Quantitative Finance and Systematic Research

Chapters 1–45, Projects 1–6, and Capstone · Complete English Text

Yuxiao Ma

October 17, 2026

Edition and Reading Notes

This book collects the accepted English teaching sources for Chapters 1–45, Projects 1–6, and the Capstone as a local complete candidate (current acceptance register: `editions/ACCEPTANCE_CURRENT.json`). Each project follows its prerequisite chapters: Project 1 after Chapter 11, Project 2 after Chapter 15, Project 3 after Chapter 21, Projects 4 and 5 after Chapter 36, Project 6 after Chapter 43, and the Capstone after Chapter 45. Study the derivations and worked examples in the text first, then cover the answers in the companion Loom and redo the key steps.

Constructed examples explain methods or test implementations; they are not market evidence. Real examples should be read with the data version, reading level, and research scope stated in each chapter. Real-data work for Projects 4–6 is incomplete; Capstone market execution remains subject to data-use rights and execution authorization; the evidence gaps around Chapter 39's old row-level predictions and some historical failed artifacts are retained. Words such as "candidate" or "pending acceptance" that remain in some project pages and historical notes record the production workflow of their time; they do not change the independent acceptance each batch has completed. Research statuses such as verified, planned, or blocked, data rights, and PIT gaps stand as written.

The learning roadmap, notation, glossary, and source index are in the reader resources. Links to other chapters or projects jump within this book; links to code, data, and Notebooks are relative to this book's location in `editions/en/body/`. Text extracted from printed code may lose indentation; use the source files.

Copyright and License

© 2026 Yuxiao Ma. Original educational content: CC BY-NC-SA 4.0.
Noncommercial sharing and adaptation require attribution and ShareAlike.
Commercial use requires separate permission. Original code: MIT; third-party terms apply.
creativecommons.org/licenses/by-nc-sa/4.0/

Contents

Chapter 1 —Quantitative Finance and the Quant Industry 1

Chapter 2 —Python for Quantitative Research: Getting Returns and Time Indices Right 21

Chapter 3 —Financial Data Engineering 48

Chapter 4 —Calculus, Rates of Change, Curvature and Local Decisions 74

Chapter 5 —Linear Algebra, Portfolio Objects and Risk 102

Chapter 6 —Probability 126

Chapter 7 —Statistical Inference: uncertainty in research conclusions 160

Chapter 8 —Regression 187

Chapter 9 —Optimization 208

Chapter 10 —Financial Markets 232

Chapter 11 —Returns and Compounding 253

Project 1 —Returns, Wealth, and Risk Description 273

Chapter 12 —Financial Statements for Quantitative Research 291

Chapter 13 —Valuation for Quants 318

Chapter 14 —CAPM and Asset Pricing 345

Chapter 15 —Factor Models 365

Project 2 —Factor Regressions and Evidentiary Limits 393

Chapter 16 —Time Series I: Dependence and ARIMA 415

Chapter 17 —Time Series II: Unit Roots, Cointegration, and Prediction 443

Chapter 18 —Volatility Modeling and Forecasting 470

Chapter 19 —State-Space Models and the Kalman Filter 499

Chapter 20 —Modern Portfolio Theory 522

Chapter 21 —Practical Portfolio Construction 550

Project 3 —Rolling Portfolios and Out-of-Sample Evaluation 600

Chapter 22 —Quantitative Research Methodology 623

Chapter 23 —Cross-Sectional Research 654

Chapter 24 —Value Factors 689

Chapter 25 —Momentum 721

Chapter 26 —Quality 751

Chapter 27 —Carry, Defensive Investing, and Low Volatility 779

Chapter 28 —Multi-Factor Models: Preprocessing, Combination, and Constraints 815

Chapter 29 —Building a Backtesting Engine 852

Chapter 30 —Backtesting Biases 890

Chapter 31 —Financial Model Validation 937

Chapter 32 —Performance Statistics & Inference 977

Chapter 33 —Risk Management 1010

Chapter 34 —Market Microstructure 1036

Chapter 35 —Transaction Costs 1054

Chapter 36 —Algorithmic Execution and Trade Scheduling 1100

Project 4 —From Fields to Signals to Execution 1144

Project 5 —Pairs Trading 1198

Chapter 37 —A Foundational Stance on Machine Learning for Finance 1239

Chapter 38 —Financial ML Pipeline 1265

Chapter 39 —Regularized Linear Models 1293

Chapter 40 —Tree-Based Models 1319

Chapter 41 —Unsupervised Learning 1351

Chapter 42 —Deep Learning Foundations and Quantitative Applications 1378

Chapter 43 —Bayesian Inference, Causal Identification, and Alternative Text Data 1423

Project 6 —Machine-Learning Forecasts, Costs, and Evidence 1476

Chapter 44 —Stochastic Calculus and Derivatives 1514

Chapter 45 —Production Quant Pipelines 1554

Capstone —Research 1598

Chapter 1 — Quantitative Finance and the Quant Industry

The central task is to turn an investment idea into a traceable chain of data, decisions, fills, and evidence. Prerequisites: elementary algebra, percentages, and basic logical reasoning. Read the programming logic first if needed; Chapter 2 develops the environment and syntax.

1. Learning goals

Distinguish quantitative finance, financial engineering, systematic trading, and algorithmic trading. Explain the problems that researchers, traders, developers, risk specialists, derivatives specialists, and execution specialists solve. Draw the path from a hypothesis to a conclusion and identify each step's inputs, outputs, and possible failures.

You should be able to turn the idea that past relative performance may persist into a testable hypothesis; record features, signals, forecasts, target weights, orders, fills, positions, and profit and loss (PnL) as separate objects; and reconcile a two-asset example with transaction costs. Job titles provide an entry point. Understanding the objects establishes the working boundary.

2. Why a quant needs this map

A historical wealth curve with a high Sharpe ratio does not show what the researcher knew before buying. It does not establish that the price was executable, that the sample retained delisted companies, or how many designs the researcher tried. Examine the process behind the curve before evaluating the result.

This chapter uses a constructed study to explain the objects. Separate the hypothesis, calculation rule, and market evidence. A technical indicator can provide an input to test; its name or appearance does not establish predictive value.

The Princeton ORFE undergraduate overview lists probability, statistics and stochastic processes, and optimization as common foundations, with finance, econometrics, and computational methods among the applications. This suggests a practical order: represent uncertainty, estimate quantities, make constrained decisions, then investigate strategies. Your role may cover one stage, but your report still needs the assumptions of the other stages. Princeton ORFE

The U.S. Securities and Exchange Commission (SEC) report on algorithmic trading discusses algorithms for investment decisions, order size, quoting, and trading risk. Automatic order submission is one application; it does not establish that a researcher has identified a new source of expected return. SEC report

3. Intuition

Start with rights, quantities, and quotes

A financial instrument specifies the rights and obligations of its holder and counterparty. Cash, securities, and contractual rights with economic value can constitute financial assets. A stock is both an instrument and an asset in an account. These categories overlap. Chapter 10 develops the trading mechanisms; here we establish the background for calculation.

Object	What you hold	Quantity and price units	Changes that affect wealth
Stock	An ownership interest; distributions depend on the company and the rights attached	Shares; currency per share	Price, entitled dividends, share count
Bond	A contractual debt claim with default risk	Units or face value; verify the quotation convention	Interest, repayment, price, default
Fund unit	A corresponding share of fund assets	Units; currency per unit	Underlying assets, fees, distributions
Derivative	Rights or obligations determined by an underlying and contract terms	Contracts; record the multiplier separately	Underlying, time, contract terms; margin differs from notional exposure

A price has a time, currency, and quotation unit. A fill price, bid, ask, and valuation price serve different purposes. With q shares valued at P currency units per share, qP has currency units. A quote of 100 does not state the amount invested or establish that the asset is cheap.

Return divides a change in wealth by starting wealth. With no cash flows and a fixed quantity, a price rise from 100 to 102 adds 2, so return is $2/100 = 0.02 = 2\%$. A holder of 100 shares earns 200 currency units, still a 2% return. Risk includes uncertainty, possible losses, and the ability to sell when needed. Beta measures one linear sensitivity, rather than the whole set of risks.

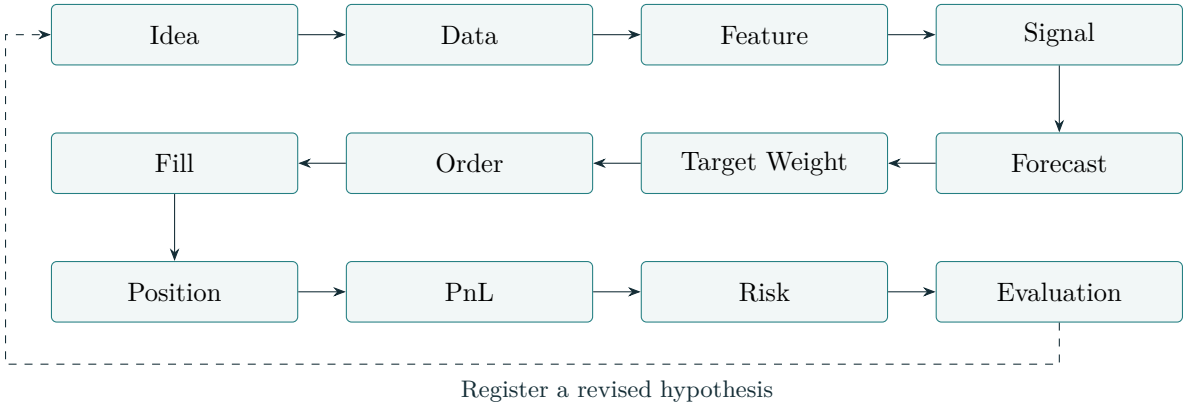
In the spot example, a long position has a positive quantity and gains value as price rises. In a simplified stock short sale, you borrow shares and sell them, assuming an obligation to return shares. A price rise increases the repurchase cost. Actual borrowing, collateral, and financing carry further conditions. Signed weights express direction here; they do not establish that short-sale proceeds are available for unrestricted use.

Consider a teaching hypothesis: information about macroeconomic demand and fundamental shocks may spread with a delay, so assets with stronger past relative performance may continue to outperform next period. Before trading, define the return measure, the historical window, the future horizon, and whether the universe includes companies that later delisted.

A historical return calculated on a decision date is a **feature**. A ranking of assets produces a **signal**. A mapping from rank to estimated future return produces a **forecast**. Risk, costs, and holding limits then inform **target weights**. Execution converts targets to **orders**. A **fill** records actual quantity, price, and time; fills update the **position**. Subsequent price and cash-flow changes produce **PnL**.

The study may fail because the proposed regularity does not exist, the data cannot test it, the forecast is weak, costs absorb the return, risk concentrates, or execution is harder than assumed. Record the point of failure to choose the next task. A more complex model does not resolve these problems by itself.

The lifecycle



The arrows show dependencies. Completing the chain does not ensure profit. The nine broad stages are:

1. **Idea:** a falsifiable hypothesis grounded in a proposed mechanism, such as slow information adjustment or compensation for risk.
2. **Data:** structured market and fundamental records with timestamps and units, organized under a point-in-time (PIT) information contract.
3. **Signal:** a direction or ranking based only on features known at decision time $t-1$.
4. **Forecast:** a conditional estimate of the next period's return distribution.
5. **Portfolio:** desired capital proportions under budget, risk, and turnover constraints.
6. **Execution:** orders, matching, and actual fill quantities, prices, and times.
7. **PnL:** monetary gains and losses and dimensionless returns from effective holdings, prices, and cash flows.
8. **Risk:** factor exposures, tail-loss budgets, and stress scenarios.
9. **Evaluation:** benchmark comparisons, friction costs, statistical and economic significance, and experiment versions.

Changing the design after evaluation adds research choices. Register a new version and retain the previous results.

4. Mathematical foundations

Definition: simple return and its scope

This chapter uses returns, weighted averages, and subtraction. The simple return of asset i over $[t-1, t]$ is

$$R_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1.$$

The prices must be positive and use the same convention. Return is dimensionless. This expression assumes no separately accounted cash flow; Chapters 3 and 11 explain total returns with dividends.

For a holding interval with no rebalancing and consistent cash-flow treatment,

$$R_{p,t} = \sum_{i=1}^{n} w_{i,t-1}^{\text{held}} R_{i,t} + w_{c,t-1}^{\text{held}} R_{c,t}.$$

Here p denotes the portfolio, n the number of risky assets, and $w_{i,t-1}^{\text{held}}$ and $w_{c,t-1}^{\text{held}}$ the actual risky-asset and cash weights after the opening fills, at the start of $[t-1, t]$. Cash return covers that same interval. Use beginning holdings, rather than targets selected with information from the interval's end.

Investing 60% in an asset returning 2%, 20% in an asset returning -1%, and 20% in zero-return cash gives a gross return of 1%. All weights share the same starting net asset value (NAV) denominator.

Gross return less trading and financing costs gives net return. With a simplified interval cost fraction c_t,

$$R_{p,t}^{\text{net}} = R_{p,t}^{\text{gross}} - c_t.$$

We measure cost against initial NAV, assume no external subscriptions or withdrawals, and simplify fee timing and reinvestment. Chapters 29 and 35 use cash ledgers to keep different cash-flow times separate.

5. Definitions

5.1 Four overlapping fields

Field	Main object	Typical question	Output and check
Quantitative finance	Mathematical, statistical, and computational representations of finance	How do we price, measure risk, allocate capital, or test a regularity?	Models, estimates, numerical methods, testable conclusions
Financial engineering	Contracts, risk transfer, and implementation structures	How do we replicate an option or design hedging and financing?	Pricing and hedging plans; no-arbitrage and risk conditions
Systematic trading	Explicit investment and trading rules	What information determines what to hold and when?	Reproducible rules, holdings, out-of-sample evaluation
Algorithmic trading	Programmed trading decisions	How do we split orders, quote, control speed, or decide trades?	Algorithms, orders, fills, execution-risk evaluation

A fund may select stocks with a model and trade them with an execution algorithm. A derivatives pricing model can belong to quantitative finance without an active investment strategy.

> **Common mistake: treating statistical arbitrage as risk free**
>
> **Statistical arbitrage** uses statistical relationships, such as relative prices, factor-neutral portfolios, and pair spreads, to construct trades. The name does not guarantee absence of risk. Estimation errors, structural changes, financing, and fills can cause losses. A no-arbitrage pricing condition and a statistical strategy's expected advantage are different claims.

High-frequency trading (HFT) commonly combines short holding periods, low-latency infrastructure, and frequent order interactions. No single threshold covers all institutions. Describe latency, order frequency, holding period, and mechanism instead of judging research quality by trades per second.

Fundamental quant encodes statements, valuation, operations, and other fundamental information as testable signals. Availability times and accounting conventions matter as much as the code.

5.2 A map of responsibilities

Role	Main responsibility	Example	Interfaces
Quant research	Hypotheses, estimation, experiments, evidence	Test momentum after costs	Data, portfolios, validation, reports
Quant trading	Trading state, risk, implementation decisions	Adapt a plan to falling liquidity	Signals, positions, execution
Quant development	Reliable data, calculation, and trading systems	Prevent misaligned data or duplicate orders	APIs, code, logs, deployment
Risk quant	Risk measures, models, constraints	Factor concentration, stress losses, margin needs	Holdings, markets, limits
Derivatives quant	Pricing, calibration, sensitivities, hedging	Option values and Greeks	Quotes, numerical methods, trading desks
Execution quant	Trading costs and execution algorithms	Balance impact against waiting risk	Parent orders, books, fills

Institutions combine and rename responsibilities. The table is a working map, rather than a universal hiring classification. Identify who chooses the hypothesis, assures the data, handles execution, and checks risk.

5.3 Keep the objects distinct

In regression attribution, **alpha** is an intercept relative to a specified model. Researchers may also use the word for expected active return or a predictive signal. State the intended meaning. **Beta** is linear sensitivity to a market or factor. A **factor** may mean a company characteristic used for sorting, a common return, or a tradable portfolio. A low P/E characteristic differs from the return on a value-factor portfolio.

Object	Example or definition	Representation	Availability and units
Feature	Cumulative return over the past 11 months	$x_{i,t}$	Calculable after the formation interval; dimensionless
Signal	Same-date percentile rank of historical return	$s_{i,t}$	Uses the available universe; often 0–1
Forecast	Conditional estimate of next month's return	$\widehat{\mu}_{i,t+1\|t}$	Produced at t, for t to $t+1$; decimal return
Target weight	Desired capital allocation after rebalancing	$w_{i,t}^{\text{target}}$	Decision after risk and cost constraints; dimensionless
Order	Instruction to buy or sell, with quantity, limit, and conditions	ID and submission parameters	Recorded at submission; no holding until filled
Fill	Actual executed quantity, price, and time	$(\Delta q_{i,t}, E_{i,t}, trade_time)$	A partial order can produce multiple fills
Position	Accumulated actual quantity	Prior quantity plus filled quantity	Updated at each fill; shares or contracts
Exposure	Capital size or sensitivity to an asset, industry, or factor	Weight, currency amount, or $B^\top w$	Specify the capital convention or risk model

Object	Example or definition	Representation	Availability and units
PnL	Change in value from holdings and cash	Final value minus initial value minus external net inflow	Currency and holding interval

In $B^\top w$, B is an asset-by-factor exposure matrix and w an asset-by-one vector, giving a factor-by-one result. Chapters 5, 15, and 21 develop it. The same capital weight can correspond to different market risk.

6. Derivations

6.1 From capital to return

Starting with V_0, allocate $V_0 w_i$ to asset i. Its final value is $V_0 w_i(1+R_i)$; cash ends at $V_0 w_c(1+R_c)$. With $\sum_i w_i + w_c = 1$,

$$V_1 = V_0 \left(1 + \sum_i w_i R_i + w_c R_c\right).$$

Divide by V_0 and subtract one to obtain Section 4's formula. The allocation is effective at the interval's start; we do not reset it using ending prices.

6.2 From trading amount to cost

Let w_i^{before} and w_i^{after} denote weights before and after filled rebalancing. Define the total risky-asset traded fraction, counting purchases and sales without halving:

$$T = \sum_{i=1}^{n} \left| w_i^{\text{after}} - w_i^{\text{before}} \right|.$$

This is total risky-asset traded notional divided by initial NAV. It excludes changes in cash weight. With fee k per unit traded, $C = V_0 kT$ and $c = C/V_0 = kT$. A fee of $k = 0.001$ means 10 basis points (bps), or 0.1%, rather than 10%. Sources may call the quantity turnover or halve it; check the convention before comparing costs.

6.3 Interpreting alpha and beta

A single-market regression takes the form

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i (R_{m,t} - R_{f,t}) + \epsilon_{i,t}.$$

R_i, R_f, and R_m are asset, risk-free, and market returns at the same currency and frequency. Alpha is the sample/model intercept, beta the sensitivity to market excess return, and epsilon the unexplained component. Use decimal returns; a daily intercept does not directly equal an annual return.

This introduces the regression objects. It does not prove the equilibrium assumptions of the capital asset pricing model (CAPM). Chapter 8 derives estimation; Chapters 14 and 15 discuss pricing and empirical tests. A positive estimated intercept still requires checks for uncertainty, omitted factors, costs, and selection bias.

6.4 Partial fills: reconcile quantities and cash

Complete example: partial fills and cash

Targets determine orders; fills determine quantities. Start from Section 8's account: 500 shares of A, 300 of B, both priced at 100, and cash of 20,000, for NAV of 100,000. The plan buys 100 A and sells 100 B. Now only 40 A fill, while all 100 B sell. This is a constructed example with 10 bps on actual traded notional, immediate payment from cash, and no external cash flow.

The purchase costs 4,000 and the sale raises 10,000. Total traded notional is 14,000 and the fee is 14. Cash becomes $20,000 - 4,000 + 10,000 - 14 = 25,986$. Actual holdings are 540 A and 200 B; post-fill NAV is $54,000 + 20,000 + 25,986 = 99,986$. Trading exchanges cash and assets; only the fee reduces NAV. The unfilled 60 A do not enter the position.

At final prices A=102 and B=99, NAV is $540(102) + 200(99) + 25,986 = 100,866$. Net PnL is 866, or 0.866% of initial NAV. The alternative check is price PnL $540(2) + 200(-1) = 880$, less the fee of 14. Using the target 600 A would overstate profit by $60(2) = 120$. If weights use post-fee NAV, their denominator is 99,986. Keep this event ledger separate from Section 8's simplified terminal fee deduction. We take fills as given; we do not model matching or financing.

7. Financial interpretation

A forecast does not specify how much capital to invest. Allocation depends on forecast uncertainty, correlations, loss budgets, costs, liquidity, and financing. High weights in two correlated stocks can concentrate the same economic risk twice.

With market beta 1.5, a one-percentage-point increase in market excess return corresponds in the regression model to about 1.5 percentage points of portfolio change, holding other effects fixed. It does not guarantee the next realization, particularly during crises or leverage constraints.

Specify what "net alpha" deducts. Commissions alone can omit borrowing, financing, spreads, and impact. Chapter 35 develops the model; here, report the included and omitted costs.

8. Numerical example

All inputs are **constructed for teaching**, rather than observations or strategy performance. We follow the nine stages.

Times use Coordinated Universal Time (UTC) and constructed days, rather than an actual exchange calendar. At D 0 16:00, prices are A 98/B 101; at D 1 16:00, A 100/B 100. Publication occurs at 16:01, ingestion at 16:02, and signal formation at 16:05. Both assets fill at 100 at D 2 09:30. The measured interval runs from D 2 09:30 to D 3 16:00, with final prices A 102/B 99 and no dividends.

Input	A	B	Purpose
D 0–D 1 historical return	$100/98 - 1 \approx 2.040816\%$	$100/101 - 1 \approx -0.990099\%$	Compare past performance
Assigned high/low signal	$+1$	-1	Enter the stated forecast mapping
Next holding-period forecast	1.5%	0.5%	Inform the portfolio decision
D 2–D 3 realized return	2%	-1%	Evaluate after the interval

At D 1, the last row is unknown. The target weights 0.6/0.2 are a specified rule, rather than a uniquely derived optimum from the two forecasts.

1. **Idea:** macroeconomic industry-demand differences may propagate slowly and create short-term persistence.
2. **Data:** use prices and returns already ingested at decision time $t-1$, with both `available_at` and `ingested_at` no later than that decision.
3. **Signal:** assign $+1$ to the stronger A and -1 to the weaker B.
4. **Forecast:** specify the teaching mapping $\widehat{\mu}_i = 0.01 + 0.005 s_i$. It gives A 1.5%, B 0.5%, and cash 0%. Constants have decimal-return units per holding period. We have neither fitted nor validated the mapping. Rank has no return units; the realized $2\%/-1\%$ are separate inputs.
5. **Portfolio:** choose long-only target weights $w_A^{\text{target}} = 0.60$, $w_B^{\text{target}} = 0.20$, and $w_c^{\text{target}} = 0.20$.
6. **Execution:** initial NAV is 100,000 with weights 0.50/0.30/0.20. At D 2's price of 100, the account holds 500 A, 300 B, and 20,000 cash. Buy 100 A and sell 100 B, each for 10,000 currency units. Assume full fills, yielding 600 A and 200 B. Distinguish currency notional from share quantity. At the start of the return interval, the total traded fraction is $T = |0.60-0.50|+|0.20-0.30| = 0.20$, or 20,000 currency units. Held weights are 0.60/0.20/0.20.
7. **PnL:** realized A/B/cash returns are $2\%/-1\%/0\%$. Gross return is $0.60(0.02) + 0.20(-0.01) + 0.20(0) = 0.01 = 1.00\%$. At $k = 0.001$, cost is $C = 100,000(0.001)(0.20) = 20$, or 0.02% of initial NAV. Net return is $1.00\% - 0.02\% = 0.98\%$. Final NAV is $100,000(1 + 0.0098) = 100,980$; net PnL is 980 currency units.
8. **Risk:** risky-asset gross and net exposure both equal 0.80; cash provides a 20% buffer. A alone falling 10%, with B and cash unchanged, produces $0.6(-0.10) = -6\%$ gross return. This is a constructed stress scenario, rather than a probability estimate.

9. **Evaluation:** a specified 50/50 A/B benchmark with no cash returns 0.5% gross. The gross difference is 0.5 percentage points; benchmark costs are unspecified, so a net difference is unavailable. One observation cannot establish mean significance. At 50 bps, strategy cost becomes 0.10% and net return 0.90%. Freeze the record and parameters to retain the trial history.

We assume full fills at interval start, no additional impact or external cash flow, and a fee deducted against initial NAV without changing the stated asset allocation. A self-financing ledger must reconcile the source of fee cash and actual quantities. This example checks arithmetic and object definitions; it is not a trading engine.

Choosing weights 0.6/0.2 after observing A 2%/B −1% and applying them to that finished interval creates look-ahead bias. The same multiplication can represent an invalid experiment.

9. Python implementation

Default accepted-reader entry point

Use the shared I3 reader at the cloned repository root (`quantweft/`). Set up its environment under README.md, then run from that reader root:

```
.venv/bin/python run_notebook.py notebooks/ch01_lifecycle.ipynb
```

The constructed notebook does not read omitted French files or download data. Each run writes to a unique `runs/` subdirectory. The historical CLI and test/data paths below refer to the course root. French calculations require authorized local inputs and manifests and were not rerun for this batch. The portable I3 CLI defaults to a fresh UUID run and rejects overwriting. When already at the reader root, do not run `cdquantweft` again. README.md and OPTIONAL_DATA.md describe setup and optional inputs. A constructed check does not revalidate market calculations.

The chapter script is `code/ch01_lifecycle.py`, with `notebooks/ch01_lifecycle.ipynb`; the run environment and steps are described in the environment notes. The script records the teaching construction and capital convention and sends no trading instructions. The original course-root command is:

```
cd quantweft
.venv/bin/python code/ch01_lifecycle.py
```

The following standalone program explains the calculation. Shared implementations and their tests live in `src`.

```
from math import isclose

def one_period_net_return(
    weights: tuple[float, ...],
    returns: tuple[float, ...],
    prior_weights: tuple[float, ...],
    cost_rate: float,
) -> tuple[float, float, float]:
    if not (len(weights) == len(returns) == len(prior_weights)):
        raise ValueError("weights and returns must have the same length")
    if cost_rate < 0:
        raise ValueError("cost_rate must be nonnegative")
    gross_return = sum(w * r for w, r in zip(weights, returns))
    turnover = sum(abs(w - old) for w, old in zip(weights, prior_weights))
    cost = cost_rate * turnover
    return gross_return, cost, gross_return - cost

gross, cost, net = one_period_net_return(
    weights=(0.6, 0.2), returns=(0.02, -0.01),
    prior_weights=(0.5, 0.3), cost_rate=0.001,
)
assert isclose(gross, 0.01)
assert isclose(cost, 0.0002)
assert isclose(net, 0.0098)
print({"gross": gross, "cost": cost, "net": net})
```

Floating-point output may show 0.009800000000000001. Use a tolerance, rather than comparing strings. The local function assumes zero cash return; it does not cover financed shorts or complex rebalancing. Chapter 29 supplies a ledger.

The next standalone block reconciles the simplified convention: specify the post-fill asset allocation, deduct the 20-unit fee at the end, and ignore the reinvestment effect of fee timing. Chapter 29 handles self-financing constraints.

```
from math import isclose
initial_nav = 100000.0
stock_end = 600 * 102.0 + 200 * 99.0
trade_amount = 100 * 100.0 + 100 * 100.0
cost_currency = trade_amount * 0.001
cash_end = 20000.0 - cost_currency
ending_nav = stock_end + cash_end
pnl_currency = ending_nav - initial_nav
assert isclose(stock_end, 81000.0)
assert isclose(cost_currency, 20.0)
assert isclose(ending_nav, 100980.0)
assert isclose(pnl_currency / initial_nav, 0.0098)
print(stock_end, cash_end, ending_nav, pnl_currency)
```

Timing contract

Suppose historical information through Monday's close forms the feature, the signal follows the close, and fills occur at Tuesday's trading start. Measure changes after the Tuesday fill. The Monday-close-to-Tuesday-open gap does not belong to a position first purchased on Tuesday.

Chapter 2 demonstrates index alignment with `shift`. A shifted row still needs consistent price endpoints. Using close-to-close returns to approximate an open-fill holding return requires a separate error disclosure.

10. Visualization

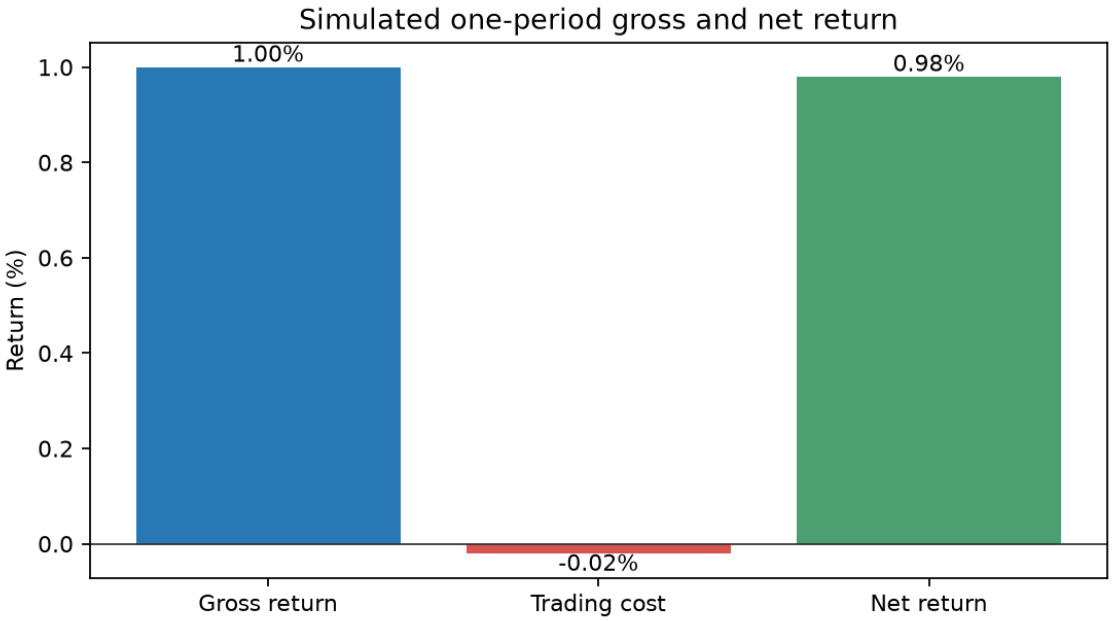


Figure 1: Constructed teaching inputs

The horizontal axis shows Gross, Cost, and Net; the vertical axis shows return percent. The cost bar is -0.02%, expressing its contribution to return; the fee itself is positive 0.02%. Check that the fee equals the gross/net difference. The 0.02-percentage-point fee is small relative to 1% gross return here. The figure illustrates constructed arithmetic and establishes no strategy advantage.

The lifecycle arrows show dependencies. An incorrect net return may arise from fill price, position timing, corporate actions, or cost units. Trace the relevant inputs.

11. Real market example

Kenneth French publishes industry-portfolio and factor returns. The course's frozen 10-industry snapshot comes from 10 Industry Portfolios; its download and hashes appear in `data/manifests/french_industries_2026-10-05.json`. Raw data are omitted from the portable reader and require authorized user-supplied inputs.

The source description assigns stocks by Standard Industrial Classification (SIC) industry at each June end and computes returns from July through the following June. You have returns on **already constructed portfolios**, rather than executable stock quantities or observations of Open, High, Low, Close, and Volume (OHLCV). This supports return-panel and regression exercises; it does not establish that you can reproduce every historical fill.

Map it to the lifecycle: industry-return differences may motivate an idea; monthly portfolio returns provide data; historical industry returns form features; ranks form signals; research weights allocate across return series. The file does not supply executable prices, constituents, or quantities for execution. Bound the statistical conclusion accordingly.

This chapter performs no momentum backtest on that sample and reports no strategy Sharpe. Chapter 2 handles loading and units, Chapter 15 factor regression, and Project 1 and later projects description and strategy validation. The database revises historical calculations, so a snapshot differs from the current page. French library and format changes

12. Common mistakes

Using a future realization as the forecast

`forecast = next_month_return` uses an outcome unavailable at prediction time. Historical realized returns may serve as training labels after their intervals finish. Keep separate columns for the feature available at `signal_time`, the forecast generated then, and the later `realized_return`. A common display date does not imply common availability.

```
past_return = -0.02
next_return = 0.03

# Wrong: use the future outcome for the current decision
wrong_weight = 1.0 if next_return > 0 else 0.0

# Use historical information first; evaluate the future afterward
correct_weight = 1.0 if past_return > 0 else 0.0
```

These inputs are constructed. The programming solution compares realized returns. In actual work, both variables need explicit return intervals.

Treating zero net exposure as zero risk

Long A 50% and short B 50% gives zero net exposure and 100% gross exposure. If A rises 10% and B 20%, the cost-free return is $0.5(0.1) - 0.5(0.2) = -5\%$. Dollar neutrality does not imply beta neutrality or protection from loss.

Confusing cost-rate units

10 bps equals $10/10000 = 0.001$. Entering 10 gives a 1000% rate; entering 0.1 gives 10%. Record units in configurations and data dictionaries, and use the 20-currency-unit hand calculation as a check.

Treating regression alpha as established advantage

An intercept depends on model, sample, frequency, factors, and estimation error. Check multiple testing, costs, and out-of-sample evidence. Discuss statistical and economic significance separately.

Equating reproducibility with valid research

Repeated output confirms that fixed inputs and code reproduce a calculation. Invalid data, future information, omitted delistings, and wrong formulas can also reproduce. Test input validity and experimental design.

13. Quant research application

Write a minimal protocol for the momentum hypothesis:

1. **Mechanism:** adjustment may occur in stages, creating association between past strength and future return. Treat this as a proposed explanation.
2. **Sample:** begin with industry portfolios for method practice. For a stock strategy, recheck the PIT universe, prices, cash flows, and borrowing.
3. **Formation:** specify months and whether to skip the most recent month. Chapter 25 develops 12–1 momentum.
4. **Decision and fill:** use information available through t, trade at an executable later time, and match return endpoints to the fill.
5. **Control:** compare equal weighting or a specified benchmark instead of reporting only the best curve.
6. **Freeze:** determine windows, weights, costs, and metrics before final out-of-sample evaluation; retain all trials.
7. **Evaluation:** report gross/net return, risk, exposure, turnover, failure windows, uncertainty, and unverified execution assumptions.

You can define a question and data contract before learning regression and backtesting software. Add a tool when it addresses a specific problem.

14. Chapter summary

Researchers turn an economic question into hypotheses and testable objects. Developers make data and calculations reproducible. Trading and execution staff convert target weights into orders and fills; fills update holdings. Risk staff assess possible losses, and holdings plus cash flows determine PnL. The lifecycle connects these responsibilities.

Features provide inputs; signals guide decisions; forecasts estimate future outcomes; targets allocate capital; orders request trades; fills record execution; positions record actual quantities; exposure measures capital or sensitivity; PnL records subsequent monetary results. Attach a model and frequency to alpha and beta. Chapters 2 and 3 develop implementation and information availability.

15. Exercises

1-E01: A program splits a large order into small orders every minute. A person chooses investment direction. Which field does this illustrate? Does it establish quant research?

1-E02: Explain feature, signal, forecast, target weight, order, fill, and position. Give a historical-return example and label decision, submission, fill, and position-effective times.

1-E03: Does long A 50%/short B 50% eliminate market risk? State the information you need.

1-E04: Industry returns support regression but do not validate stock-order execution. Name three missing inputs.

1-E05: Risky weights are 0.4/0.4 and cash 0.2. Returns are 3%/−2% and cash 0.1%. Calculate gross return.

1-E06: Risky weights change from 0.3/0.5 to 0.5/0.3, fee 5 bps per traded notional, starting NAV 200,000 currency units. Calculate cost, counting purchases plus sales without halving.

1-E07: A/B returns are 10%/20%. Hold A 0.5/B −0.5 and cash 1 at zero return. Calculate net/gross exposure and gross return.

1-N01 (identify): Classify a quote of 100 currency units per share, 100 shares held, signal +1, and a next-period forecast of 1.5%, with units. Can signal +1 directly mean a 1% forecast? When do orders arise and holdings update?

1-N02 (one step): Starting NAV 50,000; risky weights 0.40/0.40, cash 0.20. Full fills change risky weights to 0.70/0.10; cash stays 0.20. At 10 bps calculate total unhalved traded fraction, actual traded notional, fee amount, and cost fraction. Does cash-weight change enter turnover?

1-N03 (full calculation): Starting NAV 100,000; risky/cash weights change from 0.50/0.30/0.20 to 0.60/0.20/0.20. Realized returns 2%/−1%/0%, fee 10 bps. Show turnover, fee, gross/net return, ending NAV, net PnL, and risky gross/net exposure, with units.

1-B01 (partial fills): Under Section 6.4's fill state, calculate post-fill cash, quantities, NAV, and final net PnL. Explain why the target 600 A cannot determine actual A profit.

16. Programming exercises

1-P01: Run the chapter script and check gross 0.01/cost 0.0002/net 0.0098. Change the rate to 50 bps and interpret the result.

1-P02: With past return -2% and next return 3%, compare a rule selecting holdings from the future return's sign with one using the past return's sign. Define and run all variables.

1-P03: Write a check rejecting a `signal_time` of 16:05 and same-day `trade_time` 16:00. Make the next-day opening time pass. Use timezone-aware UTC objects.

1-N04 (diagnose): Targets 0.6/0.2 do not fill; holdings remain 0.5/0.3, cash 0.2. No other trade occurs; fees apply only to actual traded notional. Returns $2\%/-1\%/0\%$. Identify both errors below, calculate actual gross/net return, and repair with assertions.

```
target_weights = (0.6, 0.2)
prior_weights = (0.5, 0.3)
returns = (0.02, -0.01)
cost_rate = 10  # Wrong: treat 10 bps as the decimal rate
planned_turnover = sum(abs(w - old) for w, old in zip(target_weights, prior_weights))
gross = sum(w * r for w, r in zip(target_weights, returns))
net = gross - cost_rate * planned_turnover
print(gross, net)
```

17. Challenge problems

1-C01: Design a minimal momentum protocol. Which assumptions can you register? Which data gaps prevent an investability conclusion? Propose a control that could overturn the initial explanation.

1-C02: Name at least three gaps between the simplified fee example and an actual self-financing ledger. Propose a reconciliation identity.

1-N05 (research judgment): A historical-return ranking uses today's security list and forms its signal after the closing price is final, yet assumes fills at that same close. The report shows one high-Sharpe result without costs. Identify timing/sample defects, missing fee/trial evidence, and whether this proves actual net profitability or disproves the regularity.

18. Solutions / Solution Hints

Concepts and hand calculations

1-E01: This is an execution algorithm within algorithmic trading. Studying execution parameters and impact may constitute execution-quant research. Following a fixed plan does not establish alpha research. Human investment direction can coexist with automated execution.

1-E02: Historical cumulative return is a feature; its rank at t is a signal; an estimate made at t for a future interval is a forecast. The target is a desired capital fraction, the order a submitted instruction, the fill an actual execution record, and the position the filled share quantity. For example, decide at D 1 16:05, submit at D 2 09:29, and confirm a fill at 09:30; that quantity becomes effective at 09:30 and its return starts afterward. Unfilled orders leave quantities unchanged.

1-E03: Net capital exposure is zero. Unequal asset betas can leave nonzero market beta. Industry, stock-specific, spread, borrowing, and financing risks remain. Obtain betas, correlations, factor exposures, and trading/capital constraints.

1-E04: Historical constituents and quantities; executable quotes/depth; borrowing and financing terms. Corporate actions, calendars, and costs also need checks. Portfolio-return files supply none of these by themselves.

1-E05: $0.4(0.03) + 0.4(-0.02) + 0.2(0.001) = 0.0042 = 0.42\%$. Cash contributes 0.02 percentage points.

1-E06: $T = |0.5 - 0.3| + |0.3 - 0.5| = 0.4$. At 5bps $= 0.0005$, cost fraction is 0.0002=0.02%, or 40 currency units. A halved convention needs a matching fee convention.

1-E07: Risky net exposure $0.5 - 0.5 = 0$, gross 1, return 5%−10%=−5%. Cash weight 1 satisfies the total budget but does not offset price changes. Actual access to short-sale proceeds depends on collateral and financing; this question simplifies them.

1-N01: 100 currency/share is price,100 shares is quantity, and their product 10,000 is market value. Signal +1 is dimensionless;1.5% forecasts the next holding return. A mapping and conditions are needed to convert a score into expected return. Execution submits orders, and actual fills update positions. Unfilled orders do not increase holdings.

1-N02: $T = |0.70-0.40|+|0.10-0.40| = 0.60$. Notional is $50,000(0.60) = 30,000$, comprising 15,000 bought and 15,000 sold. At $k = 0.001$, $c = 0.0006 = 0.06\%$ and $C = 30$. Cash rebalancing is excluded from risky-asset turnover.

1-N03: $T = 0.10 + 0.10 = 0.20$; $k = 0.001$, $c = 0.0002 = 0.02\%$, $C = 20$. Gross return $0.60(0.02) + 0.20(-0.01) = 0.012 - 0.002 = 0.010 = 1.00\%$. Net 0.98%; ending NAV 100,980; PnL 980 currency units. Risky gross/net exposure both 0.80=80%.

1-B01: Actual holdings 540 A/200 B; traded notional 14,000; fee 14; cash 25,986; post-fill NAV 99,986; final NAV 100,866; PnL 866 and return 0.866% of initial NAV. Price PnL 880 less fee 14 gives 866. The unfilled 60 shares would falsely add 120 profit if you used the target. Post-fee weights divide by 99,986 and their amounts sum to that NAV.

Programming reference solutions

1-P01: 50 bps=0.005 gives cost 0.001=0.1% at turnover 0.2. Net return 0.009=0.9%. Change `cost_rate=0.005` in Section 9 and assert `isclose(net, 0.009)`. Equal gross return need not give equal net return.

1-P02: The future-informed rule obtains 3%; the historical rule holds no asset and obtains 0. The difference comes from future knowledge.

```
past_return = -0.02
next_return = 0.03
wrong_weight = float(next_return > 0.0)
correct_weight = float(past_return > 0.0)
wrong_pnl_fraction = wrong_weight * next_return
correct_pnl_fraction = correct_weight * next_return
assert wrong_pnl_fraction == 0.03
assert correct_pnl_fraction == 0.0
print(wrong_pnl_fraction, correct_pnl_fraction)
```

1-P03: Compare timezone-aware instants, rather than date strings. This checks time order; it does not establish that an exchange is open. Chapter 3 adds calendars.

```
from datetime import datetime, timezone

def check_trade_time(signal_time: datetime, trade_time: datetime) -> None:
    if signal_time.tzinfo is None or trade_time.tzinfo is None:
        raise ValueError("timezone required")
    if trade_time <= signal_time:
        raise ValueError("trade must follow signal")

signal = datetime(2026, 6, 1, 16, 5, tzinfo=timezone.utc)
wrong_trade = datetime(2026, 6, 1, 16, 0, tzinfo=timezone.utc)
try:
    check_trade_time(signal, wrong_trade)
except ValueError:
    pass
else:
    raise AssertionError("look-ahead timing was not rejected")
check_trade_time(signal, datetime(2026, 6, 2, 13, 30, tzinfo=timezone.utc))
```

The dates and UTC times are constructed; we do not claim a particular security's actual opening time.

1-N04: First,10 bps equals 0.001; entering 10 multiplies the rate by 10,000. Second, targets differ from actual holdings and planned trading differs from fills. No fills leaves holdings 0.5/0.3, actual turnover 0, and fees 0. Gross $0.5(0.02) + 0.3(-0.01) = 0.007$, net 0.7%.

```python
from math import isclose
held_weights = (0.5, 0.3)
returns = (0.02, -0.01)
actual_traded_fraction = 0.0
cost_rate = 10 / 10000.0
gross = sum(w * r for w, r in zip(held_weights, returns))
cost = actual_traded_fraction * cost_rate
net = gross - cost
assert isclose(gross, 0.007)
assert cost == 0.0
assert isclose(net, 0.007)
print(gross, cost, net)
```

This uses a specified fill state, rather than a matching model. Actual systems also record rejections, missed opportunities, and other possible fees.

Challenge solutions and grading

1-C01: Specify mechanism, universe, window endpoints, signal/return intervals, benchmark, costs, validation, and search scope. A preregistered reversal window or subsample comparison can challenge the explanation. Industry data cannot establish stock borrowing, impact, or full-market PIT validity. Grade timing/data scope 40%, falsifiable hypothesis/control 30%, validation/limits 30%; do not grade by positive return.

1-C02: Gaps include fee timing, post-trade cash, converting weights to quantities, unfilled orders, dividends/financing, and price drift between trades. Use

$$NAV_t = CASH_t + \sum_i q_{i,t} P_{i,t} - LIAB_t.$$

CASH is cash, q actual quantity, P valuation price, and LIAB liabilities. Record external flows and trade cash flows separately. Pair each asset change with its cash change; fees reduce NAV. Grade at least three gaps 50%, consistent units and event order 50%.

1-N05: Final-close input must arrive and be processed before the signal and order. Same-close execution lacks timing evidence. Today's list cannot establish historical membership; obtain entry/exit and tradability records. Add actual traded notional, rates, spreads/impact/financing, fills, trial history, benchmarks, and out-of-sample design. High Sharpe proves neither actual net profit nor absence of the economic regularity. The backtest cannot support an executable conclusion. Grade 10 points: timing 3, universe 2, fee/trial evidence 3, conclusion scope 2.

19. Further reading

- **Must know:** Princeton ORFE's official overview for foundations and applications; the SEC report's definitions and trading uses.
- **Should know:** the MIT Finance Theory I overview for valuation, investment, and risk themes.
- **Optional:** compare institutional job descriptions by responsibilities. Titles vary and do not provide a universal qualification standard.

20. References

1. **ORFE:** Princeton ORFE, Undergraduate Program. The historical course record reports reading the core overview on 2026-10-05. Official page.
2. **SEC_ALGO:** SEC, *Staff Report on Algorithmic Trading in U.S. Capital Markets*,2020. The record covers algorithmic decision uses and makes no unverified statistical claim. Official PDF.
3. **FRENCH_10:** Kenneth R. French, *Detail for 10 Industry Portfolios*. This chapter explains applicability. Portfolio description.
4. **FRENCH:** Kenneth R. French, Data Library, for format and historical-calculation revisions. Library.

Source-reading levels follow the existing source_notes.md and accepted I3 records. This batch did not revisit websites or read additional full texts.

Local shared references: source notes, accepted reader setup, optional data contract, shared script, shared notebook.

Chapter 2 — Python for Quantitative Research: Getting Returns and Time Indices Right

> Prerequisite: Chapter 1. After this chapter, you can rerun the same return analysis from a command line and a Notebook, explain missing values, label alignment, historical windows and execution timing, and reject unsorted or duplicate indices and nonpositive prices.

Time labels in this chapter use Coordinated Universal Time (UTC). NaN means Not a Number; we use this numerical marker for missing values. The marker alone does not explain why a value is missing.

1. Learning Objectives

By the end of the chapter, you should be able to:

1. Organize a repeatable analysis using Python's built-in types, conditionals, loops, functions, type hints, imports and exceptions.
2. Calculate a batch of returns with NumPy arrays, vectorization and broadcasting, and explain the meaning of every axis.
3. Use pandas date indices for returns, ranking, rolling and expanding statistics, groupby, monthly resampling and missing-value checks.
4. Express filters, column transformations and grouped aggregation in Polars, distinguishing eager execution from a lazy plan executed by `collect`.
5. Read the input conditions and convergence results of SciPy numerical methods, and use Matplotlib to plot clearly identified samples with units.
6. Rerun analysis in Jupyter Notebook and from the command line, record code changes in Git, and keep credentials and local data out of shared repositories.

To demonstrate these skills, obtain correct returns from one price column and explain its first missing return. Explicitly handle unsorted dates, duplicate indices and nonpositive prices. Verify that NaN remains in the output rather than becoming zero, and show that a signal forms before actual execution and the start of the holding interval, whose return is measured afterward.

2. Why Quantitative Research Needs Python

Many research errors are ordinary calculations applied to the wrong interval. An arithmetically correct ratio answers the wrong question if its endpoints come from different frequencies, assets or dates.

Python provides a common environment for reading data, numerical and statistical work, plotting and recording experiments. Define a function in a script, then call it from a Notebook to explore the data. After a fix, rerun it in a fresh process. NumPy operates on arrays; pandas operates on labeled tables. Both support batch operations, while pandas also aligns data by index labels. Labels clarify time series, but require explicit index checks.

We begin with three constructed teaching prices: 100, 110 and 99. Calculate the two interval returns by hand, then ask the libraries to reproduce each result. This checks arithmetic and time labels; it establishes neither a tradable strategy nor a real market observation.

3. Intuition

For the same asset, the first move from 100 to 110 is a gain; the second move to 99 is a loss. Adding price changes or percentages misses the fact that the second return acts on wealth after the first return. Section 8 works through the numbers.

Each observation needs an asset identity and an observation time. A price has a specified definition, such as a closing price or total-return index; a return has explicit start and end points. The fourth number in an array is insufficient without knowing its asset, time and interval.

An index labels each row. When pandas adds two Series, it pairs values by labels before calculating. A Monday/Tuesday Series plus a Tuesday/Wednesday Series leaves Monday and Wednesday missing because their partners are absent. Alignment can reveal mismatches or quietly reduce the usable sample. Check the resulting row count, missing positions and dates.

4. Mathematical Foundations

Let P_t be the specified price at date t, in quotation currency per unit. The dimensionless simple return is

$$R_t = \frac{P_t}{P_{t-1}} - 1, \qquad P_{t-1} > 0.$$

Here $t-1$ means the preceding observation, not necessarily the preceding calendar day. T prices provide at most $T-1$ adjacent returns. The first row has no preceding price and must remain missing, marked NaN rather than invented as zero.

A one-dimensional pandas price sequence and its return output both have shape $(T,)$, with a missing first return. With T dates in rows and N assets in columns, prices and the aligned return output have shape (T, N).

Wealth factors multiply:

$$1 + R_{0:K} = \prod_{t=1}^{K} (1 + R_t).$$

We use K for the number of holding intervals: T complete price observations give $K = T - 1$ adjacent intervals, with endpoints $P_0, \ldots, P_K$. $R_{0:K}$ is the simple return for the entire span; each R_t covers one nonoverlapping interval. Adding percentages does not compound wealth.

For positive prices, log returns $r_t = \ln(P_t/P_{t-1})$ add across time:

$$r_{0:K} = \sum_{t=1}^{K} r_t = \ln(P_K/P_0).$$

Log and simple returns differ. Funding and ordinary portfolio weights generally use simple returns; keep the two definitions separate.

Most decimal fractions have only approximate finite binary representations. The machine result of 0.1 + 0.2 need not equal 0.3 exactly; extra trailing digits do not by themselves show a formula error. Compare floating-point results with a tolerance. Accounting that requires exact decimal amounts needs a separate decimal representation and rounding rule. Preserve price units, dtypes and tolerance conventions even when using floats for returns.

5. Definitions and Objects

Python is a general-purpose interpreted language. An expression produces a value; a statement controls execution or binds a name. A variable name binds an object and does not fix its type. Common built-in objects include **int**, **float**, **bool**, **str**, **list**, **tuple**, **dict**, **set** and **None**. Numeric price columns should have a defined type rather than mix dates, notes and numbers.

`if/elif/else` selects branches; `for` visits items and `while` repeats a check. A function names input/output logic; a module is an importable Python file or library. An exception makes failure identifiable. Type hints describe expected interfaces for readers, integrated development environments (IDEs) and static checkers. They do not automatically reject incompatible inputs at runtime.

Definition: shape, axis and meaning

NumPy's `ndarray` stores elements of one dtype. Its shape gives dimension lengths; an axis is a dimension number. An array expression acts on many elements, reducing loops at the Python level. Compatible shapes permit elementwise operations; broadcasting does not validate what the numbers mean.

A pandas DataFrame has row and column labels. Labels govern alignment; `iloc` selects by position. These rules are not interchangeable. Timestamp, DatetimeIndex and PeriodIndex can represent time points or periods.

Reading Objects and Brackets

`import pandas as pd` imports a module under the short name `pd`. A dot accesses a module function or object method; `()` calls it. `=` binds the result on the right to a name on the left; `==` tests equality. `assert condition` stops the example when the condition fails. `[100, 110]` is an ordered list; `{"A": [100, 110]}` is a dictionary mapping a string key to a value. `index=dates` is a named argument specifying row labels.

A Series has one column of values and an equally long Index. A DataFrame has both row and column labels. `df["A"]` returns a Series; `df[["A"]]` uses a list of column names and returns a two-dimensional DataFrame. `df.iloc[1, 0]` selects the second row and first column using zero-based positions. `df.loc[dates[1], "A"]` selects the same cell by labels. Equal positions do not prove equal labels in an imported table.

Shape (T, N) means T rows and N columns whose meanings cannot be exchanged. Axis 0 is the row dimension; axis 1 is the column dimension. For a time-by-asset table, `mean(axis=0)` removes time and returns one mean per asset; `mean(axis=1)` removes assets and returns one

cross-sectional mean per date. `rank(axis=1)` ranks within each row. `rolling(2).mean()` moves along each column's row order and retains the date rows. Check each operation's output shape even when axis numbers coincide.

`prices[1:, :]` takes all rows from the second onward and all columns; `prices[:-1, :]` takes all but the last row. Colons slice and the comma separates row/column selections. These slices pair adjacent endpoints within a column and produce $(T-1, N)$ returns. pandas shift keeps all T rows and a first NaN. These are different output layouts for the same return definition.

Six Core Operations

1. **shift:** move values a specified number of positions along an axis. `shift(1)` places the previous row's value at the current time label, leaving the first value NaN. It changes alignment without changing index labels.
2. **Rolling window:** use a fixed-length historical window including the current row. For size W, the default `min_periods=W` requires W nonmissing observations; otherwise the result is NaN.
3. **Expanding window:** use all observations from the sample start through the current row. With `min_periods=1`, the first row has a result and the sample grows thereafter.
4. **groupby:** partition a table by discrete keys such as `asset`, calculate independently within groups, and combine the results. Grouping isolates assets but does not sort them. First sort time ascending within each group and reject duplicate keys.
5. **resample:** divide a time-indexed series into regular bins, such as weeks or months, and aggregate each bin with `.last()` or `.mean()`. A boundary label does not assert that the market was open then.
6. **Cross-sectional rank:** rank asset characteristics or returns within a time slice along `axis=1`. `pct=True` normalizes ranks to $(0, 1]$.

Polars' expression application programming interface (API) constructs column transformations. Eager mode computes each step; lazy mode first constructs a query plan for optimization and execution. SciPy supplies distributions, linear algebra, optimization, interpolation and integration on top of NumPy. Matplotlib maps numbers to graphical elements. Jupyter provides cells and a computational kernel; Git records recoverable commits.

6. Derivation

Without external cash flows, holding wealth evolves as

$$W_t = W_{t-1}(1 + R_t).$$

Expanding this recurrence gives $W_K = W_0 \prod_{t=1}^{K}(1+R_t)$ and cumulative simple return W_K/W_0-1. K still counts actual holding intervals; initial capital is not an extra zero-return observation.

The prices must share a definition and refer to the same holding. Closing-price ratios understate total return when omitted dividends matter. With splits or vendor revisions, specify whether the input is raw prices, adjusted prices or a total-return series.

The denominator must be positive. Zero makes division undefined; negative prices are outside this example's ordinary spot-asset interpretation. **compute_returns** requires an ascending, unique, timezone-aware DatetimeIndex and positive finite nonmissing prices. It preserves and propagates NaN without filling; the caller must choose and record any cleaning rule.

7. Financial Interpretation

A return is an interval statistic. A return labeled 2026-02-28 needs explicit starting and ending times. Labels often use the interval end and cannot alone establish when a signal became available.

The course distinguishes **event_time**, **available_at** (earliest availability of that version), **ingested_at** (actual receipt and storage by this system), **signal_time**, **trade_time**, **return_start** and **return_end**. Eligible inputs satisfy both

$$\text{available_at} \leq \text{signal_time}, \qquad \text{ingested_at} \leq \text{signal_time}.$$

Execution must follow the signal. Public but unreceived data cannot enter this system's historical signal. If equal timestamps do not establish order, exclude the version or retain finer receipt ordering; Chapter 3 explains this boundary. A signal requiring the finalized close cannot simply assume execution at that same close. An execution design must establish order submission, matching and latency. A common teaching setup forms a signal from closing information and trades at the next executable time, measuring returns after execution.

Simple returns compound over time and combine across assets using capital weights effective at the interval start, assuming no within-interval rebalancing, extra cash flows or financing. Log returns add over time but cannot replace simple returns in an ordinary capital-weighted portfolio. Keep daily, monthly and annualized quantities separate, and define annualization by frequency and market calendar. A missing trading day is not a zero return; filling zeros changes volatility, cumulative returns and correlations.

Historical rolling statistics must use information known by the signal time. A signal at close t may use the previous 20 observations for a later holding interval. Computing full-sample means, ranks or standard deviations and writing them back into history reads the future. Define every feature's window endpoints, minimum sample size and availability.

8. Worked Numerical Examples

The following **constructed teaching** table uses UTC-labeled observation dates, without claiming they are verified trading sessions.

Time index	Asset A price	Asset B price	Shape and meaning
t_0: 2026-01-05 UTC	100.0	50.0	Wide table $(4, 2)$, timezone-aware DatetimeIndex
t_1: 2026-01-06 UTC	110.0	52.0	Asset columns `Index(['A', 'B'])`
t_2: 2026-01-07 UTC	99.0	51.0	One extracted Series has shape $(4,)$
t_3: 2026-01-08 UTC	108.9	54.0	Numeric ndarray $(4, 2)$ has no labels

8.1 Inputs, Intermediate Values and Outputs

shift and Returns

Apply `P.shift(1)` along rows, axis 0.

Time	P_A	Previous A	R_A	P_B
t_0	100	NaN	NaN	50
t_1	110	100	$110/100 - 1 = 10\%$	52
t_2	99	110	$99/110 - 1 = -10\%$	51
t_3	108.9	99	$108.9/99 - 1 = 10\%$	54

Time	Previous B	R_B
t_0	NaN	NaN
t_1	50	$52/50 - 1 = 4\%$
t_2	52	$51/52 - 1 \approx -1.923\%$
t_3	51	$54/51 - 1 \approx 5.882\%$

At t_0 no sample observation establishes the growth base. This is a statistical boundary, not a zero return.

Rolling Mean, window=2

For A's input $[100, 110, 99, 108.9]$, `rolling(window=2).mean()` gives:

Time	Window	Calculation	Output
t_0	$\{100\}$	$1 < \text{min_periods} = 2$	NaN
t_1	$\{100, 110\}$	$(100 + 110)/2$	105
t_2	$\{110, 99\}$	$(110 + 99)/2$	104.5
t_3	$\{99, 108.9\}$	$(99 + 108.9)/2$	103.95

Expanding Mean, min_periods=1

The same A input under `expanding(min_periods=1).mean()` gives:

Time	Accumulated window	Calculation	Output
t_0	$\{100\}$	100/1	100
t_1	$\{100, 110\}$	210/2	105
t_2	$\{100, 110, 99\}$	309/3	103
t_3	$\{100, 110, 99, 108.9\}$	417.9/4	104.475

Rolling retains a fixed lookback; expanding keeps accumulating all history.

groupby in Long Format

Flattening the full table gives eight rows with `asset`, `date` and `price`. A global `df['price'].shift(1)` can pair different assets: when rows alternate by date, the preceding row may belong to another security; with asset blocks, B's first row may use A's last price. `df.groupby('asset')['price'].shift(1)` shifts separately and leaves each asset's first value NaN.

For the first two dates, sorted by date and asset:

Row	date/asset/price	Global previous	Within-asset previous	Correct return
0	t0/A/100	NaN	NaN	NaN
1	t0/B/50	100 (A)	NaN	NaN
2	t1/A/110	50 (B)	100 (A)	0.10
3	t1/B/52	110 (A)	50 (B)	0.04

The global calculation produces $50/100 - 1 = -50\%$ in row 1 and $110/50 - 1 = 120\%$ in row 2. Neither is the asset's adjacent return. Sort within groups first; groupby is not a sorting operation.

resample

`.resample('W-FRI').last()` assigns observations from Monday January 5 through Thursday January 8 to the bin ending Friday January 9. The output label is 2026-01-09, but A's value 108.9 comes from January 8, the last input observation. A bin boundary does not establish a trading session.

Cross-sectional Percentile rank

At t_1, $R_B = 4\% < R_A = 10\%$. `returns.rank(axis=1,pct=True)` compares assets on that date: B has rank 1 and percentile $1/2 = 0.5$; A has rank 2 and percentile $2/2 = 1.0$.

Axis checks on prices give `P.mean(axis=0)` = A 104.475, B 51.75, shape $(2,)$; `P.mean(axis=1)` = 75, 81, 75, 81.45, shape $(4,)$. The first averages time within columns; the second averages assets within rows. A mean price across unrelated securities has no natural investment

interpretation here; it only checks axes. Section 9's inherited array example has three different price rows and explicit weights, so its portfolio result does not belong to this table.

Label alignment gives January 5: $0.01 + \text{missing} = \text{NaN}$; January 6: $0.02 + 0.03 = 0.05$; January 7: $\text{missing} + 0.04 = \text{NaN}$. `shift(1)` retains January 5–8 labels and changes A's values to NaN/100/110/99. `shift(1,freq="D")` retains 100/110/99/108.9 and moves labels to January 6–9. Neither establishes executable prices.

With average ranks, two equal 1% returns have rank 1.5 each and percentile $1.5/2 = 0.75$. If only A is valid, its percentile is 1 while B remains NaN. Changing valid counts changes the ranking scale; report coverage.

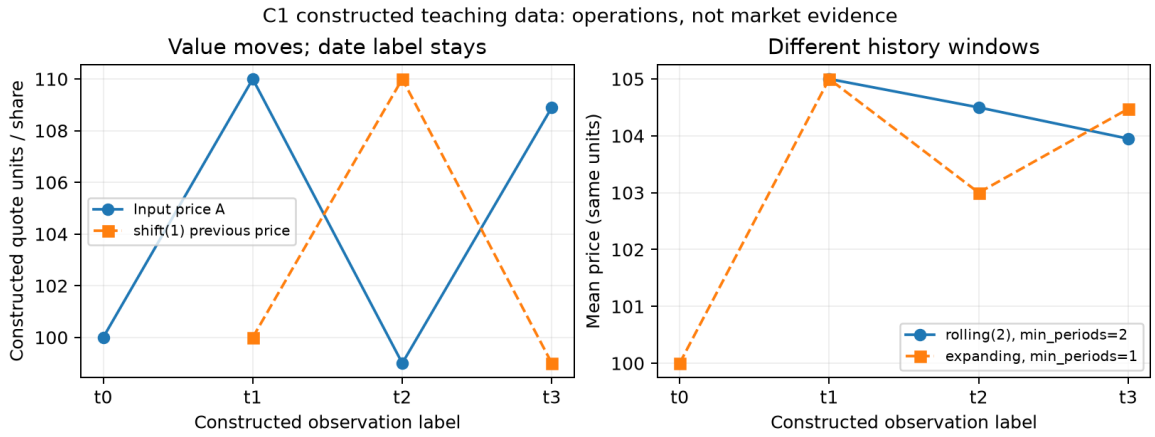


Figure 1: Shift and historical windows in the small table

The horizontal axis contains constructed t_0–t_3 labels; the vertical axis is quotation currency per unit. The left panel pairs current and preceding prices with no first predecessor. The right panel shows rolling(2) and expanding(1) diverging after t_2. It illustrates operations, without orders or market performance.

8.2 Three Price Observations, Two Compounding Intervals

Complete example: three prices, two intervals

A's first three prices, 100, 110 and 99, define two adjacent holding intervals.

- Price changes are $+10$ and -11, totaling -1 quotation unit.
- Simple returns are $+10\%$ and -10%; their sum 0% is not cumulative return.
- Wealth follows $100 \to 110 \to 99$, giving $99/100 - 1 = -1\%$.
- Log returns satisfy $\ln(1.1) + \ln(0.9) = \ln(0.99)$, matching the cumulative wealth ratio.

Machine output may contain a tail such as 0.10000000000000009. Hand and machine results agree within tolerance. Check units and dates before precision.

8.3 Compatible Shapes Can Still Use the Wrong Axis

Input conditions: time and asset axes

Consider two constructed holding intervals and two assets:

$$R = \begin{pmatrix} 0.10 & -0.05 \\ -0.10 & 0.10 \end{pmatrix}, \qquad w = (0.20, 0.80), \qquad g = (1.00, 0.50).$$

Rows are intervals; columns are assets A/B. w gives baseline risky-asset weights and g gives risky capital fractions specified before each interval. The remainder earns zero in cash; there are no fees. Weights are set at the interval start, without using its ending information to decide g. This checks axes and funding, not a risk-adjustment strategy.

A shape $(2,)$ asset vector broadcasts along the final, column dimension:

$$R \odot w = \begin{pmatrix} 0.02 & -0.04 \\ -0.02 & 0.08 \end{pmatrix}.$$

Here $\odot$ denotes elementwise multiplication. Row sums give $(-0.02, 0.06)$, or -2% and 6%.

To scale risky capital by date, use a column vector $g_{\text{col}} = (1, 0.5)^\top$, shape $(2, 1)$:

$$R \odot g_{\text{col}} = \begin{pmatrix} 0.10 & -0.05 \\ -0.05 & 0.05 \end{pmatrix}, \qquad (R \odot g_{\text{col}}) \odot w = \begin{pmatrix} 0.02 & -0.04 \\ -0.01 & 0.04 \end{pmatrix}.$$

Row sums are -2% and 3%. In the second interval the asset weights are $0.10/0.40$ and cash is 0.50; the budget remains 1.

The wrong $R \odot g$ also runs because the date count happens to equal the asset count. NumPy still broadcasts the one-dimensional vector across columns:

$$R \odot g = \begin{pmatrix} 0.10 & -0.025 \\ -0.10 & 0.05 \end{pmatrix}.$$

This halves B in both intervals instead of halving risky exposure only in the second. Multiplying by w then summing rows gives 0% and 2%, not -2% and 3%. `g[:,None]` explicitly retains the time axis with shape $(2, 1)$. Align asset weights to column labels as well. When $T = N$, a length check especially cannot establish axis meaning.

```
import numpy as np
r = np.array([[0.10, -0.05], [-0.10, 0.10]])
w = np.array([0.20, 0.80])
g = np.array([1.00, 0.50])
asset_contributions = r * w
date_contributions = (r * g[:, None]) * w
wrong_contributions = (r * g) * w
assert np.allclose(asset_contributions.sum(axis=1), [-0.02, 0.06])
assert np.allclose(date_contributions.sum(axis=1), [-0.02, 0.03])
assert np.allclose(wrong_contributions.sum(axis=1), [0.00, 0.02])
```

9. Python Implementation

Default Entry in the Accepted Reader Package

This sample carries forward the shared implementation of the accepted I3 reader. Create the environment following the reader installation instructions, then run from the reader-package root:

```
.venv/bin/python run_notebook.py notebooks/ch02_python.ipynb
```

The default Notebook uses packaged shared logic, reads no absent French data and downloads nothing. New outputs go to a unique runs subdirectory. The course-root code, lock file and historical tests below are the original working-source route; do not mix the two roots. Optional French files require authorized inputs under the optional-data contract and were not rerun for this batch. The reader CLI uses a new UUID directory and rejects overwrites. Constructed runs do not validate market data or fills.

Begin with ordinary expressions and list indexing:

```
prices = [100.0, 110.0, 99.0]
first_return = prices[1] / prices[0] - 1.0
second_return = prices[2] / prices[1] - 1.0
wealth = 100.0 * (1.0 + first_return) * (1.0 + second_return)
assert abs(wealth - 99.0) < 1e-12
print(first_return, second_return, wealth)
```

A caller supplies function inputs; the body calculates; `return` gives the result back. This standalone example demonstrates hints and explicit failure boundaries:

```
from collections.abc import Sequence
from math import isfinite
from numbers import Real


def final_wealth(initial: float, returns: Sequence[float]) -> float:
    if isinstance(initial, bool) or not isinstance(initial, Real):
        raise TypeError("initial must be a real number")
    if not isfinite(initial) or initial <= 0.0:
        raise ValueError("initial must be positive and finite")
    wealth = initial
    for value in returns:
        if isinstance(value, bool) or not isinstance(value, Real):
            raise TypeError("returns must be real numbers")
        if not isfinite(value) or value < -1.0:
            raise ValueError("simple return must be finite and at least -100%")
        wealth *= 1.0 + value
        if not isfinite(wealth):
            raise ValueError("wealth exceeds finite numeric range")
    return wealth


assert abs(final_wealth(100.0, [0.10, -0.10]) - 99.0) < 1e-12
for invalid in ([float("nan")], [float("inf")], [-1.01], [True], ["0.1"]):
    try:
        final_wealth(100.0, invalid)
    except (TypeError, ValueError):
        pass
    else:
        raise AssertionError("invalid return was accepted")
print(final_wealth(100.0, [0.10, -0.10]))
```

This teaching function rejects non-real inputs, booleans, NaN, infinity and returns below -100%, and checks compounding overflow. The research vector interface still uses Chapter 11's **wealth_index** rather than a second implementation. Allowing -100% to reduce wealth to zero does not establish defined account returns after bankruptcy. Type hints help tools but do not enforce inputs. ValueError rejects known contract violations. Catch only exception types you can handle; `except Exception: pass` hides data problems.

Imports bind reusable modules, commonly `import numpy as np` and `import pandas as pd`, or one imported function. Avoid local files called pandas.py, scipy.py or statistics.py that may shadow the intended module. An entry under `if __name__ == "__main__":` lets a script run directly and also be imported by a Notebook.

An isolated environment reduces dependency conflicts. The course interpreter and dependency versions are locked in README.md. From the course root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-a1-lock.txt
.venv/bin/python -m pip install -e . --no-deps
.venv/bin/python -m pytest tests -q
.venv/bin/python tools/run_a1.py
```

The lock file identifies the course snapshot. Temporary package exploration can use a disposable environment, but formal results return to the project environment and run from the beginning. Command-line and Notebook entry points share .venv; select its Python kernel. The src layout and editable installation make both import the same core functions.

The API is **compute_returns(prices: pd.Series, kind: Literal["simple", "log"] = "simple") -> pd.Series**, imported with `from quant_course import compute_returns`. The price index must be ascending, unique and timezone-aware; nonmissing prices must be positive and finite. A missing current price makes its return missing and also makes the following return missing because that row lacks a predecessor. The first result is NaN; nothing is filled. Check inputs and outputs explicitly. When passing this series to Chapter 11's wealth_index, exclude only the first position with no preceding price; all remaining holding intervals must be complete simple returns, with initial capital supplied separately. `dropna()` cannot erase unknown interior intervals and preserve a continuous path. Chapter 11 §9 provides the runnable bridge; that chapter remains a Chinese prerequisite until its later translation.

From the course root, run the script or start a Notebook:

```
.venv/bin/python code/ch02_python.py
.venv/bin/jupyter lab
```

The script locates the course root from its own path and adds src to the import path. The Notebook searches upward from its working directory. Both call the same return function.

Git records which code an analysis used. In an initialized course repository, inspect status, stage explicit files, inspect the staged diff, then commit:

```
git status
git add code/ch02_python.py notebooks/ch02_python.ipynb
git diff --staged
git commit -m "Add Chapter 2 Python examples"
git log --oneline -1
```

The staging area selects changes for the next commit. A commit is local; pushing is a separate action. Exclude environments, caches and local secrets:

```
.venv/
.env
*.ipynb_checkpoints/
```

Keep API keys, passwords and access tokens out of scripts, Notebook outputs and commits. Inject them through environment variables or managed credentials, and redact run records. .gitignore only prevents untracked files entering new commits. If a secret was committed, revoke and rotate it; later deleting the file does not resolve the exposure.

NumPy applies one expression to an array. In this separate three-row example, rows are dates and columns are assets; a length-N weight vector broadcasts along the final axis:

```
import numpy as np

prices = np.array([[100.0, 200.0], [110.0, 190.0], [99.0, 209.0]])
returns = prices[1:, :] / prices[:-1, :] - 1.0
weights = np.array([0.5, 0.5])
weighted_asset_returns = returns * weights
portfolio_returns = weighted_asset_returns.sum(axis=1)
assert prices.shape == (3, 2)
assert returns.shape == (2, 2)
assert np.allclose(portfolio_returns, [0.025, 0.0])
print(returns, portfolio_returns)
```

Shape compatibility does not establish that asset columns and weights are in the same order. Label and align weights before treating the result as a portfolio.

pandas aligns Series by index labels rather than row positions. This example produces three date labels, with NaN where the other side has no partner:

```
import pandas as pd

dates = pd.to_datetime(["2026-01-05", "2026-01-06", "2026-01-07"], utc=True)
left = pd.Series([0.01, 0.02], index=dates[:2], name="left")
right = pd.Series([0.03, 0.04], index=dates[1:], name="right")
combined = left + right
assert abs(combined.loc[dates[1]] - 0.05) < 1e-12
assert pd.isna(combined.loc[dates[0]])
assert pd.isna(combined.loc[dates[2]])
print(combined)
```

Executable Small-Table Operations

The following self-contained code reproduces all six operations in §8—shift, rolling, expanding, groupby, resample and rank—and asserts their values and intermediate behavior.

```
from math import isclose
import pandas as pd

dates = pd.to_datetime(["2026-01-05", "2026-01-06", "2026-01-07", "2026-01-08"], utc=
    ↪ True)
df_small = pd.DataFrame(
    {"A": [100.0, 110.0, 99.0, 108.9], "B": [50.0, 52.0, 51.0, 54.0]},
    index=dates,
)

# 1. shift and returns
df_shifted = df_small.shift(1)
df_returns = df_small / df_shifted - 1.0
assert pd.isna(df_shifted.iloc[0, 0])
assert isclose(df_returns.loc[dates[1], "A"], 0.10, rel_tol=1e-12)
assert isclose(df_returns.loc[dates[2], "A"], -0.10, rel_tol=1e-12)

# 2. rolling(2)
rolling_mean = df_small.rolling(window=2).mean()
assert pd.isna(rolling_mean.iloc[0, 0])
assert rolling_mean.loc[dates[1], "A"] == 105.0
assert rolling_mean.loc[dates[2], "A"] == 104.5

# 3. expanding(1)
expanding_mean = df_small.expanding(min_periods=1).mean()
assert expanding_mean.loc[dates[0], "A"] == 100.0
assert expanding_mean.loc[dates[1], "A"] == 105.0
assert isclose(expanding_mean.loc[dates[2], "A"], 103.0)

# 4. Cross-sectional rank(axis=1)
ranks = df_returns.rank(axis=1, pct=True)
assert ranks.loc[dates[1], "B"] == 0.5
assert ranks.loc[dates[1], "A"] == 1.0

# 5. Long-table groupby shift
df_long = (
    df_small.reset_index()
    .melt(id_vars="index", var_name="asset", value_name="price")
    .rename(columns={"index": "date"})
    .sort_values(["asset", "date"])
    .reset_index(drop=True)
)
df_long["prev_price"] = df_long.groupby("asset")["price"].shift(1)
df_long["ret"] = df_long["price"] / df_long["prev_price"] - 1.0
assert pd.isna(df_long.loc[0, "ret"])  # First row of A
assert pd.isna(df_long.loc[4, "ret"])  # First row of B
assert isclose(df_long.loc[1, "ret"], 0.10, rel_tol=1e-12)
assert isclose(df_long.loc[5, "ret"], 0.04, rel_tol=1e-12)

# 6. Weekly resample aggregation
weekly_last = df_small.resample("W-FRI").last()
assert weekly_last.index[0] == pd.Timestamp("2026-01-09", tz="UTC")
assert weekly_last.loc[pd.Timestamp("2026-01-09", tz="UTC"), "A"] == 108.9
```

Polars eager operations immediately return a DataFrame. Lazy operations build a plan and execute at collect. Filtering and selecting columns let the optimizer attempt to reduce file reading:

```
import polars as pl

data = pl.DataFrame(
    {
        "asset": ["A", "A", "B"],
        "date": [1, 2, 2],
        "return": [0.01, 0.02, -0.01],
    }
)
eager_result = (
    data.filter(pl.col("date") >= 2)
    .group_by("asset")
    .agg(pl.col("return").mean().alias("mean_return"))
)
lazy_query = (
    data.lazy()
    .filter(pl.col("date") >= 2)
    .select("asset", "return")
    .group_by("asset")
    .agg(pl.col("return").mean().alias("mean_return"))
)
print(lazy_query.explain())
lazy_result = lazy_query.collect()
assert lazy_result.height == eager_result.height
print(lazy_result)
```

Optimization depends on the actual plan and source. Check schema, missingness and sorting yourself; grouped results need not retain input order.

SciPy convergence cannot establish financial assumptions. Suppose 100 currency units are received in two years and have present value 90. Under annual compounding, solve for the unknown annual rate:

```
from scipy.optimize import brentq


def pricing_error(rate: float) -> float:
    return 100.0 / (1.0 + rate) ** 2 - 90.0

rate = brentq(pricing_error, 0.0, 1.0)
assert abs(pricing_error(rate)) < 1e-10
print(rate)
```

brentq requires opposite signs at the bracket endpoints. This example assumes specified cash-flow timing, annual compounding and no default. Actual security yields also depend on day count, calendars, cash flows, clean/dirty prices and call provisions. A root finder does not define a yield.

10. Visualization

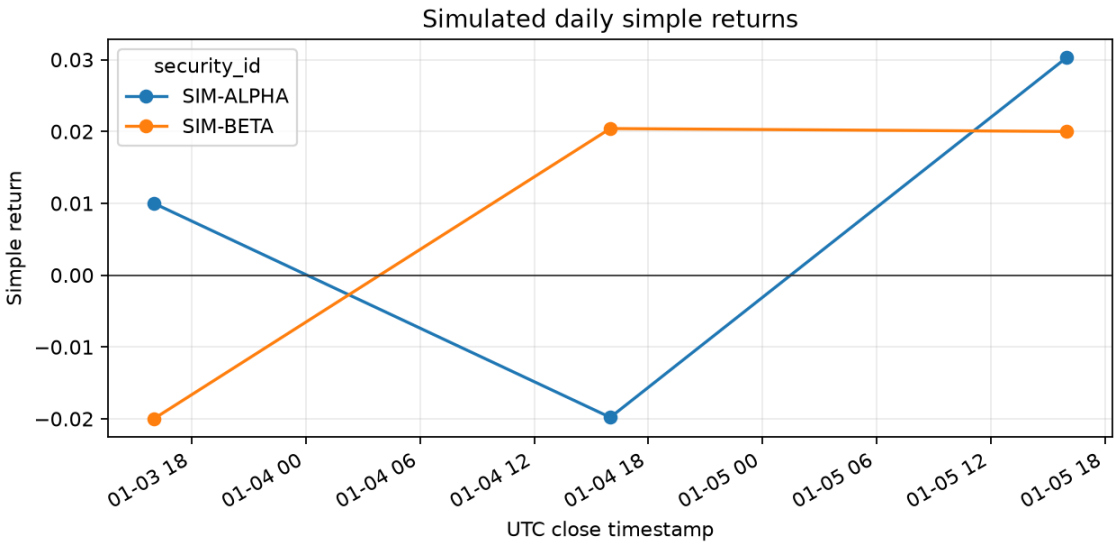


Figure 2: Simulated daily returns

This inherited figure uses two constructed securities, SIM-ALPHA and SIM-BETA. Its horizontal axis is UTC closing timestamps; its vertical axis is dimensionless simple-return decimals. The first missing return is not connected or drawn as zero. The title says simulated and the source is a local fixture. It is neither market observation nor a plot of §8's hand example.

A caption must identify simulated data, daily observations and UTC closing labels. A Figure is the whole image and an Axes is one plotting area. Set axis labels, units, legend, title and source before saving. A reader should understand the plot from the caption alone. A real French monthly plot would need its own version, frequency and return units.

11. Real Market Example: An Inherited Snapshot

Kenneth R. French Data Library publishes research-portfolio returns at monthly or daily frequencies. This chapter's snapshot selects the **Average Value Weighted Returns – Monthly** block of 10 Industry Portfolios. The inherited official description covers July 1926 through August 2026; the local processed file contains 1,202 months and ten industry columns. The original YYYYMM rows contain percentages. Processing divided them by 100 and retained YYYYMM month strings. Annual and equal-weighted blocks in the same source file must not enter this monthly value-weighted series. Daily returns come from a separate official file.

The processed monthly industry CSV has a data manifest. Download time: 2026-10-05 19:18:24 UTC. ZIP SHA-256: `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`. Processed SHA-256: `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`. Historical returns can change with CRSP revisions; preserve snapshot, download date and hashes. Fama and French retain copyright. Public download does not imply unrestricted redistribution; check terms before releasing derived data.

The monthly block begins near line 12 of the original file. Missing codes are −99.99 and −999. Processing selects that block, maps those codes to NaN, constructs a monthly PeriodIndex from YYYYMM and divides percentages by 100. The manifest records compressed, extracted and processed hashes. In the first month, 192607, NoDur is 0.0144 and Durbl is 0.1558.

Read the processed file and convert its month index:

```
import pandas as pd

raw = pd.read_csv(
    "data/processed/french_10_industries_monthly.csv",
    dtype={"month": "string"},
)
month = pd.PeriodIndex(raw.pop("month"), freq="M")
monthly_returns = raw.set_axis(month, axis="index")
assert abs(monthly_returns.loc[pd.Period("1926-07", freq="M"), "NoDur"] - 0.0144) < 1
    ↪ e-12
assert abs(monthly_returns.loc[pd.Period("1926-07", freq="M"), "Durbl"] - 0.1558) < 1
    ↪ e-12
print(monthly_returns.index.min(), monthly_returns.index.max())
```

These are portfolio returns, not daily OHLCV bars. They support exercises on frequency alignment, indices, ranking and grouping; they provide neither individual daily stock prices nor company-filing availability, historical constituents or delisting records. Do not present monthly portfolio returns as prices or future-investable decision times. This section describes format and reading rules, without claims about prediction, excess return or strategy performance.

12. Common Mistakes

Adding percentages. For 100, 110, 99, $0.10 - 0.10 = 0$, whereas compounding gives −1%. Multiply wealth factors.

Using shift(-1) as an available feature. It moves future values to current labels. This may construct a research target, but not a contemporaneously known input. Draw a timeline and check all inputs against signal_time.

Shifting a long panel globally. Alternating assets can pair A with B's price. Sort within asset groups and shift there, or use a checked date-by-asset wide table.

Ignoring alignment. Assigning a differently indexed Series to a DataFrame matches labels, not positions. Verify equal indices or reindex explicitly. Positional assignment requires proof of equal row order.

Misaligning rolling signals and future returns. `returns.rolling(20).mean()` includes the current row and retains its label. A close-t signal belongs to a later holding interval. `signal.shift(1) * returns` is a teaching row alignment; actual research still needs executable interval endpoints.

Creating observations at calendar month-end. `.resample("ME").last()` labels a monthly bin but uses its last available input. Assets may have different actual dates, and the boundary may be a nontrading day. Analyze monthly returns as returns; do not apply a price ratio again.

Treating missing as zero. A first endpoint, vendor omission, suspension or absence of trades does not establish zero return. compute_returns preserves NaN; cleaning and selection must be explicit. Filling zero before compounding invents a return.

Chained assignment. Under pandas 3.0 Copy-on-Write, `df["ret"][mask] = 0` does not modify the original object. Use `df.loc[mask, "ret"] = 0` and verify the result.

Calling decimal changes percentages. A pandas ratio of 0.01 means 1%, not 0.01%. Convert for a percent-labeled plot while retaining decimals for calculations.

Common mistake: compatible shapes, wrong axis

Broadcasting along the wrong semantic axis. (T, N) times $(N,)$ fits per-asset weights. If $T = N$, a $(T,)$ date vector also runs but multiplies columns. Print and assert shapes and document axis meanings.

Trusting an already-run Notebook. Cells can run out of order and retain stale variables. Save, restart the kernel and run all cells top to bottom. Rerun scripts in fresh processes and record library versions, input paths and parameters.

13. A Research Workflow

Start by defining assets and data: IDs, price convention, currency, frequency, source and version. Preserve raw data or a verifiable snapshot. Check time coverage, duplicate labels, sorting, dtypes, units and missingness after reading. Keep timezone information and use the market calendar to define sessions.

Use the shared return function. Start with §8's hand inputs, then verify contract failures for unsorted/duplicate dates and zero/negative prices. A missing interior price must leave its own and the next return missing. If downstream calculations need completeness, the caller explicitly selects the sample and records omissions. Automatic sorting or zero filling must not hide failures.

Define signals next. For cross-sectional ranks, establish which assets have available values on each date. Rank rows in a wide table, or group a long table by time before ranking assets. Compute historical windows within assets; shift prices and returns within the same asset. Record signal_time and each holding interval's trade_time, return_start and return_end.

Align signals to realizable subsequent returns, deduct interpretable trading and financing costs, and compare a simple benchmark. Report dates, valid counts, missingness rules, annualization, turnover and parameters. Select windows in training, assess decisions in validation and reserve testing for final evaluation. Save scripts, Notebooks, environment, configuration, figures and manifests. Report outcomes that fail the prespecified direction; a cumulative-return chart alone cannot show causation or lasting advantage.

A benchmark may equally weight the same universe or use a defined passive portfolio. Falsifying outcomes include benchmark-like returns, disappearance after costs, reliance on future data or one short period, and direction changes under small parameter variations. Python makes computations repeatable; evidence must still support the hypothesis.

14. Chapter Summary

Code defines operations; labels define their assets and times. NumPy broadcasts by shape, pandas aligns by labels, and Polars lazy API defers a plan. None chooses an appropriate financial definition for you.

Check positive prices, order, unique indices, frequency and missingness. Label returns by defined intervals, form signals from available inputs and align them to executable later holdings. Compound simple returns; use only signal-time information in windows. The command line, Notebook, plots and Git record should identify the same code, environment, snapshot and parameters.

15. Exercises

Complete answers are in §18. The first seven require no new package installation.

2-E01 — Concepts. Do Python type hints automatically validate arguments at runtime? Give an input check you would place in a financial function.

2-E02 — Concepts. An array has shape (5,3), with asset columns. How does a (3,) weight vector broadcast? How would you verify column meanings and weight order?

2-E03 — Concepts. How does pandas pair observations when adding Series with different date indices? Give a cause of alignment-induced NaN.

2-E04 — Concepts. Compare shift(1) and shift(1,freq="D"). Which moves values and which changes labels?

2-E05 — Hand Calculation. Prices move $80 \to 100 \to 90$. Calculate both simple returns, cumulative simple return and summed price changes. Explain the differences.

2-E06 — Hand Calculation. R_t is close-to-close and S_t forms at close t. At date t, which signal and interval does S.shift(1) * R use? What execution assumption is required?

2-E07 — Hand Calculation. A month's trading-day closing prices are 20, 21, 19; the first next-month price is 20. What is this month's closing observation and return? Why not invent a nontrading month-end observation at 19?

2-N01 — Identify the Mechanism.

1. Explain axis=0 and axis=1. Which computes each asset's time-series 20-observation mean and which ranks assets on a date?
2. Why must the first shift-based return be NaN? What fictional assumption does fillna(0) introduce?

3. Compare the historical information retained by rolling(window=3) and expanding(min_periods=1).

2-N02 — One Operation at a Time. For four consecutive trading-day prices [100, 110, 99, 108.9], write the shifted predecessor sequence, every rolling(2) window and mean, every expanding(1) window and mean, and each NaN position with its reason.

2-N03 — Complete Calculation. Use §8's A/B wide table. Calculate both assets' t_1, t_2, t_3 returns, the full percentile-rank matrix, and show exact agreement with sorted within-asset long-table calculations.

2-B01 — Broadcasting Axes. For §8.3, calculate every cell after asset weighting, proper date scaling and mistaken column scaling by the one-dimensional date vector. Compare the three portfolio sequences. Explain second-period cash, g[:,None]'s shape and why length checks fail when T=N.

16. Programming Exercises

2-P01 — Returns and Failure Conditions. Call compute_returns on UTC-indexed [100,110,99]; check simple returns, the first NaN and cumulative result. Assert failures for unsorted and duplicate indices, zero and negative prices. With an interior NaN, assert missing-value propagation and show that a completeness check rejects silent zero filling.

2-P02 — Multiple Assets. Construct a shuffled long table with at least two assets and six dates. Sort, shift within assets, calculate returns, three-interval rolling return means and datewise cross-sectional ranks. Assert missing first returns, no A/B predecessor contamination and prediction inputs restricted to the current or earlier date.

2-P03 — Month-end and French Format. From the course root, read the processed snapshot. Build a YYYYMM PeriodIndex without dividing its decimal returns by 100 again. Assert July 1926 NoDur=0.0144 and Durbl=0.1558; display coverage and missing counts. Use three constructed business-day prices to show that monthly .last() selects the last input observation; do not recalculate returns from monthly returns.

2-N04 — Diagnose Code. An intern uses this panel feature code:

```
# Long-table feature code to diagnose
import pandas as pd
df = pd.DataFrame({
    'asset': ['B', 'A', 'B', 'A', 'B', 'A'],
    'date': ['2026-01-02', '2026-01-01', '2026-01-01', '2026-01-03', '2026-01-03', '
    ↪ 2026-01-02'],
    'price': [52.0, 100.0, 50.0, 99.0, 51.0, 110.0]
})

# The intern shifts globally and fills zero
df['lag_price'] = df['price'].shift(1).fillna(0.0)
df['return'] = (df['price'] - df['lag_price']) / df['lag_price']
```

Identify the unsorted cross-asset mismatch and zero denominators/fictitious returns. Supply self-contained corrected code and assert within-group isolation.

17. Challenge Problems

2-C01 — Executable Signal Design. Design a monthly cross-sectional signal ranking the preceding 20 observed returns. Specify availability, signal_time, trade_time, return_start and return_end; map closing inputs to subsequent execution. Handle suspensions, missing returns, nontrading month-ends and rank ties. Extra prerequisites: calendars and portfolio weights. Ten points: timing contract 4, missingness/ties 2, interval calculations 2, costs/benchmark 2.

2-C02 — Memory and Query Plans. For T=1,000,000 and N=500, estimate float64 price-matrix bytes and the cost of an additional equally shaped difference matrix. Write a Polars lazy query filtering dates, selecting needed columns, deriving a column and aggregating by asset. Explain predicate/projection pushdown. Extra prerequisites: array bytes, schema and lazy plans. Ten points: memory 3, lazy/collect 3, optimization 2, memory/sorting limits 2.

2-N05 — Research Judgment. A report uses `df['price'].resample('ME').last()` for monthly rebalancing and claims to obtain monthly returns and immediately trade at each calendar month-end close. Diagnose:

1. Calendar boundaries versus real trading days.
2. The ordering problem in a zero-latency same-close fill.
3. Stale .last() prices for stocks suspended several days before month-end.

18. Complete Solutions

2-E01. Hints describe expected types to static tools; the interpreter does not enforce them. Explicitly check ascending/unique indices and finite positive prices, raising ValueError for violations. Inspect NaN separately; hints do not replace runtime checks.

2-E02. The three entries match columns and repeat across rows. Verify column labels/order, reindex labeled weights to prices.columns and reject missing weights. Assert the column count equals weight length and label sets agree.

2-E03. pandas pairs index labels. A/B on the left and B/C on the right produce A/B/C; A and C lack partners and become NaN. Equal row numbers are insufficient.

2-E04. shift(1) retains the index and maps the preceding value to the current label. shift(1,freq="D") retains values/order and moves labels one calendar day forward. A previous trading observation need not be yesterday.

2-E05. Returns are $100/80 - 1 = 25\%$ and $90/100 - 1 = -10\%$. Cumulative return is $1.25(0.90) - 1 = 12.5\%$. Price changes $+20, -10$ sum to $+10$ currency units, without normalization by initial wealth; this is not the sum of returns. Wealth moves $80 \to 100 \to 90$.

2-E06. At t, the shifted value is S_{t-1}, multiplied by R_t. If that interval is close $t-1$ to close t, the expression holds the earlier signal for one interval and assumes execution at its start. If the signal requires the finalized close $t-1$, establish actual execution and measure returns from that executable time.

2-E07. The last input is 19, so the monthly closing observation is 19. The preceding month's closing price is absent, so a full-month return cannot be calculated. From this month's first observation 20 to 19, the narrower interval returns -5%. A nontrading day has no new close; do not invent or repeat an observation.

2-N01.

1. axis=0 aggregates across rows/time; use it for asset time-series means. axis=1 works across columns/assets within a timestamp; use it for cross-sectional ranks.
2. The first row lacks P_{-1}. Filling zero invents an observed zero-return interval and changes the valid count from T$-$1 to T, affecting mean and volatility. Multiplication by 1 does not change ending wealth but cannot turn an endpoint into an observed holding interval.
3. Rolling retains only the latest fixed W observations and forgets older ones, suited to local changing statistics. Expanding keeps all available history from the start and grows without forgetting.

2-N02. The shifted sequence is $[\text{NaN}, 100, 110, 99]$. Rolling windows are $\{100\}$ (insufficient, NaN), $\{100, 110\}$ (105), $\{110, 99\}$ (104.5) and $\{99, 108.9\}$ (103.95). Expanding windows are $\{100\}$ (100), $\{100, 110\}$ (105), $\{100, 110, 99\}$ (103) and $\{100, 110, 99, 108.9\}$ (104.475). shift and rolling are missing at t_0 because preceding/minimum observations are absent.

2-N03. Returns to four decimals:

Time	A	B	A percentile	B percentile
t_0	NaN	NaN	NaN	NaN
t_1	$110/100 - 1 = 0.1000$	$52/50 - 1 = 0.0400$	1.0	0.5
t_2	$99/110 - 1 = -0.1000$	$51/52 - 1 = -0.0192$	0.5	1.0
t_3	$108.9/99 - 1 = 0.1000$	$54/51 - 1 = 0.0588$	1.0	0.5

Sort the long table by asset/date and compute unfilled ratios inside groups, or pct_change(fill_method=None). A yields [0.10,$-$0.10,0.10], B yields [0.04,$-$0.0192,0.0588] and both begin NaN, exactly matching their wide columns before rounding.

2-B01. Baseline contributions are (0.02,$-$0.04)/($-$0.02,0.08), with $-2\%/6\%$ returns. Correct date-scaled contributions are (0.02,$-$0.04)/($-$0.01,0.04), with $-2\%/3\%$. Wrong column scaling gives (0.02,$-$0.02)/($-$0.02,0.04), with $0\%/2\%$. Correct second-period risky weights are 0.10/0.40 and cash is 0.50. g[:,None] is (2,1), explicitly indexed by date; a one-dimensional (2,) vector aligns to the last dimension. Equal lengths cannot prove semantic alignment.

2-P01 — Reference Implementation. Run from the course root using the shared API:

```
import pandas as pd
from quant_course import compute_returns

dates = pd.date_range("2026-01-02", periods=3, freq="B", tz="UTC")
prices = pd.Series([100.0, 110.0, 99.0], index=dates, name="price")
got = compute_returns(prices, kind="simple")
assert pd.isna(got.iloc[0])
assert abs(got.iloc[1] - 0.10) < 1e-12
assert abs(got.iloc[2] + 0.10) < 1e-12
assert abs((1.0 + got.iloc[1]) * (1.0 + got.iloc[2]) - 0.99) < 1e-12

def must_raise_value_error(bad):
    try:
        compute_returns(bad, kind="simple")
    except ValueError:
        return
    raise AssertionError("invalid index or price was accepted")

must_raise_value_error(prices.iloc[::-1])
duplicated = pd.Series([100.0, 110.0], index=[dates[0], dates[0]])
must_raise_value_error(duplicated)
must_raise_value_error(pd.Series([100.0, 0.0], index=dates[:2]))
must_raise_value_error(pd.Series([100.0, -1.0], index=dates[:2]))

with_missing = pd.Series([100.0, float("nan"), 99.0], index=dates)
missing_returns = compute_returns(with_missing, kind="simple")
expected_valid = with_missing.notna() & with_missing.shift(1).notna()
expected_valid.iloc[0] = False
assert missing_returns.notna().equals(expected_valid)
silently_filled = missing_returns.fillna(0.0)
assert not silently_filled.notna().equals(expected_valid)
```

Valid returns are 0.10 and -0.10, cumulative -0.01. An interior missing price removes its current return and the following return. The last lines show why absence of NaN after filling is dangerous; actual research must record selection and missingness rather than use this fill in results.

2-P02 — Reference Implementation. This constructed six-date table ensures at least one valid three-return rolling mean and lagged signal.

```
import pandas as pd

rows = [
    ("A", "2026-01-06", 104.0), ("B", "2026-01-01", 50.0),
    ("A", "2026-01-01", 100.0), ("B", "2026-01-06", 58.0),
    ("A", "2026-01-02", 101.0), ("B", "2026-01-02", 52.0),
    ("A", "2026-01-03", 103.0), ("B", "2026-01-03", 53.0),
    ("A", "2026-01-05", 105.0), ("B", "2026-01-05", 55.0),
    ("A", "2026-01-04", 102.0), ("B", "2026-01-04", 54.0),
]
df = pd.DataFrame(rows, columns=["asset", "date", "price"])
df["date"] = pd.to_datetime(df["date"], utc=True)
df = df.sort_values(["asset", "date"]).reset_index(drop=True)
```

```
df["previous_price"] = df.groupby("asset", sort=False)["price"].shift(1)
df["return"] = df["price"] / df["previous_price"] - 1.0
df["mean_3"] = (
    df.groupby("asset", sort=False)["return"]
    .rolling(window=3, min_periods=3)
    .mean()
    .reset_index(level=0, drop=True)
)
df["signal"] = df.groupby("asset", sort=False)["mean_3"].shift(1)

cross_section = df.pivot(index="date", columns="asset", values="return")
rank = cross_section.rank(axis=1, pct=True, method="average")
assert df.groupby("asset").head(1)["return"].isna().all()
a = df.loc[df["asset"].eq("A")].reset_index(drop=True)
b = df.loc[df["asset"].eq("B")].reset_index(drop=True)
assert a.loc[1, "previous_price"] == 100.0
assert b.loc[1, "previous_price"] == 50.0
assert df.groupby("asset")["signal"].apply(lambda s: s.notna().any()).all()
print(df)
print(rank)
```

Sort before shifting within each group. The rolling window requires three valid returns; the initial rows do not yet provide them. Shift the mean again so a same-row estimate is not applied to that row's return interval. rank compares assets at one date, without mixing A's multiple dates into B's cross-section.

2-P03 — Reference Implementation. The processed returns are already decimals.

```
from pathlib import Path
import pandas as pd

path = Path("data/processed/french_10_industries_monthly.csv")
raw = pd.read_csv(path, dtype={"month": "string"})
month = pd.PeriodIndex(raw.pop("month"), freq="M")
monthly = raw.set_axis(month, axis="index")
assert abs(monthly.loc[pd.Period("1926-07", freq="M"), "NoDur"] - 0.0144) < 1e-12
assert abs(monthly.loc[pd.Period("1926-07", freq="M"), "Durbl"] - 0.1558) < 1e-12
print(monthly.index.min(), monthly.index.max(), int(monthly.isna().sum().sum()))
```

Coverage is July 1926–August 2026. Original missing codes became NaN in processing and the manifest preserves snapshot/hashes/transformation. This separate construction verifies that a monthly bin can have a non-observation boundary:

```
import pandas as pd

prices = pd.Series(
    [20.0, 21.0, 19.0],
    index=pd.to_datetime(["2026-01-28", "2026-01-29", "2026-01-30"]),
)
month_end = prices.resample("ME").last()
assert month_end.iloc[0] == 19.0
assert month_end.index[0] == pd.Timestamp("2026-01-31")
```

The label is January 31 but the value comes from January 30. Check each asset's actual observation date in larger samples. Define monthly returns from appropriate monthly prices or original monthly returns; do not take ratios of returns again.

2-N04. The unsorted global shift contaminates asset predecessors. Filling the absent price with zero creates infinite or absurd returns and hides the first statistical boundary. Correct it as follows:

```
from math import isclose
import pandas as pd

df = pd.DataFrame({
    'asset': ['B', 'A', 'B', 'A', 'B', 'A'],
    'date': ['2026-01-02', '2026-01-01', '2026-01-01', '2026-01-03', '2026-01-03', '
    ↪ 2026-01-02'],
    'price': [52.0, 100.0, 50.0, 99.0, 51.0, 110.0]
})

# 1. Explicit timestamp conversion and two-key sort
df['date'] = pd.to_datetime(df['date'], utc=True)
df = df.sort_values(['asset', 'date']).reset_index(drop=True)

# 2. Within-group shift and returns, retaining NaN
df['lag_price'] = df.groupby('asset')['price'].shift(1)
df['return'] = df['price'] / df['lag_price'] - 1.0

# Assert group isolation and first-row missingness
assert pd.isna(df.loc[0, 'return'])  # First row of A
assert pd.isna(df.loc[3, 'return'])  # First row of B
assert isclose(df.loc[1, 'return'], 0.10, rel_tol=1e-12)
assert isclose(df.loc[4, 'return'], 0.04, rel_tol=1e-12)
```

2-C01 — Reference Argument and Scoring. Form the mean and rank after the last required monthly information becomes available and enters the system. signal_time cannot precede either available_at or ingested_at; equal stamps need proof of receipt and processing. Trade at a later executable time, define return_start from the fill and return_end by the holding horizon. For a nontrading month-end, use the venue calendar to locate the preceding actual observation. Preserve missing returns and state sample conditions; use deterministic average ranks or a stable tie rule. Align subsequent returns by interval labels and include an after-cost benchmark and falsification condition. Award 4+2+2+2 points as specified; an unsupported same-close fill earns no timing-contract points.

2-C02 — Reference Argument and Scoring. Storage is $1{,}000{,}000 \times 500 \times 8 = 4{,}000{,}000{,}000$ bytes: 4.0 GB decimal or about 3.73 GiB binary. A materialized equal-sized difference array adds at least that much; temporaries, copies, results and interpreter overhead add more. Start a LazyFrame with scan, filter dates, select columns, derive with with_columns, group_by("asset").agg(…), then collect. Predicate pushdown filters early and projection pushdown reduces read columns. Inspect explain() for the actual plan. Award 3+3+2+2 points; include sorting requirements and the final result's unavoidable memory.

Here is the required concrete query using §9's constructed three-row long table: asset is a string, date an integer and return a decimal. This is a different input layout from the million-by-500 wide matrix in the memory question. Filter date≥2, retain needed columns, derive 100×return in percent units and aggregate by asset:

```
import polars as pl

data = pl.DataFrame({
    "asset": ["A", "A", "B"],
    "date": [1, 2, 2],
    "return": [0.01, 0.02, -0.01],
})
query = (
    data.lazy()
    .filter(pl.col("date") >= 2)
    .select("asset", "date", "return")
    .with_columns((100.0 * pl.col("return")).alias("return_pct"))
    .group_by("asset")
    .agg(pl.col("return_pct").mean().alias("mean_return_pct"))
)
print(query.explain())
result = query.collect().sort("asset")
assert result["asset"].to_list() == ["A", "B"]
assert result["mean_return_pct"].to_list() == [2.0, -1.0]
print(result)
```

The retained rows are A/date2/0.02 and B/date2/−0.01. Derived values are 2 and −1, so the final means are A=2%, B=−1%. For files, use scan_csv or another scan entry with explicit schema and inspect explain for actual optimization. No giant matrix is constructed; this tiny query does not measure large-table performance.

2-N05.

1. Calendar month-ends can fall on weekends or holidays. Labeling them as fills invents an execution clock.
2. A closing price is finalized after the auction and must reach this system before forming the signal. A signal formed from that close cannot simply claim a simultaneous same-close fill. Execution depends on venue hours, order type and latency; labels and shift do not establish it.
3. For a suspended stock, .last() can pick an earlier stale price and relabel it month-end. Other stocks may have current observations; pooling these values creates unequal holding spans and invalid cross-sectional comparisons.

19. Further Reading

Must know: Python's official control-flow, function, module, exception, virtual-environment and floating-point chapters; NumPy array basics and broadcasting; pandas indexing, time series, windows, GroupBy, resampling and Copy-on-Write; Pro Git's basic recording workflow. These support independent maintenance of a small return-analysis project.

Should know: Polars DataFrame, expressions, Lazy API, CSV scan and query plans; Matplotlib Quick Start's Figure/Axes lifecycle; Jupyter kernels/execution; SciPy stats, optimize and linalg. Compare APIs with the installed version.

Advanced: NumPy broadcasting memory, Polars predicate/projection pushdown, accumulated floating-point error and stable algorithms. Broadcasting may avoid copies but can create large outputs; estimate shapes and bytes first.

Optional: Pro Git's branches, remotes and recovery chapters. Working tree, staging area and commit suffice to begin recording; study branching and merging when collaboration requires them.

20. References and Reading Boundaries

Specific pages, inherited verification dates and reading limits are recorded in Chinese source audit record. NUMPY, PANDAS, POLARS and SCIPY source keys appear in the course references.md. The following official materials support terminology and API behavior; the market example uses the French official data page and the recorded local snapshot.

- Python documentation: control flow, exceptions, modules, hints, venv and floating-point arithmetic.
- NumPy User Guide: arrays and broadcasting.
- pandas User Guide: alignment, time indices, shifting, resampling, windows and Copy-on-Write.
- Polars User Guide: eager/lazy, scan, optimization and collect.
- SciPy User Guide: numerical algorithms, statistics and optimization.
- Matplotlib Quick Start: Figure/Axes, annotation and saving.
- Project Jupyter documentation: Notebook, IPython and kernel entry points.
- Pro Git: working tree, staging, commits and recovery.
- Kenneth R. French Data Library: portfolio definitions, frequencies, coverage, revisions and missing codes.

This translation introduces no new live documentation reading, download or market execution.

Chapter 3 — Financial Data Engineering

Prerequisite: Chapter 2's data and Python foundations. After this chapter, you can check market records and corporate actions, select historically visible versions, explain missingness and security identities, and specify signal times and holding-return intervals.

1. Learning Objectives

You should be able to deliver these checks for a colleague to review:

1. Explain the objects, units and intervals described by Open, High, Low, Close and Volume (OHLCV).
2. Distinguish raw prices, split-adjusted prices and dividend-reinvested total-return series, and identify the questions each answers.
3. Calculate wealth returns from share quantities and cash through splits and dividends; recognize that missing delisting returns cannot simply become zero.
4. Classify missing data rather than merge nontrading days, suspensions, zero trading and pipeline failures into one blank.
5. Define execution times using venue time zones and calendars, then compare timezone-aware timestamps in UTC.
6. Preserve event, external availability, ingestion and revision times for each fact, and apply a bitemporal point-in-time (PIT) contract.
7. Link symbol changes through permanent security IDs, rebuild the contemporaneous universe and inspect survivorship and selection.
8. Preserve source, version, quality and license constraints for reproducible, authorized research.

Deliver an auditable dictionary, list every input visible at a signal time, specify execution and holding intervals, and explain each exclusion.

2. Why Quantitative Research Needs Data Engineering

Research turns a market hypothesis into orders. Selecting securities, reading indicators, calculating returns and simulating fills all depend on inputs. Future information hidden in those inputs cannot become executable evidence through better statistics.

Suppose a Friday-close signal uses prices back-adjusted for a split announced over the weekend, earnings overwritten by a later correction and a list containing only today's surviving stocks. These records may accurately describe history as known today, yet differ from what was visible on Friday.

Follow the whole lifecycle:

Raw records → security identity and calendar → corporate actions and versions → signal-time snapshot → executable order time → holding returns → costs and performance.

A unit error needs a unit contract; a timing error needs a timing contract; a universe error needs historical membership; reproducibility needs snapshots, versions and rights. Executing a cleaning rule does not establish truth or lack of bias.

3. Intuition: Every Value Has a Source and an Availability Time

"A stock at 20.00" is incomplete. Is the currency EUR or USD? Is the quote per share, American Depositary Receipt (ADR) or pre-split unit? Is it an auction close or last intraday trade? What was the venue-local date? When did the vendor publish it and this system receive it? Was it revised?

Think of each record as a receipt with distinct clocks:

- The event clock describes the economic occurrence, such as a closing trade, ex-dividend event or reporting period.
- The release clock describes the earliest external availability, such as a company announcement or regulatory acceptance.
- The system clock records actual receipt of that version.
- The revision clock identifies a correction without erasing its predecessor.

Suppose an after-close initial earnings-per-share (EPS) report says 10, but arrives in this system later. The system may use 10 only after receipt. A future restatement to 8 belongs to later information sets, not earlier signals. A database showing only 8 today does not change what traders could have seen then.

The inherited New York Stock Exchange (NYSE) official description specifies core hours of 9:30–16:00 Eastern Time. New York 9:30 is 13:30 UTC in daylight saving time and 14:30 UTC in standard time. Treating a naive 9:30 as UTC is wrong in both seasons. Time zone is part of the record.

4. Prices, Returns, Actions and Time

4.1 OHLCV Objects and Dimensions

For security i and bar interval t, let $O_{i,t}, H_{i,t}, L_{i,t}, C_{i,t}, V_{i,t}$ denote open, high, low, close and volume. Prices are quotation currency per quoted unit; volume may mean shares, units or contracts. Daily, minute and auction intervals are different objects.

Basic internal checks require

$$L_{i,t} \le \min(O_{i,t}, C_{i,t}) \le \max(O_{i,t}, C_{i,t}) \le H_{i,t}, \qquad V_{i,t} \ge 0.$$

These are necessary, not sufficient. Multiplying every price by 100 preserves inequalities but may confuse currency units with cents. Zero volume may mean no trade or a fault sentinel. State volume units and whether prices come from official auctions, last trades, aggregated bars or vendor synthesis.

For a defined price convention,

$$R_{i,t} = \frac{P_{i,t}}{P_{i,t-1}} - 1, \qquad r_{i,t} = \ln\left(\frac{P_{i,t}}{P_{i,t-1}}\right).$$

Endpoints must share security, currency, share unit and price definition. Both returns are dimensionless decimals over $t-1$ to t. Simple return measures change per unit of initial wealth. Log returns add over consecutive intervals via $\ln(ab) = \ln a + \ln b$, but do not directly capital-weight portfolios. Without an action convention, the price ratio lacks a complete economic interpretation.

4.2 Raw Prices, Split Adjustment and Total Return

A raw price is the nominal market quote at the time. Adjustment rescales history under a specified action rule to make returns comparable; the adjusted value is not an actual historical fill quote. Price return counts quotation changes; total return also counts distributions. Raw prices support order checks, microstructure and execution; ratios across splits mix economics with a unit change.

If one old share becomes q new shares, with q measured as new shares per old share,

$$N_{\text{post}} = qN_{\text{pre}}, \qquad P_{\text{pre}}^{(\text{new unit})} = \frac{P_{\text{pre}}^{(\text{raw})}}{q}, \qquad N_{\text{pre}}P_{\text{pre}}^{(\text{raw})} = N_{\text{post}}P_{\text{post}}^{(\text{raw})}.$$

N is share quantity and P is currency per share. The last equality describes the ideal conversion without another market move: a split itself creates no wealth, but subsequent prices need not stay constant. Historical volume in new-share units can become $V_{\text{pre}}^{(\text{new unit})} = qV_{\text{pre}}^{(\text{raw})}$; verify the vendor's factors and volume definition.

Back-adjustment may be written

$$P_t^{(\text{adj})} = P_t^{(\text{raw})}A_t, \qquad A_t = \prod_{j>t} a_j.$$

Each future action j contributes a dimensionless factor a_j. Vendors differ in factor direction, special-dividend treatment, dates and precision. "Adjusted close" alone does not specify a method. It describes history rescaled using the current factors, rather than the original quote. Later factors can alter past price-level signals. Adding dividends again to a total-return-adjusted series double counts them.

For comparable share units and currencies, an entitled beginning holding and no taxes, fees or reinvestment, the simplified pre-reinvestment total return is

$$R_t^{(\text{TR})} = \frac{P_t + D_t}{P_{t-1}} - 1.$$

D_t is cash per share belonging to the interval. If a split or exchange occurs inside it, first transform quantities, prices and flows, then calculate account wealth. Declaration, record, ex-dividend and payment dates represent different events. Entitlement follows holding rights and market rules; declaration is not cash receipt.

4.3 Bitemporal Records and Versions

Economic valid time and system-record time are the two basic bitemporal axes: when a fact applies in the world and when the system records a version. Financial research also needs external release/availability. Keep **event_time**, **available_at**, **ingested_at** and **revision_time** distinct. valid_from/valid_to and system_from/system_to can preserve intervals of fact applicability and knowledge. An announcement date labels release; a vendor's report date may mean period-end, filing date or something else. Period-end describes coverage, not availability. None substitutes for a precise available_at.

For fact key k, version v is eligible at signal time s only when both gates hold:

$$\text{eligible}(v, s) = \mathbf{1}\{\text{available_at}(v) \leq s\}\mathbf{1}\{\text{ingested_at}(v) \leq s\} = 1.$$

All three clocks are comparable timezone-aware instants; $\mathbf{1}$ is an indicator. Equality requires evidence that the version was already visible at the signal. If second-resolution clocks cannot establish ordering, use strict inequality or finer timestamps/receipt sequence numbers.

Let $\mathcal{V}(k, s)$ contain versions of the same fact key satisfying both gates. Select the latest external version:

$$v^*(k, s) = \underset{v \in \mathcal{V}(k,s)}{\arg\max}\ \text{available_at}(v).$$

available_at orders source versions; ingested_at determines local eligibility, not revision order. Multiple conflicting rows at the same key and available_at require an ambiguity error and quarantine, not a row-order winner. $\max(\text{available_at}, \text{ingested_at})$ is the first eligibility time; it neither orders source versions nor replaces the original clocks.

Keep period-end, company announcement, regulator acceptance, vendor availability and ingestion distinct. A fact describes a past period without being known at its end. An as-of query reconstructs visibility at the specified instant, rather than selecting the table's latest row.

4.4 Identities, Calendars and Missingness

A permanent security key K_i identifies an economically continuous security. Map symbols, venues, currencies and valid intervals to it, and retain when the mapping entered the system. Otherwise a later-discovered identity relation can leak backward. Permanent company keys, security keys and tickers are different levels.

A venue calendar defines session dates S_e and their open, close, breaks, early closes and auction windows. It is more than weekdays. Define sessions in venue-local time and convert to UTC. IANA zone rules handle daylight saving changes; a fixed offset cannot replace them.

A missing indicator $M_{i,t}$ says no usable observation exists, not why. A separate status $Z_{i,t}$ should classify nontrading day, suspension, no trade, quote outage, delisting, uncovered security, inapplicable field or parsing failure. Do not map missing price to zero return without a rule informed by holdings, signals, calendar and reason.

5. Field Contracts

A minimal auditable market row includes:

Field	Definition and check
security_id	Permanent security ID, issuing authority, level and validity
symbol, venue	Venue and symbol with valid dates; symbols are not globally unique
event_time, session_date	Event instant and venue-local session, possibly across UTC dates
open, high, low, close, volume	Numeric units, currency, aggregation and raw/adjusted status
available_at, ingested_at	External availability and local receipt, both timezone-aware
source_id, source_version	Source and snapshot, file, API version or batch
action_id, adjustment_method	Action linkage and factor method if adjusted
quality_status	Checks, quarantine reason and repair person/program version

A financial fact also needs issuer_id, concept, unit, start/end or instant, reporting period, form, accession/document version, announcement/acceptance, vendor, revision and ingestion times. Keep announcement_date, report-period end, as-of date, revision_time and ingested_at. Matching concepts need not be comparable across currencies, durations, consolidation scopes, units or contexts.

Corporate actions change quantities, cash or identities: splits and reverse splits, cash/stock dividends, mergers, spin-offs, rights issues, exchanges, special distributions and delistings. Preserve events and effective dates, not only final adjusted prices. Distinguish price and shareholder total returns; separately record share changes, cash receivables, payments and fees.

Delisting means leaving a venue or sample, not automatically losing all wealth or deleting a return. Acquisition, bankruptcy, transfer and privatization can give different cash, exchange or liquidation proceeds. If terminal value is absent, preserve unknown and investigate; do not substitute zero or the last quote. For two sequential simple returns on the same holding and currency,

$$R_t^{(\text{combined})} = (1 + R_t^{(\text{regular})})(1 + R_t^{(\text{delist})}) - 1.$$

This does not apply when a source defines the two as alternative estimates for the same interval.

PIT data contains only versions and securities visible at a historical decision time. A valid query constrains identity, valid intervals, both clocks, version order and signal_time. Joining a quarter to market dates is insufficient.

Data quality needs evidence about completeness, timeliness, units, uniqueness, internal consistency and source reliability. A version records original payload, retrieval time, hash, software/cleaning version and revision relations; each change retains a traceable identity. A license specifies access, use, retention, display, redistribution, commercial, derivative and territorial rights. Free access or teaching use does not by itself authorize redistribution.

6. From Records to Visible Returns

6.1 Reconstruct Wealth Before Calculating Returns

Start with N_0 shares at raw price P_0. A 2:1 split gives $q = 2$, then each new share has cash entitlement $D = 1$; ending price is P_1. Entitlement belongs to both new shares. Ignore taxes/fees and reinvestment; record the dividend as a receivable at the endpoint.

$N_1 = qN_0$, with shares and new/old share units explicit. In one currency,

$$W_1 = N_1(P_1 + D) = qN_0(P_1 + D), \qquad W_0 = N_0P_0,$$

so

$$R_{0,1} = \frac{W_1}{W_0} - 1 = \frac{q(P_1 + D)}{P_0} - 1 = \frac{P_1 + D}{P_0/q} - 1.$$

Cancellation uses $N_0 > 0$ and continuity of the same holding. A split creates no wealth; entitlement rules determine dividends. A dividend declared per old share must be converted before using a per-new-share amount.

6.2 Period-end Is Not Availability

Availability conditions: both clocks must pass

All numbers and timestamps here are constructed fixture data. Initial 2023Q4 EPS=10 is released March 15, 2024 at 16:05 New York daylight time and ingested at 16:12. A 16:10 signal cannot use it; 16:15 can. Times have offset −04:00.
At $s = 16{:}10$, release $a = 16{:}05 \le s$, but ingestion $g = 16{:}12 > s$, so the conjunction fails. A later executable order may use the report after ingestion. If a vendor publishes at 16:25 but a system retrieves an authorized file at 16:24, availability still depends on the license and research contract; do not misrepresent information unavailable to other users as public.

6.3 Revisions Preserve Prior States

These constructed versions match the PIT fixtures.

Version	EPS	available_at	ingested_at	revision_time
Initial v1	10	2024-03-15 16:05 −04:00	2024-03-15 16:12 −04:00	2024-03-15 16:05 −04:00
Restated v2	8	2024-04-02 16:05 −04:00	2024-04-02 16:08 −04:00	2024-04-02 16:05 −04:00

March 15 at 16:10 has no visible initial report; 16:15 selects 10. April 2 at 16:06 still selects 10 because the restatement has not arrived; 16:10 selects 8. Rank eligible versions by available_at. A future restatement must not overwrite earlier signals.

6.4 Signal, Execution and Label Intervals

Separate signal_time, trade_time, return_start and return_end. Daily close data often becomes available after the auction, with a subsequent signal and next-session order. A same-close fill needs an explicit auction deadline, calculation latency and participation design; otherwise it uses future prices as fills.

Measure returns after the holding actually takes effect through exit or valuation. Track rights through suspensions, delisting and actions. Prediction labels additionally have label_start, label_end and label_available_at. A label ending after the training cutoff remains invisible even if its feature was already known.

7. Financial Meaning of Transformations

Raw, adjusted and total-return series measure different questions. Raw quotes support execution; split adjustment makes share units comparable; total return measures price changes plus distribution rights. Preserve corresponding fields for execution, performance and factor definitions.

A signal based on back-adjusted price levels need not use a historically visible nominal quote. Adjacent returns are invariant if a vendor later multiplies all history by one constant. If factors change differently across dates, returns can change too. Distinguish scale-invariant calculations from price-level-dependent ones and inspect source versions.

Actions affect holdings and returns; delisting affects sample boundaries. Selecting eventual survivors or securities with complete prices removes failures and difficult exits. A strategy for the historical investable universe needs that universe rebuilt, including securities that later disappeared and terminal-value quality.

Information arrives, is processed, informs a signal and only then enters an order. Returns begin after a holding could actually be established, not at the report-period end. This ordering does not guarantee profit, but excludes profits relying on unavailable inputs.

8. Worked Numerical Example

Constructed security SEC_ABC uses permanent key PERM_001 across prices, fact versions and actions. Prices are USD per share. Dates and times do not claim real NYSE securities, sessions or tradability; version times use −04:00 and can be converted to UTC. The financial event describes 2023Q4; release and receipt are separate. An EPS key must fix concept, period, currency and share unit. Split-era EPS is not comparable without the vendor's unit convention.

8.1 Filter Each Version Before Ordering It

Version	EPS: USD/report share unit	available_at	ingested_at
v1 initial	10	3/15 16:05	3/15 16:12
v2 restated	8	4/2 16:05	4/2 16:08
v_old older correction arriving late	9.5	3/20 10:00	4/20 09:00

Each cell below gives release/ingestion eligibility. "yes" means its clock is no later than the query.

Query/as-of s	v1	v2	v_old	Eligible set → selection
3/15 16:00	no/no	no/no	no/no	Empty → unknown; period-end is insufficient
3/15 16:10	yes/no	no/no	no/no	Empty → unknown; ingestion is late
3/15 16:15	yes/yes	no/no	no/no	{v1} → 10
4/2 16:06	yes/yes	yes/no	yes/no	{v1} → 10
4/2 16:10	yes/yes	yes/yes	yes/no	{v1,v2} → 8
4/21 16:00	yes/yes	yes/yes	yes/yes	All three → v2, 8

Perform both tests per version, then select the latest public version within the eligible set. v_old remains stored and eligible at the last query, but does not win. Selecting v2 globally then deleting it as unreceived at April 2 16:06 wrongly returns nothing and loses eligible v1. Release and receipt constrain external knowledge and local readability; neither proves a license.

8.2 One-Share Split and Dividend Ledger

Complete example: split and dividend ledger

All values are teaching constructions. Start with one old share at 100; a 2:1 split gives two new shares, each entitled to cash 1 and ending at 51. At the endpoint recognize the dividend receivable, ignoring payment delay, tax, reinvestment, foreign exchange (FX) effects and fees.

1. Starting wealth: 1 share $\times$ 100 currency/share = 100 currency.
2. New quantity: $1 \times 2 = 2$ shares, without wealth creation by the split.
3. Ending stock value: $2 \times 51 = 102$ currency units.
4. Receivable: $2 \times 1 = 2$.
5. Ending wealth: $102 + 2 = 104$.
6. Total return: $104/100 - 1 = 0.04 = 4\%$.

The old price becomes $100/2 = 50$ per new share. Thus $(51 + 1)/50 - 1 = 4\%$ total return, versus $51/50 - 1 = 2\%$ price return. $51/100 - 1 = -49\%$ mixes share units; 2% omits cash. Our specified entitlement belongs to the two shares held when rights are determined. Declaration, record and payment are not one event.

8.3 Connecting Records to Execution and Returns

Constructed record	Event/object	Visibility and use
3/15 filing	2023Q4 v1, released 16:05, ingested 16:12	Unusable 16:10; may inform a 16:15 research signal
D2 fill	D2 09:30, raw price 100; acknowledgement 09:30:01	Separate post-signal fill; returns start after execution, not period-end
4/2 restatement	v2 ingested 16:08	Later queries switch to 8; earlier signal logs remain
5/10 starting valuation	A separate account already holds one old share at 100	New wealth base for the ledger below, without adding the D2 order again
5/15 action	Prior constructed announcement; 2:1 split, cash entitlement 1/new share, endpoint 51	Stock 102, receivable 2, wealth 104; preserve separate rights/payment events
Later payment	Receivable 2 becomes cash 2	Receivable -2, cash $+2$; wealth does not increase again
6/28 exit	Constructed listing termination, terminal value still needs checking	Retain historical membership; no assumed continued trading; missing terminal value remains unknown

The D2 record explains the signal/fill/return interface; the May 10 ledger is a separate one-share account, not the same uninterrupted account. All prices and open states are assumed constructions, without evidence of real fills. Membership valid_from/valid_to describes economic intervals; available_at/ingested_at determines whether those membership records were known. Today's omission of SEC_ABC does not erase its past existence or prove historical investability on every date.

Keep accounting routes separate. The quantity/raw-price/cash ledger gives $2(51)+2=104$. The comparable-new-unit route gives $(51+1)/50-1=4\%$. These describe the same result; neither treats adjusted prices as fills nor adds dividends twice. A total-return index $100 \to 104$ already includes the dividend; adding another cash 2 falsely gives 106 and 6%.

Delisting counterexample. If subsequent -20% and -50% intervals apply only to stock, its compounded return is $(1-0.2)(1-0.5)-1=-60\%$. The second loss acts on the remaining 0.8. For stock 102 plus cash 2, with cash unchanged and unreinvested, ending wealth is $102(0.8)(0.5)+2=42.8$, returning about -58.846154% from 104. Unknown terminal values cannot be filled with -50%; these percentages are a separately specified constructed branch.

9. Python Implementation

Default Entry in the Accepted Reader

Follow the accepted I3 reader installation instructions, then run from the reader-package root:

```
.venv/bin/python run_notebook.py notebooks/ch03_data.ipynb
```

The default Notebook uses packaged shared logic without French files or downloads. New outputs use a unique runs directory; the reader CLI uses UUIDs and rejects overwrites. The original course-root code and historical tests below are a separate route. Optional French input requires the data contract and was not rerun. Constructed runs do not establish market or fill evidence.

The core function is in pit.py; the full example is `ch03_data.py` and its Notebook. Simulated fixtures are signals, versions and corporate actions; metadata explicitly identifies simulation.

The default interface is:

```
pit_asof_join(
    signals,
    versions,
    security_col="security_id",
    period_col="fiscal_period",
    signal_time_col="signal_time",
    available_at_col="available_at",
    ingested_at_col="ingested_at",
)
```

It matches permanent security ID and reporting period, preserves signal order/index and returns pit_effective_at. All three clocks must be timezone-aware. Both gates must precede or equal signal_time; eligible versions are ordered by available_at. Duplicate fact-key/available_at rows raise an explicit error. ingested_at is an eligibility gate, never a source-version ordering rule.

In the course root, the Python 3.12+ project environment runs `.venv/bin/python tools/run_a1.py --chapter 3`. It reads fixtures, converts time columns to UTC, calls the join and makes the figure. The constructed EPS results are no version at March 15 16:10, 10

at 16:15; still 10 at April 2 16:06 and 8 at 16:10. This demonstrates query contracts, not actual company disclosure.

Save the following standalone ledger as ch03_split_demo.py and run with Python 3.10+. It only demonstrates §8's constructed split/cash account, without duplicating PIT join. Outputs are currency and dimensionless return; no external data, random seed or NaN filling is used.

```
old_shares = 1
old_price = 100
split_ratio = 2
new_close = 51
cash_dividend_per_new_share = 1

new_shares = old_shares * split_ratio
ending_cash = new_shares * cash_dividend_per_new_share
ending_wealth = new_shares * new_close + ending_cash
total_return = ending_wealth / (old_shares * old_price) - 1

from math import isclose
assert ending_wealth == 104
assert isclose(total_return, 0.04, rel_tol=0, abs_tol=1e-12)
print(f"{ending_wealth} {total_return:.2f}")
```

Output:

```
104 0.04
```

Production account wealth also includes security value, cash, receivables and liabilities in a common base currency, with relevant FX timestamps retained.

A Self-contained Dual-clock Example

The following code handles a finite set of supplied versions with the same fact key, units and time zone. It does not replace the multi-security/period core API or its input checks. It tests five SEC_ABC signals, including late release, late receipt, restatement and a late-arriving older version.

```
import pandas as pd

# Historical versions of security SEC_ABC
versions_data = [
    {
        "version_id": "v1",
        "eps": 10.0,
        "available_at": pd.Timestamp("2024-03-15 16:05:00-04:00"),
        "ingested_at": pd.Timestamp("2024-03-15 16:12:00-04:00"),
    },
    {
        "version_id": "v2",
        "eps": 8.0,
        "available_at": pd.Timestamp("2024-04-02 16:05:00-04:00"),
        "ingested_at": pd.Timestamp("2024-04-02 16:08:00-04:00"),
    },
```

```
    {
        "version_id": "v_old",
        "eps": 9.5,
        "available_at": pd.Timestamp("2024-03-20 10:00:00-04:00"),
        "ingested_at": pd.Timestamp("2024-04-20 09:00:00-04:00"),
    },
]
df_versions = pd.DataFrame(versions_data)

def query_asof_eps(signal_time_str: str) -> float | None:
    s = pd.Timestamp(signal_time_str)
    if s.tzinfo is None:
        raise ValueError("timezone required")
    # Filter eligibility using both clocks
    eligible = df_versions[
        (df_versions["available_at"] <= s) & (df_versions["ingested_at"] <= s)
    ]
    if eligible.empty:
        return None
    if eligible["available_at"].duplicated().any():
        raise ValueError("ambiguous source time")
    # Select latest available_at among eligible versions
    latest_version = eligible.sort_values("available_at", ascending=False).iloc[0]
    return float(latest_version["eps"])

# Assert five representative cases
assert query_asof_eps("2024-03-15 16:10:00-04:00") is None  # Late ingestion
assert query_asof_eps("2024-03-15 16:15:00-04:00") == 10.0  # Initial version becomes
    ↪  eligible
assert query_asof_eps("2024-04-02 16:06:00-04:00") == 10.0  # Restatement not
    ↪ ingested
assert query_asof_eps("2024-04-02 16:10:00-04:00") == 8.0   # Restatement becomes
    ↪ eligible
assert query_asof_eps("2024-04-21 16:00:00-04:00") == 8.0   # Late older version does
    ↪  not overwrite restatement
```

10. Visualization

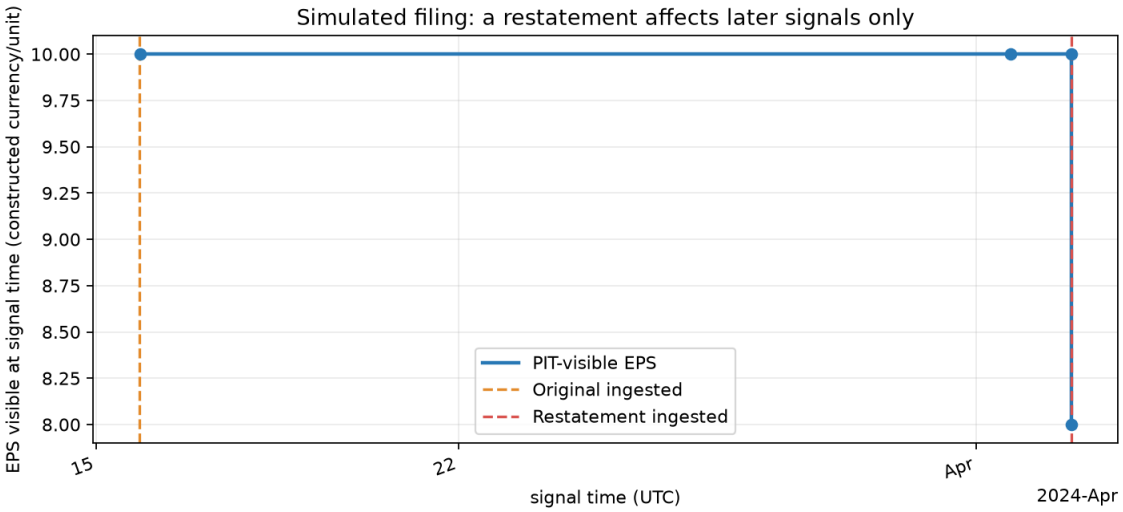


Figure 1: PIT version timeline

The horizontal axis is UTC signal time; the vertical axis is visible constructed EPS in currency per unit. The blue step shows 10 after initial ingestion, then 8 after restatement ingestion. Orange and red dashed vertical lines mark those two ingestion times. March 15 16:10 ET has no eligible EPS and no plotted value; April 2 16:06 retains 10 and 16:10 shows 8. All values are simulated fixtures.

There are no separate available_at lines and no price/return axis. The plot shows the two-gate version join, with UTC converted from local signals. It verifies visibility, without evidence of EPS predicting returns.

11. Official Interfaces and Calendars: Reading, Not a Market Experiment

The inherited reading of SEC EDGAR API documentation on 2026-10-05 covers data.sec.gov filing histories and XBRL facts. companyfacts aggregates issuer facts; frames selects the closest fact for a target calendar period using the most recently filed facts. The documentation describes real-time JSON updates following disclosures and bulk ZIP republication around 3 a.m. Eastern Time.

Today's companyfacts response is today's snapshot, not automatically a frozen earlier information set. A frames "most recently filed" selection cannot stand in for an initial report at a historical date. Preserve filing versions, issuer/concept contexts, release/acceptance, ingestion, retrieval time, raw payload, URL and hash, then apply the dual-clock query per signal. The documentation does not establish vendor availability or local ingestion for every fact.

submissions also provides filing history and current/historical names, venues and symbols. Use these as identity clues, then construct mappings with validity and system times rather than apply today's ticker to all history. A Central Index Key (CIK) identifies a filing entity, not one permanent tradable security; an issuer may have several securities or classes.

The inherited 2026 NYSE page specifies 9:30–16:00 Eastern core hours, annual holidays and some early closes after Thanksgiving and on Christmas Eve. It establishes venue/time-zone and special-session conventions, without historical tick data or proof that this chapter downloaded quotes or calendars.

Reading boundaries:

- Sources: SEC API documentation and NYSE Hours & Calendars; exact URLs appear in §20 and source_notes.md.
- Reading date: 2026-10-05. No company-fact or price sample was downloaded/extracted; no empirical frequency or sample period exists.
- Revision evidence: documented live JSON refresh and nightly ZIP mechanism; no historical API vintage was saved.
- Access: documentation describes access without authentication/API keys. Automated access still follows SEC Privacy and Security Policy; public access is not unlimited redistribution permission.
- Hashes: no data files exist for this example, hence no data hashes. Documentation pages are not market-sample snapshots.
- Experiment status: no market-return test, backtest or price comparison was run. Every EPS, split and dividend number above is explicitly constructed.

12. Common Mistakes and Repairs

12.1 Dividing Nominal Prices Across a Split

Incorrect:

```
old_price = 100
new_close = 51
naive_return = new_close / old_price - 1
print(naive_return)  # -0.49
```

−49% treats a share-unit change as loss. The account earns 4% with the two entitled shares and dividend 1 per share. For price return alone, convert the old price to 50 per new share and calculate $51/50 - 1 = 2\%$; total wealth also includes cash rights.

12.2 Treating Period-end as Release

> **Common mistake: period end is not release**
>
> Joining today's fact by fiscal_period_end/report_end_date to quarter-end market dates can introduce unreleased reports and future restatements. Filter each signal using both available_at and ingested_at. The constructed initial report released 16:05 and ingested 16:12 is absent at 16:10, visible as 10 at 16:15. The April 2 restatement remains unreceived at 16:06; only at 16:10 does 8 become eligible.

12.3 Labeling Local Time UTC

Wrong:

```
from datetime import datetime, timezone

wrong = datetime(2026, 7, 1, 9, 30, tzinfo=timezone.utc)
print(wrong)
```

This means 09:30 UTC rather than New York 09:30. In summer:

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo

local_open = datetime(
    2026, 7, 1, 9, 30, tzinfo=ZoneInfo("America/New_York")
)
print(local_open.astimezone(timezone.utc))
```

The result is 2026-07-01 13:30:00+00:00. Winter offsets differ, so use IANA rules. Repeated fall-back times need an offset or fold handling; "01:30" alone is ambiguous.

12.4 Forward-filling Every Missing Price

Incorrect:

```
closes = [10.0, None, 9.0]
filled = [10.0, 10.0, 9.0]
returns = [None, 0.0, -0.1]
print(returns)
```

This invents a 0% return on an unknown day and misplaces the intervening change. If the day is suspended, no new fill is possible then. A reopening at 9 after a known 10 gives $9/10 - 1 = -10\%$ over the Monday-to-Wednesday span, not two observed daily returns. Retain original missingness, reason, preceding valid timestamp and session span. Daily holding valuation needs a separate suspension valuation/risk-discount policy; generic forward fill does not define one.

12.5 Retaining Only Today's Symbols

A current universe applied ten years backward selects on future survival. Bankruptcy, acquisition and symbol changes disappear or misjoin. Use permanent mappings with valid_from/valid_to and historical membership snapshots, retaining both effective and received times. Check liquidation cash, exchange consideration, last quotes and delisting definitions. Unknown terminal values need investigation and defensible bounds, not zero or silent deletion.

12.6 Using "Adjusted" Without a Definition

Inspect whether factors include splits, regular dividends, special distributions and which effective dates. Adding D to a total-return-adjusted close double counts; treating a split-adjusted quote as an actual spot fill changes the object. Retain raw and adjusted values, source, factors, action types and method versions, and use them in their appropriate layers.

12.7 Treating Latest APIs as Version Archives

Current responses may update. A URL alone cannot reproduce the bytes weeks later. Preserve original JSON/files, UTC retrieval, permitted HTTP metadata, parser version, hashes and accessions. Treat each response as a new snapshot rather than overwrite the only copy.

13. Auditable Inputs for an Event Strategy

Consider the hypothesis that a larger initial financial surprise relative to expectations predicts higher next-session risk-adjusted returns. This is a design example, not an established result.

First define units: permanent security, issuer, concept, units, reporting coverage and initial filing version. Security IDs link trades/quotes; issuer IDs group company facts. Specify FX source/time and standardize differing quarter lengths or reporting conventions before comparison.

Reconstruct the information set per signal by selecting the latest version passing both clocks. Preserve initial and corrected records; do not overwrite initial EPS with the current value. Report unreceived, missing, conflicting and unsynchronized cases, including unmatched proportions and exclusions.

Rebuild the historical universe from then-known listing and tradability records. Keep subsequently delisted securities before exit. Link symbol changes by valid identity intervals; a merger does not automatically make old and new shares interchangeable.

Define execution and returns next. After-close announcements received later imply signal_time no earlier than receipt, with trade_time at a subsequent executable session. Orders/quotes must support any next-open fill assumption. Returns start after actual fills and end at a prespecified exit/valuation. Record split quantities, dividends and delisting proceeds. Deduct commissions, spreads, impact, financing and borrow costs from gross PnL for net PnL.

In a common base currency:

$$NAV_t = \sum_i N_{i,t} P_{i,t}^{(\text{base})} + \text{Cash}_t + \text{Receivables}_t - \text{Liabilities}_t,$$

$$R_{t_0,t_1}^{(\text{net})} = \frac{NAV_{t_1}}{NAV_{t_0}} - 1.$$

N is held quantity and $P^{(\text{base})}$ is per-share base-currency valuation. The ratio requires positive initial NAV, no external deposits/withdrawals, and all investment flows/costs in the ledger. With external flows, use Chapter 11's segmented measurement rather than count deposits as profit. Derivatives require their contract-specific valuation in place of spot-share value.

Compare no-signal holdings in the same universe, market/industry exposures and simple event windows. Report year, liquidity, venue and delisting strata, counts, missing coverage and cost model. An advantage relying on revised facts, surviving stocks, same-close fills or zero costs fails the executable design. If initial availability cannot be reconstructed, bound the conclusion to inability of the current dataset to identify historical executable performance.

14. Chapter Summary

A financial observation needs identity, units, venue clocks, release clocks and version. OHLCV summarizes trading activity without specifying adjustments, cash rights or tradability. Splits change quantities/units; dividends enter as rights; price and total returns follow these definitions.

Before calculating historical returns, determine whether a version was both released and ingested by signal_time and whether the security belonged to the contemporary universe. Use IANA zones and actual calendars, classify missingness, and retain delisting terminal values or explicit unknowns. Deliver code, snapshots, hashes and licenses with numbers. Bound conclusions wherever reconstruction is incomplete.

15. Concepts and Hand Calculations

Answers are in §18.

3-E01. What does each of event_time, available_at, ingested_at and revision_time answer? Why cannot period_end replace available_at?

3-E02. Which questions use raw, split-adjusted and total-return series? Give a risk in using back-adjusted levels for historical signals.

3-E03. A security has no close. Give at least four causes and explain why return cannot be zero before diagnosis.

3-E04. Distinguish ticker, permanent security ID and issuer ID. What selection does today's universe impose on historical research?

3-E05. One old share starts at 100, splits 2:1, with dividend 1/new share and ending price 51. Under the endpoint-receivable assumption and no taxes/fees, calculate stock value, cash entitlement, wealth, total return and price return; label the holding interval.

3-E06. Initial EPS 10 is released March 15 at 16:05 and ingested 16:12; EPS 8 restatement is released April 2 at 16:05 and ingested 16:08. Determine visibility at March 15 16:10/16:15 and April 2 16:06/16:10, using matching New York dates/zones.

3-E07. A holding returns −20% then −50% in a delisting phase, sequentially in comparable units. Calculate combined return and explain why addition fails.

3-N01 — Identify the Mechanism.

1. Distinguish late announcement from late ingestion and explain how each invalidates a backtest.
2. Why must eligibility use available_at≤s AND ingested_at≤s rather than OR?
3. Why cannot an already included dividend be added again? Can a split factor and cash simply be added?

3-N02 — Stepwise Version Selection. PERM_001 has A: March 15 16:05 release, 16:12 ingestion, value 10; B: April 2 16:05 release, 16:08 ingestion, value 8; C: March 20 10:00 release, April 20 09:00 ingestion, value 9.5. At April 21 16:00, identify each eligible version and the latest visible version. Why not latest-ingested C?

3-N03 — Full Ledger. Reproduce §8.2's quantities, stock value, receivable, wealth, price and total return. Then pay the receivable into cash 2; stock alone returns −20% then −50%, with unchanged unreinvested cash, no taxes/fees or external flows. Calculate stock compounded return, final stock value, account wealth, subsequent return from 104 and whole-path return from 100. Can a constructed −50% replace unknown real terminal value?

16. Programming Exercises

3-P01 — Dual-clock Query. Read the simulated signal/version fixtures, convert clocks to timezone-aware UTC and join by permanent security_id/fiscal_period through §9's API. Apply both gates, select latest available_at and preserve signal order/IDs. Assert pre_release missing, post_ingest 10, restatement_available_only 10, after_restatement_ingest 8 and other_security 77. Reject conflicting duplicate fact-key/available_at rows.

3-P02 — Missingness and Intervals. D1 close 10, suspended D2 missing, reopened D3 close 9. Return D2 missing with reason=suspended and D3 about −10% across two trading sessions. Do not invent D2 0% or label the two-session change daily; compare floats with tolerance.

3-P03 — OHLCV and Local Time. Write a standard-library row checker for finite real numbers excluding bool, parseable offset-aware time, low/high ordering, OHLC range, nonnegative volume and venue-local session_date. July 1, 2026 09:30 −04:00 converts to 13:30 UTC. Explicitly reject volume −1 and low>high; preserve bad inputs.

3-N04 — Diagnose a Join.

```
# As-of join code to diagnose
import pandas as pd

def bad_asof_select(versions_df, signal_time):
    # Error 1: ordering by ingestion to select newest version
    sorted_df = versions_df.sort_values('ingested_at', ascending=False)
    # Error 2: selecting first row using only ingestion eligibility
    valid = sorted_df[sorted_df['ingested_at'] <= signal_time]
    return valid.iloc[0]['eps']
```

Explain its late-arriving older-version defect. Supply a self-contained correction and assert that an older late receipt cannot overwrite a restatement.

17. Challenge Problems

3-C01 — Late and Out-of-order Data. A vendor sends earlier public A, later public B, and delayed batches whose ingestion order differs. Design a reproducible schema/query for latest versions both released and received by each signal. Discuss same-second boundaries, withdrawals, duplicates and historical backfills. Ten points: timing semantics 3, version retention 2, query correctness 3, exception handling 2.

3-C02 — Delisting Audit. A monthly strategy uses today's constituents over ten historical years and deletes securities without complete month-end prices. Give the shortest executable audit plan for data, mapping and falsification. Explain reporting when delisting return is absent. Ten points: PIT universe 2, permanent IDs/symbols 2, missingness/exit 2, realizable returns/costs 2, bounded conclusions 2.

3-N05 — Research Judgment. A constructed vendor contract says its total-return index already reinvests dividends; the researcher nevertheless adds the same dividend to portfolio wealth. The universe uses current membership and deletes holdings without month-end quotes. Separate the confirmed accounting error, possible selection and required evidence. Can you quantify total bias or real net return? Do not invent a vendor or loss value.

18. Complete Solutions

3-E01. event_time describes the event; available_at the specified external-source/vendor availability; ingested_at local receipt and persistence; revision_time revision release. period_end describes coverage. Earnings for a quarter ending March 31 may appear weeks later. Signals need both available_at$\leq$signal_time and ingested_at$\leq$signal_time.

3-E02. Raw prices represent contemporaneous quotes and execution checks. Split adjustment gives comparable share units; total return includes distributions. Back-adjustment can rewrite levels using later actions, making price-level signals depend on unknown factors. Even return calculations require revision and double-count checks.

3-E03. Causes include venue closure, suspension, no trade, vendor omission, delisting, inapplicable field and parsing failure. Zero return asserts an observed unchanged value; missing asserts no trustworthy price. Confusing them invents low volatility and trading opportunities. Retain statuses and define valuation/intervals separately.

3-E04. A ticker is venue/time-specific and may change or be reused. A permanent security ID represents a continuous tradable claim; issuer ID represents a company/filing entity with potentially several classes. Today's list conditions history on later survival and membership, excluding prior constituents and delisted securities.

3-N01. Late announcement delays external disclosure after the economic period; using period_end introduces not-yet-released information. Late ingestion delays local receipt after public release through networks, queues or batches; treating release as receipt assumes inputs live systems could not read. Both external availability under the contract and persistence must hold; neither proves rights. Splits change share units; cash distributions add cash. A total-return series that has already compounded the distribution cannot receive the same cash again. A dimensionless split factor and currency amount cannot be added.

3-E05. $W_0 = 100$; two shares end at $2(51) = 102$, plus receivable $2(1) = 2$, so $W_1 = 104$ and total return 4%. Comparable starting price is $100/2 = 50$, giving price return 2%. The interval runs from the initial old-share valuation to the post-split close and entitlement recognition. The example recognizes a receivable, not actual payment, belonging to both new shares.

3-E06.

Signal	Initial 10	Restated 8	Result
3/15 16:10	Released, unreceived	Unreleased	Missing unless an earlier eligible version exists
3/15 16:15	Released/received	Unreleased	10
4/2 16:06	Released/received	Released, unreceived	10
4/2 16:10	Released/received	Released/received	8

No earlier version was supplied, so the first row cannot be filled.

3-E07. Start at 1; after −20%, wealth is 0.8; after −50%, it is 0.4. Combined return is −60%. The −70% arithmetic sum wrongly applies the second loss to initial wealth.

3-N02. At April 21 16:00, A, B and C all satisfy both clocks, so the eligible set is {A,B,C}. B wins with EPS8 because April 2 release follows C's March 20 release. C's latest receipt is pipeline delay, not the newest source fact.

3-N03. Two shares give stock 102 and receivable 2, wealth 104. Starting price in new units 50 gives price return 2% and total return 4%. Payment reclassifies receivable to cash without adding wealth. Stock returns −60% and ends at $102(0.8)(0.5) = 40.8$. Add unchanged cash 2 for wealth 42.8. Subsequent account return is $42.8/104 - 1 \approx -58.846154\%$; whole-path return $42.8/100 - 1 = -57.2\%$. $104(0.4) = 41.6$ wrongly loses cash too. Unknown terminal values remain unknown and need investigation, not the constructed −50%.

3-P01 — Reference Implementation. Run from the course root; it calls the shared API and checks signal IDs and the original index.

```
from pathlib import Path
import sys

import pandas as pd

ROOT = Path.cwd()
if not (ROOT / "data" / "fixtures" / "a1_pit_signals.csv").exists():
    raise RuntimeError("run this example from the quant_course root")

sys.path.insert(0, str(ROOT / "src"))
from quant_course import pit_asof_join

signals = pd.read_csv(ROOT / "data" / "fixtures" / "a1_pit_signals.csv")
versions = pd.read_csv(ROOT / "data" / "fixtures" / "a1_pit_versions.csv")
signals["signal_time"] = pd.to_datetime(signals["signal_time"], utc=True)
for column in ("available_at", "ingested_at"):
    versions[column] = pd.to_datetime(versions[column], utc=True)

joined = pit_asof_join(signals, versions)
by_signal = joined.set_index("signal_id")

assert pd.isna(by_signal.loc["pre_release", "eps"])
assert by_signal.loc["post_ingest", "eps"] == 10
assert by_signal.loc["before_restatement", "eps"] == 10
assert by_signal.loc["restatement_available_only", "eps"] == 10
assert by_signal.loc["after_restatement_ingest", "eps"] == 8
assert by_signal.loc["other_security", "eps"] == 77
assert joined.index.equals(signals.index)
assert joined["security_id"].equals(signals["security_id"])

print(joined[["signal_id", "version_id", "eps"]].to_string(index=False))
```

Inherited actual output:

```
                signal_id      version_id  eps
             pre_release             NaN  NaN
             post_ingest     original_10 10.0
       before_restatement     original_10 10.0
restatement_available_only     original_10 10.0
  after_restatement_ingest      restated_8  8.0
           other_security other_issuer_77 77.0
```

This matches the recorded direct fixture call of 2026-10-05, without a fresh claim of execution in this translation batch. Duplicate key/available_at rows raise ValueError rather than a silent row-order winner.

3-P02 — Reference Implementation.

```
from math import isclose

observations = [
    {"date": "D1", "close": 10.0, "status": "observed"},
    {"date": "D2", "close": None, "status": "suspended"},
    {"date": "D3", "close": 9.0, "status": "observed"},
]

assert observations[1]["close"] is None
assert observations[1]["status"] == "suspended"
interval_return = observations[2]["close"] / observations[0]["close"] - 1
sessions_spanned = 2
assert isclose(interval_return, -0.1, rel_tol=0, abs_tol=1e-12)
assert sessions_spanned == 2
print(interval_return, sessions_spanned)
```

Output:

```
-0.09999999999999998 2
```

D2 remains unknown, reason=suspended; D3 spans D1 close to D3 close over two sessions and returns about -10%. Define which decision owns that label. Missing intermediate prices cannot establish two daily returns.

3-P03 — Reference Implementation. This Python 3.10+ standard-library program retains both valid and bad inputs without rewriting them.

```
from datetime import datetime
from zoneinfo import ZoneInfo
from math import isfinite
from numbers import Real

def validate_bar(row, exchange_tz):
    errors = []
    for name in ("open", "high", "low", "close", "volume"):
        value = row.get(name)
        if value is None:
            errors.append(f"{name}: missing")
        elif isinstance(value, bool) or not isinstance(value, Real) or not isfinite(
    ↪ value):
            errors.append(f"{name}: finite real number required")
    try:
        stamp = datetime.fromisoformat(row["event_time"])
    except (KeyError, TypeError, ValueError):
        return errors + ["event_time: invalid ISO timestamp"]
    if stamp.tzinfo is None:
        errors.append("event_time: timezone offset required")
    values = [row.get(k) for k in ("open", "high", "low", "close")]
    if all(isinstance(v, Real) and not isinstance(v, bool) and isfinite(v) for v in
    ↪ values):
        opn, high, low, close = values
        if low > high:
            errors.append("low exceeds high")
```

```
        if not (low <= min(opn, close) <= max(opn, close) <= high):
            errors.append("OHLC ordering invalid")
    volume = row.get("volume")
    if (isinstance(volume, Real) and not isinstance(volume, bool)
            and isfinite(volume) and volume < 0):
        errors.append("volume is negative")
    if stamp.tzinfo is not None:
        local = stamp.astimezone(ZoneInfo(exchange_tz))
        if local.date().isoformat() != row.get("session_date"):
            errors.append("session_date disagrees with local event date")
    return errors

good = {
    "event_time": "2026-07-01T09:30:00-04:00",
    "session_date": "2026-07-01",
    "open": 50.0, "high": 51.0, "low": 49.0,
    "close": 50.5, "volume": 100,
}
bad = {
    "event_time": "2026-07-01T09:30:00-04:00",
    "session_date": "2026-07-01",
    "open": 50.0, "high": 49.0, "low": 51.0,
    "close": 50.5, "volume": -1,
}

assert validate_bar(good, "America/New_York") == []
errors = validate_bar(bad, "America/New_York")
assert "low exceeds high" in errors
assert "volume is negative" in errors
for invalid in (float("nan"), float("inf"), True, "100"):
    assert "volume: finite real number required" in validate_bar(
        dict(good, volume=invalid), "America/New_York")
assert "event_time: invalid ISO timestamp" in validate_bar(
    dict(good, event_time="not a timestamp"), "America/New_York")
utc_time = datetime.fromisoformat(good["event_time"]).astimezone(ZoneInfo("UTC"))
assert utc_time.isoformat() == "2026-07-01T13:30:00+00:00"
print(utc_time.isoformat(), errors)
```

Output:

```
2026-07-01T13:30:00+00:00 ['low exceeds high', 'OHLC ordering invalid', 'volume is
    ↪ negative']
```

It rejects missing, nonfinite, mistyped or unparseable values and checks internal relationships. Asset classes need their own rules for negative prices. Offset awareness does not prove a valid session or trade; production checks still need precise calendars and quote/auction provenance.

3-N04. Sorting by ingestion lets a late March 20 version displace an already effective April 2 restatement. Ignoring external availability can also admit an embargoed version received before release. Correct both gates and order only eligible versions by available_at:

```
import pandas as pd

def correct_asof_select(versions_df: pd.DataFrame, signal_time: str) -> float | None:
    s = pd.Timestamp(signal_time)
    if s.tzinfo is None:
        raise ValueError("timezone required")
    # Both clocks must satisfy eligibility
    eligible = versions_df[
        (versions_df["available_at"] <= s) & (versions_df["ingested_at"] <= s)
    ]
    if eligible.empty:
        return None
    if eligible["available_at"].duplicated().any():
        raise ValueError("ambiguous source time")
    # Select latest available_at among eligible versions
    winning = eligible.sort_values("available_at", ascending=False).iloc[0]
    return float(winning["eps"])

# Constructed test data
versions_df = pd.DataFrame([
    {
        "version_id": "v1", "eps": 10.0,
        "available_at": pd.Timestamp("2024-03-15 16:05:00-04:00"),
        "ingested_at": pd.Timestamp("2024-03-15 16:12:00-04:00"),
    },
    {
        "version_id": "v2", "eps": 8.0,
        "available_at": pd.Timestamp("2024-04-02 16:05:00-04:00"),
        "ingested_at": pd.Timestamp("2024-04-02 16:08:00-04:00"),
    },
    {
        "version_id": "v_old", "eps": 9.5,
        "available_at": pd.Timestamp("2024-03-20 10:00:00-04:00"),
        "ingested_at": pd.Timestamp("2024-04-20 09:00:00-04:00"),
    },
])

assert correct_asof_select(versions_df, "2024-03-15 16:10:00-04:00") is None
assert correct_asof_select(versions_df, "2024-03-15 16:15:00-04:00") == 10.0
assert correct_asof_select(versions_df, "2024-04-02 16:10:00-04:00") == 8.0
assert correct_asof_select(versions_df, "2024-04-21 16:00:00-04:00") == 8.0  # Late
    ↪ older version does not overwrite
```

3-C01 — Reference Argument and Scoring. Preserve stable fact key, security, period, concept/unit, valid interval, version ID, clocks, source, raw hash, parser and withdrawal/supersession relations. Per signal, first filter the matching facts by both gates, then select by a defined source-version order. Selecting global maximum available_at before deleting late arrivals can lose an earlier eligible version.

If same-second order is unknown, exclude or use finer time/monotonic sequences. Deduplicate source/version/hash while retaining receipt events. Add withdrawal events with availability instead of erasing history. Backfills affect later signals unless evidence proves earlier visibility and possession. Award 3 timing, 2 retained versions, 3 per-signal correct reconstruction, 2 boundaries/withdrawals/duplicates/backfills.

3-C02 — Reference Argument and Scoring. Obtain membership and tradability at each historical rebalance, with validity and visibility clocks. Map permanent IDs to time-bounded symbols, distinguishing issuers, common/preferred shares, ADRs and venues. Keep later delistings before exit; investigate acquisition consideration, bankruptcy distributions, delisting returns and cessation dates. Classify closures, suspensions, outages, vendor omissions and exits, with coverage and valuation/exclusion rules. Use executable holding intervals, reproducible costs and survivor-versus-full-PIT comparisons. Unknown terminal values require reported unknown proportions and defensible bounds or censoring sensitivity, not silent zero/deletion. If key exits remain unknown, limit results to identifiable-terminal-value subsamples and state that selection may change direction. Score the five two-point categories.

3-N05. The stated contract confirms double counting of the same dividend. Reconcile through either raw-price/quantity/cash accounting or a return route already including dividends. Today's membership cannot reconstruct history; absent quotes do not erase holdings/rights. Add historical membership/visibility, permanent mappings, suspension/exit reasons, terminal proceeds, action versions and fill costs. These descriptions cannot quantify overall bias or establish losses for every omitted security. Compare affected samples and benchmarks after repair. Ten points: double count 3, universe/holdings 3, evidence 2, bounded conclusion 2.

19. Further Reading

- **Must know — SEC EDGAR API:** API, companyfacts, frames and Update Schedule explain endpoints, aggregation and refresh. They do not replace vendor/local receipt evidence.
- **Must know — NYSE Hours & Calendars:** yearly holidays and Trading Hours bind sessions to Eastern Time and identify early closes.
- **Should know — Python zoneinfo:** IANA sources, constructors and fold address conversions and repeated times. Fix time-zone database dependencies for cross-platform deployment.
- **Should know — pandas.merge_asof:** direction, sorting, tolerance and allow_exact_matches explain the time-neighbor primitive. It does not manage filing versions or constrain ingestion automatically.
- **Optional — French Data Library:** browse the directory and Historical Archives. Future factor use requires its own authorized snapshot, version, date and construction checks. This chapter downloaded no French data and makes no empirical claim from it.

20. References and Reading Boundaries

Keys are registered in shared references/sources.json and references.bib. Exact inherited reading scope and dates are in Chinese source audit record. This translation adds no new online reading or market run.

- **SEC_API:** EDGAR Application Programming Interfaces. Endpoints, companyfacts/frames, refresh, bulk data and access; no proof of vendor/local clocks.
- **NYSE_CAL:** Holidays & Trading Hours. Eastern hours, holidays and early closes; no historical quote sample.
- **PY_ZONEINFO:** zoneinfo — IANA time zone support. Database sources and daylight-saving ambiguities.
- **PANDAS_ASOF:** `pandas.merge_asof`. Sorted backward joins, grouping and tolerance; no inferred fact-validity or ingestion semantics.
- **FRENCH_LIBRARY:** Kenneth R. French Data Library. Optional directory entry, without chapter-level revision or empirical claims.

Chapter 4: Calculus, Rates of Change, Curvature and Local Decisions

Spine: use the response of a price to a yield change and the curvature of a portfolio objective to make a small change in inputs something you can calculate and check. Prerequisites: Chapter 2's Python and arrays. Section 6.3 develops column vectors, transposes and a two-by-two product before Chapter 5 collects the linear algebra. You will derive a constructed bond's first and second derivatives, read their units, and compare analytic and numerical derivatives of a quadratic objective.

1. Learning Objectives

Work from a one-variable rate of change to a local approximation with several inputs. You should be able to:

1. Define a derivative by a limit of difference quotients and distinguish continuity, differentiability and twice continuous differentiability, written C^2.
2. Read a derivative's units from its input and output.
3. Distinguish a partial derivative, total differential, gradient, directional derivative and Hessian matrix.
4. State the smoothness and remainder conditions behind first- and second-order Taylor approximations.
5. Accumulate marginal changes using the fundamental theorem of calculus.
6. Classify an interior stationary point as a minimum, maximum or saddle when the second-order test applies, and recognize when it is inconclusive.
7. Compare central differences with analytic gradients and Hessians, including the effects of step size.

The two constructed cases give concrete checks. The bond has

$$P \approx 90.70295, \qquad P' \approx -172.76752, \qquad P'' \approx 493.62148.$$

The portfolio case gives objective -0.0452, gradient $(-0.022, -0.038)^\top$ and Hessian Σ. Their inputs are teaching assumptions, not market observations.

2. Why Quantitative Research Needs Calculus

A yield rises by one basis point: how much does the formula price change? A feature changes: which way does a model score move? A portfolio weight changes: does its risk objective increase? Calculus separates the slope, the chosen direction and the curvature in each question.

A slope needs a shock size before it becomes a price change. In our bond case, -172.77 is the response per unit of decimal yield; multiply by 0.0001 for one basis point. An unconstrained weight update needs a separate budget check. A lower objective does not establish a fully invested portfolio.

The gradient describes the local response; the Hessian describes how that response changes. A definite integral accumulates marginal responses along a path. These calculations describe the chosen mathematical model. Evidence for prediction, execution and costs must come from a research design that also addresses those questions.

3. Intuition: Slope and Curvature

Read $P(y)$ as the output of the pricing rule P at yield y. Changing the input can change the output while leaving the rule fixed. For $f(x) = x^2$, input $x = 3$ gives $f(3) = 9$; x is the independent variable and $f(x)$ the dependent variable.

On a price-yield graph, put yield on the horizontal axis and price on the vertical axis. A tangent line at the current yield has slope P'. If the curve bends upward, equal yield shocks at different locations need not give equal price changes. The second derivative records that curvature.

For $f(w_1, w_2)$, think of a surface. The gradient points toward the steepest local ascent in Euclidean coordinates; among Euclidean unit directions, the negative-gradient direction gives the steepest instantaneous decrease per unit distance. A bowl has positive curvature in every direction; a saddle bends upward in one direction and downward in another. This picture says nothing by itself about global convexity, position limits, leverage or feasibility of a finite update.

4. Mathematical Foundations: Increments, Units and Shapes

4.1 Variables, Function Inputs and Powers

The parentheses in $f(x)$ specify an input, not multiplication of f by x. In $g(x, z)$, two inputs produce one output. A partial derivative changes one input while holding the others fixed.

A power means repeated multiplication: $(1 + y)^2 = (1 + y)(1 + y)$. A negative power gives a reciprocal: $a^{-2} = 1/a^2$, for $a \neq 0$. Thus $100/(1 + y)^2 = 100(1 + y)^{-2}$. For the constructed annual-compounding bond we require $y > -1$, so the discounting base is positive.

4.2 Input Increments and Difference Quotients

Change x to $x + h$. The signed input increment is h, and the output increment is

$$\Delta f = f(x + h) - f(x).$$

For $h \neq 0$, the average rate of change is

$$\frac{f(x + h) - f(x)}{h} = \frac{\Delta f}{h}.$$

Its units are output units divided by input units. For $f(x) = x^2$, moving from 2 to 2.1 gives $h = .1$, outputs 4 and 4.41, increment .41 and average rate 4.1. With $h = .01$, the rate is 4.01. As nonzero h approaches zero, the rate approaches 4. The quotient remains undefined at $h = 0$.

Differentiability lets us separate a linear part from a smaller remainder:

$$f(x+h) - f(x) = f'(x)h + o(|h|).$$

The notation $o(|h|)$ means that the remainder divided by $|h|$ tends to zero. It does not guarantee a quadratic error rate. A stronger error bound requires stronger smoothness assumptions.

4.3 Units and Dimensions

A derivative has units

$$[\text{derivative}] = [\text{output}]/[\text{input}].$$

Price uses currency units; yield is a dimensionless decimal. Accordingly, $P'(y)$ is price units per unit of decimal yield. A change of .01 is one percentage point, or 100 basis points (bp). Since one bp is one hundredth of a percentage point,

$$1 \text{ bp} = .0001, \quad 25 \text{ bp} = .0025, \quad 100 \text{ bp} = .01.$$

A derivative reported per decimal yield unit is not a price change per percentage point or per bp.

With inputs $w \in \mathbb{R}^n$, gradient entries can have different units if the coordinates do. Hessian entry H_{ij} has function units divided by the units of inputs i and j. Coordinate scaling matters when comparing gradient magnitudes or choosing a numerical step.

For $f : \mathbb{R}^n \to \mathbb{R}$, use column inputs and a column gradient, both $n \times 1$; the Hessian is $n \times n$. Then $\nabla f^\top \Delta w$ and $\Delta w^\top H \Delta w$ are scalars. Check the convention before adding transposes merely to make a program run.

Input conditions: a fixed-maturity construction

Our bond pays a single face value of 100 at the end of year two, uses annual compounding, and measures cash flow and price in the same currency. It is a closed-form teaching case.

5. Definitions

A limit describes nearby function values. If for every $\varepsilon > 0$ there is a $\delta > 0$ such that $0 < |h| < \delta$ implies $|f(x+h) - L| < \varepsilon$, then $\lim_{h\to 0} f(x+h) = L$.

Definition: continuity and differentiability

Continuity requires the function to be defined at the point and its limit to equal its value. Differentiability requires the finite limit

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}.$$

Differentiability implies continuity; continuity alone is insufficient. At zero, $|x|$ is continuous, but its left and right slopes are -1 and 1. A continuously differentiable function is C^1; continuous second partial derivatives give the usual C^2 condition.

A partial derivative $\partial f/\partial w_i$ changes coordinate i, holding the others fixed. The gradient collects the partial derivatives:

$$\nabla f(w) = \begin{bmatrix} \partial f/\partial w_1 \\ \vdots \\ \partial f/\partial w_n \end{bmatrix}.$$

At a differentiable point, the total differential is the linear part:

$$df = \nabla f(w)^\top dw, \quad f(w+\Delta w) - f(w) = \nabla f(w)^\top \Delta w + o(\|\Delta w\|).$$

It approximates a small increment; it need not equal an arbitrary finite change.

For a unit direction u, the directional derivative is

$$D_u f(w) = \lim_{h\to 0} \frac{f(w+hu) - f(w)}{h} = \nabla f(w)^\top u$$

when f is differentiable. A direction that is not normalized also scales the rate by its length. Normalize directions before comparing rates per unit distance.

The Hessian collects second partial derivatives:

$$\nabla^2 f(w) = \left[\frac{\partial^2 f(w)}{\partial w_i \partial w_j}\right]_{i,j=1}^{n}.$$

Under C^2 smoothness in a neighborhood, mixed partials agree and the Hessian is symmetric. Its curvature along u is $u^\top \nabla^2 f(w) u$.

A second-order Taylor expansion is

$$f(w+h) = f(w) + \nabla f(w)^\top h + \tfrac{1}{2} h^\top \nabla^2 f(w) h + R_3(h).$$

For C^2 functions, $R_3(h) = o(\|h\|^2)$. If you retain only the linear term, you omit the quadratic term as well: the resulting error is generally $O(\|h\|^2)$, not necessarily $o(\|h\|^2)$. For C^3 functions with the appropriate bounded third derivatives along the segment,

$$R_3(h) = \tfrac{1}{6} D^3 f(w+\theta h)[h,h,h], \qquad 0 < \theta < 1.$$

Here $D^3 f(z)[h,h,h]$ is the third directional derivative at z. If its magnitude is bounded by $M\|h\|^3$, then $|R_3(h)| \le M\|h\|^3/6$. For a quadratic polynomial this remainder is zero.

An error $O(|h|^3)$ is bounded by $C|h|^3$ for a fixed constant in a sufficiently small neighborhood. It need not equal h^3. Curvature usually gives a quadratic error after a tangent approximation; a third-order bound after a quadratic approximation needs the stated conditions. Compare finite-shock approximations with exact values using a tolerance chosen before inspecting the results.

For continuous g, a definite integral is the limit of Riemann sums. With $\Delta x = (b-a)/n$ and a point ξ_k in each subinterval,

$$\int_a^b g(x)\,dx = \lim_{n\to\infty} \sum_{k=1}^{n} g(\xi_k)\Delta x.$$

If $F' = g$ and g is continuous, the fundamental theorem of calculus gives

$$\int_a^b g(x)\,dx = F(b) - F(a).$$

Worked example: midpoint accumulation

Integrating a marginal price response per input unit over that input returns price units. For $\int_0^1 x^2 dx = 1/3$, two midpoint intervals give $.5(.25^2 + .75^2) = .3125$, short by .0208333. Ten give .3325, short by .0008333. The finite sum approximates the integral; convergence requires the relevant integrability and grid refinement conditions.

A differentiable local extremum in an open region has zero gradient. At a stationary point of a locally C^2 function, a positive definite Hessian gives a strict local minimum, a negative definite Hessian a strict local maximum, and an indefinite Hessian a saddle. A semidefinite Hessian is generally inconclusive. At the origin, x^4 is a minimum, $-x^4$ a maximum, and $x^2 - y^4$ a saddle, despite their zero gradients and semidefinite Hessians. Inspect higher-order terms or changes along different paths. A constrained boundary extremum need not have zero full-space gradient.

For differentiable functions and fixed constants,

$$(af + bg)' = af' + bg', \quad (fg)' = f'g + fg', \quad (f/g)' = (f'g - fg')/g^2 \quad (g \neq 0).$$

A constant has derivative zero; the power rule $(x^m)' = mx^{m-1}$ holds where that power is defined and differentiable. Composite functions also need the chain rule. The derivative of a product is not the product of the derivatives.

To derive the product rule, add and subtract $f(x+h)g(x)$ in the numerator. It becomes $f(x+h)[g(x+h) - g(x)] + g(x)[f(x+h) - f(x)]$. Divide by nonzero h, use continuity implied by differentiability and take the limit to obtain $fg' + gf'$. Combine this with the derivative of g^{-1} for the quotient rule. For x^2, the difference quotient is $2x + h$, tending to $2x$.

Existing partial derivatives need not imply multivariable differentiability. Define $g(x, y) = xy/\sqrt{x^2 + y^2}$ away from zero and $g(0, 0) = 0$. Both partial derivatives at zero vanish along the coordinate axes, but $g(t, t) = |t|/\sqrt{2}$. Dividing this error from the proposed zero linear part by the distance $\sqrt{2}|t|$ gives $1/2$, not zero. Continuous partial derivatives provide a common sufficient condition; merely calculating two partials does not establish a total differential.

6. Derivations: From a Bond Curve to a Portfolio Surface

6.1 Bond Derivatives and Yield Units

For our constructed bond,

$$P(y) = 100(1 + y)^{-2}, \qquad y > -1.$$

The exponent reflects two annual discount periods. Write the outer rule as $100u^{-2}$ and the inner rule as $u = 1 + y$. The power rule and chain rule give

$$P'(y) = 100(-2)(1 + y)^{-3} \frac{d(1 + y)}{dy} = -200(1 + y)^{-3}.$$

The exponent moves into the coefficient and decreases by one; the inner derivative is 1.

Check directly from the definition. For $a = 1 + y > 0$,

$$\frac{P(y + h) - P(y)}{h} = \frac{100[a^2 - (a + h)^2]}{h(a + h)^2 a^2} = -\frac{100(2a + h)}{(a + h)^2 a^2} \longrightarrow -\frac{200}{a^3}.$$

Cancel the nonzero h before taking the limit. You have not divided by zero. Differentiating again gives

$$P''(y) = (-200)(-3)(1+y)^{-4} = 600(1+y)^{-4}.$$

Throughout $y > -1$, the first derivative is negative and the second positive. A higher yield lowers this formula price, and the response changes with position. Those are properties of the formula rather than measurements of a market yield curve.

6.2 Path Derivatives and Directional Derivatives

For a specified yield path $y(t)$,

$$\frac{dP}{dt} = P'(y(t))\frac{dy}{dt}.$$

If t uses years, multiplying price per decimal yield unit by decimal yield per year gives price per year. This path fixes remaining maturity at two years. Treat t as a scenario parameter, not the full calendar-time evolution of one bond.

For a bond whose remaining maturity changes, use $P(y,\tau) = 100(1+y)^{-\tau}$:

$$\frac{dP}{dt} = P_y\frac{dy}{dt} + P_\tau\frac{d\tau}{dt}, \qquad P_\tau = -P\log(1+y).$$

In a smooth calendar-time scenario, $d\tau/dt = -1$. The terms capture yield movement and shortening maturity in this model, excluding credit, trading and cash-flow events. The calculation below isolates the yield term.

Let $y(t) = .05 + .002t$, a constructed increase of 20 bp per year. At zero,

$$\frac{dP}{dt} = (-172.7675197063)(.002) = -.3455350394$$

price units per year. The path is specified for teaching, not estimated as a forecast.

For $g(x,z) = x^2 + 3z$, $x(t) = 1 + t$, $z(t) = 2t$, at zero the partials are 2 and 3 and the path speeds 1 and 2. Therefore

$$\frac{dg}{dt} = g_x x' + g_z z' = 2(1) + 3(2) = 8.$$

At $dt = .01$, the differential predicts .08. The exact change $g(1.01, .02) - g(1,0) = .0801$ also includes the quadratic term .0001. The following self-contained code checks the path derivative and both changes.

```
face_value = 100.0
y0 = 0.05
yield_change_per_year = 0.002

def price(yield_rate: float) -> float:
    return face_value / (1.0 + yield_rate) ** 2

def yield_path(time_years: float) -> float:
    return y0 + yield_change_per_year * time_years

def price_path(time_years: float) -> float:
    return price(yield_path(time_years))

analytic_price_rate = (-200.0 / (1.0 + y0) ** 3) * yield_change_per_year
step_years = 1e-4
numeric_price_rate = (
    price_path(step_years) - price_path(-step_years)
) / (2.0 * step_years)

def g(x: float, z: float) -> float:
    return x**2 + 3.0 * z

g_rate = (2.0 * 1.0) * 1.0 + 3.0 * 2.0
dt = 0.01
exact_g_change = g(1.0 + dt, 2.0 * dt) - g(1.0, 0.0)
linear_g_change = g_rate * dt
print("bond price rate per year:", analytic_price_rate, numeric_price_rate)
print("g exact/linear change:", exact_g_change, linear_g_change)
assert abs(analytic_price_rate - numeric_price_rate) < 1e-8
assert abs(exact_g_change - 0.0801) < 1e-12
assert abs(linear_g_change - 0.08) < 1e-12
```

For a vector path,

$$\frac{d}{dt}f(w(t)) = \nabla f(w(t))^\top \frac{dw}{dt}.$$

If the gradient is nonzero, Cauchy-Schwarz gives the smallest derivative over Euclidean unit directions as $-\|\nabla f\|_2$, attained at $-\nabla f/\|\nabla f\|_2$. Changing coordinate scales or the distance metric can change the steepest direction. Geometrically, $q(h) = f(w + hu)$ has $q'(0) = \nabla f^\top u$. If u is not normalized, parameter h no longer equals distance travelled.

6.3 Gradient and Hessian of a Quadratic Objective

Use the teaching score

$$f(w) = \tfrac{1}{2}w^\top \Sigma w - \mu^\top w, \quad \Sigma = \begin{bmatrix} .04 & .01 \\ .01 & .09 \end{bmatrix}, \quad \mu = \begin{bmatrix} .05 \\ .08 \end{bmatrix}.$$

Both expected simple returns and covariance refer to the same annual interval and use decimals. A general mean-variance score has a risk-aversion coefficient:

$$f(w) = \tfrac{\lambda_{\text{risk}}}{2}w^\top \Sigma w - \mu^\top w.$$

That coefficient sets the trade-off and has inverse return units when preserving the return-unit interpretation. Here we fix its numerical value at 1 and suppress it for differentiation. The objective is a risk-return score, not a realized return. Changing decimals to percentages requires a corresponding coefficient change. Chapter 9 develops the convention; its English translation is pending a later batch.

For symmetric Σ,

$$\nabla_w(\tfrac{1}{2}w^\top \Sigma w) = \Sigma w, \quad \nabla_w(-\mu^\top w) = -\mu, \quad \nabla f = \Sigma w - \mu, \quad \nabla^2 f = \Sigma.$$

The shapes in $w^\top \Sigma w$ are $(1 \times 2)(2 \times 2)(2 \times 1)$, leaving a scalar. First,

$$\Sigma w = \begin{bmatrix} .04w_1 + .01w_2 \\ .01w_1 + .09w_2 \end{bmatrix}.$$

Multiplying by $w^\top$ gives two cross terms, $.01w_1w_2$ and $.01w_2w_1$. After multiplying by $1/2$ their sum is $.01w_1w_2$. Thus

$$f = .02w_1^2 + .01w_1w_2 + .045w_2^2 - .05w_1 - .08w_2,$$

$$f_{w_1} = .04w_1 + .01w_2 - .05, \qquad f_{w_2} = .01w_1 + .09w_2 - .08.$$

For example, the derivative of $.02w_1^2$ is $.04w_1$, and of $.01w_1w_2$ with respect to w_1 is $.01w_2$, because w_2 stays fixed. At $(.6, .4)$ these entries are $.024 + .004 - .05 = -.022$ and $.006 + .036 - .08 = -.038$.

Differentiate each gradient entry again:

$$\nabla^2 f = \begin{bmatrix} \partial f_{w_1}/\partial w_1 & \partial f_{w_1}/\partial w_2 \\ \partial f_{w_2}/\partial w_1 & \partial f_{w_2}/\partial w_2 \end{bmatrix} = \begin{bmatrix} .04 & .01 \\ .01 & .09 \end{bmatrix}.$$

The diagonal entries describe individual-coordinate curvature; .01 describes interaction between simultaneous changes. Along d, the quadratic change is $d^\top \Sigma d/2$, positive for every nonzero direction because this matrix is positive definite.

For a budget-preserving transfer $\Delta w = (-.01, .01)$, the linear contribution is $(-.022)(-.01) + (-.038)(.01) = -.00016$. The quadratic contribution is .0000055. With no higher-order remainder, the exact change is $-.0001545$, taking weights from $(.60, .40)$ to $(.59, .41)$. For a nonsymmetric matrix, the quadratic gradient uses $(\Sigma + \Sigma^\top)w/2$; our covariance is symmetric.

6.4 Integrating a Marginal Price Response

Because P' is continuous over $[.05, .06]$,

$$P(.06) - P(.05) = \int_{.05}^{.06} P'(y)\, dy = 88.9996440014 - 90.7029478458 = -1.7033038444.$$

Price per yield unit times yield units returns price units. This is an exact accumulation within the formula; holding the initial slope fixed and multiplying by .01 would give a linear approximation. The negative integral means a net decline, not a negative price. Reversing the endpoints reverses the change's sign. Finer Riemann sums accumulate more local responses over the same interval.

6.5 Central Differences

For a sufficiently smooth scalar function,

$$f(x+h) = f(x) + f'(x)h + \tfrac{1}{2}f''(x)h^2 + O(h^3),$$
$$f(x-h) = f(x) - f'(x)h + \tfrac{1}{2}f''(x)h^2 + O(h^3).$$

Subtract and divide by $2h$ to cancel the even terms:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

Under the stated smoothness, truncation error is $O(h^2)$. Floating-point cancellation prevents unlimited improvement as h shrinks. Compute a vector gradient coordinate by coordinate; Hessian diagonal entries use second central differences and mixed entries use four points.

A large step averages over too much curvature. A very small step subtracts almost equal values and amplifies rounding; it may also divide simulation noise by a tiny number. Check neighboring step sizes for a stable range. One apparently perfect step is weak evidence. Near a parameter boundary, a central difference may call the function outside its legal domain; handle the boundary explicitly or use a justified one-sided method.

6.6 From a Descent Direction to a Finite Step

A negative directional derivative does not guarantee descent at every step size. For our fixed quadratic objective, let $g = \Sigma w - \mu \neq 0$ and take $w_{new} = w - \eta g$, $\eta > 0$. Substituting into the exact second-order change gives

$$f(w - \eta g) - f(w) = -\eta g^\top g + \tfrac{1}{2}\eta^2 g^\top \Sigma g.$$

The negative term grows linearly in the step; positive curvature grows quadratically. For positive definite Σ, strict descent holds exactly when

$$0 < \eta < \frac{2g^\top g}{g^\top \Sigma g}.$$

At our current weights, $g = (-.022, -.038)^\top$, $g^\top g = .001928$, and $g^\top \Sigma g = .00016604$. The upper bound is about 23.22332. Check the intermediate products:

$$\Sigma g = (-.00126, -.00364)^\top, \quad g^\top g = .000484 + .001444 = .001928,$$

$$g^\top \Sigma g = .00002772 + .00013832 = .00016604.$$

The starting score is $-.0452$. Step .1 gives change $-.0001919698$; step 30 gives $+.016878$, raising the score to $-.028322$. At the upper bound the change is zero. When the gradient is zero, this update stays put; do not apply the bound by dividing by zero.

This exact bound belongs to this fixed quadratic score. A general differentiable objective needs a suitably small step, local curvature checks or an explicit backtracking rule. Even the descending .1 step has weights summing to 1.006. Along the existing feasible direction $d = (-1, 1)^\top$, weights $w + sd = (.6 - s, .4 + s)^\top$ preserve the budget; long-only holdings with $s \geq 0$ require $0 \leq s \leq .6$. At $s = .01$ the exact score change is $-.0001545$. Check descent and feasibility separately.

7. Financial Interpretation

$P'(y)$ measures the formula price's local sensitivity to yield. For a one-bp shock, multiply by .0001 after confirming decimal units. It supplies neither a yield forecast nor realized trading profit.

In the portfolio score, gradient entry i gives the first-order response to increasing weight i while holding the others fixed. Positive definite curvature in the global quadratic objective gives strict convexity and a unique unconstrained minimum. Investment constraints add another problem: $\mathbf{1}^\top w = 1$, $w_i \geq 0$, leverage, sectors and turnover. A full gradient step may leave the feasible set, requiring projection, constrained optimization or Lagrange multipliers.

The .1 update in Section 8.2 sums to 1.006. The transfer in Section 6.3 sums to zero as an increment and preserves the budget. For a budget-feasible small direction, $\mathbf{1}^\top d = 0$; at a zero long-only weight, $d_i \geq 0$ is also necessary. At a constrained local minimum, all feasible first-order directional changes must be nonnegative, while the full gradient can be nonzero. That necessary condition alone is not sufficient: inspect curvature or higher-order changes. A multiplier's sign depends on whether you write the budget as $\mathbf{1}^\top w - 1 = 0$ or its negative.

The fixed risk-aversion scaling makes the example a numerical score. Variance and expected return do not naturally have the same units. A financial mean-variance objective needs an explicit coefficient or normalization; these calculations cannot identify a particular investor's utility or preferences.

8. Numerical Examples

8.1 A Zero-Coupon Bond and One Basis Point

At $y = .05$,

$$P = 100/1.05^2 = 90.7029478458,$$

$$P' = -200/1.05^3 = -172.7675197063, \quad P'' = 600/1.05^4 = 493.6214848751.$$

One bp gives linear change $-.01727675197$. A .01 shock is 100 bp and gives linear change -1.7276751971. Direct substitution gives $P(.06) = 88.9996440014$, hence exact change -1.7033038444.

The quadratic approximation restores curvature:

$$P(.06) \approx P(.05) + P'(.05)(.01) + \tfrac{1}{2}P''(.05)(.01)^2 = 88.9999537230.$$

The curvature contribution is $+.0246810742$, giving approximate change -1.7029941228. Exact minus quadratic change is about $-.000309722$. Positive curvature moderates a decline after a positive yield shock and augments a rise after a negative shock. Use a tolerance to decide whether a local approximation is adequate. The following code defines the input and asserts the analytic/numerical agreement in the project environment.

```
import numpy as np
from quant_course.math_tools import finite_difference_gradient,
    ↪ finite_difference_hessian

face_value = 100.0
y = 0.05

def bond_price(point: np.ndarray) -> float:
    return face_value / (1.0 + point[0]) ** 2

price = bond_price(np.array([y]))
analytic_first = -2.0 * face_value / (1.0 + y) ** 3
analytic_second = 6.0 * face_value / (1.0 + y) ** 4
numeric_first = finite_difference_gradient(bond_price, [y], step=1e-4)[0]
numeric_second = finite_difference_hessian(bond_price, [y], step=1e-3)[0, 0]

print(f"price={price:.10f}")
print(f"first analytic/numeric={analytic_first:.10f}/{numeric_first:.10f}")
print(f"second analytic/numeric={analytic_second:.10f}/{numeric_second:.10f}")
assert np.isclose(price, 90.7029478458)
assert np.isclose(analytic_first, numeric_first, rtol=1e-6)
assert np.isclose(analytic_second, numeric_second, rtol=2e-6)
```

8.2 A Step on the Quadratic Surface

For $w = (.6, .4)^\top$,

$$\Sigma w = \begin{bmatrix} .04(.6) + .01(.4) \\ .01(.6) + .09(.4) \end{bmatrix} = \begin{bmatrix} .028 \\ .042 \end{bmatrix}, \quad \nabla f = \begin{bmatrix} -.022 \\ -.038 \end{bmatrix}.$$

The quadratic term is $.6(.028) + .4(.042) = .0336$, and $\mu^\top w = .062$, so $f = .0336/2 - .062 = -.0452$. The Hessian is Σ. Its first leading principal minor is .04 and determinant $.04(.09) - .01^2 = .0035$, both positive.

At learning rate $\eta = .1$,

$$w_{new} = w - .1\nabla f = (.6022, .4038)^\top, \qquad f(w_{new}) = -.0453919698.$$

This particular step descends, but the weights sum to 1.006. Section 6.6 shows why a larger step can increase the score. Compare analytic and central-difference quantities in one place:

```
import numpy as np
from quant_course.math_tools import finite_difference_gradient,
    ↪ finite_difference_hessian

sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
mu = np.array([0.05, 0.08])
weights = np.array([0.6, 0.4])

def objective(point: np.ndarray) -> float:
    return float(0.5 * point @ sigma @ point - mu @ point)
```

```
exact_gradient = sigma @ weights - mu
numeric_gradient = finite_difference_gradient(objective, weights)
numeric_hessian = finite_difference_hessian(objective, weights)
next_weights = weights - 0.1 * exact_gradient

print(f"f={objective(weights):.10f}")
print("gradient:", exact_gradient)
print("numeric gradient:", numeric_gradient)
print("numeric Hessian:\\n", numeric_hessian)
print("one unconstrained step:", next_weights)
print(f"new objective={objective(next_weights):.10f}")
assert np.isclose(objective(weights), -0.0452)
assert np.allclose(exact_gradient, [-0.022, -0.038])
assert np.allclose(numeric_hessian, sigma, atol=2e-7)
assert objective(next_weights) < objective(weights)
```

9. Python Implementation

Default Entry in the Accepted Reader Package

Follow the reader installation instructions. From its root, run:

```
.venv/bin/python run_notebook.py notebooks/ch04_calculus.ipynb
```

The default Notebook calls the packaged shared implementation, reads no absent French inputs and downloads nothing. New execution records and figures go to a unique runs subdirectory; reader CLI outputs use universally unique identifiers (UUIDs) and reject overwrites. Historical production tests that were not packaged remain references to original checks. Use the Notebook's assertions for the reader route. French inputs require the optional-data contract and were not rerun in this batch.

The separate original course route uses `ch04_calculus.py`, the Notebook, and shared **src/quant_course/math_tools.py**. Install its project environment, then run from the course root:

```
.venv/bin/python tools/run_a2.py --chapter 4
```

The deterministic formulas have no random seed, market observations, signal timestamp, trade timestamp or measured holding interval. A strategy application must establish input availability at signal_time, later execution at trade_time and returns from actual position activation.

The application programming interface (API) `finite_difference_gradient` takes a callable scalar function, a finite nonempty real vector and a step, defaulting to `1e-5`; it returns a floating-point vector of the same length. `finite_difference_hessian` takes the same kind of inputs, defaults to `1e-4`, and returns a square matrix matching the input dimension. Every perturbed function call must return a finite real scalar. NaN, infinity, complex or nonnumeric inputs cause explicit errors. A shared default step is a starting point; inspect coordinate scales and stability.

Hessian diagonals use $[f(x+h)-2f(x)+f(x-h)]/h^2$; mixed terms use four points. NumPy `gradient` is a different interface for sampled arrays: central differences inside, one-sided differences at boundaries, preserving shape. Supply the actual coordinates rather than treating each gap as one:

```
import numpy as np

time_years = np.array([0.0, 0.4, 1.0, 1.8, 3.0])
values = time_years**2
estimated_slope = np.gradient(values, time_years)
print("time in years:", time_years)
print("sampled function:", values)
print("estimated derivative:", estimated_slope)
assert np.isclose(estimated_slope[2], 2.0)
```

10. Visualization: Error Orders and Floating-Point Limits

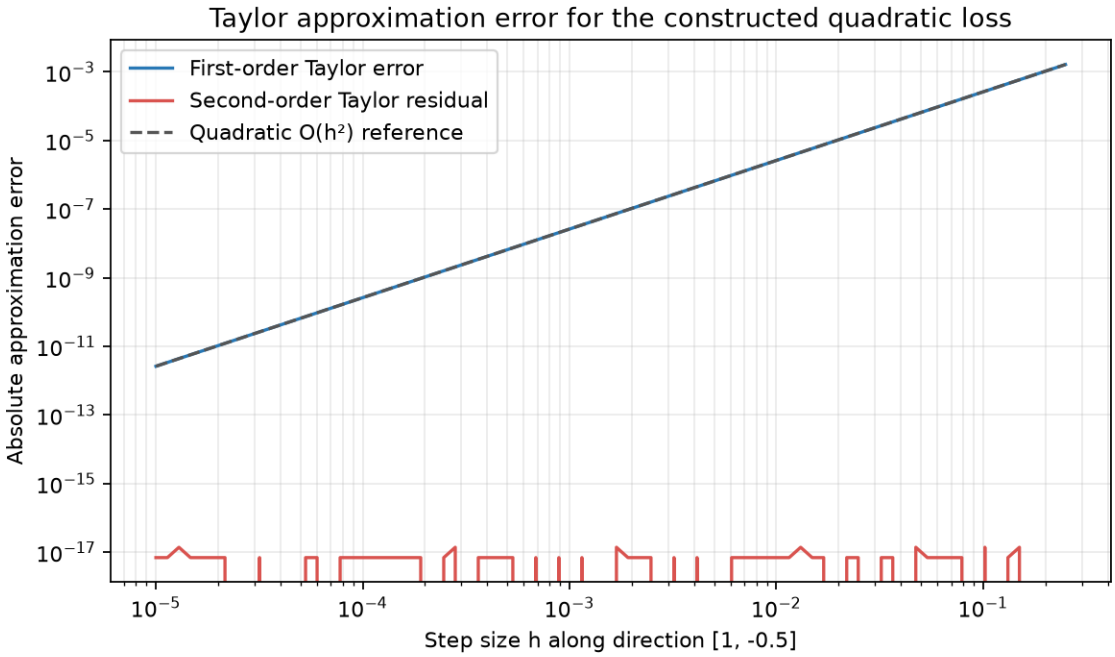


Figure 1: Taylor errors for a constructed quadratic objective

The horizontal axis is step h along direction $(1,-.5)$, on a log scale; the vertical axis is absolute approximation error in the objective's score units. Blue shows first-order error, and a dashed line supplies an $O(h^2)$ reference. Red shows the second-order residual. Because the objective is quadratic, its exact second-order remainder is zero; numerical residuals reach a rounding floor rather than continuing the theoretical power law.

Compare slopes and the plateau. The plateau does not establish a feature of the exact function. This is neither a yield curve nor measured market-price error. The bond outputs support hand calculations; the figure supports the approximation-error lesson.

11. From Constructed Formulas to Market Research

This chapter has not downloaded or analyzed bond prices, curves or cash flows. It has no empirical period, market frequency, data authorization, market-version hashes or market result. Face value 100, two-year maturity and 5% annual-compounding yield are teaching assumptions.

A fixed-income study needs security identity, coupons and payment dates, day count, yield compounding, clean or dirty price, curve source, timestamps and versions. Real cash flows require their respective discount periods; a single yield and factor need not suffice. Credit spreads, curve changes, liquidity and accrued interest affect quotes.

Specify security selection, quote observation times, yield-price matching, duplicates, missingness, accrual and cash-flow mapping to curve nodes. Check a fitted sensitivity on another period or other securities. Without those records the derivation explains a model implication rather than how the market behaved. Later financial chapters develop duration, convexity and market data; their English editions are pending later batches.

12. Common Mistakes

12.1 Calling .01 a One-Basis-Point Shock

Decimal yields require .0001 for one bp. The negative assertion below demonstrates that .01 is a different shock; an assertion that the two shocks were equal would fail.

```
import numpy as np

y = 0.05
dy_wrong = 0.01
dy_one_bp = 0.0001
price = lambda rate: 100.0 / (1.0 + rate) ** 2

wrong_change = price(y + dy_wrong) - price(y)
one_bp_change = price(y + dy_one_bp) - price(y)
print(f"wrongly labeled one bp: {wrong_change:.10f}")
print(f"actual one bp: {one_bp_change:.10f}")
assert not np.isclose(dy_wrong, dy_one_bp)
assert np.isclose(one_bp_change, -0.0172742842, atol=1e-10)
```

The exact one-bp change is $-.0172742842$; the .01 change is -1.7033038444, both in price units.

12.2 Omitting Parentheses in a Difference Quotient

Common mistake: missing parentheses

A forward difference subtracts two function values before division. Dividing only the second value changes both the result and units.

```
import numpy as np

function = lambda value: value**2
x = 3.0
h = 0.01
wrong = function(x + h) - function(x) / h
correct = (function(x + h) - function(x)) / h
print(f"wrong={wrong:.6f}, correct={correct:.6f}, exact={2*x:.6f}")
assert not np.isclose(wrong, 2.0 * x)
assert np.isclose(correct, 6.01)
```

The correct forward difference is 6.01, near the derivative 6; the wrong expression is -890.9399.

12.3 Treating a Quadratic Taylor Polynomial as an Exact Formula

A second-order Taylor expression needs its remainder except when the function's form makes the remainder zero. At zero, approximating $e^{.5}$ gives:

```
import math

x = 0.5
approximation = 1.0 + x + x**2 / 2.0
exact = math.exp(x)
error = exact - approximation
print(f"quadratic approximation={approximation:.10f}")
print(f"exact value={exact:.10f}")
print(f"omitted remainder={error:.10f}")
assert error > 0.0
```

The polynomial is 1.625; the exact value is about 1.6487212707; the omitted remainder is about .0237212707.

12.4 Calling Every Zero Gradient a Minimum

At the origin, $x^2 - y^2$ rises along x and falls along y. The first-order condition finds a stationary point, while Hessian signs and directional checks identify the saddle:

```
import numpy as np

def saddle(point: np.ndarray) -> float:
    return float(point[0] ** 2 - point[1] ** 2)

origin = np.array([0.0, 0.0])
gradient = np.array([2.0 * origin[0], -2.0 * origin[1]])
hessian = np.array([[2.0, 0.0], [0.0, -2.0]])
wrongly_called_minimum = np.allclose(gradient, 0.0)
eigenvalues = np.linalg.eigvalsh(hessian)
rise = saddle(np.array([0.1, 0.0])) - saddle(origin)
fall = saddle(np.array([0.0, 0.1])) - saddle(origin)
print("gradient:", gradient, "naive minimum label:", wrongly_called_minimum)
print("Hessian eigenvalues:", eigenvalues, "directional changes:", rise, fall)
assert wrongly_called_minimum
assert eigenvalues[0] < 0.0 < eigenvalues[1]
assert rise > 0.0 and fall < 0.0
```

12.5 Treating an Unconstrained Step as a Feasible Portfolio

The Section 8.2 step sums to 1.006. Specify constraints before choosing a projected update or feasible direction. The example retains the wrong unconstrained update for comparison and separately checks a budget-preserving step.

```
import numpy as np

sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
mu = np.array([0.05, 0.08])
weights = np.array([0.6, 0.4])

def objective(point: np.ndarray) -> float:
    return float(0.5 * point @ sigma @ point - mu @ point)

gradient = sigma @ weights - mu
unconstrained = weights - 0.1 * gradient
feasible_direction = np.array([-1.0, 1.0])
feasible = weights + 0.01 * feasible_direction
print("unconstrained weights/sum:", unconstrained, unconstrained.sum())
print("feasible weights/sum:", feasible, feasible.sum())
print("starting/feasible objective:", objective(weights), objective(feasible))
assert not np.isclose(unconstrained.sum(), 1.0)
assert np.isclose(feasible.sum(), 1.0)
assert np.all(feasible >= 0.0)
assert objective(feasible) < objective(weights)
```

12.6 Comparing Gradients Across Different Coordinate Units

A gradient mixing percentage-point and ratio coordinates mixes units. Specify scaling and its effects on gradients, Hessians and steps. Changing units changes the numerical optimization path, not the economic question. Here the derivative per bp is one ten-thousandth of the derivative per decimal unit, while the same physical shock has the same impact:

```
import numpy as np

y = 0.05
one_basis_point_in_decimal = 0.0001
price = lambda rate: 100.0 / (1.0 + rate) ** 2
derivative_per_decimal = -200.0 / (1.0 + y) ** 3
derivative_per_basis_point = derivative_per_decimal / 10000.0
impact_from_decimal_coordinate = derivative_per_decimal * one_basis_point_in_decimal
impact_from_bp_coordinate = derivative_per_basis_point * 1.0
print("derivative per decimal unit:", derivative_per_decimal)
print("derivative per basis point:", derivative_per_basis_point)
print("same shock impact:", impact_from_decimal_coordinate, impact_from_bp_coordinate
    ↪ )
assert np.isclose(impact_from_decimal_coordinate, impact_from_bp_coordinate)
```

12.7 Omitting Path Speed from the Chain Rule

P' has price/yield units; dP/dt has price/time units. A constructed yield increase of 20 bp per year has speed .002 decimal units per year:

```
import numpy as np

y = 0.05
yield_change_per_year = 0.002
derivative_price_per_yield = -200.0 / (1.0 + y) ** 3
wrong_price_rate = derivative_price_per_yield
correct_price_rate = derivative_price_per_yield * yield_change_per_year
print("wrong units/value:", wrong_price_rate, "price per yield unit")
print("chain-rule rate:", correct_price_rate, "price per year")
assert not np.isclose(wrong_price_rate, correct_price_rate)
assert np.isclose(correct_price_rate, -0.3455350394)
```

13. Connecting Local Calculations to Research Decisions

State the objective, units, frequency and estimation window. Compute its value, gradient and Hessian. A candidate direction with $\nabla f^\top d < 0$ gives first-order descent for a sufficiently small positive step; curvature can change the result for a finite step.

Estimate μ and Σ from observations available before signal_time. Declare budget, leverage, shorting, sector and turnover constraints. Compare analytic and numerical derivatives over sensible scales and steps. Project a candidate into the feasible set or use a constrained solver. Evaluate subsequent holding returns after trade_time, including trading, funding, borrow costs and weight drift, against a simple benchmark.

Keep input checks, derivative comparisons, stationarity classifications, feasibility and out-of-sample performance as separate checks. Success of one cannot fill missing evidence for another. Fees, impact and data availability can change the entire objective's relevance to executable decisions.

14. Chapter Summary

Derivatives give local rates in specified units. The chain rule combines input sensitivities with path speeds. Gradients collect coordinate rates and Hessians describe curvature. Taylor approximations require their smoothness and remainder conditions; integrals accumulate marginal responses under the fundamental theorem's conditions.

The constructed bond slopes downward and curves upward. The quadratic portfolio score has gradient $\Sigma w - \mu$ and Hessian Σ. A negative-gradient direction needs a finite-step check, and a descending update needs a feasibility check. Numerical agreement helps find implementation errors; market claims still require the relevant data, timing and cost evidence.

15. Exercises

4-N01: Inputs and Difference Quotients

For $q(x) = 2x^2 + 1$, identify independent and dependent variables and explain exponent 2. For a move from 2 to 2.1, calculate $h, q(2), q(2.1), \Delta q$ and $\Delta q/h$ in order. State what the quotient measures.

4-N02: Bond Derivatives and Basis Points

Set $u = 1 + y$ in $P(y) = 100(1 + y)^{-2}$. Derive P' and P'' using the power and chain rules. Evaluate at .05 to four decimal places. Convert a 25-bp rise to decimals, compute the linear price change and give derivative and change units.

4-N03: Diagnose a Slope Labelled as a Price Change

The following code claims to report a one-bp price change. Explain the mismatch between its name and value. Replace it with a linear price-change approximation and check against the difference of formula prices.

```
def price(y):
    return 100.0 / (1.0 + y) ** 2

y = 0.05
one_bp = 0.0001
reported_price_change = (price(y + one_bp) - price(y)) / one_bp
assert abs(reported_price_change + 172.7675) < 0.1
```

4-N04: Model Implications and Market Evidence

A researcher uses $P'(.05) = -172.7675$ as evidence that yield will rise tomorrow and that a real bond will lose about .0173 price units. Can either claim follow from the teaching formula? Before reporting a real bond's one-bp sensitivity, which model, quote and timing fields must be recorded?

4-E01: Continuity and Differentiability

Explain why differentiability implies continuity. Give a continuous but nondifferentiable function, identify the point, and compare its left and right difference quotients.

4-E02: Yield-Shock Units

Convert 1, 25 and 100 bp to decimal increments. If a program uses .01 for one bp, how many bp does it actually apply?

4-E03: A Price Slope

Derive $P'(y)$ from $100/(1+y)^2$. Explain its sign, value and units at .05. Does it predict tomorrow's yield direction?

4-E04: Total Differential and Chain Rule

For $g = x^2 + 3z$, $x(t) = 1 + t$, $z(t) = 2t$, find the two partials and dg/dt at zero and check the chain rule.

4-E05: Unit Directions

Explain normalization when comparing directional derivatives. Normalize $u = (1, 1)$.

4-E06: Gradient and Hessian

Find both for $q(x, y) = x^2+xy+2y^2$. Check positive definiteness and interpret the stationary point.

4-E07: Taylor Remainders

Which smoothness is needed for a third-order Lagrange remainder after a second-order expansion? What changes for a quadratic polynomial?

4-E08: Stationary-Point Classification

Give two-variable functions with a local minimum, local maximum and saddle at zero, checking their gradients and Hessians.

4-E09: Accumulating a Marginal Change

Compute $\int_0^1 2x\,dx$. Interpret it with an antiderivative and as accumulated marginal change.

4-B01: Direction and Finite Step Size

For the existing quadratic score at $w = (.6, .4)^\top$, derive the exact negative-gradient update and strict-descent interval. Compare steps .1 and 30 and check weight sums. Along $d = (-1, 1)^\top$ with $s \geq 0$, which steps meet the budget and long-only constraints? Why does score descent not establish better real investment returns?

16. Programming Exercises

4-P01: Check Bond Differences

At $y = .05$, compare analytic first and second derivatives with the project's central-difference functions. Use $h = 10^{-4}$ for the first and 10^{-3} for the second. Print differences and assert suitable tolerances. State the units of one-bp and 100-bp price changes.

4-P02: Check the Quadratic Score

Using the chapter's Σ, μ, w, compute the score, analytic and numerical gradients, numerical Hessian, and an unconstrained .1 update. Assert descent and report the weight sum. Explain why this has not solved a feasible portfolio problem.

4-P03: Sampled Derivatives and Integrals

Sample x^2 at nonuniform nodes $[0, .4, 1, 1.8, 3]$. Pass the coordinates to NumPy `gradient` and check the interior derivative. Approximate $\int_0^1 x^2 dx$ by midpoint sums with 10, 100 and 1000 intervals; explain the error changes.

17. Challenge Problems

4-C01: Choosing a Yield-Shock Approximation

From yield .05, compare exact, linear and quadratic changes for rises of 1, 25 and 100 bp. Declare a repeatable tolerance and decide which approximations meet it. Distinguish floating-point error from model error.

Ten points: units/shocks 2, exact and approximate values 3, error/tolerance 2, small/large-step interpretation 2, constructed rather than market identity 1. No single tolerance is mandatory; a reproducible rule is.

4-C02: Add a Full-Investment Budget

Solve

$$\min_{w} \ \tfrac{1}{2} w^{\top} \Sigma w - \mu^{\top} w, \qquad \mathbf{1}^{\top} w = 1, \quad w \geq 0.$$

Explain why $\Sigma^{-1}\mu$ is not the budget answer. Derive the budget-only equality solution, then discuss boundary candidates under nonnegativity. Supply reproducible values or full enumeration steps, and state the solver checks.

Twelve points: unconstrained/budget conflict 2, Lagrange derivation 3, nonnegative boundary analysis 3, numerical/analytic result 2, feasibility/objective verification 2. Bare library output without constraints and verification earns at most 4.

18. Complete Solutions

4-N01

x is the input and $q(x)$ the output; exponent 2 means $x \cdot x$. Here $h = .1$,

$$q(2) = 9, \quad q(2.1) = 9.82, \quad \Delta q = .82, \quad \Delta q/h = 8.2.$$

The quotient has output/input units and is the average rate on $[2, 2.1]$. More generally,

$$\frac{q(2+h) - q(2)}{h} = \frac{2[(2+h)^2 - 2^2]}{h} = \frac{8h + 2h^2}{h} = 8 + 2h \longrightarrow 8.$$

Take the limit through nonzero increments.

4-N02

With $P = 100u^{-2}$, $u = 1 + y$, the outer derivative is $-200u^{-3}$, and the inner derivative is 1. Hence

$$P' = -200(1 + y)^{-3}, \quad P'' = (-200)(-3)(1 + y)^{-4} = 600(1 + y)^{-4}.$$

At .05, these are -172.7675 and 493.6215. Their units are price/decimal yield and price/(decimal yield squared). Since 25 bp is .0025,

$$\Delta P \approx (-172.7675197063)(.0025) = -.4319$$

price units. The negative response belongs to the formula and does not predict yield movement.

4-N03

Dividing a price change by a yield change produces an average rate, not a one-bp price change. The assertion tests closeness of the quotient to the derivative within .1. Its variable name cannot change its units. Multiply the analytic slope by the shock for a linear approximation; subtract the two prices for the exact constructed change:

```
def price(y):
    return 100.0 / (1.0 + y) ** 2

y = 0.05
one_bp = 0.0001
slope = -200.0 / (1.0 + y) ** 3
linear_price_change = slope * one_bp
exact_price_change = price(y + one_bp) - price(y)
assert abs(linear_price_change - (-0.01727675197063)) < 1e-12
assert abs(exact_price_change - (-0.0172742842)) < 1e-10
```

The linear change is about $-.0172767520$, the exact change $-.0172742842$, in price units. Curvature accounts for their small difference.

4-N04

The derivative is a local slope of a single-cash-flow, two-year, annual-compounding rule at 5%. Given a one-bp rise, it supports a linear change of about $-.0173$. It contains no forecast of tomorrow's yield and does not establish a real bond's observed response.

Record identity, coupons, cash-flow dates, remaining maturity, day count, compounding, clean/dirty quote convention, quote source and observation time, yield-price matching and version. A future-price claim also needs information-timed evidence for yield direction, costs and position activation.

4-E01

Write

$$f(x+h)-f(x)=h\left(\frac{f(x+h)-f(x)}{h}\right).$$

As $h\to 0$, the finite convergent quotient times h tends to zero, proving continuity. For $|x|$ at zero, the function and limit are zero, while left/right quotients are -1 and 1, so no derivative exists.

4-E02

The increments are .0001, .0025 and .01. Labelling .01 as one bp applies 100 bp, a hundredfold shock and an absolute error of 99 bp.

4-E03

$P'=-200(1+y)^{-3}$. At .05 it is -172.7675197063 price units per decimal yield unit. Higher yield lowers this formula price locally. The slope does not estimate tomorrow's yield direction.

4-E04

$g_x=2x$, $g_z=3$. At zero, $x=1,z=0$, so partials are 2 and 3. The chain rule gives $2(1)+3(2)=8$. Direct substitution gives $g(t)=(1+t)^2+6t$, whose derivative at zero is also 8.

4-E05

An unnormalized vector mixes direction and speed in $\nabla f^{\top}u$. Normalize to compare equal-length movements. The norm of $(1,1)$ is $\sqrt{2}$; the unit direction is $(1/\sqrt{2},1/\sqrt{2})$.

4-E06

$$\nabla q=\begin{bmatrix}2x+y\\x+4y\end{bmatrix},\quad \nabla^2 q=\begin{bmatrix}2&1\\1&4\end{bmatrix}.$$

Leading minor 2 and determinant 7 are positive. The strictly convex quadratic has its unique global and strict local minimum at its stationary point $(0,0)$.

4-E07

Third differentiability along the segment is needed for the intermediate-point Lagrange expression; a uniform bound also needs bounded third derivatives. A quadratic equals its second-order Taylor polynomial, so the third-order remainder is zero.

4-E08

$x^2 + y^2$, $-x^2 - y^2$ and $x^2 - y^2$ have zero gradients at zero and Hessians $2I$, $-2I$, and diag(2, -2). These give minimum, maximum and saddle respectively.

4-E09

$$\int_0^1 2x\,dx = [x^2]_0^1 = 1.$$

The marginal response $2x$ belongs to $F = x^2$; accumulated change from zero to one equals $F(1) - F(0) = 1$.

4-P01: Reference Implementation

Run in the project's frozen environment. Define the teaching function locally and compare both derivative orders:

```
import numpy as np
from quant_course.math_tools import finite_difference_gradient,
    ↪ finite_difference_hessian

face_value = 100.0
y = 0.05
price = lambda point: face_value / (1.0 + point[0]) ** 2
exact_first = -2.0 * face_value / (1.0 + y) ** 3
exact_second = 6.0 * face_value / (1.0 + y) ** 4
numeric_first = finite_difference_gradient(price, [y], step=1e-4)[0]
numeric_second = finite_difference_hessian(price, [y], step=1e-3)[0, 0]
one_bp = price(np.array([y + 0.0001])) - price(np.array([y]))
one_hundred_bp = price(np.array([y + 0.01])) - price(np.array([y]))

print("price:", price(np.array([y])))
print("first:", exact_first, numeric_first)
print("second:", exact_second, numeric_second)
print("1 bp exact price change:", one_bp)
print("100 bp exact price change:", one_hundred_bp)
assert np.isclose(exact_first, -172.7675197063)
assert np.isclose(exact_second, 493.6214848751)
assert np.isclose(numeric_first, exact_first, rtol=1e-6)
assert np.isclose(numeric_second, exact_second, rtol=2e-6)
assert np.isclose(one_bp, -0.0172742842, atol=1e-10)
assert np.isclose(one_hundred_bp, -1.7033038444, atol=1e-10)
```

4-P02: Reference Implementation

The snippet defines its inputs. Central differences check the analytic gradient that supplies the update direction:

```
import numpy as np
from quant_course.math_tools import finite_difference_gradient,
    ↪ finite_difference_hessian

sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
mu = np.array([0.05, 0.08])
weights = np.array([0.6, 0.4])

def objective(point: np.ndarray) -> float:
    return float(0.5 * point @ sigma @ point - mu @ point)

gradient = sigma @ weights - mu
numeric_gradient = finite_difference_gradient(objective, weights)
hessian = finite_difference_hessian(objective, weights)
new_weights = weights - 0.1 * gradient
print("objective:", objective(weights))
print("gradient:", gradient)
print("numeric gradient:", numeric_gradient)
print("numeric Hessian:\\n", hessian)
print("new weights:", new_weights, "sum:", new_weights.sum())
print("new objective:", objective(new_weights))
assert np.isclose(objective(weights), -0.0452)
assert np.allclose(gradient, [-0.022, -0.038])
assert np.allclose(numeric_gradient, gradient, atol=1e-9)
assert np.allclose(hessian, sigma, atol=2e-7)
assert objective(new_weights) < objective(weights)
assert not np.isclose(new_weights.sum(), 1.0)
```

The final assertion deliberately shows the budget violation. A descending program objective has not solved the constrained problem.

4-P03: Reference Implementation

Use actual nonuniform coordinates. Refine midpoint intervals to check error reduction against the analytic integral:

```
import numpy as np

nodes = np.array([0.0, 0.4, 1.0, 1.8, 3.0])
samples = nodes**2
derivative = np.gradient(samples, nodes)
print("interior derivative at x=1:", derivative[2])
assert np.isclose(derivative[2], 2.0)

def midpoint_integral(n: int) -> float:
    edges = np.linspace(0.0, 1.0, n + 1)
    midpoints = (edges[:-1] + edges[1:]) / 2.0
    width = 1.0 / n
    return float(np.sum(midpoints**2) * width)

exact = 1.0 / 3.0
errors = [abs(midpoint_integral(n) - exact) for n in (10, 100, 1000)]
print("midpoint errors:", errors)
assert errors[0] > errors[1] > errors[2]
```

The exact integral is $[x^3/3]_0^1 = 1/3$. On the nonuniform grid, NumPy uses neighboring coordinate gaps to construct interior differences.

4-C01: Values and a Repeatable Decision Rule

Decimal yield change	Basis points	Exact change	Linear change	Quadratic change
.0001	1	-.0172742842	-.0172767520	-.0172742839
.0025	25	-.4303811146	-.4319187993	-.4303762321
.0100	100	-1.7033038444	-1.7276751971	-1.7029941228

An absolute price-error tolerance of .001 accepts all three quadratic approximations and rejects the linear 100-bp approximation. At a relative tolerance of .1% of the exact change, the 100-bp quadratic error is about .0182% and passes; the linear error is about 1.43% and fails. Declare the threshold first and inspect neighboring shocks.

Roundoff arises from finite precision, often creating an error plateau at tiny steps. Model error arises from wrong cash flows, yield conventions or economic assumptions. A smaller difference step may alter the former without repairing the latter. These figures measure the constructed formula's approximations, not a real bond's quote errors.

4-C02: Derivation and Candidate Optimum

The unconstrained stationarity equation is $\Sigma w - \mu = 0$, yielding

$$w_0 = \Sigma^{-1}\mu = (1.0571428571, .7714285714)^\top.$$

Its sum is about 1.82857, violating the budget. With the equality constraint, use

$$\mathcal{L}(w, \lambda) = \tfrac{1}{2} w^\top \Sigma w - \mu^\top w + \lambda(\mathbf{1}^\top w - 1).$$

Stationarity gives $\Sigma w - \mu + \lambda \mathbf{1} = 0$, so

$$w(\lambda) = \Sigma^{-1}(\mu - \lambda \mathbf{1}), \quad \lambda = \frac{\mathbf{1}^\top \Sigma^{-1}\mu - 1}{\mathbf{1}^\top \Sigma^{-1}\mathbf{1}}.$$

The values are

$$\lambda = .0263636364, \quad w = (.45454545, .54545455)^\top, \quad f(w) = -.0463636364.$$

Both weights are nonnegative. Positive definiteness gives a strictly convex score on the convex feasible set, making this the unique long-only global optimum as well. Boundary scores are $f(1, 0) = -.03$ and $f(0, 1) = -.035$, both higher. Verify the equality solution:

```
import numpy as np

sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
mu = np.array([0.05, 0.08])
ones = np.ones(2)
sigma_inverse_mu = np.linalg.solve(sigma, mu)
sigma_inverse_ones = np.linalg.solve(sigma, ones)
lagrange_multiplier = (ones @ sigma_inverse_mu - 1.0) / (ones @ sigma_inverse_ones)
weights = np.linalg.solve(sigma, mu - lagrange_multiplier * ones)

def objective(point: np.ndarray) -> float:
    point = np.asarray(point, dtype=float)
    return float(0.5 * point @ sigma @ point - mu @ point)

kkt_residual = sigma @ weights - mu + lagrange_multiplier * ones
print("lambda:", lagrange_multiplier)
print("constrained weights:", weights)
print("KKT stationarity residual:", kkt_residual)
print("sum:", weights.sum(), "objective:", objective(weights))
print("boundary objectives:", objective([1.0, 0.0]), objective([0.0, 1.0]))
assert np.isclose(weights.sum(), 1.0)
assert np.allclose(kkt_residual, np.zeros(2), atol=1e-12)
assert np.isclose(objective(weights), -0.0463636364)
assert np.all(weights >= 0.0)
assert objective(weights) < min(objective([1.0, 0.0]), objective([0.0, 1.0]))
```

In higher dimensions inspect active zero-weight boundaries and the Karush-Kuhn-Tucker (KKT) complementary-slackness conditions. Check primal feasibility, stationarity/dual conditions and the objective; a solver's success status alone is insufficient.

4-B01

Substitute $h = -\eta g$ into the exact quadratic increment $g^\top h + h^\top \Sigma h/2$:

$$\Delta f = -.001928\eta + .00008302\eta^2.$$

Strict descent requires $0 < \eta < 23.22331968$. At .1, change is $-.0001919698$, weights (.6022, .4038), sum 1.006. At 30, change is $+.016878$, weights $(1.26, 1.54)$, sum 2.8. Both fail the budget, and the latter also increases the score.

Along $(.6 - s, .4 + s)$, the sum remains one and nonnegativity with $s \geq 0$ requires $0 \leq s \leq .6$. Its exact change is $-.016s + .055s^2$, giving $-.0001545$ at .01. Within the feasible interval, strict descent for positive steps additionally requires $s < .016/.055$. These are fixed teaching-score conditions. Investment-return claims need estimates, information timing, execution, costs and out-of-sample evaluation.

19. Further Reading

- **Must know:** MIT 18.01SC sessions 3-5 and 48 for rates, limits, continuity and the fundamental theorem.
- **Must know:** MIT 18.02SC sessions 26-30 and 32-38 for partial derivatives, tangent approximations, optimization, chain rule, differentials and directions.
- **Should know:** NumPy `gradient` documentation for coordinate spacing and interior/boundary formulas.
- **Advanced:** Boyd and Vandenberghe, *Convex Optimization*, Section 3.1 and Chapter 9 for optimality conditions and descent methods.
- **Optional:** MIT 18.01SC Taylor's Formula resources for one-variable expansions and remainders.

20. References and Reading Boundaries

The inherited sources, exact URLs, verification methods and limits appear in the Chinese source audit record. The original exposition and derivations explain the mathematics; MIT OpenCourseWare supports topic placement and formula structure, Boyd supports convex optimality conditions, and NumPy supports sampled-array difference semantics. The bond and quadratic numbers are constructed by the author. No new external-document reading, market data download or market execution took place for this translation.

Chapter 5: Linear Algebra, Portfolio Objects and Risk

Spine: matrices let you calculate across assets; shape, orientation and sample meaning give those calculations their financial interpretation. Prerequisites: Chapter 2's arrays and indexing, and elementary algebra. Chapter 6 develops probability and covariance formally; its current Chinese edition remains the reference until a later English batch. Here we begin with a supplied covariance matrix. Principal component analysis (PCA) follows Chapter 3's information-time contract.

1. Learning Objectives

Write holdings as vectors and multi-asset returns as matrices, explaining each axis before computing. Read matrix multiplication, transpose, rank, basis and orthogonality, and recognize singular or ill-conditioned systems.

For risk, derive portfolio variance from a two-asset expansion and assess whether a candidate matrix can be a covariance matrix. For dimension reduction, read singular value decomposition (SVD) and PCA: training centering, directions, scores, explained variance and the exclusion of future rows from fitting.

You need not master every abstract vector-space theorem yet. Check dimensions, calculate small examples and understand the conditions before verifying them in code. Chapter 8 develops regression; Chapters 9 and 20 use these objects in portfolio problems. Chapter 20's English edition is already available; other later chapters remain Chinese pending translation.

2. Why Quantitative Research Needs Linear Algebra

With one stock you can calculate each number. With 500 stocks, a date has 500 returns, 500 weights, many exposure coordinates and a 500-by-500 risk matrix. Explicit shapes make the calculations manageable and expose missing terms or incorrect axis order.

A solver does not know that monthly covariance was mixed with annual expected returns, or that column 17 now represents another security. Arrays retain positions after labels are removed. Data engineering establishes identities and available times; linear algebra then computes under those contracts.

We use a return panel, a weight vector and a covariance matrix throughout. Chapter 2 introduced broadcasting; here matrix products explain the resulting portfolio returns. Chapter 21 later addresses estimation error and investment constraints.

Input conditions: axes, intervals and units

Before computing, record asset order, observation axis, return interval and units. Annual decimal returns multiplied by dimensionless weights give annual decimal portfolio returns. Covariance and $w^\top \Sigma w$ have squared-return units; their square roots have return units. Equal shapes cannot justify mixing daily, monthly and annual objects. Aggregated variance also

Continued

depends on cross-period covariance, so annualization needs time-dependence assumptions; compounded simple returns need their product rule.

3. Intuition

A weight vector is a recipe; a contemporaneous return vector records how each ingredient changes over one holding interval. Their inner product multiplies paired entries and adds them.

A matrix can map weights to returns by date, or holdings to industry and style exposures. A return panel and a risk matrix are both two-dimensional but have different axes.

Two volatile assets can move imperfectly together. Risk depends on both individual movement and joint movement. An eigenvector keeps its direction under a matrix transformation; an eigenvalue measures the scaling in that direction. For a risk matrix, these directions carry particular variances.

PCA rotates coordinates toward the greatest sample variation, then the greatest remaining orthogonal variation. Historical variance concentration alone does not identify a useful forecast or investment direction.

4. Mathematical Foundations

4.1 Vectors, Length and Transpose

Use column vectors:

$$w = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}, \qquad R_t = \begin{bmatrix} R_{1,t} \\ R_{2,t} \end{bmatrix}.$$

Weights and same-interval simple returns are dimensionless; each vector is 2×1. Transpose turns w into a 1×2 row. Then

$$w^\top R_t = w_1 R_{1,t} + w_2 R_{2,t}$$

is a scalar. Interpreting it as realized portfolio return also requires actual weights held at the interval's start.

For $a, b \in \mathbb{R}^p$, the inner product is $a^\top b = \sum_j a_j b_j$, and Euclidean length is

$$\|a\|_2 = \sqrt{a^\top a} = \sqrt{\sum_j a_j^2}.$$

Length is not gross exposure. Gross exposure uses the L1 norm $\sum_j |a_j|$. For $(.5, -.5)$, Euclidean length is about .7071 while gross exposure is 1. Select the norm for the quantity you intend to measure.

At start-of-interval weights $(.6, .4)^\top$ and returns $(.02, -.01)^\top$,

$$\begin{bmatrix} .6 & .4 \end{bmatrix} \begin{bmatrix} .02 \\ -.01 \end{bmatrix} = .6(.02) + .4(-.01) = .008.$$

This is .8%, a scalar. Choosing those weights after seeing the interval's returns would introduce future information.

4.2 Return Panels and Matrix Multiplication

Let $X \in \mathbb{R}^{n\times p}$ have observation intervals as rows and securities as columns, with $X_{ti} = R_{i,t}$. In our two-interval example, reset to fixed target weights each interval and ignore fees/cash. Then

$$y = Xw \in \mathbb{R}^{n\times 1}, \qquad y_t = \sum_i X_{ti} w_i.$$

This is a specified-weight mapping, not an automatic rebalancing backtest. Drifting positions need the actual interval weights; Chapter 29 later develops the ledger.

For $A \in \mathbb{R}^{m\times k}$ and $B \in \mathbb{R}^{k\times n}$,

$$(AB)_{ij} = \sum_{\ell=1}^{k} A_{i\ell} B_{\ell j}.$$

The shared axis k contracts, leaving $m \times n$. The output row is i, column j, and summation axis ℓ. Multiplication generally does not commute; BA may be undefined. Identity I_p has ones on the diagonal and zeros elsewhere, with $I_p w = w$.

A fixed matrix maps x linearly to Ax. For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad x = (2,-1)^\top, \quad Ax = 2\begin{bmatrix} 1 \\ 3 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

The first column is the image of $(1,0)^\top$, the second the image of $(0,1)^\top$; input coordinates combine those columns. Column order therefore has meaning.

Calculate every output cell:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 1(5)+2(7) & 1(6)+2(8) \\ 3(5)+4(7) & 3(6)+4(8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

Each entry is a row of the left matrix dotted with a column of the right. In Xw, that dot product produces one return per date. Shapes $(n \times p)(p \times 1)$ yield $n \times 1$.

The elementwise, or Hadamard, product has $(A \odot B)_{ij} = A_{ij} B_{ij}$, requiring compatible shapes or specified broadcasting. NumPy `@` multiplies matrices and `*` multiplies elements. Both can return plausible values; the operation's purpose and axes distinguish them.

4.3 Linear Combinations, Span, Basis and Rank

A linear combination of $v_1, \ldots, v_k$ is $\sum_j c_j v_j$. Their span contains all such combinations. They are linearly independent if only zero coefficients yield zero; otherwise they are dependent.

A basis is independent and spans the chosen space. The standard plane basis $(1,0)^\top, (0,1)^\top$ gives unique coordinates. A new basis changes coordinates while preserving the object.

For $\begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, columns $(1,2)^\top$ and $(2,4)^\top$ lie on the same line; the second is twice the first. The column space has dimension, or rank, 1. Columns $(1,0)^\top, (0,1)^\top$ span the plane and have rank 2. Column count measures storage; rank measures independent directions.

Rank equals both column-space and row-space dimension. A hundred stored factors can offer far fewer independent directions if many are linear combinations. The null space is $\{v : Av = 0\}$. Holdings $w, w + v$ with $Av = 0$ look identical under the mapping A. If A records only industry exposures, individual holdings and other risks can still differ.

4.4 Orthogonality and Projection

Nonzero vectors with $a^\top b = 0$ are orthogonal. A mutually orthogonal unit set is orthonormal. Perpendicularity depends on the inner product; zero column inner products in one sample do not establish economic independence or future uncorrelatedness.

Project $x \in \mathbb{R}^p$ onto nonzero $v \in \mathbb{R}^p$:

$$\text{proj}_v(x) = v\frac{v^\top x}{v^\top v}.$$

For unit v, this is $v(v^\top x)$. The residual $e = x - \text{proj}_v(x)$ satisfies $v^\top e = 0$, preparing the least-squares geometry in Chapter 8.

For $x = (3,2)^\top, v = (1,0)^\top$, the coordinate score is 3, the projected vector $(3,0)^\top$, and residual $(0,2)^\top$. A scalar score and a projected vector are different objects. The arrows lie on perpendicular horizontal and vertical axes.

4.5 Linear Systems, Inverses and Conditioning

In $Ax = b$, solve for x. A full-rank square A has a unique solution and inverse satisfying $A^{-1}A = AA^{-1} = I_p$, so algebraically $x = A^{-1}b$. A singular matrix has no inverse; the system may have no solution or multiple solutions.

Numerically prefer `solve(A,b)` over constructing the entire inverse for one right-hand side. Check the residual $Ax - b$, input contracts and financial constraints. For $A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, $b = (5,4)^\top$, substitution verifies $x = (2,1)^\top$. Multiple right-hand sides can reuse a factorization. A solver does not repair ill-conditioning.

Invertibility does not imply stability. The Euclidean condition number is

$$\kappa_2(A) = \|A\|_2\|A^{-1}\|_2 = s_{max}(A)/s_{min}(A),$$

where the norm is induced by Euclidean vector length and the s's are singular values. A tiny minimum singular value compresses a direction whose inverse mapping amplifies errors. This diagnoses relative sensitivity; it does not rank economic estimates by reliability.

5. Definitions

5.1 Symmetry and Definiteness

Definition: positive and semidefinite matrices

We use real matrices. A symmetric matrix has $A = A^\top$. It is positive definite (PD) if $v^\top Av > 0$ for every nonzero v; it is positive semidefinite (PSD) if $v^\top Av \geq 0$ for every v. PSD allows a nonzero zero-value direction.

Positive diagonals or a few positive portfolio quadratic forms cannot check every direction. For a real symmetric matrix, nonnegative eigenvalues are equivalent to PSD, strictly positive ones to PD. Floating-point tolerances must follow scale; rounding near -10^{-16} and a material negative eigenvalue $-.8$ are different findings.

5.2 Covariance Matrices

Begin with a supplied risk table ordered A, B on both axes:

$$\Sigma = \begin{bmatrix} .04 & .01 \\ .01 & .09 \end{bmatrix}.$$

Diagonal entries are individual variances, off-diagonals describe joint movement. You can compute with the table before deriving probability theory.

For a same-interval return vector R with finite second moments and $\mu = \mathbb{E}[R]$,

$$\Sigma = \mathbb{E}[(R-\mu)(R-\mu)^\top], \quad \Sigma_{ij} = \mathrm{Cov}(R_i, R_j).$$

R, μ are $p \times 1$; Σ is $p \times p$ with squared-return units. Expectation $\mathbb{E}$ is developed in the current Chinese Chapter 6, pending its English translation. Here this formula identifies symbols; calculations start from a supplied Σ.

Volatility $\sigma_i = \sqrt{\Sigma_{ii}}$ has return units. For nonzero volatilities, correlation is $\rho_{ij} = \Sigma_{ij}/(\sigma_i\sigma_j)$. Correlation keeps standardized joint movement and removes risk magnitude; a constant zero-volatility sequence has no usual correlation coefficient.

5.3 Eigenvalues and Spectral Decomposition

A nonzero q satisfying $Aq = \lambda q$ is an eigenvector with eigenvalue λ. Normalize its length as needed. A real symmetric matrix admits

$$A = Q\Lambda Q^\top, \qquad Q^\top Q = I_p.$$

Columns of Q are orthonormal eigenvectors; Λ is diagonal. A repeated-eigenvalue subspace has multiple valid bases; even a distinct-eigenvalue vector can reverse sign. NumPy `eigh` returns column eigenvectors and uses one triangle, so check symmetry independently.

5.4 SVD and PCA

Every real $X_c \in \mathbb{R}^{n\times p}$ has

$$X_c = UDV^\top.$$

For the reduced computational form, $r = \min(n, p)$: U is $n\times r$, D is $r\times r$, and $V^\top$ is $r\times p$. Singular values satisfy $s_1 \geq \cdots \geq s_r \geq 0$, and the corresponding columns of U, V are orthonormal. You may retain zero values or further truncate to actual rank. Rectangular matrices need no inverse for SVD.

PCA finds orthogonal directions of maximum projected training variance after centering. Directions are columns of V; scores are observation coordinates along them. Explained variance is another object. The covariance-PCA convention here does not automatically scale features.

Center by subtracting each training column mean:

$$X_c = X - \mathbf{1}_n \bar{x}^\top.$$

Standardization also divides by training standard deviations, removing original volatility magnitudes. For correlation-PCA, justify equal scaling and fit both means and scales only on training rows.

With k retained directions, V_k is $p \times k$, and scores $T = X_c V_k$ are $n \times k$. Projected points in original coordinates are $TV_k^\top$; add the training mean to reconstruct uncentered data. The project stores one direction per row: `components` is $k \times p$, and scores are `(X-center) @ components.T`.

For n_h held-out rows,

$$T_{held} = (X_{held} - \mathbf{1}_{n_h} \bar{x}_{train}^\top) V_k.$$

If trained with standardization, apply the frozen training scales too. Retain cutoff, feature order, center, scales and directions. Future rows enter the transform, never the past fit.

6. Derivations

6.1 Portfolio Variance from a Two-Asset Expansion

For fixed start-of-interval weights, $R_p = w^\top R$. Match frequency, interval and currency conventions, and determine weights before the random return. Random weights related to returns need conditioning or a joint model.

Linearity gives $\mathbb{E}[R_p] = w^\top \mu$, and

$$R_p - \mathbb{E}[R_p] = w^\top (R - \mu).$$

Squaring the scalar and moving deterministic weights outside expectation,

$$\begin{aligned} \mathrm{Var}(R_p) &= \mathbb{E}\{[w^\top (R-\mu)][(R-\mu)^\top w]\} \\ &= w^\top \mathbb{E}[(R-\mu)(R-\mu)^\top] w \\ &= w^\top \Sigma w. \end{aligned}$$

The shape is $(1 \times p)(p \times p)(p \times 1) = 1 \times 1$. For two assets,

$$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\Sigma_{12}.$$

Expand the four matrix cells to see both cross terms:

$$\begin{aligned} w^\top \Sigma w &= w_1(w_1\Sigma_{11} + w_2\Sigma_{21}) + w_2(w_1\Sigma_{12} + w_2\Sigma_{22}) \\ &= w_1^2\Sigma_{11} + w_1w_2\Sigma_{21} + w_1w_2\Sigma_{12} + w_2^2\Sigma_{22}. \end{aligned}$$

Symmetry combines the middle terms. Omitting them omits joint risk.

6.2 Why Covariance Is PSD

For every v,

$$v^\top \Sigma v = \mathbb{E}\{[v^\top (R-\mu)]^2\} \geq 0.$$

An expected square cannot be negative. If a nonzero combination is always zero, the covariance need not be PD. If B always returns twice A, $v = (2, -1)^\top$ gives a zero-change direction.

For $n \geq 2$ common complete rows centered by column,

$$\widehat{\Sigma} = X_c^\top X_c/(n-1), \qquad v^\top \widehat{\Sigma} v = \|X_c v\|_2^2/(n-1) \geq 0.$$

The denominator is the sample-covariance convention. This Gram-matrix proof requires every entry to come from the same X_c. Pairwise missing-data deletion can use different dates and lose this guarantee.

Centered columns sum to zero, so the sum of the n row vectors is zero and the rows are dependent. Thus $\mathrm{rank}(X_c) \leq \min(n-1, p)$. For $p \geq n$, sample covariance is necessarily rank deficient. Inverting it cannot create the missing information.

6.3 Variance Along Risk Axes

Write $\Sigma = Q\Lambda Q^\top$ and coordinates $z = Q^\top w$:

$$w^\top \Sigma w = z^\top \Lambda z = \sum_j \lambda_j z_j^2.$$

Each λ_j is the variance along a unit eigen-direction; z_j is the holding coordinate on it. These axes can change as estimates change. Calling the first one a market factor requires economic evidence.

6.4 Why PCA Takes the Largest Eigenvalue Direction

For unit v, centered scores $t = X_c v$ have mean zero and sample variance

$$t^\top t/(n-1) = v^\top \widehat{\Sigma} v, \qquad v^\top v = 1.$$

Write $v = Qa$. Orthogonality gives $\sum_j a_j^2 = 1$, so variance is $\sum_j \lambda_j a_j^2$, bounded by the largest eigenvalue. Putting all length on that direction attains the bound. The next direction must be orthogonal to the first and maximizes the remaining variance, continuing recursively.

SVD gives

$$\widehat{\Sigma} = VD^2V^\top/(n-1), \quad \lambda_j = s_j^2/(n-1), \quad \mathrm{EVR}_j = \lambda_j / \sum_\ell \lambda_\ell.$$

Explained variance ratio (EVR) is dimensionless; eigenvalues retain squared-return units. Scores $T_k = X_c V_k$ reconstruct

$$\widehat{X}_k = T_k V_k^\top + \mathbf{1}_n \bar{x}^\top.$$

All nonzero directions reconstruct the centered data. Fewer directions discard variation. For that same centered matrix, truncated SVD minimizes squared reconstruction error at the chosen rank; the error equals the sum of discarded squared singular values. This optimization has no labels, fills or future net returns in its objective.

7. Financial Interpretation

7.1 Diversification and Joint Movement

At asset volatilities 20%, 30% and correlation 1/6, covariance is .01. A 60/40 portfolio has positive joint-risk contribution, but imperfect co-movement gives volatility below the weighted 24%. Individual risks are insufficient; low historical correlation does not eliminate crisis dependence. Chapter 33 addresses stress scenarios.

7.2 Small Eigenvalues Can Amplify Errors

Solving $\Sigma x = \mu$ divides mean coordinates by risk eigenvalues. A tiny eigenvalue can turn a small expected-return change into a large position change. An apparent low-risk arbitrage direction may reflect short samples, duplicate assets or omitted costs.

Stable algorithms do not remove economic estimation error. Shrinkage, regularization and position limits can reduce sensitivity, while introducing bias and model choices that need out-of-sample checks. Chapters 8 and 21 develop these methods.

7.3 Appropriate Roles for PCA

PCA can summarize joint asset movement, diagnose near-duplicate columns and supply a candidate feature transform. Directions need separate units, budgets, shorting conventions, costs and signs before they define tradable positions.

PCA is unsupervised and uses no target labels. A high-variance direction may be common noise for a prediction task; a low-variance direction may contain target information. Covering 95% of training variance alone cannot establish trading value. Chapter 41 continues dimension reduction and model selection.

8. Numerical Examples

8.1 Two Assets: Each Risk Contribution

These supplied annual parameters are teaching constructions, not daily estimates or current market parameters:

$$\mu = (.05, .08)^\top, \quad \Sigma = \begin{bmatrix} .04 & .01 \\ .01 & .09 \end{bmatrix}, \quad w = (.6, .4)^\top.$$

Use annual simple expected returns, the same annual covariance interval, weights held at its start, no fees and no within-interval trades.

1. Expected return: $.6(.05) + .4(.08) = .062$, or 6.2%.
2. The row $w^\top\Sigma$ is $[.6(.04) + .4(.01), .6(.01) + .4(.09)] = [.028, .042]$.
3. Its dot product with weights is $.028(.6) + .042(.4) = .0336$.
4. The four original-cell contributions are .0144, .0024, .0024 and .0144, summing to .0336.
5. Volatility is $\sqrt{.0336} = .183303$, about 18.33%.

Calling .0336 a 3.36% volatility confuses variance with standard deviation. Omitting cross terms gives .0288. With unchanged asset volatilities and perfect positive correlation, covariance is .06 and portfolio volatility 24%. At zero correlation, variance is .0288 and volatility about 16.97%. These changes come from specified joint-movement assumptions, not measured allocation effects.

8.2 Rank-Deficient Systems

For

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \quad b = (3, 6)^\top,$$

both rows and columns are dependent and rank is 1. The sole independent equation is $x_1 + 2x_2 = 3$, with solutions $(3 - 2t, t)^\top$ for all real t. Null direction $(2, -1)^\top$ leaves the output unchanged.

Changing b to $(3, 7)^\top$ makes the rows inconsistent: twice a left side equal to 3 cannot equal 7. Least squares can fit approximately, but does not establish an exact solution. Chapter 8 develops its objective.

8.3 Eigenvalues, Invertibility and the Budget

For the Section 8.1 covariance,

$$\det(\Sigma - \lambda I_2) = (.04 - \lambda)(.09 - \lambda) - .01^2 = \lambda^2 - .13\lambda + .0035.$$

For a two-by-two matrix, the determinant is $ad - bc$; zero means no inverse. Here

$$\lambda_{1,2} = \frac{.13 \mp \sqrt{.0029}}{2} \approx .03807418, \ .09192582.$$

Both are positive, and the condition number is about 2.41438775. This supplied small matrix is not nearly singular; real estimation quality is a separate question.

As a linear-system exercise,

$$\Sigma^{-1} = \frac{1}{.0035} \begin{bmatrix} .09 & -.01 \\ -.01 & .04 \end{bmatrix}.$$

Multiply back: the upper-left entry is $(.09(.04) - .01^2)/.0035 = 1$, off-diagonals cancel, and the remaining diagonal is 1. Multiplying by μ gives $(1.05714286, .77142857)^\top$, sum 1.82857143. This fails a no-borrowing, fully invested two-asset budget. Solving the algebra does not supply an executable portfolio.

8.4 Three Rows: PCA and Reconstruction by Hand

Worked example: three-row PCA reconstruction

Use undated geometric data,

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}, \quad \bar{x} = (2, 4)^\top.$$

Centered rows are $(-1, -2), (0, 0), (1, 2)$, so

$$\widehat{\Sigma} = X_c^\top X_c / 2 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}.$$

Continued: the same complete example

Eigenvalues are 5 and 0. Choose $v_1 = (1,2)^\top/\sqrt{5}$, giving scores $(-\sqrt{5}, 0, \sqrt{5})^\top$, sample variance 5. Direction $(-2,1)^\top/\sqrt{5}$ has zero scores. One component explains 100% because the points lie exactly on a line; this does not imply real assets need only one factor.
The first row reconstructs as $-\sqrt{5}v_1^\top + (2,4) = (1,2)$. Reversing both direction and score signs leaves their product unchanged.

9. Python Implementation

Default Entry in the Accepted Reader Package

Follow the reader installation instructions and run from its root:

```
.venv/bin/python run_notebook.py notebooks/ch05_linear_algebra.ipynb
```

The default Notebook uses packaged shared logic without absent French inputs or downloads. New runs and figures use unique runs directories; the reader CLI uses universally unique identifiers (UUIDs) and rejects overwrites. Historical tests absent from the reader are original production-check references. Use the Notebook assertions. Optional French inputs require the data contract; this batch did not rerun them.

The shared PCA implementation takes only a training matrix and returns its mean, row-stored directions, explained variances, ratios and training scores. Require at least two rows and one column of finite real values. It neither fills NaN nor standardizes; zero total variance raises an error. The separate original-course route uses the script, Notebook and project environment:

```
.venv/bin/python tools/run_a2.py --chapter 5
```

Each following snippet is self-contained. A NumPy one-dimensional vector is not literally a two-dimensional column; `@` defines its contractions. Use `reshape(-1,1)` when an explicit column is needed.

```
import numpy as np

asset_returns = np.array([[0.02, -0.01], [-0.03, 0.04]])
weights = np.array([0.6, 0.4])
portfolio_returns = asset_returns @ weights
assert asset_returns.shape == (2, 2)
assert weights.shape == (2,)
assert portfolio_returns.shape == (2,)
assert np.allclose(portfolio_returns, [0.008, -0.002])
assert np.allclose(portfolio_returns, (asset_returns * weights).sum(axis=1))
print(portfolio_returns)
```

The rows are two supplied holding intervals, with decimal returns and A/B column order matched to weights. There is no signal or execution design, so the code checks multiplication rather than a backtest.

The second snippet verifies algebra, symmetry, spectrum and constraints. `eigh` returns ascending eigenvalues; PCA generally selects descending variance. The official interface documentation records that ordering and triangle convention; its previously read source scope is retained.

```
import numpy as np

covariance = np.array([[0.04, 0.01], [0.01, 0.09]])
expected_returns = np.array([0.05, 0.08])
weights = np.array([0.6, 0.4])
assert np.allclose(covariance, covariance.T)
eigenvalues, directions = np.linalg.eigh(covariance)
assert eigenvalues.min() > 0.0
assert np.allclose(directions.T @ directions, np.eye(2))
assert np.allclose(covariance @ directions, directions * eigenvalues)
variance = float(weights @ covariance @ weights)
assert np.isclose(variance, 0.0336)
assert np.isclose(weights @ expected_returns, 0.062)
solution = np.linalg.solve(covariance, expected_returns)
assert np.linalg.norm(covariance @ solution - expected_returns) < 1e-12
assert not np.isclose(solution.sum(), 1.0)
print(variance, np.sqrt(variance), eigenvalues, solution)
```

The third constructs covariance from one finite matrix and checks SVD against the spectrum. Its geometric rows have no timestamps or trade interpretation.

```
import numpy as np
from quant_course.math_tools import pca_fit_transform

samples = np.array([[1.0, 2.0], [2.0, 4.0], [3.0, 6.0]])
result = pca_fit_transform(samples, n_components=1)
centered = samples - result.center
sample_covariance = centered.T @ centered / (len(samples) - 1)
assert np.allclose(sample_covariance, [[1.0, 2.0], [2.0, 4.0]])
assert np.isclose(result.explained_variance[0], 5.0)
assert np.isclose(result.explained_variance_ratio[0], 1.0)
assert np.allclose(result.projection @ result.components + result.center, samples)
print(result.center, result.explained_variance, result.explained_variance_ratio)
```

The larger script constructs 252 daily rows for four fictitious assets with seed 20261005. Dates are timezone-aware Coordinated Universal Time (UTC) business-day labels without an exchange holiday calendar. Fit the first 180 rows, 2024-01-02 through 2024-09-09, and hold out 72. The inherited variance shares are about 56.96%, 20.19%, 14.36% and 8.49%. It creates no signal, fills or strategy-return result. Read actual outputs rather than changing the sample to obtain appealing percentages.

Later rows use `(X_later-center) @ components.T` with fitted parameters. Joining them to training before SVD changes the past fit. Missing-data policies belong in the training process; the shared implementation rejects missing values rather than silently zero-filling them.

10. Visualization

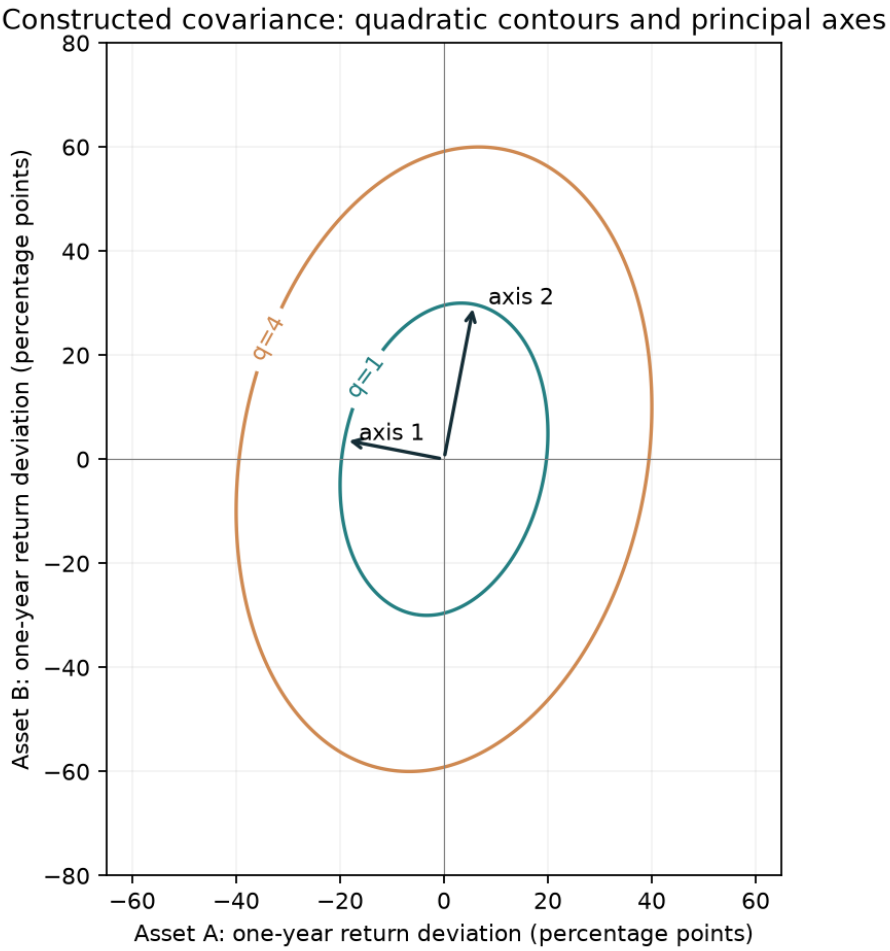


Figure 1: Quadratic contours and axes of a supplied covariance

The Section 8.1 annual matrix supplies this geometry. Axes show A/B annual return deviations in percentage points. Contours have $q = x^\top \Sigma^{-1} x = 1$ or 4; eigen-direction arrows have lengths equal to eigenvalue square roots. Inspect the long/short axes and tilt from positive covariance. Without a specified probability distribution, these ellipses are not automatically confidence regions, nor does a two-dimensional one-standard-deviation ellipse automatically contain 68% of observations. See the geometry script.

The left axes are first/second component scores in decimal-return units. Each point is a training observation, colored by row order from early to late. The first axis has larger variance because PCA selects it that way; this is not independent investment evidence, a price series or a wealth chart.

The right panel shows component index against training variance share. The first two explain about 77.15%, leaving about 22.85% of training variation discarded by a two-direction truncation. Held-out shares can differ. The plot uses centering and SVD, without per-feature standardization or whitening. Whitening would rescale component variance and is discussed later in Chapter 41.

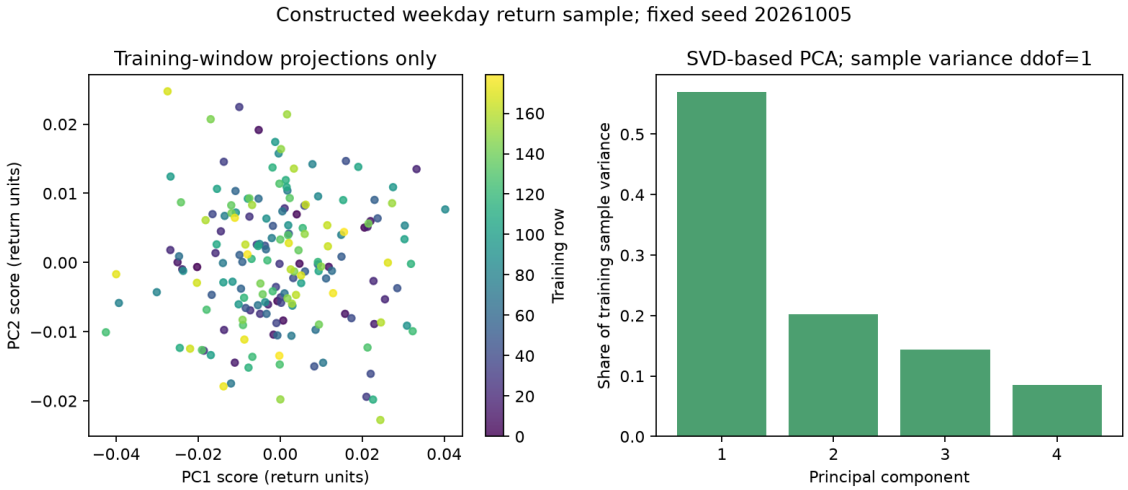


Figure 2: Constructed training PCA scores and variance shares

11. A Recorded Market-Snapshot Example

Chapter 2 already read the Kenneth R. French ten-industry monthly portfolio-return cache. Arranging it as months by industries lets you inspect common-sample covariance and PCA. These are published portfolio-return directions, not individual-stock risk models or executable industry strategies. A return-month label does not establish data availability at that month's closing auction.

The code uses complete common months from January 2010 through December 2019. The inherited executed first-three EVRs are 71.6043%, 7.5198% and 7.1064%; see the run record. They describe this recorded snapshot, fixed window and unstandardized convention. No 2020-or-later rows entered that fit. Nonetheless, the downloaded version may contain historical revisions and is insufficient to establish historical point-in-time (PIT) availability. Chapter 3 explains that version-time distinction.

```
from pathlib import Path
import numpy as np
import pandas as pd
from quant_course.math_tools import pca_fit_transform

path = Path("data/processed/french_10_industries_monthly.csv")
raw = pd.read_csv(path, dtype={"month": "string"})
monthly = raw.set_axis(pd.PeriodIndex(raw.pop("month"), freq="M"), axis="index")
train = monthly.loc["2010-01":"2019-12"].dropna(axis=0, how="any")
assert train.index.is_monotonic_increasing and train.index.is_unique
assert len(train) == 120
fit = pca_fit_transform(train.to_numpy(), n_components=3)
covariance = np.cov(train.to_numpy(), rowvar=False, ddof=1)
assert np.linalg.eigvalsh(covariance).min() >= -1e-12
assert np.isclose(fit.explained_variance.sum(), np.linalg.eigvalsh(covariance)[-3:].
    ↪ sum())
print("French snapshot: monthly published portfolio returns; not PIT prices")
print("training rows:", len(train), "features:", list(train.columns))
print("first three training EVR:", fit.explained_variance_ratio)
```

Common-sample `dropna` removes a whole month when any industry is missing. Research should report removed months and reasons, not only the final count. Here the fixed 120-row assertion prevents silently substituting another sample while keeping the same claims. PCA on uncentered price levels is not automatically a return-risk factor model.

12. Common Mistakes

12.1 Valid Dimensions with the Wrong Security Order

A two-entry dot product runs after swapping identities, but its meaning changes:

```
import numpy as np
import pandas as pd

returns = pd.Series({"A": 0.02, "B": -0.01})
weights = pd.Series({"B": 0.4, "A": 0.6})
wrong = float(returns.to_numpy() @ weights.to_numpy())
correct = float(returns @ weights.reindex(returns.index))
assert np.isclose(wrong, 0.002)
assert np.isclose(correct, 0.008)
print("positional mistake:", wrong, "aligned labels:", correct)
```

Freeze the column list before array operations. Training, risk, holdings and execution need the same security mapping. Permanent identifiers from Chapter 3 do not automatically enforce column order.

12.2 Pairwise Plausibility without Joint Consistency

Common mistake: pairwise plausibility without consistency

Consider the constructed candidate correlation matrix

$$C = \begin{bmatrix} 1 & .9 & .9 \\ .9 & 1 & -.9 \\ .9 & -.9 & 1 \end{bmatrix}.$$

Its pairwise correlations lie in $[-1, 1]$ and diagonal entries are 1, but eigenvalue $-.8$ makes it invalid. For $v = (-1, 1, 1)^\top$, $Cv = -.8v$ and $v^\top Cv = -2.4$. A variance cannot be negative. Separate windows or pairwise deletions can lose joint consistency. This is a deliberately invalid construction, not a claim about a market dataset. Investigate alignment, computation and units before choosing a model repair. Clipping all negative eigenvalues changes the risk model and must be recorded.

12.3 Calling a Successful Solve a Valid Portfolio

A small linear-system residual says little about budgets, leverage, sectors, shorting or costs. Section 8.3 is not a maximum-Sharpe portfolio: no risk-free return, budget normalization or feasible set has yet been specified.

12.4 Fitting PCA before a Time Split

A full-sample fit makes past features depend on future means and covariance. Unsupervised processing can still leak future information. Fit each training window and only transform its successor. Section 18 changes an extreme future row to test the frozen past mean.

12.5 Mixing Scales and Risk Units

Columns combining prices, dollar turnover, percentages and bp can have high variance simply because of their units. Standardization may help compare features but also removes original risk magnitudes. Choose covariance- or correlation-PCA for the question; do not rescale arbitrarily and keep interpreting the result in original return units.

12.6 Comparing Every Eigenvector Sign Literally

A direction and its negative are equivalent. Nearly equal eigenvalues can make individual vectors sensitive to perturbations while their span is more stable. Check subspaces, orthogonality, reconstruction and spectrum; identical loading signs are not the sole criterion.

13. Research Application

For a risk-panel study, specify currency, frequency, holding intervals and securities; use versions available by the training cutoff and a shared missing-data rule. Estimate covariance on the same complete sample. Check symmetry, units, minimum eigenvalue, rank and conditioning before evaluating weights.

For PCA, fit means/directions in training and project the next window with those past parameters. Save feature order, cutoff, versions, center, directions and random configuration. Compare principal subspaces across windows as well as signs. Chapters 19 and 41 later address changing structure.

Predeclare at least two ways the proposal can fail: large direction changes under tiny sample perturbations, or no improvement in out-of-sample risk prediction/downstream loss despite high training EVR. Compare equal weights, a single common-risk direction or existing economic factors rather than only complex models against one another.

This chapter hands off numerical diagnostics and risk structure, not investment performance. A backtest still needs signal, execution, holdings, costs and label timing as developed in Chapter 29.

14. Chapter Summary

Vectors, matrices and transposes encode multi-asset mappings with explicit shapes. Rank counts independent directions; invertibility determines unique square-system solutions; conditioning diagnoses error amplification. Orthogonality, projection and changes of basis explain risk and regression geometry.

Fixed-weight variance is $w^\top \Sigma w$ with all cross terms. Covariance is PSD but may be singular. Common complete samples preserve the Gram-matrix guarantee; short samples constrain rank. PCA centers past observations and selects orthogonal high-variance directions. Explained variance, prediction and trading feasibility require their respective checks.

Next, the current Chinese Chapter 6 develops probability, Chapter 8 applies projection to regression, and Chapters 9, 20 and 21 address estimated, constrained portfolio choices. Knowing the matrix objects prepares these problems without pretending to solve them already.

15. Exercises

5-E01. A panel has 120 months and ten industries. Give shapes of $X, w, Xw, X^\top X$, explain the axes and state why $X^\top X$ is not automatically sample covariance.

5-E02. Calculate the dot product of weights $(.6, .4)$ and returns $(2\%, -1\%)$. Swap the returns without changing weights. What changes, and why do both calculations run?

5-E03. Distinguish rank 1 from one stored column. For $A = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$, give a nonzero null vector and explain the missing inverse.

5-E04. Give a PSD but non-PD covariance, identify a nonzero zero-variance direction and explain why PSD does not imply independence of all assets.

5-E05. Use the annual covariance in Section 8.1 with equal weights. Calculate expected return, variance, volatility and correlation, including units.

5-E06. For centered rows $n = 20, p = 30$, explain why covariance is singular. If different pairs use different samples, can one Gram matrix still establish PSD?

5-E07. Section 8.4 retains 100% of sample variance with one component. Reverse its sign: what happens to scores, reconstruction and EVR? Does this prove future prediction?

5-N01: Identify Shapes. For $X = \begin{bmatrix} .02 & -.01 \\ -.03 & .04 \end{bmatrix}$, columns A/B, weights $(.6, .4)^\top$ reset at each interval start without fees, and $R_1 = (.02, -.01)^\top$, give shapes/meanings of X, w, Xw, $w^\top R_1$. Calculate both returns and specify covariance units and weight timing.

5-N02: Calculate Every Cell. Multiply $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, displaying the row-column dot product for all four cells. Explain why reversing multiplication needs a fresh calculation.

5-N03: Diagnose the Axis. What does this runnable code output? Change it to returns by date and assert the hand results.

```
import numpy as np

returns = np.array([[0.02, -0.01], [-0.03, 0.04]])
weights = np.array([0.6, 0.4])
wrong = (returns * weights).sum(axis=0)
print(wrong)
```

5-N04: Assess the Research Claim. A researcher stacks ten assets' full-period price levels, centers with the full-period mean and does PCA. They name the first direction a market-return factor, trade it and use high EVR as proof of strategy success. Identify at least three object/scale/timing errors, define a train/holdout process and state the evidence needed for trading claims.

16. Programming Exercises

5-P01. Diagnose a supplied real square matrix: shape, finite values, symmetry, eigenvalues and a scale-relative PSD check, rank and condition number. Test Σ, the rank-deficient example and Section 12.2's invalid correlation matrix. Reject asymmetry rather than silently fixing it.

5-P02. Center one complete training matrix and compute covariance. Use shared `pca_fit_transform` to verify full-component reconstruction, orthogonality and score variances. Retain one direction and compare squared reconstruction error with discarded squared singular values.

5-P03. Fit four training rows and transform one future row. Replace the future row by an extreme value: verify unchanged fit parameters, while showing that a leaky combined fit changes the mean. Record feature order and reject mismatched future column counts. Define all inputs locally.

17. Challenge Problems

5-C01. For near-duplicate assets with covariance $.04 \begin{bmatrix} 1 & .999 \\ .999 & 1 \end{bmatrix}$, compare linear solutions for means $(.05, .05)^\top$ and $(.051, .049)^\top$. Explain amplified holdings and propose remedies at three different levels. Merely switching to `solve` is insufficient.

5-C02. French monthly industry PCA gives high first-component EVR. A colleague claims it is the market factor and retaining it must improve Sharpe. Plan checks of economic naming, versions/timing, out-of-sample results, scales, risk and costs, and specify findings that would reject the claim.

18. Complete Solutions

18.1 Concepts and Hand Calculations

5-E01. $X : 120 \times 10$, $w : 10 \times 1$, $Xw : 120 \times 1$, $X^\top X : 10 \times 10$. Panel rows are months and columns industries; the Gram axes are both industries. Covariance also needs column centering and division by 119. An uncentered Gram matrix includes mean-level contributions.

5-E02. The correct return is $.6(.02) + .4(-.01) = .008$, .8%. After swapping returns it is $.6(-.01) + .4(.02) = .002$, .2%. Matching vector lengths only check shape, not identity; this is the same kind of positional mistake as Section 12.1.

5-E03. Stored columns and independent directions differ. The second column is twice the first, giving rank 1. $A(2, -1)^\top = 0$. If an inverse existed, multiplying that equality by it would force the nonzero vector to be zero, a contradiction.

5-E04. Take $\Sigma_0 = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$. For $v = (a, b)^\top$, $v^\top \Sigma_0 v = (a + 2b)^2 \geq 0$, establishing PSD. Direction $(2, -1)^\top$ has zero variance, so it is not PD. Perfectly related assets are compatible with PSD; orthogonality/zero correlation also do not generally establish probability independence, as Chapter 6 later explains.

5-E05. Annual expected return is .065, or 6.5%; variance is $.25(.04) + .25(.09) + .5(.01) = .0375$; volatility is $\sqrt{.0375} \approx 19.365\%$. Correlation is $.01/(.2(.3)) = 1/6$, dimensionless. Pair annual means with annual covariance.

5-E06. Centered rank is at most 19, leaving at least eleven zero-eigenvalue directions in a 30-asset covariance. Pairwise samples generally cannot be represented by one $X_c^\top X_c/(n-1)$. That loses the guarantee, not a theorem that every pairwise-deletion result is non-PSD.

5-E07. $v \to -v$ sends $t \to -t$, leaving $tv^\top$, its variance and EVR unchanged. The 100% arises from exact sample collinearity. No future targets or fees were used, so prediction and trading claims lack their evidence.

18.2 Programming Reference Implementations

5-P01. Tie the tolerance to spectral scale and report both it and the minimum eigenvalue. Diagnosis is separate from invertibility; valid PSD can be singular. The function refuses materially invalid directions rather than deleting them.

```
import numpy as np

def diagnose_symmetric_matrix(matrix, relative_tolerance: float = 1e-10) -> dict:
    raw = np.asarray(matrix)
    if raw.ndim != 2 or raw.shape[0] != raw.shape[1] or raw.shape[0] == 0:
        raise ValueError("require a non-empty square matrix")
    if not np.issubdtype(raw.dtype, np.number) or np.iscomplexobj(raw):
        raise TypeError("require real numeric values")
    values = raw.astype(float)
    if not np.isfinite(values).all():
        raise ValueError("require finite values")
    if not np.isfinite(relative_tolerance) or relative_tolerance <= 0:
        raise ValueError("tolerance must be positive and finite")
    scale = max(float(np.linalg.norm(values, ord=2)), np.finfo(float).tiny)
    if not np.allclose(values, values.T, rtol=relative_tolerance,
                       atol=relative_tolerance * scale):
        raise ValueError("input is not symmetric; do not silently symmetrize")
    eigenvalues = np.linalg.eigvalsh(values)
    tolerance = relative_tolerance * scale
    return {"eigenvalues": eigenvalues, "tolerance": tolerance,
            "psd_numerical": bool(eigenvalues.min() >= -tolerance),
            "pd_numerical": bool(eigenvalues.min() > tolerance),
            "rank": int(np.linalg.matrix_rank(values)),
            "condition_2": float(np.linalg.cond(values))}

covariance = np.array([[0.04, 0.01], [0.01, 0.09]])
singular = np.array([[1.0, 2.0], [2.0, 4.0]])
invalid = np.array([[1.0, 0.9, 0.9], [0.9, 1.0, -0.9], [0.9, -0.9, 1.0]])
assert diagnose_symmetric_matrix(covariance)["pd_numerical"]
assert diagnose_symmetric_matrix(singular)["psd_numerical"]
assert not diagnose_symmetric_matrix(singular)["pd_numerical"]
assert not diagnose_symmetric_matrix(invalid)["psd_numerical"]
assert np.isclose(diagnose_symmetric_matrix(invalid)["eigenvalues"][0], -0.8)
print(diagnose_symmetric_matrix(covariance))
```

Numerical PD depends on the tolerance and does not replace the exact definition. Even small asymmetry needs investigation and an explicit treatment rule; this teaching checker rejects it.

5-P02. Full components check reconstruction and variance conventions; a single component checks truncation error. The squared Frobenius norm $\|A\|_F^2 = \sum_{ij} A_{ij}^2$ sums squared errors in common units.

```
import numpy as np
from quant_course.math_tools import pca_fit_transform

samples = np.array([[1.0, 2.0, 0.0], [2.0, 0.0, 1.0],
                    [4.0, 3.0, 2.0], [3.0, 5.0, 1.0]])
full = pca_fit_transform(samples, n_components=3)
centered = samples - full.center
covariance = centered.T @ centered / (len(samples) - 1)
assert np.allclose(full.projection @ full.components + full.center, samples)
assert np.allclose(full.components @ full.components.T, np.eye(3))
assert np.allclose(np.var(full.projection, axis=0, ddof=1), full.explained_variance)
assert np.isclose(full.explained_variance.sum(), np.trace(covariance))
short = pca_fit_transform(samples, n_components=1)
reconstructed = short.projection @ short.components + short.center
squared_error = float(np.square(samples - reconstructed).sum())
singular_values = np.linalg.svd(centered, compute_uv=False)
assert np.isclose(squared_error, np.square(singular_values[1:]).sum())
print("discarded reconstruction error:", squared_error)
```

Large-unit columns dominate unscaled squared reconstruction error. The objective remains computable, but its research meaning requires a scale decision.

5-P03. Transform with frozen centers/directions, without refitting. Column counts can be checked, while security names/order must be saved and checked by the caller as model metadata.

```
import numpy as np
from quant_course.math_tools import pca_fit_transform

training = np.array([[0.01, -0.02], [0.02, 0.01],
                     [-0.01, 0.03], [0.0, -0.01]])
future = np.array([[0.03, 0.02]])
fit = pca_fit_transform(training, n_components=2)
saved_center = fit.center.copy()
saved_components = fit.components.copy()

def transform_past_fit(values, center, components):
    array = np.asarray(values, dtype=float)
    if array.ndim != 2 or array.shape[1] != len(center):
        raise ValueError("feature count differs from training")
    if not np.isfinite(array).all():
        raise ValueError("non-finite future features")
    return (array - center) @ components.T

scores = transform_past_fit(future, fit.center, fit.components)
changed_future = np.array([[100.0, -100.0]])
changed_scores = transform_past_fit(changed_future, fit.center, fit.components)
assert not np.allclose(scores, changed_scores)
assert np.array_equal(fit.center, saved_center)
assert np.array_equal(fit.components, saved_components)
leaky_fit = pca_fit_transform(np.vstack([training, changed_future]), n_components=2)
assert not np.allclose(leaky_fit.center, saved_center)
try:
    transform_past_fit(np.zeros((1, 3)), fit.center, fit.components)
except ValueError:
```

```
    pass
else:
    raise AssertionError("mismatched feature count accepted")
print("past center:", fit.center, "leaky center:", leaky_fit.center)
```

The assertions show this extreme future input did not change this fit. They do not prove every caller uses only past rows; callers still implement the time split and persist cutoff/version.

18.3 Challenge Reasoning and Scoring

5-C01. The common direction $(1,1)^\top/\sqrt{2}$ has eigenvalue .07996; the spread $(1,-1)^\top/\sqrt{2}$ has .00004. Their ratio is 1999. Equal means have zero spread coordinate and produce about $(.625313, .625313)^\top$. A small mean difference is amplified in the spread direction, giving $(25.625313, -24.374687)^\top$.

```
import numpy as np

covariance = 0.04 * np.array([[1.0, 0.999], [0.999, 1.0]])
first = np.linalg.solve(covariance, np.array([0.05, 0.05]))
second = np.linalg.solve(covariance, np.array([0.051, 0.049]))
assert np.isclose(np.linalg.cond(covariance), 1999.0)
assert np.allclose(second - first, [25.0, -25.0])
print(first, second)
```

At the data level, check duplicates, synchronous returns and common-sample length. At the estimation level, assess mean shrinkage and covariance regularization, recording bias. At the decision level, impose budgets, leverage, asset limits and turnover costs. Stable decomposition and residual checks support these steps without replacing them. Ten points: directions/eigenvalues 3, solutions/amplification 3, three-level remedies 3, rejection of automatic arbitrage claims 1.

5-C02. Compare component scores with independently defined market excess returns using matched currency, frequency and model. High correlation alone does not establish identity. Record snapshot version and return month; distinguish retrospective analysis from actual PIT trading, which a current revised snapshot cannot reproduce by itself.

Fit preprocessing only inside each training window. Compare covariance-PCA, correlation-PCA and simple economic baselines under predeclared downstream prediction or risk loss. In the next window, assess subspace stability, risk forecast error and target prediction rather than reusing training EVR. A strategy also needs executable asset mapping, budgets, delays, costs and net returns. Only in-sample gains, strong scale dependence, a few years driving the result or no improvement after costs cannot support the strong claim.

Ten points: 2 for economic naming and comparison, 2 for versions and timing, 2 for training and later-window checks, 2 for scales and stability, and 2 for investment constraints, costs and falsification. “PCA reduces dimensions, so the model is better” earns no method credit.

18.4 Complete Answers to the Existing N-Series

5-N01. X is 2×2, intervals by A/B; w is 2×1, start-of-interval holdings; Xw is 2×1. Returns are $.6(.02) + .4(-.01) = .008$ and $.6(-.03) + .4(.04) = -.002$. $w^\top R_1$ is a scalar .008. Weights and returns are dimensionless and covariance has squared-return units. Choosing weights after observing returns brings future information into the calculation.

5-N02.

$$AB = \begin{bmatrix} 1(5)+2(7) & 1(6)+2(8) \\ 3(5)+4(7) & 3(6)+4(8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}.$$

Reversing the operation requires rechecking shapes and meaning. Both happen to be square, so BA exists but equals $\begin{bmatrix} 23 & 34 \\ 31 & 46 \end{bmatrix}$, a different result.

5-N03. Axis zero sums dates, yielding weighted totals by asset $(-.006, .012)$, not return by date. Sum asset columns or use matrix multiplication:

```
import numpy as np

returns = np.array([[0.02, -0.01], [-0.03, 0.04]])
weights = np.array([0.6, 0.4])
by_date = (returns * weights).sum(axis=1)
via_matrix = returns @ weights
assert by_date.shape == (2,)
assert np.allclose(by_date, [0.008, -0.002])
assert np.allclose(via_matrix, by_date)
assert np.allclose((returns * weights).sum(axis=0), [-0.006, 0.012])
```

Axis one preserves rows/dates. The assertions check shape, hand results, matrix-product equivalence and the wrong-axis output.

5-N04. Price levels are not returns. A direction describes variation in its actual inputs and cannot automatically be named a return factor. Full-period centering leaks holdout observations into fitting; differing price scales also affect covariance-PCA directions. EVR does not measure forecasting, execution or net profit.

Specify objects/order, derive matching-frequency returns from information-available prices, and split training/holdout by time. Fit center and any justified standard deviations only on training data. Freeze directions, center, columns and cutoff, then transform held-out returns. Predeclare a risk or trading task, compare simple benchmarks in later windows and check stability. A trading claim additionally needs executable mapping, timing, fees and net results. Training EVR without these results cannot support it.

19. Further Reading

- **Must know:** MIT 18.06 syllabus for systems, spaces, projections, eigenvalues and positive matrices before the final SVD topic.
- **Should know:** NumPy SVD, solve and eigh: output shapes, direction storage and order.
- **Should know:** scikit-learn PCA: centering without automatic feature scaling, component/variance attributes and solver conditions.
- **Advanced:** Current Chinese Chapters 21 and 41 for shrinkage, constraints and dimension reduction, pending their English batches.

20. References and Reading Boundaries

The exposition and small examples are original teaching material, not a page-by-page textbook rewrite. Historical source verification is dated 2026-10-05; the Chinese source audit record states actual reading and limits. No new external reading, market download or market execution was performed for this translation.

MIT_LA and MIT_LA_SYLL support course-topic mapping, without claiming every video/lecture was read. NP_SVD supports decomposition, reduced shapes, ordering and reconstruction; NP_EIGH supports real-symmetric input, column directions, ascending eigenvalues and the triangle convention. NP_SOLVE and NP_COND support full-rank square systems, errors and the two-norm condition number. SK_PCA supports centering/scaling and component/variance attributes; the project uses explicit shared SVD and does not claim equivalence to every online library default. The exact official URLs are retained in the source audit record and the reading links above.

Chapter 6 — Probability

> Thread: express uncertainty as events and distributions that you can check, then update your judgment with conditional information. Simulation output does not replace checking assumptions. Prerequisites: Chapter 4 on calculus and Chapter 5 on linear algebra and covariance. After this chapter, you can compute conditional probabilities and moments, identify common distributions, explain the assumptions of the law of large numbers (LLN) and central limit theorem (CLT), and implement basic stochastic processes.

The terminology connects this chapter to course materials, NumPy/SciPy documentation and program output. When a symbol first appears, its meaning and units connect the formula to the quantity you compute.

1. Learning goals

After reading, you should be able to:

1. Define probability using a sample space and events, and update judgments with conditional probability, total probability and Bayes' rule.
2. Distinguish discrete and continuous random variables, and use probability mass functions (PMFs), probability density functions (PDFs) and cumulative distribution functions (CDFs) correctly.
3. Compute expectations, variances, covariances and conditional expectations, and explain their roles in portfolio risk and prediction.
4. Identify parameters and moment conditions for Bernoulli, binomial, normal, lognormal, Poisson and Student's t distributions.
5. State the sampling assumptions of LLN and CLT, and explain how temporal dependence, heavy tails and parameter uncertainty change conclusions.
6. Generate sample means, random walks and Poisson counting processes with fixed seeds, and distinguish constructed simulations from market evidence.

You have completed the chapter when you can work through Section 8's low-base-rate Bayes example by hand, identify nonexistent distribution moments, run Section 9's script, and explain what each figure demonstrates and what it does not.

2. Why a quant needs probability

Quantitative research asks questions involving randomness: what is the probability of a loss after a signal? How large is the risk that several assets fall together? Why does a finite-sample average differ from the population mean? Can a counting model approximate the number of arriving events?

Probability supplies a common language for these questions, but a model output is not itself a fact. A probability of 0.02 describes an event under a specified model and information set. It

does not guarantee exactly one occurrence in every fifty trials, or imply that model parameters are known without error. Separate these questions:

- **Path randomness:** the next outcome remains uncertain even with parameters specified.
- **Parameter uncertainty:** probabilities, means, volatilities and intensities come from finite-sample estimates; their true values remain unknown.
- **Model uncertainty:** the chosen distribution or process may fail to describe the target object.

Low-base-rate events can mislead intuition in risk monitoring. A detector with high sensitivity may still produce mostly false positives. Evaluate an alert using the event's prior probability. The chapter demonstrates this with constructed numbers that you can calculate by hand, then extends the same conditioning logic to returns, processes and simulations.

3. Intuition

Imagine independently drawing a red or blue card from a bag each day, with fixed red probability p. Which card appears is path randomness. If you have observed only a few days and do not know the red-card proportion, p also carries estimation uncertainty. Distinguish the two layers: fixing $p = 0.3$ in a simulation displays only the first.

The bag analogy separates individual outcomes from repeated trials. Financial returns may also exhibit volatility clustering, common shocks and temporal dependence; the analogy does not establish independence of returns.

Conditioning narrows the population you examine. Among days with a positive alert, ask what fraction actually contains the event, rather than looking only at the detector's hit rate when events occur. Bayes' rule places the prior, hit rate and false-positive rate in one denominator.

Consider a symmetric random walk with one-unit up or down steps. Each increment has population mean zero, and its sample average approaches zero under independence. Yet the cumulative position's variance grows with the number of steps, and a path can move far from zero. Convergence of average increments does not imply convergence of the cumulative position. LLN concerns sample averages under repeated trials, not convergence of a financial price path to a constant. CLT describes the approximate distribution of a standardized sample mean, not normality of individual-period returns.

Every numerical experiment here constructs samples from specified parameters and a pseudorandom number generator (PRNG). A fixed seed makes output reproducible in the same environment; it does not turn simulation into market data or establish that parameters suit real assets.

4. Mathematical foundations

4.1 Probability and events

Definition: probability models and events

Let Ω be the sample space of possible outcomes, $\mathcal{F}$ the collection of events we may discuss (an event σ-algebra), and $\mathbb{P}$ a probability measure. An event $A \in \mathcal{F}$ is a set of outcomes. Probability satisfies:

$$\mathbb{P}(A) \geq 0, \qquad \mathbb{P}(\Omega) = 1, \qquad \mathbb{P}\left(\bigcup_{j=1}^{\infty} A_j\right) = \sum_{j=1}^{\infty} \mathbb{P}(A_j) \quad \text{if } A_j \text{ are pairwise disjoint.}$$

Ω denotes all outcomes, and A_j a countable collection of disjoint events. $\mathbb{P}$ takes values in $[0, 1]$ and has no physical units. The axioms imply $\mathbb{P}(\varnothing) = 0$, $\mathbb{P}(A^c) = 1 - \mathbb{P}(A)$ and $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$. You may add union probabilities directly only when $A \cap B = \varnothing$.

4.2 Conditional probability and updating

When $\mathbb{P}(B) > 0$, the conditional probability of A given that B occurred is defined by:

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

A, B are events. The denominator is the probability mass selected by the conditioning event, and the ratio is dimensionless. Interchanging A, B generally changes the result. Independence requires $\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B)$; whenever the conditional probability exists, this is equivalent to $\mathbb{P}(A \mid B) = \mathbb{P}(A)$.

If $B_1, \ldots, B_k$ form a mutually exclusive and exhaustive partition, total probability gives

$$\mathbb{P}(A) = \sum_{j=1}^{k} \mathbb{P}(A \mid B_j)\mathbb{P}(B_j).$$

Substitution into the conditional-probability definition yields Bayes' rule:

$$\mathbb{P}(B_i \mid A) = \frac{\mathbb{P}(A \mid B_i)\mathbb{P}(B_i)}{\sum_{j=1}^{k} \mathbb{P}(A \mid B_j)\mathbb{P}(B_j)}.$$

$\mathbb{P}(B_i)$ is the prior, $\mathbb{P}(A \mid B_i)$ the likelihood, and $\mathbb{P}(B_i \mid A)$ the posterior after observing A. All are dimensionless probabilities. The denominator is the total probability of observing A across all possible states.

Conditional probability asks how common A is among cases where B has been observed. It does not establish that B causes A. A causal effect compares outcome distributions under intervention on B and without intervention on B, using an identification argument such as random assignment or control of confounding. Knowing $\mathbb{P}(A \mid B)$ alone does not supply an intervention effect.

4.3 Random variables and distributions

A random variable $X : \Omega \to \mathbb{R}$ maps outcomes to real numbers. Its cumulative distribution function (CDF) is

$$F_X(x) = \mathbb{P}(X \le x).$$

x has the same units as X. F_X is dimensionless, nondecreasing and right-continuous, with limiting values 0 and 1. For any $a < b$, $\mathbb{P}(a < X \le b) = F_X(b) - F_X(a)$.

For a discrete X, the probability mass function (PMF) is $p_X(x) = \mathbb{P}(X = x)$ and sums to 1 over the possible values. If a continuous X has a probability density function (PDF) f_X, then

$$F_X(x) = \int_{-\infty}^{x} f_X(u)\,du, \qquad \mathbb{P}(a < X \le b) = \int_a^b f_X(u)\,du.$$

u, x have the same units as X. Density f_X has inverse-variable units $1/X$, so integration yields a dimensionless probability. A continuous variable assigns zero probability to an individual point; density height is not probability.

Fix the Section 3 bag at 10 cards: 3 red and 7 blue. Replace and mix the cards after each draw. Two successive draws form one ordered trial record; encode red as success 1 and blue as failure 0. Repeating the mechanism produces another record. The model specifies independent draws with fixed success probability $p = 0.30$. Use two independent Bernoulli trials to connect these objects. Event A means both draws succeed; event B means at least one succeeds. Random variable X maps a record to its success count. The table lists each record and its probability; an event-column value of 1 means occurrence, and 0 means nonoccurrence:

Outcome ω	Outcome probability	Event A	Event B	Count X
00	$0.7 \times 0.7 = 0.49$	0	0	0
01	$0.7 \times 0.3 = 0.21$	0	1	1
10	$0.3 \times 0.7 = 0.21$	0	1	1
11	$0.3 \times 0.3 = 0.09$	1	1	2

The sample space is $\Omega = \{00, 01, 10, 11\}$; events are its subsets. The four probabilities sum to 1. An outcome probability is the probability of the singleton set $\{\omega\}$. Conditioning retains the three rows in B and then selects A within them: $\mathbb{P}(A \mid B) = 0.09/(0.21 + 0.21 + 0.09) = 3/17$. Grouping rows by X yields $\mathbb{P}(X = 0) = 0.49$, $\mathbb{P}(X = 1) = 0.42$ and $\mathbb{P}(X = 2) = 0.09$. The next section uses this distribution table to calculate expectation and variance.

4.4 Expectation, variance and covariance

When the moment exists, expectation is the probability-weighted long-run average. The discrete and continuous expressions are

$$\mathbb{E}[X] = \sum_x x p_X(x), \qquad \mathbb{E}[X] = \int_{-\infty}^{\infty} x f_X(x)\, dx.$$

X and its mean share units. Variance measures squared deviations from the mean; standard deviation has the same units as X:

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2.$$

Covariance describes the direction in which two variables deviate together from their means:

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)].$$

Covariance has units equal to the units of X times those of Y. When both variances are finite and strictly positive, correlation standardizes covariance and lies in $[-1, 1]$. Define correlation by

$$\mathrm{Corr}(X, Y) = \frac{\mathrm{Cov}(X, Y)}{\sigma_X \sigma_Y},$$

Here σ_X, σ_Y are the respective standard deviations, with units matching X, Y. Numerator and denominator units cancel.

If X, Y are independent with finite second moments, their covariance is zero; the converse does not hold. Section 12.5 uses $Y = X^2$ to show zero correlation with complete dependence.

Return to the two Bernoulli trials. The success-count distribution is $(0, 0.49), (1, 0.42), (2, 0.09)$. Thus

$$\mathbb{E}[X] = 0(0.49) + 1(0.42) + 2(0.09) = 0.60, \qquad \mathbb{E}[X^2] = 0^2(0.49) + 1^2(0.42) + 2^2(0.09) = 0.78,$$

$$\mathrm{Var}(X) = 0.78 - (0.60)^2 = 0.42.$$

First obtain probabilities from the outcomes, then weight them according to the definitions. This connects the experiment, PMF, expectation and variance in one calculation.

For a vector R of returns on n assets with fixed weights, portfolio return is $R_p = w^\top R$, with variance

$$\mathrm{Var}(R_p) = w^\top \Sigma w = \sum_{i=1}^{n} \sum_{j=1}^{n} w_i w_j \,\mathrm{Cov}(R_i, R_j).$$

w_i are dimensionless capital weights, and each entry of Σ is the covariance of the corresponding returns. With returns in decimals, variance has squared decimal-return units. Omitting cross terms and looking only at individual volatilities loses information about diversification or common movements.

4.5 Conditional expectation

Conditional expectation given event B is $\mathbb{E}[X \mid B]$. Conditional expectation $\mathbb{E}[X \mid \mathcal{G}]$ given information set $\mathcal{G}$ is a random variable using only information in $\mathcal{G}$. Its units match X. Total expectation and total variance give

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}(X \mid \mathcal{G})],$$

$$\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid \mathcal{G})] + \mathrm{Var}(\mathbb{E}[X \mid \mathcal{G}]).$$

The first term is average randomness within states; the second is variation between state-conditional means. Reporting only the first misses uncertainty in state means.

4.6 Process and convergence notation

Independent and identically distributed (i.i.d.) observations $X_1, \ldots, X_n$ are mutually independent and have the same distribution. Their sample mean is $\bar{X}_n = n^{-1} \sum_{i=1}^{n} X_i$. Convergence notions have different strengths: convergence in probability is $\xrightarrow{p}$, almost sure convergence is $\xrightarrow{a.s.}$, and convergence in distribution is $\xrightarrow{d}$. These symbols are not interchangeable.

Weak white noise (WN) is a second-order concept: $\mathbb{E}[X_t] = 0$, a finite variance constant over time, and zero autocovariance at every nonzero lag. A zero-mean i.i.d. sequence with finite variance is white noise. White noise itself does not require independence, so it is not equivalent to i.i.d. Chapter 16 develops processes with zero autocovariance and remaining dependence.

A stochastic process is a family of random variables indexed by time. A random walk accumulates increments. A Markov process makes the future conditionally independent of earlier history given the current state. A Poisson process counts arrivals over time. Brownian motion is a standard continuous-time diffusion with continuous paths. Section 6 supplies their definitions and assumptions.

5. Definitions and common distributions

5.1 Bernoulli and binomial distributions

A Bernoulli random variable $X \sim \text{Bernoulli}(p)$ takes only 0 or 1, with $\mathbb{P}(X = 1) = p$. Its mean is p and variance $p(1-p)$. Across n independent trials with the same success probability, the total $K = \sum_{i=1}^{n} X_i$ has a binomial distribution:

$$\mathbb{P}(K = k) = \binom{n}{k} p^k (1-p)^{n-k}, \quad k = 0, \ldots, n.$$

n is the dimensionless number of trials, k the number of successes, and p a dimensionless probability. Mean is np and variance $np(1-p)$. A fixed trial count, independence and equal success probabilities are model assumptions, not consequences of a variable's name.

5.2 Normal, lognormal and Student's t distributions

For a normal distribution $X \sim N(\mu, \sigma^2)$, μ has the same units as X, $\sigma > 0$ is standard deviation in those units, and variance is σ^2. The standard normal has mean 0 and variance 1. Its support is the full real line. It can approximate certain aggregated errors, but its tails still decay at an exponential-of-a-square rate.

First define Y as a dimensionless growth multiplier. If $Z = \ln Y \sim N(\mu, \sigma^2)$, then $Y = e^Z$ is lognormal and $Y > 0$. Here μ, σ are the mean and standard deviation of log growth. The multiplier's expectation is $e^{\mu+\sigma^2/2}$ and its median e^μ; the larger mean reflects right skew. For a price V measured in currency, choose a positive reference $V_0 > 0$ with matching units, write $V = V_0 Y$ and $Z = \ln(V/V_0)$. Then $\mathbb{E}[V] = V_0 e^{\mu+\sigma^2/2}$, and price variance equals multiplier variance times V_0^2. Do not take the logarithm of a dimensional price directly.

Student's t has degrees-of-freedom parameter $\nu > 0$ and standard-form center 0. Its mean exists only for $\nu > 1$ and then equals 0. Variance exists only for $\nu > 2$ and equals $\nu/(\nu - 2)$. For $1 < \nu \leq 2$, mean exists but finite variance does not; for $\nu \leq 1$, even the mean does not exist. Heavier tails than the normal do not make t a description of every heavy-tailed process.

5.3 Poisson distribution

A Poisson random variable $N \sim \text{Poisson}(\lambda)$ describes an event count over an interval in a counting process, with probabilities

$$\mathbb{P}(N = k) = e^{-\lambda} \frac{\lambda^k}{k!}, \quad k = 0, 1, \ldots.$$

$\lambda \geq 0$ is the expected count over that interval. Its numerical value is dimensionless; "events" labels the count. Mean and variance both equal λ. With interval length Δt, a homogeneous Poisson process has $\lambda = \rho \Delta t$, where ρ has units events per unit time. Independent increments, stationary increments and constant rate define this model; justify the constant-rate assumption for the application.

Distribution	Parameters and support	Mean and variance	Uses and limits

5.4 Distribution reference table

Distribution	Parameters and support	Mean and variance	Uses and limits
Bernoulli	$p \in [0,1]$; $\{0,1\}$	$p,\ p(1-p)$	One binary outcome; no automatic independence over time
Binomial	$n \in \mathbb{N}, p \in [0,1]$; $\{0,\dots,n\}$	$np,\ np(1-p)$	Fixed number of independent equal-probability trials
Normal	$\mu \in \mathbb{R}, \sigma > 0$; $\mathbb{R}$	μ, σ^2	Aggregated errors or asymptotic approximation; no guaranteed tail fit
Lognormal	$\ln Y \sim N(\mu, \sigma^2)$; $Y > 0$	$e^{\mu+\sigma^2/2}, (e^{\sigma^2}-1)e^{2\mu+\sigma^2}$	Positive multiplicative quantities; not a single-period return that can be negative
Poisson	$\lambda \geq 0$; nonnegative integers	λ, λ	Interval counts; assumptions include equal mean and variance
Student's t	$\nu > 0$; $\mathbb{R}$	Mean 0 for $\nu > 1$; variance $\nu/(\nu-2)$ for $\nu > 2$	Heavy tails at finite degrees of freedom; low degrees can lack moments

SciPy distribution objects use their own application programming interface (API) parameter names. In particular, lognormal shape=sigma and scale=exp(mu) correspond to $Y = e^Z,\ Z \sim N(\mu, \sigma^2)$ here. Check parameterization before using a library function.

5.5 Uniform and exponential distributions: simulation and waiting times

A uniform distribution $U \sim \text{Uniform}(a, b)$ has density $1/(b-a)$ on $a < b$ and zero outside. Equal-length subintervals have equal probability. Its mean is $(a+b)/2$ and variance $(b-a)^2/12$. For example, $U \sim \text{Uniform}(0,1)$ gives $\mathbb{P}(0.2 < U < 0.5) = 0.3$. Uniform random numbers can drive simulations of other distributions; this does not make returns uniformly distributed.

For an exponential distribution $T \sim \text{Exponential}(\rho)$, $\rho > 0$ is the arrival rate. Density is $\rho e^{-\rho t}$ for $t \geq 0$, survival probability is $\mathbb{P}(T > t) = e^{-\rho t}$, mean is $1/\rho$ and variance $1/\rho^2$. T has time units, while ρ and the density have inverse-time units. Rate and mean waiting time are different parameters: SciPy expon uses scale $1/\rho$.

If $N(t)$ is a homogeneous Poisson process with rate $\rho > 0$, the first arrival's waiting time satisfies $T_1 > t$ if and only if $N(t) = 0$. Hence

$$\mathbb{P}(T_1 > t) = \mathbb{P}(N(t) = 0) = e^{-\rho t}.$$

This connects the count distribution to the waiting-time distribution. Further,

$$\mathbb{P}(T_1 > s + t \mid T_1 > s) = \frac{e^{-\rho(s+t)}}{e^{-\rho s}} = e^{-\rho t}, \quad s, t \geq 0.$$

This is memorylessness. It holds in this model; it does not establish the absence of seasonality or clustering in actual order arrivals. At $\rho = 0.7$ events/hour, mean waiting time is about 1.428571 hours. The probability of waiting more than 5 hours is about 0.0301974, equal to the probability of zero arrivals in 5 hours.

6. Derivations and assumptions

6.1 Deriving Bayes' rule from definitions

The conditional-probability definition gives $\mathbb{P}(B_i \mid A) = \mathbb{P}(B_i \cap A)/\mathbb{P}(A)$. The multiplication rule for intersections gives $\mathbb{P}(B_i \cap A) = \mathbb{P}(A \mid B_i)\mathbb{P}(B_i)$, and total probability over the partition gives $\mathbb{P}(A) = \sum_j \mathbb{P}(A \mid B_j)\mathbb{P}(B_j)$. Substitute the last two expressions into the first to obtain Bayes' rule. All conditional probabilities must be defined, and the observed event A must have positive total probability.

6.2 Bernoulli and binomial moments

For $X \in \{0, 1\}$, $\mathbb{E}[X] = 0(1-p) + 1p = p$ and $X^2 = X$. Thus

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = p - p^2 = p(1-p).$$

Let $K = \sum_{i=1}^{n} X_i$. Linearity of expectation does not require independence, so $\mathbb{E}[K] = np$. Independence eliminates cross covariances, giving $\mathrm{Var}(K) = \sum_i \mathrm{Var}(X_i) = np(1-p)$. With correlated trials, the mean can remain the same, but variance must include all covariance terms.

The binomial formula selects k success positions among n positions in $\binom{n}{k}$ arrangements. Each arrangement has probability $p^k(1-p)^{n-k}$. Adding the mutually exclusive arrangement probabilities gives the PMF.

6.3 Lognormal expectation

If $Z \sim N(\mu, \sigma^2)$, the normal moment-generating function is $\mathbb{E}[e^{tZ}] = e^{t\mu + t^2\sigma^2/2}$. Setting $t = 1$ yields $\mathbb{E}[Y] = \mathbb{E}[e^Z] = e^{\mu + \sigma^2/2}$; setting $t = 2$ yields $\mathbb{E}[Y^2] = e^{2\mu + 2\sigma^2}$. Therefore

$$\mathrm{Var}(Y) = e^{2\mu + 2\sigma^2} - e^{2\mu + \sigma^2} = (e^{\sigma^2} - 1)e^{2\mu + \sigma^2}.$$

μ, σ are log-scale parameters, and the exponent is dimensionless. For a variable measured in currency, normalize by a reference unit before taking logs, then restore units to its mean and variance.

6.4 Law of large numbers

Assumptions: two laws of large numbers

LLN states that a sample average approaches the population expectation under suitable assumptions. If X_i are i.i.d. and $\mathbb{E}|X_i| < \infty$, the strong law of large numbers (SLLN) gives $\bar{X}_n \xrightarrow{a.s.} \mu$, where $\mu = \mathbb{E}X_i$. With the additional assumption of finite variance σ^2,

$$\mathbb{E}[\bar{X}_n] = \mu, \qquad \mathrm{Var}(\bar{X}_n) = \frac{\sigma^2}{n}.$$

Chebyshev's inequality gives $\mathbb{P}(|\bar{X}_n - \mu| \geq \epsilon) \leq \sigma^2/(n\epsilon^2)$. For any $\epsilon > 0$, the bound tends to zero, establishing convergence in probability. This short derivation uses finite variance; the i.i.d. sufficient condition for SLLN can be as weak as a finite absolute first moment.

6.5 Central limit theorem

The classical i.i.d. CLT gives, for $0 < \sigma^2 < \infty$,

$$\frac{\sqrt{n}(\bar{X}_n - \mu)}{\sigma} \xrightarrow{d} N(0,1).$$

n is sample size, μ the population mean and σ its standard deviation. The standardized quantity is dimensionless. The theorem concerns the **standardized sample mean** as sample size grows, not the distribution of raw observations X_i. Zero autocovariance or weak white noise alone does not justify this classical i.i.d. theorem. Finite-sample approximation depends on skewness, tails and sample size; discrete observations do not become continuous because of CLT. One proof route studies characteristic functions: finite variance gives a second-order expansion near zero, and independence makes the standardized sum's characteristic function the nth power of that expansion evaluated at $t/\sqrt{n}$. Its limit is $e^{-t^2/2}$, the standard-normal characteristic function. This explains the normal limit's origin without replacing the theorem's assumptions.

For a weakly stationary sequence with finite second moments, sample-mean variance is

$$\mathrm{Var}(\bar{X}_n) = \frac{1}{n^2}\left[n\gamma_0 + 2\sum_{k=1}^{n-1}(n-k)\gamma_k\right],$$

Here $\gamma_k = \mathrm{Cov}(X_t, X_{t-k})$ and γ_0 is marginal variance. Units are X^2. Nonzero covariances prevent simply copying σ^2/n. Dependent sequences may satisfy an LLN or CLT, but need additional conditions involving stationarity, ergodicity, mixing or moment/covariance decay. Different theorem versions use different conditions; identifying data as a time series does not establish them.

6.5a Why zero autocovariance is insufficient: a shared-scale counterexample

Complete counterexample: uncorrelated observations, nonnormal means

Let a shared random scale A take values 1 and 2 with probability 1/2 each. Draw it once at the start of the whole sequence and keep it fixed. Let ε_t be i.i.d. standard-normal innovations independent of A, and define the dimensionless teaching sequence $X_t = A\varepsilon_t$. Each observation has mean 0 and variance $\mathbb{E}[A^2] = (1+4)/2 = 2.5$. For distinct s, t, $\mathrm{Cov}(X_s, X_t) = \mathbb{E}[A^2]\mathbb{E}[\varepsilon_s\varepsilon_t] = 0$. The sequence therefore meets the second-order conditions for weak white noise.

All observations in a sequence nevertheless share one scale. Observing larger squared values changes your judgment of the other observations' volatility. A direct check gives $\mathrm{Cov}(X_s^2, X_t^2) = \mathbb{E}[A^4] - (\mathbb{E}[A^2])^2 = (1+16)/2 - 2.5^2 = 2.25 > 0$. Zero linear covariance leaves this dependence intact.

The fourth moment is $\mathbb{E}[Z^4]$. For a standard normal, the moment-generating function is $M_Z(t) = e^{t^2/2}$, with fourth derivative $M_Z^{(4)}(t) = (t^4+6t^2+3)e^{t^2/2}$. Hence $\mathbb{E}[Z^4] = M_Z^{(4)}(0) = 3$. If $Y = \sqrt{v}Z$, where v is the variance, then $\mathbb{E}[Y^4] = v^2\mathbb{E}[Z^4] = 3v^2$. We use this identity to calculate the mixture's fourth moment.

For any sample size n, conditional on A, $\sqrt{n}\bar{X}_n \sim N(0, A^2)$. Standardizing by the marginal standard deviation $\sqrt{2.5}$ gives a mixture: half its mass comes from $N(0, 0.4)$ and half from $N(0, 1.6)$. This mixture distribution has variance 1 but fourth moment $\frac{1}{2}[3(0.4)^2+3(1.6)^2] = 4.08$, whereas the standard normal's fourth moment is 3. Its distribution does not change with n, so it does not approach the standard normal. Sample-mean variance is still exactly $2.5/n$: a correct variance formula for the standard error and a valid normal interval require different checks. This shared-scale construction is not a market model. It shows why zero autocovariance cannot replace independence in the classical i.i.d. CLT.

6.6 Heavy-tail limits

Heavy tails usually mean slower tail-probability decay than a normal or exponential benchmark; definitions differ. Student's t provides a computable example: no finite variance exists for $\nu \le 2$, so inserting sample standard deviation into a classical finite-variance CLT argument lacks justification. With an even lower tail index, mean may also fail to exist. You can still calculate a sample mean, but it may not estimate a finite population parameter, and standard errors or normal intervals may fail. Check moment conditions before choosing a limit theorem, robust method or tail model. Histogram appearance alone is insufficient.

6.7 From discrete to continuous processes

A symmetric random walk uses independent ϵ_i taking values ± 1 with probability 1/2 each, and position $S_n = \sum_{i=1}^{n} \epsilon_i$. Then $\mathbb{E}S_n = 0$ and $\mathrm{Var}(S_n) = n$. Shrink time steps to $1/n$ and positions by $1/\sqrt{n}$; the heuristic expression is

$$W_t^{(n)} = \frac{1}{\sqrt{n}} \sum_{i=1}^{\lfloor nt \rfloor} \epsilon_i, \qquad \mathrm{Var}(W_t^{(n)}) = \frac{\lfloor nt \rfloor}{n} \to t.$$

Under appropriate conditions, a functional CLT relates the scaled random-walk path distribution to Brownian motion. This chapter uses variance scaling for intuition, without proving that theorem.

Standard Brownian motion $(W_t)_{t\geq 0}$ starts at $W_0 = 0$, has continuous paths and independent increments over disjoint time intervals. Its increment law is $W_t - W_s \sim N(0, t-s)$ and depends only on interval length, giving stationary increments. t uses a chosen time unit; W_t has units $\sqrt{\text{time}}$ and satisfies $\text{Var}(W_t) = t$. These define a process rather than establish a fit to asset returns. Rigorous random-walk convergence, stochastic integration and Itô's formula are left to Chapter 44.

Donsker's invariance principle supplies a precise entry point to the random-walk limit. If increments ϵ_i are i.i.d. with mean 0 and variance 1, set $S_m = \sum_{i=1}^{m} \epsilon_i$. Over a fixed finite horizon, shrinking time steps to $1/n$ and positions by $1/\sqrt{n}$ makes the path $W_n(t) = S_{\lfloor nt \rfloor}/\sqrt{n}$ converge in distribution to standard Brownian motion as n increases. Symmetric ± 1 steps meet these increment assumptions. Later courses supply the formal function-space definition and proof; here you identify the input, scaling and target process.

For stochastic calculus, Brownian motion must also be defined relative to a filtration $(\mathcal{F}_t)$. $\mathcal{F}_t$ represents information available by time t. Adaptation to $\mathcal{F}_t$ means that W_t is measurable using information then available. Future increments $W_t - W_s$ must also be independent of past information $\mathcal{F}_s$. Before Itô integration, learn filtrations, adapted processes, quadratic variation, integrals of simple predictable processes and the Itô isometry; then study Itô's formula and stochastic differential equations. Chapter 44 develops these formally. Here the process definition and scaling provide the entry point.

6.8 Total variance, portfolio variance and the Poisson limit

Let $m = \mathbb{E}[X \mid \mathcal{G}]$ and $\mu = \mathbb{E}[X]$. Decompose the deviation as $X - \mu = (X - m) + (m - \mu)$. Expand the square and take expectation. The cross term vanishes because $\mathbb{E}[X - m \mid \mathcal{G}] = 0$ and $m - \mu$ depends only on $\mathcal{G}$. Therefore

$$\text{Var}(X) = \mathbb{E}[(X-m)^2] + \mathbb{E}[(m-\mu)^2] = \mathbb{E}[\text{Var}(X \mid \mathcal{G})] + \text{Var}(m).$$

Every term has units X^2. Apply the same square-expansion argument to portfolio returns: with $R_p = w^\top R$,

$$\text{Var}(R_p) = \mathbb{E}\Big[(w^\top(R - \mathbb{E}R))^2\Big] = w^\top \mathbb{E}\Big[(R-\mathbb{E}R)(R-\mathbb{E}R)^\top\Big]\, w = w^\top \Sigma w.$$

w is dimensionless and Σ has squared-return units. These equalities require finite second moments.

The Poisson distribution also arises as a limit of rare independent events. Partition an interval of length T into m cells. In each cell, let one event occur independently with probability $\rho T/m$, temporarily excluding multiple events per cell. The total is approximately $K_m \sim \text{Binomial}(m, \rho T/m)$. Fix k and take $m \to \infty$:

$$\mathbb{P}(K_m = k) = \binom{m}{k} \left(\frac{\rho T}{m}\right)^k \left(1 - \frac{\rho T}{m}\right)^{m-k} \longrightarrow e^{-\rho T} \frac{(\rho T)^k}{k!}.$$

ρ is measured in events/time, T in time, and ρT is the dimensionless expected count. The limit depends on independent cells and the rare-event construction; it does not establish a Poisson arrival process in reality.

6.9 Markov and Poisson processes

The discrete-time Markov property states that the current state summarizes the history needed to predict the future:

$$\mathbb{P}(X_{t+1} \in A \mid X_0, \ldots, X_t) = \mathbb{P}(X_{t+1} \in A \mid X_t).$$

A is a set of states. This restricts conditional distributions; it does not claim that yesterday alone causes today's state. A finite-state, time-homogeneous Markov chain uses transition matrix P with entries $P_{ij} = \mathbb{P}(X_{t+1} = j \mid X_t = i)$ and row sums 1. Time homogeneity means these probabilities do not change with t.

A counting process $N(t)$ takes nonnegative integer values, is nondecreasing and right-continuous, and has $N(0) = 0$. A homogeneous Poisson process counts at rate $\rho \geq 0$ in events/time. Increments over disjoint intervals are independent, with

$$N(t) - N(s) \sim \text{Poisson}(\rho(t-s)), \quad 0 \leq s < t.$$

Consequently $\mathbb{E}[N(t)] = \text{Var}(N(t)) = \rho t$. Independent and stationary increments are model definitions. Clustering, intraday seasonality or changing intensity in data makes a homogeneous process an assumption to examine, not a default truth.

For $\rho > 0$, an equivalent construction uses i.i.d. interarrival times with distribution Exponential(ρ). $\rho = 0$ means no arrivals, so the waiting time is not a finite exponential variable. Exponential memorylessness corresponds to Poisson independent, stationary increments. Section 5.5 derives the first waiting time from the zero-count probability. Markov chains, Poisson processes and Brownian motion each have explicit state or increment conditions; their names cannot replace those conditions.

7. Financial interpretation

7.1 Conditioning requires an information time

In trading and risk research, known information must refer to its availability time. Let $\mathcal{F}_t$ be information available at t, and write future-return conditional mean as $\mathbb{E}[R_{t+1} \mid \mathcal{F}_t]$. Do not define the conditioning group from future outcomes. Align signals, release times, ingestion times and return intervals first; probability formulas cannot repair look-ahead bias.

7.2 Covariance links individual and portfolio risk

Changing correlation changes portfolio variance even when individual asset risks remain fixed. With weights w_1, w_2, portfolio variance includes $2w_1w_2\,\mathrm{Cov}(R_1, R_2)$. Diversification comes from imperfect comovement, not asset count alone. Frequency, overlapping intervals, missingness, tails and estimation error affect sample covariance; a historical estimate does not automatically describe the future.

7.3 Distributions are approximations, not asset labels

- Bernoulli/binomial models can describe default indicators or success counts over fixed periods, but require a default definition, observation window and treatment of common shocks.
- Normal distributions ease sums and derivations, while returns may exhibit skew, heavy tails, volatility clustering or extreme jumps.
- Lognormal distributions suit strictly positive multiplicative quantities, such as terminal values in certain price models. This does not make simple returns lognormal.
- Poisson provides a count-model benchmark. Examine overdispersion, clustering and intensity changes as possible failures.
- Student's t adds tail mass at finite degrees of freedom. If fitted degrees of freedom are too low, some moments and the metrics using them cease to exist.

7.4 State what a process model preserves

Random walks accumulate increments to illustrate path randomness and mean/variance growth. Markov processes compress historical information; specify states, time variation in transitions and parameter estimation. Poisson processes simulate counts; examine independent increments and stable intensity. Brownian motion uses continuous paths and Gaussian increments as an entry point to continuous-time models. A path looking like a price chart does not establish any of these as a price-generating process.

8. Numerical Example

All following numbers are teaching constructions for hand calculations and code checks, not market frequencies, strategy performance or risk-parameter estimates.

8.1 Low-base-rate alerts: Bayes updating

Complete example: Bayes updating for a low-base-rate alert

Let the event prior be 0.005, sensitivity (a positive alert given an event) 0.95, and false-positive rate given no event 0.05. Then

$$\mathbb{P}(E \cap +) = 0.005 \times 0.95 = 0.00475,$$

$$\mathbb{P}(+) = 0.00475 + (1 - 0.005) \times 0.05 = 0.00475 + 0.04975 = 0.05450,$$

$$\mathbb{P}(E \mid +) = \frac{0.00475}{0.05450} = 0.0871559633 \approx 8.7156\%.$$

E denotes the event and $+$ a positive alert. Sensitivity is 95%, yet event probability after a positive is about 8.72%, because nonevents dominate the population. Choose the conditional direction from the question.

Express the same model as frequencies among 200,000 objects. These are model-expected counts obtained from specified probabilities, not guaranteed counts in one realized sample:

Actual state	Positive alert	Negative alert	Total
Event	950	50	1,000
No event	9,950	189,050	199,000
Total	10,900	189,100	200,000

The positive-column denominator is $950 + 9,950 = 10,900$, giving posterior $950/10,900 = 19/218 \approx 0.08715596$. The table expands the Bayes denominator cell by cell: it contains both true and false positives.

8.2 Bernoulli counts and portfolio covariance

Across $n = 20$ independent trials with $p = 0.30$, the success total $K \sim \text{Binomial}(20, 0.30)$ has

$$\mathbb{E}[K] = 20(0.30) = 6, \qquad \text{Var}(K) = 20(0.30)(0.70) = 4.2.$$

Variance is measured in squared success-count units.

Complete example: a four-state portfolio and conditional variance

Construct two assets' daily decimal returns across four equally likely states: (R_A, R_B) takes values $(0.1, 0.1)$, $(-0.1, -0.1)$, $(0.1, -0.1)$ and $(-0.1, 0.1)$. Both assets have mean 0, variance 0.01 and zero covariance. Equal capital weights produce returns $0.1, -0.1, 0, 0$, with mean 0 and variance $(0.01 + 0.01 + 0 + 0)/4 = 0.005$. Keeping only the first two perfectly aligned states leaves each marginal variance at 0.01 but raises covariance to 0.01 and equal-weight portfolio variance to 0.01. Individual volatilities are unchanged while portfolio risks differ; the numbers isolate the covariance term's role.

Use the same construction for total expectation and variance. Let C denote the first two aligned states, with $\mathbb{P}(C) = \mathbb{P}(C^c) = 1/2$, and let $\mathcal{G} = \sigma(\mathbf{1}_C)$ represent knowing only whether C occurred. Within C, returns are $0.1, -0.1$, so conditional mean is 0 and variance 0.01.

Continued: the same complete example

Within C^c, returns are $0, 0$, so both conditional moments are 0. Thus $\mathbb{E}[R_p] = \frac{1}{2}(0) + \frac{1}{2}(0) = 0$, and

$$\mathrm{Var}(R_p) = \mathbb{E}[\mathrm{Var}(R_p \mid \mathcal{G})] + \mathrm{Var}(\mathbb{E}[R_p \mid \mathcal{G}]) = \tfrac{1}{2}(0.01) + \tfrac{1}{2}(0) + 0 = 0.005,$$

This agrees with the direct four-state calculation.

8.3 Lognormal mean and median

Let one-period log growth be $Z \sim N(0.08, 0.20^2)$ and the gross growth factor $G = e^Z$. Median is $e^{0.08} \approx 1.0833$ and expectation $\mathbb{E}[G] = e^{0.08+0.20^2/2} = e^{0.10} \approx 1.1052$. If G is terminal wealth per unit of initial wealth, its mean exceeds its median. This does not guarantee a net return and accounts for neither costs, bankruptcy, jumps nor parameter error.

8.4 Poisson counts

With constructed rate 0.7 events/hour, the 5-hour count parameter is $\lambda = 0.7 \times 5 = 3.5$. The homogeneous Poisson model gives mean and variance 3.5 over that interval. A realized path with 2 or 6 events is consistent with this: theoretical moments describe repeated sampling, not numbers every path must attain.

8.5 Student's t, random walks and Brownian scaling

A standard t distribution with $\nu = 4$ has mean 0, variance $4/(4-2) = 2$ and standard deviation $\sqrt{2}$. Reducing degrees of freedom to 1 removes both mean and variance. Finite degrees of freedom alone do not guarantee a given moment.

A symmetric random walk S_n uses $+1$ or -1 per step. After $n = 4$ steps, $\mathbb{E}[S_4] = 0$ and $\mathrm{Var}(S_4) = 4$, giving standard deviation 2 step units. For standard Brownian motion measured in hours, at $t = 4$ hours, $\mathrm{Var}(W_4) = 4$ hours and standard deviation $2\sqrt{\text{hours}}$. This converts process-definition scales; it does not estimate asset-price volatility.

8.6 A two-state Markov chain

Use the constructed transition matrix

$$P = \begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}$$

to describe one-step probabilities between states A, B; each row sums to 1. From initial row-vector probabilities $\pi_0 = (0.5, 0.5)$, one step gives $\pi_1 = \pi_0 P = (0.6, 0.4)$ and two steps $\pi_2 = \pi_1 P = (0.66, 0.34)$. Probabilities and state labels are dimensionless. Before applying this constructed matrix to markets, define states knowable in real time and estimate and examine transition stability.

9. Python Implementation

Default entry point in the accepted reader package

From the accepted I3 reader package root, create the isolated environment described in its README, then run:

```
.venv/bin/python run_notebook.py notebooks/ch06_probability.ipynb
```

This notebook calls the packaged shared implementation, reads no omitted French data and downloads nothing. Each execution writes a unique UUID subdirectory under runs and prints its location. Old production tests and the historical course-root commands below belong to the original working project; the reader's default route is the package-root notebook and its assertions. Optional historical French paths follow the reader's data contract and were not rerun in this batch.

Historical course-root route

`ch06_probability.py` implements Bayes calculations, Bernoulli/binomial moments and four fixed-seed constructed simulations. It uses local NumPy Generators without changing global random state. Out-of-range or nonfinite probabilities and undefined conditioning events raise errors. Simulations use no financial data or global shuffle.

Run from the course-project root:

```
.venv/bin/python code/ch06_probability.py --seed 641 --output-dir figures/a2
.venv/bin/python -m pytest tests/test_ch06_probability.py -q
```

Historical tests cover the Bayes denominator, closed-form moments, reproducibility with the same seed, moments of LLN/CLT constructed samples, random-walk steps, Poisson monotonicity and moments, and writing all four figures. The recorded validation environment was Python 3.14.7, NumPy 2.5.3 and SciPy 1.18.1; the course project's .venv determines dependency versions.

The next self-contained snippet checks Section 8.1. Inputs, imports and assertions are included; tolerance handles floating-point rounding:

```
import math

prior = 0.005
sensitivity = 0.95
false_positive_rate = 0.05
joint_event_and_positive = prior * sensitivity
positive_rate = joint_event_and_positive + (1 - prior) * false_positive_rate
posterior = joint_event_and_positive / positive_rate

assert math.isclose(positive_rate, 0.0545, rel_tol=0.0, abs_tol=1e-12)
assert math.isclose(posterior, 0.0871559633027523, rel_tol=0.0, abs_tol=1e-12)
print(f"P(event | positive) = {posterior:.4%}")
```

```
import numpy as np
from scipy import stats

rate_per_hour = 0.7
waiting = stats.expon(scale=1.0 / rate_per_hour)
no_arrival_in_five_hours = waiting.sf(5.0)
zero_count_probability = stats.poisson.pmf(0, mu=rate_per_hour * 5.0)
assert np.isclose(no_arrival_in_five_hours, zero_count_probability)
assert np.isclose(waiting.mean(), 1.0 / rate_per_hour)
assert np.isclose(waiting.sf(3.0 + 2.0) / waiting.sf(3.0), waiting.sf(2.0))
uniform_probability = stats.uniform.cdf(0.5) - stats.uniform.cdf(0.2)
assert np.isclose(uniform_probability, 0.3)
print("constructed waiting time mean, hours", waiting.mean())
print("constructed P(no arrival in 5 hours)", no_arrival_in_five_hours)
```

The waiting-time example explicitly uses hours; other process figures use specified constructed time units. These examples have no market sampling frequency, signal time or trade time because they contain no security records. Script process time is dimensionless teaching time. A future application to real signals must define the event window, information cutoff, execution time and return interval separately.

10. Visualization

The chapter script generated the following constructed-simulation figures. Axes carry labels, and titles state the seeds.

10.1 LLN sample-mean paths

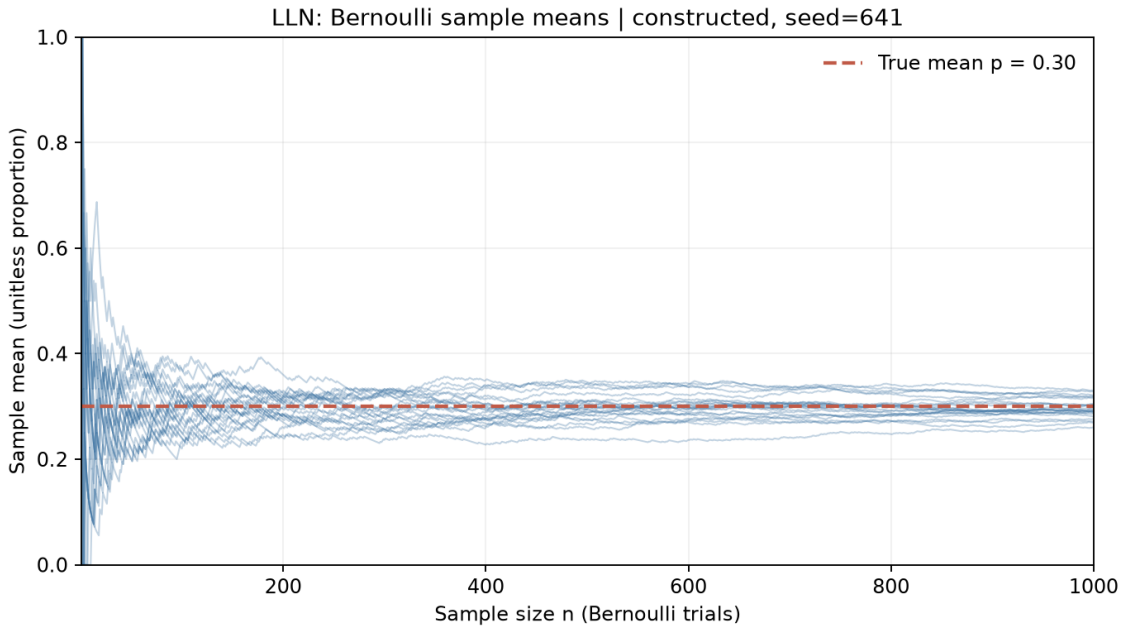


Figure 1: LLN sample-mean paths

The horizontal axis is Bernoulli trial count $n = 1, \ldots, 1000$; the vertical axis is the sample mean for each of 24 paths. The dashed line marks constructed population mean $p = 0.30$. Paths fluctuate widely early and narrow around p later. Each line is a cumulative mean of independently constructed Bernoulli observations. The figure illustrates convergence; it does not prove it.

10.2 CLT standardized sample means

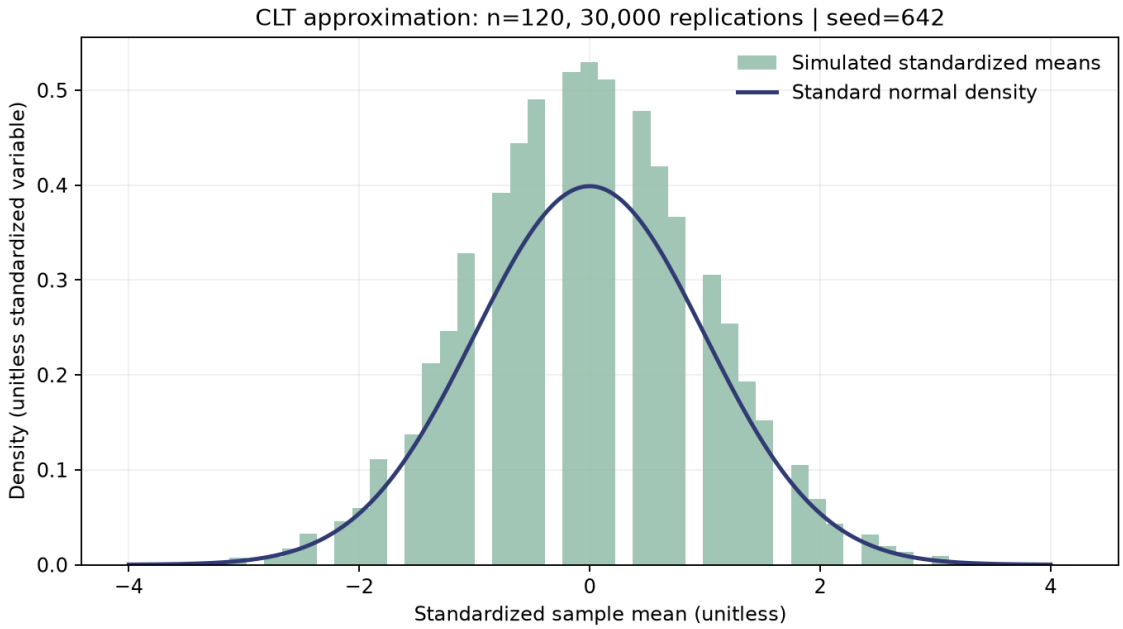


Figure 2: CLT standardized sample means

The horizontal axis is the standardized Bernoulli sample mean at $n = 120$. A density histogram uses 30,000 constructed replications; the curve is standard-normal density. At finite n the variable remains discrete, and the curve is a limiting approximation.

Each replication draws $n = 120$ independent Bernoulli($p = 0.30$) observations and computes $Z_{n,m} = (\bar{X}_{n,m} - p)/\sqrt{p(1-p)/n}$. Index $m = 1, \ldots, 30{,}000$ labels independent sample replications, not observations within a sample. The 30,000 constructed statistics check whether code follows the specified model and show its approximate sampling distribution. They do not prove CLT or establish i.i.d. real returns.

10.3 Symmetric random walks

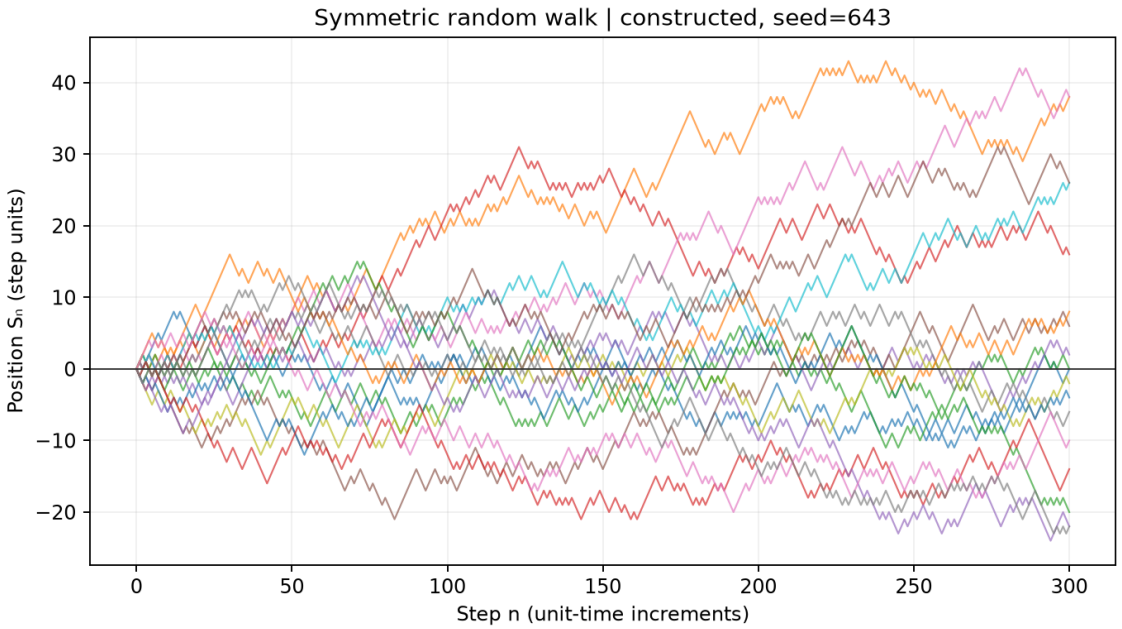


Figure 3: Symmetric random walks

The horizontal axis runs from step 0 to 300 and the vertical axis records cumulative position. Each of 18 paths uses independent $+1/-1$ increments. The figure illustrates path randomness without fitting prices or returns.

10.4 Poisson counting processes

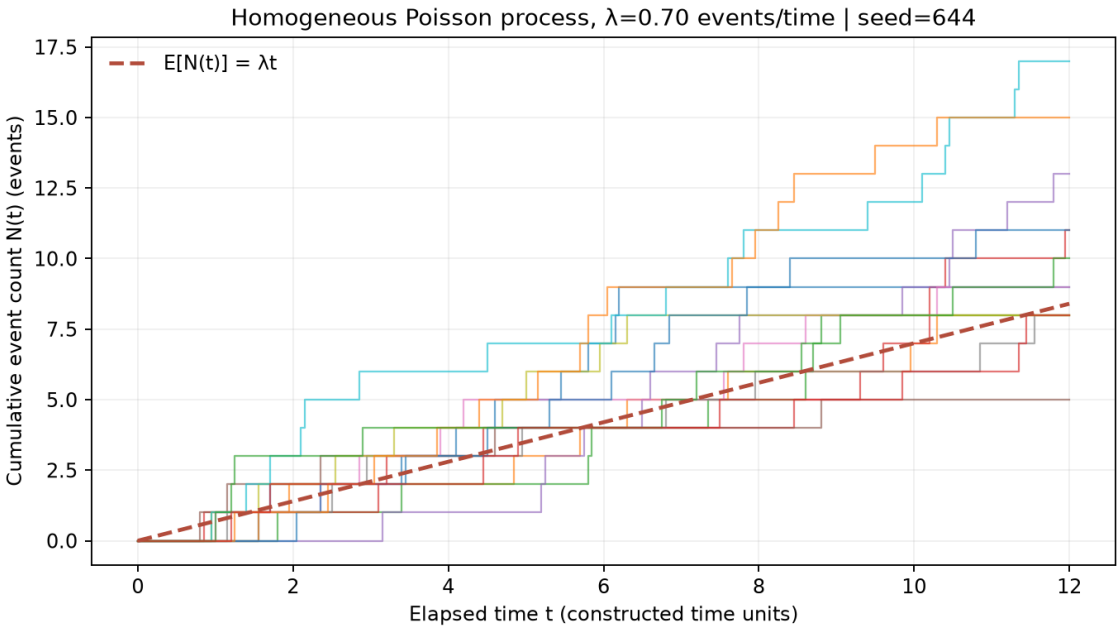


Figure 4: Poisson counting processes

The horizontal axis covers 0 to 12 constructed time units and the vertical axis cumulative count. Fourteen step paths use independent Poisson increments on 240 equal-width cells of size 0.05. The dashed line is theoretical mean ρt, with $\rho = 0.70$ events/time. Every path takes integer values and is nondecreasing.

Section 9 gives reproduction commands and seeds. These four figures illustrate definitions, convergence trends and count structure; they do not establish the same process assumptions in real data.

11. Real Market Example

This chapter downloaded or tested no real security, order, default or return data. It therefore reports no market frequencies, fitted parameters or in-/out-of-sample results, and the alert example is not empirical evidence. Official course notes and distribution APIs support definitions and methods, not conclusions that a particular market follows a distribution.

For a future real-data case, first choose a sourced, licensed dataset. Record issuer/venue, event definition, sample period, frequency, download time, version and checksum. An alert study must prevent test-period base rates, label availability or revised values leaking into the signal time. Examine interarrival distributions, intraday seasonality, clustering and intensity changes before accepting a count model. Until those checks are complete, the model remains a hypothesis to test.

12. Common Mistakes

12.1 Confusing sensitivity with event probability after an alert

$\mathbb{P}(+ \mid E)$ and $\mathbb{P}(E \mid +)$ reverse the conditional direction. Calculate them separately using Section 8.1 inputs:

```
prior = 0.005
sensitivity = 0.95
false_positive_rate = 0.05

sensitivity_value = sensitivity
posterior_value = prior * sensitivity / (
    prior * sensitivity + (1 - prior) * false_positive_rate
)
print(f"P(positive | event) = {sensitivity_value:.2%}")
print(f"P(event | positive) = {posterior_value:.4%}")
```

Outputs are 95.00% and about 8.7156%. Reporting only the first leaves the reliability of a positive alert unanswered.

12.2 Reading density height as probability

Density carries units; interval integration gives probability, while a specified point has zero probability here. The following example uses the standard normal:

```
from scipy.stats import norm

density_at_zero = norm.pdf(0.0)
probability_near_zero = norm.cdf(0.01) - norm.cdf(-0.01)
print(f"density at zero = {density_at_zero:.6f}")
print(f"P(-0.01 < Z < 0.01) = {probability_near_zero:.6f}")
```

density_at_zero is about 0.398942 and interval probability about 0.007979. The former is not $P(Z = 0)$; a continuous standard normal assigns zero probability to any specified single point.

12.3 Applying finite-variance CLT to heavy tails without checking moments

Student's t with 1 degree of freedom has no finite mean or variance. SciPy moment interfaces return nonfinite values for nonexistent moments; function conventions may represent them as infinity or NaN:

```
import numpy as np
from scipy.stats import t

mean, variance = t.stats(df=1, moments="mv")
assert not np.isfinite(mean)
assert not np.isfinite(variance)
print("df=1: population mean and variance do not exist")
```

You can calculate a finite-sample mean, but cannot standardize by a nonexistent population standard deviation and claim a guaranteed classical CLT normal approximation.

12.4 Redrawing cumulative counts independently at each time

Independently drawing $N(t_1), N(t_2), \ldots$ as separate Poisson variables can make a later count lower than an earlier one, violating a counting process's path definition. Draw independent interval increments and accumulate them instead:

```
import numpy as np

rng = np.random.default_rng(644)
rate = 0.7
horizon = 12.0
n_intervals = 240
dt = horizon / n_intervals
increments = rng.poisson(lam=rate * dt, size=n_intervals)
counts = np.concatenate(([0], np.cumsum(increments)))
assert np.all(np.diff(counts) >= 0)
assert counts[0] == 0
print(f"terminal count = {counts[-1]}")
```

Accumulation makes the path nondecreasing, with disjoint-interval counts generated as independent increments. The chapter script also generates increments before cumulative sums.

12.5 Zero correlation does not imply independence

Common mistake: inferring independence from zero correlation

Take $X \sim \text{Uniform}(-1, 1)$ and $Y = X^2$. Symmetry gives $\mathbb{E}[X] = \mathbb{E}[X^3] = 0$, hence $\text{Cov}(X, Y) = 0$. Both variances are positive and correlation is zero. Yet Y is determined completely by X, so they are dependent. Correlation describes linear comovement; its absence does not remove all conditional relationships. A nonlinear feature in a conditional model should not be discarded solely because its raw linear correlation is near zero.

13. Quant Research Application

Suppose the target is whether an extreme loss occurs within a defined future window. Build the probability study in this order:

1. **Define the event:** for example, next-trading-day portfolio simple return below a threshold. State the interval, gross/net convention, timezone and when the label becomes known.
2. **Define information:** use only information available at signal time, denoted $\mathcal{F}_t$. Features, historical base rates and label versions must meet point-in-time constraints. Here $\mathcal{F}_t$ is information at prediction time t, and R_{t+1} is return over the prespecified subsequent interval.
3. **Choose the estimand:** predict conditional event probability $\mathbb{P}(E_{t+1} = 1 \mid \mathcal{F}_t)$, rather than claiming that an alert caused the loss. Separate path randomness, finite-sample parameter uncertainty and model uncertainty.
4. **Choose baseline and conditional models:** report the unconditional base rate first, then assess whether features improve prediction. Check calibration, group base rates and out-of-sample loss. With imbalanced classes, report precision, recall and false-positive burden as well as accuracy.
5. **Examine dependence and tails:** time-series labels may overlap and returns may correlate. Use temporal splits and dependence-aware uncertainty assessments. Resampling must preserve temporal structure; a global shuffle does not establish i.i.d. observations.
6. **Translate prediction into action and include costs:** thresholds depend on false-positive/false-negative losses, capital, liquidity, execution delay and trading costs. A probability is neither a trading rule nor a net-return result.
7. **Falsify and assess robustness:** vary base rates, windows, states and thresholds; compare historical baselines; examine drift, restatements, missingness and out-of-distribution cases. Conclusions relying on one window or a few events need narrower claims.

For posterior event probability q_t after a positive, compare false-positive and false-negative losses in a simplified binary decision. Let C_{FP} be the loss from acting when no event occurs and C_{FN} the loss from not acting when it occurs. Normalize correct-decision losses to zero. Expected loss from acting is $C_{FP}(1 - q_t)$; from not acting, $C_{FN}q_t$. Making the former smaller and rearranging gives the action condition

$$q_t > \frac{C_{FP}}{C_{FP} + C_{FN}}.$$

q_t is dimensionless; express both costs in the same loss units. Real decisions also include the action's effects and opportunity costs. Section 8's 8.72% is not a universal market threshold.

14. Chapter Summary

Probability axioms define event probabilities. Conditioning and Bayes' rule update state probabilities with new information. Random variables map outcomes to numbers; distributions, expectations, variances and covariances describe values, centers, dispersion and comovement. Portfolio risk requires covariance, not just individual asset variances.

Common distributions have specific supports, parameters and moment conditions. LLN brings sample means toward population means under assumptions; CLT concerns standardized means' limiting distribution. Correlation, overlap, nonstationarity and infinite variance change available conclusions. Random walks, Markov, Poisson and Brownian processes preserve different path structures. Assess assumptions against the target data; these simulations illustrate only mathematics under constructed parameters.

15. Exercises

Questions 6-E01 through 6-E07 use constructed inputs, not market observations.

6-E01 (concepts). Define sample space, event, mutual exclusion and independence. Are mutually exclusive events necessarily independent? Give a simple example with positive event probabilities.

6-E02 (concepts). Given $\mathbb{P}(A) = 0.2$ and $\mathbb{P}(B) = 0.5$, can marginal probabilities alone determine $\mathbb{P}(A \mid B)$? Identify the missing information.

6-E03 (concepts). If a continuous variable's PDF equals 0.4 at a point, is that point's probability 40%? Explain interval probabilities and density units.

6-E04 (concepts). Distinguish the objects described by LLN and CLT, and state the classical i.i.d. finite-variance CLT assumptions. Which population moments fail to exist for t with 1 degree of freedom?

6-E05 (hand calculation). A binary signal has prior event probability 0.02, sensitivity 0.80 and false-positive rate 0.10. Compute positive-alert probability and event probability given a positive.

6-E06 (hand calculation). Repeat $X \sim \text{Bernoulli}(0.25)$ independently 16 times. Find the single-trial mean and variance, and the total-success mean and variance.

6-E07 (calculation and interpretation). $N(t)$ is a homogeneous Poisson process at 1.2 events/hour. Give the 2-hour count distribution, zero-count probability, mean and variance. State its two increment assumptions.

Additional questions 6-N01 through 6-N05 also use teaching constructions.

6-N01 (identify objects). Record two independent Bernoulli(0.30) trials as 00, 01, 10 and 11. Give the sample space and outcome probabilities, define $A = \{\text{both succeed}\}$, $B = \{\text{at least one succeeds}\}$ and the success-count random variable X, and give the support and PMF of X.

6-N02 (one calculation). Using 6-N01, calculate only $\mathbb{P}(A \mid B)$. State which outcomes conditioning removes and which rows form the denominator.

6-N03 (complete calculation). From 6-N01's table, list probabilities for $X = 0, 1, 2$, then calculate $\mathbb{E}[X]$, $\mathbb{E}[X^2]$ and $\mathrm{Var}(X)$. Does the variance agree with the sum of two independent Bernoulli variances?

6-N04 (code diagnosis). For each Bernoulli(p) sample of size n, a program computes $z = (\bar{X} - p)/\sqrt{p(1-p)}$, plots many z values and overlays standard-normal density. Identify the missing scaling, derive the current statistic's variance and correct the denominator. Explain what more replications can check and cannot prove.

6-N05 (research judgment). A fixed-seed i.i.d. simulation produces a bell-shaped sample-mean histogram, and an autocorrelation test on real monthly returns does not reject zero correlation. The researcher claims: "Real returns are proven i.i.d. normal; the signal caused subsequent losses; classical CLT intervals are valid." Evaluate all three claims and list the objects or assumptions still needed for prediction, causality and mean inference.

6-B01 (dependence counterexample). In Section 6.5a's shared-scale construction, how does drawing A once differ from independently redrawing it each period? Calculate single-period variance, cross-period linear covariance, covariance of squared observations and the fourth moment of $Z_n = \sqrt{n}\bar{X}_n/\sqrt{2.5}$. Why does $\mathrm{Var}(\bar{X}_n) = 2.5/n$ still not establish that Z_n becomes standard normal?

16. Programming Exercises

Programming questions 6-P01 through 6-P03 must run independently in a Python 3 NumPy/SciPy environment. Use a function-local default_rng with the specified seed; do not use global shuffle.

6-P01 (conditional probability). With Section 8.1's low-base-rate inputs, compute and print positive-alert probability and event probability given a positive. Assert 0.0545 and approximately 0.0871559633 using math.isclose.

6-P02 (LLN and CLT). With seed 6202, generate 100,000 Bernoulli observations at $p = 0.3$ and report prefix means for 100, 1,000 and 100,000 observations. With seed 6203, generate 20,000 binomial sample totals from samples of $n = 100$ trials and standardize by theoretical mean and standard deviation. Check an empirical standardized mean near 0 and variance near 1. Explain why these checks do not establish i.i.d. real data.

6-P03 (process paths). With seed 6204, generate a 300-step symmetric random walk and a Poisson path at rate $\rho = 0.7$ over horizon $T = 10$. Print path shapes and terminal count. Assert walk increments belong to $\{-1, 1\}$, counts never decrease and initial count is 0. Optionally call the chapter script to generate figures.

17. Challenge Problems

6-C01 (correlation and effective sample size). Let X_t be stationary with mean 0, marginal variance 1 and autocorrelation $\rho_k = 0.5^k$. Derive $\mathrm{Var}(\bar{X}_4)$ at $n = 4$ and compare with i.i.d. variance. Explain the direction of error from treating observation count as independent sample count. Scoring: covariance-sum formula 2 points; correct lag substitution 2; comparison and interpretation 2.

6-C02 (state switching, Poisson and diffusion limits). Propose a conceptual model with finite-state Markov states, state-dependent event intensity and conditional continuous returns. Explain: (a) state transitions; (b) conditional Poisson intensity; (c) structure needed for Brownian return increments; (d) real features that could invalidate the model; (e) out-of-sample testing. Give a falsifying diagnostic. Scoring: states and information times 2 points; intensity and increments 2; Brownian restrictions 2; failure mechanisms and falsification 2. Answers may differ, but must describe a testable model rather than a fact.

18. Solutions / Solution Hints

6-E01

Mutual exclusion means $A \cap B = \varnothing$; independence means the intersection probability equals the product of marginals. With $\mathbb{P}(A) > 0, \mathbb{P}(B) > 0$ and mutually exclusive events, $\mathbb{P}(A \cap B) = 0$ while $\mathbb{P}(A)\mathbb{P}(B) > 0$, so they are dependent. For example, roll a fair die once and take $A = \{1\}$ and $B = \{2\}$. If one event has probability zero, mutual exclusion can also satisfy independence.

6-E02

No. You also need $\mathbb{P}(A \cap B)$; conditional probability is $\mathbb{P}(A \cap B)/0.5$. Fréchet bounds permit the intersection to range from $\max(0, 0.2 + 0.5 - 1) = 0$ to $\min(0.2, 0.5) = 0.2$, corresponding to conditional probabilities 0 to 0.4. Adding independence gives 0.2, but two marginal probabilities alone cannot imply independence.

6-E03

No. A continuous variable assigns zero probability to a specified point. Density has reciprocal-variable units, so its height is not a dimensionless probability. Over a small interval $[a, b]$, probability is $\int_a^b f_X(x)\,dx = F_X(b) - F_X(a)$. Only when density changes little over the interval does $f_X(x)(b - a)$ approximate its probability.

6-E04

LLN brings sample means toward the population mean under assumptions; CLT describes the asymptotic normal distribution of a centered mean scaled by its standard error. The classical CLT requires i.i.d. observations and finite, positive variance. At $\nu = 1$, t has neither mean nor variance, preventing classical standardization with those population moments.

6-E05

$$\mathbb{P}(+) = 0.02(0.80) + 0.98(0.10) = 0.016 + 0.098 = 0.114.$$

$$\mathbb{P}(E \mid +) = 0.016/0.114 = 0.1403508772 \approx 14.0351\%.$$

The numerator is the joint probability of an event and a positive alert. The denominator includes true and false positives.

6-E06

Single-trial mean is $p = 0.25$ and variance $p(1-p) = 0.1875$. Over 16 independent trials, total-success mean is $16p = 4$ and variance $16p(1-p) = 3$.

6-E07

The 2-hour parameter is $\lambda = 1.2 \times 2 = 2.4$, hence $N(2) \sim \text{Poisson}(2.4)$. Zero-count probability is $e^{-2.4} \approx 0.090718$, and mean and variance are both 2.4. Disjoint-interval increments must be independent, and their distributions must depend only on interval length (stationary increments). Rate has events/hour units and variance squared-count units.

6-P01 Reference implementation

```python
import math

prior = 0.005
sensitivity = 0.95
false_positive_rate = 0.05
joint_positive_event = prior * sensitivity
positive_rate = joint_positive_event + (1 - prior) * false_positive_rate
posterior = joint_positive_event / positive_rate

assert math.isclose(positive_rate, 0.0545, rel_tol=0.0, abs_tol=1e-12)
assert math.isclose(posterior, 0.0871559633027523, rel_tol=0.0, abs_tol=1e-12)
print(f"positive_rate={positive_rate:.4f}; posterior={posterior:.8%}")
```

Expected output: positive_rate=0.0545; posterior=8.71559633%. The assertions check both total-probability denominator and conditional direction.

6-P02 Reference implementation

```
import numpy as np

p = 0.30
rng_lln = np.random.default_rng(6202)
draws = rng_lln.binomial(1, p, size=100_000)
prefix_means = [float(draws[:n].mean()) for n in (100, 1_000, 100_000)]

rng_clt = np.random.default_rng(6203)
sample_size = 100
replications = 20_000
successes = rng_clt.binomial(1, p, size=(replications, sample_size)).sum(axis=1)
z = (successes - sample_size * p) / np.sqrt(sample_size * p * (1 - p))
z_mean = float(z.mean())
z_variance = float(z.var(ddof=1))

assert len(prefix_means) == 3
assert abs(prefix_means[-1] - p) < 0.01
assert abs(z_mean) < 0.04
assert abs(z_variance - 1.0) < 0.06
print("prefix means:", [round(value, 4) for value in prefix_means])
print(f"standardized mean={z_mean:.4f}; variance={z_variance:.4f}")
```

This code generates constructed samples from the specified Bernoulli model. It checks implementation against known theoretical parameters, not whether any real returns are i.i.d.

6-P03 Reference implementation

```
import numpy as np

rng = np.random.default_rng(6204)
walk_steps = 300
walk_increments = 2 * rng.integers(0, 2, size=walk_steps) - 1
walk = np.concatenate(([0], np.cumsum(walk_increments)))

rng_count = rng
rate = 0.7
horizon = 10.0
n_intervals = 200
dt = horizon / n_intervals
increments = rng_count.poisson(lam=rate * dt, size=n_intervals)
counts = np.concatenate(([0], np.cumsum(increments)))

assert walk.shape == (301,)
assert set(np.unique(np.diff(walk))) <= {-1, 1}
assert counts.shape == (201,)
assert counts[0] == 0
assert np.all(np.diff(counts) >= 0)
print(f"walk points={walk.size}; terminal position={walk[-1]}")
print(f"count points={counts.size}; terminal count={counts[-1]}")
```

A local Generator produces both paths in sequence with seed 6204. Poisson increments are independently generated on a grid of width $T/n_{\text{intervals}}$ and accumulated. An increment above 1 permits multiple events in a cell.

6-C01 Reference derivation

The stationary sample-mean variance is

$$\operatorname{Var}(\bar{X}_n) = \frac{1}{n}\left[1 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\rho_k\right]$$

with marginal variance 1. Substitute $n = 4, \rho_k = 0.5^k$:

$$\operatorname{Var}(\bar{X}_4) = \frac{1}{4}\left[1 + 2\left(\frac{3}{4}\cdot\frac{1}{2} + \frac{1}{2}\cdot\frac{1}{4} + \frac{1}{4}\cdot\frac{1}{8}\right)\right] = \frac{33}{64} = 0.515625.$$

I.i.d. variance is $1/4 = 0.25$. Correlation provides less effective information than 4 independent observations. Ignoring positive autocorrelation understates sample-mean uncertainty and overstates precision. This finite-n calculation does not establish the given autocorrelation function in a real sequence.

6-C02 One acceptable reference argument

Let $S_t \in \{0, 1\}$ describe two market states, updated only at equal-length window boundaries using fixed transition matrix P, and held constant within each window $[t, t+\Delta)$. This is a testable benchmark only if states/transitions remain stable and the state is knowable at the window start. Assume also that window counts are conditionally independent given the full state sequence and that count innovations do not affect transitions. Given initial state $S_t = s$, use the fixed-intensity model

$$N_{t+\Delta} - N_t \mid S_t = s \sim \text{Poisson}(\rho_s\Delta).$$

ρ_s has events/time units and Δ uses the same time unit. For continuous increments, hold drift and volatility fixed within the window and make Brownian innovations independent of the state chain:

$$X_{t+\Delta} - X_t \mid S_t = s \sim N(\mu_s\Delta, \sigma_s^2\Delta).$$

Here μ_s is drift per unit time and σ_s volatility per square root of time. This is a piecewise-constant conditional construction, not a market calibration.

If states switch inside a window, the two distributions conditional only on its starting state generally fail. Given an exogenous full state path, Poisson parameter becomes $\int_t^{t+\Delta} \rho_{S_u}\, du$, and normal conditional mean and variance use corresponding drift and variance integrals. Without the intervening path, integrate over its distribution, generally producing a mixture. If states are determined endogenously by the same count or Brownian innovations, re-examine these conditional laws. Chapters 19 and 44 develop hidden states and stochastic integration; this chapter performs no Itô derivation or calibration.

Possible failures include macro-dependent transitions, states unavailable in real time, clustered event intensity, return jumps, volatility clustering, heavy tails and switching-induced parameter drift. Split out-of-sample tests by time. Freeze state and intensity estimates, then examine Poisson calibration/dispersion, count-residual correlation, standardized continuous-return residuals and tail quantiles. Compare with constant baselines and independent Bernoulli/Poisson benchmarks. Holdout overdispersion or clustered residuals falsify a simple conditional Poisson model; heavy standardized tails or discontinuous jumps weaken the Brownian return model. Preserve information availability and versions rather than filling signals with future states or revised values.

6-C02 Scoring check

Total: 8 points. Clear state variables, observable information times and transitions earn 2; state-conditional intensity, count-increment assumptions and units 2; Brownian continuous paths, independent normal increments and time-growing variance 2; at least two failure mechanisms plus a falsifying holdout diagnostic 2. Claims that real markets must be Markov/Poisson/Brownian receive no points for the corresponding assumptions.

6-N01

The sample space is $\Omega = \{00, 01, 10, 11\}$. Independence gives outcome probabilities 0.49, $0.21, 0.21, 0.09$ summing to 1. Events are $A = \{11\}$ and $B = \{01, 10, 11\}$. Random variable X maps outcomes to counts: $X(00) = 0$, $X(01) = X(10) = 1$ and $X(11) = 2$. Support is $\{0, 1, 2\}$, with PMF $0.49, 0.42, 0.09$ respectively.

6-N02

Condition B retains 01, 10 and 11 with total probability $0.21 + 0.21 + 0.09 = 0.51$. Only 11 belongs to A, so $\mathbb{P}(A \mid B) = 0.09/0.51 = 3/17 \approx 0.17647059$. The denominator is neither single-trial sensitivity nor all four rows' total probability.

6-N03

Group outcomes by X: $\mathbb{P}(X = 0) = 0.49$, $\mathbb{P}(X = 1) = 0.21 + 0.21 = 0.42$ and $\mathbb{P}(X = 2) = 0.09$. Then

$$\mathbb{E}[X] = 0(0.49) + 1(0.42) + 2(0.09) = 0.60,$$

$$\mathbb{E}[X^2] = 0^2(0.49) + 1^2(0.42) + 2^2(0.09) = 0.78, \qquad \mathrm{Var}(X) = 0.78 - (0.60)^2 = 0.42.$$

Each Bernoulli variance is $0.3(0.7) = 0.21$, and two independent trials give total variance $0.21 + 0.21 = 0.42$, agreeing with the outcome table.

6-N04

Keeping numerator $\bar{X} - p$ requires denominator $\sqrt{p(1-p)/n}$. Alternatively use numerator $\sqrt{n}(\bar{X} - p)$ and divide by $\sqrt{p(1-p)}$. The original statistic is $1/\sqrt{n}$ times the correct CLT-standardized value, with variance

$$\mathrm{Var}(z) = \frac{\mathrm{Var}(\bar{X})}{p(1-p)} = \frac{p(1-p)/n}{p(1-p)} = \frac{1}{n},$$

rather than 1. More independent replications show the statistic's distribution under the specified Bernoulli model more clearly and check implementation; they neither prove an asymptotic CLT nor establish independence and identical distributions of real returns.

6-N05

All three claims exceed the evidence. A simulation describes one construction under specified generation; even many replications check only that model's sampling distribution. Failure to reject zero autocorrelation means insufficient evidence against that test's null, not proof of zero population covariance, independence or normality. Predictive probabilities require an event, return window, signal information time and out-of-sample evaluation. Causal claims additionally require an intervention and identification through randomization or defensible confounder control. A classical i.i.d. mean-CLT interval requires independent, identically distributed observations with finite positive population variance, and a centered sample mean standardized using each sample's size n and population standard deviation. Using sample standard deviation also requires consistency and corresponding inference assumptions. Dependent-sequence methods need their own dependence and moment conditions. One plot or test cannot replace these requirements.

6-B01

Drawing shared A once preserves cross-period scale dependence. Redrawing it independently each period with independent innovations creates a different i.i.d. mechanism. $\mathbb{E}[A^2] = 2.5$ gives variance 2.5 and zero cross-period covariance, hence $\mathrm{Var}(\bar{X}_n) = 2.5/n$. Yet $\mathbb{E}[A^4] = 8.5$ and squared cross covariance $8.5 - 2.5^2 = 2.25$. Conditional on scale, Z_n has variance 0.4 or 1.6 with equal weights; the normal fourth-moment identity in Section 6.5a gives $3(0.4^2 + 1.6^2)/2 = 4.08$. The same nonnormal mixture holds for every n, so standardization produces no standard-normal limit. This argument does not infer general distributional convergence from fourth moments. The distribution is already known to be unchanged for all n; its fourth moment only checks that it is not standard normal. Second moments and zero covariance determine mean variance; a normal limit also needs suitable independence or weak-dependence conditions.

19. Further Reading

- **Must Know:** MIT 6.041SC Lectures 1–2 on sample spaces, axioms, conditioning, total probability and Bayes; Lectures 5–6 on random variables, expectation, variance and conditional expectation; Lecture 8 on continuous variables and PDF/CDF; Lecture 12 on iterated expectation and total variance. Official links appear in Section 20.
- **Should Know:** MIT 6.262 Lectures 3–4 and Chapter 2 §2.1 on convergence, Poisson arrivals and increments; MIT 6.041SC Lecture 17 introduces finite-state Markov chains.
- **Advanced:** MIT 15.070J Lecture 6 on Brownian motion, especially heuristic random-walk construction and process definitions. The notes explicitly distinguish this heuristic from a proof. Continue with rigorous limits and stochastic integration; this course develops Itô calculus in Chapter 44.
- **APIs:** NumPy Generator, binomial, normal and poisson references explain generators and parameters. SciPy binom, norm, lognorm, poisson and t references specify parameterization. Software documentation supplies interfaces, not market evidence.

20. References

Sources are official course materials or software documentation. `source_notes.md` records reading scope, retrieval and limits. This chapter has no empirical market source.

- **MIT_6041SC** — MIT OpenCourseWare, 6.041SC Probabilistic Systems Analysis and Applied Probability, Fall 2013. Source notes list links and reading scope for Lectures 1–2, 5–6, 8, 10, 12 and 17.
- **MIT_6262** — MIT OpenCourseWare, 6.262 Discrete Stochastic Processes, Spring 2011. Source notes link the consulted Lectures 3–4 and Poisson Processes Chapter 2 §2.1.
- **MIT_BROWNIAN** — MIT OpenCourseWare, 15.070J Advanced Stochastic Processes, Fall 2013, Lecture 6. Brownian definitions and heuristic random-walk construction, not a market-fitting source.
- **NUMPY_RANDOM** — NumPy, Random Generator reference, Generator.binomial, Generator.normal, Generator.poisson.
- **SCIPY_STATS** — SciPy binom, norm, lognorm, poisson and t distribution references.

Additional official APIs: SciPy expon and SciPy uniform. The original root review read definitions and parameterizations; count-to-waiting-time and zero-correlation examples are original derivations.

Chapter 7 — Statistical Inference: uncertainty in research conclusions

1. Learning goals

After this chapter, you should formulate a question about an unknown population parameter, choose a sample statistic as an estimator, and interpret the realized estimate. You should derive maximum likelihood estimates (MLEs) of mean and variance under independent normal observations; compute and interpret confidence intervals, p-values and testing errors; distinguish bias, variance, consistency and likelihood; choose resampling for temporal dependence; and perform a small Bayesian update with a clear interpretation of each output.

Record the estimand, observation mechanism and test assumptions beside the result before pressing a test button. Prerequisites are Chapter 6's expectations, conditional probability, LLN and CLT, and Chapter 4's differentiation. Chapter 8 extends this inference to regression coefficients.

The statistics script uses core functions in `statistics_tools.py`; the historical course-root Notebook provides an interactive version. Simulations are labeled constructed samples. Actual industry returns use the French snapshot saved in Chapter 2. These illustrate statistical methods without establishing a tradable strategy.

2. Why a quant needs inference

One backtest reports daily mean return 0.03%, another 0.04%. The second is larger, but if it uses only 20 days during which one news story persists, the difference is not automatically a repeatable advantage. Statistics assesses the strength of conclusions supported by finite, noisy observations, rather than making result tables appear precise.

Quantitative research faces variation in the sample mean, possible changes in the generating mechanism, and searches across many versions with only the best retained. Classical intervals mainly address the first. They do not automatically handle regime changes or record discarded experiments.

An interval can describe instability of a fixed rule's mean estimate under a specified model. Tradability, costs, information availability and whether the rule was frozen in advance still require financial and experimental design. Chapters 22 and 30–32 develop those parts.

3. Intuition

3.1 One historical path, but a sampling distribution asks about repetition

Imagine randomly weighing a batch of apples from a continually replenished supply. The supply's mean weight is an unknown population parameter; the batch is a sample. Another batch selected by the same rule yields another mean. Calculating this batch's mean is arithmetic; studying how means change across batches is inference. The analogy explains repeated sampling only. Markets have time-ordered dependence and regime changes, so trading days are not freely exchangeable apples.

A statistic is a function of the sample, such as its mean. For observed data it is a fixed number; before observation it varies with the random sample and has a sampling distribution. Keep these perspectives separate when reading intervals and tests.

3.2 Standard deviation and standard error describe different variation

Return standard deviation describes single-period volatility. Standard error (SE) describes an estimator's variability across repeated samples. More independent observations reduce the sample mean's SE, but downloading ten more years does not reduce the market's daily volatility.

For example, a daily standard deviation of 2% gives a mean SE of about 0.2% across 100 independent observations. Tomorrow's return does not thereby fluctuate by only 0.2%. Under the specified model, the estimator formed from the 100-day average has sampling variability of about 0.2%.

3.3 Nonsignificance does not establish absence of an effect

High noise and a small sample can hide a small, real mean effect. Failure to reject zero mean means that this test lacks evidence to reject it. Establishing an economically negligible effect instead requires a prespecified tolerance and equivalence or interval-bound analysis; a large p-value alone cannot establish this.

4. Mathematical foundations

4.1 Samples, parameters and estimands

Definition: parameters, estimators and realized estimates

First specify the population, target parameter and sampling unit. The population is the generating domain under study. Parameter θ is an unknown distribution feature, such as population mean μ, variance σ^2 or standard deviation $\sigma = \sqrt{\sigma^2}$. Means and standard deviations share observation units; variance uses squared units. For a fixed rule's daily returns, a unit is a trading day defined in advance, not a trade record selected afterward. n counts units and carries no return units.

Write unit i's random outcome as X_i and its observed value as x_i. Under an independent and identically distributed (IID) model, units share distribution F_θ. A statistic is a sample function, such as mean $\bar{X} = n^{-1}\sum_i X_i$, which varies before observations arrive. A statistic used to estimate θ is an estimator: for example, $\bar{X}$ estimates μ. Substituting observed data gives the estimate $\bar{x} = n^{-1}\sum_i x_i$. Uppercase denotes the random object before sampling; lowercase denotes this realization. Repeated sampling generates another n units under the declared mechanism and studies the estimator's distribution. With dependent daily returns, that mechanism must preserve temporal structure.

Specify the estimand too: unconditional mean excess return over a period, or a conditional mean in a market state? An average over surviving securities, or the complete contemporaneous population? Match the reported mean to its target, units, window and sampling unit. An API cannot make these choices for you.

4.2 Sample mean and variance

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i, \qquad S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2.$$

$\bar{X}$ is the sample-mean estimator in the units of X_i. S^2 uses denominator $n-1$ and squared observation units; sample standard deviation $S = \sqrt{S^2}$ shares observation units. Require $n \geq 2$. The realized estimates are $\bar{x}, s^2, s$. Under IID observations and finite second moments, $\mathbb{E}[\bar{X}] = \mu$ and $\mathrm{Var}(\bar{X}) = \sigma^2/n$. Section 6 explains $n-1$.

4.3 From IID to dependence: the full sample-mean variance

Without assuming independence, we still have

$$\mathrm{Var}(\bar{X}) = \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}\mathrm{Cov}(X_i, X_j).$$

Here $\mathrm{Cov}(X_i, X_j)$ is covariance between observations i, j. Under weak stationarity, the mean stays constant and lag-k autocovariance is $\gamma_k = \mathrm{Cov}(X_t, X_{t-k})$, giving

$$\mathrm{Var}(\bar{X}) = \frac{1}{n}\left[\gamma_0 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\gamma_k\right].$$

γ_0 is single-period variance. Lag k has $n-k$ observation pairs, occurring in both matrix triangles; this gives factor 2 and $1-k/n$. Zero autocovariance at all nonzero lags suffices to reduce the expression to σ^2/n. For fixed n, nonzero terms may also cancel in the weighted sum. Zero covariance does not generally mean independence.

With absolutely summable autocovariances and suitable weak-dependence assumptions, define long-run variance $\Omega = \gamma_0 + 2\sum_{k=1}^{\infty}\gamma_k$; large-sample mean variance is approximately Ω/n. Chapter 8 introduces heteroskedasticity and autocorrelation consistent (HAC) standard errors. Writing SE as $s/\sqrt{n}$ already invokes an important assumption.

5. Definitions

5.1 Bias, variance and mean squared error

$$\mathrm{Bias}_\theta(\hat{\theta}) = \mathbb{E}_\theta[\hat{\theta}] - \theta, \qquad \mathrm{MSE}_\theta(\hat{\theta}) = \mathbb{E}_\theta[(\hat{\theta} - \theta)^2].$$

$\hat{\theta} = T(X_1, \ldots, X_n)$ is an estimator. Subscript θ denotes expectation under a model with parameter θ; MSE means mean squared error. These definitions concern scalar parameters; matrix parameters need a specified loss function. Zero bias means unbiasedness.

Consistency means that an estimator approaches the true parameter in probability as sample size tends to infinity: for any $\varepsilon > 0$,

$$\Pr_\theta(|\hat{\theta}_n - \theta| > \varepsilon) \longrightarrow 0.$$

Unbiasedness concerns expectation at each finite sample size; consistency concerns the limit of an estimator sequence. Neither replaces the other. When adding market observations, ask whether the mechanism needed for that limit still applies.

Bias measures average deviation across repeated samples; variance measures spread across samples; consistency asks whether errors vanish in probability with increasing sample size; MSE combines variance and squared bias in one loss scale. They describe finite-sample center, dispersion, limiting behavior and squared loss respectively.

5.2 Likelihood and maximum likelihood

For IID continuous observations, the likelihood function is

$$L(\theta; x) = \prod_{i=1}^{n} f_\theta(x_i), \qquad \hat{\theta}_{\text{MLE}} = \arg\max_{\theta \in \Theta} L(\theta; x).$$

f_θ is model density, x fixed observed data, and Θ the permitted parameter space. For discrete data use PMFs instead. Likelihood fixes the data and varies the parameter. A continuous variable has zero probability at an exact point, so we compare densities, not the probability of this exact decimal sequence. Likelihood is not posterior parameter probability.

Dependence requires a joint or conditional distribution, for example $f_\theta(x_1) \prod_{t=2}^{n} f_\theta(x_t \mid x_{1:t-1})$. Multiplying marginal densities instead actively substitutes a different model.

Bias, variance and consistency characterize estimators' average deviations, repeated-sample spread and large-sample convergence. Likelihood compares parameter values' relative fit for fixed data. Its value is neither a parameter probability nor a description of sampling variability.

5.3 Confidence intervals, tests and errors

An interval procedure $[L(X), U(X)]$ has confidence level $1 - \alpha_{\text{sig}}$ if its coverage satisfies

$$\Pr_\theta\{L(X) \le \theta \le U(X)\} = 1 - \alpha_{\text{sig}}$$

or reaches at least the target level where appropriate. L, U are random endpoints, θ a fixed parameter, and α_{sig} the significance level, distinct from asset-pricing alpha. Approximate intervals can have coverage different from nominal; assess that approximation.

A hypothesis test specifies null H_0 and alternative H_1, then its statistic and rejection region. Type I error rejects a true H_0. Type II error fails to reject under a specified true alternative. Power is the probability of rejecting H_0 under that alternative, or one minus Type II error probability. It depends on effect size, noise, sample size and the decision rule; it is not one fixed property of a test.

For this chapter's two-sided t test, a p-value is the null-model probability of a statistic at least as extreme as observed. It is neither the probability that H_0 is true nor a strategy's failure probability.

5.4 Bootstrap and Bayesian inference

Bootstrap uses observations to approximate an unknown generating distribution and repeatedly recalculates the statistic to approximate sampling variability. Resampling must preserve the structures relevant to the question.

Bayesian inference represents parameter uncertainty with a probability distribution. Prior, likelihood and posterior satisfy

$$\pi(\theta \mid x) = \frac{L(\theta; x)\pi(\theta)}{\int_\Theta L(u; x)\pi(u)\, du}.$$

$\pi(\theta)$ is prior density, the denominator is marginal data density, and u the integration variable. Use sums for discrete parameters. Posterior probability statements condition on the model and prior. A credible interval contains a specified posterior mass and differs in meaning from a frequentist confidence interval.

Frequentist inference fixes the parameter and varies data across repeated samples; confidence describes the procedure's long-run coverage. Bayesian inference assigns posterior parameter probabilities given the model, prior and observed data. Posterior mass and coverage describe different uncertainty objects.

6. Derivations

6.1 Why sample variance divides by $n - 1$

Start with the algebraic identity:

$$\sum_i (X_i - \bar{X})^2 = \sum_i (X_i - \mu)^2 - n(\bar{X} - \mu)^2.$$

μ is the common population mean. Expanding $X_i - \bar{X} = (X_i - \mu) - (\bar{X} - \mu)$ gives cross-term sum $-2n(\bar{X} - \mu)^2$; adding $n(\bar{X} - \mu)^2$ yields the identity.

Under IID observations with finite variance, the first right-hand term has expectation $n\sigma^2$ and the second $n\,\mathrm{Var}(\bar{X}) = \sigma^2$. The squared-deviation sum therefore has expectation $(n-1)\sigma^2$, so dividing by $n - 1$ gives an unbiased variance estimator. Estimating the common center makes residuals sum to zero, leaving $n - 1$ free directions.

For correlated observations, $\mathrm{Var}(\bar{X})$ generally differs from σ^2/n, so this unbiasedness derivation cannot be reused unchanged. `ddof=1` states a computation convention; it does not prove IID data.

6.2 Gaussian mean and variance MLEs

Assume $X_i \overset{\text{IID}}{\sim} N(\mu, v)$ with $v = \sigma^2 > 0$. The log-likelihood is

$$\ell(\mu, v) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log v - \frac{1}{2v}\sum_i (x_i - \mu)^2.$$

ℓ is the natural log of likelihood. A strictly increasing logarithm preserves the maximizer. Differentiate with respect to μ:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{v}\sum_i (x_i - \mu) = 0 \quad \Rightarrow \quad \hat{\mu} = \bar{x}.$$

Substitute $\bar{x}$ and differentiate with respect to v:

$$\frac{\partial \ell(\bar{x}, v)}{\partial v} = -\frac{n}{2v} + \frac{\sum_i (x_i - \bar{x})^2}{2v^2} = 0 \quad \Rightarrow \quad \hat{v}_{\text{MLE}} = \frac{1}{n}\sum_i (x_i - \bar{x})^2.$$

With a positive squared-deviation sum, profile log-likelihood tends to negative infinity as $v \to 0$ and $v \to \infty$; its stationary point is the maximum. If all x_i are identical, the sum is zero. Setting μ to that common value makes likelihood grow without bound as $v \downarrow 0$. There is no finite MLE in the regular Gaussian model with $v > 0$. A descriptive zero variance is not a valid regular Gaussian MLE; the core function raises an explicit error.

Note that

$$\mathbb{E}[\hat{v}_{\text{MLE}}] = \frac{n-1}{n}\sigma^2.$$

The variance MLE is downward biased but remains consistent under IID observations, finite second moments and the relevant LLN. MLE need not be unbiased; finite- and large-sample properties differ.

6.3 Bias–variance decomposition of MSE

Write the estimation error as

$$\hat{\theta} - \theta = (\hat{\theta} - \mathbb{E}\hat{\theta}) + (\mathbb{E}\hat{\theta} - \theta).$$

Square and take expectation. The cross term contains $\mathbb{E}[\hat{\theta} - \mathbb{E}\hat{\theta}] = 0$, hence

$$\text{MSE}(\hat{\theta}) = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

This explains the value of shrinkage. Pulling extreme estimates toward a common center introduces bias, but a sufficiently large variance reduction can lower total error. Chapter 8 uses this in regularized regression and Chapter 21 in covariance shrinkage.

6.4 The t interval for an IID normal mean

Assumptions and pivot: an IID Gaussian mean

Under normal sampling, standardized mean $Z = (\bar{X} - \mu)/(\sigma/\sqrt{n})$ is standard normal, while $V = (n-1)S^2/\sigma^2$ is chi-squared with $n-1$ degrees of freedom and independent of Z. Therefore

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} = \frac{Z}{\sqrt{V/(n-1)}} \sim t_{n-1}.$$

t_{n-1} is Student's t with $n-1$ degrees of freedom. Estimating unknown volatility adds uncertainty and gives small-sample t heavier tails than normal. Independence of the normal sample mean and squared deviations follows from orthogonal Gaussian components. This chapter states that distribution fact without fully proving the chi-squared density.

Choose critical value $q = t_{n-1,1-\alpha_{\text{sig}}/2}$ so that $\Pr(-q \le T \le q) = 1 - \alpha_{\text{sig}}$. Because $S/\sqrt{n} > 0$, rearrangement gives

$$\mu \in \left[\bar{X} - q\frac{S}{\sqrt{n}},\ \bar{X} + q\frac{S}{\sqrt{n}}\right].$$

q is dimensionless, $S/\sqrt{n}$ the mean's SE, and endpoints share return units. This is an exact IID Gaussian interval. Nonnormal independent samples require separate asymptotic assumptions and finite-sample checks. A large dependent sample alone cannot recover the exact result.

For testing $H_0 : \mu = \mu_0$, use $t_{\text{obs}} = (\bar{x} - \mu_0)/(s/\sqrt{n})$ and two-sided p-value $2\Pr(t_{n-1} \ge |t_{\text{obs}}|)$. With the same assumptions and two-sided level, μ_0 outside the corresponding interval is equivalent to rejecting H_0. Different SEs or one-/two-sided rules break that direct correspondence.

Complete example: a one-sided z test, interval and both error types

Work through a right-sided normal test with known standard deviation. Each independent measurement is one sampling unit, with all values in the same output units. Construct four IID Gaussian observations $(1, 1, 3, 3)$, count $n = 4$ and known $\sigma = 2$. Test $H_0 : \mu = 0$ against $H_1 : \mu > 0$ at significance $\alpha = 0.05$. Exact critical value is $c = z_{0.95} = \Phi^{-1}(0.95)$; use rounded $c \approx 1.645$ for hand calculation. Compute $\bar{x} = 2$, SE $\sigma/\sqrt{n} = 1$ and statistic $z = (\bar{x} - 0)/1 = 2$. Let Φ be the standard-normal CDF, with table approximations $\Phi(1.645) \approx 0.9500$, $\Phi(2) \approx 0.9772$ and $\Phi(1) \approx 0.8413$. One-sided p-value is approximately $1 - \Phi(2) = 0.0228$, below 0.05, so reject H_0 under the prespecified rule. The one-sided 95% lower bound is approximately $2 - 1.645 \times 1 = 0.355$, giving interval $[0.355, \infty)$, excluding zero. With the exact critical value, for any fixed true μ, the procedure satisfies $\Pr_\mu(\bar{X} - c\sigma/\sqrt{n} \le \mu) = 0.95$. After observation the endpoint is fixed; 0.95 refers to repeated-sample coverage.

Type I error under this rule rejects a true H_0. Exact threshold $c = z_{0.95}$ has error probability $1 - \Phi(c) = 0.05$. Rounding the threshold to 1.645 gives about 0.049985, approximately 0.05. Type II error requires a specified alternative. At approximate value $\mu_1 = 2.645$, the hand threshold 1.645 gives $\beta \approx \Phi((1.645 - 2.645)/1) = \Phi(-1) \approx 0.1587$ and power about 0.8413.

Continued: the same complete example

Using the exact quantile for the same $\mu_1 = 2.645$ instead gives $\beta \approx 0.15862$. Null and threshold determine Type I error; alternative effect and noise determine Type II error.

6.5 Constructing a percentile bootstrap interval

After observing $x_1, \ldots, x_n$, the empirical distribution places mass $1/n$ at each observation. Each IID bootstrap replication samples n entries with replacement and computes $\bar{x}^{*(b)}$, for $b = 1, \ldots, B$. A star marks resampling and B is replication count. The percentile interval uses quantiles $\alpha_{\text{sig}}/2$ and $1 - \alpha_{\text{sig}}/2$ of those means.

The empirical distribution approximates the unknown population, and repeated empirical samples approximate the estimator's sampling variation. Finite-sample bias, skewness, heavy tails, nonsmooth statistics and boundary parameters can impair coverage. Increasing B mainly reduces Monte Carlo error in resampling calculations; it creates no new market observations.

Independent day-by-day resampling destroys neighboring relationships in a financial sequence. Mean inference then misses temporal covariance's contribution to SE. Justify an IID approximation before applying independent bootstrap to financial data.

For time series, sample blocks of neighboring observations instead. Here a circular block bootstrap uses fixed length L: choose random starts, preserve within-block order, wrap at the sample's end, concatenate and truncate to n observations. It retains short-term within-block dependence but cuts relationships at joins; the circular boundary is itself an approximation. Short blocks can omit dependence, while long blocks offer fewer effective rearrangements. Wider intervals alone do not establish correctness.

Asymptotic validity of block methods requires stationarity, relevant moments and sufficiently decaying dependence, among other assumptions. Common results also require $L \to \infty$ and $L/n \to 0$. These two rate conditions are not complete theorem assumptions and do not guarantee a fixed $L = 12, n = 180$ setting. Künsch's original study concerns block resampling of general stationary observations (Section 20); this chapter uses a circular fixed-block variant to illustrate structure.

6.6 Beta–Binomial updating

Assume $Y_i \mid p \overset{\text{IID}}{\sim} \text{Bernoulli}(p)$ and observe s successes and f failures. Prior $p \sim \text{Beta}(a, b)$ with $a, b > 0$ has density kernel $p^{a-1}(1-p)^{b-1}$; likelihood kernel is $p^s(1-p)^f$. Multiplication gives

$$\pi(p \mid y) \propto p^{a+s-1}(1-p)^{b+f-1}, \qquad p \mid y \sim \text{Beta}(a+s, b+f).$$

p is success probability, a, b dimensionless prior shape parameters, and s, f counts. A posterior remaining in the prior distribution family is conjugate. Its mean is

$$\mathbb{E}[p \mid y] = \frac{a+s}{a+b+s+f} = \frac{a+b}{a+b+n}\frac{a}{a+b} + \frac{n}{a+b+n}\frac{s}{n},$$

where $n = s + f > 0$. This is a weighted average of prior mean and sample success rate. It does not mean the prior contains (a+b) real observed trades; the weights have an algebraic interpretation. Explain the prior's economic basis and assess sensitivity.

6.7 Power and sample size: detecting a small effect

To examine power, temporarily assume IID Gaussian observations with known single-period standard deviation σ. Prespecify one-sided hypotheses $H_0 : \mu = 0$ and $H_1 : \mu > 0$. A directional hypothesis must precede the research; a positive observed mean does not justify switching from two-sided to one-sided.

Let $c = z_{1-\alpha_{\text{sig}}}$ and reject when $\sqrt{n}\bar{X}/\sigma > c$. Under specified alternative $\mu = \delta > 0$, this statistic follows $N(\sqrt{n}\delta/\sigma, 1)$, hence

$$\text{Power}(\delta) = 1 - \Phi\left(c - \frac{\sqrt{n}\delta}{\sigma}\right).$$

Φ is the standard-normal CDF, δ the prespecified effect in single-period return units, and n independent-observation count. With fixed significance, a larger effect, lower noise or more independent observations increases power. Type II error is one minus power under that alternative, not the probability of an ineffective strategy after nonrejection.

For target power $1 - \beta_{\text{II}}$, rearrangement gives the sample-size design bound, rounded up to a minimum integer:

$$n \geq \left\lceil \frac{\sigma}{\delta}\left(z_{1-\alpha_{\text{sig}}} + z_{1-\beta_{\text{II}}}\right)\right\rceil^2.$$

β_{II} is the Type II error target, not market beta. With single-period standard deviation 2%, target mean 0.1%, one-sided level 5% and power 80%, round upward to 2474 independent observations. As trading days, these span years; adding data may also introduce mechanism changes. This idealized sample size is not a market-research guarantee.

```
import numpy as np
from scipy import stats

sigma, delta, significance, target_power = 0.02, 0.001, 0.05, 0.80
critical = stats.norm.ppf(1.0 - significance)
required = int(np.ceil((sigma / delta * (critical + stats.norm.ppf(target_power)))
    ↪ **2))
for n in (100, 1000, required):
    power = stats.norm.sf(critical - np.sqrt(n) * delta / sigma)
    print("known-sigma IID Gaussian, one-sided", n, "power", power)
assert stats.norm.sf(critical - np.sqrt(required) * delta / sigma) >= target_power
```

This design illustration does not use unknown-standard-deviation t power, temporal dependence or multiple testing. Recalculate or simulate under the research model; 2474 is not a universal strategy acceptance threshold.

7. Financial interpretation

7.1 Measure payoffs, not win rate alone

Suppose each win earns 1 unit, each loss loses 2 units, every trade costs 0.1 unit, and success probability is p. Expected net payoff per trade is

$$\mathbb{E}[G] = p \times 1 - (1 - p) \times 2 - 0.1 = 3p - 2.1.$$

G is net payoff per trade, so a positive expectation requires $p > 0.7$. Even a high posterior probability that p exceeds 0.5 cannot establish positive expected strategy returns. This example fixes payoff magnitudes; actual amounts, holding times and tail risk need separate modeling.

7.2 Effect size, statistical significance and economic value

Statistical significance assesses incompatibility with the tested hypothesis. Economic value assesses whether an effect warrants costs and risk given capacity.

A daily gross mean of 0.02% with costs 0.03% has a negative net mean even if a large sample estimates the gross mean precisely. A positive mean with a wide interval may instead leave economic value uncertain. Report effect size, interval, cost convention and risk alongside one another; a p-value is not a sole strategy-ranking score.

A mean significantly different from zero also does not remove market beta. Long exposure in a rising market can have a positive mean without independent alpha. Chapters 14–15 specify asset-pricing and factor-regression benchmarks.

7.3 Effective sample size: intuition and scope

For stationary first-order autoregression (AR(1)) $X_t - \mu = \phi(X_{t-1} - \mu) + \varepsilon_t$, assume $|\phi| < 1$ and independent zero-mean innovations with finite variance. Then $\gamma_k = \gamma_0 \phi^{|k|}$. Substituting the geometric series into long-run variance gives

$$\Omega = \gamma_0 \frac{1 + \phi}{1 - \phi}, \qquad n_{\text{eff}} \approx n \frac{1 - \phi}{1 + \phi}.$$

n_{eff} is effective sample size that approximately matches an independent mean variance, not actual independent-trade count. It is a large-sample stationary AR(1) approximation for mean estimation, not a replacement for every model's degrees of freedom. At $n = 180, \phi = 0.7$ the approximation is 31.76: 180 rows need not provide 180 independent information units. Negative correlation can reduce mean variance; $n_{\text{eff}} > n$ is not a mathematical error.

8. Numerical Example

8.1 Four returns calculated by hand

Complete example: estimates and t inference from four returns

The following intervals and tests use an IID Gaussian reference model. Specifying four numbers does not establish normality or independence.

Use constructed daily decimal returns $x = (-0.01, 0.01, 0.02, 0.02)$. Mean is 0.01, or 1%; deviations are $(-0.02, 0, 0.01, 0.01)$ and squared-deviation sum 0.0006. Gaussian variance MLE is 0.00015, unbiased sample variance 0.0002, $s \approx 0.0141421$ and mean SE $s/\sqrt{4} \approx 0.0070711$.

The 95% t interval has 3 degrees of freedom and critical value about 3.182446, giving $[-0.0125033, 0.0325033]$, or daily-mean interval $[-1.2503\%, 3.2503\%]$. A zero-mean test gives $t \approx 1.414214, p \approx 0.252215$. A positive sample mean can coexist with insufficient evidence. These four returns are not market observations; compounding or annualization cannot create credible strategy performance.

8.2 Recorded fixed-seed outputs

The full script uses seed 20261007 and independent child streams for experiments. Forty constructed IID Gaussian observations have model mean 0.002 and standard deviation 0.02. The recorded realization has sample mean about −0.001500, variance MLE about 0.00045812, unbiased variance about 0.00046987, 95% interval $[-0.008432, 0.005432]$ and zero-mean p-value about 0.664051. The difference between true mean and realized estimate is the point of this experiment.

Across 3000 correct-model experiments, 95% t coverage at sample sizes 10, 30 and 100 was about 95.3333%, 94.9333% and 95.5%. A finite simulation need not yield exactly 95%. At true coverage 0.95, the SE of 3000 coverage indicators is approximately $\sqrt{0.95 \times 0.05/3000} = 0.003979$, or 0.398 percentage points. This is Monte Carlo error in simulated coverage, not a market-return SE.

The stationary constructed AR(1) sample contains 240 observations; the first 180 enter estimation, while the final 60 stay out of resampling. With $\phi = 0.7$ and innovation standard deviation 0.01, 4000 resamples gave an IID percentile interval about $[-0.002327, 0.001643]$ and a circular length-12 block interval about $[-0.003428, 0.002961]$. Blocks were wider for this particular sample; greater width is neither a universal theorem nor proof of coverage. Repeated generation and block-length sensitivity require separate checks.

Finally, prior Beta(2,2) and 6 wins/4 losses give posterior Beta(8,6), with success-probability mean $8/14 = 4/7 \approx 0.571429$. Using Section 7's payoff structure gives posterior expected net payoff $3 \times 4/7 - 2.1 \approx -0.385714$ units/trade.

9. Python Implementation

Default entry point in the accepted reader package

From the accepted I3 reader package root, create the isolated environment described in its README, then run:

```
.venv/bin/python run_notebook.py notebooks/ch07_statistics.ipynb
```

This notebook calls the packaged shared implementation, reads no omitted French data and downloads nothing. Each execution writes a unique UUID subdirectory under runs and prints its location. Old production tests and historical course-root commands below belong to the original project; the default reader route is this package-root notebook with its assertions. Optional historical French inputs follow the reader's data contract; this batch did not rerun them.

Historical course-root route

Run the following self-contained blocks from the course root using the course environment. Each supplies its imports and data. Core functions require one-dimensional finite numeric observations and do not silently discard NaNs, which could change the observation window. Before deleting missing values, investigate dates and causes as in Chapter 3 and record them.

9.1 Check estimates, intervals and tests together

```
import numpy as np
from scipy import stats
from quant_course.statistics_tools import (
    normal_mle_estimates, mean_t_confidence_interval, one_sample_t_test,
)

sample = np.array([-0.01, 0.01, 0.02, 0.02], dtype=float)
estimates = normal_mle_estimates(sample)
interval = mean_t_confidence_interval(sample, confidence_level=0.95)
test = one_sample_t_test(sample, null_mean=0.0)
assert np.isclose(estimates.mean, 0.01)
assert np.isclose(estimates.variance_mle, 0.00015)
assert np.isclose(estimates.variance_unbiased, 0.0002)
reference = stats.ttest_1samp(sample, popmean=0.0)
assert np.isclose(test.p_value, reference.pvalue)
assert (interval.lower <= 0.0 <= interval.upper) == (test.p_value >= 0.05)
print(estimates)
print(interval)
print(test)
```

These estimate a single-period mean, not annualized return, with no signal or order time. The interval object's `standard_error` is the mean's SE, not daily-return volatility. SciPy checks computation; you remain responsible for model assumptions.

9.2 Explicit failure for constant or missing samples

```
import numpy as np
from quant_course.statistics_tools import normal_mle_estimates

constant = np.array([0.01, 0.01, 0.01])
assert constant.var(ddof=0) == 0.0
for invalid in (constant, np.array([0.01, np.nan, 0.02])):
    try:
        normal_mle_estimates(invalid)
    except ValueError as error:
        print(type(error).__name__, str(error))
    else:
        raise AssertionError("invalid input must not produce a regular Gaussian MLE")
```

A constant sample can have descriptive variance zero, yet no regular Gaussian MLE exists in parameter space $v > 0$. These statements are consistent. Replacing the error with zero-filled continuation would hide the failure.

9.3 Resampling that preserves dependence

```
import numpy as np
from quant_course.statistics_tools import (
    simulate_stationary_ar1, bootstrap_mean_draws,
    moving_block_bootstrap_mean_draws, bootstrap_mean_confidence_interval,
)

streams = [np.random.default_rng(s) for s in np.random.SeedSequence(20261017).spawn
    ↪ (3)]
observations = simulate_stationary_ar1(
    n=240, phi=0.7, innovation_std=0.01, rng=streams[0],
)
train = observations[:180]
holdout = observations[180:]
assert len(holdout) == 60
point_estimate = float(train.mean())
iid = bootstrap_mean_draws(train, n_resamples=2000, rng=streams[1])
blocks = moving_block_bootstrap_mean_draws(
    train, block_length=12, n_resamples=2000, rng=streams[2],
)
for name, draws in (("IID", iid), ("circular fixed blocks", blocks)):
    interval = bootstrap_mean_confidence_interval(draws)
    print(name, "original mean", point_estimate, "interval", interval)
```

Each method in this block uses 2000 resamples; Section 8's 4000 belong to the full script. The seeds also differ, so do not tune this output to match Section 8. The percentile-interval function returns `estimate` as the average of the finite set of resampled means, which can differ slightly from the original mean. Code therefore stores the original mean separately as `point_estimate`. The standard deviation of Section 8's 4000 resampled means estimates the original mean's sampling SE. Do not divide it again by $\sqrt{4000}$ to obtain that SE: doing so mistakes computation count for market sample size.

9.4 Bayesian updating and posterior intervals

```
import numpy as np
from scipy import stats
from quant_course.statistics_tools import beta_binomial_posterior

posterior = beta_binomial_posterior(
    successes=6, failures=4, prior_alpha=2.0, prior_beta=2.0,
)
a, b = posterior.posterior_alpha, posterior.posterior_beta
credible_interval = stats.beta.ppf([0.025, 0.975], a=a, b=b)
probability_above_half = stats.beta.sf(0.5, a=a, b=b)
expected_net_payoff = 3.0 * posterior.posterior_mean - 2.1
assert np.isclose(posterior.posterior_mean, 4.0 / 7.0)
assert expected_net_payoff < 0.0
print("posterior", posterior)
print("95% equal-tail credible interval", credible_interval)
print("posterior P(p > 0.5)", probability_above_half)
print("posterior expected net payoff, units per trade", expected_net_payoff)
```

This is a 95% equal-tail credible interval, leaving 2.5% posterior mass at each end. It is neither a frequentist coverage-guaranteed interval nor a highest-posterior-density interval. `.sf(0.5)` gives posterior probability under the specified Beta–Binomial model and prior, with a meaning different from a t-test p-value.

10. Visualization

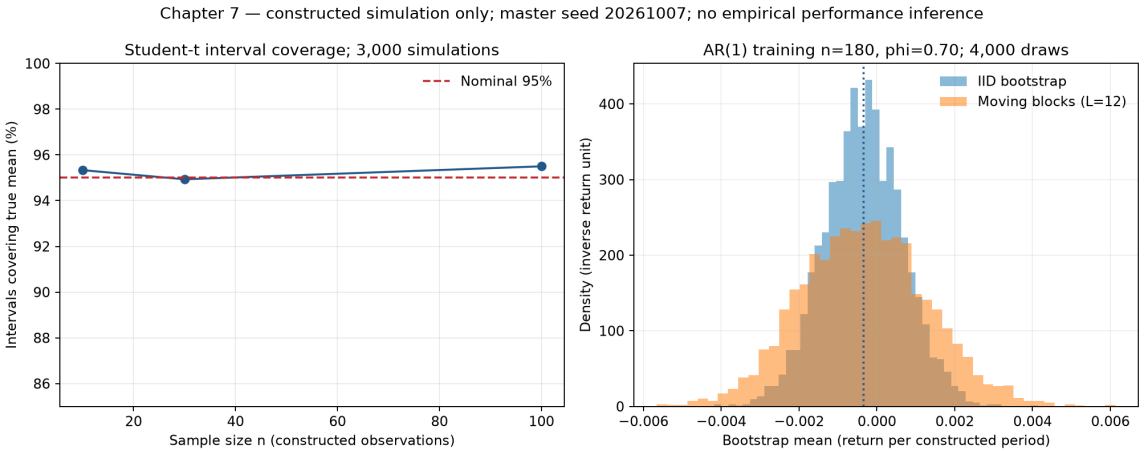


Figure 1: Repeated IID t coverage and two resampling methods for dependent observations

The left panel's horizontal axis is sample size per simulation and its vertical axis the proportion of 3000 independent experiments covering the true mean. The dashed line is nominal 95%. Observe finite-simulation fluctuations around that level; one point slightly above it does not establish superiority. This checks the specified IID Gaussian experiment, not normality or independence of real returns.

The right panel shows resampled means from the first 180 AR(1) observations, in decimal-return units. Its vertical axis is density, in reciprocal-return units; each histogram has area

approximately 1. Blocks are more dispersed in this sample because they retain some positive serial dependence. They include no future 60 observations and use no holdout outcomes to tune block length.

Distinguish raw-return distribution, the mean's sampling distribution, and the finite-sample bootstrap approximation. The figure shows the third, not a predictive interval for the next return.

11. Real Market Example

Use the saved French 10-industry monthly-return snapshot, prespecify January 2010–December 2019 as the descriptive window, and select Non-durables (NoDur). Fix the range and verify consecutive months before computing mean and an IID reference interval. "Reference" matters: executing code does not establish independent normal observations in this real-market sequence.

```
from pathlib import Path
import numpy as np
import pandas as pd
from quant_course.statistics_tools import mean_t_confidence_interval

raw = pd.read_csv(
    Path("data/processed/french_10_industries_monthly.csv"), dtype={"month": "string"
    ↪ },
)
monthly = raw.set_axis(pd.PeriodIndex(raw.pop("month"), freq="M"), axis="index")
series = monthly.loc["2010-01":"2019-12", "NoDur"]
expected = pd.period_range("2010-01", "2019-12", freq="M")
assert series.index.equals(expected)
assert np.isfinite(series.to_numpy()).all()
reference = mean_t_confidence_interval(series.to_numpy())
print("published monthly industry snapshot; not a PIT tradable security")
print("n", len(series), "sample mean", float(series.mean()))
print("IID Gaussian reference interval; assumptions unverified", reference)
print("lag-one sample correlation, diagnostic only", float(series.autocorr(lag=1)))
```

No automatic `.dropna()` occurs; missing data fail the checks. Monthly labels identify published portfolios' return months, not proof that data were public at that month-end. The recorded snapshot calculation gave monthly mean about 1.009917%, an IID-reference 95% interval about [0.428369%,1.591465%], and lag-one sample correlation about −0.188911; see the snippet record. These are window-specific calculations, not independence evidence. Near-zero sample lag correlation does not prove independence, and volatility clustering can retain dependence elsewhere. Chapters 16–18 examine serial and volatility structure; Chapter 8 introduces HAC inference.

Industry-portfolio return is not active-strategy excess return, and these data do not subtract the tutorial's actual trading costs. Record the outputs as descriptive statistics and reference-model calculations for that window, rather than claiming future monthly means will lie between the endpoints. The downloaded version can revise history; point-in-time (PIT) conditions remain unmet. Cache notes record provenance and hashes.

12. Common Mistakes

12.1 Treating a p-value as null-hypothesis probability

An incorrect claim is: "p=0.03 means only a 3% chance that zero returns are true." Testing assumes the null and calculates a statistic's tail probability; it supplies no prior or posterior for the hypotheses. Report model, statistic, two-sided rule, p-value, effect estimate and interval.

12.2 Reading a mean interval as tomorrow's return range

A mean confidence interval estimates a parameter. A prediction interval also includes a new observation's randomness. More precise mean estimates can coexist with a wide distribution of tomorrow's return. Confusing the two understates single-period loss risk.

12.3 Treating more bootstrap replications as more data

> **Common mistake: resampling count is not market sample size**
>
> Resampling the same 180 days a hundred thousand times still uses information from those 180 observations. More replications stabilize approximate quantiles. They cannot fix omitted delisted stocks, future filings used as inputs or unmodeled regime changes.

12.4 Dropping NaNs without reporting selection

Missing crisis-period prices can raise a mean if their dates are deleted. Missingness itself may relate to returns. In financial data, `dropna()` selects a sample; report removed dates, proportions and causes rather than treating it as an economically neutral cleaning step.

12.5 Reporting one significant result from 100 attempts

Assume 100 independent tests, all null hypotheses true, and each Type I error exactly 5%. Probability of at least one false rejection is

$$1 - (1 - 0.05)^{100} \approx 0.994079.$$

Here 100 counts attempts, not observations. Independence is a demonstration assumption; correlated strategy tests cannot directly use this equality. Even without knowing the total error rate, selecting the smallest p-value and interpreting it as one isolated test changes the research procedure.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(20261027)
null_samples = rng.normal(0.0, 1.0, size=(100, 60))
p_values = stats.ttest_1samp(null_samples, popmean=0.0, axis=1).pvalue
raw_rejections = int((p_values < 0.05).sum())
bonferroni_rejections = int((p_values < 0.05 / len(p_values)).sum())
assert bonferroni_rejections <= raw_rejections
print("100 independent null experiments; simulation only")
print("unadjusted rejections", raw_rejections)
print("Bonferroni rejections", bonferroni_rejections)
print("all-null independent probability of at least one raw rejection", 1 -
    ↪ 0.95**100)
```

Bonferroni correction divides the family's target level by test count and uses the union bound to control at least one Type I error. It does not require independence, but may be conservative. This example defines the test family in advance. Unrecorded features, windows and universes mean the final 100 retained tests may not be the full attempt count. Chapter 32 covers multiple testing, selection-adjusted performance and overfitting probability.

13. Quant Research Application

13.1 Write an inference contract before experimenting

For a fixed signal's mean net return, specify population, universe formation, dates, return interval, data availability, execution time, costs, test target, one-/two-sided rule, dependence treatment and parameter-search scope. Freeze splits and record every attempt. These details let a reader identify the question a result addresses.

These pure inference examples create no holdings, so `signal_time` and `trade_time` are inapplicable. Strategy returns instead must come from previously specified executable holdings and a cost model. The statistics module does not detect upstream signal leakage; Chapters 29–31 address backtesting and validation.

13.2 A rejection region is not an investment decision

Investment decisions need estimated net effects, uncertainty, tail/liquidity risks and resource constraints. Nonrejection can lead to further data collection; rejection can still lead to stopping because of capacity, exposures or failed replication. Separate statistical outputs from research-team decisions.

A mean-only model does not establish safe extreme drawdowns. Tail observations are scarce, and resampling can repeatedly replay the same few extremes. Chapter 33 develops VaR, Expected Shortfall and stress scenarios.

13.3 Study robustness rather than search for the best p-value

Prespecify reasonable block lengths, economic subperiods and cost scenarios, and report all of them. Trying settings until an interval excludes zero and reporting only that setting creates another parameter search. Sensitivity analysis should explain which assumptions matter, not select the most favorable conclusion.

14. Chapter Summary

Parameters are study targets, statistics are sample functions, and estimates are realized numbers. Unbiasedness, consistency and low MSE address different questions. Variance MLE divides by n; unbiased sample variance divides by $n-1$, with its unbiasedness relying on IID assumptions.

Confidence intervals describe procedural coverage and p-values null-model extremeness. Exact t inference requires independent normal observations. Dependence changes sample-mean variance; suitable block resampling or HAC can address some dependence under assumptions and diagnostics. A Bayesian posterior expresses conditional uncertainty differently; posterior win rates still cannot replace net-payoff analysis.

Completion means being able to state an output's object, units, window, assumptions and interpretation, beyond obtaining a test-function result.

15. Exercises

7-E01 (identify objects). A researcher says an industry's 120 monthly returns over 2010–2019 average 1%. Identify observations, estimator, estimate and possible estimand. Does the sample mean automatically equal a future population mean?

7-E02 (hand calculation). For $(-0.01, 0.01, 0.02, 0.02)$, calculate mean, squared-deviation sum, variance MLE, unbiased variance, mean SE and 95% t interval. Why do the two variances differ?

7-E03 (MSE). Two scalar estimators have bias/variance $(0, 0.04)$ and $(0.10, 0.01)$. Which do you choose under squared loss? Does this make unbiased estimators universally undesirable?

7-E04 (interval language). Distinguish "95% of parameters lie in this computed interval" from "the procedure covers a fixed parameter in about 95% of repeated samples." Also distinguish mean intervals and prediction intervals for new returns.

7-E05 (effective sample size). For stationary AR(1) with $n = 180, \phi = 0.7$, calculate large-sample mean effective sample size and SE relative to IID. Repeat for $\phi = -0.3$ and state assumptions.

7-E06 (multiple tests). For 20 independent true-null tests each at 5%, find the probability of at least one false rejection and the per-test Bonferroni level for family level 5%. Which conclusion does not require independence?

7-E07 (prior and payoff). Update Beta(2,2) with 6 wins and 4 losses. Give posterior distribution and mean. With win payoff 1, loss 2 and cost 0.1, find posterior expected net payoff and the success probability required for positive expectation.

7-N01 (identify objects). Study a fixed rule's mean daily return over a specified window. Identify unknown population parameter, random variable, realized observation, estimator, estimate and sampling unit. Explain repeated sampling here and the information a sampling mechanism must preserve for neighboring-dependent returns.

7-N02 (one calculation). A prespecified right-sided test observes $z = 1.50$ with $\Phi(1.50) = 0.9332$. Compute the one-sided p-value and decision at $\alpha = 0.05$. What does this say about the probability of the null?

7-N03 (complete calculation). Each unit is one independent measurement in common output units. Four IID normal observations are $(1, 1, 3, 3)$ and known population standard deviation $\sigma = 2$. Test $H_0 : \mu = 0$ against $H_1 : \mu > 0$ at $\alpha = 0.05$ with exact critical value $z_{0.95} \approx 1.645$. Given approximate CDF values $\Phi(1.645) \approx 0.9500$, $\Phi(2) \approx 0.9772$ and $\Phi(1) \approx 0.8413$, find mean, SE, statistic, p-value, one-sided 95% lower bound and Type I error. At specified approximate alternative $\mu_1 = 2.645$, find Type II error and power.

7-N04 (code diagnosis). A prespecified right-sided normal test uses `p = 2 * (1 - Phi(z))`. When does this equal a two-sided p-value, and when does it fail? Correct it to a right-tail formula and compute the result using $z = 1.50$ and $\Phi(1.50) = 0.9332$. What happens to the two formulas for negative z?

7-N05 (research judgment). A researcher selects the best of 40 strategy versions and reports IID right-sided $p = 0.03$, gross daily mean 0.02% and daily costs 0.03%. Returns have positive serial correlation. Which statements hold, and which claims lack support? Give at least three reporting or checking requirements.

16. Programming Exercises

7-P01 (cross-check computation). Independently generate 80 normal observations with seed 20261037, mean 0.001 and standard deviation 0.02. Calculate a 95% t interval and zero-mean test with the core functions, then cross-check SciPy and assert their two-sided correspondence. Do not make the assertion depend on whether this realization is significant.

7-P02 (block-length sensitivity). With seed 20261047, generate 300 stationary AR(1) observations. Use the first 240 as training and leave 60 unused. For lengths 4, 12 and 24, use independent streams and 2000 circular block resamples, recording intervals and widths. Do not choose block length by zero-mean rejection.

7-P03 (prior sensitivity). Keep 6 wins/4 losses fixed and use Beta(1,1), Beta(2,2) and Beta(10,10). Compute posterior means, 95% equal-tail credible intervals and expected net payoffs. Explain stronger-prior shrinkage toward 0.5 and why interval changes are not additional trading evidence.

17. Challenge Problems

7-C01 (finite-sample dependence). Derive stationary AR(1) mean variance at finite n from the double sum. At $n = 180, \phi = 0.7$, compare finite-sample SE inflation with its long-run approximation and explain omitted boundary weights. Do not treat effective sample size as universal model degrees of freedom.

7-C02 (design coverage experiments). Compare IID t, IID bootstrap and circular length-12 bootstrap coverage under IID Gaussian and stationary AR(1) generation. Specify true mean, sample size, outer replications, bootstrap repetitions, random streams, coverage/width metrics, simulation error and limits for market conclusions. Do not require a preselected winner.

18. Solutions / Solution Hints

7-E01

Observations are the 120 specified monthly industry-portfolio returns. Estimator is $\bar{X} = (1/120)\sum_{t=1}^{120} X_t$ and estimate 0.01. The estimand could be monthly industry mean under a stationary generating model, or simply a fixed-window descriptive average; choose and state which. A changed future distribution prevents automatic estimation of its mean. Industry portfolios, security populations and tradable strategies are different objects.

7-E02

Sum 0.04 divided by 4 gives 0.01. Deviations -0.02, 0, 0.01 and 0.01 give squared sum 0.0006. Dividing by 4 gives MLE variance 0.00015, and by 3 sample variance 0.0002. Standard deviation is $\sqrt{0.0002} \approx 0.0141421$; divide by 2 for mean SE about 0.0070711. Multiplying by the 3-degree critical value 3.182446 gives half-width about 0.0225033 and interval [−0.0125033,0.0325033]. MLE maximizes model density; division by $n - 1$ targets unbiasedness under IID finite second moments. Different purposes give different denominators.

7-E03

The first MSE is 0.04 and the second $0.01 + 0.10^2 = 0.02$, so choose the second here. This compares squared loss at specified parameters; other losses, bias directions or parameter spaces can change the choice. It refutes universal superiority of unbiasedness, not all usefulness of unbiased estimation.

7-E04

Frequentist parameters are fixed and data/endpoints random; the interval-generating procedure repeats. After observation the interval is fixed, and assigning 95% probability to a fixed parameter needs an additional parameter-probability model. Bayesian posteriors permit parameter probabilities conditional on model and prior. Mean intervals address estimation error; prediction intervals add new-observation randomness and are usually wider. Mechanism changes can invalidate either despite their mathematical form.

7-E05

$n_{\text{eff}} \approx 180 \times 0.3/1.7 = 31.7647$, with SE inflation about $\sqrt{(1+0.7)/(1-0.7)} \approx 2.38048$. For $\phi = -0.3$, effective sample size is approximately $180 \times 1.3/0.7 \approx 334.286$ and SE ratio $\sqrt{0.7/1.3} \approx 0.733799$. Negative correlation reduces mean variability; it does not create independent observations. These results require stationary AR(1), finite second moments and a large-sample approximation, not an unconditional substitution for t degrees of freedom.

7-E06

Under independence, probability of at least one error is $1-0.95^{20} \approx 0.641514$. The Bonferroni per-test level is $0.05/20 = 0.0025$. The first equality requires independence. The union bound applies to arbitrary events, so Bonferroni's family-error bound does not; however, individual tests must be valid and the family and attempts correctly recorded.

7-E07

Posterior is Beta(8,6), with mean $8/14 = 4/7$. Expected net payoff is $3 \times 4/7 - 2.1 = -0.385714$ units/trade. Under fixed payoffs and costs, break-even success probability is 0.7 and strictly positive expectation requires $p > 0.7$. A win rate above one half does not offset larger losses and costs.

7-P01

The reference implementation follows. Changing seeds may change significance, but computational correspondence should remain.

```
import numpy as np
from scipy import stats
from quant_course.statistics_tools import mean_t_confidence_interval,
    ↪ one_sample_t_test

sample = np.random.default_rng(20261037).normal(0.001, 0.02, size=80)
interval = mean_t_confidence_interval(sample)
test = one_sample_t_test(sample)
reference = stats.ttest_1samp(sample, popmean=0.0)
assert np.isclose(test.t_statistic, reference.statistic)
assert np.isclose(test.p_value, reference.pvalue)
assert (interval.lower <= 0.0 <= interval.upper) == (test.p_value >= 0.05)
print(interval, test)
```

Scoring: self-contained data and code; a t statistic using `ddof=1`; daily-return units; no interpretation of p-values as strategy-validity probabilities.

7-P02

```
import numpy as np
from quant_course.statistics_tools import (
    simulate_stationary_ar1, moving_block_bootstrap_mean_draws,
    bootstrap_mean_confidence_interval,
)

streams = [np.random.default_rng(s) for s in np.random.SeedSequence(20261047).spawn
    ↪ (4)]
series = 0.001 + simulate_stationary_ar1(
    n=300, phi=0.7, innovation_std=0.01, rng=streams[0],
)
train, holdout = series[:240], series[240:]
assert len(holdout) == 60
for length, rng in zip((4, 12, 24), streams[1:]):
    draws = moving_block_bootstrap_mean_draws(
        train, block_length=length, n_resamples=2000, rng=rng,
    )
    interval = bootstrap_mean_confidence_interval(draws)
    print(length, interval.lower, interval.upper, "width", interval.upper - interval.
    ↪ lower)
```

Widths need not be monotone. Finite-sample and resampling randomness can change their order. Report all three prespecified settings, explain dependence missed by short blocks and fewer rearrangements with long blocks, and use neither holdout mean nor zero exclusion to select length.

7-P03

```
from scipy import stats
from quant_course.statistics_tools import beta_binomial_posterior

for prior in (1.0, 2.0, 10.0):
    posterior = beta_binomial_posterior(
        successes=6, failures=4, prior_alpha=prior, prior_beta=prior,
    )
    ci = stats.beta.ppf(
        [0.025, 0.975], a=posterior.posterior_alpha, b=posterior.posterior_beta,
    )
    payoff = 3.0 * posterior.posterior_mean - 2.1
    print("prior", prior, "posterior mean", posterior.posterior_mean, "CI", ci, "
    ↪ payoff", payoff)
```

Posteriors are Beta(7,5), Beta(8,6) and Beta(16,14), with means about 0.583333, 0.571429 and 0.533333, and expected net payoffs about -0.35, -0.385714 and -0.50. The code computes each interval. A stronger symmetric prior places greater model weight near 0.5. Observations remain the same 10 trials, so narrower intervals do not mean new market evidence.

7-C01

Each lag has $n-k$ observation pairs and AR(1) autocovariance $\gamma_0\phi^k$. The exact finite-second-moment expression is

$$\operatorname{Var}(\bar{X}) = \frac{\gamma_0}{n}\left[1 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\phi^k\right].$$

Relative to IID mean variance γ_0/n, the bracket is the variance ratio and its square root the SE ratio. Dropping boundary weights and extending the sum to infinity gives $(1+\phi)/(1-\phi)$. Errors vanish for fixed $|\phi| < 1$ and sufficiently large n. Near a unit root, dependence decays slowly and 180 observations may not be enough.

```
import numpy as np

n, phi = 180, 0.7
lags = np.arange(1, n)
finite_ratio = 1.0 + 2.0 * np.sum((1.0 - lags / n) * phi**lags)
long_run_ratio = (1.0 + phi) / (1.0 - phi)
assert finite_ratio > 0.0
print("finite variance ratio", finite_ratio, "SE ratio", np.sqrt(finite_ratio))
print("long-run variance ratio", long_run_ratio, "SE ratio", np.sqrt(long_run_ratio))
```

Finite-sample variance ratio is about 5.580247 and SE ratio 2.362255; long-run ratios are 5.666667 and 2.380476. Finite samples contain fewer high-lag pairs. This compares mean variances at known parameters, not an exact t reference distribution after estimating dependence.

7-C02

Prespecify, for example, true mean 0.001, target single-period standard deviation and $n = 180$, and separately generate IID Gaussian and stationary AR(1) samples. Matching marginal variances requires AR innovation standard deviation equal to target standard deviation times $\sqrt{1-\phi^2}$; innovation and marginal volatility differ. Use, for example, 1000 independent outer experiments per model, three intervals per sample, 1000 bootstrap repetitions and independent streams. Record methods, block length 12 and confidence 95% before running; do not revise targets afterward.

Record coverage indicators and widths; report coverage mean, average width and simulated-coverage SE $\sqrt{\hat{c}(1-\hat{c})/M}$, where $\hat{c}$ is observed coverage and M outer-replication count. If methods share an outer sample, estimate the error of coverage differences from paired differences instead of assuming independent groups. Retain all results, including block undercoverage or wide but biased intervals.

Scoring: model and parameters 20%; original/resampling randomness 20%; coverage, width and simulation error 20%; prespecified methods and complete reporting 20%; interpretation limits 20%. The experiment describes finite-sample behavior under constructed mechanisms; it cannot establish real tradability, stable market mechanisms or future strategy returns.

7-N01

The unknown parameter is fixed-rule mean daily return μ in the target population. Trading day i has random return X_i and realized observation x_i. Sample-mean random variable $\bar{X} = n^{-1}\sum_i X_i$ estimates μ; the realized $\bar{x} = n^{-1}\sum_i x_i$ is this sample's estimate. A unit is a trading day defined by the fixed rule, not any trade row. Repeated sampling obtains another equal-length sequence under the same model and examines variation in $\bar{X}$. With correlation, simulation or resampling must preserve neighboring order and dependence; independently drawing days changes the mechanism.

7-N02

One-sided p-value is $1 - \Phi(1.50) = 1 - 0.9332 = 0.0668$. Since $0.0668 > 0.05$, do not reject H_0. It is a right-tail probability of a statistic at least as large as observed under the null and test model, not the probability that H_0 is true or proof of zero effect.

7-N03

Mean is $(1+1+3+3)/4 = 2$. Known-standard-deviation SE is $2/\sqrt{4} = 1$, so $z = (2-0)/1 = 2$. Right-tail p-value is approximately $1-\Phi(2) = 1-0.9772 = 0.0228$, below 0.05; reject H_0. The one-sided 95% lower bound is approximately $2 - 1.645(1) = 0.355$, giving interval $[0.355, \infty)$ and the same decision as the one-sided test.

Under H_0, $\bar{X} \sim N(0,1)$. Exact critical value $z_{0.95}$ gives Type I error $1 - \Phi(z_{0.95}) = 0.05$. Rounded 1.645 gives about 0.049985, or 0.05 to two decimals. At specified $\mu_1 = 2.645$, Type II error is

$$\Pr_{\mu_1}(\bar{X} \le 1.645) = \Phi\left(\frac{1.645 - 2.645}{1}\right) = \Phi(-1) = 1 - \Phi(1) \approx 0.1587.$$

Using hand threshold 1.645, power is about $1 - 0.1587 = 0.8413$. With the exact quantile and unchanged $\mu_1 = 2.645$, $\beta \approx 0.15862$ and power about 0.84138. Type II error depends on the specified alternative; "test power" must include effect size.

7-N04

The correct standard-normal two-sided expression is $2[1 - \Phi(|z|)]$. Code `2 * (1 - Phi(z))` matches it only for $z \ge 0$; for $z < 0$ it exceeds 1 and is not a probability. A prespecified right-sided alternative always uses $1 - \Phi(z)$: at $z = 1.50$, p=0.0668; for $z < 0$ the right-tail probability exceeds 0.5. Do not select the tail after observing data.

7-N05

If 0.03 is the prespecified IID one-sided gross-mean p-value, it describes one right-tail null-model calculation. It neither corrects 40-version selection nor handles serial dependence. With valid costs, the daily net estimate is $0.02\% - 0.03\% = -0.01\%$, so these numbers do not support a positive net point estimate. Gross-mean p-value is not net-mean p-value, and serial correlation can distort IID SEs and p-values.

Report all 40 versions and their selection rule. If they form a prespecified test family, Bonferroni's individual threshold is $0.05/40 = 0.00125$, with stated validity assumptions and conservatism. Freeze version and target, reassess net-return uncertainty with dependence-aware methods, and evaluate a window unused for selection. Given gross mean, costs and IID p-value alone, no positive net alpha, future return or tradability claim follows.

19. Further Reading

Start with MIT 18.05's likelihood, normal-data intervals and Bayesian conjugate priors; work two small samples by hand. Then read bootstrap materials, distinguishing interval constructions rather than treating them as one method.

For time-series resampling, read Künsch (1989) and examine dependence, moments and asymptotic block-length assumptions instead of remembering only "blocks are safe." It is an original-method source, not proof that this circular implementation instantiates all its theorems. Chapters 16–18 provide useful sequence-model foundations first.

The next chapter uses regression for conditional relationships and factor exposures. Chapters 31–32 place individual inference within model selection, multiple attempts and out-of-sample evaluation.

20. References

- MIT 18.05 topics and readings: probability and statistics course structure.
- Orloff & Bloom: Maximum Likelihood Estimates: likelihood and parameter estimation.
- Orloff & Bloom: Confidence Intervals Based on Normal Data: normal mean intervals and interpretation.
- Orloff & Bloom: Conjugate Priors, Beta and Normal: Beta updating.
- MIT 18.05: Bootstrap Confidence Intervals: resampling interval materials.
- SciPy `ttest_1samp`, Beta and bootstrap: APIs and conventions; the default interval is not this chapter's explicit percentile method.
- Künsch (1989), The Jackknife and the Bootstrap for General Stationary Observations, *The Annals of Statistics*, 17(3), 1217–1241, DOI 10.1214/aos/1176347265. Publisher abstract and author publication list were checked; full-text theorems are not claimed as verified here.
- Kenneth French: 10 industry portfolios: source of the real monthly-return cache, not a PIT individual-security trading database.

The chapter source notes record verification scope and program evidence.

Chapter 8 — Regression

Thread: from market-risk exposure to predictions available in time

Thread: regression coefficients, statistical inference and out-of-sample prediction answer different questions. Verify each against its own assumptions and information times. Prerequisites: Chapter 5 on linear algebra and Chapter 7 on statistics. By the end, you can derive and fit ordinary least squares (OLS); interpret and test linear coefficients; diagnose heteroskedasticity, autocorrelation and collinearity; compare classical, HC3 (heteroskedasticity-consistent covariance of type 3) and HAC (heteroskedasticity and autocorrelation consistent) inference; and use Ridge, Lasso and Elastic Net in chronological cross-validation pipelines.

1. Learning objectives

Write responses and features as a linear model and determine when its normal equations have a unique solution. Calculate R^2 and adjusted R^2 from residual sums of squares; test individual or joint linear restrictions using t or F and interpret p-values correctly. Explain the effects of heteroskedasticity, autocorrelation and multicollinearity, choose covariance estimates for the stated error structure, and select preprocessing and penalized regression using only past observations.

Completion means following Section 9 to run `.venv/bin/python run_notebook.py notebooks/ch08_regression.ipynb` from the accepted I3 reader-package root, checking notebook outputs and figures in each new UUID execution directory. Historical course-root CLI and figure paths remain in Section 9. Explain why this contemporaneous simulated sample establishes neither causality, empirical CAPM validity nor tradable returns.

2. Why a quant needs regression

Regression serves three roles in research: describing conditional association, testing a pricing or risk hypothesis, and measuring prediction error on held-out observations. Their code may differ by only a `fit` or `predict` call, but their statistical meanings differ. Coefficients describe model-conditional slopes; standard errors and tests depend on error structure; out-of-sample error also requires that prediction inputs were available at the relevant time. “The regression is significant, so the strategy is profitable” skips economic interpretation, information availability, transaction costs and execution checks.

Start with a linear model relating asset excess returns to market excess returns. Use the same constructed data to study classical and robust standard errors, collinearity and regularization. This extends Chapter 7’s sampling inference and prepares for real French-factor analysis in Chapter 15 and Project 2. The capital asset pricing model (CAPM) connects regression to financial interpretation; empirical analysis with real observations belongs to those later chapters.

3. Intuition

Before fitting a line, establish what one row means in the financial example:

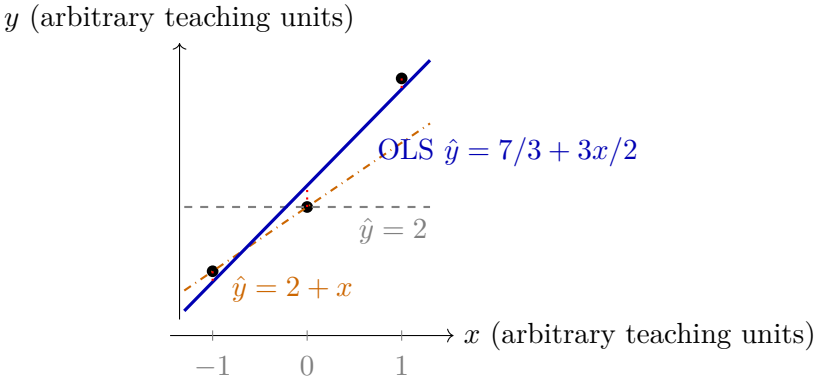
Object	Meaning here	Units and timing
Observation unit	One paired market/asset observation; 400 simulated rows	One consecutive UTC calendar day per row; not a trade, a guarantee of independent sampling, or an actual trading day
Explanatory variable	Market excess return x_t	Daily decimal return: 0.01 means 1%; the same observation period t as the response
Response variable	Asset excess return y_t	Daily decimal return recorded contemporaneously with the feature
Timing	x_t and y_t are realized returns in period t	The market return is known only at period end. It cannot predict that period's asset return at period start. Next-period prediction needs features already available at signal time to predict y_{t+1}

Rows are time-ordered and errors may be correlated across them. A row is not automatically an independent draw. This construction defines no signal, execution or holding interval.

First isolate fitting with three hand-calculable observations $x = (-1, 0, 1)^T$, $y = (1, 2, 4)^T$. Their units are arbitrary teaching units, not daily decimal returns or actual CAPM observations. Compare fitted values, vertical residuals $e_i = y_i - \hat{y}_i$ and squared loss for three candidate lines:

Candidate line	Fitted values $\hat{y}$	Residuals $e = y - \hat{y}$	Squared loss $RSS = \sum_i e_i^2$
$\hat{y} = 2$	$(2, 2, 2)$	$(-1, 0, 2)$	5
$\hat{y} = 2 + x$	$(1, 2, 3)$	$(0, 0, 1)$	1
$\hat{y} = 7/3 + 3x/2$	$(5/6, 7/3, 23/6)$	$(1/6, -1/3, 1/6)$	1/6

The diagram compares these lines against the same three points. Dotted vertical segments show OLS residuals; the OLS line has the smallest squared loss on these observations.



Squaring prevents positive and negative residuals from cancelling and gives larger deviations more weight. Loss has squared y units. For an arbitrary line $a + bx$, define

$$Q(a, b) = \sum_{i=1}^{3} (y_i - a - bx_i)^2.$$

Differentiate with respect to both scalar parameters and set the derivatives to zero:

$$\frac{\partial Q}{\partial a} = -2 \sum_i (y_i - a - bx_i) = 0, \qquad \frac{\partial Q}{\partial b} = -2 \sum_i x_i (y_i - a - bx_i) = 0.$$

Here $\sum_i x_i = 0$, $\sum_i y_i = 7$, $\sum_i x_i^2 = 2$, $\sum_i x_i y_i = 3$, so the equations reduce to $3a = 7$, $2b = 3$ and yield $a = 7/3$, $b = 3/2$. Next express these scalar conditions using a design matrix, then generalize to normal equations with n rows and k columns.

Return to the financial setting: record market excess return x_t and asset excess return y_t for each constructed daily period. An upward-sloping scatter indicates that observations with higher market returns tend to have higher asset returns. The intercept α is the model's conditional mean at $x = 0$; slope β gives the fitted average change in y for a one-unit increase in x. The vertical distance from a point to the line is its residual.

Three limits matter. A fitted association does not exclude omitted variables or reverse causality. Error variance and temporal dependence determine whether standard errors are appropriate. If x_t and y_t are contemporaneous returns, x_t is known only at period end and cannot predict y_t at its start. The code holds out the final 120 observations chronologically, but the variables remain contemporaneous. It demonstrates held-out conditional fitting error, not an executable forecast at period start.

4. Mathematical foundations

Let n be observation count, p feature count excluding the intercept, and $k = p+1$ parameter count including it. Design matrix $X \in \mathbb{R}^{n \times k}$ has an initial column of ones followed by feature columns. Response $y \in \mathbb{R}^n$, coefficients $\beta \in \mathbb{R}^k$ and errors $\varepsilon \in \mathbb{R}^n$ satisfy

$$y = X\beta + \varepsilon.$$

In the financial example, y and the market feature are daily excess returns in decimal form; 0.01 represents a single-period return of 1%. The response is asset excess return and the feature market excess return. The slope is dimensionless; intercept and residuals have daily decimal-return units, as does y. Changing to percentage points or another frequency changes parameter and error units.

The core algebraic condition is full column rank of X: $rank(X) = k$. No column can be expressed exactly by the others, and $X'X$ is invertible. Statistical inference additionally needs error assumptions; prediction also needs inputs available at the prediction time.

5. Definitions

Definition and assumptions: OLS, exogeneity and exact inference

Ordinary least squares (OLS) chooses coefficients $\hat{\beta}$ to minimize residual sum of squares:

$$\hat{\beta} = \arg\min_b (y - Xb)'(y - Xb).$$

A constant design column provides an intercept and permits a nonzero conditional intercept. Its presence alone does not guarantee uniqueness: the entire design must have full column rank. A constant feature or a column exactly determined by others destroys coefficient uniqueness. Omitting the intercept instead forces the fitted line through the origin.
Model $y = X\beta + \varepsilon$ specifies the observation relation. Conditional exogeneity $E[\varepsilon|X] = 0$ gives conditional mean $E[y|X] = X\beta$ and, with full column rank, unbiased OLS coefficients conditional on X. **Gauss–Markov assumptions** additionally require constant conditional error variance and uncorrelated errors across observations; OLS then has minimum variance among linear unbiased estimators satisfying these conditions. **Normal errors** are unnecessary for calculating OLS or obtaining unbiasedness under exogeneity. With $\varepsilon|X \sim N(0, \sigma^2 I_n)$ and the remaining model conditions, classical small-sample t/F distributions are exact.

Each output answers a different question:

- **Coefficient estimate** $\hat{\beta}$: the conditional-relation parameters minimizing this sample's residual sum of squares.
- **Standard error (SE)**: uncertainty in the coefficient estimate under the stated covariance estimator and error assumptions; not the standard deviation of observed returns.
- t/F **tests and p-values**: tests of prespecified individual or joint parameter restrictions. Reference distributions and exactness depend on error assumptions and covariance estimation.
- R^2: in-sample squared-deviation fit relative to the sample-mean benchmark for an intercept model; not causality, significance or future error.
- **Out-of-sample error**: prediction loss on held-out observations. It describes prospective prediction only when features were available at prediction time; it is not net trading return.

Fitted values are $\hat{y} = X\hat{\beta}$ and residuals $e = y - \hat{y}$. For an intercept model, the **coefficient of determination** (R^2) is

$$R^2 = 1 - \frac{RSS}{TSS}, \quad RSS = \sum_{t=1}^{n} e_t^2, \quad TSS = \sum_{t=1}^{n} (y_t - \bar{y})^2.$$

This ratio requires $TSS > 0$. A constant response has $TSS = 0$, leaving the ratio undefined even if coefficients can be fitted uniquely. It describes the fraction of in-sample total squared deviation explained by fitted values, not classification accuracy or causal validity. **Adjusted** R^2 accounts for parameter count:

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}.$$

This expression also requires $n > k = p+1$. Here p excludes the intercept; the denominator's degrees of freedom are $n-p-1 = n-k$. Unlike R^2, adjusted R^2 can fall when weak explanatory features are added.

Heteroskedasticity means $Var(\varepsilon_t|X_t)$ varies with observations or features. **Autocorrelation** means errors at different times have $Cov(\varepsilon_t, \varepsilon_s|X) \neq 0$. **Multicollinearity** means strong linear dependence among features. Heteroskedasticity and autocorrelation mainly invalidate classical covariance estimates; severe collinearity destabilizes coefficients, while perfect collinearity removes coefficient uniqueness.

6. Derivations

Define objective $Q(b) = (y - Xb)'(y - Xb)$. Expanding gives

$$Q(b) = y'y - 2b'X'y + b'X'Xb.$$

Differentiate with respect to vector b, using the symmetry of $X'X$:

$$\nabla_b Q(b) = -2X'y + 2X'Xb.$$

Setting the gradient to zero gives the **normal equations** $X'X\hat{\beta} = X'y$. Only full column rank of X permits the unique solution

$$\hat{\beta} = (X'X)^{-1}X'y.$$

Set $P = X(X'X)^{-1}X'$, giving $\hat{y} = Py$. Matrix P is symmetric and idempotent, hence the orthogonal projector onto the column space of X. The normal equations equivalently give $X'e = 0$: residuals are perpendicular to every design column. With an intercept, orthogonality to the constant column makes residuals sum to zero.

Conditional exogeneity gives $E[\hat{\beta}|X] = \beta$. If additionally $Var(\varepsilon|X) = \sigma^2 I_n$, then

$$\mathrm{Var}(\hat{\beta} \mid X) = \sigma^2(X'X)^{-1}, \qquad \widehat{\mathrm{Var}}(\hat{\beta} \mid X) = s^2(X'X)^{-1}, \qquad s^2 = \frac{RSS}{n-k}.$$

Require $n > k$; otherwise $RSS/(n-k)$ cannot estimate error variance using positive residual degrees of freedom. Calculating t/F statistics also needs the relevant standard error or s^2 to be positive. For example, perfect fitting and an estimated zero standard error do not yield a generally testable case. Classical standard errors are square roots of covariance-diagonal entries. For individual-coefficient null $H_0 : \beta_j = \beta_{j,0}$, the classical t statistic is

$$t_j = \frac{\hat{\beta}_j - \beta_{j,0}}{SE(\hat{\beta}_j)}.$$

Under IID normal errors and the remaining classical model conditions, it has a t distribution with $n-k$ degrees of freedom. Without normality, large-sample approximations are common, but the covariance estimator's applicability must be stated.

The two-sided $1-\gamma$ classical confidence interval is $\hat{\beta}_j \pm t_{1-\gamma/2,n-k} SE(\hat{\beta}_j)$. Its coverage is a repeated-sampling property, not a probability $1-\gamma$ assigned to a fixed parameter after observing this interval. A p-value is the probability, under the null and test assumptions, of a statistic at least as extreme as observed.

For q linear restrictions $H_0 : R\beta = r$, the classical F statistic is

$$F = \frac{(R\hat{\beta} - r)'[R(X'X)^{-1}R']^{-1}(R\hat{\beta} - r)/q}{s^2}.$$

Here R is a $q \times k$ restriction matrix and $r \in \mathbb{R}^q$. Require $1 \leq q \leq k$ and $\text{rank}(R) = q$: restrictions are nonredundant. With full column rank of X, matrix $R(X^\top X)^{-1}R^\top$ is then positive definite and invertible. Calculating this statistic also requires $n > k$ and $s^2 > 0$. Exact classical F inference additionally needs a correctly specified linear model, conditional exogeneity, homoskedastic errors with $\sigma^2 > 0$, and conditional normality. Under that classical normal model it has an F distribution with numerator degrees of freedom q and denominator degrees of freedom $n-k$. For one slope restriction, classical $F = t^2$. With robust covariance, joint hypotheses need the corresponding Wald/F test; the classical homoskedastic formula cannot simply be relabelled robust.

Heteroskedasticity-robust **HC3** covariance weights residuals by leverage h_{tt}:

$$\widehat{V}_{HC3} = (X'X)^{-1}\left[\sum_t x_t x_t' \frac{e_t^2}{(1-h_{tt})^2}\right](X'X)^{-1}.$$

Here x_t' is row t of X; every $h_{tt} < 1$ is an additional condition for directly calculating this HC3 formula. HC3 changes coefficient-covariance estimation, leaving OLS point estimates unchanged. It addresses unequal observation variances, not serial error correlation.

Complete counterexample: unique fitting with undefined inference formulas

Complete counterexample: full rank and positive residual degrees of freedom are insufficient. Take three dimensionless teaching observations $x = (0,0,1)^\top$ and $y = (0, 2, 1)^\top$ with an intercept. Direct calculation gives

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad X^\top X = \begin{pmatrix} 3 & 1 \\ 1 & 1 \end{pmatrix}, \quad (X^\top X)^{-1} = \begin{pmatrix} 1/2 & -1/2 \\ -1/2 & 3/2 \end{pmatrix}.$$

The determinant is 2, hence $\text{rank}(X) = k = 2$ and $n - k = 1 > 0$. Normal equations give $\hat{\beta} = (1,0)^\top$, fitted values $(1,1,1)^\top$, residuals $e = (-1,1,0)^\top$ and $RSS = 2$. Leverage is the diagonal of the hat matrix $H = X(X^\top X)^{-1}X^\top$:

$$h_{11} = h_{22} = 1/2, \qquad h_{33} = 1.$$

Observation 3 is the only point with $x = 1$ and is fitted exactly, so $e_3 = 0$. Its HC3 weight $e_3^2/(1-h_{33})^2 = 0/0$ is undefined. Do not arbitrarily set it to zero or treat a finite floating-point result as valid. Full column rank guarantees coefficient uniqueness; positive residual degrees of freedom permits a classical residual-variance estimate. Neither ensures nonzero denominators for all HC3 leverage corrections.

Continued: the same complete example

The same design checks two other denominators. Replace the response by constant $y = (1, 1, 1)^\top$: coefficients remain unique, but $TSS = RSS = 0$ and the chapter's R^2 definition becomes $1 - 0/0$, which cannot be calculated. Repeating "slope equals zero" as two restrictions gives

$$R = \begin{pmatrix} 0 & 1 \\ 0 & 2 \end{pmatrix}, \qquad r = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \qquad R(X^\top X)^{-1}R^\top = \begin{pmatrix} 3/2 & 3 \\ 3 & 6 \end{pmatrix}.$$

Both rows express the same restriction: $\operatorname{rank}(R) = 1$ and the final matrix has determinant zero. Retain one nonredundant restriction and use $q = 1$. Using a pseudoinverse while keeping $q = 2$ does not create two independent pieces of information. This example identifies formula domains, not empirical financial observations or validated inference.

HAC covariance estimates cross-period score dependence as well as heteroskedasticity. Bartlett weights are $w_\ell = 1 - \ell/(L+1)$, where L is the maximum included lag. HAC standard errors depend on stationarity, weak dependence, moments, sample length, kernel and bandwidth, among other conditions. Increasing L is not automatically more robust: a short truncation can miss dependence, while a long one can add estimation noise.

To see what HAC estimates, let $u_t = x_t e_t$ be the residual-score vector of length k. Define

$$S_\ell = \sum_{t=\ell+1}^{n} u_t u_{t-\ell}^\top, \qquad \widehat{S} = S_0 + \sum_{\ell=1}^{L} w_\ell (S_\ell + S_\ell^\top).$$

Here S_0 sums same-period score outer products; S_ℓ sums products separated by ℓ periods, with the transpose ensuring symmetry. The sandwich form before small-sample correction is

$$\widehat{V}_{\text{HAC}} = (X^\top X)^{-1} \widehat{S} (X^\top X)^{-1}.$$

The outer matrices map error-score covariance to parameter covariance; the middle retains cross-period joint variation. The implementation also uses the official API's small-sample correction. Do not compare software output entry by entry against this uncorrected expression. HC3/HAC here report t_{n-k} reference-tail probabilities using `use_t=True`. This is an approximate convention for dependent, heteroskedastic observations, without classical normal-model finite-sample exactness. HAC consistency for an infinite-memory process also needs bandwidth to grow at an appropriate rate with sample size. Fixed $L = 5$ can omit material longer lags.

Regularization penalizes coefficient size to reduce high variance. With an unpenalized intercept, write

$$\begin{aligned}
\text{Ridge:} \quad & \min_{a,b} \frac{1}{2n} \|y - a\mathbf{1} - Xb\|_2^2 + \lambda \|b\|_2^2, \\
\text{Lasso:} \quad & \min_{a,b} \frac{1}{2n} \|y - a\mathbf{1} - Xb\|_2^2 + \lambda \|b\|_1, \\
\text{Elastic Net:} \quad & \min_{a,b} \frac{1}{2n} \|y - a\mathbf{1} - Xb\|_2^2 + \lambda \left(\rho \|b\|_1 + \frac{1-\rho}{2} \|b\|_2^2 \right).
\end{aligned}$$

$\lambda \geq 0$ controls penalty strength and $0 < \rho < 1$ mixes L1 and L2 penalties. Software scales losses and penalties differently; alpha values are not comparable without their objective conventions.

Changing feature units changes coefficients and their effective penalties. Standardize features with mixed scales. Put the scaler inside each time-series cross-validation pipeline so its mean and standard deviation are fitted only on that fold's training rows. Choose penalties using validation folds inside the training window, then score the final holdout once after selection.

6.1 Why Ridge shrinks coefficients

Temporarily omit the intercept, or center features and response using training means to remove the unpenalized intercept. With loss $1/(2n)$ and penalty $\lambda\|b\|_2^2$,

$$-\frac{1}{n}X^\top(y - Xb) + 2\lambda b = 0, \qquad \hat{b}_{\text{Ridge}} = (X^\top X + 2n\lambda I)^{-1}X^\top y.$$

I is the identity in p feature dimensions. For $\lambda > 0$, the eigenvalues are $d_j^2 + 2n\lambda$, bounded below by $2n\lambda$. Curvature rises in small-singular-value directions; stability still depends on λ. For centered design decomposition $X = UDV^\top$ with nonnegative singular values d_j,

$$\hat{b}_{\text{Ridge}} = V \operatorname{diag}\left(\frac{d_j}{d_j^2 + 2n\lambda}\right) U^\top y.$$

Relative to OLS's $1/d_j$ in nonzero singular directions, each direction is multiplied by $d_j^2/(d_j^2 + 2n\lambda)$, between 0 and 1. Smaller singular values receive more shrinkage. Added bias can reduce prediction variance but need not help every holdout window. scikit-learn Ridge uses a loss not divided by $2n$, so its `alpha` corresponds to $2n\lambda$ here. Lasso uses `alpha` $= \lambda$; Elastic Net uses `alpha` $= \lambda$ and `l1_ratio` $= \rho$. Different objective scales prevent direct comparison of hyperparameter magnitudes.

Lasso's absolute value is nondifferentiable at zero, so the smooth derivative above does not apply there. For a standardized one-dimensional feature $x^\top x/n = 1$, set $z = x^\top y/n$. Removing terms independent of b leaves objective

$$\frac{1}{2}(b - z)^2 + \lambda|b|.$$

For $b > 0$ the derivative is $b - z + \lambda$, whose candidate $z - \lambda$ is feasible only if $z > \lambda$. For $b < 0$ the derivative is $b - z - \lambda$, whose candidate $z + \lambda$ is feasible only if $z < -\lambda$. At zero, the absolute-value subgradient is $[-1, 1]$, so zero is optimal when $|z| \le \lambda$. Combining the three cases gives soft thresholding:

$$\hat{b} = \operatorname{sign}(z)(|z| - \lambda)_+, \qquad (a)_+ = \max(a, 0).$$

A penalized zero coefficient does not prove zero economic effect. Lasso selections can change with the sample when features are highly correlated. Elastic Net's L2 component can mitigate some instability, but still requires out-of-sample assessment and discussion of selection uncertainty. Chapter 39 develops penalized financial prediction pipelines.

7. Financial interpretation

The CAPM single-factor regression is

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_i(R_{m,t} - R_{f,t}) + \varepsilon_{i,t},$$

R_i is asset return, R_f risk-free return and R_m market return; excess returns use a common frequency. β_i describes linear exposure to market excess return, and α_i the conditional intercept when market excess return is zero. CAPM's economic theory comes from equilibrium assumptions. Historical regression estimates a particular sample, proxy and frequency; estimated α alone establishes neither persistent abnormal return nor a causal premium.

The plotted construction sets true $\alpha = 0.0001$ per day and $\beta = 1.2$, while training estimates are 0.00064027 and 1.124669. Finite samples and correlated errors produce differences between estimates and generator parameters. These truths belong to the simulator, not real markets. Frequency, market proxy, asset universe, risk-free definition and sample period all affect economic interpretation.

Statistical significance does not establish trading value. Regression null $H_0 : \beta = 0$ concerns a conditional linear relation, not net return, market impact, borrowing, capacity or execution quality. Risk attribution needs stability and intervals; prediction needs features available at the forecast time; trading additionally needs positions, orders, holding periods and costs.

8. Numerical Example

Complete example: three-point OLS and fit statistics

Section 3 introduced these points through candidate-line calculations and scalar first-order conditions. Here matrix equations, centering and fit statistics cross-check them. Units remain arbitrary teaching units, not daily decimal returns or actual CAPM data. Take $x = (-1, 0, 1)^T$ and $y = (1, 2, 4)^T$ with intercept design $X = [\mathbf{1}, x]$.
In matrix form, the same problem uses $X = [\mathbf{1}, x]$, giving

$$X^\top X = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}, \qquad X^\top y = \begin{pmatrix} 7 \\ 3 \end{pmatrix}, \qquad X^\top X\hat{\beta} = X^\top y.$$

Full column rank makes the normal-equation solution unique: $\hat{\beta} = (7/3, 3/2)^\top$.
Centering provides an independent check of slope and intercept. Since $\bar{x} = 0$,

$$\hat{\beta}_1 = \frac{\sum_t (x_t - \bar{x})(y_t - \bar{y})}{\sum_t (x_t - \bar{x})^2} = \frac{-1 + 4}{2} = 1.5, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = \frac{7}{3}.$$

Fitted values are $(5/6, 7/3, 23/6)$ and residuals $(1/6, -1/3, 1/6)$. Thus $RSS = 1/6$, $TSS = 14/3$ and $R^2 = 1 - (1/6)/(14/3) = 27/28 \approx 0.964286$. With $n = 3, p = 1$, adjusted $R^2 = 1 - (1 - R^2) \times 2/1 = 13/14 \approx 0.928571$. These arbitrary-unit observations check algebra only; they are not financial observations.

The larger fixed-seed construction has 400 UTC calendar-day observations from 2024-01-02 through 2025-02-04: 280 for training and 120 held out. Generator parameters are $\alpha = 0.0001$, $\beta = 1.2$; errors combine innovation scales depending on absolute market return with an AR(1) recursion at $\phi = 0.55$. One-factor training OLS gives $R^2 = 0.776925$ and adjusted $R^2 = 0.776123$. For a zero-slope null, standard errors are 0.036144 classical, 0.052391 HC3 and 0.055402 HAC(5), with two-sided p-values approximately 1.474×10^{-92}, 5.966×10^{-61} and 8.014×10^{-57}. Point estimates coincide; uncertainties and test results differ. These values describe constructed data and selected covariance conventions, not null-hypothesis probabilities or significant alpha in any real asset.

9. Python Implementation

Default entry in the accepted reader package

From the accepted I3 reader-package root, create its isolated environment following the README, then run:

```
.venv/bin/python run_notebook.py notebooks/ch08_regression.ipynb
```

The notebook calls the packaged shared implementation. Its default route reads no omitted French data and downloads none. Each execution and its figures go to a unique UUID runs directory, whose actual location is printed. Old production tests and the historical commands below belong to the course working project; readers use the package-root notebook and its assertions. Optional historical data follows the reader's data contract; this batch did not rerun it.

The I3 package lacks the historical code/ch08_regression.py. Its default notebook and regression_tools implementation are included; do not alter I3 to restore a historical path.

Historical course-root route

Implementation resides in `regression_tools.py`: time-index and feature validation, `fit_ols_with_robust_covariance`, fixed-seed generator `simulate_capm_like_data` and training-only model selector `select_regularized_models`. Features and response must have exactly aligned, unique, ascending, timezone-aware indexes. Inputs must be finite real numbers; missing values are neither dropped nor filled silently. Full rank is checked after adding the intercept, and HAC's maximum lag must be below observation count. This batch's revised shared-implementation candidate additionally computes leverage from rowwise squared left singular vectors. Machine precision times the largest design dimension identifies leverage indistinguishable from one and explicitly rejects the HC3 denominator boundary; computable high-leverage samples are not rejected by a broad default closeness rule. This revision is intended for later bilingual assembly; the frozen accepted I3 module has not been rewritten. Input checks still do not certify valid inference. The teaching wrapper requires equal elapsed timestamp intervals, rejecting missing periods, irregular events and unequal month-length labels compressed into equally spaced rows. It explicitly rejects complex inputs. Empirical monthly, trading-day or event-time tasks need a declared observation clock and an appropriate implementation; do not fabricate zero returns to pass validation.

From the course-project root, run the full example and plotting route independently:

```
.venv/bin/python tools/run_a2.py --chapter 8
```

The following self-contained script runs from that project root and shows the API, training slice, covariance comparison and units:

```
from pathlib import Path
import sys
import numpy as np

sys.path.insert(0, str(Path("src").resolve()))
from quant_course.regression_tools import (
    fit_ols_with_robust_covariance,
    simulate_capm_like_data,
)

data = simulate_capm_like_data(n=400, seed=20261008, alpha=0.0001, beta=1.2)
train = data.iloc[:280]
fit = fit_ols_with_robust_covariance(
    train[["market_excess"]], train["asset_excess"], hac_maxlags=5
)
for name, result in (("classical", fit.classical), ("HC3", fit.hc3), ("HAC(5)", fit.
    ↪ hac)):
    params = np.asarray(result.params)
    standard_errors = np.asarray(result.bse)
    print(name, "alpha/day", float(params[0]),
          "beta", float(params[1]),
          "beta_SE", float(standard_errors[1]))
```

The full script reports training estimates, classical/HC3/HAC tests, near-collinear coefficients, penalties selected only within training, and final holdout MSE, and creates the figure. Dependencies are pinned in the course's local `.venv`; the corresponding standalone notebook is `ch08_regression.ipynb`. Simulation frequency is one constructed observation per UTC calendar day, not an exchange session. `market_excess_t` and `asset_excess_t` share the same row and period. The code defines no executable `signal_time`, `trade_time` or real holding interval.

10. Visualization

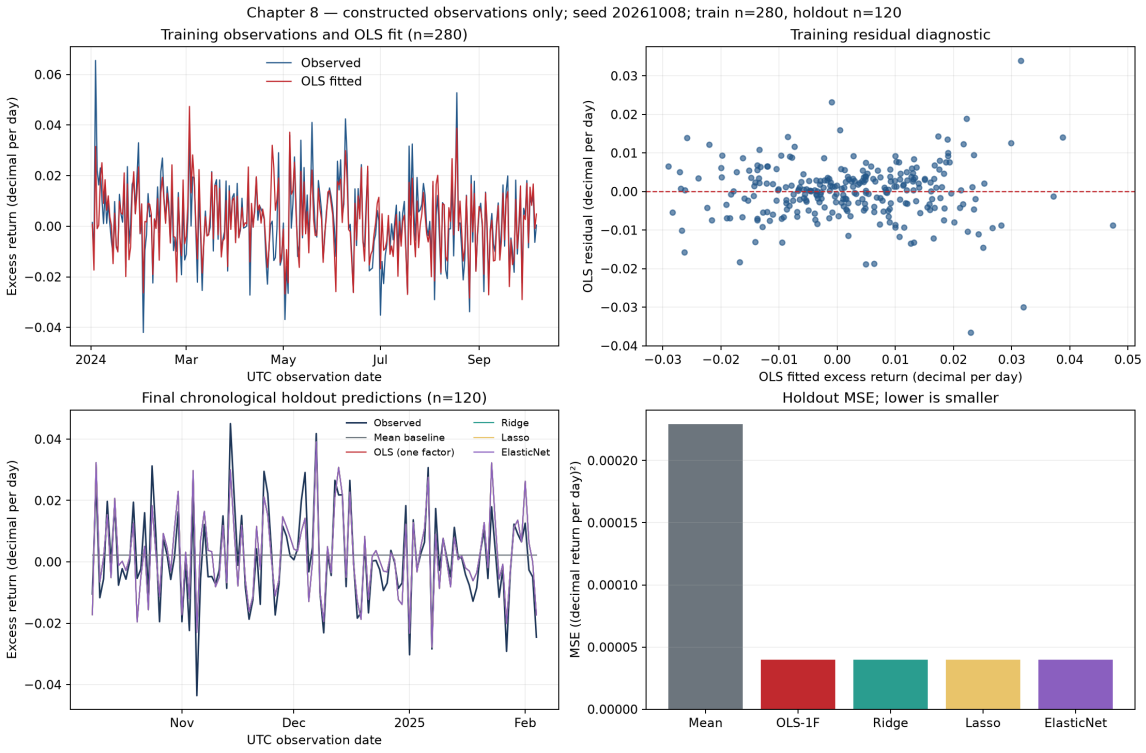


Figure 1: Contemporaneous constructed-data regression diagnostics and holdout error

The script generated this four-panel figure. Upper left: UTC observation dates against daily decimal asset excess returns, comparing 280 training observations with OLS fits. Upper right: training residuals against fitted values. Lower left: 120 held-out observations and contemporaneous conditional fits from the mean, OLS, Ridge, Lasso and Elastic Net models. Lower right: their holdout mean squared errors, measured in squared daily decimal-return units. All panels use the same fixed-seed construction; dates are observation labels.

Classical, HC3 and HAC results share an OLS slope but different uncertainty. Near-duplicate features destabilize individual slope decompositions. Penalized-model MSEs are close to one-factor OLS here. The last panel measures numerical prediction error, omitting costs and trading outcomes. Because features and response are contemporaneous, its "holdout predictions" cannot be executed at the beginning of the same return interval.

11. Real Market Example

This chapter reports no real-market regression and does not call constructed data an empirical CAPM test. The course controller previously prepared a real French 10 Industry monthly-return cache; numerical empirical work belongs to Chapter 15 and Project 2, without redownload or re-estimation here. Sharpe's original CAPM paper supplies equilibrium-theory background, not evidence for this simulation. Future use of real returns needs a download URL, snapshot date, sample interval, frequency, units, risk-free and market-proxy definitions, version, license boundary and SHA-256. Report cleaning and estimates separately from this construction.

12. Common Mistakes

Omitting the intercept. `statsmodels.OLS` does not add a constant automatically. Forcing the model through the origin changes the slope when the conditional mean permits an intercept. Incorrect call:

```
import numpy as np
import statsmodels.api as sm
x = np.array([1., 2., 3.])
y = 1. + 2. * x
fit_without_intercept = sm.OLS(y, x, missing="raise").fit()
print("slope through origin", fit_without_intercept.params[0])
```

Correction:

```
import numpy as np
import statsmodels.api as sm
x = np.array([1., 2., 3.])
y = 1. + 2. * x
design = sm.add_constant(x, has_constant="add")
fit_with_intercept = sm.OLS(y, design, missing="raise").fit()
print("intercept, slope", fit_with_intercept.params)
```

The chapter's wrapper adds an intercept and rejects an already supplied constant feature.

> **Common mistake: robust standard errors do not repair a model**
>
> **Treating robust standard errors as model repair.** HC3/HAC change estimated coefficient uncertainty, without correcting omitted variables, wrong functional form, nonstationary trends or future-information leakage. HC3 does not automatically handle autocorrelation. Do not pick the standard error that makes results most significant after inspecting outputs. State assumptions, covariance type and finite-sample limitations from the data structure first.

Standardizing all data before splitting. This leaks future means and scales into historical models. Chronological validation puts `StandardScaler` inside `Pipeline` and gives the entire pipeline to the searcher in each fold. scikit-learn's documentation explains that preprocessing `fit` on test observations leaks data. SK_PITFALLS

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.linear_model import Ridge
import numpy as np

X_train = np.arange(40., dtype=float).reshape(-1, 1)
y_train = 0.2 * X_train[:, 0] + np.sin(X_train[:, 0])

pipe = Pipeline([("scale", StandardScaler()), ("model", Ridge())])
cv = TimeSeriesSplit(n_splits=5, test_size=4, gap=1)
search = GridSearchCV(pipe, {"model__alpha": [1e-3, 1e-2, 1e-1]}, cv=cv,
                      scoring="neg_mean_squared_error")
search.fit(X_train, y_train)
```

```
print(search.best_params_, -search.best_score_)
```

Randomly shuffling time series. Ordinary random K-fold splits can train on later observations to predict earlier ones. `TimeSeriesSplit` preserves forward ordering. `gap=5` excludes five rows between training end and validation start; it does not automatically resolve arbitrary target overlap, revisions, suspensions or irregular indexes. The helper therefore requires equally spaced timestamps. Empirical finance additionally needs folds and purging rules based on the actual trading calendar.

Calling a significant p-value or high R^2 a strategy return. A p-value is a null-model tail probability for a statistic at least as extreme, given the prespecified test conditions; it is not the probability the null is true. Similarly, R^2 does not measure return after costs. Here y_t and x_t are contemporaneous, so these results establish no ability to predict y_t at period start.

13. Quant Research Application

An auditable workflow first specifies whether the question concerns contemporaneous systematic exposure, future-return prediction or a trading signal. Freeze the universe, return and factor definitions, timezone alignment, risk-free series, frequency and availability times. Audit missingness, anomalies, adjustments, delistings and revisions. Reserve training and final holdout intervals; do not select features, thresholds or penalties on the final holdout. For exposure, prespecify $H_0 : \beta = 0$ and report coefficients, covariance estimator, intervals, residual diagnostics and coefficient stability.

The construction uses 280 training rows for OLS and chronological cross-validation. `TimeSeriesSplit(n_splits=5, test_size=30, gap=5)` produces five consecutive 30-row validation windows with expanding training sizes. The full training interval is 2024-01-02 through 2024-10-07 UTC; the final 120-row holdout spans 2024-10-08 through 2025-02-04 UTC. Ridge selects `alpha=0.01`, Lasso `alpha=0.00001` and Elastic Net `alpha=0.00001, l1_ratio=0.2` using training-internal validation only. Final holdout MSEs are 0.0002291795 for the mean benchmark, 0.0000400386 for one-factor OLS, 0.0000400437 for Ridge, 0.0000400481 for Lasso and 0.0000400414 for Elastic Net, all in squared daily decimal-return units. Under this generator and split, their linear conditional-fit errors are similar. This establishes neither risk-adjusted net return nor an out-of-sample causal effect.

For prospective prediction, inputs must exist when the forecast is issued. If x_t is a market return known only after the close and the target is that day's asset return, contemporaneous feature x_t is unavailable at the open. Use information through the close of t to forecast a subsequent target such as y_{t+1}, explicitly define $signal_time_t < trade_time_{t+1}$ and the holding interval, and build labels using release times, execution delays and the trading calendar. The code performs no such shift, defines no trades and subtracts no costs; MSE cannot be interpreted as strategy performance.

With variance varying by scale, report HC3 and examine high-leverage observations. With serial dependence, HAC may be appropriate under stationarity, weak dependence and sufficient sample length; assess plausible lag bandwidths. For multi-entity panels, choose entity or two-way clustered covariance according to shared error shocks and ensure enough clusters. A large design

condition number calls for checks of units, correlations and stability. Regularization can reduce prediction variance but changes the coefficient target; ordinary OLS t statistics generally cannot be assigned to coefficients selected by tuning.

Prespecify disconfirming outcomes: coefficient directions or magnitudes change materially across reasonable periods, proxies, frequencies or definitions; training-selection gains fail on the final holdout; the contemporaneous relation disappears under the available information set; error structure exceeds the chosen inference conditions; or realistic costs, capacity and borrowing make the strategy objective fail. Narrow or withdraw the claim when these occur, rather than switching significance conventions.

14. Chapter Summary

OLS minimizes residual sum of squares to estimate a linear conditional relation. Full column rank provides uniqueness; exogeneity supports unbiased interpretation. R^2 describes in-sample fit, while standard errors and tests depend on error covariance. HC3 and HAC adjust uncertainty without repairing model specification or information timing. Ridge, Lasso and Elastic Net trade bias for variance; selection and preprocessing belong inside training-only chronological validation. Financial regression conclusions must state variable definitions, availability, sample interval, error structure and interpretation limits.

15. Exercises

8-E01 (concepts). List separate conditions for a unique OLS solution, conditional unbiasedness, and exact classical small-sample t / F inference. Which steps do not require normality?

8-E02 (interpretation). Interpret α and β in a CAPM-style excess-return regression. If $\hat{\alpha} > 0$ and its classical p-value is below 0.05, list at least three conclusions that still do not follow.

8-E03 (diagnosis). Distinguish heteroskedasticity, autocorrelation and multicollinearity and explain their main effects on OLS point estimates, classical standard errors or coefficient stability.

8-E04 (calculation). An intercept model has $n = 100, p = 4, R^2 = 0.20$. Calculate adjusted R^2 and explain how adding one feature affects R^2 and adjusted R^2.

8-E05 (tests). State nulls for a single slope and a joint zero test of three market factors. In the classical single-restriction case, how do F and t relate? Interpret the p-value.

8-E06 (methods). Daily-return residuals vary in scale with fitted values and show persistence. What do classical, HC3 and HAC estimators address? Is HAC lag 5 automatically optimal?

8-E07 (time validation). What does `gap` exclude in `TimeSeriesSplit(gap=5)`? Does it guarantee leakage-free financial labels? State feature/target availability constraints and explain the purpose of panel-clustered standard errors.

8-T01 (identify objects). Each of the 400 simulated rows contains `market_excess_t` and `asset_excess_t` for the same UTC calendar day. Identify observation unit, feature, response and units. Why does this pairing not represent a forecast before the open?

8-T02 (one calculation). Given $\hat{\beta} = 1.124669$ and HAC(5) standard error $SE = 0.055402$, calculate the observed t statistic for $H_0 : \beta = 0$.

8-T03 (complete calculation). For $x = (-1, 0, 1)^T$ and $y = (1, 2, 4)^T$, list fitted values, residuals and RSS point by point for $\hat{y} = 2$, $\hat{y} = 2 + x$ and OLS. Then calculate TSS, R^2 and adjusted R^2 using the OLS line.

8-T04 (code diagnosis). Inspect Section 12's incorrect `sm.OLS(y, x, missing="raise")` call. With $y = 1 + 2x$, what does it estimate? How do you fit an intercept, and why does the slope change?

8-T05 (research judgment). A 400-row contemporaneous simulated sample gives high R^2 and a significant HAC(5) slope. State two supported and three unsupported conclusions. What timing, validation and trading elements must be fixed before investigating next-period prediction?

8-B01 (formula-domain diagnosis). For Section 6's intercept construction $x = (0, 0, 1)^\top$ and $y = (0, 2, 1)^\top$, calculate the determinant of $X^\top X$, OLS coefficients, residuals and all three leverage values. Why do $n > k$ and full column rank still fail to ensure calculable HC3? How does R^2 become undefined with a constant response? For two copies of the same slope restriction, what happens to the joint-F inverse matrix and q?

16. Programming Exercises

8-P01 (hand calculation and code). Build an intercept regression for $x = (-1, 0, 1)^T$ and $y = (1, 2, 4)^T$ in NumPy. Solve normal equations and cross-check using statsmodels; assert agreement of coefficients, RSS, R^2 and Section 8's hand calculations.

8-P02 (inference). Use the first 280 rows from simulate_capm_like_data(n=400, seed=20261008). Fit OLS with classical, HC3 and HAC(5) covariance; report intercept/slope, standard errors and p-values for $H_0 : \beta = 0$. Assert that all point estimates coincide; do not label generator truths as estimated results.

8-P03 (chronological regularization). Build a `StandardScaler` + Ridge pipeline from near-duplicate market features in the first 280 rows. Use `TimeSeriesSplit(n_splits=5, test_size=30, gap=5)` and training-internal grid selection of `alpha`. Check training/validation ordering, five-row gaps and training-fold-only scaler fitting. Score the final 120 rows only after selection, without selecting again from that score.

17. Challenge Problems

8-C01 (collinearity counterexample). Construct highly correlated features, one nearly copying the other. Compare one- and two-feature OLS coefficients, fits and condition numbers, then Ridge. Explain why large coefficient changes establish neither changed economic effects nor causality. State acceptable reporting limits.

8-C02 (research design). Write a pre-analysis validation plan for "Can a market factor predict next-period asset excess returns?" Cover timestamps/shift, training and holdout, lagged targets, error structure, HAC bandwidth sensitivity, model selection, residuals, benchmarks, execution and disconfirmation. Explain why contemporaneous construction and HAC lag 5 do not answer the question. Score timing, inference assumptions, leakage control, trading feasibility and falsifiability from 0 to 2 each, total 10.

18. Solutions / Solution Hints

8-E01 solution. Full column rank makes $X'X$ invertible and OLS unique. Conditional exogeneity $E[\varepsilon|X] = 0$ supports unbiasedness conditional on X. Classical covariance additionally needs homoskedastic, cross-row-uncorrelated errors; conditional normality makes finite-sample t / F distributions exact. Calculating OLS and obtaining unbiasedness under exogeneity do not need normality. Without it, asymptotic inference may be available under suitable large-sample and error conditions.

8-E02 solution. α is the conditional intercept at zero benchmark return and β conditional linear market exposure. A positive intercept and small classical p-value alone establish neither correct specification, freedom from heteroskedasticity/autocorrelation/multiple-testing effects, future positive performance, causal risk premium nor profitability after costs. Examine error assumptions, sample selection, proxies, stability, versions and execution.

8-E03 solution. Heteroskedasticity changes error variance, not necessarily its conditional mean. Exogeneity can preserve unbiased OLS while classical homoskedastic standard errors fail. Autocorrelation likewise may preserve unbiasedness under exogeneity while invalidating classical covariance. Multicollinearity makes $X'X$ ill-conditioned, inflates individual-slope uncertainty and creates sensitivity to samples or specification. Perfect collinearity removes full rank and unique coefficients.

8-E04 solution. $n - p - 1 = 95$, giving

$$\bar{R}^2 = 1 - (1 - 0.20)\frac{99}{95} = 1 - \frac{79.2}{95} \approx 0.166316.$$

For intercept OLS on the same observations, R^2 cannot decrease when columns are added. Adjusted R^2 penalizes lost degrees of freedom and falls if fit improvement is insufficient. Missingness, sample changes or different fitting procedures invalidate this monotonicity comparison.

8-E05 solution. The individual null is $H_0 : \beta_m = 0$; three-factor null $H_0 : \beta_1 = \beta_2 = \beta_3 = 0$ can be written $R\beta = 0$. One linear restriction in the classical model gives $F = t^2$. Its p-value is the null-model tail probability of a statistic at least as extreme under test assumptions, not the probability the null is true.

8-E06 solution. Classical covariance needs homoskedasticity and no cross-period error correlation. HC3 handles heteroskedasticity but still needs independence or suitable cross-observation conditions. HAC includes cross-period score correlation under heteroskedasticity, requiring sufficient sample length, stationarity/weak dependence and appropriate bandwidth. A fixed truncation cannot capture arbitrarily long dependence. Script setting $L = 5$ is reproducible teaching input, not automatically optimal; justify frequency/bandwidth and check sensitivity.

8-E07 solution. `gap=5` leaves five rows between training end and validation start in every fold. It does not guarantee appropriate label intervals, releases, holding periods, timezones or transformations; design windows for target horizon and feature availability. Features must be available by signal time, signals precede execution, and target returns follow execution. Panel-clustered covariance addresses error dependence within entities or time clusters; justify clusters and account for limited cluster counts.

8-P01 reference implementation. Run from the course-project root. Normal equations, statsmodels and assertions cross-check results:

```
import numpy as np
import statsmodels.api as sm
x = np.array([-1., 0., 1.]); y = np.array([1., 2., 4.])
X = np.column_stack([np.ones(3), x])
b = np.linalg.solve(X.T @ X, X.T @ y)
fit = sm.OLS(y, X, hasconst=True, missing="raise").fit()
assert np.allclose(b, [7/3, 1.5])
assert np.isclose(np.sum((y-X@b)**2), 1/6)
assert np.isclose(fit.rsquared, 27/28)
assert np.isclose(fit.rsquared_adj, 13/14)
print(b, fit.rsquared, fit.rsquared_adj)
```

8-P02 reference implementation.

```
from pathlib import Path
import sys
import numpy as np
sys.path.insert(0, str(Path("src").resolve()))
from quant_course.regression_tools import fit_ols_with_robust_covariance,
    ↪ simulate_capm_like_data

data = simulate_capm_like_data(n=400, seed=20261008, alpha=.0001, beta=1.2)
train = data.iloc[:280]
fit = fit_ols_with_robust_covariance(
    train[["market_excess"]], train["asset_excess"], hac_maxlags=5
)
for label, result in (("classical", fit.classical), ("HC3", fit.hc3), ("HAC5", fit.
    ↪ hac)):
    params = np.asarray(result.params)
    standard_errors = np.asarray(result.bse)
    p_values = np.asarray(result.pvalues)
    print(label, "alpha", params[0], "beta", params[1],
          "beta_SE", standard_errors[1], "beta_p", p_values[1])
assert np.allclose(np.asarray(fit.classical.params), np.asarray(fit.hc3.params))
assert np.allclose(np.asarray(fit.classical.params), np.asarray(fit.hac.params))
```

Training OLS gives $\hat{\alpha} = 0.00064027$ and $\hat{\beta} = 1.124669$. Slope standard errors are 0.036144 classical, 0.052391 HC3 and 0.055402 HAC(5), giving t statistics 31.116, 21.467 and 20.300. Significance does not remove the construction's violations of classical error assumptions.

8-P03 reference implementation.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path("src").resolve()))
from quant_course.regression_tools import select_regularized_models,
    ↪ simulate_capm_like_data

data = simulate_capm_like_data(n=400, seed=20261008)
train, holdout = data.iloc[:280], data.iloc[280:]
features = ["market_excess", "market_near_duplicate"]
selected = select_regularized_models(train[features], train["asset_excess"],
                                     n_splits=5, gap=5, test_size=30)
for name, choice in selected.items():
    assert choice.training_end == train.index[-1]
    assert all(fold.train_end < fold.validation_start for fold in choice.folds)
    assert all(fold.gap_n == 5 for fold in choice.folds)
    assert all(fold.validation_end <= train.index[-1] for fold in choice.folds)
    print(name, choice.best_params, choice.best_cv_mse,
          "holdout MSE computed after selection only")
    # score only after selection; do not feed this score back to the search
    pred = choice.estimator.predict(holdout[features])
    print(float(((holdout["asset_excess"].to_numpy() - pred) ** 2).mean()))
```

A five-row gap counts observations, not a universal embargo duration. The implementation places `StandardScaler` inside each pipeline; cross-validation `fit` uses only that fold's training rows. The selected pipeline is refitted on all 280 training rows. The final 120 do not enter model or hyperparameter selection.

8-C01 reference reasoning and scoring. Simulate near-duplicate features $x_2 = x_1 + \eta$ with small noise η. A single-feature model attributes shared variation to x_1; a two-feature model tries to divide almost the same signal, so coefficients can offset with large positive and negative values while fits barely change and conditioning deteriorates. L2 shrinkage usually stabilizes predictions but changes the coefficient target; it does not identify causality. The fixed-seed one-factor condition number is 84.008; near-duplicate two-feature design 135724.250, with slopes approximately $(-2.83396, 3.95871)$. Report the construction, dependence and instability, plus specification/prediction sensitivity; do not call coefficient changes economic effects. Ten points: construction/rank 2, coefficients versus fits 2, Ridge bias/stability 2, causal limits 2, sample/units 2.

8-C02 reference plan and scoring. Define timezone and release times and generate signals only after market information exists. Align x_t with future target y_{t+1}, specifying signal, execution and return intervals. Contemporaneous x_t, y_t is not a tradable forecast. Freeze chronological training, validation and final holdout; scalers, missing-value handling and selection stay inside training folds. Set gap/purge by label spans, tune penalties inside training and use holdout once. State classical homoskedastic/no-autocorrelation assumptions. Use HC3 for heteroskedasticity and theoretically motivated HAC lag choices for weakly dependent sequences, with sensitivity

checks. Examine residuals, breaks and collinearity. Compare MSE/prediction loss with mean or simple-factor benchmarks. A trading extension also needs positions, turnover, fees, slippage, borrowing, capacity and execution. Prespecify failure to replicate across periods, failure after costs, unavailable inputs or sensitivity to reasonable lags/splits as disconfirming results. Score the five dimensions 0–2 each. This construction lacks trading times and future targets, checking regression/inference implementation and a teaching pipeline only. $L = 5$ is fixed teaching input, not a universal optimum or substitute for error diagnostics.

8-T01 solution. A row is a paired record for one simulated UTC calendar day, not guaranteed independent or a trade. `market_excess_t` is the explanatory variable and `asset_excess_t` the response, both daily excess returns in decimals; 0.01 means 1%. They are realized in the same period. Market return is known at period end, so that day's `market_excess_t` cannot predict asset return at the open. A prospective design aligns available inputs with a future target such as y_{t+1}.

8-T02 solution. $t = (1.124669 - 0)/0.055402 \approx 20.300$. This is the statistic under HAC(5) covariance; a p-value additionally needs the reference distribution and degrees of freedom. It is not an exact classical finite-sample test for dependent heteroskedastic observations.

8-T03 solution. For $\hat{y} = 2$, fits $(2, 2, 2)$ and residuals $(-1, 0, 2)$ give $RSS = 1 + 0 + 4 = 5$. For $\hat{y} = 2 + x$, fits $(1, 2, 3)$ and residuals $(0, 0, 1)$ give $RSS = 1$. OLS coefficients $\hat{\beta}_0 = 7/3$ and $\hat{\beta}_1 = 3/2$ produce fits $(5/6, 7/3, 23/6)$ and residuals $(1/6, -1/3, 1/6)$, hence $RSS = (1 + 4 + 1)/36 = 1/6$. Response mean $7/3$ gives $TSS = (-4/3)^2 + (-1/3)^2 + (5/3)^2 = 14/3$, so $R^2 = 1 - (1/6)/(14/3) = 27/28 \approx 0.964286$. With $n = 3, p = 1$, adjusted $R^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1} = 1 - (1/28)\frac{2}{1} = 13/14 \approx 0.928571$.

8-T04 solution. `sm.OLS(y, x, ...)` adds no constant and estimates a constrained through-origin slope. Section 12 uses $x = (1, 2, 3)$ and $y = (3, 5, 7)$, giving $\hat{b} = \sum x_i y_i / \sum x_i^2 = 34/14 = 17/7 \approx 2.429$. With an intercept, estimates are 1 and 2. Build a two-column design using `sm.add_constant(x, has_constant="add")`, then fit it. The origin constraint forces the slope to absorb the intercept; this difference is not caused by standard errors or the optimizer.

8-T05 solution. Supported conclusions: under this constructed sample's contemporaneous pairing, linear fitting explains part of in-sample variation; using HAC(5) covariance, the sample slope is far from zero. Neither establishes that market return causes asset return, sustainable CAPM alpha, prediction of same- or next-period returns at period start, or profitability after costs. A prospective study needs availability times, signal/execution/holding intervals, target alignment, chronological training/validation/holdout, within-training-fold scaling/tuning, gap/purge for label overlap, covariance and bandwidth sensitivity, benchmarks/costs and prespecified disconfirmation.

8-B01 solution. $X^\top X = \left(\begin{smallmatrix} 3 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ has determinant 2 and inverse $\left(\begin{smallmatrix} 1/2 & -1/2 \\ -1/2 & 3/2 \end{smallmatrix}\right)$. With $X^\top y = (3, 1)^\top$, coefficients are $(1, 0)^\top$, fits $(1, 1, 1)^\top$ and residuals $(-1, 1, 0)^\top$, giving $RSS = 2$ and $n - k = 1$. Rowwise $h_{tt} = x_t^\top (X^\top X)^{-1} x_t$ yields $(1/2, 1/2, 1)$; the third HC3 weight is $0/0$. Floating-point rounding does not theoretically repair this undefined term. A constant response gives $TSS = RSS = 0$, so R^2 is undefined here; a software convention does not change that definition. Duplicate restriction matrix $R = \left(\begin{smallmatrix} 0 & 1 \\ 0 & 2 \end{smallmatrix}\right)$ has rank 1; $R(X^\top X)^{-1} R^\top = \left(\begin{smallmatrix} 3/2 & 3 \\ 3 & 6 \end{smallmatrix}\right)$ has determinant zero and no inverse. Remove one duplicate row and test a single slope with $q =$

1. Even when all denominator conditions hold, exact classical F/t inference needs a correctly specified linear model, conditional exogeneity, positive homoskedastic variance and conditional normal errors. HC3/HAC have their own inference conditions; input validation is not an inference certificate.

19. Further Reading

- **Must Know**: statsmodels' official linear-regression documentation, especially the OLS intercept, classical covariance and result attributes; scikit-learn's `TimeSeriesSplit` and leakage guidance for folds, `gap` and training-only preprocessing.
- **Should Know**: White (1980) on heteroskedasticity-consistent covariance; MacKinnon and White (1985) on finite-sample HC comparisons; Newey and West (1987) on HAC. They explain estimator construction, without certifying a particular sample's dependence or finite-sample inference.
- **Advanced**: Hoerl and Kennard (1970) on Ridge; Tibshirani (1996) on Lasso; Zou and Hastie (2005) on Elastic Net. Compare penalties, selection stability, coefficient bias and tuning uncertainty.
- **Optional / later empirical work**: Sharpe (1964) on CAPM equilibrium theory; Chapter 15 and Project 2 use real French data. Theory does not empirically validate this construction.

20. References

Course source keys and limits appear in `references.md`: `STATSMODELS` supports regression API use, `SK_PITFALLS` training-only preprocessing and `SK_TSS` chronological splitting and `gap`. API references are statsmodels OLS, robust covariance and OLS results. See TimeSeriesSplit and data-leakage pitfalls for chronological tuning.

Method background: White, H. (1980), "A Heteroskedasticity-Consistent Covariance Matrix Estimator and a Direct Test for Heteroskedasticity," *Econometrica*, 48(4), 817–838, `JSTOR/DOI`; MacKinnon, J. G. and H. White (1985), "Some Heteroskedasticity-Consistent Covariance Matrix Estimators with Improved Finite Sample Properties," *Journal of Econometrics*, 29(3), 305–325, DOI; Newey, W. K. and K. D. West (1987), "A Simple, Positive Semi-definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix," *Econometrica*, 55(3), 703–708, NBER working-paper record. Original regularization methods: Hoerl, A. E. and R. W. Kennard (1970), "Ridge Regression: Biased Estimation for Nonorthogonal Problems," DOI; Tibshirani, R. (1996), "Regression Shrinkage and Selection Via the Lasso," DOI; Zou, H. and T. Hastie (2005), "Regularization and Variable Selection via the Elastic Net," *Journal of the Royal Statistical Society: Series B*, 67(2), 301–320, DOI. CAPM theory: Sharpe, W. F. (1964), "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk," *The Journal of Finance*, 19(3), 425–442, DOI.

Chapter 9 — Optimization

Thread: how feasible two-asset weights change with budget, leverage and long-only bounds, and how the objective and KKT conditions check a solution.

Begin with constructed one-year inputs. Assets 1 and 2 have expected total returns (0.05, 0.08) and covariance matrix

$$\Sigma = \begin{bmatrix} 0.04 & 0.01 \\ 0.01 & 0.09 \end{bmatrix}.$$

Let $w_1 = x$ and $w_2 = 1 - x$. Budget $w_1 + w_2 = 1$ and long-only lower bounds $w_1, w_2 \geq 0$ imply $x \in [0, 1]$. Compare feasible points using $J(w) = \frac{1}{2}\lambda w^\top \Sigma w - \mu^\top w$ with $\lambda = 1$:

w_1	w_2	Expected total return (annual decimal)	Variance (annual decimal return^2)	Objective $J(w)$
0	1	0.08000	0.09000	−0.03500
0.5	0.5	0.06500	0.03750	−0.04625
1	0	0.05000	0.04000	−0.03000

All three points satisfy budget and long-only constraints. They indicate direction, without proving optimality over the continuous interval. w contains dimensionless weights, μ expected total returns over the same annual holding period, and Σ squared-decimal-return covariance. Risk coefficient λ has inverse-decimal-return units, making both objective terms annual decimal returns. Return from these feasible points to the full model to derive the optimum.

Prerequisites: calculus in Chapter 4, linear algebra in Chapter 5 and regression in Chapter 8. You will prove quadratic convexity, formulate least squares, LP and mean–variance QP, distinguish necessary from sufficient KKT conditions, interpret local dual shadow prices, and check CVXPY statuses and constraint residuals.

1. Learning objectives

Separate four layers. Variables identify choices, dimensions and units. The objective compresses return, risk or fitting error to a scalar. The feasible set describes budget, long-only and capacity limits. Evidence consists of solver status, primal feasibility, dual quantities and residuals supporting the conclusion.

By the end, you should be able to:

1. Distinguish objective functions, decision variables, constraints, feasible points and optimal points.
2. Prove set/function convexity by definition, second derivatives or Hessians, recognizing positive definiteness as one sufficient route to strict convexity.
3. Formulate least squares as a convex quadratic program (QP) and state rank or tangent-space conditions for uniqueness.

4. Write linear programs (LPs) and mean–variance QPs, tracking input, output, objective and dual units.
5. Derive weak duality, complementary slackness, Karush–Kuhn–Tucker (KKT) conditions and right-hand-side shadow prices from the Lagrangian.
6. Explain Slater's role in strong duality and KKT's different logical status in nonconvex and convex problems.
7. Inspect CVXPY solver status, feasibility residuals and KKT diagnostics, including `optimal_inaccurate`, infeasibility, unboundedness and the limits of handling slightly negative covariance eigenvalues.

Numerical checks concern fixed constructions: mean–variance weights $[5/11, 6/11]$ and minimum-variance weights $[8/11, 3/11]$; a 0.4 cap on each of two assets is infeasible. At risk aversion 0.2 the optimum is boundary $[0, 1]$ with first long-only-bound dual 0.014. The constructed production LP chooses 40 A batches and 20 B batches per day for 160 thousand euros of daily contribution. These are teaching numbers, not market estimates or backtest returns.

2. Why a quant needs optimization

Statistics asks what an unknown quantity might be given observations. Optimization asks which portfolio, order or resource allocation to choose given estimates and feasible choices. They connect without replacing one another. Mean–variance optimization uses expected returns μ and covariance Σ. A solver finds weights for those inputs, without verifying their estimation accuracy or that future trades can execute at the mathematically optimal weights.

Without explicit objectives, code can confuse volatility, variance, absolute loss and drawdown. Without feasible constraints, it can output impossible short positions, excessive gross exposure or overused resources. Without status and independent residual checks, a floating-point vector in `x.value` can be mistaken for a validated strategy. Researchers must define the complete problem before asking a numerical solver to solve it.

Three structures connect the chapter. Least squares expresses Chapter 8's squared-error objective as an unconstrained convex QP. LP distributes scarce resources through a linear objective on a polyhedral feasible set. Mean–variance QP chooses between expected return and risk. Each needs an objective, feasible set and optimality check, but their units, solution methods and economic interpretations differ.

3. Intuition: draw feasible choices before seeking improvement

Put asset 1's weight on a horizontal axis and asset 2's on a vertical axis. Full investment gives line $w_1 + w_2 = 1$; nonnegative weights restrict it to the segment from $(0, 1)$ to $(1, 0)$. Each point is an allowed portfolio. The mean–variance objective becomes a quadratic curve along that segment. Optimize over the segment, rather than seeking an arbitrary zero gradient in the plane.

A convex set contains the segment between any two points. With a convex objective on that set, a feasible local optimum is global; there is no inferior local valley. Strict convexity permits

at most one optimum. Algorithms still iterate and use floating-point approximations, but the mathematical structure excludes separated local valleys.

LP level sets are parallel lines, and a linear objective commonly improves toward a polygon vertex. QP contours are ellipses or degenerate ellipses; optima may be interior or constrained to a boundary. A boundary optimum generally has nonzero full-space gradient. Ask instead whether any feasible direction can reduce the objective; constrained optimality and KKT express this.

Nonconvex objectives can have multiple local extrema; nonconvex constraints can disconnect the feasible set. A convex solver does not turn an incorrectly chosen objective into a valid research question.

4. Mathematical foundations: dimensions, units and feasible choices

Definition: variables, constraints and optimal value

Start with a general minimization problem:

$$\begin{aligned} \text{minimize}_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

Here $x \in \mathbb{R}^n$ collects n decision quantities in a column vector; f_0 is the scalar objective, f_i inequality constraints and h_j equality constraints. Feasibility requires every constraint. Optimal value $p^\star$ is the infimum of objective values over feasible points. No feasible point means infeasibility; an objective decreasing along feasible points toward $-\infty$ means unboundedness.

Dimensional checks reveal modeling errors early. For x in batches/day, $Ax \le b$ needs A_{ij} in resource-hours/batch and b_i in resource-hours/day, so Ax and b match. If c_j is thousand euros/batch, $c^\top x$ is thousand euros/day. Portfolio weights $w \in \mathbb{R}^n$ are dimensionless, expected return μ_i is a decimal over the holding period, and Σ_{ij} is same-period covariance in squared-decimal-return units. $w^\top \Sigma w$ is portfolio-return variance.

The example's holding period is one year: 0.05 means expected simple return 5% over that year, and covariance concerns annual decimal returns. Neither input is rescaled independently. Risk aversion λ has inverse-decimal-return units, making $\lambda w^\top \Sigma w$ and $\mu^\top w$ both annual decimal returns. Weights summing to 1 is a budget identity, not an observed return or trade time.

Distinguish feasibility, boundedness, attainment and numerical verification. Feasibility requires at least one allowed point; boundedness a finite best objective value; attainment a feasible point realizing that value; numerical verification sufficient precision and compliance with model contracts in the returned floating-point result. A problem can be infeasible, or feasible but unbounded. `optimal_inaccurate` reports a lower-precision candidate, distinct from `optimal`. Even `optimal` returns floating-point values requiring residual and application-precision checks. Printing weights does not establish all four layers.

5. Definitions: convex problems, LP, QP and feasibility

A set C is convex when all $x, y \in C$ and $\theta \in [0,1]$ satisfy $\theta x + (1-\theta)y \in C$. A convex combination uses nonnegative weights summing to 1; fully invested long-only portfolios form a convex simplex. Half-spaces $a^\top x \le b$, affine-equality solution sets $Ax = b$ and their intersections are convex. A convex function satisfies

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta) f(y).$$

Geometrically, the graph lies below its chords. Strict inequality for distinct x, y and $0 < \theta < 1$ defines strict convexity. Convex optimization minimizes a convex objective on a convex feasible set; equivalently it can maximize a concave objective. A local optimum is global. Strict convexity allows at most one optimum on that set, without guaranteeing its existence.

For the quadratic function

$$q(x) = \frac{1}{2} x^\top P x + c^\top x + r,$$

take symmetric $P \in \mathbb{R}^{n \times n}$, with $c \in \mathbb{R}^n$, $x \in \mathbb{R}^n$ and $r \in \mathbb{R}$. Its Hessian is P. If $P \succeq 0$, every direction d has curvature $d^\top P d \ge 0$ and q is convex. If $P \succ 0$, every nonzero direction has positive curvature and q is strictly convex. Positive definiteness is sufficient and also necessary for a quadratic to be strictly convex on the full space. On a budget hyperplane, even singular P can give strict convexity when curvature is positive in every nonzero feasible direction.

A linear program (LP) has a linear objective and affine constraints, for example

$$\begin{array}{ll} \text{maximize}_x & c^\top x \\ \text{subject to} & Ax \le b, \quad x \ge 0. \end{array}$$

With $x \in \mathbb{R}^n$ and m resource constraints, dimensions are $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ and $c \in \mathbb{R}^n$. Linear objectives are both convex and concave; the feasible region is polyhedral. LP is the QP special case $P = 0$.

General quadratic programs can be nonconvex. This chapter considers convex QP only: positive-semidefinite quadratic objective matrices with linear equalities/inequalities:

$$\begin{array}{lll} \text{minimize}_x & \frac{1}{2} x^\top P x + c^\top x & \\ \text{subject to} & Gx \le h, & P \succeq 0. \\ & Ax = b, & \end{array}$$

Here $G \in \mathbb{R}^{m \times n}$, $h \in \mathbb{R}^m$, $A \in \mathbb{R}^{p \times n}$ and $b \in \mathbb{R}^p$. Finitely many linear constraints in finite dimensions define a closed polyhedron. A coercive objective on the feasible set, or a continuous objective on a compact feasible set, are two common sufficient existence conditions, not necessary ones. A convex QP on a closed polyhedron also attains a finite optimum; details lie beyond this chapter. Convexity alone does not guarantee existence or convergence within default solver precision.

CVXPY uses disciplined convex programming (DCP) rules: minimize expressions certified convex, maximize expressions certified concave, and combine affine equalities with correctly oriented convex/concave inequalities. Passing DCP verifies expression rules, not units, economic suitability or estimation accuracy. Failing DCP need not prove mathematical nonconvexity; the expression may lie outside its rules and require an equivalent reformulation.

6. Derivations: from curvature to KKT and duality

6.1 Why a PSD Hessian makes a quadratic convex

Let $P = P^\top \succeq 0$ and choose points x, y and $\theta \in [0, 1]$. On expansion, linear term $c^\top x$ and constant r cancel in the convex-combination difference:

$$\theta q(x) + (1-\theta)q(y) - q(\theta x + (1-\theta)y) = \frac{1}{2}\theta(1-\theta)(x-y)^\top P(x-y) \geq 0.$$

Positive semidefiniteness gives the last inequality, hence convexity. Conversely, if a direction d has $d^\top P d < 0$, curvature along $x + td$ is negative, so the quadratic is not globally convex. Symmetrizing does not generally repair invalid covariance. Theoretical covariance is symmetric PSD; small numerical asymmetry may be averaged within declared tolerances, but material asymmetry or negative eigenvalues require an audit of data and estimation.

6.2 Why least squares is a QP

Let design $A \in \mathbb{R}^{m \times n}$, observations $y \in \mathbb{R}^m$ and chosen coefficients $\beta \in \mathbb{R}^n$ define least squares:

$$\min_\beta \|A\beta - y\|_2^2.$$

Expanding the square gives

$$\beta^\top A^\top A\beta - 2y^\top A\beta + y^\top y.$$

$A^\top A$ is PSD because every z satisfies $z^\top A^\top Az = \|Az\|_2^2 \geq 0$. The residual sum of squares is therefore convex quadratic. Differentiating in β gives $2A^\top(A\beta - y)$ and the normal equations $A^\top A\beta = A^\top y$. Full column rank of A gives $A^\top A \succ 0$ and unique coefficients. Otherwise fitted values remain unique but coefficients do not. Explicit inversion $(A^\top A)^{-1}$ amplifies ill-conditioning; prefer `numpy.linalg.lstsq`, QR/SVD or a reliable least-squares solver.

This does not repeat Chapter 8's inference. Optimization finds coefficients minimizing squared error; it supplies no standard errors, tests, error diagnostics or information-timing evidence for prediction.

6.3 Lagrangian, weak duality and Slater's condition

In Section 4's general minimization problem, attach nonnegative multiplier $\lambda_i \geq 0$ to inequality $f_i(x) \leq 0$ and unrestricted multiplier $\nu_j \in \mathbb{R}$ to each equality. The Lagrangian is

$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^{m} \lambda_i f_i(x) + \sum_{j=1}^{p} \nu_j h_j(x).$$

For feasible x, $\lambda_i f_i(x) \leq 0$ and equality terms vanish, giving $L(x, \lambda, \nu) \leq f_0(x)$. Taking the infimum over x gives dual function $g(\lambda, \nu) = \inf_x L(x, \lambda, \nu)$, a lower bound on every feasible primal objective. Maximizing it yields the Lagrangian dual problem. Dual optimum $d^\star$ never exceeds primal optimum $p^\star$: $d^\star \leq p^\star$. This weak duality also holds for nonconvex problems.

Equality of optimal values is strong duality. For convex problems, Slater is a sufficient constraint qualification; with finite primal optimum, it also ensures dual attainment. It is not necessary. Standard Slater requires a point in the relative interior of the common domain satisfying affine equalities and strictly satisfying every convex inequality. The refined version allows non-strict affine inequalities but still requires strict nonaffine convex inequalities. With only affine inequalities, as in these QPs, a feasible point in the domain's relative interior suffices without strict inequality; these quadratic domains are all of $\mathbb{R}^n$. Failure of this sufficient condition does not establish failure of strong duality.

6.4 When KKT conditions are necessary or sufficient

For differentiable problems, Karush–Kuhn–Tucker (KKT) conditions consist of four groups:

$$\begin{aligned}
&f_i(x^\star) \leq 0, \quad h_j(x^\star) = 0 && \text{(primal feasibility)},\\
&\lambda_i^\star \geq 0 && \text{(dual feasibility)},\\
&\lambda_i^\star f_i(x^\star) = 0 && \text{(complementary slackness)},\\
&\nabla f_0(x^\star) + \sum_i \lambda_i^\star \nabla f_i(x^\star) + \sum_j \nu_j^\star \nabla h_j(x^\star) = 0 && \text{(stationarity)}.
\end{aligned}$$

Stationarity balances objective gradients with constraint effects. Complementary slackness makes inactive-inequality multipliers zero, and positive multipliers require active constraints. An active constraint can still have zero multiplier; a boundary does not necessarily have a positive shadow price.

> **Applicability: necessary and sufficient KKT conditions**
>
> In general nonconvex problems, KKT necessity at a local optimum usually needs a constraint qualification such as linear independence of active constraint gradients. KKT points may still be maxima, saddles or other stationary points; KKT alone does not establish global optimality. For a differentiable convex problem, a feasible primal–dual pair satisfying all KKT conditions is globally primal- and dual-optimal. Inferring existence of multipliers from an optimum requires constraint qualification. Slater guarantees strong duality and dual attainment for convex problems as a sufficient, not necessary or unique, condition.

6.5 Shadow prices and perturbations

For resource bounds $Ax \leq b$, dual multipliers $\lambda^\star \geq 0$ attach to resource constraints. Near a regular LP basis, a small relaxation Δb_i of resource bound b_i for row i changes maximum contribution by approximately $\lambda_i^\star \Delta b_i$. Multipliers have objective-units/resource-units, allowing comparison of scarce capacity with added contribution. This first-order interpretation needs a stable active structure; crossing vertices, changing active sets or meeting a nondifferentiable value function prevents distant linear extrapolation.

For minimization with perturbation $f_i(x) \leq u_i$ and a differentiable value function, $\partial p^\star / \partial u_i = -\lambda_i^\star$. Relaxing upper bound u_i generally reduces minimum cost, with the sign set by this inequality orientation. In the usual maximization LP $Ax \leq b$, resource-value derivatives are nonnegative $\lambda_i^\star$. Specify objective direction and right-hand-side perturbation before interpreting a sign.

7. Financial interpretation: optimizing estimated models, not future returns

Mean–variance portfolios combine weights $w \in \mathbb{R}^n$, same-horizon expected returns $\mu \in \mathbb{R}^n$ and covariance $\Sigma \in \mathbb{R}^{n \times n}$ into a QP:

$$\begin{array}{ll} \text{minimize}_w & \frac{1}{2}\lambda w^\top \Sigma w - \mu^\top w \\ \text{subject to} & \mathbf{1}^\top w = 1, \\ & w \geq 0. \end{array}$$

The first term penalizes variance and the second rewards expected return. The negative sign makes maximizing return equivalent to minimizing negative return here. With annual decimal simple returns, w is dimensionless, $\mu^\top w$ annual expected decimal return, $w^\top \Sigma w$ annual squared-decimal-return variance, and λ inverse-decimal-return risk aversion. Both objective terms then share units.

Increasing λ emphasizes variance over estimated mean; decreasing λ emphasizes return and can push weights toward higher estimated means or constraint boundaries. Weights are model decisions conditional on estimates, not intervals for future returns. Mean estimates are noisy; covariance perturbations can change weights. Assess estimation windows, shrinkage, caps, turnover, costs, borrowing and risk budgets.

Maximum Sharpe uses expected excess returns. Let r_f be the risk-free benchmark for the same holding period and currency and define $\mu^{ex} = \mathbb{E}[R] - r_f \mathbf{1}$. Subtract same-period r_f from total-return inputs; if no risk-free benchmark is modeled, explicitly assume $r_f = 0$. Do not combine monthly excess returns with annual risk-free rates. The objective is

$$\max_w \frac{(\mu^{ex})^\top w}{\sqrt{w^\top \Sigma w}}$$

This fractional objective cannot be used directly as an ordinary QP objective under DCP. Let C be the convex set defined by budget, long-only and optional linear weight caps. First confirm that the finite-ratio candidate domain

$$D_w = \{w \in C : (\mu^{ex})^\top w > 0,\ w^\top \Sigma w > 0\}$$

is nonempty. $\Sigma \succeq 0$ alone does not ensure that. An empty original domain has no finite positive-Sharpe candidates of the type discussed here.

Budget $\mathbf{1}^\top w = 1$ is not itself invariant to scaling. For $w \in D_w$, set $y = w/((\mu^{ex})^\top w)$. Then $y \geq 0$, $(\mu^{ex})^\top y = 1$ and

$$y^\top \Sigma y = \frac{w^\top \Sigma w}{((\mu^{ex})^\top w)^2} > 0.$$

The reverse direction must preserve positive variance too: given $y \geq 0$, $(\mu^{ex})^\top y = 1$ and $y^\top \Sigma y > 0$, set $t = \mathbf{1}^\top y > 0$ and $w = y/t$, obtaining $(\mu^{ex})^\top w = 1/t > 0$ and $w^\top \Sigma w = y^\top \Sigma y/t^2 > 0$. Weight cap $w_i \leq c_i$ becomes $y_i \leq c_i \mathbf{1}^\top y$ in both directions. Denote the closed polyhedron without the positive-variance condition by $\overline{D}_y$ and the strict corresponding domain by $D_y = \{y \in \overline{D}_y : y^\top \Sigma y > 0\}$. The bidirectional mapping holds only between D_w and D_y and preserves Sharpe ordering:

$$S(w) = \frac{1}{\sqrt{y^\top \Sigma y}}.$$

The solvable closed convex QP minimizes $q(y) = y^\top \Sigma y$ on $\overline{D}_y$; distinguish it from the strict domain. Assume its minimum is attained. If $q^\star > 0$, its optimizer lies in D_y and transforms back to finite maximum Sharpe $1/\sqrt{q^\star}$. If $q^\star = 0$, the zero-variance optimizer lies outside D_y and cannot be reported as a finite-ratio optimum. Confirmed $D_w \neq \varnothing$ provides a positive-variance feasible point. Under the present convex budget/long-only/linear-cap structure, mixing it with the zero-variance point generates finite ratios tending to infinity. Section 18 proves supremum $+\infty$; the zero-variance point's own ratio is undefined. Do not draw this conclusion for an empty original domain.

$\Sigma \succ 0$ is a simple sufficient condition for every nonzero transformed vector to have positive variance, not a necessary condition for a finite optimum. Adding strict $q(y) > 0$ does not make an ordinary convex QP: a convex quadratic's strict superlevel set is generally nonconvex. Prove that transformations and mixtures preserve any costs, fixed amounts or other complex restrictions separately.

8. Numerical Example

Complete example: a two-asset return–risk problem, interior and boundary optima with KKT checks

All inputs are teaching constructions with a one-year holding period. Use annual expected decimal returns, annual covariance and risk coefficient:

$$\mu = \begin{bmatrix} 0.05 \\ 0.08 \end{bmatrix}, \qquad \Sigma = \begin{bmatrix} 0.04 & 0.01 \\ 0.01 & 0.09 \end{bmatrix}, \qquad \lambda = 1.$$

Asset 1's annual expected return is 5% and asset 2's 8%. Diagonal entries are annual variances and off-diagonals covariance. Fully invested long-only weights are $w = (x, 1-x)^\top$ with $x \in [0, 1]$.

Expected portfolio return is

$$\mu^\top w = 0.05x + 0.08(1 - x) = 0.08 - 0.03x.$$

Continued: the same complete example

Portfolio variance is

$$w^\top \Sigma w = 0.04x^2 + 2(0.01)x(1-x) + 0.09(1-x)^2 = 0.11x^2 - 0.16x + 0.09.$$

Substitution gives a one-variable convex quadratic objective:

$$J(x) = \tfrac{1}{2}(0.11x^2 - 0.16x + 0.09) - (0.08 - 0.03x) = 0.055x^2 - 0.05x - 0.035.$$

Derivative $J'(x) = 0.11x - 0.05$ vanishes at $x = 5/11 \approx 0.454545$, giving $w = (5/11, 6/11) \approx (0.454545, 0.545455)$. Since $J''(x) = 0.11 > 0$ and the point lies inside the feasible interval, it is the unique global minimum. Expected return is 0.066363636, annual variance 0.040000000 and objective $0.5(0.04) - 0.066363636 = -0.046363636$, matching the numerical solver.

For minimum variance, set the expected-return vector to zero. Equation $0.22x - 0.16 = 0$ gives $x = 8/11 \approx 0.727273$ and weights $w = (8/11, 3/11)$, with variance 0.031818182. This is below the return–risk solution's variance because the objective no longer rewards expected return.

At risk aversion $\lambda = 0.2$, adding constraints changes the solution. Without budget or bounds, stationarity $\lambda\Sigma w - \mu = 0$ gives

$$w = (\lambda\Sigma)^{-1}\mu = (37/7, 27/7), \qquad \mathbf{1}^\top w = 64/7.$$

Weights sum far above 1: the objective does not itself specify investment scale. Add budget $w_1 + w_2 = 1$ and use $w = (x, 1-x)$, giving objective $J_{0.2}(x) = 0.011x^2 + 0.014x - 0.071$ and derivative $0.022x + 0.014$. With shorting and no gross-leverage cap, stationary point $x = -7/11$ gives $w = (-7/11, 18/11)$ and gross exposure $\|w\|_1 = 25/11$. Add convex cap $\|w\|_1 \le 1.5$: the budget line's entire feasible interval is $x \in [-1/4, 5/4]$. For $x < 0$, the cap gives $1 - 2x \le 1.5$; for $x > 1$ it gives $2x - 1 \le 1.5$; all of $[0, 1]$ remains feasible. The minimum derivative on this interval is $J'_{0.2}(-1/4) = 0.0085 > 0$, positive, so the optimum is left endpoint $(-1/4, 5/4)$. Adding long-only bounds $w \ge 0$ narrows the interval to $x \in [0, 1]$ and moves the optimum to $(0, 1)$. Budget controls net weight, leverage gross long/short exposure, and lower bounds forbid shorts. This leverage variant is analytic only; the current helper has no gross-exposure-cap argument.

Now check interior and boundary KKT conditions separately. At risk aversion $\lambda = 1$ and interior $w^\star = (5/11, 6/11)$, objective gradient is

$$\nabla J(w^\star) = \Sigma w^\star - \mu = (-29/1100, -29/1100)^\top.$$

Write budget $h(w) = \mathbf{1}^\top w - 1 = 0$ and long-only constraints $-w_i \le 0$, with Lagrangian $L = J + \nu h - \ell^\top w$. Budget multiplier $\nu = 29/1100$ and lower-bound multipliers $\ell = (0, 0)$ give stationarity vector $\nabla J + \nu\mathbf{1} - \ell = (0, 0)$. Budget residual is zero, weights positive, multipliers nonnegative and complementary products $\ell_i w_i = 0$. Strictly convex objective and convex feasibility make this KKT point the unique global optimum.

At risk aversion $\lambda = 0.2$ with long-only boundary $w^\star = (0, 1)$,

$$\nabla J(w^\star) = 0.2\Sigma w^\star - \mu = (-0.048, -0.062)^\top.$$

Continued: the same complete example

Choose $\nu = 0.062$ and $\ell = (0.014, 0)$, giving stationarity $(-0.048, -0.062) + (0.062, 0.062) - (0.014, 0) = (0, 0)$. Budget residual is zero, weights nonnegative, $\ell \geq 0$ and complementary products $(\ell_1 w_1, \ell_2 w_2) = (0, 0)$. The full-space gradient is nonzero, but inward budget-line direction $(1, -1)$ has derivative $0.014 > 0$, so no long-only feasible direction decreases the objective. Point $(0.5, 0.5)$ satisfies budget and strict long-only inequalities, establishing Slater. Convexity makes feasible KKT sufficient; Slater ensures multiplier existence at an optimum. Lower-bound multipliers have annual-objective-decimal-return per weight units, not asset returns, future excess returns or invested cash.
A simultaneous per-asset cap 0.4 gives $w_1 + w_2 \leq 0.8$, contradicting full investment $w_1 + w_2 = 1$. The problem is infeasible, rather than merely lacking a good solution.

8.2 Checking least squares and a resource LP by hand

Construct features $x = (0, 1, 2, 3)$ and response $y = (1, 2, 4, 5)$ in arbitrary teaching units, not market returns. Intercept design A has rows $(1, x_i)$, giving

$$A^\top A = \begin{bmatrix} 4 & 6 \\ 6 & 14 \end{bmatrix}, \qquad A^\top y = \begin{bmatrix} 12 \\ 25 \end{bmatrix}.$$

Normal equations are $4\beta_0 + 6\beta_1 = 12$ and $6\beta_0 + 14\beta_1 = 25$. Subtract 1.5 times the first from the second to obtain $5\beta_1 = 7$, hence slope $\beta_1 = 1.4$ and intercept $\beta_0 = 0.9$. Fits $(0.9, 2.3, 3.7, 5.1)$ and residuals $(0.1, -0.3, 0.3, -0.1)$ give squared loss 0.2. Full column rank of A makes the loss strictly convex and these coefficients uniquely optimal, without establishing statistical significance.

Complete example: a resource LP and feasible dual upper bound

Construct a production LP. Let a, b be daily batch counts for products A and B, contributing 3 and 2 thousand euros/batch. Machine-hour constraint $2a + b \leq 100$ and labor-hour constraint $a + 2b \leq 80$ apply with $a, b \geq 0$. Subtract half the first binding equation from the second to obtain intersection $b = 20$, hence $a = 40$ and contribution $3a + 2b = 160$ thousand euros/day.
Checking the intersection alone is insufficient. A feasible dual gives a global upper bound: multiply the machine constraint by $4/3$ and labor constraint by $1/3$, then add:

$$3a + 2b \leq \frac{4}{3}(100) + \frac{1}{3}(80) = 160.$$

No feasible production plan exceeds 160, which the intersection attains, proving global optimality. The two nonnegative multipliers are current local resource shadow prices. Changing only the machine bound to $100 + \delta$ gives binding-constraint solution $a = 40 + 2\delta/3$ and $b = 20 - \delta/3$, with contribution $160 + 4\delta/3$. For $-60 < \delta < 60$ both quantities remain positive and the basis is unchanged; outside this interval, recheck active constraints instead of extrapolating the same marginal contribution.

9. Python Implementation

Default entry in the accepted reader package

From the accepted I3 reader-package root, create the isolated environment described in its README, then run:

```
.venv/bin/python run_notebook.py notebooks/ch09_optimization.ipynb
```

The notebook calls the packaged shared implementation. It reads no omitted French data and downloads none by default. Every execution and its figures go into a unique UUID runs directory, whose actual location is printed. Old production tests and historical commands below belong to the course working project; the default reader route uses the package-root notebook and its assertions. Optional historical data follows the reader's data contract; this batch did not rerun it.

I3 also includes code/ch09_optimization.py, but the default reader entry remains the notebook above.

Historical course-root route

Previously validated core functions are in `optimization_tools.py`, with standalone example `ch09_optimization.py`. From the course-project root, run:

```
.venv/bin/python code/ch09_optimization.py
```

The recorded script used CVXPY 1.9.3 with installed solvers `CLARABEL`, `SCS`, `SCIPY`, `HIGHS` and `OSQP`. OSQP returned `optimal` for the convex QP, while the LP used CLARABEL. Defaults can differ between environments. Report actual `problem.solver_stats.solver_name`; do not assume every installation has the same solver.

Run this minimal portfolio example independently from that project root:

```
import sys
sys.path.insert(0, "src")
import numpy as np
from quant_course.optimization_tools import solve_mean_variance

mu = np.array([0.05, 0.08])
sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
result = solve_mean_variance(mu, sigma, risk_aversion=1.0, solver="OSQP")
minimum_variance = solve_mean_variance([0.0, 0.0], sigma, solver="OSQP")
print(result.solver_status, result.solver_name, result.weights)
print(result.expected_return, result.variance, result.objective)
print("primal residuals", result.budget_residual, result.nonnegativity_violation)
print("KKT diagnostics", result.stationarity_residual, result.
    ↪ complementarity_residual)
print("min-variance", minimum_variance.weights, minimum_variance.variance)
assert np.allclose(result.weights, [5 / 11, 6 / 11], atol=1e-7)
```

The wrapper requires a finite real expected-return vector, matching finite approximately symmetric PSD covariance, positive risk aversion and feasibility tolerance, and optional weight cap in $(0, 1]$. Invalid dimensions, finite values, symmetry or semidefiniteness raise errors. Eigenvalue tolerance is $10^{-10} \max_i |d_i|$, where d_i are eigenvalues. Values below negative tolerance are rejected; slightly negative values within tolerance are clipped to zero and the matrix reconstructed. This permits only negative curvature small relative to spectral scale, without proving rounding caused it. A zero matrix has zero spectral scale and tolerance; no absolute tolerance floor is added. Symmetry is also checked relative to the largest absolute matrix entry. This is not a general remedy for economic estimation errors; the return object has no separate clipping flag.

After modeling, check `problem.is_dcp()`, solve and inspect status:

- `optimal`: proceed to independent feasibility checks.
- `optimal_inaccurate`: reject as a passed result.

The wrapper independently recomputes budget, long-only and upper-bound violations, raising an error if their maximum exceeds default 10^{-6}. It also returns stationarity and complementary-slackness residuals and checks they are finite. **It sets no acceptance threshold** for these two KKT residuals and checks neither dual signs, dual gap nor a convergence-precision certificate. They are diagnostics, not proof that every solve passed a complete KKT certificate. The objective uses the covariance after eigenvalue validation; clipping slightly negative eigenvalues makes diagnostics refer to that modified model.

Financial inputs use one-year horizons. The script contains no real prices, release dates, signals, executions or realized holding intervals. There are no `available_at`, `signal_time` or `trade_time` fields to audit, and no period-start signal predicts subsequent realized returns. Real research requires estimation-window data available before signal generation and timing signal_time $<$ trade_time $\leq$ return interval start. Holding-period length must match μ and Σ units.

10. Visualization

The script generated the following objective curve:

The horizontal axis is asset 1 weight $w_1 \in [0, 1]$; asset 2 automatically has $1 - w_1$. The vertical axis is annual-decimal-return objective $\frac{1}{2}\lambda w^\top \Sigma w - \mu^\top w$. Fixed constructed inputs generate the curve; the red star and dashed line mark interior weights $w_1 = 5/11$ and $w_2 = 6/11$. Upward curvature reflects a positive second derivative, with larger objective values toward either endpoint.

The figure describes model geometry, not asset-return history, an empirical efficient frontier or performance. Minimum -0.04636 is an objective value containing a variance penalty, not expected return, annual Sharpe or net portfolio return. The plotted objective agrees with script output.

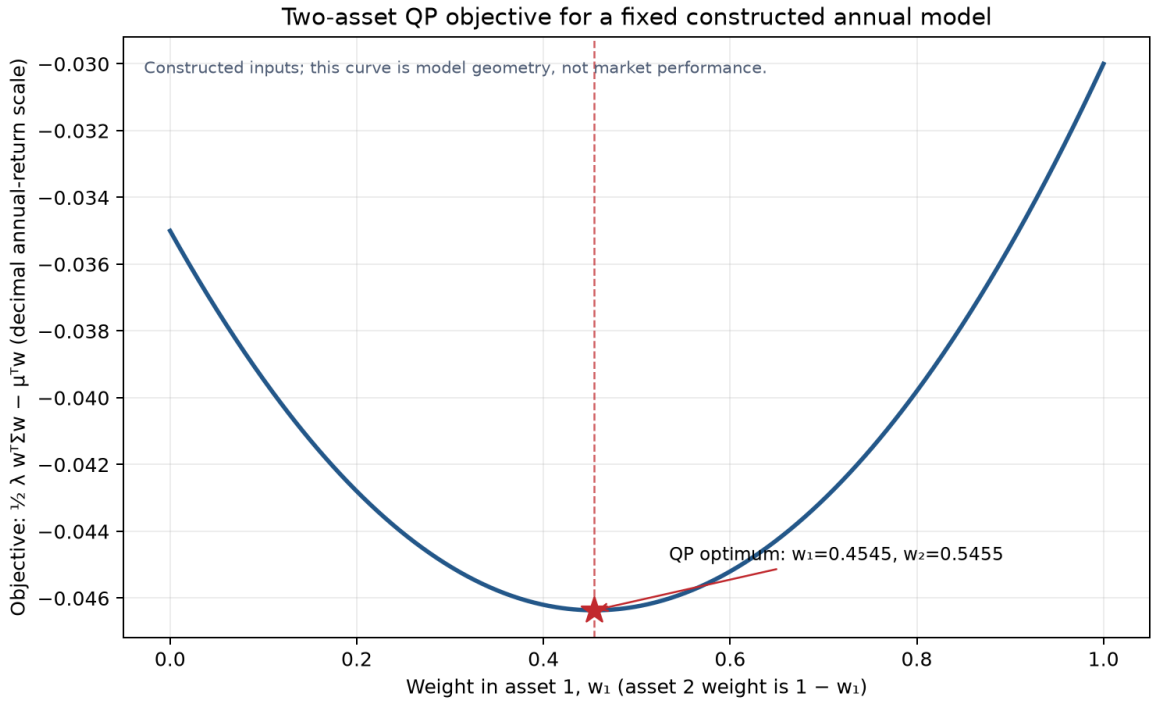


Figure 1: Two-asset portfolio objective and interior optimum

11. Real Market Example

This chapter reads no real asset prices or returns and reports no empirical portfolios, optimal market weights or out-of-sample performance. Two-asset μ and Σ, LP profits/capacity and the four least-squares points are constructions. The contribution is a method for turning estimates into decisions and checking solutions, not empirical market evidence.

A real study needs a fixed security universe and delisting treatment; total versus price returns, risk-free series, currency, adjustments and forecast frequency; and saved versions, download time, sample windows and hashes. Estimate means, covariance and features only from point-in-time available information, using training alone for tuning and risk budgets. Validate signals through order submission/execution and post-trade holding returns, comparing simple benchmarks, turnover, costs, slippage, capacity and robust parameter ranges.

12. Common Mistakes

Calling maximized convex risk a DCP problem. For PSD Σ, $w^\top \Sigma w$ is convex. DCP permits minimizing, not maximizing, a convex expression. Define the intended minimized risk–return objective and verify constraints.

```
import cvxpy as cp
import numpy as np

mu = np.array([0.05, 0.08])
sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
w = cp.Variable(2)
constraints = [cp.sum(w) == 1, w >= 0]
wrong = cp.Problem(cp.Maximize(cp.quad_form(w, sigma)), constraints)
print("wrong problem follows DCP:", wrong.is_dcp())
try:
    wrong.solve(solver="OSQP")
except cp.error.DCPError as error:
    print(type(error).__name__)
correct = cp.Problem(
    cp.Minimize(0.5 * cp.quad_form(w, sigma) - mu @ w), constraints
)
print("correct problem follows DCP:", correct.is_dcp())
correct.solve(solver="OSQP")
print(correct.status, w.value)
```

The incorrect model fails DCP with `False` and raises `DCPError`; the corrected model gives `True` and `optimal`. If the research objective truly calls for maximum risk, its risk definition may be reversed. Changing maximization to minimization is a modeling decision requiring review of the hypothesis, not merely a syntax correction.

Mixing return and covariance horizons. Annual expected returns with unscaled monthly covariance distort the risk term. Unify holding period and return scale first. Aggregating monthly independent increments to a year requires independence or appropriate weak-correlation assumptions for covariance scaling; "multiply by 12" does not verify frequency structure. One-year inputs here are constructions, not a data-annualization procedure.

Passing a low-precision status as normal success. `optimal_inaccurate` supplies a low-precision candidate; `infeasible_inaccurate` limits the precision of an infeasibility conclusion. Nonempty `value` alone can hide boundary violations. The following checks show conservative API behavior: impossible caps raise `OptimizationError` preserving status, while a materially non-PSD covariance fails before modeling.

```
import sys
sys.path.insert(0, "src")
from quant_course.optimization_tools import OptimizationError, solve_mean_variance

mu = [0.05, 0.08]
sigma = [[0.04, 0.01], [0.01, 0.09]]
try:
    solve_mean_variance(mu, sigma, max_weight=0.4, solver="CLARABEL")
except OptimizationError as error:
    print("cap result:", error.solver_status)
try:
    solve_mean_variance(mu, [[1.0, 2.0], [2.0, 1.0]], solver="CLARABEL")
except ValueError as error:
    print("covariance rejected:", str(error))
valid = solve_mean_variance(mu, sigma, solver="CLARABEL")
```

```
print(valid.solver_status, "budget residual", valid.budget_residual)
print("reported KKT diagnostics", valid.stationarity_residual, valid.
    ↪ complementarity_residual)
```

Recorded outputs show `infeasible`, rejection of the non-PSD matrix, and `optimal` for the valid QP. If numerically unstable, report status and inspect scaling, conditioning, PSD, primal constraints and solver logs; reformulate or compare other installed solvers as needed. Do not silently treat candidate weights as usable.

Turning a nonzero dual into a large-return guarantee. Interpret it under the current objective, constraint direction, active set and local perturbation. Changing a right-hand side may change vertices or units, preventing distant extrapolation. Track CVXPY inequality orientation and Lagrangian signs: `w >= 0` has a lower-bound multiplier entering stationarity with a minus sign.

Rejecting a boundary solution because the full-space gradient is nonzero. Unconstrained interior optima have zero gradients; boundaries restrict feasible directions. KKT balances objective gradients with budget and bound gradients. At risk aversion $\lambda = 0.2$ in the two-asset example, the original gradient is nonzero but the inward budget-line derivative is positive, so the boundary solution is optimal.

Sending every maximum-Sharpe problem directly to QP. The fractional objective depends on $\sqrt{w^\top \Sigma w}$; direct `Maximize` is not ordinary QP/DCP. Restrict to positive excess return and positive variance, prove bidirectional feasibility, normalize budget on transforming back, and translate fixed weight caps to homogeneous linear constraints.

13. Quant Research Application

Start with the research question: portfolio choice, execution planning or analysis of a fitting objective? Fix horizon and units, list decisions, objective, equalities, bounds and inviolable risk budgets. Portfolio inputs need security identities, return definition, currency, risk-free return, adjustments, delistings, covariance windows and mean-estimation methods. Audit `available_at`, versions and signal times, then establish convexity and feasibility before creating CVXPY expressions.

Do not transfer $w^\star$ directly into a performance report. Check status, returned variables and actual solver. Independently recompute $\mathbf{1}^\top w - 1$, minimum/maximum weights, budget and risk residuals; inspect multipliers, active constraints, stationarity and complementarity. Stress estimates: can small mean perturbations, covariance shrinkage, asset exclusions or risk-aversion changes reverse the portfolio? Local shadow prices do not replace sensitivity analysis.

At each real backtest period, estimate μ and Σ only from then-available historical observations. Generate `signal_time` before feasible `trade_time`, then map weights to an explicit post-trade `return interval`. Align holding period, estimation frequency, risk matrix and cost timing. Optimization must not use later revisions, future prices or holdout targets. Record benchmarks, turnover, fees, slippage, borrowing, capacity and rebalancing. Prespecify disconfirmation, such as weights flipping under reasonable mean estimates, excess out-of-sample risk, failure after costs or contradictory constraints/execution rules.

The script provides no point-in-time data, signals, trades or holding-period performance. It demonstrates objectives, convexity, solver outputs and structural checks, without empirical strategy effects.

Further research boundaries appear in Chapter 20: modern portfolio theory and Chapter 21: practical portfolio construction. Chapter 20 distinguishes population from estimated moments and treats plug-in frontiers as in-sample objects. Chapter 21 addresses mean/covariance error, real holdings constraints, turnover/cost ledgers and out-of-sample evaluation. A constructed optimum alone establishes neither improved real return nor executability.

14. Chapter Summary

Optimization defines an objective with its feasible region. Convex objectives on convex sets have globally optimal local optima; PSD Hessians produce typical convex QPs. Least-squares Gram matrix $A^\top A$ is PSD; LP uses linear resource bounds; mean–variance QP combines convex risk with expected return.

Lagrange multipliers incorporate constraints and relate primal and dual problems. Weak duality always provides a bound. Convexity with qualifications such as Slater can establish strong duality. Feasible KKT conditions suffice in convex problems; in general nonconvex problems they commonly provide necessary stationarity under regularity. Dual sensitivity remains local, with signs, units and perturbation directions set by constraint conventions.

Solver status is computational evidence, not economic evidence. Report solver, status, objective, primal residuals, KKT diagnostics and input scaling, while preserving availability times and evaluation intervals.

15. Exercises

Progressive short exercises

9-T01 (identify objects). Identify decisions, objective, budget equality, long-only bounds and feasible set. Give meanings/units of w, μ, Σ and λ. What does additional $\|w\|_1 \le L$ restrict?

9-T02 (one calculation). At $w = (0.5, 0.5)$ and $\lambda = 1$, calculate same-year expected return, variance and objective. Which quantity has squared-decimal-return units?

9-T03 (complete calculation). Take $\lambda = 0.2$ and $w = (x, 1 - x)$. Find unconstrained stationarity without budget, then add budget, gross cap $\|w\|_1 \le 1.5$ and long-only bounds $w \ge 0$ successively. Calculate each solution and explain which constraint changes it.

9-T04 (code diagnosis). Code receives nonempty weights.value, normalizes by its sum and returns it, although status may be optimal_inaccurate or infeasible. What statuses and numerical quantities must be checked? Why cannot normalization repair the failed original problem?

9-T05 (research judgment). Do constructed optimal weights and a low objective establish improved real return or immediate live tradability? What evidence remains necessary, and which later chapters address it?

9-E01 (definition). Use convex combinations to explain why half-spaces, affine-equality solution sets and their intersections are convex. Give a nonconvex feasible set and state what makes local optimality global.

9-E02 (mathematics). For $q(x) = \frac{1}{2}x^\top Px + c^\top x + r$, derive the Hessian and prove $P \succeq 0$ sufficient for convex q. Why is $P \succ 0$ sufficient for strict convexity on the full space? What directional condition applies on an affine budget set?

9-E03 (modeling). Write least squares as a QP and show why $A^\top A$ is PSD. What gives unique coefficients? How do RSS units relate to response units?

9-E04 (hand calculation). Use Section 8's μ, Σ with $w = (x, 1-x)$ and $\lambda = 1$. Calculate return, variance, objective and optimal weights. Why is this a global minimum, not just a stationary point?

9-E05 (duality). Distinguish weak and strong duality, Slater's guarantee and why it is not necessary. Compare KKT's logical roles in nonconvex and differentiable convex problems.

9-E06 (shadow prices). Derive machine/labor shadow values 4/3 and 1/3 thousand euros per additional resource-hour and explain their extrapolation limits.

9-E07 (status). Explain `optimal_inaccurate`, `infeasible` and `unbounded`. Why is nonempty `variable.value` insufficient? Which residuals does the course helper threshold, and which are diagnostics only?

16. Programming Exercises

9-P01 (mean–variance QP). Use `solve_mean_variance` for baseline and minimum-variance portfolios. Assert analytical objectives, returns, variances, budget residuals and weights, using annual decimal-return inputs.

9-P02 (LP and duality). Model daily A/B production in CVXPY. Each A batch uses 2 machine-hours and 1 labor-hour; B uses 1 and 2. Daily capacities are 100 and 80 hours, and net contributions 3 and 2 thousand euros/batch. Find quantities, total contribution and both resource duals; verify primal resource residuals.

9-P03 (boundary KKT and infeasibility). Solve the two-asset problem at $\lambda = 0.2$. Report boundary weights, lower-bound duals, full-space gradient and budget, stationarity and complementarity residuals. Set both caps to 0.4 and record infeasibility without continuing from a partial solution.

17. Challenge Problems

9-C01 (Sharpe transformation). Derive $y = w/((\mu^{ex})^\top w)$ from the fractional maximum-Sharpe problem, explicitly define $\mu_i^{ex} = \mathbb{E}[R_i] - r_f$ with a common holding period/currency, and declare $r_f = 0$ if used. Check the positive-excess-return/positive-variance domain under budget, long-only and optional linear caps, then prove the positive-variance bidirectional map and cap transformation. Distinguish the strict domain from the closed PSD QP: what does positive or zero optimal variance imply? Use $\Sigma = \mathrm{diag}(0, 1)$, $\mu^{ex} = (1, 0)^\top$ and $w = (x, 1 - x)^\top$ to check $0 < x < 1$ versus $x = 1$. Show through feasible mixtures when the supremum is infinite and when the original domain is empty. Explain why PD is unnecessary and strict positive variance is not an ordinary convex QP constraint. Ten points: same-period excess returns/nonempty domain 2, bidirectional positive-variance map 2, caps 1, positive/zero closed-QP values and mixture 3, PD sufficiency and strict-domain convexity limits 2.

9-C02 (strict convexity on the feasible set). Let singular $\Sigma \succeq 0$ and budget hyperplane $\mathbf{1}^\top w = 1$ apply. Give tangent-space curvature conditions, and determine whether $w^\top \Sigma w$ can fail strict convexity globally yet be strictly convex on budget-feasible directions. Score tangent space $\{d : \mathbf{1}^\top d = 0\}$ 2 points, $d^\top \Sigma d > 0$ for every nonzero feasible direction 3, null-space intersection/uniqueness 3, sufficient conditions versus existence 2.

18. Solutions / Solution Hints

9-E01 solution. Write a half-space as $f(x) = a^\top x - b \leq 0$ with affine f. For feasible x, y, convex combinations remain at most zero. Affine-equality solutions preserve $A(\theta x + (1 - \theta)y) = \theta Ax + (1 - \theta)Ay = b$. Points in an intersection belong to every constituent set, hence their connecting segment does too. A union of two separated closed disks is a counterexample: each disk is convex but their union generally is not. Both objective and feasible region must be convex. If a local optimum had a better feasible point, the connecting segment would give arbitrarily close feasible improvements, contradicting local optimality.

9-E02 solution. For symmetric P, $\nabla q(x) = Px + c$ and $\nabla^2 q(x) = P$. The quadratic convex-combination difference is $\frac{1}{2}\theta(1 - \theta)(x - y)^\top P(x - y)$, nonnegative under PSD, proving convex q. Full-space $P \succ 0$ provides positive curvature in every nonzero difference direction, hence strict convexity. On $Ax = b$, tangent directions satisfy $Ad = 0$. If every nonzero $d \in \ker A$ has $d^\top Pd > 0$, the restricted objective is strictly convex even with singular P on $\mathbb{R}^n$. With inequalities, examine the directions of relevant feasible faces. Strict convexity permits at most one optimum, without ensuring attainment.

9-E03 solution. Least squares is $\min_\beta \|A\beta - y\|_2^2 = \min_\beta(\beta^\top A^\top A\beta - 2y^\top A\beta + y^\top y)$. Every z satisfies $z^\top A^\top Az = \|Az\|_2^2 \geq 0$, so the quadratic matrix is PSD. Normal equations are $A^\top A\beta = A^\top y$. Full column rank of A makes $A^\top A$ PD and coefficients unique; otherwise some coefficient directions cannot be distinguished by the sample. If y has daily decimal-return units, residuals share them and RSS has squared-return units. If y is euros, RSS is euros squared.

9-E04 solution. With $w = (x, 1-x)$, return is $0.08 - 0.03x$ and variance $0.11x^2 - 0.16x + 0.09$.

$$J(x) = 0.055x^2 - 0.05x - 0.035, \qquad J'(x) = 0.11x - 0.05, \qquad J''(x) = 0.11 > 0.$$

Stationary point $x = 5/11 \in [0, 1]$ is the unique global interval minimum. Expected return 0.066363636, variance 0.04 and objective -0.046363636 follow. Convexity together with feasibility gives the global conclusion.

9-E05 solution. Weak duality bounds every feasible primal value below by any feasible dual bound, giving $d^\star \leq p^\star$; strong duality equates optimal values. Slater requires a point in the common domain's relative interior satisfying affine equalities and strict inequalities, and suffices for strong duality. With finite primal optimum it ensures dual attainment. Refined Slater needs only feasibility for affine inequalities while retaining relative-interior conditions. It is sufficient, not necessary; some non-strictly feasible problems still have zero dual gap. In nonconvex problems, KKT typically needs qualification to be necessary at local optima and is not sufficient. Feasible KKT suffices for global primal/dual optimality in differentiable convex problems. Inferring multiplier existence at any primal optimum needs strong duality and dual attainment; Slater is sufficient.

9-E06 solution. Maximize $3a + 2b$ subject to $2a + b \leq 100$, $a + 2b \leq 80$ and $a, b \geq 0$. Both products are positive and both capacities bind at the optimal vertex. Dual equalities are $2\lambda_m + \lambda_l = 3$ and $\lambda_m + 2\lambda_l = 2$, giving $\lambda_m = 4/3$ and $\lambda_l = 1/3$. Units are thousand euros/day divided by resource-hours/day: marginal daily contribution per additional daily resource-hour. This first-order interpretation holds locally with the same feasible basis; a new vertex requires reoptimization.

9-E07 solution. `optimal_inaccurate` reports a candidate below target precision; `infeasible` reports a determination of infeasibility; `unbounded` means the objective can tend to positive/negative infinity along feasible points, according to maximization/minimization. CVXPY can retain values under inaccurate statuses, so nonempty `value` does not prove success. The helper thresholds recomputed budget, long-only and cap violations at 10^{-6}. It recomputes stationarity/complementarity and checks finiteness but sets no acceptance threshold and calculates neither dual gap nor sign conditions.

9-T01 solution. $w = (w_1, w_2)$ is a dimensionless weight vector, objective $\frac{1}{2}\lambda w^\top \Sigma w - \mu^\top w$, budget $w_1 + w_2 = 1$ and bounds $w_i \geq 0$. Feasible weights satisfy all equalities and inequalities. Annual μ contains expected total decimal returns, Σ annual squared-decimal-return covariance, and λ inverse-decimal-return risk aversion, aligning objective units. $\|w\|_1 \leq L$ restricts gross long/short exposure without replacing net budget.

9-T02 solution. $\mu^\top w = 0.5(0.05) + 0.5(0.08) = 0.065$. Variance is $0.25(0.04) + 2(0.25)(0.01) + 0.25(0.09) = 0.0375$, so $J = 0.5(0.0375) - 0.065 = -0.04625$. Variance units are annual decimal return squared; multiplying by $\lambda = 1$ in inverse-return units aligns the risk penalty with expected return.

9-T03 solution. Without budget/bounds, stationarity $\lambda\Sigma w - \mu = 0$ gives $w = (\lambda\Sigma)^{-1}\mu = (37/7, 27/7)$. With budget, $J_{0.2}(x) = 0.011x^2 + 0.014x - 0.071$ and derivative $J'_{0.2}(x) = 0.022x + 0.014$ give stationary point $x = -7/11$, weights $(-7/11, 18/11)$ and gross exposure $25/11$. Add $\|w\|_1 \leq 1.5$: the entire feasible interval is $[-1/4, 5/4]$, from $1 - 2x \leq 1.5$ on the negative branch and $2x - 1 \leq 1.5$ above one. Its derivative is positive throughout, so optimum $(-1/4, 5/4)$. Add $w \geq 0$ to get interval $[0, 1]$ and optimum $(0, 1)$. Budget constrains net weight, leverage gross exposure and lower bounds shorts; each removes different choices.

9-T04 solution. Check actual status first. The helper accepts only `optimal`, still a floating-point result rather than an exact certificate. It rejects `optimal_inaccurate` and does not use infeasible variable values. Verify finite weights and recompute original budget, long-only, caps and any other task constraints, plus objective and units. Normalization changes the returned vector and may violate caps or other restrictions; it cannot turn failure into a feasible optimum of the original problem. Budget/long-only/uniform-cap violations use threshold 10^{-6}. Stationarity/complementarity are finite diagnostics without acceptance thresholds or a complete KKT certificate.

9-T05 solution. No. Constructed μ, Σ are fixed inputs; optimal status refers to a candidate solution of the stated model, without validating estimation, future returns or trades. A real study needs point-in-time training data, out-of-sample assessment, real constraints/financing, turnover/cost/slippage ledgers and robustness. Chapter 20 develops mean–variance theory and estimated-moment in-sample limits; Chapter 21 covers estimation error, practical constraints, costs and out-of-sample performance.

9-P01 reference implementation.

```
import sys
sys.path.insert(0, "src")
import numpy as np
from quant_course.optimization_tools import solve_mean_variance

mu = np.array([0.05, 0.08])
sigma = np.array([[0.04, 0.01], [0.01, 0.09]])
result = solve_mean_variance(mu, sigma, risk_aversion=1.0, solver="CLARABEL")
minimum_variance = solve_mean_variance([0.0, 0.0], sigma, solver="CLARABEL")
assert result.solver_status == "optimal"
np.testing.assert_allclose(result.weights, [5 / 11, 6 / 11], atol=1e-7)
np.testing.assert_allclose([result.expected_return, result.variance, result.objective
    ↪ ],
                           [0.06636363636363636, 0.04, -0.04636363636363636], atol=1e
    ↪ -7)
np.testing.assert_allclose(minimum_variance.weights, [8 / 11, 3 / 11], atol=1e-7)
np.testing.assert_allclose(minimum_variance.variance, 0.031818181818181815, atol=1e
    ↪ -7)
assert result.budget_residual < 1e-6 and result.nonnegativity_violation < 1e-6
print("QP status", result.solver_status, result.solver_name)
print("weights", result.weights, "return", result.expected_return,
      "variance", result.variance, "objective", result.objective)
print("minimum variance", minimum_variance.weights, minimum_variance.variance)
```

Reference weights are approximately $[0.45454545, 0.54545455]$, expected return 0.066363636, variance 0.04 and objective -0.046363636. Minimum-variance weights are [0.72727273, 0.27272727] with variance 0.031818182. Small differences reflect numerical tolerance.

9-P02 reference implementation.

```
import cvxpy as cp
import numpy as np

lots = cp.Variable(2, nonneg=True, name="lots_per_day")
machine = 2 * lots[0] + lots[1] <= 100
labor = lots[0] + 2 * lots[1] <= 80
problem = cp.Problem(cp.Maximize(np.array([3.0, 2.0]) @ lots), [machine, labor])
problem.solve(solver="CLARABEL")
assert problem.status == cp.OPTIMAL
np.testing.assert_allclose(lots.value, [40.0, 20.0], atol=1e-5)
np.testing.assert_allclose(problem.value, 160.0, atol=1e-5)
np.testing.assert_allclose([machine.dual_value, labor.dual_value], [4 / 3, 1 / 3],
    ↪ atol=1e-5)
assert 2 * lots.value[0] + lots.value[1] <= 100 + 1e-6
assert lots.value[0] + 2 * lots.value[1] <= 80 + 1e-6
print("status", problem.status, "solver", problem.solver_stats.solver_name)
print("lots/day", lots.value, "contribution kEUR/day", problem.value)
print("shadow values kEUR per additional resource-hour/day",
      machine.dual_value, labor.dual_value)
```

The reference solution produces 40 A and 20 B batches/day for 160 thousand euros/day. Machine dual $4/3$ and labor dual $1/3$ give marginal contribution per added daily resource-hour near the current basis.

9-P03 reference implementation.

```
import sys
sys.path.insert(0, "src")
import numpy as np
from quant_course.optimization_tools import OptimizationError, solve_mean_variance

mu = [0.05, 0.08]
sigma = [[0.04, 0.01], [0.01, 0.09]]
boundary = solve_mean_variance(mu, sigma, risk_aversion=0.2, solver="CLARABEL")
np.testing.assert_allclose(boundary.weights, [0.0, 1.0], atol=1e-6)
np.testing.assert_allclose(boundary.lower_bound_duals, [0.014, 0.0], atol=1e-6)
gradient = 0.2 * np.asarray(sigma) @ boundary.weights - np.asarray(mu)
assert not np.allclose(gradient, 0.0, atol=1e-3)
assert boundary.stationarity_residual < 1e-6
assert boundary.complementarity_residual < 1e-6
print("weights", boundary.weights, "lower duals", boundary.lower_bound_duals)
print("full-space gradient", gradient, "stationarity", boundary.stationarity_residual
    ↪ )
try:
    solve_mean_variance(mu, sigma, max_weight=0.4, solver="CLARABEL")
except OptimizationError as error:
    assert error.solver_status == "infeasible"
    print("infeasible cap confirmed:", error.solver_status)
else:
    raise AssertionError("two 0.4 caps cannot sum to a fully invested portfolio")
```

Reference boundary weights $[0, 1]$, first lower-bound dual 0.014 and full-space gradient $[-0.048, -0.062]$ give near-zero stationarity after balancing budget and active-bound normals. Two 0.4 caps leave no feasible point.

9-C01 reference derivation. Write $m = \mu^{ex} \in \mathbb{R}^n$ with $m_i = \mathbb{E}[R_i] - r_f$ and a common holding period/currency for assets and risk-free benchmark; assume $\Sigma \succeq 0$. Budget/long-only set C may include caps $w_i \le c_i$. First find a portfolio satisfying $w \in C$, $m^\top w > 0$ and $w^\top \Sigma w > 0$ to establish $D_w \neq \varnothing$. Budget feasibility or positive excess return alone is insufficient.

Set $y = w/(m^\top w)$, giving $y \ge 0$, $m^\top y = 1$ and positive variance $q(y) = w^\top \Sigma w/(m^\top w)^2 > 0$, plus caps $y_i \le c_i \mathbf{1}^\top y$ if present. Conversely, retain these conditions, especially $q(y) > 0$. From $m^\top y = 1$ follows $y \neq 0$, hence $t = \mathbf{1}^\top y > 0$. Transforming back with $w = y/t$ gives

$$\mathbf{1}^\top w = 1, \quad w \ge 0, \quad m^\top w = 1/t > 0, \quad w^\top \Sigma w = q(y)/t^2 > 0.$$

Caps return to $w_i \le c_i$, and the forward map recovers the same y. Thus D_w and D_y correspond one to one, with $S(w) = 1/\sqrt{q(y)}$; minimizing variance on the strict domain preserves maximum-Sharpe ordering. Two 0.4 caps imply $t \le 0.8t$, contradicting $t > 0$, so both domains are empty. Zero or negative excess return is outside this normalization argument.

Remove positive variance to obtain the closed convex QP:

$$\min_{y \in \overline{D}_y} q(y), \qquad \overline{D}_y = \{y \ge 0 : m^\top y = 1,\ y_i \le c_i \mathbf{1}^\top y\},$$

Omit caps when absent. Assume this closed QP attains its minimum. If $q^\star > 0$, optimizer $y^\star$ lies in the strict domain and transforms back validly, giving maximum $1/\sqrt{q^\star}$. If $q^\star = 0$, closed-QP optimizer y_0 transforms to zero-variance weights, excluded from the original finite-ratio problem.

A nonempty original domain then provides $y_1 \in \overline{D}_y$ with $q(y_1) > 0$. For $0 < \theta \leq 1$ define $y_\theta = (1-\theta)y_0 + \theta y_1$. Convex mixing preserves linear normalization, nonnegativity and caps. PSD gives implication $y_0^\top \Sigma y_0 = 0 \Rightarrow \Sigma y_0 = 0$: write $\Sigma = B^\top B$, so zero variance means $\|By_0\|^2 = 0$. Therefore

$$q(y_\theta) = \theta^2 q(y_1) > 0, \qquad S(w_\theta) = \frac{1}{\theta\sqrt{q(y_1)}} \longrightarrow +\infty, \quad w_\theta = \frac{y_\theta}{\mathbf{1}^\top y_\theta}.$$

Each w_θ is originally feasible with a finite ratio. The supremum is $+\infty$ without a finite maximizing portfolio. The zero-variance endpoint's ratio is undefined; it does not attain infinite Sharpe. If the original domain is empty, no y_1 exists for this mixing argument, so no infinite-supremum conclusion follows.

For example, take $\Sigma = \mathrm{diag}(0, 1)$ and $m = (1, 0)^\top$ without caps. For $0 < x < 1$, weights $w = (x, 1-x)^\top$ have excess return x and positive variance $(1-x)^2 > 0$, establishing a nonempty original domain; Sharpe is $x/(1-x)$. At $x = 1/2, 9/10, 99/100, 999/1000$ ratios are 1, 9, 99 and 999. The transform is $y = (1, (1-x)/x)^\top$. The closed QP admits $y_0 = (1, 0)^\top$ with variance zero; returned weights $w_0 = (1, 0)^\top$ are excluded from the original domain. Take $y_1 = (1, 1)^\top$: mixture $y_\theta = (1, \theta)^\top$ gives $S = 1/\theta$. Compare $\Sigma = 0$ with $m = (1, 0)^\top$: the closed QP remains feasible with value zero, but all portfolio variances are zero, so the original domain is empty and no such finite-ratio sequence exists.

PD is sufficient, not necessary. Keep singular PSD $\mathrm{diag}(0, 1)$ but use $m = (0, 1)^\top$: normalization fixes $y_2 = 1$, closed-QP minimum variance is 1 and Sharpe is identically 1 on original domain $w_2 > 0$, attaining a finite maximum. The strict domain need not be convex: with $\Sigma = \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ and $m = (1, 1)^\top$, normalized long-only endpoints $(1, 0)$ and $(0, 1)$ each have variance 1 while their midpoint has zero variance. Adding $q(y) > 0$ excludes the midpoint, preventing ordinary convex-QP treatment. Costs and nonlinear restrictions need separate proofs; the mixture conclusion does not automatically carry over.

9-C02 reference reasoning. Budget tangent space is $T = \{d \in \mathbb{R}^n : \mathbf{1}^\top d = 0\}$. Under $\Sigma \succeq 0$, if each nonzero $d \in T$ has $d^\top \Sigma d > 0$, different budget-feasible weights w, v have difference $d = w - v \in T$ with positive curvature. Thus $w^\top \Sigma w$ is strictly convex on the budget hyperplane, allowing at most one minimum-variance point there or in its long-only simplex. Nonzero null-space directions are allowed provided they do not intersect T nontrivially. Full-space PD is sufficient but not the only possibility. If nonzero $d \in \ker \Sigma \cap T$ exists, the quadratic is constant along that direction and is not strictly convex. A linear expected-return term may still select a unique boundary solution. Distinguish uniqueness guaranteed by strict convexity from other sources of uniqueness.

19. Further Reading

- **Must Know**: Boyd and Vandenberghe, *Convex Optimization*, §§3.1.4, 4.1–4.4, 5.1–5.2.3, 5.5.3, 5.6.1–5.6.3 and 9.1.1. Read Hessian criteria, convex problems/LP/QP, Lagrangian duality/Slater, KKT necessity/sufficiency, perturbation sensitivity and least-squares normal equations. Chapter/page records appear in `source_notes.md`.
- **Should Know**: CVXPY's DCP guide and QP example for curvature rules, mean–variance form and `dual_value` mappings.
- **Advanced**: LP, least squares, solver guide and status guide for standard forms, status, statistics and infeasibility/unboundedness.
- **Optional**: Boyd Chapter 9 on unconstrained minimization for further numerical/nonconvex discussion. It does not establish this chapter's convex-QP guarantees or replace the conditional Sharpe-transformation proof.

20. References

Source key `BOYD` points to the authors' Stanford textbook site; `CVXPY` to CVXPY's official QP documentation. Specific Boyd–Vandenberghe reading: §3.1.4 (p. 71, second-order convexity and quadratics); §§4.1–4.4 (pp. 127–153, problem definitions, convex optimization, LP/QP); §§5.1–5.2.3 (pp. 215–227, Lagrangian duality, weak/strong duality, Slater); §5.5.3 (pp. 243–244, KKT necessity/sufficiency); §§5.6.1–5.6.3 (pp. 249–252, perturbations/sensitivity); §9.1.1 (p. 458, quadratic minimization and least-squares normal equations). Official PDF: Boyd and Vandenberghe, *Convex Optimization*.

Software references are CVXPY's official DCP, QP, LP, least squares, status and solver documentation. These support expression rules, APIs, statuses and constraint duals, not real-market data, superiority of estimators or strategy performance.

Chapter 10 — Financial Markets

Thread: a trade moves through quotation, execution, clearing and settlement. Its economic return, rights, obligations and cash-flow dates depend on the instrument and contract. Prerequisite: timestamps and the data lifecycle in Chapter 1. By the end, you can trace who owes what in an equity, bond, FX or derivative trade, when assets move, and how notional exposure, margin and actual profit differ.

1. Learning objectives

Describe stocks, exchange-traded funds, bonds, FX, futures, options, swaps and repurchase agreements as checkable rights and obligations. Distinguish brokers, dealers, exchanges, central counterparties and custody/settlement institutions; arrange events by trade, clearing and settlement dates. With explicit units, calculate dividend-inclusive return, coupons, net swap payments, currency conversion, futures variation settlement, option profit at expiry and repo haircuts. Explain why short sales and margin arrangements can cause losses exceeding initial collateral.

This chapter establishes instrument terminology and cash-flow ledgers. Full bond valuation, option-pricing models and order-book execution are developed later, along with return construction, market microstructure and costs, stress risk and derivative pricing.

2. Why a quant needs market instruments

A numerical column often captures only one aspect of a contract. Equity price changes omit some dividends; fund-share prices differ from net asset value; futures index points become monetary profit only after applying a multiplier. An FX quote of "1.10" is uninterpretable without its direction, and trade dates can differ from cash-settlement dates.

These distinctions determine return denominators, information availability, loss settlement and the date assigned in a backtest. Treating futures notional exposure as invested principal gives the wrong return. Treating margin as maximum loss understates cash shortfalls before liquidation. Collapsing reporting endpoints, executions and settlement dates into one time makes a tidy ledger that may violate the contract.

Ask four questions first: what is traded, how is it quoted, what have the parties promised, and when do cash or securities transfer? Use them throughout the instruments below.

3. Intuition

Begin with three teaching ledgers, all constructed rather than historical. An investor buys 100 shares at USD 50 each; price later becomes USD 50.20 and the holder becomes entitled to USD 0.50 per share in dividends. A USD 1,000 fixed-coupon bond pays 6% annually in semiannual installments. For CME Micro E-mini S&P 500 futures (MES), the contract multiplier is USD 5 per index point.

Complete example: equity entitlement and cash events

Write the equity trade as an event table. Dates, prices and dividend arrangements are constructed. Record an established receivable separately from cash receipt, while marking securities at the price then prevailing.

Event	Rights and obligations	Quantity and amount	Cash change, investor's perspective
Trade T	Investor buys through a broker; counterparties acquire delivery obligations	100 shares × USD 50/share = USD 5,000	Not yet settled; cash is not automatically debited
Clearing	Determine deliveries; netting or a central counterparty (CCP) depends on applicable rules	100 shares against USD 5,000	Calculating obligations is separate from paying cash
Settlement	Investor pays and receives settled securities	100 shares delivered against USD 5,000	−USD 5,000
Dividend entitlement	Under the example, the holder acquires a claim against the issuer	100 shares × USD 0.50/share = USD 50	0; dividend receivable increases by USD 50
Dividend payment	Issuer pays through payment/custody arrangements	USD 50	+USD 50; receivable decreases by USD 50
Terminal valuation	Investor still holds 100 shares priced at USD 50.20/share	Securities worth USD 5,020	Valuation itself transfers no cash

Terminal wealth is USD 5,020 in securities plus USD 50 dividend cash, totaling USD 5,070. If payment remains outstanding but the entitlement is valid, cash has not increased; wealth includes the USD 50 receivable instead. Do not count both receivable and paid dividend. This simplified table omits borrowing, taxes, fees and cash interest. Check actual delivery obligations against trade confirmations and product rules. Chapter 11 constructs return intervals from these endpoint values; Chapter 29 adds execution and fee ledgers.

Equity holding-period wealth includes terminal security value and entitled dividends. Bondholders receive contractual interest and usually principal at maturity. Futures settle price changes through contract multipliers rather than requiring full cash purchase of an index basket, while demanding performance collateral. Each may respond to "a rising market," yet their cash flows differ.

Think of a contract as a promise with explicit units and dates. Units resemble quantities in a recipe; dates resemble an agreed collection time. Without either, a number has no operational meaning. The analogy is only a memory aid: actual rights come from product documents, applicable law and trading rules.

4. Mathematical foundations

State a currency for every amount: USD for dollars and EUR for euros. Prices per share use USD/share; futures multipliers use USD/index point with contract count specified. Multiplication by share or contract quantity produces USD. Returns are dimensionless ratios, usually stored as decimals: 5% is 0.05.

Let t_0 denote execution or the beginning of a holding interval and t_1 its endpoint. Signal time identifies when the strategy forms a signal from available information; trade time identifies when the order can actually execute. A return interval has explicit start and end. Execution t_0 is not automatically settlement or information availability. A backtest requires information available before signal formation and explicit execution delay. These constructed ledgers illustrate contract cash flows, not matching-engine behavior.

"One contract" is not a universal unit. Products specify size, currency, quote precision, maturity and delivery method. Define N, M, S and K where used: the units of quantity N and multiplier M, quote direction of price S, and currency and underlying unit of strike K.

5. Definitions

Stock/equity represents a company's residual ownership claim. Shareholders may receive cash dividends and voting rights, but dividends are not guaranteed fixed interest and equity usually ranks after creditors in liquidation. Ordinary-share rights, voting, distributions and corporate actions depend on issue documents and applicable law.

An **exchange-traded fund (ETF)** holds an asset basket or follows another documented fund strategy, with shares traded on a venue. Retail investors typically trade at intraday market prices. **Net asset value (NAV)** follows the fund's valuation rules; market price may trade at a discount or premium to NAV. An ETF share does not grant every investor on-demand redemption of each underlying asset. This introductory treatment follows US Investor.gov education; individual product terms differ.

A **bond** is a debt instrument promising contractual interest, principal repayment at maturity or another specified payment structure. Face value F is monetary, coupon rate c is annualized, and payment frequency m is payments/year. For a plain fixed coupon, each payment is Fc/m. Credit, seniority, collateral, early redemption and payment structure may change actual cash flows.

Foreign exchange (FX) exchanges one currency for another. Here $S_{Q/B}$ means units of quote currency Q for one unit of base currency B. With $S_{USD/EUR} = 1.10$, 1 EUR is quoted at 1.10 USD. EUR 10,000 converts to USD 11,000 at that midpoint, before spreads, fees or settlement arrangements.

A **futures contract** is a derivative traded on a standardized venue and settled daily or at the specified frequency under its rules. The **contract multiplier** converts underlying quote movements into contract monetary profit. CME MES pays USD 5 per contract per index point; E-mini S&P 500 futures (ES) pays USD 50. A multiplier sets the profit scale, rather than contract value or margin itself. Positive signed quantity is long; negative is short, describing

price exposure rather than ownership of index constituents. Periodic cash adjustment following futures-price changes is **variation margin**; contract rules govern its settlement.

An **option** gives its buyer a contractual exercise-or-abandon right and imposes the corresponding obligation on its seller. Strike price is K. Premium paid or received is an opening cash flow. A call's expiry **payoff** is $\max(S_T - K, 0)$; a put's is $\max(K - S_T, 0)$. S_T, K share the same underlying unit and quote currency. Exercise style, maturity and settlement assets depend on the contract.

A **swap** exchanges a contractual set of future cash flows. Interest-rate swaps use a notional amount to calculate interest without necessarily exchanging that principal: one side may pay fixed and receive floating, with contractual net settlement. Directions, floating benchmarks, reset dates and payment netting are specified. Central or bilateral clearing arrangements differ; the word "swap" alone determines no collateral rule.

A **repurchase agreement (repo)** sells securities with an agreement to repurchase them, economically often providing secured financing. A **haircut** is the proportion of collateral market value not financed. Contracts and market conventions determine actual haircuts, valuation frequency, margin calls and default disposal.

Definition: distinct trading, clearing and settlement responsibilities

A **broker** generally acts for customer orders or arranges trades. A **dealer** trades on its own account and bears inventory risk. A **market maker** supplies two-way quotations continuously or under specified arrangements. One institution may perform several roles; legal duties depend on jurisdiction and activity. An **exchange** organizes listings, quotes or matching. **Clearing** confirms obligations, nets them, manages risk or substitutes a central counterparty (CCP) for the original trading relationship. **Settlement** transfers securities, cash or other contractual assets. These are related stages with distinct meanings.

Margin consists of cash or securities required for performance or financing. A **securities margin loan** is broker financing secured by account assets. Futures margin resembles a **performance bond**, rather than a down payment on full notional exposure. **Initial margin** is required to open a position; **maintenance margin** is the minimum supporting continued holding. Rules and risk conditions can change amounts. **Collateral** supports performance or credit exposure, can change value and may require top-ups. Initial margin neither prepays a maximum loss nor limits losses; losses can exceed the original collateral.

6. Derivations

Hold N shares at beginning price P_0 and ending price P_1, with entitled cash dividend D per share. All prices and dividends use USD/share. Initial investment is NP_0 USD, terminal securities value NP_1 USD and dividend cash ND USD. This cash-equity example requires $N > 0$ and $P_0 > 0$; P_1 and D are finite and nonnegative in the stated model, giving a valid initial-capital denominator NP_0. Simple total return is therefore

$$R_{total} = \frac{NP_1 + ND - NP_0}{NP_0} = \frac{P_1 - P_0 + D}{P_0}.$$

Quantity N cancels because it multiplies both numerator and denominator. The derivation assumes an entitled holding-period dividend, cash receipt without reinvestment, and no taxes, fees, splits or other corporate actions. Price return is $(P_1 - P_0)/P_0$ and excludes the dividend.

A fixed-coupon bond's annual interest is face value times annual coupon rate, Fc. Dividing evenly among m payments gives $C = Fc/m$. USD 1,000 face value at annual rate 0.06 with two payments gives USD 30 per payment. Principal is separately due on the contractual repayment date. Coupon rate is neither market yield nor a direct statement of current bond price.

For a simplified single-currency fixed-for-floating swap with no principal exchange, let N be notional, r_f the period's annual floating rate, r_k the fixed annual rate and δ its year fraction. The receive-floating/pay-fixed side's net payment is

$$CF = N(r_f - r_k)\delta.$$

With $N = USD$ 1,000,000, $r_f = 3.6\%$, $r_k = 4.0\%$ and $\delta = 0.25$, net cash is $-USD$ 1,000: that side pays USD 1,000. N uses USD; rates are annualized fractions and δ is a dimensionless year fraction, so the result is USD. The formula assumes same-currency legs, one accrual interval, simple interest, no fees and payment at period end. Different benchmarks, day counts, frequencies or principal exchanges require contractual calculations. Notional is an interest base, separate from the net payment.

For futures, let q be signed contract count, positive for long and negative for short; M has units USD/(contract · index point), and ΔF_t is the change between consecutive settlement prices in index points. Periodic variation settlement is

$$VM_t = qM\Delta F_t \quad [USD].$$

A rising price produces positive cash for a long and negative cash for a short. If each settlement change is paid, several periods' nominal cash flows sum to $qM(F_T - F_0)$, provided quantity and multiplier remain constant, with no fees, additional collateral, delivery or other position changes. This sum excludes interest, reinvestment and financing on intervening balances; differently timed payments can therefore differ from financed economic profit. Futures profit is separate from a cash-security return. Dividing by chosen initial margin gives a return dependent on that capital convention, not a cash-asset return on notional principal.

Complete example: MES cash and three capital denominators

Place notional exposure, collateral balance and profit in separate columns. Take one long MES contract, USD 5 per index point, initially settled at 5,000 points, with an assumed USD 1,000 performance deposit. These are constructed conditions. No withdrawals, top-ups, interest or fees occur; balances illustrate cash needs rather than broker margin rules.

Settlement level	Reference notional USD	Period VM USD	Collateral account USD
5,000 (initial)	25,000	No return interval yet	1,000
5,020	25,100	+100	1,100
4,980	24,900	−200	900

Continued: the same complete example

5,010	25,050	+150	1,050

Cumulative trading profit is USD 50. Dividing by initial notional USD 25,000 gives 0.2%; dividing by the assumed USD 1,000 deposit gives 5%. If allocated within an initially USD 100,000 portfolio, its gross profit contribution is 0.05%. These answer different capital-base questions, so state each denominator. Actual margin calls also depend on maintenance requirements, other positions and client rules. A change in notional value is not an additional USD 50 payment.

Two covered-parity paths and a complete six-month example

FX units can be checked directly. Base-currency amount A_B has units B; multiplying quote $S_{Q/B}$ with units Q/B gives quote-currency amount $A_Q = A_B S_{Q/B}$ in Q. For simple interest, matched maturity and this quote direction, the teaching **covered interest parity (CIP)** reference is

$$F_{Q/B} = S_{Q/B}\frac{1 + r_Q\tau}{1 + r_B\tau}.$$

Start both paths with the same 1 B and express both terminal amounts in Q:

Path	Initial capital	Intermediate action	Terminal Q amount
Deposit B and cover FX	1 B	Deposit at the B rate; sell its terminal B amount forward at inception	$(1 + r_B\tau)F_{Q/B}$ Q
Convert spot to Q and deposit	1 B	Receive $S_{Q/B}$ Q at spot and deposit at the Q rate	$S_{Q/B}(1 + r_Q\tau)$ Q

With borrowing/lending at these rates, a lockable forward, no friction and matched dates, terminal values coincide: $(1 + r_B\tau)F_{Q/B} = S_{Q/B}(1 + r_Q\tau)$. Divide by the positive B accumulation factor to obtain the reference formula. If values differ, an arbitrage argument must also establish a financing leg; the starting 1 B is not free profit.

In the six-month USD/EUR example, the first path turns EUR 1 into EUR 1.015 and converts forward to USD $1.015F$. The second converts to USD 1.10 at spot, then reaches USD $1.10 \times 1.025 = 1.1275$. Hence $F = 1.1275/1.015 = 1.1108374384$ USD/EUR. The reverse quote is $1/F$ EUR/USD; reversing currency labels also swaps the interest rates in numerator and denominator.

Here r_Q, r_B are annual simple rates, τ is in years, and F and S both use Q/B. Both accumulation factors, $1 + r_Q\tau$ and $1 + r_B\tau$, must be positive. The relation equates terminal value of depositing 1 B with converting to Q and depositing there under forward-covered conversion. Actual trades also involve spreads, day counts, delivery dates, credit, financing, collateral and cross-currency basis. A conditional relation is not a promise of executable arbitrage.

7. Financial interpretation

A cash-equity purchase pays the full execution value; profit comes from price movement, dividends and corporate actions. A **short sale** typically borrows securities through securities lending, sells them and later repurchases and returns them. Rising prices increase repurchase cost; borrowing fees, substitute dividend payments, recalls, margin and compulsory buy-ins also matter. This mechanism implies no uniform short-selling rule across markets.

Bondholders have contractual payment claims but face issuer default, interest-rate, inflation, liquidity, reinvestment and early-redemption risk. The **term structure** relates rates or yields across maturities. **Duration** measures local price sensitivity to a small yield change; **convexity** measures curvature. Modified duration D_{mod} and convexity C give the local approximation

$$\frac{\Delta P}{P} \approx -D_{mod}\Delta y + \frac{1}{2}C(\Delta y)^2.$$

When Δy is a decimal annual-yield change, duration is measured in years under the compounding convention and C in years squared. A one-percentage-point increase is $\Delta y = 0.01$, or 100 basis points (bps), not one basis point; 1 bp = 0.0001. The approximation requires a local shock, substantially fixed cash flows and a consistent yield definition. It excludes full repricing, credit-spread changes, optionality and nonparallel curve shifts. Calculate these sensitivities under explicit compounding below. Chapter 27 uses zero-coupon duration for carry and rolldown; Chapter 44 develops derivative pricing. No separate, undeclared fixed-income chapter is assumed.

7.1 From cash flows to duration and convexity

Extend Chapter 4's single-payment discounting to fixed cash flows. Payment CF_i USD occurs in year t_i, discounted at one effective annual yield $y > -1$. This valuation example makes no separate accrued-interest deduction:

$$P(y) = \sum_i CF_i(1+y)^{-t_i}.$$

Differentiate each power term and sum: $P'(y) = -\sum_i t_i CF_i(1+y)^{-t_i-1}$ and $P''(y) = \sum_i t_i(t_i+1)CF_i(1+y)^{-t_i-2}$. Define modified duration $D_{mod} = -P'(y)/P(y)$ and convexity $C = P''(y)/P(y)$, converting USD price derivatives into relative sensitivity. Taylor expansion gives $P(y+\Delta y) = P(y) + P'(y)\Delta y + P''(y)(\Delta y)^2/2 + \cdots$; division by $P(y) > 0$ produces the relative-price approximation. Under annual compounding, **Macaulay duration** is $D_{Mac} = \sum_i t_i[CF_i(1+y)^{-t_i}/P]$; comparing terms yields $D_{mod} = D_{Mac}/(1+y)$. Years are the accrual basis. Semiannual compounding, separate discount curves or optional cash flows require new derivatives.

Complete example: two-year bond sensitivity and exact repricing

Construct a USD 1,000 face-value bond paying USD 60 annually and maturing in two years. Value it at $y = 5\%$:

Payment year	Cash flow USD	Discounted value USD	Contribution to $-P'$	Contribution to P''
1	60	$60/1.05 = 57.142857$	$60/1.05^2 = 54.421769$	$120/1.05^3 = 103.660512$
2	1,060	$1060/1.05^2 = 961.451247$	$2120/1.05^3 = 1831.335709$	$6360/1.05^4 = 5232.387740$
Total	1,120 (undiscounted)	$P = 1018.594104$	1885.757478	5336.048252

Thus $D_{mod} = 1.851334$ and $C = 5.238640$. Yield rises to 6%, with $\Delta y = 0.01$. The first-order term is -0.018513336 and the second-order term $+0.000261932$, totaling approximately -1.825140%. Exact repricing gives $60/1.06 + 1060/1.06^2 = 1000$ USD, a change of -1.825467%. Convexity improves this small-shock approximation without replacing discounting. Use it to estimate local rate exposure; if deteriorating credit changes cash flows, the problem is no longer one in which only y changes.

Futures notional exposure is the price level times multiplier times contract count. A constructed MES contract near 5,000 points represents 5,000 index points × USD 5/index point = USD 25,000. This reference exposure is usually neither USD 25,000 paid at inception nor margin. Profit uses subsequent point changes. Exchanges, clearing members and brokers adjust margin for products, clients, volatility and rules. Shortfalls may cause calls, reductions or liquidation; extreme losses can exceed initial collateral.

A **futures roll** closes a nearby or expiring contract and opens a later maturity. Record both contracts, execution prices, quantities, multipliers, costs and times. The near/far spread is not automatically a separate booked loss. Attribution and empirical roll costs belong to later strategy and trading chapters.

Option rights and obligations are asymmetric. Buyers usually pay premium upfront; pre-expiry profit also depends on time, implied volatility, rates, dividends and exercise rules. Expiry payoff shows only terminal intrinsic value. For a simply fully paid long option without subsequent costs, maximum contract loss is usually premium paid. Sellers owe performance; a short call can incur very large losses. One expiry-payoff line is no substitute for full risk disclosure.

Swaps and repos specify dated payment obligations rather than forming one generic "leverage" category. Swaps may settle net payments; repos provide securities-backed financing with reverse repurchase. Product documents determine clearing, collateral and default terms. Central clearing changes counterparties and requires margin without removing economic losses.

8. Numerical Example

Equity total return. Construct $N = 100$ shares, $P_0 = USD\ 50.00/share$, $P_1 = USD\ 50.20/share$ and $D = USD\ 0.50/share$. Initial investment is USD 5,000; terminal securities USD 5,020 plus dividends USD 50 give USD 5,070. Total return is $(5070 - 5000)/5000 = 0.014 = 1.4\%$. Price-only return is $(5020 - 5000)/5000 = 0.004 = 0.4\%$.

Bond coupon. USD 1,000 face value, 6% annual coupon and two payments per year give each coupon $1000 \times 0.06 \div 2 = USD\ 30$. Assume one-year holding and timely payment: USD 30 at six months, then USD 30 plus USD 1,000 principal at twelve months, totaling USD 1,030 at year end. Including the earlier coupon gives USD 1,060 received over the year. Purchase price, accrued interest, taxes, default and reinvestment are omitted.

Interest-rate swap. Construct an effective date of January 4, 2027 and first settlement on April 5, 2027, using year fraction $\delta = 0.25$ for hand calculation. Notional is USD 1,000,000, receiving floating 3.6% annually and paying fixed 4.0% over that interval. Net cash for the floating receiver is $1{,}000{,}000 \times (0.036 - 0.04) \times 0.25 = -USD\ 1{,}000$, requiring payment of USD 1,000. Only interest differences are exchanged, with no principal exchange, fees, discounting, credit or collateral costs. Dates and rates are constructed.

MES periodic settlement. The reviewed CME specification gives USD 5/index point. Construct settlements at 5,000, 5,020, 4,980 and 5,010. One long receives/pays $+20 \times 5 = +USD\ 100$, $-40 \times 5 = -USD\ 200$ and $+30 \times 5 = +USD\ 150$, totaling +USD 50, matching $5 \times (5010 - 5000) = USD\ 50$. A 300-point fall from 5,000 produces $-300 \times 5 = -USD\ 1{,}500$. This exceeds an assumed initial USD 1,000 deposit. That deposit is a drawing assumption, not a current CME margin quote or guaranteed maximum loss.

FX quote. With $S_{USD/EUR} = 1.10\ USD/EUR$, EUR 10,000 converts to USD 11,000. At annual simple USD rate 5%, EUR rate 3% and six months, the simplified reference forward is $1.10 \times (1 + 0.05 \times 0.5)/(1 + 0.03 \times 0.5) = 1.110837\ USD/EUR$. Actual two-way quotes depend on more than this equation.

Options and repo. Construct one call with 100 shares/contract, strike USD 100/share, premium USD 4/share and terminal stock USD 110/share. Long profit at expiry is $(110 - 100 - 4) \times 100 = USD\ 600$. For a short put with stock USD 90/share, strike USD 100/share and premium USD 3/share, profit is $-[(100 - 90) - 3] \times 100 = -USD\ 700$. A constructed repo with collateral worth USD 1,000,000 and a 5% haircut advances $1{,}000{,}000 \times (1 - 0.05) = USD\ 950{,}000$. The USD 50,000 value buffer does not guarantee default recovery.

Separate the call example by time and state. Opening premium paid is $4 \times 100 = USD\ 400$; at expiry $S_T = 110$ yields $(110 - 100) \times 100 = USD\ 1{,}000$, for nominal net cash USD 600. If $S_T \leq 100$ at expiry, payoff is zero and profit −USD 400. If $100 < S_T < 104$, payoff is positive but profit remains negative; break-even is 104. Here profit is expiry net profit ignoring financing and costs. Bringing opening premium to the same terminal date requires its funding cost. The short put receives USD 300 initially and pays USD 1,000 at expiry, net −USD 700. Retain both negative payoff and received premium.

9. Python Implementation

Default entry in the reader edition

From the accepted I3 reader package root, prepare an independent environment following its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch10_markets.ipynb
```

The Notebook calls the packaged shared implementation using constructed contract cash flows throughout Chapter 10. The runner saves fresh-kernel execution records to a unique runs/execution_* directory; new figures use another UUID directory. The terminal prints actual locations. Omitted French data are not read by default or downloaded. The old production tests and CLI below refer to the working-course root; reader checks use the Notebook assertions above. Optional historical data follow the data contract and are not rerun in this revision.

Historical working-course root and original implementation

Core functions in `src/quant_course/market_instruments.py` state currencies and quote directions through names, parameters and docstrings. They reject NaN, infinities, missing and invalid inputs rather than disguising them as zero. They calculate equity total return, coupons, simplified swap net cash, FX conversion/reference forwards, futures variation settlement, option expiry profit and repo advances. `code/ch10_markets.py` provides the historical CLI and teaching figure; tests use pytest. These commands run from the working-course root:

```
.venv/bin/python code/ch10_markets.py --output-dir figures/a3
.venv/bin/python -m pytest tests/test_ch10_markets.py -q
```

The original `notebooks/ch10_markets.ipynb` is opened and run sequentially from the project root. Notebook and tests call shared package functions; the CLI generates the figure and demonstration output. There is no observed price series or random simulation, so no seed is specified. Inputs are constructed cash-flow events or settlement intervals. Outputs are amounts/returns, without tradable signals, signal times, trade times or return estimation.

Interfaces require the caller to establish holding-period dividend entitlement. FX is always quote currency per one base unit; futures changes are index points between consecutive settlements. Option premium and terminal underlying prices share currency and underlying units, with explicit multiplier. Repo haircut satisfies $0 \leq h < 1$. Functions return amounts or simple returns without selecting taxes, financing, calendars, clearing or contract rules.

10. Visualization

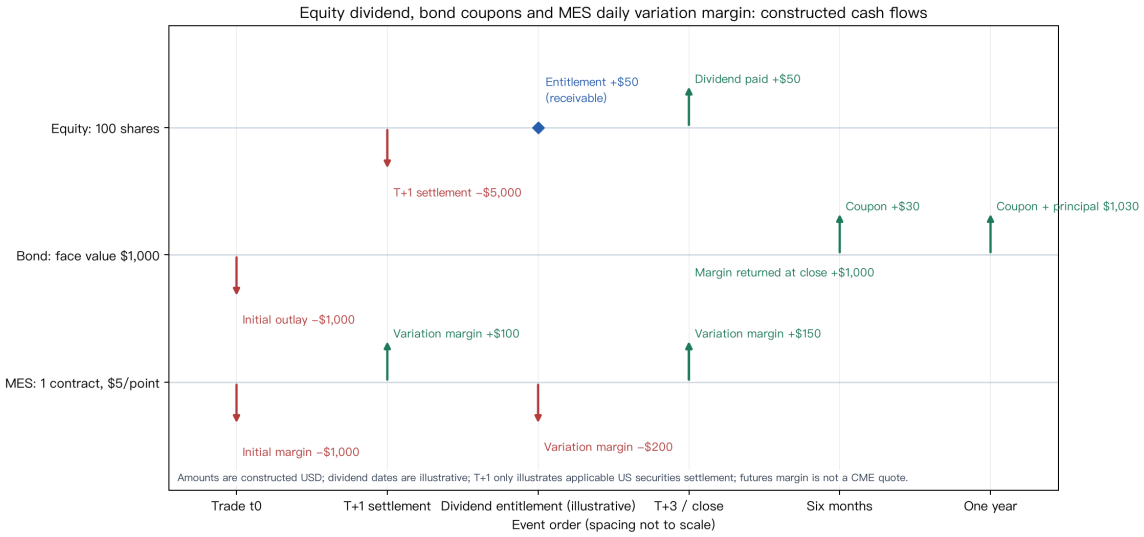


Figure 1: Constructed contract cash-flow timeline

The script generated this cash-flow timeline. Its horizontal axis lists events from trade through T+1, illustrative dividend entitlement, T+3, and semiannual/annual bond nodes. Spacing is not proportional to days or a calendar. T+1 illustrates applicable US security settlement only; dividend and other-lane dates are constructed rather than synchronized market events. Vertical lanes classify equities, a USD 1,000 face-value bond and one MES; they are not amount scales. Arrow labels use USD; upward means receipt and downward means payment/deposit. A blue diamond separately marks dividend entitlement, not cash receipt.

The equity purchase settles at the constructed T+1 node; entitlement and later dividend payment are distinct. Bond coupons follow constructed semiannual dates. Futures settle signed periodic cash flows and illustrate return of assumed collateral on closure. Collateral transfers account funds rather than constituting a trading loss. Compare timing and cash-flow nature, not arrow heights or spacing across lanes.

11. Real Market Example

This chapter neither loads/downloads market prices nor treats constructed values as executions. Actual-rule examples concern US securities settlement and CME specifications. The SEC staff FAQ identifies May 28, 2024 as the compliance date. Rule 15c6-1 covers the specified security transactions executed by US-registered brokers/dealers, with listed instrument/arrangement exceptions and some expressly agreed extended settlement at execution. For a covered trade on Tuesday May 28, 2024, business-day T+1 is Wednesday May 29. This is no common calendar for all bonds, FX, futures, repos, cross-border securities or transactions.

DTCC descriptions of NSCC Continuous Net Settlement (CNS) and DTC securities/cash book-entry transfers illustrate US infrastructure rather than defining global clearing. Reviewed CME specifications state MES as USD 5 × S&P 500 index points and ES as USD 50 × points.

Only multipliers are used, excluding changing margin quotes and notional valuations. See `source_notes.md` for pages and reading scope.

12. Common Mistakes

Calling price return equity total return. The 100-share example gives price-only return 0.4%, or 1.4% after USD 0.50/share dividend. Name the measure and verify holding-period entitlement.

Omitting FX quote direction. "1.10" alone converts nothing. USD/EUR=1.10 means USD 1.10 per EUR; the reverse EUR/USD is 1/1.10 EUR/USD. State numerator/denominator currencies, then cancel units.

> **Common mistake: margin is not a loss ceiling**
>
> **Combining futures notional, margin and profit.** MES at 5,000 points has USD 25,000 reference notional; a 20-point move produces USD 100. An assumed USD 1,000 deposit neither changes the profit formula nor caps loss. Margin supports risk management and performance.

Treating trade date as cash settlement. T+1 belongs to specific markets, instruments and rules. T is trade date; count subsequent days under the relevant business calendar. Economic exposure may begin at execution, while available cash or settled securities follow rules. Record them separately.

Drawing only option expiry payoff. It omits opening premium, pre-expiry market value, assignment risk and contract adjustments. A single upward/downward payoff curve cannot establish the risks of long and short options.

Treating a repo haircut as risk-free insurance. A haircut provides an initial difference between funding and collateral value. Price gaps, liquidity discounts, legal enforcement and counterparty risk can consume the buffer. One haircut value does not eliminate credit risk.

Calling a futures roll spread a realized loss. A roll closes the nearby position and opens a later contract. Different quotes change position reference prices but do not automatically equal realized cash loss. Outcomes depend on quantity, multipliers, close/open prices, costs, margin and underlying movements. Term-structure and roll-return attribution follow later.

13. Quant Research Application

Build a research prototype that checks units and timing first. Record instrument, contract code, quote currency/unit, multiplier, maturity, trade/settlement times and data availability. Store executions, terminal marks, dividends/coupons, variation settlement, fees and collateral changes separately. Do not force heterogeneous instruments into one return column.

For cash equities, identify adjusted-price conventions, dividend accrual/payment and tax basis. For futures, specify notional, margin-capital or NAV denominator and roll rules. For FX, state home/foreign direction and conversion date. Missing fields should fail checks or enter a

defined quality queue, never default to zero. Strategies additionally require Chapter 3 information clocks, Chapters 29–30 execution delay/look-ahead checks, and Chapters 34–36 quotes, order books and costs. This ledger alone is no backtest evidence.

14. Chapter Summary

Financial instruments specify future rights and obligations, differing in underlying, units, funding, return sources and settlement. Stocks and bonds are cash securities; ETFs are fund shares. Futures, options and swaps are derivative contracts; repo provides secured financing; FX requires a quote direction. Brokerage, trading, clearing and settlement have distinct responsibilities.

Begin calculations with units and timing. State currencies, unit prices, quantities and multipliers, then distinguish cash entitlement from payment. Total return differs from price movement; notional differs from invested capital; margin differs from maximum loss; T+1 differs from a universal settlement rule.

15. Exercises

10-E01 (concepts). What may brokers, dealers and market makers do? Must they be three separate institutions?

10-E02 (concepts). Distinguish ETF intraday price, NAV and underlying securities. Why is buying an ETF share not the same as buying every underlying security individually?

10-E03 (concepts). In the US example, what do T, clearing and settlement mean? Why is T+1 no universal rule for all global securities and derivatives?

10-E04 (concepts). Explain why futures margin is not maximum loss. Distinguish a securities margin loan from a futures performance bond.

10-E05 (concepts). What cash obligations and operational risks accompany a short sale beyond a rising stock price?

10-E06 (hand calculation). Buy 100 shares at USD 50/share; terminal price is USD 49.80/share, with entitled dividend USD 0.30/share. Calculate price and total returns, stating the denominator.

10-E07 (hand calculation). One long MES contract has USD 5/index point multiplier. Settlements are 4,000, 3,992, 4,005. Calculate both variation payments and cumulative profit. Why does assumed USD 500 collateral not cap loss?

10-E08 (hand calculation). A USD 1,000,000 notional interest-rate swap receives floating 3.6% and pays fixed 4.0%, with year fraction $\delta = 0.25$. Calculate net cash and explain whether notional transfers on this payment date.

10-P01 (programming). Use the chapter functions to calculate E06 total return and one coupon for USD 5,000 face value, 4% annual rate and quarterly payments. Print amounts/ratios and assert the results.

10-P02 (programming). Convert EUR 25,000 using $S_{USD/EUR} = 1.08\ USD/EUR$, then calculate the simplified six-month covered-parity forward with USD rate 4% and EUR rate 2%. Print direction and assert the formula.

10-P03 (programming). Construct a short MES position with settlement changes $(-12,+7)$ points and calculate cash flows. Calculate a long put with terminal underlying USD 85/share, strike USD 90/share, premium USD 2/share and multiplier 100, then a repo advance for USD 2,000,000 collateral and 3% haircut. Assert all three results.

16. Programming Exercises

10-P04. Extend a ledger function accepting dated amounts, currencies and event types, ordered by time. Sum within currency only; reject NaN, infinities or missing currency. Return daily balances and cumulative cash flows. Use separate dividend-entitlement/payment dates to demonstrate that events are not merged.

10-P05. Write a contract checker accepting instrument, quote string, multiplier, quantity and price change. Check monetary units. Reject missing FX direction, nonpositive multipliers and futures changes mislabeled as "USD per contract" instead of index-point changes paired with USD/index point.

17. Challenge Problems

10-C01 (research design). A cash ETF and MES referencing the same index each have initial USD 25,000 notional exposure. Design a comparison separating holding return, dividends/financing, fees, rolls, margin capital and settlement cash flows. Specify at least four required data items and explain why percentage price movements cannot establish replicable returns.

10-C02 (reasoning). An institution pledges bonds worth USD 1,000,000 in a repo with 5% haircut and borrows USD 950,000. Design price, liquidity/haircut and counterparty/settlement stresses. Explain worsening balances and the contractual/market data needed to assess loss. Why does the initial haircut not ensure full recovery?

Additional independent exercises

10-N01 (events and state). Use the Section 3 table: terminal stock price is 50.20 and USD 50 dividend entitlement is established but unpaid. State securities, receivable, dividend cash and total wealth before and after payment. Why should payment not increase wealth a second time?

10-N02 (two FX paths). Starting with EUR 1, spot USD/EUR=1.10, simple annual USD rate 5%, EUR rate 3% and six months, write both terminal USD paths and solve the forward. Explain the reciprocal for EUR/USD.

10-N03 (sensitivity). Use the Section 7.1 two-year bond's cash flows to compute price, modified duration and convexity. Give first-order, second-order and exact repricing for a 100bps increase. State cash-flow and compounding assumptions.

10-N04 (capital and option profit). Why does USD 50 MES profit produce 0.2%, 5% and 0.05% in Section 6? For a call with strike USD 100, premium USD 4 and multiplier 100 shares, calculate payoff/profit at terminal prices 102 and 104.

18. Solutions / Solution Hints

10-E01. Brokers transmit or arrange customer orders; dealers take inventory/counterparty exposures on their own accounts; market makers provide two-way quotes under institutional/commercial arrangements. One entity can perform several roles, with jurisdiction-, venue- and activity-specific duties and disclosure. Economic roles do not determine legal-entity count.

10-E02. ETF shares are units of fund ownership, with venue prices determined by supply/demand and NAV by valuation rules. Discounts/premiums may be temporary or persistent. Ordinary secondary-market purchases do not deliver the underlying basket on every trade. Authorized participants usually handle creation/redemption under product rules; consult the fund documents.

10-E03. T is execution date. Clearing confirms trades, calculates obligations and may net/manage risk; settlement transfers cash/securities. The SEC staff FAQ describes T+1 from May 28, 2024 for transactions covered by Rule 15c6-1 and US-registered brokers/dealers, with scope, exceptions and expressly extended-settlement cases. Other markets, instruments and arrangements require their own rules and business calendars.

10-E04. Securities margin loans finance purchases against account collateral; futures margin supports performance while profit settles through point changes and multipliers. Shortfalls may cause cash calls, reductions or forced liquidation; gaps can produce losses beyond deposits.

10-E05. A short seller must repurchase and return borrowed securities. Rising prices increase repurchase cost; borrowing fees, substitute dividends, recalls, changing borrow availability, margin and compulsory buy-ins add obligations. Rules and eligible securities vary by market and broker.

10-E06. Initial investment $100\times50 = USD\ 5,000$; terminal securities $100\times49.80 = USD\ 4,980$ and dividends $100\times0.30 = USD\ 30$ give USD 5,010. Price return $100\times(49.80-50)/5,000 = -0.004 = -0.4\%$; total return $(5,010-5,000)/5,000 = 0.002 = 0.2\%$. The denominator is initial purchase capital USD 5,000. Assume entitled dividends and no taxes or fees.

10-E07. First change $3992 - 4000 = -8$ points gives VM $-8 \times USD\ 5/point = -USD\ 40$. Second change $4005 - 3992 = +13$ points gives $+13 \times 5 = +USD\ 65$. Cumulative $-40 + 65 = +USD\ 25$ also equals $5 \times (4005 - 4000)$. USD 500 is assumed collateral. A subsequent 101-point adverse move can lose USD 505 before costs and may impose calls. Collateral does not truncate profit/loss.

10-E08. The receive-floating/pay-fixed net amount is $N(r_f - r_k)\delta$. Substitution gives $1{,}000{,}000 \times (0.036 - 0.04) \times 0.25 = -1{,}000$, so that side pays USD 1,000. In this simplified interest-only swap, notional is a calculation base and does not transfer on payment day. A principal-exchange clause would require a different ledger.

10-P01 reference implementation (run independently from the working-course root):

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd() / "src"))

from quant_course.market_instruments import bond_coupon_cashflow, equity_total_return

total_return = equity_total_return(100, 50.0, 49.8, 0.3)
coupon = bond_coupon_cashflow(5_000.0, 0.04, 4)
assert abs(total_return - 0.002) < 1e-12
assert abs(coupon - 50.0) < 1e-12
print(f"Total return={total_return:.3%}; Coupon per payment=USD {coupon:.2f}")
```

Expected output is total return 0.200% and coupon per payment USD 50.00. Equity return follows Section 6; the coupon is $5000 \times 0.04 \div 4 = 50$.

10-P02 reference implementation:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd() / "src"))

from quant_course.market_instruments import covered_fx_forward_quote,
    ↪ fx_convert_base_to_quote

spot_usd_per_eur = 1.08
usd = fx_convert_base_to_quote(25_000.0, spot_usd_per_eur)
forward = covered_fx_forward_quote(spot_usd_per_eur, 0.04, 0.02, 0.5)
assert abs(usd - 27_000.0) < 1e-10
assert abs(forward - 1.0906930693069308) < 1e-12
print(f"USD per EUR: spot={spot_usd_per_eur:.4f}, reference forward={forward:.6f}")
print(f"EUR 25,000 converts to USD {usd:,.2f} at the stated spot")
```

Units are USD/EUR, so EUR 25,000 $\times$ 1.08 USD/EUR = USD 27,000. The simplified forward is $1.08 \times 1.02/1.01 \approx 1.0906930693\ USD/EUR$, under the frictionless teaching assumptions.

10-P03 reference implementation:

```
import sys
from pathlib import Path
sys.path.insert(0, str(Path.cwd() / "src"))

from quant_course.market_instruments import (
    futures_variation_margin_usd, listed_option_pnl, repo_cash_advanced,
)

short_mes = futures_variation_margin_usd(-1, 5.0, [-12.0, 7.0])
put_pnl = listed_option_pnl(85.0, 90.0, 2.0, 1, 100, "put")
repo_cash = repo_cash_advanced(2_000_000.0, 0.03)
assert short_mes == (60.0, -35.0)
assert abs(put_pnl - 300.0) < 1e-12
assert abs(repo_cash - 1_940_000.0) < 1e-8
print(f"short MES settlement cashflows: {short_mes} USD")
print(f"long put expiry P&L: USD {put_pnl:.2f}")
print(f"repo cash advanced: USD {repo_cash:,.2f}")
```

The short receives USD 60 for a 12-point fall, then pays USD 35 for a 7-point rise, totaling USD 25. The put has USD 5/share intrinsic value, less USD 2/share premium, times 100 shares: net USD 300. Repo advances USD 2,000,000 $\times$ 97% = USD 1,940,000.

10-P04 reference implementation (independent execution):

```
from dataclasses import dataclass
from datetime import date
import math
import re

@dataclass(frozen=True)
class CashEvent:
    event_date: date
    amount: float
    currency: str
    event_type: str
    moves_cash: bool

def cash_ledger(events: list[CashEvent]) -> list[dict[str, object]]:
    ordered = sorted(events, key=lambda event: event.event_date)
    balances: dict[str, float] = {}
    rows: list[dict[str, object]] = []
    for event in ordered:
        if not isinstance(event.event_date, date):
            raise TypeError("event_date must be datetime.date")
        if (
            isinstance(event.amount, bool)
            or not isinstance(event.amount, (int, float))
            or not math.isfinite(event.amount)
        ):
            raise ValueError("amount must be a finite number")
        if (
            not isinstance(event.currency, str)
            or re.fullmatch(r"[A-Z]{3}", event.currency.upper()) is None
```

```
        ):
            raise ValueError("currency must be a three-letter code")
        if not isinstance(event.event_type, str) or not event.event_type:
            raise ValueError("event_type must be non-empty")
        if not isinstance(event.moves_cash, bool):
            raise TypeError("moves_cash must be a boolean")
        currency = event.currency.upper()
        cash_flow = float(event.amount) if event.moves_cash else 0.0
        balances[currency] = balances.get(currency, 0.0) + cash_flow
        rows.append({
            "date": event.event_date.isoformat(),
            "event_type": event.event_type,
            "currency": currency,
            "event_amount": float(event.amount),
            "cash_flow": cash_flow,
            "running_cash": balances[currency],
        })
    return rows

events = [
    CashEvent(date(2025, 1, 2), 50.0, "USD", "dividend_entitlement", False),
    CashEvent(date(2025, 1, 6), 50.0, "USD", "dividend_payment", True),
]
rows = cash_ledger(events)
assert rows[0]["date"] == "2025-01-02" and rows[0]["cash_flow"] == 0.0
assert rows[0]["running_cash"] == 0.0
assert rows[1]["date"] == "2025-01-06" and rows[1]["cash_flow"] == 50.0
assert rows[1]["running_cash"] == 50.0
print(rows)
```

The ledger retains entitlement date but increases USD cash only on payment date. It does not treat a receivable as cash. Currency balances accumulate separately. NaN, infinities, empty currency and invalid events fail. Scoring, 10 points: fields/currency checks 2, missing/nonfinite rejection 2, separate entitlement/payment dates 2, reproducible daily/cumulative balances 2, tests/units 2. Entitlement-as-cash or cross-currency summation cannot earn full credit.

10-P05 reference implementation (independent execution):

```
import math
import re

def convert_fx(amount_base: float, quote: str, quote_per_base: float) -> tuple[str,
    ↪ float]:
    if not isinstance(quote, str) or re.fullmatch(r"[A-Z]{3}/[A-Z]{3}", quote) is
    ↪ None:
        raise ValueError("quote must state QUOTE/BASE, for example USD/EUR")
    if not math.isfinite(amount_base) or not math.isfinite(quote_per_base):
        raise ValueError("FX amount and rate must be finite")
    if amount_base < 0 or quote_per_base <= 0:
        raise ValueError("amount must be non-negative and rate positive")
    quote_currency, _base_currency = quote.split("/")
    return quote_currency, amount_base * quote_per_base

def futures_pnl(
```

```
    signed_contracts: int,
    multiplier: float,
    multiplier_unit: str,
    price_change: float,
    price_change_unit: str,
) -> float:
    if isinstance(signed_contracts, bool) or not isinstance(signed_contracts, int):
        raise TypeError("signed_contracts must be an integer")
    if multiplier_unit != "USD/index point" or price_change_unit != "index point":
        raise ValueError("futures multiplier and price-change units do not match")
    if not math.isfinite(multiplier) or multiplier <= 0 or not math.isfinite(
    ↪ price_change):
        raise ValueError("multiplier must be positive and inputs finite")
    return signed_contracts * multiplier * price_change

currency, converted = convert_fx(10_000.0, "USD/EUR", 1.10)
mes_pnl = futures_pnl(-1, 5.0, "USD/index point", -12.0, "index point")
assert currency == "USD" and converted == 11_000.0
assert mes_pnl == 60.0
for invalid_pair in ("", "1.10", "EURUSD"):
    try:
        convert_fx(10_000.0, invalid_pair, 1.10)
    except ValueError:
        pass
    else:
        raise AssertionError(f"accepted ambiguous FX quote: {invalid_pair!r}")
try:
    futures_pnl(1, 0.0, "USD/index point", 1.0, "index point")
except ValueError:
    pass
else:
    raise AssertionError("accepted zero multiplier")
try:
    futures_pnl(1, 5.0, "USD/contract", 1.0, "USD")
except ValueError:
    pass
else:
    raise AssertionError("accepted mismatched units")
print(f"FX conversion: {currency} {converted:.2f}; short MES P&L: USD {mes_pnl:.2f}")
```

FX assertions reject missing/ambiguous direction; futures require matching contract × USD/index point × index point units. Scoring, 10 points: FX direction/conversion units 3, multiplier/direction checks 2, futures-unit derivation 2, counterexample tests 2, field-specific errors 1.

10-C01 reasoning and scoring, 10 points. Include execution prices/fees, ETF distributions/fund fees, daily futures settlement, margin capital, rolls/maturities, financing/cash rates, settlement times and FX conversion. Supply at least four auditable data items; distinguish ETF NAV from traded price and specify multiplier/delivery. Compare equal notional or risk budgets with an explicit denominator. A supported conclusion is that the tools have different cash flows, funding and implementation risks despite shared index exposure; percentage price changes cannot establish replicable net returns. Score denominator 2, data 3, cash-flow/cost distinctions 3, restrained causal/arbitrage claims 2.

10-C02 reasoning and scoring, 10 points. Initial buffer is USD 50,000. A 4% collateral decline leaves USD 960,000 against USD 950,000 debt, only USD 10,000 buffer before interest/fees. Raising haircut from 5% to 8% supports USD 920,000 funding on USD 1,000,000 collateral, potentially requiring USD 30,000 cash/collateral. A price gap, delayed default disposal and additional 7% liquidation discount can leave recovery below debt; disposal costs widen the gap. Assess repo agreements, valuation/top-up frequency, interest, insolvency/clearing law, liquidity and stress prices. Score at least two quantified stresses 4, contractual/execution data 3, explanation that haircut does not guarantee recovery 3. Other consistent inputs/formulas are acceptable.

10-N01 complete solution. Before payment: securities USD 5,020, dividend receivable USD 50, dividend cash zero, total USD 5,070. After payment: receivable zero, dividend cash USD 50, securities unchanged, total still USD 5,070. Converting a receivable into cash does not add a second USD 50 to wealth already containing it. Initial capital USD 5,000 gives 1.4% total return with valid entitlement and no taxes/fees.

10-N02 complete solution. EUR deposits grow to EUR $1(1 + 0.03 \times 0.5) = 1.015$ and forward conversion gives USD $1.015F$. Spot-to-USD deposits reach USD $1.10(1 + 0.05 \times 0.5) = 1.1275$. Equality gives $F = 1.1108374384$ USD/EUR, reciprocal approximately 0.9002217295 EUR/USD. Paths share initial capital, maturity and currency, with positive accumulation factors and the borrowing/quotation/cost assumptions stated above.

10-N03 complete solution. $P = 60/1.05 + 1060/1.05^2 = 1018.594104$; $-P' = 60/1.05^2 + 2120/1.05^3 = 1885.757478$; $P'' = 120/1.05^3 + 6360/1.05^4 = 5336.048252$. Divide by P to obtain $D_{mod} = 1.851334$ and $C = 5.238640$. A 100bps shock is 0.01: first-order change -1.851334%, second-order -1.825140%. Exact $P(0.06) = 1000$ gives -1.825467%. Conditions are effective annual compounding, fixed cash flows, one yield and no accrued interest, default or callability.

10-N04 complete solution. $50/25000 = 0.2\%$ uses initial reference notional; $50/1000 = 5\%$ uses assumed margin; $50/100000 = 0.05\%$ is gross contribution to initial portfolio NAV. Margin does not cap loss. At 102, payoff=$(102 - 100)100 = USD\ 200$ and profit=$200 - 400 = -USD\ 200$. At 104, payoff is USD 400 and profit zero. Funding and fees are omitted; break-even is $K + premium = 104$.

19. Further Reading

- **Must Know:** Investor.gov equity, ETF, bond, margin, short-sale and order-execution education introduces investor rights/obligations without replacing contracts or regulation.
- **Must Know:** SEC T+1 FAQ and DTCC clearing/settlement descriptions, limited to their US security/service scopes and exceptions.
- **Must Know:** CME MES/ES specifications and margin education for multipliers, quote precision and terminology; actual trading also requires current rulebooks and broker requirements.
- **Should Know:** OCC standardized-option disclosures, especially exercise, assignment and expiry. The complete document was not reviewed page by page.
- **Should Know:** CFTC swap/clearing explanations, ECB repo mechanisms and BIS FX-swap discussion for notional, cash-flow and counterparty distinctions; individual materials do not establish current market size.
- **Advanced:** MIT OpenCourseWare 15.401 fixed-income materials for yields and rate-risk measures. Section 7.1 gives fixed-cash-flow local sensitivity; Chapter 27 uses zero-coupon duration for rolldown and Chapter 44 develops derivative pricing.

20. References

See `source_notes.md` for URLs, actual reading scope, boundaries and dates. Main sources are official SEC/Investor.gov, DTCC, CME, OCC, CFTC, ECB, BIS and MIT OCW materials. Original constructed inputs illustrate arithmetic, separate from source quotes or observations. US T+1, CME multipliers and margin statements support only the named rules/contracts and are not generalized to other markets/products.

The C5 sensitivity bridge uses original derivations and constructed cash flows. The October 13, 2026 supplemental source read covered MIT Part IV transcript 17:05–19:32 on local approximations and risk use. See the batch source review for that scope and inherited evidence.

Chapter 11 — Returns and Compounding

Main thread: establish what happened in the account before choosing a return measure. Prerequisites are Chapter 2 (Python), Chapter 6 (probability), and Chapter 10 (market instruments). Total return, simple return, log return, means, growth rates, and the Sharpe ratio answer different questions. We start with wealth recursion and proceed to financial-data calculations over matching intervals. All constructed examples are identified; the historical example uses a frozen French monthly release snapshot.

1. Learning objectives

By the end of this chapter, you should be able to calculate simple return and total return from cash and security values, turn returns over consecutive intervals into a wealth path, and explain why log returns add over time but cannot be added directly across assets using capital weights. You should also distinguish the arithmetic mean, geometric mean, and compound annual growth rate (CAGR).

The programming objectives are to retain initial capital, handle a -100% loss and missing values, calculate excess return using the risk-free return for the same month, check drawdown independently, and state the annualization conventions and conditions for volatility and the Sharpe ratio. Finally, read a historical chart and distinguish the descriptions supported by the data from trading conclusions that remain unsupported.

2. Why a quant needs this

A 10% price increase does not always mean a 10% account gain. Dividends, splits, financing, exchange rates, external deposits, and trades within the interval affect the numerator or denominator. A stock falling from 100 to 98 while paying a dividend of 2 does not imply a 2% loss to its holder. Futures notional exposure and margin are also different return denominators. Chapter 10 establishes instrument cash flows; this chapter places them in a common measurement framework.

Signal research needs a specified prediction target: next-month price return, total return, excess return, or log return? Portfolio modules usually use simple returns; statistical models sometimes use log returns; long-term account growth requires compounding. Mixing these objects can make a regression or optimization program optimize the wrong target even when it runs correctly.

We first establish a benchmark account with no external cash flows, one reporting currency, and valuation at each interval end. We then identify the real circumstances that require extending this benchmark. This makes the assumptions visible rather than hiding several of them in one piece of code.

3. Intuition: the percentage denominator changes

An initial 100 rises 50% to 150, then loses 50% of 150 and ends at 75. The two percentages sum to zero, yet the account loses 25%. The second change applies to a different capital base and cannot simply cancel the first.

Currency profit and loss can be added across intervals if both are reported against the same initial 100, but that uses a different denominator convention. A research report must identify the capital base of each return before discussing additivity.

Logarithms turn multiplication into addition, making multi-period growth easier to express. They do not remove risk or merely rename simple return. The change from 100 to 150 gives $\log(1.5)$; from 150 to 75 gives $\log(0.5)$. Their sum is $\log(0.75) < 0$, correctly preserving the 25% total loss.

4. Mathematical foundations: accounts, intervals, and units

Definition and conditions: account returns and the log domain

Let $V_{t-1} > 0$ denote account wealth at the interval start and $V_t \geq 0$ wealth at the interval end in the same reporting currency. First assume no external deposits or withdrawals and that all investment cash flows are assigned correctly to the account. Simple return is defined as

$$R_t = \frac{V_t - V_{t-1}}{V_{t-1}}, \qquad V_t = V_{t-1}(1 + R_t).$$

V has currency units; R_t is dimensionless. The decimal 0.01 means 1%, not 0.01%. One basis point (bp) is 0.0001. The subscript t labels the end of a measurement interval, not an arbitrary CSV row number.

For an unlevered account with positive initial wealth, a 100% loss means $R_t = -1$ and $V_t = 0$. Log return requires the stricter condition

$$g_t = \log \frac{V_t}{V_{t-1}} = \log(1 + R_t), \qquad R_t > -1.$$

Here log is the natural logarithm. Zero terminal wealth has no finite real logarithm; negative wealth requires a liability and leverage model rather than continued use of the positive-wealth formula. The implementation must explicitly reject inputs outside its model.

A monthly label such as 2010-01 normally identifies the January 2010 return interval. It does not establish that a price, signal, or order was known at any particular intraday time. The historical example contains monthly published returns and a current snapshot, with no signals or executions. A strategy backtest must separately specify opening valuation, effective holdings, closing valuation, cash-flow attribution, and data availability and ingestion times.

5. Formal definitions: distinguish the return objects

5.1 Price return and total return

Let P_{t-1} and P_t be the endpoint prices of one unit of a security, and D_t the cash distribution belonging to its holder during the interval. In a simplified example with constant quantity, distributions credited at the interval end, and within-interval reinvestment either negligible or recorded separately,

$$R_t^{\text{price}} = \frac{P_t - P_{t-1}}{P_{t-1}}, \qquad R_t^{\text{total}} = \frac{P_t + D_t - P_{t-1}}{P_{t-1}}.$$

This requires $P_{t-1} > 0$. Prices and distributions use the same currency and per-unit basis. A split that changes quantity must preserve the economic position. If a vendor total-return series already includes dividends, do not add D_t again. Reinvestment, taxes and fees, or multiple distributions within the interval require a complete wealth ledger; this endpoint formula is not a universal formula for corporate actions.

5.2 Arithmetic mean, geometric mean, and CAGR

For T equal-length intervals with simple returns $R_1, \ldots, R_T$, the sample arithmetic mean is

$$\bar{R} = \frac{1}{T} \sum_{t=1}^{T} R_t.$$

It describes the sample average of a single-interval return, rather than the compounded growth of a path. The geometric mean is

$$G = \left[\prod_{t=1}^{T} (1 + R_t) \right]^{1/T} - 1 = \exp\left(\frac{1}{T} \sum_{t=1}^{T} g_t \right) - 1.$$

The second expression requires every $R_t > -1$. If one interval has return -1 and no later external capital is injected, terminal wealth is zero and the product definition gives a geometric mean of -1. A finite-log implementation cannot handle this boundary.

For wealth changing from $V_0 > 0$ to $V_T \geq 0$ over $Y > 0$ years with no external cash flows, define

$$\text{CAGR} = \left(\frac{V_T}{V_0} \right)^{1/Y} - 1.$$

Y is an explicit time convention. A regular monthly sample can be reported using a "$T/12$ years under the monthly-interval convention." Exact calendar time requires dates and a year-length convention. Dividing the row count of irregular data by 252 is not a time definition.

5.3 Excess return, volatility, and the Sharpe ratio

For the same interval and currency, let R_t be the asset simple total return and $R_{f,t}$ the risk-free simple return. Arithmetic excess return is

$$e_t = R_t - R_{f,t}.$$

It compares investing the same capital in the asset and the risk-free account. Growth relative to risk-free wealth is $(1 + R_t)/(1 + R_{f,t}) - 1$, which generally differs from e_t. Do not substitute relative wealth growth into the ordinary Sharpe numerator.

The sample standard deviation uses $T \geq 2$:

$$s(R) = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(R_t - \bar{R})^2}.$$

The single-interval sample Sharpe ratio is $\widehat{SR} = \bar{e}/s(e)$. The denominator is the standard deviation of **excess returns**, rather than an unspecified asset-return standard deviation. When $s(e) = 0$, the sample Sharpe ratio is undefined; do not return infinity and call it an ideal strategy. Chapter 32 addresses sampling uncertainty and selection bias.

6. Derivations: from wealth recursion to aggregate risk

6.1 Compounding and log additivity

Substitute $V_t = V_{t-1}(1+R_t)$ repeatedly. For two intervals, $V_2 = V_0(1+R_1)(1+R_2)$; induction gives

$$\frac{V_T}{V_0} = \prod_{t=1}^{T}(1 + R_t), \qquad R_{0:T} = \prod_{t=1}^{T}(1 + R_t) - 1.$$

Total return $R_{0:T}$ uses initial wealth as its denominator. Taking logs of positive wealth and applying the product rule gives

$$\log(V_T/V_0) = \sum_{t=1}^{T}\log(1 + R_t) = \sum_{t=1}^{T} g_t.$$

This is an identity. It does not require independent and identically distributed (IID), normal, or stationary returns. It requires one consistent account path, consistent cash-flow attribution, and positive wealth.

6.2 Why the geometric mean does not exceed the arithmetic mean

For positive $1 + R_t$, the natural logarithm is strictly concave. Jensen's inequality gives

$$\frac{1}{T}\sum_t \log(1 + R_t) \le \log\left(1 + \frac{1}{T}\sum_t R_t\right).$$

The exponential function is increasing, so $G \le \bar{R}$. Equality holds when all interval returns are equal. This is a sample inequality for the same realized values; it does not establish that every high-volatility asset must grow more slowly than any low-volatility asset, because means and distributions may also differ.

For small $|R|$, the Taylor expansion from Chapter 4 gives $\log(1+R) \approx R - R^2/2$. The sample mean log return is therefore approximately

$$\bar{g} \approx \bar{R} - \frac{1}{2}\overline{R^2}.$$

The squared term contains both sample variance and the squared mean. "Volatility drag" interprets a local approximation; it does not mean subtracting exactly half the variance for every distribution. Large losses, heavy tails, and returns near -100% can invalidate a low-order approximation.

6.3 Cross-sectional portfolios start with simple returns

For two assets with beginning-of-interval capital weights w_1, w_2 summing to one, no rebalancing within the interval, and no additional cash flows or financing, terminal wealth is

$$V_t = V_{t-1}\sum_i w_i(1 + R_{i,t}), \qquad R_{p,t} = \sum_i w_i R_{i,t}.$$

w_i are beginning-of-interval capital weights, not terminal weights after return-induced drift. Portfolio log return is

$$g_{p,t} = \log\left(1 + \sum_i w_i R_{i,t}\right).$$

It generally differs from $\sum_i w_i \log(1 + R_{i,t})$. If two equally weighted assets gain 50% and lose 50% in the same interval, portfolio simple and log returns are both zero, but the weighted asset log returns equal $\frac{1}{2}\log(0.75) \approx -0.143841$. Aggregation across assets and aggregation across time cannot be interchanged.

6.4 What square-root annualization assumes

Conditions: square-root scaling at a fixed horizon and at every horizon

For weakly stationary increments, every interval has variance γ_0. If the weighted covariance sum is zero at a specified q, the variance of the q-interval sum is exactly $q\gamma_0$ and its standard deviation is exactly $\sqrt{q}$ times the single-interval standard deviation. At this fixed q, the necessary and sufficient condition is

$$\sum_{k=1}^{q-1}(q-k)\gamma_k = 0.$$

Zero covariance at every nonzero lag is sufficient, but is not necessary at a fixed horizon: positive and negative terms can cancel. Requiring square-root scaling at every horizon q does require every nonzero-lag covariance to be zero under weak stationarity. Independence is sufficient for the latter property but is stronger than zero covariance.

More generally, let weakly stationary increments X_t have mean μ, finite variance γ_0, and lag covariance γ_k. Expanding the double sum gives

$$\begin{aligned}\operatorname{Var}\left(\sum_{j=1}^{q} X_j\right) &= \sum_{i=1}^{q}\sum_{j=1}^{q}\operatorname{Cov}(X_i, X_j) \\ &= q\gamma_0 + 2\sum_{k=1}^{q-1}(q-k)\gamma_k.\end{aligned}$$

The diagonal contributes q variance terms. Observations k intervals apart form $q-k$ pairs, and the reverse direction contributes the same covariance, producing the factor two. Positive correlation usually increases aggregate variance and negative correlation can reduce it; the full lag structure matters, not just one correlation coefficient.

This formula describes the **sum of increments** exactly. Log returns add over time, whereas compounded simple return is a product minus one; their standard deviations are not the same exact object. "Monthly return standard deviation times $\sqrt{12}$" may be reported as a conventional annualized measure, with the model and approximation limits stated.

In the same additive model, the q-interval mean is $q\mu$, while its risk denominator follows the formula above. The mean-to-standard-deviation ratio reduces to $\sqrt{q}\,\mu/\sqrt{\gamma_0}$ only when the weighted covariance sum at that horizon is zero. Nonstationary, serially correlated, compounded returns and risk-free returns at different frequencies do not yield an exact annual Sharpe ratio by changing a frequency constant.

7. Financial interpretation: growth, risk, and opportunity cost

The arithmetic mean answers "What is the average single-interval return?" The geometric mean describes growth along an equal-interval path; CAGR compresses initial wealth, terminal wealth, and elapsed time. CAGR does not retain ordering: rearranging the same returns preserves terminal wealth and the geometric mean but can change drawdown and liquidity pressure. An investor's ability to reach the endpoint depends on the path.

The risk-free account is an opportunity-cost benchmark. Compare a monthly asset return with risk-free return for the same month; do not subtract a full-year 4% from every monthly observation. Given a 4% effective annual rate and the additional assumption of constant monthly compounding, the effective monthly rate is $1.04^{1/12}-1$, approximately 0.327374%. Money-market simple-rate quotations, day-count conventions, or actual monthly Treasury-bill returns require their own terms rather than the same exponent formula.

A domestic-currency investment in a foreign asset also involves the exchange rate. Let S_t be the domestic-currency price of one unit of foreign currency and R_t^F the foreign-currency asset total return. Ignoring taxes, fees, and hedging,

$$1 + R_t^H = (1 + R_t^F)\frac{S_t}{S_{t-1}}.$$

H denotes domestic currency and F foreign currency; quotation direction determines the exchange-rate factor. A foreign asset gaining 10% while its currency loses 10% against domestic currency gives domestic return $1.1 \times 0.9 - 1 = -1\%$, not zero. Hedging costs and forward cash flows require a separate ledger.

8. Numerical Example

8.1 Symmetric percentages do not produce symmetric wealth

Construct two years: +50% in year one and −50% in year two. Initial wealth 100 ends at 75. The annual arithmetic mean is 0%, two-year total return is −25%, and the annual geometric mean and CAGR are both −13.3974596%. Year-two drawdown from the historical peak of 150 is −50%; total loss and peak drawdown use different denominators.

8.2 Dividends and external deposits

Construct a price change from 100 to 98 with a dividend of 2 paid at the interval end, no reinvestment, and no taxes or fees. Price return is −2% and total return is 0%. Adding 2 again to a series that already includes dividends would count the cash flow twice.

Another account starts at 100, earns 10% to reach 110, then receives an external deposit of 90 and ends at 200. The 100% increase from 100 to 200 is not investment return. Removing the end-of-interval deposit leaves investment return of 10%. A deposit within the interval cannot be assumed to occur at its end. The challenge exercise below explains segmented time-weighted and money-weighted returns.

8.3 A serial-correlation counterexample for additive increments

Construct weakly stationary increments with monthly standard deviation 0.02 and lag covariance $\gamma_k = 0.0004(0.5)^k$. These are theoretical teaching inputs, not actual security returns. The variance multiplier for a 12-interval sum is

$$12 + 2\sum_{k=1}^{11}(12-k)(0.5)^k = 32.0009765625.$$

Its standard deviation is 0.1131388113, compared with 0.0692820323 from square-root-of-12 scaling that ignores correlation: approximately 63.30% larger. This example checks risk aggregation for a sum of increments; it is not a compounded account or an empirical Sharpe estimate.

8.4 Covariance cancellation at a fixed horizon

Complete counterexample: covariance cancellation over three intervals

Consider three increments, each with variance one, with covariance matrix

$$\Sigma = \begin{pmatrix} 1 & 0.1 & -0.2 \\ 0.1 & 1 & 0.1 \\ -0.2 & 0.1 & 1 \end{pmatrix}.$$

The leading principal minors are 1, 0.99, and 0.936, all positive, so this is a valid positive-definite covariance matrix. At fixed $q = 3$, $\gamma_1 = 0.1$ and $\gamma_2 = -0.2$ give weighted sum $2(0.1) + 1(-0.2) = 0$. Hence

$$\mathrm{Var}(X_1 + X_2 + X_3) = 3 + 2[2(0.1) - 0.2] = 3.$$

The three-interval sum therefore has exactly $\sqrt{3}$ times the single-interval standard deviation, although cross-interval covariances are not all zero. At horizon two, variance is $2 + 2(0.1) = 2.2$ and square-root scaling fails. Cancellation answers only the variance question at the specified horizon; it establishes neither independence nor scaling at other horizons.

8.5 Returns and path risk from an endpoint ledger

Complete example: endpoints, intervals, and peak drawdown

The table is a constructed account path for teaching, not market history. Each row first records a valuation endpoint; neighboring endpoints define a holding-period return. Wealth is in currency units; returns and drawdowns are decimals.

Endpoint	Wealth	Interval simple return	Historical peak	Drawdown
t_0	100	—	100	0
t_1	150	0.50	150	0
t_2	75	−0.50	150	−0.50
t_3	120	0.60	150	−0.20

Four endpoints define only three return intervals. The final wealth ratio is $120/100 = 1.2$ and cumulative simple return is 20%. The arithmetic mean is 20%, the geometric mean is $1.2^{1/3} - 1 \approx 6.2659\%$, and maximum drawdown is −50%. Drawdown uses the historical peak of 150 as denominator; do not confuse it with cumulative loss from initial wealth 100. Initial wealth belongs in the wealth and drawdown path, but has no preceding endpoint. Do not fabricate a 0% return for it or include it in return means, volatility, Sharpe ratios, or elapsed time.

A later genuine holding interval with wealth 100 at both endpoints has a 0% return and must remain in the sample. If a displayed ledger fills the initial row's `ret` with zero, mark it as a structural baseline row and exclude it from statistics. That display sentinel differs from a genuine interval with zero return.

9. Python Implementation

Default execution entry point for this reader edition

From the accepted I3 reader package root, create a separate environment using its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch11_returns.ipynb
```

The Notebook calls the implementation shared within the package. Chapter 11 executes a constructed wealth path by default. French historical loading, plotting, and cache checks remain in skipped explanatory cells rather than default execution. The runner writes fresh-kernel execution records to a unique runs/execution_* directory; new Notebook figures go to a separate UUID directory. The terminal prints the actual locations. French data absent from the package are neither loaded by default nor downloaded. The old course-production tests and historical CLI below refer to the course working root; readers use assertions in the Notebook above. Optional historical data also follow the data contract. This batch did not rerun them.

Historical course working root and original implementation

Core functions are in performance.py; frozen monthly input loading is in `market_data.py`; the standalone entry point is `ch11_returns.py`. Cached historical returns are already decimals and are not divided by 100 again. This CLI requires the course root's frozen French files and manifests, rather than serving as the offline default above. The original route is:

```
.venv/bin/python code/ch11_returns.py
```

The following constructed example can be run independently from the course root:

```
import sys
sys.path.insert(0, "src")
import numpy as np
from quant_course.performance import (
    wealth_index, geometric_mean_return, cagr, drawdown_path
)

returns = np.array([0.5, -0.5])
wealth = wealth_index(returns, initial_capital=100.0)
print(wealth)
print(returns.mean(), geometric_mean_return(returns))
print(cagr(100.0, wealth[-1], elapsed_years=2.0))
print(drawdown_path(wealth))
np.testing.assert_allclose(wealth, [100.0, 150.0, 75.0])
np.testing.assert_allclose(drawdown_path(wealth), [0.0, 0.0, -0.5])

# Bridge from chapter2: exclude only the structural initial position.
import pandas as pd
from quant_course import compute_returns
prices = pd.Series([100.0, 110.0, 99.0],
    index=pd.date_range("2026-01-02", periods=3, freq="B", tz="UTC"))
price_returns = compute_returns(prices, kind="simple")
assert pd.isna(price_returns.iloc[0])
interval_returns = price_returns.iloc[1:]
if interval_returns.isna().any():
    raise ValueError("unknown holding interval; do not drop gaps")
price_path = wealth_index(interval_returns.to_numpy(), prices.iloc[0])
np.testing.assert_allclose(price_path, prices.to_numpy())
assert len(price_path) == len(prices) == len(interval_returns) + 1
```

This bridge applies only to a teaching price path with complete, consistently defined prices and no omitted cash flows. compute_returns retains the initial NaN; wealth_index accepts only simple returns over actual intervals. Exclude only the first entry by position and explicitly check for remaining missing values. Do not use dropna() to delete unknown intervening intervals. Choosing the opening price as initial capital merely reproduces the example's price values; it does not turn price return into total return. Chapter 2 uses r for log return and this chapter uses g for the same natural-log transformation. Neither belongs in wealth_index, which requires simple returns.

Distinguish endpoints from intervals when counting: wealth 100, 150, 75, 120 gives four valuation endpoints but three actual returns, 50%, −50%, and 60%. Initial wealth 100 has no

preceding value and is not a market zero return. Retain it in wealth and drawdown paths, but exclude it from return means, volatility, Sharpe samples, and CAGR elapsed time. A displayed initial ret=0 row must be explicitly excluded as a baseline; genuine 0% holding-period returns remain. Missing intervals are unknown observations, which cannot be removed by the initial-row rule.

The wealth output has one more entry than the number of return observations. Its first entry is declared initial capital, not a fabricated market price. A -100% return may reduce wealth to zero; subsequent multiplication cannot restore positive wealth without capital injection. After wealth reaches zero, that account's own subsequent returns have a zero denominator and are undefined. Multiplying hypothetical growth factors afterward only illustrates that zero wealth stays zero, not actual account returns. The added drawdown_path permits zero wealth; the time-indexed compute_drawdown in timeseries.py retains its positive-wealth contract (not developed in Chapter 2). Do not silently interchange them.

All measures reject NaN, infinity, and invalid numeric types rather than applying dropna() automatically. A missing return may represent an unknown wealth change; deleting it joins nonconsecutive intervals. Only a declared, justified research subsample can change the path. "Enough observations remain" is not a substitute for that decision.

RF requires an explicit vector of matching length and intervals; an annual rate is not broadcast automatically. Zero excess-return volatility makes the Sharpe ratio undefined. Extreme values can overflow float64 variance; the function rejects the result rather than misreporting an infinite standard deviation as Sharpe zero. The functions do not identify trading calendars or publication times. The monthly loader verifies hashes, indices, and continuity; the researcher specifies the information clock and execution contract.

10. Visualization

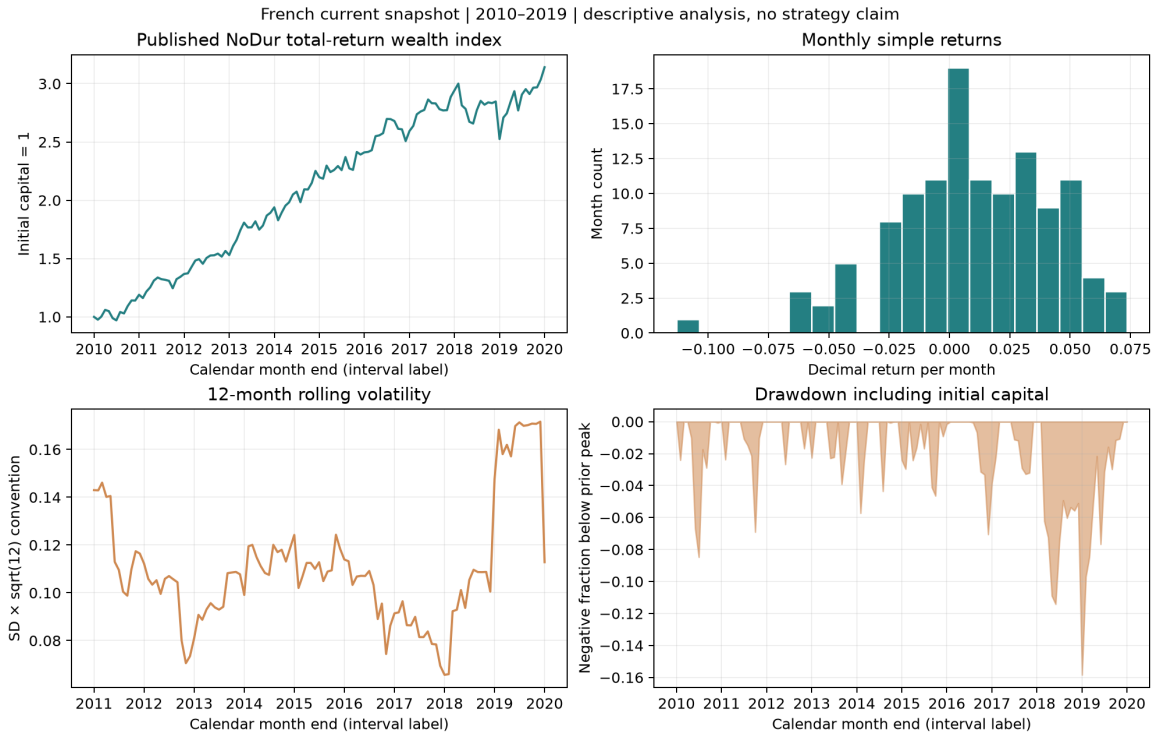


Figure 1: Monthly descriptive statistics for the NoDur industry portfolio

The four panels use the same monthly sample from 2010-01 through 2019-12 and the same calculation code.

- Upper left: the horizontal axis is calendar month-end labels; the vertical axis is a total-return wealth index starting at one. The first point at 2009-12 is chosen initial capital, not an actual 2009 stock price. Examine growth and intervening stagnation rather than judging risk from terminal wealth alone.
- Upper right: the horizontal axis is decimal monthly simple return; the vertical axis counts months in each bin. This is a 120-month frequency histogram, not proof of a population density. Extreme losses and asymmetry require more data and statistical tests.
- Lower left: the horizontal axis is month-end; the vertical axis is the preceding 12 months' sample standard deviation times the square root of 12, a conventional annualized measure. The curve starts only after 12 observations. It uses no future window and does not establish next year's volatility.
- Lower right: the horizontal axis is month-end; the vertical axis is negative drawdown from the highest wealth observed by that month. Zero indicates a new or matched high. The deeper decline in 2018 shows why terminal growth does not eliminate pressure along the path.

The wealth index compounds published portfolio returns. It is not a fund NAV, a stock-level price series, a strategy net of fees, or a product immediately available for purchase.

11. Real Market Example

The sources are Kenneth R. French Data Library's 10 Industry Portfolios and Fama/French 3 Factors. We select published monthly value-weighted industry portfolio returns in the NoDur column. RF is the same-month FF3 Treasury-bill return proxy; it is not assumed to equal a frictionless cash rate available to every investor. NoDur identifies the nondurable-consumer-goods industry portfolio, not a security ticker.

The processed caches' month labels and hashes are verified against their respective manifests. A one-to-one monthly join over 2010-01 through 2019-12 gives 120 months, with no imputation or frequency mixing. Initial capital is one. The script calculated the following values; return units are decimals:

Measure	Calculated value	Interpretation
Terminal wealth index	3.141190295	Compounded total-return wealth over 120 months, before executable-strategy costs
Monthly arithmetic mean	0.010099167	Approximately 1.009917% per month
Monthly mean log return	0.009538348	Not a monthly simple return
Monthly geometric mean	0.009583983	Approximately 0.958398% per month
CAGR (10 years under the monthly-interval convention)	0.121267992	Approximately 12.126799% per year
Monthly sample standard deviation	0.032172825	Approximately 3.217283% per month
Standard deviation times square root of 12	0.111449935	Conventional annualized measure of approximately 11.144994%
Mean monthly RF	0.000426667	Aligned actual monthly values, not a fixed annual rate
Mean monthly excess return	0.009672500	Approximately 0.967250% per month
Sharpe ratio times square root of 12	1.039249824	No serial-correlation or selection-bias adjustment
Maximum drawdown as positive loss magnitude	0.158392214	Approximately 15.839221% between month-end wealth peak and trough

These values describe this specified interval in the current published version. The 2010–2019 window is for teaching, not an independently validated preferred investment period. It defines no exit rule, signal, order, or cost model. Portfolio constituent treatment, return methodology, and historical revisions belong to the publisher; they do not establish a stock-level historical universe built by this project.

The manifests record download times and hashes from 2026-10-05. Historical data downloaded in 2026 do not establish that researchers in 2010 saw the same version. Complete point-in-time publication and ingestion histories have not been built. This is retrospective descriptive analysis and makes no claim that look-ahead bias has been eliminated at the trading level.

12. Common Mistakes

Adding returns and calling the sum total return. The incorrect calculation $+50\%-50\%=0$ ignores the changing denominator. Correct total return is $(1.5)(0.5)-1=-25\%$. Adding currency profit and loss requires a consistent capital basis and stated account cash flows.

Adding dividends again to total return. Adjusted prices do not necessarily use the same total-return convention. Read the vendor definition and Chapter 3's corporate-action contract first. Distributions already included cannot be credited twice. Split adjustment preserves the relation between quantity and price; it creates no economic return.

Common pitfall: do not apply drifted end weights retrospectively

Multiplying this interval's returns by end weights. Returns have already changed the weights; terminal weights do not describe this interval's capital allocation. Invest 50 in each of two assets gaining 50% and losing 50%. Terminal values are 75 and 25, with weights 75% and 25%. End-weight calculation gives 25%, although the account's actual return is zero.

```
import numpy as np
weights_at_start = np.array([0.5, 0.5])
asset_returns = np.array([0.5, -0.5])
values_at_end = weights_at_start * (1 + asset_returns)
weights_at_end = values_at_end / values_at_end.sum()
correct = weights_at_start @ asset_returns
wrong = weights_at_end @ asset_returns
print("correct simple return", correct, "wrong end-weight return", wrong)
assert np.isclose(correct, 0.0)
assert np.isclose(wrong, 0.25)
```

Deleting blank months before compounding. Missing is neither zero nor a market pause. If the account's wealth change is unknown, terminal wealth and maximum drawdown are not fully determined. Current core functions raise an error rather than silently joining nonadjacent intervals.

Calling annualized output a forecast of next year's risk. Square-root-of-time scaling depends on risk aggregation conditions; a rolling sample describes its historical window. Forecasting next year requires a model, estimation uncertainty, and out-of-sample evaluation. Chapter 18 develops dynamic volatility models.

Treating net deposits as investment profit. CAGR applies only to the declared path without external cash flows. With inflows and outflows, segmented time-weighted and money-weighted returns answer different questions. Recording cash-flow times matters more than choosing an attractive percentage.

13. Quant Research Application

First define the target object, frequency, and interval. For next-month total return, distinguish feature time from label end time: a training label is complete only once its interval has ended and it is available. When converting a log-return prediction to simple return, consider the distribution and estimand. $\exp(E[g]) - 1$ generally differs from $E[\exp(g) - 1]$; exponentiating a predicted mean does not automatically give expected simple return.

Next determine the account denominator. A portfolio backtest applies effective beginning-of-interval capital weights to subsequent asset total returns, then subtracts fees and financing to form net wealth. Self-financing constraints, cash holdings, stock borrowing, and derivative notional exposures require a ledger. Convert annual RF and foreign-currency benchmarks to matching intervals and definitions before subtraction; matching column names is insufficient.

Report mean, growth, volatility, drawdown, sample scope, and costs together. Store calculation values as decimals and convert to percentages for display. Retain raw and processed versions, hashes, missing-data rules, and measurement intervals. Project 1 uses these steps for a descriptive financial-data report; Chapter 29 develops execution and holdings ledgers.

14. Chapter Summary

Simple return connects neighboring wealth values. Log return turns temporal multiplication into addition on a positive-wealth path; cross-asset portfolios still start with simple returns and beginning-of-interval capital weights. The arithmetic mean describes single-interval sample returns; the geometric mean and CAGR summarize compound growth; drawdown preserves peak-to-trough path information.

Annualization involves more than changing a display unit. The variance of summed increments depends on all lag covariances. Square-root time is either scaling under specific conditions or an explicitly stated convention. RF, distributions, external deposits, currency, and the measurement clock determine what an indicator means. Define them before interpreting figures and values.

15. Exercises

11-E01 (Concepts) Explain the differences between simple return, log return, price return, and total return. State the account or cash-flow conditions for each.

11-E02 (Hand calculation) Start at 100, gain 50%, then lose 50%, with each interval lasting one year. Calculate terminal wealth, total return, arithmetic mean, geometric mean, CAGR, and maximum drawdown.

11-E03 (Portfolio) Two equally weighted assets gain 50% and lose 50% in one interval. Calculate portfolio simple and log returns, weighted asset log returns, and terminal capital weights. Explain the difference between time and cross-sectional aggregation.

11-E04 (RF) Monthly asset return is 2% and monthly RF is 0.3%. Calculate arithmetic excess return and growth relative to risk-free wealth. What monthly effective rate corresponds to an effective annual rate of 4% under constant monthly compounding?

11-E05 (Currency) A foreign-currency asset gains 10%, while the domestic-currency price of one foreign-currency unit falls 10%. Calculate domestic total return and explain why quotation direction matters.

11-E06 (Annualization) Derive the variance of a q-interval increment sum from the covariance double sum. What conditions does square-root-of-time scaling require? Why is it not the exact risk formula for compounded simple return?

11-E07 (Path) Compare three-interval returns $(50\%, -40\%, -20\%)$ and $(-40\%, 50\%, -20\%)$, starting at 100. Are terminal wealth, CAGR, and maximum drawdown the same?

16. Programming Exercises

11-P01 (Wealth boundaries) Use the core functions to check $+50\%/-50\%$, a one-interval -100% loss, and NaN input. What is the initial point? Why cannot a positive percentage restore wealth after bankruptcy?

11-P02 (Risk aggregation) For $\gamma_k = 0.0004(0.5)^k$ in Section 8.3, calculate the standard deviation of a 12-interval sum and the scaling that ignores serial correlation. Do not call the result an actual strategy's annual Sharpe ratio.

11-P03 (Historical cache) Run the frozen-cache analysis; verify the same-month NoDur/RF join, 120 consecutive months, terminal wealth, and indicators. Produce at least one chart and write three supported interpretations and two limitations.

17. Challenge Problems

11-C01 (Approximation and inequality) Prove that the equal-interval geometric mean does not exceed the arithmetic mean. Explain equality, the conditions for $\bar{g} \approx \bar{R} - \overline{R^2}/2$, and why "geometric mean equals mean minus half the variance" is not exact. Ten points: concavity and Jensen (4), exponential monotonicity and equality (2), Taylor range and second-moment term (2), extreme-return boundaries (2).

11-C02 (External cash flows) Split a year into two halves. Initial 100 earns 10% to 110; a deposit of 90 brings capital to 200; the second half earns 10% to 220. Calculate time-weighted return (TWR) and compare it with the unadjusted terminal-to-initial ratio. Write the cash-flow equation for money-weighted return and explain the difference. Ten points: segmented capital (2), TWR product (3), incorrect ratio (1), internal rate of return (IRR) equation and timing (2), the questions answered by the two returns (2).

11-N01 (Variance scaling at a fixed horizon) For weakly stationary, equal-variance increments, derive the necessary and sufficient condition for exact square-root scaling at a specified horizon q from the covariance double sum. Explain what the condition implies if it holds at every horizon. Use the Section 8.4 matrix to verify three-interval cancellation and matrix validity, then compare the two-interval result. State why this establishes neither independence nor a universal variance formula for compounded simple returns.

18. Solutions / Solution Hints

11-E01 Answer. Simple return divides the change between neighboring wealth values by beginning wealth; the benchmark account has no external inflows and positive initial wealth. Log return is the natural log of a positive wealth ratio and adds over time, requiring positive terminal wealth as well. Price return tracks only prices; total return includes attributed distributions and other investment cash flows. Splits must preserve the economic position, and within-interval reinvestment requires a ledger. Do not add dividends to a series that already includes distributions. Log return is not synonymous with total return: one may take logs of either total-return wealth or a price path, but their cash-flow coverage remains different.

11-E02 Answer. Wealth is 100, 150, 75. Total return is -25%, arithmetic mean is 0%, and the geometric mean and CAGR are $\sqrt{0.75} - 1 = -13.3974596\%$. Drawdown from peak 150 to terminal 75 is -50%; maximum drawdown may be reported as a positive loss magnitude of 50%. It differs from the 25% loss against initial capital.

11-E03 Answer. Initial weights are 0.5 each. Portfolio simple return is $0.5\times0.5+0.5\times(-0.5)=0$, so portfolio log return is $\log 1 = 0$. Weighted asset log return is $\frac{1}{2}[\log 1.5 + \log 0.5] = \frac{1}{2}\log 0.75 \approx -0.143841036$, not zero. Terminal capital is 75 and 25, with weights 0.75 and 0.25. Those weights describe the next interval's starting allocation, not this interval's return.

11-E04 Answer. Arithmetic excess return is $0.02-0.003 = 0.017$, or 1.7%. Growth relative to risk-free wealth is $1.02/1.003 - 1 \approx 0.0169491525$, approximately 1.694915%. The monthly effective rate is $1.04^{1/12} - 1 \approx 0.0032737398$, or 0.327374%. This conversion assumes constant compounding and cannot be inferred from an arbitrary annualized quotation.

11-E05 Answer. The domestic growth factor is $1.1\times0.9=0.99$, giving total return -1%. A lower domestic-currency price of one foreign-currency unit means foreign-currency depreciation. If the quotation instead gives foreign currency per domestic unit, invert the conversion factor rather than continuing to multiply by 0.9. The example ignores hedging, taxes, fees, and trading costs.

11-E06 Answer. The diagonal has q terms of γ_0 each. Positive lag k has $q-k$ pairs, with equal symmetric contributions at negative lags, giving $q\gamma_0 + 2\sum_{k=1}^{q-1}(q-k)\gamma_k$. With equal interval variances, exact square-root scaling at specified q holds if and only if $\sum_{k=1}^{q-1}(q-k)\gamma_k = 0$. Zero cross-interval covariances suffice but are not necessary at a fixed horizon because positive and negative terms can cancel. If scaling must hold at every horizon, weak stationarity requires every nonzero-lag covariance to vanish. Independence suffices but is not necessary. Compounded simple return is $\prod(1+R_t) - 1$, not an increment sum, so this is not its general exact variance formula. Log returns can be added, but then the risk object has changed.

11-E07 Answer. The first path is 100→150→90→72, with maximum drawdown $1 - 72/150 = 52\%$. The second is 100→60→90→72, with maximum drawdown $1 - 60/100 = 40\%$. Both end at 72, giving total return -28%. If each interval is one year, both CAGRs are $0.72^{1/3} - 1 \approx -10.37\%$. Ordering does not change the product, but it changes intervening peaks and troughs; equal endpoints do not imply equal holding experiences or risks.

11-P01 Reference implementation.

```
import sys
sys.path.insert(0, "src")
import numpy as np
from quant_course.performance import wealth_index, drawdown_path

first = wealth_index([0.5, -0.5], 100.0)
bankrupt = wealth_index([0.1, -1.0, 10.0], 100.0)
np.testing.assert_allclose(first, [100, 150, 75])
np.testing.assert_allclose(bankrupt, [100, 110, 0, 0])
np.testing.assert_allclose(drawdown_path(bankrupt), [0, 0, -1, -1])
try:
    wealth_index([0.1, np.nan], 100.0)
except ValueError:
    print("missing interval rejected")
else:
    raise AssertionError("unknown return cannot be silently removed")
```

The first point is explicitly chosen capital before the interval. Zero times any finite growth factor remains zero. External capital injection is a separate cash flow to record, not a larger next-interval return.

11-P02 Reference implementation.

```
import numpy as np
q = 12
variance_per_month = 0.0004
rho = 0.5
lags = np.arange(1, q)
variance_sum = variance_per_month * (
    q + 2 * np.sum((q - lags) * rho**lags)
)
actual_sd = np.sqrt(variance_sum)
zero_cov_sd = np.sqrt(q * variance_per_month)
print(actual_sd, zero_cov_sd)
assert np.isclose(actual_sd, 0.11313881131159192)
assert np.isclose(zero_cov_sd, 0.06928203230275509)
```

The declared theoretical increment sum has standard deviation 0.1131388113, versus 0.0692820323 with zero covariance. The difference comes from lag covariances. No security data, account implementation, or trading rules are involved.

11-P03 Reference implementation.

```
import runpy
import numpy as np
lesson = runpy.run_path("code/ch11_returns.py", run_name="lesson_functions")
sample = lesson["monthly_example"]()
result = lesson["summarize_monthly"](sample)
assert len(sample) == 120
assert sample.index[0].strftime("%Y%m") == "201001"
assert sample.index[-1].strftime("%Y%m") == "201912"
assert not sample.isna().any().any()
np.testing.assert_allclose(result["wealth_final"], 3.141190294871468)
lesson["make_figure"](sample, lesson["ROOT"] / "figures/a3/ch11_returns_dashboard.png
    ↪ ")
print(result)
```

This example reuses the standalone entry point's input/output and plotting wrappers; the numerical core calls performance and market_data in src. Interpretations may state that selected-window wealth rises from one to about 3.141, month-end drawdown deepens around 2018, and rolling volatility changes with the sample window. Limitations include month-end sampling missing intraday or between-day maximum drawdown, the current release not reconstructing contemporaneous data versions, no executable holdings or costs, and a teaching window not establishing superior investment selection. Grade open responses on correspondence to chart or values and specific conditions and evidence limits.

11-C01 Reference argument. For $R_t > -1$, log is strictly concave. Jensen gives $\overline{\log(1+R)} \le \log(1+\bar{R})$; exponential monotonicity implies $G \le \bar{R}$. Equality requires identical realized returns. The local second-order Taylor expansion is $\bar{g} \approx \bar{R} - \overline{R^2}/2$, while $\overline{R^2} = \overline{(R-\bar{R})^2} + \bar{R}^2$. One cannot subtract only sample variance or equate $\bar{g}$ directly with G: exponential conversion and higher-order error remain. Near -1, logs and higher-order terms change sharply; exactly -1 has no finite logarithm. The arithmetic–geometric mean inequality for non-negative growth factors can cover the zero-growth boundary, but the finite-log proof requires a separate limiting argument.

11-C02 Reference argument. The first-half growth factor is 1.1. The deposit changes capital to 200, and second-half growth is 220/200=1.1. TWR is 1.1×1.1−1=21%. The unadjusted 220/100−1=120% mixes the external 90 with investment profit.

From the investor's perspective, cash outflows are 100 at time zero and 90 at 0.5 years, with an inflow of 220 at one year. The IRR equation for annual effective money-weighted return r is

$$-100 - \frac{90}{(1+r)^{1/2}} + \frac{220}{1+r} = 0, \qquad r > -1.$$

Set $z = \sqrt{1+r} > 0$ and multiply by z^2 to obtain $100z^2 + 90z - 220 = 0$. The positive root is $z = (-90 + \sqrt{96100})/200 = 1.1$, so $r = 21\%$. Both halves earn 10%, making the effective annual return unaffected by deposit timing in this example, so the two measures coincide. They still answer different questions: TWR excludes the scale of external flows, whereas IRR reflects investor cash-flow amounts and timing. If second-half return changes to -10%, terminal wealth is

180 and TWR=−1%. Replacing terminal cash with 180 in the IRR equation gives $r \approx -6.8587\%$ from the positive root; the measures differ. Specifically, $z = (-90+\sqrt{80100})/200$, then $r = z^2 - 1$.

11-N01 Reference argument. Under weak stationarity with interval variance γ_0, the covariance double sum is $q\gamma_0 + 2\sum_{k=1}^{q-1}(q-k)\gamma_k$. At fixed q, it equals $q\gamma_0$ if and only if $\sum_{k=1}^{q-1}(q-k)\gamma_k = 0$. If the condition holds at every horizon, $q = 2$ first implies $\gamma_1 = 0$, and recursion over horizons implies zero covariance at every nonzero lag. The Section 8.4 leading principal minors are 1, 0.99, and 0.936, all positive, so the matrix is positive definite. At $q = 3$ the weighted sum is $2(0.1) - 0.2 = 0$ and three-interval variance is three; at $q = 2$ variance is $2 + 2(0.1) = 2.2$. The positive and negative covariances cancel only over these three intervals. This establishes neither independence nor square-root scaling at other horizons. The formula concerns a sum of increments, not a universal variance formula for compounded simple returns.

19. Further Reading

- **Must Know**: MIT OCW 15.401 compounding lecture, effective annual rates on slides 28–29 and real/nominal cash-flow consistency on slides 30–34.
- **Should Know**: French industry portfolio methodology and FF3 factor/RF descriptions. Use cache manifests to distinguish total returns, factor spreads, and RF.
- **Advanced**: Andrew W. Lo (2002), *The Statistics of Sharpe Ratios*, author publication record and abstract. Only metadata and the abstract were checked; the full text was not obtained. Consult the original equations when Chapter 32 develops Sharpe inference.
- **Optional**: Chapter 3 (corporate actions and dual clocks), Chapter 18 (dynamic volatility), Chapter 29 (ledgers), and Chapter 32 (statistical evaluation) progressively extend the no-external-cash-flow benchmark.

20. References

The actual external-source reading scope is recorded in `source_notes.md` and the source/metric preparation contract. The compounding source is MIT's official course. The risk aggregation formula is independently derived here by covariance expansion, not attributed equation by equation to an unread Lo full text.

Data versions, download times, units, and SHA-256 values are recorded in the industry manifest and FF3/FF5 manifest. Checkable values are in the actual metric output. Wealth and charts are original calculations, not copied source figures. Public download availability was not treated as permission to redistribute all data.

Project 1 —Returns, Wealth, and Risk Description

Project 1 Reader Guide: From a Return Table to Interpretable Wealth and Risk

Learn to read a monthly descriptive analysis: which interval each return covers, how wealth evolves, which denominator a risk measure uses and which material supports each conclusion. Review Chapter 11's simple and log returns, annualization and drawdown before attempting this project's six exercises.

1. Identify the data you are working with

This project offers two reading routes whose numerical results cannot be interchanged.

Material	Input	Purpose here
Current offline Notebook	Seed101 constructed panel, 180 months; NoDur, HiTec and Enrgy are constructed column labels, RF=0.001	Executable method demonstration and small hand calculations
Original historical report, figure and answers to Exercises 2/5	French publication snapshot of October 5, 2026; historical RF aligned by month	Read the saved historical description without rerunning it

The 2010–2024 month labels do not turn constructed inputs into market observations from that period. HiTec wealth of 4.6270, CAGR of 16.55%, conventionally annualized volatility of 15.61% and the negative 2015–2019 energy Sharpe belong to the historical report. The current constructed table cannot establish, update or overturn them.

2. Prepare the environment and run

The accepted I3 reader package declares Python 3.12 or later. Its checked lock file targets macOS `arm64/CPython` 3.14.7; other platforms have not been verified. If your reader environment is ready, proceed to the execution command below. Otherwise, run these setup commands from the package root:

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-macos-arm64-py314.lock
.venv/bin/python -m ipykernel install --prefix .venv --name python3 --display-name '
    ↪ Reader Python'
```

Installation may require network access; the default Notebook route downloads no data. From the same package root, run:

```
.venv/bin/python run_notebook.py projects/project01/notebooks/
    ↪ project01_financial_data_analysis.ipynb
```

The runner starts a fresh kernel and saves the executed Notebook and `execution.json` in a new `runs/execution_*` directory, printing its actual evidence location. The Notebook's first code cell separately creates `runs/project01_financial_data_analysis_<uuid>`, containing `constructed_monthly_returns.csv` and `project01_dashboard_CONSTRUCTED.png`. These two directories hold execution evidence and method outputs respectively. They are separate from the original French report's result locations. Each UUID is new; use the printed paths.

3. Input contract and main window

The current panel has a continuous monthly PeriodIndex. The month label identifies a return interval. Its three industry columns contain decimal simple returns: 0.01 means 1%; RF uses the same interval and decimal units. Seed101 generates normal constructed industry inputs, with RF fixed at 0.001. Do not divide the returns by 100 again. The original `load_project_data` function is not called to load French snapshots.

The Notebook selects January 2010 through December 2019, 120 months, from the 180-month panel for its main table. The code checks the complete index, count and missingness. These checks establish input consistency; they do not establish market authenticity, permissions, historical availability or tradability. The data dictionary separately explains the French sources, SIC formation rules, processed units and missing historical clocks.

4. Build wealth from simple returns, then interpret means

For one account without external cash flows,

$$W_t = W_{t-1}(1 + R_t), \qquad W_0 > 0.$$

Each percentage acts on the current capital, so growth factors must be multiplied. The Notebook's hand example starts at 100 with $+50\%$ and -50%, producing 100, 150, 75. Its arithmetic mean is 0%, cumulative return -25% and maximum drawdown from the peak of 150 is -50%. The initial point has no preceding return interval. Keep it in wealth and drawdown paths, and exclude it from return statistics.

For every period satisfying $R_t > -1$, $g_t = \log(1+R_t)$ and $\sum g_t = \log(W_T/W_0)$. The Notebook next compares the current constructed main window's simple-return and log-return means. Logs add across time; across assets, first aggregate simple returns using beginning-of-period weights. Column names ending in `pct_per_month` denote displayed percentages; numerical inputs remain decimals.

5. What each column of the metric table answers

Metric	Calculation and conditions	Interpretation
Final wealth, cumulative simple return	Initial wealth 1; multiply monthly simple-return growth factors	Terminal value and change from the initial value for this constructed path; not a price
CAGR	$(W_T/W_0)^{12/T} - 1$, no external cash flows, complete monthly span	Compound-growth summary using the stated `T/12-year` convention
Annualized volatility	Monthly sample standard deviation, ddof=1, multiplied by $\sqrt{12}$	Conventional annual scale, not an identity for annual compound-return risk
Annualized Sharpe	Same-month $e_t = R_t - RF_t$, $\sqrt{12}\bar{e}/s(e)$, $s(e) > 0$	`Mean/risk` ratio in the constructed sample, not an Alpha test
Maximum drawdown, peak month, trough month	$D_t = W_t / \max_{s \le t} W_s - 1$, loss magnitude $-\min D_t$	Pressure along the month-end peak-to-trough path; intraday peaks are unobserved

Each fixed descriptive segment contains 60 months and restarts drawdown calculations at wealth 1. The next Notebook table uses the same Seed101 constructed panel. Rankings, signs and values from the French historical report do not apply to it. The Markdown retains the accepted constructed energy-segment description: approximately +8.008% CAGR and +0.623 Sharpe in 2015–2019, versus −0.262% and −0.073 in 2020–2024. These are records for that constructed input, not newly rerun values in this revision.

6. Read the axes of the four panels

The figure uses the same constructed panel over 2010–2024. The shaded 2020–2024 region is a descriptive extension, not a holdout test.

- Upper left: month-end labels against wealth index, starting at 1 at the selected December 2009 endpoint; not a security price.
- Upper right: month-end labels against the trailing 12-month sample standard deviation multiplied by $\sqrt{12}$. The first 11 months are blank because the window is incomplete; this is not a future-volatility forecast.
- Lower left: month-end labels against negative drawdown from the running wealth peak, including initial capital. A value of −0.5 means 50% below that peak.
- Lower right: industry labels against decimal monthly simple returns in boxplots. Medians, quartiles and distant observations summarize the constructed input; unfavorable points are retained.

The original figure in the historical report uses these same four types of axes, but its data object is the French snapshot. Similar figure layouts establish neither identical inputs nor reproduction.

7. Approach the six exercises

Read the original exercises and attempt them before consulting the complete solutions. Exercise 1 uses an independent +10%/−10% hand calculation, distinct from the Notebook's +50%/−50% inputs. Exercise 3 supplies its own monthly RF; Exercise 4 compares initial-wealth and peak-wealth denominators. Read the historical report's main-window results for Exercise 2 and its fixed historical segments for Exercise 5. Do not replace historical answers with the constructed table. Exercise 6 interprets the original historical figure's axes; compare them with the current constructed figure while retaining the input distinction.

All six questions retain their original meanings, values and complete answers. The French CLI at the end of the old exercise document belongs to the historical working-course route. For current study, run only the offline Notebook command in this guide; do not activate that market route.

8. Supported conclusions and remaining responsibilities

The current run can demonstrate how shared metrics process constructed returns, windows and figures. The historical report can describe published portfolio returns for its specified snapshot and windows. Neither establishes future returns, net strategy Alpha or an executable advantage. The snapshot lacks stock-level OHLCV, historical monthly vintages, release and ingestion times, signals, orders, costs and financing ledgers.

G01 rights, subject and use remain unresolved; `G05/A10-B` is unauthorized. Technical protocol acceptance supplies no data permission. The optional input contract describes future user-supplied data requirements under separate authorization; it does not enable French execution here. See the complete English project entry. Both Notebook language projections change Markdown only: the numerical implementation, execution cells and existing outputs are preserved.

Project 1 — Financial Data Analysis

Prerequisites: Python in Chapter 2, data in Chapter 3, market instruments in Chapter 10 and return measurement in Chapter 11. Starting with monthly simple returns, identify the data and units, compound wealth, align monthly RF, calculate volatility, Sharpe and drawdown, then interpret fixed-window tables and figures.

Two distinct data objects

The current reader Notebook uses a Seed101 constructed panel of 180 months, labelled January 2010 through December 2024, with columns NoDur, HiTec, Enrgy and RF. Industry codes are method-example labels here. There are no French market observations, and the fixed RF of 0.001 is not an observed cash rate.

The original historical report uses the French publication snapshot frozen on October 5, 2026. Its main 2010–2019 window contains 120 months, with three additional 60-month descriptive

segments. The report's wealth values, industry rankings, negative results and figures retain their historical identities. The constructed run neither reproduces nor verifies those market values. Exercises 2 and 5 ask you to read this historical report.

Current execution entry

Prepare an independent reader-package environment using the reader guide, then run from the package root:

```
.venv/bin/python run_notebook.py projects/project01/notebooks/
    ↪ project01_financial_data_analysis.ipynb
```

The runner prints the location of the fresh-kernel execution record; the Notebook separately prints a unique UUID directory for method outputs. New constructed CSVs and figures are written only under the package's runs directory, preserving the original report, data and images. The French branch is skipped by default; no data are downloaded.

Read in this order

- Reader guide: inputs, actions, outputs, formulas, axes and the source material for all six exercises.
- Notebook language projection: the accepted I3 numerical cells, with revised explanatory Markdown only.
- Data dictionary and original data manifest: constructed-column contracts, historical snapshot hashes, portfolio formation dates and the periods supplying classification codes.
- Historical report, six exercises and complete solutions: report-reading questions use the original snapshot; constructed results do not replace their answers.
- Historical configuration and historical QA: fixed windows, annualization conventions and the scope of the original checks.
- Project entry: the complete English guide, dictionary, historical report and exercises.

Historical course-work route

The original command from the working course root is `.venv/bin/python projects/project01/run_project01.py`. It depends on manifest-bound French ZIP archives, extracted CSVs and processed CSVs; the original production tests are at `tests/test_project01_analysis.py`. These are historical reproduction locations, separate from the current offline Notebook entry. Consult the report and QA for the original output identities. This revision neither runs nor regenerates them.

G01 data rights, subject and use remain unresolved, and `G05/A10-B` is unauthorized. Technical acceptance of the A10-A.1 protocol supplies neither data permission nor execution authorization. The user-supplied-data contract describes future input requirements; it does not enable a market branch in this reader run.

Data Dictionary

The French source and hash-chain description in this dictionary belongs to the original October 5, 2026 snapshot. The current offline Notebook uses a Seed101 constructed panel of 180 months: column names and decimal units match, but provenance differs. Constructed RF is fixed at 0.001, separate from observed Treasury-bill returns. See the reader guide for execution.

The historical project reads frozen processed snapshots of the French 10-industry monthly table and French three-factor monthly table. During historical execution, the project manifest is checked against the course source manifests, and shared `load_monthly_snapshot` verifies processed-file SHA-256, fields, finite values, month order, uniqueness and continuity. Hash checks identify the consumed processed bytes; original ZIP and extracted-CSV hashes trace the conversion chain.

Fields

Field	Meaning	Source unit	Project input unit	Key / time	Missing values
`month`	`YYYYMM` monthly return-interval label	—	pandas monthly `PeriodIndex`	Unique calendar month; not an intramonth timestamp	Missing, duplicate, unordered or skipped months rejected
`NoDur`	Published monthly value-weighted total return of the Nondurables industry portfolio, including dividends	Percent / month	Decimal simple return / month	`month`	Original missing sentinels become empty processed fields; selected windows must be finite and continuous
`HiTec`	Published monthly value-weighted total return of the High Technology industry portfolio, including dividends	Percent / month	Decimal simple return / month	`month`	Same rule
`Enrgy`	Published monthly value-weighted total return of the Energy industry portfolio, including dividends	Percent / month	Decimal simple return / month	`month`	Same rule

Field	Meaning	Source unit	Project input unit	Key / time	Missing values
RF	Monthly one-month US Treasury-bill return proxy in the Fama/French three-factor file; not a frictionless cash rate available to every investor	Percent / month	Decimal simple return / month	month; exact one-to-one same-month alignment with industry returns	No forward filling; selected missing or infinite values rejected

Decimal 0.01 means a monthly return of 1%. Processed files already use decimals; do not divide by 100 again. Industry fields come from "Average Value Weighted Returns — Monthly." This project uses standard 10 Industry Portfolios; the library separately lists [ex. Dividends], so these are portfolio total returns including dividends. They are separate from prices, OHLCV or stock-level position-return ledgers. French forms industry portfolios at the end of June in year t. Classification preferentially uses Compustat Standard Industrial Classification (SIC) codes from fiscal years ending in calendar year t−1; when unavailable, it uses CRSP SIC codes from June of year t. Portfolio returns then cover July of t through June of t+1. Formation dates and classification-input periods are distinct. See the 10-industry details and Data Library for source methods and update information.

Time, Publication and Scope

month identifies a monthly return interval. It supplies neither intramonth event time (event_time), historical availability (available_at), signal time (signal_time) nor trade time (trade_time). Manifest download times establish only that the snapshot was obtained on October 5, 2026; they cannot recover each historical monthly release. RF and industry returns share monthly frequency; Sharpe pairs identical month labels.

The main estimation window is January 2010 through December 2019. Fixed descriptive slices cover January 2010–December 2014, January 2015–December 2019 and January 2020–December 2024. The later slice is not an out-of-sample test. The industry manifest states that public download access is no blanket redistribution license; verify applicable rights before external publication and retain attribution to Fama and French. The RF source states that its one-month Treasury-bill input changed in June 2024. This historical file is also separate from monthly point-in-time vintages. See the manifest.

Project 1 Research Report: Describing Industry Portfolio Returns and Risk

Historical report for the October 5, 2026 French snapshot. All reported results and figure identities are retained. The reproduction section describes the original working-course route; the current Seed101 offline Notebook does not rerun it. Use the English reader guide for current execution.

Findings

All three industry portfolios have positive cumulative returns in the fixed January 2010–December 2019 monthly window. High Technology has the largest final wealth index, 4.627, followed by Nondurables at 3.141 and Energy at 1.331. Higher growth does not imply lower path risk: Energy's conventionally annualized volatility is 20.03% and maximum drawdown is 40.70%, both the largest of the three. Annualized Sharpe ratios against the same-month risk-free return proxy (RF) are approximately 1.039, 1.032 and 0.217 for Nondurables, High Technology and Energy. Growth, volatility and performance relative to RF measure different aspects; one ranking is not an overall investment conclusion.

This report describes history using the publication vintage obtained for this project. The sample is neither an implementable strategy nor a historical point-in-time database. Returns cannot reconstruct actual prices, price paths, announcements, information then available, trades, financing or transaction costs. The report claims no Alpha, realizable excess return or future return.

Questions and Windows

The main question asks how Nondurables, High Technology and Energy differ in 2010–2019 compound growth, return dispersion, performance relative to same-month RF and peak-to-trough losses when measured consistently at monthly frequency. Consumer goods, technology and energy provide contrasting business categories. This small teaching sample represents neither all industries nor a winner-selection procedure.

A second question asks whether main-window rankings change across fixed historical slices. The three portfolios are also reported over 2010–2014, 2015–2019 and 2020–2024, each containing 60 continuous months. The later 2020–2024 interval is a fixed descriptive segment observed after the data update. It is not an out-of-training, holdout or out-of-sample test.

The main window has 120 months and excludes 2020–2024. Figures extend through December 2024 to display path changes, shading the extension in gray. The three industries and all intervals are fixed in the configuration. The project does not search additional industries, windows or parameters to maximize return, Sharpe or other measures.

Sources, Snapshot and Sample

Inputs come from the Kenneth R. French Data Library's US 10 Industry Portfolios monthly table and the `Fama/French` three-factor monthly table. The analysis reads `NoDur`, `HiTec`, `Enrgy` and `RF`. Industry portfolios group US listed companies by industry. Source details describe formation at the end of June using industry codes, followed by returns from July through the following June. See the 10-industry description and French Data Library.

Input snapshots were downloaded and frozen on October 5, 2026. The manifest lists SHA-256 hashes for source ZIPs, extracted CSVs and processed CSVs. Historical analysis rechecks both consumed processed-file hashes against course source manifests. Industry returns use the "Average Value Weighted Returns — Monthly" block. Source percentages are divided by 100 once and saved as decimals. RF comes from the monthly factor-file block and receives the same conversion; loading does not repeat it.

Both processed snapshots contain 1,202 rows from July 1926 through August 2026. The analysis uses January 2010–December 2024, 180 months, with all four selected fields continuous and complete; its main window is the first 120 months. Source missing sentinels −99.99 and −999 were converted to empty fields during processing. The shared loader checks hashes, column types, finite values, ascending and unique months and monthly continuity. The project neither fills returns with zero, forward-fills RF nor silently removes gaps. Industry and RF monthly indexes must match exactly; join one-to-one on `month`, then select configured windows.

The manifest supplies vintages, exact paths and hash chains; the dictionary defines fields, units and clock limitations. Inputs were obtained as a revised publication vintage, rather than contemporaneously saved monthly snapshots. French states that updates rebuild the full history, so historical returns can change. See the Data Library update description. The factor-file header declares CRSP vintage 202608. Its source also states that the one-month Treasury-bill input changes from Ibbotson Associates to the ICE BofA US 1-Month Treasury Bill Index in June 2024. See the factor description. The long history therefore cannot be interpreted as data generated contemporaneously by one frozen method.

Time and Units

`month` labels a monthly return interval: `2010-01` is the January 2010 interval, not a day or intraday observation time. Inputs do not supply each historical month's `release/ingestion` time (`available_at`), signal time (`signal_time`) or trade time (`trade_time`). This analysis forms no signals or orders. It cannot determine whether an order could be submitted or filled after a close.

Industry returns are published monthly simple portfolio total returns including dividends, with dimensionless units; `0.01` means a monthly return of 1%. The project uses the standard industry file; French separately publishes an "ex. Dividends" version. See the industry-portfolio listing. Value-weighted industry returns are separate from stock prices, `open/high/low/close/volume` (OHLCV), a present-day universe or execution prices. They describe the published portfolio's return path but cannot reconstruct an investor's prices, positions, flows or costs.

`RF` is the monthly one-month US Treasury-bill return proxy in the `Fama/French` factor file. It represents a same-month risk-free opportunity-cost reference, rather than a frictionless cash rate guaranteed to every investor. Sharpe subtracts that month's RF from its industry return. There is no annual-rate conversion or single yearly constant. The source states that the proxy input comes from Ibbotson Associates through May 2024 and the ICE BofA US 1-Month Treasury Bill Index from June 2024. See the Fama/French factor description.

From Returns to Risk: Methods and Reasons

1. Connect the monthly observations into wealth using simple returns

The source directly supplies each month's portfolio simple return, `R_t`. Without external contributions, account wealth evolves as

$$W_t = W_{t-1}(1 + R_t), \qquad W_0 = 1.$$

This recursion reflects returns measured against changing capital; monthly addition loses compounding. `W_0=1` is a chosen initial wealth unit, not an observed security price. Each plotted wealth line is a product of published growth factors, so it is called a wealth index, not a price.

2. Use log returns to explain multiplication over time

Log return is defined as `g_t = ln(1+R_t)`. All returns for the three portfolios in this sample exceed -100%, so it is defined.

$$\sum_{t=1}^{T} g_t = \ln\left(\prod_{t=1}^{T}(1 + R_t)\right).$$

The identity explains why log returns describe time-series compounding: cumulative logs add, and exponentiation recovers the wealth growth factor. It does not replace cross-asset simple-return weighting; the project does not add different assets' log returns. Tables report both arithmetic monthly simple-return means and mean monthly log returns, distinguishing typical single-month movement from time-compound growth. Shared `wealth_index` still constructs cumulative wealth from simple returns.

3. Express dispersion on an annual scale and state the convention

Volatility uses monthly simple-return sample standard deviation `s(R)` with one degree of freedom deducted; the report multiplies it by `√12`. This expresses monthly dispersion on a customary annual scale for comparison. It is conventional annualization, not an identity for annual compound-return standard deviation. Serial correlation, changing volatility and nonstationarity can distort square-root scaling. Rolling figures similarly multiply the sample standard deviation of each 12 continuous months by `√12`, leaving the first 11 months blank because the window is incomplete.

4. Measure opportunity cost with same-month RF

Sharpe uses simple excess returns `e_t=R_t-RF_t`. The project reports

$$\widehat{SR}_{ann} = \sqrt{12}\,\frac{\overline{(R-RF)}}{s(R-RF)}.$$

Subtracting same-month RF in the numerator measures the arithmetic return difference relative to a risk-free alternative for that month. The denominator is the sample standard deviation of those monthly differences. Numerator and denominator thus share the same frequency before application of the stated `√12` annualization convention. This is not a strategy-effectiveness test; no correction is made for serial dependence, sample selection or multiple comparisons. RF is subtracted once. This is neither a factor regression nor an Alpha estimate.

5. Locate losses relative to the historical wealth peak

Drawdown is defined as

$$D_t = \frac{W_t}{\max_{0\le s\le t} W_s} - 1.$$

Track the running wealth peak, then measure the current proportional distance below it. This distinguishes cumulative final growth from the largest intervening loss. Tables report positive maximum drawdown loss as `-min(D_t)` and give peak and trough months. Calculations include initial wealth 1 before the window begins; the plotted starting point is an initial-capital convention, not a market price.

6. Summarize initial-to-final wealth with CAGR

Compound annual growth rate (CAGR) compresses the initial-to-final wealth ratio into a single annual compound rate:

$$\text{CAGR} = (W_T/W_0)^{1/(T/12)} - 1.$$

Each reporting window contains whole months; `T/12` years is an explicit, reproducible interval convention. CAGR provides a common annual growth scale while hiding volatility and peaks and troughs. It is therefore reported alongside volatility and drawdown rather than replacing them.

Main-Window Results: 2010–2019

Total return, CAGR, volatility and drawdown are displayed as percentages; wealth index starts at 1. Sharpe is dimensionless. All three portfolios share mean monthly RF of approximately 0.0427%.

Industry portfolio	Final wealth index	Cumulative simple return	CAGR	Annualized volatility `s(R)√12`
Nondurables (NoDur)	3.1412	214.12%	12.13%	11.14%
High Technology (HiTec)	4.6270	362.70%	16.55%	15.61%
Energy (Enrgy)	1.3313	33.13%	2.90%	20.03%

Industry portfolio	Annualized Sharpe, `√12` convention	Maximum drawdown loss (peak → trough)
Nondurables (NoDur)	1.039	15.84% (2018-01 → 2018-12)
High Technology (HiTec)	1.032	17.68% (2018-08 → 2018-12)
Energy (Enrgy)	0.217	40.70% (2014-06 → 2016-02)

The two monthly means reflect compounding differences. Arithmetic simple-return means are 1.0099% for NoDur, 1.3848% for HiTec and 0.4053% for Enrgy per month. Mean log returns are respectively 0.9538%, 1.2766% and 0.2384% per month. Energy has a particularly large gap: compounding positive and negative months gives lower growth than a reading of its arithmetic monthly mean alone suggests. Mean logs are not another directly weightable industry portfolio return.

HiTec has the highest final wealth; Energy has the greatest dispersion and peak-to-trough loss. NoDur and HiTec have similar Sharpe values. Ranking them by a small Sharpe difference overlooks estimation uncertainty, interval selection and path risk. This report constructs neither confidence intervals nor significance tests, and does not interpret decimal differences as statistically stable superiority.

Fixed Segments: Do Descriptive Rankings Change over Time?

Recalculating the same measures over predetermined segments checks whether rankings from one ten-year description depend on the historical phase. It does not select a best interval. CAGR, conventionally annualized volatility and maximum drawdown use percentages; Sharpe is dimensionless. Each industry has 60 continuous months in every segment. Wealth restarts at 1 immediately before each window, so segment drawdown tracks peaks only within that 60-month window, without carrying over the previous window's peak.

Fixed segment	Industry	CAGR	Annualized volatility	Annualized Sharpe	Maximum drawdown
2010–2014	NoDur	17.05%	11.08%	1.481	8.47%
2010–2014	HiTec	14.84%	15.79%	0.956	15.39%
2010–2014	Enrgy	7.88%	19.25%	0.488	25.88%
2015–2019	NoDur	7.41%	11.16%	0.609	15.84%
2015–2019	HiTec	18.30%	15.55%	1.102	17.68%
2015–2019	Enrgy	−1.84%	20.86%	−0.033	28.18%
2020–2024	NoDur	4.93%	15.25%	0.235	19.03%
2020–2024	HiTec	22.04%	21.84%	0.920	33.60%
2020–2024	Enrgy	12.44%	37.50%	0.435	51.16%

NoDur has the highest CAGR in the first five years; HiTec leads in both later segments. Energy has cumulative return of −8.88% and Sharpe of −0.033 in 2015–2019, then positive cumulative return in 2020–2024. In that later period it also has the highest annualized volatility, 37.50%, and maximum drawdown, 51.16%. Rankings, excess returns and path risk therefore change with the interval; the main window does not identify a permanent winner. All windows are retrospective analyses of the downloaded vintage, supplying neither transferable next-period robustness nor out-of-sample evidence. Complete machine-readable means and `peak/trough` months are in the window-results CSV.

Figures: Read Coordinates without Turning Returns into Prices

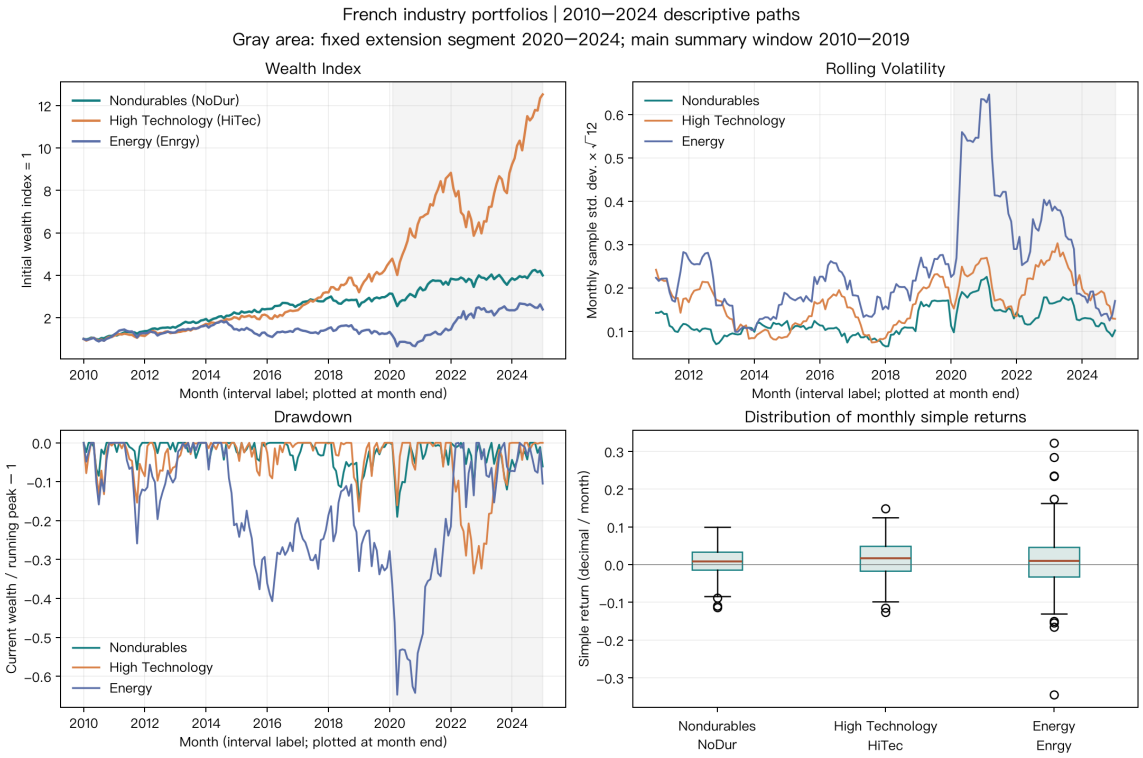


Figure 1: 2010–2024 wealth, volatility, drawdown and return distributions for three industry portfolios

Upper left: the horizontal axis uses month-end plotting labels for monthly return intervals; the vertical axis is wealth compounded from 1. Its initial point is the selected December 2009 capital, not an observed December 2009 price. Solid lines cover the entire 2010–2024 plotting window. Gray shading marks 2020–2024 as a descriptive extension after the main window. Upper right: months remain on the horizontal axis; the vertical axis is the trailing 12-month sample standard deviation multiplied by $\sqrt{12}$. Brief spikes indicate greater local dispersion, not model-predicted future risk.

Lower left: the vertical axis is `wealth / running maximum wealth - 1`. Zero means wealth is at its historical peak; −0.5 means it is half below that peak. Its time axis matches the wealth path and retains initial capital. Lower-right boxplots show the three industries horizontally and 2010–2024 decimal monthly simple returns vertically. Center lines are medians, boxes span quartiles and separate points represent distant sample months. Extreme points remain part of the actual input; unfavorable or prominent observations are not removed. Month labels supply no intramonth events or trade times.

Limitations and Conclusion

These French industry inputs teach compounding and sample statistics. They supply no individual prices, OHLCV, historical constituent lists, contemporaneous monthly vintages, precise release dates, signals, executions or costs. Updates may revise historical returns; the October 5, 2026 download time cannot become each historical month's availability time. The Treasury-bill input changes in June 2024, a distinction retained in comparisons across periods.

Square-root-12 annualization is a reporting convention. Serial dependence and conditional volatility affect annual aggregation. Sharpe is a window mean-to-standard-deviation ratio without uncertainty or serial-dependence adjustments; it is not an Alpha estimate. No position rule exists, so there are no estimable strategy transaction costs, financing, borrowing or net returns. Inventing an "after-cost Alpha" value would exceed the inputs' evidential scope.

The strongest supported description is that High Technology has the highest historical compound growth in 2010–2019 among these published, revised monthly value-weighted industry portfolios, while Energy has greater historical volatility and peak losses. Industry return rankings change across fixed historical slices. This cannot be extrapolated to future individual stocks or tradable investments, and establishes neither an asset-pricing explanation nor causality.

Historical Reproduction Route

```
.venv/bin/python projects/project01/run_project01.py
.venv/bin/python -m pytest tests/test_project01_analysis.py -q
```

The historical CLI reads frozen files and generates figures, JSON metrics and window CSVs. Tests check snapshot hashes, continuous intervals, one-to-one RF alignment and hand-checkable `wealth/drawdown` boundaries. See the QA record for the complete original checks. Historical teaching derivations and tables are in the original Notebook, designed for a clean kernel under that original data route.

Project 1 Exercises

> Exercises 2 and 5 read the original historical report. Their answers use the October 5, 2026 French snapshot, separate from the current Seed101 constructed output. The CLI at the end belongs to the historical course route; use the reader guide for the offline Notebook.

Attempt these questions before consulting the solutions. Return inputs are decimals; ignore external cash flows unless a question states otherwise.

1. **Compounding and two return measures.** Initial wealth is 100, followed by monthly simple returns of $+10\%$ and -10%. Calculate wealth at each month end, cumulative two-month simple return, arithmetic mean monthly return and mean monthly log return. Why do they differ?

2. **Read the report.** HiTec's mean monthly simple return exceeds NoDur's in 2010–2019. What wealth index, compound annual growth rate (CAGR) and volatility does the report give for HiTec? Do these values establish that "HiTec is a better future investment"? Explain.
3. **Same-month RF.** An industry returns 2.0% in a month whose RF is 0.4%. Calculate the arithmetic excess return. What unit or frequency error results from subtracting the year's annual RF of 4% from every monthly return?
4. **Peak drawdown.** Wealth follows `[1.00, 1.50, 0.75, 1.20]`. Calculate drawdown from the running peak at every point and its maximum loss. Calculate the full path's final cumulative simple return and explain why it differs from maximum drawdown. How does the intermediate wealth of 0.75, a −25% loss from the initial value, differ from maximum drawdown?
5. **Compare segments.** Use the results table to compare the three industries in 2010–2014 and 2015–2019. Which industry has the highest CAGR in each segment? How should the negative 2015–2019 Energy Sharpe ratio be interpreted? Does this test a future tradable strategy?
6. **Read the figures.** What questions do wealth index, rolling volatility, drawdown and boxplots answer? What does the initial point on the wealth figure's time axis represent, and why are the first 11 months of rolling volatility blank?

Historical course-work route for regenerating the numerical tables:

```
.venv/bin/python projects/project01/run_project01.py
```

Project 1 Complete Solutions

All six original answers are retained. Historical-report answers refer to the October 5, 2026 French snapshot, not to new Seed101 Notebook outputs.

1. Compounding and two return measures

Wealth rises from 100 to 110, then falls to 99. Cumulative two-month simple return is `99/100 - 1 = -1%`. The arithmetic mean is `(10% - 10%) / 2 = 0%`. Mean log return is

$$\frac{\log(1.10) + \log(0.90)}{2} = \frac{\log(0.99)}{2} \approx -0.5025\%/\text{month}.$$

The arithmetic mean averages the two monthly returns. Wealth multiplies their growth factors, `1.10 × 0.90 = 0.99`. Logs convert this time-series multiplication to addition; exponentiating the mean log return gives the monthly geometric return, approximately −0.5013%.

2. Read the report

HiTec's 2010–2019 wealth index is approximately 4.6270, starting at 1, for cumulative return of approximately 362.70%. Under the convention of 120 months equal to 10 years, CAGR is 16.55%; multiplying sample volatility by `√12` gives 15.61%. These are descriptive statistics for one fixed historical window. Inputs are subsequently revised published industry portfolio returns without historical availability clocks, signals, trades or costs. They establish neither future performance nor a tradable strategy.

3. Same-month RF

Arithmetic excess return is `2.0% − 0.4% = 1.6%`. Subtracting annual 4% from every monthly return treats an annual unit as a monthly unit. This is neither a same-interval comparison nor an effective-rate conversion. Use the published monthly RF when analyzing monthly returns.

4. Peak drawdown

Wealth is 1.00, 1.50, 0.75, 1.20. Drawdowns from the running peak are 0%, 0%, −50%, −20%, so maximum drawdown loss is 50%. Full-path cumulative return is `1.20 / 1.00 − 1 = 20%`. Intermediate wealth relative to initial capital gives `0.75 / 1.00 − 1 = −25%`, whereas its drawdown from the peak of 1.50 is −50%. Cumulative return uses initial wealth as its denominator; drawdown uses a historical peak and depends on path order.

5. Compare segments

In 2010–2014, NoDur CAGR is 17.05%, exceeding HiTec's 14.84% and Enrgy's 7.88%. In 2015–2019, HiTec's 18.30% exceeds NoDur's 7.41%; Energy is −1.84%. Growth rankings therefore vary with the fixed interval. Energy's annualized 2015–2019 Sharpe is approximately −0.033 because mean monthly excess return is slightly negative (−0.0578%) in that sample. This is not a guaranteed-loss law. Both intervals and the later segment are retrospective historical slices without real-time vintages, strategy signals, executions, costs or a reserved out-of-sample evaluation.

6. Read the figures

- Wealth index compounds monthly simple returns across time, showing how the specified path accumulates. Initial wealth 1 near December 2009 is a chosen normalization, not a market price.
- Rolling volatility uses the current month and previous 11 months to estimate a 12-month sample standard deviation and multiply it by $\sqrt{12}$. It displays changing local dispersion. Until 12 observations exist, the window is incomplete and the first 11 months are blank.
- Drawdown shows wealth below its preceding peak. A value of −0.5 means 50% below that peak.

- Boxplots summarize full-window monthly simple returns. The center line is the median, boxes span quartiles and individually plotted points display more distant observations. The horizontal axis identifies fixed industry portfolios; the vertical axis uses decimal monthly returns.

Chapter 12 — Financial Statements for Quantitative Research

The central task is to connect the income statement, balance sheet, and cash flow statement into company features with explicit units, periods, versions, and reproducible information-availability times. Prerequisites: Chapter 3 on data engineering and Chapter 10 on market instruments; Chapter 11 on returns and compounding provides background. After this chapter, you can explain the main links among the three statements; construct and interpret revenue, profit margins, cash flow from operations (CFO), capital expenditure (CapEx), free cash flow (FCF), working capital, and debt features; and prevent later restatements from leaking into earlier signals.

1. Learning Objectives

After reading this chapter, you should be able to do four kinds of work.

1. Trace an item in one statement to its effects on the other two. Given credit sales, depreciation, capital expenditure, or borrowing, explain how profit, balance-sheet accounts, and cash change.
2. Align the numerators, denominators, currencies, units, and periods of common financial features. Define explicit proxies for earnings before interest and taxes (EBIT), earnings before interest, taxes, depreciation and amortization (EBITDA), CFO, CapEx, FCF, selected operating working capital (Selected OWC), net debt, and FCF yield.
3. Preserve the reporting period, first public version, filing version, revision version, receipt time, and local ingestion time for every value. Determine which version was both public and in the research database at a given signal_time.
4. Check figures and definitions against actual statements. Recalculate the original Apple FY2024/FY2023 statement lines, distinguish Spire's original FY2023 10-K from its later 10-K/A, and explain what these limited cases do and do not establish.

The standard is reproducibility: a reviewer can use source links, accession numbers, statement line names, periods, and formulas to recalculate the results. Constructed data remain distinct from company disclosures, and late versions cannot overwrite values visible earlier.

2. Why Quants Need This

Factor research often compresses financial statements into a number, such as operating margin, CFO/assets, net debt/EBITDA, or FCF yield. Comparing numbers looks easy, but producing them requires choices: consolidated profit or profit attributable to the parent, ending or average assets, a negative CapEx cash-flow line or positive expenditure, the time a revision became public, and the time a vendor entered it into a database.

These choices can change rankings. One company may report revenue growth yet collect cash slowly because receivables grow faster; another may report lower profit but higher cash flow after collecting receivables. Higher CapEx depresses near-term FCF while potentially building

future capacity. Accounting features are therefore measurements with definitions and versions, rather than truths obtained simply by reading a statement.

Financial statements describe a company and reporting period; security prices belong to tradable securities, share classes, and trading times. Mapping company data to common stock also requires checks of permanent security identifiers, listing classes, currencies, share capital, tradability, and time. Chapter 13 connects enterprise value to equity value. This chapter first establishes trustworthy statement inputs, without claiming that a factor earns excess returns.

3. Intuition

Worked Example: Transactions, Three Statements, and Cash from 50 to 63

Consider a fictional company over one quarter. It delivers goods and services worth 100, collecting 80 in cash and selling 20 on credit. Cost of goods sold (COGS) is 40, cash operating expenses are 25, depreciation and amortization (D&A) are 10, interest is 5, and income tax is 4. The company buys equipment for 8 and borrows another 15.

All amounts use the constructed unit u, cover the same quarter, and describe one consolidated company with no noncontrolling interest (NCI). Opening accounts receivable (AR) and accounts payable (AP) are zero. There are no dividends, share issues, exchange-rate effects, or other adjustments beyond the listed transactions. Interest and income tax payments are classified in CFO in this example. Inventory purchases are 50: 40 paid in cash and 10 owed to suppliers. COGS of 40 leaves inventory up by 10 at quarter-end.

Purchases of property, plant and equipment (PP&E) belong to cash flow from investing activities (CFI); new borrowing belongs to cash flow from financing activities (CFF).

Quarterly transaction	Income-statement effect	Balance-sheet account changes	Cash-flow classification and calculation
Deliver sales of 100: collect 80, sell 20 on credit	Revenue +100	Cash +80, AR +20	Direct method: cash from customers +80; indirect method: revenue already enters starting profit, so deduct the AR increase of 20
Buy inventory of 50: pay 40, owe 10; recognize COGS of 40	COGS −40	Cash −40, net inventory +10, AP +10	Direct method: supplier payments −40; indirect method: deduct inventory increase 10, add AP increase 10
Pay cash operating expenses of 25	Operating expenses −25	Cash −25	CFO −25
Recognize depreciation of 10	Depreciation expense −10	Net PP&E −10	Noncash expense: add back +10 in indirect CFO
Pay interest of 5 and income tax of 4	Interest expense −5, tax expense −4	Cash −9	Both classified in CFO here: −5−4
Buy equipment for cash of 8	No immediate expense on purchase	Gross PP&E +8, cash −8	CFI −8; positive CapEx expenditure = 8
Borrow another 15	No revenue or profit effect	Debt +15, cash +15	CFF +15

Continued: the same complete example

The income statement gives gross profit $100 - 40 = 60$, operating income $60 - 25 - 10 = 25$, and net income $25 - 5 - 4 = 16$. This chapter's EBITDA proxy is $25 + 10 = 35$. Under these simplified assumptions, CFO is $35 - 5 - 4 - 20 = 6$, where 20 is the net increase in Selected OWC. The EBITDA proxy deducts neither interest, tax, nor working-capital changes; CFO reflects their operating cash effects, so the two are unequal. Check the same CFO by the indirect method: start with net income 16, add depreciation 10, deduct AR growth 20 and inventory growth 10, and add AP growth 10, totaling 6.

Of the six Selected OWC components, only AR, inventory, and AP change here: +20, +10, and +10, respectively, producing a net increase of 20. Vendor non-trade receivables, other operating current assets, and deferred revenue have zero changes. CFO already includes the cash effects of all six items. FCF deducts only the specified CapEx; do not deduct the same 20 from CFO again.

Ending stock	Opening	Quarterly change	Ending	Source of change
Cash	50	+13	63	CFO 6 + CFI (−8) + CFF 15
Accounts receivable	0	+20	20	Credit sales
Inventory	20	+10	30	Purchases 50 − COGS 40
Net PP&E	100	−2	98	Purchases 8 − depreciation 10
Total assets	170	+41	211	Sum of the four asset changes
Accounts payable	0	+10	10	Supplier credit
Borrowing	40	+15	55	New borrowing
Total liabilities	40	+25	65	AP and borrowing
Equity	130	+16	146	Net income 16, no dividends or share issues
Total liabilities and equity	170	+41	211	Ending balance-sheet identity holds

The statements answer different questions. The income statement explains accounting performance during the period; the balance sheet lists unsettled resources and obligations at the ending date; the cash flow statement explains cash rising from 50 to 63. Net income 16 enters retained earnings within equity, with no dividends or share issues here. Equipment purchases first increase assets and reduce cash; later depreciation reduces profit and net equipment value over time. Borrowing creates cash and debt, not revenue. Under this chapter's definition, $FCF = 6 - 8 = -2$. This deducts only the specified PP&E expenditure, not debt principal, dividends, acquisitions, or lease payments, and is not cash unconditionally distributable to shareholders.

All figures above are constructed for teaching. The SEC investor guide also introduces the three statements through these questions and describes indirect CFO as net-income adjustments for noncash items and changes in operating assets and liabilities. SEC Beginners' Guide to Financial Statements

4. Mathematical Foundations

Start financial-statement formulas by checking dimensions.

- Income-statement and cash-flow items are flows over a period, such as FY2024 net sales and CFO.
- Balance-sheet items are stocks at a date, such as cash, debt, and receivables on 2024-09-28.
- Amounts must share a currency and unit. A value of 118,254 in USD millions differs numerically from 118,254,000 in USD thousands; the bare number is insufficient.
- Ratios are usually dimensionless. Dividing one amount by another still requires compatible periods, currencies, and units.
- Fiscal years need not equal calendar years. Apple FY2024 ends on 2024-09-28 and FY2023 on 2023-09-30; neither is a calendar year ending December 31.

A feature table can represent a fact by the tuple (i, p, v, x, u, a): i is a permanent issuer or entity identifier, p the reporting period or instant, v the document version, x the value, u the currency and unit, and a its information-availability time. Also record local ingestion time g, statement name, line name, source URL, accession, and processing-code version. These fields let two researchers determine whether they calculated the same quantity.

5. Formal Definitions

The **income statement** summarizes revenue, expenses, and profit over a period; the **balance sheet** summarizes assets, liabilities, and equity at a date; the **cash flow statement** summarizes operating, investing, and financing effects on cash and cash equivalents over a period. SEC guidance on reading a 10-K places audited financial statements in the annual report's financial-information section. Individual items still require the company's notes and audited statements. How to Read a 10-K

The balance-sheet identity is:

$$\text{Assets}_t = \text{Liabilities}_t + \text{Equity}_t.$$

Here t is the closing date, not a period. Assets, Liabilities, and Equity are totals within the same consolidation scope. If NCI is presented within equity, Equity on the right includes it. A common-shareholder factor must instead match equity and earnings attributable to the parent's common shareholders.

This chapter reconciles the cash flow statement as follows:

$$\text{Cash}_{t_1} - \text{Cash}_{t_0} = \text{CFO} + \text{CFI} + \text{CFF} + \text{FX}.$$

t_0, t_1 are the opening and closing instants of the reporting period. CFO, CFI, and CFF are operating, investing, and financing cash flows; FX separately reconciles exchange-rate effects on cash and cash equivalents. Actual statements may differ in their coverage of cash, cash equivalents, restricted cash, and exchange adjustments: use the statement's own definition. IAS 7 also organizes cash flows into operating, investing, and financing activities and presents exchange-rate effects separately from those flows. IAS 7.11 and 7.28

Revenue is a period item recognized under the reporting entity's applicable standards and contractual facts. Revenue growth is often written as:

$$g_t = \frac{R_t}{R_{t-1}} - 1,$$

R_t and R_{t-1} are revenues for the same company, currency, unit, comparable fiscal periods, and consistent version snapshot; g_t is a dimensionless growth rate. Differences in fiscal weeks, acquisition scope, exchange rates, or revenue-recognition versions may leave the rate computable without representing the same economic change.

Operating income generally subtracts operating costs and expenses from operating revenue. Presentation and classification of restructuring or unusual items differ across companies. **EBIT** can approximate operating income, but the terms are not interchangeable in every statement. Declare the original line used rather than storing only the label "EBIT."

Analysts often construct **EBITDA** as:

$$\text{EBITDA}_t^* = \text{EBIT}_t^* + \text{D\&A}_t.$$

In the code, EBIT* uses operating income and D&A is D&A for the same period. The asterisk marks a teaching proxy. Company-reported Adjusted EBITDA may additionally exclude share-based compensation, restructuring charges, impairments, or acquisition amortization; this formula does not equal that disclosed adjusted measure.

Net income is the accounting result after applicable expenses, taxes, and other gains and losses. Consolidated net income may include profit attributable to NCI. For a factor representing the parent's common shareholders, select the profit line attributable to the parent or common shareholders. IAS 1 requires separate presentation of profit and comprehensive income attributable to parent owners and NCI. IAS 1.81A and 1.106(a)

Cash flow from operations (CFO) is net operating cash inflow over the reporting period. The indirect method usually begins with net income or loss, reverses noncash items, and adjusts for cash effects of operating assets and liabilities. CFO therefore equals neither net income nor EBITDA.

Capital expenditure (CapEx) here means cash spent on the specified PP&E purchases during the reporting period. Statements often report it as a negative investing cash flow. Convert expenditure to a positive amount for the formula:

$$\text{CapEx}_t = -\text{PP\&E cash-flow line}_t \quad \text{(when the line is nonpositive cash flow).}$$

Software development, capitalized R&D, leases, intangibles, and acquisitions classified as investing by a company are not automatically included in this CapEx definition.

Definition and Conditions: This Chapter's CFO-minus-CapEx Proxy

This chapter explicitly defines **free cash flow (FCF)** as:

$$\text{FCF}_t = \text{CFO}_t - \text{CapEx}_t = \text{CFO}_t + \text{PP\&E cash-flow line}_t.$$

CFO_t and CapEx_t must share the same period, currency, unit, and statement version. The first is signed net operating cash flow; the second is positive capital expenditure. SEC staff identifies CFO minus capital expenditures as a common FCF calculation while noting that FCF lacks a uniform definition. Readers must not be led to believe it deducts all debt service and other nondiscretionary spending. SEC Non-GAAP C&DI 102.07

Working capital (WC) has no single factor definition. To specify this chapter's scope, define the following six-item **Selected Operating Working Capital (Selected OWC)**. It is a reproducible subset of operating working capital, not complete working capital or complete net working capital:

$$\text{Selected OWC}_t = \text{AR}_t + \text{VendorRec}_t + \text{Inventory}_t + \text{OtherOpCA}_t - \text{AP}_t - \text{DeferredRevenue}_t.$$

AR is AR, VendorRec vendor non-trade receivables, Inventory inventory, OtherOpCA other current operating assets included here, AP AP, and DeferredRevenue deferred revenue liabilities. This excludes cash, security investments, borrowings, and other current liabilities. A full working-capital definition may cover other accounts; this Selected OWC subset is not a universal net-working-capital measure.

A simplified indirect method is:

$$\text{CFO}_t \approx \text{NI}_t + \text{Noncash}_t - \Delta\text{OWC}_t + \text{OtherAdj}_t.$$

NI is net income; Noncash adjusts for noncash expenses such as depreciation less noncash gains; $\Delta\text{OWC} = \text{OWC}_{t_1} - \text{OWC}_{t_0}$; and OtherAdj includes other adjustments, tax/interest classification, noncurrent operating items, exchange effects, or acquisition effects. The approximation sign matters: a selected balance-sheet subset may not reproduce the full movements across the larger set of accounts in the statement's CFO reconciliation.

Limit **debt** to explicitly defined interest-bearing financing items. The Apple example uses commercial paper and current and noncurrent term debt. Define net debt as:

$$\text{NetDebt}_t = \text{InterestBearingDebt}_t - \text{CashAndEquivalents}_t.$$

This formula does not deduct marketable securities or include lease liabilities, and it is not every industry's definition of net financial liabilities. When the EBITDA proxy is nonpositive, NetDebt/EBITDA lacks the interpretation adopted here; the code returns an undefined value rather than forcing an ordinary ratio.

A **factor** here is a cross-sectional or time-series feature obtained through an explicit transformation of financial facts, such as gross margin, CFO margin, FCF margin, net debt/EBITDA, or FCF yield. It defines a candidate signal, without automatically establishing expected returns, a causal mechanism, or a tradable strategy.

6. Derivations

6.1 Why the Statements Connect

A simplified equity roll-forward is:

$$\text{Equity}_{t_1} \approx \text{Equity}_{t_0} + \text{NetIncome}_t - \text{Distributions}_t + \text{OtherEquityChanges}_t.$$

Distributions denotes dividends or other distributions. The equity presentation of repurchases depends on standards and statement classification. Other equity changes include other comprehensive income, share issuance, currency translation, NCI, and accounting adjustments. Ending equity does not simply equal opening equity plus net income.

Indirect CFO starts from income-statement net income: the first link. CapEx is the second: equipment purchases increase PP&E and reduce balance-sheet cash, appear as investing outflows, and subsequently affect profit through depreciation. Borrowing is the third: proceeds increase cash and debt without increasing revenue; repayment reduces cash and debt, while principal repayment is not an operating expense.

These links explain how a business can simultaneously report positive net income, negative CFO, positive CapEx expenditure, and negative FCF. They answer different questions about accounting earnings, operating funding needs, investment intensity, and operating cash after defined CapEx.

6.2 From Profit to Cash

With simplified operating accounts, increases in AR and inventory consume cash; increases in AP and deferred revenue temporarily supply cash:

$$\begin{aligned}\text{CFO}_t \approx\ & \text{NI}_t + \text{D\&A}_t - \Delta\text{AR}_t - \Delta\text{VendorRec}_t \\ & - \Delta\text{Inventory}_t - \Delta\text{OtherOpCA}_t \\ & + \Delta\text{AP}_t + \Delta\text{DeferredRevenue}_t + \text{OtherAdj}_t.\end{aligned}$$

The equation connects a period flow to changes between endpoint stocks. Each ΔX_t is ending less opening balance. Positive operating-asset changes enter approximate CFO negatively; positive operating-liability changes enter positively. It assumes selected changes correspond to the relevant cash receipts and payments. Actual statements include allowances, exchange effects, acquisitions, disposals, noncurrent items, and reclassifications; this is not a company's complete indirect reconciliation.

The six changes explicitly match Selected OWC. OtherAdj excludes these six cash changes and contains only other adjustments. Ignoring or separately adjusting the noncash differences above, their working-capital changes combine into $-\Delta$Selected OWC. Once CFO includes that movement, deducting the same ΔSelected OWC again is a double deduction. To construct CFO excluding working-capital movements, state exactly which changes are added back or removed and reconcile them individually.

6.3 From Statement Items to Candidate Factors

Operating margin, net margin, and CFO margin describe operating profit, final accounting earnings attributable to a specified entity, and operating cash relative to revenue, respectively. Typical ratios are:

$$\begin{aligned}\text{OperatingMargin}_t &= \frac{\text{OperatingIncome}_t}{\text{Revenue}_t},\\ \text{NIMargin}_t &= \frac{\text{NetIncome}_t}{\text{Revenue}_t},\\ \text{CFOMargin}_t &= \frac{\text{CFO}_t}{\text{Revenue}_t}.\end{aligned}$$

Operating-margin and CFO-margin amounts must cover the same consolidation scope and fiscal period, currency, and unit. Net margin also requires an attribution convention: the equation above uses consolidated net income and consolidated revenue. For net margin attributable to the parent, this chapter divides parent-attributable net income by consolidated revenue for the same group and period. This revenue-denominator convention differs from matching common earnings to common equity for ROE or common market capitalization for P/E. Do not mechanically allocate revenue by ownership percentage to invent undefined "common-shareholder revenue." Negative net income still expresses a loss relative to the declared revenue base. With nonpositive revenue, these margins lose their ordinary business interpretation.

FCF margin and FCF yield use different denominators. For cross-company annual yield comparisons, use consistent annual or trailing-twelve-month (TTM) FCF. Label a quarterly window separately rather than treating it as annualized. A fixed window makes rankings reflect financial differences instead of differing measurement lengths:

$$\text{FCFMargin}_t = \frac{\text{FCF}_t}{\text{Revenue}_t}, \qquad \text{FCFYield}_{s,t} = \frac{\text{FCF}_{i,p,v}}{\text{MarketCapitalization}_{s,t}}.$$

i, p, v identifies the company, reporting period, and selected statement version; s, t identifies the security and market-valuation time. Yield divides period FCF from a known version by equity market capitalization at signal time, in comparable currency and units. Do not backfill the denominator with future prices or share counts. FCF yield is also not an enterprise-value yield. Chapter 13 examines EV versus equity scope.

Asset-efficiency ratios often divide a period flow by an average stock, for example:

$$\text{ROA}_t = \frac{\text{NetIncome}_t}{(\text{Assets}_{t_0} + \text{Assets}_{t_1})/2}.$$

ROA is the simple net-income/average-total-assets definition shown here. Other studies may use operating income, after-tax operating profit, or another numerator. Average assets match period flows with average resources; using only an ending balance changes the denominator to an ending-stock measure. Both prior and current statement values must come from versions visible at signal time.

6.4 Deriving Version and Availability Rules

Conditions: Two Clocks and Independent Validity

For candidate version v and signal time s, this simplified demonstration checks two availability clocks, assuming no withdrawal or non-reliance event:

$$\text{eligible}(v, s) \iff a_v \leq s \ \wedge \ g_v \leq s.$$

a_v is when the value first became public or that version became a public source; g_v is when the research pipeline confirmed receipt and made it usable by this research run; s is the signal-generation cutoff. Both clocks are necessary. A public document absent from this research database was not readable by this backtest instance then. A prerelease field ingested before publication cannot inform a market signal early either.

Among available versions, select the one with the latest source-publication time:

$$v^*(s) = \arg \max_{v:\ \text{eligible}(v,s)} a_v.$$

Sort by a_v, not ingestion time g_v. An older report ingested late must not obscure a newer report ingested on time. A revision replaces nothing before both its publication and ingestion. Conflicting rows for the same company, period, and source-availability time require stopping to resolve the version key, rather than letting file order select the value.

This is only a two-clock selector, not a complete statement-validity system. A withdrawal or non-reliance notice can change an old value's research eligibility without supplying a new number. Production selection also needs an event-time validity state $h_v(s)$: 1 means the version remains eligible for the defined research at the signal cutoff, based on status events already public and ingested; 0 means withdrawn or marked non-reliable. Full eligibility includes $a_v \leq s$, $g_v \leq s$, and $h_v(s) = 1$. Status events need their own source versions and publication/ingestion times. A notice learned later must not be backfilled into an earlier cutoff.

After Spire's non-reliance notice became visible to the research system but before an eligible restatement arrived, "latest available number" would not justify continued use of an invalid original. Mark the feature unavailable and follow a predefined missing-data policy to exclude, pause, or flag it for review. Do not fill the gap with the unpublished 2025 restatement. This chapter's code has no withdrawal-event interface or complete company status timeline. Its constructed two-clock example does not cover that real intermediate window and cannot establish historical tradable signals for Spire.

Real records often establish only a date, without precise publication time or time zone. Preserve date precision instead of inventing midnight UTC. Apple FY2024 10-K headers give period of report 2024-09-28, filed date 2024-11-01, and raw acceptance string 20241101060136; Apple's newsroom dates the FY2024 release 2024-10-31. Spire's original FY2023 period ends 2023-12-31, with filing on 2024-03-06, an August 2024 non-reliance notice for certain historical statements, and a 10-K/A filed 2025-03-03. The acceptance strings are not converted into a trading time zone here, and local vendor ingestion records are unavailable. Apple EDGAR headers, Spire original headers, Spire restatement 10-K/A

7. Financial Interpretation

Statement items describe accounting recognition; researchers then choose how to measure economic phenomena. Revenue growth measures recognized sales growth, not customer payment timing. Margins measure a profit level relative to revenue, not cash after fixed-asset replacement, debt principal, and shareholder distributions. CFO represents reported operating cash, without deducting every investment need. FCF interpretation depends on the statement and CapEx definition.

Common quality-factor intuitions ask whether profit converts to cash, assets turn over quickly, or debt increases financial fragility. Candidate measures include CFO/NetIncome, CFO/Revenue, ROA, and net debt/EBITDA. These remain measurement choices. Near-zero net income magnifies CFO/NetIncome; negative EBITDA makes net debt/EBITDA hard to interpret; low FCF during rapid capacity construction may indicate investment rather than deteriorating operations.

Working-capital changes have multiple interpretations. AR growing faster than revenue may indicate slower collection, quarter-end shipments, or a changing business mix. Delaying suppliers temporarily raises CFO, but payment terms and sustainability also matter. Strong seasonality can confound annual endpoint changes; quarterly comparisons need matching quarters and fiscal weeks.

Debt measures must match the equity entity. Gross interest-bearing debt, cash offsets, lease liabilities, supply-chain finance, and NCI may change leverage or valuation scope. Deducting illiquid restricted cash or omitting debt is not repaired by retaining the name NetDebt. Store the formula version and included account list.

Noncontrolling interest (NCI) sharpens the distinction between group net income and earnings attributable to the parent's common shareholders. Consolidating a subsidiary without owning all its equity includes amounts attributable to outside holders in consolidated profit and balance-sheet totals. Common-equity profitability and market multiples require numerators and denominators for the same equity holders. IAS 1 presents parent-owner and NCI attribution separately. IAS 1

Accounting differences also affect international samples. Interest, dividends, and certain cash-flow classifications may vary by standard and period; presentation of revenue, leases, and development expenditure can differ too. Uniform English field names do not erase these differences. Preserve original labels, standards, company extensions, accounting policies, years, and revision status.

8. Numerical Example

8.1 Three-Statement Hand Calculation: A Constructed Company

Section 3, this section, and 12-E07/12-L06 use the same fictional company, quarter, and constructed unit u, allowing line-by-line recalculation. The income statement is:

$$\text{GrossProfit} = 100 - 40 = 60, \quad \text{EBIT}^* = 60 - 25 - 10 = 25, \quad \text{NetIncome} = 25 - 5 - 4 = 16.$$

Revenue, COGS, cash operating expenses, depreciation, interest, and tax are constructed period amounts. Operating income approximates EBIT; other nonoperating items are excluded.

With opening operating assets of inventory 20 and ending AR 20, inventory 30, and AP 10, Selected OWC rises from 20 to $20 + 30 - 10 = 40$, an increase of 20. Begin with net income 16, add noncash depreciation 10, deduct AR and inventory increases $20 + 10$, and add AP growth 10 to obtain CFO 6. A quick check is $\text{NI} + \text{D\&A} - \Delta\text{OWC} = 16 + 10 - 20 = 6$.

CapEx 8 is positive expenditure; the PP&E cash-flow line records -8. This chapter's FCF is therefore $6 - 8 = -2$. With new borrowing 15, opening cash 50, and no other cash flows, ending cash is $50 + 6 - 8 + 15 = 63$.

Ending assets are cash 63, AR 20, inventory 30, and net PP&E $100 + 8 - 10 = 98$, totaling 211. Liabilities are AP 10 and debt $40 + 15 = 55$; equity rises from 130 by net income 16 to 146, also totaling 211 on the right. Equipment purchases of 8 first enter assets; depreciation of 10 reduces book PP&E through expense. Cash payment and current-period depreciation differ.

8.2 Apple FY2024 and FY2023: Recalculating Original Statement Figures

Apple's 2024 Form 10-K consolidated cash flow statement uses USD millions. For FY2024 (ending 2024-09-28) and FY2023 (ending 2023-09-30), Cash generated by operating activities is 118,254 and 110,543; Payments for acquisition of property, plant and equipment is $-9{,}447$ and $-10{,}959$. Convert the negative lines into positive CapEx expenditure:

$$\text{FCF}_{2024} = 118{,}254 - 9{,}447 = 108{,}807, \qquad \text{FCF}_{2023} = 110{,}543 - 10{,}959 = 99{,}584$$

All amounts are USD millions. Year-over-year FCF growth is $108{,}807/99{,}584 - 1 \approx 9.26\%$. This year-over-year change in FCF is neither a stock return nor sufficient evidence of a sustained improvement in cash generation.

Net sales in the same 10-K are 391,035 and 383,285, growing approximately 2.0220%. FY2024 gross profit is 180,683, giving gross margin $180{,}683/391{,}035 \approx 46.21\%$. Operating income of 123,216 gives operating margin approximately 31.51%. D&A of 11,445 produces the teaching EBITDA proxy $123{,}216 + 11{,}445 = 134{,}661$, not an Apple-announced adjusted EBITDA.

Under this chapter's debt definition, FY2024 interest-bearing debt is commercial paper 9,967 plus current term debt 10,912 plus noncurrent term debt 85,750, totaling 106,629. Deduct only cash and cash equivalents 29,943 to obtain net debt 76,686 USD millions. Marketable securities

and lease liabilities are outside this narrow definition. The statement supplies no matched historical market-capitalization input for this example. The FCF-yield code separately constructs market capitalization of 4,000,000 USD millions solely to demonstrate same-unit division. Do not report 2.72% as Apple's actual FCF yield.

8.3 Spire FY2023: Original and Restated Versions

Worked Example: Two Versions of Annual FCF and Their Limits

Spire's original FY2023 Form 10-K (accession 0000950170-24-027164) reports CFO −23,622 and PP&E purchase cash flow −30,037 in USD thousands. Convert to positive CapEx expenditure 30,037:

$$\text{FCF}_{\text{original}} = -23{,}622 - 30{,}037 = -53{,}659 \text{ USD thousands.}$$

Spire's 2025 Form 10-K/A (accession 0000950170-25-030856) restates that same FY2023 with CFO −36,307 and PP&E purchase cash flow −17,352. Thus:

$$\text{FCF}_{\text{restated}} = -36{,}307 - 17{,}352 = -53{,}659 \text{ USD thousands.}$$

Under this defined formula, FCF happens to be unchanged: the 12,685 downward CFO revision offsets the 12,685 reduction in CapEx expenditure. Spire's restatement materials explain how contract revenue and cost recognition, leases, customer notes, and other accounting matters affect multiple periods and statements. Two cash-flow lines alone miss those version changes. This single case cannot establish that other restatements must offset. Original 10-K, `10-K/A`

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I3 reader package root, set up the environment using its instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch12_statements.ipynb
```

The Notebook calls the packaged shared implementation. Chapter 12 recalculates fixed transcriptions of Apple/Spire statement facts; PIT clocks and market capitalization are constructed. By default it displays the two existing packaged figures without redrawing them. The runner writes new kernel execution records to a unique `runs/execution_*` directory; the Notebook separately creates a unique UUID chapter-method directory and prints its location. These directories serve different purposes, and existing figures are not new outputs from this batch. French data absent from the package are neither read by default nor downloaded. The CLI and old production tests below refer to the working-course root. Readers use the Notebook assertions above and need not restore unbundled production tests. The English Notebook Markdown layer

remains pending BIL23 integration. This batch executed neither Notebooks nor old numerical suites.

Historical Working-Course Root and Original Implementation

Core metric and version functions are in `statement_metrics.py`; CLI and plotting entry points are in `ch12_statements.py`; program tests are in `test_ch12_statements.py`; and the teaching Notebook is `ch12_statements.ipynb`. The core module records each FinancialFact's entity, period, currency, unit, accession, revision status, and source. FCF requires CFO and the PP&E expenditure line from the same version and rejects positive "outflow" inputs. Feature functions also check unit/version mismatches, nonfinite values, basic statement reconciliations, and CapEx signs.

Run from the project root:

```
.venv/bin/python code/ch12_statements.py
.venv/bin/python -m pytest tests/test_ch12_statements.py -q
```

The code neither downloads statements nor connects to market quotes. Apple and Spire inputs are fixed transcriptions of selected source figures. Running the example generates the two chapter figures. The following call can run independently in the project virtual environment:

```
import math
import sys

from pathlib import Path

sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.statement_metrics import (
    apple_facts,
    derive_accounting_factors,
    fcf_from_statement_lines,
    fcf_yield,
    spire_cash_flow_facts,
)

apple_2024 = derive_accounting_factors(apple_facts(2024))
assert apple_2024["fcf"] == 108_807.0
constructed_yield = fcf_yield(
    apple_2024["fcf"],
    4_000_000.0,
    currency="USD",
    statement_unit="USD millions",
    market_cap_currency="USD",
    market_cap_unit="USD millions",
    market_cap_label="constructed teaching value",
)
assert math.isclose(constructed_yield, 0.02720175, rel_tol=0, abs_tol=1e-8)

for version in ("original", "restated"):
    cfo, capex_cash_flow = spire_cash_flow_facts(version)
    print(version, fcf_from_statement_lines(cfo, capex_cash_flow))
```

Reporting frequency above is annual; FY2024/FY2023 follow Apple's fiscal years, not calendar years. A date-only financial signal timestamp cannot establish intraday availability; actual receipt and ingestion times still require external logs. The PIT helper uses explicitly SIM-labeled constructed UTC times and selects only versions whose available_at and ingested_at do not exceed signal_time. Trading must follow information arrival and strategy processing, with execution latency handled as in Chapters 3 and 29. This chapter estimates no return interval.

This chapter's `FinancialFact` is a minimal teaching structure for these transcriptions. It labels fiscal years with `fiscal_period` but does not explicitly distinguish period flows from closing-date stocks or separately store `period_start`, `period_end`, or `as_of_date`. Production panels should preserve flow start/end dates, stock closing/effective instants, and `fact_period_type`. This lightweight class is not a complete financial-data schema.

10. Visualization

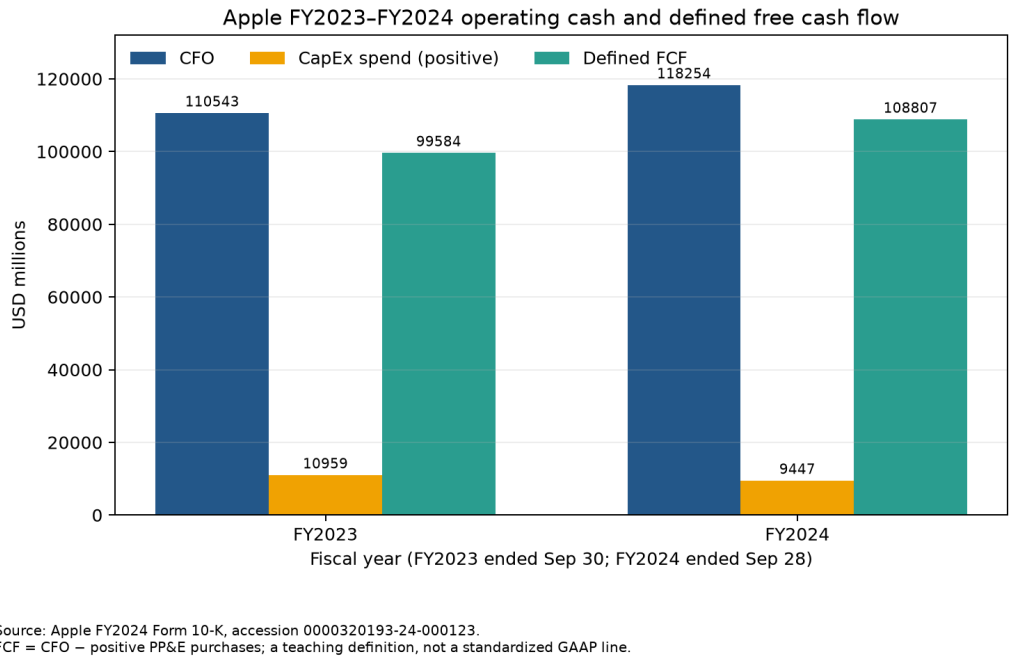
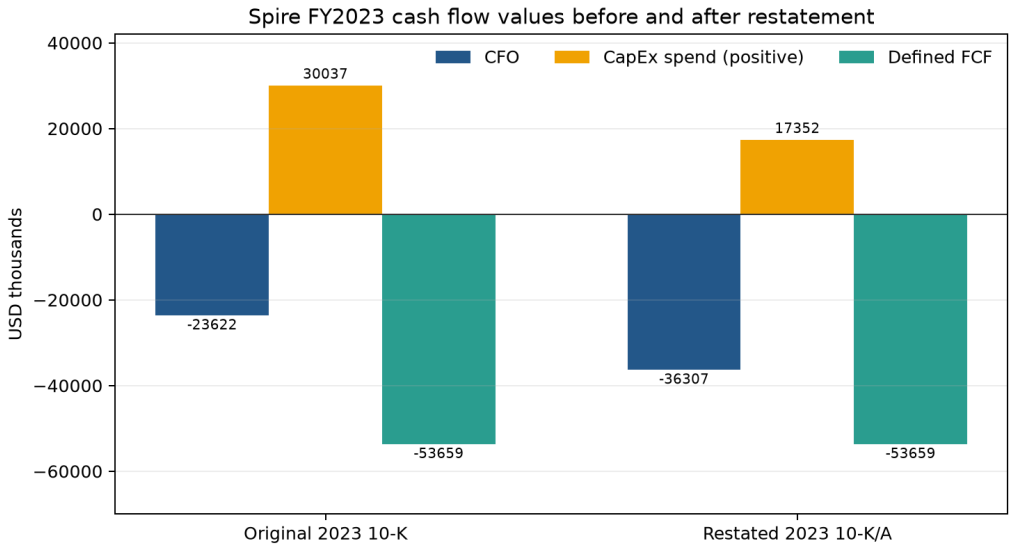


Figure 1: Apple cash flows and chapter-defined FCF for two fiscal years

The Apple FY2023/FY2024 CFO, CapEx, and FCF figure uses fiscal years horizontally and USD millions vertically. Blue bars show CFO; orange bars convert negative PP&E purchase cash flows into positive CapEx expenditure; green bars calculate CFO−CapEx. This displays two reporting years and their units, not stock prices or factor returns. FY2024 CFO and defined FCF are higher, while CapEx expenditure is lower. The chart alone does not explain why.

The Spire FY2023 original-versus-restated figure fixes the reporting period, changes document version horizontally, and uses USD thousands vertically. Blue CFO and green FCF lie below zero; orange CapEx is positive expenditure. Restated CFO is more negative and CapEx expenditure smaller, while chapter-defined FCF remains −53,659. Check accession numbers and signs before discussing whether a measure changed.



Sources: original 10-K accession 0000950170-24-027164; amended 10-K/A accession 0000950170-25-030856.
CapEx is plotted as positive spend; FCF = CFO − CapEx. Values are USD thousands.

Figure 2: Two statement versions for the same Spire fiscal year

Figure text identifies original documents, versions, and this chapter's FCF formula. Real figures come only from the relevant SEC statements; they contain no actual stock prices, vendor timestamps, or investment performance.

11. Real Market Example

This section uses actual public statements to test limited claims. These are neither complete security-level PIT data nor empirical strategy evidence.

Apple FY2024 10-K headers list period of report 2024-09-28, filed as of date 2024-11-01, and raw acceptance 20241101060136. Apple's newsroom dates the fourth-quarter/full-year FY2024 announcement 2024-10-31. The filing presents adjacent audited FY2024/FY2023 statements. Under the specified lines and definition, FCF is 108,807 and 99,584 USD millions. This establishes that the document version follows period-end and permits recalculation of two fiscal years. These pages alone do not establish the release's precise publication time, when a vendor received the full text, or when all 10-K notes became market-visible that day.

Spire's original FY2023 10-K has period-end 2023-12-31, accession 0000950170-24-027164, and filed date 2024-03-06. A subsequent Form 8-K states that certain issued statements should no longer be relied upon and require restatement. Its headers distinguish report date 2024-08-21, filed date 2024-08-27, and raw acceptance 20240827161154. On 2025-03-03 the company filed 10-K/A accession 0000950170-25-030856 with restated FY2023 statements. The three document versions remain separate; 2025 numbers do not replace original features generated in March 2024.

The company IR page provides a FY2023 results-announcement date of 2024-03-05, but its precise publication time zone was not verified here. Neither 10-K acceptance field is converted to UTC. These companies provide statement/version facts, without historical vendor ingestion logs, exchange calendars, security prices at trading instants, historical shares outstanding, delisted securities, or complete permanent-ID mappings required by this research. Thus no historical Apple or Spire FCF yield, validated PIT factor, or actual market regression is reported.

SEC companyfacts and frames facilitate XBRL retrieval. Frames organizes recent matching filing facts around a specified calendar period, while corporate fiscal years may not align. Current API results do not automatically replace historical snapshots preserving original accession numbers and version chains. SEC EDGAR API documentation

12. Common Mistakes

Treating report period-end as availability. Putting Spire's December 31 figures into the December 31 closing signal is wrong. Period-end describes measurement; first publication requires release records and research receipt requires ingestion logs. Preserve separate time fields and filter at a verifiable signal cutoff.

Collapsing filed date, acceptance, and ingestion into one timestamp. SEC accession headers show different metadata for period, filing date, and raw acceptance, without recording local database receipt. Preserve each raw field, its time-zone status, and system receipt logs; mark missing information unknown.

Pitfall: Negative Cash-Flow Lines versus Positive Expenditure

Subtracting a negative CapEx line again. Apple FY2024 PP&E cash flow is −9,447, while CapEx expenditure is +9,447. The wrong calculation is $118{,}254-(-9{,}447)=127{,}701$; the correct one is $118{,}254-9{,}447=108{,}807$. For inputs recorded as negative outflows, this is equivalently CFO plus that negative line.

```
cfo = 118_254.0
reported_ppe_cash_flow = -9_447.0
wrong = cfo - reported_ppe_cash_flow       # 127,701, sign error
capex_spend = -reported_ppe_cash_flow
correct = cfo - capex_spend                # 108,807
assert wrong == 127_701.0
assert correct == 108_807.0
```

Deducting the same working-capital changes again after CFO. The teaching company's CFO of 6 already includes AR, inventory, and AP movements. FCF is $6-8=-2$. Deducting working-capital growth 20 again gives −22, double-counting the cash use. To estimate CFO excluding a particular working-capital change, identify that same item's cash contribution in the full reconciliation and specify its removal or add-back. Do not automatically adjust again using a different balance-sheet proxy.

Calling FCF a uniform GAAP line or freely spendable cash. This formula deducts only selected PP&E CapEx from CFO, not principal repayments, dividends, acquisitions, lease payments, or other potentially necessary investment. Different research may use different CapEx coverage: preserve formula versions and included line names.

Allowing later restatements to rewrite past samples. CFO −36,307 in Spire's 2025 10-K/A and original CFO −23,622 are different versions. Using the 2025 value for an early-2024 signal embeds information absent then. Filter version metadata with both clocks and preserve each accession.

Treating frames output as calendar-year data or a historical vintage. Fiscal year-end may differ from December; frames selects recent matching facts for a calendar period. Even after reading the SEC API documentation, check original filings and period contexts. A recent API response is not proof of what a system actually saw on a past date.

Treating company facts directly as stock signals. A company may have common stock, multiple share classes, ADRs, predecessor entities, or delisted securities. CIK and ticker alone do not map every field to a tradable share class. Build a historical issuer/security mapping with permanent identifiers and specify per-share or equity-level denominators.

13. Quant Research Application

Start with a falsifiable hypothesis, such as: "Do companies with high CFO margin, measured using aligned fiscal years and public versions, subsequently have different cross-sectional returns from low-CFO-margin companies?" This remains a research question; two company cases cannot answer it.

First define the universe and measurement. Choose CFO/Revenue, CFO/average assets, or CFO/Net Income; specify statement lines, matched currency/unit, fiscal periods, revenue-recognition versions, and missing-data treatment. For FCF, define the CapEx line and sign; for NetDebt/EBITDA, specify debt accounts and whether EBITDA is disclosed or a research proxy. Define null/group handling for nonpositive numerators or denominators.

Next build source and version tables. Link every raw fact to permanent issuer ID, period, accession, version status, document line, first verified publication time, SEC filed date, parsable accepted field, system ingestion time, and cleaning-code version. Do not label current companyfacts retrieval as a historical snapshot. Preserve custom XBRL tags, units, period contexts, and normalization rules. Resolve duplicate facts by duration/instant, currency, dimensions, and revision source.

Third, select PIT versions. Both available_at and ingested_at must be no later than the signal cutoff; choose the latest-published eligible version. Date-only timestamps require more conservative date-level cutoffs with explicit assumptions. Without market time zones and trading calendars, claim no intraday availability. Only then map issuer fields to securities, share classes, and markets existing at that time.

Fourth, ensure comparability. Compute revenue growth for aligned fiscal cycles, consolidation scopes, currencies, and version snapshots. Specify ending or period-average stock denominators. Preserve event flags for acquisitions, spin-offs, accounting-policy changes, currency conversion, and restatements. Distinguish not applicable, unreported, source unavailable, unpublished, and parsing failure instead of filling all with zero.

Fifth, define the research sample and baseline. Use the historical company universe knowable then, recording delistings and identifier changes; survivorship filters may alter the cross-section. Compare the feature with a simple baseline excluding it. Preregister winsorization, industry adjustment, standardization, holding period, and trading lag. Use chronological out-of-sample evaluation and results after transaction costs rather than choosing the best full-sample transformation.

Finally, seek counterevidence. If an advantage appears only in restated versions, requires zero-filled missing values, depends on mismatched fiscal years or excluding delisted firms, or vanishes at an executable post-announcement time, report those measurement and implementation boundaries. Without real historical data and complete PIT security mappings, the work establishes statement arithmetic and engineering demonstrations, not empirical factor results.

14. Chapter Summary

The three statements describe period earnings, ending resources and obligations, and cash sources and uses. Net income, operating accounts, depreciation, CapEx, and financing connect them. Before turning these items into a factor, specify numerator, denominator, period, entity, currency, and unit, preserving original line names and formulas.

CFO and net income measure different outcomes. Convert CapEx cash-flow signs consistently; FCF depends on the declared CapEx scope. Do not deduct a working-capital proxy again when CFO already includes the same change. Revenue margins, cash conversion, and debt ratios are comparable only with matched definitions, periods, and shareholder entities.

Historical statements also have versions. Period-end, first publication, filing date, accepted field, revision announcement, and ingestion are distinct time information. A version may generate a signal only after it is public and in the research system. Later versions belong at their actual future availability times.

15. Exercises

Reference answers to conceptual questions are in Section 18.

12-E01 (Identify the statements) Credit sales increase revenue and receivables before customers pay. Which statements change first, and why might indirect CFO be below net income?

12-E02 (Levels of accounting profit) Distinguish EBIT, the EBITDA proxy, net income, and CFO in one sentence. Which may correspond directly to statement lines, and which require construction or definition checks?

12-E03 (Balance-sheet identity) A company has assets 500, liabilities 360, shareholder equity 130, and NCI 10. Does the identity hold? Can NCI 10 enter parent equity in a parent-shareholder feature?

12-E04 (Time fields) A FY2023 statement has period-end 2023-12-31, SEC filed date 2024-03-06, and local ingestion 2024-03-08. When can it first inform a strategy signal? What timing evidence is missing?

12-E05 (NWC and CFO) AR and inventory rise while AP is unchanged. What are their directions in indirect CFO? Why is deducting those same changes again after CFO wrong?

12-E06 (Shareholder scope) Consolidated net income is 100, including 15 attributable to NCI. For parent common-shareholder profitability, should the simple net-margin numerator be 100 or 85? Explain which definitions the numerator's entity must match.

Show formulas, variables, and units in hand calculations.

12-E07 (Three-statement links) Using the same fictional company from Sections 3 and 8 (all amounts in constructed unit u), calculate gross profit, EBIT, net income, CFO, FCF, ending cash, and ending assets/liabilities/equity. Explain depreciation and credit sales at each step.

12-E08 (Actual Apple statements) Apple FY2024 CFO is 118,254, the PP&E cash-flow line −9,447, and revenue 391,035, all in USD millions. Calculate positive CapEx expenditure, FCF, FCF margin, and CFO margin.

12-E09 (Spire versions) Original CFO is −23,622 and the PP&E line −30,037; restated CFO is −36,307 and the PP&E line −17,352, in USD thousands. Calculate each FCF, identify changed items, and explain whether this establishes a uniform FCF definition.

16. Programming Exercises

Section 18 contains each programming exercise's inputs, error boundaries, and reference implementation. Code uses Python, pandas, and this chapter's module. Every demonstration timestamp is explicitly constructed.

12-P01 (Sign- and unit-safe FCF) Write a function taking CFO and a negative PP&E outflow line. Validate matching currency, unit, period, and accession, then return FCF. Acceptance: Apple FY2024 returns 108,807; changing CapEx to +9,447 or mismatching versions raises an exception.

12-P02 (Two-clock PIT) Test version selection with at least two constructed records for one company and fiscal year. The first is published before ingestion; the second is published and ingested after a signal. Add a signal between publication and ingestion. Require time-zone-aware timestamps and verify that late restatements do not overwrite early signals.

12-P03 (Ratios and provenance) Calculate FCF margin, operating margin, net debt/EBITDA proxy, and revenue growth from Apple FY2024 facts. Market capitalization may only be an explicitly constructed teaching input. Assert values and unit boundaries, including rejection of direct division of USD by USD millions.

17. Challenge Problems

Open questions can have multiple reasonable solutions, but answers must preserve assumptions, versions, and counterevidence.

12-C01 (Restatement diagnosis; prerequisite: accounting reconciliations) Spire restates CFO from −23,622 to −36,307 and PP&E cash expenditure from 30,037 to 17,352; net loss also changes. Chapter-defined FCF is unchanged. Give a bounded interpretation and list at least four statements or notes to check before attributing the changes to economic cash movements, reclassification, revenue recognition, capitalization, or other causes.

12-C02 (PIT factor-data design; prerequisite: Chapter 3 data engineering) Design a minimal auditable data model for cross-company annual FCF yield. Specify fields for raw facts, versions, availability, ingestion, issuer/security mapping, market-value sources, and feature derivation. Explain conservative treatment of report dates/filing acceptance without precise verified time zones, and give two counterevidence tests that could invalidate the factor.

18. Solutions / Solution Hints

Conceptual Answers

12-E01 The income statement and balance sheet change first. If recognition conditions are met, revenue rises, AR increases, and profit rolls into equity. Cash is uncollected, so indirect CFO starts from net income and deducts the AR increase as a cash use; CFO can be below net income. Recognition timing still depends on applicable standards and contractual performance facts.

12-E02 EBIT is profit before interest and income tax, often approximated by operating income; the EBITDA proxy adds D&A to EBIT; net income further reflects interest, taxes, and other gains/losses; CFO adjusts net income to operating cash. Operating income and net income are usually reported lines, whose names, entities, and presentation require original-statement checks. Researcher-constructed EBIT+D&A must be labeled a proxy; company adjusted EBITDA differs from this formula. CFO is a cash-flow statement item with statement-specific coverage.

12-E03 Liabilities plus equity are $360 + 130 + 10 = 500$, exactly equal to assets. NCI 10 is part of consolidated equity; parent equity is 130, excluding NCI. Profit and valuation must follow the same entity convention.

12-E04 These fields alone cannot establish the earliest usable instant. 2023-12-31 is period-end, 2024-03-06 is filed date, and local ingestion occurs only on 2024-03-08. Verify first public announcement or other available_at evidence. This research instance must at least wait for ingestion, so candidate signals require $\max(\text{available_at}, \text{ingested_at}) \leq \text{signal_time}$. Date-only times support only date-level conclusions.

12-E05 AR and inventory are operating assets whose increases normally consume CFO; unchanged AP provides no cash offset from AP movements. Indirect CFO already adjusts net income for these changes. Deducting them again double-counts the cash use.

12-E06 Consolidated net income 100 less NCI-attributable profit 15 gives parent-attributable net income 85. Only with the additional assumption of no preferred dividends or other distributions requiring deduction is 85 also earnings available to common shareholders. Otherwise bridge further from parent to common earnings. Under Section 6.3, parent-attributable net margin divides parent net income by consolidated revenue for the same group and period. No revenue is supplied here, so select the earnings numerator without reporting a numerical margin. There is no directly ownership-allocated "common-shareholder revenue" line. ROE pairs common earnings with common equity, specifying average or ending equity; P/E divides common market capitalization by common earnings. Consolidated net income 100 still describes the group but includes NCI profit 15.

Hand-Calculation Answers

12-E07 All amounts use constructed unit u. Revenue 100 less COGS 40 gives gross profit 60. Deduct cash operating expenses 25 and depreciation 10 for EBIT proxy 25, then interest 5 and tax 4 for net income 16. CFO starts at 16, adds noncash depreciation 10, deducts AR growth 20 and inventory growth 10, and adds AP growth 10, giving 6. CapEx expenditure is 8, so FCF $= 6-8 = -2$. Ending cash is $50+6-8+15 = 63$. Assets are cash 63 + AR 20 + inventory 30 + net PP&E $100 + 8 - 10 = 98 = 211$. Liabilities are AP 10 + debt 55 = 65. Equity is 130+net income 16 = 146, and 65+146=211. Credit sales create revenue and AR without current cash; depreciation reduces profit and equipment book value but is added back in CFO.

12-E08 The PP&E line is a negative cash flow; positive CapEx $= -(-9{,}447) = 9{,}447$ USD millions. FCF $= 118{,}254-9{,}447 = 108{,}807$ USD millions. FCF margin $= 108{,}807/391{,}035 \approx 27.8254\%$; CFO margin $= 118{,}254/391{,}035 \approx 30.2413\%$. Both ratios are dimensionless. This is the chapter's FCF definition, not a standardized Apple GAAP line.

12-E09 Original CapEx expenditure is 30,037, so FCF $= -23{,}622-30{,}037=-53{,}659$ USD thousands. Restated CapEx is 17,352, so FCF $= -36{,}307-17{,}352=-53{,}659$ USD thousands. CFO and CapEx each change by 12,685 in offsetting directions. This establishes only an offset under this definition, not uniform FCF definitions across companies or unchanged FCF in every restatement.

Programming Reference Implementations

12-P01 The following runs independently and explicitly preserves period, accession, currency, and unit. A positive CapEx outflow sign or mismatched version/period stops execution:

```
from dataclasses import dataclass
from math import isclose, isfinite

@dataclass(frozen=True)
class Line:
    value: float
    period: str
    accession: str
    currency: str
    unit: str
    name: str

def same_frame(*lines: Line) -> None:
    keys = {(x.period, x.accession, x.currency, x.unit) for x in lines}
    if len(keys) != 1:
        raise ValueError("period, filing version, currency, and unit must match")
    if any(not isfinite(x.value) for x in lines):
        raise ValueError("line values must be finite")

def fcf(cfo: Line, capex_cash_flow: Line) -> float:
    same_frame(cfo, capex_cash_flow)
    if cfo.name != "cfo" or capex_cash_flow.name != "capex_cash_flow":
        raise ValueError("wrong line item")
    if capex_cash_flow.value > 0:
```

```
        raise ValueError("CapEx cash-flow line must be a non-positive outflow")
    return cfo.value - (-capex_cash_flow.value)

cfo = Line(118_254.0, "FY2024", "0000320193-24-000123", "USD", "millions", "cfo")
capex = Line(
    -9_447.0,
    "FY2024",
    "0000320193-24-000123",
    "USD",
    "millions",
    "capex_cash_flow",
)
assert fcf(cfo, capex) == 108_807.0
assert isclose(fcf(cfo, capex), 108_807.0, rel_tol=0, abs_tol=1e-12)

try:
    fcf(
        cfo,
        Line(
            9_447.0,
            "FY2024",
            cfo.accession,
            "USD",
            "millions",
            "capex_cash_flow",
        ),
    )
except ValueError:
    pass
else:
    raise AssertionError("positive cash outflow sign should fail")

try:
    fcf(
        cfo,
        Line(
            -9_447.0,
            "FY2024",
            "RESTATED-ACCESSION",
            "USD",
            "millions",
            "capex_cash_flow",
        ),
    )
except ValueError:
    pass
else:
    raise AssertionError("mixed filing versions should fail")
```

12-P02 All dates and CFO values below are constructed; Z denotes explicitly specified UTC inputs. The project PIT join keeps the first version invisible at 14:15, after announcement but before ingestion. A restatement likewise cannot replace the original between its publication and ingestion:

```
import sys
from pathlib import Path

import pandas as pd

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.pit import pit_asof_join

signals = pd.DataFrame({
    "issuer": ["SIM-1"] * 4,
    "period": ["FY2023"] * 4,
    "signal_time": [
        "2024-01-10T14:15:00Z",
        "2024-01-10T14:45:00Z",
        "2024-03-20T12:05:00Z",
        "2024-03-20T12:15:00Z",
    ],
})
versions = pd.DataFrame({
    "issuer": ["SIM-1", "SIM-1"],
    "period": ["FY2023", "FY2023"],
    "available_at": ["2024-01-10T14:00:00Z", "2024-03-20T12:00:00Z"],
    "ingested_at": ["2024-01-10T14:30:00Z", "2024-03-20T12:10:00Z"],
    "revision": ["original", "restated"],
    "cfo": [20.0, 14.0],
})
for frame, column in (
    (signals, "signal_time"),
    (versions, "available_at"),
    (versions, "ingested_at"),
):
    frame[column] = pd.to_datetime(frame[column], utc=True)
joined = pit_asof_join(
    signals, versions,
    security_col="issuer", period_col="period",
    signal_time_col="signal_time",
)
assert pd.isna(joined.loc[0, "cfo"])
assert joined.loc[1, "revision"] == "original"
assert joined.loc[2, "revision"] == "original"
assert joined.loc[3, "revision"] == "restated"
```

12-P03 The chapter functions below validate Apple's source version. Market value is explicitly constructed. fcf_yield raises ValueError for currency or unit mismatches:

```
import math
import sys
from pathlib import Path

sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.statement_metrics import (
    apple_facts,
    derive_accounting_factors,
    fcf_yield,
    revenue_growth,
)

fy2024 = apple_facts(2024)
fy2023 = apple_facts(2023)
metrics = derive_accounting_factors(fy2024)
growth = revenue_growth(fy2024["revenue"], fy2023["revenue"])
assert metrics["fcf"] == 108_807.0
assert math.isclose(metrics["fcf_margin"], 108_807 / 391_035, abs_tol=1e-14)
assert math.isclose(metrics["operating_margin"], 123_216 / 391_035, abs_tol=1e-14)
assert math.isclose(growth, 391_035 / 383_285 - 1, abs_tol=1e-14)
assert math.isclose(
    metrics["net_debt_to_ebitda_proxy"], 76_686 / 134_661, abs_tol=1e-14
)

# Net debt and EBITDA use the same currency and unit; this ratio is dimensionless.

yield_demo = fcf_yield(
    metrics["fcf"], 4_000_000.0,
    currency="USD", statement_unit="USD millions",
    market_cap_currency="USD", market_cap_unit="USD millions",
    market_cap_label="constructed teaching market cap",
)
assert math.isclose(yield_demo, 108_807 / 4_000_000, abs_tol=1e-14)
try:
    fcf_yield(
        metrics["fcf"], 4_000_000.0,
        currency="USD", statement_unit="USD millions",
        market_cap_currency="USD", market_cap_unit="USD",
        market_cap_label="constructed but wrong scale",
    )
except ValueError:
    pass
else:
    raise AssertionError("unit mismatch should fail")
```

Challenge Reference Arguments

12-C01 The supported conclusion is that the listed CFO and PP&E restatements offset under this FCF definition. Net loss also changes from −63,956 to −77,558 USD thousands, demonstrating that profit and cash classification differ. Do not conclude that cash flow was unrevised, or that unchanged FCF implies unchanged enterprise value, shareholder returns, or all cash flows. Check (1) the 10-K/A explanatory note and restatement background; (2) original/restated consolidated income statements, balance sheets, and cash flow statements; (3) PP&E, customer contracts, and embedded-lease accounting notes; (4) period allocation of revenue recognition and contract assets/liabilities; (5) PP&E purchases/capitalization, depreciation, and noncash adjustments; and (6) 8-K non-reliance periods and first revision announcement. An acceptable answer preserves versions, units, cash-flow signs, and the possibility of simultaneous restatement across multiple lines and notes.

Scoring: distinguish checked figures from causal explanations, 2 points; recalculate both FCF versions, 2; propose at least four relevant evidence checks, 3; explain timing/version and extrapolation limits, 2; state what new evidence could change the interpretation, 1.

12-C02 A minimal auditable model needs at least six logical tables: issuer/security mapping, raw statement facts, document versions, public/submission-time events, vendor ingestion logs, and factor runs. Facts preserve CIK/issuer ID, permanent security ID, original/standardized tags, original line name, value, currency, XBRL unit, duration/instant, fiscal start/end, quarter/year, segment/dimension, accession, version status, and source location. Version events store first_public_at and precision/time zone, filed_date, acceptance_raw, revision_public_at and accession, ingested_at, vendor/batch, and content hash. Mapping stores security class, validity start/end, listing status, and source. Derivation stores signal_time, selected accession, rule/code version, raw-line keys, currency conversion, market-denominator source, executable time, and output units.

Date-only or unverified-zone fields remain date-level; do not invent midnight UTC. Use a conservative later signal date and separately verify trading calendars and data-supply terms. Two counterevidence tests are: (1) removing a later restatement must leave earlier factors unchanged, or the filter leaks; (2) moving ingested_at after signal_time must return the last then-available version or null, or the two-clock implementation is wrong. Additional tests can cover period/currency/unit mismatches, conflicting simultaneous versions, delisted securities, and multiple share classes.

Scoring: traceable raw and derived data, 3 points; correct two-clock and revision-chain preservation, 3; versioned/timed security identity and market value, 2; at least two executable counterevidence tests, 2.

19. Further Reading

- **Must Know — SEC investor materials and company filings:** trace a transaction through all three statements; compare Apple FY2024 and Spire FY2023 original/restated filings, recording accession, period, unit, version, and notes. Investor guides do not replace accounting standards.
- **Should Know — SEC EDGAR API and Non-GAAP C&DI 102.07:** understand companyfacts/frames versus company fiscal periods, checking aggregation and common FCF-definition boundaries.
- **Advanced — IAS 7 and IAS 1:** compare cash-flow classification, FX adjustments, and parent/NCI attribution. The cited public HTML editions are 2026 and 2024, respectively; this is not a review of the full standards.

20. References

This chapter does not modify shared sources.json or references.bib. The direct links below identify official originals. Reading scope, verification date, raw figures, and timing limits are recorded in `source_notes.md`.

1. SEC, How to Read a 10-K and Beginners' Guide to Financial Statements: statement functions and operating/investing/financing cash-flow classification, Sections 3, 5, and 6.
2. SEC, EDGAR API Documentation: companyfacts/frames and calendar versus fiscal periods, Sections 6.4, 11, and 13.
3. SEC, Non-GAAP Financial Measures C&DIs, Question 102.07: common FCF definitions and limits, Sections 5, 7, and 12.
4. Apple, FY2024 Form 10-K, accession headers, 2024-10-31 results release: periods, statement lines, units, and distinct disclosure-time fields, Sections 8, 10, and 11.
5. Spire, original FY2023 Form 10-K, original accession headers, 2024 Form 8-K, 2024 8-K headers, amended FY2023 Form 10-K/A, amended accession headers: original, revision notice, restatement, cash-flow lines, and accessions, Sections 6.4, 8, 10, 11, and 18.
6. IFRS Foundation, IAS 7, 2026 public HTML and IAS 1, 2024 public HTML: IAS 7.11/7.28 and IAS 1.81A/106(a), within the limited scope in Sections 5, 7, and 19.

Chapter 13 — Valuation for Quants

Start by aligning statement cash flows, common equity market capitalization, and capital claims before calculating valuation ratios or discounted values. Prerequisites: Chapters 4, 11, and 12. After this chapter, you can bridge enterprise value (EV) to common equity value, explain common valuation ratios, construct discounted cash flow (DCF) valuations with matched cash flows and discount rates, and turn them into candidate fundamental signals without look-ahead.

1. Learning Objectives

After completing this chapter, you should be able to:

1. Recalculate enterprise value from common market capitalization and explicitly defined cash, interest-bearing debt, preferred equity, noncontrolling interest (NCI), and nonoperating assets (NOA) at the same valuation date, then bridge back to common equity value.
2. Specify the numerators, denominators, units, periods, shareholder scopes, and comparability conditions of price-to-earnings (P/E), price-to-book (P/B), EV/EBITDA, EV/EBIT, earnings yield, and free cash flow (FCF) yield.
3. Identify cash-flow claims: pair free cash flow to firm (FCFF) with the weighted average cost of capital (WACC) and EV, and free cash flow to equity (FCFE) with the cost of equity and common equity value. Understand that CFO−CapEx is often a useful proxy without automatically equaling FCFF or FCFE.
4. Derive finite-horizon DCF, WACC, and Gordon growth terminal value (TV), explain their assumptions, and use sensitivity and terminal-value share to identify what drives the result.
5. Construct candidate cross-sectional signals using point-in-time (PIT) statement availability, actual ingestion, market-valuation times, and share classes, with rules for zero, negative, and missing denominators.

Completion requires individually reproducible hand calculations, same-chapter answers covering every question ID, core Python functions importable from a clean project environment, and explicit separation of constructed inputs from company disclosures in every example.

2. Why Quants Need This

Value research involves more than ranking ratios. Statements first describe accounting profit or cash flow for a period. Researchers then determine whose cash it is, which items are operating, and which version was available at signal time. Market data price different claims at a particular instant. Only then can a numerator and denominator support cross-company comparison. A scope error anywhere can mislabel financing differences, accounting differences, or future information as cheapness.

The chain is statement lines and periods → FCFF or FCFE → matched discount rate → EV or common value → comparable ratios → PIT cross-sectional signal → industry neutrality and out-of-sample tests. Each stage inherits earlier limits. For example, dividing consolidated

EBITDA by only parent common market capitalization omits creditors, preferred holders, and noncontrolling shareholders from the value denominator. A portfolio ranking built from that mismatch has no coherent economic interpretation.

This chapter teaches measurement, valuation relationships, and research design. A low multiple does not imply low risk or future excess returns; DCF is not observable truth. Without historical market quotes, we can verify disclosed statement arithmetic and demonstrate formulas with constructed prices. The latter are not actual market capitalization, actual valuation, or measured strategy performance.

3. Intuition

Imagine two software companies in the same industry. Each market capitalization prices common-equity claims. Company A has little debt and substantial cash; Company B has more debt, preferred shares, and NCI. To ask what common holders pay per unit of their earnings, P/E pairing common market capitalization with common earnings is a reasonable start. To ask what all operating-capital providers pay per unit of operating earnings before interest, tax, and depreciation, add debt, preferred equity, and NCI to common value, deduct cash and other nonoperating assets, and divide by EBITDA over the same consolidation scope. The multiples answer different questions.

Now consider cash flow. A factory's cash must fund working capital and CapEx before distribution to creditors and shareholders. FCFF is operating cash available to all capital providers before debt service; FCFE belongs to common equity after interest, taxes, and net borrowing adjustments. Discounting the first gives operating enterprise value; discounting the second directly gives common equity value. Discounting shareholder cash at an all-capital rate, or operating cash at an equity rate, mixes distinct claims.

Begin with three questions: whose value is in the numerator, whose cash flow or profit is in the denominator, which period/version does it cover, and at what instant is the market price observed? These connect accounting, capital structure, and information timing.

4. Mathematical Foundations

Represent an amount as the signed-value, currency, and unit triple `Amount`. For example, 600 USD millions means six hundred million US dollars. Price 60 USD/share times 10 million shares gives 600 USD millions. Before adding, subtracting, or dividing amounts, align currency, unit, and required conversions. Denote dimensionless multiples by `x` and yields by decimals or percentages.

Distinguish three time objects: income/cash-flow statement flows cover period $[p_0, p_1]$; balance-sheet amounts belong to closing date p_1; prices, market capitalization, and diluted shares belong to valuation instant s. Statement versions separately have `available_at`, `ingested_at`, and accession. At research cutoff `signal_time`, both the public source and local data must be ready. Order time `trade_time` follows signal formation; returns start only once holdings actually take effect.

Amounts cannot be paired merely because both have dollar dimensions. FY2024 flows and market capitalization cover different time objects, but trailing P/E can pair FY2024 or TTM earnings already public at signal time with signal-time market value. Explicitly identify the trailing convention and verify version availability. Balance-sheet cash, debt, and common book equity use the same known closing date and disclosure version.

5. Formal Definitions

5.1 Market Capitalization, Enterprise Value, and Common Equity Value

Equity market capitalization sums each relevant common share class's price times its corresponding shares outstanding at the specified date, after currency and unit alignment. With one class, $M_s = P_s Q_s$, where P_s is the valuation-time price in quote currency/share, Q_s that class's shares then outstanding, and M_s a currency amount. Handle American depositary receipt (ADR) ratios, dual classes, and convertible securities under the declared research convention; do not mix a share price with another class's earnings per share (EPS).

Definition and Scope: The Claim Bridge from EV to Common Equity

This chapter defines **enterprise value (EV)** with an explicit claim bridge. Interest-bearing debt (D), preferred equity (PREF), and NCI are non-common claims; cash (CASH) and separately identified nonoperating assets (NOA) are deducted in constructing operating EV. NCI represents outside ownership claims in consolidated subsidiaries:

$$EV_s = M_s + D_s + PREF_s + NCI_s - CASH_s - NOA_s.$$

Start with common market capitalization. Creditors, preferred holders, and subsidiary noncontrolling holders also claim resources, but their claims are absent from parent common capitalization. Adding them covers all capital providers for the corresponding operations. Cash and separately valued NOA are excluded from the matched operating profit or cash flow, so deduct them from operating value. $CASH_s$ includes only cash judged deductible under the research rules. Do not automatically deduct restricted cash or minimum operating cash. Deduct NOA_s only once; an asset already contributing to operating cash flow cannot also be treated as a separate nonoperating asset.

All right-hand items are currency amounts corresponding to the valuation time: M_s is total common market capitalization, D_s defined interest-bearing debt, $PREF_s$ preferred claims, NCI_s NCI value, $CASH_s$ cash/equivalents deductible under the rule above, and NOA_s separately treated NOA. Units must match. This is a declared teaching convention, not a universal vendor EV definition. Pension shortfalls, leases, associate investments, and restricted cash may require additional bridges.

Common equity value deducts non-common capital claims from EV and adds back cash and other nonoperating assets:

$$M_s = EV_s - D_s - PREF_s - NCI_s + CASH_s + NOA_s.$$

This value belongs to common shareholders. Book equity is not market value and cannot substitute for it. With consolidated operating profit in the EV multiple denominator, retain

Continued

other capital-provider claims in the bridge. A consolidated, non-wholly-owned subsidiary contributes all its operating results to consolidated EBITDA, so add the corresponding external NCI value to EV, not to P/E earnings or common capitalization. Consolidation depends first on control and applicable-standard exceptions, not a single ownership percentage; joint control is not synonymous with non-wholly-owned consolidation. For associates or joint ventures accounted for using the equity method or otherwise not line-by-line consolidated, separately identify the parent-held investment value and any investment earnings already in the denominator, following the asset-scope bridge in 13-C01. Partial ownership does not automatically mean adding NCI. Valuation NCI need not equal book NCI; disclose any book-value approximation.

5.2 Equity Ratios and Earnings Yield

Price-to-earnings (P/E) divides common market capitalization by net income attributable to common shareholders, NI^{common}:

$$P/E_s = \frac{M_s}{NI^{common}_{p,v}}.$$

$NI^{common}_{p,v}$ is the company's net income attributable to that common-equity scope in period p and version v, with matched currency and unit. Conventional P/E requires positive earnings. Negative earnings are not comparable, commonly labeled N/M; zero earnings are undefined. A negative arithmetic P/E below positive P/E does not make a loss-maker cheaper.

Price-to-book (P/B) divides common capitalization by common book equity, BV^{common}: $P/B_s = M_s/BV^{common}_{p_1,v}$. Book equity belongs to the same issuer's common holders, closing date p_1, and statement version v. Nonpositive common book equity prevents conventional mechanical P/B ranking; preserve the negative-equity state and missingness reason.

Earnings yield is $NI^{common}_{p,v}/M_s$. It is the reciprocal of P/E only with positive net income and identical definitions. Negative net income produces negative yield, expressing the loss direction rather than an attractive positive-return signal.

5.3 Enterprise Multiples and Cash-Flow Yields

EV/EBITDA divides EV by EBITDA for the same period and consolidation scope; **EV/EBIT** uses EBIT. EBITDA is generally not a uniquely standardized statement line: record original lines, adjustments, and company/research definitions. These operating denominators require EV to cover corresponding debt, preferred, and NCI claims. Conventional multiples with nonpositive EBIT or EBITDA are excluded from cross-sectional ranking.

The term **free cash flow (FCF)** has many definitions. Distinguish three here:

- **Free cash flow to firm (FCFF):** convert EBIT into after-tax operating profit using operating tax rate T: $FCFF = EBIT(1-T)+D\&A-CapEx-\Delta NWC+Adj$. Depreciation and amortization (D&A) are noncash expenses; CapEx is positive expenditure; an increase ΔNWC in net operating working capital (NWC) consumes cash; Adj contains explicit other operating cash adjustments. All amounts share currency, unit, and forecast period.
- **Free cash flow to equity (FCFE):** initially restrict the simplified formula to common equity and debt only, with all items covering the same common-equity operating scope. $FCFE = NI^{common} + D\&A - CapEx - \Delta NWC + NetBorrowing + Adj$. NI^{common} is common-attributable net income. Net borrowing equals new debt less repayments; a positive amount supplies cash to equity.
- **CFO minus CapEx:** $CFO - CapEx$ is a common cash-flow proxy. Label its precise research definition or proxy status; do not automatically rename it FCFF.

All these cash flows have currency/period units. Signs matter: PP&E purchases usually appear as negative statement cash flows; formula $CapEx$ is positive expenditure, so align signs before subtracting it from CFO or operating cash flow. SEC staff describes CFO minus capital expenditures as common but nonuniform FCF and cautions that it is not discretionary cash after all debt service and nondiscretionary spending. SEC C&DI 102.07

FCF yield must identify its claim: $FCFE/M_s$ is common-equity FCF yield, and $FCFF/EV$ enterprise FCF yield. If a data product divides $CFO - CapEx$ by market capitalization, label it an equity-yield proxy with that explicit formula; do not directly compare it with FCFF/EV. Preserve and separately flag negative FCF yields. A nonpositive value denominator M_s or EV invalidates the yield.

5.4 DCF, WACC, and Terminal Value

Discounted cash flow (DCF) converts expected future cash flows to present value at the valuation date. With annual year-end flows:

$$V_0 = \sum_{t=1}^{N} \frac{CF_t}{(1+r)^t} + \frac{TV_N}{(1+r)^N}.$$

CF_t is a currency amount at the end of year t; r is the annual effective discount rate matching the cash-flow risk and claim; N is the dimensionless explicit forecast horizon; and TV_N is continuing value at year-end N. Match cash flows, TV, currencies, real/nominal conventions, and rates. Semiannual or monthly compounding requires corresponding intervals and discount factors.

The simplified common-equity/debt **weighted average cost of capital (WACC)** is:

$$WACC = \frac{E}{D+E}k_e + \frac{D}{D+E}k_d(1-T).$$

Weights use market values because the rate should reflect relative claim values at the valuation date. Historical book equity cannot directly replace market weights. If cash flows cover

all consolidated subsidiary operations, parent common capitalization alone omits NCI. A broader form is:

$$WACC = \frac{E_P k_e + NCI\, k_{NCI} + PREF\, k_p + D\, k_d(1-T)}{V_{cap}}, \qquad V_{cap} = E_P + NCI + PREF + D.$$

Amounts are valuation-time market values: E_P parent common equity, NCI NCI in that operating scope, $PREF$ preferred equity, and D included financing debt. Each k is the corresponding annual effective capital cost. Combine $E_P + NCI$ into E only under the constructed assumption that parent and NCI equity have the same risk; preferred equity retains separate weight and cost. The formula assumes V_{cap}, total gross financing claims, represents financing weights for the operating risk. Capital costs must match that risk and weight scope. Cash and NOA are deducted from operating EV, but under this assumption not from V_{cap}. Substantial lower-risk nonoperating cash can dilute observed common-equity beta and required return; cash is not necessarily risk-free, and NOA is not synonymous with cash. For pure operating FCFF, remove nonoperating-asset risk or consistently adjust costs, weights, and value scope. Do not directly combine observed whole-company equity cost with gross weights. If book debt approximates unavailable debt market value, disclose the bias assumption rather than claiming actual market weights.

See Damodaran, DCF Inputs for market weights, preferred costs, and matching rates to cash-flow risk, and Damodaran, What Is Debt? for gross/net debt and cash conventions. Gross-debt and net-debt routes handle cash risk and weights differently. Apply the selected route consistently to EV, weights, capital costs, and operating cash flows; deducting cash in EV alone does not align costs. These materials provide frameworks, not a unique issuer-level treatment of cash, NOA, NCI, or leases. Actual valuation must match operating risk, costs, and value-bridge scope.

E, D are simplified market weights when no preferred, NCI, or other separate claims exist; k_e, k_d are annual effective equity and pretax debt costs in consistent currency/tenor terms; T approximates a sustainable marginal tax rate. Target leverage, pensions, or lease debt require corresponding cash-flow and value-bridge adjustments. WACC debt must use the same lease treatment as EV and cash flows; see Section 7.2. WACC is dimensionless.

Discount FCFF at WACC to obtain operating enterprise value, then apply the claim bridge. Discount FCFE at the equity cost to obtain common equity value. An explicitly assumed capital asset pricing model (CAPM) can specify equity cost, for example $k_e = R_f + \beta ERP$, with risk-free rate R_f, market-risk exposure β, and equity risk premium (ERP), compensation for stock-market risk. Match rate frequencies and real/nominal terms. Richer risk models are possible, but convenience does not justify swapping discount rates.

Conditions: Mathematical and Economic Scope of Stable-Growth TV

Under stable growth, **terminal value (TV)** compresses perpetual post-forecast cash flows into a value at year-end N:

$$TV_N = \frac{CF_{N+1}}{r-g} = \frac{CF_N(1+g)}{r-g}, \quad r > g.$$

For enterprise valuation, CF is FCFF and $r = WACC$; for equity valuation, CF is FCFE and $r = k_e$. g is perpetual growth matched to currency and inflation convention; r, g are

Continued

dimensionless rates for the same interval. Annual cash flow divided by an annual rate gives currency value. This growth model also requires $r > -1$ and $g > -1$ for positive discount and growth factors; only then does $r > g$ ensure geometric-series convergence. These mathematical conditions do not themselves require positive CF_N. A going-concern interpretation additionally needs an explicit normalized stable-period base CF_N and a sustainable cash-flow path. A numerical formula output from negative or unstable forecasts does not establish credible mature-stage value. Nor do these conditions validate forecasts, competition, risk, or terminal-value share. Mature reinvestment, returns, leverage, margins, and growth must be jointly sustainable.

6. Derivations

6.1 From Market Capitalization to Comparable Multiples

If the market pays M_s for common equity while operating cash also serves creditors, preferred holders, and NCI, add those claims to cover all operating-capital holders. Deduct idle cash and separately identified NOA from required operating capital value. EV additions and deductions therefore define claim and asset boundaries rather than an arbitrary adjustment list. Rearrangement gives the reverse bridge: $M_s = EV - D - PREF - NCI + CASH + NOA$.

Next, match each multiple's numerator and denominator at the same claim level:

- Common market capitalization pairs with common earnings or common book equity for P/E or P/B.
- EV pairs with consolidated operating income or pre-depreciation operating income for EV/EBIT or EV/EBITDA.
- Common FCFE pairs with market capitalization, and firm FCFF with EV, for the corresponding FCF yields.

This is more than memorizing categories. The EV bridge reveals omitted capital providers; identifying the cash-flow claim determines the discount rate and valuation object.

6.2 From Accounting Cash Flow to FCFF and FCFE

Define net operating profit after tax, $NOPAT$, as $NOPAT = EBIT(1 - T)$: after-tax operating profit before financing effects. D&A reduces accounting EBIT without another current cash payment, so add $D\&A$. CapEx consumes cash, so deduct $CapEx$; operating working-capital growth consumes cash, so deduct ΔNWC. Thus:

$$FCFF = NOPAT + D\&A - CapEx - \Delta NWC + Adj.$$

This starting point precedes interest and principal payments and belongs to all financing providers. Common-attributable net income already reflects interest and tax. Add noncash D&A, deduct capital investment and working-capital growth, and add net borrowing to obtain

common-equity cash:

$$FCFE = NI^{common} + D\&A - CapEx - \Delta NWC + NetBorrowing + Adj.$$

This simplified FCFE formula and the bridge below assume only common equity and debt, the same operating scope, a realizable interest tax shield, and consistent other accounting adjustments. With preferred equity or NCI, combining parent common net income with whole-group D&A and CapEx mixes claims. Either build all-equity cash over a consistent scope and deduct preferred/NCI entitlements, or construct parent common cash item by item. Deducting NCI only in the ending EV bridge does not correct period FCFE. Under the simple debt-tax-shield assumption, $FCFE = FCFF - k_d D(1-T) + NetBorrowing$. Interest and principal can put FCFE below FCFF; new borrowing can temporarily lift it above the operating contribution. Major leverage changes complicate direct FCFE forecasts. The FCFF/WACC route can value operations first and then bridge through valuation-date debt, NCI, and other claims.

However, $CFO - CapEx$ usually starts from reported CFO after interest, depending on classification, and cash taxes. It may include exchange effects, noncash adjustments, interest classifications, and working-capital effects, without adjusting new borrowing or net capitalized items. It is useful as a statement feature; calling it FCFF or FCFE requires checking the accounting mapping and making necessary adjustments.

Worked Example: The Itemized Bridge from CFO to FCFF and FCFE

Use constructed inputs for a company with only common equity and debt to bridge an annual CFO proxy to FCFF. Assume cash interest is in CFO, cash taxes equal the tax rate times ($EBIT$ − cash interest), the interest shield is usable, and there are no nonoperating gains/losses. D&A and ΔNWC cover the same operations and year and are already fully included in CFO: EBIT 50, tax rate 25%, cash interest 8, D&A 10, positive CapEx 15, $\Delta NWC = 5$, and net borrowing 3, all in USD millions. Simplified net income is $(50-8)(1-0.25) = 31.5$, so $CFO = 31.5 + 10 - 5 = 36.5$.

Itemized CFO-to-FCFF bridge	USD millions
Reported CFO (already includes D&A add-back and ΔNWC cash effect)	36.5
Add cash interest already deducted in CFO	+8.0
Deduct the interest cash tax shield $8 \times 25\%$	−2.0
Unlevered operating cash flow	42.5
Deduct positive CapEx expenditure	−15.0
FCFF	27.5

Starting directly from EBIT also gives $50(1-0.25) + 10 - 15 - 5 = 27.5$. Do not add D&A or deduct ΔNWC again after CFO; both are already included. The simple proxy $CFO - CapEx = 36.5 - 15 = 21.5$ is below FCFF by after-tax interest $8(1-0.25) = 6$ and has not unlevered operating cash. For the same company's FCFE, retain common net income and net borrowing: $31.5 + 10 - 15 - 5 + 3 = 24.5$. Real statements can differ in interest presentation, cash taxes, other operating adjustments, and leases. Reconcile items before deciding whether the inputs support this bridge.

6.3 From Cash Flows to Discounting and Terminal Value

Move a cash flow at the end of year t back one year by dividing by $1+r$; discount a year-two flow twice to obtain $CF_2/(1+r)^2$. Summing N explicit years gives finite-horizon present value. If stable cash flows follow $CF_{N+j} = CF_{N+1}(1+g)^{j-1}$, their value at year-end N is:

$$\begin{aligned} TV_N &= \sum_{j=1}^{\infty} \frac{CF_{N+1}(1+g)^{j-1}}{(1+r)^j} \\ &= \frac{CF_{N+1}}{1+r} \sum_{j=0}^{\infty} \left(\frac{1+g}{1+r}\right)^j \\ &= \frac{CF_{N+1}}{r-g}. \end{aligned}$$

A geometric series converges when its ratio has absolute value below 1. Here $r > -1$ and $g > -1$ make the ratio positive, so convergence simplifies to $\frac{1+g}{1+r} < 1$, or $r > g$. Checking r>g alone wrongly accepts inputs such as g=-300%, outside this growth model. TV is located at year-end N and must be divided by $(1+r)^N$ to reach V_0. Adding TV directly to today's explicit-flow present value omits N years of discounting.

7. Financial Interpretation

7.1 What Multiples Compare

P/E and P/B price common equity per unit of attributable earnings or book net assets. They retain leverage and nonoperating-asset effects in common value. P/E also reflects growth, earnings volatility, repurchases, unusual charges, taxes, and accounting policies. P/B may better relate to financial firms' asset/capital constraints, but book value can diverge greatly from operating capital for businesses with substantial uncapitalized R&D or brands.

EV/EBITDA aligns an additional layer of capital-structure claims beyond net debt and uses operating results before depreciation. Companies with different capital intensity can therefore show similar EV/EBITDA yet different sustainable FCF. EV/EBIT includes D&A in operating costs and exposes asset-consumption differences, but still deducts neither CapEx nor working capital. No multiple replaces checks of cash, reinvestment, and risk.

7.2 Align Taxes, Interest, Leases, and Periods

Construct FCFF from operating profit with operating cash-tax assumptions. Do not mix interest-inclusive net income with an incorrect after-tax-interest add-back to FCFE. FCFE begins with common earnings after financing costs. Multiplying WACC debt cost by $1-T$ assumes deductible interest. Align pretax/after-tax, real/nominal, and annual/quarterly cash-flow conventions.

Leases readily introduce accounting differences into EV/EBITDA or FCF ratios. Adding lease liabilities as EV debt requires checking rent, right-of-use depreciation, and lease principal/interest classifications in EBIT/EBITDA and cash flow. If rent remains an ongoing operating expense, do not add the same commitment again as EV debt. Teaching code does not automatically add lease liabilities: this is an explicit definition choice, not automatic precision.

7.3 Limits of TV and Quantitative Signals

Post-forecast TV often accounts for much of DCF. A high terminal share indicates sensitivity to stable-period assumptions, not correctness or cheapness. At minimum, check a grid of r and g, the transition to stable growth, terminal returns on capital versus capital costs, and reinvestment sufficient for growth.

Quantitative research usually compresses fundamentals into rankable indicators rather than certifying intrinsic value for one firm. Similar industry business/accounting models can improve comparability. Industry-neutral ranking still requires checks of size, leverage, earnings quality, liquidity, estimation risk, and factor exposures. Freeze factor definitions, availability, and execution lag before backtesting; test against out-of-sample data, delistings, revisions, and transaction costs. Every ranking remains a hypothesis.

8. Numerical Example

Worked Example: Orion and Lyra's EV Bridges and All Ratios

Both companies, prices, shares, statements, and market values in this section are constructed for teaching, in USD millions, not quotes, actual companies, or strategy performance. Assume one fictional signal cutoff, with FY2024 flows already public and ingested. Orion's constructed price is USD 60/share with 10 million diluted shares, giving market capitalization $60 \times 10 = 600$. Lyra's price is USD 24/share with 20 million shares, giving 480.

Constructed input/calculation	Orion	Lyra
Common market capitalization M	600	480
Interest-bearing debt D	100	260
Preferred equity / NCI	0 / 0	20 / 40
Cash / NOA	50 / 0	30 / 20
Common net income / common book equity	30 / 180	20 / 120
EBITDA / EBIT	75 / 50	75 / 50
FCFE / FCFF	28 / 40	18 / 40
EV bridge result	$600+100-50=650$	$480+260+20+40-30-20=750$

Continued: the same complete example

FCFF and FCFE are separately specified constructed cash flows already matched to firm and common claims, not inferred from the listed net income. Lyra has NCI and preferred equity: whole-group D&A/CapEx cannot simply adjust parent common net income to reconstruct FCFE. Reconstruction would require additional attribution/distribution adjustments.
Orion's enterprise multiples are $650/75 = 8.67x$ for EV/EBITDA and $650/50 = 13x$ for EV/EBIT. Common P/E is $600/30 = 20x$, P/B $600/180 = 3.33x$, and earnings yield $30/600 = 5\%$. Pair FCFE with market capitalization for equity yield $28/600 = 4.67\%$; pair FCFF with EV for enterprise yield $40/650 = 6.15\%$.
Lyra's EV/EBITDA is $750/75 = 10x$ and EV/EBIT $750/50 = 15x$; P/E is $480/20 = 24x$, P/B $480/120 = 4x$, and earnings yield $20/480 = 4.17\%$. Equity FCF yield is $18/480 = 3.75\%$ and enterprise FCF yield $40/750 = 5.33\%$. Omitting preferred/NCI, or adding debt without deducting cash/NOA, changes EV multiples without changing operating profit: a bridge error alone can manufacture ranking differences.

A hand-calculable WACC example also uses constructed amounts and assumes common/debt market weights and costs match the simplified operating risk: $E = 600, D = 100, k_e = 10\%, k_d = 6\%, T = 25\%$, giving

$$WACC = \frac{600}{700}(10\%) + \frac{100}{700}(6\%)(1 - 25\%) \approx 9.2143\%.$$

Worked Example: Lyra's Financing Weights and Operating EV

Lyra's constructed bridge illustrates EV versus WACC weight scope: $M = 480, D = 260$, $PREF = 20, NCI = 40, CASH = 30, NOA = 20$, so $EV = 480 + 260 + 20 + 40 - 30 - 20 = 750$, with reverse bridge $750 - 260 - 20 - 40 + 30 + 20 = 480$. Assume $k_e = k_{NCI} = 10\%$, $k_p = 7\%, k_d = 6\%, T = 25\%$. The capital-claim weighting base is $V_{cap} = 480 + 40 + 20 + 260 = 800$, and

$$WACC = \frac{480(10\%) + 40(10\%) + 20(7\%) + 260(6\%)(1 - 25\%)}{800} = \frac{65.1}{800} = 8.1375\%.$$

Here 800 is gross financing claims before cash/NOA deductions; 750 is operating EV after those deductions. They serve different calculations. This example assumes cash-risk adjustments are already reflected in the specified costs; it is not an actual company cost estimate.

Worked Example: Five-Year FCFF and Year-End TV

A separate constructed DCF uses year-end FCFF of 40, 45, 50, 55, and 60 USD millions in years 1–5, WACC 9%, and stable $g = 3\%$. Year-five TV is $60(1.03)/(0.09 - 0.03) = 1{,}030$ USD millions. Discounting all five flows and TV at year-end gives EV approximately 860.57. TV present value is $1{,}030/1.09^5 \approx 669.43$, about 77.79% of total EV. These figures reproduce the formula without validating g, WACC, or forecasts.

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I3 reader package root, set up its environment, then run:

```
.venv/bin/python run_notebook.py notebooks/ch13_valuation.ipynb
```

The Notebook calls the packaged shared implementation. Orion/Lyra, forecasts, capital costs, PIT clocks, and market values are constructed; the Apple section only checks statement lines. By default it displays two existing packaged figures without redrawing them. The runner writes new kernel execution records to a unique `runs/execution_*` directory; the Notebook separately creates a unique UUID chapter-method directory and prints its location. These directories have distinct purposes; existing figures are not new batch outputs. Unbundled French data are neither read by default nor downloaded. The CLI and old production tests below refer to the working-course root. Readers use the Notebook assertions above and need not restore missing production tests. English Notebook Markdown remains pending BIL23 integration. No Notebook or old numerical suite was executed in this batch.

Historical Working-Course Root and Original Implementation

Core functions are in valuation.py, entry points and plots in `ch13_valuation.py`, tests in `test_ch13_valuation.py`, and the teaching Notebook in `ch13_valuation.ipynb`. The module downloads no data. Amounts require explicit currency/unit; multiple functions return `None` for nonpositive denominators; FCF yield preserves cash-flow signs and checks claims; PIT selection checks both `available_at` and `ingested_at`.

The same core interfaces calculate the constructed cases, from price and million-share counts through market capitalization, EV, and multiples. From the project directory, run:

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import Amount, enterprise_value, price_to_earnings

market_cap = Amount(600.0, "USD", "USD millions")
debt = Amount(100.0, "USD", "USD millions")
cash = Amount(50.0, "USD", "USD millions")
net_income = Amount(30.0, "USD", "USD millions")
ev = enterprise_value(market_cap, debt, cash)
print(
    f"EV={ev.value:.0f} USD millions; "
    f"P/E={price_to_earnings(market_cap, net_income):.2f}x"
)
```

Examples combine annual statement flows with market amounts at one constructed signal instant. PIT demonstrations use constructed UTC times marked `Z`. A later revision cannot change an earlier signal even when it covers the same fiscal period. Market snapshots cannot exceed signal_time. Example trade_time follows signal_time; real execution additionally requires

trading calendars, matching delays, and order conventions. No actual return interval is calculated here.

10. Visualization

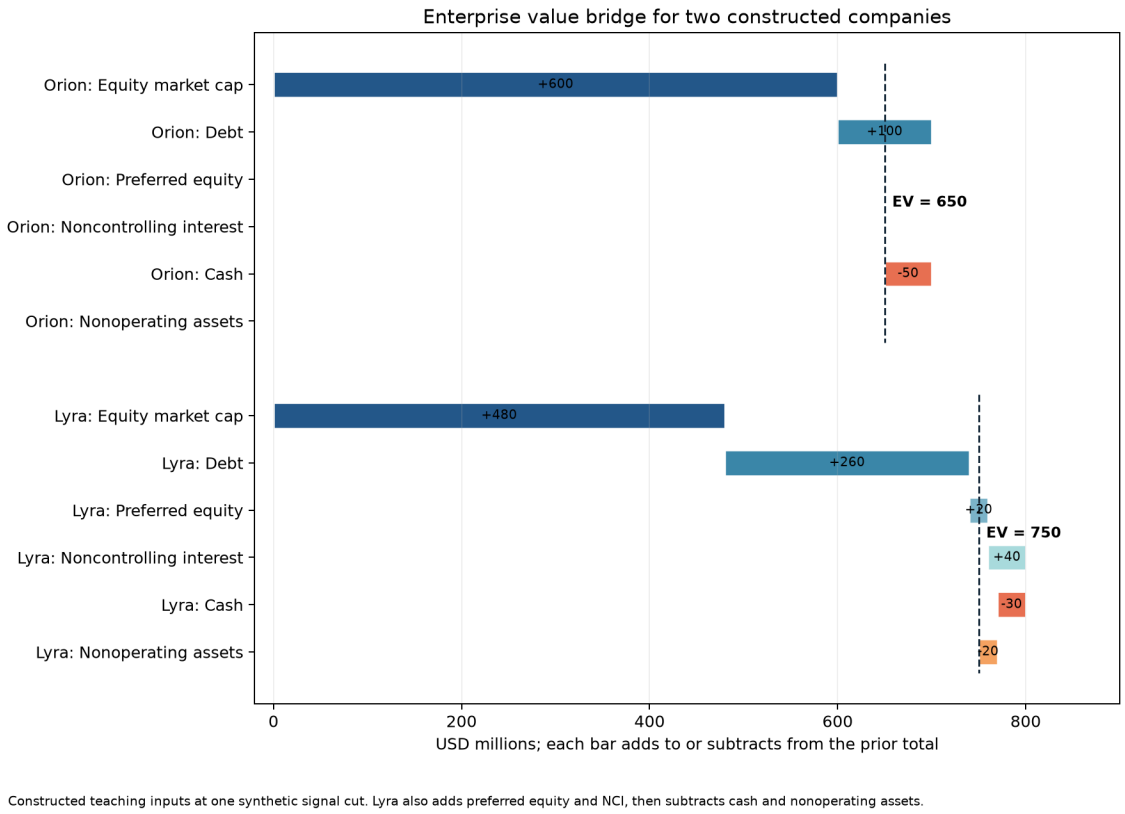


Figure 1: Enterprise-value claim bridges for constructed companies

The EV claim-bridge figure uses USD millions horizontally and constructed companies/bridge items vertically. Each bar adds to or deducts from the preceding cumulative value; dashed lines mark final EV. Lyra's preferred equity/NCI raise EV, while cash/NOA reduce it. Orion shows only market capitalization, debt, and cash.

The terminal-share sensitivity figure places annual FCFF WACC horizontally and annual perpetual growth g vertically. Colors and cell figures give TV present value as a percentage of total DCF EV. Inputs are constructed FCFF 40, 45, 50, 55, and 60 USD millions. Cells with $g \geq WACC$ are blank because the perpetual-growth formula does not converge. Terminal share rises near $g = WACC$, prompting scrutiny of stable-period assumptions rather than validating the model.

The CLI generates both figures from the listed constructed numbers, without market observations or random sampling; no random seed is needed. Axes and captions identify units and assumptions.

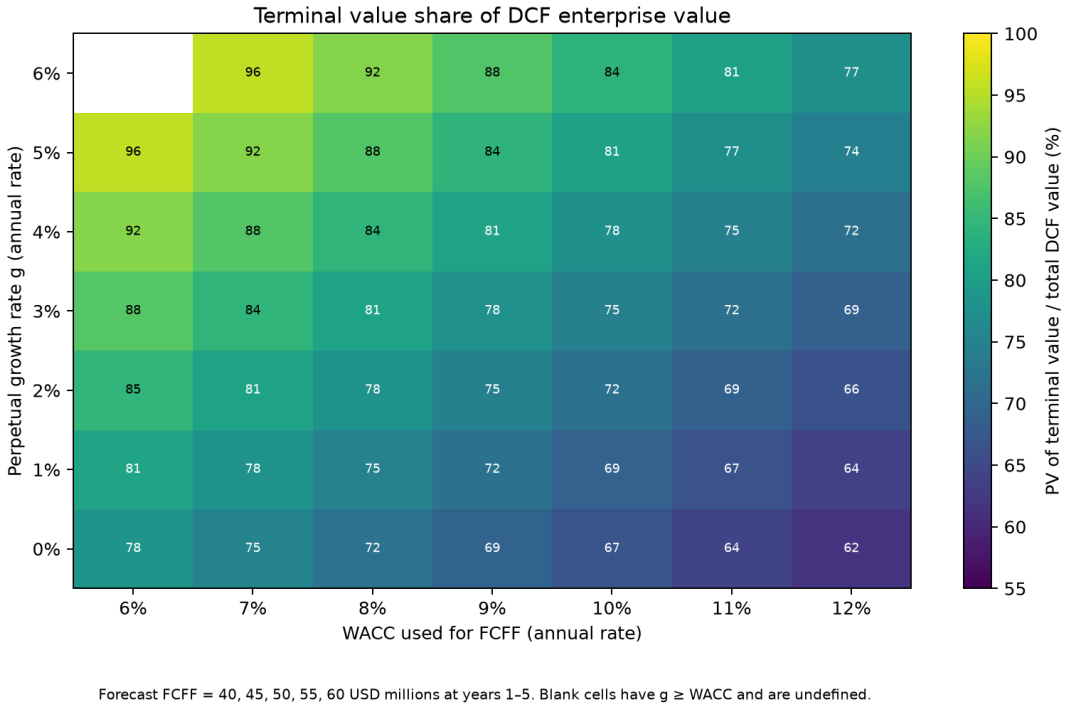


Figure 2: Sensitivity of constructed DCF terminal present-value share

11. Real Market Example

The real-source example verifies statement facts, not actual valuation or market performance. Apple FY2024 Form 10-K reports operating income 123,216 and net income 93,736 USD millions, diluted EPS USD 6.08, and weighted-average diluted shares of 15,408,095 thousand shares. FY2024 ends 2024-09-28; accession is `0000320193-24-000123`. These explain the EPS numerator/denominator without a verified matched price, security-class mapping, or historical capitalization snapshot. No actual Apple P/E, P/B, or FCF yield is calculated. Apple FY2024 Form 10-K

Issued and outstanding shares in Apple's filing differ from weighted-average diluted EPS shares. Balance-sheet shares are a closing-date stock; EPS shares are a fiscal-year weighted average. Market capitalization needs shares outstanding for the corresponding class at the valuation date. Price/EPS additionally requires the EPS period, basic/diluted choice, and share basis. Actual capitalization requires verified prices, all relevant common classes, ADR conversions, and trading times. These data were not downloaded, so there is no quote sample period, frequency, or file hash.

12. Common Mistakes

12.1 Dividing FCFE by EV

Dividing Orion FCFE 28 USD millions by EV 650 gives 4.31%, versus correct common FCF yield 28/600 = 4.67%. The mismatch inserts creditors and other claims into an FCFE denominator. Enterprise yield requires FCFF/EV instead. The core module rejects an explicit `cash_flow_basis="FCFE"` with `claim_basis="EV"` call.

```
fcfe = 28.0
equity_value = 600.0
enterprise_value_amount = 650.0
wrong_yield = fcfe / enterprise_value_amount
correct_yield = fcfe / equity_value
assert round(wrong_yield * 100, 2) == 4.31
assert round(correct_yield * 100, 2) == 4.67
```

12.2 Ranking Negative P/E as Cheapest

Arithmetic P/E for loss-makers is negative; a ranking algorithm may put the smallest negative number first among low valuations. The denominator sign changes interpretation. Mark P/E noncomparable and preserve a loss flag. Negative earnings yield can remain a separate feature with explicitly designed ranking and missingness rules.

```
pe = {"LossCo": 480.0 / -20.0, "ProfitCo": 600.0 / 30.0}
wrong_order = sorted(pe, key=pe.get)
def comparable_pe(market_cap: float, net_income: float) -> float | None:
    return market_cap / net_income if market_cap > 0 and net_income > 0 else None
safe = {
    name: comparable_pe(480.0, -20.0)
    if name == "LossCo"
    else comparable_pe(600.0, 30.0)
    for name in pe
}
assert wrong_order[0] == "LossCo"
assert safe["LossCo"] is None and safe["ProfitCo"] == 20.0
```

12.3 Automatically Calling (CFO-CapEx) FCFF

CFO often includes interest paid, cash taxes, and working-capital adjustments, while accounting standards may classify interest differently. FCFF uses after-tax operating profit less operating reinvestment. Check interest, taxes, and working capital for omissions or duplication. This code compares an accounting cash-flow proxy with operating-profit-based FCFF:

```
cfo = 44.0
capex = 15.0
reported_cfo_less_capex = cfo - capex
ebit, tax, da, change_nwc = 50.0, 0.25, 10.0, 5.0
fcff = ebit * (1 - tax) + da - capex - change_nwc
assert reported_cfo_less_capex == 29.0
```

```
assert fcff == 27.5
```

Different results alone do not establish a statement error. Trace interest, tax, noncash adjustments, working-capital definitions, capitalization, and versions.

12.4 Mismatching EV and the Operating Denominator

Pitfall: Operating Denominator and EV Claim Scope

If EBITDA is consolidated but EV omits NCI, value covers only some equity claims while profit includes all subsidiary operations. Omitting Lyra's NCI understates EV from 750 to 710 USD millions and EV/EBITDA from 10x to 9.47x. This false signal comes from a bridge-scope mismatch, not improved operations. Adding lease debt to EV while using EBITDA after corresponding lease-cost deductions may also count leases twice. Record EBITDA's consolidation scope and align NCI, leases, and other bridge items to the same economic scope.

```
market_cap, debt, preferred, nci = 480.0, 260.0, 20.0, 40.0
cash, nonoperating_assets, consolidated_ebitda = 30.0, 20.0, 75.0
wrong_ev = market_cap + debt + preferred - cash - nonoperating_assets
correct_ev = wrong_ev + nci
wrong_multiple = wrong_ev / consolidated_ebitda
correct_multiple = correct_ev / consolidated_ebitda
assert wrong_ev == 710.0 and correct_ev == 750.0
assert round(wrong_multiple, 2) == 9.47 and correct_multiple == 10.0
print(f"Omitting NCI: EV={wrong_ev:.0f}, EV/EBITDA={wrong_multiple:.2f}x")
print(f"Matched scope: EV={correct_ev:.0f}, EV/EBITDA={correct_multiple:.2f}x")
```

12.5 Pairing Current Capitalization with Future Statements or Wrong Periods

Period-end is not market availability. Backfilling an earlier signal with a later-restated FY2024 value displayed today inserts unavailable information into rankings. The code deliberately miscalculates early yield using the latest version, then uses PIT selection for the version public and ingested at signal_time: the wrong yield is 4.17%, the then-reproducible yield 4.67%. Market amounts likewise need snapshots no later than the signal, with matched share class and valuation date.

```
from datetime import date, datetime, timezone
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import FinancialVintage, select_pit_vintage

utc = timezone.utc
def vintage(available, ingested, accession, fcfe):
    return FinancialVintage(
        issuer_id="SYNTHETIC-1", security_id="SYNTHETIC-A", fiscal_period="FY2024",
        period_end=date(2024, 12, 31),
        available_at=datetime.fromisoformat(available).replace(tzinfo=utc),
```

```
        ingested_at=datetime.fromisoformat(ingested).replace(tzinfo=utc),
        accession=accession,
        currency="USD", unit="USD millions", net_income_common=30.0,
        common_book_equity=180.0, ebit=50.0, ebitda=75.0, free_cash_flow=fcfe,
        cash_flow_basis="FCFE", industry="Software",
    )

versions = [
    vintage("2025-02-15T12:00:00", "2025-02-16T09:00:00", "SIM-ORIGINAL", 28.0),
    vintage("2025-04-02T12:00:00", "2025-04-02T14:00:00", "SIM-RESTATED", 25.0),
]
signal_time = datetime(2025, 3, 31, 16, 1, tzinfo=utc)
market_cap_at_signal = 600.0  # Constructed March 31 market cap, USD millions
wrong_latest = max(versions, key=lambda row: row.available_at)
wrong_yield = wrong_latest.free_cash_flow / market_cap_at_signal
selected = select_pit_vintage(
    versions, security_id="SYNTHETIC-A", fiscal_period="FY2024", signal_time=
    ↪ signal_time
)
correct_yield = selected.free_cash_flow / market_cap_at_signal
assert wrong_latest.accession == "SIM-RESTATED"
assert selected.accession == "SIM-ORIGINAL"
assert round(wrong_yield * 100, 2) == 4.17
assert round(correct_yield * 100, 2) == 4.67
print(f"Incorrect future-restated yield={wrong_yield:.2%}; PIT yield={correct_yield
    ↪ :.2%}")
```

Real research must also preserve `period_start/end`, accession, `available_at`, `ingested_at`, `signal_time`, market-snapshot `as_of`, and share-class validity periods.

Every three-argument `MarketCapSnapshot` example explicitly assumes that at `as_of` quotes were already public and ingested, the multiplied shares were then-known historical shares, and all EV components used statement versions known at that signal instant. These are constructed demonstration assumptions. Real data need separate market-data `available_at` and `ingested_at` constrained with `as_of` by signal_time. Quote time differs from researcher receipt time; current shares cannot multiply historical prices retrospectively. The selector also lacks Chapter 12's non-reliance state machine; production must exclude known invalid versions. A restatement ingested only after the signal remains unusable even if `available_at` qualifies. Reject version conflicts or route them for review; do not silently zero-fill missing values.

12.6 Confusing Shares, Units, or Lease Policies

Price has currency/share units; multiplying shares in millions yields currency millions. Earnings must also be in millions for units to cancel. Dividing total USD capitalization by USD-millions profit creates a million-fold error. Constructed price USD 60 and 10 million shares give capitalization 600 USD millions and P/E 20x. Correct units alone do not fix multiplying one A-class price by A+B shares: with class prices 60 and 30, total capitalization should be 540, not 600 USD millions.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import market_cap_from_price

price_usd_per_share = 60.0
shares_millions = 10.0
net_income_usd_millions = 30.0
market_cap = market_cap_from_price(price_usd_per_share, shares_millions)
wrong_pe_mixed_units = (price_usd_per_share * 10_000_000) / net_income_usd_millions
correct_pe = market_cap.value / net_income_usd_millions
assert market_cap.value == 600.0
assert wrong_pe_mixed_units == 20_000_000.0 and correct_pe == 20.0

class_a_price, class_a_shares_m = 60.0, 8.0
class_b_price, class_b_shares_m = 30.0, 2.0
wrong_single_class_cap = class_a_price * (class_a_shares_m + class_b_shares_m)
correct_all_class_cap = (
    class_a_price * class_a_shares_m + class_b_price * class_b_shares_m
)
assert wrong_single_class_cap == 600.0 and correct_all_class_cap == 540.0
print(
    f"Mixing USD and USD millions: incorrect P/E={wrong_pe_mixed_units:.0f}, "
    f"correct P/E={correct_pe:.0f}x"
)
print(
    f"Omitting class B price: incorrect market cap={wrong_single_class_cap:.0f}, "
    f"correct market cap={correct_all_class_cap:.0f} USD millions"
)
```

See Section 7.2 for lease consistency. Also check ADR ratios, split adjustments, basic/diluted EPS shares, restricted cash, and nonoperating investments to avoid repeated additions or deductions.

13. Quant Research Application

To use valuation measures as candidate fundamental signals, follow this workflow.

1. **Declare assumptions and universe.** Freeze permanent security IDs, tradable share classes, industries, delisting treatment, fiscal periods, and company classification. Common-stock studies use common earnings, market values, and book values. Consolidated operating profit requires matched NCI scope in EV.
2. **Freeze statement features.** Specify trailing net income/EPS for P/E, parent common book equity for P/B, and original lines/adjustments for EV/EBITDA and EV/EBIT. Choose FCFE/market cap, FCFF/EV, or a separately named (CFO-CapEx) proxy. Preserve versions of tax, interest, lease, cash, debt, share, currency, and unit treatments.
3. **Filter and price at the cutoff.** Candidate versions require external `available_at≤signal_time` and local `ingested_at≤signal_time`. Report dates, acceptance strings, and ingestion cannot substitute for one another. Market value uses prices and shares at or before the signal. All-class aggregation, corporate actions, ADRs, FX, and

exchange calendars need independent sources. With closing prices, signal calculation and orders follow the close, `trade_time` is the next executable instant, and holding returns begin when positions actually take effect.

4. **Set numerical boundaries.** Nonpositive market value/EV invalidates the measure. Nonpositive P/E, P/B, EV/EBIT, or EV/EBITDA denominators retain loss/zero/negative-equity reasons and stay out of monotonic rankings. Preserve source/mapping/unit/version failure states. Negative earnings or FCF yield can retain its sign without entering an undesigned cheapness ranking alongside positives.
5. **Compare within industries before testing factors.** At each signal date, use preregistered directions for industry percentiles or robust standard scores, then form portfolios. Report exclusion shares for loss-makers/negative-equity firms and sensitivity to alternative rules. Use industry equal weights or medians as baselines, rather than explaining performance only against a broad index.
6. **Test counterevidence and net-of-cost results.** Use rolling training/validation/out-of-sample intervals with fixed rebalance, execution lag, suspension/delisting, and version rules. Compare industry, size, leverage, and distress exposures. Measure turnover, spreads, impact, commissions, stock borrowing, and liquidity constraints. Changing a late version must not change earlier signals. Tests must catch future capitalization, statements, constituents, or revisions. Report new results alongside industry baselines, simple book/earnings ranks, and before/after-cost outcomes.

No historical security panel, market prices, industry classifications, or execution costs are assembled here; no factor returns, significance, capacity, or investment conclusions are reported. Future empirical work should first freeze the schema and eligibility rules, then use verifiable actual version chains.

14. Chapter Summary

Valuation begins by identifying claims. Common market value pairs with common earnings/equity; EV with operations covering all capital providers. FCFE and equity cost give common value; FCFF and WACC give operating enterprise value. Cash flows, taxes, interest, leases, and NCI scope must remain consistent. EV bridges and multiples create comparable market measures; DCF discounts explicit forecasts. TV compresses long-run assumptions and often dominates. Quantitative workflows also require denominator signs, units, periods, versions, share classes, and signal/trade timing. Meeting these constraints does not establish profitable signals: industry benchmarks, out-of-sample tests, and costs remain necessary.

15. Exercises

Answers to conceptual, calculation, and interpretation questions are in Section 18.

13-E01 (Claims) Consolidated EBITDA includes all EBITDA of a non-wholly-owned subsidiary. Why does EV/EBITDA require NCI consideration? How should common capitalization be used when studying only parent shareholders?

13-E02 (EV bridge) Calculate Lyra's EV and reverse common equity value using Section 8. Explain the signs of preferred equity, NCI, cash, and NOA.

13-E03 (Multiples) For both Section 8 companies, calculate P/E, P/B, EV/EBITDA, EV/EBIT, earnings yield, FCFE/equity yield, and FCFF/EV yield. Identify each claim pairing.

13-E04 (Nonpositive denominators) A company has capitalization 500 USD millions and net loss -25; another has common book equity -60. State P/E/P/B treatment, calculate the loss-maker's earnings yield, and explain how rankings preserve the information.

13-E05 (Cash flows) With EBIT=80, tax rate 25%, D&A=12, CapEx=20, and $\Delta NWC = 5$, calculate FCFF. Then use common net income 45 and net borrowing -3 to calculate FCFE. Both input sets use USD millions.

13-E06 (WACC) $E = 700, D = 300, k_e = 9\%, k_d = 5\%, T = 20\%$. Calculate WACC and explain why weights usually use market values.

13-E07 (DCF and TV) Year-end FCFF is 30, 35, and 40 USD millions, WACC=10%, and perpetual growth $g = 2\%$. Calculate year-three TV and express today's value under year-end discounting; many decimal places are unnecessary.

13-E08 (PIT) A FY2024 10-K is public on 2025-02-15 and ingested February 16; a restatement is public/ingested on 2025-04-02. Signals are 2025-03-31 and 2025-04-03. Capitalization snapshots are 600 on March 31 and 620 on April 2, in USD millions. Original/restated FCFE is 28/25. Select each version, calculate FCF yield, and identify the market-value date used.

13-E09 (Real-source boundaries) Apple FY2024 10-K lists net income, diluted EPS, and weighted-average diluted shares. Do these alone establish actual historical P/E? Identify at least three additional data or definition checks.

16. Programming Exercises

Programming exercises run in the project `.venv` and import Section 9's core module. Each requires outputs verified by assertions.

13-P01 (Claim-safe ratios) Use `Amount`, the EV bridge, P/E, EV/EBITDA, and FCFE yield to reproduce Orion and Lyra. Assert EV 650/750, P/E 20/24, and EV/EBITDA approximately 8.67/10. Pairing FCFE with EV must be rejected.

13-P02 (Discount matching and TV sensitivity) For annual FCFF `[40,45,50,55,60]` USD millions, calculate TV and EV with WACC=9% and $g = 3\%$. Assert EV approximately 860.57 and terminal share approximately 77.79%. Traverse a WACC/g grid, returning TV only when $g < WACC$.

13-P03 (Cross-section and PIT counterevidence) Use `industry_percentile_ranks` for three profitable firms and one loss-maker missing P/E at the same signal date, including tied ranks. Move a restatement's `ingested_at` beyond signal_time and assert the early signal retains the original. Verify exclusion of market values later than signal_time.

17. Challenge Problems

Open questions allow multiple reasonable answers but require explicit scope, timing, and counterevidence. Reference arguments and scoring appear in the answers.

13-C01 (Consolidated EBITDA, NCI, and leases; prerequisite: Chapter 12) X and Y have equal operating profit. Y has substantial unconsolidated investments or consolidated non-wholly-owned subsidiaries and is lease-intensive; first identify the actual accounting scope of each category. Propose consistent EV/EBITDA paths, identify at least two seemingly conservative bridges that double-count, and list required note fields.

13-C02 (Value-signal research design; prerequisites: Chapters 3 and 12) Design a cross-company PIT FCF-yield test. Justify FCFF/EV, FCFE/market cap, or a CFO−CapEx proxy. Define versions, market prices, share classes, capitalization, industry groups, negative-denominator and missingness rules; specify signal_time, trade_time, and return interval. Include at least two counterevidence tests and an out-of-sample, benchmark, and transaction-cost comparison. Do not present unexecuted research as results.

18. Solutions / Solution Hints

Conceptual and Hand-Calculation Answers

13-E01 Consolidated EBITDA includes all subsidiary operations, while common capitalization prices only parent common-share claims. EV adds debt, preferred equity, NCI, and other non-common claims to cover that operating scope. For parent holders, use common capitalization with common-attributable earnings/book value or FCFE. From enterprise value, deduct non-common claims and add back excluded cash/NOA.

13-E02 $EV = 480+260+20+40-30-20 = 750$ USD millions. Reverse bridge: $750-260-20-40+30+20 = 480$, matching constructed common capitalization. Debt, preferred equity, and NCI add to EV; cash and NOA subtract. The signs follow claim and nonoperating-asset boundaries.

13-E03 Orion: P/E $600/30 = 20x$, P/B $600/180 = 3.33x$, EV/EBITDA $650/75 = 8.67x$, EV/EBIT $650/50 = 13x$, earnings yield $30/600 = 5\%$, FCFE/equity $28/600 = 4.67\%$, FCFF/EV $40/650 = 6.15\%$. Lyra: P/E $480/20 = 24x$, P/B $480/120 = 4x$, EV/EBITDA $750/75 = 10x$, EV/EBIT $750/50 = 15x$, earnings yield $20/480 = 4.17\%$, FCFE/equity $18/480 = 3.75\%$, FCFF/EV $40/750 = 5.33\%$. P/E, P/B, earnings yield, and FCFE yield pair common claims; EV multiples and FCFF yield pair firm-level claims. Matched constructed units cancel in amount ratios.

13-E04 Arithmetic P/E $500/(-25) = -20x$ is not a conventional comparable multiple and stays out of rankings, labeled N/M or negative earnings. Earnings yield $= -25/500 = -5\%$ can preserve the loss direction under a separate rule. Negative common book equity likewise excludes conventional P/B ranking while retaining its state. Do not rank absolute denominators or silently delete firms and call survivors the full universe.

13-E05 $FCFF = 80(1-0.25)+12-20-5 = 47$ USD millions. $FCFE = 45+12-20-5-3 = 29$ USD millions. Net borrowing -3 indicates net repayment. Net income already deducts interest; the FCFF EBIT starting point does not. Without interest and capital-structure data, their difference cannot be inferred from one another.

13-E06 $WACC = 0.7(9\%) + 0.3(5\%)(1 - 20\%) = 6.3\% + 1.2\% = 7.5\%$. Valuation reflects current market claim pricing and required returns; market weights approximate marginal capital structure. Book values are historical accounting amounts potentially far from current financing proportions. Explicitly disclose any book-debt approximation.

13-E07 $TV_3 = 40(1.02)/(0.10 - 0.02) = 510$ USD millions. With year-end flows, $V_0 = 30/1.10+35/1.10^2+40/1.10^3+510/1.10^3 \approx 469.42$ USD millions. TV is located at year-three end and receives three discount periods, like year-three FCFF. The result still depends on normalized FCFF, WACC, and sustainable growth.

13-E08 On March 31, select only the original because the restatement is not yet public/ingested; use March 31 capitalization for FCFE yield $28/600 = 4.67\%$. On April 3, select the April 2 restatement and April 2 capitalization, no later than the signal, for yield $25/620 \approx 4.03\%$. Inputs are constructed. A public restatement ingested after signal_time would remain unusable.

13-E09 No. At minimum, verify common-stock price and matched valuation time; all common classes/ADR conversions and share basis; and available statement version with trailing EPS period/shareholder scope. Also check trading time zone, dividends/splits, valuation date, and ending shares versus weighted-average EPS shares. Net income and EPS alone explain only statement earnings definitions.

Programming Answers

13-P01 Run the complete program below from the project root. It directly calls the chapter module; both `assert` checks cover EV bridges and claim restrictions.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import (
    Amount,
    enterprise_value,
    enterprise_multiple,
    fcf_yield,
    price_to_earnings,
)

def a(value: float) -> Amount:
    return Amount(value, "USD", "USD millions")
```

```
orion_ev = enterprise_value(a(600), a(100), a(50))
lyra_ev = enterprise_value(a(480), a(260), a(30), noncontrolling_interest=a(40),
                           preferred_equity=a(20), nonoperating_assets=a(20))
assert orion_ev.value == 650 and lyra_ev.value == 750
assert price_to_earnings(a(600), a(30)) == 20
assert price_to_earnings(a(480), a(20)) == 24
assert enterprise_multiple(orion_ev, a(75)) == 650 / 75
assert enterprise_multiple(lyra_ev, a(75)) == 10
assert fcf_yield(a(40), orion_ev, cash_flow_basis="FCFF", claim_basis="EV") == 40 /
    ↪ 650
try:
    fcf_yield(a(28), orion_ev, cash_flow_basis="FCFE", claim_basis="EV")
except ValueError:
    pass
else:
    raise AssertionError("FCFE must be paired with equity value")
print("Orion EV=650; Lyra EV=750; claim matching passed")
```

13-P02 The complete reference implementation follows. Change only (r,g), recalculating TV and DCF for each valid pair.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import (
    Amount,
    discounted_cash_flow_value,
    gordon_terminal_value,
)

flows = [Amount(value, "USD", "USD millions") for value in (40, 45, 50, 55, 60)]
rate, growth = 0.09, 0.03
tv = gordon_terminal_value(flows[-1], rate, growth)
value = discounted_cash_flow_value(
    flows, rate, cash_flow_basis="FCFF", discount_rate_basis="WACC", terminal_value=
    ↪ tv
)
pv_tv = tv.value / (1 + rate) ** len(flows)
assert round(tv.value, 2) == 1030.00
assert round(value.value, 2) == 860.57
assert round(100 * pv_tv / value.value, 2) == 77.79
valid = []
for test_rate in (0.08, 0.09, 0.10):
    for test_growth in (0.02, 0.03, 0.04):
        if test_growth < test_rate:
            test_tv = gordon_terminal_value(flows[-1], test_rate, test_growth)
            valid.append((test_rate, test_growth, test_tv.value))
assert len(valid) == 9
print(f"TV={tv.value:.2f}; EV={value.value:.2f}; PV(TV)/EV={pv_tv/value.value:.2%}")
```

13-P03 Section 9's core module supplies all PIT types and industry-percentile functions. This complete code checks ranks, late-ingested restatements, and future-market-value exclusion. Companies and UTC clocks are explicitly constructed, not actual corporate histories.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
from quant_course.valuation import industry_percentile_ranks

values = {"A": 0.05, "B": 0.03, "C": 0.03, "LOSS": None}
industries = {"A": "Software", "B": "Software", "C": "Software", "LOSS": "Software"}
ranks = industry_percentile_ranks(values, industries)
assert ranks == {"A": 1.0, "B": 0.25, "C": 0.25, "LOSS": None}
print(ranks)

from datetime import date, datetime, timezone
from quant_course.valuation import (
    Amount, FinancialVintage, MarketCapSnapshot,
    select_pit_vintage, select_pit_market_cap,
)
signal = datetime(2025, 3, 31, 20, tzinfo=timezone.utc)
original = FinancialVintage(
    issuer_id="SIM-ISSUER", security_id="SIM-COMMON", fiscal_period="FY2024",
    period_end=date(2024, 12, 31),
    available_at=datetime(2025, 2, 15, 8, tzinfo=timezone.utc),
    ingested_at=datetime(2025, 2, 16, 8, tzinfo=timezone.utc),
    accession="SIM-V1", currency="USD", unit="USD millions",
    net_income_common=30, common_book_equity=180, ebit=50, ebitda=75,
    free_cash_flow=28, cash_flow_basis="FCFE", industry="Software",
)
from dataclasses import replace
restated = replace(
    original, accession="SIM-V2", free_cash_flow=25,
    available_at=datetime(2025, 3, 1, 8, tzinfo=timezone.utc),
    ingested_at=datetime(2025, 4, 2, 8, tzinfo=timezone.utc),
)
assert select_pit_vintage(
    [original, restated], security_id="SIM-COMMON",
    fiscal_period="FY2024", signal_time=signal,
) == original

# Constructed assumption: prices and historical shares were public and ingested by
    ↪ as_of.
old_cap = MarketCapSnapshot("SIM-COMMON", signal, Amount(600, "USD", "USD millions"))
future_cap = MarketCapSnapshot(
    "SIM-COMMON", datetime(2025, 4, 2, 20, tzinfo=timezone.utc),
    Amount(620, "USD", "USD millions"),
)
assert select_pit_market_cap(
    [old_cap, future_cap], security_id="SIM-COMMON", signal_time=signal,
) == old_cap
print("late ingestion and future market cap excluded")
```

Challenge Answers

13-C01 First distinguish consolidated non-wholly-owned subsidiaries from investments not consolidated line by line, then fix the activities covered by the operating denominator. Partial ownership or a percentage alone does not determine an NCI addition.

Consolidated path. If consolidated EBITDA includes 100% of subsidiary operations, operating EV must cover debt, preferred equity, and external NCI over the same scope. NCI value adds to parent common capitalization in constructing EV; it does not add to P/E earnings or common capitalization itself. Retaining all subsidiary EBITDA while omitting NCI omits claims and mismatches scope, rather than double-counting. A proportionately attributable operating scope requires rebuilding debt, cash, NOA, and operating amounts consistently, with headquarters allocations and intragroup transactions explained. Multiplying consolidated EBITDA by ownership percentage and pairing it directly with common capitalization is insufficient.

Unconsolidated-investment path. Assume the investment is separately valued as a nonoperating asset. Let I be the estimated value of the parent-held equity stake and NOA_{other} other separately identified NOA. Separating that investment from assets embedded in common-equity pricing gives the core operating bridge:

$$EV_{\text{core}} = M + D + PREF + NCI - CASH - NOA_{\text{other}} - I.$$

Here NCI belongs only to the consolidated operating scope above; do not add outside holders of the unconsolidated investee again. The operating denominator must also exclude this investment's contribution. If reported EBITDA already excludes equity-method income, deduct nothing further. If the chosen EBITDA definition actually includes investment earnings, check their item, period, and attribution, then remove exactly the included part. Investment earnings are not the investee's full EBITDA and cannot casually represent a full operating contribution via ownership scaling. Conversely, after valuing core operating FCFF excluding investment earnings to obtain EV_{core}, add I back through the same asset bridge to reach parent common value:

$$M = EV_{\text{core}} - D - PREF - NCI + CASH + NOA_{\text{other}} + I.$$

Do not retain the same investment in core cash flows/TV and add its full separately estimated value again. Do not deduct its value twice or remove investment earnings from a denominator that already excludes them. A valuation route including the investment as a whole must jointly define its cash flows, risk/discount rate, and value scope; a one-sided multiple adjustment is insufficient.

Two lease paths. Under financing treatment, EV includes corresponding lease liabilities, the operating denominator uses a consistent capitalization convention, and FCF/WACC jointly address lease principal, interest, and debt cost. Alternatively, retain all cash rent as an ongoing operating expense and exclude the same lease commitment from EV; profit, cash flow, and capital costs follow that expense treatment. These are mutually exclusive internally consistent conventions.

Double-counting examples include valuing core operations from cash flows after full rent deduction and then deducting the same lease liability again in the EV-to-common bridge; or retaining an investment in core cash flows/TV and separately adding back its full value. Omitting

NCI is an omission and should be labeled separately. Check ownership/control evidence, actual consolidation/equity-method scope, book NCI and valuation basis, parent-held unconsolidated investment value and income location, subsidiary EBIT/EBITDA, intragroup transactions and headquarters costs, lease maturities/rates, right-of-use depreciation, cash rent, and lease principal/interest cash-flow classification. Disclose book-value proxies for NCI or investment market values. One balance-sheet figure cannot settle the appropriate convention.

Reference scoring (10 points): consistent claims/operating scope, 3; at least two mutually exclusive internally consistent lease paths, 3; at least two double-counting/omission examples, 2; verifiable notes and fields, 2.

13-C02 The reference design chooses FCFE/common capitalization, conditional on reliable common-attributable earnings, interest, net borrowing, and reinvestment. Major leverage changes or difficult borrowing forecasts favor FCFF/EV with explicit WACC and EV bridge. If only CFO−CapEx is available, name it a cash-flow proxy and describe accounting differences without claiming FCFF. Rows preserve permanent issuer/security IDs, share class and validity, fiscal period, currency/unit, original line, accession, available_at, ingested_at, revision status, and transformation-code version. Market snapshots preserve price/share sources, as_of, currency, ADR ratios, corporate actions, and all-common-class aggregation. Industry classifications retain same-date versions. Signals select only statements satisfying both clocks. Constructed orders execute at the next tradable instant; the return interval begins after actual position execution and ends at the next holding interval's valuation endpoint. Same-close execution of a closing signal is not the default.

Counterevidence includes injecting future restatements or prices without changing early factors; moving ingestion after a signal must force fallback or missingness; removing constituents announced only later must not alter historical universes. Out-of-sample reporting includes industry equal-weight and median baselines, predefined rebalance/ranking rules, negative-denominator/missing shares, turnover, capacity, delistings, and results before/after commissions, spreads, impact, borrowing, and financing costs. Conclusions apply only to samples, times, and definitions actually run.

Reference scoring (10 points): matched cash-flow claims and discount rates, 2; PIT and share-class/market-value versions, 3; signal/trade/return timing, 2; two executable counterevidence tests, 2; benchmark and net-of-cost out-of-sample comparison, 1.

19. Further Reading

- **Must Know — MIT OpenCourseWare, Finance Theory I, "Present Value Relations":** read Lectures I–II and slides on asset cash flows, present values, perpetuities, and compounding to reinforce discount timing. These materials are not a unique EV-bridge standard. Slides and videos
- **Should Know — Aswath Damodaran, "An Introduction to Valuation":** read DCF, equity-versus-firm valuation, and relative-valuation sections on claims, discount rates, reinvestment, and comparability. Public notes
- **Should Know — Damodaran, "Valuation" lecture notes:** read FCFE, FCFF, and model selection to compare valuation paths under changing leverage. Historical amounts and valuations are not current quotes. Notes
- **Should Know — SEC, Non-GAAP Financial Measures C&DI 102.07:** examine limits of common CFO-minus-CapEx FCF. This is US SEC staff guidance, not a global accounting-standard definition. SEC page
- **Advanced — Damodaran, "Terminal Value":** read stable growth, currency, and real/nominal constraints, especially sensitivity to (r-g). Public chapter

20. References

1. MIT OpenCourseWare, Finance Theory I — Present Value Relations: present values, perpetuities, and compounding, Sections 4, 6, and 8. Also Equities slides: dividend discounting, EPS, and P/E examples, Sections 5 and 7.
2. Aswath Damodaran, NYU Stern, Valuation: FCFE/FCFF, equity/firm valuation, WACC, and matched cash flows/rates, Sections 5–7; An Introduction to Valuation: DCF and relative valuation, Sections 5–7 and 13.
3. Aswath Damodaran, Terminal Value: stable cash flows, discount rates, growth, and TV sensitivity, Sections 5–8 and 10.
4. SEC, Non-GAAP Financial Measures C&DIs, Question 102.07: common CFO-minus-CapEx FCF is nonuniform, requires explanation, and is not freely discretionary cash, Sections 5–7 and 12.
5. Apple Inc., FY2024 Form 10-K, accession 0000320193-24-000123: period, operating income, net income, diluted EPS, EPS weighted-average shares, ending shares, and balance-sheet lines, Section 11. This supports original filing facts only, not actual capitalization, P/E, signals, or returns.

This chapter uses no secondary-market valuations or historical quotes. Source locations, reading scope, and verification limits are in `source_notes.md`.

Chapter 14 — CAPM and Asset Pricing

The argument begins with future payoffs discounted to today's prices using risk-adjusted weights. Under restrictive mean–variance equilibrium assumptions, CAPM reduces this risk adjustment to covariance exposure to the market portfolio. It provides a testable expected-return relation, rather than a prediction of next period's return or a trading instruction. Prerequisites: Chapter 5 on covariance and linear algebra, Chapter 8 on regression and robust inference, Chapter 10 on market contracts, and Chapter 11 on returns and compounding. You will distinguish no-arbitrage from equilibrium, derive CAPM by perturbing an efficient market portfolio, calculate beta and residuals by hand, reuse monthly regression code, and judge what an empirical deviation can establish.

1. Learning Objectives

Write a one-period stochastic discount factor pricing equation from future payoffs and explain how risk premiums affect prices. Distinguish no-arbitrage restrictions from equilibrium models. Calculate beta as covariance divided by market variance, and derive the Capital Asset Pricing Model (CAPM) from a local perturbation of a mean–variance efficient market portfolio. Distinguish the Capital Market Line (CML) from the Security Market Line (SML), and explain why zero beta does not mean zero total risk.

In practice, independently check the mean vector, covariance matrix, betas, and regression residuals of an explicitly constructed four-row example. The thin script corresponding to `src/quant_course/asset_pricing.py` reproduces a French monthly CAPM regression with fixed sources, SHA-256 hashes, frequency, and sample window. Its sample intercept is not tradable alpha. Successful work has consistent dimensions, hand-checkable examples, agreement between charts and script outputs, and an explicit account of what the case did not test.

2. Why Quants Need This

Before asking whether a signal predicts returns or makes money, quantitative research needs a relation between current prices and future payoffs. Ignoring state-dependent discounting treats assets with equal cash amounts in different states as equally valuable. Looking only at total volatility can mistake diversifiable variation for risk that the market must compensate. Asset pricing connects prices, exposures, and required expected returns, providing a clear null hypothesis for benchmarks, risk adjustment, and abnormal-return tests.

No-arbitrage excludes feasible free lunches without uniquely specifying every risk premium. CAPM adds investor choice, common expectations, risk-free borrowing and lending, and market clearing to obtain a particular market-beta relation. A good regression fit does not establish those assumptions. A nonzero historical intercept does not establish a financeable, executable strategy that remains profitable after costs.

Two constructed assets run through the chapter: their four-row mean total returns are equal, but their market betas are 0.5 and 1.5. Beta measures the direction and scale of market comovement; it does not rename a realized average as an expectation. Chapter 15 develops multiple factors. The historical case here is a single-factor market regression.

3. Intuition: In Which State Does the Asset Pay?

Consider a security paying EUR100 next period. Paying when incomes are ample and the marginal value of consumption is low can command a different current price from paying when incomes are strained and cash is most valuable. A state-independent average discount loses this distinction. A payoff received in a state with higher marginal value deserves more weight in its current price.

CAPM makes a strong simplification. In its one-period mean–variance world, investors adjust risk using the market portfolio and a risk-free asset. An asset that also falls when the market fares badly carries exposure that diversification cannot easily remove; equilibrium requires higher expected excess return. Beta summarizes that comovement. It does not measure every bad-state risk or guarantee that higher beta means a larger gain next period.

Asset A and Asset B both have a four-row mean total return of 7%, with excess-return betas of 0.5 and 1.5. If those rows were the complete probability distribution, they would not both satisfy the CAPM expected returns of 5% and 11% implied by a 2% risk-free rate and 8% expected market return. Here the rows are equally weighted teaching records, without a claim about long-run state probabilities. Sample means and model expectations are different objects; prediction and pricing require assumptions and evidence.

4. Mathematical Foundations: Units of Payoffs, Returns, and Risk

On a one-period timeline, today is t and the next date is $t+1$. The terminal payoff X_{t+1} and current price P_t both have currency units. The simple total return $R_{t+1} = (X_{t+1} - P_t)/P_t = X_{t+1}/P_t - 1$ is dimensionless; its gross return factor is $G_{t+1} = X_{t+1}/P_t = 1 + R_{t+1}$. Before displaying a percentage, check that an internal decimal such as 0.02 already means 2%; do not divide it by 100 again.

CAPM here relates expected simple returns over the same holding period, in the same currency and return convention. R_f is the risk-free simple return and $\mathbb{E}[R_m]$ the expected market simple total return, rather than gross return factors. Monthly returns require monthly R_f and a monthly market premium. Although dimensionless, monthly beta depends on frequency and the market proxy. An expectation under a model or population state distribution cannot be replaced by next period's realization.

For equally weighted observations $x_1, \ldots, x_n$, the sample mean is $\bar{x} = n^{-1} \sum_j x_j$ and sample variance is $s_x^2 = (n-1)^{-1} \sum_j (x_j - \bar{x})^2$, with sample covariance defined similarly. Return variances and covariances have units of squared per-period return proportions. Their ratio gives dimensionless beta. For a return vector R with k entries, the mean vector μ has dimension k, and the $k \times k$ covariance matrix Σ describes every pairwise comovement.

5. Formal Definitions: From General Pricing to Market Beta

A one-period stochastic discount factor (SDF), m_{t+1}, is a random, state-dependent weight converting terminal payoffs into current values. Under no-arbitrage and the required state-space conditions, write

$$P_t = \mathbb{E}_t[m_{t+1}X_{t+1}].$$

The units of m are current currency per unit of terminal currency; mX has the same units as P. It maps a future payoff X to a current price. States with high m have higher marginal value: greater payoffs there command higher prices. This representation is more general than CAPM and does not by itself determine the market risk premium.

No-arbitrage excludes a feasible portfolio with nonpositive initial cost, nonnegative future payoff in every positive-probability state, and either strictly negative initial cost or strictly positive payoff in at least one state. We use a frictionless finite-state setting allowing cash borrowing and lending and linear combinations of trades. State probabilities are positive; prices define a consistent linear functional on attainable payoffs. In this setting, no-arbitrage is equivalent to strictly positive state prices and hence a strictly positive SDF, without requiring market completeness. Several positive SDFs may support the same traded prices, so no-arbitrage alone does not imply CAPM. Equilibrium also requires investors to optimize subject to budgets and preferences and markets to clear. CAPM is a model with explicit equilibrium assumptions.

Separate probabilities from pricing weights. In finite states $s = 1, \ldots, S$, $p_s > 0$ is the probability and m_s the SDF in that state. The state price $q_s = p_s m_s$ is today's price of one terminal currency unit paid only in state s. When that state security cannot be traded separately, q_s is an implicit weight supporting traded-asset prices. Thus

$$P = \sum_{s=1}^{S} p_s m_s X_s = \sum_{s=1}^{S} q_s X_s.$$

p_s is dimensionless and sums to 1. Both m_s and q_s have units of current currency per terminal currency; q_s need not sum to 1. A complete market has traded payoffs spanning the entire state-payoff space, yielding unique no-arbitrage state prices. In an incomplete market, several positive weight vectors can price all traded payoffs identically while valuing unspanned payoffs differently. A positive SDF establishes existence, without uniquely determining every expected risk premium.

The remaining chapter uses simple returns R, denoted R_{t+1} in Section 4. To show the SDF identity, use gross return factors $G = 1 + R$, the risk-free simple return r_f (that is, R_f), and its gross factor $G_f = 1 + r_f$. Normalize an asset's price to 1 to obtain $1 = \mathbb{E}[mG]$. A risk-free terminal payment of 1 costs $\mathbb{E}[m]$, so $G_f = 1/\mathbb{E}[m]$. Using $\mathbb{E}[mG] = \mathbb{E}[m]\mathbb{E}[G] + \mathrm{Cov}(m, G)$ gives

$$\mathbb{E}[G] - G_f = -\frac{\mathrm{Cov}(m, G)}{\mathbb{E}[m]}.$$

Because $G = 1 + R$, gross factors and simple returns differ only by 1: $\mathrm{Cov}(m, G) = \mathrm{Cov}(m, R)$ and $\mathbb{E}[G] - G_f = \mathbb{E}[R] - r_f$. Equivalently, $\mathbb{E}[R] - r_f = -\mathrm{Cov}(m, R)/\mathbb{E}[m]$. An asset paying

less in high-SDF, high-marginal-value states requires greater expected return. The covariance uses the same one-period probability distribution. This general SDF interpretation precedes the additional restrictions on m and the market portfolio that lead to a single CAPM beta.

Systematic risk is common risk that diversification alone cannot remove. Idiosyncratic risk is variation left over relative to the market or a specified factor model. A single-factor regression is

$$R_i - R_f = \alpha_i + \beta_i(R_m - R_f) + \varepsilon_i, \qquad \beta_i = \frac{\mathrm{Cov}(R_i - R_f, R_m - R_f)}{\mathrm{Var}(R_m - R_f)}.$$

Beta is linear exposure of one-period asset excess returns to market excess returns. It depends on the sample, frequency, proxy, and estimator, rather than being an intrinsic asset constant. A fitted linear projection has in-sample residuals orthogonal to market excess returns, yet those residuals can have substantial variance. Zero beta means zero linear covariance, rather than zero total risk.

CAPM requires the following expected-return relation for any asset i:

$$\mathbb{E}[R_i] - R_f = \beta_i\big(\mathbb{E}[R_m] - R_f\big).$$

Both sides are expected simple excess returns over the same interval, measured as returns per period; beta is dimensionless. For the market portfolio with $\beta_i = 1$, the equation recovers its own risk premium. A zero-beta asset has zero CAPM expected excess return, while the market premium may remain nonzero and the asset may retain idiosyncratic risk. This is an equilibrium cross-sectional expectation restriction, rather than an identity for next period's realized returns.

The CML describes the expected return of efficient mixtures of the risk-free asset and market portfolio in mean–standard-deviation space:

$$\mathbb{E}[R_p] = R_f + \frac{\mathbb{E}[R_m] - R_f}{\sigma_m}\sigma_p.$$

Its slope is the market Sharpe ratio, the expected premium per unit of volatility. CML applies to efficient combinations of the risk-free asset and tangency market portfolio. SML relates expected return to beta for assets or portfolios under CAPM equilibrium. Their horizontal axes differ: total standard deviation for CML, market beta for SML. The two lines cannot be interchanged.

Branch condition: negative market weights and absolute values

Why must CML specify a branch? Let the market weight be w and risk-free weight $1 - w$, with R_f known and fixed for this period:

$$R_p = R_f + w(R_m - R_f), \quad \mathbb{E}[R_p] = R_f + w(\mu_m - R_f), \quad \sigma_p = |w|\sigma_m.$$

Taking the square root of w^2 produces an absolute value, rather than simply w. With a positive market premium, $w \geq 0$ selects the positive-slope efficient CML; $w > 1$ represents borrowing at the same R_f. For $w < 0$, the same standard deviation lies on a lower, negative-slope branch. With $R_f = 2\%$, $\mu_m = 8\%$, and $\sigma_m = 10\%$, weights $w = 0.5$ and $w = -0.5$

Continued

both give 5% volatility, but expected returns of 5% and −1%. Their betas are 0.5 and −0.5, distinguishable on SML. Converting total volatility to expected return using CML already selects the efficient $w \geq 0$ branch.

Equality of total- and excess-return covariances above concerns a conditional one-period distribution with R_f known initially. RF can vary across historical months, so $\mathrm{Cov}(R_i - R_f, R_m - R_f)$ and $\mathrm{Cov}(R_i, R_m)$ need not coincide. Empirical interfaces therefore subtract each month's RF and define beta using excess returns. The fact that subtracting a constant leaves covariance unchanged does not justify dropping a time-varying RF column.

Distinguish the CAPM alpha target from its estimate. Theoretical expected alpha can be defined as $\alpha_i^* = \mathbb{E}[R_i - R_f] - \beta_i \mathbb{E}[R_m - R_f]$, which CAPM restricts to zero. The time-series intercept $\hat{\alpha}$ is estimated for a particular sample, market proxy, and model. Statistical rejection of zero neither realizes a future return nor establishes an executable strategy after financing and trading costs.

6. Derivation: Perturbing an Efficient Portfolio

Assume a one-period mean–variance setting: risky assets are tradable and divisible; the initially known risk-free return is constant across states; wealth weights satisfy $\sum_j w_j = 1$. Investors borrow and lend at the same known R_f, share expected-return vectors and covariance matrices, and care only about mean and variance (or have jointly normal returns making this reduction appropriate). Market clearing aggregates their risky holdings into the market portfolio. Its variance and expected premium are positive. Locally, each risky-asset weight can be increased or decreased. Without these conditions, this derivation does not guarantee the conclusion.

Common expectations give investors the same mean–variance opportunity set. With borrowing and lending at the same risk-free rate, all select the same tangency risky portfolio; risk aversion changes its allocation against the risk-free asset. Risky-market clearing identifies that portfolio with the aggregate market. This theoretical market holds all investable risky assets at market-value weights; an index or French market factor is a proxy. Scaling the tangency allocation using R_f is equivalent to choosing the risky portfolio with the highest Sharpe ratio.

Write the market portfolio M with expected total return μ_m, standard deviation $\sigma_m > 0$, and excess return $a = \mu_m - R_f > 0$. Reallocate a small weight ε from it to asset i, retaining total weight 1:

$$R_p(\varepsilon) = (1 - \varepsilon)R_m + \varepsilon R_i.$$

The coefficients sum to 1, preserving the budget. Define the portfolio Sharpe ratio as $S(\varepsilon) = [\mu_p(\varepsilon) - R_f]/\sigma_p(\varepsilon)$. As the tangency portfolio of the common opportunity set, $\varepsilon = 0$ locally maximizes Sharpe among fully invested risky portfolios. Both positive and negative small reallocations are feasible, so the derivative is zero. The mean derivative is $\mu_i - \mu_m$; the variance derivative at zero is $2[\mathrm{Cov}(R_i, R_m) - \sigma_m^2]$, and the standard-deviation derivative divides it by $2\sigma_m$. Apply the

quotient rule before jumping to beta:

$$S'(0) = \frac{(\mu_i - \mu_m)\sigma_m - a\sigma_p'(0)}{\sigma_m^2}, \qquad \sigma_p'(0) = \frac{\mathrm{Cov}(R_i, R_m) - \sigma_m^2}{\sigma_m}.$$

Here $S'(0)$ measures the marginal Sharpe change from reallocating a small wealth share, and $\sigma_p'(0)$ the marginal standard-deviation change. Substitute the second expression into the first, set $S'(0) = 0$, and multiply by the positive σ_m:

$$0 = (\mu_i - \mu_m) - \frac{a}{\sigma_m^2}\{\mathrm{Cov}(R_i, R_m) - \sigma_m^2\}.$$

This covariance uses total returns. Subtracting the same constant one-period risk-free return leaves covariance unchanged. The coefficient a/σ_m^2 follows from tangency Sharpe optimality, rather than an arbitrarily chosen historical regression coefficient.

Substitute $a = \mu_m - R_f$ and rearrange:

$$\begin{aligned}
\mu_i - \mu_m &= \frac{\mu_m - R_f}{\sigma_m^2}\{\mathrm{Cov}(R_i, R_m) - \sigma_m^2\}, \\
\mu_i - R_f &= \frac{\mathrm{Cov}(R_i, R_m)}{\sigma_m^2}(\mu_m - R_f), \\
\mathbb{E}[R_i] - R_f &= \beta_i\{\mathbb{E}[R_m] - R_f\}.
\end{aligned}$$

The ratio is market beta. The first line links marginal expected return to marginal common risk at the tangency optimum. The second cancels the market's own component. The third rewrites total returns as excess returns, since constant R_f does not change covariance. This derives CAPM from equilibrium and efficiency without proving that real markets meet its assumptions. Different borrowing and lending rates, short-sale constraints, heterogeneous expectations, preferences beyond mean–variance, or an incomplete opportunity set can invalidate the common tangency portfolio or local first-order conditions.

7. Financial Meaning: Compensation for Marginal Common Risk

Diversification reduces the parts of security returns that do not move together. With many low-correlation assets, individual positive and negative residuals can offset. Exposure that falls with the entire market does not disappear by adding similar securities. CAPM therefore ties expected premiums to beta rather than total standard deviation. This remains conditional: omitted systematic factors can make market beta an incomplete risk description.

A beta above 1 indicates a larger historical linear excess-return response than the market itself; a negative beta indicates opposite sample linear comovement. Neither guarantees a return direction in a given period. Leverage usually increases equity sensitivity to underlying asset changes and can introduce financing constraints and bankruptcy risk. Scaling beta does not guarantee higher realized or risk-adjusted returns.

A zero-beta asset can have high variance. CAPM gives it zero expected market excess return, hence a required return of RF. This does not set the market premium to zero or eliminate total risk, loss probabilities, or holding costs. CML uses total volatility; SML uses market covariance. They describe different risk concepts.

8. Numerical Example

8.1 A Two-State Stochastic Discount Factor

Complete example: two-state prices, returns, and covariance check

This is a constructed one-period euro model, rather than market data. Two future states, ample income and strained income, each have probability 1/2. Their given positive SDFs are 0.9 and 1.1; the larger SDF values a terminal unit more highly in the strained-income state. Security X pays EUR110 and EUR90; Y pays EUR100 in both states.

Future state	Probability p	SDF m	State price q=p×m	X payoff (EUR)	q×X (current EUR)
Ample income	0.5	0.9	0.45	110	49.5
Strained income	0.5	1.1	0.55	90	49.5

Probabilities describe how often states occur; SDFs describe the value of receiving a payoff there. Only the final column sums to price. State prices sum to 1 here because this example has a zero risk-free return.

First, $\mathbb{E}[m] = (0.9 + 1.1)/2 = 1$, giving risk-free gross factor $G_f = 1$ and zero simple risk-free return. This pricing relation follows from the constructed inputs; it is not an assumption about actual euro interest rates.

Next multiply state by state: today X costs $P_X = (0.9 \times 110 + 1.1 \times 90)/2 = 99$ euros and Y costs $P_Y = (0.9 \times 100 + 1.1 \times 100)/2 = 100$ euros. Both have an expected future payment of 100, but X pays only 90 when cash is more valuable. Discounting only the average payoff loses this risk distinction.

X therefore has expected simple return $100/99 - 1 \approx 1.0101\%$, versus zero for Y. Its premium compensates for paying less in the strained-income state, without guaranteeing a gain. Its realized state returns are $110/99 - 1 \approx 11.1111\%$ and $90/99 - 1 \approx -9.0909\%$. Even this small table separates expectations from realizations.

Finally, check Section 5's identity: $\mathrm{Cov}(m, X) = 99 - 1 \times 100 = -1$ euros, hence $\mathrm{Cov}(m, G_X) = -1/99$. Substitution into $-\mathrm{Cov}(m, G_X)/\mathbb{E}[m] = 1/99$ gives exactly the expected premium. All probabilities, payoffs, and positive SDFs are given teaching inputs. The calculation demonstrates pricing weights without estimating preferences from real data.

```
import numpy as np
probability = np.array([0.5, 0.5])
discount = np.array([0.9, 1.1])
payoff = np.array([110.0, 90.0])
price = float(probability @ (discount*payoff))
mean_m = float(probability @ discount)
gross = payoff/price
mean_g = float(probability @ gross)
cov_m_g = float(probability @ ((discount-mean_m)*(gross-mean_g)))
assert np.isclose(price, 99.0)
assert np.isclose(mean_g-1/mean_m, -cov_m_g/mean_m)
print(price, mean_g-1/mean_m, gross-1)
```

8.2 Calculating Beta and Residuals from Four Rows

Complete example: four-row betas, residuals, and expected returns

The following four rows are constructed and equally weighted for hand calculation. They are neither market records nor claimed random population draws. Values are one-period decimal simple returns. RF is 2% in every row; market excess returns are $-4\%, 1\%, 11\%, 16\%$. Centered market excess returns are $(-10\%, -5\%, 5\%, 10\%)$ and constructed residuals $(1\%, -2\%, 2\%, -1\%)$, whose inner product with the centered market is zero.

Define asset excess returns as

$$R_A - R_f = 5\% + 0.5(R_m - R_f - 6\%) + \varepsilon, \quad R_B - R_f = 5\% + 1.5(R_m - R_f - 6\%) + \varepsilon.$$

This gives the total-return table:

Constructed row	RF	Mkt−RF	Asset A total return	Asset B total return	Constructed residual
1	2%	−4%	3.0%	−7.0%	1%
2	2%	1%	2.5%	−2.5%	−2%
3	2%	11%	11.5%	16.5%	2%
4	2%	16%	11.0%	21.0%	−1%

With equal row weights, the mean excess-return vector in order $(Mkt-RF, A-RF, B-RF)$ is $\mu = (6\%, 5\%, 5\%)$. Using sample covariance denominator $n - 1 = 3$ gives approximately

$$\Sigma = \begin{bmatrix} 0.008333 & 0.004167 & 0.012500 \\ 0.004167 & 0.002417 & 0.006583 \\ 0.012500 & 0.006583 & 0.019083 \end{bmatrix}.$$

Entries have squared return-proportion units. Unrounded cross-products give $\beta_A = (0.0125/3)/(0.0250/3) = 0.5$ and $\beta_B = (0.0375/3)/(0.0250/3) = 1.5$; six-decimal entries above are for display. The ordinary least squares (OLS) fitted lines are $\widehat{R_A - R_f} = 2\% + 0.5(Mkt - RF)$ and $\widehat{R_B - R_f} = -4\% + 1.5(Mkt - RF)$. Both have residuals $(1\%, -2\%, 2\%, -1\%)$. Defining beta does not eliminate residual risk.

If constructed $R_f = 2\%$ and expected simple market total return $\mathbb{E}[R_m] = 8\%$ are CAPM inputs, SML gives 5% and 11% expected returns at betas 0.5 and 1.5. Both four-row mean total returns are instead 7%. There is no arithmetic contradiction: a finite realized sample mean differs from a model expectation, and the rows were not declared an exact state distribution. Equal past means alone do not determine future pricing relations.

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I3 reader package root, establish the documented environment, then run:

```
.venv/bin/python run_notebook.py notebooks/ch14_asset_pricing.ipynb
```

The default run checks four constructed return rows, betas, and residuals, then draws new CML/SML and beta-scatter charts using RF=2% and expected market return 8%. Both new PNGs go to `runs/ch14_asset_pricing_<uuid>/`. With `RUN_FRENCH=False`, the historical French regression and residual-chart branches are disabled: the default neither reads nor downloads market snapshots. The NoDur figures and tables in Sections 10–11 remain historical reading material.

The runner separately records kernel execution in a unique `runs/execution_*` directory, distinct from the chapter UUID method directory printed by the Notebook. Constructed execution checks computational interfaces; it produces no market research result, PIT forecast, or strategy return after costs. Chapter 14's historical NoDur inference uses HAC5; Chapter 15 and Project 2 historical main tables use HAC6. Keep point estimates and inference settings identifiable rather than altering intervals for superficial agreement. This batch inspected actual source without executing these Notebooks or rerunning historical regressions.

English Markdown for the Chapter 14 Notebook belongs to BIL23; this batch does not modify it. The CLI and old production tests below refer to the course working root. Readers need not restore production tests absent from the package.

Historical Course Working Root and Original Implementation

Core calculations are in `asset_pricing.py`: `beta_from_covariance` divides covariance by variance; `fit_capm_excess` returns estimates, fitted values, residuals, and (R^2) using single-regressor OLS identities; `capm_expected_return` applies the expected-return relation. Heteroskedasticity and autocorrelation consistent (HAC) inference uses the shared monthly `fit_monthly_factor_model` rather than a second engine. Missing or nonfinite inputs raise explicit errors, without silently dropping rows.

The `ch14_asset_pricing.py` script is a thin loading, reporting, and plotting layer. From the course root:

```
.venv/bin/python code/ch14_asset_pricing.py
```

For historical data, the manifest first verifies processed-file SHA-256 hashes; NoDur, Mkt-RF, and RF must then have consecutive `PeriodIndex('M')` months. Mkt-RF already subtracts RF, so the target subtracts it only once via `NoDur - RF`. Month labels do not establish information-availability, signal, or trade timestamps: this is not a backtest. HAC maxlags=5 means five adjacent monthly observation rows. Month ends separated by 28, 30, or 31 days are not converted to artificial equal-second timestamps. Inference depends on time-series approximations and bandwidth choices; significance alone does not establish alpha.

The CLI above belongs to historical course production, rather than the accepted I3 reader default. Programs 14-P01–P03 can be pasted independently at the working root after installing the course as documented. The local `.venv/bin/python` already imports `quant_course` directly, so no additional path bootstrap was added.

10. Visualization

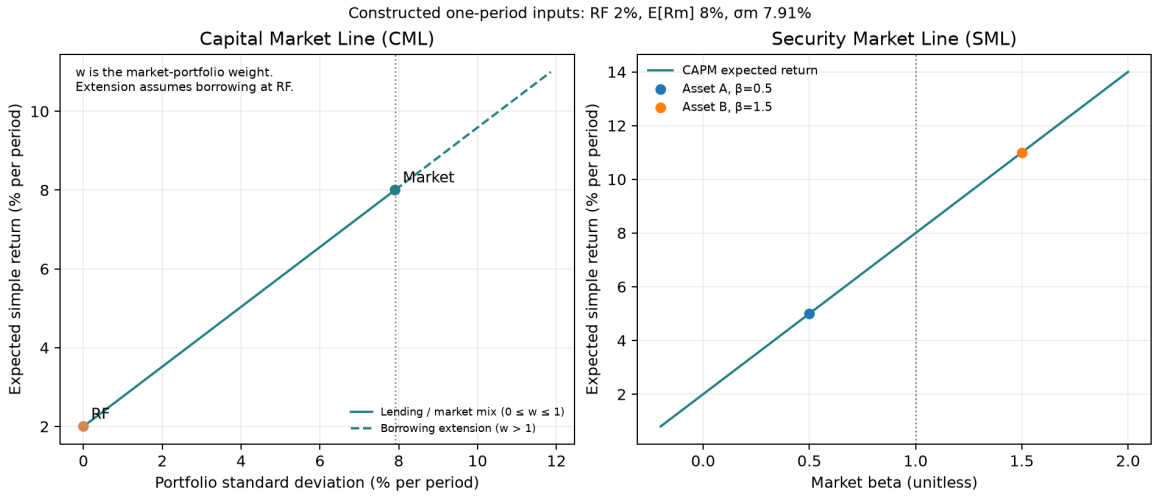


Figure 1: CML and SML under constructed inputs

Left: horizontal axis is portfolio standard deviation (% per period), vertical axis expected simple return (% per period). Market weights from 0 to 1 combine the risk-free asset and market; the dashed extension assumes borrowing at RF for weights above 1. RF=2%, expected market return=8%, and market volatility about 7.91% are constructed inputs. The 7.91% uses ddof=0 on the four rows as an assumed model risk parameter, rather than Section 8's ddof=1 sample covariance risk estimate. It does not turn those rows into a true population state distribution. Right: horizontal axis is dimensionless beta, vertical axis model expected return, with endpoints calculated under the same assumptions. These are conditional model relations, rather than market observations.

The horizontal axis shows beta estimated from the four constructed rows; the vertical axis shows their equally weighted mean total return. Orange points A and B both lie at 7%, while SML gives 5% and 11% at betas 0.5 and 1.5. The points are realized sample means and the line a constructed expectation model. Their gaps do not establish tradable alpha.

Left: Mkt-RF (% per month) horizontally and NoDur−RF (% per month) vertically; points are 120 published monthly rows from 2010–2019, with an OLS line fitted in that window. Right: return month horizontally and regression residual (% per month) vertically. These charts reveal linear association and residual time structure; one residual chart cannot establish model validity.

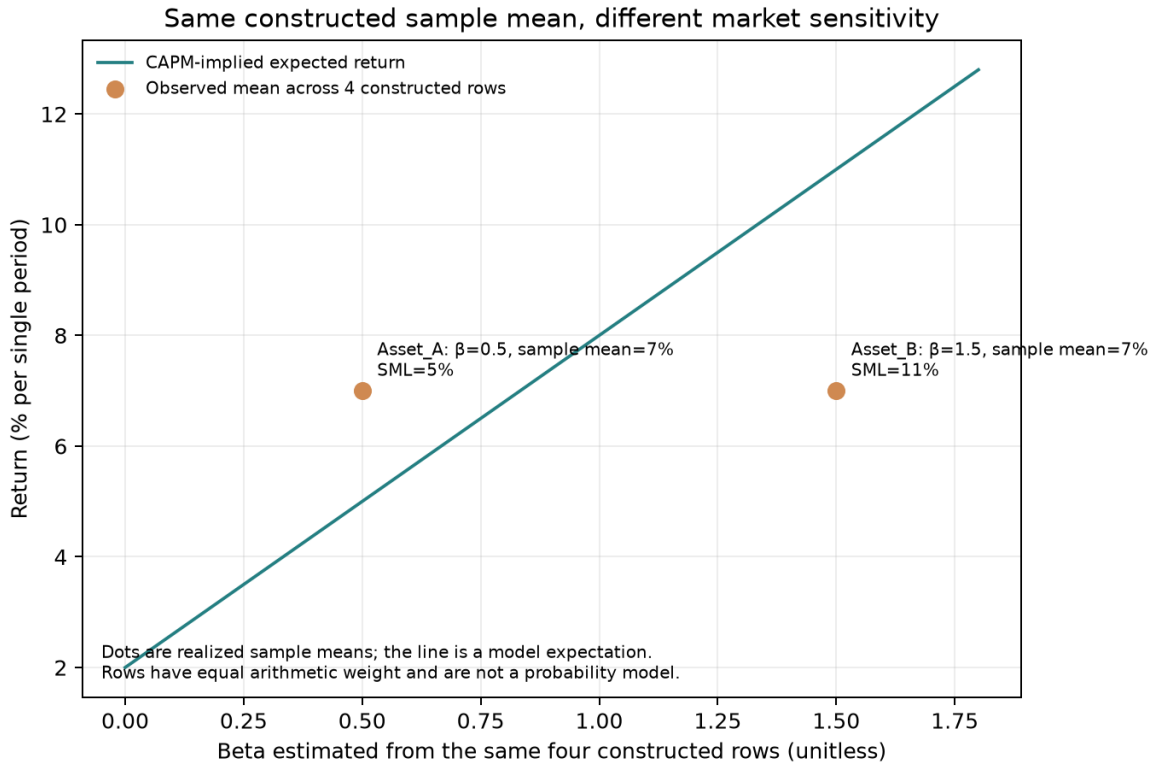


Figure 2: Betas and sample means of the constructed sample

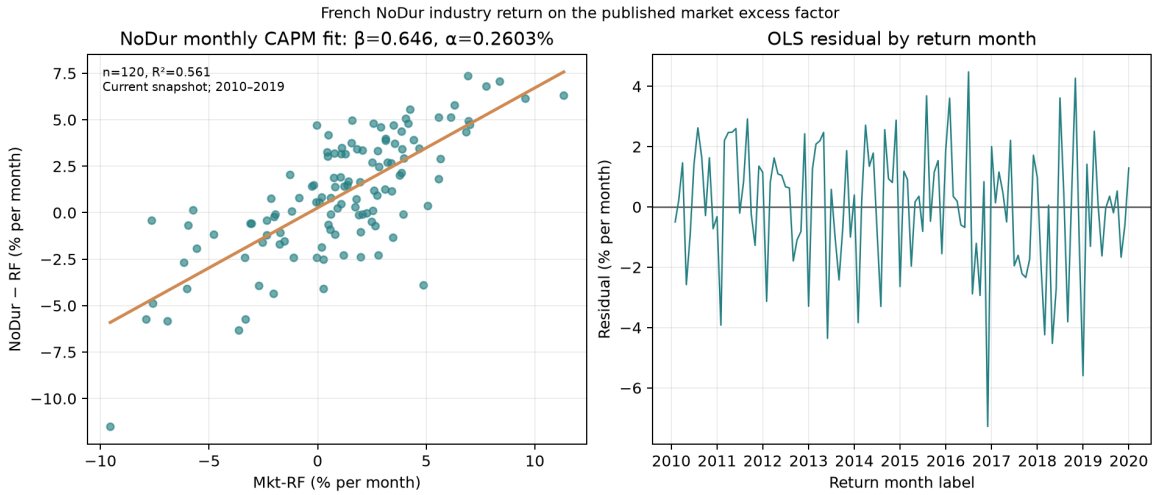


Figure 3: French NoDur CAPM regression and monthly residuals

11. Real Market Example

The historical case estimates only a single market factor. NoDur is the value-weighted nondurables industry total return from French's monthly ten-industry portfolios; Mkt-RF is published market excess return; RF is the same-frequency one-month Treasury-bill proxy. The fixed window has 120 consecutive months, 2010-01 through 2019-12, with decimal processed-file hashes checked before calculation. See `french_industries_2026-10-05.json` and `french_factors_2026-10-05.json`: industry SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`; factor SHA-256 `c19770a19a0ac47ad1c29074f5723457`

`0aeaa9725212cb4fa8146a2846c52814`. The FF source declares CRSP database vintage month 202608. Both are current published snapshots from 2026, rather than point-in-time (PIT) data available in each historical month.

The equation is $NoDur_t - RF_t = \alpha + \beta(Mkt - RF)_t + \varepsilon_t$, using monthly decimal simple returns and dimensionless beta. Estimated monthly beta is 0.6456, with sample $R^2 = 0.5608$. Monthly intercept is 0.2603%, Bartlett HAC standard error 0.1926%, reference 95% interval [−0.1210%, 0.6417%], and reference two-sided $p = 0.179$. HAC maxlags=5 counts consecutive monthly rows. Intervals and p-values are approximate under this estimation setup, rather than definitive judgments of beta or alpha.

Results are recorded in `ch14_french_capm_2010_2019.json`. This in-sample regression describes NoDur–market excess-return comovement. It does not test the complete aggregate-wealth market or include risk-free financing, costs, impact, capacity, or execution times. An intercept interval containing zero does not prove CAPM; an interval excluding zero would not alone establish a trading opportunity. Mean monthly NoDur total return is 1.0099%, RF 0.0427%, and excess return 0.9673%. These describe the fixed sample rather than next month's expectation.

12. Common Mistakes

One error subtracts RF from Mkt-RF again, producing $R_m - 2R_f$ rather than standard market excess exposure. Another regresses raw asset total returns and labels its intercept CAPM alpha; RF changes the intercept and potentially the slope. The following historical frozen-data script checks hashes and compares correct and incorrect monthly betas and intercepts:

```
from pathlib import Path
import json
import sys
import statsmodels.api as sm

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.factor_models import fit_monthly_factor_model
from quant_course.market_data import load_monthly_snapshot

industry_manifest = json.loads(
    (ROOT / "data/manifests/french_industries_2026-10-05.json").read_text()
)
factor_manifest = json.loads(
    (ROOT / "data/manifests/french_factors_2026-10-05.json").read_text()
)["datasets"]["ff3"]
industry = load_monthly_snapshot(
    ROOT / industry_manifest["processed_path"],
    industry_manifest["processed_sha256"], ["NoDur"]
)
factors = load_monthly_snapshot(
    ROOT / factor_manifest["processed_path"],
    factor_manifest["processed_sha256"], ["Mkt-RF", "RF"]
)
sample = industry.join(factors).loc["2010-01":"2019-12"]
correct = fit_monthly_factor_model(
```

```
    sample["NoDur"], sample[["Mkt-RF", "RF"]],
    factor_columns=("Mkt-RF",), hac_maxlags=5
)
wrong_factors = sample[["Mkt-RF", "RF"]].copy()
wrong_factors["Mkt-RF"] -= wrong_factors["RF"]
double_subtracted = fit_monthly_factor_model(
    sample["NoDur"], wrong_factors,
    factor_columns=("Mkt-RF",), hac_maxlags=5
)
raw_return_fit = sm.OLS(
    sample["NoDur"], sm.add_constant(sample["Mkt-RF"])
).fit()
print("correct beta:", correct.coefficient_table.loc["Mkt-RF", "estimate"])
print("beta after subtracting RF twice:", double_subtracted.coefficient_table.loc["
    ↪ Mkt-RF", "estimate"])
print("correct excess-return intercept:", correct.coefficient_table.loc["const", "
    ↪ estimate"])
print("raw-return intercept (not CAPM alpha):", raw_return_fit.params["const"])
assert round(correct.coefficient_table.loc["Mkt-RF", "estimate"], 6) == 0.645630
assert round(double_subtracted.coefficient_table.loc["Mkt-RF", "estimate"], 6) ==
    ↪ 0.646460
```

In the frozen sample, correct beta is 0.645630, versus 0.646460 after subtracting RF twice. The correct monthly excess-return intercept is 0.2603%, versus 0.3036% for raw total returns. The latter is not the CAPM excess-return intercept. Use same-month `NoDur - RF` as the target and already-adjusted `Mkt-RF` directly as the regressor.

A third error substitutes $\hat{\alpha}$ or mean sample excess return for $\mathbb{E}[R_i] - R_f$. An estimated intercept and a realized path average can both reflect finite samples, revised data, proxy choice, structural changes, and residuals. Expectations concern a target population or equilibrium, without promising next period's realization.

A fourth error treats beta as all risk. A zero-beta asset can retain large residual variance; examine covariance, beta, total variance, and residuals together. A fifth announces a strategy from a significant intercept. Estimation does not automatically address selection, multiple testing, financing, taxes, impact, trading constraints, or when a signal became available.

Do not interpret a correlation regression as causal, or claim that ten industries or a common market proxy unambiguously test the complete market portfolio. Proxy coverage and mean–variance efficiency form a joint question, central to Roll's testability critique. This does not mean every CAPM test is invalid.

13. Quant Research Application

Define the research target first. Asking whether market beta explains cross-sectional mean returns is a pricing test; whether past beta predicts next month's return is a prediction test; whether a strategy earns investable net returns is a portfolio and execution test. Their dependent variables, estimation windows, information timing, and falsification conditions differ.

Preregister theoretical restrictions, proxy, frequency, sample window, and missing-data rules. Estimate beta only from returns available before the estimation date. Prediction tests need rolling or expanding windows, with estimates and signals available before trades. Strategy tests additionally define rebalancing, execution prices, borrowing and lending, costs, impact, short availability, and capacity. Use holdouts or rolling out-of-sample evaluation, report every tried model and asset, and retain insignificant results.

A falsifiable CAPM claim is that, for a specified tradable universe and preselected proxy, the cross-sectional mean excess-return slope on beta agrees with the market premium and intercept deviations fall within sampling uncertainty. Disagreement can reflect inapplicable assumptions, an inefficient proxy, noisy expected-return estimates, selection, or omitted risk. It does not automatically identify the cause. Conclusions must match the research estimand.

14. Chapter Summary

A one-period price is the expectation of future payoff weighted by an SDF. No-arbitrage permits a broader representation than CAPM. With mean–variance investors, common expectations and risk-free financing, market efficiency, and clearing, CAPM reduces compensation to market covariance beta. Local perturbation derivatives explain its origin; the assumptions show why it is not a law of nature.

Beta measures systematic linear comovement, rather than total risk or guaranteed future return. SML plots expected return against beta; CML plots efficient-portfolio expected return against standard deviation. Four rows suffice to hand-check the constructed means, betas, and residuals. The French case illustrates monthly regression and reference HAC inference, while one proxy, historical window, and intercept cannot alone confirm a theory or trading opportunity.

15. Exercises

Full answers appear in Section 18. Variables are same-interval simple returns, in decimals unless otherwise stated.

14-E01 | Pricing and prediction. In one sentence, distinguish one-period SDF pricing from the CAPM expected-return relation. What can neither guarantee alone?

14-E02 | Hand calculation of an SDF. Two equally likely states have discount factors 1.1 and 0.9. A security pays 0 and 2 respectively. Find its price, risk-free gross factor and simple return, and its own expected gross factor and simple return. Explain the premium's sign.

14-E03 | Risk and beta. Asset–market excess-return covariance is zero and market variance positive, while asset excess-return variance is 0.04. What CAPM expected excess return follows? Is total risk zero?

14-E04 | SML and CML. With $R_f = 2\%$, $\mathbb{E}[R_m] = 8\%$, and $\sigma_m = 10\%$ (σ_m is independently given here, rather than estimated from Section 8.2), find CAPM expected return for beta=1.5 and expected return for an efficient market/RF combination with 5% standard deviation. Identify each line.

14-E05 | Probabilities and state prices. Using Section 8.1, give both state prices and explain why probabilities alone do not price X. If only Y is traded, paying EUR100 in both states at a price of EUR100, are both state prices uniquely identified? Give two strictly positive weight vectors.

14-E06 | Two branches with equal volatility. Keep Section 5's RF=2%, expected market return 8%, and volatility 10%; set market weight to -0.5. Find expected portfolio return, standard deviation, and beta. Why does the positive-slope CML not give 5%?

16. Programming Exercises

14-P01 | Reproduce the constructed example. Call the core constructor and calculate sample means and covariance in order market, A excess, B excess. Obtain both betas as covariance ratios and fit both residuals. Check means 6%, 5%, 5%; betas 0.5 and 1.5; and residuals 1%, -2%, 2%, -1%.

14-P02 | Missing values. Replace a P01 observation with `NaN` and call `fit_capm_excess`. Test explicit rejection of nonfinite values, without silently dropping rows. Restore the data and confirm unchanged results.

14-P03 | Shared monthly estimation. Construct $y_t = 0.001 + 1.2x_t + e_t$ from 60 observations on a monthly `PeriodIndex('M')` and estimate one factor with `fit_monthly_factor_model`. RF must be zero and present in the factor table; use HAC maxlags=5. Assert 60 observations, beta near 1.2, and residual mean near zero; print that the lag unit is monthly rows.

14-P04 | Historical data audit. Run the chapter script and read `ch14_french_capm_2010_2019.json`. Automatically check endpoints, 120 rows, monthly frequency, both file hashes, and the single-factor CAPM label. Do not relabel `alpha_monthly` as net strategy return.

17. Challenge Problems

14-C01 | Local perturbation derivation. Derive CAPM from tangency Sharpe maximization and feasible local perturbations. State the budget constraint, mean and variance derivatives, how the risk-free asset supports a common tangency portfolio, and where clearing identifies the market portfolio.

14-C02 | Zero beta with positive risk. Construct returns uncorrelated with market excess returns but having strictly positive variance. Explain the CAPM expected excess-return condition, its SML meaning, and risk conclusions that do not follow.

14-C03 | Estimated alpha and trading. A monthly intercept is 0.3% with HAC standard error 0.1%, reference (t=3). Give at least four further checks needed to move from a large estimated intercept to an implementable net trading opportunity.

14-C04 | Limits of the Roll critique. Explain why testing a linear return–beta relation with a market proxy also tests that proxy's mean–variance efficiency. Identify an overstatement the critique does not support and one design improvement.

18. Solutions / Solution Hints

14-E01 Answer. SDF pricing weights state payoffs to obtain today's price. CAPM expresses expected asset excess return as market beta times the expected market premium under specific equilibrium assumptions. Neither alone predicts a future month's realized return or establishes strategy profitability after financing and trading costs.

14-E02 Answer. Equal probabilities give $\mathbb{E}[m] = 0.5(1.1 + 0.9) = 1$, hence risk-free gross factor $G_f = 1/\mathbb{E}[m] = 1$ and simple return $r_f = G_f - 1 = 0$. The security costs $P = \mathbb{E}[mX] = 0.5(1.1 \cdot 0 + 0.9 \cdot 2) = 0.9$. Its gross factor is 0 or $2/0.9 = 2.2222$, averaging 1.1111; simple returns are -100% or 122.22%, with expectation 11.11%. Relative to $r_f = 0$, its expected simple premium is 11.11 percentage points. It pays only in the lower-discount-factor state and therefore covaries negatively with m. This two-state example introduces no CAPM market-state assumption.

14-E03 Answer. Beta is $0/\operatorname{Var}(R_m - R_f) = 0$, giving zero CAPM expected excess return without setting the market premium to zero. Asset variance is 0.04, standard deviation $\sqrt{0.04} = 0.2$ or 20% per period, so total risk is positive. CAPM interprets it as risk lacking linear comovement with this proxy.

14-E04 Answer. SML gives $2\% + 1.5(8\% - 2\%) = 11\%$. The efficient combination has market weight $w = \sigma_p/\sigma_m = 5\%/10\% = 0.5$ and CML expected return $2\% + 0.5(8\% - 2\%) = 5\%$. The former prices beta 1.5; the latter relates standard deviation to return only for that efficient combination.

14-E05 Answer. $q = (0.45, 0.55)$ and $P_X = 0.45 \times 110 + 0.55 \times 90 = 99$ euros. Probability weights alone give EUR100, an expected payoff rather than risk-adjusted price. With only Y traded, the restriction is $100(q_1 + q_2) = 100$. Both $(0.45, 0.55)$ and $(0.5, 0.5)$ are strictly positive and satisfy it, but price unspanned X at EUR99 and EUR100. Y's no-arbitrage price therefore does not uniquely identify state prices. The two valuations of an untraded X are not themselves executable arbitrage.

14-E06 Answer. $\mathbb{E}[R_p] = 2\% - 0.5 \times 6\% = -1\%$ and $\sigma_p = |-0.5| \times 10\% = 5\%$; with constant RF, $\beta_p = w = -0.5$. SML consistently gives $2\% - 0.5 \times 6\% = -1\%$. Positive-slope CML selects $w \geq 0$, whereas this portfolio lies on the lower branch. Total standard deviation alone loses exposure direction.

14-P01 Reference implementation.

```
import numpy as np
from quant_course.asset_pricing import (
    constructed_two_asset_sample,
    fit_capm_excess,
)

sample = constructed_two_asset_sample()
returns = np.column_stack([
    sample["Mkt-RF"],
    sample["Asset_A"] - sample["RF"],
    sample["Asset_B"] - sample["RF"],
])
mean_vector = returns.mean(axis=0)
covariance = np.cov(returns, rowvar=False, ddof=1)
fit_a = fit_capm_excess(returns[:, 1], returns[:, 0])
fit_b = fit_capm_excess(returns[:, 2], returns[:, 0])
assert np.allclose(mean_vector, [0.06, 0.05, 0.05])
assert np.allclose(covariance[1:, 0] / covariance[0, 0], [0.5, 1.5])
assert np.allclose(fit_a.residuals, [0.01, -0.02, 0.02, -0.01])
assert np.allclose(fit_b.residuals, fit_a.residuals)
```

In squared decimal-return units the sample covariance is $\begin{bmatrix} .008333 & .004167 & .012500 \\ .004167 & .002417 & .006583 \\ .012500 & .006583 & .019083 \end{bmatrix}$. The code uses order `[Mkt-RF, Asset_A_excess, Asset_B_excess]`; covariance and variance both use `ddof=1`, and their ratio gives beta.

14-P02 Reference test.

```
import numpy as np
import pytest
from quant_course.asset_pricing import constructed_two_asset_sample, fit_capm_excess

sample = constructed_two_asset_sample()
market = sample["Mkt-RF"].to_numpy()
asset = (sample["Asset_A"] - sample["RF"]).to_numpy(copy=True)
asset[1] = np.nan
with pytest.raises(ValueError, match="must be finite"):
    fit_capm_excess(asset, market)
asset[1] = 0.005
restored = fit_capm_excess(asset, market)
assert np.isclose(restored.beta, 0.5)
assert np.allclose(restored.residuals, [0.01, -0.02, 0.02, -0.01])
```

The test confirms explicit failure on a bad observation. Restoring the second excess return to 0.005 gives beta 0.5 and residuals `[.01, -.02, .02, -.01]`. Missing data do not authorize shortening the sample.

14-P03 Reference implementation.

```
import numpy as np
import pandas as pd
from quant_course.factor_models import fit_monthly_factor_model

month = pd.period_range("2015-01", periods=60, freq="M", name="month")
x = pd.Series(np.linspace(-0.08, 0.08, 60), index=month, name="Mkt-RF")
e = pd.Series(0.002 * np.sin(np.arange(60)), index=month)
rf = pd.Series(0.0, index=month, name="RF")
y = pd.Series(0.001 + 1.2 * x.to_numpy() + e.to_numpy(), index=month, name="asset")
fit = fit_monthly_factor_model(
    y,
    pd.concat([x, rf], axis=1),
    factor_columns=("Mkt-RF",),
    hac_maxlags=5,
)
assert fit.nobs == 60
assert abs(fit.coefficient_table.loc["Mkt-RF", "estimate"] - 1.2) < 0.01
assert abs(float(fit.residuals.mean())) < 1e-12
assert fit.hac_maxlags == 5  # five adjacent monthly observation rows
```

The deterministic sine error is not perfectly orthogonal to x, so beta need only be close to 1.2. Adjacent labels are consecutive monthly Periods, without artificial equal-second timestamps. HAC lag=5 means monthly rows. The helper subtracts RF once from asset returns; Mkt-RF already represents excess return and is not adjusted again.

14-P04 Reference checks. Command `.venv/bin/python code/ch14_asset_pricing.py` verifies manifest hashes before writing two QA JSONs and three charts. The frozen French record identifies `2010-01`, `2019-12`, `n_months=120`, `frequency=monthly`, and the single Mkt-RF factor. Hashes must match Section 11; investigate changed snapshots or processed files rather than altering expected values to hide differences. JSON alpha is a time-series intercept, never net strategy return.

14-C01 Reference derivation. Weights $(1-\varepsilon, \varepsilon)$ sum to 1, preserving the budget when reallocating a small position from M to i. The market tangency Sharpe ratio is locally maximal along these weights. The mean derivative is $\mu_i - \mu_m$, variance derivative $2[\mathrm{Cov}(R_i, R_m) - \sigma_m^2]$, and standard-deviation derivative divides the latter by $2\sigma_m$. Differentiate $S(\varepsilon) = [\mu_p(\varepsilon) - R_f]/\sigma_p(\varepsilon)$ and set the result to zero to obtain $0 = (\mu_i - \mu_m) - [(\mu_m - R_f)/\sigma_m^2][\mathrm{Cov}(R_i, R_m) - \sigma_m^2]$. Rearranging yields $\mu_i - R_f = [\mathrm{Cov}(R_i, R_m)/\sigma_m^2] \cdot (\mu_m - R_f)$; the covariance ratio defines beta. Weight sums preserve the budget; common mean–variance expectations and RF produce a common tangency portfolio; clearing identifies it as the market. Infeasible two-sided perturbations, incomplete asset coverage, or heterogeneous expectations prevent this argument from identifying the relation.

14-C02 Reference construction. In four equally likely states, take market excess returns $X = (-1\%, -1\%, 1\%, 1\%)$ and asset excess returns $Y = (1\%, -1\%, 1\%, -1\%)$. Both means are zero; state products $(-0.0001, 0.0001, 0.0001, -0.0001)$ average to zero covariance. Asset variance is 0.0001 and standard deviation 1%, strictly positive. Positive market variance gives beta $0/\mathrm{Var}(X) = 0$. Under CAPM, expected excess return is zero and SML gives RF at beta=0, while the market premium may remain nonzero. Total asset volatility is still 1%. This example does not establish zero volatility, no downside tails, or guaranteed principal.

14-C03 Reference answer. Check at least: (1) units, time zones, frequency, RF alignment, and whether the market factor already subtracts RF; (2) proxy coverage, omitted common risks, and parameter instability; (3) sample selection, all tried assets/frequencies/windows, holdouts, and multiple testing; (4) HAC bandwidth, residual dependence, structural changes, and small-sample approximations; (5) signal availability and executable trades; (6) fees, spreads, impact, financing, borrowing, taxes, and capacity; (7) incremental risk-adjusted results against a simple investable benchmark. Even a stable expected alpha requires an executable portfolio and risk budget. $t = 3$ is a reference regression statistic, rather than proof of free arbitrage.

14-C04 Reference answer. Roll's original critique notes that if the proxy used as the market is not mean–variance efficient, departures from a linear mean–beta relation can reflect either pricing failure or proxy inefficiency. The exact theoretical market is not fully observable, so empirical tests impose joint restrictions on a proxy. This does not imply that all CAPM tests are invalid or all proxy results uninformative. Define the investable universe and proxy coverage explicitly, test proxy efficiency alongside cross-sectional restrictions, and report sensitivity across prespecified plausible proxies rather than selecting the most favorable result.

19. Further Reading

- **Must Know:** Read the Sharpe (1964) passages on mean–variance preferences, risk-free borrowing/lending, common expectations, efficient portfolios, and the return–beta relation. Source notes delimit actual reading. They support the theory without proving its assumptions hold in data.
- **Should Know:** Princeton FIN501 one-period pricing slides 43–66 explain SDFs, state prices, no-arbitrage, and additional beta-pricing restrictions. MIT OCW 15.401 CAPM slides 3–10, 15–16, 18–27 review diversification, SML/CML, and empirical issues.
- **Advanced:** Roll (1977), original abstract and market-portfolio testability argument, explains the proxy joint-test limit. Read the original paper for the full argument rather than relying only on this chapter.
- **Optional:** Markowitz (1952) publication metadata establish the historical source of mean–variance theory; the preparation did not read its full text page by page. Fama and French (2004) official abstract introduces a later empirical review, without this chapter restating the full paper.

20. References

1. Sharpe, W. F. (1964). "Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk." *The Journal of Finance*, 19(3), 425–442. Original article and publication metadata. Supports mean–variance assumptions, efficient portfolios, and market risk; actual reading scope is in `source_notes.md`.
2. Markowitz, H. (1952). "Portfolio Selection." *The Journal of Finance*, 7(1), 77–91. Wiley metadata and DOI. Metadata only were checked, without full page-by-page reading.
3. Princeton FIN501. Lecture 02: One-Period Asset Pricing, slides 43–66. Supports general pricing kernels and SDFs; all technical conditions of the fundamental theorem are not developed here.

4. Lo, A. W. MIT OCW 15.401. The CAPM and APT, specified slides. See source notes for reading and formula-adaptation scope.
5. Roll, R. (1977). "A Critique of the Asset Pricing Theory's Tests Part I: On Past and Potential Testability of the Theory." *Journal of Financial Economics*, 4(2), 129–176. DOI, UCLA scan. This chapter uses metadata and the first-page abstract to explain the proxy joint-test limit.
6. Fama, E. F., and K. R. French (2004). "The Capital Asset Pricing Model: Theory and Evidence." *Journal of Economic Perspectives*, 18(3), 25–46. AEA official page and abstract. Metadata and abstract only were checked.
7. statsmodels official OLS and robust covariance results documentation. It supports software calls, without selecting this study's model, HAC bandwidth, or economic interpretation.
8. Kenneth R. French Data Library. 10 Industry Portfolios, Fama/French Factors. See Section 11, source notes, and manifests for snapshot scope. Official methods do not confer historical PIT eligibility.

Chapter 15 — Factor Models

Prerequisites: Chapter 8 regression, Chapter 11 returns, Chapters 12–13 statements and valuation, and Chapter 14 asset pricing.

1. Learning Objectives

When a portfolio earns more than cash, how much compensates for market and other common risks, and how much remains unexplained by the chosen model? Move from CAPM to the Fama–French Three-Factor Model (FF3) and Five-Factor Model (FF5), understand the construction of size, value, profitability, and investment factors, and complete a historical monthly regression independently.

Distinguish factors, factor loadings, exposures, intercept alpha, and residual risk. Derive how omitted variables change alpha, identify assumptions behind risk decomposition, and explain why statistical significance still does not establish tradable net profit. The historical case uses actual Kenneth French industry and factor snapshots, without inventing stock trades.

2. Why Quants Need This

A portfolio with strong historical performance may simply hold high-market-beta stocks, favor small companies, or bear value or profitability risks. Establish a benchmark before naming that compensation a unique skill.

Chapter 13 constructs cheapness characteristics from statements and prices matched to the same claims. Here we ask how a portfolio holding those characteristics moves with the market and other styles. Company characteristics and common portfolio-return variation serve different purposes. Chapters 23–28 later turn characteristics into signals available at decision time. This chapter explains risk and attributes returns; a realized same-month factor return is not a forecast input available beforehand.

Factor models also support risk management. Many stock names need not diversify risk if they share exposures. A low-dimensional coordinate system helps portfolio construction; Chapters 20–21 use it for risk and constraints, subject to data quality, out-of-sample behavior, and costs.

3. Intuition

Think of portfolio returns as an ensemble: the market is a melody most stocks respond to, and other factors capture different parts. Loadings measure response strength, rather than the fraction of capital invested in each part. Residuals contain firm events, measurement error, omitted factors, and noise. Calling the unexplained part exceptional skill is premature.

Why go beyond a market regression? Firms with similar market sensitivity may share persistent variation through financing, valuation, profitability, or reinvestment. Adding a factor tests whether this comovement explains variation previously left in residuals. A better-looking historical fit alone supplies no evidence of future validity.

A rising value factor does not mean every low-P/E stock must rise. Factors are rule-based portfolio-return differences and can be positive or negative. A company characteristic, regression exposure, and next-month return are not linked by a mechanical identity.

4. Mathematical Foundations

4.1 From One Asset to N Assets

For asset i's simple total return in month t, write:

$$y_{i,t} = R_{i,t} - R_{f,t} = \alpha_i + \boldsymbol{\beta}_i^\top \boldsymbol{f}_t + \varepsilon_{i,t}.$$

$R_{i,t}$ is its monthly total return and $R_{f,t}$ a same-currency cash proxy over the same interval, both decimal returns per month. $y_{i,t}$ is arithmetic excess return. The K×1 factor-return vector $\boldsymbol{f}_t$ and dimensionless K×1 loading vector $\boldsymbol{\beta}_i$ form an inner product; $\top$ transposes the latter into a row. The result is explained monthly excess return. α_i is a decimal monthly intercept and $\varepsilon_{i,t}$ the monthly residual.

For N assets together:

$$\boldsymbol{y}_t = \boldsymbol{\alpha} + B\boldsymbol{f}_t + \boldsymbol{\varepsilon}_t.$$

$\boldsymbol{y}_t, \boldsymbol{\alpha}, \boldsymbol{\varepsilon}_t$ are N×1, while B is N×K, with one asset's loadings per row. Bf is N×1. Dimensions prevent confusing the number of assets with the number of factors. N can be large while K is usually much smaller, giving factor risk models their dimension reduction.

4.2 Expectations and Volatility Answer Different Questions

With fixed loadings and intercepts, finite second moments, and zero population residual mean:

$$\mathbb{E}[y_i] = \alpha_i + \boldsymbol{\beta}_i^\top \boldsymbol{\mu}_f, \qquad \boldsymbol{\mu}_f = \mathbb{E}[\boldsymbol{f}_t].$$

$\boldsymbol{\mu}_f$ is the expected monthly return vector of K factors. This is an expectation decomposition. Claiming that the model correctly prices every asset additionally requires $\alpha_i = 0$ and an account of how factors represent priced risks. High statistical explanatory power alone does not establish that pricing claim.

Volatility concerns deviations from means; constant alpha adds no variance. If factors and residuals are uncorrelated:

$$\Sigma_y = B\Sigma_f B^\top + \Sigma_\varepsilon.$$

Σ_y is the N×N asset excess-return covariance matrix, Σ_f the K×K factor covariance, and Σ_ε the N×N residual covariance, all in squared monthly-return units. Shared residual shocks make Σ_ε nondiagonal. A diagonal approximation is a modeling choice; the word residual does not justify ignoring residual correlations.

In general let $C = \mathrm{Cov}(\boldsymbol{f}, \boldsymbol{\varepsilon})$, a K×N matrix. The full expression is

$$\Sigma_y = B\Sigma_f B^\top + \Sigma_\varepsilon + BC + C^\top B^\top.$$

BC is N×N and the other cross term is its transpose. Expanding $(B(f-\mu_f)+(\varepsilon-\mu_\varepsilon))(\cdots)^\top$ yields factor risk, residual risk, and both cross-covariances. Only $C = 0$ recovers the simpler formula. Factor–residual covariance C differs from cross-asset residual covariance Σ_ε.

5. Formal Definitions

5.1 CAPM, FF3, and FF5

The CAPM time-series regression uses market excess return as its sole factor:

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{M,i}(R_{M,t} - R_{f,t}) + \varepsilon_{i,t}.$$

R_M is the chosen market proxy's monthly return and β_M its loading. Chapter 14 discusses equilibrium assumptions; here published proxies estimate historical linear relations. A proxy is not the aggregate-wealth market portfolio, so a zero-intercept test jointly depends on proxy and specification.

FF3 adds size and value portfolio-return spreads:

$$y_{i,t} = \alpha_i + \beta_{M,i}MKT_t + s_i SMB_t + h_i HML_t + \varepsilon_{i,t}.$$

$MKT_t = R_{M,t} - R_{f,t}$: Small Minus Big (SMB) is the size spread and High Minus Low (HML) the high-minus-low book-to-market value spread. s_i, h_i are dimensionless loadings. High or low stock characteristics differ from positive or negative long–short returns: negative monthly HML means its value side lost to its growth side that month.

FF5 adds Robust Minus Weak (RMW) profitability and Conservative Minus Aggressive (CMA) investment:

$$y_{i,t} = \alpha_i + \beta_{M,i}MKT_t + s_i SMB_t + h_i HML_t + r_i RMW_t + c_i CMA_t + \varepsilon_{i,t}.$$

r_i, c_i are profitability and investment loadings, not discount rates r. Subscripted loadings here belong to regressions. Conservative and aggressive describe company investment growth, rather than trader courage. FF5 constructs SMB differently from FF3, as explained below.

5.2 From Characteristic Sorts to Factor Returns

To avoid confusing value exposure with greater small-cap exposure, first split stocks into Small/Big, then each size group into Value/Neutral/Growth by book-to-market. Denote the six monthly portfolio returns SV, SN, SG, BV, BN, BG. Returns within each group are value-weighted; the outer combinations below use equal weights.

$$SMB_{BM} = \frac{SV + SN + SG}{3} - \frac{BV + BN + BG}{3}, \qquad HML = \frac{SV + BV}{2} - \frac{SG + BG}{2}.$$

Every symbol is a same-month decimal return. These formulas subtract portfolio returns rather than six companies' statement figures. Size averages across value groups; value averages across size groups. This reduces confounding without guaranteeing statistical orthogonality. Record within-group and between-group weights separately when replicating factors. Official French three-factor construction

Profitability groups are Robust/Neutral/Weak; investment groups are Conservative/Neutral/Aggressive. Construct size spreads SMB_{OP} and SMB_{INV} by the same three-group averaging. FF5 uses:

$$SMB_5 = \frac{SMB_{BM} + SMB_{OP} + SMB_{INV}}{3}.$$

All three are monthly small-minus-big spreads. Let SR, BR, SW, BW denote small/big robust/weak profitability groups, and SC, BC, SA, BA small/big conservative/aggressive investment groups. Then:

$$RMW = \frac{SR + BR}{2} - \frac{SW + BW}{2}, \qquad CMA = \frac{SC + BC}{2} - \frac{SA + BA}{2}.$$

Each portfolio retains monthly-return units; denominators 2 and 3 are dimensionless equal-weight counts. This explains construction without claiming replication from stock-level statements and historical universes. The chapter reads published factors directly. Official annual sorts use lagged financial eligibility and July–June holding periods, unlike backfilling all current restatements into history. Official French five-factor description

5.3 Factor Returns, Characteristics, Loadings, and Positions

Object	Input → operation → output	Units and availability clock
Stock financial characteristic	Public book equity and market value on consistent definitions → ratio → book-to-market 0.6	Dimensionless; check publication, ingestion, and price times
Sorted portfolio	Characteristics known at cutoff and eligible universe → sorting and within-group weights → monthly SV-type return	Decimal/month; determine groups and holdings before returns realize
Factor return	Same-month group returns → high-minus-low/small-minus-big → HML-type spread	Decimal/month; observed after the month is realized and published
Regression loading	Realized asset excess returns and same-month factors → specified-window OLS → h=0.8	Dimensionless; estimated after training-window cutoff
Position weight	Available signals, risk budget, constraints → portfolio decision → stock at 4% of NAV	Dimensionless; record decisions and actual trades separately

Net Asset Value (NAV) is the weight denominator. No identity maps a loading directly to holdings: objectives, constraints, and costs are also needed. Characteristics form sorted groups, their returns form factors, and factor returns estimate loadings. Regression cannot reconstruct the stocks you actually bought.

Define exposure explicitly: it may mean a regression beta or a holdings-weighted characteristic. One is estimated from return comovement; the other uses statements and holdings. They may correlate without being the same number. Loadings need not lie between 0 and 1 or sum to 1. Highly correlated factors can destabilize individual loadings even when aggregate fitted returns remain similar.

6. Derivation

6.1 Why the Fitted Intercept Can Fall

Use an included factor x and omitted factor z. Suppose the true statistical relation is:

$$y = \alpha + \beta x + \gamma z + u, \quad \mathrm{Cov}(u, x) = \mathrm{Cov}(u, z) = 0.$$

x, z, y are same-month return variables; alpha is the intercept, beta and gamma loadings, and u a zero-mean error. Regressing only on x gives the population projection slope:

$$\widetilde{\beta} = \frac{\mathrm{Cov}(x, y)}{\mathrm{Var}(x)} = \beta + \gamma \frac{\mathrm{Cov}(x, z)}{\mathrm{Var}(x)}.$$

First apply the population single-regressor projection with an intercept, then substitute y inside covariance. Constants have zero covariance and Cov(x,u)=0. With $q = \mathrm{Cov}(x, z)/\mathrm{Var}(x)$, x absorbs some omitted-factor variation. This requires Var(x)>0.

The intercept equates fitted and observed means, so:

$$\widetilde{\alpha} = \mathbb{E}[y] - \widetilde{\beta}\mathbb{E}[x] = \alpha + \gamma\{\mathbb{E}[z] - q\mathbb{E}[x]\}.$$

An omitted factor can raise or lower the intercept; adding z can also increase alpha. A positive intercept is not automatically skill, nor does its disappearance mean the model removed actual profits. The benchmark and projection changed; investors' cash flows did not reverse when the researcher changed models.

6.2 Multivariate OLS and Loadings

For T months, the T×(K+1) design X has an initial column of ones and contemporaneous factor-return columns; y is T×1 asset excess return. Solve:

$$\widehat{\theta} = \arg\min_{\theta}(y - X\theta)^{\top}(y - X\theta).$$

$\theta = (\alpha, \beta_1, \ldots, \beta_K)^{\top}$ is a K+1 parameter vector. Squared historical explanation error is not trading profit. Expanding and differentiating gives $-2X^{\top}(y - X\theta) = 0$, hence:

$$X^{\top}X\widehat{\theta} = X^{\top}y, \qquad \widehat{\theta} = (X^{\top}X)^{-1}X^{\top}y.$$

The inverse expression requires full column rank and T>K+1; no factor may be an exact linear combination of the others. Actual software should solve with stable QR or SVD methods rather than manual inversion. Full rank identifies parameters without guaranteeing precision; near-collinearity still amplifies uncertainty. This links Chapter 5's rank and singular values to Chapter 8 regression and risk explanation.

The normal equations give $X^{\top}\widehat{\varepsilon} = 0$. The ones column makes residuals sum to zero; factor columns give in-sample orthogonality. This OLS algebra does not promise uncorrelated future residuals.

6.3 When Risk Decomposition Holds

For one asset, let $v = \boldsymbol{\beta}^{\top}\boldsymbol{f}$. Then:

$$\mathrm{Var}(y) = \mathrm{Var}(v) + \mathrm{Var}(\varepsilon) + 2\,\mathrm{Cov}(v, \varepsilon).$$

Constant alpha has no variance. Center y and expand its squared expectation. If Cov(f,epsilon)=0, the third term vanishes and the first expands as $\boldsymbol{\beta}^{\top}\Sigma_f\boldsymbol{\beta}$. Correlated factors also require cross term $2\beta_1\beta_2\,\mathrm{Cov}(f_1, f_2)$; individual volatilities cannot simply be added.

With an intercept, in-sample OLS fitted values and residuals are orthogonal. Using the same T−1 sample-variance denominator:

$$s_y^2 = s_{\widehat{y}}^2 + s_{\widehat{\varepsilon}}^2.$$

All three s² terms are centered variances over the same sample and denominator T−1. An unbiased regression-noise variance estimate may divide residual sum of squares by T−K−1 instead. That different denominator cannot be inserted directly into this identity; this explains apparent discrepancies between software residual variance and hand-calculated risk.

For fixed portfolio weights w, factor exposure is $B^\top w$ and portfolio variance:

$$\sigma_p^2 = (B^\top w)^\top \Sigma_f (B^\top w) + w^\top \Sigma_\varepsilon w.$$

w is N×1; B and Sigma have Section 4's dimensions. The first term is common-factor risk and the second residual risk, both squared monthly returns. This requires fixed weights, linear return aggregation, and orthogonality. Dynamic weights, financing, costs, and nonlinear option exposures need a fuller model.

6.4 Why Independent-Sample Standard Errors May Fail

Factors and residuals can have serial dependence and heteroskedasticity. OLS still minimizes squared error, while independent homoskedastic standard errors may understate uncertainty. We use heteroskedasticity and autocorrelation consistent covariance (HAC), monthly-row lag L=6, and Bartlett weights $a_\ell = 1 - \ell/(L+1)$.

Let $q_t = x_t \widehat{\varepsilon}_t$, where x_t is the K+1 design column including the constant, and define score cross-products $\Gamma_\ell = \sum_{t=\ell+1}^{T} q_t q_{t-\ell}^\top$. Before finite-sample correction, the sandwich is:

$$\widehat{V} = (X^\top X)^{-1} \left[\Gamma_0 + \sum_{\ell=1}^{L} a_\ell (\Gamma_\ell + \Gamma_\ell^\top) \right] \cdot (X^\top X)^{-1}.$$

V is (K+1)×(K+1) parameter covariance; square roots of diagonal entries are standard errors. Lagged score products capture cross-month dependence; downweighting distant lags limits finite-sample noise. Software additionally applies finite-sample correction and a t reference. This remains conditional approximate inference, without immunity to arbitrarily strong dependence. Neither 120 months nor six lags is universally optimal. Transparently compare 3, 6, 12 rather than choosing L by the most favorable p-value.

To see the sandwich components, use Section 8.4's four-month constructed regression. The table repeats only factors and OLS residuals needed for HAC; Section 8.4 gives asset excess returns and their derivation. There is a constant and one factor, parameters ordered (alpha, beta), with T=4 and p=2. L=1 is chosen solely for this teaching calculation; the historical French case retains L=6.

Month	Factor x (decimal/month)	OLS residual (decimal/month)
1	−0.020	0.001
2	0.000	−0.001
3	0.020	−0.001
4	0.040	0.001

With design order (constant, x), each score q_t is the design vector times the residual. The four rows form:

$$\begin{bmatrix} q_1^\top \\ q_2^\top \\ q_3^\top \\ q_4^\top \end{bmatrix} = 10^{-3} \begin{bmatrix} 1 & -0.02 \\ -1 & 0 \\ -1 & -0.02 \\ 1 & 0.04 \end{bmatrix}.$$

Multiplication gives the lag-zero and lag-one score cross-products:

$$\Gamma_0 = \begin{bmatrix} 4.0 \times 10^{-6} & 4.0 \times 10^{-8} \\ 4.0 \times 10^{-8} & 2.4 \times 10^{-9} \end{bmatrix}.$$

$$\Gamma_1 = \begin{bmatrix} -1.0 \times 10^{-6} & 0 \\ -2.0 \times 10^{-8} & -8.0 \times 10^{-10} \end{bmatrix}.$$

For L=1, Bartlett weight $a_1 = 1 - 1/(1+1) = 0.5$. The middle sandwich matrix (meat) is therefore:

$$M = \Gamma_0 + 0.5(\Gamma_1 + \Gamma_1^\top) = \begin{bmatrix} 3.0 \times 10^{-6} & 3.0 \times 10^{-8} \\ 3.0 \times 10^{-8} & 1.6 \times 10^{-9} \end{bmatrix}.$$

The design cross-product and its inverse, ordered constant then beta, are:

$$X^\top X = \begin{bmatrix} 4 & 0.04 \\ 0.04 & 0.0024 \end{bmatrix}, \qquad (X^\top X)^{-1} = \begin{bmatrix} 0.3 & -5 \\ -5 & 500 \end{bmatrix}.$$

Multiplying on both sides gives the uncorrected covariance and software finite-sample-corrected result:

$$\widehat{V}_{\text{raw}} = \begin{bmatrix} 2.20 \times 10^{-7} & -3.25 \times 10^{-6} \\ -3.25 \times 10^{-6} & 3.25 \times 10^{-4} \end{bmatrix}.$$

$$\widehat{V}_{\text{HAC}} = \frac{T}{T-p}\widehat{V}_{\text{raw}} = 2\widehat{V}_{\text{raw}} = \begin{bmatrix} 4.40 \times 10^{-7} & -6.50 \times 10^{-6} \\ -6.50 \times 10^{-6} & 6.50 \times 10^{-4} \end{bmatrix}.$$

p=2 includes the intercept, so $T/(T-p) = 4/(4-2) = 2$. Alpha's standard error is $\sqrt{4.40 \times 10^{-7}} = 0.0006633$ in decimal monthly returns; beta's is $\sqrt{6.50 \times 10^{-4}} = 0.025495$, dimensionless. The statsmodels t reference uses degrees of freedom $T - p = 2$: alpha t=3.0151, p=0.09464, and 95% interval [−0.0008541, 0.0048541]. These illustrate software settings. Four observations cannot establish reliable inference; correction and a t reference do not validate error assumptions.

The following code independently recalculates every matrix and asserts agreement with the helper's statsmodels HAC standard errors, t statistics, p-values, and intervals. Inversion is used only to display this tiny example; actual regression uses stable OLS.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))

import numpy as np
import pandas as pd
import statsmodels.api as sm
from quant_course.factor_models import fit_monthly_factor_model

x = np.array([-0.020, 0.000, 0.020, 0.040])
y = np.array([-0.013, 0.001, 0.017, 0.035])
design = np.column_stack([np.ones(len(x)), x])
params = np.linalg.lstsq(design, y, rcond=None)[0]
residual = y - design @ params
scores = design * residual[:, None]

gamma0 = scores.T @ scores
gamma1 = sum(
    np.outer(scores[t], scores[t - 1]) for t in range(1, len(scores))
)
weight1 = 1 - 1 / (1 + 1)
meat = gamma0 + weight1 * (gamma1 + gamma1.T)
bread = np.linalg.inv(design.T @ design)
raw_cov = bread @ meat @ bread
correction = len(y) / (len(y) - design.shape[1])
manual_cov = correction * raw_cov

np.testing.assert_allclose(params, [0.002, 0.8], atol=1e-14)
np.testing.assert_allclose(residual, [0.001, -0.001, -0.001, 0.001], atol=1e-14)
np.testing.assert_allclose(gamma0, [[4e-6, 4e-8], [4e-8, 2.4e-9]], atol=1e-20)
np.testing.assert_allclose(gamma1, [[-1e-6, 0], [-2e-8, -8e-10]], atol=1e-20)
np.testing.assert_allclose(
    manual_cov,
    [[4.4e-7, -6.5e-6], [-6.5e-6, 6.5e-4]],
    rtol=1e-12, atol=1e-18,
)

months = pd.period_range("2020-01", periods=4, freq="M", name="month")
factors = pd.DataFrame({"Mkt-RF": x, "RF": np.zeros(4)}, index=months)
fit = fit_monthly_factor_model(
    pd.Series(y, index=months), factors, ("Mkt-RF",), hac_maxlags=1
)
robust = sm.OLS(y, design).fit().get_robustcov_results(
    cov_type="HAC", maxlags=1, kernel="bartlett",
    use_correction=True, use_t=True,
)
np.testing.assert_allclose(robust.cov_params(), manual_cov, rtol=1e-12, atol=1e-18)
np.testing.assert_allclose(
    fit.coefficient_table["standard_error_hac"].to_numpy(), robust.bse,
    rtol=1e-12, atol=1e-14,
)
np.testing.assert_allclose(
    fit.coefficient_table.loc["const", ["t_hac", "p_hac"]].to_numpy(dtype=float),
    [robust.tvalues[0], robust.pvalues[0]],
```

```
)
np.testing.assert_allclose(
    fit.coefficient_table.loc["const", ["ci_low_hac", "ci_high_hac"]].to_numpy(dtype=
    ↪ float),
    robust.conf_int()[0],
)
assert robust.df_resid == 2
print("V_HAC:", manual_cov)
print(fit.coefficient_table[["standard_error_hac", "t_hac", "p_hac", "ci_low_hac", "
    ↪ ci_high_hac"]])
```

Scope of fixed-loading formulas

6.5 Time-Varying Loadings: Do Not Multiply Averages First

The preceding mean attribution uses fixed $\boldsymbol{\beta}$. If states or holdings make loadings $\boldsymbol{\beta}_t$, generally

$$\mathbb{E}[\boldsymbol{\beta}_t^\top \boldsymbol{f}_t] = \mathbb{E}[\boldsymbol{\beta}_t]^\top \mathbb{E}[\boldsymbol{f}_t] + \sum_{k=1}^{K} \operatorname{Cov}(\beta_{k,t}, f_{k,t}).$$

Expand each product expectation to obtain this result. Dimensionless loadings and decimal monthly factors make the covariance term a decimal monthly return. Mean loading times mean factor return is not the average contribution of an arbitrary dynamic portfolio. In two equally likely teaching states, take $(\beta, f) = (0.5, -2\%)$ and $(1.5, 4\%)$ with zero intercept and error. Contributions are -1% and 6%, averaging 2.5%; average loading 1 times average factor return 1% gives only 1%, missing a 1.5-percentage-point covariance term. This state construction provides no foreseeable rebalancing signal.

Nor may unconditional risk replace random B_t with average B in a fixed-loading formula. For a conditional model on pre-month information $\mathcal{I}_{t-1}$, require B_{t-1} to be known, conditional residual mean zero, and conditional factor–residual orthogonality before writing

$$\operatorname{Var}(\boldsymbol{y}_t \mid \mathcal{I}_{t-1}) = B_{t-1} \operatorname{Var}(\boldsymbol{f}_t \mid \mathcal{I}_{t-1}) B_{t-1}^\top + \operatorname{Var}(\boldsymbol{\varepsilon}_t \mid \mathcal{I}_{t-1}).$$

Unconditional variance additionally follows total variance: expected conditional variance plus variance of the conditional mean. Rolling estimation produces window-dependent loading estimates without proving conditional orthogonality or making this month's realized factors known beforehand. Chapters 19–21 further distinguish estimation error, conditional risk, and dynamic weights. This chapter's empirical analysis remains fixed-window contemporaneous attribution.

7. Financial Interpretation

7.1 A Risk-Premium Explanation Is Not Causal Proof

Positive factor means can relate to risk compensation, behavior, institutional constraints, or the selected sample. A regression describes return relations without deciding between mechanisms. Risk compensation needs adverse states, marginal investors, and an economic mechanism; behavioral mispricing needs an account of biases and limits to arbitrage. Correlation does not establish causation.

7.2 Why Alpha Is Relative

$\widehat{\alpha} = \bar{y} - \widehat{\boldsymbol{\beta}}^{\top} \bar{\boldsymbol{f}}$. Alpha answers how much average return remains for this sample, cash proxy, and factor set. A portfolio with market beta 0.8 and value loading 0.6 does not have positive alpha merely because it beats cash; first subtract model-explained compensation.

A significant intercept still requires scrutiny of assumptions, selection, omissions, and repeated attempts. Statistical significance differs from economic significance; insignificance does not prove alpha exactly zero. A small p-value is especially inadequate after testing many portfolios, models, and windows. Chapter 32 covers multiple testing.

7.3 Residuals Are Not a Free Lunch

Holding more unrelated assets may diversify residual risk, but zero in-sample residual mean follows from fitting an intercept rather than negligible event risk. Incomplete factors can leave many stocks exposed to shared liquidity or industry shocks. Treating every residual as independent noise can severely understate concentration.

8. Numerical Example

8.1 Hand Calculation of Factor Construction

These are constructed monthly returns: SV=4%, SN=3%, SG=2%, BV=2%, BN=1%, BG=0%. Small averages 3% and Big 1%, giving SMB_BM=2%. Value averages (4%+2%)/2= 3%; Growth averages (2%+0%)/2=1%, giving HML=2%. Equal factor returns this month do not make their long-run series identical, and one month cannot estimate correlation.

8.2 Explained Return and Alpha

Construct one month with RF=0.2%, asset R=2.6%, intercept $\alpha = 0.2\%$, market excess return MKT=2%, SMB=1%, HML=−1%, and loadings $\beta_M = 0.8, s = 0.3, h = 0.5$. Asset excess return is 2.4%. Let c denote factor-explained contribution:

$$c = 0.8 \times 2\% + 0.3 \times 1\% + 0.5 \times (-1\%) = 1.4\%,$$
$$\varepsilon = (R - RF) - (c + \alpha) = 2.4\% - 1.4\% - 0.2\% = 0.8\%.$$

The intercept is 0.2% per month and epsilon the unexplained monthly variation; all terms retain same-month return units.

This month's residual 0.8% is not its 0.2% intercept. A single month's unexplained variation differs from a regression intercept across time. Subtract cash once from asset total returns. MKT is already excess return and SMB/HML already spreads, so do not subtract RF again.

8.3 Common Risk by Hand

Construct two uncorrelated factors with monthly volatility 4% and 2%, loadings 0.8 and 0.5, and residual volatility 1%. Denote explained variance v_f and total variance v:

$$v_f = 0.8^2 \times 0.04^2 + 0.5^2 \times 0.02^2 = 0.001124, \qquad v = v_f + 0.01^2 = 0.001224.$$

v_f and v have squared monthly-return units. Taking the final square root gives about 3.4986% volatility. Square, add, and then take the root; adding 3.2%, 1%, and 1% to get 5.2% is incorrect.

With factor correlation 0.5, cross-covariance is $0.5 \times 0.04 \times 0.02 = 0.0004$. Denote the additional cross term d:

$$d = 2 \times 0.8 \times 0.5 \times 0.0004 = 0.00032, \qquad v_{corr} = 0.001224 + d = 0.001544.$$

d and v_corr have squared monthly-return units. Correlated-case monthly volatility is $\sqrt{v_{corr}}$, about 3.9294%. Correlation changes risk even with unchanged loadings.

8.4 A Four-Month Regression Ledger

Use one market factor to unpack Section 6.2's matrix algorithm into hand calculations. All values are constructed monthly decimal returns, without real companies or prediction tests. y already subtracts same-month cash:

Constructed month	x: market excess return	y: asset excess return
1	−0.020	−0.013
2	0.000	0.001
3	0.020	0.017
4	0.040	0.035

First calculate means $\bar{x} = 0.010$ and $\bar{y} = 0.010$. Centering matters because an intercept permits different levels; slope should reflect comovement around each mean rather than mistake average levels for covariance.

Second, x deviations are −0.030, −0.010, 0.010, 0.030; y deviations are −0.023, −0.009, 0.007, 0.025. Their product sum is 0.00069+0.00009+0.00007+0.00075=0.00160, and the x squared-deviation sum is 0.00090+0.00010+0.00010+0.00090=0.00200.

Third, slope $\widehat{\beta} = 0.00160/0.00200 = 0.8$. Sample covariance and variance both divide by 3, which cancels to give 0.8. One percentage point more market excess return raises fitted asset excess return by 0.8 percentage points; it does not mean an 80% capital allocation.

Fourth, restore levels: $\widehat{\alpha} = \bar{y} - 0.8\bar{x} = 0.002$, or 0.2% per month. Omitting the intercept forces slope to absorb mean differences and answers another question.

Fifth, calculate $\widehat{y} = 0.002 + 0.8x$ month by month: −0.014, 0.002, 0.018, 0.034. Subtract fitted values from y to obtain residuals 0.001, −0.001, −0.001, 0.001. Their sum and inner product with x are zero. These are solution orthogonality properties, rather than evidence that future errors are independent.

Finally, y's centered sum of squares is 0.001284 and residual sum of squares 0.000004, giving $R^2 = 1 - 0.000004/0.001284 \approx 0.996885$. High R^2 comes from deliberate construction, without market advantage. Multivariate OLS extends this comovement calculation by solving columns jointly, with each slope controlling the others. Section 6's matrix equations do this simultaneously.

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I 3 reader package root, establish its documented environment, then run:

```
.venv/bin/python run_notebook.py notebooks/ch15_factor_models.ipynb
```

Seed 20261015 constructs 120 monthly rows from 2010-01, five normal factor columns with standard deviation 0.025, RF=0.001, and one CONSTRUCTED asset equal to RF+0.002+1.0×Mkt-RF+0.3×HML plus normal error with standard deviation 0.01. CAPM, FF3, and FF5 use common underlying columns, making these constructed models strictly nested. Actual `build_summary(..., constructed=True)` checks all three ddof=1 variance identities and unchanged point estimates for HAC3/6/12. A coefficient CSV, summary JSON, and three new charts go to `runs/ch15_factor_models_<uuid>/`. CSV columns explicitly include `model` and `coefficient`. Charts show monthly alpha with HAC intervals, FF5 loadings with intervals, and arithmetic cumulative monthly attribution; titles use the CONSTRUCTED asset name. `RUN_FRENCH=False` disables loading historical French snapshots. Sections 10–11 retain the original historical NoDur charts and tables.

The runner separately records kernel execution in unique `runs/execution_*`, distinct from the Notebook's chapter UUID method directory. Constructed runs check interfaces without producing market research, PIT forecasts, or strategy returns after costs. Chapter 14 historical NoDur uses HAC5; Chapter 15 and Project 2 historical main tables use HAC6. Keep estimates and inference settings distinct rather than rewriting intervals for superficial agreement. This batch inspected source without executing these Notebooks or rerunning historical regressions.

I 3 Chapter 15 Notebook cells 4 and 18 still describe differences between official snapshot shared-name columns. This applies only to historical French data, rather than current common constructed columns. English Markdown for Chapters 14–15 and correction of those cells belong to BIL23; this batch leaves chapter Notebooks unchanged. The CLI and old production tests below refer to the working course root. Readers need not restore production tests absent from the package.

Historical Course Working Root and Original Implementation

9.1 Input Contracts Before Fitting

The historical entry is `ch15_factor_models.py`, with teaching `ch15_factor_models.ipynb`. From the course directory run `.venv/bin/python code/ch15_factor_models.py`; it writes a coefficient CSV and summary JSON with source hashes. These are experiment outputs, not replacement inputs for another regression.

The core is `fit_monthly_factor_model()` in `factor_models.py`. Asset input is a `Series` and factor input a `DataFrame`, both indexed by identical consecutive, unique, increasing monthly `PeriodIndex` labels. Original French percentages have already been converted to decimals in processed snapshots; do not divide by 100 again. NaN, infinity, booleans, complex values, missing months, and rank deficiency are rejected rather than hidden by zero filling.

April asset returns paired with May factors can still produce numbers, but answer a different financial question. Dropping missing rows after an outer join also changes the research window. Explicit failure is preferable to uninterpretable beta. A month labels a return interval, without claiming publication at its first instant.

Complete example: checking a noiseless constructed regression

This standalone constructed monthly example fixes seed 15. It verifies the estimator interface rather than backtesting a financial strategy.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
import numpy as np
import pandas as pd
from quant_course.factor_models import fit_monthly_factor_model

rng = np.random.default_rng(15)
months = pd.period_range("2000-01", periods=120, freq="M")
factors = pd.DataFrame({
    "Mkt-RF": rng.normal(0.005, 0.04, 120),
    "SMB": rng.normal(0.002, 0.02, 120),
    "HML": rng.normal(0.003, 0.02, 120),
    "RF": np.full(120, 0.001),
}, index=months)
asset = factors["RF"] + 0.002 + 0.8*factors["Mkt-RF"] + 0.3*factors["SMB"] +
    ↪ 0.5*factors["HML"]
fit = fit_monthly_factor_model(asset, factors, ("Mkt-RF", "SMB", "HML"),
    ↪ hac_maxlags=6)
assert np.allclose(fit.coefficient_table["estimate"], [0.002, 0.8, 0.3, 0.5])
assert np.max(np.abs(fit.residuals)) < 1e-14
print(fit.coefficient_table["estimate"])
```

Random error is deliberately absent so estimates can be checked exactly. Near-zero standard errors provide no real-world significance evidence. Financial returns contain noise; the historical confidence intervals below describe uncertainty in that experiment.

9.2 Inspect Returned Values, Beyond a Summary

The result stores coefficients, fitted excess returns, residuals, R^2, adjusted R^2, observation count, condition number, and HAC lag count. `estimate` gives monthly intercept or dimensionless loading, `standard_error_hac` the corresponding standard error, and `ci_low_hac/ci_high_hac` approximate 95% bounds. Intercept bounds are decimal monthly returns.

The condition number helps detect near-collinearity, but also depends on scaling when constants accompany decimal-return columns. No arbitrary threshold validates every model. Inspect loading intervals, correlations, window stability, and economic meaning together.

Historical regressions use frozen snapshots for 2010-01 through 2019-12. This retrospective descriptive experiment has no signal timestamp, trade timestamp, or orders. A `return interval` labels each published monthly portfolio-return interval. Trading research would separately require training cutoff, data availability, signal, execution, and subsequent evaluation interval.

10. Visualization

These three charts were generated from the same frozen historical entry and share Section 11's values. Read them in this order:

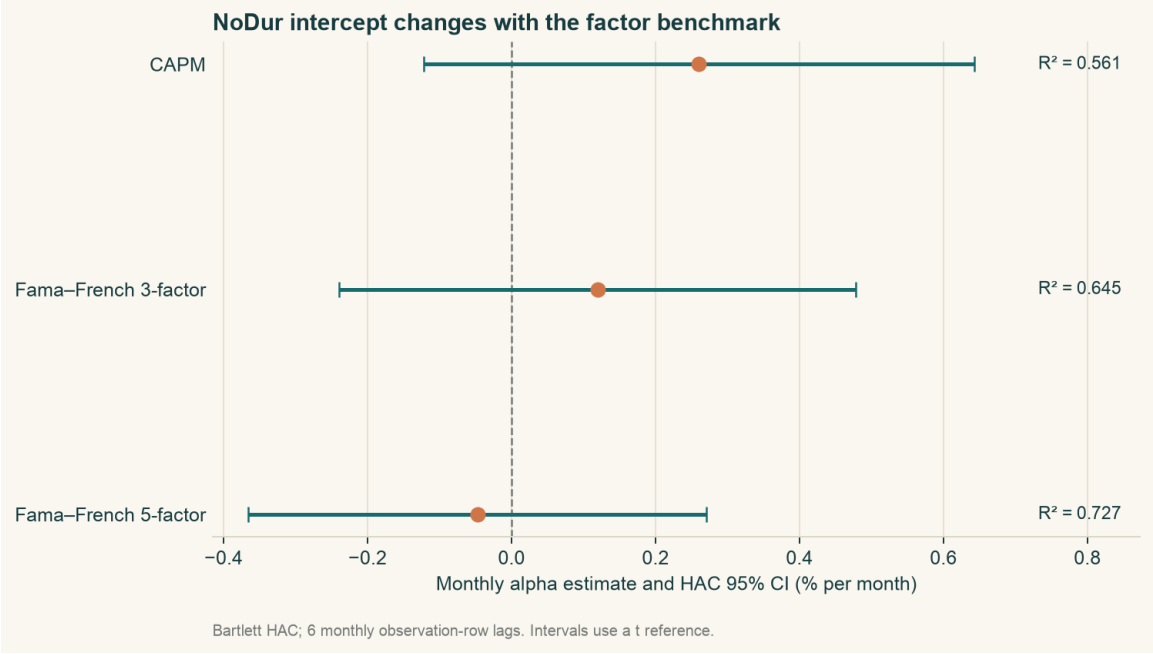


Figure 1: Historical monthly alpha and HAC intervals

1. Model intercepts and 95% HAC intervals: monthly alpha and interval horizontally (percentage points/month), CAPM/FF3/FF5 vertically. Examine benchmark changes and whether intervals cover zero, rather than only point-estimate signs.
2. FF5 loadings and intervals: dimensionless loading and HAC interval horizontally, factor names vertically. Examine market, size, value, profitability, and investment directions and uncertainty. Loadings are not holdings proportions.

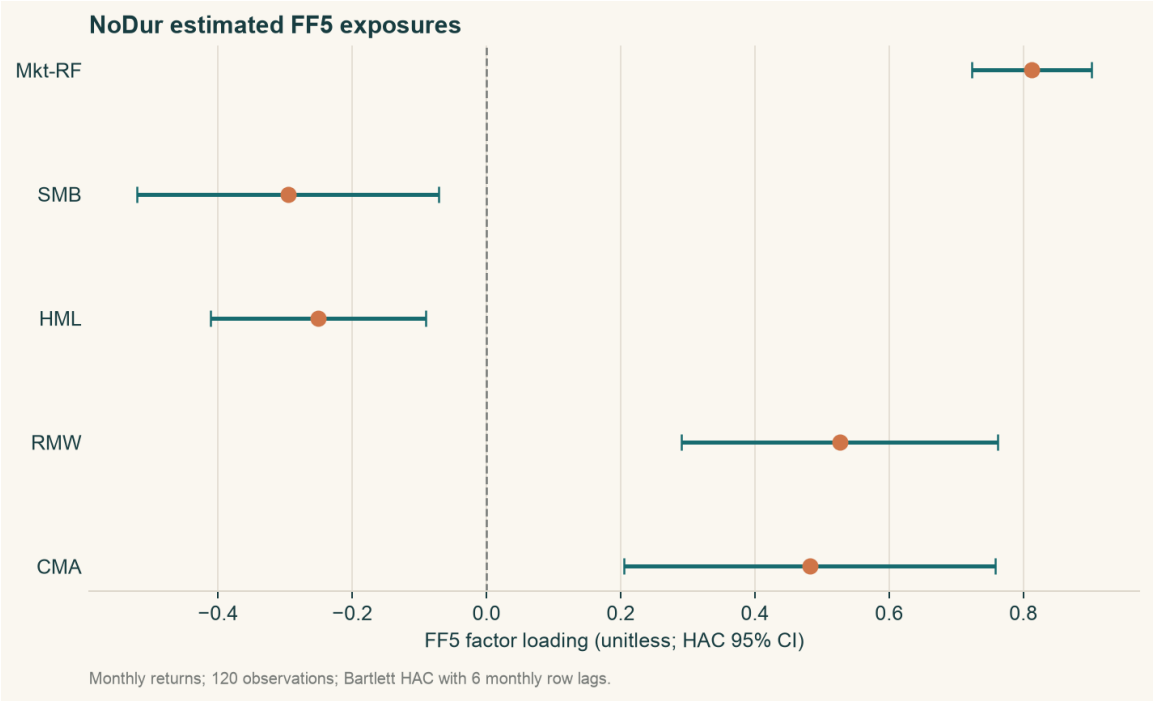


Figure 2: Historical FF5 loadings and intervals

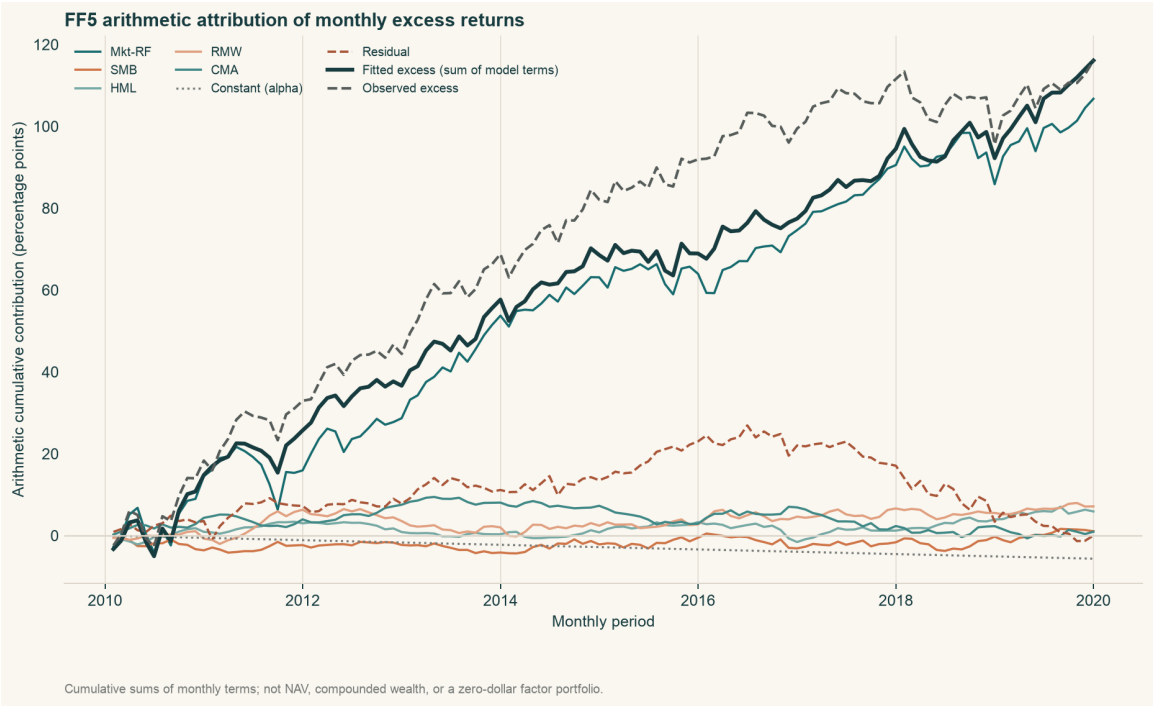


Figure 3: Historical FF5 arithmetic cumulative attribution

3. Arithmetic cumulative attribution: sample month horizontally, cumulative monthly `beta_k*f_k` contributions in percentage points vertically, alongside cumulative alpha and residuals. This is a linear attribution ledger, rather than investable NAV or compounded wealth. Monthly components sum to excess return, and arithmetic summation preserves that identity across months; it cannot replace Chapter 11's wealth product.

Do not treat a zero-net-investment long–short spread as a fund return with initial wealth 1. A factor-strategy NAV requires long/short scale, collateral, financing, rebalancing, and costs. Those ledgers are absent here; the chart supports attribution in the stated input units.

11. Real Market Example

11.1 Sample, Sources, and Evidence Scope

Inputs are published Kenneth French Nondurables (NoDur) monthly industry total returns and US FF3/FF5 factors. Snapshots were downloaded 2026-10-05; originals declare CRSP202608. The fixed 2010-01–2019-12 window contains 120 months with USD return conventions. The industry portfolio is not a directly purchasable fund; this chapter does not reconstruct constituents, prices, or historical universes.

SHA-256 hashes of processed industry, FF3, and FF5 files respectively are:

- `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`
- `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`
- `833bd925b49651428781c789c13a6ac7454d4676449e6d119782050046a57eea`

The industry manifest and factor manifest identify sources, raw and processed files, and hashes. Hashes fix current inputs without proving availability in the corresponding 2010 month. Current snapshots are not historical point-in-time (PIT) data. The official RF source changes from 2024-06; this window ends in 2019 and avoids that switch, but longer experiments must record the boundary.

11.2 Interpreting Historical Estimates

Chapter 14's same-sample CAPM uses five HAC monthly lags. This chapter uses six for all models and separately reports 3/6/12 sensitivity. Point estimates agree across chapters; intervals and p-values require their settings to accompany comparisons. These are explicit teaching choices, without a claim of formal preregistration before observing data.

The following estimates use actual frozen data, six-row Bartlett HAC, finite-sample correction, and a t reference. They are not constructed teaching values.

Model	Monthly alpha (%)	Alpha 95% interval (%)	Alpha p-value	R^2	Adjusted R^2
CAPM	0.26034	[−0.12227, 0.64295]	0.1804	0.56080	0.55708
FF3	0.11947	[−0.23925, 0.47818]	0.5108	0.64496	0.63578
FF5	−0.04716	[−0.36528, 0.27095]	0.7695	0.72651	0.71451

All three intervals contain zero, so these approximate tests do not reject zero intercepts. This does not prove the industry has no alpha. Lower FF5 sample residual variance means greater explanation in this window, without establishing better future prediction.

FF5 market, SMB, HML, RMW, and CMA loadings are about 0.81212, −0.29609, −0.25075, 0.52627, 0.48148. Controlling the other factors, NoDur has positive historical profitability and conservative-investment exposure, and negative size/value spread exposure. This neither classifies every industry company into those groups nor proves the factors causally drove returns.

11.3 An Important Observed Data Difference

Over the same 120 months, the maximum absolute difference in FF3/FF5 `Mkt-RF` is 0.0003, or 0.03 percentage points; SMB differs by up to 0.0137, or 1.37 percentage points. HML and RF agree. In 2018-05 the two raw market values are 2.66% and 2.63%, despite both headers declaring CRSP202608. The historical work checked raw values without resolving their upstream cause. Do not assume rounding, revision, or publication error.

FF5 already uses a different SMB construction, and the market columns also differ. These fits are therefore not strictly nested regressions retaining every old column and adding two. A nested F-test of only RMW/CMA is inappropriate, and the entire R^2 increase cannot be assigned to them. Isolating added columns requires fixed common market/SMB/HML series before adding RMW/CMA: an explicitly labeled custom hybrid, rather than official FF5.

11.4 Risk Decomposition and Uncertainty

All models have target sample variance 0.0010394864 in squared monthly-return units. FF5 fitted variance 0.0007551957 plus residual variance 0.0002842907 matches it. This checks the same-denominator OLS sample identity, without proving future residual independence.

At HAC lags 3,6,12, FF5 intercept remains−0.0004716374 decimal/month; its standard errors are approximately 0.0015052,0.0016058,0.0018439. Changing covariance estimation leaves OLS point estimates unchanged but alters intervals and p-values. Stable estimates across covariance choices do not establish stable future returns.

12. Common Mistakes

12.1 Subtracting RF Twice Changes the Definition

The error treats `Mkt-RF` as total market return and subtracts RF again. Test a one-month constructed case:

```
import numpy as np
rf, asset, market_excess, beta = 0.002, 0.026, 0.020, 0.8
correct_left = asset-rf
wrong_right = beta*(market_excess-rf)
correct_right = beta*market_excess
assert np.isclose(correct_left, 0.024)
assert np.isclose(correct_right-wrong_right, 0.0016)
print(f"Incorrect market contribution={wrong_right:.2%}; correct={correct_right:.2%}"
    ↪ )
```

Incorrect contribution is 1.44%, versus 1.60% correctly, a 0.16-percentage-point gap. Do not subtract RF from SMB/HML spreads either. Time-varying cash changes the entire regression relation, rather than necessarily only its intercept.

12.2 Loadings Are Not Weights; Correlations Matter

Normalizing 0.8,0.3,0.5 to sum to 1 changes fitted response coefficients. Loadings come from comovement rather than a budget. Ignoring correlations also miscalculates risk; reproduce Section 8's comparison:

```
import numpy as np
beta = np.array([0.8, 0.5])
vol = np.array([0.04, 0.02])
independent = np.diag(vol**2)
correlated = independent.copy()
correlated[0, 1] = correlated[1, 0] = 0.5*vol[0]*vol[1]
wrong_risk = np.sqrt(beta @ independent @ beta + 0.01**2)
correct_risk = np.sqrt(beta @ correlated @ beta + 0.01**2)
assert np.isclose(wrong_risk, np.sqrt(0.001224))
assert np.isclose(correct_risk, np.sqrt(0.001544))
print(f"Ignoring correlation={wrong_risk:.4%}; including correlation={correct_risk
    ↪ :.4%}")
```

Here ignoring correlation understates monthly volatility by about 0.43 percentage points. Other loading signs and negative correlations can reverse the effect. Apply the formula rather than assume correlation always increases risk.

12.3 A Current Snapshot Is Not a Forecast; Future Data Cannot Extend Training

Full-window regression supports retrospective attribution. A 2014-end estimate used as a 2015-start signal may use only training data readable at that time. Incorrect `train = sample.loc[:"2019-12"]` includes the future; cutoff `sample.loc[:"2014-12"]` fixes the time span, while publication and ingestion clocks still require checking. Slicing dates alone does not turn revised current data into PIT inputs.

These slices are executable, but no full strategy was constructed, so no Sharpe gap between incorrect and correct versions is claimed. The established difference is 120 versus 60 training months. Coefficient changes require actual fitting before reporting. Data availability, estimation, and tradable backtesting have separate evidence requirements.

12.4 Monthly Intercepts Are Not Realizable Annual Returns

Multiplying monthly alpha by 12 gives a linear annual scale. $(1+\alpha)^{12}-1$ instead assumes an account repeatedly earning the same monthly return, rather than a general intercept annualization. Historical intercepts omit trading, financing, and stock-borrow costs. Treating annualized alpha as fund earnings crosses both inference and execution boundaries.

13. Quant Research Application

13.1 Completing a Risk-Explanation Experiment

State an economic hypothesis: historical excess returns may reflect size, value, profitability, or investment exposures. Fix asset definition, window, factor vintage, currency, monthly intervals, and missing-data rules, then validate inputs and compare models. Report coefficients, intervals, errors, correlations, and method differences rather than selecting the best row.

Instead of mechanically picking the highest R^2, ask whether interpretation is plausible, loadings stable across windows, residuals still share structure, and new factors reduce out-of-sample errors or improve risk estimates. If the design only supports retrospective attribution, keep that conclusion.

13.2 Requirements for Trading Research

Residual means predicting future returns is a different hypothesis from linear explanation of realized returns. Define loadings estimated solely in training, the subsequent target and holding interval, benchmarks, turnover, costs, short feasibility, and industry exposure. Falsification must allow disappearing residuals, unstable exposures, or gross returns below costs to reject the strategy.

Training ends at t; signal s follows availability of every input; execution e follows s; evaluated returns start after e. Explaining t+1 returns using already realized t+1 factors is not an ex-ante forecast. Rolling or expanding windows only restrict fitting spans; full PIT and leakage controls remain necessary. Chapters 29–31 implement these contracts.

13.3 From Statements to Factor Exposures

Chapters 12–13 clean statements and align characteristics; Chapter 15 identifies common-return exposure; Chapters 23–28 test future-return relations; Chapters 20–21 turn expected returns and risk into weights; Chapter 35 deducts costs. A more complex factor model cannot repair any failed step. Complexity should follow the research question and bounded evidence.

14. Chapter Summary

Factor models specify comparison coordinates, estimate exposures from historical returns, then examine unexplained returns and risk. Characteristics describe companies, factor returns describe sorted portfolios, and beta describes conditional responses. State alpha together with model, sample, and cash proxy.

Same-sample OLS orthogonality supports algebraic risk decomposition; future decomposition needs additional assumptions. HAC improves inference under specified conditions, while costs and timing determine realizability. Deliverable conclusions state what was explained, what remains unproved, and how a next experiment could falsify the hypothesis.

15. Exercises

15-E01 Why are book-to-market, HML return, and a stock's HML loading different quantities? Give each unit and calculation clock.

15-E02 Keep Section 8.1's six constructed portfolios, changing only SG from 2% to−1%. Recalculate SMB_BM and HML. Why do both change?

15-E03 Using Section 8.2 with RF 0.2% and total asset return 2.6%, calculate excess return, factor contribution, fitted return including intercept, and residual. Explain why one monthly residual is not alpha.

15-E04 In Section 6.1 take alpha=0, beta=0.8, gamma=0.5, E[x]=0.5%, E[z]=0.3%, and Cov(x,z)/Var(x)=0.4. What slope and intercept result when regressing only on x?

15-E05 With uncorrelated factors, beta=(0.8,0.5), volatilities=(4%,2%), and residual volatility 1%, recalculate total variance. What changes at correlation−0.5?

15-E06 Why must residual variance using T−K−1 be adjusted before entering a sample risk identity using T−1? Does that identity establish residual independence?

15-E07 Historical NoDur FF5 monthly alpha is about−0.04716%, with 95% interval[−0.36528%,0.27095%]. Does this prove zero alpha or future underperformance?

15-E08 Why is a nested F-test adding only two columns inappropriate for the official FF3/FF5 comparison here? Design and correctly name an experiment isolating the added factors.

15-E09 | Cross-covariance. An externally fixed model has $y = f + \varepsilon$, $\varepsilon = 0.5f + u$, zero-mean uncorrelated f, u, $\mathrm{Var}(f) = 0.0004$, and $\mathrm{Var}(u) = 0.0001$. Calculate residual variance, cross-covariance, and total variance. How much does omitting the cross term change it? These are not fitted OLS residuals; explain why that matters.

15-E10 | Varying loadings. Recalculate Section 6.5's two-state mean factor contribution and $\mathrm{Cov}(\beta, f)$. Does this example alone establish an executable strategy of raising exposure in positive-factor months?

16. Programming Exercises

15-P01 Fit FF3 on Section 9.1's constructed inputs. Shift all factor months forward by one and assert rejection rather than silent inner-join fitting. Then make one factor twice another and assert rank-deficiency rejection.

15-P02 On the same constructed sample with random residuals, use HAC lags 3,6,12 and assert unchanged OLS coefficients. Report intercept standard errors and check that ddof=1 fitted plus residual sample variance equals excess-return variance.

15-P03 Recalculate CAPM, FF3, FF5 on frozen historical inputs. Report monthly alpha and intervals, SMB loadings, and R^2; check shared-name FF3/FF5 column differences. Do not claim unexecuted net alpha or stock-level replication.

17. Challenge Problems

15-C01 | Models and economic meaning. A researcher searches 100 asset windows, selects an FF5 intercept with $p<0.05$, and claims tradable net alpha. Give statistical, data, economic, and execution falsifications. Design follow-up research without substituting more same-sample searches for out-of-sample testing.

15-C02 | Portfolio risk; prerequisites Chapters 5,9. Two assets have one-factor loadings 1 and-0.5 and weights 0.5 each. Factor volatility is 4% per month; each residual volatility is 2%. Calculate total portfolio volatility first with uncorrelated residuals, then residual correlation 0.8. Why can low market exposure coexist with concentration risk?

18. Solutions / Solution Hints

18.1 Conceptual and Hand-Calculation Answers

15-E01 Book-to-market divides common-equity book value from a known statement version by consistently defined market value. It is dimensionless, with publication/ingestion and market clocks recorded separately. HML is the monthly return difference of specified high/low book-to-market portfolios, decimal/month, known after realization and publication. Loading h is a dimensionless conditional response controlling other factors in a specified training sample, using only data readable by cutoff. These are a company attribute, a realized portfolio return, and a comovement parameter.

15-E02 Small now averages $(4\%+3\%-1\%)/3 = 2\%$, Big remains 1%, giving SMB_BM=1%. Value averages 3%; Growth averages $(-1\% + 0\%)/2 = -0.5\%$, giving HML=3.5%. SG belongs to both Small and low book-to-market groups, so both formulas include it. Sorting does not automatically make spreads orthogonal.

15-E03 Step by step:

$$\begin{aligned} y &= 2.6\% - 0.2\% = 2.4\%, \\ c &= 1.6\% + 0.3\% - 0.5\% = 1.4\%, \\ \widehat{y} &= c + 0.2\% = 1.6\%, \\ \varepsilon &= y - \widehat{y} = 0.8\%. \end{aligned}$$

Here c is factor contribution; all terms are this month's return percentages. Alpha is a conditional mean shift estimated across the sample; a residual is one month's deviation from its fit. They are not interchangeable.

15-E04 The omitted-factor slope is $0.8+0.5\times0.4=1.0$; intercept $0+0.5\times(0.003-0.4\times0.005)=0.0005$, or 0.05% per month. True constructed alpha is zero. The single-factor positive intercept reflects omitted z's mean component, without newly created trading skill.

15-E05 With zero correlation, common variance 0.001124 plus residual variance 0.0001 gives 0.001224 and volatility 3.4986%. Correlation-0.5 adds cross term-0.00032, giving 0.000904 and about 3.0067%. Risk effects depend on the product of correlation and loading signs. Covariance must remain valid; impossible correlation combinations cannot be assigned arbitrarily.

15-E06 With RSS the residual sum of squares, an intercept makes residual mean zero, so sample residual variance is RSS/(T$-$1). Noise variance estimate RSS/(T$-$K$-$1) is usually larger. Different targets and denominators cannot be mixed in the same-sample identity. Zero in-sample inner products do not establish variable independence, cross-asset residual independence, or future orthogonality.

15-E07 The approximate 95% interval permits both negative and positive values and does not reject zero under current assumptions. It does not prove alpha exactly zero. Future performance also involves new data, states, costs, and realizable portfolios. Historical industry attribution cannot imply certain future underperformance. Insignificance is not proof of no effect.

15-E08 Official FF3/FF5 SMB definitions differ, and their market columns also differ numerically here. Changed old columns prevent nesting by only two additions. Fix identical MKT/SMB/HML, RF, asset, and months, then add same-vintage RMW/CMA; call it a custom extension with a fixed FF3 benchmark. Testing still requires appropriate covariance/error assumptions; an ordinary homoskedastic F-test cannot automatically replace robust testing. This also does not establish better forecasts.

15-E09 $\mathrm{Var}(\varepsilon) = 0.5^2 \times 0.0004 + 0.0001 = 0.0002$ and $\mathrm{Cov}(f, \varepsilon) = 0.5 \times 0.0004 = 0.0002$. Total variance is $0.0004 + 0.0002 + 2 \times 0.0002 = 0.0010$, also obtained directly by expanding $y = 1.5f + u$ to give $1.5^2 \times 0.0004 + 0.0001 = 0.0010$. Dropping the cross term gives only 0.0006, understating by 0.0004; volatilities are 3.1623% and 2.4495%. Loading 1 is externally assigned, leaving $0.5f$ in the remainder. A best linear projection with an intercept on this population would instead give loading 1.5 and residual u, enabling orthogonal decomposition. Arbitrary model remainders do not inherit OLS orthogonality.

15-E10 $\mathbb{E}[\beta] = 1$, $\mathbb{E}[f] = 0.01$, $\mathbb{E}[\beta f] = (-0.01 + 0.06)/2 = 0.025$, and $\mathrm{Cov}(\beta, f) = 0.025 - 1 \times 0.01 = 0.015$. Centered products $[(-0.5)(-0.03) + (0.5)(0.03)]/2$ also give 0.015. Mean contribution 2.5% contains a 1.5-percentage-point covariance term. Contemporaneous loadings and realized factors supply no pre-period predictive ability, execution, or costs. Choosing the month's loading after observing its factor realization introduces future information, without establishing strategy validity.

18.2 Complete Programming References

15-P01 This standalone example uses constructed seed 1501 and requires rejection of shifted months and exact collinearity.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
import numpy as np
import pandas as pd
from quant_course.factor_models import fit_monthly_factor_model

rng = np.random.default_rng(1501)
index = pd.period_range("2000-01", periods=120, freq="M")
f = pd.DataFrame({"Mkt-RF": rng.normal(0.005, 0.04, 120),
                  "SMB": rng.normal(0.002, 0.02, 120),
                  "HML": rng.normal(0.003, 0.02, 120),
                  "RF": np.full(120, 0.001)}, index=index)
y = f["RF"] + 0.002 + 0.8*f["Mkt-RF"] + 0.3*f["SMB"] + 0.5*f["HML"]
fit = fit_monthly_factor_model(y, f, ("Mkt-RF", "SMB", "HML"))
assert np.allclose(fit.coefficient_table["estimate"], [0.002, 0.8, 0.3, 0.5])
shifted = f.copy()
shifted.index = shifted.index + 1
rank_deficient = f.copy()
rank_deficient["SMB"] = 2*rank_deficient["Mkt-RF"]
for invalid in (shifted, rank_deficient):
    try:
        fit_monthly_factor_model(y, invalid, ("Mkt-RF", "SMB", "HML"))
    except ValueError:
```

```
        pass
    else:
        raise AssertionError("wrong months or rank deficiency must be rejected")
print("both invalid experiments rejected")
```

15-P02 Nonzero random error makes interval changes under covariance choices meaningful. Noise demonstrates the interface without claiming realistic financial states.

```
from pathlib import Path
import sys
sys.path.insert(0, str(Path.cwd() / "src"))
import numpy as np
import pandas as pd
from quant_course.factor_models import fit_monthly_factor_model

rng = np.random.default_rng(1502)
index = pd.period_range("2000-01", periods=120, freq="M")
f = pd.DataFrame({"Mkt-RF": rng.normal(0.005, 0.04, 120),
                  "SMB": rng.normal(0.002, 0.02, 120),
                  "RF": np.full(120, 0.001)}, index=index)
asset = f["RF"] + 0.002 + 0.8*f["Mkt-RF"] + 0.3*f["SMB"] + rng.normal(0, 0.01, 120)
fits = [fit_monthly_factor_model(asset, f, ("Mkt-RF", "SMB"), hac_maxlags=lag)
        for lag in (3, 6, 12)]
for model in fits:
    assert np.array_equal(model.coefficient_table["estimate"], fits[0].
    ↪ coefficient_table["estimate"])
    observed = asset-f["RF"]
    assert np.isclose(observed.var(ddof=1),
                      model.fitted_excess.var(ddof=1)+model.residuals.var(ddof=1))
    print(model.hac_maxlags, model.coefficient_table.loc["const", "standard_error_hac
    ↪ "])
```

15-P03 Load frozen inputs from manifests rather than treating existing result JSON as the calculation source. Recalculating on a current snapshot does not give historical PIT eligibility.

```
from pathlib import Path
import sys
import json
sys.path.insert(0, str(Path.cwd() / "src"))
import numpy as np
from quant_course.market_data import load_monthly_snapshot
from quant_course.factor_models import fit_monthly_factor_model

root = Path.cwd()
a = json.loads((root / "data/manifests/french_industries_2026-10-05.json").read_text
    ↪ ())
m = json.loads((root / "data/manifests/french_factors_2026-10-05.json").read_text())[
    ↪ "datasets"]
asset = load_monthly_snapshot(root/a["processed_path"], a["processed_sha256"], ["
    ↪ NoDur"])
asset = asset.loc["2010-01":"2019-12", "NoDur"]
columns = {"ff3": ("Mkt-RF", "SMB", "HML"),
           "ff5": ("Mkt-RF", "SMB", "HML", "RMW", "CMA")}
frames = {name: load_monthly_snapshot(root/m[name]["processed_path"],
                                      m[name]["processed_sha256"], [*cols, "RF"])
```

```
                    .loc["2010-01":"2019-12"]
          for name, cols in columns.items()}
expected = {"capm": (0.002603392167217396, 0.5607993620708411),
            "ff3": (0.001194674972166453, 0.6449619292529103),
            "ff5": (-0.0004716374176230468, 0.7265084935780675)}
for name in expected:
    frame = frames["ff5" if name == "ff5" else "ff3"]
    cols = ("Mkt-RF",) if name == "capm" else columns[name]
    fit = fit_monthly_factor_model(asset, frame, cols, hac_maxlags=6)
    assert fit.nobs == 120
    assert np.isclose(fit.coefficient_table.loc["const", "estimate"], expected[name
    ↪ ][0])
    assert np.isclose(fit.r_squared, expected[name][1])
    print(name, fit.coefficient_table, "R2=", fit.r_squared)
for col, difference in {"Mkt-RF": 0.0003, "SMB": 0.0137, "HML": 0.0, "RF": 0.0}.items
    ↪ ():
    measured = (frames["ff3"][col]-frames["ff5"][col]).abs().max()
    assert np.isclose(measured, difference)
    print(col, measured)
```

Monthly alpha, intervals, and R^2 appear in Section 11. SMB loadings are about-0.42424 for FF3 and-0.29609 for FF5. Different SMB columns and conditioning factors prevent attributing economic mechanisms to the loading gap alone. After-cost performance is outside these outputs.

18.3 Challenge Arguments

15-C01 Statistical falsifications include selection across 100 attempts and sensitivity to dependence, HAC bandwidth, and misspecification. Data concerns include revised statements, surviving-stock universes, month mismatch, and unavailable inputs. Economic alternatives include omitted risk, industry concentration, and liquidity exposure difficult to bear in adverse states. Execution concerns include gross returns below costs, unstable stock borrowing, and capacity. Freeze hypotheses, definitions, and training/validation windows; preserve untouched future evaluation periods; compare simple benchmarks under fixed robustness rules; retain failures. Reselecting favorable subdivisions of the searched sample is not independent out-of-sample evidence.

Score 10:1 per falsification category;2 for validation windows and clocks;2 for multiple search and test assumptions;2 for costs/capacity and negative-result rules. Alternative mechanisms are acceptable if falsifiable.

15-C02 Portfolio loading is $0.5\times1+0.5\times(-0.5)=0.25$, common variance $0.25^2\times0.04^2=0.0001$. Independent residual portfolio variance is $0.5^2\times0.02^2+0.5^2\times0.02^2=0.0002$, giving total 0.0003 and volatility 1.7321%. With correlation 0.8, residual covariance is $0.8\times0.02^2=0.00032$; added portfolio cross term $2\times0.5\times0.5\times0.00032=0.00016$ gives total 0.00046 and volatility 2.1448%. Shared residual shocks raise risk despite low market loading.

Score 10:2 for portfolio loading;2 each for both residual-variance calculations;2 for total variance/volatility;2 for explaining why low beta need not mean low concentration.

19. Further Reading

First read French's official construction pages and check columns and monthly intervals by hand; then read the verified introductions and method passages of the original papers. Separate factor construction, tests of return relations, and claims about an asset-pricing mechanism. Success at one level does not establish the others.

The 1993 paper studies common stock and bond risks; its five risk factors are not the 2015 equity FF5. The 2015 HML redundancy discussion is conditional on its US 1963–2013 sample and model, rather than a claim that value is permanently useless. Original-paper historical table values are not presented as this chapter's current-window results. 1993 journal-PDF teaching mirror, 2015 journal-PDF teaching mirror

For deeper risk models, consult classic portfolio-management texts on factor and residual covariance estimation. They are course-wide references; inaccessible chapters must not be cited as if read page by page. Chapters 18–21 develop volatility, estimation uncertainty, and portfolio risk; Chapters 23–28 turn to characteristics and prediction.

20. References

- Fama, E. F., & French, K. R. (1993). *Common risk factors in the returns on stocks and bonds*. Journal of Financial Economics,33,3–56. Uses the read introduction and selected methods, without reproducing original charts.
- Fama, E. F., & French, K. R. (2015). *A five-factor asset pricing model*. Journal of Financial Economics,116,1–22. Uses the read introduction and selected redundancy discussion, without claiming equation-by-equation verification of the entire paper.
- Kenneth R. French Data Library: three factors, five factors. Method pages and snapshot vintages are checked separately.
- statsmodels: robust covariance API. Parameters support reproducibility without replacing statistical assumptions.

Full source-reading limits appear in `source_notes.md`. The historical root acceptance identifies the prior revision's body, code, Notebook, figures, and same-chapter answers. C7 teaching additions remain subject to independent review by the original lead.

Project 2 —Factor Regressions and Evidentiary Limits

Project 2: CAPM and Fama–French Factor Regression

What this project contains

This project has two reading tracks that must remain separate:

1. The **default offline notebook** is a constructed-data method demonstration. It does not read French market data or reproduce the historical study.
2. The **research report, exercises, and solutions** preserve the original historical analysis of French industry portfolios. They describe a historical release snapshot for 2010–2019, not new output from the default notebook.

Start with this page and the Reader Guide, then run the notebook. The default entry point is the Project 2 factor-regression notebook. The research report contains the historical results, limitations, and descriptions of all five historical figures. Attempt the exercises before reading the solutions. See the data dictionary for fields, units, sources, and authorization boundaries, and the data manifest for the identities of the historical inputs.

Default notebook data and models

The notebook uses random seed 102 to construct a 120-month monthly series starting in 2010-01. It generates five factor columns from normal pseudo-random draws with mean 0 and standard deviation 0.03; RF is fixed at 0.001 per month. The constructed asset's total return is

$$R_t = RF_t + 0.002 + 1.1\, f_{\mathrm{MKT},t} + 0.4\, f_{\mathrm{SMB},t} + \epsilon_t, \qquad \epsilon_t \sim N(0, 0.015^2).$$

The two factor symbols denote constructed columns `Mkt-RF` and `SMB`.

The notebook fits CAPM (Mkt-RF), FF3 (Mkt-RF, SMB, HML), and FF5 (Mkt-RF, SMB, HML, RMW, CMA). All three specifications use the same constructed factor columns, so these constructed models are strictly nested. Every model uses HAC 6; only FF3 is also calculated with HAC 3, 6, and 12. The constructed sample is not industry data, and its results must not be confused with the nine historical regressions in the report.

The shared fitting function subtracts RF once from the constructed asset's total return. The code checks that each fit has 120 observations and asserts that excess return equals fitted value plus residual; the notebook does not calculate or assert a sample-variance decomposition. It exports three coefficient-table CSVs and a plot of FF3 residuals. Outputs go to the package-root directory `runs/project02_factor_regression_<uuid>/`. When launched through the package runner, the executed notebook and execution status are saved separately under `runs/execution_project02_factor_regression_<uuid>/`; these UUID directories serve different purposes. See the data dictionary for output columns and row order.

Run the notebook from the reader package root:

```
.venv/bin/python run_notebook.py projects/project02/notebooks/
    ↪ project02_factor_regression.ipynb
```

The package-root README has the full instructions for setting up the reader environment. The runner uses a fresh kernel and stores the executed copy and execution.json in its own run directory. This workflow runs only the default constructed-data branch; market data are not required.

Historical study and interpretation boundaries

The historical report uses the Kenneth R. French 10 industry portfolios and official `FF3/FF5` factor snapshots frozen on 2026-10-05. It estimates CAPM, FF3, and FF5 for NoDur, HiTec, and Enrgy: nine historical regressions in total. The report retains every table, negative and near-zero result, HAC 3/6/12 sensitivity, multiple-testing result, and chronological attribution error.

The common Mkt-RF and SMB series in the historical official FF3 and FF5 files are not identical, so the two historical models are not strictly nested; the R^2 increase cannot be attributed entirely to the added `RMW/CMA` factors. The default notebook compares nested specifications using the same constructed factor columns. The two sets of material use different samples, sources, and questions.

The historical report is not a causal study, real-time PIT test, ex ante forecast, trading-advantage test, or evaluation of an investable product. The reader package does not include the raw, extracted, or processed French data. The report's download date and SHA-256 identify historical inputs; they do not grant data rights. Public access does not imply permission to redistribute. G01 rights, subject, and use remain unknown; G05 does not authorize A10-B, so historical market-data execution remains blocked. Costs, financing, and borrowing are not applicable to the available report because there is no signal, position, order, fill, or transaction ledger; this does not mean costs are zero.

Historical course production entry

These commands document the original course working-root workflow; they are not requirements for the packaged default and were not rerun in this batch. The original project status is verified; this bilingual candidate is draft, pending independent acceptance. The historical CLI locates the course root from its own path, validates hashes and CRCs, and writes historical outputs and five figures. Constructed outputs must not overwrite them.

```
.venv/bin/python projects/project02/run_project02.py
.venv/bin/python -m pytest tests/test_project02_analysis.py -q
```

The old working Notebook used the `quant-course-venv` kernel to call that analysis entry. The accepted I3 default instead uses the package-root `run_notebook.py` with a fresh kernel and constructed inputs. Readers need not restore old production tests absent from the package.

Project 2 Reader Guide: From Conditional Regression to Evidence Boundaries

Learning goals

This project practices three skills: distinguishing total return from excess return; interpreting conditional Betas and model intercepts in CAPM, FF3, and FF5; and judging what fit, sensitivity, and chronological-split results can support. The default notebook teaches the method with constructed data. The historical report preserves the original French market-data results. First identify which evidence track you are reading.

Suggested sequence

1. Read the project README and keep the constructed demonstration separate from the historical report.
2. Review Chapter 8 on regression for intercepts, Betas, residuals, and confidence intervals. Read Chapter 11 on returns for total returns, monthly simple returns, and RF.
3. Read Chapter 14 on asset pricing for benchmark-model interpretation, then Chapter 15 on factor models for factor regression, OLS, and worked HAC calculations. This project applies those ideas in its equations and research report.
4. Open the default offline notebook. Read its constructed inputs and variable definitions first, then inspect the three model tables, FF3 bandwidth sensitivity, and residual plot. It does not run the historical French-data branch.
5. Attempt the eight exercises independently, then check the solutions. Finally, read the historical report, keeping method demonstration and historical evidence distinct.

Reading the default notebook

The input is a constructed sample generated with fixed seed 102: 120 consecutive months starting in 2010-01; five factor columns drawn from $N(0, 0.03^2)$; and RF fixed at 0.001. The asset's total return is constructed from RF, a 0.002 intercept, 1.1 times Mkt-RF, 0.4 times SMB, and noise with standard deviation 0.015. The shared function subtracts RF once from total return before fitting excess return.

CAPM, FF3, and FF5 use nested subsets of the same constructed factor columns, so these constructed specifications are strictly nested. All three use HAC 6. The sensitivity section refits FF3 only at HAC 3, 6, and 12. The notebook does not refit nine industry models, calculate Holm multiple-testing adjustments, or produce the five historical research figures. Any coefficient or interval belongs to the method demonstration and cannot be cited as a French-market finding.

The code asserts that each model has 120 observations and checks excess return = fitted value + residual. The notebook does not calculate or assert a variance decomposition; the variance identity in its prose is a conceptual statement. Do not describe it as a notebook check that passed.

The data dictionary lists the three CSV filenames, six columns, and parameter row order. CSV export uses index=False, so the parameter names are omitted. In order, CAPM rows are const, Mkt-RF; FF3 rows are const, Mkt-RF, SMB, HML; and FF5 rows are const, Mkt-RF,

SMB, HML, RMW, CMA. Recover the row meanings from the corresponding file and this fixed order.

The default notebook produces one plot: a line plot of FF3 residuals. Its x-axis is labeled Month, but the plotted values are the ordinals 0–119 because the residual array is plotted without date coordinates; they are not calendar dates. Its y-axis is labeled Decimal excess return, but the plotted array is the constructed FF3 excess-return residual. It therefore shows residuals in decimal units, not asset total returns or realized excess returns themselves. Notebook artifacts go to the package-root directory `runs/project02_factor_regression_<uuid>/`. The package runner separately saves the executed notebook and execution.json under `runs/execution_project02_factor_regression_<uuid>/`.

Run it through the reader package's launcher:

```
.venv/bin/python run_notebook.py projects/project02/notebooks/
    ↪ project02_factor_regression.ipynb
```

The launcher starts a fresh kernel and executes the notebook. See the reader package root README for environment setup; this project does not require downloading or supplying market data.

Reading the five historical figures

These figures belong to the historical report. They are not newly produced by the default notebook. The research report contains the figures and their full context.

Historical figure	X-axis	Y-axis and visual meaning
Alpha	Monthly intercept in percentage points per month; error bars are HAC 6 95% intervals	The nine asset-by-model estimates; the vertical line marks zero
Factor loadings	Conditional Beta and HAC 6 interval	One panel per factor; only asset-by-model combinations that include the factor are shown, and omitted terms are not plotted as zero
Variance decomposition	Fitted and residual shares of variance in the same historical sample, shown as a horizontal 100% stacked bar	The nine asset-by-model combinations; teal is fitted variance and orange is residual variance
Historical residuals	Monthly return interval, plotted at month-end	One panel per industry; realized same-month excess return less fitted value, in percentage points per month; colors distinguish CAPM, FF3, and FF5
Chronological attribution error	RMSE from applying 2010–2014 coefficients to realized same-month factors in 2015–2019, in percentage points	Horizontal bars for the nine asset-by-model combinations; this is not an ex ante forecast or strategy result

These figures summarize the original historical analysis. The common Mkt-RF and SMB series in the historical official FF3 and FF5 files differ, so those models are not strictly nested.

The current constructed notebook uses the same factor columns in all three specifications. Do not compare fit or Alpha across these evidence tracks as though they measured the same market relationship.

Reading the historical report and exercises

The historical report preserves the 120-month 2010–2019 French snapshot and all results from the nine historical regressions, including negative and near-zero estimates, HAC 3/6/12 intervals, nine-test Holm adjustment, sample variance accounting, and the contemporaneous attribution check from 2010–2014 training to 2015–2019. HiTec–FF5 attribution RMSE is 1.562 and NoDur–FF5 is 1.932 percentage points per month. These are same-month factor-attribution errors, not ex ante forecast errors.

The questions cover, in order: 1) RF and factor units; 2) conditional Beta; 3) model Alpha; 4) the sample variance identity; 5) comparability of historical FF3 and FF5; 6) HAC bandwidth; 7) negative results and multiple testing; and 8) information availability and data needed for net Alpha. The solutions retain the full reasoning and evidentiary limits.

Costs, financing, and borrowing are **not applicable** to the historical report because there is no trading signal, target position, order, execution price, or ledger; they were not set to zero. Public access to French downloads does not grant redistribution rights. The reader package contains no raw, extracted, or processed data; G01 rights, subject, and use remain unknown, and G05 does not authorize A10-B. The default constructed demonstration does not change these gates or test market results.

The retained loading-figure Chinese panel titles mean market excess return (Mkt-RF), size (SMB), value (HML), profitability (RMW), and investment (CMA), from top to bottom. The residual-figure panel titles mean nondurables (NoDur), high technology (HiTec), and energy (Energy), from top to bottom. The three model colors are teal CAPM, orange FF3, and blue FF5. English bitmap titles remain a BIL23 assembly task; these historical image bytes are retained.

Project 2 Data Dictionary and Output Notes

This dictionary distinguishes two data identities: the default notebook uses constructed data, while the historical report describes a frozen Kenneth R. French official snapshot. Do not call the current notebook's synthetic series historical observations. Paths and SHA-256 identities for the historical inputs are in the data manifest; the data files are not included in the reader package.

Default notebook: constructed inputs

Name	Meaning	Unit and generation	Time key / notes
month	Monthly index for the constructed sample	pandas monthly PeriodIndex	120 consecutive months starting in 2010-01; not historical market dates
Mkt-RF	Constructed market-factor spread	Normal pseudo-random draw with seed 102, N(0, 0.03^2), decimal per month	First column of the five-factor matrix; not French market data
SMB	Constructed size-factor spread	N(0, 0.03^2) pseudo-random draw, decimal per month	Second factor-matrix column
HML	Constructed value-factor spread	N(0, 0.03^2) pseudo-random draw, decimal per month	Third factor-matrix column
RMW	Constructed profitability-factor spread	N(0, 0.03^2) pseudo-random draw, decimal per month	Fourth factor-matrix column
CMA	Constructed investment-factor spread	N(0, 0.03^2) pseudo-random draw, decimal per month	Fifth factor-matrix column
RF	Constructed risk-free-rate term	Fixed simple return of 0.001 per month	Same in every month; used only to demonstrate the excess-return transformation
asset	One constructed asset total-return series	RF + 0.002 + 1.1 × Mkt-RF + 0.4 × SMB + ε; ε is a draw from N(0, 0.015^2); decimal per month	The regression function subtracts RF once to obtain the excess-return target y

The model-factor order in the notebook is fixed as follows. Each specification uses the same constructed factor values, so the models are strictly nested in this demonstration:

CSV file	Model row order (parameter names)	Note
`CAPM_coefficients_CONSTRUCTED.csv`	const, Mkt-RF	HAC 6
`FF3_coefficients_CONSTRUCTED.csv`	const, Mkt-RF, SMB, HML	HAC 6
`FF5_coefficients_CONSTRUCTED.csv`	const, Mkt-RF, SMB, HML, RMW, CMA	HAC 6

Each CSV is exported with index=False. Parameter names are the DataFrame index and are not saved as a column. The six actual data columns, in order, are:

Column	Meaning
estimate	OLS coefficient point estimate
`standard_error_hac`	HAC standard error
`t_hac`	HAC t-statistic
`p_hac`	HAC p-value
`ci_low_hac`	Lower endpoint of the 95% interval
`ci_high_hac`	Upper endpoint of the 95% interval

Thus, the first CSV row is const, followed by the factor rows in the order above. The filename identifies the model, but rows do not carry parameter labels. CAPM, FF3, and FF5 all use HAC 6; only FF3 is also estimated at HAC 3, 6, and 12. The code checks that every model has 120 observations and asserts y = `fitted_excess` + residual; it does not calculate or assert the variance identity.

The notebook creates one figure, `project02_residuals_CONSTRUCTED.png.` It plots FF3 `residuals.to_numpy`(), so the x-axis Month is actually the ordinal index 0–119 after the PeriodIndex dates are dropped; the y-axis label Decimal excess return denotes decimal units, but the plotted values are constructed FF3 excess-return residuals. The plot is neither total return nor the unfitted excess return.

The output directory created by the notebook's setup code cell is package-root `runs/project02_factor_regression_<uuid>/` and contains three coefficient CSVs and one PNG. When launched through `run_notebook.py`, the runner separately creates package-root `runs/execution_project02_factor_regression_<uuid>/` to store the executed notebook and execution.json. Each directory gets its own UUID and serves a different purpose; the runner's directory does not replace the notebook-artifact directory.

Historical French snapshot fields

The table below describes the source and field conventions for the historical analysis. It does not mean that these data files are included in the current reader package.

Field	Meaning	French published unit	Historical processed-file / regression unit	Time key	Historical missing-value rule

Field	Meaning	French published unit	Historical processed-file / regression unit	Time key	Historical missing-value rule
month	YYYYMM monthly return-interval label	—	pandas monthly PeriodIndex	Unique calendar month; not an announcement, signal, or execution time	Missing, duplicate, out-of-order, or missing month is not allowed
NoDur	Published dividend-inclusive value-weighted Non-Durables industry portfolio total return	Percent per month	Decimal simple return per month	month	Source sentinels were made empty during the frozen conversion; non-finite values in the window cause an error
HiTec	Published dividend-inclusive value-weighted Hi-Tech industry portfolio total return	Percent per month	Decimal simple return per month	month	Same as above
Enrgy	Published dividend-inclusive value-weighted Energy industry portfolio total return	Percent per month	Decimal simple return per month	month	Same as above
Mkt-RF	Published market-portfolio return less same-month RF spread	Percent per month	Decimal factor spread per month	month	No imputation; all values in the selected window must be finite
SMB	Published Small Minus Big spread	Percent per month	Decimal factor spread per month	month	Same as above
HML	Published High Minus Low book-to-market spread	Percent per month	Decimal factor spread per month	month	Same as above
RMW	Published Robust Minus Weak profitability spread	Percent per month	Decimal factor spread per month	month	FF5 file only; same as above
CMA	Published Conservative Minus Aggressive investment spread	Percent per month	Decimal factor spread per month	month	FF5 file only; same as above

Field	Meaning	French published unit	Historical processed-file / regression unit	Time key	Historical missing-value rule
RF	One-month U.S. Treasury-bill return proxy; not guaranteed to equal an investable cash rate	Percent per month	Decimal simple return per month	month; matched one-to-one to the asset in the same month	No forward fill; missing or infinite value causes an error

Historical French files are published in percent units and were divided by 100 once during processing; 0.01 in a processed file means a 1% monthly return or factor spread. Source sentinels −99.99 and −999 were converted to empty fields. The historical pipeline checked SHA-256 for the ZIP, archived member CSV, and processed CSV; checked ZIP CRC and member-byte identity; and used the shared monthly reader to validate hashes, columns, numeric types, finite values, date order, uniqueness, and continuity. These are records of the historical source workflow, not checks that the current constructed notebook performs on French inputs.

Models, timing, and source boundaries

In the historical regressions, CAPM takes Mkt-RF and RF from the FF3 file; historical FF3 and FF5 each use the full factor series and RF from the corresponding official file. Each historical target is industry total return less same-file, same-month RF once; published spreads such as Mkt-RF enter unchanged. The historical window is fixed at 2010-01 through 2019-12, 120 months. An additional, non-preregistered attribution check estimates on 2010–2014 and applies coefficients to contemporaneous factors in 2015–2019; it is neither PIT evidence nor an ex ante forecast.

The historical file headers identify the 202608 CRSP version. Source documentation records the January 2025 transition from CRSP FIZ to CIZ Flat File Format 2.0, which changed monthly dividend-reinvestment timing, and the June 2024 change in the RF proxy from Ibbotson Associates to the ICE BofA US 1-Month Treasury Bill Index. Historical month labels do not include `event_time`, `available_at`, `signal_time`, or `trade_time`; a single current snapshot cannot reconstruct what was knowable at each historical date.

Mkt-RF and SMB differ between the selected historical FF3 and FF5 files, so the official models are not strictly nested. The SMB construction difference is documented by the source, but the month-by-month differences have not been reproduced from underlying portfolios, and the cause of the Mkt-RF file difference is unknown. The current constructed notebook, by contrast, uses the same generated factor columns in nested models. These are different evidence identities.

Raw, extracted, and processed French files are not included in the reader package. Public access does not grant general redistribution rights; retain Kenneth R. French attribution and check current terms before external use. A10-A was accepted only within a limited scope; the A10-A.1 execution protocol is technically accepted but still awaits permissions and execution authorization; G01 rights, subject, and use remain unknown, and G05 does not authorize A10-B.

With no signal, positions, orders, fills, or transaction ledger, costs, financing, and borrowing are not applicable to the historical study; do not describe them as zero or report a net strategy return.

Project 2 Exercises

> Reader note: The data version, values, negative results, and null results below are retained from the historical study and exercises. They are historical teaching material, not new market results from the current constructed-data run of the I3 reader notebook. The data-authorization branch is skipped by default.

Answer the questions before consulting the solutions. Unless a question says otherwise, returns are expressed as decimals, and the model results come from the fixed 2010–2019 sample.

1. **Same-month RF and factor units.** In a given month, HiTec's total return is 2.0%, the same month's RF is 0.3%, and the published Mkt-RF value is 1.7%. Write down the dependent variable for that month. What would be wrong with subtracting RF from Mkt-RF again? If only the asset's total return is available and RF is not, can this be called a CAPM excess-return regression?
2. **Interpret a conditional Beta.** HiTec's FF3 market Beta is 1.158, with a HAC 95% interval of [1.075, 1.242]. Interpret the linear conditional relationship, in percentage points, when Mkt-RF rises by 1 percentage point in the same month. Does this number prove that a market move causes HiTec's return to change?
3. **Interpret model Alpha.** NoDur-CAPM Alpha is 0.2603% per month, with a HAC 95% interval of [−0.1223%, 0.6429%], nominal p = 0.1804, and nine-test Holm p = 1.0000. Explain its unit, interval, and evidentiary limits.
4. **Check the sample variance.** The total sample variance, fitted variance, and residual variance of NoDur-CAPM excess returns are 0.001039486, 0.000582943, and 0.000456543, respectively (squared decimal returns). Check the numerical identity and convert the fitted and residual shares to percentages. Can this be called a causal decomposition of future risk?
5. **Compare FF3 and FF5.** HiTec's R^2 rises from 0.864 under FF3 to 0.893 under FF5. Give at least two reasons why this difference should not be interpreted directly as the "pure incremental contribution" of the newly added `RMW/CMA` factors.
6. **HAC 3/6/12 sensitivity.** Enrgy-FF3 Alpha has the same point estimate, −0.6629% per month, under HAC 3/6/12. The 95% intervals are [−1.2950%, −0.0308%], [−1.3091%, −0.0167%], and [−1.3154%, −0.0104%], respectively. Which quantities change and which stay the same? Is there a reason to treat the three bandwidths as three independent validations?
7. **Multiple testing and negative results.** Under HAC 6, the Alpha estimates for all three energy models are negative, with single-test p-values of 0.0061, 0.0444, and 0.0249. Their Holm-adjusted values are 0.0549, 0.3111, and 0.1995. How would you state a cautious conclusion? May the report retain only the CAPM nominal p-value?
8. **Chronological split.** HiTec-FF5 Alpha estimated on 2010–2014 is −0.098% per month, and the 2015–2019 error RMSE is 1.562 percentage points per month. Why is this still not

a real-time forecast or a strategy return? List at least three types of information that a future net-Alpha study would need.

Project 2 Solutions

Reader note: The data version, values, negative results, and null results below are retained from the historical study and exercises. They are historical teaching material, not new market results from the current constructed-data run of the I3 reader notebook. The data-authorization branch is skipped by default.

The question numbers correspond to the exercises. Multiply decimal returns by 100 to convert them to percentage points; the project outputs retain decimal precision.

1. Same-month RF and factor units

The dependent variable is HiTec's total return less RF for the same month: 2.0% − 0.3% = 1.7%. Mkt-RF is already the published spread between the market portfolio return and RF, so the right-hand side uses 1.7% directly. Subtracting 0.3% again would create a series different from the official factor and could count the risk adjustment twice. If the asset's total return is available but same-month RF is not, the excess-return response defined by this project cannot be constructed; the regression also cannot quietly be relabeled as a completed CAPM excess-return regression.

2. Interpret a conditional Beta

When same-month Mkt-RF rises by 1 percentage point, the HiTec-FF3 model's fitted conditional mean excess return rises by about 1.158 percentage points. The interval [1.075, 1.242] describes coefficient uncertainty under the specified model, sample, and HAC approximation. This is conditional co-movement between the market spread and industry excess return, not a causal effect of a market move, and it does not describe future returns known before the factor occurs.

3. Interpret model Alpha

0.2603% per month is the sample intercept for NoDur against the CAPM market benchmark. Its interval crosses zero and its nominal p-value is 0.1804; for this window, there is insufficient evidence to reject a zero model-conditional mean intercept. Holm p = 1.0000 is the adjustment for a family of nine fixed asset-by-model Alpha tests. Alpha depends on the factor benchmark and sample. It is not a realizable return, a demonstrated pricing error, or future Alpha.

4. Check the sample variance

Fitted and residual sample variance sum to 0.000582943 + 0.000456543 = 0.001039486, equal to total variance at the reported precision. The fitted share is 0.000582943 / 0.001039486 ≈ 56.080%; the residual share is about 43.920%. This is sample variance accounting between an OLS projection and its residual over the same window. It does not mean the market caused 56.080% of returns or prove that the same proportions will explain future risk.

5. Compare FF3 and FF5

First, the common Mkt-RF and SMB series in the FF3 and FF5 official releases are not identical in the selected months, so the two official models are not strictly nested. Second, factor spreads may be correlated, so the R^2 change cannot be uniquely assigned to `RMW/CMA.` Third, this is in-sample fit over the same historical sample; adding variables generally helps in-sample fit, but that does not establish better forecasts, a causal increment, or net returns. To isolate the contribution of the added factors, hold the same baseline factors fixed and then add `RMW/CMA`, labeling the result as a custom specification.

6. HAC 3/6/12 sensitivity

With the same asset, sample, and factor design, the OLS intercept point estimate remains −0.6629% per month. Changing the HAC lag length to 3, 6, or 12 changes the standard error, 95% interval, and p-value because it changes which lagged covariance terms enter the covariance estimate. The three calculations reuse the same 120 months; they are neither three datasets nor independent replications. All three intervals being below zero means only that this model's single negative intercept is relatively stable across these specified bandwidths. It remains subject to the model, sample, estimation assumptions, and the nine-test Holm boundary.

7. Multiple testing and negative results

A cautious statement is: all three energy-model HAC 6 intercepts are negative, and their unadjusted p-values are below 0.05; after Holm adjustment across the nine intercepts shown in this report, none is below 0.05, and the minimum is 0.0549. Therefore, do not claim significance after adjustment, much less that future energy investing will lose money. The report must not retain only the CAPM nominal p-value: it should preserve FF3, FF5, NoDur, HiTec, and results near zero to avoid selective reporting. It should also say that Holm adjusts only the nine listed tests, not any unrun specification search.

8. Chronological split

2010–2014 and 2015–2019 are separate historical windows, but the current data are not a month-by-month point-in-time archive, so the parameter versions may not be those available at the time. More importantly, the 2015–2019 monthly factors were fully known only after they were realized within each return interval. The RMSE therefore uses ex post, contemporaneous factors and measures historical attribution error, not forecast error known in advance or the net return of a tradable strategy.

At a minimum, a net-Alpha study would also need to define an executable signal and its availability time; the historical security universe, delisting data, and constituent versions; position and rebalancing rules; order and execution times and prices; turnover and commissions, taxes, spreads, and market impact; financing and borrow costs; cash and corporate-action treatment; and a holdings, orders, fills, and cash ledger. Without these inputs, net-of-cost returns cannot be calculated and strategy feasibility cannot be assessed.

Project 2 Research Report: CAPM and Fama–French Factor Attribution

> Reader note: The data version, values, negative results, and null results below are retained from the historical study. They are historical teaching material, not new market results from the current constructed-data run of the I3 reader notebook. The package's default Project 2 notebook is a separate constructed-data demonstration: its three models use the same generated factor columns and are strictly nested. Its outputs are not the historical market results reported here. The data-authorization branch is skipped by default.

This project uses the Capital Asset Pricing Model (CAPM), the Fama–French Three-Factor Model (FF3), and the Fama–French Five-Factor Model (FF5). Ordinary Least Squares (OLS) estimates the coefficients; Heteroskedasticity and Autocorrelation Consistent (HAC) covariance estimation is used for inference; and confidence intervals (CIs) and the Holm step-down multiple-testing adjustment describe single-estimate uncertainty and adjustment across the stated test family, respectively. The theory and worked calculations are covered in Chapters 8 and 15.

Historical status of the original project: **verified** (this does not mean the market analysis was re-verified for this reader package); completed by the root author and independent reviewer. This report is based on the fixed configuration and frozen data manifest; the tables were calculated by a reproducible CLI.

Conclusions

For the 120-month historical snapshot from January 2010 through December 2019, in-sample R^2 under FF5 is 0.727, 0.893, and 0.696 for NoDur, HiTec, and Enrgy, respectively; under CAPM it is 0.561, 0.829, and 0.597. This comparison has a higher in-sample fitted variance share under FF5, but this means only that the contemporaneously published factors provide a larger linear fit to the same historical returns.

All six Alpha estimates for NoDur and HiTec have HAC 95% intervals that cross zero. All three Enrgy Alpha estimates are negative, and their individual HAC 6 intervals lie below zero. After Holm adjustment across the nine asset-by-model Alpha estimates shown in this report, the minimum p-value is 0.0549; none is below 0.05. This does not prove that energy portfolios will continue to underperform, and Alpha is not an asset attribute independent of the model. All negative and near-zero results are retained.

The study has no causal identification, historical point-in-time data, signal, order, or execution ledger. It does not test a trading advantage. Costs, financing, and borrowing are not applicable to these data; no after-cost strategy return or Alpha was calculated.

Research question, data, and timing

The questions are: What conditional relationships do the excess returns of three industry portfolios have with the same-month published spreads for the market, size, value, profitability, and investment factors? How do Beta, model-specific intercepts, R^2, and residuals change as factors are added? How sensitive are the results to the HAC bandwidth and fixed historical slices?

The assets are Non-Durables (NoDur), Hi-Tech (HiTec), and Energy (Enrgy) from Kenneth R. French's U.S. 10 industry portfolios, using value-weighted monthly total returns with dividends. The French Data Library and methodology are described in the official library, industry portfolio description, FF3 description, and FF5 description. The original studies are Fama and French (1993) and (2015): 1993, 2015.

The inputs are the current release snapshot downloaded and frozen on 2026-10-05; both the FF3 and FF5 file headers identify the 202608 CRSP version. The data manifest records the ZIP, member CSV, processed CSV path, and SHA-256 for each of the three inputs. The CLI checked all hashes, ZIP CRCs, archive members, and byte identity with the saved original CSVs, then validated the actual input through the shared monthly reader. The project did not re-download, overwrite, or rebuild the production data.

French files are published in percent units; the frozen processed CSVs were divided by 100 once, and this project reads decimal returns without another conversion. The source missing-value sentinels −99.99 and −999 were converted to empty fields during preprocessing. Any missing or infinite selected value, missing month, duplicate month, or misalignment causes an error; gaps are neither filled nor compressed. The main sample is fixed at 2010-01 through 2019-12, with exactly 120 matching monthly PeriodIndex values for each asset and factor.

Official documentation states that, beginning with the January 2025 release, the library switched from the older CRSP FIZ format to CIZ Flat File Format 2.0, which uses a different monthly dividend-reinvestment timing convention. The source of the RF proxy also changed in June 2024 from Ibbotson Associates to the ICE BofA US 1-Month Treasury Bill Index. The current historical files are one revised snapshot, not a historical point-in-time (PIT) archive. A public download is not a general redistribution license; external use must retain Fama and French attribution and check the current terms.

Month labels identify return intervals. They do not provide historical `event_time`, `available_at`, `signal_time`, or `trade_time.` The manifest's download date records when the current snapshot was obtained; it cannot establish what was knowable historically. There is no signal and no future-return label.

Model, inference, and interpretation

For asset i and month t, the target is y(i,t) = R(i,t) − RF(t), and the model is:

$$y_{i,t} = \alpha_i + \boldsymbol{\beta}_i^{\mathsf{T}} \mathbf{f}_t + \varepsilon_{i,t}.$$

RF is subtracted once from the asset's dividend-inclusive total return; Mkt-RF, SMB, HML, RMW, and CMA are published spreads and enter the right-hand side in their original form. CAPM uses Mkt-RF and RF from the FF3 file; FF3 and FF5 each use the full set of factors and RF from their corresponding official file. Units are decimal monthly returns. There is no inner join that drops months, no date shift, and no repeated percentage conversion.

OLS includes an intercept. Alpha is a conditional mean intercept for a particular sample and model benchmark; it changes when the factor set or sample changes. It is not a demonstrated pricing error, skill, or future return. Beta is a linear conditional loading given the other factors in the model. For example, Beta = 0.8 means that when a same-month factor spread rises by 1 percentage point, the fitted conditional mean excess return changes by about 0.8 percentage points. This is statistical co-movement, not causation. SMB is small minus big, HML is high minus low book-to-market, RMW is robust minus weak profitability, and CMA is conservative minus aggressive investment, using the published spreads. Portfolio Betas should not be over-interpreted as firm-level mechanisms.

The main results use HAC standard errors with a Bartlett kernel, six monthly observation-row lags, a finite-sample correction, and a t reference distribution. Standard errors, intervals, and p-values for all coefficients were also calculated with 3, 6, and 12 monthly lags. Changing the HAC bandwidth does not change the OLS point estimates; it changes inference. HAC does not add omitted variables, resolve endogeneity, remove structural change, or validate error assumptions. Small-sample intervals and p-values are approximations that depend on assumptions.

R^2 is the in-sample proportion of squared error reduced relative to the mean benchmark; adjusted R^2 accounts for degrees of freedom. With an intercept, the OLS residual sample mean is close to zero. We checked that, for the same 120-month target and ddof=1, the variances satisfy Var(y)=Var(fitted)+Var(residual). This is an in-sample accounting identity for the OLS

projection, not a causal decomposition or evidence of future predictive ability. Because factors may be correlated, marginal Betas are not converted into independent, additive factor risk shares.

The nine-test Alpha family consists of three fixed assets crossed with three fixed models. The report gives nominal HAC p-values and Holm step-down adjusted values. Holm covers only these nine displayed intercepts, not other model searches, factor definitions, or window choices. Beta intervals are pointwise and do not adjust for the full specification search.

The OLS and four-month hand calculation examples are in Chapter 15, Sections 6 and 8; the step-by-step numerical Bartlett HAC sandwich-matrix example is in Section 6.4 of that chapter. The 95% intervals in the tables are pointwise; Holm adjustment applies only to the p-values of the nine intercepts, so these intervals are not simultaneous confidence intervals.

Nine full-sample empirical results

Alpha and its confidence interval are in percentage points per month; Beta is dimensionless, with its HAC 6 95% interval in brackets. A dash means the factor is not included in that model; it does not mean the estimated value is zero. Exact results, t-statistics, and p-values for every coefficient are in the HAC 6 coefficient CSV (historical external `evidence/data` not bundled).

NoDur: Non-Durables

Model	Alpha (% / month) [95% CI]	Alpha p / Holm p	Mkt-RF	SMB
CAPM	0.2603 [−0.1223, 0.6429]	0.1804 / 1.0000	0.646 [0.539, 0.752]	—
FF3	0.1195 [−0.2392, 0.4782]	0.5108 / 1.0000	0.747 [0.646, 0.848]	−0.424 [−0.611, −0.237]
FF5	−0.0472 [−0.3653, 0.2710]	0.7695 / 1.0000	0.812 [0.723, 0.902]	−0.296 [−0.521, −0.071]

Model	HML	RMW	CMA	R^2 / adjusted R^2
CAPM	—	—	—	0.561 / 0.557
FF3	−0.117 [−0.304, 0.070]	—	—	0.645 / 0.636
FF5	−0.251 [−0.411, −0.090]	0.526 [0.291, 0.762]	0.481 [0.204, 0.759]	0.727 / 0.715

HiTec: Hi-Tech

Model	Alpha (% / month) [95% CI]	Alpha p / Holm p	Mkt-RF	SMB
CAPM	0.1414 [−0.1568, 0.4396]	0.3496 / 1.0000	1.097 [1.022, 1.171]	—
FF3	0.0051 [−0.2643, 0.2746]	0.9701 / 1.0000	1.158 [1.075, 1.242]	−0.155 [−0.306, −0.004]
FF5	0.1032 [−0.1486, 0.3550]	0.4185 / 1.0000	1.126 [1.019, 1.233]	−0.158 [−0.320, 0.003]

Model	HML	RMW	CMA	R^2 / adjusted R^2
CAPM	—	—	—	0.829 / 0.827
FF3	−0.337 [−0.485, −0.189]	—	—	0.864 / 0.861
FF5	−0.050 [−0.199, 0.099]	0.021 [−0.257, 0.299]	−0.668 [−0.962, −0.374]	0.893 / 0.888

Enrgy: Energy

Model	Alpha (% / month) [95% CI]	Alpha p / Holm p	Mkt-RF	SMB
CAPM	−0.9459 [−1.6167, −0.2752]	0.0061 / 0.0549	1.195 [1.081, 1.309]	—
FF3	−0.6629 [−1.3091, −0.0167]	0.0444 / 0.3111	1.065 [0.936, 1.195]	0.332 [0.078, 0.585]
FF5	−0.7323 [−1.3707, −0.0939]	0.0249 / 0.1995	1.090 [0.922, 1.258]	0.286 [0.049, 0.524]

Model	HML	RMW	CMA	R^2 / adjusted R^2
CAPM	—	—	—	0.597 / 0.593
FF3	0.692 [0.375, 1.008]	—	—	0.688 / 0.680
FF5	0.452 [0.046, 0.858]	−0.091 [−0.564, 0.381]	0.508 [−0.065, 1.082]	0.696 / 0.683

HiTec's market Beta is above 1; all three assets have positive market Betas. Other loading signs depend on the industry and benchmark. The HiTec FF5 SMB interval narrowly crosses zero; the Enrgy FF5 RMW and CMA intervals cross zero; and the NoDur FF3 HML interval crosses zero. All are retained. The three energy intercepts have individual HAC 6 intervals below zero, but none reaches the 5% threshold after the nine-test Holm adjustment. The intervals for the other six intercepts cross zero; HiTec FF3 is close to zero at 0.0051% per month.

HAC bandwidth sensitivity

The table gives 95% intervals for the nine Alpha estimates at each bandwidth, in percentage points per month. Full point estimates, standard errors, t-statistics, p-values, and intervals for all Alpha and Beta coefficients under HAC 3/6/12 are in the complete sensitivity CSV (historical external `evidence/data` not bundled).

Asset	Model	HAC 3	HAC 6	HAC 12
NoDur	CAPM	[−0.1276, 0.6482]	[−0.1223, 0.6429]	[−0.1532, 0.6738]
NoDur	FF3	[−0.2285, 0.4674]	[−0.2392, 0.4782]	[−0.2836, 0.5226]
NoDur	FF5	[−0.3453, 0.2510]	[−0.3653, 0.2710]	[−0.4124, 0.3181]
HiTec	CAPM	[−0.1620, 0.4448]	[−0.1568, 0.4396]	[−0.1744, 0.4572]
HiTec	FF3	[−0.2846, 0.2948]	[−0.2643, 0.2746]	[−0.2589, 0.2691]
HiTec	FF5	[−0.1768, 0.3832]	[−0.1486, 0.3550]	[−0.1449, 0.3513]
Enrgy	CAPM	[−1.6232, −0.2687]	[−1.6167, −0.2752]	[−1.5996, −0.2923]
Enrgy	FF3	[−1.2950, −0.0308]	[−1.3091, −0.0167]	[−1.3154, −0.0104]
Enrgy	FF5	[−1.3793, −0.0854]	[−1.3707, −0.0939]	[−1.3766, −0.0880]

The three individual energy intervals remain below zero at all three HAC bandwidths, but these are not three independent batches of evidence. The displayed nine-test Holm adjustment uses HAC 6 and leaves no intercept significant at 5%; separate Holm adjustments at HAC 3 or HAC 12 are not claimed. Changing the bandwidth affects inference, not the Beta or Alpha point estimates.

Fit, residuals, and risk decomposition

Residual metrics are in percentage points per month; Lag-1 is the sample correlation between adjacent months. A mean near zero is an in-sample property of OLS with an intercept. Monthly targets, fitted values, and residuals are in the residual CSV (historical external `evidence/data` not bundled).

Asset	Model	R^2	Fitted variance share	Residual variance share
NoDur	CAPM	0.561	56.080%	43.920%
NoDur	FF3	0.645	64.496%	35.504%
NoDur	FF5	0.727	72.651%	27.349%
HiTec	CAPM	0.829	82.879%	17.121%
HiTec	FF3	0.864	86.407%	13.593%
HiTec	FF5	0.893	89.273%	10.727%
Enrgy	CAPM	0.597	59.670%	40.330%
Enrgy	FF3	0.688	68.799%	31.201%
Enrgy	FF5	0.696	69.598%	30.402%

Asset	Residual SD	RMSE	MAE	Maximum absolute value
NoDur	2.137	2.128	1.686	7.278
NoDur	1.921	1.913	1.455	5.696
NoDur	1.686	1.679	1.297	5.314
HiTec	1.864	1.856	1.494	5.360
HiTec	1.661	1.654	1.364	4.075
HiTec	1.475	1.469	1.197	3.758
Enrgy	3.674	3.659	2.794	11.684
Enrgy	3.232	3.218	2.541	9.437
Enrgy	3.190	3.177	2.486	9.482

Asset	Lag-1
NoDur	−0.063
NoDur	−0.072
NoDur	−0.200
HiTec	−0.146
HiTec	−0.180
HiTec	−0.178
Enrgy	0.009
Enrgy	0.043
Enrgy	0.075

R^2 corresponds to the fitted variance share and describes in-sample variance accounting. It does not show that future volatility will fall, that factor risk is tradable, or that `RMW/CMA` alone caused the FF5 improvement.

Figure descriptions:

- Alpha figure: The x-axis is monthly Alpha (percentage points per month); the y-axis identifies each asset-by-model estimate; points show estimates, horizontal lines show HAC 6 95% intervals, and the vertical line marks zero.
- Loading figure: The five panels show market, size, value, profitability, and investment. The x-axis is conditional Beta; the y-axis identifies each asset-by-model that includes the factor; lines show HAC 6 intervals. Factors not included in a model are not shown as zero.
- Variance figure: The x-axis is each model's fitted and residual shares of excess-return variance for the same sample, as percentages; the y-axis identifies the nine models. Teal is the fitted component and orange the residual component; each bar sums to 100%.
- Residual figure: The x-axis is the monthly return interval, labeled at month-end for plotting; the y-axis is realized same-month excess return less its fitted value, in percentage points per month. The three panels are the three industries, and colors distinguish CAPM, FF3, and FF5. These are not future forecast errors.
- Chronological attribution-error figure: The x-axis is 2015–2019 same-month attribution RMSE, in percentage points; the y-axis identifies each asset-by-model. Horizontal bars show RMSE after coefficients estimated on 2010–2014 are applied to factors realized during each evaluation month. This is not an ex ante forecast or strategy result.

FF3 and FF5 factor differences

The main results retain the corresponding official series. Differences in common columns over 2010–2019 are:

Field	Maximum absolute difference (decimal / month)	Number of different months
Mkt-RF	0.0003	40 / 120
SMB	0.0137	119 / 120
HML	0	0 / 120
RF	0	0 / 120

Both original files identify the 202608 CRSP version. Locally, each was converted from percent to decimal once; the selected HML and RF series match exactly, while Mkt-RF and SMB differ. The difference in SMB construction has an official explanation: the FF3 construction description uses six size-by-book-to-market portfolios, while the FF5 construction description averages three size spreads formed on book-to-market, profitability, and investment. Thus, not all SMB construction differences can be called unknown. This project has not reproduced the month-by-month differences from the underlying portfolios. The specific cause of the Mkt-RF file-value difference is also unknown; we do not guess that it is rounding, revision, or a release error. Because the common-factor values are not identical, the official FF3 and FF5 models are not strictly nested. We do not run a nested-model F-test or attribute the full R^2 increase to the added `RMW/CMA` factors. To isolate their contribution, hold the same baseline factors fixed and then add `RMW/CMA`, labeling the result a custom specification and a new study.

Chronological attribution check

Intercepts estimated on 2010–2014 compared with the full-window intercepts (percentage points per month):

Asset	Model	Training Alpha	Full-window Alpha	Full window minus training
NoDur	CAPM	0.525	0.260	−0.264
NoDur	FF3	0.379	0.119	−0.260
NoDur	FF5	0.144	−0.047	−0.191
HiTec	CAPM	−0.147	0.141	0.288
HiTec	FF3	−0.252	0.005	0.257
HiTec	FF5	−0.098	0.103	0.201
Enrgy	CAPM	−0.752	−0.946	−0.194
Enrgy	FF3	−0.617	−0.663	−0.046
Enrgy	FF5	−0.585	−0.732	−0.148

Applying the training estimates to realized same-month factors in 2015–2019 gives these error statistics (percentage points per month):

Asset	Model	Mean error	RMSE	MAE	Maximum absolute error
NoDur	CAPM	−0.541	2.347	1.747	7.570
NoDur	FF3	−0.666	2.205	1.656	5.800
NoDur	FF5	−0.459	1.932	1.516	5.293
HiTec	CAPM	0.601	2.075	1.682	5.062
HiTec	FF3	0.506	1.798	1.495	4.224
HiTec	FF5	0.282	1.562	1.257	3.597
Enrgy	CAPM	−0.369	4.195	3.366	10.221
Enrgy	FF3	−0.242	3.660	2.994	9.773
Enrgy	FF5	−0.335	3.832	3.135	10.537

This fixed split was not preregistered. The 2015–2019 factors were known only after they were realized in each evaluation month. This is therefore a chronological coefficient-stability check and a same-month attribution error, not PIT evidence, an ex ante forecast, a strategy return, or a genuine holdout return. Monthly errors are in the chronological attribution error CSV (historical external `evidence/data` not bundled); coefficient differences are in the stability CSV (historical external `evidence/data` not bundled).

Costs, limitations, and data needed next

Costs, financing, and borrowing are **not applicable** to this project: it has no signal, target positions, turnover, orders, execution prices, or transaction ledger. Costs were not assumed to be zero, and monthly Alpha was not annualized as a strategy return.

A future net-Alpha study would need an executable signal and its generation time; the historical asset universe, delistings, and industry constituents; the data versions available at the time; weights and rebalancing rules; order and execution times; execution prices and bid-ask spreads; commissions and taxes; market impact; financing and borrowing costs; dividend and corporate-action rules; and daily or transaction-level holdings, orders, fills, and cash records. The evaluation design would also need to be fixed in advance to avoid using future factors or revised inputs. The current data contain none of these fields, so they cannot establish feasibility, net return, or a future trading advantage.

The conclusion is limited to historical series for three currently revised industry portfolios. The regressions are observational conditional associations, not a causal design; the sample is limited; Alpha depends on the factor benchmark; the SMB construction difference between FF3 and FF5 is known, but the month-by-month differences have not been reproduced, and the specific cause of the Mkt-RF file-value difference is unknown; and there are no PIT timestamps. HAC and Holm do not cover all unrun specifications or establish a future advantage.

Deliverables and reproduction

Running the historical CLI generates:

- HAC 6 coefficients (historical external `evidence/data` not bundled): all `Alpha/Beta` estimates, intervals, p-values, and R^2.
- HAC sensitivity (historical external `evidence/data` not bundled): inference for all coefficients at 3/6/12 monthly lags.
- Monthly residuals (historical external `evidence/data` not bundled), chronological attribution errors (historical external `evidence/data` not bundled), coefficient stability (historical external `evidence/data` not bundled), and the JSON summary.
- Five historical Project 2 figures are retained with the report; the chronological same-month attribution error figure is not an ex ante forecast.

The reader package's default workflow is the offline notebook listed in the project README. It uses constructed data and does not reproduce the historical French-data results in this report. Its three models share the same generated factor columns and are strictly nested; write new outputs only to a unique subdirectory under the package-root runs/ directory. The historical full-study CLI, complete test suite, and market-data branch are outside the reader package's default execution scope.

The original CLI also supports launch by absolute path from any working directory. The historical working Notebook used a clean quant-course-venv kernel; the current package default uses the fresh python3 kernel through `run_notebook.py`. Current language projection: Project 2 Notebook. See the data dictionary for fields and time keys, the data manifest for paths, hashes, and source boundaries, and the historical independent review record (not included in this reader package).

Retained bitmap language: loading-panel titles read market excess return (Mkt-RF), size (SMB), value (HML), profitability (RMW), and investment (CMA), top to bottom. Residual panels read nondurables (NoDur), high technology (HiTec), and energy (Enrgy), top to bottom; `teal/orange/blue` identify `CAPM/FF3/FF5`. The historical bytes are unchanged; BIL23 owns the remaining English bitmap-title layer.

Chapter 16 — Time Series I: Dependence and ARIMA

Roadmap: First explain how a series depends on its past over time, then estimate a model that uses only past information and evaluate it with forecasts that advance strictly in time.
Prerequisites: Chapters 6 on probability, 7 on statistics, 8 on regression, and 11 on returns and performance measures. By the end, you will be able to distinguish price levels from returns; define weak and strict stationarity; explain the difference between white noise and IID; calculate the mean, variance, and autocorrelation of an AR(1) by hand; read ACF/PACF plots; compare AR, MA, ARMA, and ARIMA models; diagnose residuals; calculate prediction intervals; and write a month-by-month expanding-window forecast.

1. Learning Objectives

By the end of this chapter, you should be able to do five things:

1. Treat a time index with a stated frequency as part of the data; identify missing months, unsorted observations, and missing values; and refuse to compress time into row numbers with no gaps.
2. State weak-stationarity conditions from finite means, variances, and autocovariances, and explain how they relate to strict stationarity, white noise, and independent and identically distributed observations.
3. Derive the stationary mean, variance, autocovariance, ACF, and multi-step forecast-error variance from the recursion $X_t = c + \phi X_{t-1} + \varepsilon_t$.
4. Use the lag operator to explain AR, MA, ARMA, and ARIMA models; understand that ACF/PACF patterns are clues for candidate models, not automatic model selectors.
5. Restrict model-order and parameter choices to the training window, use only observations available before each forecast origin, and compare forecasts with a naive benchmark.

State mathematical units consistently: monthly returns are decimals, so 0.01 means a 1% return for that month; means and forecasts are also in monthly-return decimals; variances are in squared return decimals; ACF and PACF are dimensionless; and lag k means k observed months back, not k calendar days of equal length.

2. Why Quants Need This

A time-series model turns the question "Does the recent value contain information about the next period?" into a question that can be stated and tested. In quantitative work, this often comes after data cleaning and before regression or strategy testing: first choose the target variable (a price, return, spread, or volatility proxy), describe how it depends on its own history, and then ask whether that relationship would have been available at the forecast time.

Storing daily, monthly, or quarterly data as a column of numbers discards the spacing between observations. If an asset has no observation for March 2024, applying `dropna()` and treating

February and April as adjacent steps no longer makes an AR(1) a "one-month lag." Modeling a cumulative price level as though it were a stationary return can make a trend look like strong correlation. If you choose an order using the full data and then call the second half an out-of-sample forecast, the future has already influenced the research decision.

The chapter follows a complete chain: define the observation clock → calculate autocovariance and correlation → represent dependence with a low-order model → estimate from training data → inspect what the model leaves behind → forecast forward one period at a time → compare with a naive benchmark. The time index is not decoration; it determines the meaning of a lag and the forecast target.

3. Intuition

Imagine two "monthly-return machines." Both have a long-run average monthly return of 0.5% and a long-run one-month standard deviation of 4%. The first draws a new independent card each month. The second carries forward 70% of the previous month's deviation from its long-run mean. Their averages and volatility alone do not distinguish them; if the second machine was above average last month, it is more likely to be above average this month too. The difference lies in the ordering over time.

This kind of "memory" is described by autocorrelation. Positive lag-1 autocorrelation means neighboring months usually deviate from the mean in the same direction; a negative value means they tend to move in opposite directions. A value near zero says only that linear correlation is small; it does not rule out nonlinear dependence, volatility clustering, or common external shocks.

A common misconception is that a return series that fluctuates smoothly around a level proves that its data-generating process is stationary. A structural break, a few extreme months, or a slow stochastic trend can all mislead a finite-sample plot. A plot raises a question; theoretical assumptions and later tests provide different kinds of evidence.

3.1 A Stochastic Process Is Not a Single Line

A stochastic process is a time-indexed family of random variables $\{X_t\}$: before observation, a different return could occur in each month. A realization is one set of values actually drawn from that family, $x_1, \ldots, x_n$. The three possible worlds below distinguish "across time" from "across worlds." This table identifies the objects; it is not a stationarity test.

Possible world	January return	February return	March return
A (the path actually observed)	1%	−1%	2%
B	−1%	2%	0%
C	0%	1%	−2%

The January column corresponds to the random variable X_1; row A is one realization. $\mathbb{E}[X_1]$ weights the possible worlds, while the sample mean $\bar{x}$ averages along the observed path; these are different operations. Estimating population moments from a long path also requires suitable ergodicity or law-of-large-numbers conditions; stationarity alone does not ensure them. For example, first draw a Z and then set $X_t = Z$ for every month. This process is strictly stationary, but the mean of any long path remains the draw Z and does not automatically become $\mathbb{E}[Z]$. Financial data usually provide only one historical path, so whether moments look stable within a window is an evidential question that cannot be settled by the line's appearance.

The population parameters of the two return machines can be written in one table. Let $s = 0.04$ and $\mu = 0.005$, and use a stationary initialization for the AR machine:

Population quantity	IID machine	AR(1) machine, $\phi = 0.7$
Long-run mean	0.005	0.005
One-month variance	0.0016	0.0016
Innovation standard deviation	0.04	$0.04\sqrt{1-0.7^2} = 0.0285657$
Lag-1 covariance	0	$0.7(0.0016) = 0.00112$
Lag-2 correlation	0	$0.7^2 = 0.49$

Equal marginal means and variances do not determine how neighboring months move together. The AR machine needs a smaller innovation variance because past shocks also contribute to the current month's total variation. Sample values in a finite data table will not exactly match these population parameters.

4. Mathematical Foundations

Let X_t be a random variable indexed by month; t denotes the position of a monthly observation, and X_{t-k} is k observed months earlier. If P_t is the month-end price, the simple and log returns are, respectively:

$$R_t = \frac{P_t}{P_{t-1}} - 1, \qquad r_t = \log P_t - \log P_{t-1}.$$

If prices are quoted in euros, P_t is measured in euros per share, while returns R_t and r_t are dimensionless. For small returns, $r_t \approx R_t$, but they differ for larger moves. A price level, a simple return, and a log return are three different series; a model must state which one it forecasts.

This chapter represents calendar months with a monthly `PeriodIndex`. The lag from `2024-02` to `2024-03` is one month even when the two calendar months have different numbers of days; this does not assume February and March have the same number of days. A missing month is a missing observation, not an excuse to treat "the next CSV row that remains" as a

one-month lag. The core code requires consecutive months and rejects NaN, positive or negative infinity, and complex inputs; it does not fill values automatically.

For a stationary series with finite variance, its mean, variance, and autocovariance are:

$$\mu = \mathbb{E}[X_t], \qquad \gamma(k) = \mathrm{Cov}(X_t, X_{t-k}), \qquad \rho(k) = \frac{\gamma(k)}{\gamma(0)}.$$

Here, μ has the same units as X_t; $\gamma(k)$ has units equal to the square of the variable's units; $\rho(k)$ is dimensionless, and $\rho(0) = 1$. If X_t is a monthly return, $k = 12$ means 12 observed months apart; it does not automatically mean 365 days or any fixed number of calendar days.

5. Formal Definitions

Strict stationarity requires that, for every positive integer m, every set of observation positions $t_1, \ldots, t_m$, and every common shift h, the vectors $(X_{t_1}, \ldots, X_{t_m})$ and $(X_{t_1+h}, \ldots, X_{t_m+h})$ have the same joint distribution. It compares the full distribution and dependence structure, not only the mean and variance.

Weak stationarity (also called covariance stationarity) requires finite second moments, a mean that does not vary with t, and an autocovariance $\mathrm{Cov}(X_t, X_{t-k})$ that depends only on the separation k, not on the calendar starting point t. Strict stationarity with finite second moments implies weak stationarity; the converse generally fails. An exercise in Section 18 constructs a counterexample from independent series with different marginal distributions but identical first two moments.

White noise (WN) uses a second-order definition in this chapter: $\mathbb{E}[\varepsilon_t] = 0$, $\mathrm{Var}(\varepsilon_t) = \sigma^2 < \infty$, and zero autocovariance at every nonzero lag. White-noise observations need not be independent. **Independent and identically distributed (IID)** additionally requires independence across periods and the same marginal distribution. Zero-mean, finite-variance IID noise is white noise; "correlations close to zero" do not imply IID. If the noise is jointly Gaussian, zero correlation does imply independence.

The **autocorrelation function (ACF)** lists how $\rho(k)$ changes with lag k. The sample ACF divides sample autocovariance by sample variance; different denominator conventions across software can produce small finite-sample differences. The **partial autocorrelation function (PACF)**, after controlling for the intermediate lags $X_{t-1}, \ldots, X_{t-k+1}$, measures the remaining linear relationship between X_t and X_{t-k}. It can also be viewed as the coefficient on the farthest lag in the linear projection for AR(k).

For observations $x_1, \ldots, x_n$, a common sample definition is:

$$\hat{\gamma}(k) = \frac{1}{n} \sum_{t=k+1}^{n} (x_t - \bar{x})(x_{t-k} - \bar{x}), \qquad \hat{\rho}(k) = \frac{\hat{\gamma}(k)}{\hat{\gamma}(0)}.$$

Here n is the number of sample months, k is the lag in months, and $\bar{x}$ is the sample mean. Using n or $n-k$ in the denominator gives different finite-sample estimates; the code in this chapter explicitly uses the former convention. The PACF can also be defined by linearly projecting x_t

on all variables from lag 1 through lag k and reading the population projection coefficient on the final term x_{t-k}; finite-sample estimators can differ.

An **autoregressive model** (AR(p)) writes the current value as a finite sum of its own lags and an innovation. A **moving-average model** (MA(q)) writes the current value as a linear combination of the current and previous q innovations. An **autoregressive moving-average model** (ARMA(p, q)) contains both structures. An **autoregressive integrated moving-average model** (ARIMA(p, d, q)) first differences the series d times and then describes the differenced process with an ARMA model.

The **lag operator** L is defined by $LX_t = X_{t-1}$, so $L^k X_t = X_{t-k}$. Lag polynomials write finite-lag equations compactly. A process has a stable causal stationary solution when all roots of its AR polynomial lie outside the unit circle; a model is invertible when all roots of its MA polynomial lie outside the unit circle. For the AR(1) and MA(1) models here, these conditions reduce to $|\phi| < 1$ and $|\theta| < 1$, respectively. Polynomial sign conventions vary, so check the definition before using software.

A **prediction interval**, conditional on a model and information set, gives an uncertainty range for one future observation. It is wider than a confidence interval for the conditional mean because the future innovation also varies. With normal errors, parameters treated as known, and forecast variance V_h, a common form is:

$$\hat{X}_{T+h|T} \pm z_{1-\alpha/2}\sqrt{V_h}.$$

Here $\hat{X}_{T+h|T}$ has the same units as the series, V_h is the forecast-error variance for the future observation, and $z_{1-\alpha/2}$ is a standard-normal quantile. Estimated model parameters, heavy-tailed innovations, conditional heteroskedasticity, and structural changes affect interval coverage; an interval is not a guarantee.

6. Derivations

6.1 AR(1): From the Recursion to Stationary Moments

Start with an AR(1) with an intercept:

$$X_t = c + \phi X_{t-1} + \varepsilon_t,$$

c has the same units as X_t, ϕ is dimensionless, and ε_t has the same units as X_t. Assume the innovation has mean zero and variance σ^2, and is uncorrelated with the past. If a stable stationary solution exists, take expectations:

$$\mu = c + \phi\mu \quad \Longrightarrow \quad \mu = \frac{c}{1-\phi},$$

Under the causal stationarity condition $|\phi| < 1$, the long-run mean is the expression above. Define the centered variable $Y_t = X_t - \mu$ and substitute it into the recursion to obtain:

$$Y_t = \phi Y_{t-1} + \varepsilon_t.$$

Centering separates the long-run level from variation around it; here $\mathbb{E}[Y_t] = 0$. Using the fact that the innovation is uncorrelated with the previous value Y_{t-1}, square the equation and take expectations:

$$\begin{aligned}\gamma(0) &= \mathbb{E}[Y_t^2] = \phi^2\mathbb{E}[Y_{t-1}^2] + 2\phi\mathbb{E}[Y_{t-1}\varepsilon_t] + \mathbb{E}[\varepsilon_t^2] \\ &= \phi^2\gamma(0) + \sigma^2,\end{aligned} \qquad \gamma(0) = \frac{\sigma^2}{1-\phi^2}.$$

In the causal setup used here, the innovation is a new shock in the current period and is uncorrelated with the past. For the shock weights $1, \phi, \phi^2, \ldots$ to decay and the causal representation to converge, we must have $|\phi| < 1$. Rearranging the variance equation formally gives $\sigma^2/(1-\phi^2)$, but when $|\phi| \geq 1$ it no longer yields a finite, nonnegative causal stationary variance. This chapter discusses causal AR(1); other noncausal representations are out of scope.

Next, multiply $Y_t = \phi Y_{t-1} + \varepsilon_t$ by Y_{t-k} and take expectations. For $k \geq 1$, the current innovation is uncorrelated with the past value Y_{t-k}, so:

$$\gamma(k) = \phi\gamma(k-1).$$

Repeated application of the recursion gives $\gamma(k) = \phi^k\gamma(0)$, and therefore:

$$\rho(k) = \frac{\gamma(k)}{\gamma(0)} = \phi^k, \qquad k \geq 0.$$

If $\phi = 0.7$, the autocorrelation decays as $0.7, 0.49, 0.343, \ldots$; if $\phi < 0$, its sign alternates each period; if ϕ is close to 1, decay is slow and a finite-sample plot may look highly persistent. Slow decay in a finite plot does not by itself prove a unit root; unit-root tests and correlated common trends are covered in Chapter 17.

When $|\phi| < 1$, repeated substitution writes the centered series as:

$$Y_t = \varepsilon_t + \phi\varepsilon_{t-1} + \phi^2\varepsilon_{t-2} + \cdots.$$

The remainder after a finite expansion is $\phi^m Y_{t-m}$, which vanishes as m increases for a stable causal solution with finite variance. This shows that the current value is a weighted sum of recent shocks, with old-shock weights shrinking as ϕ^j. If the innovations are IID, this causal representation also helps establish strict stationarity. Second moments under white-noise assumptions give the weakly stationary moments derived above, but do not establish invariance of every joint distribution.

6.1.1 A Stable Recursion and a Stationary Initialization Are Different Things

$|\phi| < 1$ means that the effect of an old state decays; if the process starts from an arbitrary fixed X_0, the first few periods need not have stationary moments. Iterate the recursion for h periods, substituting $c = (1-\phi)\mu$ at each step, to obtain:

$$X_h = \mu + \phi^h(X_0 - \mu) + \sum_{j=1}^{h} \phi^{h-j}\varepsilon_j.$$

Assume the future innovations are independent of X_0, have mean zero, and are uncorrelated with one another:

$$\mathbb{E}[X_h] = \mu + \phi^h(\mathbb{E}[X_0] - \mu), \qquad \operatorname{Var}(X_h) = \phi^{2h}\operatorname{Var}(X_0) + \sigma^2 \sum_{j=0}^{h-1} \phi^{2j}.$$

The first term comes from the starting value; the second comes from innovations that have entered the process. A fixed starting value has variance zero. Let $\mu = 0.005, \phi = 0.5, \sigma = 0.06$ and start from $X_0 = \mu$:

h (months)	Mean	Variance from a fixed start	Variance from stationary initialization
0	0.005	0	0.0048
1	0.005	0.0036	0.0048
2	0.005	0.0045	0.0048
3	0.005	0.004725	0.0048

The means are equal, but the variance changes over time, so the process initialized at the left is not weakly stationary. If X_0 is independent of future innovations and has mean μ and variance $\sigma^2/(1-\phi^2)$, then the second moments are stationary from the start. For strict stationarity with IID innovations, the full distribution must also be initialized from the stationary causal solution. With normal innovations, draw $X_0 \sim N(\mu, \sigma^2/(1-\phi^2))$. Discarding a simulation warm-up period only makes the starting-value effect smaller; it does not make the process exactly stationary after any finite warm-up.

6.2 How Multi-Step Forecast Errors Widen

To derive the conditional forecast mean and forecast-error variance, assume that future innovations are independent and identically distributed, and that the entire set of future innovations is independent of the information available at the forecast origin; innovations have mean zero and variance σ^2. This is stronger than unconditional second-order white noise alone. Let $\mathcal{F}_T$ denote the information through month T. The one-step AR(1) forecast mean is:

$$\hat{X}_{T+1|T} = \mu + \phi(X_T - \mu).$$

The value X_T on the right is known at the forecast time. The conditional mean of a future innovation is zero, so this is the "conditional mean of next month's return," not the return that

will necessarily be realized next month. For the following month, X_{T+1} is still unknown, so replace it with the one-step forecast just calculated:

$$\hat{X}_{T+2|T} = \mu + \phi(\hat{X}_{T+1|T} - \mu) = \mu + \phi^2(X_T - \mu).$$

For an h-step horizon, the conditional forecast mean is $\mu + \phi^h(X_T - \mu)$. The forecast error includes future innovations:

$$X_{T+h} - \hat{X}_{T+h|T} = \varepsilon_{T+h} + \phi\varepsilon_{T+h-1} + \cdots + \phi^{h-1}\varepsilon_{T+1}.$$

Under these assumptions, conditional on $\mathcal{F}_T$, each future innovation still has conditional variance σ^2, and conditional covariances between different future periods are zero. Add the terms in the conditional forecast-error variance to obtain:

$$V_h = \sigma^2 \sum_{j=0}^{h-1} \phi^{2j} = \sigma^2 \frac{1 - \phi^{2h}}{1 - \phi^2}.$$

Here V_h has units of squared monthly-return decimals. As h increases, more future innovations are unknown; as $h \to \infty$, forecast uncertainty approaches the unconditional variance $\sigma^2/(1 - \phi^2)$. This derivation treats the parameters as known; fitting a model also introduces parameter-estimation uncertainty.

How the Information Set Changes the "Best" Forecast

The conditional mean is the optimal forecast under squared-error loss, given $\mathcal{F}_T$. If we assume only second-order white noise, the AR recursion above still gives a linear forecast based on past values, but not necessarily the conditional mean among all measurable functions. A concrete counterexample is $U \sim \text{Uniform}(0, 2\pi)$, $e_t = \cos(2^t U)$ $(t = 0, 1, \ldots)$: each period has mean zero and variance $1/2$, and cosine waves at different frequencies are orthogonal, so nonzero-lag autocovariances are zero; nevertheless:

$$e_{t+1} = 2e_t^2 - 1.$$

Given the past value e_t, the next value is fully known, and $\mathbb{E}[e_{t+1} \mid e_t] = 2e_t^2 - 1$ is generally not zero. Second-order white noise cannot replace the assumption that the conditional mean of a future innovation, given the available information, is zero. The numerical forecasts in this chapter explicitly use the stronger independent-innovation assumption; with conditional heteroskedasticity, the conditional forecast-error variance must also be rederived.

6.3 MA(1), Invertibility, and ARMA

First distinguish two objects both called a moving average. The rolling price mean $M_t = (P_t + P_{t-1} + P_{t-2})/3$ is a smoothing indicator calculated from observed prices, measured in currency per share. The MA(1) process $X_t = \mu + \varepsilon_t + \theta\varepsilon_{t-1}$ is a data-generating model whose filter acts on innovations that cannot be observed directly. The first can be calculated from prices; the second requires identifying or estimating the model's innovations. Let the innovations (in return decimals) be $\varepsilon_{t-1} = 0.02, \varepsilon_t = -0.01$ and $\theta = 0.5, \mu = 0.005$, with $X_t = 0.005 - 0.01 + 0.5(0.02) = 0.005$; this does not average the two monthly returns. The three-period rolling mean of prices $100, 103, 102$ is 101.6667 currency per share.

Use the MA(1) sign convention $X_t = \mu + \varepsilon_t + \theta\varepsilon_{t-1}$. The current value combines this month's new shock with last month's shock. For the centered series, two adjacent months are, respectively, $X_t - \mu = \varepsilon_t + \theta\varepsilon_{t-1}$ and $X_{t-1} - \mu = \varepsilon_{t-1} + \theta\varepsilon_{t-2}$. If innovations have mean zero, variance σ^2, and are uncorrelated across months, then:

$$\gamma(0) = \sigma^2(1 + \theta^2), \qquad \gamma(1) = \theta\sigma^2, \qquad \gamma(k) = 0 \ (k \geq 2), \qquad \rho(1) = \frac{\theta}{1 + \theta^2}.$$

The lag-0 variance adds the variances of the two terms because ε_t and ε_{t-1} are uncorrelated. At lag 1, the two series share only ε_{t-1}; the product of its coefficients is θ, so the covariance is $\theta\sigma^2$. At separations of at least two months, no innovation is shared, so the covariance is zero. Thus the population ACF of an MA(1) cuts off after lag 1. A finite-sample ACF will not become exactly zero; it fluctuates around the population pattern.

The theoretical ACF of an MA(1) is zero from lag 2 onward, while the PACF generally decays gradually. If $|\theta| < 1$,

$$\varepsilon_t = (1 + \theta L)^{-1}(X_t - \mu) = (1 - \theta L + \theta^2 L^2 - \cdots)(X_t - \mu),$$

Because $(1 + \theta L)^{-1} = 1 - \theta L + \theta^2 L^2 - \cdots$, when $|\theta| < 1$ the weights on the right decay geometrically; substituting current and past values then recovers the innovations stably. This is called invertibility. An MA process is a finite linear combination of white-noise terms and is therefore stationary for finite θ; noninvertibility does not mean nonstationarity. Invertibility is an identification restriction that gives an observable series a unique, stable innovation representation.

The PACF of AR(p) theoretically cuts off after lag p, while its ACF usually tails off. The ACF of MA(q) cuts off after lag q, while its PACF usually tails off. ARMA models usually have both functions tail off. These are ideal population patterns; noise, roots near the unit circle, and mixtures of model structures can blur them in finite samples. Do not choose an order only by eye from the "closest-looking" plot.

6.4 Differencing and Over-Differencing

First differencing is defined by $\Delta X_t = X_t - X_{t-1} = (1 - L)X_t$. An ARIMA$(p, d, q)$ model represents $\Delta^d X_t$ with an ARMA(p, q) model. If X_t is a price, ΔX_t is a price change in currency per share; if $X_t = \log P_t$, ΔX_t is exactly the log return over that interval, and only approximates the simple return for a small change. If X_t is already a return, differencing it gives a "change in returns," not the usual holding-period return.

From input to output, first differencing subtracts last month's level from this month's level and drops the first month, which cannot be paired. For example, for month-end prices [100, 101, 103, 102] euros per share, the differenced output is $[1, 2, -1]$ euros per share; this is a price change, not a percentage return. Differencing log prices instead gives an approximation to the simple return. Differencing aims to remove a shared accumulation in levels under assumptions such as a stochastic trend; it does not automatically make every series stationary, and it changes the forecast target.

Differencing can remove a stochastic trend, but it also loses the first sample row and changes the question. If the original monthly series is already white noise, $X_t = \varepsilon_t$, differencing it gives:

$$\Delta X_t = \varepsilon_t - \varepsilon_{t-1}.$$

This is the boundary case of an MA(1) with $\theta = -1$; its variance is $2\sigma^2$, lag-1 autocovariance is $-\sigma^2$, and ACF(1) is $-1/2$. Adding a difference has instead created pronounced negative correlation, known as over-differencing. Choose d only with a reason tied to the target variable and trend; Chapter 17 discusses ADF tests, deterministic trends, and unit-root identification in detail.

6.5 Returning a Differenced Forecast to the Original Quantity

If a model forecasts $D_t = \Delta X_t$, its output is still a change. From $X_{T+h} = X_T + \sum_{j=1}^{h} D_{T+j}$, take the conditional expectation using the same information set:

$$\hat{X}_{T+h|T} = X_T + \sum_{j=1}^{h} \hat{D}_{T+j|T}.$$

The last observed price is 102 euros per share, and the predicted changes for the next two months are 1 and 2 euros per share, giving level forecasts of 103 and 105. If the two change-forecast errors v_1, v_2 each have variance 4 and covariance 1, the second-period level error is $v_1 + v_2$, with variance $4 + 4 + 2(1) = 10$ euros squared per share squared. Do not simply add the endpoints of separate one-period intervals.

Log prices add another complication: if $X = \log P$, after forecasting log level m, e^m is usually not $\mathbb{E}[P \mid \mathcal{F}_T]$. Under a conditional normal distribution with variance V, the conditional mean price is $e^{m+V/2}$, while e^m is the conditional median. Without a specified conditional distribution, do not apply this correction mechanically. The chapter's `forecast_arima` returns a forecast of whichever series it receives; if the original level is passed with $d = 1$, statsmodels has already transformed the forecast back to levels internally, so do not cumulate it again.

6.6 How Long Is One Lag?

The population ACF/PACF objects were defined in Section 5; sample estimates suggest candidates, while out-of-sample errors show how a candidate forecasts the next period. Both depend on a valid observation clock: month labels use calendar time, trading-day numbers use trading time, and transaction numbers use event time. One step of a transaction-level AR(1) means the next transaction, which may occur milliseconds or hours later; it is not automatically a one-minute forecast. For a one-minute forecast, first define a one-minute grid and specify what a no-trade interval means.

In the missing-month example, February to April is two steps. Given the February state, the April conditional-mean deviation in the same AR(1) model is multiplied by ϕ^2, and the error variance is $\sigma^2(1 + \phi^2)$. Using ϕ and σ^2 as though the rows were adjacent gets both the target and the risk wrong. The monthly interface in this chapter stops when it finds a gap; it does not fill months or choose a new clock for the researcher.

7. Financial Interpretation

When modeling an asset price P_t, a long-run upward trend can make P_t and P_{t-1} highly correlated even when the tradable return R_t is not predictable. High ACF in a price level cannot be translated directly into "next month's return is predictable." First define the return interval, then state whether the target is a month-end-to-month-end simple return, log return, or another contractual return.

If monthly returns follow a stable AR(1), a positive ϕ means that returns recently above the mean partly persist in the model; a negative ϕ means linear reversal in the model. This describes time dependence only. An estimate could reflect market microstructure, nonsynchronous trading, dividend-reinvestment conventions, crisis-regime shifts, or sample selection. The univariate model in this chapter explains no causal mechanism and says nothing about trading costs, borrow availability, or capacity.

Linear return autocorrelation and volatility clustering are separate issues. Even when the return ACF is near zero, $|R_t|$ or R_t^2 may remain persistent; the white-noise condition describes only second-order linear correlation in the series. This chapter develops ARMA tools mainly for the conditional mean; conditional-heteroskedasticity models are left to later chapters. Failure to reject the null of no residual linear autocorrelation with Ljung–Box does not prove that residuals are IID, normal, or homoskedastic.

Tradable timing cannot be inferred from a month label alone. If R_t is the return from the end of March to the end of April, the April return is not fully observable until that interval ends; using it to forecast "the April return" reverses time. A research design must specify data `available_at`, signal `signal_time`, the earliest executable `trade_time`, and the target `return interval`. French monthly portfolio files do not report historical release times month by month, so they describe the published historical series but do not provide a point-in-time training set.

8. Numerical Example

Complete example: AR conditional means, recursive code, and intervals

All values below are constructed for teaching; they are not market observations. Let:

$$X_t = 0.005 + 0.5(X_{t-1} - 0.005) + \varepsilon_t, \qquad \varepsilon_t \overset{\text{IID}}{\sim} N(0, 0.06^2).$$

This example additionally assumes that innovations are independent, normal, and independent of past information, so the linear forecasts below are also conditional means; second-order white-noise conditions alone are not enough. The long-run monthly mean return is 0.005, or 0.5%; $\phi = 0.5$, and the innovation standard deviation is 6 percentage points of monthly return. The stationary variance around the mean is:

$$\text{Var}(X_t) = \frac{0.06^2}{1 - 0.5^2} = \frac{0.0036}{0.75} = 0.0048,$$

Thus the unconditional standard deviation is about $\sqrt{0.0048} = 0.06928$, so the monthly-return standard deviation is 6.928%. The autocovariances are $\gamma(0) = 0.0048$, $\gamma(1) = 0.0024$, and $\gamma(2) = 0.0012$, respectively, corresponding to ACF values $1, 0.5, 0.25$. Each γ is in squared monthly-return units.

If this month's return is $X_T = 0.04$, which is 3.5 percentage points above the long-run mean, the next month's conditional forecast is:

$$\hat{X}_{T+1|T} = 0.005 + 0.5(0.04 - 0.005) = 0.0225.$$

The result is 2.25%, not 2% from multiplying 4% directly by 0.5. One month later, $\hat{X}_{T+2|T} = 0.005 + 0.5^2(0.035) = 0.01375$, or 1.375%. The deviation from the mean decays by a power of the lag. The same calculation is shown below for a short monthly series. Treat parameters μ and ϕ as known in this hand-worked example; the aim is to show the forecast steps, not estimate a model from three observations.

Month	Observed monthly return	Role
2025-01	0.0100 (1.00%)	Historical observation
2025-02	0.0300 (3.00%)	Historical observation
2025-03	0.0400 (4.00%)	Forecast origin T

The input is the last observed value, $X_T = 0.04$. First calculate $0.005 + 0.5(0.04 - 0.005) = 0.0225$ to obtain a conditional mean of 2.25% for 2025-04. Then replace the unknown realized 2025-04 value with its forecast and calculate $0.005 + 0.5(0.0225 - 0.005) = 0.01375$, the conditional mean of 1.375% for 2025-05.

```
import numpy as np
import pandas as pd

observed = pd.Series(
    [0.01, 0.03, 0.04],
    index=pd.PeriodIndex(["2025-01", "2025-02", "2025-03"], freq="M"),
    name="monthly_return",
)
```

Continued: the same complete example

```
mu = 0.005
phi = 0.5
x_t = float(observed.iloc[-1])
forecast_april = mu + phi * (x_t - mu)
forecast_may = mu + phi * (forecast_april - mu)

print("forecast target months:", observed.index[-1] + 1, observed.index[-1] + 2)
print("forecast means:", forecast_april, forecast_may)
assert np.isclose(forecast_april, 0.0225)
assert np.isclose(forecast_may, 0.01375)
```

The two values printed by the code match the hand calculation. It reads data only through 2025-03. To fit unknown μ, ϕ, pass `fit_arima` a sufficiently long training series ending strictly before the forecast target.

With known parameters and normal innovations, the one-step forecast-error variance is $V_1 = 0.06^2 = 0.0036$, with standard deviation 0.06. The 95% prediction interval for a future one-month return is about $0.0225 \pm 1.96(0.06) = [-0.0951, 0.1401]$, or approximately $[-9.51\%, 14.01\%]$. The two-step forecast error includes one more future innovation: $V_2 = 0.06^2(1 + 0.5^2) = 0.0045$, with standard deviation about 0.06708, so uncertainty is greater than at one step. A wide interval reflects the large one-month innovation and does not contradict predictability of the conditional mean. The interval omits estimation uncertainty in ϕ, μ, σ^2 and does not address heavy tails or volatility clustering.

Now consider an MA(1) with $\theta = 0.5, \sigma = 0.06$. Then $\gamma(0) = 0.0036(1 + 0.25) = 0.0045$ and $\gamma(1) = 0.5(0.0036) = 0.0018$, so $\rho(1) = 0.4$, while $\rho(k) = 0$ for every $k \geq 2$. Its ACF shape differs from that of an AR(1), although both can explain correlation between adjacent periods.

Finally, compare the two constructed return machines with common mean 0.005 and marginal standard deviation 0.04. Every nonzero-lag population ACF of the IID machine is zero. Set the AR(1) machine to $\phi = 0.7$; its innovation standard deviation should then be $0.04\sqrt{1 - 0.7^2} \approx 0.02857$, and its population ACF(1) is 0.7. This adjustment gives both machines the same theoretical unconditional variance, 0.04^2, without forcing their finite-sample variances to match exactly.

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I3 reader package root, create an independent environment using its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch16_time_series.ipynb
```

With seed 20261016, the default run constructs 480 months of IID and AR(1) returns from 1980-01, with mean 0.005, marginal standard deviation 0.04, and AR coefficient 0.7. The AR starting value is drawn from its stationary normal distribution; the innovation standard deviation is 0.04 times sqrt($1-0.7^2$). It checks theoretical moments and sample ACF/PACF, then trains on the first 360 months through 2009-12 and forecasts three months starting in 2010-01. Training residuals receive a Ljung–Box check with lag=12 and model_df=1. Over the final 120 months, expanding-window one-step ARIMA forecasts are compared with the mean available from each training window using RMSE.

The Notebook displays the existing bundled figure `fixtures/figures/a4/ch16_stationarity_acf.png`; displaying it does not regenerate it. `RUN_FRENCH=False` disables the historical market branch. Although it creates and prints `runs/ch16_time_series_<uuid>/`, this Notebook writes no CSV or PNG there. The runner separately saves the executed copy and status in a unique `runs/execution_*` directory; these directories serve different purposes. Constructed calculations are not evidence of market predictability or strategy net returns. The historical French figures and numbers in Sections 10–11 retain their original status.

The CLI and production tests below belong to the historical course working-root workflow and are not this package's default route. Readers need not restore production tests that are absent from the package. The I3 Chapter 16 CLI still reads French data unconditionally; the Chapter 17 CLI's historical branch requires `--french-data`. Neither replaces the default command above.

Historical Course Working Root and Original Implementation

The core implementation for this chapter is in `time_series.py`. `validate_monthly_series` requires a sorted, unique, consecutive `PeriodIndex(freq="M")` with finite real values; it explicitly rejects missing months, NaN, infinity, and complex numbers. `split_monthly_series` splits by row count into earlier and later observations without shuffling. `forecast_arima(train, ...)` receives only the training prefix; `rolling_origin_forecast` refits an expanding window before each target month and forecasts the next month. This teaching wrapper supports only nonseasonal differencing orders $d = 0$ or 1; it does not include exogenous regressors, seasonal terms, or complex missing-state estimation.

The statsmodels ARIMA likelihood implementation may issue numerical-convergence warnings for very small decimal returns. The fitting function in this chapter multiplies monthly return decimals by 100 to fit in percentage points, then scales forecast means, intervals, and residuals back to decimals; AR/MA coefficients do not change under this uniform linear scaling. `FittedARIMA.result` is the fitted statsmodels result in percentage-point units. Its intercept,

residuals, and innovation standard deviation remain in percentage points if read directly; use `.residuals` to obtain residuals in decimals.

For estimation, the input is a monthly-return series ending at the forecast origin, and the model order (p, d, q) must be specified first. `fit()` estimates parameters on that training sample; only then does `get_forecast()` produce future conditional means and prediction intervals. ACF/PACF help narrow the candidates. AIC/BIC can compare fitted candidates within the training window, followed by MAE/RMSE comparisons on a time-ordered validation window; the final holdout must not be used for repeated order selection.

For $d = 0$, this wrapper uses `trend="c"`. Statsmodels treats the trend term as a regression constant, so `const` here denotes the constant level or unconditional mean of the series. In the textbook recursion with intercept $X_t = c + \phi X_{t-1} + \varepsilon_t$, the two constants satisfy $c = (1 - \phi)\mu$; do not treat the fitted `const` as the recursion's c. For $d = 1$, the wrapper uses `trend="n"`, with no drift term estimated. The model also constrains AR roots to be stationary and MA roots to be invertible, and it raises an error for missing values. These computational settings do not prove that the data satisfy the model. Standard prediction intervals depend on the innovation structure; normal intervals also depend on a normal-error approximation, and parameter uncertainty is not automatically included when parameters have been estimated.

From the `quant_course/` project root, run the script and the focused tests:

```
.venv/bin/python code/ch16_time_series.py
.venv/bin/python -m pytest -q tests/test_ch16_time_series.py
```

The script uses fixed seed `20261016` to generate 480 months of constructed IID and AR(1) data, plots them, prints theoretical and sample moments and expanding-window simulation RMSE, and reads the French industry snapshot for descriptive statistics. The notebook `ch16_time_series.ipynb` must be rerun from its first cell using the `quant-course-venv` kernel. The notebook calls `src/` directly rather than reimplementing the core functions.

The following is a complete monthly split and one-step forecast example. Return decimals represent monthly returns; `target_period` is the target month of a forecast, not the actual `trade_time`.

```
import numpy as np

from quant_course.time_series import (
    forecast_arima,
    simulate_stationary_pair,
    split_monthly_series,
    training_mean_forecast,
)

monthly_returns = simulate_stationary_pair(
    n=120, seed=20261016, mean=0.005, marginal_sd=0.04, phi=0.7
)["ar1"]
train, test = split_monthly_series(monthly_returns, train_size=96)
prediction = forecast_arima(train, steps=1, order=(1, 0, 0))
baseline = training_mean_forecast(train, steps=1)
print("train through:", train.index[-1], "target:", prediction.index[0])
print("ARIMA forecast:", float(prediction.iloc[0]["forecast"]))
```

```
print("training-mean baseline:", float(baseline.iloc[0]))
print("realized test return, for retrospective scoring only:", float(test.iloc[0]))
assert prediction.index[0] == test.index[0]
assert np.isfinite(prediction.to_numpy()).all()
```

For sample ACF/PACF, `sample_acf_pacf` explicitly returns lags 1 through L, excluding lag 0. It uses statsmodels' ACF settings `adjusted=False, fft=False, missing="raise"` and PACF setting `method="ywadjusted"`. Finite-sample ACF/PACF confidence bounds depend on additional assumptions; crossing a line in the chapter's plot is not treated as proof of a model.

10. Visualization

The figure below shows 480 months of simulated data generated by the script with seed `20261016`. The top row has monthly observation time on the horizontal axis and monthly return in percentage points on the vertical axis. The bottom row has lag in months horizontally and dimensionless ACF vertically. Blue shows the sample ACF; orange shows the population ACF implied by the generating parameters. The IID process has an orange line at zero, while the AR(1) population correlation decays as 0.7^k. The blue line does not match the population curve at every lag because this is a finite sample. The figure contains no market returns, forecast returns, or strategy performance.

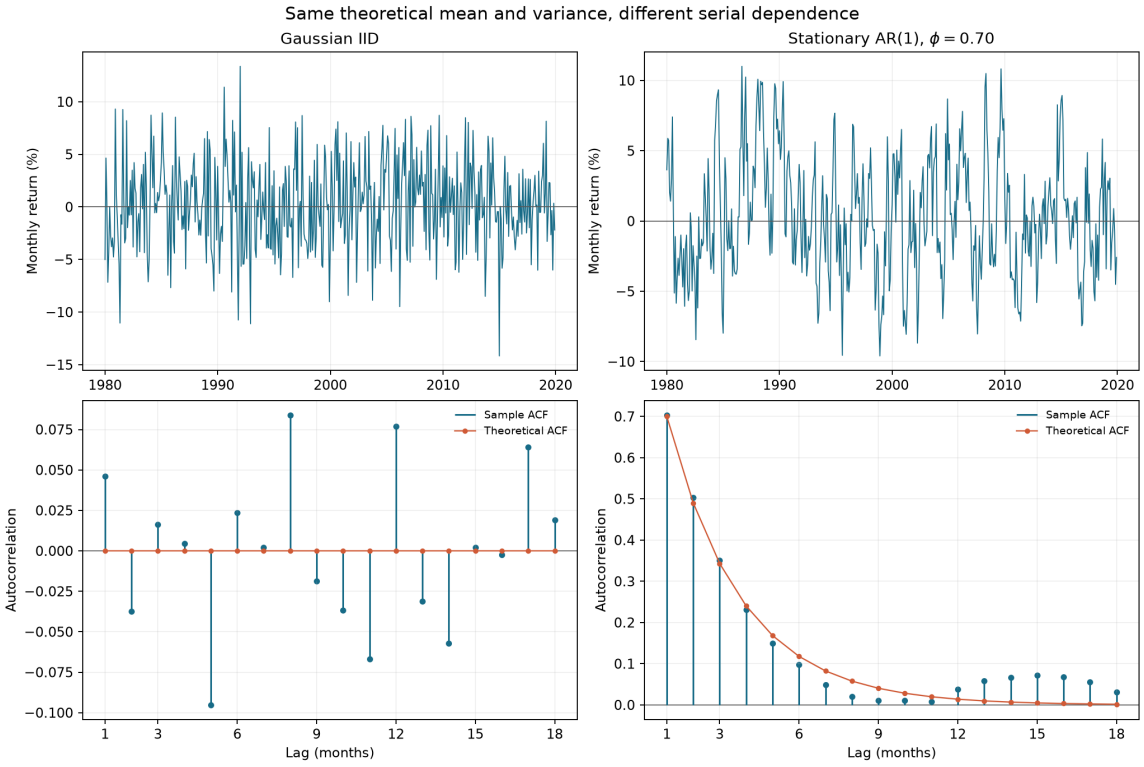


Figure 1: Constructed data with a fixed seed: monthly-return paths and sample/theoretical ACF for IID and AR(1). The top row is in monthly-return percentage points; the bottom row shows dimensionless autocorrelation, with lags measured in months.

11. Real Market Example

The local real-data sample is the current monthly value-weighted return cache for the French 10 industry portfolios, monthly industry-return CSV, with the original source at the data details page and the saved data manifest. The cache was parsed from a ZIP downloaded on 2026-10-05, selecting the "Average Value Weighted Returns — Monthly" block and dividing source percentages by 100. It contains 1,202 months, from 1926-07 through 2026-08. These are published industry-portfolio returns, not single-stock OHLCV prices or fund execution returns.

The script reads the NoDur column (consumer nondurables) and obtains a mean of 0.009411, sample standard deviation 0.045564, sample ACF(1)=0.094385, and ACF(12)=0.034820. These numbers confirm that the fields, units, and descriptive calculations ran; they do not show significance, stability, predictable returns, or strategy effectiveness. Monthly returns are not a sample whose homoskedastic IID properties are guaranteed by construction. The A1 manifest also says this cache is the current published version and cannot reconstruct the version visible in each historical month, so it cannot support a claim of a genuine historical rolling forecast.

The parsing rule preserves `month` as a `YYYYMM` string and then converts it to `PeriodIndex(freq="M")`; every industry column contains simple monthly returns in decimals. The data have no daily OHLCV block, so they cannot be converted into daily observations. The manifest has not verified a general license for redistribution outside the course; public download does not automatically permit unrestricted forwarding.

12. Common Mistakes

Treating month-end DatetimeIndex values as equally spaced days. February 29 to March 31 and March 31 to April 30 are both one observation month even though the number of calendar days differs. Use a monthly `PeriodIndex` for monthly series. If the task depends on actual days or business days, explicitly define a different time scale instead of guessing from row numbers.

Dropping missing months before calculating lags. Applying `dropna()` makes the remaining rows adjacent, so the model cannot see that the observations are two months apart. This chapter rejects gaps and missing values. To address one, return to the source, determine what the missingness means, and define an appropriate method; do not automatically fill with zero, forward-fill, or pretend the observations are equally spaced.

```
import numpy as np
import pandas as pd

from quant_course.time_series import validate_monthly_series

incomplete = pd.Series(
    [0.01, -0.02],
    index=pd.PeriodIndex(["2024-01", "2024-03"], freq="M"),
    name="monthly_return",
)
try:
    validate_monthly_series(incomplete)
```

```
except ValueError as error:
    print(type(error).__name__ + ":", error)
```

Confusing price levels with returns. For prices 100, 110, and 121, the price autocorrelation is usually high because the previous price enters the current price directly, while the simple returns are 10% and 10%. Even a good AR(1) fit to prices does not show that returns have the same predictive relationship. When needed, calculate returns for the stated research target and define the return interval separately.

Choosing an order because the ACF/PACF "cuts off." Sample correlations are estimates, and random spikes are likely somewhere among dozens of lags. A low-order plot pattern only suggests candidates. Select orders within the training window, then compare forecast loss on a time-ordered validation window. Do not repeatedly change p, q, d while looking at the final holdout.

Choosing transformations or estimating preprocessing on the full sample before splitting. A fixed causal backward difference, applied only to observations available at each origin, does not by itself use future information. Leakage can arise when full-sample decisions choose whether or how to transform, fit an estimated preprocessing step, impute values, or otherwise bring future information into past modeling decisions. Define the raw data with its availability times first; align training and validation cutoffs with forecast origins; fit each estimated transformation using training data only.

Calling a large Ljung–Box p-value "independent residuals." The test checks a set of linear autocorrelations for a selected lag set and asymptotic reference distribution. Small-sample power is limited, and it does not check autocorrelation in squared residuals, heavy tails, or parameter stability. Report the test's H_0, lags, model degrees of freedom, sample size, and residual plots, then state a bounded conclusion.

Starting from residual sample $e_1, \ldots, e_n$, calculate sample autocorrelations for lags 1 through h, then combine them as follows:

$$Q = n(n+2) \sum_{k=1}^{h} \frac{\hat{\rho}_e(k)^2}{n-k}.$$

The null hypothesis is that the population residual autocorrelations at these lags are jointly zero. Under the usual asymptotic calibration, the reference degrees of freedom are $h - m$, where m denotes `model_df` (for an ARMA fit, this usually subtracts the number of estimated AR and MA parameters). For example, when $h = 12, m = 1$, use 11 degrees of freedom. A small p-value suggests remaining linear structure; a large p-value only means this test did not provide enough evidence to reject the null.

```
import pandas as pd

from quant_course.time_series import ljung_box_diagnostic, simulate_stationary_pair

residuals = simulate_stationary_pair(n=120, seed=20261016)["iid"]
diagnostic = ljung_box_diagnostic(residuals, model_df=1, lag=12)
print(diagnostic.to_string())
print("Interpretation: a failure to reject is not proof of IID residuals.")
```

13. Quant Research Application

First narrow the research question to: "Do past monthly returns improve the point forecast for next month's return?" This is neither "Does AR(1) prove a causal momentum mechanism?" nor "Can we trade at no cost?" A complete design can follow these steps:

1. **Define the variable and clock.** Record the source, currency, frequency, adjustment/dividend convention, and monthly interval for the price or portfolio return. If a feature X_t uses data that are not complete until the end of month t, its `available_at` cannot precede the actual release time. Set `signal_time` only after all inputs are visible; `trade_time` must follow the signal and comply with the selected market's execution rules. Define target Y_{t+1} as the next month's return interval, not the same-period Y_t.
2. **Describe the data; do not infer stationarity from a plot.** Inspect separate plots of price levels and returns, missing months, outliers, distributions, and ACF/PACF. One step in a monthly PeriodIndex is one month; empirical returns are not necessarily stationary. This chapter does not perform unit-root or cointegration tests; those are covered in Chapter 17.
3. **Set training and validation windows in advance.** For example, train on the first 15 years and forecast one month at a time afterward; at each origin, refit parameters using only observations available before that origin. Choose orders and bandwidths within the training range and reserve the final period for one final validation. If the input source has no verifiable historical release time or version, downgrade the result to "descriptive analysis of the current snapshot"; do not call it a point-in-time backtest.
4. **Compare with naive benchmarks.** For returns near zero, the training mean or a zero-return forecast are clear candidates. At each origin, calculate the training mean using only earlier observations. Compute MAE/RMSE in target-return decimals and compare errors for the same forecast months. Retain the result if ARIMA does not consistently beat the benchmark.
5. **Diagnose and seek disconfirming evidence.** Check residual linear autocorrelation, distributional anomalies, parameter changes across windows, and errors across volatility regimes. Test sensitivity to prespecified model orders and forecast origins. If signal timing is invalid, forecast error does not beat the benchmark, improvement appears only in one period, or residuals/parameters show model failure, do not package correlation as predictable alpha.

At each step, the rolling function receives only the prefix observed earlier. It demonstrates data-access rules; it does not construct historical `available_at` or execute trades for you. The

current French monthly-return snapshot lacks month-by-month historical ingestion/release times and therefore cannot support a valid historical point-in-time forecast experiment.

14. Chapter Summary

Define the time step, return target, and index first; only then does a lag have economic meaning. Stationarity describes whether distributional or moment structure changes over time; white noise describes second-order correlation; ACF/PACF summarize linear relationships across lags. AR, MA, ARMA, and ARIMA models parameterize these relationships but do not automatically identify an economic mechanism.

AR(1) shows the chapter's central chain: $|\phi| < 1$ makes shock weights decay, giving the finite unconditional variance $\sigma^2/(1-\phi^2)$ and geometrically decaying ACF ϕ^k. The same recursion also gives forecasts that move toward the long-run mean and forecast-error intervals that widen with the horizon. After estimation, still check residuals and out-of-sample forecasts, and keep every input within the information set available at its forecast origin.

15. Exercises

All questions are answered in Section 18 of this chapter. Unless stated otherwise, ε_t denotes a zero-mean, finite-variance innovation. Hand calculations should show units and the key intermediate steps.

16-E01 (Concept). Distinguish strict and weak stationarity mathematically. Explain why strict stationarity implies weak stationarity when second moments are finite, then outline a construction that is weakly but not strictly stationary.

16-E02 (Concept). Is "white noise is IID" correct? Construct a process with mean zero, variance 1, zero autocovariance at every nonzero lag, and dependence; identify observable evidence of the dependence.

16-E03 (Concept). How do the typical population ACF/PACF patterns help distinguish AR(1), MA(1), and ARMA(1,1)? Explain why these patterns alone cannot determine the order.

16-E04 (Time axis). A CSV contains three monthly returns dated `2024-01, 2024-02, 2024-04`. After dropping missing rows and applying `shift(1)`, how many months apart are the second and third observations? How should time be represented and the gap handled?

16-E05 (Concept). Why does a price chart that keeps rising prove neither a unit root nor that monthly returns are unpredictable? Which question is left to Chapter 17?

16-E06 (Forecasting). How do the targets of a prediction interval and a confidence interval for the conditional mean differ? What uncertainties and distributional assumptions still need checking when using a normal AR(1) interval?

16-E07 (By hand: AR(1) moments). $X_t = 0.005 + 0.5(X_{t-1} - 0.005) + \varepsilon_t$, $\mathrm{sd}(\varepsilon_t) = 0.06$. Find the unconditional mean, variance, standard deviation, $\gamma(1)$, ACF(1), and ACF(2).

16-E08 (By hand: forecasting). For the process in E07, if this month is $X_T = 0.04$, find the conditional forecast means for next month and the month after, and the one- and two-step forecast-error variances. Assume known parameters, IID future innovations, and that the entire set of future innovations is independent of information at the forecast origin; each conditional variance is σ^2.

16-E09 (By hand: MA(1)). $X_t = \mu + \varepsilon_t + 0.5\varepsilon_{t-1}$, $\mathrm{sd}(\varepsilon_t) = 0.06$. Find $\gamma(0)$, $\gamma(1)$, ACF(1), and ACF at every $k \geq 2$. Is the model invertible?

16-E10 (By hand: differencing). If $X_t = \varepsilon_t$ is white noise with variance σ^2, calculate the variance, lag-1 autocovariance, and ACF(1) of ΔX_t. What differencing risk does this illustrate?

16-P01 (Programming). Generate two simulated 240-month series with a fixed seed. Calculate the AR(1) ACF/PACF through lag 12 and print the first lag and sample size. Then assert that strict validation rejects a sample with a missing month.

16-P02 (Programming). For a fixed-seed 240-month AR(1) simulation, train on the first 180 months and forecast the next month. Print the target month, ARIMA forecast, training-mean benchmark, and realized test value. Assert that the forecast index equals the first test month; do not pass the realized test value to the fitting function.

16-P03 (Programming). Fit ARIMA(1,0,0) to the P02 training data, run a lag-12 Ljung–Box test on training residuals, and print the test statistic, p-value, and degrees of freedom. In one sentence, state the null and what the test cannot prove.

16-C01 (Challenge: same moments, different autocorrelation). Construct monthly IID and AR(1) series with the same theoretical mean and variance. Choose a seed and sample size, plot both paths and their ACFs, and check the theoretical AR(1) ACF. Explain why finite-sample values may differ from theory and state which financial conclusions this does not support. Score 0–2 points for each of: parameters and assumptions; seed and reproducibility; moment-matching derivation; time axis/units; and evidence boundaries, for a total of 10 points.

16-C02 (Challenge: real-data research design). Use monthly French NoDur returns to design a study of whether past returns improve next-month return forecasts. Specify units, timestamps, rolling windows, benchmarks, residual diagnostics, and falsifiable conditions; assess whether the current cache supports a point-in-time backtest. Score 0–2 points for each of: source and version; target and timing; leakage-free validation; benchmarks and metrics; and limitations and disconfirming evidence, for a total of 10 points.

16. Programming Exercises

P01–P03 were listed in the previous section and respectively require a frequency contract, a forecast using historical data only, and residual diagnostics. Programming reference solutions must use the core implementation in `quant_course.time_series`; do not reimplement ACF or fitting logic in a notebook.

17. Challenge Problems

C01 compares population laws with finite-sample plots; C02 examines the gap between a public snapshot, information timing, and evidence for historical strategies. Both questions guard against projecting a simulation property or a current release date backward into information available at the time.

Additional Exercises (Independent IDs)

- **16-N01 (Identification and hand calculation; §§3.1, 6.1.1):** Run an AR(1) with unconditional variance 0.0048 from a fixed-mean starting value, $\phi = 0.5, \sigma = 0.06$. Find the variances of the first three months and explain why a stable recursion does not make the process stationary from its initial state. Must the mean of a long path converge to the population mean? Give a counterexample.
- **16-N02 (Diagnosis; §§6.2–6.3):** Given the cosine white noise with $e_t = 0.5$, find the next-period conditional mean. Why does it disprove "white noise cannot be predicted"? Separately state the inputs to an MA model and to a rolling average of prices.
- **16-N03 (Forecast reconstruction and research judgment; §§6.5–6.6):** The last level is 102; forecasts of the next two changes are 1 and 2, with error variances 4 each and covariance 1. Find the two level forecasts and the second-period error variance. If only February and April monthly returns are available, how should the lag between adjacent rows be handled? If ARIMA is fit to the original level with $d = 1$, should its output be cumulated again?

18. Solutions / Solution Hints

16-E01 Answer. Strict stationarity requires all finite-dimensional joint distributions to be invariant under a common time shift. Weak stationarity restricts only the constant mean, finite constant variance, and autocovariance that depends on lag alone. Under strict stationarity with finite second moments, the marginal mean and variance do not change over time, and a covariance between any two dates is determined by the joint distribution at their separation of k periods, so weak stationarity follows. The converse fails. Construct independent observations: at even periods, take $+1$ or -1 with probability $1/2$ each; at odd periods, take 0 with probability $1/2$, and $+\sqrt{2}$ or $-\sqrt{2}$ with probability $1/4$ each. Both types of period have mean 0 and variance 1, and all cross-period covariances are zero, so the series is weakly stationary. Its marginal distribution alternates, however, so the full series is not strictly stationary.

16-E02 Answer. Let Z_t be IID standard normal and define $\varepsilon_t = Z_t Z_{t-1}$. Its mean is zero and its variance is $\mathbb{E}[Z_t^2]\mathbb{E}[Z_{t-1}^2] = 1$. In products at any nonzero lag, at least one independent zero-mean factor appears only once, so every nonzero-lag autocovariance is zero. But adjacent squared values share Z_t^2:

$$\mathbb{E}[\varepsilon_t^2 \varepsilon_{t+1}^2] = \mathbb{E}[Z_t^4] = 3, \quad \mathbb{E}[\varepsilon_t^2]\mathbb{E}[\varepsilon_{t+1}^2] = 1, \quad \mathrm{Cov}(\varepsilon_t^2, \varepsilon_{t+1}^2) = 2.$$

Therefore adjacent innovations are not independent. White noise guarantees only second-order uncorrelatedness; it does not replace the IID assumption.

16-E03 Answer. The theoretical AR(1) ACF tails off as ϕ^k, while its PACF is zero after lag 1. The MA(1) ACF is zero after lag 1, while its PACF usually tails off. An ARMA(1,1) usually has both functions tail off. In practice, there are only finitely many observations, and ACF/PACF estimates are affected by estimation noise, mean estimation, outliers, and combinations of orders. Use plots to suggest a small candidate set, then assess it with training-window information criteria, rolling forecast loss, and residual diagnostics.

16-E04 Answer. After March is removed, `shift(1)` treats the February row as the previous observation for April even though they are two months apart. Keep the month labels, create a monthly `PeriodIndex`, and validate that its index is consecutive. Check the source and meaning of a missing month; use a clearly appropriate missing-data model or restrict the sample. Do not call the gap a one-month lag or fill it with zero by default.

16-E05 Answer. An upward trend may come from a deterministic trend, stochastic trend, structural change, adjustment convention, or sample selection; a price chart cannot distinguish these. High persistence in the price level does not establish linear predictability of the return $P_t/P_{t-1} - 1$. Chapter 17 discusses unit-root tests, deterministic trends, differencing, and cointegration; test results still depend on assumptions and sample design.

16-E06 Answer. A confidence interval for the conditional mean concerns uncertainty in estimating an unknown mean. A prediction interval concerns one future random observation and therefore includes future innovation variance. A standard normal AR(1) interval treats μ, ϕ, σ^2 as known and assumes future innovations follow the specified normal structure. In practice, also examine parameter-estimation uncertainty, heavy tails, conditional heteroskedasticity, structural change, and whether the training sample represents the forecast period. A prediction interval is not a lower bound on strategy returns.

16-E07 Step-by-step solution. Read $\mu = 0.005$ directly from the centered equation; its unit is a monthly-return decimal. The innovation variance is $0.06^2 = 0.0036$, so:

$$\gamma(0) = \mathrm{Var}(X_t) = \frac{0.0036}{1 - 0.25} = 0.0048, \quad sd(X_t) = \sqrt{0.0048} \approx 0.06928.$$

The monthly-return standard deviation is about 6.928%. Next, $\gamma(1) = \phi\gamma(0) = 0.5(0.0048) = 0.0024$, and $\mathrm{ACF}(1) = 0.0024/0.0048 = 0.5$. Applying the recursion once more gives $\gamma(2) = 0.5(0.0024) = 0.0012$ and $\mathrm{ACF}(2) = 0.25$. Variance and autocovariance are in squared monthly-return units; correlation is dimensionless.

16-E08 Step-by-step solution. The current deviation from the mean is $0.04 - 0.005 = 0.035$. Multiplying by 0.5 gives a next-period forecast deviation of 0.0175; adding back the mean gives 0.0225 (2.25%). The two-step forecast decays once more:

$$0.005 + 0.5^2(0.035) = 0.01375,$$

That is 1.375%. The one-step error variance contains one future innovation, (0.0036). The two-step error is $\varepsilon_{T+2} + 0.5\varepsilon_{T+1}$; the terms are uncorrelated, so its variance is $0.0036(1 + 0.25) = 0.0045$. The two-step forecast is less certain than the one-step forecast.

16-E09 Step-by-step solution. The MA(1) variance is the sum of the variances of two uncorrelated innovations: $\gamma(0) = 0.06^2(1 + 0.5^2) = 0.0045$. The only shared term with the previous period is $0.5\varepsilon_{t-1}$ and ε_{t-1}, so $\gamma(1) = 0.5(0.0036) = 0.0018$ and $\rho(1) = 0.0018/0.0045 = 0.4$. At lag 2 and beyond no innovations are shared, so the autocovariance and ACF are zero.

Under the chapter's convention for (1+0.5L), the root is $L = -2$, whose modulus is 2, greater than 1; the model is therefore invertible. The equivalent first-order condition is $|\theta| < 1$. Do not confuse the lag-polynomial root condition with the equation's solution $L = -1/\theta$.

16-E10 Step-by-step solution. $\Delta X_t = \varepsilon_t - \varepsilon_{t-1}$. The variance is the sum of two uncorrelated variances, $2\sigma^2$. The current difference and previous difference $\Delta X_{t-1} = \varepsilon_{t-1} - \varepsilon_{t-2}$ share only one term, with opposite signs. Thus the lag-1 autocovariance is $-\sigma^2$ and ACF(1) is $-1/2$; at all higher lags no innovations are shared and the autocovariance is zero. This is an example of differencing white noise into an MA(1) boundary process.

16-P01 Reference implementation. Run the following script by itself. It creates a consecutive monthly PeriodIndex and calculates ACF/PACF; the validator rejects an index with a missing month.

```
import pandas as pd

from quant_course.time_series import (
    sample_acf_pacf,
    simulate_stationary_pair,
    validate_monthly_series,
)

pair = simulate_stationary_pair(n=240, seed=20261016, phi=0.7)
summary = sample_acf_pacf(pair["ar1"], lags=12)
print("n:", len(pair), "lag-1 ACF:", float(summary.loc[1, "acf"]))
assert summary.index[0] == 1 and summary.index[-1] == 12
missing_month = pd.Series(
    [0.01, 0.03], index=pd.PeriodIndex(["2000-01", "2000-03"], freq="M")
)
try:
    validate_monthly_series(missing_month)
except ValueError as error:
    print("gap rejected:", error)
else:
    raise AssertionError("a missing calendar month was accepted")
```

16-P02 Reference implementation. Only the first 180 months are passed to `forecast_arima`. The program reads the realized first test-month value only for retrospective scoring and asserts that the forecast target is exactly that month.

```
from quant_course.time_series import (
    forecast_arima,
    simulate_stationary_pair,
    split_monthly_series,
    training_mean_forecast,
)

returns = simulate_stationary_pair(n=240, seed=20261016, phi=0.7)["ar1"]
train, test = split_monthly_series(returns, train_size=180)
prediction = forecast_arima(train, steps=1, order=(1, 0, 0))
baseline = training_mean_forecast(train, steps=1)
print({
    "origin": str(train.index[-1]),
    "target": str(prediction.index[0]),
    "arima": float(prediction.iloc[0]["forecast"]),
    "training_mean": float(baseline.iloc[0]),
    "realized_for_scoring": float(test.iloc[0]),
})
assert prediction.index[0] == test.index[0]
assert train.index[-1] + 1 == test.index[0]
```

16-P03 Reference implementation. `model_df=1` subtracts one degree of freedom for the estimated AR(1) coefficient; `lag=12` jointly checks the first 12 months. H_0 is that population residual autocorrelations at those lags are all zero. A large p-value does not prove IID residuals, normality, or a correct model.

```
from quant_course.time_series import (
    fit_arima,
    ljung_box_diagnostic,
    simulate_stationary_pair,
)

series = simulate_stationary_pair(n=240, seed=20261016, phi=0.7)["ar1"]
training = series.iloc[:180]
fitted = fit_arima(training, order=(1, 0, 0))
model_df = 1
lag_months = 12
test = ljung_box_diagnostic(fitted.residuals, model_df=model_df, lag=lag_months)
print(test.to_string())
print("reference chi-square degrees of freedom:", lag_months - model_df)
print("H0: the first 12 residual autocorrelations are jointly zero; non-rejection is
    ↪ not proof of IID.")
```

16-C01 Reference argument, values, and scoring. Set the marginal target standard deviation to $s = 0.04$, the mean to $\mu = 0.005$, and $\phi = 0.7$. Draw the IID control from $\mathcal{N}(\mu, s^2)$. Set the AR innovation standard deviation to $s\sqrt{1-\phi^2} = 0.0285657$, so the AR unconditional variance is also s^2, while its theoretical ACF is 0.7^k. In the chapter run with seed 20261016 and 480 months, the IID sample mean is 0.002567, sample standard deviation 0.041477, and ACF(1)= 0.046208; the AR sample mean is 0.003149, sample standard deviation 0.042033, and ACF(1)= 0.702692. Differences between sample and population ACF reflect sampling variation; sample means and standard deviations also do not exactly equal their generating parameters.

```
import numpy as np

from quant_course.time_series import sample_acf_pacf, simulate_stationary_pair

pair = simulate_stationary_pair(n=480, seed=20261016, mean=0.005, marginal_sd=0.04,
    ↪ phi=0.7)
for name in ("iid", "ar1"):
    acf_values = sample_acf_pacf(pair[name], lags=12)
    print(name, "mean", f"{pair[name].mean():.6f}", "sd", f"{pair[name].std(ddof=1)
    ↪ :.6f}", "rho1", f"{acf_values.loc[1, 'acf']:.6f}")
assert pair.index.freqstr == "M"
```

Score 0–2 points for each item: state $s_\varepsilon = s\sqrt{1-\phi^2}$ and the population moments (2); provide seed, sample size, and monthly frequency so the work can be reproduced (2); compare sample and theoretical ACF/mean/variance (2); label plot axes with monthly returns and lag months (2); explain why constructed data do not establish real-market predictability, causality, or net returns (2). A plot without the theoretical variance match earns at most 4 points.

16-C02 Reference research design, acceptable conclusion, and scoring. The current data can demonstrate converting month-label strings to a monthly PeriodIndex, validating consecutive months, and calculating return mean, standard deviation, and ACF. It does not provide the version of each observation available in each historical month, announcement or ingestion `available_at`, or the historical time when a signal first became available. A defensible conclusion is therefore: "On the current French release snapshot saved on 2026-10-05, the descriptive statistics for NoDur monthly returns are …; these data are insufficient to assess real-time historical forecasting ability." Do not treat the download date as the monthly information time back to 1926, and do not treat industry-portfolio returns as executable single-stock prices.

To extend this to a forecasting study, obtain and retain the version and timestamp visible for each observation at the time. Define the information cutoff for X_t and the holding interval for Y_{t+1}. At each origin, fit ARIMA using an expanding or rolling historical window and compare it with a zero-return or training-mean benchmark for exactly the same months. Prespecify order selection, missing-data treatment, MAE/RMSE, residual diagnostics, and one final holdout. A trading study must also delay the signal until it is actually available, record executable order prices, and deduct costs. If results are highly sensitive to the time split, sample start, or order, or fail to beat the benchmark, reject the claim of a "robust forecasting improvement." Without these data and experiments, report no result numbers.

Score 0–2 points for each item: correctly identify the data version and industry-portfolio units (2); distinguish month labels, release times, signal_time, trade_time, and the target-return interval (2); use time-ordered rolling forecasts trained only on the past (2); define a naive benchmark and out-of-sample metric in advance (2); state the lack of historical point-in-time versions and give disconfirming conditions (2). Calling the current CSV point-in-time data or treating ACF as alpha earns zero for both timing and evidence.

Answers to the Additional Exercises

16-N01. The first three variances are 0.0036, $0.0036(1 + 0.25) = 0.0045$, and $0.0036(1 + 0.25+0.0625) = 0.004725$. They change over time, so the process with a finite starting value is not weakly stationary. A stationary second-order initialization requires initial variance 0.0048 and independence from future innovations. Strict stationarity also requires matching the full initial distribution. The constant process $X_t = Z$ is strictly stationary, but its path mean remains the draw Z, which need not equal $\mathbb{E}Z$; ergodicity must be checked separately.

16-N02. The next value is $2(0.5)^2 - 1 = -0.5$, fully determined by the past. White noise restricts linear autocovariance to zero; this nonlinear squared-value relation still allows prediction. An MA model combines current and lagged innovations, while a rolling price average smooths observed prices with equal weights; a price rolling average cannot replace the innovations in an MA model.

16-N03. The level forecasts are 103 and 105; the second-period error variance is $4+4+2(1) = 10$. With February and April only, first reject the compressed "one-month lag" and investigate the missing month or a clearly appropriate missing-data model. A two-month forecast in theory uses ϕ^2 and $\sigma^2(1+\phi^2)$. For ARIMA$(p, 1, q)$ fit to the original level, the output is already restored to levels; cumulating it again would integrate twice.

19. Further Reading

- **Must Know:** statsmodels `ARIMA` and ARIMA `get_forecast`. Focus on p, d, q, trend, missing-data handling, stationarity/invertibility constraints, and the future prediction-interval API. Software documentation explains how a model is computed; it does not prove that your series meets its assumptions.
- **Must Know:** statsmodels `acf`, `pacf`, and Ljung–Box. Read about missing-data handling, denominator adjustments, PACF algorithm options, and degrees of freedom via `model_df`.
- **Should Know:** statsmodels `ArmaProcess`. Review lag-polynomial coefficient signs, AR/MA roots, and the stationarity and invertibility Boolean properties; this helps prevent mixing sign conventions for MA models across textbooks.
- **Should Know:** statsmodels ARMA: synthetic-data example. Read the sections on ACF/PACF, estimation, and residual checks for a synthetic ARMA series; the example does not replace point-in-time financial data.
- **Advanced:** The time-series analysis frameworks of Box and Jenkins and Hamilton's time-series textbook. Use them to study likelihood estimation, root conditions, forecast errors, and diagnostics in more depth. This chapter does not claim to have checked the specific editions or page numbers of these copyrighted books.

20. References

The definitions and AR(1)/MA(1) moment formulas in this chapter are derived independently; statsmodels documentation is used to verify code interfaces and parameter meanings. For full verification dates, page ranges read, runtime dependencies, and conclusions unsupported by the data, see `source_notes.md`.

- Official statsmodels documentation for `ARIMA`, `ARIMAResults.get_forecast`, `acf`, `pacf`, `acorr_ljungbox`, and `ArmaProcess` supports the implementation and API descriptions in this chapter; it does not support claims that the model is valid or that markets are predictable.
- Kenneth R. French Data Library 10 Industry Portfolios data description supports the source and field background for the downloaded portfolio file. The numbers in this chapter were cross-checked against the cache and hash manifest. The public download page is not treated as a general license for redistribution beyond the course directory.

Chapter 17 — Time Series II: Unit Roots, Cointegration, and Prediction

Through line: Two series can move closely together without sharing a stable long-run relationship. Testing, forecasting, and trading are three different questions.

Prerequisite: Chapter 16. After this chapter, you should be able to derive and interpret the augmented Dickey–Fuller (ADF) regression; distinguish deterministic from stochastic trends; estimate and recursively forecast a bivariate vector autoregression (VAR); interpret Granger predictive-causality tests correctly; assess cointegration using the Engle–Granger two-step test and its dedicated critical values; and design a pairs-trading study that does not leak future information.

1. Learning Objectives

After completing this chapter, readers should be able to use month-indexed series to answer four distinct questions: Does a series contain a unit root? Do two I(1) series have a stationary linear combination? Do lags of one variable improve forecasts of another? Can a statistical relationship be turned into a trading rule with explicit timestamps and measurable risk and costs? These questions are related in sequence, but none can replace another.

The calculations include deriving ADF's $\gamma = \phi - 1$ from $y_t = \phi y_{t-1} + \varepsilon_t$; hand-calculating a two-step bivariate VAR forecast from a small numerical example; calculating the first-stage Engle–Granger regression from 6 observations; and applying the same strict monthly interfaces to simulated data and descriptive real data. Success means understanding each statistic's null hypothesis, sample boundary, units, assumptions, and next use—not merely obtaining a small p-value.

2. Why Quants Need This

Chapter 16 asks whether a series' mean, variance, and autocorrelation can be described by a stable model. This chapter extends the question to financial series with similar long-run movements. A level regression between two steadily rising indexes can produce very high correlation and R^2 simply because of their trends. Calling a differenced VAR's predictive relationship "causality" turns incremental predictive information, conditional on a given information set, into an unsupported claim about intervention effects. Either mistake can lead researchers to mistake common risk for arbitrage.

The logic is sequential: first describe whether trends arise from deterministic terms or random shocks, then test for a unit root; if several variables are individually nonstationary, ask whether a stationary combination exists; use a suitable VAR and predictive test for stationary or differenced variables; only then turn the frozen model into a research design with delayed execution and measured risk and costs for each leg. A unit-root test does not establish cointegration; a cointegration test does not establish predictive causality; and a predictive relationship does not establish positive net trading returns.

3. Intuition

Imagine two industry total-return indexes that can each be pushed upward by new shocks every month. A shared market shock can make them rise and fall together over the long run. But if each also accumulates its own independent random error, their difference will keep drifting. A high correlation shows strong linear comovement within the sample; it does not show that the difference returns to a long-run level.

Another process can be written as $Y_t = 0.25 + 1.4X_t + S_t$, where X_t is a random walk and S_t is a stationary mean-reverting process. The levels of X_t and Y_t both drift, but $Y_t - 0.25 - 1.4X_t = S_t$ does not accumulate a stochastic trend. This combination of individually nonstationary series with a stationary combination is cointegration. The example explains the concept; it does not imply that two stocks can be traded at that ratio. Estimation error, structural stability, tradable prices, and cost evidence would still be needed.

3.1 High Correlation in the Same Window Can Arise from Two Mechanisms

Let W_t, U_t, V_t be mutually independent, zero-mean random walks accumulated from zero, with innovation variances 9, 0.25, and 0.25, respectively. The noncointegrated pair is $X_t = W_t + U_t, Y_t = W_t + V_t$. At the same $t > 0$,

$$\mathrm{Corr}(X_t, Y_t) = \frac{9t}{\sqrt{(9.25t)(9.25t)}} = \frac{9}{9.25} \approx 0.9730, \qquad \mathrm{Var}(Y_t - X_t) = 0.5t.$$

Even when the population correlation at each time is high, the difference continues to accumulate independent shocks. More generally, the variance of $Y_t - \beta X_t = (1-\beta)W_t + V_t - \beta U_t$ is $t[9(1-\beta)^2 + 0.25 + 0.25\beta^2]$, so no choice of β can eliminate all of the stochastic trends.

For the cointegrated pair, keep the same X_t and define $Y_t^* = 2X_t + S_t$, where S_t is a stationary AR(1) independent of X, with variance 0.04. Then $Y_t^* - 2X_t = S_t$ has constant variance 0.04, while the levels can still drift together. The table below shows one possible realization over the same six time points. It is a reading exercise; six points do not prove cointegration:

t	X_t	Cumulative difference $V_t - U_t$	$Y_t = X_t + (V_t - U_t)$	Stationary deviation S_t	$Y_t^* = 2X_t + S_t$
0	0	0	0	0.0	0.0
1	1	0.1	1.1	0.2	2.2
2	3	0.2	3.2	−0.1	5.9
3	2	0.3	2.3	0.1	4.1
4	4	0.4	4.4	−0.2	7.8
5	6	0.5	6.5	0.0	12.0

The sample correlation in this window exceeds 0.99 for both pairs. The conceptual distinction is whether the generating mechanism continues to accumulate errors, not which of these six-point correlations is larger. The longer samples in Figure 17-1 show how finite-sample tests distinguish the two constructions.

4. Mathematical Foundations

Throughout this chapter, t denotes an integer month; equally spaced seconds or trading days are not substitutes for calendar months. y_t may be a return (a dimensionless decimal), a return-index level (index points), or a log index (log points); these units must not be mixed within a formula. The differences $\Delta y_t = y_t - y_{t-1}$ and y_t have the same units.

For a k-dimensional vector $z_t = (z_{1t}, \ldots, z_{kt})'$, the covariance matrix is $k \times k$, and each lag-coefficient matrix in a VAR is also $k \times k$. Ordinary least squares (OLS) projects the dependent variable onto the space spanned by a constant, trend, or regressors. The residuals' orthogonality to the sample regressors is an estimation property; it does not mean the residuals are independent in the population.

White noise is not automatically independent and identically distributed. Weak stationarity requires a constant mean and variance, with covariance depending only on the lag. A unit-root process is generally not weakly stationary: accumulated shocks make its variance change with sample time. Critical values also depend on the regression's deterministic terms, number of variables, sample, and null hypothesis. This chapter does not apply ordinary Student-t or univariate ADF critical values to cointegration residuals.

5. Formal Definitions

Unit root: A series has a unit root if its autoregressive lag polynomial $1 - \phi_1 z - \cdots - \phi_p z^p$ has a root on the unit circle $|z| = 1$. Roots at different points on the circle represent different dynamics. The ordinary ADF test in this chapter primarily tests for the nonseasonal $+1$ root. For example, $y_t = y_{t-1} + \varepsilon_t$ is a random walk, in which each shock remains permanently in future levels; with uncorrelated innovations of variance σ^2, a fixed initial value gives $\mathrm{Var}(y_t) = t\sigma^2$. A -1 root produces persistent alternating signs, while other unit-circle roots can correspond to seasonal oscillations and require tests for the relevant frequencies. The ordinary ADF null $\phi = 1$ does not test every unit-circle root, so all such roots must not be interpreted as the same kind of stochastic trend. Failure to reject the $+1$ root with an ordinary ADF test means only that the sample does not provide enough evidence to reject the null; it does not prove that the root exists.

Deterministic trend: $y_t = \alpha + \beta t + u_t$. Weak stationarity of u_t constrains its mean, variance, and autocovariance, but does not ensure that the full distribution is time-invariant or, on its own, that an individual shock's response disappears. If u_t is additionally assumed to be a stable causal short-memory process, such as a causal ARMA process with absolutely summable MA coefficients, then the response of the deviation to a single shock decays. Do not attribute this stronger conclusion to weak stationarity alone. If $\beta \neq 0$, then the mean of y_t changes over time and its level is not weakly stationary. An ADF auxiliary regression can include no deterministic term (`n`), a constant (`c`), a constant and linear trend (`ct`), or a constant, linear, and quadratic trend (`ctt`). These options express modeling assumptions; they are not knobs for repeatedly searching for a small p-value.

Augmented Dickey–Fuller test (ADF): A univariate test for a unit root. The null is that the level contains a unit root; the alternative is rejection of the unit-root null conditional on the specified deterministic terms. Several lagged differences are added to absorb serial correlation in the differenced regression residual. The null distribution of the statistic is nonstandard, so it must be interpreted using the Dickey–Fuller/MacKinnon reference distribution for the same deterministic specification.

Vector autoregression (VAR): Each of k variables is predicted linearly from its own past values and the past values of the other $k-1$ variables. A stationary VAR(p) has the form $z_t = c + A_1 z_{t-1} + \cdots + A_p z_{t-p} + u_t$. Here, $z_t, c, u_t \in \mathbb{R}^k$, $A_j \in \mathbb{R}^{k\times k}$, and the lag is measured in months. This form supplies a system of predictive equations, but estimation and inference for an ordinary stable VAR require the series to meet the relevant stability conditions.

Granger predictive-causality test: For a specified sample, lag order, and information set, this test asks whether past values of x add linear information for predicting y. A common null is that all lag coefficients of x in the y equation are zero. This is not structural causality: omitted variables, sampling frequency, common shocks, and variable definitions can all affect the result.

Order of integration and cointegration: In this chapter, $I(0)$ is defined as a weakly stationary, short-memory series (with absolutely summable autocovariances). For an integer $d \geq 1$, if applying d ordinary first differences yields $I(0)$, while fewer than d differences do not yield $I(0)$, the original series is $I(d)$. This is the short-memory convention used here; weak stationarity alone does not imply short memory. If every component of a vector z_t is $I(d)$ and there is a nonzero vector b such that $b'z_t$ is reduced to $I(d-r)$ (for a positive integer r with $d-r \geq 0$), the series are cointegrated. A common case is two $I(1)$ series with a single $I(0)$ linear combination. That combination is called the cointegrating relation or long-run error. It follows the univariate order-of-integration checks and provides a long-run term for a later error-correction model (ECM).

Engle–Granger two-step test: For two candidate $I(1)$ series, the first step estimates $y_t = \alpha + \beta x_t + u_t$ by regression and obtains the residuals; the second tests whether those residuals have a unit root. Because the residuals come from an estimated long-run relationship, the second-stage statistic does not have the ordinary univariate ADF null distribution. Use cointegration-specific critical values or calibrated p-values. In statsmodels, `coint` implements this test.

Error-correction model (ECM): A representation that includes both short-run difference dynamics and the previous period's long-run cointegration deviation. If variables are cointegrated, it describes how subsequent changes adjust to the deviation; differencing alone loses this long-run information. This chapter discusses the form only in 17-C02 and does not fit an ECM.

Pairs trading: A family of long-short strategies that combines two asset series whose spread may be stable. Statistical cointegration is only the starting point for research. A strategy still needs an in-sample relationship estimate, frozen signal rules, separate signal and execution times, returns and risk measured for each leg, costs deducted, and out-of-sample testing.

6. Derivations

6.1 From AR(1) to the ADF Coefficient

Start with the AR(1) from Chapter 16: $y_t = \phi y_{t-1} + \varepsilon_t$. Subtract y_{t-1} from both sides to obtain

$$\Delta y_t = (\phi - 1)y_{t-1} + \varepsilon_t = \gamma y_{t-1} + \varepsilon_t, \qquad \gamma = \phi - 1.$$

This step turns "does the level have a unit root?" into "does the level provide mean-reverting information about next period's change?" The unit-root null $\phi = 1$ is equivalent to $\gamma = 0$. For a stationary AR(1), $|\phi| < 1$ implies $\gamma < 0$, as in the simple AR(1) model. After adding a constant or trend, the auxiliary regression is

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^{p} \psi_i \Delta y_{t-i} + \varepsilon_t.$$

In the ADF specification, lagged differences are included to reduce serial correlation in ε_t and make the regression's short-run error structure tractable. They do not change the "unit-root null" into a test of "ordinary significance of a coefficient." Reject the unit-root null at the stated significance level only if the ADF statistic for $\hat{\gamma}$ is more negative than the critical value for the same deterministic specification.

Why not use an ordinary t table? Under $H_0 : \gamma = 0$, the regressor y_{t-1} is an accumulated random shock; after scaling with sample size, it converges to a Brownian-motion path rather than a stationary variable with fixed variance. The residual sum of squares and the slope denominator therefore have a nonstandard limiting distribution. Dickey and Fuller derived the relevant distribution; MacKinnon used response-surface approximations for finite-sample critical values and p-values. A large ADF p-value means, strictly, that the current specification and sample do not reject H_0. It does not mean "nonstationarity has been proved."

6.2 Trend-Stationary and Difference-Stationary Series

If $y_t = \alpha + \beta t + u_t$ and u_t is weakly stationary, removing the deterministic mean leaves the first two moments unchanged over calendar time. This alone does not guarantee a constant distribution or imply that shock responses disappear. If u_t is also assumed to be a stable causal short-memory process, its impulse response can decay to zero. For example, in $u_t = 0.5u_{t-1} + \eta_t$, a unit innovation has a response of 0.5^h at horizon h. By contrast, $y_t = y_{t-1} + \mu + \varepsilon_t$ contains a $+1$ unit root, so the current shock remains permanently in future levels through summation:

$$y_t = y_0 + t\mu + \sum_{j=1}^{t} \varepsilon_j, \qquad \mathrm{Var}(y_t) = t\sigma_\varepsilon^2$$

(assuming independent, identically distributed shocks with finite variance). Differencing turns the $+1$ stochastic trend into $\Delta y_t = \mu + \varepsilon_t$, but loses the level relationship. Over-differencing

a trend deviation that is already stationary adds noise and moving-average structure. First ask which data-generating mechanism is plausible; then choose regression terms and differencing instead of mechanically differencing once. If the suspected root is -1 or a seasonal unit root, an ordinary first difference may not be the appropriate transformation; the derivation for a $+1$ root here does not replace seasonal-unit-root diagnostics.

How a New Shock Enters Future Levels

In the random walk $y_t = y_{t-1} + \mu + \varepsilon_t$, substituting each prior period adds another innovation. For example, let $y_0 = 100, \mu = 0$ and let the three innovations be $2, -1, 3$; the levels are then 102, $101, 104$. If only the first innovation is increased by 1, all three levels increase by 1, becoming $103, 102, 105$; the difference is larger by 1 only in the first period. With a fixed initial value, the first three variances are $\sigma^2, 2\sigma^2, 3\sigma^2$, because each period's level inherits one more independent shock. This is the calculable meaning of "accumulation."

For the trend-stationary example $y_t = 100 + 0.4t + u_t, u_t = 0.5u_{t-1} + \eta_t$, if the first innovation increases by 1, deviations in periods one, two, and three increase by $1, 0.5, 0.25$, respectively; the deterministic trend remains the same line. Detrending and differencing target different objects, so two upward-sloping lines alone do not determine which treatment to choose.

6.3 Why Residual ADF Uses Different Critical Values

Suppose $x_t, y_t \sim I(1)$. In the first stage, OLS chooses $\hat{\alpha}, \hat{\beta}$ to minimize $\sum_t (y_t - \alpha - \beta x_t)^2$, yielding $\hat{u}_t = y_t - \hat{\alpha} - \hat{\beta} x_t$. The second stage examines $\Delta\hat{u}_t = \rho\hat{u}_{t-1} + \sum_i \psi_i \Delta\hat{u}_{t-i} + v_t$. For a series or combination specified in advance, with coefficients not estimated from this test sample, the ordinary ADF reference distribution under the unit-root null may be used when the relevant deterministic-term and error assumptions hold. But $\hat{u}_t$ has been deliberately chosen by the first stage to fit y as well as possible.

Under the no-cointegration null, the long-run coefficient β is not identified by a true equilibrium relationship. The residual estimated from many level combinations has already used the sample to minimize the sum of squares. This estimation step changes the second-stage statistic's null distribution. Engle–Granger and later MacKinnon work derived or simulated critical values calibrated to the trend terms and number of regressors. In practice, use `coint(y, x)`; do not pass first-stage residuals to ordinary `adf_test` and draw a conclusion directly from its p-value.

A Pre-specified Coefficient and a Same-Sample Residual: Which Reference Table?

Object being tested	Source of coefficient	Null hypothesis and reference distribution
A pre-specified $u_t = y_t - \alpha_0 - \beta_0 x_t$	Unrelated to the test sample and treated as a fixed, known constant	Test this given series for a unit root; use the ordinary ADF distribution under its error and deterministic-term assumptions
$\hat{u}_t = y_t - \hat{\alpha} - \hat{\beta} x_t$	OLS fit on the same training sample	The candidate $I(1)$ series are not cointegrated; use the Engle–Granger-specific distribution
Residual in a new window, $y_t - \hat{\alpha}_{train} - \hat{\beta}_{train} x_t$	Estimated on an earlier training window and then frozen	A relationship-stability diagnostic; freezing does not remove training-parameter error or automatically justify ordinary ADF finite-sample calibration

The first row asks whether a given combination has a unit root; the second asks whether an estimated long-run relationship is cointegrated. The questions and sampling steps differ. For the first row, the coefficients must not be chosen after looking at this sample, and an ordinary ADF result for one combination is not equivalent to a full joint cointegration study.

Deterministic terms, lag order, and training window are all part of the test design. The intercept represents a nonzero location; a linear trend represents deterministic growth. ADF lagged differences absorb short-run dependence. Too few lags may leave serial correlation in the errors; too many consume observations and test power. Fix the candidate maximum, selection criterion, and residual checks within the training window. The Engle–Granger deterministic terms belong to the first-stage long-run equation; the second-stage residual test cannot switch to an ordinary ADF specification. In a short window, $\phi = 0.99$ still satisfies $|\phi| < 1$ but is very close to a unit root, so an ADF non-rejection may reflect low power. Report the specification, window, effective sample size, and meaning of non-rejection so the evidence is clear.

6.4 VAR and Granger Tests

For a bivariate VAR(1), let $z_t = (x_t, y_t)'$, $c = (c_x, c_y)'$, and $A_1 = (a_{ij})$. The conditional-mean forecast sets the conditional expectation of the unknown future innovation u_{T+1} to zero; recursively substituting the forecast into the same equation gives the second-period forecast. When estimating parameters from data, the lag order and coefficients must be estimated using only the training interval. If z_t are nonstationary levels, first consider differencing or a suitable model such as a VECM; do not apply the significance results of an ordinary stationary VAR without checking. If the long-run cointegration error is retained, consider a vector error-correction model (VECM).

In the y_t equation, suppose there are p lagged coefficients on x, $b_1, \ldots, b_p$. The Granger null is $H_0 : b_1 = \cdots = b_p = 0$. A joint F test asks whether these past values improve in-sample linear forecasts after controlling for lags of y and other included information. Reversing the column order changes the question: statsmodels treats the second input column as Granger-causing the first.

For example, an omitted release variable q_{t-1} may enter both x_{t-1} and y_t, making lagged x only a proxy for it. Adding q, which was observable at the time, may remove the predictive contribution of x. Intervening on x need not change q or y. Changing the sampling clock, variable order, or control information set changes the predictive question as well.

6.5 From Cointegration to a Tradable Spread

Suppose the logarithms $a_t = \log P_t^A, b_t = \log P_t^B$ of two positive indexes P_t^A, P_t^B are $I(1)$ and satisfy $s_t = a_t - \alpha - \beta b_t \sim I(0)$. Then s_t is a statistical deviation corresponding to "long A and hedge B by β." Estimate α, β in the training set, then calculate $z_t = (s_t - \bar{s}_{train})/\hat{\sigma}_{s,train}$ using the training-set mean and standard deviation. Do not re-estimate with future observations and claim the historical signal was available at the time.

Under a conceptual rule, if the month-end, observable $z_t > 2$, consider shorting the spread only at the next executable time; when $z_t < -2$, take the opposite direction; exit near zero. These thresholds are illustrative only. A log-spread coefficient does not automatically give dollar-neutral positions: specify each leg's notional or volatility risk, leverage, maximum gross exposure, rebalancing time, financing, stock-borrow terms, dividends, and trading costs. Label each signal's availability time *available_at*, generation time *signal_time*, and actual execution time *trade_time* so that $available_at \le signal_time < trade_time$.

6.6 Converting a Log Coefficient into Notional and Shares

The small change in $s_t = \log P_t^A - \alpha - \beta \log P_t^B$ is $\mathrm{d}s = \mathrm{d}P^A/P^A - \beta\,\mathrm{d}P^B/P^B$. To approximately track the spread change using dollar P&L, choose a direction $d \in \{+1, -1\}$ (+1 means long the spread) and a scale $K > 0$ in dollars, and define signed notionals as follows:

$$N_A = dK, \qquad N_B = -d\beta K, \qquad q_A = \frac{N_A}{P^A_{entry}}, \quad q_B = \frac{N_B}{P^B_{entry}}.$$

q is measured in shares; positive values are long and negative values are short. With share counts frozen, price P&L over one interval is $q_A(P^A_{exit} - P^A_{entry}) + q_B(P^B_{exit} - P^B_{entry})$, or equivalently $dKR_A - d\beta KR_B$, where R is the simple price return over the holding interval. For small returns this is approximately $dK\Delta s$; for finite changes it is not exactly equal to the change in the log spread. If dividends and financing are accounted for separately, include their cash flows. If total returns are used, do not count dividends again.

β and direction	A-leg notional	B-leg notional	Portfolio meaning
$\beta = 1.5, d = +1$	$+K$	$-1.5K$	Long A, short B; gross exposure $2.5K$, net exposure $-0.5K$
$\beta = 1.5, d = -1$	$-K$	$+1.5K$	Short A, long B
$\beta = -0.5, d = +1$	$+K$	$+0.5K$	Both legs long
$\beta = -0.5, d = -1$	$-K$	$-0.5K$	Both legs short

General cointegration does not require $\beta > 0$. Project 5's preregistered rule selects only $\beta > 0$ so that the project's spread direction corresponds to one long leg and one short leg. This is a strategy-selection rule, not part of the definition of cointegration. Even when $\beta > 0$, the notional conversion above is dollar-neutral only when $\beta = 1$. β is not a share ratio, and the intercept α is not a tradable cash leg.

Worked example: $\beta = 1.5, d = +1, K = 100$, with execution prices of 20 dollars per share for A and 50 dollars for B. The notionals are $+100, -150$ dollars, and the share counts are $+5, -3$. At exit prices of 20.6 dollars for A and 50.5 dollars for B, P&L is $5(0.6) - 3(0.5) = 1.50$ dollars. Gross exposure is 250 dollars and initial signed net notional is -50 dollars. Using 250 dollars explicitly as the gross-exposure denominator, the interval P&L ratio is 0.6%; it is not a NAV return with an undefined capital account. Deduct trading costs using actual executed notionals at entry and exit.

Fixed initial notionals do not automatically maintain the same notional each period: prices drift while the position is held, so rebalancing is needed to restore target ratios, and rebalancing incurs costs. If $K = 110$, the target share counts are $5.5, -3.3$; rounding toward zero under the convention used here gives $5, -3$. Actual notionals are $100, -150$, and the actual hedge ratio happens to remain 1.5, while gross exposure changes to 250. Rounding generally changes the ratio. For example, with $K = 100, \beta = 1.4$, target shares $5, -2.8$ round toward zero to $5, -2$, changing the actual hedge ratio from 1.4 to 1.0. Account for actual share counts and recalculate risk; do not substitute ideal target weights for executed positions.

7. Financial Interpretation

A unit root means shocks in levels may continue to affect valuation or accumulated wealth, changing the relevant risk horizon, long-run behavior, and forecast uncertainty. A trend-stationary process tends to return to its trend after a shock; each shock permanently redirects the path of a stochastic-trend process. These mechanisms call for different models and hedging horizons.

Cointegration can be understood as an empirical linear error that does not diverge over time. Its source might be cash flows, a common discount rate, policy, industry structure, or index construction. Structural change, mergers or industry reclassification, and regime changes during crises can break the relationship. A Granger predictive test says only whether historical x adds linear predictive content for y in the specified forecasting system; it does not show how an intervention on x would change y.

Pairs research turns a possible long-run constraint into a signal, but "cointegration" does not mean that the spread will revert within the intended holding period. Short-term liquidity risk, borrow recalls, financing, and trading impact can cause losses before statistical reversion. A cumulative index built from monthly industry-portfolio returns is also not two stocks that an investor can directly trade. The real-data example in this chapter is descriptive and reports no strategy returns.

8. Numerical Examples: Step by Step

8.1 Hand-Calculating the ADF Coefficient

Let $y_t = 0.7y_{t-1} + \varepsilon_t$. Subtract y_{t-1} from both sides:

$$\Delta y_t = (0.7 - 1)y_{t-1} + \varepsilon_t = -0.3y_{t-1} + \varepsilon_t.$$

Thus $\gamma = -0.3$. If $y_{t-1} = 10$, this term contributes -3 units of y to the conditional next-period change. It is not a full forecast, which would also require the constant, trend, lagged differences, and innovation. The null $\gamma = 0$ corresponds to $\phi = 1$; it is not a test of whether γ is significant under an ordinary t distribution.

8.2 Hand-Calculating the First-Stage Engle–Granger OLS

Complete example: six-point OLS and residual-test limits

Six constructed observations are $x = (1, 2, 3, 4, 5, 6)'$; define $y = 2x + u$ with $u = (1, -2, 1, 1, -2, 1)'$. Check that $\sum u_i = 0$ and $\sum(x_i - \bar{x})u_i = 0$ are both zero. Therefore, OLS with an intercept gives $\hat{\alpha} = 0$, slope $\hat{\beta} = 2$, and residuals equal point by point to u. The first step returns a long-run coefficient and residual series, not a trading signal.

The second step asks whether these residuals have a unit root. Six observations are not enough for a powerful test. More importantly, the residual coefficients were estimated from those same six points, so ordinary ADF critical values do not apply. An implementation uses Engle–Granger/MacKinnon calibration. The example illustrates the arithmetic; these six deliberately orthogonal values are not market cointegration.

```
import numpy as np
x = np.arange(1.0, 7.0)
u = np.array([1.0, -2.0, 1.0, 1.0, -2.0, 1.0])
y = 2.0 * x + u
design = np.column_stack([np.ones(len(x)), x])
alpha_hat, beta_hat = np.linalg.lstsq(design, y, rcond=None)[0]
residual = y - design @ np.array([alpha_hat, beta_hat])
print(alpha_hat, beta_hat, residual.tolist())
assert np.allclose([alpha_hat, beta_hat], [0.0, 2.0])
assert np.allclose(residual, u)
```

The outputs are, in order, an intercept of about 0, a slope of about 2, and residuals `[1, -2, 1, 1, -2, 1]`. These OLS values can be checked by hand; a formal test still requires a long, time-ordered sample.

8.3 Two-Step Forecast from a Bivariate VAR

Complete example: two-step VAR recursion

Let units be monthly return percentage points, let the final training value be $z_T = (2, -1)'$, and treat the coefficients and constant as a teaching construction:

$$c = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix}, \qquad A = \begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix}.$$

First step:

$$\widehat{z}_{T+1|T} = c + Az_T = \begin{bmatrix} 0.5 \\ -0.2 \end{bmatrix} + \begin{bmatrix} 0.4(2) + 0.2(-1) \\ 0.1(2) + 0.3(-1) \end{bmatrix} = \begin{bmatrix} 1.1 \\ -0.3 \end{bmatrix}.$$

Second step, substituting the first conditional mean (not its realized future value):

$$\widehat{z}_{T+2|T} = c + A\widehat{z}_{T+1|T} = \begin{bmatrix} 0.5 + 0.4(1.1) + 0.2(-0.3) \\ -0.2 + 0.1(1.1) + 0.3(-0.3) \end{bmatrix} = \begin{bmatrix} 0.88 \\ -0.18 \end{bmatrix}.$$

The two-step conditional forecasts are therefore 1.10 and 0.88 percentage points for the first variable, and -0.30 and -0.18 percentage points for the second. The conditional mean of a future random shock is zero, so no new shock is added in the recursion. Realized values will fluctuate around these conditional means. In code, calculate the VAR forecast from the estimated coefficients and pass the final lagged state from the training segment to forecast.

```
import numpy as np
c = np.array([0.5, -0.2])
A = np.array([[0.4, 0.2], [0.1, 0.3]])
z_t = np.array([2.0, -1.0])
z_t1 = c + A @ z_t
z_t2 = c + A @ z_t1
print(z_t1, z_t2)
assert np.allclose(z_t1, [1.1, -0.3])
assert np.allclose(z_t2, [0.88, -0.18])
```

This program performs the same matrix multiplication as the written calculation, in the same order, and asserts the two-step results. It does not insert realized future innovations into the forecast.

8.4 One Return Interval and Two Legs

Complete example: fixed-notional costs on both legs

Suppose a strategy forms a signal at month-end t, enters at the next executable time $t+1$ with 100 dollars notional long and 100 dollars short, and rebalances at $t+2$. If the long asset's total return over the holding interval from $t+1$ to $t+2$ is $r^A = 3\%$ and the short asset's total return is $r^B = 1\%$, the dollar P&L before fees is $100(0.03) - 100(0.01) = 2$. To check the one-month holding interval, charge 10 basis points on each leg at entry and again on exit, using a fixed notional of 100 dollars per leg for this example. The four trades cost 0.10 each, for total transaction costs of 0.40. The one-month stock-borrow fee of 20 basis points and financing fee of 5 basis points are also each calculated on a 100-dollar notional, costing 0.20 and 0.05, respectively. Net P&L for this example is:

$$2 - 4(0.10) - 0.20 - 0.05 = 1.35.$$

Actual ledgers should charge fees using each executed notional, the exact holding period, and the applicable rate. Fixed notional is only the convention for this hand calculation. This example checks units: monthly French data cannot decompose the interval from a month-end signal to an actual execution in the following month and cannot support estimating these costs.

9. Python Implementation

Default Entry Point for This Reader Edition

From the accepted I3 reader package root, create an independent environment using its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch17_time_series_ii.ipynb
```

The default run constructs two 900-month samples using seeds 20261017 and 20261018. In the noncointegrated simulation, the common-innovation standard deviation is 1 and the independent-innovation standard deviation is 0.2; this has different inputs from the hand-worked variances 9/0.25/0.25 in Section 3.1. The cointegrated construction uses beta=1.4, a spread AR coefficient of 0.65, innovation standard deviation 0.35, intercept 0.25, and a stationary initial spread. Ordinary ADF is applied to the known spread; the estimated same-window relationship receives EG-specific calibration. A separate two-variable VAR uses seed 1704 for 120 rows from 1990-01, trains on the first 100 months through 1998-04, and forecasts 1998-05 and 1998-06. Its generating coefficients are A=[[0.35,0.12],[0.05,0.30]], c=[0.25,−0.10], and innovation standard deviation 0.10. This Notebook does not run a Granger test.

The Notebook displays the existing bundled figure `fixtures/figures/a4/ch17_cointegration_simulation.png`; displaying it does not regenerate it. `RUN_FRENCH=False` disables the historical market branch. Although it creates and prints `runs/ch17_time_series_ii_<uuid>/`, this Notebook writes no CSV or PNG there. The runner separately saves the executed copy and status in a unique `runs/execution_*`

directory; these directories serve different purposes. Constructed calculations are not evidence of market predictability or strategy net returns. The historical French figures and numbers in Sections 10–11 retain their original status.

The CLI and production tests below belong to the historical course working-root workflow and are not this package's default route. Readers need not restore production tests that are absent from the package. The I3 Chapter 16 CLI still reads French data unconditionally; the Chapter 17 CLI's historical branch requires `--french-data`. Neither replaces the default command above.

Historical Course Working Root and Original Implementation

The chapter's core is in `cointegration.py`. It reuses Chapter 16's `validate_monthly_series` and also strictly validates multicolumn monthly tables. It requires an ascending, unique, continuous `PeriodIndex(freq="M")` and converts numeric values to finite `float64`. Missing months, NaN, infinity, booleans, and complex numbers raise errors; values are not filled automatically. The interfaces are:

Function	Input	Output and boundary
`adf_test`	One complete monthly series; `regression` deterministic terms and optional lag specification	ADF statistic, approximate p-value, lags used, sample size, and corresponding critical values
`engle_granger_test`	Two same-month aligned series with at least 24 observations; optional trend and lag specifications	Engle–Granger statistic, dedicated p-value, and critical values
Common-trend, noncointegrated simulation	Length `n` and `seed`	Shared random trend plus two independent random walks
`simulate_cointegrated_pair`	`seed`, a finite nonzero real `beta`, and $\|\phi\| < 1$	$x \sim I(1)$, $y = 0.25 + \beta x + s$, and $s \sim I(0)$; rejects `beta=0` and booleans
`fit_var_forecast`	Training panel `training`, fixed lag `lags`, and forecast horizon `steps`	Recursive forecasts indexed by future months; researchers must still verify stationarity
`granger_test`	Named, same-month aligned `caused` and `causing` series; maximum lag `max_lag`	Joint F statistic and p-value with direction stated explicitly
`screen_cointegration_pairs`	Multicolumn level table `levels` and training cutoff month `training_end`	Engle–Granger p-values and BH-adjusted p-values (often called q-values) for all pairs before the cutoff; an exploratory FDR adjustment that does not read post-cutoff values to estimate results
`total_return_index_proxy`	Monthly simple returns in decimal form `returns`	Educational total-return index (TR index) compounded from 100

Reproduce the experiments, figures, and sample statistics with:

```
.venv/bin/python code/ch17_time_series_ii.py
.venv/bin/python -m pytest tests/test_ch17_cointegration.py -q
```

The script fixes the two simulation seeds at 20261017 and 20261018; seeds are not selected based on returns or p-values. The Engle–Granger wrapper explicitly calls statsmodels 0.15.0 `coint`; it does not assemble incorrect critical values from the ordinary ADF wrapper. Perfectly or nearly collinear inputs are rejected. For `trend="n"`, if statsmodels returns NaN critical values, the interface preserves the missing values while separately checking that the test statistic and p-value are finite. All empirical forecasts and screens use only the training segment. Monthly data are indexed by month; a month is not treated as a fixed number of seconds. The Notebook calls the core module instead of duplicating the statistical implementation.

10. Visualization

Figure 17-1: Simulated comparison of a common random trend and a cointegrated pair. The two columns show the noncointegrated common-trend construction with seed 20261017 and the cointegrated construction with seed 20261018 and $\beta = 1.40$; both samples contain 900 months. The top row uses calendar month on the horizontal axis and arbitrary constructed levels on the vertical axis. Both x, y move in the same direction in each case, so the top row alone cannot determine cointegration. The bottom row shows $x - y$ on the left and the known stationary spread $y - 0.25 - 1.40x$ on the right: the left difference continues to drift, while the known stationary spread on the right fluctuates around zero. This illustrates mechanisms; it is not a market result.

Figure 17-2: Cumulative-index proxy compounded from monthly returns for 10 French industries. The horizontal axis is the calendar month of the return observation. The vertical axis is logarithmic, after compounding the 10 industries' published monthly returns as $100 \prod(1 + r_t)$ and setting the displayed base for the first month to 100. The sample covers 1926-07 through 2026-08. The industries share a long-run upward movement but have different drawdowns. This is neither a single-stock price nor an out-of-sample strategy NAV; the log scale shows long-run compounding and historical drawdowns together.

Both PNGs were generated by running `code/ch17_time_series_ii.py`. The code reads the local French CSV with its fixed hash and validates the legend, periods, and data range. They are not hand-drawn illustrations.

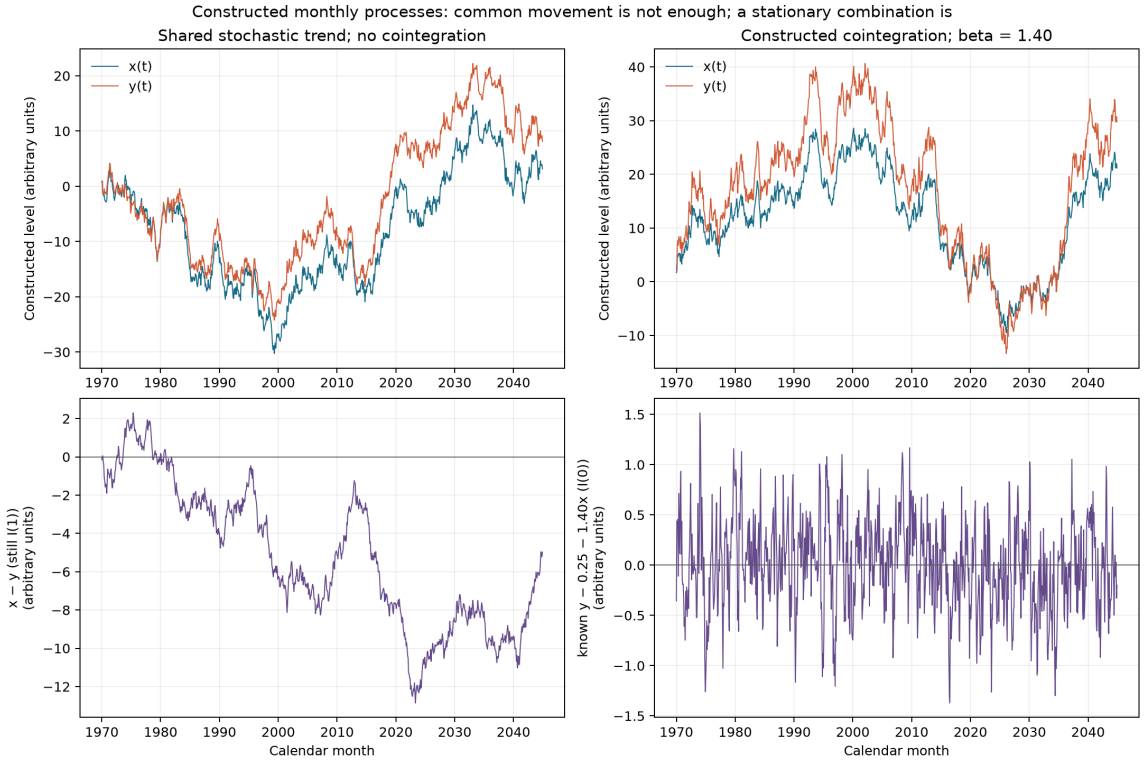


Figure 17-1: Similar level movements do not imply the same long-run statistical relationship

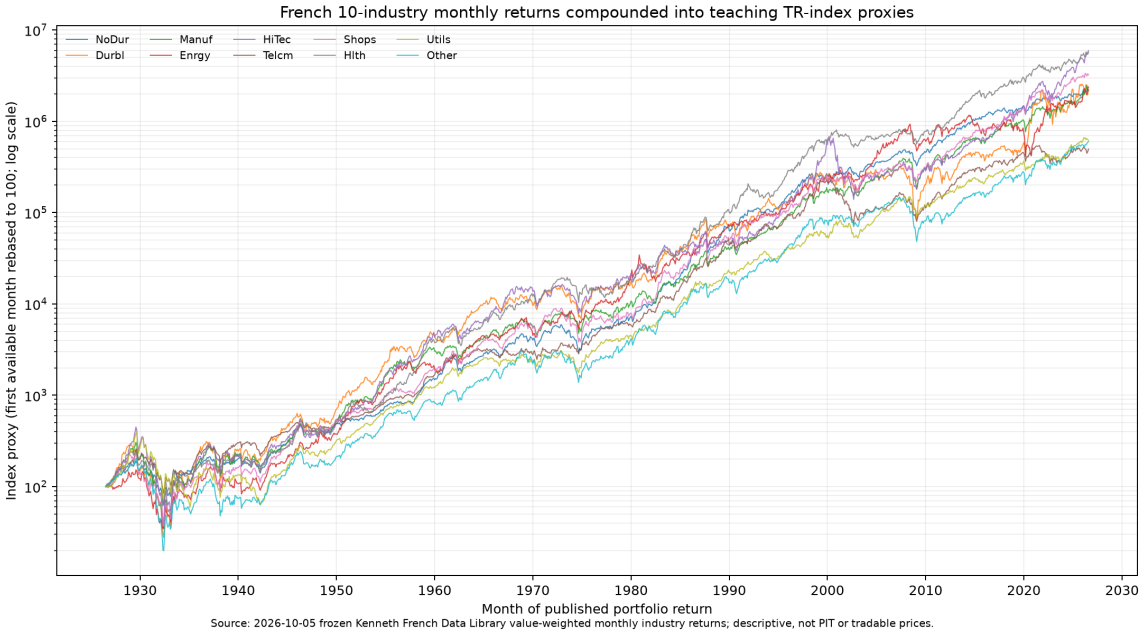


Figure 17-2: Educational TR-index proxy compounded from ten industries' published returns

11. Real Market Example

This chapter uses the current cache of the Kenneth R. French Data Library's 10 Industry Portfolios: a ZIP downloaded on 2026-10-05, using the monthly block labeled `Average Value Weighted Returns -- Monthly`, with source percentages divided by 100. The source

ZIP SHA-256 is `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`; the processed CSV SHA-256 is `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`. The cache contains 1,202 months, 1926-07 through 2026-08. See `source_notes.md` and the data manifest for source-conversion details. Publicly downloadable does not mean unrestricted redistribution is licensed.

Compound each industry's published decimal simple returns into an educational `TR Index proxy`. During the 1926-07 through 2000-12 training interval, run a constant-term Engle–Granger test on all $\binom{10}{2} = 45$ pairs; do not select trading pairs based on test-period results. The script records a minimum nominal p-value of 3.85848×10^{-9}, a median of 0.0516444, 22 of 45 nominal p-values below 0.05, and 15 pairs with Benjamini–Hochberg (BH)-adjusted q-values below 0.05. The false discovery rate (FDR) is the expected proportion of false discoveries among rejected hypotheses. The BH step-up procedure aims to control FDR; it does not control the family-wise error rate (FWER), the probability of at least one false positive. Its usual FDR-control guarantee requires independent tests or applicable conditions such as positive regression dependence on a subset (PRDS) of the true nulls. The pairs here share industry series, and neither their dependence structure nor the PRDS condition has been verified. The 15 BH-adjusted results below 0.05 are therefore exploratory adjusted statistics; they do not establish that FDR is controlled for this family. A small p-value does not validate future stability or imply profitability. The individual series' I(1) orders, stability across regimes, real-time data availability, and trading costs were not established by this step, so these numbers do not establish cointegration or support a trading conclusion.

These 10 columns are industry-portfolio returns, not 10 individual stocks. The cumulative values are index proxies constructed for this chapter's modeling, not executable adjusted prices for an actual fund and not point-in-time (PIT) data. The current release covers through 2026-08 and cannot reconstruct the version investors actually saw in each month since 1926. This chapter does not claim historical signal validity or report a strategy Sharpe ratio or return.

12. Common Mistakes

Mistake 1: The level correlation is 0.95, so the two series can be paired. A common stochastic trend can produce high correlation; test whether a linear combination is stationary. The output below is about 0.9552 and 0.457541; correlation is not a cointegration test.

```
from quant_course.cointegration import simulate_noncointegrated_common_trend,
    ↪ engle_granger_test
pair = simulate_noncointegrated_common_trend(n=900, seed=20261017)
test = engle_granger_test(pair["x"], pair["y"])
print(pair["x"].corr(pair["y"]), test.pvalue)
```

Mistake 2: Run ordinary ADF on first-stage OLS residuals and declare cointegration from the ordinary p-value. The residuals were estimated using the same I(1) data, so their null distribution differs. In the fixed simulation, ordinary ADF gives $p \approx 0.23056$, while the correct Engle–Granger test gives $p \approx 0.45754$. The values differ; the key point is that only the latter is calibrated for the no-cointegration null. Use this chapter's Engle–Granger interface or statsmodels `coint`.

```
import numpy as np
import statsmodels.api as sm
from statsmodels.tsa.stattools import adfuller
from quant_course.cointegration import simulate_noncointegrated_common_trend,
    ↪ engle_granger_test
pair = simulate_noncointegrated_common_trend(n=900, seed=20261017)
design = sm.add_constant(pair["y"].to_numpy())
residual = pair["x"].to_numpy() - design @ sm.OLS(pair["x"].to_numpy(), design).fit()
    ↪ .params
ordinary = adfuller(residual, regression="c", autolag="AIC", result_object=True)
correct = engle_granger_test(pair["x"], pair["y"])
print(ordinary.pvalue, correct.pvalue)  # approx. 0.23056 and 0.45754
```

Mistake 3: Granger significance means x structurally causes y. Reversing the column order asks a different predictive question. The result concerns only incremental predictive information for the specified sample and lags. The function makes the direction explicit and checks it against a known generating direction:

```
import numpy as np
import pandas as pd
from quant_course.cointegration import granger_test
rng = np.random.default_rng(23)
x = rng.normal(size=1000)
y = np.zeros(1000)
for t in range(1, len(y)):
    y[t] = 0.75 * x[t - 1] + rng.normal(scale=0.5)
index = pd.period_range("1950-01", periods=len(y), freq="M", name="month")
x_series = pd.Series(x, index=index, name="x")
y_series = pd.Series(y, index=index, name="y")
print(granger_test(caused=y_series, causing=x_series, max_lag=1).pvalue)
print(granger_test(caused=x_series, causing=y_series, max_lag=1).pvalue)
```

Mistake 4: Estimate the relationship on the full sample, then treat old z-scores as real-time signals. Future observations change the estimated intercept and slope. Split first and estimate using only the training period. This chapter's `screen_cointegration_pairs` accepts only the `training_end` prefix; a test changes future observations and confirms that screening results remain unchanged. Real orders also require recording data availability time.

```
import numpy as np
import pandas as pd
from quant_course.cointegration import simulate_cointegrated_pair
pair = simulate_cointegrated_pair(n=900, seed=20261018)
train = pair.loc[:pd.Period("2000-12", freq="M")]
design = np.column_stack([np.ones(len(train)), train["x"]])
alpha_hat, beta_hat = np.linalg.lstsq(design, train["y"], rcond=None)[0]
spread_train = train["y"] - alpha_hat - beta_hat * train["x"]
print(len(train), alpha_hat, beta_hat, spread_train.std(ddof=1))
```

Mistake 5: Assume adjacent rows remain one month apart after `dropna()`. Removing a missing month makes a multi-month difference look like a one-month change. The example shows the core interface rejecting this compressed index. Trace the source or specify a missing-data model; do not fill values inside the test function.

```
import numpy as np
import pandas as pd
from quant_course.cointegration import validate_monthly_frame
index = pd.period_range("2000-01", periods=4, freq="M", name="month")
series = pd.Series([0.01, np.nan, 0.02, 0.03], index=index, name="return")
try:
    validate_monthly_frame(series.dropna().to_frame())
except ValueError as error:
    print(type(error).__name__, error)
```

13. Quant Research Application

A credible pairs study should produce an auditable chain in at least the following order.

1. **State the hypothesis and falsification conditions.** For example: "Two preselected assets are each I(1) in the specified regime, their log spread is stationary across multiple training and validation windows, and signal returns cover all costs." Possible counterevidence includes different integration orders, failure to reject a unit root in the residual, relationship drift after a regime change, no out-of-sample mean reversion, or no advantage over a naive benchmark after costs.
2. **Freeze the data and clock.** Save source-file hashes, units, corporate actions, price/return definitions, and `available_at`. The industry cache in this chapter supports description of the current version of monthly portfolio returns only; it cannot give historical strategy signals PIT timestamps.
3. **Split by time.** Within the training interval, separately check both series' orders, specify deterministic terms, estimate long-run coefficients, test the specially calibrated Engle–Granger statistic, and account for multiple testing across families such as all 45 pairs. If screening pairs, pre-specify the candidate set and correction method; do not switch objects after seeing the validation period.
4. **Freeze signals and positions.** Within training, fix $\hat{\alpha}, \hat{\beta}$, spread-normalization parameters, entry/exit/stop rules, refit frequency, and each leg's dollar notional or risk budget. The cointegration regression's $\hat{\beta}$ is not a risk-neutral weight; define each leg's target volatility, total gross exposure, and single-leg limit.

5. **Align each leg's return interval.** If a signal becomes available after month t closes, do not book the full month-t return for that signal. Set the first executable time after the close, and measure both legs' next holding-period returns over the same execution interval. Monthly French published returns do not recover this execution interval, so this chapter does not calculate backtest P&L.
6. **Deduct each leg's costs and compare benchmarks.** Record commissions on both sides, spreads and impact, borrow availability and fees, financing, rebalancing turnover, halts/delistings, dividends, and taxes; compare with no-trade and naive hold/hedge benchmarks. Strategy validation also needs rolling or expanding windows, the available training prefix and next-period target at each step, uncertainty intervals, and crisis stress tests.
7. **Audit stability and failure.** Check residual ACF and structural breaks, drift in $\hat{\beta}$ within windows, industry-definition changes, spread tails during crises, and borrow recalls. For multiple pairs selected in training, adjusted p-values do not replace independent out-of-sample validation.

This is a research design, not a record that Project 5 is complete. This chapter has no timestamped data, execution prices, borrow records, or NAV series; it cannot claim that a pairs strategy passed out-of-sample and cost tests.

Project 5: Connecting Statistical Training Outputs to the Execution Interface

The current accepted entry point for Project 5 is `projects/project05/ACCEPTED_VERSION.md`, and its primary specification is preregistration v2. Its acceptance covers implementation and constructed teaching validation; real-price empirical work remains blocked. Training outputs should distinguish the long-run regression intercept and slope, Engle–Granger screening results, training-spread mean and standard deviation, and subsequent execution rules. Apply these frozen outputs at each later time point; do not renormalize using the entire future spread. If the training window changes, generate the next relationship at the pre-specified refit time rather than replacing historical fits with a future estimate.

The two chapters have distinct interface responsibilities: Chapter 16 explains the time index, effective observation count, and conditional forecast; Chapter 17 explains relationship screening and the position being held. Project 5 then calculates results using its daily trading clock, cost model, and episode definition. Passing Engle–Granger and having $\beta > 0$ are only candidate-entry conditions; a trade may still not be entered or may lose money. The primary C5 constructed result, "evidence does not support this strategy," must be retained. Do not substitute this chapter's fixed 100-dollar-notional example for the project's period-by-period share counts, actual executed notionals, and cumulative ledger. Do not upgrade the monthly French proxy into daily real trading prices.

14. Chapter Summary

First use an ADF test with the appropriate deterministic terms to ask whether the series rejects a unit root, then consider whether several I(1) variables have a stationary linear combination. Ordinary ADF and Engle–Granger tests on estimated residuals have different null distributions; the latter requires cointegration-specific critical values. A VAR links multiple variables through lags to form conditional forecasts. A Granger joint test concerns only incremental predictive power within the specified information set. Cointegration, prediction, and structural causality answer different questions; none alone establishes that trading is feasible.

For a pairs application, the key transition is from an in-sample long-run error to a real-time executable rule: estimate coefficients using past data only, trade only after the signal is available, define a common return interval for both legs, and include risk and costs in validation. Simulations help readers understand the mechanisms. The current French industry cache helps check compounding and screening counts for published returns, but does not support historical real-time trading conclusions.

15. Exercises

Attempt the exercises independently before reading the answers and derivations in Section 18.

- **17-E01 (Concept):** Two random walks have a sample correlation of 0.97. Is that enough to conclude they are cointegrated? What tests or conditions are still needed?
- **17-E02 (Unit root):** If $\hat{\gamma} = -0.08$ in the ADF regression, can you reject a unit root based on its sign alone? Explain which statistic and specification are needed.
- **17-E03 (Trend):** Distinguish the persistence of shocks in $y_t = 1 + 0.4t + u_t$ ($u_t \sim I(0)$) from that in $y_t = y_{t-1} + 0.4 + \varepsilon_t$.
- **17-E04 (Engle–Granger critical values):** Why is it wrong to use ordinary `adfuller` critical values on first-stage OLS residuals?
- **17-E05 (Causality language):** What does a significant result from `grangercausalitytests([y,x])` test? Which stronger claim does it not support?
- **17-E06 (Model selection):** What is the problem with running ADF under all three specifications `c`, `ct`, and `ctt`, then reporting only the smallest p-value?
- **17-E07 (Multiple comparisons):** If 45 pairs each receive a 5% test, why are 22 nominally significant results not 22 verified long-run relationships?
- **17-E08 (Timing):** A month-end t signal uses the full return for month t. What is wrong with calculating that signal's P&L from the same month's full return?

16. Programming Exercises

- **17-P01 (Simulation and testing):** With fixed seeds, generate 900 months of a non-cointegrated common-trend sample and a cointegrated sample. Report level correlations and the p-values from `engle_granger_test`. Acceptance criteria: the same seed reproduces the same result; the first sample has correlation above 0.9 and does not reject the no-cointegration null at 5%; the second rejects it at 1%. Label the results as simulations.
- **17-P02 (VAR time boundary):** Construct a stationary bivariate monthly series. Fit a VAR(1) using only the first 80 months and forecast two months. After replacing the final 20 months with arbitrarily large values, the model input and forecasts based on the first 80 months must remain unchanged. State the output-column order, forecast months, and units.
- **17-P03 (Multiple-pair screening):** Load the French 10-industry cache and compound 10 TRI proxies. Run Engle–Granger tests for all 45 pairs using only data through 2000-12; report p and BH q values. Add a constant to all post-cutoff observations and confirm that the training results do not change. Do not select pairs based on backtest returns.
- **17-P04 (Granger direction):** Generate white-noise x_t and $y_t = 0.75x_{t-1} + e_t$, then test $x \to y$ and $y \to x$ separately. Check the column order and interpretation; do not write a structural-causality conclusion.

17. Challenge Problems

- **17-C01 (Multiple testing and selection bias):** Explain how choosing the smallest-p-value relationship among the 45 pairs from 10 series distorts significance. Propose a reproducible validation plan for each rolling training window that does not revisit the selected pair out of sample. Additional prerequisites: multiple comparisons and time-series cross-validation. **10 points:** record all 45 pairs (2); select within training only (2); define a correction, permutation, or testing family (2); use an independent later window (2); state falsification conditions (2).
- **17-C02 (Error correction):** Starting from the cointegration error $s_t = y_t - \alpha - \beta x_t$ between two I(1) series, write an Δy_t equation with lagged differences and explain its adjustment coefficient. Describe the long-run information lost in an ordinary differenced VAR and retained in a VECM, and how structural change can invalidate the error-correction term. Additional prerequisites: linear regression and VAR. **10 points:** write lagged differences and an error term (3); explain the adjustment direction (2); distinguish long-run from short-run terms (2); give a structural-change example (2); state that this chapter does not estimate the model (1).

Additional Exercises (Independent Numbering)

- **17-N01 (Mechanism and hand calculation; §§3.1, 6.2):** The common random walk has innovation variance 9, and each independent trend has innovation variance 0.25. Find the same-t correlation between the two levels and the variance of $Y - X$; show that no β can eliminate all trends. With a fixed initial level of 100 and innovations 2, -1, and 3, how do the levels change if the first shock is increased by 1?
- **17-N02 (Test diagnosis; §6.3):** Explain which test interpretation to use for a pre-specified combination, same-sample OLS residuals, and new-window residuals using frozen training coefficients. Why does an ADF non-rejection in a short window with $\phi = 0.99$ fail to prove a unit root? State when deterministic terms and lags should be selected.
- **17-N03 (Two-leg ledger and interpretation; §6.6):** Given $K = 100, d = +1, \beta = 1.5$ and A/B execution prices of 20/50 and exit prices of 20.6/50.5, find share counts, gross exposure, net notional, and price P&L. Charge 10 basis points on each leg's actual execution notional at entry and exit, then find net P&L (no dividends, borrow, or financing). If $\beta = -0.5$, what direction does a long spread take? If $\beta = 1.4$ and shares are rounded toward zero, what is the actual hedge ratio? Explain what it means that Project 5 excludes negative beta.

18. Solutions / Solution Hints

Exercises

- **17-E01:** No. A common stochastic trend can produce high correlation without a stationary linear combination. First assess whether both series are I(1) under suitable specifications, then use a cointegration-specific procedure such as Engle–Granger to test the combination. Sample correlation provides neither critical values nor evidence that the relationship will remain stable.
- **17-E02:** No. The statistic is usually an ADF t-type statistic for $\gamma = 0$, with a nonstandard finite-sample Dickey–Fuller reference distribution. You also need the deterministic terms (`n/c/ct/ctt`), lag selection, and matching critical value. A negative slope alone is insufficient.
- **17-E03:** In the first equation, under this chapter's definition $u_t \sim I(0)$ supplies only weak-stationarity and short-memory properties; those moment conditions alone do not imply a time-invariant full distribution or a vanishing response to one shock. With the additional assumption $u_t = 0.5u_{t-1} + \eta_t$, a unit innovation changes the deviation by 0.5^h at horizon h, so its effect decays while the trend $1 + 0.4t$ is not permanently rewritten. The second equation contains a $+1$ unit root: a shock at time t remains with coefficient 1 in every later level, and variance grows with t under independent, identically distributed innovations with finite variance. In the two equations, 0.4 is respectively a deterministic slope and a per-period drift. The claim that the first equation's shocks are temporary depends on the additional stable causal assumption; it does not follow automatically from the definition of weak stationarity.

- **17-E04:** The first stage estimates the long-run coefficient from the same nonstationary levels, minimizing the residual sum of squares. Under the no-cointegration null, that coefficient is not a stable true long-run parameter; the unit-root statistic for the estimated residual has a different null distribution from a univariate ADF test on a series specified in advance. Use the dedicated critical values returned by `coint`.
- **17-E05:** statsmodels treats the second column, x, as the causing variable and the first column, y, as the dependent series. It jointly tests whether the specified lag coefficients of x in the y equation are zero. Rejection means incremental linear predictive information for the specified sample, lags, and controls; it does not show that intervening on x would cause y to change.
- **17-E06:** Repeatedly selecting the smallest p-value implicitly includes specification search as multiple testing, so the nominal p-value no longer has its original meaning. Choose deterministic terms in advance based on assumptions about the data-generating process and report sensitivity; do not report only the most significant specification.
- **17-E07:** The 45 tests can generate false positives, and the tests are dependent because they share industry series. The 22 nominally significant results describe only this exploratory family of tests. BH adjusts for FDR, and its theoretical guarantee requires independence or suitable positive dependence such as PRDS. That dependence condition has not been verified here, so 15 BH q-values below 0.05 are adjusted results, not evidence that FDR control is guaranteed; BH is not FWER control. Independent later-window testing is also needed. No adjustment guarantees relationship stability or tradability.
- **17-E08:** The signal exists only once that month's return is fully formed and available. Booking P&L over the same month's start-to-end interval uses movement that occurred before the signal existed, reversing the time order. Specify the next executable time and calculate both legs' returns over the same interval after execution. Monthly portfolio files do not provide that finer-grained data.

Programming Exercises

- **17-P01:** Run the chapter's core interfaces directly:

```
from quant_course.cointegration import (
    engle_granger_test, simulate_cointegrated_pair,
    simulate_noncointegrated_common_trend,
)
noncointegrated = simulate_noncointegrated_common_trend(n=900, seed=20261017)
cointegrated = simulate_cointegrated_pair(n=900, seed=20261018)
a = engle_granger_test(noncointegrated["x"], noncointegrated["y"])
b = engle_granger_test(cointegrated["y"], cointegrated["x"])
assert noncointegrated["x"].corr(noncointegrated["y"]) > .9
assert a.pvalue > .05 and b.pvalue < .01
print(a.pvalue, b.pvalue)  # approx. 0.457541 and 8.69312e-26
```

- **17-P02:** The following code runs independently. First generate a stationary bivariate VAR(1), passing only the first 80 months into the fit and forecast. Then replace the final 20 months in the original panel with large values and recompute using the same training prefix. Both forecasts must be identical, begin in the month after the training cutoff, and retain the same column order.

```
import numpy as np
import pandas as pd
from pandas.testing import assert_frame_equal
from quant_course.cointegration import fit_var_forecast

rng = np.random.default_rng(29)
values = np.zeros((100, 2))
for t in range(1, 100):
    values[t] = np.array([0.25, -0.10]) + np.array(
        [[0.35, 0.12], [0.05, 0.30]]
    ) @ values[t - 1] + rng.normal(0.0, 0.1, size=2)
index = pd.period_range("2000-01", periods=100, freq="M", name="month")
panel = pd.DataFrame(values, index=index, columns=["A", "B"])
training = panel.iloc[:80].copy()
forecast = fit_var_forecast(training, lags=1, steps=2)

changed = panel.copy()
changed.iloc[80:] += 500.0
forecast_again = fit_var_forecast(changed.iloc[:80], lags=1, steps=2)
assert_frame_equal(forecast, forecast_again)
assert forecast.index.tolist() == [training.index[-1] + 1, training.index[-1] + 2]
assert forecast.columns.tolist() == ["A", "B"]
print(forecast)
```

- **17-P03:** The reference implementation is located in `code/ch17_time_series_ii.py`, in `main()`: verify the frozen CSV hash; compound cached monthly returns that have already been divided by 100; set the training cutoff to 2000-12; loop over all 45 unordered pairs; calculate BH-adjusted p-values; and print every pair's unadjusted and BH-adjusted values. The core `screen_cointegration_pairs` function uses only the `training_end` prefix. A test adds 10,000 to all post-cutoff data and confirms that every result field

remains the same. The reproduced counts are 22/45 nominally significant and 15/45 with BH-adjusted p < 0.05. FDR-control conditions for this family of tests on shared series have not been verified; the results do not imply 15 tradable relationships.

- **17-P04:** The following code runs independently. The process makes x_t white noise and sets $y_t = 0.75x_{t-1} + e_t$. It tests each predictive direction on month-aligned data and asserts strong significance in the known direction, but not the reverse. Rejecting the null means incremental predictive information under the given model and sample; it is not a structural-causality conclusion.

```
import numpy as np
import pandas as pd
from quant_course.cointegration import granger_test

rng = np.random.default_rng(23)
n = 1000
x = rng.normal(size=n)
y = np.zeros(n)
for t in range(1, n):
    y[t] = 0.75 * x[t - 1] + rng.normal(scale=0.5)
index = pd.period_range("1950-01", periods=n, freq="M", name="month")
x_series = pd.Series(x, index=index, name="x")
y_series = pd.Series(y, index=index, name="y")

x_predicts_y = granger_test(caused=y_series, causing=x_series, max_lag=1)
y_predicts_x = granger_test(caused=x_series, causing=y_series, max_lag=1)
assert x_predicts_y.pvalue < 1e-8
assert y_predicts_x.pvalue > 0.05
print("x -> y p-value:", x_predicts_y.pvalue)
print("y -> x p-value:", y_predicts_x.pvalue)
```

Challenge Problems

- **17-C01, suggested argument:** For each outer training window, first freeze the 10 series names, the set of 45 pairs, deterministic terms, ADF lag rule, and multiple-testing method (for example, apply BH to p-values in the training window and also report the full unadjusted distribution). Inner rolling time-series windows may compare candidates, but each outer test window evaluates only the single model selected before that window began. Do not reselect pairs or recalculate training coefficients in the outer period. When retraining for the next period, explicitly record the old and new objects. Retain failed relationships and report drift in relationship sign and half-life, out-of-sample spread ACF, each leg's post-cost returns, and confidence intervals. If a relationship exists only in one regime, one very short window, or before costs, the hypothesis is falsified. Award points according to the 5 rubric items.
- **17-C02, suggested form:**

$$\Delta y_t = \lambda(y_{t-1} - \alpha - \beta x_{t-1}) + \sum_{i=1}^{p-1} \Gamma_{yy,i} \Delta y_{t-i} + \sum_{i=1}^{p-1} \Gamma_{yx,i} \Delta x_{t-i} + \varepsilon_t.$$

The parenthesized term is the previous period's long-run error. If y is above the long-run combination and $\lambda < 0$, the adjustment term lowers next-period Δy; stationarity, sign, and parameter stability still require empirical confirmation. A differenced VAR retains short-run difference dynamics but loses the level error; a VECM puts the cointegration error back into the model. If a regime change means the old α, β no longer describes equilibrium, the adjustment term can point in the wrong direction. The prompt asks for the model form and research boundary; the chapter's core code does not estimate a VECM. Award points according to the rubric.

Answers to the Additional Exercises

17-N01. The level correlation is $9/9.25 \approx 0.9730$, and the variance of the difference is $0.5t$. The general error variance is $t[9(1-\beta)^2 + 0.25 + 0.25\beta^2]$; the 0.25 term keeps it strictly positive for every β, so it grows with t. The original levels are 102, 101, and 104. Increasing the first shock changes them to 103, 102, and 105; each later level is higher by 1, while only the first difference is higher by 1.

17-N02. For a pre-specified fixed combination, the univariate ADF reference distribution may be used under its assumptions; same-window OLS residuals require the Engle–Granger no-cointegration distribution. A new-window residual with frozen training coefficients is a stability diagnostic; training error remains, so ordinary ADF calibration is not automatically exact. With $0.99 < 1$, the stable AR process is close to a unit root, and a small-sample test has limited power; non-rejection means only that the current evidence is insufficient. Specify deterministic terms, the maximum lag or selection rule, and the training boundary in advance; do not search future windows and report only the smallest p-value.

17-N03. $q_A = 5, q_B = -3$; gross exposure is 250 dollars and net notional is -50 dollars. Price P&L is $5(20.6 - 20) - 3(50.5 - 50) = 1.50$. Entry executions total $100 + 150 = 250$ dollars; exit executions total $103 + 151.5 = 254.5$ dollars. Total executed notional is 504.5 dollars, so the fee is $504.5(0.001) = 0.5045$ and net P&L is 0.9955 dollars. With $\beta = -0.5$, $N_A = 100, N_B = 50$, so both legs are long. Project 5's $\beta > 0$ is a strategy-screening restriction; it does not deny that general cointegrating relationships can have a negative coefficient. With $\beta = 1.4$, target shares $5, -2.8$ round toward zero to $5, -2$, giving an actual hedge ratio of $100/100 = 1$. Recalculate risk using the actual position.

19. Further Reading

- **Must Know:** Dickey & Fuller (1979), pp. 427–431: focus on §§1–2 to understand the estimator distribution under a unit-root null and the deterministic-term specification. The chapter's source notes record the pages actually read.
- **Must Know:** Engle & Granger (1987), pp. 251–255 and 260–269: read the definitions of cointegration and error correction, the two-step estimator, and the explanation that residual tests cannot use ordinary DF critical values.
- **Must Know:** MacKinnon (2010), §§1–3 and the 2010 Appendix: read how cointegration-test critical values are calibrated for regressor count, trend terms, and sample size.

- **Must Know:** Benjamini & Hochberg (1995), pp. 289–300: focus on the FDR definition and BH step-up procedure under independent tests. Benjamini & Yekutieli (2001), pp. 1165–1188, discuss the extension under positive regression dependence on a subset (PRDS); shared industry pairs must not be assumed to meet its conditions without evidence.
- **Should Know:** Granger (1969), §§4–5: read the predictive meaning of "causality" and the bivariate model, noting the author's limitations concerning omitted variables, sample frequency, and nonstationarity.
- **Should Know:** The official statsmodels 0.15.0 `adfuller`, `coint`, VAR, and `grangercausalitytests` documentation: check the code API and output direction. Software documentation describes implementation; it does not replace the original theory.
- **Advanced:** Said & Dickey (1984), *Biometrika* 71(3), 599–607, on asymptotic unit-root testing when the ARMA order is unknown. For this chapter, only the OUP bibliographic record and abstract were checked; the full text was not read. See the source notes.

20. References

- Dickey, D. A., & Fuller, W. A. (1979). "Distribution of the Estimators for Autoregressive Time Series with a Unit Root." *Journal of the American Statistical Association*, 74(366), 427–431. DOI. Supports the nonstandard distribution under the unit-root null in §6.1.
- Engle, R. F., & Granger, C. W. J. (1987). "Co-Integration and Error Correction: Representation, Estimation, and Testing." *Econometrica*, 55(2), 251–276. Official article page and abstract. Supports the two-step estimation and cointegration-residual testing in §§5 and 6.3.
- Granger, C. W. J. (1969). "Investigating Causal Relations by Econometric Models and Cross-Spectral Methods." *Econometrica*, 37(3), 424–438. DOI. Supports the predictive-causality definition and limitations in §§5 and 6.4.
- MacKinnon, J. G. (2010). "Critical Values for Cointegration Tests." Queen's Economics Department Working Paper No. 1227. PDF. Supports the cointegration critical-value calibration in §6.3.
- Benjamini, Y., & Hochberg, Y. (1995). "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." *Journal of the Royal Statistical Society: Series B*, 57(1), 289–300. DOI. Introduces and discusses FDR; the BH procedure controls FDR under the independent-test assumption and is not a family-wise error-rate procedure.
- Benjamini, Y., & Yekutieli, D. (2001). "The Control of the False Discovery Rate in Multiple Testing under Dependency." *The Annals of Statistics*, 29(4), 1165–1188. DOI. Discusses BH FDR control under positive regression dependence (PRDS); the fact that tests are correlated does not establish that its conditions hold.
- Official statsmodels 0.15.0 documentation: ADF, Engle–Granger `coint`, VAR, and Granger test. Supports code behavior only.
- Kenneth R. French Data Library. 10 Industry Portfolios. Supports the source and monthly data-block location for industry returns in §11; cache revision, hash, and use limits are documented in `source_notes.md`.

Chapter 18 — Volatility Modeling and Forecasting

Roadmap: First distinguish variation already realized along a path from conditional variance at a point in time. Then use recursions that read only past information to turn risk estimates into next-period forecasts. Prerequisites: probability (Chapter 6), statistics (Chapter 7), and time series (Chapter 16); Chapter 17 provides background, and this chapter does not use unit-root, cointegration, or VAR results. After completing this chapter, you should be able to check units and timestamps; calculate realized variance, EWMA, ARCH/GARCH, and multi-step forecasts by hand; distinguish persistence from moment conditions; explain high-frequency sampling error; and read a historical crisis-window forecast comparison with appropriate caution.

1. Chapter Objectives

By the end of this chapter, you should be able to:

1. Explain that realized volatility is a post hoc measure of a path, whereas conditional volatility is a forecast object given an information set; the two are not equal merely because both are called "volatility."
2. Understand volatility clustering from the autocorrelation of squared returns, and explain how returns themselves can be nearly uncorrelated while squared returns are clearly correlated.
3. Compute rolling variance, exponentially weighted moving average (EWMA), autoregressive conditional heteroskedasticity (ARCH), and generalized ARCH (GARCH) without leaking information across the clock.
4. Derive the finite unconditional variance meaning of $\alpha+\beta$, the GARCH multi-step expected-variance recursion, and the fourth-moment condition under a Gaussian assumption; do not conflate these conditions as a single "stationarity" measure.
5. Explain where the Gaussian conditional quasi-maximum likelihood (QML) objective comes from, the limits of the in-class fit, and why convergence does not imply a global optimum.
6. Recheck research conclusions from figures, frozen data, numerical tables, and output files; do not elevate in-sample fit or proxy-loss rankings into statistical significance, trading advantage, or a guarantee about the future.

2. Why Quant Needs This

Return forecasting and risk forecasting are different tasks. Even if tomorrow's mean return is difficult to predict, today's market state may still help predict the conditional dispersion of tomorrow's return. Volatility clustering is an intuitive motivation for this distinction: large moves often occur in clusters during some periods, while calm periods also tend to persist.

If risk is fixed at the full-sample standard deviation, extreme and quiet periods are averaged into one number. If this month's squared return is used to estimate "this month's conditional variance," the target month's post hoc outcome leaks into the forecast. Rolling, EWMA, ARCH,

and GARCH make the information set explicit, provide variance estimates that update over time, and separate the target month from the history available before it.

These methods do not answer "what should I buy?" Volatility is only one risk input. A claim that a position-sizing rule predicts returns or has tradable value would also require records of actual availability, signal-to-order delay, position rules, execution, turnover, financing, borrow, spreads, market impact, capacity, and net returns. This chapter has no trading ledger, so post-cost strategy returns are not applicable.

3. Intuition

Suppose the log returns in three subintervals of one interval are $+1\%$, -2%, and $+3\%$, respectively. Log returns add, so the net log return is $+2\%$, and its square is 0.0004; the sum of the three squared increments is $0.01^2 + (-0.02)^2 + 0.03^2 = 0.0014$. Moves in opposite directions do not cancel in the sum of squares. The first quantity answers "how far did the interval move net?"; the second answers "how much squared variation occurred within the interval?" Simple returns must be compounded first; their sum is not an exact net return.

Conditional variance is like a risk state that updates each period. A large shock yesterday raises the next period's risk estimate; if the estimate is highly persistent, risk falls back only gradually. EWMA mixes the latest squared shock with the old variance using weights; GARCH also puts the long-run risk level explicitly into the model.

A counterexample helps put this intuition in bounds: observing one very large monthly return today means the squared return is large, but does not prove that tomorrow's conditional variance must be large. A model of the dynamics is also needed, and the estimated relationship must be reproducible in the target setting. Clustering is a testable pattern of time dependence, not a law that every large decline must be followed by another.

Object	Input and calculation	Result for the three values	Availability and use

From an Interval Path to a Forecast Clock

The same set of numbers has meaning only together with its interval and information set. Fix the object first, then choose the calculation:

Object	Input and calculation	Result for the three values	Availability and use
Interval RV	Square each of the three log returns in the same interval and add	0.0014	After the interval ends; describes pathwise squared variation
Squared net log return	Add first, then square	0.0004	After the interval ends; describes the squared net change
Three-point sample variance	Mean is 0.0066667; divide the sum of squared deviations by 2	0.0006333333	After all three subintervals; describes the sample dispersion of subinterval returns
Monthly innovation squares	Reinterpret the three values as innovations for three months	0.0001, 0.0004, 0.0009	Arrive at each month-end; evaluate this month or update next month
Monthly conditional variance h_t	Past information and model recursion	See the three-period forecast in §8	Before the target month starts; forecasts that month's conditional risk

The interval for RV may be one day or one month; the subinterval coverage must be stated. The high-frequency definition in this chapter does not relabel the three rows of a monthly CSV as intraday RV. Although sample variance also has squared-return units, its target is subinterval dispersion; the objects cannot be substituted just because their units match.

Monthly clock	Known information	Permitted action
Before month t begins	$\mathcal{F}_{t-1}$, h_t, and a mean rule fixed in advance	Save the forecast for target month t
After month t ends	r_t, and ϵ_t computed using the original mean rule	Compute the squared-innovation proxy and evaluate the saved h_t
Update for next month	h_t, ϵ_t^2	Recurse to h_{t+1}; do not revise the forecast for this month

To convert units, first multiply decimal returns by 100, then multiply variance by 10,000, and finally take the square root to get volatility in percent. This order ensures $\sqrt{10,000h} = 100\sqrt{h}$, rather than multiplying variance by 100.

4. Mathematical Foundations

Let r_t be the simple return for calendar month t, measured as a dimensionless decimal at monthly frequency, so 0.02=2%. In the teaching model with mean fixed at μ, $\epsilon_t = r_t - \mu$ is the return innovation, still measured in decimal monthly returns. Let $\mathcal{F}_{t-1}$ denote the information available before the target month begins. The conditional variance is

$$h_t = \mathrm{Var}(r_t \mid \mathcal{F}_{t-1}) = \mathrm{Var}(\epsilon_t \mid \mathcal{F}_{t-1})$$

and its units are "decimal monthly return squared." The conditional standard deviation $\sigma_t = \sqrt{h_t}$ returns to decimal returns (for one month). If returns are multiplied by 100 to express them as percentages, variance is multiplied by 100^2=10,000. Calculations, losses, and chart axes must use consistent units.

Under the classical setup, standardized innovations $z_t = \epsilon_t/\sqrt{h_t}$ satisfy $E(z_t \mid \mathcal{F}_{t-1}) = 0$ and $E(z_t^2 \mid \mathcal{F}_{t-1}) = 1$. Conditional normality is an additional distributional assumption; it does not follow from those two conditions. The QML fit below uses a conditional Gaussian objective but fixes the mean at the training-sample mean; it is not joint maximum likelihood estimation of the mean and variance parameters.

Realized variance is often calculated as the sum of squared intraday log returns $x_{t,i}$ over an interval, with units of squared log return for that interval. It is a post hoc measure of a path that has already occurred; h_t is a forecast, formed before the month begins, of the conditional variation over the entire month. Without intraday prices, a genuine intraday realized variance cannot be reconstructed from a monthly-return CSV.

5. Formal Definitions

5.1 Realized Risk: Realized Variance and Volatility

Within a clearly specified interval ([t-1,t]), divide the interval into M adjacent subintervals and let $x_{t,i}$ be the log return in subinterval i. Define realized variance (RV) and realized volatility as

$$RV_t = \sum_{i=1}^{M} x_{t,i}^2, \qquad RVOL_t = \sqrt{RV_t}.$$

RV is not demeaned, divided by (M-1), or automatically annualized. It is neither sample variance nor the square of the net return over the full interval. The high-frequency limit relationship between RV and quadratic variation (QV) depends on assumptions about the price process, sampling grid, jumps, and market microstructure. RV on a finite grid will not equal the latent continuous-time integrated variance observation by observation.

5.2 Conditional Heteroskedasticity and Clustering

Conditional heteroskedasticity means that h_t changes with F_{t-1}. Clustering usually appears as weak autocorrelation in r_t but stronger autocorrelation in r_t^2 or centered innovation squares ϵ_t^2. Sample autocorrelation in squared returns is a diagnostic signal, not a causal theorem that "volatility will certainly be high in the future."

5.3 Rolling Variance, EWMA, ARCH, and GARCH

Given window W, a rolling forecast labeled for target month t can be written as

$$\widehat{h}_{t|t-1}^{(W)} = \frac{1}{W-1}\sum_{j=1}^{W}(r_{t-j} - \bar{r}_{t-W:t-1})^2.$$

The denominator (W-1) is the rolling sample-variance convention, and the target-month return r_t is excluded. This is a historical local-dispersion benchmark; it is not guaranteed to equal conditional variance.

The first-order EWMA recursion is

$$h_{t+1} = \lambda h_t + (1-\lambda)\epsilon_t^2, \quad 0 < \lambda < 1.$$

Equivalently, EWMA is ARCH(∞) with exponentially decaying weights: the weight on an older shock falls as $(1-\lambda)\lambda^j$, but the caller must supply the initial h_1 for the first recursive step. It corresponds to the GARCH(1,1) parameter boundary $\alpha = 1-\lambda, \beta = \lambda$, so $\alpha + \beta = 1$, and it has no positive constant term that separately anchors long-run variance. The standard GARCH long-run variance formula therefore does not describe its mean reversion.

Initialization is also part of the forecast rule. Expanding the finite recursion gives

$$h_{t+1} = \lambda^t h_1 + (1-\lambda)\sum_{j=1}^{t}\lambda^{t-j}\epsilon_j^2.$$

Thus, the influence of h_1 persists as λ^t, so both the initial value and λ must be fixed before the forecast origin. If two runs differ only in changing the initial value from 0.0004 to 0.0009, then with $\lambda = 0.9$ their next-period forecasts differ by $0.9(0.0005) = 0.00045$, and after two updates they differ by 0.000405. Initializing with the full test-period sample variance would let early forecasts read the future; a training-period or prespecified value makes the input boundary explicit.

ARCHq uses a finite number of lagged innovation squares:

$$h_t = \omega + \sum_{i=1}^{q}\alpha_i\epsilon_{t-i}^2.$$

GARCH(p,q) also includes lagged conditional variance. This chapter uses p for the number of lagged conditional-variance terms and q for the number of lagged innovation-square terms

(software may use a different parameter order, so check the definition):

$$h_t = \omega + \sum_{i=1}^{q} \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j h_{t-j}.$$

The focus in this chapter is GARCH(1,1):

$$h_t = \omega + \alpha \epsilon_{t-1}^2 + \beta h_{t-1}, \quad \omega > 0,\ \alpha, \beta \geq 0.$$

The units of conditional variance are determined by ω, the innovation squares, and h; each has units of decimal monthly return squared. α, β are dimensionless. ARCH(1) is the special case with $\beta = 0$. Positive coefficients make the contributions of squared shocks and the historical risk state to current variance nonnegative.

5.4 Persistence, Moments, and Strict Stationarity

Let $\rho = \alpha + \beta$. If standardized shocks have variance 1, the process is stationary in the finite-second-moment sense, and $\rho < 1$, then unconditional variance is

$$\bar{h} = E(h_t) = \frac{\omega}{1 - \alpha - \beta}.$$

This concerns only a finite second moment. It differs from strict stationarity, under which the joint distribution of the entire process is unchanged by time shifts. For scalar GARCH(1,1), under the usual stochastic-recursion assumptions, the log-contraction condition for a strictly stationary solution is written as $E \log(\alpha z_t^2 + \beta) < 0$. When $E(\alpha z^2 + \beta) = \alpha + \beta < 1$, Jensen's inequality gives a log expectation less than 0, so the finite-variance condition is sufficient to imply a solution of this kind; the converse does not hold. For example, when $\alpha = 0.1, \beta = 0.9$, $EA = 1$, but if z^2 is nondegenerate and the log expectation exists, strict Jensen's inequality can give $E \log A < 0$, while the unconditional second moment does not exist. The in-class core deliberately restricts attention to the finite-second-moment range, so it does not estimate this boundary case.

A finite second moment still does not guarantee a finite fourth moment. If $z_t \sim N(0, 1)$, the finite-fourth-moment condition for standard GARCH(1,1) is

$$E[(\alpha z_t^2 + \beta)^2] = 3\alpha^2 + 2\alpha\beta + \beta^2 < 1.$$

Therefore, "$\rho < 1$" cannot replace a fourth-moment check; asymptotic standard errors for squared returns or conclusions based on higher moments may require stronger conditions.

5.5 QML, Forecasts, and QLIKE

Gaussian conditional QML estimates parameters using a density objective formed each period from the conditional mean and variance. The in-class implementation fixes the training-sample mean first, then optimizes the scaled average objective for the variance recursion. It omits a constant; when the return scale changes, only ω is converted back to the original units. Three starting points and numerical constraints improve usability, but do not prove a global optimum or provide parameter standard errors.

Define the QLIKE (quasi-likelihood loss) for a positive variance forecast $\widehat{h}_t$ and nonnegative variance proxy q_t as

$$L(q_t, \widehat{h}_t) = \log \widehat{h}_t + \frac{q_t}{\widehat{h}_t}.$$

The common "robust ranking under an unbiased proxy" property requires conditions such as $E(q_t \mid F_{t-1}) = h_t$, as well as regularity conditions. If the squared-return proxy has a mean error, conditional bias, or timing misalignment, that guarantee cannot be applied directly. QLIKE can be negative because $\log h$ is often negative when the unit is below 1; for the same target period and units, a lower score is better, but the score is not utility, significance, or net return.

6. Derivations

6.1 Why a Fine-Grid Sum of Squares Is Related to QV

For a discrete log price X without measurement noise, partition $[0, T]$ into a grid $0 = t_0 < \cdots < t_M = T$, and let the increments be $\Delta_i X = X_{t_i} - X_{t_{i-1}}$. Define the quadratic sum over the partition as $Q_M = \sum_i (\Delta_i X)^2$. In the fine-grid limit for a continuous semimartingale, it converges in probability to the integrated variance of the continuous component; if jumps occur, the sum of squared jump magnitudes must also be added. The key assumptions are no measurement noise, a refining grid, and the required process conditions. This is not pointwise equality in a finite sample, nor is it the conditional variance given past information, h_t.

6.2 How Microstructure Noise Can Make "Higher Frequency" Worse

Write the observed log price as $Y_i = X_i + u_i$, where X_i is the efficient price and u_i is independent, mean-zero measurement noise with variance τ^2. Then

$$\Delta_i Y = \Delta_i X + (u_i - u_{i-1}).$$

If the noise is independent and identically distributed and independent of efficient-price increments, the cross-term has expectation zero after squaring and summing, while the noise-increment variance is

$$E[(u_i - u_{i-1})^2] = E(u_i^2) + E(u_{i-1}^2) - 2E(u_i u_{i-1}) = 2\tau^2.$$

Therefore, $E[\sum_{i=1}^{M}(\Delta_i Y)^2]$ is approximately the efficient-price quadratic sum plus $2M\tau^2$. For a fixed interval, as the sampling count M increases, the noise term accumulated in simple RV may also increase. Real-world noise may be serially correlated or related to changes in the efficient price; this simplified derivation cannot identify one universally best frequency for every market. Frequency selection requires empirical checks for the market, asset, sampling design, and estimation target.

6.3 Step-by-Step EWMA and GARCH Updates

EWMA updates next month's risk only after ϵ_t is observed at the end of month t: take the old forecast h_t, multiply it by λ, take the new information ϵ_t^2 from this month, multiply it by $1-\lambda$, and add them to obtain h_{t+1}. The weights sum to 1. When the model's standardized shocks have conditional variance 1, the conditional expectation of next-period variance equals current variance; there is no separate long-run anchor. If the realized shock is exactly zero, the actual update is $h_{t+1} = \lambda h_t$, so it falls; do not misread an unchanged conditional expectation as an unchanged value on every path.

GARCH also starts from the previous h_t and puts the new shock into ϵ_t^2:

$$h_{t+1} = \omega + \alpha\epsilon_t^2 + \beta h_t.$$

Take expectations and, under a finite second moment and standardized shocks with variance 1, $E(\epsilon_t^2) = E(h_t) = \bar{h}$. Hence

$$\bar{h} = \omega + (\alpha+\beta)\bar{h} \quad \Longrightarrow \quad \bar{h} = \frac{\omega}{1-\alpha-\beta},$$

which requires $\alpha + \beta < 1$. α measures the contribution of the new shock square, while β measures the contribution of the old variance state; two models with the same ρ can have different shock responses, so persistence alone does not imply identical model behavior.

6.4 Multi-Step Forecasts Are Not Cumulative Variance Across Periods

At time T, once $h_{T+1|T}$ is known, the variance forecast for the next target period is known. Later shocks have not yet occurred, so use $E_T(\epsilon_{T+1}^2) = h_{T+1|T}$:

$$h_{T+2|T} = \omega + (\alpha+\beta)h_{T+1|T}.$$

Substitute the unconditional level ($\bar{h}$) for $\omega = (1-\rho)\bar{h}$ to obtain

$$h_{T+k|T} = \bar{h} + \rho^{k-1}(h_{T+1|T} - \bar{h}), \quad k \geq 1.$$

Each value for k is the expected conditional variance for that particular future month. If the objective is cumulative log-return variance over several months, then, under the additional conditions that cross-period innovation conditional covariances are zero given the forecast-origin information and the mean is known (as in an appropriate martingale-difference sequence), add the

expected variance for each month; the last-period forecast alone is not the cumulative quantity. The core function `garch_variance_forecast` returns the former quantity.

6.5 From a Density to the Gaussian QML Objective

The conditional normality assumption gives $\epsilon_t \mid F_{t-1} \sim N(0, h_t)$, so the log density is

$$\ell_t = -\tfrac{1}{2}\{\log(2\pi) + \log h_t + \epsilon_t^2/h_t\}.$$

Maximizing $\sum_t \ell_t$ is equivalent to minimizing $1/2\sum_t[\log h_t + \epsilon_t^2/h_t]$, because $log(2\pi)$ does not depend on the parameters. The core first scales residuals by the training standard deviation s, giving $\tilde{\epsilon} = \epsilon/s$ and variance $\tilde{h} = h/s^2$, then optimizes the average objective

$$\frac{1}{2}\,\text{mean}(\log \tilde{h}_t + \tilde{\epsilon}_t^2/\tilde{h}_t),$$

and multiplies the fitted $\tilde{\omega}$ by s^2 to return to the original variance units; α, β are unchanged. Omitting the constant and taking the average makes optimization easier, but the result is not a full original-scale log likelihood that can be directly compared across different samples or scales.

When the mean and variance dynamics are correctly specified, the process satisfies the required stationarity and ergodicity, identification, integrability, and other conditions, conditional normal QML may yield consistent variance parameters. Under nonnormality, inference for variance parameters also requires a matching robust covariance estimator or stronger distributional conditions. The in-class function has no robust standard errors; it does not prove automatic validity for all nonnormal errors.

6.6 Check Proxy Eligibility Before Comparing Forecasts

Let the true conditional mean be μ_t, and let μ_0 be a training mean fixed in advance. By the definition of conditional variance,

$$E[(r_t - \mu_0)^2 \mid \mathcal{F}_{t-1}] = h_t + (\mu_t - \mu_0)^2.$$

The cross-term is zero because $E(r_t - \mu_t \mid \mathcal{F}_{t-1}) = 0$. This shows that noise in a squared-innovation proxy and mean error are distinct issues: even with a large sample, a wrongly chosen center adds conditional bias. Intramonth high-frequency RV uses more path information, but still requires checks for sampling noise, jumps, interval coverage, and fit to the forecast target; it cannot be assumed conditionally unbiased simply because it is "high frequency."

Check	Squared return / fixed-mean squared innovation	High-frequency RV
Information coverage	One net move per month	Multiple path increments in the target interval
Available ex post?	After month-end	After sampling over the target interval is complete
Proxy eligibility	Key conditions are conditional mean, centering, and conditional unbiasedness	Key conditions are sampling noise, interval coverage, and the conditional target
Fair loss comparison	Same target months, same effective sample, same squared-return units	Same, and check whether it matches the monthly forecast

If returns are uniformly multiplied by c, then $q' = c^2 q$, $\widehat{h}' = c^2 \widehat{h}$, and QLIKE becomes the original loss plus $\log(c^2)$. A uniform unit conversion shifts all model scores by the same amount and preserves rankings on the same sample; changing only forecasts or only the proxy breaks comparability. Negative scores come from the scale of this loss and the omitted-constant convention; they do not mean negative risk.

7. Financial Interpretation

Conditional volatility describes the risk scale for "how far the next-period return may lie from its conditional mean, given the information at this point in time"; it does not describe return direction. If h_t rises, a strategy's position size might be adjusted under a risk-budget rule, but that is a separate portfolio assumption. If a historical estimate is delayed, revised, subject to structural changes across market regimes, or costly to trade on, the model's numerical forecast is not equivalent to realized risk-control performance.

A high ρ means the forecast variance falls back toward $(\bar{h})$ more slowly after a shock. In the current NoDur training sample, the estimate is $\rho \approx 0.9953$, but the training sample has only 204 months, the function has boundary constraints, and the current snapshot is not data that was genuinely available at the time. Do not interpret this estimate as a natural constant, a permanently stable parameter, or reliable forecasting ability.

EWMA is easy to interpret, fast, and parsimonious, but its $\alpha + \beta = 1$ boundary means it does not revert to a long-run variance. A rolling benchmark is easy to reproduce, but its reaction speed depends on the window length. GARCH explicitly separates the new shock from the old variance, but depends on model specification and numerical estimation. Comparing them is a research diagnostic, not a leaderboard that automatically becomes a trading decision.

8. Numerical Example

Complete Example: Path Sum of Squares, Recursion, and Month-by-Month Risk

All data below are hand-calculable teaching constructs, not market observations. The RV row interprets the three values as subinterval log returns over the same interval; the remaining rows interpret them as innovations for three months to calculate different quantities by hand. Given three innovations $[0.01, -0.02, 0.03]$ (decimal monthly returns) and initial variance $h_1 = 0.0004$:

Method / period	Substitution	Variance quantity (RV or h)	Square-root quantity (RVOL or conditional standard deviation)
RV	$0.01^2 + (-0.02)^2 + 0.03^2$	0.001400	0.037417
EWMA 1	Given h_1	0.000400	0.020000
EWMA 2	$0.9(0.0004) + 0.1(0.01^2)$	0.000370	0.019235
EWMA 3	$0.9(0.00037) + 0.1((-0.02)^2)$	0.000373	0.019313
GARCH 1	Initial h_1	0.000400	0.020000
GARCH 2	$0.00001 + 0.1(0.01^2) + 0.8(0.0004)$	0.000340	0.018439
GARCH 3	$0.00001 + 0.1((-0.02)^2) + 0.8(0.00034)$	0.000322	0.017944

If GARCH is changed to ARCH(1) (set $\beta = 0$), the second and third variances are respectively $0.00001 + 0.1(0.01^2) = 0.00002$ and $0.00001 + 0.1(0.02^2) = 0.00005$: past variance no longer propagates directly. For $\omega = 0.00001, \alpha = 0.1, \beta = 0.8$, the long-run level is $\bar{h} = 0.00001/(1 - 0.9) = 0.0001$. If next-period variance is (0.0004) and $\rho = 0.9$, the expected variances for the next three months are $[0.0004, 0.00037, 0.000343]$, not the total variance for three months. If the cross-period innovation conditional covariance is zero given the forecast origin and the mean is known, the sum of the first three periods is the variance forecast over that interval.

A final percent-unit check: $h = 0.0004$ corresponds to $\sigma = 0.02 = 2\%$, and variance in percentage units is 4 (% monthly return)^2. Therefore, multiply decimal variance by 10,000 to plot it as "percentage return squared." Do not misread 0.0004 as 0.04% volatility.

9. Python Implementation

Default Standalone Entry Point

From the root directory of the bundled reader package, create an independent environment following its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch18_volatility.ipynb
```

First use the three log returns from the same interval to check RV, then reinterpret the same values as innovations over three months and check EWMA, ARCH, and GARCH. The multi-step forecast uses a separate, manually constructed `GARCHFit`: next-month variance 0.0004, training endpoint 2000-03, and 60 observations are toy metadata, not the result of fitting three observations and not a continuation of the third-month variance 0.000322 just calculated. This standalone example returns month-by-month variances of 0.0004, 0.00037, and 0.000343; 60 does not represent the sample size of a fit performed in this run.

It then constructs 360 months starting in 1980-01 using seed 20261018, mean 0.005, omega=0.00001, alpha=0.1, beta=0.8, and a 500-period burn-in; a finite burn-in is not the same as drawing an exact stationary initial state. The first 240 months are used for fitting and the final 120 for evaluation. The training mean is fixed first, then GARCH is fitted on the training segment; QLIKE is reported separately for the known true variance and squared-innovation proxy over the same evaluation period. Shape checks in the loss function do not replace callers' responsibility to align observations by target month.

By default, the Notebook writes one file: `constructed_simulation.csv`. Its directory is `runs/ch18_volatility_<uuid>/`: all 360 rows retain the true variance and `sample_role`, the forecast column has values only in the last 120 rows, and the training prefix retains NaN. It does not write a historical forecast CSV, a results JSON, or four PNGs; the historical figure set is explicitly skipped. The four figures and NoDur numbers in Sections 10–11 are retained as frozen chapter reference material and are not products of this run.

`RUN_FRENCH=False`; French data is not read or downloaded by default. Importing the chapter workflow does not mean that its historical market analysis has been executed. The chapter run directory differs from the `runs/execution_<uuid>/` directory created separately by the runner; the latter holds an execution copy and state. In-package assertions let readers check constructed calculations; production tests not included in the package do not need to be restored. Constructed results do not demonstrate market forecasting or a post-cost trading advantage.

The CLI below follows the historical course-work root workflow and requires suitable, authorized data inputs and an environment; it does not replace the default command above.

Shared Implementation and Historical Work-Root Workflow

The workflow calls only the shared implementation in `src/quant_course/volatility.py`; the chapter script handles frozen-data reading, sample splitting, outputs, and plots. It does not write a separate estimator. The script `code/ch18_volatility.py` locates the project root from its file path and can be launched from any current working directory:

```
.venv/bin/python /absolute/path/to/quant_course/code/ch18_volatility.py
```

To select custom output directories:

```
.venv/bin/python /absolute/path/to/quant_course/code/ch18_volatility.py \
  --output-dir /tmp/ch18-results --figure-dir /tmp/ch18-figures
```

The script first checks the SHA-256 hashes in the manifest for the raw ZIP, extracted CSV, and processed CSV; if any file is missing or any byte has changed, it stops. It does not download data again or overwrite raw data. It reads decimal monthly returns, splits the sample by full monthly calendar, then outputs fitted parameters, monthly variance forecasts, proxy losses, and four figures. The simulated sample uses fixed seed=20261018; the function retains true conditional variance for each period for auditability only.

The sequence is: `train=1990-01…2006-12`; the training mean is computed from the training segment only; GARCH is fitted only on the training segment; each target month's recursion uses return shocks observed before it; and the test squared-innovation proxy can be computed only after the target month ends. This information structure is a post hoc attribution of "what would have happened if historical parameters had been fixed and previous historical shocks had been observed." The download time, version-availability time, publication lag, and historical local-ingestion time for the French snapshot have not been reconstructed, so this is not a point-in-time real-time backtest.

The in-sample fitted path for the training period is an ex post state, not a real-time month-by-month training-period forecast. The core fixes the mean at the training-sample mean, and its 3 initialization points do not guarantee a global optimum; numerical bounds such as the normalized lower bound for ω and the upper bound on persistence are safeguards for teaching optimization, not economic laws. The default Notebook above reuses the same workflow and shared core but selects the constructed-data branch.

10. Reading the Figures

The four PNGs below were generated by the historical chapter workflow and are retained as frozen chapter readings; the default offline Notebook does not regenerate them:

1. Simulated volatility clustering: the x-axis is simulated month (seed=20261018); the upper panel shows monthly returns in percent and the lower panel shows the true conditional standard deviation in percent/month. Larger return magnitudes cluster during high-volatility periods. The simulation's true DGP variance is known, so it is not market evidence; finite burn-in also does not guarantee a strictly stationary initial draw.

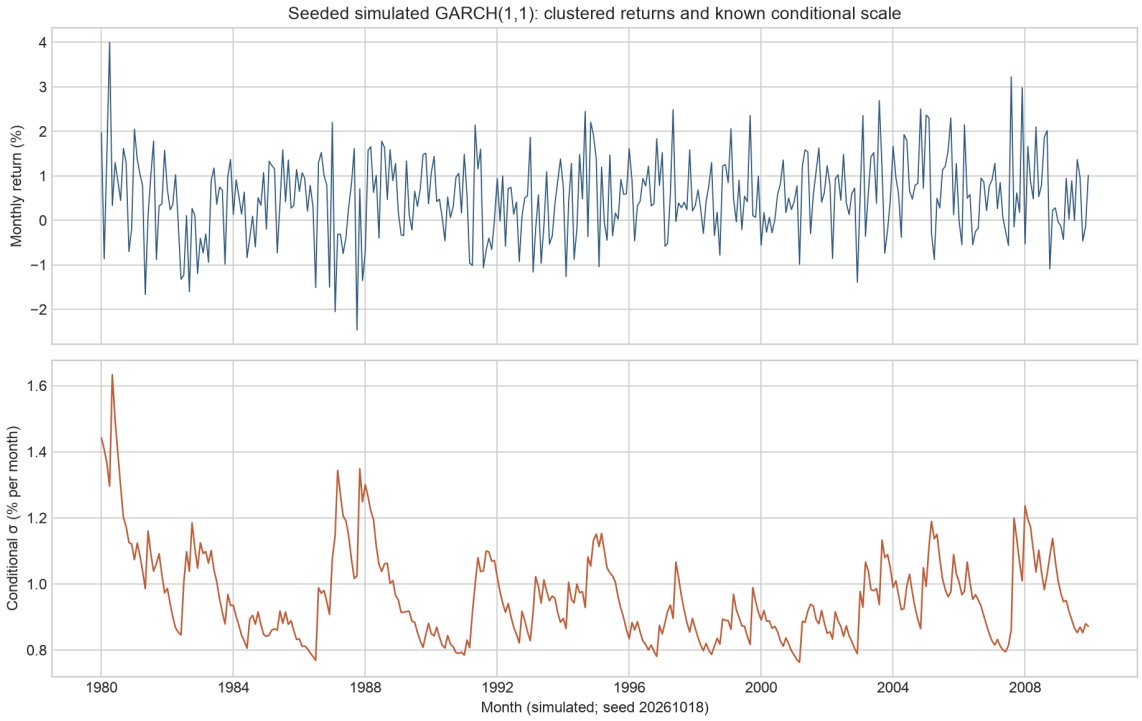


Figure 1: Simulated returns and true conditional risk

2. Return and squared-innovation autocorrelation in the real NoDur training segment: the x-axis is lag in months and the y-axis is dimensionless sample autocorrelation; the sample is the 1990–2006 training segment. Squared-innovation autocorrelations are positive at several lags, and both columns fluctuate above and below zero; the figure shows no clear, stable decay pattern. It suggests further checking for conditional heteroskedasticity, but does not establish that volatility clustering has been validated in this segment; the small-sample ACF is not a formal significance test or causal finding.

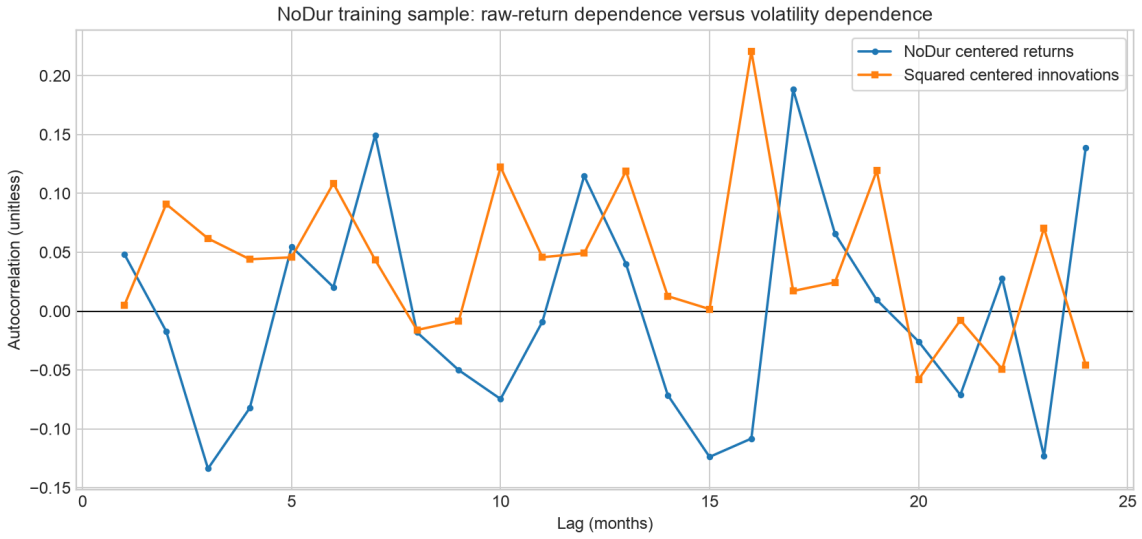


Figure 2: Real training-segment returns and squared-innovation ACF

3. NoDur crisis-window forecast comparison: the x-axis is target month from 2007–2009; the upper panel shows realized monthly returns in percent and the lower panel shows $100^2\widehat{h}$, in "percentage monthly return squared." Gray dots are the post hoc squared-innovation proxy; forecast curves are formed before the target month begins, and the recursion uses that month's shock after month-end to update the next month. The historical window includes the rise in volatility in 2008, but the proxy is extremely noisy in the figure.

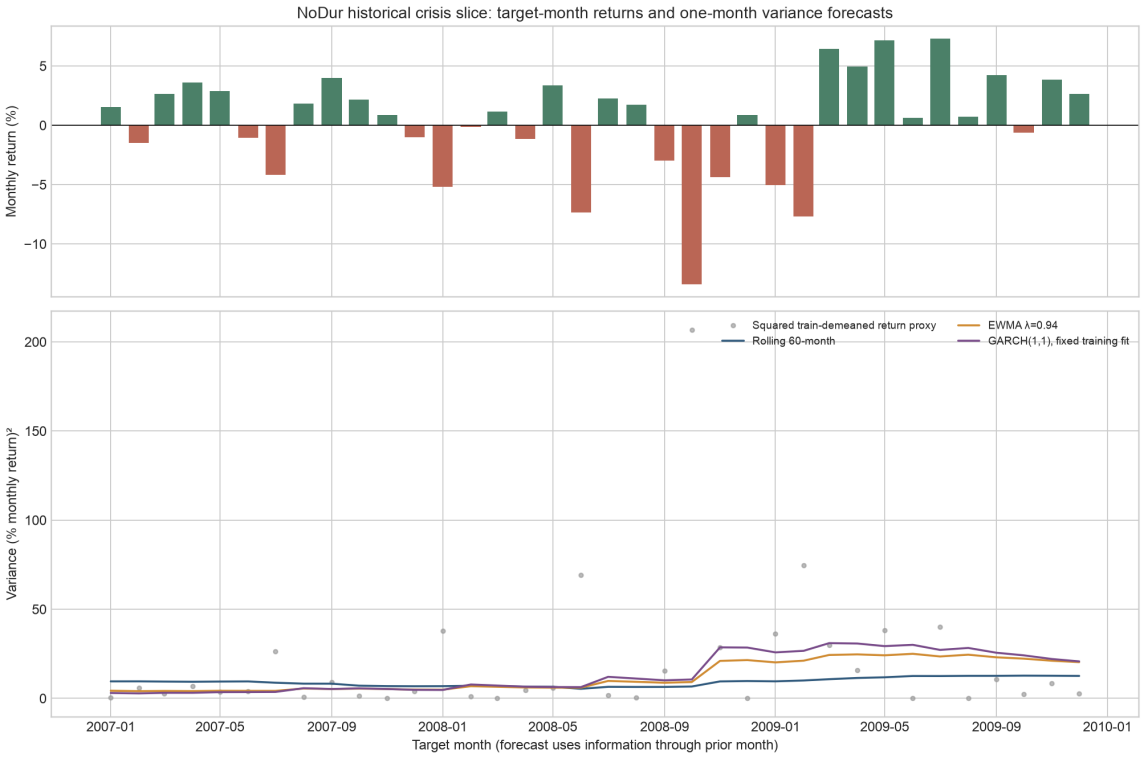


Figure 3: One-month-ahead forecasts and proxy in the historical crisis window

4. Proxy QLIKE sensitivity: the x-axis lists fixed model/window settings and the y-axis is average QLIKE over 36 target months; lower scores are better for the same sample and units. The caption identifies the proxy as $(r_t - \bar{r}_{train})^2$, not the true conditional variance. No confidence intervals, significance, or winning-model claim is provided.

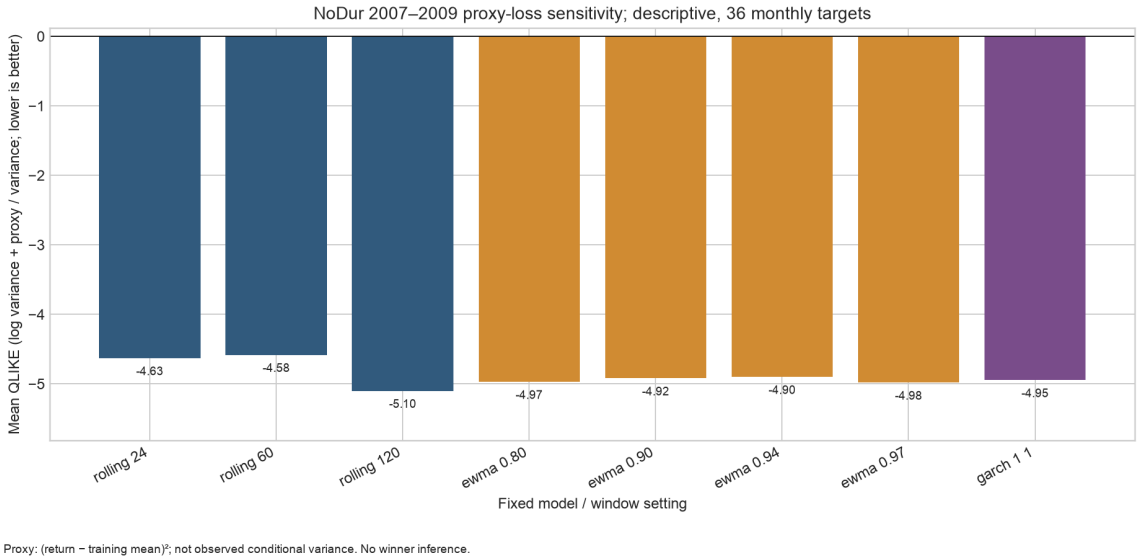


Figure 4: Proxy loss sensitivity on the same target sample

11. Real-Market Example

The case uses the French Data Library's **NoDur (Consumer Non-Durables) monthly value-weighted simple returns for the 10 industry portfolios**. The frozen snapshot was downloaded on 2026-10-05; the source table is in percent, and the processed table is converted to decimals by dividing by 100. It is monthly data, with return intervals defined by calendar month; the processed data end in 2026-08. NoDur is an industry portfolio rather than an individual stock. Its reported returns do not provide a transaction-cost or tradable-strategy ledger.

File	SHA-256	Sample contract
Raw ZIP `10_Industry_Portfolios_CSV.zip`	`ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`	Frozen file as of 2026-10-05
Extracted raw `10_Industry_Portfolios.csv`	`18ed376fc42e2327ceaf3d65a02521a6512e7f9e3001007c4da69d7db108aa82`	Official monthly value-weighted table block
Processed monthly returns `french_10_industries_monthly.csv`	`3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`	1926-07–2026-08, 1202 rows, decimal simple returns

The research design uses 1990-01 through 2006-12 as a 204-month fitting segment and 2007-01 through 2009-12 as a 36-month historical evaluation segment specified in the script in advance. The historical crisis slice shows how the recursion behaves across a change in conditions; it is not claimed as a preregistered or independent external test. The window also cannot be selected again after seeing the result and then described as prespecified. The fixed training mean for the GARCH fit is 0.0097235, and the training-sample standard deviation is 0.0389174. The fit is:

Parameter	Estimate	Meaning / units
ω	0.000004003	Constant in decimal monthly return^2
α	0.091829	Weight on the new shock square, dimensionless
β	0.903517	Weight on prior variance, dimensionless
$\alpha + \beta$	0.995346	Persistence in the teaching finite-variance model, dimensionless
$\frac{\omega}{1-\alpha-\beta}$	0.0008602	Model-implied long-run variance, decimal monthly return^2
2007-01 forecast	0.0003050	Next-month variance after updating with the last training-segment information, decimal monthly return^2

For evaluation, use $q_t = (r_t - \bar{r}_{train})^2$ as the variance proxy. The mean proxy over 36 periods is 0.0019354 (decimal return squared). QLIKE results (same aligned sample and scale; lower is better) are:

Setting	Mean QLIKE
Rolling 24	(-)4.6299
Rolling 60	(-)4.5825
Rolling 120	(-)5.0997
EWMA λ=.80 / .90 / .94 / .97	(-)4.9710 / (-)4.9207 / (-)4.9002 / (-)4.9822
Fixed-training-fit GARCH(1,1)	(-)4.9461

Rolling 120 has the lowest proxy score here; GARCH does not have the lowest loss. All settings are retained. The sample has only 36 months, squared-return proxies may be conditionally biased, the target slice is affected by crisis conditions, and no paired forecast-loss inference was conducted; therefore, no statistical winner or future trading advantage is claimed. If the mean changes in the evaluation period, the squared deviations centered on the training mean also include mean error. The theoretical conditions for proxy-based QLIKE rankings cannot be assumed to hold as a result.

The clock boundary matters: the forecast h_t uses only return shocks through (t-1); the target-month return r_t can be observed only after month-end and then used for h_{t+1}. However, actual source-data release dates, historical revisions, and the versions visible at the time have not been archived. This is historical attribution using the current frozen snapshot, not point-in-time (PIT) or genuinely real-time out-of-sample evidence.

12. Common Mistakes

Mistake	Why it is wrong	Fix
Treating `sum(intraday_returns)**2` as RV	Netting first lets subinterval moves in opposite directions cancel	Square each subinterval log return and then sum: `sum(x*x for x in returns)`
Plotting `sqrt(h)` as variance or calling `h` volatility	The units differ before and after taking the square root	Label variance "return^2" and standard deviation "return"; distinguish multiplying by 100 from multiplying by 10,000
Using `r_t**2` to forecast `h_t` for the same month	The target-month return r_t is not known before the month ends, so this leaks information	Forecast `h_t` using only ϵ_{t-1}; use the target-month square for evaluation or to update next month
Saying $\alpha + \beta < 1$ is a necessary condition for strict stationarity of every GARCH process	This is a finite-second-moment condition; strict stationarity has a different stochastic-recursion condition	Describe finite variance, strict stationarity, and the fourth moment separately; do not conflate them
Assuming EWMA $\lambda = .94$ necessarily mean-reverts	EWMA corresponds to the boundary $\alpha + \beta = 1$	State that the recursion alone does not pull variance back to a long-run level; compare initial-value effects and shock decay
Treating multi-step $h_{T+k\|T}$ as cumulative future-return variance	The core output is expected variance for each individual month	Under additional cross-period conditions, sum the variance across periods and state the forecast target
Calling squared returns true volatility	They are noisy proxies, random and possibly conditionally biased	Report model forecasts, observed proxies, and latent conditional variance separately
Treating optimizer `success=True` as proof of a global optimum or parameter significance	Multiple starts do not exhaust the parameter space; there are no standard errors	Report starts, constraints, and objective; do not claim robust inference that was not performed
Calling a low QLIKE score tradable profit	The loss is not a position, return, or trading ledger	Separately validate real-time availability, transaction costs, capacity, and net returns; none is available here

The shortest leakage demonstration is: `forecast_t = eps_t**2` uses the target month's post hoc data on the right-hand side; the correct one-step GARCH expression at the start of month t is `forecast_t = omega + alpha*eps_prev**2 + beta*h_prev`. They are not forecasts formed at the same time.

13. Applications in Quantitative Research

A reproducible volatility study should make the following chain clear.

1. **Hypothesis and target**: The question here is how a one-month variance forecast behaves under historical conditions, not whether returns can be forecast in direction or what the portfolio return will be.
2. **Data and version**: Record the French source, download date, unit conversion, missing values, hashes, release dates, and versions. A test of what was historically available requires

point-in-time data with publication times and revision versions; the current snapshot is insufficient.

3. **Clock**: Specify the training cutoff month, target month, signal calculation time, and time an order could be placed. A monthly return is observed only after the target month ends. The month-by-month recursion uses the previous month's actual shock, so future values are not backfilled into earlier forecasts.
4. **Benchmarks and candidates**: Use rolling variance as a benchmark and compare it with fixed-parameter EWMA/GARCH; prespecify candidate settings, retain sensitivity checks, and do not pick a winner from a short evaluation period.
5. **Diagnostics and loss**: Check return and squared-innovation autocorrelation, conditional-standard-deviation paths, optimization boundaries, convergence counts, and units; align target months before evaluating loss. Squared innovations are not the latent true variance; QLIKE ranking has the corresponding theoretical support only under conditions such as conditional unbiasedness of the proxy.
6. **External validation**: Recheck across different markets, periods, frequencies, and structural changes, and report failed months and instability. If converting forecasts into positions, separately analyze actual order timing, turnover, fees, market impact, financing, borrow, and capacity.
7. **Limits of the conclusion**: Statistical differences in forecast loss are not differences in trading utility; without a post-cost ledger, do not calculate "net α." This case retains the result that Rolling 120 has lower proxy loss than GARCH; it does not repeatedly retest that result to declare a winner.

This chapter has also not run a "look-ahead in the objective" trading experiment: target-period conditional variance is a risk statistic, and no rule for placing trades using it has been defined. There is no trading ledger, so costs are "not applicable," not "zero." A reproducible net-α study would still need a signal, asset holdings, execution prices and delays, a fee schedule, borrow/financing records, capacity constraints, and a net-return series.

14. Chapter Summary

RV sums the squared increments in an already observed, finely partitioned path; conditional variance h_t forecasts next-period risk using information available before the target period begins. A rolling window provides an auditable benchmark; EWMA mixes the new shock square with old variance; ARCH uses lagged shock squares; GARCH also carries forward past variance.

For a finite-second-moment GARCH(1,1) with standardized innovations of variance 1, $\alpha + \beta < 1$ gives the long-run variance $\frac{\omega}{1-\alpha-\beta}$ and mean-reverting multi-step recursion; it is not the same as strict stationarity or the fourth-moment condition. EWMA's weights sum to 1, which is precisely its boundary behavior without reversion to a separate long-run level. High-frequency RV also faces sampling and microstructure noise.

The code puts these concepts into practice through the separation of training and target data, data hashes, and the recursive clock. The historical NoDur 2007–2009 comparison shows the difference between a forecast and a squared-return proxy; it cannot establish availability at the time, a winning model, strategy profitability, or a future trading advantage.

15. Exercises

18-E01 Concept: RV and Sample Variance. In one precise sentence, compare interval RV, sample variance, and squared net return; list two path features that can make RV greater than squared net return.

18-E02 Concept: Clustering. Why can volatility clustering exist even if return autocorrelation is close to zero? Give one sample statistic to check and explain what it cannot prove.

18-E03 Concept: Stationarity and Moments. What does each of the following describe: the finite-second-moment condition, strict stationarity, and a finite fourth moment? How do they differ logically?

18-E04 Concept: EWMA. What does EWMA's λ control? Why does the boundary $\alpha + \beta = 1$ not mean "mean reversion to some long-run variance"?

18-E05 Hand Calculation: Realized Risk. For log returns $[0.01, -0.02, 0.03]$ in three subintervals of the same interval, calculate RV, RVOL, and squared net log return. Explain the units.

18-E06 Hand Calculation: EWMA. Let $\lambda = .9$, $h_1 = .0004$, and the shocks be $[.01, -.02, .03]$. Write the first three monthly forecasts h_1, h_2, h_3 under the target convention.

18-E07 Hand Calculation: ARCH/GARCH. Let $\omega = .00001$, $\alpha = .1$, $\beta = .8$, $h_1 = .0004$, and the first two shocks be .01 and $-.02$. Find GARCH h_1, h_2, h_3, then find ARCH h_2, h_3 when $\beta = 0$.

18-E08 Persistence and Higher Moments. For parameters $(\omega, \alpha, \beta) = (.00001, .1, .8)$, calculate the long-run variance and the left-hand side of the Gaussian fourth-moment condition. Does each condition hold?

18-E09 Multi-Step Forecast. For the GARCH model above, if $h_{T+1|T} = .0004$, list the variance forecast for each target month at $k = 1, 2, 3$. Why can't the last number be treated as the cumulative three-month variance?

18-E10 Hand Calculation: Noise Bias. Suppose the expected sum of squared efficient-price increments over the interval is 0.001, each observation-noise variance is $\tau^2 = .000001$, and the sampling count is M=10. Under the simple independent-noise model, what is the approximate expected observed RV? What about M=100?

18-E11 QLIKE. Write the average QLIKE. If proxy q is conditionally unbiased and candidate variance is positive, explain why forecast risk affects expected loss; then list at least two threats to the current NoDur proxy ranking.

18-P01 Implement RV. Pass the three subinterval log returns from the same interval, $[.01, -.02, .03]$, to the shared core; assert that the result is `.0014` and also display the squared net return. Explain why it is not automatically annualized.

18-P02 Forecast Clock. Use four consecutive monthly returns $[.01, .03, -.01, .02]$ and window 2; call the rolling variance forecast function (the P02 answer must include the complete code). Write the forecast for month 3, change month 3 and month 4 returns to 100, and recalculate; the month-3 forecast must remain unchanged. Explain why the first two months are NaN.

18-P03 Reproducible Research. Run the chapter CLI from any working directory and check the three input SHA-256 hashes, training and evaluation record counts, results JSON, and four figures. If a hash does not match, explain why the correct behavior is to stop rather than download the data and continue automatically.

18-C01 QML Boundary. Give three differences between core `fit_garch11` and joint normal MLE; explain what "three starting points converged" can and cannot establish. List a typical set of regularity conditions for consistent QML variance estimation; do not claim it holds for arbitrary nonnormality.

18-C02 High-Frequency Microstructure. Starting from $Y_i = X_i + u_i$, derive the extra term in the expected RV from independent noise. Give one realistic assumption that may fail and design a study to test sampling frequency; do not assume one universally best frequency.

18-C03 Historical vs. PIT. Explain why the French NoDur 2007–2009 results in this chapter are neither historical PIT evidence nor a preregistered test. List the ledger fields still needed for a net strategy test, and distinguish "costs not applicable" from "costs are zero."

18-C04 Constructed Counterexample: Strict Stationarity and Variance. For $\alpha = .1$, $\beta = .9$, and (z~ N(0,1)), compare $E(\alpha z^2 + \beta)$ with $E\log(\alpha z^2 + \beta)$. Explain what follows under the standard stochastic-recursion condition and what finite moment cannot be asserted.

Teaching Supplements (Independent IDs)

18-R01 Initialization Effects. Use the expansion in §5.3 for EWMA to prove that the forecast difference from two initial values decays geometrically. If the initial values differ by 0.0005 and lambda=0.9, what is the difference after two updates?

18-R02 Proxy and Unit Diagnostics. The true conditional mean is 0.01, the fixed center is 0, and the true conditional variance is 0.0004. What is the conditional expectation of the squared-return proxy? If returns are uniformly converted from decimals to percentages, by how much does QLIKE increase, and does the ranking change?

18-R03 Research Judgment. A researcher compares intraday RV with a monthly forecast and declares it unbiased because its proxy mean is close to the mean of monthly squared returns. Which conditions and records are missing?

16. Programming Exercises

Complete **18-P01–18-P03**. Deliver a runnable cell or script, assertions, run command, Python and dependency environment, output paths, data version, and clock description. P02 must display the target index and NaN warmup; do not silently fill warmup rows with zeros. P03 must not touch frozen raw files or introduce a network download.

17. Challenge Problems

Complete **18-C01–18-C04**. Answers may use equivalent reasoning, but must state assumptions, units, information set, and testable steps.

- For full credit on C01, distinguish the fixed training-sample mean from joint MLE; explain that consistency and robust inference are different questions; state at least three of the following: correct conditional mean/variance, stationarity and ergodicity, identification, and finite integrability; and note that the core has no parameter standard errors.
- For full credit on C02, expand the noise difference and squared expectation and propose a frequency-sensitivity plan; identify the limits of the simple derivation, including correlated noise, correlation between price changes and noise, jumps, and asynchronous observations.
- For full credit on C03, identify the missing data version/release times and historical availability, the ex post crisis slice, and the absence of position or execution ledgers; provide at least six net-return audit fields.
- For full credit on C04, use the strict Jensen inequality to distinguish $EA = 1$ from $E \log A < 0$; explain that a strictly stationary solution can exist while the second moment formula is infinite or does not hold; do not present the chapter's finite-variance implementation as covering this region.

18. Complete Answers and Hints

18-E01

RV is the sum of squared log returns over subintervals within the same interval; sample variance centers on the sample mean and divides by (M-1); squared net return nets the returns first and then squares. Repeated reversals reduce the net amount, and more nonzero subintervals add to the sum of squares, so path cancellation and back-and-forth movement within an interval can both make RV greater than squared net return.

18-E02

If the signs or linear variation in innovations are unpredictable but their conditional scale changes with market conditions, then r_t can have low autocorrelation while r_t^2 inherits persistence in the scale state. Check the sample ACF of centered squared innovations. It is a structural diagnostic; it cannot establish a causal mechanism, predict future direction, or show economic significance.

18-E03

The finite-second-moment condition $\alpha+\beta < 1$ ensures a finite unconditional-variance solution in this chapter's setup. Strict stationarity means that the joint distribution of the process does not change under time shifts; for the GARCH stochastic recursion, $E\log(\alpha z^2+\beta) < 0$ is commonly used as a condition. With Gaussian shocks, a finite fourth moment additionally requires $3\alpha^2 + 2\alpha\beta+\beta^2 < 1$. The latter is stronger; a finite second moment suffices for the corresponding strictly stationary solution but is not necessary for strict stationarity, and strict stationarity does not ensure a second moment exists.

18-E04

λ is the retention rate on the old variance state, and $1-\lambda$ is the weight on the latest shock square. The recursion allocates all weight between the "old forecast" and the "new shock"; it has no additional ω pulling the state toward a separate long-run level. It lies on the $\alpha+\beta=1$ boundary, so the finite-variance mean-reversion expression $\omega/(1-\alpha-\beta)$ does not apply.

18-E05

$$RV = .01^2 + .02^2 + .03^2 = .0001 + .0004 + .0009 = .0014,$$

$$RVOL = \sqrt{.0014} \approx .0374166 = 3.7417\%.$$

Net log return = .02, and squared net log return = .0004. RV is measured in squared decimal returns over the interval; RVOL is measured in decimal returns over the interval. There is no annualization factor.

18-E06

$$h_1 = .0004, \quad h_2 = .9(.0004) + .1(.01^2) = .00037,$$

$$h_3 = .9(.00037) + .1((-.02)^2) = .000373.$$

The third innovation, .03, generates h_4; it cannot affect h_3, which was formed before that month began.

18-E07

$$h_1 = .0004, \quad h_2 = .00001 + .1(.01^2) + .8(.0004) = .00034,$$

$$h_3 = .00001 + .1((-.02)^2) + .8(.00034) = .000322.$$

With $\beta = 0$, $h_2 = .00001 + .1(.0001) = .00002$, and $h_3 = .00001 + .1(.0004) = .00005$. All are in squared decimal monthly returns.

18-E08

$$\bar{h} = .00001/(1 - .1 - .8) = .0001.$$

The Gaussian fourth-moment left-hand side is $3(.1)^2 + 2(.1)(.8) + (.8)^2 = .83 < 1$, so both conditions hold. This result establishes a finite fourth moment only under the specified Gaussian standardized-shock and model assumptions; it cannot be generalized to arbitrary shock distributions.

18-E09

$\rho = .9, \bar{h} = .0001.$

$$h_{T+1|T} = .0004, \quad h_{T+2|T} = .0001 + .9(.0004 - .0001) = .00037,$$

$$h_{T+3|T} = .0001 + .9^2(.0004 - .0001) = .000343.$$

These are risk forecasts for one month each. Cumulative interval variance is the sum of monthly variances only under additional conditions, including zero cross-period innovation conditional covariance given the forecast origin and a known mean; $h_{T+3|T}$ alone does not represent the three-month total.

18-E10

The variance of each noise difference is $2\tau^2 = .000002$. At M=10, the extra term is approximately .00002, so expected RV is (approximately 0.00102); at M=100, the extra term is approximately .0002, so expected RV is (approximately 0.0012). This uses a simplified independent and identically distributed noise model; it is not a frequency rule for every market.

18-E11

$$\bar{L} = \frac{1}{T}\sum_{t}(\log \widehat{h}_t + q_t/\widehat{h}_t).$$

If $E(q_t \mid F_{t-1}) = h_t^\star$, the conditional expected loss as a function of positive forecast $\widehat{h}$ is $\log \widehat{h} + h_t^\star/\widehat{h}$, with derivative $1/\widehat{h} - h_t^\star/\widehat{h}^2$; it is minimized at $\widehat{h}{=}h_t^\star$. In the actual NoDur case, the squared proxy may be affected by differences between the training and evaluation means, high noise in a single-period square, revisions to the current data version, only 36 months of data, and date misalignment.

18-P01

Reference implementation:

```
from quant_course.volatility import realized_variance

parts = [0.01, -0.02, 0.03]
rv = realized_variance(parts)
net_squared = sum(parts) ** 2
assert abs(rv - 0.0014) < 1e-12
assert abs(net_squared - 0.0004) < 1e-12
print(rv, net_squared)
```

Output: `0.0014 0.0004`. The function does not demean, divide by the sample count, or annualize; the caller must record the interval and sampling design from which these returns came.

18-P02

```
import pandas as pd
from quant_course.volatility import rolling_variance_forecast

idx = pd.period_range("2000-01", periods=4, freq="M")
r = pd.Series([0.01, 0.03, -0.01, 0.02], index=idx)
f = rolling_variance_forecast(r, window=2)
assert f.iloc[:2].isna().all()
assert abs(f.iloc[2] - 0.0002) < 1e-12  # sample variance of 0.01 and 0.03
r2 = r.copy(); r2.iloc[2:] = 100.0
f2 = rolling_variance_forecast(r2, window=2)
assert f2.iloc[2] == f.iloc[2]
```

The month-3 forecast uses only the two observations from months 1 and 2. Their sample variance is $(.01 - .02)^2 + (.03 - .02)^2$ divided by $2 - 1$, which equals .0002. The first two target months do not have two previous observations, so they remain NaN. Changing the current or future return cannot change the month-3 forecast.

18-P03

Run from a current directory outside the repository:

```
cd /tmp
/absolute/path/to/quant_course/.venv/bin/python \
  /absolute/path/to/quant_course/code/ch18_volatility.py \
  --output-dir /tmp/ch18-results --figure-dir /tmp/ch18-figures
```

Check the JSON: `train_n=204`, `evaluation_n=36`, training endpoint `2006-12`, test window `2007-01`⋯`2009-12`, the three hashes match the manifest, and the paths to four figures exist. A hash mismatch means the inputs are no longer the frozen evidence for this study; silently downloading a new version would change the research object and its version. The correct action is to stop, report the difference, and establish a separate manifest for the new snapshot.

18-C01

The in-class function first computes and fixes the training-segment sample mean, then optimizes only the variance parameters; joint normal maximum likelihood optimizes the mean and variance parameters together in the same likelihood. The core objective is normalized by the scale of training residuals, omits a constant, and takes the average; after fitting, ω is multiplied back by the variance scale. Convergence from three starting points shows that at least one feasible numerical solution was found from those starts and the code selected the one with the lower objective; it does not prove a global optimum, a correct model, significant parameters, or future stability.

Typical consistency conditions can include a correctly specified (or explicitly quasi-correct) conditional mean/variance, a stationary and ergodic process, a finite and integrable objective, identified parameters with a parameter space satisfying the required conditions, and numerical optimization close to the objective's extremum. Under nonnormality, inference usually also requires a corresponding robust variance estimate; the core provides no robust standard errors. Convergence of an in-sample fit does not replace these theoretical and empirical conditions.

18-C02

Expand ΔY, square, sum, and take expectations. The efficient-price term gives the efficient price's squared variation; under independent, zero-mean noise, the cross-term has expectation zero; each noise difference Δu has variance $2\tau^2$, for an increase of approximately $2M\tau^2$ across M intervals. Real-world noise may be serially correlated or correlated with efficient-price changes; bid–ask bounce, asynchronous trades, and jumps can also change the error structure.

First define the asset, trading hours, and target; select several sampling intervals in advance; estimate RV using the same cleaning, alignment, and interval rules, while retaining jump/noise diagnostics; compare error and robustness across dates and liquidity regimes; where possible, combine estimates of microstructure noise with alternative realized measures; and report estimation uncertainty. Do not generalize one frequency chosen in one paper for one asset to all settings.

18-C03

The data are the current French snapshot downloaded on 2026-10-05, with no historical month-by-month release versions, availability times, or local-ingestion timestamps; historical NoDur values may have been revised. The crisis slice is fixed in the script, not preregistered or an independent external test. Although the parameters are fitted before evaluation begins, the test still uses existing historical target-month returns and later recursive updates; this does not reconstruct real-time operations.

A post-cost strategy also needs: signal time, target weights/positions, executable order time, order/execution prices, spreads and fees, turnover, market impact, financing/borrow, cash/margin, capacity limits, handling of unfilled/delayed orders, and a benchmark and net-return series. Without a strategy and order ledger, costs are not applicable; this does not mean costs were set to zero or that net returns have no costs.

18-C04

Let $A = 0.1z^2 + 0.9$. For $z \sim N(0,1)$, EA=0.1(1)+0.9=1. A is nondegenerate, and if $E|\log A| < \infty$, strict Jensen gives $E \log A < \log EA = 0$. The stochastic-coefficient recursion $h_t = \omega + A_{t-1}h_{t-1}$ can therefore have a strictly stationary solution under standard conditions. However, EA=1 means the finite unconditional variance formula $\omega/(1 - EA)$ does not exist, and the second moment is not finite; strict stationarity alone does not justify reporting a finite long-run h. The in-class core restricts parameters to $\alpha + \beta < 1$, so it does not fit this example.

Complete Answers to the Teaching Supplements

18-R01. Subtracting the two recursions with the same shock sequence gives $\delta_{t+1} = \lambda\delta_t$, so after two updates the difference is $0.9^2 \times 0.0005 = 0.000405$. The initial value must be set before forecasting begins.

18-R02. The conditional expectation is $0.0004 + (0.01 - 0)^2 = 0.0005$, giving a conditional bias of 0.0001. Uniformly converting both forecast and proxy increases the loss by log(10000), approximately 9.210340; all models on the same sample shift by the same constant, so the ranking is unchanged.

18-R03. First check whether intraday RV covers the complete target month, the return definition and units, sampling noise and jumps, whether the forecast was saved before the month, and whether the effective samples match. Unbiasedness requires the conditional expectation to match the target; close unconditional sample means do not prove conditional unbiasedness and do not establish a significant win or net return.

19. Further Reading

- **Must Know — Bollerslev (1986), §§2–3.** Read the GARCH representation, weak-stationarity variance condition, and GARCH(1,1) fourth-moment condition; the corresponding passages and author-PDF page references for this chapter are recorded in `source_notes.md`.
- **Must Know — Andersen, Bollerslev, Diebold & Labys (2003), Introduction and §2.** Read about the asymptotic relationship between high-frequency sums of squared returns and quadratic variation/integrated variance, including the jump term; do not misread a limit result as equality between finite-grid RV and conditional variance.
- **Should Know — Hansen & Lunde (2006), Introduction, theoretical framework, empirical section and conclusion.** Read how microstructure noise and frequency choice change realized-variance precision; frequency conclusions from that paper's sample apply only to its sample and setting.
- **Should Know — Patton (2011), §2 propositions and conclusion.** Read the requirements for forecast-comparison losses with noisy but conditionally unbiased variance proxies; the proxy used in this chapter does not automatically meet them.
- **Should Know — SciPy `minimize(method="SLSQP")` official documentation.** Learn the constrained-optimization API, stopping precision, and solver status; stopping precision does not prove a global optimum.
- **Optional — Official ARCH/GARCH forecast and EWMA API documentation.** Use it to check one- and multi-step variance targets and alignment; this chapter does not install or use the `arch` estimator.
- **Advanced — Engle (1982).** The original paper is the source of ARCH, but this work checked only the bibliographic information on the author's official page and did not obtain the full paper. You may obtain it through an institutional library and check the specific test derivation; this chapter does not cite unread passages as authority.

20. References

1. French Data Library. **10 Industry Portfolios**, monthly average value-weighted returns. The original download URL is in the data manifest; the study snapshot, processing, and hashes are listed in Section 11. The table contains currently downloadable historical values, not historical point-in-time vintages.
2. Bollerslev, T. (1986). "Generalized autoregressive conditional heteroskedasticity." *Journal of Econometrics*, 31(3), 307–327. DOI: 10.1016/0304-4076(86)90063-1. This chapter checks the key moment conditions against §§2–3 of the author's original journal PDF, physical PDF pages 3–6; the specific reading range is in `source_notes.md`.
3. Andersen, T. G., Bollerslev, T., Diebold, F. X., & Labys, P. (2003). "Modeling and forecasting realized volatility." *Econometrica*, 71(2), 579–625. DOI: `10.1111/1468-0262.00418`. This chapter reads the Introduction and opening of §2; the derivation coverage is noted in the source notes.
4. Hansen, P. R., & Lunde, A. (2006). "Realized variance and market microstructure noise." *Journal of Business & Economic Statistics*, 24(2), 127–161. DOI: `10.1198/073500106000000071`.
5. Patton, A. J. (2011). "Volatility forecast comparison using imperfect volatility proxies." *Journal of Econometrics*, 160(1), 246–256. DOI: `10.1016/j.jeconom.2010.03.034`.
6. Engle, R. F. (1982). "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation." *Econometrica*, 50(4), 987–1008. DOI: `10.2307/1912773`. The full paper was not obtained for this work, so no specific theorem or within-paper step is attributed to it.
7. SciPy documentation. `scipy.optimize.minimize`, SLSQP method. Optimizer API and stopping criterion only.
8. `arch` 7.2 documentation. Forecasting; EWMA variance process. Documentation consulted for definitions and alignment; the estimator was not installed or used.

Source locations, historical reading ranges, and verification limits are detailed in `source_notes.md`.

Chapter 19 — State-Space Models and the Kalman Filter

Core idea: First use the state transition to form a prior, then use the innovation from a new observation to update the hidden state. The timestamp determines whether that update can inform a trade. Prerequisites: Chapter 5, linear algebra; Chapter 6, conditional Gaussian distributions; Chapter 8, regression; Chapter 16, monthly time indexing. By the end of this chapter, you will be able to derive the scalar-observation Kalman update from a conditional Gaussian distribution, calculate a one-dimensional dynamic beta by hand, explain a two-dimensional alpha/beta state and its hedge ratio, distinguish filtering from smoothing, and build a research workflow that keeps future information out of historical signals.

1. Learning objectives

After reading, you should be able to write a state-space representation and state the units and dimensions of its states, observations, and noise; derive the prediction, innovation, innovation variance, Kalman gain, and posterior covariance from a joint Gaussian distribution; calculate a beta update by hand; distinguish the information used by a filtered state from that used by a smoothed state; and run the core function, fixed-seed simulation, and frozen-data example while explaining which investment claims they do not validate.

You have completed this chapter when your hand calculations match the code, every state and observation has a clear timestamp, and you can find complete answers to the original 13 questions and the 3 added teaching questions in Section 18.

2. Why Quant practitioners use state-space models

A historical regression beta is the average market exposure over a particular sample window. If exposure changes over time, a fixed beta averages away those changes. Rolling ordinary least squares (OLS) repeatedly changes the sample window, so both the window length and the re-estimation point affect the result.

A state-space model separates the problem into two processes: how latent exposure changes and how current returns vary around that exposure. A dynamic beta is an estimate of market exposure that changes over time. Each month, as new returns arrive, you compare them with the model's prior expectation and update the beta estimate. The process-noise variance specifies how much state change the model allows; the observation-noise variance describes how much returns deviate from the relationship with the state.

This decomposition gives risk estimation a recursive update rule. It does not determine whether beta is stable, how large process noise should be, whether a hedge can be executed, or whether dynamic beta outperforms rolling OLS. You still need training-period and out-of-sample evidence to answer those questions.

3. Intuition: the state changes behind the scenes while returns mix signal and noise

You can observe monthly asset and market returns, but not the asset's true sensitivity to the market. That sensitivity is a latent state. Monthly residuals can also reflect industry shocks, portfolio changes, and measurement error.

The filter processes each month in sequence:

Timing	Information available	Action	Output
Before the month begins	Observations through the prior month	Project forward with the state-transition model	Prior mean and variance for the month
After this month's returns arrive	This month's asset and market returns	Subtract the prior expectation from the realized value	Innovation and its variance
Month-end update	The prior, innovation, and variance above	Adjust the state based on uncertainty	Filtered state incorporating this month's observation

The prior distribution is the state distribution before the observation month begins; the posterior distribution is the state distribution conditional on this month's observation. The predicted state can inform risk controls during the month, but the filtered state does not exist until this month's returns arrive. They refer to different information sets.

4. Mathematical setup and notation

Suppose the state has d components and each period provides one scalar observation:

$$\mathbf{x}_t = F_t \mathbf{x}_{t-1} + \boldsymbol{\eta}_t, \qquad y_t = \mathbf{h}_t \mathbf{x}_t + \varepsilon_t.$$

$\mathbf{x}_t \in \mathbb{R}^{d\times 1}$ is the state vector and latent state; $F_t \in \mathbb{R}^{d\times d}$ is the state-transition matrix; $\mathbf{h}_t \in \mathbb{R}^{1\times d}$ is the observation-design row vector; and $y_t \in \mathbb{R}$ is the current observation. The state noise is $\boldsymbol{\eta}_t \in \mathbb{R}^{d\times 1}$, and the observation noise is $\varepsilon_t \in \mathbb{R}$.

In this chapter, let

$$\boldsymbol{\eta}_t \sim N(\mathbf{0}, Q_t), \qquad \varepsilon_t \sim N(0, R_t),$$

where $Q_t \in \mathbb{R}^{d\times d}$ is the state-noise covariance and $R_t > 0$ is the scalar observation-noise variance. The conditional covariance of the state distribution given the information available, which is also the covariance of state-estimation error, is denoted by P. Covariance matrices are positive semidefinite, and the observation-noise variance is positive. Here, N denotes a Gaussian distribution.

The implementation starts from $\mathbf{x}_{-1}$ and applies one state transition before observing y_0. This convention separates the initial state from the first observation. If the initial value is mistakenly treated as $\mathbf{x}_0$ and then transitioned again, the first-period prior changes. A random walk with $F = 1$ hides this offset, so the core check also uses $F = 2$ to test the first step. This chapter uses different notation from the general representation in statsmodels 0.15.0. The official

documentation uses Z_t for the observation design, H_t for the observation-noise covariance, T_t for the state transition, R_t for the noise-selection matrix, and Q_t for the noise covariance. Thus, this chapter's $\mathbf{h}_t$ corresponds to the official Z_t; this chapter's F_t corresponds to the official T_t; and this chapter's observation variance R_t corresponds to the official H_t. If the official state noise passes through a selection matrix, the state covariance in this chapter equals $R_t Q_t R_t^\top$. The official R_t is a matrix, not this chapter's observation-noise variance.

Which object does each of the three uncertainties belong to?

Quantity	Random object	Role in the recursion
Q_t	New state shock $\boldsymbol{\eta}_t$	State-change uncertainty added after the transition
R_t	Observation error ε_t	Noise that remains in the observation given the state
$P_{t\|t-1}$, $P_{t\|t}$	State minus its prior/posterior estimate	Estimation-error covariance computed from the information available

Q, R are model inputs specified in this chapter; P is the state uncertainty produced by the recursion. The state itself can change each month, and even when Q is fixed, P can still change as observations update it. Q is not a beta value, and R is not the asset's total-return variance; the latter also includes the part projected from the state through the design row.

Notation note: the scalar h_t in Chapter 18 is the conditional variance of monthly returns; the bold $\mathbf{h}_t$ in this chapter is the design row that maps the state to the observation. Here, P is the state-error covariance, while S is the innovation variance. In the code, `observation_design` is the design, `process_covariance` corresponds to Q, and `observation_variance` corresponds to R; the interface follows the existing implementation.

5. Formal definitions: hidden states, observations, and model assumptions

A latent state is a variable the model describes but cannot directly observe. Dynamic beta is one example. The observation equation links it to visible returns, while the state equation describes how it changes from one period to the next.

Filtering computes, in time order,

$$\hat{\mathbf{x}}_{t|t} = E[\mathbf{x}_t \mid y_0, \ldots, y_t],$$

The subscript $t|t$ means that the estimate uses observations through t. Prediction is computed before the current observation arrives:

$$\hat{\mathbf{x}}_{t|t-1} = E[\mathbf{x}_t \mid y_0, \ldots, y_{t-1}].$$

Smoothing computes

$$\hat{\mathbf{x}}_{t|T} = E[\mathbf{x}_t \mid y_0, \ldots, y_T], \quad T > t.$$

A smoothed state includes observations after the target month, so it is useful for explaining the full history after the fact. This chapter does not implement a smoother; do not describe smoothing as a code output or a real-time signal.

The conditional Gaussian posterior in this chapter requires the following assumptions:

1. The initial state $\mathbf{x}_{-1}$ has a Gaussian distribution with known mean and covariance.
2. State noise and observation noise are zero-mean Gaussian in each period; noise is independent across periods.
3. State noise, observation noise, and the initial state are mutually independent.
4. F_t, Q_t, R_t, and the design $\mathbf{h}_t$ are known at the relevant time. With a random design, the stated independence assumptions must continue to hold conditional on that design. If a market input is correlated with a state shock, the current recursion does not automatically correct for that endogeneity.
5. The observation variance is positive, the covariance is positive semidefinite, and the innovation variance is invertible. With one scalar observation, the innovation variance is a positive scalar.

The conditional expectations below also assume that the model matrices and an appropriate exogenous observation design are given. In the dynamic-beta example, the current month's market return f_t is not fully known until month-end: it is not needed for the state prediction at the start of the month, and substituting realized f_t into the expected observation gives a contemporaneous conditional fit, not a forecast of the asset return made at the start of the month.

The Gaussian assumption ensures that each conditional distribution remains Gaussian, so the mean and covariance describe the full posterior. If the distribution is non-Gaussian, linear projection can still give a linear minimum mean-square-error estimate; the recursive quantities alone do not establish that the full posterior is Gaussian. The assumptions determine the scope of the conclusions.

6. Deriving one update from a conditional Gaussian distribution

First, let the state estimate through $t-1$ be

$$\mathbf{x}_{t-1} \mid \mathcal{Y}_{t-1} \sim N(\mathbf{m}_{t-1}, P_{t-1}), \qquad \mathcal{Y}_{t-1} = \{y_0, \ldots, y_{t-1}\}.$$

6.1 Predict with the state equation

Substitute the state transition into the conditional mean:

$$\mathbf{m}_{t|t-1} = E[F_t\mathbf{x}_{t-1} + \boldsymbol{\eta}_t \mid \mathcal{Y}_{t-1}] = F_t\mathbf{m}_{t-1}.$$

Zero-mean, independent state noise does not change the predicted mean. Apply the linear-transformation rule to the covariance, then add the covariance of the independent noise:

$$P_{t|t-1} = F_tP_{t-1}F_t^\top + Q_t.$$

This variance describes the model's uncertainty about the state before this period's return arrives.

6.2 Calculate the prior observation and innovation

The observation equation gives the prior observation mean $\hat{y}_{t|t-1} = \mathbf{h}_t \mathbf{m}_{t|t-1}$. The innovation is the new observation minus the prior observation:

$$v_t = y_t - \mathbf{h}_t \mathbf{m}_{t|t-1}.$$

The conditional covariance between the state and observation, and the innovation variance, are

$$C_t = \mathrm{Cov}(\mathbf{x}_t, y_t \mid \mathcal{Y}_{t-1}) = P_{t|t-1}\mathbf{h}_t^\top,$$

$$S_t = \mathrm{Var}(v_t \mid \mathcal{Y}_{t-1}) = \mathbf{h}_t P_{t|t-1}\mathbf{h}_t^\top + R_t.$$

C_t is a $d \times 1$ vector, and S_t is a scalar. It projects state uncertainty into return units and adds the observation-noise variance of returns.

See the update weight through scalar precision

For a scalar prior with $P^- > 0$ and $R > 0$, and a known nonzero design H that satisfies the exogeneity condition above, the observation y/H is the state plus noise with variance R/H^2. Completing the square gives

$$P^+ = \left(1/P^- + H^2/R\right)^{-1}, \qquad m^+ = P^+\left(m^-/P^- + Hy/R\right).$$

The two information precisions add. Let $w = H^2P^-/(H^2P^- + R) = KH$; then

$$m^+ = (1-w)m^- + w(y/H), \qquad 0 < w < 1.$$

The convex weight is $w = KH$, not K, which has state/observation units. If $H = 0$, the observation is unrelated to the state, $K = 0$, and the posterior remains the prior; do not divide by H. A degenerate prior with $P^- = 0$ also cannot use the inverse-precision formula, but the original covariance recursion still applies.

Hold $m^- = 1, H = 0.02, y = 0.03$ fixed (the innovation remains 0.01) along with the previous-period $P = 0.04, F = 1$, and change only the model inputs:

Q	R	$P^- = P + Q$	S	K
0.01	0.0001	0.05	0.00012	8.333333
0.06	0.0001	0.10	0.00014	14.285714
0.01	0.0004	0.05	0.00042	2.380952

Q	KH	m^+
0.01	0.166667	1.083333
0.06	0.285714	1.142857
0.01	0.047619	1.023810

For this fixed positive design, increasing Q raises prior uncertainty and gives the observation more influence on the state; increasing R lowers observation precision and reduces the adjustment. This illustrates how the model allocates information; it does not establish that any particular Q, R is better in markets.

6.3 Obtain the posterior from the conditional formula for a joint Gaussian

Conditional on the historical observations, the state and new observation have a joint Gaussian distribution:

$$\begin{bmatrix} \mathbf{x}_t \\ y_t \end{bmatrix} \sim N\left(\begin{bmatrix} \mathbf{m}_{t|t-1} \\ \mathbf{h}_t \mathbf{m}_{t|t-1} \end{bmatrix}, \begin{bmatrix} P_{t|t-1} & P_{t|t-1}\mathbf{h}_t^\top \\ \mathbf{h}_t P_{t|t-1} & S_t \end{bmatrix} \right).$$

Apply the conditional-mean and Schur-complement formulas for a partitioned Gaussian distribution to obtain

$$\mathbf{m}_{t|t} = \mathbf{m}_{t|t-1} + P_{t|t-1}\mathbf{h}_t^\top S_t^{-1} v_t,$$

$$P_{t|t} = P_{t|t-1} - P_{t|t-1}\mathbf{h}_t^\top S_t^{-1}\mathbf{h}_t P_{t|t-1}.$$

Define the Kalman gain

$$K_t = P_{t|t-1}\mathbf{h}_t^\top S_t^{-1},$$

The mean can then be written as $\mathbf{m}_{t|t} = \mathbf{m}_{t|t-1} + K_t v_t$. The gain comes from the covariance between the state and observation; it converts an innovation measured in return units into an adjustment measured in state units. The filter passes $(\mathbf{m}_{t|t}, P_{t|t})$ to the next-period prediction.

The implementation uses the Joseph covariance update:

$$P_{t|t} = (I - K_t\mathbf{h}_t)P_{t|t-1}(I - K_t\mathbf{h}_t)^\top + K_t R_t K_t^\top.$$

Under the model assumptions, this equals the conditional-covariance formula above; under finite-precision rounding, it better preserves symmetry and positive semidefiniteness. With one scalar observation, R_t is a scalar and K_t is $d \times 1$.

7. Financial interpretation: two-dimensional alpha, beta, and a dynamic hedge ratio

Write the asset and market monthly excess simple returns as

$$y_t = \alpha_t + \beta_t f_t + \varepsilon_t, \qquad \mathbf{x}_t = \begin{bmatrix} \alpha_t \\ \beta_t \end{bmatrix}, \qquad \mathbf{h}_t = \begin{bmatrix} 1 & f_t \end{bmatrix}.$$

Thus, the observation equation is $y_t = \mathbf{h}_t\mathbf{x}_t + \varepsilon_t$. The design matrix $\mathbf{h}_t$ has dimension 1×2, the state vector has dimension 2×1, and their product is a scalar monthly return. Set the state transition to $F = I_2$, so alpha and beta each follow a random walk around their previous levels. The example program uses fixed parameters:

Parameter	Value	Meaning
Initial mean $\mathbf{m}_{-1}$	$[0, 1]^\top$	Initial alpha is 0 and beta is 1
Initial covariance P_{-1}	$\mathrm{diag}(0.0004, 0.04)$	Initial state variances for alpha and beta
Transition F	I_2	Alpha and beta means continue as random walks
Process covariance Q	$\mathrm{diag}(10^{-6}, 10^{-5})$	Monthly state-change variance
Observation variance R	0.0004	Residual variance of monthly decimal returns

These are teaching values specified in advance; the program does not estimate them from the full sample.

β_t is the asset's first-order linear exposure to excess market returns. If the asset position has dollar notional A, the target short notional in the market proxy is $\beta_t A$. If you hold q_A shares of the asset at price P_A and the market proxy trades at price P_M, both are quoted in the same currency and use the same return interval, then the number of market-proxy shares is

$$q_M = -\beta_t \frac{q_A P_A}{P_M}.$$

When $\beta_t > 0$, this formula establishes a short position in the market; when $\beta_t < 0$, it establishes a long position to offset the asset's negative market exposure. The actual position is also constrained by currency conversion, margin, basis, stock lending, execution timing, and costs. A beta estimate does not replace those implementation conditions.

The state components have different units: alpha is a monthly decimal return, and beta is a dimensionless exposure. Alpha variance is measured in return squared, beta variance in beta squared, and their covariance in return times beta. Estimating the alpha state is not a test of whether a strategy's alpha is statistically significant.

Worked example: a two-dimensional state and cross-covariance

The same update applies to a two-dimensional hand calculation. Let the prior mean be $\mathbf{m}^- = [0, 1]^\top$, prior covariance $P^- = \mathrm{diag}(0.0004, 0.04)$, design $\mathbf{h} = [1, 0.02]$, observation $y = 0.03$, and noise variance $R = 0.0001$. This is a teaching construction using monthly returns in decimal units. The prior observation mean is 0.02, and the innovation is $v = 0.01$. The cross-covariance between state and observation is

$$C = P^-\mathbf{h}^\top = \begin{bmatrix} 0.0004 \\ 0.0008 \end{bmatrix}, \qquad S = \mathbf{h}P^-\mathbf{h}^\top + R = 0.000516.$$

Thus, $K = C/S = [0.775194, 1.550388]^\top$, and the posterior mean is

$$\mathbf{m}^+ = \mathbf{m}^- + Kv = \begin{bmatrix} 0.007752 \\ 1.015504 \end{bmatrix}.$$

The same return innovation updates both alpha and beta. The posterior covariance is

$$P^+ = P^- - CC^\top/S \approx \begin{bmatrix} 0.0000899225 & -0.0006201550 \\ -0.0006201550 & 0.0387596899 \end{bmatrix}.$$

The negative off-diagonal element means that, under this prior and observation design, the errors in alpha and beta are correlated conditional on this return; it is not a standalone risk variance.

8. Calculating dynamic beta by hand: how one return updates the state

Worked example: one scalar beta update and its timing

The following is a teaching construction, not a market observation. The state contains only beta. The previous-period posterior mean is 1 and the posterior variance is $P_{t-1} = 0.04$. Set the random-walk transition to $F = 1$ and the process variance to $Q = 0.01$. This month's excess market return is $f_t = 0.02$, the asset's excess return is $y_t = 0.03$, and the observation-noise variance is $R = 0.0001$.

The prior beta mean is $m_{t|t-1} = 1 \times 1 = 1$, and the prior variance is

$$P_{t|t-1} = 1^2(0.04) + 0.01 = 0.05.$$

The prior return is $f_t m_{t|t-1} = 0.02$. Therefore, the innovation is

$$v_t = 0.03 - 0.02 = 0.01.$$

The scalar observation design is $h_t = f_t = 0.02$, so the innovation variance is

$$S_t = 0.02^2 \times 0.05 + 0.0001 = 0.00002 + 0.0001 = 0.00012.$$

The Kalman gain is

$$K_t = \frac{0.05 \times 0.02}{0.00012} = 8.333333.$$

The posterior beta mean is

$$m_{t|t} = 1 + 8.333333 \times 0.01 = 1.083333.$$

Check the posterior variance with the scalar update:

$$P_{t|t} = 0.05 - K_t(0.02)(0.05) = 0.041666667.$$

Beta and beta variance are measured in beta and beta squared, respectively. f_t, y_t are monthly decimal returns, and R, S_t are in squared decimal returns. The units of K_t are beta per decimal return. A value greater than 1 is entirely possible; it is not a probability. Only in special cases, such as a scalar observation and a state measured in the same units, is it natural to view the gain as a weight between 0 and 1.

The market return f_t is not fully known until this month's returns are realized, so $m_{t|t}$ in this example is a month-end risk estimate. If you need a beta hedge ratio at the start of the month, use the then-available $m_{t|t-1}$ and confirm that the market proxy and trading arrangements can be executed at that time.

State prediction and return prediction use different clocks

Timing	Quantity that can be computed	Quantity still unknown or unavailable for backfilling
Before the month	$F\mathbf{m}_{t-1}$, $FP_{t-1}F^\top + Q$	The current state shock and the full monthly asset and market returns
When the month-end market factor f_t arrives	$[1, f_t]\mathbf{m}^-$ using the realized design	This is not an asset-return forecast made before the month
When the month-end asset return arrives	The innovation, filtered state, and next-month plan	The filtered state cannot change the position already held this month
After the sample ends	A smoothed historical estimate using future observations	It cannot be a real-time signal for the target month; smoothing is not implemented here

To forecast an asset return before the month begins, you need to specify a conditional distribution for future f_t and account for its joint relationship with the state. You cannot substitute f_t, which is known only at month-end, and call the result a forecast made before the month.

Return beta, price-regression coefficients, and share counts also need separate interpretations. If another model uses $\log P_A = a + b \log P_B + e$, then with a, b held fixed the local relationship is $dP_A/P_A \approx b\, dP_B/P_B$, and the approximate local hedge share count is $-bq_AP_A/P_B$. This converts local price sensitivity; it does not mean setting $q_B = -bq_A$, nor does it automatically interpret a log-price cointegration coefficient as monthly-return beta. Finite price changes, changing coefficients, cash, and costs still belong in the actual ledger. This chapter retains the single-leg and two-dimensional state-estimation interfaces and does not change the Project 5 share-count protocol.

9. Python implementation and execution

Default standalone entry point

From the root of the bundled reader package, create a separate environment using its environment instructions, then run:

```
.venv/bin/python run_notebook.py notebooks/ch19_state_space.ipynb
```

First check the prior, innovation, gain, and posterior for one scalar update. Then compare the two-dimensional alpha/beta conditional Gaussian formula with the core implementation term by term. The two-dimensional example specifies the prior covariance directly, uses an identity transition, and sets Q to 0; do not confuse it with the dynamic process parameters in Section 7.

Next, use seed 20261019 to construct a sample starting in 1980-01 and spanning 480 months. Draw the initial state from a Gaussian distribution with mean [0.001,0.9] and covariance diag(0.0001,0.04); set Q=diag(0.000004,0.00004) and R=0.0004; draw market returns from a Gaussian distribution with mean 0 and standard deviation 0.04. Each period, transition the latent state first and then generate the observation. Comparing the true Q filter with a filter

whose beta process variance is zero gives one constructed path; retain the result in Section 18 where the misspecified filter has lower RMSE, and do not use it to recommend a model.

By default, the Notebook writes `constructed_latent.csv` and `constructed_filter.csv` to `runs/ch19_state_space_<uuid>/`. It displays the two simulated PNGs already included in `fixtures/figures/a4/` and does not generate new images. The other two French figures and historical numbers in Sections 10–11 are frozen chapter reading material, not default-run outputs.

`RUN_FRENCH=False`; French data are not read or downloaded by default. Importing the chapter workflow does not run its historical market analysis. The chapter run directory is separate from the runner's `runs/execution_<uuid>/`, which stores an execution copy and status. Assertions included in the package let readers check the constructed calculations; the production tests not included in the package do not need to be restored. Constructed results do not establish market predictability or a trading advantage after costs.

The CLI below follows the historical course-work root path and requires suitable, authorized data inputs and an environment; it does not replace the default command above.

Shared implementation and historical course-work path

The recursion is implemented by **kalman_filter** in the state-space core. The chapter script `code/ch19_state_space.py` reuses that function; it validates the frozen French data, runs a fixed-parameter analysis, and outputs tables and figures. The default Notebook above reuses the shared core and constructed-simulation branch, including the conditional-Gaussian check and hand-calculation assertions. The historical French analysis is handled separately by the CLI below.

The code below reproduces the example in Section 8; its outputs remain in beta and monthly decimal-return units:

```
import numpy as np
from quant_course.state_space import kalman_filter

result = kalman_filter(
    observations=np.array([0.03]),
    observation_design=np.array([[0.02]]),
    initial_state=np.array([1.0]),
    initial_covariance=np.array([[0.04]]),
    transition=np.array([[1.0]]),
    process_covariance=np.array([[0.01]]),
    observation_variance=0.0001,
)
assert np.isclose(result.predicted_covariances[0, 0, 0], 0.05)
assert np.isclose(result.innovations[0], 0.01)
assert np.isclose(result.innovation_variances[0], 0.00012)
assert np.isclose(result.gains[0, 0], 8.333333333333333)
assert np.isclose(result.filtered_states[0, 0], 1.0833333333333333)
assert np.isclose(result.filtered_covariances[0, 0, 0], 0.04166666666666667)
```

You can run the CLI from any directory by using an absolute path. Put outputs in a new directory to preserve existing data and figures. The script does not download French data and checks the SHA-256 hashes of the six raw, extracted, and processed files. Overwriting results with the same names requires the explicit –**overwrite** option.

```
python /path/to/quant_course/code/ch19_state_space.py \
  --output-dir /tmp/ch19-output \
  --figure-dir /tmp/ch19-figures
```

The core API in this chapter uses fixed F, Q, R and allows the observation design to vary by period; it does not implement general period-specific transitions and noise matrices. The core function requires one finite observation in every period, rejects missing values, and does not compress or skip months. The dynamic-beta helper also checks the monthly indexes of the two return series. If a data source is missing a month, first find out why, then choose a missing-data update that preserves the time step or revise the sample; missing-data updates are not implemented in this chapter's core.

10. Figures: viewing the state, innovation, and uncertainty

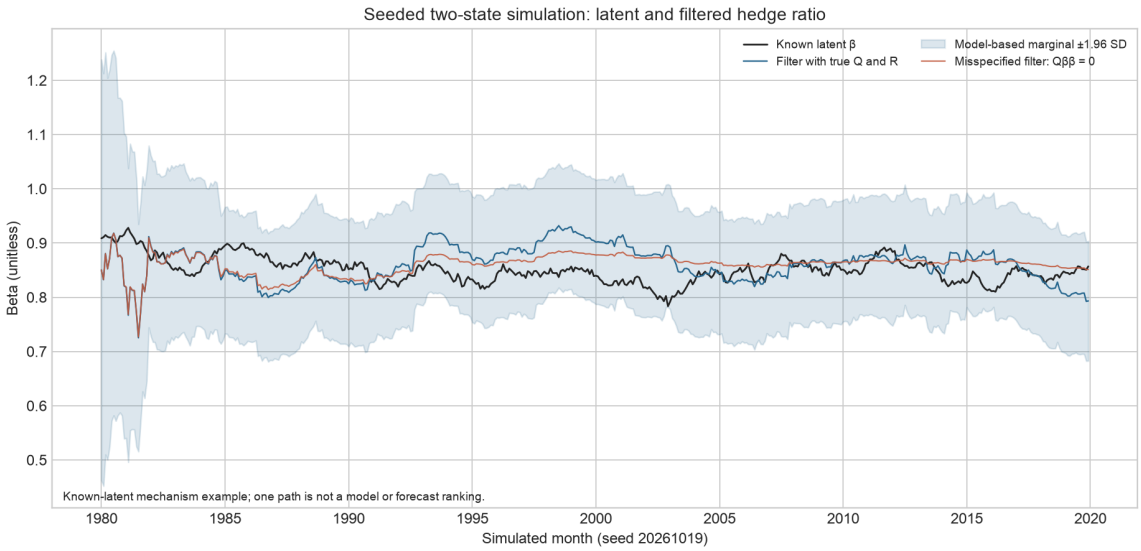


Figure 1: Known latent beta and two filtered estimates

In Figure 1, the horizontal axis is the simulated month generated with fixed seed 20261019, and the vertical axis is dimensionless beta. The black line is the known latent beta; the blue line is filtered beta using the true Q, R; the light-blue band shows the model's marginal ±1.96 standard deviations; and the red line comes from a filter that misspecifies the beta process variance as zero. The figure shows how filtering follows the state and why one simulated path cannot rank models universally.

Figure 2 uses simulated months on the horizontal axis. The top panel's vertical axis is monthly alpha in percent; the middle panel shows $v_t/\sqrt{S_t}$; and the bottom panel shows the beta gain $K_{\beta,t}$, in beta per return. The dashed lines at ±1.96 in the middle panel are reference lines

Figure 2: Simulated alpha, standardized innovation, and beta gain

under a Gaussian distribution, not a formal model test on their own; the gain in the bottom panel is not a probability.

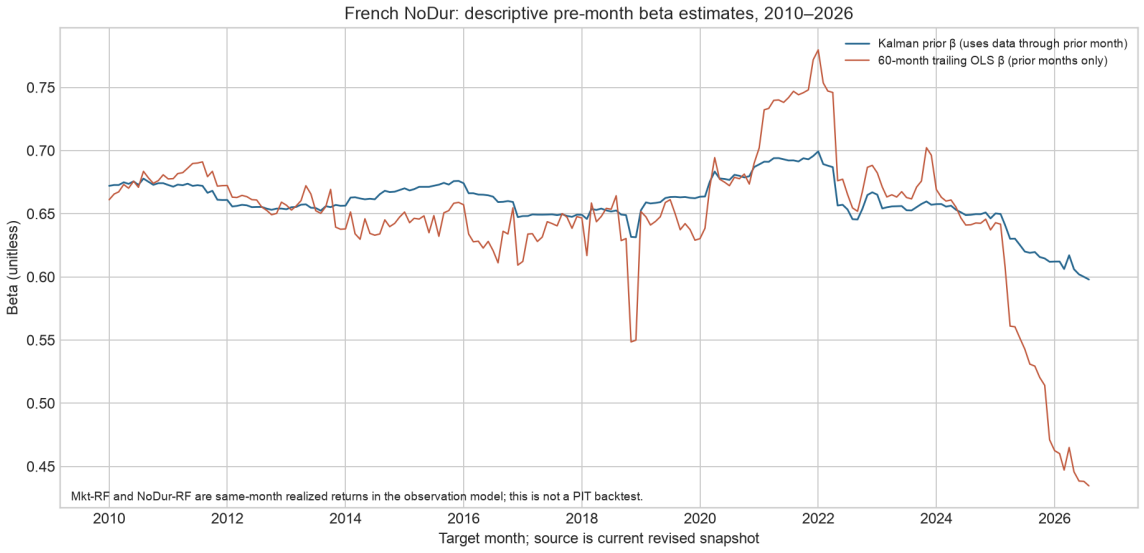


Figure 3: Prior beta and 60-month rolling OLS for French NoDur

Figure 3 uses target month on the horizontal axis and dimensionless beta on the vertical axis. Before each month begins, the blue line uses only earlier observations to obtain the Kalman prior; the red line is the OLS slope over the 60 months through the prior month. Both lines come

from the same currently revised history. Their difference shows how parameters and windows change the estimate; it does not show that one method makes more money.

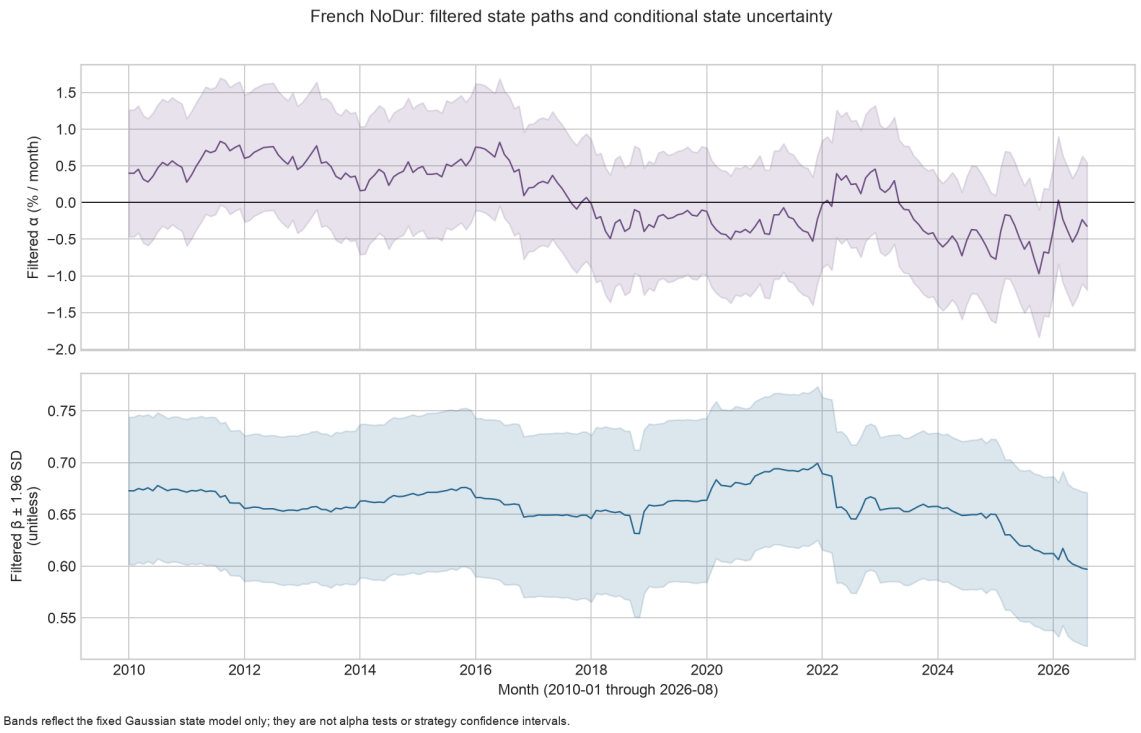


Figure 4: Filtered alpha/beta and model-conditional uncertainty for French NoDur

Figure 4 covers months from 2010-01 through 2026-08. The top panel's vertical axis shows filtered alpha (monthly percent) and its conditional marginal ±1.96 standard deviations; the bottom panel shows dimensionless filtered beta with the same type of interval. The shading reflects only the conditional state variance under the fixed Gaussian state model. It excludes parameter-selection uncertainty and is not an alpha-significance interval or a strategy-return interval.

11. Real-market example: the French NoDur industry portfolio

The example uses monthly returns for the 10 industry portfolios and the three-factor file from the Kenneth R. French Data Library. NoDur is an industry portfolio, not an individual stock. The asset input is the NoDur monthly total return minus RF; the market input is Mkt-RF, which already subtracts RF, so the code does not subtract RF again. Returns are in decimals over calendar months.

The frozen download date is 2026-10-05, and the sample runs from 1926-07 through 2026-08, for 1,202 months. The analysis checks monthly continuity with the monthly-series reader and verifies six hashes in the local manifests for the raw ZIPs, extracted CSVs, and processed CSVs. See the French factor manifest and industry-data manifest. The processed industry CSV has SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`; the FF3 CSV has SHA-256 `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`.

All initial states and P_{-1}, Q, R are teaching values; the program does not fit them on the full sample. The full-sample recursion is descriptive only and is not a point-in-time (PIT) backtest. Figures 3 and 4 show only the period from 2010-01 onward.

Month	Kalman prior beta	Month-end filtered beta	Prior 60-month OLS beta
2010-01	0.672096	0.672764	0.661122
2020-03	0.675173	0.683395	0.670480
2026-08	0.597956	0.596971	0.434646

By 2026-08, the month-end filtered beta is 0.596971, with a model-conditional standard deviation of about 0.03788. The prior beta for that month is 0.597956, and the rolling OLS beta is 0.434646. The filtered value and rolling slope use different state noise, windows, and shrinkage; this difference alone cannot tell us which estimate is closer to future exposure.

The data come from one currently revised version, without public timestamps or historical versions for each period. The analysis therefore makes no claim of point-in-time availability, a no-look-ahead strategy return, alpha outperformance, or actual hedge performance. The French Library page provides downloadable data; the local manifests do not record blanket permission for external redistribution. This chapter retains the source and hashes and does not repackage this snapshot.

12. Common mistakes

Mistake	Problem	Correction
Treating $K = 8.3333$ as a confidence probability greater than 1	The gain is measured in beta per return, not probability; the conversion ratio changes when the market return is small	Check the state units of Kv and the resulting update
Using filtered beta to place an order at the start of the same month	Filtered beta uses that month's asset and market returns	Use predicted beta before the month starts; update the next-period state after month-end
Treating a full-sample smoother as a historical signal	A smoothed state is conditional on observations after the target time	Use only filtered or predicted states for real-time signals
Subtracting RF a second time from Mkt-RF	The factor file already provides excess market returns	Subtract RF once from the NoDur total return only
Fitting Q, R and initial beta on the full sample	The parameters absorb future sample information, creating leakage even if the recursion itself is causal row by row	Estimate parameters in a training window and freeze them out of sample or update them by a prespecified rule
Dropping a missing month and continuing the recursion	The number of state transitions becomes misaligned	Preserve the monthly index and specify a missing-data policy; this chapter's function raises an error for a missing month or NaN
Treating a state interval as a test of alpha significance	The conditional variance includes only state uncertainty under the specified model	For an alpha test, handle parameter uncertainty, identification, and multiple selection separately

13. Quant research application: testing whether a dynamic hedge ratio improves risk control

A narrow hypothesis to test is whether, with a tradable market proxy and a prespecified month-end timestamp, a market-neutral portfolio formed using the beta prior available before the month begins reduces out-of-sample risk or tracking error. This is a risk hypothesis, not a claim that beta filtering predicts market direction.

1. Record the data source, release date, receipt time, return interval, and units. Confirm that the industry portfolio and market proxy can be traded at the specified weights.
2. Choose a training window. Use only training data to estimate or specify the initial alpha and beta mean and covariance $\mathbf{m}_{-1}, P_{-1}$, along with Q, R. Record the parameter choices, window, and selection rules, then freeze them for the test period.
3. At the end of period $t-1$, after receiving complete returns, save $\hat{\mathbf{x}}_{t-1|t-1}$ and use the state equation to generate the next period's beta prior. Under the random-walk model, the mean satisfies $\hat{\beta}_{t|t-1} = \hat{\beta}_{t-1|t-1}$.
4. Establish the hedge at an executable time and hold it through the month. At month-end, calculate the realized hedged-portfolio return from actual asset and market returns. Do not backfill the month-end filtered beta to the start of the month.
5. Using the same training information, compare static OLS beta with rolling OLS beta over the prior 60 months. Report unhedged and hedged volatility, tail loss, turnover, transaction costs, and execution constraints.
6. Repeat the process in later months. Check the mean, serial correlation, and tails of standardized innovations; test sensitivity to Q, R; and explain how conclusions change under model misspecification.

Specify in advance what evidence would reject the hypothesis. For example, reject the claim that "this dynamic hedging approach reduces achievable risk" if out-of-sample risk does not improve or if the improvement disappears after turnover and costs. If PIT data, a historical market proxy, or a cost model is missing, report descriptive risk paths at most.

14. How the pieces fit together

The state transition carries the prior-period posterior into the current-period prior; the observation design maps the state to visible returns; the innovation measures the gap between the current observation and its prior; the innovation variance measures uncertainty in that gap; the gain adjusts the state based on its covariance with the observation; and the updated covariance carries post-observation uncertainty into the next period. The Gaussian and independent-noise assumptions let these means and variances describe the full conditional posterior.

In finance, the beta state provides a dynamic estimate for current-period risk. Whether an estimate can inform an order depends on whether it was available before the order. Conditional variance, parameter estimation, and strategy risk represent different sources of uncertainty and should be reported separately.

15. Exercises

Complete answers to every question appear in Section 18.

19-E01 (Concept) In the two-dimensional alpha/beta model, identify the state, observation, design vector, state noise, and observation noise. Which item can only be observed after month-end?

19-E02 (Dimensions and units) State the dimensions and units of $\mathbf{x}_t, \mathbf{h}_t, Q, R$ in the alpha/beta model. Explain why the alpha-beta covariance is not the same as either the alpha variance or the beta variance.

19-E03 (By hand) Let the previous-period posterior have $m = 1, P = 0.04$, with transition $F = 2$ and state-noise variance $Q = 0.01$. The current observation design is $H = 1$, the return observation is $y = 2.10$, and the observation-noise variance is $R = 0.03$. Following this chapter's convention, transition from x_{-1} and calculate the current prior, innovation, S, K, posterior mean, and posterior variance.

19-E04 (Interpretation) The Kalman gain in Section 8 is greater than 1. Explain its units and identify the error in interpreting it as a probability.

19-E05 (Information timing) Let t be March. Monthly market and asset returns are not complete until the last trading day of March. When can $\hat{\beta}_{t|t-1}$ and $\hat{\beta}_{t|t}$ be used to form positions?

19-E06 (Assumptions) Market return f_t and the current state shock η_t are correlated because of the same event. Can the original conditional Gaussian update still be interpreted as the correct posterior? State what condition is missing.

19-E07 (Distribution) If state and observation noise still have zero means and known covariances but are heavy-tailed, which conclusions remain valid as linear-projection recursions, and which claims about a "full Gaussian posterior" must be withdrawn?

19-E08 (Parameter leakage) A researcher uses all observations from 1926–2026 to choose Q, R, then reruns the filter month by month. Identify where leakage occurs and describe an acceptable training/test treatment.

Added teaching questions (separately numbered)

19-R01 Precision derivation and boundary cases. Complete the square in a scalar Gaussian prior and observation density to write the posterior precision and mean. Why is the weight KH? What happens when H=0?

19-R02 Hand calculation comparing Q and R. Use the three rows in the Section 6 table to calculate each posterior variance. Explain why the same innovation produces different adjustments.

19-R03 Position and timing check. Hold 10 shares of an asset priced at 50, with the proxy priced at 100 and log-price coefficient b=1.2. How many shares are needed for the local hedge? If b is a filtered state at the end of the current month, can it determine the position for that month?

16. Programming exercises

19-P01 (Hand-calculation regression check) Use the core function to reproduce the Section 8 example. Assert the prior variance, innovation, innovation variance, gain, posterior beta, and posterior variance individually, with tolerance no greater than 10^{-10}.

19-P02 (Prefix causality) Construct 8 months of single-state data and save the filtered states for the first 5 months. Change only observations from month 6 onward and run the same filter with the same parameters. Assert that the first 5 filtered states are identical and that the final state in the changed segment differs.

19-P03 (Time boundaries) Give the dynamic-beta function a pair of continuous indexes with one month missing and a pair of monthly series containing NaN. Confirm that the function raises an error in both cases; do not drop the missing month and continue fitting.

17. Challenge questions

19-C01 (Unexpected ranking in a simulation) With fixed seed 20261019, use the accompanying script to generate 480 periods of latent alpha and beta. Compare the beta root-mean-square error (RMSE) and marginal model-interval coverage for a correctly specified filter using the true Q and another filter that sets beta's process variance to zero. If the misspecified filter has lower RMSE, can you recommend it? Explain the limits on extending the conclusion.

19-C02 (Research design) Design a point-in-time dynamic-beta hedging evaluation for a real asset. Specify data versions, timestamps, parameter estimation, static and rolling benchmarks, return accounting, costs, and failure conditions. Explain how to avoid using the same month's month-end filtered beta to backfill its position.

18. Exercise answers and derivations

19-E01

Let $\mathbf{x}_t = [\alpha_t, \beta_t]^\top$; monthly asset excess return y_t is the observation; $\mathbf{h}_t = [1, f_t]$ maps the state to returns; $\boldsymbol{\eta}_t$ allows alpha and beta to change over time; and ε_t is the deviation of realized returns from the state relationship. If f_t and y_t are both current-month returns, the complete observation is not known until the last trading day of that month. At the start of the month, you can form $\hat{\mathbf{x}}_{t|t-1}$ but cannot know this month's filtered state.

19-E02

$\mathbf{x}_t$ has dimension 2×1; $\mathbf{h}_t$ has dimension 1×2; Q has dimension 2×2; and R is a scalar. Alpha is measured in monthly decimal returns, while beta is dimensionless. Thus, alpha variance is in return squared, beta variance in beta squared, and $\mathrm{Cov}(\alpha, \beta)$ in return times beta. The off-diagonal matrix elements describe joint variation and cannot be treated as either diagonal variance.

19-E03

Apply the first transition:

$$m^- = 2(1) = 2, \qquad P^- = 2^2(0.04) + 0.01 = 0.17.$$

The expected observation is $1 \times 2 = 2$, so $v = 2.10 - 2 = 0.10$. Then

$$S = 1^2(0.17) + 0.03 = 0.20, \qquad K = \frac{0.17}{0.20} = 0.85.$$

The posterior mean is $2 + 0.85(0.10) = 2.085$. The posterior variance is

$$P^+ = 0.17 - 0.85(1)(0.17) = 0.0255.$$

The first prior mean is 2; skipping the transition would incorrectly give 1. This is why the first-step timing check uses $F = 2$ rather than $F = 1$.

19-E04

The numerator PH of the gain has units of beta times return, and the denominator S has units of return squared, so K has units of beta per return. After multiplying by the innovation v, Kv is in beta units. Treating K as a probability would incorrectly force it below 1 and obscure how market-return scaling affects the update.

19-E05

$\hat{\beta}_{t|t-1}$ is conditional on data through the end of February and can be used as the hedge weight before March begins, provided the asset and market proxy can be traded as planned. $\hat{\beta}_{t|t}$ includes both March returns and can only be used after March ends, to update the April plan. Using it to determine the March position would backfill future information from the same month.

19-E06

You cannot directly call this update the correct conditional posterior. Given the design, the derivation assumes that the state shock, observation error, and initial state are independent. If f_t and η_t are correlated, observing f_t may itself provide information about the state shock, so the prior mean and covariance conditional on that design must be recalculated; the original P^- cannot simply be reused. If the observation error is also correlated with the state, the cross-covariance terms must be rederived. Include the correlation mechanism in the joint conditional distribution, or rewrite the information set and observation equation, then derive the recursion again.

19-E07

With linearity, finite second moments, and correct covariances, the mean and covariance recursions can still be interpreted as linear projections and error-covariance calculations. Heavy tails do not ensure that the conditional distribution remains Gaussian, so these values cannot be claimed to describe the full posterior density; an interval of ± 1.96 standard deviations also does not automatically have 95% coverage. To assess the full distribution or tail risk, specify a heavy-tailed model and recalibrate.

19-E08

Leakage occurs in the model selection for Q, R: the researcher lets 2026 observations determine the filtering rule used for every step since 1926. First freeze the training cutoff and select parameters using training data. In the test period, apply the frozen values or re-estimate under a prespecified expanding-window rule, using only history available at each point. Determine the initial-state mean and covariance using the same boundary. A causal month-by-month recursion does not undo leakage from having seen future data during parameter selection.

19-P01 through 19-P03

Run the following Python program from the repository root with PYTHONPATH=src python -. The first block checks the hand calculation, the second changes only the future suffix, and the third checks missing-month and NaN boundaries.

```
import numpy as np
import pandas as pd
from quant_course.state_space import kalman_filter, dynamic_beta_filter

# 19-P01: Hand-check the scalar beta update.
manual = kalman_filter(
    [0.03], [[0.02]], initial_state=[1.0],
    initial_covariance=[[0.04]], transition=[[1.0]],
    process_covariance=[[0.01]], observation_variance=0.0001,
)
actual = [
    manual.predicted_covariances[0, 0, 0],
```

```
    manual.innovations[0],
    manual.innovation_variances[0],
    manual.gains[0, 0],
    manual.filtered_states[0, 0],
    manual.filtered_covariances[0, 0, 0],
]
expected = [0.05, 0.01, 0.00012, 8.333333333333333,
            1.0833333333333333, 0.04166666666666667]
np.testing.assert_allclose(actual, expected, rtol=0, atol=1e-12)

# 19-P02: Changes to future observations must not rewrite past filtered states.
y = np.arange(8, dtype=float) / 10
h = np.ones((8, 1))
kwargs = dict(
    initial_state=[0.0], initial_covariance=[[1.0]],
    transition=[[1.0]], process_covariance=[[0.02]],
    observation_variance=0.3,
)
original = kalman_filter(y, h, **kwargs)
changed_y = y.copy()
changed_y[5:] += 100
changed = kalman_filter(changed_y, h, **kwargs)
np.testing.assert_allclose(original.filtered_states[:5],
                           changed.filtered_states[:5], rtol=0, atol=0)
assert not np.allclose(original.filtered_states[5:],
                       changed.filtered_states[5:])

# 19-P03: Do not compress missing months or silently accept NaN.
months = pd.period_range("2025-01", periods=3, freq="M")
market = pd.Series([0.01, 0.02, -0.01], index=months)
asset = pd.Series([0.02, 0.01, 0.00], index=months)
parameters = dict(
    initial_beta=1.0, initial_variance=0.04,
    process_variance=0.01, observation_variance=0.0001,
)
for bad_asset, bad_market in [
    (asset.drop(months[1]), market.drop(months[1])),
    (asset.copy().where(pd.Series([True, False, True], index=months)), market),
]:
    try:
        dynamic_beta_filter(bad_asset, bad_market, **parameters)
    except (TypeError, ValueError):
        pass
    else:
        raise AssertionError("expected invalid monthly input to be rejected")
```

19-C01

The script uses a fixed seed to generate 480 periods, known latent states, and two sets of filtered results. The historical constructed run produced beta RMSE 0.0454210 for the filter using the true Q and 0.0364814 for the filter that sets $Q_{\beta\beta}$ to zero; the marginal 95% standard-deviation band for the true-parameter filter has coverage of 1.00 on this path. This path does not support the ranking "filtering with the true Q is better"; it gives the opposite RMSE ranking once.

You cannot use this to recommend $Q_{\beta\beta} = 0$. Error on a single path depends on the latent path, observation noise, and initial-state draw. RMSE measures recovery of the known latent beta, not net hedged returns. To compare methods, estimate the error distribution across multiple independent paths on a prespecified parameter grid, and report state error, interval coverage, response to turning points, and the specific risk objective separately. A simulation ranking still does not establish an investment advantage in the French data.

19-C02

An acceptable research plan should provide a checkable record for every timestamp:

- Record the data source, release version, availability time for each row, sample period, currency, return definition, missing-data treatment, and hash. Use price series that represent a tradable market; French factors can serve as a statistical benchmark but do not guarantee that a corresponding tradable product exists.
- Estimate or select $F, Q, R, \mathbf{m}_{-1}, P_{-1}$ in the training window and freeze them together with the window and hyperparameter grid. Also specify a static training-period OLS benchmark and a rolling OLS benchmark over the prior 60 months. During testing, read only versions available at the time.
- At the end of period $t-1$, calculate and record $\hat{\beta}_{t|t-1}$. Hold the hedge at this weight from the next month's first executable time. Record actual asset returns, market returns, and costs for month t in month t's P&L. The $\hat{\beta}_{t|t}$ computed only at month-end can affect positions in t+1 or later.
- Report unhedged and hedged returns, volatility, tail risk, beta residuals, turnover, stock-borrowing/financing/market-impact costs, and data and execution limits for each weighting rule. Tune parameters only within rolling training windows.
- Without PIT history, do not describe a currently revised history as a genuine historical backtest. If the market proxy cannot be traded at the target weight, or risk does not fall after costs, reject the claim of achievable risk improvement.

A suitable conclusion could be "This parameterization has better risk metrics than the benchmark in the available sample, but the difference is sensitive to Q and costs," or "We found no out-of-sample improvement." The score checks whether timestamps and data versions are clear (2 points), whether parameters and benchmarks are fair (2 points), whether returns and costs can be recomputed (2 points), and whether failure conditions were prespecified and the conclusion appropriately limited (2 points). The design fails if it has any one of these omissions: same-month filtered-beta leakage, treating a currently revised history as PIT, or omitting costs.

Full answers to the added teaching questions

19-R01. Given a scalar Gaussian prior with $P^- > 0, R > 0$, let

$$A = \frac{1}{P^-} + \frac{H^2}{R}, \qquad B = \frac{m^-}{P^-} + \frac{Hy}{R}, \qquad C = \frac{(m^-)^2}{P^-} + \frac{y^2}{R}.$$

The exponent of the prior density times the observation density is $-1/2$ times the following quadratic:

$$\frac{(x-m^-)^2}{P^-} + \frac{(y-Hx)^2}{R} = Ax^2 - 2Bx + C = A\left(x - \frac{B}{A}\right)^2 + C - \frac{B^2}{A}.$$

The linear coefficient in the quadratic is $-2B$; after multiplication by $-1/2$, the linear coefficient in the exponent is $+B$. The final two terms do not contain x and are absorbed into the normalizing constant. Comparing with the Gaussian density exponent $-(x-m^+)^2/(2P^+)$ gives

$$P^+ = \frac{1}{A} = \left(\frac{1}{P^-} + \frac{H^2}{R}\right)^{-1}, \qquad m^+ = \frac{B}{A} = P^+\left(\frac{m^-}{P^-} + \frac{Hy}{R}\right).$$

The first hand example gives $A = 24, B = 26, C = 29$, so the expanded expression is $24x^2 - 52x + 29$, the posterior variance is $1/24$, and the mean is $26/24$. $KH = H^2P^-/(H^2P^- + R)$ is a dimensionless convex weight, while K has state/observation units. When $H = 0$, $K = 0$, so the posterior equals the prior; do not use y/H. If $P^- = 0$, the prior is a degenerate point mass, and the inverse-precision derivation does not apply. With $R > 0$, the original covariance recursion still gives $K = 0, P^+ = 0, m^+ = m^-$.

19-R02. Calculate $P^+ = P^-R/(H^2P^- + R)$. The three results, in order, are 0.041666667, 0.071428571, and 0.047619048. Increasing Q makes the prior more uncertain in this example; increasing R makes the observation less precise. They increase and decrease KH, respectively.

19-R03. The local share count is -1.2×10×50/100=-6 shares; writing -12 shares confuses notional with share count. Local log-price sensitivity is not automatically return beta. A month-end filtered state can be used at a later executable time after the next state transition; it cannot be backfilled into the current month.

19. Further reading

- **Must Know**: Kalman (1960), read pp. 35–45 of the original paper for the problem statement, state recursion, and Theorem 3. The paper uses early notation and a specific measurement setup; the scalar observation-error formula in this chapter is derived from the conditional distribution of a joint Gaussian and does not attribute every modern implementation to that paper.
- **Must Know**: statsmodels 0.15.0, *Time Series Analysis by State Space Methods*. Read the matrix names and dimensions in the general state-space form, the lower-order state-space models, and the distinction between KalmanFilter and KalmanSmoother.
- **Should Know**: the local state-space core and chapter Notebook. Recalculate this chapter's examples in prior, innovation, and update order, then inspect the time-index validation.

- **Advanced**: the official statsmodels state-space chapters on custom models, initialization, likelihood, and result objects. This chapter does not include parameter-estimation or smoother code; as you read further, watch for differences between software conventions and this chapter's notation.

20. References and source scope

1. **CH19_KALMAN1960**: R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35–45, 1960. DOI: 10.1115/1.3662552. See source_notes.md for the reading scope, mirror notes, and claims supported by this source.
2. **CH19_STATSMODELS**: statsmodels 0.15.0, Time Series Analysis by State Space Methods. This chapter follows its model overview and maps the differing notation in the text.
3. **FRENCH**: Kenneth R. French Data Library, 10 Industry Portfolios and Fama/French Factors. The actual snapshots and input hashes are recorded in two local manifests; see source_notes.md for the reading scope.

Chapter 20 — Modern Portfolio Theory

Portfolio risk depends on weights and covariance. A mean–variance frontier is an exact mathematical object conditional on its input moments; estimated moments give an in-sample frontier. Prerequisites: Chapter 5 for matrices and definiteness, Chapter 9 for Lagrange multipliers and convex quadratic programming, Chapter 11 for simple/excess returns, Chapter 14 for CML/SML, and Chapter 15 for covariance and factor risk. You will derive portfolio variance, calculate two-asset portfolios, derive the GMV and target-return frontier, check tangency conditions, and distinguish population from estimated moments.

1. Learning goals

1. State portfolio mean and variance for n assets, including the dimensions, units, interval, and timing of $\mathbf{w}, \boldsymbol{\mu}, \Sigma$.
2. Explain how correlation changes a two-asset risk–return curve and find the threshold for a long-only portfolio to reduce risk below the least volatile asset.
3. Derive the global minimum variance (GMV) portfolio, exact-target minimum-variance portfolio, and frontier hyperbola with Lagrange multipliers; identify its inefficient branch.
4. Derive the tangency portfolio and optimal capital allocation line (CAL) under a single-period model with equal borrowing/lending rates; check the positive normalization denominator.
5. Interpret two historical French 10-industry estimated frontiers and extreme weights as in-sample descriptions.

Check the hand calculations against Section 9, retain complete solutions in Section 18, and identify axes, units, and samples in each figure. The inherited 13 core questions, two C10 bridge questions, and new 20-B01 all retain answers.

2. Why a quant needs portfolio theory

Chapter 11 measures single-asset returns and volatility; Chapter 14 discusses CAPM pricing of systematic risk. Allocating across assets raises three questions:

- Mean return is a weighted average of asset means.
- Risk is not a weighted average of asset volatilities. Assets with volatilities 15% and 25% can give an equal-weight volatility of 5% or 20%, depending on joint movement.
- Which weights deserve consideration under a risk budget, and which are dominated regardless of preference?

Modern portfolio theory (MPT) uses covariance to calculate risk and constrained optimization to select candidates. In the lifecycle, it follows signals/forecasts and precedes execution: upstream estimates of expected return and risk inform target weights.

Using weighted-average volatility misrepresents diversification. Calling the optimizer's output "the answer" ignores dependence on input moments. Chapter 21 and Project 3 handle estimation and implementation; here we develop the exact mathematics.

3. Intuition: correlation determines how much risk can offset

Take constructed annualized means 6%/10% and volatilities 15%/25%. Allocate w_1 to asset 1 and $1 - w_1$ to asset 2. Mean moves linearly between the two asset means; risk depends on correlation ρ.

Calculate an ordinary portfolio first. Set $\rho = 0.2$ and weights 50/50, fixed over one period. In annualized decimal units,

$$\begin{aligned}
\sigma_{12} &= \rho\sigma_1\sigma_2 = 0.2(0.15)(0.25) = 0.0075,\\
\Sigma &= \begin{pmatrix} 0.0225 & 0.0075 \\ 0.0075 & 0.0625 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix},\\
E[R_p] &= 0.5(0.06) + 0.5(0.10) = 0.08,\\
\mathrm{Var}(R_p) &= 0.5^2(0.0225) + 0.5^2(0.0625) + 2(0.5)(0.5)(0.0075)\\
&= 0.005625 + 0.015625 + 0.003750 = 0.025,\\
\sigma_p &= \sqrt{0.025} = 0.158114.
\end{aligned}$$

The first two contributions come from each asset's own variance; the third from joint movement. Two cross-products appear when expanding the square. Mean is 8% and volatility 15.81%, rather than the weighted-average 20%. A variance of 0.025 is not volatility 2.5%. Covariance has squared-return units, w is a two-by-one capital vector, and every matrix element uses the same frequency. Section 6.1 generalizes the expansion.

The horizontal axis is annualized volatility (%) and the vertical axis annualized expected return (%). This constructed curve restricts weights to 0–1. Locate the 50/50 point and the leftmost GMV. Points below the GMV mean have higher-return counterparts at equal risk. Section 8 calculates GMV and the 8% target. Optimality is conditional on the stated moments.

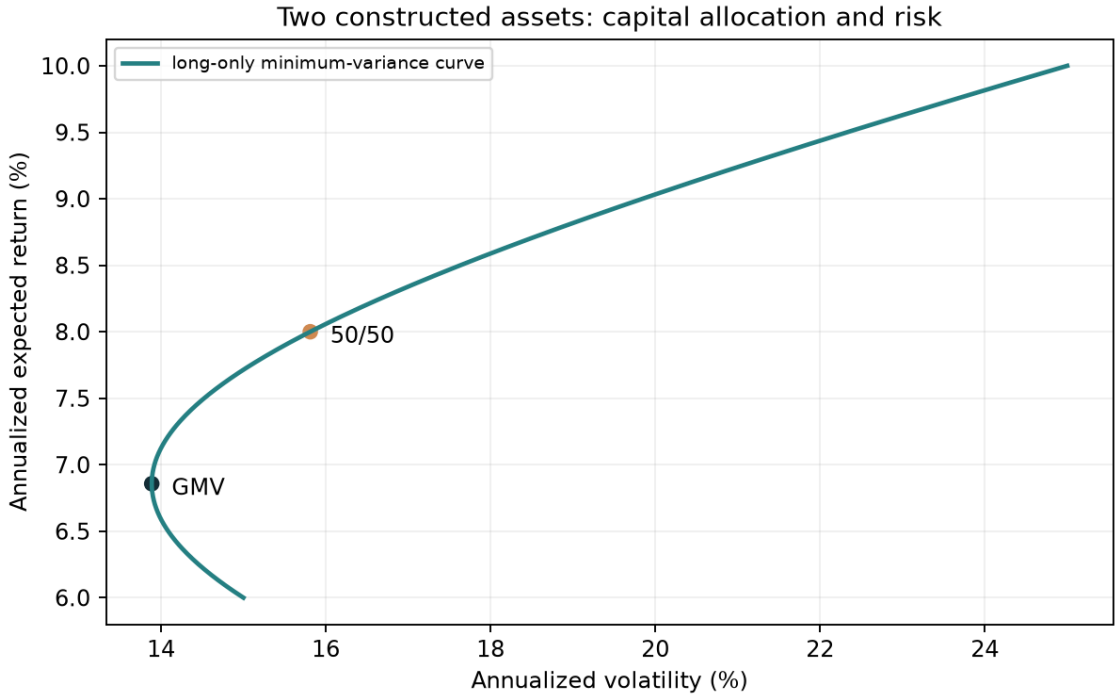


Figure 1: Constructed teaching inputs

Change only correlation:

Correlation	Lowest long-only volatility, $0 \le w_1 \le 1$	Interpretation
+1.0	15.00%, all asset 1	Joint movements leave no offset in the long-only set
+0.5	14.90%	A small offset
0.0	12.86%	Uncorrelated components reduce variance
−0.5	9.28%	Opposing movements allow more offset
−1.0	0.00%	A riskless combination within this pair

The chapter CLI calculated these teaching numbers. Correlation below 1 is not sufficient to reduce long-only risk below the safer asset. At 0.5, improvement is small; above $\sigma_1/\sigma_2 = 0.6$, the long-only minimum holds only asset 1. Section 6.2 derives the threshold.

The axes show annualized portfolio volatility and mean (%). Only ρ varies in Section 8's constructed inputs. Solid segments restrict w_1 to 0–1; dashed segments permit shorting. Moving from asset 1 toward asset 2 raises mean linearly; volatility may first fall and then rise. Lower correlation bends the curve left. At +1 the long-only segment is straight; at −1 it touches zero volatility. At +0.5 the leftmost point lies just left of asset 1, consistent with weak diversification near the 0.6 threshold.

Correlation measures linear co-movement over the specified horizon. It does not give the probability of joint tail crashes or ensure stability next period.

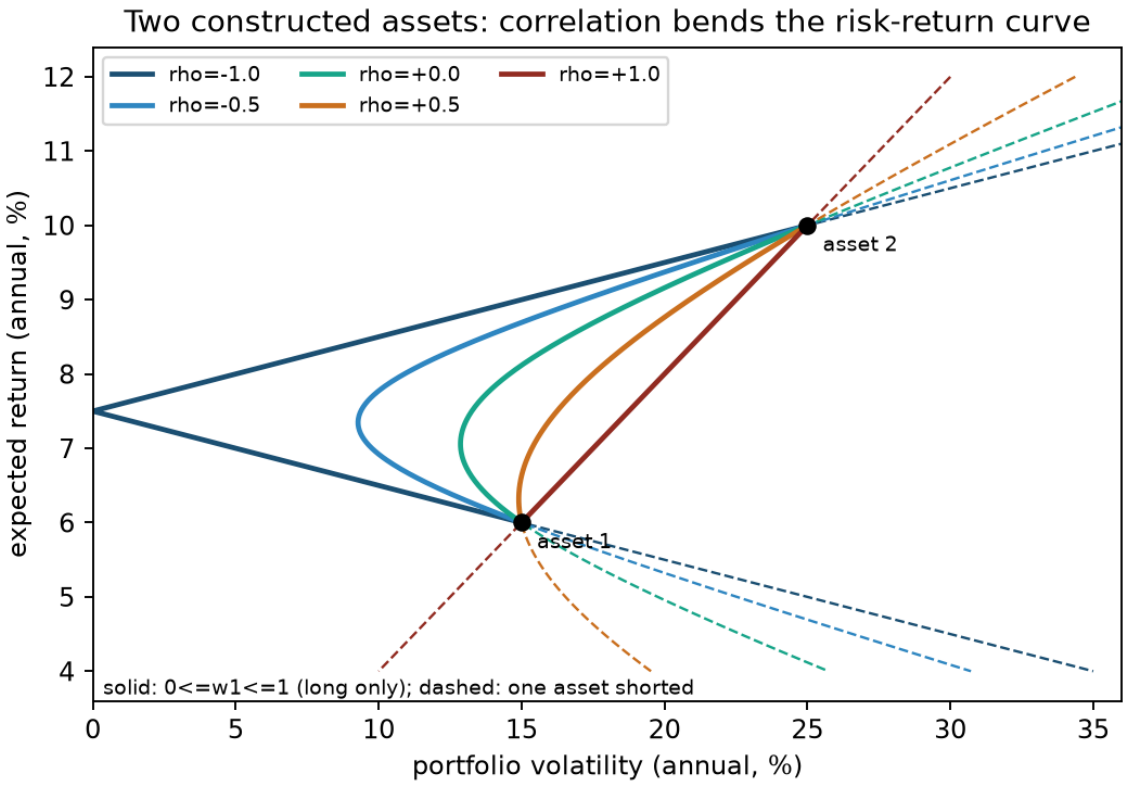


Figure 2: Constructed teaching inputs

4. Mathematical foundations and notation

Use a single holding period $[t, t+1]$. Set weights at t, then observe returns at $t+1$.

- n: number of risky assets.
- $\mathbf{R} \in \mathbb{R}^{n\times 1}$: simple returns in decimals; $R_i = 0.02$ means 2%.
- $\mathbf{w} \in \mathbb{R}^{n\times 1}$: invested amount divided by starting NAV; a negative weight means a short position, with proceeds accounted for elsewhere.
- $\mathbf{1} \in \mathbb{R}^{n\times 1}$: ones. $\mathbf{1}^\top w = 1$ allocates the full budget to risky assets.
- $\boldsymbol{\mu} = E[\mathbf{R}] \in \mathbb{R}^{n\times 1}$: expected decimal returns per period.
- $\Sigma = \mathrm{Cov}(\mathbf{R}) \in \mathbb{R}^{n\times n}$: entries $\sigma_{ij} = \mathrm{Cov}(R_i, R_j)$, in squared-return units; diagonals σ_i^2 are variances.
- r_f: a deterministic risk-free return for the same currency and period.

$w^\top$ is 1-by-n, $w^\top\mu$ a scalar, Σw n-by-one, and $w^\top\Sigma w$ a scalar.

Input conditions: one holding period

Keep frequency consistent across mean, covariance, and risk-free return. Constructed examples use annualized inputs; French calculations use monthly inputs. Multiplying monthly mean and covariance by 12 assumes IID returns for this reporting convention and ignores compounding. GMV weights do not change under proportional covariance scaling; tangency weights require matching r_f and mean frequencies.

A covariance matrix is symmetric and positive semidefinite (PSD): $w^\top \Sigma w = \mathrm{Var}(w^\top R) \geq 0$ for any w. The inverse-based formulas require positive definiteness (PD), so every nonzero direction has strictly positive variance and Σ^{-1} exists. Singular PSD matrices remain legitimate covariance matrices; only the inverse formulas fail.

4.1 A valid covariance can still be singular

Construct $R_1 = 0.06 + Z$ and $R_2 = 0.10 + Z$, with $E[Z] = 0$ and $\mathrm{Var}(Z) = 0.04$. Then

$$\Sigma = \begin{pmatrix} 0.04 & 0.04 \\ 0.04 & 0.04 \end{pmatrix}.$$

Eigenvalues 0.08 and 0 make it PSD but not PD. Every budget-one vector $(w, 1-w)$ has random component Z and variance 0.04, yet mean $0.10 - 0.04w$. A minimum variance exists, but all such vectors attain it: GMV is not unique. Failure to invert the matrix does not make portfolio mean or risk undefined.

Measure both legs against the same reference capital. The 0.04 below is deterministic PnL per unit of that reference, not return divided by zero net investment. The zero-budget direction $(1, -1)$ has constant return -0.04; reversing it gives $+0.04$. Under unrestricted, cost-free shorting, this constructed model contains an arbitrage direction. Long-only constraints prevent unlimited scaling. Do not interpret the toy model as an implementable trade. Adding a small diagonal term makes the matrix invertible by changing the risk input. Explain the estimation or regularization rationale instead of claiming the unique solution to the original problem.

If both assets instead have identical returns, the same covariance remains singular, but the zero-budget direction returns 0. Singularity alone therefore does not establish arbitrage. Check means and the feasible set too.

5. Definitions

Definition: portfolio return and expectation

Portfolio return and expected return:

$$R_p = \sum_i w_i R_i = w^\top R, \qquad \mu_p = E[R_p] = w^\top \mu.$$

The return identity is exact for beginning weights with no intraperiod trading: $V_{t+1} = \sum_i w_i V_t (1 + R_i)$, divided by V_t minus one. Do not substitute it for multiperiod compounding, intraperiod rebalancing, or log-return aggregation.

Covariance and portfolio variance:

$$\sigma_p^2 = \sum_i \sum_j w_i w_j \sigma_{ij} = w^\top \Sigma w, \qquad \sigma_p = \sqrt{\sigma_p^2}.$$

Section 6.1 derives the expression.

Diversification: variance reduction, at a fixed budget, from assets that do not move together perfectly. It does not mean an increase in return.

Feasible set and minimum-variance frontier: minimize variance for each target mean m, subject to $\mathbf{1}^\top w = 1$ and $\mu^\top w = m$. With unrestricted shorting, PD covariance, and unequal means, the frontier lies on the right branch of a hyperbola in (σ_p, μ_p) space.

GMV: the minimum-variance portfolio under the budget constraint alone, at the frontier's leftmost point.

Efficient frontier: the segment with mean at least the GMV mean. The lower branch is inefficient because the upper branch offers a higher mean at the same volatility.

Sharpe ratio: $S_p = (\mu_p - r_f)/\sigma_p$. Sharpe's 1994 discussion uses differential returns, representing a zero-investment strategy. With fixed r_f, the differential is portfolio return minus the risk-free return. Here the ratio uses population moments (ex ante); a historical estimate is ex post.

Risky and risk-free assets: the ideal model allows unlimited borrowing and lending at the same r_f, without margin, taxes, costs, or shorting restrictions. Risky weights w leave cash $1 - \mathbf{1}^\top w$; negative cash denotes borrowing.

Tangency portfolio: the budget-one risky portfolio maximizing Sharpe with positive excess mean, when it exists. Its line to r_f is the optimal CAL.

Population versus estimated moments: μ, Σ are parameters of the generating process; $\hat{\mu}, \hat{\Sigma}$ are historical-window estimates. "Optimal" refers to the input moments. An estimated frontier is an in-sample object; Section 11 and Chapter 21 address its relationship to future results.

6. Derivations

6.1 From a double sum to a quadratic form

Start with the variance definition,

$$\mathrm{Var}(R_p) = E[(R_p - \mu_p)^2].$$

Substitute $R_p - \mu_p = \sum_i w_i(R_i - \mu_i)$, then multiply out the finite sums:

$$(R_p - \mu_p)^2 = \Big(\sum_i w_i(R_i - \mu_i)\Big)\Big(\sum_j w_j(R_j - \mu_j)\Big) = \sum_i \sum_j w_i w_j (R_i - \mu_i)(R_j - \mu_j).$$

Beginning weights are constants. Linearity of expectation, with finite second moments, gives

$$\mathrm{Var}(R_p) = \sum_i \sum_j w_i w_j E[(R_i - \mu_i)(R_j - \mu_j)] = \sum_i \sum_j w_i w_j \sigma_{ij}.$$

There are n^2 terms: n own-variance terms and $n(n-1)$ off-diagonal terms. Each of the $n(n-1)/2$ distinct covariances appears twice. Since $(\Sigma w)_i = \sum_j \sigma_{ij} w_j$, left multiplication by $w^\top$ yields the same sum:

$$\sigma_p^2 = w^\top \Sigma w.$$

Weights have no units and covariance has squared-return units, so variance has squared-return units.

For two assets,

$$\sigma_p^2 = w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2w_1w_2\rho\sigma_1\sigma_2.$$

Only the cross-term contains correlation. It explains the change in curvature.

Equal weighting and asset count. For n assets with common volatility σ, common pairwise correlation ρ, and $w_i = 1/n$,

$$\sigma_p^2 = n\frac{\sigma^2}{n^2} + n(n-1)\frac{\rho\sigma^2}{n^2} = \frac{\sigma^2}{n} + \left(1 - \frac{1}{n}\right)\rho\sigma^2 \longrightarrow \rho\sigma^2.$$

At $\sigma = 20\%, \rho = 0.3$, volatility is 20% for $n = 1$, $\sqrt{0.004 + 0.0108} = 12.17\%$ for $n = 10$, and $\sqrt{0.012} = 10.95\%$ in the limit. Rewrite as $\sigma^2(1-\rho)/n + \rho\sigma^2$: the first component diversifies away; the common component remains. An equicorrelation matrix is PSD only if $-1/(n-1) \le \rho \le 1$. A fixed negative correlation cannot remain valid as $n \to \infty$, so the fixed-correlation limit requires $\rho \ge 0$. This is the portfolio calculation behind Chapter 14's systematic-risk discussion.

6.2 Two-asset minimum variance and the correlation threshold

Set $w_2 = 1 - w_1$ and differentiate:

$$\frac{d\sigma_p^2}{dw_1} = 2w_1\sigma_1^2 - 2(1-w_1)\sigma_2^2 + 2(1-2w_1)\sigma_{12}.$$

Setting this to zero and collecting w_1 gives

$$w_1^* = \frac{\sigma_2^2 - \sigma_{12}}{\sigma_1^2 + \sigma_2^2 - 2\sigma_{12}}.$$

The denominator is $\mathrm{Var}(R_1 - R_2)$. It is positive unless the return difference is almost surely constant. Unequal means with identical random components can still give zero denominator. A positive denominator gives a positive second derivative and a minimum.

Assume $\sigma_1 < \sigma_2$. Then $w_1^* < 1$ is equivalent to $\sigma_{12} < \sigma_1^2$, or

$$\rho < \frac{\sigma_1}{\sigma_2}.$$

The numerator is positive because $\sigma_2^2 - \sigma_{12} = \sigma_2(\sigma_2 - \rho\sigma_1) \ge \sigma_2(\sigma_2 - \sigma_1) > 0$. Thus the strict threshold gives $0 < w_1^* < 1$: both assets appear in the long-only minimum. In the example, the threshold is 0.6. Below it, minimum volatility falls below 15%; above it, $w_1^* > 1$, requiring a short in asset 2. The long-only optimum then holds only asset 1. At $\rho = 0.5$, the small improvement to 14.90% reflects proximity to the threshold.

6.3 GMV via Lagrange multipliers

Solve

$$\min_w \ \tfrac{1}{2} w^\top \Sigma w \quad \text{subject to} \quad \mathbf{1}^\top w = 1.$$

The factor $1/2$ simplifies differentiation without changing the solution. Define $L = \frac{1}{2} w^\top \Sigma w - \lambda(\mathbf{1}^\top w - 1)$.

1. Symmetry gives $\nabla_w L = \Sigma w - \lambda \mathbf{1}$.
2. The first-order condition is $\Sigma w = \lambda \mathbf{1}$. Marginal variance contributions $(\Sigma w)_i$ are equal. Moving capital toward a lower contribution would otherwise reduce variance.
3. PD covariance permits $w = \lambda \Sigma^{-1} \mathbf{1}$.
4. The budget gives $\lambda A = 1$, with $A = \mathbf{1}^\top \Sigma^{-1} \mathbf{1} > 0$, since the inverse is also PD.
5. Hence

$$w_{\text{GMV}} = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top \Sigma^{-1} \mathbf{1}}, \qquad \sigma^2_{\text{GMV}} = \lambda^2 \mathbf{1}^\top \Sigma^{-1} \mathbf{1} = \frac{1}{A}.$$

The objective is strictly convex and the constraint affine, so the feasible stationary point is the unique global minimum. Scaling covariance by any positive c scales its inverse by $1/c$, which normalization cancels. GMV weights therefore remain unchanged under proportional monthly/annualized covariance or decimal/percent conversion.

The implementation `global_minimum_variance_weights` solves $\Sigma z = \mathbf{1}$ and normalizes z, avoiding an explicit inverse.

6.4 Exact target return and the frontier hyperbola

Add target mean m:

$$\min_w \ \tfrac{1}{2} w^\top \Sigma w \quad \text{subject to} \quad \mathbf{1}^\top w = 1, \quad \mu^\top w = m.$$

With $L = \frac{1}{2} w^\top \Sigma w - \lambda(\mathbf{1}^\top w - 1) - \gamma(\mu^\top w - m)$, the first-order condition gives

$$\Sigma w = \lambda \mathbf{1} + \gamma \mu, \qquad w = \lambda \Sigma^{-1} \mathbf{1} + \gamma \Sigma^{-1} \mu.$$

Define

$$A = \mathbf{1}^\top \Sigma^{-1} \mathbf{1}, \quad B = \mathbf{1}^\top \Sigma^{-1} \mu, \quad C = \mu^\top \Sigma^{-1} \mu, \quad D = AC - B^2.$$

A has inverse-squared-return units, B inverse-return units, C is dimensionless, and D has inverse-squared-return units. Target m is a return; λ has squared-return units and γ return units. These constants summarize the input moments; they are not new observed data.

Substitute into both constraints:

$$\lambda A + \gamma B = 1, \qquad \lambda B + \gamma C = m.$$

This two-by-two system has determinant D. Cramer's rule gives

$$\lambda = \frac{C - Bm}{D}, \qquad \gamma = \frac{Am - B}{D}.$$

In the inner product $\langle x, y \rangle = x^\top \Sigma^{-1} y$, Cauchy–Schwarz gives $B^2 \le AC$, with equality precisely when μ is proportional to $\mathbf{1}$. If all means are equal, the two constraints repeat or conflict, and

no return choice remains. Unequal means give $D > 0$; the target-weight function rejects the equal-mean degeneracy.

Simplify variance using the first-order condition:

$$\begin{aligned}\sigma_p^2 &= w^\top \Sigma w \\ &= w^\top(\lambda \mathbf{1} + \gamma\mu) && \text{first-order condition} \\ &= \lambda + \gamma m && \text{budget and target} \\ &= \frac{Am^2 - 2Bm + C}{D} && \text{substitute and collect.}\end{aligned}$$

Variance is quadratic in m, producing a hyperbola in (σ_p, m). Minimizing over m gives $m = B/A$ and variance $(C - B^2/A)/D = 1/A$, agreeing with GMV.

- The upper branch $m \geq B/A$ is efficient.
- The lower branch $m < B/A$ is inefficient. Reflection to target $2B/A - m$ gives equal risk and higher mean. The target-weight function returns either branch; check efficiency yourself.

Two-fund separation. $w(m)$ is affine in m. Affine combinations of two distinct frontier portfolios, with coefficients summing to one, remain on the minimum-variance frontier. Allowing coefficients outside 0–1 generates the whole frontier. Targets below B/A generate its inefficient lower branch, rather than efficient portfolios.

Two assets. With $n = 2$, the two equality constraints uniquely determine weights; optimization reduces to solving equations. For $n \geq 3$, $n - 2$ free dimensions remain for variance minimization.

6.5 Cash, tangency, and the CAL

Risky weights need not sum to one. Cash $1 - \mathbf{1}^\top w$ earns r_f, giving return $r_f + w^\top(R - r_f\mathbf{1})$. Define $e = \mu - r_f\mathbf{1}$. For target excess return $\epsilon > 0$, solve

$$\min_w \ \tfrac{1}{2} w^\top \Sigma w \quad \text{subject to} \quad e^\top w = \epsilon.$$

Then

$$\begin{aligned}L(w, \kappa) &= \tfrac{1}{2} w^\top \Sigma w - \kappa(e^\top w - \epsilon), \\ \nabla_w L &= \Sigma w - \kappa e = 0.\end{aligned}$$

Symmetry gives the quadratic gradient; the constraint gradient is e. Both ϵ and multiplier κ have same-frequency return units; H below is dimensionless. Thus $w = \kappa\Sigma^{-1}e$, and substitution into the target gives $\kappa H = \epsilon$, where

$$H = e^\top \Sigma^{-1} e = C - 2r_f B + r_f^2 A.$$

PD inverse gives $H > 0$ for $e \neq 0$. Completing the square gives $H = A(r_f - B/A)^2 + D/A \geq D/A > 0$ when means differ. Therefore $\sigma_p^2 = \epsilon^2/H$, or $\epsilon = \sqrt{H}\,\sigma_p$. The optimal CAL begins at $(0, r_f)$ with slope $\sqrt{H}$, the maximum attainable Sharpe for this feasible set.

Call it the **capital market line (CML)** only with Chapter 14's CAPM equilibrium, market clearing, and the market portfolio as tangency fund. Constructed moment inputs do not establish

equilibrium, so our plots say CAL. Investors share direction $\Sigma^{-1}e$ and scale it by risk preference: Tobin-style two-fund separation.

Normalize the direction to a risky-only budget-one portfolio:

$$w_T = \frac{\Sigma^{-1}(\mu - r_f\mathbf{1})}{\mathbf{1}^\top\Sigma^{-1}(\mu - r_f\mathbf{1})}, \qquad \mathbf{1}^\top\Sigma^{-1}(\mu - r_f\mathbf{1}) = B - r_f A.$$

A positive denominator requires $r_f < B/A$, the GMV mean. Then $e^\top w_T = H/(B - r_f A) > 0$, and the fund lies at the upper-branch tangency. If $r_f > B/A$, the denominator is negative; normalized excess mean is negative and the fund lies on the lower branch with the lowest Sharpe. The optimal cash allocation instead shorts that fund. At $r_f = B/A$, the direction sums to zero and cannot normalize to a budget-one risky fund. The CAL still exists with slope $\sqrt{D/A}$. `tangency_weights` rejects nonpositive denominators.

Direct Sharpe maximization gives the same direction. For

$$S(w) = \frac{e^\top w}{\sqrt{w^\top\Sigma w}},$$

positive scaling leaves the ratio unchanged (degree-zero homogeneity). Its stationary condition is

$$e\sqrt{w^\top\Sigma w} - \frac{e^\top w}{\sqrt{w^\top\Sigma w}}\Sigma w = 0,$$

so $\Sigma w \propto e$. The ratio itself is not concave. With long-only constraints, Chapter 9 Section 7 and 9-C01 establish the convex-QP substitution $y = w/(e^\top w)$, conditional on positive excess return and homogeneous rewriting of weight caps. Chapter 21 implements it for rolling portfolios.

List the ideal assumptions: one period, unlimited equal-rate borrowing/lending, no costs/taxes/margin/shorting restrictions, common moments across investors, and divisible assets. If borrowing costs more than lending earns, the two rates generally give different tangencies. Between them, an investor may follow a segment of the risky efficient frontier, with lending/borrowing lines outside. Do not draw two lines joined at a presumed common tangency. Different investors may hold different risky funds.

6.6 Long-only KKT and why clipping is not optimization

Adding $w \geq 0$ replaces the general closed form with numerical quadratic programming (QP). For long-only GMV, Chapter 9's Karush–Kuhn–Tucker (KKT) conditions require λ and $\nu \geq 0$ such that

$$\Sigma w = \lambda\mathbf{1} + \nu, \qquad \nu_i w_i = 0.$$

Positive holdings have $\nu_i = 0$ and equal marginal variances. A zero-weight asset has marginal variance no lower than λ; buying it in exchange for a held asset does not reduce variance at the optimum.

Setting negative unrestricted GMV weights to zero and rescaling typically violates these conditions. Removing shorts changes hedges and leaves unequal marginal variances among the surviving assets. In Section 11, the 2000–2009 clipped portfolio has monthly volatility 3.42% versus 3.34% for the constrained QP. The CLI uses Chapter 9's `solve_mean_variance` for the long-only frontier.

7. Financial interpretation

Covariance determines marginal portfolio variance. Asset i's contribution to the current portfolio depends on $(\Sigma w)_i = \mathrm{Cov}(R_i, R_p)$, rather than its standalone variance. This describes risk; it does not establish risk compensation. CAPM equilibrium in Chapter 14 connects market covariance to expected excess return.

The frontier does not choose a point for you. It excludes dominated portfolios. Risk aversion or a risk budget chooses among efficient points, as in Chapter 9's utility $w^\top \mu - \frac{\gamma}{2} w^\top \Sigma w$.

Sharpe has a scope. It reduces mean and volatility to one number and suits portfolios that can be scaled with risk-free borrowing/lending. A higher-Sharpe fund then offers a higher mean at any volatility. Leverage limits, heavy tails, and skewness require additional drawdown/tail measures (Chapters 32–33).

Means choose direction; covariance determines hedging. $\Sigma^{-1}e$ is linear in excess means. The inverse amplifies mean differences in directions with low variance, including differences between highly correlated assets. Small mean-estimation changes can create large opposing positions; Section 11 illustrates extreme weights.

Implementation matters. Known population moments and unlimited cost-free financing/shorting are idealizations. Financing spreads, borrowing fees, margin, turnover costs, and estimation error affect the apparent Sharpe. Chapter 21 addresses them.

8. Numerical example: two assets

Worked example: a two-asset portfolio in five steps

These are constructed annualized decimal inputs:

$$\mu = \begin{bmatrix} 0.06 \\ 0.10 \end{bmatrix}, \quad \sigma_1 = 0.15,\ \sigma_2 = 0.25,\ \rho = 0.2, \quad \Sigma = \begin{bmatrix} 0.0225 & 0.0075 \\ 0.0075 & 0.0625 \end{bmatrix}, \quad r_f = 0.02.$$

Covariance is $0.2(0.15)(0.25) = 0.0075$.

Step 1: GMV.

$$w_1 = \frac{0.0625 - 0.0075}{0.0225 + 0.0625 - 2(0.0075)} = \frac{0.055}{0.070} = \frac{11}{14} = 0.785714, \quad w_2 = \frac{3}{14} = 0.214286.$$

Mean is $0.06(11/14) + 0.10(3/14) = 0.068571$. Variance is

$$\begin{aligned} \sigma_G^2 &= (11/14)^2(0.0225) + (3/14)^2(0.0625) \\ &\quad + 2(11/14)(3/14)(0.0075) \\ &= 0.013890 + 0.002870 + 0.002526 = 0.019286. \end{aligned}$$

Volatility 13.887% is below 15%, consistent with correlation 0.2 below 0.6.

Step 2: frontier constants. $\det \Sigma = 0.0225(0.0625) - 0.0075^2 = 0.00135$, so

$$\Sigma^{-1} = \frac{1}{0.00135} \begin{bmatrix} 0.0625 & -0.0075 \\ -0.0075 & 0.0225 \end{bmatrix}.$$

Continued: the same complete example

$\Sigma^{-1}\mathbf{1} = [0.055, 0.015]^\top/0.00135 = [40.7407, 11.1111]^\top$, and $\Sigma^{-1}\mu = [0.003, 0.0018]^\top/0.00135 = [2.2222, 1.3333]^\top$. Thus

$$A = 51.8519, \quad B = 3.5556, \quad C = 0.06(2.2222) + 0.10(1.3333) = 0.26667, \quad D = 1.18519.$$

Check $B/A = 0.068571$ and $1/A = 0.019286$.

Step 3: target 8%. Budget and target give $0.06w_1 + 0.10(1 - w_1) = 0.08$, hence $w_1 = 0.5$. The frontier formula gives

$$\sigma^2 = \frac{51.8519(0.0064) - 2(3.5556)(0.08) + 0.26667}{1.18519} = \frac{0.331852 - 0.568889 + 0.266667}{1.18519} = 0.025.$$

Direct expansion $(0.25(0.0225)+0.25(0.0625)+2(0.25)(0.0075))$ agrees; volatility 15.81%. Displayed constants are rounded; use full precision for computation.

Step 4: tangency. $B - r_f A = 3.5556 - 1.0370 = 2.5185 > 0$. The direction is

$$\Sigma^{-1}(\mu - 0.02\mathbf{1}) = [2.2222 - 0.8148, 1.3333 - 0.2222]^\top = [1.4074, 1.1111]^\top.$$

Normalize to $w_T = (19/34, 15/34) = (0.558824, 0.441176)$. Mean 0.077647, variance 0.022889, volatility 15.129%, and Sharpe $(0.077647 - 0.02)/0.15129 = 0.38103$.

Step 5: check with H. $H = 0.26667 - 0.14222 + 0.02074 = 0.14519$; $\sqrt{H} = 0.38103$. A 10%-volatility CAL allocation holds $0.10/0.15129 = 66.1\%$ in the tangency fund and 33.9% cash, with mean $0.02 + 0.38103(0.10) = 5.81\%$.

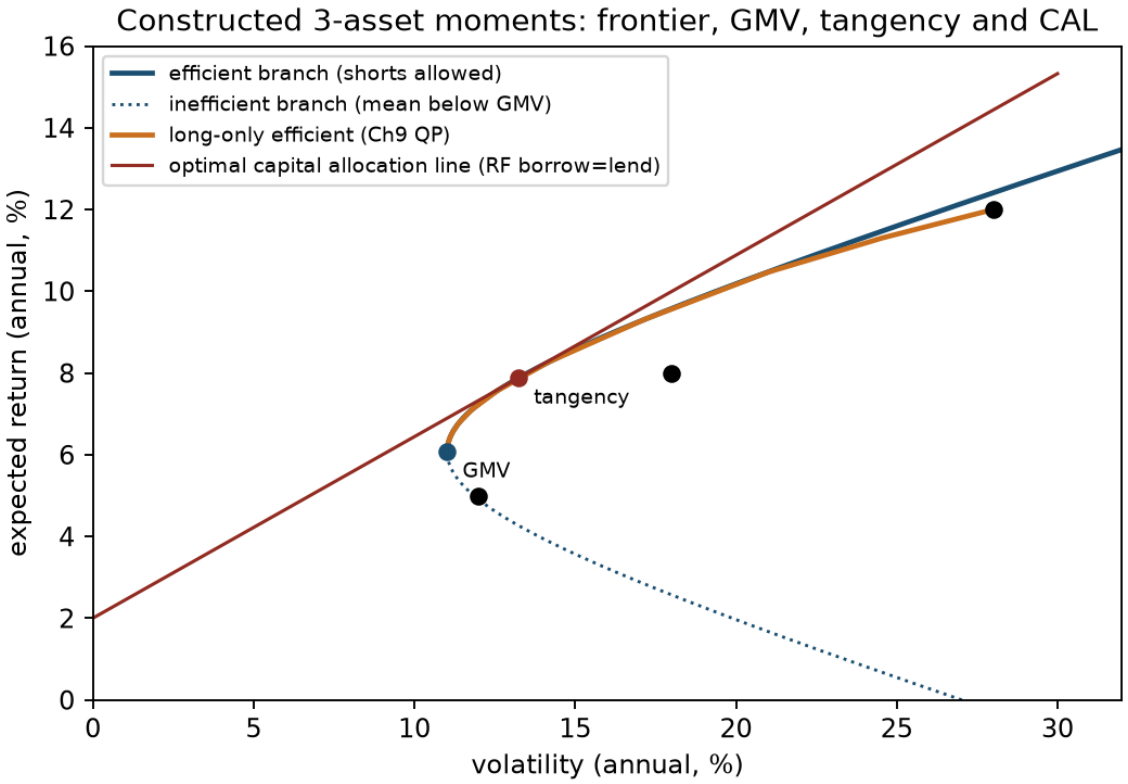


Figure 3: Constructed teaching inputs

The three-asset constructed inputs are annualized means 5%/8%/12%, volatilities 12%/18%/28%, correlations 0.3/0.1/0.4, and $r_f = 2\%$. Navy solid shows the unrestricted efficient frontier and dotted its inefficient lower branch; orange shows Chapter 9's long-only QP frontier; red shows CAL. GMV is leftmost, at volatility 11.02% and mean 6.07%. CAL touches at volatility 13.24%, mean 7.88%, Sharpe 0.444. At low risk, long-only and unrestricted frontiers coincide; when a weight reaches zero, the constrained frontier separates and lies inside the unrestricted opportunity set. It ends at the highest-mean single asset.

9. Python implementation

Default accepted-reader entry point

From the shared I3 reader root, after README.md setup, run:

```
.venv/bin/python run_notebook.py notebooks/ch20_portfolio_theory.ipynb
```

The constructed notebook does not read omitted French inputs or download data. Each run writes a fresh `runs/` directory. Historical CLI/test/data paths below refer to the course root. The French route requires authorized local data and manifests and was skipped in this batch. The portable I3 CLI defaults to a new UUID run and rejects overwriting. Stay at the reader root for its commands. See README.md and OPTIONAL_DATA.md; a constructed notebook check does not rerun historical market calculations.

`src/quant_course/portfolio_theory.py` supplies:

- `portfolio_moments`: mean, variance, volatility for given weights;
- `global_minimum_variance_weights`: budget-constrained GMV;
- `minimum_variance_target_weights`: exact-target frontier;
- `tangency_weights`: positive-denominator tangency fund.

Weight functions impose a budget but permit shorts/leverage and solve linear systems. Moment calculations reject nonsymmetric/non-PSD matrices; inverse-based weights additionally require numerical PD, and target weights require unequal means. Moment evaluation accepts singular PSD input. Long-only frontiers reuse `src/quant_course/optimization_tools.py` and Chapter 9's `solve_mean_variance`.

`code/ch20_portfolio_theory.py` recalculates Section 8, draws figures, and provides an optional descriptive French route. It reads local files, checks manifest SHA-256, and does not download. The portable default output is a fresh UUID's results/figures directories; specify fresh targets for the optional historical route:

```
.venv/bin/python code/ch20_portfolio_theory.py --output-dir /tmp/ch20-out --figure-
    ↪ dir /tmp/ch20-fig
```

`notebooks/ch20_portfolio_theory.ipynb` calls the shared functions and asserts results. Historical tests are `tests/test_ch20_portfolio_theory.py` and `tests/test_ch20_workflow.py`; they are not bundled as original tests in the reader. Use the course source and its evidence records for these paths.

The block below recalculates Section 8 in annualized decimals:

```
import numpy as np
from quant_course.portfolio_theory import (
    global_minimum_variance_weights, minimum_variance_target_weights,
    portfolio_moments, tangency_weights,
)

mu = np.array([0.06, 0.10])
cov = np.array([[0.0225, 0.0075], [0.0075, 0.0625]])
rf = 0.02

gmv = global_minimum_variance_weights(cov)
assert np.allclose(gmv, [11 / 14, 3 / 14])
m = portfolio_moments(gmv, mu, cov)
assert np.isclose(m.expected_return, 0.068571428571) and np.isclose(m.variance,
    ↪ 0.019285714286)

ones = np.ones(2)
A = ones @ np.linalg.solve(cov, ones)
B = ones @ np.linalg.solve(cov, mu)
C = mu @ np.linalg.solve(cov, mu)
D = A * C - B ** 2
assert np.isclose(B / A, m.expected_return) and np.isclose(1 / A, m.variance)

target = minimum_variance_target_weights(mu, cov, target_return=0.08)
assert np.allclose(target, [0.5, 0.5])
assert np.isclose(target @ cov @ target, (A * 0.08 ** 2 - 2 * B * 0.08 + C) / D)

tangent = tangency_weights(mu, cov, risk_free_return=rf)
assert np.allclose(tangent, [19 / 34, 15 / 34])
t = portfolio_moments(tangent, mu, cov)
sharpe = (t.expected_return - rf) / t.volatility
assert np.isclose(sharpe ** 2, C - 2 * rf * B + rf ** 2 * A)
print(round(sharpe, 5))  # 0.38103
```

The constructed model sets beginning weights for one common interval. French month labels denote calendar-month returns. Each window's moments use all 120 months, with no signal or fill time: these are historical descriptions. The loader rejects missing/duplicate months and nonfinite values instead of dropping rows.

10. Visualization

Read the correlation plot beside Section 3 and the three-asset/cash plot beside Section 8. The next plot shows historical changes in estimated moments.

Axes are monthly volatility (%) and monthly mean (%). Panels use 2000-01–2009-12 and 2010-01–2019-12,120 months each. Black dots are the 10 industries, navy the estimated unrestricted frontier, orange long-only, and red the in-sample tangency line. Industry locations change across windows: HiTec has negative mean in the first and the highest in the second. The first tangency lies beyond the plot, with monthly volatility above 9% and gross exposure about 5.8.

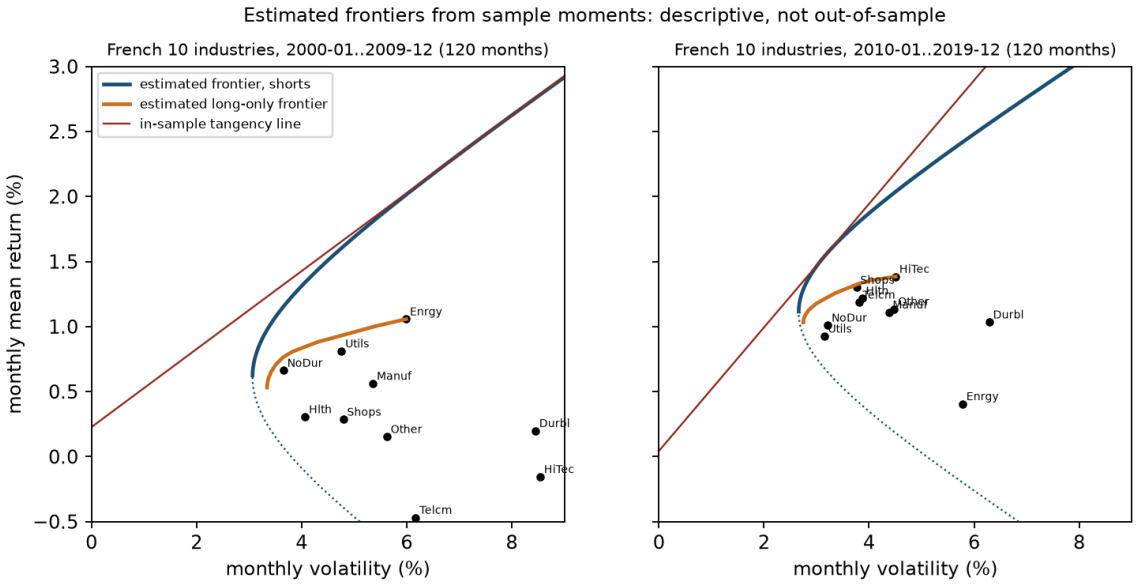


Figure 4: Historical estimated frontiers; inherited descriptive evidence

These plots describe what follows *if sample moments equal population moments.* They do not forecast the next window.

11. Real market example: estimated French 10-industry frontiers

The historical source is Kenneth R. French's value-weighted monthly total returns for 10 Industry Portfolios and RF from the FF3 file, downloaded and frozen 2026-10-05. The full source sample spans 1926-07–2026-08,1,202 months. Processed industry CSV SHA-256 is `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`. `src/quant_course/french_frozen.py` checks six raw/extracted/processed files against manifests. These are currently revised historical data without historical vintages or reception logs; they are not PIT data. Industry research portfolios are not tradable products with execution records. Raw data are omitted from the portable reader.

For each window, historical calculations used sample means, covariance with divisor $T-1$, and window-average RF. Monthly results were:

Window	Mean RF	GMV vol	Long-only min vol	1/N vol
2000-01–2009-12	0.227%	3.07%	3.34%	4.45%
2010-01–2019-12	0.043%	2.66%	2.75%	3.58%

Window	In-sample tangency Sharpe	Tangency gross exposure	Covariance condition number
2000-01–2009-12	0.300	5.84	86.8
2010-01–2019-12	0.475	2.80	101.1

Weights in percent:

Fund	NoDur	Durbl	Manuf	Enrgy
2000 s GMV	48.2	−5.5	−18.5	9.5
2000 s tangency	159.2	6.4	29.9	51.3
2000 s long-only min	40.6	0.0	0.0	2.4
2010 s GMV	21.7	−13.6	−13.4	−7.3
2010 s tangency	−20.0	−26.3	12.6	−43.4
2010 s long-only min	23.4	0.0	0.0	0.0

Fund	HiTec	Telcm	Shops	Hlth
2000 s GMV	0.6	−1.1	43.2	34.3
2000 s tangency	24.4	−122.9	51.0	−10.8
2000 s long-only min	0.0	0.0	13.0	31.0
2010 s GMV	3.4	1.2	13.8	11.5
2010 s tangency	23.4	28.8	39.2	7.0
2010 s long-only min	0.0	0.0	11.1	15.2

Fund	Utils	Other
2000 s GMV	14.0	−24.7
2000 s tangency	19.5	−108.2
2000 s long-only min	13.0	0.0
2010 s GMV	52.3	30.3
2010 s tangency	49.0	29.7
2010 s long-only min	49.7	0.6

Interpretation against theory:

- GMV and long-only minimum variance have lower in-sample volatility than 1/N in both windows:3.07%/3.34% versus 4.45%, and 2.66%/2.75% versus 3.58%. They use covariance without mean estimates. Persistence out of sample requires Project 3's design.
- Their large long holdings include low-volatility, lower-correlated industries such as NoDur, Hlth, Utils, and Shops. Held-asset marginal variances satisfy the relevant unrestricted or constrained conditions. Within a window, the constraint explains the difference between GMV and long-only minimum variance.
- Clipping and normalizing GMV gives 3.42%/2.88% volatility, above QP's 3.34%/2.75%.
- Tangency weights change sharply: NoDur +159% to −20%, Enrgy +51% to −43%. Their in-sample Sharpe exceeds each individual industry's, using the same data for estimation and evaluation.
- RF 0.227%/month exceeds the later window's 0.043%. Both lie below the corresponding GMV means 0.612%/1.109%, so both tangency denominators are positive.

These results show how fixed formulas respond to different historical windows. They do not evaluate earlier weights in the next window. That requires frozen rolling train/test rules and turnover/cost ledgers, as in Chapter 21 and Project 3. This section supplies no out-of-sample, after-cost, or tradability conclusion.

12. Common mistakes

Error	Consequence	Correction
Volatility $\sum_i w_i\sigma_i$	Ignores covariance; equality holds for perfectly correlated long holdings, while other long-only cases generally give an overestimate; shorts need separate calculation	Use $\sqrt{w^\top \Sigma w}$
Correlation below 1 guarantees improvement below the safer asset	Above σ_1/σ_2, the long-only optimum holds only the safer asset	Calculate the threshold
Calling the whole frontier efficient	Includes dominated lower-branch portfolios	Restrict to $m \geq B/A$
Monthly mean/covariance with annual r_f	Wrong excess mean/direction; denominator can change sign	Match periods
Clipped GMV called long-only optimal	Violates KKT;3.42% versus QP 3.34%	Solve constrained QP, check status/residuals
Normalize at $r_f \geq B/A$	Negative excess mean or zero denominator	Check $B - r_f A > 0$

Error	Consequence	Correction
Passing small monthly covariance directly to a solver	Historical CLARABEL output said optimal but exceeded the core's 10^{-6} budget tolerance	In a zero-mean problem scale covariance by its largest absolute entry; still check status and residuals
Present in-sample Sharpe 0.475 as attainable strategy Sharpe	Reuses estimation data for evaluation	Evaluate rolling out-of-sample, after costs

Runnable comparison:

```
import numpy as np
from quant_course.portfolio_theory import global_minimum_variance_weights,
    ↪ tangency_weights

cov = np.array([[0.0225, 0.0075], [0.0075, 0.0625]])
w = np.array([0.5, 0.5])
wrong_vol = w @ np.sqrt(np.diag(cov))          # Weighted-average volatility
right_vol = np.sqrt(w @ cov @ w)               # Portfolio volatility
print(round(wrong_vol, 4), round(right_vol, 4))  # 0.2 0.1581

mu_monthly = np.array([0.06, 0.10]) / 12
cov_monthly = cov / 12
annual_rf = 0.02
try:  # Monthly means with annual RF: denominator turns negative
    tangency_weights(mu_monthly, cov_monthly, risk_free_return=annual_rf)
except ValueError as error:
    print("rejected:", error)
fixed = tangency_weights(mu_monthly, cov_monthly, risk_free_return=annual_rf / 12)
assert np.allclose(fixed, [19 / 34, 15 / 34])
assert np.allclose(global_minimum_variance_weights(cov_monthly), [11 / 14, 3 / 14])
```

13. Quant research application

A falsifiable question is whether monthly rolling estimated tangency portfolios, under frozen windows/estimators/constraints, improve after-cost out-of-sample Sharpe over equal weight and minimum variance, which do not estimate means.

1. Record source, version, return definition, sample, missing-data rules, and hashes. Revised French industry data support bounded descriptive research without historical vintages.
2. At month $t-1$ end, estimate moments over $[t-W, t-1]$, set weights, and hold month t. Add a month's information lag as sensitivity if publication delay is relevant.
3. Freeze window length, covariance estimator, caps, cost rate, and fallback before evaluating. Do not tune on the out-of-sample period.
4. Separate drifted pretrade weights, turnover, costs, gross return, and net return.
5. Compare 1/N and minimum variance, plus ERC as a risk-contribution benchmark. Objectives, inputs, and feasible sets differ, so return differences do not isolate a causal contribution from mean estimation.
6. Report uncertainty intervals for return and Sharpe differences, for example paired block bootstrap; retain negative and zero results.

7. Lack of improvement after costs, or improvement restricted to isolated parameter settings, does not support usefulness.

13.1 From moments to capital: four interfaces

With NAV 1,000,000, a 10% target means 100,000 currency units. A fully invested risky portfolio satisfies $\mathbf{1}^\top w = 1$; cash equals $1 - \mathbf{1}^\top w$, with negative cash denoting financing. Section 6 permits shorts; Chapter 21's unconstrained BL/Kelly permits financing. Do not treat these as Project 3's fully invested long-only weights. Constraints define permitted holdings; objectives select among them.

Project 3's frozen monthly ten-industry specification uses:

Strategy	Objective/input rationale	Feasible set and fallback
Equal weight (EW)	10% each, no moment estimation; tests whether estimation/optimization adds value	Long-only, fully invested; predefined optimization fallback
Minimum variance (MinVar)	Minimize $w^\top \hat{\Sigma} w$, without mean forecasts	Long-only, full investment,40% per-asset cap; log failure and fall back to EW
Maximum Sharpe (MaxSharpe)	Training-window excess mean divided by predicted volatility; estimates both moments	Same 40% cap; no positive-excess feasible portfolio or solver failure triggers EW
Equal risk contribution (ERC)	Covariance only;10% volatility-contribution share per asset	Positive weights, full investment, no 40% cap; log Newton failure and use EW

The shared MinVar/MaxSharpe cap aids their comparison; ERC has a wider feasible set. Observed ERC weights below 40% do not establish a guaranteed cap. Strategy differences mix objectives, estimates, and constraints. Report net mean, Sharpe, volatility, drawdown, turnover, and concentration. Minimum variance need not maximize return; equal risk shares need not improve net Sharpe. Chapter 21 separates descriptions from difference intervals.

13.2 Two information clocks, one execution time

Month t names the holding return. This is a backtest clock, rather than a historical reception log:

Specification	W-month training interval	Decision/virtual fill	Holding interval
Frozen Project 3 lag 0	$[t-W, t-1]$, ends at $t-1$	End of $t-1$, assuming its data have arrived	End of $t-1$ to end of t
Teaching lag 1/project sensitivity	$[t-W-1, t-2]$, ends at $t-2$	Still end of $t-1$; one-month information buffer	Same holding month t

For January 1990 with $W = 60$, lag 0 uses January 1985–December 1989; lag 1 uses December 1984–November 1989. Both virtually rebalance at December 1989 end. Evaluation does not shift one month.

20-C02 adopts lag 1 as a conservative teaching convention. Project 3 retains idealized lag 0 at the month boundary and checks lag 1 sensitivity. A currently revised French snapshot has no contemporaneous publication versions or fill logs. Lag 1 changes the window; it does not prove historical arrival or tradability. It supports comparisons under a limited rolling specification, with historical availability and execution requiring separate evidence.

14. Chapter map

Covariance connects asset risks through $\sigma_p^2 = w^\top \Sigma w$. Minimizing under a budget gives GMV; adding a target mean gives the hyperbola, with the upper branch efficient. Equal-rate unlimited cash borrowing/lending gives a common risky direction and optimal CAL; a normalized positive-Sharpe tangency fund additionally requires $r_f < B/A$. The inverse-based weights require PD covariance; definitions of mean/risk require finite second moments and allow singular PSD covariance. Long-only constraints call for Chapter 9's QP.

Check input horizons and units, select the objective/feasible set, and treat estimated frontiers as in-sample relationships. Use out-of-sample and after-cost evidence to assess a portfolio decision (Chapter 21/Project 3).

15. Exercises

Full solutions follow in Section 18.

20-E01 (dimensions): For 10 monthly assets, give dimensions/units of $w, \mu, \Sigma, \Sigma w, w^\top \Sigma w$. Interpret $(\Sigma w)_i$.

20-E02 (variance expansion): $w = (0.5, 0.3, 0.2)$, volatilities 0.10/0.20/0.30, correlations $0.5/0/-0.2$ for 12/13/23. Write all nine terms and calculate variance/volatility.

20-E03 (two-asset GMV): Volatilities 0.20/0.30 and correlation 0.5. Calculate GMV weights, variance, volatility, and long-only feasibility.

20-E04 (threshold): Prove that with $\sigma_1 < \sigma_2$, both assets have positive long-only minimum-variance weights iff $\rho < \sigma_1/\sigma_2$. Above the threshold, can unrestricted GMV risk fall below σ_1?

20-E05 (limit): Equal-weight n assets with volatility 25% and common correlation 0.4. Calculate volatility for $n = 1, 5, 20, \infty$. Identify diversifiable/common variance.

20-E06 (lower branch): Use Section 8 constants for target 5%. Find weights/variance and a higher-mean frontier portfolio at the same risk.

20-E07 (tangency): For Section 8 assets with $r_f = 0.08$, calculate the denominator, normalized direction's weights/mean, and CAL slope. Classify the normalized fund.

20-E08 (estimation): Under IID, a monthly volatility 4.5% and 120 observations give what mean standard error? Compare it with industry monthly mean differences 0.1%–0.5% and explain changing tangency weights.

20-B01 (singularity and mean): For Section 4.1, calculate mean/variance for any budget-one weights, the minimum-variance solution set, and zero-budget direction's return. If both means become 6%, does singularity still establish arbitrage?

16. Programming exercises

20-P01: Write `frontier_constants(mu, cov)` returning A, B, C, D. Use Section 8's three-asset figure inputs and targets 4%/7%/11% to assert the frontier-variance formula and GMV mean B/A.

20-P02: With the authorized frozen French source for 2010-01–2019-12, calculate unrestricted GMV, clipped/normalized GMV, and Chapter 9 long-only QP. Assert QP volatility no greater than clipped volatility and report both monthly values.

20-P03: Prove invariance of GMV/target weights when mean×100, covariance×10000, target×100. Construct monthly moments with annual r_f, confirming rejection when the denominator is negative. Show a mismatched example with positive denominator that still returns weights; the function has no frequency metadata. Match frequencies and recover 19/34,15/34.

17. Challenge problems

20-C01: Prove $D > 0$ unless all means agree; prove an affine combination of two frontier points remains on the frontier; prove equality of the tangency slope and the line slope from (0, r_f). Check the three-asset example. Grade 2 points per proof and 2 for the numerical check.

20-C02: Design a test of rolling estimated tangency after-cost superiority to 1/N. Specify data, timing, windows, estimators, constraints, failure/fallback, cost ledger, inference, and falsification. Explain why in-sample Sharpe cannot establish it.

C10 bridge questions, with separate stable IDs

20-R01: For fully invested ten-industry EW/MinVar/MaxSharpe/ERC, identify moments and which methods carry a 40% cap. Why does slightly higher ERC net Sharpe not isolate an algorithm advantage under common constraints? At NAV 1,000,000, give EW amounts.

20-R02: January 1990 holding month, $W = 60$: give lag 0/lag 1 training endpoints, virtual fill, and holding interval. Does lag 1 prove contemporaneous French availability or industry-portfolio execution?

18. Solutions / Solution Hints

20-E01

w:10-by-one, dimensionless NAV fractions. μ:10-by-one, monthly decimal returns. Σ:10-by-ten, squared monthly returns. Σw:10-by-one, squared-return units. $w^\top \Sigma w$:scalar monthly variance, whose square root is monthly volatility. $(\Sigma w)_i = \sum_j \sigma_{ij} w_j = \mathrm{Cov}(R_i, R_p)$, half the partial derivative of variance with respect to w_i, or marginal variance contribution.

20-E02

Covariances are $\sigma_{12} = 0.010$, $\sigma_{13} = 0$, $\sigma_{23} = -0.012$. The nine terms are three diagonals: $0.25(0.01) = 0.0025$, $0.09(0.04) = 0.0036$, $0.04(0.09) = 0.0036$; two 12 terms of $0.15(0.010) = 0.0015$; two 13 terms of 0; two 23 terms of $0.06(-0.012) = -0.00072$. Sum $0.0025 + 0.0036 + 0.0036 + 0.0030 - 0.00144 = 0.01126$; volatility 10.61%. Weighted-average volatility 17% differs because correlations are imperfect, with a negative 23 contribution.

20-E03

$\sigma_{12} = 0.5(0.2)(0.3) = 0.03$.

$$w_1 = \frac{0.09 - 0.03}{0.04 + 0.09 - 0.06} = \frac{0.06}{0.07} = 0.857143, \quad w_2 = 0.142857.$$

Variance $0.734694(0.04) + 0.020408(0.09) + 2(0.857143)(0.142857)(0.03) = 0.029388 + 0.001837 + 0.007347 = 0.038571$, volatility 19.64%. Both weights are positive, as correlation 0.5 is below 0.667.

20-E04

Section 6.2 gives $w_1^* < 1 \iff \sigma_{12} < \sigma_1^2 \iff \rho < \sigma_1/\sigma_2$. Also $w_1^* > 0$ because $\sigma_{12} \leq \sigma_1\sigma_2 < \sigma_2^2$. The interior minimum is thus long-only iff the strict threshold holds, with risk below the safer asset. Above it, $w_1^* > 1$ shorts asset 2; the derivative at $w_1 = 1$ is $2(\sigma_1^2 - \sigma_{12}) < 0$, so increasing w_1 strictly reduces variance. At equality, GMV is asset 1. Borrowing, margin, and financing costs lie outside this model.

20-E05

$\sigma^2 = 0.0625$, common variance 0.025, and $\sigma_p^2 = 0.0625/n + (1 - 1/n)0.025$. At 1, variance 0.0625 and volatility 25%; at 5, $0.0125 + 0.02 = 0.0325$,18.03%; at 20, $0.003125 + 0.02375 = 0.026875$,16.39%; at infinity,0.025,15.81%. The diversifiable component $\sigma^2(1-\rho)/n$ vanishes; the common component $\rho\sigma^2$ remains.

20-E06

$0.06w_1 + 0.10(1 - w_1) = 0.05$ gives $1.25/-0.25$. The frontier variance is

$$\frac{51.8519(0.0025) - 2(3.5556)(0.05) + 0.26667}{1.18519} = 0.034375,$$

with full-precision constants, volatility 18.54%. Direct expansion $0.035156+0.003906-0.004688 = 0.034375$ agrees. Mirror target $2B/A-0.05 = 0.087143$ gives $w_1 = 0.32143$ and the same variance, with mean 3.71 percentage points higher. The 5% target is inefficient.

20-E07

$B - 0.08A = 3.5556 - 4.1481 = -0.5926 < 0$. No positive-excess, budget-one tangency fund exists, although optimal CAL with cash still exists. Direction $[-1.0370, 0.4444]$ normalizes to $1.75/-0.75$; mean $0.06(1.75) - 0.10(0.75) = 0.03 < 0.08$. It is on the lower branch and minimizes Sharpe. $H = 0.26667 - 0.56889 + 0.33185 = 0.02963$, so CAL slope is 0.1721. The positive-excess cash solution shorts that fund, corresponding to positive κ times the unnormalized negative-net-investment direction. The core function rejects this normalization.

20-E08

IID standard error $0.045/\sqrt{120} = 0.004108 = 0.4108\%$ per month. With serial dependence, use long-run variance, for example HAC, instead of mechanically dividing by $\sqrt{T}$. Errors of this size rival 0.1%–0.5% monthly mean gaps and can change rankings. $\Sigma^{-1}\hat{e}$ amplifies differences between correlated assets, causing large weight changes. NoDur's +159% to −20% illustrates instability across windows, without identifying its cause from two windows alone. Covariance also has estimation error; Chapter 21 discusses its typical relative precision.

20-B01

For $(w, 1-w)$, mean is $0.10-0.04w$ and variance 0.04. All real w minimize budget-constrained variance; the inverse formula is unavailable. Using equal reference-capital notionals, direction $1/-1$ has deterministic PnL per reference unit of -0.04 and its reverse $+0.04$. These do not divide by zero net investment. The reverse direction yields an arbitrage under unrestricted cost-free shorting. When both means are 6%, the direction returns 0 while covariance remains singular. Long-only constraints restrict $w \in [0, 1]$ and prevent unlimited scaling.

20-P01

```python
import numpy as np
from quant_course.portfolio_theory import global_minimum_variance_weights,
    ↪ minimum_variance_target_weights

def frontier_constants(mu, cov):
    ones = np.ones(len(mu))
    inv_one, inv_mu = np.linalg.solve(cov, ones), np.linalg.solve(cov, mu)
    A, B, C = ones @ inv_one, ones @ inv_mu, mu @ inv_mu
    return A, B, C, A * C - B ** 2

sigma = np.array([0.12, 0.18, 0.28])
corr = np.array([[1, 0.3, 0.1], [0.3, 1, 0.4], [0.1, 0.4, 1]])
cov = np.outer(sigma, sigma) * corr
mu = np.array([0.05, 0.08, 0.12])
A, B, C, D = frontier_constants(mu, cov)
for m in (0.04, 0.07, 0.11):
    w = minimum_variance_target_weights(mu, cov, target_return=m)
    assert np.isclose(w @ cov @ w, (A * m * m - 2 * B * m + C) / D)
g = global_minimum_variance_weights(cov)
assert np.isclose(g @ mu, B / A) and np.isclose(g @ cov @ g, 1 / A)
print(np.round([B / A, np.sqrt(1 / A)], 4))  # [0.0607 0.1102]
```

The assertions check each exact-target variance, GMV mean, and GMV variance using shared implementations.

20-P02

```
import numpy as np
from quant_course.french_frozen import load_industries_and_rf
from quant_course.optimization_tools import solve_mean_variance
from quant_course.portfolio_theory import global_minimum_variance_weights

returns, rf, _ = load_industries_and_rf()
cov = returns.loc["2010-01":"2019-12"].cov().to_numpy()
gmv = global_minimum_variance_weights(cov)
clipped = np.clip(gmv, 0, None)
clipped /= clipped.sum()
qp = solve_mean_variance(np.zeros(10), cov / np.max(np.abs(cov))).weights
vol = lambda w: float(np.sqrt(w @ cov @ w))
assert vol(gmv) <= vol(qp) <= vol(clipped)
print(round(100 * vol(clipped), 3), round(100 * vol(qp), 3))  # 2.876 2.751
```

QP satisfies equal marginal variance on positive holdings; clipping does not. Unrestricted GMV has the lowest volatility 2.661%, with shorts in Durbl, Manuf, and Enrgy. Historical displayed clipped/QP values are 2.876%/2.751%; this batch did not rerun the French input route.

20-P03

```
import numpy as np
from quant_course.portfolio_theory import (
    global_minimum_variance_weights, minimum_variance_target_weights,
    ↪ tangency_weights)

mu = np.array([0.06, 0.10]); cov = np.array([[0.0225, 0.0075], [0.0075, 0.0625]])
assert np.allclose(global_minimum_variance_weights(cov),
    ↪ global_minimum_variance_weights(cov * 10000))
assert np.allclose(minimum_variance_target_weights(mu, cov, target_return=0.08),
                   minimum_variance_target_weights(mu * 100, cov * 10000,
    ↪ target_return=8.0))
try:
    tangency_weights(mu / 12, cov / 12, risk_free_return=0.02)
    raise AssertionError("this example has a negative normalization denominator")
except ValueError:
    pass
assert np.allclose(tangency_weights(mu / 12, cov / 12, risk_free_return=0.02 / 12),
    ↪ [19 / 34, 15 / 34])

# No frequency metadata: another mismatched input can return weights.
wrong_but_accepted = tangency_weights(np.array([0.03, 0.04]), cov / 12,
    ↪ risk_free_return=0.02)
assert np.isfinite(wrong_but_accepted).all()
```

Monthly means 0.005/0.00833 are below annual $r_f = 0.02$, making this denominator negative. Use 0.02/12 for the matching period. Dividing means/covariance by 12 here illustrates consistency; it does not establish such a conversion for actual market moments. The other mismatched example returns finite weights because the numerical API lacks frequency metadata.

20-C01

(1) PD inverse defines $\langle x, y\rangle = x^\top \Sigma^{-1} y$. Cauchy–Schwarz gives $B^2 \le AC$, with equality iff $\mu = c\mathbf{1}$. Unequal means imply $D > 0$.

(2) $w(m) = \lambda(m)\Sigma^{-1}\mathbf{1} + \gamma(m)\Sigma^{-1}\mu$, with affine multipliers. For $m_a \neq m_b$ and any real a, $a\lambda(m_a) + (1-a)\lambda(m_b) = \lambda(am_a + (1-a)m_b)$, and likewise for gamma. Hence $aw(m_a) + (1-a)w(m_b) = w(am_a + (1-a)m_b)$, on the minimum-variance frontier. It may be on its inefficient branch.

(3) Differentiate $\sigma^2 = (Am^2 - 2Bm + C)/D$: $2\sigma\, d\sigma = (2Am - 2B)dm/D$, so $dm/d\sigma = D\sigma/(Am - B)$. At tangency,

$$m_T = \frac{C - r_f B}{B - r_f A}, \qquad \sigma_T^2 = \frac{H}{(B - r_f A)^2}.$$

For $B - r_f A > 0$, the line slope is $(m_T - r_f)/\sigma_T = \sqrt{H}$. Since $Am_T - B = D/(B - r_f A)$, the tangent slope also equals $D\sigma_T(B - r_f A)/D = \sqrt{H}$. The three-asset CLI gives slope 0.4442, mean 7.879%, volatility 13.237%; $(0.07879 - 0.02)/0.13237 = 0.4442$.

Grade 2 per proof and 2 for the numerical check; subtract 1 for omitting PD or positive denominator conditions.

20-C02

A complete plan includes:

- **Data:** source, version, hashes, total-return/currency convention, dates, missingness and delisting. French industry snapshots permit bounded descriptive conclusions.
- **Timing:** estimate only with received information; account for publication lag. Project 3 uses $[t - W, t - 1]$; this teaching lag 1 uses $[t - W - 1, t - 2]$. Both hold month t.
- **Prespecification:** W, sample/shrinkage covariance, mean estimator, shorts/caps, fees, fallback, all before evaluation. An example fallback is EW when no positive-excess feasible portfolio exists.
- **Ledger:** drifted pretrade weights, unhalved $\sum |\Delta w|$, cost, gross and net returns.
- **Evaluation:** out-of-sample net Sharpe, volatility, drawdown, turnover, concentration; compare 1/N/MinVar; paired block-bootstrap intervals for return and Sharpe differences; retain all cost/window sensitivity settings.
- **Falsification:** no after-cost improvement or improvement only under isolated settings does not establish superiority.

Teaching reference with an information lag. Freeze a 60-month rolling window, monthly rebalancing, long-only 40% cap, sample means and shrinkage covariance. Compare EW, MinVar, and MaxSharpe. For holding month t, use received data only through $t-2$; the extra month is a conservative convention, not a guarantee of publication by all suppliers. Fall back to EW when no positive-excess feasible portfolio exists and log the reason. Calculate turnover from drifted pretrade weights; report 0/10/25/50 bps fees with all sensitivities retained. Move forward through the evaluation period without retuning. Use a paired block bootstrap for net-return/Sharpe differences, plus concentration and drawdown. An advantage appearing only at

zero cost or a single window gives insufficient implementation evidence. The French snapshot exercises the workflow but does not supply historical publication, delisted-stock, or executable-price records.

In-sample Sharpe reuses data for choosing and evaluating weights, allowing the optimizer to exploit estimation noise. Grade 2 each for timing/data, prespecification/fallback, ledger/cost, inference/falsification. Full-sample estimation or shuffled K-fold fails this design.

20-R01

EW estimates no moments; MinVar/ERC estimate covariance; MaxSharpe also estimates excess means. All are long-only and fully invested; only MinVar/MaxSharpe impose 40%. ERC needs positive risk budgets and PD covariance in this implementation; observed weights below 40% do not ensure a cap. Objective, estimation, and feasible set differ, so a Sharpe ranking cannot isolate an algorithm effect; also report paired difference intervals. At NAV 1,000,000, EW targets 100,000 per asset; amounts drift with returns before rebalancing.

20-R02

Lag 0 trains 1985-01–1989-12; lag 1 trains 1984-12–1989-11. Both decide and virtually fill at December 1989 end, holding through January 1990 end. Lag 1 removes the latest month and adds an earlier one, keeping 60 months. It provides a buffer without establishing contemporaneous versions, reception, tradability, constituents, or capacity. Those require separate records.

19. Further reading

- **Must know:** Markowitz 1952, "Portfolio Selection," for E–V motivation, covariance/diversification, and residual common risk. Suggested reading 77–81 is broader than the recorded actual reading of 77–80's first part. The original excludes shorting; do not attribute this chapter's general n-asset closed forms to that article.
- **Must know:** MOSEK Cookbook Chapter 2, Sections 2.1–2.2, for mean–variance formulations and budget/leverage/turnover constraints.
- **Should know:** Sharpe 1994 for ex ante and ex post ratios and differential returns; Boyd and Vandenberghe for QP/KKT alongside Chapter 9.
- **Advanced:** Stanford CVX Short Course's portfolio example for numerical trade-offs and constraints; this recommendation does not claim a new reading.
- **Optional:** notebook/CLI in order: load, verify hashes, estimate, optimize, plot.

20. References

1. **CH20_MARKOWITZ1952:** Harry Markowitz, "Portfolio Selection," *Journal of Finance*7(1),77–91,1952. Berkeley mirror. Historical actual reading: extracted text on 77–80's first part; later geometric pages and scanned formulas were not fully inspected. Closed-form derivations here are author calculations.
2. **CH20_MOSEK_COOKBOOK:** *Portfolio Optimization Cookbook*1.6.0, Chapter 2. Historical record covers 2.1–2.2 and the start of 2.3;2.4 code was not run.
3. **SHARPE1994:** William F. Sharpe, "The Sharpe Ratio," *Journal of Portfolio Management*, Fall 1994. Author's page. Recorded reading covers differential returns, ex-ante/ex-post definitions, and aggregation without serial correlation, rather than the full fund-comparison discussion.
4. **BOYD:** Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*. Author's site. Inherited shared source.
5. **CVXPY:** Documentation. The historical chapter used Chapter 9's CLARABEL path and did not add a document reading.
6. **FRENCH:** 10 Industry Portfolios and Fama/French Factors. Historical source/manifest checks remain version-bound; no new download or empirical run occurred in BIL01.

Local shared references: source notes, accepted reader setup, optional data contract, shared script, shared notebook, Chinese Chapter 5, Chinese Chapter 9, Chinese Chapter 11, Chinese Chapter 14, Chinese Chapter 15.

Chapter 21 — Practical Portfolio Construction

Prerequisites: Chapter 9 on convex quadratic programming and KKT conditions, Chapter 15 on multi-factor risk models, Chapter 18 on volatility estimation, and Chapter 20 on modern portfolio theory and the unconstrained analytical solution for the mean–variance tangency portfolio. After completing this chapter, you will be able to estimate covariance matrices from a training window using Ledoit–Wolf shrinkage and check conditional positive definiteness and out-of-sample risk error; transform long-only maximum-Sharpe optimization, whose objective is not concave, exactly into an equivalent convex quadratic program (convex QP); solve for an equal risk contribution (ERC) portfolio; combine market equilibrium and subjective views with the Black–Litterman model; derive the Kelly criterion's log-growth rate and explain fractional-Kelly constraints; calculate asset drift, turnover, and trading costs; and objectively evaluate portfolio out-of-sample performance after costs.

1. Chapter Objectives

After studying this chapter, you should be able to:

1. Calculate estimation error in sample means and covariance matrices, and quantify why uncertainty in expected returns can damage optimized weights more severely than the covariance matrix;
2. Derive the optimal shrinkage intensity toward a scaled identity matrix in Ledoit–Wolf (2004), then use training-window data to compute a positive-definite covariance matrix for out-of-sample use;
3. Calculate two-sided rebalancing turnover, drifted portfolio weights, and trading-friction drag, and maintain an accounting ledger for gross and post-cost net returns;
4. Distinguish dollar-neutral, beta-neutral, and sector-neutral portfolios; explain how absolute sector neutrality across mutually exclusive and exhaustive sectors implies dollar neutrality but generally does not imply beta neutrality; and explain that Euclidean projection minimizes only weight distance;
5. Derive the Euler decomposition of portfolio volatility, calculate marginal risk contributions and relative risk shares, and solve for ERC weights using damped Newton's method with a Newton-decrement stopping rule;
6. Restate the long-only maximum-Sharpe variable substitution proved in Chapter 9, which converts the problem into a convex quadratic program, and implement a solution workflow with a feasibility precheck for weight caps;
7. Derive the Black–Litterman reverse-optimization prior, the block-Gaussian posterior mean and posterior-mean covariance, and verify the orthogonality property of view tilts;
8. Derive the binary-bet log-growth optimum under the Kelly criterion, its discrete second-order expansion, and its diffusion-limit quadratic proxy; analyze the tail-ruin mechanism under full Kelly and the risk–return trade-off under half Kelly;
9. Use a multi-factor risk model to decompose portfolio variance and volatility into factor and specific-risk contributions;

10. Using the French 10 industry historical snapshot and Project 3's 440-month rolling backtest, objectively evaluate each strategy's post-cost rolling out-of-sample performance and statistical uncertainty on the currently revised historical snapshot, without treating it as point-in-time (PIT) tradable evidence.

You have completed the chapter when you can independently reproduce all numerical examples and pass the code assertions, understand the assumptions and failure boundaries of every model, complete all exercises in Sections 15–17, and check the full derivations and code in Section 18.

2. Why Quants Need Practical Portfolio Construction

In Chapter 20's mean–variance theory (Modern Portfolio Theory, MPT), tangency-portfolio weights are given by the analytical expression $\mathbf{w}^* \propto \Sigma^{-1}\boldsymbol{\mu}_{excess}$. In live quantitative investing, however, directly substituting the historical sample mean $\hat{\boldsymbol{\mu}}$ and sample covariance matrix S into this expression (the plug-in estimator) usually produces extreme and unstable weights.

This failure arises from three unavoidable practical challenges.

First, **the inversion step sharply amplifies parameter-estimation error**. Financial time series are full of low signal-to-noise-ratio observations. The annualized standard error of an expected-return estimate is often on the same order as the mean itself, while the smallest eigenvalues of a high-dimensional covariance matrix are often close to zero. Inverting a noisy matrix greatly magnifies the directions associated with these small eigenvalues, leading the optimizer to take large long and short positions in directions with the most in-sample noise. In quantitative finance, this phenomenon is sometimes called an "error maximizer."

Second, **the theoretical tangency portfolio does not model real trading frictions or constraints**. The analytical solution assumes unrestricted short selling, no borrowing-rate penalty, no market-impact or liquidity cost, and no trading commissions. In institutional investing, however, managers face strict long-only rules, position limits, sector and market-risk budgets, and slippage caused by turnover. If these real-world boundaries are ignored, apparently strong paper returns can be eroded by monthly rebalancing costs.

Finally, **diversification is often misunderstood in practice**. A nominally equal-weighted allocation is not necessarily diversified by risk. If a portfolio contains highly volatile assets, a small number of them may dominate total portfolio risk. Likewise, a long–short portfolio described as "market neutral" may still carry substantial systematic market beta or sector-tilt risk if it satisfies only dollar neutrality.

The goal of practical portfolio construction is to use mathematically rigorous shrinkage estimation, convex-optimization transformations, risk budgeting, and Bayesian prior constraints to turn noisy forecasts into real holdings that are implementable out of sample, have controlled risk, and can withstand trading frictions.

3. Intuition: From Sample Noise to Implementable Holdings

To see why unconstrained optimization is fragile, consider a deliberately simple thought experiment.

Suppose the market has two assets, each with a true monthly excess return of 1.0%, monthly volatility of 4.0%, and a correlation of 0.7 (a constructed teaching example). The two assets are perfectly symmetric, so the true optimal tangency portfolio clearly holds 50% of each.

Now suppose a researcher estimates the inputs from the past 60 months. Due to sampling variation, asset 1's sample mean happens to be high at 1.3%, while asset 2's happens to be low at 0.7%. This small sampling perturbation of 0.3% is entirely plausible (the standard error is about $4.0\%/\sqrt{60} \approx 0.52\%$).

Substituting these estimates into $\Sigma^{-1}\boldsymbol{\mu}_{excess}$, the two assets' equal volatilities imply that Σ^{-1} is proportional to $\begin{bmatrix} 1 & -0.7 \\ -0.7 & 1 \end{bmatrix}$. The resulting direction is proportional to $(1.3 - 0.7 \times 0.7,\ 0.7 - 0.7 \times 1.3) = (0.81, -0.21)$, giving normalized weights of roughly (135%, −35%), even though the true optimum is (50%, 50%). Evaluated using the true parameters, both portfolios have expected excess returns of 1.0% per month, but monthly volatility is $4\% \times \sqrt{1.35^2 + 0.35^2 - 2(0.7)(1.35)(0.35)} = 4.53\%$ for (135%, −35%), compared with $4\% \times \sqrt{0.85} = 3.69\%$ for the equal-weight portfolio. The monthly Sharpe ratio falls from 0.271 to 0.221. The higher the correlation, the more the same mean-estimation error is amplified (at $\rho = 0.5$, the weights are about (95%, 5%)).

This simple example reveals the core logic of practical portfolio construction:

1. **Shrinkage guards against covariance ill-conditioning**: With a limited sample, the extreme eigenvalues of the sample covariance matrix can be badly distorted. Shrinking the matrix toward a structured target, such as a diagonal matrix or factor model, at a theoretically chosen rate can dampen very small eigenvalues and keep the inverse from turning local noise into extreme positions.
2. **Constraints are the first line of defense against mean-estimation errors**: Long-only constraints rule out short positions and mathematically cap one mechanism by which estimated means create extreme weights. Although long-only constraints invalidate the analytical solution, they restrict optimization to the simplex and can prevent extreme hedges.
3. **Risk parity avoids relying on unreliable expected-return estimates**: When expected returns cannot be estimated precisely, a portfolio that uses the covariance matrix to equalize assets' contributions to total risk avoids one source of estimation error. It still depends on a risk model, so any improvement in return or risk must be compared out of sample.
4. **Trading frictions shape the turnover boundary**: Price changes cause asset weights to drift away from their targets. If the portfolio is forced back to the exact target every month, commissions and market-impact costs from turnover accumulate quickly. A balance is needed between tracking error from drift and turnover costs.

Simulation of Estimation Error and Comparison of Strategy Distributions

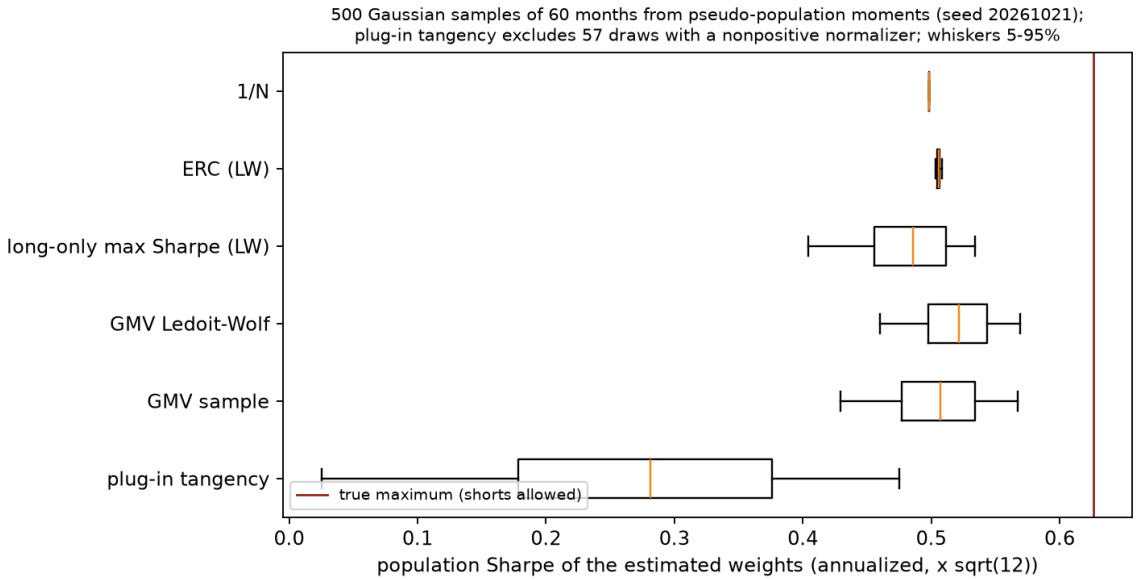


Figure 1: Estimation-error simulation comparison

- **Horizontal axis**: Annualized population Sharpe ratio realized when each strategy's estimated weights are applied to the true population (multiplied by $\sqrt{12}$);
- **Vertical axis**: The six strategies compared (plug-in tangency, GMV sample, GMV Ledoit–Wolf, long-only max Sharpe (LW), ERC (LW), and 1/N);
- **Units**: Dimensionless Sharpe ratio;
- **Sample and setup**: The full-sample monthly first and second moments of the French 10 industries are treated as a fixed "pseudo-population." With random seed 20261021, 500 independent Gaussian Monte Carlo samples are drawn, each 60 months long (representing a five-year rolling window);

- **How to read the figure and what to look for**:
  1. The theoretical maximum true tangency Sharpe ratio is marked by the red vertical line (about 0.6262 annualized);
  2. The unconstrained plug-in tangency portfolio performs worst, with a median annualized Sharpe ratio of just 0.2812 and a 10th percentile of about 0.079 (the whiskers show the 5th–95th percentiles). In addition, 57 samples failed outright because the denominator was non-positive; those failures were excluded from the plotted distribution, making the box plot substantially optimistic. The long-only max-Sharpe (LW) strategy had another 11 samples with no positive-excess-return portfolio in the training sample, triggering the input-feasibility failure described in Section 12.3; those were also excluded from the box plot;
  3. Adding a long-only constraint raises the median for long-only max Sharpe (LW) to 0.4859, reducing dispersion caused by estimation error;
  4. Strategies that do not rely on expected-return estimates (GMV LW, ERC LW, and 1/N) are the most stable: the median annualized Sharpe is 0.5215 for GMV (LW), 0.5057 for ERC (LW), and a stable 0.4983 for 1/N;
- **Limitation**: This figure shows a statistical simulation of sampling error only. All asset returns are strictly Gaussian; real-market jumps, fat tails, and structural breaks are not included.

4. Mathematical Foundations and Notation

Before turning to the derivations, we standardize notation, matrix dimensions, and units throughout the chapter:

- **Number of assets and sample periods**: The number of assets is denoted by n (or by p for the cross-sectional dimension), and the number of periods in the historical training window by T.
- **Return vector**: The vector of single-period simple returns at time t is $\mathbf{r}_t = (r_{1,t}, \ldots, r_{n,t})^\top \in \mathbb{R}^n$. This chapter uses monthly decimal returns by default. Explicitly labeled annualized teaching examples are exceptions and must not be mixed with monthly empirical inputs; for example, a monthly return of 1% is written as 0.01.
- **Expected-excess-return vector**: $\boldsymbol{\mu}_{excess} = \boldsymbol{\mu} - r_f\mathbf{1} \in \mathbb{R}^n$, where $\boldsymbol{\mu} = E[\mathbf{r}]$, r_f is the risk-free rate, and $\mathbf{1} = (1, \ldots, 1)^\top$ is the all-ones vector.
- **Covariance matrix**: $\Sigma \in \mathbb{R}^{n\times n}$ is the single-period return covariance matrix and is positive semidefinite (PSD). If invertibility is required, it must be positive definite (PD), denoted by $\Sigma \succ 0$.
- **Portfolio-weight vector**: $\mathbf{w} = (w_1, \ldots, w_n)^\top \in \mathbb{R}^n$. w_i is the fraction of portfolio net asset value (NAV) invested in asset i. Fully invested means $\mathbf{1}^\top\mathbf{w} = 1$, and long-only means $\mathbf{w} \geq \mathbf{0}$.
- **Matrix Frobenius-norm convention**: Following Ledoit & Wolf (2004), this chapter normalizes the Frobenius norm of a $p \times p$ matrix A by its dimension p:

$$\|A\|^2 = \frac{1}{p}\operatorname{tr}(AA^\top) = \frac{1}{p}\sum_{i=1}^{p}\sum_{j=1}^{p} a_{ij}^2,$$

The matrix inner product is defined as $\langle A, B\rangle = \frac{1}{p}\operatorname{tr}(AB^\top)$. Unless stated otherwise, $\|\cdot\|$ denotes this normalized norm.

- **Time and information set**: Maintain strict causality. In the idealized month-boundary test, if a rebalance decision is made and executed at the end of month $t-1$, the historical training window available for that decision is restricted to $[t-W, t-1]$ (or to $[t-W-1, t-2]$ when a one-month data-publication lag is assumed). The current-period return $\mathbf{r}_t$ is never known before the end of month t. Observing returns, calculating weights, and executing at the same month-end does not guarantee real-world availability; a live system must record when data are received and when the next trade can occur.

5. Formal Definitions: Estimation Error, Neutrality Constraints, and Risk Budgets

5.1 Mean-Estimation Error and Covariance Ill-Conditioning

Suppose asset returns are independently and identically distributed, with true mean $\boldsymbol{\mu}$ and true covariance Σ. The sample-mean estimator is $\hat{\boldsymbol{\mu}} = \frac{1}{T}\sum_{t=1}^{T}\mathbf{r}_t$, and its estimation-error standard deviation is:

$$\mathrm{SE}(\hat{\mu}_i) = \frac{\sigma_i}{\sqrt{T}}.$$

If an asset has monthly volatility $\sigma = 4\%$ (about 13.86% annualized), then with $T = 120$ months (10 years of history), the standard error of its monthly mean is:

$$\mathrm{SE}(\hat{\mu}_i) = \frac{0.04}{\sqrt{120}} \approx 0.003651 \quad (0.365\%/\text{month}, \text{about annually: } 4.38\%).$$

For a hypothetical stock, if a researcher assumes an annualized excess return of only $5\% \sim 8\%$ (a teaching assumption, not a market statistic verified in this chapter), even 10 years of data may not narrow the expected-return confidence interval enough to determine its sign reliably.

The sample covariance matrix, centered by the sample mean and divided by T, is defined as:

$$S = \frac{1}{T}\sum_{t=1}^{T}(\mathbf{r}_t - \hat{\boldsymbol{\mu}})(\mathbf{r}_t - \hat{\boldsymbol{\mu}})^\top.$$

Equation (12) in Section 2.2 of Ledoit and Wolf (2004) rewrites Lemma 2.1 in terms of eigenvalues: in expectation, the dispersion of sample eigenvalues around their common mean equals the dispersion of the true eigenvalues plus $E\|S-\Sigma\|^2$. This shows that estimation noise inflates eigenvalue dispersion in the population sense; it does not guarantee that each sample maximum or minimum eigenvalue moves in a particular direction. The distortion becomes more pronounced as p/T increases. When $p > T$, S is rank-deficient: at least $p-T+1$ eigenvalues are exactly zero under exact arithmetic (after centering, the rank is at most $T-1$), so the sample inverse S^{-1} does not exist. This does not imply that the population matrix Σ is singular.

5.2 Three Definitions of Portfolio Neutrality

Quantitative investors often seek to eliminate exposure to specific macroeconomic or systematic risks. Common forms of "neutrality" include:

1. **Dollar neutrality / capital neutrality**:

$$\mathbf{1}^\top \mathbf{w} = \sum_{i=1}^{n} w_i = 0.$$

   The total market value of long positions equals that of short positions, leaving net exposure equal to 0.
2. **Beta neutrality**: Let $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_n)^\top \in \mathbb{R}^n$ be the vector of regression slopes of the assets relative to a benchmark index, such as the S&P 500. A beta-neutral portfolio satisfies:

$$\boldsymbol{\beta}^\top \mathbf{w} = \sum_{i=1}^{n} w_i \beta_i = 0.$$

   At the margin, the portfolio has no direct linear exposure to movements in the overall benchmark return.
3. **Sector neutrality**: Suppose the market is partitioned into K mutually exclusive sectors, and let I_k be the set of asset indices in sector k. Sector neutrality requires net holdings to be zero in every sector:

$$\sum_{i \in I_k} w_i = 0, \quad \forall k \in \{1, \ldots, K\}.$$

These conditions are distinct, but they are not independent. If the mutually exclusive sectors cover all assets, summing across sectors gives

$$\mathbf{1}^\top \mathbf{w} = \sum_{k=1}^{K} \sum_{i \in I_k} w_i = 0.$$

Therefore, the absolute sector neutrality used in this chapter necessarily implies dollar neutrality, but generally does not imply beta neutrality. Dollar neutrality generally implies neither sector nor beta neutrality, and beta neutrality generally implies neither of the other two. For example, within one sector, weights $(0.5, -0.5)$ and betas $(1.5, 0.5)$ produce zero net sector weight and zero net dollar exposure, but beta exposure is 0.5. Conversely, weights $(0.5, -1.0)$ with betas $(1, 0.5)$ have zero beta exposure but net weight of -0.5. If "sector neutrality" instead means an active-weight constraint relative to a fully invested benchmark $\mathbf{w}_b$, namely $Z(\mathbf{w} - \mathbf{w}_b) = 0$, it implies $\sum_i w_i = \sum_i w_{b,i}$, not zero absolute net weight. Here Z is a $K \times n$ matrix whose rows are sector indicator vectors; this benchmark-relative constraint should be distinguished from the absolute neutrality defined in this section.

5.3 Risk Contributions and the Euler Decomposition

Portfolio volatility is the function $\sigma(\mathbf{w}) = \sqrt{\mathbf{w}^\top \Sigma \mathbf{w}}$. Because $\sigma(a\mathbf{w}) = a\sigma(\mathbf{w})$ for any positive scalar $a > 0$, portfolio volatility is **homogeneous of degree 1** in the weight vector $\mathbf{w}$. The derivatives and risk shares below require $\sigma(\mathbf{w}) > 0$ and a symmetric positive-semidefinite matrix Σ. By Euler's theorem for homogeneous functions:

$$\sigma(\mathbf{w}) = \sum_{i=1}^{n} w_i \frac{\partial \sigma(\mathbf{w})}{\partial w_i}.$$

This gives rise to three core definitions in the risk-contribution framework:

- **Marginal risk contribution (MRC)**:

$$\mathrm{MRC}_i = \frac{\partial \sigma(\mathbf{w})}{\partial w_i} = \frac{(\Sigma \mathbf{w})_i}{\sqrt{\mathbf{w}^\top \Sigma \mathbf{w}}} = \frac{(\Sigma \mathbf{w})_i}{\sigma(\mathbf{w})}.$$

  The economic meaning of MRC_i is the rate of change in total portfolio volatility when the weight of asset i increases by a small amount. It has the same units as return volatility (monthly decimal units here). This is a partial derivative that holds all other coordinates fixed. If increasing asset i requires selling asset j to preserve the budget, the change along the self-financing direction is MRC_i−MRC_j; it should not be read as MRC_i alone.
- **Absolute risk contribution (RC)**:

$$\mathrm{RC}_i = w_i \, \mathrm{MRC}_i = \frac{w_i (\Sigma \mathbf{w})_i}{\sigma(\mathbf{w})}.$$

  By Euler's theorem, the absolute risk contributions of all assets sum exactly to total portfolio volatility: $\sum_{i=1}^{n} \mathrm{RC}_i = \sigma(\mathbf{w})$.
- **Risk fraction / relative risk contribution**:

$$p_i = \frac{\mathrm{RC}_i}{\sigma(\mathbf{w})} = \frac{w_i (\Sigma \mathbf{w})_i}{\mathbf{w}^\top \Sigma \mathbf{w}}.$$

  Relative risk shares are dimensionless and sum to one: $\sum_{i=1}^{n} p_i = 1$. They are not probabilities. If a weight and its marginal risk contribution have opposite signs, p_i can be negative, meaning that the position offsets some risk in the current portfolio. A long position can also have a negative risk contribution; a short position multiplied by a negative marginal contribution has a positive contribution.

5.4 Equal Risk Contribution (ERC) Portfolios

An equal risk contribution (ERC) portfolio requires every asset to contribute equally in absolute terms to total portfolio volatility, over the fully invested long-only simplex:

$$\mathrm{RC}_i = \mathrm{RC}_j \iff w_i (\Sigma \mathbf{w})_i = w_j (\Sigma \mathbf{w})_j, \quad \forall i, j \in \{1, \ldots, n\},$$

Thus, each asset carries exactly $1/n$ of the relative risk share: $p_i = 1/n$.

Check the two-asset case before memorizing an ERC formula. In this annualized-decimal teaching example, $\sigma_1 = 0.2, \sigma_2 = 0.3, \rho = 0.2$, so

$$\Sigma = \begin{pmatrix} 0.04 & 0.012 \\ 0.012 & 0.09 \end{pmatrix}.$$

Set the two risk contributions equal, first canceling their common positive denominator σ_p:

$$\begin{aligned} w_1^2(0.04) + w_1 w_2(0.012) &= w_1 w_2(0.012) + w_2^2(0.09), \\ 0.04 w_1^2 &= 0.09 w_2^2, \\ 0.2 w_1 &= 0.3 w_2 \quad \text{(positive weights give the positive square root).} \end{aligned}$$

Combining this with $w_1 + w_2 = 1$ gives $(w_1, w_2) = (0.6, 0.4)$. Substituting these weights back into the risk calculation:

$$\begin{aligned} \Sigma \mathbf{w} &= (0.0288, 0.0432)^\top, \\ \sigma_p^2 &= 0.6(0.0288) + 0.4(0.0432) = 0.03456, \\ \mathrm{RC}_1 &= \mathrm{RC}_2 = 0.01728/\sqrt{0.03456} = 0.092952. \end{aligned}$$

Volatility is 18.59%, and the two risk shares are 50% each even though the capital weights are 60% and 40%. The two-asset cross terms cancel, so when volatility is positive and $\rho \in (-1, 1)$, ERC weights do not depend on correlation; correlation still changes total volatility. With three or more assets, these pairwise cancellations do not occur, so inverse-volatility weights are not a general substitute for ERC.

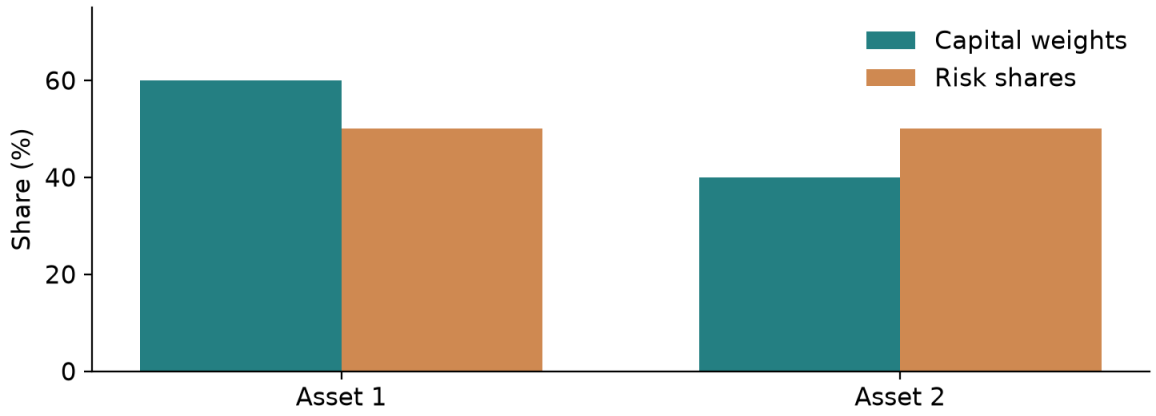


Figure 2: Difference between capital weights and risk shares

The horizontal axis shows the two assets and the vertical axis shows shares (%). Capital weights are 60%/40%, while risk shares are 50%/50%. This figure checks only the annualized example above; it does not show that ERC has higher returns. Section 6.3 explains how to solve for a specified risk budget under a general covariance matrix.

6. Core Derivations: Shrinkage, Convex-QP Transformation, and Risk Decomposition

6.1 Deriving the Ledoit–Wolf Linear Shrinkage Estimator

Ledoit & Wolf (2004) seek a linear combination $\Sigma^* = \rho_1 I + \rho_2 S$. In the theoretical derivation, the shrinkage target is μI, where $\mu = \langle \Sigma, I \rangle = \frac{1}{p}\operatorname{tr}(\Sigma)$ is the mean of the true eigenvalues. The feasible estimator later replaces μ with its sample counterpart, $m = \langle S, I \rangle$. Let the true, unknown covariance matrix be Σ. The theoretical objective is to minimize expected squared loss under the normalized Frobenius norm:

$$\min_{\rho_1,\rho_2} E\left[\|\Sigma^* - \Sigma\|^2\right] = E\left[\|\rho_1 I + \rho_2 S - \Sigma\|^2\right].$$

Using an orthogonal decomposition and expanding the expectation, define three key geometric distance parameters. In this subsection, δ^2 denotes only the total dispersion relevant to shrinkage; it is not the square of the risk-aversion coefficient δ in Section 6.4:

1. Expected squared distance from the sample covariance to the target: $\delta^2 = E\left[\|S - \mu I\|^2\right]$;
2. Squared distance from the true covariance to the target: $\alpha^2 = \|\Sigma - \mu I\|^2$;
3. Expected estimation error of the sample covariance relative to the true covariance: $\beta^2 = E\left[\|S - \Sigma\|^2\right]$.

First derive the result under the theoretical benchmark $E[S] = \Sigma$. Under this condition, the Pythagorean structure holds in expectation. Centered outer products satisfy the condition when the true mean is known. In a finite sample, a matrix centered using the sample mean and divided by T instead has $E[S] = (T-1)\Sigma/T$, so the following relation is not an unconditional finite-sample identity. The practical shrinkage estimator uses an asymptotic construction:

$$\delta^2 = \alpha^2 + \beta^2.$$

The proof of Lemma 2.1 uses only $E[S] = \Sigma$: when expanding $E\|S - \Sigma + \Sigma - \mu I\|^2$, the cross term $2\langle E[S - \Sigma], \Sigma - \mu I\rangle = 0$. Rewrite Σ^* as the convex combination $\Sigma^* = \kappa(\mu I) + (1-\kappa)S$ (the original paper first proves that the optimal target scale is exactly μ), and substitute it into the squared-loss objective:

$$\|\Sigma^* - \Sigma\|^2 = \|(1-\kappa)(S - \Sigma) - \kappa(\Sigma - \mu I)\|^2.$$

Expanding the expectation and using the first-moment unbiasedness that makes the cross term vanish gives a quadratic in the shrinkage intensity κ:

$$E\left[\|\Sigma^* - \Sigma\|^2\right] = (1-\kappa)^2\beta^2 + \kappa^2\alpha^2.$$

Differentiate with respect to κ and set the derivative to zero:

$$-2(1-\kappa)\beta^2 + 2\kappa\alpha^2 = 0 \implies \kappa^* = \frac{\beta^2}{\alpha^2 + \beta^2} = \frac{\beta^2}{\delta^2}.$$

Thus, the theoretically optimal shrinkage intensity is exactly the share of total dispersion attributable to estimation-error variance, β^2/δ^2. Because the population matrix Σ is unknown in practice, Ledoit & Wolf construct asymptotically unbiased feasible estimators for the corresponding quantities:

- Estimate of the target scale: $m = \frac{1}{p}\operatorname{tr}(S)$;

- Estimate of total sample dispersion: $d^2 = \|S - mI\|^2 = \frac{1}{p}\sum_{i=1}^{p}\sum_{j=1}^{p}(s_{ij} - m\delta_{ij})^2$;
- Asymptotically unbiased estimate of estimation error: Let the rows of the centered sample matrix be $\mathbf{x}_t \in \mathbb{R}^p$, with the outer product for each period $\mathbf{x}_t\mathbf{x}_t^\top$. Since S is the average of these T outer products, the error variance of the average is approximately the dispersion of a single outer product multiplied by $1/T$. Lemma 3.4 in the original paper therefore uses $\bar{b}^2 = \frac{1}{T^2}\sum_t \|\mathbf{x}_t\mathbf{x}_t^\top - S\|^2$ to estimate β^2. Expanding with $\sum_t \mathbf{x}_t^\top S\mathbf{x}_t = T\|S\|_F^2$ gives:

$$\bar{b}^2 = \frac{1}{T^2 p}\left(\sum_{t=1}^{T}\|\mathbf{x}_t\|_2^4 - T\sum_{i=1}^{p}\sum_{j=1}^{p}s_{ij}^2\right);$$

- Truncated shrinkage intensity: $b^2 = \min(\max(\bar{b}^2, 0), d^2)$, giving the feasible shrinkage intensity:

$$\hat{\kappa} = \frac{b^2}{d^2} \in [0, 1].$$

  The resulting feasible shrinkage estimator is positive semidefinite; it is positive definite under the following condition:

$$S^* = \hat{\kappa}(mI) + (1 - \hat{\kappa})S.$$

  Because the target matrix mI is positive definite as long as at least one asset has nonzero variance, when $\hat{\kappa} > 0$, the smallest eigenvalue of S^* is strictly positive even if S is rank-deficient because $T < p$. The shrunk matrix is therefore numerically invertible.

6.2 Convex-QP Reformulation of Long-Only Maximum Sharpe Ratio

The original long-only optimization problem for maximizing a portfolio's Sharpe ratio is:

$$\max_{\mathbf{w}} \frac{\boldsymbol{\mu}_{excess}^\top \mathbf{w}}{\sqrt{\mathbf{w}^\top \Sigma \mathbf{w}}} \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{w} = 1, \quad \mathbf{w} \geq \mathbf{0}, \quad \mathbf{w} \leq \mathbf{w}_{\max}.$$

The objective is a ratio, not a concave function, so it cannot be passed directly to CVXPY under Disciplined Convex Programming (DCP) rules. Section 7 of Chapter 9 and exercise 9-C01 already prove the variable substitution below and its conditions. We restate it here as implemented in this chapter; Lesniewski's Baruch MFE Lecture 5 gives the same transformation.

Proof: The discussion below concerns a finite Sharpe ratio and requires every feasible portfolio with positive excess return to have positive variance; $\Sigma \succ 0$ is a simple sufficient condition. If a zero-risk direction has positive excess return, the Sharpe denominator is zero and the QP objective may be zero. This is a degenerate or infinite-Sharpe case, not an ordinary finite optimum.

Keep three separate gates in view. Budget feasibility requires $nw_{\max} \geq 1$. Feasibility of positive excess return requires $\max_w \mu_{excess}^\top w > 0$ under the weight cap. A finite Sharpe ratio also requires the risk denominator of every compared portfolio to be nonzero. $\Sigma \succ 0$ guarantees this for nonzero weights. If the matrix is only positive semidefinite and a feasible w satisfies both $w^\top \Sigma w = 0$ and $\mu_{excess}^\top w > 0$, it lies on a zero-risk, positive-excess-return boundary where ordinary finite Sharpe is undefined; a QP optimum of 0 cannot be interpreted as a finite maximum Sharpe ratio. A singular matrix does not necessarily create this boundary, but the risk domain must be checked explicitly. The Project 3 implementation requires numerically positive-definite input and does not introduce a separate zero-risk strategy.

Assume that at least one portfolio in the feasible set has strictly positive expected excess return, that is, $\max_{\mathbf{w}} \boldsymbol{\mu}_{excess}^{\top}\mathbf{w} > 0$. Define the scalar transformation factor:

$$\kappa = \frac{1}{\boldsymbol{\mu}_{excess}^{\top}\mathbf{w}} > 0,$$

and define the scaled weight vector:

$$\mathbf{y} = \kappa\mathbf{w} \in \mathbb{R}^n.$$

Since $\mathbf{w} \geq \mathbf{0}$ and $\kappa > 0$, it follows that $\mathbf{y} \geq \mathbf{0}$. The constraints map algebraically as follows:

1. Expected excess return is normalized:

$$\boldsymbol{\mu}_{excess}^{\top}\mathbf{y} = \kappa(\boldsymbol{\mu}_{excess}^{\top}\mathbf{w}) = 1;$$

2. The investment-budget constraint becomes:

$$\mathbf{1}^{\top}\mathbf{y} = \kappa(\mathbf{1}^{\top}\mathbf{w}) = \kappa;$$

3. The weight-cap constraint becomes:

$$w_i \leq w_{\max} \iff y_i \leq w_{\max}\kappa.$$

Consider the reciprocal of the squared objective:

$$\begin{aligned}\frac{1}{\mathrm{SR}(\mathbf{w})^2} &= \frac{\mathbf{w}^{\top}\Sigma\mathbf{w}}{(\boldsymbol{\mu}_{excess}^{\top}\mathbf{w})^2} \\ &= (\kappa\mathbf{w})^{\top}\Sigma(\kappa\mathbf{w}) \quad \text{(the same positive scaling)} \\ &= \mathbf{y}^{\top}\Sigma\mathbf{y} \quad \text{(substitute the definition)}.\end{aligned}$$

Thus, maximizing the Sharpe ratio is equivalent to minimizing its squared reciprocal:

$$\min_{\mathbf{y}\in\mathbb{R}^n,\kappa\in\mathbb{R}} \mathbf{y}^{\top}\Sigma\mathbf{y}$$

subject to the linear constraints:

$$\begin{cases}\boldsymbol{\mu}_{excess}^{\top}\mathbf{y} = 1, \\ \mathbf{1}^{\top}\mathbf{y} = \kappa, \\ \mathbf{y} \geq \mathbf{0}, \\ \mathbf{y} \leq w_{\max}\kappa, \\ \kappa > 0.\end{cases}$$

When $\Sigma \succeq 0$, $\mathbf{y}^{\top}\Sigma\mathbf{y}$ is convex; when $\Sigma \succ 0$, it is strictly convex and the solution is unique. All constraints are affine equalities or inequalities, so this is a convex quadratic program. The strict inequality $\kappa > 0$ need not be passed to the solver: $\boldsymbol{\mu}_{excess}^{\top}\mathbf{y} = 1$ forces $\mathbf{y} \neq \mathbf{0}$, and because $\mathbf{y} \geq \mathbf{0}$, we have $\kappa = \mathbf{1}^{\top}\mathbf{y} > 0$. The reverse mapping $\mathbf{w} = \mathbf{y}/\kappa$ sends any feasible $(\mathbf{y}, \kappa)$ back to a feasible portfolio with positive excess return. The feasible solutions of the two problems are in one-to-one correspondence, and their objective values equal $1/\mathrm{SR}^2$. After solving for the optimum $(\mathbf{y}^*, \kappa^*)$, recover the optimal portfolio weights for the original problem directly using the inverse transformation:

$$\mathbf{w}^* = \frac{\mathbf{y}^*}{\kappa^*} = \frac{\mathbf{y}^*}{\sum_{i=1}^n y_i^*}.$$

This completes the proof.

6.3 Convex Log-Barrier Formulation and Damped Newton Algorithm for Risk Budgeting (ERC)

Section 3.4 of Maillard, Roncalli, and Teiletche (2009) formulate ERC as a minimum-volatility problem subject to $\sum_i \ln x_i \geq c$. The unconstrained log-barrier form below corresponds to that formulation, with the constraint multiplier absorbed into the objective; the derivation and implementation here are developed in this chapter. Consider the log-barrier function:

$$\min_{\mathbf{y}\in\mathbb{R}^n,\mathbf{y}>\mathbf{0}} f(\mathbf{y}) = \frac{1}{2}\mathbf{y}^\top\Sigma\mathbf{y} - \sum_{i=1}^{n} b_i \ln y_i,$$

where $b_i > 0$ are the specified risk-budget weights, satisfying $\sum b_i = 1$ (for ERC, $b_i = 1/n$). The gradient of this objective is:

$$\nabla f(\mathbf{y}) = \Sigma\mathbf{y} - \frac{\mathbf{b}}{\mathbf{y}}, \quad \text{where } \left(\frac{\mathbf{b}}{\mathbf{y}}\right)_i = \frac{b_i}{y_i}.$$

The Hessian matrix is:

$$\nabla^2 f(\mathbf{y}) = \Sigma + \operatorname{diag}\left(\frac{\mathbf{b}}{\mathbf{y}^2}\right).$$

As long as $\Sigma \succeq 0$ and $b_i > 0$, the diagonal terms are strictly positive and $\nabla^2 f(\mathbf{y}) \succ 0$. The objective is therefore strictly convex on $\mathbf{y} > \mathbf{0}$, and any minimizer, if it exists, is unique. The condition $\Sigma \succ 0$ guarantees existence, but is not necessary. For example, although $\Sigma = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ is singular, with $b_1 = b_2 = 1/2$ it still has a unique minimizer $\mathbf{y} = (1/2, 1/2)^\top$, which normalizes to equal weights. The key danger is a nonnegative zero-risk direction: if some nonzero $\mathbf{v} \geq \mathbf{0}$ satisfies $\mathbf{v}^\top\Sigma\mathbf{v} = 0$, then along $\mathbf{y} = \mathbf{y}_0 + t\mathbf{v}$, with $\mathbf{y}_0 > 0$ and $t \to \infty$, positive semidefiniteness implies $\Sigma\mathbf{v} = 0$. The quadratic term stays constant while at least one logarithmic term tends to negative infinity, so no minimizer exists. The core function therefore requires a numerically positive-definite covariance matrix. At a first-order optimum, the gradient is zero:

$$(\Sigma\mathbf{y}^*)_i - \frac{b_i}{y_i^*} = 0 \iff y_i^*(\Sigma\mathbf{y}^*)_i = b_i.$$

Because the variance contribution $y_i(\Sigma\mathbf{y})_i$ is homogeneous of degree 2 in $\mathbf{y}$ (while the volatility contribution $y_i(\Sigma\mathbf{y})_i/\sigma(\mathbf{y})$ is homogeneous of degree 1), normalize $\mathbf{y}^*$ onto the simplex:

$$\mathbf{w}^* = \frac{\mathbf{y}^*}{\sum_{j=1}^n y_j^*}.$$

Normalization does not change the relative proportions of the assets' risk contributions:

$$w_i^*(\Sigma\mathbf{w}^*)_i = \frac{y_i^*(\Sigma\mathbf{y}^*)_i}{\left(\sum y_j^*\right)^2} = \frac{b_i}{\left(\sum y_j^*\right)^2} \propto b_i.$$

The denominator also needs to be checked. Let $s = \sum_j y_j^*$. Summing the first-order conditions gives $\mathbf{y}^{*\top}\Sigma\mathbf{y}^* = \sum_i b_i = 1$, so

$$\sigma^2(\mathbf{w}^*) = 1/s^2, \qquad \frac{w_i^*(\Sigma\mathbf{w}^*)_i}{\sigma^2(\mathbf{w}^*)} = \frac{b_i/s^2}{1/s^2} = b_i.$$

This proves the risk-budget proportions, not the budget amounts. In the two-asset example above, set $\mathbf{y} = (0.6, 0.4)^\top / \sqrt{0.03456}$; the quadratic form is 1 and each contribution is 0.5. Normalization then recovers the original weights. Finding a minimizer, reaching the algorithm's stopping threshold, obtaining a sufficiently small budget residual, and deciding that a strategy is worth trading are four separate judgments.

Damped Newton Algorithm and Stopping Rule:

1. Initial point: start at the inverse-diagonal-standard-deviation point $\mathbf{y}^{(0)} = (1/\sqrt{\Sigma_{11}}, \ldots, 1/\sqrt{\Sigma_{nn}})^\top$, then normalize so that $\mathbf{y}^\top \Sigma \mathbf{y} = 1$.
2. Compute the Newton direction: $\Delta\mathbf{y} = -[\nabla^2 f(\mathbf{y})]^{-1} \nabla f(\mathbf{y})$.
3. Compute the Newton decrement (Boyd & Vandenberghe, Section 9.5):

$$\lambda^2 = -\nabla f(\mathbf{y})^\top \Delta\mathbf{y}.$$

   $\lambda^2/2$ is a quadratic-model estimate of $f(\mathbf{y}) - f^*$ (Boyd–Vandenberghe, Section 9.5.1), not a general strict upper bound. Stop the iteration when $\lambda^2 \leq 10^{-14}$, then check the risk-share deviations.
4. Backtracking line search: use the Armijo condition to choose a step size $\alpha \in (0, 1]$, ensuring that the new point remains in the feasible region $\mathbf{y} + \alpha\Delta\mathbf{y} > \mathbf{0}$.

6.4 Bayesian Derivation of the Black–Litterman Model and Its Gain Form

The Black–Litterman model combines market-equilibrium implied returns and investor views into an expected-return vector. The original Black and Litterman (1992) paper was not available for this chapter; the setup and formulas below are based on Appendix B of He and Litterman (1999).

1. **Reverse optimization determines the prior**: Assume that investors as a whole hold the market portfolio $\mathbf{w}_{mkt}$. By the first-order condition for mean–variance optimization, the market-equilibrium expected-return vector $\mathbf{\Pi}$ must satisfy:

$$\mathbf{\Pi} = \delta \Sigma \mathbf{w}_{mkt},$$

   Here δ is the aggregate risk-aversion coefficient for the market as a whole (He & Litterman 1999 commonly use $\delta = 2.5$).
2. **Prior distribution**: Treat the expected-return vector itself as a random variable with a prior normal distribution centered on equilibrium returns:

$$\boldsymbol{\mu} \sim N(\mathbf{\Pi}, \tau\Sigma),$$

   The scalar $\tau > 0$ scales confidence by the prior variance relative to the variance of realized returns. All returns here are expected excess returns at the same frequency; total returns would also include the cash return over the same period.
3. **View system**: Suppose the investor holds k linear views, expressed in matrix form as:

$$P\boldsymbol{\mu} = \mathbf{Q} + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim N(\mathbf{0}, \Omega),$$

   Here $P \in \mathbb{R}^{k \times n}$ is the view-selection matrix, $\mathbf{Q} \in \mathbb{R}^k$ is the vector of view returns, and $\Omega \in \mathbb{R}^{k \times k}$ is the covariance matrix of view errors. He and Litterman assume view errors are

independent of prior expected-return errors and that errors across views are independent, which is why the view-error covariance is usually taken to be diagonal. The first independence assumption is also needed for this chapter's block-Gaussian covariance derivation.

4. **Deriving the posterior distribution (block-normal conditional update)**: The joint distribution of $\boldsymbol{\mu}$ and the observed views $\mathbf{v} = P\boldsymbol{\mu} + \boldsymbol{\varepsilon}$ is:

$$\begin{bmatrix} \boldsymbol{\mu} \\ \mathbf{v} \end{bmatrix} \sim N\left(\begin{bmatrix} \boldsymbol{\Pi} \\ P\boldsymbol{\Pi} \end{bmatrix}, \begin{bmatrix} \tau\Sigma & \tau\Sigma P^\top \\ P\tau\Sigma & P\tau\Sigma P^\top + \Omega \end{bmatrix} \right).$$

Applying the conditional-normal formula gives the posterior expected-return mean $\boldsymbol{\mu}_{BL}$ and posterior-mean covariance V_{BL}:

$$\boldsymbol{\mu}_{BL} = \boldsymbol{\Pi} + \tau\Sigma P^\top (P\tau\Sigma P^\top + \Omega)^{-1}(\mathbf{Q} - P\boldsymbol{\Pi}),$$

$$V_{BL} = \tau\Sigma - \tau\Sigma P^\top (P\tau\Sigma P^\top + \Omega)^{-1} P\tau\Sigma.$$

Define the Kalman-form gain matrix $M = \tau\Sigma P^\top (P\tau\Sigma P^\top + \Omega)^{-1}$. The update can then be written compactly as:

$$\boldsymbol{\mu}_{BL} = \boldsymbol{\Pi} + M(\mathbf{Q} - P\boldsymbol{\Pi}).$$

This gain form does not require Ω itself to be invertible, but it does require $S_v = P\tau\Sigma P^\top + \Omega$ to be invertible. When $\Omega = 0$, $\Sigma \succ 0$, and $\tau > 0$, the rows of P must be linearly independent. Repeating the same certain view makes S_v singular; assigning two different values to that duplicated view is even contradictory. The core implementation raises an explicit error rather than hiding the issue with an inverse or pseudoinverse. V_{BL} measures posterior uncertainty about the unknown mean; it is not the covariance of next-period asset returns. If we separately model $R = \mu + \varepsilon$, with $\mathrm{Cov}(\varepsilon) = \Sigma$, and assume the new return noise is independent of the posterior mean, the law of total variance gives the posterior predictive covariance $\Sigma + V_{BL}$. The portfolio tilt below uses Σ as the return-risk penalty, preserving the original model interface. If predictive risk is used instead, the objective and weights must both be recomputed; the two covariance matrices cannot be treated as interchangeable.

5. **Orthogonality of the optimal portfolio tilt**: Substitute the posterior mean into the mean–variance optimal allocation:

$$\mathbf{w}^* = (\delta\Sigma)^{-1}\boldsymbol{\mu}_{BL} = (\delta\Sigma)^{-1}\left[\delta\Sigma\mathbf{w}_{mkt} + \tau\Sigma P^\top (P\tau\Sigma P^\top + \Omega)^{-1}(\mathbf{Q} - P\boldsymbol{\Pi})\right].$$

Expanding and rearranging gives:

$$\mathbf{w}^* = \mathbf{w}_{mkt} + P^\top \left[\frac{\tau}{\delta}(P\tau\Sigma P^\top + \Omega)^{-1}(\mathbf{Q} - P\boldsymbol{\Pi})\right] = \mathbf{w}_{mkt} + P^\top \boldsymbol{\lambda}.$$

This expression shows that, without constraints, the deviation of portfolio weights from market weights lies in the column space of $P^\top$, meaning it is a linear combination of the view portfolios. If a column of P is all zeros (no view concerns that asset), its portfolio weight remains at the market weight, although its posterior mean can still change through correlations (asset 2 in Section 8.5). This property generally no longer holds after adding constraints such as long-only or weight caps.

6.5 The Kelly Criterion and Deriving Log Growth

Kelly (1956) studied capital-allocation rules that maximize the geometric growth rate of wealth over the long run.

Discrete binary bet: Suppose an investor bets a fraction $f \in [0, 1)$ of capital. The probability of winning is p, and the net odds are b: a one-unit stake earns b units net if it wins and loses the one-unit stake if it loses. Expected log growth in one period is:

$$g(f) = E[\ln(W_{t+1}/W_t)] = p \ln(1 + bf) + (1 - p) \ln(1 - f).$$

Differentiate with respect to f:

$$g'(f) = \frac{pb}{1 + bf} - \frac{1 - p}{1 - f}.$$

Set $g'(f) = 0$:

$$pb(1 - f) = (1 - p)(1 + bf) \implies pb - pbf = 1 - p + bf - pbf \implies bf = pb - (1 - p).$$

Solving gives the well-known Kelly-optimal bet fraction:

$$f^* = p - \frac{1 - p}{b} = \frac{bp - (1 - p)}{b}.$$

With symmetric odds ($b = 1$), the optimal fraction simplifies to $f^* = 2p - 1$. When $bp \leq 1 - p$ (expected net return is non-positive), $g'(0) = pb - (1 - p) \leq 0$ and g is concave, so every $f > 0$ is no better than not betting. Therefore, $f^* = \max\{(bp - (1 - p))/b, 0\}$; the core function `kelly_binary_fraction` applies this truncation.

Notation note: On printed page 920 of Kelly's (1956) paper in the *Bell System Technical Journal*, the letter q denotes the probability that a signal is transmitted correctly (a win), p denotes the probability of a transmission error (a loss), and the bet fraction is denoted by ℓ. This chapter follows modern financial convention, using p for the probability of winning, $1 - p$ for the probability of losing, and f for the bet fraction.

Second-order expansion and quadratic proxy for discrete asset allocation: For assets with continuous returns, let single-period excess return be $\mathbf{r} - r_f\mathbf{1}$, with mean $\mathbf{m}$ and covariance Σ. A second-order Taylor expansion of single-period portfolio log growth is:

$$\ln(1 + r_p) \approx r_p - \frac{1}{2} r_p^2.$$

Let $a = E[r_p] = r_f + \mathbf{w}^\top \mathbf{m}$. The cash return r_f is deterministic over this holding period, so $E[r_p^2] = \mathrm{Var}(r_p) + a^2 = \mathbf{w}^\top \Sigma \mathbf{w} + a^2$. Taking the expectation of the Taylor polynomial gives the full discrete second-order expression:

$$g_{2,d}(\mathbf{w}) = a - \tfrac{1}{2} \left[\mathbf{w}^\top \Sigma \mathbf{w} + a^2 \right].$$

The term a^2 cannot be removed from the second-order term without explanation. For example, if r_p is equally likely to be 0.1 or 0.2, then $a = 0.15$, the variance is 0.0025, and the full second-order mean is 0.1375; omitting the squared mean would give 0.14875. This polynomial approximation also assumes returns stay within a region where the log expansion is valid, and cannot replace a check of the ruin boundary.

The code below uses a local model that also ignores the squared drift term:

$$g_q(\mathbf{w}) = r_f + \mathbf{w}^\top \mathbf{m} - \frac{1}{2}\mathbf{w}^\top \Sigma \mathbf{w}.$$

This is an additional approximation. A continuous-time diffusion limit is justified only when, over a time step Δt, the mean is $O(\Delta t)$ and the variance is $O(\Delta t)$, making $a^2 = O(\Delta t^2)$. Fixed monthly data do not automatically satisfy this limit. `quadratic_log_growth` calculates only g_q, and `gaussian_kelly_weights` optimizes only this explicit proxy objective. For the full discrete second-order expression, the gradient is $(1-a)\mathbf{m} - \Sigma\mathbf{w}$, and the unconstrained solution should be $(1-r_f)(\Sigma + \mathbf{m}\mathbf{m}^\top)^{-1}\mathbf{m}$, which generally differs from $\Sigma^{-1}\mathbf{m}$. Taking the first-order condition with respect to the weight vector $\mathbf{w}$:

$$\nabla g(\mathbf{w}) = \mathbf{m} - \Sigma\mathbf{w} = \mathbf{0} \implies \mathbf{w}^* = \Sigma^{-1}\mathbf{m}.$$

Here $\mathbf{w}^*$ gives the nominal weights of the risky assets as fractions of net asset value; there is no fully invested constraint. The remainder, $1 - \mathbf{1}^\top \mathbf{w}^*$, is held in the risk-free asset; a negative remainder represents borrowing at the risk-free rate (in 21-E08, $w^* = 3.2$ means borrowing 2.2 times net asset value). If we introduce a fractional-Kelly scaling parameter $c \in (0, 1]$ and set the portfolio weights to $\mathbf{w}_c = c\mathbf{w}^*$, then:

- Growth-rate ratio: $g(c\mathbf{w}^*) - r_f = c\mathbf{w}^{*\top}\mathbf{m} - \frac{1}{2}c^2\mathbf{w}^{*\top}\Sigma\mathbf{w}^* = (2c - c^2)\left[\frac{1}{2}\mathbf{m}^\top\Sigma^{-1}\mathbf{m}\right]$;
- Return-variance ratio: $\mathrm{Var}(c\mathbf{w}^*) = c^2\,\mathrm{Var}(\mathbf{w}^*)$. Under half Kelly ($c = 0.5$):
- Under the quadratic proxy objective above, growth above the cash baseline is $2(0.5) - 0.5^2 = 75\%$ of full Kelly;
- With the same input covariance, variance is $0.5^2 = 25\%$ of full Kelly. Under the known-moment quadratic approximation, half Kelly gives up one quarter of the growth rate in exchange for a three-quarter reduction in variance. In an ideal wealth stochastic differential equation with bounded coefficients, no jumps, continuous rebalancing without limit, and no margin liquidation, the stochastic exponential can keep wealth positive; the current discrete monthly model offers no such guarantee. Strictly Gaussian simple returns have an unbounded left tail, so any position with nonzero variance has positive probability of $r_p \le -1$, making exact $E\ln(1 + r_p)$ ill-defined. The quadratic proxy can still return a number, but that does not make it a valid exact Kelly solution. Even if the asset price cannot fall below zero, a discrete 3.2-times position (ignoring the cash return) would exhaust net asset value after a single-period asset loss greater than 31.25%. Jumps, margin, and financing restrictions widen this model error. If estimated $\mathbf{m}$ is too high, "full Kelly" already exceeds true Kelly; fractional Kelly provides protection against that error.

Kelly Growth-Rate and Variance Trade-Off Curves

- **Horizontal axes**: Bet fraction f (%) in the left panel and Kelly multiplier c in the right panel;
- **Vertical axes**: Expected log growth per period in the left panel; growth-rate and variance multiples relative to full Kelly in the right panel;

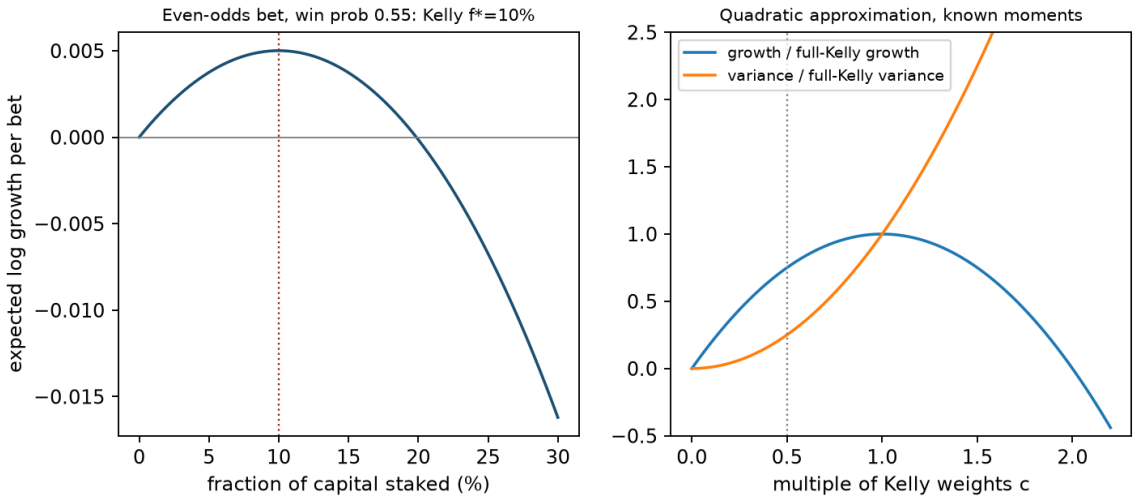


Figure 3: Kelly log growth and fractional-Kelly trade-off

- **How to read the figure and what to look for**:
  1. In the left panel, for $p = 0.55, b = 1$, log growth peaks at $f^* = 10\%$, with a maximum of 0.005008. At twice the Kelly position ($f = 20\%$), expected log growth turns negative. Under this example's assumptions of an IID bet, continued proportional betting, and infinitely many repetitions, wealth almost surely tends to zero;
  2. In the Gaussian approximation shown in the right panel, half Kelly (the gray dashed line at $c = 0.5$) achieves 75% of the theoretical peak growth rate while its variance is only 25% of full Kelly;
- **Limitation**: The right panel shows only a quadratic growth proxy with known moments and the squared-mean term omitted; the 75% refers to proxy growth above the cash baseline. Gaussian simple returns also have an unbounded left tail and cannot guarantee that exact log utility exists. Fat tails, jumps, financing, and margin constraints further change the position size that can be sustained.

6.6 Factor Risk Decomposition

The factor model from Chapter 15, $\mathbf{r} = B\mathbf{f} + \boldsymbol{\varepsilon}$, gives $\Sigma = B\Sigma_f B^\top + \Sigma_\varepsilon$. Here B is an $n \times K$ loading matrix, $\mathrm{Cov}(\mathbf{f}) = \Sigma_f$, the specific covariance Σ_ε is diagonal (consistent with Chapter 15 notation; in this chapter Ω is reserved for the Black–Litterman view covariance), and factor and specific returns are uncorrelated. Portfolio exposure is $\boldsymbol{\beta}_p = B^\top \mathbf{w}$ (a $K \times 1$ vector), so

$$\mathbf{w}^\top \Sigma \mathbf{w} = \boldsymbol{\beta}_p^\top \Sigma_f \boldsymbol{\beta}_p + \mathbf{w}^\top \Sigma_\varepsilon \mathbf{w}.$$

Asset i's variance contribution, $w_i(\Sigma\mathbf{w})_i$, can also be split into a factor component, $w_i(B\Sigma_f\boldsymbol{\beta}_p)_i$, and a specific component, $w_i^2(\Sigma_\varepsilon)_{ii}$. Consider a constructed single-factor example with $\boldsymbol{\beta} = (1.1, 0.9)$, $\Sigma_f = 0.03$, $\Sigma_\varepsilon = \mathrm{diag}(0.02, 0.01)$, and $\mathbf{w} = (0.5, 0.5)$ (annualized decimal units). Then $\beta_p = 1.0$; factor variance is 0.03, specific variance is $0.25(0.02) + 0.25(0.01) = 0.0075$, total variance is 0.0375, and volatility is 19.36%. The factor accounts for 80% of variance. By asset, asset 1 contributes $0.5(1.1)(0.03)(1.0) = 0.0165$ from the factor and 0.005 from specific risk; asset 2 contributes 0.0135 and 0.0025, respectively. The four components sum to 0.0375, and asset 1's risk

share is 57.3%. Equal capital weights do not imply equal risk; factor exposures determine most of the risk here. This decomposition depends on the factor-model specification, and parameter-estimation error in loadings and factor covariance also flows into the contribution estimates.

7. Financial Interpretation: Leverage, Costs, and the Diversification Illusion

7.1 Leverage and Position-Exposure Measures

Long–short quantitative strategies must distinguish two measures of portfolio exposure:

- **Net exposure**: $\sum_{i=1}^{n} w_i$. This measures net capital or net market-value exposure; market beta exposure must be calculated separately as the sum of each weight times its asset beta. For a market-neutral strategy, net exposure is usually anchored near 0.
- **Gross exposure**: $\sum_{i=1}^{n} |w_i|$. This measures the total capital invested as a leverage multiple. For example, with net exposure equal to 0, gross exposure of 2.0 (or 200%) means that, on a net asset value of 100 million yuan, the portfolio holds 100 million yuan long and 100 million yuan short.
- **Leverage**: When weights are already normalized by net asset value, leverage is gross exposure, $\sum_i |w_i|$; do not divide by net asset value again. If the inputs are amounts, first divide the sum of the absolute long and short market values by net asset value. With asset returns fixed and risky positions scaled by a common factor, their gross-return contribution and fees under a linear proportional-cost model scale by that factor. Financing costs, market impact, and tail liquidation risk can change nonlinearly; proportional scaling is not guaranteed.

7.2 Trading Frictions, Portfolio Drift, and Turnover Drag

Over a full investment period, asset weights change for two reasons: passive price changes create "drift," while active portfolio rebalancing creates "turnover." Suppose beginning-of-period weights are $\mathbf{w}$ (with $\sum w_i = 1$) and realized returns during the period are $\mathbf{r} = (r_1, \ldots, r_n)^\top$. At the end of the period, the naturally evolved weight of asset i as a share of total portfolio value (the pre-trade drifted weight) is:

$$w_i^+ = \frac{w_i(1 + r_i)}{1 + \mathbf{w}^\top \mathbf{r}}.$$

If next period's target weights are $\mathbf{w}^{target}$, the normalized weight-turnover proxy for buys and sells combined (not divided by 2) is:

$$\text{Turnover} = \sum_{i=1}^{n} |w_i^{target} - w_i^+|.$$

If the one-way trading-cost rate is c (for example, 10 basis points, or $c = 0.0010$), the trading-cost drag for this period is $c \times \text{Turnover}$. Under the model, this proxy cost is deducted at the start of the holding period, and the target weights are applied to the remaining capital. The resulting modeled net return is:

$$r_{net} = (1 - c \cdot \text{Turnover})(1 + r_{gross}) - 1.$$

This is not an exact self-financing execution ledger: actual trade amounts also depend on post-cost net asset value, so the fee fraction q should satisfy $q = c\sum_i |(1-q)w_i^{target} - w_i^+|$. This chapter and Project 3 use $q \approx c\,\text{Turnover}$ as a low-cost approximation and do not separately charge the additional adjustment needed to fund fees; see Section 3 of the Project 3 report. Annualized average fees, $12E[q]$, differ from the annualized average gross-to-net return difference, $12E[q(1 + r_{gross})]$. Higher turnover increases costs, but whether those costs erase an advantage must be calculated; high turnover alone does not prove that a strategy loses money.

7.3 The Diversification Illusion and the Failure of Geometric Projection

Intuitively, holding more assets may appear to spread risk. But when risk contributions are extremely uneven, investors fall into the **diversification illusion**. Constructed example: stocks have 16% annualized volatility, bonds 5%, and their correlation is 0.2; capital weights are 60% stocks and 40% bonds. Let subscripts s and b denote stocks and bonds: $(\Sigma\mathbf{w})_s = 0.6(0.0256) + 0.4(0.2)(0.16)(0.05) = 0.016$, $(\Sigma\mathbf{w})_b = 0.4(0.0025) + 0.6(0.0016) = 0.00196$, and portfolio variance is $0.6(0.016) + 0.4(0.00196) = 0.010384$. Stocks account for $0.0096/0.010384 = 92.4\%$ of risk: the 40% bond allocation carries only about 7.6% of risk.

Another common conceptual error is to try to satisfy neutrality constraints using a "Euclidean projection." Suppose an alpha model produces weights $\mathbf{w}$ that do not satisfy a neutrality constraint $A\mathbf{w} = \mathbf{0}$. The unweighted Euclidean projection onto the nearest feasible point solves:

$$\min_{\mathbf{v}} \|\mathbf{v} - \mathbf{w}\|_2^2 \quad \text{s.t.} \quad A\mathbf{v} = \mathbf{0},$$

Its closed-form solution is:

$$\mathbf{v}^* = \mathbf{w} - A^\top (AA^\top)^{-1} A\mathbf{w}.$$

The serious financial flaw is that Euclidean-distance minimization takes place in an abstract mathematical space with no financial dimensions. It is completely unaware of differences in asset volatility, correlations, or liquidity frictions. When constraint rows are correlated, the projection may greatly shrink the original long and short weights (in Section 8.3, gross exposure falls from 1.6 to 0.2). Whether the projection preserves the source of expected returns must be reevaluated using a risk model and return forecasts; a more common approach is to include neutrality conditions as constraints in an optimization with a risk term.

8. Numerical Examples

This section gives five benchmark numerical examples with explicit inputs, step-by-step hand calculations, and clearly specified units.

8.1 Ledoit–Wolf Covariance-Shrinkage Example by Hand (4×2 Matrix, Percentage Units)

Complete Example: Calculating Covariance Shrinkage Step by Step

Input: The training window contains $T = 4$ observations on $p = 2$ assets. These are constructed returns for four equal-length observation periods, in percentage units per period; covariance is measured in squared percentage units. The sample has already been demeaned, and no exchange frequency is specified, so these are not monthly market observations:

$$X = \begin{bmatrix} 3.0 & 2.0 \\ -1.0 & -2.0 \\ 2.0 & 3.0 \\ -4.0 & -3.0 \end{bmatrix}.$$

Hand calculation:

1. **Sample covariance matrix** $S = \frac{1}{T} X^\top X$:

$$\begin{aligned} (X^\top X)_{11} &= 3^2 + (-1)^2 + 2^2 + (-4)^2 = 30, \\ (X^\top X)_{12} &= 3(2) + (-1)(-2) + 2(3) + (-4)(-3) = 26, \\ (X^\top X)_{22} &= 2^2 + (-2)^2 + 3^2 + (-3)^2 = 26. \end{aligned}$$

   Divide by $T = 4$:

$$S = \begin{bmatrix} 7.5 & 6.5 \\ 6.5 & 6.5 \end{bmatrix}.$$

2. **Shrinkage-target scale** m:

$$m = \frac{1}{p} \operatorname{tr}(S) = \frac{7.5 + 6.5}{2} = 7.0.$$

3. **Squared sample dispersion** $d^2 = \|S - mI\|^2$:

$$S - mI = \begin{bmatrix} 7.5 - 7.0 & 6.5 \\ 6.5 & 6.5 - 7.0 \end{bmatrix} = \begin{bmatrix} 0.5 & 6.5 \\ 6.5 & -0.5 \end{bmatrix}.$$

   Squared Frobenius norm (divided by $p = 2$):

$$d^2 = \frac{1}{2}\left(0.5^2 + 6.5^2 + 6.5^2 + (-0.5)^2\right) = \frac{1}{2}(0.25 + 42.25 + 42.25 + 0.25) = \frac{85.0}{2} = 42.5.$$

4. **Estimation-error variance,** $\bar{b}^2$: Calculate the squared norm of each row, $\|\mathbf{x}_t\|_2^2$:
   - $t = 1$: $3^2 + 2^2 = 13$, whose square is $13^2 = 169$;
   - $t = 2$: $(-1)^2 + (-2)^2 = 5$, whose square is $5^2 = 25$;
   - $t = 3$: $2^2 + 3^2 = 13$, whose square is $13^2 = 169$;
   - $t = 4$: $(-4)^2 + (-3)^2 = 25$, whose square is $25^2 = 625$. The sum of fourth powers is $\sum_{t=1}^{4} \|\mathbf{x}_t\|_2^4 = 169 + 25 + 169 + 625 = 988$. Sum of squared sample-matrix entries:

$$\sum_{i,j} s_{ij}^2 = 7.5^2 + 6.5^2 + 6.5^2 + 6.5^2 = 56.25 + 3(42.25) = 56.25 + 126.75 = 183.0.$$

   Substitute into the formula to calculate $\bar{b}^2$:

$$\bar{b}^2 = \frac{988 - 4(183.0)}{T^2 p} = \frac{988 - 732}{4^2 \times 2} = \frac{256}{32} = 8.0.$$

Continued: the same complete example

5. **Shrinkage intensity and shrunk estimator**:

$$b^2 = \min(\max(8.0, 0), 42.5) = 8.0 \implies \hat{\kappa} = \frac{b^2}{d^2} = \frac{8.0}{42.5} = \frac{16}{85} \approx 0.18823529.$$

Calculate the shrunk covariance matrix $S^* = \hat{\kappa}(7.0I) + (1 - \hat{\kappa})S$:

- $S^*_{11} = \frac{16}{85}(7.0) + \frac{69}{85}(7.5) = \frac{112+517.5}{85} = \frac{629.5}{85} \approx 7.40588235$;
- $S^*_{12} = S^*_{21} = \frac{69}{85}(6.5) = \frac{448.5}{85} \approx 5.27647059$;
- $S^*_{22} = \frac{16}{85}(7.0) + \frac{69}{85}(6.5) = \frac{112+448.5}{85} = \frac{560.5}{85} \approx 6.59411765$。

6. **Financial comparison**:
   - Sample correlation coefficient: $\rho = \frac{6.5}{\sqrt{7.5 \times 6.5}} = \sqrt{\frac{13}{15}} \approx 0.930949$;
   - Correlation after shrinkage: $\rho^* = \frac{5.27647059}{\sqrt{7.40588235 \times 6.59411765}} \approx \frac{5.27647059}{\sqrt{48.8352595156}} \approx \frac{5.27647059}{6.9882229154} \approx 0.755052$。The correlation falls from the ill-conditioned 0.931 to 0.755, substantially improving the matrix condition number.

8.2 Hand Calculation of Drift, Turnover, and the Cost Ledger

Complete Example: Drift, Turnover, and Proxy Costs

Input: The two-asset portfolio starts with equal weights, $\mathbf{w} = (0.50, 0.50)$. Monthly returns are $r_1 = +10.0\%$ (0.10) and $r_2 = -10.0\%$ (−0.10). The next-period target remains equal-weighted, $\mathbf{w}^{target} = (0.50, 0.50)$. The one-way trading-cost rate is $c = 10$ bps $= 0.0010$.

Hand calculation:

1. Portfolio gross return: $r_{gross} = 0.5(0.10) + 0.5(-0.10) = 0.0$;
2. Drifted weights:

$$w_1^+ = \frac{0.5(1 + 0.10)}{1 + 0.0} = 0.55, \quad w_2^+ = \frac{0.5(1 - 0.10)}{1 + 0.0} = 0.45;$$

3. Turnover:

$$\text{Turnover} = |0.50 - 0.55| + |0.50 - 0.45| = 0.05 + 0.05 = 0.10 \quad (10\% \text{ NAV});$$

4. Trading-cost drag: cost $= 0.0010 \times 0.10 = 0.0001$ (1 basis point);
5. Net return after costs:

$$r_{net} = (1 - 0.0001)(1 + 0.0) - 1 = -0.0001 \quad (-0.01\%).$$

8.2.1 Checking Exact Self-Financing in Currency Amounts

Complete Example: Amounts Balance After Fees

The target in the previous example is unchanged, but trades still occur. Pre-trade net asset value is $V^- = 100$, measured in units of ten thousand yuan (1 million yuan in total), with existing holdings of 550,000 and 450,000 yuan. After costs, net asset value is $V^+ = (1-q)V^-$; if the post-cost target remains 50/50, the new holdings are $50(1-q)$ / $50(1-q)$ ten-thousand yuan. Since fees are paid from within the portfolio, trade differences must be calculated from these new amounts:

$$q = c\sum_i |(1-q)w_i^{target} - w_i^+|.$$

In this example, with $c = 0.001$ and $q < 0.1$, sales are $0.05 + 0.5q$ of pre-trade net asset value and purchases are $0.05 - 0.5q$, still totaling 0.10. Thus the exact value is $q = 0.0001$. The fee is 100 yuan, leaving new holdings of 499,950 yuan each; 50,050 yuan is sold and 49,950 yuan is purchased, so the 100-yuan difference pays the fee exactly. The proxy and exact formula happen to agree in this symmetric example; they do not generally agree.

Now change the target so all capital is allocated to asset 1: $w^{target} = (1, 0)$. Selling asset 2 accounts for 0.45 of pre-trade net asset value, while buying asset 1 accounts for $0.45 - q$, so

$$q = 0.001(0.90 - q), \qquad q = \frac{0.0009}{1.001} = 0.0008991009.$$

The exact fee is 899.1009 yuan. The proxy charges 900 yuan using turnover of 0.90 based on pre-trade net asset value, an overstatement of 0.8991 yuan. Project 3 reuses the proxy; this exact hand calculation must not be described as a recomputed Project 3 result.

For a fully invested long-only target, $\sum_i w_i^{target} = 1$. The mapping $F(q) = c\sum_i |(1-q)w_i^{target} - w_i^+|$ has a Lipschitz constant no greater than c, by the triangle inequality for absolute values. When $0 \le c < 1/2$, $F(0) \le 2c < 1$ and $F(1) = c < 1$. A continuous convex function on [0,1] reaches its maximum at an endpoint, so F maps this interval into itself; the contraction mapping theorem gives a unique nonnegative fee solution. This result assumes no external cash flows, one common proportional fee rate, a fully invested long-only target, and positive net asset value. It does not cover actual market impact, financing, or different buy and sell rates.

These three layers of accounting answer different questions: drifted weights explain why a trade occurs; the proxy makes fixed specifications easier to compare; and the exact equation conserves capital under the specified proportional-fee model. Project 3 has no orders, market-impact estimates, or execution logs, so neither equation establishes live trading costs.

8.3 Implications Among the Three Neutralities and a Geometric-Projection Failure Example

Complete Example: Neutrality Constraints and Independent Constraint Rows

Input: The four-asset position vector is $\mathbf{w} = (0.5, 0.3, -0.4, -0.4)^\top$. Market betas are $\boldsymbol{\beta} = (1.2, 0.8, 1.0, 0.6)^\top$. The assets belong to sectors $[A, B, A, B]$, respectively. **Hand check**:

- Net dollar exposure: $0.5 + 0.3 - 0.4 - 0.4 = 0.0$ (dollar neutral);
- Gross exposure: $|0.5| + |0.3| + |-0.4| + |-0.4| = 1.6$ (160% leverage);
- Beta exposure: $0.5(1.2) + 0.3(0.8) - 0.4(1.0) - 0.4(0.6) = 0.60 + 0.24 - 0.40 - 0.24 = 0.20 \neq 0$ (not beta neutral);
- Net sector exposure: Sector A (assets 1 and 3): $0.5 + (-0.4) = +0.10 \neq 0$; Sector B (assets 2 and 4): $0.3 + (-0.4) = -0.10 \neq 0$ (not sector neutral). These values disprove the assumption that dollar neutrality automatically implies beta or sector neutrality. Now apply the unweighted Euclidean projection $\mathbf{v} = \mathbf{w} - A^\top (AA^\top)^{-1} A\mathbf{w}$ using a constraint matrix A for the three neutralities above. Use the independent constraint rows $A = [\mathbf{1}^\top; \boldsymbol{\beta}^\top; Z_A]$, where $Z_A = (1, 0, 1, 0)$. The dollar-neutrality and sector-A rows already imply the sector-B row. Adding $Z_B = (0, 1, 0, 1)$ would make the rows dependent because $\mathbf{1}^\top = Z_A + Z_B$, rendering $AA^\top$ non-invertible. The redundant row must be removed; the four-row matrix cannot be inverted directly. The projection weights for the three-row independent version are:

$$\mathbf{v}^* = (0.05, -0.05, -0.05, 0.05)^\top.$$

Gross exposure falls sharply from 1.6 to 0.20 (a reduction of 87.5%). In this example, the Euclidean-nearest point satisfying all neutrality conditions greatly shrinks total exposure. This is a consequence of these particular constraints and holdings; it does not show that every geometric projection fails. The projection minimizes weight distance, not risk or trading costs, so its suitability depends on the research objective.

8.4 Four-Asset Risk-Budget Example from Maillard, Roncalli, and Teiletche (2009)

Complete Example: Four-Asset Risk Budgeting

Input: The vector of annualized volatilities for four assets is $\boldsymbol{\sigma} = [0.10, 0.20, 0.30, 0.40]^\top$. The correlation matrix is:

$$C = \begin{bmatrix} 1.0 & 0.8 & 0.0 & 0.0 \\ 0.8 & 1.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 1.0 & -0.5 \\ 0.0 & 0.0 & -0.5 & 1.0 \end{bmatrix}.$$

The covariance matrix is $\Sigma = \operatorname{diag}(\boldsymbol{\sigma}) C \operatorname{diag}(\boldsymbol{\sigma})$. **Results and features**:

1. **Equal-weighted portfolio (1/N, $\mathbf{w} = [0.25, 0.25, 0.25, 0.25]^\top$)**:
   - Total portfolio volatility: $\sigma = 11.51\%$ (0.115109);
   - Marginal risk contributions: MRC $= [0.0565, 0.1216, 0.0652, 0.2172]^\top$;

Continued: the same complete example

- Relative risk shares: $p = [12.26\%, 26.42\%, 14.15\%, 47.17\%]^\top$. Asset 4, the most volatile asset, accounts for almost half of total risk (47.17%), revealing substantial nominal diversification but actual concentration.

2. **Long-only minimum-variance portfolio (Long-Only Min Var)**:
   - Weight vector: $\mathbf{w} = [74.48\%, 0.00\%, 15.17\%, 10.34\%]^\top$;
   - Total portfolio volatility: $\sigma = 8.63\%$ (0.086303);
   - Relative risk shares: exactly equal to the weights, $[74.48\%, 0.00\%, 15.17\%, 10.34\%]^\top$. Marginal risk contributions are identical across held assets (all equal to 0.086303). The unheld asset 2 has a marginal risk contribution above portfolio volatility (0.138085), satisfying the KKT complementary-slackness condition.
3. **Equal risk contribution portfolio (ERC)**:
   - Weight vector: $\mathbf{w} = [38.36\%, 19.18\%, 24.26\%, 18.20\%]^\top$;
   - Total portfolio volatility: $\sigma = 10.29\%$ (0.102934);
   - Relative risk shares: exactly 25.00% per asset ($p = [0.25, 0.25, 0.25, 0.25]^\top$).
4. **Checking the monotonicity theorem**:

$$\sigma_{mv}(8.63\%) \leq \sigma_{erc}(10.29\%) \leq \sigma_{1/n}(11.51\%).$$

This agrees with the mathematical ordering theorem of Maillard et al. (2009).

8.5 Black–Litterman Two-Asset Example by Hand

Complete Example: View Update and Unconstrained Holdings

Input: The annualized covariance matrix for the two assets is $\Sigma = \begin{bmatrix} 0.04 & 0.02 \\ 0.02 & 0.09 \end{bmatrix}$. Market benchmark weights are $\mathbf{w}_{mkt} = (0.60, 0.40)^\top$, and the aggregate market risk-aversion coefficient is $\delta = 2.5$. The uncertainty-scaling coefficient is $\tau = 0.05$. The subjective view is an absolute view on asset 1, with an expected annualized excess return of 10.0% (0.10). Thus, $P = [1.0, 0.0]$ and $Q = [0.10]$. Calibrate the view covariance using the He–Litterman proportion: $\Omega = \tau P \Sigma P^\top = 0.05(0.04) = 0.002$. **Hand calculation**:

1. **Reverse-optimization equilibrium return, $\mathbf{\Pi} = \delta \Sigma \mathbf{w}_{mkt}$**:

$$\Sigma \mathbf{w}_{mkt} = \begin{bmatrix} 0.04(0.6) + 0.02(0.4) \\ 0.02(0.6) + 0.09(0.4) \end{bmatrix} = \begin{bmatrix} 0.024 + 0.008 \\ 0.012 + 0.036 \end{bmatrix} = \begin{bmatrix} 0.032 \\ 0.048 \end{bmatrix}.$$

Multiply by $\delta = 2.5$:

$$\mathbf{\Pi} = 2.5 \begin{bmatrix} 0.032 \\ 0.048 \end{bmatrix} = \begin{bmatrix} 0.08 \\ 0.12 \end{bmatrix} \quad \text{(asset 1: 8.0\% annually; asset 2: 12.0\% annually).}$$

2. **Calculating the middle term and gain matrix**:

$$P\tau\Sigma P^\top + \Omega = 0.05(0.04) + 0.002 = 0.002 + 0.002 = 0.004.$$

Continued: the same complete example

Gain vector:

$$M = \tau\Sigma P^\top (0.004)^{-1} = 0.05 \begin{bmatrix} 0.04 \\ 0.02 \end{bmatrix} \frac{1}{0.004} = \begin{bmatrix} 0.002 \\ 0.001 \end{bmatrix} \times 250 = \begin{bmatrix} 0.50 \\ 0.25 \end{bmatrix}.$$

3. **Posterior expected-return mean**: View excess: $Q - P\boldsymbol{\Pi} = 0.10 - 0.08 = 0.02$.

$$\boldsymbol{\mu}_{BL} = \begin{bmatrix} 0.08 \\ 0.12 \end{bmatrix} + \begin{bmatrix} 0.50 \\ 0.25 \end{bmatrix} (0.02) = \begin{bmatrix} 0.08 + 0.010 \\ 0.12 + 0.005 \end{bmatrix} = \begin{bmatrix} 0.090 \\ 0.125 \end{bmatrix} \quad (9.0\%, 12.5\%).$$

4. **Posterior-mean covariance matrix**:

$$V_{BL} = \tau\Sigma - MP\tau\Sigma = \begin{bmatrix} 0.002 & 0.001 \\ 0.001 & 0.0045 \end{bmatrix} - \begin{bmatrix} 0.50 \\ 0.25 \end{bmatrix} [0.002, 0.001] = \begin{bmatrix} 0.0010 & 0.0005 \\ 0.0005 & 0.00425 \end{bmatrix}.$$

5. **Optimal unconstrained weights, $\mathbf{w}^* = (\delta\Sigma)^{-1}\boldsymbol{\mu}_{BL}$**:

$$\delta\Sigma = 2.5 \begin{bmatrix} 0.04 & 0.02 \\ 0.02 & 0.09 \end{bmatrix} = \begin{bmatrix} 0.10 & 0.05 \\ 0.05 & 0.225 \end{bmatrix}.$$

Determinant: $\det = 0.10(0.225) - 0.05^2 = 0.0225 - 0.0025 = 0.020$. Inverse matrix:

$$(\delta\Sigma)^{-1} = \frac{1}{0.020} \begin{bmatrix} 0.225 & -0.05 \\ -0.05 & 0.10 \end{bmatrix} = \begin{bmatrix} 11.25 & -2.50 \\ -2.50 & 5.00 \end{bmatrix}.$$

Calculate the weights:

$$\mathbf{w}^* = \begin{bmatrix} 11.25(0.090) - 2.50(0.125) \\ -2.50(0.090) + 5.00(0.125) \end{bmatrix} = \begin{bmatrix} 1.0125 - 0.3125 \\ -0.225 + 0.625 \end{bmatrix} = \begin{bmatrix} 0.70 \\ 0.40 \end{bmatrix}.$$

Key insight: Asset 2 is not directly included in the view, but its positive covariance with asset 1 (0.02) raises its posterior expected return from 12.0% to 12.5%. At the portfolio-allocation level, however, asset 2's optimal holding stays exactly at its market weight of 0.40. The entire excess tilt (+0.10) goes to the asset named in the view, asset 1, whose weight increases from 0.60 to 0.70. This confirms the orthogonality of view tilts in the He–Litterman model. These unconstrained risky-asset weights sum to 1.10, so the cash weight is −0.10, meaning that 10% of net asset value is borrowed. If borrowing is not allowed, the problem must be re-solved with a budget constraint; the weights cannot simply be normalized and still described as the same optimum.

Black–Litterman View Confidence and Portfolio-Weight Curves

Continued: the same complete example

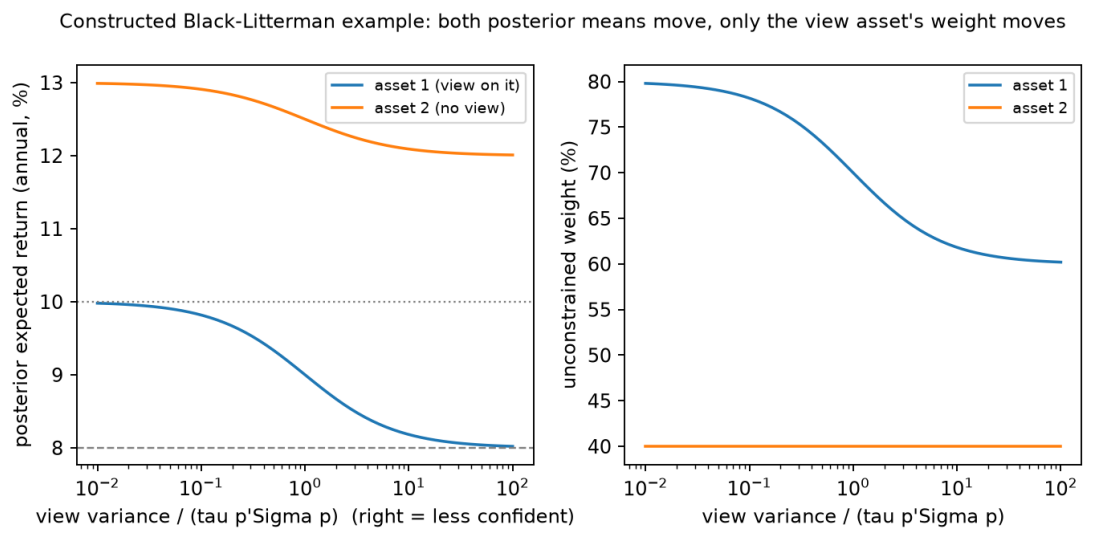


Figure 4: Black–Litterman view confidence and portfolio weights

- **Horizontal axis**: View-variance scaling multiplier $\Omega/(\tau P\Sigma P^\top)$ (log scale; moving right means lower confidence and much greater view variance);
- **Vertical axis**: Posterior expected return (annualized %) in the left panel and optimal unconstrained portfolio weights (%) in the right panel;
- **Units**: Percent (%);
- **How to read the figure and what to look for**:
  1. In the left panel, when confidence is very high (far left), asset 1's posterior expected return approaches the subjective view of 10.0%, and asset 2 approaches 13.0%. As confidence fades to the right, the two expected returns smoothly return to their prior equilibrium values of 8.0% and 12.0%;
  2. In the right panel, asset 2, which is not included in the view, stays at a constant optimal weight of 40.0%, regardless of view confidence. Asset 1 bears the full tactical adjustment, smoothly declining from 80% to its benchmark weight of 60%;
- **Limitation**: This is a constructed two-asset teaching model and does not consider interaction effects from multiple non-orthogonal views or constrained quadratic programming.

9. Python Implementation and Assertion Checks

Default Standalone Run Entry Point

From the root of the companion reader package, create an isolated environment following the package instructions and run:

```
.venv/bin/python run_notebook.py notebooks/ch21_portfolio_construction.ipynb
```

By default, `RUN_FRENCH=False`. The Notebook checks LW shrinkage, MRT four-asset risk, BL, Kelly, neutrality constraints, and drift in sequence, and checks boundary cases such as finite Sharpe ratios, ERC existence, posterior-mean covariance, and the quadratic growth proxy. It creates only the chapter's `RUN_DIR`, writes no CSV or PNG files, and skips the historical figures. Figures in Sections 3, 5.4, 6.5, 8.5, and 11 are existing reader materials, not outputs of this run. The chapter run directory differs from the separate `runs/execution_ch21_portfolio_construction_<uuid>/` directory created by the runner, which stores an execution copy and status.

This chapter's 21-E08 uses a monthly excess mean of 0.008 and variance of 0.0025, giving a quadratic-proxy weight of 3.2. The Notebook instead uses 0.006 and 0.0016, giving a weight of 3.75; these are different constructed inputs. The exact binary half-Kelly growth ratio is about 74.9268%; the quadratic growth proxy gives 75% at half of its own optimal risky-asset weight, and this ratio should not be treated as a general result for exact binary growth. The neutrality constraint should be assessed as described in Section 5.2: absolute sector neutrality across mutually exclusive and exhaustive sectors implies dollar neutrality and generally does not imply beta neutrality.

For Project 3's standalone entry point, constructed outputs, and historical report, see the project guide. The call and assertions for the shared implementation are shown below. The course workspace root must have the course package and its dependencies installed; a bare Python interpreter in an unconfigured environment is not a complete reader entry point.

Calling the Shared Implementation

The code below shows how to call the core module `src/quant_course/portfolio_construction.py` and uses assertions to check Sections 8.1–8.3 and 8.5. Full code for the four-asset ERC example in Section 8.4 is in the answer to 21-P01. The code requires the course package and its dependencies.

```
import numpy as np
from quant_course.portfolio_construction import (
    ledoit_wolf_identity, risk_contributions, risk_budget_weights,
    max_sharpe_long_only, black_litterman, he_litterman_view_covariance,
    kelly_binary_fraction, binary_log_growth, gaussian_kelly_weights,
    quadratic_log_growth, exposure_report, project_to_constraints,
    drift_weights, turnover,
)

# 1. Ledoit-Wolf 4x2 percentage-matrix shrinkage hand-check
x_hand = np.array([[3.0, 2.0], [-1.0, -2.0], [2.0, 3.0], [-4.0, -3.0]])
lw_res = ledoit_wolf_identity(x_hand)
assert np.isclose(lw_res.target_scale, 7.0)
assert np.isclose(lw_res.intensity, 8.0 / 42.5)
np.testing.assert_allclose(lw_res.sample_covariance, [[7.5, 6.5], [6.5, 6.5]], atol=1
    ↪ e-12)
np.testing.assert_allclose(
    lw_res.covariance,
    [[7.40588235, 5.27647059], [5.27647059, 6.59411765]],
    atol=1e-6,
)

# 2. Black-Litterman two-asset hand-check
cov_bl = np.array([[0.04, 0.02], [0.02, 0.09]])
w_mkt = np.array([0.6, 0.4])
p_view = np.array([[1.0, 0.0]])
q_view = np.array([0.10])
tau_param = 0.05
omega_view = he_litterman_view_covariance(p_view, cov_bl, tau=tau_param)
bl_res = black_litterman(
    cov_bl, w_mkt, risk_aversion=2.5, tau=tau_param,
    view_matrix=p_view, view_returns=q_view, view_covariance=omega_view,
)
np.testing.assert_allclose(bl_res.equilibrium_returns, [0.08, 0.12], atol=1e-12)
np.testing.assert_allclose(bl_res.posterior_mean, [0.09, 0.125], atol=1e-12)
np.testing.assert_allclose(bl_res.weights, [0.7, 0.4], atol=1e-12)

# 3. Kelly log-growth hand-check (p=0.55, b=1)
f_star = kelly_binary_fraction(0.55, 1.0)
assert np.isclose(f_star, 0.10)
g_full = binary_log_growth(0.10, 0.55, 1.0)
g_half = binary_log_growth(0.05, 0.55, 1.0)
g_double = binary_log_growth(0.20, 0.55, 1.0)
assert np.isclose(g_full, 0.0050083668, atol=1e-6)
assert np.isclose(g_half, 0.0037526078, atol=1e-6)
assert g_double < 0  # Double Kelly: expected log growth is negative; long-run wealth
    ↪  almost surely tends to 0

# 4. Neutrality and projection checks
w_demo = np.array([0.5, 0.3, -0.4, -0.4])
beta_demo = np.array([1.2, 0.8, 1.0, 0.6])
sectors_demo = ["A", "B", "A", "B"]
rep = exposure_report(w_demo, beta_demo, sectors_demo)
assert np.isclose(rep.net, 0.0)
```

```
assert np.isclose(rep.beta, 0.2)
a_mat = np.vstack([np.ones(4), beta_demo, [1.0, 0.0, 1.0, 0.0]])
proj_w = project_to_constraints(w_demo, a_mat)
np.testing.assert_allclose(a_mat @ proj_w, 0.0, atol=1e-12)

# 5. Drift and turnover ledger checks
w_init = np.array([0.5, 0.5])
r_month = np.array([0.10, -0.10])
w_drift = drift_weights(w_init, r_month)
np.testing.assert_allclose(w_drift, [0.55, 0.45], atol=1e-12)
turnover_val = turnover(w_drift, w_init)
assert np.isclose(turnover_val, 0.10, atol=1e-12)
```

10. Visualization

This section examines four core charts generated in advance. All figures were produced from fixed seeds and frozen data.

The figures appear near the relevant discussion: the estimation-error simulation in Section 3, Kelly growth in Section 6.5, BL view confidence in Section 8.5, and French industry risk shares in Section 11. Their original inputs, axes, units, and limitations are retained. The new two-asset figure in Section 5.4 distinguishes capital shares from risk shares.

11. Empirical Example Using Real-Market Data

The empirical example uses a frozen snapshot from the Kenneth R. French Data Library:

- **Asset universe**: Value-weighted monthly returns for 10 U.S. equity industries (NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils, Other), plus the Fama–French three-factor risk-free rate (RF);
- **Sample period**: 1926-07 to 2026-08 (1,202 calendar months);
- **Raw-file hash**: SHA-256 for `industry_raw_zip`: `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`;
- **Data status**: The French data are a currently revised academic version, not a point-in-time (PIT) historical snapshot. Monthly data are published with a lag, historical values may be revised, and the local manifest does not preserve historical versions. A zero-month lag is an idealized month-boundary decision test.

For the 120-month period from 2010-01 to 2019-12, the observed performance of four representative strategies using the Ledoit–Wolf covariance estimator is recorded in `outputs/ch21/ch21_results.json`:

Portfolio strategy	Monthly volatility	Highest-weight industries and weights	Lowest-weight industries and weights	Risk-diversification characteristics
Equal-weighted portfolio (1/N)	3.51%	Each industry at 10.0%	Each industry at 10.0%	Uneven risk shares (Durbl 15.1% vs Utils 4.5%)
Long-only minimum variance	2.73%	Utils (47.45%), NoDur (23.31%)	Durbl, Manuf, Enrgy, HiTec (0.0%)	Highly concentrated in low-volatility defensive assets; 6 industries have zero weight
Long-only maximum Sharpe	2.88%	Utils (36.08%), Shops (27.40%)	NoDur, Durbl, Manuf, Enrgy, Other (0.0%)	Chases assets with high historical returns; 5 industries have zero weight
Equal risk contribution (ERC)	3.24%	Utils (17.62%), NoDur (12.42%)	Durbl (6.41%), Enrgy (7.32%)	Nonzero holdings in all industries; risk share is exactly 10.0% each

The data show that the long-only minimum-variance portfolio has the lowest volatility but pays for it with highly concentrated holdings (the two largest industries account for more than 70%). Under the fully invested constraint, the ERC portfolio balances volatility reduction with broader market coverage.

Risk Decomposition Across the French 10 Industries in the Historical Window

- **Horizontal axis**: French 10 industry codes (NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils, Other);
- **Vertical axis**: Nominal portfolio weights (%) in the upper panel and relative risk shares (%) in the lower panel;
- **Units**: Percent (%);
- **Sample**: 2010-01 to 2019-12 (120 months), using a Ledoit–Wolf shrunk covariance matrix;
- **How to read the figure and what to look for**:
  1. In the equal-weighted portfolio (1/N), every industry has a weight of exactly 10.0%, but utilities (Utils) account for only 4.52% of risk in the lower panel, while durable goods (Durbl) and energy (Enrgy) account for 15.09% and 12.97%, respectively;
  2. The long-only minimum-variance portfolio concentrates its weights in defensive industries (Utils 47.45%, NoDur 23.31%, and Hlth 15.27%); its risk shares match its weights exactly;
  3. In the ERC portfolio, each industry's risk share in the lower panel is 10.0%. To compensate for the low volatility of Utils, ERC raises its portfolio weight to 17.62% and reduces Durbl to 6.41%;
  4. Monthly volatility comparison: minimum variance (2.73%) $<$ maximum Sharpe (2.88%) $<$ ERC (3.24%) $<$ 1/N (3.51%);

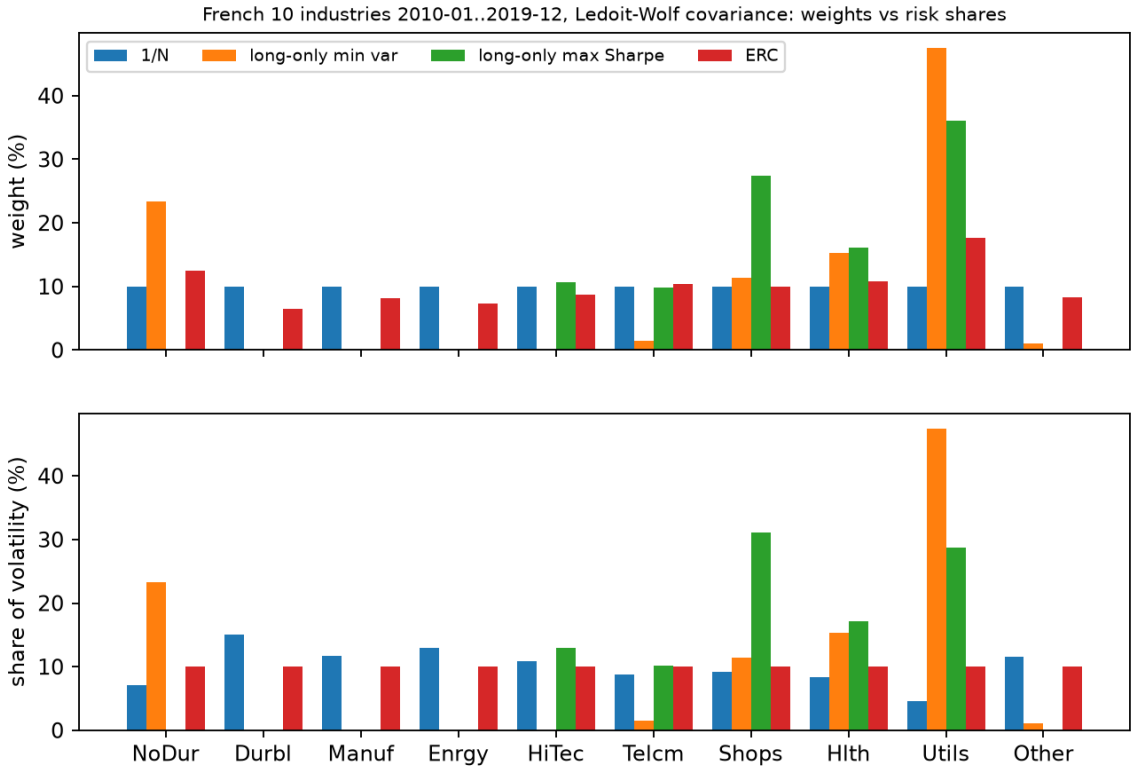


Figure 5: French 10-industry weights and risk-share panel

- **Limitation**: This figure is a static, ex-post description of one historical window, 2010–2019. It is not a rolling out-of-sample result or a live-strategy conclusion.

12. Common Mistakes and Debugging

In quantitative portfolio-construction work, researchers often encounter the following pitfalls because of floating-point precision and the interaction between solver implementations and financial constraints:

12.1 A Covariance Matrix on a Small Numerical Scale Can Produce a False Solver-Tolerance Failure

- **Symptom**: When calculating a long-only minimum-variance portfolio, pass a monthly covariance matrix directly to a quadratic-programming solver (its elements are often on the order of $10^{-3} \sim 10^{-4}$). The solver reports `optimal`, but independently checking the original constraints sometimes finds a budget residual $|\sum w_i - 1|$ above the default tolerance of 10^{-6}, so the quality-control module rejects the result as designed.
- **Possible cause**: Solver stopping rules depend on residual and duality-gap tolerances that are scaled to the problem. When quadratic coefficients are very small, the objective is insensitive to weight deviations, so the solver may report convergence before the budget residual falls below 10^{-6}. This section records the symptom and fix; it does not diagnose each internal CLARABEL criterion.

- **Fix**: Before passing the problem to the mean–variance solver, normalize the covariance matrix by its largest absolute entry: $\tilde{\Sigma} = \Sigma / \max_{i,j} |\Sigma_{ij}|$. Because the mean is zero in the pure minimum-variance problem, multiplying the entire objective by a positive scalar does not change the minimizer $\mathbf{w}^*$, but it brings the quadratic gradient back to a well-scaled $O(1)$ range.

12.2 False Non-Convergence Stagnation in Damped Newton Log-Barrier Iterations

- **Symptom**: When solving the ERC log-barrier objective, setting the line-search tolerance near floating-point machine precision (for example, 10^{-12}) can make the algorithm stall near the optimum and falsely report "maximum iteration count exceeded without convergence."
- **Mechanism**: When the objective is on the scale of $O(1)$, double-precision floating-point arithmetic cannot resolve a decrease on the order of 10^{-16}, and Armijo backtracking shrinks the step size to machine zero.
- **Fix**: Following Boyd & Vandenberghe (2004), Section 9.5, use the Newton decrement $\lambda^2 = -\nabla f(\mathbf{y})^\top \Delta\mathbf{y} \le 10^{-14}$ as the convergence criterion, then check the absolute deviation in risk shares after solving ($\max_i |p_i - b_i| \le 10^{-6}$).

12.3 Missing a Feasibility Check for Maximum Sharpe with Weight Caps

- **Symptom**: The only check is whether any asset has positive excess return; if so, the convex QP is constructed and sent to the solver. With a per-asset cap such as $w_{\max} = 0.4$, CLARABEL may report `infeasible`.
- **Mechanism**: A single asset with positive excess return does not imply that the simplex contains a portfolio with positive excess return. If the only asset with a positive excess return is capped at 40%, the remaining 60% of capital must go to assets with negative excess returns, so the maximum excess return on the simplex may still be strictly negative.
- **Fix**: Before constructing the QP, use a greedy fill in descending order of excess return to calculate the theoretical maximum excess return on the capped simplex. If the maximum is non-positive, raise an explanatory `OptimizationError` in advance.

The code below independently verifies the scale-normalization fix and the feasibility precheck for weight caps:

```
import numpy as np
from quant_course.optimization_tools import OptimizationError, solve_mean_variance
from quant_course.portfolio_construction import max_sharpe_long_only

# 1. Covariance scale-normalization check
cov_tiny = np.array([[0.0004, 0.0001], [0.0001, 0.0009]])
cov_scaled = cov_tiny / float(np.max(np.abs(cov_tiny)))
res_scaled = solve_mean_variance(np.zeros(2), cov_scaled)
np.testing.assert_allclose(res_scaled.weights, [8 / 11, 3 / 11], atol=1e-6)

# 2. Maximum-Sharpe cap feasibility precheck
mu_infeasible = np.array([0.004, -0.004, -0.004])
cov_diag = np.diag([0.01, 0.01, 0.01])
try:
```

```
    max_sharpe_long_only(
        mu_infeasible, cov_diag, risk_free_return=0.0, max_weight=0.4,
    )
except OptimizationError as exc:
    assert exc.solver_status == "infeasible_by_inputs"
```

13. Quantitative Research in Practice: Project 3 Rolling Out-of-Sample Evaluation

As an end-to-end engineering check of quantitative portfolio construction, `projects/project03/src/project03_backtest.py` implements a rolling out-of-sample evaluation system covering 1990-01 to 2026-08 (440 evaluation months).

13.1 Backtest Design and Parameter Baseline

- **Primary backtest specification**:
  - Training window: rolling estimation with $W = 60$ months;
  - Covariance estimator: Ledoit–Wolf shrinkage toward a scaled identity matrix;
  - Constraints: long-only ($w_i \geq 0$); single-asset cap $w_{\max} = 40\%$ for minimum variance and maximum Sharpe; 10% per asset for equal weight; no cap for ERC (the sample maximum is about 19.83%);
  - Rebalance timing and lag: zero-month lag (optimize with the then-current cross-section of moments and rebalance at the end of month $t-1$);
  - Trading-cost baseline: total two-sided turnover multiplied by 10 bps ($c = 0.0010$).
- **Strategies compared**:
  1. `equal_weight`: equal-weighted 1/N each month (baseline);
  2. `min_variance`: long-only minimum-variance portfolio with a weight cap;
  3. `max_sharpe`: long-only maximum-Sharpe portfolio with a weight cap;
  4. `erc`: equal risk contribution portfolio.

13.2 Evaluation Results and Statistical Inference

Based on `projects/project03/outputs/summary.json` and sensitivity.csv, the core performance statistics over 440 months are:

Evaluation metric	Equal weight (1/N)	Minimum variance (Min Var)	Maximum Sharpe (Max Sharpe)	Equal risk contribution (ERC)
Annualized net excess return	9.05%	7.03%	8.01%	8.66%
Annualized net volatility	14.32%	11.78%	14.36%	13.37%
Annualized net Sharpe ratio	0.631	0.596	0.558	0.648
Annualized gross Sharpe ratio	0.634	0.605	0.576	0.650
Compound annual growth rate (CAGR)	11.25%	9.42%	10.11%	10.98%
Maximum drawdown	-48.09%	-36.41%	-41.49%	-45.52%
Average monthly turnover	2.58%	8.60%	21.45%	2.80%
Annualized average fees as share of prior NAV	3.36 bps	10.57 bps	25.96 bps	3.63 bps
Annualized average gross-to-net return difference (cost drag)	3.37 bps	10.65 bps	26.20 bps	3.64 bps
Average effective number of assets (N_{eff})	10.00	4.03	3.49	9.31
Failures triggering fallback	0	0	1	0

- **Horizontal axis**: Backtest month (1990-01 to 2026-08); **vertical axis**: cumulative net wealth from an initial investment of 1 yuan (log scale);
- **Main observations**:
  1. No optimized strategy credibly beats the naive equal-weighted 1/N portfolio on net returns. 1/N has the highest annualized net excess return (9.05%) and the highest ending net wealth;
  2. In this sample, the minimum-variance portfolio reduces full-period volatility (11.78% vs 14.32%) and sharply narrows maximum drawdown from −48.09% to −36.41%, while its annualized average net return is lower than equal weight. These are joint observations from the same historical sample and do not identify a causal mechanism in which lower risk causes lower returns. Volatility and drawdown differences are descriptive; no confidence intervals are available for those differences, so they cannot be

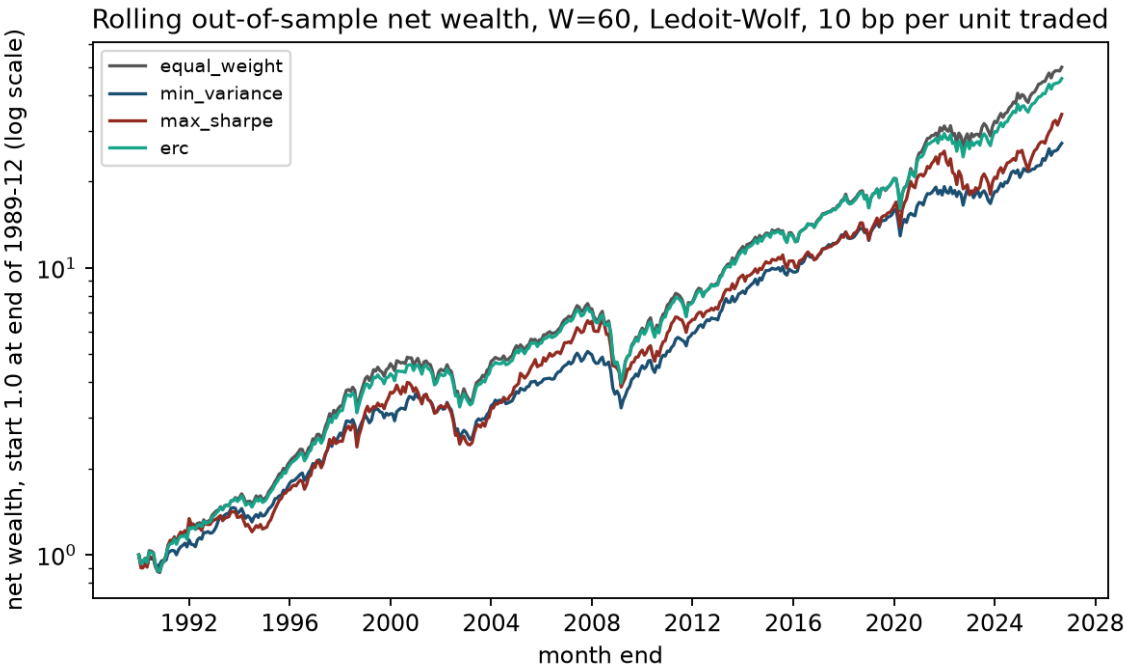


Figure 6: Project 3 net-wealth comparison

called statistically significant. The average-return difference has a separate, unadjusted single-comparison interval in Section 13.3;

3. The maximum-Sharpe strategy uses rolling sample means and has average monthly turnover of 21.45% (8.3 times that of equal weight), with an annualized average gross-to-net return difference of 26.20 basis points (average fees equal 25.96 basis points of prior NAV). After costs, its net Sharpe ratio is only 0.558, the lowest of the four strategies.

13.3 Statistical Inference (Paired Circular Block Bootstrap)

To assess the statistical significance of strategy-return and Sharpe differences, Project 3 uses 5,000 circular moving-block bootstrap samples (12-month blocks, seed 20261006). Sharpe differences use paired resampling:

- **Minimum variance vs equal weight**: The monthly average-return difference is -0.168%, with a 95% confidence interval of $[-0.331\%, -0.005\%]$. Only 2.22% of resampled differences are above zero, so the main-specification average-return difference has an **unadjusted single-comparison interval below zero**. No multiple-comparison adjustment was made across three strategies and two metrics, so this is not a family-wise significance result;
- **Maximum Sharpe vs equal weight**: The monthly average-return difference is -0.086%, with a 95% confidence interval of $[-0.275\%, +0.106\%]$, which includes 0. The Sharpe-ratio difference is -0.073, with a 95% interval of $[-0.236, +0.089]$; there is no statistical advantage;
- **ERC vs equal weight**: Although ERC's net Sharpe ratio (0.648) is slightly higher than that of equal weight (0.631), their paired Sharpe difference (+0.016) has a 95% confidence

interval of $[-0.014, +0.050]$, which clearly includes 0. Therefore, ERC cannot be claimed to be statistically significantly better than equal weight.

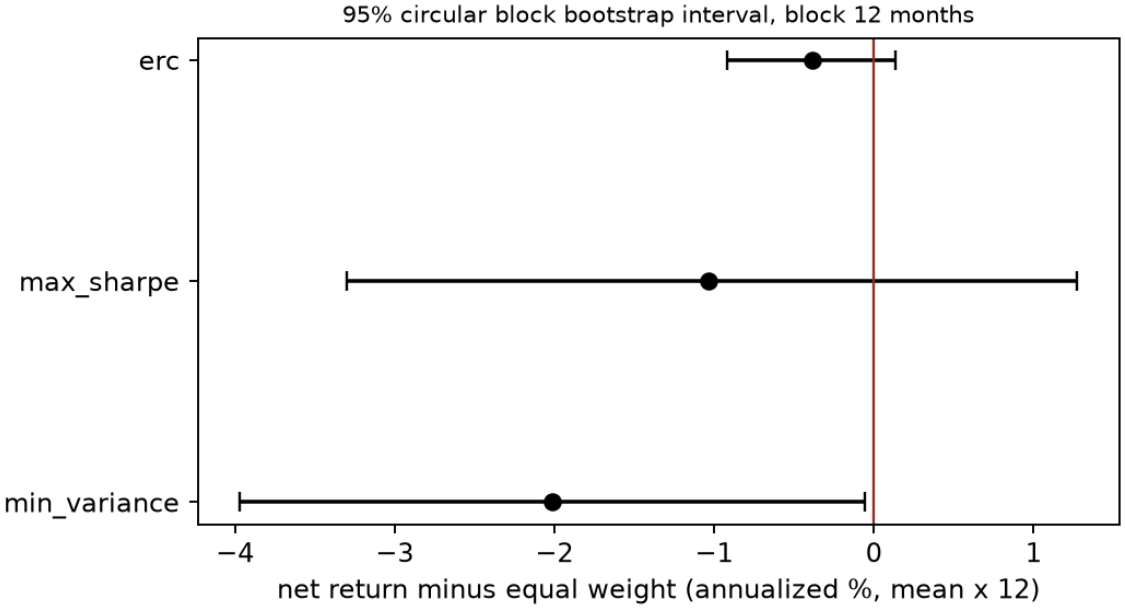


Figure 7: Project 3 bootstrap difference distribution

The horizontal axis shows the annualized average net-return difference relative to equal weight (percentage points, calculated as the monthly mean difference multiplied by 12); the vertical axis lists the three strategies. Dots show sample differences and horizontal lines show the 95% paired circular-block bootstrap intervals. Check whether each interval crosses the vertical zero-difference line. This figure does not show Sharpe differences and applies no multiple-comparison adjustment.

13.4 Cost and Window Sensitivity Analysis

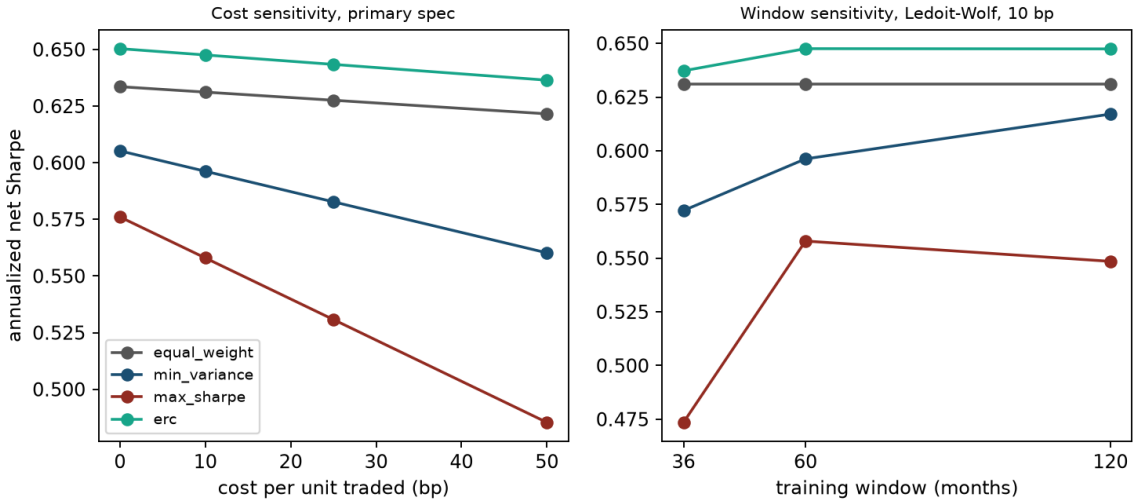


Figure 8: Project 3 cost and window sensitivity

The horizontal axis in the left panel shows cost per unit of combined buy-and-sell turnover (bp); the horizontal axis in the right panel shows training-window length (months). Both vertical axes show annualized net Sharpe. Compare each strategy's cost slope and changes across window lengths; do not choose parameters retrospectively from the highest point in the figure. These data come from prespecified sensitivity specifications on the same frozen sample and are not an independent out-of-sample test.

- **Cost sensitivity**: When trading costs rise from 0 to 50 bps:
  - The annualized Sharpe ratio for equal weight declines only slightly, from 0.634 to 0.622;
  - ERC declines from 0.650 to 0.636;
  - Maximum Sharpe's annualized net Sharpe falls from 0.576 to 0.485 (about 15.8%), indicating greater cost sensitivity; this does not establish that the wealth path collapses.
- **Window and lag sensitivity**: With a 36-month window, maximum Sharpe has 14 fallback months under both sample covariance and Ledoit–Wolf estimation (all industry training-window excess means are negative in 8 months; in 6 months no positive-excess portfolio exists under the 40% cap). These fallbacks are caused by mean estimation and are unrelated to the covariance estimator; at 10 bp, net Sharpe falls to 0.474. Ledoit–Wolf provides a modest improvement for minimum variance (W60: 0.596 versus 0.577 with sample covariance). A one-month lag barely changes the results (ERC 0.648 versus 0.649).

The answer file also reports net Sharpe of about 0.587 for maximum Sharpe at W=24 and 25 equal-weight fallbacks. This window was proposed after seeing the main results, so it is a post hoc exploration. It is outside the 12 frozen window/covariance/lag sensitivity specifications and does not replace the W=60 primary specification. Exploration can generate a question to prespecify next time, but it does not increase the number of independent validations in this round.

13.5 Warning About Hindsight Overfitting

Use sample moments from the full evaluation period (1990-01..2026-08), with the same long-only constraint and 40% cap, to calculate a hindsight maximum-Sharpe portfolio (it uses information from the evaluation period and is not tradable):

- The monthly realized Sharpe of the fixed-weight gross-return path is 0.21774, or **0.75428** annualized, using the standard deviation of same-month excess returns as the denominator. The optimizer's objective for the supplied moments is 0.75386, using the covariance of asset gross returns as its denominator; these must not be described as the same quantity;
- By comparison, the realized annualized Sharpe ratio for rolling out-of-sample maximum Sharpe is only **0.5580**. With zero costs, rolling maximum Sharpe is 0.576; with the 10 bp proxy cost it is 0.558. The full-sample fit uses information from the evaluation period, and its estimation window and fixed weights also differ from the rolling weights. The difference from the rolling net figure does not isolate a causal effect of look-ahead bias or measure a tradable advantage.

14. Chapter Summary and Decision Logic

1. **Estimation error dominates risk**: Expected-return estimation error is a primary source of failure in mean–variance optimization. Without reliable alpha forecasts, use shrinkage to control covariance ill-conditioning or fall back to minimum-variance or risk-parity portfolios.
2. **Long-only constraints and caps are natural forms of regularization**: Long-only constraints rule out unbounded short leverage, while per-asset caps prevent a single extreme position. A maximum-Sharpe problem with these constraints can be transformed exactly into a convex quadratic program using a scale variable (proved in Chapter 9).
3. **Keep neutrality concepts distinct**: Dollar, beta, and sector neutrality should be added explicitly as constraints in the optimizer. Euclidean projection optimizes only weight distance; if the objective includes risk and costs, use the corresponding objective function. Full-coverage sector constraints also imply dollar neutrality, so redundant rows should be removed.
4. **Risk budgets balance actual risk exposures**: ERC distributes risk evenly across assets, preventing high-volatility assets from dominating the portfolio. Damped Newton's method with a Newton-decrement stopping rule can solve ERC efficiently and reliably.
5. **Trading costs can determine whether a strategy survives live trading**: Portfolio managers must balance rebalancing frequency, tracking error, and turnover frictions. Without significant alpha to support it, a high-turnover strategy's net returns can be consumed by trading costs.
6. **Use benchmarks and try to falsify the strategy**: A naive equal-weighted portfolio (1/N) should be the first benchmark for an optimized strategy. Apparent optimization gains that have not been checked against trading costs and bootstrap uncertainty are often only in-sample statistical illusions.

15. Conceptual and Mathematical Exercises

21-E01 (Scale of estimation error) Using the IID and stable-variance assumptions from Section 5.1, suppose a stock has monthly volatility of 6% and a historical observation window of 36 months. Calculate the standard error of its monthly mean (%). If the point estimate of monthly excess return is 0.5%, construct a 95% confidence interval and explain why tangency-portfolio optimization is highly vulnerable to this error.

21-E02 (Ledoit–Wolf target and intensity) Briefly explain why Ledoit–Wolf shrinkage uses the scaled identity matrix mI, rather than the fixed unscaled identity matrix I, as its target. What is the asymptotic behavior of the shrinkage intensity $\hat{\kappa}$ as $T \to \infty$?

21-E03 (Drift and turnover calculation) A portfolio contains three assets with beginning-of-period weights $\mathbf{w} = (0.4, 0.4, 0.2)$. Their returns for the month are +20%, −10%, and 0%, respectively. Calculate the pre-trade drifted weights $\mathbf{w}^+$ at month-end. If the next-period target weights return to their initial values, calculate total turnover. If the one-way cost is 15 bps, calculate the cost drag on net asset value.

21-E04 (Constructing a counterexample for the neutrality conditions) Construct a 3-asset portfolio weight vector $\mathbf{w}$ and beta vector $\boldsymbol{\beta}$ such that the portfolio is dollar neutral ($\mathbf{1}^\top \mathbf{w} = 0$) but its total portfolio beta has absolute value at least 0.5.

21-E05 (Proving the Euler risk decomposition) Under a symmetric positive-semidefinite covariance matrix, suppose portfolio volatility satisfies $\sigma(\mathbf{w}) > 0$, and portfolio variance is $\sigma^2(\mathbf{w}) = \mathbf{w}^\top \Sigma \mathbf{w}$. Use the multivariable chain rule to prove rigorously that $\frac{\partial \sigma(\mathbf{w})}{\partial w_i} = \frac{(\Sigma \mathbf{w})_i}{\sigma(\mathbf{w})}$, and verify that $\sum_{i=1}^n w_i \, \mathrm{MRC}_i = \sigma(\mathbf{w})$.

21-E06 (Analytical derivation of two-asset ERC) Let two assets have volatilities σ_1, σ_2 and correlation $\rho \in (-1, 1)$. Starting from the definition $w_1(\Sigma \mathbf{w})_1 = w_2(\Sigma \mathbf{w})_2$, prove that the fully invested long-only solution is $w_1 = \frac{\sigma_2}{\sigma_1 + \sigma_2}$, and that this weight is exactly independent of the correlation ρ.

21-E07 (Recovering Black–Litterman equilibrium returns) Suppose the market contains two assets, with market weights $\mathbf{w}_{mkt} = (0.7, 0.3)^\top$, covariance matrix $\Sigma = \begin{bmatrix} 0.05 & 0.01 \\ 0.01 & 0.02 \end{bmatrix}$, and market risk-aversion coefficient $\delta = 3.0$. Calculate the reverse-optimized equilibrium expected-return vector $\mathbf{\Pi}$.

21-E08 (Deriving the continuous Kelly approximation) Write the second-order Taylor expansion of portfolio log growth. Let a single asset's excess return be $\mu - r_f = 0.008$, with variance $\sigma^2 = 0.0025$. Both inputs are monthly decimals. Using the quadratic proxy that omits the squared-mean term, calculate the full-Kelly optimal leverage and the monthly proxy growth of full and half Kelly relative to the cash baseline; do not calculate absolute growth with an unspecified cash rate.

16. Programming Exercises

21-P01 (Numerical test of the four-asset MRT example) Write a standalone test script using the four-asset inputs from Maillard et al. (2009) in Section 8.4. Call `quant_course.portfolio_construction` and the optimization tools to calculate weights and risk shares for equal weight, long-only minimum variance, and ERC. Use `assert` to check that volatility satisfies $\sigma_{mv} \leq \sigma_{erc} \leq \sigma_{1/n}$.

21-P02 (Verifying shrinkage of a rank-deficient covariance matrix) Generate a Gaussian random sample matrix of shape 10×20 ($T < p$). Call the Ledoit–Wolf shrinkage function from the Section 9 import list. Assert that the original sample covariance matrix is rank-deficient (rank ≤ 10) and that every eigenvalue of the shrunk covariance matrix is strictly greater than 0 (positive definite and invertible).

21-P03 (Testing the long-only maximum-Sharpe feasibility precheck under a weight cap) Construct a 3-asset excess-mean vector $\boldsymbol{\mu}_{excess}$ with components 0.004, -0.004, and -0.004, and covariance $0.01I_3$. With a per-asset cap of 0.40, assert that the long-only maximum-Sharpe function in the Section 9 import list identifies that no positive-excess-return portfolio exists under the cap and raises `OptimizationError`; the budget constraint itself remains feasible, for example with equal weights.

17. Challenge Problems

21-C01 (Analytical decomposition of risk contributions under a factor model) Suppose asset returns follow a general multi-factor model in which factors and specific returns are uncorrelated, but the factors need not be orthogonal: $\mathbf{r} = B\mathbf{f} + \boldsymbol{\varepsilon}$, where factor covariance is Σ_f and specific covariance is diagonal, Σ_ε (using Chapter 15 notation).

1. Derive the decomposition of portfolio variance into factor risk and specific risk;
2. Prove that each factor's absolute risk contribution can be written as $\mathrm{RC}_k = \beta_{p,k}\frac{(\Sigma_f\boldsymbol{\beta}_p)_k}{\sigma(\mathbf{w})}$, where $\boldsymbol{\beta}_p = B^\top\mathbf{w}$;
3. Construct a numerical example with 2 assets and 1 factor, and calculate the risk shares of every component in full.

21-C02 (Designing a robust portfolio-construction and backtesting protocol) For a long–short equity strategy, design a complete practical portfolio-construction plan. The plan must clearly specify:

1. Data timestamps and causal separation;
2. The covariance estimator and treatment of ill-conditioning;
3. Risk-neutrality constraints and total-leverage controls;
4. Turnover controls and logic for smoothing target weights;
5. Explicit evaluation failure conditions (the quantitative evidence that would require rejecting the strategy for launch).

17.1 Integrated Review Questions

21-R01 (Fee conservation in currency amounts) Using the 55/45 pre-trade weights, 1 million yuan of net asset value, and c=10bp from Section 8.2, calculate the exact fees and purchase/sale amounts for 50/50 and 100/0 targets. Compare them with the turnover proxy based on prior net asset value. Do both examples establish actual slippage?

21-R02 (Mathematical objects and statistical judgments) Diagnose each case: (a) a fully invested long-only weight cap is feasible, but maximum excess mean is non-positive; (b) a zero-risk portfolio has positive excess return; (c) whether the BL posterior-mean covariance can directly replace return risk; and (d) lower sample volatility for MinVar, a single-comparison return-difference interval below 0, an ERC Sharpe difference interval crossing 0, and the post hoc W24 result. For each, state which conclusions can and cannot be drawn.

18. Exercise Solutions and Explanations

21-E01

- Standard error of the monthly mean:

$$\mathrm{SE}(\hat{\mu}) = \frac{0.06}{\sqrt{36}} = \frac{0.06}{6} = 0.010 \quad (1.00\%).$$

- 95% confidence interval (using the normal critical value 1.96):

$$0.5\% \pm 1.96 \times 1.0\% = [-1.46\%, +2.46\%].$$

- The confidence interval is very wide and does not even establish whether true expected excess return is positive or negative. Unconstrained tangency optimization calculates $\Sigma^{-1}\hat{\boldsymbol{\mu}}$. Substituting either -1.46% or $+2.46\%$ could swing weights by hundreds of percentage points between extreme short and long positions, treating pure sampling noise as a certain signal.

21-E02

- Choosing the scaled identity matrix mI ensures that the shrinkage target has exactly the same trace and total-energy scale as the sample covariance ($\mathrm{tr}(mI) = \mathrm{tr}(S)$). If the unscaled identity matrix I were used, changes in the scale of asset returns (for example, converting decimals to percentages) would severely distort the overall eigenvalue scale of the shrunk matrix.
- As $T \to \infty$ with fixed p, the law of large numbers implies that the sample covariance matrix S converges almost surely to the true covariance Σ. Estimation-error variance $\beta^2 \to 0$, while $d^2 \to \alpha^2$. As long as the true Σ is not a scaled identity matrix ($\alpha^2 > 0$), the feasible shrinkage intensity $\hat{\kappa} = b^2/d^2 \to 0$, and the estimator converges asymptotically back to the sample covariance S. If Σ is exactly μI, both quantities tend to 0 and the intensity need not tend to 0; in that case the shrinkage target itself is correct.

21-E03

- Portfolio gross return:

$$r_{gross} = 0.4(0.20) + 0.4(-0.10) + 0.2(0.0) = 0.08 - 0.04 + 0.0 = +0.04 \quad (4.0\%).$$

- Drifted weights:

$$w_1^+ = \frac{0.4(1+0.20)}{1.04} = \frac{0.48}{1.04} \approx 0.461538;$$
$$w_2^+ = \frac{0.4(1-0.10)}{1.04} = \frac{0.36}{1.04} \approx 0.346154;$$
$$w_3^+ = \frac{0.2(1+0.0)}{1.04} = \frac{0.20}{1.04} \approx 0.192308.$$

- Total turnover (buys and sells combined):

$$\begin{aligned}\text{Turnover} &= |0.4 - 0.461538| + |0.4 - 0.346154| + |0.2 - 0.192308| \\ &= 0.061538 + 0.053846 + 0.007692 = 0.123076 \quad (12.31\%\ \text{NAV}).\end{aligned}$$

- Cost drag (15 bps = 0.0015):

$$\text{cost} = 0.0015 \times 0.123076 \approx 0.0001846 \quad (1.85\ \text{bps}).$$

21-E04

- Let the weights be $\mathbf{w} = (0.5, 0.0, -0.5)^\top$. Check dollar neutrality: $\sum w_i = 0.5 + 0.0 - 0.5 = 0.0$.
- Let the beta vector be $\boldsymbol{\beta} = (1.5, 1.0, 0.3)^\top$. The portfolio's total beta is:

$$\boldsymbol{\beta}^\top \mathbf{w} = 0.5(1.5) + 0.0(1.0) - 0.5(0.3) = 0.75 - 0.15 = 0.60.$$

  Thus, $|\boldsymbol{\beta}^\top \mathbf{w}| = 0.60 \geq 0.5$.

21-E05

Every division by volatility below requires $\sigma(\mathbf{w}) > 0$; this derivative does not apply at a zero-risk point.

- Apply the chain rule to $\sigma(\mathbf{w}) = (\mathbf{w}^\top \Sigma \mathbf{w})^{1/2}$:

$$\frac{\partial \sigma(\mathbf{w})}{\partial w_i} = \frac{1}{2}(\mathbf{w}^\top \Sigma \mathbf{w})^{-1/2} \cdot \frac{\partial (\mathbf{w}^\top \Sigma \mathbf{w})}{\partial w_i}.$$

  By the vector derivative rule for a quadratic form, $\frac{\partial (\mathbf{w}^\top \Sigma \mathbf{w})}{\partial \mathbf{w}} = 2\Sigma\mathbf{w}$, so the derivative with respect to its i-th component is $2(\Sigma\mathbf{w})_i$. Substituting gives:

$$\frac{\partial \sigma(\mathbf{w})}{\partial w_i} = \frac{1}{2\sigma(\mathbf{w})} \cdot 2(\Sigma\mathbf{w})_i = \frac{(\Sigma\mathbf{w})_i}{\sigma(\mathbf{w})}.$$

- Verify the Euler identity:

$$\sum_{i=1}^{n} w_i \,\mathrm{MRC}_i = \sum_{i=1}^{n} w_i \frac{(\Sigma\mathbf{w})_i}{\sigma(\mathbf{w})} = \frac{\mathbf{w}^\top (\Sigma\mathbf{w})}{\sigma(\mathbf{w})} = \frac{\sigma^2(\mathbf{w})}{\sigma(\mathbf{w})} = \sigma(\mathbf{w}).$$

  This completes the proof.

21-E06

- For two assets, the covariance matrix is $\Sigma = \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix}$. Calculate their absolute risk contributions:
$$\mathrm{RC}_1 = \frac{w_1(\Sigma\mathbf{w})_1}{\sigma(\mathbf{w})} = \frac{w_1(w_1\sigma_1^2 + w_2\rho\sigma_1\sigma_2)}{\sigma(\mathbf{w})};$$
$$\mathrm{RC}_2 = \frac{w_2(\Sigma\mathbf{w})_2}{\sigma(\mathbf{w})} = \frac{w_2(w_1\rho\sigma_1\sigma_2 + w_2\sigma_2^2)}{\sigma(\mathbf{w})}.$$
- Set $\mathrm{RC}_1 = \mathrm{RC}_2$:
$$w_1^2\sigma_1^2 + w_1w_2\rho\sigma_1\sigma_2 = w_1w_2\rho\sigma_1\sigma_2 + w_2^2\sigma_2^2.$$
The cross term $w_1w_2\rho\sigma_1\sigma_2$ cancels exactly from both sides, giving:
$$w_1^2\sigma_1^2 = w_2^2\sigma_2^2.$$
Under the long-only constraint $w_1, w_2 > 0$, take the positive square root to obtain:
$$w_1\sigma_1 = w_2\sigma_2 \iff \frac{w_1}{w_2} = \frac{\sigma_2}{\sigma_1}.$$
- Combining this with the fully invested constraint $w_1 + w_2 = 1$:
$$w_1 + w_1\frac{\sigma_1}{\sigma_2} = 1 \implies w_1\left(\frac{\sigma_1 + \sigma_2}{\sigma_2}\right) = 1 \implies w_1 = \frac{\sigma_2}{\sigma_1 + \sigma_2}.$$
The formula contains no correlation coefficient ρ. This completes the proof.

21-E07

- Calculate $\Sigma\mathbf{w}_{mkt}$:
$$\begin{bmatrix} 0.05 & 0.01 \\ 0.01 & 0.02 \end{bmatrix}\begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.05(0.7) + 0.01(0.3) \\ 0.01(0.7) + 0.02(0.3) \end{bmatrix} = \begin{bmatrix} 0.035 + 0.003 \\ 0.007 + 0.006 \end{bmatrix} = \begin{bmatrix} 0.038 \\ 0.013 \end{bmatrix}.$$
- Multiply by $\delta = 3.0$:
$$\mathbf{\Pi} = 3.0\begin{bmatrix} 0.038 \\ 0.013 \end{bmatrix} = \begin{bmatrix} 0.114 \\ 0.039 \end{bmatrix} \quad (11.4\%, 3.9\%).$$

21-E08

The following calculations use a monthly quadratic-growth proxy relative to the cash baseline, omitting the squared-mean term; this is not exact discrete expected log utility.

- First write the second-order expansion. Let portfolio simple return be $R_p = r_f + w(R - r_f)$, with mean $a = r_f + w(\mu - r_f)$ and variance $w^2\sigma^2$. From $\ln(1 + x) \approx x - x^2/2$,
$$E[\ln(1 + R_p)] \approx a - \tfrac{1}{2}E[R_p^2] = a - \tfrac{1}{2}\{w^2\sigma^2 + a^2\}.$$
This uses $E[R_p^2] = \mathrm{Var}(R_p) + a^2$. Returns must be sufficiently small, and the ruin boundary and higher-order remainder must be controlled; the second-order expansion alone does not guarantee that exact log utility exists under an arbitrary distribution.

- This exercise then uses the lower-order monthly proxy that omits the squared-mean term a^2:
$$g(w) \approx r_f + w(\mu - r_f) - \frac{1}{2}w^2\sigma^2.$$
- Full-Kelly optimal weight:
$$w^* = \frac{\mu - r_f}{\sigma^2} = \frac{0.008}{0.0025} = 3.20 \quad (3.20 \text{ times leverage}).$$
- Expected excess log growth under full Kelly:
$$\begin{aligned} g(w^*) - r_f &= 3.20(0.008) - \tfrac{1}{2}(3.20^2)(0.0025) \\ &= 0.0256 - 0.0128 = 0.0128 \quad (1.28\%). \end{aligned}$$
- Half-Kelly weight, $w_{half} = 1.60$:
$$\begin{aligned} g(w_{half}) - r_f &= 1.60(0.008) - \tfrac{1}{2}(1.60^2)(0.0025) \\ &= 0.0128 - 0.0032 = 0.0096 \quad (0.96\%). \end{aligned}$$

  Check: $0.0096/0.0128 = 0.75$, exactly 75% of full-Kelly growth.

Reference Implementation and Assertions for 21-P01 to 21-P03

The Python code below can be run independently from the repository root to complete the checking assertions:

```python
import numpy as np
from quant_course.portfolio_construction import (
    ledoit_wolf_identity, risk_budget_weights, risk_contributions,
    max_sharpe_long_only,
)
from quant_course.optimization_tools import solve_mean_variance, OptimizationError

# 21-P01: MRT 2009 four-asset numerical check and volatility ordering
vol_mrt = np.array([0.10, 0.20, 0.30, 0.40])
corr_mrt = np.array([
    [1.0, 0.8, 0.0, 0.0],
    [0.8, 1.0, 0.0, 0.0],
    [0.0, 0.0, 1.0, -0.5],
    [0.0, 0.0, -0.5, 1.0],
])
cov_mrt = np.outer(vol_mrt, vol_mrt) * corr_mrt

# 1/N
ew_res = risk_contributions(np.full(4, 0.25), cov_mrt)
assert np.isclose(ew_res.volatility, 0.1151086, atol=1e-4)
np.testing.assert_allclose(ew_res.fractions, [0.123, 0.264, 0.141, 0.472], atol=1e-3)

# Long-only Min Var
mv_weights = solve_mean_variance(np.zeros(4), cov_mrt).weights
mv_res = risk_contributions(mv_weights, cov_mrt)
assert np.isclose(mv_res.volatility, 0.086303, atol=1e-4)
np.testing.assert_allclose(mv_weights, [0.7448, 0.0, 0.1517, 0.1034], atol=1e-3)
```

```
# ERC
erc_weights = risk_budget_weights(cov_mrt).weights
erc_res = risk_contributions(erc_weights, cov_mrt)
assert np.isclose(erc_res.volatility, 0.102934, atol=1e-4)
np.testing.assert_allclose(erc_weights, [0.3836, 0.1918, 0.2426, 0.1820], atol=1e-3)
np.testing.assert_allclose(erc_res.fractions, 0.25, atol=1e-6)

# Verify the volatility-ordering theorem
assert mv_res.volatility <= erc_res.volatility <= ew_res.volatility

# 21-P02: Shrinkage of a rank-deficient covariance matrix for T < p
rng = np.random.default_rng(20261021)
x_low_rank = rng.standard_normal((10, 20))
lw_low_rank = ledoit_wolf_identity(x_low_rank)
assert np.linalg.matrix_rank(lw_low_rank.sample_covariance) <= 10
min_eig = np.linalg.eigvalsh(lw_low_rank.covariance).min()
assert min_eig > 0.0
assert lw_low_rank.intensity > 0.0

# 21-P03: Maximum-Sharpe cap infeasibility precheck test
mu_infeasible = np.array([0.004, -0.004, -0.004])
cov_diag = np.diag([0.01, 0.01, 0.01])
try:
    max_sharpe_long_only(
        mu_infeasible, cov_diag, risk_free_return=0.0, max_weight=0.4,
    )
except OptimizationError as exc:
    assert "capped long-only portfolio has positive expected excess return" in str(
    ↪ exc)
else:
    raise AssertionError("expected OptimizationError was not raised")
```

21-C01

Conditions: $\mathrm{Cov}(\mathbf{f}, \boldsymbol{\varepsilon}) = 0$, Σ_ε is diagonal, and $\sigma(\mathbf{w}) > 0$. Individual contributions can be negative when factors are correlated; the formula does not require Σ_f to be diagonal.

1. **Variance decomposition**: Under the factor model $\mathbf{r} = B\mathbf{f} + \boldsymbol{\varepsilon}$, the return covariance matrix is $\Sigma = B\Sigma_f B^\top + \Sigma_\varepsilon$. Total portfolio variance is:

$$\begin{aligned}\sigma^2(\mathbf{w}) &= \mathbf{w}^\top \Sigma \mathbf{w} = \mathbf{w}^\top B \Sigma_f B^\top \mathbf{w} + \mathbf{w}^\top \Sigma_\varepsilon \mathbf{w} \\ &= \boldsymbol{\beta}_p^\top \Sigma_f \boldsymbol{\beta}_p + \sum_{i=1}^{n} w_i^2 (\Sigma_\varepsilon)_{ii},\end{aligned}$$

   Here $\boldsymbol{\beta}_p = B^\top \mathbf{w} \in \mathbb{R}^K$ is the portfolio's net exposure to the K common factors.
2. **Factor risk-contribution decomposition**: Write the result from step 1 as a sum over components:

$$\sigma^2(\mathbf{w}) = \sum_{k=1}^{K} \beta_{p,k} (\Sigma_f \boldsymbol{\beta}_p)_k + \sum_{i=1}^{n} w_i^2 (\Sigma_\varepsilon)_{ii}.$$

Divide both sides by $\sigma(\mathbf{w})$ and define

$$\mathrm{RC}_{factor,k} = \beta_{p,k}\frac{(\Sigma_f \boldsymbol{\beta}_p)_k}{\sigma(\mathbf{w})}, \qquad \mathrm{RC}_{spec,i} = \frac{w_i^2(\Sigma_\varepsilon)_{ii}}{\sigma(\mathbf{w})},$$

Then $\sum_k \mathrm{RC}_{factor,k} + \sum_i \mathrm{RC}_{spec,i} = \sigma(\mathbf{w})$. Treating σ as a function of factor exposure $\boldsymbol{\beta}_p$ and specific weights gives $\partial\sigma/\partial\beta_{p,k} = (\Sigma_f\boldsymbol{\beta}_p)_k/\sigma$, so $\mathrm{RC}_{factor,k}$ is exactly the Euler form "exposure × marginal risk."

3. **Two-asset, one-factor example**: Let $\beta = [1.2, 0.8]^\top$, factor variance $\Sigma_f = 0.04$, specific covariance $\Sigma_\varepsilon = \mathrm{diag}(0.01, 0.02)$, and weights $\mathbf{w} = [0.6, 0.4]^\top$.
   - Portfolio factor exposure: $\beta_p = 0.6(1.2) + 0.4(0.8) = 0.72 + 0.32 = 1.04$;
   - Factor-variance component: $\sigma^2_{factor} = (1.04)^2 \times 0.04 = 0.043264$;
   - Specific-variance component: $\sigma^2_{spec} = 0.6^2(0.01) + 0.4^2(0.02) = 0.0036 + 0.0032 = 0.0068$;
   - Total portfolio variance: $\sigma^2 = 0.043264 + 0.0068 = 0.050064 \implies \sigma \approx 0.22375$ (22.375%);
   - Absolute factor risk contribution: $\mathrm{RC}_{factor} = \frac{0.043264}{0.22375} \approx 0.19336$ (share 86.42%);
   - Absolute specific risk contribution: $\mathrm{RC}_{spec} = \frac{0.0068}{0.22375} \approx 0.03039$ (share 13.58%).

21-C02

Scoring rubric (10 points total):

- **Timing and causal separation (2 points)**: At decision time $t-1$, use only historical data that are available; generate signals strictly before the opening or rebalance; never use future returns to estimate means or covariance.
- **Covariance estimation and numerical robustness (2 points)**: Use Ledoit–Wolf shrinkage or a structured factor-model estimate; never directly invert an ill-conditioned sample covariance matrix; include scale normalization and positive-definiteness safeguards in the optimizer.
- **Realistic constraint architecture (2 points)**: Clearly specify total exposure (leverage), per-stock caps, sector-neutrality and beta-neutrality constraints; do not use a purely geometric Euclidean projection that discards information.
- **Cost and turnover controls (2 points)**: Include a turnover penalty (such as a quadratic or absolute-value cost term) or buffer rules in target weights, and calculate actual net returns after costs.
- **Rigorous falsification and launch thresholds (2 points)**: Prespecify non-negotiable falsification criteria. For this challenge, one may propose an illustrative teaching threshold (not a universal industry standard): under this example's governance rule, reject the strategy for launch if its net Sharpe ratio does not significantly outperform the 1/N benchmark under paired bootstrap, if it is too sensitive to trading costs (a 10 bps cost increase reduces Sharpe by more than 20%), or if optimizer failures trigger fallback in more than 1% of backtest periods. Readers may propose different thresholds but must explain the risk budget, economic objective, and evidence requirements set in advance. 1/N is not the only suitable benchmark for every trading task.

21-R01

50/50 target: q=0.0001, fees are 100 yuan, and the new holdings are 499,950 yuan each. Sell 50,050 yuan and buy 49,950 yuan; the 100-yuan difference pays the fee. Total traded amount is 100,000 yuan, matching the proxy. For a 100/0 target: q=0.0009/1.001=0.0008991009, fees are 899.1009 yuan, and the new amount in asset 1 is 999,100.8991 yuan. Buy 449,100.8991 yuan of asset 1 from the initial 550,000 yuan position and sell 450,000 yuan of asset 2; the 899.1009-yuan difference pays the fee. Total traded amount is 899,100.8991 yuan, which yields the fee when multiplied by 0.001. The proxy charges 900 yuan on 900,000 yuan of turnover. The capital-conservation check holds, but the model has no actual slippage, market impact, orders, or capacity data, so it does not establish live trading costs.

21-R02

(a) Budget feasibility does not imply that the positive-Sharpe transformation is available. Preserve the reason for an equal-weight fallback; do not call every position infeasible. (b) The denominator of an ordinary finite Sharpe ratio is 0. A positive-semidefinite convex QP may return an objective of 0, but this cannot be read as a finite optimal ratio. (c) V_BL represents uncertainty in the estimated mean; only under a hierarchical model with independent new noise is predictive risk Sigma+V_BL. The original BL allocation uses Sigma, and changing the risk objective requires solving again. (d) Lower volatility or drawdown is only a sample description. An unadjusted mean-difference interval supports only that single primary-specification comparison; it does not establish family-wise significance or significant volatility differences. An ERC interval crossing 0 is neither proof of equivalence nor a probability of superiority. W24 is a post hoc exploration; it does not add a prespecified sensitivity validation or replace the primary specification.

19. Further Reading

- **Must Know**:
  - Ledoit & Wolf (2004), read Sections 2.1 and 3 of the authors' working-paper version to learn the derivation of the feasible shrinkage-intensity estimator and the Frobenius-norm definition;
  - Maillard, Roncalli & Teiletche (2009/2010), read Sections 1–3.4 to learn the Euler decomposition of volatility, the two-asset independence result, and the convex log-barrier formulation.
- **Should Know**:
  - He & Litterman (1999), read Appendix B to learn how reverse optimization determines the prior, how the gain-form posterior mean is derived, and why view tilts are orthogonal;
  - DeMiguel, Garlappi & Uppal (2009), read Section 3 of the NBER conference-paper version to understand rolling out-of-sample backtesting, turnover calculated from pre-trade drifted weights, and the practical challenges of the naive 1/N benchmark.

- **Advanced**:
  - Boyd & Vandenberghe, *Convex Optimization*, Section 9.5, for convergence analysis of damped Newton's method and the Newton-decrement stopping criterion;
  - Kelly (1956), read printed pages 919–920 of the *Bell System Technical Journal* scan to understand the binary geometric log-growth maximum theorem and ruin boundary.
- **Optional**:
  - Lesniewski's Baruch MFE Lecture 5 (2019 file), "Maximizing the Sharpe ratio," for comparison with the variable substitution in Chapter 9;
  - Browse the companion core code `src/quant_course/portfolio_construction.py` and applied module `projects/project03/src/project03_backtest.py`.

20. References

1. **CH21_LEDOITWOLF2004**: Olivier Ledoit and Michael Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, 88(2), 365–411, 2004. DOI: 10.1016/S0047-259X(03)00096-4. Working-paper version: `http://www.ledoit.net/ole1a.pdf`. The actual reading scope is documented in the source reading record.
2. **CH21_MRT2009**: Sébastien Maillard, Thierry Roncalli, and Jérôme Teiletche, "On the properties of equally-weighted risk contributions portfolios," *Journal of Portfolio Management*, 36(4), 60–70, 2010. DOI: `10.3905/jpm.2010.36.4.060`. Working-paper version: `http://www.thierry-roncalli.com/download/erc.pdf`. The actual reading scope is documented in the source reading record.
3. **CH21_HELITTERMAN1999**: Guangliang He and Robert Litterman, "The Intuition Behind Black-Litterman Model Portfolios," Goldman Sachs Asset Management Research, 1999. Duke-hosted copy: `https://people.duke.edu/~charvey/Teaching/BA453_2006/GS_The_intuition_behind.pdf`. The actual reading scope is documented in the source reading record.
4. **CH21_KELLY1956**: J. L. Kelly, Jr., "A New Interpretation of Information Rate," *Bell System Technical Journal*, 35(4), 917–926, 1956. DOI: `10.1002/j.1538-7305.1956.tb03809.x`. Scan: `https://archive.org/details/bstj35-4-917`. The actual reading scope is documented in the source reading record.
5. **CH21_DGU2009**: Victor DeMiguel, Lorenzo Garlappi, and Raman Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?" *Review of Financial Studies*, 22(5), 1915–1953, 2009. DOI: `10.1093/rfs/hhm075`. NBER conference-paper version: `https://users.nber.org/~confer/2006/si2006/ap/uppal.pdf`. The actual reading scope is documented in the source reading record.
6. **SHARPE1994**: William F. Sharpe, "The Sharpe Ratio," *Journal of Portfolio Management*, Fall 1994 (volume, issue, and pages were not verified in the webpage read). Author's webpage: `https://web.stanford.edu/~wfsharpe/art/sr/sr.htm`.
7. **LESNIEWSKI2019**: Andrew Lesniewski, "Optimization Techniques in Finance — 5. Convex optimization. II," Baruch College MFE lecture notes (file name

Opt_Lecture5_2019.pdf). https://mfe.baruch.cuny.edu/wp-content/uploads/2014/12/Opt_Lecture5_2019.pdf.

8. **BOYD**: Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. https://web.stanford.edu/~boyd/cvxbook/.
9. **CVXPY**: Steven Diamond and Stephen Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, 17(83), 1–5, 2016. https://www.cvxpy.org/.
10. **FRENCH**: Kenneth R. French Data Library, 10 Industry Portfolios and Fama/French Factors. The data snapshot and hashes are recorded in the local manifest; the actual reading scope is documented in the source reading record.

Project 3 —Rolling Portfolios and Out-of-Sample Evaluation

Project 3 — Portfolio Optimization

This project contains a default offline constructed demonstration and a complete historical research reader. Start with the Standalone Run Guide, then read the Data Dictionary, six exercises, complete solutions, and historical research report. The default demonstration does not reproduce the historical French report.

Research objective and data

Prerequisite chapters: 9, 18, 20, 21.

Assets in the same currency, or verifiable `industry/ETF` total-return data; verify vendor availability, historical coverage, and survivorship limitations separately.

Workflow

1. Implement Equal Weight, Minimum Variance, Maximum Sharpe, and Risk `Parity/ERC.`
2. At each rebalancing point, estimate returns and covariance using only the preceding window.
3. Add weight caps, cash, transaction costs, and feasibility checks.
4. Compare out-of-sample net performance, turnover, concentration, and parameter sensitivity.

Timing and evaluation rules

For all inputs, specify `event_time`, `available_at`, `signal_time`, `trade_time`, the return interval, and the label interval when applicable. Execute a signal formed at the close at the next tradable point. Select the model and parameters on the training data; use the test data only for evaluation. Specify cost, financing, and borrowing conventions in the configuration.

Deliverables

- `data_manifest.json`: download source, licensing boundary, sample, units, SHA-256, and data version.
- `data_dictionary.md`: fields, units, primary keys, timing, and missing-value rules.
- `notebooks/`: execute from a clean kernel; reuse `src/` for core logic.
- `src/`, `tests/`, `configs/`: runnable scripts, core tests, and frozen configuration.
- `figures/`: axis descriptions, data version, and code version.
- `research_report/`: hypotheses, methods, `train/test`, metrics, risk, costs, robustness, limitations, and conclusions.
- `solutions.md`: complete reference solutions to the six exercises, with question numbers and historical result checks.

Implementation checks

Do not formulate Maximum Sharpe as a convex QP without a proof; verify ERC risk contributions; check that covariance is PSD and that solver status and budget constraints are valid.

Report limitations caused by unavailable data and retain all zero and negative results. Validate core logic with pytest, scripts, and a cold start of the Notebook; do not substitute teaching simulations for measured results.

Historical research deliverable (2026-10-06)

The items above describe the full research-task contract and its checks; this section describes the historical research and must not be mistaken for output from the default offline demonstration.

- Data: value-weighted monthly returns for the 10 French industries and FF3 RF, frozen snapshot dated 2026-10-05 (current revised history, not point-in-time (PIT); industry portfolios are not directly tradable). `data_manifest.json`, data_dictionary.md.
- Frozen configuration: `configs/project03.json`. Main specification: W=60, Ledoit–Wolf, 10 bp per unit of turnover, a 40% cap for Minimum Variance and Maximum Sharpe (ERC has no cap), lag 0; evaluation 1990-01..2026-08 (440 months); sensitivity grid W$\in\{36,60,120\}$ $\times$ {sample, LW} $\times$ lag$\{0,1\}$ $\times$ cost$\{0,10,25,50\}$ bp.
- Code: `src/project03_backtest.py` (ledger, fallback, metrics, block bootstrap) reuses `src/quant_course/portfolio_construction.py`, `optimization_tools.py`, `performance.py`, and `statistics_tools.py`; entry point `run_project03.py`; tests `tests/test_project03_backtest.py`.
- Timing: for holding month t, estimate and trade weights at the end of month t−1 using [t−W, t−1] (or [t−W−1, t−2] with lag 1); calculate turnover from weights after drift; net return = (1−cost)(1+gross)−1.
- Results: research report. After costs, none of the three estimated-moment portfolios credibly outperformed equal weight; the unadjusted single-comparison interval for the annualized mean net-return difference of Minimum Variance versus equal weight lies below zero (about −2.0% annualized; the 95% block-bootstrap interval excludes 0), with lower volatility and drawdown. Maximum Sharpe had the highest turnover and was most sensitive to costs and window length. The interval for the ERC versus equal-weight Sharpe difference crosses 0. All negative and zero results are retained.
- Figures: `figures/projects/project03_net_wealth.png`, `project03_weights.png`, `project03_cost_window_sensitivity.png`, `project03_bootstrap_difference.png`.
- Notebook: `notebooks/project03_portfolio_oos.ipynb`. Exercises and solutions: exercises.md, solutions.md.

The following is the historical course-work-root route. It requires suitable, authorized data and an appropriate environment; it is not the reader package's default command:

```
.venv/bin/python projects/project03/run_project03.py
.venv/bin/python -m pytest tests/test_project03_backtest.py -q
```

Accounting conventions: the ledger includes the information cutoff, trade interval, and drifted pre-trade weights; costs use a normalized weight-turnover proxy. See report §3 for the exact self-financing amount equation and its error bound. Annualized mean fees and the realized gross-minus-net return difference are reported separately; statistical conclusions say only that the advantage screen was not passed, and do not misstate a nonsignificant result as proof of ineffectiveness.

Project 3 Standalone Run Guide

This project connects Chapter 21's stable estimation, long-only constraints, risk budgeting, and drift-adjusted costs into a rolling evaluation process. First complete the default offline constructed demonstration, then use the complete historical report to practice statistical interpretation; their inputs, windows, and evidentiary status differ.

Default entry point and inputs

From the root of the reader package, create an isolated environment using the package instructions, then run:

```
.venv/bin/python run_notebook.py projects/project03/notebooks/project03_portfolio_oos
    ↪ .ipynb
```

By default, the Notebook runs only the constructed branch and does not run the French branch. Seed 103 generates 48 months of independent normally distributed monthly simple returns for four assets, `A/B/C/D`, starting in 2000-01, with mean 0.005 and standard deviation 0.03; RF is constant at 0.001. The first 24 months provide the initial training window, followed by 24 holding months from 2002-01 through 2003-12. `Spec(window=24,max_weight=0.4)` has Ledoit–Wolf and lag 0 as its other defaults. Equal weights are each 0.25; Minimum Variance and Maximum Sharpe have a 0.4 cap, while ERC has no cap.

Each month uses only the preceding 24 months for estimation and then holds for one month. The first month starts from cash, so the weight-turnover proxy is 1; thereafter, buy-and-sell turnover is calculated from drifted weights, without dividing by 2. The cost rate is 10 bp, and net return is calculated as `(1-cost)*(1+gross)-1`; this is a cost proxy, not an exact self-financing transaction-amount ledger. Check that the final training month precedes the holding month and that each of the four strategies has 24 rows, for 96 rows in total when complete.

Expected outputs

The project `RUN_DIR` is `runs/project03_portfolio_oos_<uuid>/`, which writes only six files:

- `constructed_primary_metrics.csv`;
- `equal_weight_ledger_CONSTRUCTED.csv`;
- `min_variance_ledger_CONSTRUCTED.csv`;
- `max_sharpe_ledger_CONSTRUCTED.csv`;
- `erc_ledger_CONSTRUCTED.csv`;
- `project03_wealth_CONSTRUCTED.png`.

The runner separately creates `runs/execution_project03_portfolio_oos_<uuid>/` for an execution copy and status; do not confuse it with the project run directory. The constructed wealth plot includes an initial value of 1, for 25 points total. Its original "Holding month" x-axis uses position indices 0 through 24; 0 is before the first holding month and is not a calendar date. By default, the workflow does not download or read French data, run the historical main specification, parameter grid, bootstrap, or hindsight analysis, or generate their `summary.json` and related files. Any saved old outputs are records carried with the original files, not evidence from this run.

Reading the complete historical material

The README describes the research objective and full research contract; the dictionary lists historical fields and explains how the default data differ; the six exercises and complete solutions retain all derivations, code, and historical results; the report presents all four strategies and the full `440-month/1760-row` historical study, with main specification W60, 10 bp, LW, lag 0, and all negative and zero results.

P3-02 requires the historical `primary_ledger.csv`; historical calculations for P3-05 require French inputs and the course-work-root code. Neither is included in the default constructed outputs. If these inputs are unavailable, read the complete historical reference solution; there is no requirement to run the market-data branch. The default constructed W24 result does not reproduce the historical post hoc W24 result (net Sharpe about 0.587, with 25 fallbacks).

The historical report's paired circular-block bootstrap with 5000 replications and 12-month blocks, and its unadjusted intervals across three strategies × two metrics, support conclusions only under the stated resampling scheme. An interval crossing zero does not establish equivalence, and failing the screen is not a formal rejection by a joint test. French is a current revised snapshot, not PIT; the published industry portfolios are not directly tradable. Data rights and research gates remain governed by the original requirements; public download availability does not grant new authorization to use or redistribute data.

Notebooks in both languages

The Chinese and English project Notebooks are projections of the same accepted reader source; only the Markdown language and default-run instructions differ. Code cells, saved outputs, and metadata remain unchanged. This round does not rerun the Notebook. Final reader-package path assembly and run verification remain for integration; this guide does not describe language-layer checks as new numerical or market validation.

Project 3 Data Dictionary

This dictionary first lists fields from the historical French study; differences in the default constructed data are described in the final section. All returns are simple returns in decimals. A month label YYYY-MM denotes the return measurement interval [previous month-end close, current month-end close], not a publication date.

Frozen inputs

Field	Source file	Unit	Primary key	Timing meaning	Missing-value rule
month	Two processed CSVs	YYYY-MM	Unique, consecutive, ascending	Return-interval label	Missing, duplicate, or out-of-order months raise an error; rows are not deleted
NoDur … Other (10 columns)	`data/processed/french_10_industries_monthly.csv`	Monthly total return in decimals (including dividends), value-weighted	month	Interval [t−1 month-end, t month-end]	Any non-finite value raises an error; source sentinels −99.99/−999 were converted to blanks during processing
RF	`data/processed/french_factors_3_monthly.csv`	Monthly one-month Treasury bill return in decimals	month	Same as above	Same as above

Before use, the reader `src/quant_course/french_frozen.py` verifies the SHA-256 of the 6 raw, extracted, and processed files; it does not connect to the internet or modify them.

Timing fields (ledger)

Field	Meaning
`holding_month`	Month t held and used to measure returns
`train_start`, `train_end`	First and last month of the training window; lag 0 uses $[t-W, t-1]$, lag 1 uses $[t-W-1, t-2]$
`information_cutoff`	Label of the last month in the training window; with lag 1 it is one month before the decision month and is not a historical publication time
`trade_at` / `return_interval_start` / `return_interval_end`	Idealized month-end trade and return-interval boundaries
`decision_at`	Decision and trade time: month $t-1$ end-of-month close (idealized)
`event_time` / `available_at`	This dataset has no row-level availability times; historical French values are revised and monthly publication is delayed. Lag 0 assumes that the return for month $t-1$ is available at the end of month $t-1$; this is an idealized assumption, not a PIT fact. Lag 1 is a sensitivity check
`signal_time` / `trade_time`	Both equal `decision_at`; there is no intraday execution model
return interval	Calendar month of `holding_month`

Output: `primary_ledger.csv` (main specification W60, Ledoit–Wolf, 10 bp, 40% cap, lag 0)

Field	Unit	Definition
strategy	—	`equal_weight` / `min_variance` / `max_sharpe` / erc
status	—	`ok` or `fallback:<exception class>:<reason>` (falls back to equal weight)
turnover	Proportion of NAV	$\sum_i \lvert w_{i,t}^{target} - w_{i,t}^{pre} \rvert$; pre-trade weights are the prior month's target weights after drift from that month's returns. The first month starts from cash, so turnover=1. Any additional adjustment amount caused by fees is not included; this is not exact transaction-amount turnover
`pre_weight_*` / `target_weight_*` / `pre_trade_cash_weight`	Proportion of NAV	Per-industry drifted pre-trade asset weights and post-cost target weights; cash is 1 before the first investment and 0 thereafter
`gross_return`	Monthly decimal	$\mathbf{w}_t^{target\top}\mathbf{R}_t$
cost	Proportion of NAV	$c\times$turnover, $c = 10$ bp (cost sensitivities additionally use 0/25/50 bp)
`net_return`	Monthly decimal	$(1-\text{cost})(1+\text{gross})-1$

Field	Unit	Definition
`risk_free`, `net_excess`	Monthly decimal	Same-month RF; `net_return` − RF
`max_weight`	Proportion	Largest target weight
`effective_n`	Number of assets	$1/\sum_i w_i^2$
`max_risk_fraction_ex_ante`	Proportion	Largest single-asset volatility contribution (Euler decomposition), calculated from the training-window covariance estimate

Other outputs

- `primary_target_weights_<strategy>.csv`: target weights for each holding month (rows = holding months, columns = industries).
- `sensitivity.csv`: all metrics for 12 specifications (windows 36/60/120 × covariance `sample/ledoit_wolf` × lag 0/1) × 4 strategies × 4 cost bands; nothing is filtered out.
- `subperiods.csv`: main-specification subperiods 1990-01..2007-12 and 2008-01..2026-08.
- `fallback_log.csv`: fallback records for all specifications (month, strategy, reason).
- `summary.json`: main-specification metrics, block-bootstrap intervals versus equal weight, hindsight portfolio, fallback counts, data source, and hashes.

Metric definitions

Annualized mean excess return = 12 × monthly mean (net − RF); annualized volatility = $\sqrt{12}$ × monthly net-return standard deviation (ddof=1); Sharpe = $\sqrt{12}$ × monthly mean excess return / monthly excess-return standard deviation (`performance.sharpe_ratio`); CAGR uses 440/12 years; maximum drawdown is the minimum drawdown on the wealth path (negative); average monthly turnover excludes initial investment. $\sqrt{12}$ annualization is the report's convention and does not adjust for serial correlation.

`ann_mean_fee` = 12×mean fee as a proportion of old NAV; `ann_cost_drag` = 12×mean(`gross_return`−`net_return`) = 12×mean(cost×(1+`gross_return`)); these must not be conflated. ERC has no 40% cap, and equal weight is fixed at 10%; the `max_weight` parameter constrains only Minimum Variance and Maximum Sharpe.

Default offline constructed data and outputs

See the Standalone Run Guide for the default entry point. Seed 103 generates independent normally distributed monthly simple returns for assets `A/B/C/D`, starting in 2000-01, over 48 months, with mean 0.005 and standard deviation 0.03; RF is constant at 0.001. These are constructed teaching data, not industry data or market estimates. There are 24 training months and 24 holding months (2002-01 through 2003-12), with 96 ledger rows across the four strategies when complete.

`constructed_primary_metrics.csv` summarizes the four strategies and is saved with the DataFrame default `index=True`, so the first column contains strategy row names. The

default CAGR uses 24/12=2 years; 440/12 years above applies only to the historical report. The four `<strategy>_ledger_CONSTRUCTED.csv` files use the ledger fields above, with asset suffixes `A/B/C/D`, window 24, LW, lag 0, and cost 10 bp. Equal weight assigns 0.25 to each asset; only Minimum Variance and Maximum Sharpe have a 0.4 cap, while ERC is uncapped. The initial cash investment has turnover 1; thereafter, weights drift before turnover is calculated, and fees remain a weight-turnover proxy. The default run does not produce the historical `primary_ledger.csv`, `sensitivity.csv`, or `summary.json`.

`project03_wealth_CONSTRUCTED.png` uses returns from the 24 constructed holding months. Its wealth series includes an initial value of 1, for 25 points total. The original "Holding month" labels correspond to position indices 0 through 24, not calendar dates; 0 is before the first holding month. Do not treat the constructed W24 output as a reproduction of the historical report's post hoc W24 result.

Project 3 Exercises

See solutions.md for complete solutions and the guide for the default run instructions. Exercises P3-01, P3-03, P3-04, and P3-06 can be answered using the inputs and historical tables provided in this book. P3-02 requires the historical `primary_ledger.csv`, and the historical calculation in P3-05 requires French inputs; neither is generated by the default constructed branch. If these materials are unavailable, read the complete historical reference solutions; there is no need to start the market-data branch.

P3-01 (Manual ledger calculation) At the start of month 1, a two-asset portfolio has target weights (0.6, 0.4). The assets return +5% and −10% in month 1. At the start of month 2, the target weights are again (0.6, 0.4), with a cost of 25 bp per unit of turnover. The assets return +2% and +1% in month 2. Calculate the drifted weights at the end of month 1, turnover at the start of month 2, cost, and gross and net returns in month 2.

P3-02 (Ledger identities) Read `outputs/primary_ledger.csv`. For every row, verify cost $= 0.001 \times$ turnover and net $= (1 - \text{cost})(1 + \text{gross}) - 1$. Also verify that, for each strategy, `train_end` in each of two consecutive months precedes `holding_month`.

P3-03 (Why lag does not affect equal weight) In `sensitivity.csv`, the equal-weight metrics are identical under lag 0 and lag 1, while Minimum Variance differs slightly. Explain why, and what this implies for conclusions about the importance of the timing assumption.

P3-04 (Interpreting intervals) ERC's annualized net Sharpe is 0.016 higher than equal weight, with a 95% block-bootstrap interval of [−0.014, 0.050], while the mean net-return difference is −0.38% per year. Can you say that ERC outperforms equal weight? Write wording for a conclusion you would accept.

P3-05 (Unregistered exploration) Someone proposes changing the window to 24 months "to see if it gets better." Use the functions in `src/project03_backtest.py` to run Maximum Sharpe with W=24 and all other main-specification parameters unchanged. Report annualized net Sharpe and explain how this result should be labeled in the report.

P3-06 (Research design) If this project were to become a tradable study, list the data, timing, costs, and evaluation elements that must be replaced or added, and give one piece of evidence that would overturn the claim that "ERC reduces risk concentration without sacrificing returns."

Project 3 Exercise Solutions

The six complete solutions and historical measured-output comments dated 2026-10-06 are retained below; they have not been rerun in this round. The code belongs to the course-work-root route and requires the course dependencies, historical outputs, and French inputs; the default reader's constructed branch does not provide these historical artifacts. P3-01 can be solved by hand. When the materials are unavailable, read the historical reference solutions for P3-02 and P3-05; do not conflate results from the constructed run with a historical reproduction. See the Standalone Run Guide.

```
import sys
from pathlib import Path
import numpy as np, pandas as pd
ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "projects/project03/src"))
from quant_course.portfolio_construction import drift_weights, turnover
from project03_backtest import Spec, apply_costs, metrics, run_strategy
from quant_course.french_frozen import load_industries_and_rf

# P3-01
pre = drift_weights([0.6, 0.4], [0.05, -0.10])
to = turnover(pre, [0.6, 0.4]); cost = 0.0025 * to
gross = 0.6 * 0.02 + 0.4 * 0.01; net = (1 - cost) * (1 + gross) - 1
print(np.round(pre, 6), round(to, 6), round(cost, 9), gross, round(net, 9))

# [0.636364 0.363636] 0.072727 0.000181818 0.016 0.015815273

# P3-02
led = pd.read_csv(ROOT / "projects/project03/outputs/primary_ledger.csv")
assert np.allclose(led["cost"], 0.001 * led["turnover"])
assert np.allclose(led["net_return"], (1 - led["cost"]) * (1 + led["gross_return"]) -
    ↪ 1)
assert (pd.PeriodIndex(led["train_end"], freq="M") < pd.PeriodIndex(led["
    ↪ holding_month"], freq="M")).all()
print("ledger rows", len(led))  # 1760 = 4 strategies x 440 months

# P3-05 (exploratory, not pre-registered)
r, rf, _ = load_industries_and_rf()
ledger, _ = run_strategy("max_sharpe", r, rf, Spec(window=24, covariance="ledoit_wolf
    ↪ ", max_weight=0.4, lag=0),
                         "1990-01", "2026-08")
m = metrics(apply_costs(ledger, 10))
print(round(m["sharpe_net_ann"], 3), m["fallbacks"], round(m["avg_monthly_turnover"],
    ↪ 3))  # 0.587 25 0.287
```

P3-01

Month 1 gross return is $0.6(0.05) + 0.4(-0.10) = -0.01$. The drifted weights are $w_i(1 + r_i)/(1 + R_p)$: $0.6 \times 1.05/0.99 = 0.636364$, $0.4 \times 0.90/0.99 = 0.363636$. At the start of month 2, the weights are reset to (0.6, 0.4), so turnover is $|0.6 - 0.636364| + |0.4 - 0.363636| = 0.072727$ (total buys and sells, not divided by 2). This problem uses a cost proxy based on normalized weight changes and does not solve the post-cost self-financing transaction-amount equation. Cost is $0.0025 \times 0.072727 = 0.000182$. Month 2 gross return is $0.6(0.02) + 0.4(0.01) = 0.016$, and net return is $(1 - 0.000182)(1.016) - 1 = 0.015815$. If you forget drift and assume the weights are still (0.6, 0.4), you will calculate turnover as 0 and omit the cost.

P3-02

All three assertions pass, and the ledger contains 1,760 rows. The first confirms a cost rate of 10 bp; the second confirms that cost is deducted from NAV at the trade and then applied to the holding month's growth; the third confirms that no row is trained on data from its holding month or later.

P3-03

Equal weight uses no estimate from the training window, and its target weights are always 1/10, so lag changes only the `train_start`/`train_end` labels, not weights, turnover, or returns. Minimum Variance uses an estimated covariance matrix; lag 1 shifts its monthly window one month earlier, slightly changing the weights (main-specification net Sharpe 0.596 versus 0.597). This shows that the project's conclusions are insensitive to the assumption that data are available at month-end; it does not prove that the data were available then. A lag test measures sensitivity to the assumption and cannot turn revised data into historical vintages.

P3-04

You cannot say that ERC outperforms equal weight. The Sharpe-difference point estimate is positive, but its interval crosses zero; the mean net-return difference is negative, and its interval also crosses zero. The analysis compares three strategies and two metrics, without correcting for multiple comparisons. Acceptable wording: "In the idealized monthly backtest of French 10 industries over 1990–2026, this sample did not detect a net Sharpe advantage for ERC over equal weight (this is not proof of return equivalence) (difference 0.016, 95% block-bootstrap interval −0.014 to 0.050). ERC allocated the ex ante risk share evenly across industries, while equal weight had an average maximum ex ante risk share of 15.9%." This is a difference in risk structure, not an advantage in returns.

P3-05

For W=24, Maximum Sharpe has annualized net Sharpe of 0.587, higher than the main specification's 0.558 at W=60, but: (1) in 25 of 440 months, it fell back to equal weight because no positive-excess-return portfolio was feasible under the 40% cap or all industry excess means were negative; some of the "improvement" comes from holding equal weight; (2) average monthly turnover was 28.7%, well above W=60's 21.5%, and Sharpe fell faster as the cost tier increased; (3) this window was proposed only after the main results had been seen. The report should label it "post hoc exploration, not pre-registered," separate it from the 12 pre-specified specifications, and not use it to replace the main specification. A formal test would require evaluating it on a new data segment or a pre-specified set of windows, with a correction for multiple attempts.

P3-06

Replace or add:

- Data: historical total returns and prices for tradable instruments (for example, industry ETFs or replicable constituent portfolios), with timestamped historical versions; record delistings and constituent changes.
- Timing: at each month-end, the information set contains only data published by then; use an available closing or opening price and record execution delay.
- Costs: bid-ask spread, commissions, market impact (as a function of trade size and liquidity), plus ETF management fees, borrowing fees if shorting, and cash drag.
- Evaluation: pre-specified out-of-sample period, turnover and capacity, net returns, drawdown, concentration; block bootstrap or other inference robust to dependence; correction for multiple comparisons.

Evidence that could overturn the claim that "ERC reduces risk concentration without sacrificing returns" includes: in a pre-registered tradable sample, ERC's ex post risk shares (calculated from realized returns) remain concentrated in a few industries; or the block-bootstrap interval for its mean net return relative to equal weight lies entirely below zero (for example, an upper bound below -0.5% per year); or its after-cost Sharpe is significantly lower than equal weight.

Project 3 Research Report: Rolling Out-of-Sample Comparison of Four Long-Only Portfolios

Historical research date: 2026-10-06. This report fully preserves the historical inputs, values, and negative results; the default offline Notebook runs only an independent constructed example, as described in the guide. This language revision did not rerun the report.

1. Research question and falsifiable hypothesis

Chapter 20 explains that mean-variance optimization is exact when the moments are known; Chapter 21 explains that when moments are estimated, the optimizer amplifies estimation error. Using the same data and timing rules, this project compares four long-only, fully invested portfolios rebalanced monthly:

Code	Portfolio	What must be estimated
`equal_weight`	Equal weight `1/N`	Nothing
`min_variance`	Long-only minimum variance, 40% cap per asset	Covariance
`max_sharpe`	Long-only maximum Sharpe, 40% cap per asset	Covariance and excess means
erc	Equal Risk Contribution	Covariance

Pre-specified hypothesis (H1): among the three portfolios based on estimated moments, at least one outperforms equal weight in out-of-sample Sharpe or mean net return after costs, and the direction of that advantage remains unchanged across windows, covariance estimators, and cost tiers. If the difference interval crosses zero or is negative under the main specification, or if an advantage appears only under some sensitivity settings, the project has not passed its pre-specified advantage screen. This screen is not a calibrated joint hypothesis test.

Author's handoff statement: the main specification was entered in `configs/project03.json` before viewing this project's out-of-sample results: a 60-month window, Ledoit–Wolf shrinkage covariance, 10 bp per unit of turnover, a 40% cap, lag 0, and evaluation from 1990-01 through 2026-08 (440 months). The author later fixed two numerical implementation issues (the ERC stopping criterion and a feasibility pre-check for capped portfolios) and added an interval for the Sharpe difference; both are recorded in `qa/a4/claude_a4/revision_log.md`. These changes did not alter any strategy parameters. This is the author's local statement and revision record; it has no timestamp from an independent registration platform. Because the study uses existing historical data, it is not a prospectively registered trial.

2. Data

- Assets: value-weighted monthly total returns (including dividends), in decimals, for the 10 Industry Portfolios from Kenneth R. French Data Library.
- Risk-free return: the RF column from the FF3 file, aligned by month.
- Snapshot: downloaded and frozen on 2026-10-05; the reader verified SHA-256 hashes for 6 raw, extracted, and processed files (see `data_manifest.json`).
- Available sample: 1926-07 through 2026-08; the earliest month used in this project is 1979-12 (first training window for W=120, lag 1).

Limitation: these are current revised historical values, with no historical vintages or receipt timestamps; they are not point-in-time (PIT) data. The industry portfolios are published research portfolios, not tradable products with `bid/ask` prices, fills, and capacity data; the source's

methods handle classification, constituent changes, and delistings, which this project did not audit. See data_dictionary.md for field definitions.

3. Timing contract and tradability assumptions

For holding month t:

1. Training window: with lag 0, $[t-60, t-1]$; with lag 1, $[t-61, t-2]$. Only these months are used to estimate covariance and means.
2. Decision and trade: at the end-of-month close of $t-1$, with execution at the close implied by the returns and no intraday impact.
3. Holding: no trades during month t; weights drift by month-end.
4. Cost: deducted from NAV at trade as $c \times \sum|w^{target} - w^{pre}|$.

Lag 0 assumes the return for month $t-1$ is available at the end of month $t-1$. French data are not actually published then, so the main specification uses an idealized month boundary. The lag-1 sensitivity shows that this assumption has little effect on the results (Section 8), but adding a lag does not turn the current revised snapshot into tradable historical data. Cost tiers of 0/10/25/50 bp are hypothetical scenarios, not estimates of actual slippage.

The project charges a **normalized weight-turnover proxy**: it estimates the fee from the difference between drifted weights measured against old NAV and target weights, then invests the post-fee funds at the target weights. This is not an exact self-financing model of transaction amounts. For an exact trading-amount convention, let q be fees as a fraction of old NAV. The vector of actual traded amounts as a fraction of old NAV is $(1-q)\mathbf{w}^{target} - \mathbf{w}^{pre}$, and it must satisfy

$$q = c \sum_i \left| (1-q) w_i^{target} - w_i^{pre} \right|.$$

Here c is the cost rate per unit of traded amount, $\mathbf{w}^{pre}$ is the pre-trade asset weight normalized by old NAV (all zero in the first month), and $\mathbf{w}^{target}$ is the post-fee target weight normalized by new NAV. The current code uses $q \approx c \sum_i |w_i^{target} - w_i^{pre}|$; it does not separately count the additional adjustment caused by fees. The exact first-month fee is $c/(1+c)$, while the current proxy is c; when $c = 0.001$, these are 0.000999001 and 0.001, respectively. The approximation differs little at low costs, but this proxy ledger must not be described as a real-trade ledger.

4. Methods

- Covariance: Ledoit–Wolf (2004) shrinkage toward a scaled identity matrix (main specification); sample covariance (sensitivity). See `src/quant_course/portfolio_construction.py`'s `ledoit_wolf_identity`.
- Minimum Variance: Chapter 9's `solve_mean_variance`, with zero means. Covariance is first divided by its largest element (which does not change the weights and avoids solver-tolerance distortion); 40% cap.
- Maximum Sharpe: `max_sharpe_long_only` uses the variable substitution $(\mathbf{y}, \kappa)$ to form a convex quadratic program (Chapter 9, Section 7, and 9-C01 prove the conditions for the transformation; Chapter 21 restates and implements it). Means are sample means of

industry returns less RF in the training window; 40% cap. This is not an "unproven convex QP": the ratio objective itself is non-concave; convexity comes from the transformation.

- ERC (no 40% weight cap): `risk_budget_weights` applies Newton iterations to $\frac{1}{2}\mathbf{y}^\top \Sigma \mathbf{y} - \sum_i b_i \ln y_i$. After stopping, it checks that each asset's volatility share differs from 1/10 by less than 10^{-6}, or raises an error. By definition each asset has a 10% capital weight under equal weighting; ERC is an uncapped risk-budget solution. Its observed maximum weight in the main sample is about 19.83%, but that is a sample result, not a constraint. Only Minimum Variance and Maximum Sharpe explicitly have a 40% cap; feasibility should be rechecked on other data.
- Fallback: only an exception caught by the try block containing the optimizer call causes an equal-weight holding for that month, with the reason recorded. Data validation, covariance estimation, and later `risk_contributions` are outside that catch; their failure can terminate the run. It is incorrect to say that every input error triggers a fallback. No shuffled K-fold is used, and no tuning is done during the evaluation period.

5. Ledger consistency

For every strategy and month, the revised `primary_ledger.csv` records the training window, status, pre-trade weights (the prior month's target weights after drift from its returns), target weights, turnover, cost, gross return, net return, and RF, including per-industry `pre_weight_*` and `target_weight_*` columns and an initial cash weight. `tests/test_project03_backtest.py` checks that: each training window ends before its holding month; changing future returns does not change past weights and returns; turnover uses drifted pre-trade weights; the net-return formula is $= (1 - \text{cost})(1 + \text{gross}) - 1$; gross return equals target weights times that month's returns; long-only, fully invested, and cap constraints hold; and fallbacks are recorded. The portfolio starts from cash before investing. There is no cash, borrowing, or short selling during the holding period; RF is used only for excess returns. `information_cutoff` is the last month in the training window; `decision_at`/`trade_at` always remains at the end of the month before holding. A lag moves the information cutoff earlier but does not move the trade time.

6. Main-specification results (W60, Ledoit–Wolf, 10 bp, 40% cap, lag 0; 1990-01..2026-08, 440 months)

Metric	Equal weight	Minimum Variance	Maximum Sharpe	ERC
Annualized mean net excess return	9.05%	7.03%	8.01%	8.66%
Annualized net volatility	14.32%	11.78%	14.36%	13.37%
Annualized net Sharpe	0.631	0.596	0.558	0.648
Annualized gross Sharpe	0.634	0.605	0.576	0.650
Net CAGR	11.25%	9.42%	10.11%	10.98%
Maximum drawdown (net)	−48.1%	−36.4%	−41.5%	−45.5%
Average monthly turnover (excluding first month)	2.6%	8.6%	21.5%	2.8%
Annualized mean fee as a share of old NAV	0.03%	0.11%	0.26%	0.04%
Annualized mean gross-minus-net return difference (bp)	3.37	10.65	26.20	3.64
Average maximum weight	10.0%	35.9%	37.8%	15.5%
Average effective number of assets $1/\sum w^2$	10.0	4.0	3.5	9.3
Average maximum ex ante risk share	15.9%	35.4%	43.5%	10.0%
Number of fallback months	0	0	1	0

Figure 1: The x-axis is month-end; the y-axis is net wealth on a logarithmic scale, with 1.0 at 1989-12 month-end. All four lines include 10 bp costs. Reading the plot: Equal weight and ERC nearly overlap at the top; Minimum Variance has shallower drawdowns in 2000–2002 and 2008 but the slowest long-run growth; Maximum Sharpe has the most volatile path.

Figure 2: Three stacked area plots, with holding month on the x-axis and target weight (summing to 1) on the y-axis; colors identify the 10 industries. Reading the plot: Minimum Variance remains concentrated in Utils, NoDur, and Hlth and often reaches the 40% cap; Maximum Sharpe rotates its holdings as a whole every few months, following the industry with the highest mean over the most recent 60 months (such as Enrgy in 2003–2008 and HiTec after 2019); ERC weights change slowly and remain close to equal weight.

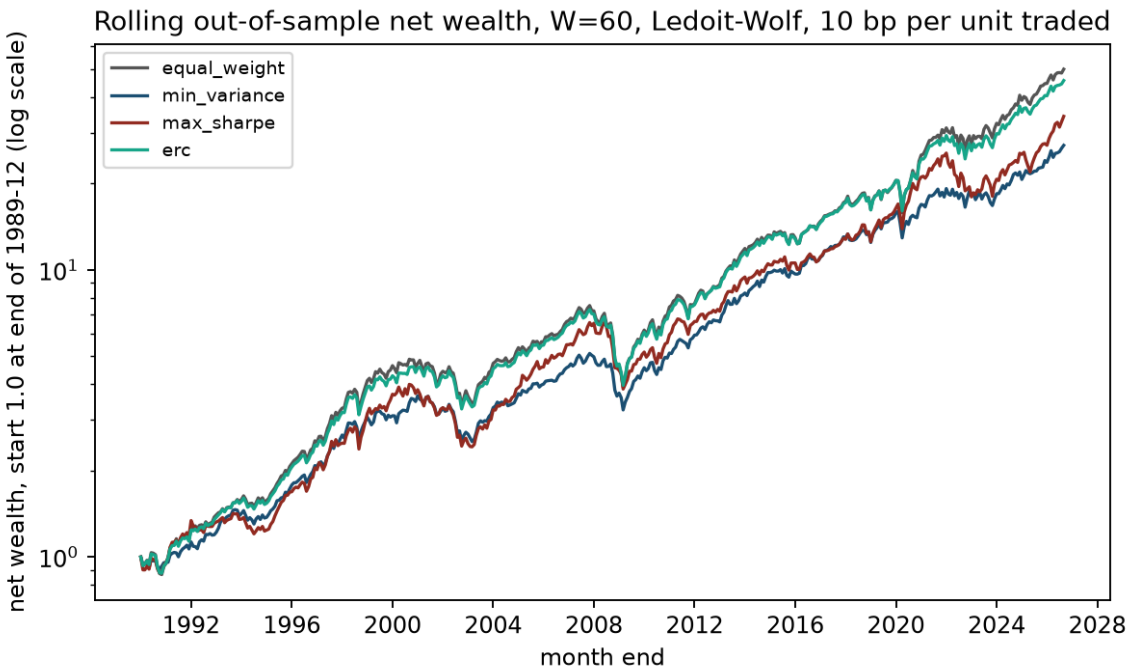


Figure 1: Net wealth

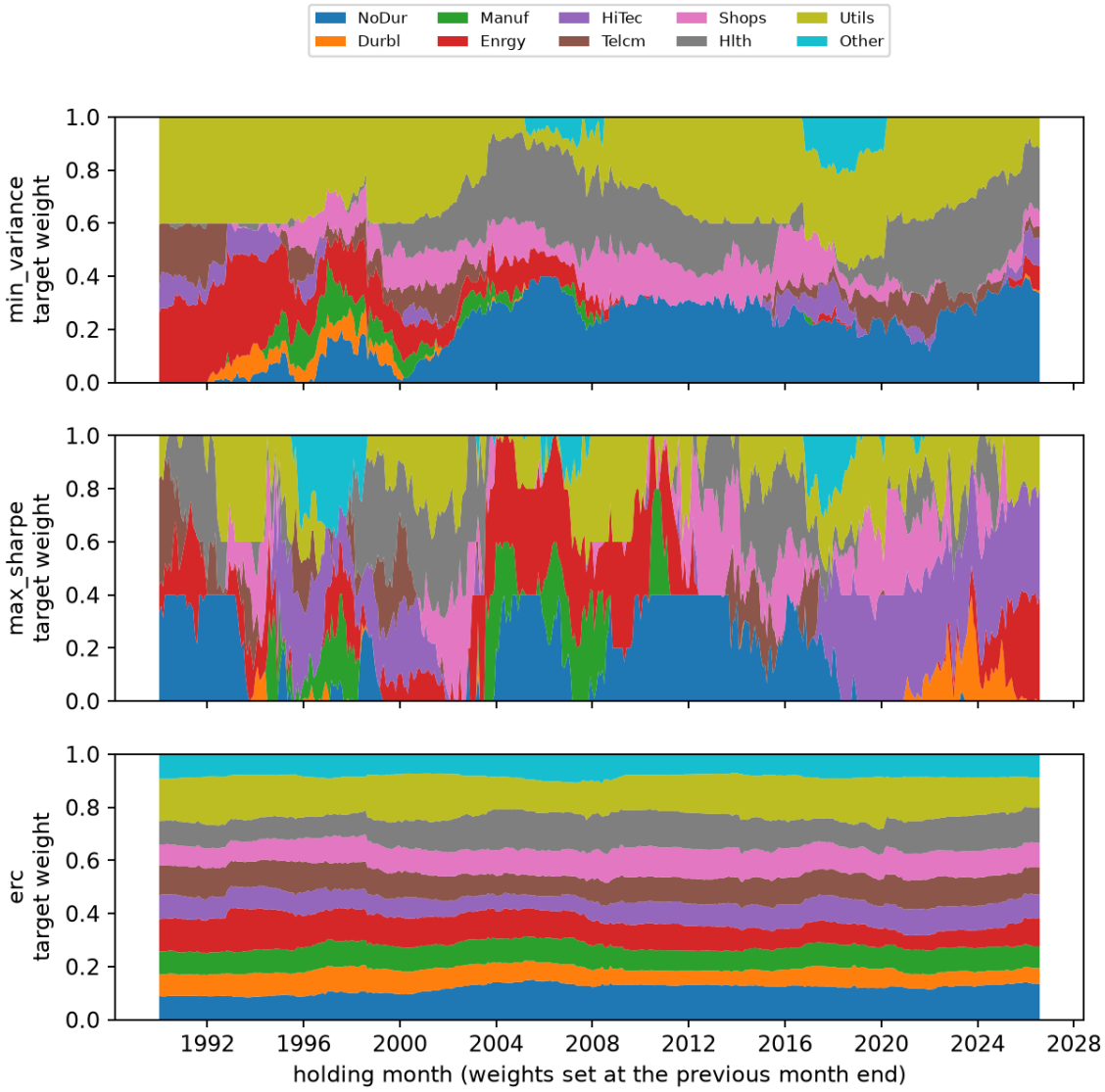


Figure 2: Target weights

7. Inference: are the differences distinguishable from zero?

A circular moving-block bootstrap (12-month block length, 5,000 replications, seed 20261006) estimates 95% percentile intervals for the mean of "strategy monthly net return − equal-weight monthly net return," and for paired-bootstrap differences in annualized net Sharpe:

Strategy	Mean net-return difference (annualized)	95% interval	Share of draws >0	Sharpe difference
Minimum Variance	−2.01%	[−3.97%, −0.06%]	2.2%	−0.035
Maximum Sharpe	−1.04%	[−3.30%, 1.27%]	18.4%	−0.073
ERC	−0.38%	[−0.92%, 0.13%]	7.8%	+0.016

Strategy	95% interval	Share of draws >0
Minimum Variance	[−0.174, 0.101]	29.2%
Maximum Sharpe	[−0.236, 0.089]	18.0%
ERC	[−0.014, 0.050]	84.1%

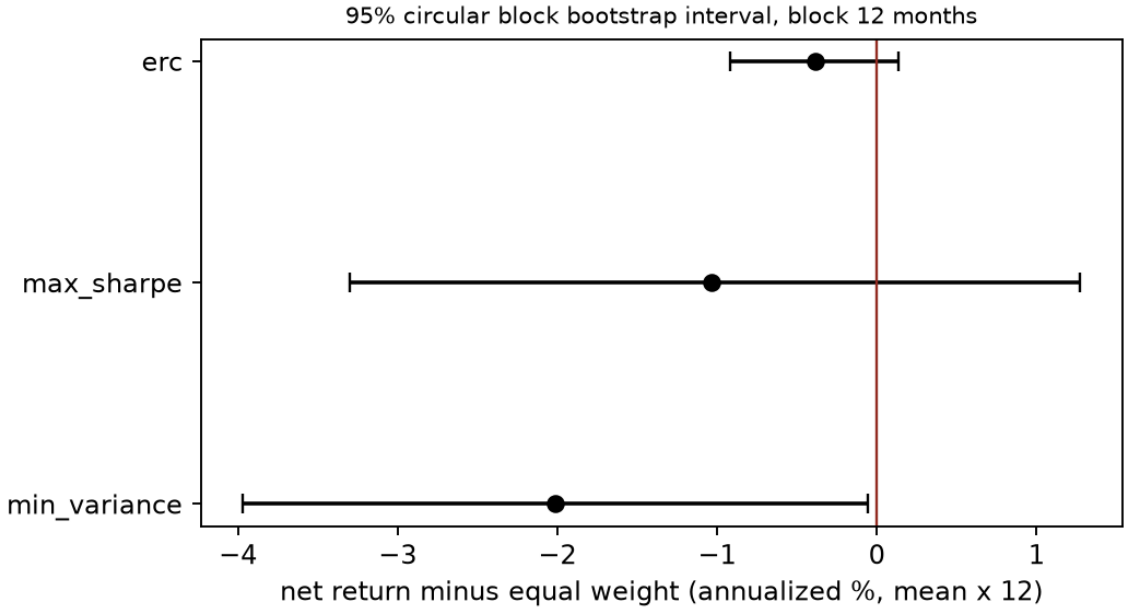


Figure 3: Relative to equal weight

Figure 3: The x-axis shows the annualized mean net-return difference (%); points are point estimates and error bars are 95% block-bootstrap intervals. The red line is zero.

Interpretation: Minimum Variance's mean net-return difference lies below zero in the unadjusted single-comparison 95% interval; volatility is lower, and it is indistinguishable from equal weight on a risk-adjusted basis. Maximum Sharpe is negative on both measures, with intervals crossing zero. ERC's Sharpe point estimate is slightly higher (0.016), but its interval crosses zero and its mean return is slightly lower. None of the three portfolios passed H1's research screen. An interval crossing zero means there is insufficient evidence of an advantage; it does not prove the strategies equivalent or that the overall advantage is zero. The circular-block method also assumes

sampled blocks can represent the target process; blocking does not remove nonstationarity across 1990–2026. Percentile intervals and the "share of draws >0" are neither posterior probabilities of an advantage nor calibrated joint p-values. The intervals reflect uncertainty under this resampling scheme only; they do not include model selection, data revisions, or multiple comparisons (the report compares 3 strategies × 2 metrics and applies no multiple-testing correction).

8. Robustness: window, covariance estimator, lag, and cost

The complete table is in `outputs/sensitivity.csv` (12 specifications × 4 strategies × 4 cost tiers, 192 rows total; all are retained). Annualized net Sharpe at 10 bp:

Specification	Equal weight	Minimum Variance	Maximum Sharpe	ERC
W36, LW, lag 0	0.631	0.572	0.474	0.637
W60, LW, lag 0 (main)	0.631	0.596	0.558	0.648
W120, LW, lag 0	0.631	0.617	0.549	0.647
W60, sample covariance, lag 0	0.631	0.577	0.551	0.648
W60, LW, lag 1	0.631	0.597	0.556	0.649

Cost tiers (main specification, 0/10/25/50 bp): Equal weight 0.634/0.631/0.628/0.622; Minimum Variance 0.605/0.596/0.583/0.560; Maximum Sharpe 0.576/0.558/0.531/0.485; ERC 0.650/0.648/0.643/0.636.

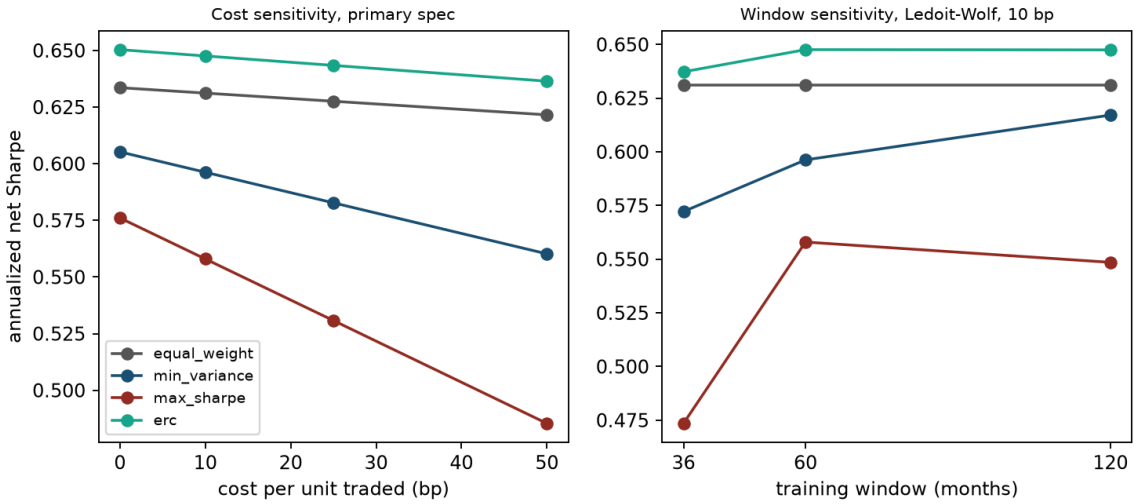


Figure 4: Sensitivity

Figure 4: In the left panel, the x-axis is cost per unit of turnover (bp) and the y-axis is annualized net Sharpe. In the right panel, the x-axis is the training window (months), with cost fixed at 10 bp. Reading the plot: Maximum Sharpe has the steepest slope (highest turnover); shorter windows make mean estimates noisier and worsen Maximum Sharpe and Minimum Variance; ERC and equal weight are insensitive to both.

Findings:

- The ranking is the same across all 12 specifications: ERC is close to equal weight, Minimum Variance is next, and Maximum Sharpe ranks lowest.
- Shrinkage covariance helps Minimum Variance slightly (W60: 0.596 versus 0.577) and has almost no effect on ERC.
- A 1-month lag barely changes the results. This shows that the conclusion is not sensitive to the assumption of availability at month-end; it does not show that the data are PIT.
- At W36, Maximum Sharpe has 14 fallback months: in 8 months all industries had negative excess means in the training window; in 6 months, at least one industry had a positive mean but no positive-excess-return portfolio was feasible under the 40% cap (around 2009).

9. Subperiods

Period	Equal weight	Minimum Variance	Maximum Sharpe	ERC
1990-01..2007-12 net Sharpe	0.611	0.494	0.534	0.628
2008-01..2026-08 net Sharpe	0.653	0.686	0.579	0.668
1990-01..2007-12 maximum drawdown	−31.3%	−30.9%	−39.4%	−28.7%
2008-01..2026-08 maximum drawdown	−45.8%	−34.7%	−41.5%	−43.5%

Minimum Variance has a higher Sharpe than equal weight only in the later subperiod. The ranking changes by subperiod, so it cannot be extrapolated.

10. Concentration, risk, and return sources

Minimum Variance and Maximum Sharpe average about 4 effective assets, with maximum ex ante risk shares of 35%–44%; by construction, ERC assigns about 10% of ex ante volatility to each industry. The industries' contributions to annualized mean gross return under the main specification (average $w_{i,t}R_{i,t} \times 1200$, %/year):

Industry	Equal weight	Minimum Variance	Maximum Sharpe	ERC
NoDur	1.03	2.01	1.57	1.20
Durbl	1.36	0.19	−0.12	0.91
Manuf	1.23	0.46	1.01	1.07
Enrgy	1.20	1.19	1.79	1.21
HiTec	1.60	0.43	3.11	1.34
Telcm	0.78	0.02	0.28	0.78
Shops	1.25	1.13	0.90	1.23
Hlth	1.21	1.45	1.17	1.30
Utils	0.99	2.89	0.49	1.46
Other	1.11	0.07	0.77	0.89

Industry	Equal weight	Minimum Variance	Maximum Sharpe	ERC
Total	11.76	9.84	10.97	11.39

Minimum Variance held less of Durbl, HiTec, and Manuf, whose evaluation-period mean returns were higher, and allocated more weight to Utils and NoDur, whose means were lower. This accounts for its roughly 2% annualized return gap: lower volatility came with lower mean return, not "free" risk reduction. This is an accounting decomposition of contemporaneous returns, not a causal attribution.

11. Hindsight comparison (not tradable)

Using sample moments over the entire 1990-01..2026-08 evaluation period and the same 40% cap, the maximum-Sharpe weights produce an annualized Sharpe of 0.754 for a fixed-weight target portfolio's gross-return path (the denominator is the sample standard deviation of same-month gross return minus RF) (Utils 31%, Hlth 29%, HiTec 18%, Shops 16%, Enrgy 4%, NoDur 2%). The rolling version has 0.576 at the same zero cost, and 0.558 after 10 bp fees. The solver's objective value for the supplied sample moments is 0.753856, with asset gross-return covariance in the denominator; the path-based value is 0.754281, with realized excess-return standard deviation in the denominator. Varying RF by month makes them slightly different, so they must not be described as the same number. The in-sample portfolio uses all information from the evaluation period; its estimation window, fixed weights, and rolling strategy also differ. These differences do not separately identify a causal effect of look-ahead bias and cannot be treated as tradable performance.

12. Solver failures and fallback records

Under the main specification, Maximum Sharpe had 1 fallback month (2003-04, when no positive-excess-return portfolio was feasible under the 40% cap); the other strategies had 0. Records for all 12 specifications are in `outputs/fallback_log.csv`. Two implementation problems occurred during development: ERC Newton stalled near 10^{-12}, and the old code incorrectly reported "not converged" and fell back to equal weight; CLARABEL returned infeasible for capped infeasible problems. Both have been fixed and their causes documented. After the fixes, ERC has 0 fallbacks, and all infeasible cases are identified in the input pre-check.

13. Negative and zero results

The project's main finding is negative: for 10 industries, monthly data, and long-only constraints, none of the three estimated-moment portfolios credibly outperformed equal weight after costs. Maximum Sharpe is the only portfolio that estimates means; it also performed worst, had the highest turnover, and was most sensitive to costs and window length. This is directionally consistent with Chapter 21's estimation-error simulation and the working-paper version of DeMiguel, Garlappi, and Uppal (the project read only the methods and abstract of their 2006 conference paper). The intervals for ERC's differences from equal weight cross zero: this sample

detects no return advantage and does not establish equivalence. Its ex ante risk distribution is more even, but this project does not infer higher returns from that structure.

14. Limitations

- Current revised history, not PIT; industry portfolios are not directly tradable; borrowing, financing, taxes, and capacity constraints are excluded.
- There are only 10 assets, so the conclusions do not generalize to a large stock universe.
- Monthly decisions ignore the intramonth path and execution timing.
- Costs are modeled as linear rates, with no market impact or nonlinear scale effects.
- The block bootstrap assumes within-block dependence captures serial correlation; no multiple-comparison correction was applied.
- Only one cap (40%) and one mean estimator (sample mean) were tested; more variants increase selection bias and should be pre-registered separately.

15. Conclusion

Under this project's data and assumptions, none of the three strategies passed H1's research screen; this is not a formal statistical rejection of an "overall advantage." Equal weight and ERC are difficult to distinguish in out-of-sample net Sharpe. Minimum Variance reduces volatility and drawdown but has a lower point estimate and a mean-return difference below zero in the unadjusted single-comparison interval. Maximum Sharpe has the highest turnover and is most sensitive to costs. Estimation error in the mean is a theoretically possible mechanism, but this comparison does not separately identify its causal contribution. A high in-sample Sharpe does not provide evidence of a tradable advantage.

16. Historical reproduction route

```
.venv/bin/python projects/project03/run_project03.py
.venv/bin/python -m pytest tests/test_project03_backtest.py tests/
    ↪ test_ch21_portfolio_construction.py -q
```

The commands above are historical course-work-root commands and require the corresponding environment and authorized data. The historical record shows a runtime of about 25 seconds; this is not a promise about the current default run. This route writes to `projects/project03/outputs/` and `figures/projects/project03_*.png`; the default offline constructed entry point does not run it or generate this report's bootstrap or sensitivity tables. Notebook: `notebooks/project03_portfolio_oos.ipynb`. Exercises and solutions: exercises.md, solutions.md.

17. Sources and review scope

The rolling window and turnover after weight drift follow DeMiguel, Garlappi, and Uppal, 2006 conference paper Section 3 (printed pp. 10–12, equations 10–13); the `return-difference/annualization` boundary follows Sharpe's 1994 author webpage, "Definitions" and "Time Dependence." The historical review read the cited sections; it does not claim to have read the 2009 journal version. See Chapter 21 source notes for the authors' source-review scope for `covariance/ERC`; data source and hashes are in the manifest. Project metrics and intervals were computed by local code and are not statistics from the cited papers.

Chapter 22 — Quantitative Research Methodology

Core idea: Freeze the hypothesis, sample, and evaluation method before seeing the final test results, and record every attempt. Otherwise, no number called "significant" can answer whether it reflects a real pattern or noise discovered through search. Prerequisites: Chapter 3 (data timing and inventories), Chapter 7 (mean inference and bootstrap), Chapter 11 (compounding returns), Chapter 15 (regression and HAC), and Chapter 21 (portfolio construction and turnover). By the end, you will be able to write and hash a frozen research plan with ten stages; maintain a trial ledger; calculate Bonferroni, Holm, and Benjamini–Hochberg adjustments by hand; derive the expected inflation when taking the maximum of N null trials; distinguish "writing the plan first" from "mining after the fact" and explain why a local hash cannot prove which came first; and complete a holdout evaluation on real French industry data that includes a negative result.

1. Chapter Objectives

This chapter addresses a question shared by Chapters 22–28: when you claim that "this signal works," why should anyone believe that you did not simply choose the best of twenty versions?

After completing this chapter, readers should be able to do five things:

1. Put the inputs, outputs, audit trail, and typical failure for each of the ten stages—"economic hypothesis → data → cleaning → features → signal → portfolio → backtest → validation → cost model → out of sample"—in a table, and explain how each stage connects to Chapters 3, 7, 11, 15, and 21.
2. Write a frozen research plan: hypothesis and sign, sample windows, a single primary metric, numerical pass/fail criteria, cost assumptions, and a trial budget; calculate its SHA-256; and explain what the hash does and does not prove.
3. Maintain a trial ledger that records the specification, metric, sample size, and status of every evaluated variant, including abandoned attempts and undefined results.
4. Calculate and explain $1-(1-\alpha)^M$, Bonferroni, Holm, and Benjamini–Hochberg by hand; write the expected maximum of N independent standard normal variables and check the approximation; and understand why correlated variants are not the same as independent trials.
5. Follow a complete discovery–holdout evaluation, including a result that fails its criterion on real data, and write the conclusion according to the rule set in advance.

The standard for "complete" is not a small p-value. Any reader should be able to use only your plan, ledger, code, and data hashes to recompute every reported number and see every variant you tried.

2. Why Quant Needs This

Chapters 14–21 gave you measurement tools: regression and HAC standard errors (Chapter 15), unit roots and cointegration (Chapter 17), and portfolio construction and costs (Chapter 21). Part IV begins "systematic alpha research," where the same historical data are repeatedly used to test many signals. More tools also mean more ways to make mistakes: you can change the lookback window, winsorization threshold, number of buckets, sample start, or label horizon. Each change produces a new t-statistic.

Two facts show that this is not merely a theoretical concern. First, in the abstract of their 2014 NBER working paper, Harvey, Liu, and Zhu wrote that hundreds of papers and hundreds of factors had tried to explain the cross-section of expected stock returns. Given this extensive data mining, they argued that continuing to use the usual t-ratio threshold of 2.0 for a newly discovered factor was unjustified in that multiple-testing setting; their multiple-testing framework suggested a t-ratio above 3.0 (corresponding to a p-value of 0.27%) for new factors. They also noted that the true number of attempts includes unpublished failures, so even 3.0 may be too low. Second, Gelman and Loken (2013, unpublished manuscript) pointed out that multiple comparisons arise through "forking paths" even when a researcher performs only one analysis and holds the hypothesis in advance, if analytical details—such as which observations to exclude, which contrast to measure, or which variables to control—depend on what the data look like. They also emphasized that a selected analysis being a deterministic function of the observed data does not remove the multiple-comparison problem: a p-value asks what would happen with another possible dataset.

Together, these points determine the logic of this chapter: first define which "attempts" count; then use a frozen plan to lock in the choices that can be fixed; evaluate on a holdout sample; put adjusted significance thresholds into the decision rule; and decide what to report. This chapter does not teach a particular signal or provide conclusions for the later chapters on Value, Momentum, or Quality. It provides a process that makes those chapters' conclusions inspectable.

3. Intuition

Imagine trying to prove that you can guess coin flips correctly. In the first approach, before flipping the coin, you write down the rule: "I will guess ten times, and I count as skilled if I get at least nine right." Then you flip. In the second approach, you flip ten times, look at the results, then decide whether you meant to predict the first five flips or the odd-numbered ones, and find a rule that describes the sequence. Both approaches might report "nine correct," but only the first makes that number informative.

In financial research, the "rule" corresponds to a set of choices: how many months to look back, how many to skip, whether to winsorize, how many buckets to use, which sample period to choose, and whether to exclude small-cap stocks. You may not feel that you are "fishing," because each choice has a rationale. The problem is that if the data looked different, you could find equally good reasons for a different set of choices. That freedom to "always find a reasonable story after the fact" is what Gelman and Loken call forking paths.

State the limits of the simplification. There are 32 independent opportunities only if each attempt uses a fresh, independent batch of ten coin flips. Different post hoc subsets or rules applied to the same ten flips are not thereby independent. Financial variants are different: signals with 11-month and 12-month lookbacks are highly correlated, and their t-statistics tend to rise and fall together. The Monte Carlo experiment in this chapter will show that 32 mutually correlated variants correspond to only about 4–5 effective independent trials. Here, the "effective number" is matched to the mean of the maximum absolute statistic. It describes only one moment; it is not the size of the test family and does not guarantee the same tail probability. Bonferroni still counts the actual 32 attempts. Correlation does not let you pretend you looked only once.

3.1 From an Economic Question to a Deliverable Research Object

Start by asking, "Why might stocks with a high book-to-market ratio earn higher returns next month?" One economic explanation is that investors require compensation for distress risk; another is that prices underreact to information. Both imply a positive-direction prediction, but they call for different falsification tests and risk controls. So write the mechanism first, then a measurable statement: "Within a prespecified stock universe, financial-statement availability rule, and set of months, higher B/M corresponds to higher returns next month." Each step below delivers a concrete object.

Step	Operation and units in this example	Recipient of the output
Observable input	Same-vintage book equity and contemporaneous market capitalization that were public and ingested by the cutoff, both in the same currency	Feature calculation
Feature	$B/M =$ book equity/market capitalization, dimensionless; nonpositive equity is set to missing under a fixed rule	Same-date preprocessing
Signal	Percentile rank of valid B/M values on the same date, in the range 0–1; record the specification separately if comparing an industry-relative version	Prediction evaluation and weight construction
Prediction	The primary prediction is a positive relative ordering; to predict return magnitudes, estimate and freeze the return mapping in the discovery window	Holdout label evaluation
Portfolio	Buy the high bucket and sell the low bucket, with each leg's weights summing to $+1$ and -1, for gross exposure of 2; list constraints and tradable universe separately	Trade/position ledger
Evaluation	IC by date, returns before and after costs, benchmark differences, and valid dates; decide under the frozen rule	Research conclusion and next research log

A "high signal" is not a prediction of "an 8% return"; a percentile gives only an ordering. IC evaluates ordering or a linear relationship, while portfolio returns also depend on weights, execution timing, and costs. Chapter 23 covers prediction evaluation; Chapters 21 and 29 cover weights and the ledger. When the data have already been examined repeatedly, this process can constrain the choices made now, but it cannot erase past exposure.

4. Mathematical Foundations

This chapter uses only:

- The one-sample mean t-statistic. If $x_1, \ldots, x_T$ are monthly observations of a metric, the sample mean is $\bar{x} = \frac{1}{T}\sum_t x_t$, and the sample standard deviation is s, then $t = \bar{x}/(s/\sqrt{T})$. The denominator $s/\sqrt{T}$ is the standard error of the mean and has the same units as x_t; t is dimensionless. The metric here is the monthly cross-sectional Information Coefficient (IC, defined in Chapter 23 and treated here simply as a dimensionless monthly correlation), and T is the number of months.
- Long-run variance and the Newey–West adjustment. If x_t is serially correlated, the variance of its mean is no longer s^2/T. The Bartlett-kernel long-run variance is $\text{LRV} = \gamma_0 + 2\sum_{\ell=1}^{L}(1 - \frac{\ell}{L+1})\gamma_\ell$, where $\gamma_\ell = \frac{1}{T}\sum_{t=\ell+1}^{T}(x_t - \bar{x})(x_{t-\ell} - \bar{x})$; the estimated variance of the mean is LRV/T. $L = 0$ gives the independent and identically distributed formula (with divisor T, not $T-1$). Chapter 15 used this for regression coefficients; here it applies directly to a constant-only regression.
- The union bound in probability theory (Boole's inequality): for any events $A_1, \ldots, A_m$, $\Pr(\cup_i A_i) \le \sum_i \Pr(A_i)$, with no independence requirement.
- Order statistics: the maximum $M_N = \max_i Z_i$ of N independent, identically distributed variables satisfies $\Pr(M_N \le x) = F(x)^N$.
- The null-hypothesis property of p-values: for a valid test, $\Pr(p \le u) \le u$ for $u \in [0, 1]$; equality holds for an exact p-value calibrated to the correct continuous null distribution; that p-value is uniform under the null.

Frequency convention: unless stated otherwise, t in this chapter tests the mean of a monthly metric, and the label horizon is expressed in months as h. Overlapping labels ($h > 1$) make the metric series serially correlated across adjacent months. Taking $L = h - 1$ is common, but can still underestimate variance in a finite sample; the Monte Carlo experiment in this chapter will show this.

5. Formal Definitions

Research hypothesis: A falsifiable statement that names an object, direction, and horizon, such as "stocks with a high book-to-market ratio earn higher returns than stocks with a low book-to-market ratio over the next month (positive sign)." The directional economic hypotheses in this chapter require a clear object, sign, and horizon. "The value factor works" supplies no specific decision criterion and invites post hoc interpretation. This does not rule out a prespecified two-sided primary test: it need not predict the sign, but must specify the object, horizon, null hypothesis, and two-sided rejection rule in advance.

Research plan and freeze: A research plan is a file specifying the hypothesis, stock universe, data sources, discovery and holdout windows, signal definition, label definition, portfolio rule, benchmark, single primary metric, numerical pass/fail criteria, cost assumptions, and trial budget (the maximum number of variants to inspect). To freeze it, compute the SHA-256 of its normalized JSON and record the local clock time. The hash carries the previous step, "write down the plan," into the next step, "evaluate the holdout," as a reference that cannot be silently changed; the next definition explains its limits.

Local freeze and preregistration: Under collision-resistance and related cryptographic assumptions, a local freeze lets you compare normalized current content with a trusted saved digest. A match supports content identity with that reference; the digest itself records no writing or editing history, and the timestamp comes from your own machine. Preregistration requires a registration maintained outside the research team with an independent timestamp, such as a public registry or a file held by a third party. Only that provides external evidence that the plan preceded the data results. This chapter's `ResearchPlan` and `freeze_plan` perform only a local freeze. Gelman and Loken also noted that preregistration of public data analyzed repeatedly by many people has "little point," because a researcher cannot pretend to know nothing about the data. The French industry data used here fall into this category, so the conclusion in Section 11 can serve only as a process demonstration.

Trial and trial ledger: A trial is any evaluation of a variant whose results you read to judge whether a hypothesis holds. The ledger is an append-only table of trial ID, specification, metric, sample size, status (primary / run / abandoned), and notes. A repeated run of the same specification still gets a row. An attempt with an undefined result (NaN) also gets a row, because you still looked at the data. The number of ledger rows is a lower-bound input for multiple-testing adjustments.

Research log: The ledger records "what was done and what it produced"; the log records "why it was done and why it changed." Keep them separate: the ledger is data, the log is narrative.

Discovery sample, holdout sample, and embargo: Use the discovery sample to choose specifications; evaluate on the holdout only once, after the specification is fixed. If a label spans h months, the label for the last signal in the discovery sample uses returns from the first few months of the holdout. Dropping h months of signals between the two periods (an embargo) avoids leakage from overlapping labels; this is a precursor to the purging in Chapter 30.

Benchmark: A reference that answers, "What would happen without this signal?" This chapter uses three kinds: a no-signal benchmark (equal-weight holdings), a permutation placebo (randomly shuffle asset labels within each month, preserving that month's label set and the time order), and a simple published rule (for example, fixed 12-1 momentum).

Ablation: With all other steps fixed, remove one component at a time (winsorization, rank transform, lag, and so on) and observe how the result changes to identify the role of each component. Ablations are trials and must be entered in the ledger.

Negative result and reproducibility: A negative result is one that does not support the hypothesis under the prespecified criterion; report it as fully as a positive result. Reproducibility means that anyone with the code, data hashes, random seeds, and ledger obtains the same numbers.

Family-wise error rate and false discovery rate (FWER and FDR): Among M hypotheses, let V be the number of true null hypotheses rejected, and R the total number rejected. FWER $= \Pr(V \geq 1)$; FDR $= \mathbb{E}[V/\max(R, 1)]$, the expected fraction of false rejections among all rejections. HLZ (2014) bound FDR by FWER: if $V \geq 1$ and $R > 0$, then $V/R \leq 1$, so $\mathbb{E}[V/R \mid R > 0] \Pr(R > 0) \leq \Pr(V \geq 1)$.

Ten-stage research pipeline: The table below is the main structure of this chapter. Every stage has inputs, outputs, an audit trail, and a typical failure. Read it from left to right: the output of one stage becomes the input to the next. The plan is frozen at Stage 1, the cost model is at Stage 9, and holdout evaluation is at Stage 10.

Stage	Input	Output and audit trail	Typical failure	Related chapters

Stage	Input	Output and audit trail	Typical failure	Related chapters
1 Economic hypothesis	Mechanism, literature, your prior	One sentence with sign, object, and horizon; plan file and hash	No sign; can be interpreted arbitrarily after the fact	22
2 Data	Source, authorization, version	Source URL, download time, SHA-256, `available_at`	Passing a current revision off as historically available data	3, 13
3 Cleaning	Raw table	Written rules for missing values and outliers; cleaning log	Deciding whom to exclude after seeing returns	3
4 Features	Cleaned data	Feature definition with window, units, and availability time	Using data that arrive after `signal_time`	11, 12
5 Signal	Features	Signal transformed cross-sectionally on the same date	Full-sample standardization; using a future mean today	23, 28
6 Portfolio	Signal	Target weights, constraints, turnover	Mixing constraints into the signal definition	21, 23
7 Backtest	Weights, returns	Label interval, lags, gross returns, and ledger	Not counting repeated iterations on the same data	22, 29, 30
8 Validation	Backtest results	Holdout evaluation, benchmarks, ablations, permutations	Tuning on the holdout sample	22, 31
9 Cost model	Turnover, fee rates	Gross/net returns; capacity assumptions	Applying fees to the wrong weight changes	21, 35
10 Out of sample	Frozen specification	One-time evaluation; report negative results too	Changing the hypothesis after seeing results and calling it “out of sample”	22, 31

Time fields run through all stages: when an event occurs (`event_time`); when the system can access it (`available_at`); when it is ingested locally (`ingested_at`); the decision time (`signal_time`); execution time (`trade_time`); and the return and label intervals. An executable study requires both `available_at` and `ingested_at` to be no later than the decision cutoff, and `signal_time < trade_time`; measure returns from the actual execution price. This chapter’s month-end label treats a hypothetical month-end trade as a research proxy. A later label month does not mean that all returns occurred after a real trade. See the fields and clocks in Section 5.2 of Chapter 23.

5.1 A Small Ledger: How a Primary Hypothesis, Exploration, and Holdout Coexist

The five rows below are a constructed teaching example, not a historical market result from this chapter. The plan first specifies the primary B/M specification and positive direction, discovery window D, gap window, and holdout window H, and registers "at most five evaluations in the discovery stage." The p-values in the table are only for demonstrating multiple-testing calculations.

ID	Specification and status	Results read from	p-value	Conclusion status
D01	Primary specification, same-date percentile rank	D	0.01	Primary-hypothesis discovery result
D02	Changed to industry-relative ranks after seeing D01	D	0.04	Post hoc exploration
D03	Changed the winsorization threshold after seeing results	D	0.03	Post hoc exploration
D04	Constant signal; IC undefined	D	Missing	Undefined attempt; retain reason and sample size
D05	Best result after changing label horizon	D	0.005	Post hoc exploration; cannot be relabeled as the primary specification

These five attempts serve the same discovery decision and, under this chapter's all-attempts convention, form one test family, $M = 5$. D04 has no valid p-value, so set $p = 1$ for the calculation; it has not been "tested as nonsignificant." In the original row order, the Bonferroni-adjusted p-values are $(0.05, 0.20, 0.15, 1, 0.025)$; Holm gives $(0.04, 0.09, 0.09, 1, 0.025)$; BH gives $(0.025, 0.05, 0.05, 1, 0.025)$. Section 6 explains the multipliers, cumulative maxima/minima, and error-rate conditions. Padding with 1 preserves the number of attempts; it cannot repair an invalid p-value or a data-dependent test design.

Record the holdout evaluation separately as H01 and link it to the question "does it evaluate D01 or D05, selected in the discovery window?" If the plan registers only D01 as the primary confirmatory test, report the holdout result for D05 as post-exploration validation. If the plan prespecifies the complete algorithm of selecting a specification in the discovery window and confirming it on the holdout, H01 evaluates the specification chosen by that algorithm. Fix either plan before opening H; do not switch between them after seeing H01. Once the holdout has been read, changing the threshold is another attempt: add a ledger row and retain the original H01.

The local hash binds the plan's contents; a third party's record supports the freeze time; the fact that the French history was used before records prior exposure. These answer different questions—"what was the content?", "when was it written?", and "what historical data were

already known?"—and cannot replace one another. Separating the discovery and holdout windows reduces label overlap, but does not guarantee statistical independence in financial time series; inference still requires checking dependence and sample size.

6. Derivations

6.1 Family-Wise Error Rate Under Multiple Testing

Suppose there are M independent tests, each rejecting a true null hypothesis with probability α. When all null hypotheses are true, the probability of at least one rejection is

$$\text{FWER} = 1 - (1 - \alpha)^M.$$

The derivation uses only independence: the probability of no rejection is $1-\alpha$, the probability of no rejection in all M tests is $(1 - \alpha)^M$, and the complement is the desired probability. For $\alpha = 0.05$, the values are 0.05 for $M = 1$, 0.226 for $M = 5$, 0.401 for $M = 10$, 0.806 for $M = 32$, and 0.994 for $M = 100$. To keep family-wise error at α, the single-test level under independence is $1 - (1 - \alpha)^{1/M} \approx \alpha/M$ (Šidák).

Interpretation: with 32 independent attempts, the probability of at least one "significant" result is 81%, which is why the plan needs a "trial budget." This assumes the tests are independent. Dependence does not guarantee that the probability is below 81%; the specific constructed experiment in Section 13 measured 23.6%, which cannot be generalized as a bound for arbitrary dependence structures.

6.2 Bonferroni Correction and the Union Bound

No independence is required. Let I_0 be the index set of true null hypotheses, with $m_0 = |I_0| \le M$. Bonferroni rejects when $p_i \le \alpha/M$. Then

$$\Pr(\text{there exists } i \in I_0 : \ p_i \le \alpha/M) \ \le \ \sum_{i \in I_0} \Pr(p_i \le \alpha/M) \ \le \ m_0 \cdot \frac{\alpha}{M} \ \le \ \alpha.$$

The first step is the union bound, the second uses $\Pr(p \le u) \le u$ under the null, and the third uses $m_0 \le M$. Equivalently, compare the adjusted p-value $p_i^{\text{Bonf}} = \min(Mp_i, 1)$ with α. The cost is lower power: every test must meet the α/M threshold.

6.3 Why Holm's Step-Down Procedure Also Controls FWER

Sort $p_{(1)} \le \cdots \le p_{(M)}$ and let the corresponding hypotheses be $H_{(1)}, \ldots, H_{(M)}$. Holm's procedure finds the smallest k such that $p_{(k)} > \alpha/(M+1-k)$, rejects $H_{(1)}, \ldots, H_{(k-1)}$, and does not reject the rest. HLZ (2014, Section 4.4.2) gives this procedure; the FWER proof below is provided by this book's author, not by that paper.

Let j be the position of the first true null hypothesis in the sorted list. If Holm makes at least one false rejection, it must reject $H_{(j)}$ (the procedure rejects in order, so passing the first true null means making a false rejection). Rejecting $H_{(j)}$ means that $p_{(k)} \le \alpha/(M+1-k)$ for every $k \le j$, and in particular $p_{(j)} \le \alpha/(M+1-j)$. All $j-1$ hypotheses before it are false nulls, so $j-1 \le M-m_0$, or $M+1-j \ge m_0$. Thus $p_{(j)} \le \alpha/m_0$. This event is contained in "at least one true null has a p-value $\le \alpha/m_0$," whose probability is at most $m_0 \cdot \alpha/m_0 = \alpha$ by the union bound. QED.

Comparison: Holm and Bonferroni use the same threshold at Step 1 (α/M). The threshold then rises step by step as $\alpha/(M+1-k)$, so Holm rejects every hypothesis Bonferroni rejects and does not require independence either. The adjusted p-values are $p^{\text{Holm}}_{(i)} = \min\{\max_{j\le i}(M-j+1)p_{(j)}, 1\}$, using a cumulative maximum to ensure monotonicity.

6.4 Benjamini–Hochberg and the False Discovery Rate

The BH procedure finds the largest k such that $p_{(k)} \le \frac{k}{M}q$ and rejects $H_{(1)}, \ldots, H_{(k)}$. The adjusted p-values ("q-values") are $p^{\text{BH}}_{(i)} = \min_{j\ge i}\{\min(\frac{M}{j}p_{(j)}, 1)\}$. The known result is that FDR $\le \frac{m_0}{M}q$ when tests are independent or satisfy a positive-dependence condition. This chapter does not prove that result and has not read the original Benjamini–Hochberg paper; it uses the procedure itself as a filter for "exploratory discoveries." Chapter 17 used it for cointegrating pairs. FWER requires virtually no false rejections; FDR permits a small share of false rejections. FDR fits exploratory work, while a claim about a single factor calls for something closer to the FWER standard.

6.5 How Large Is the Maximum of N Null Trials?

Let $Z_1, \ldots, Z_N$ be independent, identically distributed $N(0,1)$ variables, and let Φ and φ be the CDF and density. The maximum has CDF $\Phi(x)^N$ and density $N\varphi(x)\Phi(x)^{N-1}$, so

$$\mathbb{E}[\max_i Z_i] = \int_{-\infty}^{\infty} x\, N\varphi(x)\Phi(x)^{N-1}\, dx.$$

For $N=2$, calculate by hand: $\max(Z_1, Z_2) = \frac{Z_1+Z_2}{2} + \frac{|Z_1-Z_2|}{2}$. The first term has expectation zero; $Z_1 - Z_2 \sim N(0,2)$ and $\mathbb{E}|Z_1 - Z_2| = 2/\sqrt{\pi}$, so the expected maximum is $1/\sqrt{\pi} = 0.5642$.

Bailey and López de Prado (2014, Eq. (1), Appendix A.1) give this large-N approximation:

$$\mathbb{E}[\max_i Z_i] \approx (1-\gamma)\,\Phi^{-1}\Big(1-\frac{1}{N}\Big) + \gamma\,\Phi^{-1}\Big(1-\frac{1}{Ne}\Big),$$

where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant and e is the base of the natural logarithm. If the Sharpe estimates from separate trials are normally distributed under the null with mean 0 and standard deviation σ, the expected maximum is σ times the expression above. This chapter checks it by numerical integration:

N	Approximation	Numerical integration	Relative error
2	0.520	0.564	−7.9%
5	1.193	1.163	+2.5%
10	1.575	1.539	+2.3%
32	2.100	2.070	+1.5%
100	2.531	2.508	+0.9%
1000	3.255	3.241	+0.4%

The approximation is inaccurate for small N; the authors describe it as a "large N approximation." For a numerical interpretation, the null standard deviation of a monthly Sharpe estimate from $T = 120$ months is about $1/\sqrt{120} = 0.0913$. Take the maximum of 32 independent null trials: the expected value is about $0.0913 \times 2.100 = 0.192$ per month, or about 0.66 annualized after multiplying by $\sqrt{12}$. In other words, from a family of strategies with no signal, the best one looks like it has an annualized Sharpe of 0.66 on average. This assumes independent trials and approximately independent, identically distributed returns.

6.6 Why Evaluation After Selection Requires New Data

Suppose you select the variant with the largest t in the discovery sample, and call its discovery statistic t_D^*. Since t_D^* is the maximum of N values, it is upward biased; Section 6.5 quantifies the bias. The distribution of the holdout statistic t_H depends on the relationship between the holdout data and the discovery data. Under the null, if the holdout data are independent of the selection process, the distribution of t_H does not depend on the selection: using the discovery sign as the direction, the sign-aligned statistic $\operatorname{sign}(t_D^*)\, t_H$ has mean zero. A genuinely positive IC in the final fixed evaluation direction gives a positive holdout effect in that direction. The discovery sign can be wrong; a positive raw IC alone does not guarantee a positive conditional mean under every selected sign alignment. This is why a holdout can distinguish "noise found by searching" from a "real but overestimated effect." The zero-mean argument requires independence between holdout data and the selection process; an embargo prevents label overlap but does not automatically guarantee independence in financial time series. Also avoid repeatedly reading the holdout and continuing to select specifications.

6.7 From t to the Sharpe Ratio

If monthly long-short returns x_t are approximately independent and identically distributed, the monthly Sharpe is $\widehat{SR}_m = \bar{x}/s$, so $t = \bar{x}/(s/\sqrt{T}) = \widehat{SR}_m\sqrt{T}$. For example, with $\widehat{SR}_m = 0.15$ and $T = 120$, $t = 0.15 \times \sqrt{120} = 1.643$, with a two-sided p-value of 0.100. An annualized Sharpe of $\widehat{SR}_m\sqrt{12} = 0.52$ looks good, but with 120 months of data it is not reliably distinguishable from zero. This conversion depends on independence; use Newey–West when the series is autocorrelated.

7. Financial Interpretation

A hypothesis needs an economic mechanism first. A hypothesis suitable for a plan should fit at least one of three mechanisms: risk compensation (the feature represents risk exposure for which investors demand a higher return), behavioral underreaction to information (prices adjust slowly to public information), or institutional or financing constraints (some investors are forced to buy or sell). Chapters 24–27 give the mechanisms and counterarguments for Value, Momentum, Quality, and Carry. A mechanism does not prove that a signal works; it limits which variants you can legitimately try. If the mechanism says "winners over the past year continue to win," you should not change it to "winners over the past seven weeks" because the data look better.

The threshold changes with the number of attempts. HLZ's (2014) recommendation of $t > 3.0$ follows from a multiple-testing framework; it is not a natural constant. They also acknowledge that a factor derived from first principles may warrant a lower threshold than a purely empirical discovery, but 2.0 is no longer suitable for either type. In this chapter's experiment, the two-sided Bonferroni threshold for 32 variants is $z = 3.16$. This issue is not new: when reporting 32 momentum strategies, Jegadeesh and Titman (1993, printed p. 69) noted that a single $t = 4.28$ has a Bonferroni probability below 0.0006 across 32 tests; they placed the number of tests beside the result. If there is truly one prespecified, well-calibrated two-sided normal test, the 5% threshold is about 1.96. So "report how many times you tried" is more fundamental than "choose a threshold."

A holdout sample has limited meaning. A holdout separates "selection" from "evaluation," but it is not a new world: it comes from the same market, the same data provider, and usually the same proxy. A signal that passes on the holdout may still have very low capacity, negative returns after costs, or fail in the next regime. A method controls FWER or FDR when the marginal p-values are valid and the specified selection and dependence conditions hold; an individual study can still report a false positive. Error-rate control also does not prove that returns are positive after costs.

Negative results carry information. If you write in advance that "holdout $t \geq 2.0$ counts as support" and the result is 1.70, the rule says "not supported." A positive point estimate close to the threshold is a reason to continue research, but it does not let you change the threshold to 1.5 after the fact. Changing it opens another forking path.

8. Numerical Example: Work Through Every Step

Every number in this section can be checked by hand; the code in Section 9 checks it again with assertions.

8.1 A Five-Trial Ledger and Three Adjustments

Complete Worked Example: Three Adjustments for Five Tests

Suppose the ledger contains five variants with discovery-sample t-statistics of 2.4, 1.1, 0.4, 1.9, and 2.1, respectively. Treat them as two-sided standard normal tests: $p = 2[1 - \Phi(|t|)]$.

Trial	t	Two-sided p
a	2.4	0.0164
b	1.1	0.2713
c	0.4	0.6892
d	1.9	0.0574
e	2.1	0.0357

Looking only at trial a, $p = 0.0164 < 0.05$ appears significant. But it is the best of five attempts. With $M = 5$:

- Bonferroni: $p^{\text{Bonf}} = \min(5p, 1)$: a is 0.082; e is 0.179; d is 0.287; b and c are 1. None is below 0.05.
- Holm: sort $p_{(1)} = 0.0164$ (a), $p_{(2)} = 0.0357$ (e), $p_{(3)} = 0.0574$ (d), $p_{(4)} = 0.2713$ (b), and $p_{(5)} = 0.6892$ (c). The multipliers $5, 4, 3, 2, 1$ give 0.082, 0.143, 0.172, 0.543, 0.689. They are already nondecreasing, so these are the adjusted p-values (a 0.082, e 0.143, d 0.172, b 0.543, c 0.689). None is below 0.05.
- BH: calculate $\frac{M}{\text{rank}}p$: $0.0164 \times 5/1 = 0.082$, $0.0357 \times 5/2 = 0.0893$, $0.0574 \times 5/3 = 0.0957$, $0.2713 \times 5/4 = 0.339$, and $0.6892 \times 5/5 = 0.689$. Taking the cumulative minimum from the end leaves the same values; the smallest q-value is 0.082, which does not pass the 5% FDR threshold.

Conclusion: $t = 2.4$ looks like a discovery on its own, but it does not once the five attempts are counted. This conclusion depends only on the number of ledger rows, regardless of your subjective intention to "look only at a." If you froze a plan to look only at a and evaluated only a as specified, then $M = 1$ and $p = 0.0164$ is valid. That is the value of a frozen plan.

8.2 Another Example with Four p-Values

Complete Worked Example: Adjusting Four p-Values

$p = (0.01, 0.04, 0.03, 0.005)$, $\alpha = 5\%$, and $M = 4$. Bonferroni gives (0.04, 0.16, 0.12, 0.02); two pass ($p = 0.005$ and 0.01). For Holm, sort $0.005, 0.01, 0.03, 0.04$ and multiply by $4, 3, 2, 1$: 0.02, 0.03, 0.06, 0.04. The cumulative maximum is 0.02, 0.03, 0.06, 0.06, which maps back to the input order as (0.03, 0.06, 0.06, 0.02); again, two pass. For BH, $0.005 \times 4/1 = 0.02$, $0.01 \times 4/2 = 0.02$, $0.03 \times 4/3 = 0.04$, and $0.04 \times 4/4 = 0.04$. Taking the cumulative minimum from the end leaves these values, which map back to (0.02, 0.04, 0.04, 0.02); all four pass at 5%. The strictness ordering of the three methods for adjusted p-values is Bonferroni $\geq$ Holm $\geq$ BH. This ordering also holds in the chapter's unit test for 50 random p-values.

8.3 Hash of the Frozen Plan

The code in Section 9 writes a value-hypothesis plan as a `ResearchPlan`. With the local clock fixed to 2026-10-06 09:00 UTC, it produces a SHA-256 whose first 16 characters are `40d38a74d67e0c08`. If only the criterion changes from "holdout $t \geq 2.0$ and positive sign" to "holdout $t \geq 1.5$ and positive sign," the hash becomes `e8ba36254e34603b`$\cdots$, completely different. If a colleague has already retained a trusted copy of the original digest, a digest comparison will identify modified content, under the collision-resistance assumption. Keeping a file and digest that can both be replaced does not establish a tamper-evident history. A hash also cannot prove that the data had not been seen before the plan was written.

8.4 Family-Wise Error and the Bonferroni Threshold for 32 Variants

For 32 independent tests at a 5% level, $1 - 0.95^{32} = 0.8063$. The Bonferroni single-test level is $0.05/32 = 0.0015625$, corresponding to $z = \Phi^{-1}(1 - 0.05/64) = 3.163$ for a two-sided test. Section 6.5 concerns the one-sided $\max Z$, whose exact expected value is about 2.070 and whose approximate value is 2.100. Here we have a two-sided $\max |Z|$; the expected value for 32 independent variables is about 2.347 (by numerical integration). These are not interchangeable. Thus the 1.96 threshold is exceeded often, while 3.16 is exceeded only rarely.

9. Python Implementation

Reusable code for this chapter is in `research_protocol.py`; cross-sectional and IC tools are in `factor_research.py` (covered in Chapter 23); and monthly momentum windows are in `factor_signals.py`. Interfaces:

Function/class	Input	Output and limits
`ResearchPlan`	Hypothesis, stock universe, data sources, four window endpoints, signal/label/portfolio definitions, benchmark, primary metric, criteria, cost assumptions, trial budget	Frozen data class; raises an error if any required field is empty, the budget is not a positive integer, or the discovery and holdout windows overlap
`freeze_plan`	Plan and optional local clock	`FrozenPlan`, with SHA-256 of normalized JSON and a local time string; not an external timestamp
`TrialLedger.record`	Specification dictionary, metric, sample size, status, notes	Appends a row; metric must be a finite real number or NaN; rejects infinity, Boolean values, and unknown statuses; `n_trials` counts every row
`bonferroni_adjust`, `holm_adjust`, `benjamini_hochberg_adjust`	One-dimensional p-value array	Returns adjusted p-values in input order; rejects empty arrays, out-of-range values, and NaN
`expected_max_standard_normal`, `..._exact`	Integer N	DSR approximation ($N \geq 2$) and numerical integration

Function/class	Input	Output and limits
`split_discovery_holdout`	Monthly `PeriodIndex` and plan	Two disjoint sets of month labels; raises an error when windows and sample have no overlap

The first code sample freezes a value-hypothesis plan and demonstrates hashing:

```
import dataclasses
from datetime import datetime, timezone

from quant_course.research_protocol import ResearchPlan, freeze_plan

plan = ResearchPlan(
    hypothesis="Cheap stocks (high B/M) earn higher next-month returns (sign +)",
    universe="constructed 200 names x 180 months",
    data_sources=("constructed panel, seed 20261006",),
    discovery_start="2000-01", discovery_end="2009-12",
    holdout_start="2010-01", holdout_end="2014-12",
    signal_definition="percentile rank of B/M within month, no lag",
    label_definition="next-month simple return",
    portfolio_rule="top quintile minus bottom quintile, equal weight",
    benchmark="equal-weight universe",
    primary_metric="t of mean IC: Pearson correlation of within-month signal ranks
    ↪ with next-month return, Newey-West lag 0, holdout",
    decision_rule="holdout t >= 2.0 and positive sign",
    cost_assumption="10 bp per unit traded", trial_budget=1,
)
clock = datetime(2026, 10, 6, 9, 0, tzinfo=timezone.utc)
frozen = freeze_plan(plan, now=clock)
edited = freeze_plan(dataclasses.replace(plan, decision_rule="holdout t >= 1.5 and
    ↪ positive sign"), now=clock)
assert frozen.sha256.startswith("40d38a74d67e0c08")
assert edited.sha256.startswith("e8ba36254e34603b") and edited.sha256 != frozen.
    ↪ sha256
print(frozen.sha256[:16], edited.sha256[:16])
```

The second code sample repeats the five trials in Section 8.1: it counts the ledger, applies three adjustments, and checks them against the hand calculations:

```
import numpy as np
from scipy import stats

from quant_course.research_protocol import (
    TrialLedger, benjamini_hochberg_adjust, bonferroni_adjust, holm_adjust,
    ↪ pad_untested_attempts,
)

t = np.array([2.4, 1.1, 0.4, 1.9, 2.1])
ledger = TrialLedger()
for name, value in zip("abcde", t):
    ledger.record({"variant": name}, metric=float(value), n_obs=120, status="primary"
    ↪  if name == "a" else "run")
ledger.record({"variant": "f"}, metric=float("nan"), n_obs=0, status="abandoned",
    ↪ note="constant signal: IC undefined")
```

```
assert ledger.n_trials == 6          # the undefined attempt still counts as a look
    ↪ at the data

p = 2 * stats.norm.sf(t)
np.testing.assert_allclose(np.round(p, 4), [0.0164, 0.2713, 0.6892, 0.0574, 0.0357])
np.testing.assert_allclose(np.round(bonferroni_adjust(p), 3), [0.082, 1.0, 1.0,
    ↪ 0.287, 0.179], atol=1e-3)
np.testing.assert_allclose(np.round(holm_adjust(p), 3), [0.082, 0.543, 0.689, 0.172,
    ↪ 0.143], atol=1e-3)
np.testing.assert_allclose(np.round(benjamini_hochberg_adjust(p), 3), [0.082, 0.339,
    ↪ 0.689, 0.096, 0.089], atol=1e-3)
full = pad_untested_attempts(p, ledger.n_trials)            # the undefined attempt
    ↪ enters the family as p = 1: family size 6
np.testing.assert_allclose(np.round(bonferroni_adjust(full)[:5], 3), [0.098, 1.0,
    ↪ 1.0, 0.345, 0.214], atol=1e-3)
np.testing.assert_allclose(np.round(holm_adjust(full)[:5], 3), [0.098, 0.814, 1.0,
    ↪ 0.230, 0.179], atol=1e-3)
np.testing.assert_allclose(np.round(benjamini_hochberg_adjust(full)[:5], 3), [0.098,
    ↪ 0.407, 0.827, 0.115, 0.107], atol=1e-3)
print(ledger.to_frame()[["trial_id", "status", "metric"]])
```

Note that `ledger.n_trials` is 6, not 5: the sixth result is undefined, but you read the data to obtain it. **The primary result should use the full-ledger family size** $M = 6$: use `pad_untested_attempts` to add attempts with no p-value to the family as $p = 1$ (they can never be rejected, but they count in the multiplier). This gives Bonferroni $(0.098, 1, 1, 0.345, 0.214)$, Holm $(0.098, 0.814, 1, 0.230, 0.179)$, and BH $(0.098, 0.407, 0.827, 0.115, 0.107)$. The three assertions above adjust only the five valid p-values ($M = 5$); that is a different convention and may be included only as a sensitivity note, not as a substitute for the full-ledger result. If you report both conventions, explain the difference. The hand calculation in Section 8.1 uses only the first five for teaching simplicity.

The third code sample checks the accuracy of the expected-maximum approximation (the table in Section 6.5):

```
import numpy as np

from quant_course.research_protocol import expected_max_standard_normal,
    ↪ expected_max_standard_normal_exact

exact_two = expected_max_standard_normal_exact(2)
assert abs(exact_two - 1 / np.sqrt(np.pi)) < 1e-9                # hand value 1/sqrt
    ↪ (pi) = 0.5642
for n, tol in ((10, 0.04), (100, 0.03), (1000, 0.02)):
    approx, exact = expected_max_standard_normal(n),
    ↪ expected_max_standard_normal_exact(n)
    assert abs(approx - exact) < tol
print(round(expected_max_standard_normal(32), 3), round(
    ↪ expected_max_standard_normal_exact(32), 3))   # 2.1 2.07
```

Default offline entrypoint and historical experiment are separate. From the root of the accompanying I3 reader package, run:

```
.venv/bin/python run_notebook.py notebooks/ch22_research_methodology.ipynb
```

The default is `RUN_FRENCH=False`. The notebook demonstrates freezing and hashing, a six-row ledger and the two family-size conventions for five valid p-values, the expected maximum, one single 32-variant constructed world, and simulations with 60 null worlds and 60 alternative worlds (not the 500 each used for the historical charts in Sections 10 and 13). The notebook creates only a chapter-level `RUN_DIR`; it does not write CSV/PNG files or generate the historical chart library. The runner separately creates `runs/execution_ch22_research_methodology_<uuid>/` to save an execution copy and status; do not conflate this with the chapter-level directory. The default demonstration with five valid p-values does not imply that there is a sixth valid p-value; interpret the family adjustment for the failed attempt using the earlier convention of padding with 1.

The full historical CLI is `code/ch22_research_methodology.py`. It includes the French branch and 500 simulations per world; it is not the default offline command. Its historical seeds are fixed at 20261006 (null world) and 20261007 (alternative world), and the local freeze plan is written before the holdout numbers are calculated; the local clock provides no external evidence of ordering. The historical numbers and charts below come from the original experiment; readers do not need to rerun it.

NaN policy: the ledger accepts NaN metrics and counts them; `ic_summary` drops NaN dates and records them in `n_missing`; after dropping, the Newey–West lags are computed by position in the valid array, so internal date gaps require separate checks. Summary calculation raises an error if fewer than three valid dates remain. The data frequency is monthly; `signal_time` is month-end, the label is the following month's simple return, and the return measurement interval is that following month. French industry monthly returns come from a revised historical snapshot, not a point-in-time vintage, so `available_at` is in practice later than month-end. Treating month-end as the signal time is an idealized assumption in this chapter.

10. Reading the Figures

Figure 22-1: The ten-stage research pipeline. Two rows of five boxes, read left to right and then top to bottom, with each box showing a stage name and its audit trail. There is no arrow from Stage 10 back to Stage 1: holdout results must not rewrite the hypothesis they were intended to test. This is a process diagram with no data.

Figure 22-2: How search creates significance in a constructed world, and how a holdout distinguishes two worlds. Each of two worlds has 500 independent simulations, each with 200 stocks, a 120-month discovery sample, a 3-month embargo, and a 60-month holdout; the null-world seed is 20261006 and the alternative-world seed is 20261007 (the original chart's shared title lists only 20261006, which must not be used to conflate the seeds). The left panel shows the null world (true IC is zero): blue is $|t|$ in the discovery sample for the single prespecified variant; orange is the maximum $|t|$ among 32 variants. The dashed line is 1.96, and the purple dotted line is the Bonferroni threshold of 3.16 for 32 tests. Interpretation: orange shifts to the right overall;

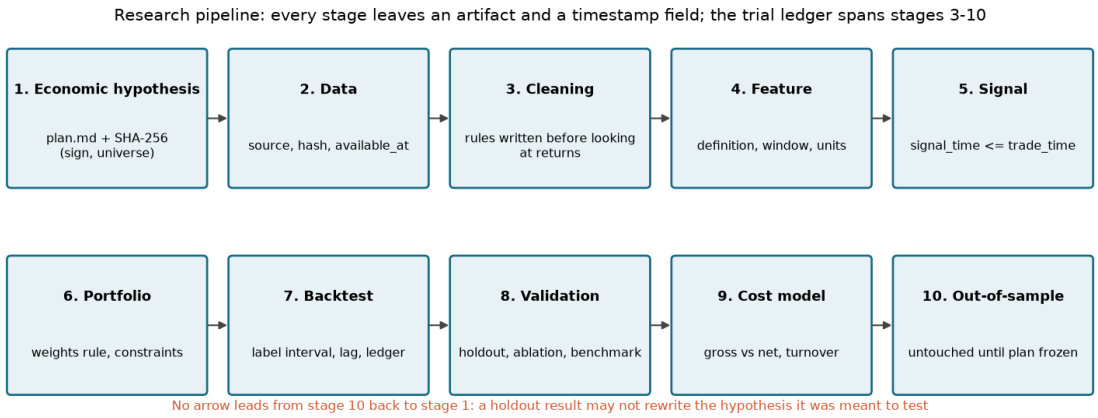


Figure 22-1: Ten stages of the research pipeline and their audit trails

23.6% exceed 1.96, compared with 5.4% for blue. The right panel shows the holdout t-statistic of the best variant chosen in the discovery sample, aligned to its discovery sign: orange (null world) is centered around zero, while blue (true IC about 0.02) lies mostly to the right of zero. These are constructed data to illustrate a mechanism, not market evidence.

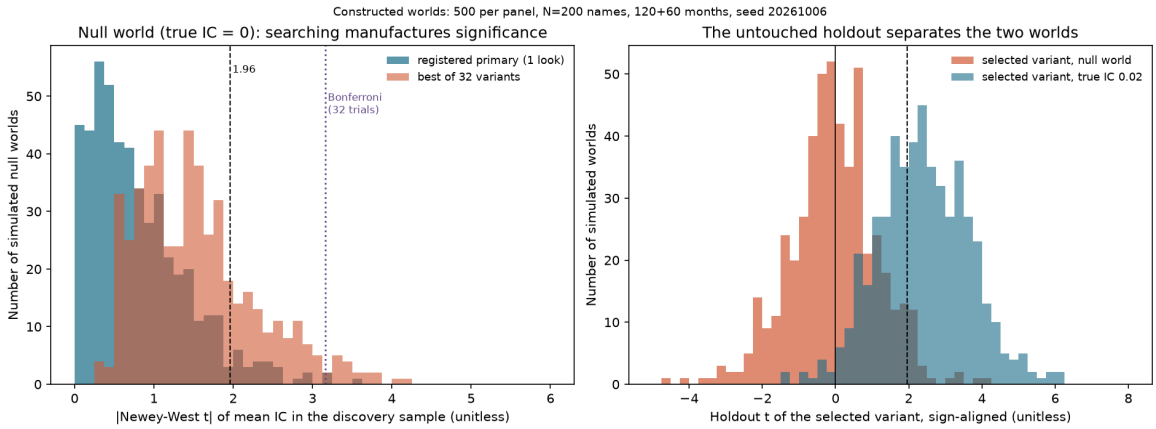


Figure 22-2: Selecting the best of 32 variants—false significance in the null world and the role of holdout evaluation

Figure 22-3: Twenty-four lookback-window variants for French industry momentum. In the left panel each point is a (lookback months, skip months) pair. The x-axis is the discovery-sample t from 1926-07 through 1999-12; the y-axis is the holdout-sample t from 2000-01 through 2026-07. The black circle marks the prespecified primary specification, (11, 1). In the right panel, variants are sorted by discovery t from highest to lowest, and the orange line shows the holdout t for the same variant. Interpretation: 22 of the 24 variants have $t > 2$ in the discovery sample; none exceeds 2 in the holdout; the top-ranked variant, (1, 0), has a holdout t of only 0.26. The sample consists of revised monthly returns for 10 industry portfolios, not executable trades.

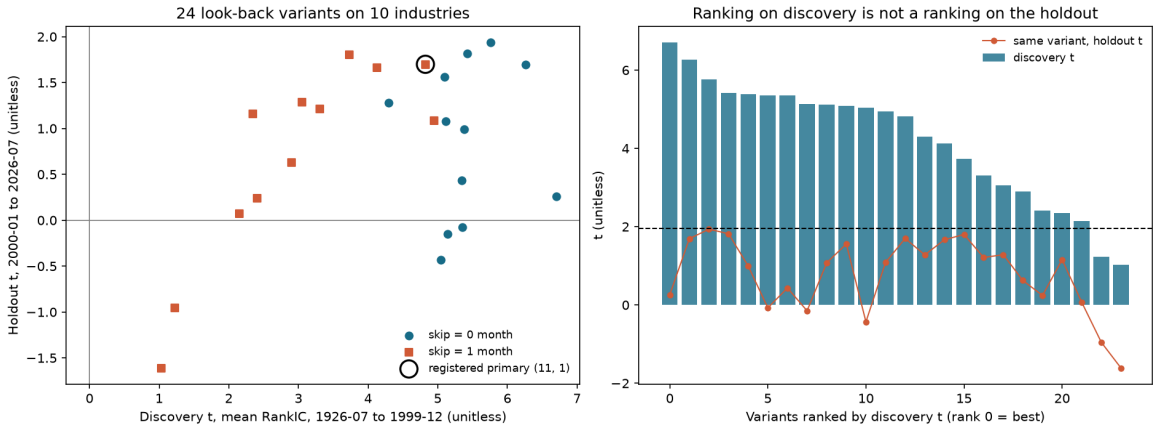


Figure 22-3: Discovery and holdout t-statistics for 24 lookback-window variants

11. Historical Market Example

Data. Use the French 10-industry value-weighted monthly returns frozen on 2026-10-05: source Ken French Data Library, 10 Industry Portfolios, ZIP SHA-256 `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`, processed CSV SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`, 1,202 months (1926-07 to 2026-08), in decimal monthly simple returns. Publicly downloadable does not mean freely redistributable. This chapter does not download new data.

Plan (frozen). Hypothesis: industries with higher past 12-1 returns have higher returns next month (positive sign). Signal: at month-end s, compound returns over the 11 months from $s-11$ through $s-1$, excluding the current month; label: simple return in the next month, $s+1$; cross-section: 10 industries; primary metric: the t-statistic of the mean monthly Spearman rank correlation (Rank IC) in the holdout sample (Newey–West lag 0); criterion: holdout $t \geq 2.0$ and positive; cost assumption: 10 bp per unit of turnover, used only for the long-short portfolio demonstration; discovery sample through 1999-12, holdout sample from 2000-01 through 2026-07; trial budget of 24 variants (lookback 1–12 months, skip 0 or 1 month). The notes field says that this plan was written after reading the relevant literature and using the same data in earlier chapters; it is not a blind plan and was not externally registered.

Results. All numbers were produced by `code/ch22_research_methodology.py` and independently recomputed by `qa/a5/claude/recompute/ch22_recompute.py` (without importing course code, using a monthly loop and `scipy.stats.spearmanr`); the two agree.

Variant (lookback, skip)	Discovery months	Discovery mean Rank IC	Discovery t	Holdout months
Registered primary specification (11, 1)	871	0.0688	4.82	319
Discovery-sample winner (1, 0)	882	0.0925	6.70	319
(12, 0)	871	0.0877	6.26	319
(6, 1)	876	0.0433	3.05	319

Variant (lookback, skip)	Holdout mean Rank IC	Holdout t
Registered primary specification (11, 1)	0.0414	1.70
Discovery-sample winner (1, 0)	0.0061	0.26
(12, 0)	0.0412	1.69
(6, 1)	0.0304	1.28

Of the 24 variants, 22 have discovery-sample $t > 2$ and none has holdout $t > 2$. The ledger has 24 rows (the registered primary specification is the (11,1) row).

Apply the prespecified criterion. The primary specification's holdout $t = 1.70 < 2.0$, so the conclusion is "not supported." The point estimate is positive (mean Rank IC 0.041); the difference between the discovery estimate (0.0688, standard error about 0.0143) and holdout estimate (0.0414, standard error about 0.0243) is about 0.027, with a standard error of about 0.028. The difference itself is not significant: the data neither prove that momentum "disappeared" after 2000 nor prove "no decay." A permutation benchmark (randomly shuffling labels among the 10 industries each month 2,000 times while preserving each month's label set) gives a one-sided p-value of 0.050, close to $1 - \Phi(1.70) = 0.045$, and adds no new information. These conclusions apply only to revised monthly returns for these 10 industry portfolios.

Multiple-testing adjustment. For the two-sided p-values of the 24 discovery-sample variants, the winner's raw p-value is about 2×10^{-11}, with a Bonferroni-adjusted value of about 5×10^{-10}; Holm and BH are the same. It remains highly significant after adjustment. This shows that, under the approximate p-values and family definition used, the discovery-window tests still reject the null after adjustment; it does not identify why the holdout estimate is smaller. Multiple-testing adjustments control error rates for rejection decisions; they do not debias the selected effect size and cannot rule out the winner's curse. Changes in the effect, selection bias, and sampling variation may all contribute; the difference above, 0.027 with standard error 0.028, does not establish decay. The lower holdout statistic also reflects estimation precision, so it cannot identify a cause.

Switch to a long-short portfolio and costs (demonstration). Each month, sort the 10 industries into five groups of two, go long the top group and short the bottom group, equal-weight each leg, for gross exposure of 2 and net exposure of 0. Turnover is the sum of absolute changes in target weights between adjacent months (buys and sells combined, not halved, ignoring drift). For the primary specification (11,1), the discovery-sample gross mean is 0.561% per month ($t = 3.69$), mean turnover is 0.861, and net return at 10 bp per unit is 0.475%; the holdout gross mean is 0.403% ($t = 1.21$), turnover is 1.019, and net return is 0.301%. For the discovery-sample winner (1,0), the discovery gross mean is 0.861% ($t = 6.33$), the holdout gross mean is 0.405% ($t = 1.25$), turnover is 3.056, and net return is 0.099% per month: turnover is three times higher and costs consume about three-quarters of the gross return. The 10 bp is an assumption, and industry portfolios are not executable trade series; the net returns here illustrate only the direction and relative size of gross returns and costs.

What this example does and does not show. It demonstrates that the registered primary specification, the discovery-sample winner, and the holdout sample can lead to very different conclusions; "not supported under the criterion" is not the same as "the estimate is significantly zero." It does not prove that industry momentum does not exist, validate any conclusion in Chapter 25, or provide an independent test of 12-1 momentum: the holdout is not pristine, the author has read the momentum literature and used these data in earlier chapters, the cross-section has only 10 assets, and the data are revised historical values.

12. Common Errors

Error 1: Report only the best of N variants. In a null world, the probability that the maximum $|z|$ among 20 independent variants exceeds 1.96 is $1 - 0.95^{20} = 0.642$.

```
import numpy as np

rng = np.random.default_rng(7)
z = rng.standard_normal((20000, 20))                    # 20 independent null z-
    ↪ statistics per "paper"
reported_best = (np.abs(z).max(axis=1) > 1.96).mean()   # report only the best of
    ↪ 20
single = (np.abs(z[:, 0]) > 1.96).mean()                # report the one pre-
    ↪ committed test
bonferroni = (np.abs(z).max(axis=1) > 3.0).mean()       # hurdle for 20 looks: z =
    ↪  3.02 two-sided at 5%
assert abs(reported_best - (1 - 0.95**20)) < 0.015 and abs(single - 0.05) < 0.01 and
    ↪ bonferroni < 0.06
print(round(reported_best, 3), round(single, 3), round(bonferroni, 3))
```

Fix: report all variants or register one variant and state the budget in the plan. If search is unavoidable, use a Bonferroni or Holm threshold.

Error 2: Change the criterion after seeing results. The hash in Section 8.3 shows that changing one character in the criterion changes the entire hash. Fix: put the criterion in the plan and hash it; if a change is necessary, create a new version and record in the ledger and log that it was "changed after seeing the holdout result."

Error 3: Repeatedly select using the holdout. If you evaluate 10 independent variants on the holdout and choose the best, the null-world probability of at least one "significant" result is $1 - 0.95^{10} = 0.401$; the holdout has become a second discovery sample. Fix: evaluate only the single specification (or a small, explicitly counted number) selected in the discovery sample. Figure 22-3 calculates holdout t-statistics for all 24 variants to illustrate the chart; this is for teaching only. In real research it would amount to 24 holdout evaluations. The table in Section 11 still applies the criterion only to the registered primary specification and states this limitation explicitly.

Error 4: Use future data in standardization or winsorization parameters. Full-sample standardization of a time series makes early signals depend on later data.

```
import numpy as np

x = np.array([0.5, -0.2, 0.1, 0.4, -0.1, 3.0])        # pretend x[5] arrives later
full = (x - x.mean()) / x.std(ddof=0)                  # full-sample: uses the future
causal = np.array([(x[: i + 1] - x[: i + 1].mean())[-1] / (x[: i + 1].std(ddof=0) or
    ↪ 1.0) for i in range(len(x))])
x2 = x.copy(); x2[5] = -3.0                            # the future value changes ...
full2 = (x2 - x2.mean()) / x2.std(ddof=0)
causal2 = np.array([(x2[: i + 1] - x2[: i + 1].mean())[-1] / (x2[: i + 1].std(ddof=0)
    ↪  or 1.0) for i in range(len(x2))])
assert not np.allclose(full[:5], full2[:5])            # ... and rewrites earlier "
    ↪ signals"
assert np.allclose(causal[:5], causal2[:5])            # expanding-window version
    ↪ does not
print(np.round(full[0], 3), np.round(full2[0], 3))
```

Cross-sectional standardization across all stocks on the same day is not this error, because all the same-date data are available before `signal_time`; Chapter 28 distinguishes the two cases.

Error 5: Count only "successful" trials. The ledger must include abandoned and undefined attempts (`n_trials` is 6 in the second code sample in Section 9). Counting only successes is equivalent to reducing M to the most favorable number.

Error 6: Naive t-statistics with overlapping labels. Three-month labels make the IC series correlated across adjacent months. In the null world, the standard deviation of t for the three-month-label variant is 1.26 (1.04 for the one-month-label variant). Even with Newey–West lag 2, the rejection rate for $|t| > 1.96$ on three-month labels is 11.8%, above the nominal 5%. In a finite sample, Newey–West underestimates variance. Fix: use a longer lag, nonoverlapping sampling, or calibrate the threshold separately for each h; Chapter 23 explains this in detail.

Error 7: Selection favors the statistic with the most biased standard error. In the same null world, the selected "best variant" has a 4.6% rate of holdout $t > 1.96$ (aligned to the discovery sign), not 2.5%. Of the selected variants, 61.8% use three-month labels even though they make up only half of the variants. Their null t-statistics have higher variance, making them more likely to become the maximum; their holdout statistics also have higher variance. Selection favors the class with underestimated standard errors.

13. Quant Research Application: Compare an A Priori Fixed Plan with Post Hoc Mining for the Same Value Hypothesis

This section compares the value-hypothesis plan from Section 8.3 with a "mining version" on the same constructed data. The data are for teaching: 200 stocks, a 120-month discovery sample, a 3-month embargo, and a 60-month holdout. The feature is heavy-tailed monthly data (Student-t with 4 degrees of freedom, variance-normalized) with autocorrelation 0.9, representing book-to-market ranks; next-period return equals slope× feature plus heavy-tailed noise (monthly standard deviation 0.08). The slope is 0 in the null world and 0.0016 in the alternative world, corresponding to population IC of about 0.020. The two worlds are otherwise identical. Each world is simulated 500 times.

Path A (a priori fixed specification): evaluate only variant 0 in the frozen plan (rank transform, lag 0, one-month label), and decide under the criterion. In the simulation, each month's statistic is the "Pearson correlation between the rank-transformed signal and raw returns" (for vectorization, returns are not ranked), so it differs from Chapter 23's Rank IC, which ranks both sides. Other variants use "the Pearson correlation between the transformed signal and raw returns," consistent with the plan's primary-metric wording. The real-data analysis in Section 11 uses Spearman Rank IC, ranking both sides. Path B (post hoc mining): evaluate 32 variants (rank/raw value/1% winsorization/5% winsorization × signal lags of 0–3 months × label horizons of 1 or 3 months), select the one with the largest discovery-sample $|t|$, and evaluate it on the holdout.

Metric (500 simulations)	Null world: Path A	Null world: Path B	Alternative world: Path A	Alternative world: Path B
Share with discovery-sample $\|t\| > 1.96$	5.4%	23.6%	82.6%	93.6%
Share with discovery-sample $\|t\| > 3.16$ (Bonferroni threshold for 32 trials)	—	3.6%	—	65.8%
Share with holdout $t > 1.96$ (Path B aligned to the discovery sign)	2.8%	4.6%	57.6%	61.8%
Share of cases in which Path B selects the registered variant 0	—	5.8%	—	5.0%

Interpretation. In the null world, Path B raises the false-positive rate from 5.4% to 23.6%, not to $1 - 0.95^{32} = 81\%$: the variants are correlated. Matching the mean maximum discovery-sample $|t|$ of 1.53 implies about 4.6 effective independent trials, not 32. Using the Bonferroni threshold of 3.16 for the actual 32 attempts, the rejection rate in these 500 null-world simulations is 3.6%. This

is not a general FWER guarantee: the union bound requires every marginal p-value to be valid, and the Newey–West approximate test may be miscalibrated here because the labels overlap. The 4.6 obtained by matching the mean maximum absolute statistic matches one moment and does not identify a tail threshold; Chapter 23 explains the Newey–West truncation and kernel-weight issue. In the alternative world, the power of Path A is 82.6%, while the holdout confirmation rate after selection on Path B is not much higher than A (61.8% versus 57.6%): search brings no reliable extra benefit, only more opportunities for false reporting. The holdout has only 60 months, so even when the signal is real, only about six in ten pass in the alternative world. This is a power issue, not a failure of the method.

Implication for Project 4: before a multifactor project starts, write a frozen plan containing at least the twelve items in Section 5, and specify the ledger format, embargo rule, and one-time holdout evaluation. Without this plan, no "significant" result from Project 4 can go into the conclusion. Chapter 28 will make preprocessing, combination, and constraints separate steps in the plan. The same applies to the pair screening in Project 5: the ledger must record how many cointegration tests were run during training (see the preparation layer at `projects/project05/preparation/`).

Benchmark and falsification condition: use equal-weight holdings (no signal) and a permutation benchmark; write before running that "the hypothesis is not supported if holdout $t < 2.0$ or the sign is negative."

14. Chapter Summary

This chapter divides a factor study into ten stages, each leaving an inspectable record. Four relationships matter. First, a frozen plan constrains choices that can be fixed before seeing results, while the ledger records every attempt whose result was read. Second, independent and well-calibrated tests give $1 - (1 - \alpha)^M$. With valid marginal p-values, Bonferroni and Holm control FWER without independence, while BH additionally requires independence or suitable positive dependence. In the particular constructed example, the mean maximum absolute statistic among 32 variants matches about 4.6 independent trials. This describes one moment; it cannot replace the actual family size or guarantee a tail threshold. Third, a holdout separates selection and evaluation. Avoiding repeated access and imposing an embargo reduce label leakage but do not automatically make financial data independent of the selection process; inference still requires checking dependence and marginal p-value calibration. Fourth, retain results that fail the prespecified criterion: the industry primary specification has holdout $t = 1.70$, below the 2.0 threshold. None of these steps automatically establishes profitability after costs.

Limits: a local hash is not preregistration; the French industry data are revised historical data and have been used in earlier chapters; a cross-section of 10 assets has low testing power; costs are assumed values; and the chapter is a methodological demonstration, not an empirical verdict on any factor.

15. Exercises

22-E01 (Conceptual) What can a local freeze and an external preregistration each prove, and what can neither prove? If a colleague asks, "Does your hash timestamp predate the data?", how should you answer?

22-E02 (Conceptual) Give an example of financial research with one regression and a hypothesis held in advance that still has forking paths. Identify the decision points.

22-E03 (By hand) For 20 independent tests at level $\alpha = 1\%$, what is the family-wise error rate? What single-test level gives exactly 1% FWER (Šidák)? How far is it from Bonferroni?

22-E04 (By hand) Let $p = (0.002, 0.011, 0.020, 0.040, 0.300)$, $M = 5$, and $\alpha = 5\%$. Calculate the Bonferroni, Holm, and BH adjusted p-values, and list the tests that pass under each method.

22-E05 (Derivation) Use $\max(Z_1, Z_2) = \frac{Z_1+Z_2}{2} + \frac{|Z_1-Z_2|}{2}$ to derive $\mathbb{E}[\max(Z_1, Z_2)] = 1/\sqrt{\pi}$. Explain why the DSR approximation is inaccurate at $N = 2$.

22-E06 (Interpretation) In the null world, 32 variants have a mean maximum $|t|$ of 1.53, corresponding to about 4.6 effective independent trials. Explain what "effective number of independent trials" means. Compare Bonferroni using the actual $M = 32$ with replacing that count by $M = 4.6$: what conditions, power costs, or tail-error risks apply to each? Do both approaches necessarily contain an error in principle?

22-E07 (By hand) On a holdout sample, you evaluate 10 mutually independent null variants and report the best one. What is the probability that at least one has $|z| > 1.96$? What does this imply for the rule "evaluate only once on the holdout"?

22-E08 (By hand) A long-short strategy has $T = 60$ months and an estimated monthly Sharpe of 0.20. Calculate t, annualized Sharpe, and the two-sided p-value (normal approximation). If it is entered in a ledger with five variants, does it pass 5% after Bonferroni?

22-R01 (Ledger diagnosis, further practice) For the five-row ledger in Section 5.1, give the raw p-value vector used for the family, the Holm sorting multipliers, and the IDs rejected by each method at 5%. If you change the label horizon to the best value after H01, can that evaluation still be called an unexposed holdout? Could a file hash change the answer?

16. Programming Exercises

22-P01 Write your own `ResearchPlan` for any hypothesis, freeze it, and save it as JSON. Then change one field and verify that the hash differs; assert that overlapping discovery and holdout windows raise an error. Acceptance: all three assertions pass.

22-P02 Use a ledger to record eight variants, including one undefined result. Calculate two-sided p-values, use `pad_untested_attempts` to bring the full-ledger family size to $M = 8$, and output a table of the three adjusted p-values. Acceptance: `n_trials==8`, family size is 8, and Bonferroni $\geq$ Holm $\geq$ BH for every test.

22-P03 Simulate 32 null z-statistics with an equicorrelation of 0.8. Across 20,000 simulations, calculate the mean maximum $|z|$ and the share exceeding 1.96, then use numerical integration to infer the equivalent number of independent trials. Acceptance: mean near 1.65 and equivalent number between 5 and 7.

17. Challenge Problems

22-C01 (Proof) Prove FDR $\leq$ FWER and give a strict example with FDR $<$ FWER (specify the values of V and R). Additional prerequisite: expectation and conditional probability. Grading: proof 40 points, example 40 points, practical interpretation 20 points.

22-C02 (Open-ended) Write a complete frozen plan for 12-1 momentum in Chapter 25: fill in all twelve items, specify ledger fields and embargo rules, describe the one-time holdout process, and list at least three concrete numerical outcomes that would lead you to conclude that the hypothesis is "not supported." Grading: plan completeness 30 points, falsifiability of criteria 30 points, limitations for data repeatedly analyzed by others 20 points, cost and benchmark design 20 points.

18. Complete Solutions and Reference Implementations

22-E01 A local freeze binds normalized JSON to a digest. Under collision-resistance and related cryptographic assumptions, matching a trusted saved digest supports identity with the reference content. It does not prove that the file was never changed and later restored, or establish when it was written. A file and digest that can both be replaced do not establish their own trustworthy history. The local clock can be altered and cannot show that the plan preceded exposure. An external registry holds content or a digest with a third-party timestamp, providing evidence of registration chronology; prior data exposure must still be logged. Answer the colleague: "The current content can be compared with a trusted saved digest. The local time is not external evidence of chronology. These data were used before, so this is a process demonstration."

22-E02 Example: study whether returns over the past 12 months predict returns next month. You hold the hypothesis in advance and run only one regression, but before seeing the results you did not fix whether to remove low-priced stocks, use value weights, exclude financial stocks, or choose the sample start. If the effect appears mainly among small-cap stocks, you can find a reasonable rationale for "exclude microcaps"; if it appears among large-cap stocks, you can find a rationale for "use value weights." Regardless of how many regressions you actually ran, these data-dependent choices are forking paths.

22-E03 FWER $= 1 - 0.99^{20} = 0.1821$. The Šidák single-test level is $1 - (0.99)^{1/20}$: $\ln 0.99 = -0.0100503$, divided by 20 gives -0.00050251, and exponentiating gives 0.99949762, so the complement is 0.00050238. Bonferroni is $0.01/20 = 0.0005$. The difference is 2.4×10^{-6}, or about 0.5% relatively: Šidák is slightly more permissive, and the two are nearly identical in this range.

22-E04 $M = 5$, with p-values already sorted in ascending order. Bonferroni $5p$ gives 0.010, 0.055, 0.100, 0.200, 1.000; only the first ($p = 0.002$) passes at 5%. Holm multipliers $5, 4, 3, 2, 1$ give 0.010, 0.044, 0.060, 0.080, 0.300, already monotone; the first and second ($p = 0.002$, 0.011) pass. BH gives $p \cdot 5/\text{rank}$: 0.010, 0.0275, 0.0333, 0.050, 0.300. Taking the cumulative minimum from the end leaves these values; the first four pass at ≤ 0.05 (the fourth is exactly 0.05 and counts as passing). In total, Bonferroni rejects 1, Holm 2, and BH 4; the more permissive error-rate definition allows more rejections.

22-E05 Both $Z_1 + Z_2$ and $Z_1 - Z_2$ are centered normal variables, and the expectation of the first term is 0. Let $W = Z_1 - Z_2 = \sqrt{2}\,G$, where $G \sim N(0,1)$. Then $\mathbb{E}|G| = \sqrt{2/\pi}$, so $\mathbb{E}|W| = \sqrt{2}\sqrt{2/\pi} = 2/\sqrt{\pi}$ and $\mathbb{E}[\max] = \frac{1}{2} \cdot 2/\sqrt{\pi} = 1/\sqrt{\pi} = 0.5642$. The approximation gives 0.520, an underestimate of 7.9%. The authors call it a large-N approximation. At $N = 2$, $\Phi^{-1}(1 - 1/N) = \Phi^{-1}(0.5) = 0$; the second term is $\Phi^{-1}(1 - 1/(2e)) = \Phi^{-1}(0.816) = 0.90$. Weighting it by 0.5772 gives 0.52. The asymptotic expansion from extreme-value theory has non-negligible error at small N. (Readers can check with `expected_max_standard_normal(2)`.)

22-E06 Here N_{eff} is defined by matching one moment: the expected maximum of N_{eff} independent $|N(0,1)|$ variables equals the observed mean of 1.53 in this constructed example. Numerical inversion gives about 4.6. This does not mean that the full maximum distributions are the same, and it gives no tail-probability or FWER guarantee. Bonferroni with the actual $M = 32$ uses a threshold of about 3.16 and controls FWER by the union bound when marginal p-values are valid; independence is not required, though power may be lower. The 3.6% rejection rate in this null world and 65.8% in the alternative world are only results from this simulation; the overlapping-label/Newey–West calibration limitations remain. Replacing 32 with 4.6 would relax the threshold to about 2.55, but matching a mean does not establish that this threshold controls 5% FWER. Nor can it provide such a guarantee when the dependence structure is unknown or marginal p-values are invalid. Keep the full-ledger adjustment, or rerun the entire selection process under a resampling design that preserves both the null and dependence structure and estimates the tail of the maximum distribution; the validity of the resampling design must itself be justified.

22-E07 $1 - 0.95^{10} = 0.4013$. Evaluating 10 variants on the holdout and selecting the best gives a Type I error rate of about 40%; the holdout has become a second discovery sample. The rule "evaluate only once on the holdout" means evaluating only the specification selected after freezing (or a small number that is explicitly counted).

22-E08 $t = 0.20\sqrt{60} = 1.549$; annualized Sharpe $0.20\sqrt{12} = 0.693$; two-sided $p = 2(1 - \Phi(1.549)) = 0.121$. In a five-variant ledger, Bonferroni gives $p = \min(5 \times 0.121, 1) = 0.606$, which does not pass. Here, $t = \widehat{SR}_m\sqrt{T}$ assumes monthly returns are approximately independent and identically distributed.

22-R01 The family's raw p-value vector is $(0.01, 0.04, 0.03, 1, 0.005)$. In ascending order the IDs are D05, D01, D03, D02, D04; Holm multipliers are $5, 4, 3, 2, 1$. The products are 0.025, $0.04, 0.09, 0.08, 1$, and the cumulative maxima are $0.025, 0.04, 0.09, 0.09, 1$. Mapping back to the original rows gives the values in Section 5.1. At $p_{\text{adj}} \leq 0.05$, Bonferroni and Holm reject D01 and D05; BH rejects D01, D02, D03, and D05 (including equality). The BH FDR guarantee additionally requires independence or an applicable positive-dependence condition. Changing the horizon after H01 uses holdout information, so the data can no longer be called unexposed; a hash binds file contents but cannot undo the history of what was read.

22-P01 Reference Implementation

```
import dataclasses
import json
from datetime import datetime, timezone

import pytest

from quant_course.research_protocol import ResearchPlan, freeze_plan

plan = ResearchPlan(
    hypothesis="Higher past 12-1 return predicts higher next-month return (sign +)",
    universe="10 French industries", data_sources=("french_10_industries_monthly.csv"
    ↪ ,),
    discovery_start="1926-07", discovery_end="1999-12", holdout_start="2000-01",
    ↪ holdout_end="2026-07",
    signal_definition="12-1 compounded return", label_definition="next-month simple
    ↪ return",
    portfolio_rule="top 2 minus bottom 2, equal weight", benchmark="equal-weight 10
    ↪ industries",
    primary_metric="t of mean RankIC in holdout", decision_rule="holdout t >= 2.0 and
    ↪  positive",
    cost_assumption="10 bp per unit traded", trial_budget=24,
)
clock = datetime(2026, 10, 6, 9, 0, tzinfo=timezone.utc)
first = freeze_plan(plan, now=clock)
second = freeze_plan(dataclasses.replace(plan, trial_budget=25), now=clock)
assert first.sha256 != second.sha256
saved = json.dumps({"sha256": first.sha256, "frozen_at_local": first.frozen_at_local
    ↪ }, indent=1)
assert json.loads(saved)["sha256"] == first.sha256
with pytest.raises(ValueError):
    dataclasses.replace(plan, holdout_start="1999-06")        # overlaps the
    ↪ discovery window
```

22-P02 Reference Implementation

```
import numpy as np
import pandas as pd
from scipy import stats

from quant_course.research_protocol import (
    TrialLedger, benjamini_hochberg_adjust, bonferroni_adjust, holm_adjust,
    ↪ pad_untested_attempts,
)

t_values = [2.8, 2.1, 1.7, 1.2, 0.9, 0.5, -0.4]
ledger = TrialLedger()
for i, value in enumerate(t_values):
    ledger.record({"variant": i}, metric=value, n_obs=120, status="primary" if i == 0
    ↪  else "run")
ledger.record({"variant": 7}, metric=float("nan"), n_obs=0, status="abandoned", note=
    ↪ "constant signal")
assert ledger.n_trials == 8
p = np.append(2 * stats.norm.sf(np.abs(np.array(t_values))), np.nan)          # the
    ↪ undefined attempt has no p-value
family = pad_untested_attempts(p[:-1], ledger.n_trials)                          #
    ↪ ... and enters the family as p = 1: M = 8
assert family.size == 8 and family[-1] == 1.0
table = pd.DataFrame({"t": t_values + [np.nan], "p": family, "bonf":
    ↪ bonferroni_adjust(family), "holm": holm_adjust(family),
                      "bh": benjamini_hochberg_adjust(family)})
assert (table["bonf"] >= table["holm"] - 1e-12).all() and (table["holm"] >= table["bh
    ↪ "] - 1e-12).all()
assert (table["bonf"].iloc[-1], table["holm"].iloc[-1], table["bh"].iloc[-1]) ==
    ↪ (1.0, 1.0, 1.0)
print(table.round(4))
```

(The seven t-values plus one undefined attempt make eight ledger rows. Add the undefined attempt to the family as $p = 1$, so the primary convention is $M = 8$. Adjusting only the seven valid p-values ($M = 7$) is a different convention and may be reported only as a sensitivity note.)

22-P03 Reference Implementation

```
import numpy as np
from scipy import integrate, optimize, stats


def mean_max_abs_normal(n):
    cdf = lambda x: 2 * stats.norm.cdf(x) - 1
    pdf = lambda x: 2 * stats.norm.pdf(x)
    return integrate.quad(lambda x: x * n * pdf(x) * cdf(x) ** (n - 1), 0, 12, limit
    ↪ =200)[0]


rng = np.random.default_rng(1)
m, rho, worlds = 32, 0.8, 20000
common = rng.standard_normal((worlds, 1))
z = np.sqrt(rho) * common + np.sqrt(1 - rho) * rng.standard_normal((worlds, m))
observed = np.abs(z).max(axis=1).mean()
```

```
n_eff = optimize.brentq(lambda n: mean_max_abs_normal(n) - observed, 1.0, 500.0)
assert 1.55 < observed < 1.75 and 5.0 < n_eff < 7.0
assert abs(mean_max_abs_normal(32) - 2.347) < 0.01
print(round(observed, 3), round(n_eff, 2), round((np.abs(z).max(axis=1) > 1.96).mean
    ↪ (), 3))
```

The output is approximately 1.648, 5.92, and 0.258: the maximum absolute value of 32 variants with correlation 0.8 matches about six independent trials on this one moment. Under the null, taking the maximum leads to rejection at 1.96 in 26% of cases, far below the 81% for independent tests.

22-C01 Let V be the number of false rejections and R the total number of rejections. If $V \geq 1$, then $R \geq 1$ and $V/R \leq 1$; if $V = 0$, then $V/\max(R, 1) = 0$. Therefore, $V/\max(R, 1) \leq \mathbf{1}\{V \geq 1\}$. Taking expectations gives FDR $\leq \Pr(V \geq 1) =$ FWER. For a strict example, let $M = 10$, with $m_0 = 5$ true nulls and five false nulls that are almost surely rejected. If one true null is falsely rejected, then $V = 1$, $R = 6$, and $V/R = 1/6 < 1$, while $\mathbf{1}\{V \geq 1\} = 1$. If this occurs with positive probability, FDR is strictly lower than FWER. Meaning: FDR allows a few errors among many true discoveries and is more useful in exploration; a claim about a single factor calls for the stricter FWER. Grade the three parts as in the original.

22-C02 (Scoring Guide and Reference) A sample plan: hypothesis: "the higher the compounded return over the past 12-1 months, the higher the return next month (positive sign)"; state whether the stock universe includes small caps and financials; specify total-return price data and `available_at`; give discovery and holdout endpoints; signal: 12-1 (11-month window, skip 1 month); label: next month; portfolio: equal-weight decile long-short, rebalanced monthly; benchmarks: equal-weight and permutation; primary metric: holdout Rank IC t; criterion: prespecify holdout Rank IC $t \geq 2.0$ with a positive sign as the sole primary confirmation; list after-cost portfolio results separately as secondary evaluation with risk constraints. If multiple metrics jointly confirm, state "all must pass" or a multiplicity-adjusted joint rejection rule before opening the holdout; do not select whichever is significant after the fact. Give a specific cost assumption in basis points; trial budget (for example, 12); ledger fields; embargo rule (one-month label, one-month embargo); and one-time holdout process. The following are illustrative constructed values, not estimates from the French history. Three outcomes that fail the primary hypothesis criterion are: mean holdout Rank IC 0.02 with $t = 1.70$; mean -0.01 with $t = -2.3$ (wrong direction); and mean 0.001 with $t = 0.1$ (insufficient precision). Separately, if a nonnegative mean net return was prespecified as an economic-feasibility gate, a net mean of -0.002 fails that secondary gate; it does not automatically overturn the sole IC statistical hypothesis if that passed. Report statistical confirmation and after-cost economic feasibility separately. A different study with net return as its primary hypothesis must fix its test and multiplicity rule before opening the holdout. State the limits: the data have been repeatedly used by others, the local hash is not external preregistration, and costs are assumptions. Grade the four parts as in the original. Acceptable variations in the twelve items and thresholds are fine if they are falsifiable and fixed before running.

19. Further Reading

- Must know: Harvey–Liu–Zhu working paper Sections 4.3–4.4 (definitions and examples of FWER, FDR, Bonferroni, and Holm); read the abstract and introduction first, then these sections. Purpose: understand why $t > 2$ is no longer enough.
- Must know: Gelman–Loken, Sections 1.1–1.2 and 4.3, on forking paths and the limits of preregistration.
- Should know: the first half of Bailey–López de Prado's *Deflated Sharpe Ratio*, Eq. (1) and Appendix A.1; read the full construction again in Chapter 32.
- Should know: the definition in Section 2 of Bailey et al.'s PBO paper, on overfitting in the selection process; used in Chapters 30–32.
- Advanced: the original papers by Holm (1979), Benjamini–Hochberg (1995), and Benjamini–Yekutieli (2001), to check the conclusions in Sections 6.3 and 6.4; these originals have not been read for this chapter.
- Optional: instructions from any public preregistration platform to learn how external timestamps work.

20. References

Citation keys and reading ranges are listed in `source_notes.md`.

- CH22_HLZ2014: Harvey, C. R., Liu, Y., Zhu, H. (2014). "… and the Cross-Section of Expected Returns." NBER Working Paper 20592. Supports Sections 2, 5, 6.3, and 7; the 2016 RFS journal version has not been read.
- CH22_DSR2014: Bailey, D. H., López de Prado, M. (2014). "The Deflated Sharpe Ratio." Authors' 2014-07-31 preprint. Supports the equation in Section 6.5.
- CH22_PBO2015: Bailey, D. H., Borwein, J. M., López de Prado, M., Zhu, Q. J. (2015). "The Probability of Backtest Overfitting." Authors' preprint. Used only as a conceptual preview in Sections 5 and 19.
- CH22_GELMANLOKEN2013: Gelman, A., Loken, E. (2013). "The garden of forking paths." Unpublished working paper. Supports Sections 2, 3, and 5.
- CH22_JT1993: Jegadeesh, N., Titman, S. (1993). "Returns to Buying Winners and Selling Losers." *Journal of Finance* 48(1), printed pages 68–69; supports the Bonferroni discussion of 32 strategies in Section 7.
- FRENCH: Kenneth R. French Data Library, 10 Industry Portfolios (frozen snapshot and hash in Section 11 and `data/manifests/french_industries_2026-10-05.json`).

Chapter 23 — Cross-Sectional Research

The central idea: At one point in time, rank many assets by a signal. Use the relationship between that ranking and next-period returns to measure prediction, then use quantile portfolios to answer what it would mean to hold the assets. These answer different questions. Prerequisites: Chapter 8 (regression), Chapter 15 (factor models and HAC), and Chapter 22 (research methodology). By the end, you will be able to calculate ranks, percentiles, and quantile buckets in a cross-section with ties and missing values; distinguish Pearson IC from Rank IC; derive why a pooled correlation across dates is not the same as per-date IC; construct quantile long-short portfolios and calculate turnover; derive the effective sample size of an IC series with overlapping labels; and use the two-step Fama–MacBeth method to estimate average returns per unit of signal.

1. Learning objectives

You should be able to:

1. By hand, calculate ranks, percentiles, and q-quantile buckets for a single-date cross-section with ties and missing values (`NaN`), and explain the tie rule at each step.
2. Calculate the Information Coefficient (IC) and Rank IC for each date, aggregate them over time, and use a Newey–West adjustment for inference; explain why a pooled correlation cannot replace per-date IC.
3. Distinguish three time objects: `signal_time` (when the signal is formed), `trade_time` (when the trade executes), and the label interval (the return-measurement period that begins after the signal). Construct labels with `lead_return` and `forward_label` so the label interval does not overlap the signal window.
4. Construct quantile, decile, and long-short portfolios; describe three weighting schemes (equal weight, market-cap weight, and signal-proportional weight); and derive why a signal-proportional portfolio's return equals IC $\times \sigma_r$.
5. Calculate IC decay and turnover; explain how signal persistence relates to forecast persistence in a specified constructed model, and how weights and costs affect trading frequency.
6. Explain when overlapping labels make the naive t statistic too large, derive the variance inflation factor (VIF), and use the two-step Fama–MacBeth method to estimate average returns per unit of standardized signal.

Completion standard: Given a signal table and a return table indexed by date and asset, you can specify every column in the research table, state when each column becomes available, and reproduce every number in Section 8.

2. Why quants need this

Chapter 22 explains how to record experiments honestly. This chapter covers the measurements used most often in each experiment. Most multifactor strategies use a panel with many assets over many dates: at each month-end, hundreds of assets have signal values, and each has a return in the following month. You need to answer two questions: Do assets with higher signals earn higher returns next month (prediction)? And how much would you earn, and how quickly would you trade, by holding assets ranked on the signal (holding)?

These questions require different tools. Prediction concerns only the relative order of assets within each date, so rank within each date first and then aggregate across dates. If you put all assets from all dates into one correlation, market-wide moves and shifts in the signal's overall level enter the calculation, producing a number unrelated to ranking skill. A holding analysis turns ranks into weights and accounts for changes in weights between dates (turnover) and costs.

The chapter proceeds as follows: define within-date ranking and tie rules; measure prediction with per-date IC, aggregate over time for inference, and handle dependence caused by overlapping labels; turn ranks into quantile portfolios and derive the link between IC and portfolio returns; examine how signal persistence relates to IC decay in a specified model and how weights affect turnover; and use Fama–MacBeth regression to estimate average return per unit of signal with a standard error. Each factor in Chapters 24–28 follows this sequence.

3. Intuition

Imagine a school that gives an exam every month, with a different subject and difficulty each time. You want to know whether a student's preview score predicts their final-exam score. If you pool every student and every month into one correlation, the easy months (when everyone's scores are high) may coincide with months when preview scores are also high (perhaps the preview questions happened to be easy). That creates a correlation unrelated to individual ability. The right approach is to rank students within each exam, ask whether those who ranked higher on the preview also ranked higher on the final, and then average the results across exams. That is per-date IC.

A quantile portfolio answers a different question. You do not need to predict each student's exact score. You can select the top 20% by preview rank and compare their average final-exam score with the bottom 20%. The difference is the long-short return. It depends on how you form the groups, how you weight students within each group, and how many students you replace each month.

The analogy has limits. Exam subjects differ. In finance, a common market shock creates cross-sectional dependence among assets within a month. Dependence across months is a separate question: examine return dynamics, signal persistence, and overlapping labels rather than inferring time dependence directly from a same-month common shock. The students remain the same group a year later, while asset rankings by signal change from month to month. In the stable linear construction introduced later, the speed of that change is related to the duration of IC. The following sections quantify these issues.

3.1 Work through a small asset-by-date table

The following is a teaching construction; the return label is not available on the signal date. An asset with a missing signal is excluded from that date's ranking. An asset with a missing label can still have a signal percentile, but it cannot enter that date's IC calculation. A–E are fixed asset identifiers.

Date	Field	A	B	C
January	Signal available by the cutoff	1	2	3
January	Ex post next-month return label	0.01	Missing	0.03
February	Signal available by the cutoff	1	1	1
February	Ex post next-month return label	0.01	0.02	0.03
March	Signal available by the cutoff	4	3	2
March	Ex post next-month return label	0.01	0.02	0.03

Date	D	E
January	4	Missing
January	0.02	0.04
February	1	Missing
February	0.04	Missing
March	1	Missing
March	0.04	Missing

In January, the number of valid signals is $N_{\text{signal},t} = 4$. The ranks across all signals are $(1, 2, 3, 4)$, and the percentiles are $(0, 1/3, 2/3, 1)$. With $q = 2$, the bucket formula $\lfloor 2(r - 0.5)/4 \rfloor + 1$ gives $(1, 1, 2, 2)$. As a tie variant, change C's signal to 2. The average ranks become $(1, 2.5, 2.5, 4)$, the percentiles $(0, 0.5, 0.5, 1)$, and the buckets $(1, 2, 2, 2)$. Do not split a tied group to force equal bucket sizes. The rest of the chapter uses the original table with C=3. For IC, only A, C, and D form valid pairs, so $n_{\text{pair},t} = 3$. Re-rank within those three pairs: signal ranks $(1, 2, 3)$, label ranks $(1, 3, 2)$, and both means are 2. The centered cross-product sum is 1, each sum of squares is 2, and Rank IC is $1/\sqrt{2 \times 2} = 0.5$.

If you rank all four signals first and then drop B, the signal ranks become $(1, 3, 4)$, which gives a correlation of about 0.655 with the label ranks. That is not the specified Rank IC. Missing labels change the set of pairs, and therefore the ranking problem; deleting one precomputed rank is not enough.

February has four valid pairs, but the signal is constant, so IC is undefined. March has four valid pairs and Rank IC $= -1$. Thus the panel has three signal dates and per-date pair counts $(3, 4, 4)$, but only $T_{\text{valid}} = 2$ nonmissing IC observations. Their time-series mean is $(0.5 - 1)/2 = -0.25$. Do not treat February's undefined IC as zero, which would give $-1/6$, or treat the 11 pairs as 11 independent dates. Two valid dates illustrate the calculation but do not support a robust statistical conclusion.

4. Mathematical foundations

Notation. Dates $t = 1, \ldots, T$ are monthly. Assets are indexed by $i = 1, \ldots, N_t$, where $N_t = N_{\text{signal},t}$ is the number of assets with valid signals on date t and may vary over time. Let $n_{\text{pair},t}$ be the number of pairs with both signal and label available, and T_{valid} the number of dates with nonmissing IC. In earlier examples, n or N means the number of valid signals for a single-date ranking and the number of valid pairs for an IC calculation; in time-series inference, T must mean the number of valid dates. Signal $s_{i,t}$ is known at `signal_time` t (month-end); its units depend on the signal definition (a ratio, a return, and so on). Label $y_{i,t}$ is the simple return over a later interval, expressed as a decimal. For example, the next-month label is $y_{i,t} = R_{i,t+1}$; the label over h months is $y_{i,t} = \prod_{k=1}^{h}(1 + R_{i,t+k}) - 1$. A cross-section fixes t and operates across i.

Within-date statistics. Calculate each IC statistic on the set of valid pairs. In the equations below, N_t therefore means $n_{\text{pair},t}$, not the number of assets with signals only. The within-date sample mean is $\bar{s}_t = \frac{1}{N_t}\sum_i s_{i,t}$; the population-form variance is $\sigma^2_{s,t} = \frac{1}{N_t}\sum_i (s_{i,t} - \bar{s}_t)^2$ (divide by N_t, or `ddof=0`); and the covariance is $\text{Cov}_t(s, y) = \frac{1}{N_t}\sum_i (s_{i,t} - \bar{s}_t)(y_{i,t} - \bar{y}_t)$. The within-date Pearson correlation is $\text{IC}_t = \text{Cov}_t(s, y)/(\sigma_{s,t}\sigma_{y,t})$.

Law of total covariance. Treat all (i, t) observations from the T dates as one large sample (with N assets per date and NT observations in total). The population covariance equals the average within-date covariance plus the covariance between date-level means. This is the basis for the pooled-correlation issue in the next section.

Newey–West. Chapter 22, Section 4 gives the formula. Here we use it to aggregate monthly IC over T dates. $L = 0$ corresponds to the independent, identically distributed formula (divisor T).

Autoregressive signal of order 1 (AR(1)). If $z_{i,t} = \rho z_{i,t-1} + \sqrt{1 - \rho^2}\,\epsilon_{i,t}$, with $|\rho| < 1$, independent and identically distributed innovations ϵ of mean 0 and variance 1 that are independent of past signals, and a stationary initialization (signal mean 0 and variance 1), then $\text{Var}(z) = 1$ and $\text{Corr}(z_{i,t}, z_{i,t+j}) = \rho^j$.

Conditional expectation under a bivariate normal distribution. If (Z, r) is bivariate normal, with $Z \sim N(0, 1)$, r having mean μ_r and standard deviation σ_r, and correlation ρ_{IC} with Z, then $\mathbb{E}[r \mid Z] = \mu_r + \rho_{\text{IC}}\sigma_r Z$. For a standard normal Z, the upper-tail conditional mean is $\mathbb{E}[Z \mid Z > c] = \varphi(c)/(1 - \Phi(c))$, where φ is the standard normal density.

Frequency convention: monthly data; `signal_time` is month-end; the label begins in the following month.

5. Formal definitions

Cross-section and panel. A cross-section is the asset vector at a fixed date t. A panel is a table with dates as rows and assets as columns; the row index is `signal_time`. Use NaN as the only missing-value marker. Reject infinities, complex numbers, booleans, text, and `None`: they either have no numerical meaning or may be silently treated as 0 or 1.

Rank and ties. For valid (non-NaN) signals on date t, sort values in ascending order; the kth smallest value has rank k. If m values are equal, they occupy ranks $k, \ldots, k+m-1$, and each receives their average, $k + (m-1)/2$ (the average-rank rule). Equivalently, $\text{rank}_i = \#\{j : s_j < s_i\} + \big(\#\{j : s_j = s_i\} + 1\big)/2$. Drop NaN before ranking; it occupies no rank and remains missing in the output.

Percentile. If there are $n \geq 2$ valid items, define percentile $u_i = (\text{rank}_i - 1)/(n-1) \in [0, 1]$. For $n = 1$, define it as 0.5; if all values are tied, each receives 0.5. Percentiles put ranks from dates with different N_t on a common scale.

q-quantile buckets. Let $\tilde{u}_i = (\text{rank}_i - \frac{1}{2})/n \in (0, 1)$ and assign bucket $g_i = \lfloor q\,\tilde{u}_i \rfloor + 1 \in \{1, \ldots, q\}$. Bucket 1 has the lowest signals; bucket q has the highest. Tied values have the same average rank and therefore enter the same bucket. If there are fewer than q valid items, set the whole row to NaN because at least one bucket would be empty. $q = 5$ gives quintiles; $q = 10$ gives deciles.

Information Coefficient and Rank IC. IC_t is the Pearson correlation between $\{s_{i,t}\}_i$ and $\{y_{i,t}\}_i$ on date t. Rank IC is the Pearson correlation of their average ranks, also called Spearman correlation. Use only pairs for which both values are valid, and calculate ranks within that paired set. Under this chapter's minimum-pair convention, do not report IC when there are fewer than three pairs (a mathematical correlation of ± 1 is still possible with two nonconstant pairs). A constant value on either side also makes the correlation undefined. The functions return NaN in both cases, not zero.

IC series, ICIR, and hit rate. For $\{\text{IC}_t\}_{t=1}^{T}$, report the mean $\overline{\text{IC}}$ and standard deviation s_{IC} (`ddof=1`). The Information Coefficient Information Ratio (ICIR) is $\overline{\text{IC}}/s_{\text{IC}}$; it is a per-period ratio, not annualized. The hit rate is the fraction of dates with $\text{IC}_t > 0$. Under independent dates, ICIR relates to the Chapter 22 t statistic by $t = \text{ICIR} \cdot \sqrt{T}$.

IC decay and signal decay. Hold signal $s_{i,t}$ fixed and calculate its IC with single-period returns $R_{i,t+k}$ at leads $k = 1, 2, \ldots$. The resulting function of k describes how forecast power fades with horizon.

Quantile, decile, and long-short portfolios. On date t, sort assets into q buckets by signal. There are three within-bucket weighting schemes: equal weight; market-cap weight, using market capitalization known at `signal_time`; and signal-proportional weight, proportional to the demeaned signal. A long-short portfolio buys bucket q and sells bucket 1, with one unit of capital in each leg, gross exposure of 2, and net exposure of 0. Its gross return excludes costs.

Turnover. This chapter uses two definitions. Target-weight turnover is $\sum_i |w_{i,t} - w_{i,t-1}|$ (purchases plus sales, without dividing by two, and excluding weight changes caused by price drift). Membership turnover is the fraction of the prior-period bucket members that are absent from the current bucket. The former is used for cost estimates; the latter is an intuitive measure of signal stability.

Overlapping labels and effective sample size. If each month produces an IC from a label spanning $h > 1$ months, adjacent IC observations share some return months; the labels overlap. For a stationary series with a finite long-run variance, define $n_{\text{eff}} = T/\text{VIF}$ as the number of dates that would give the same variance for the mean. If the net contribution from autocorrelation is positive, $n_{\text{eff}} < T$; if negative, it can exceed T. It is neither a count of new observations nor the degrees of freedom for a test.

Two-step Fama–MacBeth method. First, for each date t, run a cross-sectional regression across assets: $y_{i,t} = a_t + b_t\, x_{i,t} + e_{i,t}$. Then treat the time series of b_t as data and calculate its mean and standard error. Usually, x is the signal standardized within that date, so b_t has units of label return per one standard deviation of signal.

Panel dependence. Returns across assets on the same date are correlated because of common shocks (time effects); residuals for the same asset across dates may be correlated because of persistent characteristics (asset or firm effects). Fama–MacBeth handles the former but gives standard errors that are too small under the latter (Petersen 2005). Newey–West adjusts for time-series dependence but does not address cross-sectional dependence.

5.1 Three counts and two divisor conventions

Use $N_{\text{signal},t}$ for percentile ranking and bucket assignment; re-rank the paired observations for IC using $n_{\text{pair},t}$; use T_{valid} for time-series means and inference. They are all counts, but count different objects. If dates are frequently missing, report the calendar span, missing dates, and their causes. Equal weighting across valid dates is not weighting by the number of asset pairs.

For a date with fixed $n = n_{\text{pair},t} \geq 3$, let C be the centered cross-product sum and $S_s, S_y > 0$ the two sums of squares. If both use the same cross-sectional divisor $n - d$, where $d = 0$ or 1, then

$$\text{IC}_t = \frac{C/(n-d)}{\sqrt{S_s/(n-d)}\sqrt{S_y/(n-d)}} = \frac{C}{\sqrt{S_s S_y}}.$$

The numerator and denominator share the same positive factor, so it cancels and correlation is dimensionless. This cancellation holds when covariance and standard deviations use a consistent divisor; it does not mean arbitrary mixtures of divisors have no effect. For the January ranks, $C = 1$ and $S_s = S_y = 2$. With `ddof=0`, Cov $= 1/3$ and $\sigma = \sqrt{2/3}$; with `ddof=1`, Cov $= 1/2$ and $\sigma = 1$. Both correlations equal 0.5.

Use `ddof=1` for the time-series standard deviation s_{IC}, estimating variation across dates with $(T_{\text{valid}} - 1)$. The ordinary standard error of the mean is $s_{\text{IC}}/\sqrt{T_{\text{valid}}}$. In Section 3.1, the two deviations are $+0.75, -0.75$, their squared sum is 1.125, $s = \sqrt{1.125} = 1.06066$, the standard error is 0.75, and the naive $t = -1/3$. Do not take the square root of the number of cross-sectional pairs.

In the current HAC implementation, sample autocovariances all use the divisor T_{valid}. Thus, at $L = 0$, $\widehat{\text{Var}}_{\text{NW}}(\bar{\text{IC}}) = (T_{\text{valid}} - 1)s^2/T_{\text{valid}}^2$, which differs from the ordinary s^2/T_{valid} by a finite-sample factor. The two values in the example are 0.28125 and 0.5625. These are different estimation conventions, not an error in the correlation formula.

Missing-month interfaces. `ic_series` first intersects the dates and assets in its two inputs; missing dates outside that intersection do not appear in its returned missing-value count and must be checked against the original calendar separately. `ic_summary` first counts and drops NaN, then passes the compressed array of valid ICs to `newey_west_variance_of_mean`. The latter, when called directly, requires a complete array of finite values and raises `ValueError` on NaN; it does not drop missing values. Newey–West lags count positions in the input array. Thus, January and March in Section 3.1 become adjacent valid observations after compression, and $L = 1$ means a one-observation gap, not one calendar month. The return-label constructors instead require a continuous monthly index and reject missing months. Retain missing IC dates and audit their causes. For a HAC analysis with a true "one-month lag," do not relabel a result from the compressed sequence; define a separate protocol for a continuous date index and missing values, or use a bounded complete window. Do not fill in zeros to manufacture a complete IC series.

5.2 Five clocks for financial reports, signals, and trades

The dates below reuse the event record for constructed P102/FY2013 in Project 4; they are not newly collected point-in-time data.

Clock	Current field and record	What it permits
Reporting period	`period_end`: 2013-12-31	Identifies the fiscal year, but does not prove public availability
Publication and ingestion	`available_at`: 2014-04-17 16:05 UTC; `ingested_at`: 2014-04-18	The same version is usable only if both precede the cutoff
Signal cutoff	April 2014 month-end; the program's comparison cutoff is 2014-05-01 00:00 UTC	Select the latest period/version for which both fields are strictly before the cutoff
First actually tradable time	The first time in May 2014 with an actually tradable price	Occurs after the signal; a monthly table cannot supply a specific execution price
Label interval	Current proxy: April month-end close to May month-end close; execution label: actual trade price to exit price	The proxy includes a return segment before the actual trade; the two are not equivalent

The original 10-K version was available in April. A restatement published on May 20 and ingested on May 21 is usable only from the May signal onward. The cutoff uses a strict comparison against the first instant of the following month; it does not force an unknown timestamp precision into "before 23:59:59 at month-end." A month-end return label answers a hypothetical month-end execution question. A real portfolio return must start at the actual execution price and connect to the cash/position and corporate-action ledger. An integer-month `lag` cannot replace a trading calendar.

6. Derivations

6.1 Quantile bucket sizes and ties

Without ties, ranks take all values in $1, \ldots, n$, and $\tilde{u} = (r - \frac{1}{2})/n$ gives equally spaced points in $(0, 1)$ at intervals of $1/n$: $\frac{1}{2n}, \frac{3}{2n}, \ldots$. Bucket g contains ranks satisfying $(g-1)/q \le \tilde{u} < g/q$, or $r - \frac{1}{2} \in [(g-1)n/q,\ gn/q)$. A half-open interval of length $L = n/q$ contains either $\lfloor L \rfloor$ or $\lceil L \rceil$ points from a unit-spaced grid $(\frac{1}{2}, \frac{3}{2}, \ldots)$, so bucket sizes differ by at most 1. For $n = 10, q = 5$, each bucket has 2 assets; for $n = 11, q = 5$, bucket sizes are 2 or 3. With ties, all values in a tied group share an average rank, so bucket sizes need not be even and a bucket may be empty. That is a consequence of ties, not a code defect.

6.2 Spearman correlation is Pearson correlation of ranks; the no-ties formula

For n assets without ties, ranks r_i and q_i are each permutations of $1, \ldots, n$. Therefore $\bar{r} = \bar{q} = \frac{n+1}{2}$ and $\sum_i (r_i - \bar{r})^2 = \sum_i (q_i - \bar{q})^2 = \frac{n(n^2-1)}{12}$. Let $d_i = r_i - q_i$. Then

$$\begin{aligned} \sum_i d_i^2 &= \sum_i (r_i - \bar{r})^2 + \sum_i (q_i - \bar{q})^2 \\ &\quad - 2\sum_i (r_i - \bar{r})(q_i - \bar{q}), \\ \sum_i (r_i - \bar{r})(q_i - \bar{q}) &= \frac{n(n^2-1)}{12} - \frac{1}{2}\sum_i d_i^2. \end{aligned}$$

Pearson correlation divides the cross-product sum by the geometric mean of the two sums of squares, which is $\frac{n(n^2-1)}{12}$. Hence

$$\rho_S = 1 - \frac{6\sum_i d_i^2}{n(n^2-1)}.$$

This formula applies only when there are no ties. With ties, calculate Pearson correlation on average ranks. See the example in Section 8.3.

6.3 Why pooled correlation is not per-date IC

Suppose each date has the same N assets and all NT observations are placed in one correlation. Let date t have means $\bar{s}_t, \bar{y}_t$, and let the overall means be $\bar{\bar{s}}, \bar{\bar{y}}$. Total covariance is

$$\mathrm{Cov}_{\mathrm{pool}}(s, y) = \underbrace{\frac{1}{T}\sum_t \mathrm{Cov}_t(s, y)}_{\text{average within-date covariance}} + \underbrace{\frac{1}{T}\sum_t (\bar{s}_t - \bar{\bar{s}})(\bar{y}_t - \bar{\bar{y}})}_{\text{covariance between date means}}.$$

To derive this, write each observation as $s_{i,t} - \bar{\bar{s}} = (s_{i,t} - \bar{s}_t) + (\bar{s}_t - \bar{\bar{s}})$, and do the same for y. When multiplied and summed within each date, the cross-terms vanish because $\sum_i (s_{i,t} - \bar{s}_t) = 0$. The first term contains only within-date ranking information. The second is co-movement

between the overall signal level and return level on each date: in finance, "the signal is higher overall when the market is up overall." It says nothing about which asset is stronger than another. Pooled correlation adds both terms and divides by total variance; per-date IC first demeans within each date, eliminating the second term. Section 8.4 gives an example with per-date ICs of $+1$ and -1 (mean zero), while pooled correlation is 0.89, entirely because of the second term.

6.4 A signal-proportional portfolio return equals $\mathrm{IC} \times \sigma_r$

Standardize date t's signal within the date: $z_i = (s_i - \bar{s})/\sigma_s$, using the population-form standard deviation. Then $\bar{z} = 0$ and $\frac{1}{N}\sum z_i^2 = 1$. Set weights to $w_i = z_i/N$ (net exposure 0). The portfolio return is

$$R_p = \sum_i w_i y_i = \frac{1}{N}\sum_i z_i y_i = \frac{1}{N}\sum_i z_i(y_i - \bar{y}) = \mathrm{Cov}_t(z, y) = \mathrm{IC}_t \cdot \sigma_{y,t}.$$

The second equality uses $\sum_i z_i = 0$, the third uses the covariance definition, and the fourth uses $\sigma_z = 1$. Thus, the current return of a signal-proportional portfolio equals that date's IC times the cross-sectional dispersion of returns, $\sigma_{y,t}$. This identity does not assume normality. It connects prediction (IC) and holding returns: for the same IC, a date with greater return dispersion earns more. Gross exposure for this portfolio is $\frac{1}{N}\sum_i |z_i|$, about 0.80 if the signal is normally distributed. Multiplying all weights by $2/\left(\frac{1}{N}\sum_i |z_i|\right)$ gives a gross-exposure-2 version, and scales the portfolio return by the same factor.

6.5 Expected long-short return by quantile under bivariate normality

Under the bivariate normal assumption in Section 4, $\mathbb{E}[y \mid z] = \mu_y + \mathrm{IC}\,\sigma_y z$. The upper-tail bucket has probability $1/q$: $z > c_q$, where $c_q = \Phi^{-1}(1 - 1/q)$. Its average return is $\mu_y + \mathrm{IC}\,\sigma_y$ $\mathbb{E}[z \mid z > c_q] = \mu_y + \mathrm{IC}\,\sigma_y \cdot q\,\varphi(c_q)$. The lower-tail bucket is symmetric with the opposite sign. Therefore

$$\mathbb{E}[\text{long-short}] = 2\,\mathrm{IC}\,\sigma_y\,q\,\varphi(c_q).$$

Numerically, for $q = 5$, $c = 0.8416$, $\varphi = 0.2800$, and $q\varphi = 1.400$; for $q = 10$, $c = 1.2816$, $\varphi = 0.1755$, and $q\varphi = 1.755$; for $q = 3$, $q\varphi = 1.091$. With IC $= 0.05$ and $\sigma_y = 8\%$, quintile long-short earns about $2 \times 0.05 \times 0.08 \times 1.400 = 1.12\%$ per month, and decile long-short about 1.40% per month. In this static model, finer buckets increase expected long-short returns but leave fewer assets in each bucket and increase the sampling noise of each bucket mean. Compare the mean with its standard error to assess signal-to-noise; bucket count alone does not determine the direction. Turnover also depends on signal dynamics and ranking/weight rules, so the static formula says nothing about turnover. This derivation assumes a bivariate normal distribution; with heavy tails and nonlinearities, treat it as an order-of-magnitude reference.

6.6 Fama–MacBeth: regress each date, then average over time

The ordinary least squares (OLS) slope in date t's regression $y_{i,t} = a_t + b_t z_{i,t} + e_{i,t}$ is

$$\hat{b}_t = \frac{\sum_i z_{i,t}(y_{i,t} - \bar{y}_t)}{\sum_i z_{i,t}^2} = \frac{N \operatorname{Cov}_t(z, y)}{N} = \mathrm{IC}_t \cdot \sigma_{y,t} \quad \text{(when } z \text{ is standardized within date)},$$

which is the same as the portfolio return in Section 6.4. Thus, each period's Fama–MacBeth slope is the return on a signal-proportional portfolio. The second step is $\bar{b} = \frac{1}{T}\sum \hat{b}_t$, with $\mathrm{se}(\bar{b}) = s_b/\sqrt{T}$ under independent periods. Petersen (2005, equations (8) and (9)) gives this estimator and variance formula and states the assumption that period coefficients are independent. If the $\hat{b}_t$ series is autocorrelated, this course uses Newey–West inference. That is a design choice in this textbook, not a recommendation from Petersen's paper. Petersen separately tests Fama–MacBeth standard errors adjusted for first-order autocorrelation and finds that they remain too small when firm effects are persistent.

How this differs from IC: $\overline{\mathrm{IC}}$ weights each month equally, while $\bar{b}$ weights each month by $\sigma_{y,t}$, giving more weight to months with greater dispersion. Their t statistics therefore differ. Neither is right or wrong: IC asks whether the ranking contains information; the FM slope asks how much return each unit of signal earns on average. The historical industry data in Section 11 give $t = 4.99$ for IC and $t = 3.35$ for the FM slope.

6.7 IC decay and signal persistence in a specified constructed model

Constructed setting: next-period return is $y_{i,t+1} = \beta z_{i,t} + \epsilon$, signal z follows an AR(1) process with coefficient ρ, and ϵ is independent of past signals. The covariance between $z_{i,t}$ and the return k months later, $y_{i,t+k} = \beta z_{i,t+k-1} + \epsilon$, is $\beta\rho^{k-1}$, reflecting signal autocorrelation. Therefore

$$\mathrm{IC}_k = \mathrm{IC}_1 \cdot \rho^{k-1}.$$

When $0 < \rho < 1$, the signal/IC half-life in this construction is $\ln(0.5)/\ln\rho$ months: the number of months for IC to halve starting from lead 1. Halving occurs at lead $k = 1 + \ln(0.5)/\ln\rho$. For $\rho = 0.85$, the half-life is 4.27 months ($k \approx 5.27$); for $\rho = 0.9$, it is 6.58 months ($k \approx 7.58$). In this stable linear prediction model, greater persistence means slower IC decay. If ranking and weighting rules also remain stable, turnover may be lower. Actual prediction mechanisms, rank crossings, universe changes, and costs can alter this relationship; signal autocorrelation alone does not determine a tradable holding period.

6.8 Overlapping labels and autocorrelation in the IC series

Let the signal follow an AR(1) process with coefficient ρ, and let date t's h-month label be $S_{i,t} = \sum_{k=1}^{h} R_{i,t+k}$. Under the null, returns are independent and identically distributed with variance σ^2 and are unrelated to the signal. The leading term in IC is $\mathrm{IC}_t \approx \frac{1}{N\sqrt{h}\sigma} \sum_i z_{i,t} S_{i,t}$, where z is standardized. This is a large-cross-section approximation: sampling variation in the within-date mean is a higher-order term, and the sample standard deviations in the denominator are replaced by population values $\sigma_z = 1$ and $\sigma_S = \sqrt{h}\sigma$. For $0 < j < h$,

$$\mathrm{Cov}(\mathrm{IC}_t, \mathrm{IC}_{t+j}) = \frac{1}{N^2 h\sigma^2} \sum_i \mathbb{E}[z_{i,t} z_{i,t+j}]\, \mathbb{E}[S_{i,t} S_{i,t+j}] = \frac{1}{N^2 h\sigma^2} \cdot N\rho^j \cdot (h-j)\sigma^2 = \frac{\rho^j (h-j)}{Nh},$$

only the same-asset terms are nonzero (under the null, signals and returns for different assets are independent), and the labels share $h - j$ return months. Meanwhile, $\mathrm{Var}(\mathrm{IC}_t) = 1/N$. Thus,

$$\mathrm{Corr}(\mathrm{IC}_t, \mathrm{IC}_{t+j}) = \rho^j \Big(1 - \frac{j}{h}\Big), \quad 0 < j < h, \qquad 0 \ (j \geq h).$$

For a stationary series of finite length T, the variance of its mean is

$$\mathrm{Var}(\overline{\mathrm{IC}}) = \frac{\sigma^2_{\mathrm{IC}}}{T} \mathrm{VIF}_T,$$
$$\mathrm{VIF}_T = 1 + 2 \sum_{j=1}^{T-1} \left(1 - \frac{j}{T}\right) \mathrm{Corr}(\mathrm{IC}_t, \mathrm{IC}_{t+j}).$$

There are $T-j$ pairs at lag j, which gives the factor $1-j/T$; stationarity makes the covariance depend only on lag. The following VIF omits that pair-count factor and is the long-run variance approximation when T is large relative to h:

$$\mathrm{VIF} = 1 + 2 \sum_{j=1}^{h-1} \rho^j \Big(1 - \frac{j}{h}\Big), \qquad n_{\mathrm{eff}} = T/\mathrm{VIF}.$$

At the limits, when $\rho = 0$ (no signal persistence), VIF $= 1$ and the naive t statistic is not affected; when $\rho = 1$, VIF $= h$ and $n_{\mathrm{eff}} = T/h$. **Under the specified null-world construction with a large cross-section, independent assets, and independent monthly returns, overlapping labels and signal persistence jointly generate this dependence.** Other data-generating processes can also make ICs dependent through serially correlated returns, common shocks, or time-varying cross-sections and missingness. This formula does not show that signal persistence is the only source in every model. For example, with $\rho = 0.85$ and $h = 3$, VIF $= 1 + 2(0.85 \times \frac{2}{3} + 0.7225 \times \frac{1}{3}) = 2.615$. With $T = 120$ months, the long-run approximation gives $n_{\mathrm{eff}} = 45.9$, and the naive t is overstated by a factor of $\sqrt{2.615} = 1.62$. Retaining the finite-120-period pair-count factor gives

$$\mathrm{VIF}_{120} = 1 + 2\left[\frac{119}{120} 0.85 \frac{2}{3} + \frac{118}{120} 0.85^2 \frac{1}{3}\right]$$
$$= 2.59753.$$

The equivalent number of dates is about 46.20. "Exact" here applies only to the finite-period count under the specified approximate autocorrelation model, not to the exact distribution of a finite-cross-section IC.

If the same model instead uses $\rho = -0.5, h = 3$, the long-run VIF $= 1 + 2[-0.5(2/3) + 0.25(1/3)] = 0.5$. For 120 periods it is 0.50278, giving about 238.67 equivalent dates, more than 120. Negative serial correlation cancels variation in the mean, so here the naive standard error is too large. The case $\rho = 1$ is the boundary where the signal is perfectly persistent: long-run VIF is h, while the finite-period expression still needs the factor $1 - j/T$. Therefore, T/h is not a general formula for effective dates.

Newey–West uses Bartlett weights $(1 - j/(L + 1))$. Setting $L = h - 1$ gives $(1 - j/h)$. These weights multiply the **estimated** autocovariances, which already contain the structure $\rho^j(1 - j/h)$; the kernel downweights them a second time. For nonnegative persistence coefficients in this construction, the long-run population value recovered is only

$$\mathrm{VIF}_{\mathrm{NW}} = 1 + 2\sum_{j=1}^{h-1} \rho^j \left(1 - \frac{j}{h}\right)^2.$$

With $\rho = 0.85, h = 3$, $\mathrm{VIF}_{\mathrm{NW}} = 1.916$, below the model's long-run VIF of 2.615. The t statistic therefore remains overstated by about $\sqrt{2.615/1.916} = 1.17$. The same issue occurs even when $\rho = 1$: the model's long-run VIF $= h$, while Newey–West recovers only $(2h^2 + 1)/(3h)$ (2.11 for $h = 3$ and 8.03 for $h = 12$). This is a population difference caused by the kernel's fixed truncation length, which downweights positive autocovariances; it is not sampling error. The finite-sample simulations in Sections 10 and 12 show that rejection rates remain too high. A truncated estimate with all weights equal to 1 has no such population bias, but the sample estimate is not guaranteed to be positive. For overlapping labels, this chapter therefore prioritizes nonoverlapping samples or reporting both conventions.

7. Financial interpretation

IC is association, not return. A reported average IC of 0.05 means that the time average of per-date ICs is 0.05, not that IC equals 0.05 every month. A single IC_t remains a cross-sectional correlation at one date. Pearson IC measures linear association between raw values; Rank IC converts both sides to ranks. By Section 6.4, $R_p = \mathrm{IC} \cdot \sigma_y$. With $\sigma_y = 8\%$ and IC=0.05, the monthly return is 0.4% for the z/N portfolio (net exposure 0, gross exposure about 0.8). This is a gross return; costs, capacity, and financing are not included. The numbers in this chapter come only from constructed panels and 10 industry portfolios; they do not make a claim about a "typical IC" in any market.

Weights connect prediction and holding. The same IC series can produce very different portfolios: an equal-weight decile long-short portfolio uses ranks and is less sensitive to outliers; a market-cap-weighted quintile concentrates in larger assets, increasing capacity but also noise; signal-proportional weights use signal magnitude and are sensitive to heavy-tailed signals. The constructed examples in Sections 11 and 13 show large differences in gross returns and t statistics across these schemes. A claim that a "factor works" must specify the weighting rule.

Signal persistence informs trading choices. If the predictive direction and structure are stable, high persistence motivates evaluating lower-frequency rebalancing. A short-lived signal calls for comparing the incremental returns from faster rebalancing with its costs. An IC-decay curve describes associations at different leads in the sample. A tradable holding period also needs to be tested using actual weights, holdings, and costs; turnover helps quantify the associated expense.

Economic meaning of within-date processing. Ranking within each date before aggregating ensures that assets are compared at the same time and against the same macroeconomic backdrop. A broad market move (for example, valuations rise in one month and the market rises the next) is not counted as signal skill. This is the "same-date cross-sectional transformation" emphasized in Chapter 28.

7.1 Four different questions on the same panel

Use the two-date, four-asset panel from Section 8.4: on date A, signal $(1, 2, 3, 4)$ and return $(0.10, 0.12, 0.14, 0.16)$; on date B, signal $(11, 12, 13, 14)$ and return $(0.26, 0.24, 0.22, 0.20)$. The date means rise from $(2.5, 0.13)$ to $(12.5, 0.23)$, even though the within-date relationship switches from positive to negative.

Quantity	Hand calculation	Question answered
Pooled correlation	$2/\sqrt{210 \times 0.024} = 0.891$	How do date-level averages and within-date variation move together?
Cross-sectional Pearson/Rank IC	A is $+1$, B is -1, equal-date mean 0	Which asset is stronger on the same date?
FM slope on raw signal	$+0.02, -0.02$; return per unit of raw signal	How much return corresponds to one unit of signal?
Standardized FM and z/N gross portfolio	Both dates have $\sigma_s = \sqrt{5/4} = 1.118$; slope/gross return is ± 0.02236	How much return corresponds to one cross-sectional standard deviation of signal?

The standardization uses the same paired observations and the population-form standard deviation; the identity in Section 6.6 holds for single-signal OLS with an intercept. It does not carry over to partial slopes in a multiple-signal regression, Rank IC, or a different selected universe. On each date, $z = (-1.342, -0.447, 0.447, 1.342)$, $w = z/4$, and gross exposure $G = \sum |w| = 0.89443$. If scaled to gross exposure 2, the multiplier is $2/G = \sqrt{5} = 2.236$, and the two gross returns become ± 0.05. These weights use the same reference capital base, and their net sum is zero; gross exposure of 2 does not mean the capital base is 2.

Only after subtracting costs can we discuss net returns: $R_{\text{net}} = R_{\text{gross}} - c \sum_i |\Delta w_i|$ is this chapter's target-weight cost proxy, where c is the fee per unit of total purchases and sales. Drift, actual executed turnover, financing, borrow, and slippage belong in a separate execution ledger. If the two unscaled gross returns cancel and the proxy cost is 0.001 per period, the net mean is -0.001. An average IC of zero does not tell us those costs; a positive IC also does not equal cost-adjusted alpha (residual return relative to a specified risk model and benchmark).

8. Worked numerical examples

All numbers are produced by this chapter's CLI function `hand_examples()` and independently checked with a different implementation in `qa/a5/claude/recompute/ch23_recompute.py`.

8.1 Ranks and percentiles with ties and missing values

Worked example: Ranks and Percentiles

For a single-date cross-section, let the signal be $x = (3,\ 1,\ \text{NaN},\ 3,\ 2,\ 7,\ 5,\ 5)$ across 8 assets, with 1 missing value and $n = 7$ valid values. In ascending order they are $1, 2, 3, 3, 5, 5, 7$: the value 1 has rank 1; 2 has rank 2; the two 3s occupy ranks 3 and 4 and each receive 3.5; the two 5s occupy ranks 5 and 6 and each receive 5.5; and 7 has rank 7. The ranks are $(3.5,\ 1,\ \text{NaN},\ 3.5,\ 2,\ 7,\ 5.5,\ 5.5)$. The percentiles $(r-1)/6$ are $(0.4167,\ 0,\ \text{NaN},\ 0.4167,\ 0.1667,\ 1,\ 0.75,\ 0.75)$.

8.2 Quantile buckets

For $n = 10$ untied values $(1, \ldots, 10)$ and $q = 5$, $\tilde{u} = (r - 0.5)/10$, so $5\tilde{u} = 0.25, 0.75, 1.25, \ldots$. The bucket numbers $\lfloor \cdot \rfloor + 1$ are $(1, 1, 2, 2, 3, 3, 4, 4, 5, 5)$. With ties, let $x = (1, 2, 2, 2, 5, 6, 7, 8)$, $n = 8$, and $q = 4$. The average ranks are $(1, 3, 3, 3, 5, 6, 7, 8)$; $\tilde{u} = (r - 0.5)/8 = (0.0625, 0.3125, 0.3125, 0.3125, 0.5625, 0.6875, 0.8125, 0.9375)$; and $4\tilde{u} = (0.25, 1.25, 1.25, 1.25, 2.25, 2.75, 3.25, 3.75)$. Bucket numbers are $(1, 2, 2, 2, 3, 3, 4, 4)$. The three tied values all enter bucket 2, so it contains 3 assets; the other buckets contain 1, 2, and 2.

8.3 Pearson IC and Rank IC: an outlier

For six assets on one date, let signal $s = (1, 2, 3, 4, 5, 20)$ and next-period return $y = (0.02, 0.01, 0.04, 0.03, 0.06, -0.05)$. Then $\bar{s} = 5.833$, $\bar{y} = 0.01833$; $\sum(s - \bar{s})(y - \bar{y}) = -1.0617$, $\sum(s - \bar{s})^2 = 250.83$, and $\sum(y - \bar{y})^2 = 0.007083$. Pearson IC is $-1.0617/\sqrt{250.83 \times 0.007083} = -0.796$. Signal ranks are $1, \ldots, 6$; return ranks are $(3, 2, 5, 4, 6, 1)$ (sort y ascending: -0.05 has rank 1, 0.01 rank 2, 0.02 rank 3, 0.03 rank 4, 0.04 rank 5, and 0.06 rank 6, then read the ranks in the assets' original order). Thus, $d = (-2, 0, -2, 0, -1, 5)$, $\sum d^2 = 34$, and Rank IC $= 1 - 6 \times 34/(6 \times 35) = 0.0286$. The outlier signal value of 20 pulls Pearson IC down to -0.80, despite the near absence of ranking information. Rank IC looks only at order and is about 0.03. Which measure is more suitable depends on whether the signal-return relationship is linear and whether the outlier is meaningful. For heavy-tailed signals, this chapter defaults to Rank IC.

8.4 Pooled correlation and per-date IC

Two dates, four assets each. Date A: $s = (1, 2, 3, 4)$, $y = (0.10, 0.12, 0.14, 0.16)$, within-date IC $= +1$. Date B: $s = (11, 12, 13, 14)$, $y = (0.26, 0.24, 0.22, 0.20)$, within-date IC $= -1$ (higher signal means lower return within that date). The mean per-date IC is 0. For the pooled sample of eight points, $\bar{s} = 7.5$, $\bar{y} = 0.18$, $\sum(s - \bar{s})(y - \bar{y}) = 1.10 + 0.90 = 2.00$ (1.10 from A and 0.90 from B), $\sum(s - \bar{s})^2 = 210$, and $\sum(y - \bar{y})^2 = 0.024$. Pooled correlation is $2.00/\sqrt{210 \times 0.024} = 0.891$. Decompose the covariance using sample-count divisors: within-date terms divide by $N = 4$, and the pooled term by $NT = 8$. Date A's within-date covariance is $0.10/4 = 0.025$; B's is -0.025; their average is 0. The covariance between date means is $\frac{1}{2}[(2.5 - 7.5)(0.13 - 0.18) + (12.5 - 7.5)(0.23 - 0.18)] = \frac{1}{2}(0.25 + 0.25) = 0.25$. The pooled covariance, $2.00/8 = 0.25$, comes entirely from the second term: date B has both a higher signal level and a higher return level.

8.5 Quintile long-short returns and turnover

Worked example: Long-Short Returns, Turnover, and Costs

There are $N = 10$ assets over two months. Month 1 signal is $(5, 3, 9, 1, 7, 2, 10, 4, 8, 6)$ and next-period return is $(0.02, 0.01, 0.05, -0.01, 0.03, 0.00, 0.06, 0.01, 0.04, 0.02)$. The signals are their own ranks. With $q = 5$, bucket numbers are $(3, 2, 5, 1, 4, 1, 5, 2, 4, 3)$. Bucket 5 (signals 9 and 10) contains assets 3 and 7 (one-based positions 3 and 7, meaning the third and seventh assets), with returns 0.05 and 0.06 and mean 0.055. Bucket 1 (signals 1 and 2) has returns -0.01 and 0.00 and mean -0.005. The long-short return is 0.060. Month 2 signal is (5, 4, 9, 1, 6, 3, 10, 2, 7, 8) and returns are $(0.01, 0.02, 0.03, -0.02, 0.02, 0.01, 0.03, 0.00, 0.04, 0.05)$. Bucket 5 (signals 9 and 10) has returns 0.03 and 0.03, mean 0.03; bucket 1 (signals 1 and 2) has returns -0.02 and 0.00, mean -0.01. The long-short return is 0.040. Each asset in a leg has weight ± 0.5. The bucket 5 membership is unchanged between months; bucket 1 changes from assets 4 and 6 to assets 4 and 8. Target-weight turnover is $|0 - (-0.5)| + |-0.5 - 0| = 1.0$; membership turnover is 0 for bucket 5 and 0.5 for bucket 1. At a cost of 10 bp per unit of turnover, month 2 net long-short return is $0.040 - 0.0010 = 0.039$.

8.6 The return on a z/N portfolio equals IC $\times \sigma_r$

Five assets: $s = (2, 4, 6, 8, 10)$, $y = (0.01, 0.00, 0.03, 0.02, 0.04)$. Then $\bar{s} = 6$, $\sigma_s = \sqrt{40/5} = 2.828$, and $z = (-1.414, -0.707, 0, 0.707, 1.414)$. The portfolio return is $R_p = \frac{1}{5}\sum z_i y_i = \frac{1}{5}(-0.01414 + 0 + 0 + 0.01414 + 0.05657) = 0.011314$. Pearson IC $= \sum(s - \bar{s})(y - \bar{y})/\sqrt{\sum(s - \bar{s})^2 \sum(y - \bar{y})^2} = 0.16/\sqrt{40 \times 0.001} = 0.8$; $\sigma_y = \sqrt{0.001/5} = 0.014142$; and IC $\times \sigma_y = 0.011314$. The two values match.

8.7 A three-period Fama–MacBeth calculation

Three dates, four assets. Date 1: $s = (1, 2, 3, 4)$, $y = (0.01, 0.02, 0.04, 0.05)$, $\sum(s-\bar{s})(y-\bar{y}) = 0.07$, and $\sum(s-\bar{s})^2 = 5$, so $\hat{b}_1 = 0.014$. Date 2: $s = (4, 3, 2, 1)$, $y = (0.02, 0.02, 0.01, 0.00)$, giving $0.035/5 = 0.007$. Date 3: $s = (1, 3, 2, 4)$, $y = (0.00, 0.03, 0.02, 0.04)$, giving $0.065/5 = 0.013$. The mean is $\bar{b} = 0.011333$. Deviations are $(0.002667, -0.004333, 0.001667)$, their squared sum is 2.867×10^{-5}, $s_b = \sqrt{2.867 \times 10^{-5}/2} = 0.003786$, se $= 0.003786/\sqrt{3} = 0.002186$, and $t = 5.18$. With only three periods, the t statistic has 2 degrees of freedom; these numbers illustrate the mechanics only and are not for inference.

8.8 Effective sample size with overlapping labels

For $\rho = 0.85$ and $h = 3$, VIF $= 2.615$. With $T = 120$, $n_{\text{eff}} = 45.9$, and a naive $t = 3.0$ adjusts to $3.0/\sqrt{2.615} = 1.86$. At the extreme $\rho = 1$, $n_{\text{eff}} = T/h = 40$.

9. Python implementation

The core is in `factor_research.py`, and the monthly momentum window is in `factor_signals.py`. Distinguish three time dependencies. Same-date ranks, percentiles, and buckets use only row t. Formation-period features such as `formation_return` use the permitted historical window before the cutoff; changes to future inputs must not rewrite past signals. The evaluation labels `lead_return` and `forward_label` deliberately use future returns after the fact. They must never enter the feature set available at signal time. Future-dependent labels define what is being measured; they do not authorize forward-looking predictors.

Function	Input	Output and edge cases

Function	Input	Output and edge cases
`rank_cross_section`, `percentile_cross_section`	One-dimensional numeric vector (`NaN` means missing)	Average ranks/percentiles; an empty cross-section is all NaN; infinities, booleans, text, `None`, and complex numbers raise an error
`quantile_bucket`	Vector and bucket count $q \geq 2$	Bucket numbers 1..q; ties stay together; all NaN if valid item count is below q
`factor_rank`	Date-by-asset panel	Percentiles by row; rows with fewer than `min_names` are all NaN
`lead_return`, `forward_label`	Return panel with a continuous monthly `PeriodIndex`	Single-period return at lead k; compounded labels controlled by `lag` and `horizon`; missing calendar months in the input index raise an error, while labels beyond the end of the sample or labels requiring a missing return are NaN
`information_coefficient`, `ic_series`	Signals and labels	per-date Pearson/Spearman; ranks within valid pairs; NaN for fewer than `min_obs` or a constant input
`ic_summary`, `newey_west_variance_of_mean`	IC series and lag count	`ic_summary` returns the mean, standard deviation, ICIR, naive and NW t, n_{eff}, and hit rate; it first drops and counts NaNs. The NW helper returns the variance of the mean and raises an error if called directly with NaN. Lags count positions in the supplied valid array; check for internal calendar gaps first.
`ic_decay`	Signals, returns, and a set of leads	Mean IC and t for each lead
`bucket_returns`	Signals, labels, bucket count, and optional weight panel	Mean return by bucket and `LS`; missing or nonpositive weights are excluded
`quantile_long_short_weights`, `signal_proportional_weights`	Signals	Buy the top bucket and sell the bottom bucket (one unit per leg); demeaned signal-proportional weights (gross exposure can be specified)
`target_turnover`, `bucket_membership_turnover`	Weight panel; signals and bucket numbers	$\sum \lvert\Delta w\rvert$; fraction of bucket members replaced
`fama_macbeth`	Labels and one or more signal panels	Per-period slopes, mean, naive and NW standard errors; dates with too few observations or rank-deficient design matrices are skipped and listed

Execution assumption for monthly labels. In the one-month example (`lead_return` with $k = 1$, or `forward_label` with `horizon=1, lag=0`), the monthly return in row $t + 1$ is attached to row t. Other settings select or compound the future interval specified by the interface. A monthly return starts at the prior month-end close. If the actual trade executes at the first tradable time in the next month, the first part of that monthly return has already occurred before execution. With month-end data only, this label is a research proxy under a hypothetical month-end execution assumption, not an executable return. If open or execution prices are available, start the label at the execution price. `lag` is a whole-month execution delay, not an intraday execution model. Interpret the monthly results in Chapters 22–28 this way (the French industry example is also a revised aggregate return series, not an execution series).

The tie, missing-value, and ranking rules in one place: (1) use average ranks for ties; do not break them by column order; (2) drop NaN before ranking; (3) calculate Rank IC ranks within the pairs for which both signal and label are valid; (4) return NaN rather than 0 when a date has a constant input or fewer than three pairs; (5) do not use `shift` across a missing month in a return panel; and (6) calculate quantile buckets separately by date, not from full-panel quantiles.

Code block 1: verify ranks, percentiles, buckets, and hand-calculated values (Sections 8.1–8.2).

```
import numpy as np

from quant_course.factor_research import percentile_cross_section, quantile_bucket,
    ↪ rank_cross_section

x = np.array([3, 1, np.nan, 3, 2, 7, 5, 5])
np.testing.assert_allclose(rank_cross_section(x), [3.5, 1, np.nan, 3.5, 2, 7, 5.5,
    ↪ 5.5])
np.testing.assert_allclose(percentile_cross_section(x), [5 / 12, 0, np.nan, 5 / 12, 1
    ↪  / 6, 1, 0.75, 0.75])
np.testing.assert_array_equal(quantile_bucket(np.arange(1.0, 11.0), 5), [1, 1, 2, 2,
    ↪ 3, 3, 4, 4, 5, 5])
np.testing.assert_array_equal(quantile_bucket([1, 2, 2, 2, 5, 6, 7, 8], 4), [1, 2, 2,
    ↪  2, 3, 3, 4, 4])

# the bucket of a tied value does not depend on the order of the columns
shuffled = quantile_bucket([7, 2, 5, 2, 8, 1, 2, 6], 4)
np.testing.assert_array_equal(shuffled, [4, 2, 3, 2, 4, 1, 2, 3])
print(rank_cross_section(x))
```

Code block 2: Pearson IC and Rank IC (Section 8.3), pooled correlation and per-date IC (Section 8.4).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import ic_series, information_coefficient

s = np.array([1, 2, 3, 4, 5, 20.0]); y = np.array([0.02, 0.01, 0.04, 0.03, 0.06,
    ↪ -0.05])
assert abs(information_coefficient(s, y, method="pearson") - (-0.79648)) < 1e-5
assert abs(information_coefficient(s, y, method="spearman") - 1 / 35) < 1e-12       #
    ↪  1 - 6*34/(6*35)

idx = pd.period_range("2020-01", periods=2, freq="M")
signal = pd.DataFrame([[1, 2, 3, 4.0], [11, 12, 13, 14.0]], index=idx, columns=list("
    ↪ abcd"))
label = pd.DataFrame([[0.10, 0.12, 0.14, 0.16], [0.26, 0.24, 0.22, 0.20]], index=idx,
    ↪  columns=list("abcd"))
per_date = ic_series(signal, label, method="pearson")["ic"]
pooled = np.corrcoef(signal.to_numpy().ravel(), label.to_numpy().ravel())[0, 1]
np.testing.assert_allclose(per_date, [1.0, -1.0])
assert abs(per_date.mean()) < 1e-12 and abs(pooled - 0.89087) < 1e-5
print(per_date.round(3).tolist(), round(pooled, 4))
```

Code block 3: quintile long-short, weights, turnover, and costs (Section 8.5).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import (
    bucket_membership_turnover, bucket_returns, quantile_long_short_weights,
    ↪ target_turnover,
)

idx = pd.period_range("2020-01", periods=2, freq="M")
cols = list("abcdefghij")
signal = pd.DataFrame([[5, 3, 9, 1, 7, 2, 10, 4, 8, 6.0], [5, 4, 9, 1, 6, 3, 10, 2,
    ↪ 7, 8.0]], index=idx, columns=cols)
label = pd.DataFrame([[.02, .01, .05, -.01, .03, .00, .06, .01, .04, .02],
                      [.01, .02, .03, -.02, .02, .01, .03, .00, .04, .05]], index=idx
    ↪ , columns=cols)
buckets = bucket_returns(signal, label, 5)
np.testing.assert_allclose(buckets["LS"], [0.060, 0.040])
w = quantile_long_short_weights(signal, 5)
assert w.sum(axis=1).abs().max() < 1e-12 and np.allclose(w.abs().sum(axis=1), 2.0)
turnover = target_turnover(w)
assert abs(turnover.iloc[1] - 1.0) < 1e-12
np.testing.assert_allclose(bucket_membership_turnover(signal, 5).iloc[1], 0.0)
    ↪       # top bucket unchanged
np.testing.assert_allclose(bucket_membership_turnover(signal, 5, bucket=1).iloc[1],
    ↪ 0.5)   # bottom bucket: 1 of 2 replaced
net_second = buckets["LS"].iloc[1] - 0.0010 * turnover.iloc[1]
    ↪       # 10 bp per unit of weight traded
assert abs(net_second - 0.039) < 1e-12
print(buckets.round(4))
```

Code block 4: the z/N identity (Section 8.6) and the three-date hand calculation for Fama–MacBeth (Section 8.7).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import fama_macbeth

s = np.array([2, 4, 6, 8, 10.0]); y = np.array([0.01, 0.00, 0.03, 0.02, 0.04])
z = (s - s.mean()) / s.std(ddof=0)
assert abs((z * y).sum() / 5 - np.corrcoef(s, y)[0, 1] * y.std(ddof=0)) < 1e-15
    ↪       # R_p = IC * sigma_y
assert abs(np.corrcoef(s, y)[0, 1] - 0.8) < 1e-12

idx = pd.period_range("2020-01", periods=3, freq="M")
cols = list("abcd")
signal = pd.DataFrame([[1, 2, 3, 4.0], [4, 3, 2, 1.0], [1, 3, 2, 4.0]], index=idx,
    ↪ columns=cols)
label = pd.DataFrame([[.01, .02, .04, .05], [.02, .02, .01, .00], [.00, .03, .02,
    ↪ .04]], index=idx, columns=cols)
fm = fama_macbeth(label, {"signal": signal}, min_extra_obs=0)
np.testing.assert_allclose(fm.slopes["signal"], [0.014, 0.007, 0.013])
row = fm.summary.loc["signal"]
assert abs(row["mean"] - 0.011333333) < 1e-8 and abs(row["se_naive"] - 0.0021858) < 1
    ↪ e-6 and abs(row["t_naive"] - 5.185) < 1e-3
print(fm.slopes.round(4).T)
```

Code block 5: label construction and "change the future without changing the past" (`signal_time` and label interval are separate).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import factor_rank, forward_label, lead_return
from quant_course.factor_signals import formation_return

rng = np.random.default_rng(3)
idx = pd.period_range("2015-01", periods=40, freq="M")
returns = pd.DataFrame(rng.normal(0.01, 0.05, (40, 12)), index=idx, columns=[f"S{i:02
    ↪ d}" for i in range(12)])
signal = factor_rank(formation_return(returns, window=11, skip=1))        # 12-1
    ↪ percentile signal, row s uses months <= s-1
label = forward_label(returns, horizon=1)                                 # label[s]
    ↪ = return of month s+1
np.testing.assert_allclose(label.iloc[0], returns.iloc[1])
np.testing.assert_allclose(lead_return(returns, 2).iloc[0], returns.iloc[2])
assert label.iloc[-1].isna().all()                                        # no label
    ↪ until the future arrives

shocked = returns.copy(); shocked.iloc[30:] = 0.5                         # change
    ↪ months from 30 on
signal2 = factor_rank(formation_return(shocked, window=11, skip=1))
pd.testing.assert_frame_equal(signal.iloc[:31], signal2.iloc[:31])        # signals
    ↪ up to month 30 cannot see months >= 30
print(signal.iloc[11:13].round(2))
```

For Code block 5, changing returns from month 30 onward leaves signal rows 0 through 30 unchanged. Row 30 uses months 19 through 29 and skips month 30, so the 12-1 signal excludes that month's return. This verifies that the signal has not been shifted one month too far forward or backward.

Default offline entrypoint. From the root of the bundled I3 reader package, run:

```
.venv/bin/python run_notebook.py notebooks/ch23_cross_sectional_research.ipynb
```

The default is `RUN_FRENCH=False`. The notebook demonstrates ranks and ties, IC, pooled correlation, long-short returns and turnover, z/N, FM, a constructed panel of 300 assets by 180 months, a date-level example, and analytical VIFs for $h = 1, 3, 6, 12$. It does not run the 600-replication overlapping-label simulation from Section 10. It creates only a chapter-level `RUN_DIR` ; it does not write CSV/PNG files or generate the historical figure gallery. The runner creates `runs/execution_ch23_cross_sectional_research_<uuid>/` with an execution copy and status. The historical full CLI `code/ch23_cross_sectional_research.py` includes French data and overlap simulations and is not the default offline command. The historical seeds and results below are carried over from the existing experiment; they do not need to be rerun. CLI seeds: constructed panel 20261023; pooled-correlation example 20261024; overlapping-label simulations 20261025 ($\rho = 0.85$) and 20261026 ($\rho = 0$). Data are monthly; `signal_time` is month-end; labels are returns in later months; the French snapshot is revised history, not point-in-time data.

10. Reading the figures

Figure 23-1: IC decay and decile returns in a constructed panel. The constructed panel has $N = 300$ assets over 180 months. The signal is a heavy-tailed AR(1) process with autocorrelation 0.85. Next-period return is β·signal ($\beta = 0.004$), plus a market component and heavy-tailed noise. The left plot shows average Rank IC by lead k (the single-period return in month k after the signal month-end), with error bars at ± 2 standard errors; the orange dashed line starts at the first sample Rank IC and multiplies it by 0.85^{k-1}. It borrows the Pearson-IC decay shape in the linear model of Section 6.7 as an order-of-magnitude reference, not an exact decay theorem for Rank IC in a heavy-tailed finite panel. The measured values are near this reference curve, slightly higher at long horizons but within the error bars. The right plot shows equal-weighted average monthly return by decile, which rises roughly monotonically from bucket 1 to bucket 10. Constructed data; mechanism demonstration.

Figure 23-2: Pooled correlation and per-date IC. In this construction, the signal equals "within-date noise" plus a "date-level component." The date-level component has a 0.5 correlation with next-month market returns; returns contain only the market component and noise unrelated to the signal. Each point in the left plot is a month, with next-month market return (%) on the horizontal axis and the cross-sectional mean signal for that month on the vertical axis; they move together. The right plot is a histogram of per-date IC over 120 months. The black line marks the mean, 0.005, and the orange dashed line marks pooled correlation, 0.256. The pooled correlation lies far outside the IC distribution because it measures date-level co-movement, not assets' ranking skill.

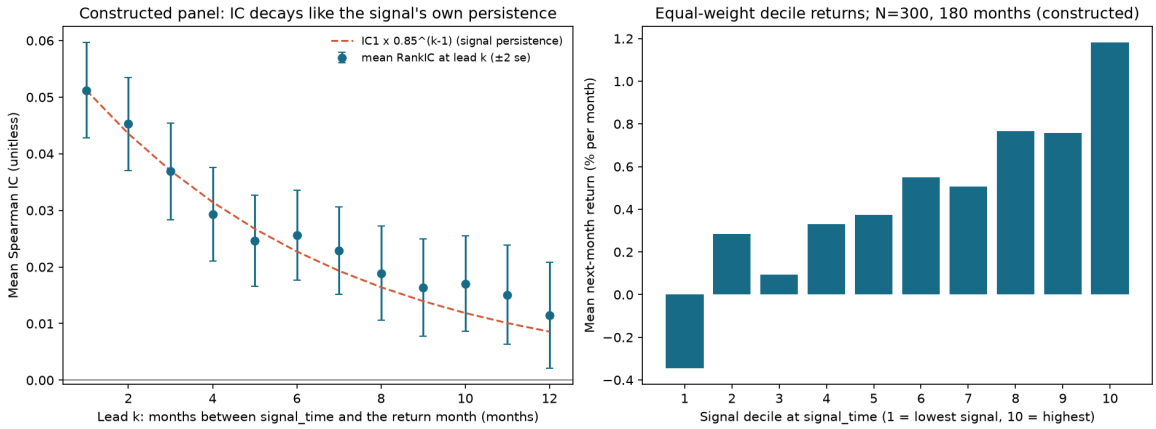


Figure 23-1: IC decay and decile returns in a constructed panel

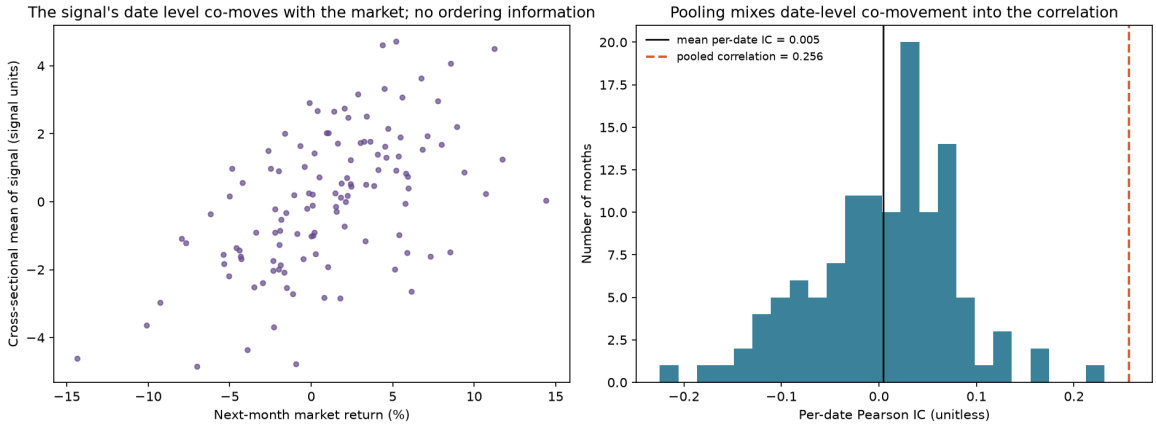


Figure 23-2: Pooled correlation versus per-date IC

Figure 23-3: The size of three t statistics with overlapping labels. In a null world where returns and signals are independent, the construction has 120 months, 100 assets, signal autocorrelation 0.85, and 600 replications. The left plot shows the standard deviation of t statistics; the dashed line at 1 is a standard-normal/asymptotic reference. A finite-sample Student statistic need not have standard deviation exactly 1. The right plot shows the actual rejection rate for a nominal 5% test. The three bars represent the naive test (treating monthly IC as independent), Newey–West with lag $h-1$, and a sample that takes every hth month (nonoverlapping, using the t-distribution critical value). At $h = 12$, the naive t statistic has standard deviation 2.7 and a 45% rejection rate. Newey–West reduces these to 1.38 and 15%, still above 5%. The nonoverlapping sample restores a rate near 5%, at the cost of using only T/h observations.

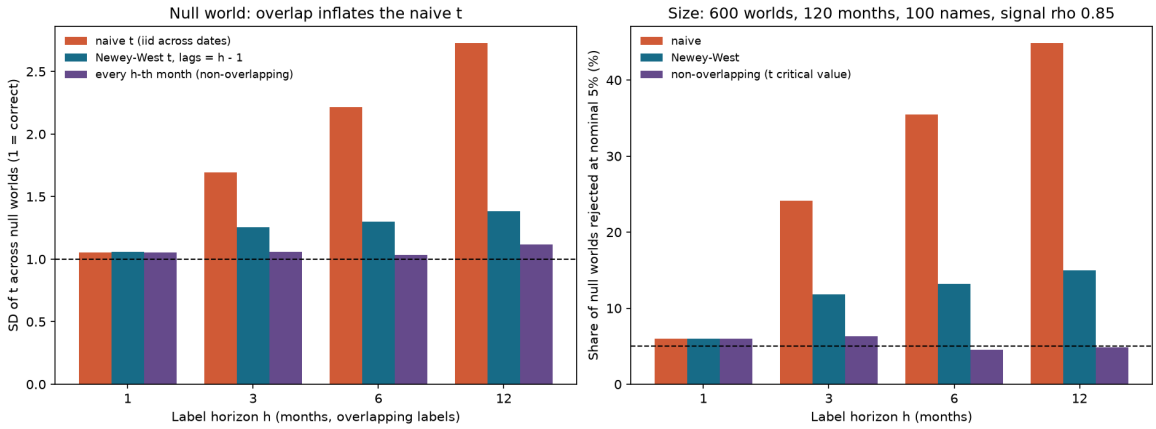


Figure 23-3: Effect of overlapping labels on the t statistic

Figure 23-4: Rank IC and decay for a 12-1 signal on the 10 French industries. The left plot shows a rolling 120-month average Rank IC (horizontal axis: signal month-end); the orange line is the full-sample mean of 0.061. The right plot shows mean Rank IC at each lead, with error bars at ±2 Newey–West standard errors (lag 0). The rolling IC varies widely by period, from near zero to above 0.12. Average IC is positive for the first 6 months, turns negative starting in month 9, and reaches -0.028 ($t = -2.37$) at month 12. But that negative value should not be treated as a significant reversal when selecting among 12 leads: the two-sided Bonferroni threshold for $M = 12$ is $\Phi^{-1}(1 - 0.05/(2 \times 12)) \approx 2.865$. The sample consists of 10 revised industry portfolios, not executable trades.

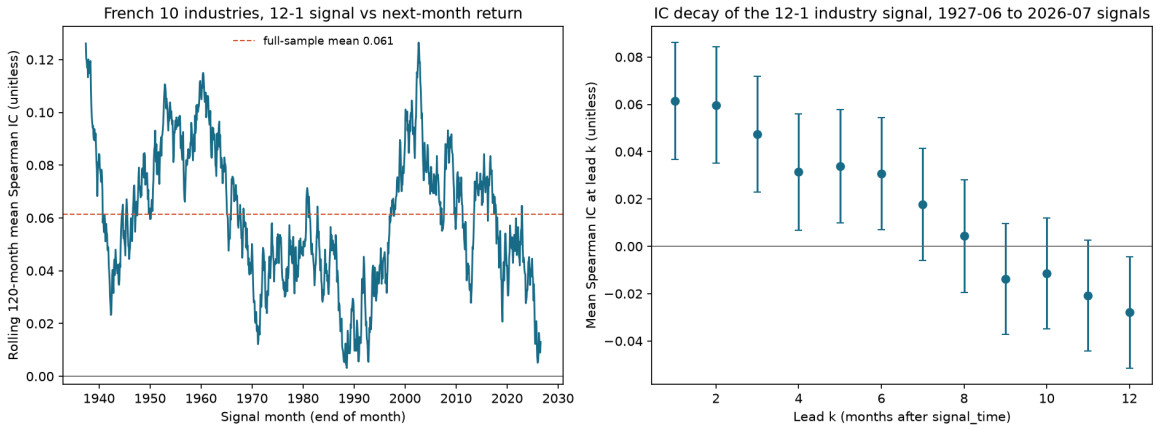


Figure 23-4: Rolling Rank IC and IC decay for a French-industry 12-1 signal

11. Historical market example

Data and definitions. Use the same frozen value-weighted monthly returns for the French 10 industry portfolios as in Chapter 22 (ZIP SHA-256 `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5`; processed CSV SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`; 1,202 months from 1926-07 through 2026-08, in decimal simple returns per month; source and license are documented in Chapter 22 and `data/manifests/french_industries_2026-10-05.`

`json`). Signal $s_{i,t}$: at month-end t, the compounded return over the prior 11 months from $t-11$ through $t-1$ (12-1, excluding the current month). Label: the simple return for the next month, $t+1$. The cross-section has $N = 10$ industries; 1,190 months have both signal and label (1927-06 through 2026-07). This previews Chapter 25; here it demonstrates cross-sectional tools and is not a conclusion about momentum.

per-date Rank IC. Full-sample mean Rank IC is 0.0615 (standard deviation 0.425, hit rate 55.5%, $t = 4.99$, Newey–West lag 0, $T = 1190$). By subperiod: 1927-06 through 1963-12, 0.0868 ($t = 4.41$, 439 months); 1964-01 through 1999-12, 0.0505 ($t = 2.45$, 432 months); and 2000-01 through 2026-07, 0.0414 ($t = 1.70$, 319 months). The last period is exactly the holdout evaluation from Chapter 22, with the same $t = 1.70$. The standard deviation of monthly IC is large, 0.425, because each cross-section has only 10 assets. The approximate sampling standard error of monthly Rank IC is $1/\sqrt{9} = 0.33$, so it is meaningful only after aggregating many months.

IC decay. Mean Rank IC for leads 1 through 12 is 0.0615, 0.0597, 0.0474, 0.0315, 0.0338, 0.0307, 0.0178, 0.0044, −0.0137, −0.0114, −0.0208, and −0.0279. The corresponding t statistics are 4.99, 4.84, 3.88, 2.56, 2.82, 2.59, 1.50, 0.37, −1.17, −0.98, −1.77, and −2.37. In this sample, mean IC is positive for the first six leads, while later point estimates are near zero or negative. These are sample associations in the same revised history, not a general "effective holding period." Choosing among 12 leads is a multiple-comparison problem. The reported t statistics all use HAC lag 0, whose independent-date/calibration assumption still needs to be checked. Do not generalize the decay relationship from the stable AR linear model in Section 6.7 to real signals.

Quintile portfolios (two industries per bucket). Average next-month returns for buckets 1 through 5 are 0.759%, 0.890%, 0.981%, 1.101%, and 1.278% per month, increasing monotonically. The mean long-short return (bucket 5 minus bucket 1, with two industries equally weighted in each leg) is 0.519% per month, with $t = 3.64$ when months are treated as independent. Mean target-weight turnover is 0.904 (two assets per leg, gross exposure 2; on average, about 45% of gross exposure is replaced each month). The mean membership turnover of bucket 5 is 22.9%.

Fama–MacBeth. Standardize the signal within each date using the population-form standard deviation and fit $y_{i,t} = a_t + b_t z_{i,t} + e_{i,t}$ each month, regressing the label on the standardized signal with an intercept. The average slope is $\bar{b} = 0.1686\%$ per month per one standard deviation of signal, with Newey–West (lag 0) $t = 3.35$. The mean intercept is 1.00%, the industry's average monthly return. The identity $\hat{b}_t = \mathrm{IC}_t \sigma_{y,t}$ holds across all 1,190 months with maximum absolute error below 10^{-15}. The two t statistics differ (4.99 for IC and 3.35 for FM) because FM gives more weight to months with higher $\sigma_{y,t}$.

What this example shows and does not show. It shows that one signal can be measured through per-date IC, quantile portfolios, and FM slopes, with all three giving a positive direction but different significance; that the IC series varies widely across periods; and that a decay curve describes lead associations in this sample but does not directly set a tradable horizon. It does not show that the finding is tradable: 10 industry portfolios are revised published returns, not execution prices, and Chapter 22 only used assumed costs for illustration. Nor does it show that inference is robust: the cross-section has only 10 assets, subperiod t statistics differ, and the summary over 12 leads involves multiple testing. The same data have been used repeatedly in Chapter 22, so do not treat this t statistic as independent evidence.

12. Common mistakes

Mistake 1: Treating a correlation pooled across dates as IC. In the two-date example in Section 8.4, per-date IC is +1 and −1 (mean 0), while pooled correlation is 0.891. In the 120-month constructed example in Figure 23-2, pooled correlation is 0.256 while mean per-date IC is 0.005.

```
import numpy as np
import pandas as pd

from quant_course.factor_research import ic_series, ic_summary

rng = np.random.default_rng(5)
months, names = 120, 200
level = rng.standard_normal(months)                                    # date-level
    ↪ driver
signal = pd.DataFrame(rng.standard_normal((months, names)) + 2 * level[:, None],
                      index=pd.period_range("2010-01", periods=months, freq="M"))
label = pd.DataFrame(0.045 * level[:, None] + 0.08 * rng.standard_normal((months,
    ↪ names)), index=signal.index)
pooled = np.corrcoef(signal.to_numpy().ravel(), label.to_numpy().ravel())[0, 1]
per_date = ic_summary(ic_series(signal, label, method="pearson")["ic"])
assert pooled > 0.2 and abs(per_date.mean) < 0.02 and abs(per_date.t_naive) < 3     #
    ↪  ordering has no information
print(round(pooled, 3), round(per_date.mean, 4), round(per_date.t_naive, 2))
```

Mistake 2: Breaking ties by row order. If you use "ordinal ranking" (`argsort`), assigning four tied signals to buckets depends on column order. Reordering the same data changes the portfolio.

```
import numpy as np

from quant_course.factor_research import quantile_bucket

x = np.array([0.0, 0.0, 0.0, 0.0, 1.0, 2.0, 3.0, 4.0])
average = quantile_bucket(x, 4)
reordered = quantile_bucket(x[::-1], 4)[::-1]
np.testing.assert_array_equal(average, reordered)                      # average-
    ↪ rank rule: order-independent
ordinal = np.argsort(np.argsort(x, kind="stable"), kind="stable")      # ordinal
    ↪ ranks 0..7 break ties by position
ordinal_rev = np.argsort(np.argsort(x[::-1], kind="stable"), kind="stable")[::-1]
bucket_ord = lambda ranks: (4 * (ranks + 0.5) // 8).astype(int) + 1
assert not np.array_equal(bucket_ord(ordinal), bucket_ord(ordinal_rev))  # ordinal
    ↪ rule: depends on column order
print(average.tolist(), bucket_ord(ordinal).tolist(), bucket_ord(ordinal_rev).tolist
    ↪ ())
```

Mistake 3: Ranking the full table before dropping NaN, or filling NaN with zero before ranking. In the first case, ranks include assets that were later dropped. In the second, missing assets are treated as if they had a zero signal. The library functions rank within the valid pairs.

Mistake 4: Shifting the label by one month (look-ahead). Using the same month's return as a prediction label (conceptually $k = 0$, a parameter the current `lead_return` rejects) includes already realized information. The example deliberately pairs the signal directly with same-month `returns` to demonstrate the error. In the example below, the constructed same-month return is correlated 0.7 with the signal, while the next month is entirely unrelated.

```
import numpy as np
import pandas as pd

from quant_course.factor_research import ic_series, ic_summary, lead_return

rng = np.random.default_rng(9)
idx = pd.period_range("2010-01", periods=120, freq="M")
returns = pd.DataFrame(0.08 * rng.standard_normal((120, 100)), index=idx)
signal = returns.mul(0.7).add(0.08 * rng.standard_normal((120, 100)) * 0.714)
    ↪    # built FROM this month's return (a leak)
same_month = ic_summary(ic_series(signal, returns, method="pearson")["ic"])
next_month = ic_summary(ic_series(signal, lead_return(returns, 1), method="pearson")[
    ↪ "ic"])
assert same_month.mean > 0.5 and abs(next_month.t_naive) < 3
print(round(same_month.mean, 3), round(next_month.mean, 4))
```

Mistake 5: Using a naive t statistic with overlapping labels. In a null world with signal autocorrelation 0.85 and $T = 120$, the naive rejection rate is about 24% for $h = 3$ and 45% for $h = 12$ (Figure 23-3). Remedy: use a nonoverlapping sample or at least Newey–West, recognizing that it remains optimistic (rejection rate 12–15%).

Mistake 6: Bucketing on full-sample quantiles. If cut points use the full panel, an early date's bucket depends on future data. `quantile_bucket` calculates separately for each date. Applying `pd.qcut` to an entire long-form panel would make this mistake.

Mistake 7: Using market capitalization or a universe from a later date. The `cap` used for value weights and the stock universe must reflect versions known at `signal_time`. Using market capitalization or delisting status that became available after month-end is look-ahead. The time-field tables in Chapter 24 and Project 4 list each field's `available_at`.

Mistake 8: Reading "significant t" as "profitable return." In the constructed panel, the decile long-short gross return is 1.525% per month with $t = 10.4$, but monthly target-weight turnover is 1.43 at gross exposure 2. At 10 bp per unit, the proxy cost is about 14 bp per month, only a fraction of the gross return. In the historical industry example, a high-turnover variant would consume more than half its gross return (Chapter 22). The cost share depends on signal persistence and must be reported alongside gross return.

13. Applications in quantitative research

Research table for the signal layer. Project 4's (multifactor strategy) research interface can be represented as a long-form table with primary key (`date`, `security_id`): raw feature, within-date percentile, bucket, next-month return, h-month label, industry, and market capitalization. The current implementation uses date-by-asset panels and separate metadata; the teaching long-form example is not an existing export file. Input features, industry, and market capitalization need records showing availability and ingestion before the cutoff. Ex post labels mark the evaluation interval and need not be available on the signal date. All functions in this chapter use this table as their input/output model.

Complete chain on a constructed panel (chapter experiment). The panel has $N = 300$, $T = 180$, and an AR(1) signal with $\rho = 0.85$ and true $\mathrm{IC}_1 \approx 0.050$. Results: mean Pearson IC 0.0501 ($t = 12.1$, hit rate 83.8%); Rank IC 0.0512 ($t = 12.1$). The plotted Rank-IC decay is compared with the reference shape 0.85^{k-1}. Section 6.7 gives a Pearson-IC law for its specified linear model, not an exact Rank-IC law for this heavy-tailed finite panel. Equal-weight decile long-short return is 1.525% per month ($t = 10.4$); equal-weight quintile return is 0.999%; market-cap-weighted quintile return is 0.836% ($t = 4.90$); and signal-proportional return at gross exposure 2 is 1.042% ($t = 12.2$). For the same IC, gross returns differ by more than 20% and t statistics by a factor of 2.5 across the three weighting schemes. This is why a claim that a "factor works" cannot omit the weighting rule. The z/N portfolio's mean return is 0.401% per month, the same as the Fama–MacBeth mean slope of 0.4013% (identity in Section 6.4). After adding a pure-noise signal, its FM slope has $t = -1.0$ and the true-signal slope is unchanged.

Project 4 constraints. (1) Report per-date IC and Rank IC, IC decay, quantile monotonicity, and turnover for each factor. (2) For overlapping labels, use nonoverlapping observations or report Newey–West as well. (3) Put missing-value and tie rules in the configuration. (4) Put long-short weights, gross exposure, and the cost convention in the configuration. (5) Record every experiment in Chapter 22's ledger. The signal-layer work in this chapter does not establish a strategy's return after costs.

Layer	Current acceptance entrypoint and scope	Use in this chapter

13.1 Project 4 acceptance scope and tradable subspace

Layer	Current acceptance entrypoint and scope	Use in this chapter
A5 signal preparation	`qa/a5/codex_followup_fixes/acceptance.md`: constructed PIT, same-date processing, signal contract; missing real EV-bridge fields must not default to zero	Reads the feature and date contract; does not accept a complete strategy
A6 ledger interface	`qa/a6/codex_resume/acceptance.md`: constructed prices, listing filters, delisting, and ledger integration	Reads execution constraints; filtering after full-universe neutralization still leaves residual exposure
A7-B cost experiment	`projects/project04/cost_execution_lab/ACCEPTED_VERSION.md`: limited teaching cost/execution experiment	Cost sensitivity is a constructed experiment, not real alpha or capacity

A four-asset counterexample shows why filtering is not re-optimization. Industry 1 has weights $(0.3, -0.3)$ on A and B; industry 2 has weights $(0.2, -0.2)$ on C and D. Before filtering, each industry's weights sum to zero, total net exposure is 0, and gross exposure is 1. If B is untradable and set to zero, the resulting weights are $(0.3, 0, 0.2, -0.2)$. Industry 1 exposure is 0.3, net exposure is 0.3, and gross exposure is 0.7. Even if scaled by $1/0.7$ to restore gross exposure to 1, industry 1 and net exposure remain $3/7$, not zero.

A complete Project 4 run must reconstruct or project weights over the assets tradable at that time, then check final nonzero weights for industry exposure, net exposure, gross exposure, single-asset caps, budget/cash, and all other prespecified constraints. If the feasible set is empty, define a fallback; do not claim that filtering preserves neutrality. A6 has already recorded this open interface. This chapter example gives readers a counterexample and checks; it did not change the shared optimizer or run new empirical work. Complete P4 methods, implementation, real data, and trading validation remain in the later project queue.

14. Chapter summary

Cross-sectional research turns "many assets on the same date" into testable quantities: rank within each date (ranks, percentiles, quantile buckets), measure prediction within each date (IC and Rank IC), and then treat dates as the observations for aggregation. Pooled correlation mixes in co-movement in date-level averages. In the specified constructed model, overlapping labels and persistent signals make the IC series autocorrelated; the long-run effective sample size is T/VIF. Other data-generating processes need separate checks. We connect prediction to holding with weights: a signal-proportional portfolio earns IC $\times \sigma_y$, while quantile portfolios depend on

bucket and weighting rules. In the specified model, signal persistence relates to IC decay; actual turnover also depends on weights and the universe. Fama–MacBeth expresses average return per unit of signal with a standard error, but its weighting differs from IC. In the historical French industry data, the same signal has a consistent direction across per-date IC, quantile portfolios, and FM, while significance varies by measure and period. Scope limits: the constructed panel demonstrates a mechanism; a 10-industry cross-section is small; French data are revised history; and this chapter measures costs only through turnover, so it does not draw strategy conclusions.

15. Exercises

23-E01 (conceptual) Why is the signal time month-end and why does the label begin in the following month? List the five time fields that a monthly research table must identify, and state which two require a strict ordering.

23-E02 (by hand) For the date-level cross-section $x = (4, 2, 2, \text{NaN}, 9, 7, 2, 5, 7)$, calculate average ranks, percentiles, and bucket numbers for $q = 3$.

23-E03 (by hand) Five assets have signal $s = (1, 2, 3, 4, 12)$ and next-period return $y = (0.03, 0.01, 0.05, 0.04, -0.06)$. Calculate Pearson IC and Rank IC, and explain the difference.

23-E04 (derivation) Prove that without ties, $\sum(r_i - \bar{r})^2 = n(n^2 - 1)/12$, and use it to derive Spearman's formula $1 - 6\sum d^2/(n(n^2 - 1))$.

23-E05 (by hand) Two dates with three assets each. A: $s = (1, 2, 3)$, $y = (0.01, 0.02, 0.03)$. B: $s = (5, 6, 7)$, $y = (0.07, 0.06, 0.05)$. Calculate per-date IC, its mean, and the pooled correlation; use the law of total covariance to explain which term accounts for the difference.

23-E06 (by hand) Four assets have $s = (1, 3, 5, 7)$ and $y = (0.02, 0.01, 0.05, 0.04)$. Calculate z and verify that $\frac{1}{N}\sum z_i y_i = \text{IC} \times \sigma_y$.

23-E07 (by hand) Two months, eight assets, and $q = 4$. Month 1 signals: $(3, 1, 8, 5, 6, 2, 7, 4)$; month 2 signals: $(3, 2, 8, 1, 6, 5, 7, 4)$. Next-period returns for month 1: $(0.03, 0.00, 0.06, 0.02, 0.04, 0.01, 0.05, 0.02)$; for month 2: $(0.02, -0.01, 0.05, 0.00, 0.03, 0.02, 0.04, 0.01)$. Calculate each month's long-short return, target-weight turnover, and the membership replacement rate in each of the two buckets. If the cost is 10 bp per unit of turnover, what is the net return in month 2?

23-E08 (calculation) Using the long-run approximation in Section 6.8, let signal autocorrelation be $\rho = 0.9$, label horizon $h = 4$, and $T = 240$ months. Calculate VIF, n_{eff}, the adjusted value of a naive $t = 3.1$, and the VIF and adjusted t recovered in population by Newey–West with $h - 1 = 3$ lags.

23-E09 (interpretation) A signal has AR(1) coefficient 0.9. What is its half-life? If a signal's IC half-life is only one month, how would you choose a rebalancing frequency, and why?

23-R01 (pairs and dates; further practice) Redo Section 3.1: give $N_{\text{signal},t}$, $n_{\text{pair},t}$, IC, and T_{valid} for all three dates, and calculate the time-series standard error. Why can you not use ranks from the full signal set or fill February with zero? What does HAC lag 1 mean in the currently compressed series?

23-R03 (model conditions; further practice) Under the specified large-cross-section dependence model in Section 6.8, calculate finite-period VIF and equivalent date count for $\rho = 0.85$ and -0.5 with $T = 120, h = 3$. Explain which conclusions do not generalize to arbitrary IC series.

23-R02 (portfolio and interface; further practice) For Section 7.1, report the raw and standardized FM slopes, the gross exposure of z/N, and the scaling to gross exposure 2. Then, for Section 13.1 after filtering out B, check industry, net, and gross exposure. Which of these steps can establish cost-adjusted alpha?

16. Programming exercises

23-P01 Write a function that takes a date-by-asset panel and returns the average next-month return for five buckets and the long-short return, using `quantile_bucket` and `lead_return`. Verify that the five buckets are monotonic and the long-short return is positive on constructed data. It must handle NaN and ties without error. Acceptance: one assertion for each of the three input types.

23-P02 On a constructed panel, calculate per-date Pearson IC and pooled correlation. Create a case where the signal's date-level component co-moves with the market, then verify that pooled correlation is much larger than mean per-date IC. Acceptance: pooled correlation > 0.2 and absolute mean IC < 0.02.

23-P03 Simulate a null world with an AR(1) signal ($\rho = 0.85$) and independent returns, using $h = 3$ overlapping labels. Compare rejection rates for naive t, Newey–West with lag 2, and a nonoverlapping sample. Acceptance: naive rejection rate is clearly above 5%, while the nonoverlapping rate is around 5%.

17. Challenge exercises

23-C01 (derivation) Under a bivariate normal assumption, derive the expected return of a q-quantile long-short portfolio, $2\,\mathrm{IC}\,\sigma_y\,q\,\varphi(c_q)$, and calculate $q\varphi(c_q)$ for $q = 3, 5, 10, 20$. Discuss the trade-off between long-short return and single-bucket sampling noise as buckets become narrower. Additional prerequisite: the conditional mean of a normal upper tail. Scoring: derivation 40 points, numerical values 30, trade-off discussion 30.

23-C02 (open-ended) Design the cross-sectional research table for the Value factor in Chapter 24. List each column name and source, the `available_at` rule, missing-value and tie rules, label interval, and industry and market-cap treatment. Give at least three tests that would make you conclude that a positive per-date IC is spurious. Scoring: time-field correctness 30 points, missing-value and tie rules 20, test design 30, treatment of overlapping labels and panel dependence 20.

18. Complete answers and reference implementations

23-E01 The chapter uses month-end signals based on information that was available and ingested before the cutoff, with actual execution after the signal. Five time objects: reporting period (`period_end`)/event time; publication and ingestion (`available_at`/`ingested_at`); `signal_time` (the current program uses the first instant of the next month as a strict cutoff); `trade_time`; and label start/end. Inputs must precede the signal cutoff, and `signal_time < trade_time`. Executable returns begin at the actual execution price. The current next-month month-end label is a research proxy under a hypothetical month-end execution assumption and includes a pre-execution return segment. A later calendar month alone does not establish correct execution timing.

23-E02 The valid values are $(4, 2, 2, 9, 7, 2, 5, 7)$, with $n = 8$. Sorted values are $2, 2, 2, 4, 5, 7, 7, 9$: the three 2s occupy ranks 1, 2, and 3 and each receive average rank 2; 4 has rank 4; 5 has rank 5; the two 7s occupy ranks 6 and 7 and each receive 6.5; 9 has rank 8. The ranks, retaining NaN, are $(4, 2, 2, \text{NaN}, 8, 6.5, 2, 5, 6.5)$. Percentiles $(r-1)/7$ are $(0.4286, 0.1429, 0.1429, \text{NaN}, 1, 0.7857, 0.1429, 0.5714, 0.7857)$. For $q = 3$, $\tilde{u} = (r-0.5)/8$: rank 2 gives 0.1875, 4 gives 0.4375, 5 gives 0.5625, 6.5 gives 0.75, and 8 gives 0.9375. The values of $3\tilde{u}$ are 0.5625, 1.3125, 1.6875, 2.25, and 2.8125; bucket numbers are $(2, 1, 1, \text{NaN}, 3, 3, 1, 2, 3)$.

23-E03 $\bar{s} = 4.4$ and $\bar{y} = 0.014$. Pearson IC is -0.865; the outlier signal of 12 corresponds to the lowest return, -0.06. Return ranks are $(3, 2, 5, 4, 1)$ and signal ranks are $(1, 2, 3, 4, 5)$. Thus $d = (-2, 0, -2, 0, 4)$, $\sum d^2 = 24$, and $\rho_S = 1 - 6 \times 24/(5 \times 24) = 1 - 1.2 = -0.2$. Pearson is sensitive to the outlier, which drives nearly all of the result; Rank IC considers only order, and -0.2 shows that the ordering relationship is also weak. If 12 is a real extreme signal, Pearson may capture it better; if it is a data error, Rank IC is more robust.

23-E04 Ranks are a permutation of $1, \ldots, n$, so $\bar{r} = (n+1)/2$ and $\sum r_i^2 = n(n+1)(2n+1)/6$. Therefore, $\sum (r_i - \bar{r})^2 = n(n+1)(2n+1)/6 - n(n+1)^2/4 = n(n+1)[2(2n+1) - 3(n+1)]/12 = n(n+1)(n-1)/12 = n(n^2-1)/12$. For the rest, see Section 6.2: define $d_i = r_i - q_i$, expand $\sum d_i^2$, and use the equal sums of squares for the two permutations to obtain $\sum (r_i - \bar{r})(q_i - \bar{q}) = n(n^2-1)/12 - \frac{1}{2}\sum d_i^2$. Dividing by $n(n^2-1)/12$ gives the result.

23-E05 Date A has s and y moving together, so IC $= +1$. On B, s rises while y falls, so IC $= -1$. The mean is 0. In the pooled sample of six points, $\bar{s} = 4$ and $\bar{y} = 0.04$. For A, deviations are $s - 4 = (-3, -2, -1)$ and $y - 0.04 = (-0.03, -0.02, -0.01)$, with cross-product sum $0.09 + 0.04 + 0.01 = 0.14$. For B, deviations are $s - 4 = (1, 2, 3)$ and $y - 0.04 = (0.03, 0.02, 0.01)$, with cross-product sum $0.03 + 0.04 + 0.03 = 0.10$. Total cross-product is 0.24; divide by $NT = 6$ for pooled covariance 0.04. The sums of squares are $\sum (s - \bar{s})^2 = 28$ and $\sum (y - \bar{y})^2 = 0.0028$. Pooled correlation is $0.24/\sqrt{28 \times 0.0028} = 0.857$. Decomposition: A's within-date covariance is $\sum (s-2)(y-0.02)/3 = 0.02/3 = 0.00667$, and B's is -0.00667, averaging to 0. Covariance between date means is $\frac{1}{2}[(2-4)(0.02-0.04) + (6-4)(0.06-0.04)] = \frac{1}{2}(0.04+0.04) = 0.04$. All pooled covariance comes from the second term: date B has higher signal and return levels.

23-E06 $\bar{s} = 4$, $\sigma_s = \sqrt{(9+1+1+9)/4} = \sqrt{5} = 2.236$, and $z = (-1.342, -0.447, 0.447, 1.342)$. Then $\frac{1}{4}\sum z_i y_i = \frac{1}{4}(-0.02683 - 0.00447 + 0.02236 + 0.05367) = 0.01118$. Pearson IC is 0.7071. $\sigma_y = \sqrt{\frac{1}{4}\sum(y-0.03)^2} = \sqrt{(0.0001+0.0004+0.0004+0.0001)/4} = 0.015811$, so IC $\times\, \sigma_y = 0.011180$. They match.

23-E07 Month 1 has $q = 4$, $n = 8$, and signals equal to ranks. Bucket assignment is $\lfloor 4(2r-1)/16 \rfloor + 1$: ranks 1 and 2 go to bucket 1; ranks 3 and 4 to bucket 2; ranks 5 and 6 to bucket 3; and ranks 7 and 8 to bucket 4. Bucket 4 in month 1 contains signals 7 and 8, assets 3 and 7, with returns 0.06 and 0.05 and mean 0.055. Bucket 1 contains signals 1 and 2, assets 2 and 6, with returns 0.00 and 0.01 and mean 0.005. Long-short return is 0.050. In month 2, bucket 4 has signals 7 and 8 (assets 3 and 7) with returns 0.05 and 0.04 and mean 0.045. Bucket 1 has signals 1 and 2 (assets 4 and 2) with returns 0.00 and −0.01 and mean −0.005. Long-short return is 0.050. Each leg uses weights ±0.5. Bucket 4 membership is unchanged; bucket 1 changes from assets 2 and 6 to assets 2 and 4. Target-weight turnover is $0.5 + 0.5 = 1.0$; membership replacement is 0 for bucket 4 and 0.5 for bucket 1. Month 2 net return is $0.050 - 0.0010 \times 1.0 = 0.049$.

23-E08 VIF $= 1 + 2[0.9 \times \frac{3}{4} + 0.81 \times \frac{2}{4} + 0.729 \times \frac{1}{4}] = 1 + 2(0.675 + 0.405 + 0.18225) = 3.5245$; $n_{\text{eff}} = 240/3.5245 = 68.1$; adjusted $t = 3.1/\sqrt{3.5245} = 1.65$. The population Newey–West value is $\text{VIF}_{\text{NW}} = 1 + 2[0.9 \times (\frac{3}{4})^2 + 0.81 \times (\frac{2}{4})^2 + 0.729 \times (\frac{1}{4})^2] = 1 + 2(0.50625 + 0.2025 + 0.04556) = 2.5086$, giving $t = 3.1/\sqrt{2.5086} = 1.957$. The naive $t = 3.1$ looks highly significant; the adjusted value is about 1.65. Newey–West's 1.96 remains optimistic and sits near the 5% critical value.

23-E09 Under the stable linear construction in Section 6.7, half-life is $\ln 0.5 / \ln 0.9 = 6.58$ months. If IC half-life is one month, forecast power halves within a month. Compare monthly rebalancing with shorter holding periods, while recognizing that turnover and fees may be higher. Decide using cost-adjusted returns under the frozen weighting rule, not IC alone. If half-life is longer (for example, 6.6 months), lower-frequency rebalancing is a candidate. Whether turnover falls and how much gross return is lost must be calculated under the specified weights, holding interval, rank changes, and costs; ρ alone does not guarantee it.

23-R01 Valid signal counts across the three dates are $(4, 4, 4)$; paired counts are $(3, 4, 4)$; IC values are $(0.5, \text{NaN}, -1)$; and there are two valid dates. January must be re-ranked on A, C, and D, giving 0.5. Ranking all signals first and filtering later gives about 0.655. February's constant signal contains no ranking information, so correlation is undefined; filling it with zero changes the estimand. The mean is −0.25, sample standard deviation 1.06066, standard error 0.75, and naive $t = -1/3$. After compressing the array, lag 1 is one observation from January to March; it cannot be described as one month. Two dates do not support a statistical research conclusion.

23-R03 Finite-period VIF is $1 + 2\sum_{j=1}^{2}(1 - j/120)\rho^j(1 - j/3)$. For $\rho = 0.85$, it is 2.59752778 and 120/VIF = 46.1978. For $\rho = -0.5$, it is 0.50277778 and the equivalent date count is 238.6740. The earlier 2.615 and 45.9 are long-run approximations. With negative correlation, VIF is below 1, so do not state that "correlation" always means smaller standard errors or fewer effective dates. These results depend on the specified autocorrelation structure and do not solve the exact finite-cross-section problem with compounded labels and common shocks.

23-R02 Raw FM slopes are ± 0.02; standardized slopes are ± 0.02236; gross exposure of z/N is 0.89443. Multiplying by $\sqrt{5}$ gives gross exposure 2 and returns of ± 0.05. Filtering out B gives industry sums $(0.3, 0)$, net exposure 0.3, and gross exposure 0.7. Scaling by $10/7$ gives industry sums $(3/7, 0)$, net exposure $3/7$, and gross exposure 1. Reconstruct the portfolio in the tradable universe and verify its final constraints. These are associations, gross returns, and constraint calculations; without actual costs and benchmark-risk attribution, none establishes cost-adjusted alpha.

23-P01 reference implementation

```
import numpy as np
import pandas as pd

from quant_course.factor_research import bucket_returns, lead_return


def five_bucket_table(signal: pd.DataFrame, returns: pd.DataFrame) -> pd.Series:
    table = bucket_returns(signal, lead_return(returns, 1), 5).dropna()
    return table.mean()


rng = np.random.default_rng(11)
idx = pd.period_range("2010-01", periods=120, freq="M")
cols = [f"S{i:03d}" for i in range(200)]
signal = pd.DataFrame(rng.standard_normal((120, 200)), index=idx, columns=cols)
returns = 0.004 * signal.shift(1) + pd.DataFrame(0.08 * rng.standard_normal((120,
    ↪ 200)), index=idx, columns=cols)
signal.iloc[5, :20] = np.nan                        # missing values
signal.iloc[7] = np.round(signal.iloc[7], 0)        # many ties on one date
mean = five_bucket_table(signal, returns)
assert mean.loc[5] > mean.loc[1] and mean["LS"] > 0
assert np.all(np.diff(mean.loc[[1, 2, 3, 4, 5]].to_numpy()) > -0.0015)   # weakly
    ↪ increasing within noise
print(mean.round(5).to_dict())
```

23-P02 reference implementation: See the code under Mistake 1 in Section 12, which reports pooled correlation alongside per-date IC. Set the coefficient on `2 * level` to 0 and verify that both values are close to zero.

23-P03 reference implementation

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(13)
months, names, h, rho, worlds = 120, 100, 3, 0.85, 400
naive, nw, non = [], [], []
for _ in range(worlds):
    total = months + h
    z = np.empty((total, names)); z[0] = rng.standard_normal(names)
    for t in range(1, total):
        z[t] = rho * z[t - 1] + np.sqrt(1 - rho**2) * rng.standard_normal(names)
    r = 0.08 * rng.standard_normal((total, names))
    label = sum(r[k:k + months] for k in range(1, h + 1))
```

```
    zs = z[:months] - z[:months].mean(axis=1, keepdims=True); ls = label - label.mean
    ↪ (axis=1, keepdims=True)
    ic = (zs * ls).sum(axis=1) / np.sqrt((zs**2).sum(axis=1) * (ls**2).sum(axis=1))
    naive.append(ic.mean() / (ic.std(ddof=1) / np.sqrt(months)))
    d = ic - ic.mean(); lrv = d @ d / months + 2 * sum((1 - j / h) * (d[j:] @ d[:-j])
    ↪  / months for j in range(1, h))
    nw.append(ic.mean() / np.sqrt(lrv / months))
    sub = ic[::h]; non.append(sub.mean() / (sub.std(ddof=1) / np.sqrt(len(sub))))
reject = lambda a, c=1.96: float((np.abs(a) > c).mean())
assert reject(naive) > 0.15 and reject(nw) > reject(non) and reject(non, stats.t.ppf
    ↪ (0.975, 39)) < 0.09
print(reject(naive), reject(nw), reject(non, stats.t.ppf(0.975, 39)))
```

23-C01 From Section 4, $\mathbb{E}[y \mid z] = \mu_y + \mathrm{IC}\sigma_y z$. The upper-tail bucket is $z > c_q$, with probability $1/q$. Then $\mathbb{E}[z \mid z > c_q] = \int_{c_q}^{\infty} z\varphi(z)dz/(1/q) = q\,\varphi(c_q)$, since $\int_c^{\infty} z\varphi(z)dz = \varphi(c)$. The lower-tail mean is $\mathbb{E}[z \mid z < -c_q] = -q\varphi(c_q)$. Therefore, long-short return is $2\,\mathrm{IC}\sigma_y q\varphi(c_q)$. Values of $q\varphi(c_q)$: for $q = 3$, $c = 0.4307$, $\varphi = 0.3636$, giving 1.091; for $q = 5$, $c = 0.8416$, $\varphi = 0.2800$, giving 1.400; for $q = 10$, $c = 1.2816$, $\varphi = 0.1755$, giving 1.755; for $q = 20$, $c = 1.6449$, $\varphi = 0.1031$, giving 2.063. Expected long-short return rises with q, but each bucket has fewer assets, N/q. The sampling standard error of a bucket mean is approximately $\sigma_y/\sqrt{N/q}$ (conditional variance approximately $\sigma_y^2(1 - \mathrm{IC}^2)$). With $N = 300, q = 20$, each bucket has 15 assets and its mean standard error is about $0.08/\sqrt{15} = 2.1\%$, far above the signal's expected contribution. Finer buckets raise expected long-short return, but also increase single-bucket noise and concentrate holdings in fewer assets. Compare the mean with its standard error to assess the long-short signal-to-noise ratio; noise alone does not determine the direction. The static normal formula contains no signal-transition process, so it cannot show that turnover must rise. Calculate actual turnover from dynamic ranks and weights. Fix q in the research plan. Score using the original three criteria.

23-C02 (scoring guide and reference). Suggested columns: `date` (month-end `signal_time`), `security_id` (permanent identifier), `book_equity`, and `market_cap` (latest version of each with `available_at ≤ signal_time`, using Chapter 13's `select_pit_vintage`); `b_m` (book equity/market cap; set to NaN and record `reason` when equity ≤0); `industry` (classification known at `signal_time`); `rank_pct` (within-date percentile, with a separate column for the within-industry version); `bucket`; `label_1m`; and `label_3m` (with intervals documented). Missing values: retain nonpositive equity and missing market cap as NaN; do not fill with zero. Ties: use average ranks. Three tests for a spurious positive IC: (1) delay `available_at` by one month and see whether IC disappears (timing test); (2) rank within industry and see whether IC remains positive (industry-effect test); (3) remove the largest 5% of assets by market cap and see whether IC remains positive (size-concentration test). A permutation test is another option. Score using the original four criteria.

19. Further reading

- **Must know:** Petersen (2005 NBER working paper), abstract and introduction, then the section containing equations (8) and (9), to understand the assumptions behind Fama–MacBeth standard errors; useful for recognizing when FM t statistics are too large.
- **Should know:** Jegadeesh and Titman (1993), Section I, on overlapping holding-period portfolios and turnover; read in depth in Chapter 25.
- **Should know:** Chapter 22, Section 6, on multiple testing, and Chapter 15 on HAC.
- **Advanced:** Original papers by Fama and MacBeth (1973), Newey and West (1987), and the chapters on IC and the Fundamental Law in Grinold and Kahn. These works have not been read for this chapter; obtain their texts before checking notation and conclusions.
- **Optional:** Any publicly available factor research report; practice identifying when each column in its "research table" becomes available.

20. References

Citation keys and reading scope are listed in `source_notes.md`.

- **CH23_PETERSEN2005:** Petersen, M. A. (2005/2006). "Estimating Standard Errors in Finance Panel Data Sets: Comparing Approaches." NBER Working Paper 11280. Supports the discussion of Fama–MacBeth estimation and standard errors in Sections 5 and 6.6; the 2009 RFS final version has not been read.
- **CH23_JT1993:** Jegadeesh, N., and Titman, S. (1993). "Returns to Buying Winners and Selling Losers." *Journal of Finance* 48(1). Supports the discussion of overlapping holding periods.
- **CH23_SCIPY_RANKDATA:** SciPy 1.18.1 documentation for `scipy.stats.rankdata`, supporting the average-rank definition.
- **FRENCH:** Kenneth R. French Data Library, 10 Industry Portfolios (frozen snapshot).

Chapter 24 — Value Factors

Through line: Turn "cheap relative to fundamentals" into a directionally consistent signal whose numerator and denominator use matching definitions and whose inputs were available at signal_time. Each careless step can create an illusion that a value effect exists. Prerequisites: Chapter 12 (financial statements), Chapter 13 (valuation), and Chapter 23 (cross-sectional research). By the end, you should be able to define the numerator, denominator, direction, and appropriate industry for book-to-market (B/M), earnings yield (E/P), free-cash-flow yield (FCF Yield), and enterprise value to EBITDA (EV/EBITDA); handle negative denominators; remove industry-level differences with same-date, within-industry percentiles; select point-in-time versions using available_at and ingested_at and detect look-ahead; explain value traps; read published HML value-factor returns; and explain why they do not test a per-stock value signal.

1. Chapter Objectives

Readers should be able to answer: At a given month-end, how exactly do we calculate whether a company is "cheap," and which of those numbers were known at that time?

1. For each value ratio, state its numerator, denominator, units, and direction (does a larger ratio mean cheaper or more expensive?), and distinguish yield form from multiple form.
2. Specify clear missing-value rules for negative earnings, negative book equity, zero or negative enterprise value (EV), and nonpositive EBITDA, and explain the source of bias under each rule.
3. Explain why EV and EBITDA are unsuitable for financial firms such as banks and insurers, and why we rank within industry on the same date rather than across the whole market.
4. Use latest_available_values to select point-in-time fundamentals at each month-end from a financial-statement version table containing restatements, and give three counterexamples to using period-end or the current restated value.
5. Reproduce three effects with constructed data: dilution of IC by industry-level differences, a calendar-signal-month IC diagnostic for look-ahead book values, and value-trap concentration in the cheapest decile.
6. Read published monthly HML returns and report their mean, t-statistic, subsamples, and maximum drawdown, while explaining that they describe an aggregate factor return and do not test a per-stock value signal.

Completion criterion: Every value signal can be accompanied by a field table listing its source, period_end, available_at, ingested_at, and units, along with an explanation of the signal month in which each value becomes available.

2. Why Quant Needs This

Chapter 23 introduced cross-sectional tools: given a column of signals known at signal_time, we can calculate period-by-period ICs and quantile portfolios. Value is the first factor for which we must construct that column ourselves, and errors at this stage almost never trigger an error message. A ratio with a negative denominator still computes; a period-end value can still be aligned with month-end; and book-to-market ratios for banks and software companies can still be ranked together. Every mistake produces a plausible-looking number, and some make the signal appear more effective.

The logic is: define the ratios and their direction (Sections 4–5); handle boundaries and matching definitions (Sections 6.1–6.3); answer "when was this number knowable?" (Section 6.4); quantify three errors' effects on IC with a constructed panel (Sections 10–13); and then examine published value-factor returns (Section 11). Chapters 25–28 reuse these conclusions: Chapter 26's quality measures face the same denominator and availability-time problems, and Chapter 28 uses within-industry transformations in preprocessing.

3. Intuition

"Cheap" is a relative statement. If two companies each earn 5 per year, one selling for 50 is cheaper than the other selling for 80. Turning that statement into a signal takes four steps: choose an accounting measure of "how much is earned" (book equity, earnings, or cash flow); choose a price measure of "what it costs" (market capitalization or enterprise value); put the ratio in a consistent direction (larger means cheaper); and rank companies on the same date and within the same industry.

Each step has a trap. When an accounting measure is negative, it usually appears in the numerator of yield forms such as B/M and E/P, but may appear in the denominator of multiple forms such as P/E. A ratio may therefore be negative, but that does not mean "cheaper"; it means the company is losing money or has negative equity. The price measure and accounting measure must refer to the same claim on the company: cash flow available to shareholders is divided by equity market capitalization, while cash flow available to all capital providers is divided by enterprise value. Accounting figures come from financial statements, which may be published weeks or months after the fiscal year ends and may later be restated. Before publication, the market does not know those figures, so the signal cannot know them either. Capital structure and accounting practices also differ across industries. Banks tend to have structurally high book-to-market ratios; ranking them on the same list as software companies ranks industries rather than individual companies.

A simplifying boundary: this chapter uses a constructed panel with only two industries to demonstrate these mechanisms. Real markets have dozens of industries, thousands of stocks, quarterly statements, and many more accounting differences. Constructed data can illustrate mechanisms and magnitudes; it cannot replace research on real data.

3.1 First, Reconcile a Company's Capital Claims

Complete Worked Example: The EV Bridge, Cash-Flow Pairing, and a Net-Cash Counterexample

Suppose the signal date is the end of March 2025. A nonfinancial company has market capitalization of 600, debt of 200, cash of 100, noncontrolling interests of 20, preferred equity of 10, and nonoperating assets of 30; all amounts are in millions of euros. Value the debt and other bridge items consistently with Chapter 13. Keep cash and nonoperating assets separate to avoid deducting them twice. The complete bridge is:

$$V = 600 + 200 - 100 + 20 + 10 - 30 = 700.$$

Each step answers, "How much would it cost to acquire the corresponding capital claim?" The 600 is common-equity market capitalization. Debt, noncontrolling interests, and preferred equity add other claims. Cash and separately identified nonoperating assets are deducted because their value does not need to be supported by operating cash flow. EV is neither revenue nor book total assets. Every bridge item must be available at the same cutoff time. Market capitalization is a market quantity for that month; the accounting bridge items come from versions already available. The values need not originate on the same calendar day, but their respective timestamps and lags must be recorded.

Suppose annual FCFF is 56, after-tax interest is 12, and net borrowing is 4, and this simplified cash-flow bridge has no separate payment adjustment for preferred equity or noncontrolling interests (if such payments exist, adjust for their allocation separately). Cash flow available to common shareholders, FCFE, is $56 - 12 + 4 = 48$. The firm-level yield is $56/700 = 8\%$; the shareholder-level yield is $48/600 = 8\%$. They happen to match because of the chosen numbers; changing net borrowing or the capital structure would change them. Using $56/600 = 9.333\%$ incorrectly mixes the firm cash flow with the bridge multiple $700/600 = 1.1667$; it does not show that the company earns more.

Net-cash counterexample: market capitalization is 100, debt is 20, and cash is 80, with all other bridge items zero. Then $V = 40 < M = 100$. With FCFF of 8, the correctly matched yield is $8/40 = 20\%$; the mismatched $8/100 = 8\%$ understates it. In general,

$$\text{FCFF}/M = (V/M)(\text{FCFF}/V);$$

for positive M, V, the multiplier V/M exceeds 1 exactly when the total net bridge $V - M$ is positive. If EV is nonpositive, this chapter marks the EV ratio missing and calls for the bridge to be checked; it does not label the company "infinitely cheap."

4. Mathematical Foundations

Notation. Let month-end t be signal_time. For asset i, values known at t are: market capitalization $M_{i,t}$ (share price times shares outstanding, in currency units); book equity $B_{i,t}$ (book value attributable to common shareholders, from the most recent available statement, in the same units as M); earnings $E_{i,t}$ (net income attributable to common shareholders); free cash flow $F_{i,t}$; enterprise value $V_{i,t}$; and $\text{EBITDA}_{i,t}$. Unless stated otherwise, accounting measures use the most recent available fiscal-year value. Units: B/M is a stock-to-stock ratio. E/M, F/M, F/V, and EBITDA/V use cumulative flows for a fixed fiscal year; the course displays them as decimals or percentages and always states the annual window. In continuous-time terms, their dimension is year^{-1}. V/EBITDA is a multiple of annual EBITDA, expressed in years. An annual accounting yield is not a monthly investment return, and we do not multiply or divide it by 12 to convert it.

Enterprise-value bridge. Chapter 13 defines $V = M+\text{debt}-\text{cash}+\text{noncontrolling interests}+\text{preferred equity} - \text{nonoperating assets}$, with every item in the same currency and on the same scale. The bridge items must be nonnegative.

Ranking and monotonic transformations. For positive x, the function $x \mapsto 1/x$ is strictly decreasing, so $x_i < x_j \iff 1/x_i > 1/x_j$. Thus the ascending rank of V/EBITDA is exactly the descending rank of EBITDA/V. Reciprocals are strictly decreasing separately on the positive and negative half-lines, but the same ranking conclusion cannot be applied across zero: $-1 < 1$, while $1/(-1) < 1/1$. The reciprocal is undefined at zero. We therefore cannot rank P/E for companies with negative earnings as though they had positive earnings.

Same-date, within-industry percentiles. For the n_g assets with valid values in a given industry on a given date, define percentile $u_i = (\text{rank}_i - 1)/(n_g - 1)$ (Chapter 23's definition, with average ranks for ties). If an industry has fewer than m valid assets, set the percentile to NaN.

Point-in-time (PIT) versions. Each financial-statement version v has a period_end (the end of the reporting period), available_at (when the information first became externally available), and ingested_at (when it entered the local system). Given cutoff c (the final instant of month-end, in UTC), the known set is

$$K(c) = \{v : \ \text{available_at}_v \le c, \ \text{ingested_at}_v \le c\}.$$

Frequency convention: monthly; signal_time is month-end; prices and market capitalization are measured at month-end; accounting measures use fiscal years; and the signal predicts the following month's return.

5. Formal Definitions

Book-to-market ratio (B/M). $\text{BM}_{i,t} = B_{i,t}/M_{i,t}$. This is a yield form: larger means cheaper. When book equity is zero or negative, the Fama–French factor construction includes only companies with positive book equity. This chapter does the same by default and sets B/M to NaN for nonpositive book equity.

Earnings yield (E/P). $E_{i,t}/M_{i,t}$, the reciprocal of the price-to-earnings ratio (P/E). Larger means cheaper. Negative earnings produce a negative yield, not a "cheapest" observation, so Section 6.1 makes an explicit choice about whether to retain negative values for ranking.

Free-cash-flow yield (FCF Yield). Shareholder-level yield: free cash flow to equity (FCFE) divided by market capitalization. Firm-level yield: free cash flow to the firm (FCFF) divided by enterprise value. These definitions must be matched (Section 6.2).

EV/EBITDA. A multiple form; smaller means cheaper. EBITDA is earnings before interest, taxes, depreciation, and amortization. If EBITDA is nonpositive, the multiple is meaningless and is set to NaN.

Industry-relative value. Transform a signal into a percentile within the same industry on the same date, so that companies are compared only with firms having similar accounting structures.

Value trap. An asset that looks cheap based on its ratios but whose expected return is not high and may even be negative, often because fundamentals are deteriorating (earnings are falling or debt is rising) and the price has already declined. This is not a new ratio; it describes a subset of observations for which the "cheap" signal does not hold.

Point-in-time versions, reporting periods, and restatements. period_end is the end of the period to which a financial figure belongs; it is not the date the figure became available. available_at is the earliest time that version could be obtained externally. A restatement is a later update to a figure for the same period. A signal month may use only versions published by that month; later restatements cannot be backfilled. Chapter 3 and Chapter 13 introduced pit_asof_join and select_pit_vintage. This chapter's latest_available_values does something different: for each month-end, it selects the latest version of the latest known reporting period.

Value-factor return (HML). The published High Minus Low factor is HML $= \frac{1}{2}(\text{Small Value} + \text{Big Value}) - \frac{1}{2}(\text{Small Growth} + \text{Big Growth})$, where Value and Growth are value-weighted portfolios formed on B/M, and Small and Big are size groups. This is an aggregate return series, not a per-stock signal.

The difference between a characteristic and a factor return. B/M is a characteristic belonging to company i in month t. HML is a factor return belonging to month t; it does not belong to any one company. The two parts of this chapter address those two objects: the constructed panel demonstrates how to build a characteristic correctly, while the French HML series describes an aggregate return.

6. Derivations

6.1 Ratio Direction, Negative Values, and Missing-Value Rules

Direction. Yield forms (B/M, E/P, and FCF/EV) become cheaper as they rise; multiple forms (EV/EBITDA and P/E) become cheaper as they fall. When both numerator and denominator are positive, ascending multiple ranks correspond to descending ranks of the reciprocal yield (Section 4), but the values, z-scores, and composites are not equivalent. Before combining signals, align their ranking direction. cheapness_percentile(kind="multiple") ranks the negative of the multiple, $-x$, rather than taking a reciprocal and dividing by values close to zero.

Nonpositive accounting benchmark. If $B \leq 0$ (negative equity), B/M is nonpositive. Retaining negative values puts these companies at the "most expensive" end and hides the fact that they may be distressed. Setting them to zero forces records with no valid definition into the ranking, where their position depends on the other observations. Setting them to NaN removes them from the sample. Each rule creates a different bias: using NaN excludes potentially poor companies and can make the value portfolio look cleaner; retaining negative values mixes leverage distress into the ranking. This chapter includes only companies with positive book equity, consistent with the Fama–French construction (database description), and requires the report to state the share excluded.

Negative earnings. If $E < 0$, E/P is negative. Retain its sign in the yield form: a loss-making company has a "negative yield" and ranks toward the expensive end, consistent with "larger means cheaper." But P/E is negative for a loss-making company, so a naive ranking would call it the "cheapest." The multiple form must therefore set a nonpositive denominator to NaN. yield_ratio retains the sign of the numerator; ev_to_ebitda requires both EBITDA and EV to be positive.

Zero or negative market capitalization and enterprise value. Nonpositive market capitalization indicates a data error. Negative enterprise value (cash exceeds market capitalization plus debt) means that the bridge's accounting definitions should be checked; the multiple is meaningless.

6.2 Matching Numerator and Denominator: FCFF and FCFE

Cash flow available to equity holders equals firm cash flow less net payments to creditors: FCFE = FCFF − after-tax interest + net borrowing. FCFF belongs to shareholders and creditors together, so it is divided by $V = M +$ net debt $+ \ldots$. FCFE belongs to shareholders, so it is divided by M. What happens if FCFF is divided by M?

$$\frac{\text{FCFF}}{M} = \frac{V}{M} \cdot \frac{\text{FCFF}}{V} = \left(1 + \frac{\text{net debt} + \text{other bridge items}}{M}\right) \frac{\text{FCFF}}{V}.$$

The identity holds for positive or negative FCFF, but the size of V/M depends on all net bridge items: debt minus cash plus noncontrolling interests and preferred equity minus nonoperating assets. Positive net debt alone is insufficient. For example, with $M = 100$, net debt 20,

nonoperating assets 50, and all other items zero, $V = 70 < M$. For $M, V > 0$, positive total net bridge items give $V/M > 1$; negative net bridge items give a ratio below 1.

For $M, V > 0$, the difference is $FCFF/M - FCFF/V = FCFF(V - M)/(MV)$. The mismatched yield is higher when FCFF and the total net bridge $V - M$ have the same sign, lower when their signs differ, and equal when either is zero. Thus, for positive FCFF, a positive total net bridge raises the mismatched yield; negative FCFF reverses the direction. In Lyra's positive-FCFF example in Section 8.5, $V/M = 750/480 = 1.5625$, so the correct 5.33% becomes a mismatched 8.33%, an amplification of about 56%. With other bridge items and operating definitions held fixed, the mismatch can introduce capital-structure exposure; its direction depends on the complete bridge and the cash-flow sign. See the net-cash counterexample in Section 3.1.

6.3 Why Rank Within Industry: Dilution of IC by Industry-Level Differences

Suppose each company's log book-to-market ratio is $\ell_i = \log(B_i/M_i) = a_{g(i)} + u_i$, where a_g is the industry level and u_i is the within-industry deviation. Assume next-period returns depend only on u_i (economically: within the same industry, cheaper companies earn higher returns), so $y_i = \beta u_i/\sigma_u + \epsilon_i$. There are two equally weighted industries, with level difference $\Delta = a_{\text{banks}} - a_{\text{manufacturing}}$, so $\text{Var}(a) = (\Delta/2)^2$. When we use the full-market ℓ_i as a Pearson signal, its correlation with u_i is

$$\text{Corr}(\ell, u) = \frac{\sigma_u}{\sqrt{\sigma_u^2 + (\Delta/2)^2}},$$

using $\text{Cov}(\ell, u) = \text{Var}(u) = \sigma_u^2$ (industry levels a_g and within-industry deviations u_i are uncorrelated and $\text{E}[u] = 0$) and $\text{Var}(\ell) = \sigma_u^2 + \text{Var}(a)$. Because returns depend only on u_i, the full-market signal's correlation with returns equals the within-industry signal's correlation multiplied by this factor, so it is attenuated. In the constructed panel, $\Delta = 0.9$ (mean log B/M is -0.1 for banks and -1.0 for manufacturing) and $\sigma_u \approx 0.55$ (including the extra displacement of trap companies). The attenuation factor is about $0.55/\sqrt{0.55^2 + 0.45^2} = 0.77$. The constructed experiments in Sections 10 and 13 measure raw-rank IC of 0.0256 and within-industry IC of 0.0360, a ratio of 0.71, on the same order as 0.77. The measured statistic is a rank correlation, and within-industry deviations in the panel are not normally distributed, so this is a comparison of magnitudes. Within-industry ranking removes a_g from the comparison and recovers information in u_i.

6.4 Point-in-Time Selection and Causality

For security identifier s and signal cutoff c, select

$$v^*(s, c) = \arg\max_{v \in K_s(c)} \big(\text{period_end}_v,\ \text{available_at}_v,\ \text{ingested_at}_v\big) \quad \text{(lexicographically)},$$

that is, among known versions, select the latest reporting period, then the latest publication for that period, then the latest ingestion. Properties: (1) Causality: adding a version whose

available_at or ingested_at is later than c does not change $K_s(c)$ and therefore does not change $v^*(s, c)$. The tests verify this by randomly adding future versions. (2) A restatement of an older period does not replace a newer period: if the old period's restatement arrives after the first release of a newer period, the newer period has a later period_end in $K_s(c)$, so v^* remains the newer period. (3) A restatement is used only if it was published and ingested before c; its older reporting period does not make it available earlier. (4) Two rows for the same security, period, and available_at with different values are ambiguous; the function rejects them rather than guessing.

Three common substitutes introduce look-ahead: treat period_end as the availability date (so a December 31 fiscal-year figure is used in a January signal even though the annual report is not published until March); backfill history with the last version available today; or use download time or file-creation time instead of available_at. Sections 8.4 and 12 give numerical examples.

6.5 Diagnosing Look-Ahead IC by Calendar Signal Month

The specified construction puts book-value news g into the look-ahead signal and a same-direction price response κg into the filing-month return; before publication, the PIT signal still uses the old value. A positive covariance loading alone does not guarantee a higher correlation, because correlation also divides by the signal and return standard deviations. For a counterexample, let u, g be uncorrelated with unit variance, the label be $y = u + 0.1g$, the PIT signal be u, and the look-ahead signal be $u + g$. Covariance rises from 1 to 1.1, but

$$\mathrm{Corr}(u, y) = \frac{1}{\sqrt{1.01}} = 0.995037, \qquad \mathrm{Corr}(u + g, y) = \frac{1.1}{\sqrt{2 \times 1.01}} = 0.773957.$$

Thus this chapter's higher look-ahead IC is an observed direction for its specified mechanism, parameters, and sample, not a general inequality or a guarantee for every finite-sample repetition. An audit must compare PIT and look-ahead signals on the same valid pairs, rather than infer a higher correlation from a higher covariance.

Figure 24-2 uses signal month on the horizontal axis and next-month returns as labels. In this construction, disclosure occurs 60–110 days after a December 31 fiscal year-end (late February/March through April). The observed ICs differ for January–March signals and coincide from April onward. Interpret the actual signal/label alignment; do not call the horizontal-axis months disclosure months. Other forms of look-ahead can increase, decrease, or leave IC unchanged. Better historical performance also needs an availability-time audit.

6.6 The Mixture Mean for Value Traps

Suppose a share s of the cheapest decile consists of trap companies, with the rest being normal companies. The portfolio's mean return is a mixture: $\bar{r} = (1-s)\bar{r}_{\text{normal}} + s\,\bar{r}_{\text{trap}}$. Given $\bar{r}$, s, and the mean return of the normal companies after removing traps, we can solve for $\bar{r}_{\text{trap}} = (\bar{r} - (1-s)\bar{r}_{\text{normal}})/s$. In the constructed panel, $s = 44.7\%$; the cheapest decile earns 1.058% per month including traps and 1.361% after removing them. The implied trap mean is 0.683%. The trap share is 4.6 times the overall share of 9.7%: because their prices have already fallen, trap companies have high B/M and cluster at the "cheapest" end. This decomposition requires knowing which companies are traps, which a real researcher does not know. In practice, traps can only be identified in advance using observable fundamentals such as earnings changes, leverage, or cash-flow signs. That is why Chapter 26's quality measures and Chapter 28's factor composite matter.

6.7 HML Portfolio Arithmetic

HML is an arithmetic combination of four value-weighted portfolio returns: the average of two Value portfolios (Small Value and Big Value) minus the average of two Growth portfolios (Small Growth and Big Growth). This averaging balances the Small/Big bucket weights: each side contains half Small and half Big. It does not ensure that the two sides have exactly equal market capitalization or size-factor beta. If the four portfolio returns for a month are 1.4%, 0.8%, 0.4%, 0.6%, then HML $= \frac{1}{2}(1.4+0.8) - \frac{1}{2}(0.4+0.6) = 1.10 - 0.50 = 0.60\%$. Using only Big stocks gives $0.8 - 0.6 = 0.2\%$: this evaluates the value spread among Big stocks only, a different portfolio from the four-leg balanced construction. HML is a zero-cost long-short spread: it is the difference between two long legs and two short legs and does not deduct financing, borrow fees, or trading costs. It is not an investable net return.

7. Financial Interpretation

A value signal measures the distance between price and fundamentals, not company quality. A high B/M means the market places a low price on each unit of book equity. The reason may be that the market expects low cash flows, or that the company is temporarily undervalued. The ratio cannot distinguish these cases. Under a mispricing explanation, a subsequent price recovery requires the market to have been overly pessimistic. That is not a necessary condition for every value premium; risk compensation and proxy measurement may also generate a positive association. Value traps remind us that a low price may reasonably reflect persistently deteriorating fundamentals.

This chapter does not decide whether a value premium exists or is stable. Published HML averaged 0.35% per month from 1926–2026 ($t = 3.43$), but averaged -0.44% per month over the 168 months from 2007–2020 ($t = -1.99$), with a maximum drawdown of about 58%. The same data contain both long-run positive returns and a more than decade-long trough. Fama and French (2015, printed pages 12–13) found that, in US data from 1963–2013, HML added little description of average returns after profitability and investment factors were included. They

also noted that the value premium itself remained large and was still targeted by investment managers. So a claim about whether "the value factor is useful" must specify the sample, model, and question.

Risk and behavioral explanations are contested; this chapter does not adjudicate them. One view treats the value premium as compensation for some form of risk; another treats it as an underreaction of prices to information. The original sources have not been read for this chapter (see the source notes), so the text does not reproduce either side's argument. The Quant requirement is the same under either view: the signal must be constructible at signal_time, costs must be deducted, and failure modes (value traps, industry concentration, and factor crowding) must be discussed.

Industry definitions are an economic issue, not only a technical adjustment. For banks, "debt" is an operating input and interest is a cost, so EV and EBITDA are not meaningful. Bank book equity is constrained by regulatory capital requirements, and its B/M level is not comparable to that of a software company. Within-industry percentiles answer "Is it cheap relative to its peers?" rather than "Is it cheap across the whole market?" The tradeoff is that this discards information about value differences between industries: if the entire banking industry is cheap, within-industry ranks will not show it. Chapter 28 makes this an explicit choice through industry neutralization and industry-exposure constraints.

When we compare matters more than which formula we use. Two researchers may use the same B/M formula, but if one uses a new book value before the annual report is published, some signals formed before annual-report publication have higher IC against next-month returns in this chapter's specified construction. The formulas match; the data timestamps do not. Reproducible value research must publish both the formula and the timing rule.

7.1 From "Cheap" to a Testable Economic Hypothesis

A ratio describes the relation between price and an accounting anchor; a hypothesis is also needed to predict return direction. The risk-compensation hypothesis says that the cheap group may face more severe operating decline or financing pressure, so investors require a higher return. It predicts not only average returns but also losses in bad states. The mispricing hypothesis says that the market may extrapolate bad news too far; later cash flows may turn out better than expected and prices may recover. It requires distinguishing expectations from later information in advance, rather than labeling winners "undervalued" after the fact. The proxy-measurement hypothesis says that high B/M may proxy for capital-intensive industries, leverage, or accounting treatment of intangible assets, so the apparent value effect may come from these differences.

These are mechanisms to test, not literature conclusions verified by reading the original papers for this chapter. Chapter 22 can preregister three questions: Are cheap-portfolio returns concentrated in compensation for bad states? Do returns vary with observable errors in cash-flow expectations? How much association remains after industry and size controls? Controls can also remove a genuine economic channel, so a "cleaner residual signal" cannot be assumed from its name. Chapter 23's IC answers only whether ranks and next-period labels are associated in the given sample; a positive IC or positive gross spread alone cannot identify a mechanism or establish net alpha (an intercept relative to a specified risk model).

Value traps make this chain concrete: falling prices mechanically raise B/M, but if fundamentals keep deteriorating, the low price may be a reasonable expectation. Quality measures add another set of observable, ex ante measures for earnings, cash flow, and leverage, but they cannot guarantee that every trap is removed. Chapter 26 discusses how those measures are formed; Chapter 28 discusses how to combine them.

8. Numerical Worked Examples: Calculate Each Step

All figures are produced by hand_examples() and independently checked in `ch24_recompute.py` using a different implementation. The companies are teaching constructions (Orion and Lyra come from Chapter 13; the others are constructed for this chapter), not real companies.

8.1 Four Definitions for Two Companies in the Same Industry (Orion and Lyra)

Amounts are in millions of dollars, and share counts are in millions of shares. Orion: price 60, 10 shares, market capitalization 600; debt 100, cash 50, so enterprise value is $600+100-50=650$; book equity 180, net income 30, FCFE 28, FCFF 40, and EBITDA 75. Lyra: price 24, 20 shares, market capitalization 480; debt 260, cash 30, noncontrolling interests 40, preferred equity 20, and nonoperating assets 20, so enterprise value is $480+260-30+40+20-20=750$; book equity 120, net income 20, FCFE 18, FCFF 40, and EBITDA 75.

Definition	Orion	Lyra	Cheaper
B/M	180/600 = 0.300	120/480 = 0.250	Orion
E/P	30/600 = 5.00%	20/480 = 4.17%	Orion
FCFE/market capitalization	28/600 = 4.67%	18/480 = 3.75%	Orion
FCFF/EV	40/650 = 6.15%	40/750 = 5.33%	Orion
EV/EBITDA	650/75 = 8.67	750/75 = 10.00	Orion

All five definitions say Orion is cheaper. Note that both companies have exactly the same EBITDA and FCFF (75 and 40); the difference is entirely in enterprise value. Lyra's leverage makes it more expensive on an EV basis, though not necessarily on a market-capitalization basis.

8.2 Seven Companies in Two Industries: Marketwide and Within-Industry Ranks

Manufacturing has four companies: Orion, Lyra, Forge (price 20, 30 shares, market capitalization 600, book equity 480), and Gear (price 50, 8 shares, market capitalization 400, book equity 200). Banking has three: Harbor (price 30, 20 shares, market capitalization 600, book equity 540), Summit (price 45, 10 shares, market capitalization 450, book equity 360), and Delta (price 12, 40 shares, market capitalization 480, book equity 336).

Company	Industry	B/M	Marketwide average rank	Marketwide percentile	Within-industry percentile
Orion	Manufacturing	0.300	2	0.167	0.333
Lyra	Manufacturing	0.250	1	0.000	0.000
Forge	Manufacturing	0.800	5.5	0.750	1.000
Gear	Manufacturing	0.500	3	0.333	0.667
Harbor	Banking	0.900	7	1.000	1.000
Summit	Banking	0.800	5.5	0.750	0.500
Delta	Banking	0.700	4	0.500	0.000

Calculation: across all $n = 7$ companies, the ascending values are $0.25, 0.30, 0.50, 0.70, 0.80, 0.80, 0.90$. The two 0.80 observations occupy ranks 5 and 6, so each receives the average rank 5.5. Percentile is $(r - 1)/6$. In manufacturing, the ascending values are $0.25, 0.30, 0.50, 0.80$, with percentiles $(r - 1)/3 = 0, \frac{1}{3}, \frac{2}{3}, 1$. In banking, the ascending values are $0.70, 0.80, 0.90$, with percentiles $0, \frac{1}{2}, 1$. Interpretation: Forge (manufacturing) and Summit (banking) both have B/M of 0.80, so they tie marketwide. Within their industries, Forge is the cheapest manufacturer, while Summit is only in the middle of the banks. Delta is at marketwide percentile 0.5, the overall middle, but is the least cheap bank within its industry. Two of the marketwide top three (Harbor, Forge, Summit) are banks. That is an industry-level difference, not company-specific information.

8.3 Negative Denominators and Missing-Value Rules

Five companies (units are as above):

Company	Net income E	Book equity B	Market cap M	EV V
Normal	30	180	600	650
Loss-making	−10	40	100	150
Negative book equity	8	−25	200	260
Zero market cap (data error)	5	50	0	10
Negative EBITDA	12	70	300	320

Company	EBITDA	E/P	B/M	EV/EBITDA
Normal	75	0.050	0.300	8.67
Loss-making	20	−0.100	0.400	7.50
Negative book equity	30	0.040	NaN	8.67
Zero market cap (data error)	5	NaN	NaN	2.00
Negative EBITDA	−4	0.040	0.233	NaN

Rules: the denominator of E/P and EV/EBITDA must be positive; B/M requires both book equity and market capitalization to be positive. yield_ratio retains the sign of the numerator (the loss-making company has −0.10); the multiple form sets nonpositive EBITDA or nonpositive EV to NaN. In the fourth row, EV/EBITDA can still be calculated (2.0), while E/P and B/M cannot. This shows that different definitions can have different missing-value sets for the same company. When combining signals, either handle missing values separately or use a common intersection (Chapter 28).

A positive denominator close to zero. Strict positivity does not guarantee safety: with EV=100 and EBITDA=0.01, the multiple is 10,000; with EBITDA=-0.01, it is NaN. Two nearly identical companies land at opposite ends. The policy is to specify a lower bound or other treatment in the research plan in advance (this chapter sets no lower bound and only requires a positive denominator), report coverage excluded or missing, and run a sensitivity check after adding a threshold (for example, set the ratio to NaN when EBITDA as a share of EV is below a threshold). Ranking limits the effect of an extreme multiple's magnitude, but it does not change the fact that this company ranks at the "most expensive" end. Ranking therefore cannot substitute for this policy.

8.4 Point-in-Time Selection: Four Versions for One Company

Version table for security S1:

Reporting period	Value (book equity)	available_at	ingested_at
FY2021	80	2022-03-15 16:05	2022-03-15 16:12
FY2022	100	2023-03-15 16:05	2023-03-15 16:12
FY2022 restated	90	2023-11-20 16:05	2023-11-20 16:12
FY2023	120	2024-03-12 16:05	2024-03-12 16:12

Book equity used at each month-end: End of February 2023: FY2022 has not been published yet (it is not released until March 15), so use FY2021, value 80. End of March 2023: FY2022 has been published, so use 100. End of October 2023: still use 100; the restatement has not yet been published. End of November 2023: the restatement was published on November 20, so select the latest release for that period, value 90. End of February 2024: still use 90. End of March 2024: FY2023 has been published and is the latest period, so use 120. Three counterexamples: treating period_end as the availability date would use FY2022's 100 at the end of January 2023; backfilling with the last version in the full history would use 90 at the end of March 2023, although the market then knew only 100; and if the March 15, 2023 version had not entered the local system until April 2, the local system would still know only 80 at the end of March and would know 100 the following month. Test test_pit_late_ingestion_and_month_end_boundary covers the last case and the one-second month-end boundary case.

8.5 Leverage Amplification from Dividing FCFF by Market Capitalization

For Lyra, $V/M = 750/480 = 1.5625$. The correctly matched value is $\text{FCFF}/V = 40/750 = 5.33\%$; the mismatched value is $\text{FCFF}/M = 40/480 = 8.33\%$, or 1.5625 times the correct value. For comparison, Orion has $V/M = 650/600 = 1.083$, so the mismatched yield is $40/600 = 6.67\%$ versus the correct 6.15%, an 8.3% amplification. The mismatch changes the gap between Lyra and Orion from 0.82 percentage points under the correct definition (6.15%−5.33%) to a reversed difference of 1.67 percentage points: under the mismatch Orion is at 6.67% and Lyra at 8.33%, making Lyra appear cheaper.

8.6 HML Portfolio Arithmetic

Suppose the returns on the four value-weighted portfolios for a month are: Small Value 1.4%, Big Value 0.8%, Small Growth 0.4%, and Big Growth 0.6%. Then HML $= \frac{1}{2}(1.4+0.8) - \frac{1}{2}(0.4+0.6) = 0.60\%$. Using only Big stocks gives $0.8 - 0.6 = 0.20\%$.

8.7 Dilution of IC by Industry-Level Differences

Let $\Delta = 0.9$ and $\sigma_u = 0.5$. Then $\mathrm{Var}(a) = (0.45)^2 = 0.2025$, and the attenuation factor is $0.5/\sqrt{0.25 + 0.2025} = 0.743$. If the population within-industry IC is 0.036, the full-market IC is approximately $0.036 \times 0.743 = 0.027$. The constructed panel measures 0.0256. (Here we use a pure Gaussian base deviation with $\sigma_u = 0.5$ for a hand calculation; Section 6.3 includes the extra displacement of trap companies and uses $\sigma_u \approx 0.55$, giving a factor of about 0.77.)

9. Python Implementation

The core implementation is in `factor_signals.py` (value ratios and point-in-time selection) and `factor_research.py` (group_percentile). The Chapter 13 valuation.py definitions Amount, enterprise_value, FinancialVintage, and select_pit_vintage remain unchanged; this chapter reuses their enterprise-value bridge.

Function	Input	Output and boundaries
positive_denominator_ratio	Two date-by-asset tables with matching labels	Numerator divided by denominator; NaN if denominator is nonpositive or either value is missing; can require a positive numerator; mismatched labels, booleans, and infinities raise errors
book_to_market	Book equity and market capitalization	B/M; nonpositive book equity is NaN by default
yield_ratio	Cash flow or earnings and the matching claim value	Retains the sign of the numerator; denominator must be positive
ev_to_ebitda	Enterprise value and EBITDA	Multiple; both EV and EBITDA must be positive
cheapness_percentile	Ratio table, kind="yield" or "multiple"	Same-date percentile, where 1 is cheapest; multiple values are ranked in reverse rather than inverted
group_percentile	Signal table, industry labels (static or time-varying), minimum group size	Same-date within-industry percentile; NaN when too few valid assets are in the group or the label is missing
latest_available_values	Long-form version table, monthly PeriodIndex, value columns	Latest version of the latest known reporting period for each security at each month-end; requires time zones; errors on ambiguous rows; cutoff is the final instant of month-end

Default offline entry point (run from the root of the I3 reading package):

```
.venv/bin/python run_notebook.py notebooks/ch24_value_factors.ipynb
```

The default is RUN_FRENCH=False. It shows hand-calculated ratios, missing-value policies, industry rankings, and PIT versions, then demonstrates a constructed panel with 300 companies, 108 months, and seed 20261031. It creates only the chapter RUN_DIR and writes no CSV, PNG, or JSON files or historical figure library. The runner separately writes an execution copy and status under `runs/execution_<stem>_<uuid>/`. The Notebook's English language layer and full-package paths are assembled in BIL23. The following are references to the historical full CLI and tests, not default offline commands:

```
.venv/bin/python code/ch24_value_factors.py
.venv/bin/python -m pytest tests/test_ch24_value.py tests/test_ch24_workflow.py -q
```

CLI seed: 20261031 (constructed panel). Data frequency: monthly; signal_time is the final instant of month-end in UTC; the signal predicts next month's return. French HML is a published aggregate monthly spread read after hash verification, with revised history.

Code example 1: five definitions, the EV bridge, and negative-denominator rules (Sections 8.1 and 8.3).

```
import numpy as np
import pandas as pd

from quant_course.factor_signals import book_to_market, ev_to_ebitda, yield_ratio
from quant_course.valuation import Amount, enterprise_value

unit = "USD millions"
lyra_cap = Amount(24.0 * 20.0, "USD", unit)
lyra_ev = enterprise_value(lyra_cap, Amount(260, "USD", unit), Amount(30, "USD", unit
    ↪ ),
                           noncontrolling_interest=Amount(40, "USD", unit),
    ↪ preferred_equity=Amount(20, "USD", unit),
                           nonoperating_assets=Amount(20, "USD", unit))
assert lyra_ev.value == 750.0
assert abs(40 / lyra_ev.value - 0.053333) < 1e-6 and abs(40 / lyra_cap.value -
    ↪ 0.083333) < 1e-6   # FCFF/EV vs the mismatched FCFF/market cap

idx = pd.period_range("2026-06", periods=1, freq="M")
cols = ["normal", "loss", "neg_book", "zero_cap", "neg_ebitda"]
earn = pd.DataFrame([[30.0, -10.0, 8.0, 5.0, 12.0]], index=idx, columns=cols)
book = pd.DataFrame([[180.0, 40.0, -25.0, 50.0, 70.0]], index=idx, columns=cols)
cap = pd.DataFrame([[600.0, 100.0, 200.0, 0.0, 300.0]], index=idx, columns=cols)
ev = pd.DataFrame([[650.0, 150.0, 260.0, 10.0, 320.0]], index=idx, columns=cols)
ebitda = pd.DataFrame([[75.0, 20.0, 30.0, 5.0, -4.0]], index=idx, columns=cols)
ep, bm, mult = yield_ratio(earn, cap).iloc[0], book_to_market(book, cap).iloc[0],
    ↪ ev_to_ebitda(ev, ebitda).iloc[0]
assert abs(ep["loss"] + 0.10) < 1e-12 and np.isnan(ep["zero_cap"])
    ↪    # sign of the flow kept; zero market cap -> NaN
assert np.isnan(bm["neg_book"]) and abs(bm["neg_ebitda"] - 70 / 300) < 1e-12
assert abs(mult["zero_cap"] - 2.0) < 1e-12 and np.isnan(mult["neg_ebitda"])
print(pd.DataFrame({"E/P": ep, "B/M": bm, "EV/EBITDA": mult}).round(4))
```

Code example 2: marketwide ranking and within-industry percentiles (Section 8.2).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import factor_rank, group_percentile

firms = ["Orion", "Lyra", "Forge", "Gear", "Harbor", "Summit", "Delta"]
industry = pd.Series(["Mfg"] * 4 + ["Bank"] * 3, index=firms)
bm = pd.DataFrame([[0.30, 0.25, 0.80, 0.50, 0.90, 0.80, 0.70]], index=pd.period_range
    ↪ ("2026-06", periods=1, freq="M"), columns=firms)
raw = factor_rank(bm).iloc[0]
inside = group_percentile(bm, industry).iloc[0]
np.testing.assert_allclose(raw.to_numpy(), [1 / 6, 0, 0.75, 1 / 3, 1, 0.75, 0.5])
np.testing.assert_allclose(inside.to_numpy(), [1 / 3, 0, 1, 2 / 3, 1, 0.5, 0])
assert raw["Forge"] == raw["Summit"] and inside["Forge"] > inside["Summit"]
    ↪         # tied overall, ordered inside the industry
print(pd.DataFrame({"raw": raw, "within_industry": inside}).round(3))
```

Code example 3: point-in-time version selection, including restatements, late ingestion, and the month-end boundary (Section 8.4).

```
import numpy as np
import pandas as pd

from quant_course.factor_signals import latest_available_values

versions = pd.DataFrame({
    "security_id": ["S1"] * 4,
    "period_end": pd.to_datetime(["2021-12-31", "2022-12-31", "2022-12-31", "
    ↪ 2023-12-31"]),
    "available_at": pd.to_datetime(["2022-03-15 16:05", "2023-03-15 16:05", "
    ↪ 2023-11-20 16:05", "2024-03-12 16:05"], utc=True),
    "ingested_at": pd.to_datetime(["2022-03-15 16:12", "2023-03-15 16:12", "
    ↪ 2023-11-20 16:12", "2024-03-12 16:12"], utc=True),
    "book_equity": [80.0, 100.0, 90.0, 120.0]})
months = pd.period_range("2023-02", "2024-03", freq="M")
pit = latest_available_values(versions, months, ["book_equity"])["book_equity"]["S1"]
assert [pit[pd.Period(m)] for m in ("2023-02", "2023-03", "2023-10", "2023-11", "
    ↪ 2024-02", "2024-03")] == [80, 100, 100, 90, 90, 120]

late = versions.copy()                                                  # filed Mar
    ↪  15 but entered the local database on Apr 2
late.loc[1, "ingested_at"] = pd.Timestamp("2023-04-02 09:00", tz="UTC")
pit_late = latest_available_values(late, months, ["book_equity"])["book_equity"]["S1"
    ↪ ]
assert pit_late[pd.Period("2023-03")] == 80 and pit_late[pd.Period("2023-04")] == 100

future = pd.concat([versions, versions.iloc[[3]].assign(book_equity=999.0,
    ↪ available_at=pd.Timestamp("2025-01-01", tz="UTC"),
                                                        ingested_at=pd.Timestamp("
    ↪ 2025-01-01", tz="UTC"),
                                                        period_end=pd.Timestamp("
    ↪ 2024-12-31"))], ignore_index=True)
```

```
changed = latest_available_values(future, months, ["book_equity"])["book_equity"]["S1
    ↪ "]
pd.testing.assert_series_equal(pit, changed)                                    # a version
    ↪  published after the last signal changes nothing
print(pit.to_dict())
```

Code example 4: calendar-month diagnosis for look-ahead (Sections 6.5 and 12).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import group_percentile, ic_series, lead_return
from quant_course.factor_signals import book_to_market

rng = np.random.default_rng(5)
months, firms = 72, 200
idx = pd.period_range("2015-01", periods=months, freq="M")
cols = [f"F{i:03d}" for i in range(firms)]
cap = pd.DataFrame(np.exp(rng.standard_normal((months, firms))), index=idx, columns=
    ↪ cols)
book_old = pd.DataFrame(np.exp(rng.standard_normal((months, firms))), index=idx,
    ↪ columns=cols)
returns = pd.DataFrame(0.07 * rng.standard_normal((months, firms)), index=idx,
    ↪ columns=cols)
book_new = book_old.copy()
for year in sorted({p.year for p in idx}):
    news = rng.standard_normal(firms)                                           # book-
    ↪ value news filed in March of this year
    march, early = pd.Period(f"{year}-03", "M"), [pd.Period(f"{year}-01", "M"), pd.
    ↪ Period(f"{year}-02", "M")]
    if march in idx:
        returns.loc[march] += 0.05 * news                                       # the price
    ↪  reacts to the news in the filing month
    for p in early:
        if p in idx:
            book_new.loc[p] = book_old.loc[p] * np.exp(0.8 * news)              # look-
    ↪ ahead: the new book value is used before it is public
groups = pd.Series(["A"] * 100 + ["B"] * 100, index=cols)
label = lead_return(returns, 1)
ic_pit = ic_series(group_percentile(book_to_market(book_old, cap), groups), label)["
    ↪ ic"].dropna()
ic_look = ic_series(group_percentile(book_to_market(book_new, cap), groups), label)["
    ↪ ic"].dropna()
by_month = lambda s: s.groupby([p.month for p in s.index]).mean()
gap = by_month(ic_look) - by_month(ic_pit)
assert gap[2] > 0.15 and gap[2] == gap.max()                                    # Feb
    ↪ signals predict the March filing reaction
assert np.allclose(gap.drop([1, 2]), 0.0)                                       # months
    ↪ with identical books give identical IC
print(gap.round(3).to_dict())
```

Code example 5: checking direction and input validation with positive_denominator_ratio.

```
import numpy as np
import pandas as pd
import pytest

from quant_course.factor_signals import cheapness_percentile,
    ↪ positive_denominator_ratio

idx = pd.period_range("2026-06", periods=1, freq="M")
yields = pd.DataFrame([[0.05, 0.08, 0.02, np.nan]], index=idx, columns=list("abcd"))
multiples = pd.DataFrame([[10.0, 6.0, 20.0, np.nan]], index=idx, columns=list("abcd")
    ↪ )
np.testing.assert_allclose(cheapness_percentile(yields, kind="yield").iloc[0, :3],
    ↪ [0.5, 1.0, 0.0])
np.testing.assert_allclose(cheapness_percentile(multiples, kind="multiple").iloc[0,
    ↪ :3], [0.5, 1.0, 0.0])
with pytest.raises(ValueError):
    positive_denominator_ratio(yields, yields.rename(columns={"d": "z"}))
    ↪    # labels must match
with pytest.raises(ValueError):
    positive_denominator_ratio(yields.where(yields < 1, np.inf), yields)
    ↪    # infinity is rejected, NaN is the only missing marker
print("ok")
```

10. Reading the Figures

Figure 24-1: Dilution of IC by industry levels. The constructed panel has 300 companies (150 each in manufacturing and banking), 108 months, and fixed seed 20261031. The left panel shows each company's $\log(B/M)$ at the end of January 2015: the industry-wide levels differ by about 0.9 (black lines mark the industry means), and within-industry dispersion is on the same order as the between-industry difference. The right panel shows mean Spearman IC for three signals (error bars are ± 2 standard errors): marketwide B/M rank, 0.0256; within-industry rank, 0.0360; and within-industry rank using fiscal-year-end book values (look-ahead), 0.0414. Interpretation: industry levels reduce full-market IC by about one third; look-ahead makes within-industry IC about 15% higher and therefore makes the signal look better. This uses constructed data.

Figure 24-2: Calendar-signal-month IC diagnostic for look-ahead. The x-axis is the calendar month of the signal (January through December); the y-axis is mean IC across all signals in that calendar month. Blue shows point-in-time book values; orange shows look-ahead book values. The two differ only in January, February, and March (in the construction, filings occur from late February through April). In February, IC is 0.0186 versus 0.0557; in March, 0.0533 versus 0.0729; from April onward the series coincide. Interpretation: look-ahead does not add the same amount to every month; it concentrates in the period after fiscal year-end but before filing. Splitting IC by calendar month is a low-cost way to detect an availability-time problem.

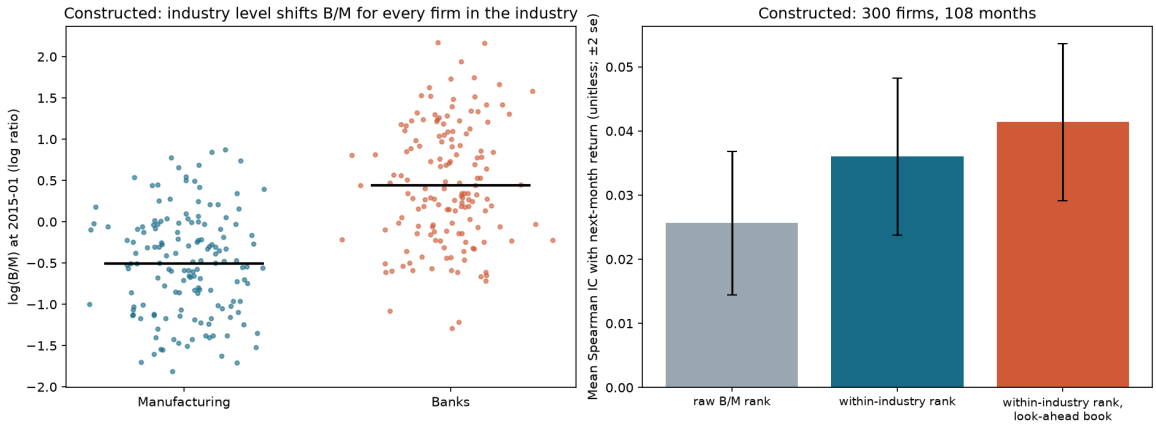


Figure 24-1: Industry-level dilution of IC and look-ahead inflation

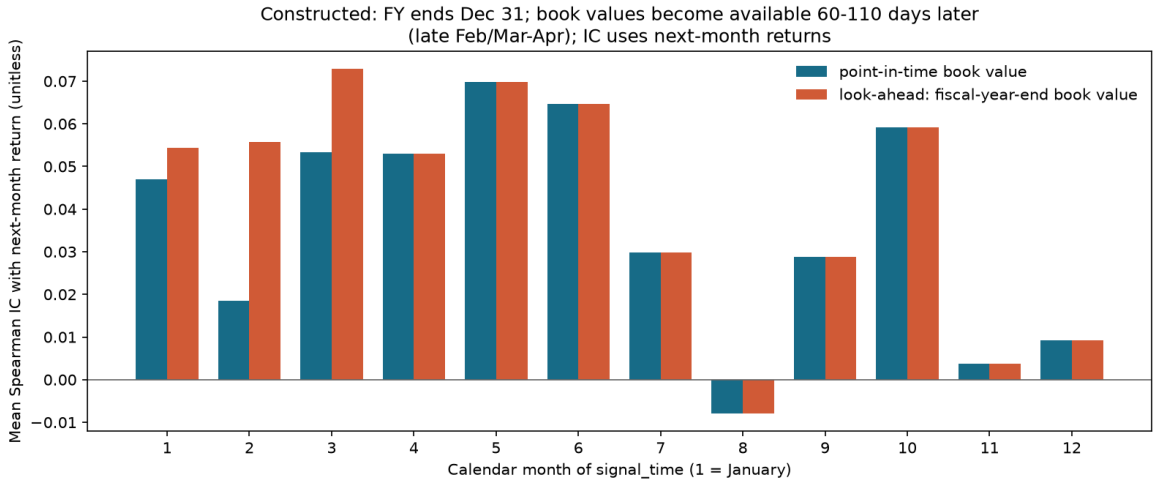


Figure 24-2: Calendar-month IC using point-in-time and look-ahead book values

Figure 24-3: Decile returns and value traps. The left panel shows equal-weighted mean next-month returns (%) for within-industry-percentile deciles, with decile 10 the cheapest. Returns rise overall from 0.089 to 1.058 (decile 3 at 0.514 is below decile 2 at 0.583, and deciles 5 and 6 are slightly below decile 4; the pattern is not strictly monotonic). The right panel shows the trap-company share in each decile: 44.7% of the cheapest decile are traps (the overall share is 9.7%, shown by a dashed line), while the seven most expensive deciles have almost none. Interpretation: traps cluster at the "cheapest" end because prices have already fallen and B/M is high, lowering that decile's mean return. Constructed data.

Figure 24-4: Published HML. The left panel shows the compounded cumulative HML monthly spread (log scale, starting at 1, sample July 1926–August 2026); the shaded area marks 2007–2020. The right panel shows the rolling 120-month mean HML (% per month). Interpretation: the long-run path rises, but there is a roughly 14-year decline after 2007; the rolling mean falls to about −0.5% per month. The value premium is not constant. These are published aggregate spreads and exclude financing and trading costs.

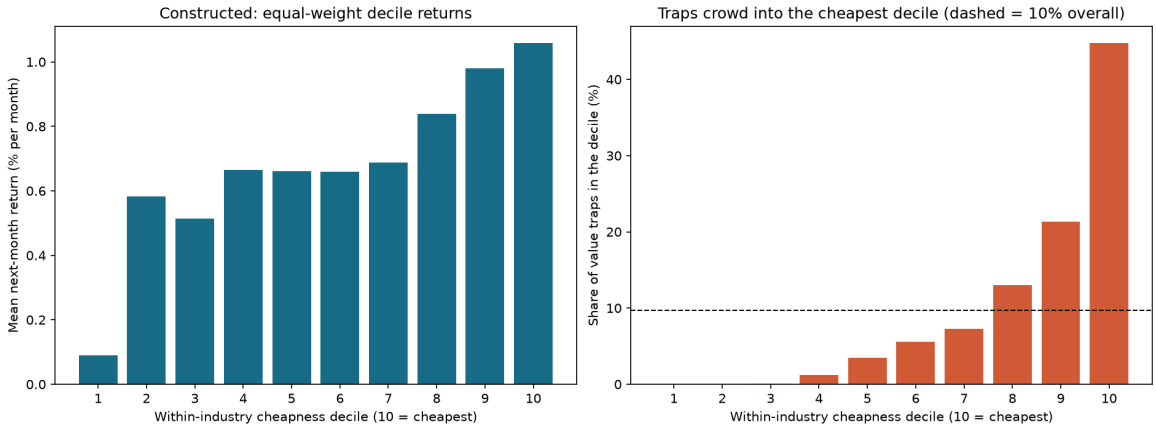


Figure 24-3: Within-industry decile returns and value-trap shares

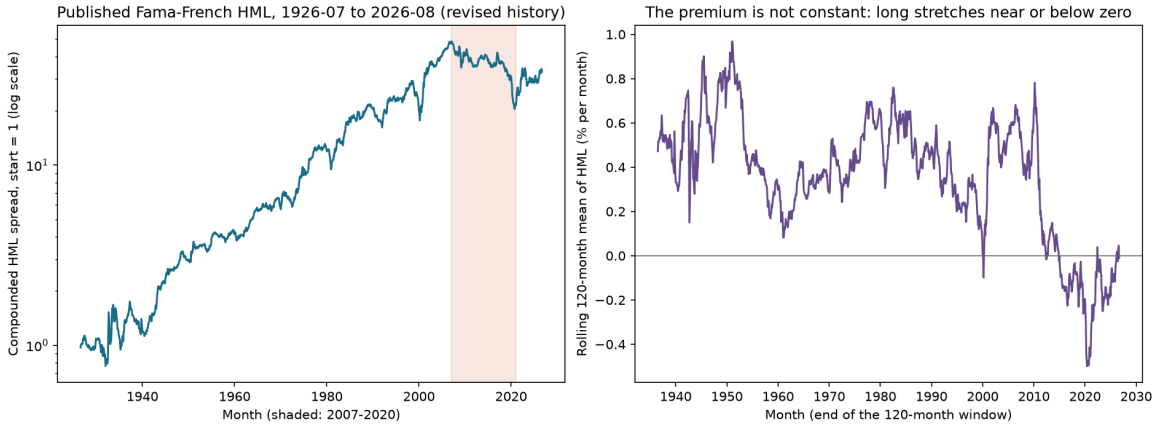


Figure 24-4: Cumulative Fama–French HML and rolling mean

11. Historical Market Example

Data and limitations. We use the French three-factor monthly data frozen on 2026-10-05: Ken French Data Library: F-F Research Data Factors. The processed CSV has SHA-256 `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814` and contains 1,202 months (July 1926–August 2026). Units are decimal monthly spread returns. Data before August 2026 may be revised as CRSP updates its history. Publicly downloadable data are not necessarily free for unrestricted redistribution. As described in Section 6.7 and the source notes, HML uses book equity from fiscal year $t-1$, market capitalization from December of year $t-1$, forms portfolios in July, and holds them through the following June; it includes only companies with positive book equity. This is a published aggregate return, not a per-stock B/M signal, not a point-in-time version, and not an executed trade.

Descriptive results. Across all 1,202 months, mean HML is 0.352% per month (standard deviation 3.55%, $t = 3.43$). By subsample:

Period	Months	Mean (% per month)	t	Standard deviation (% per month)
1926-07 to 1963-06	444	0.450	2.17	4.37
1963-07 to 1999-12	438	0.349	2.83	2.58
2000-01 to 2026-08	320	0.218	1.14	3.43
2007-01 to 2020-12	168	−0.443	−1.99	2.88

Maximum drawdown of the compounded series is 57.8%, occurring in 2020-09. The correlation with market excess returns is 0.22 and with SMB is 0.13. The t-statistics treat months as independent observations; HML is serially correlated, and no Newey–West correction was applied. The 2007–2020 window was selected after the fact. Its $t = -1.99$ is not a test conclusion; it only describes the absence of positive returns in that sample. Selecting the window is itself an experiment and should be recorded in the Chapter 22 ledger.

What we can and cannot say. We can say that the published value spread has a significantly positive mean over 1926–2026 and was negative from 2007–2020, with substantial variation in mean and volatility across subsamples. We cannot say: (1) that per-stock B/M signals worked or failed during those years; HML is an arithmetic combination of six value-weighted portfolios, not the return from any single B/M ranking; (2) that a company will rise because its B/M is high; or (3) that Fama–French's (2015) conclusion that HML is redundant in a five-factor model applies to 1926–1963 or international data. Their original result is limited to US data from 1963–2013.

Data needed for per-stock value research are not available here. At minimum, per-stock point-in-time value research requires each company's financial-statement versions and available_at; price, shares, and market capitalization for every signal month; a historical stock universe including delisted stocks and delisting returns; and historical versions of industry classifications. According to the source notes read on 2026-10-06, the US Securities and Exchange Commission's (SEC) Financial Statement Data Sets provide "as filed" XBRL values from January 2009 through June 2026, with filed and accepted (acceptance date/time) fields. They are free to redistribute with attribution, subject to rate limits and a requirement to declare a User-Agent. But they contain no prices, shares, stock universe, or delisting returns; prevrpt only flags that a submission was later amended and does not provide a complete restatement chain. The data alone are therefore insufficient for per-stock value research, and XBRL is unavailable before 2009. This chapter did not download the data. The Project 4 preparation gap table lists what is missing (../../../../projects/project04/preparation/).

12. Common Mistakes

Mistake 1: Letting industry differences contaminate a marketwide ranking. Section 8.2: two of the marketwide top three are Harbor and Summit, both banks; the constructed panel's marketwide IC is 0.0256 versus 0.0360 within industries. Fix: use same-date within-industry percentiles and report small industries excluded.

Mistake 2: Dividing FCFF by market capitalization. Section 8.5: Lyra's FCFF yield changes from 5.33% to 8.33%, reversing its ranking relative to Orion. Fix: match the definitions: FCFF over EV, FCFE over market capitalization.

```
import pandas as pd

from quant_course.factor_signals import yield_ratio

idx = pd.period_range("2026-06", periods=1, freq="M")
cols = ["Orion", "Lyra"]
fcff = pd.DataFrame([[40.0, 40.0]], index=idx, columns=cols)
cap = pd.DataFrame([[600.0, 480.0]], index=idx, columns=cols)
ev = pd.DataFrame([[650.0, 750.0]], index=idx, columns=cols)
right, wrong = yield_ratio(fcff, ev).iloc[0], yield_ratio(fcff, cap).iloc[0]
assert right["Orion"] > right["Lyra"] and wrong["Lyra"] > wrong["Orion"]
    ↪          # the ranking reverses
assert abs(wrong["Lyra"] / right["Lyra"] - 750 / 480) < 1e-12
print(right.round(4).to_dict(), wrong.round(4).to_dict())
```

Mistake 3: Taking absolute values of negative numbers or setting them to zero. Taking the absolute value of negative earnings makes a loss-making company look cheap; setting it to zero puts it in the middle. Fix: keep the sign in yield form, set nonpositive denominators to NaN in multiple form, and report the missing share.

```
import numpy as np
import pandas as pd

from quant_course.factor_research import factor_rank

idx = pd.period_range("2026-06", periods=1, freq="M")
pe = pd.DataFrame([[12.0, 8.0, -20.0, 15.0]], index=idx, columns=list("abcd"))
    ↪ # firm c has negative earnings: P/E = -20
naive = factor_rank(-pe).iloc[0]                        # cheapest = smallest P/E: the
    ↪  loss-maker looks cheapest
fixed = factor_rank(-pe.where(pe > 0)).iloc[0]          # non-positive P/E is missing
assert naive["c"] == 1.0 and np.isnan(fixed["c"]) and fixed["b"] == 1.0
print(naive.round(2).to_dict(), fixed.round(2).to_dict())
```

Mistake 4: Treating period-end as the availability date. Section 8.4: using period_end would make FY2022 available by the end of January 2023 even though it was not published until March. Fix: use available_at and ingested_at and select with latest_available_values. The calendar-month diagnosis in Figure 24-2 can reveal the error.

Mistake 5: Backfilling history with today's restated values. Section 8.4: the March 2023 month-end would incorrectly use 90, which was not available until November. Fix: preserve every version in the version table. Without version history, a point-in-time study is not possible; label the work as a teaching construction.

Mistake 6: Reading "cheap" as "will rise." In the constructed panel, 44.7% of the cheapest decile consists of traps. Without identifying them, the cheapest leg of a value long-short portfolio is dragged down (1.058% versus 1.361%). Fix: combine the signal with Chapter 26's quality measures and Chapter 28's factor composite. A standalone value backtest should report the trap share or observable proxies such as leverage and cash-flow signs.

Mistake 7: Applying HML conclusions to individual stocks. See Section 11. HML is an arithmetic spread of six value-weighted portfolios; value weighting gives large companies more influence. An equal-weighted long-short portfolio of individual-stock B/M deciles can behave very differently.

13. Quantitative Research Application

Project 4's value-field contract. Every value signal needs a field table. Example (the complete table is in ../../../../projects/project04/preparation/):

Field	Source	Time fields	Units	Missing-value rule
book_equity	Financial statements	period_end, available_at, ingested_at	Reporting currency	Nonpositive values become NaN, with reason recorded
market_cap	Price × shares	Month-end signal_time	Reporting currency	Nonpositive values become NaN
industry	Versioned classification	available_at	Label	Missing becomes NaN
ratio_bm	Calculation	signal_time	Dimensionless	Follows book_equity and market_cap
label_1m	Following month's return	Start and end of label interval	Decimal	Missing becomes NaN

Research workflow (replicating the constructed panel; the Chapter 22 ledger constraints also apply). First freeze the plan: hypothesize that companies with higher B/M within the same industry on the same date have higher returns the following month; use within-industry percentile as the signal, next-month return as the label, the t-statistic of Rank IC as the primary metric, and $t \geq 2$ as the criterion. Then calculate three variants: marketwide ranking, within-industry ranking, and look-ahead book values (diagnostic only, excluded from the main conclusion). Results: 108 months and 300 companies; Spearman ICs are 0.0256 ($t = 4.58$), 0.0360 ($t = 5.87$), and 0.0414 ($t = 6.76$), respectively; hit rates are 70.1%, 73.8%, and 75.7%; within-industry decile long-short spread is 0.969% per month ($t = 5.10$), while the marketwide-ranked spread is 0.478%. Calendar-month IC shows look-ahead is elevated only from January through March. Trap share is 44.7%. These numbers depend on the constructed-data parameters (industry-level difference of 0.9, value effect of 0.4% per standard deviation, and 10% traps). They

would change under different parameters; their purpose is to demonstrate mechanisms, not to estimate the real-world value premium.

Constraints for Project 4. (1) Pass value fields through latest_available_values or Chapter 3's pit_asof_join; without a version table, label them "constructed for teaching." (2) Put missing-value rules and exclusion shares in the configuration. (3) Report within-industry and marketwide rankings side by side. (4) Include look-ahead and calendar-month IC as data audits. (5) Report observable proxies for value traps. (6) Record all variants in the Chapter 22 ledger. Project 4 cannot be completed until Chapters 29–36; this chapter completes only the signal-layer fields and rules.

13.1 Current Seven-Field Interface: How This Chapter's Variables Enter Project 4

The current signal_contract.json is p04-prep-0.2; the general preprocessing rules are in frozen_rules.json, version p04-prep-0.1. The files have different responsibilities. The seven fields are three value measures (book_to_market, earnings_yield, ev_to_ebitda_cheapness), one momentum measure (mom_12_1), and three quality measures (gp_a, neg_accrual, roe_avg). FCF Yield is a teaching variable in this chapter and is not implemented among the current seven fields. OP is not a signal at this layer either.

For the constructed universe, EV is only market_cap + debt - cash. The contract explicitly sets noncontrolling interests, preferred equity, and nonoperating assets to zero. This is a construction assumption; missing fields for a real company must not be treated as zero. Real use requires complete point-in-time bridge items in consistent currencies and scales, with the full bridge checked. The clarification of the old machine-readable contract is closed; see acceptance.md. This chapter only explains that already-closed boundary.

The three chapters share the following interface table. Signal-layer neutralization and portfolio-layer projection are separate: the former changes signals, the latter changes weights. Signal neutralization does not guarantee that the resulting portfolio is tradable or neutral.

Layer	Current action	What readers should check
Raw seven fields	Apply each formula; convert the EV multiple to a reverse percentile so larger means cheaper	Valid denominators and complete PIT fields; do not first set every missing value to zero
Same-date winsorization	Winsorize each field at the 1st/99th percentiles, requiring at least 30 valid names	Changes extreme magnitudes; does not create missing observations
Same-date residualization	OLS on industry dummies and log market capitalization; use residuals	Economic meaning of controlled exposures and sample coverage
Same-date standardization	z-scores with sample standard deviation, ddof=1, requiring at least 10 valid names	The raw EV direction change and standardization are separate steps
Within-style composite	Equal-weight valid fields and renormalize, requiring at least one field	A company with one field and a company with three fields have different amounts of evidence
Across-style composite	Equal-weight valid styles and renormalize, requiring at least two styles	Below the threshold, remain missing and report coverage loss

Layer	Current action	What readers should check
Optional portfolio	Long-short quintiles; optional industry-weight projection	C13 has corrected listed support/gross 2/60-name threshold; real eligibility and post-execution residuals still need separate checks

How to read the small example: after same-date winsorization, neutralization, and standardization, a company's three value z-scores are $(1.2, \text{NaN}, 0.6)$. Each of the two valid fields gets half the weight, so its value score is 0.9. Momentum is -0.3, quality is 0.6, and the three-style equal-weight composite is $(0.9-0.3+0.6)/3 = 0.4$. If quality is missing, average the two available styles: $(0.9-0.3)/2 = 0.3$. If only value remains, the score is missing because it falls below the two-style minimum; it is not 0.9. These are interface calculations, not return forecasts.

Older A6 records identified nonzero weights outside the listed support and loss of neutrality after clearing those columns. C13 has since granted limited acceptance to projection on the listed support (including listed hedge coordinates without alpha), restoration of gross exposure to 2, and the 60-listed-signal-name threshold; execution cancellations and closing-drift residuals are reported separately. See the current portfolio-support interface. Real trading eligibility, borrowing, costs, capacity, and a complete real-data P4 remain unfinished. Old Notebooks and A5/A6 outputs retain their historical identity; the source correction does not mean they were rerun. General IC and time-inference methods are in Chapter 23; experiment budgeting is in Chapter 22.

14. Chapter Summary

A value signal divides an accounting measure by a price measure, aligns the direction, and ranks companies within the same peer group on the same date. Denominators must be positive and definitions must match; otherwise false bargains appear, as with negative book equity, nonpositive EBITDA, or FCFF divided by market capitalization. Industry levels are not company-specific information: banks and software companies have different B/M levels, and same-date within-industry percentiles remove that industry component. Every signal value needs an available_at timestamp. In the specified construction, period-end or today's restated values raise IC for some signal months; that direction is not a general guarantee, and stronger results still require a timing audit. Value traps are observations for which "cheap" fails; they cluster in the cheapest decile. Published HML is an arithmetic spread of six value-weighted portfolios; it has a positive long-run mean and was negative from 2007–2020. It describes aggregate returns, not a per-stock signal test. Scope: the constructed panel demonstrates mechanisms; HML has a revised history; this chapter has no per-stock point-in-time data, and the Project 4 data gaps remain for the user to decide whether to fill.

15. Exercises

24-E01 (conceptual). Why, for positive x, does the ascending rank of V/EBITDA equal the descending rank of EBITDA/V, but this equivalence fail when the numerator may be negative? Are the two forms equivalent when z-scored or winsorized?

24-E02 (calculation). A company has price 40, 25 million shares, common book equity 700, net income attributable to common shareholders 80, FCFE 60, debt 500, cash 100, noncontrolling interests 50, preferred equity 0, nonoperating assets 30, EBITDA 200, and FCFF 90 (all amounts are in millions of dollars). Calculate enterprise value, B/M, E/P, FCFE yield, FCFF/EV, EV/EBITDA, and the mismatched FCFF/market capitalization and its amplification relative to the correctly matched value.

24-E03 (calculation). On one date, Industry A has four companies with B/M values (0.6, $0.4, 0.4, 0.9$), and Industry B has three with values $(1.2, 0.8, 1.0)$. Calculate full-market average-rank percentiles and within-industry percentiles.

24-E04 (calculation). Security S2 has the following versions: FY2020, book equity 50, externally available 2021-03-01 16:00 and ingested the same day at 16:10; FY2021, 55, available 2022-02-20 16:00 and ingested at 16:10; restated FY2021, 48, available 2022-10-05 16:00 and ingested 2022-10-07 09:00; FY2022, 70, available 2023-03-10 16:00 and ingested 2023-04-03 09:00 (all times are UTC). Which book-equity value is used at the ends of February 2022, September 2022, October 2022, March 2023, and April 2023? Explain why.

24-E05 (calculation). Four value-weighted portfolio returns are: Small Value 1.4%, Big Value 0.8%, Small Growth 0.4%, and Big Growth 0.6%. Calculate HML and explain the difference between using only Big stocks and using all four portfolios.

24-E06 (derivation and calculation). Derive $\text{FCFF}/M = (V/M) \cdot (\text{FCFF}/V)$. What are the matched yields for Lyra (market cap 480, EV 750, FCFF 40, FCFE 18)? How does the mismatch change its ranking relative to Orion (market cap 600, EV 650, FCFF 40)?

24-E07 (calculation). There are two equally weighted industries with level difference $\Delta = 0.9$ and within-industry standard deviation $\sigma_u = 0.5$. Find the correlation attenuation factor for full-market versus within-industry ranking. If the within-industry IC is 0.036, what is the approximate full-market IC?

24-E08 (calculation). Traps make up 44.7% of the cheapest decile. Its mean monthly return is 1.058% including traps and 1.361% after removing them. Find the traps' mean monthly return. If a researcher cannot identify traps, how should the result be reported?

24-N01 (claims and mechanisms). Recalculate the complete EV, FCFE, two correctly matched yields, and mismatched yield from the inputs in Section 3.1, one term at a time. Then explain why the net-cash counterexample means EV need not exceed market capitalization. Give one test of risk compensation and one test of proxy measurement, and explain why a positive IC cannot distinguish them.

16. Programming Exercises

24-P01. For a version table with five companies and three filing versions per company (first FY2021, restated FY2021, and late-ingested FY2022), use latest_available_values to produce an eight-month book-equity panel. Assert that: the restatement is not backfilled; late ingestion delays availability by one month; and adding a future version does not change historical values. Acceptance: all three assertions pass.

24-P02. On a constructed panel (you may call value_world in `ch24_value_factors.py`), calculate Rank IC for within-industry B/M and marketwide B/M, and check that the former is larger. Change the industry-level difference Δ by changing the bank mean in the initial log_bm0, and observe how full-market IC changes. Acceptance: explain the direction of the population Pearson attenuation factor in Section 6.3 and report finite-sample Rank ICs for each Δ under a fixed data-generating process and seed. Do not present sample monotonicity as a general claim.

24-P03. Implement a "calendar-month IC diagnostic": given a signal table and next-month labels, return mean IC for all 12 calendar months. On constructed data with look-ahead, detect the anomalous month (the month with the largest point-in-time versus look-ahead difference is the month before filing). Acceptance: the detected months include February.

17. Challenge Problems

24-C01 (derivation). Let two companies have yields $y_1 = 0.10$ and $y_2 = 0.02$. Compare "average the yields" with "average the multiples, then take the reciprocal": $\bar{y}$ versus $1/\overline{(1/y)}$. Explain why averaging several value ratios as yields first gives a different result from averaging the multiples first (harmonic versus arithmetic mean), and why this chapter ranks before combining. Prerequisite: mean inequalities. Scoring: calculation 30, derivation 40, discussion of combination 30.

24-C02 (open-ended). Design a Project 4 field contract and timeline for the Chapter 26 quality measures together with this chapter's value signals. List each field, source, period_end, available_at, ingested_at, units, and missing-value rule; draw a timeline showing the signal month, filing, restatement, and label interval; identify what data are missing for per-stock research. Scoring: field completeness 25, correct timeline 30, missing-value rules and bias discussion 25, data gaps 20.

18. Complete Answers and Reference Implementations

24-E01. If $x_i < x_j$ are both positive, then $1/x_i > 1/x_j$, so the ascending and descending ranks correspond one-to-one. With negative values, for example $x = (-1, 1, 2)$ for a, b, and c, the ascending ranks of x are $(1, 2, 3)$. The reciprocals are $(-1, 1, 0.5)$; their descending ranks are b first, c second, and a third, or $(3, 1, 2)$. The ranks no longer correspond (and the reciprocal is undefined when $x = 0$). Z-scoring and winsorization act on different number scales in the two forms. For positive values they give opposite rankings, but the distances between values differ; reciprocal values become large when the denominator is close to zero. Under the same symmetric

quantile-winsorization policy on the same positive sample, the tail members correspond in reverse rank, so one cannot generally claim that different companies will be clipped. However, nonlinear reciprocal distances, interpolated cutoffs, and post-winsorization values are usually not equivalent. Therefore z-scores and weighted composites will usually differ. Rank first (using percentiles), then combine, to avoid the numerical problems introduced by reciprocals.

24-E02. Market capitalization $= 40 \times 25 = 1000$; enterprise value $= 1000 + 500 - 100 + 50 + 0 - 30 = 1420$. B/M $= 700/1000 = 0.70$; E/P $= 80/1000 = 8.00\%$; FCFE yield $= 60/1000 = 6.00\%$; FCFF/EV $= 90/1420 = 6.34\%$; EV/EBITDA $= 1420/200 = 7.10$. The mismatched FCFF/market cap is $90/1000 = 9.00\%$, or $9.00/6.338 = 1.42 = V/M$ times the correctly matched value (1420/1000).

24-E03. The seven full-market values in ascending order are $0.4, 0.4, 0.6, 0.8, 0.9, 1.0, 1.2$. The two 0.4 observations occupy ranks 1 and 2 and each receive average rank 1.5; 0.6 is rank 3, 0.8 rank 4, 0.9 rank 5, 1.0 rank 6, and 1.2 rank 7. Percentile is $(r-1)/6$: $0.4 \to 0.0833$, $0.6 \to 0.3333$, $0.8 \to 0.5$, $0.9 \to 0.6667$, $1.0 \to 0.8333$, $1.2 \to 1$. Industry A has $0.4, 0.4, 0.6, 0.9$, with ranks 1.5, 1.5, 3, and 4 and percentiles $(r-1)/3$: 0.1667, 0.1667, 0.6667, and 1. Industry B has $0.8, 1.0$, 1.2, with ranks 1, 2, and 3 and percentiles 0, 0.5, and 1. For companies in the order $(0.6, 0.4, 0.4, 0.9, 1.2, 0.8, 1.0)$, the full-market percentiles are $(0.333, 0.083, 0.083, 0.667, 1, 0.5, 0.833)$ and the within-industry percentiles are $(0.667, 0.167, 0.167, 1, 1, 0, 0.5)$. Industry B's 0.8 is in the middle of the full market (0.5) but the least cheap within its industry (0).

24-E04. End of February 2022: 55, because FY2021 was published and ingested on 2022-02-20. End of September 2022: 55, because the restatement on October 5 has not yet been published. End of October 2022: 48, because the restatement was available on October 5 and ingested on October 7, both before October 31; select the latest version of FY2021. End of March 2023: 48, because FY2022 was externally available on March 10 but did not enter the local system until April 3, so the local system did not know it on March 31. End of April 2023: 70, because it has been ingested and is now the latest reporting period. In each case, the known set requires both available_at and ingested_at to be no later than the cutoff; then select the latest reporting period and the latest version.

24-E05. HML $= \frac{1}{2}(1.4 + 0.8) - \frac{1}{2}(0.4 + 0.6) = 1.10 - 0.50 = 0.60\%$. Using only Big stocks gives $0.8 - 0.6 = 0.20\%$, a spread using only Big stocks that omits the Small Value/Small Growth difference (which is larger: $1.4 - 0.4 = 1.0\%$). Averaging all four portfolios gives each side half Small and half Big by bucket weight; it does not ensure zero market-capitalization or size beta.

24-E06. Since $V = M \cdot (V/M)$, multiply and divide FCFF by V to get $\text{FCFF}/M = (\text{FCFF}/V) \times V/M$. Lyra: FCFE/market cap $= 18/480 = 3.75\%$; FCFF/EV $= 40/750 = 5.33\%$; mismatched FCFF/market cap $= 40/480 = 8.33\%$; $V/M = 1.5625$. Orion: FCFF/EV $= 40/650 = 6.15\%$; mismatched yield $= 40/600 = 6.67\%$ ($V/M = 1.083$). Under the correctly matched definition, Orion's 6.15% exceeds Lyra's 5.33%, so Orion is cheaper. Under the mismatch, Lyra's 8.33% exceeds Orion's 6.67%, reversing the ranking.

24-E07. Attenuation factor is $\sigma_u/\sqrt{\sigma_u^2 + (\Delta/2)^2} = 0.5/\sqrt{0.25 + 0.2025} = 0.5/0.6727 = 0.743$. Approximate full-market IC is $0.036 \times 0.743 = 0.0268$. This is an approximation for Pearson correlation of log ratios; rank correlation and nonnormality change the exact value.

24-E08. Solve for the trap mean:

$$\bar{r}_{\text{trap}} = (1.058 - (1 - 0.447) \times 1.361)/0.447 = 0.683147651\% \text{ per month.}$$

If we instead use an alternative rounded share of 44.74% while keeping the two displayed, rounded return inputs, the result is 0.683753688%, displayed as 0.684%. This explains the display difference in the old answer; it must not be mixed with the precision of the question's 44.7%. The original estimate and its monthly aggregation basis remain historical; they were not re-estimated. If traps cannot be identified, report the observed result including traps (1.058%), the trap share or observable proxies (shares with falling earnings, leverage, or negative cash flow), and that this is a "known-trap" decomposition in a constructed panel. Traps are not directly observable in a real market.

24-P01 Reference Implementation

The three versions are counted per company: 15 input rows, with eight signal months from August 2022 through March 2023.

```
import numpy as np
import pandas as pd

from quant_course.factor_signals import latest_available_values

# Five companies; each has three filing versions before any future test row is added.
base = {f"S{i}": 60.0 + 20.0 * i for i in range(1, 6)}
rows = []
for sec, value in base.items():
    for period, available, ingested, book in [
        ("2021-12-31", "2022-03-15 16:00", "2022-03-15 16:10", value),
        ("2021-12-31", "2022-10-05 16:00", "2022-10-07 09:00", value - 10),
        ("2022-12-31", "2023-02-20 16:00", "2023-03-02 09:00", value + 40),
    ]:
        rows.append({"security_id": sec, "period_end": pd.Timestamp(period),
                     "available_at": pd.Timestamp(available, tz="UTC"),
                     "ingested_at": pd.Timestamp(ingested, tz="UTC"), "book_equity":
    ↪ book})
versions = pd.DataFrame(rows)
months = pd.period_range("2022-08", periods=8, freq="M")
table = latest_available_values(versions, months, ["book_equity"])["book_equity"]

# Restatement becomes known in October, never backfilled into August or September.
assert table.shape == (8, 5) and all(table.loc["2022-09", s] == v and table.loc["
    ↪ 2022-10", s] == v - 10 for s, v in base.items())

# The February filing is ingested in March: February retains the old value.
assert all(table.loc["2023-02", s] == v - 10 and table.loc["2023-03", s] == v + 40
    ↪ for s, v in base.items())

# Add one later version per company only for the causality check.
future = versions.groupby("security_id", sort=True).tail(1).copy()
```

```
future["period_end"] = pd.Timestamp("2024-12-31")
future["available_at"] = pd.Timestamp("2025-03-01", tz="UTC")
future["ingested_at"] = pd.Timestamp("2025-03-02", tz="UTC")
future["book_equity"] = 999.0
pd.testing.assert_frame_equal(table, latest_available_values(pd.concat([versions,
    ↪ future], ignore_index=True), months, ["book_equity"])["book_equity"])
print(table.to_string())
print("PASS: 5 companies, 3 versions each, 8 months; restatement, late ingestion,
    ↪ future invariance")
```

24-P02 Reference Implementation

```
import importlib.util
from pathlib import Path

import numpy as np

from quant_course.factor_research import group_percentile, ic_series, lead_return
from quant_course.factor_signals import book_to_market, cheapness_percentile

root = Path.cwd()
spec = importlib.util.spec_from_file_location("ch24", root / "code/ch24_value_factors
    ↪ .py")
ch24 = importlib.util.module_from_spec(spec); spec.loader.exec_module(ch24)
world = ch24.value_world(seed=7, n_firms=200, months=60)
    ↪       # smaller world for speed
bm = book_to_market(world["pit_book"], world["cap"])
label = lead_return(world["returns"], 1)
raw = ic_series(cheapness_percentile(bm, kind="yield"), label)["ic"].mean()
inside = ic_series(group_percentile(bm, world["industry"]), label)["ic"].mean()
assert inside > raw > 0
    ↪        # industry-relative beats raw
print(round(raw, 4), round(inside, 4))
```

How to vary the industry-level difference Δ: copy value_world and change -0.1 in np.where(industry == "Bank", -0.1, -1.0) to -1.0 + delta, using $\Delta \in \{0, 0.45, 0.9, 1.5\}$ as optional constructed sensitivities. The population Pearson approximation declines as Δ rises; finite-sample Rank IC need not be monotonic, and within-industry results may also change when the panel is regenerated. Fix the seed and mechanism, report the complete results, and do not retain only a monotone run.

24-P03 Reference implementation: use Code example 4 in Section 9. Rewrite gap as a function that takes two signal tables and returns the IC difference for all 12 calendar months; check that gap.idxmax()==2.

24-C01. With $y = (0.10, 0.02)$, the arithmetic mean yield is $\bar{y} = 0.06$. The corresponding multiples are $(10, 50)$; their mean is 30, whose reciprocal is $1/30 = 0.0333$. This is the harmonic mean $2/(1/0.10+1/0.02) = 0.0333$, below the arithmetic mean. Thus "average the multiple, then take the reciprocal" gives 3.33%, not the 6.00% from "average the yield." Averaging multiples gives more weight to the most expensive company (the one with the largest multiple). Conclusion: convert value ratios to same-direction percentiles (rank them) before combining with weights (Chapter 28), so an extreme multiple caused by a denominator close to zero does not dominate. Score using the three original categories.

24-C02 (scoring guidance and reference). Example fields: book_equity (period_end, available_at as the filing acceptance time, ingested_at, reporting currency); market_cap (month-end price times shares, with point-in-time price source); EBITDA and EV bridge items (each with its own timing); industry (historical classification versions); CFO and net_income (quality measures, aligned to the same version). Timeline: fiscal year-end → acceptance (available_at) → local ingestion → signal month-end → next-month execution → label interval → possible later restatement. Data gaps: price and shares, historical stock universe and delisting returns, historical industry classifications, XBRL before 2009, and the full restatement chain. The SEC data sets cover "as filed" financial figures from 2009 through June 2026 but do not include prices. Missing-value rules: set negative book equity and nonpositive EBITDA to NaN and report the share. Score using the four original categories.

24-N01 Complete Answer. EV is 700 million euros; FCFE is 48; FCFF/EV and FCFE/M are each 8%; mismatched FCFF/M is 9.333%, or 1.1667 times the correctly matched firm-level yield. The net-cash company's EV is 40, below its market capitalization of 100, so the correctly matched 20% yield is higher while the mismatched yield is 8%. A risk test could compare bad-state losses and risk compensation in advance; a proxy test could report the association before and after industry/size controls, including coverage. Both require frozen definitions and a holdout period. A positive IC shows only an association in the sample; risk, mispricing, and measurement proxies can all generate it.

19. Further Reading

- **Must know:** Ken French Data Library, "Description of Fama/French Factors." Read the Construction and Stocks paragraphs to understand HML's portfolio arithmetic and book-value timing.
- **Must know:** Asness, Moskowitz, and Pedersen (2013), "Value and Momentum Everywhere," printed pages 936–937; read the discussion of value definitions and the six-month lag in book values. Chapter 25 covers momentum.
- **Should know:** Fama and French (2015), Section 7, "HML: a redundant factor"; note the sample and model to which the result is limited.
- **Should know:** SEC "Financial Statement Data Sets" documentation; read the filed, accepted, and prevrpt fields and the limits of "as filed" data.
- **Advanced:** Read the original Fama–French (1992, 1993) and Lakonishok–Shleifer–Vishny (1994) papers to check the initial design of value measures. This chapter has not read these papers.

- **Optional:** Chapter 13's valuation chapter for derivations of the EV bridge and FCFE/FCFF.

20. References

Citation keys and reading scope are listed in `source_notes.md`.

- CH24_FF_FACTORS_PAGE: Kenneth R. French Data Library, "Description of Fama/French Factors." Supports HML arithmetic and the use of fiscal-year $t-1$ values, positive book equity, and July portfolio formation.
- CH24_FF5_PAGE: The same library, "Description of Fama/French 5 Factors (2x3)."
- CH24_AMP2013: Asness, C. S., Moskowitz, T. J., and Pedersen, L. H. (2013). "Value and Momentum Everywhere." *Journal of Finance* 68(3), printed pages 936–937.
- CH24_FF2015: Fama, E. F., and French, K. R. (2015). "A five-factor asset pricing model." *Journal of Financial Economics* 116(1), abstract, Section 7 (pages 12–13), and Table 8 note (page 14).
- CH24_SEC_FSDS: SEC DERA Financial Statement Data Sets page and documentation; SEC Accessing EDGAR Data; SEC website information policy.
- FRENCH: Frozen snapshot of French three-factor data (`french_factors_2026-10-05.json`).

Chapter 25 — Momentum

Through line: Turn "past winners keep winning" into precisely bounded windows, directionally explicit positions, and returns whose costs can be measured. Distinguish cross-sectional momentum (XSMOM) from time-series momentum (TSMOM, also called trend); they are different strategies. Prerequisites: Chapter 11 (returns and compounding) and Chapter 23 (cross-sectional research). After this chapter, you should be able to write the exact signal-month endpoints of 12–1 momentum without an off-by-one error; explain the difference between $\prod(1+r)-1$ and a sum; construct both a winner-minus-loser cross-sectional portfolio and a time-series trend portfolio; reduce turnover with overlapping holdings and derive the turnover formula; identify data-definition problems involving stock splits, futures rolls, and continuous contracts, and survivorship in the sample; and report results using real monthly returns for 10 industries, including negative results, costs, and crash states.

1. Chapter goals

Readers should be able to:

1. For signal month s (at month-end), write the 12–1 formation interval (months $s-11,\ldots,s-1$, 11 months in all), the skipped month (s), and the holding interval ($s+1$); explain that the 12–0 time-series momentum window of Moskowitz, Ooi, and Pedersen (2012; hereafter MOP) includes the most recent month.
2. Calculate by hand and distinguish $\prod(1+r)-1$, $\sum r$, and $\sum \ln(1+r)$, and give an example in which sum-based and compound-return rankings are reversed.
3. Construct a cross-sectional momentum portfolio (long the highest formation-return bucket and short the lowest) and a time-series momentum portfolio (go long or short each asset according to the sign of its own past return, with positions scaled by ex ante volatility). Explain net exposure (the sum of signed weights, equivalently long weights minus the absolute short weights), gross exposure (the sum of absolute weights), and the relevant benchmark for each.
4. Construct a portfolio with K months of overlapping holdings, derive the adjacent-month weight change $(w^{(t)}-w^{(t-K)})/K$, and compare turnover (the monthly sum of absolute weight changes) and costs for $K=1,3,6$ using real industry data.
5. Identify and correct three data-definition errors: false losers caused by unadjusted stock splits and dividends, false returns created by stitching futures prices across rolls, and survivorship bias from backtesting only the stocks that exist today.
6. Report 12–1, window variants, a $J\times K$ grid (including a Bonferroni threshold), time-series trend versus an always-long benchmark, and crash states for 10 industry portfolios; state clearly what cannot be concluded.

Completion criterion: Every momentum signal should include a timeline marking formation months, the skipped month, `signal_time`, `trade_time`, and the holding month. You should be able to explain what changes when the window is one month longer or shorter than the standard definition.

2. Why Quant needs this

Momentum is the factor in Part IV most easily "implemented incorrectly." Its definition fits in one sentence ("assets that rose more in the past keep rising"), but moving a window endpoint by one month, summing returns instead of compounding, failing to adjust prices for splits, stitching futures prices directly, or using a universe of only surviving companies can all change the result without producing an error. Momentum also has a clear crash risk: strategies with positive long-run means can suffer very large losses in a small number of months. Those losses cluster in rebounds after market declines, so reporting only the mean and its t-statistic is incomplete.

The chapter proceeds as follows: first make the window endpoints and compounding exact (Sections 6.1–6.2); then specify the two portfolios using overlapping holdings and volatility scaling (Sections 6.3–6.4); explain why their risks and benchmarks differ (Section 6.5); examine crash states (Section 6.6); address data definitions (Sections 6.7–6.8); and finish with a full report on 10 industry portfolios (Section 11). We use the period-by-period IC, quantile portfolios, and inference methods chosen for the dependence structure from Chapter 23; the experiment ledger in Chapter 22 constrains window searches; Chapter 27 extends the futures-roll issue to carry.

3. Intuition

An asset that rose the most over the past year tends, on average, to rise a little more next month than an asset that fell the most over the past year: this is cross-sectional momentum. If an asset's own price has trended upward over the past year, it is more likely to continue rising next month: this is time-series momentum, also called trend. The first compares assets' relative positions and is always one long and one short, with net position near zero. The second looks only at each asset on its own; if most assets are rising, the portfolio is net long and its return includes the market's overall moves.

The intuition has two further details. First, why skip the most recent month? Individual-stock monthly returns exhibit short-term reversal: last month's winners often give some of it back next month, in connection with liquidity and market microstructure (AMP 2013, printed p. 936). Thus, the "1" in 12–1 removes the most recent month from the formation period. Second, why consider the holding period? Signals update each month. Replacing the entire portfolio every month creates high turnover; instead, let each new vintage occupy $1/K$ of the target book. The replaced quantity is a vintage share, while actual turnover still depends on the difference between old and new weights. The tradeoff is a slower response (the overlapping-holdings design of Jegadeesh and Titman 1993).

Simplifying boundary: the real-data example in this chapter has only 10 industry portfolios (revised value-weighted monthly returns). They do not exhibit the microstructure reversals of individual stocks; the cross section is small; and the observations are not an executable trade series. This chapter does not support conclusions about individual stocks.

3.1 Fixing the 12–1 clock with actual calendar months

Complete worked example: compound returns month by month

The signal month is December 2025. All monthly returns in the table are split-adjusted total returns that include cash dividends. They are shown in percentage points for readability; divide by 100 before entering them in Python. Start with wealth of 1 and multiply by $1 + r$ row by row instead of adding the returns.

Return month	Monthly return (%)	Cumulative wealth (shown to six decimals)	Role in this 12–1 signal
2025-01	3	1.030000	$s - 11$, first observation
2025-02	-2	1.009400	second observation
2025-03	5	1.059870	third observation
2025-04	1	1.070469	fourth observation
2025-05	-4	1.027650	fifth observation
2025-06	6	1.089309	sixth observation
2025-07	2	1.111095	seventh observation
2025-08	-1	1.099984	eighth observation
2025-09	4	1.143984	ninth observation
2025-10	3	1.178303	tenth observation
2025-11	-2	1.154737	$s - 1$, eleventh observation
2025-12	5	Not multiplied into this signal	Most recent month s, skipped
2026-01	-6	Unknown when the signal is formed	Holding month and next-month label $s + 1$

The displayed cumulative values are for reading only; retain full precision internally at every step. The 11-observation compounded signal is $1.1547369892 - 1 = 15.4737\%$; the sum is only 15%. Including December 2025 gives 12–0: $1.1547369892 \times 1.05 - 1 = 21.2474\%$. The -6% return in January 2026 does not enter any December 2025 signal; that is the boundary imposed by the causal clock.

The general formula follows from the table: the start is $s-(12-1) = s-11$, the end is $s-1$, and the count is $(s-1)-(s-11)+1 = 11$. In the current interface, `window=11` is the number of observations and `skip=1` means to skip the most recent 1 month; do not infer 12–1 from the name and set `window=12`. Knowing the month-end input does not mean it can be traded at that same closing price. Actual `trade_time` must be a tradable time after the information becomes available, with fills, holidays, halts, and costs recorded. Here we have only a monthly-return-label proxy.

4. Mathematical foundations

Notation. Asset i's monthly simple return is $R_{i,t}$ (a unitless decimal for month t, known at the end of that month). Signal month s means the end of month s: the decision is made then, and the holding month is $s+1$. Windows use "lookback span–skip" notation: 12–1 looks back 12 months and skips the most recent 1 month, so it actually uses $12-1=11$ months of returns.

Compounded return. For a set of months $\mathcal{W}$, $F = \prod_{k\in\mathcal{W}}(1+R_{i,k}) - 1$. Expanding, $\prod_k(1+R_k)-1 = \sum_k R_k + \sum_{j<k} R_j R_k + \cdots + \prod_k R_k$; the sum is only the first-order term. When monthly returns are on the order of $\pm 5\%$, the second-order terms over a year are on the order of $0.5\times(\sum R)^2 - 0.5\sum R^2$. Log returns add: $\ln(1+R)$ gives $\ln(1+F) = \sum_k \ln(1+R_k)$.

Time-series signal. Asset i's sign is $\text{sign}(F_i) \in \{-1, 0, +1\}$. Volatility $\sigma_{i,s}$ is the annualized standard deviation estimated using returns available through month-end s. Portfolio weight is $w_{i,s} = \text{sign}(F_{i,s}) \cdot (\sigma^*/\sigma_{i,s})/S_s$, where σ^* is each asset's target annualized volatility and S_s is the number of assets with valid inputs that month.

Overlapping holdings. Form one new "vintage" (a batch of target weights) $w^{(s)} \in \mathbb{R}^N$ each month and hold it for K months. The portfolio at the end of month s is $B_s = \frac{1}{K}\sum_{j=0}^{K-1} w^{(s-j)}$.

Bear-market state. $I_s = \mathbf{1}\{\prod_{k=s-23}^{s}(1+M_k) - 1 < 0\}$, where M_k is the market total return. This uses only the 24 months through month-end s. This is the definition in Daniel and Moskowitz (2014 working paper; note to Table 3).

Frequency convention: monthly. `signal_time` is month-end; execution is at the next tradable time; the return interval is month $s+1$; all weights update monthly.

5. Formal definitions

Formation interval, skipped month, and holding interval. The formation interval contains the months used to calculate the signal. For 12–1, these are $s-11, \ldots, s-1$. The skipped month is excluded between the formation and holding intervals; for 12–1 it is month s. The holding interval is the month held after the signal is decided: month $s+1$. The French database's "prior (2–12)" corresponds to the same definition: the portfolio is formed at the end of month $t-1$ and held in month t, using formation months $t-12, \ldots, t-2$ (French momentum-factor description page).

Cross-sectional momentum. On a given date, rank assets by their formation returns, buy the highest bucket, and sell the lowest (Jegadeesh and Titman 1993 use deciles). Its direction follows relative ranks rather than the sign of each asset's absolute return; portfolio market risk still depends on the two legs' exposures.

Time-series momentum (TSMOM). Each asset's own past-return sign determines its long or short direction, and the position size is inversely proportional to ex ante volatility (Moskowitz, Ooi, and Pedersen 2012, equation (5): $\text{sign}(r_{t-12,t}) \cdot (40\%/\sigma_t)\, r_{t,t+1}$, where σ_t is the ex ante volatility known at the end of month t; the formation window includes the most recent month).

Short-term reversal. The negative predictive relation of the most recent month's return, associated with liquidity and market microstructure for individual stocks (AMP 2013). This chapter tests whether it appears in industry portfolios.

Trend. Used synonymously with time-series momentum: persistence in an asset's own price direction.

Momentum crash. Sustained or extreme negative returns in momentum strategies, concentrated in rebound months after market declines (Daniel and Moskowitz 2014).

Rebalancing and overlapping holdings. Update signals and adjust positions monthly. Overlapping holdings replace a vintage share equal to $1/K$ each month; this does not guarantee that trading volume falls by exactly $1/K$. In this course, the target weights are reset each month to the equal-weighted average, ignoring price drift; this differs from actual buy-and-hold without rebalancing.

Total return, price return, and adjusted prices. Price return includes only price changes; total return also accounts for dividends and splits. An unadjusted price after a split treats the split as a crash.

Roll and continuous contract. Before a futures contract expires, move to the next contract. A "continuous price" made by joining prices of different contracts has a jump on the roll date; that jump is not a return earned by holding any one contract.

Survivorship bias. A backtest includes only companies that still exist, omitting companies (including those later delisted) that were investable at each historical `signal_time`.

6. Derivations

6.1 Exact endpoints for 12–1

Signal month s is at month-end. The formation return is

$$F_{i,s} = \prod_{k=1}^{11}(1 + R_{i,s-k}) - 1,$$

using months $s-11, s-10, \ldots, s-1$, 11 months in all; month s is skipped; and the holding month is $s+1$. Look back 12 months from $s-11$ through s and remove the most recent 1 month: hence 12–1. MOP (2012) uses $r_{t-12,t}$ for its time-series signal, that is, $\prod_{k=0}^{11}(1 + R_{s-k}) - 1$, covering months $s-11, \ldots, s$, 12 months with none skipped. The windows differ by one month and are not the same signal. Three ways to make a one-month endpoint error are: (1) shift the window back one month ($s-12, \ldots, s-2$), dropping the most recent valid information; (2) shift it forward one month ($s-10, \ldots, s$), including the month that should be skipped; or (3) misread 12–1 as 12 months minus 1 month but still use 12 observations ($s-12, \ldots, s-1$). Row s of `formation_return(returns, window=11, skip=1)` is exactly $s-11, \ldots, s-1$. A test verifies that changing returns in month s and later does not change that row (Chapter 23, Section 9, code segment five).

6.2 Why $\prod(1+r)-1$, not $\sum r$

Returns compound over time: principal of 1 becomes $\prod(1+R_k)$ after n months, so cumulative return is that value minus 1. Whether a sum understates or overstates compounding depends on the return path: $\prod(1+R_k)-1=\sum R_k+\sum_{j<k}R_jR_k+\ldots$. In a second-order approximation, at a fixed arithmetic sum, a larger sum of squared returns lowers the compound return. A strict ranking requires additional conditions; for example, a mean-preserving spread in positive wealth factors lowers their sum of logs. Variance alone cannot determine the ranking of compound returns for arbitrary multi-month paths. The following two-month example can be calculated exactly. Asset X returns $+50\%, -50\%$: sum 0, compounded return $1.5\times 0.5-1=-25\%$. Asset Y returns $0, 0$: sum 0, compounded return 0. By the sum, X and Y tie; by compounding, Y ranks above X. Log returns agree with compounding: $\ln 1.5+\ln 0.5=-0.288=\ln 0.75$. Therefore, all signals in this chapter use compounding. If the data are log returns, sum them and then exponentiate. `formation_return` sums `log1p` for simple returns and then applies `expm1`; a missing return cell or insufficient history makes the window NaN. A missing calendar-month row violates the continuous-index requirement and raises an error.

6.3 Turnover of overlapping-holdings portfolios

Form a new batch of weights $w^{(s)}$ each month and hold it for K months; the portfolio is $B_s=\frac{1}{K}\sum_{j=0}^{K-1}w^{(s-j)}$. The difference between adjacent months is

$$B_s-B_{s-1}=\frac{1}{K}\big(w^{(s)}-w^{(s-K)}\big)$$

because B_s and B_{s-1} share the middle $K-1$ vintages and differ only by the newly added $w^{(s)}$ and the just-removed $w^{(s-K)}$. Thus target-weight turnover is

$$\text{turn}_s=\sum_i|B_{i,s}-B_{i,s-1}|=\frac{1}{K}\big\|w^{(s)}-w^{(s-K)}\big\|_1.$$

Each vintage has gross exposure of 2 (1 per leg), so $\|w^{(s)}-w^{(s-K)}\|_1\le 4$ and turnover is bounded above by $4/K$. For $K=1$, it is the change between adjacent months. The more persistent the signal, the more slowly $\|w^{(s)}-w^{(s-K)}\|_1$ tends to grow with K, so turnover falls more. In the real industry data, mean turnover for $K=1,3,6$ is 0.904, 0.518, and 0.364 (Section 11). For $K=3$, mean $\|\cdot\|_1$ is about $3\times 0.518=1.55$, greater than 0.904 for $K=1$: the portfolio from three months earlier differs more than last month's portfolio, but only one-third of it needs to be revised each month. The cost is a slower response: with a K-month holding period, the portfolio contains, on average, $(K-1)/2$ months of stale information.

6.4 Time-series momentum exposure and benchmark

Weights are $w_{i,s} = \text{sign}(F_{i,s})\,(\sigma^*/\sigma_{i,s})/S_s$. Gross exposure is $\sum |w_i| = \frac{\sigma^*}{S}\sum_i |\text{sign}(F_i)|/\sigma_i$. It can be simplified to a sum of inverse volatilities only when every valid signal is nonzero; a zero signal has zero weight. Net exposure is $\sum w_i = \frac{\sigma^*}{S}\sum_i \text{sign}_i/\sigma_i$ and depends jointly on the shares and volatilities of long and short assets. Hand calculation (Section 8.3): three assets, signals $(+1, -1, +1)$, volatilities $(10\%, 20\%, 40\%)$, and $\sigma^* = 40\%$. The positions before division are $(4, -2, 1)$; divide by 3 to get $(1.333, -0.667, 0.333)$, gross exposure 2.333, and net exposure 1.0. Net exposure describes an imbalance in nominal long and short positions, but does not by itself establish the size or sign of market beta; market beta also depends on each asset's exposure. Most assets in this industry sample carry equity-market risk. When most trend signals are positive, the portfolio is net long. So evaluate time-series momentum against an "always long, scaled the same way" benchmark: $r^{\text{TS}}_{s+1} = \sum_i w_{i,s} r_{i,s+1}$ versus $r^{\text{AL}}_{s+1} = \sum_i (\sigma^*/\sigma_{i,s}/S)\, r_{i,s+1}$. MOP (2012) also regressed on an always-long benchmark ("a positive alpha in 90% of the cases"). In the real industry data, the time-series 12–0 monthly mean is 1.391%, versus 1.779% for always long with the same scaling; the difference is -0.389% ($t = -1.05$). In this sample, trend filtering did not outperform simply holding long.

6.5 Cross-sectional versus time-series momentum

Cross-sectional momentum is relative: equal long and short legs give zero net exposure and balance nominal capital. Its market beta is $\sum_i w_i \beta_i$ and is not necessarily zero just because $\sum_i w_i = 0$. It mainly expresses relative strength, but may still bear market-direction risk. Time-series momentum is absolute: each asset is compared only with its own past, so the portfolio can be entirely long or entirely short and its net exposure changes with market conditions. Both use past-return signals, but their portfolio properties differ; their correlation is only 0.42 in the industry sample, and their benchmarks differ: cross-sectional momentum is compared with zero (or equal weight), while time-series momentum is compared with always long.

6.6 Crash states: conditional means

Classify each signal month as bear or non-bear according to whether the market's cumulative return over the prior 24 months is negative. Let the next-month conditional means of cross-sectional momentum be $\bar{r}_1$ and $\bar{r}_0$, and let the bear-state share be p. Then the unconditional mean is $p\bar{r}_1 + (1-p)\bar{r}_0$. In the industry data, the bear state occurs 16.2% of the time (191 months), with a mean of -0.179% per month (standard deviation 7.05%). The other 987 months have a mean of $+0.644\%$ (standard deviation 4.37%). Volatility is higher and the mean is negative in the bear state; 7 of the 10 worst months occur after a bear state, while the market rose in 8 of those months (table in Section 11). This direction is consistent with Daniel and Moskowitz's (2014, printed p. 8) description of individual-stock winner-minus-loser portfolios, but this chapter's sample contains 10 industry portfolios, not that paper's individual-stock deciles, so this is not a replication.

6.7 Prices, splits, dividends, and total return

Let split ratio ρ convert 1 old share into ρ new shares, P^{post} be the post-split price per share, D the cash dividend per new share, and P^{pre} the pre-split price. Total return is $R = (\rho P^{\text{post}} + \rho D)/P^{\text{pre}} - 1$; the unadjusted price change is $P^{\text{post}}/P^{\text{pre}} - 1$. The constructed example in Chapter 3 has $P^{\text{pre}} = 100$, $\rho = 2$, $P^{\text{post}} = 51$, and $D = 1$: total return is $(102 + 2)/100 - 1 = +4\%$, while the unadjusted price change is -49%. A momentum strategy using unadjusted prices would treat this stock as a loser. This month's return is understated by $4 - (-49) = 53$ percentage points. The difference over the entire formation signal must also be multiplied by the wealth factor from the other months in the window (see below); it cannot be treated as 49 percentage points.

6.8 Roll stitching: a continuous price is not a return

Here and in Section 8.5 and 25-E06, chaining within-contract price ratios defines a return index: ignore fees and collateral interest, fully collateralize each segment at its starting contract price, and adjust notional exposure with wealth before multiplying the wealth factors. A fixed one-contract position instead requires currency P&L for each segment and an explicit capital denominator; it is not the same capital return.

Contract A is priced at 100 and 102 at t_0, t_1. At t_1 it is rolled into contract B, priced at 106 and 107 at t_1, t_2. Joining the prices gives 100, 102, 106, 107 and "returns" of $+2\%, +3.92\%, +0.94\%$. The middle $+3.92\%$ is $106/102 - 1$, comparing two different contracts: A is closed at 102 at t_1, while B is opened at 106 at the same time; the holder did not earn that 3.92%. The returns actually available are $+2\%$ while holding A and $107/106 - 1 = +0.94\%$ while holding B. If B's price is below A's (spot premium / downward-sloping futures curve), stitching creates a false negative return. AMP (2013, Appendix A.5) calculates commodity-futures returns by compounding daily excess returns on the most liquid contract, rather than stitching prices. A time-series momentum signal on such prices would treat the roll jump as a trend; Chapter 27 decomposes roll returns separately.

7. Financial interpretation

This chapter does not decide where momentum returns come from. Jegadeesh and Titman (1993) report winner-minus-loser returns for U.S. individual stocks in 1965–1989; Asness, Moskowitz, and Pedersen (2013) use the same simple measure across multiple asset classes; Moskowitz et al. (2012) report time-series momentum that persists for 1 to 12 months and partially reverses across 58 futures. This chapter has not read the original papers in enough detail to restate their explanations (such as underreaction to information or risk compensation). The Quant requirement is that, whatever the explanation, the signal be constructible at `signal_time`, costs be included, and crashes be reported separately.

Industry momentum differs from individual-stock momentum. The skipped month in 12–1 is used for individual stocks to avoid short-term reversal (AMP 2013, printed p. 936). AMP also notes stronger momentum without skipping the most recent month for equity-index futures and currencies (printed p. 937). An industry portfolio is a value-weighted average of many stocks, so bid-ask spreads and liquidity-related microstructure effects are averaged out. The industry results here are consistent with that distinction: the 1–0 window (last month's return) has a long-short monthly mean of +0.740% ($t = 5.61$), while 1–1 (the historical specification name; `window=1, skip=1`, using only month $s - 1$) has a mean of −0.028% ($t = -0.21$); the 12–0 mean of 0.621% is greater than the 12–1 mean of 0.519%. This does not show that individual-stock momentum should not skip a month. It shows only that skipping did not help in these 10 industry portfolios.

Turnover is a major source of momentum costs. Monthly target-weight turnover for cross-sectional 12–1 is 0.904 (gross exposure 2). Assuming 10 bp per unit of turnover gives a monthly cost of 9 bp and reduces 0.519% to 0.427%. $K = 3$ reduces turnover to 0.518 and gives a net return of 0.423%; t rises from 2.99 to 3.06. The 10 bp per unit is an assumption, not an estimate for industry portfolios; stock-borrow and financing costs are not included.

Report crash risk alongside the mean. The cumulative compound return of cross-sectional 12–1 on the 10 industries has a maximum drawdown of about 64% at its worst point. Its 10 worst monthly losses range from 13.6% to 34.8%, with 7 following a bear state. A strategy with a positive mean may still be impossible for an investor to hold.

The time-series trend return contains market exposure. In the industry sample, trend signals are positive in most asset-months (67.4% of assets are long); average net exposure is 1.05, so the 1.391% monthly mean includes an equity risk premium. Relative to always long, the mean difference is -0.389% per month with t=-1.05; no statistically clear positive increment is observed. This is not an estimated regression alpha.

7.1 Three questions and three mechanisms to test

Cross-sectional momentum asks: "On the same date, do past winners outperform past losers next month?" It needs a common asset universe and buys winners while selling losers. Even if every stock fell in the past, it can still take both sides as long as relative ranks are available. Time-series momentum asks: "Does this asset's own past direction continue next month?" It uses zero return, or another explicitly stated excess-return threshold, as the cutoff; when all assets point the same way, the portfolio can be net long or net short. Short-term reversal asks: "Do last month's winners give back their gains?" If the signal is defined as the negative of the most recent month's return, a higher value means a larger recent decline. This is a different specification from 12–1, which excludes the latest month; one name cannot stand in for the other.

Underreaction is one hypothesis: information takes time to enter prices, so past trends continue. It can be tested using information events defined in advance and subsequent drift. Risk compensation is another: winners and losers have different state exposures, and returns come with losses during some rebounds. The crash-state description in Section 11 can motivate such a test, but cannot prove the cause. Measurement proxies may also arise from industry trends, stale trading, or liquidity; they require the corresponding universe and controls for available quotes.

All three can coexist with positive historical returns; past profits cannot decide among these explanations. These are research questions, not extensions of the original empirical findings.

Skipping a month is a specification set in advance, with the cost of giving up the most recent information; whether it helps depends on the asset and data definition. The positive 1–0 result for industry portfolios and the negative time-series difference versus always long are both retained. With individual stocks, futures, or currencies, inputs must be defined again. Do not choose the main specification after looking back at whichever of the 19 variants in this chapter performed best.

8. Numerical worked examples: calculate each step

All numbers are generated by `hand_examples()` and independently checked by `qa/a5/claude/recompute/ch25_recompute.py`. These are constructed teaching examples.

8.1 The 12–1 window: returns over fourteen months

An asset's monthly simple returns over months 1–14 are $0.03, -0.02, 0.05, 0.01, -0.04, 0.06, 0.02, -0.01, 0.04, 0.03, -0.02, 0.05, -0.06, 0.02$. The signal is formed at the end of month 13, and month 14 is the holding month. The 12–1 formation interval is months 2–12, 11 months; month 13 (-6%) is skipped.

- Compounded return: $\prod_{k=2}^{12}(1+R_k)-1 = 17.716\%$; sum: 17.0%; difference: 0.716 percentage points.
- Shift the window one month earlier (months 1–11): 15.474%; shift it one month later (months 3–13, including the skipped month): 12.911%; include month 13 in a 12-month window (months 2–13, the 12–0 window): 10.653%.

Interpretation: for the same asset, the signal differs by 2 to 7 percentage points across three erroneous windows. The 12–0 signal is 7 percentage points lower than 12–1 because the skipped month was -6%. If many assets give back gains in the latest month in the same way, 12–0 will rank them lower.

8.2 Cross-sectional momentum: six assets

The six assets' 12–1 formation returns are $(A, B, C, D, E, F) = (30\%, 12\%, 5\%, -2\%, -10\%, -25\%)$, and their next-month returns are $(2\%, 3\%, 1\%, 0\%, 1\%, -2\%)$. Take the top two and bottom two: long A and B, short F and E. The long-leg average is $(2+3)/2 = 2.5\%$; the short-leg average is $(-2+1)/2 = -0.5\%$; long-short return is $2.5-(-0.5) = 3.0\%$. Each leg has capital of 1, so gross exposure is 2 and net exposure is 0.

8.3 Time-series momentum: volatility scaling

Three assets have signals $(+1, -1, +1)$, ex ante annualized volatilities $(10\%, 20\%, 40\%)$, and target volatility 40%. Individual positions are $(4, -2, 1)$; divide by $S = 3$ to get weights (1.333, $-0.667, 0.333$). Gross exposure is 2.333, and net exposure is 1.0. Next-month returns are (1%, $-2\%, 3\%$); asset contributions are $(1.333\%, 1.333\%, 1.0\%)$, for a portfolio return of 3.667%. Interpretation: the low-volatility asset gets the largest weight; the portfolio is net long 1.0, so it is not neutral.

8.4 Stock split and dividend

Constructed example from Chapter 3: the pre-split price is 100, 1 share splits into 2 shares (a U.S. 2-for-1 split), the post-split price is 51, and the dividend is 1 per new share. Total return is $(2 \times 51 + 2 \times 1)/100 - 1 = 4\%$; the unadjusted price change is $51/100 - 1 = -49\%$.

8.5 Futures roll

Contract A: 100, 102; contract B at the roll: 106, 107. The stitched price series is $100, 102, 106, 107$, with stitched returns of 2.00%, 3.92%, 0.94%. The available returns are 2.00% (holding A) and 0.94% (holding B); 3.92% at the roll is a false return.

8.6 Overlapping holdings and turnover

There are two assets and six months of vintage weights: $(0.5, -0.5)$ for the first two months, then a reversal to $(-0.5, 0.5)$ from month 3 onward, with the later months unchanged. For $K = 1$, adjacent-month turnover is $0, 2, 0, 0, 0$; turnover in the reversal month is 2.0. For $K = 3$, portfolio B is the average of the latest three vintages. The table starts when three vintages are available: month 3 is $\frac{1}{3}(0.5 + 0.5 - 0.5) = 0.167$; month 4 is $\frac{1}{3}(0.5 - 0.5 - 0.5) = -0.167$; month 5 is -0.5; and month 6 is -0.5 (weights for the first asset; the second asset has the opposite sign). Turnover shown in the table is $0.667, 0.667, 0$. If the portfolio had remained $(0.5, -0.5)$ through month 2, its month-2 weight would already be $(0.5, -0.5)$, so there would also be turnover of 0.667 from month 2 to 3. The three months over which the reversal is spread would trade $0.667 \times 3 = 2.0$, the same total as one 2.0 trade for $K = 1$: the total quantity is unchanged, but the monthly trades are smaller. If the signal repeatedly reverses, $K = 3$ can offset some round trips and reduce total turnover. In the real industry data, mean turnover for $K = 1, 3, 6$ is 0.904, 0.518, and 0.364 (Section 11).

8.7 Bear-market state

All market monthly returns are +1% except month 3, which is −35%. At signal month $s = 24$ (end of month 24), the window runs from month 1 through 24 and includes month 3; cumulative return is $1.01^{23} \times 0.65 - 1 = -18.3\% < 0$, so the state is 1. At $s = 26$, the window from month 3 through 26 still includes month 3, so the state is 1. At $s = 27$, the window from month 4 through 27 no longer includes month 3; cumulative return is $1.01^{24} - 1 = +26.97\%$, so the state is 0. The state flips in the first month after the window has rolled past the crash month.

9. Python implementation

The window is calculated by `formation_return` in `factor_signals.py` (introduced in Chapter 22); portfolio construction and costs are in the new module `momentum.py`; cross-sectional bucketing, IC, and Newey–West inference reuse Chapter 23's `factor_research.py` .

Function	Input	Output and boundary
`formation_return`	Monthly return panel with a continuous `PeriodIndex`, `window`, `skip`	Compounded formation return. Signal month s uses months $s - skip - window + 1, \ldots, s - skip$; a missing return cell or an out-of-range window gives NaN; a missing calendar-month row raises an error; returns below −100% raise an error.
`cross_sectional_momentum_weights`	Return panel, window, skip, bucket count, holding months K	Average of the latest K monthly vintages (top bucket $+1/n$, bottom bucket $-1/n$); rows without enough history are NaN.
`blend_vintages`	Weight panel, K	Rolling average; fewer than K vintages gives NaN.
`time_series_sign`	Excess-return panel, window, skip	$+1/-1/0$; any missing observation in the window gives NaN.
`ex_ante_volatility`	Return panel, window 36, minimum 12	Annualized rolling standard deviation; a teaching substitute, not the daily exponentially weighted estimate in the original paper.
`tsmom_weights`	Sign, volatility, target volatility	$\text{sign} \cdot \sigma^*/\sigma/S$; nonpositive volatility or target raises an error.
`strategy_returns`	Weights (at signal month), returns	Month-s weights times month-$s+1$ returns; a missing return for a nonzero weight raises an error.
`net_of_costs`	Gross returns, weights, unit cost	Gross return minus $c \times \sum \lvert\Delta w\rvert$ (target to target, without drift).
`trailing_bear_state`	Market total-return series, window	Mark 1 when the past 24 months (including the current month) compound to a negative return; insufficient window gives NaN.

Default offline entry point (run from the I3 reader-package root):

```
.venv/bin/python run_notebook.py notebooks/ch25_momentum.ipynb
```

The default is `RUN_FRENCH=False`; it displays windows, compounding, CS/TS weights, and the 24-by-6 vintage identity with seed 2 ($K = 1, 3, 6$). All five market-data cells are skipped. By default, it does not import the ch25 module, calculate stock-split or futures-roll examples, or build the historical grid. The runner also writes an execution copy and status to `runs/execution_<stem>_<uuid>/`. The notebook's English-language layer and the package-wide paths are assembled in BIL23. The following full historical CLI and test references are not the default offline command:

```
.venv/bin/python code/ch25_momentum.py
.venv/bin/python -m pytest tests/test_ch25_momentum.py tests/test_ch25_workflow.py -q
```

Frequency is monthly; `signal_time` is month-end; execution is at the next tradable time; the return interval is month $s + 1$. French industry data and FF3 were read from revised, hash-checked files. Unit trading cost of 10 bp is an assumption, not an estimate.

Code segment one: 12–1 window and three erroneous windows (Section 8.1).

```
import numpy as np
import pandas as pd

from quant_course.factor_signals import formation_return

r = [0.03, -0.02, 0.05, 0.01, -0.04, 0.06, 0.02, -0.01, 0.04, 0.03, -0.02, 0.05,
    ↪ -0.06, 0.02]
frame = pd.DataFrame({"a": r}, index=pd.period_range("2020-01", periods=14, freq="M")
    ↪ )
s = pd.Period("2021-01", "M")                                    # month 13: signal
    ↪ at the end of this month
f121 = formation_return(frame, window=11, skip=1).loc[s, "a"]    # months 2..12
f120 = formation_return(frame, window=12, skip=0).loc[s, "a"]    # months 2..13,
    ↪ nothing skipped (time-series style)
early = np.prod(1 + np.array(r[0:11])) - 1                      # months 1..11
late = np.prod(1 + np.array(r[2:13])) - 1                       # months 3..13
assert abs(f121 - 0.177159) < 1e-6 and abs(f120 - 0.106530) < 1e-6
assert abs(early - 0.154737) < 1e-6 and abs(late - 0.129112) < 1e-6
assert abs(sum(r[1:12]) - 0.17) < 1e-12                          # the sum, 0.716
    ↪ percentage points lower
print(round(f121, 4), round(f120, 4), round(early, 4), round(late, 4))
```

Code segment two: sum and compound return produce opposite rankings (Section 6.2).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import rank_cross_section
from quant_course.factor_signals import formation_return

idx = pd.period_range("2020-01", periods=2, freq="M")
returns = pd.DataFrame({"X": [0.50, -0.50], "Y": [0.0, 0.0]}, index=idx)
compound = formation_return(returns, window=2, skip=0).iloc[-1]
total = returns.sum()
assert np.isclose(total["X"], 0.0) and np.isclose(total["Y"], 0.0)
    ↪            # sums tie
assert np.isclose(compound["X"], -0.25) and np.isclose(compound["Y"], 0.0)
    ↪            # compounding separates them
assert np.isclose(np.log1p(returns["X"]).sum(), np.log(0.75))
    ↪            # log returns add up to the compound return
assert list(rank_cross_section(compound.to_numpy())) == [1.0, 2.0]
    ↪            # X is the loser
print(compound.round(4).to_dict())
```

Code segment three: six-asset cross-sectional and time-series weights (Sections 8.2 and 8.3).

```
import numpy as np
import pandas as pd

from quant_course.factor_research import quantile_long_short_weights
from quant_course.momentum import tsmom_weights

idx = pd.period_range("2026-06", periods=1, freq="M")
names = list("ABCDEF")
formation = pd.DataFrame([[0.30, 0.12, 0.05, -0.02, -0.10, -0.25]], index=idx,
    ↪ columns=names)
w = quantile_long_short_weights(formation, 3).iloc[0]                  # terciles of
    ↪ 6 names: top 2 long, bottom 2 short
nxt = np.array([0.02, 0.03, 0.01, 0.00, 0.01, -0.02])
np.testing.assert_allclose(w.to_numpy(), [0.5, 0.5, 0, 0, -0.5, -0.5])
assert abs(w.to_numpy() @ nxt - 0.03) < 1e-12 and abs(w.sum()) < 1e-12 and abs(w.abs
    ↪ ().sum() - 2.0) < 1e-12

sig = pd.DataFrame([[1.0, -1.0, 1.0]], index=idx, columns=list("xyz"))
vol = pd.DataFrame([[0.10, 0.20, 0.40]], index=idx, columns=list("xyz"))
tw = tsmom_weights(sig, vol, target_vol=0.40).iloc[0]
np.testing.assert_allclose(tw.to_numpy(), [4 / 3, -2 / 3, 1 / 3])
assert abs(tw.abs().sum() - 7 / 3) < 1e-12 and abs(tw.sum() - 1.0) < 1e-12
    ↪             # gross 2.33, net 1.0: not market neutral
assert abs(tw.to_numpy() @ np.array([0.01, -0.02, 0.03]) - 0.036667) < 1e-6
print(tw.round(3).to_dict())
```

Code segment four: turnover formula for overlapping holdings, $\text{turn}_s = \|w^{(s)} - w^{(s-K)}\|_1 / K$ (Section 6.3).

```
import numpy as np
import pandas as pd

from quant_course.momentum import blend_vintages

rng = np.random.default_rng(2)
idx = pd.period_range("2020-01", periods=24, freq="M")
vint = pd.DataFrame(rng.choice([-0.5, 0.0, 0.5], size=(24, 6)), index=idx)          #
    ↪ arbitrary vintage weights
for K in (1, 3, 6):
    blended = blend_vintages(vint, K)
    turnover = blended.diff().abs().sum(axis=1, min_count=1)
    formula = pd.Series([np.abs(vint.iloc[i] - vint.iloc[i - K]).sum() / K for i in
    ↪ range(len(vint))], index=idx)
    np.testing.assert_allclose(turnover.iloc[K:].to_numpy(), formula.iloc[K:].
    ↪ to_numpy(), atol=1e-12)
    assert turnover.iloc[:K].isna().all()                                          # no
    ↪ turnover before K vintages exist
print("turnover = ||w_s - w_(s-K)||_1 / K holds for K = 1, 3, 6")
```

Code segment five: stock split, dividend, and futures roll (Sections 8.4 and 8.5).

```
pre, ratio, dividend, post = 100.0, 2.0, 1.0, 51.0
assert abs((ratio * post + ratio * dividend) / pre - 1 - 0.04) < 1e-12 and abs(post /
    ↪ pre - 1 + 0.49) < 1e-12

a_prices, b_prices = [100.0, 102.0], [106.0, 107.0]
spliced = a_prices + b_prices
returns_spliced = [spliced[i + 1] / spliced[i] - 1 for i in range(3)]
tradable = [a_prices[1] / a_prices[0] - 1, b_prices[1] / b_prices[0] - 1]
assert abs(returns_spliced[1] - (106 / 102 - 1)) < 1e-15 and abs(returns_spliced[1] -
    ↪ 0.039216) < 1e-6      # fake return at the roll
assert abs(tradable[0] - 0.02) < 1e-15 and abs(tradable[1] - (107 / 106 - 1)) < 1e-15
print([round(x, 4) for x in returns_spliced], [round(x, 4) for x in tradable])
```

Code segment six: bear-state window boundary (Section 8.7) and "no future information" for industry weights.

```
import numpy as np
import pandas as pd

from quant_course.momentum import cross_sectional_momentum_weights,
    ↪ trailing_bear_state

idx = pd.period_range("2000-01", periods=30, freq="M")
m = pd.Series(0.01, index=idx); m.iloc[2] = -0.35                              # month 3
    ↪ is the crash month
state = trailing_bear_state(m, window=24)
assert np.isnan(state.iloc[22]) and state.iloc[23] == 1.0 and state.iloc[25] == 1.0
    ↪ and state.iloc[26] == 0.0

rng = np.random.default_rng(4)
returns = pd.DataFrame(rng.normal(0.01, 0.05, (48, 10)), index=pd.period_range("
    ↪ 2010-01", periods=48, freq="M"))
```

```
base = cross_sectional_momentum_weights(returns, window=11, skip=1, n_buckets=5,
    ↪ hold_months=3)
shocked = returns.copy(); shocked.iloc[30:] = 0.3
changed = cross_sectional_momentum_weights(shocked, window=11, skip=1, n_buckets=5,
    ↪ hold_months=3)
pd.testing.assert_frame_equal(base.iloc[:31], changed.iloc[:31])          # months
    ↪ >= 30 cannot change weights decided up to month 30
print(state.iloc[[22, 23, 25, 26]].tolist())
```

10. Reading the figures

Figure 25-1: Window timeline. The horizontal axis is calendar month 1 through 15, with the signal formed at the end of month 13. Row one: cross-sectional 12–1 uses months 2–12 (blue, 11 months), skips month 13 (hatched), and holds in month 14 (orange). Row two: time-series 12–0 uses months 2–13 (purple, 12 months) and does not skip a month. Row three: a common error shifts the window one month earlier (gray, months 1–11). Interpretation: the three rows give different signals.

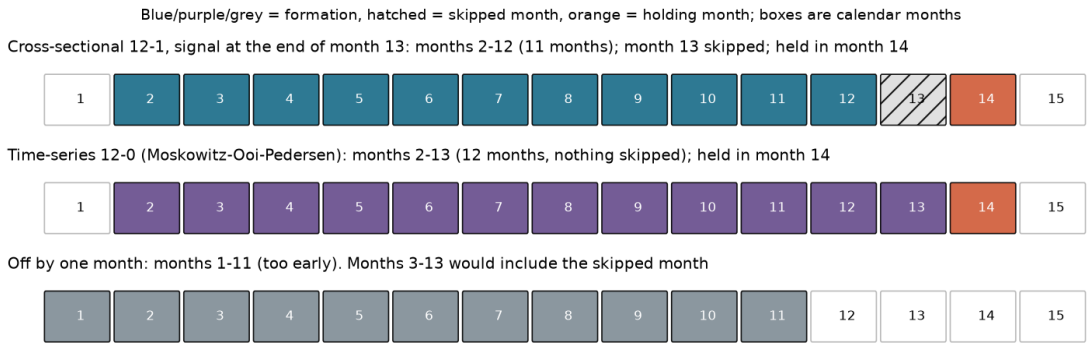


Figure 25-1: 12–1 and 12–0 window endpoints

Figure 25-2: Cumulative portfolio returns and worst months for 10 industries. The left panel shows compound wealth (log scale, starting at 1) from July 1927: cross-sectional 12–1 long-short ($K = 1$ and $K = 3$, gross returns, gross exposure 2), time-series 12–0 (excess returns, volatility-scaled, gross exposure about 2.5), and always long with the same scaling. Interpretation: cross-sectional long-short rises over the long run but by less; time-series trend moves in the same direction as always long, and always long is higher after 2000. The right panel shows monthly cross-sectional 12–1 long-short returns (percent per month), with orange dots marking the 10 worst months. The sample is revised industry portfolios, not executed trades.

Figure 25-3: t-statistics for the $J \times K$ grid. Rows are formation spans $J \in \{3, 6, 9, 12\}$ (using months $s - J + 1, \ldots, s - 1$; $J = 12$ is standard 12–1); columns are holding periods $K \in \{1, 3, 6, 12\}$. Each cell shows the Newey–West t-statistic (lag $K - 1$) for a five-quantile long-short portfolio (two industries per bucket). The nominal two-sided normal Bonferroni threshold for 16 strategies is $z = 2.955$. Valid family-wise error-rate control requires calibrated marginal p-values; this grid retains the NW/normal approximation. Lag $K - 1$ is a frozen specification, not a sufficient bandwidth for every dependence structure. Interpretation: only $(12, 1)$ at 3.64, (12,

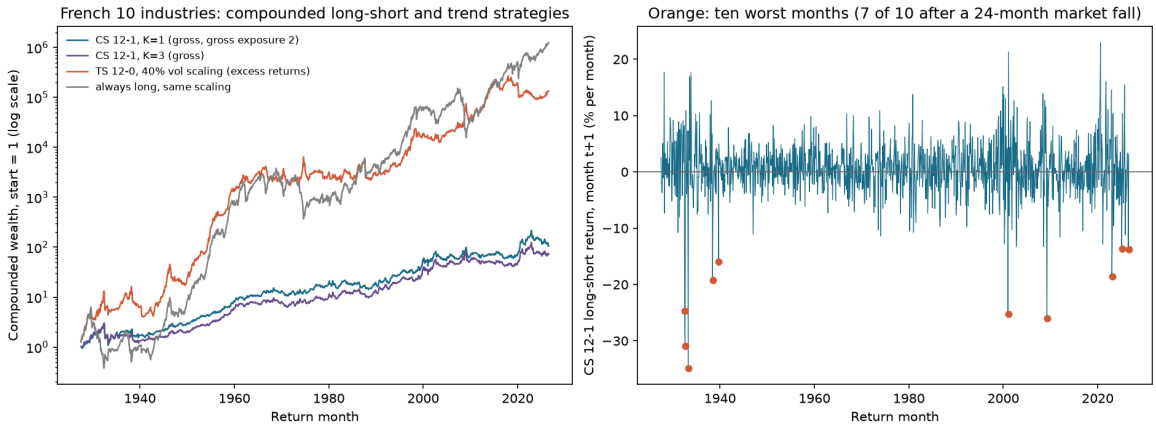


Figure 25-2: Cumulative cross-sectional, time-series, and always-long returns; worst cross-sectional months

3) at 3.43, and $(9, 6)$ at 3.18 exceed the threshold; $(9, 3)$ at 2.94 is just below it; short formation spans and short holding periods are not significant.

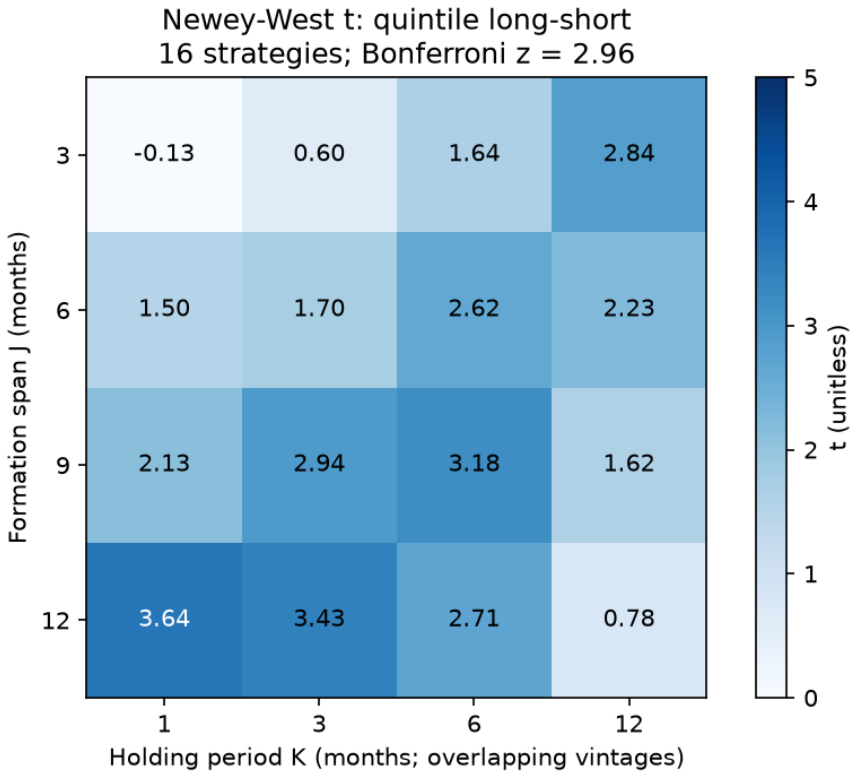


Figure 25-3: t-statistics by formation span and holding period

Figure 25-4: Crash states. Left: classify signal months by whether the market's cumulative total return over the prior 24 months was negative; show the next-month average cross-sectional 12–1 long-short return: +0.644% across 987 non-bear months and −0.179% across 191 bear months (vertical bars are plus or minus two standard errors). Right: each dot is a month, with the market's total return on the horizontal axis and the long-short return on the vertical axis; bear-state observations are orange. Interpretation: the largest losses occur in months of strong market gains (the lower right), often after a bear state.

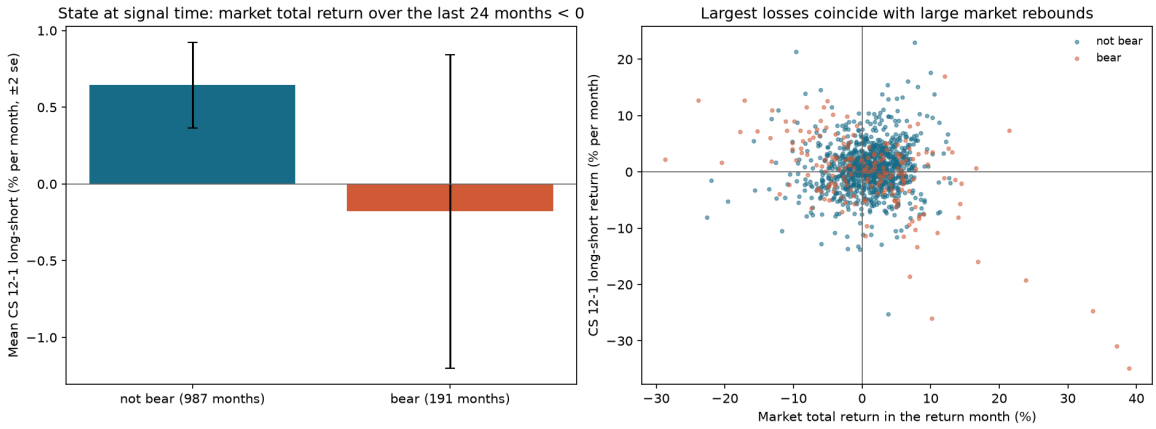


Figure 25-4: Bear states and long-short returns

11. Historical market case

Data. The same frozen French 10-industry value-weighted monthly returns as Chapters 22–24 (processed CSV SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`, July 1926 through August 2026, 1,202 months) and FF3 Mkt-RF and RF (processed SHA-256 `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`). The industry portfolios are revised published returns, 10 assets, not individual stocks and not an execution series. No new data were downloaded for this chapter.

Cross-sectional 12–1 (five quantiles, two industries per bucket, equal-weighted, formed monthly). Signal months run from June 1927 through July 2026. There are no ties in 12–1 formation returns in any of the 1,191 months with formation returns, so each bucket has exactly two industries. The one-month variants (1–0 and 1–1) each have ties in 60 months; ties receive average ranks, so an extreme bucket may contain 1 or 3 industries, but each long and short leg still sums to 1.

K	Gross mean (% per month)	Gross t (NW lag $K-1$)	Net mean (10 bp per unit)	Net t	Mean turnover
1	0.519	3.64	0.427	2.99	0.904
3	0.474	3.43	0.423	3.06	0.518
6	0.347	2.71	0.308	2.40	0.364

Averaging conventions. For K=1/3/6, gross returns have 1190/1188/1185 valid months and net returns have 1189/1187/1184. The first valid target has no prior comparable weight, so turnover and net return are missing. Mean turnover uses the valid rows of weight differences themselves and may include a final row without a next-month return label. Thus gross mean minus net mean need not equal 10 bp times this turnover column. Align gross returns and turnover to the same valid months when comparing costs. This table preserves each historical series' own convention; do not infer a new cost or revise history from the rounded displays.

Monthly volatility is 4.9% for $K = 1$; annualized gross Sharpe is 0.37 for $K = 1$, and net Sharpe is 0.30. Net Sharpe for $K = 3$ is 0.31, higher than for $K = 1$ or $K = 6$. Costs are turnover

multiplied by a unit cost; borrowing, financing, and capacity are omitted. The 10 bp value is assumed.

Window variants ($K = 1$).

Window	Months	Mean (% per month)	t
12–1 (months $s-11..s-1$)	1190	0.519	3.64
12–0 (months $s-11..s$)	1190	0.621	4.41
1–0 (month s)	1201	0.740	5.61
1–1 (month $s-1$)	1200	−0.028	−0.21
6–1 (months $s-5..s-1$)	1196	0.209	1.50
3–1 (months $s-2..s-1$)	1199	−0.018	−0.13

This table has six rows. Combined with the 16 strategies in the $J \times K$ grid, it includes 12–1, 6–1, and 3–1 in both tables; this section evaluates 19 distinct cross-sectional variants in total (the separately reported time-series evaluations are not part of this count and must still be included in the complete research ledger). Under Chapter 22's ledger, the two-sided Bonferroni threshold for 19 tests is $z = 3.01$. The values above it are 1–0 (5.61), 12–0 (4.41), 12–1 (3.64), $(12, 3)$ (3.43), and $(9, 6)$ (3.18); the others are below. The 1–0 (last-month-return) result is positive at the industry level (no short-term reversal); 1–1, which uses only month $s-1$, is about zero.

$J \times K$ grid (NW t-statistics; monthly mean in % in parentheses).

$J \backslash K$	1	3	6	12
3	−0.13 (−0.02)	0.60 (0.06)	1.64 (0.13)	2.84 (0.17)
6	1.50 (0.21)	1.70 (0.22)	2.62 (0.28)	2.23 (0.19)
9	2.13 (0.30)	2.94 (0.39)	3.18 (0.39)	1.62 (0.15)
12	3.64 (0.52)	3.43 (0.47)	2.71 (0.35)	0.78 (0.08)

Time-series momentum (excess returns; 40% target annual volatility per asset; volatility is the standard deviation over the prior 36 months, with at least 12 months).

Portfolio	Mean (% per month)	t	Monthly SD (%)	Annualized Sharpe
TS 12–0 (includes most recent month)	1.391	5.46	8.79	0.55
TS 12–1 (skips most recent month)	1.197	4.66	8.86	0.47
Always long (same scaling)	1.779	5.79	10.61	0.58

Portfolio	Mean gross exposure	Mean net exposure
TS 12–0 (includes most recent month)	2.50	1.05
TS 12–1 (skips most recent month)	2.50	1.01
Always long (same scaling)	2.50	2.50

TS 12–0 is 0.389% per month below always long (difference-series $t = -1.05$). Monthly returns for cross-sectional 12–1 ($K = 1$) and time-series 12–0 have correlation 0.42. On average, 67.4% of asset signals are long.

Crash states. There are 191 signal months (16.2% of months with a defined state) in which the market's total return (Mkt-RF plus RF) over the past 24 months is negative. The next-month long-short mean is −0.179%, with standard deviation 7.05%; the remaining 987 months have mean +0.644% and standard deviation 4.37%. The 10 worst months are:

Signal month → return month	Long-short (%)	Bear state	Market total return in return month (%)
1933-03 → 1933-04	−34.83	Yes	+38.91
1932-07 → 1932-08	−30.90	Yes	+37.15
2009-03 → 2009-04	−25.97	Yes	+10.18
2000-12 → 2001-01	−25.21	No	+3.79
1932-06 → 1932-07	−24.73	Yes	+33.63
1938-05 → 1938-06	−19.23	Yes	+23.89
2022-12 → 2023-01	−18.55	Yes	+6.96
1939-08 → 1939-09	−15.94	Yes	+16.90
2026-06 → 2026-07	−13.80	No	−0.28
2025-01 → 2025-02	−13.63	No	−2.11

Seven follow a bear state; the market's total return is positive in eight return months. The maximum drawdown of cross-sectional 12–1 ($K = 1$, gross) is 64.3%.

What can and cannot be said. We can say that 12–1 cross-sectional long-short returns were positive and significant on 10 industry portfolios over 1927–2026 ($t = 3.64$, gross), with $t = 2.99$ after assumed costs; turnover falls as K rises; results vary substantially by window and holding period, and only some long-formation/short-holding cells exceed the multiple-testing threshold; and the worst months cluster in rebounds after bear states. We cannot say: (1) that individual-stock momentum exists or does not exist, because the sample contains 10 industry portfolios; (2) that skipping a month is useless, because industry portfolios lack individual stocks' microstructure reversals; (3) that post-2000 weakening is significant: Chapters 22 and 23 report $t = 1.70$ for the same data over 2000–2026; (4) that time-series trend beats always long—it does not in this sample; (5) that the net return is achievable, because 10 bp is assumed and borrowing, financing, and capacity are omitted; or (6) that the link between crashes and market rebounds has been proved—this is a description, not a replication of Daniel and Moskowitz's result.

12. Common mistakes

Mistake 1: shifting the window by one month. Section 8.1: the same asset's 12–1 signal is 17.72%, one month earlier it is 15.47%, one month later it is 12.91%, and 12–0 is 10.65%. Fix: use `formation_return(window=11, skip=1)` and pass Chapter 23, code segment five's test that "changing the future does not change the past."

Mistake 2: summing instead of compounding. Section 6.2: $+50\%, -50\%$ sums to 0 but compounds to -25%. Fix: compound simple returns; add log returns and then exponentiate.

Mistake 3: inconsistent skipped months in cross-sectional and time-series momentum. Comparing MOP's 12–0 with JT's 12–1 confounds the window difference with the strategy difference: in the industry data, the means of 0.621% and 0.519% differ by 0.10 percentage points. Fix: report them side by side and label every window.

Mistake 4: using unadjusted prices after a split. Section 8.4: the split causes a 49% price decline while total return is +4%. In the example below, a "false loser" after a split ranks last and a true loser ranks ahead of it.

```
import numpy as np
import pandas as pd

from quant_course.factor_research import rank_cross_section

idx = pd.period_range("2020-01", periods=3, freq="M")
prices = pd.DataFrame({"split_stock": [100.0, 51.0, 52.0], "true_loser": [100.0,
    ↪ 90.0, 85.0]}, index=idx)   # 2-for-1 split in month 2 (+$1 dividend per new
    ↪ share)
unadjusted = prices.iloc[-1] / prices.iloc[0] - 1
adjusted_split = (2 * 51.0 + 2 * 1.0) / 100.0 * (52.0 / 51.0) - 1
    ↪ # total-return chain: split month, then 52/51
adjusted = pd.Series({"split_stock": adjusted_split, "true_loser": 85.0 / 100.0 - 1})
rank_u, rank_a = rank_cross_section(unadjusted.to_numpy()), rank_cross_section(
    ↪ adjusted.to_numpy())
assert rank_u[0] == 1.0 and rank_a[0] == 2.0
    ↪  # unadjusted: split stock is the worst; adjusted: it is the best
print(unadjusted.round(3).to_dict(), adjusted.round(3).to_dict())
```

Mistake 5: stitching futures prices directly. Section 6.8: the stitched series creates a false 3.92% return. A time-series signal would mistake the roll jump for a trend.

```
import numpy as np

a_prices, b_prices = [100.0, 102.0, 104.0], [108.0, 109.0, 111.0]          # contract
    ↪  B trades at a premium: contango
spliced = np.array([100.0, 102.0, 108.0, 109.0, 111.0])                     # roll
    ↪ between the second and third prices
naive_return = spliced[-1] / spliced[0] - 1                                  # "12-
    ↪ month" return from the spliced series
true_a = a_prices[1] / a_prices[0] - 1
true_b = b_prices[2] / b_prices[0] - 1
true_total = (1 + true_a) * (1 + true_b) - 1                                  # hold
    ↪ A to the roll, then hold B
```

```
assert naive_return > true_total + 0.04                                    # the
    ↪ splice overstates the move by the roll gap
assert np.sign(naive_return) == np.sign(true_total) == 1
print(round(naive_return, 4), round(true_total, 4))
```

Mistake 6: backtesting a universe of current survivors. Taking today's listed companies back through history uses future information—"survived until today"—to select the stocks. In the fixed-winner-leg example, omitting delisted losers overstates the future return of the loser leg and understates WML. With a dynamic universe, it also changes winners, ranks, and weights, so the bias direction is not fixed. Numerical example: 10 formation-period losers; 9 later survive and return an average of +20%, and 1 delists at −100%. The actual loser-leg mean is $(9 \times 20 - 100)/10 = +8\%$; a backtest with only survivors reports +20%. Fix: use constituents as of `signal_time` and include realized delisting returns in the subsequent labels; do not select history using today's list or feed future delisting returns into the signal. This chapter has no individual-stock data; the Project 4 preparation layer records this as a data gap.

Mistake 7: using a naive t-statistic for overlapping-holding returns. Portfolio returns with K-month holdings are serially correlated. This chapter uses Newey–West with a frozen lag of $K - 1$; separately check whether it covers the actual dependence. Chapter 23 showed that lag $K - 1$ can still be optimistic in a finite sample. Here, the reported t is an NW value and a rough magnitude reference only.

Mistake 8: ignoring turnover and costs. Monthly turnover for cross-sectional 12–1 is 0.904 (gross exposure 2). A 10 bp unit cost removes 0.092 percentage points, about 18% of the gross mean; the $K = 1$ net t falls from 3.64 to 2.99. Fix: report costs beside gross returns and state that borrowing and financing are omitted.

Mistake 9: leaving window searches out of the ledger. This chapter evaluates 19 distinct cross-sectional strategies across window variants and the $J \times K$ grid, so the threshold should be about $z = 3.01$, not 1.96. Fix: record the tests as required by Chapter 22 and draw conclusions only from the prespecified main specification.

13. Quantitative research application

Momentum fields in Project 4 and the checklist for real research. The current machine signal calls `formation_return(window=11, skip=1)`. The price-contract column is named `total_return` and is defined as a constructed decimal monthly total return; `signal_time` is month-end. The constructed price layer does not record an actual `available_at`. The table below lists descriptive fields needed for future real research (for example, `total_return_m` is not a current machine column). Do not describe its vendor or availability-time fields as data already obtained at this layer.

Field	Source	Time field	Unit	Missing-data rule
`total_return_m`	Total-return prices or vendor total return	Month-end (`event_time`) and `available_at`	Decimal	A missing return cell makes the window NaN; reject a missing calendar-month row before calculation, do not fill it.
`mom_12_1`	Calculated	`signal_time`	Decimal	NaN with fewer than 12 months of history.
`rank_pct`	Same-date percentile	`signal_time`	0–1	NaN with fewer than 2 valid assets on that date.
`universe_member`	Historical constituents	Version available at `signal_time`, including delisted firms	Boolean	Without historical constituents, do not claim to have backtested.
`label_1m`	Next-month return, including delisting return	Label interval start and end	Decimal	Use the delisting return in the delisting month; do not drop it as NaN.

Research workflow. First write a frozen plan (Chapter 22): the hypothesis that assets with higher same-date 12–1 returns have higher next-month returns; the 12–1 signal window; $K = 1$; period-by-period Rank IC with NW t as the primary metric; decision rule and cost assumption. Then evaluate the main specification. Treat window variants and the K grid as robustness checks, record them in the ledger, and do not use them to change the main conclusion. Report gross and net returns, turnover, mean and worst month in crash states, window variants, and the difference from the benchmark (cross-sectional momentum versus zero; time-series momentum versus always long).

What this chapter completes and leaves open. Completed: the core window and portfolio functions, and a full report on 10 industries. Not completed: individual-stock momentum (historical constituents, delisting returns, and total-return prices are missing), a cost-adjusted Project 4 strategy (Chapters 35–36), or dynamic momentum. Chapter 28 combines momentum with value and quality.

13.1 Return definitions and the Project 4 interface, field by field

Object to calculate	Required inputs	What cannot substitute
Monthly stock total return	Share-count changes, splits, cash distributions, and delisting treatment, or a total-return series with these adjustments explicitly defined	Difference in unadjusted closing prices; adding dividends again to a total-return series that already includes them
Stock price return	Split-adjusted price, explicitly labeled as excluding cash distributions	Calling it total-return momentum
Futures return	P&L while holding the same contract, roll rules, notional-exposure/margin convention, and collateral-interest treatment	Price difference across stitched contracts; it is not profit from executing a roll
Current constructed P4 total return	Decimal monthly simple total return defined by the generator, continuous monthly index, sampled using contemporaneous membership	Exchange-traded prices or evidence of live fills

For the stock example, if the wealth product over the other formation months is G, the correct signal is $G(1+R_{\text{total}})-1$ and the unadjusted signal is $G(1+R_{\text{raw}})-1$. Their difference is $G(R_{\text{total}}-R_{\text{raw}})$. The current-month difference in the split example in Section 6.7 is 53 percentage points; if $G = 1.10$, the formation-signal difference is 58.3 percentage points. A futures price change is also only a simple teaching proxy per unit of notional exposure; a margin-account return and a total return including collateral interest use different denominators.

Project 4's `mom_12_1` actually calls `formation_return(..., 11, 1)`. Its return input is the constructed monthly total return in the contract, and the month-$s+1$ return is used as the label after signal month s. This layer has no source of actual adjusted prices, daily next-tradable-time data, or fill verification. The raw formation signal enters the unified preprocessing and synthesis flow in Chapter 24, Section 13.1; it is not a position by itself. Label alignment, Rank IC, and time-series inference use Chapter 23. The NW lag for overlapping holdings is an existing specification; do not treat $K-1$ as universally sufficient for every serial-dependence structure.

14. Chapter summary

Momentum is determined by three things: window endpoints (12–1 uses months $s-11$ through $s-1$, while MOP's 12–0 includes the most recent month; a one-month difference means a different signal), compounding (summing can change the ranking), and portfolio construction (cross-sectional long-short has zero net exposure; time-series trend can be net long or net short and must be compared with always long). Overlapping holdings replace a $1/K$ vintage share each month; under the rule that resets average target weights and ignores drift, target-weight turnover is $\|w^{(s)}-w^{(s-K)}\|_1/K$. Data definitions determine whether results are credible: use total returns that account for splits and dividends, returns on the held futures contract rather than stitched

prices, and constituents available at `signal_time`. On 10 industry portfolios, 12–1 is positive and remains positive after assumed costs, but only a few of 19 distinct cross-sectional variants pass the multiple-testing threshold; time-series trend does not beat always long; and the worst months cluster in rebounds after bear states. Scope: industry portfolios are not individual stocks; costs are assumed; the crash description is not a replication; this chapter has no individual-stock data.

15. Exercises

25-E01 (by hand). An asset's monthly returns over months 1–14 are $0.04, -0.03, 0.02, 0.05, -0.01, 0.03, 0.02, -0.02, 0.01, 0.06, -0.04, 0.02, 0.03, -0.05$. The signal is formed at the end of month 13. Calculate 12–1, 12–0, and the returns for windows shifted one month earlier and one month later; also calculate the arithmetic sum for 12–1.

25-E02 (by hand). Eight assets have 12–1 formation returns $(40\%, 25\%, 18\%, 10\%, 10\%, -5\%, -12\%, -30\%)$ and next-month returns $(3\%, 1\%, 4\%, 0, 2\%, 1\%, -2\%, -3\%)$. Take the highest and lowest buckets for $q = 4$ (resolve ties with average ranks). Find the $q = 4$ bucket assignments, long-short weights, and next-month long-short return.

25-E03 (by hand). Four assets have signs $(+1, -1, -1, +1)$, ex ante annualized volatilities $(8\%, 16\%, 32\%, 20\%)$, target volatility 40%, and next-month returns $(1.5\%, -3\%, 4\%, -1\%)$. Find the weights, gross exposure, net exposure, portfolio return, and each asset's contribution.

25-E04 (by hand). Two assets have seven months of vintages: $(0.5, -0.5)$ for the first three months and $(-0.5, 0.5)$ for the next four. Find target-weight turnover for $K = 1$ and $K = 3$, and verify $\text{turn}_s = \|w^{(s)} - w^{(s-K)}\|_1 / K$.

25-E05 (by hand). A stock is priced at 90 and undergoes a 3-for-1 split; after the split it is priced at 31 and pays a cash dividend of 0.5 per new share. Calculate total return and the unadjusted price change. If this month is in a 12–1 formation window, by how many percentage points is the signal understated?

25-E06 (by hand). Contract A is priced at 50 and 52; at the roll, contract B is priced at 55 and 56.1. Find the "returns" from the stitched price series, the available returns, and the false return at the roll.

25-E07 (by hand). All market monthly returns are +1% except month 3, which is −35%. What is the bear state for the 24-month window at signal months $s = 23, 24, 26, 27$? Explain why the state changes.

25-E08 (concept and calculation). Asset X returns $+50\%, -50\%$ over two months and Y returns $0, 0$. Rank them by sum and by compounding. Then give a three-month example in which the sum-based and compound-return rankings are opposite.

25-E09 (interpretation). Table 11 reports a monthly mean of 1.391% for TS 12–0 and 1.779% for always long. Explain why 1.391% cannot be treated as trend alpha and how to test whether trend adds value.

25-N01 (calendar and data diagnostic). Calculate 12–1, 12–0, and the sum from the actual calendar-month table in Section 3.1; mark the formation, skipped, and holding times. If someone multiplies the January 2026 return into the signal, identify the leakage. Then, let the wealth product for the other formation months in the split example in Section 6.7 be $G = 1.10$; calculate the difference in the formation signal caused by using unadjusted prices, and explain why the stitched jump at a futures roll is not earned.

16. Programming exercises

25-P01. Write a test that, for the 14 monthly returns, asserts that `formation_return( window=11, skip=1)` in month 13 equals the value in Section 8.1 and that changing returns in month 13 and later does not change that row. Acceptance criterion: both assertions pass.

25-P02. Generate vintage weights for 24 months and 10 assets, then calculate turnover for $K = 1, 3, 6$ and verify $\|w^{(s)} - w^{(s-K)}\|_1/K$. Acceptance criterion: error is less than 10^{-12} for all three K values.

25-P03. On a constructed random-walk panel with positive drift (10 assets, 480 months, monthly drift 0.5%, monthly volatility 4%), calculate time-series 12–0 and always long with the same scaling; report their means, t-statistics, and the t-statistic of the difference series, and interpret the results. Acceptance criterion: always-long mean exceeds the time-series portfolio mean and the difference-series $t < -2$.

17. Challenge problems

25-C01 (proof). Prove that adjacent-month turnover for an overlapping-holdings portfolio $B_s = \frac{1}{K}\sum_{j=0}^{K-1} w^{(s-j)}$ equals $\|w^{(s)} - w^{(s-K)}\|_1/K$. If every vintage has gross exposure 2, give the turnover upper bound and state when it is attained. Explain why increasing K does not necessarily reduce turnover by exactly $1/K$. Prerequisite: the triangle inequality for absolute values. Grading: proof 40, upper bound 30, discussion 30.

25-C02 (open-ended). Write a complete frozen plan and data-gap table for the Project 4 momentum signal: window, total-return definition, historical constituents and delisting returns, cost and stock-borrow assumptions, crash-state reporting, and multiple-testing count. Identify fields unavailable in the course's current data. Grading: plan completeness 25, correct time fields 25, data gaps 25, crash and cost treatment 25.

18. Complete solutions and reference implementations

25-E01. 12–1: compound months 2–12, $\prod_{k=2}^{12}(1 + R_k) - 1$. The month-by-month product is $0.97 \times 1.02 \times 1.05 \times 0.99 \times 1.03 \times 1.02 \times 0.98 \times 1.01 \times 1.06 \times 0.96 \times 1.02 = 1.1101$, or +11.01% (rounded to 11.009%); the sum is 11.0%. For 12–0 (months 2–13, adding month 13's +3%), $1.1101 \times 1.03 = 1.1434$, or +14.34%. One month earlier (months 1–11): 13.19%; one month later (months 3–13): 17.88%. The four windows differ by 2 to 7 percentage points.

25-E02. In the given asset order, the ascending ranks for formation returns $(40, 25, 18, 10, 10, -5, -12, -30)\%$ are $-30 \to 1$, $-12 \to 2$, $-5 \to 3$, the two 10s tied across ranks 4 and 5 each receive 4.5, $18 \to 6$, $25 \to 7$, and $40 \to 8$. Thus ranks are $(8, 7, 6, 4.5, 4.5, 3, 2, 1)$. With $q = 4$ and $n = 8$, bucket number is $\lfloor 4(2r-1)/16 \rfloor + 1$: $r = 8 \to 4$, $7 \to 4$, $6 \to 3$, $4.5 \to 3$, $3 \to 2$, $2 \to 1$, and $1 \to 1$, giving $(4, 4, 3, 3, 3, 2, 1, 1)$. The top bucket contains the first two assets (formation returns 40% and 25%); the bottom contains the last two (-12% and -30%). Each leg has weights of ± 0.5. Long-short return is $\frac{1}{2}(3\% + 1\%) - \frac{1}{2}(-2\% - 3\%) = 2\% + 2.5\% = 4.5\%$.

25-E03. Unnormalized position is sign $\times\, 0.40/\sigma$: $(+5, -2.5, -1.25, +2)$. Divide by 4 to get weights $(1.25, -0.625, -0.3125, 0.5)$. Gross exposure is 2.6875; net exposure is 0.8125. Contributions are $1.25 \times 1.5\% = 1.875\%$, $-0.625 \times (-3\%) = 1.875\%$, $-0.3125 \times 4\% = -1.25\%$, and $0.5 \times (-1\%) = -0.5\%$. The portfolio return is 2.0%.

25-E04. For $K = 1$, adjacent-month turnover is $0, 0, 2, 0, 0, 0$; the reversal in month 4 trades 2.0. For $K = 3$, $B_3 = (0.5, -0.5)$; $B_4 = \frac{1}{3}(0.5 + 0.5 - 0.5) = (0.1667, -0.1667)$; $B_5 = \frac{1}{3}(0.5 - 0.5 - 0.5) = (-0.1667, 0.1667)$; $B_6 = (-0.5, 0.5)$; and $B_7 = (-0.5, 0.5)$. Turnover is $0.667, 0.667, 0.667, 0$. For month 4, $\|w^{(4)} - w^{(1)}\|_1/3 = \|(-0.5, 0.5) - (0.5, -0.5)\|_1/3 = 2/3$. Months 5 and 6 work the same way; for month 7, $\|w^{(7)} - w^{(4)}\| = 0$. Total turnover is 2.0, but only 0.667 is traded per month.

25-E05. Total return is $(3 \times 31 + 3 \times 0.5)/90 - 1 = (93 + 1.5)/90 - 1 = +5.0\%$; the unadjusted price change is $31/90 - 1 = -65.6\%$. The split-month difference is $5.0 - (31/90 - 1) \times 100 = 70.5555556$ percentage points (displayed as 70.6). If the wealth product for the other formation months is G, the full formation-signal difference is $G \times 70.5555556$ percentage points. Since G is not given in the exercise, the full signal difference is not uniquely determined. It equals the one-month difference only when $G = 1$.

25-E06. Stitched prices are 50, 52, 55, 56.1, giving returns of 4.00%, 5.77%, and 2.00%. Available returns: hold A for $52/50 - 1 = 4.00\%$; no return at the roll itself; hold B for $56.1/55 - 1 = 2.00\%$. The false return at the roll is $55/52 - 1 = 5.77\%$. The total price change across the stitched series is $56.1/50 - 1 = 12.2\%$; the available total return is $1.04 \times 1.02 - 1 = 6.08\%$.

25-E07. Number months from 1 through 30; the window is months $s-23, \ldots, s$ and requires 24 months. At $s = 23$, history is insufficient, so the state is NaN. At $s = 24$, months 1–24 include month 3, so cumulative return is $1.01^{23} \times 0.65 - 1 = -18.3\% < 0$ and state is 1. At $s = 26$, months 3–26 still include month 3, so state is 1. At $s = 27$, months 4–27 exclude month 3, cumulative return is $1.01^{24} - 1 = +26.97\%$ and state is 0. The window has moved past the crash month, so the cumulative return no longer includes it.

25-E08. X sums to 0 and compounds to -25%; Y sums to 0 and compounds to 0. They tie by the sum, but Y ranks above X by compounding. Three-month example: X returns $+30\%, +30\%, -40\%$, sum $+20\%$, compound return $1.3 \times 1.3 \times 0.6 - 1 = +1.4\%$. Y returns $+10\%, +5\%, +4\%$, sum $+19\%$, compound return $1.1 \times 1.05 \times 1.04 - 1 = +20.1\%$. By sums, X (20%) ranks above Y (19%); by compound returns, Y (20.1%) ranks above X (1.4%): the rankings reverse.

25-E09. The trend portfolio's average net exposure is 1.05 and 67% of assets are classified as long, so the 1.391% mean includes an equity risk premium (after scaling). Always long with the same scaling earns 1.779%. Test trend's incremental value using the difference series (TS minus always long) or an intercept from a regression of TS on always long. The difference-series mean is -0.389% ($t = -1.05$); the sign and significance of the regression intercept depend on the estimated regression. This chapter reports only the difference series and finds no statistically clear positive increment in this sample. Nonsignificance does not prove that the increment is zero.

25-P01 reference implementation

```
import numpy as np
import pandas as pd

from quant_course.factor_signals import formation_return

r = [0.03, -0.02, 0.05, 0.01, -0.04, 0.06, 0.02, -0.01, 0.04, 0.03, -0.02, 0.05,
    ↪ -0.06, 0.02]
idx = pd.period_range("2020-01", periods=14, freq="M")
frame = pd.DataFrame({"a": r}, index=idx)
s = idx[12]
base = formation_return(frame, window=11, skip=1)
assert abs(base.loc[s, "a"] - (np.prod(1 + np.array(r[1:12])) - 1)) < 1e-12
shocked = frame.copy(); shocked.iloc[12:] = 0.9                                    #
    ↪ month 13 and later change
pd.testing.assert_frame_equal(base.iloc[:13], formation_return(shocked, window=11,
    ↪ skip=1).iloc[:13])
```

25-P02 reference implementation: see code segment four in Section 9, keep the K=1/3/6 loop and replace its input with `vint = pd.DataFrame(rng.choice([-0.5, 0.0, 0.5], size=(24, 10)), index=idx, columns=[f"A{i}" for i in range(10)])`. Leave the difference, formula, and assertions unchanged; this supplies the required ten columns.

25-P03 reference implementation

```
import numpy as np
import pandas as pd

from quant_course.momentum import ex_ante_volatility, strategy_returns,
    ↪ time_series_sign, tsmom_weights

rng = np.random.default_rng(3)
idx = pd.period_range("1980-01", periods=480, freq="M")
returns = pd.DataFrame(rng.normal(0.005, 0.04, (480, 10)), index=idx, columns=[f"A{i}
    ↪ " for i in range(10)])
vol = ex_ante_volatility(returns)
sig = time_series_sign(returns, window=12, skip=0)
ts = strategy_returns(tsmom_weights(sig, vol, target_vol=0.40), returns).dropna()
always = strategy_returns(tsmom_weights(pd.DataFrame(1.0, index=idx, columns=returns.
    ↪ columns).where(vol.notna()), vol, target_vol=0.40), returns).dropna()
diff = (ts - always).dropna()
t = diff.mean() / (diff.std(ddof=1) / np.sqrt(len(diff)))
assert always.mean() > ts.mean() and t < -2                    # independent returns
    ↪ : no trend to exploit; the 12-month sign filter shorts after a negative
    ↪ cumulative return and gives up the drift
print(round(100 * ts.mean(), 3), round(100 * always.mean(), 3), round(t, 2))
```

The output is about 0.500, 1.565, and −5.14: with independent returns and only positive drift, sign filtering shorts after negative cumulative returns over the past 12 months and gives up the drift, so the difference is significantly negative (across five seeds, all t-statistics are below -2.8). This is a sign-filter example for the specified IID positive-drift construction; it does not prove that positive return autocorrelation is a general necessary condition for trend to add value. Nonlinear predictability, state dependence, and risk scaling can also affect the comparison. The industry-sample difference-series $t = -1.05$ means only that no statistically clear mean advantage is observed; it does not identify the cause or prove there is no increment.

25-C01. $B_s - B_{s-1} = \frac{1}{K}\left(\sum_{j=0}^{K-1} w^{(s-j)} - \sum_{j=1}^{K} w^{(s-j)}\right) = \frac{1}{K}(w^{(s)} - w^{(s-K)})$ because the two sums share terms $j = 1, \ldots, K-1$. Taking the ℓ_1 norm gives $\text{turn}_s = \|w^{(s)} - w^{(s-K)}\|_1 / K$. If each vintage has gross exposure 2, the triangle inequality gives $\|w^{(s)} - w^{(s-K)}\|_1 \le \|w^{(s)}\|_1 + \|w^{(s-K)}\|_1 = 4$, so $\text{turn}_s \le 4/K$. Equality holds if and only if $w_i^{(s)} w_i^{(s-K)} \le 0$ for every asset i (the asset has opposite signs in the two vintages, or at least one weight is zero). As K rises, the denominator grows, but the numerator $\|w^{(s)} - w^{(s-K)}\|_1$ also changes; there is no general theorem that it rises monotonically with K (a periodic signal can make it rise and then fall). Therefore turnover does not necessarily decline strictly as $1/K$. In the industry data, mean numerator is about 1.55 for $K = 3$ and about 0.90 for $K = 1$. Grading follows the three items in the original.

25-C02 (grading guidance and reference). Plan elements: state the hypothesis and sign (winners have higher next-month returns; positive direction); use 12–1 total returns; use historical constituents as of `signal_time`, including delisting returns; use an 11-month window and skip 1 month; equal-weight the highest and lowest deciles; set $K = 1$ as the main specification; state separate unit-turnover and stock-borrow cost assumptions; use period-by-period Rank IC with NW t as the primary metric; define the crash state as a negative market total return over the prior 24 months and report conditional means and worst months; specify the experiment budget

and ledger; reserve a holdout sample. Data gaps: individual-stock total returns and adjusted prices, historical constituents and delisting returns, borrow rates, and shares available to borrow are all absent from the course's current data. Grading follows the four original criteria.

25-N01 complete solution. January–November 2025 has 11 observations and cumulative wealth of 1.1547369892, so 12–1 is 15.4737%; the sum is 15%. Including December gives 12–0 at 21.2474%. December is the skipped month; January 2026 is the next-month holding label and is unknown when the signal is formed, so including it is look-ahead. In the split example, the correct monthly total return is 4% and the unadjusted monthly return is -49%, a 53-percentage-point difference. Multiplying by $G = 1.10$ gives a formation-signal difference of 58.3 percentage points. At a futures roll, selling A and buying B at the same time does not create a price path from A at 102 to B at 106; the price difference between two different claims is not a holding return. Calculate P&L separately for each contract held.

19. Further reading

- Must Know: Jegadeesh and Titman (1993), opening of §§I and II; read the J/K strategies, overlapping holdings, and their Bonferroni discussion of 32 tests.
- Must Know: Asness, Moskowitz, and Pedersen (2013), printed pp. 936–937; read the MOM2–12 definition and reason for skipping a month.
- Should Know: Moskowitz, Ooi, and Pedersen (2012), §§2.4 and 4.1; read volatility scaling and equation (5) for TSMOM.
- Should Know: Daniel and Moskowitz (2014 working paper), abstract and note to Table 3; read the crash-state definition. The journal version was not read.
- Advanced: Original paper by Moskowitz and Grinblatt (1999) on industry momentum; not read for this chapter.
- Optional: French database Mom-factor description page; check the definition of prior (2–12).

20. References

Citation keys and the scope of what was read are listed in `source_notes.md`.

- CH25_JT1993: Jegadeesh, N., and Titman, S. (1993). *Journal of Finance* 48(1), printed pp. 68–69.
- CH25_AMP2013: Asness, C. S., Moskowitz, T. J., and Pedersen, L. H. (2013). *Journal of Finance* 68(3), printed pp. 936–937.
- CH25_MOP2012: Moskowitz, T. J., Ooi, Y. H., and Pedersen, L. H. (2012). *Journal of Financial Economics* 104(2), §§2.4 and 4.1.
- CH25_DM2014: Daniel, K., and Moskowitz, T. J. (2014). NBER Working Paper 20439, abstract, printed p. 8, and note to Table 3; journal version not read.
- CH25_FRENCH_MOM_PAGE: Ken French database Mom-factor description page.
- FRENCH: Frozen French industry and FF3 snapshots (`data/manifests/`).

Chapter 26 — Quality

Through line: Quality is not a single number. It has several accounting dimensions: profitability, leverage, the cash content of earnings, and stability. For each dimension, ask what the numerator is, what the denominator is, whether both come from the same financial-statement version, and whether the market knew them at that point in time. Prerequisites: Chapter 12 (financial statements), Chapter 23 (cross-sectional research), and Chapter 24 (value and point-in-time fundamentals). After this chapter, you should be able to decompose ROE into net margin, turnover, and the equity multiplier, and explain why rising ROE does not necessarily mean better operations; distinguish ending from average denominators and handle negative or near-zero equity; calculate gross profit to assets, operating profitability to equity, accrual ratios, and earnings stability; construct ratios from one financial-statement version; explain financial-sector conventions; and interpret the published RMW factor return while recognizing that it does not test an individual-stock quality signal.

1. Chapter goals

Readers should be able to answer: Is a company "high quality" because it truly earns more, or because it borrows more? Does a number in an earnings ratio represent cash, and was it public at the time?

1. State the numerator, denominator, unit, direction, and relevant industries for ROE, ROA, GP/A, OP, leverage, the accrual ratio, and earnings stability.
2. Use the DuPont decomposition ROE = net margin × asset turnover × equity multiplier to derive the effect of leverage, ROE $= \rho + (\rho - i)D/E$, and use two companies to show how "operating improvement" and "more leverage" can produce similar ROE changes.
3. Explain the choice between average and ending balances, why negative or near-zero equity makes ROE meaningless, and why both year-end equity balances must be positive before averaging them.
4. Apply the "same financial-statement version" rule: numerator and denominator must come from the same filing; a restated numerator cannot be divided by a pre-restatement denominator.
5. Reproduce with a constructed panel that net-income measures mix in accruals; most cross-sectional ROE variance comes from leverage; near-zero equity can severely distort Pearson IC; Spearman is more robust to extreme magnitudes, but ranks and sample filtering can still change it; and the Pearson composite identity for two uncorrelated standardized dimensions, then compare it in scale with a constructed rank IC.
6. Read published monthly RMW returns and report the mean, t-statistic, subperiods, drawdown, and correlations; explain that this describes an aggregate factor return and does not test any stock-level quality signal.

Completion criterion: Every quality signal includes a field table specifying each field's reporting period, `available_at`, `ingested_at`, unit, and industries where it is undefined.

2. Why Quant needs this

A value factor asks, "Is the price cheap relative to fundamentals?" A quality factor asks, "Are the fundamentals themselves good?" Together, they provide more information available in advance for identifying value traps: a company that is cheap but unprofitable, highly leveraged, or supported by accruals may rank well on value and poorly on quality. Chapter 28 combines the two, but every quality metric must stand on its own before combination.

Quality metrics have the same difficulty as value metrics, but it is less visible. The numerator comes from the income statement (a flow over a period), while the denominator comes from the balance sheet (a stock at a point in time); their time conventions naturally differ. The denominator is often equity close to zero or even negative. Profit and liabilities mean different things in financial firms. Net income includes amounts unsupported by cash. Financial statements may be restated. Chapter 24's point-in-time selection rules apply directly, with one additional rule: the numerator and denominator of a ratio must come from the same version.

The chapter proceeds as follows: define the measures (Section 5); derive why leverage changes ROE (6.1); address denominators (6.2), measures that do not directly contain a financing multiplier (6.3), accruals (6.4), estimation error in stability (6.5), same-version selection (6.6), and financial firms (6.7); quantify how standardized dimensions combine (6.8); and finish with a constructed panel and published RMW returns (Sections 10–11).

3. Intuition

Last year, two companies both raised ROE from 15% to 20%. Company A cut costs and sold more effectively, so it earned more profit per dollar of sales. Company B borrowed money to repurchase shares, shrinking its equity; the same profit divided by less equity raised ROE. Both ROEs rose, but A improved its operations while B took on more risk. Decomposing ROE into "margin $\times$ turnover $\times$ leverage" reveals the difference.

Another common confusion is profit versus cash. Net income is an accounting number. Part of it comes from changes in receivables, inventory, and accrued expenses and does not correspond to cash received. If the profit-cash difference comes from accruals that later reverse, it may be followed by lower earnings. Investment, seasonal working capital, and accounting classifications can also cause a difference; its direction must be tested. Subtracting net cash from operating activities from net income, then dividing by average assets, gives the accrual ratio. It measures the gap between accounting earnings and operating cash flow; it is not "cash" itself.

A third issue is timing. A financial statement may be released weeks after the fiscal year ends, and its reported values may later be revised. For every number in a quality signal, ask: who could see it at the signal month-end?

Simplifying boundary: the constructed panel in this chapter represents these phenomena with a minimal structure (one latent quality factor, one accrual tendency, and random leverage). Real-company accounting is much more complex.

3.1 One available filing, worked through completely

Complete worked example: one filing version to five ratios

The teaching company's FY2024 financial statements were released on 2025-03-20 and ingested locally on 2025-03-21, both in UTC. The reporting currency is millions of euros. Profit and cash-flow figures cover all of 2024. Comparative balances come from this same filing; do not substitute a restated prior-year value from today's database.

Item	FY2024 flow or year-end balance	FY2023 comparative balance in the same filing
Revenue, cost of goods sold	1000, 600	Not needed for this example
SG&A, interest	200, 20	Not needed for this example
Net income attributable to common shareholders, net cash from operating activities	90, 120	Not needed for this example
Total assets	1000	800
Common equity	400	300
Interest-bearing debt	300	Not needed for this example

Total liabilities between assets of 1000 and equity of 400 are 600; interest-bearing debt is 300 and other liabilities are another 300. So this example does not assume $E = A - D$ and cannot be inserted directly into the one-interest-bearing-debt model in Section 6.1. First calculate $\bar{A} = (800 + 1000)/2 = 900$ and $\bar{E} = (300 + 400)/2 = 350$. Gross profit is 400; GP/A $= 400/900 = 0.444444 = 44.4444\%$; ROE $= 90/350 = 25.7143\%$. The raw accrual ratio is $a = (90 - 120)/900 = -3.3333\%$; P4 `neg_accrual` is $= -a = +3.3333\%$, a positive value meaning operating cash flow exceeds net income. Debt to ending assets is $300/1000 = 30\%$.

French's OP definition uses book equity. Here, use ending equity of 400 explicitly: OP $= (1000 - 600 - 200 - 20)/400 = 45\%$. Using average equity of 350 instead would give 51.4286%, a different course variant that must not be called the same specification as the original. Annual profit-to-balance measures are accounting ratios of operating performance over a year. They may be displayed as decimals or percentages; they are not monthly investment returns and should not be "monthlyized" by multiplying or dividing by 12. In continuous-time dimensions, a flow-to-stock ratio has a per-year frequency; in this course, a fixed annual window is displayed as a dimensionless ratio and must always be labeled with its annual convention.

At the end of February 2025, FY2024 was not yet available. At the end of March, both availability conditions are satisfied, so this row can be calculated. Using it before release on March 20 or local ingestion on March 21 would be look-ahead for this system. Each month, select a complete row version using Chapter 24, Section 6.4, and only then calculate the ratios; do not interpolate an annual flow into monthly realized earnings.

4. Mathematical foundations

Notation. For company i and a fiscal year, let sales be S, cost of goods sold be C, selling, general, and administrative expenses be G, interest expense be I, net income attributable to common shareholders be N, operating cash flow be F, total assets be A, debt be D, and book equity be $E = A - D$. Note that this is a simplification for this chapter: it assumes all liabilities are interest-bearing debt D. A real balance sheet also has non-interest-bearing operating liabilities such as accounts payable and deferred revenue; equity is $A - L$, where $L \geq D$ is total liabilities. Both the DuPont derivation and the constructed panel use $E = A - D$; with real statements, distinguish interest-bearing debt from total liabilities. Income-statement items are flows over a fiscal year; balance-sheet items are stocks at fiscal year-end. Use one currency and scale. Flow-to-balance ratios must carry an annual window (in continuous-time units, per year, including asset turnover, ROE, and GP/A). In this course, fixed fiscal-year cumulative flows divided by balances are shown as unitless decimals or percentages, while debt/assets and the equity multiplier are stock-to-stock ratios. Neither convention describes monthly investment returns.

Average balances. Profit earned during a fiscal year corresponds to average assets during that year. A common calculation is $\bar{A} = (A_{\text{year-end}} + A_{\text{year-start}})/2$; use the same convention for $\bar{E}$. The year-start balance is the prior fiscal year-end balance. A 10-K reports balance sheets for both the current and prior years, so both balances can come from one filing.

DuPont identity. When $S, A, E > 0$,

$$\frac{N}{E} = \frac{N}{S} \cdot \frac{S}{A} \cdot \frac{A}{E}.$$

The three factors are net margin, asset turnover, and the equity multiplier (also called financial leverage).

Precision of the sample standard deviation. For n independent normal observations, the variance of sample standard deviation s is approximately $\sigma^2/(2(n-1))$, with relative standard error $1/\sqrt{2(n-1)}$.

Combining two signals. Let standardized signals z_1, z_2 be uncorrelated with variance 1, and a standardized label y have variance 1 and correlations ρ_1, ρ_2 with the two signals. Then $\text{Corr}(z_1 + z_2,\, y) = \dfrac{\rho_1 + \rho_2}{\sqrt{2}}$.

Frequency convention: accounting quantities are annual; signals and returns are monthly. `signal_time` is month-end, and the signal uses the most recent fiscal year that was available.

5. Formal definitions

Return on Equity (ROE). $N/\bar{E}$, using average or ending common equity as the denominator; undefined when equity is nonpositive.

Return on Assets (ROA). $N/\bar{A}$. It does not use equity, but net income has already deducted interest, so ROA contains some leverage information.

Gross Profitability (GP/A). $(S-C)/\bar{A}$. Gross profit is revenue minus cost of goods sold. Novy-Marx's 2012 author manuscript records gross profits-to-assets, normalized by book assets rather than equity. The course and P4 use average assets as the course-interface convention; do not attribute "average" to a paper definition that has not been checked. The numerator does not directly deduct interest. Financial firms have no cost-of-goods-sold field in this sense, so this measure does not apply to them.

Operating Profitability (OP). Fama and French's (2015, note to Table 8) definition is $(S-C-G-I)/E$: revenue minus cost of goods sold, SG&A, and interest expense, divided by book equity. Include only firms with positive book equity. The numerator deducts interest and the denominator is equity, so OP contains leverage.

Leverage. This chapter uses both D/A (debt to total assets, unitless; between 0 and 1 in the simplified model with $E = A-D > 0$, but it can exceed 1 for a real company with nonpositive equity) and the equity multiplier A/E. These describe capital structure, not profitability.

Accrual ratio. $(N-F)/\bar{A}$: net income minus net cash from operating activities, divided by assets. It measures the size of the difference between net income and operating cash flow relative to average assets: the higher the value, the larger the portion by which net income exceeds operating cash flow (scaled by assets); the lower or more negative the value, the more operating cash flow exceeds net income. It is not "the proportion of profit not yet received in cash." Company A has $N = 10$, $F = 5$, and $\bar{A} = 100$, so $a = 5\%$ and the difference is 50% of net income. Company B has $N = 20$, $F = 12$, and $\bar{A} = 100$, so $a = 8\%$, while the difference is only 40% of net income. To discuss the difference as a share of net income, define a separate ratio $(N-F)/N$ and handle $N \leq 0$ separately.

Earnings stability. The sample standard deviation of the most recent n fiscal years of ROA (or ROE); lower means more stable. Usually $n = 5$; fewer than n years gives NaN.

Same-version rule. The numerator and denominator used to form a ratio must come from the same financial statement and version. After a restatement is released, recalculate using all values from the restated filing; before it is released, continue to use the original values.

RMW (Robust Minus Weak). A published profitability-factor spread: $\frac{1}{2}$(Small Robust+ Big Robust) $-$ $\frac{1}{2}$(Small Weak + Big Weak), formed from six value-weighted portfolios sorted on size and operating profitability (French database description page). It is an aggregate return, not a stock-level signal.

Convention for using "quality." In this chapter, "quality" refers only to the calculable accounting dimensions above, not unmeasured concepts such as management or a moat. Different studies combine dimensions differently; this chapter does not call any one combination "the standard definition of a quality factor."

6. Derivations

6.1 DuPont decomposition and the effect of leverage on ROE

Consider a company with debt D and an after-tax borrowing cost i. Compare cases with fixed assets, operating return, and debt interest; require positive equity and assume all liabilities are this debt. After-tax quantities must be matched under the simplifying conditions of a common tax rate, deductible interest, and a tax shield usable in the current period. Do not assume an immediate tax shield unconditionally when the company has severe losses. Let net operating profit after tax (NOPAT) be $= \rho A$, where ρ is the after-tax return on operating assets. Net income is NOPAT minus after-tax interest: $N = \rho A - iD$. Equity is $E = A - D$. Therefore,

$$\text{ROE} = \frac{N}{E} = \frac{\rho A - iD}{E} = \rho\frac{A}{E} - i\frac{D}{E} = \rho\Big(1 + \frac{D}{E}\Big) - i\frac{D}{E} = \rho + (\rho - i)\frac{D}{E}.$$

The identity used is $A/E = 1 + D/E$. Interpretation: with no debt ($D/E = 0$), ROE equals ρ. If $\rho > i$, every additional unit of D/E raises ROE by $(\rho - i)$. For $\rho = 8\%, i = 3\%$, values $D/E = 0, 0.5, 1, 2$ give ROE of $8\%, 10.5\%, 13\%, 18\%$. Thus ROE can rise solely because leverage rose, while ρ stays unchanged. DuPont splits the change into three factors: leverage changes the third factor, A/E; operating improvement changes the first and second. Section 8.1 works through two companies numerically.

The cost of leverage: the same D/E lowers ROE when $\rho < i$ and increases ROE volatility (the multiplier amplifies variation in ρ). A company whose high ROE comes from high leverage is more fragile.

6.2 Denominators: average, ending, and near-zero equity

Sensitivity of ROE to the denominator. ROE $= N/E$, $\partial\text{ROE}/\partial E = -N/E^2$, and a first-order approximation gives relative change $\delta\text{ROE}/\text{ROE} \approx -\delta E/E$. With equity $E = 400$, a one-unit equity error changes ROE by about 0.25% in relative terms (not 0.25 percentage points). With equity $E = 2$, a one-unit error is so large that the first-order approximation fails: increasing equity from 2 to 3 lowers ROE by 33%; decreasing it from 2 to 1 raises ROE by 100%. With $N = 40$, equity of 2 gives $40/2 = 2000\%$; equity of 3 gives $40/3 = 1333.3\%$, a drop of 666.7 percentage points; equity of 1 gives $40/1 = 4000\%$, a rise of 2000 percentage points. Near-zero equity turns accounting noise into extreme values.

Negative equity. When $E < 0$ and $N > 0$, N/E is negative; when $E = 0$, the ratio is undefined. Both are excluded under the course rule. The negative-equity ratio looks like "very poor profitability" but actually reflects insolvency; the sign has changed meaning. This chapter requires positive equity, consistent with Fama–French's inclusion condition (French database description page).

Average-balance trap. Ending equity is −30 and beginning equity is 100. The simple average is 35, which is positive, and $N = 40$ gives ROE = 114%: a very high profitability figure for a company insolvent at year-end. Therefore, require both balances to be positive before averaging them (`average_balance(..., floor=0.0)`).

Near-zero equity. Even positive equity may need a floor; for example, require equity to exceed 5% of assets (the `floor` parameter). The threshold is a research choice: write it into the plan and report the fraction excluded.

Ending versus average balances. Profit is a flow accumulated over the fiscal year, so average balances match it better. Ending balances refer to fiscal year-end and are simpler to use; that reference date does not mean they have been publicly released or locally ingested. Both ending and average conventions must wait until the same filing satisfies the available_at/ingested_at cutoff. The two are similar for companies with stable growth, but can differ substantially when equity changes rapidly (Section 8.3: $N = 40$, equity falls from 500 to 100; ending ROE is 40%, average ROE is 13.3%). State the denominator choice in the plan and use a consistent convention across metrics.

6.3 Why gross profit to assets does not directly contain a financing multiplier

Net income $N = (S - C - G - I - \cdots)(1 - \tau)$ includes interest I, which equals debt times the interest rate, so N is related to capital structure. Gross profit $S - C$ is before interest. Dividing it by assets A (the total capital, regardless of whether it is debt or equity) neither subtracts interest nor uses a denominator tied to the financing structure. Therefore, in a comparison with sales, costs, and assets held fixed, borrowing interest or an equity denominator does not directly change GP/A; it measures gross profit per asset. Actual borrowing can affect investment, working capital, and operating costs, so this does not imply statistical independence between real-world GP/A and leverage. OP deducts interest in the numerator and uses equity in the denominator, so it is affected by leverage. That is the substantive difference between the measures, not an implementation detail.

6.4 Accruals: the gap between earnings and cash

Net cash from operating activities F is operating cash flow on the cash-flow statement; net income N is measured on an accrual basis. The difference $N - F$ is the total accrual gap. It includes working-capital changes, depreciation, and other noncash items, and is also affected by cash-flow-statement classification. Working-capital changes (receivables, inventory, accounts payable, and so on) and noncash items such as depreciation can contribute. The accrual ratio is $a = (N - F)/\bar{A}$. A high a means net income exceeds operating cash flow by more relative to assets; F is itself a net cash amount, so $N - F$ does not identify, transaction by transaction, which profit has not yet been received in cash. If accruals later reverse, future earnings may fall. This is a mechanism hypothesis; the direction and size of accruals' predictive relation with returns is an empirical question (this chapter has not read Sloan 1996 and does not restate its results). Direction convention: lower raw accrual ratio a is better; the quality signal is $-a$, so higher transformed values are better. Do not mix up the direction of the raw ratio and its negative.

6.5 Estimation error in earnings stability

Measure stability with the sample standard deviation s_n of ROA over n years. Suppose annual ROA is approximately independent and normal, with population standard deviation σ_i. Then $\mathrm{Var}(s_n) \approx \sigma_i^2/(2(n-1))$, with relative standard error $1/\sqrt{2(n-1)}$: 35% for $n = 5$ and 24% for $n = 10$. Thus, when ranking companies by five-year standard deviation, two companies whose true volatilities differ by 30% can easily be ranked in the opposite order in the sample. These values show that a short-history standard deviation has low relative precision; they do not prove it is noisier than every level estimate, whose precision also depends on the mean, volatility, sample size, and dependence structure. Practical implication: assign NaN when a company has fewer than n years; do not fill a five-year measure with a two-year standard deviation. When combining stability with level metrics, lower its weight or use a longer window. This is an approximation under normality and independence. Financial data often have serial dependence and heavy tails, so estimate actual error according to the data's dependence structure; this approximation cannot establish that stability is always noisier.

6.6 Same-version rule: a false change caused by a restatement

Suppose a company originally reports $N = 40$, $E = 400$, and ROE = 10%. It later restates the values to $N = 28$, $E = 350$. Using one version gives $40/400 = 10\%$ before the restatement and $28/350 = 8\%$ afterward. If only the numerator is updated after the restatement while the denominator remains from the original version (combining the database's "latest net income" with the "first ingested equity"), the result is $28/400 = 7\%$: an apparent change of -3 percentage points, versus the true -2 percentage-point change, overstated by 1 percentage point. Worse, using the restated values before the restatement was released is look-ahead (Chapter 24, Section 6.4). Therefore, retrieve an entire row at once with `latest_available_values`; all numerators and denominators must come from that row. This includes the prior year-end balance (the comparative balance in the same 10-K), which is why `assets_prev` and `equity_prev` are stored in the same row.

6.7 Financial firms

Banks and insurers have balance sheets dominated by financial liabilities. Interest is both a cost and operating revenue, and they do not have cost of goods sold in the same sense; the numerators for GP/A and OP are undefined, as are EV and EBITDA (Chapter 24). A bank's equity multiplier is often around 10, versus 1 to 2 for a nonfinancial company, so the same ROE has a different meaning in the two industries. Treatment: set GP/A and OP to NaN for financial firms (Novy-Marx's author manuscript also excludes them); rank ROE and ROA within industry; report coverage for each metric. In the constructed panel, the median bank equity multiplier is about 11.2, versus about 1.5 for nonfinancial companies.

6.8 Combining two standardized dimensions: the arithmetic of correlation

The following is an identity for standardized Pearson variables on the same paired sample. Uncorrelated does not mean independent; the specified correlation matrix must be valid, and the composite variance must be nonzero. Let z_1, z_2 be standardized and uncorrelated, and y standardized, with $\mathrm{Corr}(z_k, y) = \rho_k$. Then $\mathrm{Cov}(z_1 + z_2, y) = \rho_1 + \rho_2$, $\mathrm{Var}(z_1 + z_2) = 2$, and the correlation of the composite signal is $(\rho_1 + \rho_2)/\sqrt{2}$. In the constructed panel, IC is 0.0241 for GP/A and 0.0418 for −accruals; latent quality and accrual tendency are independent by construction, and the average same-date rank correlation of the two signals is −0.073 (near zero, but not zero). Under zero correlation, the prediction is $(0.0241+0.0418)/\sqrt{2} = 0.0466$. Adjusting for $r_{12} = -0.073$ gives $0.0659/\sqrt{2(1 + r_{12})} = 0.0659/1.362 = 0.0484$. The measured IC of the equal-weighted rank average is 0.0473, between the two predictions. This is a scale comparison only: averages across dates of Rank IC and rank correlation do not exactly obey the single-sample standardized Pearson identity. Conditions: the two signals are uncorrelated, weights are equal after separately standardizing each signal, and weights are not tuned on the test period. If their correlation is $r_{12} > 0$, the denominator becomes $\sqrt{2(1 + r_{12})}$ and the composite gain shrinks. This is the starting point for Chapter 28's discussion of factor correlations and weighting.

7. Financial interpretation

Quality has multiple dimensions. Profitability levels (GP/A, OP, ROA, ROE) ask "how much does the company earn?" Leverage asks "how much does it rely on borrowing?" Accruals ask "how much cash is in earnings?" Stability asks "how volatile are earnings?" Dimensions can point in opposite directions: a highly leveraged company can have high ROE but be fragile; a company with high accruals can report high earnings but little operating cash. Before compressing them into one "quality score," know what each dimension measures.

ROE mixes in leverage. In the constructed panel, cross-sectional variance of ROE is 5.6 times that of ROA (0.0233 versus 0.0042). The mean leverage D/A of the top ROE quintile, averaged across dates, is 0.419; for the bottom quintile it is 0.342; and for the full sample it is 0.340. The high-ROE group is more leveraged. Leverage in the top and bottom GP/A quintiles is nearly identical (0.332 and 0.329). These are numerical illustrations of the mechanisms in Sections 6.1 and 6.3.

Net-income measures contain accruals. In the constructed panel, profit N includes an accrual component negatively related to returns (a construction assumption). Net-income measures ROA (IC 0.0113) and ROE (0.016) are therefore weaker than GP/A, which does not contain accruals (0.0241). This does not show that GP/A is better in real markets. It shows only that if net income mixes in a component related to future returns in the opposite direction, using net income directly dilutes the signal. Whether real-world accruals have this direction must be tested with data.

The relation between quality and value is an empirical question. Fama and French (2015, Section 7) find that value stocks tend to have weaker profitability unconditionally; after controlling for other factors, value stocks behave like robustly profitable stocks. In the constructed panel, quality and value are independent by design, so it cannot answer this question. Published RMW and HML returns have correlation 0.095 (Section 11).

Economic meaning of RMW. Published RMW averages 0.25% per month ($t = 3.03$) over 1963–2026. It is an aggregate spread before costs and financing. It does not show which accounting metric is better or whether an individual-stock portfolio bought on quality can be implemented.

Availability timing. As in Chapter 24, Novy-Marx's author manuscript starts using a fiscal year's accounting data at the end of June in the next calendar year; Fama–French's RMW also forms in July of year t using fiscal-year $t-1$ data. Researchers must state their own lag rule.

7.1 Why profitability might relate to next-period returns

High profitability does not automatically imply high stock returns: if the price already fully reflects those cash flows, a buyer gets no free excess return. A risk hypothesis might ask whether highly profitable companies have lower risk and therefore lower required returns, or how different quality dimensions behave in bad states. A mispricing hypothesis might ask whether investors underweight earnings persistence or underreact to differences between cash and accruals. A proxy hypothesis might ask whether a measure stands in for industry, size, asset measurement, or leverage. Each question requires observations and tests designed in advance; the word "quality" alone does not imply a positive premium.

The accrual gap especially requires economic judgment. A growing company may increase receivables, temporarily making operating cash flow lower than profit; it may also be recognizing revenue aggressively. The same accrual ratio does not identify the cause. The course's constructed panel builds a negative effect of accrual tendency on returns into its generator to demonstrate how the signal behaves under that assumption; it does not decide the mechanism for real companies. The original Sloan and Piotroski papers have not been read here, so no empirical results from them are added. RMW is a published portfolio spread and cannot substitute for single-stock GP/A or OP fields; it also does not test the accounting quality in this example.

8. Numerical worked examples: calculate each step

All numbers are generated by `hand_examples()` and independently checked using a different implementation in `qa/a5/claude/recompute/ch26_recompute.py`. The companies are constructed for teaching.

8.1 Same increase in ROE: operating improvement versus more leverage

This section uses a simplified DuPont calculation with ending balances, unlike P4's average balances in Section 3.1. Amounts are in millions of dollars. Last year, Companies A and B were identical: net income 60, sales 1000, total assets 600, and equity 400. ROE = 60/400 = 15%; net margin 60/1000 = 6%; turnover 1000/600 = 1.667; equity multiplier 600/400 = 1.5; product $0.06 \times 1.667 \times 1.5 = 15\%$.

This year, Company A's net income rises to 90 and everything else stays fixed: margin 9%, turnover 1.667, multiplier 1.5, and ROE = 22.5%—an operating improvement. Company B borrows 100 to repurchase shares: equity falls to 300, debt rises by 100, after-tax interest is about 3.75, and net income is $60 - 3.75 = 56.25$. Sales and assets do not change. Margin falls to 5.625%, turnover remains 1.667, the multiplier is 600/300 = 2.0, and ROE is $0.05625 \times 1.667 \times 2 = 18.75\%$. Company B's margin falls even as ROE rises; the increase comes from the multiplier.

8.2 Effect of leverage on ROE

Let $\rho = 8\%$ and after-tax debt cost $i = 3\%$. For $D/E = 0, 0.5, 1, 2$, ROE is $8\%, 10.5\%, 13\%, 18\%$ (using ROE $= \rho + (\rho - i)D/E$). Balance-sheet check: assets 200, debt 100, equity 100, NOPAT $= 0.08 \times 200 = 16$, after-tax interest $0.03 \times 100 = 3$, net income 13, and ROE $= 13/100 = 13\%$.

8.3 Denominator: ending, average, and near-zero equity

Let $N = 40$ and assets be 1000.

Company	Beginning equity	Ending equity	Ending ROE	Average equity
Stable	400	400	10.0%	400
Shrinking equity	500	100	40.0%	300
Near-zero equity	300	2	2000%	151
Negative at year-end	100	−30	NaN	35 (unguarded)

Company	Average ROE	If ending equity is below 2% of assets
Stable	10.0%	Keep
Shrinking equity	13.3%	Keep
Near-zero equity	26.5%	Set ending ROE to NaN ($2 < 20$)
Negative at year-end	114.3% (unguarded)	Require both balances positive first: NaN

Interpretation: ending and average values differ most for companies whose equity changes rapidly. Averaging can hide negative ending equity by turning it positive (and reporting ROE of 114%). With a threshold, near-zero equity becomes NaN; report the fraction excluded.

8.4 Gross profit, operating profitability, and accruals

A company in this simplified example has identical beginning and ending assets, so 800 is also same-version average assets; the accrual example likewise uses 500 as average assets. Sales are 1000, cost of goods sold 600, SG&A 200, interest 20, total assets 800, and book equity 400. GP/A $= (1000 - 600)/800 = 50\%$; OP $= (1000 - 600 - 200 - 20)/400 = 45\%$. If interest is 60, GP/A is unchanged and OP is $= (1000 - 600 - 200 - 60)/400 = 35\%$: OP falls with leverage while GP/A does not. Two companies have net income 50 and assets 500. Company A's operating cash flow is 80, so its accrual ratio is $(50 - 80)/500 = -6\%$ (operating cash exceeds net income by 30). Company B's operating cash flow is 20, so the ratio is $+6\%$ (net income exceeds operating cash by 30, or 6% of assets).

8.5 False change from mixing versions

Originally, $N = 40, E = 400$ (ROE 10%); after restatement, $N = 28, E = 350$. Same-version ROE after restatement is $28/350 = 8\%$; the mixed-version result is $28/400 = 7\%$. The change from 10% to 7% is -3 percentage points, while the true same-version change from 10% to 8% is -2 percentage points.

8.6 Estimation error in stability

For a five-year standard deviation of ROA, relative standard error is $1/\sqrt{2 \times 4} = 0.354$. Suppose two companies' true standard deviations are 4% and 5.2%, a 30% difference. Each sample standard deviation has 35% relative error, so the sample ranking can readily reverse. Under a normal approximation, the standard deviation of the log ratio of two independent estimates is about $\sqrt{2} \times 0.354 = 0.5$; the true log ratio $\ln 1.3 = 0.26$ is only half a standard deviation, implying about a 30% chance of reversal. The exact value from the F distribution is 31%.

8.7 Combining two dimensions

Let $\rho_1 = 0.0241$ and $\rho_2 = 0.0418$. Then $(\rho_1 + \rho_2)/\sqrt{2} = 0.0466$. If the signals have correlation 0.5, $\sqrt{2(1 + 0.5)} = 1.732$, and composite correlation is $0.0659/1.732 = 0.0380$, 18% lower than when they are uncorrelated.

9. Python implementation

Field definitions are in the new module `quality.py`. Same-version fundamental selection reuses Chapter 24's `latest_available_values`, which can retrieve multiple columns at once with the entire row from one version. Ranking and IC reuse Chapter 23's `factor_research.py`.

Function	Input	Output and boundary
`average_balance`	Beginning and ending balances	$(c+p)/2$; with a `floor`, either balance at or below it gives NaN (for equity use `floor=0.0`).
`guarded_ratio`	Numerator, denominator, `floor` (number or table)	Preserves the numerator's sign; denominator at or below `floor` gives NaN.
`dupont`	Net income, sales, assets, equity	Net margin, turnover, equity multiplier, and ROE (product of the first three); margin requires positive sales; turnover requires positive sales/assets; the multiplier requires positive assets/equity; net income retains its sign; ROE requires positive sales, assets, and equity.
`gross_profit_to_assets`	Revenue, cost of goods sold, assets	$(S-C)/A$; caller must pass average assets $\bar{A}$ (function does not calculate an average); missing cost of goods sold for a financial firm gives NaN.
`operating_profitability`	Revenue, cost of goods sold, SG&A, interest, book equity	Fama–French (2015) OP; any of four missing numerator items gives NaN; equity must be positive.
`accrual_ratio`	Net income, net cash from operating activities, assets	$(N-F)/A$; caller must also pass average assets.
`leverage_ratio`	Debt, assets	D/A; negative debt raises an error.
`earnings_stability`	Annual-ratio history (rows are fiscal years)	Sample standard deviation over the latest n years; fewer than `min_years` gives NaN.
`roe_from_leverage`	ρ, i, D/E	$\rho+(\rho-i)D/E$; negative D/E or nonfinite input raises an error.

Default offline entry point (run from the I3 reader-package root):

```
.venv/bin/python run_notebook.py notebooks/ch26_quality.ipynb
```

The default is `RUN_FRENCH=False`. The stability example uses 100,000 worlds and seed 11 (the body's historical example uses 200,000). The composite example uses 400,000 observations and resets seed 5 for each r_{12} (the body uses 600,000 and a continuous RNG). `quality_world` and `quality_study` default to seed 20261107. The runner also writes an execution copy and status under `runs/execution_<stem>_<uuid>/`. The notebook's English-language layer and package-wide paths are assembled in BIL23. The following historical full CLI and test references are not the default offline command:

```
.venv/bin/python code/ch26_quality.py
.venv/bin/python -m pytest tests/test_ch26_quality.py tests/test_ch26_workflow.py -q
```

By default, this builds only a chapter-level RUN_DIR and does not write CSV, PNG, or JSON files or generate a historical figure library.

Frequency: accounting measures are annual; signals and returns are monthly. `signal_time` is the final instant of month-end (UTC). Version selection follows Chapter 24, Section

6.4, using `latest_available_values`. Each filing contains both the current and prior-year balance sheets, so average balances come from the same filing. CLI seed: 20261107.

Code segment one: DuPont decomposition and leverage effect (Sections 8.1 and 8.2).

```
import pandas as pd

from quant_course.quality import dupont, roe_from_leverage

idx = pd.period_range("2026-06", periods=1, freq="M")
mk = lambda v: pd.DataFrame([[v]], index=idx, columns=["f"])
before = dupont(mk(60.0), mk(1000.0), mk(600.0), mk(400.0))
ops = dupont(mk(90.0), mk(1000.0), mk(600.0), mk(400.0))          # operations
    ↪ improve: margin 6% -> 9%
lev = dupont(mk(56.25), mk(1000.0), mk(600.0), mk(300.0))         # buy back 100 of
    ↪ equity with debt; after-tax interest 3.75
r = lambda d, k: float(d[k].iloc[0, 0])
assert abs(r(before, "roe") - 0.15) < 1e-12 and abs(r(ops, "roe") - 0.225) < 1e-12
    ↪ and abs(r(lev, "roe") - 0.1875) < 1e-12
assert r(lev, "net_margin") < r(before, "net_margin") and r(lev, "equity_multiplier")
    ↪  > r(before, "equity_multiplier")   # ROE rose although margin fell
assert abs(roe_from_leverage(0.08, 0.03, 1.0) - 0.13) < 1e-12 and abs((16 - 0.03 *
    ↪ 100) / 100 - 0.13) < 1e-12              # balance-sheet check
print(round(r(ops, "roe"), 4), round(r(lev, "roe"), 4))
```

Code segment two: denominator rules (Section 8.3).

```
import numpy as np
import pandas as pd

from quant_course.quality import average_balance, guarded_ratio

idx = pd.period_range("2026-06", periods=1, freq="M")
cols = ["steady", "shrinking", "tiny", "negative"]
ni = pd.DataFrame([[40.0] * 4], index=idx, columns=cols)
end = pd.DataFrame([[400.0, 100.0, 2.0, -30.0]], index=idx, columns=cols)
prev = pd.DataFrame([[400.0, 500.0, 300.0, 100.0]], index=idx, columns=cols)
assets = pd.DataFrame([[1000.0] * 4], index=idx, columns=cols)
roe_end = guarded_ratio(ni, end).iloc[0]
roe_avg_unguarded = guarded_ratio(ni, average_balance(end, prev)).iloc[0]
roe_avg = guarded_ratio(ni, average_balance(end, prev, floor=0.0)).iloc[0]
roe_floor = guarded_ratio(ni, end, floor=0.02 * assets).iloc[0]
assert abs(roe_end["tiny"] - 20.0) < 1e-12 and np.isnan(roe_end["negative"])
    ↪               # 2000%, and no ratio for negative equity
assert abs(roe_avg_unguarded["negative"] - 40 / 35) < 1e-12
    ↪                # the average hides the negative balance: 114%
assert np.isnan(roe_avg["negative"]) and abs(roe_avg["shrinking"] - 40 / 300) < 1e-12
assert np.isnan(roe_floor["tiny"]) and abs(roe_floor["shrinking"] - 0.4) < 1e-12
    ↪               # 2 < 2% of assets (20): dropped
print(pd.DataFrame({"ending": roe_end, "average, unguarded": roe_avg_unguarded, "
    ↪ average, both > 0": roe_avg, "ending, 2% floor": roe_floor}).round(3))
```

Code segment three: GP/A, OP, accruals, and leverage (Section 8.4).

```
import pandas as pd

from quant_course.quality import accrual_ratio, gross_profit_to_assets,
    ↪ operating_profitability

idx = pd.period_range("2026-06", periods=1, freq="M")
mk = lambda *v: pd.DataFrame([list(v)], index=idx, columns=["low_debt", "high_debt"
    ↪ ][: len(v)])
rev, cogs, sga = mk(1000.0, 1000.0), mk(600.0, 600.0), mk(200.0, 200.0)
interest, assets, equity = mk(20.0, 60.0), mk(800.0, 800.0), mk(400.0, 400.0)
gpa = gross_profit_to_assets(rev, cogs, assets).iloc[0]
op = operating_profitability(rev, cogs, sga, interest, equity).iloc[0]
assert (gpa == 0.5).all()
    ↪           # GP/A ignores interest
assert abs(op["low_debt"] - 0.45) < 1e-12 and abs(op["high_debt"] - 0.35) < 1e-12
    ↪          # OP falls when interest rises
acc = accrual_ratio(mk(50.0, 50.0), mk(80.0, 20.0), mk(500.0, 500.0)).iloc[0]
assert abs(acc["low_debt"] + 0.06) < 1e-12 and abs(acc["high_debt"] - 0.06) < 1e-12
    ↪          # cash-backed vs not (columns reused as labels)
print(gpa.to_dict(), op.round(3).to_dict(), acc.round(3).to_dict())
```

Code segment four: same-version rule and restatement (Section 8.5).

```
import pandas as pd

from quant_course.factor_signals import latest_available_values
from quant_course.quality import guarded_ratio

versions = pd.DataFrame({
    "security_id": ["S1", "S1"], "period_end": pd.to_datetime(["2022-12-31", "
    ↪ 2022-12-31"]),
    "available_at": pd.to_datetime(["2023-03-15 16:05", "2023-11-20 16:05"], utc=True
    ↪ ),
    "ingested_at": pd.to_datetime(["2023-03-15 16:12", "2023-11-20 16:12"], utc=True)
    ↪ ,
    "net_income": [40.0, 28.0], "equity": [400.0, 350.0]})
    ↪    # restated NI and equity
months = pd.period_range("2023-10", "2023-11", freq="M")
pit = latest_available_values(versions, months, ["net_income", "equity"])
same_version = guarded_ratio(pit["net_income"], pit["equity"])["S1"]
old_equity = latest_available_values(versions, months[:1], ["equity"])["equity"]["S1"
    ↪ ].iloc[0]   # the equity known in October, kept after the restatement
mixed_nov = pit["net_income"]["S1"].iloc[1] / old_equity
assert abs(same_version.iloc[0] - 0.10) < 1e-12 and abs(same_version.iloc[1] - 0.08)
    ↪ < 1e-12
assert abs(mixed_nov - 0.07) < 1e-12
    ↪      # new numerator over the old denominator: a false -3 pp change
print(same_version.round(3).to_dict(), round(mixed_nov, 3))
```

Code segment five: estimation error in earnings stability and probability of a rank reversal (Sections 6.5 and 8.6).

```
import numpy as np

rng = np.random.default_rng(11)
worlds = 200_000
from scipy import stats

for n in (5, 10):
    expected = float(stats.f.sf((0.06 / 0.04) ** 2, n - 1, n - 1))           # exact
    ↪ : s1^2/s2^2 = (sigma1/sigma2)^2 F(n-1, n-1); 0.226 and 0.121
    s1 = rng.normal(0.0, 0.04, (worlds, n)).std(axis=1, ddof=1)             # true
    ↪ sigma 4%
    s2 = rng.normal(0.0, 0.06, (worlds, n)).std(axis=1, ddof=1)             # true
    ↪ sigma 6% (50% higher)
    reversed_share = float((s1 > s2).mean())                                  #
    ↪ sample ranking opposite to the truth
    assert abs(reversed_share - expected) < 0.01                              # the
    ↪ lognormal approximation (0.209, 0.112) is a little low
    print(n, round(reversed_share, 3), round(1 / np.sqrt(2 * (n - 1)), 3))
```

Code segment six: IC after combining two uncorrelated signals (Sections 6.8 and 8.7).

```
import numpy as np

rng = np.random.default_rng(5)
rho1, rho2, n = 0.0241, 0.0418, 600_000


def composite_ic(r12: float) -> float:
    """Correlation of z1 + z2 with y when corr(z1, y) = rho1, corr(z2, y) = rho2 and
    ↪ corr(z1, z2) = r12."""
    cov = np.array([[1.0, r12, rho1], [r12, 1.0, rho2], [rho1, rho2, 1.0]])
    z1, z2, y = rng.multivariate_normal(np.zeros(3), cov, size=n).T
    return float(np.corrcoef(z1 + z2, y)[0, 1])


zero, half = composite_ic(0.0), composite_ic(0.5)
assert abs(zero - (rho1 + rho2) / np.sqrt(2)) < 0.004 and abs(half - (rho1 + rho2) /
    ↪ np.sqrt(2 * 1.5)) < 0.004
assert half < zero                                                            # a
    ↪ correlated second signal shrinks the gain
print(round(zero, 4), round(half, 4))
```

10. Reading the figures

Figure 26-1: Leverage, ROE, and near-zero equity. Left: with an after-tax debt cost of 3% (dashed line), three curves show ROE against D/E for operating returns $\rho = 4\%, 8\%, 12\%$. When $\rho > i$, ROE rises with slope $\rho - i$; when $\rho < i$, it falls. Right: ROE for 300 nonfinancial companies in one month of the constructed panel. The left column uses unguarded ending-equity ROE; the right uses average equity with both balances positive and requires equity to exceed 5% of assets. The unguarded column has extreme values (hundreds of percent); the guarded values fall into a more reasonable range. Interpretation: near-zero-equity outliers are not economic information. Constructed data.

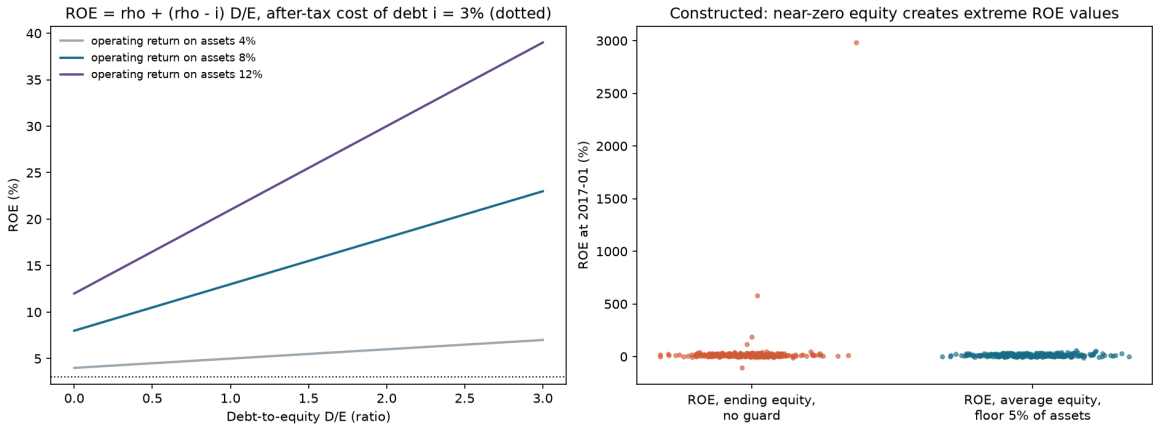


Figure 26-1: Leverage's effect on ROE and outliers caused by near-zero equity

Figure 26-2: IC for different quality measures. Constructed panel with 300 nonfinancial companies and 132 months, using fixed seed 20261107. Bars show mean Spearman IC (vertical lines are approximately plus or minus two standard errors), in order: ROE (ending equity), ROE (average equity), ROE (average equity with equity above 5% of assets), ROA, GP/A, OP, −accruals, −leverage, and the equal-weighted rank average of GP/A and −accruals. Interpretation: net-income measures (ROE, ROA) are weaker than gross profit (GP/A) and operating profitability (OP); −accruals is strongest, and the composite is strongest; −leverage is near zero (leverage is unrelated to returns by construction). These are results of the constructed setup, not estimates for real markets.

Figure 26-3: Published RMW. Left: compounded cumulative RMW monthly spread (log scale, starting at 1, July 1963 through August 2026). Right: rolling 120-month mean RMW (percent per month). Interpretation: a slow long-run rise, positive most of the time, with a drawdown around 2000 (maximum drawdown about 42%, occurring in 2000-02). This is a published aggregate spread, before costs and financing.

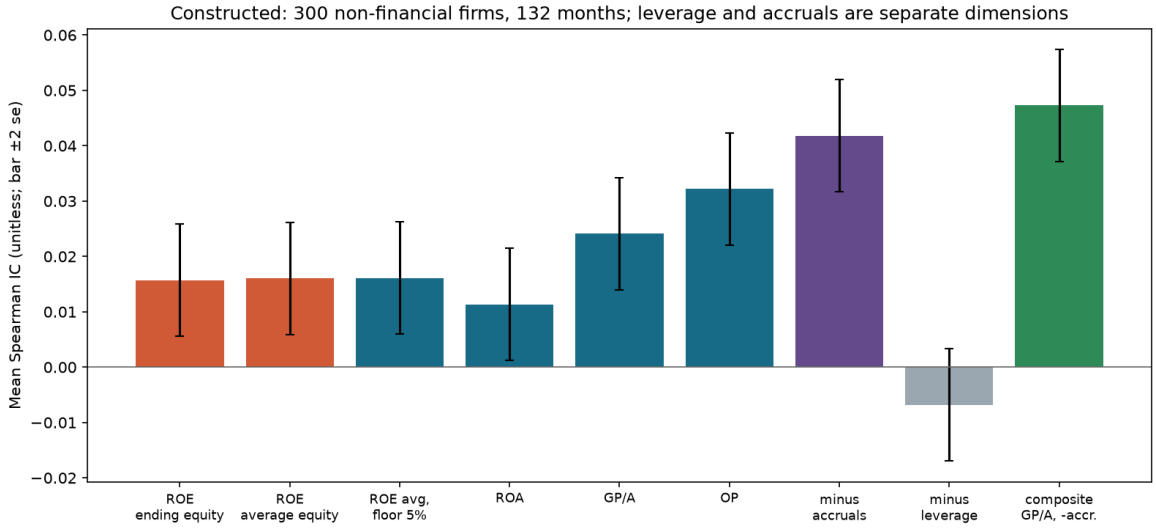


Figure 26-2: IC for quality measures and their composite

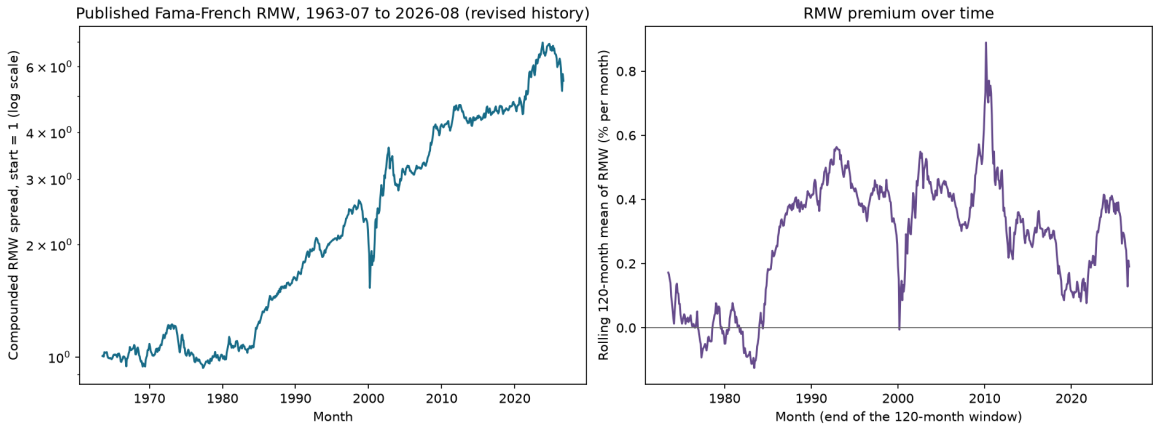


Figure 26-3: Cumulative and rolling mean of Fama–French RMW

11. Historical market case

Data and limitations. Use the French five-factor (2×3) monthly data frozen on 2026-10-05: Ken French Data Library: F-F Research Data 5 Factors 2x3. The processed CSV has SHA-256 `833bd925b49651428781c789c13a6ac7454d4676449e6d119782050046a57eea` and 758 months (July 1963 through August 2026), in decimal monthly spread returns. Publicly downloadable does not mean freely redistributable. RMW construction is described in Section 5 and the source notes: six value-weighted portfolios are formed by size and operating profitability using fiscal year $t-1$, formed in July and held through the following June; only firms with positive equity and available revenue and cost items are included. This is a published aggregate return, not a stock-level profitability signal, not point-in-time statement-version data, and not an execution record.

Descriptive results.

Period	Months	Mean (% per month)	t	Standard deviation (% per month)
1963-07 through 1999-12	438	0.173	2.26	1.60
2000-01 through 2026-08	320	0.359	2.16	2.98
2007-01 through 2020-12	168	0.234	1.90	1.59
Full sample	758	0.251	3.03	2.29

Maximum drawdown of compound returns is 41.8%, occurring in 2000-02. Correlation with market excess return is −0.19, with SMB −0.33, with HML +0.10, and with CMA +0.02. The reported t-statistics treat months as independent observations and are naive; the two subperiods were selected after the fact, so their t-statistics are not confirmatory tests. Volatility is notably higher after 2000 (2.98% versus 1.60%).

What can and cannot be said. We can say that the published profitability spread averaged positive returns over 1963–2026; mean returns are positive in both subperiods, while standard deviations differ by nearly a factor of two; RMW is negatively correlated with SMB and nearly uncorrelated with HML. We cannot say: (1) that any one accounting metric (GP/A, OP, ROE, or accruals) is the better quality measure, since RMW uses only operating profitability; (2) that high-OP individual stocks earn higher returns, since RMW is the arithmetic difference of six value-weighted portfolios; (3) what the quality–value relation is for individual stocks: Fama–French's (2015, Section 7) unconditional and conditional descriptions apply to their sample, and this chapter has no individual-stock test; or (4) that the strategy is tradable, has capacity, or is profitable after costs.

What is missing for stock-level quality research. We need every version and `available_at` for each company-year's revenue, cost of goods sold, SG&A, interest, net income, operating cash flow, assets, equity, and debt; historical stock universes and delistings; historical industry classifications; prices and market capitalizations. The SEC Financial Statement Data Sets listed in Chapter 24 provide as-filed XBRL values and acceptance timestamps from 2009 onward, but mappings from tags (Extensible Business Reporting Language, XBRL, labels) to accounting concepts and coverage/consistency of items such as cost of goods sold require separate review; the datasets do not contain prices or delisting returns. No such data were downloaded for this chapter. The Project 4 preparation-layer gap table lists these items.

12. Common mistakes

Mistake 1: interpreting higher ROE as operating improvement. Section 8.1: Company B's margin falls while its ROE rises. Fix: always report the three DuPont factors, or use GP/A, which does not directly deduct interest, in a comparison holding operations and assets fixed; report leverage as well. ROA uses net income after interest and still contains financing information.

Mistake 2: averaging away negative equity. Section 8.3: ending equity −30 and beginning equity 100 average to 35, producing ROE of 114%. Fix: require both balances to be positive before averaging.

Mistake 3: allowing near-zero equity into the ranking. In the constructed panel, Pearson IC for unguarded ending-equity ROE is 0.0017 ($t = 0.36$), rising to 0.0177 ($t = 3.22$) after the guard. Spearman ICs in this construction are similar (0.0157 and 0.0161). Spearman is robust to extreme magnitude changes that do not alter ranks, but changes in ranks, pairwise reordering after filtering, and changes in the universe can still affect it. Fix: set a denominator threshold and report exclusions; use Rank IC by default for rank-based signals.

```
import numpy as np
import pandas as pd

from quant_course.factor_research import information_coefficient
from quant_course.quality import guarded_ratio

rng = np.random.default_rng(8)
n = 300
quality = rng.standard_normal(n)
equity = np.exp(rng.normal(5.0, 0.3, n)); equity[:6] = [1.0, 0.5, 2.0, 0.8, 1.5, 3.0]
    ↪           # six near-zero equity firms
income = 40 + 15 * quality + rng.normal(0, 5, n)
ret = 0.01 * quality + 0.03 * rng.standard_normal(n)
idx = pd.period_range("2026-06", periods=1, freq="M")
frame = lambda v: pd.DataFrame([v], index=idx, columns=range(n))
raw = guarded_ratio(frame(income), frame(equity)).iloc[0].to_numpy()
    ↪            # no floor: ROE of ~4000% for the tiny firms
guard = guarded_ratio(frame(income), frame(equity), floor=frame(np.full(n, 20.0))).
    ↪ iloc[0].to_numpy()
p_raw, p_guard = information_coefficient(raw, ret, method="pearson"),
    ↪ information_coefficient(guard, ret, method="pearson")
s_raw = information_coefficient(raw, ret, method="spearman")
assert p_guard > p_raw + 0.1 and abs(s_raw - information_coefficient(guard, ret,
    ↪ method="spearman")) < 0.1
print(round(p_raw, 3), round(p_guard, 3), round(s_raw, 3))
```

Mistake 4: mixing versions. Section 8.5: using a new numerator over an old denominator gives a 3-percentage-point decline instead of the true 2-point decline, an extra 1-point drop. Fix: select the whole row at once.

Mistake 5: calculating GP/A or OP for financial firms. Banks have no cost of goods sold in this definition; replacing revenue with interest income and cost with interest expense produces a different measure. Fix: set these two metrics to NaN for financial firms, rank ROE and ROA within industry, and report coverage. In the constructed panel, all bank GP/A values are NaN (CLI assertion).

Mistake 6: treating net income as cash. Section 8.4: two companies with the same net income have operating cash flow of 80 and 20; this does not identify, transaction by transaction, which profit was received in cash. Fix: calculate the accrual ratio separately and read net-income measures alongside accruals.

Mistake 7: ranking stability from only a few years. Sections 6.5 and 8.6: the relative error of a five-year standard deviation is 35%; the probability that sample ranks reverse for two companies whose true standard deviations differ by 30% is about 30%. When the true difference is 50% (26-E06), the exact reversal probabilities are about 23% for 5 years and 12% for 10 years. Fix: require enough years, reduce the weight, or use a longer window.

Mistake 8: tuning composite weights on the test period. The composite assumptions in Sections 8.7 and 6.8 set equal weights in advance. Choosing weights based on test-period IC inflates the composite IC through post hoc selection (Chapter 22). Fix: fix weights in the plan or estimate them only in the training period.

13. Quantitative research application

Quality field contract (Project 4 signal layer).

Field	Source	Time fields	Unit	Missingness and convention
`revenue`, `cogs`, `sga`, `interest`	Income statement	`period_end`, `available_at`, `ingested_at`	Reporting currency	`cogs` is missing for financial firms; set GP/A and OP to NaN.
`net_income`, `cfo` (cash flow from operations)	Income statement, cash-flow statement	Same as above	Reporting currency	Must be from the same version as assets.
`assets`, `assets_prev`, `equity`, `equity_prev`, `debt`	Balance sheet (current-year and prior-year comparative balances)	Same as above	Reporting currency	Current P4 average equity first requires both year-end equity values positive, then average equity above 5% of average assets; average assets follow the interface convention.
`gp_a`, `neg_accrual`, `roe_avg`	Current P4 signal calculation	`signal_time`	Decimal	Calculate from one same-version row; OP, ROA, and leverage are separate teaching measures.
`industry`	Versioned classification	`available_at`	Label	Used for within-industry ranking.

Research workflow (constructed panel). Freeze a plan: the hypothesis that companies with higher gross profit to assets and a lower accrual ratio on the same date have higher next-month returns; the signal is the equal-weighted average of the same-date percentiles of those two dimensions; the primary statistic is the t-statistic of Rank IC. Evaluate each dimension first, then the composite; fix weights before observing returns. Results: 132 months and 300 nonfinancial companies; GP/A Rank IC 0.0241 ($t = 4.58$), −accruals 0.0418 ($t = 8.31$), composite 0.0473

($t = 9.00$, hit rate 72.5%); mean monthly returns for the highest and lowest GP/A quintiles are 1.039% and 0.594%, a long-short spread of 0.445% ($t = 4.26$). These directions and magnitudes are known under the constructed setup, so they demonstrate that the functions and workflow behave as specified, not a real-world effect.

Constraints for Project 4. (1) Retrieve an entire row at once with `latest_available_values` for every ratio. (2) Report coverage and exclusions for each metric. (3) Handle financial firms separately. (4) Report net-income measures together with accruals and leverage. (5) Require enough years for stability. (6) Fix composite weights in advance; multiple dimensions enter Chapter 28's factor-correlation and crowding analysis. (7) Record every variant in Chapter 22's ledger.

13.1 Metric mapping and coverage: first ask "who is still rankable?"

The current P4 quality fields are only `gp_a`, `neg_accrual`, and `roe_avg`, with formulas corresponding to 44.4444%, +3.3333%, and 25.7143% in Section 3.1. The current seven fields do not include OP. The standalone `quality.operating_profitability` function is available for teaching; the existence of this function does not mean P4 has integrated OP. The paper's gross profits-to-assets, the course's average-asset GP/A, and French's equity-denominator OP are three distinct definitions; do not substitute one for another. RMW is a portfolio return formed under French's convention, not company-level OP input.

P4's actual `roe_avg` guard operates in two steps: `average_balance(equity, equity_prev, floor=0.0)` first excludes any record with a nonpositive year-end equity balance; then `guarded_ratio(..., floor=0.05 * assets_avg)` excludes average equity that is not strictly greater than 5% of average assets. A value equal to the threshold is excluded. This is a prespecified interface threshold, not a universal definition of ROE. The 2% threshold in Section 8.3 and the 5% ending-equity threshold in 26-E03 are separate teaching-policy examples and must not be described as the P4 rule.

Consider four companies with average assets of 1000 and net income of 40, and beginning/ending equity as follows: A (80,120), B (10,30), C (120,-20), and D (40,60). Company A has average equity 100, greater than 50, so ROE of 40% is valid. B has average equity 20, at or below 50, and is excluded. C is excluded first because ending equity is nonpositive (its unguarded average is exactly 50). D's average equity is 50, equal to the threshold, and is excluded. Of four companies with complete inputs, only one passes: valid ROE coverage is 25%, with 75% lost. Accruals or GP/A may still be available according to their own fields. Missingness is not a zero quality score. Report coverage by size and industry and each failure reason, since filtering may systematically change the universe.

Raw quality fields enter the unified winsorization, industry/size residualization, standardization, and synthesis flow in Chapter 24, Section 13.1. Missing GP/A for financial firms changes which dimensions are available; renormalizing within style does not restore missing economic evidence. A residualized quality score and the final weight projection by industry are separate stages. Distinguish older A6 weights outside the support from C13's limited correction: listed-support projection, gross restoration to 2, and the 60-name threshold are accepted; execution and drift residuals are reported separately. See the current-interface entry in Chapter 24, Section

13.1. Real eligibility, borrowing, fees, capacity, and a complete real-data P4 remain unfinished; old outputs have not thereby been rerun.

14. Chapter summary

Quality comprises several accounting dimensions. ROE is the product of net margin, turnover, and leverage; leverage can raise ROE without improving operations: ROE $= \rho + (\rho - i)D/E$. GP/A does not directly deduct interest and does not directly contain an equity-financing multiplier when operations and assets are held fixed. Net-income measures mix in accruals. The accrual ratio is the net-income/operating-cash-flow gap scaled by average assets, not the gap as a share of earnings. A five-year earnings-standard-deviation estimate has about 35% relative error and noisy rankings. Denominators must be positive and away from zero; require both year-end balances to be positive before averaging; draw the numerator and denominator from the same financial-statement version; handle industries without cost of goods sold separately. For two uncorrelated dimensions, composite IC is approximately $(\rho_1 + \rho_2)/\sqrt{2}$; correlation reduces the gain. Published RMW averages 0.25% per month ($t = 3.03$), is an aggregate spread, and does not test stock-level signals. Scope: effects in the constructed panel are assumptions; RMW is revised historical data; this chapter has no point-in-time stock-level data.

15. Exercises

26-E01 (by hand). A company's first-year net income is 36, sales 600, total assets 400, and equity 150. In year two, net income is 39.6, sales 660, total assets 440, and equity 110 (it borrowed to repurchase shares). Calculate net margin, turnover, equity multiplier, and ROE for both years; explain the source of the ROE change.

26-E02 (by hand). Operating return $\rho = 10\%$, after-tax debt cost $i = 4\%$, and $D/E = 1.5$. Find ROE. Then set $\rho = 3\%$ and $i = 4\%$; find ROE and interpret its sign.

26-E03 (by hand). Net income is 24 and assets are 800. Three companies have (beginning equity, ending equity): (200, 180), (150, 6), and (80, −10). Find ending ROE, unguarded average ROE, average ROE only when both balances are positive, and ending ROE subject to ending equity being above 5% of assets.

26-E04 (by hand). Revenue is 2400, cost of goods sold 1500, SG&A 500, interest 100, total assets 2000 (same-version beginning and ending assets are both 2000, so average assets are also 2000), and book equity is 800. Find GP/A and OP. If interest is 300, what are the two values? With net income 120, operating cash flow 60, and average assets 1000, what is the accrual ratio? What if operating cash flow is 180?

26-E05 (by hand). Originally, net income is 55 and equity is 500; after restatement, net income is 44 and equity is 440. Find both same-version ROEs, the value using the new numerator over the old denominator, and the false change caused by mixing versions.

26-E06 (calculation). What is the relative standard error of a standard deviation estimated over 5 years and over 10 years? If two companies' true standard deviations are 4% and 6%, what are the approximate probabilities that their sample rankings reverse using 5-year and 10-year samples?

26-E07 (calculation). Two standardized signals have correlations $\rho_1 = 0.03$ and $\rho_2 = 0.02$ with the label. What is the IC of the equally weighted composite if they are uncorrelated and if their correlation is 0.3?

26-E08 (concept and calculation). A bank has ROA of 1% and an equity multiplier of 11; a software company has ROA of 8% and an equity multiplier of 1.5. Calculate both ROEs and explain why ranking them by ROE across the full market is inappropriate. What alternative would you use?

26-N01 (filing version and coverage). Recalculate the five ratios in Section 3.1 and label the sign, denominator, and annual unit; explain why February values are unavailable and why French OP does not switch to average equity. Use the four companies in Section 13.1 to check P4's ROE threshold and coverage loss, and explain why GP/A and RMW cannot be substituted for each other.

16. Programming exercises

26-P01. Use `dupont` to reproduce 26-E01 and assert that the entire ROE change comes from the equity multiplier (net margin and turnover are unchanged). Acceptance criterion: assertions for all three factors pass.

26-P02. Construct a version table with one restatement, use `latest_available_values` to retrieve numerator and denominator together, and assert the ROEs before and after restatement as well as that "new numerator over old denominator" differs from both. Acceptance criterion: all three assertions pass.

26-P03. Simulate two signals with correlation 0.5 and correlations 0.03 and 0.02 with the label. Calculate the IC of their equally weighted composite and compare it with $(\rho_1 + \rho_2)/\sqrt{2(1 + r_{12})}$. Acceptance criterion: error below 0.005.

17. Challenge problems

26-C01 (derivation). Extend Section 6.1. Let pretax operating profit (earnings before interest and taxes, EBIT) over assets be $r_a = \text{EBIT}/A$, tax rate be τ, and pretax debt rate be r_d. Derive $\text{ROE} = (1 - \tau)\big[r_a + (r_a - r_d)D/E\big]$ and explain the mapping to ρ and i in Section 6.1. Discuss the effect of leverage on ROE and ROE volatility when $\rho < i$. Prerequisite: identity $A/E = 1 + D/E$. Grading: derivation 50, mapping 20, discussion 30.

26-C02 (open-ended). For a Project 4 quality signal, write a data-gap table and frozen plan. For every field, state whether it exists in the course's current data, a source where real data could be obtained, its time fields, and missing-data rules. Explain which conclusions cannot be written without these data. List the three checks required before combining quality and value: correlation, coverage, and timing. Grading: gap table 30, frozen plan 30, claims that cannot be made 20, pre-combination checks 20.

18. Complete solutions and reference implementations

26-E01. Year one: net margin $36/600 = 6\%$, turnover $600/400 = 1.5$, equity multiplier $400/150 = 2.667$, and ROE $= 36/150 = 24\%$ (the product $0.06 \times 1.5 \times 2.667 = 0.24$). Year two: net margin $39.6/660 = 6\%$, turnover $660/440 = 1.5$, equity multiplier $440/110 = 4.0$, and ROE $= 39.6/110 = 36\%$. Margin and turnover do not change; the multiplier rises from 2.667 to 4.0, accounting for all 12 percentage points of ROE increase.

26-E02. ROE $= 0.10 + (0.10 - 0.04) \times 1.5 = 0.19$. When $\rho = 3\% < i = 4\%$, $0.03 + (0.03 - 0.04) \times 1.5 = 0.015$ (1.5%). Leverage lowers ROE below ρ because asset returns are below borrowing costs; more borrowing loses more.

26-E03. Ending ROEs: $24/180 = 13.3\%$; $24/6 = 400\%$; and $24/(-10) = -240\%$ (the sign has lost its meaning, so report NaN). Unguarded averages: $(200+180)/2 = 190 \to 12.6\%$; $(150+6)/2 = 78 \to 30.8\%$; and $(80-10)/2 = 35 \to 68.6\%$. Require both balances positive before averaging: 12.6%, 30.8%, NaN. Require ending equity greater than 5% of assets (40): 13.3%, NaN ($6 < 40$), NaN ($-10 < 40$). The third company's 68.6% is an artifact of averaging away insolvency at year-end.

26-E04. GP/A $= (2400-1500)/2000 = 45\%$; OP $= (2400-1500-500-100)/800 = 37.5\%$. With interest of 300, GP/A remains 45%; OP is $= (2400 - 1500 - 500 - 300)/800 = 12.5\%$. Accrual ratios: $(120-60)/1000 = +6\%$ and $(120-180)/1000 = -6\%$.

26-E05. Same version: original $55/500 = 11\%$ and restated $44/440 = 10\%$, a true change of -1 percentage point. Mixing gives $44/500 = 8.8\%$, an apparent change of $8.8 - 11 = -2.2$ percentage points, 1.2 percentage points more decline than the true change.

26-E06. Relative standard error $1/\sqrt{2(n-1)}$: 0.354 for $n = 5$ and 0.236 for $n = 10$. The standard deviation of the log ratio of two independent estimates is approximately $\sqrt{2}$ times the relative standard error: 0.5 for $n = 5$ and 0.333 for $n = 10$. The true log ratio is $\ln(6/4) = 0.405$. Under the normal approximation for the log ratio, reversal probabilities are $\Phi(-0.405/0.5) = \Phi(-0.81) = 0.209$ for $n = 5$ and $\Phi(-0.405/0.333) = \Phi(-1.216) = 0.112$ for $n = 10$. These are approximations and slightly low. The exact sample-variance ratio satisfies $s_1^2/s_2^2 \sim (\sigma_1/\sigma_2)^2 F(n-1, n-1)$, giving $P(F_{n-1,n-1} > 2.25) = 0.226$ for $n = 5$ and 0.121 for $n = 10$. The simulation in Section 9, code segment five, centers around the exact values.

26-E07. If uncorrelated: $(0.03 + 0.02)/\sqrt{2} = 0.0354$. If correlation is 0.3: $0.05/\sqrt{2(1+0.3)} = 0.05/1.612 = 0.0310$.

26-E08. Bank ROE = 1% × 11 = 11%; software ROE = 8% × 1.5 = 12%. Their ROEs are close, but net income/assets (ROA) differs by a factor of eight. ROA here already deducts interest, so it is not the operating return ρ before financing in Section 6.1. The bank's 11% comes with 11-fold leverage; the software company's 12% comes from asset profitability. Ranking by ROE across the whole market puts very different risks on one scale. Rank within industry and interpret alongside financing and asset definitions; for nonfinancial companies GP/A may also be examined. Handle financial firms separately; do not describe ROA as excluding leverage.

26-P01 reference implementation

```
import pandas as pd

from quant_course.quality import dupont

idx = pd.period_range("2026-06", periods=1, freq="M")
mk = lambda v: pd.DataFrame([[v]], index=idx, columns=["f"])
y1 = dupont(mk(36.0), mk(600.0), mk(400.0), mk(150.0))
y2 = dupont(mk(39.6), mk(660.0), mk(440.0), mk(110.0))
g = lambda d, k: float(d[k].iloc[0, 0])
assert abs(g(y1, "roe") - 0.24) < 1e-12 and abs(g(y2, "roe") - 0.36) < 1e-12
assert abs(g(y1, "net_margin") - g(y2, "net_margin")) < 1e-12 and abs(g(y1, "
    ↪ asset_turnover") - g(y2, "asset_turnover")) < 1e-12
assert g(y2, "equity_multiplier") > g(y1, "equity_multiplier")                    #
    ↪  all of the ROE change is leverage
```

26-P02 reference implementation

```
import pandas as pd

from quant_course.factor_signals import latest_available_values
from quant_course.quality import guarded_ratio

versions = pd.DataFrame({
    "security_id": ["S1", "S1"], "period_end": pd.to_datetime(["2022-12-31", "
    ↪ 2022-12-31"]),
    "available_at": pd.to_datetime(["2023-03-15 16:00", "2023-09-10 16:00"], utc=True
    ↪ ),
    "ingested_at": pd.to_datetime(["2023-03-15 16:10", "2023-09-10 16:10"], utc=True)
    ↪ ,
    "net_income": [55.0, 44.0], "equity": [500.0, 440.0]})
months = pd.period_range("2023-08", "2023-09", freq="M")
pit = latest_available_values(versions, months, ["net_income", "equity"])
roe = guarded_ratio(pit["net_income"], pit["equity"])["S1"]
old_equity = pit["equity"]["S1"].iloc[0]
mixed = pit["net_income"]["S1"].iloc[1] / old_equity
assert abs(roe.iloc[0] - 0.11) < 1e-12 and abs(roe.iloc[1] - 0.10) < 1e-12
assert abs(mixed - 0.088) < 1e-12 and mixed != roe.iloc[0] and mixed != roe.iloc[1]
```

26-P03 reference implementation

```
import numpy as np

rng = np.random.default_rng(6)
n, r12, rho1, rho2 = 600_000, 0.5, 0.03, 0.02
cov = np.array([[1.0, r12, rho1], [r12, 1.0, rho2], [rho1, rho2, 1.0]])          #
    ↪ corr(z1,y)=rho1, corr(z2,y)=rho2, corr(z1,z2)=r12
z1, z2, y = rng.multivariate_normal(np.zeros(3), cov, size=n).T
sim = np.corrcoef(z1 + z2, y)[0, 1]
assert abs(sim - (rho1 + rho2) / np.sqrt(2 * (1 + r12))) < 0.005
print(round(sim, 4), round(0.05 / np.sqrt(3.0), 4))
```

26-C01. Assume a common tax rate, deductible interest, and a tax shield usable in the current period; hold asset/debt and operating-return/interest-rate conventions fixed. Losses and deferred tax shields require separate modeling. Pretax operating profit is EBIT $= r_a A$, interest is $r_d D$, net income is $N = (1-\tau)(r_a A - r_d D)$, and equity is $E = A - D$. Then

$$\text{ROE} = N/E = (1-\tau)(r_a A - r_d D)/E = (1-\tau)\big[r_a(1+D/E) - r_d D/E\big] = (1-\tau)\big[r_a + (r_a - r_d)D/E\big],$$

using $A/E = 1 + D/E$. The mapping to Section 6.1 is $\rho = (1-\tau)r_a$ (after-tax operating return) and $i = (1-\tau)r_d$ (after-tax debt cost), so $(1-\tau)(r_a - r_d) = \rho - i$ and the formula returns to $\rho + (\rho - i)D/E$. If $\rho < i$, the leverage term is negative and leverage lowers ROE. ROE's sensitivity to r_a is $(1-\tau)(1+D/E)$, so more leverage amplifies variation in r_a and ROE volatility rises with D/E. Grading follows the original three criteria.

26-C02 (grading guidance and reference). In the course's current data, French RMW is an aggregate spread; constructed companies in `valuation.py` have very few fields; and there are no versioned stock-level revenue, cost of goods sold, SG&A, interest, or operating-cash-flow data, nor prices, historical universes, or industry history. The SEC Financial Statement Data Sets can supply as-filed values and `filed` and `accepted` timestamps from 2009 onward, but XBRL tag mapping for items such as cost of goods sold, industry classifications, prices, and delisting returns must be sourced separately (and the data's access restrictions followed; see Chapter 24). Claims that cannot be made: returns to a stock-level quality factor, its correlation with value, or its after-cost performance. Three checks before combination: same-date rank correlation of quality and value metrics (which determines composite gain), coverage and exclusions for each metric (which determine sample selection), and each field's `available_at` and lag rule (which determine look-ahead). Grading follows the original four criteria.

26-N01 complete solution. Average assets are 900 and average equity is 350. GP/A is 44.4444%; ROE is 25.7143%; the raw accrual ratio is -3.3333%, so its negative is +3.3333%; debt to ending assets is 30%. French OP is also 180/400 = 45%; 51.4286% using average equity is a course variant. GP/A, ROE, accruals, and OP use fixed fiscal-year flow/balance conventions; debt to ending assets is an ending stock-to-stock ratio; none is a monthly investment return. The report had not yet been released or ingested in February; it is available only at March month-end. Only company A passes the four-company threshold example; B is below 5% of average assets, C has nonpositive ending equity, and D equals the threshold. Coverage is 25%, with 75% lost.

GP/A is a company accounting characteristic; RMW is an aggregate spread of size/OP portfolios. They describe different objects and use different denominators, so neither can replace or validate the other.

19. Further reading

- Must Know: Fama–French (2015), note to Table 8 and Section 7; read the OP definition and the unconditional relation between value and profitability.
- Must Know: Ken French database five-factor description page; read how RMW is formed and its sample requirements.
- Should Know: Novy-Marx, *The Other Side of Value*, June 2012 author manuscript; read the introduction and data description to understand the asset denominator and exclusion of financial firms. The journal version was not read.
- Should Know: Chapter 12 of this course (financial statements), definitions of net income, cash flow, and accruals.
- Advanced: Original papers by Sloan (1996) on accruals and Piotroski (2000) on fundamental scoring; check empirical findings on accruals and stability measures. Not read for this chapter.
- Optional: "Selected Financial Data" in any public 10-K; practice finding each field's reporting period and comparative balances.

20. References

Citation keys and scope of reading are listed in `source_notes.md`.

- CH26_FF5_PAGE: Kenneth R. French Data Library, "Description of Fama/French 5 Factors (2x3)."
- CH26_FF2015: Fama, E. F., and French, K. R. (2015). *Journal of Financial Economics* 116(1), note to Table 8 (printed p. 14), Section 7 (pp. 12–13).
- CH26_NOVYMARX2012: Novy-Marx, R. "The Other Side of Value: The Gross Profitability Premium," June 2012 author manuscript; journal version, JFE 108(1), not read.
- CH24_SEC_FSDS: SEC Financial Statement Data Sets description (see Chapter 24 source notes).
- FRENCH: Frozen French five-factor snapshot (`data/manifests/french_factors_2026-10-05.json`).

Chapter 27 — Carry, Defensive Investing, and Low Volatility

Through line: Carry is "how much a position would earn if market conditions stayed unchanged." It can be calculated mechanically in advance. Low volatility and low beta raise different research questions: Is the security market line flatter than CAPM predicts? What return difference remains after scaling by beta, and how do financing costs and constraints change the comparison? Carry starts from a cash-flow definition; the low-risk question requires empirical evidence. Both require attention to leverage and losses under stress, as well as average returns. Prerequisites: Chapter 10 (cash flows in financial markets: futures, FX, and bonds), Chapter 18 (volatility), and Chapter 23 (cross-sectional research). After this chapter, you should be able to derive carry for futures, FX forwards, zero-coupon bonds, and commodity curves from holding-period cash flows; distinguish the return earned under "unchanged prices" from price changes; explain why spliced continuous futures prices are not tradable PnL; write down the weights, leverage, and financing costs of a beta-sorted, leveraged low-beta portfolio (betting against beta, BAB); explain how shrinkage determines whether BAB is actually market neutral; report low-beta and BAB results for 10 industry portfolios; and report negative results, costs, and crashes alongside positive results.

1. Chapter goals

Readers should be able to answer: Does a high-interest-rate currency "earn the interest differential," or does it earn a return when the exchange rate stays unchanged? Where does the roll return on a commodity futures contract come from? Why is a high-minus-low portfolio's average carry nonnegative when constructed on a common capital basis, yet its realized return can be negative? Why does a low-beta portfolio need leverage to be compared with the market, and how do financing costs and constraints change the conclusion?

1. Write the futures carry definition $C_t = (S_t - F_t)/X_t$ (S_t is the spot price, F_t the futures price, and X_t the collateral capital), and derive carry for FX forwards, zero-coupon bonds, and commodity futures curves, stating each formula's quote direction, frequency, and assumptions.
2. Decompose a position's return as $r_{t+1} = C_t + \Delta S_{t+1}/X_t$ (use F_t as the denominator only when the contract is fully collateralized, $X_t = F_t$). Explain where the "unchanged price" assumption is counterfactual and how the part that is "taken back" (take-back) through price changes determines carry's predictive power.
3. Explain the difference between a continuous futures price (made by stitching together prices of different expiries) and the tradable return on a single contract; calculate their difference from the roll spread; and verify in a constructed world that tradable = carry + price change.
4. Distinguish low volatility, low beta, and defensive investing; use $\sigma_i^2 = \beta_i^2\sigma_m^2 + \sigma_{\varepsilon,i}^2$ to show why low beta does not imply low volatility.
5. State BAB's weights, leverage, and return, and prove market neutrality under estimated betas. Distinguish the noiseless and noisy models: under the ranking, positive-denominator,

and conditional-expectation assumptions in Section 6.9, derive the sign of true exposure; in the noise model, optimal shrinkage gives conditional neutrality, while the unconditional expectation also requires integrability.

6. Report the average excess returns and alphas of the low- and high-beta halves, BAB returns, sample market-regression beta estimates, subperiods, cost sensitivities, and worst months using the frozen French monthly data for 10 industries. State that these are 10 value-weighted industry portfolios, not individual stocks or a trade-by-trade execution series.
7. In a constructed world with a known data-generating process, reproduce a carry portfolio with positive average carry but negatively skewed returns and deep drawdowns, with a large take-back component. Use 200 new seeds to explain what a single path can and cannot establish.

Completion criterion: Every carry or low-risk signal must have a field table stating quote direction, tenor, `signal_time`, financing and collateral assumptions, roll rule, and conclusions that cannot be drawn without actual forward quotes, futures curves, and financing rates.

2. Why carry and low risk need careful comparison

Value, momentum, and quality ask "what to buy." This chapter asks where a return comes from and how much market exposure is involved in comparing it. An FX forward typically requires margin rather than payment of the full contract principal. Its premium or discount reflects the interest differential, but exchange-rate changes can offset carry. Carry is a counterfactual calculation under a stated quote and capital convention; it does not guarantee an average realized return.

The low-risk question starts with CAPM's benchmark, $E[r_i] = \beta_i E[r_m]$: if expected excess return is proportional to beta, expected return per unit of beta is the same. Whether the security market line is flatter in the data is an empirical question. Only under the relevant relationship might low-beta portfolios compare favorably after beta scaling. Scale the groups to the same estimated market exposure, then report returns, risk, and estimation error. Leverage also requires financing; spreads, margin requirements, and forced liquidation may change the result. Financing constraints are part of the research question, rather than a reason to assume that low-risk portfolios must earn more.

Three common misinterpretations explain why the definitions need to be precise. First, treating "high interest rates" as "high carry": reversing the quote direction reverses the sign. Second, treating a continuous futures price as trading PnL: a stitched price jumps on the roll date, and nobody can earn that jump. Third, treating "leveraged low beta" as "market neutral": if the beta used for scaling is shrunk toward 1, true exposure need not be zero under the specified model (Section 6.9); the historical market-regression beta estimate is about -0.2 in Section 11, which does not make it a known true beta. None of these errors raises an exception.

The chapter's logic is as follows: derive carry from holding-period cash flows (6.1), then write the specific formulas for FX (6.2), bonds (6.3), and commodities (6.4); decompose returns into carry and price changes (6.5), handle rolls and splicing (6.6), and portfolio carry (6.7); then turn to low risk: the beta-volatility decomposition (6.8), BAB construction and neutrality (6.9), and

leverage, financing, and crashes (6.10); finally, report results from a constructed world (Section 10) and 10 industry portfolios (Section 11).

3. Intuition

Suppose you borrow dollars and buy a yen deposit. The yen deposit has a lower interest rate, while the dollar rate is higher, so you pay the interest differential each year. Reverse the trade—borrow yen and deposit dollars—and you earn the interest differential. This is a carry trade. It makes money when the exchange rate does not move far enough against you. If the high-interest-rate currency depreciates by exactly the interest differential (uncovered interest parity, UIP, holds), the average return is zero. The empirical "carry premium" means UIP fails on average; it does not mean every year is profitable.

The same intuition applies to commodity futures. Suppose the nearby contract is 100 and the deferred contract is 98 (backwardation). You buy the deferred contract. If the curve does not move, it gradually "slides" toward the nearby price as expiry approaches, earning you 2/98. Conversely, if the nearby contract is 100 and the deferred contract is 101 (contango), buying the deferred contract loses 1/101 per month if the curve stays unchanged. This return comes from the curve's shape, not from a price rise or fall. A continuous contract joins nearby and deferred prices end to end, so its price appears to "jump" on the roll date. Nobody earns that jump.

The intuition for low volatility and low beta is this: if risk premia are proportional to beta, then expected return per unit of beta is the same. If the security market line is flatter than the theory predicts, low-beta assets have higher returns per unit of beta and high-beta assets have lower returns. Leveraging the low-beta assets to beta 1 produces an expected return above the market, at the cost of leverage. A study that omits leverage constraints, financing costs, and risk presents only a paper Sharpe ratio.

Scope boundary: the real-data example in this chapter uses only 10 industry portfolios (revised value-weighted monthly returns). It has no actual FX forwards, futures curves, or financing rates. Carry's numerical examples are hand calculations and a constructed world.

3.1 Start with a cash-flow ledger, then name the return

Why can the same forward-price spread produce different reported returns? Because a contract PnL is a currency amount, while a return also needs an invested-capital denominator. **Collateral capital** X is the denominator; **contract notional** is the price multiplied by the number of contract units. They are different quantities. The table below fixes one contract with a unit multiplier. Futures and forwards have different settlement and margin mechanisms; here we use the chapter's simplified one-period ledger, with the collateral account earning the risk-free rate, and do not model daily margin calls.

Time	Known at this point	Account action	Still unknown
Signal/entry t	Spot S_t, forward F_t to expiry, capital X_t, and the one-period interest rate	Deposit X_t in the collateral account and enter a long position in one unit of the underlying	Future spot, margin changes, and trading impact
Counterfactual calculation	Only the current quotes above	Assume expiry spot remains at S_t and calculate $(S_t - F_t)/X_t$	This is carry, not a future observation
Expiry $t+1$	Only now is S_{t+1} known	Return $X_t(1+r^f)$ from the collateral account; contract PnL is $S_{t+1} - F_t$	Record the two cash flows separately, then divide by X_t

Complete worked example: Capital changes returns

Let $S_t = 102, F_t = 100, X_t = 50, r^f = 1\%$ (a one-period decimal rate). The counterfactual contract PnL is 2, so carry is $2/50 = 4\%$. If the actual expiry spot is 99, the contract loses 1 and the collateral account earns 0.5; ending equity is $50.5 - 1 = 49.5$. Total return is -1% and excess return is -2%. The decomposition is $4\% + (99 - 102)/50 = -2\%$. If the same contract instead has $X = 100$, carry falls to 2% and excess return to -1%; collateral interest of 1 offsets the contract loss, leaving a total return of 0. More capital changes the return and leverage, not the PnL on that one contract.

The FX example in Section 8 uses $X = F$, so $C = S/F - 1$; this does not mean a "zero-cost contract has infinite returns." Section 6.3 first calculates the cash bond's **total holding-period return**, then subtracts the financing benchmark over the same period. When reading carry for different assets, first ask: "What are the cash flows, what is the denominator, and what has been subtracted?" Actual financing spreads, margin calls, and borrowing constraints require additional inputs; they cannot be hidden inside the risk-free collateral assumption.

4. Mathematical foundations

Notation. Use one period (one month). S_t is the spot price at time t, in units of "local currency per unit of the underlying" (for FX, local currency per unit of foreign currency); F_t is the futures (forward) price for delivery exactly at $t+1$, in the same units; X_t is the capital (collateral) allocated to one contract, at least equal to margin; and r^f is the one-period risk-free rate (a decimal), which the collateral capital earns. All prices are positive. Frequency: monthly; `signal_time` is month-end t, when S_t, F_t are known; the return period runs from t to $t+1$.

Futures return. Allocate X_t and hold the contract for one period. Its end-of-period value is $X_t(1+r^f) + F_{t+1} - F_t$, so total and excess returns per unit of capital are

$$r^{\text{tot}}_{t+1} = \frac{F_{t+1} - F_t}{X_t} + r^f, \qquad r_{t+1} = \frac{F_{t+1} - F_t}{X_t}.$$

Convergence at expiry. At expiry, the contract price equals spot: $F_{t+1} = S_{t+1}$.

Carry. Hold the spot price fixed at $S_{t+1} = S_t$, so $F_{t+1} = S_t$:

$$C_t = \frac{S_t - F_t}{X_t}, \qquad X_t = F_t \text{ when } C_t = \frac{S_t}{F_t} - 1.$$

Return decomposition. Add and subtract S_t:

$$r_{t+1} = \frac{S_{t+1} - F_t}{X_t} = \underbrace{\frac{S_t - F_t}{X_t}}_{C_t} + \underbrace{\frac{S_{t+1} - S_t}{X_t}}_{\text{price change}}$$

(the same denominator applies when $X_t = F_t$). These are Koijen, Moskowitz, Pedersen, and Vrugt's definitions (NBER Working Paper w19325, August 2013 version, equations (2)–(6)).

Beta and volatility. Regress an asset's excess return r_i on the market's excess return r_m: $r_i = \alpha_i + \beta_i r_m + \varepsilon_i$, with $\beta_i = \mathrm{Cov}(r_i, r_m)/\mathrm{Var}(r_m) = \rho_{im}\sigma_i/\sigma_m$. Assume finite second moments and strictly positive market variance, and let ε_i be the population linear-projection residual, orthogonal to the market. Then $\sigma_i^2 = \beta_i^2\sigma_m^2 + \sigma_{\varepsilon,i}^2$. An arbitrary error term does not automatically satisfy this decomposition.

BAB weights. At a month-end, let $\hat{\beta}$ be the estimated beta vector for n securities, $z_i = \mathrm{rank}(\hat{\beta}_i)$ (use average ranks for ties), $\bar{z} = \frac{1}{n}\sum z_i$, and $k = 2/\sum_i |z_i - \bar{z}|$. Then

$$w_H = k\,(z - \bar{z})^+, \qquad w_L = k\,(z - \bar{z})^-, \qquad \mathbf{1}^\top w_H = \mathbf{1}^\top w_L = 1,$$

where $x^+ = \max(x, 0)$ and $x^- = \max(-x, 0)$. The low-beta leg w_L puts its largest weights on the lowest-beta securities. (Frazzini and Pedersen's draft, 2013-05-10, equation (16).)

BAB return. Let $\hat{\beta}_L = w_L^\top\hat{\beta}$, $\hat{\beta}_H = w_H^\top\hat{\beta}$, and next-period excess returns be r^e, with $r_L = w_L^\top r^e$ and $r_H = w_H^\top r^e$:

$$r^{\mathrm{BAB}} = \frac{1}{\hat{\beta}_L}\,r_L - \frac{1}{\hat{\beta}_H}\,r_H.$$

(Equation (17); the draft writes this as $r - r^f$, which is equivalent to excess returns.) The estimated beta of each leg must be strictly positive. If all betas are tied, the rank weights are undefined.

Frequency convention: monthly; estimate beta using data through month-end s; BAB return is for month $s + 1$.

5. Formal definitions

Carry. The return on a position under "unchanged market conditions"; for futures, $C_t = (S_t - F_t)/X_t$. It is one component of expected return that can be calculated in advance; it is not expected return itself.

Currency carry trade. A portfolio that goes long high-carry currencies and short low-carry currencies. For forwards, carry is the interest-rate differential $(r^* - r)/(1 + r)$.

Forward premium. $F/S - 1$: the forward price relative to spot; a positive value means the forward is at a premium.

Covered interest parity (CIP). With simple interest rates, $F = S(1 + r)/(1 + r^*)$, where S is quoted as "local currency/foreign currency," r is the local interest rate, and r^* is the foreign interest rate.

Term structure and curve slope. A curve of prices or yields for the same underlying at different expiries. In this chapter's constructed world, the commodity curve slope $g = (S - F)/S$ measures the shape of the curve.

Backwardation and contango. When the nearby contract is more expensive than the deferred contract (a downward-sloping curve, with positive carry on a deferred long), the market is in backwardation. When the deferred contract is more expensive than the nearby contract, it is in contango.

Convenience yield. The net return δ from holding a physical commodity rather than a futures contract, after storage costs.

Roll yield. In this chapter, the return earned by holding a contract as it approaches expiry while the curve stays unchanged; it is the manifestation of carry along the futures curve. It comes from the shape of the price curve, not from a change in spot.

Roll-down. The price change that results when a bond's maturity shortens and it moves along an unchanged yield curve toward a lower yield.

Continuous contract. A price series formed by joining prices of different expiries end to end. It includes the spread between two contracts on the roll date; that spread is not a return.

Portfolio carry. $C^{\mathrm{p}} = \sum_i w_i C_i$ (KMPV equation (24)).

Low volatility. Buying securities ranked by (ex ante) volatility. **Low beta.** Buying securities ranked by (ex ante) beta. They are different: $\sigma^2 = \beta^2 \sigma_m^2 + \sigma_\varepsilon^2$, so a low-beta security may have high idiosyncratic volatility.

Defensive. The operational definition in this chapter: an asset or portfolio that loses materially less than the market in down-market months, typically (but not necessarily) low-beta, low-volatility, or high-quality. Measure it with the conditional mean and conditional beta in down months; it is not a standard classification.

Betting against beta (BAB). A self-financing portfolio that goes long the low-beta leg levered to beta 1 and short the high-beta leg delevered to beta 1.

Funding constraints and funding risk. Limits on borrowing, margin requirements, and financing costs. When constraints tighten, forced deleveraging causes leveraged positions to lose money at the same time. Frazzini and Pedersen's model predicts that high-beta assets are overpriced, BAB earns positive returns, and BAB returns are low when funding tightens (draft abstract); this chapter does not test the model.

Crash. In this chapter, the left tail of a portfolio's return distribution: large losses clustered in a small number of months because of common shocks rather than diversified idiosyncratic shocks.

6. Derivations

6.1 From holding-period cash flows to carry

A futures contract does not require the full principal upfront. Allocate capital X_t to one contract (collateral above the margin requirement, for example, invested in a short-term Treasury), and at expiry receive $X_t(1 + r^f)$ plus the contract's PnL, $F_{t+1} - F_t$. Excess return per unit of capital is $(F_{t+1} - F_t)/X_t$; total return is $r^f + (F_{t+1} - F_t)/X_t$ (Section 4). Convergence at expiry, $F_{t+1} = S_{t+1}$, links the contract return to spot: $r_{t+1} = (S_{t+1} - F_t)/X_t$. Set $S_{t+1} = S_t$ to obtain $C_t = (S_t - F_t)/X_t$. Read it this way: carry is positive when the futures price is below spot ($F < S$), and negative when it is above spot. $X = F$ is called full collateralization; then $C = S/F - 1$.

This is a one-period case in which the contract expires exactly at period end. For a longer-dated contract, replace "price unchanged" with "curve unchanged" (Sections 6.3 and 6.4).

6.2 FX forwards: interest differentials and quote direction

Let S be quoted as "local currency/foreign currency," and let r and r^* be the local and foreign simple interest rates for the same period. Covered interest parity (CIP) gives $F = S(1 + r)/(1 + r^*)$. The carry on a long foreign-currency forward (buying one unit of foreign currency at F for delivery at expiry) is

$$C = \frac{S - F}{F} = \frac{S}{F} - 1 = \frac{1 + r^*}{1 + r} - 1 = \frac{r^* - r}{1 + r}.$$

When the foreign rate is higher ($r^* > r$), $F < S$ and $C > 0$. The forward is a zero-cost contract; its return is an excess return, and carry is the interest differential normalized by $X = F$. KMPV equation (7) and footnote 6 do use the reciprocal of the local rate for their normalization. That normalization should not be interpreted directly as a same-notional replication that delivers one fixed unit of foreign currency at maturity: today, that replication requires depositing $1/(1 + r^*)$ units of foreign currency, at a cost of $S/(1 + r^*) = F/(1 + r)$ units of local currency. Keep the replication amount separate from the denominator used to report the return.

Deposit comparison with an explicit denominator. Convert one unit of local currency into foreign currency and deposit it for one period. Its local-currency value at maturity is $\frac{S_T}{S}(1 + r^*)$; a local-currency deposit grows to $1 + r$. Since $(1 + r^*)/S = (1 + r)/F$, excess return $\left[\frac{S_T}{S}(1 + r^*) - (1 + r)\right]/(1 + r) = \frac{S_T - F}{F}$. Algebraically, this equals the excess return on a forward with $X = F$ (Section 8.1), because the denominator is the local deposit's ending value $1 + r$, not its initial principal of 1. The foreign-currency deposit's total return is $S_T(1+r^*)/S-1$; its arithmetic excess return is that total return minus r. Neither should be conflated with the forward's total return. Separate two scenarios. If the exchange rate stays unchanged, $S_T = S$, the foreign deposit's

arithmetic excess return on initial local capital of 1 is $r^* - r$. Dividing by the local deposit's ending value $1 + r$ gives relative excess return $(r^* - r)/(1 + r)$, equal to forward carry with $X = F$. If instead $S_T = F$, the foreign currency depreciates by exactly $1 - (1 + r)/(1 + r^*)$, offsetting its interest advantage, and excess return relative to the local deposit is zero. Here "break-even" refers to the local-deposit benchmark, not to initial principal. These are different scenarios.

Quote direction. With the reciprocal quote, $S' = 1/S$ and $F' = 1/F$, applying the same formula $(S' - F')/F'$ gives $F/S - 1$ (the sign is reversed). Therefore, fix the convention: S, F in the formula are quoted as "local currency/foreign currency," and carry is the long-foreign-currency carry (the counterexample in Section 8.1).

UIP and carry trades. Uncovered interest parity says $E[S_T] = F$: the expected depreciation of the foreign currency exactly offsets the interest differential, leaving the long-forward average return at zero. A profitable carry trade requires $E[S_T] > F$, meaning UIP fails on average. This is an empirical question. This chapter has no actual FX data; it only includes a specified "take-back" fraction in a constructed world (Sections 6.5 and 10).

6.3 Zero-coupon bonds: yield, roll-down, and an unchanged curve

Let $y(\tau)$ be the annual-compounded zero-coupon yield curve at maturity τ years. The price of a zero-coupon bond with face value 1 and maturity T is $P_T = (1 + y_T)^{-T}$, where $y_T = y(T)$. Hold it for h years ($h < T$). If the curve is unchanged, it becomes a bond with $T - h$ years to maturity and price $(1 + y_{T-h})^{-(T-h)}$. Total return is

$$1 + \text{carry} = \frac{(1 + y_T)^T}{(1 + y_{T-h})^{T-h}} = \underbrace{(1 + y_T)^h}_{\text{1+yield component}} \times \underbrace{\left(\frac{1 + y_T}{1 + y_{T-h}}\right)^{T-h}}_{\text{1+roll-down component}},$$

using $(1 + y_T)^T = (1 + y_T)^h (1 + y_T)^{T-h}$. On a flat curve, $y_{T-h} = y_T$ and the roll-down factor is 1. On an upward-sloping curve ($y_{T-h} < y_T$), roll-down is positive: the bond moves toward the short end, its yield falls, and its price rises. Excess return equals total return minus the short-term financing rate over h years, $[(1 + y_h)^h - 1]$. KMPV (equations (13)–(15)) shows the same structure for coupon bonds: carry consists of yield and roll-down; the yield component less the short rate is the term spread. A modified-duration approximation is

$$\text{roll-down} \approx -D_{\text{mod}} (y_{T-h} - y_T),$$

where D_{mod} is for the bond after roll-down (the $(T - h)$-year zero-coupon bond, $(T - h)/(1 + y)$). In the numerical example in Section 8.2, using the duration of the original T-year bond overstates roll-down.

Assumptions: zero-coupon bonds, annual compounding, linear interpolation of yields by maturity, no credit or liquidity premium, and an "unchanged curve" counterfactual (the actual curve changes every day).

6.4 Commodities: convenience yield and curve slope

A cash-flow view of holding physical commodities: buy spot S and hold for one period. The return is the net convenience yield δ minus financing cost r (borrowing to buy spot). No arbitrage gives $F = S(1 + r - \delta)$ (following KMPV equation (9)). If spot is unchanged, carry on a long futures position is

$$C = \frac{S - F}{F} = \frac{\delta - r}{1 + r - \delta}.$$

Spot prices for commodities are often not cleanly observable, so use the spread between two adjacent futures contracts: nearby F^1 (with T_1 periods remaining) and the next-nearby F^2 (with $T_2 > T_1$). If the nearby price stays unchanged, the F^2 contract slides toward F^1 over $T_2 - T_1$ periods. A linear per-period proxy for carry is

$$C = \frac{F^1 - F^2}{F^2\,(T_2 - T_1)}$$

(KMPV equation (10)). It is the price spread over the maturity difference, divided by capital; it is not an exact monthly compounded return for an arbitrary maturity. If the total endpoint price ratio is compounded evenly by period, calculate $(F^1/F^2)^{1/(T_2-T_1)} - 1$ instead, with the unchanged-curve path stated explicitly. If $F^i = S(1 + (r - \delta)T_i)$ is linear, then $F^1 - F^2 = S(r - \delta)(T_1 - T_2)$ and $C = (\delta - r)S/F^2$, consistent with the formula above. Read it this way: in backwardation ($F^1 > F^2$), $C > 0$; in contango, $C < 0$. Section 8.3 uses nearby 100 and next-nearby 101, one month apart, so $C = -1/101 = -0.99\%$ per month.

6.5 Return decomposition and "take-back"

Take the conditional expectation of the identity from Section 4, $r_{t+1} = C_t + \Delta S_{t+1}/F_t$ (when $X = F$):

$$E_t[r_{t+1}] = C_t + E_t\Big[\frac{\Delta S_{t+1}}{F_t}\Big].$$

Carry is one component of expected return; the other is the expected price change, which depends on the risk premium and expectations (KMPV equation (6)). Suppose the expected price change is a fraction of carry: $E_t[\Delta S/F] = -\gamma C_t$. Then $E_t[r] = (1 - \gamma)C_t$. If $\gamma = 1$, the price change exactly offsets carry (the UIP case in FX), leaving zero average return. If $\gamma = 0$, price is a martingale and all carry is realized. If $0 < \gamma < 1$, some carry is taken back. If the cross-sectional regression is $r_{i,t+1} = a + bC_{i,t} + u$, with slope $b = 1 - \gamma$, then $b = 1$ means carry is fully realized and $b = 0$ means price changes fully offset it.

The constructed world in Section 10 sets $\gamma = 0.5$, but shock loadings $\ell_i = 2\,\mathrm{pct}_i - 1$ depend on carry rank, so the unconditional expected monthly cross-sectional slope is below $1-\gamma$: $E[b] \approx (1-\gamma) - p\,E[m]\,\overline{\mathrm{Cov}(\ell, C)/\mathrm{Var}(C)}$, where $p = 1.2\%$ is the monthly shock probability and $E[m] = 0.05 + 0.02\sqrt{2/\pi} = 0.066$ is the expected shock size. The last term is the average across months of $\mathrm{Cov}(\ell, C)/\mathrm{Var}(C)$. The theoretical value for this path is 0.23; the median of monthly slope means over 200 new seeds is 0.273 (the median theoretical value is 0.272; the 5th and 95th percentiles

are −0.14 and 0.56). The measured value on this path is 0.44: it has only 3 shock months (4.3 expected), the average over the 356 non-shock months is 0.65, and the average over shock months is −24.0. Thus, the slope below $1-\gamma$ comes from the specified crash loading, not merely sampling error.

6.6 Continuous-contract splicing and tradable PnL

Let $c(t)$ be the contract held on date t, with price $F_{t,c}$. Tradable return is the price ratio for the same contract on two dates:

$$r_t^{\text{trade}} = \frac{F_{t+1,c(t)}}{F_{t,c(t)}} - 1.$$

The return on the spliced price $S_t^* = F_{t,c(t)}$ is

$$r_t^* = \frac{S_{t+1}^*}{S_t^*} - 1 = \frac{F_{t+1,c(t+1)}}{F_{t,c(t)}} - 1.$$

Their difference is

$$r_t^* - r_t^{\text{trade}} = \frac{F_{t+1,c(t+1)} - F_{t+1,c(t)}}{F_{t,c(t)}}.$$

It is zero when there is no roll ($c(t+1) = c(t)$); when the contract changes, it is the new-minus-old contract price at $t+1$, divided by the old price. In contango, the new contract costs more, so the spliced price jumps upward on the roll date. Nobody earns this amount: at $t+1$, traders sell the old contract at its quote and buy the new contract at its quote. The roll itself creates no PnL. The cost of contango appears as the new contract later converges downward toward spot. Over time, the spliced series is close to spot, while the tradable series is spot plus roll yield (Section 8.4 and the constructed world in Section 10). A spliced price can also become negative or make price ratios invalid, so research should compound single-contract returns rather than calculate returns from a spliced price (the commodity-futures return construction of Asness, Moskowitz, and Pedersen 2013; see Chapter 25's source notes).

6.7 A high-minus-low portfolio's carry is nonnegative, but its return need not be

For portfolio weights w_i (long weights sum to +1, short weights to −1, or rank weights), portfolio carry is $C^{\text{p}} = \sum_i w_i C_i = \overline{C}_{\text{long}} - \overline{C}_{\text{short}}$ (KMPV equations (24) and (25)). Going long high carry and short low carry makes $C^{\text{p}} \geq 0$ by construction. The return decomposition in Section 6.5 gives realized return = C^{p} + price change; the average price-change component can offset much of carry.

The nonnegative result requires nonnegative weights within each long and short leg, comparable capital bases, and each long position's carry to be no lower than each short position's carry. It can be zero when all values are equal or when there is a boundary tie. Carry observed at formation can still vary over time. Let D_t be the portfolio's price-change component; then

$\mathrm{Var}(r) = \mathrm{Var}(C^{\mathrm{p}}) + \mathrm{Var}(D) + 2\mathrm{Cov}(C^{\mathrm{p}}, D)$, and skewness also depends on the joint distribution. Conditional on the formation-time information set, known carry does not contribute to the current conditional innovation; this differs from unconditional time-series risk. Thus, "portfolio carry is positive" and "return is positive" are two different statements. In the constructed world in Section 10, average portfolio carry is 0.516% per month, while average return is 0.179% per month.

6.8 Low volatility and low beta: decomposition

$\sigma_i^2 = \beta_i^2\sigma_m^2 + \sigma_{\varepsilon,i}^2$. Let $\sigma_m = 16\%$. Asset X has $\beta = 0.5$ and idiosyncratic volatility 25%; asset Y has $\beta = 1.2$ and idiosyncratic volatility 5%. Then

$$\sigma_X = \sqrt{(0.5 \times 0.16)^2 + 0.25^2} = 26.2\%, \qquad \sigma_Y = \sqrt{(1.2 \times 0.16)^2 + 0.05^2} = 19.8\%.$$

X has lower beta but higher volatility, so ranking by volatility and by beta gives opposite orders (27-E08). "Low volatility" and "low beta" are different signals. They are often correlated empirically, but they are not identical (Section 11: among the 10 industries, the low-beta and low-volatility halves overlap by an average of 3.6 industries).

CAPM says $E[r_i] = \beta_i E[r_m]$ for excess returns. If the security market line is flatter than this, $\alpha_i = E[r_i] - \beta_i E[r_m]$ is positive for low-beta assets and negative for high-beta assets. Suppose a low-beta portfolio has $\beta_p = 0.7$ and is held unlevered. To compare it with the market, lever it by $1/0.7 = 1.43$ to reach beta 1. If expected excess return is independent of beta (a flat SML), the levered portfolio's expected excess return is $1.43E[r_p]$, above the market's. CAPM has no cost of leverage; in practice, leverage has costs and constraints (Section 6.10).

6.9 BAB: weights, neutrality, and shrinkage

Construction. Use w_L, w_H from Section 4, with estimated leg betas $\hat{\beta}_L, \hat{\beta}_H$, and return $r^{\mathrm{BAB}} = r_L/\hat{\beta}_L - r_H/\hat{\beta}_H$. Each leg is scaled to beta 1 (both leg betas must be positive; only when $0 < \hat{\beta}_L < 1 < \hat{\beta}_H$ do the positions separately satisfy $1/\hat{\beta}_L > 1$ and $1/\hat{\beta}_H < 1$; high-low ranking alone does not guarantee that the betas straddle 1). The draft reports that U.S. equity BAB is on average long 1.4 dollars of low-beta stocks, financed by borrowing 1.4 dollars in the risk-free asset, and short 0.7 dollars of high-beta stocks (that draft's sample, page 19).

Neutrality under estimated beta. Apply the portfolio weights directly to the estimates used for scaling: $\hat{\beta}^{\mathrm{BAB}} = \hat{\beta}_L/\hat{\beta}_L - \hat{\beta}_H/\hat{\beta}_H = 1 - 1 = 0$. This algebra needs nonzero estimated leg betas, not equality between estimates and true values; the chapter's construction additionally requires positive leg betas. Equality with true betas is needed only to extend this result to true market neutrality. In Section 8.5's special case, each asset's return is exactly the scaling β_i times the market return, so BAB return is zero.

True exposure. Actual portfolio beta is $\beta^{\text{BAB}} = \beta_L^T/\hat{\beta}_L - \beta_H^T/\hat{\beta}_H$, where β^T denotes the true beta of each leg.

Shrinkage. Frazzini and Pedersen use $\hat{\beta} = w\hat{\beta}^{\text{TS}} + (1-w)\cdot 1$ with $w = 0.6$ (equation (15) and following; draft page 17), shrinking toward the cross-sectional mean of 1. A common strictly positive w does not change the ranking; $w = 0$ makes all estimates equal to 1, eliminating rank information, and the existing `bab_weights` function errors on an all-tied vector. A positive shrinkage weight leaves the membership of the two legs unchanged and only changes their scaling (leverage). Consider two cases.

(a) **No noise.** Use the shrunk beta for scaling, while true beta is the unshrunk value x: $\hat{\beta} = wx + (1-w)$. Define $g(x) = x/(wx+1-w)$. Then $\beta^{\text{BAB}} = g(x_L) - g(x_H)$, with $0 < x_L < 1 < x_H$ and $0 < w \leq 1$. State the domain first: g has a pole at $x = -(1-w)/w$, and $g'(x) = (1-w)/(wx+1-w)^2 > 0$ for $w < 1$ only guarantees that g is increasing on each side of the pole; it does not allow a comparison across the pole. The scaling betas for the two legs, $wx_L + 1 - w$ and $wx_H + 1 - w$, are both strictly positive, which means both lie in the same interval to the right of the pole; $x_L > 0$ ensures this automatically. Thus $g(x_L) < g(x_H)$ and $\beta^{\text{BAB}} < 0$. When $w = 1$, $g(x) = 1$ and $\beta^{\text{BAB}} = 0$. Without the positive-denominator condition, the conclusion can reverse: with $x_L = -1$, $x_H = 2$, and $w = 0.6$, the low-beta leg's scaling beta is -0.2, and $g(x_L) - g(x_H) = 5 - 1.25 = +3.75$. With $w = 1$ and $x_L = 0$, the expression is $0/0$. This is the existing policy of `bab_returns`: if either weighted leg beta is ≤ 0 in any month, it errors instead of "levering" a negative-beta leg to 1. For $x_L = 0.7$, $x_H = 1.3$, and $w = 0.6$, the numerical result is $0.7/0.82 - 1.3/1.18 = 0.854 - 1.102 = -0.248$ (27-C01).

(b) **With noise.** Let $\hat{\beta} = \beta + e$, with e independent of β, $\beta \sim (1, \sigma_\beta^2)$, and $e \sim (0, \sigma_e^2)$. Then $E[\beta \mid \hat{\beta}] = 1 + w^*(\hat{\beta} - 1)$, where $w^* = \sigma_\beta^2/(\sigma_\beta^2 + \sigma_e^2)$. This is exact when the pairs (β_i, e_i) are cross-sectionally independent and jointly normal; otherwise it is the best linear prediction. If estimation errors are correlated across securities, selecting the legs by extreme ranks creates additional selection bias.

When $w = w^*$, the shrunk beta is exactly the conditional expectation of true beta. Conditional on the entire vector $\hat{\beta}$, leg weights and scaling denominators s_L, s_H are constants, so the conditional true exposures are exactly $E[\beta_L^T \mid \hat{\beta}]/s_L = 1$ and $E[\beta_H^T \mid \hat{\beta}]/s_H = 1$. BAB's conditional true beta is therefore zero (there is no Jensen adjustment; the denominators must be defined, $s_L, s_H \neq 0$. The unconditional expectation additionally requires the ratio to be integrable, which cannot be assumed under normal tails and is not claimed here).

The sign comparison requires more conditions: the estimated leg betas must straddle 1, $\hat{\beta}_L < 1 < \hat{\beta}_H$, and both scaling denominators s_L, s_H must be strictly positive ($s_H > 0$ follows automatically when $\hat{\beta}_H > 1$). Under these conditions, if $w > w^*$ (too little shrinkage), the low-beta leg's conditional mean true beta exceeds the beta used for scaling and conditional mean BAB exposure is positive. If $w < w^*$ (too much shrinkage), conditional mean exposure is negative. Merely requiring nonzero denominators is insufficient: with $\hat{\beta}_L = -2$, $\hat{\beta}_H = 2$, $w^* = 0.5$, and $w = 0.6$, $s_L = -0.8$ and conditional net exposure is -0.3125, contrary to the rule that too little shrinkage gives positive exposure. The simulation in 27-P03 uses true beta with mean 1 and standard deviation 0.25, noise standard deviation 0.25 ($w^* = 0.5$), 500 securities, and averages over 200 simulations: at $w = 1$, $\beta^{\text{BAB}} = +0.47$; at $w = 0.5$, -0.0006; and at $w = 0.2$, -0.24.

Interpretation. Whether a constant shrinkage weight of 0.6 makes BAB neutral depends on the noise in the beta estimates. In the 10-industry sample in Section 11, the regression beta is about -0.20 for $w = 0.6$ and about $+0.05$ without shrinkage, consistent with the mechanism in which shrinkage changes exposure. But a large portfolio and a 60-month window do not identify the variance of beta estimation noise, nor do they imply that w^* is close to 1. This chapter does not estimate true beta or identify the causal source of negative exposure.

6.10 Leverage, financing, and crashes

Financing. The long leg must borrow $\max(1/\hat{\beta}_L - 1, 0)$ units (0.27 on average in Section 11). If the borrowing rate exceeds the risk-free rate by s per month, the monthly cost is $s\big[\max(1/\hat{\beta}_L - 1, 0) + \max(1/\hat{\beta}_H - 1, 0)\big]$. A financing spread of 1% annualized lowers the mean BAB return for the 10 industries from 0.2045% to 0.1818% per month (Section 11). Financing spreads are only one cost; margin requirements, forced liquidations, and short-sale constraints are not included in this chapter's calculation.

Crash. Suppose a global shock J occurs with low probability, and high-carry currencies' losses are proportional to their carry rank: $r_i = (1-\gamma)C_i + \text{noise} - \ell_i\, m\, J$, with loading $\ell_i = 2\,\text{pct}_i - 1 \in [-1, 1]$ (pct_i is the carry percentile rank). The long leg's ℓ is about $+0.8$ and the short leg's about -0.8, so the portfolio loading on J is about 1.6. All currencies share the same shock, so adding more currencies does not diversify it away. Expected crash loss per month is about $p\,E[m] \times 1.6$. The common J shock creates a concentrated left tail, but noise, changes in carry over time, and their dependence also affect volatility and skewness; it is inaccurate to say all risk comes only from shock months (in Section 10, the 3 shock months average -9.0% and other months average $+0.26\%$).

6.11 What kind of neutrality does BAB mean?

The two BAB legs each have weights summing to 1, then are scaled by $L = 1/\hat{\beta}_L$ and $H = 1/\hat{\beta}_H$. The net weight in risky assets $w = Lw_L - Hw_H$ is $L - H$; gross exposure is $L + H$ (the legs do not overlap). Estimated beta exposure is $L\hat{\beta}_L - H\hat{\beta}_H = 0$, but the **net dollar exposure is generally not zero**. To express the PnL as a self-financing portfolio, add a risk-free-asset weight of $-(L-H)$. Risk and financing weights then sum to zero. This is a self-financing factor expressed per unit of reference capital, not a deposit for which percentage returns can be calculated from zero initial wealth.

In Section 8.5, $L = 1.4815$ and $H = 0.7547$, so risky-asset net exposure is 0.7268 and gross exposure is 2.2362. Using total returns directly adds $(L-H)r^f$; the risk-free financing leg exactly offsets it, so $Lr^e_L - Hr^e_H$ is equivalent to the complete self-financing ledger. The additional borrowing cost in Section 6.10 uses the course's leg-by-leg financing convention, $s[\max(L-1, 0) + \max(H-1, 0)]$: in this example, it charges only the long leg's amount above 1, or 0.4815. This is not a complete broker net-financing or margin ledger, and cannot be interchanged with net financing based on $L - H$ or short-borrow fees. The original sensitivity result is retained under this explicit convention.

For a separate homogeneous comparison with gross exposure equal to 2, scale both legs by $k = 2/(L+H) > 0$. The zero estimated-beta constraint still holds, and returns and fees measured directly on positions under this convention must be recalculated. Net exposure remains $k(L-H)$ and does not automatically become zero. This chapter's existing BAB figures are not scaled this way; the gross-2 constraint in P4 is a separate contract for Chapter 28's stock-industry portfolios and should not be applied to historical BAB results. Conditions on positive denominators, true and estimated beta, and shrinkage remain as described in Section 6.9.

7. Financial interpretation

Carry is a mechanical component of expected return, not the whole return. It can be calculated from today's prices (using rates or forwards for FX, the curve slope for commodities, and the yield curve for bonds) without estimating a risk-premium model. But the identity in Section 6.5 shows that expected price changes matter too: they can take back carry (as with FX UIP) or amplify it. Researchers can observe C_t directly; they need to test γ.

Carry returns are negatively skewed. The crash mechanism in Section 6.10 explains how carry positions earn small, steady returns in ordinary periods but incur concentrated losses during global shocks (funding stress or rising risk aversion). In the constructed world in Section 10, all 200 new seeds have negative skewness; median skewness is -1.4, and 74% of worlds have a maximum drawdown above 25%. Whether this holds in real data requires actual FX, futures, and financing data.

A roll is not a return. On the roll date, a continuous contract jumps by the spread between the new and old contracts (Section 6.6). Calculating returns from this spliced price treats curve shape (roll yield) as spot-price movement. Tradable return equals carry plus price change, while the spliced price approximates spot (in the constructed world in Section 10, spliced mean return is 0.093% per month, spot is 0.090%, and tradable mean return is 0.205%).

Low beta is not low risk. Low beta means only that an asset has less market co-movement; its idiosyncratic risk may be high (Section 6.8). BAB's return has two sources: alpha from low-beta assets and negative alpha from high-beta assets when the SML is flat, plus leverage. Leverage has costs and constraints. Frazzini and Pedersen's model uses constraints to explain why high-beta assets are overpriced (draft abstract); this chapter uses the model's construction but does not test its mechanism.

Defensive does not mean cheap. The low-beta half loses less in down-market months (Section 11: average -2.86% versus -4.55%), but its downside and upside betas are nearly the same (0.81 versus 0.79). This sample does not show a clear asymmetry between up- and down-market beta. The lower beta is consistent with smaller losses, but this diagnostic cannot attribute the entire difference in losses causally to beta.

8. Numerical examples: calculate each step by hand

All figures are generated by `hand_examples()` and independently checked with a different implementation in `qa/a5/claude/recompute/ch27_recompute.py`. The companies, currencies, and contracts are constructed for teaching.

8.1 FX forwards: interest differential, quote direction, and break-even exchange rate

Complete worked example: FX quotation, capital, and the deposit denominator

The quote is "local currency/foreign currency," $S = 100$. The local annual interest rate is $r = 2\%$, and the foreign annual rate is $r^* = 6\%$, both simple rates for one year. CIP gives $F = 100 \times 1.02/1.06 = 96.2264$. Carry on a long foreign-currency forward is

$$C = \frac{S - F}{F} = \frac{100 - 96.2264}{96.2264} = 3.9216\% = \frac{r^* - r}{1 + r} = \frac{0.04}{1.02}.$$

The forward premium is $F/S - 1 = -3.7736\%$ (a forward discount). Break-even spot is $S_T = F = 96.2264$, so the long forward makes money if foreign-currency depreciation is less than 3.7736%. If expiry spot is 96, the forward return is $(96 - 96.2264)/96.2264 = -0.2353\%$. The equivalent deposit comparison is: the foreign-currency deposit converts back to $1.06 \times 0.96 = 1.0176$ local currency, versus 1.02 in the local deposit; excess return is $(1.0176 - 1.02)/1.02 = -0.2353\%$, relative to the local deposit's ending value, and matches the $X = F$ forward's excess return. The foreign-currency deposit's total return on initial principal of 1 is 1.76%, and its arithmetic excess return is $1.76\% - 2\% = -0.24\%$. If the forward is paired with capital F earning 2% collateral interest for the same period, its total return is $2\% - 0.235294\% = 1.76470588\%$. These total returns use different denominators and accounts, so they need not match. The foreign rate is 4 percentage points higher, but the foreign currency depreciates by 4%, so it earns no positive excess return. The deposit still has a positive total return; failure to earn excess return does not mean the account made no profit.

Quote-direction trap: switch to "foreign currency/local currency." Then $S' = 0.01$, $F' = 1/96.2264 = 0.0103922$, and the same formula, $(S' - F')/F'$, gives -3.7736%. The sign reverses, and the value is $F/S - 1$.

8.2 Zero-coupon bond carry and roll-down

Annual-compounded yield curve: 2% at 1 year, 3% at 2 years, 3.5% at 3 years, and 4% at 5 years. Buy a 3-year zero-coupon bond and hold it for one year, assuming the curve stays unchanged. Current price is $1.035^{-3} = 0.90194$; one year later it is a 2-year bond priced at 3%, or $1.03^{-2} = 0.94260$. Total return is $0.94260/0.90194 - 1 = 4.5073\%$. Decomposition: yield component $1.035 - 1 = 3.5\%$; roll-down component $(1.035/1.03)^2 - 1 = 0.9732\%$; and $(1.035)(1.009732) - 1 = 4.5073\%$. Subtract the 2% one-year financing rate to get excess carry of 2.5073%. Duration approximation: modified duration of the 2-year bond after roll-down is $2/1.03 = 1.9417$, so estimated roll-down is $\approx 1.9417 \times (0.035 - 0.03) = 0.9709\%$, 0.002 percentage points below the exact 0.9732%. Using the original 3-year bond's duration, $3/1.035 = 2.899$,

would give 1.449%, an overestimate of about 50%. On a flat curve (all yields at 3.5%), roll-down is zero and total return is 3.5%.

8.3 Commodity curve

The nearby contract (one month to expiry) is 100 and the next-nearby contract (two months to expiry) is 101. Per month, $C = (100-101)/(101\times 1) = -0.990\%$ (contango; a long next-nearby contract loses 0.99% per month if the curve stays unchanged). If the nearby contract is 100 and next-nearby is 98 instead, $C = (100-98)/98 = +2.041\%$ per month (backwardation).

8.4 Rolling: spliced prices and tradable returns

The return index below ignores collateral interest and fees, is fully collateralized at the starting contract price each period, and adjusts notional with wealth (fractional units are allowed). It is not the cash account for a fixed one-contract position. Spot remains at 100 and the deferred curve is higher by one price unit per period. Contract A is priced at 102, 101, and 100 in periods 0, 1, and 2 (converging to spot at expiry in period 2). At period 2, roll into contract B, whose prices at periods 2 and 3 are 103 and 102. Hold A through period 2, then hold B:

Interval	Tradable return (same contract at both endpoints)	Return from spliced prices
Period 0→1	$101/102 - 1 = -0.980\%$	-0.980%
Period 1→2	A: $100/101 - 1 = -0.990\%$	Spliced series 101→103: $+1.980\%$
Period 2→3	B: $102/103 - 1 = -0.971\%$	-0.971%

Cumulative tradable return is -2.913% (spot is unchanged, while contango costs about 1% per month). The spliced series $102 \to 101 \to 103 \to 102$ has a cumulative return of 0%, matching spot. The period 1→2 difference is $1.980 - (-0.990) = 2.970\% = (103-100)/101$, the new-old contract price difference at period 2 divided by the old price (Section 6.6). Nobody earns the + 1.98% in the spliced series.

8.5 BAB: weights, leverage, return, and neutrality

Estimated betas for four securities are $0.6, 0.9, 1.1, 1.4$. Ranks are $1, 2, 3, 4$, $\bar{z} = 2.5$, deviations are $(-1.5, -0.5, 0.5, 1.5)$, $\sum |z - \bar{z}| = 4$, and $k = 2/4 = 0.5$. Low-beta leg weights are $w_L = (0.75, 0.25, 0, 0)$; high-beta leg weights are $w_H = (0, 0, 0.25, 0.75)$. Each leg's weights sum to 1. Their betas are $\hat{\beta}_L = 0.75\times 0.6 + 0.25\times 0.9 = 0.675$ and $\hat{\beta}_H = 0.25\times 1.1 + 0.75\times 1.4 = 1.325$. Leverage is $1/0.675 = 1.4815$ on the low-beta leg and $1/1.325 = 0.7547$ on the high-beta leg. Next month's excess returns are $(1.2\%, 1.5\%, 1.0\%, 2.0\%)$, so $r_L = 1.275\%$ and $r_H = 1.75\%$:

$$r^{\text{BAB}} = \frac{1.275\%}{0.675} - \frac{1.75\%}{1.325} = 1.8889\% - 1.3208\% = 0.5681\%.$$

Neutrality check: if each security's next-month excess return is exactly $\beta_i \times 1\%$ (the market rises 1%), then $r_L = 0.675\%$, $r_H = 1.325\%$, and $r^{\text{BAB}} = 1\% - 1\% = 0$. Financing cost: the

long leg borrows $1.4815 - 1 = 0.4815$ units. If the monthly borrowing spread is 0.2%, cost is $0.2\% \times 0.4815 = 0.0963\%$, leaving BAB return of 0.4718%. The short leg is delevered and does not borrow.

8.6 Portfolio carry and return

Monthly carry for four currencies is $0.4\%, 0.2\%, -0.1\%, -0.3\%$. Go long the first two and short the last two, with weights ± 0.5. Portfolio carry is $0.5(0.4 + 0.2) - 0.5(-0.1 - 0.3) = 0.5\%$, equal to "average long carry of 0.3% minus average short carry of −0.2%" (KMPV equation (25)). Suppose next month's price changes $\Delta S/F$ are $-0.3\%, -0.1\%, 0, +0.2\%$. Currency returns $= C + \Delta S/F$ are $0.1\%, 0.1\%, -0.1\%, -0.1\%$. Portfolio return is $0.5(0.1 + 0.1) - 0.5(-0.1 - 0.1) = 0.2\%$, only 40% of portfolio carry: price changes take back 60%.

8.7 Shrinkage and true exposure

An original beta of 0.5 shrinks to $0.6 \times 0.5 + 0.4 = 0.7$; 1.8 shrinks to 1.48. If the low-beta leg's true beta is 0.7 and the high-beta leg's is 1.3, while scaling uses the shrunk values 0.82 and 1.18, true portfolio beta is $0.7/0.82 - 1.3/1.18 = 0.854 - 1.102 = -0.248$ (Section 6.9(a)).

9. Python implementation

The carry functions are in the new module `carry.py`; beta and BAB are in `low_risk.py`. Ranking reuses Chapter 23's `factor_research.py`. FX forward reference quotes reuse Chapter 10's existing `covered_fx_forward_quote`.

Function	Inputs	Output and boundaries
`futures_carry`	Spot, futures price, optional collateral	$(S - F)/X$; defaults X to F; prices must be positive; arrays are matched by position
`forward_premium`	Spot, forward	$F/S - 1$
`fx_carry_from_rates`	Local and foreign interest rates	$(r^* - r)/(1 + r)$; rates must exceed -1; quote is local/foreign
`commodity_curve_carry`	F^1, F^2, T_1, T_2	$(F^1 - F^2)/(F^2(T_2 - T_1))$; $T_2 > T_1$
`zero_coupon_carry`	Curve, maturity, holding period, optional financing rate	Total return, yield component, roll-down component, excess return; curve must cover T and $T - h$
`roll_down_approximation`	Modified duration, two yields	$-D(y_{\text{after roll-down}} - y_{\text{current}})$
`portfolio_carry`	Weights and carry (same labels)	$\sum w_i C_i$; missing carry for a nonzero weight raises an error
`held_contract_returns`	Contract-price table and contract held each day	$F_{t+1,c}/F_{t,c} - 1$; missing held-contract prices raise an error; last row is NaN
`naive_splice_returns`	Same	Return from spliced prices (includes the spread on the roll date); last row is NaN

Function	Inputs	Output and boundaries
`rolling_beta`	Asset excess returns, market excess return, window, minimum months	Row s uses only months $s - \text{window} + 1$ through s; market data may not be missing
`vol_corr_beta`	Same, with volatility and correlation windows	$\rho\sigma_i/\sigma_m$; defaults to 12 and 60 months, with at least 12 and 36 months
`vasicek_shrink`	Beta, weight, prior	$w\hat{\beta} + (1-w) \cdot \text{prior}$; defaults are 0.6 and 1
`bab_weights`	Beta vector for a month	w_L, w_H (each leg sums to 1; missing-beta weights are 0); ties use average ranks; all-tied vector errors
`bab_returns`	Excess returns, beta, optional borrowing spread	BAB, each leg, leg betas and leverage; errors if either leg beta is nonpositive or a held security's next-month return is missing
`state_beta`	Asset, market, Boolean condition	Conditional slope and intercept

The default offline reading entry point is the accepted reader package's `notebooks/ch27_carry_lowrisk.ipynb`: `RUN_FRENCH=False`, 10 FX currencies by 360 months (seed 20261114), 240 commodity months (seed 20261115), FX sweep defaulting to 60 worlds, and beta-noise defaulting to 100 worlds by 500 names. It creates only a new `RUN_DIR` and does not write historical figures, CSV, or JSON. The 200-seed and French results and this chapter's four historical figures are existing historical evidence and are not rerun by the default path. The runner also saves a separate execution copy and status. Code blocks are retained in full for study; the following are historical CLI/test entry points from the original engineering project, not reader defaults. Do not run them when the standalone package does not include those engineering entry points:

```
.venv/bin/python code/ch27_carry_lowrisk.py
.venv/bin/python -m pytest tests/test_ch27_carry.py tests/test_ch27_lowrisk.py tests/
    ↪ test_ch27_workflow.py -q
```

Frequency: monthly; `signal_time` is the last instant of month-end (UTC); beta and volatility use data only through that month-end; BAB returns in the following month. Seeds: FX world 20261114 and commodity world 20261115.

Code block one: carry for FX, bonds, and commodities (Sections 8.1–8.3).

```
from quant_course.carry import (
    commodity_curve_carry, fx_carry_from_rates, futures_carry,
    ↪ roll_down_approximation, zero_coupon_carry,
)

s, r_local, r_foreign = 100.0, 0.02, 0.06
f = s * (1 + r_local) / (1 + r_foreign)                    # covered parity; quote
    ↪  = local currency per foreign
assert abs(f - 96.2264) < 1e-4
c = futures_carry(s, f)
assert abs(c - fx_carry_from_rates(r_local, r_foreign)) < 1e-12 and abs(c - 0.04 /
    ↪ 1.02) < 1e-12
```

```
assert futures_carry(1 / s, 1 / f) < 0 < c                    # the inverted quote
    ↪ flips the sign
forward_return = (96.0 - f) / f                               # spot ends at 96
deposit_excess = (96.0 / s * (1 + r_foreign) - (1 + r_local)) / (1 + r_local)
assert abs(forward_return - deposit_excess) < 1e-12 and abs(forward_return +
    ↪ 0.0023529) < 1e-7

z = zero_coupon_carry([1, 2, 3, 5], [0.02, 0.03, 0.035, 0.04], 3.0, 1.0,
    ↪ financing_rate=0.02)
assert abs(z["total"] - (1.035 ** 3 / 1.03 ** 2 - 1)) < 1e-12
assert abs((1 + z["yield_component"]) * (1 + z["roll_down_component"]) - (1 + z["
    ↪ total"])) < 1e-12
approx = roll_down_approximation(2 / 1.03, 0.035, 0.03)
assert abs(approx - 0.0097087) < 1e-7 and approx < z["roll_down_component"]
assert abs(commodity_curve_carry(100.0, 101.0, 1.0, 2.0) + 1 / 101) < 1e-15
```

Code block two: rolls, spliced prices, and tradable returns (Section 8.4).

```
import numpy as np
import pandas as pd

from quant_course.carry import held_contract_returns, naive_splice_returns

dates = pd.period_range("2026-01", periods=4, freq="M")
prices = pd.DataFrame({"A": [102.0, 101.0, 100.0, np.nan], "B": [np.nan, np.nan,
    ↪ 103.0, 102.0]}, index=dates)
chosen = pd.Series(["A", "A", "B", "B"], index=dates)
trade = held_contract_returns(prices, chosen).iloc[:3]
naive = naive_splice_returns(prices, chosen).iloc[:3]
assert np.allclose(trade, [101 / 102 - 1, 100 / 101 - 1, 102 / 103 - 1])
assert np.allclose(naive, [101 / 102 - 1, 103 / 101 - 1, 102 / 103 - 1])
gap = naive - trade
assert gap.iloc[0] == 0 and gap.iloc[2] == 0 and abs(gap.iloc[1] - (103 - 100) / 101)
    ↪  < 1e-12   # new minus old price at the roll, over the old price
assert abs((1 + naive).prod() - 1) < 1e-12 and abs((1 + trade).prod() - 1 + 0.029126)
    ↪  < 1e-6    # stitched 0%, tradable -2.91%
```

Code block three: BAB weights, returns, neutrality, and shrinkage (Sections 8.5 and 8.7).

```
import pandas as pd

from quant_course.low_risk import bab_returns, bab_weights, vasicek_shrink

beta = pd.Series([0.6, 0.9, 1.1, 1.4], index=list("ABCD"))
wl, wh = bab_weights(beta)
assert wl.tolist() == [0.75, 0.25, 0.0, 0.0] and wh.tolist() == [0.0, 0.0, 0.25,
    ↪ 0.75]
bl, bh = float(wl @ beta), float(wh @ beta)
assert abs(bl - 0.675) < 1e-12 and abs(bh - 1.325) < 1e-12
r = pd.Series([0.012, 0.015, 0.010, 0.020], index=beta.index)
by_hand = (wl @ r) / bl - (wh @ r) / bh
idx = pd.period_range("2026-01", periods=2, freq="M")
betas = pd.DataFrame([beta.to_numpy()] * 2, index=idx, columns=beta.index)
out = bab_returns(pd.DataFrame([r.to_numpy()] * 2, index=idx, columns=beta.index),
    ↪ betas)
```

```
assert abs(out["bab"].iloc[0] - by_hand) < 1e-15 and abs(by_hand - 0.0056813) < 1e-7
neutral = bab_returns(pd.DataFrame([beta.to_numpy() * 0.01] * 2, index=idx, columns=
    ↪ beta.index), betas)
assert abs(neutral["bab"].iloc[0]) < 1e-15                    # every asset returns
    ↪ exactly beta x market: BAB = 0
g = lambda x, w: x / (w * x + 1 - w)                          # true beta / scaling
    ↪ beta when the scaling beta is shrunk with weight w
assert abs(g(0.7, 0.6) - g(1.3, 0.6) + 0.2480) < 1e-3         # realised BAB beta
    ↪ -0.248
assert abs(g(0.7, 1.0) - 1.0) < 1e-12 and abs(g(1.3, 1.0) - 1.0) < 1e-12
assert abs(vasicek_shrink(0.5) - 0.7) < 1e-12 and abs(vasicek_shrink(1.8) - 1.48) < 1
    ↪ e-12
```

Code block four: return decomposition and portfolio carry (Sections 6.5 and 6.7).

```
import numpy as np
import pandas as pd

from quant_course.carry import futures_carry, portfolio_carry

s0 = np.array([100.0, 100.0, 100.0, 100.0])
f = np.array([99.0, 99.7, 100.2, 100.6])                      # contango makes the
    ↪ last two carry negative
s1 = np.array([99.6, 99.8, 100.1, 100.0])                     # month-end spot
c = futures_carry(s0, f)
r = (s1 - f) / f
assert np.allclose(r, c + (s1 - s0) / f)                      # return = carry +
    ↪ price change
w = pd.Series([0.5, 0.5, -0.5, -0.5], index=list("ABCD"))
cp = portfolio_carry(w, pd.Series(c, index=w.index))
assert abs(cp - (c[:2].mean() - c[2:].mean())) < 1e-15 and cp > 0
assert float(w.to_numpy() @ r) < cp                           # the price change
    ↪ took part of the carry back
```

10. Reading the figures

Figure 27-1: Smoothed cumulative carry and actual wealth of the carry portfolio (constructed world). Left: the orange dashed line cumulatively adds portfolio carry ($1+\sum C^{\mathrm{p}}$); the blue line compounds actual long-short returns (starting at 1, for 359 months from 1996); gray vertical lines mark global-shock months. Right: histogram of monthly long-short returns, with shock months in orange. Read it this way: cumulative carry is a smooth rising line, while actual returns plunge in shock months and later recover. Over 359 months, average return is only about one-third of average carry; maximum drawdown is 32%, the worst month is -12.2%, and skewness is -0.84. This is a constructed world with specified take-back fraction $\gamma = 0.5$ and shock mechanism.

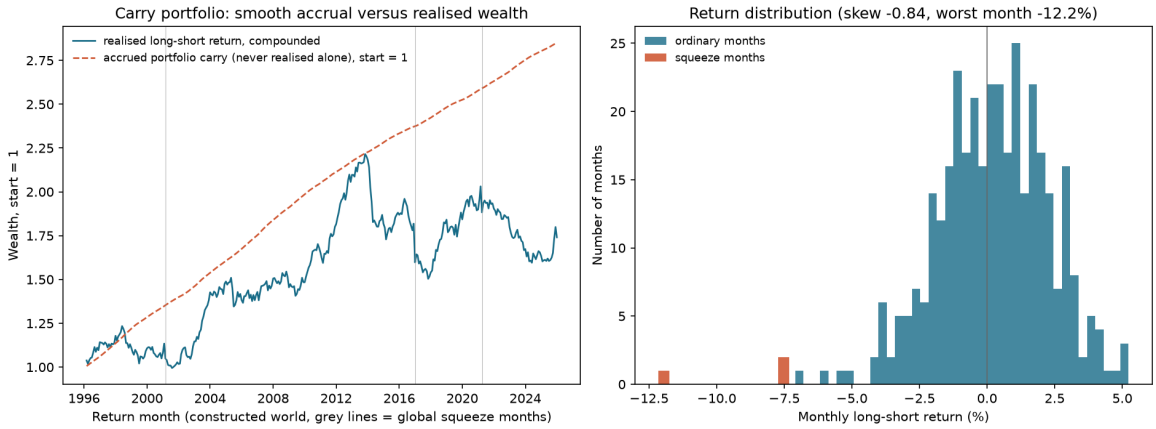


Figure 27-1: Cumulative carry and actual wealth in a constructed world

One path versus 200 new seeds. The figure above shows one path from one seed (20261114), with only 3 shock months (average −9.0%; the other months average +0.26%). Run the same data-generating process once for each of 200 new seeds (20261115 to 20261314; these are integer seeds, not dates; the commodity world also uses seed 20261115, and the same integer in different examples does not by itself establish independent random streams). Median average return is 0.148% per month (5th percentile −0.072%, 95th percentile 0.407%); 11.5% of worlds have negative average returns. Skewness is negative in all worlds (median −1.42, 5th percentile −2.26, 95th percentile −0.36). Median maximum drawdown is 32.2% (5th and 95th percentiles 51.7% and 17.1%), and 74% of worlds exceed 25%. In all 200 worlds, actual average returns are below average portfolio carry (the latter ranges from 0.36% to 0.91% per month). Read it this way: negative skewness and drawdowns in this figure are typical of this process, not one lucky path. But they follow from the specified shock mechanism (1.2% shock probability, specified loadings and sizes), and do not tell us how heavy the left tail of actual FX carry is.

Figure 27-2: Three series from the same contracts (constructed commodity world). Left (log scale, starting at 1): blue is tradable return from holding the same expiry contract over each period and compounding its returns (ignore collateral interest/fees, fully collateralize each period, and adjust notional with wealth using fractional units; this is not a fixed one-contract cash ledger); orange is the compounded spliced-price return; gray dotted line is spot. Right: curve slope $g = (S - F)/S$, with blue shading for backwardation (a long earns the slope) and orange shading for contango (a long pays the slope). Read it this way: spliced prices nearly coincide with spot, while the tradable series moves away from spot in a direction determined by the sign and persistence of the curve slope. The figure shows prolonged contango from about 2010 to 2015, when tradable returns lag spot substantially; after 2016, backwardation prevails most of the time and tradable returns lead spot.

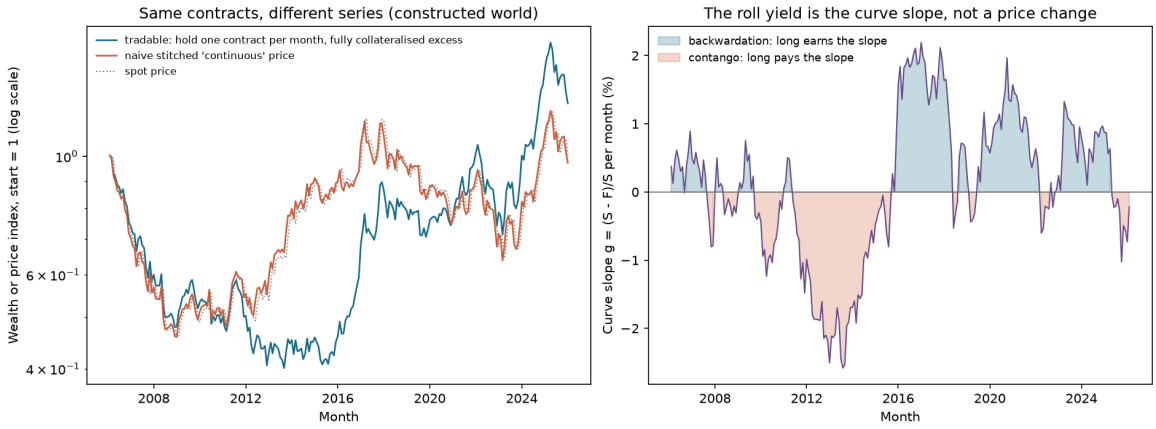


Figure 27-2: Tradable returns, spliced prices, and spot

Figure 27-3: Security market line and beta-sorted industry portfolios. Left: full-sample beta on the horizontal axis (unitless) and monthly mean excess return (in %) on the vertical axis for 10 industries. The gray dashed line is the CAPM line through the market's mean excess return (0.70% per month); the orange line fits the 10 points (slope 0.41). Right: excess wealth (log scale, starting at 1) for the low-beta and high-beta halves (5 equally weighted industries each), sorted by 60-month rolling beta shrunk with $w = 0.6$. Read it this way: the fitted line is flatter than the CAPM line, and the low-beta half has higher long-run wealth. The 10 points and full-sample betas are ex post estimates and are descriptive only.

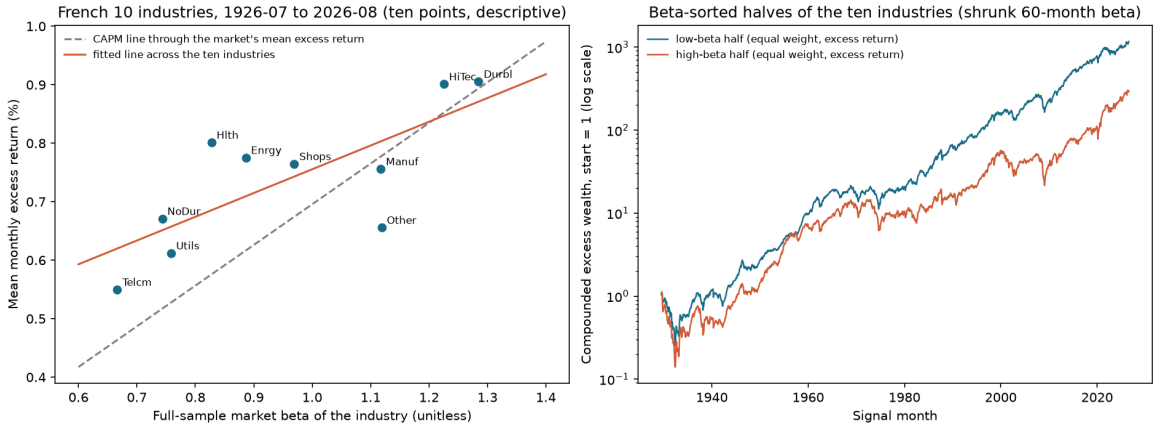


Figure 27-3: Security market line and beta-sorted portfolios

Figure 27-4: Monthly BAB returns and drawdowns (10 industries). Left: each point is one month, with market excess return (%) on the horizontal axis and BAB return (% per month) on the vertical axis; orange points mark BAB's 10 worst months. Right: drawdown in compounded BAB wealth. Read it this way: the scatter slopes down (sample regression beta is negative, −0.20), and all 10 worst months occur in months when the market rose. Maximum drawdown is 57%. These BAB returns are constructed from 10 industries, beyond the published aggregate returns, and exclude costs and financing spreads.

The figure's reference to "rebounds" means a positive market return in the same return month on the horizontal axis. It does not establish a sequence in which a rebound precedes or causes a bad BAB month.

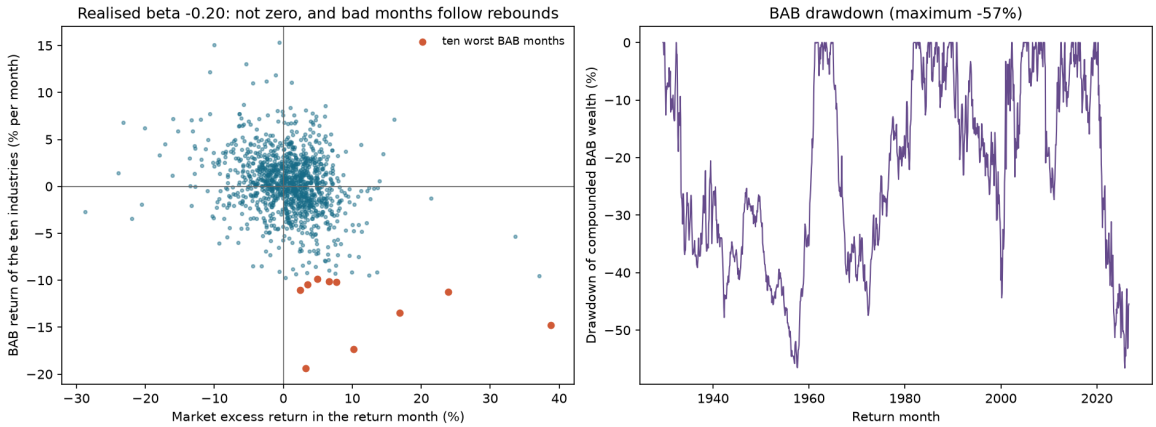


Figure 27-4: Monthly BAB returns and drawdown

11. Real-market example

Data and limitations. As in Chapters 22–25, use the frozen French value-weighted monthly returns for 10 industries (processed CSV SHA-256 `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`), from 1926-07 through 2026-08, or 1,202 months, and FF3 Mkt-RF and RF (processed SHA-256 `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`). Industries are assigned by four-digit SIC code each June and held from July through the following June (French database description page). These are revised published returns for 10 value-weighted industry portfolios, not individual stocks, point-in-time vintages, or tradable executions. All returns are excess returns (subtracting RF). The first signal month is 1929-06 (at least 36 months of beta history required); the last is 2026-07, for 1,166 signal months and return months from 1929-07 through 2026-08. **This chapter has no actual FX forwards, futures curves, or financing rates, so the carry section has no real-data evidence; it contains only hand calculations and a constructed world.**

Specification (fixed in advance). Beta: 60-month rolling OLS (at least 36 months), shrunk with $w = 0.6$ toward prior 1 (Frazzini and Pedersen's constant). BAB: rank weights (equation (16)) for 10 industries, with no financing spread. Low-beta and low-volatility halves: rank by shrunk beta or standard deviation of monthly returns over the past 12 months, and equally weight the lowest 5 against the highest 5. Primary metric: BAB's mean monthly return and its regression on the market.

Low-beta and low-volatility halves (1,166 signal months; t treats months as independent observations, a naive statistic).

Portfolio (5 equally weighted industries)	Mean excess return (% per month)	t	Monthly standard deviation (%)	Annualized Sharpe
Low-beta half	0.706	5.42	4.44	0.55
High-beta half	0.681	3.74	6.21	0.38
Low-volatility half	0.623	4.60	4.63	0.47
High-volatility half	0.763	4.35	6.00	0.44

Portfolio (5 equally weighted industries)	Market beta	Alpha (% per month, t)
Low-beta half	0.78	0.196 (4.21)
High-beta half	1.14	−0.065 (−1.71)
Low-volatility half	0.82	0.085 (1.95)
High-volatility half	1.10	0.047 (1.16)

The two sorts overlap by an average of 3.6 industries. The low-beta and high-beta halves have almost the same mean excess return (0.71% versus 0.68%), so the low-beta half has a higher Sharpe and positive alpha. This is consistent with a flatter security market line, but describes one 100-year sample of 10 industries; it does not test the low-beta effect among individual stocks. The fitted line through the full-sample beta and mean excess return of 10 industries has slope 0.41 (% per month per unit of beta), while the market's mean excess return is 0.70%; the fitted line is flatter than the CAPM line.

BAB (1,166 months). Mean return is 0.2045% per month, $t = 1.99$ (Newey–West lag 6: 2.01), standard deviation 3.50%, and annualized Sharpe 0.20. Regression on the market: alpha 0.334% ($t = 3.38$), **beta** $= -0.197$ **($t = -10.7$)**, and $R^2 = 0.09$. Average leverage is 1.27 on the long leg and 0.88 on the short leg; average leg betas are 0.79 and 1.13; average leg excess returns are 0.645% and 0.715%.

Shrinkage weight used for scaling (same ranking, only leverage changes).

Scaling beta	Mean (% per month)	t	Market beta	Alpha (% per month, t)
Raw ($w = 1.0$)	0.466	3.94	+0.052	0.432 (3.63)
$w = 0.8$	0.322	3.04	−0.081	0.375 (3.54)
$w = 0.6$ (fixed primary specification)	0.205	1.99	−0.197	0.334 (3.38)

In this industry sample, more shrinkage used for scaling produces a more negative market-regression beta estimate; the estimate under no shrinkage is near zero. This is consistent with the mechanism in Section 6.9, but these regressions do not identify true beta, optimal shrinkage, or a causal mechanism. However, $w = 0.8$ and $w = 1.0$ were added after seeing the results, so their higher t statistics are not clean tests. The three shrinkage settings amount to three comparisons; if that is the full test family, the marginal p-values are valid, and a normal approximation is used,

the two-sided 5% Bonferroni reference threshold is about $z = 2.39$ (Chapter 22). The primary setting's $t = 1.99$ does not exceed it. This does not adjust for all subperiod, cost, and grouping variants in the chapter; naive t statistics should not be treated as exact normal tests.

Subperiods ($w = 0.6$).

Return months	Months	Mean (% per month)	t	Market beta	Alpha (% per month, t)
1929-07 to 1963-06	408	0.103	0.72	−0.18	0.235 (1.80)
1963-07 to 1999-12	438	0.153	1.05	−0.14	0.234 (1.62)
2000-01 to 2026-08	320	0.405	1.56	−0.33	0.613 (2.47)

All three subperiod means are positive, and none individually exceeds 2 in absolute t-statistic. Standard deviation is higher after 2000 (4.63%). These subperiods follow the dates used in Chapters 23–26; they were not selected for this chapter.

Financing-spread sensitivity ($w = 0.6$, borrowing rate exceeds the risk-free rate by annualized s; this is an assumption, not data).

s (annualized)	0	0.5%	1%	2%	4%
Mean (% per month)	0.205	0.193	0.182	0.159	0.114
t	1.99	1.88	1.77	1.55	1.11

Down and up months. Classify return months after the fact by whether the market excess return is positive (a diagnostic, not a tradable signal): 462 down months and 704 up months. The low-beta half averages −2.86% in down months and +3.05% in up months; the high-beta half averages −4.55% and +4.11%. The low-beta half's down- and up-market betas are 0.81 and 0.79; the high-beta half's are 1.13 and 1.19. The low- and high-volatility halves average −3.16% and −4.25% in down months.

Worst months and drawdowns. BAB's worst month is 2001-01: −19.3% (market excess return in the same month: +3.3%). All of the 10 worst months—2001-01, 2009-04, 1933-04, 1939-09, 1938-06, 2000-02, 1999-01, 1999-12, 2023-01, and 2026-05—coincide with positive market excess returns; the largest is +38.8% in 1933-04. Maximum drawdown is 56.6%, with the trough in 2025-10. Correlations are −0.30 with market excess return, −0.31 with SMB, and +0.03 with HML.

What can and cannot be said. We can say that, in the 1929–2026 sample of 10 value-weighted industry portfolios, the low-beta half has nearly the same mean excess return as the high-beta half, with lower risk; BAB has a positive mean return (+0.20% per month, $t \approx 2$); its mean is positive in each of three subperiods but not significant in any one; its sample regression exposure is negative, consistent with the shrinkage model, although this chapter does not identify the cause; and its worst months occur during market rebounds. We cannot say: (1) that an individual-stock low-beta premium exists or does not exist; results for 10 industries do not test Frazzini and Pedersen's individual-stock or cross-asset results; (2) that funding constraints explain BAB

returns, because this chapter has no funding-constraint data; (3) that the strategy is tradable, because actual borrow fees, margin, short-sale constraints, and financing rates are unavailable and costs are assumed; or (4) that any t statistic has been adjusted for multiple testing. The chapter evaluates 3 shrinkage settings, 3 subperiods, 5 financing spreads, and 4 overlapping groupings.

What a per-asset carry study is missing. FX: spot and forward quotes at the same time and maturity (including bid and ask), quote direction and settlement identity, and deposit or implied rates for the relevant tenor. Commodities: prices for adjacent contracts, expiry dates, and roll rules. Bonds: zero-coupon or coupon yield curves and the cheapest-to-deliver bond for futures. All assets: financing rates, margin, and fees, plus evidence that these data were available at `signal_time` . The chapter did not download these data; the Project 4 preparation-layer gap table lists them one by one.

12. Common mistakes

Mistake 1: Reversing the quote direction. Substituting a "foreign currency/local currency" price into a "local currency/foreign currency" formula reverses the carry sign (Section 8.1: -3.77% versus $+3.92\%$). Fix: write the units of S beside every formula and test that the reciprocal quote gives the opposite sign (code block one).

Mistake 2: Treating high interest rates as all of carry. Carry is the return under "unchanged prices"; foreign-currency appreciation or depreciation determines the realized result (Section 8.1: a foreign rate 4 percentage points higher is offset by 4% depreciation). Fix: report carry and price change separately (Section 6.5).

Mistake 3: Treating continuous futures prices as tradable PnL. A spliced price contains the new-old contract spread on the roll date (Sections 6.6 and 8.4). Fix: calculate returns from the two endpoint prices of the same contract, then compound them (code block two); report the roll rule and expiry dates.

Mistake 4: Mixing contracts with different maturities without dividing by the maturity difference. Commodity carry is a per-unit-time quantity; if $T_2 - T_1$ is not one month, divide by it (Section 6.4). When comparing commodities, use common units.

Mistake 5: Using the wrong duration for roll-down. Use the duration of the shorter bond after roll-down; using the original bond's duration overstates roll-down (Section 8.2: 1.449% versus the exact 0.973%).

Mistake 6: Treating portfolio carry as expected return. Under Section 6.7's common-capital and leg-weight conditions, a portfolio sorted on carry has nonnegative carry; strict positivity additionally requires a difference between the high and low groups. Price changes can take back some carry, and realized returns can be negative (Section 10: average return is only about 35% of average carry).

Mistake 7: Confusing low beta with low volatility, or vice versa. $\sigma^2 = \beta^2\sigma_m^2 + \sigma_\varepsilon^2$, so the two rankings can be opposite (27-E08); among 10 industries, the low-beta and low-volatility halves overlap by only 3.6 industries on average (Section 11).

Mistake 8: Treating scaling with shrunk beta as market neutrality. In the noisy model in Section 6.9, too much shrinkage produces negative conditional true exposure under the stated assumptions, including leg estimates straddling 1 and positive scaling denominators; the unconditional mean additionally requires integrability. The -0.20 in Section 11 is a historical regression estimate, not an identified causal mechanism. Fix: always report BAB's market regression beta and the shrinkage weight used for scaling. Algebraic neutrality under estimated betas does not establish true market neutrality; empirical evaluation must report sample regression estimates and their uncertainty.

Mistake 9: Assuming leverage is free. The BAB long leg borrows $1/\hat{\beta}_L - 1$ units; borrowing spreads, margin, and short-sale costs affect returns (sensitivity in Sections 8.5 and 11). When comparing levered and unlevered Sharpe ratios, state the financing assumption.

Mistake 10: Using future data in beta or volatility. Centered windows and full-sample beta are unavailable at `signal_time`. Fix: row s may use only months $\leq s$, and test that changing the final row does not change earlier beta estimates:

```
import numpy as np
import pandas as pd

from quant_course.low_risk import rolling_beta

rng = np.random.default_rng(3)
idx = pd.period_range("2000-01", periods=60, freq="M")
m = pd.Series(rng.normal(0, 0.04, 60), index=idx)
a = pd.DataFrame({"x": 1.2 * m + rng.normal(0, 0.01, 60)}, index=idx)
b = rolling_beta(a, m, window=24, min_periods=12)
a2 = a.copy(); a2.iloc[-1, 0] += 0.5                          # change only the last
    ↪ month
b2 = rolling_beta(a2, m, window=24, min_periods=12)
assert b["x"].iloc[:-1].equals(b2["x"].iloc[:-1]) and b["x"].iloc[-1] != b2["x"].iloc
    ↪ [-1]
```

Mistake 11: Treating ex post "down-market months" as a signal. Classifying a month in Section 11 by whether the market fell is a diagnostic; the signal date cannot know the market's direction in the following month. A usable state variable must use only information available by s (the bear-market state in Chapter 25).

Mistake 12: Drawing conclusions from one path or one choice. In the constructed world, 3 shock months determine skewness; in the real-data analysis, 3 shrinkage settings by 3 subperiods make a 9-cell overlapping grid, but do not cover every cost or grouping variant examined (Chapter 22). Fix: repeat with new seeds, log every variant, and do not treat an ex post selected specification as the primary result.

13. Quantitative research applications

Candidate cross-asset field contract (not current Project 4 machine fields).

Field	Source	Time field	Unit	Missing data and convention
`spot`, `forward`	FX quotes (bid and ask)	Quote time, settlement date	Local currency/foreign currency	Quote direction, tenor, bid/ask; absent from the course's current data
`futures_curve`	Settlement prices for each futures expiry	Quote date, expiry date	Price	Roll rule; absent from the course's current data
`yield_curve`	Zero-coupon or coupon yield curve	Date	Annual decimal	Compounding convention; absent from the course's current data
`financing_rate`, `margin`	Broker or repo	Date	Decimal, ratio	Borrowing spread and margin; absent from the course's current data
`market_excess`	FF3 Mkt-RF	Month	Decimal	Same asset universe; available (frozen snapshot)
`beta`, `vol`	Calculated	`signal_time`	Unitless, annualized	Record window, minimum months, and shrinkage weight in the plan; use only data through s

Research workflow (10 industries, real data). Frozen plan: test whether industries with lower 60-month shrunk beta have no lower next-month excess return and whether BAB's average return is positive. Signal is `vasicek_shrink(rolling_beta(...), 0.6)`; primary metrics are BAB's mean monthly return and its market-regression beta. Report the two halves first, then BAB; fix the shrinkage weight and window before viewing results; place all other specifications in the Chapter 22 ledger. Results are in Section 11: BAB mean is 0.2045% per month ($t = 1.99$), sample regression beta is -0.20, and all three subperiod means are positive but individually insignificant; a 2% annual financing spread reduces the mean to 0.159%. The constructed-world results provide limited engineering evidence for the functions and process on the verified inputs; they do not prove correctness for arbitrary inputs and are not real-world effect estimates.

Constraints for Project 4. (1) Estimate beta and volatility using only data through the signal month, and record the shrinkage weight and window in the plan. (2) For every portfolio described as "market neutral," report its market-regression beta and t statistic. (3) Report leverage, borrowing assumptions, and financing-spread sensitivity for leveraged portfolios. (4) Do not make empirical claims about carry or futures without actual quotes, curves, and financing data; see the gap table above. (5) Report the worst month, maximum drawdown, and same-month market return. (6) Log every variant. (7) Address low-risk factors' correlations and crowding alongside value, momentum, and quality in Chapter 28.

13.1 From factor family to current Project 4: verify inputs before integration

The current P4 machine contract synthesizes only seven raw fields for stock value, momentum, and quality (Chapters 24–26). This chapter retains carry, low risk, and BAB as candidate methods. The interface table below explains why they are not yet part of that synthesis; "covered in the chapter" does not mean "implemented in the engineering pipeline."

Candidate method	What this chapter can calculate	Corresponding input or contract currently missing from P4
FX carry	$(S - F)/X$ given same-tenor quotes	Spot and forward at the same time, settlement date, bid/ask, capital, and financing convention
Bond carry	Holding return and roll-down under an unchanged curve	Cash flows and yield curve by bond, maturity and compounding convention, financing, and executable prices
Commodity carry	Adjacent-expiry price spread and same-contract PnL	Curve with contract identities, expiry, roll, collateral, and fees
Low volatility/low beta/BAB	Risk estimates through signal time, ranking, and scaling	Point-in-time stock returns and market benchmark, estimation window, shrinkage, financing, and borrow rules

French industry monthly returns support only the aggregate industry diagnostics already in this chapter. They do not supply the per-stock fields above and do not establish a real carry premium. Before integration, specify quotes and denominators, the information clock, missing-data handling, and degenerate-case policy; then Chapter 28 can define the synthesis and portfolio constraints. The current C13 P4 repair only aligns the declared supported universe, 60-name threshold, and industry projection/gross exposure; it does not add the factors in this chapter. The research ledger and IC inference in Chapters 22–23 and the seven-field definitions in Chapters 24–26 remain prerequisites; this chapter does not rerun their empirical tests.

14. Chapter summary

Carry is the return under "unchanged market conditions": for futures, $C = (S - F)/X$; for FX forwards, $C = (r^* - r)/(1 + r)$ when quoted local/foreign; for zero-coupon bonds, it is the yield component plus roll-down; for commodities, it is $(F^1 - F^2)/(F^2(T_2 - T_1))$. Return equals carry plus price change, which can take back much of carry. A high-minus-low portfolio's carry is nonnegative under the stated common-capital and weight conditions, but its realized return can still be negative. Spliced continuous futures prices are not tradable returns; their difference is the roll spread. Low beta and low volatility differ: $\sigma^2 = \beta^2\sigma_m^2 + \sigma_\varepsilon^2$. BAB leverages the low-beta leg and deleverages the high-beta leg; it is market neutral under estimated beta. In the noiseless model, shrinking toward 1 on the positive-denominator interval produces negative exposure. In the noisy model, optimal shrinkage gives conditional neutrality under all conditions in Section 6.9; the sign under over- or under-shrinkage also requires leg estimates to straddle 1 and positive denominators. Historical negative beta does not identify that mechanism.

For the 10 industry portfolios: the low-beta half has nearly the same mean excess return as the high-beta half, with less risk; BAB averages 0.20% per month ($t \approx 2$), has sample regression beta -0.20, and has positive but individually insignificant mean returns in all subperiods. Its worst months occur during market rebounds, and maximum drawdown is 57%. In the constructed FX world, average return is only about one-third of average carry, with negative skewness. Scope boundary: 10 industries are not individual stocks; actual FX, futures, and financing data are unavailable; costs and financing spreads are assumed; and the shock mechanism in the constructed world is specified.

15. Exercises

27-E01 (calculation). The quote is dollars per euro, $S = 1.10$; the annual dollar rate is 4%, the annual euro rate is 2%, and both are one-year simple rates. Assume CIP. Find F, the carry on a long euro forward (calculate once from prices and once from rates), and carry on a short euro forward. If spot is 1.12 after one year, find the return on the long forward and verify it using a deposit comparison.

27-E02 (calculation). Zero-coupon yields (annual compounded): 3% at 1 year, 3.5% at 2 years, 4% at 3 years, and 4.5% at 5 years. Buy a 3-year zero-coupon bond and hold for one year, assuming the curve is unchanged. Find total return, yield component, roll-down component, and excess return after 3% financing. Then approximate roll-down with modified duration and explain which bond's duration to use.

27-E03 (calculation). A commodity's nearby contract (one month to expiry) is 80, and the next-nearby contract (three months to expiry) is 82. Using the linear proxy in Section 6.4 and capital equal to the deferred contract price of 82, calculate monthly carry. If the same collateral capital earns 0.3% per month, what is total return? Is the curve in backwardation or contango?

27-E04 (calculation). Contract A is priced at 60, 59, and 58 in periods 0, 1, and 2; contract B is priced at 61 and 60 in periods 2 and 3. Hold A through period 2, then roll into B. Ignore interest and fees; fully collateralize each period and adjust notional with wealth (fractional units allowed). Calculate each return and cumulative return for the rebalanced index and the spliced price series, plus their difference on the roll date. Verify that the difference equals the new-old contract spread at the roll date divided by the old price.

27-E05 (calculation). Monthly carry for four currencies is $0.5\%, 0.3\%, -0.2\%, -0.6\%$, and next month's price changes $\Delta S/F$ are $-0.4\%, -0.2\%, +0.1\%, +0.5\%$. Go long the first two and short the last two with weights ± 0.5. Calculate portfolio carry, portfolio return, the fraction taken back, and the cross-sectional regression slope of return on carry.

27-E06 (calculation). Estimated betas for four securities are $0.5, 0.8, 1.3, 1.6$ and next month's excess returns are $1.0\%, 1.2\%, 1.5\%, 2.5\%$. Calculate BAB weights, the two leg betas and leverage, and BAB return. If borrowing costs 0.3% more per month, calculate long-leg borrowing cost and net return.

27-E07 (calculation). The true betas of a low-beta leg and high-beta leg are 0.4 and 1.6, with no estimation noise. Calculate each leg's true exposure and BAB's true beta when scaling with shrinkage weights $w = 1$ and $w = 0.6$ toward a prior of 1.

27-E08 (concept and calculation). Market volatility is 16%. X has $\beta = 0.5$ and idiosyncratic volatility 25%; Y has $\beta = 1.2$ and idiosyncratic volatility 5%. Calculate each asset's total volatility, state their rankings by beta and by volatility, and explain why a low-beta portfolio needs leverage to be compared with the market.

27-N01 (cash-flow and interface diagnosis). A contract has $S = 102, F = 100, r^f = 1\%$, and actual expiry spot is 99. (a) For each of $X = 50, 100$, calculate carry, price-change component, excess return, collateral interest, and total return. (b) For BAB legs with betas $0.675, 1.325$, state risky-asset net exposure, gross exposure, estimated beta, and financing-leg weight. Does this make the position dollar neutral? (c) If you wanted to add FX carry to current P4, list at least three missing inputs and explain why existing `mom_12_1` cannot substitute for a forward quote.

16. Programming exercises

27-P01. Use `zero_coupon_carry` to reproduce 27-E02. Assert that total return equals (1+yield component)(1+roll-down component)-1 and matches the hand calculation; also assert that the duration approximation is below exact roll-down. Acceptance: all three assertions pass.

27-P02. Use `held_contract_returns` and `naive_splice_returns` to reproduce 27-E04. Assert that the roll-date difference equals the new-old contract price spread on that date divided by the old price, that the difference is zero on non-roll dates, and that cumulative spliced return is 0. Acceptance: all four assertions pass.

27-P03. Simulate 500 securities: true beta has mean 1 and standard deviation 0.25; estimation noise has standard deviation 0.25 (optimal shrinkage weight 0.5). Use `bab_weights` to construct BAB and calculate true BAB beta (the ratio of true beta to the beta used for scaling) at $w = 1, 0.5, 0.2$, averaging over 200 simulations. Acceptance: at $w = 1$, the result exceeds 0.3; at $w = 0.5$, its absolute value is below 0.05; at $w = 0.2$, it is below -0.15.

17. Challenge problems

27-C01 (derivation). Let BAB's true leg betas be $0 < x_L < 1 < x_H$, and let the beta used for scaling be $\hat{\beta} = wx + (1 - w)$ with $0 < w \leq 1$ (so both scaling betas are strictly positive). Derive true portfolio beta $= g(x_L) - g(x_H)$, where $g(x) = x/(wx + 1 - w)$. Prove that it is strictly negative for $w < 1$ and zero for $w = 1$. Now suppose estimated beta is noisy, $\hat{\beta} = \beta + e$ (each security has $\beta_i \sim N(1, \sigma_\beta^2)$ and $e_i \sim N(0, \sigma_e^2)$, all mutually independent). Prove that shrinking with $w^* = \sigma_\beta^2/(\sigma_\beta^2 + \sigma_e^2)$ makes $E[\beta_i \mid \hat{\beta}_i]$ equal the beta used for scaling, and gives each leg conditional true exposure 1 given $\hat{\beta}$ (on the observation region $s_L, s_H \neq 0$). Under $\hat{\beta}_L < 1 < \hat{\beta}_H$ and $s_L, s_H > 0$, explain why too little and too much shrinkage give exposures of opposite signs. Give an example showing that removing the positive-denominator condition can reverse the conclusion. If only moments and independence are given, without normality, state

the result as "best linear prediction." Additional prerequisite: linear least squares (Chapter 6). Grading: first part 40, second part 40, sign discussion 20.

27-C02 (open-ended). Write a data-gap table and frozen plan for a "real-data study of FX carry." For each field (spot, forward, bid/ask, quote direction, settlement date, interest rate, financing spread), state a real source, time field, and missing-data rule. Explain which conclusions cannot be drawn without those data. Give two metrics that separately test "average carry is positive" and "carry return is positive," and two crash-risk statistics. Grading: gap table 30, frozen plan 30, claims that cannot be made 20, metrics 20.

18. Complete answers and solution notes

27-E01. $F = 1.10 \times 1.04/1.02 = 1.12157$. For a long euro forward, $C = (S - F)/F = (1.10 - 1.12157)/1.12157 = -1.923\%$; from rates, $(r^* - r)/(1 + r) = (0.02 - 0.04)/1.04 = -1.923\%$, as expected. For a short euro forward, $C = (F - S)/F = +1.923\%$ (the dollar rate is higher, so the dollar side earns the differential). If spot is 1.12, the long-forward return is $(1.12 - 1.12157)/1.12157 = -0.140\%$. Deposit comparison: the euro deposit converts to dollars at $1.12/1.10 \times 1.02 = 1.03855$, versus 1.04 for the dollar deposit; excess return is $(1.03855 - 1.04)/1.04 = -0.140\%$, matching the forward. Avoiding an excess loss requires spot at least equal to F, or euro appreciation of at least about 1.96%. Strictly positive forward excess return requires spot strictly above F.

27-E02. $y_3 = 4\%$ and $y_2 = 3.5\%$. Total return is $= 1.04^3/1.035^2 - 1 = 1.124864/1.071225 - 1 = 5.0073\%$. Yield component is 4.0%; roll-down is $(1.04/1.035)^2 - 1 = 0.9685\%$; and $1.04 \times 1.009685 = 1.050073$. After 3% financing, excess return is 2.0073%. Duration approximation: modified duration of the 2-year zero after roll-down is $2/1.035 = 1.9324$, giving roll-down of approximately $1.9324 \times (0.04 - 0.035) = 0.9662\%$, versus exact 0.9685%, a difference of 0.002 percentage points. Using the 3-year bond's duration before roll-down, $3/1.04 = 2.885$, gives 1.442%, an overestimate.

27-E03. $C = (80 - 82)/(82 \times 2) = -1.2195\%$ per month (simple annualized: -14.63%, assuming the curve is unchanged); this is contango because the deferred contract is more expensive. Under the question's linear monthly proxy and capital convention, collateral earning 0.3% per month gives total return of $-1.2195\% + 0.3\% = -0.9195\%$ per month.

27-E04. Tradable returns: A's $59/60 - 1 = -1.667\%$ and $58/59 - 1 = -1.695\%$; B's $60/61 - 1 = -1.639\%$. Cumulative return is $0.98333 \times 0.98305 \times 0.98361 - 1 = -4.918\%$. Spliced prices $60, 59, 61, 60$ yield -1.667%, $+3.390\%$ (59→61), and -1.639%, with cumulative return 0. The roll-date difference is $3.390 - (-1.695) = 5.085\%$, equal to $(61 - 58)/59 = 5.085\%$: the period-2 spread between new contract B at 61 and old contract A at 58, divided by the old price 59. The curve is in contango because the new contract is more expensive. The spliced series counts that 5.1% as a return, but nobody earns it.

27-E05. Portfolio carry is $0.5(0.5+0.3)-0.5(-0.2-0.6)=0.4+0.4=0.8\%$. Currency returns $= C+\Delta S/F$ are $0.1\%, 0.1\%, -0.1\%, -0.1\%$; portfolio return is $0.5(0.2)-0.5(-0.2)=0.2\%$. Take-back is 0.6%, or 75% of carry; realized return is 25% of carry. For the cross-sectional regression, $\bar{C}=0$ and $\bar{r}=0$, so the slope is $\sum Cr/\sum C^2=(0.05+0.03+0.02+0.06)/(0.25+0.09+0.04+0.36)=0.16/0.74=0.216$ (calculated in percentage points, but the ratio is scale-invariant). Under the linear parameterization in Section 6.5 alone, this implies $1-b\approx 78.38\%$. This is a description of that four-currency cross-sectional OLS; it is not the same parameter as the 75% portfolio take-back above, and does not identify a population γ exactly.

27-E06. Ranks are $1,2,3,4$: $w_L=(0.75,0.25,0,0)$ and $w_H=(0,0,0.25,0.75)$. $\hat{\beta}_L=0.75\times 0.5+0.25\times 0.8=0.575$; $\hat{\beta}_H=0.25\times 1.3+0.75\times 1.6=1.525$; leverage is $1/0.575=1.739$ and $1/1.525=0.656$. $r_L=0.75\times 1.0+0.25\times 1.2=1.05\%$, and $r_H=0.25\times 1.5+0.75\times 2.5=2.25\%$. Thus, $r^{\mathrm{BAB}}=1.05/0.575-2.25/1.525=1.826\%-1.475\%=0.351\%$. The long leg borrows 0.739 units; borrowing cost is $0.3\%\times 0.739=0.222\%$, leaving $0.351-0.222=0.129\%$. The short-leg leverage is below 1, so it does not borrow.

27-E07. At $w=1$, scaling beta equals true beta, so long-leg true exposure is $0.4/0.4=1$, short-leg exposure is $1.6/1.6=1$, and BAB beta is 0. At $w=0.6$, scaling betas are $0.6\times 0.4+0.4=0.64$ and $0.6\times 1.6+0.4=1.36$. Long-leg true exposure is $0.4/0.64=0.625$, short-leg true exposure is $1.6/1.36=1.176$, and BAB beta is $0.625-1.176=-0.551$. Shrinking toward 1 leaves the low-beta leg underlevered and the high-beta leg insufficiently delevered, producing negative net exposure. A truly neutral short leg would have size $1/1.6=0.625$, but scaling by the shrunk beta gives $1/1.36=0.735294$. The larger short position therefore carries more negative true exposure.

27-E08.

$$\sigma_X=\sqrt{(0.5\times 0.16)^2+0.25^2}=\sqrt{0.0064+0.0625}=26.2\%,$$
$$\sigma_Y=\sqrt{(1.2\times 0.16)^2+0.05^2}=\sqrt{0.036864+0.0025}=19.8\%.$$

By beta, X is low and Y high; by volatility, Y is low and X high, so the rankings are reversed. A low-beta portfolio has $\beta_p<1$; to match the market's market exposure it needs leverage of $1/\beta_p$ (1.43 when $\beta_p=0.7$). Leverage brings financing costs and constraints.

27-P01

```
from quant_course.carry import roll_down_approximation, zero_coupon_carry

z = zero_coupon_carry([1, 2, 3, 5], [0.03, 0.035, 0.04, 0.045], 3.0, 1.0,
    ↪ financing_rate=0.03)
assert abs(z["total"] - (1.04 ** 3 / 1.035 ** 2 - 1)) < 1e-12 and abs(z["total"] -
    ↪ 0.050073) < 1e-6
assert abs((1 + z["yield_component"]) * (1 + z["roll_down_component"]) - 1 - z["total
    ↪ "]) < 1e-12
assert abs(z["excess_total"] - 0.020073) < 1e-6
assert roll_down_approximation(2 / 1.035, 0.04, 0.035) < z["roll_down_component"]
```

27-P02

```
import numpy as np
import pandas as pd

from quant_course.carry import held_contract_returns, naive_splice_returns

dates = pd.period_range("2026-01", periods=4, freq="M")
prices = pd.DataFrame({"A": [60.0, 59.0, 58.0, np.nan], "B": [np.nan, np.nan, 61.0,
    ↪ 60.0]}, index=dates)
chosen = pd.Series(["A", "A", "B", "B"], index=dates)
trade, naive = held_contract_returns(prices, chosen).iloc[:3], naive_splice_returns(
    ↪ prices, chosen).iloc[:3]
gap = naive - trade
assert abs(gap.iloc[1] - (61 - 58) / 59) < 1e-12 and gap.iloc[0] == 0 and gap.iloc[2]
    ↪  == 0
assert abs((1 + naive).prod() - 1) < 1e-12 and abs((1 + trade).prod() - 1 + 0.049180)
    ↪  < 1e-6
```

27-P03

```
import numpy as np
import pandas as pd

from quant_course.low_risk import bab_weights

n, worlds = 500, 200
realised = {}
for w in (1.0, 0.5, 0.2):
    betas = []
    for k in range(worlds):
        rng = np.random.default_rng(1000 + k)
        true = rng.normal(1.0, 0.25, n)                              # true betas
        est = true + rng.normal(0.0, 0.25, n)                        # noisy estimates;
    ↪ optimal weight 0.25^2 / (0.25^2 + 0.25^2) = 0.5
        scaled = pd.Series(w * est + (1 - w))                        # shrunk beta used
    ↪ for ranking and scaling
        wl, wh = bab_weights(scaled)
        betas.append((wl * true).sum() / (wl * scaled).sum() - (wh * true).sum() / (
    ↪ wh * scaled).sum())
    realised[w] = float(np.mean(betas))
assert realised[1.0] > 0.3 and abs(realised[0.5]) < 0.05 and realised[0.2] < -0.15
```

27-C01. True portfolio beta is $w_L^\top \beta^T / \hat{\beta}_L$ for the long leg and $w_H^\top \beta^T / \hat{\beta}_H$ for the short leg. Without noise, $\hat{\beta} = wx + (1-w)$, so the two legs' exposures are $g(x_L)$ and $g(x_H)$, where $g(x) = x/(wx+1-w)$. Its derivative is $g'(x) = \big[(wx+1-w) - wx\big]/(wx+1-w)^2 = (1-w)/(wx+1-w)^2$, which is strictly positive when $w < 1$. The function has a pole at $x = -(1-w)/w$; a positive derivative means it increases on either side of the pole, not across it. The stated $0 < x_L < x_H$ makes both scaling betas $wx + 1 - w$ positive, so both points are to the right of the pole in the same interval. Therefore $g(x_L) < g(x_H)$ and BAB true beta is negative. For example, if $x_L = -1$, $x_H = 2$, and $w = 0.6$, scaling betas are -0.2 and 1.6, and the difference in exposures is $5 - 1.25 = +3.75$. When $w = 1$, $g \equiv 1$ and the difference is zero. Numerical example ($x_L = 0.7$, $x_H = 1.3$, $w = 0.6$): $0.7/0.82 - 1.3/1.18 = 0.8537 - 1.1017 = -0.248$.

With noise, leg membership is determined only by ranks of $\hat{\beta}$. Suppose the pairs (β_i, e_i) are cross-sectionally independent and jointly normal. Conditional on the full vector $\hat{\beta}$, $E[\beta_i \mid \hat{\beta}] = E[\beta_i \mid \hat{\beta}_i] = 1 + w^*(\hat{\beta}_i - 1)$, where $w^* = \mathrm{Cov}(\beta, \hat{\beta})/\mathrm{Var}(\hat{\beta}) = \sigma_\beta^2/(\sigma_\beta^2 + \sigma_e^2)$. Scaling beta is $s_i = w\hat{\beta}_i + 1 - w$. The long-leg scaling beta is $s_L = \sum_i w_{L,i} s_i$, and its true beta is $\beta_L^T = \sum_i w_{L,i}\beta_i$, so $E[\beta_L^T \mid \hat{\beta}] = w^*\hat{\beta}_L + 1 - w^*$. If $w = w^*$, this equals s_L, and the long leg's conditional exposure is $E[\beta_L^T/s_L \mid \hat{\beta}] = E[\beta_L^T \mid \hat{\beta}]/s_L = 1$ (s_L is constant, so there is no Jensen term). The short leg is the same, so conditional on $\hat{\beta}$, BAB's true-beta expectation is zero (requires $s_L, s_H \neq 0$; an unconditional expectation additionally requires the ratios to be integrable, which cannot be assumed under normal tails and is not proved here).

When $w \neq w^*$, let $m = w^*\hat{\beta} + 1 - w^*$ be the conditional expectation. Then $s - m = (w - w^*)(\hat{\beta} - 1)$. Suppose $\hat{\beta}_L < 1 < \hat{\beta}_H$ and $s_L, s_H > 0$ (consistent with `bab_returns` rejecting nonpositive leg betas; when $\hat{\beta}_H > 1$, $s_H > 0$ automatically). If $w > w^*$, then $s_L < m_L$, so $m_L/s_L > 1$, while $s_H > m_H$, so $m_H/s_H < 1$; net exposure is positive (too little shrinkage). If $w < w^*$, both inequalities reverse and exposure is negative (too much shrinkage). Positive denominators are necessary: with $\hat{\beta}_L = -2$, $\hat{\beta}_H = 2$, $w^* = 0.5$, and $w = 0.6$, $s_L = -0.8$, $m_L = -0.5$, so the ratio is $0.625 < 1$ and net exposure is $0.625 - 1.5/1.6 = -0.3125$. The numerical results in 27-P03 $(+0.47, -0.0006, -0.24)$ are consistent. Grading: first part (derivative and sign) 40, second part (conditional expectation) 40, sign discussion 20.

27-C02 (grading guide and reference answer). The course's current data contain no FX spot or forward quotes, futures curves, yield curves, or financing spreads; they do contain French Mkt-RF, RF, and monthly returns for 10 industries. Each row of the gap table should include field, required real source (for example, historical dealer quotes or exchange settlement prices), time field (`available_at`), and missing-data rules (missing bid/ask and holiday-adjusted settlement date). Without those fields, do not make empirical claims about the size or stability of a carry premium, crash probability, or tradability, or present constructed-world numbers as FX market estimates. Two metrics: (1) average portfolio carry and average return separately, with their ratio and confidence interval; (2) cross-sectional return-on-carry slope b (corresponding to $1 - \gamma$). Two crash metrics: skewness and worst month (with the same-month global-shock proxy value), plus maximum drawdown. The frozen plan must state quote direction, tenor, roll and financing assumptions, and number of variants before viewing returns.

27-N01 complete answer. (a) Contract carry PnL is $102 - 100 = 2$, actual PnL is $99 - 100 = -1$, and spot change is $99 - 102 = -3$. At $X = 50$: carry 4%, price change -6%, excess return -2%; interest is 0.5, ending equity is 49.5, and total return is -1%. At $X = 100$: carry 2%, price change -3%, excess return -1%; interest is 1, ending equity is 100, and total return is 0. Both decompositions equal actual contract PnL divided by capital plus the risk-free return for the period. (b) $L = 40/27 = 1.481481$, $H = 40/53 = 0.754717$; risky-asset net exposure is $1040/1431 = 0.726765$, gross exposure is $3200/1431 = 2.236198$, estimated beta is $L(0.675) - H(1.325) = 0$, and financing-leg weight is $-1040/1431$. Market-beta neutrality does not mean dollar neutrality. This is factor PnL per unit of reference capital. (c) At minimum, the signal-time same-tenor spot and forward, quote direction and settlement date, bid/ask, and collateral/financing assumptions are required. `mom_12_1` is a window of realized stock returns; a forward is today's price for future delivery. They are different economic objects available at different times. Current P4 has no FX fields, so momentum cannot fill the gap.

19. Further reading

- Must know: Koijen, Moskowitz, Pedersen, and Vrugt, *Carry* (NBER Working Paper w19325, August 2013 version). Read the definition in Section 2 (equations (2)–(7), (9), (10), (13)–(15)) and portfolio carry (equations (24) and (25)); the journal version has not been read.
- Should know: Frazzini and Pedersen, *Betting Against Beta* (author manuscript dated 2013-05-10), BAB construction (equations (14)–(17)) and the five predictions in the abstract; the theory and empirical sections and journal version have not been read.
- Should know: Chapter 10 of this course (financial markets), on forward, futures, and bond cash flows; and Chapter 18 (volatility), on ex ante volatility estimation.
- Advanced: Read the original work by Lustig et al. on FX carry and Brunnermeier and Pedersen on funding liquidity and market liquidity, and verify empirical findings about carry crashes and funding constraints; not read for this chapter.
- Optional: Find exchange-published futures contract specifications and settlement-price histories, and practice identifying expiry dates, roll rules, and margin requirements.

20. References

Citation keys and scope of reading are listed in `source_notes.md`.

- CH27_KMPV2013: Koijen, R. S. J., Moskowitz, T. J., Pedersen, L. H., Vrugt, E. B. *Carry*. NBER Working Paper 19325, August 2013 version (working paper; journal version not read); equations (2)–(7), (9)–(10), (13)–(15), (24)–(25), and footnote 6.
- CH27_FP2013: Frazzini, A., Pedersen, L. H. *Betting Against Beta*, author manuscript dated 2013-05-10 (working paper; journal version not read); abstract, equations (14)–(17), and relevant text on pages 16–21.
- CH27_FRENCH_IND: Kenneth R. French Data Library, "Detail for 10 Industry Portfolios" page.
- CH25_AMP2013: Asness, Moskowitz, and Pedersen (2013), commodity futures return construction (Chapter 25 source notes).
- FRENCH: Frozen snapshots of French 10-industry and FF3 data (`data/manifests/french_industries_2026-10-05.json`, `data/manifests/french_factors_2026-10-05.json`).

Chapter 28 — Multi-Factor Models: Preprocessing, Combination, and Constraints

Through line: A multi-factor workflow has three distinct responsibilities. Preprocessing changes the scale, outliers, and unwanted exposures of an individual signal, using only the cross-section for the same date. Combination determines how several signals are added, with weights fixed before seeing evaluation-period returns. Constraints determine which exposures the final portfolio may have. Mixing these steps leads to errors that do not raise exceptions but make results unreliable: choosing weights from test-period returns, standardizing with full-sample means, or orthogonalizing in an order chosen after seeing the results. Prerequisites: Chapter 24 (value and point-in-time fundamentals), Chapter 25 (momentum), Chapter 26 (quality), Chapter 27 (carry and low risk), Chapter 23 (cross-sectional research), and Chapter 21 (portfolio construction). After this chapter, you should be able to define same-date z-scores, winsorization, ranks, and normal scores and describe their IC properties; use same-date regressions to `neutralize` industries and size and explain how this relates to linear projection of weights; derive the composite IC of two correlated signals and explain why orthogonalization order matters; describe two treatments for missing factors and their scale biases; explain why full-sample standardization is unusable; and use published factor returns to show that correlations can change sign over time and to derive the Sharpe-ratio formula after diversification.

1. Chapter goals

Readers should be able to answer: What steps turn a raw signal into a tradable portfolio, and which date's data does each step use? Why can neutralizing a signal recover IC that an industry shift has diluted? Why does the order matter when two correlated factors are combined? What should we do when a company is missing one factor? Can weights be chosen from performance in the evaluation period?

1. Distinguish the responsibilities of the three layers: preprocessing (one signal, same-date cross-section), combination (multiple signals, weights fixed in advance), and constraints (portfolio-level exposure limits); explain what data each layer may use.
2. Define and implement same-date winsorization, z-scores, robust z-scores, ranks, and normal scores; state which transformations preserve Spearman IC and which change Pearson IC.
3. Derive the attenuation factor $1/\sqrt{1+\sigma^2_{\text{noise}}/\sigma^2_{\text{true}}}$ in Pearson IC when a signal contains a component unrelated to returns (an industry shift, size loading, or outlier), and verify it on a constructed panel using 60 new seeds.
4. Derive the algebra of same-date regression neutralization: with all industry indicators, the residual is the within-group demeaned value; with a size control, the procedure is two-step Frisch–Waugh–Lovell; residuals are orthogonal to controls; residuals do not change in a redundant design (industry indicators plus an intercept); for weights proportional to the signal, linear-constraint projection and neutralization produce the same vector, but not for quantile weights.

5. Derive the composite IC of two standardized signals, $(\rho_1+\rho_2)/\sqrt{2(1+r_{12})}$, and the IC after sequential orthogonalization; explain order dependence; derive the IC-maximizing weights proportional to $R^{-1}\rho$ and state the requirements.
6. Explain how "zero fill" and "reweight over available factors" change scale, using the variance formula $(1+(k-1)\bar{r})/k$, and report coverage.
7. Explain the look-ahead problem in full-sample standardization, give an expanding version that uses only the past, and show a numerical counterexample.
8. Read published FF5 factor returns: the correlation matrix, the opposite signs of HML–RMW correlations in 1963–1999 and 2000–2026, and the range of rolling correlations; derive and check the Sharpe ratio for an equal-risk average.
9. Report returns, risk, and exposures for three portfolio constructions (quantile weights on neutralized signals, quantile weights on the raw composite, and the raw composite projected to industry neutrality), and state that these are results from a constructed panel.

Completion criterion: Every multi-factor signal includes a process table stating the order of each step, which date it uses, when and by whom each parameter was fixed (winsorization quantiles, z-score degrees of freedom, minimum factor count, weights), and the share of names dropped or imputed.

2. Why the multi-factor workflow needs clear boundaries

Chapters 24–27 each construct a factor: value, momentum, quality, and carry/low risk. Their units differ (ratios, returns, accounting ratios, beta); their cross-sectional distributions differ (some are heavy-tailed); they contain components unrelated to returns (industry shifts, size); they are correlated; and they cover different sets of names. Compressing them into one sortable score requires a series of researcher choices. Each choice can be judged by evaluation-period returns, so each choice is a degree of freedom. Chapter 22 explains how to count these degrees of freedom; this chapter explains their mathematical objects and which dates each step may use.

This chapter has three rules. First, preprocessing uses only the same-date cross-section: the output for row t depends only on the inputs in row t. This assumes all fields, industry labels, version selections, and parameter policies for that row were available at signal time. Being in the same row does not by itself establish point-in-time eligibility. Second, combination weights are written into the plan before evaluation-period returns are seen. They may be estimated in a discovery period, but evaluation must use an independent holdout, with the choice recorded in the ledger. Third, constraints belong at the portfolio level. They may be equivalent to signal-level neutralization (in the linear case) or differ from it (for quantile weights); do not substitute one for the other without reporting the distinction.

The sequence is: define same-date transforms (6.1), derive attenuation from noise (6.2) and the algebra of neutralization (6.3); then derive composite IC and order dependence (6.4), weight choice (6.5), and missing-factor treatments (6.6); next address look-ahead in time-series standardization (6.7), factor correlation and diversification (6.8), and the definition of crowding (6.9); finally report results on a constructed panel (Section 10) and published factor returns (Section 11).

3. Intuition

Imagine three analysts scoring 300 stocks each. The first analyst's scores include an industry-wide optimistic shift (every technology stock gets 2 extra points). The second occasionally assigns one or two stocks outlier scores of ± 12. The third analyst's score is related to market capitalization. Sorting directly by these scores mostly compares industries for the first analyst, lets the outliers inflate variance for the second, and creates a small-cap long-short portfolio for the third. Preprocessing removes these return-unrelated components, step by step, before combination: pull in outliers (winsorization), regress out industry and size effects (neutralization), then put the scales on a common footing (z-scores).

Combination is a separate task: how should the three analysts' scores be added? If two analysts' scores are highly correlated, they are saying much the same thing, so adding them provides little new information. If we let the first analyst speak, then listen to only the part of the second analyst that the first did not capture (orthogonalization), the order changes what remains from the second. If we assign more weight to whoever performed better in the past, and "the past" is the evaluation period, we are choosing weights from the answer.

Constraints come last: must the final long-short portfolio have zero net position in each industry? Should market or size exposure be limited? Signal neutralization can approximate this step, or weights can be projected directly. In the linear case these are the same operation.

Simplifying boundary: In this chapter's constructed panel, industry shifts, size loadings, and outliers are disturbances specified by the author, so neutralization and winsorization help. In real data, an industry shift may itself predict returns; in that case neutralization would remove true signal and the conclusion would reverse. This chapter does not simulate that case.

3.1 From one row of data to positions: each step produces a different object

Complete worked example: from raw values to a composite score

Use the six-name cross-section from Section 8.2, with industries A/A/A/B/B/B and raw values $(1, -1, 0, 11, 10, 9)$. To isolate the steps, this example adds no outlier, applies no winsorization, and includes no size control; the production workflow still follows the frozen rule of winsorizing first, then taking industry/size residuals, then computing z-scores. The within-industry means are 0 and 10, so the residuals are $e = (1, -1, 0, 1, 0, -1)$. Their mean is 0 and their sum of squares is 4; the sample standard deviation is $s = \sqrt{4/5} = 0.894427$, so $z = e/s$. These six names form an **explicit small-example configuration**; they are not an exception to P4's production minimum of 60 names.

Now add a second signal that has already been standardized on the same date: $z_2 = (1, 0, -1, 0, -1, 1)/s$. Both signals use only information from the current cross-section. The equal-weight composite is $c = (z + z_2)/2$; the labels are not used at any step.

Name/industry	Raw x	Residual e	z	z_2	Composite c
a/A	1	1	1.1180	1.1180	1.1180
b/A	-1	-1	-1.1180	0	-0.5590
c/A	0	0	0	-1.1180	-0.5590

Continued: the same complete example

d/B	11	1	1.1180	0	0.5590
e/B	10	0	0	-1.1180	-0.5590
f/B	9	-1	-1.1180	1.1180	0

Raw values retain their economic direction; residuals represent "relative to peers and controls"; z-scores put values on a comparable scale; and the composite is a ranking score. None is a dollar position yet. If f is missing the second factor, reweighting over the available factor gives $c_f = -1.1180$, while zero fill gives -0.5590. If the two factors represent different styles and the contract requires at least two, f's composite should be missing; reweighting must not be used to bypass that minimum. A missing score affects eligibility for alpha ranking; whether f may serve as an industry hedge coordinate belongs to the support-set contract at the next layer (Section 6.10).

Why do we need this chapter after defining fields in Chapters 24–26? Ratios, returns, and accounting quantities cannot be added directly; peer residuals, scale, coverage, and weights must be settled one layer at a time. Report the missing-name list, factor correlations, and position intersection together, but position similarity alone does not establish real crowding (Section 6.9).

4. Mathematical foundations

Notation. Date t is a month-end; stocks are $i = 1, \ldots, n_t$; raw signal $x_{i,t}$; next-period return label $y_{i,t+1}$ for the interval $t \to t+1$. A same-date transform means that the output in row t is calculated only from the inputs in row t. The valid-name set $\mathcal{V}_t$ contains names with a nonmissing signal and all nonmissing controls. Units: signals are dimensionless; returns are monthly decimals. Frequency: monthly; `signal_time` is month-end; the evaluation label is for the following month.

Same-date z-score. $z_{i,t} = (x_{i,t} - \bar{x}_t)/s_t$, where $\bar{x}_t$ and s_t are the sample mean and sample standard deviation (`ddof=1`) over $\mathcal{V}_t$. If $s_t = 0$ or $|\mathcal{V}_t| < 3$, the row is undefined (NaN).

Same-date winsorization. Take the day's quantiles $q_t^-, q^+{}_t$ over $\mathcal{V}_t$ (linear interpolation, such as the 1st and 99th percentiles), and set

$$\tilde{x}_{i,t} = \min\{\max\{x_{i,t}, q_t^-\}, q_t^+\}.$$

Ranks and normal scores. Use average rank $r_{i,t}$; percentile rank $(r-1)/(n-1)$ (Chapter 23); normal score $\Phi^{-1}\big((r - \frac{1}{2})/n\big)$.

Neutralization. For each date t, run cross-sectional OLS: $x_{i,t} = \sum_g \alpha_{g,t}\mathbf{1}\{i \in g\} + \gamma_t c_{i,t} + e_{i,t}$, where g indexes industries (include all categories' indicators and do not add a separate intercept), and c is a control such as log market capitalization. The neutralized signal is $e_{i,t}$.

Orthogonalization. Regress signal $x^{(2)}$ on $x^{(1)}$ within the same date, including an intercept, and take the residual: $e^{(2)} = x^{(2)} - a - bx^{(1)}$.

Combination. For standardized signals $z^{(k)}$ and weights w_k, $c_{i,t} = \sum_k w_k z_{i,t}^{(k)} / \sum_k |w_k|$ (when all factors are available).

Composite IC. Let the standardized signals have variance 1 and correlation matrix R ($K \times K$); let the label have variance 1 and correlation vector ρ with the signals. Then $\mathrm{Corr}(w^\top z, y) = w^\top \rho / \sqrt{w^\top R w}$. ρ and R must be **jointly feasible**: the augmented matrix $\begin{pmatrix} R & \rho \\ \rho^\top & 1 \end{pmatrix}$ must be a correlation matrix. If R is symmetric positive definite and invertible, the Schur complement makes this equivalent to $\rho^\top R^{-1} \rho \leq 1$ (the squared multiple correlation of the label with the signals; it also implies $|\rho_i| \leq 1$). For example, $R = I_2, \rho = (0.9, 0.9)$ is infeasible ($0.81 + 0.81 = 1.62 > 1$); plugging it into the formula would produce an "IC" greater than 1. The code rejects such input rather than clipping it.

Diversification of factor returns. For k factor returns with individual Sharpe ratio S, average pairwise correlation $\bar{\rho}$, and an equal-risk average: $S_p = S\sqrt{k} / \sqrt{1 + (k-1)\bar{\rho}}$.

5. Formal definitions

Same-date cross-sectional transform. The output for date t uses only inputs from date t. Winsorization, z-scoring, neutralization, orthogonalization, and combination in this chapter are all same-date transforms.

Time-series standardization. Standardize a signal's time series using its mean and standard deviation. A **full-sample fit** estimates the mean and standard deviation using all dates, including future dates; an **expanding** version uses only rows $\leq t$ for row t.

Winsorization. Move values outside a quantile interval to the nearest endpoint. This changes magnitude but preserves the ordering among names that were not winsorized.

Z-score and MAD-based robust z-score. The first uses the mean and standard deviation; the second uses the median and 1.4826× the median absolute deviation (MAD).

Normal scores. Use ranks to map any distribution to standard-normal quantiles. This discards raw magnitudes and is more robust to extreme magnitudes that do not change ranks within a fixed set of paired observations. Adding an outlier name, changing missingness, or introducing ties can still change the mapping; normal scores do not make Pearson IC immune to change.

Neutralization. Regress on controls such as industry indicators and size within the same date, then take residuals. **Industry-neutral** means that residuals sum to zero within every industry.

Orthogonalization. Regress on an existing signal and take residuals. Sequentially orthogonalizing multiple signals is a Gram–Schmidt procedure; the **order** determines what each signal retains.

Combining signals and choosing weights. Options include equal weights, fixed economic weights, weights proportional to discovery-period IC, and the IC-maximizing $R^{-1}\rho$. Except for the first two, these require estimation; the estimation period and evaluation period must be separate.

Missing factor. A name has a missing value for one factor. Treatments include **zero fill** (zero is the z-score mean), **reweight over available factors** (divide weights by the sum of absolute available-factor weights), **drop** (require all factors), and a **minimum factor count** (return NaN below the required count).

Factor correlation. Distinguish **signal correlation** (correlation between two signals in the same-date cross-section) from **return correlation** (time-series correlation between two factor-portfolio returns). The former affects composite IC; the latter affects diversification.

Crowding. The operational definition in this chapter is a situation where many investors hold similar positions, so returns fall together when a factor is forced to deleverage. It can be described using position overlap (the intersection share of same-date top-quintile sets) and return correlation during stress periods. This chapter reports only constructed overlap values and has no actual holdings data, so it does not draw conclusions about real crowding.

Constraints. Linear restrictions on portfolio weights, $Aw = b$ (dollar neutrality, industry neutrality, or zero size exposure). Chapter 21's `project_to_constraints` finds the feasible weights closest in Euclidean distance.

6. Derivations

6.1 How same-date transforms affect IC

For a daily cross-section of signal x and label y, $\mathrm{Corr}(ax + b, y) = \mathrm{Corr}(x, y)$ for $a > 0$ and any b. A same-date z-score therefore leaves that date's Pearson IC unchanged, and also leaves its Spearman IC unchanged. Its purpose is to put signals on comparable scales so that combination weights have comparable meaning. A strictly order-preserving transform preserves Spearman IC for the same set of valid signal/label pairs and a consistent tie rule; average ranks and normal scores must use the same tie `groups`. Pearson IC generally changes. Normal scores reduce the influence of extreme magnitudes, but adding names, changing the valid set, or changing ranks can still change results. Winsorization creates ties, so it may also change Spearman IC; it is not always "nearly unchanged." Pearson IC may change substantially as well.

6.2 Attenuation of IC by a disturbance

Suppose the observed signal in a daily cross-section is $x = s + u$, where s is the true signal (variance σ_s^2) and u is a disturbance (industry shift, size loading, outliers, etc.; variance σ_u^2). Assume they are independent, have zero means, and u is unrelated to label y. Then $\mathrm{Cov}(x, y) = \mathrm{Cov}(s, y)$ and $\mathrm{Var}(x) = \sigma_s^2 + \sigma_u^2$, so

$$\mathrm{Corr}(x, y) = \mathrm{Corr}(s, y)\, \frac{1}{\sqrt{1 + \sigma_u^2/\sigma_s^2}}.$$

When disturbance variance equals true-signal variance, IC falls to $1/\sqrt{2} = 0.71$ of its original value. In the constructed panel (Section 10), true-signal variance is 1. Disturbances are: industry shifts (one shift per date for each of 10 industries, with cross-sectional variance about 0.9); size loadings (value -0.5, momentum 0, quality 0.3, with size variance 1, giving variances $0.25, 0, 0.09$); and outliers (probability 2%, standard deviation 8, variance $0.02 \times 64 = 1.28$). The predicted attenuation in raw-signal IC is $1/\sqrt{1 + 0.9 + c^2 + 1.28}$: 0.540 for value, 0.561 for momentum, and 0.553 for quality. Section 10 measures 0.553, 0.592, and 0.576 across 60 new seeds, within 0.04 of the predictions. This shows that attenuation can be calculated in advance; it is not a post hoc explanation.

Winsorization removes most of the outlier variance; neutralization removes industry shifts and size loadings. This is the arithmetic behind "each step recovers some of the true IC."

The assumptions matter: the disturbance must be unrelated to the label. If an industry shift itself predicts returns, it is not noise, and neutralization will remove it. Whether to `neutralize` is a research hypothesis and must be written into the plan.

6.3 Algebra of neutralization

All industry indicators. Let D be the $n \times G$ industry-indicator matrix, with G categories and exactly one category per stock. The solution to $\min_\alpha \|x - D\alpha\|^2$ is the within-group mean, so the residual is $e_i = x_i - \bar{x}_{g(i)}$: demean within group. Residuals sum to zero within each industry.

With a size control. Let $\tilde{x}$ and $\tilde{c}$ be signal and size after demeaning within industry. The Frisch–Waugh–Lovell theorem says that the residual from regressing x simultaneously on D and c equals the residual from first removing the industry effect and then regressing $\tilde{x}$ on $\tilde{c}$:

$$e = \tilde{x} - \hat{\gamma}\,\tilde{c}, \qquad \hat{\gamma} = \frac{\sum_i \tilde{x}_i \tilde{c}_i}{\sum_i \tilde{c}_i^2}.$$

The displayed division requires $\sum_i \tilde{c}_i^2 > 0$. If it is zero, size is already in the industry column space and adds no explanatory direction; the residual remains $\tilde{x}$, without division by zero. All steps must use the same valid-pair set.

Thus e is orthogonal to size ($\sum_i e_i c_i = 0$), and sums to zero within each industry.

Invariance under a redundant design. Adding a constant column (the sum of all industry indicators) makes the design matrix rank deficient. The residual is the projection of x onto the orthogonal complement of the column space, so it depends only on the column space. Adding a redundant constant does not change that space; the residual is therefore independent of which column is dropped and of `lstsq`'s minimum-norm choice. (In the test, residuals with and without a constant control column differ by less than 10^{-10}.)

An industry with one stock. Its indicator uses a degree of freedom by itself, so its residual is always zero. If a date has too few valid names (fewer than 2 residual degrees of freedom), the row is NaN, not zero.

Relation to linear projection of weights. Let weights be $w = x$ (proportional to the signal), with constraint $Aw = 0$, where the rows of A are industry-indicator rows (possibly with an additional size-exposure row). Chapter 21's `project_to_constraints` computes $w - A^\top(AA^\top)^{-1}Aw$, which is exactly the OLS residual from regressing x on the columns of $A^\top$. Thus, for weights proportional to the signal, "`neutralize` the signal, then take positions proportional to it" and "take positions proportional to the signal, then project" produce the same vector. Quantile weights (top quintile $+1/m$, bottom quintile $-1/m$) are a nonlinear function of the signal: taking quantiles after signal neutralization and projecting after taking quantiles produce different portfolios (see Section 10). Note that the sum of all industry-indicator rows is the dollar row, so do not add a separate dollar row (`project_to_constraints` requires linearly independent rows).

6.4 Composite IC and orthogonalization order

Assume $|r_{12}| < 1$, so the residual variance is strictly positive, and that the label and signals are jointly feasible. For two standardized signals with label correlations ρ_1, ρ_2 and signal correlation r_{12}, the equal-weight composite $z_1 + z_2$ has covariance $\rho_1 + \rho_2$ and variance $2(1 + r_{12})$. Its IC is $(\rho_1 + \rho_2)/\sqrt{2(1 + r_{12})}$; when $r_{12} = 0$, this is Chapter 26's $(\rho_1 + \rho_2)/\sqrt{2}$.

Sequential orthogonalization. Keep z_1 first, then regress z_2 on z_1 and take the residual $e_2 = z_2 - r_{12}z_1$ (variance $1 - r_{12}^2$, uncorrelated with z_1). Standardize it to $\tilde{e}_2 = e_2/\sqrt{1 - r_{12}^2}$. Then $\mathrm{Cov}(\tilde{e}_2, y) = (\rho_2 - r_{12}\rho_1)/\sqrt{1 - r_{12}^2}$. Adding the two uncorrelated unit-variance signals gives

$$\mathrm{IC}_{1\to 2} = \frac{1}{\sqrt{2}}\Big[\rho_1 + \frac{\rho_2 - r_{12}\rho_1}{\sqrt{1 - r_{12}^2}}\Big].$$

Reversing the order gives $\mathrm{IC}_{2\to 1} = \frac{1}{\sqrt{2}}\big[\rho_2 + (\rho_1 - r_{12}\rho_2)/\sqrt{1 - r_{12}^2}\big]$. The two generally differ. For $\rho_1 = 0.04, \rho_2 = 0.02, r_{12} = 0.5$, the values are $\mathrm{IC}_{1\to 2} = 0.0283$, $\mathrm{IC}_{2\to 1} = 0.0386$, equal weight 0.0346, and optimal 0.0400 (below). In this example, $\rho_2 = r_{12}\rho_1$: all of the second factor's predictive power comes from its correlation with the first. Once the first is removed, the second contains no incremental information; putting the first factor first and orthogonalizing the second leaves only the latter's noise.

K **signals.** Require the correlation matrix to be positive definite, so each Cholesky residual variance is positive. Using the Cholesky factorization $PRP^\top = LL^\top$ (where P permutes the order), the unit residuals from Gram–Schmidt are $L^{-1}Pz$, and their correlations with the label are $L^{-1}P\rho$. Thus the IC of their equal-weight combination after sequential orthogonalization is $\mathbf{1}^\top L^{-1}P\rho/\sqrt{K}$ (`sequential_orthogonal_ic`).

IC-maximizing weights. Maximize $w^\top\rho/\sqrt{w^\top Rw}$. The Lagrange condition is $\rho = \lambda Rw$, so $w^* \propto R^{-1}\rho$ and the maximum IC is $\sqrt{\rho^\top R^{-1}\rho}$. In the example, $R^{-1}\rho = (0.04, 0)$, which is proportional to $(0.04 - 0.5 \times 0.02,\ 0.02 - 0.5 \times 0.04) = (0.03, 0)$. Thus, after normalizing, all weight goes to the first factor and the IC is $0.04 = \rho_1$. This requires positive-definite R, nonzero ρ, and joint feasibility with the label. If $\rho = 0$, every nonzero direction has IC zero, so there is no unique optimal direction and the zero vector cannot be normalized; the API raises an error. The true optimum formula uses true values; estimating them requires independent discovery-period data.

Interpretation. There is no "correct" order. Choosing the order with the highest evaluation-period IC is multiple testing (Chapter 22). A defensible approach is to state in advance that the economically most basic factor comes first and to record all order variants in the ledger.

6.5 Weight choice and tuning optimism

Equal weights require no estimation. Weights proportional to discovery-period IC, $\hat{w}_k \propto \max(\widehat{\mathrm{IC}}_k, 0)$, require estimating each factor's mean IC. With T months and n names per month, the standard error of mean IC is about $1/\sqrt{nT}$ (for independent dates; each day's IC is approximately $1/\sqrt{n}$). At $n = 300, T = 60$, this is 0.0075. In the constructed panel, true ICs for value, momentum, and quality are 0.014, 0.049, and 0.033. Estimation noise is as large as value's true IC, so estimated weights can randomly push it to zero or inflate it. Choosing the best performer by the same discovery-period measure is optimistic in expectation across repeated samples due to selection bias; an individual realization need not be optimistic. The IC-proportional rule also does not exactly maximize discovery-period composite IC and is not guaranteed to beat equal weights in a holdout. Across 60 new worlds in Section 10, IC-proportional discovery weights beat equal weights in the holdout in only 31.7% of worlds; the "best discovery-period single factor" did so in 3.3%.

6.6 Missing factors: whose scale changed?

Suppose there are K standardized, equally weighted signals. A name has k available factors. Here $\bar{r}$ is the average pairwise correlation within that available subset, and we assume the subset signals still have unit variance in the conditional distribution under discussion. If missingness depends on signal value or label, the complete-population moments below cannot be carried over unchanged.

Reweight over available factors: $c = \frac{1}{k}\sum_{j\in\text{available}} z_j$, with variance $\frac{1}{k^2}[k + k(k-1)\bar{r}] = \frac{1+(k-1)\bar{r}}{k}$. When $\bar{r} = 0$, this is $1/k$; when $\bar{r} < 0$, it lowers the average's variance further than the independent case. For example, with $k = 2$ and $\bar{r} = -0.5$, the variance is 0.25, below 0.5 under independence. The correlation structure must be feasible and the resulting variance nonnegative.

Zero fill: $c = \frac{1}{K}\sum_{j\in\text{available}} z_j$, with variance $\frac{k}{K^2}[1 + (k-1)\bar{r}]$.

For the constructed panel's three factors, $\bar{r} = (-0.3 - 0.3 + 0.2)/3 = -0.133$. Names with all three factors have composite variance $(1 + 2 \times (-0.133))/3 = 0.244$ (standard deviation 0.49). A name with only one factor has variance 1 under reweighting (standard deviation 1, or 2.0 times the complete-name standard deviation) and variance 1/9 under zero fill (standard deviation 0.33, or 0.67 times the complete-name standard deviation). Without restandardizing, reweighting gives names with fewer factors greater variance, while zero fill compresses their scale. Variance alone cannot establish tail probabilities for arbitrary distributions. The tail shares below are observed in this construction under its specified missingness mechanism. In Section 10, these names are 5.7% of valid names; under reweighting they account for 8.8% of the top quintile, and under zero fill for 3.5%.

Missingness mechanism. Under missing completely at random (MCAR), the scale comparison above applies to the corresponding subset moments. If missingness depends on the signal (for example, loss-making companies lack a profitability ratio), conditional moments may also change; bias is not necessarily mild or larger. This chapter simulates only random missingness. Recommended practice: set a minimum number of factors (such as 2), report coverage, run sensitivity checks for both treatments, and record variants in the ledger.

6.7 Look-ahead in time-series standardization

Standardizing each stock's signal time series using the full-sample mean and standard deviation makes row t depend on data after t. Counterexample: for a stock's signal sequence $(1, 2, 3, 10)$, at the third observation $t = 3$, the past-only expanding version (mean 2, standard deviation 1) gives $z = (3 - 2)/1 = +1.0$; the full sample (mean 4, standard deviation 4.08) gives $z = (3 - 4)/4.08 = -0.245$. The sign flips solely because the future value 10 enters the full-sample mean. Rule: every transform in row t uses only rows $\leq t$. Test by changing the last row; earlier outputs must not change (`expanding_zscore` has this test). Same-date cross-sectional transforms use only the current row, but point-in-time eligibility also requires that the input version and policy for that row were known at the time.

6.8 Factor correlation, diversification, and unstable correlations

For an equal-risk average of k factors, each with Sharpe ratio $S = \mu/\sigma$ and average pairwise correlation $\bar{\rho}$, the portfolio mean is μ (the equal-weight average), and variance is $\sigma^2[1+(k-1)\bar{\rho}]/k$. Therefore,

$$S_p = \frac{\mu}{\sigma\sqrt{[1 + (k-1)\bar{\rho}]/k}} = \frac{S\sqrt{k}}{\sqrt{1 + (k-1)\bar{\rho}}}.$$

When $\bar{\rho} = 1$, $S_p = S$ (no diversification); when $\bar{\rho} = 0$, it is $S\sqrt{k}$. The correlation matrix must be positive semidefinite, so $\bar{\rho}$ cannot be below $-1/(k-1)$. At that lower bound, portfolio variance is zero and the Sharpe ratio is undefined (`diversified_sharpe` requires a value strictly above the bound). Examples for $S = 0.3, k = 3$: $\bar{\rho} = 0$ gives 0.520; 0.2 gives 0.439; 0.8 gives 0.322.

If the factors have equal risk but different Sharpe ratios, replace S in the numerator with their average Sharpe ratio; this requires component standard deviations to be equal under the same evaluation convention. Scaling by past volatility does not guarantee equal realized risk in the future evaluation sample.

Correlation is estimated. Under an i.i.d. approximately bivariate-normal model and a large-sample approximation, the standard error of the sample correlation from T monthly observations is about $(1-\rho^2)/\sqrt{T}$ when the true correlation is ρ. At $T = 60, \rho = 0$, this is 0.13. A correlation matrix estimated from 60 months can therefore move substantially; in Section 11, HML and RMW (published value and quality factors) have opposite signs in 1963–1999 and 2000–2026.

6.9 Crowding: definition and what can be measured

Measuring crowding requires knowing how much capital holds similar positions, which calls for holdings, short-borrow rates, fund flows, and similar data. This chapter has none of those, so it can do only two things: (i) measure signal-level position overlap as the intersection share of two factors' same-date top-quintile names; under independence this should be 0.20; and (ii) measure return-level synchronicity using factor-return correlations and rolling correlations. The first is calculated in the constructed panel (value/momentum 0.10, value/quality 0.10, momentum/quality 0.28); the second uses published factor returns (Section 11). Neither establishes "crowding"; each is only one observable aspect of it.

6.10 Projection on the support set: who is neutral, and when?

The **support set** $\mathcal{S}_t = \{i : \text{listed}_{i,t} = \text{True}\}$ contains names listed at signal month-end. The **alpha-valid set** is the subset with nonmissing composite scores; the two are not the same. First form fixed top-quintile long-short weights w^0 on the alpha-valid set; weights outside the support set must be zero. An optional industry projection may use names in $\mathcal{S}_t$ that have no valid score to carry hedge weights. The resulting portfolio is no longer a pure top/bottom-score-quintile portfolio, and that must be disclosed.

Let $u = w^0_{\mathcal{S}_t}$, and let A_t contain industry-indicator rows only for names in that support set, with every name assigned to exactly one nonmissing industry. Construct only independent rows; their sum is already the net-position row, so do not add it again. If each row has at least one coordinate, $A_t A_t^\top$ is a positive diagonal matrix of industry counts. The support-set form of Chapter 21's nearest-point projection is

$$v = \arg\min_{A_t v = 0} \|v - u\|^2 = u - A_t^\top (A_t A_t^\top)^{-1} A_t u.$$

Projecting the full-column weights first and then zeroing unlisted names generally differs from this formula: masking deletes coordinates that used to offset industry net positions. If an

industry has only one listed name, its constraint sets that name's weight to zero; this is not rank deficiency of the full matrix, but the absence of any nonzero hedge direction in that industry. An empty support set, no initial long-short portfolio, or a numerically zero projection must explicitly result in cash. Missing classifications or misaligned axes must fail explicitly; do not silently replace invalid inputs with zero.

Gross exposure and homogeneous constraints. If $g_v = \sum_{i \in \mathcal{S}_t} |v_i|$ is above the numerical-zero threshold, set $k = 2/g_v > 0$, $w_{\mathcal{S}_t} = kv$, and all other coordinates to zero. Then $A_t(kv) = kA_t v = 0$; industry net positions and total net position remain zero, gross exposure returns to 2, long exposure is 1, and short exposure is -1. If g_v is only rounding noise, do not amplify it to 2. Projection gives the nearest point; **the final result after rescaling is generally no longer the original nearest point**. Restoring gross changes the position budget, not the return parameters. For a nonzero target $A_t v = b$, $A_t(kv) = kb$; unless $k = 1$ or $b = 0$, this proof does not apply. Current P4 adds no nonzero-target strategy.

Two thresholds. P4's min_names_per_date=60 counts $|\mathcal{S}_t|$, not the number of nonmissing values in each field. With 59 names, the date is ineligible for preprocessing and the portfolio holds cash. Sixty names opens universe eligibility only: a field still needs 30 values for winsorization, at least 10 residual values for a z-score, at least one field within a style, and at least two styles across the composite. No later combination may restore a date that failed eligibility; the portfolio entry point checks the 60-name requirement again so externally supplied scores cannot bypass it.

Clock and residuals. At month-end t, use then-listed status and signals to form a target. At $t + 1$, delisting cancellations or liquidations, partial fills, and price drift can all cause actual holdings to differ from the target. Report separately: target weights outside the signal-time support set, industry net positions, total net, and gross; the same quantities after execution-target filtering; and actual closing holdings normalized by current NAV. Post-execution information may be used for execution and diagnosis, but cannot be written back into the month-t scores or support set. Current P4 listing status is a constructed input; it has no real halt status, name-level price-availability, or borrow fields. A name marked listed is not necessarily truly tradable.

7. Financial interpretation

Preprocessing removes components unrelated to returns; it is not signal beautification. At each step, ask whether the removed component is really noise. Industry shifts, size loadings, and outliers are noise in the constructed panel, so each step recovers some IC (single-world momentum Pearson IC: 0.027 raw, 0.038 after winsorization, 0.048 after neutralization, versus true value 0.053). If a "disturbance" actually carries return information (real industry momentum or a size effect), preprocessing removes it.

Combination gains come from diversification, not magic. The benefit of adding two correlated factors is limited by their correlation; the composite-IC formula shows why. If all of one factor's predictive power comes from its correlation with another, it adds no incremental information.

Order and weights are researcher degrees of freedom. Orthogonalization order, weight choice, missing-factor treatment, and winsorization quantiles can all look "better" or "worse" in the evaluation period. Fix these choices in advance, record all variants, and evaluate tuning on an independent holdout.

Neutralization and constraints. For linear weights, signal neutralization and weight projection agree; for quantile weights, they differ. Therefore, "industry neutral" must say which operation was used: `neutralize` the signal and then rank, or rank and then project the weights. The two have different exposures, volatility, and turnover (Section 10).

Correlation is unstable. The correlation between published value and quality factors has opposite signs in different 20–30-year periods. Choosing weights from an estimated correlation matrix therefore relies on an estimate that can change sign. The diversification arithmetic (the S_p formula) is conditional on average correlation, which itself changes.

8. Numerical examples: work through each step

All values are produced by `hand_examples()` and independently checked by `qa/a5/claude/recompute/ch28_recompute.py`. The stocks and signals are teaching constructions.

8.1 Winsorization, z-scores, and normal scores

On one date, 11 stocks have signals $1, 2, \ldots, 10, 100$ (the last is an outlier).

- **Z-scores without winsorization:** mean $155/11 = 14.09$, standard deviation 28.64. The outlier's $z = (100 - 14.09)/28.64 = 3.00$; the other ten are compressed between -0.457 (signal 1) and -0.143 (signal 10), making them hard to distinguish.
- **Winsorization (10th and 90th percentiles, linear interpolation):** $(n-1)q = 10 \times 0.1 = 1$ and 9, which are the 2nd and 10th order statistics, so $q^- = 2$ and $q^+ = 10$. Values become $2, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10$, with mean $66/11 = 6$. The sum of squared deviations is $16 + 16 + 9 + 4 + 1 + 0 + 1 + 4 + 9 + 16 + 16 = 92$; sample variance is $92/10 = 9.2$ and standard deviation is 3.033. Signals 10 and 100 both have $z = +1.319$; signal 5 has $z = -0.330$, slightly below its un-winsorized value of -0.317, but the ten ordinary names now span -1.32 to $+1.32$.
- **Normal scores:** $\Phi^{-1}((r-1/2)/11)$. Signal 100 (rank 11) gets $\Phi^{-1}(10.5/11) = 1.691$; signal 10 (rank 10) gets $\Phi^{-1}(9.5/11) = 1.097$. Magnitudes are discarded; only ranks remain.

8.2 Industry neutralization: reversing the sign

There are six stocks: industry A is $\{a, b, c\}$ and industry B is $\{d, e, f\}$. Signal $x = (1, -1, 0,\ 11, 10, 9)$ and next-month returns $y = (5\%, 3\%, 4\%,\ -3\%, -4\%, -5\%)$. Industry B has a higher overall signal (shift of 10) and lower returns, while within each industry a higher signal corresponds to a higher return.

- **Raw Pearson IC:** $\bar{x} = 30/6 = 5$, $x - \bar{x} = (-4, -6, -5, 6, 5, 4)$, $\sum(x - \bar{x})y = -1.16$, $\sum(x - \bar{x})^2 = 154$, and $\sum y^2 = 0.01$. Thus IC $= -1.16/\sqrt{154 \times 0.01} = -0.935$.
- **After neutralization:** industry means are 0 and 10, so residuals are $e = (1, -1, 0,\ 1, 0, -1)$ and sum to zero within each industry. $\sum ey = 0.04$, $\sum e^2 = 4$, so IC $= 0.04/\sqrt{4 \times 0.01} = +0.200$.

The industry shift is unrelated to the within-industry ordering but points opposite to the industry return. Before neutralization, IC is -0.94; afterward it is $+0.20$.

8.3 Linear-constraint projection and neutralization produce the same vector

Use x from 8.2 again, with weights $w = x$. Let the two rows of constraint matrix A be the indicators for industries A and B (their sum is the dollar row, so do not add it again). $AA^\top = \text{diag}(3, 3)$, $Ax = (0, 30)$, and $w - A^\top(AA^\top)^{-1}Ax = x - (0, 0, 0, 10, 10, 10) = (1, -1, 0, 1, 0, -1)$. This matches the 8.2 residual exactly. After projection, total net position and each industry's net position are zero. This holds for weights proportional to the signal, not quantile weights. In Section 10, the mean maximum industry net position is 0.053 after neutralizing the signal and taking quintiles, but zero after taking quintiles on the raw composite and projecting the weights.

8.4 Two correlated signals: equal weights, two orthogonalization orders, and the optimum

Complete worked example: four ways to combine correlated signals

Let $\rho_1 = 0.04,\ \rho_2 = 0.02,\ r_{12} = 0.5$ (Section 6.4).

Combination	IC
Use z_1 only	0.0400
Equal-weight $z_1 + z_2$	$0.06/\sqrt{2 \times 1.5} = 0.0346$
First z_1, then standardized residual of z_2 on z_1	$\frac{1}{\sqrt{2}}[0.04 + (0.02 - 0.02)/0.866] = 0.0283$
First z_2, then standardized residual of z_1 on z_2	$\frac{1}{\sqrt{2}}[0.02 + (0.04 - 0.01)/0.866] = 0.0386$
Optimal weights $\propto R^{-1}\rho = (0.04, 0) \to (1, 0)$	$\sqrt{\rho^\top R^{-1}\rho} = 0.0400$

Order moves IC from 0.0283 to 0.0386, a 36% difference; equal weights lie between them. The optimum equals the first factor alone because $\rho_2 = r_{12}\rho_1$: all the second factor's predictive power comes from its correlation with the first.

8.5 Missing factors

Two factors have weights $(0.5, 0.5)$. Name s_1 has both factors, $z = (2, 2)$, and composite 2.0. Name s_2 has only the second factor, $z = 1$:

- Reweight over available factors: $0.5 \times 1/0.5 = 1.0$ (preserves the z-score scale).
- Zero fill: $0.5 \times 1/1.0 = 0.5$ (shrunk toward 0).

Section 6.6 gives the broader example: for three factors with average correlation -0.133, complete names have composite standard deviation 0.49; names with one factor have standard deviation 1 after reweighting (2.0 times larger) and 0.33 after zero fill (0.67 times as large).

8.6 Look-ahead in standardization

A stock's signal sequence is $(1, 2, 3, 10)$. At the third observation, using only the past $\{1, 2, 3\}$ (mean 2, standard deviation 1) gives $z = +1.0$; the full sample (mean 4, standard deviation 4.082) gives $z = (3 - 4)/4.082 = -0.245$. The sign reverses because the full-sample mean uses the future value 10.

8.7 Diversification

Three factors each have Sharpe ratio 0.3. Average correlation 0 gives $0.3\sqrt{3} = 0.520$; 0.2 gives $0.520/\sqrt{1.4} = 0.439$; 0.8 gives $0.520/\sqrt{2.6} = 0.322$. The correlation matrix $\begin{pmatrix} 1 & 0.2 & 0.4 \\ 0.2 & 1 & 0 \\ 0.4 & 0 & 1 \end{pmatrix}$ has average pairwise correlation $(0.2 + 0.4 + 0)/3 = 0.2$.

8.8 Three-coordinate support set in a four-name universe: counterexample to project-then-mask

There are four names a, b, c, d; industry A is a, b, and industry B is c, d. The weights $w = (0.4, -0.4, 0.3, -0.3)$ are neutral over the full set and have gross exposure 1.4. Suppose a is not listed at signal time. Projecting as before (the weights do not change), then zeroing a, gives $(0, -0.4, 0.3, -0.3)$: industry A net -0.4, total net -0.4, and gross 1.0. An unlisted name is not a negligible implementation detail.

The correct support set is (b, c, d), with original vector $u = (-0.4, 0.3, -0.3)$:

$$A_t = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \quad A_t A_t^\top = \operatorname{diag}(1, 2), \quad A_t u = (-0.4, 0).$$

Subtracting the industry means gives $v = (0, 0.3, -0.3)$, gross 0.6. Multiplying by $2/0.6 = 10/3$ gives final weights $(0, 0, 1, -1)$ (embedded back into the full column set). The unlisted a has weight zero; the sole name in industry A, b, is also forced to zero; industry B net and total net are zero; gross is 2. The budget is restored, but the original four quantile positions are not preserved.

Now consider a missing-alpha hedge coordinate. All four names are listed, but only a and c have scores, with initial long-short weights $(1, 0, -1, 0)$. The two industry means are 0.5 and -0.5, so projection gives $v = (0.5, -0.5, -0.5, 0.5)$, already with gross 2. Names b and d have no score but carry hedge positions. This is the declared optional projection semantics; it does not invent alpha values for missing names. If only a and c are listed and each is the sole name in its industry, projection is zero and the portfolio holds cash; do not divide by zero or scale noise to "restore gross."

For contrast, use a nonzero target: industry A has one coordinate and requires $v_a = 0.2$. Even if the projection satisfies the constraint, multiplying by 2 changes the target to 0.4. A nonzero target requires resolving the constrained problem; it cannot be scaled without qualification. All calculations in this section are independent hand examples, not new market results.

9. Python implementation

Functions are in the new module `factor_preprocessing.py`; ranking, IC, and quantile weights reuse Chapter 23's `factor_research.py`; linear-constraint projection reuses Chapter 21's `project_to_constraints`.

Function	Input	Output and limits
`winsorize_cross_section`	Signal panel, lower and upper quantiles, minimum name count	Winsorize each row to its quantile interval; if valid names are below the minimum, return an all-NaN row; quantiles must satisfy $0 \le$ lower $<$ upper ≤ 1
`zscore_cross_section`	Panel, variance divisor adjustment (`ddof=0` or `1`), minimum name count	Compute $(x - \bar{x})/s$ row-wise; rows below the minimum or with zero standard deviation are NaN
`mad_zscore_cross_section`	Panel	Compute $(x - \text{median})/(1.4826\ \text{MAD})$; rows with zero MAD are NaN
`normal_scores`	Panel	Compute $\Phi^{-1}((r - 1/2)/n)$ using average ranks
`neutralize`	Signal, industries (static or date-indexed table), control dictionary	Same-date OLS residuals; a name missing any input is NaN; use the valid pair set for the signal/all controls/group; a date with $n - \text{rank}(X) < 2$ is all NaN; if the date has enough degrees of freedom, a singleton-industry residual is zero
`orthogonalize`	Signal, list of panels to regress on	Same-date residual after regression on an intercept and those panels
`combine_signals`	Signal dictionary, weight dictionary, minimum factor count, missing='reweight' or 'zero'	Composite panel and count of available factors with nonzero weight per name (`min_factors` counts only these); weights must be finite real numbers and not all zero
`expanding_zscore`	Panel, minimum periods	Each row uses mean and standard deviation from rows $\le t$

Function	Input	Output and limits
`composite_ic`	Correlation vector with label, correlation matrix, weights	Compute $w^\top \rho / \sqrt{w^\top R w}$; R must be symmetric, have unit diagonal, and be positive definite; also require $\rho^\top R^{-1} \rho \leq 1$ (joint feasibility), otherwise raise an error
`optimal_composite_weights`	Same inputs	Return $R^{-1}\rho$ (normalized to absolute weights summing to 1) and its IC; error if jointly infeasible or all entries of ρ are zero (no unique optimal direction)
`sequential_orthogonal_ic`	Same inputs plus order	IC of the equal-weight composite after sequential orthogonalization
`diversified_sharpe`, `average_pairwise_correlation`	Sharpe ratio, factor count, average correlation; or correlation matrix	Compute $S\sqrt{k}/\sqrt{1+(k-1)\bar{\rho}}$ (error if portfolio variance $1+(k-1)\bar{\rho}$ is nonpositive, including the tolerance band at the lower bound); or mean of off-diagonal elements

The default offline reading entry point is the accepted reader package's `notebooks/ch28_multifactor.ipynb`: `RUN_FRENCH=False`; it constructs 300 names by 120 months (seed 20261121), and the default tuning sweep uses 60 worlds. Create a new `RUN_DIR` only; do not write historical plots/CSV/JSON. The French analysis and four figures in this chapter are existing historical evidence and are not rerun by the default path. The runner also saves an execution copy and state. Code blocks are retained in full for study; the following are historical CLI/test entry points from the original project, not reader defaults. Do not run them when the isolated package does not include these project entry points:

```
.venv/bin/python code/ch28_multifactor.py
.venv/bin/python -m pytest tests/test_ch28_preprocessing.py tests/test_ch28_workflow.
    ↪ py -q
```

Frequency: monthly; `signal_time` is month-end; each row t of same-date transforms uses only row t; `expanding_zscore` for row t uses only rows $\leq t$. Seeds: constructed panel 20261121; 60 new worlds starting at 20261122.

Code block one: winsorization, z-scores, and normal scores (Section 8.1).

```
import numpy as np
import pandas as pd

from quant_course.factor_preprocessing import normal_scores, winsorize_cross_section,
    ↪  zscore_cross_section

x = pd.DataFrame([[1.0, 2, 3, 4, 5, 6, 7, 8, 9, 10, 100]], index=pd.period_range("
    ↪ 2026-06", periods=1, freq="M"))
w = winsorize_cross_section(x, lower=0.1, upper=0.9, min_names=10)
assert w.iloc[0].tolist() == [2, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10]
assert abs(w.iloc[0].mean() - 6.0) < 1e-12 and abs(w.iloc[0].std(ddof=1) - np.sqrt
    ↪ (9.2)) < 1e-12
z_raw, z_win = zscore_cross_section(x), zscore_cross_section(w)
```

```
assert abs(z_raw.iloc[0, 10] - 3.0) < 0.001 and z_raw.iloc[0, :10].between(-0.46,
    ↪ -0.14).all()   # the ten ordinary names are squeezed together
assert abs(z_win.iloc[0, 9] - 4 / np.sqrt(9.2)) < 1e-12 and abs(z_win.iloc[0, 9] -
    ↪ z_win.iloc[0, 10]) < 1e-12
ns = normal_scores(x)
assert abs(ns.iloc[0, 10] - 1.6906) < 1e-4 and abs(ns.iloc[0, 9] - 1.0968) < 1e-4
```

Code block two: industry neutralization and linear-constraint projection (Sections 8.2 and 8.3).

```
import numpy as np
import pandas as pd

from quant_course.factor_preprocessing import neutralize
from quant_course.portfolio_construction import project_to_constraints

cols = list("abcdef")
sig = pd.DataFrame([[1.0, -1.0, 0.0, 11.0, 10.0, 9.0]], index=pd.period_range("
    ↪ 2026-06", periods=1, freq="M"), columns=cols)
ret = np.array([0.05, 0.03, 0.04, -0.03, -0.04, -0.05])
groups = pd.Series(list("AAABBB"), index=cols)
res = neutralize(sig, groups=groups)
assert np.allclose(res.iloc[0], [1, -1, 0, 1, 0, -1], atol=1e-12)
assert abs(np.corrcoef(sig.iloc[0], ret)[0, 1] + 0.93475) < 1e-5 and abs(np.corrcoef(
    ↪ res.iloc[0], ret)[0, 1] - 0.2) < 1e-12
A = np.array([[1, 1, 1, 0, 0, 0], [0, 0, 0, 1, 1, 1]], dtype=float)         #
    ↪ industry indicator rows; their sum is the dollar row
proj = project_to_constraints(sig.iloc[0].to_numpy(), A)
assert np.allclose(proj, res.iloc[0], atol=1e-12) and abs(proj.sum()) < 1e-12 and np.
    ↪ allclose(A @ proj, 0)
```

Code block three: combining two correlated signals and order dependence (Section 8.4).

```
import numpy as np

from quant_course.factor_preprocessing import composite_ic, optimal_composite_weights
    ↪ , sequential_orthogonal_ic

R = np.array([[1.0, 0.5], [0.5, 1.0]])
rho = [0.04, 0.02]
assert abs(composite_ic(rho, R) - 0.06 / np.sqrt(3)) < 1e-12
a, b = sequential_orthogonal_ic(rho, R, [0, 1]), sequential_orthogonal_ic(rho, R, [1,
    ↪  0])
assert abs(a - 0.04 / np.sqrt(2)) < 1e-12 and abs(b - (0.02 + 0.03 / np.sqrt(0.75)) /
    ↪  np.sqrt(2)) < 1e-12 and b > a
w, best = optimal_composite_weights(rho, R)
assert np.allclose(w, [1.0, 0.0]) and abs(best - 0.04) < 1e-12 and abs(composite_ic(
    ↪ rho, R, w) - 0.04) < 1e-12
assert best > b > composite_ic(rho, R) > a                                   #
    ↪ 0.0400 > 0.0386 > 0.0346 > 0.0283
```

Code block four: missing-factor scale (Sections 8.5 and 6.6).

```
import numpy as np
import pandas as pd

from quant_course.factor_preprocessing import combine_signals

idx = pd.period_range("2026-06", periods=1, freq="M")
z = {"f1": pd.DataFrame([[2.0, np.nan]], index=idx, columns=["s1", "s2"]), "f2": pd.
    ↪ DataFrame([[2.0, 1.0]], index=idx, columns=["s1", "s2"])}
rw, count = combine_signals(z, {"f1": 0.5, "f2": 0.5}, missing="reweight")
zero, _ = combine_signals(z, {"f1": 0.5, "f2": 0.5}, missing="zero")
assert rw.iloc[0].tolist() == [2.0, 1.0] and zero.iloc[0].tolist() == [2.0, 0.5] and
    ↪ count.iloc[0].tolist() == [2, 1]

rng = np.random.default_rng(0)
R = np.array([[1.0, -0.3, -0.3], [-0.3, 1.0, 0.2], [-0.3, 0.2, 1.0]])
full = rng.multivariate_normal(np.zeros(3), R, size=300_000)                    #
    ↪ three standardized factors, mean correlation -0.133
var3 = full.mean(axis=1).var()
assert abs(var3 - (1 + 2 * (-0.1333)) / 3) < 0.005                              #
    ↪ (1 + (k-1) r) / k with k = 3
reweighted_one, zero_filled_one = full[:, 0], full[:, 0] / 3                    # a
    ↪  name that has only the first factor
assert abs(reweighted_one.var() / var3 - 1 / 0.2444) < 0.15 and abs(zero_filled_one.
    ↪ var() / var3 - (1 / 9) / 0.2444) < 0.02
```

Code block five: look-ahead in standardization (Section 8.6).

```
import pandas as pd

from quant_course.factor_preprocessing import expanding_zscore

x = pd.DataFrame({"x": [1.0, 2.0, 3.0, 10.0]}, index=pd.period_range("2026-01",
    ↪ periods=4, freq="M"))
past = expanding_zscore(x, min_periods=3)
full = (x["x"] - x["x"].mean()) / x["x"].std(ddof=1)
assert abs(past["x"].iloc[2] - 1.0) < 1e-12 and abs(full.iloc[2] + 0.244949) < 1e-6
    ↪  # opposite signs at the third observation
changed = x.copy(); changed.iloc[3, 0] = -100.0
    ↪   # change only the last row
assert expanding_zscore(changed, min_periods=3)["x"].iloc[:3].equals(past["x"].iloc
    ↪ [:3])
```

Code block six: diversification (Section 8.7).

```
import numpy as np

from quant_course.factor_preprocessing import average_pairwise_correlation,
    ↪ diversified_sharpe

assert abs(diversified_sharpe(0.3, 3, 0.0) - 0.3 * np.sqrt(3)) < 1e-12
assert abs(diversified_sharpe(0.3, 3, 0.2) - 0.43916) < 1e-5 and abs(
    ↪ diversified_sharpe(0.3, 3, 0.8) - 0.32225) < 1e-5
assert abs(average_pairwise_correlation(np.array([[1, 0.2, 0.4], [0.2, 1, 0.0], [0.4,
    ↪  0.0, 1]])) - 0.2) < 1e-12
assert abs(diversified_sharpe(0.3, 3, 1.0) - 0.3) < 1e-12                          #
    ↪ perfectly correlated: no diversification
```

10. Reading the figures

Constructed panel. 300 stocks, 120 months (119 months with labels), seed 20261121. Three true signals (value, momentum, quality), with same-date correlations $-0.3, -0.3, 0.2$. Next-month returns load on the true signals by $(0.0030, 0.0040, 0.0025)$, plus a common industry shock (standard deviation 2%) and idiosyncratic noise (7%). Observed raw signals equal true signals plus industry shifts (for every factor and industry, an AR(1) process with coefficient 0.9 and variance 1), size loadings $(-0.5, 0, 0.3)$, and outliers with probability 2% and standard deviation 8. The analytic correlations between true signals and labels are 0.0144 for value, 0.0493 for momentum, and 0.0329 for quality (return standard deviation 7.30%). This is a known data-generating process, not market evidence.

Figure 28-1: Winsorization and stepwise recovery of IC. Left: histogram (log vertical axis) of one day's value signal; orange is raw (with outliers at -12 and $+6$), blue is winsorized at the same-date 1st/99th percentiles. Right: mean monthly IC over five treatments for each factor (119 months): raw Pearson, winsorized Pearson, winsorized plus neutralized Pearson, winsorized plus neutralized Spearman, and true-signal Pearson. Momentum rises from 0.027 to 0.038 to 0.048, versus true value 0.053. Quality is 0.025, 0.028, and 0.031, versus true value 0.032. Value's IC is small (true value 0.014; sampling error in one world is about 0.005), so its trend is not visible in one world; see the average over 60 new seeds below.

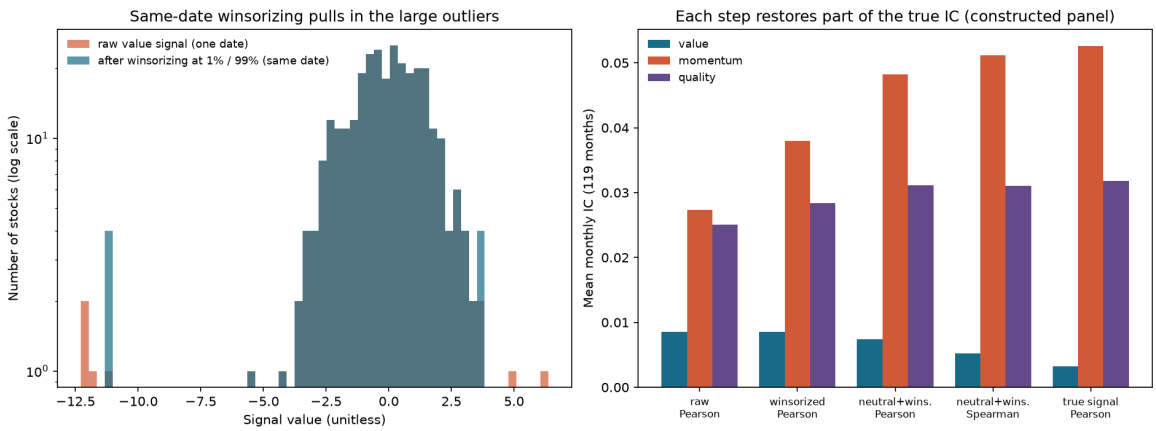


Figure 28-1: Winsorization and stepwise IC recovery

Mean Pearson IC and attenuation across 60 new worlds (raw, after winsorization, and after neutralization plus winsorization, relative to the true signal).

Factor	True-signal IC	Raw / true	Winsorized / true	Neutralized and winsorized / true	Raw / true predicted in 6.2
Value	0.0159	0.553	0.629	0.878	0.540
Momentum	0.0496	0.592	0.692	0.881	0.561
Quality	0.0327	0.576	0.676	0.859	0.553

Figure 28-2: Cross-section before and after neutralization. Left: one day's winsorized value signal plotted by industry. Right: residuals after neutralizing industry and size. Orange horizontal lines show the same-day industry means. On the left, industry means are visibly separated (industry shifts explain much of the signal variation); on the right, each industry's mean is zero (the largest absolute industry mean on one date is 1.9×10^{-15}, and residual/size correlation is -1.0×10^{-17}). Several points remain at -9 to -11: these are outlier stocks that were winsorized at -11.3 before neutralization, so they remain outliers afterward. Removing them would require a second winsorization after neutralization, which is another variant to record in the ledger.

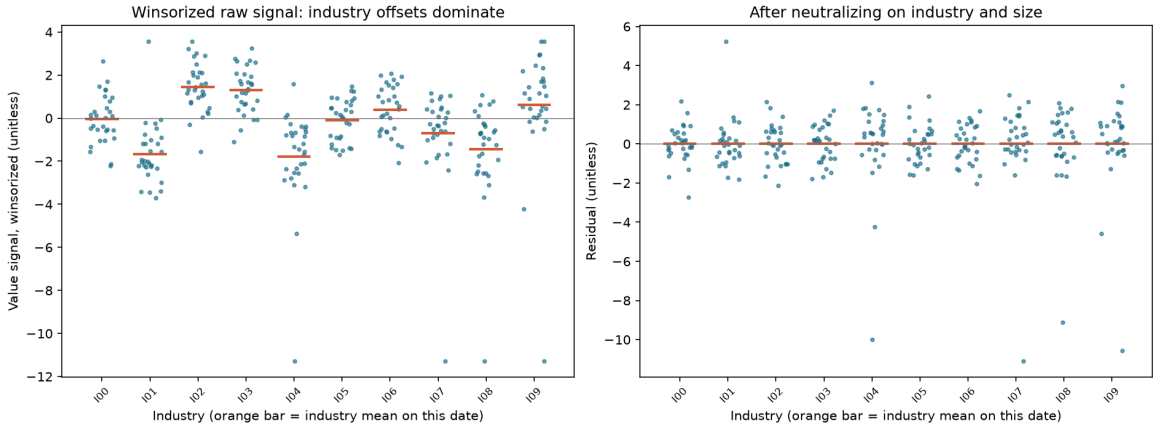


Figure 28-2: Industry cross-section before and after neutralization

Combination (Pearson IC, 119 months; values in parentheses are analytic values using the true signals and true correlation matrix).

Combination	Measured Pearson IC	Measured Spearman IC	Analytic IC
Equal weight	0.0562	0.0519	0.0651
True-loading weights ("oracle" after the fact)	0.0582	0.0548	0.0665
Sequentially `orthogonalize` value → momentum → quality	0.0500	0.0467	0.0595
Sequentially `orthogonalize` quality → momentum → value	0.0577	0.0532	0.0661

Combination	Measured Pearson IC	Measured Spearman IC	Analytic IC
Sequentially `orthogonalize` momentum → value → quality	0.0553	0.0511	0.0649
Sequentially `orthogonalize` momentum → quality → value	0.0563	0.0518	0.0639

The best analytic single-factor IC is 0.0493. Order changes IC from 0.0595 to 0.0661 analytically, with measured IC moving in the same direction. Measured Pearson IC is 0.008–0.010 below the analytic value. About 0.006 is sampling variation in this world: an equal-weight combination of the true signals has measured Pearson IC 0.0590 ($t = 11.7$), 0.006 below the analytic 0.0651, with a standard error of about 0.005. Another 0.003 is additional loss from using processed signals rather than the true signals (winsorization and neutralization also remove some true signal).

Figure 28-3: Combination and tuning. Left: mean Pearson IC for combinations of the same three processed signals (horizontal bars show ±2 standard errors); red dashed line marks the analytic optimum, 0.0665. Right: in 60 new worlds, weights estimated from the first 60 discovery months (orange) and the best discovery-period single factor (purple) are evaluated over the next 59 holdout months as IC minus equal-weight holdout IC. Orange values cluster on both sides of zero, more often negative (positive in only 31.7% of worlds); purple values are almost all negative (positive in 3.3%).

The red line is the oracle optimum under the stipulated population DGP moments, not a hard upper bound on every finite-sample measured IC. Likewise, Figure28-1's "stepwise recovery" describes the specified disturbance mechanism and reported averages; it does not guarantee a monotone increase for value at every step in a single world.

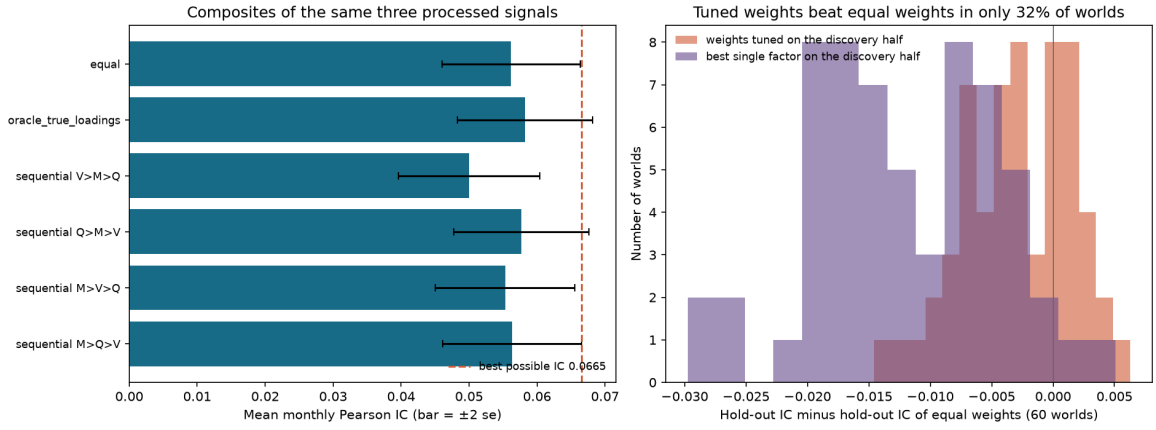


Figure 28-3: Holdout performance by combination and tuning method

Mean discovery- and holdout-period Spearman IC across 60 new worlds.

Weight rule	Discovery period	Holdout period	Discovery minus holdout
Equal weight	0.0578	0.0565	0.0013
Proportional to discovery-period IC	0.0569	0.0537	0.0032
Best discovery-period single factor	0.0459	0.0445	0.0014

Missing factors (15% random missingness for each processed factor; one world, 119 months).

Treatment	Mean Spearman IC	Coverage	Share of top quintile from single-factor names
Zero fill (minimum 1 factor)	0.0467	99.6%	3.5% (these are 5.7% of valid names)
Reweight over available factors (minimum 2)	0.0468	94.0%	—
Reweight over available factors (minimum 1)	0.0458	99.6%	8.8% (these are 5.7% of valid names)
Drop (require all three factors)	0.0511	61.3%	—

Interpretation: IC differences are small under random missingness, and the dropped-name treatment uses a different name set, so the numbers are not directly comparable. Scale bias is visible in top-quintile membership: reweighting makes single-factor names appear 1.55 times as often (8.8% versus 5.7%); zero fill reduces their representation to 0.62 times as often.

Constraints (one world, 119 months, gross returns before costs).

Portfolio	Mean monthly return (%)	t	Monthly standard deviation (%)	Mean maximum absolute industry net position	Mean absolute size exposure

Portfolio	Mean monthly return (%)	t	Monthly standard deviation (%)	Mean maximum absolute industry net position	Mean absolute size exposure
Neutralized signals, quintile weights	1.188	9.29	1.395	0.053	0.084
Raw composite (winsorized and z-scored, not neutralized), quintile weights	0.992	6.15	1.758	0.361	0.258
Same as above, with weights projected to industry neutrality	0.870	10.19	0.931	5×10^{-17}	0.230

The projected portfolio has exactly zero industry net and the lowest volatility (0.93%), so it has the highest t-statistic. Its average return is below that of the neutralized-signal quintile portfolio. These are different portfolios: projection means the weights are no longer "$+1/m$ in the top quintile and $-1/m$ in the bottom quintile." The comparison is in one constructed world and does not establish which method is better; it shows that they differ and must be identified. Position overlap: top-quintile intersection is 10.3% for value/momentum, 10.4% for value/quality, and 27.7% for momentum/quality (20% under independence).

Figure 28-4: Correlations among published factor returns. Left: monthly return correlation matrix for the five FF5 factors and two strategies constructed from 10 industry portfolios (cross-sectional momentum 12–1 and BAB), from 1963-07 through 2026-08 (758 months). Right: rolling 60-month correlation between HML and RMW. The rolling correlation ranges from -0.85 (1981-01) to $+0.82$ (2002-05). On the left, HML/CMA correlation is 0.68, HML/RMW is 0.10, and industry momentum/HML is -0.15.

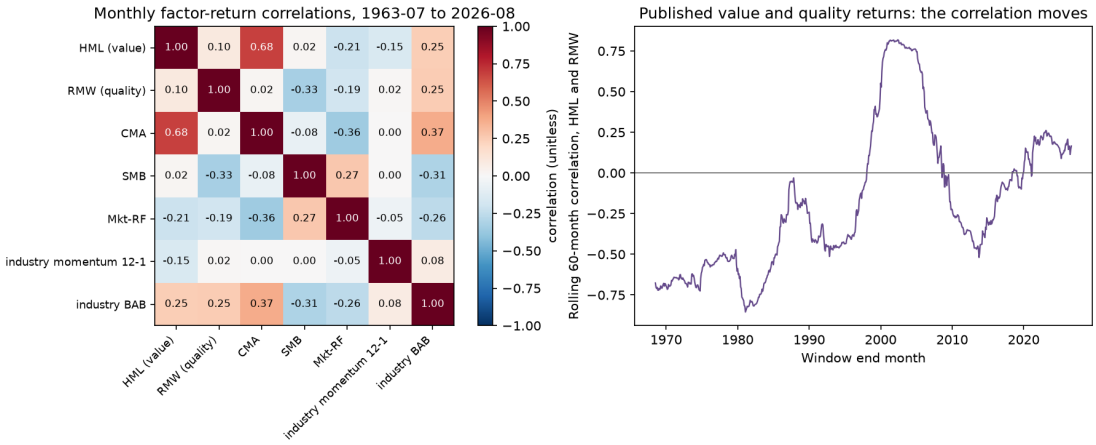


Figure 28-4: Factor return correlation matrix and rolling correlation

11. Real-market example

Data and limits. This uses the French five-factor (2×3) monthly data frozen on 2026-10-05 (processed CSV SHA-256: `833bd925b49651428781c789c13a6ac7454d4676449e6d119782050046a57eea`; 758 months, 1963-07 through 2026-08), plus the French 10-industry and FF3 snapshots used in Chapters 25 and 27 (see Chapter 25 source notes for processed hashes). Factor construction follows the French database's five-factor description: HML is one-half (Small Value + Big Value) minus one-half (Small Growth + Big Growth); RMW and CMA (conservative minus aggressive investment) are formed similarly. These are published aggregate returns, not stock-level signals, point-in-time versions, or executable trades. The two "industry strategies" are built from 10 value-weighted industry portfolios: cross-sectional 12–1 momentum (quintiles, two industries per bucket, $K = 1$) and BAB (60-month beta, shrinkage 0.6). They are neither the published UMD and BAB factors nor individual-stock strategies. Return months are aligned on the common period 1963-07 through 2026-08.

Correlation matrix (758 months).

	HML	RMW	CMA	SMB
HML (value)	1.00	0.10	0.68	0.02
RMW (quality)	0.10	1.00	0.02	−0.33
CMA	0.68	0.02	1.00	−0.08
SMB	0.02	−0.33	−0.08	1.00
Mkt-RF	−0.21	−0.19	−0.36	0.27
Industry momentum 12–1	−0.15	0.02	0.00	0.00
Industry BAB	0.25	0.25	0.37	−0.31

	Mkt-RF	Industry momentum	Industry BAB
HML (value)	−0.21	−0.15	0.25
RMW (quality)	−0.19	0.02	0.25
CMA	−0.36	0.00	0.37
SMB	0.27	0.00	−0.31
Mkt-RF	1.00	−0.05	−0.26
Industry momentum 12–1	−0.05	1.00	0.08
Industry BAB	−0.26	0.08	1.00

Unstable correlation. HML/RMW correlation is 0.095 over the full sample; −0.335 from 1963-07 through 1999-12; and +0.337 from 2000-01 through 2026-08. Rolling 60-month correlation ranges from −0.855 (window ending 1981-01) to +0.818 (ending 2002-05); the latest window is 0.165. The two subperiods use the date split carried over from Chapters 23–27; they were not selected for this chapter.

Equal-risk average. Take the monthly return series for HML, RMW, and industry momentum; divide each by its own expanding standard deviation through the prior month (at least 60 months, using only data through $t-1$; drop the first 60 months), then average. Over 698 months (1968-07 through 2026-08), annualized component Sharpe ratios are 0.341 (HML), 0.380 (RMW), and 0.294 (industry momentum), for an average of 0.338. Average pairwise correlation is −0.035. The equal-risk average has a Sharpe ratio of 0.608. Substituting the average component Sharpe into the simplified equal-risk formula, $S\sqrt{3}/\sqrt{1+2\bar{\rho}}$, also rounds to 0.608. The unrounded portfolio estimate is 0.6077680037077575; the simplified formula gives 0.6079788525663137, a difference of about 0.00021085. Expanding scaling by past risk does not force realized standard deviations to be exactly equal over these 698 months, so the close match is an approximation, not an identity. Full-sample standard deviation scaling (which looks ahead) gives a Sharpe ratio of 0.590 over the same months. Interpretation: the diversification arithmetic is close in-sample; look-ahead scaling did not raise the Sharpe in this sample, but it was unavailable at `signal_time` and is not a usable method. The numerical closeness is specific to this dataset.

What can and cannot be said. We can say that the published value and quality factors have opposite correlations across subperiods from 1963 through 2026, illustrating short-window estimation uncertainty and cross-period drift; this does not imply that every 60-month matrix is necessarily unstable. We can also say that the in-sample Sharpe of an equal-risk average of three low-correlation return series is close to the simplified formula. We cannot say: (1) that this portfolio has a 0.6 Sharpe out of sample; the value uses three components and 698 in-sample months, HML and RMW are published revised histories, and industry momentum uses only 10 industries; (2) that value and quality factors are "crowded," because this chapter has no holdings, short-borrow rates, or fund-flow data; (3) how a stock-level multi-factor portfolio performs, because the stock-level experiment here is only a constructed panel; or (4) anything about costs, capacity, or tradability.

12. Common errors

Error 1: Standardizing with a full-sample fit. Using full-sample mean and standard deviation for each stock's time series makes row t depend on the future (Section 8.6: $+1.0$ versus -0.245). Fix: use same-date cross-sectional z-scores or an expanding version; test that changing the last row leaves earlier outputs unchanged (code block five).

Error 2: Winsorizing with quantiles from the entire panel. Pooling all dates to calculate the 1st and 99th percentiles uses other dates' distributions, including the future. Fix: compute cross-sectional quantiles separately for each date (`winsorize_cross_section`).

Error 3: Choosing winsorization or z-score parameters by evaluation-period IC. The choice of quantiles (1%/99% or 5%/95%), degrees of freedom, and a second winsorization can all look different in the evaluation period. Fix: specify them in advance and record variants in the ledger (Chapter 22).

Error 4: Neutralizing with hindsight industry labels. Industry classifications can be revised; a label unavailable at `signal_time` introduces future information. Fix: use the version known at the time (date-indexed `groups`) and report the share of missing labels.

Error 5: Removing true signal through neutralization. If an industry shift itself carries returns (real industry momentum or a value effect), neutralization removes it. Fix: state whether neutralization is a research hypothesis and report IC before and after; do not transfer a favorable constructed-world result to real data.

Error 6: Using too many controls. Every added control consumes degrees of freedom; on a date with eight stocks, ten industry indicators leave no residual variation. Fix: return NaN when degrees of freedom are insufficient (the `neutralize` rule) and report how many dates were dropped.

Error 7: Confusing signal neutralization with linear-constraint projection. They agree for weights proportional to the signal (Section 8.3) and differ for quantile weights (Section 10: mean maximum industry net is 0.053 versus 5×10^{-17}, with different volatility and returns). Fix: identify which operation was used.

Error 8: Choosing orthogonalization order after evaluation. Order can change IC by 36% (Section 8.4). Fix: state a rule in advance, such as "the foundational factor comes first," and record all orders.

Error 9: Choosing composite weights from evaluation-period performance. Discovery-period IC-proportional weights beat equal weights in the holdout in only 31.7% of worlds (Section 10); the best single factor does so in 3.3%. Fix: fix weights in advance, or estimate them in discovery and evaluate them in an independent holdout, with the process recorded.

Error 10: Failing to report coverage and scale after zero fill or reweighting. In this constructed random-missingness sample, reweighting makes single-factor names appear in the top quintile 1.55 times as often as their baseline share; zero fill pushes them toward the middle (0.62 times). Fix: set a minimum factor count, report coverage and the single-factor share, and test sensitivity to both treatments.

Error 11: Configuring weights from one 60-month correlation matrix. HML/RMW correlation is −0.34 in 1963–1999 and +0.34 in 2000–2026, with a rolling range from −0.86 to +0.82 (Section 11). Fix: report estimation uncertainty and multiple periods; state that the diversification formula depends on average correlation.

Error 12: Confusing signal correlation with return correlation. Signal correlation (same-date cross-section) determines composite IC; return correlation (time series of factor returns) determines diversification. Their signs can differ (constructed panel: signal correlations of −0.3, while all true signals have positive next-month return loadings).

Error 13: Treating position overlap as proof of crowding. Top-quintile overlap is only one aspect; assessing crowding requires holdings, short-borrow, and fund-flow data (Section 6.9).

13. Quant research application

Multi-factor workflow contract (Project 4 signal layer).

Step	Transform	Which date's data?	Who fixes the parameter, and when?
1	Point-in-time version selection (Chapters 24–26, `latest_available_values`)	Available before signal month	Plan specifies availability rule
2	Same-date winsorization (P4: 1%/99%, at least 30 field values)	Same-date cross-section	Fixed in the plan before seeing returns
3	Same-date neutralization (industry and log market capitalization)	Same-date cross-section, with industry label version known at the time	Plan
4	Same-date z-score (P4 uses `ddof=1` and minimum 10; the hand-calculation example uses minimum 3)	Same-date cross-section	Plan
5	Combination (equal weights, minimum 2 factors, reweight over available factors)	Same date	Plan; weight variants recorded in the ledger
6	Quantile weights or weights proportional to the signal; linear constraints (Chapter 21 projection)	Same date	Plan
7	Evaluation: IC, quantile returns, turnover, exposures	Next-month label; independent holdout	Discovery/holdout split fixed in the plan

Research workflow (constructed panel with known data-generating process). Frozen plan: hypothesize that "the equal-weight composite of three processed factors has positive same-date Rank IC with next-month returns and exceeds each single factor's IC"; primary measure is the composite's mean Rank IC and its t-statistic. Preprocessing is fixed as 1%/99% winsorization, neutralization by industry and size, and z-scoring; weights are fixed at equal weight. Result: composite Rank IC is 0.0519 ($t = 10.5$), slightly above momentum's 0.0511, but the composite's own t-statistic does not test the difference between the two. Supporting "better than any single factor" requires a separate paired, same-month difference evaluation; that additional test is not in this batch. Pearson IC is 0.0562. Sequential-orthogonalization IC ranges from 0.0467 to 0.0532, depending on order; missing-factor IC ranges from 0.0458 to 0.0511. Across 60 new worlds, discovery-period tuning beats equal weights in the holdout in only 31.7% of worlds. The construction provides checkable expectations. These results provide limited engineering evidence about the tested inputs and workflow; they do not prove correctness for arbitrary inputs and are not real-world effects.

Variants in the ledger. Winsorization (yes/no) × neutralization (yes/no) × combination (equal weight, true loadings, four sequential orders) × missing-data treatment (four options) × constraint (three options). Not every combination was evaluated, but every evaluated variant must be recorded; all reported t-statistics are naive and unadjusted for multiple testing.

Constraints for Project 4. (1) Use only data available by the signal month; same-date transforms use that date's cross-section. (2) Use industry labels known at the time. (3) Fix weights, winsorization quantiles, minimum factor count, and orthogonalization order in advance. (4) Report coverage per factor and composite, shares of dropped or imputed names, and the share of single-factor names at both tails. (5) Report IC before and after neutralization, plus industry and size exposures. (6) State whether the constraint is signal neutralization or weight projection. (7) Report correlation uncertainty and values for multiple periods. (8) Do not claim "crowding" without actual holdings data.

13.1 How to read the C13 protocol correction alongside historical results

The original constructed panel, figures, and French aggregate diagnostics in Sections 9–11 remain tied to their own inputs; they are not rewritten as a new P4 run. The old A5/A6 P4 projected over all 240 columns and then filtered by listing status. Its historical record includes 1,842 name-months zeroed out and a maximum industry net position of 0.0280748663 after filtering. This exposed a mismatch between protocol and implementation; a claim that "industry projection was already applied" does not replace a support-set check.

C13 corrected the protocol while preserving the meaning of the original configuration: enforce the 60-name threshold at preprocessing and portfolio entry; apply optional industry projection only on the signal-time listed support set; restore configured gross of 2 after the homogeneous projection; explicitly record degenerate cases as cash; and report A6 target and actual-holding residuals separately. The original seven fields, ranking and combination formulas, data-generating process, seed, and two frozen configurations are unchanged. The new source hashes, old/new sample, weight, return, and cost differences, and checks are documented in the accepted P4 protocol-alignment note.

This correction is a normative alignment made after the old results had been seen. It is not a new blind preregistration, independent market test, or tuned strategy. Keep favorable and unfavorable return changes. Restoring configured gross and neutrality does not establish alpha, borrow availability, or capacity. Chapter 29 remains responsible for execution ledgers and cash/fee reconciliation; execution-day states do not flow backward into the signal.

14. Chapter summary

A multi-factor workflow has three layers: preprocessing (same-date cross-sectional transforms that change scale, outliers, and unwanted exposures), combination (weights fixed in advance), and constraints (portfolio-level exposures). When a signal contains a component unrelated to returns, Pearson IC is attenuated by $1/\sqrt{1+\sigma_u^2/\sigma_s^2}$; winsorization and neutralization can recover it. Same-date regression neutralization is within-group demeaning plus Frisch–Waugh–Lovell. For weights proportional to the signal, it is the same vector as linear-constraint projection; for quantile weights, it is not. The equal-weight composite IC of two correlated signals is $(\rho_1 + \rho_2)/\sqrt{2(1+r_{12})}$; sequential orthogonalization is order-dependent; the optimal weights $\propto R^{-1}\rho$ require true values; and in the 60 constructed worlds and settings reported here, discovery-period tuning beats equal weights in the holdout in only 31.7% of worlds. In general, tuning is not guaranteed to outperform equal weights. Zero fill and reweighting over available factors compress and expand scale, respectively. Full-sample standardization uses future data. Factor-return correlations can change sign over time, and the diversification formula $S\sqrt{k}/\sqrt{1+(k-1)\bar{\rho}}$ depends on average correlation. The selected checks on the constructed panel roughly match analytic predictions; they do not prove that every step or arbitrary input is correct. Published HML/RMW correlation has opposite signs in 1963–1999 and 2000–2026. The in-sample Sharpe of an equal-risk average of three low-correlation series is close to the simplified equal-risk formula, but is not an exact identity. Scope: disturbances in the constructed panel are specified by design; published factor data are revised histories; there are no actual holdings, so no crowding claim is made.

15. Exercises

28-E01 (calculation). Same-date signals for 11 stocks are $(3, 4, 5, 6, 7, 8, 9, 10, 11, 12, -50)$. Winsorize at the 10th and 90th percentiles using linear interpolation. Find the winsorized values, mean, sample standard deviation, z-score of signal 12, and z-score of signal -50 without winsorization. Give the normal score of signal -50.

28-E02 (calculation). There are 8 stocks; industry A is the first four and industry B the last four. Signal $(2, 0, 1, 3,\ 10, 8, 12, 6)$ and next-month returns $(6, 2, 4, 8,\ -1, -3, 1, -5)\%$. Find the industry-neutral residuals, raw Pearson IC, and neutralized Pearson IC; explain the signs.

28-E03 (calculation). Use the 28-E02 signal as weights w and let A contain the two industry-indicator rows. Find $w - A^\top(AA^\top)^{-1}Aw$ and compare it with the 28-E02 residuals. If instead the weights are $+1/2$ for the top 25% and $-1/2$ for the bottom 25%, what is each industry's net position after projection? What is it before projection?

28-E04 (calculation). Two standardized signals have $\rho_1 = 0.05, \rho_2 = 0.03, r_{12} = 0.4$. Find the IC of the equal-weight composite, the ICs for both sequential orthogonalization orders, the optimal weights, and the optimal IC. Explain which order is better and why.

28-E05 (calculation). Four equally weighted factors each have mean zero and variance one, and every pair of distinct factors has correlation 0.25. Assume missingness is completely at random (MCAR), so these conditional moments still hold within every available subset, given the missingness mask. The overall average correlation of four factors alone would not be sufficient. A name has $k = 1, 2, 4$ available factors. Find the composite-score variance under reweighting over available factors and under zero fill. Compare the scales and state what distribution and missingness assumptions are needed to discuss representation in the tails.

28-E06 (calculation). A stock's signal sequence is $(4, 6, 8, 20)$. Find the expanding z-score (sample standard deviation) and full-sample z-score for the third observation. Explain why the latter is unavailable at `signal_time`.

28-E07 (calculation). Four factors each have Sharpe ratio 0.4 and average correlation 0.25. Find the Sharpe ratio of the equal-risk average. What average correlation would make it 0.6? What is the lower bound on average correlation?

28-E08 (conceptual). For each method, say whether it is available at `signal_time` and why: (a) z-score each date's cross-section; (b) z-score each stock's signal time series using its full-sample mean and standard deviation; (c) winsorize using 1st and 99th percentiles calculated over the whole panel; (d) `neutralize` within the date using industry labels known in the signal month; (e) `orthogonalize` in the order with the highest evaluation-period IC; (f) scale factor returns by an expanding standard deviation through the prior month.

28-N01 (support set, budget, and causal clock). Use the four names, weights, and industry labels from Section 8.8. (a) When a is unlisted, report industry net, total net, and gross after full-column projection followed by masking, then give the weights and residuals from projecting only on b, c, and d and restoring gross to 2. (b) All four names are listed, initial quantile weights are $(1, 0, -1, 0)$, and b and d have no alpha. Find the projected weights and explain whether this remains a pure quantile portfolio. (c) How should dates with 59 or 60 signal-time listed names be handled when a field has only 29 values? (d) After execution, suppose only c's long 0.8 and d's short 0.6 in industry B are filled. Report industry net, total net, and gross; may these fills be used earlier to change the signal? (e) For $A = (1, 1), b = 1, v = (0.5, 0.5)$, does scaling gross to 2 preserve the constraint?

16. Programming exercises

28-P01. Reproduce 28-E02 using `neutralize`: assert the residuals and the raw and neutralized ICs. Then add a size-control column and assert that residuals are orthogonal to size and sum to zero within each industry. Acceptance: all four assertions pass.

28-P02. Simulate two standardized signals with correlation 0.5 and one label ($\rho_1 = 0.04$, $\rho_2 = 0.02$, 400,000 observations). Use explicit regressions to `orthogonalize` in both orders, calculate the composite IC, and compare it with `sequential_orthogonal_ic`. Acceptance: each order's IC is within 0.006 of its formula, and the second order's IC is more than 0.006 above the first.

28-P03. Simulate 100 worlds, each with 300 stocks and 120 months. Three mutually independent standard-normal signals contribute equally to the label. Estimate each factor's mean IC over the first 60 months, use its positive part as a weight (normalized), and compare mean IC for tuned versus equal weights over the next 60 months. Acceptance: mean holdout IC under equal weights exceeds the tuned mean, and tuning beats equal weights in fewer than 40% of worlds.

17. Challenge problems

28-C01 (derivation). For two standardized signals, prove that the IC difference after sequential orthogonalization is

$$\mathrm{IC}_{1\to 2} - \mathrm{IC}_{2\to 1} = \frac{\rho_1 - \rho_2}{\sqrt{2}}\left[1 - \sqrt{\tfrac{1+r_{12}}{1-r_{12}}}\right],$$

and discuss which order is better when $r_{12} > 0$ and when $r_{12} = 0$. Then prove that the IC of the optimal weights is at least as large as that of either order. Prerequisite: the Cauchy–Schwarz inequality. Grading: derive the identity (50), discuss the signs (25), prove that the optimum is no worse (25).

28-C02 (open-ended). Write a frozen multi-factor workflow plan and data-gap table for Project 4. List the data date used at every step (version selection, winsorization, neutralization, z-scoring, combination, constraints, evaluation), who fixes each parameter and when, required fields (including historical industry classifications and market capitalization), and the number of variants recorded. Explain which conclusions cannot be made without those data. Give the three checks required before combination: correlations, coverage, and point-in-time eligibility. Grading: workflow table (30), gap table (30), unsupported conclusions (20), pre-combination checks (20).

18. Complete answers and worked solutions

28-E01. Sorted values are $-50, 3, 4, \ldots, 12$. Quantile positions are $(n-1)q = 10 \times 0.1 = 1$ and 9, giving $q^- = 3$ (the second order statistic) and $q^+ = 11$ (the tenth). After winsorization, $-50 \to 3$ and $12 \to 11$, so the original order becomes $(3, 4, 5, 6, 7, 8, 9, 10, 11, 11, 3)$. Mean $77/11 = 7$; sum of squared deviations $16 + 9 + 4 + 1 + 0 + 1 + 4 + 9 + 16 + 16 + 16 = 92$; sample variance 9.2; standard deviation 3.033. Signal 12 is winsorized to 11, so $z = (11 - 7)/3.033 = 1.319$. Without winsorization, mean $25/11 = 2.27$, standard deviation 17.57, and signal -50 has $z = (-50 - 2.27)/17.57 = -2.975$; the other ten are compressed between $+0.04$ and $+0.55$. The normal score for -50 is rank 1, $\Phi^{-1}(0.5/11) = -1.691$.

28-E02. Industry A's mean is 1.5, giving residuals $(0.5, -1.5, -0.5, 1.5)$; industry B's mean is 9, giving residuals $(1, -1, 3, -3)$. Raw IC: signal mean 5.25, $\sum(x-\bar{x})y = -0.75$, $\sum(x-\bar{x})^2 = 137.5$, $\sum(y-\bar{y})^2 = 0.0138$ ($\bar{y} = 0.015$), so IC $= -0.75/\sqrt{137.5 \times 0.0138} = -0.544$. After neutralization, $\sum ey = 0.3$ (because residuals sum to zero, this equals $\sum e(y-\bar{y})$), $\sum e^2 = 25$, so IC $= 0.3/\sqrt{25 \times 0.0138} = +0.511$. Industry B has a higher signal level and lower returns (an inverse industry-level relationship), while within industries higher signals have higher returns. Raw IC mostly reflects the industry-level relationship; neutralized IC reflects the within-industry relationship.

28-E03. $AA^\top = \mathrm{diag}(4,4)$ and $Aw = (6, 36)$. Subtract $A^\top(1.5, 9)$: $w - (1.5, 1.5, 1.5, 1.5, 9, 9, 9, 9) = (0.5, -1.5, -0.5, 1.5, 1, -1, 3, -3)$, matching the 28-E02 residuals entry by entry. For quantile weights, the top two names are 12 (industry B) and 10 (industry B); the bottom two are 0 (A) and 1 (A). Assign $+1/2$ to the two B names and $-1/2$ to the two A names. Before projection, A net is -1 and B net is $+1$ (far from neutral). Then $Aw = (-1, +1)$ and $AA^\top = \mathrm{diag}(4,4)$. The correction is -0.25 for each A name and $+0.25$ for each B name, subtracted from the original weights. After projection, the weights are all ± 0.25: industry A, in original order, is $(0.25, -0.25, -0.25, 0.25)$; industry B is $(0.25, -0.25, 0.25, -0.25)$. Both industries have zero net, but the weights are no longer the original top-25% and bottom-25% portfolio.

28-E04. Equal weight: $(0.05 + 0.03)/\sqrt{2 \times 1.4} = 0.0478$. For $1 \to 2$: $\frac{1}{\sqrt{2}}[0.05 + (0.03 - 0.4 \times 0.05)/\sqrt{0.84}] = \frac{1}{\sqrt{2}}[0.05 + 0.0109] = 0.0431$. For $2 \to 1$: $\frac{1}{\sqrt{2}}[0.03 + (0.05 - 0.4 \times 0.03)/\sqrt{0.84}] = \frac{1}{\sqrt{2}}[0.03 + 0.0415] = 0.0505$. Optimal weights are $R^{-1}\rho \propto (0.05 - 0.4 \times 0.03,\ 0.03 - 0.4 \times 0.05) = (0.038, 0.010)$; normalized weights are $(0.792, 0.208)$, and IC is $\sqrt{\rho^\top R^{-1}\rho} = 0.0512$. The $2 \to 1$ order (put the weaker second factor first, then add the first factor's incremental information) is better: IC 0.0505, close to the optimum 0.0512. This agrees with the identity in 28-C01: $(\rho_1 - \rho_2)/\sqrt{2}\,[1 - \sqrt{1.4/0.6}] = 0.01414(1 - 1.5275) = -0.00746$, so $\mathrm{IC}_{1\to2} - \mathrm{IC}_{2\to1} = 0.0431 - 0.0505 = -0.0075$.

28-E05. By the stated equicorrelation and MCAR assumptions, any available subset, conditional on its mask, retains zero means, unit variances, and pairwise correlation $\bar{r} = 0.25$. The same correlation parameter can therefore be used for each k. Reweighting variance is $(1 + (k-1) \times 0.25)/k$: for $k = 1$, 1; for $k = 2$, 0.625; for $k = 4$, 0.4375. Zero-fill variance is $k[1 + (k-1) \times 0.25]/16$: for $k = 1$, 0.0625; for $k = 2$, 0.156; for $k = 4$, 0.4375. Under reweighting, single-factor names have $1/0.4375 = 2.29$ times the variance of complete names (1.51 times the standard deviation). If we additionally assume the `groups` have the same distributional shape and missingness is independent of values, the larger variance can make them overrepresented beyond a fixed symmetric tail threshold; variance alone cannot prove an arbitrary tail probability. Under zero fill, their variance is $0.0625/0.4375 = 0.143$ times as large (standard deviation 0.38 times as large), so their scale shrinks. Only with further assumptions, such as the same distributional shape, can we infer underrepresentation in both tails.

28-E06. The first three observations are $(4, 6, 8)$, with mean 6 and sample standard deviation 2; the third value has $z = (8-6)/2 = +1.0$. The full sample $(4, 6, 8, 20)$ has mean 9.5 and sample standard deviation 7.19; $z = (8 - 9.5)/7.19 = -0.209$. The sign changes because the full-sample mean and standard deviation use the future value 20, which was not available at the third observation's `signal_time`.

28-E07. $S_p = 0.4 \times 2/\sqrt{1 + 3 \times 0.25} = 0.8/1.3229 = 0.605$. For $S_p = 0.6$, $\sqrt{1 + 3\bar{\rho}} = 0.8/0.6 = 1.333$, so $1 + 3\bar{\rho} = 1.778$ and $\bar{\rho} = 0.259$. The lower bound is $-1/(k-1) = -1/3$; the average correlation cannot be lower because the correlation matrix must be positive semidefinite. At the bound, the denominator of the Sharpe formula tends to zero and the formula is undefined.

28-E08. (a) Usable with conditions: same-date fields and industry-label versions and their parameter policy must be known at `signal_time`, and only that cross-section may be used; being in one row does not itself establish point-in-time eligibility. (b) Not usable: the full-sample mean and standard deviation include the future. (c) Not usable: this uses distributions from other dates, including future dates; calculate cross-sectional quantiles separately for each date. (d) Usable if the industry-label version was known at `signal_time`. (e) Not usable: choosing order from evaluation results tunes to the test set; fix the order in advance and record all variants. (f) Usable: an expanding standard deviation through the prior month uses no future data.

28-P01

```
import numpy as np
import pandas as pd

from quant_course.factor_preprocessing import neutralize

cols = list("abcdefgh")
sig = pd.DataFrame([[2, 0, 1, 3, 10, 8, 12, 6.0]], index=pd.period_range("2026-06",
    ↪ periods=1, freq="M"), columns=cols)
y = np.array([0.06, 0.02, 0.04, 0.08, -0.01, -0.03, 0.01, -0.05])
groups = pd.Series(list("AAAABBBB"), index=cols)
res = neutralize(sig, groups=groups)
assert np.allclose(res.iloc[0], [0.5, -1.5, -0.5, 1.5, 1.0, -1.0, 3.0, -3.0])
raw_ic, neutral_ic = np.corrcoef(sig.iloc[0], y)[0, 1], np.corrcoef(res.iloc[0], y)
    ↪ [0, 1]
assert abs(raw_ic + 0.5445) < 1e-4 and abs(neutral_ic - 0.5108) < 1e-4
size = pd.DataFrame([[1.0, 2.0, 0.5, 3.0, 2.5, 1.5, 0.2, 4.0]], index=sig.index,
    ↪ columns=cols)
res2 = neutralize(sig, groups=groups, controls={"size": size})
assert abs(np.dot(res2.iloc[0], size.iloc[0])) < 1e-10 and abs(res2.iloc[0].groupby(
    ↪ groups).sum()).max() < 1e-10
```

28-P02

```
import numpy as np

from quant_course.factor_preprocessing import sequential_orthogonal_ic

rng = np.random.default_rng(1)
n = 400_000
z = rng.multivariate_normal([0.0, 0.0], [[1.0, 0.5], [0.5, 1.0]], size=n)
y = 0.04 * z[:, 0] + np.sqrt(1 - 0.04 ** 2) * rng.normal(size=n)               # corr
    ↪ (z1, y) = 0.04, corr(z2, y) = 0.02


def sequential(first: int, second: int) -> float:
    e = z[:, second] - np.polyval(np.polyfit(z[:, first], z[:, second], 1), z[:,
    ↪ first])    # residual of the second on the first
    return float(np.corrcoef(z[:, first] + e / e.std(), y)[0, 1])
```

```
a, b = sequential(0, 1), sequential(1, 0)
assert abs(a - 0.0283) < 0.006 and abs(b - 0.0386) < 0.006 and b - a > 0.006
assert abs(a - sequential_orthogonal_ic([0.04, 0.02], [[1, 0.5], [0.5, 1]], [0, 1]))
    ↪ < 0.006
```

28-P03

```
import numpy as np

rng = np.random.default_rng(5)
worlds, n, months = 100, 300, 120


def mean_ic(sig: np.ndarray, lab: np.ndarray) -> np.ndarray:
    s = (sig - sig.mean(axis=1, keepdims=True)) / sig.std(axis=1, keepdims=True)
    l = (lab - lab.mean(axis=1, keepdims=True)) / lab.std(axis=1, keepdims=True)
    return (s * l).mean(axis=1)                                                  #
    ↪ per-month Pearson IC


hold_eq, hold_tuned = [], []
for _ in range(worlds):
    z = rng.normal(size=(months, n, 3))
    label = 0.05 * z.sum(axis=2) + rng.normal(size=(months, n))                  #
    ↪ three equally good, independent signals
    half = months // 2
    disc = np.array([mean_ic(z[:half, :, k], label[:half]).mean() for k in range(3)])
    w = np.maximum(disc, 0.0)
    w = w / w.sum() if w.sum() > 0 else np.ones(3) / 3
    hold_eq.append(mean_ic(z[half:].sum(axis=2), label[half:]).mean())
    hold_tuned.append(mean_ic(z[half:] @ w, label[half:]).mean())
hold_eq, hold_tuned = np.array(hold_eq), np.array(hold_tuned)
assert hold_eq.mean() > hold_tuned.mean() and (hold_tuned > hold_eq).mean() < 0.4
```

28-C01. Let $\mathrm{IC}_{1\to 2} = \frac{1}{\sqrt{2}}[\rho_1 + (\rho_2 - r\rho_1)/\sqrt{1-r^2}]$ and $\mathrm{IC}_{2\to 1} = \frac{1}{\sqrt{2}}[\rho_2 + (\rho_1 - r\rho_2)/\sqrt{1-r^2}]$, where $r = r_{12}$. Their difference is $\frac{1}{\sqrt{2}}[(\rho_1 - \rho_2) + ((\rho_2 - r\rho_1) - (\rho_1 - r\rho_2))/\sqrt{1-r^2}]$. Since $(\rho_2 - r\rho_1) - (\rho_1 - r\rho_2) = (\rho_2 - \rho_1)(1+r)$, this becomes $\frac{\rho_1 - \rho_2}{\sqrt{2}}[1 - (1+r)/\sqrt{1-r^2}]$. Because $(1+r)/\sqrt{(1-r)(1+r)} = \sqrt{(1+r)/(1-r)}$, this is the claimed identity. If $r > 0$, then $\sqrt{(1+r)/(1-r)} > 1$, so the bracket is negative; when $\rho_1 > \rho_2$, $\mathrm{IC}_{1\to 2} < \mathrm{IC}_{2\to 1}$: put the weaker factor first, then add the stronger factor's incremental information. If $r < 0$, the bracket is positive, so the stronger factor should come first. If $r = 0$, the bracket is zero and the orders agree (the residual is just the signal itself). For any weight vector w, $\mathrm{IC} = w^\top \rho / \sqrt{w^\top R w} \le \sqrt{\rho^\top R^{-1} \rho}$. By Cauchy–Schwarz, let $v = R^{1/2} w$; then $w^\top \rho = v^\top R^{-1/2} \rho \le \|v\| \, \|R^{-1/2}\rho\| = \sqrt{w^\top R w}\sqrt{\rho^\top R^{-1}\rho}$. A sequentially orthogonalized combination is a linear combination $w^\top z$, so its IC cannot exceed the optimum. Grading: identity derivation (50), sign discussion (25), proof that the optimum is no worse (25).

28-C02 (grading guidance and reference). Available course data include French five-factor returns, 10-industry monthly returns, and a constructed panel. Missing are historical point-in-time industry classifications, market capitalization, and point-in-time versions of stock-level fundamentals. For each row in the workflow table, state the transformation, data date, parameters, and who fixes them and when. For each gap, state the field (industry classification, log size, or raw field for each factor), required real source, time fields, and missingness rule. Unsupported claims include stock-level multi-factor portfolio returns, capacity or costs, any crowding conclusion, or treating constructed-panel IC values as real IC. The three checks before combination are: (1) correlations—same-date signal and factor-return correlations, including their values across periods; (2) coverage—coverage for each factor and whether missingness is related to the signal; and (3) point-in-time eligibility—each field's `available_at`, industry-label version, and the data date used by every transformation.

28-N01 (complete answer). (a) After masking, weights are $(0, -0.4, 0.3, -0.3)$: industry A net -0.4, industry B net 0, total net -0.4, gross 1. The support-restricted projection is $(0, 0, 0.3, -0.3)$ with gross 0.6; multiplying by $10/3$ gives $(0, 0, 1, -1)$, with industry and total net zero, gross 2, and zero outside the support set. (b) Industry net positions initially are 1 and -1, with means 0.5 and -0.5. Subtracting the means gives $(0.5, -0.5, -0.5, 0.5)$, net 0 and gross 2. b and d may hedge within the listed support set but have no alpha ranking eligibility, so the result is no longer a pure quantile portfolio. (c) With 59 names, the date is ineligible, preprocessing is missing, and the target is cash; a complete field cannot restore eligibility. With 60 names, the universe passes the threshold, but a 29-value field does not meet the 30-value winsorization threshold and should be missing. That field cannot enter style combination. Other fields/styles may still form a portfolio if they meet their own thresholds. (d) Holdings $(0, 0, 0.8, -0.6)$, assuming all values are normalized by the same reported NAV: industry B net and total net are both 0.2; gross is 1.4. This is a post-execution residual and cannot enter the month-end signal in advance. A real ledger must also state the NAV, price, and fee normalization conventions. (e) $Av = 1$ and gross is 1; scaling to $2v = (1, 1)$ gives $A(2v) = 2 \neq 1$. The homogeneous proof for target $b = 0$ does not extend to a nonzero target; current P4 has no such strategy.

19. Further reading

- Must Know: Kenneth French Data Library, five-factor description; review the construction and sample conditions for HML, RMW, CMA, and SMB.
- Must Know: Chapter 22 of this course, the experiment ledger and multiple testing (Bonferroni, Holm, Benjamini–Hochberg); every degree of freedom in a multi-factor workflow belongs in the ledger.
- Should Know: Gelman and Loken on forking paths (working paper, referenced in Chapter 22 source notes); read their discussion of analyses being selected in response to data.
- Should Know: Chapter 21's `project_to_constraints` and exposure reporting; review the linear-constraint projection derivation.
- Advanced: Asness, Moskowitz, and Pedersen on combining value and momentum across asset classes; Fama and French (2015) on redundancy of the value factor in the five-factor model. This chapter has not reviewed their findings on factor correlation and combination.

- Optional: Any public fund-holdings disclosure, to practice measuring the overlap between the top quintiles of two factors. This course has no holdings data.

20. References

Citation keys and reading scope are listed in `source_notes.md`.

- CH28_FF5_PAGE: Kenneth R. French Data Library, "Description of Fama/French 5 Factors (2x3)" (construction and sample conditions for HML, RMW, CMA, SMB, and the market).
- CH22_GELMANLOKEN2013: Gelman, A. and Loken, E. (2013), unpublished working paper (see Chapter 22 source notes).
- CH25_FRENCH_IND: French 10-industry and FF3 snapshots (Chapter 25 source notes), plus the constructed 12–1 momentum and BAB strategies (Chapters 25 and 27).
- FRENCH: Frozen French five-factor snapshot (data/manifests/french_factors_2026-10-05.json).

Chapter 29 — Building a Backtesting Engine

A backtesting engine turns a target weight decided at time s into actual shares and a cash balance at a later executable time. A self-financing ledger keeps those units separate. It lets us attribute NAV changes to price movements, dividends, interest, delistings, and execution fees rather than treating an optimizer's output as an already executed position. Prerequisites: Chapter 3 (data engineering and point-in-time contracts), Chapter 11 (returns and risk), Chapter 21 (portfolio construction, drift, and turnover), and Chapter 22 (research methodology). After this chapter you should be able to derive the reconciliation identity, solve the post-fee NAV equation, explain passive weight drift, distinguish execution and marking prices, and identify when a vectorized calculation represents the same strategy as a share ledger.

1. Learning goals

A model asks for 50% in stock A, 30% in stock B, and 20% in cash. That does not yet tell us how many shares the account owns. Fees reduce the allocation base; execution happens later than the decision; prices move between execution and the close. Dividends, splits, and delistings change the state again.

The chapter develops four capabilities:

1. Derive cash and share conservation, including the equation $V' = V_{\text{pre}} - c\sum_i |w_i^* V' - x_i|$. Under $c\sum_i |w_i^*| < 1$, prove global uniqueness of the real root and its positive-root condition $c\sum_i |x_i| < V_{\text{pre}}$. Derive a closed form for known trade signs and prove finite monotone Newton/subgradient termination in exact arithmetic. Keep that theorem distinct from the floating-point implementation's tolerance and 100-iteration cap.
2. Explain the difference between targets and actual weights: relative price movements, fees, delayed execution, and, in models that impose them, whole-share constraints. A signal computed from a closing price cannot execute at that same close. Here lag is a number of observations in the price index.
3. Use `src/quant_course/backtest.py` to inspect `fills`, `positions`, actual `weights`, and `reconcile`. For positive, finite NAV, check the relative residual against 10^{-12}; report the absolute currency residual separately for a nonpositive terminal NAV, without dividing by zero or a negative value. This is an engineering threshold for this implementation at the stated scale, not a universal floating-point accuracy guarantee. It does not establish that prices or timestamps are valid.
4. State sufficient conditions for a vectorized drift model and the ledger to agree, including aligned initial states, execution calendars, and rebalancing policies. Diagnose a mismatch before interpreting a return curve.

Completion task: reproduce all four observation dates in Section 8, including shares, fees, cash, and NAV to four decimal places. Report the largest reconciliation residual and explain its units.

2. Why a quant needs this

```
Raw data -> Factor engineering -> Portfolio optimization -> Target weights
            -> Backtesting ledger -> Performance and risk evaluation
 Chapters 3,23-28       Chapters 20-21     Chapter 29       Chapters 32-33
```

Several errors enter precisely at the transition from targets to holdings:

- **Look-ahead fills.** Computing a signal after the close and assigning it that close's execution price reverses the information and order clocks. Orders need a later execution event.
- **Invisible friction.** Multiplying weights by returns can omit the cash paid for every rebalance. Small recurring costs reduce both current wealth and the base that earns later returns.
- **Drift blindness.** Forward-filling target weights between monthly rebalances silently describes repeated resets if used as actual holdings. Relative asset returns change a buy-and-hold portfolio's weights.
- **Broken corporate-action accounting.** A split changes shares as well as price. A cash dividend credits a long holder and debits a short holder. A delisting settles a residual amount or a zero recovery. Combining a dividend-inclusive return series with explicit dividends counts the same benefit twice.

This chapter builds a one-currency ledger with no external flows, deterministic complete fills, and proportional fees. Chapter 30 uses it to examine eight backtesting biases; Chapter 31 addresses time separation and validation; Chapter 32 develops performance inference, including PSR, DSR, and PBO; Chapter 33 uses holdings and cash for risk measures and stress tests.

3. Intuition: two assets and cash

3.1 Start with the allocation base

After the close on Friday, January 30, 2026, the account has USD 1,000 cash and no shares. The decision is

$$w_A^* = 0.5, \qquad w_B^* = 0.3, \qquad w_{\text{cash}}^* = 0.2.$$

The fee rate is $c = 0.002$, or 20 bps of one-way traded notional.

Buying USD 500 of A and USD 300 of B would cost another USD 1.60 in fees. Cash would be USD 198.40 and NAV USD 998.40, so the cash fraction would be 198.40/998.40 = 19.87%, not 20%.

Instead, apply the target fractions to NAV **after** fees:

$$\begin{aligned} V' + \text{Fees} &= 1000, \\ V' + 0.002(0.5 + 0.3)V' &= 1000, \\ 1.0016V' &= 1000, \\ V' &= 998.4026. \end{aligned}$$

The purchases total USD 798.7220, fees are USD 1.5974, and cash is USD 199.6805. Using unrounded values, cash is exactly $0.2V'$.

3.2 Execution and subsequent drift

At the next observation, February 2, execution prices are $E_A = 10, E_B = 20$. The account buys 49.92013 shares of A and 14.97604 shares of B. At that day's close, $P_A = 11, P_B = 19$:

- A is worth USD 549.1214;
- B is worth USD 284.5447;
- cash is USD 199.6805;
- closing NAV is USD 1033.3466.

Thus

$$w_A = 53.14\%, \qquad w_B = 27.54\%, \qquad w_{\text{cash}} = 19.32\%.$$

The targets were hit at the execution price and post-fee denominator. Closing weights differ because prices moved afterward.

On February 27, the closing prices are USD 12.1 and USD 19. NAV is USD 1088.2588, with A at 55.50%, B at 26.15%, and cash at 18.35%. A new decision requests the original 50/30/20 allocation.

At the March 2 execution prices $(12, 20)$, the old shares are worth USD 599.0415 and USD 299.5208. Pre-trade NAV is USD 1098.2428, and execution-price weights are 54.55%, 27.27%, and 18.18%. Solving for post-fee NAV gives USD 1098.0830. Sell 4.1667 shares of A (USD 50.0000) and buy 1.4952 shares of B (USD 29.9041). The resulting shares are 45.7535 and 16.4712. Fees are USD 0.1598 and cash USD 219.6166. At closing prices $(12, 21)$, holdings are worth USD 894.9376 and closing NAV is USD 1114.5542.

The chain is now explicit:

$$\begin{aligned}\text{Data} \to \text{Signal} \to \text{Target weights} \to \text{Execution lag} \\ \to \text{Orders/fills} \to \text{Shares} \to \text{Costs} \\ \to \text{PnL} \to \text{NAV} \to \text{Metrics}.\end{aligned}$$

4. Mathematical foundations

4.1 Notation, units, and clocks

Let $\mathcal{T} = \{t_0, \ldots, t_T\}$ be a unique, strictly increasing price index and let $\mathcal{A} = \{1, \ldots, N\}$ identify the assets.

- $P_{i,t} > 0$ is the closing mark, in currency per share.
- $E_{i,t} > 0$ is the execution-price input, such as a specified open or VWAP. If omitted, the engine sets $E = P$. That is a closing-mark execution proxy, with zero post-execution price PnL, not evidence of an actual open or VWAP fill. It still executes an earlier signal, never a signal formed at that same close.
- A real-share ledger uses unadjusted trade prices and explicit corporate actions. Section 11 instead declares synthetic total-return proxy units and adds no dividends again; those units are not actual shares.

- $q_{i,t-1}$ is the previous closing share count. A split ratio $S_{i,t}$ changes it to $q'_{i,t-1} = q_{i,t-1}S_{i,t}$; absent a split, $S = 1$. A delisted holding is settled into cash and its shares set to zero.
- $q_{i,t}$ is the signed closing holding: positive for a long, negative for a short. C_t is closing cash in currency units.
- Closing NAV is $V_t = C_t + \sum_i q_{i,t}P_{i,t}$. Immediately before execution,

$$V_{t,\text{pre}} = C_{t,\text{start}} + \text{Interest}_t + \text{Div}_t + \text{DelistCash}_t + \sum_{i \notin \text{Delist}} q'_{i,t-1}E_{i,t}.$$

- V'_t is NAV just after execution and fees, still valued at E.
- $w^*_{i,t}$ is a target fraction, with $\sum_i |w^*_{i,t}| \le$ max_gross and implicit cash target $1 - \sum_i w^*_{i,t}$. Actual closing weight is $w_{i,t} = q_{i,t}P_{i,t}/V_t$.
- $c \ge 0$ is the proportional fee rate: 0.001 = 10 bps. r_{cash} is the simple cash rate for one observation interval.
- lag $\in \mathbb{Z}_{\ge 1}$ counts **price-index observations**. A signal at row k executes at row $k +$ lag. With monthly French data, lag 1 means the next monthly observation, not one day or 24 hours. Do not add integers to date strings.

4.1.1 Three denominators answer different questions

Weights are dimensionless; shares have share units; prices have currency/share units. The quantities $x_i, V_{\text{pre}}, V', \Delta M_i$ are all currency amounts. Compute target dollars $w^*_i V'$ first, then divide by the same execution price to obtain shares.

> **Pre-trade, post-fee, and closing NAV**
>
> The pre-trade denominator is V_{pre}, the target denominator is post-fee V', and the reporting denominator is closing V_t. With no fees and zero **aggregate** post-execution price PnL, all three coincide. Unchanged prices are sufficient, but offsetting gains and losses also suffice. The execution target constrains $q_{i,t}E_{i,t}/V'$, not $q_{i,t}P_{i,t}/V_t$. A's 53.14% closing weight in Section 3 is therefore consistent with a correctly executed 50% target.

4.1.2 From Project 4 targets to the ledger

Project 4's accepted signal layer emits target rows at signal time. Listed support, the 60-name threshold, and gross exposure 2 belong to that row's construction. The ledger then delays it by price-index observations and converts it into cash and shares. Gross-2 long/short targets require explicit `allow_short=True` and a compatible `max_gross`; the default long-only, gross-1 configuration cannot accept them.

The base `run_backtest` raises if a nonzero target refers to an asset already delisted at execution; it does not automatically filter targets. The A6 wrapper preserves the raw signal and filters a separate execution copy. That copy retains signal-row indices, which does not imply the strategy knew future eligibility at signal time. The cost walk-forward consumer separately checks the contemporaneous listed state in its execution loop and settles old delisted holdings first. These are different consumers.

Report the original signal target, the execution-adjusted target, and actual closing holdings separately. Eligibility changes and price drift can change industry neutrality, net, and gross exposure. Do not rewrite an old signal using an execution-day status. This teaching interface carries forward the accepted C13 behavior; it neither reruns Project 4 nor establishes real borrow, suspension, capacity, or partial-fill feasibility.

4.2 Assumptions and domain

- **A1: closed, self-financing account.** No deposits or withdrawals after initialization.
- **A2: deterministic complete execution.** Every order fills at its supplied E, without partial fills, queues, slippage, or impact.
- **A3: monotonicity domain.** $c\sum_i |w_i^*| < 1$.
- **A4: positive equity and affordable liquidation.** At a scheduled rebalance, $V_{\text{pre}} > 0$. Otherwise the run terminates at execution without trading. Under A3, a positive post-fee root additionally requires $c\sum_i |x_i| < V_{\text{pre}}$; otherwise the rebalance is rejected. This condition describes this equation and API domain, not every conceivable cost model.

5. Definitions and state transitions

5.1 The objects in the chain

Object	Input and role	Output	Condition
Signal timestamp	Information available by decision close s	Target row w_s^*	No later information
Execution lag	Number of observations from signal row to execution row	Execution at row $k + \text{lag}$	Integer lag at least 1
Execution price E	Explicit execution benchmark	Price for orders/fills	Default $E = P$ is only a closing-mark proxy
Post-fee NAV V'	$V_{\text{pre}}, x, w^*, c$	Target dollars w^*V'	A3 and positive-root feasibility
Fills table	Share changes and execution prices	Quantities, notional, fees	Cash funds net purchases and fees
Reconciliation	Opening state, flows, and closing state	Currency residual	Engineering relative check for positive finite NAV; absolute currency residual for a nonpositive terminal row

5.2 Event order within an observation

```
1. Interest on the cash balance from the previous interval
2. Corporate actions: split shares, pay dividends, settle delistings
3. Scheduled execution:
   value old holdings at E, inspect pre-trade equity
   solve post-fee NAV, form orders, update shares and cash, pay fees
4. Mark remaining holdings at closing P
5. Terminate if closing NAV is nonpositive
```

An execution-time ruin stops before step 4; later closing-price recovery is not observed by that terminated account. When there is no trade, old holdings simply carry through to the close.

5.2.1 Follow one day's cash and shares

Complete example: one day's cash and shares

Start with cash 200, eight shares, and previous price 100, so NAV is 1000. Cash earns 0.001 for the interval. Today a 2:1 split occurs before a dividend of 1 per **new** share. Let $E = 49$, $P = 50, w^* = 0.8, c = 0.002$. No asset delists. Retain full intermediate precision and round only displayed answers.

Clock/action	Calculation and reason	State/output
Previous close	Shares and cash are the state; the target is an instruction	$C = 200, q = 8, V = 1000$
Interest	$200(0.001)$, before today's trade	Interest 0.2000; cash 200.2000
Corporate action	$8(2) = 16$ shares, then dividend $16(1)$	16 shares; $C_{\text{pre}} = 216.2000$
Execution valuation	Use $E = 49$, not the later close	$x = 784, V_{\text{pre}} = 1000.2000$
NAV solution	$v = 1000.2 - 0.002\lvert 0.8v - 784\rvert$	$V' = 1000.1677$; target value 800.1342
Order and fee	$\Delta M = 0.8V' - 784$, $\Delta q = \Delta M/49$	Buy 16.1342 dollars, 0.3293 shares; fee 0.0323
Cash/shares	$C = 216.2 - \Delta M - 0.002\lvert\Delta M\rvert$	Cash 200.0335; shares 16.3293
Closing mark	Only now use $P = 50$	Holdings 816.4635; NAV 1016.4970; stock weight 80.3213%

Assuming a buy gives

$$V' = \frac{1000.2 + 0.002(784)}{1 + 0.002(0.8)} = 1000.1677.$$

Substitution yields $0.8V' - 784 = 16.1342 > 0$, confirming that trade sign. Multiple assets can have different signs but share one NAV base; we do not solve a separate portfolio NAV for each asset.

For reconciliation, the split-adjusted old price is $100/2 = 50$. Old shares incur $16(49 - 50) = -16$ before execution, offset by dividend cash 16. The **new** shares then earn $16.3293(50 - 49) = 16.3293$. Therefore

$$V_t - 1000 = 0.2 + 16 - 16 + 16.3293 - 0.0323 = 16.4970.$$

Using the old 16 shares for the second segment would omit the new purchase's closing gain.

5.3 Explicit engine policies

Boundary	Behavior	Reason
Execution-time $V_{\text{pre}} \leq 0$	Record `terminated_at`; stop without orders	Do not trade an account with no positive equity
Under A3, $c\sum\lvert x_i\rvert \geq V_{\text{pre}}$	Reject rebalance with `ValueError`	Post-fee root is nonpositive
Closing NAV ≤ 0	Record termination; truncate later dates	No resurrection of the account
NaN targets	Reject; write 0 for zero exposure	Investment intent must be explicit
Duplicate/unsorted dates	Reject	Unique increasing clock required
Missing/nonpositive held or traded price	Reject; no forward-fill	A fill or valuation cannot use an invented price
Target gross above `max_gross`	Reject	Explicit exposure limit
Negative target without `allow_short=True`	Reject	Short model must be explicitly selected
Target already equals holdings	No fills and no fees	Do not inflate turnover with empty records
All-zero target	Liquidate to cash	Cash is a valid allocation

5.4 Three boundaries, three stopping points

1. At a scheduled execution, $V_{\text{pre}} \leq 0$ stops the run at E before a NAV solution or trade. A favorable later close is excluded.
2. Positive execution equity with unaffordable liquidation is a rejected input configuration, not a negative-NAV trade curve. For long value 1000, short value -990, cash 0, and $c = 0.01$, equity is only 10 while liquidation fees are 19.9. Under A3 the root is nonpositive. Equality gives a zero root and is also rejected.
3. A valid trade can still be followed by closing NAV ≤ 0 if $E \to P$ losses exhaust equity. That run stops at the closing mark. Ordinary portfolio weights no longer have a positive denominator.

Negative cash pays interest: -100 at rate 0.001 becomes -100.1. The simplified engine credits all short-sale proceeds to cash. It does not model borrow fees, rebates, segregated collateral, separate margin funding rates, or margin calls. `allow_short=True` selects that simplified convention; Challenge 29-C02 develops an extension rather than claiming it already exists.

6. Derivations

6.1 The self-financing reconciliation identity

Let the previous closing state be $(C_{t-1}, q_{t-1}, P_{t-1})$. In a closed account, the following operations fully determine the next state.

1. Interest is $I_t = C_{t-1} r_{\text{cash}}$.
2. After a split, $q'_{i,t-1} = q_{i,t-1} S_{i,t}$ and the comparable old price is $\widetilde{P}_{i,t-1} = P_{i,t-1}/S_{i,t}$. Their product equals the pre-split value.

3. Dividend cash is $D_t = \sum_i q'_{i,t-1} D_{i,t}$. For delisted names, cash settlement is $K_t = \sum_{j\in\mathcal{D}} q'_{j,t-1} K_{j,t}$, and their previous marked value is $L_t = \sum_{j\in\mathcal{D}} q'_{j,t-1} \widetilde{P}_{j,t-1}$.
4. For live names, $x_{i,t} = q'_{i,t-1} E_{i,t}$ and

$$V_{t,\mathrm{pre}} = C_{t-1} + I_t + D_t + K_t + \sum_{i\notin\mathcal{D}} x_{i,t}.$$

5. Trade $\Delta q_{i,t} = q_{i,t} - q'_{i,t-1}$ and pay $F_t = c \sum_{i\notin\mathcal{D}} |E_{i,t}\Delta q_{i,t}|$. Cash conservation gives

$$C_t = C_{t-1} + I_t + D_t + K_t - \sum_{i\notin\mathcal{D}} E_{i,t}\Delta q_{i,t} - F_t.$$

6. Closing NAV is $V_t = C_t + \sum_i q_{i,t} P_{i,t}$.

Substitute the cash equation into the NAV difference:

$$\begin{aligned}
V_t - V_{t-1} &= \left(C_t + \sum_i q_{i,t}P_{i,t}\right) - \left(C_{t-1} + \sum_i q'_{i,t-1}\widetilde{P}_{i,t-1}\right) \\
&= I_t + D_t + K_t - F_t - \sum_{i\notin\mathcal{D}} E_{i,t}(q_{i,t} - q'_{i,t-1}) \\
&\quad + \sum_i q_{i,t}P_{i,t} - \sum_i q'_{i,t-1}\widetilde{P}_{i,t-1} \\
&= I_t + D_t + (K_t - L_t) - F_t \\
&\quad + \sum_{i\notin\mathcal{D}} q'_{i,t-1}(E_{i,t} - \widetilde{P}_{i,t-1}) \\
&\quad + \sum_{i\notin\mathcal{D}} q_{i,t}(P_{i,t} - E_{i,t}).
\end{aligned}$$

The six components are interest, net dividends, delisting PnL, pre-execution price PnL on old shares, post-execution price PnL on new shares, and negative fees. This is an exact algebraic identity, not an approximation.

Define

$$\mathrm{residual}_t = (V_t - V_{t-1}) - \mathrm{explained}_t.$$

Residuals have currency units (USD). Define $|\mathrm{residual}_t|/V_t$ only when V_t is positive and finite. The threshold 10^{-12} is an engineering check for this implementation at the stated scale. For a nonpositive terminal NAV, report the absolute currency residual separately; do not divide by zero or a negative NAV. Investigate nonfinite NAV as an invalid result. This threshold is not a universal floating-point truncation bound. The inherited USD 100,000 constructed-account run reported a largest absolute residual around 3.1×10^{-11} dollars; report absolute and relative errors separately.

At execution-time ruin, the final observation is at E, not the close, so omit the $E \to P$ term. For example, initial equity 100 with a short of ten shares at price 10 leaves cash 200. If the next execution price is 150, equity is $200-10(150) = -1300$, a change of -1400. A later close at 5 would imply a USD 1450 recovery, but the stopped account does not observe it. The existing `test_execution_ruin_reconciles_without_later_close_recovery` checks this boundary.

A zero residual establishes internal conservation. It does not establish correct prices, legal information times, or absence of look-ahead features.

6.2 Post-fee NAV: equation, closed form, and uniqueness

At execution, old holdings have values $x_i = q_{i,\text{pre}} E_i$ and pre-trade NAV $V_{\text{pre}} = C_{\text{pre}} + \sum_i x_i$. Targets require new values $w_i^* V'$, so traded dollars are

$$\Delta M_i = w_i^* V' - x_i.$$

Fees are $c|\Delta M_i|$, hence

$$V' = V_{\text{pre}} - c \sum_i |w_i^* V' - x_i|.$$

Theorem 1: existence, uniqueness, and positive-root feasibility

If $c \sum_i |w_i^*| < 1$, the continuous piecewise-linear function

$$f(v) = v + c \sum_i |w_i^* v - x_i| - V_{\text{pre}}$$

has exactly one real root. For $V_{\text{pre}} > 0$, that root lies in $(0, V_{\text{pre}}]$ if and only if $c \sum_i |x_i| < V_{\text{pre}}$.

Proof. Away from kinks,

$$f'(v) = 1 + c \sum_i w_i^* \operatorname{sgn}(w_i^* v - x_i) \geq 1 - c \sum_i |w_i^*| > 0.$$

All one-sided slopes share this positive lower bound. Thus f is strictly increasing, tends to $-\infty$ as $v \to -\infty$ and to $+\infty$ as $v \to +\infty$. Continuity and monotonicity imply one real root.

At the upper endpoint,

$$f(V_{\text{pre}}) = c \sum_i |w_i^* V_{\text{pre}} - x_i| \geq 0,$$

so the root is at most V_{pre}. At zero,

$$f(0) = c \sum_i |x_i| - V_{\text{pre}}.$$

Strict monotonicity gives $V' > 0$ exactly when $f(0) < 0$.

For $x = (1000, -990), C = 0, V_{\text{pre}} = 10, c = 0.01$, liquidation costs 19.9 and $f(0) = 9.9 > 0$. Under the theorem's target-gross condition, the unique root is nonpositive. `run_backtest` rejects that rebalance with an infeasibility error. A nonpositive pre-trade NAV instead triggers termination without a trade. ■

Theorem 2: closed form with known trade signs

Let $s_i = \operatorname{sgn}(w_i^* V' - x_i) \in \{-1, 0, 1\}$. Then

$$|w_i^* V' - x_i| = s_i(w_i^* V' - x_i).$$

For a zero-trade term, $s_i = 0$ is valid; either sign ± 1 also gives the same zero product. After solving, verify all assumed signs.

Expand and collect terms:

$$\begin{aligned} V' &= V_{\text{pre}} - c\sum_i s_i(w_i^* V' - x_i), \\ V'\left(1 + c\sum_i s_i w_i^*\right) &= V_{\text{pre}} + c\sum_i s_i x_i, \\ V' &= \frac{V_{\text{pre}} + c\sum_i s_i x_i}{1 + c\sum_i s_i w_i^*}. \end{aligned}$$

Under A3 the denominator is positive.

All buys from cash. If $x = 0$ and all nonzero targets are long, all nonzero trades are buys and

$$V' = \frac{V_{\text{pre}}}{1 + c\sum_i w_i^*}. \qquad \frac{1000}{1 + 0.002(0.5 + 0.3)} = 998.4026.$$

One sale and one purchase. If asset 1 is sold and asset 2 bought,

$$V' = \frac{V_{\text{pre}} + c(x_2 - x_1)}{1 + c(w_2^* - w_1^*)}.$$

Newton/subgradient steps: exact theory and floating point

Use the ordinary derivative at a smooth point and a legal subgradient at a kink:

$$d_k = 1 + c\sum_i w_i^* s_{i,k}, \qquad s_{i,k} = \begin{cases} \operatorname{sgn}(w_i^* v_k - x_i), & w_i^* v_k - x_i \neq 0, \\ \text{any value in } [-1, 1], & w_i^* v_k - x_i = 0. \end{cases}$$

Stop first if $f(v_k) = 0$. Otherwise update $v_{k+1} = v_k - f(v_k)/d_k$. A3 gives $d_k \geq 1 - c\sum_i |w_i^*| > 0$.

1. The function is continuous, convex, and piecewise linear. Starting at $v_0 = V_{\text{pre}}$ puts us to the right of the root because $f(v_0) \geq 0$.
2. The supporting-line inequality at the root is $0 = f(V') \geq f(v_k) + d_k(V' - v_k)$. Rearranging gives
$$V' \leq v_k - \frac{f(v_k)}{d_k} \leq v_k.$$
The second inequality is strict until the root is reached. Iterates decrease without crossing the root.
3. Only terms with $w_i^* \neq 0$ have possible kinks at x_i/w_i^*. A zero weight produces a constant term; never divide by it. At most N distinct kinks create at most $N + 1$ affine intervals.

4. Within an affine interval a Newton step reaches that line's zero. If it is not the true root, the step must cross into an interval to the left. A nonterminal step taken exactly at a kink also moves left past that kink. There are finitely many intervals, so exact arithmetic reaches the root in at most $N + 1$ updates. This is a geometric theorem, not a floating-point test report for every dimension. ■

The actual solver selects `np.sign(0)=0`, stops when $|f(v)| \le 10^{-12}V_{\text{pre}}$, and raises after 100 unsuccessful iterations. Rounding, tolerance, and that cap remain implementation conditions; finite exact termination does not promise measured zero error for every input.

6.3 The geometry of weight drift

Assume fixed shares, no corporate actions or external flows, zero cash interest, and positive beginning and ending NAV. With $r_{i,t+1} = P_{i,t+1}/P_{i,t} - 1$,

$$r_{p,t+1} = \frac{V_{t+1}}{V_t} - 1 = \sum_i w_{i,t} r_{i,t+1}.$$

Since $q_i P_{i,t+1} = w_{i,t} V_t (1 + r_{i,t+1})$,

$$w_{i,t+1} = w_{i,t} \frac{1 + r_{i,t+1}}{1 + r_{p,t+1}}.$$

1. **Signed direction matters.** Subtraction gives

$$w_{i,t+1} - w_{i,t} = w_{i,t} \frac{r_{i,t+1} - r_{p,t+1}}{1 + r_{p,t+1}}.$$

   A positive weight rises when the asset outperforms the portfolio. A negative weight becomes more negative: $w_i = -0.2, r_i = 10\%, r_p = 0$ gives -0.22. Its numerical weight falls while absolute short exposure increases. A zero holding or equal relative return can leave a weight unchanged.
2. **Normalization follows from conservation.** Including cash,

$$\sum_i w_{i,t+1} + w_{\text{cash},t+1} = \frac{\sum_i w_{i,t}(1 + r_{i,t+1}) + w_{\text{cash},t}}{1 + r_{p,t+1}} = 1.$$

3. **Drift creates reset trades.** For small fees,

$$\text{Turnover} \approx \sum_i |w_i^* - w_{i,\text{drift}}|.$$

   The approximation uses a pre-trade weight difference, while exact orders use V' and execution-price values. If E differs from the previous close, revalue before measuring drift. Return dispersion can enlarge reset trades in a specified example; it does not guarantee monotone turnover for every long/short portfolio and target.

6.4 Execution lag and future-perturbation checks

"Compute a score after today's close and buy at that same close" reverses the order of events. Closing prices must first be published, ingested, and processed; the closing auction has already ended. Under this close-decision contract a signal at row k can execute only at row $k + \text{lag}$, with lag at least 1. The base engine rejects smaller lag. A daily next row is usually the next trading observation; a monthly next row is a month, not a day.

Let D, W denote prices and targets. Hold the historical information, asset identifiers, initial state, execution calendar, and policy fixed. A legal perturbation after row k should leave historical state on common coordinates unchanged:

$$\mathcal{L}_{0:k}(D, W) = \mathcal{L}_{0:k}(\widetilde{D}, \widetilde{W}).$$

Adding an asset can change the output schema. Align common historical identifiers and separately check the new historical zero holdings rather than demanding byte-identical whole DataFrames of different shapes.

The existing `test_future_perturbation_does_not_change_past` covers specific future price and target changes; the historical CLI has its own constructed checks. Finite perturbations can reveal leakage along tested paths. They do not prove invariance to every future modification. Structural state-transition reasoning and point-in-time input checks are still required, as Chapter 30 explains.

6.5 When a vectorized calculation represents the same ledger

A common exploratory expression is

```
portfolio_return = (target_weights.shift(1) * returns).sum(axis=1)
```

Theorem 3: sufficient conditions for equality

Let $r_t^{\text{ledger}} = V_t/V_{t-1} - 1$. Compare it with `drifted_weight_returns`, which computes returns using the old drifted weights and then resets weights at a specified trading close.

First align initial cash/holdings, **already lagged execution dates**, and the rebalancing policy. The helper takes trade dates, not signal dates. Then the following are sufficient for period-by-period equality:

1. Zero fees, $c = 0$.
2. Zero cash rate.
3. Execution at the closing mark, $E = P$, for an earlier signal.
4. Exact passive drift between rebalances and no corporate actions outside the vectorized model.

To see why, start from the same state. Between two closing executions both models apply the same asset returns to the same exposures. At a common execution close, no fees or price gap change wealth, and both reset to the same target. Induction gives equal NAV returns at every observation.

The conditions are sufficient, not necessary. If there are no trades in a window, a positive configured fee rate causes zero actual fees. With aligned state, zero cash interest, and $E = P$, equality can still hold there.

Counterexample: a fixed target is not a drifting holding. Initially A and B each have value 50 in a NAV-100 account. A gains 100% on day 1 and loses 50% on day 2; B is unchanged. Do not rebalance in between.

- Day 1 NAV becomes 150; actual weights are 2/3 and 1/3.
- Day 2 A falls from 100 to 50, B remains 50, and NAV returns to 100. The second-day return is $100/150 - 1 = -33.33\%$; cumulative return is 0%.
- Reusing 50/50 weights gives day-1 return 50% and day-2 return $0.5(-0.5) = -25\%$. Compounding yields $1.5(0.75) - 1 = 12.5\%$.

That extra 12.5% comes from an implicit free reset of A's weight. If a strategy explicitly rebalances daily and meets the zero-cost assumptions, fixed-weight dot products can describe that different strategy. The error is calling it the original no-rebalance strategy.

7. Financial interpretation

7.1 Fees reduce the compounding base

With average one-way monthly turnover T_{month}, immediate fees are roughly cT_{month} of NAV. Later returns compound on the remaining wealth.

The inherited French 10-industry example spans 1926-07 through 2026-08, or 1202 monthly observations. Monthly equal-weight rebalancing has mean turnover around 2.38% (28.6% annualized):

- At 0 bps, terminal NAV is USD 353,584,236; CAGR 11.02% (0.11022), annual volatility 17.80%, and annual Sharpe 0.495.
- At **10 bps**, $c = 0.001$, or one thousandth, terminal NAV is USD 343,599,177; CAGR 10.99% (0.10990), cumulative fees USD 1,013,676, and annual CAGR drag 3.17 bps.
- At 50 bps, $c = 0.005$, terminal NAV is USD 306,406,322; CAGR 10.86% (0.10863), cumulative fees USD 4,571,698, and annual drag 15.86 bps.

At 10 bps, $0.0238(0.001) = 0.00238\%$ of NAV per average month, under 0.24 bps. Over the historical century, the 50-bps scenario's terminal wealth is over USD 47 million lower, roughly 13.3%. This is a scenario comparison on the same return proxy, not a new market-cost estimate.

7.2 Real shares and total-return proxy units

A real-share ledger submits orders at unadjusted prices and handles splits and dividends explicitly. In a 2:1 split from price 100 to 50, shares double; changing price alone invents a loss. A dividend-inclusive index already includes the dividend benefit and must not receive that cash flow again.

Section 11 uses another coherent teaching model: synthetic total-return proxy units, with no added dividend or split events. It permits consistent proportional-fee comparisons but does not establish real whole-share, borrow, or opening-price execution. If prices are multiplied by a constant and fractional shares divided by that constant, dollar holdings and purely proportional fees are unchanged. Whole-share rules, fixed fees, and actual entitlement units require separate treatment.

A constant rescaling of back-adjusted historical prices does not automatically make returns forward-looking. Leakage depends on whether a feature uses unavailable future levels, adjustment information, or events. Check each model's units and information contract rather than declaring every adjusted series invalid.

8. Complete numerical example

Complete example: four dates of entry, drift, rebalancing, and reconciliation

This is the frozen two-asset hand example from `qa/a6/claude/plan.md`. The calculation below retains intermediate precision and rounds only displayed values.

8.1 Inputs

Initial cash is USD 1000.0000; $c = 0.002$, cash rate is zero, and targets are A 0.5, B 0.3, cash 0.2. Lag is one price-index observation.

Row	Date	E_A	E_B	P_A
0	2026-01-30	10.0	20.0	10.0
1	2026-02-02	10.0	20.0	11.0
2	2026-02-27	12.0	19.5	12.1
3	2026-03-02	12.0	20.0	12.0

Row	P_B	Event
0	20.0	First closing signal
1	19.0	First execution, then closing mark
2	19.0	Hold and drift; second closing signal
3	21.0	Second execution, then closing mark

8.2 January 30: form the signal

The account remains entirely in cash: $q_A = q_B = 0, C = 1000, V_0 = 1000$. The closing signal requests 50/30/20. Lag 1 means no trade on this first row.

Continued: the same complete example

8.3 February 2: execute the initial allocation

1. **Pre-trade state.** Cash is 1000, old holdings $x_A = x_B = 0$, and $V_{1,\text{pre}} = 1000$.
2. **Solve the allocation base.** Both nonzero trades are buys:
$$V_1' = \frac{1000}{1 + 0.002(0.5 + 0.3)} = \frac{1000}{1.0016} = 998.4026.$$
3. **Shares and fees.** A's target dollars are $0.5V_1' = 499.201278$, so $q_A = 499.201278/10 = 49.92013$ and its fee is 0.9984. B's target dollars are $0.3V_1' = 299.520767$, so $q_B = 299.520767/20 = 14.97604$ and its fee is 0.5990. Total fees are 1.5974.
4. **Cash.** $C_1 = 1000 - 499.201278 - 299.520767 - 1.597444 = 199.6805$. With unrounded values, $C_1/V_1' = 0.2$ exactly.
5. **Closing mark.** A is worth $49.920128(11) = 549.121406$ and B $14.976038(19) = 284.544728$. Holdings total 833.666134, giving
$$V_1 = 199.680511 + 833.666134 = 1033.3466.$$
The period return is $1033.346645/1000 - 1 = 3.3347\%$; closing weights are 53.14%, 27.54%, and 19.32%.

8.4 February 27: hold, drift, decide again

1. No trade occurs: shares remain 49.920128 and 14.976038; cash stays 199.680511 and fees are zero.
2. At $(12.1, 19)$, A is worth 604.033546, B 284.544728, and holdings total 888.578275. Hence
$$V_2 = 199.680511 + 888.578275 = 1088.2588.$$
The return is $1088.258786/1033.346645 - 1 = 5.3140\%$.
3. Divide by closing NAV:
$$w_A = 55.50\%, \qquad w_B = 26.15\%, \qquad w_{\text{cash}} = 18.35\%.$$
4. At this close, form the second 50/30/20 target. It executes at the next indexed observation.

8.5 March 2: rebalance

1. **Value old holdings at execution prices.** With $E = (12, 20)$, $x_A = 599.041534$, $x_B = 299.520767$ and
$$V_{3,\text{pre}} = 199.680511 + 599.041534 + 299.520767 = 1098.2428.$$
2. **Determine and verify signs.** A's pre-trade share of NAV is about 54.55%, above 50%; B's is 27.27%, below 30%. Try $s_A = -1, s_B = +1$:
$$\begin{aligned} V_3' &= \frac{1098.242812 - 0.002(599.041534 - 299.520767)}{1 + 0.002(-0.5 + 0.3)} \\ &= \frac{1097.643770}{0.9996} = 1098.0830. \end{aligned}$$

Continued: the same complete example

3. **Trade and pay fees.** A's target value is 549.041502, giving $\Delta M_A = -50.000032$, $\Delta q_A = -4.166669$, new shares 45.75346, and fee 0.1000. B's target value is 329.424901, giving $\Delta M_B = 29.904134$, $\Delta q_B = 1.495207$, new shares 16.47125, and fee 0.0598. Both assumed signs are confirmed; total fees are 0.1598.
4. **Update cash.**

$$\begin{aligned} C_3 &= C_2 - (\Delta M_A + \Delta M_B) - \text{Fees}_3 \\ &= 199.680511 - (-50.000032 + 29.904134) - 0.159808 \\ &= 219.6166. \end{aligned}$$

   Unrounded cash is exactly $0.2V_3'$.
5. **Mark at the close.** At $(12, 21)$, A is worth about 549.041496 and B 345.896145. Carrying unrounded shares gives holdings 894.937648 and

$$V_3 = 219.616601 + 894.937648 = 1114.5542.$$

   The period return is $1114.554248/1088.258786 - 1 = 2.4163\%$. Closing weights are A 49.26%, B 31.03%, and cash 19.70%. Small differences among sums of displayed components are rounding, not extra cash flows.

8.6 Reconcile all four observations

Date	NAV change	Before-trade price PnL	After-trade price PnL	Negative fees
2026-01-30	0.0000	0.0000	0.0000	0.0000
2026-02-02	33.3466	0.0000	34.9441	−1.5974
2026-02-27	54.9121	54.9121	0.0000	0.0000
2026-03-02	26.2955	9.9840	16.4712	−0.1598

Date	Explained	Residual	Cash-identity difference
2026-01-30	0.0000	0	0
2026-02-02	33.3466	1.56×10^{-13}	0
2026-02-27	54.9121	-7.82×10^{-14}	0
2026-03-02	26.2955	-7.11×10^{-15}	2.84×10^{-14}

These residuals are from the inherited frozen run, not a newly rerun historical experiment. The independent hand calculation uses unrounded intermediate values. The inherited largest residual is 1.56×10^{-13} dollars, and the largest cash-identity difference 2.84×10^{-14} dollars.

9. Python Implementation

The default reader route is `notebooks/ch29_backtest_engine.ipynb` in the accepted I3 reader package. From that package's root, run:

```
python run_notebook.py notebooks/ch29_backtest_engine.ipynb
```

The Notebook defaults to `RUN_FRENCH=False`. It computes the four-date ledger, weight drift, `constructed_study`, and extreme-policy teaching examples. The historical French tables and figures in Section 11 are not new outputs from this default run. The runner saves the executed Notebook and `execution.json` under a unique `runs/execution_<stem>_<uuid>/`, including failed executions. The Notebook also creates its own `runs/ch29_backtest_engine_<uuid>/`; its main results are in-kernel tables and printed output, so do not assume everything is saved to that inner directory. Read Section 11's data boundary first. The chapter Notebook's English language layer, a portable complete package, and cold-start verification remain pending BIL23; this chapter is not a promise of an independently portable English software package.

All core backtesting and reconciliation functions in this chapter are implemented in `src/quant_course/backtest.py`. The code below shows how to call the engine in a standalone Python environment and reconcile the results end to end:

```
import numpy as np
import pandas as pd
from quant_course.backtest import BacktestConfig, reconcile, run_backtest,
    ↪ solve_post_fee_nav

# 1. Build the price and signal data
dates = pd.DatetimeIndex(["2026-01-30", "2026-02-02", "2026-02-27", "2026-03-02"])
mark_prices = pd.DataFrame(
    {"A": [10.0, 11.0, 12.1, 12.0], "B": [20.0, 19.0, 19.0, 21.0]},
    index=dates,
)
exec_prices = pd.DataFrame(
    {"A": [10.0, 10.0, 12.0, 12.0], "B": [20.0, 20.0, 19.5, 20.0]},
    index=dates,
)
target_weights = pd.DataFrame(
    {"A": [0.5, 0.5], "B": [0.3, 0.3]}, index=[dates[0], dates[2]]
)

# 2. Configure and run the backtest
config = BacktestConfig(initial_cash=1000.0, fee_rate=0.002, lag=1)
result = run_backtest(
    mark_prices, target_weights, config, exec_prices=exec_prices
)

# 3. Print the holdings table and trade blotter
print("Fill count:", len(result.fills))
print("Final NAV:", round(result.ledger["nav"].iloc[-1], 4))

# 4. Check the daily reconciliation
rec = reconcile(result, mark_prices, config, exec_prices=exec_prices)
max_residual = rec["residual"].abs().max()
```

```
print("Maximum reconciliation residual:", max_residual)
assert max_residual < 1e-12
```

Commands and script output: Advanced entry points: inspect the full course CLI, its parameters, historical branches, and output paths before choosing to run it or the tests:

```
.venv/bin/python code/ch29_backtest_engine.py
.venv/bin/python -m pytest -q tests/test_ch29_backtest.py tests/test_ch29_workflow.py
```

These full-course entry points may generate outputs, call historical data, or use fixed default directories. They are separate from the default offline reader route. Check the actual scripts and parameters before running, and protect frozen qa results. This revision does not execute the runner, complete Notebook, CLI, course suite, or French study.

---

10. Interpreting the Figures

The four figures below are retained from the historical outputs of `code/ch29_backtest_engine.py`; they were not redrawn or rerun for this batch. Full captions explain the crowded local titles and legends. The summaries do not contain the complete NAV paths, so they cannot be used to reconstruct full curves. The original figures are stored in `figures/a6/`:

10.1 Asset Decomposition and Reconciliation Residuals in the Hand-Worked Ledger

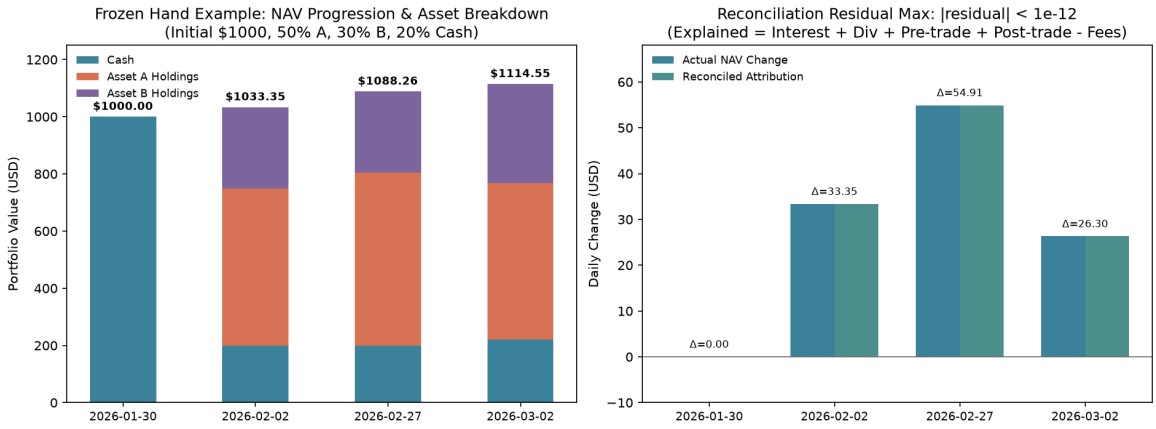


Figure 1: Asset decomposition and reconciliation residuals in the hand-worked ledger

- **Left panel (asset composition bars):** The x-axis shows dates and the y-axis shows total portfolio assets (USD). It shows how the initial USD 1,000 in cash is converted into assets A and B plus a cash reserve on 2026-02-02; NAV then expands as A rises on 02-27, and the portfolio rebalances successfully on 03-02 to reach USD 1,114.55.
- **Right panel (attribution comparison):** The x-axis shows dates and the y-axis shows daily NAV changes (USD). Actual NAV changes (dark blue bars) closely match the changes explained by the reconciliation attribution (teal bars); the maximum absolute residual is less than 1.57×10^{-13} dollars.

- **Key observation:** Assets and cash satisfy the self-financing conservation identity at every point, with each flow clearly accounted for.

10.2 Weight Drift and Rebalancing Turnover

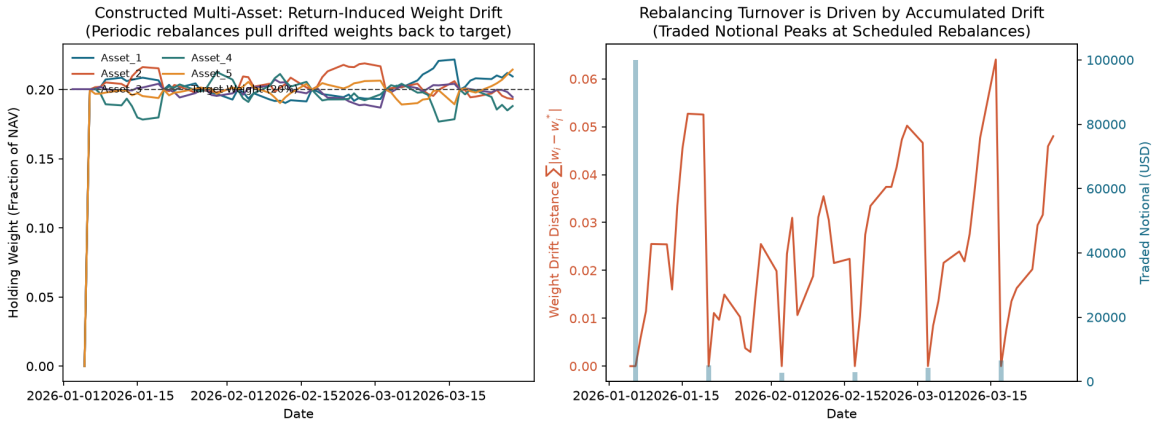


Figure 2: Weight drift and rebalancing turnover

- **Left panel (holdings-weight paths):** The x-axis shows dates and the y-axis shows portfolio weights. In the constructed market with 5 assets, the target weight is fixed at 20% (black dashed line). The five asset lines in the crowded legend at the left of the original figure are the actual weights of the five assets; the black dashed line represents their common target and is not an additional asset path. The curves retain their historical identity. During each 10-trading-day interval between rebalances, relative asset returns produce a pronounced sawtooth pattern of divergence; weights return to 20% on each rebalancing date.
- **Right panel (drift distance and traded notional):** The x-axis shows dates. The left axis shows the L_1 distance between holdings weights and target weights, $\sum |w_i - w_i^*|$ (orange line); the right axis shows actual traded notional (blue-gray bars).
- **Key observation:** Rebalancing turnover is a direct consequence of accumulated weight drift. In this fixed constructed setup, larger drift is associated with larger reset trades; cross-sectional dispersion alone is not a general monotonic-turnover theorem.

10.3 Vectorized Backtest Compared with the Event Ledger

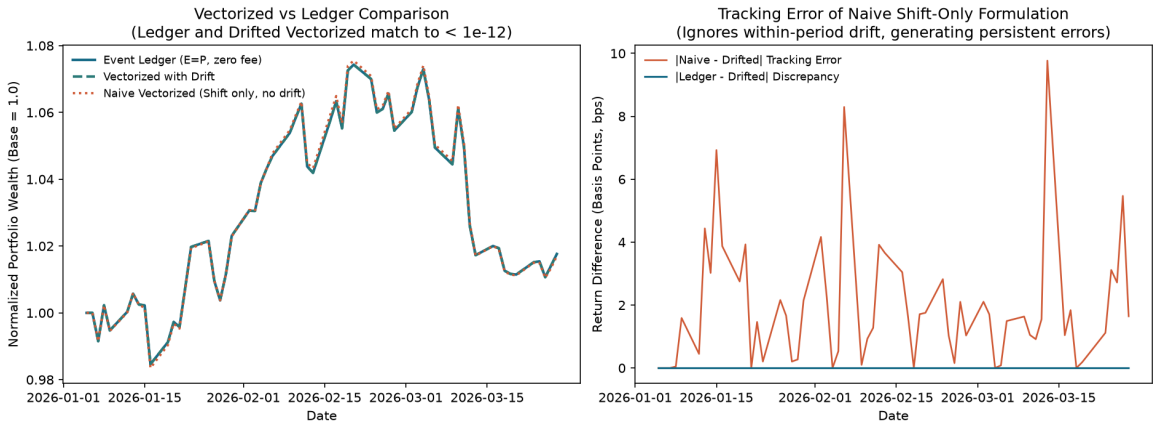


Figure 3: Vectorized backtest compared with the event ledger

- **Left panel (NAV paths):** The x-axis shows dates and the y-axis shows normalized NAV. The dark blue line (event ledger with zero fees and closing-price execution) and the green dashed line (vectorized algorithm with weight drift) **overlap exactly (difference** $< 10^{-15}$**)**; the orange dotted line (naive `shift` vectorization) diverges visibly.
- **Right panel (single-period tracking error):** The x-axis shows dates and the y-axis shows return differences (basis points, bps). The dark blue line (difference between the ledger and drift-aware vectorization) is zero to machine precision. The orange line (difference between naive vectorization and actual returns) shows persistent deviations on non-rebalancing dates, with a maximum daily deviation of about **9.77 bps** (0.000977).
- **Key observation:** Vectorized backtests that ignore weight drift have material model bias and cannot replace a ledger based on actual holdings.

10.4 Trading-Cost Sensitivity and Compounding Drag (Kenneth French 10 Industries)

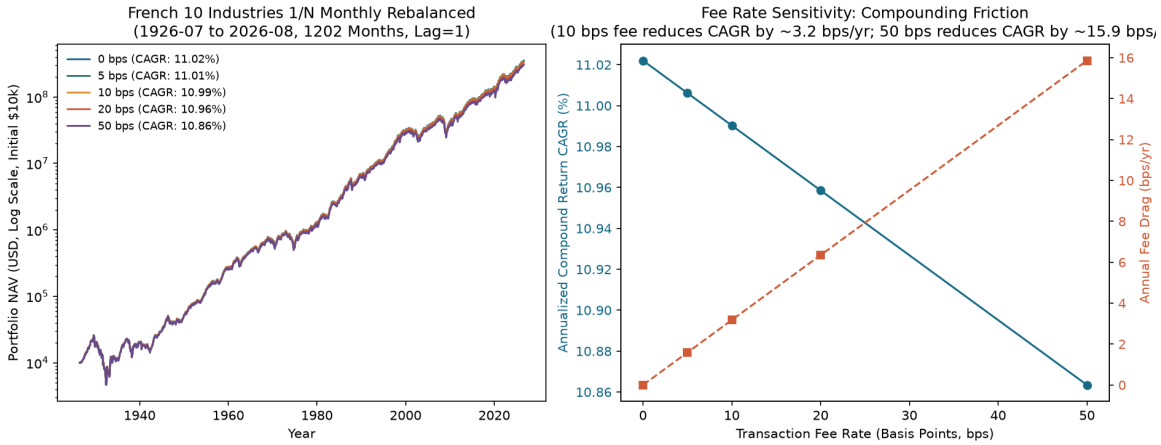


Figure 4: Trading-cost sensitivity and compounding drag (Kenneth French 10 industries)

- **Left panel (full-sample log NAV curves):** The x-axis shows years (1926–2026) and the y-axis shows portfolio NAV on a logarithmic scale (initial value USD 10,000). It shows the divergence in NAV paths under five fee rates: 0 bps, 5 bps, 10 bps, 20 bps, and 50 bps.
- **Right panel (annualized CAGR and friction drag; the original title is cut off at the edge, and this gives its full meaning):** The x-axis shows the trading fee rate (bps), the left axis shows net annualized compound growth rate (CAGR, %), and the right axis shows annualized performance drag (bps).
- **Key observation:** As the fee rate rises from 0 bps to 50 bps, the annualized return falls from 11.02% to 10.86% (each 10 bps of fees lowers CAGR by about 3.2 bps per year; a 50 bps fee lowers CAGR by about 15.9 bps per year).

11. A Ledger Case Using Historical Return Proxies

11.1 Historical Data Specification and Sources

This section retains an earlier historical proxy-ledger study. French raw, extracted, and processed data are not bundled in the default I3 reader package; the paths below identify sources in the full project. Data-use rights, subject, and purpose remain unresolved. Public availability is not authorization to redistribute data or execute research. Acceptance of the A10-A limited-scope design and A10-A.1 technical protocol does not replace data rights or execution authorization; G05/A10-B remains unauthorized. Aggregated monthly industry returns are not company-level point-in-time (PIT) inputs. Real membership, tradable prices, corporate actions, borrow/financing, capacity, and trading calendars require separate checks. These curves do not demonstrate deployable market returns. This section reports an empirical analysis using monthly value-weighted returns for the **10 Industry Portfolios** published by the Kenneth French Data Library:

- **Data file:** `data/processed/french_10_industries_monthly.csv`
- **Checksum (SHA-256):** `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950` (the corresponding manifest is `data/manifests/french_industries_2026-10-05.json`, SHA-256 `3b71cd5f781f9790cc380195c22635e03e9c9e27a5f43cd2d0d46fb12efa8271`)
- **Risk-free rate file:** `data/processed/french_factors_3_monthly.csv` (SHA-256 `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`).
- **Sample period:** July 1926 through August 2026 (1,202 calendar months in total).
- **Industries:** NoDur (consumer nondurables), Durbl (consumer durables), Manuf (manufacturing), Enrgy (energy), HiTec (high technology), Telcm (telecommunications), Shops (retail and wholesale), Hlth (healthcare), Utils (utilities), and Other.
- **Conventions and timing:**
  1. The data contain only discrete month-end observations. The backtest therefore sets lag = 1: an industry-allocation target calculated at month-end s is filled at the proxy price level at the next month-end, $s+1$ (a full one-month execution lag; the execution price E equals the mark price P at that point).
  2. After adding a base price level of 100 at 1926-06, the first return month, 1926-07, enters at that month-end proxy price, $E = P$. There is no pre-entry equity-price PnL. Its return is zero when $c = 0$ and negative from entry fees when $c > 0$; holding-period price returns start subsequently. The 1202 return observations include this initial execution month; do not replace them with 1201 months and re-estimate the frozen CAGR.
  3. The number of years used for CAGR is based on the total number of months: $Y = 1202/12 \approx 100.167$ years, and $\text{CAGR} = (\text{NAV}_T/\text{NAV}_0)^{12/1202} - 1$.
  4. The Sharpe ratio uses aligned monthly Kenneth French FF3 risk-free rates (RF, in decimal form). It is annualized by multiplying the mean monthly excess return divided

by its sample standard deviation by $\sqrt{12}$; its statistical properties are discussed in Chapter 32.

5. The industry total-return index series is a **Total-Return Proxy Level**. It includes reinvested dividends and is not an individual-stock trading price. The backtest does not add external cash dividends a second time.

11.2 Strategy Design and Backtest Results

The strategy is a simple **monthly 1/N rebalancing strategy**:

- **Decision frequency:** Set target weights at each month-end (10% per industry).
- **Execution:** Execute at the following month-end using lag = 1, with initial cash of USD 10,000.
- **Fee rates:** $c \in \{0, 5, 10, 20, 50\}$ bps.

Measured Performance Comparison (1926-07 to 2026-08)

Holdings path and performance (same fee-rate experiment):

Trading fee tier	Ending NAV (USD)	Cumulative total return	Annualized return (CAGR)	Annualized volatility	Annualized Sharpe ratio
0 bps (frictionless)	**353,584,236**	+3,535,742%	**11.02%**	17.80%	0.495
5 bps	348,555,862	+3,485,459%	**11.01%**	17.80%	0.495
10 bps	**343,599,177**	+3,435,892%	**10.99%**	17.80%	0.494
20 bps	333,896,778	+3,338,868%	**10.96%**	17.80%	0.492
50 bps	**306,406,322**	+3,063,963%	**10.86%**	17.80%	0.487

Trading frictions and realized turnover:

Trading fee tier	Cumulative fees (USD)	Annualized fee drag (bps)	Average monthly turnover
0 bps (frictionless)	0.00	0.0	2.38%
5 bps	513,420	1.59	2.38%
10 bps	1,013,676	**3.17**	2.38%
20 bps	1,975,717	6.35	2.38%
50 bps	4,571,698	**15.86**	2.38%

Empirical findings using the historical data:

1. **Equivalence check:** At 0 bps, the maximum absolute difference between the monthly return series from the ledger and the vectorized output from `drifted_weight_returns` is 6.52×10^{-16}, again confirming Theorem 3 at machine precision.
2. **Measured drift bias:** If rebalancing is changed to once every 12 months (annual rebalancing), the naive `shift` vectorization that ignores within-year drift reaches a maximum monthly tracking deviation of **2.41%** (241 basis points).
3. **Cost sensitivity:** Average monthly turnover for the 1/N strategy is about 2.38% (about 28.6% annualized). As the fee rate rises from 0 bps to 50 bps, the annualized Sharpe ratio

declines gradually from 0.495 to 0.487, CAGR falls by about 15.9 basis points, and ending NAV declines by about USD 47.18 million (about 13.3%).

---

12. Common Errors

12.1 Error 1: Filling a Closing Signal at the Same Close (Look-Ahead Fill)

- **Why it is wrong:** If a signal is calculated at the close and filled at that same closing price, the backtest effectively uses the full day's future price movement to trade retroactively.
- **Verification code:** The backtest engine enforces lag ≥ 1 and rejects any zero-lag input that would look ahead.

```
import pandas as pd
import pytest
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.date_range("2026-01-05", periods=3, freq="B")
mark = pd.DataFrame({"A": [10.0, 11.0, 12.0]}, index=dates)
tw = pd.DataFrame({"A": [1.0]}, index=[dates[0]])

# The engine raises ValueError when it detects lag=0 (a closing signal filled at the
    ↪ same close)
with pytest.raises(ValueError, match="lag"):
    run_backtest(mark, tw, BacktestConfig(initial_cash=1000.0, lag=0))
print("run_backtest strictly rejects a look-ahead fill with lag=0.")
```

12.2 Error 2: Using Adjusted Prices and Adding Cash Dividends Again (Double Counting)

- **Why it is wrong:** A total-return adjusted price already embeds reinvested dividends. Booking cash dividends again counts the return twice. Unadjusted prices must instead be paired with explicitly recorded dividend and split events.
- **Verification code:**

```
import numpy as np
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.date_range("2026-01-05", periods=3, freq="B")
mark = pd.DataFrame({"A": [10.0, 10.0, 5.0], "B": [20.0, 20.0, 20.0]}, index=dates)
tw = pd.DataFrame({"A": [1.0], "B": [0.0]}, index=[dates[0]])
cfg = BacktestConfig(initial_cash=100.0)

# Unadjusted prices plus a recorded split preserve the $100 market value
splits = pd.DataFrame({"A": [np.nan, np.nan, 2.0]}, index=dates)
res_split = run_backtest(mark, tw, cfg, splits=splits)
assert np.isclose(res_split.ledger["nav"].iloc[2], 100.0)
```

```
# Without the recorded split, NAV incorrectly falls to $50
res_no_split = run_backtest(mark, tw, cfg)
assert np.isclose(res_no_split.ledger["nav"].iloc[2], 50.0)
print("With unadjusted prices, splits must be booked explicitly.")
```

12.3 Error 3: Replacing Weight Drift with a Simple `shift(1)` (Naive Vectorized Drift Blindness)

- **Why it is wrong:** Naive `shift` vectorization assumes that the portfolio is rebalanced to target weights for free every period, ignoring the weight drift caused by relative asset returns.
- **Verification code:**

```
import numpy as np
import pandas as pd
from quant_course.backtest import drifted_weight_returns

# Assume there is no rebalancing for 2 months and both initial weights are 50%
px = pd.DataFrame({"A": [10.0, 20.0, 10.0], "B": [10.0, 10.0, 10.0]}, index=pd.
    ↪ date_range("2026-01-01", periods=3))
tw = pd.DataFrame({"A": [0.5], "B": [0.5]}, index=[pd.Timestamp("2026-01-01")])

# 1. Drift-aware vectorization
r_drift = drifted_weight_returns(px, tw)

# 2. Naive shift vectorization
r_naive = (tw.reindex(px.index).ffill().shift(1).fillna(0) * px.pct_change(
    ↪ fill_method=None).fillna(0)).sum(axis=1)

print("True drifted return at t=2:", round(r_drift.iloc[2], 4))
print("Naive shift return at t=2:", round(r_naive.iloc[2], 4))
assert abs(r_naive.iloc[2] - r_drift.iloc[2]) > 0.08
```

12.4 Error 4: Ignoring Post-Fee NAV Scaling and Creating a Cash Deficit

- **Why it is wrong:** If the strategy does not call `solve_post_fee_nav` to solve for V', and instead trades using V_{pre}, the fees have no funding source and cash becomes negative.
- **Verification code:**

```
import numpy as np
from quant_course.backtest import solve_post_fee_nav

v_pre = 1000.0
w = np.array([0.5, 0.5])  # Intended 100% fully invested long exposure
c = 0.002

# Naive, incorrect approach: order at full V_pre and create a negative cash balance
order_val_naive = w * v_pre
fees_naive = c * float(np.abs(order_val_naive).sum())
cash_deficit = v_pre - float(order_val_naive.sum()) - fees_naive
assert cash_deficit < 0  # $2.0 cash overdraft!
```

```
# Correct approach: solve for post-fee NAV V' with solve_post_fee_nav
v_post = solve_post_fee_nav(v_pre, np.zeros(2), w, c)
order_val_correct = w * v_post
fees_correct = c * float(np.abs(order_val_correct).sum())
cash_end = v_pre - float(order_val_correct.sum()) - fees_correct
assert np.isclose(cash_end, 0.0)  # Cash is exactly zero; there is no overdraft
print(f"Naive cash deficit: {cash_deficit:.2f}, Post-fee NAV solves exact cash: {
    ↪ cash_end:.4f}")
```

---

13. Applications in Quantitative Research

13.1 A Standardized Research Workflow from Strategy Idea to Post-Fee NAV

A sound quantitative research report should apply the following five standard checks before presenting a strategy's NAV:

1. **Timing audit:** State the signal formation timestamp (`signal_time`) and execution timestamp (`trade_time`). The report must establish that `available_at` $\leq$ `signal_time` $<$ `trade_time`.
2. **Price-type check:** A real-share ledger should use unadjusted closing prices in `mark_prices` and record corporate actions. A synthetic total-return proxy should be declared as using virtual units, without booking dividends twice. If execution uses an opening price or VWAP, provide `exec_prices` separately.
3. **Self-financing reconciliation:** Run `reconcile` and check the relative residual only for positive, finite NAV:
$$\max_{t:\,0<V_t<\infty} \frac{|\text{residual}_t|}{V_t} \leq 10^{-12}$$
If there are no such periods, the relative check is inapplicable; an empty set is not a pass. Report the absolute currency residual separately for a nonpositive terminal NAV, and investigate nonfinite values. The threshold is an engineering check at the stated scale; also report the maximum absolute error. Passing reconciliation is a necessary check of internal accounting consistency, not sufficient evidence that prices and timestamps are correct. Investigate possible omitted cash flows, double-counted dividends, or split-share mismatches.
4. **Tiered cost stress test:** Do not report only a zero-fee or single-fee NAV chart. Consider showing the Sharpe ratio and CAGR decay curves under $c \in \{0, 5, 10, 20, 50\}$ bps, and calculate the strategy's **break-even fee rate**.
5. **Falsification and benchmark comparison:** Set a benchmark portfolio (such as a 1/N equal-weight buy-and-hold strategy) and test whether the strategy's excess return is merely exposure to overall market beta or passive exposure from unrebalanced drift.

14. Chapter Summary

This chapter establishes the core accounting framework for quantitative textbook Part V. The concepts and rules fit together as follows:

1. **Self-financing and the conservation law:** In an account with no external cash flows, the change in portfolio NAV follows this accounting identity:

$$\Delta\text{NAV}_t \equiv \text{Interest}_t + \text{Div}_t + \text{DelistNet}_t + \Delta P_{\text{pre}} + \Delta P_{\text{post}} - \text{Fees}_t$$

   For positive, finite NAV, use Section 6.1's engineering threshold for the relative residual and report absolute currency errors separately. Exclude nonpositive terminal NAV from that division and investigate nonfinite results. Internal reconciliation does not prove that market data or timestamps are correct.
2. **The nonlinear nature of post-fee NAV:** Trading costs directly reduce the capital available for allocation. Target holdings must therefore be sized from post-fee NAV, V', by solving the piecewise-linear equation $V' = V_{\text{pre}} - c\sum|w^*V' - x|$. When $c\sum|w^*| < 1$, the real root is globally unique over $\mathbb{R}$; the root lies in $(0, V_{\text{pre}}]$ if and only if $c\sum|x_i| < V_{\text{pre}}$. Starting from V_{pre}, Newton's method has a convex, piecewise-linear objective: its tangent-line zero decreases monotonically and always remains at or above the true root. It terminates in finitely many steps in exact arithmetic; the floating-point implementation uses a relative-residual threshold and a 100-iteration limit.
3. **Actual holdings generally drift away from target weights:** Target weights hold only momentarily, once a trade completes. Under the assumptions in Section 6.3, without rebalancing weights evolve as $w_{i,t+1} = w_{i,t}\frac{1+r_{i,t+1}}{1+r_{p,t+1}}$; equal returns, zero holdings, or a pure-cash portfolio can remain unchanged. Naive `shift` vectorization that ignores drift produces returns detached from actual holdings.
4. **Causal timing and the execution-lag constraint (lag ≥ 1):** A signal calculated at the close cannot be filled at that same day's closing price. The intraday event order is fixed: settle interest $\to$ apply corporate actions $\to$ execute at price $E \to$ mark to market at closing price P. Invariance to future perturbations is a structural requirement. Existing finite constructed tests cover selected paths; they do not prove invariance to every future change. Complete signals and inputs still require point-in-time contracts and structural reasoning.
5. **Building on this chapter:** The reconciled ledger, holdings table, and trade blotter produced here become analysis tools in **Chapter 30** to expose how 8 common backtest biases distort NAV; in **Chapter 31** to construct rigorous time-series splits; in **Chapter 32** to calculate rigorous out-of-sample statistical inference; and in **Chapter 33** for portfolio risk modeling.

15. Conceptual and Mathematical Questions

- **29-E01:** Starting from the beginning- and end-of-period cash and holdings definitions in the self-financing ledger of Section 6.1, write out the complete algebraic derivation including dividends, splits, delistings, pre-trade price changes, post-trade price changes, and trading fees, and prove that the theoretical reconciliation residual is exactly 0.
- **29-E02:** Suppose a portfolio starts entirely in cash, with $V_{\text{pre}} = 10,000$ dollars. Its target weights are 0.4, 0.3, and 0.2 in 3 stocks, with 0.1 retained as cash. The proportional trading fee rate is $c = 0.0015$. Use the closed-form solution in Theorem 2 to calculate by hand the post-fee NAV V', total fees payable, purchase amount for each stock, and ending cash balance. Verify that cash as a share of V' is exactly 10%.
- **29-E03:** Discuss in detail how the derivative of $f(v) = v + c\sum_i |w_i^* v - x_i| - V_{\text{pre}} = 0$ changes sign when $c\sum_{i=1}^N |w_i^*| \geq 1$. Explain why the function may then lose strict monotonicity and can have multiple roots or no root. Use the counterexample $C = 0, x = (1000, -990), V_{\text{pre}} = 10, c = 0.01$ to explain the economic meaning of a nonpositive root when $c\sum |x_i| \geq V_{\text{pre}}$, and describe the engine's policy response.
- **29-E04:** At period t, a portfolio holds two stocks with weights $w_{1,t} = 0.6$ and $w_{2,t} = 0.4$, with no cash. Over $[t, t+1]$, stock 1 returns +20% and stock 2 returns -10%. If no trades occur during the interval, calculate the drifted weights of both stocks at the end of period $t+1$.
- **29-E05:** Why should a Total Return Index not be used as an unadjusted trading price in an execution module? If a strategy uses total-return prices as input and also passes in a cash-dividend table, how will this distort NAV?
- **29-E06:** Explain why the four conditions in Theorem 3 are sufficient, but not necessary, for a vectorized backtest and an event-ledger backtest to match day by day. Give a specific scenario in which the two series remain exactly equal over a particular interval despite a fee rate $c > 0$. Then analyze why a vectorized backtest diverges when $c = 5$ bps and the execution price is the opening price, $E \neq P$.

29-N01: Diagnose Units and Boundaries Along the Ledger Clock (New)

Redo the small ledger in Section 5.2.1. First calculate $C_{\text{pre}}, x, V_{\text{pre}}, V', \Delta M, \Delta q, C_t, q_t, V_t$, then independently verify it with the conservation decomposition. Identify two errors: writing $0.8V'$ as a share count, and still multiplying the post-trade price P&L by the old 16 shares. Finally explain whether the default $E = P$ means execution at the open, whether monthly-index `lag=1` means the next day, and why the recovery value at the same day's close cannot be used after pre-execution NAV falls to or below 0.

---

16. Programming Questions

Programming Implementation and Boundary-Acceptance Questions

- **29-P01:** Write a standalone Python function named `solve_nav_closed_form` that takes V_{pre}, the vector of current holding values x, target weights w^*, and fee rate c. Assuming the signs of the trade directions are known, use the analytical expression in Theorem 2 to calculate V'. Add assertions that numerically compare this function with the Newton solution from `solve_post_fee_nav` and verify that they agree to machine precision.
- **29-P02:** Use `run_backtest` and `reconcile` to write an automated test script that fully reproduces the two-asset worked example in Section 8. Assert the share counts, fees, ending cash, ending NAV, and reconciliation residual for all 4 dates. Use an absolute floating-point tolerance of `abs=1e-4`.
- **29-P03:** Write a comparison experiment that generates random price data for 3 assets over 60 trading days. Run `drifted_weight_returns` and the naive forward-filled expression `(tw.shift(1) * ret).sum(axis=1)`, then calculate and print the maximum return deviation between them on non-rebalancing dates.
- **29-P04:** Implement a ruin-termination unit test. Build a leveraged portfolio with a -100% short position in stock X and a +100% long position in stock Y. When a sharp rise in X drives account NAV below zero, assert that the backtest engine triggers termination (`terminated_at` is not None) and generates no NAV for any later date.

---

17. Challenge Questions

Advanced Challenges and Further Research

- **29-C01 (No-Trade Bands and Convex Optimization with Proportional Trading Costs):** In real-market rebalancing, quantitative funds often use no-trade bands to avoid unnecessary fees from small weight deviations: a trade is triggered only when the actual holding weight is farther from the target than a tolerance δ (for example, $|w_i - w_i^*| > \delta$), and the holding is adjusted only to the band's boundary. Formulate this mathematically as a one-period utility-maximization problem with an L_1 friction penalty, and explain how it corresponds to convex optimization.
- **29-C02 (Complete Ledger Modeling of Short-Borrow Costs and Securities-Lending Interest):** This chapter's engine assumes that `allow_short=True` incurs no additional borrow fee. In real securities lending, a short position incurs a daily borrow cost at an annualized borrow rate r_{borrow}, while short-sale proceeds are usually subject to rehypothecation and collateral and rebate-rate restrictions. Derive the discrete cash and NAV conservation recursions that include borrow costs, and explain how to extend the attribution terms in `reconcile` accordingly.

18. Complete solutions

29-E01: derive the conservation identity

After a split, $q'_{i,t-1} = q_{i,t-1}S_{i,t}$ and $\widetilde{P}_{i,t-1} = P_{i,t-1}/S_{i,t}$. The previous value is unchanged: $q'\widetilde{P} = qP$. Let I, D, K, L, F denote interest, signed dividends, delisting cash, previous value of delisted shares, and trading fees, as in Section 6.1. For live names,

$$C_t = C_{t-1} + I + D + K - \sum_{i \notin \mathcal{D}} E_i(q_{i,t} - q'_{i,t-1}) - F.$$

Substitute into the difference of closing NAVs:

$$\begin{aligned}
\Delta V_t &= C_t + \sum_i q_{i,t}P_{i,t} - C_{t-1} - \sum_i q'_{i,t-1}\widetilde{P}_{i,t-1} \\
&= I + D + K - F - \sum_{i \notin \mathcal{D}} E_i(q_{i,t} - q'_{i,t-1}) \\
&\quad + \sum_i q_{i,t}P_{i,t} - \sum_i q'_{i,t-1}\widetilde{P}_{i,t-1} \\
&= I + D + (K - L) - F \\
&\quad - \sum_{i \notin \mathcal{D}} E_i q_{i,t} + \sum_{i \notin \mathcal{D}} E_i q'_{i,t-1} \\
&\quad + \sum_{i \notin \mathcal{D}} q_{i,t}P_{i,t} - \sum_{i \notin \mathcal{D}} q'_{i,t-1}\widetilde{P}_{i,t-1} \\
&= I + D + (K - L) - F \\
&\quad + \sum_{i \notin \mathcal{D}} q'_{i,t-1}(E_i - \widetilde{P}_{i,t-1}) \\
&\quad + \sum_{i \notin \mathcal{D}} q_{i,t}(P_{i,t} - E_i).
\end{aligned}$$

All steps are exact rearrangements. Thus actual NAV change minus these components is exactly zero in real arithmetic. Split units and delisting removal are essential to the cancellation; execution-time termination uses E as the final mark and excludes the last segment.

29-E02: buy from cash

All three trades are buys from $x = 0$. The stock target sum is $0.4 + 0.3 + 0.2 = 0.9$, leaving cash fraction 0.1. Therefore

$$V' = \frac{10000}{1 + 0.0015(0.9)} = \frac{10000}{1.00135} = 9986.5182.$$

Purchases are 3994.6073, 2995.9555, and 1997.3036 in currency units, totaling 8987.8664. Fees are

$$0.0015(8987.866389) = 13.4818.$$

Cash is

$$10000 - 8987.866389 - 13.481800 = 998.6518.$$

Using unrounded values, $998.651821/9986.518210 = 0.1$, exactly the requested cash fraction.

29-E03: the theorem's domain and the financial domain

The function is convex and piecewise linear even outside A3. Away from kinks its derivative is $1 + c\sum_i w_i^* \operatorname{sgn}(w_i^* v - x_i)$; at a zero term select any value in $[-1, 1]$ for a legal subgradient.

When $c\sum |w_i^*| \geq 1$, the far-left slope is $1 - c\sum |w_i^*| \leq 0$ and the far-right slope $1 + c\sum |w_i^*| > 0$. Slopes are nondecreasing. There may be no root, a single root, two roots, or a flat interval of roots; a convex function cannot repeatedly cross the axis.

- With $x = 0, C = V_{\text{pre}} = 100, w^* = 1, c = 2$, $f(v) = v + 2|v| - 100$ has roots $100/3$ and -100. The **positive financial root** is unique: buy $100/3$ in notional, pay $200/3$ in fees, and retain zero cash. NAV is $100/3$. The negative algebraic root is not a financial candidate.
- With $x = 200, C = -100, V_{\text{pre}} = 100, w^* = 1, c = 2$, the minimum of $v + 2|v - 200| - 100$ occurs at $v = 200$ and equals 100, so there is no real root.

Failure of A3 removes a sufficient global-monotonicity condition; it does not make every positive-root account impossible. `solve_post_fee_nav` deliberately rejects inputs outside that supported domain.

Even inside A3, positivity additionally requires $f(0) < 0$. For $x = (1000, -990), C = 0$, $V_{\text{pre}} = 10, c = 0.01$, liquidation fees are $0.01(1990) = 19.9$ and $f(0) = 9.9 > 0$. The theorem's unique root is nonpositive, so the engine refuses that rebalance. Equality also fails the positive-equity requirement. This is distinct from a pre-execution ruin, which terminates without trading.

29-E04: compute signed drift

Portfolio return is $0.6(0.2) + 0.4(-0.1) = 0.08$. Hence

$$w_{1,t+1} = \frac{0.6(1.2)}{1.08} = \frac{2}{3} = 66.67\%, \qquad w_{2,t+1} = \frac{0.4(0.9)}{1.08} = \frac{1}{3} = 33.33\%.$$

The weights sum to 1. Stock 1 rises by **6.67 percentage points**, and stock 2 falls by 6.67 percentage points.

29-E05: do not count dividends twice

A total-return index compounds a dividend-reinvestment convention into its changes. If the engine also receives the corresponding dividend cash table, the same economic benefit enters twice and NAV is overstated.

An index level is not an actual per-share quote. Treating a high accumulated index level as a real quote misstates whole-share and entitlement units. Nevertheless, proportional fees and dollar holdings are invariant to a constant price rescaling with inverse fractional-share scaling. A declared synthetic proxy with no additional dividends is coherent for Section 11's teaching comparison; it does not establish actual security execution.

29-E06: sufficient is not necessary

With aligned initial state, execution dates, and policy, Theorem 3's four conditions make both systems value the same exposure and cash flows at every step. Induction then gives identical returns.

For a counterexample to necessity, consider an interval with no trades, zero cash rate, and $E = P$. A configured positive fee rate, even 10 bps, causes no fee when traded notional is zero. Equality can therefore hold without a zero configured rate.

With actual trading at 5 bps, fees reduce cash and the allocation base; a bare dot product omits them. If $E \neq P$, old shares earn the previous-close-to-E segment and new shares earn E-to-close. A single close-to-close return cannot generally allocate both correctly. Thus the simple formula generally fails for the same strategy, but no trade, no holdings, or an offsetting coincidence can still produce equality. Positive fees and unequal price inputs alone do not force a difference at every observation.

29-N01: follow the units through the clock

Interest is $200(0.001) = 0.2$; the split creates 16 shares and dividend cash 16. Thus $C_{\text{pre}} = 216.2, x = 16(49) = 784, V_{\text{pre}} = 1000.2$. On the buy segment,

$$V' = \frac{1000.2 + 0.002(784)}{1 + 0.002(0.8)} = 1000.1677.$$

The target-dollar change is $\Delta M = 0.8V' - 784 = 16.1342 > 0$, confirming the buy. Divide by 49 to get $\Delta q = 0.3293$ shares. Fees are 0.0323; cash is 200.0335; new shares 16.3293; closing NAV 1016.4970.

The independent check is

$$V_t - 1000 = 0.2 + 16 + 16(49 - 50) + q_t(50 - 49) - \text{fee} = 16.4970.$$

The quantity $0.8V'$ has dollar units, not share units. Post-trade PnL uses the new shares, not the old 16. Default $E = P$ is a closing-mark proxy, not an open; monthly lag 1 is one monthly observation. Nonpositive execution equity stops at E without later recovery; positive equity with unaffordable costs is rejected; a valid trade followed by nonpositive closing NAV stops at P.

29-P01: complete runnable solution

```
import numpy as np
import pytest
from quant_course.backtest import solve_post_fee_nav


def solve_nav_closed_form(
    v_pre: float, x: np.ndarray, w: np.ndarray, c: float, signs: np.ndarray
) -> float:
    """Post-fee NAV from Theorem 2, conditional on consistent trade signs."""
    numerator = v_pre + c * float(np.sum(signs * x))
    denominator = 1.0 + c * float(np.sum(signs * w))
    return numerator / denominator


# Compare the closed form and the existing solver.
v_pre = 1000.0
x = np.array([400.0, 500.0])
w = np.array([0.6, 0.2])
c = 0.01

# Try a buy in asset 1 and a sale in asset 2; verify signs after solving.
signs = np.array([1.0, -1.0])

v_closed = solve_nav_closed_form(v_pre, x, w, c, signs)
v_newton = solve_post_fee_nav(v_pre, x, w, c)

print(f"Closed-form V': {v_closed:.6f}")
print(f"Newton solver V': {v_newton:.6f}")
assert np.isclose(v_closed, v_newton, rtol=0, atol=1e-10)
assert np.array_equal(np.sign(w * v_closed - x), signs)
print("P01 verification passed successfully!")
```

29-P02: complete runnable solution

```
import numpy as np
import pandas as pd
import pytest
from quant_course.backtest import BacktestConfig, reconcile, run_backtest

dates = pd.DatetimeIndex(["2026-01-30", "2026-02-02", "2026-02-27", "2026-03-02"])
mark = pd.DataFrame(
    {"A": [10.0, 11.0, 12.1, 12.0], "B": [20.0, 19.0, 19.0, 21.0]},
    index=dates,
)
execp = pd.DataFrame(
    {"A": [10.0, 10.0, 12.0, 12.0], "B": [20.0, 20.0, 19.5, 20.0]},
    index=dates,
)
tw = pd.DataFrame({"A": [0.5, 0.5], "B": [0.3, 0.3]}, index=[dates[0], dates[2]])
cfg = BacktestConfig(initial_cash=1000.0, fee_rate=0.002, lag=1)
```

```
res = run_backtest(mark, tw, cfg, exec_prices=execp)
led = res.ledger
rec = reconcile(res, mark, cfg, exec_prices=execp)

# Complete four-date state: q_A, q_B, cash_end, fees, nav.

# Independently computed closed-form expectations; rtol=0 fixes absolute tolerance.
expected = np.array([
    [0.0, 0.0, 1000.0, 0.0, 1000.0],
    [49.920128, 14.976038, 199.680511, 1.597444, 1033.346645],
    [49.920128, 14.976038, 199.680511, 0.0, 1088.258786],
    [45.753458, 16.471245, 219.616601, 0.159808, 1114.554248],
])
actual = np.column_stack([
    res.positions[["A", "B"]].to_numpy(),
    led[["cash_end", "fees", "nav"]].to_numpy(),
])
assert led.index.equals(dates)
np.testing.assert_allclose(actual, expected, rtol=0, atol=1e-4)
assert rec["residual"].abs().max() < 1e-12
assert rec["cash_identity"].abs().max() < 1e-12
print("P02: all four dates, both positions, cash, fees, NAV and reconciliation
    ↪ checked.")
```

The five columns cover both share counts, cash, fees, and NAV on every input date. Absolute tolerance is 10^{-4} with relative tolerance disabled. Reconciliation and the cash identity are checked independently to 10^{-12}.

29-P03: complete runnable solution

```
import numpy as np
import pandas as pd
from quant_course.backtest import drifted_weight_returns

rng = np.random.default_rng(2026)
dates = pd.date_range("2026-01-01", periods=60, freq="B")
shocks = rng.normal(0, 0.02, size=(60, 3))
px = pd.DataFrame(100.0 * np.exp(np.cumsum(shocks, axis=0)), index=dates)

# Form equal-weight signals every 20 observations; delay execution by one row.
sig_dates = dates[[0, 20, 40]]
tw = pd.DataFrame(1.0 / 3.0, index=sig_dates, columns=px.columns)

trade_w = tw.copy()
trade_w.index = dates[np.searchsorted(dates, tw.index) + 1]

r_drift = drifted_weight_returns(px, trade_w)
r_naive = (
    trade_w.reindex(dates).ffill().shift(1).fillna(0)
    * px.pct_change(fill_method=None).fillna(0)
).sum(axis=1)

non_trade = ~dates.isin(trade_w.index)
```

```
max_error = np.abs(r_naive[non_trade] - r_drift[non_trade]).max()
print(f"Max tracking error between naive shift and drifted: {max_error:.6f}")
assert max_error > 1e-4
print("P03 passed: naive shift-only formulation diverges as predicted!")
```

29-P04: complete runnable solution

```
import pandas as pd
import pytest
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.date_range("2026-01-05", periods=4, freq="B")

# X jumps from 10 to 25 after the short is established.
mark = pd.DataFrame(
    {"X": [10.0, 10.0, 25.0, 30.0], "Y": [20.0, 20.0, 20.0, 20.0]}, index=dates
)

# Short X and go long Y.
tw = pd.DataFrame({"X": [-1.0], "Y": [1.0]}, index=[dates[0]])
cfg = BacktestConfig(
    initial_cash=100.0, allow_short=True, max_gross=2.0, lag=1
)

res = run_backtest(mark, tw, cfg)

print("Terminated at:", res.terminated_at)
print("Ledger length:", len(res.ledger))
print("Final NAV:", res.ledger["nav"].iloc[-1])

assert res.terminated_at == dates[2]
assert len(res.ledger) == 3
assert res.ledger["nav"].iloc[-1] <= 0.0
print("P04 passed: ruin policy correctly terminated bankrupted backtest!")
```

29-C01: the static no-trade region

Let u be current drifted weights, w the chosen new weights, μ expected returns, $\gamma > 0$ risk aversion, $\Sigma \succ 0$ the covariance matrix, and $c > 0$ the cost coefficient. With no further investment constraints, consider the static approximation

$$\max_{w} \; U(w) = \mu^\top w - \frac{\gamma}{2} w^\top \Sigma w - c\|w - u\|_1.$$

Strict concavity gives a unique optimum. The subgradient optimality condition is

$$0 \in \gamma\Sigma w - \mu + c\,\partial\|w - u\|_1.$$

At $w = u$, every absolute-value subgradient coordinate can take any value in $[-1, 1]$. Thus no trade is optimal if and only if

$$|\mu_i - (\gamma\Sigma u)_i| \le c \quad \text{for every } i.$$

This is a region of **current states** u, not a fixed box that must contain all optimal weights. With frictionless optimum $w^0 = (\gamma\Sigma)^{-1}\mu$,

$$\mathcal{N} = \{u : \|\gamma\Sigma(u - w^0)\|_\infty \le c\}.$$

Positive definiteness makes $\gamma\Sigma$ invertible. The region is therefore a bounded closed convex parallelotope centered at w^0. It is generally tilted with correlated assets and axis-aligned when Σ is diagonal. A merely positive-semidefinite matrix can have a nullspace that makes the region unbounded; the bounded-parallelotope claim would fail.

Outside this region solve the same subgradient equation. A traded coordinate satisfies $\mu_i - (\gamma\Sigma w)_i = c\,\mathrm{sgn}(w_i - u_i)$; an unchanged coordinate has marginal benefit within $[-c, c]$. The kink can therefore leave coordinates exactly unchanged. "Trade to the boundary" here means that remaining marginal benefit reaches marginal cost in this static model. It is not a proof that every hand-chosen threshold δ is dynamically optimal. The L_1 weight difference approximates costs and does not simultaneously solve Section 6.2's exact fee-adjusted denominator. A dynamic policy needs future utility and state transitions.

Assessment: objective and conditions 40%; subgradients and kinks 30%; state-space geometry and limitations 30%.

29-C02: free cash, collateral, and short costs

This is a teaching extension beyond the base engine; `allow_short=True` does not implement margin accounting. First specify a convention. During a fixed-holding interval assume no new trades, splits, or delistings. Let q_i be signed shares, F free cash, and R restricted collateral. Then

$$V = F + R + \sum_i q_i P_i.$$

Positive free cash earns r_l, negative free cash pays r_b, and collateral earns r_R. Use the example's declared day-count fraction $h = \Delta d/360$; this is not a universal market rule. Define

$$I_F = h\{r_l \max(F_{t-1}, 0) - r_b \max(-F_{t-1}, 0)\}, \qquad I_R = h r_R R_{t-1},$$

$$B = h \sum_i r_{\mathrm{borrow},i} |\min(q_i P_{i,t-1}, 0)|, \qquad D = \sum_i q_i D_{i,t}.$$

Borrow cost B uses the previous short market value. A different contract may specify a different basis. The collateral return I_R already contains this example's rebate; do not add the rebate again. Signed dividends D are negative for a short.

Settle interest, dividends, and borrow cost into free cash, then transfer ΔR_t to collateral, positive for a transfer out of free cash:

$$F_t = F_{t-1} + I_F + I_R + D - B - \Delta R_t,$$
$$R_t = R_{t-1} + \Delta R_t.$$

Add the accounts and substitute into NAV:

$$V_t - V_{t-1} = \sum_i q_i(P_{i,t} - P_{i,t-1}) + D + I_F + I_R - B.$$

The internal collateral transfer cancels. This is the complete fixed-holding reconciliation identity.

With new trades at E, also add $-\sum_i E_i \Delta q_i -$Fees to free cash, update shares, and recompute collateral under the stated rule. Split price PnL into old holdings' previous-close-to-E segment and new holdings' E-to-close segment, as in Section 6.1; add the interest terms and subtract borrow cost. A transfer still creates no NAV loss. If free cash cannot meet margin, the model must specify borrowing, position reduction, or termination rather than silently ignoring the call.

Complete constructed ledger, in dollars. Set interest, rebates, and transaction fees to zero. Start with equity 100 and short one share at 50. Total cash becomes 150; lock the sale proceeds 50 plus extra margin 25. Thus $F = 75, R = 75, q = -1$, and NAV is $75 + 75 - 50 = 100$. Next the share price rises to 55; pay a dividend of 1 and borrow cost of 5:

Step	Free cash F	Collateral R	Short value qP	NAV
After initial short and collateral transfer	75	75	−50	100
Price rises to 55	75	75	−55	95
Pay dividend 1 and borrow cost 5	69	75	−55	89
Raise collateral to 1.5(55) = 82.5; transfer 7.5 internally	61.5	82.5	−55	89

The final check is $61.5 + 82.5 - 55 = 89$. The loss of 11 comprises price loss 5, dividend compensation 1, and borrow cost 5. The collateral transfer of 7.5 has zero NAV effect. Borrow cost 5 is a declared input for this teaching ledger, not output from the current engine; compatible rate/day-count inputs could generate it through the formula above. An extended `reconcile` should show free-cash interest, collateral rebate, signed dividends, borrow cost, and transfer cash identities separately. Adding a borrow-fee row alone does not implement margin.

Assessment: conventions, cash recurrence, dividends and borrow costs 50%; NAV decomposition, transfer cancellation, and complete numerical ledger 50%.

19. Further reading

These are inherited reading pointers, not newly verified external sources for this revision.

- **Core reading:** pandas' Time Series / Date Functionality guide and `DataFrame.pct_change` documentation, to understand index order, alignment, slicing, and missing-value behavior: https://pandas.pydata.org/docs/user_guide/timeseries.html.
- **Core reading:** Kenneth R. French Data Library's 10-industry description, for value weighting and reconstruction conventions: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_10_ind_port.html.

- **Further background:** Boyd and Vandenberghe, *Convex Optimization* (2004), https://web.stanford.edu/~boyd/cvxbook/. Not read for this chapter; a background pointer for convex functions and scalar root finding.
- **Advanced background:** Frazzini, Israel, and Moskowitz (2018), "Trading Costs of Asset Pricing Anomalies," https://www.aqr.com/Insights/Research/Working-Paper/Trading-Costs-of-Asset-Pricing-Anomalies. Not read for this chapter; none of this chapter's numerical assumptions is attributed to that paper.

20. References

1. [**PANDAS**] The pandas development team. (2026). *pandas documentation (v3.0)*. https://pandas.pydata.org/docs/.
2. [**FRENCH_10**] French, K. R. (2026). *Detail for 10 Industry Portfolios*. Kenneth R. French Data Library. https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_10_ind_port.html.

Chapter 30 — Backtesting Biases

A backtest can run without errors while importing future information, conditioning on survivors, selecting a lucky maximum, or miscounting cash and shares. This chapter traces those channels through the self-financing ledger, a point-in-time (PIT) universe, a complete trial ledger, and explicit trading costs. Prerequisites: Chapter 3 (data engineering and PIT contracts), Chapter 22 (research methodology and trial records), and Chapter 29 (the backtesting ledger). You will diagnose eight biases, derive the truncated-normal and search-maximum examples, inspect finite future-perturbation tests, and distinguish constructed demonstrations from revised, aggregated historical returns.

1. Learning goals

A dangerous backtest need not crash. A smooth NAV curve may use an after-close report before its release, discard companies that later delisted, or hide unsuccessful specifications. The problem is the information and accounting contract, not whether the program returns a number.

After reading, you should be able to:

1. Derive the mechanisms of look-ahead, survivorship, selection, data snooping, overfitting, corporate-action errors, parameter mining, and cost errors. Calculate a truncated-normal survivor premium, the expected maximum of null statistics, and Bonferroni/Holm adjustments; prove the cash-and-share accounting of splits and dividends.
2. Ask which information was used, when it was published and ingested, and where it entered the decision. Explain why `lag=1` alone cannot repair revised financial statements or late data delivery.
3. Read the wrong and repaired implementations on the Chapter 29 ledger, retain every candidate in `research_protocol.TrialLedger`, and compare pre-cutoff signals, fills, positions, and NAV under specified future perturbations. Passing finitely many checks does not prove every future input channel is harmless.
4. Separate company-level PIT histories from academic industry aggregates. Revised, non-PIT and nontradable snapshots can illustrate methods without establishing executable alpha.

Completion means explaining each contaminated channel, following the default reader route in Section 9, and solving the chapter's questions. The historical JSON, figures, and 20-seed results retain their original run identities; this teaching revision does not rerun the complete CLI, Notebook, or study.

2. Why a quant needs this

Bias diagnosis connects signal research to validation and ledger execution:

```
Prices and announcements (Chapter 3) -> signals (Chapters 23–28)
Search and trial records (Chapter 22) -> bias diagnosis (this chapter)
Diagnosis -> ledger execution (Chapter 29) -> validation and risk (Chapters 31–33)
```

The main failure channels are:

- **Look-ahead leakage:** a closing decision cannot trade retroactively at that close or an earlier open using a later announcement.
- **Survivorship and selection conditioning:** today's surviving top-100 stocks are not an unconditioned historical universe. Applying their final membership backward erases failed companies and their losses.
- **Data snooping and mining:** reporting the best of 50 lookbacks or 100 factors without the unsuccessful trials disguises a search maximum as a single test.
- **Corporate-action distortion:** ignoring split shares, using adjusted levels as real-share quotes, or paying cash dividends on top of dividend-inclusive levels breaks the intended ledger.
- **Frictional blindness:** zero-cost assumptions can materially alter the wealth of a high-turnover strategy. Costs need the actual traded-notional denominator.

The eight diagnoses use one declared constructed market and the same Chapter 29 accounting core; the examples below retain the historical outcomes, including negative results.

3. Intuition

3.1 Three stocks and three dates

Before Monday's open, the account has USD 1,000 cash and ten B shares marked at the previous trading close of USD 100 each: total NAV is USD 2,000. A will announce earnings, B will split 2:1, and C will delist. Let t_0 be Monday's signal date, t_1 Tuesday's execution date, and t_2 Wednesday's closing settlement.

```
Monday close -> Tuesday open -> Tuesday close -> Wednesday close
Decision s      Execution E      Mark P          Final mark/settlement
```

1. **An after-close announcement.** A announces at Monday 16:30; Tuesday opens 10% higher, from 100 to 110. A leaked version backdates the signal or uses zero lag to buy at Monday's 100 and collect an unavailable jump. If a separate pre-open decision contract permits it and publication, ingestion, and the order cutoff are all satisfied, the earliest fill still uses Tuesday's post-jump 110. Under this chapter's close-decision contract, the earliest signal is Tuesday close, filled Wednesday open. Neither route earns a jump that occurred before entry.
2. **Removing a future delisting.** C settles at zero on Wednesday. Taking Wednesday's survivor list back to Monday, or deleting an existing C position and its loss, is the error.

Wednesday's contemporaneous listing status may legitimately constrain new investment; it cannot rewrite past eligibility or cancel settlement of an old position.

3. **A false split cliff.** B splits after Tuesday's close: its mechanical quote changes from 100 to 50 and ten shares become twenty. Leaving shares at ten records B at 500 instead of 1,000. The position appears to lose 50%, but with the unchanged 1,000 cash the account falls from 2,000 to 1,500: a false **25% NAV loss**. Proper share adjustment preserves the position's 1,000 value under this no-news mechanical example.

4. Mathematical foundations

4.1 Information, units, and clocks

Let $\mathcal{T} = \{t_0, \dots, t_T\}$ be an increasing, unique sequence of price observations. Keep three event timestamps separate:

- t_{period}: the economic reporting-period end;
- t_{avail}: source publication;
- t_{ingest}: actual receipt, validation, and database availability.

At an actual decision close s, the information filtration is

$$\mathcal{F}_s = \sigma\{e : \max(t_{\text{avail}}(e), t_{\text{ingest}}(e)) \le s\}.$$

The random target vector w_s^* must be **measurable with respect to** $\mathcal{F}_s$; a random vector is not an event belonging to that sigma-algebra. Execution is at price-index position $k + \text{lag}$ with integer lag ≥ 1, rather than an integer added to a timestamp.

Prices $P_{i,t} > 0$ and $E_{i,t} > 0$ are respectively raw closing marks and execution prices, in USD/share. The split multiplier $S_{i,t} > 0$ is new shares per old share, with default 1. Cash dividend $D_{i,t} \ge 0$ is quoted per post-split share, and $C_{i,t}^{\text{delist}} \ge 0$ is settlement cash per share. A closing-mark proxy is not evidence of an executable open.

4.2 Search maxima and multiple tests

Under an abstract null of M independent standard-normal statistics Z_m, let $M_M = \max_m Z_m$. Then

$$\Pr(M_M \le z) = \Phi(z)^M,$$
$$\mathbb{E}M_M = M \int_{-\infty}^{\infty} z\Phi(z)^{M-1}\phi(z)\, dz.$$

For $M = 50$, `expected_max_standard_normal_exact(50)` gives the inherited numerical value **2.2491**. The Bailey–López de Prado approximation is

$$\mathbb{E}M_M \approx (1-\gamma)\Phi^{-1}(1 - 1/M) + \gamma\Phi^{-1}(1 - 1/(Me)),$$

and `expected_max_standard_normal(50)` gives **2.2763**. The first-order Gumbel asymptotic approximation is

$$\sqrt{2\ln M} - \frac{\ln\ln M + \ln(4\pi)}{2\sqrt{2\ln M}} + \frac{\gamma}{\sqrt{2\ln M}},$$

giving **2.3072**. The latter is a large-M approximation, slightly above the integral at 50.

These are expectations, not a promise that every observed maximum exceeds 2.24. Putting the expected maximum into a one-sided tail gives about 0.0123; it is not the expected selected p-value. Fifty correlated momentum Sharpes do not automatically satisfy this independent standard-normal null. Formal inference needs valid nominal p-values and an explicit search family.

5. Definitions

Bias	Mathematical or operational trigger	Contaminated channel	Typical consequence
Look-ahead	w_s^* is not $\mathcal{F}_s$-measurable; publication or ingestion is later than the decision	Future information enters a past order	Unavailable jumps may enter NAV
Survivorship	Historical universe uses $\{i : \text{alive}_i(T) = 1\}$	Final survival conditions the past	Left-tail settlement losses disappear
Selection	Selection uses full-sample or future performance, such as a full-period Sharpe maximum	Outcomes determine admission	Lucky historical winners are selected
Data snooping	Only the best of M tests is disclosed; the trial ledger omits the others	Unadjusted multiple comparisons	Nominal inference ignores the search
Overfitting	Excess flexibility minimizes in-sample residuals	Noise is fitted as structure	In-sample fit fails to generalize
Corporate-action distortion	Split shares or dividends are omitted or counted twice	Cash-and-share accounting	A false position cliff or duplicated income
Parameter mining	$\theta^* = \arg\max_\theta \text{Sharpe}(\theta)$ is treated as a stable law	A noisy parameter surface	Fragile selected parameters
Cost-model error	Fees use NAV instead of actual traded notional, or are omitted	Friction is disconnected from trades	High turnover is mispriced

5.1 How the channels connect

```
Misaligned event clocks -> look-ahead
Outcome-conditioned sample -> survivorship / selection
Unrecorded search -> snooping / parameter mining / overfitting
Misstated ledger mechanics -> corporate actions / costs
```

6. Derivations

6.1 Conditions behind a constructed look-ahead return gap

Suppose an earnings surprise $\Delta_{i,t}$ is published at 16:30 after close t, with opening gap

$$g_{i,t+1} = \alpha + \beta r_{m,t+1} + \kappa \Delta_{i,t} + \varepsilon_{i,t+1}.$$

This is a declared teaching DGP. $\kappa > 0$ alone proves no general positive return difference. Define intraday return $d_{i,u} = P_{i,u}/E_{i,u} - 1$. Assume publication and ingestion finish before the next close. A legal close decision first uses the surprise at $t+1$, executes at $t+2$ with lag 1, and earns $r_c = d_{i,t+2}$. A wrong version backdates to the $t-1$ signal, enters at t open, and holds to $t+1$ close:

$$1 + r_w = (1 + d_{i,t})(1 + g_{i,t+1})(1 + d_{i,t+1}).$$

To first order in small returns, $r_w \approx d_{i,t} + g_{i,t+1} + d_{i,t+1}$. Conditional on the selected announcement event Top, suppose non-announcement intraday returns have the same mean μ_d, the market and idiosyncratic noise are independent of selection with zero conditional mean, and $\alpha = 0$. Only then do we obtain

$$\mathbb{E}[r_w - r_c \mid \text{Top}] \approx \mu_d + \kappa \mathbb{E}[\Delta_{i,t} \mid \text{Top}].$$

The approximate gap is positive only if the right side is positive. The arms have different holding periods; the extra intraday period and compounding cross-terms cannot be silently discarded. Correlated selection/noise, large returns, or nonstationary intraday distributions require a new calculation. A leaked strategy remains invalid even if its performance is worse. Section 12.1 reports the historical constructed comparison without turning it into a universal theorem.

6.2 Survivor conditioning in a truncated-normal model

Approximate an evaluation-period return indicator by $X \sim N(\mu_X, \sigma_X^2)$ with $\sigma_X > 0$, and define survival in this teaching threshold model by $X \geq d$. Let

$$\alpha = \frac{d - \mu_X}{\sigma_X}, \qquad p_{\text{delist}} = \Phi(\alpha), \qquad \lambda(\alpha) = \frac{\phi(\alpha)}{1 - \Phi(\alpha)}.$$

Then

$$\mathbb{E}[X \mid X \geq d] = \mu_X + \sigma_X \lambda(\alpha),$$
$$\mathbb{E}[X \mid X \geq d] - \mu_X = \sigma_X \lambda(\alpha) > 0.$$

At fixed location and scale, λ rises with the threshold and p_{delist}. The relation is monotone and nonlinear, not proportional. A normal variable is unbounded below; it is an approximation for a truncated indicator, not a universal default mechanism for limited-liability stock simple returns.

The inherited `bias_2_survivorship.truncated_normal_check` uses $\mu_X = 0$, $\sigma_X = 0.3$, $d = -0.3$. Thus $\alpha = -1$, $\Phi(-1) = 0.1587$, $\lambda(-1) \approx 0.2420/(1 - 0.1587) = 0.2876$, and

$$\mathbb{E}[X \mid X \geq -0.3] \approx 0.3(0.2876) = 0.0863.$$

Excluding the delisted lower tail and reporting survivors alone raises the conditional average by about **8.63 percentage points** relative to the unfiltered population.

6.3 Splits and dividends in the raw-price ledger

Before execution, Chapter 29's ledger applies the split $q'_{i,t-1} = q_{i,t-1} S_{i,t}$, credits dividend cash $q'_{i,t-1} D_{i,t}$, and uses the combined mechanical reference

$$P^{\text{adj}}_{i,t-1} = P_{i,t-1}/S_{i,t} - D_{i,t}.$$

Because the dividend is per post-split share,

$$\begin{aligned} V_{t,\text{pre}} &= C_{t-1} + q_{i,t-1} S_{i,t} D_{i,t} + q_{i,t-1} S_{i,t} (P_{i,t-1}/S_{i,t} - D_{i,t}) \\ &= C_{t-1} + q_{i,t-1} P_{i,t-1} = V_{t-1}. \end{aligned}$$

For ten shares at 100, a 2:1 split and a dividend of 1 per new share give twenty shares, USD 20 cash, and reference price $100/2 - 1 = 49$: $20(49) + 20 = 1000$. Using $(100 - 1)/2 = 49.5$ instead falsely records 1010, an extra USD 10.

This is mechanical value neutrality with no other price news, trading, fees, or interest. It does not imply every real ex-date has zero NAV return: Section 8's inherited sample retains a normal -1.29% market move. Ignoring the multiplier halves the affected position under a pure 2:1 split; the percentage account loss also depends on cash and other holdings.

7. Financial Interpretation

7.1 Why do quantitative teams systematically produce backtest bias?

Backtest bias is widespread because the organizational structure of quantitative investment research contains strong incentives and engineering blind spots:

1. **Publication and reporting incentives**: In investment institutions, an evaluation system focused too heavily on the Sharpe ratio from a single backtest may encourage researchers to report selectively the best in-sample result. Researchers who candidly disclose many failed parameter choices have little incentive to do so, while selectively reporting one best-performing variant is more likely to pass an investment review. In hypothesis-testing terms, this is equivalent to leaving multiple testing uncontrolled (see Chapter 22).
2. **The high cost and engineering burden of point-in-time databases**: Maintaining a point-in-time database that preserves every historical earnings-release timestamp, every subsequent accounting restatement, and every delisted stock requires substantial commercial data purchases and data-cleaning work. Beginners often download "clean" data from open-source websites that have been adjusted for later corporate actions and filtered to currently listed securities. The time-series integrity has already been lost at the input stage.
3. **The gap between vectorized code and live trading**: Modern data-science libraries such as Pandas and Polars favor vectorized computation. A simple `.shift(-1)` or `df.corr()` can run extremely quickly, but when researchers write these lines, they can easily forget the underlying micro-sequence of placing and queueing orders, exchange matching, and cash transfers in an actual market.

---

8. Numerical Example

Complete Worked Example: Constructed-Market Inputs, Split, and Delisting Ledger

8.1 Specification of the constructed teaching dataset

To investigate eight forms of bias systematically, we construct a strictly reproducible standard market environment in `code/ch30_backtest_biases.py`:

- **Random seed**: `SEED = 20261030` (one global seed; cherry-picking seeds is prohibited).
- **Asset universe**: 36 stocks, labeled `S01` through `S36`.
- **Date range**: 2024-01-02 through 2025-12-31, with 522 synthetic Monday-to-Friday observations. This is not an exchange calendar and includes exchange holidays (for example, 2025-01-01). Annualized metrics such as Sharpe and volatility use the convention of 252 observations per year; this does not mean that there are 522 actual trading days.

Continued: the same complete example

  - In-sample (IS): calendar year 2024, with 261 weekday-grid observations (2024-01-02 through 2024-12-31).
  - Out-of-sample (OOS): calendar year 2025, with 261 weekday-grid observations (2025-01-01 through 2025-12-31).
- **Preconfigured corporate actions**:
  - `S05` undergoes a 2:1 split on 2024-06-25 (trading day 125; $S = 2.0$).
  - `S08` pays a cash dividend of USD 1.00 per share on trading days 60, 180, 300, and 420.
  - `S12` pays a cash dividend of USD 1.50 per share on trading days 100, 220, 340, and 460.
  - `S35` encounters operating difficulties and is delisted on 2024-09-10 (trading day 180), liquidated for USD 2.00 per share in cash, and removed from the universe.
  - `S36` encounters operating difficulties and is delisted on 2025-05-06 (trading day 350), liquidated for USD 1.00 per share in cash, and removed from the universe.
- **Ex-dividend-date price convention**: The ledger adjusts share count by S_t first, then pays the per-share dividend D_t on the new share count. Therefore, when D_t is quoted per post-split share, the combined event reference price is $P_{t-1}/S_t - D_t$.
- **After-hours earnings calendar**: Every 21 weekday-grid observations, 12 stocks are selected at random to announce earnings after the close (16:30), causing an earnings gap at the next observation day's open. Because the announcement arrives after that day's close, the causal example generates a signal at the next observation day's close and executes at the following observation day's open with `lag=1`.

8.2 Hand calculation: the day-125 split and day-180 delisting liquidation

We calculate key points from the ledger run (all figures are quoted from `ch30_results.json`):

1. S05 split on trading day 125 (2024-06-25)

Source: `bias_6_corporate_actions.day_125_split_hand_check` in the JSON file.

- **Closing ledger on trading day 124 (2024-06-24)**:
  - S05 close $P_{124} = 75.1314$ USD, position $q_{124} = 500.8387$ shares, market value USD 37,628.74.
  - S08 close $P_{124} = 74.2556$ USD, position $q_{124} = 500.9080$ shares, market value USD 37,195.22.
  - Cash balance $C_{124} = 500.9080$ USD; total net asset value $\mathrm{NAV}_{124} = 75,324.87$ USD.
- **Open and close on trading day 125 (2024-06-25)**:
  - S05 undergoes a 2:1 split, with execution price $E_{125} = 37.5104$ USD and closing mark $P_{125} = 37.4605$ USD.
  - S08 execution price $E_{125} = 73.9312$ USD and closing mark $P_{125} = 72.5206$ USD (there was no trade that day, so the execution price is for reference only).

Continued: the same complete example

- **Compliant Fixed ledger (with the splits table passed in)**:
  - S05 shares double to $q_{125} = 500.8387 \times 2.0 = 1001.6775$ shares.
  - S05 closing position value $= 1001.6775 \times 37.4605 = 37,523.31$ USD.
  - S08 closing position value $= 500.9080 \times 72.5206 = 36,326.16$ USD.
  - The cash balance remains unchanged at $= 500.9080$ USD (its numeric equality to S08's 500.9080 shares is purely coincidental; the units differ: one is dollars and one is shares).
  - Total NAV $\text{NAV}_{125} = 37523.31 + 36326.16 + 500.91 = 74,350.38$ USD.
  - Daily return $= (74350.38 - 75324.87)/75324.87 = -0.012937$ (ordinary market movement of -1.29%).
- **Faulty Split-Ignored ledger (without the splits table)**:
  - The share count is not adjusted and remains at 500.8387 shares.
  - The position is recorded as $500.8387 \times 37.4605 = 18,761.66$ USD, which is USD 18,761.66 below the correct value.
  - Total NAV $\text{NAV}_{125} = 18761.66 + 36326.16 + 500.91 = 55,588.73$ USD.
  - Daily return $= (55588.73 - 75324.87)/75324.87 = -0.262014$ (an artificial ledger loss of 26.20% appears).

2. S35 delisting liquidation on trading day 180 (2024-09-10)

Source: `bias_2_survivorship.s35_delisting_hand_check` in the JSON file.

- On the day before delisting (2024-09-09, day 179):
  - The strategy holds $q = 63.7348$ shares of S35; the closing price is $P_{179} = 28.7926$ USD.
  - Pre-delisting market value $= 63.7348 \times 28.7926 = 1835.09$ USD.
- Delisting liquidation on trading day 180:
  - Liquidation cash price $C^{\text{delist}} = 2.00$ USD/share.
  - The ledger recovers $= 63.7348 \times 2.00 = 127.47$ USD in cash.
  - The S35 share position is reduced to zero, and USD 127.47 is credited to the cash account.
  - The ledger records a direct delisting capital loss of $= 1835.09 - 127.47 = 1707.62$ USD.

A universe containing only surviving stocks would conceal this real liquidation loss of USD 1,707.62 entirely.

9. Python Implementation

The complete course entrypoint for the bias analyses in this chapter is `code/ch30_backtest_biases.py`. By default, first run the following from the root of the accepted I3 reader package:

```
python run_notebook.py notebooks/ch30_backtest_biases.ipynb
```

The Notebook skips the French branch by default and computes only the constructed example. Initialization creates a unique `runs/ch30_backtest_biases_<uuid>/` directory; the runner separately saves an execution copy and `execution.json` under `runs/execution_<stem>_<uuid>/`, retaining failed runs as well. English translation of the chapter Notebook, the full package, and cold-start work are assigned to BIL23. The complete course calls below are provided to explain the original method, including the 522-observation constructed dataset and the full bias study; they were not run during this revision. Before running them independently, check the actual parameters and output paths, and do not overwrite historical QA.

Daily returns used to generate research signals are calculated from unadjusted closing prices, split ratios, cash dividends, and delisting liquidation prices according to ledger events. Orders are executed and daily positions are valued using raw, unadjusted prices. A general close-based signal formed at the close on date d may use historical features only through that close; after `lag=1`, its holding interval is from the next grid observation's open through its close. The regression label is the gross open-to-close return over that interval and does not assign the overnight gap before execution to the new position. If information is released after the close at 16:30, it can first be used in a signal formed at the next observation day's close, then executed at the open of the following observation day.

```
import importlib.util
from pathlib import Path
import numpy as np
import pandas as pd

# Dynamically load the independently executable Chapter 30 module.
root = Path.cwd()
spec = importlib.util.spec_from_file_location("ch30", root / "code/
    ↪ ch30_backtest_biases.py")
ch30 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ch30)

# Generate the constructed market dataset.
data = ch30.build_constructed_market(seed=20261030)
assert len(data["dates"]) == 522
assert len(data["assets"]) == 36
print(f"Dataset initialized: {len(data['assets'])} assets, {len(data['dates'])}
    ↪ synthetic weekday observations.")
```

```
import importlib.util
from pathlib import Path

root = Path.cwd()
spec = importlib.util.spec_from_file_location("ch30", root / "code/
    ↪ ch30_backtest_biases.py")
ch30 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ch30)

data = ch30.build_constructed_market(seed=20261030)

# Run the look-ahead bias and end-to-end future-perturbation tests.
s1 = ch30.study_lookahead(data)
pert = ch30.study_future_perturbation(data)
print(f"Lookahead diff Sharpe: {s1['diff_sharpe']:.4f}")
print(f"Perturbed future inputs: {pert['perturbed_future_inputs']}")
print(f"Fixed look-ahead pre-cutoff diffs: {pert['fixed_lookahead_diffs_up_to_cutoff
    ↪ ']}")
print(f"Fixed selection pre-cutoff diffs: {pert['fixed_selection_diffs_up_to_cutoff
    ↪ ']}")
print(f"Wrong full-sample selection pre-cutoff diffs: {pert['
    ↪ wrong_selection_diffs_up_to_cutoff']}")
print(f"Fixed pipeline invariant: {pert['is_fixed_invariant']}")
print(f"Wrong pipeline causality violated: {pert['is_wrong_causality_violated']}")
assert pert["is_fixed_invariant"] == 1
assert pert["is_wrong_causality_violated"] == 1
```

For the `filled_orders` metric in the perturbation results, if the fill keys (signal time, trade time, asset) differ, the metric returns `1.0` as a limited marker of identity change. If the keys match, it returns the maximum absolute difference in fill quantity, price, notional, and fees. The other continuous metrics each return the maximum absolute difference in their corresponding outputs.

10. Visualization

The saved historical results from `code/ch30_backtest_biases.py` are located in `qa/a6/codex_resume/ch30/results_v2/`, and the corresponding four figures are in `qa/a6/codex_resume/ch30/figures_v2/`. This revision reuses those curves; it does not rerun the code or redraw the figures. The v1 results and figures remain in `results/` and `figures/`; the figures used in the textbook are updated only after the v2 snapshot has been saved.

10.1 Figure 1: NAV paths for look-ahead and survivorship bias

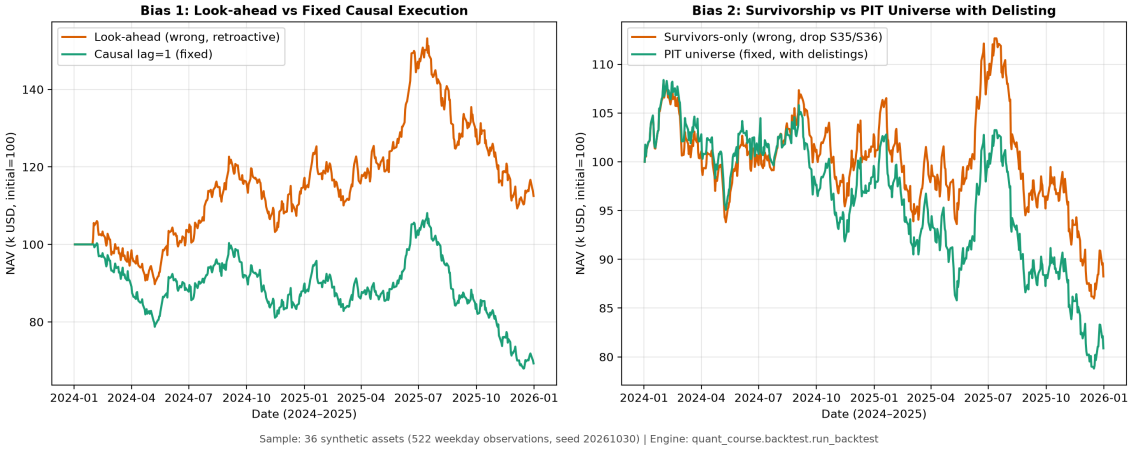


Figure 1: NAV paths for look-ahead and survivorship bias

- **Left panel (Panel A: Look-ahead bias)**:
  - Horizontal axis: synthetic Monday-to-Friday date grid (2024–2025; 522 observations, including exchange holidays); vertical axis: NAV (thousands of USD, initial value = 100).
  - Comparison: the orange solid line is the look-ahead strategy (Wrong, retroactive), with ending NAV of `112500.12` USD and Sharpe of `0.3730`; the green solid line is the causal strategy (decision at the close of the observation day after the announcement, then execution after `lag=1`), with ending NAV of `69300.37` USD and Sharpe of `-0.7760`.
  - Main finding: the look-ahead strategy backdates after-hours announcement information to a signal before the announcement and establishes a position before the announcement gap. The causal strategy waits until a signal can be formed at the close and then executes.
- **Right panel (Panel B: Survivorship bias)**:
  - Horizontal axis: the same 522-observation synthetic weekday grid; vertical axis: NAV (thousands of USD).
  - Comparison: the orange solid line is the survivors-only strategy (Wrong, survivors-only), with ending NAV of 88.25 k USD and Sharpe of -0.2950; the green solid line is the true point-in-time strategy with delistings (Fixed, with delistings), with ending NAV of 80.87 k USD and Sharpe of -0.5565.
  - Main finding: the universe containing only surviving stocks erases the two steep declines of S35 and S36 and creates a false impression of resilience.

10.2 Figure 2: Data-snooping multiple comparisons and selection bias

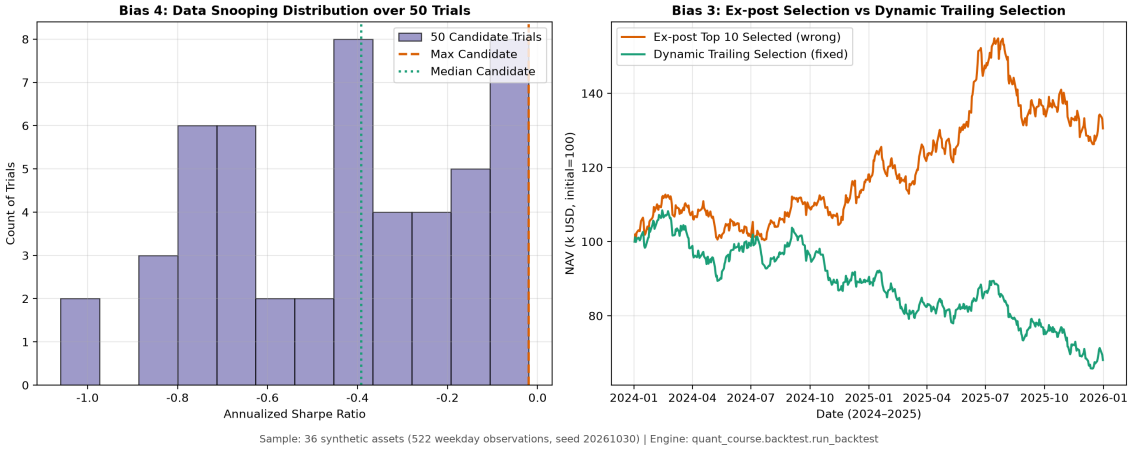


Figure 2: Data-snooping multiple comparisons and selection bias

- **Left panel (Panel A: Data-snooping distribution)**:
  - Horizontal axis: annualized Sharpe ratio; vertical axis: frequency of trials.
  - Main finding: among 50 momentum strategies with different lookback periods, the median Sharpe is -0.3921 and even the maximum is -0.0193. Under the null, the expected maximum of 50 independent standard-normal trials is 2.2763 (BLdP approximation) and 2.2491 (exact value); the Bonferroni-adjusted p-value in this example is 1.0000. The candidate strategies are correlated and their returns do not satisfy the independent-normal assumption, so this is a teaching example about multiple comparisons, not a formal significance test.
- **Right panel (Panel B: Selection bias)**:
  - Horizontal axis: the same 522-observation synthetic weekday grid; vertical axis: NAV (thousands of USD).
  - Comparison: the orange solid line is the 10 stocks with the best full-period performance selected ex post (Wrong, ex-post), with ending NAV of 130.54 k USD and Sharpe of 0.8164; the green solid line is a dynamic screen based on a rolling 60-day economic-return Sharpe (Fixed, trailing 60d), with ending NAV of 68.06 k USD and Sharpe of -0.9108.

10.3 Figure 3: Overfitting decay and parameter-mining grid surface

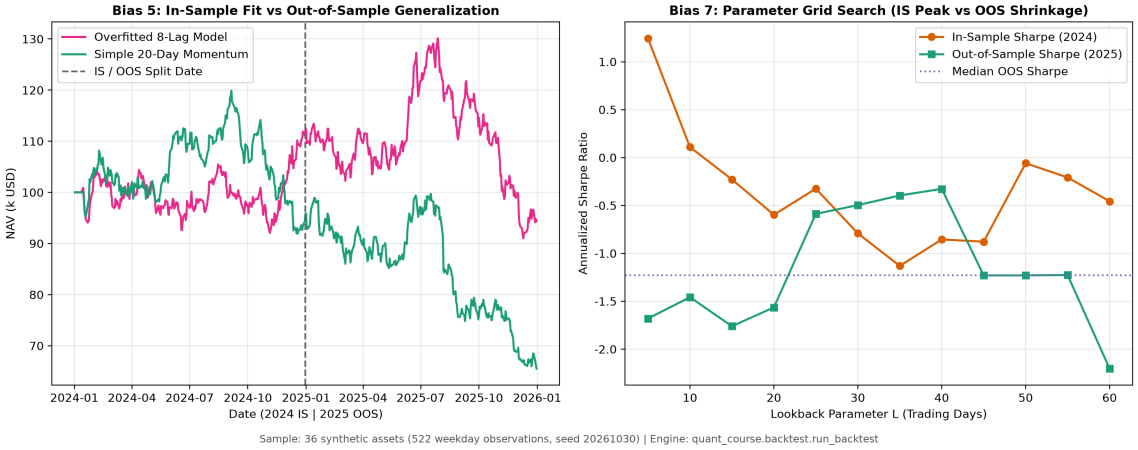


Figure 3: Overfitting decay and parameter-mining grid surface

- **Left panel (Panel A: In-sample and out-of-sample overfitting comparison)**:
  - Horizontal axis: date, with 2024-12-31 as the IS/OOS split; vertical axis: NAV.
  - Main finding: the model uses eight close-to-close economic returns available through the signal close to predict the next trading day's gross open-to-close return. The unregularized regression (magenta) has IS Sharpe of 0.7041, falling to -0.6788 OOS; the 20-day momentum benchmark (green) has IS Sharpe of -0.1243 and OOS Sharpe of -1.5639. Neither shows stable OOS performance, and a single split cannot establish that the simple strategy is superior.
- **Right panel (Panel B: Parameter-mining grid)**:
  - Horizontal axis: lookback parameter L (5 to 60 days); vertical axis: annualized Sharpe ratio.
  - Main finding: the parameter selected in the training period, $L^* = 5$ days, has IS Sharpe of 1.2443 and 2025 OOS Sharpe of -1.6799, a shrinkage of 2.9242. The selected parameter's OOS performance is below the median OOS value across the full grid, -1.2301; the grid's OOS results are for diagnosis only and do not enter parameter selection.

10.4 Figure 4: Corporate-action accounting and transaction-cost sensitivity

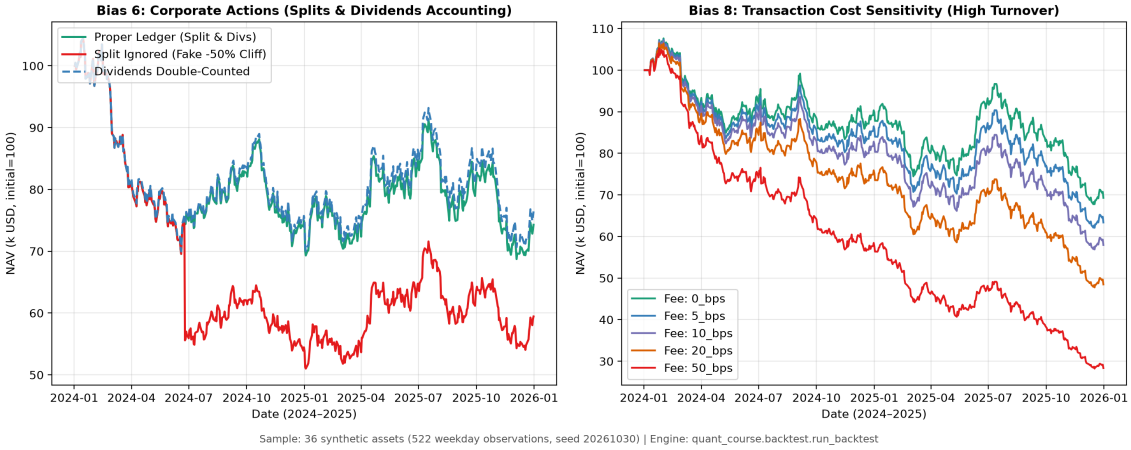


Figure 4: Corporate-action accounting and transaction-cost sensitivity

- **Left panel (Panel A: Corporate-action accounting)**:
  - Main finding: the original figure's "Fake -50% Cliff" refers to the mechanical halving of one stock's market value when its shares are not adjusted; it is not the NAV decline of this portfolio. The historical sample for this portfolio shows -26.20%, while the cash/B mini-example in Section 3 shows -25%. The red line that ignores the split drops 26.20% on day 125 and ends at NAV of 59.45 k USD; the correctly reconciled green line transitions smoothly and ends at NAV of 74.28 k USD; the blue dashed line shows the artificial inflation from double-crediting dividends while using adjusted prices (ending NAV 76.44 k USD).
- **Right panel (Panel B: Transaction-cost ladder)**:
  - Horizontal axis: the same 522-observation synthetic weekday grid; vertical axis: NAV.
  - Main finding: in the high-turnover momentum strategy (annualized turnover of 6912.25% in the 10 bps scenario), ending NAV is 69.29 k USD (CAGR -16.23%) with a 0 bps one-way fee, 57.96 k USD (CAGR -23.15%) with a 10 bps fee, and 28.36 k USD (CAGR -45.58%) with a 50 bps fee. NAV and Sharpe both decline strictly as the fee rate rises.

11. Real Market Example

11.1 Historical-method example and its data-rights boundary

This section retains the historical methodological illustration previously based on Kenneth R. French's data. The raw, extracted, and processed data are not included in the default I3 package, and the rights, subjects, and intended uses remain unresolved. The A10-A design and A10-A.1 technical protocol do not replace execution authorization; G05/A10-B has not been authorized. The historical study is not rerun here. The previously used monthly returns for 10 industry portfolios illustrate selection bias over a long span of real historical data:

- **Data-source URL**: `https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/10_Industry_Portfolios_CSV.zip`
- **Industry-data manifest**: `data/manifests/french_industries_2026-10-05.json` (SHA-256: `3b71cd5f781f9790cc380195c22635e03e9c9e27a5f43cd2d0d46fb12efa8271`)
- **Raw industry ZIP SHA-256**: `ebd937a4b995d821867aeb956a96f09a666386f38fa82516b7d1767ee4a49ae5` (hash of the raw archive; it differs from the hash of the JSON manifest below)
- **Processed industry file**: `data/processed/french_10_industries_monthly.csv` (SHA-256: `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`)
- **Risk-free-rate manifest**: `data/manifests/french_factors_2026-10-05.json` (SHA-256: `597c9703afa623a63b0d5fbf36dc941aac90d00948aeca302f151ed3c854000b`)
- **Processed risk-free-rate file**: `data/processed/french_factors_3_monthly.csv` (SHA-256: `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`)
- **Sample period and frequency**: 1926-07 through 2026-08, 1,202 consecutive monthly observations, with returns in decimal form per period.
- **Disclaimer and non-PIT status**: Kenneth French's 10 industry data are **aggregate, value-weighted industry portfolios for the full market**. The historical data have been recalculated after multiple revisions; they are a **non-point-in-time (non-PIT) snapshot and do not contain directly tradable, unadjusted constituent-stock quotes**. This example cannot support a claim of live-trading alpha and is only a methodological illustration for academic research.

11.2 Fifty-year split: ex-post selection versus ex-ante selection

We compare two non-overlapping historical periods without assuming that the return series are statistically independent (figures are quoted from `ch30_results.json["real_market_french"]`):

- **Period 1 (1926-07 through 1975-12; 594 months)**: Sharpe ratios for the 10 industries are calculated over Period 1. The three highest-ranked industries are:
  1. `Hlth` (Health): Sharpe = `0.4657`
  2. `Telcm` (Telecom): Sharpe = `0.4116`
  3. `HiTec` (High Tech): Sharpe = `0.4106`

  The equal-weighted portfolio of the top three industries in Period 1 has a Sharpe of `0.4701`, compared with `0.4250` for the unconditional equal-weighted benchmark across all 10 industries; its excess Sharpe is `+0.0451`.
- **Full-sample view (1926-07 through 2026-08; 1,202 months)**: In the ex-post full-sample calculation, the top three industries by full-period Sharpe change to `NoDur` (0.5099), `Hlth` (0.5044), and `Shops` (0.4577).
- **Period 2 evaluation (1976-01 through 2026-08; 608 months)**: We compare the actual performance of three strategies in Period 2:
  1. **Wrong version (top three selected ex post using the full sample, with look-ahead selection bias)**: Holding the ex-post full-sample picks NoDur, Hlth, and Shops produces a Period 2 Sharpe of **`0.6519`** (CAGR = `13.15%`), with excess Sharpe of `+0.0372` relative to the equal-weighted 10-industry benchmark.
  2. **Fixed version (top three selected ex ante using Period 1 only)**: Holding only the Period 1 picks (Hlth, Telcm, and HiTec) in Period 2 produces an actual Sharpe of **`0.5658`** (CAGR = `12.46%`), with excess Sharpe of `-0.0489` relative to the equal-weighted 10-industry benchmark.
  3. **Benchmark portfolio (unconditional equal weight across 10 industries)**: The Period 2 Sharpe is `0.6147` (CAGR = `12.75%`).

  Metric definition and decay:

- Sharpe difference between the wrong and fixed versions: `0.6519 - 0.5658 = +0.0862` (inflated by ex-post selection).
- Change in the ex-ante portfolio's excess Sharpe (Excess Decay): its excess Sharpe relative to the benchmark changes from `+0.0451` in Period 1 to `-0.0489` in Period 2; the decline in excess Sharpe relative to the benchmark is:

$$\text{Excess Decay} = (-0.0489) - (+0.0451) = -0.0940$$

This value shows that the fixed portfolio selected using the first half of the sample did not maintain its excess Sharpe advantage over the benchmark in the second half.

12. Common Mistakes

This section examines eight forms of bias in depth. Each follows the same structure: "incorrect example → why it is wrong → runnable incorrect code → corrected code → numerical difference on the same data → limitations that remain after the correction."

12.1 Bias 1: Look-ahead bias

- **Incorrect example**: In a monthly-rebalanced model, a company releases a positive earnings preannouncement after the close at 16:30 on Friday, 2024-03-29. The researcher labels the announcement as a March feature and generates a trade instruction before the close on March 29, or even executes it that day.
- **Why it is wrong**: The information release time $t_{\text{avail}} > s$ (Friday's close), so the information is not yet available at the Friday decision and execution times.
- **Runnable incorrect code** (shows the timestamp being backdated):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=3)
P = pd.DataFrame({"A": [10.0, 11.0, 12.0]}, index=dates)
E = pd.DataFrame({"A": [10.0, 11.0, 12.0]}, index=dates)

# Incorrect: the announcement occurs after the close on date 1 (16:30), but the
    ↪ signal is backdated to date 0.

# With lag=1, entry is early at 11 on date 1; this position does not earn the
    ↪ preceding 10-to-11 gap.
announcement_date = dates[1]
leaked_signal_date = dates[0]
tw_wrong = pd.DataFrame({"A": [1.0]}, index=[leaked_signal_date])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_wrong = run_backtest(P, tw_wrong, cfg, exec_prices=E)
assert res_wrong.fills["trade_time"].iloc[0] == dates[1]
```

This three-row example demonstrates backdating and invalid early entry only. The account buys at 11 on date 1 and is marked at 11 at that close, so its closing NAV remains USD 1,000; it does not earn the preceding 10-to-11 gap. The holding-period construction across the announcement in §6.1 and the historical results below use their own data and protocols; they are not outputs of this three-row code.

Corrected code (the announcement is released at 16:30 after the close on dates[1]; under a close-decision convention, the signal is formed at the next day's close on dates[2], and lag=1 executes at the open on dates[3]):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)
P = pd.DataFrame({"A": [10.0, 11.0, 11.0, 11.0]}, index=dates)
E = pd.DataFrame({"A": [10.0, 11.0, 11.0, 11.0]}, index=dates)

# The 16:30 announcement on date[1] is unavailable at its close.
announcement_date = dates[1]
causal_signal_date = dates[2]
tw_fixed = pd.DataFrame({"A": [1.0]}, index=[causal_signal_date])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_fixed = run_backtest(P, tw_fixed, cfg, exec_prices=E)
assert res_fixed.fills["signal_time"].iloc[0] == dates[2]
assert res_fixed.fills["trade_time"].iloc[0] == dates[3]
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_1_lookahead"]`):
  - fixed (decision at the close of the observation day after the announcement): ending NAV = 69300.37 USD, Sharpe = -0.7760.
  - wrong (future announcement backdated to a signal before its release): ending NAV = 112500.12 USD, Sharpe = 0.3730.
  - Difference: NAV difference +43199.74 USD; Sharpe difference +1.1490 (sign: wrong > fixed).
- **Empirical distribution across 20 additional random seeds** (quoted from `ch30_results.json["multi_seed_distribution"]["lookahead_diff_sharpe"]`):
  - Mean difference: +1.1100; standard deviation: 0.2150; range: [0.7854, 1.6389]; positive-difference share: 100.0%. The main-seed difference, 1.1490, lies within the observed range across these 20 draws.
- **Limitations that remain after the correction**: lag=1 means only that a signal executes at least on the next observation; by itself, it does not prove that an after-hours announcement was ingested before the order cutoff. This chapter's default uses after-hours announcements in a signal formed at the next observation day's close. An earlier decision requires the actual announcement time, receipt/ingestion time, and order cutoff time, with their ordering demonstrated. Causal timestamps still do not resolve one-price limit-up conditions, trading suspensions, execution capacity, or later restatements.

12.2 Bias 2: Survivorship bias

- **Incorrect example**: Export "currently listed stocks" from a market-data application and use that list as the backtest universe for several preceding years.
- **Why it is wrong**: The current list is a survivor set selected by past outcomes and automatically excludes failed companies that defaulted, were delisted, or were removed from the exchange.
- **Runnable incorrect code** (builds the universe using future survival status):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)

# Incorrect: at the start of the backtest (t=0), use end-of-period survival status to
    ↪ remove stock C, which is delisted on date 2.
all_universe = ["A", "B", "C"]
future_delisted = ["C"]
survivors_only = [s for s in all_universe if s not in future_delisted]
P_surv = pd.DataFrame({"A": [10.0, 11.0, 12.0, 13.0], "B": [20.0, 21.0, 22.0, 23.0]},
    ↪ index=dates)
tw_wrong = pd.DataFrame({"A": [0.5], "B": [0.5]}, index=[dates[0]])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_wrong = run_backtest(P_surv, tw_wrong, cfg)
assert "C" not in res_wrong.positions.columns
```

- **Corrected code** (uses the point-in-time full universe and records delisting):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)

# Correction: retain the full universe that existed at the time; the delistings
    ↪ mapping liquidates the position for cash on the delisting date.
P_all = pd.DataFrame({
    "A": [10.0, 11.0, 12.0, 13.0],
    "B": [20.0, 21.0, 22.0, 23.0],
    "C": [10.0, 5.0, float("nan"), float("nan")]
}, index=dates)
tw_fixed = pd.DataFrame({"A": [0.33], "B": [0.33], "C": [0.34]}, index=[dates[0]])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_fixed = run_backtest(P_all, tw_fixed, cfg, delistings={"C": (dates[2], 1.0)})
assert res_fixed.ledger["delist_cash"].iloc[2] > 0
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_2_survivorship"]`):
  - fixed (PIT universe with delisting liquidation): ending NAV = 80874.56 USD, Sharpe = -0.5565.
  - wrong (survivors only): ending NAV = 88245.23 USD, Sharpe = -0.2950.
  - Difference: NAV difference +7370.67 USD; Sharpe difference +0.2616 (sign: wrong > fixed).
- **Empirical distribution across 20 additional random seeds** (quoted from `ch30_results.json["multi_seed_distribution"]["survivorship_diff_sharpe"]`):
  - Mean difference: +0.3174; standard deviation: 0.0306; range: [0.2677, 0.3894]; positive-difference share: 100.0%. The main-seed difference of 0.2616 and the 20 positive differences are retained as observations from their respective constructed draws; they do not establish a robust direction across all markets or seeds.
- **Limitations that remain after the correction**: A PIT universe and recorded delisting liquidation still do not resolve off-exchange transfer slippage at a discount during a delisting suspension, or the omission of micro-cap delisted stocks from early vintages of a historical full-universe database.

12.3 Bias 3: Selection bias

- **Incorrect example**: Inspect the full-sample results for the stock universe over 2024–2025, then put the best-performing stocks into the backtest portfolio in advance.
- **Why it is wrong**: The selection rule is conditional on future full-sample outcomes.
- **Runnable incorrect code** (selects stocks using a full-sample ex-post metric):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)
P = pd.DataFrame({"A": [10.0, 15.0, 18.0, 20.0], "B": [20.0, 18.0, 16.0, 15.0]},
    ↪ index=dates)

# Incorrect: after the full sample ends, select stocks using their mean return over
    ↪ all dates; this small example has no corporate actions.
full_sample_top = P.pct_change().dropna().mean().idxmax()
assert full_sample_top == "A"
tw_wrong = pd.DataFrame({full_sample_top: [1.0]}, index=[dates[0]])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_wrong = run_backtest(P[[full_sample_top]], tw_wrong, cfg)
assert res_wrong.ledger["nav"].iloc[-1] > 1000.0
```

- **Corrected code** (dynamic ex-ante selection using only a historical rolling window):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=6)
P = pd.DataFrame({"A": [10.0, 11.0, 10.0, 12.0, 11.0, 13.0],
                  "B": [10.0, 9.0, 10.0, 9.0, 11.0, 10.0]}, index=dates)

# This small example has no corporate actions. At each close decision, compare only
    ↪ the preceding two days of returns.
daily_returns = P.pct_change()
rows = []
for i in range(2, len(dates) - 1):
    trailing_mean = daily_returns.iloc[i - 1:i + 1].mean()
    selected = trailing_mean.idxmax()
    rows.append({"A": float(selected == "A"), "B": float(selected == "B")})
tw_fixed = pd.DataFrame(rows, index=dates[2:-1])
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_fixed = run_backtest(P, tw_fixed, cfg)
assert tw_fixed.nunique().iloc[0] > 1 or tw_fixed.nunique().iloc[1] > 1
assert res_fixed.ledger["nav"].iloc[-1] > 0
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_3_selection"]`; this comparison uses the full PIT universe that includes delisted S35 and S36):
  - fixed (rolling dynamic forward selection): ending NAV = 68057.83 USD, Sharpe = -0.9108.
  - wrong (10 best stocks selected ex post): ending NAV = 130536.57 USD, Sharpe = 0.8164.
  - Difference: NAV difference +62478.74 USD; Sharpe difference +1.7272 (sign: wrong > fixed).
- **Empirical distribution across 20 additional random seeds** (quoted from `ch30_results.json["multi_seed_distribution"]["selection_diff_sharpe"]`):
  - Mean difference: +1.2671; standard deviation: 0.3014; range: [0.6812, 1.8841]; positive-difference share: 100.0%.
- **Limitations that remain after the correction**: A dynamic ex-ante rule still needs a declared target population. If a "continuously profitable in the past" condition was known at the time, it can define a legitimate prespecified investment universe. The corresponding selection problem arises only when this conditional population is presented as an unconditional result for the whole market or future eligibility is written back into the historical record.

12.4 Bias 4: Data snooping

- **Incorrect example**: Try 50 momentum signals with different lookback periods on the same historical dataset, report only the one with the highest backtest Sharpe, and claim to have found significant alpha.
- **Why it is wrong**: Taking the maximum of independent random variables systematically inflates the nominal test statistic and ignores extreme-value inflation under multiple testing.
- **Runnable incorrect code** (reports only the maximum without logging the multiple trials):

```
import numpy as np

# Fifty candidate return series on a fixed sample; the incorrect approach reports
    ↪ only the best Sharpe.
rng = np.random.default_rng(30)
trial_returns = rng.normal(0.0002, 0.01, size=(522, 50))
sharpes = trial_returns.mean(axis=0) / trial_returns.std(axis=0, ddof=1) * np.sqrt
    ↪ (252)
best_metric = float(sharpes.max())
assert np.isfinite(best_metric)
```

- **Corrected code** (logs every trial in TrialLedger and adjusts the full p-value vector with Bonferroni):

```
import numpy as np
from scipy import stats
from quant_course.research_protocol import TrialLedger, bonferroni_adjust

# Retain all 50 candidate return series from the incorrect example and record every
    ↪ trial.
rng = np.random.default_rng(30)
trial_returns = rng.normal(0.0002, 0.01, size=(522, 50))
ledger = TrialLedger()
p_values = []
for m, daily_returns in enumerate(trial_returns.T):
    sharpe = float(daily_returns.mean() / daily_returns.std(ddof=1) * np.sqrt(252))
    t_stat = sharpe * np.sqrt(len(daily_returns) / 252)
    p_values.append(float(2 * stats.norm.sf(abs(t_stat))))
    ledger.record({"candidate": m + 1}, metric=sharpe, n_obs=len(daily_returns))

assert ledger.n_trials == 50
p_bonf_all = bonferroni_adjust(np.array(p_values))

# Pass the full vector of 50 p-values to apply the multiple-comparison adjustment
    ↪ automatically.
assert (p_bonf_all >= np.array(p_values)).all()
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_4_data_snooping"]`):
  - Number of candidate trials: 50.
  - Maximum selected Sharpe: -0.0193 (median: -0.3921; mean: -0.4369).
  - Expected maximum across 50 normal trials under the null: exact value = 2.2491; BLdP approximation = 2.2763; Gumbel approximation = 2.3072.
  - Nominal p-value = 0.9782; Bonferroni-adjusted p-value = 1.0000.
- **Limitations that remain after the correction**: A trial ledger can constrain attempts made by mechanical code, but it cannot resolve informal trials left unrecorded in a researcher's mind (Mental Trials) or the data snooping produced by decades of collective trial and error on public data in academia.

12.5 Bias 5: Overfitting

- **Incorrect example**: In 2024 (the in-sample, IS period), fit the next trading day's return using eight historical-return features, then treat the in-sample fit as future performance.
- **Why it is wrong**: The model has too many degrees of freedom, forcing it to memorize idiosyncratic in-sample noise without structural generalization.
- **Runnable incorrect code** (fits a high-degree polynomial only in sample):

```
import numpy as np

# Incorrect: fit only on the training sample and leave no out-of-sample validation.
rng = np.random.default_rng(2)
x = rng.uniform(-1, 1, 30)
y = 0.6 * x + rng.normal(0, 0.3, size=30)
x_is, y_is = x[:20], y[:20]
poly_overfit = np.polyfit(x_is, y_is, deg=8)
assert len(poly_overfit) == 9

# The target includes random noise; fit on training only is not out-of-sample
    ↪ evidence.
```

- **Corrected code** (constrains the model and validates on a held-out sample):

```
import numpy as np

# Fix the random seed; split the training and held-out periods in time order, and fit
    ↪  using only the training segment.
rng = np.random.default_rng(2)
x = rng.uniform(-1, 1, 30)
y = 0.6 * x + rng.normal(0, 0.3, size=len(x))
split = 20
poly_overfit = np.polyfit(x[:split], y[:split], deg=8)
poly_simple = np.polyfit(x[:split], y[:split], deg=1)
train_mse_overfit = np.mean((y[:split] - np.polyval(poly_overfit, x[:split])) ** 2)
train_mse_simple = np.mean((y[:split] - np.polyval(poly_simple, x[:split])) ** 2)
test_mse_overfit = np.mean((y[split:] - np.polyval(poly_overfit, x[split:])) ** 2)
```

```
test_mse_simple = np.mean((y[split:] - np.polyval(poly_simple, x[split:])) ** 2)
assert train_mse_overfit < train_mse_simple
assert test_mse_overfit > test_mse_simple
```

- In this fixed random example, the degree-8 model's training MSE is about 0.0382 and its held-out MSE is about 0.2412. A single holdout can assess generalization over that period, but cannot establish future stability.
- The strategy-level label in the constructed-market CLI is strictly the gross open-to-close return on the trading day after the signal-day close; the eight features are economic close-to-close returns available through the signal close. The overnight gap occurs before execution and is not included in the new position's label. Backtest NAV is still calculated from raw prices, corporate actions, and the fee ledger.
- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_5_overfitting"]`):
  - Eight-feature regression: IS Sharpe = 0.7041; OOS Sharpe = -0.6788; change = -1.3829.
  - 20-day momentum benchmark: IS Sharpe = -0.1243; OOS Sharpe = -1.5639; change = -1.4396. Neither strategy generalized in this constructed sample, so this does not support a claim that the simple benchmark is superior.
- **Limitations that remain after the correction**: A simple out-of-sample split can reveal overfitting, but cannot resolve iterative leakage from repeatedly inspecting OOS results and then changing the model. Chapter 31 formalizes this issue further with Purged CV and Embargo.

12.6 Bias 6: Incorrect corporate-action handling

- **Incorrect examples**:
  - Error A: use unadjusted prices directly in the backtest without passing a splits table to the engine.
  - Error B: backtest using adjusted prices while also enabling cash credits through a dividends table.
- **Why they are wrong**:
  - Error A halves the share price on the ex-split date without doubling the share count, creating an artificial 50% crash.
  - Error B credits dividends a second time in the cash ledger even though the adjusted prices already reflect dividend reinvestment, double-counting the dividend return.

- **Runnable incorrect code** (shows the artificial drop caused by omitting the split parameter):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)

# Incorrect: S05 undergoes a 2:1 split, but the splits parameter is omitted, causing
    ↪ an artificial crash.
P = pd.DataFrame({"S05": [100.0, 100.0, 50.0, 50.0]}, index=dates)
tw = pd.DataFrame({"S05": [1.0, 1.0, 1.0, 1.0]}, index=dates)
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_wrong = run_backtest(P, tw, cfg)

# On day 2 the position is held through the split, but the share count is not
    ↪ adjusted; NAV drops 50%.
assert res_wrong.ledger["nav"].iloc[2] == 500.0
```

- **Corrected code** (passes raw prices and a split-multiplier table):

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest, reconcile

dates = pd.bdate_range("2024-01-02", periods=4)

# Correction: pass unadjusted prices and the split-multiplier table.
P = pd.DataFrame({"S05": [100.0, 100.0, 50.0, 50.0]}, index=dates)
splits = pd.DataFrame({"S05": [1.0, 1.0, 2.0, 1.0]}, index=dates)
tw = pd.DataFrame({"S05": [1.0, 1.0, 1.0, 1.0]}, index=dates)
cfg = BacktestConfig(initial_cash=1000.0, lag=1)
res_fixed = run_backtest(P, tw, cfg, splits=splits)
rec = reconcile(res_fixed, P, cfg, splits=splits)
assert rec["residual"].abs().max() < 1e-12
```

- **Error B and its correction: double-crediting a dividend when using adjusted prices**. The flat adjusted-price series below already incorporates the total return from a USD 1 dividend. Passing a dividends table to the ledger as well adds another USD 10. Production backtests should use unadjusted execution/mark prices with a clearly identified source, and pass corporate actions separately:

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=4)
tw = pd.DataFrame({"A": [1.0]}, index=[dates[0]])
dividends = pd.DataFrame({"A": [0.0, 0.0, 1.0, 0.0]}, index=dates)
cfg = BacktestConfig(initial_cash=1000.0, lag=1)

# Wrong: adjusted close is flat, while the ledger also credits a 1 USD dividend.
P_adjusted = pd.DataFrame({"A": [100.0, 100.0, 100.0, 100.0]}, index=dates)
wrong = run_backtest(P_adjusted, tw, cfg, dividends=dividends)

# Fix: use raw prices; record the 100-to-99 ex-date move and the 1 USD/share cash.
```

```
P_raw = pd.DataFrame({"A": [100.0, 100.0, 99.0, 99.0]}, index=dates)
fixed = run_backtest(P_raw, tw, cfg, dividends=dividends)
assert fixed.ledger["nav"].iloc[2] == 1000.0
assert wrong.ledger["nav"].iloc[2] == 1010.0
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_6_corporate_actions"]`):
  - fixed (compliant unadjusted-price accounting): ending NAV = 74281.57 USD; return on the split date = -0.0129.
  - split_ignored: ending NAV = 59445.09 USD; return on the split date = -0.2620 (an artificial 26.20% drop).
  - dividend_double_counted: ending NAV = 76444.05 USD (an overstatement of USD 2,162.48).
  - Maximum absolute ledger-reconciliation residual: 2.55e-11 USD (an internal accounting check at the stated scale; it does not prove that market-price inputs or all trading conditions are correct).
- **Limitations that remain after the correction**: Correctly entering splits and dividends can maintain cash-and-share accounting, but does not resolve complex subsidiary spin-offs, rights-offering dilution discounts, or differences in dividend withholding taxes across international markets.

12.7 Bias 7: Parameter mining

- **Incorrect example**: Run a dense in-sample grid search over momentum lookback days $L \in \{5, 10, \ldots, 60\}$ and choose the in-sample optimum, $L^* = 5$ days, as the live-trading parameter.
- **Why it is wrong**: It treats a local maximum from a discrete sample as a deep market invariant and ignores the parameter surface's flatness and the risk of overfitting.
- **Runnable incorrect code**:

```
import importlib.util
from pathlib import Path

spec = importlib.util.spec_from_file_location("ch30", Path("code/ch30_backtest_biases
    ↪ .py"))
ch30 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ch30)
data = ch30.build_constructed_market(seed=20261030)

# The parameter-scan function returns the actual in-sample performance of the
    ↪ backtest.
result = ch30.study_parameter_mining(data)
grid = result["grid"]
best_is = max(grid, key=lambda row: row["is_sharpe"])

# Incorrect interpretation: treat the in-sample peak as a future-realizable Sharpe.
claimed_future_sharpe = best_is["is_sharpe"]
```

```
assert best_is["lookback"] == result["best_is_lookback"]
```

- **Corrected code** (selects parameters in the training period and evaluates the selected parameter only in the test period):

```
import importlib.util
from pathlib import Path

spec = importlib.util.spec_from_file_location("ch30", Path("code/ch30_backtest_biases
    ↪ .py"))
ch30 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ch30)
data = ch30.build_constructed_market(seed=20261030)
result = ch30.study_parameter_mining(data)
best_L = result["best_is_lookback"]  # selected on the 2024 IS segment only
selected_oos_sharpe = result["best_param_oos_sharpe"]
print(f"Selected on 2024 only: L={best_L}; 2025 Sharpe={selected_oos_sharpe:.4f}")
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_7_parameter_mining"]`, after correcting the warm-up window):
  - Best in-sample parameter: $L^* = 5$ days; in-sample Sharpe = 1.2443.
  - Actual out-of-sample performance for that parameter: OOS Sharpe = -1.6799.
  - Median OOS value across the full grid: -1.2301.
  - OOS shrinkage: 2.9242 (from a positive mean return to the negative-return region).
- **Empirical distribution across 20 additional random seeds** (quoted from `ch30_results.json["multi_seed_distribution"]["parameter_mining_oos_shrinkage"]`):
  - Mean OOS shrinkage: 0.3974; standard deviation: 1.4743; positive-shrinkage share: 55.0%. The shrinkage direction differs across random seeds, so the evidence does not support a universal effect.
- **Limitations that remain after the correction**: Showing the full parameter surface reveals fragility, but does not resolve structural regime shifts that cause the true optimal parameter to drift over time.

12.8 Bias 8: Incorrect cost modeling

- **Incorrect example**: Develop a high-turnover momentum strategy with fee_rate = 0.0 in the backtest and claim that its annualized return is robust.
- **Why it is wrong**: It ignores market frictions. The proportional-fee illustration used consistently in this chapter has a material compounding effect on a high-turnover portfolio.

- **Runnable incorrect code**:

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=3)
P = pd.DataFrame({"A": [10.0, 11.0, 12.0]}, index=dates)
cfg_zero = BacktestConfig(initial_cash=1000.0, fee_rate=0.0, lag=1)
res_zero = run_backtest(P, pd.DataFrame({"A": [1.0]}, index=[dates[0]]), cfg_zero)
assert res_zero.ledger["fees"].sum() == 0.0
```

- **Corrected code**:

```
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=3)
P = pd.DataFrame({"A": [10.0, 11.0, 12.0]}, index=dates)
cfg_fee = BacktestConfig(initial_cash=1000.0, fee_rate=0.001, lag=1)
res_fee = run_backtest(P, pd.DataFrame({"A": [1.0]}, index=[dates[0]]), cfg_fee)
assert res_fee.ledger["fees"].sum() > 0.0
```

- **Numerical difference on the same data** (quoted from `ch30_results.json["bias_8_cost_modeling"]`):
  - Annualized turnover: 6912.25%.
  - Fee rate of 0 bps: ending NAV = 69293.03 USD; CAGR = -16.23%; Sharpe = -0.7744; total fees = 0.0 USD.
  - Fee rate of 5 bps: ending NAV = 63373.14 USD; CAGR = -19.76%; Sharpe = -0.9870; total fees = 7465.42 USD.
  - Fee rate of 10 bps: ending NAV = 57958.45 USD; CAGR = -23.15%; Sharpe = -1.1984; total fees = 14318.22 USD.
  - Fee rate of 20 bps: ending NAV = 48476.06 USD; CAGR = -29.50%; Sharpe = -1.6158; total fees = 26386.78 USD.
  - Fee rate of 50 bps: ending NAV = 28356.82 USD; CAGR = -45.58%; Sharpe = -2.7949; total fees = 52420.09 USD.

  The metrics decline strictly as the fee rate rises.

- **Limitations that remain after the correction**: The proportional-fee model covers linear commissions and regulatory fees, but does not resolve nonlinear market impact for large orders (such as the square-root law), stock-borrow costs and short-sale constraints, or liquidity droughts in extreme markets.

13. Quant Research Application

In professional institutional quantitative research, backtest-bias defenses cannot rely on researchers' diligence alone. They must be built into fully automated CI/CD gates:

```
[Researcher submits strategy code and configuration]
               |
               v
   [Gate 1: automated future-perturbation invariance check]
   (inject Gaussian noise after cutoff time t; assert historical signals/fills/nav
    before t remain exactly identical)
               | (pass)
               v
   [Gate 2: static scan of point-in-time (PIT) contracts]
   (check release/ingestion/decision channels; shift(-1) may validly construct
    labels, but its actual support must be checked when used for features)
               | (pass)
               v
   [Gate 3: TrialLedger audit]
   (extract the cumulative number of trials M; calculate Bonferroni/Holm
    thresholds and expected extreme-value inflation)
               | (pass)
               v
   [Gate 4: unadjusted-price, self-financing ledger execution and cost ladder]
   (bind actual shares to unadjusted prices/corporate actions; do not credit
    dividends again for a declared total-return proxy; check fees and reconciliation)
               | (pass)
               v
[Eligible to proceed to Chapter 31's out-of-sample splits and Chapter 32's inference
   ↪ process]
```

14. Chapter Summary

14.1 Bias-diagnostic checklist: "What information, when available, and where used?"

Before deploying any backtest, quantitative engineers should rigorously check each item in the following table:

Review dimension	Core checklist question	Red flags	Architectural solution
Information timing (Available When)	Was this field officially released by the exchange or data vendor by the backtest decision time s?	Using period_end or placing after-hours announcements directly into same-day features	Filter using both release and ingestion clocks, then check the decision time and delayed execution
Universe completeness (Which Universe)	Does the asset universe include all names that went bankrupt, were delisted, or were suspended during the backtest period?	Using the latest current-stock list; filtering symbols based on a special identifier	Maintain a dynamic PIT constituent database with delisting liquidation-price mappings
Selection objectivity (Used Where)	Was the choice of names or factors based on historical performance over the full sample?	"Allocate the portfolio to the stocks/industries with the best performance over the past three years"	Build the benchmark using only historical information available up to each cross-section date
Search multiplicity (How Many Tried)	How many variants did the code and algorithms try before the final strategy was chosen?	The paper or report omits the number of trials and reports only one attractive backtest	Require TrialLedger to record all attempts and calculate adjusted thresholds
Generalization	Does the number of free parameters match the effective number of independent observations?	Very high in-sample Sharpe followed by a rapid fall to negative returns out of sample	Strictly separate training and test sets, and introduce regularization and cross-validation gates
Accounting conservation	Do share count and cash accounting preserve value through splits and dividends?	Using adjusted prices as execution prices; NAV suddenly halves on the ex-date	Input only actual unadjusted prices and run the reconcile ledger
Parameter surface (Sensitivity)	Is the optimum an isolated peak? How does performance change under small parameter perturbations?	A small change in a 5-day parameter flips backtest Sharpe from positive to negative	Scan the full parameter-sensitivity surface and evaluate shrinkage toward the median
Realism of frictions (Friction Reality)	How does the strategy perform under explicit proportional-fee, slippage, and stock-borrow assumptions?	Assuming zero commissions; high-turnover strategy is highly sensitive to the fee rate	Run the 0/5/10/20/50 bps ladder and calculate the turnover threshold

15. Exercises

- **30-E01 (Conceptual: point-in-time causal inequality)**: On a discrete trading timeline, let the signal decision time be the close at s, and let the order execution time be $t_{\text{trade}} = s + \text{lag}$. If a macroeconomic data release occurs at 16:30 on date s, write the point-in-time causal inequality that constrains when the strategy may use that data. Explain separately whether ingestion at 09:45 on the next day prevents a decision at that day's close, and why it would violate a different premarket-decision contract that requires the order to be placed before 09:30 the next day. Do not conflate the two clocks.
- **30-E02 (Mathematical: survivorship-bias premium under a truncated normal distribution)**: Suppose the one-period stock return is $R \sim \mathcal{N}(0.01, 0.04^2)$. If a strategy counts only surviving names whose return is at least -0.03, use numerical values of the standard-normal density ϕ and cumulative distribution function Φ to calculate by hand the conditional mean return $\mathbb{E}[R \mid R \geq -0.03]$ for the surviving sample. Then calculate the one-period upward bias in the mean and the difference in the annualized arithmetic mean after multiplying by 252. Explain why the latter is not an achievable compounded annual return.
- **30-E03 (Mathematical: expected maximum-Sharpe inflation under multiple testing)**: Under the null hypothesis (true Sharpe ratio equal to 0), a quantitative team independently evaluates $M = 100$ factors. Use the extreme-value expectation approximation

$$\mathbb{E}[M_M] \approx \sqrt{2 \ln M} - \frac{\ln \ln M + \ln(4\pi)}{2\sqrt{2 \ln M}} + \frac{\gamma}{\sqrt{2 \ln M}}$$

  to calculate by hand the expected test statistic for the best factor. Then use the null distribution of the maximum, or an effective multiplicity-adjusted p-value, to determine whether a one-sided nominal statistic of 2.0 is significant at a 5% family-wise level. Do not compare it only with the expected maximum.
- **30-E04 (Proof: split neutrality in an unadjusted-price self-financing ledger)**: On date t, suppose the number of shares changes from q_{t-1} to $q_{t-1} \cdot S_t$, while the closing price changes from P_{t-1} to P_{t-1}/S_t. Prove that on a pure ex-split date with no cash dividend and no rebalancing, the ledger's daily reconciliation residual residual_t is identically 0 and the first derivative of total NAV with respect to the split ratio S_t is 0 everywhere.
- **30-E05 (Derivation: transaction-cost breakeven for a high-turnover strategy)**: Define $\tau_{L1} = \sum_i |\Delta x_i| / V_{\text{pre}}$ as the total L1 turnover used by the ledger (buy notional plus sell notional), and let c be the one-way proportional trading-fee rate. First define whether r_g is the accounting return before fees on the same holdings, or the no-fee counterfactual return after fees are deducted before reinvestment. Write the end-of-period wealth-growth equation after transaction costs for each definition. Derive an approximate lower bound on the gross-return-to-turnover ratio $\frac{r_g}{\tau_{L1}}$ required for the strategy to achieve positive long-run compound growth ($\mathbb{E}[\ln(1 + r_{\text{net}})] > 0$). If one-sided turnover is defined as $\tau_{1w} = \tau_{L1}/2$, how does the cost coefficient in the equation change?

---

16. Programming Exercises

- **30-P01 (Future-perturbation non-intrusion checker)**: Write a general Python check function `assert_future_perturbation_invariant` that takes a strategy function, market data, and a cutoff date. Perturb raw mark prices, execution prices, announcements, splits, dividends, and future delisting-liquidation data after the cutoff, then rerun the strategy. Assert that target weights, fills, positions, and NAV through and including the cutoff date match the original results. The strategy interface should return `(target_weights, BacktestResult)`.
- **30-P02 (Automatic multiple-testing adjustment using TrialLedger)**: Based on `TrialLedger` in `src/quant_course/research_protocol.py`, write an analysis function that takes a trial ledger and valid nominal p-values fully aligned by `trial_id`. Return the maximum and median metric, all nominal p-values, the Bonferroni and Holm adjusted sequences (in original candidate order and sorted order), and the rejected candidate IDs. Rejecting the null hypothesis does not establish that a strategy is "truly significant."
- **30-P03 (Point-in-time dynamic constituent filter and delisting-liquidation allocator)**: Write a dynamic-weight function that takes mappings of listing and delisting dates and, on each monthly rebalance date, outputs a $1/N$ equal-weight target-weight matrix containing only the stocks still listed then. If a stock is delisted while held, close it at the given delisting liquidation price and return the cash to the cash ledger.
- **30-P04 (Double-counting detector for adjusted prices and an unadjusted ledger)**: Write a defensive validation module that requires callers to provide the price basis from source metadata (`raw_unadjusted` or `adjusted`). Allow prices to enter a ledger together with a cash-dividend table only when the basis is explicitly declared `raw_unadjusted`. An ex-dividend price drop may produce a diagnostic warning, but cannot by itself prove whether prices are adjusted; explain why the diagnostic can produce false positives and false negatives.

---

17. Challenge Problems

- **30-C01 (Open challenge: PIT causal isolation for restated financial statements)**: In real company financial statements, listed companies often publish restatements (10-K/A) several quarters or even years later, revising historical quarterly net income and equity. If a quantitative team wants to backtest a stock-selection strategy based on return on equity (ROE), design an underlying data architecture with a version table, an effective timestamp (`pit_effective_at`), and a dual-key alignment mechanism. Show why historical signal generation cannot be contaminated by future accounting restatements.
- **30-C02 (Open challenge: estimating effective independent trials in a continuous parameter search)**: In a moving-average crossover strategy, short-period parameter $S \in$

$[5, 20]$ and long-period parameter $L \in [30, 120]$ are continuous or densely gridded, and the return series for neighboring grid points are highly correlated. Applying Bonferroni directly using the total number of grid points (for example, $15 \times 90 = 1350$) may be conservative. Propose an estimate based on eigenvalue decomposition of the return-correlation matrix or an effective number of tests (M_{eff}), and give a counterexample showing when that estimate can fail.

- **30-C03 (Derivation challenge: feasibility of a closed-form solution for nonlinear market impact and a self-financing equation)**: Chapter 29 gives the post-fee NAV equation under proportional linear fees: $V' = V - c\sum_i |w_i V' - x_i|$. Let $z_i = |\Delta x_i| = |w_i V' - x_i|$, and consider total cost

$$C_i(z_i) = c_1 z_i + c_2 \sigma_i z_i \sqrt{z_i / \text{ADV}_i},$$

where z_i and ADV_i are notionals in the same currency, σ_i is a dimensionless return volatility, c_1 is a fee rate, and c_2 is dimensionless. Give sufficient conditions that guarantee a unique positive root on $[0, V_{\text{pre}}]$, explain why Newton iteration is no longer presumed to converge monotonically, and describe how to use a bracketing root-finder if the conditions fail, including which conclusions can and cannot be drawn.

---

18. Solutions / Solution Hints

18.1 Full solutions to conceptual and derivation problems

30-E01 Solution

1. The release and ingestion clocks jointly determine the available time: $a(e) = \max(t_{\text{avail}}, t_{\text{ingest}})$. A valid feature requires $a(e) \le s_{\text{close}}$; weights must be measurable with respect to $\mathcal{F}_s$, and execution occurs at a later valid observation step.
2. Under the teaching clock in this problem, the market closes at 15:00 and the announcement arrives at 16:30, so it cannot be used for that day's close. If it has been ingested by 09:45 the next day, it can be used for that day's 15:00 close decision; after lag=1, execution occurs at the open of the following observation day. The 09:45 ingestion does not introduce look-ahead on this route.
3. If the contract instead forms a premarket order before 09:30 the next day, ingestion at 09:45 is after the decision and cutoff, so the data cannot be assumed available. The same lag label cannot replace checking the ordering of the actual release, ingestion, and order-cutoff times. If ingestion occurs later than the next close, the close-decision route in this problem must also be delayed.

30-E02 Solution

1. **Standardized truncation point**:
$$\alpha = \frac{-0.03 - 0.01}{0.04} = \frac{-0.04}{0.04} = -1.0$$
2. **Tabulated values**: The standard-normal density is $\phi(-1.0) = \frac{1}{\sqrt{2\pi}} e^{-0.5} \approx 0.24197$. The cumulative distribution is $\Phi(-1.0) = 1 - \Phi(1.0) \approx 1 - 0.84134 = 0.15866$.
3. **Inverse Mills ratio**:
$$\lambda(-1.0) = \frac{\phi(-1.0)}{1 - \Phi(-1.0)} = \frac{0.24197}{0.84134} \approx 0.28760$$
4. **Conditional expectation**:
$$\mathbb{E}[R \mid R \geq -0.03] = 0.01 + 0.04 \times 0.28760 = 0.01 + 0.01150 = 0.02150 \quad (2.150\%)$$
5. **Spurious upward bias**: The unrounded one-period mean difference is about 0.011504, or 1.1504 percentage points. Multiplying by 252 gives about 2.8990, or an annualized arithmetic-mean difference of 289.90 percentage points. This is a rescaling under the one-period truncation model; it is neither an achievable compounded annual return nor a conclusion under a condition that the same company survives to year-end.

30-E03 Solution

1. **Calculate the logarithmic terms**: Given $M = 100$, $\ln 100 \approx 4.60517$. $\sqrt{2 \ln M} = \sqrt{2 \times 4.60517} = \sqrt{9.21034} \approx 3.03485$. $\ln \ln 100 = \ln(4.60517) \approx 1.52723$. $\ln(4\pi) = \ln(12.56637) \approx 2.53102$.
2. **Substitute into the formula for the expected maximum**:
$$\mathbb{E}[M_{100}] \approx 3.03485 - \frac{1.52723 + 2.53102}{2 \times 3.03485} + \frac{0.57722}{3.03485}$$
$$= 3.03485 - \frac{4.05825}{6.06970} + 0.19020 = 3.03485 - 0.66861 + 0.19020 \approx 2.55644$$
3. **Post-selection significance**: The value 2.55644 is an asymptotic expectation approximation; it is not a 5% critical value. For the maximum of 100 independent one-sided standard-normal tests, at an observed maximum of $z = 2$ the family-wise p-value is
$$p_{\max} = 1 - \Phi(2)^{100} \approx 0.8999.$$
This is greater than 0.05, so the family-wide null cannot be rejected. Bonferroni also gives $\min(1, 100[1 - \Phi(2)]) = 1$. For a two-sided statistic, use the corresponding null distribution of the maximum absolute value; this p-value does not automatically apply to Sharpe ratios from correlated strategies.

30-E04 Solution

1. **Definition of the ledger-recursion residual**: By the Chapter 29 formula, the explained equity change on the ex-date is:

$$\text{explained}_t = \Delta C_t + q_{t,\text{pre}}(E_t - P_{t-1}^{\text{adj}}) + q_{t,\text{post}}(P_t - E_t) - \text{fees}_t$$

2. **Substitute the variables for a pure split day**: There is no rebalancing trade ($q_{t,\text{post}} = q_{t,\text{pre}}$, $\text{fees}_t = 0$) and no cash dividend ($\Delta C_t = 0$). Before the open, the position is adjusted to $q_{t,\text{pre}} = q_{t-1}S_t$; the ex-date reference price is $P_{t-1}^{\text{adj}} = P_{t-1}/S_t$. Substituting and expanding:

$$\text{explained}_t = (q_{t-1}S_t)(E_t - P_{t-1}/S_t) + (q_{t-1}S_t)(P_t - E_t)$$

$$= (q_{t-1}S_tE_t - q_{t-1}P_{t-1}) + (q_{t-1}S_tP_t - q_{t-1}S_tE_t) = q_{t-1}S_tP_t - q_{t-1}P_{t-1}$$

   The actual NAV change for the day is:

$$\text{NAV}_t - \text{NAV}_{t-1} = (C + q_{t-1}S_tP_t) - (C + q_{t-1}P_{t-1}) = q_{t-1}S_tP_t - q_{t-1}P_{t-1}$$

   The two quantities are equal, so the residual $\text{residual}_t = (\text{NAV}_t - \text{NAV}_{t-1}) - \text{explained}_t \equiv 0$.
3. **Verify neutrality of the derivative**: Immediately after the split, NAV is $V_{\text{split}} = C_{t-1} + (q_{t-1}S_t) \cdot (P_{t-1}/S_t) = C_{t-1} + q_{t-1}P_{t-1}$. Thus, $\frac{\partial V_{\text{split}}}{\partial S_t} = 0$; the first derivative with respect to the split multiplier is identically zero, proving value neutrality.

30-E05 Solution

1. **Accounting return before fees**: Let r_g be the accounting P&L before fees on the same actual transaction/holding path, measured relative to V_{pre}, with costs measured against the same denominator. Let $\tau = \sum |\Delta x|/V_{\text{pre}}$. Under this definition,

$$1 + r_{\text{net}} = 1 + r_g - c\tau$$

   is an accounting identity. It is not the no-fee counterfactual return on the same risk weights minus fees. If $r_g^{(0)}$ is the no-fee counterfactual return after fees are deducted first and the remaining capital is reinvested, then

$$1 + r_{\text{net}} = (1 - c\tau)(1 + r_g^{(0)}),$$

   which includes the cross-term $-c\tau r_g^{(0)}$ and also requires $c\tau < 1$. The two accounting sequences cannot be mixed.
2. **Necessary condition**: When $1 + r_{\text{net}} > 0$ and the relevant expectations are integrable, Jensen's inequality gives

$$0 < \mathbb{E}\ln(1 + r_{\text{net}}) \leq \ln(1 + \mathbb{E}r_{\text{net}}).$$

   Thus, under the accounting-return definition, $\mathbb{E}r_g > c\mathbb{E}\tau$ is necessary but not sufficient. If $0 < \mathbb{E}\tau < \infty$, then $\mathbb{E}r_g/\mathbb{E}\tau > c$. Random turnover cannot be replaced directly by a fixed value of τ.

3. **Small-return second-order approximation**:

$$\mathbb{E}\ln(1+r_{\text{net}}) \approx \mathbb{E}r_{\text{net}} - \tfrac{1}{2}\mathbb{E}[r_{\text{net}}^2].$$

The second moment equals variance plus squared mean; the squared mean can be dropped only under a further small-mean/diffusion approximation. Correlation between turnover and gross return also affects the second moment of net return; it cannot unconditionally be replaced by σ_g^2.

4. **One-sided turnover convention**: With $\tau_{1w} = \tau/2$, the accounting fee is $2c\tau_{1w}$, so the necessary lower bound on the gross-return ratio becomes $\mathbb{E}r_g/\mathbb{E}\tau_{1w} > 2c$. A 10 bps fee therefore means 10 bps per unit of L1 turnover and 20 bps per unit of one-sided turnover; covering fees alone does not ensure positive log growth.

30-C03 Solution

Let $z_i(v) = |w_i v - x_i|$, and define

$$C_i(z) = c_1 z + c_2\sigma_i z\sqrt{z/\text{ADV}_i}, \qquad f(v) = v + \sum_i C_i(z_i(v)) - V_{\text{pre}}.$$

Here, z_i and ADV are notionals in the same currency, so z_i/ADV_i is dimensionless; σ_i is a dimensionless return volatility, c_1 is a linear fee rate, and c_2 is dimensionless. Therefore C_i has units of currency.

On $0 \le v \le V_{\text{pre}}$, $z_i(v) \le |w_i|V_{\text{pre}} + |x_i|$, and for $z > 0$:

$$C_i'(z) = c_1 + \frac{3}{2}c_2\sigma_i\sqrt{z/\text{ADV}_i} \le L_i := c_1 + \frac{3}{2}c_2\sigma_i\sqrt{(|w_i|V_{\text{pre}} + |x_i|)/\text{ADV}_i}.$$

On each smooth interval, $f'(v) \ge 1 - \sum_i L_i|w_i|$. Thus, if $c_1, c_2, \sigma_i \ge 0$, $\text{ADV}_i > 0$, and

$$\sum_i L_i|w_i| < 1,$$

then f is strictly increasing between kinks. By continuity, it remains strictly increasing across points where $w_i v = x_i$. Since costs are nonnegative, $f(V_{\text{pre}}) \ge 0$. If also $f(0) < 0$, the intermediate value theorem guarantees a root in the interval, and strict increase makes it the unique positive root.

If this sufficient condition fails, one cannot conclude that no solution exists, nor claim uniqueness. Consider the single-asset counterexample $w = 1$, $x = 50$, $V_{\text{pre}} = 49$, $c_1 = 0$, $\text{ADV} = 1$, and $c_2\sigma = 0.3$. The impact cost must retain the executed notional z, so

$$f(v) = v + 0.3|v - 50|^{3/2} - 49.$$

Numerically, $f(40) = 0.48683298 > 0$, $f(45) = -0.64589803 < 0$, and $f(49) = 0.3 > 0$. Thus there are at least two positive roots in $[0, 49]$; bracketing solvers find 41.29509609 and 48.38332833. Here $f(0) = 57.06601718 > 0$, and the derivative bound in the sufficient condition is $L|w| = 1.5(0.3)\sqrt{99} = 4.47744347 > 1$. The sufficient uniqueness condition and its endpoint sign-change premise therefore do not hold. This example directly refutes the claim that the cost function unconditionally makes the full equation monotone, but does not imply that a solution generally fails to exist.

Convexity and the correct starting point: Each C_i is nonnegative, nondecreasing, and convex for $z \geq 0$; $z_i(v)$ is convex, so f is convex as well. Suppose the interval contains a largest root $r \in (0, V_{\text{pre}}]$. Start at the upper endpoint, $v_0 = V_{\text{pre}}$, and stop if $f(v) = 0$. For $v > r$ with $f(v) > 0$, any valid subgradient d satisfies

$$0 = f(r) \geq f(v) + d(r - v), \qquad d \geq f(v)/(v - r) > 0.$$

Therefore $r \leq v - f(v)/d < v$: the iteration does not cross the largest root and decreases monotonically. On a finite convex interval containing the iterates, if the function is continuous and its subgradients are locally bounded, a limit that is not a root would imply a positive lower bound on the step size, a contradiction. The iteration therefore converges to the largest root; once it reaches a root, it must stop rather than divide by the derivative. This does not require the derivative to be positive over the entire real line. In the counterexample above, starting from 49 gives approximately 49, 48.45454545, 48.38436224, 48.38332855, and 48.38332833, converging to the larger financial root.

What is lost under nonlinear impact is the finite-step termination guarantee available for the piecewise-linear equation in Chapter 29. There is no unconditional guarantee for arbitrary starting values, zero or negative derivatives, no root, or iterates outside the interval above. When $c_1 > 0$, kinks require care; the derivative of the impact term $z^{3/2}$ tends to zero, rather than diverging, at zero. If a sign-changing bracket for the continuous function is found, bisection converges to a root within that bracket, but does not choose the economically relevant root for the researcher. Equal signs at the endpoints do not rule out two interior roots, and Newton failure does not prove that no root exists.

18.2 Reference implementations for programming exercises

30-P01 Solution

```
import numpy as np
import pandas as pd

def _perturb_future_inputs(market_data, cutoff_date, seed=42):
    data = {k: (v.copy(deep=True) if isinstance(v, pd.DataFrame) else v)
            for k, v in market_data.items()}
    dates = data["dates"]
    after = dates > cutoff_date
    rng = np.random.default_rng(seed)
    for key, scale in (("mark_prices", 0.15), ("exec_prices", 0.12)):
        shocks = np.exp(rng.normal(0.0, scale, size=data[key].loc[after].shape))
        data[key].loc[after] = data[key].loc[after] * shocks
    data["earnings_surprises"].loc[after] = rng.normal(
        0.0, 1.0, size=data["earnings_surprises"].loc[after].shape
    )
    data["splits"].loc[after] *= np.exp(
        rng.normal(0.0, 0.08, size=data["splits"].loc[after].shape)
    )
```

```
    data["dividends"].loc[after] *= 1.5
    if after.any():
        data["dividends"].loc[dates[after][0], data["dividends"].columns[0]] += 0.25
    data["delistings"] = {
        asset: (date, cash * (1.5 if date > cutoff_date else 1.0))
        for asset, (date, cash) in data["delistings"].items()
    }
    return data


def assert_future_perturbation_invariant(strategy_func, market_data, cutoff_date, tol
    ↪ =1e-11):
    # strategy_func returns (target_weights, BacktestResult).
    w0, r0 = strategy_func(market_data)
    w1, r1 = strategy_func(_perturb_future_inputs(market_data, cutoff_date))

    def max_frame_diff(a, b):
        if not a.index.equals(b.index) or list(a.columns) != list(b.columns):
            return float("inf")
        x = a.to_numpy(dtype=float)
        y = b.to_numpy(dtype=float)
        if not np.array_equal(np.isnan(x), np.isnan(y)):
            return float("inf")
        finite = np.isfinite(x) & np.isfinite(y)
        if not np.array_equal(np.isfinite(x), np.isfinite(y)):
            return float("inf")
        if not np.array_equal(x[~finite & ~np.isnan(x)], y[~finite & ~np.isnan(y)]):
            return float("inf")
        return float(np.max(np.abs(x[finite] - y[finite]))) if finite.any() else 0.0

    f0 = r0.fills.loc[r0.fills["trade_time"] <= cutoff_date].sort_values(
        ["trade_time", "signal_time", "asset"]
    ).reset_index(drop=True)
    f1 = r1.fills.loc[r1.fills["trade_time"] <= cutoff_date].sort_values(
        ["trade_time", "signal_time", "asset"]
    ).reset_index(drop=True)
    keys = ["signal_time", "trade_time", "asset"]
    if not f0[keys].equals(f1[keys]):
        raise AssertionError("pre-cutoff filled orders differ")
    diffs = {
        "target_weights": max_frame_diff(w0.loc[w0.index <= cutoff_date],
                                         w1.loc[w1.index <= cutoff_date]),
        "filled_orders": max_frame_diff(f0[["quantity", "price", "notional", "fee"]],
                                        f1[["quantity", "price", "notional", "fee"
    ↪ ]]),
        "positions": max_frame_diff(r0.positions.loc[:cutoff_date],
                                    r1.positions.loc[:cutoff_date]),
        "nav": max_frame_diff(r0.ledger.loc[:cutoff_date, ["nav"]],
                              r1.ledger.loc[:cutoff_date, ["nav"]]),
    }
    if max(diffs.values()) >= tol:
        raise AssertionError(f"Future perturbation changed pre-cutoff outputs: {diffs
    ↪ }")
    return diffs
```

```
from quant_course.backtest import BacktestConfig, run_backtest

dates = pd.bdate_range("2024-01-02", periods=5)
prices = pd.DataFrame({"A": [100.0, 100.0, 101.0, 99.0, 98.0]}, index=dates)
market_data = {
    "dates": dates,
    "mark_prices": prices.copy(),
    "exec_prices": prices.copy(),
    "earnings_surprises": pd.DataFrame(0.0, index=dates, columns=["A"]),
    "splits": pd.DataFrame(1.0, index=dates, columns=["A"]),
    "dividends": pd.DataFrame(0.0, index=dates, columns=["A"]),
    "delistings": {"A": (dates[4], 1.0)},
}

def strategy_func(data):
    weights = pd.DataFrame({"A": [1.0]}, index=[data["dates"][0]])
    result = run_backtest(
        data["mark_prices"], weights, BacktestConfig(initial_cash=1000.0, lag=1),
        exec_prices=data["exec_prices"], splits=data["splits"],
        dividends=data["dividends"], delistings=data["delistings"],
    )
    return weights, result

cutoff_date = dates[1]
print("P01 invariance check passed:", assert_future_perturbation_invariant(
    ↪ strategy_func, market_data, cutoff_date))

# Changes in missingness must fail even when the alternative value is zero.
```

30-P02 Solution

The metrics are ranking summaries; do not treat metric multiplied by $\sqrt{n}$ as a test statistic without justification. This problem additionally takes the full set of valid nominal p-values as input. If using an IID-normal approximation for annualized Sharpe, first state the frequency A, use $z \approx SR\sqrt{n/A}$, and specify the one- or two-sided tail probability; dependent returns require an appropriate standard error. The p-values below are supplied teaching inputs and are not claimed to be estimated from the metric.

```
import numpy as np
import pandas as pd
from quant_course.research_protocol import TrialLedger, bonferroni_adjust,
    ↪ holm_adjust

def evaluate_ledger_multiplicity(ledger, nominal_p, alpha=0.05):
    df = ledger.to_frame()
    if df.empty or not 0 < alpha < 1:
        raise ValueError("nonempty ledger and 0<alpha<1 required")
    ids = df["trial_id"].to_numpy(int)
    if not isinstance(nominal_p, pd.Series) or nominal_p.index.has_duplicates:
        raise ValueError("unique trial-id-indexed nominal p-values required")
    if set(nominal_p.index) != set(ids):
        raise ValueError("every attempted candidate needs one valid nominal p")
    p = nominal_p.reindex(ids).to_numpy(float)
```

```
    if not np.isfinite(p).all() or ((p < 0) | (p > 1)).any():
        raise ValueError("p-values must be finite in [0,1]")
    metrics = df["metric"].to_numpy(float)
    finite = metrics[np.isfinite(metrics)]
    bonf = bonferroni_adjust(p)
    holm = holm_adjust(p)
    table = pd.DataFrame({"trial_id": ids, "metric": metrics,
                          "nominal_p": p, "bonferroni_p": bonf, "holm_p": holm})
    ordered = table.sort_values(["nominal_p", "trial_id"]).reset_index(drop=True)
    ordered["holm_threshold"] = alpha / np.arange(len(ids), 0, -1)
    ordered["step_pass"] = ordered["nominal_p"] <= ordered["holm_threshold"]
    ordered["holm_step_reject"] = ordered["step_pass"].cummin()
    return {"n_trials": len(ids),
            "max_metric": float(finite.max()) if finite.size else float("nan"),
            "median_metric": float(np.median(finite)) if finite.size else float("nan"
    ↪ ),
            "table": table, "holm_sequence": ordered,
            "rejected_bonferroni": ids[bonf <= alpha].tolist(),
            "rejected_holm": ids[holm <= alpha].tolist()}

ledger = TrialLedger()
for k, metric in enumerate([0.5, -0.2, 0.3], 1):
    ledger.record({"candidate": k}, metric=metric, n_obs=100)
res = evaluate_ledger_multiplicity(ledger, pd.Series({3: 0.040, 1: 0.008, 2: 0.019}))
assert res["n_trials"] == 3
assert np.allclose(res["table"]["bonferroni_p"], [0.024, 0.057, 0.120], rtol=0, atol
    ↪ =1e-14)
assert np.allclose(res["table"]["holm_p"], [0.024, 0.038, 0.040], rtol=0, atol=1e-14)
assert res["rejected_bonferroni"] == [1]
assert res["rejected_holm"] == [1, 2, 3]
print(res["table"], res["holm_sequence"])
```

The Holm sorted thresholds are 0.05/3, 0.05/2, and 0.05; after any step fails, stop rejecting at later steps. Candidate IDs remain traceable, and duplicate, failed, and undefined metrics still count as trials. If a valid p-value cannot be supplied for an attempt, do not simply delete it to shrink the family. A prespecified conservative placeholder p-value of 1 may be used and disclosed, but this does not repair invalidity in the other p-values.

30-P03 Solution

This example demonstrates listing eligibility, a month-end signal with next-observation execution, and delisting cash. It does not present this three-name toy example as an actual P4 universe satisfying the 60-name minimum. Prices are unadjusted; there are no dividends, interest, or fees. Month-end dates are specified explicitly in the input.

```
import numpy as np
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest, reconcile

def generate_pit_weights(rebalance_dates, assets, listings, delistings):
    w = pd.DataFrame(0.0, index=rebalance_dates, columns=assets)
    for d in rebalance_dates:
        alive = [a for a in assets if listings[a] <= d and
                 (a not in delistings or d < delistings[a][0])]
        if alive:
            w.loc[d, alive] = 1.0 / len(alive)
    return w

dates = pd.to_datetime(["2026-01-30", "2026-02-02", "2026-02-27", "2026-03-02"])
assets = ["A", "B", "C"]
listings = {"A": dates[0], "B": dates[2], "C": dates[0]}
delistings = {"C": (dates[3], 2.0)}
prices = pd.DataFrame({"A": [10., 10., 12., 12.], "B": [np.nan, np.nan, 20., 20.],
                       "C": [10., 10., 9., np.nan]}, index=dates)
raw = generate_pit_weights(dates[[0, 2]], assets, listings, delistings)
assert np.allclose(raw.iloc[0], [0.5, 0., 0.5], rtol=0, atol=1e-14)
assert np.allclose(raw.iloc[1], [1/3, 1/3, 1/3], rtol=0, atol=1e-14)

# Execution-only copy: do not rewrite the historical signal or rescale survivors.
execution_copy = raw.copy()
for d in raw.index:
    execution_date = dates[dates.get_loc(d) + 1]
    for a, (exit_date, _) in delistings.items():
        if exit_date <= execution_date:
            execution_copy.loc[d, a] = 0.0
cfg = BacktestConfig(initial_cash=1000., lag=1)
try:
    run_backtest(prices, raw, cfg, delistings=delistings)
    raise AssertionError("base ledger must reject an already-delisted nonzero target"
    ↪ )
except ValueError as exc:
    assert "delist" in str(exc).lower()
result = run_backtest(prices, execution_copy, cfg, delistings=delistings)
rec = reconcile(result, prices, cfg, delistings=delistings)
assert result.ledger.loc[dates[3], "delist_cash"] == 100.0
assert result.positions.loc[dates[3], "C"] == 0.0
assert np.isclose(result.ledger.loc[dates[3], "nav"], 700., rtol=0, atol=1e-10)
assert np.isclose(result.ledger.loc[dates[3], "cash_end"], 700/3, rtol=0, atol=1e-10)
assert rec["residual"].abs().max() < 1e-10
print(raw, execution_copy, result.ledger, result.positions)
```

The initial execution buys 50 shares each of A and C. Before delisting, A is worth 600 and C is worth 450; C is liquidated for 100, realizing a loss of 350 and reducing NAV from 1,050 to 700. In the execution-only copy, C is set to zero and A and B remain at one-third each; they are not rescaled. Therefore, A and B each have USD 700/3 in holdings and USD 700/3 remains in cash. Listing information constrains new targets, while an existing position in a delisted stock is liquidated by the base ledger event. The base run_backtest does not automatically filter targets;

explicitly copying eligibility here is a teaching wrapper, not a claim that the signal already knew about a future delisting.

30-P04 Solution

```
import pandas as pd

def validate_price_basis(price_basis, prices, dividends, tol_ratio=0.5):
    if price_basis != "raw_unadjusted":
        raise ValueError("cash dividends require source-declared raw_unadjusted
    ↪ prices")
    if not prices.index.equals(dividends.index) or list(prices.columns) != list(
    ↪ dividends.columns):
        raise ValueError("prices and dividends must have matching dates and assets")
    diagnostics = []
    for col in prices.columns:
        for d, div_val in dividends[col][dividends[col] > 0].items():
            t_loc = prices.index.get_loc(d)
            if t_loc > 0:
                price_drop = prices[col].iloc[t_loc - 1] - prices[col].iloc[t_loc]
                if price_drop < div_val * tol_ratio:
                    diagnostics.append(
                        f"{col} {d}: price drop {price_drop:.2f} is below "
                        f"{tol_ratio:.2f} x dividend {div_val:.2f}; basis is not
    ↪ inferred"
                    )
    return diagnostics

dates = pd.bdate_range("2024-01-02", periods=2)
dividends = pd.DataFrame({"A": [0.0, 1.0]}, index=dates)

# Raw prices may show no drop if a market gain offsets the dividend.
P_raw_offset = pd.DataFrame({"A": [10.0, 10.0]}, index=dates)
warnings = validate_price_basis("raw_unadjusted", P_raw_offset, dividends)
assert len(warnings) == 1

# Adjusted prices may fall by more than the dividend because of a market loss.
P_adjusted_down = pd.DataFrame({"A": [10.0, 8.0]}, index=dates)
try:
    validate_price_basis("adjusted", P_adjusted_down, dividends)
    raise AssertionError("adjusted price basis was not rejected")
except ValueError as exc:
    assert "raw_unadjusted" in str(exc)

print("P04 basis metadata enforced; price-drop check is diagnostic only.")
```

18.3 Challenge-problem scoring criteria and reference arguments

30-C01 Full reference answer: restatements and ingestion

The original filing, v1, is released on March 1 and ingested on March 2: net income is 10, equity is 100, and ROE is 10%. Restatement v2 is released on May 1 and ingested on May 3: net income is 5, equity is 50, and ROE is still 10%. An April signal may read only v1; v2 cannot be used to recalculate the April signal after the fact. If the restated net income is updated while v1 equity is retained, the result is incorrectly $5/100 = 5\%$, while the ratio using fields from the same version is $5/50 = 10\%$.

Use the primary key `(security_id, fiscal_period, version_id)` and retain every version and source without overwriting prior rows. Record both the source-system release time `source_available_at` and the time the program actually receives the record, `ingested_at`. To simulate data actually delivered in production at the time, define availability as the later of the two. A scholarly PIT reconstruction based only on the source release time is a separate protocol and must not be described as reproducing actual receipt timing.

For each security, report period, and signal time s, first retain versions with $a_v \leq s$, then select the one with the greatest a_v among them; use all financial fields from that same version. If multiple versions have the same timestamp, break ties using a prespecified source sequence/version ordering rather than choosing after looking at results. A simulation of actual delivery on May 2 still uses v1; v2 becomes available only on or after May 3. As a test, arbitrarily alter restated values received only after May 3: April and May 2 signals must remain unchanged. If they change, the join or version fields have leaked future information.

30-C02 Full reference answer: dimension is not maximum-statistic calibration

Suppose there are $M = 3$ standardized trials with a correlation matrix whose diagonal entries are 1 and off-diagonal entries are $\rho = 0.5$. Its eigenvalues are $2, 0.5, 0.5$, which sum to 3 and have squared sum 4.5. The Spectral Participation Ratio is

$$d_{\text{eff}} = \frac{(\sum_i \lambda_i)^2}{\sum_i \lambda_i^2} = \frac{3^2}{2^2 + 0.5^2 + 0.5^2} = 2.$$

For a general correlation matrix, this is also $M^2/\text{tr}(\mathbf{\Sigma}_M^2)$. It equals M for the identity matrix and 1 when all trials point in exactly the same direction. This measures how many principal directions share the variance; it describes an effective dimension and cannot unconditionally replace the number of independent trials when calibrating a maximum Sharpe or DSR. Chapter 32 separately presents an approximation for trial correlations from the cited working paper and its limitations.

A clear counterexample is two trials, Z and $-Z$, where $Z \sim N(0, 1)$. The correlation matrix has eigenvalues 2 and 0, so the Spectral Participation Ratio is 1. But the maximum across the two trials is $|Z|$, whose expectation is $\sqrt{2/\pi} \approx 0.7979$, whereas the expected value from only

one standard-normal trial is 0. If one mechanically treats "dimension 1" as "one selection," the selection gain is misestimated. This counterexample holds even without heavy tails or regime shifts.

A continuous parameter search also requires a frozen search region, a complete trial ledger, the dependence structure, and a valid resampling scheme under the null. With time-dependent data, rows must not be assumed exchangeable by default. Gaussian random-field extreme-value methods require additional smoothness assumptions, covariance geometry, and sources; they are outside the methods validated in this chapter. When valid calibration cannot be established, retain all attempts, use conservative multiple-testing adjustment and causal holdout validation, and interpret results narrowly. Do not claim that a single number from a correlation matrix resolves search risk.

30-C03 Scoring rubric and reference argument

- **Full-credit criteria (10 points)**:
  1. State $z_i(v) = |w_i v - x_i|$, $C_i(z) = c_1 z + c_2 \sigma_i z \sqrt{z/\mathrm{ADV}_i}$, and $f(v) = v + \sum_i C_i(z_i(v)) - V_{\mathrm{pre}}$, and correctly explain the units of each term (3 points).
  2. Give the sufficient condition $\sum_i L_i |w_i| < 1$ on $[0, V_{\mathrm{pre}}]$, using the definition of L_i above; with $f(0) < 0$ and $f(V_{\mathrm{pre}}) \geq 0$, prove that there is a unique positive root in the interval (3 points).
  3. Explain that piecewise-linear absolute-value costs can have kinks and that the derivative on smooth intervals can be zero or negative; arbitrary-start Newton iteration therefore has no general guarantee. But if a convex function has a root on the interval, iteration from the upper endpoint can converge monotonically to the largest root, without a finite-step guarantee under nonlinear costs. Bisection requires a sign-changing bracket; failure of the sufficient condition does not mean no root exists, and multiple roots may occur (4 points).

---

19. Further Reading

For readers who want to study frontier and classic scholarship on backtest bias, the following list provides a categorized reading guide. **Disclosure**: The analyses in the chapter body are based entirely on local derivations and observations. The external works below were not opened and read directly during this session and are marked **not reviewed**.

- **Must Know (classic works, not reviewed)**:
  - *White, H. (2000).* "A Reality Check for Data Snooping". *Econometrica*, 68(5), 1097–1126.
    - Suggested reading: focus on Section 2, which constructs the bootstrap reality check. Understand how resampling correlated return series controls family-wise error and addresses the limitation of assuming independence.
  - *Brown, S. J., Goetzmann, W., Ibbotson, R. G., & Ross, S. A. (1992).* "Survivorship Bias in Performance Studies". *The Review of Financial Studies*, 5(4), 553–580.

    - Suggested reading: focus on Section 3's derivation of fund survival and truncated distributions; compare it with the inverse Mills ratio in Section 6.2 of this chapter.
- **Should Know (advanced essentials, not reviewed)**:
  - *Lo, A. W., & MacKinlay, A. C. (1990).* "Data-Snooping Biases in Tests of Financial Asset Pricing Models". *The Review of Financial Studies*, 3(3), 431–467.
    - Suggested reading: read the philosophical and statistical discussion of "data that have already been tested by previous researchers are no longer unbiased."
  - *Harvey, C. R., Liu, Y., & Zhu, H. (2016).* "… and the Cross-Section of Expected Returns". *The Review of Financial Studies*, 29(1), 5–68.
    - Suggested reading: read the analysis of multiple-testing corrections for the Factor Zoo and why contemporary quantitative research requires new-factor t-statistic thresholds to rise above 3.0.
- **Advanced (frontier extensions, not reviewed)**:
  - *Bailey, D. H., Borwein, J. M., López de Prado, M., & Zhu, Q. J. (2015).* "The Probability of Backtest Overfitting". Working-paper version dated 2015-02-27; Chapter 32 provides a reading of this version and a record of the local source.
    - Suggested reading: Chapter 32 formally derives its CSCV (Combinatorially Symmetric Cross-Validation) and PBO (Probability of Backtest Overfitting) algorithms.

---

20. References

The following list identifies sources cited in this chapter and their locally verified hash records:

- `FRENCH_10_INDUSTRIES`: Kenneth R. French 10 Industry Portfolios (Monthly, 1926-07 to 2026-08).
  - Manifest file: `data/manifests/french_industries_2026-10-05.json` (SHA-256: `3b71cd5f781f9790cc380195c22635e03e9c9e27a5f43cd2d0d46fb12efa8271`)
  - Industry data file: `data/processed/french_10_industries_monthly.csv` (SHA-256: `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`)
  - Source URL: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/10_Industry_Portfolios_CSV.zip
- `FRENCH_FACTORS_RF`: Kenneth R. French Risk-Free Rate (Monthly, 1926-07 to 2026-08).
  - Manifest file: `data/manifests/french_factors_2026-10-05.json` (SHA-256: `597c9703afa623a63b0d5fbf36dc941aac90d00948aeca302f151ed3c854000b`)
  - Factor data file: `data/processed/french_factors_3_monthly.csv` (SHA-256: `c19770a19a0ac47ad1c29074f57234570aeaa9725212cb4fa8146a2846c52814`)
  - Source URL: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/F-F_Research_Data_Factors_CSV.zip

- `QUANT_COURSE_BACKTEST`: Chapter 29 Frozen Backtest Ledger (`src/quant_course/backtest.py`).
  - Function interfaces: `run_backtest`, `BacktestConfig`, `reconcile`, `solve_post_fee_nav`.
- `QUANT_COURSE_RESEARCH_PROTOCOL`: Chapter 22 Research Methodology (`src/quant_course/research_protocol.py`).
  - Classes and functions: `TrialLedger`, `expected_max_standard_normal`, `expected_max_standard_normal_exact`, `bonferroni_adjust`, `holm_adjust`.
- `QUANT_COURSE_CH30_CLI`: Chapter 30 Implementation CLI (`code/ch30_backtest_biases.py`).
  - Current result output: `qa/a6/codex_resume/ch30/results_v2/ch30_results.json`.

English translation of the chapter Notebook and assembly, portable paths, and cold-start verification of the shared reader package remain assigned to BIL23. English chapter prose does not establish completion of the full English software package.

Chapter 31 — Financial Model Validation

Main thread: trace the information used by every label and feature through time. Purging removes overlapping label intervals; embargo removes specified post-test feature look-backs; a causal walk-forward protocol evaluates predictions using information available at deployment time. These controls address their declared channels and do not by themselves establish that an entire research pipeline is free of leakage. Prerequisites: Chapters 7, 16, 22, 29, and 30. You will derive half-open interval overlap, calculate removal and retention sets, diagnose row-count gaps in panels, implement purged splits and nested selection, and interpret constructed and historical industry-return comparisons.

1. Chapter Objectives

A subtle source of apparent generalization is the interaction between a forward-return label and a validation boundary. When the target is a five-day return, training features may all precede the test block while their labels still include returns realized inside it. A chronological split of feature rows alone does not resolve that problem.

After this chapter you should be able to:

1. Derive the overlap condition for $I_i = (t_i, e_i]$,

$$\max(t_i, t_j) < \min(e_i, e_j),$$

and explain why touching endpoints do not overlap. Calculate the purge set for horizon h, establish the single-asset equivalence to `gap=h-1` on a regular observation calendar, and derive the residual $(h-1)(N-1)$ overlapping rows in the specified balanced panel.
2. Distinguish the economic roles and directions of purge and embargo. Purge checks label intersections on both sides of a test block; embargo checks later training features whose historical support reaches back into the test span. Distinguish offline purged K-fold stability diagnostics from deployment-time walk-forward evaluation.
3. Use `src/quant_course/validation.py` to construct single-asset and panel splits, rolling and expanding windows, and nested selection. Fit the scaler and each candidate model only on the current inner training fold; select hyperparameters using inner validation scores. Refit on the current outer training set after selection, and use the outer test set only for evaluation.
4. Check invalid intervals, touching endpoints, irregular calendars, multiple assets per date, and labels whose availability a_i is later than settlement e_i. Interpret numerical differences under the actual constructed or French-industry protocol.

Completion means calculating the 15-observation example by hand and reconciling it with the code:

$$R_{\text{purge}} = \{2, 3, 4, 5, 9, 10, 11, 12\},$$

$$R_{\text{embargo}} = \{13, 14\}, \qquad S_{\text{final}} = \{0, 1\}.$$

Then follow the default I3 reader route in §9 and complete the questions and answers. The stored five-seed and historical French results are inherited evidence. This revision does not rerun the full CLI, Notebook, runner, or historical studies. Correct splits alone do not validate every input, execution, or feature channel.

2. Why Quants Need It

Validation connects feature engineering to portfolio construction, the Chapter 29 execution ledger, and Chapter 32 performance inference:

```
Data/features --> Validation and information-aware splits --> Portfolio targets
  (Ch 3,23–28)                  (Ch 31)                        (Ch 20–21)
    --> Ledger reconciliation (Ch 29) --> Performance inference (Ch 32)
```

Under genuinely IID sampling, a declared target distribution and training size, and no continued model selection using outer results, random K-fold can estimate the predictive risk of the corresponding training procedure. That does not make it an unconditional unbiased estimate of future deployment risk. Financial data present several additional issues:

- **Overlapping labels.** In the additive approximation $y_t^{\text{add}} = \sum_{k=1}^{5} r_{t+k}$, adjacent labels share four realized returns. The compounded definition appears in §4.1. Both definitions require tracking the interval $(t, t+h]$; random row allocation may place the same future realized return in both training and test labels.
- **Feature look-back leakage.** Moving averages, momentum, and smoothing use historical observations. When a K-fold split trains on dates after a historical test block, those later training features can reach back into that test period.
- **Global preprocessing.** Fitting `StandardScaler` or PCA before splitting exposes the training procedure to test means, variances, or other distributional information.
- **Rows versus dates.** `TimeSeriesSplit(gap=...)` removes rows. With many securities on each date, four removed rows need not cover even one complete trading date.

This chapter formalizes the timing diagnosis in Chapter 30. The resulting out-of-sample candidate sequences feed the multiple-testing methods in Chapter 32, including PBO and CSCV, subject to those methods' own assumptions.

3. Intuition

3.1 Why the Last Training Labels Already Contain Test Returns

Suppose features are formed at each daily close and predict the next five trading days. Assign observations $t = 0, 1, 2, 3, 4$ to training and begin testing at $t = 5$. The feature rows are chronological, but examine their labels:

- The observation at close 4 predicts the return from close 4 to close 9.
- The first test observation at close 5 predicts the return from close 5 to close 10; the next predicts close 6 to close 11.
- More precisely, the training label at 4 contains $r_5, \ldots, r_9$, while the test label at 5 contains $r_6, \ldots, r_{10}$. They share four periods, $r_6, \ldots, r_9$. At close 5, r_5 has already been realized, while the training label remains unsettled until 9. Calling all five returns $r_5, \ldots, r_9$ future test returns would be inaccurate.

Treat each sample as an information interval, rather than a point on a row index.

3.2 The Frozen 15-Observation Example, Step by Step

Use observation starts $t \in \{0, 1, \ldots, 14\}$ on a regular trading calendar and horizon $h = 5$. Each feature is available at its observation close. Its label covers

$$I_i = (t_i, t_i + 5],$$

with daily returns $\{r_{t_i+1}, \ldots, r_{t_i+5}\}$. Choose test starts $Q = \{6, 7, 8\}$ and all other starts as candidate training observations:

$$C = \{0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 14\}.$$

```
Test start 6: (6,11] contains returns 7,8,9,10,11.
Test start 7: (7,12] contains returns 8,9,10,11,12.
Test start 8: (8,13] contains returns 9,10,11,12,13.
The test-label span runs from start 6 to latest end 13.
```

First inspect the candidates before the test block:

- At 0, compare $(0, 5]$ with $(6, 11]$: $\max(0, 6) = 6$ is not less than $\min(5, 11) = 5$. Retain 0.
- At 1, $(1, 6]$ contains $r_2, \ldots, r_6$, while $(6, 11]$ contains $r_7, \ldots, r_{11}$. They touch at the close-6 base price but share no return increment. The overlap test gives $6 < 6$, which is false. Retain 1.
- At 2, $\max(2, 6) = 6 < \min(7, 11) = 7$. Return r_7 is shared. Remove 2.
- At 3, $6 < \min(8, 11) = 8$. Remove 3.
- At 4, $6 < \min(9, 11) = 9$. Remove 4.
- At 5, $6 < \min(10, 11) = 10$. Remove 5.

Thus the pre-test removal set is $\{2, 3, 4, 5\}$ and its survivors are $\{0, 1\}$.

Next inspect the candidates after the test block, comparing with the final test label $(8, 13]$:

- At 9, $(9, 14]$ overlaps because $9 < 13$. Remove 9.
- At 10, $(10, 15]$ overlaps because $10 < 13$. Remove 10.
- At 11, $(11, 16]$ overlaps because $11 < 13$. Remove 11.
- At 12, $(12, 17]$ overlaps because $12 < 13$. Remove 12.
- At 13, $(13, 18]$ only touches $(8, 13]$: $13 < 13$ is false. Retain 13 at this stage.
- At 14, $(14, 19]$ does not overlap because $14 < 13$ is false. Retain 14 at this stage.

After purge, the retained set is $\{0, 1, 13, 14\}$.

Finally impose the declared feature embargo. Here the feature uses three past return periods, involving four prices: at 13 it uses prices on $[10, 13]$, and at 14 on $[11, 14]$. This is not a moving average of only three price points. Both supports reach into the test-label span ending at 13. With $m = 3$, the post-test embargo band is

$$[e_{\max}, e_{\max} + m) = [13, 16).$$

Starts 13 and 14 lie inside that band and are removed. The final training set is $\{0, 1\}$. Purge and embargo are explicit geometric filters with different inputs.

3.3 Reconcile the Removal and Retention Ledgers

The complete one-way filtering chain is

$$\begin{aligned} Q &= \{6, 7, 8\}, \\ C &= \{0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 14\}, \\ R_{\text{purge}} &= \{2, 3, 4, 5, 9, 10, 11, 12\}, \\ S_{\text{after_purge}} &= \{0, 1, 13, 14\}, \\ R_{\text{embargo}} &= \{13, 14\}, \\ S_{\text{final}} &= \{0, 1\}. \end{aligned}$$

There are $12 - 8 = 4$ survivors after purge and $4 - 2 = 2$ after embargo. Including future starts 13 and 14 makes this an offline purged-K-fold-style diagnostic, not a deployment training set available at time 6. Label ends remain $i + 5$, so the last label ends at 19. Fifteen observation starts do not mean that all label returns end at 14; an actual dataset must supply those later returns or remove unsettled labels.

Two touching cases illustrate the distinction. $(1, 6]$ ends with r_6 while $(6, 11]$ starts with r_7. Likewise, $(13, 18]$ starts with r_{14} after $(8, 13]$ ends with r_{13}. Both 1 and 13 pass purge, but 13 fails the feature embargo. Touching answers only the label-overlap question.

If a prediction is made at time 6 and label 1 settles at $e_1 = 6$ but is released at $a_1 = 7$, availability removes it first. Settlement does not automatically make a label readable.

4. Mathematical Foundations

4.1 Observation Coordinates and Trading Calendars

Let $\mathcal{T} = \{t_0, t_1, \ldots, t_{T-1}\}$ be strictly increasing trading dates. Observation i has feature cutoff t_i, and its features may use only information in $\mathcal{F}_{t_i}$. Its label ends at $e_i > t_i$ and has support $I_i = (t_i, e_i]$.

For a common date sequence $\tau_0 < \cdots < \tau_K$, write $t_i = \tau_{p_i}$ and $e_i = \tau_{q_i}$. Defining $r_k = P_{\tau_k}/P_{\tau_{k-1}} - 1$ gives the compounded simple-return label

$$y_i = \frac{P_{\tau_{q_i}}}{P_{\tau_{p_i}}} - 1 = \prod_{k=p_i+1}^{q_i} (1 + r_k) - 1.$$

Writing the horizon as $e_i - t_i$ is appropriate only after encoding the endpoints as common observation positions. Adding h directly to irregular dates is not valid. Map asset-specific settlement dates to a common ordered coordinate first.

4.2 Interval Algebra

For nondegenerate half-open intervals $I_1 = (t_1, e_1]$ and $I_2 = (t_2, e_2]$,

$$I_1 \cap I_2 = (\max(t_1, t_2), \min(e_1, e_2)],$$

where the intersection is empty if its lower endpoint is at least its upper endpoint. Hence

$$I_1 \cap I_2 \neq \emptyset \iff \max(t_1, t_2) < \min(e_1, e_2).$$

When $e_1 = t_2$, both bounds are e_1 and $(e_1, e_1] = \emptyset$. Endpoint touching does not overlap.

5. Formal Definitions

Observation time t_i. The feature cutoff, normally the daily close in this chapter.

Label end time e_i. The end of the outcome interval. Require $e_i > t_i$; the implementation raises `ValueError` for $e_i \leq t_i$.

Availability time a_i. The earliest timestamp at which the settled or released label is recorded and readable. Liquid-market closing labels may have $a_i = e_i$, while fund NAVs, international assets, and financial-statement measures can have $a_i > e_i$. A deployment training label must satisfy $a_i \leq t_{\text{predict}}$. The current `validation.py` interface accepts only t_i, e_i, not an independent a_i. The caller must filter candidates using actual availability before each fit; delayed a_i cannot be replaced by e_i.

Candidate set C. Construct it for the intended evaluation. Deployment walk-forward takes past observations with labels available by prediction time. Offline purged K-fold may use future training blocks whose labels are available by the later evaluation time; it cannot claim that those blocks were available for the historical prediction. Let Q be the test set, with $C \cap Q = \emptyset$.

Purging. Define the removal set and then its complement:

$$R_{\text{purge}} = \{i \in C : \exists j \in Q,\ \max(t_i, t_j) < \min(e_i, e_j)\},$$

$$S_{\text{after_purge}} = C \setminus R_{\text{purge}}.$$

One overlap with any test label is sufficient for removal. No overlap, including mere touching, passes this filter. Here R means removed and S means survivors. The panel policy conservatively checks time globally rather than exempting overlapping labels because their asset identifiers differ.

Embargo. Inspect only post-test candidates that have already passed purge:

$$R_{\text{embargo}} = \{i \in S_{\text{after_purge}} : t_i \geq t_{\text{test,max}}, \\ e_{\text{test,max}} \leq t_i < e_{\text{test,max}} + m\},$$

$$S_{\text{final}} = S_{\text{after_purge}} \setminus R_{\text{embargo}}.$$

Here $t_{\text{test,max}} = \max_{j \in Q} t_j$ and $e_{\text{test,max}} = \max_{j \in Q} e_j$. The buffer $m \geq 0$ has the same units as the time coordinate; trading-observation periods are not automatically calendar days. This chapter chooses a buffer covering the declared maximum feature look-back. The left endpoint is included, the right excluded, and $m = 0$ removes nothing. The current API converts t, e to floats. It does not parse dates, reconcile asset calendars, or enforce a. Encode a common ordered coordinate and express m in that unit. A finite buffer covers the stated finite support; it does not eliminate the infinite memory of an untruncated EMA.

The filtering invariant is

$$S_{\text{final}} \subseteq S_{\text{after_purge}} \subseteq C.$$

A removed sample never reappears during embargo or later concatenation. A past-only walk-forward has no post-test candidates and thus $R_{\text{embargo}} = \emptyset$. Under strict deployment conditions

$$a_i \geq e_i, \qquad a_i \leq t_{\text{fit}} \leq \min_{j \in Q} t_j,$$

we already have $e_i \leq \min t_j$, excluding overlap with future test returns; an explicit interval check remains a useful implementation safeguard. Offline future-block diagnostics can still require two-sided purging. Conversely, nonoverlap does not establish availability: a touching historical label may be released late.

TimeSeriesSplit gap g. The number of sample rows omitted between the training block and test block, not a number of dates.

Purged walk-forward. A forward-moving design with historical training candidates and a subsequent test block; check the trailing training labels for overlap.

Purged K-fold. Split time into K contiguous, unshuffled blocks. For each test block, use the other past and future blocks as candidates and apply purge and embargo.

Nested validation. The outer split evaluates out-of-sample performance. Inner splits are confined to the current outer training set and select hyperparameters or features. Fit preprocessing means and variances only on the current training fold.

6. Derivations

6.1 Proof of the Necessary and Sufficient Overlap Condition

Theorem 1. For $t_1 < e_1$ and $t_2 < e_2$,

$$I_1 \cap I_2 \neq \emptyset \iff \max(t_1, t_2) < \min(e_1, e_2).$$

Proof. Membership in the intersection requires both $t_1 < x \leq e_1$ and $t_2 < x \leq e_2$. Combining the inequalities gives

$$\max(t_1, t_2) < x \leq \min(e_1, e_2).$$

Such a real x exists exactly when the lower bound is strictly less than the upper bound. If so, every point in the displayed half-open interval belongs to both intervals. If the lower bound is at least the upper bound, no point satisfies both inequalities and the intersection is empty. This proves both directions.

6.2 Purge Counts and Gap Equivalence for One Regular Asset

Assume fixed integer horizon $h \geq 2$ and regular observation positions, with $e_i = t_i + h$. For a contiguous test block starting at t_{test} and containing n_{test} starts, the union of its label intervals is

$$\bigcup_{j \in \mathcal{S}_{\text{test}}} (t_j, e_j] = (t_{\text{test}}, t_{\text{test}} + n_{\text{test}} - 1 + h].$$

For a prior candidate $t_i < t_{\text{test}}$, Theorem 1 reduces overlap with the first test label to

$$\max(t_i, t_{\text{test}}) < \min(t_i + h, t_{\text{test}} + h) \iff t_{\text{test}} < t_i + h \iff t_i > t_{\text{test}} - h.$$

The integer starts to remove are therefore

$$\{t_{\text{test}} - h + 1, \ldots, t_{\text{test}} - 1\},$$

whose count is

$$(t_{\text{test}} - 1) - (t_{\text{test}} - h + 1) + 1 = h - 1.$$

This count presumes those preceding observations exist. `TimeSeriesSplit` uses the exclusive slice end

$$\text{train_end} = \text{test_start} - \text{gap}.$$

For one regular asset, `gap=h-1` sets that end to $t_{\text{test}} - h + 1$ and removes exactly the same trailing observations. This equivalence relies on the stated calendar, horizon, and one-row-per-date conditions.

6.3 Why the Same Row Gap Fails in a Panel

Theorem 2. Suppose a complete balanced panel has exactly $N \geq 2$ assets per date, sorted by `(date, asset)`, fixed integer horizon $h \geq 2$, a test block beginning at a complete-date boundary, and at least $h-1$ preceding consecutive dates. The test date T_{test} begins at row $R_{\text{test}} = T_{\text{test}}N$. Using the row gap $h-1$ leaves exactly

$$N_{\text{leak}} = (h-1)(N-1)$$

overlapping training rows.

Proof. Every asset on dates $T_{\text{test}} - h + 1, \ldots, T_{\text{test}} - 1$ overlaps the test-label span. Thus the required removal count is

$$N_{\text{true_purge}} = (h-1)N.$$

The row-based algorithm instead removes only

$$\{R_{\text{test}} - (h-1), \ldots, R_{\text{test}} - 1\},$$

a total of $h-1$ rows. Those rows lie within the true overlap band. They cover at most $\lfloor (h-1)/N \rfloor$ complete dates and part of one further date, so the remaining count is

$$N_{\text{leak}} = (h-1)N - (h-1) = (h-1)(N-1).$$

For the frozen $N = 3, h = 5$ example, dates 2–5 contain 12 overlapping rows. A four-row gap removes all three assets on date 5 and the third asset on date 4. Eight rows remain: three on date 2, three on date 3, and two on date 4. This proves the result. Missing assets or dates, variable horizons, insufficient history, or a boundary inside a date require direct interval calculations instead of this closed form.

---

7. Financial Interpretation

7.1 Why Do Financial Features and Labels Have Long Memory?

In classical statistics, residuals are often assumed to be independent and identically distributed white noise. Financial asset prices, however, are jointly driven by fundamentals, macroeconomic liquidity, and strategic interactions among investors.

If compounding period-by-period simple returns is approximated by summing small returns, an h-period label can be written as a rolling sum, $y_t^{\text{add}} = \sum_{k=1}^{h} r_{t+k}$. When the returns r_t have equal variance and are pairwise uncorrelated, as under a homoskedastic martingale difference process, the covariance at lag $0 \leq \ell < h$ is:

$$\text{Cov}(y_t^{\text{add}}, y_{t+\ell}^{\text{add}}) = (h-\ell)\sigma_r^2$$

This is an exact result for additive labels; the covariance is zero when $\ell \geq h$. It is not a general exact covariance formula for the compounded simple-return label defined in §4.1.

For the compounded simple-return label in §4.1, $Y_t = \prod_{k=1}^{h}(1+r_{t+k})-1$, an exact expression follows only under the additional assumption that simple returns are independent and identically distributed over time. Let $G = 1 + r$, $m_1 = \mathbb{E}[G]$, and $m_2 = \mathbb{E}[G^2]$. Then, for $0 \le \ell < h$:

$$\mathrm{Cov}(Y_t, Y_{t+\ell}) = m_2^{h-\ell} m_1^{2\ell} - m_1^{2h}$$

If $\mathbb{E}[r] = 0$ and $\mathrm{Var}(r) = \sigma_r^2$, this becomes $(1 + \sigma_r^2)^{h-\ell} - 1$, which is approximated by $(h - \ell)\sigma_r^2$ only when the variance is small. Overlapping intervals therefore induce statistical dependence between labels, but its strength depends on the return distribution, holding period, and lag. Interval overlap alone does not imply that labels are necessarily highly correlated. The Purge logic still applies: it prevents the same realized return from entering both a training label and a test label.

The effect of label overlap on model evaluation depends on the data-generating process, model, feature structure, and test window. In this chapter's Ridge construction, the average score difference before and after purging is small across five seeds; this result alone does not characterize low-capacity models in general. A separate 1-NN construction illustrates how an unpurged boundary sample can provide a path for memorizing information shared with a test label. The size of that effect is not determined by model capacity alone and must be examined under the specific data and split.

7.2 How to Choose Between Purged K-Fold and Walk-Forward

Quantitative teams often debate whether model validation should use strictly forward Walk-Forward or whether Purged K-Fold, which trains on both sides of a test block, is acceptable. These approaches answer different scientific questions:

1. **Purged Walk-Forward answers the deployment question.**
   - It follows the one-way causal direction of time. At every prediction point, the model can use only past history.
   - It can reflect how regime shifts, structural breaks, and distribution drift affect a strategy. Deployment evaluation should use a forward protocol whose inputs were available at the time. Passing the split does not establish that the real inputs, trading costs, or other channels are permissible.
2. **Purged K-Fold answers the cross-regime parameter-stability question.**
   - After removing overlapping labels with Purge and preventing feature lookback from crossing the test block with Embargo, it allows future data to train a model evaluated on a historical interval.
   - It asks: "If an economic relationship holds in 2024, are its parameters stable in 2018?" By using more of the sample, it can test whether the model overfits a particular historical period. However, its test score cannot replace a deployment test along the real-time axis.

8. Numerical Example

This section fully reproduces and works through the frozen example from §3.2, then checks each step against the standardized results exported by the CLI.

Worked Example: 15 Label Starts and a 45-Row Panel

8.1 Example Inputs

- **Sample sequence:** Daily observations for one asset, indexed by $t_i = i$, where $i \in \{0, 1, \ldots, 14\}$, for 15 trading days.
- **Forward label horizon:** $h = 5$ trading days. The label interval is $I_i = (t_i, t_i + 5]$.
- **Test block:** $\mathcal{S}_{\text{test}} = \{6, 7, 8\}$.
- **Candidate training set:** $\mathcal{S}_{\text{train}} = \{0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 14\}$.
- **Feature lookback window:** $m = 3$ trading days.

8.2 Step-by-Step Calculation and Reconciliation Table

The table lists the interval endpoints for every candidate sample, whether it overlaps the test block, and the resulting decision:

Sample i	Observation t_i	Label end e_i	Half-open interval I_i	Overlap test against the test block
0	0	5	$(0, 5]$	$\max(0, 6) = 6 \not< \min(5, 11) = 5$
1	1	6	$(1, 6]$	$\max(1, 6) = 6 \not< \min(6, 11) = 6$
2	2	7	$(2, 7]$	$\max(2, 6) = 6 < \min(7, 11) = 7$
3	3	8	$(3, 8]$	$\max(3, 6) = 6 < \min(8, 11) = 8$
4	4	9	$(4, 9]$	$\max(4, 6) = 6 < \min(9, 11) = 9$
5	5	10	$(5, 10]$	$\max(5, 6) = 6 < \min(10, 11) = 10$
9	9	14	$(9, 14]$	$\max(9, 8) = 9 < \min(14, 13) = 13$
10	10	15	$(10, 15]$	$\max(10, 8) = 10 < \min(15, 13) = 13$
11	11	16	$(11, 16]$	$\max(11, 8) = 11 < \min(16, 13) = 13$
12	12	17	$(12, 17]$	$\max(12, 8) = 12 < \min(17, 13) = 13$

Continued: the same complete example

13	13	18	$(13, 18]$	$\max(13, 8) = 13 \not< \min(18, 13) = 13$
14	14	19	$(14, 19]$	$\max(14, 8) = 14 \not< 13$

Sample i	Status	Final disposition
0	No intersection	Retain
1	Endpoint touch, not overlap	Retain
2	Overlap	Purge
3	Overlap	Purge
4	Overlap	Purge
5	Overlap	Purge
9	Overlap	Purge
10	Overlap	Purge
11	Overlap	Purge
12	Overlap	Purge
13	Endpoint touch; retained by Purge	Remove by Embargo
14	No intersection; retained by Purge	Remove by Embargo

- **Purge-stage reconciliation:**
  - Samples before the test block removed: $\{2, 3, 4, 5\}$ (key: `hand_example.removed_by_purge_pre_test`).
  - Samples after the test block removed: $\{9, 10, 11, 12\}$ (key: `hand_example.removed_by_purge_post_test`).
  - Training samples retained by Purge: $\{0, 1, 13, 14\}$ (key: `hand_example.purged_train_indices`).
- **Embargo-stage reconciliation:**
  - Latest label end in the test block: $e_{\max} = 13$.
  - Embargo interval: $[e_{\max}, e_{\max} + m) = [13, 16)$.
  - The observations $t_i \in \{13, 14\}$ for post-test samples $\{13, 14\}$ both fall in $[13, 16)$, so those samples are removed (key: `hand_example.removed_by_embargo`).
  - Final retained training set: $\{0, 1\}$ (key: `hand_example.final_train_indices`).

8.3 Hand Check: Why gap Fails on a Cross-Sectional Panel

Suppose three assets are observed over 15 trading days, giving 45 rows arranged strictly by (date, asset):

- Rows $0, 1, 2$ are date 0; rows $3, 4, 5$ are date 1; ...; rows $18, \ldots, 26$ are dates 6, 7, and 8, the test set (nine rows).
- A correct causal Purge removes all samples on dates 2, 3, 4, and 5, or rows $6, \ldots, 17$ (12 rows). It retains only the six rows for dates 0 and 1, rows $0, \ldots, 5$ (key: `hand_example.panel_purged_train_indices`).
- If TimeSeriesSplit(gap=4) is called incorrectly, the test set starts at row 18 and the training boundary is $18 - 4 = 14$. The algorithm retains rows $0, \ldots, 13$, or 14 rows.

Continued: the same complete example

- The overlapping rows left in training are:
  - Date 2: rows 6, 7, and 8 (three rows).
  - Date 3: rows 9, 10, and 11 (three rows).
  - Date 4: rows 12 and 13 (two rows).
  - The total is exactly $3 + 3 + 2 = 8$ rows (key: `hand_example.panel_tss_overlapping_count`). Their indices are $[6, 7, 8, 9, 10, 11, 12, 13]$ (key: `hand_example.panel_tss_overlapping_rows`).

---

9. Python Implementation

By default, walk_forward_splits uses purge_overlap=True and requires both observation times t and label end times e, each with the same number of entries as the samples. If either is missing, it raises an error rather than silently falling back to an ordinary time split. For a comparison experiment, Purge can be explicitly disabled with purge_overlap=False, but time order alone does not guarantee that labels have settled. For example, with n=15, train_size=6, test_size=3 and e=t+5, the first fold has candidate training samples 0–5 and test samples 6–8; after purging, only training samples 0 and 1 remain. Omitting the intervals would leave overlapping labels for training samples 2–5.

The chapter's core algorithm is in src/quant_course/validation.py. From the root of the accepted I3 reader package, the default command is:

```
python run_notebook.py notebooks/ch31_model_validation.ipynb
```

The default Notebook computes a new single-seed run for the four schemes on 500 dates, 1-NN, purged K-Fold, and nested construction; it does not merely read the five-seed summary. The French branch is skipped by default. The Notebook creates a unique `runs/ch31_model_validation_<uuid>/` directory and writes figures there. The runner separately saves an execution copy and execution.json under `runs/execution_<stem>_<uuid>/`; failed runs are retained as well. Neither was executed for this revision. Full assembly and cold-start verification of the English Notebook and shared runner remain for BIL23. The following code is an entry point for reading the implementation and for a small independent example. It does not direct the reader to overwrite historical CLI defaults.

9.1 Interval-Overlap Test and Basic Purge / Embargo Functions

The following code shows how to implement vectorized overlap checks and filtering efficiently in NumPy, without loops:

```
import numpy as np
from quant_course.validation import check_label_intervals, intervals_overlap, purge,
    ↪ embargo

# Construct the hand-worked example with 15 observations
t = np.arange(15, dtype=float)
e = t + 5.0
test_idx = np.array([6, 7, 8])
cand_train = np.array([0, 1, 2, 3, 4, 5, 9, 10, 11, 12, 13, 14])

# 1. Apply Purge
purged_train = purge(cand_train, test_idx, t, e)
assert np.array_equal(purged_train, [0, 1, 13, 14])

# 2. Assert that endpoint touching is not overlap: sample 1 (1, 6] and test sample 6
    ↪ (6, 11]
assert not intervals_overlap(t[1], e[1], t[6], e[6])

# 3. Apply Embargo (m = 3)
final_train = embargo(purged_train, test_idx, t, e, m=3)
assert np.array_equal(final_train, [0, 1])
print("Hand-worked Python Purge assertions passed; final training set:", final_train.
    ↪ tolist())
```

9.2 Causal Walk-Forward and Purged K-Fold Generators

```
import numpy as np
from quant_course.validation import walk_forward_splits, purged_kfold_splits

# 1. Expanding causal Walk-Forward
n_samples = 30
t_series = np.arange(n_samples, dtype=float)
e_series = t_series + 3.0  # 3-day forward return

splits_wf = walk_forward_splits(
    n_samples=30,
    train_size=10,
    test_size=5,
    step=5,
    expanding=True,
    t=t_series,
    e=e_series,
    purge_overlap=True,
)
print(f"Number of Walk-Forward folds: {len(splits_wf)}")

# First fold: test [10..14], candidate training [0..9], overlapping samples 8 and 9,
    ↪ retain [0..7]
```

```
train0, test0 = splits_wf[0]
assert np.array_equal(test0, np.arange(10, 15))
assert np.array_equal(train0, np.arange(8))

# 2. Date-grouped Purged K-Fold for panel data
dates_panel = np.repeat(np.arange(20), 2)  # 20 dates, 2 assets per date, 40 samples
    ↪ total
e_panel = dates_panel + 2.0
splits_pkf = purged_kfold_splits(
    n_samples=40,
    n_splits=4,
    t=dates_panel,
    e=e_panel,
    embargo_m=1,
    groups_date=dates_panel,
)
print(f"Number of Purged K-Fold folds: {len(splits_pkf)}")
train_f0, test_f0 = splits_pkf[0]
print(f"Rows in test fold 0: {len(test_f0)}, rows in retained training set: {len(
    ↪ train_f0)}")
```

9.3 Nested Hyperparameter Selection with Purge in Both Outer and Inner Folds

The example explicitly uses a forward-array convention: period_returns[i] is the synthetic return realized after feature time i, over $(i, i+1]$. It corresponds to the formal notation r_{i+1}, not to r_i realized at time i. Thus y[i] sums the returns over $(i, i+1]$ and $(i+1, i+2]$, and settles at $e_i = i+2$. X_all[i+1] is used only to generate the second synthetic future return; it does not enter the features available at time i. This example uses additive labels to demonstrate overlap purging and does not claim that they equal two-period compounded simple returns.

```
import numpy as np
from sklearn.linear_model import Ridge
from quant_course.validation import nested_hyperparameter_evaluation

rng = np.random.RandomState(42)
N = 120
X_all = rng.randn(N + 1, 2)
X = X_all[:-1]

# period_returns[i] is realized over (i, i+1], driven by X_all[i] known at i.
period_returns = 0.5 * X_all[:, 0] + rng.randn(N + 1) * 0.1

# Two-period forward additive-return approximation.
y = period_returns[:-1] + period_returns[1:]
t_obs = np.arange(N, dtype=float)
e_obs = t_obs + 2.0
a_obs = e_obs.copy()  # This example assumes labels are available when their
    ↪ intervals end.

candidate_outer_splits = [
    (np.arange(60), np.arange(60, 90)),
    (np.arange(90), np.arange(90, 120)),
```

```
]
outer_splits = []
for train_idx, test_idx in candidate_outer_splits:
    prediction_time = np.min(t_obs[test_idx])
    eligible_train = train_idx[a_obs[train_idx] <= prediction_time]
    outer_splits.append((eligible_train, test_idx))

def inner_split_fn(out_tr, t_arr, e_arr):
    n = len(out_tr)
    folds = []
    for start, stop in [(n // 3, 2 * n // 3), (2 * n // 3, n)]:
        in_train = np.arange(start)
        in_val = np.arange(start, stop)
        prediction_time = np.min(t_arr[out_tr[in_val]])
        # The caller filters availability; the helper applies interval Purge.
        in_train = in_train[a_obs[out_tr[in_train]] <= prediction_time]
        folds.append((in_train, in_val))
    return folds

param_grid = [{"alpha": 0.01}, {"alpha": 1.0}, {"alpha": 100.0}]

def model_factory(p):
    return Ridge(alpha=p["alpha"])

def scorer(m, X_te, y_te):
    return -float(np.mean((m.predict(X_te) - y_te) ** 2))

nested_res = nested_hyperparameter_evaluation(
    X, y, t_obs, e_obs, outer_splits, inner_split_fn, param_grid, model_factory,
    ↪ scorer, scale_features=True
)
print("Nested selection outer-fold test scores:", nested_res["outer_scores"])
print("Best parameter selected in each outer fold:", nested_res["best_params_per_fold
    ↪ "])
print("Outer train/test indices actually used:", nested_res["outer_splits_used"])
print("Inner train/validation indices actually used:", nested_res["inner_splits_used"
    ↪ ])
```

The helper applies label-interval Purge separately to the outer training candidates and each inner training set, and records the global indices actually used. In the pure forward example, the caller first filters labels by $a_i \leq t_{\text{predict}}$; here, $a_i = e_i$ is set explicitly. If publication delays mean that actual $a_i > e_i$, the real a_i values must be used for the same pre-filtering. A Purged K-Fold diagnostic may include training samples from after the test block; those samples are not part of the information set available at deployment.

The chapter CLI's synthetic study on 20 assets reported the following results, independent of the short structural example above (key: nested_selection):

- Honest outer-test mean MSE: 0.014231 (key: `nested_selection.mean_nested_outer_mse`).
- Test-set-tuned, or "cheating," MSE: 0.014195 (key: `nested_selection.mean_naive_cheating_mse`).
- Difference: 0.000036 (about 0.25%).

Why the difference is small: In this synthetic study, the candidate hyperparameter grid contained five α values, the data-generating process was primarily linear, and Ridge regularization was relatively strong. The small gap therefore describes these test folds only. It is specific to this model, search grid, seed, and sample split. Tuning on the test set makes that set part of model selection, so its score is no longer an independent evaluation. The actual bias depends on the search, data, and validation design; this example does not imply a fixed increase.

---

10. Visualization

The chapter's code/ch31_model_validation.py generated four high-resolution figures in figures/a6/, each corresponding to the derivations and numerical experiments in the text.

10.1 Label Intervals for the Worked Example

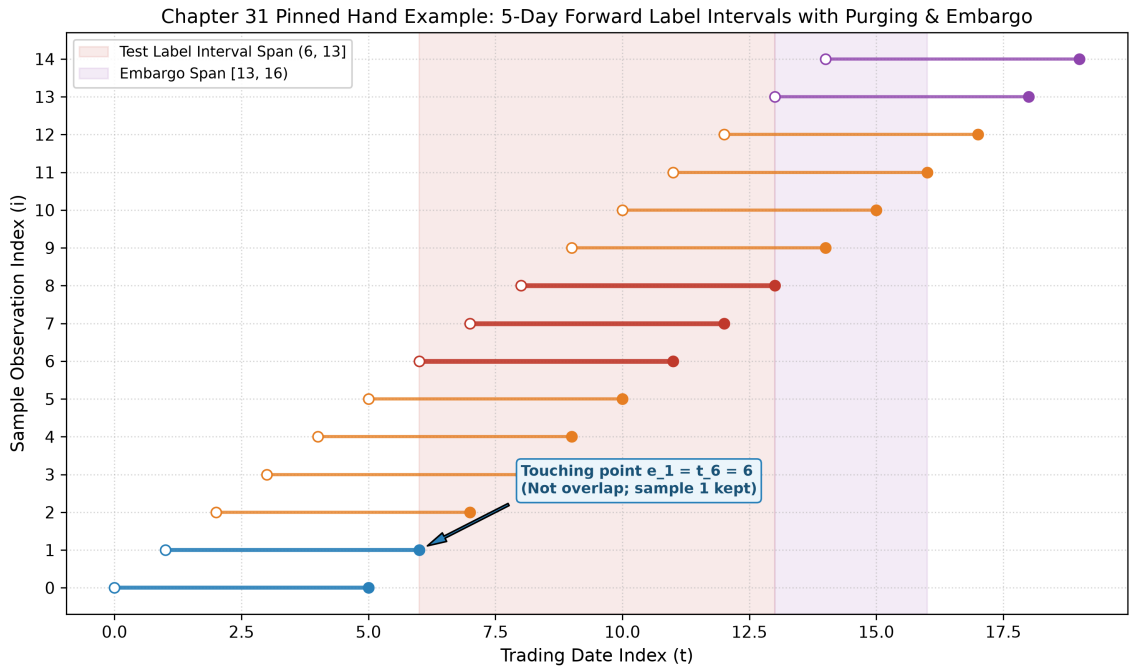


Figure 1: Chapter 31 Hand Label Intervals

- **File path:** figures/a6/ch31_hand_label_intervals.png.

- **Axes and units:** The horizontal axis is the shared observation position t (a discrete integer extending through the final label endpoint, 19; 15 label starts do not mean the labels end at 14). The vertical axis is the sample observation index i (integer, 0 to 14).
- **Samples and marks:** All 15 half-open return intervals $(t, t+5]$ are shown. Left endpoints are hollow circles and right endpoints are filled circles. Test-block samples $\{6, 7, 8\}$ are thick dark-red lines; samples $\{2, 3, 4, 5, 9, 10, 11, 12\}$ removed by Purge for overlapping labels are orange; samples $\{13, 14\}$ removed by Embargo for feature lookback penetration are purple; and the final clean training samples $\{0, 1\}$ are thick dark-blue lines.
- **What to look for:**
  1. The filled right endpoint of sample 1's label interval $(1, 6]$ coincides exactly at $t = 6$ with the hollow left endpoint of test sample 6's interval $(6, 11]$. An arrow labels this "Endpoint touch, not overlap (touching point $e_1 = t_6 = 6$)," showing why sample 1 is safely retained.
  2. The label interval for post-test sample $t = 13$ does not overlap the test block, but it falls inside the purple embargo span $[13, 16)$, showing how Embargo protects against feature lookback.

10.2 Leakage Ablation Comparison and Breakdown

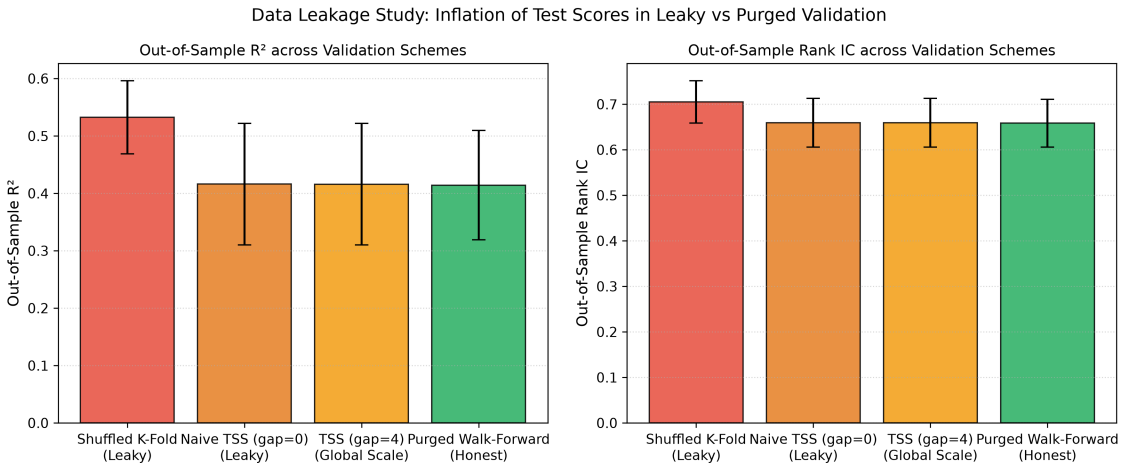


Figure 2: Chapter 31 Leakage Comparison

- **File path:** figures/a6/ch31_leakage_comparison.png.
- **Axes and units:** The left panel shows out-of-sample coefficient of determination R^2 (dimensionless); the right panel shows out-of-sample rank correlation, Rank IC (dimensionless). The horizontal axis compares four validation schemes. Error bars show the standard deviation across five random seeds ([20261031, 20261032, 20261033, 20261034, 20261035]).

The table reports the observed five-seed mean values (key: `leakage_experiment.schemes`):

Validation scheme	Out-of-sample R^2 (mean ± SD)	Out-of-sample Rank IC (mean ± SD)	Out-of-sample MSE	Leakage and isolation properties
1. Shuffled K-Fold	0.5327 ± 0.0638	0.7054 ± 0.0465	0.01177	Time shuffled; globally normalized features
2. Naive TSS (gap=0)	0.4160 ± 0.1059	0.6593 ± 0.0537	0.01286	Not shuffled; no Purge; globally normalized features
3. TSS (gap=4)	0.4160 ± 0.1060	0.6594 ± 0.0537	0.01286	Not shuffled; four rows removed; global normalization
4. Purged Walk-Forward	0.4143 ± 0.0952	0.6583 ± 0.0526	0.01290	Strictly time-causal; Purge applied; training-set normalization

- **What the comparison shows:**
  1. **Observed differences between protocols:** The saved mean R^2 difference between Shuffled K-Fold and Purged Walk-Forward is 0.1184 (the Rank IC difference is 0.0471). The gap between the first two schemes is 0.1167, while the gap between the latter two protocols is about 0.0017. These are not matched ablations: Shuffled K-Fold and TSS use different training dates and sample sizes, while TSS and Purged Walk-Forward also differ in date grouping, Purge, and scaling range. The value 0.1167/0.1184, or about 98.6%, cannot be called a causal share attributable to shuffling; nor can 0.0017 be attributed to Purge alone. Identifying the contribution of any one change requires matching the training samples, test samples, preprocessing, and other conditions.
  2. **Boundary effects in the current linear-model comparison:** In this Ridge construction with 20 assets, 500 days, and five seeds, the four-day label overlap at the boundaries of five large test blocks (80 days each) accounts for less than 2% of the training samples. The observed scores before and after purging are close. This describes the current design only; it does not show that low-capacity models are generally unaffected by leakage or that leakage effects occur only in high-capacity models or particular features.
  3. **Unpurged inflation with a high-capacity memorization model (1-NN):** To illustrate how label overlap can affect a high-capacity memorization model, a 1-nearest-neighbor (1-NN) model was evaluated in daily forward steps on a 30-asset panel (key: overlap_memorization_experiment).
     - The unpurged 1-NN model had a mean out-of-sample Rank IC of 0.1194 (the five seeds were 0.0977, 0.0942, 0.1691, 0.1109, and 0.1249).
     - The causal Purged result had a mean Rank IC of 0.0108 (the five seeds were 0.0007, −0.0189, 0.0784, 0.0061, and −0.0121). These are observations under this data-generating process and a finite set of seeds, not a guarantee about true population predictive power.

– In this construction, the unpurged model can match a test asset to its own unpurged sample from the previous day; the two labels share 80% of their returns. The five-seed mean Rank IC difference is 0.1086 (key: `overlap_memorization_experiment.rank_ic_inflation`). This shows how some highly memorizing models may exploit label overlap. Together with the Ridge result, it does not establish a general monotonic relationship between model capacity and leakage bias.

10.3 Comparison of Row-Count gap with Interval Purge

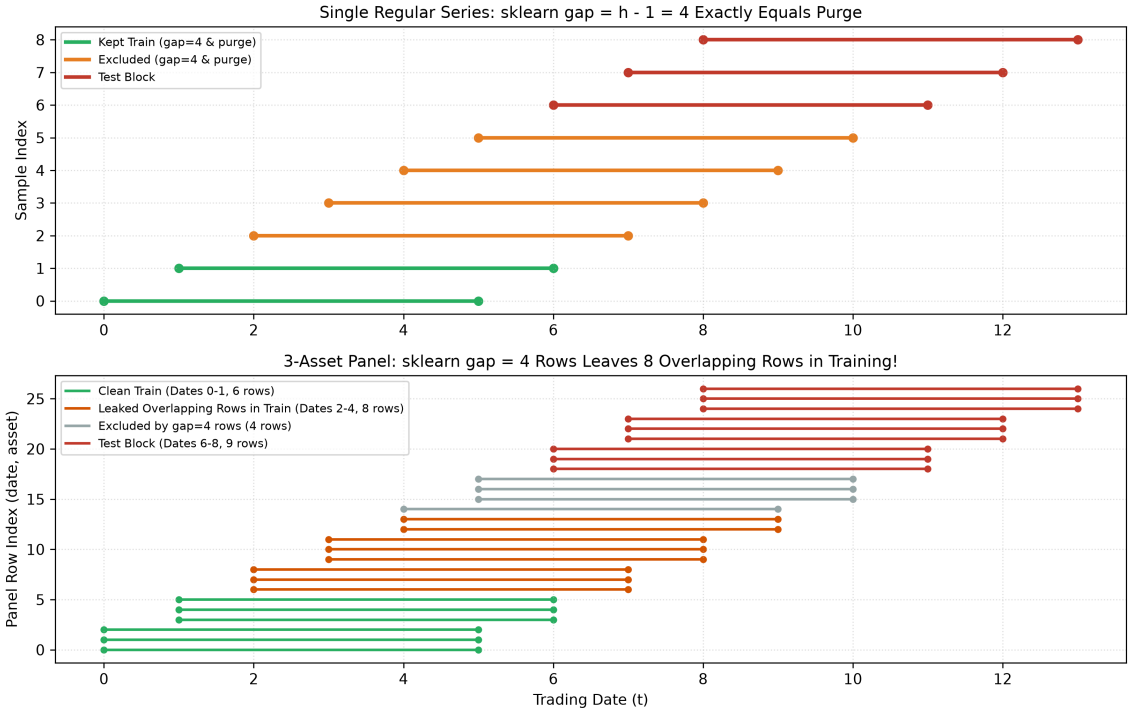


Figure 3: Chapter 31 Gap vs Purge Panel

- **File path:** figures/a6/ch31_gap_vs_purge_panel.png.
- **Axes and units:** The upper panel shows a single-asset sequence (sample indices 0–8); the lower panel shows a three-asset panel (row indices 0–26, ordered by (date, asset)). Both horizontal axes show trading-day index t.
- **What to look for:**
  1. In the single-asset sequence, gap=4 removes samples 2, 3, 4, and 5, exactly matching Purge row by row.
  2. The panel-data plot shows the substantial residual overlap: gap=4 removes only rows 14–17 (gray), while rows 6–13 (dark orange, covering eight rows across dates 2, 3, and 4) remain exposed in the training set, visually confirming Theorem 2.

10.4 Mean-Summary Plot for Rolling Validation on Real Monthly Industry Returns

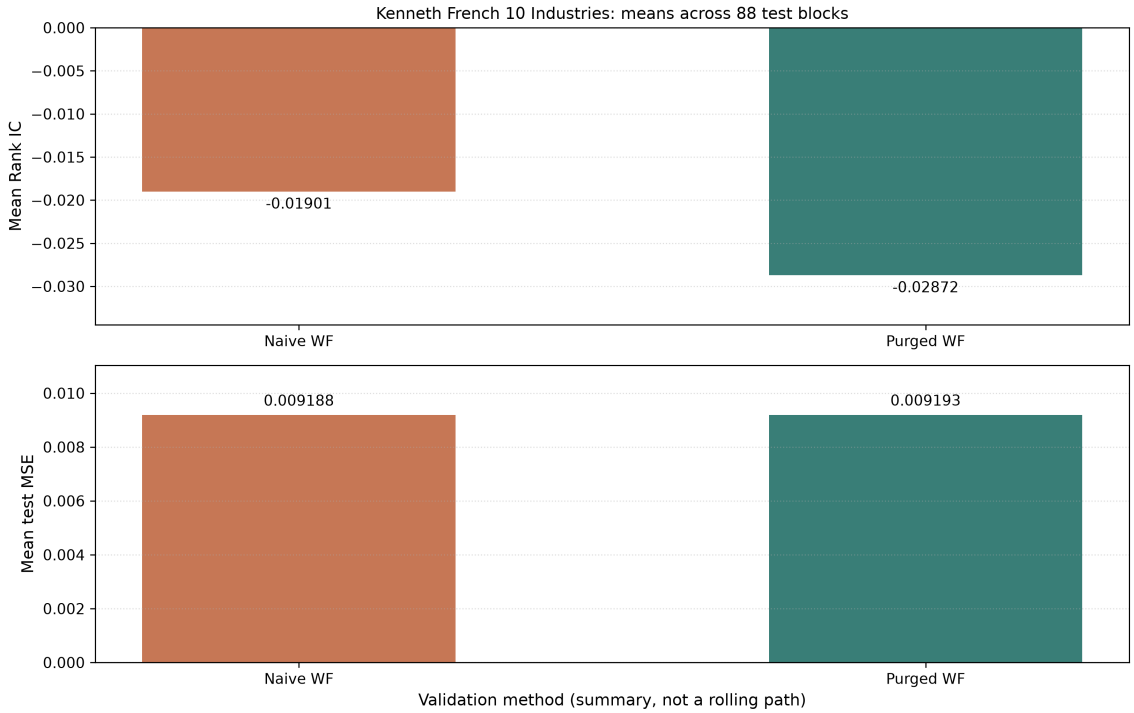


Figure 4: Chapter 31 French Rolling Validation

- **File path:** figures/a6/ch31_french_rolling_validation.png.
- **Axes and units:** The horizontal axis compares naive and purged validation. The upper panel shows mean Rank IC across 88 test blocks; the lower panel shows mean test MSE. Bar heights show only the saved summary means. They do not show the block-level paths, stability across blocks, or statistical significance of the method difference.
- **What to look for:**
  1. With three-month forward overlapping labels, mean test MSE is 0.009188 for naive Walk-Forward and 0.009193 for Purged Walk-Forward.
  2. On the real academic aggregate industry data, the observed mean Rank IC is −0.01901 and −0.02872, respectively. The experiment reports no uncertainty for these means and no significance test, so the negative means do not establish that positive Alpha is absent in the population. The figure reports only the observed results for this dataset, model, and validation window.

---

11. Real-Market Example

This section preserves a historical methodological illustration; it is not a newly acquired dataset or rerun. Raw, extracted, and processed French files are absent from the default I3 package, whose Notebook skips this branch. Rights, subjects, and intended uses need separate review. A10-A/A10-A.1 do not replace G05/A10-B execution authorization.

To illustrate model validation on real-market assets, we use the frozen Kenneth R. French 10 Industry Portfolios monthly total-return data in the workspace, covering 1926-07 through 2026-08 (1,202 monthly observations).

11.1 Data Properties and Scope

- **Sources:** Kenneth R. French Data Library: 10 Industry Portfolios (value-weighted, monthly) and the FF3 risk-free rate (RF).
- **Files and hash verification:**
  - Data manifest: data/manifests/french_industries_2026-10-05.json (SHA-256: `3b71cd5f781f9790cc380195c22635e03e9c9e27a5f43cd2d0d46fb12efa8271`).
  - Processed return table: data/processed/french_10_industries_monthly.csv (SHA-256: `3b097d810c295d64eedfa1ece7067ff2a0707f5ca90e33935df9949251f70950`).
- **Academic-use and tradability boundaries:** These data are academic aggregate portfolios, have been revised historically, and are not point-in-time snapshots (is_point_in_time: false). They also do not represent the tradable liquidity of individual securities. This example therefore supports methodological instruction only and does not make a live-trading Alpha claim.

11.2 Design: Three-Month Overlapping Forward Returns and Rolling Validation

- **Prediction target:** Three-month compounded forward return ($h = 3$):

$$y_{i,t} = \prod_{k=1}^{3}(1 + R_{i,t+k}) - 1$$

- **Feature engineering:**
  - 12-month trailing momentum: $X_{0,i,t} = \prod_{k=0}^{11}(1 + R_{i,t-k}) - 1$.
  - One-month short-term reversal: $X_{1,i,t} = R_{i,t}$.
- **Rolling Walk-Forward windows:**
  - Fixed training window: 120 months (10 years).
  - Test block: 12 months (1 year).
  - Step size: 12 months (annual tuning).
  - The full sample produces 88 non-overlapping 12-month calendar test blocks. Forward labels can still overlap across blocks, so the blocks are not necessarily statistically independent (key: `french_market_experiment.eval_blocks`).
- **Protocol comparison:**
  1. **Naive Walk-Forward:** The training set consists of the 120 months immediately before each test block, with no Purge. The return intervals for the final two training labels, y_{t-1} and y_{t-2}, extend into the first two months of the test block.
  2. **Purged Walk-Forward:** Purge removes the two months at the end of the training set whose return intervals overlap the test set.

11.3 Observed Results and Reconciliation

The table reports CLI results preserved in an earlier independent review; they were not rerun for this revision (`build/codex_a6/final_learning_review/ch31/recheck/results/ch31_results.json`):

Validation scheme	Mean out-of-sample MSE	Mean out-of-sample Rank IC	Evaluation blocks	Lookahead status
Naive Walk-Forward (unpurged)	0.009188	−0.01901	88	Return labels from the final two training months overlap
Purged Walk-Forward	0.009193	−0.02872	88	Overlapping labels are removed

Interpretation:

1. With a 120-month training sample and only two months of overlap (1.67% of the training window), the out-of-sample MSE values are very close: 0.009188 versus 0.009193.
2. The mean out-of-sample Rank IC is slightly negative for both schemes in this split (−0.01901 and −0.02872). The table provides no standard errors, confidence intervals, or test. It supports reporting these sample means only; it does not establish that the population relationship is negative, statistically insignificant, or devoid of Alpha. These academic aggregate portfolios also do not support a tradability conclusion.

---

12. Common Mistakes

12.1 Mistake 1: Using Randomly Shuffled K-Fold as the Default for Financial Time Series

- **Incorrect example and why it fails:** This ignores time order and the label-dependence window. After shuffling, adjacent samples can appear in both the training and test sets. When their label windows share information, the test score may be optimistic; the magnitude depends on the data-generating process, features, model, and split. The code below is a fixed synthetic counterexample and does not establish a general bias magnitude.

- **Runnable comparison code:**

```
import numpy as np
from sklearn.linear_model import Ridge
from sklearn.model_selection import KFold
from quant_course.validation import walk_forward_splits

rng = np.random.RandomState(42)
N, h = 100, 5
X_all = rng.randn(N + h - 1, 2)
X = X_all[:N]

# period_returns[k] belongs to (k,k+1], so y[i] settles at i+h.
period_returns = 0.6 * X_all[:, 0] + rng.normal(0, 0.2, len(X_all))
y = np.array([period_returns[i:i+h].sum() for i in range(N)])
t = np.arange(N, dtype=float)
e = t + h
a = e.copy()
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores_leaky = [Ridge().fit(X[tr], y[tr]).score(X[te], y[te]) for tr, te in kf.split(
    ↪ X)]
splits = walk_forward_splits(N, 50, 10, step=10, t=t, e=e)
scores_forward = []
for tr, te in splits:
    tr = tr[a[tr] <= t[te].min()]
    assert np.all(e[tr] <= t[te].min())
    scores_forward.append(Ridge().fit(X[tr], y[tr]).score(X[te], y[te]))
assert np.isfinite(scores_leaky).all() and np.isfinite(scores_forward).all()
print("Shuffled / forward:", np.mean(scores_leaky), np.mean(scores_forward))
```

The old centered convolution, convolve(X, ones(5)/5, mode="same"), depends on $X_{t-2}, \ldots, X_{t+2}$ at interior points. Labeling it as $(t, t+5]$ does not describe its true support, and matching the interval length to five does not fix the metadata. This example instead uses an explicit forward array: each label i sums five synthetic returns over $(i, i+1], \ldots, (i+4, i+5]$, with $e_i = a_i = i+5$. Future X values are used only to generate future returns, not as features available at the prediction time. This is a new small teaching example, not the saved five-seed study below, and it does not assert that shuffled scores must be higher on every sample.

- **Observed score differences:** In the five-seed synthetic panel study, the four schemes had the following results:
  1. Shuffled K-Fold: mean $R^2 = 0.5327$ (Rank IC = 0.7054).
  2. Naive TSS (gap=0): $R^2 = 0.4160$ (Rank IC = 0.6593).
  3. TSS (gap=4, global normalization): $R^2 = 0.4160$ (Rank IC = 0.6594).
  4. Causal Purged Walk-Forward: $R^2 = 0.4143$ (Rank IC = 0.6583). In this linear Ridge construction, the R^2 difference between Shuffled K-Fold and Purged Walk-Forward is 0.1184. The observed difference between the first two protocols is 0.1167, and the difference between TSS and the final protocol is about 0.0017. Because the training/test blocks, samples, and preprocessing all differ, these values do not support a one-factor causal decomposition. In a separate 1-NN construction, the mean Rank IC was 0.1194 without Purge and 0.0108 with Purge (a difference of 0.1086). These are simulation results under their respective designs, not general laws about model capacity, leakage, or bias magnitude.

12.2 Mistake 2: Treating TimeSeriesSplit's gap as a Number of Days in a Multi-Asset Panel

- **Incorrect example:** On a daily panel of 20 stocks, a researcher predicts five-day forward returns and assumes that TimeSeriesSplit(gap=4) removes four days.
- **Why it fails:** TimeSeriesSplit's gap parameter counts **sample rows**, not dates. In a 20-stock panel, gap=4 removes only four rows, less than one trading day's records; more than three days of overlapping data remain leaked.
- **Fix:** Use quant_course.validation.purge to filter explicitly by the trading-date array, or derive indices from unique dates before calling TimeSeriesSplit.
- **Numerical difference:** In the three-asset example, gap=4 left eight overlapping rows in the training set.

12.3 Mistake 3: Normalizing Features on the Full Sample Before Splitting

- **Why it fails:** Future test-set means and extreme values are incorporated into the full feature matrix, so the training features use a baseline containing information about the test period's structure.
- **Runnable comparison code:**

```
import numpy as np
from sklearn.preprocessing import StandardScaler

rng = np.random.RandomState(42)
X = rng.randn(100, 2) + 2.0

# Incorrect: normalize on the full sample
X_scaled_all = StandardScaler().fit_transform(X)
X_test_leaky = X_scaled_all[80:]

# Correct: fit on the training set only
scaler = StandardScaler()
```

```
X_train_clean = scaler.fit_transform(X[:80])
X_test_clean = scaler.transform(X[80:])

# Check that the two transformations differ
assert not np.allclose(X_test_leaky, X_test_clean)
```

- **Observed score difference:** Fitting the scaler globally brings test-period distribution information into the training feature baseline. In this small example, the two transformations differ, but the direction of any estimated volatility or mean-drift error is not guaranteed for every sample.

12.4 Mistake 4: Applying Embargo to Training Samples Before the Test Block

- **Why it fails:** This misunderstands the causal role of Embargo. Embargo guards against backward-looking features reaching into the test labels and is therefore necessary only for training samples located after the test block. Under this chapter's post-test definition, a purely historical forward split has no post-test candidates, so the embargo-removal set is empty. Historical dependency gaps or feature warm-up periods for other purposes are separate constraints; this point does not rule them out.
- **Fix:** In code, require is_after_test = t_i >= t_test_max and apply the interval filter $[e_{\max}, e_{\max} + m)$ only to samples satisfying that condition.

12.5 Mistake 5: Treating Endpoint Touching as Overlap and Removing Too Many Samples

- **Incorrect interpretation:** A researcher assumes there is overlap when one sample settles at the close on day 6 and the test prediction starts after the close on day 6.
- **Economic meaning:** The earlier label's return ends at r_6, while the later label begins at r_7, so the half-open intervals have an empty intersection. Sharing a reference closing price does not mean the price is non-random, nor does it imply that the two returns are statistically independent. The actual order of close settlement, data ingestion, and prediction must still be checked against the event clock; endpoint contact alone does not establish that those events were ordered correctly.

---

13. Quant Research Application

13.1 End-to-End Validation Pipeline for Quantitative Research

In industry quantitative research, an immutable, leakage-resistant model-validation process is a core safeguard in strategy development. A standard pipeline has four stages:

```
Stage 1: Freeze the protocol and plan (Ch 22 ResearchPlan)
    - Specify the primary metric (Rank IC), test horizon h, maximum feature lookback
    ↪ m, and asset universe.
Stage 2: Define sample intervals and construct labels (Ch 3, 31)
    - Record each sample's observation time t_i, label end e_i, and availability time
    ↪  a_i.
    - At each prediction time, first filter for a_i <= t_predict; the current helper
    ↪ performs interval Purge using (t_i, e_i) only.
Stage 3: Run the outer causal Walk-Forward (Outer Purged Walk-Forward)
    - Set a historical training window; check a-eligibility and then label overlap.
    ↪ In a purely historical forward split, the post-test Embargo set is empty.
Stage 4: Select hyperparameters in nested inner folds (Inner Nested Selection)
    - Generate subfolds only within the current outer training set; fit normalization
    ↪  only on each inner training set.
```

13.2 Practical Falsification Rules and Negative Criteria

After building the validation pipeline, a quantitative team should run the following falsification checks:

1. **Future Invariance Test:** With random seeds, identifiers and column schema, and algorithm policy held fixed, apply specified perturbations to valid future inputs and compare model parameters and hyperparameters at shared historical coordinates. Differences beyond the declared tolerance warrant investigation for leakage or nondeterminism. Passing this finite set of tests does not establish validity under every future change or the legality of all input timestamps.
2. **Shuffling Ablation Check:** Compare Purged Walk-Forward with shuffled K-Fold. Similar or reversed scores alone do not establish a lack of temporal dependence or model underfitting; a large difference alone cannot be attributed uniquely to label memorization. First match the evaluation target, training/test dates and sample sizes, and preprocessing. Then investigate distribution and feature support, while preserving results from every protocol.

14. Chapter Summary

14.1 Conceptual Flow

```
First choose the purpose: historical deployment with available labels, or offline
    ↪ diagnosis with future blocks.
Candidate C -> Does its label overlap?
  Yes -> R_purge: remove the sample and terminate this branch.
  No -> S_after_purge -> Is it post-test and inside the buffer?
    Yes -> R_embargo: remove the sample and terminate this branch.
    No -> S_final -> Scale and select separately within each inner and outer fold.
```

Neither removal node has an outgoing edge. Only $S_{\text{after_purge}}$ proceeds to the Embargo check; there is no path back from either removal set to the final set. Choose the deployment or diagnostic branch before generating candidates. Filtering cannot turn a future training block into a deployment split. Fit the scaler and tune within the training folds after filtering, applying the interval rules separately in the inner and outer folds.

For a numerical walk-through of the diagram, sample 2 overlaps and enters R_{purge}, where its branch ends. Sample 1 touches an endpoint, passes through the no-overlap branch to $S_{\text{after_purge}}$, and enters S_{final} because it is before the test block. Sample 13 also passes Purge because it only touches an endpoint, but its observation time falls in $[13, 16)$, so it enters R_{embargo} and its branch ends. Calling both removed and surviving sets "purged" would hide the direction that matters most.

Validation objective and question	Recommended scheme	Purge policy	Embargo policy	Feature-normalization fit range	Main limitation or warning

14.2 Decision Rules for Validation Schemes

Researchers should choose a validation architecture according to the research question:

Validation objective and question	Recommended scheme	Purge policy	Embargo policy	Feature-normalization fit range	Main limitation or warning
Simulate real trading deployment	Expanding or rolling Purged Walk-Forward	Strictly remove the final $h-1$ periods of the training set	Not needed; all training data precede the test block	Restricted to the current outer training fold	Sample sizes vary over time; early folds have fewer samples
Test stability across macroeconomic regimes	Purged K-Fold with contiguous time blocks	Remove overlapping samples on both sides	Apply an m-period embargo only to post-test training blocks	Restricted to the current outer training fold	Includes future training on past data and cannot replace a deployment test over time
Search multiple hyperparameters and select features	Nested Walk-Forward validation	Purge outer and inner folds independently	Apply according to the position of each test block	Fit independently on each outer and inner training set	Computation grows substantially; keep the parameter grid manageable
Validate a multi-asset cross-sectional panel	Date-grouped Purged Panel Split	Purge globally by date across assets	Embargo globally by date across assets	Daily cross-sectional normalization or global fit on training data	Do not use row-count gap; filter by trading-date index

15. Exercises

31-E01: Testing for Overlap Between Half-Open Label Intervals

For one asset's daily data, three samples have the following time parameters:

- Sample A: $t_A = 2,\ e_A = 6$.
- Sample B: $t_B = 5,\ e_B = 9$.
- Sample C: $t_C = 6,\ e_C = 10$.

Determine:

1. Do samples A and B overlap? Give their intersection.
2. Do samples A and C overlap? Explain why or why not.

31-E02: Purge Sample Count on a Regular Single-Asset Calendar

A single-asset trading calendar is continuous and has no missing observations. The forecast holding period is $h = 10$ trading days. If the test set starts at trading-day index $t = 100$, derive the index range of training samples before the test set that Purge removes, and give the total number of samples.

31-E03: Why Embargo Is Not Needed in a Purely Forward Split

Why does a purely forward Walk-Forward split, in which every training sample occurs before the test block, require Purge but no Embargo? Prove this from the perspective of causal information flow and the mathematics of feature lookback windows.

31-E04: Availability Later Than Label Settlement

A factor label is based on a listed company's quarterly return on equity (ROE). Assume the fiscal quarter ends at $e_i = 2025\text{-}03\text{-}31$, but the actual publication and ingestion time is $a_i = 2025\text{-}04\text{-}30\ 10{:}00$. This is a fact stipulated by the exercise, not inferred from a statutory filing deadline.

If the model prediction time is $t_{\text{predict}} = 2025\text{-}04\text{-}15\ 15{:}00$, may this sample enter the current training set? What bias would arise if it were used as a training sample on 2025-04-15?

31-N01: Find the "Resurrected Sample" in the Flowchart (New)

Someone connects "overlap → remove by Purge" to "not post-test → retain," then adds the retained item to the final training set. Use sample 2 to explain the error and write the four correct sets with their containment relationships. Explain why samples 1 and 13, which both touch endpoints, have different final outcomes. If the deployment time is 6 and sample 1 has $a_1 = 7$, at which gate should it be blocked?

---

16. Programming Exercises

31-P01: Implement a Half-Open Interval Intersection Test

Write a function `is_overlapping(t1: float, e1: float, t2: float, e2: float) -> bool` and verify that it:

1. Raises `ValueError` when an input has $e \leq t$.
2. Correctly returns whether the half-open intervals $(t_1, e_1]$ and $(t_2, e_2]$ intersect.
3. Returns `False` for the endpoint-touching pair $(1, 6]$ and $(6, 11]$.

31-P02: Implement Purge for a Cross-Sectional Panel

Write a function `panel_purge(train_dates: np.ndarray, test_dates: np.ndarray, h: int) -> np.ndarray`. The inputs are row-date arrays from a five-asset panel, with multiple rows on the same date. The test set contains samples from specified dates. Return the row indices retained in the training set after Purge, and assert that no observations on overlapping dates remain.

31-P03: Demonstrate the Failure of scikit-learn TimeSeriesSplit gap

Write a standalone Python script that constructs a synthetic panel with three assets over 15 trading days, uses labels with $h = 5$ days, and sets dates 6–8 as the test set. Run `TimeSeriesSplit(gap=4)`, calculate the number of overlapping rows left in training exactly, and verify whether it is exactly eight.

31-P04: Implement a Strictly Leakage-Resistant Nested Evaluator

Using `sklearn.linear_model.Ridge`, implement a hyperparameter selector with two outer causal forward folds and two inner validation folds. Fit feature-scaling parameters only on the inner training set, and report the uncontaminated mean squared error on the outer test sets.

---

17. Challenge Problems

31-C01: Purge Interval Algebra for Irregular Calendars and Suspended Assets (Open Derivation)

In real A-share or Hong Kong markets, an asset may be suspended for several weeks, or different assets may use different delivery and settlement calendars.

1. Formally define label intervals that incorporate each asset's individual suspension calendar.
2. Prove that, on an irregular calendar, the simplified addition $e_i = t_i + h$ based on trading-day index t can cause local lookahead or over-purge.
3. Propose a rubric for designing a robust Purge engine aligned by actual calendar days or tradable tick timestamps.

31-C02: A Counterexample to Purged K-Fold Under a Nonstationary Regime Shift (Theory and Numbers)

Construct a two-stage Markov regime-switching data-generating process, such as a low-volatility bull market followed by a high-volatility bear market.

Show that even if Purged K-Fold completely removes overlapping labels and feature-embargo leakage, training on future bear-market data while testing a historical bull-market interval can still make the model parameters differ substantially from the parameters best for the historical regime. Give quantitative evidence and recommendations.

31-C03: Purged Validation Gates Combined with a Multiple-Testing Trial Ledger (Integrated Design)

Combine the TrialLedger from Chapter 22 with the future-perturbation test from Chapter 30 to design an integrated quantitative strategy-admission verification protocol. The protocol must record the Purged splits used to evaluate every hyperparameter variant, account for the number of effective independent trials, and retain a final holdout-test safeguard.

31-C04: Extending Validation to Continuous Labels and Event-Driven Triple-Barrier Labels

Financial applications often use take-profit, stop-loss, and time-expiry triple-barrier labels, in which the label end time e_i is a random variable triggered by the price path.

Derive and prove that, under triple-barrier labeling, each sample's e_i is no longer a constant $t_i + h$. Explain how to compute variable-length Purge and Embargo sets dynamically.

18. Solutions and Solution Hints

31-E01 Solution

1. **Samples A and B:**
   - Sample A has interval $I_A = (2, 6]$, and sample B has interval $I_B = (5, 9]$.
   - For the overlap test, $\max(t_A, t_B) = \max(2, 5) = 5$ and $\min(e_A, e_B) = \min(6, 9) = 6$.
   - Since $\max(t_A, t_B) = 5 < \min(e_A, e_B) = 6$, the samples **overlap**.
   - Their intersection is $(\max(t_A, t_B), \min(e_A, e_B)] = (5, 6]$.
2. **Samples A and C:**
   - Sample A has interval $I_A = (2, 6]$, and sample C has interval $I_C = (6, 10]$.
   - The overlap test gives $\max(t_A, t_C) = \max(2, 6) = 6$ and $\min(e_A, e_C) = \min(6, 10) = 6$.
   - The condition $\max(t_A, t_C) < \min(e_A, e_C)$ does not hold, because $6 < 6$ is false. The samples **do not overlap**.
   - Economically, sample A's return settles at the close on day 6, while sample C's return starts after the close on day 6. They touch at one endpoint on the time axis but share no holding-period return.

31-E02 Solution

- As derived in §6.2, when the test block starts at $t_{\text{test}} = 100$, its first sample has interval $(100, 100 + 10] = (100, 110]$.
- A historical training sample at t_i has interval $(t_i, t_i + 10]$.
- The necessary and sufficient condition for overlap with the test set is $t_i + 10 > 100 \iff t_i > 90$.
- Since $t_i < 100$, the exact range of historical training indices removed by Purge is:

$$t_i \in \{91, 92, 93, 94, 95, 96, 97, 98, 99\}$$

- The total removed is $99 - 91 + 1 = 9 = h - 1$ samples.

31-E03 Solution

- **Proof:**
  1. Embargo is defined to prevent the **feature lookback window of post-test training samples** from reaching into the test set's realized-return label intervals.
  2. In a purely forward Walk-Forward split, every training sample satisfies $t_{\text{train}} < t_{\text{test,min}}$.
  3. When historical training features are calculated from technical indicators or rolling features, they use information from earlier times, at indices $\leq t_{\text{train}} < t_{\text{test,min}}$. They lie entirely before the test set.
  4. Historical training features therefore cannot, in the causal time order, reach into the future test interval.

5. Thus a purely forward training set has no feature-lookback penetration risk. Purge is still needed to remove label overlap at the end of the training set, but Embargo is not needed.

31-E04 Solution

- **Decision:** This sample **must not** enter the current training set.
- **Reason and bias:**
  - Although the operating period reflected by the financial statement ended at e_i = 2025-03-31, at the model decision time t_{predict} = 2025-04-15 15:00 the stipulated actual availability time is April 30 at 10:00. The statement is not yet readable on April 15 at 15:00. If only the statutory latest filing deadline were known, without the actual publication and ingestion time, that would not establish whether the information was unknown on April 15; the actual timestamp would need to be checked.
  - Treating the sample as known training data on April 15 would fit model parameters using information that had not yet been disclosed, creating classic look-ahead bias.
  - The correct condition for a sample to enter training is the availability-time constraint $a_i \le t_{\text{predict}}$.

31-N01 Full Solution (New)

The interval $I_2 = (2, 7]$ for sample 2 overlaps $I_6 = (6, 11]$, so sample 2 must be removed. "Not post-test" can determine whether a candidate that has **not already been removed** is exempt from Embargo; it cannot restore a sample removed by Purge. The sets are $R_{\text{purge}} = \{2, 3, 4, 5, 9, 10, 11, 12\}$, $S_{\text{after_purge}} = \{0, 1, 13, 14\}$, $R_{\text{embargo}} = \{13, 14\}$, and $S_{\text{final}} = \{0, 1\}$. The final set is contained in the Purge survivors, which are contained in the candidate set.

The return set for sample 1 ends at r_6, while sample 6's begins at r_7. The return set for sample 13 begins at r_{14}, while sample 8's ends at r_{13}. Neither pair overlaps. Sample 13 is in the post-test buffer $[13, 16)$, however, so its feature lookback reaches into the test labels and it must be embargoed. If $a_1 = 7 > 6$, sample 1 should be removed before deployment candidates are generated, not left for the interval-touching test to decide.

31-P01 Reference Implementation

```
def is_overlapping(t1: float, e1: float, t2: float, e2: float) -> bool:
    if e1 <= t1 or e2 <= t2:
        raise ValueError("Label end e must be strictly greater than observation time
    ↪ t")
    return max(t1, t2) < min(e1, e2)

# Test assertions
assert is_overlapping(2, 7, 6, 11) is True
assert is_overlapping(1, 6, 6, 11) is False  # Endpoint touching is not overlap
assert is_overlapping(0, 5, 6, 11) is False
```

31-P02 Reference Implementation

```
import numpy as np
from quant_course.validation import purge

def panel_purge(train_dates: np.ndarray, test_dates: np.ndarray, h: int) -> np.
    ↪ ndarray:
    # Dates are common integer observation positions, not datetime strings.
    if isinstance(h, bool) or h < 1 or int(h) != h:
        raise ValueError("h must be a positive integer observation horizon")
    e_train = train_dates + float(h)
    e_test = test_dates + float(h)
    cand_idx = np.arange(len(train_dates))
    test_idx = np.arange(len(test_dates))
    # Concatenate the full sample for the standard purge
    all_t = np.concatenate([train_dates, test_dates])
    all_e = np.concatenate([e_train, e_test])
    test_offset_idx = np.arange(len(train_dates), len(all_t))
    purged = purge(cand_idx, test_offset_idx, all_t, all_e)
    return purged

# Verification
dates_tr = np.repeat(np.arange(6), 5)  # Five assets on every date 0..5.
dates_te = np.repeat(np.array([6, 7, 8]), 5)
kept = panel_purge(dates_tr, dates_te, h=5)
assert np.array_equal(kept, np.arange(10))  # dates 0 and 1: all ten rows
assert not np.any(np.maximum(dates_tr[kept, None], dates_te[None, :]) <
                  np.minimum(dates_tr[kept, None] + 5, dates_te[None, :] + 5))
```

31-P03 Reference Implementation

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit

dates = np.repeat(np.arange(15), 3)  # 45 rows
test_idx = np.arange(18, 27)         # dates 6..8, 9 rows total
tss = TimeSeriesSplit(n_splits=3, test_size=9, gap=4)
train_tss, test_tss = next(tss.split(np.arange(45)))
assert np.array_equal(test_tss, test_idx)  # Full 45-row sample: first test rows
    ↪ 18..26
assert np.array_equal(train_tss, np.arange(14))
e = dates + 5
overlapping_rows = [i for i in train_tss if any(
    max(dates[i], dates[j]) < min(e[i], e[j]) for j in test_tss)]
assert overlapping_rows == list(range(6, 14))
print("Overlapping rows left in training:", len(overlapping_rows))
assert len(overlapping_rows) == 8
```

31-P04 Reference Implementation

The forward array period_returns[i] represents the return over $(i, i+1]$, corresponding to the formal notation r_{i+1}. The label y[i] is the sum of the next two returns, with $e_i = a_i = i + 2$. Future array values generate the label and must not enter the current feature vector. Each outer fold has two distinct inner validation windows. Each uses only earlier training rows whose labels are available; the helper then applies Purge, and scaling is fit only on that training fold.

```
import numpy as np
from sklearn.linear_model import Ridge
from quant_course.validation import nested_hyperparameter_evaluation

rng = np.random.RandomState(123)
N = 100
X_all = rng.randn(N + 1, 2)
X = X_all[:-1]
period_returns = 1.2 * X_all[:, 0] + rng.randn(N + 1) * 0.2
y = period_returns[:-1] + period_returns[1:]
t = np.arange(N, dtype=float)
e = t + 2.0
a = e.copy()  # Availability equals interval end in this example.

candidate_splits = [(np.arange(50), np.arange(50, 75)), (np.arange(75), np.arange(75,
    ↪  100))]
outer_splits = []
for train_idx, val_idx in candidate_splits:
    prediction_time = np.min(t[val_idx])
    outer_splits.append((train_idx[a[train_idx] <= prediction_time], val_idx))
def inner_split(out_tr, t_arr, e_arr):
    n = len(out_tr)
    folds = []
    for start, stop in [(n // 3, 2 * n // 3), (2 * n // 3, n)]:
        in_train = np.arange(start)
```

```
        in_val = np.arange(start, stop)
        prediction_time = np.min(t_arr[out_tr[in_val]])
        in_train = in_train[a[out_tr[in_train]] <= prediction_time]
        folds.append((in_train, in_val))
    return folds

res = nested_hyperparameter_evaluation(
    X, y, t, e, outer_splits, inner_split,
    [{"alpha": 0.1}, {"alpha": 10.0}],
    lambda p: Ridge(alpha=p["alpha"]),
    lambda m, Xt, yt: -float(np.mean((m.predict(Xt) - yt) ** 2)),
    scale_features=True
)
assert len(res["outer_scores"]) == 2
assert all(len(folds) == 2 for folds in res["inner_splits_used"])
for folds in res["inner_splits_used"]:
    for fold in folds:
        tr = np.array(fold["train_idx"]); va = np.array(fold["val_idx"])
        assert tr.size and va.size and t[tr].max() < t[va].min()
        assert np.all(a[tr] <= t[va].min())
        assert not np.any(np.maximum(t[tr, None], t[None, va]) <
                          np.minimum(e[tr, None], e[None, va]))
print("Outer MSE:", [-s for s in res["outer_scores"]])
print("Purged outer splits:", res["outer_splits_used"])
print("Purged inner splits:", res["inner_splits_used"])
```

31-C01 Full Reference Answer: Irregular Calendars

First state the economic target. This problem defines settlement as the next two valid price observations for that asset, not the next two calendar days. A real study could instead use a fixed common-calendar horizon; those are different labels and should not be treated as interchangeable. Let $\tau_i(t, 2)$ return the second valid observation date for asset i after t. Asset A has observable dates $0, 1, 2, 8, 9, 10$, with no executable price during its suspension; asset B is observable every day. A's label at $t = 2$ ends at $e = 9$, giving interval $(2, 9]$. B's label at $t = 2$ ends at 4, giving interval $(2, 4]$. Let the test sample be $(8, 10]$. Check each interval:

$$\max(2, 8) = 8 < \min(9, 10) = 9$$

Thus A and the test interval share $(8, 9]$, so A must be purged. For B, the condition $8 < 4$ is false, so B is retained. If A's end were incorrectly set to $2 + 2 = 4$, the true overlap would be missed. Training eligibility also requires the actual label-availability time $a_i \le t_{\text{predict}}$; a post-suspension settlement value cannot be available in advance. All intervals here remain half-open, so endpoint touching is not overlap.

For precise Purge, use the actual end time $e_i = \tau_i(t_i, h)$ for every row. A training sample is retained only if:

$$\max(t_i, t_j) \ge \min(e_i, e_j), \qquad \forall j \in S_{\text{test}}.$$

This is the requirement that it not overlap any test sample, rather than a check against an arbitrary representative date. If a single conservative endpoint is required, take $e^{\max}(t) =$

$\max_i e_i(t)$ across assets on the same date and purge against $(t, e^{\max}(t)]$. This envelope contains each actual interval, so it will not miss an overlap, but it may wrongly remove B's non-overlapping training sample in this example and reduce the usable sample size. A longer interval does not provide more information. Report the difference between the conservative and precise rules.

31-C02 Full Reference Answer: Purging Does Not Remove Mechanism Changes

First define a two-state Markov chain with $S_0 = 0$ and transition matrix $\begin{pmatrix} 1-p & p \\ 0 & 1 \end{pmatrix}$, where $0 < p < 1$. Consider the path $S_0, \ldots, S_{2m-1} = 0$ and $S_{2m}, \ldots, S_{3m-1} = 1$. Its probability is $(1-p)^{2m-1}p > 0$. This is a positive-probability path under an absorbing two-stage mechanism; it does not imply the same error on every realization or every path.

Sample i is formed at $i-1$. Its one-period label $y_i = 2S_i$ settles at i and is then available, with interval $I_i = (i-1, i]$. The endpoint is settled before fitting at the same timestamp. Divide the path into three equal contiguous blocks, B0, B1, and B2, each with m labels. Labels are 0 in B0 and B1, and 2 in B2. The adjacent return intervals at block boundaries only touch, so correct Purge does not change the following training sets.

For a bidirectional K-Fold test on B0, train on B1 and B2. The training-label mean is $(0+2)/2 = 1$, so the B0 test MSE is $(0-1)^2 = 1$. But at the historical B0 time, B2 has not yet occurred; this model is not a deployable historical prediction. Under strict forward validation, train on B0 and test on B1. The training mean is 0 and the MSE is 0. When B2 is the first block after the regime change, training on B0 and B1 still predicts 0, giving B2 test MSE $(2-0)^2 = 4$. These folds do not represent the same task unless their periods are stated.

The point is that interval purging removes information sharing caused by intersecting label windows; it does not make future training data available in the past or ensure identical distributions across blocks. Bidirectional Purged K-Fold can be used as a clearly bounded historical-regime diagnostic. Deployment evaluation requires forward training, filtering by actual availability times, and checking sensitivity to regime changes and windows. Forward validation also cannot guarantee a stable future distribution. This is not a formal definition of a "conditional-variance tester," and passing Purge does not prove that every form of information leakage has been eliminated.

31-C03 Full Reference Answer: Trial Ledgers and Admission Gates

Do not arbitrarily reduce the number of trials used for FWER correction based on feature correlation or spectral effective dimension. Retain every attempt in the Chapter 22 ledger. If an effective-trial approximation is used, separately construct the null distribution of the search statistic and state the dependence assumptions it requires; otherwise apply conservative Bonferroni or Holm correction to the full family. The Chapter 30 Z versus $-$Z counterexample shows that dimension 1 does not mean that only one trial was attempted.

Freeze the three specifications in the same hypothesis family, along with the primary metric, label horizon, Purge rule, sample window, and test method. Record every attempt whose result

has been inspected. Suppose the three valid individual p-values are 0.008, 0.019, and 0.040, and the family-wise significance level is 0.05. Bonferroni sets each threshold to $0.05/3 = 0.016667$, so only the first result passes.

For Holm, sort the p-values in ascending order and compare them successively with $0.05/3$, $0.05/2$, and 0.05: $0.008 \leq 0.016667$, $0.019 \leq 0.025$, and $0.040 \leq 0.05$. All three pass in this example. If a step fails, stop rejecting subsequent hypotheses. Both methods control the FWER for a predefined hypothesis family and allow dependence between tests, but each individual p-value must itself be valid. Ordinary standard errors that assume independence cannot be substituted for overlapping labels or correlated folds and then described as corrected. Where needed, use time-block resampling or an estimator appropriate to the dependence structure, and disclose the block length and uncertainty.

Select the primary specification using only the discovery period and inner training validation; do not use the outer test for selection. Set economic thresholds, such as post-cost performance, maximum turnover, and worst-case scenarios, then run a completely unused holdout period once. Report it whether it passes or fails. If the model is changed after inspecting the holdout results, that period has become part of discovery and can no longer be called an untouched test. Passing all three tests also does not prove three tradable Alphas; economic significance, execution costs, capacity, and validation in a new period remain necessary.

31-C04 Full Reference Answer: Event Endpoints and Availability Times

Given profit and loss barriers and a maximum observation horizon h, the label end time is:

$$e_i = \min(t_{\text{profit}}, t_{\text{loss}}, t_i + h).$$

A barrier that is never triggered can be represented by $+\infty$. For example, sample A forms at $t_A = 0$ with maximum horizon 5, and its profit barrier is first triggered at 3. Then $e_A = 3$ and $I_A = (0, 3]$. Sample B forms at 2 and hits its loss barrier at 4, so $I_B = (2, 4]$. They share $(2, 3]$; if B is a test sample, A must be purged. A different test interval, $(3, 5]$, only touches A's interval at an endpoint, so A is not purged.

Do not set every sample's endpoint to the maximum horizon. That would incorrectly extend $(0, 3]$ to $(0, 5]$ and remove samples that touch but do not overlap. Nor should a "barrier-trigger day" automatically be described as an executable settlement time. With daily data, the order in which barriers were hit intraday may be unknown; a gap can execute beyond the barrier, and execution on the next day changes the measured return endpoint. First state whether the label uses the first barrier mark or the actual exit execution return, and define e_i accordingly. If the label also requires exit execution, fees, or settlement data, record the actual availability time a_i separately. First filter for $a_i \leq t_{\text{predict}}$, then pass the realized intervals $(t_i, e_i]$ to the Purge function. Record these times and assumptions in the data dictionary and trial ledger.

---

19. Further Reading

1. **Official scikit-learn model-selection documentation (Must Know):**
   - Read the source implementation of `sklearn.model_selection.TimeSeriesSplit` and the time-series-splitting discussion in the official User Guide.
   - Focus on the difference between how the `gap` parameter counts observations in a single time series versus a multi-sample cross-section, and understand its row-count limitation.
2. **Extensions to financial cross-validation and information purging (Advanced, not reviewed):**
   - Marcos López de Prado (2018), *Advances in Financial Machine Learning*, John Wiley & Sons, Chapter 7, "Cross-Validation in Finance."
   - **Status:** No print or electronic copy of this book is present in the local workspace, and it has not been reviewed. This is only a further-reading lead; no priority claim is based on a source that has not been reviewed. This chapter presents interval algebra and conditional derivations directly and does not claim to establish an independent universal standard.
3. **Nested hyperparameter optimization and generalization assessment (Should Know):**
   - Read a standard machine-learning textbook's derivation of the asymptotic unbiasedness of generalization-error estimates from nested cross-validation.
   - Understand how the inner and outer loops must preserve one-way causal flow under time dependence.

---

20. References

1. `SK_TSS`: scikit-learn documentation for `sklearn.model_selection.TimeSeriesSplit`, version 1.9.1 in the historical 2026-10-07 record. This revision does not claim it is the latest online version. Historical local source inspection confirmed `train_end = test_start - gap` and its row-count-based truncation.
2. `SK_PITFALLS`: scikit-learn documentation, "Common pitfalls and recommended practices in machine learning validation." Official guidance on leakage from full-sample preprocessing and feature contamination.
3. `PBO`: Bailey, D. H., Borwein, J., López de Prado, M., & Zhu, Q. J. (2014). "Pseudo-Mathematics and Financial Charlatanism: The High Probability of Overfitting in Financial Investment." *Notices of the AMS*.
4. `FRENCH_10_IND`: Kenneth R. French Data Library, 10 Industry Portfolios (value-weighted, monthly), 1926-07 through 2026-08 (1,202 months). Manifest file SHA-256: `3b71cd5f781f9790cc380195c22635e03e9c9e27a5f43cd2d0d46fb12efa8271`.

5. Chapter code and historical independent-review results, not rerun for this revision: `code/ch31_model_validation.py`; numerical output: `build/codex_a6/final_learning_review/ch31/recheck/results/ch31_results.json`; historical review figures: `build/codex_a6/final_learning_review/ch31/recheck/figures/ch31_*.png`. Existing historical QA and figures cited in the chapter have not been overwritten.

Chapter 32 — Performance Statistics & Inference

Thread: First describe the returns the account has realized. Then interpret the performance evidence in light of sample length, moment conditions, and the complete search record. Finally, use relative out-of-sample ranks to diagnose selection.

Prerequisites: Chapter 7, Chapter 11, Chapter 22, Chapter 29, and Chapter 31. After this chapter, you will be able to calculate and interpret all 11 descriptive measures; conduct PSR and DSR significance tests and a CSCV probability-of-backtest-overfitting diagnosis; recognize multiple-testing traps; and produce a traceable performance-inference report containing byte hashes and a trial ledger. A hash is not an external timestamp or tamper-proof certification.

---

1. Learning Objectives

After completing this chapter, you should be able to:

1. **Calculate descriptive statistics and distinguish their conventions.** Keep simple returns separate from log returns, and arithmetic averages separate from the compound growth rate (CAGR), a geometric average. Calculate annualized volatility, per-period and annualized Sharpe ratios, the Sortino ratio, maximum drawdown (MaxDD), the Calmar ratio, hit rate, Profit Factor, tracking error, and information ratio; connect these measures to the turnover recorded in the actual ledger in Chapter 29.
2. **Make statistical inferences under non-normality (PSR).** Understand how higher moments—skewness γ_3 and kurtosis γ_4—inflate the estimated variance of the Sharpe ratio; use the Probabilistic Sharpe Ratio (PSR) formula and its one-sided hypothesis test; and distinguish the probability of skill from the probability of making money.
3. **Account for selection bias and multiple testing (DSR).** Understand how the maximum Sharpe ratio is inflated when the best result is selected from N independent trials; derive the Deflated Sharpe Ratio (DSR) and its threshold SR_0; and understand the heuristic estimate of the effective number of independent trials when trials are correlated.
4. **Use Combinatorially Symmetric Cross-Validation and the Probability of Backtest Overfitting (CSCV/PBO).** Understand the matrix-partition algorithm, relative rank $\bar{\omega}$, and the Logit transform λ; distinguish the diagnostic role of CSCV from that of a time-ordered deployment backtest (Walk-Forward).
5. **Recognize inference failure conditions and empirical checks.** Understand variance inflation due to serial dependence, heavy tails, and extreme regime changes, and apply the full inference workflow to the Kenneth French market data.

2. Why Quant Needs This

One of the most common illusions in a quantitative research lifecycle is the "strategy champion trap." A researcher tries 200 combinations of moving-average periods, exit thresholds, or cross-sectional factor weights, then selects a "champion strategy" with an annualized Sharpe ratio of 2.5. The researcher reports to the investment committee with great excitement, claiming overwhelming statistical significance.

Yet six months after deployment, the strategy suffers a severe drawdown. Why can a perfect historical backtest fail to reproduce its performance in live trading?

Classical financial statistics textbooks often assume that asset returns are independent and identically distributed (i.i.d.) normal observations and test one prespecified strategy once. Modern quantitative research faces two unavoidable violations:

1. **Hidden risk from non-normal returns.** Hedge funds and quantitative strategies—especially mean-reversion, leveraged arbitrage, and short-volatility strategies—often exhibit negative skew and high excess kurtosis (heavy tails). Ignoring negative skew and heavy tails can substantially understate the estimation variance of the Sharpe ratio and greatly overstate statistical significance.
2. **Selection bias from multiple hypothesis tests.** Even if all 200 candidate models are generated by pure Gaussian white noise with no predictive power, the expected maximum sample Sharpe ratio is naturally greater than zero. Reporting only the winning point estimate without disclosing the trial search ledger is a classic form of data snooping.

A quantitative researcher therefore needs inferential statistics as well as descriptive return measures: use PSR to measure estimation uncertainty under stated moment conditions, DSR to raise the comparison threshold after a search, and CSCV to examine the relative out-of-sample ranks of candidates selected in sample.

3. Intuition

3.1 A Coin-Toss Stock-Picking Contest

Imagine a contest with 1,000 participants, each of whom tosses a coin 10 times. A head counts as a win and a tail as a loss.

For one fair-coin participant, the probability of getting 10 heads in a row is $p = 2^{-10}$. The expected number of perfect records among 1,000 participants is $1000p = 0.9765625$. This does not mean that exactly one participant will win every contest. If participants are also independent, the probability that at least one gets 10 heads is $1-(1-2^{-10})^{1000} \approx 0.623576$. If that "champion" claims to have a special ability to predict the future, an investor who sees only the champion's 10-for-10 record (the point estimate) may be impressed. An investor who sees the full list of 1,000

participants (the trial ledger) will recognize that the perfect record can arise as an extreme value from random sampling.

Searching for quantitative strategies works the same way:

- **PSR intuition:** If a participant earns very little on heads but loses heavily on tails (negative skew and heavy tails), the confidence band around the long-run Sharpe ratio is wide even if the win rate is respectable.
- **DSR intuition:** Given N entrants, extreme-value theory raises the comparison threshold SR_0 under the zero-skill hypothesis. Exceeding the threshold is statistical evidence against that hypothesis, but does not prove repeatable true alpha.
- **CSCV intuition:** Divide history into blocks. If the champion selected on some blocks tends to rank poorly relative to other candidates on the remaining blocks, the "optimization" may have selected a candidate that performs worse out of sample. The strict definition uses $\bar{r} < N/2$; CSCV also uses $\bar{r} \leq (N+1)/2$.

---

4. Mathematical Setup

4.1 Notation and Assumptions

- **Return intervals:** Equal-length discrete periods $t = 1, 2, \ldots, T$, with per-period simple returns $r_t \in (-1, +\infty)$ while the account is solvent. Wealth calculations also allow the boundary $r_t = -1$: without external capital, account wealth is then absorbed at zero and later multiplicative returns cannot revive it. A simple return below -100% requires a different negative-wealth model and is rejected by the current interface.
- **Annualization factor:** Let A be the number of equally spaced periods in a year (daily $A = 250$, monthly $A = 12$).
- **Benchmark and risk-free series:** A risk-free return vector $r_{f,t}$ and benchmark portfolio return vector $r_{B,t}$, both at the same frequency and of equal length.
- **Excess returns:** $r_{e,t} = r_t - r_{f,t}$. Their sample mean and standard deviation at the same frequency are

$$\hat{\mu}_e = \frac{1}{T}\sum_{t=1}^{T} r_{e,t}, \qquad \hat{\sigma}_e = \sqrt{\frac{1}{T-1}\sum_{t=1}^{T}(r_{e,t} - \hat{\mu}_e)^2}.$$

- **PSR moments:** The PSR implementation estimates skewness and Pearson's raw kurtosis from excess returns. `psr_from_returns` uses bias-corrected skewness and `scipy.stats.kurtosis` with `fisher=False` and `bias=False`. The normal benchmark has raw kurtosis $\gamma_4 = 3$, not excess kurtosis.

4.2 Inputs, Assumptions, and Interpretation

For a high-Sharpe curve, first ask what happened, then ask how much evidence it provides relative to a threshold. Convert frequencies before substituting values; do not add an annualized label after the calculation.

Input or output	Frequency and units	Conditions and interpretation
$r_t, r_{f,t}, r_{e,t}$	Same observation period; dimensionless simple returns	Subtract contemporaneous observations; do not subtract an annual risk-free rate directly from monthly returns
$\widehat{SR}, SR^*, SR_0$	Per-period, dimensionless ratios at the same frequency	The observed value and comparison threshold use the same scale
n, γ_3, γ_4	Number of observations at the same frequency; dimensionless skewness and Pearson raw kurtosis	Normal raw kurtosis is 3; matching two moments does not prove normality
PSR standard error	Standard error of the per-period Sharpe ratio	Assumes i.i.d. observations, finite fourth moment, and an asymptotic approximation; serial dependence cannot be fixed by merely increasing n
$V[\widehat{SR}], N$	Sharpe variance at the same frequency; number of trials	Variance scales with the square of the Sharpe frequency conversion; N depends on an independent-search model and is not automatically the number of candidate columns
PSR, DSR, PBO	Dimensionless tail area or fraction of partitions	PSR is threshold inference, DSR is conditional calibration, and PBO is a candidate-rank diagnostic; they answer different questions

If $SR_{ann} = \sqrt{A}\,SR$, then $V[SR_{ann}] = A\,V[SR]$. For example, with an annualized Sharpe of 2.5 and 250 trading days per year, first compute the daily Sharpe $2.5/\sqrt{250} = 0.158114$, then compare it with the daily threshold 0.113172; the difference at the same frequency is 0.044942. Compute the standard error from the native daily returns first. If these already computed quantities are then displayed consistently in annualized units, multiply the observed value, threshold, and standard error by $\sqrt{250}$; z is unchanged. Do not leave daily n and moments unchanged while substituting an annualized Sharpe into the per-period D formula. Changing only the observed Sharpe creates spurious significance (32-L02).

The annualization convention follows from sums of returns: under independence, identical distribution, and finite variance, the mean of an A-period sum is $A\mu$, its standard deviation is $\sqrt{A}\,\sigma$, and their ratio is $\sqrt{A}\,\mu/\sigma$. Normality is not required for this scaling identity. Actual annual wealth from simple returns is $\prod_t(1+r_t)$, not $1+\sum_t r_t$, so this Sharpe annualization does not replace CAGR. Serial dependence requires adding the covariances across lags (32-L04).

5. Formal Definitions

5.1 Descriptive Measures

1. **Wealth index and CAGR.** Starting from initial capital $W_0 > 0$, the compounded wealth path is
$$W_t = W_0 \prod_{i=1}^{t} (1 + r_i).$$
Over a span of Y years, the compound annual growth rate is
$$\mathrm{CAGR} = \left(\frac{W_T}{W_0} \right)^{1/Y} - 1.$$
CAGR equals -100% if and only if bankruptcy occurs ($W_T = 0$).
2. **Maximum drawdown (MaxDD) and Calmar ratio.** The historical wealth peak at time t is $M_t = \max_{0 \le s \le t} W_s$. The drawdown series is $D_t = (W_t - M_t)/M_t \in [-1, 0]$. Maximum drawdown is its nonnegative absolute value, $\mathrm{MaxDD} = \max_t |D_t|$. The Calmar ratio is
$$\mathrm{Calmar} = \frac{\mathrm{CAGR}}{\mathrm{MaxDD}}.$$
If MaxDD $= 0$, Calmar is undefined and an error is raised.
3. **Per-period and annualized Sharpe ratios.** The per-period Sharpe ratio is defined at the native frequency of the return series and has no annualization factor:
$$\hat{\mathrm{SR}} = \frac{\hat{\mu}_e}{\hat{\sigma}_e}.$$
The conventional annualized Sharpe ratio is $\hat{\mathrm{SR}}_{\mathrm{ann}} = \sqrt{A}\,\hat{\mathrm{SR}}$.
4. **Sortino ratio and downside deviation.** Let the target return threshold be τ at the same frequency. The full-sample downside deviation includes all n observations:
$$\mathrm{DD}_\tau = \sqrt{\frac{1}{n} \sum_{t=1}^{n} \min(r_t - \tau, 0)^2}.$$
The per-period Sortino ratio is $(\bar{r} - \tau)/\mathrm{DD}_\tau$. Its annualized form conventionally multiplies by $\sqrt{A}$; downside deviation and the target do not automatically scale according to the independent-sum standard-deviation theorem. If downside deviation is zero, the ratio is undefined. The target must be finite.
5. **Hit rate and Profit Factor.**
   - **Strict hit rate, excluding flat periods:**
$$\mathrm{HitRate} = \frac{\sum \mathbf{1}_{\{r_t > 0\}}}{\sum \mathbf{1}_{\{r_t \neq 0\}}}.$$
   - **Profit Factor, the ratio of total gains to the absolute value of total losses:**
$$\mathrm{ProfitFactor} = \frac{\sum_{r_t > 0} r_t}{|\sum_{r_t < 0} r_t|}.$$

This chapter gives each period equal weight. This is neither average winning return divided by average losing return nor a transaction Profit Factor calculated from actual monetary trade P&L. If there are no losses, Profit Factor is undefined. The hit-rate denominator excludes zero-return periods; it is also undefined when every period is flat.

6. **Tracking error and information ratio.** Active returns are $r_{a,t} = r_t - r_{B,t}$. Annualized tracking error is $\mathrm{TE} = \hat{\sigma}(r_a)\sqrt{A}$. The annualized information ratio is $\mathrm{IR} = (\bar{r}_a/\hat{\sigma}(r_a))\sqrt{A}$.
7. **Ledger turnover.** Connect this measure to the actual backtest ledger in Chapter 29. Turnover for one rebalance is total traded notional divided by pre-trade NAV:

$$\mathrm{Turnover}_t = \frac{\mathrm{traded_notional}_t}{\mathrm{nav_pre_trade}_t}.$$

5.2 Inference Measures

1. **Probabilistic Sharpe Ratio (PSR).** Under i.i.d. returns with a finite fourth moment, skewness and heavy tails adjust the estimated variance of the sample per-period Sharpe ratio. For a same-frequency benchmark SR^*,

$$\mathrm{PSR}(\mathrm{SR}^*) = \Phi\left(\frac{(\hat{\mathrm{SR}} - \mathrm{SR}^*)\sqrt{n-1}}{\sqrt{1 - \hat{\gamma}_3\hat{\mathrm{SR}} + \frac{\hat{\gamma}_4 - 1}{4}\hat{\mathrm{SR}}^2}}\right).$$

2. **Deflated Sharpe Ratio (DSR).** When the best Sharpe ratio is selected from N independent trials, the expected maximum random Sharpe threshold under the zero-skill hypothesis is

$$\mathrm{SR}_0 = \sqrt{V[\{\hat{\mathrm{SR}}_n\}]}\left((1-\gamma)\Phi^{-1}\left(1 - \frac{1}{N}\right) + \gamma\Phi^{-1}\left(1 - \frac{1}{Ne}\right)\right),$$

where $\gamma \approx 0.5772156649$ is the Euler–Mascheroni constant. This uses an independent Gaussian Sharpe approximation and applies to integer $N \geq 2$. Even at $N = 2$ it is an approximation: the exact expected maximum of two standard normal variables is $1/\sqrt{\pi} \approx 0.564190$, while the approximation gives 0.519755. With $N = 1$ there is no search premium; the zero-mean noise threshold is 0, and one can use a single-test PSR. The current DSR API explicitly requires $N \geq 2$ and does not substitute 1 into the inverse CDF. The expected maximum is a comparison threshold, not a critical value controlling the family-wise error rate of the whole search. DSR is not automatically an exact post-selection p-value. The Deflated Sharpe Ratio is

$$\mathrm{DSR} = \mathrm{PSR}(\mathrm{SR}_0).$$

3. **Combinatorially Symmetric Cross-Validation (CSCV) and PBO.** Divide the synchronized $T \times N$ candidate return matrix by rows into an even number S of equal-sized blocks. Choose any $S/2$ blocks for the in-sample set J and use the complementary blocks as the out-of-sample set $\bar{J}$. There are $C(S, S/2)$ combinations.
Select the best strategy in sample:

$$n^* = \arg\max_n \hat{\mathrm{SR}}_{n,\mathrm{IS}}.$$

Rank all N strategies in the test set, with the best ranked N. The test-set relative rank and Logit of the strategy selected in sample are

$$\bar{\omega}_c = \frac{\bar{r}_{n^*}}{N+1}, \qquad \lambda_c = \ln\left(\frac{\bar{\omega}_c}{1-\bar{\omega}_c}\right).$$

The current CSCV convention counts partitions whose test-set relative rank is at or below the midpoint: $\lambda \leq 0 \iff \bar{r} \leq (N+1)/2$. The local PBO Definition 2.2 instead uses $\bar{r} < N/2$. These are different thresholds and may produce different counts for even N as well; the difference is not merely an equality convention at one shared threshold.

$$\mathrm{PBO}_{\mathrm{CSCV}} = \frac{1}{C(S, S/2)} \sum_c \mathbf{1}_{\{\lambda_c \leq 0\}}.$$

---

6. Derivations

6.1 Algebra for the PSR Denominator Variance Term

Following Mertens (2002) and Bailey & López de Prado (2012a, Equation 8), the asymptotic variance expansion of the sample Sharpe ratio for non-normal i.i.d. returns is

$$\sigma^2_{\hat{\mathrm{SR}}} = 1 + \frac{1}{2}\mathrm{SR}^2 - \gamma_3 \mathrm{SR} + \frac{\gamma_4 - 3}{4}\mathrm{SR}^2.$$

Consider the two polynomial terms involving SR^2:

$$\begin{aligned}\frac{1}{2}\mathrm{SR}^2 + \frac{\gamma_4 - 3}{4}\mathrm{SR}^2 &= \frac{2}{4}\mathrm{SR}^2 + \frac{\gamma_4 - 3}{4}\mathrm{SR}^2 \\ &= \frac{\gamma_4 - 3 + 2}{4}\mathrm{SR}^2 \\ &= \frac{\gamma_4 - 1}{4}\mathrm{SR}^2.\end{aligned}$$

Substituting this identity into the asymptotic variance expression gives

$$\sigma^2_{\hat{\mathrm{SR}}} = 1 - \gamma_3 \mathrm{SR} + \frac{\gamma_4 - 1}{4}\mathrm{SR}^2.$$

The expression above is the asymptotic variance coefficient before scaling by sample size; it is not the finite-sample variance itself. Equation (11) uses an $n-1$ denominator and substitutes sample moments to obtain the following standard-error estimate. This approximation does not become an exact finite-sample result merely because of the Bessel correction:

$$\hat{\sigma}_{\hat{\mathrm{SR}}} = \sqrt{\frac{1 - \hat{\gamma}_3 \hat{\mathrm{SR}} + \frac{\hat{\gamma}_4 - 1}{4}\hat{\mathrm{SR}}^2}{n-1}}.$$

Under normality, $\gamma_3 = 0$ and $\gamma_4 = 3$, so the denominator term reduces to the classical Lo (2002) expression $\sqrt{1 + \frac{1}{2}\hat{\mathrm{SR}}^2}$. Matching these two moments does not establish that returns are normally distributed. When $\hat{\mathrm{SR}} > 0$, more negative skewness or larger raw kurtosis increases the variance term; the direction of the PSR change also depends on which side of the threshold the observed Sharpe lies, as discussed below.

The financial interpretation is to estimate the error in "mean divided by volatility" before comparing it with a threshold. Let

$$D = 1 - \gamma_3 \mathrm{SR} + \frac{(\gamma_4 - 1)\mathrm{SR}^2}{4}.$$

Then the estimator's standard error is approximately $\sqrt{D/(n-1)}$; subtracting the threshold from the observed value and dividing by this standard error gives the PSR z-statistic. Increasing n only reduces sampling error within a model whose conditions hold. It cannot make a nonexistent population fourth moment finite or make dependent observations independent.

The direction relative to the threshold also matters. For a fixed positive Sharpe, a moment change that increases D moves PSR toward 0.5: if the observed Sharpe is above the threshold, z falls and PSR moves down toward 0.5; if it is below, the negative z rises toward zero and PSR moves up toward 0.5; if it equals the threshold, PSR remains 0.5. Thus, "heavy tails always lower PSR" is missing the condition that the observed value is above the threshold (32-E02).

6.2 Extreme-Value Approximation for the Maximum of Independent Standard Normals

Let $Z_1, Z_2, \ldots, Z_N \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1)$, and let $M_N = \max(Z_1, \ldots, Z_N)$. Its cumulative distribution function is $F_{M_N}(x) = [\Phi(x)]^N$ and its density is $f_{M_N}(x) = N\phi(x)[\Phi(x)]^{N-1}$.

The exact expected value can be calculated by numerical integration:

$$E[M_N] = \int_{-\infty}^{+\infty} x\, N\phi(x)[\Phi(x)]^{N-1}\, dx.$$

By extreme-value theory (the Fisher–Tippett–Gnedenko theorem), standardized normal maxima converge weakly to a Gumbel distribution. Bailey & López de Prado (2014) give the following large-sample analytic approximation using the Euler–Mascheroni constant $\gamma \approx 0.5772$:

$$E[M_N] \approx (1-\gamma)\Phi^{-1}\left(1 - \frac{1}{N}\right) + \gamma\Phi^{-1}\left(1 - \frac{1}{Ne}\right).$$

The CLI output `expected_max_standard_normal_comparison` reports:

- $N = 10$: exact value 1.538753, BLdP approximation 1.574598, difference +0.035846;
- $N = 50$: exact value 2.249074, BLdP approximation 2.276303, difference +0.027229;
- $N = 100$: exact value 2.507594, BLdP approximation 2.530603, difference +0.023009.

The absolute error in these three examples decreases as N grows. The acceptable error depends on the required precision; the approximation must not be described as an exact expectation.

---

7. Financial Interpretation

7.1 Why PSR Is Not the "Probability of Making Money"

Beginners sometimes misread $\mathrm{PSR}(0) = 0.95$ as "there is a 95% probability that this strategy will make money in the future." That is a serious statistical error.

Under i.i.d. returns with a finite fourth moment and an asymptotic normal approximation for the Sharpe estimator, $\mathrm{PSR}(\mathrm{SR}^*)$ is the normal tail area for a threshold comparison. For the one-sided null $H_0 : \mathrm{SR} \leq \mathrm{SR}^*$, at the boundary $\mathrm{SR} = \mathrm{SR}^*$, the corresponding approximate p-value is $1 - \mathrm{PSR}$. Thus PSR can be expressed as a one-sided frequentist confidence level at a prespecified significance level. It is not a posterior probability about a fixed unknown parameter.

PSR measures statistical evidence for the estimated Sharpe relative to a specified threshold; it does not give the probability of a future profit-and-loss path. A strategy with 99% PSR and a small positive expected return can still have a per-period probability of loss close to 50%.

7.2 The Trial Ledger and "Alpha from Nothing"

In the chapter's zero-skill simulation, $M = 200$ independent noise strategies each have $T = 1250$ daily return observations. For seed `20261032`, the champion's annualized Sharpe is 1.2318 and its naive one-sided p-value is 0.002941. Yet the annualized DSR threshold is 1.3257 (daily 0.083843) and DSR is 0.417160. These are results for that simulation and the DSR assumptions; they neither prove nor disprove alpha for any real strategy.

---

8. Numerical Examples

This section compares hand-calculation inputs and reproducible outputs from the CLI result `qa/a6/codex_resume/ch32/results/ch32_results.json`.

Complete worked example: six-period descriptive measures

8.1 Hand-Calculating Descriptive Measures for Six Monthly Observations

Consider a six-month monthly return series ($A = 12$, over $Y = 0.5$ years):

- Strategy returns: $r = [0.04, -0.02, 0.03, -0.01, 0.05, -0.03]$
- Risk-free returns: $r_f = [0.002, 0.002, 0.002, 0.002, 0.002, 0.002]$
- Benchmark returns: $r_B = [0.02, -0.01, 0.01, 0.00, 0.03, -0.01]$
- Initial capital: $W_0 = 1000.0$

```
[Hand calculations]
1. Wealth path [hand_descriptive_metrics.wealth_path]:
   W_0 = 1000.0
```

Continued: the same complete example

```
   W_1 = 1000.0 * 1.04 = 1040.0
   W_2 = 1040.0 * 0.98 = 1019.2
   W_3 = 1019.2 * 1.03 = 1049.776
   W_4 = 1049.776 * 0.99 = 1039.27824
   W_5 = 1039.27824 * 1.05 = 1091.242152
   W_6 = 1091.242152 * 0.97 = 1058.504887

2. CAGR and maximum drawdown [hand_descriptive_metrics.cagr, max_drawdown,
    ↪ calmar_ratio]:
   CAGR = (1058.504887 / 1000.0)^(1 / 0.5) - 1 = (1.058504887)^2 - 1 = 12.0433%
    ↪ (0.120433)
   Historical peaks are set at t=1 (1040), t=3 (1049.78), and t=5 (1091.24).
   The maximum drawdown occurs at t=6, falling from the t=5 peak:
   (1058.504887 - 1091.242152) / 1091.242152 = -3.0000%
   MaxDD = 0.030000 (3.00%)
   Calmar = 0.120433 / 0.03 = 4.014420

3. Volatility and Sharpe ratios [hand_descriptive_metrics.per_period_sharpe,
    ↪ annualized_sharpe]:
   Excess returns r - rf = [0.038, -0.022, 0.028, -0.012, 0.048, -0.032]
   Mean excess return = 0.048 / 6 = 0.008000
   Sample standard deviation (ddof=1) = sqrt(0.005800 / 5) = sqrt(0.001160) =
    ↪ 0.0340588
   Per-period Sharpe = 0.008000 / 0.0340588 = 0.234888
   Annualized Sharpe = 0.234888 * sqrt(12) = 0.813676
   Annualized volatility = std(r) * sqrt(12) = 0.0340588 * sqrt(12) = 0.117983
    ↪ (11.7983%)

4. Sortino ratio (tau=0) [hand_descriptive_metrics.annualized_sortino]:
   Downside returns: [0, -0.02, 0, -0.01, 0, -0.03]
   Sum of squared downside returns = 0.0004 + 0.0001 + 0.0009 = 0.001400
   Full-sample downside deviation DD = sqrt(0.001400 / 6) = 0.0152753
   Per-period Sortino = (0.01 - 0) / 0.0152753 = 0.654654
   Annualized Sortino = 0.654654 * sqrt(12) = 2.267787

5. Hit rate and Profit Factor [hand_descriptive_metrics.hit_rate_strict,
    ↪ profit_factor]:
   Positive-return periods: 3 (t=1, 3, 5); negative-return periods: 3 (t=2, 4,
    ↪ 6)
   Hit rate = 3 / 6 = 50.00% (0.500000)
   Total gains = 0.04 + 0.03 + 0.05 = 0.120000
   Total losses = |-0.02 - 0.01 - 0.03| = 0.060000
   Profit Factor = 0.12 / 0.06 = 2.000000

6. Tracking error and information ratio [hand_descriptive_metrics.tracking_error
    ↪ , information_ratio]:
   Active returns r - rB = [0.02, -0.01, 0.02, -0.01, 0.02, -0.02]
   Mean active return = 0.02 / 6 = 0.0033333
   Sample standard deviation of active returns = sqrt(0.0017333333 / 5) =
    ↪ 0.0186190
   Annualized tracking error TE = 0.0186190 * sqrt(12) = 0.064498 (6.4498%)
```

Continued: the same complete example

```
Annualized IR = (1/300)/sqrt(13/37500)*sqrt(12) = sqrt(5/13); rounded to 6 d.
↪ p.: 0.620174
```

Complete worked example: two-period compounding

8.2 Arithmetic and Geometric Averages at Opposite Extremes (Volatility Drag)

Consider two period returns: $+50\%$ in the first period and -50% in the second (initial capital 1.0):

- Ending capital: $W_2 = 1.0 \times 1.5 \times 0.5 = 0.75$ (a 25% loss);
- Arithmetic average: $(0.50 + (-0.50))/2 = 0.00\%$ (`hand_descriptive_metrics.arithmetic_vs_geometric.arithmetic_mean`);
- Geometric average: $\sqrt{1.5 \times 0.5} - 1 = \sqrt{0.75} - 1 = -13.3975\%$ (`hand_descriptive_metrics.arithmetic_vs_geometric.geometric_mean`).

Financial takeaway: A zero arithmetic average does not mean the account broke even. High volatility can impose a severe compounding drag (volatility drag), so asset allocation should be evaluated using compound geometric growth.

Complete worked example: all six CSCV partitions

8.3 Frozen Hand-Worked CSCV Example (8 Observations × 3 Strategies, S=4)

Under the frozen specification, consider this return matrix for 8 periods and 3 strategies:

$$M = \begin{bmatrix} 0.02 & 0.01 & -0.01 \\ 0.01 & 0.00 & 0.02 \\ -0.01 & 0.02 & 0.00 \\ 0.03 & -0.01 & 0.01 \\ 0.00 & 0.01 & 0.02 \\ -0.02 & 0.03 & 0.01 \\ 0.01 & -0.02 & 0.03 \\ 0.02 & 0.00 & -0.01 \end{bmatrix}$$

Split the rows into four blocks of two: $B_0 = \{0, 1\}$, $B_1 = \{2, 3\}$, $B_2 = \{4, 5\}$, and $B_3 = \{6, 7\}$. Choose $S/2 = 2$ blocks for training. There are $C(4, 2) = 6$ symmetric partitions. The hand calculations for all six are shown below.

The in-sample selection and returns (arrays list strategies 0, 1, and 2 in that order):

Partition c	Training blocks J	In-sample Sharpe by strategy	Best strategy n^*
1	$\{0, 1\}$	$[0.7319, 0.3873, 0.3873]$	**0**
2	$\{0, 2\}$	$[0.1464, 0.9934, 0.7071]$	**1**
3	$\{0, 3\}$	$[2.5981, -0.1987, 0.3638]$	**0**
4	$\{1, 2\}$	$[-0.0000, 0.7319, 1.2247]$	**2**
5	$\{1, 3\}$	$[0.7319, -0.1464, 0.4392]$	**0**
6	$\{2, 3\}$	$[0.1464, 0.2402, 0.7319]$	**2**

Complementary test blocks and the Sharpe ratios for all strategies:

Continued: the same complete example

Partition c	Test blocks $\bar{J}$	Test-set Sharpe by strategy
1	$\{2,3\}$	$[0.1464, 0.2402, 0.7319]$
2	$\{1,3\}$	$[0.7319, -0.1464, 0.4392]$
3	$\{1,2\}$	$[-0.0000, 0.7319, 1.2247]$
4	$\{0,3\}$	$[2.5981, -0.1987, 0.3638]$
5	$\{0,2\}$	$[0.1464, 0.9934, 0.7071]$
6	$\{0,1\}$	$[0.7319, 0.3873, 0.3873]$

Ranks and PBO classifications for each partition:

Partition c	Test rank $\bar{r}^*$ of n^*	Relative rank $\bar{\omega}_c$	Logit λ_c	$\lambda \leq 0$	$\bar{r}^* < 1.5$
1	1.0	0.25	-1.0986	Yes	Yes
2	1.0	0.25	-1.0986	Yes	Yes
3	1.0	0.25	-1.0986	Yes	Yes
4	2.0	0.50	0.0000	Yes	No
5	1.0	0.25	-1.0986	Yes	Yes
6	2.0	0.50	0.0000	Yes	No

Closer look at the threshold conventions:

- In partitions 4 and 6, the selected strategy n^* has rank 2 in the test set (middle of the three candidates). Its relative rank is $\bar{\omega} = 2/(3+1) = 0.50$, so its Logit is $\lambda = \ln(0.5/0.5) = 0.0000$, exactly zero.
- **CSCV integral convention ($\lambda \leq 0$):** All six partitions satisfy $\lambda \leq 0$, so $\text{PBO}_{\text{CSCV}} = 6/6 = 1.000000$ (`pinned_hand_cscv.pbo_cscv`).
- **Strict Definition 2.2 convention ($\bar{r} < N/2 = 1.5$):** Only partitions 1, 2, 3, and 5 have rank 1, below 1.5; partitions 4 and 6 have rank 2. Thus $\text{PBO}_{\text{strict}} = 4/6 \approx 0.666667$ (`pinned_hand_cscv.pbo_strict`).

This module reports both thresholds. For $N = 4$, the strict rule counts only rank 1, while CSCV counts ranks 1 and 2. For $N = 10$, the respective counts are ranks 1–4 and 1–5, so the difference is not limited to odd N. In-sample ties for the best result are rejected by the current API using a tolerance; out-of-sample ties receive average rank. Each half must contain at least two observations, and each candidate must have nonzero variance. PBO describes relative ranking among candidates, not the probability of an absolute loss.

---

9. Python Implementation

The chapter's core algorithms are stored in `src/quant_course/inference.py`; the full CLI tool is `code/ch32_performance_statistics.py`.

From the root of the accepted I3 reader package, run `python run_notebook.py notebooks/ch32_performance_statistics.ipynb`. The Notebook creates a unique `runs/ch32_performance_statistics_<uuid>/` directory, computes a fresh champion

example with one seed and 1,250 observations and 200 candidate strategies, and calculates 252 CSCV partitions; later cells mainly display results already held in the kernel. It is therefore more than a static replay, and the static chapter figures are not new outputs from that Notebook run. French data is skipped by default. The runner also writes an execution copy and `execution.json` to `runs/execution_<stem>_<uuid>/`; failed runs are retained too. The English Notebook and portable packaging/cold-start work belong to BIL23, and are not executed as part of this chapter workflow.

The working CLI still includes French data, multi-seed champion analysis, and AR research and should not be launched just to read this chapter. If separately authorized, use a new output directory and do not overwrite frozen QA results. The following is a local API demonstration.

```
import sys
from pathlib import Path
sys.path.insert(0, "src")

import numpy as np
from quant_course.inference import (
    probabilistic_sharpe_ratio,
    deflated_sharpe_ratio,
    pbo_cscv,
    per_period_sharpe,
)

# 1. Verify the PSR example from Bailey & López de Prado (2012a)
psr_norm = probabilistic_sharpe_ratio(0.458, 0.0, 24, skew=0.0, kurt=3.0)
psr_skew = probabilistic_sharpe_ratio(0.458, 0.0, 24, skew=-2.448, kurt=10.164)
psr_36m = probabilistic_sharpe_ratio(0.458, 0.0, 36, skew=-2.448, kurt=10.164)

assert round(psr_norm, 3) == 0.982
assert round(psr_skew, 3) == 0.913
assert round(psr_36m, 4) == 0.9535
print("PSR paper example verified.")

# 2. Verify the DSR example from Bailey & López de Prado (2014)
sr_daily = 2.5 / np.sqrt(250.0)
var_sr_daily = 0.5 / 250.0
dsr_100, sr0_100 = deflated_sharpe_ratio(sr_daily, var_sr_daily, 100, 1250, skew
    ↪ =-3.0, kurt=10.0)
dsr_46, sr0_46 = deflated_sharpe_ratio(sr_daily, var_sr_daily, 46, 1250, skew=-3.0,
    ↪ kurt=10.0)

assert round(sr0_100, 4) == 0.1132 and round(dsr_100, 4) == 0.9004
assert round(sr0_46, 4) == 0.1004 and round(dsr_46, 4) == 0.9505
print("DSR paper example verified.")
```

10. Visualization

This chapter carries forward four historical figures. For this batch, only the threshold label on the first figure is corrected using the existing six-partition cache; the historical analysis is not rerun. Figures are stored in `figures/a6/`, and each plot identifies its sample size, physical axis units, and theoretical data source.

10.1 Ranks and Logits for the Six Hand-Worked CSCV Partitions

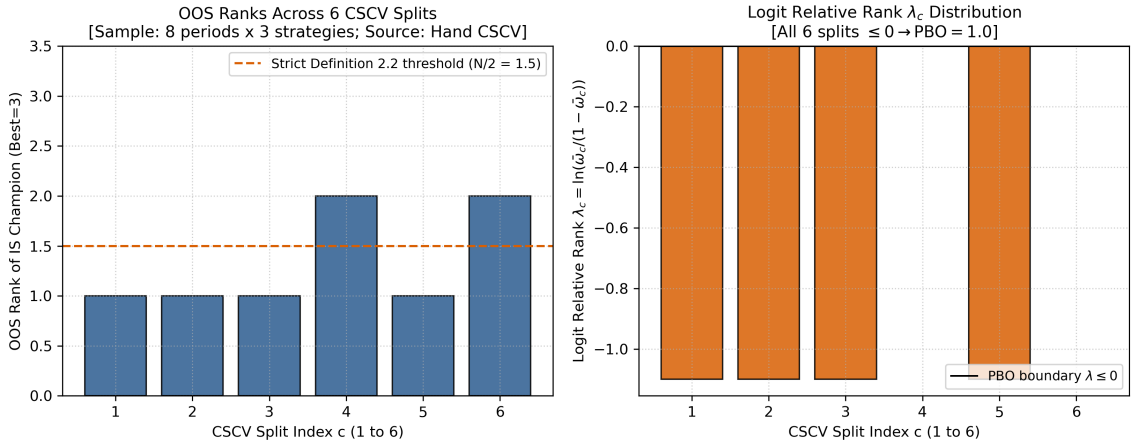


Figure 1: Hand-worked CSCV partitions

- **Left plot:** The horizontal axis is the index c of the six combinatorial partitions; the vertical axis is the test-set rank of the in-sample winning strategy ($N = 3$). The dashed line is the strict Definition 2.2 threshold $N/2 = 1.5$; the actual median rank of three candidates is 2.
- **Right plot:** The horizontal axis is the partition index and the vertical axis is the Logit statistic λ_c. All six bars are ≤ 0 (partitions 4 and 6 are exactly zero), illustrating why PBO = 1.0.

10.2 Non-Normality Penalty in the Probabilistic Sharpe Ratio (PSR)

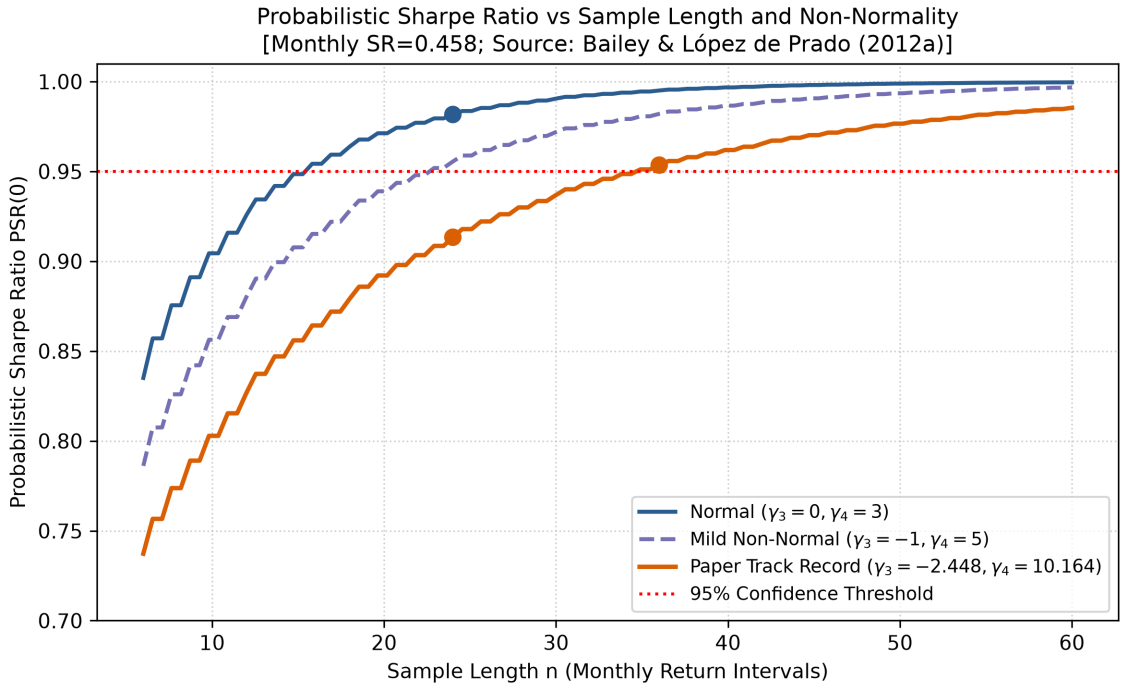


Figure 2: PSR sensitivity curve

- **Horizontal axis:** Sample length n (monthly periods, from 6 to 60 months).
- **Vertical axis:** PSR(0); the red dashed line marks a 95% confidence level.
- **Observation:** For these fixed inputs, normal moments give PSR(0) = 0.982, while negative-skew, heavy-tailed moments give 0.913 at $n = 24$. Increasing the latter sample length to $n = 36$ gives 0.9535. If a decision-maker prespecifies a one-sided 95% threshold, this 36-month example crosses it. This is a sensitivity calculation for specified moments and an observed Sharpe, not a general minimum-sample-size result.

10.3 Penalty for the Number of Trials in the Deflated Sharpe Ratio (DSR)

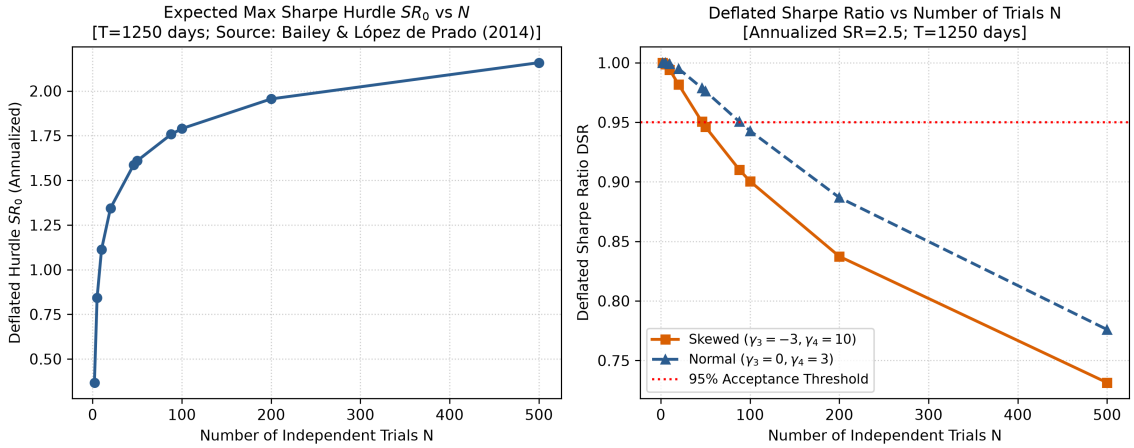


Figure 3: DSR trial-count penalty curve

- **Left plot:** The horizontal axis is the number of independent trials N (shown range 2–500); the vertical axis is the expected extreme-value threshold SR_0 for annualized random Sharpe ratios.
- **Right plot:** The horizontal axis is the trial count N; the vertical axis is the DSR threshold-comparison measure under the stated assumptions.
- **Observation:** The figure reproduces Bailey & López de Prado (2014), Exhibit 2, over the trial-count range 2–500. At the specified points $N = 46$ and $N = 100$, the threshold SR_0 rises. For an annualized Sharpe of 2.5 and the prespecified 0.95 criterion, DSR is 0.9505 at $N = 46$ and 0.9004 at $N = 100$. This conditional approximation does not prove true alpha or general deployment eligibility.

10.4 Comparing the Champion-Trap Experiment Distributions

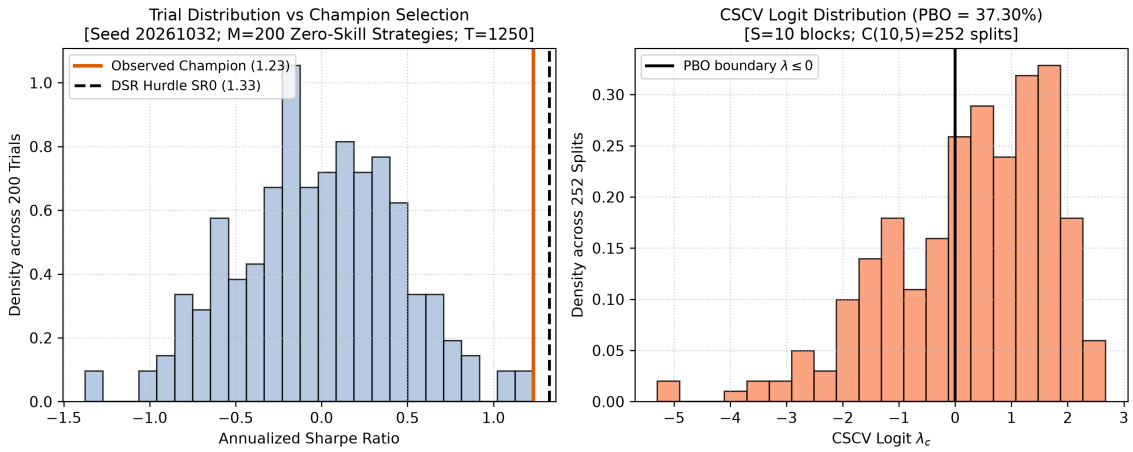


Figure 4: Champion-trap experiment distributions

- **Left plot:** The Sharpe distribution of $M = 200$ pure-noise candidates in one constructed world, with the selected champion marked; this is not a sampling distribution of champions across repeated worlds.
- **Right plot:** The distribution of CSCV Logit values λ_c, describing deterioration in the selected champion's relative out-of-sample rank. A negative Logit does not mean an absolute loss. In the French example in Section 11, PBO is 28.14% while the fraction of out-of-sample losses is 0%, demonstrating the distinction.

---

11. Real-Market Example

11.1 A 100-Year History of Kenneth French's 10 Industry Portfolios

We use `quant_course.french_frozen.load_industries_and_rf` to read the 10 Kenneth French value-weighted industry portfolios and the monthly FF3 risk-free rate. The data snapshot date is 2026-10-05. This is a revised historical series, not point-in-time data, and the industry portfolios are not strategies with transaction records.

To meet the CSCV divisibility requirement, the example takes the latest 1,200 months (1926-09 through 2026-08) and subtracts the monthly FF3 risk-free rate from each industry's total return to obtain excess returns. Set $S = 12$ (100 months per block), yielding all $C(12, 6) = 924$ partitions.

11.2 Disclosure of Empirical-Test Results

The `french_market_experiment` output reports:

1. **Industry excess-return profile.** The table gives the complete CLI result for the final 1,200 months. Annualized arithmetic excess return and volatility are computed from monthly statistics using factors of 12 and $\sqrt{12}$, respectively. Sharpe is calculated from monthly excess returns and then multiplied by $\sqrt{12}$. NoDur and Hlth have the highest annualized excess Sharpe ratios in this sample.

Industry	Annualized excess return	Annualized volatility	Annualized Sharpe
NoDur	8.01%	15.80%	0.5073
Durbl	10.69%	28.59%	0.3740
Manuf	9.01%	21.56%	0.4182
Enrgy	9.30%	22.28%	0.4175
HiTec	10.78%	24.79%	0.4348
Telcm	6.58%	16.16%	0.4072
Shops	9.20%	20.06%	0.4587
Hlth	9.57%	19.07%	0.5020
Utils	7.31%	18.82%	0.3882
Other	7.82%	22.04%	0.3548

2. **CSCV diagnostics.**
   - $\mathrm{PBO}_{\mathrm{CSCV}} = 28.14\%$ (`pbo_cscv = 0.281385`); under the strict-rank convention it is 23.59% (`pbo_strict = 0.235931`).
   - The fraction of out-of-sample losses is 0.00%; mean annualized in-sample/out-of-sample Sharpe is 0.5473/0.4508, with mean decay of 0.0964.
   - The regression slope of out-of-sample on in-sample Sharpe is -0.7975.

Limits on interpretation: These results describe only the revised historical industry portfolios under the selected CSCV partitions. They do not test a timing signal or industry momentum/quality factor, and they do not include trading, fills, fees, or investable capacity. CSCV preserves blocks but does not simulate deployment in chronological order. These statistics cannot establish future returns, tradable alpha, or suitability for launch.

---

12. Common Mistakes

12.1 Error 1: Annualizing Volatility with $\sqrt{A}$ for Serially Correlated Returns

- **Example of the error and why it occurs:** When the full weighted autocorrelation sum of asset returns is positive (as can occur with illiquid assets or smoothed private-equity valuations), one-period variance does not accumulate as if periods were independent. Blindly multiplying by $\sqrt{A}$ can materially understate true annualized portfolio volatility and inflate the annualized Sharpe ratio.

- **Executable comparison:**

```python
import numpy as np

r = np.array([0.010, 0.011, 0.0105, 0.0115, 0.011, 0.012, 0.0115, 0.0125])

# Incorrect: multiply mechanically by sqrt(250)
naive_ann_vol = float(np.std(r, ddof=1) * np.sqrt(250.0))

# Example assumption: returns follow an AR(1) process with known phi=0.3

# Long-run variance inflation includes every lag rho_k = phi**k
phi = 0.3
if not -1.0 < phi < 1.0:
    raise ValueError("AR(1) coefficient must lie strictly between -1 and 1")
inflation = (1.0 + phi) / (1.0 - phi)
adjusted_ann_vol = float(naive_ann_vol * np.sqrt(inflation))
assert inflation > 0.0 and adjusted_ann_vol > naive_ann_vol
```

The exact finite-A variance of a sum is

$$\operatorname{Var}\left(\sum_{t=1}^{A} r_t\right) = A\sigma^2\left[1 + 2\sum_{k=1}^{A-1}(1 - k/A)\rho_k\right].$$

This counts the $A-k$ pairs at lag k. For an AR(1), $\rho_k = \phi^k$; only as a long-span approximation can the infinite sum $1+2\sum_{k\geq 1}\phi^k = (1+\phi)/(1-\phi)$ replace it. The code above uses this long-run approximation, not the exact 250-period formula. For a general process, $\rho_1 > 0$ alone does not determine the sign of the full weighted sum.

- **Numerical difference:** With AR(1) parameter $\phi = 0.3$, the long-run variance inflation factor is $(1 + 0.3)/(1 - 0.3) \approx 1.857$, giving a standard-error inflation factor of 1.363. In empirical work, estimate ϕ on the training sample and report its uncertainty. For a general correlated series, use all relevant autocorrelation lags or a HAC variance estimate; a single sample ρ_1 is not an exact long-run variance correction.

12.2 Error 2: Substituting the Total Number of Strategy Columns M for the Number of Independent Trials N in DSR

- **Example of the error and why it occurs:** If 200 strategies are only small parameter variations, the number of columns may not equal the number of independent searches. Average Pearson correlation is also insufficient to determine the maximum-value distribution. It does not identify an exact "true independent trial count" or guarantee the direction of an adjustment. The following compares the numerical output of one explicitly stated heuristic.

- **Executable comparison:**

```
import math
from quant_course.inference import deflated_sharpe_ratio, implied_independent_trials

# Observed daily Sharpe 0.15, variance 0.002, total trials M=200
m_trials = 200

# Incorrect: treat 200 columns as 200 independent trials
dsr_naive, sr0_naive = deflated_sharpe_ratio(0.15, 0.002, m_trials, 1250)

# Heuristic sensitivity: interpolate N_hat using average correlation rho=0.5
n_hat = implied_independent_trials(m_trials, avg_corr=0.5)
dsr_heuristic, sr0_heuristic = deflated_sharpe_ratio(0.15, 0.002, int(round(n_hat)),
    ↪ 1250)

assert sr0_naive > sr0_heuristic
assert dsr_heuristic > dsr_naive
```

- **Numerical difference:** With average correlation 0.5, the heuristic gives noninteger $\hat{N} = 0.5 + 0.5 \times 200 = 100.5$. The current API accepts integer N, so the example uses Python's `round(100.5) == 100` as an approximation. The API also clips the interpolation to $[1, M]$; a value above M from negative correlation is returned as M. This is an implementation policy, not an extreme-value calibration theorem. The discretization does not change the fact that the formula is a heuristic or imply that its adjusted threshold establishes true alpha.

12.3 Error 3: Confusing the Arithmetic Mean with the Geometric Growth Rate

- **Example of the error and why it occurs:** Treating the simple arithmetic mean of period returns as compound performance ignores volatility drag.
- **Executable comparison:**

```
import numpy as np

# Two-period symmetric returns: +50% and -50%
r_seq = np.array([0.50, -0.50])
arith_mean = float(r_seq.mean())
geom_mean = float(np.sqrt(np.prod(1.0 + r_seq)) - 1.0)

assert arith_mean == 0.0
assert round(geom_mean, 6) == -0.133975
```

- **Numerical difference:** The arithmetic mean is 0.00%, but ending wealth is only 75% of initial capital (a geometric average return of -13.3975%). High volatility erodes compound growth and can hide bankruptcy risk behind an arithmetic-average illusion.

13. Quantitative Research Application

A Complete Performance-Inference Workflow: From Trial Ledger to Capital-Allocation Gate

An auditable quantitative-research workflow can be organized into four steps. Choose thresholds in advance for the decision at hand; PSR, DSR, and PBO do not have a universal launch threshold outside a model and risk preference.

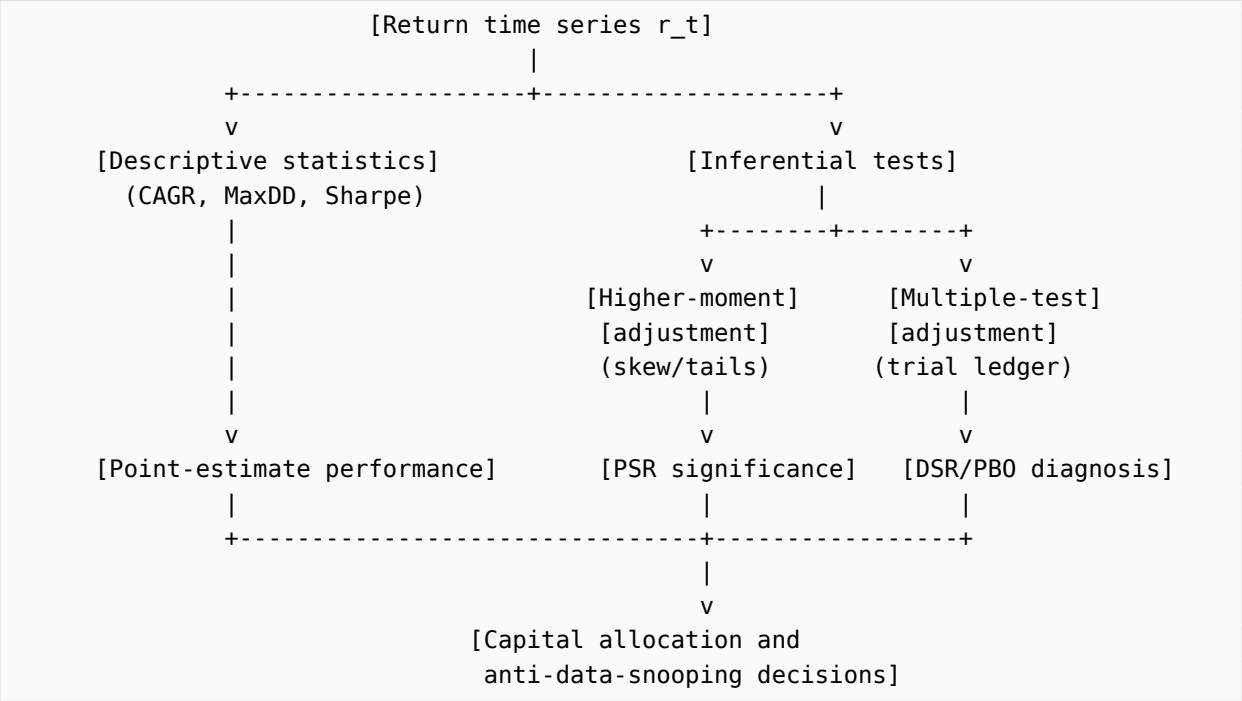
```
[Step 1: Register the trial ledger]
  Record each parameter change and factor addition/removal in the TrialLedger (
    ↪ Chapter 22)
  Record total trials M, sample length T, and evaluation frequency
        ↓
[Step 2: Audit descriptive measures]
  Calculate CAGR, MaxDD, Calmar, volatility, hit rate, and Profit Factor
  Connect to the Chapter 29 ledger to audit turnover and slippage costs
        ↓
[Step 3: Account for non-normality and single-test significance (PSR)]
  Estimate sample skewness gamma3 and kurtosis gamma4
  Calculate PSR(0) at the native frequency and interpret it against a prespecified
    ↪ one-sided significance level
        ↓
[Step 4: Adjust for selection and multiple trials (DSR + CSCV)]
  Record dependence structure; if the effective number of trials is unknown, disclose
    ↪  the unknown and a sensitivity range instead of forcing an exact N_hat
  Calculate the expected maximum random-noise threshold SR0 and report DSR with its
    ↪ assumptions, frequency, and trial count
  Run CSCV with S blocks; report PBO, loss fraction, and partition performance, using
    ↪  thresholds set in advance for the decision
```

The transition from Chapter 32 to Chapter 33 is explicit: PSR, DSR, and PBO determine how to interpret performance evidence; the next chapter uses actual positions to calculate losses, exit capacity, and limits. Strong evidence does not replace risk constraints, and a search diagnostic does not directly generate a sell order.

---

14. Chapter Summary

14.1 Concept Map



14.2 Disclosure Rules for a Rigorous Quantitative Performance Report

A performance report should disclose the information below and select inference methods appropriate to the question, sample, and model conditions. Do not show only the winning curve from a search or force an inference number when required conditions are missing:

1. **Sample period and frequency:** State the start and end dates, return sampling frequency, and annualization factor A.
2. **Higher-moment characteristics:** Report annualized volatility, the bias-corrected sample skewness $\hat{\gamma}_3$ actually used (bias correction does not guarantee strict unbiasedness for every distribution), and raw kurtosis $\hat{\gamma}_4$.
3. **Actual search ledger:** Disclose the total number of models M tried during backtest selection, the parameter space, and the average correlation $\hat{\rho}$.
4. **Inference measures:** When the relevant moment, search, and candidate conditions are met, report PSR(0), threshold SR_0, DSR, or both CSCV PBO conventions. When they are not usable, explain the conditions and missing information.
5. **Trading and capacity constraints:** Use the actual Chapter 29 ledger to disclose average per-period turnover and estimated market-impact costs.

---

15. Exercises

- **32-E01 (Conceptual distinction):** What time-series assumptions underlie the square-root scaling $\widehat{\mathrm{SR}}_{\mathrm{ann}} = \sqrt{A}\,\widehat{\mathrm{SR}}$? Distinguish the roles of i.i.d. returns and normality in this scaling relationship and in classical significance inference. If returns have positive first-order autocorrelation $\rho_1 > 0$, what bias results from applying this formula directly?
- **32-E02 (Mathematical property):** In the PSR in Equation (11), assume the observed per-period Sharpe $\widehat{\mathrm{SR}} > 0$ is fixed. Using derivatives, analyze how the denominator variance term and PSR change when skewness $\hat{\gamma}_3$ decreases (becomes more negative) or kurtosis $\hat{\gamma}_4$ increases (heavier tails).
- **32-E03 (Extreme-value statistics):** Under an independent Gaussian approximation for sample Sharpe ratios, explain why, even when the true expected return is zero, the expected maximum sample Sharpe tends to infinity as the number of trials $N \to \infty$.
- **32-E04 (Algorithm conventions):** In CSCV, suppose there are an odd number of strategies, $N = 3$, and the best strategy has test-set rank 2. State the difference in the conclusion from the strict rank rule in Definition 2.2 of Bailey et al. and the continuous integral estimate in Algorithm 2.3.

---

16. Programming Exercises

- **32-P01 (Sortino and downside deviation):** Write an independent Python function to calculate the Sortino ratio, taking a return series that includes a zero return and a target threshold τ. Verify the full-sample denominator convention using n observations, and raise a controlled `ValueError` when downside deviation is zero.
- **32-P02 (DSR-threshold sensitivity):** Call `quant_course.inference.deflated_sharpe_ratio`. For annualized variance $V = 0.5$, output a table of daily and annualized expected-maximum Sharpe thresholds SR_0 at trial counts $N \in [2, 10, 50, 100, 500]$.
- **32-P03 (Hand-reproducing CSCV for 8 observations):** Use `quant_course.inference.pbo_cscv` on the 8×3 matrix in Section 8.3 and write assertions for the Logit statistic and PBO result in all six partitions.
- **32-P04 (Extracting ledger turnover):** Write a test script that constructs a two-asset rotation ledger with `quant_course.backtest` and accurately calculates the turnover on each rebalance date with `quant_course.inference.turnover_from_ledger`.

---

17. Challenge Problems

- **32-C01 (Deriving the autocorrelation correction):** Derive the asymptotic inflation factor $(1+\phi)/(1-\phi)$ for the sample-mean variance of a stationary AR(1) process $r_t = \phi r_{t-1} + \epsilon_t$, with $|\phi| < 1$, relative to the i.i.d. case. Then use the CLI with a specified seed, sample length, and number of simulations to report the realized type-I error rate of a two-sided 5% normal test that ignores serial dependence, and explain the Monte Carlo sampling error.
- **32-C02 (Limits of the correlated-trial interpolation):** Critique the equal-weight average-correlation interpolation $\hat{N} = \hat{\rho} + (1-\hat{\rho})M$ in Appendix A.3 of the DSR. Give a counterexample showing why nonlinear dependence (such as tail dependence in a copula) can cause this linear formula to understate the true extreme-value risk.
- **32-C03 (Joint Walk-Forward and CSCV diagnostics):** Design a research-review workflow that combines CSCV and a strictly forward, purged Walk-Forward procedure. Explain the question each answers and what additional trading, data, and risk evidence is still required.
- **32-C04 (Heavy tails and the PSR assumption boundary):** Compare the realized one-sided 5% rejection rates for zero-mean standard normal, variance-standardized Student-t_5, and Student-t_4 returns at several values of n, using the sample Sharpe, skewness, and Pearson raw kurtosis in PSR. Explain why the population fourth moment of t_4 does not exist and therefore the finite-fourth-moment assumption for the PSR asymptotic variance formula fails. The simulation describes the behavior of this plug-in rule for the specified finite samples; it does not establish a general sample-size compensation theorem.

---

18. Solutions and Solution Hints

32-E01 — Suggested Answer

1. **Conditions for scaling:** For the sum of per-period excess returns, if the observations are i.i.d. with finite variance, then $E[\sum_{t=1}^{A} r_t] = A\mu$ and $\mathrm{Var}(\sum_{t=1}^{A} r_t) = A\sigma^2$. The Sharpe ratio of the sum is therefore $\sqrt{A}\,\mu/\sigma$. Normality is not required for this square-root scaling; it is an additional distributional assumption for classical normal-based confidence intervals or tests. With compounded simple returns, annual wealth is a product rather than the sum of periodic returns, so square-root annualization is usually a convention or approximation.
2. **Bias from positive autocorrelation:** For a stationary weakly dependent series, the variance of the sample mean scales with the long-run variance. Under an AR(1), the asymptotic factor is $1 + 2\sum_{k=1}^{\infty}\phi^k = (1+\phi)/(1-\phi)$. Positive first-order autocorrelation in an AR(1) makes this factor greater than 1. Using $\sqrt{A}$ anyway understates the standard deviation of an aggregated return and therefore overstates annualized Sharpe. For a general

series, inference should use the full autocorrelation structure or a HAC standard error; ρ_1 alone does not determine the long-run variance.

32-E02 — Suggested Answer

The denominator variance term is

$$D = 1 - \hat{\gamma}_3 \hat{\mathrm{SR}} + \frac{\hat{\gamma}_4 - 1}{4} \hat{\mathrm{SR}}^2.$$

For $\hat{\mathrm{SR}} > 0$:

- $\frac{\partial D}{\partial \hat{\gamma}_3} = -\hat{\mathrm{SR}} < 0$. Thus, as skewness becomes more negative ($\hat{\gamma}_3$ decreases), D increases monotonically.
- $\frac{\partial D}{\partial \hat{\gamma}_4} = \hat{\mathrm{SR}}^2/4 > 0$. Thus, as kurtosis increases (heavier tails), D increases monotonically.

The statistic is

$$Z = \frac{(\hat{\mathrm{SR}} - \mathrm{SR}^*)\sqrt{n-1}}{\sqrt{D}}.$$

If observed Sharpe is above the threshold, an increase in D lowers Z and lowers PSR under the standard normal CDF Φ. If observed Sharpe is below the threshold, the negative Z moves toward zero and PSR rises toward 0.5. If observed Sharpe equals the threshold, PSR is unchanged at 0.5. This calibrates estimation error in a threshold comparison.

32-E03 — Suggested Answer

By the asymptotic theory of standard normal extremes, as $N \to \infty$, the maximum of N independent standard normal variables has the expansion

$$E[\max(Z_1, \ldots, Z_N)] \sim \sqrt{2 \ln N} - \frac{\ln \ln N + \ln(4\pi)}{2\sqrt{2 \ln N}} + \frac{\gamma}{\sqrt{2 \ln N}}.$$

Since $\lim_{N\to\infty} \sqrt{2 \ln N} = +\infty$, the expected maximum noise value grows without bound even when the strategy pool consists entirely of zero-mean white noise. This is a statement about an expectation under the model; it does not guarantee that every finite search produces a specified high value. Nor does the Gaussian approximation make the sample Sharpe ratio's exact distribution Gaussian.

32-E04 — Suggested Answer

For $N = 3$ and $\bar{r}^* = 2$:

1. **Definition 2.2** uses the condition $\bar{r}^* < N/2 = 1.5$. Since $2 \geq 1.5$, this partition is **not counted as overfit**.
2. **Algorithm 2.3** gives relative rank $\bar{\omega} = 2/(3+1) = 0.50$ and Logit $\lambda = \ln(0.5/0.5) = 0$. Its integral convention includes partitions with $\lambda \leq 0$, so this partition **is counted as overfit**.

The two rules use different thresholds: strict $r < N/2$ and CSCV $r \leq (N+1)/2$. At $N = 4$, the first counts only rank 1 and the second counts ranks 1 and 2. At $N = 10$, they count ranks 1–4 and ranks 1–5, respectively. The difference also occurs for even N; choose and disclose the convention before looking at results.

32-P01 — Reference Implementation and Checks

```
import math
from quant_course.inference import sortino_ratio

def independent_sortino(returns, target=0.0, periods_per_year=1.0):
    x = [float(v) for v in returns]
    if not x or not all(math.isfinite(v) for v in x):
        raise ValueError("returns must be nonempty and finite")
    if not math.isfinite(target) or not math.isfinite(periods_per_year) or
    ↪ periods_per_year <= 0:
        raise ValueError("target and annualization must be finite; annualization
    ↪ positive")
    downside = math.sqrt(sum(min(v-target, 0.0)**2 for v in x) / len(x))
    if downside == 0:
        raise ValueError("downside deviation is zero")
    return (sum(x)/len(x)-target) / downside * math.sqrt(periods_per_year)

r = [0.04, 0.0, -0.02, 0.03, -0.01, 0.05, -0.03]
expected_dd = math.sqrt(0.0014 / 7)
expected = (0.06 / 7) / expected_dd * math.sqrt(12)
value = independent_sortino(r, periods_per_year=12)
assert math.isclose(value, expected, rel_tol=1e-12)
assert math.isclose(value, sortino_ratio(r, target_return=0.0, periods_per_year=12),
    ↪ rel_tol=1e-12)
assert not math.isclose(expected_dd, math.sqrt(0.0014 / 3))
for values, target in [([0.0, 0.02], 0.0), (r, math.nan), (r, math.inf), (r, -math.
    ↪ inf)]:
    try:
        independent_sortino(values, target)
    except ValueError:
        pass
    else:
        raise AssertionError("undefined/nonfinite case must fail")
print(value, expected_dd)
```

The zero-return observation remains in the full-sample denominator $n = 7$; the denominator must not be changed to 3, the number of negative returns. For a nonzero target, subtract the

same-frequency target from each observation first. Undefined cases should raise an error. The non-finite-target guard in the shared API is checked separately by a direct test of the candidate implementation.

32-P02 — Reference Implementation and Checks

```
import sys
from pathlib import Path
sys.path.insert(0, "src")

import math
import numpy as np
from quant_course.inference import deflated_sharpe_ratio

var_daily = 0.5 / 250.0
sr_daily = 2.5 / math.sqrt(250.0)
expected_sr0_ann = np.array([0.367523, 1.113409, 1.609589, 1.789406, 2.158463])
sr0_ann = []

for n in [2, 10, 50, 100, 500]:
    dsr, sr0 = deflated_sharpe_ratio(sr_daily, var_daily, n, 1250, skew=-3.0, kurt
    ↪ =10.0)
    sr0_ann.append(sr0 * math.sqrt(250))
    print(f"N={n:3d} -> daily SR0={sr0:.4f}, annualized SR0={sr0*math.sqrt(250):.4f},
    ↪  DSR={dsr:.4f}")
np.testing.assert_allclose(sr0_ann, expected_sr0_ann, rtol=0.0, atol=1e-6)
print("32-P02 tests passed.")
```

32-P03 — Reference Implementation and Checks

```
import sys
from pathlib import Path
sys.path.insert(0, "src")

import numpy as np
from quant_course.inference import pbo_cscv

rows = [
    [0.02, 0.01, -0.01],
    [0.01, 0.00, 0.02],
    [-0.01, 0.02, 0.00],
    [0.03, -0.01, 0.01],
    [0.00, 0.01, 0.02],
    [-0.02, 0.03, 0.01],
    [0.01, -0.02, 0.03],
    [0.02, 0.00, -0.01]
]
res = pbo_cscv(rows, n_blocks=4)
assert res.n_splits == 6
assert res.pbo_cscv == 1.0
assert round(res.pbo_strict, 4) == 0.6667
expected_best = [0, 1, 0, 2, 0, 2]
```

```
expected_ranks = [1.0, 1.0, 1.0, 2.0, 1.0, 2.0]
expected_omega = [0.25, 0.25, 0.25, 0.50, 0.25, 0.50]
expected_lambdas = [-1.098612, -1.098612, -1.098612, 0.0, -1.098612, 0.0]
np.testing.assert_array_equal([p.best_strategy_index for p in res.partitions],
    ↪ expected_best)
np.testing.assert_allclose([p.oos_rank_best for p in res.partitions], expected_ranks,
    ↪  rtol=0.0, atol=1e-12)
np.testing.assert_allclose([p.relative_rank_omega for p in res.partitions],
    ↪ expected_omega, rtol=0.0, atol=1e-12)
np.testing.assert_allclose([p.logit_lambda for p in res.partitions], expected_lambdas
    ↪ , rtol=0.0, atol=1e-6)
print("32-P03 tests passed.")
```

32-P04 — Reference Implementation and Checks

```
import sys
from pathlib import Path
sys.path.insert(0, "src")

import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest
from quant_course.inference import turnover_from_ledger

dates = pd.date_range("2026-01-05", periods=4, freq="B")
mark = pd.DataFrame({"A": [10.0, 11.0, 12.1, 12.0], "B": [20.0, 19.0, 19.0, 21.0]},
    ↪ index=dates)
execp = pd.DataFrame({"A": [10.0, 10.0, 12.0, 12.0], "B": [20.0, 20.0, 19.5, 20.0]},
    ↪ index=dates)
tw = pd.DataFrame({"A": [0.5, 0.5], "B": [0.3, 0.3]}, index=[dates[0], dates[2]])
cfg = BacktestConfig(initial_cash=1000.0, fee_rate=0.002, lag=1)

res = run_backtest(mark, tw, cfg, exec_prices=execp)
turnover = turnover_from_ledger(res.ledger)

assert turnover.iloc[0] == 0.0
assert round(turnover.iloc[1], 4) == 0.7987
assert turnover.iloc[2] == 0.0
assert round(turnover.iloc[3], 4) == 0.0728
print("32-P04 tests passed.")
```

P04 carries forward the accepted BIL15A ledger: closing signals on dates[0] and dates[2] execute on dates[1] and dates[3], respectively, at the specified execution prices. Turnover uses pre-trade NAV as its denominator and total buy and sell notional as its numerator; the 0.002 fee rate is applied to traded notional in the self-financing ledger. Closing weights on trade dates may differ from target weights because of prices and fees, so target-weight changes cannot replace ledger turnover.

32-C01 — Challenge Argument and Rubric

- **Suggested argument:** For an AR(1) process, the autocovariance function is $\gamma_k = \frac{\sigma_\epsilon^2}{1-\phi^2}\phi^{|k|}$. The variance of the full sum is

$$\operatorname{Var}\left(\sum_{t=1}^{T} r_t\right) = T\gamma_0 + 2\sum_{k=1}^{T-1}(T-k)\gamma_k.$$

  As T becomes large, the inflation in the variance of the sample mean relative to the i.i.d. case is

$$1 + 2\sum_{k=1}^{\infty}\phi^k = \frac{1+\phi}{1-\phi}.$$

  At $\phi = 0.5$, the factor is 3, so the standard error is $\sqrt{3} \approx 1.732$ times the i.i.d. standard error.

  The CLI's `boundary_experiments.ar1_dependence` uses seed `20261032`, $T = 1000$, 2,000 simulated series, a two-sided normal z-test, and nominal $\alpha = 0.05$. At $\phi = 0.5$, the test that ignores dependence has a simulated rejection rate of $487/2000 = 24.35\%$ (the zero-correlation baseline is $87/2000 = 4.35\%$). This quantifies undercoverage in this finite simulation. Using a true rejection rate near 0.2435, the binomial Monte Carlo standard error is about 0.0096. Do not treat the result from one simulation as an exact population probability.
- **Rubric:**
  - Show the geometric-series derivation completely (40 points).
  - Give the closed form $(1+\phi)/(1-\phi)$ and explain its meaning (30 points).
  - Interpret the numerical result through type-I-error inflation or confidence-interval undercoverage (30 points).

32-C02 — Challenge Argument and Rubric

Let X take values -3, -1, 1, and 3 with probability $1/4$ each, so its mean is 0 and variance is 5. Construct three joint distributions with the same complete marginals:

1. **Joint distribution with the larger maximum:** First choose a mapping independently: with probability $5/8$, use the adjacent swap $g = (-1, -3, 3, 1)$; with probability $3/8$, use the reversal $h = -X$. Then let Y be the chosen mapping applied to X. Its joint probability matrix (rows for X and columns for Y, each ordered as -3, -1, 1, 3) is

$$P_H = \frac{1}{32}\begin{pmatrix} 0 & 5 & 0 & 3 \\ 5 & 0 & 3 & 0 \\ 0 & 3 & 0 & 5 \\ 3 & 0 & 5 & 0 \end{pmatrix}.$$

   Every row and column sums to $8/32 = 1/4$. Under the adjacent swap, $E[Xg(X)] = (3+3+3+3)/4 = 3$; under reversal, $E[X(-X)] = -5$. The mixture covariance is therefore $(5/8)3 + (3/8)(-5) = 0$, so the Pearson correlation is zero. But $E[\max(X, g(X))] = (-1-1+3+3)/4 = 1$, while the maximum under reversal has expectation $E|X| = 2$. Therefore $E[\max(X, Y)] = 5/8 + 6/8 = 11/8$.

2. **Independent joint distribution:** Each of the 16 cells has probability 1/16. The maximum has probabilities $(1, 3, 5, 7)/16$ at the four support points, so its expectation is $(-3-3+5+21)/16 = 5/4$; covariance is zero.
3. **Joint distribution with the smaller maximum:** Choose $Y = X$ or $Y = -X$ with probability 1/2 each. Each of the four cells associated with either mapping has probability 1/8, and the marginals remain uniform. Covariance is $(5-5)/2 = 0$. The maximum has expectation $\frac{1}{2}E[X] + \frac{1}{2}E|X| = 1$.

For $M = 2$ and average correlation 0, the interpolation gives $\hat{N} = 2$ in all three cases but cannot distinguish expected maxima of 11/8, 5/4, and 1. The independent model understates the maximum by 1/8 in the first example and overstates it by 1/4 in the third. Nonlinear dependence alone does not determine the direction. This is a teaching counterexample with a finite four-point marginal; it is not an exact Gaussian DSR calibration. Pearson correlation measures linear association for any variables with finite second moments, not only elliptically distributed variables, but does not characterize the full joint tail. When $M > T$, a demeaned sample matrix has rank at most $T - 1$, so its correlation matrix can be singular. PCA or mutual-information dimensions can describe dependence, but are not exact trial counts without extreme-value calibration.

Rubric: 40 points for the complete marginals, joint probabilities, and moment calculations; 40 points for the three expected maxima and direction comparisons; 20 points for explaining the marginal model and DSR applicability boundary.

32-C03 — Challenge Argument and Rubric

- **Suggested argument:** CSCV is a symmetric, block-based diagnostic of backtest-selection bias. It uses the relative rank of candidate strategies on test blocks to measure whether the selected candidate falls below the median. It does not simulate deployment in chronological order and therefore cannot by itself test temporal progression, regime change, or causal availability. A Purged Walk-Forward procedure designed for the research question can provide a chronological out-of-sample evaluation; the two methods answer different questions. Freeze research rules and the search ledger first, use CSCV to examine selection stability, then use rolling forward windows to examine performance out of time, applying purge/embargo according to label overlap. Neither procedure guarantees future profits, executable trading, or live-trading suitability.
- **Rubric:**
  - Explain the difference between CSCV's symmetric block diagnostic and Walk-Forward's causal forward deployment logic (50 points).
  - Construct a research and risk-review workflow that combines them (50 points).

32-C04 — Challenge Argument and Rubric

- **Suggested argument and reproducible simulation:** A t_4 distribution has finite variance but an infinite population fourth moment. Therefore, the PSR asymptotic variance formula, which assumes a finite fourth moment, has no established calibration guarantee for t_4. The code below uses variance-standardized returns from each degrees-of-freedom distribution under a zero-mean null, estimates sample Sharpe, skewness, and Pearson raw kurtosis, and counts each simulation with $\mathrm{PSR}(0) > 0.95$ as a rejection. This experiment compares the plug-in rule in finite samples; its rejection rates are not interpreted as a valid significance level under t_4.

```
import numpy as np
from scipy import stats

rng = np.random.default_rng(20261007)
reps = 20_000
observed_counts = []
for n in [24, 60, 120, 240]:
    for label, df in [("Normal", None), ("t5", 5), ("t4", 4)]:
        x = rng.normal(size=(reps, n)) if df is None else rng.standard_t(df, size=(
    ↪ reps, n)) * np.sqrt((df - 2) / df)
        sr = x.mean(axis=1) / x.std(axis=1, ddof=1)
        g3 = stats.skew(x, axis=1, bias=False)
        g4 = stats.kurtosis(x, axis=1, fisher=False, bias=False)
        d = 1 - g3 * sr + (g4 - 1) * sr**2 / 4
        assert np.all(d > 0)
        psr = stats.norm.cdf(sr * np.sqrt(n - 1) / np.sqrt(d))
        rejected = int(np.count_nonzero(psr > 0.95))
        observed_counts.append(rejected)
        print(n, label, rejected, rejected / reps)
np.testing.assert_array_equal(observed_counts, [935, 1267, 1280, 968, 1116, 1191,
    ↪ 1002, 1109, 1158, 1024, 1039, 1096])
```

The results for seed `20261007` are below. Within each n, the code generates Normal, t_5, and t_4 samples in that order:

n	Normal rejection rate	t_5 plug-in rejection rate	t_4 plug-in rejection rate
24	4.675% (935/20000)	6.335% (1267/20000)	6.400% (1280/20000)
60	4.840% (968/20000)	5.580% (1116/20000)	5.955% (1191/20000)
120	5.010% (1002/20000)	5.545% (1109/20000)	5.790% (1158/20000)
240	5.120% (1024/20000)	5.195% (1039/20000)	5.480% (1096/20000)

The population raw kurtosis of t_5 is 9, and its fourth moment is finite; the population raw kurtosis of t_4 does not exist. Sample kurtosis is always a finite number in these simulations, but that does not restore the population moment condition for t_4. Values near 5% and their changes with n describe only this Monte Carlo configuration; they do not give a general "number of months required for calibration" threshold.

- **Rubric:**
  - Explain the fourth-moment boundary between t_4 and t_5 and the applicability conditions for the PSR asymptotic approximation (50 points).
  - Reproduce the rejection rates, identify Monte Carlo error, and avoid extrapolating them into a general sample-size threshold (50 points).

---

19. Further Reading

1. **Must know**
   - Bailey, D. H., Borwein, J., López de Prado, M., & Zhu, J. Q. (2015). "The Probability of Backtest Overfitting." SSRN Working Paper. **Source reading note:** Sections 2.1, 2.2, 3.1, and 4 are identified as read in the source manuscript.
   - Bailey, D. H., & López de Prado, M. (2014). "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality." *Journal of Portfolio Management*, 40(5). **Source reading note:** The relevant portions of the local author manuscript dated 2014-07-31 are identified as read; it was not compared page by page with the final journal version.
   - Bailey, D. H., & López de Prado, M. (2012a). "The Sharpe Ratio Efficient Frontier." *Journal of Risk*, 15(2). **Source reading note:** The relevant portions of Equations 8–11 in the local author manuscript dated 2012-07 are identified as read; it was not compared page by page with the final journal version.
2. **Should know**
   - Lo, A. W. (2002). "The Statistics of Sharpe Ratios." *Financial Analysts Journal*, 58(4), 36–52. **Source reading note:** Not read; listed as a classic reference for exact Sharpe-ratio corrections under serial dependence.
   - Mertens, E. (2002). "Variance of the IID estimator in Lo (2002)." Working paper, University of Basel. **Source reading note:** Not read; listed for the non-normal i.i.d. return asymptotic variance expansion.
3. **Advanced**
   - Harvey, C. R., Liu, Y., & Zhu, H. (2016). "…and the Cross-Section of Expected Returns." *Review of Financial Studies*, 29(1), 5–68. **Source reading note:** Not read; listed for multiple-testing and factor-discovery thresholds in cross-sectional asset pricing.
   - Harvey, C. R., & Liu, Y. (2014). "Backtesting." Working paper, Duke University. **Source reading note:** Not read; listed for sequential testing and double multiple-testing control.
4. **Optional**
   - Opdyke, J. D. (2007). "Comparing Sharpe Ratios: So Where's the Catch?" *Journal of Risk*, 9(4), 31–60. **Source reading note:** Not read; listed for testing under broad stationarity and ergodicity assumptions.

20. References

- [**PBO**] Bailey, D. H., Borwein, J., López de Prado, M., & Zhu, J. Q. (2015). "The Probability of Backtest Overfitting." SSRN Working Paper 2326253.
- [**DSR**] Bailey, D. H., & López de Prado, M. (2014). "The Deflated Sharpe Ratio: Correcting for Selection Bias, Backtest Overfitting and Non-Normality." *Journal of Portfolio Management*, 40(5), 94–107.
- [**PSR**] Bailey, D. H., & López de Prado, M. (2012a). "The Sharpe Ratio Efficient Frontier." *Journal of Risk*, 15(2), 13–44.
- [**KENNETH_FRENCH**] French, K. R. (2026). "Data Library: 10 Industry Portfolios." Dartmouth Tuck School of Business.

Chapter 33 — Risk Management

Main thread: Turn portfolio positions into a loss distribution with an explicit direction, unit, horizon, and set of conditions. Identify which exposures generate losses, which scenarios amplify them, and how the risk constraint feeds into the next executable target weights.

Prerequisites: Chapter 15 on factor models, Chapter 18 on volatility, Chapter 21 on portfolio exposures, Chapter 29 on the ledger, Chapter 30 on bias, Chapter 31 on validation, and Chapter 32 on inference. After this chapter, you should be able to define VaR and ES, recompute a discrete tail, distinguish sources of risk, revalue actual ledger positions, set liquidity and margin constraints, and correctly interpret a VaR backtest.

1. Learning Objectives

After completing this chapter, you should be able to start from explicit positions and a return frequency, define the loss variable, and consistently state both the confidence level and holding horizon. You should be able to define Value-at-Risk (VaR) as a quantile and Expected Shortfall (ES) by quantile integration and threshold-atom probability; explain when ES differs from a strict-tail or inclusive-threshold conditional mean; compare historical, normal, Student-t, and Monte Carlo estimates on the same portfolio; report net exposure, gross exposure, leverage, concentration, factor exposure, liquidation days, and margin separately; apply scenario shocks to the actual positions in Chapter 29; calculate VaR exceedances using data outside the training window and explain the limits of the Kupiec unconditional coverage test; and finally convert risk limits into next-period target weights that Chapter 29 can execute.

The completion standard is not memorizing a "safe number." You should be able to recompute outputs from inputs, identify where parameters came from, distinguish constructed scenarios from empirical data, check when information became available, and state what a metric cannot answer. Numerical examples in the body can be found in the CLI artifact `ch33_results.json`; each example lists its corresponding JSON keys for item-by-item reconciliation.

2. Why Quant Needs This

Chapter 29 already converts target weights into a NAV ledger with signal dates, delayed execution, fees, cash, and positions. Chapter 30 explains that an incorrect timestamp or execution assumption can inflate NAV without causing a program error. Chapter 31 requires model selection and final evaluation to be kept separate, while Chapter 32 cautions against treating a strong return curve as direct evidence of future performance. This chapter carries those lessons forward to a more specific question: how much could the positions already generated lose over a stated horizon? Could the loss come from direction, concentration, common factors, tails, liquidity, or financing constraints?

Researchers often reduce risk to volatility and multiply it by a rule-of-thumb factor. That can miss skewness, fat tails, jumps, concentrated positions, changing correlations, gaps, trading

impact, and margin. A model's low VaR does not mean its extreme losses are mild. Nor do two portfolios with the same variance necessarily have the same ES or liquidation path. Risk management must also reach beyond the report: if a stress loss exceeds the limit, target positions should be reduced; target weights can be executed only after the signal exists and at a permitted execution time, never by writing them back to prices that have already occurred.

This chapter uses three kinds of evidence that must not be conflated. Probability distributions and parameterized models are assumptions. Simulations with fixed inputs and random seeds are reproducible teaching constructions. The frozen industry returns are an actually published historical series, but they are revised, backfilled data. None is presented as a complete answer to portfolio risk.

3. Intuition

Start with a discrete loss variable. Record gains as negative losses. Possible outcomes are losses of -10, 0, 5, and 20, with probabilities 0.50, 0.30, 0.15, and 0.05. Let the confidence level be $\alpha = 0.90$. The cumulative distribution reaches 0.50, 0.80, 0.95, and 1.00 at these outcomes, respectively, so the first loss at which cumulative probability reaches α is 5. Under the lower empirical definition, this is VaR.

ES asks a harder question: what is the average loss over the worst tail interval? The tail probability is $1 - \alpha = 0.10$. The threshold 5 accounts for probability mass 0.15, but the worst tail needs only 0.05 of it. The threshold 20 contributes another 0.05. The weighted tail average is $(5 \times 0.05 + 20 \times 0.05)/0.10 = 12.5$. The conditional mean strictly above VaR is only 20; the conditional mean at or above VaR includes all probability mass at 5 and gives 8.75. Neither is the required mean over the worst 10% tail.

This example introduces the chapter's central idea: VaR is a quantile, while ES averages over an interval of quantiles. When a distribution has an atom at the threshold, only the portion of threshold mass needed to fill the tail belongs in the calculation. The CLI expands this input into 20 equally weighted observations and saves the CDF value, VaR, ES, strict-threshold mean, and inclusive-threshold mean. See the full quantile step function and tail area in `ch33_discrete_quantile_es.png`.

JSON comparison: `pinned_discrete_case.loss_values`, `probabilities`, `alpha`, `scenario_count`, `cdf_at_var`, `var`, `es`, `strict_tail_conditional_mean`, and `inclusive_tail_conditional_mean`.

4. Mathematical Foundations

Let V_t denote portfolio value at time t, and let h denote the holding horizon. Define monetary loss as

$$L_{t,h} = -(V_{t+h} - V_t).$$

Thus $L > 0$ means a loss and $L < 0$ means a gain. If using simple returns rather than monetary amounts, write the return loss explicitly as $L^R_{t,h} = -R_{t,h}$, where $R_{t,h} = (V_{t+h} - V_t)/V_t$. Monetary loss is measured in the account's currency; return loss is a dimensionless proportion. Do not compare a USD amount and a decimal return in the same column.

Every calculation in this chapter must state both α and h. For example, "VaR is 0.10" is incomplete: the reader does not know whether this is 10% of NAV, USD 0.10, or a one-month or one-day loss. The CLI's constructed portfolio uses monthly simple returns to generate negative-return losses. The ledger stress example uses positions in shares, prices in USD per share, and per-asset shocks over the same stated simple-return interval; execution occurs on the next ledger date after the signal. In an application, replace these frequencies and times with the strategy's actual trading frequency.

4.1 Dimensional Ledger for Risk Inputs

Object	Unit or frequency	Conditions to fix before substitution
$L_{t,h}$, monetary VaR/ES	Account currency, same h	Positive means loss; valuation object and horizon match
$L^R = -R$, return VaR/ES	Dimensionless, same h	State the NAV denominator; do not compare directly with currency amounts
α	Dimensionless confidence level	Report with h; 99% daily is not 95% monthly
q_i, P_i, r_i	Shares, currency/share, dimensionless	$q_i P_i r_i$ is monetary PnL; take its negative for loss
$w_i = q_i P_i / NAV$, cash weight	Dimensionless	Normalize by NAV at the same time; list cash separately and do not divide it by NAV again
ADV_i, π_i, D_i	Shares/trading day, dimensionless, trading days	$\lvert q_i \rvert / (\pi_i ADV_i)$ is a fixed-capacity approximation

With positive NAV and no external cash flows over the same valuation interval, $L_{t,h} = V_t L^R_{t,h}$. Under scaling by a fixed positive number, VaR and ES scale by V_t as well. If the NAV denominator or cash-flow definition differs, rebuild the return ledger first; do not apply this conversion directly.

Let $F_L(\ell) = P(L \le \ell)$ be the loss CDF. In a discrete distribution, probability mass can be concentrated at a single loss value. The quantile definition uses "at least reaches," not linear interpolation. If a sample contains ties, the empirical quantile may also fall on an atom. A continuous model usually has no atom near its quantile, but that continuity assumption must not be silently carried over to a discrete ledger or a small empirical sample.

5. Formal Definitions

At confidence level $\alpha \in (0,1)$, define VaR as the lower quantile of the loss distribution:

$$\mathrm{VaR}_\alpha(L) = \inf\{\ell \in \mathbb{R} : F_L(\ell) \geq \alpha\}.$$

This is a loss threshold. It does not describe how severe losses beyond the threshold are, and it does not mean that this loss "will never be exceeded." If a day's loss exceeds VaR, that is an exceedance. When exceedance rates are used in backtesting, the theoretical probability equals $1-\alpha$ directly only for a continuous predictive distribution or when a consistent randomization rule has been specified for threshold atoms.

This chapter defines ES as the average of the quantile function over the upper-tail confidence interval:

$$\mathrm{ES}_\alpha(L) = \frac{1}{1-\alpha}\int_\alpha^1 \mathrm{VaR}_u(L)\,du.$$

If $v = \mathrm{VaR}_\alpha(L)$, the same quantity can be written as

$$\mathrm{ES}_\alpha(L) = \frac{E[L\mathbf{1}\{L > v\}] + v\,[P(L \leq v) - \alpha]}{1-\alpha}.$$

Right-tail ES requires the positive part of L to be integrable, $E[L^+] < \infty$; otherwise, the tail mean may not exist or may be infinite. A finite empirical sample has a finite empirical mean, but inferring population ES from a finite sample still depends on tail assumptions and sample size. The quantile-integral definition and threshold-atom adjustment used here are consistent with Acerbi and Tasche; the historical A6/C15 page-level reading scope and notation conversion are recorded in `source_notes.md`.

6. Derivation

In the discrete example, the quantile function $u \mapsto \mathrm{VaR}_u$ is a step function. Split the tail integral over intervals where the quantile is constant, then divide by the tail length $1-\alpha$ to obtain ES. For a distribution with a point mass, integrate only over the part of the threshold's quantile interval that is actually covered by the tail. No extra assumption that the threshold lies exactly on a probability-grid boundary is needed.

Now consider probability mass. The mass strictly above threshold v is $1 - F_L(v)$. The tail needs total mass $1-\alpha$, so some of the cumulative mass at or below the threshold does not belong to that tail. The difference is $F_L(v)-\alpha$; multiplying it by v supplies exactly the missing threshold contribution. Add that contribution to the total loss contribution strictly above the threshold and divide by the tail probability to obtain the formula in the previous section. For a continuous distribution, $P(L = v) = 0$, and at an ideal threshold $F_L(v) = \alpha$, so the adjustment vanishes. For a discrete distribution, the adjustment generally cannot be omitted.

For equally weighted observations $L_1, \ldots, L_n$, first sort $L_{(1)} \leq \cdots \leq L_{(n)}$, and let $k = \lceil n\alpha \rceil$. The empirical VaR is $L_{(k)}$. Let $v = L_{(k)}$, let $S_>$ be the sum of sample losses strictly greater than v, and let $m_\leq$ be the number of sample values at or below v. Then

The mathematical ceiling is defined over exact real numbers. In implementation, do not rely directly on floating-point multiplication `alpha * n` followed by a ceiling: `0.14 * 100` is slightly greater than 14, although empirical cumulative probability `14 / 100` has already reached 0.14. The core function searches directly for the first order satisfying `k / n >=alpha`; regression tests also check that a probability just above 0.14 selects the next order.

$$\widehat{\mathrm{ES}}_\alpha = \frac{S_>/n + v(m_\le/n - \alpha)}{1-\alpha}.$$

This fractionally weights the sample mass at the threshold. Before computing, validate that the input is a nonempty finite real vector and that α lies strictly between 0 and 1; otherwise, sorting, proportions, or the denominator have no intended meaning. Convergence of empirical ES to population ES requires tail integrability and suitable sampling conditions; with few observations, the mean of the worst observations remains noisy. Acerbi and Tasche's Proposition 4.1 discusses the limit of averages of tail order statistics for independent and identically distributed samples. This does not eliminate finite-sample error or prove that a financial time series is i.i.d.

6.1 Tail Integrals for Continuous Models

For continuous $L = \mu + sZ$, $s > 0$, let z be the α-quantile of Z. The tail probability is $1-\alpha$, and

$$ES_\alpha = \frac{1}{1-\alpha}\int_z^\infty (\mu + sx) f_Z(x)dx = \mu + \frac{s}{1-\alpha}\int_z^\infty x f_Z(x)dx.$$

The standard normal density satisfies $\phi'(x) = -x\phi(x)$, so

$$\int_z^\infty x\phi(x)dx = [-\phi(x)]_z^\infty = \phi(z).$$

If $L \sim N(\mu, \sigma^2)$, and $z_\alpha = \Phi^{-1}(\alpha)$, then

$$\mathrm{VaR}_\alpha = \mu + \sigma z_\alpha, \qquad \mathrm{ES}_\alpha = \mu + \sigma\frac{\phi(z_\alpha)}{1-\alpha}.$$

If instead T_ν is a location-0, scale-1 Student-t random variable (not a unit-variance variable), with standardized density f_ν and quantile $t_{\nu,\alpha}$, then after location shift and scaling by s,

$$\mathrm{VaR}_\alpha = \mu + st_{\nu,\alpha}, \qquad \mathrm{ES}_\alpha = \mu + s\frac{\nu + t_{\nu,\alpha}^2}{\nu-1}\frac{f_\nu(t_{\nu,\alpha})}{1-\alpha}.$$

Write $f_\nu(x) = c_\nu(1 + x^2/\nu)^{-(\nu+1)/2}$. Differentiate $g(x) = (1 + x^2/\nu)^{-(\nu-1)/2}$:

$$g'(x) = -\frac{\nu-1}{\nu}x(1 + x^2/\nu)^{-(\nu+1)/2}.$$

When $\nu > 1$, g tends to zero at positive infinity, so

$$\int_z^\infty x f_\nu(x)dx = \frac{c_\nu\nu}{\nu-1}g(z) = \frac{\nu + z^2}{\nu-1}f_\nu(z).$$

Substituting into the continuous tail integral gives the formula above; $\nu > 1$ ensures an integrable tail mean. Only if σ is interpreted as a standard deviation and the variance is matched do we need $\nu > 2$. These are distinct conditions.

The Student-t mean and ES exist when degrees of freedom exceed 1; variance is finite only when degrees of freedom exceed 2. This chapter passes a mean and standard deviation, so the implementation requires $\nu > 2$ and uses $s = \sigma\sqrt{(\nu-2)/\nu}$ to preserve the standard deviation. SciPy's loc and scale parameters describe location and scale; scale itself is not the Student-t standard deviation. Do not apply the familiar normal ES formula directly to a t distribution.

6.2 Kupiec Unconditional Coverage: From Likelihood to Test

Let $I_t = \mathbf{1}\{L_t > VaR_{t,\alpha}\}$. Under the null, assume these indicators are i.i.d. Bernoulli(q), where $q = 1-\alpha \in (0,1)$. In n periods with x exceedances, the likelihood of the specified ordered sequence is $L(p) = p^x(1-p)^{n-x}$; if only the count is used, multiplying by the binomial coefficient does not change the likelihood ratio because it cancels.

For $0 < x < n$, setting $\ell'(p) = x/p - (n-x)/(1-p) = 0$ gives $\hat{p} = x/n$; the second derivative is negative, so this is the maximum. With zero or all exceedances, the maximum is at $p = 0$ or $p = 1$, respectively. Comparing fixed q with free p,

$$\Lambda = \frac{q^x(1-q)^{n-x}}{(x/n)^x(1-x/n)^{n-x}},$$

$$LR_{uc} = -2\log\Lambda = 2\left[x\log\frac{x/n}{q} + (n-x)\log\frac{1-x/n}{1-q}\right].$$

Use $\lim_{u\downarrow 0} u\log u = 0$ to define zero-count log terms. When $x = 0$, $LR = -2n\log(1-q)$; when $x = n$, $LR = -2n\log q$; both are finite. The commonly used χ^2_1 survival function gives an asymptotic p-value; few exceedances or a boundary count do not guarantee exact small-sample calibration. Independence as Bernoulli, a continuous predictive distribution or atom randomization, threshold-estimation uncertainty, and overlapping holding periods are separate conditions to check. Splitting training and test windows does not itself establish independence. The 1,500-period/74-exceedance example in the body, the 1,082-period/5-exceedance French example, and the 250-period/0-exceedance Loom example are three different windows.

7. Financial Interpretation

Let w_i be the risk-asset weight relative to NAV. Net exposure is $\sum_i w_i$; gross exposure is $\sum_i |w_i|$. In a simplified ledger with no other liabilities, cash weight can be written as $1 - \sum_i w_i$. For monetary market values, divide the signed net amount and absolute gross amount by NAV to obtain net and gross leverage. Here the weights are already NAV-normalized, so leverage is the signed or absolute weight sum; do not divide by NAV again. A net exposure near zero means only that long and short market values offset; it does not mean there is no position risk, financing cost, or squeeze risk.

For a single market factor, portfolio beta can be written $\beta_p = \sum_i w_i\beta_i$. Multi-factor exposure is $B^\top w$, where each factor loading row is aligned with its asset. Dollar neutrality means net dollar

exposure is zero; beta neutrality requires weighted beta to be zero; industry neutrality requires net weights to satisfy a constraint for each industry. These are different conditions. Concentration can be measured by HHI, the sum of squared weights normalized within total risk assets, or by the largest risk asset's share of total risk exposure. HHI describes weight dispersion only; it does not mean that assets are weakly correlated.

Leverage must also be read together with financing rules. Cash yield, borrow fees, rebates, funding rates, collateral haircuts, trading impact, and margin all change the cash path and the sustainable position size. The example ledger sets the per-period cash rate to 0. Borrow fees, rebates, and margin financing above the cash rate are not modeled. This simplification must not be read as saying that real financing costs are zero. Initial margin is the capital required to open a position; maintenance margin is the capital level that must not be breached while holding it. If the parameters are constructed, label them explicitly rather than presenting them as broker rules.

A stress scenario answers "what would the ledger show if these specified conditions occurred?" It is not "how likely is this?" For quantity q_i, current mark P_i, and simple-return shock r_i, per-asset PnL is $q_i P_i r_i$, and portfolio loss is the negative of total PnL. Setting multiple assets to adverse gaps constructs a common-shock scenario. Changing assumptions about scenario dependence does not turn this deterministic PnL into a probability model. Estimating a tail probability related to dependence requires a separately specified joint distribution and an assessment of estimation error.

Liquidity can first be approximated transparently as $D_i = |q_i|/(\pi_i\, ADV_i)$, where ADV_i is shares traded per trading day, π_i is the planned fraction of average daily volume, and D_i is measured in trading days. This assumes that the position can be traded each day at the participation rate; it does not model queueing, price impact, halts, or endogenous volume. It is a capacity screen, not an execution guarantee.

8. Numerical Example

Complete worked example: Four risk estimates for the same portfolio

The table applies historical, parametric, and simulation methods to the same two-asset portfolio. Inputs are assets A and B; weights 0.60 and 0.40; monthly simple-return mean inputs 0.0015 and 0.0020; standard deviation inputs 0.04 and 0.06; constructed correlation 0.45; Student-t degrees of freedom 5; constructed historical sample size 3,000; and confidence level 0.99. Historical and Monte Carlo methods have fixed seeds, so rerunning the same version produces the same result. These values come from `same_portfolio_risk_methods` and `teaching_inputs.portfolio_dgp`; they are not market estimates.

Estimation method	Mean loss	Standard deviation	VaR	ES
Constructed historical sample	−0.00168209	0.04066389	0.09780078	0.12252880

Continued: the same complete example

Normal parametric model	−0.00168209	0.04066389	0.09291625	0.10669587
Student-t parametric model	−0.00168209	0.04066389	0.10430684	0.13856101
Student-t Monte Carlo	Estimated from the same historical portfolio moments	Same standard-deviation convention	0.10337409	0.13853132

The corresponding JSON keys, in table-row order, are:

- `same_portfolio_risk_methods.historical`
- `same_portfolio_risk_methods.normal_fitted`
- `same_portfolio_risk_methods.student_t_fitted`
- `same_portfolio_risk_methods.monte_carlo`

The simulation draws Student-t losses with the same mean and standard deviation; its sample VaR and ES are close to the parametric Student-t results. A finite simulation sample need not match parametric values digit for digit. The counterexample is more important: under the same standard deviation, the normal model has lower VaR and ES than the t model. Increasing the normal standard deviation until its VaR matches the t VaR requires a standard deviation of 0.04556023, but the resulting normal ES is still only 0.11974567, below the t ES of 0.13856101. Thus scaling normal volatility can match one quantile but cannot automatically reproduce the tail severity of a fat-tailed model. Figure `ch33_loss_model_comparison.png` shows loss densities and VaR locations; the horizontal axis is the negative of monthly simple return and the vertical axis is density per unit loss.

Complete worked example: Exposure, stress, exit, and the next order

Next consider exposures. Constructed target weights are 0.50 and 0.30; factor loadings are A: Market 1.00, Value 0.40; and B: Market 1.20, Value −0.20. Market beta exposure is 0.86, Market factor exposure is 0.86, and Value factor exposure is 0.14. Net and gross leverage are both 0.80, and cash weight is 0.20. HHI within risk assets is 0.53125, and the largest asset is 0.625 of total risk assets. A separate dollar-neutral counterexample holds a 0.50 long and a −0.50 short: net exposure is 0, gross exposure is 1, and beta is still 0.60. Even if the two positions are in the same industry, offsetting industry net exposure does not remove gross risk. All values are under JSON key `exposures`.

Next apply an overnight gap to the Chapter 29 ledger. NAV on the signal date is USD 1,088.2588. A holds 49.9201278 shares at a mark of USD 12.10 per share; B holds 14.97603834 shares at USD 19.00 per share. Scenario simple returns are −0.15 for A and −0.25 for B. A's PnL is −USD 90.6050; B's is −USD 71.1362; together they produce a conditional loss of USD 161.7412, or 0.14862385 of current NAV. These calculations use existing quantities and marks, not target weights in place of current positions.

In the same stress state, raise the dependence assumption from 0.45 to 0.90, set participation to 0.10, and use normal ADV of 150 shares per day for A and 60 for B. Stress ADV is reduced to 0.25 of normal. Under the fixed-participation formula, normal liquidation times are

Continued: the same complete example

3.32800852 and 2.49600639 trading days; under stress they are 13.31203408 and 9.98402556. The dependence value is an input to a constructed stress state. The deterministic gap PnL comes from the stated return shocks; it is not generated as a probability from an estimated correlation matrix.

Initial/maintenance margin rates rise from the constructed 0.55/0.35 to 0.85/0.75. After stress, gross market value is USD 726.8371 and stressed NAV is USD 926.5176. The margin requirement rises, but maintenance margin headroom remains positive at USD 381.3898, so no maintenance-margin call is triggered. Do not turn "not triggered" into "margin is safe": the ratios here are teaching inputs, and the example does not model intraday paths, post-gap margin calls, or forced execution.

Finally, set a conditional gap-loss limit at 0.08 of current NAV. The loss cap is USD 87.0607, and the stress loss exceeds it, so linear scaling of loss gives a size factor of 0.5382716. Scale current ledger weights to target weights of A 0.29876543 and B 0.14074074. Through the Chapter 29 ledger, which executes one period after the signal, the actual execution sells 22.59777199 shares of A and 7.25352124 of B and incurs the relevant fees. Post-execution weights differ slightly from the target because prices, fees, and execution timing all enter the ledger. The next step is to rerun the stress on actual post-trade positions, not to treat "target weights satisfy the linear limit" as completed risk control. Full inputs and outputs are under `ledger_stress_and_limit`; `ch33_ledger_stress.png` shows per-asset USD PnL.

8.1 Three Times from Risk Trigger to Actual Reduction

Time	Known ledger object	Risk decision or action
Signal date t, before execution	Actual shares q, mark P, current NAV	Calculate USD 161.7412 conditional loss and the 0.08 NAV limit; generate scaled target
Next permitted trading-day execution time	Target weights, execution price, fees, and available volume	This example sells 22.59777199 shares of A and 7.25352124 of B; the target itself has not reduced positions at time t
Re-mark after execution	Actual remaining shares, cash, and new NAV	Recalculate exposure, exit time, stress, and margin; then reassess the limit

This table carries forward a saved ledger result; the three times are not three new strategy experiments. Under a fixed shock, loss scaled only at the original marks is linear. Price movement, cost, rejection, or partial execution can make actual risk differ from the target. The constructed dependence state changing from 0.45 to 0.90 did not enter this fixed qPr arithmetic and cannot explain the USD 161.7412 value.

9. Python Implementation

Core functions are in risk.py, grouped by purpose:

- Distributions and tails: `empirical_var_es`, `normal_var_es`, `student_t_var_es`, `simulate_student_t_losses`.
- Revaluation and validation: `scenario_pnl`, `lagged_rolling_var`, `kupiec_pof`.
- Exit and financing: `liquidation_days`, `margin_requirements`.

The chapter CLI is `ch33_risk_management.py`. It reuses the Chapter 29 ledger, the existing exposure-report function, and the frozen French data reader.

Readers should start from the accepted I3 package root:

```
python run_notebook.py notebooks/ch33_risk_management.ipynb
```

In I3, `analyze(include_french=False)` skips French by default. The Notebook freshly calculates 3,000 constructed historical observations and 50,000 Monte Carlo losses, writes a results JSON, and generates four figures. It creates a unique `runs/ch33_risk_management_<uuid>/`; the I3 CLI uses its own `runs/cli_ch33_<uuid>/`. The runner also saves an execution copy, `execution.json`, and failure status under `runs/execution_<stem>_<uuid>/`. Therefore this Notebook must not be described as a static replay that writes no experiment files. This batch only inspected the entry points statically; it did not execute the full Notebook or CLI. The working-source `code/ch33_risk_management.py` still has an `analyze()` function with no `include_french` argument and always calls French, unlike the I3 wrapper; the two entry points must not be described as fully synchronized. The English Notebook, assembly, and cold start belong to BIL23. If a separate run is authorized, specify a new output directory and do not overwrite frozen QA results or figures.

Check core inputs separately by function. `scenario_pnl` (simple-return scenario revaluation) rejects non-positive prices and simple returns below -100%. Empirical loss vectors and unbounded Student-t parametric losses allow losses above 1; they belong to a different model domain and must not be hard-clipped. The Student-t API takes standard deviation and requires $\nu > 2$. State the loss currency/proportion, shares, USD per share, ADV in shares per trading day, and holding period. Relevant test entry points are `tests/test_ch33_risk.py` and `tests/test_ch33_workflow.py`; this batch only makes local checks of the new display example and does not rerun the historical full workflow.

10. Visualization

The chapter CLI generates four figures; their filenames are listed in the results JSON at `artifacts.figure_files`.

Discrete Quantiles and ES Tail Area

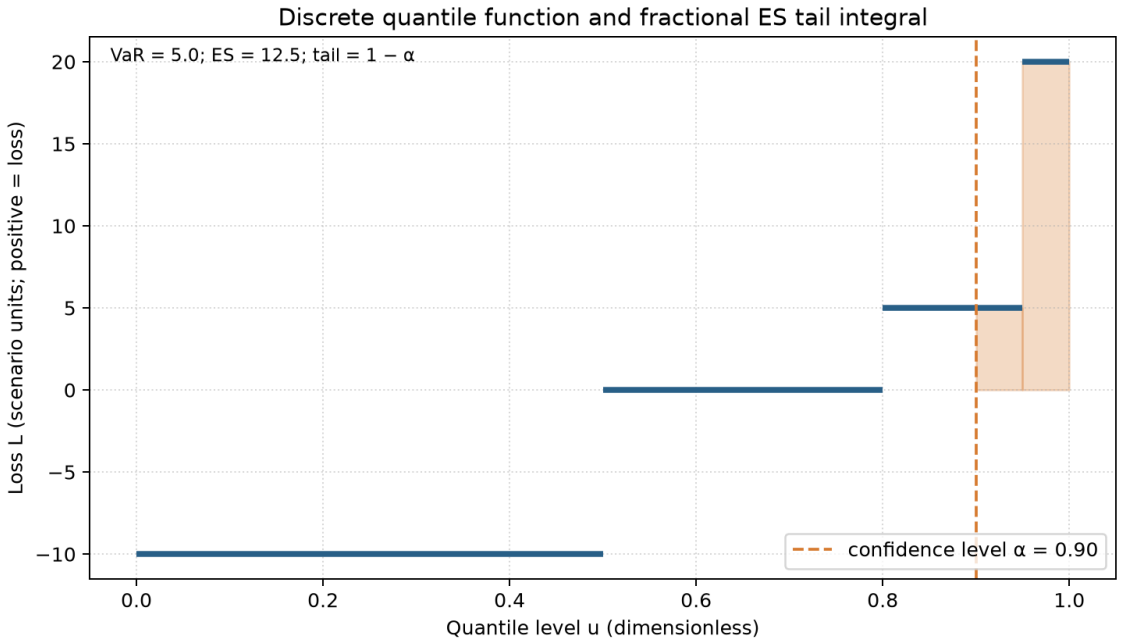


Figure 1: Discrete quantiles and ES tail area

The horizontal axis is dimensionless quantile level u; the vertical axis is scenario loss units. The sample is the fixed discrete teaching distribution in the body. The step function is the quantile function, and the orange area includes only the quantile mass to the right of the confidence threshold. This shows why threshold value 5 contributes only the amount needed to fill the tail.

Same-Portfolio Risk Model Comparison

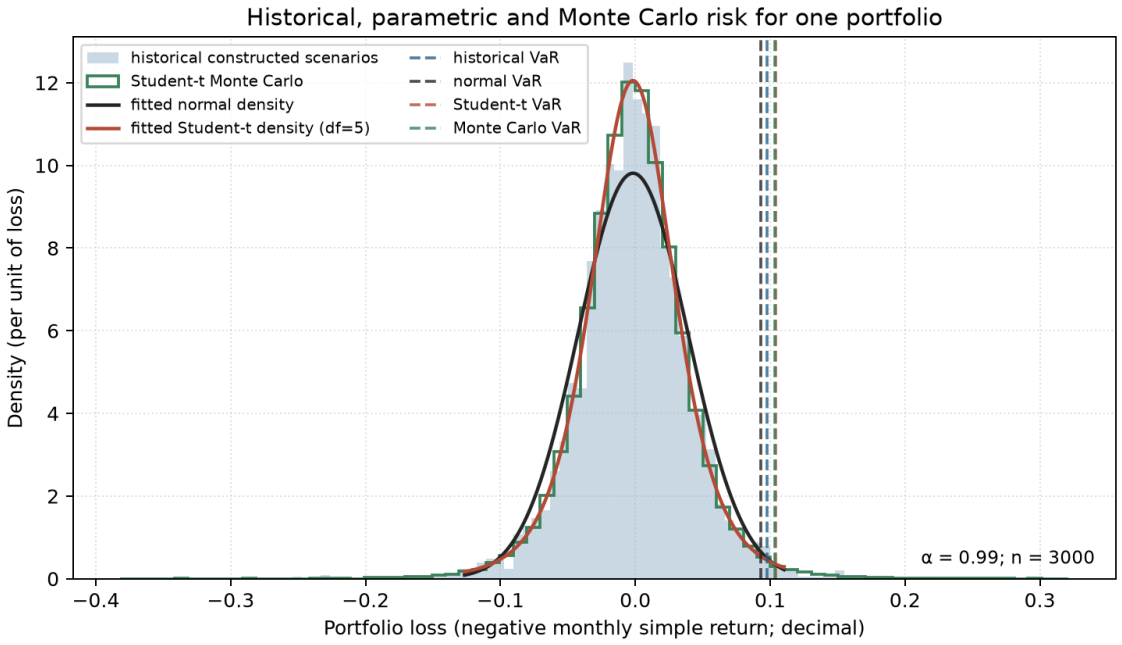


Figure 2: Same-portfolio risk model comparison

The horizontal axis is the negative of the portfolio's monthly simple return; the vertical axis is density per unit loss. The sample contains constructed historical losses and Monte Carlo losses generated from fitted Student-t parameters; the curves are normal and Student-t densities fitted to the same portfolio, and the vertical dashed lines mark the four methods' VaR. When reading the tail position, also consult the ES table. The horizontal extent of a density plot cannot establish what tails outside the model look like.

Overnight Ledger Stress Contributions

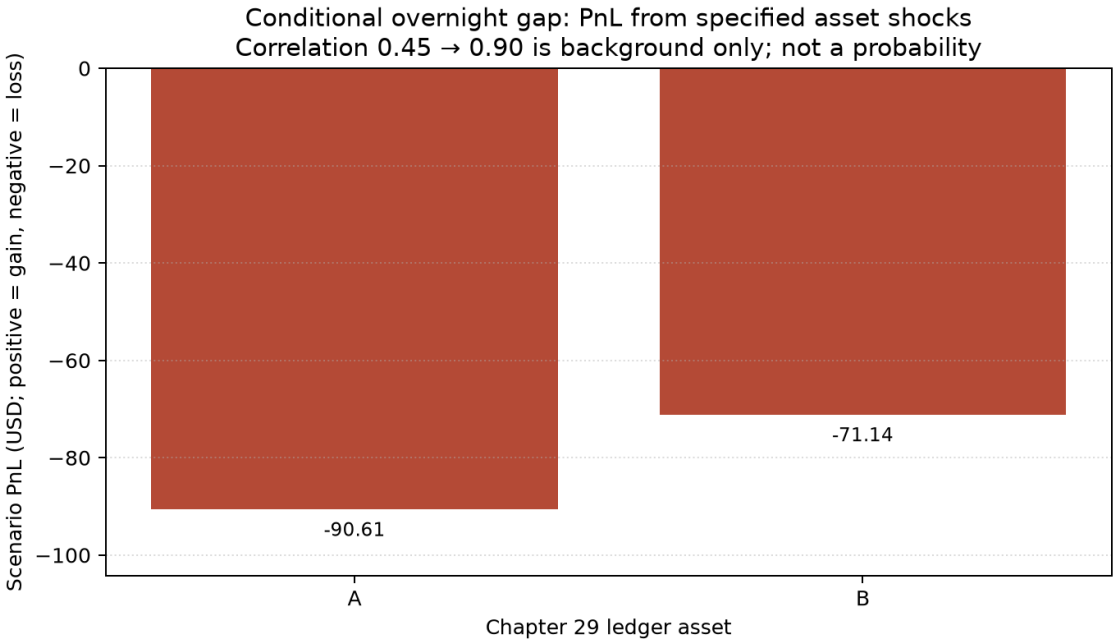


Figure 3: Overnight ledger stress contributions

The horizontal axis lists the Chapter 29 teaching-ledger assets; the vertical axis is scenario PnL in USD, where positive is gain and negative is loss. Correlation text in the figure is a constructed state assumption, not an event probability.

Windowed VaR Backtest

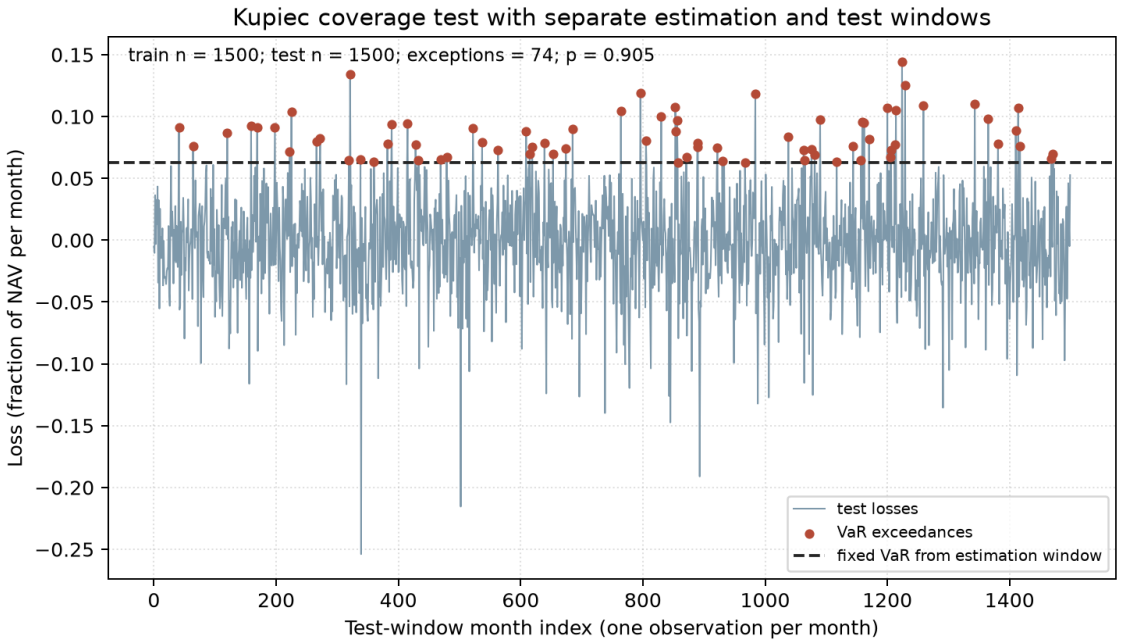


Figure 4: Windowed VaR backtest

The horizontal axis is the monthly observation index within the test window; the vertical axis is NAV-proportional loss. The VaR level is estimated only from the preceding training window; red points show observations strictly above the threshold. Sample size, exceedance count, and p-value in the caption all come from the same JSON result.

A figure cannot replace explanations of parameters, sample range, fees, and timestamps. If inputs need to change, use a new output directory and rerun; do not silently overwrite existing figures or numbers.

11. Real-Market Example

In addition to constructed samples, the CLI reads the frozen Kenneth R. French Data Library monthly value-weighted returns for ten industries in the project. The input snapshot is dated 2026-10-05 and covers 1,202 months from 1926-07 through 2026-08, with ten industry portfolios. Each month, the ten published portfolios are equally weighted, using simple monthly returns. The first 120 months (1926-07 through 1936-06) form the estimation window; the remaining 1,082 months (1936-07 through 2026-08) form the test window. The training-period 95% VaR is 0.14302 and ES is 0.20204833; there are 5 exceedances in the test period, with a Kupiec LR statistic of 76.69493441 and asymptotic p-value of 1.995144691255254e−18. Reproducible fields are under JSON key `french_real_example`.

This is a published historical industry-portfolio series, but it is the current revised, back-filled version. It has no point-in-time version, historical receipt timestamps, or actual trades in investor-tradable constituents. The monthly label denotes the calendar period being measured, not that the return was known before month end. Nor is this a strategy trading ledger. This

example can illustrate how one historical-window VaR and test-window exceedance count behave on this series; it cannot establish real-time strategy performance, trading capability, or a tradable backtest. Six SHA-256 hashes for the raw, extracted, and processed industry data and FF3 data are in `french_real_example.sha256`; download sources and transformation records are in `data/manifests/french_industries_2026-10-05.json` and the FF3 manifest. Download and use remain subject to the data provider's terms; public availability does not itself grant unrestricted redistribution rights.

12. Common Mistakes

Reversing the signs of positive return and positive loss. A positive portfolio return is a gain, but this chapter's $L = -R$ is negative. If code passes portfolio return directly to a VaR function that treats positive values as losses, the loss-tail direction is reversed. Ledger PnL uses qPr, and loss takes the negative; `scenario_pnl` returns monetary PnL, not loss.

Treating VaR as maximum loss, or writing ES as a threshold-conditional mean. VaR is not a guaranteed cap. When a distribution has probability mass at the threshold, neither the conditional mean strictly above VaR nor the conditional mean at or above VaR equals quantile-integral ES. The following code builds the same empirical sample from the results JSON and prints the correct value alongside both incorrect alternatives. Run it from the repository root:

```
import json
import sys
from pathlib import Path

import numpy as np

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.risk import empirical_var_es

result = json.loads((ROOT / "qa/a6/codex_resume/ch33/results/ch33_results.json").
    ↪ read_text())
case = result["pinned_discrete_case"]
counts = np.rint(np.asarray(case["probabilities"]) * case["scenario_count"]).astype(
    ↪ int)
losses = np.repeat(case["loss_values"], counts)
var, es = empirical_var_es(losses, case["alpha"])
print(var, es, losses[losses > var].mean(), losses[losses >= var].mean())
```

The outputs are VaR, quantile-integral ES, strict-tail mean, and inclusive-threshold mean; itemized results are in `pinned_discrete_case`. The implementation error is usually not the sorting itself but ignoring that only part of the threshold mass belongs in the tail.

Equating high volatility with fat tails. Increasing the normal standard deviation raises tail losses but leaves the distributional shape normal. The variance-matched counterexample in the body shows that even after increasing normal VaR to the Student-t VaR, normal ES remains lower. State the distribution, parameter convention, estimation window, and frequency; "conservative volatility" alone is not enough.

Treating dollar neutrality as risklessness, or mixing gross and net into one ratio. Long and short market values can offset, while gross exposure, beta, factors, industries, borrow costs, and squeeze risk remain. A zero net position within the same industry does not mean individual securities can be liquidated easily. Report signed net and absolute gross values separately and state each denominator.

Calling scenario loss a probability, or an ADV formula an execution guarantee. A fixed return shock describes a conditional outcome. Correlation assumptions, gap size, and liquidity discounts are constructed inputs; without a calibrated model and event frequency, do not claim "the probability is X."

Using one observation both to estimate VaR and to check its exceedance. This lets the threshold adapt to the loss already observed and creates look-ahead bias. Estimate fixed historical VaR from a training window first, then count exceedances in a later, separate test window. For rolling VaR at target period t, use only lookback data before t; the training-index interval must not contain t.

Treating non-rejection by the POF test as proof that the model is correct. The Kupiec test checks only whether the unconditional exceedance rate is compatible with $q = 1-\alpha$; it does not determine whether exceedances cluster, tail amounts are misestimated, or predictions are stable. The degrees-of-freedom-1 chi-square p-value is asymptotic. With zero or few exceedances, the likelihood-ratio boundary can be calculated stably, but the small-sample test may have almost no power to identify a problem. Non-rejection is not proof of accuracy.

13. Quant Research Application

Place risk work in the research process by freezing the risk question first rather than choosing a statistics package. State the portfolio, currency, loss sign, confidence level α, horizon h, asset set, information availability time, and decision to be supported. Fixed VaR estimates one threshold in a training window and suits a later fixed-window coverage diagnostic. Rolling historical VaR is closer to a period-by-period updated forecast, but every forecast must use only information available at that time, and calculation and backtesting must move forward by target time.

Next, check consistency between return inputs and the ledger. Availability must precede signal formation, and signal formation must precede execution. Align market prices, corporate actions, cash, fees, quantities, and return frequency. Chapter 29 target weights are not current actual weights; stress revaluation must use actual shares and marks on the correct date. If the portfolio contains options or other nonlinear positions, the linear qPr shock is inadequate for repricing; use the relevant valuation model to revalue each scenario.

Then assign complementary methods to different questions: use net/gross exposure, beta, and factor exposures to locate risk sources; use VaR for a threshold at the stated confidence

level; use ES for tail severity; use concentration to identify a dominant position; use ADV for a rough exit-time screen; use joint stress scenarios for conditional ledger losses; and use margin and financing assumptions to check cash constraints. Differences across methods are not a reason to pick the result that looks best; they can reveal model risk. A higher ES from a fat-tailed parametric model does not make it the true data-generating process, and a historical threshold does not automatically cover unseen events.

For model backtesting, separate the estimation and test windows. The example estimates VaR in one constructed monthly-return window and then counts strict exceedances in a non-overlapping later window. It has 74 exceedances compared with 75 expected under the null, LR statistic 0.01409458, and asymptotic p-value 0.9054967146341367. This says only that the exceedance count in this particular test window is not strong unconditional-coverage evidence against the null; it does not prove tail amounts, independence, or stability are correct. Kupiec's source notes limited power for tail validation; this chapter uses only the Bernoulli count likelihood and does not repeat the source's older wording about test properties. If clustered exceedances matter, specify a separate independence test and first read its source and finite-sample properties.

Risk limits must lead to a next-period action. In the example, the conditional-loss limit is 0.08 of current NAV; stress loss exceeds the limit, so current positions are scaled by the linear scenario-loss ratio and sent to the ledger for execution at next trading day's price. Execution has a ledger interval and includes fees and price differences; closing weights do not equal signal targets. The research report should retain the trigger value, input positions, scaling factor, filled quantities, fees, and post-trade reassessment. If costs prevent reaching the target or exit days exceed the constraint, choose a more conservative executable target rather than only changing the wording in the report.

The historical-data example uses published French industry portfolios only to demonstrate the method. The current revised industry data have no record of when they were received historically; splitting them into training and test windows cannot establish what an investor could access at the time. A real-time tradability claim requires point-in-time data, portfolio constituents, and trading-cost evidence; this example does not provide those.

13.1 Zero Exceedances: Point Probability and Test p-Value Are Different

33-L04 separately considers a 99% one-day VaR and 250 days of forecasts made in advance. Under the null of independent daily exceedances with probability 0.01, $P(X = 0) = 0.99^{250} \approx 0.08106$ is the point probability of exactly zero exceedances. Kupiec's boundary likelihood ratio is $-500\log(0.99) \approx 5.0252$; asymptotic chi-square calibration with one degree of freedom gives a p-value of about 0.025. These use different statistics or rejection regions: 8.1% is not automatically an arbitrary two-sided exact-test p-value. An exact test must first specify its one-sided or two-sided rule and discrete endpoints. The constructed backtest in the body is instead 95% monthly, with 1,500 training and 1,500 test observations; do not conflate it with this daily hand calculation.

14. Chapter Summary

A risk report should connect "what are the positions?", "how is loss defined?", "what information is used to estimate it?", "how is it revalued under scenarios?", and "how does the threshold affect the next trade?" into a traceable chain. State direction, units, confidence level, and horizon first; report VaR and ES while retaining fractional threshold mass for discrete distributions; decompose net/gross leverage, beta, factor, industry, concentration, liquidity, financing, and margin; stress actual positions rather than ideal targets; assess exceedance rates in a separate test window; and finally translate the limit into a target that can be executed after the signal at the permitted time, then reassess using actual fills.

A useful risk statement should include at least: the object and timestamp, loss sign and unit, confidence level and horizon, method and training data, VaR and ES, major factors and concentration, stress scenario and its conditional nature, liquidity and margin assumptions, out-of-sample exceedance count and test limitations, and the target and actual fills after a triggered limit. Reporting only one VaR number or one historical-backtest p-value is not enough to describe the losses a strategy can withstand.

15. Exercises

33-E01 (Discrete Threshold). Use JSON's `pinned_discrete_case`. Write the cumulative distribution at the loss value containing VaR, calculate the two tail contributions to ES by hand, and explain why neither the strict-exceedance mean nor inclusive-threshold mean is ES.

33-E02 (Distributional Shape). Use `same_portfolio_risk_methods.variance_matched_tail_counterexample`. State what is held fixed and what changes in the normal versus Student-t comparison. Explain why increasing normal standard deviation can match VaR but still fail to match ES.

33-E03 (Exposure). Use `exposures`. Explain the meanings of net leverage, gross leverage, cash weight, Market factor exposure, Value factor exposure, and HHI. Then use `neutrality_counterexample` to distinguish dollar neutrality from beta neutrality and industry neutrality.

33-E04 (Stress and Exit). Use `ledger_stress_and_limit`. Recompute both assets' scenario PnL from shares and prices; compare exit days under normal and stressed ADV; and explain why positive margin headroom can coexist with a triggered risk limit.

33-E05 (Coverage Test). Use `risk_model_backtest`. State the estimation and test window boundaries, repeat the test-window exceedance count, LR, and p-value, and name at least two conclusions that non-rejection cannot establish. Explain why the first rolling forecast can use only earlier observations.

16. Programming Exercises

33-P01 (Fractional-Tail Implementation). Read the discrete inputs from the results JSON, call `empirical_var_es`, and dynamically recompute VaR, ES, and both incorrect alternative means. Use assertions to verify the result keys. Tests for NaN and invalid confidence levels must raise input errors.

33-P02 (Exposure Reconciliation). Read `teaching_inputs.ledger_stress` from the CLI inputs and JSON output, call the existing `exposure_report`, and reconcile net exposure, gross exposure, and beta. Also verify that the dollar-neutral counterexample still has nonzero beta and gross exposure.

33-P03 (From Limit to Fill). Read the actual ledger shares, marks, stress returns, NAV, and cap for the signal date from JSON; call `scenario_pnl` to recompute stress loss and the scaling factor. Check the sign of the CLI execution orders and explain why target weights can differ from post-fill weights.

33-P04 (Kupiec Boundary). Read α and the test-window sample size from `risk_model_backtest`, call `kupiec_pof` to recompute the observed result, and recalculate the zero- and all-exceedance boundary likelihoods. All statistics should be finite; explain the continuous limit $0 \log 0 = 0$ and why the p-value is still asymptotically calibrated.

33-P05 (Leak-Free Rolling Forecast). Use the eight deterministic losses supplied in Section 18 (without calling the full CLI or `analyze`), then call `lagged_rolling_var`. Check that the first forecast receives only the immediately preceding window, that the output length is correct, and that no forecast contains its target-period loss.

17. Challenge Problems

33-C01 (Atoms and Risk Measures). For `pinned_discrete_case`, write the quantile step function and prove that the fractional-threshold formula equals the quantile integral. Then discuss how the ES result depends on integrability of the positive part of loss. If the data are negated and represented as profit, state which tail integral and sign must change.

33-C02 (What Unconditional Coverage Omits). Use the exceedance count and sample size in `risk_model_backtest.test_window` to construct two differently ordered exceedance sequences with the same sample exceedance rate but different clustering. Prove that Kupiec POF gives both the same likelihood-ratio statistic, and identify a further test question that requires reading and implementation.

33-C03 (Availability of a Real Series). Use `french_real_example` to draft a minimal research-reconciliation plan. Distinguish what the current data can and cannot support. List fields and execution evidence needed for a point-in-time interpretation of a historical simulation, and state criteria to continue, pause, or reject a strategy conclusion.

18. Solutions / Solution Hints

33-E01 Solution. The cumulative probabilities at losses -10, 0, 5, and 20 are 0.50, 0.80, 0.95, and 1.00, respectively, so the smallest loss reaching 0.90 is VaR = 5. The tail has mass 0.10; it takes all 0.05 mass at 20 and 0.05 of the mass at 5. Thus ES = $(5 \times 0.05 + 20 \times 0.05)$ / 0.10 = 12.5. The conditional mean strictly above VaR is 20; the inclusive-threshold mean includes all 0.15 mass at 5 and is 8.75. The former has less than 10% tail mass, while the latter includes 20% mass; ES takes exactly the worst 10%, not an arbitrary one-sided conditional event. JSON check: `pinned_discrete_case`.

33-E02 Solution. The comparison holds the portfolio, loss mean, and standard deviation fixed, while changing the tail shape from normal to Student-t with 5 degrees of freedom. Normal VaR/ES are 0.09291625 and 0.10669587; the t model gives 0.10430684 and 0.13856101. Raising the normal standard deviation to 0.04556023 matches the t VaR; the normal ES rises only to 0.11974567 and remains below the t ES. The normal tail keeps its shape: a single scale change stretches the same distribution family and cannot reproduce the thicker tail shape. This counterexample does not prove that every sample is generated by a t distribution; the construction rule and seed are in `teaching_inputs.portfolio_dgp`.

33-E03 Solution. The target risk weights sum to 0.80, as do their absolute values, so net and gross leverage are both 0.80 and cash weight is 0.20. Market exposure is $0.50 \times 1.00 + 0.30 \times 1.20 = 0.86$; Value exposure is $0.50 \times 0.40 + 0.30 \times (-0.20) = 0.14$. HHI within risk assets is $(0.50/0.80)^2 + (0.30/0.80)^2 = 0.53125$. NAV-normalized weights must not be confused with HHI normalized within risk assets. In the counterexample, weights sum to zero and absolute weights sum to 1, but beta is 0.60; the net industry amount is also zero if they are in the same industry. Therefore dollar neutrality does not imply beta neutrality, zero gross risk, or adequate liquidity.

33-E04 Solution. PnL is $q_i P_i r_i$. For A, $49.9201278 \times 12.1 \times (-0.15) = -90.6050$ USD; for B, $14.97603834 \times 19 \times (-0.25) = -71.1362$ USD. Negative PnL gives total conditional loss of USD 161.7412. The liquidation approximation is $D = |q|/(\text{participation rate} \times ADV)$; normal and stressed days are in `ledger_stress_and_limit.liquidity`. Lower ADV lengthens the exit time proportionally. Positive maintenance margin headroom is a local result under constructed margin ratios and stressed NAV. The risk limit separately compares conditional loss with an NAV fraction, so one metric being within its limit does not undo a breach of the other. Here maintenance margin headroom is USD 381.3898, yet the risk limit is breached.

33-E05 Solution. The training indices are [0, 1500), and test indices are [1500, 3000); they do not overlap. The test window has 74 exceedances, LR 0.01409458, and p-value 0.9054967146341367. Non-rejection does not prove that tail severity is accurate, exceedances are independent, the distribution is stable, or extreme stress is covered. For rolling target index 250, only indices 0 through 249 are available; including the target loss in its own threshold estimate uses the future outcome and leaks information. All fields are in `risk_model_backtest`.

33-P01 Reference Implementation (run from the repository root):

```
import json
import sys
from pathlib import Path

import numpy as np

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.risk import empirical_var_es

result = json.loads((ROOT / "qa/a6/codex_resume/ch33/results/ch33_results.json").
    ↪ read_text())
case = result["pinned_discrete_case"]
counts = np.rint(np.asarray(case["probabilities"]) * case["scenario_count"]).astype(
    ↪ int)
losses = np.repeat(case["loss_values"], counts)
var, es = empirical_var_es(losses, case["alpha"])
assert np.isclose(var, case["var"]) and np.isclose(es, case["es"])
assert losses[losses > var].mean() == case["strict_tail_conditional_mean"]
assert losses[losses >= var].mean() == case["inclusive_tail_conditional_mean"]
for bad_values, bad_alpha in (([np.nan], case["alpha"]), (losses, np.nan), (losses,
    ↪ 0.0), (losses, 1.0)):
    try:
        empirical_var_es(bad_values, bad_alpha)
    except (TypeError, ValueError):
        pass
    else:
        raise AssertionError("non-finite input must be rejected")
print(var, es)
```

The assertions cover the threshold-atom adjustment. Tests also verify rejection of NaN and confidence-level boundaries; invalid inputs must not be converted into silent fill values.

33-P02 Reference Implementation (run from the repository root):

```
import importlib.util
import json
from pathlib import Path

import numpy as np

ROOT = Path.cwd()
spec = importlib.util.spec_from_file_location("ch33", ROOT / "code/
    ↪ ch33_risk_management.py")
ch33 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ch33)
from quant_course.portfolio_construction import exposure_report

result = json.loads((ROOT / "qa/a6/codex_resume/ch33/results/ch33_results.json").
    ↪ read_text())
inputs = ch33.INPUTS["ledger_stress"]
weights = np.asarray(inputs["initial_target_weights"])
report = exposure_report(weights, inputs["betas"], inputs["sectors"])
saved = result["exposures"]
```

```
assert report.net == saved["net_exposure"]
assert report.gross == saved["gross_exposure"]
assert report.beta == saved["market_beta"]
neutral = saved["neutrality_counterexample"]
nw = np.asarray(inputs["neutrality_example_weights"], dtype=float)
nb = np.asarray(inputs["neutrality_example_betas"], dtype=float)
assert np.array_equal(nw, [0.5, -0.5]) and np.array_equal(nb, [1.5, 0.3])
net, gross, beta = float(nw.sum()), float(np.abs(nw).sum()), float(nw @ nb)
assert net == 0 and gross == 1 and np.isclose(beta, 0.6)
assert np.isclose(net, neutral["net_leverage_on_nav"])
assert np.isclose(gross, neutral["gross_leverage_on_nav"])
assert np.isclose(beta, neutral["beta"])
print(saved["factor_exposures"], saved["concentration_hhi_within_gross_risk_assets"])
```

This reuses an existing exposure interface and adds no engine; industry, beta, and HHI still need to be interpreted separately.

33-P03 Reference Implementation (run from the repository root):

```
import json
import sys
from pathlib import Path

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.risk import scenario_pnl

result = json.loads((ROOT / "qa/a6/codex_resume/ch33/results/ch33_results.json").
    ↪ read_text())
case = result["ledger_stress_and_limit"]
names = list(case["positions_at_signal_shares"])
pnl = scenario_pnl(
    [case["positions_at_signal_shares"][a] for a in names],
    [case["mark_prices_at_signal_currency_per_share"][a] for a in names],
    [case["overnight_gap_simple_returns"][a] for a in names],
)
loss = -float(pnl.sum())
risk = case["risk_limit"]
scale = risk["loss_cap_currency"] / loss
assert round(loss, 4) == case["conditional_stress_loss_currency"]
assert round(scale, 7) == risk["scale_factor"]
assert all(fill["quantity"] < 0 for fill in risk["actual_execution_fills"])
print(loss, scale, risk["target_weights_after_limit"], risk["
    ↪ actual_close_weights_after_execution"])
```

Sell quantities are negative. First compute the current stress loss from the actual shares held on the signal date, their current marks, and the specified return shocks. Divide the loss cap by that loss to obtain the scaling factor, then use it to form new target weights. The shares subsequently filled, execution prices, fees, and time interval all affect post-trade weights, which can therefore differ from the signal targets.

33-P04 Reference Implementation (run from the repository root):

```
import json
import sys
from pathlib import Path

ROOT = Path.cwd()
sys.path.insert(0, str(ROOT / "src"))
from quant_course.risk import kupiec_pof

result = json.loads((ROOT / "qa/a6/codex_resume/ch33/results/ch33_results.json").
    ↪ read_text())
backtest = result["risk_model_backtest"]
test = backtest["test_window"]
calculated = kupiec_pof(test["realized_exceedances"], test["observations"], backtest[
    ↪ "alpha"])
for count in backtest["boundary_count_checks"].values():
    if isinstance(count, dict):
        edge = kupiec_pof(count["exceedances"], count["n_obs"], backtest["alpha"])
        assert edge["lr_uc"] == count["lr_uc"]
        assert edge["p_value"] == count["p_value"]
assert abs(calculated["lr_uc"] - test["lr_uc"]) < 1e-8
assert calculated["p_value"] == test["p_value"]
print(calculated)
```

For $x = 0$ or $x = n$, `kupiec_pof` handles zero log terms using the continuous likelihood limit $0 \log 0 = 0$, avoiding the logarithm of zero. Boundary counts produce finite LR statistics, but the asymptotic chi-square p-value should not be read as an exact small-sample probability when exceedances are few. With all 1,500 observations as exceedances, the chi-square survival function returns 0.0 in double precision; this is numerical underflow, not an exact zero probability.

33-P05 Reference Implementation (local deterministic input; does not call the full `analyze`):

```
import numpy as np
from quant_course.risk import lagged_rolling_var

# Define a small constructor in place of the full 3,000-observation/MC/French
    ↪ research entry point.
def constructed_losses():
    return np.array([0.0, 0.2, -0.1, 0.4, 0.1, 0.3, -0.2, 0.5])

losses = constructed_losses()
window, alpha = 3, 0.75
actual = lagged_rolling_var(losses, window, alpha)
expected = []
for t in range(window, len(losses)):
    past = sorted(float(v) for v in losses[t-window:t])
    # Independent order-statistic selection; do not repeat empirical_var_es here.
    k = next(k for k in range(1, window+1) if k/window >= alpha)
    expected.append(past[k-1])
    changed = losses.copy()
    changed[t:] += 10.0
    altered = lagged_rolling_var(changed, window, alpha)
    np.testing.assert_array_equal(altered[:t-window+1], actual[:t-window+1])
```

```
np.testing.assert_allclose(actual, expected, rtol=0, atol=1e-14)
np.testing.assert_allclose(actual, [0.2, 0.4, 0.4, 0.4, 0.3], rtol=0, atol=1e-14)
assert len(actual) == len(losses)-window
assert expected[0] == max(losses[:3])
print(actual)
```

The returned item j corresponds to target position $t = window + j$; each calculation independently uses only window $[t - window, t)$. Changing the loss at t and afterward does not change forecasts through t. This example covers an explicitly finite window; it does not prove that arbitrary data or feature channels are leak-free. Missing calendar months still require a separate definition of the valid calendar. The original objective of a deterministic constructed input is retained here with a minimal explicit example, avoiding the full CLI that would call French and Monte Carlo research.

33-C01 Reference Argument. The empirical quantile function is constant on each probability-mass interval. The tail integral sums the rectangular areas strictly above the threshold, then adds the portion of the threshold interval to its right. Multiplying the threshold by $F(v) - \alpha$ gives exactly this second area. Dividing by the tail length gives the ES definition. Fractional threshold weight therefore follows directly from the discrete quantile integral; it is not an empirical patch. Generalizing to the population requires right-tail integrability $E[L^+] < \infty$. If profit is $X = -L$, with profit quantile function q_X, use

$$ES_\alpha(L) = -\frac{1}{1-\alpha}\int_0^{1-\alpha} q_X(u)du.$$

At an atom endpoint, negating a lower quantile requires a corresponding one-sided convention; they are not pointwise identical without it. The endpoint has measure zero in the integral, so it does not change the ES equality. See Acerbi and Tasche, Definition 2.6, Proposition 3.2, and Proposition 4.1 (the historical A6/C15 reading scope is in the source notes).

33-C02 Reference Argument. Construct two binary sequences with length `test_window.observations` from the results JSON. Both contain `test_window.realized_exceedances` ones; place the ones evenly across the first sequence and consecutively in the second. Their sample rates x/n are identical. The Kupiec POF likelihood depends only on total count x and sample size n, so both sequences have exactly the same LR and p-value; the test cannot distinguish clustering. An independence question requires conditional transitions or another sequence test. Before adding one, read its source for assumptions, zero-count handling, and finite-sample behavior. Non-rejection by POF is not a substitute for an independence test.

33-C03 Reference Argument. The current data support slicing, combining, and counting exceedances in the published monthly industry returns included in this snapshot. They do not support historical availability, historical constituents, execution prices, capacity, fees, tradable strategy returns, or future risk. A point-in-time review needs, at minimum, each version's publication and receipt time, values before and after revision, field definitions, publication timezone, historical industry membership and weights, signal availability time, order and fill records, and evidence on fees and market capacity. If versions cannot be traced, pause a real-time conclusion. If current revised values are backfilled into historical signals, reject any claim that this is leak-free deployment evidence. The exercise does not require a positive research result; credit should reward a clear distinction between "describing a historical series" and "tradable deployment."

19. Further Reading

- **Must Know:** Carlo Acerbi and Dirk Tasche, *On the Coherence of Expected Shortfall*. The historical A6/C15 reading record for this chapter covers Definition 2.6, Proposition 3.2, equation (3.3), and relevant pages for Proposition 4.1. Use it to understand quantile integration and the ES definition at discrete threshold atoms; this is a local reading, not the full paper.
- **Should Know:** Paul H. Kupiec, *Techniques for Verifying the Accuracy of Risk Measurement Models*, FEDS 95-24. The historical A6/C15 reading record covers selected pages on daily monitoring, failure-count likelihood, binomial likelihood, and estimation error in historical simulation. Use it to understand the unconditional-coverage setup and limits on tail-validation power. This is a FEDS discussion paper; the reading was not the full journal article.
- **Advanced (not read):** Christoffersen's conditional coverage and exceedance-independence methods. Before using these tests, read the original work and state model assumptions, calibration, and boundary-count handling.
- **Optional:** Official SciPy v1.18.0 documentation for `scipy.stats.t`. Use it to check the `ppf`, `pdf`, `rvs`, `loc`, and `scale` API semantics; it is not the source for the ES derivation.
- Basel market-risk capital rules and regulatory backtesting guidance were **not read** for this chapter. The body makes no capital-compliance or regulatory-threshold claim.

20. References

- [AT2002-v5] Carlo Acerbi and Dirk Tasche, "On the Coherence of Expected Shortfall," arXiv:cond-mat/0104295v5, 2002. https://arxiv.org/pdf/cond-mat/0104295v5. Historical A6/C15 citation scope: discrete-tail atom adjustment, quantile integration, and related definitions and arguments for tail-sample averages; local reading only. See source_notes.md for specific pages.
- [KUPIEC1995-FEDS95-24] Paul H. Kupiec, "Techniques for Verifying the Accuracy of Risk Measurement Models," Finance and Economics Discussion Series 95-24, Federal Reserve Board, cover dated May 1995; interior title page dated April 1995. https://fraser.stlouisfed.org/files/docs/publications/bog_feds/feds_1995-024.pdf. The historical A6/C15 scope uses only the monitoring setup and count likelihood from the pages read; it does not generalize every claim in the source.
- [SCIPY-T] SciPy v1.18.0 Manual, `scipy.stats.t`. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.t.html. The historical A6/C15 citation uses it only to confirm the Student-t API and location/scale parameterization.
- [FRENCH-FROZEN] Kenneth R. French Data Library, 10 Industry Portfolios, value-weighted monthly block; FF3 risk-free series. Industry data: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/10_Industry_Portfolios_CSV.zip; details: https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/Data_Library/det_10_ind_port.html. See the corresponding project data-manifest JSON for the course's frozen snapshot, download

date, use statement, processed fields, and checksums. These are current revised historical values.

- [A6-33-PLAN] Project-local frozen plan `qa/a6/claude/plan.md` and Chapter 33 contract terms in `A6_CONTRACT.md`. These support course interfaces, notation, and teaching scope, not external empirical evidence.

The historical A6/C15 reading scope, method use, and limitations for each formula and empirical result are consolidated in `source_notes.md`.

Chapter 34 — Market Microstructure

Prior engineering evidence is cited under its original acceptance status. Through-line: translate quotes shown on a screen into specific orders, venue rules, visible contra-side liquidity, actual fills, positions, and profit and loss, while recording when each step occurs and when its information becomes available. Prerequisites: Chapter 10, Markets and Traders; and Chapter 29, Backtest Ledgers. Chapter 33, Risk Management, provides background. After reading, you should be able to recompute a book sweep, explain why the midpoint is not a promise of execution, identify what snapshots and OHLCV cannot prove, and carry execution limits back into research design.

1. Learning Objectives

After completing this chapter, you should be able to:

1. Describe a quote at a specified venue and time using the bid, ask, midpoint, and quoted spread; report each with its currency, per-share or per-unit convention, and timestamp.
2. Distinguish market orders, limit orders, an order book, displayed depth, a queue, and queue position. Explain each order type's price and fill-quantity constraints.
3. Hand-calculate the quantity filled across multiple price levels, notional, volume-weighted average price (VWAP), and residual quantity; reconcile quotes, fills, positions, cash, and PnL separately.
4. Distinguish the quoted spread, slippage, price movement within the book caused by a sweep, price impact, and adverse selection; state direction, benchmark, horizon, and units for every cost figure.
5. Explain that first-in, first-out (FIFO) is only one possible priority rule at certain venues and for certain order classes; assess queue position and fills from the applicable rule version, order events, and timestamps rather than from a snapshot.
6. Distinguish event time, the time data became available to the researcher, decision time, order-send and order-receive times, fill time, valuation time, and the return interval. Explain which matching outcomes OHLCV cannot identify.
7. Run and inspect this chapter's single-venue static displayed-depth calculator; explain why it is not a full matching engine and must not be described as an order-book extension to Chapter 29's daily-frequency ledger.

2. Why Quant Needs It

A strategy can correctly predict direction and still fail to realize its forecast because it buys above its valuation, lacks enough sell-side depth, receives no fill, or fills too late. If a backtest treats the closing price as a frictionless execution price, it turns a research mark into a trading fact. Market microstructure breaks that leap into separate steps: quotes and the order book describe the state displayed by a venue at a given time; orders specify price and quantity constraints; matching rules allocate contra-side liquidity among competing orders; only fills change cash and positions; valuation afterward creates mark-to-market or liquidation PnL.

Chapter 10 surveys venues, brokers, market makers, and market participants; this chapter continues by explaining how they meet through quotes and orders. Chapter 29's ledger converts target weights into deterministic ledger fills after a specified lag and calculates fees, cash, and positions. It assumes the full target quantity can be filled as configured; it does not model partial fills, queues, or book walking. This chapter explains those mechanisms but does not call a daily-frequency ledger an order-book matching engine. Chapter 33 treats liquidity and liquidation days as risk constraints; this chapter explains why visible depth and price conditions matter to those constraints.

The chapters that follow divide the work as follows: Chapter 34 defines orders, quotes, books, and fill evidence; Chapter 35 connects actual fills to costs and a self-financing ledger; Chapter 36 develops the clock for splitting orders and implementation shortfall. Project 4's Cost Execution Lab already contains a constructed `ExecutionSlot`, simulated fills, segmented financing and borrow, and an impact model, but it has no real order book or broker fills. Project 5's preparation layer therefore gains no order-book support. A static visible-book sweep and a model-based execution simulation are separate interfaces: the latter does not connect this chapter's snapshot calculator to real matching, and its outputs cannot validate a real queue.

3. Intuition

Start with a single-venue snapshot explicitly labeled as a constructed teaching example. Prices are in USD per share, quantities are shares, and the example tick is USD 0.01 per share:

Bid	Displayed shares	Ask	Displayed shares
100.00	300	100.02	200
99.98	400	100.04	300

The best bid is 100.00, the best ask is 100.02, the midpoint is 100.01, and the displayed spread is 0.02 per share. The midpoint lies between the two quotes, but neither side promises to trade with you at 100.01. If you want an immediate 350-share purchase, it first consumes the 200 shares offered at 100.02, then 150 shares at 100.04. Whether enough quantity is available depends on contra-side depth, not on whether a midpoint exists.

If we remove only the sell quantity consumed by the buy order and assume the bid is unchanged, the remaining best ask is 100.04 and the new displayed midpoint is 100.02; the arrival midpoint remains 100.01. This static update shows that a midpoint is tied to a time and does not guarantee execution.

These four things are not interchangeable: 100.00 and 100.02 are quotes; 350 shares are the order request; the 200-share and 150-share fills are outputs of this model; the post-purchase holding is 350 shares. Cash falls by USD 35,010, while the holding marked at the arrival midpoint is worth USD 35,003.50, a difference of USD 6.50. If we then assume an immediate 350-share sale against the same frozen snapshot, the bid-side depth produces proceeds of USD 34,999 and round-trip PnL of -USD 11. That sale is a separate hypothetical sweep; it is not evidence that the market would remain unchanged after the purchase.

Before reading a return curve, ask five questions: Which prices were visible? When did my order arrive? Which price, time, or other priority rule applied? How much could actually fill? Is PnL marked at the arrival midpoint or realized at a liquidatable price?

3.1 Connecting Three Chapters: From Request to Ledger to Evaluation

Read the table below as one order moving through the process. Prices are USD per share and amounts are USD; knowing a value also requires knowing when it became available.

Object	Quantity or unit	Effective or available time	Use in the next step
Quotes b, a and midpoint P_{arr}	USD/share	Quote event and its `available_at`	Set the arrival benchmark; does not promise a fill
Direction d and request Q	$d = \pm 1$; Q is signed shares	Sent after the decision is locked	Constrains requested quantity; does not post to cash
Actual fills f_j, total $F = \sum_j f_j$	Signed shares	After fill confirmation becomes available	Update H and execution cash flow
Residual $R = Q - F$	Signed shares	Unfilled quantity confirmed at cutoff	Enters opportunity-cost evaluation
Notional and explicit fees Ξ	USD	Actual fills and fee events	$C_1 = C_0 - \sum_j f_j P_j - \Xi$
Available mark and $C + HP_{\text{mark}}$	USD/share; USD	After the mark becomes available	Update NAV; a new mark applies only to later borrow intervals
IS	USD	After the window ends and its terminal value is available	Compare with the decision benchmark; do not deduct it from cash again

This chapter's static calculator uses a nonnegative order size with buy/sell direction supplied separately. Its nonnegative fill quantity Q_f should be converted to the signed objects used in Chapters 35–36 as $Q = dQ_{\text{abs}}$, $F = dQ_f$, and $R = d(Q_{\text{abs}} - Q_f)$. Each per-level sweep quantity $q_j \geq 0$ must likewise be multiplied by d before it updates a position; do not add a sell fill to holdings as if it were a buy.

Consider a small **constructed partial-fill** example connecting the chapters. The decision is to buy 100 shares at a decision price of 10.00. The arrival benchmark is 10.02. Only 60 shares fill at an average price of 10.05, with an explicit fee of 0.30; the mark available at cutoff is 10.10. Initially, there is only USD 2,000 in cash and no position. After the fill:

$$R = 100 - 60 = 40, \quad H_1 = 60,$$

$$C_1 = 2{,}000 - 60 \times 10.05 - 0.30 = 1{,}396.70.$$

If only 10.02 is available as the mark at fill time, NAV is $1{,}396.70 + 60 \times 10.02 = 1{,}997.90$; once 10.10 is available at cutoff, NAV is 2,002.70. The unfilled 40 shares do not enter the position, and opportunity cost does not reduce cash. Chapter 35 first proves this posting; Chapter 36 then decomposes IS for the same example: delay 2.00, execution 1.80, opportunity 3.20, and explicit fee 0.30, for a total of 7.30. A paper account that buys all 100 shares at the decision price has a cutoff value of 2,010, a difference of $2{,}010 - 2{,}002.70 = 7.30$. Thus actual NAV can exceed

its initial value while still falling short of the paper target. The fill outcome is specified in the example; it is not identified from a quote snapshot.

4. Mathematical Foundations

Scope of the Static-Snapshot Definitions

Let the best executable ask be a_t and the best executable bid be b_t, both quoted in USD per share or in the quote units of the traded contract. This chapter defines the following only when valid two-sided quotes are present and $b_t < a_t$:

$$m_t = \frac{a_t + b_t}{2}, \qquad S_t = a_t - b_t.$$

m_t is the midpoint, in the same units as the quote; S_t is the absolute quoted spread, in price per share or per unit. The relative spread $s_t = S_t/m_t$ is dimensionless, consistent with Chapter 35. If the tick size is δ, quote prices must follow the applicable venue's price-increment rules. This chapter's constructed example sets $\delta = \$0.01$/share; it does not state a rule for every security, venue, or date.

Order quantity Q is measured in shares; fill j has quantity q_j shares and price P_j in USD per share. Notional $N = \sum_j q_j P_j$ is in USD. When total filled quantity $Q_f = \sum_j q_j > 0$,

$$\text{VWAP} = \frac{\sum_j q_j P_j}{\sum_j q_j}.$$

VWAP is in USD per share. If there is no fill, VWAP is undefined and must not be replaced by zero. Net position H is in shares and cash C is in USD; ignoring other assets and liabilities, account equity at price p is $C + Hp$, in USD. Fills change cash and positions; they do not automatically change an externally recorded quote.

Time also has units and meaning. For a market-data record, `event_time` is the time marked for the market event; `available_at` is the earliest time the record can enter the research workflow; `ingested_at` is when the local system received it. `decision_time` is when the signal is actually produced; `order_send_time` and `order_receive_time` are when the client sends and the venue receives the order; `fill_time` is the fill time; `mark_time` is the valuation time. A returns interval must also state whether its endpoints are open or closed, the currency of account, and how cash flows are assigned. One timestamp field cannot replace this contract.

All example prices and quantities must be finite. The code uses `Decimal` for prices and amounts and integers for share counts; it does not silently convert NaN, infinity, or fractional shares into valid inputs. Time examples use ISO 8601 strings with a UTC timezone. Without real event data, time fields describe only the required data contract; they are not presented as observations.

5. Formal Definitions

Quotes and Marks

Bid and Ask. At a specified venue and time, the bid is the price a buyer is willing to pay and the ask is the price a seller is willing to accept. The order book can have multiple price levels; in this chapter, "best" means the highest bid and lowest ask currently visible in the book. A best quote consolidated across venues requires a separately defined data source, synchronization rule, and quote-eligibility standard.

Quoted Spread and Midpoint. When two-sided quotes are valid and uncrossed, the quoted spread is the best ask minus the best bid; the midpoint is their arithmetic average. The midpoint is a reference mark, not a guarantee that anyone will trade at that price. A one-sided market, a halt, an anomalously crossed quote, or asynchronous data from different venues does not meet this simplified definition.

Market Order. An order instruction that prioritizes execution as soon as possible. It generally does not set a price ceiling or floor when submitted and may consume multiple price levels. A specific venue may have protection prices, routing, pauses, or other rules. If a snapshot does not show enough quantity, this chapter's calculator reports only the visible depth covered and the residual; it does not infer how a real exchange would handle the residual.

The U.S. Securities and Exchange Commission's Investor.gov page on Limit Orders explains the buy and sell price bounds for limit orders and that execution is not guaranteed; this educational page does not replace applicable venue rules or a brokerage agreement.

Limit Order. An order with a price boundary. A buy limit order cannot fill above its limit; a sell limit order cannot fill below its limit. It may receive no fill or only a partial fill. A price touching the limit shows only that an aggregate price reached the boundary; it does not prove that the order reached the front of the queue, became active at the venue, or filled any quantity. Broker rules and order parameters may add conditions.

Order Book, Depth, and Queue. An order book is the state of unfilled orders a specific venue receives, displays, or maintains; what is visible depends on the data product and rules. Depth is the quantity visible at one or more price levels in the selected data view. A queue is the competition among orders at the same price or priority tier; queue position is an order's relative position under the applicable priority rules. A snapshot may show price levels and aggregate quantities without showing complete order identities, hidden quantity, arrival order, priority class, or the full event sequence at the time.

Liquidity. The ability to complete a buy or sale at a given time, price tolerance, quantity, and market state. Common observables include spread, depth, replenishment speed, market impact, fill rate, and time required to exit; each describes a different dimension. High trading volume does not mean any arbitrary quantity can execute at the midpoint.

Slippage. A signed deviation oriented so that positive values represent cost: Slippage $= d(P - P_{\text{ref}})$, where $d = +1$ for an aggressive buy and $d = -1$ for an aggressive sell; P is the fill price and P_{ref} is an explicit reference price, both in USD per share. State the benchmark and time. With the decision price as the benchmark, slippage mixes quote changes, delay, spread, and price movement within the book; the interpretation differs when the benchmark is the arrival midpoint. Slippage is not a fixed, independently additive fee.

Price Impact. The change in price after a trade relative to a chosen benchmark. In this chapter's fill-price decomposition, it is represented by the change in the post-trade midpoint at a specified horizon; this ex post change can include information, public news, and common market movements. Without a counterfactual or identification design, an observed markout alone does not prove that the trade caused the price change.

Adverse Selection. The risk that a liquidity provider's counterparty knows more about asset value and therefore trades aggressively only when the quote is unfavorable to the provider. It is a mechanism involving information asymmetry between counterparties, not a synonym for any unfavorable markout. Theoretical mechanisms, sample estimates, and causal explanations for individual fills are different levels of evidence.

Tick Size and Venue Fragmentation. Tick size is the smallest permitted price increment for quotes or trades under the rules in force for a particular security, venue, order class, and rule version. Venue fragmentation means that liquidity in the same security is distributed across multiple venues or trading systems. Venues can have different market models, data visibility, and priority rules; one venue's snapshot is not the executable depth of the whole market.

6. Derivations

1. From static displayed depth to an auditable sweep. Let the order direction be $d = +1$ for a buy and $d = -1$ for a sell. A buy consumes asks from low to high; a sell consumes bids from high to low. Let D_j be displayed quantity at level j. This recurrence uses a nonnegative requested size $Q_{\text{abs}} = |Q|$ and a nonnegative residual $R_0 = Q_{\text{abs}}$, with direction supplied separately:

$$q_j = \min(R_{j-1}, D_j), \quad R_j = R_{j-1} - q_j, \quad N = \sum_j q_j P_j.$$

If $R_K > 0$ after all visible levels in the snapshot have been traversed, the only conclusion is that the snapshot's visible depth did not cover this portion of the order. Filling R_K at the last level, at the midpoint, or automatically canceling it would require additional matching or order-parameter rules; this algorithm infers none of them. If $Q_f = Q_{\text{abs}} - R_K > 0$, VWAP is N/Q_f; if $Q_f = 0$, VWAP is undefined.

2. Decomposing execution price around the midpoint. Let $d = +1$ denote an aggressive buyer-initiated fill and $d = -1$ an aggressive seller-initiated fill; P is the fill price, m_0 is the reference midpoint at order arrival, and m_1 is the midpoint after a prespecified horizon. Define the per-share measures:

$$\begin{aligned} \text{Effective Spread} &= 2d(P - m_0), \\ \text{Realized Spread} &= 2d(P - m_1), \\ \text{Signed Price Impact} &= 2d(m_1 - m_0). \end{aligned}$$

Algebraically,

$$2d(P - m_0) = 2d(P - m_1) + 2d(m_1 - m_0),$$

so Effective Spread = Realized Spread + Signed Price Impact. All three quantities are in USD per share; multiply by filled shares to obtain amounts. The factor of 2 converts the one-sided per-share deviation from the arrival midpoint into a full-spread-equivalent measure. For a one-sided fill of nonnegative size $Q_f = |F|$, the one-sided cost measured from the arrival midpoint is $Q_f\, d(P - m_0) = Q_f$ EffectiveSpread/2; Q_f EffectiveSpread is its full-spread-equivalent amount, so do not deduct both for the same side. If slippage already measures actual fills relative to the arrival midpoint, it already includes that side's half-spread and book walking; adding those components again would double-count. This identity ensures consistency in the decomposition, but it does not prove that $m_1 - m_0$ is causal impact from the order, nor does it mean a cost model has accounted for every fee.

3. Connecting order cash flows to account equity. If initial cash is C_0 and initial holdings are H_0, buying q_j shares at price P_j gives $C_1 = C_0 - \sum_j q_j P_j$ and $H_1 = H_0 + \sum_j q_j$. Equity marked at m is $NAV_m = C_1 + H_1 m$; choosing a different liquidatable price p_{exit} is a separate valuation or exit assumption. Do not misstate the difference between these values as PnL already realized by the order itself.

4. Separating cost components without double-counting. For the symmetric example in this chapter, with Q shares traded on each side, no fees, and both sweeps against the same frozen book, round-trip loss can be calculated as "half the quoted spread at the arrival midpoint on each side + the buy-side amount paid above the best ask from book walking + the sell-side amount received below the best bid from book walking." If cost is already measured against the arrival price as Slippage $= d(P - P_{\text{ref}})$, the 350-share buy deviation is USD 6.50 and the hypothetical sale deviation is USD 4.50; both already include half-spread and book walking, so do not deduct them again. Comparing post-trade midpoint changes or impact with execution spread also depends on the markout horizon.

5. Why FIFO cannot be inferred from an assumption. If the rule specifies time priority at the same price, relative order within that price tier requires order-arrival and modification events. If the rules include displayed versus non-displayed priority, parity, or other classes, a timestamp alone is insufficient. Rule documents must match the venue, security class, effective version, and time period. Aggregate depth at one instant has none of this historical path, so inferring an order's precise queue rank from a snapshot is unsound.

7. Financial Interpretation

Liquidity providers post executable quotes in the book and bear the risks of price changes before quotes update and of being selected by better-informed counterparties; liquidity demanders cross the spread to obtain immediacy. Spreads can compensate for inventory, processing, competition, and information risk, but the spread value alone cannot identify how much comes from each source. The classic Glosten–Milgrom theory shows that, in a model with traders who have heterogeneous information and a market maker, trade direction updates the market maker's assessment of value and adverse selection can enter quotes. This chapter relies only on the mechanism as described at the publisher-abstract level; it does not expand that account into empirical coefficients or a law for all markets.

Identify cost components separately: the quoted spread is the difference between the best bid and ask at the time; book walking is the mechanical change in fill price when a sweep consumes visible levels; slippage is total fill deviation from a selected benchmark; price impact is an operational measure of post-trade price change, with causal interpretation requiring additional identification; and adverse selection is an information mechanism. They may be related, but similar names do not make them additive.

Venue priority models are not uniform. NYSE Equities describes market models across its trading venues; Nasdaq Equity 4 Rule 4757 specifies price, display status, and priority tiers for applicable order classes. These references to the venue pages and rules read on 2026-10-09 illustrate only that rules vary by venue. Execution analysis must also check the traded security, order type, venue routing, and rule version; these examples do not establish a universal ordering rule for other markets.

Market fragmentation means executable depth must account for multiple venues, different update times, routing, and order protection. Asynchronous consolidated quotes can combine prices that did not actually coexist. The SEC Investor.gov overview of order execution explains that brokers may route orders to exchanges, other venues, market makers, or ECNs. A transaction-cost study should record source, time-synchronization tolerance, venue coverage, and conditional missingness. If it observes a static book from only one venue, its conclusion is limited to that venue's snapshot.

Chapter 34 complete numerical example: all steps and conclusions

8. Numerical Example

Example A: Quotes, cross-level fills, holdings, and PnL. Use the constructed teaching snapshot in Section 3; the tick is set to USD 0.01 per share for this example only. The best bid is USD 100.00 per share and the best ask is USD 100.02 per share; $m_0 = (100.00 + 100.02)/2 = 100.01$ USD/share and $S = 100.02 - 100.00 = 0.02$ USD/share.

Continued: the same complete example

Level-by-level calculation for a 350-share purchase:

Level	Shares available to buy	Shares filled here	Price (USD/share)	Notional (USD)
Ask 1	200	200	100.02	20,004
Ask 2	300	150	100.04	15,006
Total	500	350	Total	35,010

VWAP $= 35{,}010/350 = 100.028571\ldots$ USD/share. Before the purchase, cash is USD 35,010 and holdings are zero; with no explicit fee, cash becomes zero and holdings become 350 shares. Marked at the arrival midpoint, equity is $0+350\times100.01 = 35{,}003.50$ USD, or USD 6.50 below initial cash. Relative to the arrival midpoint, the actual buy price is $100.028571\ldots - 100.01$ higher per share, for a total of exactly USD 6.50.

Now make a separate assumption: sell immediately against the same frozen snapshot. Sell 300 shares at 100.00 and 50 shares at 99.98, for proceeds of $300 \times 100 + 50 \times 99.98 = 34{,}999$ USD and VWAP $= 99.997142\ldots$ USD/share. The hypothetical round-trip PnL for both executions is $34{,}999 - 35{,}010 = -11$ USD. A midpoint mark is not a realized sale; conversely, the same-snapshot sale does not prove the real market could repeatedly provide these quantities.

The same USD 11 loss can be recomputed using the frozen-book decomposition: on each side, $350 \times 0.01 = 3.50$ USD is paid or given up relative to the arrival midpoint, for USD 7 total; the 150 buy shares above the best ask cost an additional $150 \times (100.04 - 100.02) = 3$ USD; the 50 sell shares below the best bid forgo $50 \times (100.00 - 99.98) = 1$ USD. Thus $7 + 3 + 1 = 11$ USD. This example has no fees, market response, or replacement orders.

Example B: Signs of effective spread, realized spread, and signed impact. For an aggressive buyer, $d = +1$, $P = 100.02$, $m_0 = 100.01$, and $m_1 = 100.03$ USD/share: Effective Spread $= 0.02$, Realized Spread $= -0.02$, and Signed Price Impact $= 0.04$ USD/share, so $0.02 = -0.02 + 0.04$. For an aggressive seller, $d = -1$, $P = 100.00$, $m_0 = 100.01$, and $m_1 = 99.98$: Effective Spread $= 0.02$, Realized Spread $= -0.04$, and Signed Price Impact $= 0.06$ USD/share, so $0.02 = -0.04 + 0.06$. In these constructed data, the later midpoint decline makes the aggressive seller's fill price favorable relative to the new midpoint, but it does not prove the seller had private information. A single fill or one-horizon markout cannot identify the source of information.

9. Python Implementation

The core function `sweep_displayed_book(book, aggressor_side, quantity)` reads visible contra-side depth, price level by price level, from one static book. It returns fills by price and size, total notional, VWAP, and the quantity not covered by the remaining visible depth. Prices use `Decimal`; share counts use positive integers. With no fill, VWAP is `None`. Invalid direction, nonpositive or noninteger quantity, nonfinite or nonpositive price, off-tick prices, duplicate levels, out-of-order levels, and a crossed book raise errors; the function does not silently fill in or sort inputs.

From the project root, run:

```
python code/ch34_market_microstructure.py --output-dir "runs/ch34_$(python -c 'import
    ↪  uuid; print(uuid.uuid4())')"
.venv/bin/python -m pytest tests/test_microstructure.py -q
```

The command creates `ch34_sweep_case.json` and `figures/ch34_order_book_sweep.png` in a unique `runs` subdirectory. By default, the I3 reader-package Notebook calls the shared microstructure logic to compute a fresh small static sweep and saves one figure in a unique `RUN_DIR`; it does not download external data or set a random seed. Use the outer runner described in `projects/PROJECT_READER.md` from the package root and save the execution copy and execution JSON. Do not rewrite the source Notebook or frozen QA outputs. The English Notebook language layer and full-package cold start are deferred to BIL23.

The policy for NaN is rejection: this kernel accepts only finite `Decimal` prices and integer share counts, not table missing values. Execution price, quantity, and cash amount have units of USD/share, shares, and USD, respectively. This function does not infer the data clock; the caller must first ensure the snapshot is eligible for the order's decision and arrival times. This code is a teaching example, not a real data frequency, tradable time series, or seeded experiment.

10. Visualization

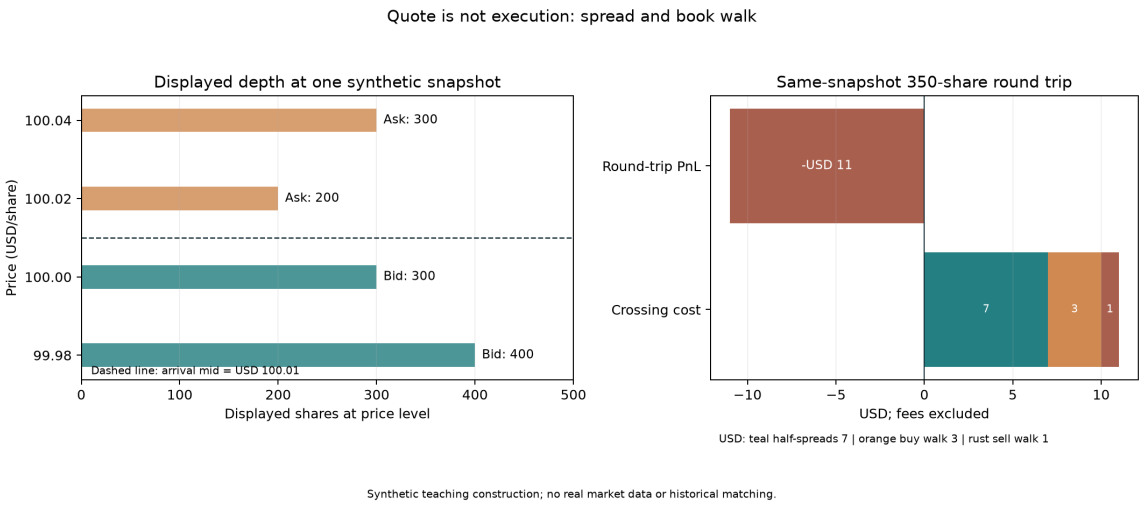


Figure 1: Price-level depth in a static single-venue order-book snapshot and a decomposition of hypothetical round-trip costs for 350 shares. Left: displayed shares on the horizontal axis and USD per share on the vertical axis; right: amounts in USD. All data are constructed for teaching.

The left panel plots displayed shares at each price level, in USD per share, with a dashed line for the arrival midpoint. The ask book first offers 200 shares at 100.02; a 350-share buy must then move to 100.04 to be fully covered in this static construction. The right panel, in USD, shows each side's half-spread and the round-trip decomposition of book walking alongside hypothetical round-trip PnL. The plotted sample is only a constructed snapshot with two bid levels and two ask levels; it does not represent a historical or live market.

Do not interpret "displayed shares" in the figure as guaranteed fills. The calculator deliberately assumes displayed quantity is executable at that snapshot. The figure does not show queues, cancellations, hidden quantity, latency, or routing across the whole market.

11. Real-Market Example

This batch did not download, read, or estimate real fills, order books, or return series. All numerical examples and the figure are labeled as constructed teaching material, so there is no real-data license, frequency, sample period, or data hash to report. The references in this section establish only that certain rule descriptions and data-format documents exist; they do not show that we observed the market during the periods they describe.

A future study using real order-book data should specify at least the vendor or exchange, product and license, rule and feed versions, security identifiers and venues, sample period, event-time precision and timezone, source publication or availability time, local receipt time, correction, deduplication, sorting, and gap-handling procedures, coverage of displayed and hidden orders, corporate-action and halt handling, and preservation of raw event records with a summary hash. A single snapshot is insufficient for historical queue reconstruction: the event messages must be sufficient to rebuild order additions, executions, reductions or cancellations, price changes, venue state, and same-timestamp ordering, together with the applicable matching rules.

The Nasdaq TotalView-ITCH 5.0 specification reviewed for this chapter encodes order additions, executions, cancellations, and deletions as separate message types, showing how an event feed covers different information from an aggregate snapshot. The reference is used only to illustrate message categories; this batch did not obtain, parse, or analyze real event messages from the feed and reports no fill statistics or market effects from it.

12. Common Mistakes

Treating the midpoint as an executable price. Incorrect: `entry_price = book.midpoint()`. In Example A, it values 350 shares at USD 35,003.50 while the purchase notional is USD 35,010, a USD 6.50 difference. Correction: label the midpoint explicitly as a benchmark or mark, and calculate notional from actual fills or from an explicitly stated sweep.

Treating a price touch as a limit-order fill. Incorrect rule: if a bar's high or low touches the limit, book a full fill. A touch does not reveal how much queue was ahead at that price, whether enough contra-side quantity traded during the interval, whether the order had arrived, or whether the high came before the low. Correction: label the rule as a price-touch proxy and include no-fill and partial-fill bounds; claim an order was matched only when the data and rules are sufficient to reconstruct it.

Treating every queue as FIFO. Incorrect rule: `queue_position = arrival_timestamp_rank`. This ignores venue rules, display class, priority changes after repricing or quantity reduction, and order status. Correction: identify the priority algorithm for the venue, product, and rule version, then replay the corresponding order-level events; if the information is missing, report "position not identifiable."

Calling every post-fill midpoint move causal impact or private information. m_1 is an ex post mark affected by concurrent news, other orders, and broad market movements. Correction: state the fixed horizon, report the signed markout identity first, then explain what counterfactual or identification strategy is needed for causal attribution.

Using OHLCV to prove micro-execution quality. The same daily bar can have open = close = 100, high = 101, and low = 99 without showing whether 101 or 99 came first, the quotes, the distribution of volume at each price, or the queue. A daily-frequency ledger can compare strategies under a prespecified bar rule; its results cannot establish queue fills for a limit order or order-book impact.

13. Quant Research Application

Suppose the research question is: "Does the return remain positive after buy- and sell-side execution frictions when a signal has greater predictive strength?" This is not the same question as whether "the signal significantly predicts returns." The workflow is:

1. **Define the estimand.** Specify the security universe, decision frequency, holding interval, base currency, how target quantity is formed, order direction, fee convention, and the cost-adjusted return to compare. State whether the target is a strategy-portfolio effect or an execution difference for a particular order type.
2. **Lock inputs available at the time.** Retain `event_time`, `available_at`, `ingested_at`, and venue for each market-data record; retain `decision_time` for the signal; generate `order_send_time`, `order_receive_time`, and `fill_time`. Verify that the source was eligible for use at the decision time and that clocks are synchronized. A snapshot with `available_at` later than `decision_time` cannot be backfilled as a decision input.
3. **Define falsifiable order rules.** Fix direction, quantity, limit or protection price, time in force, routing, and halt treatment before reading outcomes. Report execution price separately against the arrival midpoint, decision price, and close to avoid choosing a favorable benchmark after seeing results.
4. **Start with low-capability baselines.** Compare a frictionless closing-price mark, a quote-midpoint mark, a model using explicit spread and fee-rate proxies, and rule-driven replay when event-level data are available. Differences among baselines describe measurement limits; they do not license selecting one column as "the most realistic."
5. **Check executability.** Compare order quantity with visible depth at the same time and venue; quantify partial-fill rate, unfilled quantity, book walking, waiting time, and subsequent markout. Group volume, fill rate, and spreads by security and market state rather than hiding the tails in one aggregate average.
6. **Retain out-of-sample evaluation.** Calibrate parameters and cost models only on past training windows, then freeze execution in the next time window. When connecting to Chapter 29's ledger, record actual filled quantity, cash, holdings, and return interval. Without real order events, state that the result remains a model assumption; do not present it as a real backtest.
7. **Pre-register disconfirming conditions.** Do not claim a signal is executable if the tradable subset has too many partial fills, net cost by order direction consumes returns, returns

exist only at an unavailable closing price, small changes in cost parameters change the conclusion, or data became available after the decision. Positive out-of-sample returns after costs still require judgment about model uncertainty and multiple tests.

A daily OHLCV project can answer how a ranking signal performs under a prespecified daily-bar execution proxy. It cannot answer when a particular order reached the front of a queue at a price, or verify tick-level matching order. Chapter 35's cost estimates and Chapter 36's execution strategies can extend this question only when the observations and timing support them.

14. Chapter Summary

Quotes are a venue's displayed state; the midpoint is a reference mark; an order is a conditional instruction; fills are individual execution outcomes; and holdings and PnL are ledger states. When a buy sweeps the ask or a sell sweeps the bid, calculate price and quantity level by level. Quantity beyond the best price causes book walking; insufficient depth leaves an uncovered residual. Spread, slippage, impact, and adverse selection interact, but rely on different definitions and evidence.

Rules and queues depend on venue, order class, and version. A snapshot supports only limited claims about the time it shows; reconstructing a full history requires ordered events and the applicable rules. Data time, decision time, and order time cannot be collapsed into one date. OHLCV can support an explicitly stated bar proxy but cannot prove order-book execution quality. This chapter's calculator sweeps only visible depth on a single, static, uncrossed book; it does not modify Chapter 29's ledger and is not an execution engine for a future project.

15. Exercises

Question numbers are fixed for mapping between the body and the Loom notes; all values are constructed for teaching.

34-E01 (Concepts). In one sentence each, explain bid, ask, midpoint, and quoted spread. Can a snapshot showing only the best bid and ask establish that a 1,000-share buy order could fill at the midpoint? What information is missing?

34-E02 (Hand Calculation and PnL). Use Example A's order book. List the price and quantity at each level for a 350-share market buy and a hypothetical 350-share sale back, both VWAPs, the cash difference, and round-trip PnL. Explain why the -USD 6.50 mark at the arrival midpoint is not the same as round-trip execution PnL.

34-E03 (Hand Calculation and Limits). Verify the effective-spread identity using the buyer-initiated data in Example B. Change m_1 to USD 100.00 per share and recalculate Realized Spread and Signed Price Impact. Can this prove that the buy order caused the price decline?

34-E04 (Rules and Identification). A snapshot shows aggregate displayed quantity of 8,000 shares at one price. Can a researcher recover how much was already at that price before the order, the order's queue position, or how much quantity was later canceled? List the evidence needed for each.

16. Programming Exercises

34-P01 (Order-Book Sweep). From the project root, use the project's virtual environment to run the complete reference code block in Section 18. The code must include its own imports for `Decimal`, `Level`, `OrderBook`, and `sweep_displayed_book`, as well as construction of Example A's book. Assert the level-by-level fills for the 350-share buy, notional of USD 35,010, and VWAP. Then use a book with only 100 displayed shares on the ask side to verify 100 shares filled and 250 residual; also verify zero fill, full residual, and `None` VWAP with an empty ask side. Do not modify the core function to pretend it supports queues.

34-P02 (Input Boundaries and No Fill). From the project root, run the complete named tests listed in Section 18: crossed books, unsorted books, and off-tick prices should be rejected; a buy against an empty ask side should return zero fills, the full residual, notional of zero, and VWAP = `None`. Explain that `None` is this API's no-fill convention; NaN is not zero, and failing to check `filled` or applying `fillna(0)` to missing values can misrepresent a no-fill or undefined state as a zero-valued result.

17. Challenge Problems

34-C01 (Minimal Event-Replay Design). Design the smallest data contract that can verify the historical queue position and fill of a limit order. Specify event fields, availability-time filtering, same-timestamp ordering, rule version, order status, and at least three failure conditions. Do not assume a snapshot alone can replay the order.

Scoring: time/event fields, 3 points; venue rules and sequence, 2 points; add, execute, cancel/reprice state, 2 points; availability time and clock, 1 point; failure or non-identifiability boundary, 2 points.

34-C02 (OHLCV Limits and Research Statement). Construct two complete trade-print paths, each with four prints of 100 shares at event times 1, 2, 3, and 4; both must aggregate to O = 100, H = 101, L = 99, C = 100, V = 400 shares. Assume each print record becomes available 0.1 after its event time. Fix the same buy limit order in both paths: 100 shares, limit price 99, DAY through the close. The strategy decides and sends at $t = 2.5$; assume zero latency and arrival at that same time. Assume no immediately executable ask and no queue at price 99 on arrival. Separately state OHLCV-external aggressor-side and toy-priority assumptions: the order remains valid after arrival; later, a 100-share seller-initiated print at 99 first matches this sole resting order; other prints do not consume it. Explain these assumptions separately from the print paths: print order alone must not be taken as proof of a fill. List each print's time, price, and volume, explain why one path fills and the other does not, and give a bounded research statement and the next data needed.

Scoring: two feasible paths, 2 points; explanation of missing sequence, queue, and quantity, 3 points; cautious statement, 2 points; explicit next data needs and question, 3 points.

18. Solutions / Solution Hints

This section gives calculations for the E questions, reference assertions for P questions, and acceptable research reasoning for C questions. Open-ended problems do not have one universal market rule or algorithm; check answers against the stated scoring criteria and do not treat one example as a rule for every exchange.

34-E01. The bid is the buyer's displayed price at the venue and time; the ask is the seller's price; the midpoint is their average; and the quoted spread is ask minus bid. No. No seller has promised to provide a fill at the midpoint. More evidence is needed about executable contra-side depth when the order arrives, venue and routing, clock sequence, order protection conditions, applicable rules, and market state that could affect execution.

34-E02. Buy: 200 at 100.02 and 150 at 100.04, for notional of $(20{,}004 + 15{,}006) = 35{,}010$ USD and VWAP 100.028571... USD/share. Sell back: 300 at 100.00 and 50 at 99.98, for proceeds of $(30{,}000{+}4{,}999) = 34{,}999$ USD and VWAP 99.997142... USD/share. With no fees, hypothetical round-trip PnL is -11 USD. Initial cash is 35,010; after the buy, cash is zero and holdings are 350 shares. Marked at the 100.01 arrival midpoint, equity is 35,003.50 and mark PnL is -6.50. The midpoint mark does not execute a sale; the same-snapshot hypothetical sale incurs another USD 4.50 relative to the midpoint, so the two PnL measures differ.

34-E03. Original buyer example: Effective $= 2(100.02 - 100.01) = 0.02$; Realized $= 2(100.02 - 100.03) = -0.02$; Signed Impact $= 2(100.03 - 100.01) = 0.04$ USD/share, so $0.02 = -0.02 + 0.04$. If $m_1 = 100.00$, Realized $= 2(100.02 - 100.00) = 0.04$ and Impact $= 2(100.00 - 100.01) = -0.02$; still, $0.02 = 0.04 - 0.02$. This cannot prove causal price impact: concurrent news, other trades, and common market movements can also change m_1.

34-E04. No, not from the snapshot. Evidence includes order-level additions and identifiers, precise venue times, executions, reductions/cancellations/deletions, whether repricing retains priority, venue-state or auction events, data-feed display classes and gap checks, and the applicable market-model version. Verify that events were available at the relevant time and order them under venue rules; if key messages are missing, classify the queue as non-identifiable instead of inventing a FIFO position.

34-P01. From the project root, run `.venv/bin/python -` and paste the complete code below. It reads no Python variables from outside this block and does not copy the core algorithm.

```
from decimal import Decimal

from quant_course.microstructure import Level, OrderBook, sweep_displayed_book

book = OrderBook(
    bids=(Level(Decimal("100.00"), 300), Level(Decimal("99.98"), 400)),
    asks=(Level(Decimal("100.02"), 200), Level(Decimal("100.04"), 300)),
)
buy = sweep_displayed_book(book, "buy", 350)
assert [(fill.price, fill.size) for fill in buy.fills] == [
    (Decimal("100.02"), 200),
    (Decimal("100.04"), 150),
]
assert buy.notional == Decimal("35010.00")
```

```
assert buy.vwap == Decimal("35010") / 350

thin = OrderBook(bids=(), asks=(Level(Decimal("100.02"), 100),))
partial = sweep_displayed_book(thin, "buy", 350)
assert (partial.requested, partial.filled, partial.remaining) == (350, 100, 250)
assert partial.notional == Decimal("10002.00")
assert partial.vwap == Decimal("100.02")

empty = OrderBook(bids=(), asks=())
no_fill = sweep_displayed_book(empty, "buy", 350)
assert (no_fill.filled, no_fill.remaining, no_fill.notional, no_fill.vwap) == (
    0, 350, Decimal("0"), None
)
```

The named tests listed under 34-P02 cover crossed books, unsorted books, off-tick inputs, and an empty contra side. Residual quantity is not filled quantity; this function does not simulate a queue or subsequent handling of the residual order.

34-P02. Complete named validation is performed by the two pytest nodes below: the first parameterized test covers crossed, unsorted, and off-tick books; the second covers an empty contra side. From the project root, run:

```
.venv/bin/python -m pytest -q \
  tests/test_microstructure.py::test_rejects_crossed_unsorted_or_off_tick_book \
  tests/test_microstructure.py::test_empty_contra_side_returns_no_fill
```

The crossed-book case violates the required uncrossed-book condition because the best bid is not below the best ask. Bids must be listed from highest to lowest and asks from lowest to highest; the function rejects out-of-order levels rather than silently sorting them. An off-tick price violates the permitted price increment and is rejected rather than rounded. An empty ask side returns `filled=0`, `remaining=requested`, `notional=Decimal("0")`, and `vwap=None`. `None` is the API's explicit no-fill marker; NaN is neither zero nor a trade at price zero. Check `filled` first. Do not apply `fillna(0)` to `None` or NaN and then treat the result as an executed fill or a zero-cost outcome.

34-C01. One acceptable answer can include: venue/security/order ID; message type (add, execute, cancel, delete, reprice); price, remaining displayed/hidden quantity, side, order type, and display class; source `event_time` and sequence number, source `available_at`, and local `ingested_at`; client-send, venue-receive, acknowledgement, and fill times; venue session/trading state; feed-sequence continuity; effective rule version and priority; and a unified UTC clock with a policy for same-timestamp ordering. Include only research inputs with `available_at <= decision_time`. If sequence numbers have gaps, clocks cannot order events, reused order IDs cannot be disambiguated, the rule version is unknown, or required messages are missing, refuse to claim a certain queue position or fill; report the interval as non-identifiable.

34-C02. In the table, t_1, t_2, t_3, t_4 are event times 1, 2, 3, and 4. Every print is 100 shares and becomes available 0.1 after its event time; count each share once in volume:

Path	Time and price (shares)
A	$t_1 : 100$ (100), $t_2 : 101$ (100), $t_3 : 99$ (100), $t_4 : 100$ (100)
B	$t_1 : 100$ (100), $t_2 : 99$ (100), $t_3 : 101$ (100), $t_4 : 100$ (100)

Both paths have $O = 100, H = 101, L = 99, C = 100, V = 400$. The same DAY buy order for 100 shares at 99 is decided and sent at $t = 2.5$, with zero assumed latency and arrival at that time; assume it remains active through the close. At arrival, the ask is above 99 and there is no queue at 99. In path A, a 100-share seller-initiated print at 99 occurs at $t = 3$; under the stated toy-priority rule, it matches this sole resting buy order first, so the order fills completely. Other prints do not consume the order. In path B, the 99 print occurred at $t = 2$ before arrival; there is no later seller-initiated print at 99, so the order remains unfilled at the close. Aggressor side, continued order validity, an empty queue, and toy priority are assumptions outside OHLCV; print order alone does not prove a fill. A cautious conclusion is: "Performance under a daily-bar proxy does not prove a real limit-order fill; identical OHLCV cannot identify order sequence, contra-side quantity, or queue." Assessing fill rates, opportunity cost from non-fills, and markouts requires venue-level prints or order events with `event_time` and `available_at`.

19. Further Reading

Must Know

- U.S. Securities and Exchange Commission / Investor.gov, "Limit Orders": Check the price bounds for buy and sell limits and the fact that a fill is not guaranteed. Use the page only for basic order properties; specific venues and broker instructions remain subject to applicable rules and agreements.
- U.S. Securities and Exchange Commission / Investor.gov, "Executing an Order": Read the overview of how brokers may route orders to an exchange, another venue, a market maker, or an ECN; it is not a record of a specific order's routing or an evaluation of best execution.
- Nasdaq Equity 4, Rule 4757: Use only when discussing Nasdaq priority by price, time, and display class for specified order classes. Check the rule version, effective date, and applicable securities.

Should Know

- NYSE Equities, "Equities": Read the exchange's short descriptions of its different market models and understand why FIFO cannot be applied uniformly even across one exchange group; consult the relevant venue rule for a specific conclusion.
- Nasdaq, TotalView-ITCH 5.0 Technical Specification: Review message types such as order addition, execution, cancellation, and deletion to understand how an event stream differs from a snapshot. The specification is not real trading data for this chapter; do not draw conclusions from clauses that were not read.

Advanced

- Lawrence R. Glosten and Paul R. Milgrom (1985), "Bid, ask and transaction prices in a specialist market with heterogeneously informed traders," *Journal of Financial Economics* 14(1), 71–100, DOI 10.1016/0304-405X(85)90044-3. Only the publisher abstract was reviewed here. It is useful for further study of informed trading and quote formation; this chapter uses only a mechanism-level description consistent with the abstract.

20. References

For source-by-source review scope, see `course/zh/chapters/ch34/source_notes.md` and `references/sources.json`. The sources support order rules, message categories, and an abstract-level mechanism description; they do not provide the constructed market data or cost parameters. Chapters, rules, and full papers not read are not used as evidence.

Chapter 35 — Transaction Costs

Prior engineering evidence is cited under its original acceptance status. Through-line: map optimizer target weights and ideal prices through spread, slippage, impact, fees, financing, and borrow costs to scenario-specific fill quantities, cash changes, and ending net asset value (NAV). Prevent frictions already embedded in fill prices from being charged again as cash expenses, and establish self-financing and capacity-sensitive boundaries for net-return accounting. Prerequisites: Chapter 29, Backtest Ledgers; and Chapter 34, Market Microstructure. After reading, you should be able to separate the pricing logic for commissions, spreads, slippage, temporary impact, and permanent impact; derive the closed-form purchasable quantity under a self-financing constraint; post cash entries without double counting; hand-reconcile a complete implementation-shortfall ledger with partial fills and multi-asset frictions; and build turnover and capacity sensitivity analyses, including a strategy's break-even AUM.

1. Learning Objectives

In quantitative investing and multifactor research, paper portfolios and portfolios net of costs use different assumptions about fills and expenses. An optimizer that ignores execution constraints may recommend turnover beyond what a strategy can support. Realized results also depend on spreads, impact, commissions, financing, borrow, execution windows, and fill data. The examples in this course use explicit constructed inputs to demonstrate accounting methods; they do not describe a real strategy or market sample.

After completing this chapter, you should be able to demonstrate five professional competencies:

1. **Cost categories and units.** Distinguish explicit fees (commissions, exchange fees, and taxes), implicit costs (bid-ask spread, slippage, temporary impact, and permanent impact), and position-carry costs (financing interest and borrow fees). Use USD per share, shares, total cash in USD, basis points (bps, where 1 bps = $0.01\% = 10^{-4}$), and annualized percentages consistently; do not mix dimensions in an equation.
2. **Avoiding double charges and reconciling the ledger.** Understand how implicit costs enter the execution price. Prove that once the fill price includes the half-spread and temporary impact, debiting the actual fill notional from cash must not be followed by a second cash debit for spread or impact. Prove that permanent impact changes the subsequent reference-price path and is reflected through valuation, rather than posted again as a cash flow.
3. **Solving for self-financing trades.** Understand how trading frictions feed back into purchasable asset quantities. Given cash and target weights, write the nonlinear self-financing equation with slippage and fees, and derive the corrected purchase quantity for single- and multi-asset cases so a backtest does not overdraw cash.
4. **Reconciling implementation shortfall (IS).** Attribute the difference between a paper decision and the executed outcome to delay cost, execution friction, opportunity cost on

unfilled shares, and explicit fees. Prove the algebraic decomposition and its equality to the direct cash-difference method.

5. **Capacity and calibration boundaries.** Apply square-root and linear impact models within their limits; distinguish constructed scenario inputs from empirical calibration to institutional data; and derive the relationship among turnover, AUM, and net returns, including the AUM at which a strategy reaches break-even.

Completion criterion: you can independently hand-calculate the multi-asset example in Section 8 and match its fill quantities, prices, cash balance, fee details, and ending NAV to the cent. You can also identify the code and accounting defects in the double-charge example in Section 12 and state the correct repair.

2. Why Quant Needs It

A quantitative strategy's life cycle runs from a signal to net profit:

```
[Factor signals / Alpha model]
        |
        v (decision time: signal_time)
[Portfolio optimizer: target weights w*]
        |
        v (order submission and routing: order_send_time)
[Execution algorithm and matching: trade_time] ---> spread, impact, slippage,
    ↪ commission
        |
        v (actual fills: F = Q - R)
[Account and ledger update: fill_time] ---> cash debit/credit, position update,
    ↪ financing/borrow accrual
        |
        v (valuation: mark_time)
[NAV and capacity assessment] ---> determines the capital scale the strategy can
    ↪ support
```

Ignoring trading costs overstates the net return a paper portfolio can realize:

- **High-turnover scenario:** Costs accumulate on actual traded notional in the tradable risky assets. Suppose a constructed example invests the entire portfolio in tradable stocks, has one-sided annual turnover $\tau_{\text{turnover}} = 15$, and incurs an average cost of 15 bps per dollar of actual traded notional. Using total stock turnover of $2\tau_{\text{turnover}}A$, the cost is about $2 \times 15 \times 15 = 450$ bps of beginning-of-period NAV. Changes in the cash weight do not enter stock traded notional. This is arithmetic only, not a universal input for any strategy or market.
- **Liquidity differences:** The same share quantity can incur different costs under different quoted depth, volume, and volatility conditions. Without historical order-book and trade data for each security, a researcher can run transparent scenarios but cannot present an assumed spread as a market fact.
- **Borrow constraints:** Short orders also depend on borrow availability, rates, recalls, and the conditions for buying to cover. Without timestamped historical borrow records, a

backtest must use explicit scenario inputs; it cannot assume that every stock is always borrowable.

- **Scale sensitivity:** As assets under management rise, target orders may become larger relative to available liquidity. How costs scale depends on the execution model and calibration data. This chapter uses constructed functions to illustrate that dependence; it gives no universal capacity threshold.

Chapter 29 established a discrete self-financing ledger, and Chapter 34 unpacked limit order books and market matching. This chapter connects the two: it models spreads and impact, feeds them into Chapter 29's cash ledger, and shows how trading frictions affect portfolio compounding. It also supplies the cost-accounting foundations for Chapter 36's execution methods—TWAP, forecast-volume VWAP, POV, and implementation shortfall.

3. Intuition

3.1 A Used-Car Analogy

A used-car purchase is an intuitive way to think about trading costs. Suppose the fair midpoint valuation of a model is USD 20,000:

1. **Bid-ask spread:** A dealer will pay USD 19,500 if you sell the car (the bid) and ask USD 20,500 if you buy the same model (the ask). The USD 1,000 difference is the quoted spread. At the USD 20,000 midpoint valuation, a buyer immediately incurs half the spread, or USD 500.
2. **Explicit commission and fees:** The motor-vehicle authority charges a USD 200 transfer fee and transaction tax. This is an explicit cash expense.
3. **Market impact:** Suppose you need 50 cars that afternoon to start a rental business, but local dealers have too little inventory and must source more cars at higher prices. The first car costs USD 20,500, the twentieth costs USD 21,000, and the fiftieth costs USD 22,000. Your large, urgent purchase has moved prices against you: this is market impact.
4. **Temporary versus permanent impact:** After you leave with the 50 cars, the buying pressure subsides and the dealer's ask falls from USD 22,000 to USD 20,800. To compare like with like, suppose the pre-trade midpoint is USD 20,000 and the half-spread is USD 500, so the pre-trade ask is USD 20,500. During the busiest part of the purchase, the ask reaches USD 22,000. If the post-trade ask is USD 20,800 and the half-spread remains USD 500, the corresponding midpoint is USD 20,300. This constructed path labels the USD 1,200 difference between the busy-period ask and the post-trade ask as temporary, and the USD 300 difference between the post-trade and pre-trade midpoints as permanent. It is a specified hypothetical path: a finite observation window cannot establish that impact is permanent or that prices will recover at any particular speed.
5. **Financing and stock borrow:** If you borrow money from a bank to buy the car, you pay financing interest. If you borrow a friend's car, sell it, and promise to buy another one to return in a month, you pay a rental fee analogous to stock-borrow cost.

3.2 Why You Must Not Charge the Same Cost Twice

A common accounting error is to charge a friction twice. Suppose you buy a car from a dealer for USD 20,500. The fair midpoint value is USD 20,000, so the purchase price already includes the USD 500 half-spread. You also pay a USD 200 transfer fee.

Your cash falls by USD 20,500 + USD 200 = USD 20,700. You receive one car, valued at USD 20,000 at the midpoint. Your net assets therefore fall by USD 700: the USD 500 half-spread plus the USD 200 transfer fee.

The USD 500 spread loss is already reflected in the USD 20,500 purchase price. If backtest code debits USD 20,500 for the car and then separately debits a USD 500 "spread fee," cash falls by USD 21,200. The spread has been charged twice.

The accounting rule is simple: **the fill price absorbs spread and impact, explicit fees are debited separately, and modeled permanent price changes enter through valuation; never post the same cost twice.**

4. Mathematical Foundations

Before defining each trading cost, we fix the notation, variable dimensions, and units.

4.1 Core Variables and Dimensions

At a rebalance time, let the portfolio contain N risky assets and one risk-free cash account.

- $P_t^{\text{mid}} \in \mathbb{R}_{++}^N$: midpoint-price vector at time t, in USD per share.
- $a_t, b_t \in \mathbb{R}_{++}^N$: best ask and bid vectors, in USD per share, with $b_t < P_t^{\text{mid}} < a_t$.
- $S_t = a_t - b_t \in \mathbb{R}_{++}^N$: quoted spread in USD per share. The relative spread is $s_t = S_t / P_t^{\text{mid}}$, dimensionless or reported in bps.
- $Q \in \mathbb{R}^N$: signed parent-order quantity in shares. $Q_i > 0$ is a buy and $Q_i < 0$ is a sell.
- $F \in \mathbb{R}^N$: signed filled quantity in shares. It satisfies $|F_i| \le |Q_i|$; when $F_i \neq 0$, $\operatorname{sgn}(F_i) = \operatorname{sgn}(Q_i)$. Zero fills are allowed.
- $R = Q - F \in \mathbb{R}^N$: signed cutoff remainder in shares.
-

> **Nonzero fills and empty sums**
>
> Define the nonzero-fill set $\mathcal{I}_F = \{i : F_i \neq 0\}$. For each filled asset $i \in \mathcal{I}_F$, its fill VWAP $P_i^{\text{fill}} > 0$ is in USD per share. If $F_i = 0$, fill VWAP is undefined (represented by `NaN` or `None`) and must never be set to zero. Every sum involving P_i^{fill} or $\operatorname{sgn}(F_i)$ is over $i \in \mathcal{I}_F$; an empty sum is zero.

- $C_t \in \mathbb{R}$: available cash in USD at time t. $C_t < 0$ means cash borrowing or leverage.
- $H_t \in \mathbb{R}^N$: holdings in shares at time t. $H_{i,t} < 0$ means a short position.
- $\text{NAV}_t \in \mathbb{R}$: net asset value in USD, marked at the stated valuation prices.

4.2 Basis Points and Percentages

Small trading proportions are often stated in basis points (bps):

$$1 \text{ bps} = 0.01\% = 10^{-4} = 0.0001.$$

For example, 15 bps equals 0.15% or 0.0015. At a share price of USD 50.00, a 20-bps spread corresponds to an absolute amount of

$$\Delta P = 50.00 \times 20 \times 10^{-4} = \$0.10/\text{share}.$$

4.3 From Confirmed Fills to Valuation: Post First, Then Compare Benchmarks

Continue the small connection example from Chapter 34. A request for $Q = 100$ shares does not change holdings. Only the actual fill of $F = 60$ shares changes cash from USD 2,000 to USD 1,396.70 and shares from 0 to 60. The fill price of USD 10.05 already includes the execution deviation from the quote; debit only the explicit USD 0.30 fee separately. When the mark moves from USD 10.02 to USD 10.10, cash stays unchanged, the position value rises by $60(10.10 - 10.02) =$ USD 4.80, and NAV moves from USD 1,997.90 to USD 2,002.70.

The four IS components for this buy are

$$100(10.02 - 10.00) + 60(10.05 - 10.02)$$
$$+40(10.10 - 10.02) + 0.30 = 7.30.$$

The USD 3.20 opportunity cost did not buy any shares, and the USD 2.00 delay cost was not paid to anyone. Both are attribution components. If total IS were debited from cash after the fill had already been posted, NAV would be incorrectly reduced to USD 1,995.40, a second charge of USD 7.30. Section 6.2 proves the general signed identity; Section 8 preserves the full multi-asset, corporate-action, and financing ledger example.

5. Formal Definitions

This section gives the mathematical and financial definitions of the main components of trading costs.

5.1 Commissions and Explicit Fees

Definition 5.1 (Explicit Fees). Explicit fees are cash expenses charged directly for trading activity by a broker, clearing house, exchange, or regulator. Denote them by $\Xi \geq 0$, in USD. They commonly include:

1. **Per-share commission:** $\Xi_{\text{share}} = c_{\text{share}} \sum_i |F_i|$, where c_{share} is a fee per share (for example, \$0.005/share).
2. **Notional commission:** $\Xi_{\text{notional}} = c_{\text{notional}} \sum_i |F_i| P_i^{\text{fill}}$, where c_{notional} is a dimensionless rate (for example, 1.5 bps = 0.00015).
3. **Other transaction fees and taxes:** where a model includes these items, their schedules must be specified for the relevant venue, product, and account. This chapter makes no claim about the current fee or tax rules of any jurisdiction. The sell-side term in the equation below is an optional modeling term, not a universal legal rule.

The combined explicit-fee function is

$$\Xi = \sum_{i \in \mathcal{I}_F} \left(c_{\text{share},i} |F_i| + c_{\text{notional},i} |F_i| P_i^{\text{fill}} + \mathbb{I}_{\{F_i < 0\}} \tau_{\text{tax},i} |F_i| P_i^{\text{fill}} \right).$$

Conditions and properties: In this chapter's ledger model, commission is fill-based and applies only to **actual fills** F_i; an unfilled remainder R_i does not incur the modeled fill commission. Account-specific order-submission or cancellation fees are outside this model.

5.2 Bid-Ask Spread

Definition 5.2 (Effective and Realized Spread). Let an order reach the market at time t_{arr}, when the best bid is b, the best ask is a, and the midpoint is $P^{\text{mid}} = (a + b)/2$.

1. **Quoted spread:** $S^{\text{quote}} = a - b$. The relative quoted spread is $s^{\text{quote}} = (a - b)/P^{\text{mid}}$.
2. **Effective half-spread cost:** For a child order filled at P^{fill} at time t, the one-sided move from the arrival midpoint equals half the spread in the no-impact limit. For signed fill quantity F, a buy ($F > 0$) executes at the ask and a sell ($F < 0$) at the bid:

$$P^{\text{fill}} = P^{\text{mid}} + \text{sgn}(F) \cdot \frac{S^{\text{quote}}}{2}.$$

The total spread amount paid on the fill, in USD, is

$$\text{Cost}_{\text{spread}} = |F| \cdot \frac{S^{\text{quote}}}{2} = F \cdot (P^{\text{fill}} - P^{\text{mid}}).$$

Condition: This formula assumes the order size does not exceed displayed quantity at the best level. When an order consumes multiple levels, depth slippage and market impact must also be considered.

5.3 Slippage

Definition 5.3 (Slippage). Slippage is the adverse deviation of the actual fill price P^{fill} from a specified reference price P^{ref}:

$$\text{Slip} = \text{sgn}(F) \cdot (P^{\text{fill}} - P^{\text{ref}}).$$

- If the reference price is the decision-time midpoint P^{decision}, slippage includes market drift during decision delay and all execution frictions after arrival.
- If the reference price is the arrival midpoint P^{arrival}, slippage includes half-spread and immediate market impact.
- The total slippage amount is $F \cdot (P^{\text{fill}} - P^{\text{ref}})$. Positive slippage means execution was worse than the reference benchmark.

5.4 Market Impact: Temporary versus Permanent

A large order can disturb supply and demand and move prices against the trader. Almgren and Chriss (2000) formalized a decomposition of this effect into temporary and permanent impact.

Definition 5.4 (Temporary Impact). A trader demanding immediate liquidity temporarily consumes order-book depth, making the fill VWAP worse than the contemporaneous midpoint in the absence of the trader's order. In this model, this component enters the fill price but not the post-trade reference midpoint. That modeling convention does not establish that real prices recover quickly; recovery speed must be tested with event data.

Suppose a trade is executed at rate $v = |F|/\Delta t$ over a window Δt. Temporary impact raises a buy price and lowers a sell price:

$$P^{\text{fill}} = P_t^{\text{mid}} + \text{sgn}(F) \cdot \left(\frac{S_t}{2} + g(|F|, \Delta t) \right),$$

where $g(|F|, \Delta t) \geq 0$ is the temporary-impact function. A common classical specification is a power law:

$$g(|F|, \Delta t) = \eta \cdot \sigma_{P,1\text{d}} \cdot \left(\frac{|F|}{V_{\text{slot}}} \right)^{\alpha}.$$

Fix the volatility measurement window in advance. For example, let $\sigma_{r,1\text{d}}$ be the sample standard deviation of daily returns over the 60 complete trading days before the decision, excluding post-decision data. Define $\sigma_{P,1\text{d}} = P_t^{\text{mid}} \sigma_{r,1\text{d}}$ in USD per share; this is the standard deviation of the one-day price increment. Then g is in USD per share. The coefficient η and participation-rate exponent are dimensionless; V_{slot} is total market volume in the slot; $\eta > 0$ is a liquidity coefficient; and $\alpha \in [0.5, 1.0]$ is the impact exponent ($\alpha = 1$ is linear impact and $\alpha = 0.5$ is square-root impact).

Definition 5.5 (Permanent Impact). In this model, a price change attributed to the trader's order and retained in the post-trade reference midpoint is called permanent impact. The formula is a definition of a model component; it does not mean that a finite observation window has established permanent real-world impact, that private information caused the change, or that a causal effect has been identified:

$$P^{\text{mid}}_{t+\Delta t} = P^{\text{mid}}_t + \operatorname{sgn}(F) \cdot h(|F|),$$

where $h(|F|) \geq 0$ is the permanent-impact function. Huberman and Stanzl (2004) discuss restrictions on round-trip trading within their price-impact framework, impact structure, and no-quasi-arbitrage/no-price-manipulation assumptions. Their result must not be generalized beyond those model conditions as an empirical law. In this chapter's simplified specification, linear permanent impact is only a teaching model:

$$h(|F|) = \gamma \cdot \sigma_{P,1\text{d}} \cdot \left(\frac{|F|}{V_{\text{day}}} \right),$$

where $\gamma > 0$ is dimensionless and h is in USD per share. The same ex ante volatility window is used, and V_{day} is a daily-volume proxy. The parameters are not market-calibrated; the equation defines a scenario only.

Mapping to the execution engine. The equations above are abstract volatility-scaled scenarios. The current fill model in `quant_course.execution_costs` uses

$$g_{\text{engine}} = k_{\text{temp}} P_{\text{current}} \left(\frac{|F|}{V_{\text{realized}}} \right)^{\alpha},$$

$$h_{\text{engine}} = k_{\text{perm}} P_{\text{current}} \frac{|F|}{V_{\text{realized}}}.$$

The coefficients k_{temp} and k_{perm} are dimensionless scenario coefficients. The current implementation neither estimates daily volatility nor uses a real L2 queue. With the same reference price and denominator, one can write $k_{\text{temp}} = \eta \sigma_{r,1\text{d}}$ as a parameterization correspondence only; this does not mean the implementation takes calibrated volatility as an input. The denominator of the permanent term above is slot realized volume, whereas the abstract formula uses a daily-volume proxy; if those denominators differ, the coefficients cannot be equated directly. Realized volume affects fill capacity and fill price only; it must not leak into ex ante VWAP/POV scheduling. The Chapter 36 AC diffusion parameter σ is in USD per share per square root of time; neither $\sigma_{r,1\text{d}}$ nor $\sigma_{P,1\text{d}}$ has that unit.

For example, with arrival price 50, daily return standard deviation 2%, slot participation 4%, $\eta = 0.5$, and $\alpha = 0.5$, we have $\sigma_{P,1\text{d}} = 1.00$ USD/share and $g = 0.10$ USD/share. If the absolute spread is $S = 0.04$, the buy fill price is 50.12. The equivalent temporary scenario coefficient is $k_{\text{temp}} = 0.01$, not 0.5. This arithmetic example has no calibration meaning.

Key distinction:

- **Temporary impact** $g(|F|)$ affects only this order's **fill price** $P^{\mathbf{fill}}$; it does not change the model's post-trade equilibrium midpoint.
- **Permanent impact** $h(|F|)$ changes the **post-trade midpoint** $P^{\mathbf{mid}}_{t+\Delta t}$, which affects subsequent arrival prices and the mark-to-market value of positions already held.

5.5 Financing Interest and Stock-Borrow Fees

Definition 5.6 (Cash-Financing Cost). When an investor buys assets with leverage and the cash balance becomes negative ($C_t < 0$), the broker charges interest under the agreed borrowing rate. Between ledger events t_{j-1} and t_j, the cash balance is unchanged. Let Δd_j be the actual number of days and D the day-count basis (this batch uses ACT/360). The signed net interest for the interval is

$$I_j = C_{j-1}\, r(C_{j-1}) \frac{\Delta d_j}{D}, \qquad r(C) = \begin{cases} r_{\text{credit}}, & C \geq 0, \\ r_{\text{debit}}, & C < 0. \end{cases}$$

With negative cash, the financing expense is $|C_{j-1}| r_{\text{debit}} \Delta d_j / D$, so $I_j < 0$. With positive cash, $I_j > 0$. At month-end or after a fill or mark event, I_j is posted to cash immediately, and the next interval uses the updated balance as its interest base. Each interval therefore uses simple ACT/360 interest, with discrete event-level capitalization across intervals; it is not equivalent to treating all monthly trades as if they occurred at one time and charging interest once for the month.

Definition 5.7 (Stock-Borrow Fees). An investor holding a short position ($H_{i,t} < 0$) borrows shares from a lender and pays a borrow fee. The fee is calculated daily from the absolute short market value and the annual borrow rate:

$$B_j = \sum_{i: H_{i,j-1} < 0} |H_{i,j-1}| \cdot P^{\text{available}}_{i,j-1} \cdot b_i \cdot \frac{\Delta d_j}{D},$$

where $b_i \geq 0$ is the annual borrow rate for asset i. A rate on an extremely scarce security is often called a special borrow rate.

Financing and borrow use Chapter 29's single monotone ledger event clock. For interval $[t_{j-1}, t_j]$, first calculate I_j and B_j using the opening cash, short quantities, and the latest mark already available at the interval start, then post $I_j - B_j$ to cash. Only after that do we process the fill or new mark at t_j. A price first observed at the end of an event affects later borrow intervals; it is not backfilled into earlier costs. A month with no fills must still be settled through month-end. If no mark is available for a short asset, stop and report the missing mark; do not substitute a rate or future price.

In this batch's constructed Project 4 execution window, each slot-end midpoint is recorded as a new mark event, even when the slot has no fill. The same rule applies to a position that remains open but is not rebalanced that month. The old mark accrues through slot end; the new mark is used only for later intervals. This slot-end mark availability is an explicit experiment convention, not a claim that real market quotes are available with the same latency.

Segmented accounting example (ACT/360, zero commission): On day 0, cash is USD 100,000. Short 100 shares of A at USD 100, with the same USD 100 mark available, so cash after the short sale is USD 110,000. The annual cash rate is 4% and A's annual borrow rate is 1.5%. For day 0 through day 10, accrue on the opening cash and short position: interest is $110{,}000 \times 4\% \times 10/360 =$ USD 122.2222, and borrow is $100 \times 100 \times 1.5\% \times 10/360 =$ USD 4.1667. Post the net amount, then buy 50 shares of B at USD 100 on day 10; cash becomes USD 105,118.0556. From day 10 through day 30, interest is $105{,}118.0556 \times 4\% \times 20/360 =$ USD 233.5957, and A's borrow fee is $100 \times 100 \times 1.5\% \times 20/360 =$ USD 8.3333. Total interest is USD 355.8179. If the day-10 mark remains USD 100, total borrow is USD 12.50. If a new USD 105 mark is first observed on day 10, it applies only to the next 20 days: borrow for that interval is USD 8.75 and total borrow is USD 12.9167. It does not revise the first 10 days. If all fills were treated as occurring at month-end, the interest and borrow costs on the opening cash and short position from day 0 through day 10 would be omitted.

Note: If a shorted security pays a dividend while the short is open, the short seller must compensate the lender for the full cash dividend (a manufactured dividend). This is debited directly from the cash account.

5.6 Turnover Conventions

Definition 5.8 (Turnover and the Tradable-Security Set). Let the tradable risky securities be $\mathcal{S}_{\text{risky}} = \{1, \ldots, N\}$, in addition to a cash account. Let the pre-rebalance weights be w_{t^-} and target weights be w_t^*.

1. **One-sided risky-asset turnover:**

$$\tau_{\text{risky, one-sided}} = \frac{1}{2} \sum_{i \in \mathcal{S}_{\text{risky}}} |w_{i,t}^* - w_{i,t^-}|.$$

2. **Total requested risky-asset notional turnover:**

$$\tau_{\text{risky, notional}} = \sum_{i \in \mathcal{S}_{\text{risky}}} |w_{i,t}^* - w_{i,t^-}| = 2\tau_{\text{risky, one-sided}},$$

   corresponding to requested notional $V_{\text{requested}} = \tau_{\text{risky,notional}}\,\text{NAV}$. Actual commissions, spread, and temporary impact are charged only on actual risky-asset traded notional $\sum_{i \in \mathcal{S}_{\text{risky}}} |F_i| P_i^{\text{fill}}$. The unfilled part enters opportunity-cost attribution only.
3. **All-asset weight reallocation** may be defined separately as

$$\tau_{\text{all, one-sided}} = \frac{1}{2} \left(\sum_{i \in \mathcal{S}_{\text{risky}}} |\Delta w_i| + |\Delta w_{\text{cash}}| \right),$$

   but it is not stock traded turnover. A change in cash weight is a settlement difference; it does not create a stock fill, spread, impact, or commission. Do not use it in $V_{\text{trade}} = 2\tau_{\text{turnover}} A$.

For a monthly portfolio, report annualized turnover as 12 times average monthly turnover. Intraday execution slices do not turn monthly signals or returns into daily-frequency statistics.

6. Derivations

6.1 Cash-Ledger Recursion and Proof Against Double Charging

Following Chapter 29's event clock and the A7-B interface contract, we derive the ledger recursion from discrete accounting conservation.

Immediately before rebalancing at t_{pre}, let the account state be $(C_{\text{pre}}, H_{\text{pre}})$. During the execution window, the signed filled quantities are $F \in \mathbb{R}^N$; a fill VWAP P_i^{fill} is defined only for $i \in \mathcal{I}_F$. Let explicit trading fees be Ξ. At completion time t_{post}, cash is

$$C_{\text{post}} = C_{\text{pre}} - \sum_{i \in \mathcal{I}_F} F_i P_i^{\text{fill}} - \Xi.$$

The new holdings vector is

$$H_{\text{post}} = H_{\text{pre}} + F.$$

Proposition 6.1 (Embedded Costs Must Not Be Charged Again). Suppose that for every $i \in \mathcal{I}_F$, the fill price satisfies the microstructure relation

$$P_i^{\text{fill}} = P_i^{\text{mid}} + \operatorname{sgn}(F_i) \left(\frac{1}{2} S_i + g_i(|F_i|) \right).$$

Then the half-spread and temporary impact have already been fully settled through the cash account. They must not be deducted a second time as cash fees.

Proof. Substitute P_i^{fill} into the cash recursion:

$$C_{\text{post}} = C_{\text{pre}} - \sum_{i \in \mathcal{I}_F} F_i \left[P_i^{\text{mid}} + \operatorname{sgn}(F_i) \left(\frac{1}{2} S_i + g_i(|F_i|) \right) \right] - \Xi.$$

Since $F_i \operatorname{sgn}(F_i) = |F_i|$, expansion gives

$$C_{\text{post}} = C_{\text{pre}} - \sum_{i \in \mathcal{I}_F} F_i P_i^{\text{mid}} - \sum_{i \in \mathcal{I}_F} |F_i| \frac{S_i}{2} - \sum_{i \in \mathcal{I}_F} |F_i| g_i(|F_i|) - \Xi.$$

The terms on the right are, respectively:

- $\sum_{i \in \mathcal{I}_F} F_i P_i^{\text{mid}}$: transaction cash measured at the frictionless midpoint;
- $\sum_{i \in \mathcal{I}_F} |F_i| S_i / 2$: the cash consumed by the half-spread;
- $\sum_{i \in \mathcal{I}_F} |F_i| g_i(|F_i|)$: the cash consumed by temporary impact; and
- Ξ: explicit trading commissions and fees.

Thus C_{post} has already deducted the notional transaction cash, half-spread, temporary impact, and commission exactly once. If the ledger separately posts another "spread deduction" or "impact deduction," cash becomes

$$C_{\text{err}} = C_{\text{post}} - \sum_{i \in \mathcal{I}_F} |F_i| \frac{S_i}{2} - \sum_{i \in \mathcal{I}_F} |F_i| g_i(|F_i|),$$

which double counts the spread and temporary impact and creates a fictitious cash loss. This proves the claim. □

6.2 Equivalence Between NAV Changes and Implementation Shortfall

The Perold (1988) entry is included here as a bibliographic record only. The 2026-10-09 source review checked publisher and author metadata; it did not read the paper itself. This batch carried forward that historical status without a new search or full-text review. The SEC (2003) document notes that, at the time, there was no universally accepted method for calculating portfolio implementation shortfall and discusses the concept's historical origins. This section adopts an explicit teaching convention: **the difference between paper-portfolio PnL and actual-portfolio PnL**. The decomposition into decision price, arrival price, cutoff price, and unfilled opportunity cost is the convention stated in this chapter; it is not a claim that all current markets or regulatory regimes use the same formula.

Suppose the investment decision is made at t_{dec}, with decision benchmark vector P^{dec} and signed parent-order target Q. The order reaches the market at t_{arr}, when the midpoint vector is P^{arr}. The actual fill vector is F; fill VWAP P_i^{fill} is defined only for $i \in \mathcal{I}_F = \{i : F_i \neq 0\}$; the unfilled remainder is $R = Q - F$. At the end of the window or evaluation time t_{term}, the market mark vector is P^{term}. Every subscript i below refers to the same asset.

Proposition 6.2 (Four-Factor Implementation-Shortfall Decomposition Equals the Direct-Dollar Method). Define implementation shortfall (positive values denote a cost loss) by

$$\mathrm{IS} = \mathrm{PnL}_{\mathrm{paper}} - \mathrm{PnL}_{\mathrm{actual}}.$$

It decomposes exactly into the following four terms:

$$\begin{aligned}\mathrm{IS} = &\underbrace{\sum_i Q_i (P_i^{\mathrm{arr}} - P_i^{\mathrm{dec}})}_{\text{Delay}} \\ &+ \underbrace{\sum_{i \in \mathcal{I}_F} F_i (P_i^{\mathrm{fill}} - P_i^{\mathrm{arr}})}_{\text{Execution}} \\ &+ \underbrace{\sum_i R_i (P_i^{\mathrm{term}} - P_i^{\mathrm{arr}})}_{\text{Opportunity}} + \underbrace{\Xi}_{\text{Explicit fees}}.\end{aligned}$$

This decomposition is exactly equal to the direct dollar-difference method:

$$\mathrm{IS} = \sum_{i \in \mathcal{I}_F} F_i (P_i^{\mathrm{fill}} - P_i^{\mathrm{dec}}) + \sum_i R_i (P_i^{\mathrm{term}} - P_i^{\mathrm{dec}}) + \Xi.$$

Proof. For each asset i, use the same signed convention for long and short positions: $Q_i > 0$ is a buy and $Q_i < 0$ is a sell.

- **Paper-portfolio PnL:** The paper portfolio assumes that the entire parent order Q is filled at P^{dec} at decision time and incurs no explicit fee. At t_{term}, it marks the paper holdings at P^{term}, so

$$\mathrm{PnL}_{\mathrm{paper}} = \sum_i Q_i (P_i^{\mathrm{term}} - P_i^{\mathrm{dec}}).$$

- **Actual-portfolio PnL:** The actual portfolio holds only the filled quantity F and is marked at P^{term} at cutoff. The actual fill payment and explicit fees are $\sum_{i\in\mathcal{I}_F} F_i P_i^{\text{fill}} + \Xi$. Therefore,

$$\begin{aligned}\text{PnL}_{\text{actual}} &= \sum_i F_i P_i^{\text{term}} - \left(\sum_{i\in\mathcal{I}_F} F_i P_i^{\text{fill}} + \Xi\right) \\ &= \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{fill}}) - \Xi.\end{aligned}$$

Subtract the two:

$$\text{IS} = \sum_i Q_i (P_i^{\text{term}} - P_i^{\text{dec}}) - \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{fill}}) + \Xi.$$

Using the remainder identity $Q = F + R$, split the paper-PnL term, asset by asset, into filled and unfilled quantities:

$$\begin{aligned}\text{IS} = &\sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{dec}}) + \sum_i R_i (P_i^{\text{term}} - P_i^{\text{dec}}) - \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{fill}}) \\ &+ \Xi.\end{aligned}$$

Combining the terms involving F gives

$$\begin{aligned}&\sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{dec}}) - \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{term}} - P_i^{\text{fill}}) \\ &= \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{fill}} - P_i^{\text{dec}}).\end{aligned}$$

This yields the direct dollar-difference method:

$$\text{IS} = \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{fill}} - P_i^{\text{dec}}) + \sum_i R_i (P_i^{\text{term}} - P_i^{\text{dec}}) + \Xi.$$

Next, introduce the arrival midpoint P^{arr} as an intermediate benchmark in each term:

$$P_i^{\text{fill}} - P_i^{\text{dec}} = (P_i^{\text{arr}} - P_i^{\text{dec}}) + (P_i^{\text{fill}} - P_i^{\text{arr}}), \quad i \in \mathcal{I}_F,$$

$$P_i^{\text{term}} - P_i^{\text{dec}} = (P_i^{\text{arr}} - P_i^{\text{dec}}) + (P_i^{\text{term}} - P_i^{\text{arr}}), \quad i = 1, \ldots, N.$$

Substituting gives

$$\begin{aligned}\text{IS} = &\sum_{i\in\mathcal{I}_F} F_i \left[(P_i^{\text{arr}} - P_i^{\text{dec}}) + (P_i^{\text{fill}} - P_i^{\text{arr}})\right] \\ &+ \sum_i R_i \left[(P_i^{\text{arr}} - P_i^{\text{dec}}) + (P_i^{\text{term}} - P_i^{\text{arr}})\right] \\ &+ \Xi \\ = &\sum_i (F_i + R_i)(P_i^{\text{arr}} - P_i^{\text{dec}}) + \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{fill}} - P_i^{\text{arr}}) + \sum_i R_i (P_i^{\text{term}} - P_i^{\text{arr}}) + \Xi.\end{aligned}$$

Using $F + R = Q$ again,

$$\text{IS} = \sum_i Q_i (P_i^{\text{arr}} - P_i^{\text{dec}}) + \sum_{i\in\mathcal{I}_F} F_i (P_i^{\text{fill}} - P_i^{\text{arr}}) + \sum_i R_i (P_i^{\text{term}} - P_i^{\text{arr}}) + \Xi.$$

The four terms represent, in order:

1. $\sum_i Q_i (P_i^{\text{arr}} - P_i^{\text{dec}})$: **delay cost** from price movement between the decision and the order's arrival at the market.

2. $\sum_{i\in\mathcal{I}_F} F_i(P_i^{\text{fill}} - P_i^{\text{arr}})$: **execution-friction cost** from spread and market impact on actual fills; this empty sum is zero when there are no fills.
3. $\sum_i R_i(P_i^{\text{term}} - P_i^{\text{arr}})$: **opportunity cost** from market movement on the unfilled remainder.
4. Ξ: **explicit cash fees** paid to exchanges and brokers.

The algebraic identity follows. □

This chapter assigns the decision-to-arrival price movement on the full parent order Q to the delay term, then measures filled and residual terms from the arrival price. Other studies or systems may allocate that movement across filled quantities or use a different opportunity-cost benchmark. Component attribution can differ if the decision price, cutoff mark, and quantity conservation are applied consistently; the direct dollar total is fixed by the algebraic identity.

6.3 Deriving the Self-Financing Share Adjustment

In Chapter 29, an optimizer may propose a target weight w^*. If an analyst naively treats $w^* \cdot \text{NAV}_{\text{pre}}$ as the amount that can be invested, then discovers after submitting the order that commission and spread have caused a cash deficit, the system will fail. This section derives the self-financing closed-form solution when trading frictions are included.

Consider a single-asset buy. The account has available cash $C_0 > 0$, no debt, and no holdings, and wants to buy a stock. The arrival midpoint is P^{mid}. The absolute half-spread is $S/2 = P^{\text{mid}}s/2$ in USD per share. Temporary impact is linear, $g(F) = \lambda_{\text{impact}}F$, where $\lambda_{\text{impact}} > 0$ has units USD/share^2 and differs from Chapter 36's USD^{-1} λ_{risk}. Let the per-share commission be c_{share} and the proportional commission rate be c_{notional}.

For a purchase of $F > 0$ shares, the actual fill price per share is

$$P^{\text{fill}}(F) = P^{\text{mid}} + \frac{S}{2} + \lambda_{\text{impact}}F.$$

The total cash required is

$$\text{CashRequired}(F) = F \cdot P^{\text{fill}}(F) + c_{\text{share}}F + c_{\text{notional}}FP^{\text{fill}}(F).$$

The strict self-financing constraint requires total outlay not to exceed available cash:

$$(1 + c_{\text{notional}})F\left(P^{\text{mid}} + \frac{S}{2} + \lambda_{\text{impact}}F\right) + c_{\text{share}}F = C_0.$$

Rearranging gives a quadratic in the share quantity F:

$$\underbrace{(1 + c_{\text{notional}})\lambda_{\text{impact}}}_{A}F^2 + \underbrace{\left[(1 + c_{\text{notional}})\left(P^{\text{mid}} + \frac{S}{2}\right) + c_{\text{share}}\right]}_{B}F - \underbrace{C_0}_{C}$$

$$= 0.$$

Since $A > 0$, $B > 0$, and $C > 0$, the discriminant $\Delta = B^2 + 4AC > 0$, and the equation has a unique positive real root:

$$F^* = \frac{2C}{B + \sqrt{B^2 + 4AC}}.$$

If nonlinear impact is ignored ($\lambda_{\text{impact}} = 0$), the equation reduces to a linear one:

$$F^*_{\text{linear}} = \frac{C_0}{(1 + c_{\text{notional}})\left(P^{\text{mid}} + \frac{S}{2}\right) + c_{\text{share}}}.$$

Thus, frictions strictly reduce the physical number of shares that can be purchased. If commission and half-spread are omitted from the denominator and the order is instead set to $F = C_0/P^{\text{mid}}$, the account will be overdrawn by $FP^{\text{fill}} + \Xi - C_0 > 0$.

6.4 Deriving Strategy Capacity and the Decline in Marginal Net Returns

This section uses a dimensionally closed square-root scenario model to illustrate sensitivity to scale. Let A be NAV in USD, α_{gross} the gross excess-return rate over the same evaluation period, τ_{turnover} the one-sided turnover of the tradable risky-security set over that period (cash changes are not stock trades), D the number of trading days in that period, and ADV_{market} the average daily dollar volume of the relevant stocks in USD/day. Actual total traded risky-asset notional is

$$V_{\text{trade}} = 2\tau_{\text{turnover}} A,$$

while the matching market-volume proxy over the evaluation period is $D\,ADV_{\text{market}}$. Define the participation-rate proxy as

$$p = \frac{V_{\text{trade}}}{D\,ADV_{\text{market}}}.$$

This is an input for scenario comparisons, not a liquidity guarantee that can be used directly to place orders.

Assume a fixed cost rate c_0 per dollar actually traded and a square-root impact rate $\gamma\sqrt{p}$. Both c_0 and γ are dimensionless decimals and must refer to the same evaluation period:

$$C_{\text{period}} = V_{\text{trade}} \left(c_0 + \gamma \sqrt{\frac{V_{\text{trade}}}{D\,ADV_{\text{market}}}} \right).$$

Therefore,

$$\alpha_{\text{net}}(A) = \alpha_{\text{gross}} - 2\tau_{\text{turnover}} c_0 - \gamma (2\tau_{\text{turnover}})^{3/2} (D\,ADV_{\text{market}})^{-1/2} A^{1/2}.$$

Let $a = \alpha_{\text{gross}} - 2\tau_{\text{turnover}} c_0$ and $k = \gamma(2\tau_{\text{turnover}})^{3/2}/\sqrt{D\,ADV_{\text{market}}}$. If $a > 0$ and $k > 0$, the scale at which net return reaches zero is $A_{\text{break}} = (a/k)^2$. Dollar net profit is $aA - kA^{3/2}$, with an interior maximum at $A_{\text{opt}} = 4A_{\text{break}}/9$. If $a \le 0$, fixed costs have already consumed gross returns, so this model has no positive profitable capacity.

The ratio 4/9 follows only from this particular net-return function. It is not an industry-wide optimal-size constant; empirical capacity requires volume, order-flow, and cost data calibrated by security and time period.

The time frequency must be consistent. The standard monthly example uses 21 trading days, monthly gross return, and one-sided monthly turnover. If annualized turnover is reported, derive it by multiplying monthly turnover by 12; do not mix annualized turnover with monthly gross return or $D = 21$ in the same capacity formula.

7. Financial Interpretation

Returning from mathematical formulas to the actual mechanics of financial trading, quantitative researchers should understand three economic principles.

7.1 Spreads and Liquidity Provision

A spread can compensate liquidity providers for inventory risk, adverse selection, and operating costs. The relative importance of these factors varies across market structures, order types, and market conditions; a spread is not a fixed "insurance premium" determined by a single cause. Temporary impact describes the deviation of a fill from its benchmark when a trader demands immediacy.

7.2 The Relationship Between Frequency, Turnover, and Cost

Higher turnover causes a fixed fee rate to recur across more actual traded notional. Whether costs overwhelm gross returns depends on trade size, asset liquidity, and execution method. A convenient constructed scenario for hand calculation assumes a portfolio fully invested in tradable stocks, one-sided annualized turnover $\tau_{\text{turnover}} = 15$, and combined fees of 15 bps per dollar of actual traded notional. On a total traded stock-notional basis, turnover is approximately $2\tau_{\text{turnover}}A$, so costs are about $2 \times 15 \times 0.0015 = 4.5\%$ of NAV. If idle cash weights change, that change is excluded from stock traded notional. All figures are arithmetic assumptions.

7.3 Asymmetry in the Availability and Timing of Stock-Borrow Fees

Borrow fees depend on the security, the available loan supply, and the billing time; they are not constants determined solely by a short signal. General availability and scarce loan supply can correspond to different rates. Whether a borrow is available, when it may be recalled, and how much liquidity is needed to cover the short position require timestamped borrow and execution records for backtesting. Without such sources, the examples in this book demonstrate only the cash accounting identity "rate × actual borrowed market value × accrual fraction"; they do not infer real loan availability or recall losses.

Chapter 35 complete numerical example: all steps and conclusions

8. Numerical Example

This section presents a complete constructed teaching example that runs through all the formulas and can be checked by hand. All values are explicitly labeled as constructed for teaching.

8.1 Initial Account and Market State

Consider a portfolio with assets under management of USD 1,000,000.00.

Continued: the same complete example

- **Current time:** 2026-10-12 09:15:00 EDT (America/New_York, UTC-04:00), the decision and signal time t_{dec}.
- **Execution time:** 2026-10-12 09:30:00 EDT (America/New_York, UTC-04:00), when the order reaches the market, t_{arr}.
- **Initial holdings and cash at t_{pre}:**
  - Stock A (large-cap, liquid): 5,000 shares held; arrival midpoint P_A^{mid} = \$100.00/share; market value USD 500,000.00 (50.0%).
  - Stock B (mid-cap growth): 0 shares held; arrival midpoint P_B^{mid} = \$50.00/share; market value USD 0.00 (0.0%).
  - Stock C (security used for a short hedge): 0 shares held; arrival midpoint P_C^{mid} = \$40.00/share; market value USD 0.00 (0.0%).
  - Cash balance C_0: USD 500,000.00 (50.0%).
  - Initial NAV: $\text{NAV}_0 = 500{,}000.00 + 0.00 + 0.00 + 500{,}000.00 = \$1{,}000{,}000.00$.

8.2 Market Quotes and Friction Parameters (Constructed for Teaching)

Quote inputs (all prices are USD per share; quoted spread is ask minus bid):

Asset	Arrival midpoint P^{mid} (USD/share)	Best bid (USD/share)	Best ask (USD/share)	Quoted spread (USD/share; bps)
Stock A	USD 100.00	USD 99.98	USD 100.02	USD 0.04 (4 bps)
Stock B	USD 50.00	USD 49.95	USD 50.05	USD 0.10 (20 bps)
Stock C	USD 40.00	USD 39.96	USD 40.04	USD 0.08 (20 bps)

In this example, the per-share price offset from impact equals a slope times the filled quantity. The linear-impact slope is measured in USD/share^2, commission in USD/share, and borrow fees are annualized. Slopes are calculated from the exact fractions shown below. Fill prices and mark prices are rounded to cents per share before being posted; aggregate cash, fees, and NAV use unrounded internal amounts and are displayed to cents. The resulting prices and dollar totals happen to fall exactly on cent values. The permanent-impact slope γ_i here is not the dimensionless volatility-scaled coefficient from §5.4.

Asset	Temporary-impact slope λ_{impact} (USD/share^2)	Permanent-impact slope γ (USD/share^2)	Commission c (USD/share)	Annual borrow rate b
Stock A	1.0×10^{-5}	$0.01/3{,}000 = 3.333\ldots \times 10^{-6}$	USD 0.005	0.5%
Stock B	$0.10/12{,}000 = 8.333\ldots \times 10^{-6}$	$0.04/12{,}000 = 3.333\ldots \times 10^{-6}$	USD 0.005	1.0%
Stock C	1.5×10^{-5}	5.0×10^{-6}	USD 0.005	3.65% (hard to borrow, HTB)

8.3 Intended Rebalance Targets and Order Generation

At 09:15:00, the optimizer assigns new target weights based on factor scores. Target weights are locked at 09:15:00. To avoid using a future price as a decision input, this constructed example explicitly stipulates that each asset's decision midpoint equals its arrival midpoint, $P^{\text{dec}} = P^{\text{arr}}$, with both set to the prices in the table. At 09:30:00, the already-locked target

Continued: the same complete example

notionals are converted to share quantities using the arrival midpoints then available. This equal-price assumption is part of the example; it is not an observation or forecast of a future price.

- Stock A: reduce to a target weight of 20.0% (target market value USD 200,000.00, or a target of 2,000 shares).
  - Order A: sell 3,000 shares ($Q_A = -3{,}000$ shares).
- Stock B: establish a 60.0% target weight (target market value USD 600,000.00, or 12,000 shares).
  - Order B: buy 12,000 shares ($Q_B = +12{,}000$ shares).
- Stock C: establish a short position with a target weight of −8.0% (target market value −USD 80,000.00, or a short sale of 2,000 shares).
  - Order C: sell short 2,000 shares ($Q_C = -2{,}000$ shares).
- Cash: target residual weight of 28.0%.

8.4 Step-by-Step Fill Prices and Cost Details

All orders fill completely during the execution window from 09:30:00 to 10:00:00 ($F = Q$, $R = 0$).

1. Stock A Execution (Sell 3,000 Shares, $F_A = -3{,}000$)

- Half-spread deduction: $S_A/2 = 0.04/2 =$ USD 0.02/share.
- Temporary-impact deduction: $g_A(|F_A|) = \lambda_{\text{impact},A}|F_A| = 1.0 \times 10^{-5} \times 3{,}000 =$ USD 0.03/share.
- Actual average sale price:

$$P_A^{\text{fill}} = P_A^{\text{mid}} - \frac{S_A}{2} - g_A(|F_A|) = 100.00 - 0.02 - 0.03 = \text{USD } 99.95/\text{share}.$$

- Actual traded notional before net cash proceeds: $|F_A|P_A^{\text{fill}} = 3{,}000 \times 99.95 =$ USD 299,850.00.
- Stock A commission: $c_{\text{share}}|F_A| = 0.005 \times 3{,}000 =$ USD 15.00.
- Permanent impact changes the market midpoint: $h_A(|F_A|) = (0.01/3{,}000) \times 3{,}000 =$ USD 0.01/share. The new benchmark midpoint after execution is $P_{A,\text{post}}^{\text{mid}} = 100.00 - 0.01 =$ USD 99.99/share.

2. Stock B Execution (Buy 12,000 Shares, $F_B = +12{,}000$)

- Half-spread addition: $S_B/2 = 0.10/2 =$ USD 0.05/share.
- Temporary-impact addition: $g_B(|F_B|) = \lambda_{\text{impact},B}|F_B| = (0.10/12{,}000) \times 12{,}000 =$ USD 0.10/share.
- Actual average purchase price:

$$P_B^{\text{fill}} = P_B^{\text{mid}} + \frac{S_B}{2} + g_B(|F_B|) = 50.00 + 0.05 + 0.10 = \text{USD } 50.15/\text{share}.$$

- Actual purchase notional: $F_B P_B^{\text{fill}} = 12{,}000 \times 50.15 =$ USD 601,800.00.
- Stock B commission: $c_{\text{share}}|F_B| = 0.005 \times 12{,}000 =$ USD 60.00.

Continued: the same complete example

- Permanent impact changes the market midpoint: $h_B(|F_B|) = (0.04/12{,}000) \times 12{,}000 =$ USD 0.04/share. The new benchmark midpoint after execution is $P^{\text{mid}}_{B,\text{post}} = 50.00 + 0.04 =$ USD 50.04/share.

3. Stock C Execution (Short-Sell 2,000 Shares, $F_C = -2{,}000$)

- Half-spread deduction: $S_C/2 = 0.08/2 =$ USD 0.04/share.
- Temporary-impact deduction: $g_C(|F_C|) = \lambda_{\text{impact},C}|F_C| = 1.5 \times 10^{-5} \times 2{,}000 =$ USD 0.03/share.
- Actual average short-sale price:

$$P^{\text{fill}}_C = P^{\text{mid}}_C - \frac{S_C}{2} - g_C(|F_C|) = 40.00 - 0.04 - 0.03 = \text{USD } 39.93/\text{share}.$$

- Notional proceeds from the short sale: $|F_C|P^{\text{fill}}_C = 2{,}000 \times 39.93 =$ USD 79,860.00.
- Stock C commission: $c_{\text{share}}|F_C| = 0.005 \times 2{,}000 =$ USD 10.00.
- Permanent impact changes the market midpoint: $h_C(|F_C|) = 5.0 \times 10^{-6} \times 2{,}000 =$ USD 0.01/share. The new benchmark midpoint after execution is $P^{\text{mid}}_{C,\text{post}} = 40.00 - 0.01 =$ USD 39.99/share.

8.5 Cash Account and Holdings Reconciliation

The cash ledger follows the formula derived in Section 6. This constructed example treats short-sale proceeds as immediately usable cash and imposes no restricted margin, collateral haircut, or broker restriction on the use of short-sale proceeds. This is an assumed account-contract boundary, not a statement about the cash-availability rules of every account:

$$C_{\text{post}} = C_0 - \left(F_A P^{\text{fill}}_A + F_B P^{\text{fill}}_B + F_C P^{\text{fill}}_C\right) - (\Xi_A + \Xi_B + \Xi_C).$$

Substituting the values:

- $F_A P^{\text{fill}}_A = (-3{,}000) \times 99.95 = -\text{USD } 299{,}850.00$ (a sale, so cash increases by USD 299,850.00).
- $F_B P^{\text{fill}}_B = (+12{,}000) \times 50.15 = +\text{USD } 601{,}800.00$ (a purchase, so cash decreases by USD 601,800.00).
- $F_C P^{\text{fill}}_C = (-2{,}000) \times 39.93 = -\text{USD } 79{,}860.00$ (a short sale, so cash increases by USD 79,860.00).
- Total commission: $\Xi = 15.00 + 60.00 + 10.00 =$ USD 85.00.

Net cash change:

$$\begin{aligned}\Delta C &= -(-299{,}850.00) - (+601{,}800.00) - (-79{,}860.00) - 85.00 \\ &= +299{,}850.00 - 601{,}800.00 + 79{,}860.00 - 85.00 \\ &= -\text{USD } 222{,}175.00.\end{aligned}$$

Ending cash balance:

$$C_{\text{post}} = 500{,}000.00 - 222{,}175.00 = \text{USD } 277{,}825.00.$$

Update share holdings and market values using the new midpoints $P^{\text{mid}}_{\text{post}}$:

- Stock A: 5,000 − 3,000 = 2,000 shares held; market value $2{,}000 \times 99.99 =$ USD 199,980.00.

Continued: the same complete example

- Stock B: $0 + 12{,}000 = 12{,}000$ shares held; market value $12{,}000 \times 50.04 =$ USD 600,480.00.
- Stock C: $0 - 2{,}000 = -2{,}000$ shares held (short); short-liability value $-2{,}000 \times 39.99 =$ $-$USD 79,980.00.
- Cash: USD 277,825.00.

Ending NAV ($\mathrm{NAV}_{\mathrm{post}}$):

$$\mathrm{NAV}_{\mathrm{post}} = 199{,}980.00 + 600{,}480.00 - 79{,}980.00 + 277{,}825.00 = \text{USD } 998{,}305.00.$$

Relative to initial NAV of USD 1,000,000.00, the rebalance reduces NAV by

$$\Delta\mathrm{NAV} = 998{,}305.00 - 1{,}000{,}000.00 = -\text{USD } 1{,}695.00 \quad (-16.95 \text{ bps}).$$

8.6 Cost-Attribution Ablation and Exact Implementation-Shortfall Reconciliation

We attribute the USD 1,695.00 NAV loss term by term:

Item	Stock A (USD)	Stock B (USD)	Stock C (USD)	Total (USD)
Explicit commissions on actual fills	-15.00	-60.00	-10.00	-85.00
Half-spread and temporary impact (already in fill price)	-150.00	-1,800.00	-140.00	-2,090.00
Change in ending holdings mark after permanent impact, $H_{\mathrm{post}}\Delta P^{\mathrm{mid}}$	-20.00	+480.00	+20.00	+480.00
Ending NAV change	**-185.00**	**-1,380.00**	**-130.00**	**-1,695.00**

Reconciliation. Execution shortfall relative to the arrival midpoint, excluding revaluation of existing holdings, is USD 85 in commissions plus USD 2,090 in fill slippage, for implementation shortfall of USD 2,175. The ending holdings are marked at the post-permanent-impact midpoints, producing a total mark change of +USD 480. Therefore,

$$\Delta\mathrm{NAV} = -2{,}175 + 480 = -1{,}695 \text{ USD}.$$

The fill price already includes half-spread and temporary impact. Cash is posted using the actual fill price and commission; the table reports spread and impact as attribution components and does not deduct them from cash again.

Now calculate implementation shortfall independently using the arrival price as benchmark ($P^{\mathrm{dec}} = P^{\mathrm{arr}}$, with no unfilled remainder, $R = 0$):

$$\mathrm{IS} = \sum_{i \in \{A,B,C\}} F_i (P_i^{\mathrm{fill}} - P_i^{\mathrm{arr}}) + \Xi.$$

- Stock A: $(-3{,}000) \times (99.95 - 100.00) = (-3{,}000) \times (-0.05) = +$USD 150.00.
- Stock B: $(+12{,}000) \times (50.15 - 50.00) = (+12{,}000) \times (+0.15) = +$USD 1,800.00.
- Stock C: $(-2{,}000) \times (39.93 - 40.00) = (-2{,}000) \times (-0.07) = +$USD 140.00.

Continued: the same complete example

- Explicit commissions: USD 85.00.

Total implementation shortfall:

$$\text{IS} = 150.00 + 1{,}800.00 + 140.00 + 85.00 = \text{USD } 2{,}175.00 \quad (21.75 \text{ bps}).$$

Think it through: Why is implementation shortfall (USD 2,175.00) not equal to the NAV loss (USD 1,695.00)? Implementation shortfall measures realized execution friction relative to the arrival price. Ending NAV also marks holdings at the new midpoint. In this example, the constructed permanent-impact path changes the ending holdings mark by $-20 + 480 + 20 = +\text{USD } 480$, so NAV falls by $2{,}175 - 480 = \text{USD } 1{,}695$. This mark gain is a valuation result along the constructed price path, not a second cash receipt, and it does not offset the commissions paid on fills. The post-execution midpoint is stipulated as the permanent-impact mark in this constructed example; this finite-horizon path is not empirical evidence that real-world impact is permanent.

8.7 Multi-Period Stock-Borrow Fees and Holdings Returns

Assume the portfolio holds these positions for 30 calendar days ($\Delta d = 30$ days, using the Actual/360 day-count convention). There is no rebalance during the period, and market prices remain at their post-rebalance levels.

- Under the same constructed assumption, short-sale proceeds remain usable cash over these 30 days and earn the positive cash-credit rate. If an actual broker segregates or restricts these proceeds, the cash-yield and financing treatment must be recalculated under that contract.
- The USD 277,825.00 cash balance earns a 2.0% annual cash-credit rate:

$$\text{Income}_{\text{cash}} = 277{,}825.00 \times 0.02 \times \frac{30}{360} = \text{USD } 463.04.$$

- Stock C has a short market value of USD 79,980.00 and an annual borrow-fee rate of 3.65% (hard to borrow, HTB):

$$\text{Cost}_{\text{borrow}} = 79{,}980.00 \times 0.0365 \times \frac{30}{360} = \text{USD } 243.27.$$

  At month-end, after the borrow fee is charged and cash interest is credited, net cash is

$$C_{30} = 277{,}825.00 + 463.04 - 243.27 = \text{USD } 278{,}044.77.$$

  The net financing return on funds tied up by these positions over one month is $+463.04 - 243.27 = +\text{USD } 219.77$.

8.8 Constructed Cost-Sensitivity Experiment

Suppose the monthly gross excess-return rate is 120 bps, one-sided monthly turnover of the tradable stock universe is τ_{turnover}, the month has 21 trading days, and the stock universe has average daily dollar volume of USD 100,000,000. Fixed cost per dollar traded is 8.25 bps and the square-root impact coefficient is $\gamma = 0.05$. Every number is constructed for teaching and

Continued: the same complete example

excludes financing and borrow fees. If annualized turnover is reported externally, multiply the monthly figure by 12; inside the formula, all quantities remain on a monthly basis. Each row uses $V_{\text{trade}} = 2\tau_{\text{turnover}}A$, $p = V_{\text{trade}}/(21 \times ADV)$, and unit traded-notional cost $c_0 + \gamma\sqrt{p}$. Changes in cash weights are excluded from V_{trade}.

AUM (USD)	One-sided monthly turnover τ_{turnover}	Total traded notional (USD)	Impact rate (bps/traded notional)	Total cost rate (bps/traded notional)
1,000,000	0.20	400,000	6.90	15.15
1,000,000	1.00	2,000,000	15.43	23.68
1,000,000	3.00	6,000,000	26.73	34.98
10,000,000	1.00	20,000,000	48.80	57.05
20,000,000	1.00	40,000,000	69.01	77.26

AUM (USD)	Cost (USD)	Net return (bps/NAV)	Net return (USD)
1,000,000	606.03	113.94	11,393.97
1,000,000	4,736.07	72.64	7,263.93
1,000,000	20,985.67	-89.86	-8,985.67
10,000,000	114,090.01	5.91	5,909.99
20,000,000	309,026.22	-34.51	-69,026.22

The first two columns show how turnover and scale have different effects on unit traded-notional impact. For the one-sided monthly-turnover scenario of 1, the model gives a break-even AUM of about USD 11.25 million and a maximum net dollar return at about USD 5.00 million. Both are teaching values implied by these assumptions, not historical backtest results or investment advice.

9. Python Implementation

This chapter and Chapter 36 share `src/quant_course/execution_costs.py`; the experiment report and notebook call the same cost and execution core. The static I3 notebook inspection records that its default path has four cells, uses a small constructed ledger/IS example, and produces one figure. The Notebook creates its own unique `RUN_DIR` for fresh numerical and graphical outputs. The outer runner separately creates a unique execution directory for the executed Notebook copy and `execution.json`. This is not a full Project 4 CLI run. The English Notebook language layer remains assigned to BIL23. Common objects include:

Interface	Purpose
`ExecutionConfig`	Commission, half-spread, temporary/permanent impact, participation cap, positive-cash credit rate, negative-cash borrowing rate, borrow fee, and day-count convention; inputs are checked at construction. If no separate borrowing rate is supplied, the cash rate is used for compatibility.
`ParentOrder`	Signed parent order, decision/arrival times, and prices. Buy quantities are positive; sell quantities are negative.

Interface	Purpose
`ExecutionSlot`	Send, fill, and slice times, plus forecast, observed, and realized external volume; `observation_time` establishes that a POV observation was available before the send time.
`execute_order`	Generates child orders and simulates fills using `TWAP`, an ex ante forecast curve `VWAP`, or lagged observed volume `POV`.
`ExecutionPortfolioLedger`	Posts each fill to cash and holdings; spread and temporary impact enter only through the fill price, while permanent impact enters subsequent marks.

The example below shows the actual API for a parent order, one execution slot, IS decomposition, and cash accounting. The test file checks the complete boundaries, buy/sell direction, partial and zero fills, and leakage counterexamples through regression tests.

```
import datetime as dt
from datetime import timezone
import math

from quant_course.execution_costs import (
    ExecutionConfig, ExecutionPortfolioLedger, ExecutionSlot,
    ParentOrder, execute_order,
)

utc = timezone.utc
start = dt.datetime(2026, 10, 12, 13, 30, tzinfo=utc)
parent = ParentOrder(
    order_id="order-1", asset="XYZ", signed_quantity=1_000,
    decision_time=start - dt.timedelta(minutes=5), parent_received_at=start,
    ↪ arrival_time=start,
    decision_price=99.95, arrival_price=100.00,
)
slot = ExecutionSlot(
    slot_index=0, slot_start=start, slot_end=start + dt.timedelta(minutes=30),
    send_time=start, child_venue_arrival_at=start,
    fill_time=start + dt.timedelta(minutes=15), forecast_frozen_at=start - dt.
    ↪ timedelta(minutes=1),
    forecast_external_volume=5_000, observed_external_volume=0,
    realized_external_volume=5_000, mid_price=100.00,
)
config = ExecutionConfig(
    commission_rate=0.0005,       # 5 bps of actual fill notional
    half_spread_bps=2.5,
    temp_impact_coef=0.001,
    max_participation=0.20,
)
result = execute_order(
    parent, "TWAP", [slot], config,
    terminal_mark_price=100.10,
    terminal_mark_time=slot.slot_end,
)
```

```
assert result.signed_parent_quantity == result.signed_filled_quantity + result.
    ↪ signed_remainder_quantity
assert math.isclose(
    result.is_decomposition.total_is_decomposition,
    result.is_decomposition.total_is_direct,
    rel_tol=0.0, abs_tol=1e-10,
)

ledger = ExecutionPortfolioLedger(100_000.0, config, parent.decision_time)
for fill in sorted(result.fills, key=lambda item: item.fill_at):
    # Mark at the price observable with this fill; do not backfill the later terminal
    ↪  mark.
    ledger.post_fill(parent.asset, fill, {parent.asset: fill.mid_price})
ledger.accrue_interest_and_borrow(
    result.terminal_mark_time, {parent.asset: result.terminal_mark_price}
)
assert ledger.reconcile_ledger()["is_exact"] == 1.0
```

`fill_vwap` is `None` when there are no fills. Commission on executed trades is calculated from actual traded notional. The cash update deducts only the fill payment and commission, without deducting spread or temporary impact a second time after those costs have entered the fill price. Permanent impact affects only the subsequent price path and NAV marks. The complete interface, causal constraints, and exercise implementations are documented in the tests and Project 4 experiment files.

---

10. Visualization

This section reuses two constructed teaching figures inherited from the input candidate; this language batch did not regenerate them. Their curve formulas and inputs correspond to Section 8.8; the ablation figure comes from Project 4's fixed-seed, 24-month, layer-by-layer cost scenario. They illustrate relationships internal to the model, not market capacity or estimates of real performance.

10.1 Net Excess Return and Strategy-Capacity Curve

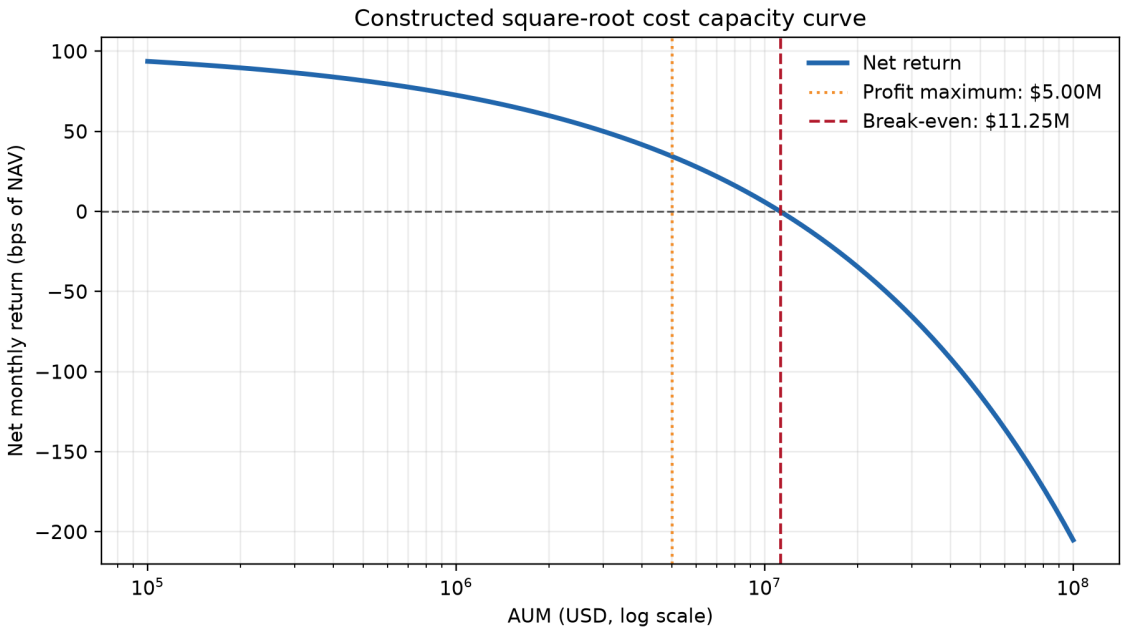


Figure 1: Net return and scale under constructed monthly cost assumptions

The figure fixes monthly gross excess return at 120 bps, one-sided monthly turnover at 1, 21 trading days, market ADV at USD 100 million per day, fixed unit traded-notional cost at 8.25 bps, and square-root impact coefficient at 0.05. The horizontal axis is AUM in USD on a logarithmic scale; the vertical axis is monthly net return. Under these assumptions, break-even AUM is about USD 11.248 million, and maximum net dollar return occurs at about USD 4.999 million. These values apply only to the parameters shown in the figure.

10.2 Rebalance-Cost Ablation Waterfall

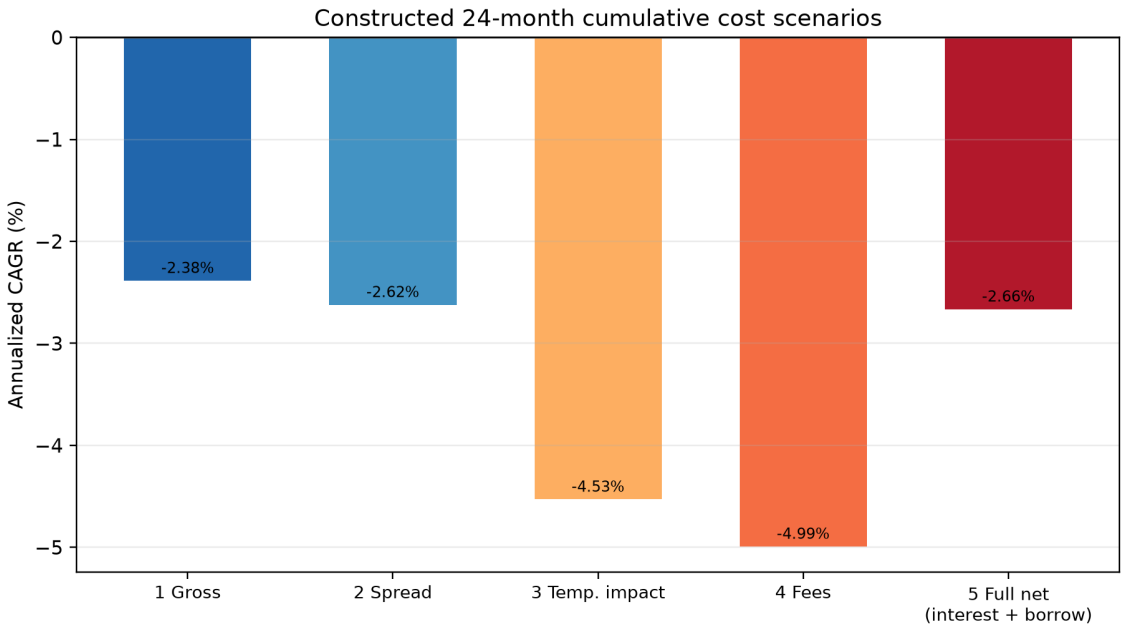


Figure 2: Layer-by-layer friction ablation in the constructed Project 4 scenario

The vertical axis is annualized CAGR calculated from the staged NAV series in the 24-month constructed sample. Each bar enables one additional cost layer in a fixed order. Since the sample path, cash interest, and borrow fees interact, this is a cumulative scenario comparison; differences between adjacent bars should not be interpreted as universal causal costs for individual components.

10.3 Fix the Result Identity Before Reading the Figures

The 24-month cost-ablation figure and the original A7-B R2R6 values retain their historical constructed-path identity; they are not the result of the current signal-interface refresh dated 2026-10-14. The historical signal implementation SHA-256 is `e16d97c61fea2f8cab1e7b80a6f20442232507e8b35b4bb2c1a1a2b51f0d0373`; do not combine the old figure with a new results table.

The current consumer was refreshed and its scope-limited acceptance recorded by the original main controller using the original CLI/config/seed `20261210` and six fixed experiments. The entrypoint is `projects/project04/cost_execution_lab/ACCEPTED_CURRENT_SIGNAL_20261014.md`. The current signal implementation SHA-256 is `b5357c4122116bacecb585ffe6625a7d14b9dc85c3f65c7ea1dffc744e0a4fd8`, and the summary SHA-256 is `3dece7d2fd979d29c5b09b03bc19f54f508963c8e738d6fddc1e58de745e0b4e`. These are all **CONSTRUCTED_TEACHING_DATA**.

For the current 36-period out-of-sample sample, CAGR is GROSS −1.9560%, TWAP −2.3459%, forecast VWAP −2.3327%, and lagged POV −6.0717%; corresponding Sharpes are −0.3334, −0.4014, −0.3994, and −1.0800. The high-friction negative case has full-sample

net CAGR −7.4392% and Sharpe −1.5050. These negative results are retained; comparisons apply only to the same constructed path and fixed scenarios. The consumer refresh followed a correction to an exposed input implementation; it involved no new preregistration and did not calibrate real Alpha, costs, or capacity. This batch reuses that accepted summary and did not rerun the CLI or copy the complete raw CSV.

11. Real-Market Example

In institutional quantitative investing, real trading-cost structures can differ substantially from assumptions in academic models. This section summarizes cost characteristics and the limits of the current quantitative-research data using public regulatory reports and institutional empirical studies.

11.1 Findings in Institutional Empirical Literature

1. **AQR study of anomaly-trading costs (Frazzini, Israel, and Moskowitz, 2012):**
   - The AQR team analyzed a database of real institutional executions from 1998 to 2011, covering 19 developed markets and close to USD 1 trillion in trading.
   - The working paper reports that estimated costs for several anomaly portfolios, based on actual execution costs, were below estimates in earlier literature based on aggregated transaction or quote data. It also uses portfolio optimization to study the tradeoff between cost and tracking error and reports differences across styles.
   - These are results for that institution's sample, period, and portfolio designs. Its impact parameters cannot be generalized as universal constants, and the paper does not establish that every quantitative strategy retains significant returns after costs.
2. **U.S. SEC Staff Report on Algorithmic Trading (2020):**
   - The report describes core algorithmic-trading processes in U.S. equities, liquidity provision by automated market makers, and order routing in a multi-venue market.
   - It summarizes improvements in several market-quality and liquidity-provision measures under normal conditions and notes that some algorithm types may amplify risk during unusual stress or volatility. It also describes the market's multi-venue, automated structure. This chapter does not extrapolate the report into a general empirical claim about spread or depth differences between large-cap and micro-cap stocks.
3. **SEC request for comment on mutual-fund transaction-cost disclosure (2003):**
   - In this historical request for comment, the SEC said that there was then no universally accepted method for calculating portfolio implementation shortfall. It compared measurement approaches and requested comments on specific algorithms, formulas, and the treatment of opportunity cost. This 2003 concept paper does not establish a current universal regulatory formula. Researchers should state and freeze the benchmark used in their experiment.

11.2 Data Available in the Current Teaching Environment and Known Gaps

The inventory in `qa/a7/ch35_ch36_batch/SOURCE_DATA_GAP.md` defines the data boundaries for the current course stage:

- **Available data:** The local environment contains monthly industry and factor-return snapshots from Kenneth French (used for macro and broad asset-pricing demonstrations) and constructed monthly signals from the Project 4 preparation layer.
- **Missing real-world materials:** The local environment does not have per-stock intraday Level 2/Level 3 order-book replay data, daily historical intraday stock-borrow rates and availability flags, or live execution records containing broker dark-pool and smart-routing fills.
- **Boundary:** All friction parameters in this chapter, examples, and later experiments—including impact coefficients λ_{impact}, participation caps, and borrow rates—are **carefully designed teaching inputs**. Readers should learn the internally consistent cost model and ledger reconciliation method, but must not treat these constructed inputs as production-market calibrations or infer evidence of a real, profitable Alpha.

---

12. Common Mistakes

Four trading-cost errors commonly arise in quantitative engineering and strategy research.

Mistake 1: Double-Charging Frictions Embedded in the Fill Price as Cash Fees

What goes wrong: A backtest matching engine first adjusts the fill price P^{fill} for spread and impact, then deducts the spread and slippage again as separate cash fees.

Incorrect code:

```
# Incorrect: double charging
fill_price = mid_price + 0.5 * spread + impact  # The buy price already includes the
    ↪ increase.

# Deduct the purchase payment.
cash -= qty * fill_price

# Error: deduct the half-spread and impact loss again!
cash -= qty * (0.5 * spread + impact)
```

Consequence and numerical error: For a trade buying 10,000 shares at a midpoint of USD 100, with a half-spread of USD 0.05 and temporary impact of USD 0.10, the actual fill price is USD 100.15. The payment is USD 1,001,500, including USD 1,500 of friction. The incorrect code over-debits cash by USD 1,500, doubles the modeled trading friction, and may falsely eliminate a strategy that would otherwise be profitable.

Correct repair:

```
# Correct ledger entry: the fill price absorbs friction; deduct explicit fees
    ↪ separately.
fill_price = mid_price + 0.5 * spread + impact
cash -= (qty * fill_price + explicit_commission)
```

Mistake 2: Multiplying by 2 Again After Total Traded Notional Already Includes Both Sides

What goes wrong: Let one-sided turnover be $\tau_{\text{turnover}} = \frac{1}{2}\sum_i |w_i^* - w_i^-|$. If total traded notional is first computed as $V_{\text{trade}} = 2\tau_{\text{turnover}}\,\text{NAV}$, it already includes both buys and sells. Multiplying $2V_{\text{trade}}$ by the cost rate per dollar traded double counts total notional. If V_{trade} was instead computed directly from the two-sided weight change $\sum_i |w_i^* - w_i^-|$, it must not be multiplied by 2 again either.

Incorrect and corrected forms:

```
# Incorrect: total_notional already includes both buy and sell notional.
total_notional = np.sum(np.abs(target_weights - current_weights)) * portfolio_nav

# Incorrect: cost_total = 2.0 * total_notional * single_trip_fee_rate

# Correct:
one_sided_turnover = 0.5 * np.sum(np.abs(target_weights - current_weights))

# Total cost, using a rate per dollar traded:
cost_total = total_notional * single_trip_fee_rate

# Equivalently, start from one-sided turnover and multiply by 2 only once:
cost_from_one_sided = (2.0 * one_sided_turnover) * portfolio_nav *
    ↪ single_trip_fee_rate
```

Mistake 3: Charging Full Commission on Unfilled or Partially Filled Orders

What goes wrong: In the ledger model used here, a limit order with no fill ($F = 0$, $R = Q$) is incorrectly charged broker commission and stamp tax based on the parent-order quantity Q.

Consequence and repair: In this model, explicit fill commission is tied to actual executed quantity F. The unfilled remainder R contributes opportunity-cost attribution in implementation shortfall; it is not charged the modeled fill commission in the cash ledger. Other account-specific order-submission or cancellation fees are outside this model.

```
# Correct repair
actual_commission = commission_per_share * abs(filled_qty)  # Use filled_qty, not
    ↪ parent_qty.
```

Mistake 4: Setting Fill VWAP to Zero When There Are No Fills, Which Breaks Return Calculations

What goes wrong: When a stock is limit-up for the day or liquidity dries up and the fill quantity is zero, the code defaults `fill_vwap` to `0.0`.

Consequence and repair: If `fill_vwap = 0.0`, later calculations of slippage or return from the fill price produce the nonsensical result $(0.0 - P_{\text{arrival}})/P_{\text{arrival}} = -100\%$, resembling a catastrophic loss.

```
# Correct repair: with no fills, VWAP must be None or np.nan.
fill_vwap = (total_fill_notional / total_filled_qty) if total_filled_qty > 0 else np.
    ↪ nan
```

---

13. Quantitative-Research Applications

In quantitative research, transaction costs should not be added as an after-the-fact patch. They belong as **an ex ante constraint embedded directly in the portfolio-optimization objective**.

13.1 Cost-Aware Portfolio Optimization

A conventional mean-variance optimizer maximizes expected return while penalizing risk:

$$\max_{w} \quad w^T \mu - \frac{\lambda_{\text{risk}}}{2} w^T \Sigma w.$$

Here λ_{risk} applies to returns and return variance at the same frequency and is a local tradeoff parameter in a return objective. It differs from the USD^{-1} parameter required by Chapter 36's dollar-IS objective; the numerical values cannot be reused across those objectives.

This frictionless optimizer can induce frequent small weight changes and generate turnover whose marginal return is below its trading cost. A cost-aware optimizer places trading costs directly in its objective:

$$\max_{w} \quad w^T \mu - \frac{\lambda_{\text{risk}}}{2} w^T \Sigma w - \frac{1}{\text{NAV}} \sum_{i=1}^{N} \text{Cost}_i(\Delta w_i), \qquad \Delta w_i = w_i - w_{i,\text{prev}}.$$

If Cost_i is in dollars, divide by NAV before subtracting it from the dimensionless return objective. Write cost as $c_i|\Delta w_i| + \kappa_i(\Delta w_i)^2$. Sufficient conditions for a convex QP are fixed NAV > 0, a symmetric positive semidefinite covariance matrix Σ at the same frequency, $\lambda_{\text{risk}} \geq 0$, dollar cost coefficients $c_i \geq 0$ and $\kappa_i \geq 0$, and linear portfolio constraints. Introduce $u_i \geq \Delta w_i$ and $u_i \geq -\Delta w_i$, replacing the absolute-value penalty in the maximization objective by $-c_i u_i/\text{NAV}$. This gives an equivalent convex quadratic program (QP). If $c_i > 0$, the penalty makes $u_i = |\Delta w_i|$ at an optimum; if $c_i = 0$, u_i may be slack without changing that term or the optimal objective value. Nonnegativity matters: with $\Delta w = 0$, $c = -1$, NAV $= 1$, and zero return and risk terms, the original objective is 0, whereas the relaxed problem becomes unbounded as $u \to \infty$.

13.2 Trade Buffers and No-Trade Zones

A researcher can define a no-trade band $[w_i^* - \delta_i, w_i^* + \delta_i]$ around a target weight and trade only when holdings drift beyond the threshold. For the static trading problem in Chapter 29, a no-trade region can be characterized by the applicable KKT conditions; an arbitrary rectangular buffer should not be treated as optimal under a general correlated covariance matrix. Buffer widths are model parameters: set and freeze them on training data, then compare net returns, turnover, and risk on time-held-out test data. Do not assume in advance that a buffer preserves a given fraction of Alpha or reduces costs by a given fraction. A trade buffer must also respect actual-fill and cash constraints.

---

14. Chapter Summary

This chapter closes the loop from microstructure frictions to self-financing portfolio accounting:

1. **Cost categories and pricing logic:** Explicit fees (such as commissions and applicable taxes or fees) are separate cash deductions. Among implicit costs, half-spread and temporary impact are embedded directly in the fill VWAP P^{fill}; permanent impact changes the new market midpoint and affects later valuation. Financing and borrow fees are settled by period based on balances and the day-count convention.
2. **The core rule against double charging:** Because P^{fill} already absorbs the half-spread and impact through a worse buy or sell price, do not deduct spread or impact again as a separate fee after posting the actual fill payment to cash.
3. **Self-financing rebalances:** Frictions reduce the physical quantity of an asset that can be purchased. On the buy side, solve the nonlinear self-financing equation to determine order size and prevent an overdraft.
4. **Four-component implementation-shortfall decomposition:**

$$\begin{aligned}\text{IS} = \text{Delay} + \text{Execution friction} \\ + \text{Opportunity cost} + \text{Explicit fees}.\end{aligned}$$

This decomposition is algebraically equivalent to the direct-dollar method and provides a benchmark for evaluating trading algorithms.

5. **Strategy capacity and life cycle:** Marginal market impact rises nonlinearly with AUM, so excess returns may eventually encounter a capacity ceiling. Under the square-root net-return function specified here, the dollar-profit maximum is 4/9 of break-even scale; this is not a market constant that can be extrapolated directly.

---

15. Exercises

35-E01 Conceptual Exercise: Avoiding Double Charges When Frictions Are Embedded in Fill Price

A quantitative researcher writing an event-driven matching engine designs the following ledger logic. For a buy, the matching engine calculates the fill price as $P^{\text{fill}} = P^{\text{mid}} + 0.5 \times S + \lambda_{\text{impact}} Q$. The cash-settlement function then executes:

```
cash_balance -= Q * fill_price
cash_balance -= Q * (0.5 * S)  # Researcher's note: deduct spread loss.
cash_balance -= Q * (impact_coef * Q)  # Researcher's note: deduct impact loss.
cash_balance -= commission
```

Clearly identify the serious accounting and mathematical error, write the correct cash-recursion formula, and calculate how much more cash the incorrect logic deducts when $Q =$ 1,000 shares, $P^{\text{mid}} = \$50.00$, $S = \$0.04$, $\lambda_{\text{impact}} Q = \$0.08$, and commission is USD 5.00.

35-E02 Hand Calculation: Tradable Risk-Asset Turnover and Total Traded Notional

A multi-asset portfolio manages USD 5,000,000 and contains three stocks plus cash. Assume the pre- and post-trade weights use the same NAV and a common mark, and all orders fill at that mark, so the weight changes map to actual traded stock notional. At the start of the month its weights are $w_{\text{pre}} = [0.40, 0.30, 0.20, 0.10]$ (the fourth entry is cash); the optimizer's month-end target weights are $w^* = [0.20, 0.50, 0.10, 0.20]$. Under Section 5.6, capacity and transaction frictions use the tradable risky-security set $\mathcal{S}_{\text{risky}} = \{1, 2, 3\}$; cash is excluded from stock turnover.

1. Calculate one-sided turnover for the tradable risky securities, $\tau_{\text{risky,one-sided}}$.
2. Calculate the two-sided total traded notional in the stocks, in USD.
3. If the combined one-way stock-trading fee and slippage rate is 12 bps, applied to actual buy and sell notional, calculate the total friction cost in USD and in basis points of total portfolio assets.

35-E03 Hand Calculation: Full Four-Factor IS Decomposition with a Partial Fill

A quantitative fund executes a large buy order:

- At decision time t_{dec}, the stock midpoint is $P^{\text{dec}} = \$80.00$. The planned parent order is $Q = 10{,}000$ shares.
- When the order reaches the venue at t_{arr}, the midpoint has risen to $P^{\text{arr}} = \$80.10$.
- At the end of the execution window, the actual fill is $F = 8{,}000$ shares at an average price of $P^{\text{fill}} = \$80.25$; the remaining $R = 2{,}000$ shares are canceled unfilled.
- At the cutoff evaluation time t_{term}, the stock midpoint is $P^{\text{term}} = \$80.50$.
- The broker charges a fixed commission of USD 0.005 per share on the actual fill.

Calculate separately: (1) delay cost, execution-friction cost, opportunity cost, and explicit fees; and (2) total implementation shortfall in USD, cross-checked using the direct-dollar method.

35-E04 Hand Calculation: Stock-Borrow Fees and Financing Interest

A leveraged long-short hedge fund holds the following positions on May 1, 2026:

- Long assets with a market value of USD 15,000,000.00.
- A short book of five stocks, with annual borrow rates $[0\%, 12\%, 0\%, 0\%, 0\%]$ and absolute short market values [USD 4,900,000, USD 100,000, 0, 0, 0], respectively; total short market value is USD 5,000,000.00.
- Available cash is −USD 2,000,000.00 (a leveraged borrowing balance), with an annual financing rate of 6.00%.
- The positions remain unchanged through May 31, 2026 (30 days), using the Actual/360 day-count convention.

Calculate each security's borrow fee, financing interest, and total erosion in USD. Explain why applying a simple average borrow rate to total short market value gives the wrong answer.

---

16. Programming Exercises

35-P01 Programming Exercise: Basic Trading-Cost Calculator and Anti-Double-Charge Unit Test

Write a pure function named `execution_ledger_step`. It takes prior cash, a holdings dictionary, a list of confirmed fill records (the signed quantity in each record is the actual fill F, not the parent-order quantity Q), and microstructure-friction parameters; it returns updated cash and a holdings dictionary.

Requirements:

1. Calculate half-spread and linear impact for buys and sells.
2. Include an explicit per-share commission.
3. Assert that zero fills incur no commission, buy fills are not below the midpoint, and sell fills are not above the midpoint.
4. Write pytest unit tests with a counterexample showing that a second spread deduction fails the assertion.

35-P02 Programming Exercise: Solver from Self-Financing Target Weights to Actual Filled Shares

Write a numerical solver named `solve_purchasable_shares`. Given available cash C, target-stock arrival midpoint P^{mid}, half-spread $S/2$, linear-impact coefficient λ_{impact}, and per-share commission c_{share}:

1. Solve for the physical share quantity F^* that makes total outlay (purchase payment plus commission) equal to C, retain an integer number of shares by rounding down.
2. Return the unused cash residual after rounding down.
3. Verify that when frictions are ignored ($s = 0$, $\lambda_{\text{impact}} = 0$, and $c = 0$), the solution reduces exactly to $\lfloor C/P^{\text{mid}} \rfloor$.

35-P03 Programming Exercise: Strategy-Capacity Sensitivity and Break-Even Curve

Implement `simulate_capacity_curve`. Gross return, one-sided turnover, average daily dollar volume (ADV), and the D trading days must all refer to the same evaluation period; use $D = 252$ for an annual example and $D = 21$ for this chapter's monthly example. Inputs are gross excess return as a decimal, one-sided turnover τ_{turnover} for that period, fixed cost in bps per actual traded notional, square-root impact coefficient γ, market average daily dollar volume ADV, and the number of trading days in that period. From

$$V_{\text{trade}} = 2\tau_{\text{turnover}} A, \qquad p = V_{\text{trade}}/(D\,ADV), \qquad c_{\text{trade}} = c_0 + \gamma\sqrt{p},$$

return net Alpha and dollar net profit over a logarithmic AUM grid, and analytically calculate break-even scale and the AUM maximizing dollar net return. Validate the inputs. If fixed cost already exceeds gross Alpha, or if the impact coefficient is zero, return "no finite capacity solution" explicitly instead of fabricating a number.

17. Challenge Problems

35-C01 Challenge: Tradeoff Between Nonlinear Impact and a Per-Slot Penalty over a Fixed Total Duration

Suppose a trader must buy a total of X shares within a total time horizon T. The trader divides the order evenly across N independent discrete slots, trading $x = X/N$ shares per slot. Temporary market impact within each slot follows the power law $g(x) = \eta x^{\alpha}$, where $\alpha > 0$. This construction assumes that liquidity resets independently in each slot and that g does not vary with slot duration T/N.

This is not a general conclusion that "more slices necessarily reduce impact within a fixed total horizon." With equal execution across slots, each slot lasts $\Delta t = T/N$ and execution speed is $v = (X/N)/(T/N) = X/T$, which is unchanged by N. If impact depends only on speed, $g = g(v)$, its per-share impact term does not fall just because the number of slices increases. The decreasing-in-N cost in this exercise follows from a different explicit assumption: per-slot impact is modeled as a function of the slot quantity x, and liquidity resets independently in each slot.

Also assume that each additional slot incurs a fixed exogenous penalty, $\text{RiskCost}(N) = \beta N$, with $\beta > 0$. Under fixed T, βN is specified directly as a per-slot model cost; it is not a standard delay-risk term derived from a risk process over the total horizon.

1. Write total execution cost (total temporary-impact dollars plus the risk cost) as a function of N.
2. Derive the analytic optimizer N^* when impact is linear ($\alpha = 1.0$).
3. Discuss the convexity of the objective and the extremum condition for the classic square-root impact case ($\alpha = 0.5$).

35-C02 Challenge: Net-Return Risk with Borrow Recalls and State-Dependent Costs

Construct an annual long-short portfolio example. Let gross excess return R_g (assuming a zero benchmark or risk-free rate) have mean 12% and standard deviation 8%, so the gross Sharpe is 1.5. Annual borrow fees equal 2% of NAV, and normal trading costs equal 1%. If R_g is not excess return relative to a zero benchmark, specify the benchmark before recalculating Sharpe. A high-spread state occurs with probability 10% and adds 0.60% of NAV in costs relative to the normal state. A recall event occurs with probability 5% and causes an additional forced-liquidation loss of 20% of NAV. First assume the cost events are mutually independent and independent of gross return.

1. Calculate the mean, variance, and net Sharpe, and explain why the result depends on the constructed parameters in this problem.
2. Now consider a leveraged short. If the security jumps up by a proportion u at recall, short notional exposure divided by NAV is L_s, and spread plus impact to cover the position is c_{liq} as a proportion of NAV, write the single-event loss as $L_s u + c_{\text{liq}}$. Give a sufficient condition

under which one recall reduces NAV to zero or below, and explain how a positive recall probability then implies a positive probability of a single-event ruin.

3. Distinguish "negative geometric net growth, with long-run wealth tending to zero" from "hitting a liquidation or bankruptcy boundary within a finite horizon." Explain why the latter also requires an explicit loss support or liquidation-threshold condition.

---

18. Solutions and Solution Hints

35-E01 Full Solution

1. **Locate the error.** In the first line, `cash_balance -= Q * fill_price`, the defined `fill_price` already includes half-spread $0.5S$ and temporary impact $\lambda_{\text{impact}}Q$. The account therefore pays the full purchase cost $Q(P^{\text{mid}} + 0.5S + \lambda_{\text{impact}}Q)$. The second and third lines deduct $Q(0.5S)$ and $Q(\lambda_{\text{impact}}Q)$ again, double charging spread and market impact.
2. **Correct recursion:**

$$\text{cash_balance}_{\text{after}} = \text{cash_balance}_{\text{before}} - Q \cdot P^{\text{fill}} - \text{commission}.$$

3. **Amount over-deducted:** The error equals the sum of the second and third lines:

$$\begin{aligned}\Delta\text{Cash}_{\text{err}} &= Q(0.5S) + Q(\lambda_{\text{impact}}Q) \\ &= 1{,}000 \times 0.02 + 1{,}000 \times 0.08 \\ &= 20.00 + 80.00 = \text{USD } 100.00.\end{aligned}$$

The erroneous code removes an extra USD 100.00 from the account.

35-E02 Full Solution

1. **One-sided turnover for tradable risky securities $\tau_{\text{risky,one-sided}}$.** The absolute stock-weight changes are 0.20 for asset 1, 0.20 for asset 2, and 0.10 for asset 3. Their sum is 0.50, so

$$\tau_{\text{risky, one-sided}} = \frac{1}{2}\sum_{i \in \mathcal{S}_{\text{risky}}} |\Delta w_i| = \frac{1}{2}(0.50) = 0.25 \quad (25.0\%).$$

If the cash-weight change of 0.10 is included in a separately reported all-asset reallocation rate, the result is 0.30. That is a different statistic and cannot be used to infer stock traded notional.

2. **Two-sided total traded notional in stocks.** Only the stocks (assets 1, 2, and 3) trade in the market; cash is a passive settlement account. The sum of absolute stock-weight changes is

$$\sum_{i=1}^{3} |\Delta w_i| = 0.20 + 0.20 + 0.10 = 0.50.$$

Thus two-sided stock traded notional is

$$V_{\text{trade}} = 5{,}000{,}000 \times 0.50 = \text{USD } 2{,}500{,}000.00.$$

3. **Friction cost and basis points.** The combined one-way friction rate is 12 bps = 0.0012. Total friction is

$$\text{Cost} = V_{\text{trade}} \times 0.0012 = 2{,}500{,}000 \times 0.0012 = \text{USD } 3{,}000.00.$$

In basis points of total portfolio assets (USD 5 million),

$$\frac{3{,}000}{5{,}000{,}000} = 0.0006 = 6.0 \text{ bps}.$$

In other words, 25% one-sided stock turnover corresponds to 50% two-sided stock traded notional and cost erosion of 6.0 bps of the total portfolio.

35-E03 Full Solution

Given $Q = 10{,}000$, $F = 8{,}000$, $R = 2{,}000$; $P^{\text{dec}} = \text{USD } 80.00$, $P^{\text{arr}} = \text{USD } 80.10$, $P^{\text{fill}} = \text{USD } 80.25$, $P^{\text{term}} = \text{USD } 80.50$; and $c_{\text{share}} = \text{USD } 0.005$.

1. **Calculate each of the four terms.**
   - Delay cost:

$$\text{Delay} = Q(P^{\text{arr}} - P^{\text{dec}}) = 10{,}000(80.10 - 80.00) = 10{,}000 \times 0.10 = \text{USD } 1{,}000.00.$$

   - Execution friction:

$$\text{Execution} = F(P^{\text{fill}} - P^{\text{arr}}) = 8{,}000(80.25 - 80.10) = 8{,}000 \times 0.15 = \text{USD } 1{,}200.00.$$

   - Opportunity cost:

$$\text{Opportunity} = R(P^{\text{term}} - P^{\text{arr}}) = 2{,}000(80.50 - 80.10) = 2{,}000 \times 0.40 = \text{USD } 800.00.$$

   - Explicit fee:

$$\text{Explicit} = F\, c_{\text{share}} = 8{,}000 \times 0.005 = \text{USD } 40.00.$$

2. **Total implementation shortfall and cross-check.** The four terms sum to

$$\text{IS} = 1{,}000.00 + 1{,}200.00 + 800.00 + 40.00 = \text{USD } 3{,}040.00.$$

Check with the direct-dollar method:

$$\begin{aligned} \text{IS}_{\text{direct}} &= F(P^{\text{fill}} - P^{\text{dec}}) + R(P^{\text{term}} - P^{\text{dec}}) + \Xi \\ &= 8{,}000(80.25 - 80.00) + 2{,}000(80.50 - 80.00) + 40.00 \\ &= 8{,}000 \times 0.25 + 2{,}000 \times 0.50 + 40.00 \\ &= 2{,}000.00 + 1{,}000.00 + 40.00 = \text{USD } 3{,}040.00. \end{aligned}$$

The two methods agree exactly.

35-E04 Full Solution

1. **Stock-borrow fees.** Calculate the fee security by security from short market value, borrow rate, and holding period:

$$B = 4{,}900{,}000 \times 0\% \times \frac{30}{360} + 100{,}000 \times 12\% \times \frac{30}{360} = \text{USD } 1{,}000.00.$$

   The simple equal-weight average rate is 2.4%. Multiplying it by total short market value would give $5{,}000{,}000 \times 2.4\% \times 30/360 = \text{USD } 10{,}000$, ten times the correct fee. The correct aggregate rate must be weighted by short market value and holding time; an unweighted average alone does not determine borrow cost.

2. **Financing interest.** The negative cash borrowing balance is USD 2,000,000.00. At an annual financing rate of $r_{\text{debit}} = 6.00\%$,

$$\text{Cost}_{\text{financing}} = 2{,}000{,}000 \times 0.0600 \times \frac{30}{360} = 120{,}000 \times \frac{1}{12} = \text{USD } 10{,}000.00.$$

3. **Total erosion:**

$$\text{TotalCost} = 1{,}000.00 + 10{,}000.00 = \text{USD } 11{,}000.00.$$

   In this constructed month, cash falls by USD 11,000. This result depends only on the constructed positions, rates, holding period, and Actual/360 assumption in the problem.

35-P01 Reference Implementation

```
from typing import Dict, List, Any
import numpy as np
import pytest

def execution_ledger_step(
    cash_pre: float,
    positions_pre: Dict[str, float],
    orders: List[Dict[str, Any]],
) -> tuple[float, Dict[str, float]]:
    cash_post = cash_pre
    positions_post = positions_pre.copy()

    for order in orders:
        symbol = order["symbol"]
        qty = float(order["signed_qty"])
        mid = float(order["mid_price"])
        half_spread = float(order["half_spread"])
        impact_coef = float(order.get("impact_coef", 0.0))
        commission_per_share = float(order.get("commission_per_share", 0.0))

        if qty == 0.0:
            continue

        direction = np.sign(qty)
        impact = impact_coef * abs(qty)
        fill_price = mid + direction * (half_spread + impact)
```

```
        # Boundary and economic assertions.
        if direction > 0:
            assert fill_price >= mid, "A buy fill price must not be below the
    ↪ midpoint."
        else:
            assert fill_price <= mid, "A sell fill price must not be above the
    ↪ midpoint."

        commission = commission_per_share * abs(qty)

        # Correct accounting: do not deduct spread twice.
        cash_post -= (qty * fill_price + commission)
        positions_post[symbol] = positions_post.get(symbol, 0.0) + qty

    return cash_post, positions_post

# Automated test checks.
def test_anti_double_counting():
    cash_0 = 100_000.0
    pos_0 = {"A": 0.0}
    orders = [{
        "symbol": "A",
        "signed_qty": 1000.0,
        "mid_price": 50.0,
        "half_spread": 0.02,
        "impact_coef": 1e-4, # Impact = 0.10.
        "commission_per_share": 0.005
    }]
    cash_1, pos_1 = execution_ledger_step(cash_0, pos_0, orders)
    # fill_price = 50.0 + 0.02 + 0.10 = 50.12.
    # expected cash = 100000 - (1000 * 50.12 + 5) = 100000 - 50125 = 49875.0.
    assert np.isclose(cash_1, 49875.0, atol=1e-4)
    assert np.isclose(pos_1["A"], 1000.0, atol=1e-4)

    # An extra spread debit is the counterexample: it must fail the cash assertion.
    with pytest.raises(AssertionError):
        assert np.isclose(cash_1 - 1000.0 * 0.02, 49875.0, atol=1e-4)


def test_zero_fill_and_sell_side_boundary():
    cash_0 = 100_000.0
    pos_0 = {"A": 100.0}
    cash_1, pos_1 = execution_ledger_step(cash_0, pos_0, [{
        "symbol": "A", "signed_qty": 0.0, "mid_price": 50.0,
        "half_spread": 0.02, "commission_per_share": 0.01,
    }])
    assert cash_1 == cash_0 and pos_1 == pos_0

    cash_2, pos_2 = execution_ledger_step(cash_1, pos_1, [{
        "symbol": "A", "signed_qty": -10.0, "mid_price": 50.0,
        "half_spread": 0.02, "impact_coef": 1e-4,
        "commission_per_share": 0.005,
    }])
    assert cash_2 > cash_1 and pos_2["A"] == 90.0
```

35-P02 Reference Implementation

```python
import numpy as np

def solve_purchasable_shares(
    cash: float,
    mid_price: float,
    half_spread: float,
    impact_coef: float,
    commission_per_share: float,
) -> tuple[int, float]:
    """
    Solve for the integer number of buy shares under trading frictions:
    F * (mid + half_spread + impact_coef * F) + commission * F <= cash
    Quadratic: impact_coef * F^2 + (mid + half_spread + commission) * F - cash = 0
    """
    values = (cash, mid_price, half_spread, impact_coef, commission_per_share)
    if not all(np.isfinite(value) for value in values):
        raise ValueError("All inputs must be finite.")
    if cash < 0 or mid_price <= 0 or half_spread < 0 or impact_coef < 0 or
    ↪ commission_per_share < 0:
        raise ValueError("Cash, price, or cost inputs are outside the allowed range."
    ↪ )
    if cash == 0:
        return 0, 0.0

    A = impact_coef
    B = mid_price + half_spread + commission_per_share
    C = cash

    if A == 0.0:
        # Linear limiting case.
        f_exact = C / B
    else:
        # Equivalent positive root; avoids subtracting nearly equal values for small
    ↪ impact coefficients.
        f_exact = 2 * C / (B + np.sqrt(B**2 + 4 * A * C))

    f_int = int(np.floor(f_exact))

    # Calculate actual cash used and residual cash.
    actual_fill_price = mid_price + half_spread + impact_coef * f_int
    cash_used = f_int * actual_fill_price + commission_per_share * f_int
    residual_cash = cash - cash_used

    assert residual_cash >= 0.0, "Actual outlay must not exceed available cash."
    return f_int, residual_cash
```

35-P03 Reference Implementation

```python
import math
import numpy as np

def simulate_capacity_curve(
    alpha_gross: float,
    tau_period: float,
    fixed_cost_bps: float,
    impact_gamma: float,
    adv_market_per_day: float,
    trading_days_in_period: int = 252,
    n_points: int = 100,
) -> dict:
    values = (alpha_gross, tau_period, fixed_cost_bps, impact_gamma,
    ↪ adv_market_per_day)
    if not all(math.isfinite(x) for x in values):
        raise ValueError("all numeric inputs must be finite")
    if tau_period <= 0 or fixed_cost_bps < 0 or impact_gamma < 0:
        raise ValueError("return and cost inputs are outside their allowed range")
    if adv_market_per_day <= 0 or trading_days_in_period <= 0 or n_points < 2:
        raise ValueError("ADV and period lengths must be positive")

    aum = np.logspace(5, 8, n_points)
    traded_notional = 2 * tau_period * aum
    participation = traded_notional / (trading_days_in_period * adv_market_per_day)
    cost_per_traded_dollar = fixed_cost_bps * 1e-4 + impact_gamma * np.sqrt(
    ↪ participation)
    net_alpha = alpha_gross - (traded_notional / aum) * cost_per_traded_dollar
    dollar_profit = aum * net_alpha

    alpha_after_fixed = alpha_gross - 2 * tau_period * fixed_cost_bps * 1e-4
    k = impact_gamma * (2 * tau_period) ** 1.5 / math.sqrt(
        trading_days_in_period * adv_market_per_day
    )
    if alpha_after_fixed > 0 and k > 0:
        breakeven_aum = (alpha_after_fixed / k) ** 2
        optimal_aum = 4 * breakeven_aum / 9
    else:
        breakeven_aum = None
        optimal_aum = None

    return {
        "aum": aum,
        "net_alpha": net_alpha,
        "dollar_profit": dollar_profit,
        "breakeven_aum": breakeven_aum,
        "optimal_aum": optimal_aum,
    }

# Constructed monthly example from section 8.8, expressed with its 21-day horizon.
r = simulate_capacity_curve(0.012, 1.0, 8.25, 0.05, 100_000_000, 21)
assert abs(r["breakeven_aum"] - 11_247_862.5) < 1.0
assert abs(r["optimal_aum"] - 4_999_050.0) < 1.0
```

```
assert simulate_capacity_curve(-0.01, 1.0, 0.0, 0.05, 100_000_000, 21)["breakeven_aum
    ↪ "] is None
assert simulate_capacity_curve(0.012, 1.0, 0.0, 0.0, 100_000_000, 21)["optimal_aum"]
    ↪ is None
```

The function returns an analytic break-even scale of about USD 11.25 million and a maximum net-dollar-return scale of about USD 5.00 million under the monthly teaching parameters. Recalculate for other inputs; do not carry these values over.

35-C01 Open-Challenge Argument and Grading Rubric

1. **Construct the total-cost function.** In N discrete slots, the quantity traded per slot is $x = X/N$. Temporary impact per share in each slot is $\eta x^{\alpha} = \eta(X/N)^{\alpha}$. The impact outlay in one slot is
$$x \cdot \eta x^{\alpha} = \eta x^{1+\alpha} = \eta \left(\frac{X}{N}\right)^{1+\alpha}.$$
Total temporary-impact dollars across N slots are
$$\text{Impact}_{\text{total}}(N) = N \times \eta \left(\frac{X}{N}\right)^{1+\alpha} = \eta X^{1+\alpha} N^{-\alpha}.$$
The total execution-cost function is
$$J(N) = \eta X^{1+\alpha} N^{-\alpha} + \beta N.$$
2. **Analytic solution for linear impact ($\alpha = 1.0$).** When $\alpha = 1.0$, $J(N) = \eta X^2 N^{-1} + \beta N$. The first-order condition is
$$\frac{dJ}{dN} = -\eta X^2 N^{-2} + \beta = 0,$$
giving
$$N^* = X\sqrt{\frac{\eta}{\beta}}.$$
The second derivative is $d^2J/dN^2 = 2\eta X^2 N^{-3} > 0$, so the objective is strictly convex on the continuous domain $N > 0$, and its continuous critical point is the unique global minimum. The actual number of slots must be an integer $N \geq 1$. If $N^* < 1$, choose $N = 1$; otherwise compare the costs at the feasible adjacent integers $\lfloor N^* \rfloor$ and $\lceil N^* \rceil$. This solution applies only to the stated construction with per-slot liquidity resets and an exogenous βN penalty; it is not a general theorem for execution over a fixed duration.
3. **Extremum for square-root impact ($\alpha = 0.5$).** When $\alpha = 0.5$, $J(N) = \eta X^{1.5} N^{-0.5} + \beta N$. The first-order condition is
$$\frac{dJ}{dN} = -0.5\eta X^{1.5} N^{-1.5} + \beta = 0.$$
The critical point is
$$N^* = \left(\frac{0.5\eta X^{1.5}}{\beta}\right)^{2/3} = \left(\frac{\eta}{2\beta}\right)^{2/3} X.$$
The second derivative is $d^2J/dN^2 = 0.75\eta X^{1.5} N^{-2.5} > 0$, again making the continuous objective strictly convex. The discrete choice still respects the feasible boundary $N \geq 1$ and compares costs at adjacent integers. This analytic curve describes only the stated exogenous-penalty and per-slot-reset assumptions.

Grading rubric (100 points):

- Correctly construct $J(N)$ and identify the dimensional conversion from per-slot impact to total impact dollars (30 points).
- Derive the linear-impact optimizer, verify second-order convexity, and explain the economic intuition (30 points).
- Solve the square-root-impact first-order condition and verify the second-order sufficient condition (25 points).
- Apply the discrete integer constraint and explain how to choose between rounding down and up in practice (15 points).

35-C02 Reference Argument and Grading Rubric

1. Let the high-spread-state indicator be $Z \sim \text{Bernoulli}(0.10)$ and the recall indicator be $Y \sim \text{Bernoulli}(0.05)$, independent of each other and of gross return. Under the problem's cost definition,
$$R_{\text{net}} = R_g - 0.02 - 0.01 - 0.006Z - 0.20Y.$$
Therefore,
$$\mathbb{E}[R_{\text{net}}] = 0.12 - 0.02 - 0.01 - 0.006(0.10) - 0.20(0.05) = 0.0794,$$
or 7.94%. The variance is
$$0.08^2 + 0.006^2(0.10)(0.90) + 0.20^2(0.05)(0.95) = 0.00830324.$$
The standard deviation is about 9.1122%, and the net Sharpe is approximately $0.0794/0.091122 \approx 0.87$. A gross Sharpe of 1.5 alone does not determine net Sharpe; this decline follows from the numerical assumptions in this problem.
2. If a recall event's loss as a fraction of NAV is $L_s u + c_{\text{liq}}$, a value of at least 1 means that a single price jump in this model can exhaust all NAV. If the event probability $p_{\text{recall}} > 0$, then the probability of at least one event over a finite horizon is positive. Under the simplifying assumption of independent events with a fixed annual probability, the probability over T years is $1 - (1 - p_{\text{recall}})^T$. Real margin-liquidation thresholds may be above zero NAV, and actual jump and impact distributions require data estimation.
3. If one event cannot exhaust NAV, but positive-NAV returns are independent and identically distributed, satisfy $\mathbb{E}[\log(1 + R_{\text{net}})] < 0$, and have an integrable logarithm, long-run geometric wealth tends to zero almost surely by the law of large numbers. The second-order approximation
$$\mathbb{E}[\log(1 + R)] \approx \mathbb{E}[R] - \tfrac{1}{2}\mathbb{E}[R^2]$$
should be used only for small return magnitudes and when returns remain well above $R = -1$; it is not equivalent to bankruptcy within a finite horizon.

Grading rubric (100 points): Correct mean, variance, and Sharpe calculations (35 points); derivation of single-recall NAV loss and the positive-event-probability condition (35 points); correct distinction between negative geometric growth and finite-horizon liquidation (20 points); explanation of independence and constructed-parameter boundaries (10 points).

19. Further Reading

To build a complete academic and industry understanding of transaction costs, market microstructure, and algorithmic execution, read the following sources in sequence.

1. **Must Know**
   - **Perold, A. F. (1988).** *The Implementation Shortfall: Paper vs. Reality. The Journal of Portfolio Management*, 14(3), 4–9. JPM publisher record.
     - *Why read it:* A bibliographic lead related to implementation shortfall. The 2026-10-09 source review checked publisher and author bibliographic metadata only; it did not read the paper's text. This chapter does not rely on this entry for a specific definition, formula, or priority claim. This batch carried forward that historical status without a new search or full-text review.
   - **Almgren, R., & Chriss, N. (2000).** *Optimal Execution of Portfolio Transactions. Journal of Risk*, 3(2), 5–39.
     - *Why read it:* A classic foundation for algorithmic execution and impact modeling. Read Sections 1–3 on the physical decomposition of temporary and permanent impact and the derivation of a discrete-time mean-variance optimal execution trajectory.
2. **Should Know**
   - **Frazzini, A., Israel, R., & Moskowitz, T. J. (2012).** *Trading Costs of Asset Pricing Anomalies.* AQR Working Paper.
     - *Why read it:* A representative industry study using institutional order data to compare actual execution costs with hypothetical spread estimates in CRSP data. Read its empirical methods and the differences between the two estimates.
   - **Huberman, G., & Stanzl, W. (2004).** *Price Manipulation and Quasi-Arbitrage. Econometrica*, 72(4), 1247–1275. DOI.
     - *Why read it:* Under its assumptions about time-independent price updates and the market, the paper discusses the form of permanent impact needed to exclude quasi-arbitrage. Its result cannot be generalized beyond the model conditions as a universal empirical law.
3. **Advanced**
   - **Bouchaud, J.-P., Gefen, Y., Potters, M., & Wyart, M. (2004).** *Fluctuations and Response in Financial Markets: The Subtle Nature of "Random" Price Changes. Quantitative Finance*, 4(2), 176–194.
     - *Why read it:* Optional follow-up reading for a later source review. The current source record does not include a review of this paper, so no summary of its derivations is provided here. This chapter does not rely on it for claims about square-root impact or decay kernels.
4. **Optional**
   - **U.S. Securities and Exchange Commission (SEC) (2020).** *Staff Report on Algorithmic Trading in U.S. Capital Markets.*

  - *Why read it:* For background on order routing, market-maker internalization, and dark-pool execution in today's fragmented, multi-venue markets.

---

20. References

The core references cited in this chapter and the specific claims they support are bounded as follows:

- PEROLD_1988: Perold, A. F. (1988), "The Implementation Shortfall: Paper vs. Reality," *The Journal of Portfolio Management*, 14(3), 4–9. JPM publisher record.
  - *Supported claim:* The 2026-10-09 source review checked the HBS author page and publisher bibliographic record, supporting only the author's name, title, and publication details. The full paper was not read; this entry does not support the specific definition or formula in Section 6.2 or a priority claim.
- AQR_COST: Frazzini, A., Israel, R., & Moskowitz, T. J. (2012), *Trading Costs of Asset Pricing Anomalies*, AQR Capital Management Working Paper. URL.
  - *Supported claim:* Provides context in Section 11 about institutional trade data at scale (1998–2011, 19 developed markets, and close to USD 1 trillion of executions) and differences from estimates in academic models. Its parameters must not be applied directly as constructed inputs for this chapter.
- ALMGREN_CHRISS_2000: Almgren, R., & Chriss, N., *Optimal Execution of Portfolio Transactions*, *Journal of Risk*, 3(2), 5–39 (2000). DOI: 10.21314/JOR.2001.041.
  - *Supported claim:* Supports the classic mathematical definition and modeling framework in Sections 5.4 and 6.1 that separates temporary impact (which erodes the immediate fill price) from permanent impact (which shifts the midpoint).
- SEC_IS_2003: U.S. Securities and Exchange Commission, *Request for Comments on Measures To Improve Disclosure of Mutual Fund Transaction Costs*, Release Nos. 33-8349, 34-48952, IC-26313 (2003). URL.
  - *Supported claim:* Supports the historical regulatory context in Sections 5.3 and 11.1 linking implementation shortfall to the paper portfolio, and the historical statement that the SEC said in 2003 there was no universally accepted method for portfolio implementation shortfall. It does not support claims about current universal formulas or every regulatory regime.
- CESARI_MARZO_ZAGAGLIA_2012: Cesari, R., Marzo, M., & Zagaglia, P., *Effective Trade Execution*, arXiv:1206.5324 [q-fin.TR] (2012). URL.
  - *Supported claim:* Supports cross-checking the terminology in Sections 5 and 14 for execution frictions, TWAP/VWAP scheduling benchmarks, and implementation-shortfall classifications. It is background literature, not the sole implementation standard for an industry.
- SEC_ALGO: U.S. Securities and Exchange Commission, *Staff Report on Algorithmic Trading in U.S. Capital Markets* (2020). URL.
  - *Supported claim:* Supports the summary in Section 11.1 of multi-venue market structure, the effects of algorithmic trading, changes in several market-quality and liquidity

measures under normal conditions, and risks during stress or volatility. It does not support a general claim about spread, depth, or cost differences between large-cap and micro-cap stocks.

- `HUBERMAN_STANZL_2004`: Huberman, G., & Stanzl, W., *Price Manipulation and Quasi-Arbitrage*, *Econometrica*, 72(4), 1247–1275 (2004). DOI.
  - *Supported claim:* Used only for the model-boundary note in Section 5.4: under time-independent price updates, absence of quasi-arbitrage, and independent temporary and permanent components, only the permanent-update component is required to be linear. This is not a universal empirical law.

Chapter 36 — Algorithmic Execution and Trade Scheduling

Existing engineering evidence is cited under its original acceptance identity. Main thread: convert portfolio target weights into signed parent orders, balance market impact against execution risk on a discrete execution clock, and attribute and benchmark execution quality ex post through an implementation shortfall decomposition. Prerequisites: Chapter 9, Optimization Foundations; Chapter 34, Market Microstructure; and Chapter 35, Transaction Costs and Capacity. After reading, you should be able to schedule TWAP, forecast-volume VWAP, and POV orders; compute the implementation shortfall decomposition with partial and unfilled quantities; derive the discrete Almgren–Chriss mean-variance objective; and reconcile cash and holdings under a causal clock.

1. Learning Objectives

After completing this chapter, you should be able to:

1. Distinguish four execution benchmarks—Decision Price, Arrival Price, Ex-Post Market VWAP, and Terminal Mark Price—and explain their evaluation settings and economic meanings.
2. Construct discrete child-order schedules for a signed parent order using Time-Weighted Average Price (TWAP), forecast-volume weighting (Forecast-Volume VWAP), and lagged observed volume participation (POV).
3. Distinguish decision, send, arrival, fill, marking, and portfolio-signal times, and explain why monthly signal weights must not be presented as intraday microstructure signals.
4. Compute signed implementation shortfall (IS), decompose it exactly into delay cost, execution cost, unfilled opportunity cost, and explicit fees, and prove its algebraic identity with the direct monetary comparison of paper and actual portfolios.
5. Handle partial-fill and no-fill boundaries. When there are no fills, mark fill VWAP as undefined (NaN/None) to avoid division by zero or misreporting the average as zero.
6. Explain the microstructure difference between temporary and permanent price impact, derive the efficient execution frontier under the discrete-time Almgren–Chriss mean-variance objective, and identify its assumptions and limits.
7. Reconcile changes in cash, holdings, and net asset value (NAV) after actual fills, without charging cash a second time for price impact already embedded in the fill price.

2. Why Quant Needs This

A strategy constructed on paper may show an excellent Sharpe ratio. Once its orders reach a real market, that paper profit can quickly erode. If a trader sends a market order for tens of thousands of shares, the order can sweep through the shallow top levels of the book and incur substantial temporary market impact. If the trader avoids impact by spreading the order over several days, the price may drift upward with the asset, making later shares more expensive and potentially leaving the target position incomplete at the end of the window.

Unfilled quantity affects execution quality relative to the paper target. Suppose a strategy plans to buy an order but the execution algorithm fills only 20%, after which the price rises. The potential gain on the remaining 80%, measured against the arrival-price benchmark, enters IS as opportunity cost. This is a performance-attribution item, not an actual cash-book expense.

Chapters 14–15 established expected asset returns and factor pricing. Chapter 29 built the portfolio backtest ledger and converted weight changes into target share counts. Chapter 34 introduced visible limit-order-book depth, quoted spreads, and queue priority. Chapter 35 quantified spreads, commissions, borrow costs, and impact functions. This chapter handles the implementation stage of the trading pipeline: it takes the target from Chapters 29 and 35, shows how to distribute the total demand over discrete execution windows, and attributes execution quality after trading. The goal of an execution algorithm is not to predict future returns. It is to balance market-impact cost, which tends to increase with trading speed, against execution risk, as prices drift while execution takes longer.

3. Intuition

Consider a simple scenario. At 09:28, you decide to buy 10,000 shares of a stock whose midpoint is USD 100.00. The order reaches the venue at the 09:30 open, when the price has moved to USD 100.20. You must complete the trade between 09:30 and 10:30.

You face two extremes:

- **Extreme 1:** At 09:30, submit all 10,000 shares as one market order. Visible depth on the first five ask levels is limited, so your buy order consumes them immediately and pushes the average fill price to USD 101.50. You incur substantial temporary impact, but eliminate the risk that the stock continues to rise during the next hour.
- **Extreme 2:** Split the 10,000 shares into 60 child orders of about 167 shares per minute. Each child order is only a small fraction of that minute's volume, so the book barely notices you and temporary impact is small. However, if external buying pressure moves the price from USD 100.20 to USD 102.00 during the hour, later child orders become increasingly expensive and the final average fill price may reach USD 101.10. Moreover, if 2,000 shares remain unfilled at 10:30 and must be canceled, you miss all of the gains from a subsequent surge in those shares.

Execution algorithms operate between these extremes:

- **TWAP** divides the parent order evenly across time slots. It suits assets with relatively uniform liquidity or cases without a volume forecast.
- **Forecast VWAP** allocates the parent order according to a volume curve frozen before trading. If the forecast curve is heavier around the open and close, the schedule assigns more shares to those intervals.
- **POV** sets the pace of subsequent requests from the chosen denominator convention and the latest fully observed market volume. Actual fills remain subject to current liquidity and capacity.

Implementation shortfall is the final scorecard. It starts its ledger when you decide to trade and decomposes the result into delay loss between the decision and order submission, slippage

on actual fills, missed gains or losses on unfilled quantity, and explicit commissions paid to the broker.

4. Mathematical Foundations

Let Q be the signed parent-order quantity for an asset in the portfolio (shares; $Q > 0$ for a buy and $Q < 0$ for a sell). Divide the execution window into N discrete time slots indexed by $k \in \{1, 2, \ldots, N\}$, each of length Δt.

Variables and Notation

- t_d: timestamp when the portfolio manager makes the rebalance decision; it maps to the interface field `decision_time` and benchmark price P_{decision} (USD/share).
- t_p: timestamp when the algorithm receives the parent order; it maps to `parent_received_at` and satisfies $t_d \le t_p$.
- t_{arrival}: arrival-benchmark time used for IS; it maps to `arrival_time` and price P_{arrival} (USD/share). This is an attribution benchmark and must not be conflated with child-order send time or venue-arrival time.
- For each child order k, interface fields `child_send_at`, `child_venue_arrival_at`, and `fill_at` map respectively to $t_{\text{send},k}$, $t_{\text{venue},k}$, and $t_{\text{fill},k}$. Fields `observed_available_at` and `forecast_frozen_at` record when volume data became available and when the forecast was frozen. The slot-end market mark becomes available only at `slot_end` and is recorded in `slot_mark_available_at`. Even when there is no fill, first settle the interval using the old mark, then apply the slot-end mark. The terminal mark time is t_{terminal}, mapped to `terminal_mark_time`.
- t_k: start time of execution slot k.
- q_k: signed child-order quantity requested in slot k (shares), with the same sign as Q.
- F_k: signed shares actually matched in slot k, satisfying $|F_k| \le |q_k|$. If nonzero, its direction matches q_k and Q. This chapter considers a single parent order in one direction, so $\sum_k |F_k| = |F|$; this identity must not be generalized to mixed buys and sells.
- $P_{\text{fill},k}$: average fill price in slot k (USD/share). It is undefined when $F_k = 0$.
- F: cumulative signed shares filled over the execution window:

$$F = \sum_{k \in \mathcal{F}} F_k$$

- R: signed shares remaining unfilled and canceled at the end of the execution window (the remainder, in shares), satisfying the conservation identity:

$$Q = F + R$$

- t_{terminal}: execution-window end and terminal-mark timestamp. The corresponding terminal mark is P_{terminal} (USD/share).
- V_k^{forecast}: forecast external-market volume for slot k, frozen before the window begins (shares).

- V_{k-1}^{obs}: actual external-market volume in slot $k-1$, fully completed and observed before child order k is sent (shares).
- V_k^{realized}: actual external-market volume during slot k (shares). This is still in the future at time t_k and must not be used to decide the child order.
- c_{comm}: explicit commission rate per share (USD/share). Total explicit fees are $C_{\text{fees}} = |F|\, c_{\text{comm}}$.

Single-Direction Fills and Empty Sums

Every sum involving fill prices is taken over the nonzero-fill set $\mathcal{F} = \{k : F_k \neq 0\}$, and an empty sum is defined as zero. With no fills, do not read an undefined average fill price or compute $0 \times$ undefined.

From the Ledger to Execution Evaluation: Continue the Same 100-Share Example

Chapter 34's quotes do not determine actual fills; Chapter 35 books only 60 shares and USD 0.30 in fees. After the window closes, Chapter 36 checks IS using $Q = 100, F = 60, R = 40$: $2.00 + 1.80 + 3.20 + 0.30 = 7.30$. The direct method gives $60(10.05 - 10.00) + 40(10.10 - 10.00) + 0.30 = 7.30$. This equals paper NAV minus actual NAV and does not change cash of USD 1,396.70 or the actual holding of 60 shares. If windows overlap, before sending a new child order subtract already-sent but unconfirmed quantity from the unfinished parent-order capacity. $R = 40$ is the terminal remainder; during execution, do not treat all unconfirmed requests as capacity available for resending.

Now consider selling 100 shares, with only 60 filled, decision price 10.00, arrival price 9.98, fill price 9.95, and terminal price 9.90. Then $Q = -100, F = -60, R = -40$, and the four components remain $2.00, 1.80, 3.20, 0.30$. An account initially holding 100 shares and USD 1,000 in cash ends with USD 1,596.70 in cash and 40 shares, for terminal NAV of USD 1,992.70. The ideal account that sold all 100 shares at the decision price has NAV of USD 2,000; the difference is USD 7.30. Direction follows from signed quantities, not from forcing sell-price differences to be absolute values.

Price Process and Market-Impact Assumptions

In this discrete linear-impact teaching specification, the exogenous price increment unaffected by trading follows arithmetic Brownian motion. After adding contemporaneous permanent impact to the midpoint, the price recursion for signed fills under this chapter's buy-positive, sell-negative convention is:

$$S_k = S_{k-1} + \sigma(\Delta t)^{1/2}\xi_k + \gamma F_k$$

where:

- S_k is the constructed midpoint at the end of period k, including prior permanent impact (USD/share); it is not an observable "true fundamental price."
- σ is the price-diffusion coefficient. If time is measured in days, $\sigma\sqrt{1 \text{ day}}$ is the standard deviation of the one-day price increment. Dividing by a locally constant reference price

gives the corresponding daily return volatility in Chapter 35. Its units are USD / (share · time$^{1/2}$).

- $\xi_k \sim \mathcal{N}(0,1)$ are independent standard-normal shocks.
- F_k is signed fill quantity in period k; buys are positive and sells negative. For a sell liquidation, $F_k = -n_k$, so the permanent-impact term is $-\gamma n_k$.
- γ is the permanent market-impact coefficient (USD/share^2); permanent impact enters subsequent midpoint prices.

Within each slot, the fill uses S_{k-1} as its reference, with temporary impact applied to the fill price; noise and permanent price updates occur afterward. Thus the final noise arrives after liquidation. The zero execution variance for $N = 1$ follows from this model's timing. Arithmetic Brownian prices do not guarantee positivity over every path, which must not be confused with the positive-price input domain of an engineering implementation.

The actual fill price includes temporary market impact:

$$\tilde{S}_k = S_{k-1} + \operatorname{sgn}(F_k)\,\eta\left(\frac{|F_k|}{\Delta t}\right)$$

where η is the temporary-impact coefficient (USD · time/share^2). Temporary impact affects only the current fill and does not enter later-slot price updates.

5. Formal Definitions

5.1 Execution Clock and Causal Ordering

In a quantitative trading system, execution, observation, and matching span several clocks. The system must record the signal-freeze time t_{signal}, the decision time t_d when target shares are locked, parent-order receipt time t_{parent}, each child-order send time $t_{\text{send},k}$, venue-arrival time $t_{\text{venue},k}$, fill-report time $t_{\text{fill},k}$, the time POV external volume becomes available to the algorithm $t_{\text{obs},k}$, forecast-freeze time $t_{\text{forecast},k}$, and cancellation and terminal-mark time t_{terminal}. Also record the arrival benchmark time t_{arrival} and price P_{arrival} used by IS. In this teaching interface it is the agreed parent-order arrival benchmark; it does not replace any child-order send or venue-arrival time.

The timestamps must satisfy these causal constraints:

$$t_{\text{signal}} \le t_d \le t_{\text{parent}} \le t_{\text{arrival}} \le t_{\text{send},k} \le t_{\text{venue},k} \le t_{\text{fill},k} \le t_{\text{terminal}}$$

$$t_{\text{obs},k} \le t_{\text{send},k}, \qquad t_{\text{forecast},k} \le t_{\text{send},k}.$$

`arrival_time` is only the IS benchmark anchor; it does not mean that the algorithm received the parent order or that a child order arrived at the venue. Slot windows may overlap, and the engine processes slots by index while enforcing nondecreasing child send times. Before each send, the algorithm counts only fills strictly earlier than that send and reserves sent-but-unconfirmed quantities against the parent-order remainder. A send at the same timestamp as a fill confirmation occurs first, so that simultaneous fill cannot affect sizing for that send. POV observations must satisfy `observed_available_at <= child_send_at`, and the forecast curve must

be frozen before the first child order is sent. The current simulation uses zero transmission delay, so `child_send_at == child_venue_arrival_at`. These are enforceable causal-clock constraints; a later child order need not wait for the previous slot to end.

A monthly rebalance signal represents target holdings derived from monthly financial or low-frequency market data and remains frozen throughout the execution day. The algorithm must not disguise a monthly signal as intraday high-frequency alpha or feed future monthly closing data backward into the intraday execution algorithm.

5.2 Time-Weighted Average Price (TWAP)

TWAP distributes the parent order evenly across N time slots fixed in advance.

The planned child quantity in slot k is:

$$q_k^{\text{TWAP}} = \frac{Q}{N}, \qquad \forall k \in \{1, 2, \ldots, N\}$$

Conditions:

1. Fix the total execution window $[t_0, t_{\text{terminal}}]$ and the number of slots N before trading begins.
2. Do not dynamically add to or reduce parent order Q during the window.

5.3 Forecast-Volume VWAP Schedule

A VWAP schedule aims to match the shape of child orders to the market's expected intraday volume curve.

Let $V_k^{\text{forecast}} > 0$ be the forecast external-market volume in each slot, set before the window begins. The forecast-volume share is:

$$w_k = \frac{V_k^{\text{forecast}}}{\sum_{j=1}^{N} V_j^{\text{forecast}}}, \qquad \text{with } \sum_{k=1}^{N} w_k = 1, \quad w_k \geq 0$$

The planned child quantity in slot k is:

$$q_k^{\text{VWAP}} = Q\, w_k$$

Conditions:

1. The weight vector $\boldsymbol{w} = (w_1, \ldots, w_N)^\top$ must depend only on historical statistics (for example, the median volume in each slot over the previous 20 trading days) and must be frozen completely at t_0.
2. The schedule must not use future realized external volume V_k^{realized} from the current trading day as an input.

The expressions above, like q_k in Section 5.2, define an ex-ante planned curve. When actual volume is insufficient, this chapter's `execute_order` mechanically reallocates unfilled quantity across later slots based on the current parent remainder. This capacity-constrained catch-up rule does not read future volume and is not an adaptive schedule that updates price or total-volume forecasts from intraday information.

5.4 Ex-Post Market VWAP Benchmark

Ex-post market VWAP is a public external benchmark price used to evaluate institutional execution quality after the whole window has ended:

$$\mathrm{VWAP}_{\mathrm{market}} = \frac{\sum_{k=1}^{N} P_k^{\mathrm{mkt}} V_k^{\mathrm{realized}}}{\sum_{k=1}^{N} V_k^{\mathrm{realized}}}$$

Here, P_k^{mkt} and V_k^{realized} are the average price and volume of the external market in slot k, excluding the institution's own fills. If a benchmark including own fills is used instead, its inclusion rule must be stated separately. This benchmark can be computed only ex post; it is not an input to the ex-ante schedule.

5.5 Lagged-Observed Percentage of Volume (POV)

POV, also called a participation-rate algorithm, is reactive. Instead of specifying a complete fixed allocation in advance, it sets target order pace from available liquidity information under a chosen denominator convention. Actual participation remains subject to fills, cancellations, and capacity.

Under a strict causal clock, the algorithm does not know future realized external volume V_k^{realized} in the current slot when it sends child order k at the slot start. Thus the child quantity can be computed only from V_{k-1}^{obs}, the latest fully settled and observed prior-slot volume.

For a participation rate $\rho \in (0, 1)$, two denominator conventions are in use:

The following participation-rate equations first define an uncapped reference request. After the minimum with the remainder is applied, the actual request may fall below that participation rate; actual fills are also constrained by current-slot capacity.

Definition A (exclusive denominator, external-market volume only):

$$\rho = \frac{|q_k|}{V_{k-1}^{\mathrm{obs}}} \implies q_k^{\mathrm{POV}} = \mathrm{sgn}(Q) \cdot \min\left(|Q_{\mathrm{rem},k}|,\ \rho V_{k-1}^{\mathrm{obs}}\right)$$

For a nonoverlapping hand calculation, assume prior fills have all been confirmed before the next send and no orders are in flight; only then can one write $Q_{\mathrm{rem},k} = Q - \sum_{j<k} F_j$. In general, send capacity must subtract both already confirmed fills and same-direction quantities still reserved in flight. Do not infer availability merely from slot indices.

Definition B (inclusive denominator, including own fills):

$$\rho = \frac{|q_k|}{V_{k-1}^{\mathrm{obs}} + |q_k|} \implies |q_k| = \frac{\rho}{1-\rho} V_{k-1}^{\mathrm{obs}}$$

The child quantity is therefore:

$$q_k^{\mathrm{POV}} = \mathrm{sgn}(Q) \cdot \min\left(|Q_{\mathrm{rem},k}|,\ \frac{\rho}{1-\rho} V_{k-1}^{\mathrm{obs}}\right)$$

Boundary and validity conditions:

1. **First-slot boundary:** At the first open-market slot, $k = 1$, no prior continuous-trading volume is yet observable, so $V_0^{\text{obs}} = 0$. Under the contract, POV sends zero in the first slot and waits until that slot's volume has settled before following the market.
2. **Limit test:** Definition B has a finite solution only for $0 \leq \rho < 1$. As $\rho \to 1$ with positive external volume, the uncapped inverse formula diverges; after capping at the finite remaining order, the actual request remains finite. Implementations often also cap requests by remaining quantity, a participation-policy ceiling, and market capacity. A ceiling such as 0.5 is an implementation choice, not a universal threshold implied by the identity.

5.6 Implementation Shortfall (IS)

Perold (1988) is retained here as further reading; the historical check covered author and publisher metadata, without reading the full text. The SEC (2003) discusses cost measurement through the difference between paper and actual portfolios and traces an early source of the idea to Treynor (1981). That request for comments supplies conceptual background, rather than prescribing this chapter's formula. The decomposition below is an explicit teaching convention, not a claim of a single industry-standard method.

For signed parent order Q, let the actual fill sequence be $\{(F_k, P_{\text{fill},k})\}_{k=1}^{N}$, cumulative filled quantity be $F = \sum_k F_k$, and unfilled remainder be $R = Q - F$. Define total implementation shortfall as:

$$\text{IS} = \text{IS}_{\text{delay}} + \text{IS}_{\text{exec}} + \text{IS}_{\text{opp}} + C_{\text{fees}}$$

Its components are:

1. **Delay cost (slippage from decision):**

$$\text{IS}_{\text{delay}} = Q(P_{\text{arrival}} - P_{\text{decision}})$$

This measures losses from adverse price movement between the portfolio manager's decision and the order's arrival at the market.

2. **Execution slippage cost:**

$$\text{IS}_{\text{exec}} = \sum_{k \in \mathcal{F}} F_k(P_{\text{fill},k} - P_{\text{arrival}})$$

When $F \neq 0$, if fill VWAP is defined as $P_{\text{fillVWAP}} = \frac{\sum_{k \in \mathcal{F}} F_k P_{\text{fill},k}}{F}$, then execution cost can also be written as:

$$\text{IS}_{\text{exec}} = F(P_{\text{fillVWAP}} - P_{\text{arrival}})$$

This measures slippage of actual fills relative to the arrival benchmark.

3. **Unfilled opportunity cost:**

$$\text{IS}_{\text{opp}} = R(P_{\text{terminal}} - P_{\text{arrival}})$$

This measures potential mark-to-market gains or losses missed because the remainder R was not traded by the deadline.

4. **Explicit fees:**

$$C_{\text{fees}} = \sum_{k \in \mathcal{F}} |F_k| c_{\text{comm}} = |F| c_{\text{comm}}$$

These are explicit commissions and taxes paid to the broker or venue.

By convention, IS > 0 means positive execution cost: actual account NAV is below the frictionless paper portfolio's NAV.

6. Derivations

6.1 Algebraic Proof of the Two Equivalent Representations of IS

In practice, implementation shortfall can be computed in two ways:

1. A four-component decomposition aligned with the workflow: delay, execution, opportunity, and explicit fees.
2. The direct amount method, which directly compares the terminal value of the theoretical paper portfolio with the actual execution cash ledger.

Theorem. The four-component decomposition is algebraically identical to the direct amount method.

Proof. Under the direct amount method, the frictionless paper portfolio is assumed to buy all Q shares at P_{decision} at the decision time t_d. At the terminal time t_{terminal}, the paper position's market value is QP_{terminal}. It initially spent QP_{decision}, so its theoretical book profit is:

$$\Pi_{\text{paper}} = Q(P_{\text{terminal}} - P_{\text{decision}})$$

In actual execution, we buy F_k shares in each slot at $P_{\text{fill},k}$ and pay fees C_{fees}. At the terminal time, the F shares held are marked at P_{terminal}. The actual economic profit is:

$$\Pi_{\text{real}} = FP_{\text{terminal}} - \sum_{k \in \mathcal{F}} F_k P_{\text{fill},k} - C_{\text{fees}}$$

Define implementation shortfall as the difference between paper and actual profit:

$$\text{IS} = \Pi_{\text{paper}} - \Pi_{\text{real}}$$

Substituting the profit expressions gives:

$$\text{IS} = Q(P_{\text{terminal}} - P_{\text{decision}}) - \left[FP_{\text{terminal}} - \sum_{k \in \mathcal{F}} F_k P_{\text{fill},k} - C_{\text{fees}} \right]$$

Using quantity conservation, $Q = F + R$, expand QP_{terminal} as $(F + R)P_{\text{terminal}}$:

$$\text{IS} = (F + R)P_{\text{terminal}} - QP_{\text{decision}} - FP_{\text{terminal}} + \sum_{k \in \mathcal{F}} F_k P_{\text{fill},k} + C_{\text{fees}}$$

Cancel the FP_{terminal} terms:

$$\text{IS} = \sum_{k \in \mathcal{F}} F_k P_{\text{fill},k} + RP_{\text{terminal}} - QP_{\text{decision}} + C_{\text{fees}}$$

Since $\sum_{k\in\mathcal{F}} F_k = F$ and $Q = F + R$, decompose QP_{decision} as $FP_{\text{decision}} + RP_{\text{decision}} = \sum_{k\in\mathcal{F}} F_k P_{\text{decision}} + RP_{\text{decision}}$:

$$\text{IS} = \sum_{k\in\mathcal{F}} F_k(P_{\text{fill},k} - P_{\text{decision}}) + R(P_{\text{terminal}} - P_{\text{decision}}) + C_{\text{fees}}$$

This is the classic direct amount formula.

To prove its identity with the four-component decomposition, insert the arrival price P_{arrival} into each price difference above. For filled quantity:

$$P_{\text{fill},k} - P_{\text{decision}} = (P_{\text{fill},k} - P_{\text{arrival}}) + (P_{\text{arrival}} - P_{\text{decision}})$$

For unfilled quantity:

$$P_{\text{terminal}} - P_{\text{decision}} = (P_{\text{terminal}} - P_{\text{arrival}}) + (P_{\text{arrival}} - P_{\text{decision}})$$

Substituting both expansions into the direct amount formula:

$$\begin{aligned}
\text{IS} &= \sum_{k\in\mathcal{F}} F_k \left[(P_{\text{fill},k} - P_{\text{arrival}}) + (P_{\text{arrival}} - P_{\text{decision}})\right] \\
&\quad + R\left[(P_{\text{terminal}} - P_{\text{arrival}}) + (P_{\text{arrival}} - P_{\text{decision}})\right] + C_{\text{fees}} \\
&= \sum_{k\in\mathcal{F}} F_k(P_{\text{fill},k} - P_{\text{arrival}}) + \left(\sum_{k\in\mathcal{F}} F_k\right)(P_{\text{arrival}} - P_{\text{decision}}) \\
&\quad + R(P_{\text{terminal}} - P_{\text{arrival}}) + R(P_{\text{arrival}} - P_{\text{decision}}) + C_{\text{fees}}
\end{aligned}$$

Collect the common factor $(P_{\text{arrival}} - P_{\text{decision}})$:

$$\left(\sum_{k\in\mathcal{F}} F_k + R\right)(P_{\text{arrival}} - P_{\text{decision}}) = (F + R)(P_{\text{arrival}} - P_{\text{decision}}) = Q(P_{\text{arrival}} - P_{\text{decision}})$$

Rearranging the terms:

$$\begin{aligned}
\text{IS} = &\underbrace{Q(P_{\text{arrival}} - P_{\text{decision}})}_{\text{IS}_{\text{delay}}} \\
&+ \underbrace{\sum_{k\in\mathcal{F}} F_k(P_{\text{fill},k} - P_{\text{arrival}})}_{\text{IS}_{\text{exec}}} \\
&+ \underbrace{R(P_{\text{terminal}} - P_{\text{arrival}})}_{\text{IS}_{\text{opp}}} \\
&+ \underbrace{C_{\text{fees}}}_{\text{Explicit Fees}}
\end{aligned}$$

This proves the result.

The proof shows the sign symmetry: the signed formulas are self-consistent for either a buy ($Q > 0$) or a sell ($Q < 0$). If a buy faces rising prices, the price difference is positive and IS rises. If a sell faces falling prices, $P_{\text{fill}} - P_{\text{arrival}} < 0$ while $F_k < 0$, so the product is positive and IS likewise records the loss from a lower sale price.

Other papers or trading systems may allocate the decision-to-arrival price move across filled quantity or use a different opportunity-cost benchmark. This chapter assigns the full Q-share decision-to-arrival move to delay cost and measures fills and remainder from arrival price. The components can be allocated differently, but as long as each item is settled consistently against the same decision and terminal prices, the direct-amount total is fixed by the identity.

6.2 Deriving the Discrete Almgren–Chriss Execution Cost–Risk Frontier

Consider liquidating an initial position of $X > 0$ shares (a sell order) over total time $T = N\Delta t$. Let x_k be the remaining inventory at timestamp t_k, the end of time slot k, with boundary conditions:

$$x_0 = X, \qquad x_N = 0$$

The number of shares sold in slot k is $n_k = x_{k-1} - x_k \geq 0$. This section fixes a deterministic static schedule at the outset; n_k and x_k are nonrandom. The expectation and variance derivations below do not directly apply to inventory that adapts to noise.

From the price process in Section 4:

$$S_k = S_{k-1} + \sigma(\Delta t)^{1/2}\xi_k - \gamma n_k$$

The fill price includes temporary impact:

$$\tilde{S}_k = S_{k-1} - \frac{\eta}{\Delta t}n_k$$

Total liquidation proceeds are $W = \sum_{k=1}^{N} n_k\tilde{S}_k$. Relative to the initial arrival value XS_0, implementation shortfall, excluding explicit commissions, is the random variable:

$$x = XS_0 - W = XS_0 - \sum_{k=1}^{N} n_k\tilde{S}_k$$

Step 1: Derive expected shortfall $E[x]$. Expand the fundamental price recursion:

$$S_{k-1} = S_0 + \sigma(\Delta t)^{1/2}\sum_{j=1}^{k-1}\xi_j - \gamma\sum_{j=1}^{k-1}n_j$$

Substitute into the fill-price expression:

$$\tilde{S}_k = S_0 + \sigma(\Delta t)^{1/2}\sum_{j=1}^{k-1}\xi_j - \gamma\sum_{j=1}^{k-1}n_j - \frac{\eta}{\Delta t}n_k$$

Total proceeds are:

$$\sum_{k=1}^{N} n_k\tilde{S}_k = S_0\sum_{k=1}^{N}n_k + \sigma(\Delta t)^{1/2}\sum_{k=1}^{N}n_k\sum_{j=1}^{k-1}\xi_j - \gamma\sum_{k=1}^{N}n_k\sum_{j=1}^{k-1}n_j - \frac{\eta}{\Delta t}\sum_{k=1}^{N}n_k^2$$

Since $\sum_{k=1}^{N} n_k = X$ and $E[\xi_j] = 0$, expected proceeds are:

$$E\left[\sum_{k=1}^{N} n_k\tilde{S}_k\right] = XS_0 - \gamma\sum_{k=1}^{N}n_k\sum_{j=1}^{k-1}n_j - \frac{\eta}{\Delta t}\sum_{k=1}^{N}n_k^2$$

Use the discrete-sum identity:

$$\sum_{k=1}^{N} n_k \sum_{j=1}^{k-1} n_j = \frac{1}{2}\left[\left(\sum_{k=1}^{N} n_k\right)^2 - \sum_{k=1}^{N} n_k^2\right] = \frac{1}{2}X^2 - \frac{1}{2}\sum_{k=1}^{N} n_k^2$$

Substituting this into expected shortfall gives:

$$E[x] = XS_0 - E\left[\sum_{k=1}^{N} n_k \tilde{S}_k\right] = \frac{1}{2}\gamma X^2 + \left(\frac{\eta}{\Delta t} - \frac{1}{2}\gamma\right)\sum_{k=1}^{N} n_k^2$$

This section assumes $\eta/\Delta t > \gamma/2$. This is a model condition for a strictly convex objective and the unique solution below, not a universal market fact. Define the adjusted temporary-impact parameter $\tilde{\eta} = \frac{\eta}{\Delta t} - \frac{1}{2}\gamma > 0$. Expected execution cost is then:

$$E[x] = \frac{1}{2}\gamma X^2 + \tilde{\eta}\sum_{k=1}^{N} n_k^2$$

The discrete identity regroups total cost as a constant and a path-dependent term:

$$\sum_{k=1}^{N} n_k \sum_{j<k} n_j = \frac{1}{2}\left(X^2 - \sum_{k=1}^{N} n_k^2\right).$$

The term $\gamma X^2/2$ is the path-invariant constant after this regrouping; it is not the original discrete permanent-impact loss. The original permanent-impact term

$$\gamma \sum_{k} n_k \sum_{j<k} n_j$$

still depends on the schedule. Its $-\gamma \sum_k n_k^2/2$ component combines with temporary impact to form $\tilde{\eta} = \eta/(\Delta t) - \gamma/2$.

Numerical check: let $\gamma = 0.01$, $\eta/\Delta t = 0.02$, and $X = 1{,}000$. For schedule $[1{,}000, 0]$, the original permanent term is 0; for $[500, 500]$, it is $0.01 \times 500 \times 500 = 2{,}500$. The regrouped constant $\gamma X^2/2 = 5{,}000$ is the same for both paths; the path difference appears through the adjusted temporary-impact squared term.

Here x_t denotes remaining inventory decreasing from X to 0. For the sell schedule, with cumulative sales $y_t = X - x_t$, the permanent-impact cost rate is $\gamma y_t \dot{y}_t$, whose total is

$$\int_0^T \gamma y_t \dot{y}_t \, dt = \gamma X^2/2.$$

Using fixed endpoints $x_0 = X, x_T = 0$, the same total integral can also be written

$$-\int_0^T \gamma x_t \dot{x}_t \, dt = \gamma X^2/2.$$

This is an integral identity after evaluating the boundary terms; the integrands are not equal at each instant. In Challenge 36-C01, x_t instead denotes cumulative buy progress from 0 to X, so the buy-cost rate $\gamma x_t \dot{x}_t$ integrates with a positive sign.

Step 2: Derive shortfall variance $V[x]$. All uncertainty in shortfall comes from the Brownian-motion term

$$\sigma(\Delta t)^{1/2}\sum_{k=1}^{N} n_k \sum_{j=1}^{k-1}\xi_j.$$

Reverse the order of summation:

$$\sum_{k=1}^{N} n_k \sum_{j=1}^{k-1}\xi_j = \sum_{j=1}^{N-1}\xi_j \sum_{k=j+1}^{N} n_k = \sum_{j=1}^{N-1}\xi_j x_j$$

because the ξ_j are independent with unit variance. Thus:

$$V[x] = \mathrm{Var}\left(\sigma(\Delta t)^{1/2}\sum_{j=1}^{N-1} x_j\xi_j\right) = \sigma^2\Delta t\sum_{k=1}^{N} x_k^2$$

Execution variance is therefore the accumulated squared remaining inventory over time: a larger position held for longer carries more price risk.

Step 3: Solve the optimization problem. Expected shortfall is measured in USD, while variance is measured in USD^2. The risk-aversion parameter λ_{risk} must therefore have units USD^{-1} so the terms can be added. It is distinct from the linear impact slope λ_{impact} (USD/share^2) in Chapter 35. The time-slot duration Δt has units of time; it is not dimensionless turnover τ_{turnover}. For trader risk aversion $\lambda_{\text{risk}} \geq 0$, form the mean-variance objective:

$$\min_{\{x_k\}_{k=1}^{N-1}} U(x) = E[x] + \lambda_{\text{risk}}V[x] = \frac{1}{2}\gamma X^2 + \tilde{\eta}\sum_{k=1}^{N}(x_{k-1}-x_k)^2 + \lambda_{\text{risk}}\sigma^2\Delta t\sum_{k=1}^{N-1} x_k^2$$

For any intermediate state x_k $(1 \leq k \leq N-1)$, the first-order condition is:

$$\frac{\partial U}{\partial x_k} = 2\tilde{\eta}(x_k - x_{k-1}) - 2\tilde{\eta}(x_{k+1}-x_k) + 2\lambda_{\text{risk}}\sigma^2\Delta t\, x_k = 0$$

Divide by $2\tilde{\eta}$ and rearrange into a second-order linear difference equation with constant coefficients:

$$x_{k+1} - \left(2 + \frac{\lambda_{\text{risk}}\sigma^2\Delta t}{\tilde{\eta}}\right)x_k + x_{k-1} = 0$$

The characteristic equation is $r^2 - 2\cosh(\kappa\Delta t)r + 1 = 0$, where κ satisfies

$$2\cosh(\kappa\Delta t) = 2 + \frac{\lambda_{\text{risk}}\sigma^2\Delta t}{\tilde{\eta}}.$$

Here the inventory nodes use $t_k = k\Delta t$, elapsed time from the start, unlike the slot-start times in Section 4. The units of κ are time^{-1}, making $\kappa\Delta t$ dimensionless. With boundary conditions $x_0 = X$ and $x_N = 0$, the unique analytic solution of the difference equation is:

$$x_k = \frac{\sinh(\kappa(T-t_k))}{\sinh(\kappa T)}X$$

Special cases. When $\lambda_{\text{risk}} = 0$ or $\sigma = 0$, $\kappa = 0$ and the continuous limit applies. By L'Hôpital's rule:

$$x_k = \lim_{\kappa \to 0} \frac{\kappa(T - t_k)}{\kappa T} X = \left(1 - \frac{k}{N}\right) X$$

Then $n_k = x_{k-1} - x_k = X/N$, so the optimal liquidation path degenerates to strictly linear liquidation, or TWAP.

When $\lambda_{\text{risk}} > 0$ and $\sigma > 0$, before maturity the hyperbolic-sine curve gives a decreasing, convex remaining-inventory path ($x''(t) = \kappa^2 x(t) > 0$). The trader sells faster early (front-loads execution), accepting higher temporary impact in exchange for quickly reducing exposure and lowering variance.

7. Financial Interpretation

7.1 Agency and the Choice of Benchmark

The benchmark chosen for an execution algorithm directly affects the trader's incentives:

1. **Arrival Price:** When an order is meant to stay close to the price at the agreed post-decision arrival point, arrival price can evaluate delay and subsequent execution. Waiting is risky if the signal is expected to decay or price drift is adverse; the direction and speed of drift must be estimated from data and do not follow from the benchmark definition itself.
2. **Market VWAP:** When the goal is to compare fills with the market's average traded price over a defined window, use this benchmark. The schedule still needs an ex-ante forecast-volume curve; market VWAP is computed from eligible market trades after the window ends.

7.2 The Economic Meaning of Unfilled Opportunity Cost

If evaluation counts slippage only on filled shares and sets the opportunity term on canceled remainder, $R(P_{\text{terminal}} - P_{\text{arrival}})$, to zero, the direction of the bias depends on its sign: a positive term means IS is understated, a negative term means it is overstated, and a zero term leaves IS unchanged. Opportunity cost is performance attribution relative to a benchmark, not a cash expense on unfilled shares.

For example, consider a constructed order to buy 50,000 shares. If only 5,000 shares fill before the deadline and the remaining 45,000 are canceled, a terminal price above arrival price means that fill slippage alone does not capture the result of the full decision. Under this chapter's IS convention, $R(P_{\text{terminal}} - P_{\text{arrival}})$ records the opportunity cost relative to arrival price. It is not an actual cash-book expense.

For a counterexample, take $Q = R = 100$, $F = 0$, $P_{\text{decision}} = P_{\text{arrival}} = 10$, $P_{\text{terminal}} = 9$, and zero fees. Correct IS is USD -100; omitting the opportunity term gives 0 and overstates IS by USD 100.

7.3 Microstructure of Temporary and Permanent Impact

- In this model, temporary impact affects only the current fill price and does not carry into the next slot's midpoint. This is a modeling assumption. In real markets, liquidity replenishment and price decay vary, and event data are needed to estimate them; there is no guarantee that the price recovers within tens of seconds.
- Permanent impact is the lasting effect of trading on subsequent prices in the price-update model. Information content, liquidity-provider inventory, and other mechanisms may all contribute; the decomposition alone does not identify a causal source. Huberman and Stanzl (2004), under a framework with uncertain prices, price impact from trading volume, and time-independent permanent price updates, prove that excluding quasi-arbitrage requires linear permanent price impact. Their abstract notes that when temporary impact is also present, only the permanent component must be linear. This theorem is not an empirical law for every market-impact path.

7.4 Model Assumptions and Practical Limits

The Almgren–Chriss model provides a clear closed-form solution, but has important limits in live deployment:

1. It assumes the fundamental price is a driftless random walk. Quantitative rebalancing often involves decaying alpha signals, and an asset may have a deterministic drift component.
2. It assumes market liquidity and volatility are constant throughout the day. Many equity samples show higher trading volume near the open and close, but the shape, spread, and depth vary by market, asset, time, and event; schedules must be re-estimated on suitable data.
3. The static schedule in this chapter does not adjust its speed in response to intraday price shocks. If the objective permits dynamic feedback and the inputs support real-time estimation, dynamic programming or a Hamilton–Jacobi–Bellman (HJB) method can construct another class of execution strategy. That requires additional modeling and validation.

Chapter 36 complete numerical example: all steps and conclusions

8. Numerical Example

This section gives one unified, constructed teaching example. Every input is labeled as constructed for instruction. The purpose is to check causal timestamps, child-order schedules, matching, IS decomposition, and cash/position reconciliation.

8.1 Setup and Inputs

Consider a signed buy parent order:

- Parent-order size: $Q = +10{,}000$ shares (buy).
- Decision time: $t_d =$ 09:28:00; decision price $P_{\text{decision}} =$ \$100.00/share.
- Arrival time: $t_0 =$ 09:30:00; arrival price $P_{\text{arrival}} =$ \$100.20/share.
- Execution window: four discrete 15-minute slots, ending at $t_{\text{terminal}} =$ 10:30:00.
- Terminal mark: $P_{\text{terminal}} =$ \$101.00/share.

Continued: the same complete example

- Commission rate: $c_{\text{comm}} = \$0.005$/share.
- Participation cap: actual fills in each slot may not exceed 20% of realized external volume in that slot ($\rho_{\text{cap}} = 0.20$).

The constructed UTC window below is not a real market open. Each slot's price serves as both an external average-price proxy and midpoint proxy, and the example explicitly sets our fill equal to it (zero half-spread and zero impact of both kinds). In general execution, they differ.

The external market stream and forecast data are constructed for instruction:

Slot k	Time interval	Forecast external volume V_k^{forecast}	Prior observed external volume V_{k-1}^{obs}	Realized external volume V_k^{realized}	Fill price $P_{\text{fill},k}$
1	09:30–09:45	40,000	0 (no intraday volume before open)	35,000	100.30
2	09:45–10:00	30,000	35,000	25,000	100.50
3	10:00–10:15	20,000	25,000	20,000	100.70
4	10:15–10:30	10,000	20,000	20,000	100.90
Total		100,000	—	100,000	—

The ex-post market VWAP across external market volume is:

$$\begin{aligned}\text{VWAP}_{\text{market}} &= \frac{35{,}000 \times 100.30 + 25{,}000 \times 100.50 + 20{,}000 \times 100.70 + 20{,}000 \times 100.90}{100{,}000} \\ &= \frac{10{,}055{,}000}{100{,}000} \\ &= 100.5500 \text{ USD/share}.\end{aligned}$$

8.2 Algorithm 1: Time-Weighted Average Price (TWAP)

1. **Schedule.** There are $N = 4$ slots. The planned child quantity in each is $q_k^{\text{TWAP}} = 10{,}000/4 = 2{,}500$ shares.
2. **Fills and capacity.**
   - Slot 1: cap $20\% \times 35{,}000 = 7{,}000 \geq 2{,}500$, so $F_1 = 2{,}500$.
   - Slot 2: cap $20\% \times 25{,}000 = 5{,}000 \geq 2{,}500$, so $F_2 = 2{,}500$.
   - Slot 3: cap $20\% \times 20{,}000 = 4{,}000 \geq 2{,}500$, so $F_3 = 2{,}500$.
   - Slot 4: cap $20\% \times 20{,}000 = 4{,}000 \geq 2{,}500$, so $F_4 = 2{,}500$. Cumulative fill is $F = 10{,}000$ shares and remainder is $R = 10{,}000 - 10{,}000 = 0$ shares.
3. **Fill VWAP.**

$$P_{\text{fillVWAP}} = \frac{2{,}500(100.30 + 100.50 + 100.70 + 100.90)}{10{,}000} = \frac{1{,}006{,}000}{10{,}000} = 100.6000 \text{ USD/share}$$

4. **IS decomposition.**
   - Delay cost: $\text{IS}_{\text{delay}} = 10{,}000(100.20 - 100.00) = \$2{,}000.00$.
   - Execution cost: $\text{IS}_{\text{exec}} = 10{,}000(100.6000 - 100.20) = \$4{,}000.00$.
   - Opportunity cost: $\text{IS}_{\text{opp}} = 0(101.00 - 100.20) = \$0.00$.
   - Explicit commission: $C_{\text{fees}} = 10{,}000(0.005) = \$50.00$.
   - Total IS: $\text{IS}_{\text{total}} = 2{,}000.00 + 4{,}000.00 + 0.00 + 50.00 = \$6{,}050.00$.

Continued: the same complete example

5. **Direct-amount check.**

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= \sum_{k=1}^{4} F_k(P_{\mathrm{fill},k} - P_{\mathrm{decision}}) + R(P_{\mathrm{terminal}} - P_{\mathrm{decision}}) + C_{\mathrm{fees}} \\ &= 2{,}500(0.30 + 0.50 + 0.70 + 0.90) + 0 + 50.00 \\ &= 2{,}500(2.40) + 50.00 = 6{,}000.00 + 50.00 = \$6{,}050.00\end{aligned}$$

The two calculations agree exactly.

8.3 Algorithm 2: Ex-Ante Forecast-Volume VWAP

1. **Schedule.** Total forecast volume is $\sum V_k^{\mathrm{forecast}} = 100{,}000$ shares. Forecast weights are $w_1 = 0.40, w_2 = 0.30, w_3 = 0.20, w_4 = 0.10$. Planned child quantities are:
   - $q_1 = 10{,}000(0.40) = 4{,}000$ shares.
   - $q_2 = 10{,}000(0.30) = 3{,}000$ shares.
   - $q_3 = 10{,}000(0.20) = 2{,}000$ shares.
   - $q_4 = 10{,}000(0.10) = 1{,}000$ shares.
2. **Fills and capacity.**
   - Slot 1: cap $7{,}000 \geq 4{,}000$, so $F_1 = 4{,}000$.
   - Slot 2: cap $5{,}000 \geq 3{,}000$, so $F_2 = 3{,}000$.
   - Slot 3: cap $4{,}000 \geq 2{,}000$, so $F_3 = 2{,}000$.
   - Slot 4: cap $4{,}000 \geq 1{,}000$, so $F_4 = 1{,}000$. Cumulative fill is $F = 10{,}000$ shares and remainder $R = 0$ shares.
3. **Fill VWAP.**

$$\begin{aligned}P_{\mathrm{fillVWAP}} &= \frac{4{,}000(100.30) + 3{,}000(100.50) + 2{,}000(100.70) + 1{,}000(100.90)}{10{,}000} \\ &= \frac{1{,}005{,}000}{10{,}000} \\ &= 100.5000 \text{ USD/share.}\end{aligned}$$

   The forecast-VWAP fills average 100.5000, better than the full external-market VWAP of 100.5500, because a larger share of the order was completed in the lower-priced early slots.
4. **IS decomposition.**
   - Delay cost: $\mathrm{IS}_{\mathrm{delay}} = 10{,}000(100.20 - 100.00) = \$2{,}000.00$.
   - Execution cost: $\mathrm{IS}_{\mathrm{exec}} = 10{,}000(100.5000 - 100.20) = \$3{,}000.00$.
   - Opportunity cost: $\mathrm{IS}_{\mathrm{opp}} = \$0.00$.
   - Explicit commission: $C_{\mathrm{fees}} = \$50.00$.
   - Total IS: $\mathrm{IS}_{\mathrm{total}} = 2{,}000.00 + 3{,}000.00 + 0.00 + 50.00 = \$5{,}050.00$.
5. **Direct-amount check.**

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= 4{,}000(0.30) + 3{,}000(0.50) + 2{,}000(0.70) + 1{,}000(0.90) + 50.00 \\ &= 1{,}200 + 1{,}500 + 1{,}400 + 900 + 50 \\ &= \$5{,}050.00.\end{aligned}$$

The results agree.

Continued: the same complete example

8.4 Algorithm 3: Lagged-Observed POV with Partial Fills

Set the target participation rate to $\rho = 0.10$ under Definition A (exclusive denominator).

1. **Causal schedule and fills.**
   - **Slot 1 (09:30):** No intraday observation is yet available, so $V_0^{\text{obs}} = 0$ and $q_1 = 0$ shares. Fill $F_1 = 0$; the remaining demand for slot 2 is $Q_{\text{rem},2} = 10{,}000$ shares.
   - **Slot 2 (09:45):** The complete external volume from slot 1 is now observed: $V_1^{\text{obs}} = 35{,}000$. Request $q_2 = \min(10{,}000, 0.10 \times 35{,}000) = 3{,}500$ shares. The current capacity cap is $20\% \times 25{,}000 = 5{,}000 \geq 3{,}500$, so $F_2 = 3{,}500$. Remaining demand for slot 3 is $Q_{\text{rem},3} = 10{,}000 - 3{,}500 = 6{,}500$ shares.
   - **Slot 3 (10:00):** Slot 2's external volume is observed: $V_2^{\text{obs}} = 25{,}000$. Request $q_3 = \min(6{,}500, 0.10 \times 25{,}000) = 2{,}500$ shares. Capacity is $20\% \times 20{,}000 = 4{,}000 \geq 2{,}500$, so $F_3 = 2{,}500$. Remaining demand for slot 4 is $Q_{\text{rem},4} = 6{,}500 - 2{,}500 = 4{,}000$ shares.
   - **Slot 4 (10:15):** Slot 3's external volume is observed: $V_3^{\text{obs}} = 20{,}000$. Request $q_4 = \min(4{,}000, 0.10 \times 20{,}000) = 2{,}000$ shares. Capacity is $20\% \times 20{,}000 = 4{,}000 \geq 2{,}000$, so $F_4 = 2{,}000$. At the window close, the unfilled remainder is $R = 4{,}000 - 2{,}000 = 2{,}000$ shares.
2. **Filled and unfilled totals.** Cumulative fill is $F = 0 + 3{,}500 + 2{,}500 + 2{,}000 = 8{,}000$ shares; canceled remainder is $R = 2{,}000$. Conservation check: $F + R = 8{,}000 + 2{,}000 = 10{,}000 = Q$.
3. **Fill VWAP.**

$$\begin{aligned} P_{\text{fillVWAP}} &= \frac{3{,}500(100.50) + 2{,}500(100.70) + 2{,}000(100.90)}{8{,}000} \\ &= \frac{351{,}750 + 251{,}750 + 201{,}800}{8{,}000} \\ &= \frac{805{,}300}{8{,}000} = 100.6625 \text{ USD/share.} \end{aligned}$$

4. **IS decomposition.**
   - Delay cost: $\text{IS}_{\text{delay}} = 10{,}000(100.20 - 100.00) = \$2{,}000.00$.
   - Execution cost: $\text{IS}_{\text{exec}} = 8{,}000(100.6625 - 100.20) = \$3{,}700.00$. Equivalently, $3{,}500(0.30) + 2{,}500(0.50) + 2{,}000(0.70) = 1{,}050 + 1{,}250 + 1{,}400 = \$3{,}700.00$.
   - Opportunity cost: $\text{IS}_{\text{opp}} = 2{,}000(101.00 - 100.20) = 2{,}000(0.80) = \$1{,}600.00$.
   - Explicit commission (charged only on actual fills): $C_{\text{fees}} = 8{,}000(0.005) = \$40.00$.
   - Total IS: $\text{IS}_{\text{total}} = 2{,}000.00 + 3{,}700.00 + 1{,}600.00 + 40.00 = \$7{,}340.00$.
5. **Direct-amount check.**

$$\begin{aligned} \text{IS}_{\text{direct}} &= \sum_{k=1}^{4} F_k(P_{\text{fill},k} - P_{\text{decision}}) + R(P_{\text{terminal}} - P_{\text{decision}}) + C_{\text{fees}} \\ &= [3{,}500(0.50) + 2{,}500(0.70) + 2{,}000(0.90)] + 2{,}000(1.00) + 40.00 \\ &= (1{,}750 + 1{,}750 + 1{,}800) + 2{,}000 + 40.00 \\ &= 5{,}300 + 2{,}000 + 40 = \$7{,}340.00 \end{aligned}$$

   Both formulas agree.

Continued: the same complete example

8.5 Boundary Case: Undefined VWAP with No Fills

Consider a separate constructed boundary with zero capacity and no fills. Keep decision price 100, arrival price 100.20, terminal price 101, and set $F = 0, R = 10{,}000$. This boundary is specified manually; it does not claim that `execute_order` simulates a limit book or a trading halt.

1. **Fill VWAP status:**

$$P_{\text{fillVWAP}} = \frac{0}{0} \implies \text{undefined (NaN/None)}$$

   Preserve the undefined state; never record it as 0.00 in the ledger.

2. **IS decomposition:**
   - Delay cost: $\text{IS}_{\text{delay}} = 10{,}000(100.20 - 100.00) = \$2{,}000.00$.
   - Execution cost: $\text{IS}_{\text{exec}} = \$0.00$; there are no fills, so the slippage sum is empty and equals zero.
   - Opportunity cost: $\text{IS}_{\text{opp}} = 10{,}000(101.00 - 100.20) = \$8{,}000.00$.
   - Explicit fees: $C_{\text{fees}} = 0(0.005) = \$0.00$.
   - Total IS: $\text{IS}_{\text{total}} = 2{,}000.00 + 0.00 + 8{,}000.00 + 0.00 = \$10{,}000.00$.

3. **Direct-amount check:**

$$\text{IS}_{\text{direct}} = R(P_{\text{terminal}} - P_{\text{decision}}) + 0 = 10{,}000(101.00 - 100.00) = \$10{,}000.00$$

   Even at the no-fill boundary, opportunity cost captures the unrealized move from arrival to terminal price. Together with delay cost, it captures the full decision-to-terminal difference for the paper order, so the formulas remain consistent.

8.6 Symmetry Check: Signed Sell Parent Order

Now let the parent order be a sale of $Q = -10{,}000$ shares, with decision price $P_{\text{decision}} = 100.00$, arrival price $P_{\text{arrival}} = 99.80$ (a price decline), slot fill prices $99.70, 99.50, 99.30, 99.10$, and terminal price $P_{\text{terminal}} = 99.00$. Under TWAP, let each fill be $F_k = -2{,}500$ shares and $R = 0$:

- Delay cost: $(-10{,}000)(99.80 - 100.00) = (-10{,}000)(-0.20) = +\$2{,}000.00$.
- Fill VWAP: $P_{\text{fillVWAP}} = 99.4000$.
- Execution cost: $(-10{,}000)(99.4000 - 99.80) = (-10{,}000)(-0.40) = +\$4{,}000.00$.
- Opportunity cost: $\$0.00$.
- Explicit commission: $|-10{,}000| \times 0.005 = \$50.00$.
- Total IS: $+\$6{,}050.00$.

Direct amount:

$$\begin{aligned}\text{IS}_{\text{direct}} &= (-2{,}500)[(99.70 - 100) + (99.50 - 100) + (99.30 - 100) + (99.10 - 100)] + 50.00 \\ &= (-2{,}500)(-2.40) + 50.00 \\ &= +\$6{,}050.00.\end{aligned}$$

This verifies the signed formula's symmetry for buys and sells.

9. Python Implementation

The default I3 reader package uses a self-contained four-cell notebook with a small constructed IS example. It writes JSON and one figure into a unique RUN_DIR; the outer runner run_notebook.py at the package root preserves an execution copy and execution JSON. This is not a complete CLI, a Project 4 historical walk-forward, or a market experiment. The English notebook-language layer and full-package cold-start validation are deferred to BIL23.

Engineering execution uses the dimensionless-coefficient model described in Chapter 35, Section 5.4. The discrete Almgren–Chriss derivation in Section 6.2 is an analytic teaching model under its stated assumptions; it must not be presented as if the engine had been calibrated with the same diffusion or risk parameters. For the current Project 4 result identity, see Chapter 35, Section 10.3 and the current acceptance entrypoint; old R2R6 material is historical constructed evidence only. This chapter and the experiment share the quant_course.execution_costs module; they do not create a second fee, scheduling, or IS engine. ExecutionSlot keeps ex-ante forecast volume, pre-send observed volume, and current realized volume separate, and records availability in observation_time. execute_order returns fills, terminal remainder, a signed IS decomposition, and a direct-amount check.

The code below reproduces the constructed forecast-VWAP example in Section 8. To match the hand-calculation table, spread and both temporary and permanent impact are set to zero, and only per-share commission is booked; market volume still imposes the participation-capacity constraint.

```
import datetime as dt
from quant_course.execution_costs import (
    ExecutionConfig, ExecutionPortfolioLedger, ExecutionSlot,
    ParentOrder, execute_order,
)

UTC = dt.timezone.utc
start = dt.datetime(2026, 10, 9, 9, 30, tzinfo=UTC)
forecast = [40_000, 30_000, 20_000, 10_000]
observed = [0, 35_000, 25_000, 20_000]
realized = [35_000, 25_000, 20_000, 20_000]
mid = [100.30, 100.50, 100.70, 100.90]
slots = []
for k in range(4):
    slot_start = start + dt.timedelta(minutes=15 * k)
    observation_time = None if k == 0 else slot_start
    slots.append(ExecutionSlot(
        slot_index=k,
        slot_start=slot_start,
        slot_end=slot_start + dt.timedelta(minutes=15),
        send_time=slot_start,
        fill_time=slot_start + dt.timedelta(minutes=7),
        forecast_external_volume=forecast[k],
        observed_external_volume=observed[k],
        realized_external_volume=realized[k],
        mid_price=mid[k],
        observation_time=observation_time,
    ))
```

```
parent = ParentOrder(
    order_id="teaching-buy", asset="XYZ", signed_quantity=10_000,
    decision_time=start - dt.timedelta(minutes=2), arrival_time=start,
    decision_price=100.00, arrival_price=100.20,
)
config = ExecutionConfig(
    commission_rate=0.0,
    commission_per_share=0.005,
    half_spread_bps=0.0, temp_impact_coef=0.0, perm_impact_coef=0.0,
    max_participation=0.20,
)
result = execute_order(
    parent, "VWAP", slots, config,
    terminal_mark_price=101.00,
    terminal_mark_time=slots[-1].slot_end,
)
assert result.signed_parent_quantity == result.signed_filled_quantity + result.
    ↪ signed_remainder_quantity
assert abs(result.is_decomposition.identity_discrepancy) < 1e-9
assert result.fill_vwap == 100.50
assert abs(result.is_decomposition.total_is_decomposition - 5_050.00) < 1e-8

ledger = ExecutionPortfolioLedger(2_000_000.00, config, start)
for fill in result.fills:
    ledger.post_fill("XYZ", fill, {"XYZ": fill.mid_price})
assert ledger.positions["XYZ"] == 10_000
assert ledger.reconcile_ledger()["is_exact"] == 1.0
```

The fill price already includes spread and temporary impact. The cash ledger deducts only actual fill consideration and commission. The total_is_decomposition field is an evaluation metric relative to decision price and must not be deducted from cash again. Tests in the same module cover formal boundaries, sells, and no fills.

10. Visualization

The two figures below are inherited from the accepted original and were not rerun in this batch. The original figures/build_figures.py script generates them from explicitly listed constructed parameters and also saves a numerical CSV for the Almgren–Chriss frontier. They illustrate model directions, not execution costs or optimal strategies for real securities.

Figure 1: Cumulative Fills for the Same Parent Order

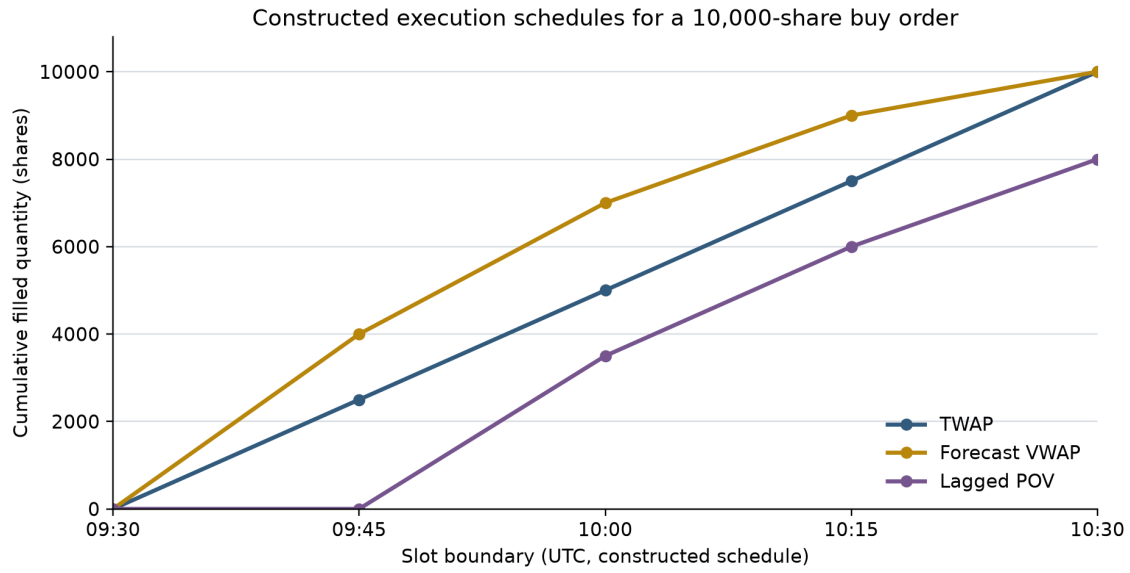


Figure 1: Cumulative shares filled under four-slot constructed schedules

TWAP requests 2,500 shares per slot. Forecast VWAP allocates according to the ex-ante 40/30/20/10% curve. Lagged POV makes no request in the first slot because no volume has yet been observed, then requests 10% of the previous slot's volume, subject to a 20% cap on current external volume. The vertical axis is cumulative shares filled and the horizontal axis is the constructed clock's slot boundaries. POV ends with 2,000 shares remaining.

Figure 2: Discrete Almgren–Chriss Expected Cost and Risk

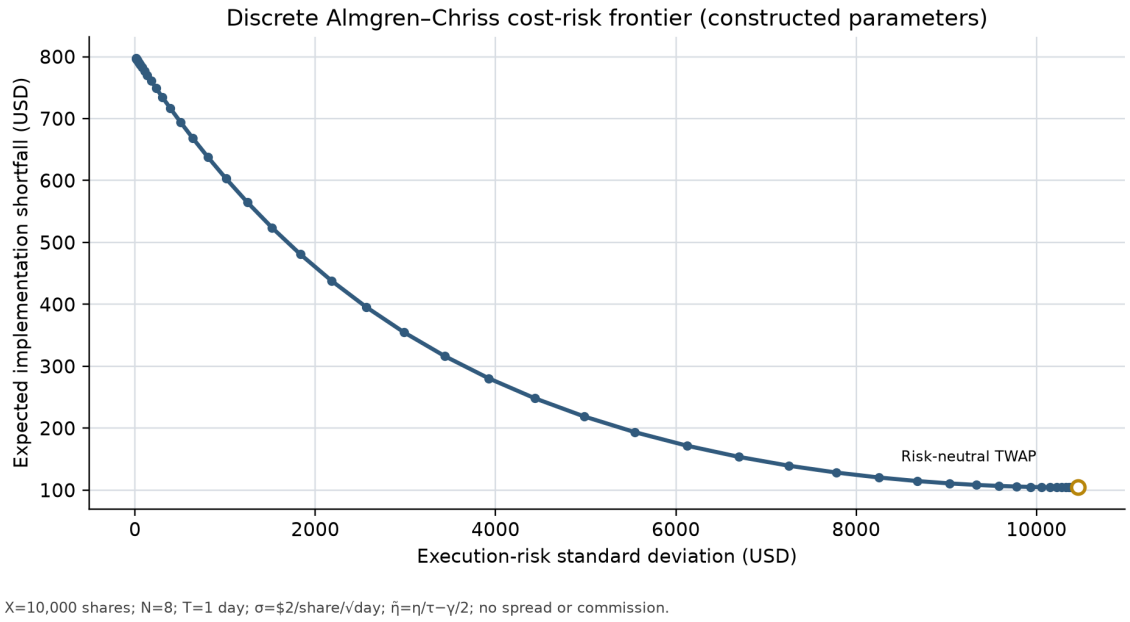


Figure 2: Constructed discrete Almgren–Chriss cost-risk frontier

Each point is calculated from the discrete inventory recursion, with a fixed 10,000-share parent order, 8 slots, and a total duration of one day; $\sigma = 2$ USD/share/$\sqrt{\text{day}}$, $\eta = 10^{-6}$ USD $\cdot$ day/share^2, and $\gamma = 10^{-7}$ USD/share^2, with adjusted temporary-impact coefficient $\tilde{\eta} = \eta/\Delta t - \gamma/2 > 0$. Commission and spread are excluded. The risk-neutral endpoint is equal-quantity TWAP; as risk aversion increases, the same constructed model trades higher expected cost for lower execution risk.

11. Evidence Boundary and TCA Study Design

This batch has no real tick-by-tick fills, order-book data, or institutional execution sample. It therefore reports no real-security case or empirical slippage. This section follows the data boundary recorded in qa/a7/ch35_ch36_batch/SOURCE_DATA_GAP.md and offers a study-design example for future work with appropriately authorized data.

11.1 Available Data and Gaps

1. **No tick-level or Level 2 feeds.** The current course environment contains daily and monthly factor-analysis data, such as Kenneth French industry portfolio returns. It has neither purchased nor mounted full-market historical millisecond trade prints, five- or ten-level order-book snapshots, nor a full-depth message feed such as Nasdaq TotalView-ITCH. The order-book sweeps and slot fills shown in the chapter are constructed for instruction and cannot be used to infer historical tick-level execution slippage for particular US or A-share securities.
2. **No borrow or financing series.** Historical security-level borrowable lists, borrow-fee rates and hard-to-borrow surcharges, and broker recall records are unavailable. The empirical feasibility of short sales cannot be closed end to end.
3. **No broker TCA sample.** There are no institutional parent-order execution details for large orders. Values such as square-root impact constants and temporary-impact shares are teaching parameters, not empirical estimates for real markets.

11.2 Example TCA Design with Real Data

With appropriately authorized market and order data, a researcher could preregister the relevant conventions for the question at hand. The steps below are an example analysis workflow, not a universal industry standard:

1. **Collect and align data.** Record the OMS/EMS parent-order decision timestamp t_d, route-send time t_{send}, and exchange fill-confirmation time t_{fill} at microsecond or nanosecond precision. Align fills to the relevant exchange's consolidated market-data feed.
2. **Capture benchmarks.**
   - **Arrival midpoint:** use the national best bid and offer midpoint at the agreed parent-order arrival instant, $P_{\text{arrival}} = (P^{\text{bid}}_{\text{NBBO}} + P^{\text{ask}}_{\text{NBBO}})/2$.
   - **Market VWAP:** define the trade window and eligible prints according to the research question, market-data source, and benchmark. Inclusion rules for auctions, off-exchange or block prints, corrections, and late reports vary by benchmark and must be documented in advance; do not assume that one set should always be excluded.
3. **Factorized impact regression.** If appropriately authorized data become available, a future study could regress parent-order slippage on trading characteristics. No estimation is performed in this batch:

$$\text{Slippage}_i = \alpha + \beta_1 \sigma_i \left(\frac{|Q_i|}{\text{ADV}_i} \right)^{0.5} + \beta_2 \text{Spread}_i + \epsilon_i$$

   Here i indexes orders, Q_i is signed share quantity (positive for buys and negative for sells), and $\text{ADV}_i > 0$ is the security's average daily share volume. Use one day's share volume in the denominator so that $|Q_i|/\text{ADV}_i$ is a fraction of daily volume; do not mix shares with dollar volume. The dependent variable Slippage_i is a direction-adjusted price deviation in USD/share, for example $\text{sgn}(Q_i)(P_{\text{fill},i} - P_{\text{benchmark},i})$. Both σ_i, the standard deviation of one-day price changes estimated from a window frozen before the decision, and Spread_i, the full spread, are in USD/share. The intercept α and residual ϵ_i therefore also have

units USD/share, while β_1 and β_2 are dimensionless. For a sell order with $Q_i = -1000$ and $\text{ADV}_i = 100000$, the original $\sqrt{-0.01}$ is outside the real domain; the corrected value is $\sqrt{0.01} = 0.1$. This is a future study design, with no real-data calibration in this batch. Slippage or costs under constructed parameters do not establish real strategy capacity.

12. Common Mistakes

This section covers six common errors in execution algorithms and trade scheduling. Each includes a symptom, underlying cause, incorrect code, a repair, and numerical implications.

12.1 Mistake 1: Look-Ahead Bias in POV Scheduling

- **Symptom:** Backtests show implausibly precise POV tracking even when volume suddenly drops, with exceptionally small slippage.
- **Cause:** When sending a child order in period k, the scheduler uses current-period realized external volume V_k^{realized} as the denominator, even though the period has not ended. A trader at 09:30 cannot know total market volume from 09:30 to 09:45.
- **Incorrect code:**

```
# Incorrect: use future current-slot volume to size the current child order
def bad_pov_schedule(parent_q, current_slot_realized_vol, participation_rate):
    target = participation_rate * current_slot_realized_vol
    return math.copysign(min(abs(parent_q), target), parent_q)
```

- **Repair:**

```
# Correct: use only the previous slot's fully settled observed volume
def good_pov_schedule(
    remaining_q, prev_slot_observed_vol, participation_rate
):
    target = participation_rate * prev_slot_observed_vol
    return math.copysign(min(abs(remaining_q), target), remaining_q)
```

- **Numerical implication:** If period-k realized volume is far above the previous slot's observed volume, the incorrect scheduler uses hindsight volume in the current request and creates look-ahead bias. The effect on actual prices depends on the price path; this error alone does not establish that the strategy must gain or lose.

12.2 Mistake 2: Charging Cash Twice for Price Impact

- **Symptom:** Backtest cash consumption greatly exceeds expectation, and NAV plunges as trading volume grows.
- **Cause:** The model computes a fill price $P_{\text{fill}} = P_{\text{arrival}} + \Delta P_{\text{impact}}$, settles the purchase at P_{fill}, and then deducts an extra impact reserve from cash:

$$\text{Cash} \leftarrow \text{Cash} - FP_{\text{fill}} - F\Delta P_{\text{impact}}.$$

- **Incorrect code:**

```
# Incorrect: the fill price includes impact, which is then charged again in cash
fill_price = arrival_price + temporary_impact
cash -= fill_qty * fill_price
cash -= fill_qty * temporary_impact  # Critical error: the impact is charged twice
```

- **Repair:**

```
# Correct: the fill price already includes temporary impact; deduct fill
    ↪ consideration and commission
fill_price = arrival_price + temporary_impact
cash -= fill_qty * fill_price + commission
```

- **Numerical implication:** The USD 4,000 in Section 8's TWAP example is the fill-price difference relative to arrival. In the constructed example, spread and temporary impact are zero; the difference comes from the price path and is not an estimate of temporary impact. Double charging would invent an additional USD 4,000 cash loss and severely understate strategy feasibility.

12.3 Mistake 3: Reporting Zero VWAP for No Fills

- **Symptom:** When an order has no fills, the risk report shows an average fill price of USD 0.00 and later computes a fictitious 100% slippage loss, triggering an alert.
- **Cause:** The code computes notional divided by volume and avoids division by zero by supplying a default 0.0.
- **Incorrect code:**

```
# Incorrect: return 0.0 when there are no fills
vwap = (
    sum(p * q for p, q in fills) / sum(q for p, q in fills)
    if fills
    else 0.0
)
```

- **Repair:**

```
# Correct: no-fill VWAP is mathematically undefined; explicitly return None or math.
    ↪ nan
total_qty = sum(q for p, q in fills)
vwap = (
    sum(p * q for p, q in fills) / total_qty
    if total_qty != 0.0
    else math.nan
)
```

- **Numerical implication:** In Section 8.5, no-fill VWAP is undefined, execution slippage is zero, and delay cost of USD 2,000 plus opportunity cost of USD 8,000 yields total shortfall of USD 10,000. If undefined VWAP is replaced by zero and parent quantity $Q = 10{,}000$ is incorrectly substituted for $F = 0$, execution slippage is miscomputed as $10{,}000(0 - 100.20) = -\$1{,}002{,}000$, breaking the attribution logic.

12.4 Mistake 4: Misrepresenting Monthly Weights as Intraday Signals

- **Symptom:** A report attributes the performance of a monthly signal, routed intraday, to the execution algorithm without distinguishing signal-generation frequency from execution frequency.
- **Cause:** The strategy calculates monthly multifactor weights, such as book-to-market and ROE, from month-end data, but the backtest lets the scheduler time execution from intraday movements. This conflates low-frequency cross-sectional signals with high-frequency execution.
- **Repair principle:** Keep the architecture in distinct layers:
  - **Portfolio layer:** Update target weights on a monthly or daily clock and generate a static signed parent order Q.
  - **Execution layer:** Receive the frozen Q and schedule child orders on an intraday discrete clock for liquidity tracking and risk control only; do not change the total position or direction unilaterally.

12.5 Mistake 5: Extreme Over-Ordering from Ambiguous POV Denominators

- **Symptom:** A trader sets participation at 20% but mixes inclusive and exclusive denominators, so requests differ from the intended target and may raise execution costs.
- **Cause:** The exclusive convention is Definition A, q/V_{ext}; the inclusive convention is Definition B, $q/(V_{\text{ext}}+q)$. If the trader intends Definition B (20% of all volume) but the code uses Definition A, it requests $0.20V_{\text{ext}}$, an inclusive rate of only $0.20/1.20 = 16.67\%$. Conversely, if the intended external-volume rate is 20% but the inverse formula $0.2/(1-0.2) = 0.25$ is used, the requested child quantity is 25% larger than intended (0.25 rather than 0.20 times external volume).
- **Repair principle:** Put the denominator convention in execution configuration and reports, and unit-test the formula. The Project 4 teaching implementation uses the exclusive denominator. A production interface supporting multiple definitions must validate the selected convention and participation-rate range.

12.6 Mistake 6: Omitting Initial NAV from Maximum Drawdown

- **Symptom:** The report shows zero maximum drawdown after a loss in the first evaluation month, or only 10% after two consecutive 10% losses.
- **Cause:** It compounds N interval returns into a wealth series but omits initial wealth $W_0 = 1$ from the running peak.
- **Correct convention:** For interval returns $r_1, \ldots, r_N$, first construct $W_0 = 1$ and $W_k = \prod_{j=1}^{k}(1+r_j)$, then calculate

$$\max_{0\le k\le N}\left\{1-\frac{W_k}{\max_{0\le j\le k} W_j}\right\}.$$

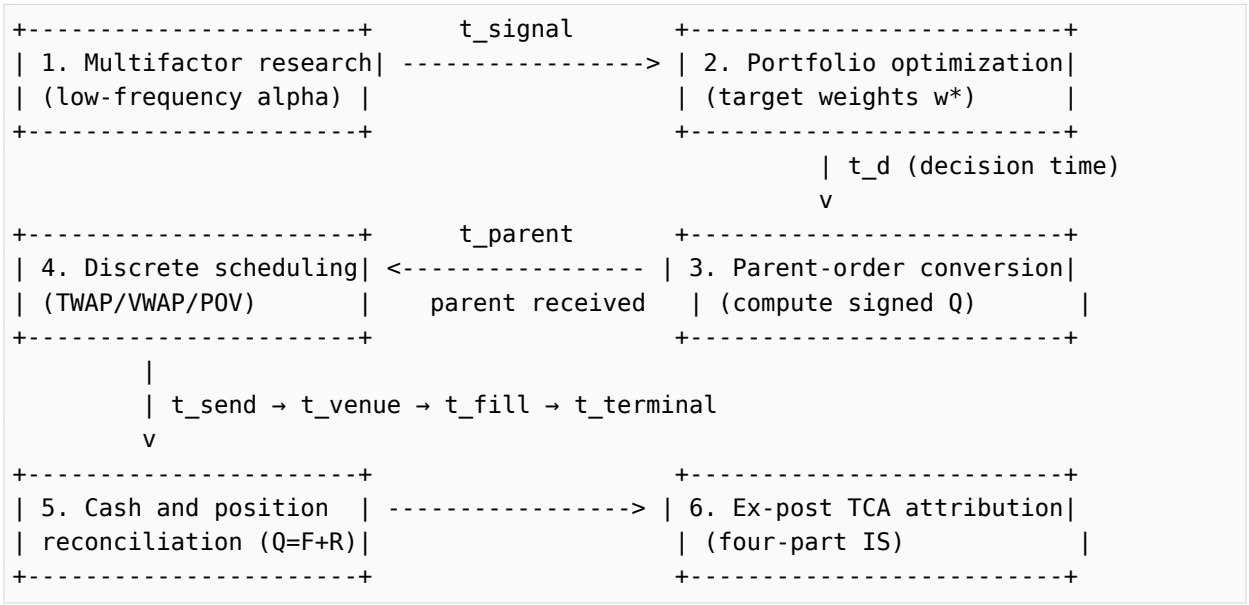
  MDD therefore uses $N+1$ wealth points, while returns, volatility, and Sharpe continue to use the original N interval returns.

- **Regression example:** MDD is 10% for $[-10\%]$; for $[-10\%, -10\%]$, wealth falls from 1 to 0.81 and MDD is 19%; a monotonically rising path has MDD 0. Project 4 recomputes this for overall, training, and test windows rather than reusing an old value that happened to be unchanged when the initial point was omitted.

13. Quant Research Application

13.1 End-to-End Institutional Execution Research Pipeline

In a rigorous quantitative investment-management system, execution is not an isolated order-submission script; it connects alpha signals to realized portfolio NAV. Its engineering loop has six causal stages:



1. **Parent-order contract.** At decision time t_d, the portfolio optimizer uses latest NAV NAV_d and target weight w_i^* to compute the target share demand for asset i:

$$Q_i = \mathrm{round}\left(\frac{w_i^* \cdot \mathrm{NAV}_d}{P_{\mathrm{decision},i}}\right) - \mathrm{Holdings}_i$$

2. **Execution-strategy comparison.** Stratify orders by $|Q_i|/\mathrm{ADV}_i$, volatility, spread, time of day, urgency, and risk limit. Compare TWAP, ex-ante forecast VWAP, lagged POV, and other schedules using common benchmarks, cost conventions, and out-of-sample periods. Any routing threshold must be calibrated on relevant order/fill data and tested out of sample; this chapter sets no universal ADV-percentage threshold.
3. **Post-trade TCA report.** After execution, generate an IS attribution dashboard comparing algorithm fill VWAP with external-market VWAP and quantifying how delay, execution slippage, and opportunity cost affect strategy-level performance.

13.2 Benchmark Comparison and Falsification Tests

Evaluation can preregister benchmarks, risk constraints, and falsification criteria to avoid selecting favorable conventions after seeing results:

1. Controls may include immediate execution, TWAP, ex-ante frozen forecast VWAP, and lagged POV for the same order and window. Include fees, unfilled opportunity cost, and data latency before comparing IS and risk. A complex algorithm warrants further study only if it adds value against a predefined objective in an out-of-sample interval.
2. A capacity stress test may report ranges under a justified set of spreads, impact functions, borrow and financing costs, participation caps, and partial-fill cases. Impact functions and participation caps require calibration on relevant data. Constructed stress scenarios show model sensitivity; they do not prove a paper signal genuine, false, or executable.

14. Chapter Summary

This chapter addresses how to turn a portfolio's signed parent order into market fills safely and efficiently. It develops a complete framework from clock separation and scheduling through impact modeling and IS attribution.

The main conclusions and decision rules are:

1. **Causal time is essential.** The data model and ledger must keep signal, decision, arrival, send, fill, and terminal-mark times distinct. Feeding future realized volume or ex-post market VWAP into the current child-order request is look-ahead bias.
2. **Schedules answer different questions.** TWAP allocates target quantity over time; forecast VWAP uses a volume curve frozen in advance; POV adjusts the request using a declared volume denominator and observations available at send time. Under constant-volatility and impact assumptions, the Almgren–Chriss mean-variance model trades expected cost against execution risk; its trajectory is not a universal prescription.
3. **IS aligns fills and missed quantity.** Implementation shortfall places the decision-to-arrival move, fill prices, terminal value of unfilled quantity, and explicit fees in one monetary framework. This chapter's four components are algebraically identical to the direct-amount method, but empirical comparisons must still state benchmark prices, windows, and fee conventions.
4. **Boundary handling determines engineering robustness.** If a price limit or exhausted liquidity results in no fills ($F = 0$), fill VWAP is mathematically undefined. Store NaN/None and use opportunity cost to capture the economic performance of the missed trade.

15. Exercises

36-E01 (Conceptual Understanding: Sign Symmetry of Implementation Shortfall)

At 09:30, a quantitative hedge-fund manager decides to sell 20,000 shares of a stock (signed parent order $Q = -20{,}000$). The decision price is USD 50.00 per share. The order reaches the execution system at 09:35, when the price has fallen to USD 49.80. By the close, the algorithm has filled 15,000 shares at an average price of USD 49.50 per share. The remaining 5,000 shares are unfilled and canceled; the closing terminal mark is USD 49.00 per share. Commission is USD 0.005 per share.

Use the signed four-component IS decomposition to calculate delay cost, execution cost, opportunity cost, explicit commission, and total IS. Verify the economic meaning of the signs.

36-E02 (Hand Check: TWAP and Forecast-VWAP Schedules)

A trader must buy $Q = +6{,}000$ shares of an asset over three time slots. The ex-ante historical forecast volumes are $V_1^{\text{forecast}} = 10{,}000$, $V_2^{\text{forecast}} = 20{,}000$, and $V_3^{\text{forecast}} = 30{,}000$ shares. The realized fill prices are USD 20.10, 20.30, and 20.50 per share. Assume every child-order request fills in full.

Calculate by hand:

1. The TWAP child requests in all three slots and the final fill VWAP.
2. The forecast-VWAP child requests in all three slots and the final fill VWAP.
3. Explain why, in a monotonically rising price environment, forecast VWAP has a worse fill VWAP than TWAP.

36-E03 (Conceptual Analysis: POV Denominators and Extreme Behavior)

Consider the two POV denominator conventions:

- Definition A (exclusive denominator): $\rho = q_k / V_{k-1}^{\text{obs}}$.
- Definition B (inclusive denominator): $\rho = q_k / (V_{k-1}^{\text{obs}} + q_k)$.

Answer the following:

1. If the target participation rate is $\rho = 0.20$ and previous observed external volume is 40,000 shares, calculate the current child request under each definition.
2. Prove that under Definition B, the uncapped reference request diverges as $\rho \to 1$ when external volume is positive. Explain why the actual finite request differs, and identify the practical market constraint behind this behavior.

36-E04 (Boundary Derivation: IS and VWAP Status with No Fills)

A quantitative market-making strategy submits a buy limit parent order for $Q = +5{,}000$ shares, with decision price $P_{\text{decision}} = \text{USD } 30.00$ and arrival price $P_{\text{arrival}} = \text{USD } 30.10$. The price jumps upward and continues to rise, so the limit order receives no fills during the session ($F = 0$). The closing terminal price is $P_{\text{terminal}} = \text{USD } 31.50$. The explicit commission rate is USD 0.002 per share.

1. State the status of fill VWAP P_{fillVWAP} and explain why it cannot be set to 0.00.
2. Calculate the four IS components step by step. Verify that total IS exactly matches the direct amount $R(P_{\text{terminal}} - P_{\text{decision}})$.

36-E05 (Mathematical Derivation: Risk-Neutral Limit of Almgren–Chriss)

In the Almgren–Chriss (2000) mean-variance liquidation model, the objective is $\min(E[x] + \lambda_{\text{risk}} V[x])$.

1. Write the discrete Euler–Lagrange difference equation.
2. Prove rigorously that as trader risk aversion $\lambda_{\text{risk}} \to 0$, the analytic solution degenerates to a constant-speed linear liquidation path (TWAP).

16. Programming Exercises

36-P01 (Programming Exercise: Causal Adapter for the Shared Execution Engine)

Write a thin adapter `run_teaching_order(...)` that accepts a parent order, algorithm name, time slots, configuration, terminal price, and terminal time; calls `quant_course.execution_costs.execute_order`; and returns the result. Do not duplicate TWAP, VWAP, POV, or fee algorithms in the chapter or Project directory.

Tests should cover:

1. $Q = F + R$; for POV, the first child request is zero when no prior observation exists.
2. For a slot k, perturb only its realized external volume and that of later slots while holding forecast volume and pre-send observed volume fixed. The request for child order k must not change in response to that future-volume perturbation. The realized fill may change.
3. For forecast VWAP, keep the frozen forecast curve fixed and change realized volume. The current sent request must not read the current slot's future volume; later requests may legitimately change due to prior actual fills and remaining quantity.
4. With a zero participation-capacity limit, there are no fills and fill VWAP is undefined.

36-P02 (Programming Exercise: Shared IS-Ledger Adapter)

Write a thin adapter `reconcile_order(parent_order, fills, terminal_price)` that derives the remainder from the signed parent order and fills, then calls `quant_course.execution_costs.compute_is_decomposition`.

Tests should include buys, sells, partial fills, and no fills. Check $Q = F + R$, equality between the four-part decomposition and the direct amount method, fees charged only on actual fills, and undefined fill VWAP when $F = 0$. Keep fees and IS attribution separate; do not deduct IS a second time from the cash ledger.

17. Challenge Problems

36-C01 (Open Derivation: Optimal Execution Speed with a Decaying Signal)

Let $T, X, \eta > 0$, $\alpha_0 \geq 0$, $\rho_a > 0$, and $\lambda_{\text{risk}}, \sigma \geq 0$. The units of α_0 are USD/share/time and those of ρ_a and κ are time^{-1}. Let x_t be cumulative buy progress in shares, with $x_0 = 0$, $x_T = X$, and trading speed $v_t = \dot{x}_t \geq 0$. The exogenous price drift is $\alpha(t) = \alpha_0 e^{-\rho_a t}$; the permanent-impact coefficient is γ, the temporary-impact coefficient is $\eta > 0$, and the price-diffusion coefficient is σ. Use the mean-variance approximation, with the risk term defined as the price exposure of the uncompleted target inventory.

1. Write the expected execution-cost and variance functionals relative to the decision price. Let $A(t) = \int_0^t \alpha(s)\, ds$, and explain why total permanent impact $\frac{1}{2}\gamma X^2$ does not depend on the schedule for a monotone completed path.
2. Derive the Euler–Lagrange equation for the schedule-dependent objective. Using $\int_0^T A(t)\dot{x}_t\, dt = A(T)X - \int_0^T \alpha(t)x_t\, dt$, give an equivalent objective that is convenient for variation.
3. For $\kappa > 0$ and $\rho_a \neq \kappa$, derive the analytic solution satisfying both endpoint constraints, where $\kappa^2 = \lambda_{\text{risk}}\sigma^2/\eta$. Also give the resonance solution for $\rho_a = \kappa > 0$ and the branch for $\kappa = 0$.
4. Compare the effects of signal decay, total signal area, risk aversion, and a speed cap on how front-loaded the schedule is. Do not interpret a boundary-layer solution without a speed cap as an executable instantaneous trade.
5. To isolate the signal effect, fix two buy paths with the same endpoints and let $\Delta x(t) = x_{\text{fast}}(t) - x_{\text{slow}}(t) \geq 0$. Under fixed peak drift α_0 and, separately, fixed finite-horizon signal area $S_0 = \int_0^T \alpha_\rho(t)\, dt$, derive the derivative of the signal-reward difference $\Delta B(\rho) = \int_0^T \alpha_\rho(t)\Delta x(t)\, dt$. Explain whether this implies a global reversal of the optimal speed.

36-C02 (Model Problem: Dark-Pool Fill Probability, Adverse Selection, and Routing Threshold)

Let the buy parent order have size Q, and route fraction $z \in [0,1]$ to a dark pool. The expected per-share cost on a lit venue is c_A; dark-pool cost per filled share is $c_B < c_A$. Dark-pool fill probability is $p(z)$. Any unfilled dark-pool quantity is later routed to the lit venue and incurs additional per-share adverse-selection cost $\Delta(z) \geq 0$. First assume linear per-share costs with no capacity limit or interaction impact.

1. Write expected cost $C(z)$ and derive the condition under which some dark-pool routing is cheaper than sending the entire order to the lit venue.
2. Suppose p is nonincreasing and Δ is nondecreasing as the share of informed traders rises. Explain how the dark-pool advantage changes, and derive the threshold at which the corner solution becomes $z^* = 0$. Identify an additional mechanism that could produce an interior solution.
3. Give two engineering implications, distinguishing model thresholds from quantities that require calibration on real fill data.

18. Solutions / Solution Hints

36-E01 Answer

1. **Inputs.**
   - Signed parent order: $Q = -20{,}000$ shares (sell).
   - Decision price: $P_{\text{decision}} = \text{USD } 50.00$.
   - Arrival price: $P_{\text{arrival}} = \text{USD } 49.80$.
   - Fill quantity and average price: $F = -15{,}000$ shares, $P_{\text{fillVWAP}} = \text{USD } 49.50$.
   - Unfilled remainder: $R = Q - F = -20{,}000 - (-15{,}000) = -5{,}000$ shares.
   - Terminal mark: $P_{\text{terminal}} = \text{USD } 49.00$.
   - Commission: $c_{\text{comm}} = \text{USD } 0.005/\text{share}$.
2. **Four components.**
   - **Delay cost:**
   $$\begin{aligned} \text{IS}_{\text{delay}} &= Q(P_{\text{arrival}} - P_{\text{decision}}) \\ &= (-20{,}000)(49.80 - 50.00) \\ &= (-20{,}000)(-0.20) = +\text{USD } 4{,}000.00. \end{aligned}$$
   The price fell by USD 0.20 in the five minutes between decision and order submission, worsening the sale benchmark and creating USD 4,000 in delay cost.
   - **Execution cost:**
   $$\begin{aligned} \text{IS}_{\text{exec}} &= F(P_{\text{fillVWAP}} - P_{\text{arrival}}) \\ &= (-15{,}000)(49.50 - 49.80) \\ &= (-15{,}000)(-0.30) = +\text{USD } 4{,}500.00. \end{aligned}$$
   The 15,000 shares filled another USD 0.30 below arrival price. The product of two negative terms is positive, recording USD 4,500 in slippage cost.

- **Opportunity cost:**

$$\begin{aligned}\mathrm{IS}_{\mathrm{opp}} &= R(P_{\mathrm{terminal}} - P_{\mathrm{arrival}}) \\ &= (-5{,}000)(49.00 - 49.80) \\ &= (-5{,}000)(-0.80) = +\text{USD } 4{,}000.00.\end{aligned}$$

  The unfilled 5,000 shares were not sold at USD 49.80, and the closing price fell further to USD 49.00. The missed chance to avoid this USD 4,000 mark-to-market loss is recorded as opportunity cost.
- **Explicit commission:** $C_{\mathrm{fees}} = |F|c_{\mathrm{comm}} = 15{,}000(0.005) = \text{USD } 75.00$.
- **Total IS:** $\mathrm{IS}_{\mathrm{total}} = 4{,}000 + 4{,}500 + 4{,}000 + 75 = +\text{USD } 12{,}575.00$.

3. **Direct-amount check:**

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= F(P_{\mathrm{fillVWAP}} - P_{\mathrm{decision}}) + R(P_{\mathrm{terminal}} - P_{\mathrm{decision}}) + C_{\mathrm{fees}} \\ &= (-15{,}000)(49.50 - 50.00) + (-5{,}000)(49.00 - 50.00) + 75 \\ &= (-15{,}000)(-0.50) + (-5{,}000)(-1.00) + 75 \\ &= 7{,}500 + 5{,}000 + 75 = +\text{USD } 12{,}575.00.\end{aligned}$$

   The methods agree. All components are positive and measure the amount received less than under the paper target of selling the full order at decision price. A sell order can reduce a long holding; it is not the PnL of a short strategy. An existing short position could instead benefit from a price decline.

36-E02 Answer

1. **TWAP schedule and average price.**
   - There are $N = 3$ slots and parent quantity $Q = 6{,}000$ shares.
   - Each requested quantity is $q_1 = q_2 = q_3 = 6{,}000/3 = 2{,}000$ shares.
   - Fill VWAP:

$$\begin{aligned}P_{\mathrm{fillVWAP}}^{\mathrm{TWAP}} &= \frac{2{,}000(20.10) + 2{,}000(20.30) + 2{,}000(20.50)}{6{,}000} \\ &= \frac{20.10 + 20.30 + 20.50}{3} \\ &= 20.3000 \text{ USD/share}.\end{aligned}$$

2. **Forecast-VWAP schedule and average price.**
   - Total forecast volume is $10{,}000 + 20{,}000 + 30{,}000 = 60{,}000$ shares.
   - Weights are $w_1 = 1/6, w_2 = 2/6, w_3 = 3/6$.
   - Requested quantities:
     - $q_1 = 6{,}000(1/6) = 1{,}000$ shares.
     - $q_2 = 6{,}000(2/6) = 2{,}000$ shares.
     - $q_3 = 6{,}000(3/6) = 3{,}000$ shares.
   - Fill VWAP:

$$\begin{aligned}P_{\mathrm{fillVWAP}}^{\mathrm{VWAP}} &= \frac{1{,}000(20.10) + 2{,}000(20.30) + 3{,}000(20.50)}{6{,}000} \\ &= \frac{20{,}100 + 40{,}600 + 61{,}500}{6{,}000} \\ &= \frac{122{,}200}{6{,}000} \approx 20.3667 \text{ USD/share}.\end{aligned}$$

3. **Economic interpretation.** In these constructed monotonically rising prices, buying earlier lowers the arithmetic average price. Forecast VWAP schedules 50% of the order in slot 3, so its average of USD 20.3667 is above TWAP's USD 20.3000. This example distinguishes benchmark tracking from timing cost; it does not establish a performance ranking for other price paths.

36-E03 Answer

1. **Child requests under each definition.**
   - Definition A (exclusive denominator):

$$q = \rho V_{k-1}^{\text{obs}} = 0.20(40{,}000) = 8{,}000 \text{ shares.}$$

   - Definition B (inclusive denominator):

$$q = \frac{\rho}{1-\rho} V_{k-1}^{\text{obs}} = \frac{0.20}{1-0.20}(40{,}000) = \frac{0.20}{0.80}(40{,}000) = 0.25(40{,}000) = 10{,}000 \text{ shares.}$$

2. **Singularity and market constraint.** Under Definition B, $\rho = q/(V_{\text{ext}} + q)$. Solving for q gives $q = \frac{\rho}{1-\rho} V_{\text{ext}}$. When external volume is positive and the request is not capped by the parent remainder, $1 - \rho \to 0$ as $\rho \to 1$, so $q \to +\infty$. The inclusive denominator means "our quantity as a share of external plus our own volume." With $V_{\text{ext}} > 0$, a finite child order cannot attain $\rho = 1$; it can only approach it as quantity grows without bound. Capping at finite parent remainder keeps the actual request finite and may prevent it from attaining the target rate. Engineering systems should impose finite limits from order remainder, risk policy, and market capacity; the equation itself does not imply a 50% threshold.

36-E04 Answer

1. **Fill VWAP status.** Since cumulative filled quantity is $F = 0$, the denominator of the weighted-average price is zero. Thus $P_{\text{fillVWAP}} = 0/0$ is mathematically undefined. The system must represent it as None or math.nan. Never assign 0.00; later analyses could misread that as buying the stock for zero and report nonsensical profit or loss.
2. **Four-component calculation and check.**
   - Remainder: $R = Q - F = 5{,}000 - 0 = 5{,}000$ shares.
   - Delay cost:

$$\text{IS}_{\text{delay}} = 5{,}000(30.10 - 30.00) = 5{,}000(0.10) = +\text{USD } 500.00.$$

   - Execution cost: sum over the nonzero-fill set $\mathcal{F} = \{k : F_k \neq 0\}$. With no fills, $\mathcal{F} = \varnothing$ and the empty sum is defined as USD 0.00. Do not construct an undefined fill VWAP or write $0 \times$ undefined.
   - Opportunity cost:

$$\begin{aligned} \text{IS}_{\text{opp}} &= R(P_{\text{terminal}} - P_{\text{arrival}}) \\ &= 5{,}000(31.50 - 30.10) = 5{,}000(1.40) \\ &= +\text{USD } 7{,}000.00. \end{aligned}$$

- Explicit fees: $C_{\text{fees}} = \sum_{k\in\mathcal{F}} |F_k| c_{\text{comm}} = \text{USD } 0.00$.
- Total IS: $\text{IS}_{\text{total}} = 500.00 + 0.00 + 7{,}000.00 + 0.00 = +\text{USD } 7{,}500.00$.
- Direct-amount check:

$$\begin{aligned}\text{IS}_{\text{direct}} &= R(P_{\text{terminal}} - P_{\text{decision}}) + 0 \\ &= 5{,}000(31.50 - 30.00) \\ &= 5{,}000(1.50) = +\text{USD } 7{,}500.00.\end{aligned}$$

The totals agree exactly. Under this chapter's convention, when there are no fills the full difference from the decision-price paper target falls into delay and opportunity cost. These are benchmark-attribution components, not cash deductions.

36-E05 Answer

1. **Discrete Euler–Lagrange equation.** In the Almgren–Chriss model:

$$U(x) = E[x] + \lambda_{\text{risk}} V[x] = \frac{1}{2}\gamma X^2 + \tilde{\eta}\sum_{k=1}^{N}(x_{k-1} - x_k)^2 + \lambda_{\text{risk}}\sigma^2\Delta t \sum_{k=1}^{N-1} x_k^2.$$

For an interior node x_k, $1 \le k \le N-1$, the stationary condition is

$$2\tilde{\eta}(x_k - x_{k-1}) - 2\tilde{\eta}(x_{k+1} - x_k) + 2\lambda_{\text{risk}}\sigma^2\Delta t\, x_k = 0.$$

Rearranging gives the discrete Euler–Lagrange equation:

$$x_{k+1} - \left(2 + \frac{\lambda_{\text{risk}}\sigma^2\Delta t}{\tilde{\eta}}\right) x_k + x_{k-1} = 0.$$

2. **Limit as $\lambda_{\text{risk}} \to 0$.** The coefficient $\lambda_{\text{risk}}\sigma^2\Delta t/\tilde{\eta}$ tends to zero, and the difference equation becomes

$$x_{k+1} - 2x_k + x_{k-1} = 0 \iff (x_{k+1} - x_k) - (x_k - x_{k-1}) = 0.$$

Thus the first difference, and equivalently the shares liquidated per slot $n_k = x_{k-1} - x_k$, is constant: $n_k = C$. With boundary values $x_0 = X$ and $x_N = 0$, summing gives $\sum_{k=1}^{N} n_k = NC = X$, hence $C = X/N$. The remaining inventory is

$$x_k = X - kC = \left(1 - \frac{k}{N}\right) X.$$

Therefore, when the trader assigns no weight to price risk ($\lambda_{\text{risk}} = 0$), minimizing expected temporary-impact cost yields constant-speed liquidation, or TWAP. This proves the limit.

36-P01 Reference Implementation and Tests

```
from dataclasses import replace
from quant_course.execution_costs import execute_order


def run_teaching_order(parent_order, algorithm, slots, config, terminal_price,
    ↪ terminal_time):
    return execute_order(
        parent_order, algorithm, slots, config,
        terminal_mark_price=terminal_price,
        terminal_mark_time=terminal_time,
    )


# Execute Section 9 first; reuse its parent, slots, config, dt and terminal mark.
result = run_teaching_order(parent, "POV", slots, config, 101.00,
                            slots[-1].slot_end)
assert result.signed_parent_quantity == result.signed_filled_quantity + result.
    ↪ signed_remainder_quantity
assert result.child_orders[0].signed_quantity == 0.0

# Current-slot realized volume affects fills, not the already-sized child request.
perturbed = [slot if slot.slot_index < 1 else
             replace(slot, realized_external_volume=slot.realized_external_volume *
    ↪ 3)
             for slot in slots]
result_perturbed = run_teaching_order(parent, "POV", perturbed, config, 101.00,
                                      slots[-1].slot_end)
assert result.child_orders[0].signed_quantity == result_perturbed.child_orders[0].
    ↪ signed_quantity
assert result.child_orders[1].signed_quantity == result_perturbed.child_orders[1].
    ↪ signed_quantity

# In this nonbinding-capacity example all requests match; this is not a universal
    ↪ claim.
vwap = run_teaching_order(parent, "VWAP", slots, config, 101.00,
                          slots[-1].slot_end)
vwap_perturbed = run_teaching_order(parent, "VWAP", perturbed, config, 101.00,
                                    slots[-1].slot_end)
assert [c.signed_quantity for c in vwap.child_orders] == [
    c.signed_quantity for c in vwap_perturbed.child_orders
]

# A binding capacity case: current request is unchanged; later catch-up may change.
low = replace(config, max_participation=0.02)
base = run_teaching_order(parent, "VWAP", slots, low, 101.00, slots[-1].slot_end)
changed_slots = [replace(slots[0], realized_external_volume=70_000), *slots[1:]]
changed = run_teaching_order(parent, "VWAP", changed_slots, low, 101.00, slots[-1].
    ↪ slot_end)
assert base.child_orders[0].signed_quantity == changed.child_orders[0].
    ↪ signed_quantity
assert base.fills[0].signed_quantity != changed.fills[0].signed_quantity
assert base.child_orders[1].signed_quantity != changed.child_orders[1].
    ↪ signed_quantity
```

```
# Zero capacity participation produces no fills and an undefined fill VWAP.
zero_config = replace(config, max_participation=0.0)
zero = run_teaching_order(parent, "VWAP", slots, zero_config, 101.00,
                          slots[-1].slot_end)
assert zero.signed_filled_quantity == 0.0 and zero.fill_vwap is None
```

In the first example, when capacity is not binding all VWAP requests happen to remain equal; the added low-capacity example is what distinguishes a current request that does not read future volume from later requests that may catch up lawfully. The shared engine's tests/test_execution_costs.py also contains volume-perturbation and VWAP-forecast-isolation tests. This adapter calls only the shared implementation.

36-P02 Reference Implementation and Tests

```
from dataclasses import replace
from quant_course.execution_costs import compute_is_decomposition, execute_order


def reconcile_order(parent_order, fills, terminal_price):
    filled = sum(fill.signed_quantity for fill in fills)
    remainder = parent_order.signed_quantity - filled
    return compute_is_decomposition(parent_order, fills, remainder, terminal_price)


# Execute Section 9 and P01 first. A genuinely partial buy uses a 2% capacity cap.
partial_config = replace(config, max_participation=0.02)
buy = run_teaching_order(parent, "VWAP", slots, partial_config, 101.00,
                         slots[-1].slot_end)
assert [fill.signed_quantity for fill in buy.fills] == [700, 500, 400, 400]
assert buy.signed_filled_quantity == 2_000
assert buy.signed_remainder_quantity == 8_000
buy_is = reconcile_order(parent, buy.fills, buy.terminal_mark_price)
assert abs(buy_is.explicit_fees - 10.00) < 1e-9
assert abs(buy_is.identity_discrepancy) < 1e-9
assert buy_is.explicit_fees == sum(fill.commission for fill in buy.fills)

sell_parent = replace(parent, order_id="teaching-sell", signed_quantity=-5_000,
                      decision_price=100.40, arrival_price=100.20)
sell = execute_order(sell_parent, "TWAP", slots, config,
                     terminal_mark_price=101.00,
                     terminal_mark_time=slots[-1].slot_end)
sell_is = reconcile_order(sell_parent, sell.fills, sell.terminal_mark_price)
assert abs(sell_is.identity_discrepancy) < 1e-9

no_fill = reconcile_order(parent, [], 101.00)
assert no_fill.fill_vwap is None
assert no_fill.explicit_fees == 0.0
assert abs(no_fill.identity_discrepancy) < 1e-9
```

In the constructed 2% capacity case, the fills are 700, 500, 400, and 400 shares: $F = 2{,}000$, $R = 8{,}000$, and fees are USD 10. The four components are USD 2,000 delay, USD 700 execution,

USD 6,400 opportunity, and USD 10 fees, for total IS of USD 9,110. The direct-amount check is also USD 9,110. Execution fees come from each actual Fill. compute_is_decomposition reports cost attribution only and does not modify ledger cash.

36-C01 Reference Argument and Rubric

Reference Derivation

Let $A(t) = \int_0^t \alpha(s)\,ds = \frac{\alpha_0}{\rho_a}(1 - e^{-\rho_a t})$. For a monotone buy path, permanent-impact cost is $\int_0^T \gamma x_t \dot{x}_t\,dt = \frac{1}{2}\gamma X^2$. Expected execution cost relative to decision price and the risk term are:

$$E[\text{IS}] = \frac{1}{2}\gamma X^2 + \int_0^T \{\eta \dot{x}_t^2 + A(t)\dot{x}_t\}\,dt, \qquad V[\text{IS}] = \sigma^2 \int_0^T (X - x_t)^2\,dt.$$

Integration by parts gives $\int A\dot{x} = A(T)X - \int \alpha x$. Since $A(T)X$ is independent of the schedule, the schedule-dependent objective can be written:

$$J[x] = \int_0^T \{\eta \dot{x}_t^2 - \alpha(t)x_t + \lambda_{\text{risk}}\sigma^2(X - x_t)^2\}\,dt.$$

Set $\kappa^2 = \lambda_{\text{risk}}\sigma^2/\eta$. The Euler–Lagrange equation is

$$\frac{\partial L}{\partial x} - \frac{d}{dt}\frac{\partial L}{\partial \dot{x}} = 0 \quad \Longrightarrow \quad \ddot{x}_t - \kappa^2 x_t = -\kappa^2 X - \frac{\alpha_0}{2\eta}e^{-\rho_a t}.$$

For $\kappa > 0$ and $\rho_a^2 \neq \kappa^2$, set

$$B = -\frac{\alpha_0}{2\eta(\rho_a^2 - \kappa^2)}.$$

A general solution is

$$x_t = X + Be^{-\rho_a t} + C\cosh(\kappa t) + D\sinh(\kappa t),$$

where $C = -X - B$ and

$$D = \frac{-Be^{-\rho_a T} - C\cosh(\kappa T)}{\sinh(\kappa T)},$$

which satisfies both endpoints.

At resonance, $\rho_a = \kappa > 0$, set $a = \alpha_0/(2\eta)$. A particular solution is $y(t) = ate^{-\kappa t}/(2\kappa)$ because

$$(d^2/dt^2 - \kappa^2)(te^{-\kappa t}) = -2\kappa e^{-\kappa t}.$$

Then take

$$x(t) = X + y(t) - X\cosh(\kappa t) + D\sinh(\kappa t), \qquad D = \frac{X\cosh(\kappa T) - y(T)}{\sinh(\kappa T)}.$$

When $\kappa = 0$ (including $\sigma = 0$), solve $x'' = -ae^{-\rho_a t}$ directly:

$$x(t) = \frac{X + a(e^{-\rho_a T} - 1)/\rho_a^2}{T}\,t + \frac{a}{\rho_a^2}(1 - e^{-\rho_a t}).$$

If $\alpha_0 = 0$ as well, this reduces to Xt/T. These are stationary points without nonnegative-speed and speed-cap constraints. Check $\dot{x} \geq 0$ and the cap over the full path. If a constraint is violated,

re-solve the convex variational problem with boundary and total-quantity constraints; pointwise clipping breaks the total or stationarity conditions and is not a constrained optimum.

To compare the signal effect alone, fix two candidate inventory paths and let $\Delta x(t) = x_{\text{fast}}(t) - x_{\text{slow}}(t) \geq 0$. Their signal-reward difference is $\Delta B(\rho) = \int_0^T \alpha_\rho(t)\Delta x(t)\,dt$. At fixed peak $\alpha_\rho(t) = \alpha_0 e^{-\rho t}$,

$$\frac{d\Delta B}{d\rho} = -\alpha_0 \int_0^T te^{-\rho t}\Delta x(t)\,dt \leq 0.$$

For the same early-inventory difference, faster decay therefore reduces capturable signal reward. Under a fixed peak, the finite-horizon signal area $\int_0^T \alpha_\rho\,dt = \alpha_0(1-e^{-\rho T})/\rho$ also changes with ρ.

With fixed finite-horizon signal area S_0, set

$$\alpha_\rho(t) = \frac{S_0\rho e^{-\rho t}}{1-e^{-\rho T}}, \qquad m_\rho = \frac{1}{\rho} - \frac{T}{e^{\rho T}-1}.$$

Then

$$\frac{d\Delta B}{d\rho} = \int_0^T \alpha_\rho(t)(m_\rho - t)\Delta x(t)\,dt.$$

Here m_ρ is the mean time under the normalized signal weights. The derivative's sign depends on how $\Delta x(t)$ is distributed across early and late times; fixed signal area alone does not imply that the reward difference must rise. These derivatives compare signal reward for two fixed paths only. The globally optimal speed also depends on temporary impact, risk, capacity, and speed constraints, so they do not establish that the optimal strategy reverses. With fixed peak, finite-horizon area changes with decay rate; finite temporary impact and participation or speed limits smooth and constrain front-loading.

Rubric

Correctly state $A(t)$ and the cost and risk units (25 points); derive the integration-by-parts objective and correct Euler–Lagrange equation (25); give an analytic solution satisfying both endpoints and discuss resonance and constraints (30); distinguish the fixed-peak and fixed-signal-area comparisons accurately (20).

36-C02 Reference Argument and Rubric

Reference Argument

Under linear per-share costs, expected cost is

$$C(z) = Q\{(1-z)c_A + z[p(z)c_B + (1-p(z))(c_A + \Delta(z))]\}.$$

Relative to routing the full order to the lit venue, the per-share cost difference on the dark-routed quantity is

$$M(z) = p(z)c_B + (1-p(z))(c_A + \Delta(z)) - c_A = (1-p(z))\Delta(z) - p(z)(c_A - c_B).$$

Thus $C(z)/Q = c_A + zM(z)$. The quantity $M(z)$ is the per-share cost difference at a given z between dark routing and sending that quantity entirely to the lit venue. If p and Δ are constant

in z, then $M < 0$ makes full dark routing cheaper, $M > 0$ makes the all-lit corner $z^* = 0$ cheaper, and $M = 0$ makes them equivalent. If p or Δ varies with z, however, the derivative of total per-share order cost is $M(z) + zM'(z)$. A positive $M(z)$ at a single point does not prove that marginally increasing dark routing is costly or that the global optimum is $z = 0$. If $M(z) \geq 0$ over the entire feasible interval, then $C(z) \geq C(0)$ and the all-lit corner is a global minimum. If informed-trader share s rises so that $p'(s) \leq 0$ and $\Delta'(s) \geq 0$, then

$$M'(s) = (1 - p)\Delta' - p'(c_A - c_B + \Delta) \geq 0.$$

Adding dark-pool capacity, venue-interaction impact, or nonlinear fill probabilities can produce an interior optimum.

In engineering, estimate fill probability, conditional slippage after a fill, the cost of rerouting failures, and venue fees using information available at the same decision time. Validate routing thresholds by asset, time, and order type, and report them alongside unfilled-order risk and urgency. Toxicity metrics and timeout rules may be candidate features or controls, but should not be treated as effective signals without sample validation.

Rubric

Clearly define the cost model and routing share (25 points); derive the correct $M(z)$ threshold and explain equality and corner cases (35); state the assumptions for comparative statics without overgeneralizing (25); distinguish estimated inputs from engineering controls (15).

19. Further Reading

To guide readers at different stages, the literature in this chapter is grouped into four levels.

Essential

- Perold, A. F. (1988). *The Implementation Shortfall: Paper vs. Reality. The Journal of Portfolio Management*, 14(3), 4–9. JPM publisher record. Further reading: the historical check verifies author and publisher metadata only, not the full text or specific sections. The teaching decomposition is this chapter's stated convention.
- Almgren, R., & Chriss, N. (2000). *Optimal Execution of Portfolio Transactions. Journal of Risk*, 3(2), 5–39. Reading focus: the discrete-time price-evolution equation in Section 2 and the efficient-frontier derivation in Section 3. Work through the discrete variational derivation for temporary and permanent impact.

Recommended

- Busseti, E., & Boyd, S. (2015). *Volume Weighted Average Price Optimal Execution*. Stanford manuscript. Reading focus: the distinction between a static schedule fixed before trading and a dynamic schedule updated with live intraday information, including the convex-optimization approach when total market volume is random.
- U.S. Securities and Exchange Commission (SEC, 2003). *Request for Comments on Measures To Improve Disclosure of Mutual Fund Transaction Costs*, Release Nos. 33-8349, 34-48952. Reading focus: the regulator's public request for comments on measuring hidden mutual-fund transaction costs, including the debates and evolving boundaries of implementation-shortfall definitions and disclosure.

Advanced

- Cesari, R., Marzo, M., & Zagaglia, P. (2012). *Effective Trade Execution: A Review of Algorithmic Trading*. Working paper, arXiv:1206.5324. Reading focus: engineering implementations of TWAP, VWAP, and POV in European and US markets, and variational solutions under different impact functions.
- Huberman, G., & Stanzl, W. (2004). *Price Manipulation and Quasi-Arbitrage*. *Econometrica*, 72(4), 1247–1275. Publisher page. Reading focus: conditions for excluding quasi-arbitrage under their time-independent price-update framework. The abstract distinguishes permanent and temporary impact; do not restate it as an unconditional empirical law.

Optional

- Kyle, A. S. (1985). *Continuous Auctions and Insider Trading*. *Econometrica*, 53(6), 1315–1336. Reading focus: the microstructure model in which informed traders transmit order flow to a market maker through discrete or continuous orders (Kyle's lambda).
- Frazzini, A., Israel, R., & Moskowitz, T. J. (2012). *Trading Costs of Asset Pricing Anomalies*. AQR working paper. Reading focus: empirical tests using nearly one trillion dollars of real large-hedge-fund trading records to study execution capacity and turnover limits for factors such as momentum and value.

20. References

The core academic and official regulatory sources cited in this chapter correspond to entries in references.md and references/sources.json. Each citation states its scope and limits:

1. ALMGREN_CHRISS_2000:
   - Citation: Robert Almgren and Neil Chriss, "Optimal Execution of Portfolio Transactions," *The Journal of Risk*, 3(2), 5–39 (2000). DOI: 10.21314/JOR.2001.041.
   - Supports: Sections 4, 6.2, 10, and 15 on the mean-variance efficient execution frontier and the discrete derivation with linear permanent and temporary impact.
   - Scope: The text uses only a teaching simplification under driftless random-walk and constant-volatility assumptions; the closed-form solution is not a guarantee of best execution in real markets.
2. BUSSETI_BOYD_VWAP:
   - Citation: Enzo Busseti and Stephen Boyd, "Volume Weighted Average Price Optimal Execution," Stanford University manuscript (2015). Paper page.
   - Supports: Sections 5.3 and 5.4, which strictly distinguish static forecast-volume weighting from the ex-post external-market VWAP benchmark.
   - Scope: The paper distinguishes static solutions from intraday adaptive solutions. Its NYSE empirical results are literature context only, not empirical output from this course's teaching simulation.
3. CESARI_MARZO_ZAGAGLIA_2012:
   - Citation: Riccardo Cesari, Massimiliano Marzo, and Paolo Zagaglia, "Effective Trade Execution: A Review of Algorithmic Trading," arXiv preprint arXiv:1206.5324 (2012). arXiv record.
   - Supports: Sections 5, 7, and 19 on the evolution and classification of TWAP, VWAP, and POV in institutional trading.
   - Scope: This is a 2012 academic working paper. Its terminology and algorithm review provide academic context; they are not current regulation or a single production standard.
4. SEC_IS_2003:
   - Citation: U.S. Securities and Exchange Commission, "Request for Comments on Measures To Improve Disclosure of Mutual Fund Transaction Costs," Release Nos. 33-8349, 34-48952, IC-26313 (2003). SEC document.
   - Supports: Sections 5.6, 6.1, and 19 on implementation shortfall in mutual-fund transaction-cost disclosure and the boundaries of its components.
   - Scope: This official request for comments reflects regulatory discussion of hidden transaction-cost measurement; it is not a current, uniformly binding legal formula for execution algorithms.

5. AQR_COST:
   - Citation: Andrea Frazzini, Ronen Israel, and Tobias J. Moskowitz, "Trading Costs of Asset Pricing Anomalies," AQR Capital Management working paper (2012). AQR paper page.
   - Supports: Sections 2, 11, and 13 on estimates of implementation shortfall for large institutional orders and background on turnover constraints to factor capacity.
   - Scope: The paper uses historical proprietary execution data from a specific large asset manager. Its empirical cost parameters must not be generalized as universal constants for all securities or teaching strategies.
6. HUBERMAN_STANZL_2004:
   - Citation: Gur Huberman and Werner Stanzl, "Price Manipulation and Quasi-Arbitrage," *Econometrica*, 72(4), 1247–1275 (2004). DOI: 10.1111/j.1468-0262.2004.00531.x.
   - Supports: Section 7.3's conditional summary of the abstract: under time-independent permanent price impact, excluding quasi-arbitrage requires linear impact; with temporary impact as well, the abstract requires only a linear permanent component.
   - Scope: The publisher record and abstract, plus the abstract and model discussion in the authors' institutional PDF, were checked. The result is not generalized into an empirical law for every market.
7. PEROLD_1988:
   - Citation: André F. Perold, "The Implementation Shortfall: Paper vs. Reality," *The Journal of Portfolio Management*, 14(3), 4–9 (1988). JPM publisher record.
   - Supports: Bibliographic identity only: author, title, journal, volume, issue, and pages. It does not support specific chapter passages, teaching definitions, or formulas.
   - Scope: The underlying literature check was performed on 2026-10-09; this BIL17A batch inherits that historical bibliographic review of author/publisher metadata, without reading the full text or performing a fresh external source check. The SEC (2003) traces an early source of implementation shortfall to Treynor (1981), so Perold is not described as its originator.

Project 4 —From Fields to Signals to Execution

Project 4 Reader Guide: From Fields to Signals and Execution

The question to answer first

Value, momentum, and quality fields differ in units and coverage. First ask whether each input was known at signal time. Then preprocess and combine under same-date policies, and finally define the support on which positions may exist. Signal targets, execution-filtered targets, and realized holdings are three objects. Neutral targets do not guarantee neutrality after fills.

Read the project objective, signal method, event timeline, seven-field contract, dictionary, fixed rules, real-input gaps, and source feasibility in order. Chapters 24–26 define the economics; Chapter 28 derives residuals, scaling, combination, and constraints. Chapter 27 `carry/BAB` methods are not in the current seven-field contract. After the signal layer, continue with Chapter 29, the ledger-interface guide, and the cost lab; consult its data dictionary, research report, and complete six-question solutions as needed. The cost lab's Exercise 1–6 and answers remain separate; this BIL15B candidate adds new constructed P4-Txx integrative exercises and complete solutions.

Default runs and user-supplied inputs

The accepted I 3 shared project reader instructions govern the reader kernel, clean-kernel execution, package-root discovery, and new `RUN_DIR.` This guide inherits that full `offline/user-supplied-input` boundary. Public availability does not establish permission to download, use, or redistribute. A real-data branch requires separate rights and execution authorization.

The signal Notebook's Chinese Markdown projection and English Markdown projection preserve I 3's 19 cells, 9 code cells, and `code/output/metadata` identity. Their present logical locations support language review. A final standalone reader package must place the selected language at the default project path and rebind package-root discovery and relative links. Current teaching review does not establish a standalone-package cold start.

The default uses seed 20261210, 240 companies including 40 banks, and 96 months, with no real company data. Its first cell creates `runs/p04_signal_layer_<uuid>/`; a new image goes to `figures/a5/p04_signal_layer.png` within that directory. The runner separately saves an execution `copy/status.` The default signal path uses no French data and does not run historical `CLI/pytest` entry points. Markdown descriptions of historical results do not mean that these tables are regenerated.

Check the current signal method in order

1. Form support from listing status known at signal time, with at least 60 listed names. Require both filing availability and ingestion before the cutoff, and select an entire filing version together.
2. Preserve economic direction in the seven fields. Nonpositive denominators and inapplicable bank measures become missing, not zero.
3. Same-date 1%/99% winsorization needs 30 values. `Industry/size` OLS uses a common complete-pair set and at least 2 residual degrees of freedom. Sample z-scores need 10 values.
4. Each style needs at least 1 available nonzero-weight field; the composite needs at least 2 styles. Renormalize over available terms and preserve missingness when a gate fails.
5. Alpha validity determines only the initial quintile ranking. Optional industry projection uses all listed support coordinates, allowing listed names without alpha to hedge. Off-support weights are exactly zero.
6. For a nonzero projection satisfying homogeneous `industry/net-zero` constraints, use one positive scalar to restore gross 2. No long–short direction or a degenerate projection means cash. A nonzero constraint target cannot use this scaling argument.
7. Report `industry/net/gross`, support residuals, coverage, and missingness separately. Post-execution information belongs to the ledger and diagnostics and cannot rewrite signals.

Read three types of results separately

Historical `A5/A6` constructed results retain the old full-column projection and negative findings. C13 corrects the `support/gross/60-name` protocol; see the current note. The default Notebook uses the current constructed method and cannot turn old tables into a new run. None of these establishes real-market alpha, universal absence of leakage, or tradable capacity.

Cost materials separately identify historical R2R6 and the 2026-10-14 current-signal summary. The English cost Notebook and its Chinese projection start from frozen I3: fixed-parent-order and `partial/zero-fill` examples are computed afresh; the other walk-forward, sensitivity, ablation, and ledger-audit displays replay the accepted summary and are not reruns. The base `run_backtest` raises on a nonzero target that is already delisted when it reaches execution. The A6 wrapper zeros such targets in an execution copy, without rewriting the signal or rescaling other names. The cost walk-forward consumer instead checks listing status on each execution date and processes existing delisted holdings first. These are separate execution paths. Complete bilingual `cost/execution/backtest` teaching continues through Chapters 29–31, with consumer review in Chapters 35–36; full P4 teaching still depends on BIL15B validation acceptance, and real-market research remains incomplete.

Real PIT statements, total returns, historical membership, capitalization, industry versions, trading calendars, `financing/borrowing`, and capacity gaps remain. This BIL15B candidate now includes new P4 integrative exercises and complete solutions, with independent P4-Txx IDs. The reading route is signal layer → Chapter 29 ledger → cost lab → Chapter 30 bias → Chapter 31 validation → these constructed cases. Cost Exercise 1–6 remain separate. The candidate counts toward full P4 teaching only after the root independently accepts BIL15B, alongside

the accepted BIL14 and BIL15A dependencies. BIL17 consumer review and BIL23 English Notebook, portable assembly, cold start, and global-link checks remain. Real-market research remains planned pending PIT data and rights, borrowing, a trading calendar, and capacity evidence.

Project 4 — Multi-Factor Equity Strategy

The reader's route

This project turns the seven equity fields in Chapters 24–26 into PIT signals, preprocessed scores, a composite, and portfolio targets. Chapters 29–31 connect those targets to the ledger and validation; Chapter 34 establishes quotes and executable prices; Chapter 35 connects actual fill notional to cash costs; Chapter 36 distinguishes requests, fills, cutoff remainders, and IS before returning to the accepted cost lab and complete solutions. Carry and low-risk methods from Chapter 27 are candidates for a later extension and are not in the current machine contract. The complete real-market project remains planned. An offline constructed example does not replace real PIT research.

Start with the reader guide, then read the signal layer, field contract, fixed rules, event timeline, dictionary, gaps, and source feasibility. Runtime and user-supplied-input rules follow the accepted I 3 shared reader contract. Every new output belongs in a unique subdirectory of `runs/` at the package root.

Prerequisites are Chapters 12, 23–26, 28–31, and 34–36. The default signal Notebook uses constructed data only; the guide links its two Markdown language projections. The accepted BIL15B teaching adds new P4 integrative teaching exercises and complete solutions, using constructed cases to connect signals, execution, the ledger, and validation. These are new teaching additions, not earlier P4 assignments or real-market research. The cost lab's Exercise 1–6 and answers remain separate and keep their existing numbering.

Research objective and data

The full project needs company-level PIT statements, filing versions, historical universe membership, delistings, and total returns, with explicit data rights and gaps. Without a historical market-wide sample, report only the sample actually available. Never backfill historical membership with today's constituents. The current teaching universe has 240 constructed companies, including 40 banks, over 96 months. It contains no real company-level PIT data. French aggregate returns cannot replace those inputs.

Workflow

1. Construct value, momentum, and quality fields under fixed definitions.
2. Apply the frozen same-date winsorization, neutralization, standardization, missing-data policy, and style combination.
3. Require at least 60 signal-date listed names before forming quintile long–short targets. Optional industry projection uses listed coordinates only and restores gross 2. Check industry, size, `net/gross`, and support residuals separately. Beta checks still require the relevant missing inputs.
4. Add transaction costs, `borrowing/financing`, and capacity assumptions separately. Distinguish signal targets from positions after execution.
5. Report walk-forward results, ablations, factor correlations, robustness, and negative results. Select parameters in training; use independent test data only for evaluation.

Time and evaluation rules

Record `event_time`, `available_at`, `ingested_at`, `signal_time`, `trade_time`, and `return/label` intervals for each input. Require `available_at ≤ signal_time < trade_time` and also check ingestion time. A closing signal executes at the next tradable time. Future returns must not enter preprocessing, version selection, or parameter choice. State cost, financing, and borrowing conventions in configuration and record every attempted variant.

Deliverables for the complete study

- `data_manifest.json`: sources, license boundaries, sample, units, SHA-256, and versions.
- `data_dictionary.md`: fields, keys, time fields, and missing-data rules.
- `notebooks/`: clean-kernel entry points reusing shared core implementations.
- `src/`, `tests/`, and `configs/`: implementation, boundary checks, and frozen settings.
- `figures/`: axis meanings and `data/code` versions.
- `research_report/`: hypotheses, `train/test` design, metrics, risk, costs, robustness, limits, and all `zero/negative` results.
- Integrative exercise solutions: complete answers to the accepted constructed teaching cases; they do not establish real-market research or a complete empirical P4 study. Cost-lab solutions keep their separate entry point.

Market conclusions require data that may lawfully be used for that purpose. Passing pytest or constructed checks cannot replace market validation.

Current methods and historical results

Preserve the old A5 signal and A6 constructed-ledger records. Full-240-column projection, gross 2.11, and 1,842 off-support name-months describe a historical inconsistency. The current support entry point and protocol note explain the corrected listed-support projection, 60-name gate, and gross 2. Historical numbers are not new C13 results. A normative correction is neither a new blind preregistration nor evidence of alpha.

Complete bilingual teaching for Chapters 1–33 and P4 has been accepted through BIL14, BIL15A, BIL15B, and the subsequent acceptance entry points. Teaching acceptance does not establish completion of real-market research. Chapters 34–36 connect executable quotes, actual fill notional and cash fees, and the request–fill–remainder–IS objects to the existing cost lab. This consumer bridge earns no duplicate P4 completion credit.

Reading route for the Chapter 29 ledger, costs, and execution

Follow the signal layer → Chapter 29 ledger → ledger-interface guide → cost lab, including its data dictionary, research report, and complete six-question solutions → Chapter 30 bias → Chapter 31 validation → new integrative exercises and complete solutions. The new constructed cases use independent P4-Txx IDs; the cost lab's Exercise 1–6 remain separate and keep their original numbering. Chapters 30–31 and the complete P4-T01–03 exercises and answers were independently accepted with BIL15B. For the execution bridge, read Chapter 34 quotes and fill evidence → Chapter 35 fill notional and cash costs → Chapter 36 scheduling, fills, cutoff remainders, and IS, then return to the cost lab's Exercise 1–6 and P4-T01–03 solutions to reconcile the consumers. This BIL17A bridge is a candidate pending independent acceptance. BIL23 English Notebook, portable assembly, cold start, and global-link checks remain. Real-market research remains planned; PIT data and rights, borrowing, a trading calendar, and capacity evidence are still missing.

Use the English cost Notebook or its Chinese projection for the interactive reading path. It follows the frozen I3 Notebook: the fixed-parent-order and `partial/zero-fill` examples are computed afresh; the walk-forward, sensitivity, ablation, and ledger-audit displays replay the accepted 2026-10-14 current-signal summary and were not rerun here. The cost configuration has 97 input months beginning in 2013-12; the first is warm-up, leaving 96 return months from 2014-01 through 2021-12. That is a different construction from the signal Notebook's 96 input months.

The historical R2R6 record and the 2026-10-14 current-signal summary have different input and result identities. The base `run_backtest` ledger does not filter a nonzero target that is already delisted on its execution date; it raises an error. The A6 wrapper zeros such targets only in an execution copy, preserving the original signal and leaving other weights unscaled. The cost walk-forward consumer separately checks listing status on each execution date and processes delisted existing holdings first. These are distinct consumers. Real PIT data, multi-factor alpha, `financing/borrowing`, and capacity remain unverified.

Project 4 — Signal Preparation: Value, Momentum, and Quality

This is a preparation layer, not a complete Project 4. It covers point-in-time (PIT) fundamental selection, a field contract, fixed `preprocessing/combination` rules, and counterexamples. It contains no transaction-cost, `borrowing/financing`, capacity, or walk-forward evaluation and no results about real firms or real factor returns. No security-level PIT statements, prices, or historical universe are available here. Every number uses a constructed teaching universe with seed 20261210 and a known data-generating process. Complete real-market evidence remains unverified.

Contents and boundaries

Item	Status	File
Dictionary: fields, units, keys, times, missingness	Available	`data_dictionary.md`
Source feasibility: historically read SEC data-set documentation and `price/universe` gaps	Available	`source_feasibility.md`
PIT events: period-end, acceptance, ingestion, `signal/trade` times, labels, `restatement/delisting`	Available	`pit_event_graph.md`
`Value/Momentum/Quality` contract	Available	`configs/signal_contract.json`, `field_contract.md`
Fixed winsorize→neutralize→z-score, combination, quintiles, industry projection	Frozen `configuration/hash`, not external preregistration	`configs/frozen_rules.json`, `rules.md`
Reader Notebook, six counterexamples, historical engineering entry points	Default uses Notebook only; historical `CLI/tests` are outside offline execution scope	`notebooks/p04_signal_layer.ipynb`; local repository for engineering sources
`Constructed/real` labels and gaps	Available	This file, `gap_table.md`
Costs, `borrowing/financing`, capacity, walk-forward, `ablation/robustness`, report	Absent from this signal layer	—
Real security-level PIT statements, `prices/total` returns, `membership/delistings`	Missing	`gap_table.md`

Label every `data/result` type

Data or result	Type
240 firms:200 nonfinancial and 40 banks;96 months January 2014–December 2021;2,257 filing versions including 97 restatements;29 delistings	Constructed teaching universe with no built-in `value/momentum/quality` effect
Availability 60–110 days after year-end; restatements 130–260 days later	Constructed rule; real availability requires SEC acceptance, subject to source checks
Rolling Rank-IC diagnostics	Historical constructed zero-effect world, t near± 1; nonsignificance alone does not establish absence of leakage, so causal perturbations are checked separately
French five `factors/ten` industries	Real aggregate data, not used by this layer and unsuitable for security-level signals

Default offline reading and outputs

Runtime and user-input contracts follow the accepted reader package's shared instructions; see the guide for both language entry points. The default runs constructed Notebook cells only, not historical `CLI/pytest.` The first cell creates `runs/p04_signal_layer_<uuid>/`; its image goes to `figures/a5/p04_signal_layer.png` inside that directory. The runner separately saves execution `copy/status` without writing frozen sources. Language projections preserve I 3 `code/output/metadata` identity and change Markdown only.

"Historical independent evidence, not bundled" identifies a historical source; it is neither an executable path nor a current output directory. The following `A5/A6` results are retained historical numbers and are not revalidated by the default Notebook. The original independent recomputation selected PIT values and seven fields cell by cell from version tables and repeated the preprocessing chain in numpy without course signal helpers. It is not rerun in this edition.

A5 historical constructed results: retain identity and values

Timing perturbation. Changing all post-June 2018 contents—returns, cap, and later-available fundamentals multiplied by 3—left the seven `fields/composite` bitwise unchanged through the cutoff,54 months, maximum difference 0. Later composite values changed, maximum 1.87. This supports the selected no-lookahead check and confirms a real perturbation; it is not a proof for arbitrary inputs.

Use period-end as availability: lookahead. About 18–20% of company-months use filings not yet submitted: `B/M` 20.3%, earnings yield 20.5%, ROE 20.4%, `GP/A18.1`%. For `February/March` signal months whose filings enter in `March/April`, naive-minus-PIT leakage correlates with next-month returns: `B/M` 0.079 (t = 3.8,14 months), `GP/A0.078` (t = 4.1). Earnings `yield/ROE` are near 0 because constructed news enters assets rather than income; other months are near-0.01 from restatements. This is a constructed leakage channel, not a real-market effect estimate.

One-version restatement rule. P102 FY2013 original available 2014-04-17: income `6.26/equity` 230.13; amendment available 2014-05-20: income `5.36/equity` 213.54. April uses the original, May onward the amendment. Mixed `numerator/denominator` ROE 2.33% differs from same-version 2.72% before and 2.51% after.

Survivor-only universe. In June 2019, keeping final survivors removes 13 names:224 versus 211. Next-month equal-weight returns are 1.13% versus 1.19%; all-month mean 0.818% versus 0.833%. There are 29 later delistings with assumed−25% delisting-month return.

Nonpositive denominators and financial firms. Of 20,579 listed cells,240 have nonpositive equity: naive `B/M` is negative, minimum−0.357; policy sets NaN, smallest positive 0.0006. Naive `EV/EBITDA` has 90 negative values, excluded under the rule. Bank `EV/EBITDA` and `GP/A` are missing. Mean field coverage among listed names: `B/M` 96.3%, earnings yield 97.4%, `EV/EBITDA` 80.4%, momentum 86.1%, `GP/A`80.8%, negative accrual 97.4%, ROE 94.7%.

Full-sample standardization. Full-sample per-name time-series z-scores of the composite disagree in sign with past-only expanding values in 11.2% of 17,258 cells.

A5 historical `portfolios/exposures`, before protocol correction and without costs. Quintile long–short gross 2.0, net below10^{-16}, mean maximum absolute industry net 0.062, mean absolute log-cap exposure 0.133. Old industry projection: gross 2.11, industry net3×10^{-17}, size exposure 0.094. These are historical full-column projection values, not a new current-C13 run. This signal layer reports no `return/Sharpe` strategy evaluation.

Zero-effect IC diagnostics. Value 0.0024 ($t = 0.35$), momentum 0.0056 (0.85), quality 0.0074 (1.02), composite 0.0071 (1.03). Momentum has 84 evaluation months because 12–1 needs a 12-month formation history; the others have 93. Nonsignificance is a historical diagnostic baseline, not proof of zero effects or no leakage. Real IC needs security-level PIT data.

Attempt counts. Seven raw fields, three styles, one composite, two portfolio methods, one fixed IC-diagnostic set, six rule counterexamples rather than alternative strategies. The historical record states that winsorization 1%/99%, ddof 1, minimum counts, and weights were written in `frozen_rules.json` before labels were viewed; local hashes alone do not independently prove that precedence.

Time fields

`event_time` is fiscal period-end or return month. `available_at` is public filing availability, constructed here; real applications require verified SEC acceptance. `ingested_at` is one day later in this universe. `signal_time` is the last instant of month-end UTC. `trade_time` is the first subsequent tradable price; next-month labels approximate this at monthly frequency, not daily precision. `Return/label` intervals are the next calendar month. A version is usable in month$t$ iff both availability and ingestion strictly precede the first instant of the next month UTC; see `pit_event_graph.md`.

Current support protocol and historical findings

Old A5 projected on all 240 columns and A6 then zeroed unlisted names. Retained evidence includes 1,842 off-support name-months and maximum post-filter industry net 0.0280748663. Current C13 corrects that mismatch: protocol, support entry. Preserve old `numbers/negative` findings without calling the issue still unfixed or relabeling them as new results.

The current method requires at least 60 signal-date listed names; an ineligible date has missing signals and cash targets. Optional industry projection builds rows only on listed support and may use listed names without alpha as hedges. Off-support weights are zero. After homogeneous `industry/net-zero` projection, restore gross 2 with one scalar; no direction or degeneracy means explicit cash. This is not a pure extreme-quintile portfolio and does not guarantee neutrality after execution. The fixed current constructed run has mean neutral-target gross 2 and off-support weight 0. Cancellations, partial fills, and price drift are a separate clock, reported in the protocol note.

This set completes signal teaching only. Full `cost/execution/backtest` explanations, dictionary, historical report, and six complete solutions continue with Chapters 29–31; Chapters 35–36 revisit execution consumers. Real PIT data, `financing/borrowing`, and capacity remain missing. The complete real-market project remains planned.

Project 4 PIT Event Timeline

A filing version passes through different events before entering a signal. Each event has a time field and a distinct use. Dates below belong to constructed company P102's FY2013 restatement example; values appear in the signal README.

Stage	Constructed time	Field and purpose
Fiscal year-end	2013-12-31	`period_end`: identifies the reporting period, not availability
Original 10-K accepted	2014-04-17 16:05 UTC	`available_at`; real-source candidate: SEC `accepted`
Original version ingested	2014-04-18	`ingested_at`; one constructed day after publication
Month-end signal	2014-04-30 23:59:59 UTC	`signal_time`; the original version is usable
Next tradable price	First tradable time in May 2014	`trade_time`; approximated at monthly frequency
`Return/label` interval	May 2014	Next calendar month; evaluation only
`10-K/A` restatement accepted	2014-05-20	Restatement `available_at`
Restatement ingestion and signal	Ingested 2014-05-21; used from May month-end	Require public availability and ingestion; replace the original for the same period

Rules

1. **Availability:** a version is usable in signal month$t$ iff both `available_at` and `ingested_at` strictly precede the first instant of the next month UTC. April 2014 uses the original; May onward uses the restatement. March and earlier have no filing for this fiscal year.
2. **Selection:** among available versions, choose the latest `period_end`, then the latest `available_at` within that period. An older-period restatement must not displace a newer period. Once available, use the restatement. All values, including prior-year comparatives, come from one version.
3. **Time-field roles:** `period_end` identifies a reporting period and is never an availability timestamp. `available_at` controls public knowledge; `ingested_at` prevents later database additions from entering earlier signals. `signal_time` is the calculation time. Do not execute at a price preceding `trade_time`. Labels follow signal time and are used only for evaluation.
4. **Industry and universe:** use industry versions known then and signal-date listing membership, including future delistings. Include delisting-month returns, then mark later returns missing.
5. **Price events:** `splits/dividends` affect total returns and cap. Use adjusted total returns, not spliced prices. This constructed layer has no corporate actions.
6. **Monthly approximation:** a signal calculated after month-end close should trade at the next tradable time. A close-to-close next-month return instead assumes execution at the preceding month-end close. That is up to one trading session apart from the intended execution contract. Daily `prices/calendars` remain a full-project gap; see `gap_table.md`.

Historical constructed counterexamples and checks: not new runs

Broken rule	Counterexample in `signal_layer.py`	Recorded observation
Rules 1/3: use period-end as availability	`counterexample_period_end`	About 18–20% of cells use unsubmitted filings; leakage correlates with next-month returns for `February/March` signal months, `B/M` 0.079 and$t = 3.8$
Rule 2: one `version/restatements`	`counterexample_restatement`	Original in April, restated in May; mixed ROE 2.33%, single-version 2.72% or 2.51%
Rule 3: no future input	`counterexample_future_perturbation`	Perturb everything after cutoff; earlier signals unchanged bitwise over 54 months, max difference 0
Rule 4: as-of membership	`counterexample_survivorship`	Survivors-only universe drops 13 names in June 2019; mean equal-weight monthly return 0.833% versus 0.818%
Rule 2 and denominator policy	`counterexample_denominators`	240 nonpositive-equity cells; naive negative `B/M` becomes NaN under the rule

Broken rule	Counterexample in `signal_layer.py`	Recorded observation
Same-date preprocessing	`counterexample_full_sample_standardization`	Full-sample and past-only standardizations differ in sign in 11.2% of cells

Events not covered

Mergers, `name/ticker` changes, CIK-to-security mapping, after-hours announcement assignment (the historically read SEC note assigns acceptance after 5:30 pm EST to the next business day), concurrent `10-K/A` filings, and nonfinancial inputs such as analyst `expectations/news.` These need resolution at the real-data stage; see `source_feasibility.md`.

Project 4 Field Contract: Value, Momentum, and Quality

Machine-readable version: `configs/signal_contract.json` (its SHA-256 is recorded in the result). Chapters 24, 25, and 26 derive the value, momentum, and quality formulas; Chapter 28 covers preprocessing.

Style	Field	Formula	Direction	Denominator and missing-data rule	Financial firms

Style	Field	Formula	Direction	Denominator and missing-data rule	Financial firms
Value	`book_to_market`	Equity / market cap	Larger is cheaper	Nonpositive equity or market cap → NaN	Applicable: bank book equity is meaningful
Value	`earnings_yield`	Net income / market cap	Larger is cheaper	Nonpositive cap → NaN; keep the sign of losses, which are not "cheapness"	Applicable
Value	`ev_to_ebitda_cheapness`	Negative same-date percentile of `EV/EBITDA`; EV = cap + debt − cash	Larger is cheaper	Nonpositive EV or EBITDA → NaN	NaN: `EV/EBITDA` is not meaningful for banks
Momentum	`mom_12_1`	$\prod_{k=s-11}^{s-1}(1+r_k)-1$	Larger is stronger	Missing month in the window → NaN; skip month s	Applicable
Quality	`gp_a`	(Revenue − cost of goods sold) / average assets	Larger is better	Missing cost of goods sold → NaN	NaN: no comparable cost of goods sold
Quality	`neg_accrual`	−(Net income − operating cash flow) / average assets	Larger is better	Any missing input → NaN	Applicable
Quality	`roe_avg`	Net income / average equity	Larger is better	Both year-end equity balances must be positive and average equity must exceed 5% of average assets; otherwise NaN	Applicable, but leverage differs; rank within industry through neutralization

The EV bridge is simplified: minority interest, preferred stock, and nonoperating assets are assumed zero. Chapter 24's complete bridge adds minority interest and preferred stock and subtracts nonoperating assets. Real data must supply and reconcile these three fields before use; the constructed universe has none of them.

General rules:

- **One version:** numerator, denominator, and prior-year comparatives come from the same filing version. Never splice a ratio across versions.
- **Average balances:** use current and prior year-end balances from that same filing for both assets and equity.

- **Missing is not zero:** missing inputs, nonpositive denominators, and inapplicable financial-sector measures yield NaN, not zero or an industry mean. Renormalize within styles and require at least 2 styles across styles.
- **Not listed:** names outside the signal-month universe have NaN in every raw field.
- **Momentum uses returns:** use total-return panels rather than spliced prices; total returns must carry corporate actions and delisting returns.
- **Common direction:** every field points toward "better" or "cheaper." Accruals and `EV/EBITDA` already reverse direction within the field definition.

Project 4 Signal Preparation Data Dictionary

Field names match `configs/signal_contract.json` and `signal_layer.py`. All data here are constructed (`frame.constructed = true`, `has_real_company_data = false`). The real-source column describes inputs needed for a complete Project 4, not data already obtained.

A. Input tables

A1. `versions`: filing versions

Primary key: `security_id` + `period_end` + `available_at`. An original filing and its restatement for the same fiscal year occupy separate rows.

Field	Meaning	Unit	Time role	Missingness/convention	Real source needed

Field	Meaning	Unit	Time role	Missingness/convention	Real source needed
security_id	Security/company identifier	String	—	Required	SEC cik mapped to a security master; missing
period_end	Fiscal year-end/reporting period	Date	event_time	Required; not availability time	SEC period
available_at	Public availability of this version	Zoned UTC timestamp	available_at	Fiscal year-end 16:05 UTC +60–110 days; restatements 130–260 days	SEC accepted; time zone still to verify in source_feasibility.md
ingested_at	Entry into this database	UTC timestamp	ingested_at	Constructed as available_at +1 day	Actual download/ingestion timestamp
form	10-K or 10-K/A	String	—	Keep both original and amendment	SEC form, prevrpt
revenue, cogs, sga, interest	Revenue, cost of goods sold, selling/general/administrative expense, interest expense	Reporting currency, common scale	Same version	Banks have NaN cogs and ebitda	XBRL tag mapping still to verify
net_income, cfo	Net income, operating cash flow	Same	Same	Same	Same
assets, assets_prev, equity, equity_prev, debt, cash, ebitda	Current/prior year-end assets/equity, debt, cash, EBITDA	Same	Same	About 3% of company-years have near-zero/negative equity; prior comparatives are stored in the same filing	Same

Every numeric value in a selected row must come from the same filing. latest_available_values selects the complete row together.

A2. `returns`, `market_cap`, `listed`

Primary key: month × `security_id`.

Field	Meaning	Unit	Time role	`Missingness/convention`
`returns`	Monthly simple total return	Decimal	Return month; `event_time` within month, no `available_at` field	NaN outside listed life; delisting-month−0.25 is constructed, followed by NaN
`market_cap`	Month-end capitalization	Currency	Month-end	NaN outside listed life
`listed`	As-of month-end universe membership	Boolean	Known at `signal_time`	Include later delistings; never backfill with today's membership

A3. `industry`

Primary key: `security_id`. Six nonfinancial industries plus `Bank`, static in the constructed universe. Real applications need the version known at signal time. SEC `sic` is the value at filing.

B. Signal-layer outputs

Each output is a month × security panel. Row$t$ uses information available by$t$ only. Every raw field is NaN for unlisted names.

Output	Meaning	Direction	Missingness
`book_to_market`	Equity / market cap	Larger is cheaper	Nonpositive `equity/cap` → NaN
`earnings_yield`	Net income / market cap	Larger is cheaper; keep losses negative	Nonpositive cap → NaN
`ev_to_ebitda_cheapness`	Negative same-date percentile of `EV/EBITDA`; EV=cap+debt−cash	Larger is cheaper	Nonpositive EV or EBITDA → NaN; banks NaN
`mom_12_1`	$\prod(1+r)-1$ over months$s-11..s-1$	Larger is stronger	Any missing window month → NaN
`gp_a`	(Revenue−COGS) / average assets	Larger is better	Banks NaN
`neg_accrual`	−(Net income−operating cash flow) / average assets	Larger is better	Any missing input → NaN
`roe_avg`	Net income / average equity	Larger is better	Both year-end equity values must be positive; average equity must exceed 5% of average assets
Processed$z$	Same-date winsorization → `industry/log-cap` residual → z-score	—	Too few valid names → entire row NaN

Output	Meaning	Direction	Missingness
`Style/composite` scores	Equal weights within styles, available-term renormalization and minimum 1 field; across styles minimum 2 styles	Larger is more bullish	Fewer than 2 styles → NaN, no zero filling
Quintile long–short weights	Top$+1/m$, bottom$-1/m$, gross 2; optional listed-support industry projection with gross restoration	—	Listed count below 60, no long–short direction, or degenerate projection → cash; listed names without alpha may hedge, off-support weights zero

The EV bridge assumes minority interest, preferred stock, and nonoperating assets are zero; the universe lacks these fields. Real use must complete Chapter 24's bridge and reconcile it.

C. Time fields and rules

`available_at ≤ signal_time < trade_time`; signal time is the last instant of month-end UTC. A version is usable in month$t$ only if both `available_at` and `ingested_at` strictly precede the first instant of the next month. Future returns are labels only and enter no signals, preprocessing, or parameter selection.

D. Current gates and output identity

The listed-name gate 60, winsorization count 30, z-score count 10, and within-style `1/across-style` 2 coverage gates are separate. OLS uses the same valid pairs for signal, all controls, and group, with at least 2 residual degrees of freedom. Combination availability counts only nonzero-weight terms. Industry-projected holdings may include no-alpha hedges and cannot be described as pure extreme-quintile holdings. A5 full-column gross 2.11 is historical; current C13 support targets restore gross 2 and report execution residuals separately.

Project 4 Fixed Preprocessing and Combination Rules

Configuration: `configs/frozen_rules.json`; result files record its local hash. This is not an external preregistration: local `timestamps/hashes` cannot prove precedence over analysis. The original record says rules were written before labels were viewed; its hash does not independently establish that order. C13 is a normative correction of existing semantics after historical results were read, not a new blind preregistration. Changing a rule requires recomputation and an entry in the trial ledger (Chapter 22).

Gate the universe before same-date preprocessing

Require at least 60 signal-date listed names. Otherwise every signal that date is missing and the target is cash, with reason `below_min_listed_names`. This counts listed support, not nonmissing field values. Fields, controls, industry labels, and policies must have been known at signal time. OLS uses the common valid set for the signal, all controls, and group; if $n - \text{rank}(X) < 2$, the entire row is NaN.

Step	Rule	Purpose and limit
1 Winsorize	Per `field/date`, use that day's 1% and 99% quantiles; at least 30 valid names, else NaN row	Limit extreme-value variance; use no other dates (Chapter 28 §6.1 and mistake 2)
2 Neutralize	Same-date OLS on all industry indicators and log cap; take residuals; insufficient residual degrees of freedom → NaN row	Remove `industry/size` exposure (§6.2–6.3). If industry predicts returns, this can remove true signal; state the hypothesis and report `before/after` IC
3 z-score	Same-date sample standard deviation, ddof 1; at least 10 names; zero dispersion → NaN row	Put fields on comparable scales

Keep the order winsorize → neutralize → z-score. Outlying residuals can remain after neutralization because winsorization came first (Chapter 28 Figure 28-2). A second winsorization is a separate recorded variant.

Combine

- Within a style: equal field weights, renormalized over available fields, minimum 1 field (`value`:3, `momentum`:1, `quality`:3).
- Across styles: equal weights, renormalized over available styles, minimum 2 styles; otherwise NaN. Do not zero-fill or substitute a single style.
- Weights use no evaluation-period performance. Estimate adaptive weights in a separate discovery period and evaluate on a holdout; Chapter 28's constructed experiment found discovery IC weighting beat equal weights in 31.7% of holdout worlds.
- Renormalization changes the scale of names with fewer factors. Tail representation also depends on distributions and missingness, not variance alone. Keep Chapter 28's constructed findings separate from this layer's historical 10.8% two-style coverage. This layer requires 2 styles. Availability counts only nonzero-weight terms.

Form portfolios

- Quintile long–short: top quintile $+1/m$, bottom quintile $-1/m$, gross 2 and net 0. Too few valid names for five buckets means cash.
- Current optional industry neutrality: project only on signal-date listed support, with independent indicator rows for industries present. Their sum is the net row, so do not add it twice. Listed names without alpha may hedge; off-support weights are exactly zero. Scale a nonzero projection by one positive factor to gross 2, preserving homogeneous `industry/net-zero` constraints. No long–short direction or a degenerate projection means explicit cash; never amplify numerical noise. Historical full-column gross 2.11 is superseded by this protocol. Nonzero targets cannot use the same scaling argument. Neutralize-then-rank and rank-then-project remain different portfolios (Chapter 28 §6.3 and §10).
- Excluded from this layer: transaction costs, `borrowing/financing`, capacity, turnover constraints, risk controls beyond rebalancing frequency, and beta checks. Only `industry/size` exposures are available here.

Evaluate diagnostics, not a strategy

Monthly Rank IC against next-month returns and naive t values treating months as independent; coverage and exposures. No net return, Sharpe ratio, or drawdown is reported by this layer.

Count attempts

Seven fields, three styles, one composite, two portfolio constructions, and one IC-diagnostic set. Six counterexamples check rules rather than offer alternative strategies. Each preprocessing parameter has one setting, not a grid.

Report signal targets, execution-filtered targets, and realized close holdings normalized by current NAV separately, with `industry/net/gross` and support residuals. Execution information cannot rewrite signals; listed status does not establish real `halt/borrow` eligibility. See the current protocol.

Project 4 Gap Table

Missing input	Consequence	Signal-layer treatment	Responsibility
Company-level PIT fundamentals, restatements, `accepted`, and prior-year comparatives	No real `value/quality` signals	Constructed universe; see `source_feasibility.md` for the historical SEC-document reading scope. Tag mapping, time zones, and annual-value filtering remain unchecked	User-supplied data or separately authorized download, followed by verification
Security prices, total returns, delisting returns, corporate actions	No real momentum or labels	Constructed returns; −25% delisting-month return is an assumption	User-supplied or authorized source
Historical universe and constituent changes	Survivorship bias	Constructed as-of membership; no claim of a real universe	Same
Historical market cap and shares outstanding	Value ratios and size neutralization	Constructed month-end cap	Same
Historical industry classifications	PIT neutralization	Constructed static labels; SEC `sic` is filing-specific	Same
Daily prices and trading calendar	Next-tradable-time execution of closing signals	Monthly approximation, differing by up to one trading session; see rule 6 in `pit_event_graph.md`	Same
Beta and market data	`Beta/exposure` checks	Not implemented here; `industry/size` exposures only	Complete-project stage
Costs, `borrowing/financing`, capacity	Net returns	Absent from this signal layer; separate constructed `cost/execution` teaching does not establish real borrowing or capacity	Complete-project stage
Walk-forward, factor ablation, robustness, report	Complete project	Absent here; historical constructed `ledger/cost` reports exist, with replay distinct from new computation	Complete-project stage
Data rights and licenses	Permitted `use/redistribution`	No restricted data downloaded	User confirmation
Minority interest, preferred stock, nonoperating assets in EV	Real-company `EV/EBITDA` would not reconcile with Chapter 24; discrepancy depends on missing items	`versions` lacks these fields; zero is an explicit constructed-universe assumption only	Add and reconcile at the real-data stage

This is a table of real signal-input gaps. It does not deny the existing constructed ledger and cost teaching. Listed-support projection is corrected under the current C13 protocol; real security-level data, trading eligibility, financing, and borrowing gaps remain.

Project 4 Source Feasibility

Sources actually read in the historical work

The SEC Financial Statement Data Sets readme was obtained on 2026-10-06 and its SUB, TAG, and NUM field definitions read. The copy belongs to historical independent evidence not bundled with the offline reader; that phrase is an identity note, not a path.

Recorded fact	Location	PIT implication
EX-101 XBRL attachments cover forms 10-K,`10-K/A`,10-Q,`10-Q/A` and others; filing dates begin 2009-04-15	§2	No pre-2009 coverage; XBRL filers only
Values are as filed, uncorrected, including amendments and duplicates	§2–3	Preserve restatement versions and select with `accepted`
SUB has one row per submission, key `adsh`; `cik`, `sic` at filing, `fye`, `form`, `period`, `fy`, `fp`, `filed`, `accepted`, `prevrpt`; acceptance after 5:30 pm EST counts as next-business-day filing	§5.1	`accepted` is an availability candidate; `period` maps to period-end; `prevrpt` marks amended submissions; filing-specific `sic` may supply then-known labels
NUM key: `adsh, tag, version, coreg, ddate, qtrs, uom`; `value` is unscaled; `qtrs=0` means an instant value	§5.3	Annual flows need `qtrs=4`, balances `qtrs=0`; verify units
Quarterly publication; filings accepted after 5:30 pm EST on the quarter's final business day go to the next quarterly release	§1	Ingestion cannot precede publication. Quarterly-package ingestion may be much later than `accepted`

Limits of that reading. The readme did not explicitly specify the `accepted` time zone; it mentioned EST in explanatory text. Verify before converting to UTC. Mappings from XBRL tags to net income, assets, equity, revenue, COGS, SG&A, interest expense, and operating cash flow, tag coverage, custom-tag proportions, and TAG values were not read or verified. No quarterly package was downloaded. Candidate tags below are an unverified checklist.

Other sources: not obtained, `rights/conventions` unchecked

Need	Candidate	Status	Note
Security `monthly/daily` total and delisting returns	`Academic/commercial` price database or exchange	Not obtained	No `vendor-license/adjustment` check; user chooses and supplies
Cap and historical shares outstanding	Same	Not obtained	Filing share capital is not arbitrary-date shares outstanding
Historical `universe/constituent` changes	Same	Not obtained	Required for real as-of membership; constructed layer demonstrates rules only
Historical industry classifications	Filing-specific SEC `sic`, commercial classification	Partial candidate	SIC differs from GICS; match research design
Borrow `availability/rates` and transaction costs	Broker	Not obtained	Outside this signal layer
Published aggregate factors	Frozen French snapshot	Available but unsuitable	Aggregate spread returns cannot support security-level signals

Implication

A real-data Project 4 cannot run before prices, cap, and historical membership are supplied. SEC data sets are a candidate for post-2009 as-filed fundamentals and acceptance times, subject to tag mapping, `accepted` time-zone, and annual-value-selection checks. The preparation layer therefore demonstrates rules with constructed data and makes no real-company claims.

Candidate XBRL tags: not verified

Net income, assets, equity, revenue, COGS, SG&A, interest expense, and operating cash flow each have common and custom tags; companies may change tags over time. Read TAG and several actual filings before producing mappings and coverage tables.

This is the historical 2026-10-06 reading scope. The language revision did not revisit SEC online, download quarterly packages, or verify tag mappings. Source feasibility does not establish data authorization.

Project 4 — A6 Ledger Interface (Constructed Example)

This page explains how the A6 interface sends P4 signal targets to the Chapter 29 base ledger. It is a constructed teaching interface, not a complete P4 project or a real-market backtest. The accepted C13 support rules define the current signal convention; A6 records made with a full 240-column projection remain historical evidence.

Reading route

Read the Chapter 29 ledger, then this guide and the A6 interface source. Continue with the cost and execution lab, and use the English cost Notebook or the Chinese cost Notebook to study order slicing, partial fills, and costs. The Notebook freshly computes only the fixed-parent-order and `partial/zero-fill` examples; its walk-forward, sensitivity, ablation, and ledger-audit sections display the accepted summary.

A6 inputs, prices, and clock

Stage	Actual interface	How to read it and its limits
Signal	`preparation/signal_layer.py` creates month-end targets. The A6 entry point first checks the signal-date `listed` support and then passes the targets to the ledger.	The accepted C13 rules require at least 60 listed names; a valid industry projection uses listed coordinates and restores gross 2 with one positive scale factor. See the current support record and protocol note. This differs from the full-240-column projection retained as historical evidence.
Constructed prices	`p4_prices` builds `P = 100 × cumprod(1 + r)` from monthly total returns and sets `E = P.shift(1)`.	The first visible price includes the first month's return and need not equal 100. Returns already include dividends, so this interface does not pass `dividends` again. These are constructed `mark/execution` proxies, not actual trade prices.
Execution clock	`BacktestConfig(lag=1)` schedules a month-end signal at `s` for execution at the next monthly index, using `E = P_s`.	The constructed assumption is a fill at the prior month-end level at the start of the next month, with no overnight gap, no impact, and a complete fill. The monthly return is measured from execution through month-end. An index step is not an exchange trading calendar.
Corporate actions and delistings	The base engine processes interval interest, `splits/dividends`, delisting cash, execution, and then the closing mark.	On the delisting date, held shares are settled at the last visible constructed price. Because the input is a total-return series, adding the dividend again would double count it.

Stage	Actual interface	How to read it and its limits
Fees and shorts	The A6 example uses proportional fees of 0, 10, or 25 bps, with `allow_short=True` and `max_gross=2.25`; targets are reached using post-fee NAV.	The base ledger uses deterministic complete fills. It does not model partial fills, market impact, borrow fees, rebates, or margin interest beyond the configured cash rate. Allowing shorts is a constructed assumption, not evidence of borrow availability or financing cost.

Execution-date delistings: wrapper and base ledger

The base `run_backtest` does not remove delisted names automatically. It raises an error if a nonzero target reaches execution after the asset has delisted. The A6 wrapper makes a copy of the target weights, sets targets to zero when the name is no longer eligible on the execution date, does not rescale the remaining weights, and passes that copy to the base ledger. It leaves the original signal targets unchanged. The copy keeps the original signal-date index; this execution adjustment does not mean the signal knew about a future delisting and is not an alpha feature.

The cost lab's `run_walk_forward_backtest` is a separate consumer. On each actual execution date it checks the `listed` state available then to set the new target, and settles already-held delisted positions through the ledger before handling new orders. It does not call the A6 wrapper's `drop_targets_delisted_at_execution`, and it should not be described as automatic filtering by the base `run_backtest`.

Output identity and run limits

The legacy shared A6 entry point runs both P4 and P5 and writes to fixed `qa/a6/claude/results/p04_p05/` and `figures/a6/` paths. It is not a standalone P4 entry point for this reader package. This revision does not run that CLI or regenerate historical results. Any "full 240 columns, gross 2.11, 1,842 off-support name-months" figures belong to the old interface identity; the current C13 listed support, 60-name gate, and gross-2 correction have separate accepted records. Constructed outputs describe workflow behavior only. They are not evidence of alpha, real execution costs, tradable capacity, or a complete P4 study.

A complete P4 study still needs company-level PIT data, historical membership, real execution prices and corporate actions, `borrowing/financing` terms, capacity assumptions, and the relevant data-use rights. The six original cost exercises and complete answers are in the cost solutions. The cost Notebook demonstrates fixed-parent orders and `partial/zero` fills; its saved-summary sections do not recompute the full studies. These six exercises do not replace a whole-project P4 assignment or research report.

Code entry points

- A6 shared interface source
- Chapter 29 base ledger
- Cost and execution guide
- Cost lab research report

Project 4 Cost & Execution Lab (A7-B Implementation Suite)

Course: Quantitative Portfolio Strategy & Market Microstructure

Teaching scope: costs, execution, and ledger interfaces

Data status: Constructed teaching data with fixed random seed `20261210`; not market evidence

Author: Yuxiao Ma

Interface and data-boundary contracts: `qa/a7/ch35_ch36_batch/INTERFACE_CONTRACT.md` and `qa/a7/ch35_ch36_batch/SOURCE_DATA_GAP.md` at the repository root

1. Objective and scope

This lab extends frictionless reference-price fills into a fill-level ledger with constructed spread, impact, commission, financing, and borrow costs. Readers can trace how those assumptions affect cash, positions, NAV, and execution evaluation. Time-Weighted Average Price (TWAP) slices a parent order evenly over time; Volume-Weighted Average Price (VWAP) allocates child orders using a volume forecast frozen in advance; Percentage of Volume (POV) sizes children only from lagged volume available when each child is sent.

The shared engine and project adapter live at `src/quant_course/execution_costs.py` and `projects/project04/cost_execution_lab/p4_cost_bridge.py`, relative to the full course repository root. The fixed-parent and partial-fill examples are small constructed ledgers; they do not substitute for full strategy results. Project 4 also depends on other signal, configuration, data, and code files, so these documents are not a stand-alone software package.

2. Paths and result identities

All paths below are relative to the full course repository root. This directory provides four bilingual cost-reading documents alongside two Notebook language projections; code, configuration, and saved summaries retain their versioned repository paths.

```
projects/project04/cost_execution_lab/
├── README.md
├── config.json
├── data_dictionary.md
├── p4_cost_bridge.py
├── research_report.md
├── solutions.md
└── cost_execution_lab.ipynb
src/quant_course/execution_costs.py
build/batches/INTEGRATION-PREP/p04_cost_refresh_20261014/runs/current_signal_01/cli/
    ↪ summary_results.json
fixtures/projects/project04/summary_results_current_signal_20261014.json
qa/a7/ch35_ch36_batch/independent_acceptance/rev2/r2r6/candidate/projects/project04/
    ↪ cost_execution_lab/outputs/a7b_rev2r6_20261009/summary_results.json
```

Keep three identities separate: A7-B R2R6 on 2026-10-09 is the historical baseline using the old signal layer; the 2026-10-14 current-signal refresh is a separately accepted constructed engineering-consumer summary; and the I3 reader Notebook recomputes only the fixed-parent and liquidity-stress examples while reading the longer-run results from the current saved summary. A shared seed does not mean different signal inputs or runs share an element-by-element historical path.

3. Core interface, clock, and ledger conventions

Parent and child quantities are signed: buys are positive and sells are negative. For one parent order, the signed fill total is $F = \sum_i f_i$, the cutoff remainder is R, and $Q = F + R$ must hold. The engine's IS interface also requires each fill on a nonzero parent to have the parent's direction; examples must not combine offsetting fills from multiple parents. With zero fills, fill VWAP is undefined and is recorded as `None`/`NaN`, never 0; there is no fill commission without an actual fill.

Fill prices use

$$P_{\text{fill}} = P_{\text{mid}} + \operatorname{sgn}(f)(\text{half-spread} + \text{temporary impact}).$$

Half-spread and temporary impact are already embedded in the fill price, so the cash ledger does not debit them again. Permanent impact shifts later mid prices and enters NAV through later marks. If spread or impact is charged twice and the duplicate is also entered as an explicit fee in both IS calculations, the decomposed and direct IS amounts may still be equal. That equality does not establish correct fee treatment. Check fill prices, commission fields, and cash postings separately.

Implementation Shortfall (IS) is decomposed into delay cost, fill cost relative to arrival, cutoff remainder opportunity cost, and explicit fees. Under $Q = F + R$, valid fill direction, and one fee entry, the decomposition equals the direct-amount calculation. Arrival price is a benchmark timestamp distinct from parent receipt or child venue arrival.

The simulation's default capacity rate uses an exclusive denominator, $\rho_{\text{excl}} = |f|/V_{\text{external}}$, so slot capacity is $\rho_{\text{excl}} V_{\text{realized}}$. With an inclusive denominator, $\rho_{\text{incl}} = |f|/(V_{\text{external}} + |f|) = \rho_{\text{excl}}/(1 + \rho_{\text{excl}})$; the reverse conversion is $\rho_{\text{excl}} = \rho_{\text{incl}}/(1 - \rho_{\text{incl}})$. For example, 20% exclusive is 16.67% inclusive, while 20% inclusive is 25% exclusive. The code includes rate-conversion and capacity helpers, but this project's configuration and fill simulation use the exclusive convention.

VWAP's forecast curve is frozen before the first child send; POV uses lagged volume already available at that send. Current-slot realized volume is not visible when the child is sent; it can only constrain that slot's later fill and enter post-trade summaries. At each send, fills that occurred strictly earlier may be used; a fill confirmed at the same timestamp comes after the send and cannot inform that child. Unconfirmed earlier children remain reserved to avoid duplicate orders. If an earlier fill changes and is known before a later send, that later child may legitimately change with the remaining parent quantity; this is feedback from past fills, not use of future volume. A fixed POV schedule does not automatically raise its participation rate to catch up with the parent; a remainder may persist at cutoff.

The cash ledger accrues financing on a monotone UTC event clock. Each interval uses its starting cash, short shares, and then-available marks with simple `ACT/360` accrual. Interest is posted at an event and becomes part of the next interval's cash basis. Fills, every slot-end mark (including no-fill slots), terminal marks, and month-end are settlement events. A mark first observed at an event cannot be used to price the interval that just ended.

Portfolio returns are month-end to month-end, with initial NAV as the first interval's starting value. Rebalance sizing and turnover use NAV after financing has settled through the rebalance time. Turnover includes tradable risky assets only; changes in cash weight do not count as stock trades. One-sided turnover is half the sum of absolute risky fill notional divided by rebalance NAV; total turnover is the unhalved amount. Reported annual turnover is the arithmetic monthly mean multiplied by 12. Thirteen intraday slots do not create thirteen portfolio-return periods.

4. Reading and reproduction path

Read the data dictionary, research report, and complete answers to all six exercises, then compare them with the interface code, configuration, and versioned result summaries at the repository-root paths above. The constructed volume curve is estimated from 60 samples from a declared U-shaped DGP, not from observed market volume.

The I3 reader candidate Notebook is at `projects/project04/cost_execution_lab/cost_execution_lab.ipynb`. It loads the project configuration, recomputes the frozen volume curve, the 10,000-share fixed-parent `buy/sell` comparison, and a 5,000-share liquidity-stress example, and checks the small examples against the saved current-signal summary. Its longer-run 96-month walk-forward results, high-friction scenario, sensitivities, ablation, and ledger audits are replayed from the saved current summary; the Notebook does not rerun the

full walk-forward study by default. Running it creates a unique `runs/cost_execution_lab_<random-id>/` directory at the repository root and writes images there, so it needs the full repository and its runtime dependencies. This documentation candidate does not include those run files.

The cost configuration explicitly requests 97 input months starting in 2013-12. The first month is warm-up, producing 96 return months from 2014-01 through 2021-12: 60 training months (2014-01 through 2018-12) and 36 test months (2019-01 through 2021-12). Do not treat a signal function's 96-month default input as the same sequence as the cost CLI's 97-month configured input.

5. How to read the results

The old-signal R2R6 test-period `CAGR/annualized` Sharpe are negative for every track: GROSS −2.09%/−0.34, TWAP −2.72%/−0.44, VWAP −2.71%/−0.44, and POV −6.62%/−1.12. Its high-friction scenario has net `CAGR/Sharpe` of −8.01%/−1.54. These results describe the accounting and cost changes on that constructed path; they do not show positive gross alpha being erased by costs.

The 2026-10-14 current-signal refresh uses the same cost configuration with 97 input months and 96 return months, but has a different signal-consumer identity. Its 36-month test `CAGR/annualized` Sharpe are: GROSS −1.9560%/−0.3334, TWAP −2.3459%/−0.4014, VWAP −2.3327%/−0.3994, and POV −6.0717%/−1.0800. The current high-friction scenario has net `CAGR/Sharpe` of −7.4392%/−1.5050. Its sensitivity grid contains both positive and negative values; no winner was selected from the grid, which is not a strategy-selection or empirical conclusion.

The research report gives the fixed 10,000-share parent-order IS results, liquidity-stress table, full historical negative results, current refresh summary, and path definitions. Do not compare numbers without their result identity and constructed assumptions.

6. Data gaps

Inputs are not point-in-time historical financial statements, an actual historical security universe, tick-level order books, broker fills or commission records, point-in-time borrow availability, or calibrated intraday volume. The constructed execution day is the third calendar day of each month in UTC, not an exchange calendar. Each run has 13 half-hour slots starting from 09:30 through 15:30 UTC, fill events 15 minutes after each slot starts, a 16:00 final slot end, and a 16:00:30 terminal mark. The results do not establish real alpha, realized trading costs, commercial capacity, or complete Project 4 strategy validity. Target-weight scaling changes requested turnover and gross exposure together, so it is a joint scenario rather than an isolated turnover intervention.

Project 4 Cost & Execution Lab: Data Dictionary

Date: 2026-10-15 (bilingual reader draft; data and code versions remain identified by their own summaries)

Version: 1.0.0

Status: Constructed teaching data, not market evidence; seed `20261210`

Reference contracts: `qa/a7/ch35_ch36_batch/INTERFACE_CONTRACT.md` and `qa/a7/ch35_ch36_batch/SOURCE_DATA_GAP.md` at the repository root

1. Scope and empirical boundary

Inputs and intermediate values in this lab are generated deterministically from a synthetic data-generating process (DGP) with fixed pseudo-random seed `20261210`. The repository does not contain historical point-in-time company financial statements, tick-level order books, actual borrow fees or availability, or broker execution reports for this experiment.

The parameters, backtest returns, and capacity estimates are not empirical facts about actual securities or commercial strategies. This dataset is used to check ledger identities, causal event ordering, and Implementation Shortfall (IS) decomposition under controlled conditions.

The cost configuration explicitly requests 97 monthly input dates starting in 2013-12; the first month is warm-up. Evaluation returns cover the 96 months from 2014-01 through 2021-12, divided chronologically into 60 training months (2014-01 through 2018-12) and 36 test months (2019-01 through 2021-12). This differs from the signal module's 96-month default input in some functions. A shared seed does not establish an identical element-by-element historical path.

2. Event clock and causal ordering

All timestamps are timezone-aware UTC with an explicit offset. The constructed path has zero child transmission delay, so child send and venue arrival are equal. Slot windows may overlap. Sends are processed in slot order. Before sizing each child, the engine applies only fills that occurred strictly before that send and reserves quantities of earlier children whose fill events are still pending. At an equal timestamp, the send precedes fill confirmation. This permits overlapping slots without using information unavailable at send time.

$$t_{\text{decision}} \leq t_{\text{parent received}} \leq t_{\text{arrival benchmark}} \leq t_{\text{child send}} = t_{\text{child venue arrival}} \leq t_{\text{fill}}$$

The two information cutoffs are independent requirements: `observed_available_at <= child_send_at` and `forecast_frozen_at <= child_send_at`. Current-slot realized external volume is unknown at child send; it can only constrain later fill capacity and enter

post-trade statistics. The forecast-volume curve is frozen before the first child send. Later sends need not wait for the previous slot to end: the engine uses fills available by the send and reserves unresolved earlier child quantities. At a given send, only confirmed fills that occurred strictly earlier may be used. A later child can legitimately change if an earlier fill changes and is known before that later send; this is past-fill feedback, not future-volume leakage.

The code's default capacity convention uses an exclusive denominator, `own_fill / external_volume`. The constructed execution date is the third calendar day of each month in UTC; it is not an exchange calendar. The 13 half-hour slots begin at 09:30, 10:00, …, 15:30 UTC. Each fill event is 15 minutes after its slot starts, and the terminal mark occurs at 16:00:30 UTC.

Timestamp field	Format / type	Causal availability requirement	Description
`decision_time`	ISO-8601 UTC	At or before parent receipt	Portfolio decision timestamp.
`parent_received_at`	ISO-8601 UTC	`decision_time <= parent_received_at`	Time the broker receives the parent order.
`arrival_time`	ISO-8601 UTC	`parent_received_at <= arrival_time <= child_send_at`	Timestamp for the IS arrival-price benchmark; distinct from parent receipt and child venue arrival.
`child_send_at`	ISO-8601 UTC	After parent receipt; equal to venue arrival in this constructed path	Child dispatch time.
`child_venue_arrival_at`	ISO-8601 UTC	At or after child send; equal in this constructed path	Time the child reaches the venue.
`observed_available_at`	ISO-8601 UTC or null for zero observed volume	At or before child send	Availability time of lagged volume used to size a POV child.
`forecast_frozen_at`	ISO-8601 UTC	At or before child send	Time the VWAP forecast profile is frozen.
`fill_at`	ISO-8601 UTC or null for no fill	When present, at or after venue arrival and no later than slot end	Time an actual fill executes and becomes available to the ledger.
`slot_start`, `slot_end`	ISO-8601 UTC	`child_send_at <= slot_start < slot_end`; slot windows may overlap	Start and end of each 30-minute execution slot.
`slot_fill_event_at`	ISO-8601 UTC	`slot_start < slot_fill_event_at <= slot_end`	Scheduled `fill/capacity` event time, retained even for a no-fill attempt.
`slot_mark_available_at`	ISO-8601 UTC	Equal to `slot_end`; after fills in the slot	Availability time of the constructed slot-end mid mark, whether or not a fill occurred.
`slot_mark_price`	`float64`, `USD/share`	Positive; available at `slot_mark_available_at`	Constructed post-slot mark; same-slot permanent impact is reflected after the fill.

Timestamp field	Format / type	Causal availability requirement	Description
`terminal_mark_time`	ISO-8601 UTC	At or after the constructed execution window	Availability time of the terminal mark used to value the remainder and update the later borrow basis.

3. Execution data schemas

3.1 Parent orders table (`parent_orders`)

Primary key: `order_id`

Field	Type	Units	Constraints / validation	Description
`order_id`	string	—	Unique	Unique parent-order identifier.
`asset`	string	—	Security identifier	Security identifier, for example `P001`.
`signed_quantity`	`float64`	shares	Finite; positive for a buy, negative for a sell, zero for no order	Target parent quantity, Q.
`decision_time`	datetime	UTC	At or before `parent_received_at`	Portfolio rebalance decision time.
`parent_received_at`	datetime	UTC	At or after `decision_time`	Time the broker receives the parent order.
`arrival_time`	datetime	UTC	At or after parent receipt and no later than child send	Timestamp anchoring the IS arrival-price benchmark.
`decision_price`	`float64`	`USD/share`	Finite and greater than 0	Reference mid at decision time, P_{decision}.
`arrival_price`	`float64`	`USD/share`	Finite and greater than 0	Reference mid at `arrival_time`, P_{arrival}.

3.2 Execution slots table (`execution_slots`)

Primary key: `(asset, slot_index)`

Field	Type	Units	Causal access policy	Description
`slot_index`	`int32`	index	$0 \le k < K$	Sequential 30-minute slot index within the day.
`child_send_at`	datetime	UTC	At or before venue arrival	Child-order dispatch time.
`child_venue_arrival_at`	datetime	UTC	At or after child send; equal in this constructed path	Time the child reaches the venue.
`slot_start`	datetime	UTC	At or after child send	Start of the execution interval.
`slot_end`	datetime	UTC	After slot start	End of the execution interval.
`fill_at`	datetime	UTC	At or after venue arrival and within the slot	Fill time.
`mid_price`	`float64`	`USD/share`	Observable before the trade	Unaffected mid-market price before this slot's child fill.
`forecast_external_volume`	`float64`	shares	Frozen before send	Forecast volume profile, used only by VWAP allocation.
`observed_external_volume`	`float64`	shares	Observed before send	Lagged external volume, used only to size POV child orders.
`observed_available_at`	datetime or null	UTC	No later than child send	Availability timestamp for lagged volume; positive observed volume requires a timestamp.
`forecast_frozen_at`	datetime	UTC	No later than child send	Time the forecast profile was frozen.
`realized_external_volume`	`float64`	shares	Realized after send	Current-slot realized external volume; used only for fill capacity and post-trade statistics.

3.3 Child fills table (`child_fills`)

Primary key: `fill_id`

Field	Type	Units	Description
`fill_id`	string	—	Unique fill identifier.
`order_id`	string	—	Foreign key to the parent order.
`slot_index`	`int32`	index	Slot in which the fill occurred.
`fill_at`	datetime	UTC	Fill time.
`signed_quantity`	`float64`	shares	Executed quantity f_i; positive for a buy and negative for a sell.
`fill_price`	`float64`	`USD/share`	Realized price, $P_{\text{fill}} = P_{\text{mid}} + \text{sgn}(f)(\text{half-spread} + \text{temporary impact})$.
`notional`	`float64`	USD	Gross traded value, $\lvert f \rvert P_{\text{fill}}$.
`half_spread_usd`	`float64`	`USD/share`	Half-spread attribution embedded in the fill price.
`temp_impact_usd`	`float64`	`USD/share`	Temporary-impact attribution embedded in the fill price.
`perm_impact_usd`	`float64`	`USD/share`	Permanent price shift applied to subsequent mid prices.
`commission`	`float64`	USD	Explicit fee paid on actual executed notional.

4. Invariants and accounting identities

1. **Quantity conservation**

$$Q = F + R$$

Here Q is the signed parent quantity, $F = \sum_i f_i$ is total signed filled quantity, and R is the signed cutoff remainder. With no fill, $F = 0$ and $R = Q$; fill VWAP is `None`/`NaN`, never 0. The IS function checks this invariant for one parent, requires each nonzero fill to match the parent's direction, and prevents fills above the parent quantity. It is not a general check for fill tables that net across parents.

2. **Cash ledger conservation and financing**

For each consecutive ledger-event interval $[t_{j-1}, t_j]$, `ACT/360` simple accrual uses only the interval-start basis:

$$I_j = C_{j-1} r(C_{j-1}) \frac{\Delta d_j}{360}, \qquad B_j = \sum_{i:H_{i,j-1}<0} \lvert H_{i,j-1} \rvert P_{i,j-1}^{\text{available}} b_i \frac{\Delta d_j}{360}.$$

Cash is updated at each fill, slot-end mark (including no-fill slots), terminal mark, and month-end event:

$$C_j = C_{j-1} + I_j - B_j - \sum_i f_i P_{\text{fill},i} - \sum_i \text{commission}_i.$$

Each interval uses simple `ACT/360` accrual. Interest posted at an event becomes part of the cash basis for the next interval, which is discrete event capitalization. At a fill or new-mark event, settle the old basis through that event time before applying the fill or newly observed mark. A mark unavailable at the interval start cannot price that interval's borrow fee. Half-spread and temporary impact are included in P_{fill} and must not be debited again. Positive cash uses `cash_rate_annual`; negative cash uses `cash_debit_rate_annual`. If a configuration omits the debit-rate field, the engine reuses `cash_rate_annual`.

3. **NAV identity**

$$\text{NAV}_t = C_t + \sum_j q_{j,t} P_{\text{mark},j,t}.$$

4. **Implementation Shortfall direct identity**

For one parent order with direction-matched fills and $Q = F + R$, the direct amount is

$$\text{IS}_{\text{direct}} = \sum_i f_i (P_{\text{fill},i} - P_{\text{decision}}) + R(P_{\text{terminal}} - P_{\text{decision}}) + \sum_i \text{commission}_i,$$

and the four-part decomposition is

$$\begin{aligned}\text{IS}_{\text{decomposition}} &= Q(P_{\text{arrival}} - P_{\text{decision}}) \\ &+ \sum_i f_i (P_{\text{fill},i} - P_{\text{arrival}}) + R(P_{\text{terminal}} - P_{\text{arrival}}) \\ &+ \sum_i \text{commission}_i.\end{aligned}$$

They are algebraically identical. This check confirms only that the two computations agree. If a duplicate `spread/impact` debit is also included in both fee terms, the identity may still pass. Check the separate economic meaning of embedded fill costs, explicit commissions, and cash postings.

5. **Participation-rate denominator and capacity**

The exclusive rate is $\rho_{\text{excl}} = |f|/V_{\text{external}}$, ranges from 0 to ∞, and gives capacity $\rho_{\text{excl}} V_{\text{external}}$. At 0, capacity is 0; at 1, it equals external volume. The inclusive rate is $\rho_{\text{incl}} = |f|/(V_{\text{external}} + |f|)$, ranges from 0 to 1, and converts by $\rho_{\text{incl}} = \rho_{\text{excl}}/(1+\rho_{\text{excl}})$ and $\rho_{\text{excl}} = \rho_{\text{incl}}/(1-\rho_{\text{incl}})$. Thus 20% exclusive is 16.67% inclusive; 20% inclusive is 25% exclusive; and 100% exclusive is 50% inclusive. At 100% inclusive, the exclusive rate tends to infinity and the capacity helper returns no upper bound. The current fill simulation uses the exclusive convention. POV child requests still use lagged observed external volume at send; realized volume only constrains the later actual fill.

5. Monthly evaluation and turnover

The signal layer, portfolio returns, and walk-forward evaluation are monthly. Each constructed rebalance has 13 half-hour execution slots, not 13 intraday portfolio returns. Month-end-to-month-end return uses prior month-end NAV as its denominator; the initial portfolio value starts the first period. Financing between that mark and the next decision falls into the following month-end return. Trade sizing and turnover use NAV after financing has settled through the rebalance timestamp.

Turnover and fill-notional statistics include tradable risky securities only. Requested dollar notional uses requested shares at the decision price; filled dollar notional uses executed shares at the actual fill price, including spread and temporary impact. Slippage may make filled turnover slightly higher or lower than requested turnover even when fill quantities do not exceed the parent order. Monthly one-sided filled turnover is half the sum of absolute risky fill notional divided by rebalance NAV; monthly total filled risky turnover is the unhalved sum divided by that NAV. Cash-weight changes create no stock fills, spread, impact, or commission. Annual values shown in reports are the arithmetic mean of monthly turnover rates multiplied by 12.

6. Outputs and result versions

`summary_results.json` stores experiment summaries and balance-sheet reconciliations. `ledger_audit.csv` stores 96 monthly records per strategy, including `nav_period_start`, `nav_rebalance`, month-end `cash/positions/available` marks, and NAV. `financing_audit.csv` records each positive-duration financing interval's starting cash, short shares, marks and availability times, rates, elapsed days, interest, and borrow fee for independent segment recalculation. `execution_clock.csv` records the timestamp chain, information cutoffs, frozen forecast, child requests, fill status, slot-end mark time and price, and terminal-mark time. `mark_events.csv` exports `fill/delist`, all slot-end (including no-fill and `held/no-order`), terminal, and month-end marks so the latest available prior mark for each borrow interval can be checked independently.

The old A7-B R2R6 summary contains 7,574 financing intervals, 1,434,399 mark events, and 753,285 execution-clock rows. Of its order slots, 231,780 had no fill but still received an available slot-end mark. R2R6 charts and the executed Notebook belong to its historical candidate tree. The 2026-10-14 current-signal refresh is a separate result identity. The I3 reader Notebook reads the saved current summary for its long-run walk-forward, sensitivity, ablation, and ledger sections; this does not mean the Notebook reran those studies. All generated paths are constructed teaching data, not market observations.

Project 4 Cost & Execution Lab: Research Report

Batch: A7-B, Chapters 35–36 integration

Historical version: R2R6, 2026-10-09 (old signal-layer identity; kept separate from the 2026-10-14 refresh)

Seed: 20261210

Status: Constructed teaching experiment; not empirical evidence or full Project 4 acceptance

1. Scope and data boundary

This module connects the existing monthly Project 4 signal layer to a new execution and ledger interface. Its purpose is to teach order scheduling, partial fills, transaction-cost attribution, implementation shortfall, financing, and reconciliation on a reproducible constructed path.

The cost configuration explicitly uses a constructed 240-firm universe (including 40 banks), 97 monthly input dates starting in December 2013, and 96 evaluation returns from January 2014 through December 2021; the first input month is warm-up. It uses a 60-month training period (January 2014–December 2018) and 36-month test period (January 2019–December 2021). Intraday illustrations use 13 half-hour slots and a volume profile frozen from training inputs. Seed 20261210 fixes the generated path. The experiment applies the configured annual borrow-rate scalar uniformly across names; this is a single-rate teaching scenario, not a point-in-time per-security borrow series. The cost CLI's 97-month configured input is distinct from 96-month defaults in some signal-layer functions; a shared seed does not make the inputs element-for-element identical.

The inputs are not point-in-time historical financial statements or a historical security universe. The repository has no historical `order-book/quote` stream, broker fill and fee records, point-in-time borrow availability, or calibrated intraday volume data for this experiment. The outputs below do not establish real alpha, real-world execution costs, strategy capacity, or complete Project 4 validity. The data boundary is recorded separately in `qa/a7/ch35_ch36_batch/SOURCE_DATA_GAP.md`.

2. Method and accounting contract

The shared implementation is `src/quant_course/execution_costs.py`; the adapter and experiments are in this directory. Parent and child quantities are signed, with $Q = F+R$. The IS function checks this identity for one parent order, requires each nonzero fill to have the parent's direction, and rejects fills beyond the parent quantity; it is not a general validation of offsetting fills combined across parents. Fill price includes half-spread and temporary impact, so those components are not debited from cash a second time. Permanent impact changes later marks.

Commissions apply only to actual fills; zero-fill VWAP is undefined and zero-fill commissions are zero. If a duplicated `spread/impact` debit is also inserted as a fee in both IS formulas, the two IS totals may still agree; equality alone cannot prove correct fee semantics.

TWAP schedules evenly by time. Forecast VWAP uses a pre-scheduled volume profile frozen before the first child send. POV uses only volume fully observed and available by the child-send time; current-slot realized volume constrains fill capacity and is used for post-trade summaries. Capacity uses the exclusive convention, own fill divided by external volume; 20% exclusive corresponds to 16.67% of combined volume. The code also provides conversion and capacity helpers for an inclusive denominator, under which 20% inclusive corresponds to 25% exclusive. The benchmark's realized market VWAP is an evaluation statistic, not a forecast available to the scheduler. Child send and venue arrival are equal in this zero-latency teaching path; all timestamps are timezone-aware.

Slot windows may overlap. Before each send, the engine applies only fills with `fill_at < child_send_at` and reserves the full quantity of sent children whose fill event remains unresolved. At equal timestamps, child sends precede fill confirmation. Later sends may use earlier fills already confirmed before those sends; that is legitimate past-fill feedback. A fixed POV schedule does not raise its participation rate to catch up with the parent, and a remainder may persist at cutoff. The IS `arrival_time` is a benchmark timestamp, separate from parent receipt and child venue arrival. POV volume observations and VWAP forecasts must each be available by the child send.

Cash interest and borrow are settled on one monotone UTC event clock. Within each interval, cash interest and borrow use simple `ACT/360` formulas on the cash balance, short shares, and last mark available at the interval start. Accrual is posted at each fill, every slot-end mark (including no-fill slots), the execution-window terminal mark, and month-end, so posted cash interest enters the next interval basis. At an event, the old balance and prior mark accrue through the event time before the fill or newly observed mark is applied. A mark first available at an event cannot price the interval ending there. A held security with no rebalance order still receives constructed slot-end and terminal marks. Empty months still settle through month-end. The monthly ledger, `financing_audit.csv`, `execution_clock.csv`, and `mark_events.csv` expose the event bridge and mark-availability history for independent segment-level recomputation. This is the declared constructed-path timing convention, not a claim about a live venue feed.

Implementation Shortfall includes decision-to-arrival delay, fills relative to arrival, the cutoff remainder marked at the terminal price, and explicit fees. The direct and decomposed calculations are both produced by the engine. Portfolio trading metrics include only tradable risky securities, excluding cash. One-sided filled risky-notional turnover is half of filled risky notional divided by rebalance NAV; total filled risky-notional turnover is the unhalved amount. Requested and filled turnover are separate fields. Annual summaries are the arithmetic mean of monthly turnover over the stated evaluation window multiplied by 12. Cash-weight changes do not create stock fills or fee notional. The 13 half-hour execution slots describe each constructed rebalance window; the signal layer and return series remain monthly. The annualized Sharpe shown here is arithmetic monthly mean divided by monthly standard deviation, multiplied by the square root of 12.

3. Fixed-parent algorithm comparison

Each algorithm receives the same constructed 10,000-share parent order, reference prices, 13-slot schedule, and cost assumptions. Total IS is in basis points of parent arrival notional and includes the realized price path, delay, fill-price execution component, and explicit fees.

Side	Algorithm	Fill VWAP (USD)	Total IS (USD)	Total IS (bps)	Identity
Buy	TWAP	105.272	55,167.99	552.74	Pass
Buy	Forecast VWAP	105.162	54,065.56	541.70	Pass
Buy	Lagged POV	106.185	64,303.27	644.27	Pass
Sell	TWAP	97.389	25,677.77	257.01	Pass
Sell	Forecast VWAP	97.469	24,872.43	248.95	Pass
Sell	Lagged POV	95.426	45,293.33	453.35	Pass

These outcomes describe one simulated price path; they do not isolate a general ranking of algorithms. IS values also include price movement from arrival to later fills and may differ by direction.

4. Partial fills and zero-fill boundary

The constructed stress test submits a 5,000-share buy order under the indicated participation cap. (R=Q-F). Monetary values below are USD.

Max participation	Filled	Fill fraction	Remainder	Fill VWAP (USD)
25%	4,775.13	95.50%	224.87	114.549
10%	2,002.83	40.06%	2,997.17	107.736
5%	1,001.42	20.03%	3,998.58	105.153
1%	200.28	4.01%	4,799.72	102.627
0%	0	0%	5,000	Undefined

Max participation	Unfilled opportunity cost	Total IS
25%	4,392.17	74,137.52
10%	27,824.21	43,426.01
5%	23,356.42	28,569.15
1%	15,168.42	15,704.79
0%	12,500.00	12,500.00

At zero participation, the engine reports no fill VWAP and charges no commission. Total IS is not monotone in this table because the constructed terminal mark and fill prices determine the opportunity and execution terms; it should not be read as a universal benefit from lower participation.

5. Walk-forward results

The constructed portfolio is evaluated over all 96 monthly returns, with the first 60 returns used for training and the last 36 for testing. Each cell is CAGR / annualized Sharpe.

Track	All 96 months	Train 60 months	Test 36 months
Gross	-0.26% / -0.03	0.85% / 0.20	-2.09% / -0.34
TWAP	-0.73% / -0.12	0.48% / 0.12	-2.72% / -0.44
Forecast VWAP	-0.71% / -0.11	0.51% / 0.13	-2.71% / -0.44
Lagged POV	-4.99% / -0.94	-4.00% / -0.80	-6.62% / -1.12

Gross performance is already negative over the full path and test window. These values therefore do not show positive gross alpha being erased by costs. The point of the exercise is the accounting and evaluation workflow, not a real-strategy conclusion.

6. Constructed high-friction scenario

The high-friction configuration increases spread, commission, temporary and permanent impact, borrow rate, and lowers the participation cap. In this one constructed path, gross CAGR / Sharpe are -0.26% / -0.03 and net CAGR / Sharpe are -8.01% / -1.54. The difference is a scenario comparison, not evidence about whether actual alpha survives trading costs.

7. Sensitivity scenarios

The commission grid is 0, 2.5, 5, 10, and 20 bps; the maximum participation grid is 0.1%, 0.25%, 0.5%, and 1%. The reported response variable is constructed net performance. At 0.25% participation, selected fee points are:

Commission	Net CAGR	Net Sharpe	Annual filled one-sided risky turnover
0 bps	-2.19%	-0.37	4.83x
5 bps	-2.66%	-0.46	4.83x
20 bps	-4.07%	-0.71	4.84x

The target-weight-scale sweep also changes requested turnover and gross exposure:

Target scale	Net CAGR	Net Sharpe	Annual filled one-sided risky turnover
0.5x	1.00%	0.37	2.40x
1.0x	-2.66%	-0.46	4.83x
2.0x	-10.75%	-0.95	9.84x

These are joint constructed scenarios. The scale experiment does not identify a pure turnover effect, and observed differences across participation caps are conditional on this path and the engine's `fill/mark` conventions.

8. Cumulative cost ablation

The five stages are a sequential 24-month constructed path; they are not independent causal estimates of each cost component.

Stage	CAGR	Sharpe	Max drawdown
Gross	-2.38%	-0.42	7.16%
Plus spread	-2.62%	-0.46	7.41%
Plus temporary impact	-4.53%	-0.81	10.30%
Plus commissions	-4.99%	-0.89	11.04%
Full net with cash interest and borrow	-2.66%	-0.46	7.49%

The final step includes modeled cash interest (4% credit on positive cash; 6% debit on negative cash) and borrow fees. Its rebound is path-dependent and does not show that borrow fees improve performance.

9. Ledger and audit evidence

The R2R6 rerun summary includes balance-sheet reconciliation, monthly returns and turnover, a segment-level financing audit, and exported mark events. All performance tables and figures below are sourced from `outputs/a7b_rev2r6_20261009/summary_results.json`; earlier rev2, R2R3, R2R4, R2R5 and v8 outputs remain separate prior-run baselines. `independent_recalc.json` hashes the summary, monthly ledger, financing segments, marks, execution clock, and config used for its checks. Each financing interval is independently reconstructed from prior cash, prior short positions, the latest exported mark available at the interval start, rates, `ACT/360` elapsed days, and event order. The R2R6 export contains 7,574 financing intervals, 1,434,399 mark events, and 753,285 execution-clock rows; 231,780 order slots had no fill and still receive an available slot-end mark. The new clock export records both `slot_start` and `slot_fill_event_at`. This generated path has zero overlapping windows; overlap and equal-time sizing are tested in the execution engine for POV, TWAP and VWAP.

The R2R6 rerun leaves the two-decimal performance tables above unchanged from R2R5; those tables also matched R2R4. The following table preserves the historical rev2-to-rev2r2 event-mark correction comparison; R2R6 repeats the corrected event path and confirms those values from its own exported intervals:

Strategy	Prior rev2 interest (USD)	Corrected rev2r2 / R2R3 interest (USD)	Prior rev2 borrow (USD)	Corrected rev2r2 / R2R3 borrow (USD)
TWAP	314,756.010458	314,756.011551	123,797.835654	123,797.835649
Forecast VWAP	315,186.047455	315,186.048535	123,963.165790	123,963.165868
Lagged POV	264,217.869948	264,217.895606	104,334.788931	104,334.721055

The old rev2 export had 3,816 financing intervals and no separate mark-event file. The corrected rev2r2 model added the full event-granular intervals and mark events; the R2R6 rerun confirms these interval counts: TWAP 2,802, VWAP 2,802, POV 1,650, plus 320 gross intervals without financing exposure. The input universe, seed, decision rules, and 96-month evaluation window are unchanged. R2R6 regenerates the complete result set and extends the execution clock schema; its constructed CLI schedule has no overlaps, so summary metrics remain the same.

10. Current-signal refresh (2026-10-14) and I3 Notebook default behavior

The current-signal refresh summary is at `build/batches/INTEGRATION-PREP/p04_cost_refresh_20261014/runs/current_signal_01/cli/summary_results.json`, relative to the repository root. The I3 reader candidate carries the same bytes at `fixtures/projects/project04/summary_results_current_signal_20261014.json`. This is a constructed engineering-consumer refresh based on the accepted current signal layer. The old A7-B R2R6 output and Notebook identities were not rewritten. The current summary also uses the cost configuration's 97 input months, 96 return months, and 60/36 split, but it must be cited as its own result identity rather than merged with old R2R6 results.

Each cell below is CAGR / annualized Sharpe:

Track	All 96 months	Train 60 months	Test 36 months
GROSS	-0.2316% / -0.0232	0.8176% / 0.2022	-1.9560% / -0.3334
TWAP	-0.4592% / -0.0688	0.6902% / 0.1735	-2.3459% / -0.4014
VWAP	-0.4333% / -0.0636	0.7240% / 0.1810	-2.3327% / -0.3994
POV	-4.4919% / -0.8849	-3.5313% / -0.7453	-6.0717% / -1.0800

In the current refresh's high-friction scenario, GROSS CAGR / Sharpe are -0.2316% / -0.0232 and net CAGR / Sharpe are -7.4392% / -1.5050; net CAGR is negative.

Selected current sensitivity points at a 0.25% participation cap are:

Commission	Net CAGR	Net Sharpe	Annual filled one-sided risky turnover
0 bps	-1.9791%	-0.3563	4.6352x
5 bps	-2.4334%	-0.4428	4.6368x
20 bps	-3.7873%	-0.6976	4.6418x

The current target-scale scenarios are:

Target scale	Net CAGR	Net Sharpe	Annual filled one-sided risky turnover
0.5x	1.0917%	0.4286	2.3072x
1.0x	-2.4346%	-0.4430	4.6368x
2.0x	-10.1724%	-0.9546	9.4117x

The current summary's 24-month ablation is:

Stage	CAGR	Sharpe	Max drawdown
Gross	-2.4271%	-0.4531	6.9367%
Plus spread	-2.6540%	-0.4974	7.1978%
Plus impact	-4.4075%	-0.8334	9.9626%
Plus commissions	-4.8509%	-0.9178	10.6818%
Full net with cash interest and borrow	-2.4346%	-0.4430	7.0912%

The current sensitivity summary contains both positive and negative values. The 0.5x target-scale row is positive, but no strategy or parameter winner was selected. Scaling changes requested turnover and gross exposure together, so it remains a joint scenario.

The I3 reader Notebook is at `projects/project04/cost_execution_lab/cost_execution_lab.ipynb`. Its default run recomputes the frozen U-shaped profile, the 10,000-share fixed-parent comparison, and the 5,000-share liquidity-stress example, then checks their corresponding values against the saved current summary. It replays the 96-month walk-forward results, high-friction scenario, sensitivities, ablation, and ledger audits from that saved summary; it does not rerun those longer studies by default. Running the Notebook creates a unique run directory at repository-root `runs/` and writes images. These are not new experiments or market-data verification in this author batch.

11. Reproduction and handoff boundary

This report is one of the four bilingual cost-reading documents, accompanied by two Notebook language projections. Source code, configuration, frozen R2R6 evidence, the current-signal summary, and the I3 reader Notebook remain at their versioned paths relative to the full repository root; these documents do not copy the complete project into a portable software package. R2R6 evidence is under `qa/a7/ch35_ch36_batch/independent_acceptance/rev2/r2r6/candidate/projects/project04/cost_execution_lab/outputs/a7b_rev2r6_20261009/`. The current-signal refresh remains a constructed engineering consumer, not verification of real costs or capacity; the full real-data Project 4 strategy has not been accepted. The I3 Notebook's computation and replay boundary is described in Section 10.

Project 4 Cost & Execution Lab: Reference Solutions

Course Module: Project 4 Multi-Factor Strategy Execution Lab

Contract Reference: `qa/a7/ch35_ch36_batch/INTERFACE_CONTRACT.md`

Purpose: Complete reference solutions to pedagogical exercises in transaction-cost accounting and execution algorithms. All Project 4 performance examples are constructed scenarios, not empirical alpha evidence.

> Result identity: Exercise 6 and its historical metrics retain the original R2R6 old-signal values. They are not new market results from the 2026-10-14 current-signal summary. The accounting and timing tests discussed in Exercises 3–4 also cannot establish correct fee semantics from an IS identity alone.

Exercise 1: Algebraic Proof & Verification of Implementation Shortfall Direct Identity

Problem Statement

Given a signed parent order Q, filled child quantities f_i summing to $F = \sum_i f_i$, unexecuted remainder $R = Q - F$, decision price P_{decision}, arrival price P_{arrival}, terminal mark price P_{terminal}, and explicit fees $\text{Fees} = \sum_i c_i$:

1. Prove algebraically that the 4-part Implementation Shortfall decomposition is identically equal to the direct amount method.
2. What is the value of realized fill VWAP when $F = 0$? How does the decomposition evaluate under zero fill?

Solution

1. Algebraic Derivation

These formulas apply to one parent order. Each nonzero child fill must have the parent order's direction, and quantity conservation $Q = F + R$ must hold. Do not combine offsetting fills across parent orders and treat them as one valid case. The engine's `compute_is_decomposition` checks that domain. Recall the definitions from `INTERFACE_CONTRACT.md`:

$$\begin{aligned}\mathrm{IS}_{\mathrm{decomposition}} &= \underbrace{Q(P_{\mathrm{arrival}} - P_{\mathrm{decision}})}_{\text{Delay Cost}} \\ &+ \underbrace{\sum_i f_i(P_{\mathrm{fill},i} - P_{\mathrm{arrival}})}_{\text{Realized Execution Cost}} \\ &+ \underbrace{R(P_{\mathrm{terminal}} - P_{\mathrm{arrival}})}_{\text{Unfilled Opportunity Cost}} \\ &+ \mathrm{Fees}\end{aligned}$$

$$\mathrm{IS}_{\mathrm{direct}} = \sum_i f_i(P_{\mathrm{fill},i} - P_{\mathrm{decision}}) + R(P_{\mathrm{terminal}} - P_{\mathrm{decision}}) + \mathrm{Fees}$$

To prove equality, expand each price difference in $\mathrm{IS}_{\mathrm{direct}}$ relative to P_{arrival}:

$$P_{\mathrm{fill},i} - P_{\mathrm{decision}} = (P_{\mathrm{fill},i} - P_{\mathrm{arrival}}) + (P_{\mathrm{arrival}} - P_{\mathrm{decision}})$$

$$P_{\mathrm{terminal}} - P_{\mathrm{decision}} = (P_{\mathrm{terminal}} - P_{\mathrm{arrival}}) + (P_{\mathrm{arrival}} - P_{\mathrm{decision}})$$

Substitute these into $\mathrm{IS}_{\mathrm{direct}}$:

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= \sum_i f_i\Big[(P_{\mathrm{fill},i} - P_{\mathrm{arrival}}) + (P_{\mathrm{arrival}} - P_{\mathrm{decision}})\Big] \\ &+ R\Big[(P_{\mathrm{terminal}} - P_{\mathrm{arrival}}) + (P_{\mathrm{arrival}} - P_{\mathrm{decision}})\Big] \\ &+ \mathrm{Fees}\end{aligned}$$

Group terms:

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= \sum_i f_i(P_{\mathrm{fill},i} - P_{\mathrm{arrival}}) + R(P_{\mathrm{terminal}} - P_{\mathrm{arrival}}) \\ &+ \left(\sum_i f_i + R\right)(P_{\mathrm{arrival}} - P_{\mathrm{decision}}) + \mathrm{Fees}\end{aligned}$$

By definition of the parent order invariant, $F = \sum_i f_i$ and $Q = F + R$, hence:

$$\sum_i f_i + R = Q$$

Substituting Q yields:

$$\begin{aligned}\mathrm{IS}_{\mathrm{direct}} &= \sum_i f_i(P_{\mathrm{fill},i} - P_{\mathrm{arrival}}) + R(P_{\mathrm{terminal}} - P_{\mathrm{arrival}}) \\ &+ Q(P_{\mathrm{arrival}} - P_{\mathrm{decision}}) + \mathrm{Fees} \equiv \mathrm{IS}_{\mathrm{decomposition}}\end{aligned}$$

Q.E.D.

2. Zero-Fill Boundary ($F = 0$)

When $F = 0$, no shares are executed:

- **Fill VWAP**: The formula $P_{\text{fill_VWAP}} = \frac{\sum f_i P_{\text{fill},i}}{F}$ has zero denominator and is **mathematically undefined** (`None` or `NaN`). It must never be recorded as 0.0.
- **Realized Execution Cost**: $\sum_i f_i(P_{\text{fill},i} - P_{\text{arrival}}) = 0$.
- **Remainder**: $R = Q$.
- **Explicit Fees**: Fees = 0 (no commissions on zero fill).
- **Total IS**:

$$\text{IS} = Q(P_{\text{arrival}} - P_{\text{decision}}) + R(P_{\text{terminal}} - P_{\text{arrival}}) = Q(P_{\text{terminal}} - P_{\text{decision}})$$

The identity holds seamlessly. It checks the arithmetic of the two IS calculations; if a duplicated fee is inserted into both fee totals, the difference may still be zero. Check the fee and cash-flow semantics separately.

Exercise 2: Participation Rate Boundary & Denominator Conversion

Problem Statement

A quantitative trader configures a participation limit of $\rho = 20\%$.

1. If ρ is defined with denominator excluding own fills (exclusive rate: own_fill/external_vol), what fraction of the total consolidated tape does the trader execute?
2. If the exchange reports an inclusive rate of $\rho_{\text{incl}} = 20\%$ (denominator includes own fills: own_fill/(external_vol + own_fill)), convert this to the equivalent exclusive rate.
3. State the boundary conversions at $\rho = 0$ and $\rho = 1$.

Solution

1. Exclusive to Inclusive Conversion

Let f = own_fill and V = external_volume.
Exclusive rate: $\rho_{\text{excl}} = f/V = 0.20 \implies f = 0.20V$.
Inclusive rate:

$$\rho_{\text{incl}} = \frac{f}{V+f} = \frac{0.20V}{V+0.20V} = \frac{0.20}{1.20} = \frac{1}{6} \approx 16.67\%$$

The trader executes 16.67% of total combined tape.

2. Inclusive to Exclusive Conversion

Given $\rho_{\text{incl}} = \frac{f}{V+f} = 0.20$:

$$f = 0.20(V+f) \implies 0.80f = 0.20V \implies \rho_{\text{excl}} = \frac{f}{V} = \frac{0.20}{0.80} = 0.25 = 25.0\%$$

3. Boundary Conditions

- **At $\rho = 0.0$:** Both rates equal 0.0. Max fill capacity is 0 shares.
- **At $\rho_{\text{excl}} = 1.0$:** own_fill $= 1.0 \times$ external_vol, so $\rho_{\text{incl}} = 1.0/(1.0 + 1.0) = 0.50$ (50% of total tape).
- **At $\rho_{\text{incl}} = 1.0$:** $\rho_{\text{excl}} = \rho_{\text{incl}}/(1 - \rho_{\text{incl}})$ tends to infinity, so the capacity helper returns no upper bound. This mathematical boundary does not guarantee an actual fill. The project configuration and `simulate_slot_fill` use the exclusive denominator; do not treat an inclusive 20% limit as the configured 20% exclusive limit.

Exercise 3: Cash Ledger Conservation & Duplicate Fee Prevention

Problem Statement

All monetary amounts in this exercise are USD. A long order of 1,000 shares is executed at mid price $S = 100.00$, with half-spread of 0.03 per share, temporary impact of 0.02 per share, permanent impact of 0.01 per share, and a 5 bps commission rate.

1. Compute the realized fill price P_{fill} and the cash deducted upon fill execution.
2. Explain why deducting the spread (30) or temporary impact (20) a second time from cash constitutes a fatal accounting error.
3. How does permanent impact enter portfolio NAV?

Solution

1. Fill Price and Cash Ledger Entry

- Half-spread: 0.03
- Temporary impact: 0.02
- Fill price:

$$P_{\text{fill}} = S + \text{sgn}(f) \cdot (\text{half_spread} + \text{temp_impact}) = 100.00 + (0.03 + 0.02) = 100.05/\text{share}$$

- Gross trade value: $1,000 \times 100.05 = 100,050.00$.
- Commission (5 bps): $100,050.00 \times 0.0005 = 50.025$.
- Cash debit:

$$\text{Cash_After} = \text{Cash_Before} - (1,000 \times 100.05) - 50.025 = \text{Cash_Before} - 100,100.025$$

2. Duplicate Fee Prevention

Because the fill price $P_{\text{fill}} = 100.05$ already embeds the half-spread (0.03) and temporary impact (0.02), the trader pays the extra 50.00 $(1,000 \times 0.05)$ through the price of the purchased shares. If the ledger also subtracts 50.00 as a cash fee while the explicit commission field still contains only the actual 50.025 commission, cash no longer matches the ledger equation:

$$\Delta\text{Cash}_{\text{erroneous}} = -(100.05 \times 1,000) - 50.025 - 50.00 = -100,150.025$$

The extra 50.00 is a cash residual. However, if an erroneous duplicate is also entered as a second fee in both the ledger and both IS calculations, the accounting checks and the `direct/decomposed` IS identity may still agree. The identity alone does not prove that the fee was counted only once.

3. Permanent Impact Treatment

Permanent impact (0.01) shifts the subsequent market equilibrium mid price:

$$S_{\text{new}} = 100.00 + 0.01 = 100.01$$

This enters portfolio valuation naturally when positions are marked to market $(1,000 \times 100.01)$ at subsequent closes. It is never charged as an explicit cash fee.

Exercise 4: Causal Volume Leakage Defense via Adversarial Perturbation

Problem Statement

Describe an adversarial unit test that checks a VWAP or POV execution algorithm for future-volume leakage.

Solution

Methodology

1. Create chronologically ordered execution slots $S = [s_0, s_1, \ldots, s_{K-1}]$. Each slot has a frozen forecast volume $V_{\text{forecast},k}$, lagged observed volume $V_{\text{observed},k}$, realized volume $V_{\text{realized},k}$, and `fill/confirmation` time.
2. Schedule parent order Q and save the child request at each send time as C_{base}.
3. Perturb only realized volume for the current or future slots, for example by multiplying it by 100 or setting it to zero. Keep the information available at the send being tested identical: frozen forecasts, lagged observations, fills already confirmed strictly before that send, and reservations for unresolved earlier children.
4. Compare the child request at each tested send.
5. **Assertion:** With the send-time information set held fixed, changing realized volume that is not yet visible cannot change the child request at that send. VWAP requests use the frozen forecast curve; POV requests use the lagged observed volume. Realized volume can constrain the later fill, not the earlier request.

A changed fill in an earlier slot may legitimately change a later child request if that fill has been confirmed before the later send, because the remaining parent quantity has changed. A fill confirmed at the same timestamp as a send is unavailable to that send in this engine. Therefore, do not require all later requests to match pathwise when the perturbation changes fills that have become known by then; compare under the same information set at each send.

The existing tests are:

```
tests/test_execution_costs.py::test_vwap_schedule_adversarial_perturbation_invariance
tests/test_execution_costs.py::test_pov_schedule_lagged_causality
```

The VWAP test covers invariance of the frozen schedule to realized-volume perturbation. The POV test verifies that POV sends zero in slot 0 when no lagged volume is observed and that changing slot 1 realized volume leaves its request unchanged when cumulative fills are held at zero. These tests do not guarantee pathwise equality of later requests when confirmed past fills differ and change the remaining parent quantity.

Exercise 5: Trade-Off Analysis of Execution Schedulers

Question

Why did VWAP outperform TWAP in Total Implementation Shortfall, while POV severely lagged in the fixed-parent-order experiment?

Solution

1. **VWAP vs TWAP**: In this constructed experiment, the external-volume path uses a U-shaped intraday profile. Because TWAP trades constant quantities per slot ($Q/13$), its participation rate is higher in the modeled low-volume midday slots. Forecast VWAP follows the pre-frozen constructed curve. For the 10,000-share buy order, VWAP IS is 541.70 bps versus TWAP 552.74 bps; for the sell order the values are 248.95 and 257.01 bps. The result illustrates this path and does not establish a general empirical ranking.
2. **POV difference**: POV sizes each child from *lagged volume already observed* at the send. Slot 0 has no earlier observed volume, so it sends 0 shares. Later child requests are bounded by the remaining parent quantity and the configured participation rate times lagged external volume. A fixed POV schedule does not roll an unfilled amount into larger later orders or raise its participation rate to catch up. If later observed volume is low or realized capacity binds, remainder R can remain at cutoff and incur opportunity cost.

Exercise 6: Turnover, Fees, and Net Results in the Constructed Path

Question

In the old-signal R2R6 constructed walk-forward run, filled one-sided annual turnover is about 5.2383x and filled total-notional turnover is about 10.4767x NAV. Commission is 5 bps and half-spread is 2.5 bps per traded dollar. Estimate the mechanical fee-and-spread drag, then compare it with the `gross/TWAP` CAGR difference and explain why those values need not match.

Solution

The old-signal R2R6 rerun reports annual filled one-sided risky turnover $\tau_{one} = 5.2383$ and total risky-notional turnover $V/NAV = 10.4767$ for TWAP, approximately $2\tau_{one}$ in this constructed long-short path. Filled dollar notional uses actual execution prices, while requested notional uses the decision price; this small difference does not imply a share overfill. Applying the configured proportional fee rates to filled risky notional gives a simple pre-impact estimate:

$$\text{Commission} = 10.4767 \times 5 \text{ bps} = 52.4 \text{ bps}, \qquad \text{Half-spread} = 10.4767 \times 2.5 \text{ bps} = 26.2 \text{ bps}.$$

The combined fee-and-spread estimate is about 78.6 annualized bps under the turnover metric's monthly rebalance-NAV normalization, before impact and financing. In this old-signal corrected constructed path, gross CAGR is -0.26% and TWAP CAGR is -0.73%, a difference of about -46.9 bps. The two numbers need not match: CAGR compounds a monthly path, while the simple estimate omits `temporary/permanent` impact, cash interest, borrow, and changes in the sequence of marked portfolio values. Gross is already negative, so this example does not show positive alpha being erased by costs.

The 2026-10-14 current-signal refresh is a separate saved summary identity. Its results are documented in the research report; do not substitute those values for the R2R6 figures used in this exercise.

Project 4: New Integrative Teaching Exercises

This constructed, integrative teaching expansion was added in BIL15B and uses independent P4-Txx identifiers. It is not a historical P4 assignment or new real-market research. The original cost Exercises 1–6 and their complete solutions retain their original identities. This set's complete solutions are listed separately in the same project. Read the reader guide, Chapters 29–31, and the cost materials first.

Shared inputs and simplifications

The three shared price-observation dates are D0=2026-01-30, D1=2026-02-02, and D2=2026-02-27. The decision is made at the D0 close; lag=1 executes at the next price-index observation, D1. The 60 constructed securities are S01..S60: the first 30 are in industry G1 and the remaining 30 in G2. In each industry, the first 15 securities have a processed score of +1 and the remaining 15 have −1; the common size exposure is set to 0. The scores are supplied, already-processed teaching inputs: do not re-estimate the seven-field alpha or present them as the original 240-company × 96-month DGP. All securities are listed at D0 and have complete scores, so the support threshold is at least 60. Industry demeaning and gross-2 rescaling are the portfolio steps in this exercise; other real-research gates must be checked separately.

For the version-timing example, S01 has two versions for the same reporting period. Version v1 is published at D0 14:00 and ingested at 14:05, with profit `10/equity` 100. Version v2 is published at D0 16:30 and ingested at D1 09:45, with profit `5/equity` 50. Both ROE values happen to be 10%; versions must not be spliced.

S60 delists on D1 with liquidation at 2 per share; S02 delists on D2 with liquidation at 2 per share. All other securities survive throughout. Initial `NAV/cash` is 1000 USD with no holdings. All execution prices are 10 USD; at the D1 close S01 is 11 and the other surviving securities are 10; closing prices remain unchanged on D2. S60 has no valid quote from D1 onward, and S02 has no valid quote from D2 onward; these are represented as NaN and accompanied by explicit delisting events. Assume no interest, dividends, splits, impact, borrowing fees, lot-size constraints, or margin calls. The fee rate is 0.001 on total traded notional. The base engine must be called with `allow_short`=True and `max_gross`=2; this does not establish that real borrowing is available.

P4-T01: Versions, Support, and Signals

1. Which version is available at the D0 close? Can v2 be used at the D1 close? What if the decision is made before 09:30 on D1? Explain why 5/100 is invalid.
2. Use the generation rule to calculate the mean score in each industry, the 60-name support, each security's weight after rescaling to gross 2, the portfolio net weight, and each industry's net weight.
3. At D1, only 59 securities remain. Can a new signal that passes the ≥60-name gate be generated? How should S60's weight in the original D0 signal be retained? Distinguish the signal available at that time from execution eligibility.

P4-T02: Execution Copy, Post-Fee NAV, and Realized Holdings

1. At execution on D1, set S60 to 0 without rescaling the other 59 weights. Calculate the execution copy's gross and net exposure and cash target. Explain what happens if the base `run_backtest` receives the original nonzero target for a security already delisted on the execution date.
2. Solve for post-fee NAV V', the fee, shares in each surviving security, and cash. Give D1 closing NAV and S01's realized closing weight. Why is it not equal to 1/30?
3. On D2, how are the existing S02 shares liquidated? Find the liquidation cash, price loss, and final NAV. Explain why S60's and S02's delistings cannot both be handled by simply "dropping the column and ignoring the loss." Do not overwrite the D0 signal with the execution copy or restore `gross/neutrality.`
4. Call the base ledger on this small input to check shares, fees, cash, NAV, and reconciliation; do not run the full `P4/A6` CLI, Notebook, or historical research. The answer gives an independently checkable table and entry-point code.

P4-T03: Label Availability, Forward Selection, and Result Identity

This is a separate, small label timeline. Do not describe its few samples as passing the 60-name portfolio gate. Sample start positions are i=0..14, with label intervals (i,i+2] and `a_i`=i+2, except `a_4`=7. Fitting and prediction take place at 6, with test start positions 6..8. At a shared endpoint, settle and ingest first, then fit and predict.

1. Which of the candidate historical rows 0..5 can train at the fit time? Do any overlap test intervals? Is there an embargo under the chapter's post-test definition?
2. Inner validation A: at 2, train on the available history and validate on sample 2. Inner validation B: train at 3 and validate on sample 3. List the training rows for each. Given validation labels (1,−1), candidate predictions A=(0.5,−0.5) and B=(1,0), calculate the MSE in both folds and select a parameter. Which rows are used to fit scaling?
3. Run the outer test only for selected candidate A. The labels are (0.2,−0.2,0), and predictions are (0.1,0.1,0.1); find the MSE. Given the complete family's valid nominal p-values (0.04,0.20), apply `Bonferroni/Holm` at the 5% level. Does selecting A establish tradable alpha?
4. Compare this new exercise with the old R2R6 results, the 2026-10-14 current-signal cost summary, the I3 default Notebook's fresh `computation/replay`, and real research, which is still planned. If holdout performance is negative or one algorithm performs worse, what must be preserved? Which teaching, consumer, assembly, and data-rights gates remain?

Project 4: Complete Solutions to the New Integrative Teaching Exercises

These solutions correspond to the new P4-T01–T03 set in this batch. They use stated assumptions for constructed teaching examples; the values were checked with Fraction and a minimal 3-date × 60-security ledger. The seven-field alpha was not re-estimated, and no historical DGP or cost study was replayed. The original six cost exercises have separate solutions; their IDs and evidence are not merged here.

P4-T01: Complete Solution

Version clock. At the D0 close, only v1 is available because its publication and ingestion are both complete. Version v2 has a=max(publication, ingestion)=D1 09:45, so it is available at the D1 close, but not for an order before 09:30 on D1. Combining v2 profit with v1 equity gives the invalid ratio 5/100=5%; each internally consistent ratio, 10/100 or 5/50, is 10%. The fact that the values happen to match does not validate the version join.

Support and targets. Each industry has 15 scores of +1 and 15 of −1, so its mean score is 0 and demeaning leaves the scores unchanged. The absolute scores sum to 60. The rescaling factor to gross 2 is 2/60=1/30. Each target weight is ±1/30; portfolio net is 0, gross is 2, and each industry's net weight is 0. This checks only the portfolio step for supplied, already-processed scores; it does not claim market validation of the real seven fields, ingestion quality, all OLS exposures, or industry-coverage gates.

At D1, support is 59<60, so a new signal cannot be said to pass this gate. Preserve the D0 signal as it was. S60's −1/30 remains a valid target at that earlier time; at execution, separately set it to 0 based on its then-current eligibility. The executed ledger may no longer be neutral or have gross 2. Do not present execution adjustment as a historical alpha correction.

P4-T02: Complete Solution

1. Execution copy and fee denominator

After removing S60's short target, there are 30 longs and 29 shorts: gross=59/30, net=1/30, and the cash target is 1−net=29/30. The base `run_backtest` raises an error if given a nonzero target for a security already delisted on the execution date. The `wrapper/execution` layer must explicitly create an eligibility copy while preserving the original signal. Do not rescale or re-neutralize this copy.

Mark the uninvested account at E=10, giving Vpre=1000. Total new traded notional is (59/30)V′, so

$$V' = 1000 - 0.001(59/30)V' = \frac{30{,}000{,}000}{30{,}059} = 998.0371935194.$$

The fee is 59000/30059=1.9628064806 USD. Each surviving security's shares are ±V′/300=±3.3267906451, and cash is (29/30)V′=29000000/30059=964.7692870688 USD. The

base engine credits short-sale proceeds in full to cash; this is a simplification, not a model of borrow or segregated collateral. See 29-C02 for the complete extension.

2. Full state check table

State	Shares, cash, or amount (USD)	NAV (USD)
Initial, D0	Cash 1000; all shares 0	1000
After D1 execution	30 `longs/29` shorts, each $\pm V'/300$ shares; cash $29V'/30$; fee $1000-V'$	998.0371935194
D1 close	S01 price 10→11, gain $V'/300$; other surviving prices unchanged	$301V'/300$=1001.3639841645
After D2 liquidation	S02 long position recovers $2V'/300$= 6.6535812901; price loss $8V'/300$= 26.6143251605	$293V'/300$=974.7496590040

S01's realized D1 closing weight is $(11V'/300)/(301V'/300)=11/301$=3.65448505%, while its execution target is 1/30=3.33333333%. The difference comes from price movement after execution and the closing NAV denominator; the target is hit against post-fee NAV at execution, so the closing weight need not remain unchanged.

S60 was already delisted before its initial position was opened, so the account never held it and has no liquidation P&L for S60; it cannot be sold short. S02, however, is already held and must be taken to zero shares on D2 in return for the explicit 2-per-share liquidation value. Do not erase the prior loss by deleting the historical column. D2 cash is 964.7692870688+6.6535812901; risk-asset value falls by 33.2679064506, giving a net loss of 26.6143251605.

3. Minimal executable check

```
import numpy as np
import pandas as pd
from quant_course.backtest import BacktestConfig, run_backtest, reconcile

assets = [f"S{i:02d}" for i in range(1, 61)]
score = np.array([1 if (i-1) % 30 < 15 else -1 for i in range(1, 61)])
d = pd.to_datetime(["2026-01-30", "2026-02-02", "2026-02-27"])
w = pd.DataFrame([score / 30], index=[d[0]], columns=assets)
execution_copy = w.copy(); execution_copy.loc[d[0], "S60"] = 0
p = pd.DataFrame(10., index=d, columns=assets)
p.loc[d[1]:, "S01"] = 11.; p.loc[d[1]:, "S60"] = np.nan
p.loc[d[2], "S02"] = np.nan
e = p.copy(); e.loc[d[1], "S01"] = 10.
exits = {"S60": (d[1], 2.), "S02": (d[2], 2.)}
cfg = BacktestConfig(initial_cash=1000., fee_rate=.001, lag=1,
                     allow_short=True, max_gross=2.)
r = run_backtest(p, execution_copy, cfg, exec_prices=e, delistings=exits)
v = 30000000 / 30059
assert np.isclose(r.ledger.iloc[1]["fees"], 1000-v, rtol=0, atol=1e-9)
assert np.isclose(r.ledger.iloc[1]["cash_end"], v*29/30, rtol=0, atol=1e-9)
assert np.isclose(r.positions.loc[d[1], "S01"], v/300, rtol=0, atol=1e-9)
assert r.positions.loc[d[2], "S02"] == 0
```

```
assert np.isclose(r.ledger.iloc[2]["nav"], v*293/300, rtol=0, atol=1e-9)
assert reconcile(r, p, cfg, exec_prices=e, delistings=exits)["residual"].abs().max()
    ↪ < 1e-9
```

The absolute 1e−9 threshold is a currency check for this USD 1000 account, not a general floating-point precision guarantee for all account sizes. Reconciliation checks internal accounting only.

P4-T03: Complete Solution

Eligibility and intervals. At fit=6, candidates with a≤6 are 0, 1, 2, and 3. Sample 4 has e4=6 but a4=7, so it is not available; sample 5 has e5=a5=7 and is also unavailable. The latest retained label ends at 5, while testing begins at (6,8], so the intervals do not intersect. A purely historical candidate set has no post-test embargo; this does not rule out other historical dependence gaps.

Two genuine inner validations. Fitting at 2, sample 0 has e=a=2 and is available, while sample 1 is not available until a=3. Thus train on {0} and validate on {2}. Fitting at 3, train on {0,1} and validate on {3}. Fit scaling on each fold's training rows only; after selecting a parameter, refit preprocessing and the model on the outer-available rows {0,1,2,3}, then evaluate outer rows 6..8. The predictions are supplied for teaching and do not claim a stable model estimated from these tiny samples.

Candidate	Squared error at validation 2	Squared error at validation 3	Mean across folds
A	$(1-0.5)^2=0.25$	$(-1+0.5)^2=0.25$	0.25
B	$(1-1)^2=0$	$(-1-0)^2=1$	0.50

The inner folds select A. The outer MSE is

$$\frac{(0.2-0.1)^2+(-0.2-0.1)^2+(0-0.1)^2}{3}=\frac{11}{300}\approx 0.03666667.$$

Do not use the outer result to switch the choice to B.

For valid nominal p-values of 0.04 and 0.20, Bonferroni-adjusted values are 0.08 and 0.40; Holm-adjusted values are 0.08 and 0.20. The first Holm step has $0.04>0.05/2$, so stop without rejecting; neither result rejects at the 5% family-wise level. A better MSE, being selected, and having tradable alpha are three different things. A p-value must itself be valid; this MSE table does not generate one, and model correlation is no reason to shrink the family size arbitrarily.

Evidence ledger. The new `60-name/3-date` exercise and small label example are BIL15B teaching additions, not a new study on the original 240-company × 96-month signal world or the `97-input/96-return-month` cost world. Keep the old R2R6 full-period GROSS CAGR −0.26% and TWAP −0.73% separate from the 2026-10-14 refresh's full-period GROSS −0.2316% and TWAP −0.4592%. Preserve `algorithm/scenario` negative and zero results; do not describe an already-negative gross return as "positive alpha erased by fees." In the I3 cost Notebook, fixed-parent-order and `partial/zero-fill` examples are computed afresh while other displays replay the saved summary; the Chapter 31 default recomputes a one-seed constructed case. None of these is a rerun of historical results for this batch.

The root may register complete P4 teaching only after accepting B14 signal, A `ledger/cost`, and B validation integration. BIL17 consumer review and BIL23 chapter-level English Notebooks, full-package assembly, cold start, and global references remain pending. Real P4 market research remains planned; this exercise does not resolve PIT data, data rights, real membership, tradable prices, `financing/borrowing`, capacity, or calendar requirements. `G05/A10-B` has not been authorized. Report any negative holdout result. If the model is changed after looking at holdout data, that holdout has become discovery data and cannot again be called untouched test data.

Project 5 —Pairs Trading

Project 5 — Pairs Trading

The reader's route and the identity of the evidence

This is the BIL17B candidate for complete bilingual teaching, pending independent acceptance. Real-market research remains planned and blocked; it is not verified. The implementation and constructed demonstrations have limited acceptance through c5, ALIGN, and the later whole-share execution boundary. Read the historical c5 acceptance, protocol-alignment acceptance, and current whole-share boundary acceptance in that order. The last supersedes ALIGN's earlier treatment of fractional actual holdings. Those historical records remain unchanged.

Start with stationarity and cointegration in Chapters 16–17, accounting and evaluation in Chapters 29–31, and the accepted material in Chapters 34–36. A quote is not a fill, and requested quantity Q is not actual filled quantity F. Reserve capacity for orders in flight and explain the disposition of terminal remainder $R = Q - F$. Only confirmed fills change cash and holdings; subsequent marks determine net asset value (NAV). Implementation shortfall (IS) attributes performance relative to a decision benchmark; it is not deducted from cash again.

Material	Data identity and purpose	What it cannot establish
A5 screening preparation	Revised, non-PIT, non-tradable diagnostics on real French aggregate industry returns, plus separate constructed counterexamples	Security-level pairs PnL or a borrowable trading strategy
A6 accounting interface	A separate constructed price pair and two-leg ledger, seed 20261207	A single price experiment combined with A5 or c5
c5 / ALIGN / v2 report	Three constructed pairs over 1,600 business days, seed 20261010; both old positive v1 and negative v2 results retained	Evidence about the eight real-security pairs; v2 was frozen after constructed results had been seen

Read the exercises and complete solutions, then consult the dictionary, report, and execution boundary. A5 and A6 are appendices about different layers, not parts of one experiment.

The default Notebook and a full research run

The accepted I3 package's `projects/project05/notebooks/p05_pairs.ipynb` explicitly defaults to constructed data. It freshly runs the three-pair `screened/ungated` main route, producing formation, ledger, and performance tables and one figure. It does not merely load an old cache, and it does not run the full CLI `cost/threshold` grids. Read explanatory prose, historical replay tables, and new constructed outputs according to their labels. The complete English Notebook language layer, portable package, and full cold-start validation belong to BIL23; this batch neither edits nor executes the Notebook.

From the configured I3 package root, run an execution copy through the outer runner:

```
.venv/bin/python run_notebook.py projects/project05/notebooks/p05_pairs.ipynb
```

The Notebook writes new outputs to a unique `RUN_DIR` under package-root `runs/`. The runner separately preserves the executed Notebook and `execution.json` in a unique `runs/execution_...` directory, including failures. The package reader contract describes supplied inputs and output boundaries; repository readers enter the package at `outputs/quant_course_reader_candidate_20261014`. `notebooks/build_notebook.py` rewrites the source Notebook and is a maintenance tool, not the ordinary reader entrypoint. Historical `CLI/test` commands below identify evidence; their presence does not mean they were rerun for this translation. Real data still require permission, field, PIT, and execution-authorization gates.

Research status: the real-market project remains planned. The following task contract is retained; constructed methods and results have their own acceptance identities.

> The A5 screening preparation layer has historical limited acceptance. See preparation/README.md. It covers training-only correlation and cointegration screening, hedge ratios, test counts and critical values, counterexamples, and descriptive diagnostics on French industry portfolios. It has no `entry/exit` rules, two-leg costs, financing, or backtest. Screening is not return evidence and does not complete real-market Project 5.

Research objective and data

Prerequisites: Chapters 16, 17, 29, 30, 31, 34, 35, and 36.

Obtain adjusted price histories for two or more real assets, validating calendars, corporate actions, and borrowing constraints. Retain the number of searches when selecting pairs at scale. Signals use total-return series; execution uses unadjusted prices with corporate actions accounted for separately.

Workflow

1. Screen correlation and test cointegration using training data only.
2. Estimate hedge ratios; construct the spread and training-standardized z-score.
3. Freeze `entry/exit`, risk controls, trade lag, and flat-state rules.
4. Account for both legs' costs, short financing, and mismatch risk.
5. Evaluate walk-forward performance and examine breakdowns and failure windows.

Timing and evaluation

Every input must identify `event_time`, `available_at`, `signal_time`, `trade_time`, return interval, and label interval where applicable. A closing signal executes at the next tradable instant. Fit and choose parameters in training; use test data for evaluation only. Document transaction, financing, and borrowing conventions in configuration.

Deliverables

- `data_manifest.json`: source, license boundary, sample, units, SHA-256, and data version.
- `data_dictionary.md`: fields, units, keys, timing, and missing-data rules.
- `notebooks/`: clean-kernel execution, reusing shared `src/` logic.
- `src/`, `tests/`, `configs/`: implementation, core tests, and frozen configuration.
- `figures/`: axis definitions and `data/code` versions.
- `research_report/`: hypotheses, methods, `training/test` design, metrics, risks, costs, robustness, limits, and conclusions.
- `solutions.md`: all ten exercises and complete answers, reference implementation, and historical checks. Constructed reference results do not constitute real-market acceptance.

Acceptance criteria

Correlation does not imply cointegration. Use the applicable residual-test critical values. Do not screen and evaluate on the same test interval. A hedge ratio is not a capital weight.

Report data limits and retain all zero and negative results. Tests, scripts, and clean Notebook execution must verify the implementation within their stated scope; teaching simulations cannot replace empirical findings.

> The A6 constructed accounting interface has historical limited acceptance: backtest_preparation/README.md. It demonstrates frozen training rules, two-leg execution, and reconciliation on explicitly constructed tradable prices. It does not use French industry portfolios as pairs-trading PnL; the real-market project remains planned.

Historical interface evidence: A6 acceptance. Its scope is constructed prices and self-financing accounting, not the complete project.

A7-C historical constructed method, 2026-10-10

c5 has limited acceptance; real research remains incomplete.

- **Real-price empirical research is blocked.** The 2026-10-10 source decision recorded licensing gaps in the options then checked; no applicable institutional authorization was obtained. This is neither a fresh review of current terms nor a universal claim about every free source. See the source decision. `data_manifest.json` remains blocked; `--data real` rejects execution with exit code 3.
- Preregistration v1: eight pairs, 2010–2025, rolling `504/126` windows, EG $p < 0.05$, $|z|$ thresholds `2/0.5/4`, and `cost/borrow` scenarios; frozen before any price download.
- Implementation: validation, forward total returns, formation and EG, signals, two-leg accounting, `financing/borrow/dividends/splits`, partial fills, IS, and walk-forward evaluation. Constructed panel; CLI; figures; tests.
- Report, exercises, solutions, reference code, historical source Notebook (readers execute the I3 package copy above), active practice, and dictionary.
- Constructed results demonstrate the method and implementation: cash residuals around 10^{-9} USD, equality of decomposed and direct IS, and formation estimates unchanged by future perturbations. The old positive screened Sharpe came from a false-positive selection and changed sign with a 252-day formation window. Even these constructed results do not satisfy the rule for supporting the strategy.

```
.venv/bin/python projects/project05/run_project05.py --data constructed --out <new-
    ↪ directory>
.venv/bin/python -m pytest projects/project05/tests/test_p05_pairs.py -q
```

c5 repairs and historical acceptance, 2026-10-10

c4 required revision; c5 addressed R01–R05 and introduced v2. Real data remain blocked, and constructed results are not market evidence.

- v2 selects EG $p < 0.05$ and $\beta > 0$, limiting the study to `long/short` relative value, not asserting that a negative beta cannot be hedged. It was frozen after constructed results were seen and before real-data analysis. v1, clarifications `A1/A2`, and c1–c4 results remain unchanged. The CLI accepts v2 and records its SHA-256 and specification in `summary.json` .
- R01 replaces same-day full volume with ADV20 known at the signal close, in post-split shares, for the 09:30 budget. This is an uncalibrated opening scenario; full-day volume is a post-close diagnostic. R02 marks both legs at the 09:30 open after corporate actions and before fills, using the opening mark for intraday borrow. R04 implements closed-episode win rates, holding periods, and fill-notional turnover.
- v2 selects `CA_CB` three times; screened portfolio excess Sharpe is −0.15. Every cost- and threshold-grid value is negative. The preregistered conclusion is that the evidence does not support the strategy. Old v1 screened Sharpe 0.39 remains in c1–c4 evidence.

ALIGN protocol alignment and the later boundary, 2026-10-14

ALIGN implementation r1 and exercise revision r2 received limited acceptance. `ACCEPTED_VERSION` retains the c5 identity; the later whole-share boundary supersedes ALIGN's earlier fractional-holding description. Entry targets are truncated in signal-close units before split conversion; the IS reference price separately handles dividends. The real-data gate binds the actually consumed preregistration bytes to the manifest's main-specification digest. Constructed custom configurations are labelled `constructed_custom_exploration`; real data remain blocked. Historical author handoffs are under `qa/a10/project05_protocol_alignment/` ; readers use the ordered acceptance records above rather than an author conversation.

Current whole-share execution domain

The 2026-10-14 boundary and limited acceptance specify consistent decimal entitlement conversion and whole-account failure if a split would produce fractional actual holdings. A residual holding must not be labelled exited. Historical ALIGN and c5 identities remain. Real fractional execution or cash in lieu still requires a separate accepted policy. Empirical research remains blocked and the complete real-market project is not verified.

Project 5 — Exercises

These constructed exercises inherit the limited `c5/ALIGN` acceptance and the later whole-share boundary. This BIL17B bilingual teaching candidate awaits independent acceptance. Complete answers are in solutions.md; reference implementation: `projects/project05/solutions_code.py`. Code exercises use the **teaching constructed panel** in `src/p05_constructed.py`. It demonstrates methods, not market evidence. Real-price research remains blocked; see report Section 2.

Prerequisites: Chapters 16–17 (stationarity and cointegration), Chapters 29 and 31 (accounting and walk-forward evaluation), and Chapters 35–36 (costs, execution, and IS).

P5-01 — Hedge units, hand calculation. Training regression $\log TR_y = \alpha + \beta \log TR_x + e$ gives $\beta = 1.2$. Signal-close prices are $P_y = 80$ and $P_x = 50$. For this simplified share-sizing exercise, assume no split or dividend between the signal close and entry, and enter at those prices; do not separately model overnight price changes or execution costs. Take a long-spread position with target gross exposure USD 1,000,000. Using $w_y = 1/(1 + |\beta|)$ and $w_x = -\beta/(1 + |\beta|)$, calculate dollar exposures, share quantities truncated toward zero, absolute and signed share ratios, and net dollar exposure. Why is the share ratio different from β? Does it still equal $|\beta|P_y/P_x$ exactly after rounding? In a level regression $P_y = a + bP_x + e$, what are the units of b?

P5-02 — Corporate actions, hand calculation. This is a corporate-action scenario independent of the P5-01 entry: before the events, the account already holds 5,681 shares of y and is short 10,909 shares of x, with preceding closes of 80 and 50. At the next opening, y splits 2:1 and x goes ex-dividend by USD 0.30 per share. State both share and cash changes. Then apply $TR_t/TR_{t-1} = r_t(C_t + D_t)/C_{t-1}$: y previously closed at 80 and closes post-split at 40.6; x

previously closed at 50 and closes ex-dividend at 49.9. What happens if vendor-adjusted prices are also used for execution?

P5-03 — Financing and borrow, hand calculation. Return to the original P5-01 entry, not the `post-split/dividend` P5-02 account. Initial cash is USD 1,000,000 with no holdings; ignore entry fees. This constructed cash model allows all short-sale proceeds to earn interest. Initial NAV is therefore also USD 1,000,000; gross target exposure alone would not establish it. Find post-entry cash. Friday 16:00 to Monday 09:30 lasts 65.5 hours in UTC or over a weekend without a DST change. Assume DFF 5.33% is known at interval start; this is a problem input, not a new historical rate check. The cash credit rate is DFF minus 25 bp; annual borrow is 25 bp; value the short at the available Friday close of USD 50. Under `ACT/360`, calculate days, interest, borrow charge, and ending cash.

P5-04 — IS across partial fills on two days, hand calculation. An exit sells $Q = -10{,}000$ shares. The decision price, in post-event signal-close units, is 41.00. Day 1 opens at 40.80 and fills only 6,000 shares, with a 2 bp half-spread and 5 bp impact. Day 2 opens at 40.50 and fills the remaining 4,000, with the same 2 bp half-spread and 4 bp impact. Commission is USD 0.005 per share. Use Chapter 36's single-arrival convention to calculate delay, execution, fees, and total IS; cross-check the direct-dollar method. Examine this specific error: retain the original full-Q first-arrival delay, reanchor day-2 execution to its opening price, and add F_2 times "day-2 open minus decision price" as delay. How much is overcounted? If first-day delay is allocated only to F_1, is reanchoring necessarily wrong?

P5-05 — Correlation is not cointegration, code. For constructed `TA/TB`, calculate level correlation, daily-return correlation, and EG p in each 504-day formation window. Why can high correlation not replace a cointegration test?

P5-06 — Leakage diagnostics, code. Multiply every price after the first formation endpoint by 1.5. Re-estimate that window's `Formation` and compare β. Then substitute the full-sample spread mean for the formation mean. Explain the leakage and its effect on z-scores. Run `leakage_checks()` in `solutions_code.py`: what does each of its three channels rule out, and why does passing not prove that all leakage is absent?

P5-07 — Critical values, code. For first-window `CA/CB` OLS residuals, report the EG statistic, its 5% critical value, and the ordinary ADF 5% critical value. Which determines the conclusion? How does the conclusion change if the ordinary ADF cutoff is used incorrectly?

P5-08 — Selection and multiple testing, code and interpretation. Run the three constructed pairs under v2, EG $p < 0.05$ and $\beta > 0$, with screened and ungated sleeves. How many tests are performed, how many pass EG, and how many are selected by v2? If all nulls hold, how many $p < 0.05$ outcomes are expected? Compare excess Sharpe, closed-episode win rate, holding days, and turnover. Explain the bias from showing only selected or profitable pairs. Why is `IA_IB` window 5 excluded? Can negative β be hedged with same-direction legs? What can and cannot be learned from the difference between old v1 screened Sharpe 0.39 and v2? What changes with a 252-day formation window?

P5-09 — Partial fills and leg risk, code. Set $W_0 = 10^9$ and the opening-fill budget to 1% of ADV20. Compare `matched` and `independent`: partial-fill rows, exit-retry rows, and the

largest first-fill deviation "actual share ratio / ordered share ratio − 1." What does cancellation of an entry remainder imply for PnL? Why can cancellation of an exit remainder not be treated as closing the position? What sample, capacity, and whole-share-domain limits constrain daily retries? Why can full-day volume not determine a 09:30 fill?

P5-10 — Research judgment. Real-price data are blocked by licensing. Write a report conclusion stating what is and is not supported, where authorized data would enter the pipeline, what must be rerun, and which results would lead to "the evidence does not support the strategy."

Project 5 — Complete Solutions

These solutions inherit the limited `c5/ALIGN` constructed teaching evidence and explain it under the latest whole-share boundary. This batch did not rerun the historical CLI or reference solution. The original numbers were printed by `.venv/bin/python projects/project05/solutions_code.py` on 2026-10-10, with candidate identity `qa/a7/project05_batch/CANDIDATE_MANIFEST_c5.json`. P5-05 through P5-09 use the teaching constructed panel, not market data. Old c1–c4 results under v1 remain in `qa/a7/project05_batch/evidence/`; the results below are c5's.

P5-01

Use the simplified entry assumptions: no split or dividend between the signal close and entry, with entry prices `80/50.` This is share sizing without a separate overnight-price or execution-cost model.

Weights are $w_y = 1/2.2 = 0.454545$ and $w_x = -1.2/2.2 = -0.545455$. Dollar exposures are +454,545.45 and −545,454.55. Truncating toward zero gives

$$n_y = \lfloor 454{,}545.45/80 \rfloor = 5{,}681, \qquad n_x = -\lfloor 545{,}454.55/50 \rfloor = -10{,}909.$$

The absolute share ratio is 10,909/5,681 = 1.920261 and the signed ratio is −1.920261. Before rounding, they are $|\beta| P_y/P_x = 1.92$ and $-\beta P_y/P_x$ respectively; afterward they hold only approximately. Net dollar exposure is 5,681(80) − 10,909(50) = −90,970, so the position is not dollar neutral.

Log-regression β is an elasticity: a 1% change in x is associated with a β% change in y. With fitted coefficients frozen and the residual unchanged, the local first-order hedge offsets this common percentage movement. Residual changes, coefficient drift, jumps, and rounding risk remain. Thus the dollar ratio involves β, while the share ratio additionally involves P_y/P_x.

For $P_y = a + bP_x + e$, units are $(\text{USD/share}_y)/(\text{USD/share}_x) = \text{share}_x/\text{share}_y$: one share of y is hedged with b shares of x. The signed hedge ratio is $n_x/n_y = -b$ and the absolute ratio is $|b|$. The level slope and log elasticity cannot be interchanged.

P5-02

This independent corporate-action scenario starts with pre-event holdings of 5,681/−10,909 and preceding closes of `80/50.` It does not convert the same opening event in P5-01 a second time.

The y holding doubles from 5,681 to 11,362 shares, with no split cash flow; its marking price is divided by two. The short x position pays dividend compensation $-10{,}909(0.30) = -3{,}272.70$ USD. Total-return steps are $2(40.6 + 0)/80 = 1.015$ for y and $(49.9 + 0.30)/50 = 1.004$ for x.

Execution and marking use unadjusted prices, with corporate actions booked separately. Signals use the forward-built TR index. Executing at vendor-adjusted prices and then also booking `splits/dividends` counts the same event twice, as explained in Chapter 29.

P5-03

Return to the original P5-01 entry: USD 1,000,000 initial cash, no holdings, no entry fees, and all short-sale proceeds available to earn interest in this constructed model. Target gross exposure alone does not determine starting NAV; the specified account does.

Post-entry cash is $C_0 = 1{,}000{,}000 - 5{,}681(80) + 10{,}909(50) = 1{,}090{,}970$. Marked holdings are worth $454{,}480 - 545{,}450 = -90{,}970$, so $NAV = C_0 - 90{,}970 = 1{,}000{,}000$.

In UTC, or on a weekend without a DST change, Friday 16:00 to Monday 09:30 is 65.5 hours: $\Delta d = 65.5/24 = 131/48$ days. DFF 5.33% is assumed known at the start; it is not a newly checked historical observation. Keep the exact day count until final display rounding:

$$I = 1{,}090{,}970(0.0533 - 0.0025)\frac{131/48}{360} \approx 420.1497,$$

$$B = 10{,}909(50)(0.0025)\frac{131/48}{360} \approx 10.3377,$$

$$C_1 = C_0 + I - B \approx 1{,}091{,}379.8121.$$

Rates, cash, and the short mark are available at interval start. Monday's newly observed opening price applies only to later intervals (Chapter 35). This calculation does not use the post-corporate-action P5-02 account.

P5-04

P5-04: complete two-day IS calculation and repeated-delay counterexample

Let $Q = -10{,}000$, $F_1 = -6{,}000$, and $F_2 = -4{,}000$, with no terminal remainder. The decision price is $d = 41.00$, first arrival is $a_1 = 40.80$, and the day-2 open is $a_2 = 40.50$. Commission is USD `0.005/share`. The half-spread and impact are already embedded in fill prices:

$$p_1 = 40.80(1 - 0.0002 - 0.0005) = 40.77144,$$

$$p_2 = 40.50(1 - 0.0002 - 0.0004) = 40.47570.$$

Keeping the first-arrival anchor gives

$$D = Q(a_1 - d) = 2{,}000,$$

$$E_1 = F_1(p_1 - a_1) = 171.36, \qquad E_2 = F_2(p_2 - a_1) = 1{,}297.20,$$

$$C = (|F_1| + |F_2|)0.005 = 50,$$

$$IS = D + E_1 + E_2 + C = 3{,}518.56.$$

The direct-dollar check is $F_1(p_1 - d) = 1{,}371.36$ and $F_2(p_2 - d) = 2{,}097.20$. Their sum plus 50 is again 3,518.56.

The specific algorithm producing 4,318.56 retains full-Q delay $D = 2{,}000$, reanchors day-2 execution to a_2, and adds $D_2 = F_2(a_2 - d) = 2{,}000$. Now $E_2' = F_2(p_2 - a_2) = 97.20$, hence

$$IS_{bad} = 2{,}000 + 171.36 + 97.20 + 2{,}000 + 50 = 4{,}318.56.$$

The extra 800 is $F_2(a_1 - d) = (-4{,}000)(-0.20)$: the first-arrival delay already charged to the original 4,000-share remainder.

Reanchoring itself need not be wrong. Allocate first-day delay only to F_1, giving $D_1' = F_1(a_1 - d) = 1{,}200$. Then $D_1' + E_1 + D_2 + E_2' + C = 3{,}518.56$, correctly. Two other errors leave the original execution anchor unchanged and merely append $F_2(a_2 - a_1) = 1{,}200$ or $F_2(a_2 - d) = 2{,}000$. They overcount by 1,200 and 2,000 respectively, not by this problem's 800. The implementation stores one first-arrival price per parent order, so a cross-day retry does not charge that delay again.

P5-05

Window	Level correlation	Daily-return correlation	EG p
0	0.833	0.810	0.725
1	0.539	0.781	0.544
2	0.860	0.779	0.483
3	0.964	0.796	0.350
4	0.976	0.793	0.655
5	0.876	0.810	0.950
6	0.735	0.802	0.731
7	0.559	0.798	0.866

None of eight windows has $p < 0.05$. Each series combines the shared random trend with its own independent random walk. Returns can therefore be highly correlated while every linear combination retains an uncancelled random-walk component. The spread is nonstationary in this construction. Level correlation between nonstationary series may be high through spurious association; return correlation describes comovement. Neither tests whether the spread returns to a mean.

P5-06

Multiplying prices after the formation endpoint by 1.5 leaves $\beta = 1.191788$ and every `Formation` field unchanged: formation reads only rows at or before its endpoint. The formation spread mean is zero because OLS includes an intercept; the full-sample spread mean is 0.011635. Full-sample centering or scaling uses trading-period prices, changes threshold-crossing dates, and introduces future information into the backtest. Freeze formation α, β, mean, and standard deviation; only insert new prices in the trading period.

Historical runnable check `leakage_checks()` in `solutions_code.py` covers three channels: (a) formation parameters cannot depend on later prices; (b) orders from signals on or before close t, and NAV before t, cannot depend on prices after t (historical check 2014-09-15: True); (c) a 09:30 fill cannot depend on that day's full-day volume (2014-09-16: True). Relevant tests: `pytest projects/project05/tests/test_p05_pairs.py -k "leakage or fill_day_volume"`. These are necessary checks of three channels, not proof that all implementation leakage is absent.

P5-07

The EG statistic is -3.0636, the MacKinnon cointegrating-residual 5% cutoff is -3.3483, and $p = 0.0958$. The ordinary ADF 5% cutoff is -2.8673. Use the EG cutoff. Estimating the first-stage cointegrating regression produces a nonstandard null distribution for the residual test; match deterministic specification, dimension, and sample conventions (Chapter 17).

A smaller OLS RSS alone does not establish faster mean reversion: multiplying the same random walk by 0.1 reduces RSS to 0.01 of its original value but leaves its ADF statistic unchanged under the same specification. Incorrectly using -2.87 would reject at 5% and claim cointegration;

the correct result fails to reject no cointegration. Even though CA/CB were constructed as cointegrated, with $\phi = 0.97$, 504-day power is limited. Failure to reject is not evidence that cointegration is absent.

P5-08

Main specification: v2 selects EG $p < 0.05$ and $\beta > 0$.

Return and risk summary (all eight rows; trade activity follows).

Pair	Gate	Total return	Excess Sharpe	Signed trough min(drawdown)
CA_CB	screened	0.0606	−0.1432	−0.0187
CA_CB	ungated	0.1171	0.4771	−0.0187
TA_TB	screened	0.0710	missing (no trades)	0.0000
TA_TB	ungated	−0.0602	−0.8558	−0.0917
IA_IB	screened	0.0710	missing (no trades)	0.0000
IA_IB	ungated	−0.0328	−0.2298	−0.3048
Portfolio	screened	0.0676	−0.1485	−0.0058
Portfolio	ungated	0.0081	−0.4717	−0.1232

Trade activity (the same eight rows; continued).

Pair	Gate	Closed episodes	Win rate	Mean holding days	Annual one-sided turnover
CA_CB	screened	4	0.25	37.25	1.00
CA_CB	ungated	8	0.50	33.50	2.01
TA_TB	screened	0	missing	missing	0.00
TA_TB	ungated	6	0.50	47.17	1.55
IA_IB	screened	0	missing	missing	0.00
IA_IB	ungated	8	0.625	47.63	2.03
Portfolio	screened	4	0.25	37.25	0.33
Portfolio	ungated	22	0.545	42.36	1.86

There are 24 tests, three pairs across eight windows. Four pass EG; v2 selects three, CA_CB windows 2–4. IA_IB window 5, $p = 0.047$ and $\hat{\beta} = -1.907$, is excluded by $\beta \leq 0$. If all nulls hold and every null p-value is correctly calibrated, linearity of expectation gives $24(0.05) = 1.2$ outcomes below 0.05 without requiring independence. Overlapping windows and shared series are not 24 independent trials. There is one within-window BH $q < 0.10$ outcome, a descriptive result without a demonstrated FDR guarantee across all dependent windows. The 7.1% return of no-trade sleeves is cash interest; their tiny excess return is interest-discretization error, so Sharpe, win rate, and holding days are missing.

Selection effects matter. Constructed cointegrated CA_CB has only four screened trades and a negative excess Sharpe, while ungated is positive. Screening can miss a true relationship and admit an accidental one. v1, retained in c1–c4, selected four windows and produced screened portfolio excess Sharpe 0.39, largely from one false-positive IA_IB selection. Excluding it under v2 yields −0.15. This does not show that v2 is better: its long/short relative-value scope

and timing repairs were frozen after constructed results were seen, as disclosed. `IA_IB`'s lack of cointegration follows from its independent-random-walk construction and the testing setting, not merely from the sign of its fitted beta.

Negative beta can be hedged with same-direction legs. For $\beta = -1.2$, long-spread dollar amounts about 454,500 and 545,500 have the same sign; if $d\log y = -1.2\, d\log x$, their first-order total PnL is zero. v2 excludes this only to restrict scope to one long and one short leg. At a 252-day formation window, portfolio excess Sharpe is -0.0211 screened and -0.7668 ungated. Report every pair and window, including ungated controls.

Win rate and holding days describe position episodes: first fill leaving flat through the fill returning both legs to zero. A new entry before an exit completes belongs to the same episode (`reentry`). Episode PnL includes fill cash flows, commission, holding-period dividends, and borrow charges, but not cash interest. Positions still open at sample end do not enter win rate. Turnover uses actual fill notional: two-sided $\sum |q| P^{fill}$; annual one-sided turnover is 0.5 times that amount, divided by mean daily NAV, times $252/N$. It does not use requested order quantity.

P5-09

The opening budget is 1% of ADV20 available at the signal close, converted to fill-day post-split shares. This uncalibrated opening scenario is not a realized-participation guarantee. `matched` has 14 partial-fill rows and four exit retries; the first-fill ratio deviation is zero at printed precision. `independent` has 14 partial-fill rows and five retries, with deviation as large as 0.6376: actual hedging can differ from the ordered ratio by about 64%, leaving direction risk.

Canceling an entry remainder reduces or changes exposure without a predetermined PnL sign: a smaller entry may avoid either profit or loss. Whole-share rounding leaves small mismatch even with `matched`; unequal fill fractions create further mismatch under `independent`. Canceling an exit remainder does not remove the remaining position. Depending on actual fills, the remainder may still be a matched pair, a mismatched pair, or a single leg. For example, start with Y100 and X$-$200; a matched exit that closes Y50 and X$-$100 leaves Y50 and X$-$100. The share ratio remains 2, but the account is not flat. With beta 2, equal leg prices, and a constant residual, the common-factor exposure of that remainder is still zero to first order for a small move. This protocol therefore submits daily replacement orders to complete the intended exit; cancellation must not be recorded as a completed close. Completion remains subject to sample length, capacity, and the current whole-share execution domain, with no unconditional promise to reach zero. A remainder replaced by a new entry receives opportunity-cost attribution, and a remainder at sample end uses the final closing mark. Full-day volume is a post-close diagnostic (`day_participation_diag`) and cannot change the 09:30 fill.

P5-10 — Reference conclusion

We can claim implementation of v2's training-only screening, frozen spread, next-opening execution, two-leg costs, and financing ledger. Constructed checks show cash reconciliation residuals around 10^{-9} USD, equivalent `decomposed/direct` IS, formation estimates invariant to later-price perturbations, and examples of correlation without cointegration, wrong critical values, and selection effects. We cannot claim that any real pair is cointegrated, profitable, or effective after costs, or that the scenario cost, borrow, or capacity levels are empirical estimates.

Once authorized data are frozen in `data/processed/p05/` and `data_manifest.json`, restart at that data node: TR construction → `formation/tests` for all eight pairs and windows → `screened/ungated` accounting → cost, threshold, formation-window, and partial-fill grids → figures, Notebook, and report. v1 and v2 share the `claim_rule`: a positive main result must survive the cost grid and `screened/ungated` selection comparison before being described as supporting the strategy. If portfolio excess Sharpe becomes nonpositive at higher costs, or any screened advantage depends on isolated `pairs/windows`, conclude that the evidence does not support the strategy and retain all negative results. Threshold grids and the 252-day window are descriptive; do not select a better variant and re-report the same sample.

P5-ALIGN — Signal shares and the IS reference price

At signal close 100, NAV 1,000,000, and weight +0.5, a next-day dividend of 1 with no split leaves a 5,000-share target and IS reference 99. Weight −0.5 gives −5,000 shares. With a 2:1 split and dividend 1, truncate to 5,000 first, then convert to 10,000 shares; the reference is 49. NAV 1,001 gives 5.005 before truncation, hence 5 then 10 shares. An ex-dividend price must not inflate the target. At NAV 1,199, truncating 5.995 then multiplying by two gives 10; converting before truncation would incorrectly give 11.

Decimal entitlement conversion does not authorize fractional actual holdings or their exit. Under the latest boundary, input 0.3333333333333333 is not exact 1/3. A flat account with a five-share target converted at 1.5 has a 7.5-share request, fills seven whole shares, and cancels the entry remainder. An existing five-share holding that would split into 7.5 instead causes whole-account failure before that day's overnight accrual and corporate-action posting. Products $4(1.5) = 6$ and $5(0.2) = 1$ remain supported. Do not remove failed windows and continue reporting a fixed-sample main result. Real fractional execution or cash in lieu needs a separately accepted policy; see EXECUTION_BOUNDARY.md.

Project 5 Data Dictionary

Status: inherits the limited acceptance of `c5/v2` and the ALIGN field contract, with the latest whole-share boundary taking precedence. This is a bilingual teaching candidate, not new market-data validation. The real panel is blocked (`data_manifest.json`); the same field contract is implemented and tested by the teaching constructed panel in `src/p05_constructed.py`.

Input 1: Daily panel (long format)

Primary key: (date, ticker). Every ticker must cover every trading day; a missing ticker-day raises an error. No forward filling and no dropping rows across gaps.

Field	Type / unit	Definition	Time
date	Date (New York trading day)	Trading day	`event_time`: open at 09:30 and close at 16:00
ticker	String	Security identifier	—
open	`USD/share`, unadjusted	Original unadjusted open-price proxy (not an observable mid); used as the execution reference, `first_arrival_price`, and the 09:30 mark, including for an existing position with no fill	Available at 09:30
close	`USD/share`, unadjusted	Closing price; valuation and signal	Available at 16:00
volume	Shares, unadjusted	Full-day volume; ADV20 (through the signal close) determines the ex ante opening execution budget	Available after the close; volume on the execution date is used only in a post-close diagnostic and does not change the 09:30 execution
`split_ratio`	New shares / old share	Ratio on the ex-date; 1 when there is no event	Known before the open on the ex-date
dividend	USD per post-event share	Cash per share on the ex-date; 0 when there is no event	Known before the open on the ex-date

Validation: all numeric values are finite; open, close, and `split_ratio` > 0; volume and dividend ≥ 0; no duplicate primary keys.

Input 2: Interest rates

Field	Unit	Definition
date	Observation date	FRED DFF observation date (real); constant 2% in the constructed panel
rate	Annualized decimal	Effective federal funds rate

Availability rule: the value observed on date s is available beginning the next day. Trading day d uses the last observation strictly before d. The overnight accrual interval uses the rate available on the preceding trading day; the intraday interval uses the rate available on the current day.

Derived series

Name	Formula	Use
TR index	$TR_t = TR_{t-1} r_t (C_t + D_t)/C_{t-1}$, with the first value equal to 1	Signal and formation-period regression; not used for execution
sigma20	20-day sample standard deviation of log TR returns, through the signal close	Impact
ADV20	Mean volume over the 20 days through the signal close, converted to post-split shares on the execution date	Impact denominator; opening execution budget = 5% × ADV20 (an uncalibrated scenario that does not guarantee an actual participation rate)
Decision price	$C_t/r_{t+1} - D_{t+1}$	IS reference price only; target shares are first rounded toward zero using the signal-close price, then multiplied by the split ratio

File	Key / rows	Main fields

Outputs (`run_project05.py`; each run writes to a new directory)

File	Key / rows	Main fields
summary.json	1	`data_class`; preregistration `path/version/SHA256` and v2 summary (selection rule, opening budget, event order); code SHA256; window, test, EG-pass, and selected counts; scenarios and rules
formations.csv	(pair, window)	Formation `start/end`, n, alpha, beta, `spread_mean/sd`, `eg_stat/pvalue/crit_5`, correlation, half-life, within-window BH q, `eg_pass` ($p < 0.05$), selected (v2: $p < 0.05$ and beta > 0), and `first/last` signal dates in the trading period
nav.csv	date × pair|gate	Daily closing NAV (USD); first row is formation-period-end `W_0`
orders.csv	Each execution attempt × leg	`order_id`, kind (`entry/exit/entry_skipped/superseded_remainder/unfilled_at_end`), `signal_date`, `fill_date`, `attempt_ordered`, filled, remainder, `decision/first_arrival/open/fill` prices, commission, `is_delay/execution/fees/opportunity`, `budget_adv20`; post-close diagnostics `day_volume_diag` and `day_participation_diag`
trades.csv	Each trade (position episode)	status (`closed/open`), `start_date`, `end_date`, `holding_days` (trading days), `calendar_days`, `entry_orders`, reentries, `same_sign_legs`, `fill_flow`, commission, dividends, borrow, `open_mtm`, pnl (excluding cash interest)
`ledger_events.csv`	Events	ACCRUAL (interval `start/end`, days, balance, rate, interest, borrow, short market-value basis), SPLIT, DIVIDEND, `OPEN_MARK`, FILL, MARK

File	Key / rows	Main fields
metrics.csv	(pair, gate) and PORTFOLIO	`Total/annualized` return, volatility, excess Sharpe (missing when no trades), maximum drawdown, `closed_trades`, `open_trades`, `win_rate`, `mean/median_holding_days` (missing when there are no closed trades), `traded_notional_two_sided`, `turnover_one_sided_ann`, cumulative cash by item, `cash_residual`
robustness.csv	Grid point × (pair, gate)	Same as metrics, plus grid and grid parameters
`rates_used.csv`	date	Policy rate available on each trading day

Amounts are in USD; returns are decimals; timestamps use `America/New_York.`

In the report and legacy metrics table, drawdown is the non-positive min(drawdown) (for example, −0.0187), not a positive drawdown magnitude as used in the prose; preserve the sign in the original table. `attempt_ordered` is the request on this attempt, filled is the actual whole-share execution, and remainder is the remainder from this attempt; an order record is not a position. Orders in transit and cutoff status must be distinguished from cash and NAV using the event ledger. If a split would make an actual position fractional, the entire account fails before overnight interest accrual and corporate-action booking; see Execution Boundary.

Project 5 Research Report: Preregistered Walk-Forward Pairs Trading

Status: inherits the `c5/v2` constructed results and the limited-accepted ALIGN alignment; the current execution domain is governed by the 2026-10-14 whole-share boundary. BIL17B is a teaching-integration candidate. Historical result date: 2026-10-10; `protocol/boundary` date: 2026-10-14. **Real-price empirical work: blocked (data permissions).** Every numeric result in this report comes from the explicitly labeled teaching constructed panel. It shows only that the workflow, ledger, and testing methods can run under the contract; it is not evidence of profitability or cointegration for any real pair. The old c1–c4 results under preregistration v1 remain unchanged in qa/a7/project05_batch/evidence/ and are not reused as c5 conclusions.

1. Research question and falsifiable hypothesis

Chapter 17 explains that if two non-stationary prices have a stationary linear combination (cointegration), the spread returns toward its mean; correlation describes only synchronous movement and does not guarantee spread reversion. This project asks whether, among **eight economically related U.S.-listed pairs fixed in advance**, screening for cointegration only in the training segment and freezing the parameters, then trading relative value one long and one short by z-score in the next segment, produces robust positive excess returns after two-leg costs, borrow, and financing.

Decision rule (`claim_rule`, unchanged between v1 and v2): a positive result in the primary specification counts as "support" only if it persists across both the cost grid and the selection-effect comparison of screened versus ungated; otherwise the conclusion is "the evidence does not support this strategy," and all zero and negative results must be retained. The threshold grid and 252-day formation window are descriptive only; they must not be used to choose a version and then report on the same sample.

2. Data and status

2.1 Preregistered universe and versions

v1 (SHA256 92e5f20c…4b22) was frozen before any price download: eight pairs, `EWA/EWC`, `KO/PEP`, `XOM/CVX`, `GDX/GLD` (expected to be weak), `XOP/XLE`, `HD/LOW`, `V/MA`, and `IVV/SPY` (a same-index control); sample 2010-01-04 through 2025-12-31; all windows for all eight pairs count toward the number of tests.

v2 (`configs/preregistration_v2.json`, SHA256 9af9dc84…87c3) was frozen after the constructed results had been seen but before any real-data analysis, and replaces v1 for all new runs. v1, clarifications `A1/A2`, and all c1–c4 results remain unchanged. Changes in v2 and their reasons:

Item	v1	v2	Reason
Selection rule	EG $p < 0.05$	EG $p < 0.05$ and beta > 0	Restricts the project to long-short relative-value pairs; this is not because a negative beta cannot be hedged mathematically
Opening execution capacity	5% of full-day volume on the execution date	5% of ADV20 known through the signal close, converted to post-split shares on the execution date	Full-day volume is not available until after the close, so using it to determine a 09:30 fill in v1 was look-ahead (c4 acceptance R01)
Intraday event order	Accrual → corporate action → execution	Accrual → corporate action → mark both legs at the 09:30 open → execution → accrual	The intraday borrow basis for an existing short with no fill must use the opening price (c4 acceptance R02)

Item	v1	v2	Reason
Evaluation outputs	Turnover, win rate, and holding days were listed but not implemented	Definitions supplied and implemented (§4)	c4 acceptance R04

The universe, dates, windows, test level, thresholds, positions, and all cost-scenario values were unchanged. The P5-ALIGN alignment implementation: the real-data entry point first reads the original preregistration bytes and matches their SHA-256 to the fixed primary-specification summary in the data manifest; data loading begins only after a match. Custom constructed configurations are labeled exploration; this report uses frozen v2.

2.2 Source review and access block

Real prices require unadjusted opens and closes, volume, split ratios, and cash dividends per share, with permissions that allow scripted download, persistent storage, internal bank training use, and delivery of derived statistics. The following carries forward only the historical review from 2026-10-10; current terms were not rechecked in this batch, and these specific cases are not generalized into a blanket prohibition on all free sources (see `qa/a7/project05_batch/DATA_SOURCE_DECISION.md`).

Source	Finding
Yahoo Finance	Terms prohibit automated collection without prior permission and commercial use; no download
Alpha Vantage	License limited to personal, non-commercial use
Tiingo	Free tier does not permit persistent storage; organizations must purchase a commercial plan
Stooq	Bot verification encountered; not bypassed
Nasdaq Data Link (Sharadar `SEP/SFP`)	Procurement candidate pending verification: general terms limit use to internal business purposes and are subject to the Order Form; external distribution of derived products requires a contract or written approval; third-party terms also apply. Coverage for the 16 securities, complete date coverage, unadjusted OHLCV, split, and dividend fields have not been verified
Other free sources and historical borrow data	Personal-use restrictions, prohibition on scripted collection, or not found (scout report; some items were not independently reread by the primary reviewer)

Accordingly, `data_manifest.json` has status=blocked, and the CLI with –data real refuses to run with exit code 3 and writes no files (c5 test). Whether any specific subscription covers this project's internal training use must be confirmed by the institution's contract; this report does not record authorization.

2.3 Teaching constructed panel (the sole source of results in this report)

`src/p05_constructed.py` (seed 20261010, 1,600 business days) designs three pairs of log total-return paths: `CA/CB` are cointegrated (beta = 1.2, AR(1) residual, phi = 0.97); `TA/TB` share a random trend but each also has an independent random walk (not cointegrated); `IA/IB` are independent. It then backs out unadjusted closes using $C_t = (C_{t-1}/r_t)(TR_t/TR_{t-1}) - D_t$ (a 2-for-1 split in CA on day 700, a 3-for-1 split in TB on day 1100, and quarterly dividends), so the constructed data pass through the same corporate-action code as real data.

3. Time contract

Field	Definition
`event_time`	Open (09:30) and close (16:00) on trading day d, `America/New_York`
`available_at`	Open price available at 09:30; close and full-day volume available after 16:00; the rate for DFF observation date s is available from the next day (publication lag approximated as one calendar day)
`signal_time`	Close at t: z-score uses only TR through t and frozen formation-period parameters
`trade_time`	Open of the next trading day, t+1; no execution at the same close
return interval	Between consecutive closing marks; NAV starts at `W_0` at the formation-period-end close, so N+1 points give N returns
label interval	None (signals use no future-return labels), so walk-forward does not require purging or embargo

Event order on the same day: overnight accrual (rate available from the prior close, prior-close mark) → split → dividend → mark both legs at the 09:30 open → opening execution → intraday accrual (rate available at the current open, opening mark) → closing mark.

4. Method

Formation (training segment only). For each 504-day formation window, run OLS of $\log TR_y = \alpha + \beta \log TR_x + e$, and freeze alpha, beta, residual mean, and residual standard deviation. The EG test uses statsmodels coint (constant, AIC lag); critical values come from the MacKinnon cointegration-residual distribution, not the ordinary ADF (Chapter 17). v2 selection rule: unadjusted p < 0.05 and beta > 0. The within-window BH q-value is reported but is not used to select pairs.

Signal. $z_t = (s_t - \bar{s})/\hat{\sigma}_s$, where `s_t` uses frozen alpha and beta applied to new TR. Enter when $|z| \geq 2$ (if $z > 0$, short the spread: short y and long x); exit when $|z| \leq 0.5$ or when z crosses zero; stop out when $|z| \geq 4$ and do not re-enter in the same window. Submit a forced-exit request at the final close of the window; completion remains subject to the sample, capacity, and whole-share execution domain.

Hedge and position sizing. For a log-price spread, beta is an elasticity (when x changes by 1%, y changes by beta% on average): dollar exposures are $w_y = s/(1+|\beta|)$ and $w_x = -s\beta/(1+|\beta|)$, giving gross exposure equal to one times signal-close NAV. At the signal, target shares are $n_t = \text{trunc}(wW_t/C_t)$, rounded toward zero first and then converted to next-day post-split shares $r_{t+1}n_t$. The ex-dividend event on the next day does not increase the target share count, and positions are not rebalanced while held. Before rounding, the absolute share ratio is $|n_x|/|n_y| = |\beta|P_y/P_x$ and the signed ratio is $n_x/n_y = -\beta P_y/P_x$; after rounding the ratios are only approximate. For the price-level regression $P_y = a + bP_x + e$, the signed hedge ratio is −b (Exercise P5-01). When beta < 0, both legs move in the same direction; same-direction positions can also hedge a negatively related pair to first order (with beta = −1.2, the two legs have dollar exposures of 454,500 and 545,500 in the same direction, and if d log y = −1.2 d log x, first-order total PnL is zero). v2 simply restricts the project to long-short pairs. The IS decision reference price is computed separately using next-day opening corporate-action adjustments: $P^{dec} = C_t/r_{t+1} - D_{t+1}$.

Execution and costs (scenarios, uncalibrated). Let Q be requested quantity and F actual nonzero fill; first apply capacity and whole-share rules to determine F, and update `cash/positions` only from F. Charge each leg separately: commission 0.005 `dollars/share`; half-spread 1 bp for ETFs and 2 bps for individual stocks; temporary impact $\eta\sigma_{20}\sqrt{|F|/ADV_{20}}$, with eta = 0.1 and sigma20 and ADV20 known through the signal close. Here open is an unadjusted open-price proxy, not an observable mid. Execution price = open × (1 + sign(F) × (half-spread + impact)); spread and impact are included in the execution price, and only commission is separately deducted from cash (Chapter 35's no-double-counting rule). **Opening execution budget:** each leg can fill at most 5% of ADV20 known through the signal close, converted to post-split shares on the execution date. This is an uncalibrated opening-execution scenario, not actual opening-market capacity, and does not guarantee an actual participation rate. Full-day volume on the execution date and |q|/full-day volume are used only in a post-close diagnostic; they do not change the 09:30 fill. Under the matched policy, both legs fill in the same proportion, based on the smaller executable proportion; the entry remainder is canceled on arrival. Exit remainders are not treated as closed merely by canceling them; they are worked day by day under the protocol. Completion is constrained by finite sample length, capacity, and the whole-share support domain. The independent policy is only a stress scenario.

Financing and borrow. `ACT/360.` For cash at the start of the interval, accrue cash × (DFF − 25 bps when cash is positive; DFF + 50 bps when cash is negative). Borrow fee = |`q_short`| × `P_available` × 0.25% × `delta_d/360`, using actual short shares and the mark available at interval start: prior close for the overnight interval, and the 09:30 open for the intraday interval; on a split date, use post-split shares and the post-split open price. Shorts compensate dividends on the ex-date. No historical borrow-fee or lendable-quantity data are available; borrow recall is represented only by a `locate_unavailable` scenario.

Implementation shortfall (IS). Sum only over the set of actual nonzero fills; an empty sum is zero. Do not multiply a zero fill by an undefined average price. Each parent order has one first-arrival price: delay $Q(P^{arr} - P^{dec})$ is counted once; each fill $F_k(P_k^{fill} - P^{arr})$ is counted for that fill, including fills of carried remainders on later days. A remainder superseded by a new

entry, or an exit remainder still unfilled at the sample end, incurs opportunity cost; an entry remainder canceled on arrival has zero opportunity cost (Chapter 36).

Trade statistics. Define one trade (episode) by the position: from the first fill that leaves flat to the fill that returns both legs to zero. If a new entry order occurs while an exit is incomplete, it is merged into the same trade and recorded as a reentry. Trade PnL = fill cash flow − commission + dividends − borrow fee; it excludes cash interest. Win rate is the fraction of closed trades with positive PnL; holding days are trading days from the first fill to the closing fill. An open trade at the sample end is counted separately and excluded from win rate. If there are no closed trades, win rate and holding days are missing. Turnover uses actual filled amounts only: two-sided traded notional = $\sum |q| P^{fill}$; annualized one-sided turnover = 0.5 × two-sided/average daily NAV × $252/N$; order quantities are not used. Excess Sharpe is missing for a sleeve with no trades (its excess returns are only residual error from discrete accrual).

Walk-forward. Roll 504-day formation windows and 126-day trading windows with a 126-day step. For each pair, run two independent ledgers: screened (trade only in windows that pass the v2 screen) and ungated (trade every window). Each starts with `W_0` = 1,000,000 dollars; the portfolio is the sum of equal-capital sleeves.

5. Implementation checks (constructed panel)

- Historical c5 tests: 40 passed. Historical P5-ALIGN checks: 52 passed (including the new share-count and real-entry counterexamples). c5 additions: changing only full-day volume on the execution date leaves that day's 09:30 fill, price, cash, and pre-close NAV unchanged item by item; the opening budget matches ADV20 × 5% calculated by hand from the input panel (including split conversion); the intraday borrow basis for an existing short with no fill, a one-leg execution day, and a split day is independently derived from input prices and actual positions; per-trade PnL, win rate, holding days, and two-sided traded amount match manual calculations; open trades are excluded from win rate; v2 excludes windows with beta ≤ 0 and the CLI rejects v1.
- Mutation checks: corresponding new tests fail if capacity is changed back to full-day execution-date volume, the open mark is removed, or the beta > 0 condition is removed (c5 author checks).
- The ledger cash identity, cash = `W_0` + interest − borrow + dividends − traded amount − commissions, has residuals around 10^{-9} dollars across the ledgers.

The P5-ALIGN historical phase reran the primary scenario and all robustness scenarios (not rerun in this batch); 11 CSVs (including orders, cash events, NAV, episodes, metrics, grid, and stress orders) were byte-for-byte identical to c5, so the following figures and values across five unchanged charts retain c5 identity. A targeted actual-order example shows: signal close 100, weight 0.5, NAV one million, and next-day dividend 1 produced 5,050 shares under the old order and 5,000 after the fix, while the IS reference price remained 99. Positive and negative legs, simultaneous `split/dividend`, and rounding order passed. Independent cash reconciliation included actual fills, commission, dividends, interest, and borrow. On the real-data entry point, changing a threshold with the same schema is rejected by the summary gate; matching bytes pass the summary gate, but real data still exits with code 3 and no output.

6. Constructed demonstration results (not market evidence)

There are 24 tests (3 pairs × 8 windows); EG $p < 0.05$ occurs 4 times; v2 selects 3 windows (`CA_CB` windows 2–4). `IA_IB` window 5 ($p = 0.047$, estimated beta $= -1.907$) is excluded because beta ≤ 0. If all tests were null and each p-value were correctly calibrated, linearity of expectation gives $24 \times 5\% = 1.2$ values below 0.05; independence is not required. Overlapping windows and shared series do not make these 24 tests independent. Only one within-window BH $q < 0.10$ (`CA_CB` window 3); this is a within-window description and does not establish FDR control across all dependent windows.

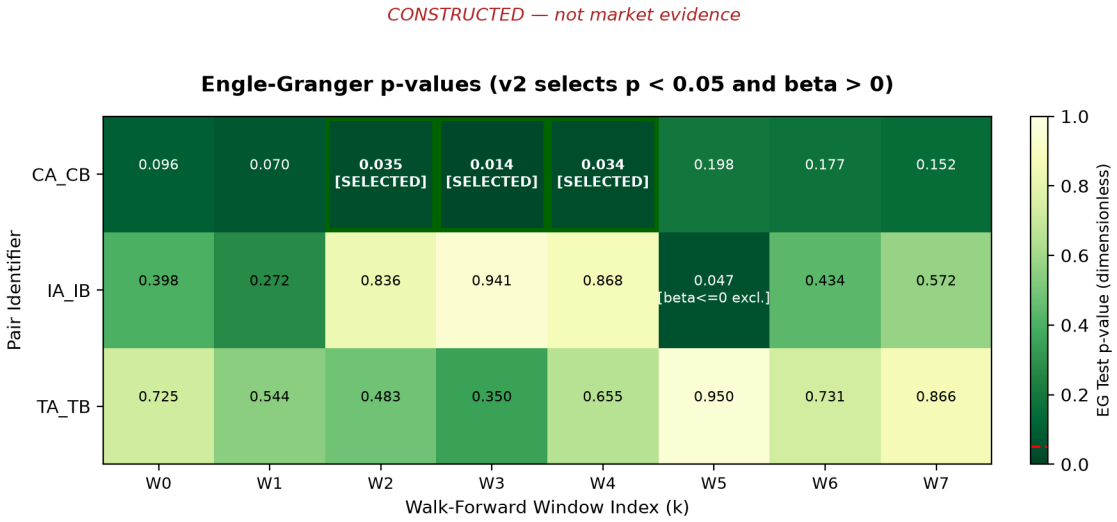


Figure 1: EG p-value heatmap

Figure 1. The horizontal axis is the walk-forward window; the vertical axis is the pair. Each tile shows the formation-period EG p-value, with v2-selected windows and the window excluded for beta ≤ 0 marked. `CA_CB` (true cointegration) passes in only 3 windows; p-values in the others are 0.07–0.20: test power is limited with a 504-day window and phi = 0.97. `IA_IB` (truly independent) has one false positive in window 5 ($p = 0.047$); its selection has no true cointegration basis, which follows from the constructed data-generating process and multiple testing, not from the sign of beta alone. `TA_TB` (common trend, not cointegrated) never passes.

Performance and risk metrics (all eight rows; trade statistics follow).

Pair	Mode	Total return	Annualized	Annualized volatility	Excess Sharpe	Signed deepest drawdown, min(drawdown)
CA_CB	screened	6.06%	1.48%	1.63%	−0.14	−1.87%
CA_CB	ungated	11.71%	2.81%	2.26%	0.48	−1.87%
TA_TB	screened	7.10%	1.73%	0.06%	missing	0.00%
TA_TB	ungated	−6.02%	−1.54%	3.74%	−0.86	−9.17%
IA_IB	screened	7.10%	1.73%	0.06%	missing	0.00%
IA_IB	ungated	−3.28%	−0.83%	9.24%	−0.23	−30.48%
Portfolio	screened	6.76%	1.65%	0.55%	−0.15	−0.58%
Portfolio	ungated	0.81%	0.20%	3.11%	−0.47	−12.32%

Trade statistics (the same eight rows; continued).

Pair	Mode	Closed trades	Win rate	Mean holding (trading days)	Annualized one-sided turnover
CA_CB	screened	4	25%	37.3	1.00
CA_CB	ungated	8	50%	33.5	2.01
TA_TB	screened	0	missing	missing	0.00
TA_TB	ungated	6	50%	47.2	1.55
IA_IB	screened	0	missing	missing	0.00
IA_IB	ungated	8	62.5%	47.6	2.03
Portfolio	screened	4	25%	37.3	0.33
Portfolio	ungated	22	54.5%	42.4	1.86

There are 1,008 daily returns (constructed dates 2013-12-06 to 2017-10-18). The cash-credit rate is the constructed 2% rate minus 25 bps, so even sleeves with no trades earn about 7% total return. No sleeve has an open trade at the sample end.

Figure 2. The vertical axis is dollar NAV, starting at `W_0`; solid line is screened and dashed line is ungated; the lower-right panel sums the three equal-capital pairs. **Interpretation:** under v2, the screened portfolio's excess Sharpe is −0.15 and comes only from 4 `CA_CB` trades (25% win rate); the truly cointegrated `CA_CB` trades less often when screened and performs worse than ungated. Under v1 (c1–c4), the screened portfolio selected 4 times and had a 0.39 Sharpe, mainly due to the false-positive `IA_IB` selection; excluding it under v2 makes the result negative. This does not mean "v2 is better" or "screening works": v2 was a design change frozen after seeing constructed results because of project scope and timing fixes, and the change was disclosed in the preregistration. Both versions show that screening can miss a true relationship and admit an accidental one.

Figure 3. The `CA_CB` window 2 frozen-spread z-score. Dashed lines are ±2 entry thresholds, dotted lines are ±0.5 exit thresholds, and dash-dot lines are ±4 stop-loss thresholds. The horizontal axis uses `fill_date`, while z comes from the corresponding `signal_date`; the mark is therefore the signal-day z displayed at the execution-date position, not a z recalculated on the execution date. This display debt is carried forward under `B23/P5_PLOT_CLOCK_CARRY_FORWARD.` The original figure was not redrawn, and changing the caption does not mean it was

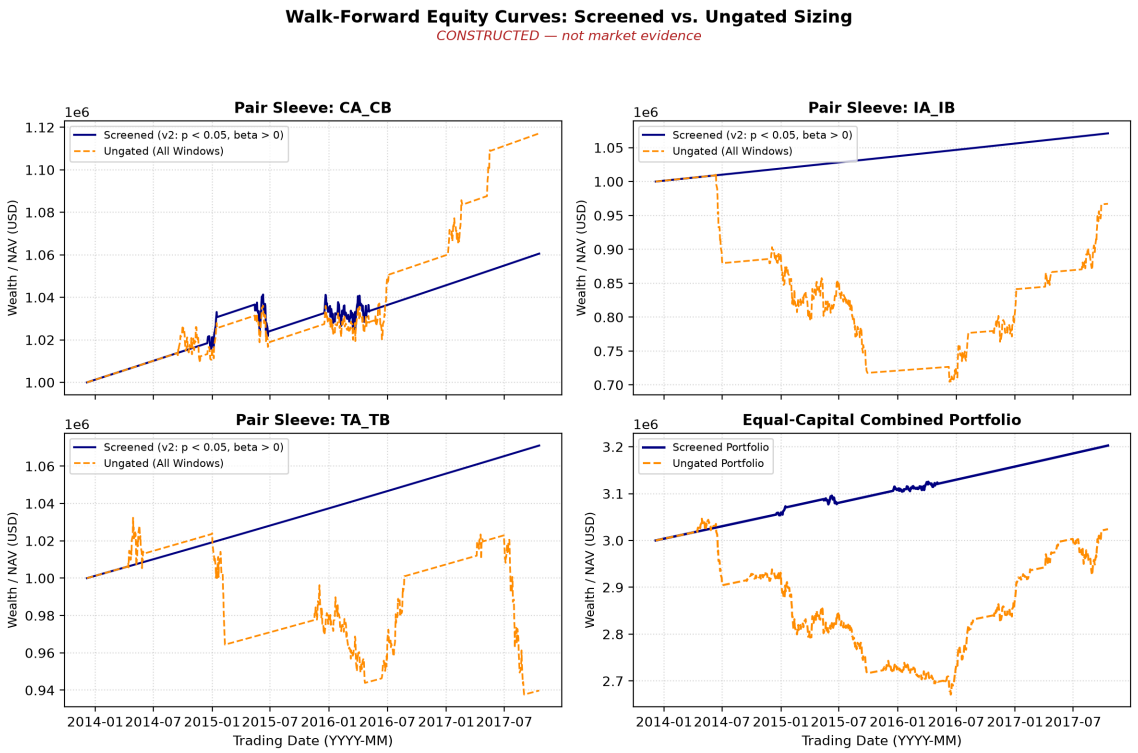


Figure 2: Screened and ungated NAV

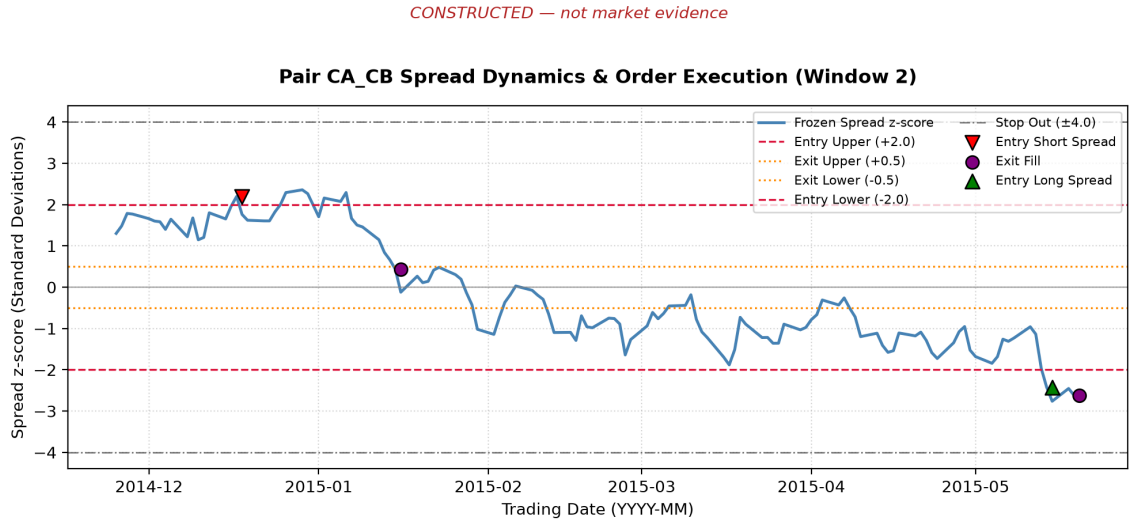


Figure 3: Single-window spread

repaired. Liquidation at the end of the window is triggered by the forced-close rule, not by mean reversion.

Figure 4. Cumulative cash items for screened sleeves (USD). At the constructed interest rate, interest is much larger than trading frictions. For the four `CA_CB` trades: spread 1,658, impact 442, commission 426, borrow 704 dollars; net dividends after short-dividend compensation are +826 dollars. The pair's dividend differential is a real cash item and must not be "removed" with adjusted prices.

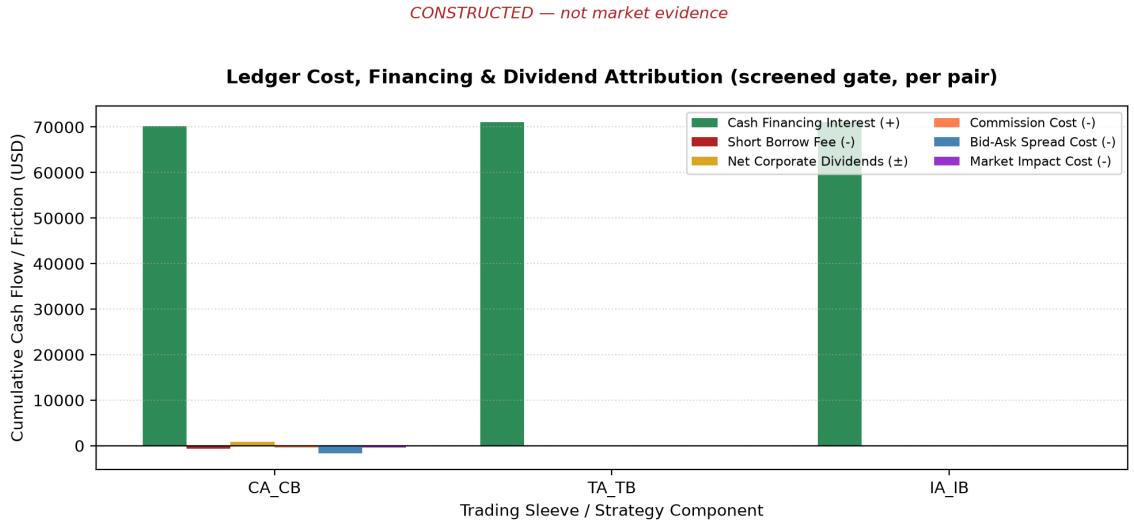


Figure 4: Cost and financing attribution

7. Robustness (constructed; all grid points are in robustness.csv)

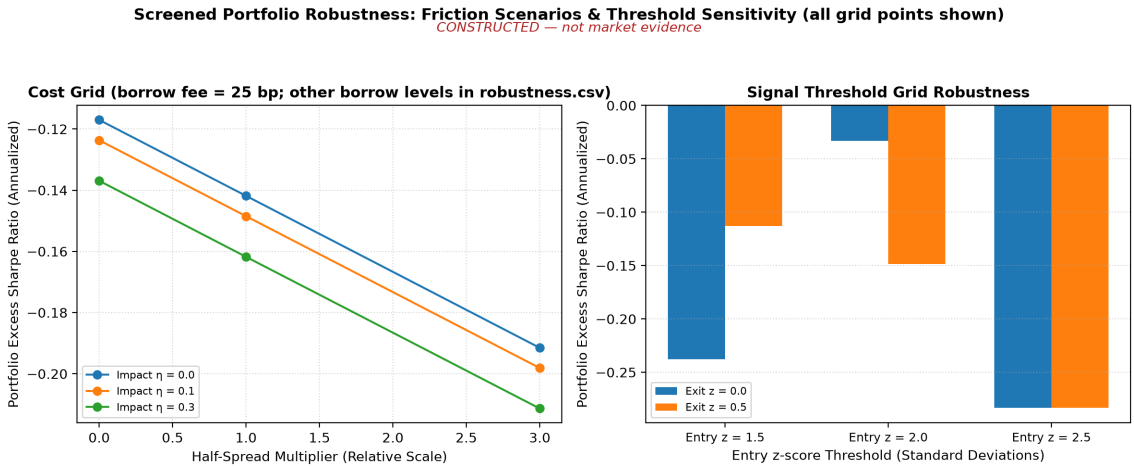


Figure 5: Robustness

Figure 5. Left: screened-portfolio excess Sharpe over the cost grid with borrow at 25 bps, varying half-spread multiplier and eta. Right: `entry/exit` threshold grid.

- Cost grid (27 cells): screened-portfolio excess Sharpe is negative in every cell, from −0.41 to −0.12; when borrow fees rise from 25 bps to 5%, the range falls from −0.21 to −0.12 to −0.41 to −0.32.
- Threshold grid (6 cells): −0.28 to −0.03, all negative. The grid is descriptive only; the primary specification was not changed in response.
- 252-day formation window: screened-portfolio excess Sharpe −0.02, ungated −0.77.
- Partial-fill stress (`W_0` = 10^9, budget = 5% of ADV20): under matched, 4 entry parent orders (8 legs) fill only 1.6%–2.0% of the submitted quantity (median 1.9%). Portfolio returns are mainly interest; the capacity budget, not the signal, determines the result.

Under the independent policy, the actual share ratio of the first fills can deviate from the submitted ratio by about 64% (Exercise P5-09, budget = 1% of ADV20). Post-close diagnostics show actual fills reaching up to 6.4% of full-day volume, above the 5% ex ante budget: the budget does not guarantee actual participation.

Under the preregistered decision rule, the primary constructed-panel specification has no positive result that could count as "support"; the conclusion is that the evidence does not support the strategy. All negative results are retained.

8. Limitations

The latest whole-share support domain is checked before any daily cash entry: if a split would make an actual position 7.5 shares, the entire account fails before overnight accrual or corporate-action booking; exact decimal entitlement conversion is not a guarantee of executable fractional shares. A 7.5-share request from a flat position can instead fill 7 whole shares and cancel the entry remainder; this is a different state. Do not delete a failed window and continue reporting a fixed sample; exit remainders are not guaranteed to reach zero before the sample ends. The historical ALIGN ending section once allowed actual fractional holdings to be retained; it is now superseded by the latest boundary, while the historical entry point remains unchanged.

1. Real-price empirical work is blocked; none of the above is a conclusion about real assets.
2. Costs, impact, borrow fee, spread, and opening-execution budget are scenarios and are not calibrated. There are no quote or opening-session volume data; the opening execution price is not a mid-price. Permanent impact is absent.
3. Borrow availability and recall are represented only as scenarios; no historical borrow data are available.
4. Dividends are booked on ex-dates (not payment dates); the DFF publication lag is approximated as one day.
5. The constructed panel has 3 pairs and 8 windows; power and the magnitude of selection effects depend on its parameters and do not generalize.
6. Preregistration is a freeze of local-file hashes, not registration on an external platform. v2 was frozen after the constructed results were seen, with its reasons disclosed; it was frozen before any real-data analysis and must not be changed during the real-data phase.
7. Trade PnL excludes cash interest. Win rate and holding days describe only a small number of trades in the constructed sample (4 screened-portfolio trades) and have no statistical significance.

9. Conclusion and rerun path

What can be said: preregistered v2's design, screening, frozen spread, next-day opening two-leg execution, ex ante opening budget, `cost/financing/borrow/dividend/split` ledger, IS decomposition, and trade statistics are implemented and were checked with behavioral tests and mutation tests. On the constructed panel, the workflow exposes correlation ≠ cointegration, incorrect critical values, and screening-selection effects. **What cannot be said:** whether the eight preregistered real pairs are cointegrated, whether the strategy is profitable, or the real levels of costs and capacity.

Once data are available (institutional permission confirmed, `field/sample` validation complete, files under data/processed/p05/, and `data_manifest.json` status=frozen), rerun v2 from the data node: build TR → form and test all windows for the 8 pairs → `screened/ungated` ledgers → cost, threshold, formation-window, and partial-fill grids → figures → Notebook → report sections 6–9. Existing constructed results must not be reused as real-data conclusions.

10. Reader run route and historical review

Ordinary readers should run `.venv/bin/python run_notebook.py projects/project05/notebooks/p05_pairs.ipynb` from the root of the accepted I3 package. The new constructed main route writes to the unique `RUN_DIR`; the outer runner also retains an execution copy of the Notebook and execution.json. It fresh-runs formation and ledgers for three `screened/ungated` constructed pairs, and produces a table and one figure; it does not run the full CLI grids listed below. The English Notebook layer, portable package, and cold start belong to BIL23 and were not executed in this batch. The commands below are for historical review and maintenance, not a list ordinary readers must run one by one; `build_notebook.py` writes the source Notebook.

```
.venv/bin/python projects/project05/run_project05.py --data constructed --out <
    ↪ new_directory>/cli
.venv/bin/python projects/project05/make_figures.py --run <new_directory>/cli --out <
    ↪ new_directory>/figures
.venv/bin/python -m pytest projects/project05/tests/test_p05_pairs.py -q
.venv/bin/python projects/project05/solutions_code.py
cd projects/project05 && ../../.venv/bin/python notebooks/build_notebook.py
```

The CLI reads `configs/preregistration_v2.json` by default and records its SHA256, selection rule, opening budget, and event order in summary.json; it refuses to run if the output directory already exists. Prerequisites: Chapters 16 and 17 (stationarity, cointegration, and EG critical values); Chapter 29 (unadjusted prices and corporate-action ledger); Chapter 31 (walk-forward); Chapter 35 (costs, financing, borrow, and no double counting); Chapter 36 (execution clock and IS). Part VI interface comparison: `qa/a7/project05_batch/PART_VI_CONSISTENCY_c5.md`.

Project 5 Whole-Share Execution and Split-Entitlement Boundary

2026-10-14. The target at the signal close is still rounded toward zero and then converted to the share units on the execution date. Actual executions remain whole-share only. This note adds no broker fractional-share execution or cash-in-lieu mechanism.

Split entitlement is calculated by exact decimal multiplication of the input float's current text representation. For example, 100 shares × 0.29 gives 29 shares. The same conversion is used for holdings, targets, parent exit orders, carried orders across days, and superseded order quantities, avoiding truncation of 28.999… to 28 shares. If an integer entitlement cannot be represented exactly in the current float ledger, the system raises an explicit error. The ratio field is the actual decimal value supplied as input; 0.3333333333333333 does not mean the exact fraction `1/3`.

The mathematical ledger retains $5 \times 1.5 = 7.5$ shares and the corresponding dividend entitlement. If an actual holding is about to become a non-integer share count, the whole-share execution simulator rejects the entire path before that day's overnight accrual and corporate action are booked. The error lists the pair, security, date, prior share count, and ratio. Do not remove the failed window and continue reporting a fixed sample. A CLI exception path may leave an empty directory or intermediate files from earlier steps; only a complete successful summary and run status count as evidence of completion.

A non-integer split ratio does not itself trigger rejection: $4 \times 1.5 = 6$ and $5 \times 0.2 = 1$ are supported; a split event on a flat position still has 0 shares. When flat, a 5-share target multiplied by 1.5 creates a 7.5-share order; under the existing whole-share execution rule, 7 shares actually fill and the entry remainder is canceled. This differs from an existing 5-share holding receiving a 7.5-share entitlement.

Exit remainders are worked over day by day, subject to capacity, finite sample length, and the support domain. At the sample end, any unfilled portion is recorded under the existing protocol as opportunity cost and an open episode. The current simulator does not silently mark a half-share residual position as closed. If a real sample produces fractional entitlements, an actual fractional-execution rule or a documented cash-in-lieu basis for quantity, amount, timing, and fees is required, followed by separate acceptance of the corresponding protocol.

This revision accepts only the explicit execution support domain and consistency of share-count conversion. Historical constructed data, preregistration `v1/v2`, cost and threshold grids, and negative results retain their historical identity. The 14 boundary checks and 52 related regressions passed in that historical round; this does not mean that the full CLI grid, Notebook, or market research was rerun. Real empirical work remains blocked, and the complete Project 5 is not verified.

Project 5 — Screening Preparation Layer (Pairs Trading)

Status: **preparation layer, not complete Project 5.** It has no `entry/exit` logic, two-leg costs, short-selling finance, positions, or backtest. Results in this directory must not be described as "a completed pairs-trading strategy" or as evidence that "the screened pairs have a return advantage." The A5 preparation layer has historical limited acceptance; this document inherits that diagnostic status only and does not register a verified real strategy.

What is and is not here

Content	Status	File
Frozen screening rules (alpha, BH q, Engle–Granger trend term, two training windows, and simulation seed) and their SHA-256	Available (local file hash, not external preregistration)	`configs/screening_rules.json`
Report of training-segment correlation and Engle–Granger cointegration screening, hedge ratio, test count, and critical values	Available (the training segment is cut first and passed to the shared helper for validation and screening; missing values after the training segment do not affect whether screening can run. The later window is validated separately for completeness and raises an error if incomplete.)	`screening_layer.py`
Frozen-spread diagnostics after the training segment (half-life and ADF p-value)	Available (diagnostics, not strategy evaluation)	Same file
Counterexamples: correlation ≠ cointegration; hedge ratio ≠ capital weights; false-positive counts under the null; selection effects from "picking the best pair"	Available	Same file; `tests/test_p05_screening.py`
Notebook (cold start) and figure	Available	`notebooks/p05_screening.ipynb`; `figures/a5/p05_screening.png`
Data dictionary, source feasibility, time fields, and gap table	Available	data dictionary, source feasibility, and gap table
`Entry/exit`, risk controls, trading lag, flat-position state	**Not available**	—
Two-leg costs, shorting and financing, borrow availability, mismatch risk	**Not available**	—
Real adjusted prices for stocks or ETFs, corporate actions, historical universe	**Not available (gaps)**	See `gap_table.md`

Real and constructed data

Data or result	Class	Description
Total-return index proxies compounded from French value-weighted monthly returns for 10 industries (1926-07 to 2026-08)	**Real, aggregated, revised, non-point-in-time, non-tradable**	Ten industry portfolios, not securities; they cannot be borrowed or traded. Used to demonstrate training-segment screening and diagnostics. Hash check is recorded in `quant_course.french_frozen.`
Two simulated series from Chapter 17 (common trend without cointegration; cointegrated pair)	Constructed	Used only for the "correlation ≠ cointegration" counterexample
Null calibration from 10 independent random walks and the "pick the best pair" selection effect	Constructed	All pairs are known to be null; used to show false positives and selection bias

Run

The commands below locate the historical A5 screening diagnostic and its tests. They were not rerun in this batch and are not the P5 three-pair constructed Notebook or the full c5 grid. Ordinary readers should start from the project guide; these commands will not overwrite the historical results here.

```
.venv/bin/python projects/project05/preparation/screening_layer.py
.venv/bin/python -m pytest tests/test_p05_screening.py -q
```

Results are written to `qa/a5/claude/results/p05/p05_screening_results.json` and `figures/a5/p05_screening.png`; an existing output is not overwritten. The independent recomputation script is `qa/a5/claude/recompute/p05_recompute.py` (it calls statsmodels coint, adfuller, and numpy directly; it does not call `quant_course.cointegration`).

Observed results (real aggregated data; descriptive only, not strategy evidence)

Test count and critical values. Two training windows; each tests all 45 unordered pairs among 10 industries, for **90 pairwise tests** in total (null and selection-effect simulations are separate and are not included in the real-data test count). Engle–Granger uses the MacKinnon cointegration critical values from statsmodels coint with an intercept: window 1 (training through 1963-06, 444 months), 5% critical value −3.350 (1% −3.921, 10% −3.054); window 2 (training through 1999-12, 882 months), 5% critical value −3.343. The ordinary ADF 5% critical value (about −2.87) is not appropriate for estimated residuals. Bonferroni per-pair p thresholds are `0.05/45` = 0.00111 in each window and `0.05/90` = 0.000556 over all 90 tests (this table does not use them to change classifications). Benjamini–Hochberg q-values are an exploratory FDR adjustment; pairs share series, and independence or PRDS conditions have not been established (see the note in cointegration.py).

Training window	Number of pairs	$p < 0.05$	Expected false positives under the null (45×0.05)	BH $q < 0.10$	$p < 0.00111$
1926-07 to 1963-06	45	5	2.25	1	1
1926-07 to 1999-12	45	6	2.25	0	0

The only pair passing BH. Window 1: Durbl (Durable Goods) and Manuf (Manufacturing): EG statistic -4.871, $p = 0.0003$, $q = 0.0127$; training-segment hedge ratio (log-index-level regression, y = alpha + beta x), beta = 1.126; training log-level correlation 0.994 and monthly log-return correlation 0.895; training half-life about 19 months. **Using frozen alpha and beta in the subsequent window, 1963-07 to 1999-12:** the spread half-life is about 239 months, its standard deviation increases from 0.142 to 0.251, and the ADF p-value for the frozen spread is 0.89 (no evidence of mean reversion). No pair passes BH in window 2.

Null calibration (constructed: 10 independent random walks, 444 months, 200 repetitions). On average, 2.81 pairs per repetition are "significant" at the 5% level (if all p-values are calibrated under the null, the expected count is 2.25 by linearity of expectation; independence is not required); 80% of repetitions have at least one "significant" pair. For BH ($q = 0.10$), the average is 0.335 pairs, and at least one pair passes in 14% of repetitions. **Selection effect:** selecting the pair with the smallest training p-value in each repetition gives a mean training p-value of 0.029, with 81% below 0.05. Using the frozen hedge ratio in the subsequent 120 months, the mean ADF p-value for its spread is 0.476, and only 4% are below 0.05 (all pairs are constructed nulls).

Correlation $\neq$ cointegration (constructed). For a pair with a common trend but no cointegration, level correlation is 0.983, monthly-change correlation is 0.960, and EG $p = 0.339$ (statistic -2.37, 5% critical value -3.35). For a cointegrated pair, level correlation is 0.999, monthly-change correlation is 0.967, and EG $p \approx 10^{-13}$. The change correlations are nearly identical and do not distinguish the pairs. **Hedge ratio $\neq$ capital weights (a separate price-level regression example):** let `P_y` = a + `bP_x` + e, with b = 1.5 (units: `share_x/share_y`), `P_y` = 120, and `P_x` = 40. Go long 1 unit of y (value 120) and short 1.5 units of x (value 60); the absolute capital shares, measured against gross exposure 180, are `2/3` and `1/3`; the signed weights are `+2/3` and `−1/3`, with net dollar exposure 60, not capital at a 1.5-to-1 ratio.

This b is not the elasticity from the log regression above. If the log elasticity beta is 1.5, with Y = 120 and X = 40, going long 1 Y share requires shorting 4.5 X shares to first order (180 dollars), not 1.5 shares. The shared helper's price-level meaning is correct; the beta field name in the historical JSON is retained, and in that example its meaning is this price-level b. Neither the helper nor the historical JSON is changed. This first-order hedge assumes the coefficient is frozen and the residual is unchanged; it does not eliminate all spread risk.

What can and cannot be said. It is reasonable to say that, in real aggregated data for 10 industry portfolios, the training segment screened only one pair that passed BH, and that pair

did not maintain mean reversion in the subsequent window. Under the null, 45 tests produce about 2.8 unadjusted "significant" results on average, so the observed counts of 5 and 6 at $p < 0.05$ cannot by themselves rule out chance. Training significance for the pair selected as "best" does not generalize. It is not reasonable to say that these industry portfolios are tradable pairs, that screening one pair indicates return potential, or anything about costs, financing, shorting, capacity, or the cause of a cointegration breakdown. "Did not pass screening" does not mean "not cointegrated"; test power is limited, and the 444-month, 10-industry sample is also very specific.

Time fields (summary; see `data_dictionary.md` for detail)

`event_time`: calendar month covered by industry-portfolio returns. `available_at`: not recorded in the database; the history is revised, so it is not point-in-time. `signal_time`: month-end of the last month in the training window (1963-06 or 1999-12); screening and the hedge ratio use only data through that date. `trade_time`: undefined (this layer does not trade). return interval: calendar month. label interval: none (screening is on levels and does not use future returns as labels).

Project 5 Preparation-Layer Data Dictionary

A. Data actually used by the preparation layer (real, aggregated, revised)

Field	Meaning	Unit	Primary key	Time field	Missingness and convention
month	Calendar month covered by industry-portfolio returns, PeriodIndex(freq="M")	Month	Row (unique, increasing, contiguous)	`event_time` (the month containing the return interval); **no `available_at`**	The shared monthly validator rejects missing months, duplicates, and disorder in the training segment, and requires `train_end` to exist. The diagnostic segment's index must exactly equal the complete monthly sequence from the month after `train_end` through `test_end`; `validate_monthly_frame` checks its values. Data after `test_end` is not checked.
NoDur…Other (10 columns)	Monthly simple returns for 10 value-weighted industry portfolios in the French database	Decimal	month × industry	Same as above	Revised historical data, not point-in-time vintages; processed-data hash is in `data/manifests/french_industries_2026-10-05.json`
<industry>_TR_proxy	Total-return index proxy formed by compounding monthly returns, starting at 100	Index points	month × industry	Same as above	Descriptive transformation, not a tradable price; `total_return_index_proxy`

Field	Meaning	Unit	Primary key	Time field	Missingness and convention
<industry> (log level)	ln(TR proxy), the object used for screening and the hedge ratio	Log points	month × industry	Same as above	Cointegration is a statement about levels, so it is tested on log levels

B. Screening outputs

Field	Meaning	Unit
`series_1`, `series_2`, `n_train`	Unordered pair (lexicographic order) tested and number of training months	— / months
statistic, pvalue	Engle–Granger statistic and MacKinnon p-value (statsmodels coint, with an intercept)	Dimensionless
`qvalue_bh`	Benjamini–Hochberg adjusted p-value	Dimensionless
alpha, beta	Training-segment log-level OLS: `series_1` = alpha + beta × `series_2`	Log points / dimensionless (**ratio in log-index units, not a capital weight**)
`corr_log_levels_train`, `corr_monthly_log_returns_train`	Training-segment log-level correlation and monthly log-return correlation	Dimensionless
`half_life_train_months`, `half_life_test_months`	AR(1) half-life of the frozen spread in the training and subsequent windows (infinity when phi is not in (0, 1))	Months
`spread_sd_train`, `spread_sd_test`, `test_mean_spread_in_train_sd`	Spread standard deviations and subsequent-window mean relative to the training spread standard deviation	Log points / dimensionless
`adf_pvalue_test_frozen_spread`	ADF p-value for the frozen spread in the subsequent window (the hedge ratio is frozen, so the ordinary ADF p-value is used)	Dimensionless

C. Fields required by complete Project 5 but absent from this layer

Field	Required convention
`adj_close`, dividend, `split_factor`	Security-level adjusted prices and corporate actions, source and license record, hash, unit, and currency
`trade_calendar`	Trading calendar, holidays, and halts; needed to execute at the next tradable time after the close signal
`borrow_available`, `borrow_fee`, `recall_flag`	Borrow availability, fee, and recall by date (`available_at`)
bid, ask, volume	Convention for two-leg costs and capacity
`delisting_return`, `universe_membership`	Historical universe membership, delisting, and total return
`event_time`, `available_at`, `signal_time`, `trade_time`, `return_interval`	Time fields for each input; `available_at` $\leq$ `signal_time` $<$ `trade_time`

Project 5 Gap Table

These are gaps in the A5 screening-preparation layer, not claims that the later `c5/ALIGN` constructed implementation is missing. Real-data, rights, and point-in-time gaps remain; a constructed implementation does not close them.

Missing item	Impact	Treatment in this layer	Who must supply it

Missing item	Impact	Treatment in this layer	Who must supply it
Security-level adjusted prices (including corporate actions)	Real pair screening and backtesting cannot be performed	Demonstrate with aggregated French industry portfolios; label them non-tradable	User supplies or authorizes a data source
Historical universe and delistings	Survivorship bias	No securities are used, so this is not addressed; no claim of unbiased results	Same
Borrow availability, fee, and recall	Cost and feasibility of the short leg	Not implemented; listed under `not_included`	Same
Two-leg cost convention	Net returns	Not implemented	Same
`available_at` (data-availability time)	Point-in-time validity	Not available for French data; declared non-point-in-time	Cannot be supplied
`Entry/exit`, risk controls, trading lag, flat-position state	Strategy layer	Not implemented (required by the contract)	Complete Project 5 phase
Multiple out-of-sample walk-forward windows	Robustness	Gives only "post-training-window diagnostics" for two windows; does not evaluate walk-forward performance	Complete Project 5 phase
Structural-break and cointegration-breakdown tests	Failure windows	Describes only frozen-spread half-life and ADF; no breakpoint test	Complete Project 5 phase
Diagnostics when the subsequent window contains missing values	Frozen-spread ADF and half-life require a complete monthly series	Validate screening and the subsequent window separately. The training segment must include `train_end`; the subsequent window must contain exactly every month from the month after `train_end` through `test_end`, once each. Missing months, missing endpoints, duplicates, an empty window, NaN, or infinity raise an error; no imputation or time compression is used. Data after `test_end` is not included in validation.	The complete Project 5 phase must decide the real-data missingness rule; the preparation layer rejects incomplete input

Project 5 Source Feasibility

This table carries forward the 2026-10-05 frozen snapshot and the source status at the time of the A5 preparation layer. Current vendor terms were not rechecked in this batch. The later 2026-10-10 source decision is also retained with its historical date; institutional use still requires verification against the applicable contract, fields, and rights. See the current project guide.

Need	Candidate source	Status in this layer	Record
Price-level series suitable for testing	French database monthly value-weighted returns for 10 industries (2026-10-05 frozen snapshot)	**Used**: read-only, hash checked	Revised historical data, aggregated portfolios rather than securities, and not borrowable; `quant_course.french_frozen` records the source page and hash
Daily adjusted prices for individual stocks or ETFs	Data source to be chosen by the user (exchange, vendor, or broker)	**Not obtained**	Nothing downloaded; licenses, adjustment conventions, and delisting treatment were **not checked** for each source and must not be assumed; please provide or authorize
Corporate actions (splits, dividends) and historical universe	Same as above	**Not obtained**	Missing; see `gap_table.md`
Borrow availability and fees	Broker or securities-lending market data	**Not obtained**	Typically not public free data; not checked. Without it, no conclusion about short-selling costs can be written
Two-leg cost convention (bid–ask spread, commissions, impact)	Broker execution records or market `bid/ask` quotes	**Not obtained**	Same as above

Conclusion: the preparation layer can demonstrate training-segment screening only with aggregated French data. Complete Project 5 requires user-supplied security-level data and borrow data. Until then, there is no data support for a statement that any particular asset pair can be traded as a pairs trade.

Project 5 — A6 Backtest-Interface Preparation Layer (Constructed Pairs, Not a Completed Project)

Status: the historical A6 constructed interface has limited acceptance. This layer itself has no real pairs, historical borrow or financing data, or complete P5 study. Later `c5/ALIGN` implemented walk-forward analysis and a report using another constructed panel, and Chapters 34–36 have also been accepted; the old A6 gaps must not be described as missing code in the current full P5. The real-market project remains `planned/blocked.`

Run

The commands below are historical A6 maintenance and review entry points, not the current default P5 Notebook in the I3 package. New readers should start from the current project guide and use its unique `RUN_DIR`. A6 results cannot be joined with A5 or c5 results as if they were one experiment. This batch did not rerun A6.

A6 shares a CLI with Project 4:

```
.venv/bin/python projects/project04/backtest_preparation/backtest_interfaces.py
```

Results are stored under the p05 key in the same JSON (`qa/a6/claude/results/p04_p05/p04_p05_backtest_interfaces.json`); the figure is in the right panel of `figures/a6/p04_p05_backtest_interfaces.png`. Test: `tests/test_ch29_project_interfaces.py`.

Why not use the A5 French industry pairs?

The A5 screening layer (`preparation/screening_layer.py`, read-only) uses total-return levels for the French 10 value-weighted industry portfolios: aggregated, revised, non-point-in-time, and not directly tradable. Turning those screening results into "pair PnL" would misstate descriptive diagnostics as tradable returns, which the contract prohibits. This layer instead uses **explicitly labeled constructed tradable prices**: two constructed assets, Y and X, with log X as a random walk and $\log Y = a + 1.5 \log X + u$, where u is AR(1) (phi = 0.95), seed 20261207; the open is the previous close multiplied by a small random gap.

Stage	Method in this layer	Time

Interface contract (P5)

Stage	Method in this layer	Time
Training	First 500 trading days: OLS of log Y on [1, log X] to estimate alpha and beta; freeze the training-segment spread mean and standard deviation; freeze entry at $\|z\| > 2$ and exit at $\|z\| < 0.5$	Training segment only (tests confirmed that price perturbations in the test segment do not change these estimates)
Signal	Starting from the final training close, update the state (flat / long spread / short spread) from z at each daily close; submit orders only when the state changes	`signal_time` = close at t
Capital weights	For a log spread, X dollar exposure = beta × Y dollar exposure. Scale to gross exposure of 1: for a long spread, `w_Y` = 1/(1+beta), `w_X` = −beta/(1+beta). beta is a ratio in log units, **not** a capital weight or share ratio (share ratio = beta · `P_Y/P_X`; the JSON gives the value at first entry)	—
Execution	Chapter 29 `run_backtest`, lag = 1, execution price = constructed open; each leg is charged separately (0 / 10 bps)	`trade_time` = open at t+1
Shorting	`allow_short`=True; borrow fees and rebates are not modeled (constructed assumption)	—

Meaning of the results

The JSON reports the training estimate (the difference between the estimated beta and the true value 1.5), order and fill counts, test-period returns under two fee rates, total fees, and reconciliation residual. Cointegration is guaranteed by design in this constructed world, so positive returns show only that the rules and ledger can run; they do not show that a real pair will be profitable. This layer does not estimate the effects of cointegration breakdown, mismatch, or borrow constraints.

Missing real-data inputs

Unadjusted execution prices for two or more real securities, corporate actions, borrowable quantity and fees, `open/VWAP` execution prices, and two-leg impact; the number of screening searches within the training segment (using the A5 ledger) and multiple testing; rolling walk-forward and cointegration-breakdown monitoring.

Root agent's final rerun: current JSON. The author's historical default output refuses to overwrite; reruns must specify a new output directory.

Chapter 37. A Foundational Stance on Machine Learning for Finance

Central point: predictive accuracy can improve while investment results after costs deteriorate; financial machine learning must evaluate prediction, decision, cost, and evidence on the same ledger. Prerequisites: Chapter 7, Statistical Inference; Chapter 22, Research Methods and Multiple Testing; Chapter 31, Model Validation; Chapter 35, Transaction Costs. After this chapter, you should be able to explain with a hand-computable counterexample why accuracy and net returns can diverge; state how overlapping labels, regime shifts, and multiple attempts affect the strength of evidence; and provide an evaluation checklist covering out-of-sample performance, stability, costs, tails, and economic logic.

1. Learning Objectives

After reading this chapter, you should be able to complete these six verifiable tasks:

1. Given a set of daily returns and long-short positions, calculate directional accuracy, gross returns, turnover, costs, and net returns by hand, and write the identity relating accuracy to returns, $\sum_t p_t r_t = \sum_t |r_t|(2c_t - 1)$ $(p_t = \pm 1)$.
2. Explain four structural challenges in financial prediction—non-stationarity, low signal-to-noise ratio, temporal dependence, and regime shifts—and give one runnable numerical example for each.
3. Calculate the expected in-sample Sharpe ratio of the best of N strategies with zero true returns, and explain why it is not a discovery.
4. Classify data leakage into five types—target leakage, look-ahead features, preprocessing leakage, selection leakage, and overlapping-label leakage—and give one erroneous implementation and its repair for each.
5. Describe how financial problems differ from Kaggle-style competitions in their data-generating process, test sets, and evaluation metrics, and choose baselines accordingly.
6. Set evaluation gates for Chapters 38–40 and Project 6: check out-of-sample performance, stability, costs, tails, and economic logic together.

Completion criteria: independently solve every question in Sections 15–17, with answers consistent with Section 18; explain the inputs, outputs, and limitations of the historical cached results in Section 9. The distinction between the default reader entry point and rerun identity is also described in Section 9; this batch does not rerun simulations.

2. Why Quant Needs This

Suppose you are reviewing a daily directional prediction model. The researcher reports that test-set accuracy in predicting up versus down rose from 52% to 55%. That figure might lead you to approve deployment, or it might lose you money. Accuracy treats every day as equally important, while portfolio returns are booked in proportion to each day's return magnitude. A model can guess correctly on 100 quiet days and miss on 3 crash days, achieving high accuracy while losing money. If the model also reverses its position every day, transaction costs can consume the remaining profit.

Earlier chapters prepared the pieces. Chapter 7 covers sampling error in estimators; Chapter 22 covers freezing research plans and multiple testing; Chapter 31 covers label intervals, purging, and walk-forward evaluation; and Chapter 35 covers costs. This chapter places those pieces in a machine-learning context and asks a more fundamental question: when training a prediction model in finance, what are you optimizing, and why should you believe the result?

This chapter does not train a complex model. Chapter 38 builds a date-based pipeline with fitting confined to training data; Chapter 39 makes controlled comparisons using regularized linear models; Chapter 40 introduces tree models. The conclusions here become their acceptance gates.

From Prediction Scores to an Account: Fix Three Tables First

Keep three tables in view as you read these chapters: an information table records what was available to read when the model was fitted and a decision was made; a prediction table records the dates, assets, and predictions; and a ledger records when trades occurred, how positions changed, and how much cost was paid. This chapter first links the last two tables; Chapter 38 completes the information table; Chapter 39 compares estimators within the same pipeline.

That path also determines the comparison order: first fix the clock and baselines, then select models with training-only validation, and finally report both prediction-level and portfolio-level results for a frozen test period. Seeing higher accuracy does not let you skip execution and costs and read the result directly as higher net returns.

3. Intuition

Start with a ten-day constructed example (for teaching, not market data). One asset has the following open-to-open returns (%) over ten days:

$$r = (+0.4,\ +0.3,\ -0.2,\ +0.5,\ -3.0,\ +0.4,\ +0.2,\ -0.1,\ +0.3,\ -2.5).$$

Days 5 and 10 are two large declines; the other eight days have small moves. Each model predicts a direction every day, and you hold one unit long (+1) or short (−1) according to that direction.

- Model B guesses correctly on all eight small-move days and predicts an increase on both large-decline days. Its accuracy is 80%.

- Model A misses on three small-move days and predicts a decline on both large-decline days. Its accuracy is 70%.

Accuracy alone would lead you to choose B. Look at the account instead: on each of the two large-decline days, B loses the full size of the decline, while A earns that amount. Correct or incorrect calls on small days are worth only a few tenths of a percentage point; on large days they are worth several percentage points. B also reverses its position on Days 3 and 8, so it turns over more than A. Section 8 calculates that, after a cost of 0.1% per unit of turnover, A earns about +5.69% compounded over ten days while B loses about 4.00%.

The example is deliberately small. Its purpose is to make the mechanism visible: accuracy and profitability aggregate the same sequence of correct and incorrect calls with different weights. In real data, fat-tailed returns let a small number of days determine much of the profit and loss, strengthening this mechanism. The simplifying boundaries should also be clear: positions are only ± 1, costs are linear in position changes, and there is no market impact or capacity constraint. The cost models in Chapters 35 and 36 are more complete.

4. Mathematical Foundations

The notation used in this chapter is as follows. All observations are daily, and returns are decimals (0.004 means 0.4%).

Symbol	Meaning	Unit / dimension
$t = 1, \ldots, T$	Trading-day index	Dimensionless
r_t	Simple return from the open on Day t to the open on Day $t+1$	Decimal per day
$\hat{d}_t \in \{-1, +1\}$	Directional prediction made by the model after the close on Day $t-1$	Dimensionless
p_t	Position established at the open on Day t and held until the next open, as a multiple of wealth	Wealth multiple
$c_t = \mathbf{1}\{\text{sign}(p_t) = \text{sign}(r_t)\}$	Whether the direction was correct on Day t	0 or 1
κ	Cost per unit change in position	Decimal (proportion of wealth)
$\tau_t = \lvert p_t - p_{t-1} \rvert$, $p_0 = 0$	Turnover on Day t	Wealth multiple
W_t	Wealth at the end of Day t, with $W_0 = 1$	Currency units (normalized)

Timing convention: the signal time is the close on Day $t-1$, the trade time is the open on Day t, and the return interval runs from the open on Day t to the open on Day $t+1$. A signal formed at the close is not executed at that same close, consistent with the Chapter 29 ledger convention `lag>=1`. This ten-day example uses open execution to make hand calculations easy. Chapters 38 and 39 and Project 6 have only closing prices, so they use "execute at the close of the next trading day after the signal close, with the label starting at the execution close." Both conventions are internally consistent; do not mix them.

Statistical tools used here: the variance of a sample mean, the relationship between correlation and the coefficient of determination (Chapters 7 and 8), and the maximum of normal

random variables (Chapter 32; multiple-testing principles in Chapter 22). If the expression for $\mathrm{Var}(\bar{x})$ under serial correlation is unfamiliar, review the Newey–West section in Chapter 32.

5. Formal Definitions

Prediction and decision. A prediction is a mapping from an information set to an estimate of a target variable, $f : \mathcal{F}_{t-1} \to \hat{y}_t$; a decision maps a prediction (and costs and risk constraints) to a position, $g : (\hat{y}_t, p_{t-1}, \text{constraints}) \to p_t$. Investment results are determined by $p_t r_t - \kappa\tau_t$, so they depend on g, not only on f. This distinction matters because the same f paired with different g can produce very different net returns. Model evaluation must state whether it evaluates f or the combination of f and g.

Directional accuracy / hit rate. On days when $p_t \neq 0$ and $r_t \neq 0$, $\mathrm{Acc} = \frac{1}{T'}\sum_t c_t$. Conditions: define it only on days with a position and nonzero returns; exclude zero-return days, where "direction" is meaningless. It carries over the output of a classification model and gives equal weight to every day.

Magnitude-weighted accuracy. $\mathrm{Acc}_{|r|} = \sum_t |r_t| c_t / \sum_t |r_t|$. This auxiliary quantity is introduced to connect accuracy with the gross return of a ± 1 position; the next section derives the relationship.

Boundary of the teaching-proxy ledger. This chapter uses the absolute change in target position, $|p_t - p_{t-1}|$, as a proxy for turnover. It does not process each trade, position drift, or target rebalancing after costs, and therefore is not equivalent to the exact self-financing ledger in Chapter 29. The return sums and compounded wealth below are calculated within this explicitly stated proxy model.

Net returns and the wealth path. Net return on Day t is $n_t = p_t r_t - \kappa\tau_t$, and wealth evolves as $W_t = W_{t-1}(1 + n_t)$. T daily returns correspond to $T + 1$ wealth points, $W_0, \ldots, W_T$. The example in this chapter excludes the exit cost on an open position at the end of the period; if included, add $\kappa|p_T|$.

Non-stationarity. The process is non-stationary if the joint distribution of (x_t, y_t) changes with t. A prediction-relevant special case is a conditional mean $E[y_t \mid x_t]$ whose shape changes over time, such as a changing linear slope b_t. The stationarity in Chapter 16 concerns the moments of an individual series; here the concern is whether the prediction relationship itself remains stable.

Regime shift. One form of non-stationarity: there is a date t^* and the prediction relationship is a different constant before and after it, with $b_t = b^{(1)}$ for $t < t^*$ and $b_t = b^{(2)}$ for $t \geq t^*$. The change point is not known in advance.

Signal-to-noise ratio. For $y = bx + \varepsilon$, where x and ε are independent, the signal-to-noise ratio is $\mathrm{SNR} = b^2 \mathrm{Var}(x)/\mathrm{Var}(\varepsilon)$; the population coefficient of determination is $R^2 = \mathrm{SNR}/(1 + \mathrm{SNR}) = \rho^2$, where ρ is the correlation between x and y. In financial-return prediction, ρ is usually only a few percent and R^2 is on the order of a few thousandths (see the evidence limits in Section 11).

Temporal dependence. Observations are correlated with one another. The positive correlation in the overlap structure studied here reduces the effective information available for estimating the mean, although under negative autocorrelation the effective sample size can also exceed the number of observations. The focus here is **overlapping labels**: computing a future h-day return label every day means adjacent labels share $h-1$ daily returns.

Multiple testing and selection bias. Try N configurations and report the best one, and the expected reported value will exceed the true value of any one configuration. Chapter 22 already covers FWER, FDR, and trial ledgers; here we use the idea to explain that tuning and changing features are also trials.

Data leakage. Data leakage occurs when information unavailable at prediction time is used to train or select a model. This chapter classifies five routes by which the information enters: target leakage (a feature is constructed from the label itself or its future portion); look-ahead features (the feature becomes available after the signal time); preprocessing leakage (scaling, imputation, or dimensionality reduction uses test data); selection leakage (the test set is used to select a model, feature, or threshold); and overlap leakage (training-label intervals intersect test-label intervals). All five mix information unavailable at deployment into test performance.

Baseline. A simple rule compared with a candidate model on the same sample, under the same timing convention and costs. This book uses three defaults: a zero prediction (no position), the training-mean prediction, and a simple rule (such as buy-and-hold or the momentum rule in Chapter 25). An out-of-sample number without a baseline cannot be interpreted.

Out-of-sample R^2. For test observations y_t, predictions $\hat{y}_t$, and baseline predictions $\bar{y}_t^{\text{base}}$, define $R^2_{\text{OS}} = 1 - \sum_t (y_t - \hat{y}_t)^2 \big/ \sum_t (y_t - \bar{y}_t^{\text{base}})^2$. With the training mean as baseline, it compares whether "the model is better than the historical mean" (Campbell–Thompson's definition for index returns); with zero as baseline, it compares whether "the model is better than making no prediction" (the definition for individual-stock returns in Gu–Kelly–Xiu, motivated by how noisy an individual stock's historical mean is). The two baselines give different numbers; state which one is used.

These concepts have a logical order: the prediction–decision distinction tells you what to evaluate; non-stationarity, low signal-to-noise ratio, and temporal dependence explain why evidence is weak; multiple testing and leakage explain how evidence can be artificially inflated; and baselines and R^2_{OS} provide a reference for comparison.

6. Derivations

6.1 The Identity Between Accuracy and Gross Returns

Assumptions: $p_t \in \{-1, +1\}$ and $r_t \neq 0$.

First, write one-day gross return according to whether the direction is correct. If $c_t = 1$, p_t and r_t have the same sign, so $p_t r_t = |r_t|$; if $c_t = 0$, they have opposite signs, so $p_t r_t = -|r_t|$. Combining the two cases gives

$$p_t r_t = |r_t| \, (2c_t - 1).$$

Second, sum over t:

$$\sum_{t=1}^{T} p_t r_t = \sum_t |r_t|(2c_t - 1) = 2\sum_t |r_t| c_t - \sum_t |r_t| = \Big(\sum_t |r_t|\Big)\big(2\,\mathrm{Acc}_{|r|} - 1\big).$$

The last equality follows from the definition of magnitude-weighted accuracy. Gross returns are positive if and only if $\mathrm{Acc}_{|r|} > 1/2$. When all $|r_t|$ are equal, ordinary accuracy Acc equals weighted accuracy for any pattern of correct and incorrect predictions. When magnitudes differ, it generally cannot substitute for weighted accuracy, although the two can happen to be equal. For example, with magnitudes $(1, 2, 3, 4)$ and only the first and fourth predictions correct, both accuracies are $1/2$. If flat positions are allowed, both accuracy measures must be restricted to the same evaluation subset on which there is a position and a nonzero return; the identity above still applies only to ± 1 positions on that subset.

Use: This identity shows that classification loss (0–1 loss) and investment loss are not the same objective. When you train a directional model with cross-entropy or accuracy, errors on large- and small-move days receive equal weight; the investor's cost is weighted by $|r_t|$. If you want to train a directional model, you can weight observations by $|r_t|$ in the loss or directly regress returns. **When it does not apply:** if position size varies with prediction confidence, then $p_t \neq \pm 1$ and the identity does not hold; return to $\sum p_t r_t$ itself.

6.2 Costs Separate Net Returns

$$\sum_t n_t = \sum_t p_t r_t - \kappa \sum_t \tau_t, \qquad \tau_t = |p_t - p_{t-1}| \in \{0, 1, 2\}.$$

Reversing a ± 1 position from long to short gives $\tau_t = 2$: first sell one unit of the long position, then sell one unit short. If directional predictions are approximately random, the probability of a reversal is about $1/2$, and average daily turnover is about 1, or about 252 units annualized. At $\kappa = 0.1\%$, annual costs are about 25%. This is what Chapter 35 means when it says that "costs are a function of trading frequency." Summation is only an approximation; the exact value uses $W_T = \prod(1 + n_t)$. Their difference consists of second-order terms and is about 0.1 percentage point in the ten-day example (Section 8).

6.3 Signal-to-Noise Ratio and R^2

Assumptions: $y = bx + \varepsilon$, $E[\varepsilon] = 0$, ε is independent of x, $\mathrm{Var}(x) = \sigma_x^2$, and $\mathrm{Var}(\varepsilon) = \sigma_\varepsilon^2$.

$$\mathrm{Var}(y) = b^2\sigma_x^2 + \sigma_\varepsilon^2, \qquad R^2 = \frac{b^2\sigma_x^2}{b^2\sigma_x^2 + \sigma_\varepsilon^2} = \frac{\mathrm{SNR}}{1 + \mathrm{SNR}}.$$

Also, $\mathrm{Cov}(x, y) = b\sigma_x^2$ and $\rho = b\sigma_x/\sqrt{b^2\sigma_x^2 + \sigma_\varepsilon^2}$, so $\rho^2 = R^2$. A correlation of 0.05 corresponds to $R^2 = 0.0025$, or 0.25%.

This does not mean the signal is useless. Chapter 23 already shows that a portfolio constructed cross-sectionally with weights $w_i = z_i/N$ on a standardized signal z_i has a one-period return equal to $\mathrm{IC}_t \cdot \sigma_{y,t}$, where $\sigma_{y,t}$ is the cross-sectional standard deviation of returns at time t. If IC stays near 0.05 each period with little variation, portfolio returns can be stable even though each stock's R^2 is only 0.25%. A low signal-to-noise ratio changes the **amount of evidence**: you need many independent cross sections and many periods to distinguish 0.05 from 0; at the same time, even small amounts of leakage or overfitting can fabricate an apparent 0.05.

6.4 Autocorrelation and Effective Sample Size for Overlapping Labels

Assumptions: Daily returns u_t are independent and identically distributed with variance σ^2; the h-day label on Day t is $Y_t = \sum_{j=1}^{h} u_{t+j}$.

Y_t and Y_{t+k} $(0 \le k < h)$ share $h - k$ daily returns:

$$\mathrm{Cov}(Y_t, Y_{t+k}) = (h-k)\sigma^2, \qquad \mathrm{Var}(Y_t) = h\sigma^2, \qquad \rho_k = \frac{h-k}{h}.$$

For $k \ge h$, they no longer overlap, so $\rho_k = 0$. For the sample mean $\bar{Y} = \frac{1}{n}\sum_{t=1}^{n} Y_t$, its variance (when $n \gg h$) is

$$\begin{aligned}\mathrm{Var}(\bar{Y}) &\approx \frac{\mathrm{Var}(Y_t)}{n}\Big(1 + 2\sum_{k=1}^{h-1}\rho_k\Big) \\ &= \frac{h\sigma^2}{n}\Big(1 + 2\sum_{k=1}^{h-1}\frac{h-k}{h}\Big) = \frac{h\sigma^2}{n}\cdot h.\end{aligned}$$

The second equality uses $\sum_{k=1}^{h-1}(h-k) = h(h-1)/2$, so the term in parentheses equals $1 + (h-1) = h$. Compared with the independent-sample formula $h\sigma^2/n$, the variance is inflated by a factor of h, and the **effective sample size** is about n/h. With $h = 5$, treating 1,000 overlapping labels as independent when calculating a t statistic would inflate the t statistic by about $\sqrt{5} \approx 2.24$. Under the known construction from independent daily returns, the correction is to use the exact covariance or mean-variance formula derived here, or to keep only non-overlapping dates. Truncating at the known MA order without kernel tapering is not the same estimator as fixed-bandwidth Bartlett Newey–West. For the shared implementation with NW lag $q = h - 1$, weights are $1 - k/h$ and its limiting factor is $1 + 2\sum_{k=1}^{h-1}(1 - k/h)\rho_k = (2h^2 + 1)/(3h)$; at $h = 5$ this is 3.4, not the long-run variance factor 5. Covering all nonzero covariance lags does not

guarantee that a fixed Bartlett bandwidth exactly recovers the overlap correction or is consistent; see Chapters 23 and 32 for general bandwidth and consistency conditions.

For finite $n \geq h$, the exact variance inflation factor for the sample mean is $1 + 2\sum_{k=1}^{h-1}(1 - k/n)\rho_k = h - (h^2 - 1)/(3n)$; effective sample size is defined as n divided by this factor. For a general series, if the sum of the weighted autocovariances is negative, the factor can be below 1 and the effective sample size can exceed n.

Scope: if daily returns themselves are autocorrelated or exhibit volatility clustering, ρ_k will differ from $(h - k)/h$; the inflation factor will no longer be exactly h. Newey–West can still be used, but the lag choice needs separate consideration.

6.5 The Maximum Sharpe Ratio Across Multiple Attempts

Assumptions: The daily returns of N strategies are independent, each has a true mean of 0, and each strategy has T observations. The estimated daily Sharpe ratio $\widehat{SR}_d$ is approximately distributed as $N(0, 1/T)$, and the annualized estimate $\widehat{SR} = \sqrt{252}\,\widehat{SR}_d \approx N(0, 252/T)$.

$$E\Big[\max_{1\leq i\leq N} \widehat{SR}_i\Big] \approx \sqrt{\frac{252}{T}}\, E\Big[\max_{1\leq i\leq N} Z_i\Big], \qquad Z_i \overset{iid}{\sim} N(0,1).$$

The first approximation comes from the normal approximation. $E[\max Z]$ can be computed by numerical integration (the exact integration function in Chapter 32's `research_protocol` module) or by the Bailey–López de Prado approximation $(1-\gamma)\Phi^{-1}(1-1/N)+\gamma\Phi^{-1}(1-1/(Ne))$, where $\gamma \approx 0.5772$. With $T = 504$ (two years), $\sqrt{252/504} = 0.707$; for $N = 100$, $E[\max Z] \approx 2.51$, so the best strategy's expected annualized Sharpe ratio is about 1.77. All strategies have a true Sharpe ratio of 0.

Use: when you see "the best model selected from 100 feature combinations had an annualized Sharpe of 1.8," first compare it with 1.77. It is nearly the expected value under the zero null and is not a discovery. Chapter 32 discusses the Deflated Sharpe Ratio; trial registration and multiple-testing principles are covered in Chapter 22. The expected maximum is a reference point, not an exact post-selection p-value or FWER.

6.6 Out-of-Sample R^2 Under a Regime Shift

Assumptions: $y_t = b_t x_t + \varepsilon_t$, with $x_t, \varepsilon_t \overset{iid}{\sim} N(0,1)$ and independent; the slope is $+b$ in Regime 1 and $-b$ in Regime 2; use the OLS estimate $\hat{b}$ from Regime 1 (with intercept approximately 0) to predict Regime 2, with zero prediction as the baseline.

$$E[(y - \hat{b}x)^2] = E[(-bx + \varepsilon - \hat{b}x)^2] = (b + \hat{b})^2 + 1, \qquad E[y^2] = b^2 + 1.$$

Conditional on the fitted fixed $\hat{b}$, the population risk ratio is $1 - [(b + \hat{b})^2 + 1]/(b^2 + 1)$. Finite-test-sample R^2_{OS} is a ratio of two sample sums of squares, and its expectation is not equal to the population risk ratio; ignoring the fitted intercept is also an approximation. Thus,

$R^2_{\text{OS}} \approx 1 - \dfrac{(b+\hat{b})^2+1}{b^2+1}$. If $\hat{b} \approx b$, then $R^2_{\text{OS}} \approx -\dfrac{3b^2}{1+b^2}$; for $b = 0.1$, this is about -2.97%. As $\hat{b}$ moves from b toward $-b$, $(b+\hat{b})^2$ falls; $\hat{b} < b$ alone is not enough to ensure lower loss, and moving farther past $-b$ increases the loss again. The point is that when a prediction relationship reverses, the model performs worse than making no prediction, while in-sample R^2 will not warn you. A rolling window can track the change, but the shorter the window, the greater the estimation noise; Chapter 37-C01 asks you to weigh this trade-off.

7. Financial Interpretation

Why large days determine profit and loss. Daily returns are fat-tailed: the absolute returns on a small number of days are far larger than those on a typical day (Chapters 11 and 33). The sum $\sum_t p_t r_t$ is dominated by those days, while accuracy weights them the same as quiet days. A model that learns that quiet days rise slightly can have high accuracy; its errors on crisis days can cost far more than its ordinary gains. The tail losses emphasized in risk management (Chapter 33) are precisely what accuracy cannot see.

Why a low signal-to-noise ratio is the norm. If a publicly available signal could predict returns reliably and by a large amount, traders would buy assets predicted to rise until the expected returns were absorbed into prices. Any remaining discoverable predictability is therefore small or requires risk or costs as compensation (the factor-premium explanation in Chapters 14 and 24–27). So a "very low R^2" is not itself a model flaw in finance; a "very high R^2" should first make you suspect leakage.

Why relationships change. Market participants learn and exploit published patterns; regulation, market structure, and interest-rate regimes change; and capital flowing into the same strategy compresses its returns. These all make $E[y \mid x]$ move over time. Chapter 37-C01 considers the training-window length to use in response.

How this differs from Kaggle-style competitions. Use competitions that allow repeated public-leaderboard submissions and assume approximately identically distributed data as a simplified contrast; individual competitions may instead have time-based splits, submission limits, and distribution shifts. In finance, you have only one historical path, and a test period can truly be used only once; distributions drift over time; the objective is risk-adjusted returns after costs rather than a prediction metric; and your trades affect prices and costs (Chapters 35 and 36). Common competition techniques such as random shuffling followed by K-fold cross-validation or repeated tuning against a public leaderboard can introduce overlap leakage under the overlapping-label structure studied here, while repeated testing and tuning introduce selection leakage. Diagnose the specific problems with random splits from the information and label structure.

Implementability. A model's value depends on whether its predictions can be computed before `trade_time`, traded at a reasonable cost, and scaled under capacity and borrow constraints. These are not included in the model's loss function and must be added to the evaluation.

Chapter 37 complete numerical example: all steps and conclusions

8. Numerical Example

8.1 Ten-Day Case: Inputs

(Constructed for teaching.) Daily returns r are as in Section 3; cost is $\kappa = 0.001$ (0.1% per unit of turnover); $p_0 = 0$. In the table, p^B, p^A, and p^{BH} are the positions for Model B, Model A, and buy-and-hold, respectively.

Day t	1	2	3	4
r_t (%)	+0.4	+0.3	−0.2	+0.5
p^B	+1	+1	−1	+1
p^A	+1	+1	+1	+1
p^{BH}	+1	+1	+1	+1

Day t	5	6	7	8
r_t (%)	−3.0	+0.4	+0.2	−0.1
p^B	+1	+1	+1	−1
p^A	−1	−1	−1	−1
p^{BH}	+1	+1	+1	+1

Day t	9	10
r_t (%)	+0.3	−2.5
p^B	+1	+1
p^A	+1	−1
p^{BH}	+1	+1

8.2 Hand-Calculating Accuracy and Gross Returns

$\sum_t |r_t| = 0.4 + 0.3 + 0.2 + 0.5 + 3.0 + 0.4 + 0.2 + 0.1 + 0.3 + 2.5 = 7.9$ (%).

- B: wrong on Days 5 and 10, so $\sum_{\text{wrong}} |r| = 3.0 + 2.5 = 5.5$. Acc $= 8/10 = 0.80$. Gross return $= (7.9 - 5.5) - 5.5 = -3.1\%$. $\mathrm{Acc}_{|r|} = 2.4/7.9 = 0.304$.
- A: wrong on Days 3, 6, and 7, so $\sum_{\text{wrong}} |r| = 0.2 + 0.4 + 0.2 = 0.8$. Acc $= 0.70$. Gross return $= (7.9 - 0.8) - 0.8 = +6.3\%$. $\mathrm{Acc}_{|r|} = 7.1/7.9 = 0.899$.
- Buy-and-hold: wrong on the 4 down days, so $\sum_{\text{wrong}} |r| = 0.2 + 3.0 + 0.1 + 2.5 = 5.8$, Acc $= 0.60$, and gross return $= 2.1 - 5.8 = -3.7\%$.

Check A against the identity $\sum p_t r_t = (\sum |r_t|)(2\mathrm{Acc}_{|r|} - 1)$: $7.9 \times (2 \times 0.899 - 1) = 7.9 \times 0.797 = 6.30$.

8.3 Turnover and Costs

- B: $p_0 = 0 \to +1$ (1), Day 3 $+1 \to -1$ (2), Day 4 $-1 \to +1$ (2), Day 8 (2), and Day 9 (2). Total: 9 units; cost: $9 \times 0.1\% = 0.9\%$.
- A: initial position 1, Day 5 $+1 \to -1$ (2), Day 9 (2), and Day 10 (2). Total: 7; cost: 0.7%.
- Buy-and-hold: 1; cost: 0.1%.

Continued: the same complete example

8.4 Net Returns: Approximate and Exact

	Acc	$\text{Acc}_{\lvert r\rvert}$	Sum of gross returns	Turnover
A	0.70	0.899	+6.30%	7
B	0.80	0.304	−3.10%	9
Buy-and-hold	0.60	0.266	−3.70%	1
Zero position	—	—	0	0

	Cost	Additive approximate net return	Exact compounded net return
A	0.70%	+5.60%	+5.6879%
B	0.90%	−4.00%	−3.9980%
Buy-and-hold	0.10%	−3.80%	−3.8072%
Zero position	0	0	0

The exact column comes from $W_{10} = \prod_{t=1}^{10}(1 + n_t)$. For example, for A on Day 1, $n_1 = 0.004 - 0.001 = 0.003$, so $W_1 = 1.003$; for B on Day 3, $n_3 = (-1)(-0.002) - 2 \times 0.001 = 0$, so $W_3 = W_2 = 1.006009$. The complete wealth path is stored in `course/zh/chapters/ch37/figures/ch37_results.json`; its 11 wealth points correspond to 10 daily returns. Model B, despite having the highest accuracy, loses to the zero-position baseline as well as to buy-and-hold.

8.5 A Hand Calculation for the Scale of R^2

If a feature has correlation 0.03 with next-day returns, then $R^2 = 0.03^2 = 0.0009$. If the daily return standard deviation is 2%, the standard deviation of the prediction component in a linear projection with an intercept is $|\rho|\sigma_y = 0.0006 = 0.06\%$. This is not an upper bound on the predictable component for arbitrary nonlinear models, nor is it expected trading profit. A cost of 0.1% per unit of turnover cannot be directly subtracted from a prediction standard deviation as if they were the same kind of return; net returns also depend on the position rule, reversal probability, holding period, and threshold. Diversification across assets, smoother positions, or lower-frequency trading may improve after-cost performance, but it still has to be checked against the actual ledger.

9. Python Implementation

Script: `code/ch37_ml_philosophy.py`. The historical engineering script can recompute the constructed experiments; even if a new `--out` is specified, the working-tree version may overwrite the chapter's original figures, so this chapter does not provide a direct overwrite instruction. The Chapter 37 Notebook in the default reader package recomputes the ten-day case, 100,000 overlapping days, three groups of 200 zero-signal experiments each, and the regime shift, then writes the ten-day wealth chart in a unique run directory; this batch explains that behavior only and did not execute it. Historically, the CLI calculated $N = 1$ first and then 10/100/1000, while the Notebook started at 10, so they consume random numbers differently. The same seed, 3702, does not mean the two paths produce the same individual results. The Notebook's English-language layer is scheduled for BIL23.

Inputs consist only of teaching parameters defined inside the script and fixed seeds (3701 for the regime shift, 3702 for multiple testing, and 3703 for overlapping labels); outputs are `ch37_results.json` (which refuses to overwrite an existing file) and three figures. Data frequency is daily. In the ten-day case, `signal_time` is the previous day's close, `trade_time` is the current day's open, and the return interval runs from the open to the next open. Missing-data policy: if an input contains NaN or inf, raise an error; do not impute.

Core functions:

- `decision_ledger(returns, positions, cost_per_unit)`: returns daily gross returns, turnover, net returns, and $T + 1$ wealth points.
- `magnitude_identity(returns, positions)`: checks the identity in Section 6.1.
- `overlap_autocorrelation(h, n_days, seed)`: sample autocorrelations and theoretical values $(h - k)/h$ for overlapping labels.
- `max_null_sharpe(n_strategies, n_days, n_sims, seed)`: best in-sample Sharpe under zero true signal and the selected strategy's holdout Sharpe; reuses Chapter 32's `quant_course.research_protocol.expected_max_standard_normal(_exact)`.
- `regime_shift(n_per_regime, slope, seed)`: trains in Regime 1 and evaluates in Regime 2.

Tests: `tests/test_ch37_philosophy.py`. The following snippet can be run on its own to check the ten-day case:

```
import numpy as np

r = np.array([0.004, 0.003, -0.002, 0.005, -0.030, 0.004, 0.002, -0.001, 0.003,
    ↪ -0.025])
models = {
    "A": np.array([1, 1, 1, 1, -1, -1, -1, -1, 1, -1]),
    "B": np.array([1, 1, -1, 1, 1, 1, 1, -1, 1, 1]),
}
kappa = 0.001
for name, p in models.items():
    turnover = np.abs(np.diff(np.concatenate([[0], p])))
    net = p * r - kappa * turnover
    wealth = np.concatenate([[1.0], np.cumprod(1 + net)])
    acc = np.mean(np.sign(p) == np.sign(r))
    assert np.isclose((p * r).sum(), (np.abs(r) * (2 * (np.sign(p) == np.sign(r)) -
    ↪ 1)).sum())
    print(name, f"acc={acc:.2f}", f"turnover={turnover.sum()}", f"net={wealth[-1] -
    ↪ 1:+.6f}", f"points={wealth.size}")
```

Expected output: `A acc=0.70 turnover=7 net=+0.056879 points=11` and `B acc=0.80 turnover=9 net=-0.039980 points=11`.

Other values from the historically frozen run (`ch37_results.json`):

- Overlapping labels, $h = 5$, 100,000 daily samples: sample autocorrelations $(0.802, 0.604, 0.403, 0.203, 0.004)$; theoretical values $(0.8, 0.6, 0.4, 0.2, 0)$.
- Multiple testing, $T = 504$ days and 200 simulations: for $N = 10, 100, 1000$, simulated mean best in-sample annualized Sharpe ratios were 1.138, 1.814, and 2.304; formula values were 1.088, 1.773, and 2.292; mean Sharpe ratios of the selected strategies in equal-length holdout periods were -0.094, -0.052, and $+0.105$. The zero-signal construction makes the population expectation for an independent holdout period 0, but finite Monte Carlo means can differ; do not characterize all three groups alike as "indistinguishable from 0." For cached $N = 1000$, the mean over 200 runs was 0.10456881, the descriptive standard error was 0.04797481, and the ordinary normal-approximation 95% interval was about [0.01054, 0.19860]. This is a post hoc description of cached results, not a new preregistered test and not market alpha.
- Regime shift, 1,000 observations per regime and $b = 0.1$: the estimated Regime 1 slope was $+0.0682$; in Regime 2, R^2_{OS} relative to zero prediction was -1.95%; within each regime the population R^2 was $0.01/1.01 = 0.99\%$. Substituting $\hat{b} = 0.0682$ into the formula from Section 6.6 gives $1 - [(0.1682)^2 + 1]/1.01 \approx -1.81\%$. The difference from the simulated result comes from finite-sample variation in x and intercept estimation.

10. Visualization

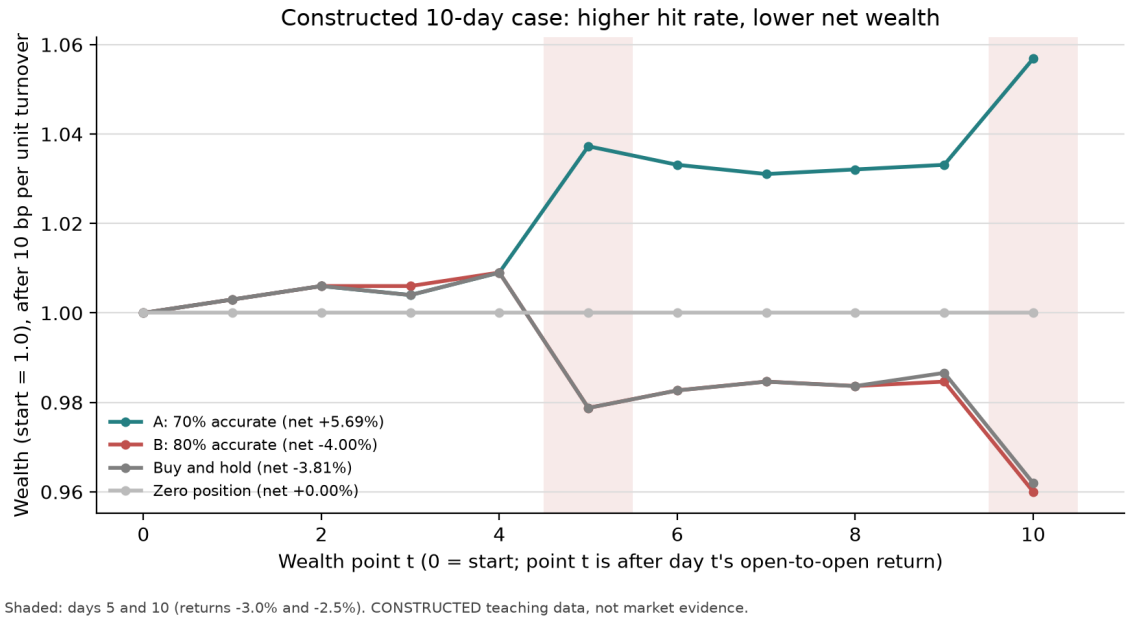


Figure 37-1. Wealth path in the ten-day constructed case

Figure 37-1: The horizontal axis is the wealth-point index (0 is the starting point; point t follows the return on Day t); the vertical axis is wealth after deducting a 0.1% cost per unit of turnover, with starting wealth 1.0. Shading marks the two large declines on Days 5 and 10. Observation: B and buy-and-hold fall together on both large-decline days, while A rises on those same two days; B has the highest accuracy and the lowest ending wealth. Constructed teaching data.

Figure 37-2. Rolling slope under a regime shift

Figure 37-2: The horizontal axis is the observation index at the end of a rolling window; the vertical axis is the OLS slope of y on x over a 250-observation window (change in y per unit of

x). The dashed line is the true slope, which changes from +0.1 to −0.1 at observation 1,000. Observation: after the shift, the rolling estimate takes about one window length to catch up; using the old slope before then loses money. The title gives R^2_{OS} for training in Regime 1 and evaluating in Regime 2. Constructed data.

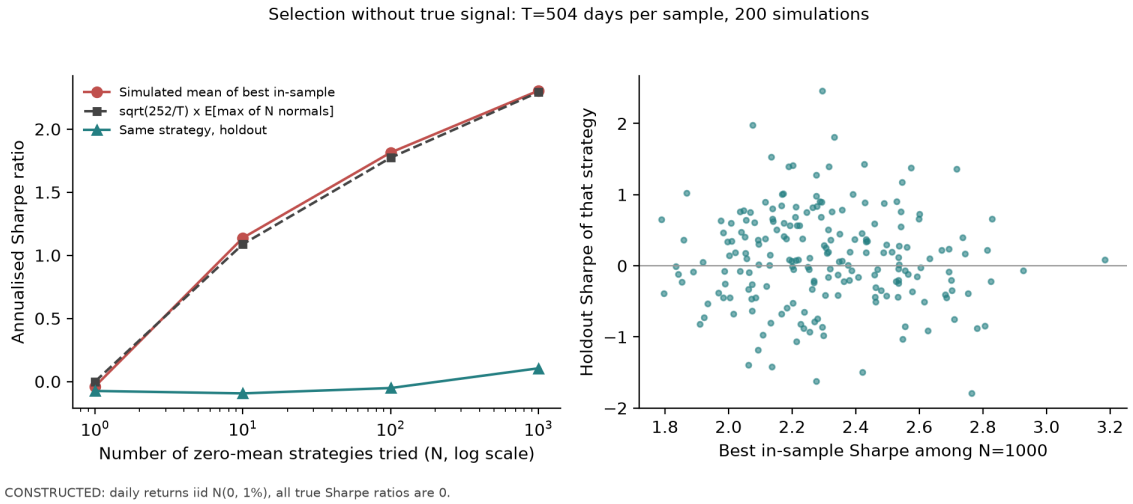


Figure 37-3. Best Sharpe ratio under zero true signal

Figure 37-3: In the left panel, the horizontal axis is the number N of zero-return strategies tried (log scale) and the vertical axis is annualized Sharpe ratio. The red line is the mean best in-sample Sharpe over 200 simulations; the dashed line is $\sqrt{252/T}\,E[\max Z_N]$; the green line is the mean holdout Sharpe of the selected strategy. In the right panel, each dot for $N = 1000$ shows one simulation's pair (best in-sample Sharpe, holdout Sharpe). Observation: the in-sample best rises with $\log N$, while holdout Sharpe values scatter around 0 without regard to in-sample level. Constructed data; all true Sharpe ratios are 0.

11. Real-Market Example

This chapter reports no measured real-market results. The reason is that the course currently has no licensed daily individual-stock price panel available for internal training (the Project 5 data decision documented terms restrictions for sources such as Yahoo, Alpha Vantage, and Tiingo); the real-data status for Project 6 is blocked. Without data, this section states only how a real test should be conducted and what can be supported by the literature.

The test to conduct once data are available. Fix the stock universe and sample period in advance; use features available at the close to predict returns from the next trading day's open to open; and, in a date-based walk-forward test, report together: directional accuracy, magnitude-weighted accuracy, IC and R^2_{OS} (relative to both 0 and the training mean), returns of a prediction-based long-short portfolio before and after costs, turnover, maximum drawdown, and the tail (worst 1% of days). If accuracy exceeds 50% while after-cost returns are not positive, that reproduces the mechanism in this chapter's ten-day example.

What the literature can support. The following are passages the author actually read in the cited papers (see `source_notes.md` for details). They use other authors' samples and definitions; they are not measurements from this book:

- Gu, Kelly, and Xiu (2020, RFS) conduct large-scale machine-learning predictions of monthly excess returns on U.S. equities. Equation (19) (p. 2249) defines out-of-sample R^2 relative to a **zero prediction**, without subtracting the mean; the authors explain that for individual stocks, historical-mean predictions are typically much worse than zero predictions, and switching to the historical mean as the baseline raises monthly R^2 by about three percentage points for all methods. In Table 1 (p. 2250), OLS with all features has -3.46%; OLS using only size, value, and momentum has 0.16%; and elastic net with Huber loss has 0.11%. The text reports that a three-layer neural network reaches about 0.40%. In the introduction, the same paper reports annualized out-of-sample Sharpe ratios of 1.35 (value-weighted) and 2.45 (equal-weighted) for a decile long-short portfolio sorted by neural-network predictions. These figures illustrate two things at once: individual-stock monthly R^2 is below 1%, and low R^2 can coexist with a substantial portfolio Sharpe ratio. These are before-cost figures; the equal-weighted result is more sensitive to small-cap stocks and costs.
- Campbell and Thompson (2008; NBER working paper w11468 version, Equation (1), p. 5) define market-index-return R^2_{OS} relative to the historical mean through the previous period, and note that even a small out-of-sample R^2 can be economically meaningful to investors.

The two papers use different baselines: zero for cross-sectional stock predictions, and the historical mean for index timing. Chapters 38 and 39 and Project 6 in this book report both baselines and state what each means.

12. Common Mistakes

12.1 Choosing a Trading Model by Accuracy

Error: rank models on test-set accuracy and select B. Why it is wrong: accuracy weights each day equally, while trading returns are weighted by $|r_t|$ (Section 6.1).

```
import numpy as np
r = np.array([0.004, 0.003, -0.002, 0.005, -0.030, 0.004, 0.002, -0.001, 0.003,
    ↪ -0.025])
A = np.array([1, 1, 1, 1, -1, -1, -1, -1, 1, -1]); B = np.array([1, 1, -1, 1, 1, 1,
    ↪ 1, -1, 1, 1])
acc = lambda p: np.mean(np.sign(p) == np.sign(r))
wrong_choice = max(("A", A), ("B", B), key=lambda kv: acc(kv[1]))[0]          # Wrong
    ↪ : choose by accuracy
pnl = lambda p: (p * r).sum() - 0.001 * np.abs(np.diff(np.concatenate([[0], p]))).sum
    ↪ ()
right_choice = max(("A", A), ("B", B), key=lambda kv: pnl(kv[1]))[0]           # Fix:
    ↪  choose by after-cost return
print(wrong_choice, right_choice, round(pnl(A), 4), round(pnl(B), 4))          # B A
    ↪ 0.056 -0.04
```

Fix: evaluate with an objective aligned to the decision and report at least accuracy, magnitude-weighted accuracy, and after-cost returns.

12.2 Ignoring Turnover

Error: report only gross returns. Why it is wrong: a model that reverses direction has high turnover; in the ten-day example, B's 0.9% cost exceeds A's 0.7%, and an illustrative annualization must first divide the ten-day cumulative cost by 10 to obtain a daily mean, then multiply by 252. Do not multiply the ten-day cumulative difference directly by 252; this extrapolation does not guarantee a constant future turnover rate. Fix: include turnover and costs in the same evaluation table; use the cost definition from Chapter 35.

12.3 Selecting the Best Strategy on the Full Sample, Then Reporting It on the Same Sample

Erroneous code (zero true signal, 100 strategies; choose the best and report its in-sample Sharpe):

```
import numpy as np
rng = np.random.default_rng(0)
ins, outs = rng.normal(0, 0.01, (504, 100)), rng.normal(0, 0.01, (504, 100))
sr = lambda x: np.sqrt(252) * x.mean(0) / x.std(0, ddof=1)
k = int(np.argmax(sr(ins)))
print("reported (in-sample) Sharpe:", round(float(sr(ins)[k]), 2))     # Wrong: about
    ↪ 1.5–2.2
print("same strategy, holdout:", round(float(sr(outs[:, [k]])[0]), 2))  # Fix: only
    ↪ holdout is evidence
```

This book's run prints 1.92 and 0.83. The sampling standard deviation of a single holdout Sharpe is about $\sqrt{252/504} = 0.71$, so 0.83 is one draw; the mean holdout value across 200 simulations in Figure 37-3 is close to 0. Why: the selection process itself creates a high Sharpe (Section 6.5). Fix: freeze the research plan and evaluate on a holdout period never used for selection; also report the number of attempts and apply Chapter 22's multiple-testing correction and Chapter 32's DSR evaluation.

12.4 Treating Overlapping Labels as Independent Observations

Error: compute a 5-day forward return every day and use $s/\sqrt{1000}$ as the standard error for 1,000 dates. Why it is wrong: adjacent labels have correlation 0.8, 0.6, … (Section 6.4), so the effective sample size is about 200. Fix:

```
import numpy as np
rng = np.random.default_rng(1)
u = rng.normal(0.0005, 0.01, 1005)
y = np.convolve(u, np.ones(5), mode="valid")[:1000]          # 1000 overlapping 5-day
    ↪ labels
naive_se = y.std(ddof=1) / np.sqrt(len(y))
nonoverlap = y[::5]
honest_se = nonoverlap.std(ddof=1) / np.sqrt(len(nonoverlap))
print(round(honest_se / naive_se, 2))                        # about 2.2, close to
    ↪ sqrt(5)
```

When the covariance in this construction is known, use the exact mean-variance formula in Section 6.4. Chapter 23's `ic_summary(nw_lags=h-1)` is an established Bartlett-kernel diagnostic; a fixed $h-1$ lag is not an exact overlap correction (Section 6.4).

12.5 Five Types of Data Leakage

Type	Erroneous implementation	Fix
Target leakage	"Past 5-day return" feature is mistakenly computed with `shift(-5)` and is actually the future 5-day return	Register `available_at` for each feature and assert `available_at` $\leq$the exact decision cutoff (the difference between date and time is covered in Chapter 38)
Look-ahead feature	Use the fiscal period-end date as the availability time for a quarterly financial statement	Use the announcement/database-ingestion time (point-in-time data in Chapters 3 and 12)
Preprocessing leakage	Standardize the full sample, then split into training and test sets	Fit the scaler only on training rows (the Chapter 38 pipeline)
Selection leakage	Inspect test results, change features, and test again	Use the test only once; for a revised design, use a new holdout and record it in the trial ledger
Overlap leakage	Randomly shuffle into K folds when training-label intervals cover test-label intervals	Split by date and exclude training rows according to actual label intervals (Chapters 31 and 38)

Chapter 38 gives runnable counterexamples and numerical differences for preprocessing leakage and random K-fold splitting.

12.6 No Baseline

Error: report "out-of-sample $R^2 = 0.3\%$, so the model works." Why it is wrong: we do not know whether 0.3% is relative to zero or the training mean, or how a simple rule would perform. Fix: report zero prediction, training mean, and a simple rule in the same table, on the same test period, and under the same costs; state the baseline for R^2_{OS}.

13. Applying This to Quant Research

Turn this chapter into an evaluation protocol and apply it in Chapters 38–40 and Project 6.

Hypothesis. Write a falsifiable statement, for example: "The set of features X available at the close has positive predictive power for ranking cross-sectional returns over the next h days, and a dollar-neutral portfolio constructed from those predictions has positive net returns at a cost of κ per unit of turnover."

Design (freeze before looking at results). Specify the asset universe, sample period, frequency, signal time / trade time, label interval, feature list, model families and hyperparameter grid, training window, rebalancing frequency, cost assumptions, evaluation metrics, and pass thresholds. Record which results were seen before freezing (Chapter 22's `TrialLedger`).

Check all five together.

1. Out-of-sample: use date-based walk-forward evaluation; report prediction metrics (IC and R^2_{OS} relative to both 0 and the training mean) separately from portfolio metrics (after-cost return and Sharpe).
2. Stability: report IC and returns by fold and year; check whether coefficients or feature importance keep the same sign across folds and whether the relationship reverses around regime shifts.
3. Costs: use at least two cost levels; report turnover and the difference between before- and after-cost results.
4. Tails: report maximum drawdown, the worst 1% of days, and directional accuracy on large-move days.
5. Economic logic: explain why the signal should work and who bears the cost of the return; without an answer, even a strong result remains a candidate.

Baselines. Zero prediction (no position), training mean (the same prediction for every asset in a cross section, yielding a flat ranking portfolio), and one simple rule. A candidate model must outperform a baseline on the same test period and under the same costs to move to the next step.

Falsification conditions (any one means the hypothesis is not supported). After-cost net returns are not positive; R^2_{OS} relative to the training mean is not positive and the IC Newey–West t statistic is not significant; results occur in only one subperiod; or results depend on significance before correcting for the number of attempts.

Null and negative results. Preserve and report them. In the constructed experiments in Chapter 39 and Project 6, some models lose to a zero-position baseline after costs; those results belong in the report just as much as the positive ones.

14. Chapter Summary

Accuracy, R^2, and IC measure predictions; net returns result from decisions and costs. For ± 1 positions, the identity $\sum p_t r_t = (\sum |r_t|)(2\text{Acc}_{|r|} - 1)$ connects them. Equal return magnitudes are sufficient for ordinary accuracy to substitute directly for magnitude-weighted accuracy; when magnitudes differ, it generally cannot. Financial prediction has a low signal-to-noise ratio, and an R^2 of a few thousandths is common; whether a portfolio is useful depends on cross-sectional breadth, turnover, and costs. Overlapping labels reduce effective sample size by about $1/h$; trying N configurations raises the best Sharpe by about $\sqrt{252/T}\, E[\max Z_N]$; and a prediction relationship can reverse, making out-of-sample R^2 negative. Leakage has five entry paths, and every one can bring information unavailable at deployment into test performance. Therefore, evaluate a financial machine-learning model on out-of-sample performance, stability, costs, tails, and economic logic together, and always compare it with zero prediction, the training mean, and a simple rule.

15. Exercises

37-E01 (Hand Calculation: Accuracy and Net Returns)

Daily returns (%) are $r = (+1.0, -0.2, +0.3, -4.0, +0.2)$. Model C positions are $(+1, -1, +1, +1, +1)$ and Model D positions are $(-1, -1, -1, -1, +1)$. The cost is $\kappa = 0.1\%$ per unit of turnover and $p_0 = 0$. For each model, calculate accuracy, magnitude-weighted accuracy, the sum of gross returns, turnover, and additive approximate net returns, and explain which one is better.

37-E02 (Turnover)

Prove that with ± 1 positions, if the probability of reversing direction each day is q and is independent of the past, expected turnover over T days is $1 + 2q(T-1)$. What is the expected annual cost when $q = 0.5$, $T = 252$, and $\kappa = 0.1\%$?

37-E03 (Regime Shift)

In the model from Section 6.6, suppose the perfect Regime 1 estimate $\hat{b} = b$ is used to predict Regime 2, with $b = 0.2$. Find R^2_{OS} relative to zero prediction. What is the result if the true slope in Regime 2 is 0, so the relationship disappears?

37-E04 (Signal-to-Noise Ratio)

For $y = bx + \varepsilon$, let $\sigma_x = 1$ and $\sigma_\varepsilon = 2\%$. What value of b yields a correlation of 0.05? What are R^2 and SNR in this case?

37-E05 (Overlapping Labels)

For overlapping labels with $h = 10$, what are the theoretical autocorrelations at lags 1, 5, and 10? Approximately what is the effective sample size for 2,520 daily overlapping labels?

37-E06 (Multiple Testing)

There are $T = 252$ days and $N = 50$ zero-return strategies. Use $E[\max Z_{50}] \approx 2.25$ to estimate the expected best annualized Sharpe. How would you respond if someone reported "the best strategy has an annualized Sharpe of 2.0"?

37-E07 (Leakage Classification)

Classify each case as a type of leakage and give a fix: (a) impute missing feature values with the full-sample feature median; (b) use test-period IC to decide which five features to keep; (c) use a post hoc revised database version for the "analyst forecast revision" feature; (d) evaluate a model with 20-day return labels using random 10-fold cross-validation.

37-E08 (How This Differs from Competitions)

List three differences between financial prediction and a typical machine-learning competition in data generation or evaluation, and explain which common practice each difference can invalidate.

37-E09 (Baselines)

For a model on a test period, $\sum(y-\hat{y})^2 = 0.981$, $\sum y^2 = 0.990$, and $\sum(y-\bar{y}_{\text{train}})^2 = 0.985$. Calculate R^2_{OS} relative to zero and to the training mean, and explain why they differ.

37-T01 (New: From Score to Three Tables)

A researcher reports 55% accuracy but does not specify data-ingestion, execution, or cost timing. Write one field that must be supplied for each of the information, prediction, and ledger tables, and explain whether the next-open example in this chapter and the next-close examples in Chapters 38–39 can directly share a label.

16. Programming Exercises

37-P01 (Decision Ledger)

Implement `decision_ledger(returns, positions, cost_per_unit)` to return daily turnover, net returns, and $T+1$ wealth points. Acceptance assertions: (1) the ten-day examples for A and B have compounded net returns of $+0.056879$ and -0.039980 (tolerance 10^{-6}); (2) the number of wealth points equals the number of returns plus 1; (3) when costs are 0, ± 1 positions satisfy the identity in Section 6.1; (4) inputs containing NaN, inf, or unequal lengths raise `ValueError`; (5) raise an error if any day's net return is $\leq -100\%$.

37-P02 (Effective Sample Size)

Write a function that takes h and a daily-return series, generates overlapping labels, and returns (a) sample autocorrelations through lag h, and (b) the ratio of the standard error from a non-overlapping subsample to that from the full overlapping sample. For $h = 5$ and 100,000 daily returns drawn from $N(0, 1\%)$, verify that (a) each absolute difference from $(h-k)/h$ is below 0.02 and (b) the ratio is between 2.0 and 2.5.

37-P03 (Selection Bias)

Simulate $N \in \{10, 100, 1000\}$ zero-return strategies, each with $T = 504$ days, for 200 repetitions; record the best in-sample Sharpe and its holdout Sharpe. Acceptance criteria: the difference between the in-sample mean and $\sqrt{252/T}\, E[\max Z_N]$ is below 0.15, and the absolute holdout mean is below 0.25.

17. Challenge Problems

37-C01 (The Window-Length Trade-Off)

In the model in Section 6.6, the true slope changes from b to $-b$ at an unknown time. You use an OLS slope estimated on the most recent L observations to predict the next period. (1) Write the expected estimated slope at the sth period after the change ($s < L$). (2) State the effect of L on estimation variance (when $\sigma_x = \sigma_\varepsilon = 1$, the OLS slope variance is approximately $1/L$). (3) If a shift occurs on average every M periods, discuss how to choose L. Additional prerequisites: Chapters 17 and 19. Scoring: correct derivations for (1) and (2), 30% each; for (3), a qualitative bias–variance trade-off and a statement of how to check robustness when M cannot be known in advance, 40%.

37-C02 (Design an Evaluation Protocol)

Write a one-page evaluation protocol for a study that "uses 20 closing-price features to predict cross-sectional returns over the next 5 days." Cover the five checks in Section 13, three baselines, what will be frozen, and falsification conditions. Open-ended; see Section 18 for the rubric.

18. Solutions / Solution Hints

37-E01

The total absolute return is $1.0 + 0.2 + 0.3 + 4.0 + 0.2 = 5.7$. For C: it is wrong on Day 4 (position $+1$ against -4.0), so Acc $= 4/5 = 0.80$, $\text{Acc}_{|r|} = 1.7/5.7 = 0.298$, and gross returns are $1.7 - 4.0 = -2.3\%$. Turnover is $0 \to +1$ (1), Day 2 $+1 \to -1$ (2), and Day 3 $-1 \to +1$ (2), for 5 total; cost is 0.5%, and additive approximate net returns are -2.8%. For D: it is wrong on Days 1 and 3, so Acc $= 0.60$; the absolute returns on wrong days sum to 1.3; $\text{Acc}_{|r|} = 4.4/5.7 = 0.772$; and gross returns are $4.4 - 1.3 = +3.1\%$. Turnover is 1 initially and 2 on Day 5 when $-1 \to +1$, for 3 total; cost is 0.3%, and approximate net returns are $+2.8\%$. D is better despite lower accuracy.

37-E02

The initial position on Day 1 has turnover 1. On each subsequent day, the position reverses with probability q, producing turnover 2, and otherwise turnover 0. Expected turnover is $1 + \sum_{t=2}^{T} 2q = 1 + 2q(T-1)$. At $q = 0.5$ and $T = 252$, this is $1 + 251 = 252$ units; costs are $252 \times 0.1\% = 25.2\%$ per year, on an additive basis.

37-E03

For $\hat{b} = b = 0.2$, $R^2_{\text{OS}} = 1 - [(0.4)^2 + 1]/(0.04 + 1) = 1 - 1.16/1.04 = -11.5\%$. If the true Regime 2 slope is 0, then $y = \varepsilon$, $E[(y - \hat{b}x)^2] = 1 + \hat{b}^2 = 1.04$, and $E[y^2] = 1$, so $R^2_{\text{OS}} = -4\%$. The model is worse than zero prediction even after the relationship disappears because it continues to take a position.

37-E04

$\rho = b\sigma_x / \sqrt{b^2\sigma_x^2 + \sigma_\varepsilon^2} = 0.05$, so $b^2 = 0.0025(b^2 + 0.0004)$, $b^2(1 - 0.0025) = 10^{-6}$, and $b = 0.001001$ (0.1001% per unit of x). $R^2 = 0.0025$; SNR $= R^2/(1 - R^2) = 0.002506$.

37-E05

$\rho_1 = 0.9$, $\rho_5 = 0.5$, and $\rho_{10} = 0$. Effective sample size is about $2520/10 = 252$.

37-E06

$\sqrt{252/252} \times 2.25 = 2.25$. A reported value of 2.0 is below the expected best value under the zero null and cannot be treated as a discovery; require a holdout result, the number of attempts, and a Deflated Sharpe calculation (Chapter 32; multiple testing in Chapter 22).

37-E07

(a) Preprocessing leakage: calculate the median on training rows only, or use the cross-sectional median on each date if only that date's data are available. (b) Selection leakage: select features within training data (inner validation) and evaluate on the test set only once. (c) Look-ahead feature: use the point-in-time version and filter by data-ingestion time. (d) Overlap leakage: 20-day labels overlap across random folds; switch to date-based walk-forward evaluation and exclude rows according to their actual label intervals.

37-E08

Reference answers (any three, with the invalidated practice matched to each): (1) Distribution shift: a simplified competition is approximately identically distributed, while time drift in finance means random splitting may put future regimes into training; diagnose using the actual information structure. (2) The test set can be used only once: some competitions allow repeated public-leaderboard submissions, but repeatedly inspecting a financial test period creates selection leakage. (3) The objective differs: a competition metric is the objective, while the financial objective is risk-adjusted return after costs, so high accuracy can lose money. (4) Trading affects prices: a competition prediction does not change the answer, while trading size affects costs and capacity. Scoring: 1 point for each difference and 1 point for its corresponding invalidated practice.

37-E09

Relative to zero: $1 - 0.981/0.990 = 0.91\%$. Relative to the training mean: $1 - 0.981/0.985 = 0.41\%$. The training mean itself explains $1 - 0.985/0.990 = 0.51\%$ (the test-period mean is nonzero and has the same sign as the training mean), so the model adds only 0.41% beyond the mean.

37-P01 Reference Implementation and Tests

The reference implementation is `decision_ledger` in `code/ch37_ml_philosophy.py`; tests are in `tests/test_ch37_philosophy.py` and cover the ten-day values, number of wealth points, the identity, non-finite inputs, and the -100% boundary. Run:

```
.venv/bin/python -m pytest tests/test_ch37_philosophy.py -q
```

Numerical check: A's wealth at point 5 is 1.037269 and at the end is 1.056879; B's ending wealth is 0.960020.

37-P02

Reference implementation: `overlap_autocorrelation`. The run in this book (seed 3703) has sample autocorrelations $(0.802, 0.604, 0.403, 0.203, 0.004)$; the ratio of standard errors from the non-overlapping and overlapping samples is about 2.2 in the Section 12.4 snippet. If the ratio is outside [2.0, 2.5], check whether the labels used `mode="valid"` and were truncated to equal lengths.

37-P03

Reference implementation: `max_null_sharpe`. The run in this book (seed 3702) gives in-sample means 1.138, 1.814, and 2.304; formula values are 1.088, 1.773, and 2.292, with each difference below 0.06; holdout means are -0.094, -0.052, and $+0.105$.

37-C01 Reference Argument

(1) The window contains $L - s$ observations from the old regime and s from the new regime. With x independent and identically distributed, the OLS slope is approximately a sample-count-weighted average of the two slopes: $E[\hat{b}] \approx \frac{(L-s)b+s(-b)}{L} = b(1-2s/L)$; it crosses zero at $s = L/2$ and fully catches up at $s = L$. (2) With a stable fixed-design scale and independent, homoskedastic errors, slope variance is about $\sigma^2/(L\sigma_x^2)$ and thus varies as $1/L$; the shorter L is, the larger the variance. (3) The average bias loss after a shift is approximately proportional to L/M, while the variance loss is approximately proportional to $1/L$. Under the approximations of isolated shifts, independent design within the window, and no interference from adjacent shifts, if average loss is written as $AL/M + B/L$, differentiation gives $A/M - B/L^2 = 0$, so $L \approx \sqrt{BM/A}$. The constants depend on the signal, noise, and drift process; this is not an exact globally optimal window. If M is unknown, report results for multiple L values and check whether the conclusion holds only at one L. Scoring follows the weights in the question; a conclusion without derivation earns at most 40%.

37-C02 Scoring Rubric

10 points total: one point for each of the five checks (with a concrete metric, such as "Newey–West t statistic for IC, lag 4"); one point for the three baselines; one for a freeze list that includes timing and costs; one for operational falsification conditions; one for stating which results were seen before freezing and recording them; and one for explaining how null and negative results will be retained. Acceptable conclusions include "insufficient evidence"; treating constructed-data results as real evidence loses 3 points.

37-T01 Full Answer

The information table could add feature `available_at`, label `label_available_at`, `fit_time`, or the exact signal cutoff, and check that the information was readable then. The prediction table could add signal date, asset, prediction, or evaluation-date set. The ledger could add `trade_time`, position, turnover, or cost. Any one field for each table is sufficient if its role is explained. A 55% accuracy figure does not specify these contracts and cannot imply net returns. Next-open and next-close use different return starting points, so they cannot share a label without redefining it; Chapter 38 separates exact timestamps from date indices.

19. Further Reading

- Must Know: Chapter 22 (research plans and multiple testing), Chapter 32 (DSR and HAC), and Chapter 31 (label intervals, purging, and walk-forward evaluation). Use these for the tools in Sections 6.4–6.5.
- Must Know: scikit-learn, "Common pitfalls and recommended practices" (`SK_PITFALLS`), especially the sections on data leakage and preprocessing within training data. Use this as a software-side counterpart to the leakage classification in Section 12.5.

- Should Know: Bailey, Borwein, López de Prado, and Zhu (2017) on the probability of backtest overfitting (PBO). Use this to extend the selection-bias discussion in Section 6.5 to portfolio-level overfitting probability.
- Should Know: the machine-learning asset-pricing and backtesting-protocol literature listed in Section 20; read the definitions of out-of-sample R^2 and the portfolio-evaluation sections.
- Optional: Chapters 35 and 36 for a review of turnover costs and execution.

20. References

- SK_PITFALLS: scikit-learn, "Common pitfalls and recommended practices." Supports the definition of preprocessing leakage in Section 5 and the repair in Section 12.5.
- PBO: Bailey, D. H., Borwein, J., López de Prado, M., and Zhu, Q. J. (2017). "The probability of backtest overfitting." Journal of Computational Finance. Background for selection bias in Section 6.5.
- DSR: Bailey, D. H. and López de Prado, M. (2014). "The Deflated Sharpe Ratio." Supports the approximation for $E[\max Z]$ in Section 6.5 (implemented by Chapter 32's `expected_max_standard_normal`).
- CH22_HLZ2014: Harvey, Liu, and Zhu. "…and the Cross-Section of Expected Returns." Background on multiple testing in factor research for Section 5.
- CH37_GKX2020: Gu, S., Kelly, B., and Xiu, D. (2020). "Empirical Asset Pricing via Machine Learning." Review of Financial Studies 33(5), 2223–2273. Author's close reading: Section 1.8, Equation (19), p. 2249 (R^2_{OS} relative to zero prediction and the rationale); Table 1, p. 2250 (OLS −3.46%, OLS-3 0.16%, ENet+H 0.11%); Section 2.2 (NN3 0.40%); and the introduction's long-short portfolio Sharpe ratios 1.35/2.45. Supports the baseline definition for R^2_{OS} in Section 5 and the evidence limits in Section 11.
- CH37_CT2008: Campbell, J. Y. and Thompson, S. B. (2008). "Predicting Excess Stock Returns Out of Sample: Can Anything Beat the Historical Average?" Review of Financial Studies 21(4), 1509–1531; the author read the NBER w11468 version, Equation (1), p. 5. Supports the historical-mean baseline definition in Section 5.
- CH37_AHM2019: Arnott, R., Harvey, C. R., and Markowitz, H. (2019). "A Backtesting Protocol in the Era of Machine Learning." Journal of Financial Data Science 1(1), 64–74. A source reconnaissance report had read the full text (introduction and transaction-cost section); the author did not reopen it, so it is listed only as further reading for the evaluation protocol in Section 13.
- The full academic definitions of leakage (Kaufman et al., 2012) and the paper on "accuracy versus economic returns" (Leung et al., 2021) could not be accessed; this chapter does not cite their specific conclusions. The relevant material is supported by the derivation in Section 6.1 and SK_PITFALLS. See `course/zh/chapters/ch37/source_notes.md` for the author's reading record.

Chapter 38: Financial ML Pipeline

Main thread: Turn "when each feature is known, which returns the label covers, and when the model is fitted" into a contract, then make splitting, preprocessing, tuning, and evaluation follow that contract. Prerequisites: Chapter 23, cross-sectional research (IC, grouping, panels); Chapter 31, model validation (label intervals, purging, embargo, walk-forward); and Chapter 37. After completing this chapter, you will be able to: record signal_time, trade_time, and the label interval for every row in a daily panel; determine by hand which training rows are available at a given fit time; preprocess and tune with a training-only Pipeline; and explain why shuffled K-fold can rate pure-noise features as "effective."

1. Learning Objectives

1. Record six times for every panel row: feature availability (`available_at`), `signal_time`, `trade_time`, `label_start`, `label_end`, and `label_available_at`, and use them to calculate labels by hand.
2. Prove that, at fit time s, a row with signal date k is available for training if and only if $k + \ell + h \leq s$; with no missing data, the last $\ell + h - 1$ signal dates before the test date are excluded. Explain how this differs from purging based on interval overlap.
3. Split a panel with date-grouped walk-forward: all asset rows on a date must stay on the same side.
4. Distinguish transforms that use only same-day cross-sectional information (no training fit required) from transforms fitted across dates (must be fitted only on training rows); implement the latter with a `Pipeline`.
5. Place hyperparameter selection inside the outer training set and explain why inner folds must follow the same availability rule.
6. Report prediction-layer metrics (R^2_{OS}, IC, calibration slope) separately from portfolio-layer results (net-of-cost returns, Chapter 39 and Project 6).
7. Quantify the costs of three incorrect practices with runnable counterexamples: shuffled K-fold, full-sample feature selection, and starting the label at the signal close.

Completion criteria: Sections 15–17 can be completed independently; trace the frozen inputs and outputs in Sections 8–10 and explain the identity boundary between a fresh run of Section 9 and a frozen replay. This batch does not run or retrain.

2. Why Quant Needs This

A researcher brings you the following result: 40 stocks, five years of daily data, and 22 features produce an out-of-sample $R^2 = 56.7\%$ under five-fold cross-validation. In this constructed example, the true feature loadings are all zero, so this large positive value is not evidence of a genuinely available signal. Inspect the data-generating process and leakage paths; do not claim that a high R^2 is mathematically impossible in any financial problem based only on an empirical rule of thumb. The problem is the pipeline: each daily label is a five-day return, so adjacent

labels share four return days; the company features change slowly, so today's value is almost the same as yesterday's; after shuffling, a test row's "neighbors" are training rows for the same stock one or two days earlier or later, with nearly identical features and heavily overlapping labels. On the test set, the model is looking up rows, not forecasting. Section 8 reproduces this 56.7% using pure-noise features.

The error is often not in the model but in the timing: when a feature becomes available, which period the label covers, and when the model is fitted. This chapter writes those times into a contract and makes every step check it. Chapter 31 introduced label intervals, purging, and walk-forward for a single series. This chapter handles panels (multiple assets on the same date) and the full preprocessing–tuning–evaluation chain, and places the implementation in `quant_course.ml_pipeline` for Chapter 39 and Project 6 to share.

3. Intuition

Imagine building a model after the close on Monday, March 2. You have all prices through Monday's close and want to predict the return from "buying at tomorrow's close and holding for two days."

- The sample from the previous Wednesday (k) has a label covering Thursday's close through Monday's close. By Monday's close you have observed it, so this row can be used for training.
- The previous Thursday's label runs from Friday's close through Tuesday's close. Tuesday has not arrived, so the row cannot be used.
- The previous Friday's label runs from Monday's close through Wednesday's close. It is also not yet complete.

Thus, even when a sample's **signal date** is before the model date, its **label** may not have finished accruing and cannot enter training. Including it lets the model see future returns. The number of excluded dates is determined by "execution lag + holding period," not by an interval chosen by intuition.

A panel adds another issue: there are 40 stocks on the same date. Their features are available together, and their labels cover the same period. If 20 stocks from March 2 go into training and the other 20 into testing, then at test time the model has already learned from the training stocks that "the market rose after March 2." The split unit must therefore be the date, not the row.

4. Mathematical Setup

Symbol	Meaning	Unit / dimension
$D_1 < D_2 < \cdots < D_T$	Trading dates; each date represents that day's close	date
$i = 1, \ldots, N$	Asset	—
$\ell \geq 1$	Execution lag: number of trading days from signal close to execution close	trading days
$h \geq 1$	Prediction horizon (holding period)	trading days
$x_{i,k} \in \mathbb{R}^p$	Features for asset i available at the close on signal date D_k	each feature's own unit
$P_{i,k}$	Closing price	currency / share
$y_{i,k} = P_{i,k+\ell+h}/P_{i,k+\ell} - 1$	Label: h-day simple return from the execution close	decimal
$(D_{k+\ell}, D_{k+\ell+h}]$	Label interval (open on the left, closed on the right)	—
s	Date index of fit time: fit the model after the close on D_s and predict test rows starting with signal date D_s	—

Timing convention: `signal_time` $= D_k$; `trade_time` = `label_start` $= D_{k+\ell}$; `label_end` = `label_available_at` $= D_{k+\ell+h}$. This book assumes prices and features from the same close become available together after the close. If your data system ingests them a day later, move both `available_at` and `label_available_at` one day later; the derivation below still applies.

Keep dates separate from exact decision times. `signal_time` records the **date** of the signal, D_k. By default, the exact decision time τ_k is the signal cutoff after that day's closing price is finalized—for example, the close is at 16:00 and the cutoff is 16:10 in the exchange's local time zone. A feature can be used for the D_k signal if and only if `available_at` $\leq \tau_k$. Data released after the cutoff (for example, earnings at 16:30) cannot be used for the D_k signal and can first be used for D_{k+1}. **After-hours signal variant:** if a study explicitly sets its cutoff later that evening (for example, 20:00) and guarantees that the data are ingested before then, the 16:30 data can be used for the D_k signal. The data dictionary must state the cutoff, time zone, and mapping from `available_at`; execution still occurs at $D_{k+\ell}$. The constructed data in Chapters 38–39 and Project 6 use only the default cutoff: every feature is derived from closing prices or specified as available at the close, so the variant does not affect them.

Intraday clock: snapshot first, then fit, then signal

The following is a standalone teaching construction in the exchange's local time zone. Except for the next-trading-day execution in the last row, all intraday and after-hours events occur on trading date D_s. Date indices only establish order; times such as 16:00 and 16:08 tell us whether a program can read the data. D_s is not an arbitrary after-hours time.

Time	Event	Treatment in this fit or signal
16:00	Close; a historical training row's label interval has just ended	Still wait for label ingestion; ending does not mean it can be read
16:08	The day's price-derived features are ingested	Available to the 16:10 signal; also readable in the 16:09 fit snapshot
16:09	`fit_time`: lock the training snapshot and start fitting	Use only historical features already readable and mature labels
16:09:40	Fit completes	In time for the signal cutoff; later-arriving data are not added to the fitted model
16:10	τ_s: generate and freeze the signal	Apply the fitted transform to today's features, ingested at 16:08
16:20	The label that ended at 16:00 above is ingested	After `fit_time`; it cannot enter this training run
16:30	Earnings are released (assume they are ingested at the same time)	After τ_s; cannot enter this prediction
Close on D_{s+1}	Execution point for this example with $\ell = 1$	The label starts at this execution close, not at 16:10

Rows used in this fit have $k < s$, a point-in-time version of each historical feature fixed according to its own signal cutoff and readable in the fit snapshot, and `label_available_at` no later than 16:09. Rows for the current date, $k = s$, are used only for prediction. The 16:08 feature for the current date is not added to a training row with no label yet, nor backfilled into the 16:09 snapshot. If another historical row's label was ingested at 15:55 and its historical features are complete, it can enter training; the row ingested at 16:20 is excluded, even if both rows' `label_end` is no later than that day's close.

Three checks guard three gates: training data must be readable by `fit_time`; prediction features must be readable by τ_s; and fitting must finish by τ_s. If the model finishes fitting at 16:12, it cannot produce the 16:10 signal. Follow a predeclared protocol to delay the signal or use a model that was already complete; do not backdate the runtime. Chapter 37's hand-worked example executes at the next open; this chapter, Chapter 39, and Project 6 execute at the next close. Keep the return starting points in the two sets of examples distinct.

Prerequisites used here: Chapter 31's interval-overlap condition $\max(t_1, t_2) < \min(e_1, e_2)$; the projection property of OLS in Chapter 8; and daily IC in Chapter 23.

5. Formal Definitions

Feature and feature availability time (`available_at`). A feature is a quantity $x_{i,k}$ that can be computed on the signal date. Record `available_at` for each feature; it must satisfy `available_at` $\leq \tau_k$ (the decision time for signal date D_k; see Section 4). Features derived from closing prices (past five-day returns, past 20-day volatility) are available after the close is finalized and before the cutoff. Earnings released only after the close at 16:30 have an `available_at` equal to the release time, even if the "reporting period" predates the signal date; they cannot be used for that day's signal under the default cutoff (Chapter 12, PIT). This makes "did the feature use the future?" a rule that code can assert.

Feature engineering. The process of constructing features from raw data. There are two types by information source:

- **Cross-sectional transform:** uses data only from the same signal date, such as that day's cross-sectional ranks, cross-sectional z-scores, or within-industry demeaning (Chapters 23 and 28). It is computed independently on each date, requires no "training fit," and brings in no information from other dates.
- **Pooled / time-series transform:** estimates parameters from data across multiple dates, such as full-sample mean–variance standardization, filling missing values with the full-sample median, PCA, feature selection, or quantile-clipping thresholds. Estimate its parameters using training rows only.

Label and prediction horizon. The label $y_{i,k}$ is the quantity the model predicts. By default in this book, it is the h-day simple return starting at the execution close; h is the prediction horizon. The label interval is $(D_{k+\ell}, D_{k+\ell+h}]$.

Label availability time (`label_available_at`). The time when the label value first becomes readable to the training program. In this chapter's daily construction, it is assumed to be available at the same close as `label_end`; a real system must record the ingestion time instead of assuming equality. It is different from `signal_time`: an early signal date does not mean the label is already known.

Cross-sectional panel. A long table keyed by (`signal_time`, `asset`). Rows with the same `signal_time` share the same set of timestamps.

Fit time (`fit_time`) and available training set. Fit the outer fold at the close on D_s. Candidate training rows have signal dates before D_s. With exact timestamps, historical features and labels must both be readable at or before `fit_time`. The date formula below abbreviates `label_available_at` $\le$ `fit_time` as `label_available_at` $\le D_s$ only under the constructed assumption that data are available synchronously at the close:

$$\mathcal{U}(s) = \{(i,k) : \ k < s, \ k + \ell + h \le s, \ x_{i,k} \text{ and } y_{i,k} \text{ are both nonmissing}\}.$$

Date-grouped walk-forward. Sort the unique dates. A test block consists of consecutive dates $D_s, \ldots, D_{s+m-1}$; training dates are $D_1, \ldots, D_{s-1}$ (expanding window) or the most recent L dates (rolling window). Move the entire block forward each time. All asset rows on a date always remain on the same side.

Purging (excluding by label interval). Chapter 31's purging removes training rows whose label intervals overlap test label intervals. This chapter writes the rule as availability by $k + \ell + h \le s$; Section 6.1 proves that it implies non-overlap and is stricter.

Gap and embargo. A gap is an additional set of dates left empty between the end of training and the start of testing. An embargo removes training rows immediately after a test block (when K-fold allows training rows after the test block) if their feature lookback windows cover the test label interval. Both should follow from the data's actual timing structure: the gap follows from label intervals and availability time; embargo length follows from the feature lookback window. Do not replace this with a uniform "leave five days" rule (Section 6.2).

Preprocessing and Pipeline. Preprocessing is a transform across dates applied to features before fitting a model. `sklearn.pipeline.Pipeline` chains several transforms and an estimator into one object. On `fit`, each step is fitted in sequence only to the training data passed in; on `predict`, the fitted parameters transform new data (official documentation `SK_PIPE_DOC`).

Hyperparameter selection. Choosing quantities such as regularization strength or number of features that are not directly estimated during training. Rule: use only outer-training rows; within those rows, run date walk-forward inner folds, each following the same availability rule; after selection, refit on all outer-training rows and predict the outer test block. Outer-test labels must not enter any fit, standardization, or selection.

Evaluation. Two layers. Prediction layer: R^2_{OS} (against zero and the training mean), daily Spearman IC and its Newey–West t-statistic (lag $h-1$), and calibration slope. Portfolio layer: convert predictions to positions under an explicit rule, execute with a lag, then measure net-of-cost returns, turnover, and drawdown (Chapter 29 ledger; Chapter 39 and Project 6).

Model calibration. Whether predictions and realized values agree in magnitude. For regression predictions, the calibration slope is $b = \mathrm{Cov}(y, \hat{y})/\mathrm{Var}(\hat{y})$. When $\mathrm{Var}(\hat{y}) > 0$, in the linear-projection sense, $0 < b < 1$ means forecast amplitude is too large relative to the linearly calibrated conditional mean, while $b > 1$ means it is too small; $b = 0$ means no linear information, and $b < 0$ means a reversed relationship, not merely a confidence-scaling issue. For example, forecasts $(-1, 0, 1)$ and realizations $(1, 0, -1)$ give $b = -1$ even though their variances are equal. Also check the intercept $a = E[y] - bE[\hat{y}]$. Even $a = 0, b = 1$ does not guarantee $E[y \mid \hat{y}] = \hat{y}$; nonlinear or group-specific bias may remain. A reliability diagram grouped by prediction compares average predictions with average realizations within each group (see `SK_CALIBRATION` for the classification-probability version).

6. Derivations

6.1 Availability Rule, Number of Excluded Dates, and Its Relation to Overlap Purging

Assumptions: equally spaced date indices; no missing data; $\ell, h \geq 1$; fit at D_s; the first test row has signal date D_s.

First, the availability condition. The label for signal date k is known at the close on $D_{k+\ell+h}$. Fit time is the close on D_s, so the row is available if and only if

$$k + \ell + h \leq s \iff k \leq s - \ell - h.$$

Second, the number of excluded dates. Candidate training dates are $k = 1, \ldots, s-1$; available rows satisfy $k \leq s - \ell - h$. The excluded dates are $k = s - \ell - h + 1, \ldots, s - 1$, a total of

$$(s-1) - (s - \ell - h) = \ell + h - 1$$

dates, or that number times the asset count N in rows. With $\ell = 1, h = 5$, five signal dates are excluded.

Third, availability implies non-overlap. The earliest test label interval is $(D_{s+\ell}, D_{s+\ell+h}]$; all other test intervals are later. An available training row satisfies $k + \ell + h \leq s < s + \ell$, so its interval ends no later than the start of any test interval. Under Chapter 31's condition $\max(t_1, t_2) < \min(e_1, e_2)$, the intervals do not overlap.

Fourth, overlap purging alone is insufficient. The training interval $(D_{k+\ell}, D_{k+\ell+h}]$ overlaps the first test interval $(D_{s+\ell}, D_{s+\ell+h}]$ if and only if $s+\ell < k+\ell+h$, or $k > s-h$. Overlap purging excludes only $h-1$ dates; the availability rule excludes $\ell + h - 1$. The additional ℓ dates have labels that end before the test interval starts, but after D_s; their returns have not occurred by fit time.

Use: In implementation, filter each row by `label_available_at`; the number excluded then follows naturally from the intervals. The row-wise rule still works when dates are missing or assets have different label intervals, whereas "always leave h days" can fail. `ml_pipeline.date_walk_forward` implements this and uses "training and test intervals must not overlap" as a defensive assertion.

A more conservative convention: If labels must be ingested by a close **before** fit time (`label_available_at` $< D_s$), exclude one additional date. Both conventions avoid leakage; this book chooses the same-close-available convention used for features.

6.2 When a Gap or Embargo Is Needed, and How Long It Should Be

Walk-forward (all training is before testing): The availability rule in Section 6.1 already handles labels that cross the boundary; no additional embargo is needed. If the system requires at least g days between fitting and deployment, that is a deployment delay and belongs in `trade_time`, not in an arbitrary training-row deletion.

scikit-learn's `TimeSeriesSplit(gap=g)`: `gap` is the **number of samples** excluded between the end of the training set and the test set (`SK_TSS`). In a panel, a sample is a row, not a date: with 40 rows per day, `gap=5` leaves only one-eighth of a day. Even `gap=5*40` becomes misaligned with dates as soon as a stock is missing on any date. Use date rules for panels, not a row-count gap.

Purged K-fold (training blocks may be after the test block): First purge using the real label intervals; future training rows may also overlap test labels and cannot be skipped just because their signal dates are later. Then check the feature windows of the remaining future training rows. If a feature uses the past L return increments, its support is $(D_{k-L}, D_k]$. For a single test-label interval $(D_a, D_e]$, the two intervals overlap if and only if both $k > a$ and $k - L < e$. For a disjoint union of test-label intervals, check each interval; using the enclosing interval is conservative. Only when rows are explicitly restricted to $k > e$ is the latter condition alone sufficient, in which case delete $e < k \leq e+L-1$. Do not call this a generally strict minimum for all future rows. If the price endpoint is also counted as shared information, or the rule uses $k \leq e + L$, one additional date is deleted under a conservative protocol.

Chapter 31's window rule must use its stated coordinates and half-open endpoints. Counted relative to the last label end e, the finite return-increment lookback is controlled by L. If counted from the last test signal date instead, e itself changes with ℓ, h; the total gap cannot be said to be independent of h without qualification. `rev_5` corresponds to five days; `mom_60_5` depends on prices as far back as 60 days and can use a conservative 60-day protocol. EMA has no exact finite lookback endpoint, and quarterly variables cannot simply be treated as finite return windows. Walk-forward has no training rows after the test block; label maturity handles the past-side boundary, so there is no future-side embargo.

This chapter's CLI date-block diagnostic actually passes signal-date ordinals and `label_end`; it purges using the conservative `(signal_date, label_end]` envelope, not the true `(label_start, label_end]` interval in this section. The embargo parameter defaults to zero. Its result table is an offline diagnostic, not full feature isolation or deployment evaluation; `date_walk_forward` separately filters by the true label interval and maturity time.

6.3 Training-Only Standardization: When It Changes Results and When It Does Not

Claim (three levels): With-intercept OLS and training design matrix $Z = [\mathbf{1}, X] \in \mathbb{R}^{n\times(p+1)}$; standardize $\tilde{x} = S^{-1}(x - \mu)$, where S is an invertible diagonal matrix and μ, S are arbitrary fixed constants. Then $\tilde{Z} = [\mathbf{1}, \tilde{X}]$, with $\tilde{X} = (X - \mathbf{1}\mu^\top)S^{-1}$.

1. **Training fitted values are always unchanged:** whether or not Z has full column rank, OLS fitted values are the same under the two designs.
2. **A corresponding coefficient representation always exists:** any standardized coefficients $(\tilde{a}, \tilde{b})$ can be written in the original coordinates as $b = S^{-1}\tilde{b}$ and $a = \tilde{a} - \mu^\top S^{-1}\tilde{b}$. Both representations give the same predictions at every new point.
3. **Predictions at new points: unchanged with full column rank, not guaranteed with rank deficiency:** If Z has full column rank ($\operatorname{rank} Z = p + 1$, so $n \geq p + 1$, and no feature is an exact linear combination of other features and the intercept), the OLS solution is unique. The solutions under the two designs correspond, so they give the same prediction for every new point x. If Z is rank deficient, the training data do not identify coefficient components in the null-space directions. The software's default solution (scikit-learn `LinearRegression` first centers using training means to handle the intercept, then returns a minimum-norm slope solution) chooses a different "minimum norm" in the two coordinate systems. Predictions generally differ for new points outside the training data's linear relationships. If a new point satisfies those same linear relationships, the predictions are guaranteed to agree; at other points they may coincide accidentally, but there is no general guarantee.

When reading these three levels, distinguish "changing coordinates for the same coefficient vector" from "letting software solve again." Level 2 is an algebraic identity that holds at every new point. Level 3 asks whether refitting is guaranteed to select that corresponding coefficient vector. Full column rank ensures uniqueness; rank deficiency ensures only that the training projection is unique. The full-rank condition is on the augmented matrix Z, including the intercept, not just on the number of feature columns; invertibility of S rules out zero-variance features.

Proof: (1) $\tilde{Z} = ZT$, where

$$T = \begin{pmatrix} 1 & -\mu^\top S^{-1} \\ 0 & S^{-1} \end{pmatrix}$$

is invertible, so $\mathrm{col}(\tilde{Z}) = \mathrm{col}(Z)$. OLS fitted values are the orthogonal projection of y onto the column space. The projection is unique and independent of the basis for that space, so fitted values are the same; full rank is not needed. (2) Substitute directly: $\tilde{a} + \tilde{b}^\top S^{-1}(x - \mu) = a + b^\top x$. (3) With full column rank, $\tilde{Z}^\top \tilde{Z} = T^\top Z^\top ZT$ is invertible, and the unique solution is $\tilde{\beta} = T^{-1}\hat{\beta}$. The coordinate correspondence in (2) gives identical predictions at new points. With rank deficiency, the normal-equation solution set is $\hat{\beta} + \ker Z$; the new-point prediction $z^\top \beta$ is not unique over this set unless z is orthogonal to $\ker Z$ (that is, the new point satisfies the same linear relationships as the training data). `LinearRegression`'s minimum-norm solution minimizes slope norm $\|b\|$ in the original coordinates and $\|\tilde{b}\| = \|Sb\|$ in standardized coordinates. These norms differ, so the selected solutions generally differ. □

Rank-deficient counterexample (constructed): Training $X = \begin{pmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{pmatrix}$, $y = (1, 2, 3)$; the second column is exactly twice the first. The new point $x = (1, 5)$ does not satisfy this relationship.

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = np.array([[1.0, 2.0], [2.0, 4.0], [3.0, 6.0]]); y = np.array([1.0, 2.0, 3.0]);
    ↪ new = np.array([[1.0, 5.0]])
raw = LinearRegression().fit(X, y)
std = make_pipeline(StandardScaler(), LinearRegression()).fit(X, y)
print(np.round(raw.predict(X), 6), np.round(std.predict(X), 6))     # identical
    ↪ training fit
print(round(raw.predict(new)[0], 4), round(std.predict(new)[0], 4))  # 2.2 1.75
```

Both models have training fitted values $(1, 2, 3)$. In the original coordinates, minimizing $\|b\|$ subject to $b_1 + 2b_2 = 1$ gives $b = (0.2, 0.4)$, intercept 0, and new-point prediction $0.2 + 0.4 \times 5 = 2.20$. In standardized coordinates, the training standard deviations are $s = (0.8165, 1.6330)$; minimizing $\|Sb\|$ gives $b = (0.5, 0.25)$ after converting back to the original units, intercept 0, and new-point prediction $0.5 + 0.25 \times 5 = 1.75$. Both satisfy the only combination identified by the training data, $b_1 + 2b_2 = 1$. Their difference lies entirely in the null-space direction $(2, -1)$, about which the data contain no information. Practical implication: the statement "standardization does not change OLS predictions" holds only with full column rank. Exact collinearity is common in panel data—for example, including two units of the same feature or the dummy-variable trap—and breaks that statement. Standardization parameters must still be estimated from training rows, whether or not the design is full rank.

Thus, with full column rank, "standardize using full-sample means and variances" does not affect OLS predictions; with rank deficiency, even OLS can be affected (as above). Standardization can generally affect:

- Ridge and Lasso: the penalty $\|b\|$ depends on feature scales, so changing μ, S can change the solution (Chapter 39).
- kNN and clustering: distances depend on scale.
- Threshold rules, clipping, missing-value imputation, and feature selection: their parameters are estimated from data, so full-sample estimates directly use test-period information.

In this constructed example, **feature selection** produces substantial leakage. The size of leakage from different preprocessing methods depends on the data and design; this case does not establish a universal ranking. Section 8.3 shows that selecting 10 of 200 pure-noise features by full-sample correlation gives walk-forward $R^2_{\text{OS}} = +0.62\%$ and IC 4.8%; putting selection inside training gives -1.21%. The uniform rule is: every transform with parameters estimated across dates belongs in a `Pipeline` and must be refitted in each training fold.

6.4 Why Shuffled K-Fold Turns Noise into a Signal

Setup: Features are slow AR(1) variables with daily correlation $\rho = 0.98$; labels are five-day returns, with adjacent-day label correlation approximately $(h - j)/h$ (Chapter 37, Section 6.4); features and labels are truly unrelated.

After shuffling, the training set almost always contains $(i, k \pm 1)$ and $(i, k \pm 2)$ for a test row (i, k). For each feature, the expected squared distance between two rows is $E(x_{i,k} - x_{i,k+j})^2 = 2(1 - \rho^j)$, which is 0.04 when $j = 1$; between different assets it is 2. kNN's nearest neighbors are almost all adjacent dates for the same asset, and their labels correlate 0.8 and 0.6 with the test label. The prediction is approximately the average label of "nearby dates," and is therefore highly correlated with the test label. This is overlap leakage (Chapter 37, Section 12.5): the model uses features to identify "which stock and which dates" a row represents, then borrows overlapping labels.

A linear model (Ridge) cannot "look up" rows. On the same data, shuffled K-fold gives about $+0.05\%$, while walk-forward gives -1.65%. The gap is small, but the direction is the same: random splitting lets training and test share spurious relationships from the same period. Flexible models (trees and nearest neighbors in Chapter 40) amplify this effect into the 56.7% result in Section 8.

6.5 Decomposing Prediction Error into "Level and Cross-Section," and Calibration

Assumption: Each date d has N_d assets. Let the date-level means be $\bar{y}_d, \bar{\hat{y}}_d$, and define demeaned values $y^\circ = y - \bar{y}_d$, $\hat{y}^\circ = \hat{y} - \bar{\hat{y}}_d$.

$$\sum_d \sum_i (y_{i,d} - \hat{y}_{i,d})^2 = \sum_d N_d (\bar{y}_d - \bar{\hat{y}}_d)^2 + \sum_d \sum_i (y^\circ_{i,d} - \hat{y}^\circ_{i,d})^2.$$

Derivation: $y-\hat{y} = (\bar{y}_d - \bar{\hat{y}}_d) + (y^\circ - \hat{y}^\circ)$. The cross term after squaring is $2(\bar{y}_d - \bar{\hat{y}}_d)\sum_i (y^\circ_{i,d} - \hat{y}^\circ_{i,d}) = 0$, because the demeaned values sum to zero within each date.

Financial interpretation: The first term is the error in "guessing how much the whole market will rise this week"; the second is the error in "guessing which asset will do better." The return of a dollar-neutral portfolio (weights sum to zero) depends only on the information corresponding to the second term. Spearman IC measures only ranks, which are unchanged by demeaning; cross-sectional SSE and cross-sectional R^2 still depend on prediction magnitudes and scale. A model may do poorly on the first term (market noise dominates the level forecast, giving negative R^2_{OS}) yet be useful on the second (positive cross-sectional R^2 and IC). The constructed panel in Section 8.4 illustrates this.

Calibration: A pooled calibration slope b mixes the two terms. The cross-sectional slope $b^\circ = \sum \hat{y}^\circ y^\circ / \sum (\hat{y}^\circ)^2$ uses only the second. If trades are triggered when "predicted returns exceed costs" (Section 13), cross-sectional calibration is needed: estimate b° on the **inner validation set** first, then—if the corresponding linear calibration model holds—use $b^\circ \hat{y}^\circ$ as a linear approximation to expected excess return. A pooled slope cannot be read directly as the true conditional expectation for every asset or state. Estimating the calibration coefficient on the test set is selection leakage.

7. Financial Interpretation

A time contract is risk control. If a backtest uses one extra day of future returns, that day does not exist in live trading. Writing every time into the data dictionary lets code check "what did I know at time t?" instead of relying on a researcher's memory.

Why group by date. Stocks on the same date experience a common market shock. If rows from the same date are randomly split, the model learns that day's market direction from training stocks and "predicts" it for test stocks. At deployment, that information is not available from the signal-date close.

Level and cross-section. Many stock-selection strategies are dollar-neutral: weights summing to zero cancel a common level term in $w^\top y$. This does not ensure beta neutrality, and it does not mean every such portfolio depends only on ranks. The actual interface demeans predictions and normalizes them by their L1 absolute sum. Multiplying them by a common positive constant or adding a common constant leaves weights unchanged; an arbitrary rank-preserving transformation generally changes them. For example, predictions $(-1, 0, 1)$ give weights $(-1/2, 0, 1/2)$, while rank-preserving predictions $(-1, -0.9, 1)$ give $(-7/26, -3/13, 1/2)$. Spearman IC uses only ranks; cross-sectional R^2 also depends on squared errors and scale.

Tuning instability is information. The Ridge regression in Section 8.4 selects regularization strengths ranging from 10^4 to 10^7 across 12 outer folds. This is a reason to inspect the actual inner `inner_scores` MSE curves and stability; parameter jumps alone do not prove that the curve is flat or identify a unique cause. Treating the model as "very stable" would be a misreading. The variation is a cue to focus on IC stability and portfolio-layer results rather than one particular hyperparameter.

Calibration and trading thresholds. If the rule is "trade only when predicted return exceeds one-way cost," overly dispersed forecasts ($0 < b < 1$) can trigger frequent trades and costs; overly conservative forecasts ($b > 1$) can miss opportunities. Under an explicit linear assumption, the cross-sectional calibration slope provides a scale adjustment. The intercept, group-level bias, and actual costs implied by the rule must still be checked.

Chapter 38 complete numerical example: all steps and conclusions

8. Numerical Example

8.1 Hand Calculation: 3 Assets × 12 Days, $\ell = 1$, $h = 2$

(Teaching construction.) Closing prices:

k	1	2	3	4
A	100	101	103	102
B	50	50.5	50	49.5
C	20	20	20.2	20.4

k	5	6	7	8
A	104	106	105	107
B	49	49.5	50	51
C	20.2	20	20.4	20.6

k	9	10	11	12
A	110	108	111	112
B	51.5	52	51	52.5
C	20.6	20.8	21	20.8

The label is $y_{i,k} = P_{i,k+3}/P_{i,k+1} - 1$, with interval $(D_{k+1}, D_{k+3}]$. For example, for A at $k = 1$, execution is on D_2 (101), the endpoint is D_4 (102), and $y = 102/101 - 1 = 0.009901$. For B at $k = 1$, $y = 49.5/50.5 - 1 = -0.019802$. For C at $k = 3$, execution is on D_4 (20.4), the endpoint is D_6 (20.0), and $y = 20.0/20.4 - 1 = -0.019608$. If instead we used the signal-date close D_3 (20.2) as the starting price and D_5 (20.2) as the endpoint, we would get 0; that is the error in Section 12.4.

Signal date k	`label_end`	y_A	y_B	y_C
1	D_4	0.009901	−0.019802	0.020000
2	D_5	0.009709	−0.020000	0.000000
3	D_6	0.039216	0.000000	−0.019608
4	D_7	0.009615	0.020408	0.009901
5	D_8	0.009434	0.030303	0.030000
6	D_9	0.047619	0.030000	0.009804
7	D_{10}	0.009346	0.019608	0.009709
8	D_{11}	0.009091	−0.009709	0.019417
9	D_{12}	0.037037	0.009615	0.000000
10–12	beyond sample	missing	missing	missing

One fold: Training dates $k = 1, \ldots, 6$; test dates $k = 7, 8, 9$; fit time D_7. The availability condition is $k + 3 \le 7$, or $k \le 4$. The available training rows are for $k = 1, \ldots, 4$, a total of $4 \times 3 = 12$ rows; $k = 5, 6$ are excluded, a total of $(\ell + h - 1) \times N = 2 \times 3 = 6$ rows. With overlap purging alone, overlap with the first test interval $(D_8, D_{10}]$ occurs if and only if $k + 3 > 8$, so only $k = 6$ is excluded. The interval for $k = 5$, $(D_6, D_8]$, does not overlap a test

Continued: the same complete example

interval, but ends at the close on D_8, so it is unknown at fit time D_7. Figure 38-1 draws these intervals on one timeline.

8.2 Shuffled K-Fold, Date-Block K-Fold, and Walk-Forward (Pure Noise)

Constructed panel: 40 assets, 1,260 trading days, and 22 slow-moving features (AR(1), $\rho = 0.98$), all with true loading 0; five-day labels; seed 3801. R^2_{OS} relative to the zero forecast:

Method	Ridge ($\alpha = 1$)	kNN (5 neighbors)
Shuffled 5-fold (incorrect)	+0.05%	**+56.67%**
Date-block purged 5-fold (Chapter 31, diagnostic only)	−0.73%	−43.31%
Date walk-forward (11 folds, deployment convention)	−1.65%	−40.73%

Date-block K-fold removes the "same-stock adjacent-date lookup" caused by shuffling, but training blocks may still come after test blocks, so it does not represent deployment. Walk-forward corresponds to "use only the past." With pure-noise features, both date-based methods are negative: the model fits noise and performs worse out of sample than the zero forecast.

8.3 Full-Sample Feature Selection (Pure Noise)

30 assets, 756 days, 200 pure-noise features, select 10, Ridge, date walk-forward (seed 3802):

Method	R^2_{OS} (vs. zero)	Mean daily Spearman IC
Select by full-sample |correlation| (incorrect)	+0.617%	+0.0476
Training-set selection inside `Pipeline` (`SelectKBest`)	−1.214%	−0.0101

The incorrect method produces an "IC of about 0.05" from pure noise—right in the range Chapter 37 describes as a "real but weak signal," and difficult to spot by eye.

8.4 Complete Panel Case

Constructed panel: 40 assets, 1,260 days; two real features q_1, q_2 (daily expected-return loadings of 0.04% and 0.02% per unit), 20 noise features, and three price features (`rev_5`, `mom_60_5`, `vol_20`); $\ell = 1$, $h = 5$; expanding-window walk-forward, first training window of 504 dates, test blocks of 63 dates, 12 folds total (first fit date 2016-12-08); Ridge, $\alpha \in \{10^2, \ldots, 10^7\}$, two inner date folds with 63 validation dates per fold and at least 252 training dates; seed 3803.

- Each outer fold excludes 200 rows $= (\ell + h - 1) \times N = 5 \times 40$, consistent with Section 6.1. In the first outer fold, `mom_60_5` requires 60 days of history, so 2,400 rows are dropped as missing (60×40).
- Selected α values: $10^6, 10^7, 10^6, 10^7, 10^7, 10^5, 10^5, 10^4, 10^5, 10^4, 10^7, 10^5$.
- Prediction layer (30,000 test rows, 750 dates): $R^2_{\text{OS}} = -0.325\%$ relative to zero and $+0.089\%$ relative to the training mean; cross-sectional (demeaned within date) $R^2 = +$

Continued: the same complete example

0.092%; mean daily Spearman IC 0.0255, Newey–West $t = 2.46$ (lag 4); directional hit rate 49.7%.

- Calibration: pooled slope 0.36 (level forecasts are too dispersed), cross-sectional slope 1.18 (cross-sectional forecasts are slightly conservative). By within-date prediction decile, the highest-decile minus lowest-decile mean demeaned five-day return is about $0.096\% - (-0.332\%) = 0.43\%$; the middle deciles are nonmonotonic.

Read these numbers using the decomposition in Section 6.5: level error makes pooled R^2 negative, while the cross-sectional component contains positive ranking information. The hit rate of 49.7% is below 50%, even as IC is positive. This again illustrates Chapter 37's point: directional accuracy is not the evaluation metric for this kind of strategy.

9. Python Implementation

Core module: `src/quant_course/ml_pipeline.py` (shared by Chapter 39 and Project 6). Main interfaces:

- `make_forward_labels(close, horizon, execution_lag=1)`: returns a long table keyed by (`signal_time`, `asset`) with `trade_time`, `label_start`, `label_end`, `label_available_at`, and `y`; labels at the end of the sample are missing.
- `date_walk_forward(signal_time, label_start, label_end, label_available_at, *, min_train_dates, test_dates, step=None, expanding=True, max_train_dates=None)`: date-grouped walk-forward; training rows satisfy `label_available_at` $\leq$ `fit_time`; records `n_excluded_unavailable`. It does not accept the real vendor feature `available_at`; the caller remains responsible for point-in-time feature versions and the exact fit snapshot. Raises `RuntimeError` if training and test intervals overlap.
- `fit_select_predict(...)` / `walk_forward_predict(...)`: a Pipeline with `StandardScaler` and a linear estimator; selects hyperparameters using inner date walk-forward (minimum mean validation MSE; ties go to the earlier grid value); includes `zero` and `train_mean` baselines.
- `prediction_metrics(labels, pred, horizon, train_mean_pred=None)`: R^2_{OS} (relative to zero and the training mean), IC (reuses Chapter 23's `ic_series` / `ic_summary`, `nw_lags=h-1`), and hit rate.
- `simulate_return_panel`, `price_features`, `stack_features`: construct the panel and features (for teaching).

Missing-data policy: Drop and count training rows with missing features or labels; do not predict and count test rows with missing features; raise an error if `inf` occurs.

Chapter script: `code/ch38_ml_pipeline.py`. The historical working script's default figure path may overwrite chapter figures; do not run it to validate the frozen values. The accepted I3 reader Notebook performs fresh constructed calculations, writes to a unique run directory, and generates one label-timeline figure; its English-language layer is deferred to BIL23. This batch did not run the CLI, Notebook, or course API.

Seeds 3801–3803; outputs `ch38_results.json` (refuses to overwrite) and three figures. The "incorrect" practices in Sections 8.2 and 8.3 are used only to measure bias; the code marks them `WRONG`, and they are never used to select a model. Tests: `tests/test_ml_pipeline.py` (22 cases, including row-by-row boundaries, future-data perturbation, test-label perturbation, missing values, and integration with the Chapter 29 ledger).

The hand-worked example can be reproduced as follows:

```
import pandas as pd
from quant_course import ml_pipeline as mp

close = pd.DataFrame({
    "A": [100, 101, 103, 102, 104, 106, 105, 107, 110, 108, 111, 112],
    "B": [50, 50.5, 50, 49.5, 49, 49.5, 50, 51, 51.5, 52, 51, 52.5],
    "C": [20, 20, 20.2, 20.4, 20.2, 20, 20.4, 20.6, 20.6, 20.8, 21, 20.8],
}, index=pd.bdate_range("2026-03-02", periods=12), dtype=float)
labels = mp.make_forward_labels(close, horizon=2, execution_lag=1)
fold = mp.date_walk_forward(labels.index.get_level_values(0), labels["label_start"],
    ↪ labels["label_end"],
                            labels["label_available_at"], min_train_dates=6,
    ↪ test_dates=3)[0]
used = sorted({close.index.get_loc(d) + 1 for d in labels.index.get_level_values(0)[
    ↪ fold.train_rows]})
print(round(labels.loc[(close.index[0], "A"), "y"], 6), used, fold.
    ↪ n_excluded_unavailable)
```

Expected output: `0.009901 [1, 2, 3, 4] 6`.

10. Visualization

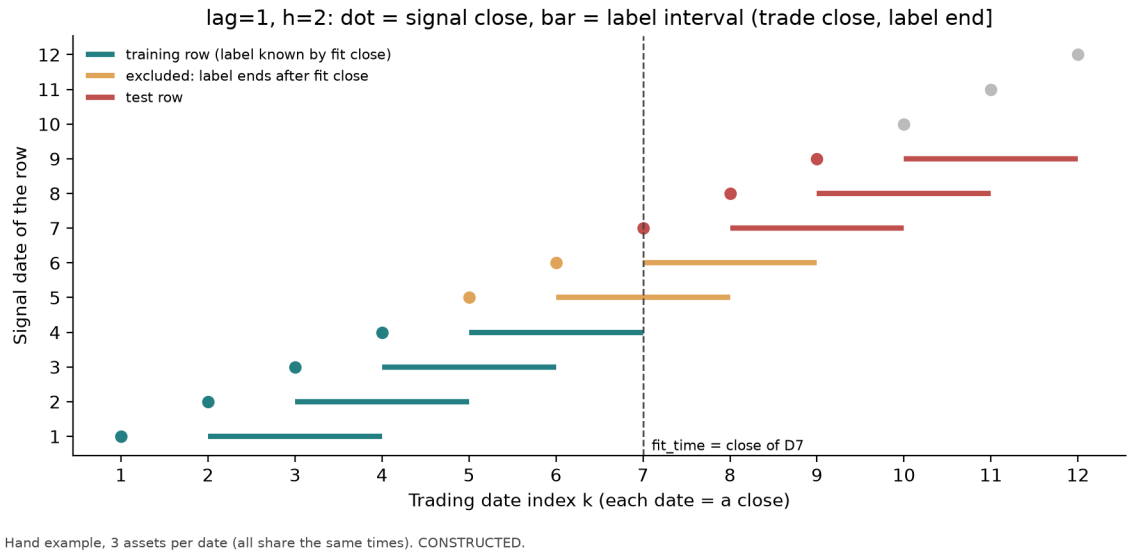


Figure 1: Label intervals and fit time for the hand-worked example

Figure 38-1: The horizontal axis is trading-date index k (each date is one close); the vertical axis is the row's signal date. Dots mark signal closes; horizontal bars show label intervals (execution close, label endpoint]. The vertical dashed line is fit time at the close on D_7. Green marks

available training rows (labels ended by D_7); orange marks excluded training candidates (labels end after D_7); red marks test rows. Observe that the orange interval at $k = 5$ overlaps no red interval but is still excluded. Constructed data.

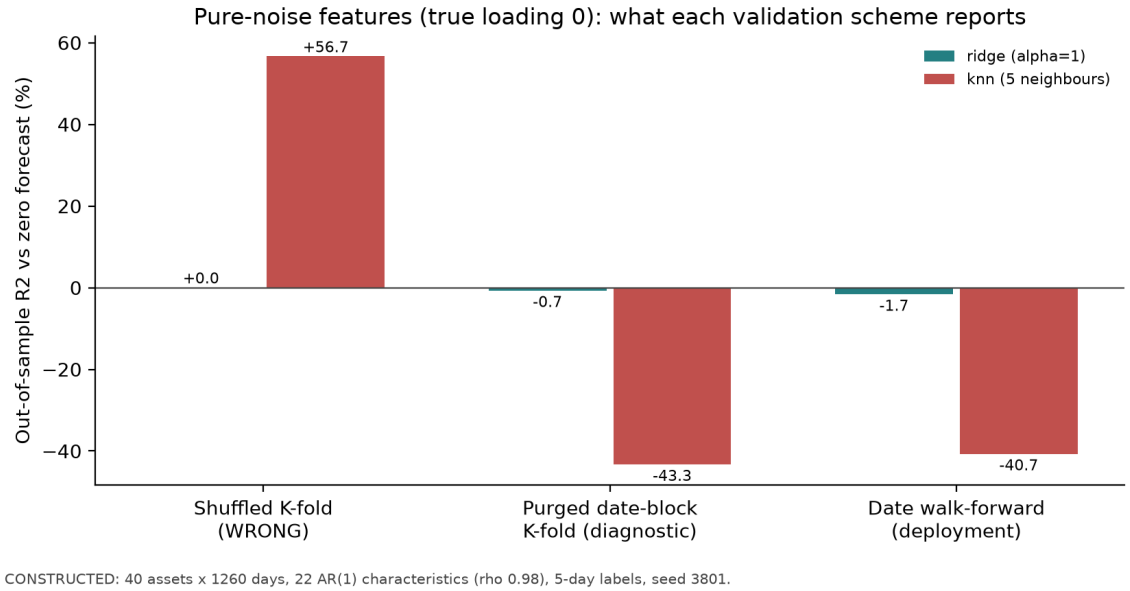


Figure 2: Results from three validation methods with pure-noise features

Figure 38-2: The horizontal axis shows three validation methods; the vertical axis is out-of-sample R^2 relative to the zero forecast (%). The two colors are Ridge ($\alpha = 1$) and five-neighbor regression. True feature loadings are 0. Observe that shuffled K-fold reports +56.7% for kNN, while both date-based methods are negative. Constructed data, seed 3801.

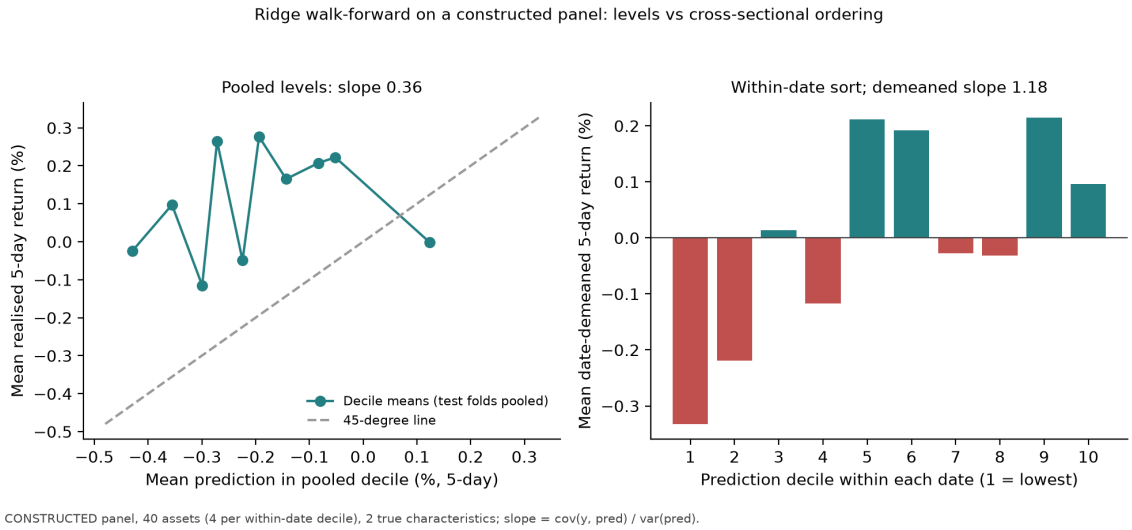


Figure 3: Level calibration and cross-sectional ranking

Figure 38-3: Left: horizontal axis is the mean prediction within pooled deciles (%, five-day); vertical axis is the group's mean realized five-day return (%); the dashed line is the 45-degree line. Right: horizontal axis is prediction decile within each date (four assets per group); vertical axis is the group's mean date-demeaned five-day return (%). Observe that the left-panel points

are far from the 45-degree line (level forecasts are unreliable); the high-minus-low spread on the right is about 0.43%, but the middle groups are nonmonotonic. Constructed panel, seed 3803.

11. Real-Market Example

This chapter contains no measured real-market results. Project 6's real-data status is blocked: the course does not yet have a licensed daily individual-stock price panel approved for internal training (see the fields and authorization requirements in `projects/project06/data_manifest.json`). Once data are obtained, applying this chapter's pipeline to real data will require three additional treatments not present in the constructed data:

1. Corporate actions: use adjusted returns for labels, but unadjusted prices plus dividend and split events for execution and the ledger (Chapters 29 and 30); do not mix the two.
2. Listings, delistings, and the stock universe: freeze the universe membership as of the signal date and include delisting returns in labels (Chapter 30, survivorship bias).
3. Asynchronous data: ingest financial features at their announcement times (Chapter 12); `available_at` may be months later than the reporting period.

Report real results separately from the constructed results. The constructed results are not evidence.

12. Common Mistakes

12.1 Using a Row-Count Gap for a Panel

```
import numpy as np
from sklearn.model_selection import TimeSeriesSplit

n_dates, n_assets, h = 30, 4, 5
rows = np.repeat(np.arange(n_dates), n_assets)            # date id of each panel row
tr, te = list(TimeSeriesSplit(n_splits=3, gap=h).split(rows))[0]
last_d, first_d = rows[tr[-1]], rows[te[0]]
print("train rows on its last date:", int((rows[tr] == last_d).sum()), "of", n_assets
    ↪ )
print("date gap between train and test:", first_d - last_d)   # WRONG: 1 date,
    ↪ contract needs lag+h-1 = 5
```

Expected output: `train rows on its last date: 1 of 4` and `date gap between train and test: 1`. The reason is that `gap` counts rows (the local scikit-learn 1.9.1 documentation says it excludes samples from the end of each training set). In this example, the last training row, 24, belongs to date 6; the first test row, 30, belongs to date 7, so the difference in date indices is 1. The maturity condition requires the most recent training signal date to be at least 6 dates before the first test date, that is, to exclude five full signal dates in between ($\ell + h - 1 = 5$). The code's `WRONG` comment mixes up the difference in date indices and the number of excluded dates; do not read that comparison as the contract. Only one stock from the last training date remains in training, and rows from the same date can also fall on different

sides across the gap and test block. Fix: `ml_pipeline.date_walk_forward`, which groups by date and filters by `label_available_at`.

12.2 Full-Sample Feature Selection or Imputation

Incorrect: fit `SelectKBest(...).fit(X_all, y_all)` first, then run walk-forward. The numerical cost is in Section 8.3: pure noise gets IC 0.048. Fix: put selection in a `Pipeline` and refit it in every training fold.

12.3 Shuffled K-Fold

Incorrect: use `KFold(5, shuffle=True)` for a panel with overlapping labels. Cost: Section 8.2 (kNN +56.7%). Fix: date walk-forward; use date-block purged K-fold only for an explicitly diagnostic purpose and pair it with embargo (Section 6.2).

12.4 Starting the Label at the Signal Close

```
import pandas as pd
close = pd.Series([20.0, 20.0, 20.2, 20.4, 20.2, 20.0])
k, lag, h = 2, 1, 2                                    # signal at index 2 (k=3
    ↪ in 1-based terms)
wrong = close[k + h] / close[k] - 1                     # WRONG: starts at the
    ↪ signal close
right = close[k + lag + h] / close[k + lag] - 1         # starts at the trade
    ↪ close
print(round(wrong, 6), round(right, 6))                 # 0.0 -0.019608
```

Reason: a signal at the close cannot execute at that same close (Chapter 29, `lag>=1`). Starting the label at the signal close includes a day's return that cannot be traded. Fix: `make_forward_labels(..., execution_lag=1)`.

12.5 Treating a Five-Day Label as a Daily Holding Return

Incorrect: rebalance daily using five-day labels and treat the average five-day label as the portfolio's daily return. Adjacent five-day labels overlap by four days, while a portfolio rebalanced daily actually holds for one day. Fix: calculate NAV from daily prices at the portfolio layer (Chapter 29 ledger), and state the rebalancing frequency separately from the label interval. Project 6 rebalances every five signal dates and records NAV daily.

12.6 Calibrating or Tuning on the Test Set

Incorrect: estimate the calibration slope during the test period and use it to evaluate that same period. Fix: estimate calibration coefficients only on the inner validation set, just like hyperparameters. The test in `tests/test_ml_pipeline.py` that changes test labels to absurd values while requiring predictions to stay unchanged guards this rule.

13. Quant Research Application

Write this chapter as the pipeline specification for Project 6 (constructed data validate only the specification itself):

1. **Data dictionary:** `event_time`, `available_at`, units, and missing-data rules for each field; `signal_time`, `trade_time`, label interval, `label_available_at`, and frequency (`projects/project06/data_dictionary.md`).
2. **Labels:** $\ell = 1$, $h = 5$, simple return starting at the execution close.
3. **Splitting:** expanding-window, date-grouped walk-forward; freeze the first training window and test-block length in advance; the availability rule automatically excludes $\ell + h - 1$ dates.
4. **Preprocessing:** compute cross-sectional transforms within each date; put all across-date transforms in a `Pipeline`.
5. **Tuning:** date walk-forward inside the outer training set; freeze the grid in advance and record the selection in each fold.
6. **Evaluation:** at the prediction layer, report R^2_{OS} against both baselines, cross-sectional R^2, IC and Newey–West t, and calibration slope. At the portfolio layer, apply an explicit rule (cross-sectional demeaning, gross exposure 1, rebalance every five days, execution lag $\ell = 1$, proportional costs) and report net-of-cost returns, turnover, and drawdown.
7. **Baselines:** zero forecast and training mean (constant across the cross-section, so the portfolio is flat).
8. **Falsification criteria:** cross-sectional IC has an insignificant Newey–West t; net-of-cost returns are not positive; or results depend on a single fold.

14. Chapter Summary

A signal date before fit time does not mean the label is known. Filter each panel row by `label_available_at` $\leq$ `fit_time`; the number of excluded dates, $\ell + h - 1$, follows from the intervals and is ℓ more than overlap-only purging. Split by date. Fit transforms that estimate parameters across dates only within training folds. With full column rank, standardization does not affect OLS; with rank deficiency, training fitted values remain unchanged but new-point predictions may change. Regularization, nearest neighbors, and feature selection are sensitive, and the feature-selection example here shows substantial leakage whose size depends on the design. Shuffled K-fold lets slowly varying features and overlapping labels "look up" each other, so pure noise can yield a large positive R^2. In evaluation, separate level error from cross-sectional error: dollar neutrality removes a common level, but actual continuous weights still depend on the relative magnitudes of demeaned predictions. Calibration should also be cross-sectional and estimated on the inner validation set.

15. Exercises

38-E01 (Availability Time)

Using this book's default cutoff τ_k (16:10 after the close on D_k is finalized), give the `available_at` for each item and decide whether it can be used for the D_k signal. Then state which conclusions change if the cutoff is moved to 20:00 that evening: (a) the D_k closing price; (b) the full-day trading volume on D_k; (c) quarterly earnings released at 16:30 on D_k; (d) a constituent change announced after hours on D_k and effective on D_{k+5}.

38-E02 (Hand-Calculated Labels and Available Rows)

Use the prices in Section 8.1, $\ell = 1, h = 2$. (a) Calculate the label and label interval for B at $k = 5$ and C at $k = 6$. (b) If fit time changes to D_8 and test dates are $k = 8, 9, 10$, which signal-date rows can be used for training? How many rows are excluded?

38-E03 (Number of Excluded Dates)

For $(\ell, h) = (1, 5)$, $(2, 3)$, and $(1, 1)$, calculate the number of dates excluded by the availability rule and by overlap purging alone. When $h = 1$, why does overlap purging exclude no rows while the availability rule still excludes ℓ dates?

38-E04 (Embargo)

A study uses date-block purged K-fold and features `rev_5` (five-day lookback) and `mom_60_5` (60-day lookback), with label horizon $h = 5$. How many trading days should the embargo be? Is embargo still needed if the study switches to expanding-window walk-forward?

38-E05 (Standardization Invariance and Its Conditions)

(a) Prove that with-intercept OLS training fitted values are invariant to affine standardization; if the training design $[\mathbf{1}, X]$ has full column rank, predictions are also invariant at every new point. (b) Give a rank-deficient counterexample where training fitted values are the same but new-point predictions differ, and explain why. (c) Give a counterexample showing that Ridge predictions can change.

38-E06 (The Lookup Mechanism in Shuffled K-Fold)

Features are AR(1) with $\rho = 0.98$ and labels are overlapping five-day returns. For a unit-variance feature, calculate the expected squared feature difference for the same asset at lags of one and three days, along with the corresponding label correlations. Explain why kNN can achieve a high R^2 under shuffled K-fold.

38-E07 (Rows in Nested Tuning)

An outer fold is fitted at D_{600}, with $\ell = 1, h = 5$ and expanding-window training dates D_1, $\ldots, D_{599}$. The inner process has two validation blocks of 63 days each and a minimum of 252 training dates. Give the validation-date ranges for the two inner folds, each fit time, and the condition inner training rows must satisfy.

38-E08 (Level–Cross-Section Decomposition)

There are two assets on each of two dates: on date 1, $y = (0.02, 0.00)$ and $\hat{y} = (0.03, 0.01)$; on date 2, $y = (-0.01, 0.01)$ and $\hat{y} = (0.00, -0.02)$. Calculate total squared error, the level term, and the cross-sectional term. Is the model useful for a dollar-neutral portfolio?

38-T01 (Added: Maturity, Snapshot, and Signal)

Use the exact timeline in Section 4. (a) Historical row A's label is ingested at 15:55 and row B's at 16:20; both rows' features were ingested the previous day. Which row can be used in the 16:09 fit? (b) A same-day feature is ingested at 16:08 and earnings at 16:30; which can be used for the 16:10 signal? (c) If fitting finishes at 16:12, what should happen to the 16:10 signal? (d) Why is checking only the date of `label_end` insufficient?

16. Programming Exercises

38-P01 (Labels and Timing)

Use `make_forward_labels` to reproduce the table in Section 8.1. Acceptance criteria: (1) `y` for the nine signal dates × three assets matches the table (tolerance 10^{-6}); (2) rows with $k \geq 10$ have missing `label_end` and missing `y`; (3) `execution_lag=0` and `horizon=True` raise `ValueError`; (4) changing any price to 0 or `inf` raises an error.

38-P02 (Feature Selection Inside a Pipeline)

On `simulate_return_panel(n_noise=200, signal_loading=0)`, compare full-sample selection with selection inside a `Pipeline`. Acceptance criteria: both use the same `date_walk_forward` folds; report R^2_{OS} and mean IC for both methods; assert that test predictions from the Pipeline method do not change when test labels change (replace test labels with the constant 5.0 and require elementwise-identical predictions).

38-P03 (Counterexample to a Row-Count Gap)

Construct an unbalanced panel (one or two stocks missing on some dates) and compare the last training date and first test date from `TimeSeriesSplit(gap=h*N)` and `date_walk_forward`. Acceptance criteria: find at least one fold where `TimeSeriesSplit` puts the same date on both sides, or where the last training row's `label_end` is later than the first test signal date.

17. Challenge Problems

38-C01 (Derive the Embargo for Purged K-Fold)

Let the longest feature lookback be L trading days and the label have parameters (ℓ, h). For a training row after the test block, state the condition under which its feature lookback window intersects any test label interval, and derive the minimum embargo length. Explain how the length depends on L and h when counted from the last label endpoint e versus the last signal date. Then discuss whether embargo can remove leakage when features are slowly changing company variables (for example, book-to-market, updated quarterly). Prerequisite: Chapter 31. Grading: condition and derivation 50%, argument about both coordinate systems and their dependence relationship 20%, discussion of slowly changing features 30%.

38-C02 (Calibration and a Trading Threshold)

The cross-sectional calibration slope on the inner validation set is $b^\circ = 0.6$; one-way cost is 0.10%; and, on a given date, two assets sampled from a larger cross-section have demeaned predictions of +0.25% and −0.05% (five-day). (a) What is each calibrated expected excess return? (b) Under the rule "trade only when calibrated $|\hat{y}^\circ|$ exceeds round-trip cost of 0.20%," do you trade either asset? (c) Why does this rule require cross-sectional calibration when a ranking portfolio does not? Open-ended; see Section 18 for the rubric.

18. Solutions / Solution Hints

38-E01

With the default cutoff τ_k = 16:10: (a) The closing price is available after it is finalized (around 16:00), so it can be used. (b) Full-day trading volume is finalized with the close; it can be used if the vendor delivers it before 16:10, otherwise its `available_at` is the actual delivery time and it cannot be used for D_k. (c) `available_at` is the 16:30 release (and ingestion) time, after the cutoff, so it cannot be used for the D_k signal; it can first be used for the D_{k+1} signal and executed on D_{k+2}. (d) `available_at` is the after-hours announcement time, after the cutoff, so it cannot be used for D_k; D_{k+5} is the effective date, not the availability time. With a 20:00 cutoff and data ingested before then, (c) and (d) become usable for the D_k signal; execution remains on D_{k+1}. (a) and (b) do not change. Under either cutoff, record the cutoff, time zone, and `available_at` in the data dictionary; do not infer availability from the date alone.

38-E02

(a) B at $k = 5$: execution is on D_6 (49.5), endpoint on $D_{5+1+2} = D_8$ (51), so $y = 51/49.5 - 1 = 0.030303$, with interval $(D_6, D_8]$. C at $k = 6$: price goes from D_7 (20.4) to D_9 (20.6), so $y = 0.009804$, with interval $(D_7, D_9]$. (b) At fit time D_8, availability requires $k + 3 \le 8$, or $k \le 5$. Candidate dates are $k = 1, \dots, 7$; $k = 1, \dots, 5$ are available (15 rows), and $k = 6, 7$ are excluded (6 rows).

38-E03

The availability rule excludes $\ell + h - 1$ dates: $(1, 5) \to 5$, $(2, 3) \to 4$, $(1, 1) \to 1$. Overlap purging excludes $h - 1$: 4, 2, and 0. When $h = 1$, overlap of the training interval $(D_{k+\ell}, D_{k+\ell+1}]$ with the test interval $(D_{s+\ell}, \dots]$ would require $k > s - 1$; there is no such candidate training date. But the label for $k = s - 1$ ends at $D_{s+\ell}$, after fit time D_s, so it is not yet known.

38-E04

First purge overlapping labels; then, relative to the last test `label_end`, apply a conservative embargo based on the longest 60-day feature lookback. Under the strict half-open return-increment interval, delete through $e + 59$. Counting the price endpoint as shared information or using $k \le e + 60$ excludes one additional date; state the coordinates explicitly (Section 6.2). Expanding-window walk-forward has only training rows before the test, so no embargo is needed; the availability rule already handles labels crossing the boundary.

38-E05

(a) See Section 6.3: equal column spaces imply equal training fitted values (full rank is not needed); with full column rank, the OLS solution is unique, and the coefficient correspondence implies equal predictions at new points. (b) In the rank-deficient example in Section 6.3, the second column of X is twice the first; both training fits are $(1, 2, 3)$. At the new point $(1, 5)$, the raw `LinearRegression` prediction is 2.20 and the `StandardScaler` -then-regression prediction is 1.75. The solution set is $\hat{\beta} + \ker Z$; the minimum-norm solutions in the two coordinate systems differ in the null-space direction $(2, -1)$. The new point does not satisfy the training relationship $x_2 = 2x_1$, so its prediction depends on this unidentified component. (c) For example, with one feature $x = (1, 2, 3, 4)$, $y = (1, 2, 3, 5)$, Ridge with $\alpha = 1$ gives a different prediction at $x = 5$ when fitted without standardization versus when x is multiplied by 10 before fitting (the coefficient in the penalty αb^2 changes with the unit). You can verify this with scikit-learn:

```
import numpy as np
from sklearn.linear_model import Ridge
x = np.array([[1.0], [2.0], [3.0], [4.0]]); y = np.array([1.0, 2.0, 3.0, 5.0])
p1 = Ridge(alpha=1.0).fit(x, y).predict([[5.0]])[0]
p2 = Ridge(alpha=1.0).fit(10 * x, y).predict([[50.0]])[0]
print(round(p1, 4), round(p2, 4))   # 5.4583 5.9935 -> different
```

38-E06

$E(x_k - x_{k+j})^2 = 2(1-\rho^j)$: for $j = 1$, 0.04; for $j = 3$, $2(1-0.98^3) = 0.1176$. Label correlation is $(5-j)/5$: 0.8 and 0.4. After shuffling, the nearest neighbors of a test row are almost all adjacent dates for the same asset (their feature distance is much smaller than 2, the distance between assets). Their labels are highly correlated with the test label, so the prediction is an average of overlapping labels.

38-E07

Outer training dates are 1..599, but outer-fold rows available by fit time satisfy $k \le 600-6 = 594$. Within these 594 dates, start at date 253 and form 63-day validation blocks: 253..315, 316..378, 379..441, 442..504, and 505..567. The remaining 568..594 dates are fewer than 63 and do not form a validation block. The final two inner validation blocks are $k = 442..504$ and $k = 505..567$, with fit times D_{442} and D_{505}. Inner training rows must satisfy $k + 6 \le 442$ (or ≤ 505), that is, they end at $k = 436$ (or 499). Outer-test labels for $k \ge 600$ never enter. These indices can be reproduced with `date_walk_forward` on a 700-day single-asset series.

38-E08

Total error: date 1 has $(-0.01)^2 + (-0.01)^2 = 0.0002$; date 2 has $(-0.01)^2 + (0.03)^2 = 0.0010$; total 0.0012. Level term: date 1 has $2 \times (0.01 - 0.02)^2 = 0.0002$; date 2 has $2 \times (0 - (-0.01))^2 = 0.0002$; total 0.0004. Cross-sectional term: on date 1, demeaned $y^\circ = (0.01, -0.01)$ and $\hat{y}^\circ = (0.01, -0.01)$ give zero error. On date 2, $y^\circ = (-0.01, 0.01)$ and $\hat{y}^\circ = (0.01, -0.01)$ give error 0.0008; total 0.0008. Thus $0.0004 + 0.0008 = 0.0012$. The ranking is entirely correct on date 1 and entirely reversed on date 2, so mean cross-sectional IC over these two dates is 0. With symmetric equal long–short weights, the two-date average gross advantage is 0; positive costs make it worse. These two observations do not establish that the population has no information.

38-P01

Complete reference method: construct the 3×12 prices from Section 9, then, for each asset and $k = 1, \ldots, 9$, independently compute `close.iloc[k+2]/close.iloc[k]-1` (here the Python loop variable k is written starting at 1, and `iloc[k]` is the execution date), and compare it with the label at signal position `k-1`. Check the final three dates for missing values. The following zero-based assertions are more direct:

```
for j in range(9):
    for asset in close:
        got = labels.loc[(close.index[j], asset)]
        expected = close[asset].iloc[j+3] / close[asset].iloc[j+1] - 1
        assert abs(got['y'] - expected) < 1e-6
        assert got['label_start'] == close.index[j+1]
        assert got['label_end'] == close.index[j+3]
assert labels.loc[(close.index[9:], slice(None)), 'y'].isna().all()
```

Then call with `execution_lag=0` and `horizon=True`, and with copies in which a price is changed to 0 or `inf`; use `try/except ValueError` to confirm that each invalid input is rejected, and explicitly `raise AssertionError` if it is not. These are reference methods for readers; this batch did not run the course API. Existing test location: `tests/test_ml_pipeline.py::test_01_hand_computed_labels`.

38-P02

Reference implementation: `code/ch38_ml_pipeline.py::selection_leak`. In the book's run, full-sample selection gives +0.617% / IC 0.0476; selection inside `Pipeline` gives −1.214% / IC −0.0101. Full method: first fix the same date folds. In the incorrect branch, fit `SelectKBest` on `X_all,y_all`, then train fold by fold. In the correct branch, build `Pipeline([StandardScaler, SelectKBest, Ridge])` in each fold and call `fit` with training rows only. Write both branches' predictions back to the original index and calculate metrics on the same valid evaluation set. For the perturbation check, keep the original folds, features, and inner rules fixed and replace only the outer-test labels in that fold:

```
changed = labels.copy()
changed.iloc[fold.test_rows, changed.columns.get_loc('y')] = 5.0

# fitted transformations and selection use fold.train_rows only
p1 = fit_training_only_pipeline(X, labels, fold).predict(X.iloc[fold.test_rows])
p2 = fit_training_only_pipeline(X, changed, fold).predict(X.iloc[fold.test_rows])
np.testing.assert_array_equal(p1, p2)
```

Here, `fit_training_only_pipeline` means the per-fold training-only selection function described above; it reads only `X.iloc[fold.train_rows]` and the corresponding nonmissing training labels. Test labels must not be used to choose k or tune hyperparameters. Existing complete engineering references are `selection_leak` and `tests/test_ml_pipeline.py::test_host_predictions_do_not_depend_on_test_labels`. This batch did not recompute the simulation.

38-P03

Following the correction already applied in I3: use `rows=np.repeat(np.arange(30),4)[1:]` to remove one row from date 0, leaving 119 rows. In the first fold of `TimeSeriesSplit(n_splits=3,gap=20)`, the final training row is index 11, from original date 3; the first test row is index 32, from original date 8. With $\ell = 1, h = 5$, the last training label does not end until date 9, so it is unknown at fit date 8 and satisfies the second failure criterion. With at most four rows per date, `gap=20` does not put the same date on both sides; do not invent that as the failure.

```
rows = np.repeat(np.arange(30), 4)[1:]
tr, te = next(TimeSeriesSplit(n_splits=3, gap=20).split(rows))
assert len(rows) == 119
assert (tr[-1], te[0], rows[tr[-1]], rows[te[0]]) == (11, 32, 3, 8)
assert rows[tr[-1]] + 1 + 5 > rows[te[0]]
```

For the date-based version, first block by unique dates, then filter rows individually by `label_available_at <= fit_time`. Check that training and test dates do not overlap, every retained label is mature, and retained training-label intervals do not overlap test-label intervals (overlap within either set is allowed). See the interface in Section 9; do not infer maturity from a row-count gap.

38-C01 (Reference Argument for the Embargo Derivation)

First purge true label overlap. A remaining future training row has finite return-increment feature support $(D_{k-L}, D_k]$. For a continuous test-label union $(D_a, D_e]$, the intervals overlap if and only if both $k > a$ and $k - L < e$. Only when rows are restricted to $k > e$ is the second condition alone sufficient; delete $e < k \leq e + L - 1$. Do not delete through this endpoint for every future row and call it a strict minimum. If the test-label union is disjoint, check each interval; using its envelope is conservative. Example: with one test signal at 0 and $\ell = h = L = 1$, the test label is $(D_1, D_2]$. For future $k = 1$, the label $(D_2, D_3]$ and feature window $(D_0, D_1]$ are both disjoint, so retain the row. For $k = 2$, the label does not overlap, but the feature window $(D_1, D_2]$ does, so delete it. Excluding the price endpoint too, or using $k \leq e + L$, deletes one additional date as a conservative protocol. Counting from the label endpoint requires only L; counting from the last test signal date also reflects the movement of the label endpoint with $\ell + h$, so the interval cannot be said to be independent of h. A quarterly company variable is not a finite return lookback, and EMA has no exact finite lookback cutoff. Check point-in-time availability and price dependence of the actual feature. Grade the condition and coordinate derivation, the difference between the two counting conventions, and the boundary for slowly changing variables.

38-C02 (Grading Rubric)

(a) $0.6 \times 0.25\% = 0.15\%$ and $0.6 \times (-0.05\%) = -0.03\%$. (b) Neither exceeds 0.20%, so neither asset is traded. Without calibration, the first asset would trigger a trade, induced by an overly dispersed forecast. (c) A pure rank portfolio depends only on order. Actual continuous weights are not pure rank weights, but a common positive rescaling also cancels after demeaning and L1 normalization. An arbitrary rank-preserving transform does not guarantee unchanged weights. A threshold rule compares returns with costs in common units and requires an explicit linear-calibration assumption; one slope alone does not establish full calibration. Grading: (a) and (b) 30% each, (c) 40%. Estimating b° on the test set loses all credit for (c).

38-T01 (Complete Answer)

(a) Row A satisfies 15:55 $\leq$ 16:09 and its historical features are readable, so it may enter training. Row B's 16:20 ingestion is after 16:09, so it cannot enter—even though its label ended at 16:00. (b) The 16:08 feature satisfies `available_at` $\leq$ 16:10 and can be used; the 16:30 earnings release cannot. The current-day feature is used only for prediction; model parameters still come from historical training rows locked at 16:09. (c) A fit completing at 16:12 cannot be described as having generated a 16:10 signal. Under a predeclared protocol, delay the signal and revise the execution convention, or use an already completed older model. Do not choose between these after seeing the result. (d) `label_end` tells when the return interval ended; `label_available_at` tells when the program could first read the label. Vendor delay separates the two. The daily rule $k + \ell + h \leq s$ is equivalent to a maturity check only under the same-close-availability assumption.

19. Further Reading

- **Must Know:** Chapter 31 (label intervals, purging, embargo, walk-forward, and nested selection). This chapter adds panels and availability times to its single-series version.
- **Must Know:** scikit-learn's *Common pitfalls* (`SK_PITFALLS`), sections on leakage and `Pipeline`; the `Pipeline` and `GridSearchCV` user guides (`SK_PIPE_DOC`, `SK_GRID_DOC`), including the explanation that `fit` acts only on training data.
- **Should Know:** Cawley and Talbot (2010), on overfitting in model selection and selection bias (`CH38_CT2010`); read the abstract and Section 5 to understand why tuning and evaluation must be separated.
- **Should Know:** López de Prado (2018), Chapter 7 (purged K-fold and embargo, `CH38_PRADO2018`); the author checked only the publisher's table of contents, not the chapter text.
- **Optional:** scikit-learn's probability calibration guide (`SK_CALIBRATION`), for reliability diagrams for classification.

20. References

- `SK_TSS`: scikit-learn `TimeSeriesSplit`. Supports the statement in Section 6.2 that `gap` counts samples (the reconnaissance report read the 1.9.1 docstring; the author checked the `gap` parameter description in the local 1.9.1 version; see `source_notes`).
- `SK_PITFALLS`: scikit-learn, *Common pitfalls and recommended practices*. Supports preprocessing leakage in Section 5 and Section 12.2.
- `SK_PIPE_DOC`, `SK_GRID_DOC`: scikit-learn user guides for Pipeline / composite estimators and hyperparameter search (1.9.1). Support the Pipeline definition in Section 5.
- `SK_CALIBRATION`: scikit-learn user guide to probability calibration. Supports the classification version of reliability diagrams in Section 5.
- `CH38_CT2010`: Cawley, G. C., Talbot, N. L. C. (2010). "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation." *JMLR* 11, 2079–2107. Supports separating tuning from evaluation in Section 5.

- `CH38_PRADO2018`: López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley. Chapter 7 table of contents. For further reading only.
- Chapter 31, `quant_course.validation` (`intervals_overlap`, `purged_kfold_splits`, `embargo`), and Chapter 23, `quant_course.factor_research` (`ic_series`, `ic_summary`). See `course/zh/chapters/ch38/source_notes.md` for the reading record.

Chapter 39 Regularized Linear Models

The historical display follows the frozen A8-R1 results; the older A8-A accepted version is preserved. This chapter does not retrain models or fabricate historical predictions. Main thread: when features are numerous and signals are weak, OLS coefficient variance can overwhelm out-of-sample performance. Ridge, Lasso, and Elastic Net trade controlled bias for lower variance, but regularization strength must be selected within the training data, and results must be evaluated against zero-prediction and training-mean baselines and after costs.

Prerequisites: Chapter 8 regression (OLS, standardization, introduction to regularization in software) and Chapter 38 pipelines. After this chapter, you will be able to: write the objective functions and scaling conventions for three scikit-learn estimators; hand-calculate four estimators under an orthogonal design; derive the bias, variance, and optimal strength $\alpha^* = \sigma^2/w^2$ for a single Ridge coefficient; explain L1/L2 geometry and their behavior under collinearity; and compare out-of-sample prediction metrics and after-cost portfolio results for six models on the Chapter 38 pipeline.

1. Learning Objectives

1. Write the objective functions for OLS, Ridge, Lasso, and Elastic Net; explain the scikit-learn 1.9.1 convention that Ridge does not divide by $2n$, while Lasso and Elastic Net do, and that the intercept is not penalized.
2. Hand-calculate all four estimators under the orthogonal design $X^\top X = nI$; verify the results with code.
3. Derive the closed-form Ridge solution, singular-value shrinkage factors, and the bias, variance, mean squared error, and optimal strength $\alpha^* = \sigma^2/w^2$ for a single coefficient.
4. Derive the Lasso soft-thresholding rule and the orthogonal Elastic Net solution; use geometry to explain why L1 produces zero coefficients while L2 does not actively produce a sparse solution.
5. Explain why standardization must be fitted within the training data and how it determines whether the penalty treats each feature equally.
6. Explain OLS sign reversals, Ridge sharing, and Lasso's choice of one feature under collinearity, and why coefficients cannot be interpreted directly as causal effects.
7. Use the date-based walk-forward and training-only tuning from Chapter 38 to compare out-of-sample and after-cost performance for zero prediction, the training mean, OLS, Ridge, Lasso, and Elastic Net on a constructed panel with more noise features than effective signals.

Completion criteria: Sections 15–17 can be completed independently; trace the frozen inputs and outputs in Sections 8–10 and explain the fresh-run versus frozen-replay boundary in Section 9; this batch does not run or retrain.

2. Why Quant Needs This

Cross-sectional stock-selection models often have dozens to hundreds of candidate features, of which only a few are genuinely useful, and those signals are weak. Under fixed-design and conditional zero-mean error assumptions, full-rank OLS coefficients are unbiased, but they can have very high variance. When there are many features and little effective information in the sample (Chapter 37: overlapping labels and low signal-to-noise ratio), that variance becomes out-of-sample error. Gu, Kelly, and Xiu (2020) report the following results for monthly U.S. individual-stock data: the out-of-sample R^2 for OLS using all features is -3.46%, OLS using only three variables is 0.16%, and Elastic Net is 0.11% (see the verified reading locations in Sections 11 and 20). Regularization does not make a model "smarter"; it makes the model less prone to errors caused by noise.

Chapter 8 already demonstrated regularization selection within the training data for a single series. This chapter adds three things: exact scaling of the objective functions (the α values of different software packages are not interchangeable); a quantitative bias–variance derivation; and evaluation against baselines and costs in a panel pipeline.

3. Intuition

Consider an example with two features and four observations:

$$X = \begin{pmatrix} 1 & 1 \\ 1 & -1 \\ -1 & 1 \\ -1 & -1 \end{pmatrix}, \qquad y = \begin{pmatrix} 3 \\ 1 \\ -1 \\ -3 \end{pmatrix}.$$

Both columns have mean 0 and population standard deviation 1, and they are orthogonal, so $X^\top X = 4I$. The OLS coefficients are $(2, 1)$.

- Ridge pulls both coefficients toward 0 by the same proportion: at $\alpha = 4$, they become $(1, 0.5)$.
- Lasso subtracts the same amount from each coefficient and stops at 0: at $\alpha = 1.5$, the coefficients become $(0.5, 0)$, so the second feature is removed.
- Elastic Net first subtracts an amount and then shrinks proportionally: at $\alpha = 1$ and L1 ratio 0.5, the coefficients become $(1, 1/3)$.

"Shrink proportionally" and "subtract a fixed amount" are the differences between L2 and L1 penalties, derived in Section 6. A simplified boundary: real features are not orthogonal, so coefficients affect one another and the closed-form solution is this simple only under orthogonality. Still, every number in this example can be reproduced exactly with scikit-learn (Section 9).

4. Mathematical Foundations

Symbol	Meaning	Dimension / units
n	Number of training sample rows	—
$X \in \mathbb{R}^{n \times p}$	Feature matrix centered and standardized on the training set	Each column is in units of one training-set standard deviation
$y \in \mathbb{R}^n$	Centered label	Decimal return
$w \in \mathbb{R}^p$	Coefficient	Return (decimal) per unit of standardized feature
$\alpha \geq 0$	Regularization strength (scikit-learn notation)	Depends on the estimator; see below
$\rho \in (0, 1)$	Elastic Net L1 ratio (scikit-learn `l1_ratio`)	—
$X = UDV^\top$	Singular value decomposition; d_j is a singular value	—
$z = X^\top y/n$	Sample covariance between each feature and the label under an orthogonal design	Return units

Intercept: center X and y on the training set first; the intercept is the training mean and is not penalized. scikit-learn does the same when `fit_intercept=True`. Prerequisites: the OLS normal equations and projection in Chapter 8, and subgradients in Chapter 9 convex optimization (used in the Lasso derivation).

5. Formal Definitions

OLS (ordinary least squares). $\hat{w}^{\mathrm{OLS}} = \arg\min_w \|y - Xw\|_2^2$. Every finite design matrix has a least-squares solution. When the matrix is rank deficient, the coefficients are generally not unique, but the training projection is unique. Invertibility of $X^\top X$ is the condition for unique coefficients and the ordinary inverse formula (full column rank; with a centered, intercept-inclusive design, there must also be enough residual degrees of freedom). Unbiased coefficients require separate statistical assumptions, such as fixed design and conditional zero-mean errors.

Ridge. In scikit-learn 1.9.1:

$$\hat{w}^{\mathrm{R}} = \arg\min_w \ \|y - Xw\|_2^2 + \alpha\|w\|_2^2.$$

The residual sum of squares in the objective is **not divided by** n. The L2 penalty stabilizes coefficients when $X^\top X$ is ill-conditioned or p is large.

Lasso (least absolute shrinkage and selection operator). In scikit-learn 1.9.1:

$$\hat{w}^{\mathrm{L}} = \arg\min_w \ \frac{1}{2n}\|y - Xw\|_2^2 + \alpha\|w\|_1.$$

The residual sum of squares is divided by $2n$. The L1 penalty can shrink a coefficient to exactly 0, so it performs feature selection.

Elastic Net. In scikit-learn 1.9.1:

$$\hat{w}^{\mathrm{EN}} = \arg\min_{w} \frac{1}{2n}\|y - Xw\|_2^2 + \alpha\rho\|w\|_1 + \frac{\alpha(1-\rho)}{2}\|w\|_2^2.$$

It combines selection (L1) and stabilization (L2), making it useful when features are correlated and a sparse solution is desired.

Consequences of the scaling conventions. Rewrite Ridge in the same form as Lasso: $\|y - Xw\|^2 + \alpha\|w\|^2 = 2n\big[\frac{1}{2n}\|y - Xw\|^2 + \frac{\alpha}{2n}\|w\|^2\big]$. If another package writes the objective as $\frac{1}{2n}\|y - Xw\|^2 + \frac{\lambda}{2}\|w\|^2$, then $\lambda/2 = \alpha/(2n)$, or $\lambda = \alpha/n$. The same numerical value, $\alpha = 1$, represents very different strengths in Ridge and Lasso. If the training sample size n changes, the relative strength of the same Ridge α changes too. When comparing software (such as glmnet) or sample lengths, convert the scaling; do not copy values directly.

Regularization path. Take a sequence of α values from large to small to obtain a curve of coefficients as α changes. Coordinate descent solves along a decreasing α grid, using the solution at the previous α as the starting value for the next (warm start; Friedman, Hastie, and Tibshirani 2010, verified in the reconnaissance report).

Scaling (standardization). Use the training-set mean μ_j and standard deviation s_j to transform a feature to $(x_j - \mu_j)/s_j$. A regularization penalty $\|w\|$ depends on coefficient units, so "treating each feature equally" has meaning only after standardization. Estimate μ and s only on training rows (Chapter 38 Pipeline).

How Scale Enters the Penalty: Converting Coefficients Back to Original Units

Let x be the original feature, b its slope in original units, and let the training standard deviations form an invertible diagonal matrix S. The standardized feature is $u = S^{-1}(x - \mu)$ and its coefficient is w. Since $w^\top u = b^\top(x - \mu)$, we have $b = S^{-1}w$ and $w = Sb$; the intercept absorbs centering and is not penalized. Therefore, the standardized penalties for identical predictions are

$$\|w\|_2^2 = \sum_j s_j^2 b_j^2, \qquad \|w\|_1 = \sum_j s_j|b_j|.$$

The penalty is on "the predicted contribution from a one-training-standard-deviation change," not an equal sum of raw slopes measured in different units. Without standardization, if the recording unit for x_2 is multiplied by 100, preserving the same predictions requires dividing b_2 by 100. Its L2 penalty becomes $1/10^4$ of the original, and its L1 penalty becomes $1/100$. After standardization, s_2 is also multiplied by 100, so $s_2 b_2$ is unchanged and the unit change cancels. A constant column with $s_j = 0$ cannot be handled by this invertible-transform proof; it must be handled according to the actual preprocessing rule.

Also check the label units. Here y is a decimal return, the standardized x is dimensionless, and w remains in return units. Under this convention, Ridge α is dimensionless, while Lasso α has return units. Elastic Net uses one numerical α to control both the L1 and L2 terms, so it must be interpreted with the label unit fixed; there is no single physical unit for it that suits both terms. If y is multiplied by $c > 0$ and predictions and coefficients must also be multiplied by c,

Ridge keeps the same α; Lasso needs $\alpha' = c\alpha$; Elastic Net must preserve both $\alpha'\rho' = c\alpha\rho$ and $\alpha'(1-\rho') = \alpha(1-\rho)$, which gives

$$\alpha' = \alpha[(1-\rho) + c\rho], \qquad \rho' = \frac{c\rho}{(1-\rho) + c\rho}.$$

Derive this by dividing the transformed objective by c^2: the residual term returns to its original scale, the L1 term gains a factor of $1/c$, and the L2 term is unchanged. If ρ is fixed, mechanically multiplying Elastic Net α by 100 does not preserve both terms. Record decimal-return and percentage-return units together with the grid. This is a unit conversion for the objective function, not retuning in this batch.

Bias–variance. The mean squared error of an estimator $\hat{w}$ is $E\|\hat{w} - w\|^2 = \|E\hat{w} - w\|^2 + \operatorname{tr}\operatorname{Var}(\hat{w})$. Regularization increases the first term and reduces the second.

Nested tuning. Select α (and ρ) only on date-based walk-forward folds within the outer training rows; evaluate the outer test once (Chapter 38).

Baselines. Evaluate zero prediction and the training-mean prediction on the same folds, with the same portfolio rule and costs (Chapter 37).

Sparsity and collinearity. Sparsity means that many coefficients are exactly 0. Collinearity means that features are highly correlated and the smallest eigenvalue of $X^\top X$ is close to 0.

6. Derivations

6.1 Closed-Form Solutions for OLS and Ridge

Take the gradient of the Ridge objective and set it to 0:

$$\nabla_w\big(\|y - Xw\|^2 + \alpha\|w\|^2\big) = -2X^\top(y - Xw) + 2\alpha w = 0 \;\Rightarrow\; \hat{w}^{\mathrm{R}} = (X^\top X + \alpha I)^{-1}X^\top y.$$

When $\alpha > 0$, $X^\top X + \alpha I$ is positive definite, so a unique solution always exists, even when $p > n$. At $\alpha = 0$, the optimization problem is OLS; only the ordinary inverse formula requires invertibility. When the matrix is rank deficient, the pseudoinverse can select a minimum-norm solution.

Orthogonal design ($X^\top X = nI$): $\hat{w}^{\mathrm{OLS}} = X^\top y/n = z$, and $\hat{w}^{\mathrm{R}} = X^\top y/(n+\alpha) = \frac{n}{n+\alpha}z$. Every coefficient is multiplied by the same shrinkage factor, $n/(n+\alpha)$. In the example from Section 3, $n = 4$, $z = (2, 1)$, and $\alpha = 4$; the factor is $1/2$, giving $(1, 0.5)$.

Singular-value form for a general design: Substituting $X = UDV^\top$ gives

$$X\hat{w}^{\mathrm{R}} = \sum_j u_j \frac{d_j^2}{d_j^2 + \alpha} u_j^\top y.$$

For the derivation, if we use the rank-r compact SVD $X = U_r D_r V_r^\top$, then V_r does not span the entire right-hand space. Therefore, when $\alpha > 0$, we must write

$$(X^\top X + \alpha I)^{-1} = V_r (D_r^2 + \alpha I_r)^{-1} V_r^\top + \frac{I - V_r V_r^\top}{\alpha}.$$

The last term acts on the null space; multiplying on the left by X makes it 0. The hat matrix is therefore still $U_r D_r (D_r^2 + \alpha I_r)^{-1} D_r U_r^\top$. We could also use a full right-singular basis and pad with zero singular values, but the first compact-SVD term alone is not the whole inverse matrix. Ridge multiplies fitted values in each principal-component direction u_j by $d_j^2/(d_j^2 + \alpha)$: directions with less variance (d_j small) are shrunk the most. Under collinearity, the smallest d_j is very small; this is exactly the direction in which OLS is unstable (ESL Section 3.4.1, verified in the reconnaissance report).

6.2 Bias, Variance, and Optimal Strength for a Single Ridge Coefficient

Assumptions: One standardized feature, $x^\top x = n$, $\sum x = 0$; $y = wx + \varepsilon$, with independent errors ε_i of mean 0 and variance σ^2.

$$\hat{w} = \frac{x^\top y}{n + \alpha} = \frac{nw + x^\top \varepsilon}{n + \alpha}.$$

Expectation: $E\hat{w} = \frac{n}{n+\alpha} w$, with bias $-\frac{\alpha}{n+\alpha} w$. For the variance, $\mathrm{Var}(x^\top \varepsilon) = \sigma^2 x^\top x = n\sigma^2$, so $\mathrm{Var}\,\hat{w} = \frac{n\sigma^2}{(n+\alpha)^2}$. The mean squared error is

$$\mathrm{MSE}(\alpha) = \frac{\alpha^2 w^2 + n\sigma^2}{(n + \alpha)^2}.$$

Differentiate with respect to α. The numerator is $2\alpha w^2 (n+\alpha)^2 - 2(n+\alpha)(\alpha^2 w^2 + n\sigma^2)$. Set it to 0 and divide by $2(n + \alpha)$: $\alpha w^2 (n + \alpha) - \alpha^2 w^2 - n\sigma^2 = 0$, hence $n\alpha w^2 = n\sigma^2$,

$$\alpha^* = \frac{\sigma^2}{w^2}, \qquad \mathrm{MSE}(\alpha^*) = \frac{\sigma^2}{n + \alpha^*} < \frac{\sigma^2}{n} = \mathrm{MSE}(0).$$

For the last step, substitute $\alpha^* w^2 = \sigma^2$; the numerator becomes $\alpha^* \sigma^2 + n\sigma^2 = \sigma^2 (n + \alpha^*)$.

This optimal formula and strict improvement require $w \neq 0$ and $\sigma^2 > 0$. If $w = 0$ and $\sigma^2 > 0$, MSE is $n\sigma^2/(n+\alpha)^2$ and tends to 0 as the strength increases; there is no finite optimum given by σ^2/w^2. If the noise is 0 and the signal is nonzero, the optimum is $\alpha = 0$. Expectations and variances are conditional on the fixed design.

Financial interpretation: The weaker the signal (w small) and the larger the noise (σ large), the stronger the optimal shrinkage. α^* does not depend on n because scikit-learn Ridge does not divide by n. **When this formula does not apply:** w and σ are unknown, so α^* cannot be calculated directly; select α by validation within the training data. With multiple correlated features, there is no similarly simple formula.

6.3 Lasso Soft Thresholding

Assumptions: Orthogonal design $X^\top X = nI$, with the scikit-learn convention that the squared loss is divided by $2n$.

Expand the objective: $\frac{1}{2n}\|y - Xw\|^2 = \frac{1}{2n}y^\top y - z^\top w + \frac{1}{2}\|w\|^2$ (using $X^\top X = nI$ and $z = X^\top y/n$). The objective separates by coordinate; each coefficient solves

$$\min_{w_j} \ \tfrac{1}{2}w_j^2 - z_j w_j + \alpha|w_j|.$$

The subgradient condition is $0 \in w_j - z_j + \alpha\,\partial|w_j|$. If $w_j > 0$, then $w_j = z_j - \alpha$, which requires $z_j > \alpha$. If $w_j < 0$, then $w_j = z_j + \alpha$, which requires $z_j < -\alpha$. If $w_j = 0$, we need $|z_j| \le \alpha$. Together,

$$\hat{w}_j^{\mathrm{L}} = S(z_j, \alpha) = \operatorname{sign}(z_j)\,(|z_j| - \alpha)_+.$$

In the example from Section 3, $z = (2, 1)$ and $\alpha = 1.5$: $S(2, 1.5) = 0.5$ and $S(1, 1.5) = 0$.

6.4 Orthogonal Elastic Net Solution

Expanding as before, each coordinate solves

$$\min_{w_j} \ \tfrac{1}{2}w_j^2 - z_j w_j + \alpha\rho|w_j| + \tfrac{\alpha(1-\rho)}{2}w_j^2 = \min_{w_j} \ \tfrac{1+\alpha(1-\rho)}{2}w_j^2 - z_j w_j + \alpha\rho|w_j|.$$

The same subgradient argument as in Section 6.3, with the quadratic coefficient changed to $1 + \alpha(1 - \rho)$, gives

$$\hat{w}_j^{\mathrm{EN}} = \frac{S(z_j, \alpha\rho)}{1 + \alpha(1-\rho)}.$$

First soft-threshold, then shrink proportionally. Example: $\alpha = 1$, $\rho = 0.5$ gives $S(z,$ $0.5)/1.5 = (1.5/1.5,\ 0.5/1.5) = (1, 1/3)$.

6.5 Geometry: Why L1 Produces Zero Coefficients

Lasso is equivalent to minimizing the residual sum of squares subject to $\|w\|_1 \leq t$ (a value of t corresponds to a value of α through Lagrangian duality). The residual-sum-of-squares contours are ellipses centered at the OLS solution. The L1 constraint set is a diamond, with corners on the coordinate axes. Expand the residual contours outward from the OLS center until they first touch the constraint set; often they touch a corner, where some coordinates are exactly 0. The L2 constraint set is a circle with no corners, so its point of tangency is generally not on a coordinate axis. Ridge therefore does not actively produce sparse support. This does not mean that a coordinate can never be 0: under an orthogonal design, Ridge also gives a zero coefficient when $z_j = 0$. If the OLS solution is already inside the constraint set, the constraint is inactive. In a general design, shrinkage is along singular directions; it does not guarantee that every original coordinate decreases in absolute value or that coefficient rankings stay fixed (the classic illustration in ESL Figure 3.11, verified in the reconnaissance report). Under orthogonality, this geometry is the soft-thresholding rule from Section 6.3: coordinates with $|z_j| \leq \alpha$ land on a corner.

6.6 Collinearity: What OLS, Ridge, and Lasso Do

Assumptions: Two standardized features with sample correlation r, such that $X^\top X/n = \begin{pmatrix} 1 & r \\ r & 1 \end{pmatrix}$.

OLS variance is $\mathrm{Var}(\hat{w}) = \sigma^2 (X^\top X)^{-1} = \frac{\sigma^2}{n(1-r^2)} \begin{pmatrix} 1 & -r \\ -r & 1 \end{pmatrix}$. Each coefficient's variance is $1/(1-r^2)$ times that under orthogonality (the variance inflation factor, VIF). At $r = 0.99$, VIF $= 50.25$ and the standard deviation is about 7.1 times larger. The stable "sum" direction and the equal-splitting description below are for $r \approx +1$; when $r \approx -1$, the stable sum/difference directions switch. The estimated coefficients have correlation $-r$: if one is overestimated, the other is underestimated, and they are often of opposite signs, even though the sum $\hat{w}_1 + \hat{w}_2$ is estimated quite precisely.

For Ridge, the eigenvalues of $X^\top X$ are $n(1+r)$ and $n(1-r)$, with eigenvectors $(1,1)/\sqrt{2}$ and $(1,-1)/\sqrt{2}$. Ridge shrinks the "sum" direction by $\frac{n(1+r)}{n(1+r)+\alpha}$ and the "difference" direction by $\frac{n(1-r)}{n(1-r)+\alpha}$. The latter is much smaller, so Ridge nearly eliminates the difference and the two coefficients become similar (an equal split). When the two features are identical ($r = 1$), Ridge splits the coefficient exactly equally (proof left as 39-C01).

For Lasso, fixing the sum of same-sign coefficients keeps the L1 penalty $|w_1| + |w_2|$ constant. The entire objective is indifferent to the allocation only when the two columns are identical ($r = 1$), so the fit also depends only on their sum (39-C01). With high but imperfect correlation, the residual term still distinguishes allocations. Full column rank makes the squared loss strictly convex and the Lasso solution unique, although selection can vary across samples and may retain only one variable. For example, let $X^\top X/n$ have diagonal entries 1 and off-diagonal entries 0.99, and let $y = X_1 + X_2$. Coefficients $(1,1)$ and $(2,0)$ both have L1 norm 2, but the latter has a normalized squared loss higher by 0.01. Theorem 1 of Zou and Hastie (2005) gives Elastic Net's

"grouping effect": the more highly correlated the features and the larger the L2 component, the closer their coefficients are (verified in the reconnaissance report; this chapter uses only the result, which is independently illustrated by the Section 8.3 simulation).

6.7 What Does Ridge Predict When α Is Large?

When α is sufficiently large relative to all eigenvalues of $X^\top X$, and the cross-sectional projection direction is nondegenerate, as $\alpha \to \infty$ we have $(X^\top X + \alpha I)^{-1} \approx I/\alpha$, $\hat{w}^{\mathrm{R}} \approx X^\top y/\alpha$, and $\hat{y} = \bar{y} + x^\top X^\top y/\alpha$. The prediction **level** approaches the training mean, while the direction of the **cross-sectional differences** approaches $X^\top y$ (the sample covariance of each feature with the label), with the whole vector scaled by $1/\alpha$. Thus, a purely rank-based portfolio can have approximately the same holdings if the ranks are stable; the actual demeaned/L1-normalized continuous weights also approach the same limit when their leading term is the same and nonzero. Whether finite $\alpha = 10^5$ and 10^7 are close depends on the actual eigenvalues; it is not guaranteed. MSE still penalizes magnitude and calibration errors even when ranks are unchanged. At low signal-to-noise ratios, stronger shrinkage may reduce estimation noise; actual inner validation must determine whether the grid boundary is optimal. The limit does not imply that level error dominates, and shrinking predictions does not remove the labels' cross-sectional error. For example, take three assets with $y = (-0.01, 0, 0.01)$, $p = (-0.02, 0, 0.02)$, and predictions ap for $a > 0$. Both means are zero, level error is zero, and Spearman IC stays at 1, but cross-sectional SSE is $2(0.01 - 0.02a)^2$: zero at $a = 0.5$ and 0.0002 at $a = 1$. Thus MSE and IC can prefer different strengths. The frequent largest-grid choice in Section 8.4 is an actual validation result; that choice alone does not identify the source of error.

7. Financial Interpretation

Coefficients are not causal effects. A standardized coefficient w_j means that, holding other features constant in the training sample's joint distribution, a one-training-standard-deviation increase in feature j raises the linear prediction for the label by w_j. It does not mean that "increasing this feature raises returns": a feature may proxy for an omitted variable; with collinearity, a case where other features stay "constant" may scarcely occur in the data; and regularization pulls coefficients toward 0 according to the penalty, so coefficient size also depends on α. In the Section 8.3 simulation, both true coefficients are 1, yet OLS assigns opposite signs to them in 41.8% of samples.

Why a zero coefficient does not mean "useless." Lasso may set a coefficient to 0 because a feature is truly irrelevant, because it is highly correlated with another selected feature, or because its signal is weaker than the current α. In Section 8.4, Lasso selects no features in 5 of 12 folds, leaving those portfolios in cash.

Regularization and portfolios. Dollar neutrality removes the common level; actual continuous weights also depend on the relative magnitudes of demeaned predictions. A common positive scaling cancels under L1 normalization. General Ridge does not multiply all original coefficients by a common factor, so it can change ranks and weights; Lasso removes features and can change ranks; large OLS coefficients on noisy features can add noise to the ranking. Thus, when the prediction-layer R^2_{OS} differences are small, portfolio-level results can still differ, and the reverse can also happen.

Costs widen the gap. A portfolio rebalanced every 5 days has annual turnover of about 14–24 times (Section 8.4). A one-way cost of 10 basis points consumes about 1.4%–2.4% of returns per year. Models with lower turnover fare better after costs.

Chapter 39 complete numerical example: all steps and conclusions

8. Numerical Example

8.1 Hand-Calculate Four Estimators (Constructed Teaching Example)

Use X and y from Section 3. We have $n = 4$, $X^\top y = (8, 4)$, and $z = (2, 1)$.

Estimator	Formula	Result
OLS	z	$(2,\ 1)$
Ridge, $\alpha = 4$	$\frac{n}{n+\alpha} z = \frac{4}{8} z$	$(1,\ 0.5)$
Lasso, $\alpha = 1.5$	$S(z, 1.5)$	$(0.5,\ 0)$
Elastic Net, $\alpha = 1, \rho = 0.5$	$S(z, 0.5)/(1 + 0.5)$	$(1,\ 0.3333)$

The maximum difference between the scikit-learn 1.9.1 fits and the closed-form solutions is 2.2×10^{-16}. For this X, `StandardScaler` is the identity transformation.

8.2 Bias and Variance

Let $n = 50$, $w = 0.2$, and $\sigma = 1$. Then $\alpha^* = \sigma^2/w^2 = 25$. For OLS, MSE $= \sigma^2/n = 0.02$. At α^*, the bias is $-\frac{25}{75} \times 0.2 = -0.0667$, squared bias is 0.004444, and variance is $50/75^2 = 0.008889$, for a total of 0.013333—one-third below OLS. The mean squared errors from 4,000 simulations match the formula curve (Figure 39-2).

8.3 Collinearity Simulation

Settings: $n = 40$, two features with correlation 0.99, true coefficients $(1, 1)$, noise standard deviation 1, and 500 repetitions (seed 3902).

Method	Mean coefficients	Coefficient standard deviations	Fraction with one coefficient equal to 0	Fraction with a negative sign
OLS	(0.967, 0.982)	(1.211, 1.216)	0	41.8%
Ridge, $\alpha = 10$	(0.866, 0.865)	(0.127, 0.126)	0	0
Lasso, $\alpha = 0.1$	(0.932, 0.920)	(0.745, 0.751)	44.8%	0

VIF $= 1/(1 - 0.99^2) = 50.25$. OLS averages are close to the true values (unbiased), but the sign in any one sample is unreliable. Ridge is biased (about 0.87) but stable. In nearly half of the samples, Lasso retains only one feature.

Continued: the same complete example

8.4 A Panel with More Noise Features Than Effective Signals

Constructed panel (seed 3904): 40 assets, 1,260 days; 3 true features (daily expected-return loadings of 0.04%, 0.02%, and 0.01% per unit), 40 noise features, and 3 price features, for 46 features in total. $\ell = 1$, $h = 5$; expanding-window date-based walk-forward, a 504-day initial training window, 63-day test blocks, 12 folds (first fit time: 2016-12-08); two inner validation blocks of 63 days each. Grids: Ridge $\alpha \in \{10^2, \ldots, 10^7\}$; Lasso $\alpha \in \{3 \times 10^{-3}, 10^{-3}, 3 \times 10^{-4}, 10^{-4}, 3 \times 10^{-5}\}$; Elastic Net $\alpha \in \{3 \times 10^{-3}, 10^{-3}, 3 \times 10^{-4}, 10^{-4}\} \times \rho \in \{0.2, 0.5, 0.8\}$. Portfolio: cross-sectional demeaning, gross exposure 1, rebalance every 5 signal days, execute at the next close, proportional one-way costs of 0 or 10 basis points; NAV recorded daily; initial capital 1 million.

Model	R^2_{OS} vs. zero	vs. training mean	Mean IC (model's own valid days)	Own valid days / candidate label days
Zero prediction	0	+0.070%	—	0/750
Training mean	−0.070%	0	—	0/750
OLS	−0.699%	−0.629%	0.0316	750/750
Ridge	−0.001%	+0.069%	0.0349	750/750
Lasso	−0.037%	+0.033%	0.0380	441/750
Elastic Net	+0.025%	+0.095%	0.0499	624/750

Model	Total return / Sharpe before costs	Total return / Sharpe after costs	Annual turnover	Maximum drawdown after costs
Zero prediction	0 / —	0 / —	0	0
Training mean	0 / —	0 / —	0	0
OLS	+34.51% / 1.62	+25.08% / 1.23	24.2	−4.43%
Ridge	+37.31% / 1.71	+27.67% / 1.33	24.2	−5.41%
Lasso	+20.83% / 1.37	+15.91% / 1.08	13.9	−4.18%
Elastic Net	+41.49% / 2.08	+33.64% / 1.75	19.0	−5.27%

Dates and inference display (A8-R1): The table reports the frozen summary means of each model's IC on its own valid dates; these are not rankings on a common date set. There are 750 candidate label dates; Ridge has 750 valid dates, Lasso 441, and Elastic Net 624. Zero prediction and the training mean are constants, so their IC is undefined and must not be entered as an observed 0. Chapter 39's frozen row-level predictions are missing, so the common dates and paired mean/difference for Ridge–Lasso and Ridge–Elastic Net, as well as the reason for each daily omission, **could not be recomputed**. The summaries confirm only that each has fewer than 750 valid dates; rows from the separate P06 experiment cannot be used to fill them in. Old fold records support that some Lasso folds were all zero, but do not independently establish the reason for every missing date.

First ask which dates are averaged. For model m, define $D_m = \{d : \text{IC}_{m,d} \text{ is defined}\}$. Its own-date mean is $|D_m|^{-1} \sum_{d \in D_m} \text{IC}_{m,d}$. For a paired comparison with Ridge, let $D_{mR} = D_m \cap D_R$, and restrict both models to D_{mR}:

$$\Delta_{mR} = \frac{1}{|D_{mR}|} \sum_{d \in D_{mR}} (\text{IC}_{m,d} - \text{IC}_{R,d}).$$

This quantity is undefined when the common set is empty. Take the intersection first and then the difference to compare both models on the same market dates. In an independent

Continued: the same complete example

constructed example, Ridge has ICs of 0.1, 0.9, and 0.2 on dates 1, 2, and 3; model m has ICs of 0.2 and 0.3 only on dates 1 and 3. Their own-date means are 0.4 and 0.25, which makes m appear worse. On the common set $\{1, 3\}$, Ridge's mean is 0.15 and m's is 0.25, so the paired difference is instead +0.10. This example only shows that date selection can change the question; it does not supply any of this chapter's historical paired values for 750/441/624 days. The frozen P06 rows can illustrate how to form P06's own intersection; they cannot serve as Chapter 39 row records.

The old shared implementation first dropped missing IC values and then calculated NW(4) by the remaining observation index. The fourth-observation lag can span more than four trading days and therefore cannot be called "four-trading-day HAC." The new candidate removes the old table's t-statistic and significance judgment; the old summary/PDF still preserves the historical t-statistic. There is no new calendar-aware HAC estimate here, and inability to recompute is not stated as a proven error. Any future re-estimation needs a separate protocol and the frozen row-level data.

(Test period: 750 signal days and 756 daily returns; Sharpe is annualized with a risk-free rate of 0.)

Reading the table:

1. OLS has the worst R^2_{OS} (-0.70%), below the training mean, but its IC is still positive and the portfolio still makes money. Prediction-layer and portfolio-layer rankings differ.
2. Ridge's R^2_{OS} is almost 0: it selects $\alpha = 10^7$ in 4 folds, 10^6 in 2 folds, and 10^5 in 6 folds (the mechanism in Section 6.7). The prediction level approaches the training mean while ranking information remains. Its own-date mean IC and portfolio result are higher than OLS; row-by-row alignment on common dates has not been recomputed.
3. In 5 of 12 folds (folds 1–4 and 12), Lasso selects $\alpha = 3 \times 10^{-3}$, which shrinks all coefficients to 0; predictions are constant and the portfolio stays in cash. The other folds retain 4–25 features. Its IC is calculated on only 441 dates with cross-sectional differences, so its own-date mean cannot be treated as a paired comparison with Ridge's 750 days. Elastic Net has only 624 valid days as well. The lower total return mainly reflects the all-zero folds.
4. Elastic Net has a 1.75 after-cost Sharpe on this frozen ledger. Its 0.050 IC is a descriptive mean on its own 624 valid days and does not support an IC advantage on common dates that have not been recomputed. This result is **for this constructed panel**, where the true features are independent and signal strengths were set by the author. It does not show that Elastic Net is better on real data.
5. Costs: 10 basis points one way reduces the average annualized return by about 2.4 percentage points (OLS 10.08% $\rightarrow$ 7.65%), consistent with annual turnover of 24.2 times $\times$ 0.1%.

9. Python Implementation

Script: `code/ch39_regularized_models.py`; shared module: `quant_course.ml_pipeline` (Chapter 38), where `backtest_predictions` connects predictions to the Chapter 29 ledger, `run_backtest` (lagged execution, proportional fees, and N returns corresponding to $N+1$ wealth points). The historical engineering entrypoint can retrain, but retraining cannot be used to fabricate the missing 3904 historical row-level predictions; the working script may also overwrite the original figures. The accepted I3 default Notebook only replays the frozen summary and display figures; it does not run the historical training. This batch did not run the CLI/Notebook/API; the English Notebook language layer is deferred to BIL23.

Seeds 3901–3904; outputs: `ch39_results.json` and three figures. Daily frequency; `signal_time` is the close on D_k, `trade_time` is the close on D_{k+1}, and the label interval is 5 days; missing-value treatment follows Chapter 38. Borrow fees, financing spreads, market impact, and capacity are not modeled. Tests: `tests/test_ch39_regularized.py` (hand-calculated estimators, α^*, soft thresholding) and `tests/test_ml_pipeline.py`.

Hand-calculation example:

```
import numpy as np
from sklearn.linear_model import ElasticNet, Lasso, LinearRegression, Ridge

X = np.array([[1.0, 1.0], [1.0, -1.0], [-1.0, 1.0], [-1.0, -1.0]])
y = np.array([3.0, 1.0, -1.0, -3.0])
for name, model in [("ols", LinearRegression()), ("ridge a=4", Ridge(alpha=4.0)),
                    ("lasso a=1.5", Lasso(alpha=1.5)), ("enet a=1 r=0.5", ElasticNet(
    ↪ alpha=1.0, l1_ratio=0.5))]:
    print(name, np.round(model.fit(X, y).coef_, 4))
```

Expected: `ols [2. 1.]`, `ridge a=4 [1.  0.5]`, `lasso a=1.5 [0.5 0. ]`, `enet a=1 r=0.5 [1.  0.3333]`.

Single-coefficient bias–variance:

```
import numpy as np
n, w, sigma = 50, 0.2, 1.0
mse = lambda a: (a * w / (n + a)) ** 2 + sigma ** 2 * n / (n + a) ** 2
grid = np.logspace(-1, 3, 4001)
print(round(grid[np.argmin(mse(grid))], 2), round(mse(sigma ** 2 / w ** 2), 6), sigma
    ↪  ** 2 / n)
```

Expected: `25.0 0.013333 0.02`.

10. Visualization

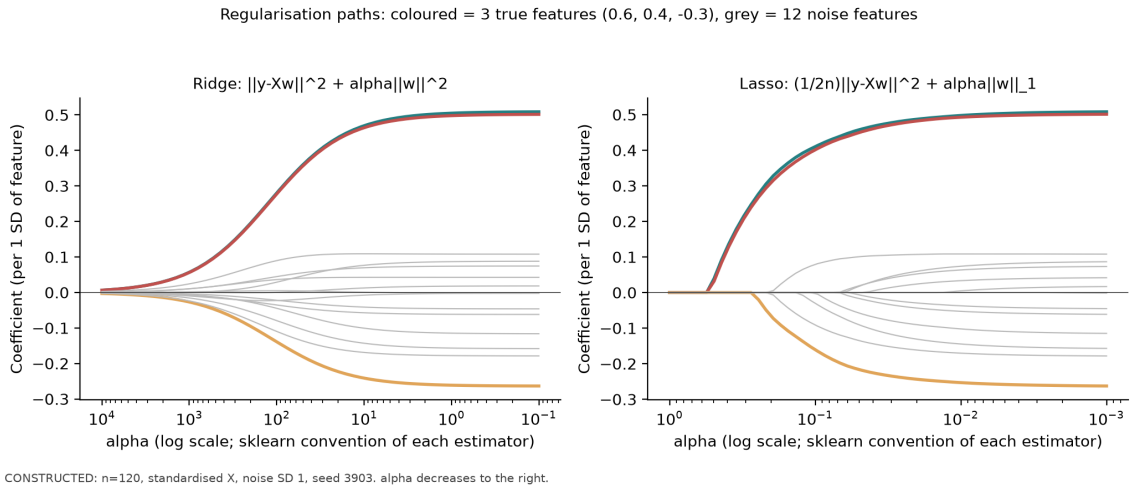


Figure 39-1 Ridge and Lasso regularization paths

Figure 39-1: The horizontal axis is α (log scale, decreasing to the right, using each estimator's scikit-learn convention); the vertical axis is the coefficient for each standardized feature. Colored lines represent three true features (true coefficients 0.6, 0.4, and -0.3); gray lines represent 12 noise features; $n = 120$, seed 3903. Observations: Ridge coefficients move smoothly away from 0 as α decreases, with all features entering together; Lasso coefficients enter from 0 one at a time. The first two true features enter at about $\alpha \approx 0.50$, the third at 0.25, and some noise features below 0.19. As $\alpha \to 0$, the estimates for the first two true features are both about 0.50 (their true values are 0.6 and 0.4): with $n = 120$, the standard error of one OLS coefficient is about $1/\sqrt{120} \approx 0.09$, which is sampling error. Constructed data.

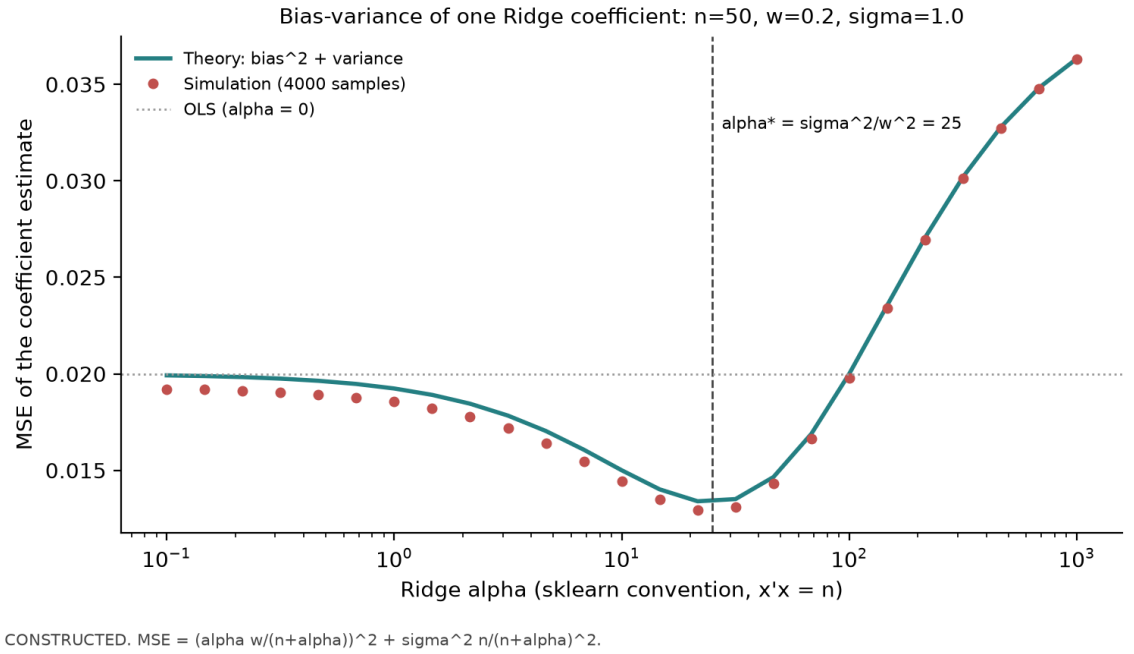


Figure 39-2 Bias–variance for a single Ridge coefficient

Figure 39-2: The horizontal axis is Ridge α (log scale); the vertical axis is the mean squared error of the coefficient estimate. The solid line is the formula $(\alpha w/(n+\alpha))^2+\sigma^2 n/(n+\alpha)^2$, dots are 4,000 simulations, the dotted horizontal line is OLS, and the vertical dashed line is $\alpha^* = 25$. Observation: the curve reaches its minimum (0.0133) at α^*; when α is too large, bias dominates, the mean squared error rises, and eventually exceeds OLS. Constructed data, seed 3901.

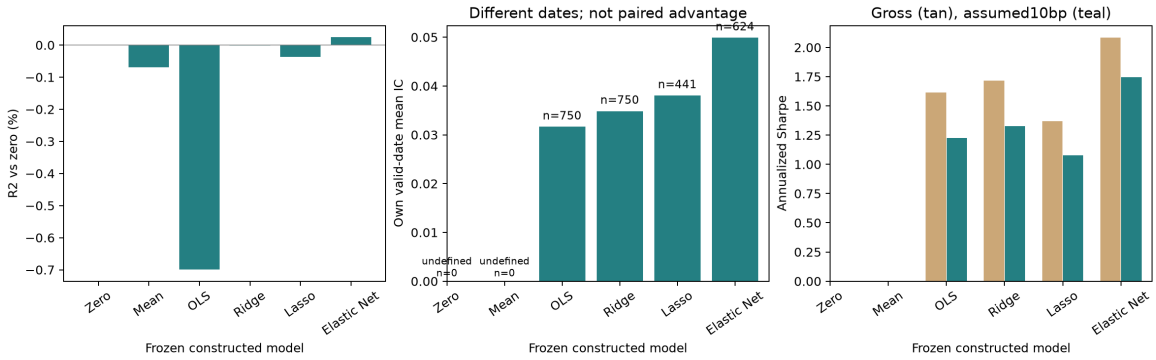


Figure 39-3 Prediction metrics and portfolio results for six models

Figure 39-3: All three panels use the six models on the horizontal axis. Left: R^2_{OS} relative to zero prediction (%). Center: the mean daily Spearman IC over each model's own valid dates, with n labeled above each bar; IC is undefined for zero prediction and the training mean, so those entries are left blank and labeled $n = 0$. The dates differ; no paired-advantage inference is made. Right: annualized Sharpe for dollar-neutral portfolios; light bars are before costs and dark bars are after one-way costs of 10 basis points. Zero prediction and the training mean remain in cash, so their Sharpe is undefined and left blank. Observation: the rankings in the left panel differ from those in the center and right panels; costs lower all model Sharpes by 0.3–0.4. Constructed panel, seed 3904.

11. Real-Market Example

This chapter reports no measured real-market numbers: Project 6 real data are blocked (see `projects/project06/data_manifest.json`). The literature results read by the author (Gu, Kelly, and Xiu 2020, Table 1, p. 2250; U.S. individual stocks, monthly, before costs) are: OLS with all features, out-of-sample $R^2 = -3.46\%$; OLS using only size, value, and momentum, 0.16%; and Elastic Net with Huber loss, 0.11%, with zero prediction as the baseline (Equation 19, p. 2249). This is directionally consistent with Section 8.4: with many features, unconstrained OLS performs poorly out of sample, while regularization or dimension reduction brings R^2 back to a small positive value. The magnitudes are not comparable to this book's constructed results because the samples, frequencies, features, and baselines differ.

Once data are available, report all columns of the Section 8.4 table for the same frozen stock universe and sample period, and add each fold's α, Lasso support-set size, and coefficient-sign stability.

12. Common Mistakes

12.1 Regularizing Without Standardizing

```
import numpy as np
from sklearn.linear_model import Lasso
rng = np.random.default_rng(0)
x1 = rng.normal(size=200); x2 = rng.normal(size=200)
y = 0.5 * x1 + 0.5 * x2 + rng.normal(size=200)
X_pct = np.column_stack([x1, x2 * 100])           # x2 recorded in percent instead of
    ↪ decimal
coef = Lasso(alpha=0.3).fit(X_pct, y).coef_        # WRONG: penalty treats a 1-unit
    ↪ change equally
print(np.round(coef, 4))
X_std = (X_pct - X_pct.mean(0)) / X_pct.std(0)    # fit mean/sd on training rows only
print(np.round(Lasso(alpha=0.3).fit(X_std, y).coef_, 4))
```

The book's output is `[0.2408 0.0054]` and `[0.2243 0.2528]`. The error is that the coefficient for x_2, recorded in percent, is 100 times smaller for the same information, but the penalty is unchanged. In the first output, the x_2 coefficient 0.0054 converts to 0.54 in decimal units, so it is barely shrunk, while x_1 is reduced from about 0.5 to 0.24. After standardization, both are shrunk equally (second output). Fix: standardize on training rows before regularization (Pipeline); when reporting coefficients, state that they are "per training standard deviation."

12.2 Comparing Ridge and Lasso α Directly

Incorrect: "Both models use $\alpha = 1$, so the penalties are the same." Why: scikit-learn Ridge does not divide by $2n$, while Lasso does (Section 5). Fix: tune each independently; convert to a common convention for comparison.

12.3 Selecting α on the Full Sample

Incorrect: use `RidgeCV` or `LassoCV` to select α with random K-fold cross-validation on the full sample, then report out-of-sample results on part of that sample. Why: selection used the test period, and random folds introduce overlap leakage (Chapter 38). Fix: outer date-based walk-forward, with inner date-based walk-forward for parameter selection (`ml_pipeline.fit_select_predict`).

12.4 Treating Coefficients as Causal Effects or Feature-Importance Rankings

Incorrect: "Lasso selected feature A and not B, so A drives returns and B is irrelevant." Why: under collinearity, Lasso chooses one feature (in Section 8.3, only one of the two is retained in 44.8% of samples), and the choice changes with the sample and α (support sets range from 0 to 25 across folds in Section 8.4). Fix: report selection frequency and sign stability across folds; causal questions require a separate identification design.

12.5 Looking Only at R^2 or Only at Sharpe

Incorrect: select the model with the highest R^2_{OS}, or report only before-cost Sharpe. Why: in Section 8.4, Ridge has R^2 near 0 yet outperforms OLS at the portfolio layer; costs reduce Sharpe by 0.3–0.4. Fix: report metrics at both layers and under both cost levels, against the zero-prediction and training-mean baselines.

12.6 Fitting `StandardScaler` on All Dates in a Panel

Error: fit `StandardScaler().fit(X_all)` first, then run walk-forward. If the training design is full rank, OLS predictions are unaffected; with exact collinearity (rank deficiency), OLS predictions for new points can also change with standardization (the Chapter 38 Section 6.3 counterexample: 2.20 versus 1.75). Ridge and Lasso can usually be affected: the penalty is based on standard deviations from the full sample, including test-period information. Fix: fit inside a Pipeline.

13. Quant Research Application

Hypothesis: Among 46 features available at the close, a few weakly predict the next 5-day cross-sectional return; a regularized linear model can produce positive after-cost dollar-neutral returns and outperform zero prediction and the training mean.

Design (frozen): Use the panel, folds, grids, portfolio rules, and costs from Section 8.4; record the selected hyperparameters and Lasso support set for each fold.

Tests: Prediction layer (out-of-sample R^2 against both baselines, each model's own valid-date and common paired-date IC; inference on t-statistics is no longer displayed for this historical case); portfolio layer (returns, Sharpe, turnover, and drawdown before and after costs); stability (fold-by-fold IC and feature selection); two cost levels.

Falsification conditions: After-cost Sharpe is not positive; a paired test on dates frozen in advance shows no advantage (the missing historical row-level data in this chapter prevented evaluation); the conclusion holds in only a few folds; or the best model changes with the grid boundary (for example, Ridge always selects the largest α, prompting a check of the inner validation curve and boundary, not a switch to IC tuning after looking at outer results; the objective and comparison protocol must be frozen in advance, see 39-C02).

Tuning judgment: Compare a grid boundary or a different evaluation criterion within the training data under a protocol frozen in advance; do not infer "the IC objective must be used" from outer-test performance.

How to state the conclusion: Elastic Net has a higher after-cost Sharpe on this chapter's frozen ledger. IC means calculated on different valid dates alone do not establish a paired prediction advantage. On real data, compare again under the same protocol and retain negative results.

14. Chapter Summary

Under assumptions such as fixed design and conditional zero-mean errors, OLS is unbiased but can have high variance. Under orthogonality, Ridge shrinks by $n/(n+\alpha)$; in a general design, it shrinks by $d_j^2/(d_j^2+\alpha)$. Lasso uses soft thresholding $S(z,\alpha)$ to produce zero coefficients; Elastic Net thresholds first and then shrinks. In scikit-learn, Ridge does not divide the residual sum of squares by $2n$, while Lasso and Elastic Net do, so α cannot be compared across estimators. For a single coefficient with $w \neq 0$ and $\sigma^2 > 0$, Ridge's optimal strength is $\alpha^* = \sigma^2/w^2$; at zero signal, the optimum is the limit of infinite shrinkage, and weaker signals call for stronger shrinkage. Under collinearity, OLS signs are unstable, Ridge splits, and Lasso may choose one feature; coefficients are not causal effects. In a panel pipeline, standardization and tuning both take place within the training data. Prediction-layer and portfolio-layer rankings can differ, MSE tuning can favor heavy shrinkage, and turnover determines how costs widen the gap. Always include zero prediction and the training mean in the same table.

15. Exercises

39-E01 (OLS)

Use the X from Section 3 and $y = (2, 0, 2, -4)$. Compute $X^\top y$, z, and the OLS coefficients. What is the intercept?

39-E02 (Ridge and Scaling Convention)

Continue from 39-E01. (a) Find the scikit-learn Ridge coefficients at $\alpha = 2$ and $\alpha = 12$. (b) Another package writes Ridge as $\frac{1}{2n}\|y - Xw\|^2 + \frac{\lambda}{2}\|w\|^2$. What λ is equivalent to scikit-learn $\alpha = 12$?

39-E03 (Bias–Variance)

Let $n = 250$, $w = 0.05$, and $\sigma = 1$. Find α^*, the bias, variance, and mean squared error at α^*, and compare with OLS.

39-E04 (Lasso Soft Thresholding)

Use the z from 39-E01. Find the Lasso coefficients at $\alpha = 0.5$, 1, and 2. At which α do all coefficients become 0?

39-E05 (Elastic Net)

Continue from 39-E01. Find the Elastic Net coefficients at $\alpha = 1$ with $\rho = 0.8$ and $\rho = 0.2$.

39-E06 (Standardization)

Feature x_2 is recorded as a percentage (its recorded values are 100 times the decimal values). Without standardization, how do the OLS and Ridge coefficients and Lasso's selection of x_2 change? Which one is unaffected? Explain.

39-E07 (Collinearity)

Two standardized features have correlation $r = 0.95$, with $n = 100$ and $\sigma = 1$. Find the standard deviation of one OLS coefficient, the VIF, and Ridge shrinkage in the "sum" and "difference" directions at $\alpha = 10$.

39-E08 (Interpreting Coefficients)

Ridge estimates a coefficient of 0.002 for the standardized feature "book-to-market ratio" (5-day return per training standard deviation). The feature's training-set standard deviation is 0.4. (a) Convert this to the coefficient per unit of book-to-market ratio. (b) Give two reasons why this does not justify saying that "increasing book-to-market by 0.4 will raise 5-day return by 0.2%."

39-T01 (New: Two Units and Two Penalty Terms)

(a) If the unstandardized feature x_2 is multiplied by 100, how do its L1 and L2 penalties change when preserving the same predictions? Why does training-set standardization cancel this effect? (b) If decimal return y is multiplied by 100, how must α change for Ridge and Lasso to preserve the scaled predictions? (c) For Elastic Net with original $\alpha = 0.01$ and $\rho = 0.5$, write the α' and ρ' that preserve both penalty terms.

39-T02 (New: Own-Date Mean and Paired Difference)

Using the three-day independent constructed example in Section 8.4, write D_R, D_m, and D_{mR}, then calculate the two own-date means and the paired difference on the common set. Can counts of valid days alone complete this calculation?

16. Programming Exercises

39-P01 (Regularization Path)

On constructed data with $n = 120$ and 15 standardized features (3 true and 12 noise features), plot the Ridge and Lasso paths. Acceptance checks: (1) all Lasso coefficients are 0 at the largest α; (2) at the smallest α, the maximum absolute difference between the Lasso coefficients and OLS is less than 0.01; (3) report the first nonzero α for every feature; (4) fix the seed and label the axis with the convention used.

39-P02 (Panel Comparison)

Use `ml_pipeline` to reproduce the Section 8.4 table. Acceptance checks: (1) each fold's training rows satisfy label_available_at $\leq$ fit_time (read `DateFold`); (2) zero prediction and the training mean have total portfolio return 0; (3) for the same predictions, after-cost NAV is no higher than before-cost NAV; when fees are positive and the wealth factor is positive, the path endpoint must be strictly lower; the zero/mean cash baselines must be equal; (4) output the hyperparameters for every fold; (5) without changing any settings, rerun with seed 3905 and report whether the conclusion changes (retain the result and do not select a seed).

17. Challenge Problems

39-C01 (Ridge and Lasso under Perfect Collinearity)

Two features are identical ($x_1 = x_2 = x$, with $x^\top x = n$). Let $\alpha > 0$. (1) Prove that the Ridge solution is unique and that $\hat{w}_1 = \hat{w}_2 = \frac{x^\top y}{2n+\alpha}$. (2) Prove that the Lasso solution set is a line segment and write it out. (3) Is the Elastic Net solution unique when $\alpha(1-\rho) > 0$? What if $\alpha = 0$? Additional prerequisite: Chapter 9 convexity. Scoring: one-third each for parts (1), (2), and (3).

39-C02 (Tune on MSE or IC?)

In Section 8.4, Ridge selects α using inner validation MSE and often selects the largest value. Design a tuning scheme that uses **mean daily Spearman IC** on the inner validation set as its criterion. (a) Explain how it differs from the MSE criterion under the mechanism in Section 6.7. (b) Identify risks from the new criterion (for example, more selection opportunities or sensitivity to a few dates). (c) Explain how to compare the two criteria without looking at the outer test. This is an open-ended question; the scoring guide is in Section 18.

39-C03 (Date and Legacy t-Statistic Display Diagnostic)

Ridge, Lasso, and Elastic Net have 750, 441, and 624 valid dates, respectively, but only summary data. Can their IC means be compared directly? Can P06 predictions fill in paired evidence for this chapter? If the valid trading-sequence indices are 0, 2, 3, 5, and 8, which two days does the fourth observation lag after dropping missing values connect?

18. Solutions and Solution Hints

39-E01

For $X^\top y$, the first column gives $2+0-2+4=4$ and the second gives $2-0+2+4=8$. Thus $z=(1,2)$ and OLS is $(1,2)$. $\bar{y}=0$, so the intercept is 0.

39-E02

(a) $\frac{4}{4+\alpha}z$: at $\alpha=2$, $(0.6667, 1.3333)$; at $\alpha=12$, $(0.25, 0.5)$. (b) Multiplying $\frac{1}{2n}\|y-Xw\|^2+\frac{\lambda}{2}\|w\|^2$ by $2n$ gives $\|y-Xw\|^2+n\lambda\|w\|^2$, so $n\lambda=\alpha$, and $\lambda=12/4=3$.

39-E03

$\alpha^*=1/0.0025=400$. Bias $=-\frac{400}{650}\times 0.05=-0.03077$; squared bias is 0.000947; variance is $250/650^2=0.000592$; MSE is $0.001538=1/650$. OLS is $1/250=0.004$. Shrinkage lowers MSE to 38% of OLS, at the cost of an estimate whose expectation is only 38% of the true value (250/650).

39-E04

$z=(1,2)$. At $\alpha=0.5$: $(0.5, 1.5)$; at $\alpha=1$: $(0,1)$; at $\alpha=2$: $(0,0)$. All coefficients are 0 when $\alpha\geq\max|z_j|=2$.

39-E05

At $\rho=0.8$: $S(z,0.8)/(1+0.2)=(0.2,1.2)/1.2=(0.1667,1)$. At $\rho=0.2$: $S(z,0.2)/(1+0.8)=(0.8,1.8)/1.8=(0.4444,1)$.

39-E06

OLS: if the training design has full column rank, the x_2 coefficient is divided by 100, while predictions stay the same (Chapter 38, Section 6.3). If the design is exactly collinear (rank deficient), the software's default minimum-norm solution changes with the units, and predictions for new points can differ. Ridge: the penalty on x_2 becomes relatively weaker (the same prediction contribution needs only 1/100 of the coefficient), so x_2 is shrunk less and predictions change. Lasso: similarly, x_2 becomes easier to select and x_1 relatively easier to remove. Only OLS with full column rank is unaffected. After standardizing on the training rows, all three are invariant to the recording units.

39-E07

VIF $= 1/(1 - 0.9025) = 10.26$; OLS coefficient standard deviation $\sqrt{10.26/100} = 0.320$ (0.1 under orthogonality). In the "sum" direction, Ridge shrinks by $\frac{n(1+r)}{n(1+r)+\alpha} = \frac{195}{205} = 0.951$; in the "difference" direction, by $\frac{5}{15} = 0.333$.

39-E08

(a) $0.002/0.4 = 0.005$ per unit. (b) Any two: the coefficient is a conditional linear prediction in the training distribution, not an intervention effect; book-to-market may proxy for omitted risk or other features (collinearity); Ridge has shrunk the coefficient toward 0, so its magnitude depends on α; or "increasing by 0.4" may coincide with changes in other features in the data.

39-P01

Reference implementation: `code/ch39_regularized_models.py::path_data`. In the book's run, the first-entry α values for the first three true features under Lasso are 0.4954, 0.4954, and 0.2454; the first three coefficients at the smallest α are 0.508, 0.501, and -0.263. Complete reference method: use a fixed seed-3903 generator, first draw 120×15 independent normal features and standardize them on this training set, set the first three loadings to $(0.6, 0.4, -0.3)$ and the rest to 0, add noise with standard deviation 1, and center y. Fit Lasso on a decreasing 60-point grid from 10^0 to 10^{-3} and Ridge on a 60-point grid from 10^{-1} to 10^4; fit each grid on the same X, y and plot each coefficient column. Coordinate descent for Lasso can update $w_j \leftarrow S(x_j^\top (y - Xw + x_j w_j)/n, \alpha)/(x_j^\top x_j/n)$, iterating until both coefficient changes and KKT residuals meet tolerance; alternatively, call `lasso_path`. Assertions and output:

```
assert np.max(np.abs(lasso_coefs[:, 0])) < 1e-8
ols = LinearRegression(fit_intercept=False).fit(X, y).coef_
assert np.max(np.abs(lasso_coefs[:, -1] - ols)) < 0.01
for j in range(X.shape[1]):
    nz = np.flatnonzero(np.abs(lasso_coefs[j]) > 1e-8)
    print(j, None if len(nz) == 0 else alphas_l[nz[0]])
```

Distinguish the three true features from the twelve noise features in the legend, label each horizontal axis with its estimator's convention, and save the maximum/minimum grid endpoints and nonzero tolerance. This gives the method; the historical path was not rerun.

39-P02

Reference implementation: `panel_case` and `backtest_predictions`. The full procedure below is provided for a future explicitly authorized experiment; this batch does not run a 3904 retraining or seed 3905.

1. Fix every constructed parameter from Section 8.4; run `simulate_return_panel` → `make_forward_labels(horizon=5,execution_lag=1)` → combine `price_features` and characteristics → `stack_features`; verify the 46 columns.
2. Call `date_walk_forward` on the same label table (504/63 dates), saving each fold's fit_time, train_rows, and test_rows. Assert that each training label is mature, each training signal date precedes the fit date, and no training label interval overlaps any test label interval (overlap among five-day label intervals within the training set or within the test set is allowed).
3. Use the fixed six-model grids from Section 8.4; call `walk_forward_predict(..., inner_val_dates=63,inner_folds=2,inner_min_train_dates=252)` for each model; save row-level predictions and each fold's `chosen_params`, then calculate the two baseline R^2 values, each model's valid IC date set, and common date sets.
4. For each model, use the same predictions in `backtest_predictions(close,pred, rebalance_every=5,fee_rate=0.0)` and with fee 0.001; compare NAV on identical dates, costs, and turnover. Zero and training-mean predictions should have total return 0; the after-cost endpoint should be lower on paths with nonzero fees and actual trades, and equal when there is no trading. Each path must have $N + 1$ wealth points for N daily returns.
5. Preserve all settings exactly, then change only the seed to 3905, save in a separate unique directory, and retain both positive and negative results and parameter choices; do not return later to select a favorable seed. This new run has a new identity and cannot fill in the missing 3904 historical rows. Report its results whatever they are. If authorization to run has not been granted, as in this batch, record "not run" and do not invent results.

Use the tests beginning with `test_host_backtest` in `tests/test_ml_pipeline.py`. Displayed numbers are in Section 8.4; the own-date IC counts of 750/441/624 are from the frozen summary. Common dates and paired means could not be recomputed; the old compressed-index NW t-statistic is not a four-trading-day HAC result. Do not retrain to fabricate historical rows. Fold hyperparameters are saved in `ch39_results.json`. Step (5) must be reported whether its result is favorable or not.

39-C01 Reference Proof

(1) The objective is $\|y-(w_1+w_2)x\|^2+\alpha(w_1^2+w_2^2)$. Let $s=w_1+w_2$. For fixed s, $w_1^2+w_2^2$ is minimized at $w_1=w_2=s/2$, so the optimal solution is an equal split. Minimizing $\|y-sx\|^2+\alpha s^2/2$ over s then gives $s=\frac{2x^\top y}{2n+\alpha}$ and $\hat{w}_j=\frac{x^\top y}{2n+\alpha}$. The objective is strictly convex, so the solution is unique. (2) The Lasso objective depends only on s and $|w_1|+|w_2|$; for same-sign coefficients, $|w_1|+|w_2|=|s|$. The optimal $s^*=S(x^\top y/n,\alpha)$, and the solution set is $\{(w_1,w_2):w_1+w_2=s^*,\ w_1w_2\geq 0\}$, a line segment. When $s^*=0$, it collapses to a point. Here $\alpha>0$ ensures that opposite-sign allocations increase the L1 cost; when $\alpha=0$, all allocations summing to $x^\top y/n$ are optimal, including opposite-sign allocations, so the set is a full line. (3) When $\alpha(1-\rho)>0$, the L2 term makes the objective strictly convex, so the solution is unique and splits equally. $\rho<1$ alone does not guarantee uniqueness when $\alpha=0$.

39-C02 (Tune on MSE or IC?) Scoring Guide

Full credit: 10 points. (a) State that MSE penalizes magnitude and calibration errors, while IC uses cross-sectional ranks; even when common positive scaling preserves ranks, the criteria may prefer different strengths: 3 points. (b) Identify at least two risks of the new criterion, such as high daily IC noise, selection bias from more choices, or non-unique choices when IC is insensitive to strength (tie-breaking): 3 points. (c) Compare the two criteria without looking at the outer test, using the same "select then evaluate" process on inner folds (for example, with a second-level holdout on the last portion of the outer training data), or freeze the criterion in advance: 4 points. Using the outer test to compare the criteria loses 4 points.

39-C03 Complete Answer

Do not treat own-date IC means calculated over different date sets as a paired advantage on common dates. Define the valid sets D_m and $D_m\cap D_R$, then calculate both models' means and their difference on the common set. With summary data alone, these paired quantities could not be recomputed. P06 is a different experiment and cannot substitute. After missing dates are dropped, the fourth-observation lag connects trading-sequence index 8 to index 0, an 8-trading-day gap; this is not four-trading-day HAC. The old t-statistic remains in the old accepted summary, but is not presented as new inference; do not retrain to fabricate historical rows.

39-T01 Complete Answer

(a) The original slope b_2 is divided by 100, so L1 becomes 1/100 and L2 becomes 1/10,000 of their original values. The training standard deviation s_2 is multiplied by 100, so the standardized coefficient $w_2 = s_2 b_2$ is unchanged and the penalty is unchanged. (b) Ridge's objective and penalty are both multiplied by 10,000, so α is unchanged. Lasso's loss is multiplied by 10,000 but $|w|$ only by 100, so $\alpha' = 100\alpha$. (c) Elastic Net: $\alpha' = 0.01(0.5 + 100 \times 0.5) = 0.505$, $\rho' = 50/50.5 = 100/101$. Check that $\alpha'\rho' = 0.5 = 100\alpha\rho$ and $\alpha'(1-\rho') = 0.005 = \alpha(1-\rho)$. The original-unit L1 and L2 coefficients after scaling meet the required equivalences; no new model fit is needed.

39-T02 Complete Answer

$D_R = \{1, 2, 3\}$, $D_m = \{1, 3\}$, and $D_{mR} = \{1, 3\}$. Ridge's own-date mean is $(0.1 + 0.9 + 0.2)/3 = 0.4$; m's own-date mean is $(0.2 + 0.3)/2 = 0.25$. On the common set, Ridge's mean is 0.15 and m's is 0.25; the paired difference is $\Delta_{mR} = [(0.2 - 0.1) + (0.3 - 0.2)]/2 = 0.10$. The counts give neither the date membership nor the values on those dates, so they cannot yield a paired difference. The historical row-level gap in this chapter remains unrecomputed; P06 cannot replace those data.

19. Further Reading

- Must know: Chapter 8 (OLS, standardization, regularization selection for a single series) and Chapter 38 (the pipeline).
- Must know: scikit-learn 1.9.1 documentation for `Ridge`, `Lasso`, and `ElasticNet` (references `CH08_DOC_08`–`CH08_DOC_10`); read the objective functions and descriptions of `alpha` and `l1_ratio`.
- Should know: Hastie, Tibshirani, and Friedman, `The Elements of Statistical Learning`, 2nd ed., Section 3.4 (reference `ESL_2ND_ED`): singular-value shrinkage for Ridge, Lasso, and L1/L2 geometry.
- Should know: the grouping effect in Theorem 1 of Zou and Hastie (2005) (reference `CH08_DOC_17`); coordinate descent and regularization paths in Friedman, Hastie, and Tibshirani (2010) (reference `CH39_FHT2010`).
- Should know: Section 2.2 and Table 1 of Gu, Kelly, and Xiu (2020) (reference `CH37_GKX2020`): out-of-sample comparisons of linear, regularized, and nonlinear models for large-scale stock prediction.
- Optional: the original papers by Hoerl and Kennard (1970) and Tibshirani (1996) (references `CH08_DOC_15` and `CH08_DOC_16`, registered in Chapter 8 as publisher records).

20. References

- CH08_DOC_08, CH08_DOC_09, CH08_DOC_10: scikit-learn Ridge, Lasso, and ElasticNet. The author checked the three objective functions in the docstrings for local scikit-learn 1.9.1 (see source_notes).
- ESL_2ND_ED: Hastie, T., Tibshirani, R., and Friedman, J. (2009). The Elements of Statistical Learning, 2nd ed., Section 3.4; the reconnaissance report records reading Equations (3.41), (3.44), and (3.47), Table 3.4, and Figure 3.11; the author did not reopen the book (the download page requires browser interaction). The derivations in Sections 6.1–6.5 are internally consistent and were checked with the Section 8.1 code.
- CH08_DOC_17: Zou, H., and Hastie, T. (2005). "Regularization and variable selection via the elastic net." JRSS-B 67(2), 301–320. The grouping effect in Theorem 1 was verified in the reconnaissance report.
- CH39_FHT2010: Friedman, J., Hastie, T., and Tibshirani, R. (2010). "Regularization Paths for Generalized Linear Models via Coordinate Descent." Journal of Statistical Software 33(1), 1–22. The reconnaissance report records reading Sections 1–2.
- CH37_GKX2020: Gu, Kelly, and Xiu (2020), Equation (19), p. 2249, and Table 1, p. 2250, read by the author.
- Reading record: course/zh/chapters/ch39/source_notes.md.

Chapter 40: Tree-Based Models

Author: Yuxiao Ma. The existing A8-B constructed implementation has been accepted within a limited scope and is pending formal integration; this does not mean that real research is complete. Core idea: trees approximate nonlinearities and interactions with piecewise-constant predictions; random forests reduce variance by averaging; gradient boosting iteratively repairs residuals along the negative gradient. Their depth, leaf size, number of rounds, and learning rate must all be selected using date-based validation inside the training data. Feature importance is a bounded diagnostic; economic conclusions still depend on a portfolio compared with benchmarks under the same accounting and costs. Prerequisites: the Chapter 38 pipeline (date-based walk-forward validation, label-availability times, and in-training tuning) and Chapter 39 regularized linear models (bias–variance and benchmark comparison). After completing this chapter, you should be able to hand-calculate an optimal split and two rounds of boosting; derive the random-forest variance formula; explain why built-in early stopping in sklearn 1.9.1 HistGradientBoosting cannot be used directly for a panel; compare tree and linear models on predictive metrics and after-cost results within the Chapter 38 pipeline; compute date-wise permutation importance and explain its limits with correlated features.

1. Chapter Objectives

1. Write down the split objective for a regression tree, derive "split gain $= \frac{n_L n_R}{n}(\bar{y}_L - \bar{y}_R)^2$," and hand-calculate a split for eight stocks.
2. Explain how depth and leaf sample size (`min_samples_leaf`) constrain tree variance, and identify which splits are infeasible under a given constraint.
3. Derive the variance of an average of B trees with pairwise correlation ρ, $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, and explain why random forests sample features at random.
4. Derive why each round of gradient boosting fits residuals under squared loss and how the learning rate shrinks each step; hand-calculate two rounds of boosting; explain how the HistGradientBoosting leaf value $-G/(H+\lambda)$ relates to Ridge shrinkage in Chapter 39.
5. Select depth, leaf size, and number of rounds (early stopping) by date within the training data, and explain why sklearn's built-in random holdout for early stopping leaks information in panels.
6. Compute date-wise permutation importance, compare it with mean decrease in impurity (MDI), and explain how correlated features, the sample period, and multiple comparisons affect it; explain SHAP's additivity and limitations.
7. Compare out-of-sample and after-cost performance for zero prediction, the training mean, OLS, Ridge, a single tree, a random forest, and gradient boosting on an interaction panel and the Chapter 39 linear panel; use the Project 6 v2 protocol for ablation and stability reporting.

Completion criteria: You can independently complete Sections 15–17; `code/ch40_tree_models.py` reproduces the numbers in Sections 8–10.

2. Why Quant Needs Tree Models

Linear models assume that each feature has a fixed slope with respect to expected returns. Two common departures in cross-sectional research are effects that vary with another feature (interactions—for example, a price-volume signal that works only for illiquid stocks) and non-monotone effects (both tails are unusual, while the middle is ordinary). Linear models require the researcher to add interaction and piecewise terms by hand, and the number of candidate combinations grows quickly with the number of features.

Tree models automatically search for "which feature and threshold should split the sample." Gu, Kelly, and Xiu (2020) report higher out-of-sample R^2 for random forests and gradient-boosted trees than for linear benchmarks on monthly U.S. individual-stock data (see Sections 11 and 20 for the specific figures and review status). The same study also illustrates the costs: trees have many hyperparameters, are sensitive to the sample period, and can change their feature-importance rankings across training windows. This chapter checks these costs within the Chapter 38 pipeline: depth and number of rounds are selected only on date-based validation windows inside the training data; importance is computed on in-training validation windows; and economic value is compared with zero prediction, the training mean, and linear models using the same accounting and costs.

3. Intuition

There are eight stocks. Feature x is the cross-sectional rank of the past five-day return (1 = largest decline, 8 = largest gain), and label y is next week's return (%). These are constructed teaching values:

Stock	1	2	3	4
x	1	2	3	4
y (%)	−2	−1	−1	0

Stock	5	6	7	8
x	5	6	7	8
y (%)	2	1	3	2

A depth-1 tree (a stump) asks one question: "Is $x \le s$?" It then predicts one constant on each side. At $s = 4$, the left-side mean is -1%, the right-side mean is 2%, and the overall mean is 0.5%. Without a split, the sum of squared errors is 22; after the split it is $2 + 2 = 4$, a reduction of 18. Section 8 calculates each threshold; $s = 4$ gives the largest reduction.

The boundary of this example: it has one feature and no independent validation set from later dates. A tree can keep splitting until each leaf contains one stock, reducing training error to 0. That zero training error reveals the risk of overfitting, but these eight rows cannot prove that the next week must be worse; evaluation requires validation or test data from later dates. Depth and leaf-size constraints, random-forest averaging, and the learning rate and number of boosting rounds all address the problem of splitting too finely.

4. Mathematical Setup

Symbol	Meaning	Dimension / unit
n	Number of sample rows in a node (or training set)	—
$x_i \in \mathbb{R}^p$	Feature vector for row i; x_{ij} is feature j	Original feature units (trees do not require standardization)
y_i	Label: simple return from `trade_time` to `label_end`	Decimal; hand calculations use %
R_m	Feature-space region corresponding to leaf m	—
c_m	Prediction in leaf m	Same unit as y
$\mathrm{SSE}(S) = \sum_{i \in S}(y_i - \bar{y}_S)^2$	Sum of squared errors in node S	Squared units of y
B	Number of trees in the forest	—
σ^2, ρ	Prediction variance of one tree at a fixed x; correlation between two tree predictions	Units of y squared; dimensionless
$F_m(x)$	Prediction after boosting round m	Same unit as y
$\nu \in (0, 1]$	Learning rate (sklearn `learning_rate`)	—
M	Number of boosting rounds (HGB `max_iter`)	—
$L(y, F) = \frac{1}{2}(y - F)^2$	Squared loss	Units of y squared

Prerequisites: Chapter 8 least squares (the mean minimizes squared error), Chapter 9 gradient descent, Chapter 39 bias–variance decomposition and the Ridge shrinkage factor $n/(n + \alpha)$, and Chapter 38 date-grouped walk-forward validation with label_available_at $\leq$ fit_time. The clock in this chapter follows Chapter 38: `signal_time` is the close of D_k, `trade_time` is the close of D_{k+1}, and `label_end` = `label_available_at` = D_{k+1+h}, where $h = 5$.

5. Formal Definitions

Regression tree (CART). Recursive axis-aligned splits partition feature space into disjoint rectangular regions $R_1, \ldots, R_J$, with a constant prediction in each region:

$$f(x) = \sum_{m=1}^{J} c_m \, \mathbf{1}\{x \in R_m\}.$$

At each split, choose feature j and threshold s, dividing node S into $S_L = \{i : x_{ij} \leq s\}$ and $S_R = \{i : x_{ij} > s\}$. The default `squared_error` criterion in sklearn 1.9.1 `DecisionTreeRegressor` chooses (j, s) to minimize $\mathrm{SSE}(S_L) + \mathrm{SSE}(S_R)$; the leaf value is the mean of y in that leaf. The algorithm is greedy, one level at a time, and is not guaranteed to find a global optimum. Not every rectangular partition can be represented by a recursive tree of a given depth. Trees address the need for linear models to specify interactions and thresholds by hand; the inputs are features and labels, and the output is a piecewise-constant prediction.

Depth and leaf samples. `max_depth` limits the number of splits from root to leaf, allowing at most 2^d leaves. `min_samples_leaf` requires each leaf to contain at least the specified number of training rows; a split that violates this condition is infeasible. Both constraints limit model complexity and therefore variance. In a panel, a row is one stock on one day, and rows from the same day are highly correlated. "1,000 rows" is far from 1,000 independent observations (see the effective sample discussion in Chapter 37), so leaf sizes generally need to be much larger than in nonfinancial applications.

Bagging and random forests. Bagging draws B bootstrap samples from the training set and grows one tree on each; predictions are averaged. At each split, a random forest considers only a random subset of `max_features`, reducing dependence across trees. In sklearn 1.9.1, `RandomForestRegressor` defaults to `max_features=1.0` (no feature subsampling, equivalent to bagging); `max_samples` controls how many rows each tree draws.

Gradient boosting. Starting from a constant F_0, each round fits a shallow tree h_m to the negative gradient of the loss with respect to the current predictions, then adds it with a learning rate:

$$F_m(x) = F_{m-1}(x) + \nu\, h_m(x), \qquad m = 1, \ldots, M.$$

This extends Chapter 9 gradient descent from parameter space to "prediction-function" space: a tree approximates the direction of each step. sklearn 1.9.1 `HistGradientBoostingRegressor` (HGB) first bins each feature into at most 255 bins using training data, then searches for splits using sums of gradients and second derivatives within bins.

Learning rate and early stopping. The learning rate ν shrinks each step. A smaller step generally requires more rounds, while fitting less noise in each round. The number of rounds M is a hyperparameter: too few underfit; too many fit noise. Early stopping halts when validation error no longer decreases. Validation must be held out by date from within the training rows; this chapter selects M by recording validation error round by round on the inner date-based walk-forward folds in Chapter 38.

Mean decrease in impurity (MDI). For each feature, sum the weighted SSE reductions from all splits that use the feature, then normalize. sklearn's documentation refers to this as "Gini importance" and warns that it is biased toward features with many possible split values. It is computed on training data.

Permutation importance. On a validation set that was not used for fitting, shuffle a feature's values and measure the increase in loss:

$$I_j = \frac{1}{R}\sum_{r=1}^{R}\Big[\mathrm{MSE}\big(y, f(X^{(\pi_r,j)})\big) - \mathrm{MSE}\big(y, f(X)\big)\Big],$$

where $X^{(\pi_r,j)}$ denotes column j permuted according to permutation π_r on repetition r. For the panel in this book, permutation is performed **within each `signal_time`** (cross-sectionally), never across dates, so date-level market conditions are preserved; the decline in daily Spearman IC is also reported. Group permutation applies the same row permutation to all columns in a group, preserving their joint distribution.

SHAP (Shapley additive explanations). Given a prediction $f(x)$, distribute $f(x) - Ef(X)$ across features using Shapley values from cooperative game theory:

$$\phi_j = \sum_{S \subseteq N \setminus \{j\}} \frac{|S|!\,(p - |S| - 1)!}{p!} \big[v(S \cup \{j\}) - v(S)\big],$$

where $v(S)$ is the expected prediction when only the values of features in S are known. Two common definitions of v are the interventional form $v(S) = E\, f(x_S, X_{\bar{S}})$ (the omitted features $X_{\bar{S}}$ are sampled from the background joint marginal distribution, independently of fixed x_S; this does **not** mean that omitted columns are sampled independently of one another) and the conditional form $v(S) = E[f(X) \mid X_S = x_S]$. The definitions give different answers when features are correlated. The book's environment does not have the shap package installed; Section 8 computes exact Shapley values by enumerating a small example.

Ablation. A pre-registered comparison run that removes a feature group or swaps out a component, using the same folds, portfolio construction, and costs, to answer: "How much does this component contribute to out-of-sample results?"

6. Derivations

6.1 Leaf Values and Split Gain

The leaf value is the mean. A constant leaf prediction c minimizes $\sum_{i \in S}(y_i - c)^2$. Differentiating with respect to c gives $-2\sum(y_i - c) = 0 \Rightarrow c = \bar{y}_S$. This uses squared loss; under absolute loss, the leaf value is the median.

Within-group and between-group decomposition. Split parent node S into S_L and S_R, and let $\bar{y}$ be the parent mean. Add and subtract each side's mean before expanding (the cross term vanishes because $\sum_{i \in S_L}(y_i - \bar{y}_L) = 0$):

$$\mathrm{SSE}(S) = \mathrm{SSE}(S_L) + \mathrm{SSE}(S_R) + n_L(\bar{y}_L - \bar{y})^2 + n_R(\bar{y}_R - \bar{y})^2.$$

Simplifying the between-group term. Since $\bar{y} = (n_L\bar{y}_L + n_R\bar{y}_R)/n$, $\bar{y}_L - \bar{y} = \frac{n_R}{n}(\bar{y}_L - \bar{y}_R)$ and $\bar{y}_R - \bar{y} = -\frac{n_L}{n}(\bar{y}_L - \bar{y}_R)$. Substituting gives:

$$\Delta \equiv \mathrm{SSE}(S) - \mathrm{SSE}(S_L) - \mathrm{SSE}(S_R) = \frac{n_L n_R^2 + n_R n_L^2}{n^2}(\bar{y}_L - \bar{y}_R)^2 = \frac{n_L n_R}{n}(\bar{y}_L - \bar{y}_R)^2.$$

Two implications follow. First, gain grows with the squared difference between the two means, but split comparisons must also include the sample-balance factor $n_L n_R/n$; comparing only the mean difference is insufficient. Second, $n_L n_R/n$ is largest when the two sides have equal sample sizes and is about 1 when one side has only one row. Isolating one extreme stock can still yield substantial gain, which is what `min_samples_leaf` is meant to guard against. For the Section 3 example at $s = 4$, $\frac{4 \cdot 4}{8}(-1 - 2)^2 = 18$.

Decision use. The formula shows what a tree looks at: the product of the squared difference in group mean returns and the sample-balance weight. It does not look at slope or rank correlation. A feature useful for ranking but with a small difference in group means may never be selected.

6.2 Averaging Reduces Variance: $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$

Fix a feature point x. Treat each tree prediction $T_b(x)$ as a random variable (randomness comes from bootstrap sampling and feature sampling at splits). Assume the trees are identically distributed, each with variance σ^2, and every pair has correlation $\rho \geq 0$. Then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \frac{1}{B^2}\Big[B\sigma^2 + B(B-1)\rho\sigma^2\Big] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2.$$

The first step expands the variance of the sum into B variances and $B(B-1)$ covariances, each covariance equal to $\rho\sigma^2$. As $B \to \infty$, the second term vanishes and variance approaches $\rho\sigma^2$: averaging many trees removes only the part on which trees differ, not the part they share. Random feature sampling may lower ρ, but it may also change the bias and variance of an individual tree; compare overall error. A smaller `max_features` is not guaranteed to be better. The formula matches Equation (15.1) in the second edition of ESL (see Section 20 for review status).

Financial interpretation: every tree sees the same chance market episode within a training window, so averaging cannot remove it. A longer sample, coarser leaves, or validation on a different window is needed to expose that error.

6.3 Gradient Boosting Fits Residuals Round by Round under Squared Loss

Initialization. $F_0 = \arg\min_c \sum_i L(y_i, c) = \bar{y}$ (the same derivation as in Section 6.1).

Negative gradient. For $L(y, F) = \frac{1}{2}(y - F)^2$, $-\partial L/\partial F = y - F$. At round m, the pseudo-residual is $r_{im} = y_i - F_{m-1}(x_i)$.

Fit and update. Fit a shallow tree h_m to $\{(x_i, r_{im})\}$; its leaf value is the mean residual within the leaf (equivalently, the least-squares solution under squared loss). Then update $F_m = F_{m-1} + \nu h_m$.

Learning-rate shrinkage. Suppose each round selects the same partition. Let the initial mean residual in leaf R be $\bar{r}_R^{(0)}$. One round reduces every row's residual by $\nu\bar{r}_R^{(0)}$, so the leaf's mean residual becomes $(1-\nu)\bar{r}_R^{(0)}$; by induction, after m rounds it is $(1-\nu)^m\bar{r}_R^{(0)}$. With $\nu = 1$, the residual is removed in one step; with $\nu = 0.1$, about $0.9^{10} \approx 35\%$ remains after 10 rounds. This is a special case (the partition changes across rounds in real boosting). It illustrates that a small learning rate breaks one large correction into many small ones, each of which can be rejected by subsequent validation error. That is why "learning rate × number of rounds" must be tuned jointly.

HGB leaf value. sklearn 1.9.1 HGB computes a leaf value using the gradient sum $G = \sum g_i$ and Hessian sum $H = \sum h_i$ as $-G/(H + \lambda)$, then multiplies it by the learning rate (source: `compute_node_value` in `splitting.pyx` and shrinkage in `grower.py`; see D06 and Section 20). Under squared loss $L = \frac{1}{2}(y - F)^2$, $g_i = F - y_i = -r_i$ and $h_i = 1$:

$$\text{leaf value} = \nu \cdot \frac{\sum_{i \in R} r_i}{n_R + \lambda} = \nu \cdot \frac{n_R}{n_R + \lambda} \bar{r}_R.$$

l2_regularization $= \lambda$ multiplies the mean residual in the leaf by $n_R/(n_R + \lambda)$, which has the same form as the Ridge shrinkage factor $n/(n + \alpha)$ under orthogonal design in Chapter 39: smaller leaves are shrunk more.

6.4 Early Stopping Must Hold Out by Date

When the training set has more than 10,000 rows, HGB's `early_stopping="auto"` turns early stopping on and uses `train_test_split` (`test_size=validation_fraction`) to **randomly** sample 10% of training rows as a validation set (local source lines 533–581). This creates two problems in panels: other stocks from the same day remain in the training set, so validation rows share that day's market shock with training rows; and five-day label intervals overlap, with adjacent signal dates sharing four days of returns, so the validation labels have almost been "seen" on the training side. Random row holdout breaks the time-out-of-sample validation design and may understate error. In the constructed panel in Section 12.1, it indeed stops too late, but this is not a theorem that it always selects more rounds or always worsens test results. The correct approach is to disable built-in early stopping, treat the number of rounds as a hyperparameter, record validation MSE at every round (`staged_predict`) on the inner date-based walk-forward folds in Chapter 38, choose the minimum of the mean curve, and refit on all outer-training rows using that number of rounds. Evaluate the outer test window only once. Section 12.1 compares the numbers from random early stopping and date-based early stopping.

6.5 Permutation Importance with Correlated Features

Let $x_2 = x_1$ (the columns are identical), and suppose the fitted model uses only x_1 (this is an assumption about the fitted model; when gains tie, feature search, random state, and implementation conventions can affect the result, so one cannot assume the first column is selected). Permuting x_2 alone leaves the prediction unchanged, so $I_2 = 0$; under the predictor, labels, and MSE specified in Section 8.4, permuting x_1 damages useful prediction, so $I_1 > 0$. Merely reading a column does not guarantee a loss increase on every validation set. The same information receives a high or low rank depending on which column the model happened to use. If a model uses the columns as substitutable predictive proxies (for example, in some forests), the unpermuted column may preserve information and dilute single-column importance. Reading both columns does not itself guarantee this. Joint permutation measures the loss change under the group perturbation. The sign and substitution behavior depend on the model, validation distribution, and loss. For example, take two rows with $x_1 = x_2 = (-1, 1)$, $y = (0, 0)$, and a fixed $f = x_1 - x_2$. Baseline MSE is zero; swapping either column alone gives MSE 4, while swapping both together

leaves MSE zero. This algebraic counterexample does not claim that a forest fitted to these data would learn that function.

The second issue is extrapolation. If x_1 and x_2 are highly, but not perfectly, correlated, permuting x_1 alone can create combinations that do not occur in reality (high x_1 with low x_2). The model's predictions on these points have no support in the training data, so the importance measures its behavior in unfamiliar regions. Strobl et al. (2008) proposed conditional importance to address this issue (see Section 20 for review status).

Third, importance depends on the sample period: the result belongs to the window and fold used to compute it. In this book, importance used for any selection is computed only on in-training validation windows; importance computed on an outer test window is a post hoc diagnostic and must not feed back into that window or any later model, parameter, or feature selection.

6.6 Shapley-Value Additivity and Limitations

Take $p = 2$, an interventional definition, and a single background point b: $v(\varnothing) = f(b)$, $v(\{1\}) = f(x_1, b_2)$, $v(\{2\}) = f(b_1, x_2)$, and $v(\{1,2\}) = f(x)$. Substituting into the definition gives:

$$\phi_1 = \tfrac{1}{2}\big[v(\{1\}) - v(\varnothing)\big] + \tfrac{1}{2}\big[v(\{1,2\}) - v(\{2\})\big], \quad \phi_2 = \tfrac{1}{2}\big[v(\{2\}) - v(\varnothing)\big] + \tfrac{1}{2}\big[v(\{1,2\}) - v(\{1\})\big].$$

Adding gives $\phi_1 + \phi_2 = v(\{1,2\}) - v(\varnothing) = f(x) - f(b)$, the additivity property (local accuracy). Limitations: (1) SHAP explains the **model**, not the world. If the model uses only x_1, the interventional Shapley value for x_2 is 0 even if x_2 contains the same information in the data. (2) Interventional and conditional definitions allocate credit differently for correlated features; neither automatically equals "economic importance." (3) Interaction credit is split according to a rule (see the $f = 2x_1x_2$ example in Section 8.4); the split follows from the axioms, not from discovering a mechanism. (4) Like permutation importance, SHAP depends on the sample period and background distribution and can vary across folds. (5) It does not provide a causal explanation: a "large SHAP value" does not mean that changing the feature would change returns.

7. Financial Interpretation

What trees capture. A tree predicts different mean returns on either side of a split and is well suited to conditional effects: a signal may work only for a particular class of stocks. For example, in the interaction panel in Section 8, q_1 has a loading only when $q_2 > 0$. A linear model can learn only an average slope (about half the true conditional slope), while a tree can first split on q_2 and then on q_1. Whether such a structure exists or is stable in real markets requires empirical evidence; constructed data can demonstrate only the mechanism.

Effects of piecewise-constant predictions on portfolios. All stocks in a leaf receive the same prediction, creating many tied cross-sectional ranks. With shallow trees and large leaves, a dollar-neutral portfolio is effectively long and short across a few "buckets"; Spearman IC is less informative when there are many ties. Forests and boosting add many trees and reduce ties, but their predictions remain step functions.

Extrapolation. A feature value outside the training range falls into an outermost leaf, where the prediction remains constant; it does not extrapolate along a slope as a linear model would. In extreme markets, this can protect against amplified predictions while also hiding effects in a new region.

Turnover and costs. Tree predictions jump at thresholds: a small feature change across a split can move a prediction and portfolio weight by a full step. This can increase turnover; a small accuracy advantage may be erased by costs. Section 13 compares after-cost results using the same Chapter 29 accounting.

Implementability and timing. A strictly monotone transformation preserves training-point order and separable training partitions (ignoring floating-point effects, ties, and binning boundaries), but it does not guarantee identical predictions for new points after a nonlinear transformation. For training data $x = (0, 2)$ and $y = (0, 1)$, the original midpoint threshold is 1. On the nonnegative domain, the strictly increasing transform $g(x) = x^2$ changes the midpoint threshold to 2. At a new point $x = 1.2$, the original tree predicts 1; after transformation, $1.44 \leq 2$, so it predicts 0. A positive affine rescaling (for example, standardizing with a fixed training mean and positive standard deviation) preserves the corresponding midpoint. This does not mean that preprocessing can be done arbitrarily: winsorization by full-sample quantiles, bins fitted on the full sample, and HGB bins fitted on the full sample all let training data see the test-period distribution. Compute a cross-sectional rank only after the complete, contemporaneously available cross-section for that `signal_time` is ready; fit transformations and bins across time using training rows only.

Chapter 40 complete numerical example: all steps and conclusions

8. Numerical Example

All numbers in this section are constructed for teaching. Sections 8.1–8.4 are hand calculations, checked against scikit-learn 1.9.1 by the hand-calculation portion of `code/ch40_tree_models.py`; Section 8.5 comes from the registered main simulations in the same script (seed and settings in its docstring and D08 of `qa/a8/ch40_ch41_batch/DECISION_LOG.md`). Section 8.6 is the D11 fivefold sensitivity added after seeing the main results, not a primary result specified in advance.

Continued: the same complete example

8.1 One Split: Calculate Each Threshold

Use the eight stocks from Section 3 (the unit of y is %, the overall mean is 0.5, and SSE = 22):

Threshold s	n_L	$\bar{y}_L$	$\bar{y}_R$	SSE_L
1.5	1	−2.000	0.857	0.000
2.5	2	−1.500	1.167	0.500
3.5	3	−1.333	1.600	0.667
4.5	4	−1.000	2.000	2.000
5.5	5	−0.400	2.000	9.200
6.5	6	−0.167	2.500	10.833
7.5	7	0.286	2.000	19.429

Threshold s	SSE_R	Gain Δ	Closed-form value
1.5	14.857	7.143	7.143
2.5	10.833	10.667	10.667
3.5	5.200	16.133	16.133
4.5	2.000	**18.000**	18.000
5.5	2.000	10.800	10.800
6.5	0.500	10.667	10.667
7.5	0.000	2.571	2.571

The closed-form column uses $\frac{n_L n_R}{n}(\bar{y}_L - \bar{y}_R)^2$. For example, at $s = 1.5$, $\frac{7}{8} \times 2.857^2 = 7.143$; at $s = 4.5$, $\frac{16}{8} \times 3^2 = 18$. The optimal threshold is 4.5 (sklearn places the threshold midway between adjacent values), with left-leaf value −1% and right-leaf value 2%. `DecisionTreeRegressor(max_depth=1)` returns the same threshold, 4.5, and leaf values $(-1, 2)$. With `min_samples_leaf=3`, only $s \in \{3.5, 4.5, 5.5\}$ are feasible, and 4.5 remains optimal. If y_8 changes from 2 to 9, the optimal split becomes one that isolates stock 8; the leaf-size constraint then takes effect (see 40-E02 in Section 15).

8.2 Random-Forest Variance

Let $\sigma^2 = 1$ and $\rho = 0.3$. For $B = 1, 10, 100, \infty$, the variance of the average is 1, 0.37, 0.307, 0.3. Increasing the forest from 10 to 100 trees reduces variance by only 0.063; the remaining 0.3 is determined by correlation. If random feature sampling lowers correlation to 0.1 at the cost of raising individual-tree variance to 1.2, then the limit as $B \to \infty$ is $0.1 \times 1.2 = 0.12$, much lower than 0.3. This arithmetic illustrates the random forest tradeoff: sacrifice individual-tree performance in exchange for lower correlation across trees.

Continued: the same complete example

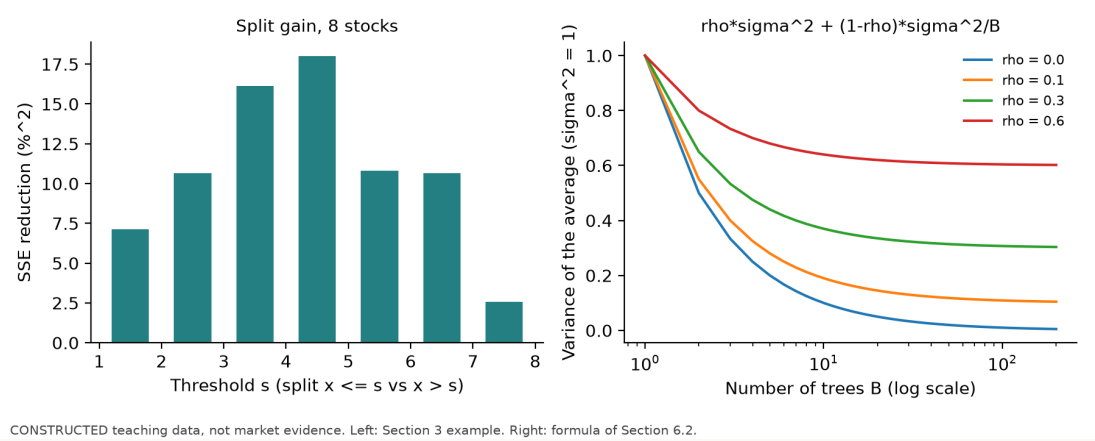


Figure 40.1. Left: SSE reduction at each threshold for the eight-stock example in Section 3, in %², peaking at 4.5. Right: $\rho\sigma^2+(1-\rho)\sigma^2/B$ versus the number of trees B on a logarithmic horizontal axis, with $\sigma^2 = 1$. Constructed values. Takeaway: beyond a few dozen trees, the curve is nearly flat; its limiting level is set by ρ.

8.3 Two Boosting Rounds with Learning Rate 0.5

Round 0: $F_0 = \bar{y} = 0.5$, residuals $r = (-2.5, -1.5, -1.5, -0.5, 1.5, 0.5, 2.5, 1.5)$, and training MSE $= 22/8 = 2.75$.

Round 1: Find the best stump for the residuals. Because the residuals are y minus a constant, the SSE reduction is the same as in Section 8.1; the optimal threshold remains 4.5. The mean residual is -1.5 in the left leaf and 1.5 in the right leaf. Thus $F_1 = 0.5 + 0.5\times(\mp 1.5)$: predictions are -0.25 for the first four stocks and 1.25 for the last four. Training MSE is 1.0625.

Round 2: The new residuals are $(-1.75, -0.75, -0.75, 0.25, 0.75, -0.25, 1.75, 0.75)$. The optimal threshold is now 3.5: the left-leaf mean is $(-1.75 - 0.75 - 0.75)/3 = -13/12$, and the right-leaf mean is $(0.25 + 0.75 - 0.25 + 1.75 + 0.75)/5 = 0.65$. Then $F_2 = F_1 + 0.5\,h_2$: predictions for stocks 1–3 are $-0.25 - 13/24 = -0.7917$, for stock 4 are $-0.25 + 0.325 = 0.075$, and for stocks 5–8 are 1.575. Training MSE is 0.5344.

The setup below produces exactly the same F_2 values on these eight points $(-0.791667, \ldots, 0.075, 1.575, \ldots)$. Training MSE continues to fall in rounds 3 and 4, to 0.3205 and 0.2014. The number of rounds to keep must be decided by validation error outside the training window.

```
HistGradientBoostingRegressor(max_iter=2, learning_rate=0.5, max_leaf_nodes=2,
                              min_samples_leaf=1, early_stopping=False)
```

Leaf-value shrinkage. On the same data, with `max_iter=1`, `learning_rate=1`, `max_depth=1`, and `l2_regularization=4`, the residual sum in the left leaf is -6 for its four samples, giving a leaf value of $-6/(4+4) = -0.75$ and prediction $0.5 - 0.75 = -0.25$; the right-leaf value is $+0.75$, giving prediction 1.25. sklearn returns $(-0.25, 1.25)$. Without λ, the leaf values would be ∓ 1.5; $\lambda = 4$ multiplies them by $4/8$.

Continued: the same complete example

8.4 Small Examples of Permutation Importance and Shapley Values

Twin features. There are four stocks on one day, with $x_1 = (-1.5, -0.5, 0.5, 1.5)$, $x_2 = x_1$, and $y = (-0.9, -0.4, 0.2, 1.1)$ (%). Suppose the fitted model uses only x_1: $f = 0.6x_1$. The baseline MSE is 0.015. Use the fixed permutation "shift forward by two positions," $\pi = (3, 4, 1, 2)$:

- Permute x_1 only: predictions become $0.6 \times (0.5, 1.5, -1.5, -0.5)$, MSE is 1.575, and importance is 1.560.
- Permute x_2 only: predictions do not change, so importance is 0.
- Permute both columns together: importance is 1.560.

x_2 contains exactly the same information as x_1, yet its importance is 0. If the model happened to select x_2, the ranking would reverse.

Shapley values for an interaction. Let $f(x_1, x_2) = 2x_1x_2$, $x = (1, 1)$, and background point $b = (0, 0)$. Then $v(\varnothing) = 0$, $v(\{1\}) = f(1, 0) = 0$, $v(\{2\}) = 0$, and $v(\{1, 2\}) = 2$. Thus $\phi_1 = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 2 = 1$, $\phi_2 = 1$, and their sum is $2 = f(x) - f(b)$. The interaction contribution is split equally. For the twin-feature model $f = 0.6x_1$ at $x = (1.5, 1.5)$, $\phi = (0.9, 0)$.

8.5 Run Results: Interaction and Linear Panels (Main Results)

Setup (registered before the run). There are 40 assets and 1,260 trading days. The conditional mean for the interaction panel is $0.0004 \times g$, where $g = q_1 \mathbf{1}\{q_2 > 0\} + 0.8(|q_3| - \sqrt{2/\pi})$. The linear panel reuses the Chapter 39 generator, with conditional mean $0.0004 \times (q_1 + 0.5q_2 + 0.25q_3)$. Each panel has 20 pure-noise features plus rev_5, mom_60_5, and vol_20, for a total of 26 columns. The setup uses $h = 5$, a one-day lag, an expanding outer window (at least 504 days of training, 63 test days per fold, 12 folds), and the last two 63-day validation windows for inner tuning; the grid matches Project 6 v2. Portfolios are dollar neutral, rebalance every five signal dates, and trade at the next close. Each row's R^2_{OS} is relative to the training mean; IC is the mean daily Spearman IC over the test period.

Panel	Model	R^2_{OS} vs training mean	Mean IC	After-cost Sharpe 10 bp	20 bp
Interaction	OLS	−0.803%	−0.0072	−1.15	−1.53
Interaction	Ridge	−0.075%	−0.0098	−1.25	−1.63
Interaction	Single tree	−0.342%	0.0019	−0.10	−0.56
Interaction	Random forest	−0.116%	−0.0030	−0.93	−1.43
Interaction	Gradient boosting	−0.010%	0.0069	−0.63	−1.17
Linear	OLS	−0.614%	0.0122	−0.06	−0.44
Linear	Ridge	−0.146%	0.0128	0.04	−0.35
Linear	Single tree	−0.343%	0.0057	0.59	0.14
Linear	Random forest	−0.018%	0.0120	0.22	−0.29
Linear	Gradient boosting	−0.040%	−0.0003	−0.50	−1.06

Continued: the same complete example

Panel	Model	Annual turnover
Interaction	OLS	24.5
Interaction	Ridge	24.4
Interaction	Single tree	31.6
Interaction	Random forest	30.8
Interaction	Gradient boosting	36.7
Linear	OLS	24.1
Linear	Ridge	23.8
Linear	Single tree	28.1
Linear	Random forest	30.2
Linear	Gradient boosting	35.8

Zero prediction and the training-mean portfolio are both flat. Every model has negative R^2_{OS} relative to the training mean. In the interaction panel, all absolute IC Newey–West t-statistics are below 1.2; the largest in the linear panel is 1.35. **The tree models did not find the interaction signal.** Inner validation reflects this too: in 11 of 12 folds, gradient boosting selects only 1–14 rounds, and the single tree selects the shallowest depth, 2, in 8 folds. Signal scale matters: in the interaction panel, the standard deviation of g is about 0.86, the daily standard deviation of the conditional mean is about 3.4×10^{-4}, and idiosyncratic volatility is 2% per day. If the signal is approximated as constant over five days and idiosyncratic shocks as independent, ignoring compounding and the heterogeneous-beta common term, the cumulative signal-to-idiosyncratic-noise variance ratio is approximately $5(3.4 \times 10^{-4}/0.02)^2 = 0.001445$, or 0.145% (on the order of 0.1%). This is a conditional scale calculation, not a measured five-day statistic from the panel. This run did not detect the interaction over 12 folds and roughly 750 test days. In the linear panel, the single tree's Sharpe of 0.59 comes from a few favorable splits in 12 folds; its IC is only 0.0057 and should not be treated as evidence.

8.6 Signal-Strength Sensitivity (Added Post Hoc, D11)

After viewing the main results, both panels' loadings were increased fivefold to 0.002 to show what trees can do when a signal is detectable; all other settings were unchanged. This was not a registered main result. Sharpes of 5–10 are far beyond any realistic strategy and indicate only an excessively strong constructed signal.

Continued: the same complete example

Panel (5×)	Model	R^2_{OS} vs training mean	Mean IC (NW t)	After-cost Sharpe 10 bp	Annual turnover
Interaction	OLS	−0.069%	0.057 (5.5)	2.45	23.5
Interaction	Ridge	0.196%	0.057 (5.5)	2.54	23.9
Interaction	Single tree	1.012%	0.090 (10.0)	4.51	33.4
Interaction	Random forest	0.970%	0.108 (11.2)	4.95	27.8
Interaction	Gradient boosting	1.162%	0.119 (13.0)	5.55	34.9
Linear	OLS	4.146%	0.214 (21.6)	10.24	22.6
Linear	Ridge	4.201%	0.215 (21.5)	10.27	22.2
Linear	Single tree	3.101%	0.196 (19.9)	8.79	28.1
Linear	Random forest	3.812%	0.221 (22.3)	10.15	22.6
Linear	Gradient boosting	4.043%	0.214 (22.0)	10.25	25.0

On the interaction panel, tree models achieve about twice the IC of linear models: linear models can use only the average slope on q_1 (theoretically half the conditional slope) and the linear parts of q_2 and q_3 (the linear projection of $|q_3|$ is 0); trees can first split on q_2, then on q_1, and split both tails of q_3. On the linear panel, Ridge, gradient boosting, and the forest are nearly identical, while the single tree is clearly worse: approximating a straight line with steps requires many splits, and the selected depth—usually 2–4 in 11 of the 12 folds—is not enough. On the interaction panel, inner validation selects 20–270 rounds for gradient boosting (mostly 39–112), far more than the 1–14 rounds in the main result. When a signal is present, validation error keeps declining and early stopping selects more rounds.

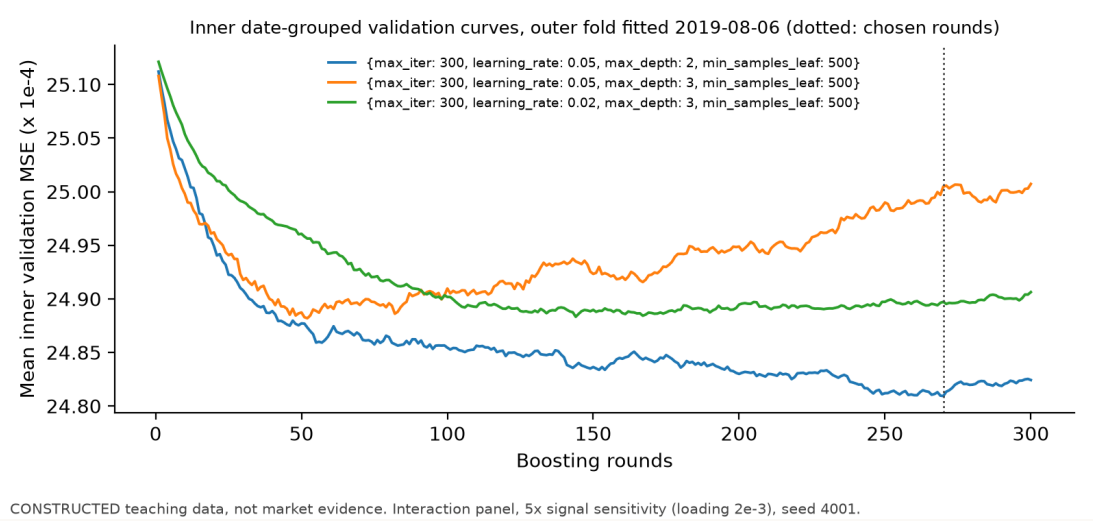


Figure 40.2. Inner date-based validation curve for the last outer fold of the fivefold interaction panel (fit date 2019-08-06). Horizontal axis: boosting rounds. Vertical axis: mean MSE across the two inner validation windows ($\times 10^{-4}$). Three lines show three parameter sets; the dashed line marks the selected 270 rounds. Constructed data. Takeaways: the depth-2, learning-rate-0.05 curve reaches its minimum near round 270; the depth-3, learning-rate-0.05 curve bottoms at round 52 and then rises; the learning-rate-0.02 curve declines more slowly and bottoms at round 144. The minima differ by less than 0.08×10^{-4}; selection uses only validation windows within the training data.

Continued: the same complete example

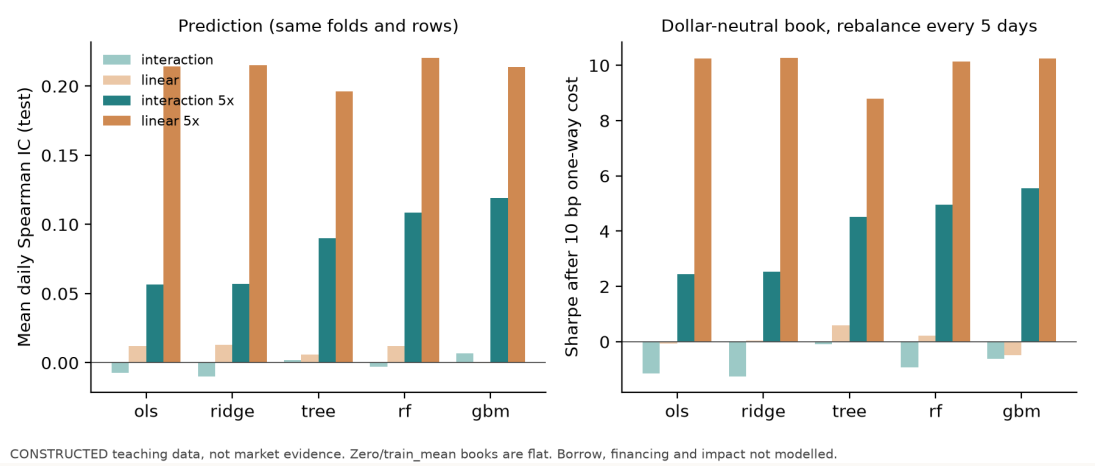


Figure 40.3. Test-period mean IC (left) and Sharpe after a one-sided 10 bp cost (right) for five models on four panels (the two main-result panels in lighter shades and the two fivefold sensitivity panels in darker shades). Same folds, rows, portfolio rule, and accounting. Constructed data. Takeaways: all bars are close to zero in the main results; under the sensitivity, tree models lead on the interaction panel and are close to linear models on the linear panel.

Importance. On the fivefold interaction panel, the random forest computes date-wise permutation importance on the last inner validation window of each fold (three repeats). Mean MSE increases for q_1, q_2, q_3 are $0.318, 0.099, 0.061 \times 10^{-4}$; across 12 folds, q_1 and q_3 enter the top 10 every time and q_2 does so in 11 folds. The pure-noise feature `noise_05` also enters the top 10 in 7 folds, with a mean increase of 0.018×10^{-4}. The mean Spearman correlation between importance rankings in adjacent folds is 0.36 (range 0.05–0.64). In the main result (weak signal), the top features are all noise features (`noise_05`, `noise_04`, noise_18), and adjacent-fold ranking correlation averages only 0.12. In the last fold, MDI ranks q_1, q_2, q_3 first (0.235, 0.150, 0.135); each of the other 23 columns receives about 0.02–0.034, summing to about 0.48. Under permutation importance, noise columns are near zero, with `noise_05` the largest at 0.038×10^{-4}. In the last fold of the main-result panel, the top four MDI features are all noise columns, while q_1 and q_2 each receive only 0.033.

Continued: the same complete example

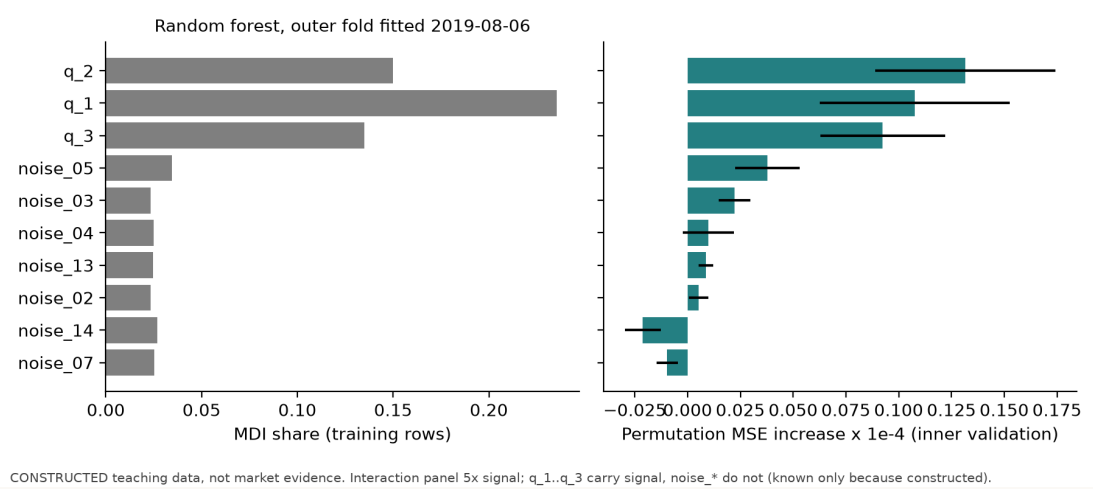


Figure 40.4. Random forest on the last outer fold of the fivefold interaction panel. Left: MDI shares on training rows. Right: date-wise permutation MSE increase on the last inner validation window ($\times 10^{-4}$; error bars are standard deviations over five repeats). Constructed data, where q_1–q_3 have signal and noise_* do not (known only because the data are constructed). Takeaway: noise features account for nearly half of MDI, while permutation importance puts them near zero, although `noise_05` still ranks among the higher features.

9. Python Implementation

- Core module: `src/quant_course/tree_models.py` (contract in Section 1 of `qa/a8/ch40_ch41_batch/INTERFACE_CONTRACT.md`): `make_estimator`, `fit_select_predict_tree`, `walk_forward_predict_tree`, `permutation_importance_by_date`, and `simulate_interaction_panel`. The outer and inner folds, label-availability times, and missing-value rules reuse Chapter 38's `ml_pipeline.date_walk_forward`; evaluation and portfolio accounting reuse `prediction_metrics` and `backtest_predictions` (Chapter 29 `run_backtest`). Tests: `tests/test_tree_models.py`.
- Chapter script. To reproduce, pass only `--out`. The `--figures-dir` option overwrites source images in this chapter's figures/ directory; use it only when intentionally refreshing source images:

```
.venv/bin/python code/ch40_tree_models.py --out <new-directory>
```

On this machine, the four panels, seven models, importance calculations, and demonstration take 4–13 minutes (the random forest uses `n_jobs=-1` and runs more slowly alongside other tasks). Outputs are `results.json` and four figures; the script refuses to use an existing output directory.

- Seed: 4001 for the panels and models; permutation importance also uses 4001. With `early_stopping=False`, HGB is deterministic for a given dataset.

- Timing: daily frequency; `signal_time` is the close of D_k; `trade_time` is the close of D_{k+1}; the return interval is from the close of D_{k+1} to the close of D_{k+6}; `label_available_at` is D_{k+6}.
- Missing values: drop and count training rows with any missing feature or label; do not predict a test row with any missing feature (the stock remains flat for that period); raise an error for infinite values. The first 60 days are excluded from training because mom_60_5 is missing.

Minimal example (one fold, random forest, in-training tuning and importance):

```
from quant_course import ml_pipeline as mp, tree_models as tm
close, chars = tm.simulate_interaction_panel(n_assets=12, n_days=420, n_noise=4,
                                             signal_loading=2e-3, seed=1)
labels = mp.make_forward_labels(close, horizon=5, execution_lag=1)
X = mp.stack_features(chars, labels.index)
folds = mp.date_walk_forward(labels.index.get_level_values(0), labels["label_start"],
                             labels["label_end"], labels["label_available_at"],
                             min_train_dates=250, test_dates=63)
grid = [{"n_estimators": 50, "min_samples_leaf": 50, "max_features": 0.5}]
res = tm.fit_select_predict_tree(X, labels, folds[0], model="rf", seed=1, param_grid=
    ↪ grid,
                                 importance_repeats=2, inner_val_dates=42,
                                 inner_min_train_dates=126)
imp = res.inner_importance["d_mse_mean"].round(8).to_dict()
print(res.chosen_params, res.n_train_used, imp)
```

10. Visualization

All four figures are generated by `code/ch40_tree_models.py` and stored in `course/zh/chapters/ch40/figures/`: Figure 1 (Sections 8.1–8.2); Figure 2 (Section 8.6, inner validation curves and selected number of rounds); Figure 3 (Sections 8.5–8.6, IC and after-cost Sharpe across four panels); Figure 4 (Section 8.6, MDI and permutation importance). Each caption identifies the axes, units, sample, and key observation; each figure is marked as constructed data.

11. Real-Market Example

Status: experiment pending. This course does not have an authorized daily individual-stock panel (`projects/project06/data_manifest.json`; `real_data.status` = blocked), so this chapter reports no real-market results.

Literature comparison (not evidence from this book): Gu, Kelly, and Xiu (2020) use monthly U.S. individual-stock data from 1957–2016 with 920 features (sample description starting on p. 2248), an initial training set of 1957–1974, validation set of 1975–1986, and test set of 1987–2016. They refit annually, expanding the training set by one year and rolling the 12-year validation set forward; they do not use cross-validation, to preserve time order (p. 2249 and footnote 32). Out-of-sample monthly R^2 is 0.33% for the random forest and 0.34% for gradient-boosted trees with Huber loss (Table 1 on p. 2250; described in the text on p. 2251), compared with −3.46% for

full-variable OLS over the same period. The forest averages shallow trees with one to five levels (p. 2251). For the large-cap stock subsample, tree and neural-network R^2 ranges from 0.52% to 0.70% (p. 2252). Their data, frequency, features, and pre-cost measurement are not comparable to the constructed experiments in this book. The paper illustrates the scale of tree-model results in monthly cross-sections, where the scale is tenths of a percent.

What would be needed for follow-up: an authorized daily individual-stock panel (unadjusted prices, volume, splits and dividends, delisted stocks, and dated index constituents), covering at least eight years. Required fields and authorization are specified in `data_manifest.json`. Once the data are available, rerun from the data node under the Project 6 v2 protocol.

12. Common Mistakes

12.1 Using sklearn's Built-In Early Stopping on a Panel

Mistake: Set `early_stopping=True` and `validation_fraction=0.1` in `HistGradientBoostingRegressor` (the default "auto" also turns it on for more than 10,000 rows). **Why:** built-in early stopping randomly samples 10% of the training rows for validation; other stocks from the same day and overlapping labels remain on the training side (Section 6.4). Difference on the main interaction-panel result in Section 8.5, across 12 outer folds, with learning rate 0.05, depth 3, minimum leaf size 500, at most 1,000 rounds, and stopping after 20 rounds without improvement:

	Mean rounds	Mean validation MSE ($\times 10^{-4}$)	Mean test MSE ($\times 10^{-4}$)
Built-in random-holdout early stopping	1,000 (all 12 folds ran to the limit)	22.06 (random holdout rows)	26.01
Inner date-based validation for round selection (book method)	1–104, median 8	—	24.72
Zero prediction	—	—	24.76

The random-holdout validation error kept falling, so early stopping never triggered. Test error was higher in all 12 folds than under date-based selection and also higher than for zero prediction. The random-holdout MSE of 22.06 is about 15% below test error, a direct measure of its optimism. **Fix:** set `early_stopping=False` and select the number of rounds with `staged_predict` on inner date-based folds (the tree_models factory rejects early-stopping-related parameters for gbm).

```
import numpy as np
from sklearn.ensemble import HistGradientBoostingRegressor
X = np.random.default_rng(0).normal(size=(12_000, 5))
y = X[:, 0] * 0.1 + np.random.default_rng(1).normal(size=12_000)
auto = HistGradientBoostingRegressor(max_iter=50, random_state=0).fit(X, y)  # "auto"
print(auto.do_early_stopping_, auto.validation_fraction)  # True 0.1: random 10%
    ↪ holdout
```

Output: True 0.1.

12.2 Selecting the Number of Rounds on the Test Window

Mistake: Inspect MSE round by round on the outer test window and report the best round. On the same panel with fixed parameters and 300 rounds per fold, selecting the best round on the test window gives R^2_{OS} (relative to zero) of +0.26%; the selected rounds are 10, 23, 58, 5, 1, 1, 1, 1, 82, 24, 2, and 1, with no stable pattern. Fixing the model at 300 rounds gives −1.61%. The book's method (selecting parameters and rounds by inner date-based validation; gradient boosting in Section 8.5) gives +0.16%. The +0.26% cannot be obtained live because test-window labels are unknown when orders are placed; it is the best value selected using test data, not achievable out-of-sample performance. **Fix:** like all other hyperparameters, select rounds only within the training data and evaluate on the outer test window once.

12.3 Assuming Trees Need No Preprocessing, So Full-Sample Preprocessing Is Fine

Mistake: Winsorize by full-sample quantiles, or fit bins or standardization on the full sample, before passing data to a tree. **Why:** a strictly monotone transformation preserves training order but does not guarantee new-point predictions under the nonlinear midpoint rule (counterexample in Section 7); winsorization and binning are not one-to-one strictly monotone transformations either. If thresholds or bin edges are set by the full-sample distribution, test-period extremes can change training-period values and therefore the splits. **Fix:** fit transformations across time only on training rows; compute a cross-sectional rank for a `signal_time` only once all required cross-sectional information is available. HGB's binning happens inside fit using only the rows passed to it, which satisfies this requirement.

12.4 Selecting Features with MDI or Test-Period Importance

Mistake: Keep the top k features from the final model's `feature_importances_` or from permutation importance on the outer test window, then report out-of-sample results. **Why:** MDI is computed on training rows and favors continuous features with many possible split points (in Section 8.6, noise columns receive about half the total MDI share); test-period importance uses test labels. **Fix:** select features only using permutation importance on in-training validation windows (`select_top_k`; the Project 6 v2 top-10 ablation). Test-period importance is only a post hoc diagnostic.

12.5 Reading an Importance Ranking as an Economic Mechanism

Mistake: Report that "`noise_05` is the fourth most important feature." **Why:** when the signal is weak, permutation importance can assign positive values to noise columns that happen to correlate with returns in a validation window; correlated features can substitute for one another (the twin-feature example in Section 8.4); rankings vary across folds (adjacent-fold correlation is 0.12 in the main result). **Fix:** report stability across folds (whether each feature enters the top 10) and group importance, treating importance as a diagnostic rather than a discovery.

13. Quantitative Research Application

Question. On the constructed panel frozen for Project 6, do tree models improve out-of-sample prediction and after-cost portfolios relative to zero prediction, the training mean, and regularized linear models? Are results stable, and which feature groups contribute?

Protocol (`projects/project06/configs/p06_trees_v2.json`, frozen before the first v2 run, SHA-256 60101509…). It inherits the v1 data generator, two registered scenarios (baseline and loading sign reversal at 75% of the sample), features (3 price features + 3 loaded features + 40 zero-loading features), $h = 5$, a one-day lag, an expanding outer window (at least 504 days, 63 test days per fold, 16 folds), the last two 63-day inner validation windows, dollar-neutral portfolios, and the Chapter 29 accounting at one-sided costs of 0/10/20 bp. Additions: a single tree (depth 2/3/4/6, at least 1,000 rows per leaf); random forest (200 trees, `max_features=0.33`, leaf sizes 500/2,000, `max_samples=0.5`); gradient boosting (HGB, up to 300 rounds, learning rates 0.05/0.02, depths 2/3, minimum leaf size 500, with round selection on inner date-based folds); and PCA+Ridge (an auxiliary ablation for Chapter 41). Importance: date-wise permutation on the last inner validation window of each fold (3 repeats), plus joint permutation of price and feature groups; outer-test permutation importance and random-forest MDI are post hoc diagnostics only. Ablations: remove price features, use price features only, and use the top 10 features selected within training. Stability: selected parameters and IC by fold, adjacent-fold importance-rank correlation, and three seeds. Comparison statistics: daily IC differences relative to Ridge and Newey–West t-statistics (lag 4). No multiple-comparison correction is applied; all results are descriptive. **Results already viewed before the protocol was frozen:** constructed results from A8-A v1 had been reviewed and were itemized in the configuration, so this is not a real-market preregistration made before seeing results. The six v1 baseline models were rerun with unchanged v1 code and match the A8-A c3 summary value by value (209 values in each scenario, maximum relative difference 0).

Results: baseline scenario (constructed data; R^2_{OS} relative to training mean; IC is the mean daily Spearman IC on each model's **own dates with defined IC**, with Newey–West t-statistics in parentheses (lag 4 trading days, reported only for continuous date series); ΔIC is the mean daily difference from Ridge on **dates where both models have IC**; Sharpe is shown at one-sided costs of 0/10/20 bp; turnover is annualized; maximum drawdown at 10 bp: OLS −14.2%, Ridge −12.9%, Lasso −9.3%, Elastic Net −7.2%, single tree −20.0%, random forest −12.3%, gradient boosting −11.0%, PCA+Ridge −12.0%):

Model	R^2_{OS}	IC (t)	ΔIC (t)	Sharpe 0 bp
OLS	−0.839%	0.0203 (2.24)	+0.0004 (0.18)	0.79
Ridge	−0.113%	0.0198 (2.22)	—	0.84
Lasso	−0.083%	0.0179 (—)[a]	+0.0017 (—)[a]	0.36
Elastic Net	−0.010%	0.0309 (—)[a]	+0.0103 (—)[a]	1.19
Single tree	−0.393%	0.0054 (0.64)	−0.0144 (−1.61)	0.21
Random forest	+0.037%	0.0266 (2.86)	+0.0067 (1.06)	1.13
Gradient boosting	+0.023%	0.0154 (1.79)	−0.0044 (−0.61)	0.76
PCA+Ridge	−0.008%	0.0135 (1.45)	−0.0063 (−0.86)	0.62

Model	10 bp	20 bp	Turnover
OLS	0.39	0.00	24.3
Ridge	0.45	0.07	23.6
Lasso	0.01	−0.33	16.3
Elastic Net	0.81	0.43	22.0
Single tree	−0.28	−0.75	35.1
Random forest	0.64	0.16	29.2
Gradient boosting	0.22	−0.32	36.2
PCA+Ridge	0.25	−0.13	23.3

Results: loading-reversal scenario (loading reverses from trading day 1,134; folds 11–16 occur after the reversal):

Model	R^2_{OS}	IC (t)	ΔIC (t)	Sharpe 0 bp
Ridge	−0.161%	0.0087 (0.95)	—	0.23
Elastic Net	−0.061%	0.0152 (—)[a]	+0.0061 (—)[a]	0.45
Single tree	−0.604%	−0.0120 (−1.32)	−0.0207 (−2.19)	−0.66
Random forest	−0.042%	0.0129 (1.35)	+0.0042 (0.66)	0.41
Gradient boosting	−0.020%	0.0041 (0.46)	−0.0045 (−0.59)	0.16
PCA+Ridge	−0.053%	0.0063 (0.68)	−0.0023 (−0.31)	0.24

Model	10 bp	20 bp	Turnover
Ridge	−0.15	−0.53	23.7
Elastic Net	0.08	−0.29	20.2
Single tree	−1.13	−1.60	36.2
Random forest	−0.08	−0.57	29.7
Gradient boosting	−0.38	−0.92	36.5
PCA+Ridge	−0.14	−0.52	23.5

(See the research report for OLS and Lasso; Lasso's Sharpe after 10 bp is −0.57.)

[a] **Date counts.** There are 1,002 test dates with mature labels. OLS, Ridge, PCA+Ridge, and all three tree models have defined IC on all 1,002 consecutive dates. Sparse models have constant predictions on some dates (all coefficients shrink to zero), so cross-sectional correlation is undefined: baseline Lasso has IC on only 630 dates and Elastic Net on 882; the corresponding counts after sign reversal are 567 and 819. Their mean IC and Ridge's mean over 1,002 dates are therefore not measured on the same dates, and ΔIC is not the difference between the two columns' means. In the baseline scenario, Lasso's own IC is 0.0179 over 630 dates, while Ridge's is 0.0161 on those same dates, giving a paired difference of +0.0017; Elastic Net's own IC is 0.0309 over 882 dates, while Ridge's is 0.0206 on those dates, giving +0.0103. After sign reversal, Lasso has 0.0010 versus Ridge's 0.0031 on 567 dates, a paired difference of −0.0021; Elastic Net has 0.0152 versus 0.0091 on 819 dates, a paired difference of +0.0061. In the frozen summary, the t-statistics for these two sparse series were computed after dropping missing dates and treating the remaining valid dates as consecutive observations in NW(4). Consecutive valid observations may be several trading days apart, so those are not HAC statistics with a four-trading-day bandwidth; no such statistics are reported here. Date counts were independently recomputed from the frozen

predictions and label CSVs; see `projects/project06/research_report/v2_ic_date_bases.json`.

Ablations (Sharpe after 10 bp; ΔIC relative to the full-feature version of the same model):

Ablation	Baseline IC	Baseline ΔIC (t)	Baseline Sharpe	Reversal IC
Forest: remove prices	0.0207	−0.0059 (−2.05)	0.27	0.0082
Forest: prices only	0.0131	−0.0135 (−1.09)	−0.05	0.0141
Forest: in-training top 10	0.0146	−0.0120 (−1.85)	0.28	0.0082
Boosting: remove prices	0.0080	−0.0074 (−1.56)	−0.22	−0.0013
Boosting: prices only	0.0145	−0.0009 (−0.08)	0.44	0.0130
Boosting: in-training top 10	0.0197	+0.0043 (0.58)	0.14	0.0141

Ablation	Reversal ΔIC (t)	Reversal Sharpe
Forest: remove prices	−0.0047 (−1.62)	−0.34
Forest: prices only	+0.0013 (0.10)	−0.14
Forest: in-training top 10	−0.0047 (−0.77)	−0.06
Boosting: remove prices	−0.0055 (−1.14)	−0.42
Boosting: prices only	+0.0089 (0.77)	0.19
Boosting: in-training top 10	+0.0099 (1.22)	−0.22

Stability. Across three random-forest seeds, IC is 0.0266, 0.0232, and 0.0237; Sharpe after 10 bp is 0.64, 0.52, and 0.44; pairwise prediction correlations range from 0.962 to 0.982. HGB with `early_stopping=False` is identical across all three seeds for the given data. This is implementation determinism, not data or time stability. Gradient boosting selects 2–96 rounds by fold (mostly 4–56), and 14 of 16 folds select depth 2 for the single tree. Mean adjacent-fold correlation of in-training permutation-importance rankings is −0.10 for the single tree, 0.01 for the forest, and 0.09 for boosting; q_1 enters the top 10 in 11 of 16 folds for both the forest and boosting, while zero-loading `noise_04` also enters the forest's top 10 in 11 folds. Group permutation: mean MSE increase is about zero for the price group (within $\pm 0.002 \times 10^{-4}$ for all three tree models) and about 0.05×10^{-4} for the feature group.

Interpretation.

1. **Prediction:** No model has R^2_{OS} relative to the training mean above 0.04%. For the tree models, the IC differences relative to Ridge on continuous dates (four-trading-day NW) are all within $|t| < 2.2$. The only $|t| > 2$ occurs for the single tree under loading reversal, where it is worse ($t = -2.19$; unadjusted two-sided approximation $p \approx 0.029$; about 0.17 after Bonferroni correction for the 3 tree models × 2 scenarios, with ablations and seed comparisons also registered). This is descriptive, not a confirmatory significant finding. The data-generating process is linear and gives the tree models no nonlinear structure to exploit, consistent with Sections 8.5–8.6.
2. **Costs:** Tree-model annual turnover is 29–36, higher than the linear models' 16–24 (piecewise-constant predictions jump by a full step when crossing a threshold). The random forest's pre-cost Sharpe in the baseline scenario is 1.13 but falls to 0.16 after 20 bp. Elastic Net ranks first or near first across both scenarios and all three cost levels.

3. **Ablation and substitutes:** Removing price features lowers IC for both tree models; with price features only, boosting nearly retains its full-feature IC, while the price group's joint-permutation importance is near zero. This indicates overlapping information between price features and feature columns: past returns include slowly varying expected returns (feature AR coefficient 0.98). When feature columns are available, they can substitute for the price group, so permuting that group causes little loss; removing it can still remove an independent source of noise and some signal. In the reversal scenario, boosting with price features only does better; momentum-like features update with recent returns faster than coefficients learned on features before the reversal. These are observations from two scenarios only, with all t-statistics below 1.3.
4. **PCA+Ridge:** The selected number of components is usually 30 (10 of 16 folds in the baseline scenario), with 5 selected in five folds and 15 in one fold. Its R^2 relative to the training mean is better than Ridge in both scenarios, and its IC is lower than Ridge in both. Its 10/20 bp Sharpe is worse in the baseline scenario, while in the reversal scenario, $-0.14/-0.52$ is slightly higher than Ridge's $-0.15/-0.53$. The 43 constructed q and noise series are mutually independent, unit marginal variance, and each follows an AR(1) with coefficient 0.98. The other three price features come from the return process and are indirectly linked to the signal, so all 46 features do not form an independent, equal-variance population. Validate finite-sample PCA directions separately from predictive power (Chapter 41, Section 6.4).
5. **Importance:** q_1 and q_2 are the only loaded features that consistently appear near the top (q_3 has one-quarter the loading of q_1), but several zero-loading features also frequently enter the top 10. Without ground truth, the table cannot distinguish them.

Teaching decision criterion (not a joint threshold in the frozen JSON). If a conservative teaching criterion requires the paired IC difference to have NW $t \geq 2$ in both scenarios and Sharpe after 20 bp to exceed both Ridge and Elastic Net, no tree model qualifies. The frozen `claims_rule` requires that constructed data not be used to claim economic value or alpha and that zero and negative results be retained; it does not register the joint threshold above. Constructed data cannot support an alpha claim, and real-data empirical work remains blocked. Full tables, fold-level records, importance CSVs, and figures are in `projects/project06/research_report/v2_trees.md`.

14. Chapter Summary

Trees divide feature space into rectangles and predict the mean within each region. Their split criterion balances the difference in group mean returns against sample balance ($\frac{n_L n_R}{n}\Delta\bar{y}^2$); depth and leaf-size constraints limit isolating a few extreme stocks in their own leaves. Random forests reduce variance by averaging, removing only the part on which trees differ and leaving $\rho\sigma^2$. Reducing `max_features` may lower ρ, but it may also weaken an individual tree and change bias or variance; validate the tradeoff within training data. Under squared loss, gradient boosting fits residuals round by round and shrinks each step with the learning rate; select the number of rounds jointly with the learning rate using in-training validation. HGB's L2 regularization has the same shrinkage form as Chapter 39 Ridge. Hyperparameters for all three model classes must be selected only on date-grouped validation windows within training data; sklearn's random-holdout

early stopping was optimistic in this panel. Importance is a diagnostic for a model in a particular window: MDI favors continuous features with many split points, permutation importance can be diluted by correlated features or inflated by chance correlations with noise, and Shapley values explain the model rather than an economic mechanism. The constructed experiments show that trees do not discover an interaction by themselves when the signal is weak; when it is detectable, trees outperform linear models on the interaction structure and perform about as well as Ridge on the linear structure. Decide whether to use trees by comparing them with zero prediction, the training mean, and regularized linear models under the same folds, accounting, and costs.

15. Exercises

40-E01 (Hand-calculating a split). There are six stocks, $x = (1, \ldots, 6)$, and $y = (1, 1, 2, 4, 5, 5)$ (%). Calculate the gain at all five thresholds, identify the best split and the two leaf values, and verify the best gain using $\frac{n_L n_R}{n}(\bar{y}_L - \bar{y}_R)^2$.

40-E02 (Leaf-size constraint). Use the Section 3 data with `min_samples_leaf=3`. Which thresholds are feasible, and what is the best split? If y_8 changes from 2 to 9, what are the best splits with and without the constraint?

40-E03 (Random-forest variance). One tree has $\sigma^2 = 4$ and $\rho = 0.25$. Find the variance for $B = 1, 4, \infty$. For a second setting, $\sigma^2 = 5$ and $\rho = 0.1$; what is the variance for $B = 4$ and $B = \infty$? Which setting is better for large B, and why?

40-E04 (One boosting round). Use the Section 3 data and learning rate 0.25 for one round of stump boosting. Write down F_0, the residuals, the selected threshold, the leaf values, F_1, and the training MSE.

40-E05 (L2 leaf value). Under squared loss, an HGB leaf has four samples with residual sum 6, `l2_regularization=4`, and learning rate 1. What is the leaf value? What is it at learning rate 0.1? Explain its relationship to the Ridge shrinkage factor using $n_R/(n_R + \lambda)$.

40-E06 (Permutation and correlation). In the twin-feature example in Section 8.4, why is the importance of x_2 zero? If the model is changed to $f = 0.3x_1 + 0.3x_2$, how do the three importance values compare when permuting x_1 alone, x_2 alone, or both together? Exact values are not required.

40-E07 (Shapley values). Let $f(x_1, x_2) = x_1 + 2x_1x_2$, $x = (1, 1)$, and background point (0, 0). Find ϕ_1, ϕ_2, verify additivity, and explain how the interaction term $2x_1x_2$ is allocated.

40-E08 (Early-stopping leakage). In one paragraph, explain why randomly holding out 10% of rows for early stopping on a panel produces lower validation error than test error and selects too many rounds in this panel; explain why random-row holdout is not a date-out-of-sample validation. Which two numbers in the Section 12.1 table show this directly?

40-E09 (Interpreting importance). In the main result in Section 8.6, the random forest's top three permutation-importance features are all noise. Give two reasons and name two stability measures you would include in a report.

16. Programming Exercises

40-P01 (Split search). Implement best_split(x, y, min_leaf), which returns the best threshold (the midpoint between adjacent values), gain, and two leaf values for a one-dimensional feature. Acceptance criteria: (1) the Section 3 data return $(4.5, 18.0, -1.0, 2.0)$; (2) on a random, nondegenerate example with a feasible positive-gain split and no tie for the optimum, results match `DecisionTreeRegressor(max_depth=1, min_samples_leaf=`min_leaf) in threshold and leaf values; (3) gain equals the closed form $\frac{n_L n_R}{n}\Delta\bar{y}^2$; (4) if all y values are equal and feasible candidates exist, returning a zero-gain split is allowed, while sklearn may keep the root as a leaf; if x is constant or the leaf constraint leaves no feasible split, return None.

40-P02 (Date-grouped round selection). For one outer fold, select the HGB round count on inner date-based folds using `staged_predict` (the function defaults to `inner_folds=1`, the final fold; use this default for this exercise. The project's two-fold setting is different). The result must match `chosen_params["max_iter"]` from `fit_select_predict_tree` with model="gbm". Acceptance criteria: (1) the two values match; (2) changing the outer test labels does not change the selected rounds or test predictions; (3) passing `early_stopping=True` to `make_estimator` raises an error.

40-P03 (Date-wise permutation). Implement permutation within each `signal_time`, shuffling a column independently by date. Acceptance criteria: (1) the multiset of that column's values is unchanged within each date; (2) values are never exchanged across dates; (3) a fixed model that does not read a noise column has exactly zero importance, while a model fitted on a finite sample with a truly zero-loading noise feature is not guaranteed to have exactly zero importance in that estimate, so report the repeated-estimate distribution; the signal feature should have a clearly positive importance; (4) in group permutation, all columns in a group use the same row permutation.

17. Challenge Problems

40-C01 (Interpret the Project 6 v2 results). Read the tables in Section 13 and `projects/project06/research_report/v2_trees.md`. (a) What is the direction of the tree models' IC differences (Newey–West t-statistics) and after-cost Sharpe differences relative to Ridge in the two registered scenarios? Are these consistent with Sections 8.5–8.6? (b) What does each of the three registered ablations (remove price features, price features only, in-training top 10) show? (c) How stable are predictions and after-cost Sharpe across seeds? (d) Which results would you cite against the claim that "gradient boosting discovered alpha"? Additional prerequisites: Chapter 37 benchmarks and Chapter 32 performance measurement.

40-C02 (Real-data protocol). Write a one-page research protocol for tree models on a real daily individual-stock panel: features and their availability times, labels, outer and inner folds, grid and round selection, importance window, ablations, cost scenarios, seeds, benchmarks, and falsification conditions. Explain how you would freeze the protocol before seeing real results, and which `A8-A/A8-B` constructed results you have already seen and must disclose.

18. Complete Solutions

40-E01. The overall mean is 3 and SSE $= 4+4+1+1+4+4 = 18$. Gains are 4.8 at threshold 1.5, 12.0 at 2.5, **16.667** at 3.5, 12.0 at 4.5, and 4.8 at 5.5. The best threshold is 3.5: the left leaf is $4/3 = 1.333$ and the right leaf is $14/3 = 4.667$; the closed form is $\frac{3\cdot 3}{6}(10/3)^2 = 16.667$. Gains are symmetric around the middle because the data are antisymmetric around their midpoint.

40-E02. Feasible thresholds are 3.5, 4.5, and 5.5 (at least three stocks on each side); the best is 4.5 (gain 18). When $y_8 = 9$, the overall mean is 1.375. Without the constraint, the best threshold is 7.5, isolating stock 8: $\frac{7\cdot 1}{8}(0.286-9)^2 = 66.45$. With the constraint, compare only 3.5, 4.5, and 5.5. The gains are 45.1 at 4.5, $\frac{15}{8}(-1.333-3.0)^2 = 35.2$ at 3.5, and $\frac{15}{8}(-0.4-4.333)^2 = 42.0$ at 5.5, so 4.5 remains best. The constraint prevents the tree from creating a leaf for just one stock.

40-E03. For $\sigma^2 = 4, \rho = 0.25$, the variances at $B = 1, 4, \infty$ are 4, 1.75, and 1. For $\sigma^2 = 5, \rho = 0.1$, they are $0.5 + 0.9 \times 5/4 = 1.625$ at $B = 4$ and 0.5 at $B = \infty$. The second setting is better for large B ($0.5 < 1$), because only $\rho\sigma^2$ remains in the limit: reducing correlation matters more than reducing individual-tree variance. At $B = 4$, they are close (1.625 versus 1.75).

40-E04. $F_0 = 0.5$; residuals are $(-2.5, -1.5, -1.5, -0.5, 1.5, 0.5, 2.5, 1.5)$; the best threshold is 4.5 and the leaf values are -1.5 and $+1.5$. Thus $F_1 = 0.5 \pm 0.25 \times 1.5$: predictions are 0.125 for the first four stocks and 0.875 for the last four. Training MSE is $[(-2.125)^2 + (-1.125)^2 \cdot 2 + (-0.125)^2 + (1.125)^2 + (0.125)^2 + (2.125)^2 + (1.125)^2]/8 = 1.7656$. At learning rate 0.5, the one-round MSE is 1.0625; with a smaller learning rate, each round reduces error less.

40-E05. The leaf value is $\nu \cdot \frac{\sum r}{n_R+\lambda} = 1 \times 6/8 = 0.75$; at learning rate 0.1 it is 0.075. Without L2, the mean residual is 1.5. With $\lambda = 4$, it is multiplied by $n_R/(n_R+\lambda) = 4/8$, the same form as Ridge shrinkage under orthogonal design: smaller leaves receive more shrinkage.

40-E06. The model $f = 0.6x_1$ does not use x_2, so permuting x_2 leaves all predictions unchanged and its importance is zero. This indicates which column the model used, not whether x_2 contains information. If $f = 0.3x_1 + 0.3x_2$, permuting either feature alone leaves the other half of the signal intact, so both importances are equal and clearly smaller than the importance of permuting them together. With one feature permuted, the prediction becomes $0.3x_{1,\pi} + 0.3x_1$; the main error comes from shuffling half the signal. Group permutation measures the information in the pair.

40-E07. $v(\varnothing) = 0$, $v(\{1\}) = f(1,0) = 1$, $v(\{2\}) = f(0,1) = 0$, and $v(\{1,2\}) = 3$. Then $\phi_1 = \frac{1}{2}(1-0) + \frac{1}{2}(3-0) = 2$ and $\phi_2 = \frac{1}{2}(0-0) + \frac{1}{2}(3-1) = 1$. Their sum is $3 = f(x) - f(b)$. The main effect of x_1, equal to 1, is assigned entirely to feature 1; the interaction contribution of 2 is split equally.

40-E08. Randomly held-out validation rows and training rows come from the same dates: they share same-day market shocks, persistent characteristics across the same stocks, and overlapping five-day labels (adjacent signal dates share four days of returns). Thus the labels of validation rows have almost been "seen" on the training side. As the model fits the training data more closely, validation error falls and early stopping is delayed, selecting too many rounds. In the table, the random-holdout validation MSE of 22.06 is about 15% below its test MSE of 26.01, and all 12 folds run to the limit of 1,000 rounds; date-based round selection has test MSE 24.72.

40-E09. Reasons: (1) the signal is weak, so the model primarily fits chance relationships between noise features and returns in the training window, and some of those relationships sometimes persist into the adjacent validation window; (2) there are only three permutation repeats, and importance is calculated on one 63-day validation window. Stability measures include the number of folds in which each feature enters the top 10 and the Spearman correlation between importance rankings in adjacent folds (0.12 on average in the main result). Group importance and ablations using price features only or characteristic features only are also useful.

40-P01 reference implementation.

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor


def best_split(x, y, min_leaf=1):
    o = np.argsort(x, kind="stable")
    x, y = np.asarray(x, float)[o], np.asarray(y, float)[o]
    n, best = len(y), None
    sse = lambda v: float(((v - v.mean()) ** 2).sum())
    for s in range(min_leaf, n - min_leaf + 1):
        if s == 0 or s == n or x[s - 1] == x[s]:
            continue
        L, R = y[:s], y[s:]
        gain = sse(y) - sse(L) - sse(R)
        assert np.isclose(gain, s * (n - s) / n * (L.mean() - R.mean()) ** 2)
        if best is None or gain > best[1] + 1e-12:
            best = ((x[s - 1] + x[s]) / 2, gain, L.mean(), R.mean())
    return best


x8, y8 = np.arange(1, 9.0), np.array([-2, -1, -1, 0, 2, 1, 3, 2.0])
assert np.allclose(best_split(x8, y8), (4.5, 18.0, -1.0, 2.0))
rng = np.random.default_rng(0)
for leaf in (1, 5):
    x, y = rng.normal(size=60), rng.normal(size=60)
    t = DecisionTreeRegressor(max_depth=1, min_samples_leaf=leaf).fit(x[:, None], y)
    thr, gain, l, r = best_split(x, y, leaf)
    assert np.isclose(thr, t.tree_.threshold[0])
    assert np.allclose([l, r], t.tree_.value[1:, 0, 0])
assert best_split(x8, np.ones(8))[1] == 0.0
assert best_split(np.ones(8), y8) is None
assert best_split(x8, y8, min_leaf=5) is None
print("ok")
```

40-P02 reference implementation.

```
import numpy as np, pytest
from quant_course import ml_pipeline as mp, tree_models as tm
close, chars = tm.simulate_interaction_panel(n_assets=10, n_days=400, n_noise=3,
                                             signal_loading=2e-3, seed=2)
labels = mp.make_forward_labels(close, horizon=5, execution_lag=1)
X = mp.stack_features(chars, labels.index)
f = mp.date_walk_forward(labels.index.get_level_values(0), labels["label_start"],
                         labels["label_end"], labels["label_available_at"],
                         min_train_dates=250, test_dates=40)[0]
grid = [{"max_iter": 40, "learning_rate": 0.1, "max_depth": 2, "min_samples_leaf":
    ↪ 50}]
kw = dict(model="gbm", param_grid=grid, inner_folds=1,
          inner_val_dates=40, inner_min_train_dates=120)
res = tm.fit_select_predict_tree(X, labels, f, **kw)
xtr, ytr = X.iloc[f.train_rows], labels["y"].iloc[f.train_rows]
ok = xtr.notna().all(axis=1) & ytr.notna()
lab = labels.iloc[f.train_rows][ok]
inner = mp.date_walk_forward(lab.index.get_level_values(0), lab["label_start"],
                             lab["label_end"], lab["label_available_at"],
                             min_train_dates=120, test_dates=40)[-1]
xi, yi = xtr[ok], ytr[ok]
est = tm.make_estimator("gbm", grid[0], seed=0)
est.fit(xi.iloc[inner.train_rows], yi.iloc[inner.train_rows])
yv = yi.iloc[inner.test_rows].to_numpy()
curve = [np.mean((yv - p) ** 2) for p in est.staged_predict(xi.iloc[inner.test_rows])
    ↪ ]
assert int(np.argmin(curve)) + 1 == res.chosen_params["max_iter"]
lab2 = labels.copy()
lab2.iloc[f.test_rows, lab2.columns.get_loc("y")] = 9.9
res2 = tm.fit_select_predict_tree(X, lab2, f, **kw)
assert res2.chosen_params == res.chosen_params
assert res2.predictions.equals(res.predictions)
with pytest.raises(ValueError):
    tm.make_estimator("gbm", {"early_stopping": True}, seed=0)
print(res.chosen_params["max_iter"])
```

40-P03 reference implementation.

```
import numpy as np, pandas as pd
from quant_course import tree_models as tm
rng = np.random.default_rng(5)
dates = pd.bdate_range("2020-01-01", periods=60)
idx = pd.MultiIndex.from_product([dates, [f"A{i}" for i in range(15)]])
X = pd.DataFrame({"sig": rng.normal(size=len(idx)), "noise": rng.normal(size=len(idx)
    ↪ ),
                  "twin": 0.0}, index=idx)
X["twin"] = X["sig"]
y = pd.Series(0.5 * X["sig"].to_numpy() + rng.normal(scale=0.5, size=len(idx)), index
    ↪ =idx)
predict = lambda d: 0.5 * d["sig"].to_numpy()  # a model that reads 'sig' only
imp = tm.permutation_importance_by_date(predict, X, y, n_repeats=5, seed=0)
assert imp.loc["sig", "d_mse_mean"] > 0.3
assert abs(imp.loc["noise", "d_mse_mean"]) < 1e-12 and imp.loc["twin", "d_mse_mean"]
    ↪ == 0
```

```
g = tm.permutation_importance_by_date(predict, X, y, groups={"pair": ["sig", "twin"
    ↪ ]},
                                      n_repeats=5, seed=0)
assert g.loc["pair", "d_mse_mean"] > 0.3
print(imp["d_mse_mean"].round(3).to_dict())
```

The multiset, no-cross-date, and joint group-permutation properties should also be checked directly, rather than relying only on the importance output:

```
Xp = X.copy()
for date in X.index.get_level_values(0).unique():
    loc = np.flatnonzero(X.index.get_level_values(0) == date)
    order = rng.permutation(len(loc))
    # Read and write within the same date; paired columns use the same order.
    Xp.iloc[loc, X.columns.get_indexer(["sig", "twin"])] = (
        X.iloc[loc[order]][["sig", "twin"]].to_numpy())
    assert sorted(Xp.iloc[loc]["sig"]) == sorted(X.iloc[loc]["sig"])
    assert sorted(map(tuple, Xp.iloc[loc][["sig", "twin"]].to_numpy())) == (
        sorted(map(tuple, X.iloc[loc][["sig", "twin"]].to_numpy())))
assert np.array_equal(Xp["sig"], Xp["twin"])
```

Here, a fixed model that does not use noise gives exactly zero importance; a model fitted on data may happen to use noise, and its finite-sample permutation importance can be positive or negative. With multiple inner folds, average the validation MSE across the same active folds at each round and select the minimum; do not use only the final fold.

40-C01 complete reference answer. Sharpe differences below subtract the two-decimal values shown in the tables, rather than rounding differences calculated from the raw summaries. In the baseline scenario, the paired ΔIC values for the single tree, forest, and boosting are -0.0144, $+0.0067$, and -0.0044, with descriptive NW t-statistics of -1.61, 1.06, and -0.61. Their Sharpe differences from Ridge at 10 bp are -0.73, $+0.19$, and -0.23. Under loading reversal, ΔIC is -0.0207, $+0.0042$, and -0.0045, with t-statistics -2.19, 0.66, and -0.59; 10 bp Sharpe differences from Ridge are -0.98, $+0.07$, and -0.23. The 20 bp differences are -0.82, $+0.09$, and -0.39 in the baseline scenario and -1.07, -0.04, and -0.39 after reversal. Only the unadjusted absolute t for the single tree under reversal exceeds 2, in the worse direction; all others are below 2. This does not support adopting tree models consistently. The weak-signal panel in Section 8 also retains its failure, while the fivefold interaction panel shows that a strong constructed interaction can be detected; it cannot be transferred to the project's linear data-generating process.

For the forest, removing price features lowers IC in both scenarios; Sharpe after 10 bp falls from 0.64 to 0.27 in the baseline scenario and from -0.08 to -0.34 after reversal. Forest results using price features only are $-0.05/-0.14$. For boosting, removing prices gives Sharpe of $-0.22/-0.42$, while using prices only gives $0.44/0.19$, above the full-feature results of $0.22/-0.38$. Using the top 10 features gives forest Sharpe of $0.28/-0.06$ and boosting Sharpe of $0.14/-0.22$, so improvement is inconsistent. Price features are derived from a return path that includes slowly varying expected returns and overlap in information with the q features; near-zero group permutation importance does not imply that removing the group or using it alone has no value. Importance rankings in the 63-day inner windows are noisy: adjacent-fold forest ranking correla-

tion is only 0.01, and `noise_04` and q_1 each enter the top 10 in 11 of 16 folds, clear evidence of instability.

Across three forest seeds, IC values are 0.0266, 0.0232, and 0.0237, and Sharpe after 10 bp is 0.64, 0.52, and 0.44; pairwise prediction correlations range from 0.962 to 0.982. High prediction correlation can coexist with a 0.20 Sharpe difference. HGB with built-in early stopping disabled is identical across three seeds for the given data; this is implementation determinism, not data or time stability. All results are constructed, multiple comparisons are unadjusted, and most after-cost values are negative under loading reversal. Importance cannot provide causal interpretation. Together, these facts contradict the claim that the model "discovered market alpha."

40-C02 complete protocol example. This is a research design that could be frozen separately once authorized data become available; it does not claim that authorization has been granted or that the study has been run. Use a licensed point-in-time daily individual-stock panel. Preserve raw unadjusted closes, volume, splits and dividends, delisted stocks, and dated index membership. Record available_at for each field and use the signal cutoff time τ_k: every input must satisfy available_at $\leq \tau_k$. If using same-day closing features, form the signal no earlier than the time those closing data actually become available, and wait until the required full cross-section is complete. Use adjusted prices to construct returns; use transaction prices and corporate actions in the Chapter 29 accounting; do not backfill historical membership with later constituents.

Form the signal at the close of D_k, trade at the close of D_{k+1}, and let the five-day label mature at the close of D_{k+6}. Use an expanding outer window with at least 504 training days and 63 test days per fold; exclude labels that have not matured by fitting time. Use the last two 63-day inner validation windows with at least 252 training days, preserving date order. Compare zero prediction, the training mean, Ridge, Elastic Net, and all three tree models on identical available rows, portfolio construction, and accounting. Search single-tree depths 2/3/4/6 with at least 1,000 samples per leaf; a 200-tree forest with `max_features=0.33` and leaf sizes 500/2,000; and HGB with at most 300 rounds, learning rates 0.02/0.05, depths 2/3, minimum leaf size 500, and round selection by average inner MSE with random early stopping disabled. Lock all grids and seeds 6061/6062/6063 before seeing results.

Compute importance by date on inner validation windows, using joint permutation of price and feature groups; test-window importance is diagnostic only. Register three ablations: remove price features, use prices only, and use the top 10 features selected within training. Report cases with no improvement. Center each cross-section's portfolio weights and set total absolute weight to 1; rebalance every five days. Show costs at 0/10/20 bp, and state that borrow fees, financing, market impact, and capacity are not modeled. Report paired IC differences on common dates, missing-date counts, R^2 relative to zero and the training mean, after-cost annualized return, Sharpe, drawdown, and turnover, plus fold and seed ranges. Do not describe NW(4) after dropping sparse dates as a four-trading-day HAC.

Pre-specify an adoption rule requiring paired advantages under both scenarios and better performance than linear benchmarks after 20 bp, with multiple-comparison handling determined in advance. Do not adopt if the advantage occurs only in one window or seed, or only before costs. A t-threshold alone is not sufficient economic evidence. Freeze the configuration, data license and SHA, timestamp, and inventory of results already viewed before running. Disclose all constructed

results already seen from Chapters 37–41 and Project 6, especially the post hoc fivefold panel and 50-seed diagnostics. Any later grid change requires a new version and retention of the earlier negative results. If access to real inputs has not been authorized, this protocol remains blocked.

19. Further Reading

- **Must know:** Breiman (2001), "Random Forests," Sections 1–2 and 11: the forest strength–correlation decomposition. Hastie, Tibshirani, and Friedman, The Elements of Statistical Learning, 2nd edition, Sections 9.2 (trees), 10.10–10.12 (gradient boosting and shrinkage), and 15.2 (Equation 15.1, forest variance).
- **Should know:** Friedman (2001), Section 3 (the pseudo-residual algorithm) and Section 5 (shrinkage). scikit-learn 1.9.1 user guide, "Permutation feature importance" (its relationship to MDI and the warning about correlated features). Gu, Kelly, and Xiu (2020), Sections 1.1 and 2.2: tree-model setup and scale in monthly cross-sections.
- **Advanced:** Strobl et al. (2007, 2008): bias in variable importance and conditional permutation importance. Lundberg and Lee (2017), and Lundberg et al. (2020): SHAP and TreeSHAP. Aas, Jullum, and Løland (2021), and Janzing et al. (2020): tradeoffs between conditional and interventional definitions with correlated features.
- **Optional:** sklearn `HistGradientBoostingRegressor` source: lines 533–581 of `gradient_boosting.py` and `compute_node_value` in `splitting.pyx`.

20. References

"Read by the author" below refers to the original course author's historical review recorded in source_notes. BIL19 inherits that status; this revision did not reopen the original publications or perform a new online source review.

Full links and reviewed locations are listed in `references.md`, `references/sources.json`, and this chapter's `source_notes.md`.

- SK_PITFALLS: scikit-learn, "Common pitfalls." Used in Sections 7 and 12.3 for preprocessing within training data. Review: inherited from the A8-A registration.
- SK_TREE_SRC: local sklearn 1.9.1 source: `tree/_classes.py` (warning in the `feature_importances_` docstring); `ensemble/_forest.py` (`RandomForestRegressor` default `max_features=1.0`); `ensemble/_hist_gradient_boosting/gradient_boosting.py`, lines 533–581 (`early_stopping="auto"` and `train_test_split`); `splitting.pyx`, `compute_node_value` ($-G/(H+\lambda)$); and `grower.py` (shrinkage = `learning_rate`). Used in Sections 5, 6.3–6.4, and 12.1. Review: **read by the author** (local source).
- CH37_GKX2020: Gu, S., Kelly, B., & Xiu, D. (2020). "Empirical asset pricing via machine learning." RFS, 33(5), 2223–2274. Used in Sections 2 and 11. Review: **read by the author**: pages 2248–2249 on sample and splits, footnote 32, page 2251 on RF 0.33% / GBRT+H 0.34% and shallow trees, and page 2252 on 0.52%–0.70% for large-cap stocks.
- CH40_BREIMAN2001: Breiman, L. (2001). "Random forests." Machine Learning, 45, 5–32. Used in Sections 5, 6.2, and 19. Review: reconnaissance report (R).

- ESL_2ND_ED: Hastie, Tibshirani, and Friedman (2009). The Elements of Statistical Learning, 2nd ed. Used for Equation (15.1) in Section 6.2 and Section 6.3. Review: reconnaissance report (R: Section 15.2, p. 588, Equation 15.1); not reread by the author.
- CH40_FRIEDMAN2001: Friedman, J. H. (2001). "Greedy function approximation: a gradient boosting machine." Annals of Statistics, 29(5), 1189–1232. Used in Section 6.3. Review: reconnaissance report (R).
- CH40_STROBL2007, CH40_STROBL2008: Strobl et al. (2007), BMC Bioinformatics, 8:25; and (2008), 9:307. Used in Section 6.5. Review: reconnaissance report (R); full texts not read by the author.
- CH40_LL2017, CH40_LUNDBERG2020: Lundberg and Lee (2017), NeurIPS; Lundberg et al. (2020), Nature Machine Intelligence, 2, 56–67. Used in Section 5 and Section 6.6. Review: reconnaissance report (R).
- CH40_AJL2021: Aas, Jullum, and Løland (2021). Artificial Intelligence, 298, 103502. Used in Section 6.6. Review: reconnaissance report (R).
- SK_PERMIMP_DOC: scikit-learn user guide, "Permutation feature importance." Used in Sections 5 and 6.5. Review: reconnaissance report (R); author checked the sklearn 1.9.1 warning in the MDI docstring.

For specific passages and source assignments, see `course/zh/chapters/ch40/source_notes.md`.

Notebook Reader's Guide

The I3 Chapter 40 notebook has 11 cells, including 5 code cells. It freshly computes the hand-calculation checks and the seven-model fivefold interaction panel, and writes two figures to a new RUN_DIR; it does not read an old cache. It does not run the full CLI's four panels or all importance calculations; the fivefold run remains a D11 post hoc diagnostic. Notebook language handling is being done in BIL23; this batch did not run or modify the notebook.

Chapter 41: Unsupervised Learning

Author: Yuxiao Ma. The existing A8-B constructed implementation has limited-scope acceptance pending formal integration; this does not mean that real research is complete. Main thread: PCA and clustering do not use labels; they compress variance and distance structure. High explained variance does not imply predictive power, and a cluster-derived state is not an economic mechanism. These methods can enter a tradable study only when scaling, principal components, and cluster centers are fitted within the training window, and later dates are projected or assigned afterward. Full-sample PCA, hindsight state labels, and state probabilities smoothed with future data all leak information. Prerequisites: Chapter 5 linear algebra (eigendecomposition, orthogonal projection), Chapter 15 factor models (covariance, factor exposures), Chapter 16 Time Series I (rolling windows and autocorrelation), and Chapter 38 pipelines (training-only fitting and date-based walk-forward). By the end, you should be able to calculate the principal component and explained-variance ratio of a 2×2 covariance matrix by hand; explain how scaling changes principal components; calculate out-of-sample reconstruction R^2 for training-fitted PCA and explain why it is lower than the in-sample value in this example without treating that ordering as a per-window theorem; use k-means to assign rolling market-state features online and test cluster stability; calculate one hidden Markov filter update by hand and distinguish filtered from smoothed probabilities; and use constructed data to show how much hindsight state labels can improve the apparent performance of a strategy.

1. Learning Objectives

1. State the PCA objective (the maximum-variance direction is an eigenvector of the covariance matrix), calculate eigenvalues, loadings, and explained-variance ratios for a 2×2 example, and explain sign indeterminacy and this book's sign convention.
2. Explain the difference between covariance PCA and correlation-matrix PCA (standardize first), and why standardization parameters must be estimated only from training rows.
3. Fit PCA within a training window, project later dates, and calculate out-of-sample reconstruction R^2; use the noise eigenvalue bound for a random matrix to explain why in-sample explained variance is inflated.
4. Use a constructed example to show that high explained variance by itself does not guarantee predictive power for future returns, and test this in Project 6 v2's PCA+Ridge ablation.
5. State the k-means objective and Lloyd iterations; explain how distance depends on scaling, why cluster labels are arbitrary, and why the number of clusters must be selected; calculate the adjusted Rand index (ARI) under row bootstrap resampling and explain that it measures sensitivity to row perturbations only, while temporal stability requires block resampling or rolling refits.
6. Assign market states online using features that depend only on past data; write the filtering recursion for a two-state hidden Markov model and calculate one step by hand; explain that smoothed probabilities use future observations.

7. Compare five market-timing rules after costs in a constructed two-state market: hindsight true state, smoothed probability, centered rolling volatility (which uses future data), filtered probability (traded with a one-day lag), and online training-fitted k-means assignment.

Completion criteria: Sections 15–17 can be completed independently; `code/ch41_unsupervised.py` reproduces the numbers in Sections 8–10.

2. Why Quant Needs It

Two common tasks do not rely on labels. The first is risk compression: estimating a return covariance matrix for hundreds of stocks requires estimating tens of thousands of parameters, while most common return variation can be explained by a few directions (market, industry, and style). Statistical factor models use PCA to find these directions for risk estimation, residual-return construction, or hedging. The second is regime detection: a researcher wants to know whether the market is currently calm or volatile in order to adjust positions, leverage, or models.

Both tasks invite time leakage. When PCA estimates a covariance matrix using the full sample, principal components for early dates already contain later volatility structure. When a state model smooths over the full sample, a day may be labeled "volatile" partly because of declines in the following days. Both practices make research results look better, but neither is available in live trading. This chapter pairs each tool with a version fitted within the training window and used out of sample, then uses constructed data to quantify the gap.

3. Intuition

PCA. For returns recorded in percentage points, the covariance matrix (in squared percentage points) for two stocks is $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: each has variance 2 and their covariance is 1. Variance along $(1,1)/\sqrt{2}$ (moving together) is 3; variance along $(1,-1)/\sqrt{2}$ (moving in opposite directions) is 1. The first principal component explains $3/4 = 75\%$ of total variance. Intuitively, PCA finds the direction along which the data are most spread out; for stock returns, that is usually the direction in which assets move together.

Clustering. Plot each day's "past-month annualized volatility" against its "past-three-month cumulative return." Calm up-market days cluster in a low-volatility, positive-return corner, while crisis days cluster in a high-volatility, negative-return corner. k-means assigns each point to its nearest center. Intuitively, clustering finds days that are close to one another.

Boundary. Neither example involves future returns. The first principal component explains the most variance, but that says nothing about its ability to predict anything next period. "Cluster 1" is simply a set of days close in distance; calling it a "crisis regime" is a name supplied by the researcher. Section 8 demonstrates both points numerically.

4. Mathematical Foundations

Symbol	Meaning	Dimension / units
$X \in \mathbb{R}^{T \times N}$	Training-window data matrix: T dates and N columns (asset returns or features)	Returns in decimals or percent; features in their original units
$\mu,\ s$	Column means and sample standard deviations in the training window (`ddof=1`)	Same units as the columns
$\tilde{X} = (X - \mathbf{1}\mu^\top)D_s^{-1}$	Training-window data after centering (and optionally standardizing), with $D_s = \mathrm{diag}(s)$ or I	—
$\Sigma = \tilde{X}^\top \tilde{X}/(T-1)$	Sample covariance matrix (the correlation matrix after standardization)	Units squared
$\lambda_1 \geq \cdots \geq \lambda_N,\ v_1, \ldots, v_N$	Eigenvalues and unit eigenvectors (loadings) of Σ	Units squared; dimensionless
$V_k = [v_1, \ldots, v_k]$	First k loadings	$N \times k$
$z_t = V_k^\top \tilde{x}_t$	Principal-component scores on date t	Same units as $\tilde{x}$
$K,\ c_1, \ldots, c_K$	Number of clusters and cluster centers	—
$s_t \in \{0, \ldots, K-1\}$	Latent state on date t	—
$P_{ij} = \Pr(s_t = j \mid s_{t-1} = i)$	Transition matrix	Rows sum to 1
$\phi(x; \mu_j, \sigma_j)$	Normal density for an observation in state j	1 / unit

Timing convention. State features and principal components are calculated at close t using data through t. The trading rule forms a signal at close t and executes at close $t+1$ (a one-day lag, consistent with the Chapter 29 ledger); the return interval runs from close $t+1$ to close $t+2$. Prerequisites: Chapter 5 eigen-decomposition and the spectral theorem (a symmetric matrix has orthogonal eigenvectors), Chapter 15 factor-model covariance and factor exposures, and Chapter 16 Time Series I (rolling windows and autocorrelation) for state features.

5. Formal Definitions

Principal component analysis (PCA). The first loading is $v_1 = \arg\max_{\|v\|=1} v^\top \Sigma v$; component j maximizes the same objective subject to being orthogonal to the previous $j-1$ components. The solution is an eigenvector of Σ, and $v_j^\top \Sigma v_j = \lambda_j$. PCA answers "how can a few orthogonal directions summarize the common variation in many columns?" Its input is training-window data; its outputs are loadings, eigenvalues, and scores.

Explained-variance ratio. $\lambda_j / \sum_i \lambda_i$. It measures the fraction of training-window total variance along that direction and does not use labels.

Out-of-sample reconstruction R^2. Use the training-window μ, s, V_k to transform later observations $\tilde{x}_t$:

$$R^2_{\text{rec}} = 1 - \frac{\sum_t \|\tilde{x}_t - V_k V_k^\top \tilde{x}_t\|^2}{\sum_t \|\tilde{x}_t\|^2}.$$

On the training window, this equals the sum of the explained-variance ratios for the first k components (when the training matrix is nonzero, the factors $1/(T-1)$ cancel between numerator and denominator). On later dates, it measures how much of the new data the training-derived directions still explain. With the training center and positive scales fixed, $V_k^\top V_k = I$, and a positive denominator, orthogonal decomposition gives $0 \le R^2_{\text{rec}} \le 1$; this is not predictive R^2, which can be negative. If the denominator is zero, the mathematical expression is undefined as $0/0$; this book's API returns 1 by convention. Standardization requires positive training standard deviations for every column and enough observations to define them. Explained-variance ratios are mathematically undefined when total training variance is zero. Without standardization, a constant column is allowed when total variance remains positive; for example, diag(1,0) has explained shares 1 and 0. Out of sample, reconstruction can be either below or above the training share; there is no per-window inequality.

Sign indeterminacy. Both v and $-v$ are eigenvectors. This book's convention is to choose the sign so the loading elements sum to a positive value; if the sum is zero, the first nonzero element must be positive. Different software or windows may produce opposite signs, so align signs before comparing scores. This convention resolves sign only; equal eigenvalues still allow arbitrary rotations within the corresponding subspace. In that case, compare subspaces.

Scaling. In covariance PCA, columns with larger variance dominate. In correlation-matrix PCA, each column is first divided by its training standard deviation. The choice depends on the use case: covariance PCA is common for risk compression because higher-volatility assets contribute more risk; feature reduction commonly standardizes first because variances in different units are not directly comparable.

k-means. Given K, minimize within-cluster squared distance, $\sum_t \|x_t - c_{k(t)}\|^2$ (called inertia by scikit-learn). Lloyd's algorithm alternates between assigning each point to its nearest center and updating each center to the mean of its assigned points. The objective never increases. With a deterministic tie rule that prevents cycling, consistent empty-cluster handling, and strict descent whenever the partition changes, the ideal Lloyd iteration terminates at a fixed point. In software, however, a tolerance or maximum-iteration limit may stop the algorithm before an exact fixed point is reached. Results depend on initialization; scikit-learn's `n_init` runs multiple initializations and keeps the best. Distance depends on each column's units, so standardize using training rows first.

Arbitrary cluster labels (label switching). The labels 0 and 1 have no intrinsic meaning and may be swapped after refitting. In this book, after fitting, clusters are relabeled in ascending order of their centers on the training-window "volatility" column. Use the Hungarian algorithm to match state labels when comparing them element by element; ARI itself is invariant to label permutations and requires no prior matching.

Cluster stability. Perturb the data (for example, with bootstrap resampling or a different window), refit clustering, and compare assignments. The adjusted Rand index (ARI) corrects agreement between two partitions for chance: perfect agreement is 1, and random partitions have an expected value around 0. The perturbation determines what the result means. An IID row bootstrap of dates measures only sensitivity to replacing some rows. The state features use overlapping 21-day and 63-day windows, so neighboring dates are highly correlated; row resampling does not establish temporal stability. Use block bootstrap or rolling refits for that.

Regime detection. Assign dates to a small number of "states" for conditional decisions. This chapter distinguishes three types of labels: **online labels** (using data through t only at time t), **filtered probabilities** $\Pr(s_t = j \mid x_{1:t})$, and **smoothed probabilities** $\Pr(s_t = j \mid x_{1:T})$. The latter use observations after t and are hindsight labels.

Hidden Markov model (HMM). The state s_t is a Markov chain with transition matrix P. Conditional on the entire latent-state path, observations are independent across dates, with $x_t \mid s_t \sim N(\mu_{s_t}, \sigma^2_{s_t})$. Conditioning on the state at just one date does not imply independence across the full observation sequence. Hamilton (1989) is a classic Markov-switching model for macroeconomic and financial time series (see the reading notes in Section 20).

Factor interpretation. Naming a principal component "market," "size," or "value" is an interpretation made by comparing its loadings with known characteristics. Loadings can rotate across windows, so the name is not automatic.

6. Derivations

6.1 The Maximum-Variance Direction Is an Eigenvector

Maximize $v^\top \Sigma v$ subject to $v^\top v = 1$. The Lagrangian is $v^\top \Sigma v - \lambda(v^\top v - 1)$. Differentiating with respect to v gives $2\Sigma v - 2\lambda v = 0 \Rightarrow \Sigma v = \lambda v$. The objective value is $v^\top \Sigma v = \lambda v^\top v = \lambda$, so the maximizing eigenvector is the one with the largest eigenvalue. Since Σ is symmetric, the spectral theorem gives an orthogonal eigenbasis; subsequent principal components take the next eigenvalues under orthogonality constraints. Conditions: Σ is estimated from training data; when eigenvalues are equal, the direction is not unique (as in Section 3 if covariance is zero and variances are equal, when every direction is equivalent).

2 × 2 example. $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ has characteristic equation $(2-\lambda)^2 - 1 = 0$, so $\lambda = 3, 1$. For $\lambda = 3$, $(2-3)a + b = 0 \Rightarrow a = b$, giving the unit vector $(1,1)/\sqrt{2}$. The explained share is $3/(3+1) = 75\%$.

6.2 Scaling Changes the Principal Components

Suppose the two columns have variances 4 and 1 and correlation 0.5, so their covariance matrix is $\begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix}$. The characteristic equation is $\lambda^2 - 5\lambda + 3 = 0$, giving $\lambda = \frac{5 \pm \sqrt{13}}{2} = 4.303$, 0.697. The first loading is approximately $(0.957, 0.290)$ and explains 86.1%; it is driven mainly by the higher-variance column. After standardization, the correlation matrix is $\begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, with eigenvalues 1.5 and 0.5, loading $(1,1)/\sqrt{2}$, and explained share 75%. With the same data, changing the scaling changes the direction of the "first principal component." If s is estimated from the full sample, the training-window components contain volatility information from the test period.

6.3 In-Sample Explained Variance Is Inflated: A Noise Eigenvalue Bound

Suppose there are N mutually independent columns, each with variance σ^2, and no common factors, observed over T rows. All population covariance eigenvalues equal σ^2, but sample covariance eigenvalues spread out. Under independent, identically distributed entries and suitable finite-moment conditions, and the limit $N, T \to \infty$ with $q = N/T \in (0, 1)$ fixed, the sample eigenvalue distribution converges to the Marchenko–Pastur distribution, whose support is

$$\lambda_\pm = \sigma^2\big(1 \pm \sqrt{q}\big)^2.$$

Laloux et al. (1999) applied this to financial correlation matrices (see the reading notes in Section 20). With $N = 50, T = 60$, $\lambda_+ \approx 3.66\sigma^2$: the first principal component of pure noise appears to explain $3.66/50 \approx 7\%$ of the in-sample variance, while its true share is $1/50 = 2\%$. This result relies on independence and a large-sample limit. Financial returns have heavy tails and volatility clustering, so it is an asymptotic reference line, not a hard finite-sample upper bound or a ready-made test. For $q > 1$, the point mass at zero eigenvalues must also be included; this example is not in that case. Practical use: compare in-sample explained variance with this line; more directly, calculate out-of-sample reconstruction R^2 (Section 8.2).

6.4 High Explained Variance Does Not Guarantee Predictive Power

Suppose two features are independent, $\operatorname{Var} x_1 = 10$, $\operatorname{Var} x_2 = 0.1$, and the label is $y = \beta x_2 + \varepsilon$, with zero-mean error ε that is uncorrelated with x_1, x_2 and has finite variance. The first covariance-PCA component is the x_1 direction and explains $10/10.1 \approx 99\%$ of feature variance. With only this component ($k = 1$), the population-optimal linear principal-component regression (PCR) has $R^2 = 0$ (assuming positive label variance), because y depends only on x_2. PCA ranks directions by $\operatorname{Var}(v^\top x)$ and does not use $\operatorname{Cov}(v^\top x, y)$. The first principal component of stock returns is often the market, which explains most common variation, but dollar neutrality $\sum w_i = 0$ is not market-beta neutrality $\sum w_i \beta_i = 0$. For example, weights $(0.5, -0.5)$ and betas $(2, 1)$ have zero net dollars but market exposure 0.5. Only weights orthogonal to the market loading

hedge that direction. Information useful for cross-sectional prediction may lie in a low-variance direction.

6.5 Monotonic Descent of k-means

With centers fixed, assigning every point to its nearest center cannot increase the objective, because each point takes its minimum distance. With assignments fixed, replacing each center with its cluster mean minimizes within-cluster squared distance (as in Section 6.1 of Chapter 40: the mean is the optimal constant under squared loss). Alternating the two steps makes the objective non-increasing and bounded below by 0. A lower bound guarantees convergence of the objective value, but alone does not imply termination in finitely many steps. With finitely many partitions, a deterministic tie rule that prevents cycling, and fixed empty-cluster handling, if every partition change strictly decreases the objective, the algorithm terminates in finitely many steps. Implementations may also stop at a tolerance or iteration cap. The result is generally a local fixed point, not a global optimum; different initializations can reach different partitions, one reason to test cluster stability separately.

6.6 Filtering Recursion and Smoothing

Let $\alpha_t(j) = \Pr(s_t = j \mid x_{1:t})$.

1. **Prediction:** $\Pr(s_t = j \mid x_{1:t-1}) = \sum_i \alpha_{t-1}(i) P_{ij}$, by the law of total probability and the Markov property.
2. **Update:** $\alpha_t(j) = \dfrac{\phi(x_t; \mu_j, \sigma_j) \Pr(s_t = j \mid x_{1:t-1})}{\sum_k \phi(x_t; \mu_k, \sigma_k) \Pr(s_t = k \mid x_{1:t-1})}$, by Bayes' rule and conditional independence of observations given the states.

With parameters fixed from the training window, α_t depends only on $x_1, \ldots, x_t$ and can be calculated at close t. If parameters are fitted on the full sample, they themselves contain future information. Smoothed probabilities $\Pr(s_t = j \mid x_{1:T})$ also use the backward recursion $\beta_t(i) \propto \sum_j P_{ij} \phi(x_{t+1}; \mu_j, \sigma_j) \beta_{t+1}(j)$ and are normalized after multiplying by $\alpha_t(i)$; they therefore contain $x_{t+1}, \ldots, x_T$. A research report may use smoothed probabilities to mark historical "crisis periods"; using them as signals that could have been traded at the time leaks future data.

The hand calculation in Section 8.3 uses these parameters: the stationary distribution under $P = \begin{pmatrix} 0.98 & 0.02 \\ 0.05 & 0.95 \end{pmatrix}$ is $(5/7, 2/7) \approx (0.714, 0.286)$, with mean durations of $1/(1 - 0.98) = 50$ and $1/(1 - 0.95) = 20$ days.

7. Financial Interpretation

Statistical factors and economic factors. PCA returns directions of maximum training-window variance. Economic factors such as market, size, and value come from theory or portfolio sorts. They may be similar (the first principal component often correlates highly with an equal-weighted market) or different. Statistical factors avoid having to specify factors in advance, but loadings rotate across windows, signs are arbitrary, and names require later interpretation. For a risk model, also test whether the factors explain as much variance out of sample.

Uses of risk compression. A low-rank covariance matrix from the first k principal components plus a diagonal residual (the statistical version of the Chapter 15 factor model) can reduce estimation error. Residual returns (after removing the first few components) can be used to construct statistical-arbitrage signals. Both uses require refitting on the then-current training window at every rebalance date.

States and mechanisms. Clustering or an HMM may group "high-volatility, negative-return" days together; that is descriptive. Calling the group a "crisis regime" and reducing exposure assumes that subsequent returns will also be worse. This prediction must be tested out of sample. State persistence and detection delay (a filtered probability may take several down days to switch) also reduce returns.

Tradability. Every state rule must answer: what do you know at close t? Filtered probabilities, features from past windows, and cluster centers fitted in the training window are available. Smoothed probabilities, centered rolling windows, and states labeled after a full-sample fit are not. Even a correctly timed rule pays for position changes; Section 8.4 compares the rules using the same simplified cost proxy.

Chapter 41 complete numerical example: all steps and conclusions

8. Numerical Example

All numbers in this section are constructed for teaching: Sections 8.1 and 8.3 are hand calculations checked against code; Sections 8.2 and 8.4 come from simulations in `code/ch41_unsupervised.py` (the seeds are recorded in the script's docstring), not market data.

8.1 Hand-Calculating Principal Components and Scaling

Covariance matrix $\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$**.** From Section 6.1, $\lambda = (3, 1)$. The first loading is $(0.7071, 0.7071)$, whose elements sum to a positive value. The second is $\pm(0.7071, -0.7071)$; its elements sum to zero, so the convention chooses a positive first element: $(0.7071, -0.7071)$. The explained ratios are $(0.75, 0.25)$. In the code, `loadings[:, j]` is the jth loading (a column), and row i gives the loading of variable i on each principal component.

Covariance matrix $\begin{pmatrix} 4 & 1 \\ 1 & 1 \end{pmatrix}$**.** $\lambda = (4.3028, 0.6972)$, the first loading is $(0.9571, 0.2898)$, and it explains 86.06%. **Correlation matrix** $\begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$**.** $\lambda = (1.5, 0.5)$, the first loading is $(0.7071, 0.7071)$, and it explains 75%. The script reproduces all three results with constructed

Continued: the same complete example

data whose sample covariance equals the specified matrix, using `unsupervised_tools.pca_fit` (`results.json` under `hand.pca`).

```
import numpy as np
for S in ([[2, 1], [1, 2]], [[4, 1], [1, 1]], [[1, 0.5], [0.5, 1]]):
    w, v = np.linalg.eigh(np.array(S, float))
    w, v = w[::-1], v[:, ::-1]
    v = v * np.where(v.sum(axis=0) < 0, -1, 1)  # sign convention: loadings sum
    ↪ > 0
    print(np.round(w, 4), np.round(v[:, 0], 4), np.round(w / w.sum(), 4))
```

Output:

```
[3. 1.] [0.7071 0.7071] [0.75 0.25]
[4.3028 0.6972] [0.9571 0.2898] [0.8606 0.1394]
[1.5 0.5] [0.7071 0.7071] [0.75 0.25]
```

8.2 In-Sample and Out-of-Sample Results: Factor and Pure-Noise Panels

Two constructed return panels have 50 assets and 1,000 days. **Factor panel:** three factors with daily volatilities of 1.0%, 0.5%, and 0.3%; idiosyncratic volatility of 1.5%; and loadings drawn from $N(0,1)$ (add 1 to the first loading column so average exposure to the market factor is positive). **Noise panel:** 50 independent columns drawn from $N(0, 1.5\%^2)$, with no common factors. Every 21 days, standardize the past T days, fit $k = 3$ component PCA, and calculate out-of-sample reconstruction R^2 over the next 21 days. Compare that with a single PCA fitted once on all 1,000 days ("full-sample"), also evaluated on each test window.

Panel	T	In-sample share (first 3)	Out-of-sample reconstruction R^2	Full-sample PCA test-window R^2	Principal-angle cosines[b]
Factor	60	0.454	0.388	0.420	0.993 / 0.944 / 0.736
Factor	250	0.429	0.409	0.419	1.000 / 0.997 / 0.988
Noise	60	0.186	0.061	0.085	0.867 / 0.659 / 0.286
Noise	250	0.113	0.060	0.085	0.971 / 0.919 / 0.627

[b] The three principal-angle cosines between the leading three-dimensional PCA subspaces of adjacent training windows, in descending order, averaged across all adjacent windows.

Interpretation. (1) In the noise panel, the true share for three directions is $3/50 = 6\%$. Out-of-sample reconstruction is about 6.0%, while in-sample explained variance is 18.6% ($T = 60$) and 11.3% ($T = 250$), a direct example of the noise bound in Section 6.3. (2) Full-sample PCA gets 8.5% on test windows, higher than the honest 6.0%, because the fit has already seen the test data. The gap is smaller for the factor panel (0.420 versus 0.388) because the factor structure is strong. (3) The final column reports the three principal-angle cosines between adjacent windows' leading three-dimensional PCA subspaces (`subspace_cosines`): take the singular values of $V_{\text{old}}^{\top} V_{\text{new}}$ in descending order, then average across adjacent windows. At $T = 60$, the smallest average is 0.736, indicating a poorly aligned direction between the two three-dimensional subspaces; the subspace itself changes over time. Principal angles describe the relation between subspaces, not any particular principal-component axis: swapping PC1 and PC3 within the same subspace leaves all three principal-angle cosines at 1, although the

Continued: the same complete example

corresponding axis cosines are 0, 1, and 0. Thus it is incorrect to say "the third principal component rotated by a certain amount." The average cosine is also not the average angle because arccos is nonlinear; arccos(0.736) is not the average rotation angle. For $T = 250$, all three average cosines are approximately 0.988 or higher. The eigenvalues of the final 250-day sample correlation matrix are 15.92, 3.64, and 2.21 for the factor panel; the largest noise-panel eigenvalue is 1.91. The Marchenko–Pastur upper edge is $(1+\sqrt{50/250})^2 = 2.094$. The factor panel's third eigenvalue is only slightly above the edge, so in real data such a direction can be difficult to distinguish from noise.

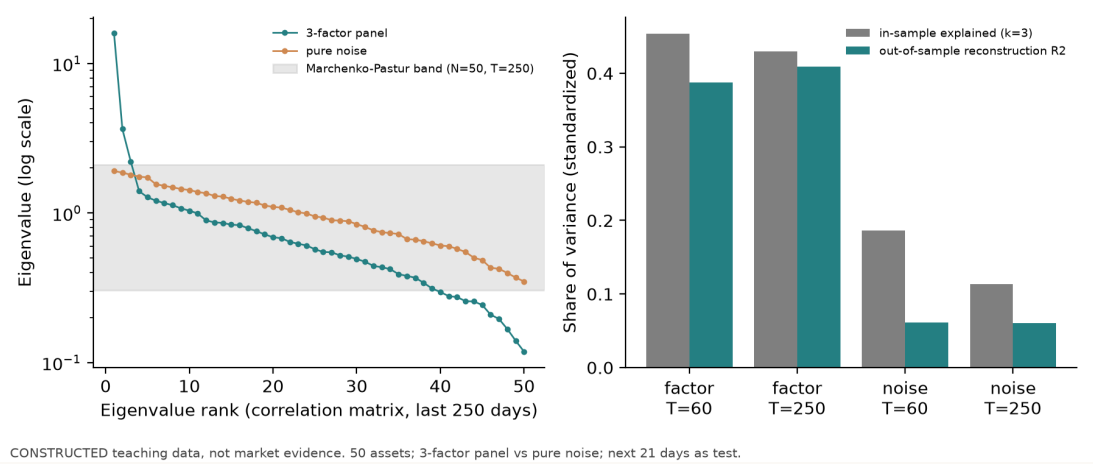


Figure 1, Chapter 41: The left plot shows eigenvalues of the final 250-day correlation matrix on a log scale, with the gray band marking the Marchenko–Pastur range for $N = 50$, $T = 250$. The right plot compares in-sample explained shares (gray) and reconstruction R^2 for the next 21 days (teal) across four settings. Constructed data. Observation: all noise-panel eigenvalues lie within the gray band, while in-sample shares are two to three times their out-of-sample values.

8.3 Hand-Calculating One Filtering Step

Two states: daily returns in the calm state (0) follow $N(0.05\%, 0.7\%^2)$; in the volatile state (1), they follow $N(-0.1\%, 2\%^2)$. Let $P = \begin{pmatrix} 0.98 & 0.02 \\ 0.05 & 0.95 \end{pmatrix}$. Yesterday's closing filtered probability is $\alpha_{t-1} = (0.9, 0.1)$, and today's return is $x_t = -2.5\%$.

1. **Prediction:** $(0.9 \times 0.98 + 0.1 \times 0.05,\ 0.9 \times 0.02 + 0.1 \times 0.95) = (0.887, 0.113)$.
2. **Densities** (units of 1 / decimal return): $\phi(-0.025; 0.0005, 0.007) = 0.0749$ (3.64 standard deviations away), and $\phi(-0.025; -0.001, 0.02) = 9.709$ (1.2 standard deviations away).
3. **Update:** $0.887 \times 0.0749 = 0.0664$ and $0.113 \times 9.709 = 1.0972$. Normalizing gives $\alpha_t = (0.0664, 1.0972)/1.1636 = (0.057, 0.943)$.

A one-day drop of -2.5% raises the volatile-state probability from 11.3% to 94.3%. If today's return were -1.0%, the two densities would be 18.50 and 18.03, nearly equal, and $\alpha_t = (0.890, 0.110)$, so the probability would be determined mostly by the prior. At $+0.3\%$, it would be $(0.955, 0.045)$. The script reproduces these three results with `hmm_filter_smooth(`

Continued: the same complete example

`x=[x_t], init=predicted_probability)` (hand.hmm). The stationary distribution is $(5/7, 2/7)$, with mean durations of 50 and 20 days.

```
import numpy as np
from scipy.stats import norm
P = np.array([[0.98, 0.02], [0.05, 0.95]])
pred = np.array([0.9, 0.1]) @ P
for x in (-0.025, -0.010, 0.003):
    lik = np.array([norm.pdf(x, 0.0005, 0.007), norm.pdf(x, -0.001, 0.02)])
    print(x, np.round(pred, 3), np.round(pred * lik / (pred * lik).sum(), 4))
```

8.4 Regime Timing: Hindsight Labels, Online Labels, and Costs

Constructed market: 3,000 days, with states generated from the Markov chain above (seed 4102); the first 1,000 days are the training window and the remaining 2,000 are the test window. The rules use two position sizes: hold the market (weight 1) when classified as calm, and hold cash (weight 0) otherwise. "Lagged" follows the information clock in Section 4: a signal at close t is executed at close $t + 1$ (one-day execution lag), then earns the return on day $t + 2$. In a daily-return series, this means shifting the signal forward by two periods. "Same-day" aligns the label directly with the return on day t and is only a leakage illustration. A valid rule forms its signal at close t, executes at close $t + 1$, and earns the return on $t + 2$. One-way turnover costs 10 basis points. HMM parameters are estimated on the training window only: a two-component Gaussian mixture provides the means and standard deviations of the states, and a transition matrix is estimated from daily hard-classification counts in the training period with add-one smoothing. This is an approximation using hard classifications from a Gaussian mixture and transition counts; it is not maximum likelihood estimation of the joint HMM likelihood by Baum–Welch. k-means uses past 21-day annualized volatility and 63-day cumulative return, standardized and fitted in the training window with $K = 2$. The threshold for the "past volatility" rule is the median volatility in the training window.

Rule (2,000 test days)	Available when?	Annualized return	Sharpe	Maximum drawdown
Buy and hold	Available	−5.89%	−0.22	−58.6%
True state, same-day	Unavailable	+6.61%	0.71	−17.8%
Smoothed probability, same-day	Leaks	+2.27%	0.28	−19.5%
Smoothed probability, lagged execution	Leaks	−5.38%	−0.28	−47.6%
Centered 21-day volatility, lagged	Leaks	+2.21%	0.33	−13.1%
Filtered probability (training-estimated parameters), lagged	Available	−8.89%	−0.51	−61.0%
Filtered probability (true parameters), lagged	Parameters unavailable	+2.73%	0.28	−26.8%
Online k-means state, lagged	Available	−2.83%	−0.15	−42.7%
Past 21-day volatility, lagged	Available	+3.54%	0.41	−19.9%

Continued: the same complete example

Rule (2,000 test days)	Annual turnover	State accuracy
Buy and hold	0.1	0.747
True state, same-day	8.7	1.000
Smoothed probability, same-day	33.9	0.871
Smoothed probability, lagged execution	33.9	0.871
Centered 21-day volatility, lagged	12.9	0.704
Filtered probability (training-estimated parameters), lagged	39.4	0.856
Filtered probability (true parameters), lagged	11.5	0.940
Online k-means state, lagged	5.9	0.792
Past 21-day volatility, lagged	12.7	0.689

Let u_d denote the signal formed at close (distinct from the latent state s_d). With returns indexed by their ending date, $r_d = C_d/C_{d-1} - 1$. Verify the timing with two adjacent signal dates (the dates below are illustrative trading dates):

Signal formed (close)	Execution (next close)	Holding-return interval	Position in return series
October 12	October 13	Close October 13 to close October 14	r_{14} multiplied by u_{12}
October 13	October 14	Close October 14 to close October 15	r_{15} multiplied by u_{13}

The numerical results in this chapter use a fixed-weight-change proxy: $w_d = u_{d-2}$, $\text{turn}_d = |w_d - w_{d-1}|$, and $r_d^{net} = w_d r_d - c\text{turn}_d$. Costs are recorded on the return-ending row. The proxy does not model rebalancing amounts after portfolio-value drift, cash before and after execution, or the event ordering of trades and costs, so it is not equivalent to the event-based self-financing ledger in Chapter 29. The historical proxy numbers and omitted mechanics are retained here; this teaching revision does not silently change the engineering implementation. Thus, the weight on return-ending date d comes from u_{d-2}. Shifting only one period would pair a newly formed signal with a return interval that had already occurred before execution. The "same-day" comparison in the table uses $u_d r_d$ and also violates this clock.

Results from a single path are highly sensitive to chance. As a post hoc robustness diagnostic (D11; not a registered primary result or confirmatory evidence), the same design was repeated with 50 additional seeds. The table shows median Sharpe and the share of paths in which Sharpe exceeded buy and hold on that same path:

Rule	Median Sharpe	Share beating buy and hold	Median annual turnover
Buy and hold	0.02	—	0.1
True state, same-day (unavailable)	0.86	1.00	7.2
Smoothed probability, same-day (leaks)	0.76	0.98	32.3
Smoothed probability, lagged (leaks)	0.05	0.54	32.3
Centered volatility, lagged (leaks)	0.63	1.00	8.8
Filtered probability (training-estimated parameters)	−0.03	0.38	38.3
Filtered probability (true parameters, unavailable)	0.38	0.90	12.2
Online k-means state	0.29	0.90	6.1
Past volatility	0.35	0.78	8.8

There are four takeaways. (1) Same-day hindsight labels earn the most because they use that day's return: after a large daily decline, the smoothed probability classifies that day as

Continued: the same complete example

volatile, so the rule is "conveniently" out of the market on that day. When the same smoothed probability is executed on the valid schedule ($t+1$ execution, return on $t+2$), median Sharpe falls from 0.76 to 0.05. (2) A centered window uses volatility from the next 10 days, and it still leads when executed on the valid schedule (median 0.63; it beats buy and hold on all 50 paths). "Lagged" does not mean "free of leakage." (3) Valid online rules retain some value (median 0.29–0.35), well below the leakage-based rules. (4) The HMM estimated on the training window performs worst because of parameter estimation: daily hard classification labels many ordinary calm-period fluctuations as volatile. The estimated transition matrix is $\begin{pmatrix} 0.874 & 0.126 \\ 0.629 & 0.371 \end{pmatrix}$ (true diagonal values 0.98 and 0.95); filtered probabilities switch frequently, generating 38–39 turnovers per year and costs that consume returns. The same filter with true parameters has median Sharpe 0.38. State accuracy and returns do not move together: the training-parameter filter has accuracy 0.856, higher than the past-volatility rule's 0.689, but much lower returns.

Cluster stability on training-window features, using 20 IID row bootstrap samples (a sensitivity check to row perturbations only, not a temporal-stability test): at $K = 2$, mean ARI is 0.943 and the minimum is 0.723; at $K = 3$, mean ARI is 0.971 and the minimum is 0.815; at $K = 4$, mean ARI is 0.722 and the minimum is 0.364. The $K = 4$ partition is clearly less stable under resampling. After a true calm-to-volatile transition, the median delay until the filtered probability of calm $P(\text{calm} \mid x_{1:t})$ first falls below 0.5 is 1 day (29 transitions). Figure 2 plots its complement, $P(\text{volatile} \mid x_{1:t})$, and the corresponding first crossing above 0.5. This quick response also comes with many false alarms, which drive turnover.

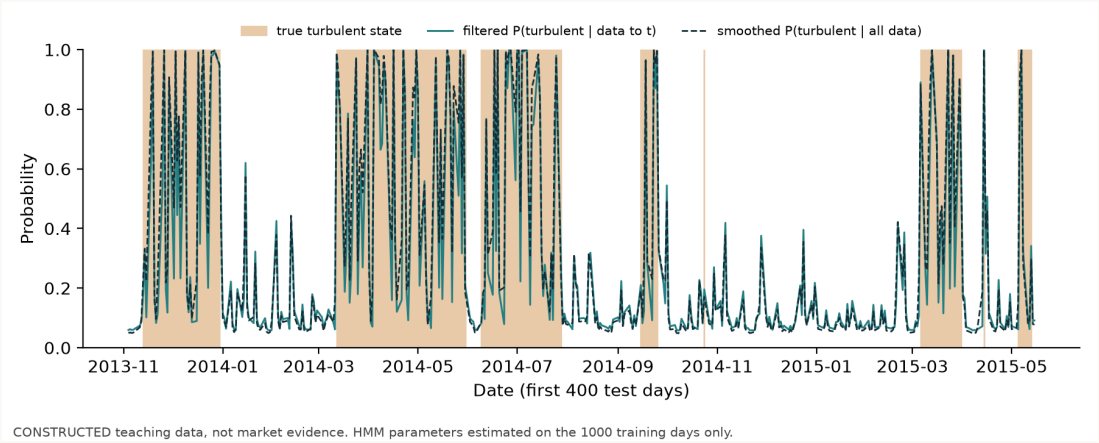


Figure 2, Chapter 41: The first 400 test days. Shading marks the true volatile state; the solid line is filtered probability $P(\text{volatile} \mid \text{data through } t)$, and the dashed line is smoothed probability $P(\text{volatile} \mid \text{the full sample})$. Constructed data; HMM parameters are estimated only on the 1,000-day training window. Observation: the smoothed curve rises before a transition because it uses later observations; the filtered curve can react only after a decline and produces frequent false alarms during calm periods.

9. Python Implementation

- Core module: `src/quant_course/unsupervised_tools.py`.
- Interface contract: Section 2 of `qa/a8/ch40_ch41_batch/INTERFACE_CONTRACT.md`. Public functions: `pca_fit`, `pca_project`, `pca_reconstruction_r2`, `subspace_cosines`, `trailing_state_features`, `fit_kmeans_states`, `assign_states`, `align_labels`, `cluster_stability`, `hmm_filter_smooth`, `simulate_markov_regimes`, and `simulate_factor_panel`. Tests: `tests/test_unsupervised_tools.py`.
- Chapter script, approximately 7 seconds. Pass only –out when reproducing results; –figures-dir `course/zh/chapters/ch41/figures` overwrites the source figures and should be used only when intentionally refreshing them:

  ```
  .venv/bin/python code/ch41_unsupervised.py --out <new-output-dir>
  ```

  Output: `results.json` and three figures. The command refuses to use an existing output directory.
- Environment: Python 3.14.7, numpy 2.5.3, pandas 3.0.6, scikit-learn 1.9.1, and scipy 1.18.1 (`.venv`).
- Seeds: factor panel 4101; noise panel 4103; state market 4102; variance-and-predictive-power example 4104; robustness diagnostic 4200–4249. `KMeans` and Gaussian-mixture `random_state` values match the corresponding seeds.
- Missing values: `pca_fit` and `pca_project` raise an error on NaN (the caller must handle them first); `trailing_state_features` returns NaN when the window is insufficient; `assign_states` returns NaN for rows containing NaN, and the rules treat NaN as a flat position.
- Timing: daily frequency; signal at close t; execution at close $t+1$; return interval from close $t+1$ to close $t+2$. Leakage-comparison rules are named with `[leaks]`, `[impossible]`, or `[oracle params]`.

Minimal example of training-only PCA and out-of-sample projection:

```
import numpy as np, pandas as pd
from quant_course import unsupervised_tools as ut
R, B = ut.simulate_factor_panel(300, 20, factor_vols=(0.01, 0.005), idio_vol=0.015,
    ↪ seed=1)
fit = ut.pca_fit(R.iloc[:250], 2, standardize=True)  # mean, SD, loadings: training
    ↪ rows only
scores = ut.pca_project(R.iloc[250:], fit)           # later dates use the training
    ↪ mean/SD
r2 = ut.pca_reconstruction_r2(R.iloc[250:], fit)
print(fit.explained_ratio.round(3), round(r2, 3), scores.shape)
```

10. Visualization

- Figure 1 (`figures/fig41_1_pca_in_vs_out.png`, Section 8.2): the horizontal axis is eigenvalue rank and the vertical axis is eigenvalue (log scale); the gray band marks the Marchenko–Pastur range. The right plot's horizontal axis shows four `panel/window` combinations and its vertical axis shows variance shares. Compare noise eigenvalues with the MP range and in-sample with out-of-sample results.
- Figure 2 (`figures/fig41_2_filtered_vs_smoothed.png`, Section 8.4): date on the horizontal axis and probability on the vertical axis; observe the "foresight" in smoothed probabilities.
- Figure 3 (`figures/fig41_3_regime_rules.png`): wealth after 10 basis points of cost across 2,000 test days (log scale, starting at 1). Dashed and dotted lines are rules that use information unavailable at trade time.

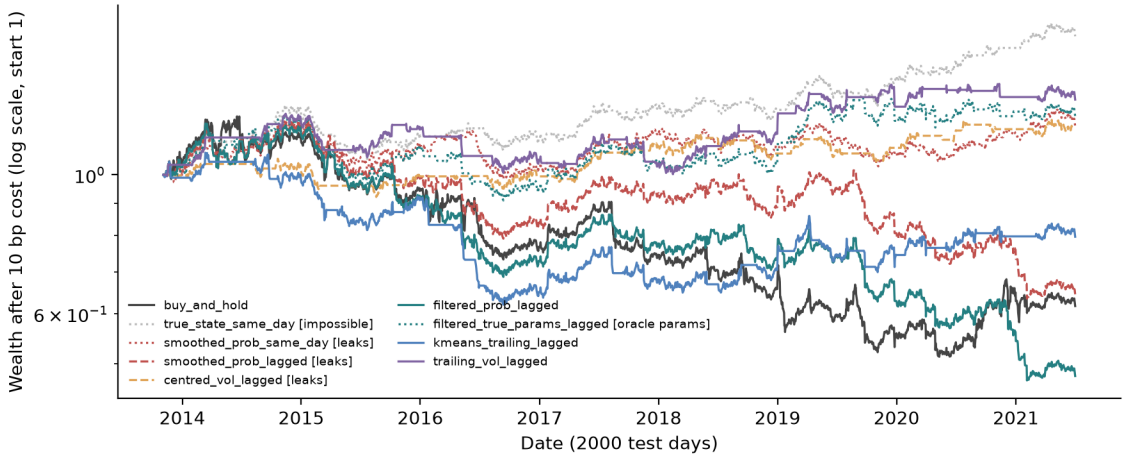


Figure 3, Chapter 41: After-cost wealth for nine rules over the test period (log scale). Constructed data, one path (seed 4102). Observation: same-day hindsight labels and centered volatility are at the top; valid online rules are much lower; the filter with training-estimated parameters falls below buy and hold because of high turnover.

11. Real-Market Example

Status: Experiment pending. This course does not have authorization to use a daily stock or index return panel (`projects/project06/data_manifest.json`; real_data.status = blocked), so this chapter reports no real-market principal components, states, or timing results. To conduct the study, obtain: (1) at least eight years of daily stock returns with point-in-time constituents (including delisted stocks), for training-only PCA, out-of-sample reconstruction R^2, and an eigenvalue-bound comparison; (2) daily returns and trading-cost assumptions for a tradable index, for state rules; and (3) permission to display derived statistics in internal training materials. Once the data are available, preregister the windows, k, K, parameter-estimation method, and costs under the protocol in Section 13.

Literature comparison (not evidence from this book): Laloux et al. (1999) used the return correlation matrix for S&P 500 constituents and found that most eigenvalues fell inside the Marchenko–Pastur range, with only a few (the largest corresponding to the market) clearly outside it. Ang and Timmermann (2012) review applications of regime-switching models in asset pricing. Reading status is documented in Section 20.

12. Common Mistakes

12.1 Full-Sample Standardization and PCA

Mistake: `StandardScaler().fit(R)` and `PCA().fit(R)` use all dates, and the resulting scores are treated as daily risk factors or features. **Why:** test-period means, volatilities, and correlation structure enter the training-period transformation. **Impact** (noise panel in Section 8.2): full-sample PCA has test-window reconstruction R^2 of 8.5%, compared with 6.0% for training-only PCA; the in-sample share is higher at 11.3%–18.6%. **Fix:** at each rebalance date, fit `pca_fit` on the past window only and use `pca_project` on later dates.

```
import numpy as np, pandas as pd
from quant_course import unsupervised_tools as ut
noise = pd.DataFrame(np.random.default_rng(4103).normal(0, 0.015, (1000, 50)))
noise.columns = [f"A{i:02d}" for i in range(50)]
full = ut.pca_fit(noise, 3, standardize=True)               # WRONG: sees the test
    ↪ window
train = ut.pca_fit(noise.iloc[:250], 3, standardize=True)   # right: training window
    ↪ only
test = noise.iloc[250:271]
print(round(ut.pca_reconstruction_r2(test, full), 3),
      round(ut.pca_reconstruction_r2(test, train), 3),
      round(float(train.explained_ratio.sum()), 3))
```

The output 0.115 0.077 0.112 means that, on this single 21-day test window, full-sample PCA has reconstruction R^2 of 11.5%, training-only PCA has 7.7%, and the training-window in-sample share is 11.2%. Section 8.2 reports averages over all windows (8.5%, 6.0%, and 11.3%).

12.2 Treating Smoothed Probabilities or Hindsight Labels as Trading Signals

Mistake: fit an HMM on the full sample and use smoothed probabilities, or label each day with a "crisis period" confirmed after the fact and align it with the same day's return. **Why:** the label uses future observations; same-day alignment also uses the return being evaluated. **Impact** (Section 8.4, median over 50 seeds): the same-day smoothed-probability Sharpe is 0.76, versus 0.05 when executed on the valid schedule (execute at $t+1$, earn the $t+2$ return). **Fix:** use only filtered probabilities $\Pr(s_t \mid x_{1:t})$, estimate parameters only on the training window, and execute the signal one day later.

12.3 Centered Rolling Windows

Mistake: `ret.rolling(21, center=True).std()`. **Why:** the value for date t uses $t+1, \ldots, t+10$. Even when the signal is executed at close $t+1$ and earns the return on $t+2$, data from $t+3, \ldots, t+10$ still arrive after the return being earned. **Impact:** median Sharpe over 50 seeds is 0.63, versus 0.35 for the past-window version. **Fix:** use `rolling(21)` (right-aligned by default), and test that changing data after t does not change features at t or earlier.

12.4 Unaligned Cluster Labels

Mistake: refit k-means in each window and compare "cluster 1" directly. **Why:** labels are arbitrary and can switch between fits. **Impact:** partitions $[0, 0, 1, 1, 2, 2]$ and $[1, 1, 0, 0, 2, 2]$ are identical (ARI $= 1$), but elementwise agreement is only $1/3$. **Fix:** relabel clusters by a meaningful center feature (volatility in this book) or use `align_labels` for Hungarian matching; ARI itself is label-invariant.

12.5 Selecting Principal Components for Prediction by Explained Variance

Mistake: principal-component regression keeps only components that "explain 99% of the variance." **Why:** PCA does not use labels. **Impact** (constructed example in Section 6.4; first half for training and second half for testing): the first covariance-PCA component explains 98.96% of feature variance, but its out-of-sample R^2 is -0.07%; using two components gives 69.6%. After standardization, the first loading is $(0.707, 0.707)$ and out-of-sample R^2 is 31.2%. **Fix:** treat k as a hyperparameter and select it by prediction error in date-based training-only validation (as in Project 6 v2's PCA+Ridge ablation in Section 13).

12.6 Evaluating Market Timing by State Accuracy

Mistake: report "85.6% state-classification accuracy" as evidence that a strategy works. **Why:** accuracy weights each day equally and ignores the size of returns on misclassified days and turnover costs. **Impact:** the training-parameter filter in Section 8.4 has accuracy 0.856 and Sharpe -0.51; the past-volatility rule has accuracy 0.689 and Sharpe 0.41. **Fix:** report after-cost returns, turnover, and drawdown together.

13. Quant Research Application

Hypothesis. Do high-variance directions (principal components) of cross-sectional features contain predictive information about future five-day returns? This auxiliary ablation was specified in the Project 6 v2 protocol before v2 runs (`configs/p06_trees_v2.json`, `pca_ridge`), not a standalone strategy project. The v1 constructed results had already been viewed; this was not an ex-ante real-market preregistration.

Method. For each outer fold in the date-based walk-forward of Chapter 38, fit `StandardScaler`, PCA(k), and Ridge(α) on training rows only. Select $k \in \{5, 15, 30\}$ and $\alpha \in \{10^4, 10^6\}$ by inner date-validation MSE. Prediction, dollar-neutral portfolios, lagged execution, and cost ledger are otherwise identical to Ridge. Benchmarks are zero prediction, the training mean, and Ridge on the original features.

Controls and interpretation limits. The 43 q and noise series are independent across series and have unit marginal variance, but each follows an AR(1) process with coefficient 0.98. Three additional price features come from the return path and are indirectly related to q. The full 46-dimensional feature set is not an IID, homoskedastic population, and finite-sample principal directions are not prespecified signal directions. Truncated PCA may discard signal or reduce estimation variance; which effect dominates must be tested, and improvement cannot automatically be called leakage. A necessary control retains all $k = 46$ components, the same training scaler, the same α, and no whitening. Under the orthogonal rotation V, $\|y - XVb\|^2 + \alpha\|b\|^2 = \|y - X\beta\|^2 + \alpha\|\beta\|^2$, where $\beta = Vb$, so theoretical predictions match original-feature Ridge (apart from numerical error). This equivalence does not hold after truncation, and truncation is not guaranteed to help by denoising. Read historical results by metric and scenario: with the training mean as the R^2 benchmark, PCA+Ridge has higher R^2 than Ridge in both scenarios, while both remain below zero; its IC is lower in both; its base-case Sharpe at 10/20 bp is lower, while in the sign-reversal case $-0.14/-0.52$ is slightly higher than Ridge's $-0.15/-0.53$. Results are in the Project 6 v2 table in Chapter 40, Section 13, and `projects/project06/research_report/v2_trees.md`.

Clustering and states. This batch does not add cluster states to Project 6: the constructed "sign-reversal" scenario changes the relation between cross-sectional features and returns, while market-level volatility and returns are unchanged. Clustering on market-state features therefore has no identifiable target. A state-rule study requires a separate protocol (Section 17, 41-C02).

14. Chapter Summary

PCA and clustering answer "how are the data distributed?" rather than "what will happen next?" PCA eigenvalues give training-window variance shares. In-sample shares are biased upward by noise; out-of-sample reconstruction R^2 and noise bounds provide more honest measures. Scaling determines the directions, and both scaling parameters and components must be estimated within the training window. Clustering depends on scaling, the number of clusters, and initialization; cluster labels have no intrinsic meaning. Row resampling tests sensitivity to row perturbations only; block resampling or rolling refits are needed for temporal stability. State labels divide into three types: online labels and filtered probabilities can be traded; smoothed

probabilities and centered windows use future information; hindsight labels aligned with same-day returns also use the return being evaluated. Even valid state rules face parameter-estimation error, detection delay, false alarms, and turnover costs, so evaluate them by after-cost returns rather than classification accuracy.

15. Exercises

41-E01 (by hand) Let $\Sigma = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$. Find the eigenvalues, unit loadings (using this book's sign convention), and the first component's explained-variance ratio.

41-E02 (scaling) Two uncorrelated columns have variances 9 and 1. (a) What are the first covariance-PCA loading and explained share? (b) What is special about correlation-matrix PCA after standardization? (c) Using (a) and (b), explain when the "first principal component" depends on the units of measurement.

41-E03 (noise bound) There are $N = 100$ independent assets and $T = 400$ days. Find the Marchenko–Pastur interval. With no common factors, what share does the first in-sample principal component appear to explain, and what is its true share?

41-E04 (interpretation) Why is a dollar-neutral portfolio with zero market exposure almost unaffected by the first return principal component (which is approximately the market)? What does this say about the relationship between explained variance and portfolio predictive power?

41-E05 (k-means) For one-dimensional data $\{0.10, 0.12, 0.15, 0.30, 0.35, 0.40\}$, $K = 2$, and initial centers 0.10 and 0.15, work through Lloyd iterations to convergence. Show assignments, centers, and objective value (inertia) at every step.

41-E06 (filtering) Use the parameters in Section 8.3. Convert returns, means, and standard deviations to decimal returns for density calculations ($-1\% = -0.010$, $0.7\% = 0.007$). Yesterday's filtered probability is $(0.6, 0.4)$ and today's return is -1.0%. Find the predicted probabilities, the two densities, and today's filtered probabilities.

41-E07 (timing) Which of the following are available at close t for execution at $t + 1$? (a) past 21-day volatility; (b) volatility from rolling(21, `center=True`); (c) filtered probability with parameters estimated in the training window; (d) filtered probability from an HMM fitted on the full sample; (e) smoothed probability from an HMM fitted on the full sample; (f) nearest-center assignment for past features using k-means fitted in the training window.

41-E08 (cluster labels) What are the ARI and elementwise agreement for partitions $[0, 0, 1, 1, 2, 2]$ and $[1, 1, 0, 0, 2, 2]$? What are they for $[0, 0, 1, 1, 2, 2]$ and $[0, 1, 0, 1, 2, 2]$?

16. Programming Exercises

41-P01 (training-only PCA) Input: generate 500 days and 30 assets with `simulate_factor_panel`, factor_vols=(0.01, 0.004), idio_vol=0.015, and seed=7. Implement a rolling window ($T = 120$, 20 test days, step size 20) with $k = 2$ training-only PCA. Return each window's in-sample explained share and out-of-sample reconstruction R^2. Acceptance assertions: (1) for this specified construction and seed, mean in-sample share $\geq$ mean out-of-sample R^2; this is not a theorem for arbitrary data or windows; (2) multiplying all data after a given test window by 10 leaves that window and all earlier window results unchanged; (3) the function never re-estimates means or standard deviations on a test window (as a construction check, adding a constant to the entire test window changes out-of-sample R^2).

41-P02 (filtering does not look ahead) Use `hmm_filter_smooth` on the 500-day series from `simulate_markov_regimes` with p_stay=(0.97, 0.9), mus=(0.0005, -0.001), sigmas=(0.008, 0.02), and seed=3. Acceptance assertions: (1) changing $x_{t+1..T}$ leaves filtered[:t+1] unchanged and changes smoothed[:t+1]; (2) each row sums to 1; (3) verify one update against the numbers in Section 8.3. Boundary condition: at $T = 1$, the two probabilities are equal.

41-P03 (state-rule backtest) Given a daily return series and a 0/1 signal, implement the simplified fixed-weight proxy "signal at close t, execute at close $t + 1$, earn the $t + 2$ return, one-way cost c" (cost charged on the return-ending row). Return annualized return, Sharpe, turnover, and maximum drawdown (with $N + 1$ wealth points). Acceptance assertions: (1) a constant-one signal with $c = 0$ matches buy and hold (from day 3 onward); (2) on the Section 8.4 data, the same-day-aligned version using the true state has higher returns than the two-day-lagged version; (3) with $c > 0$, at least one nonzero turnover, and positive wealth multipliers at every step, the after-cost terminal wealth is strictly below that for $c = 0$; if the portfolio is flat throughout and has zero turnover, they are equal.

17. Challenge Problems

41-C01 (Project 6 v2 PCA ablation) Compare the results and selected k values by fold for `pca_ridge` in Chapter 40, Section 13, and `projects/project06/research_report/v2_trees.md`. (a) How do the results compare with the metric-by-metric reference and interpretation limits in Section 13? (b) Give a constructed data-generating process in which PCA+Ridge outperforms Ridge on the original features and explain what real-world situation it represents. (c) Explain why you cannot change the k grid to "exactly the best value" after seeing v2 results and report that result. Additional prerequisite: Chapter 39.

41-C02 (honest state-study design) Design a state-timing study for a tradable index. Specify the number of states, parameter estimation (for example, Baum–Welch EM on training windows only), online signal, execution time, costs, evaluation metrics, benchmarks, out-of-sample period, and falsification condition. Explain how you prevent leakage from Sections 12.2 and 12.3 and how you report the effects of parameter-estimation error (as in the true-parameter comparison in Section 8.4). Additional prerequisites: Chapter 35 on trading costs, Chapter 31 on validation, and Chapter 32 on performance evaluation.

18. Complete Solutions

41-E01. $(3-\lambda)^2-1=0 \Rightarrow \lambda=4,2$. For $\lambda=4$, $a=b$, so the loading is $(0.7071, 0.7071)$ (positive sum). For $\lambda=2$, $a=-b$, giving a zero sum; choose the first element positive, $(0.7071, -0.7071)$. The explained share is $4/6=66.7\%$.

41-E02. (a) The covariance matrix is $\mathrm{diag}(9,1)$, the first loading is $(1,0)$, and its explained share is 90%. (b) After standardization, the correlation matrix is the identity; both eigenvalues are 1, every unit direction is an eigenvector, and the principal component is not unique. Each direction explains 50%. (c) When columns are weakly correlated and their variance difference mainly reflects their units, the first covariance-PCA component is simply the highest-variance column. Changing that column's units (for example, from percent to decimal) changes the PCA result. For predictive features, standardize using training rows only.

41-E03. $q=100/400=0.25$, so $\lambda_\pm=(1\pm0.5)^2=2.25,\ 0.25$ for a correlation matrix with $\sigma^2=1$. The first in-sample component appears to explain about $2.25/100=2.25\%$; its true share is $1/100=1\%$. This is the upper-edge location under a large-sample limit; the largest finite-sample eigenvalue fluctuates around it.

41-E04. If stock i has return $r_i=\beta_i f+e_i$, then a portfolio has return $\sum_i w_i r_i = f\sum_i w_i\beta_i + \sum_i w_i e_i$. When $\sum_i w_i\beta_i=0$, the market term disappears. Dollar neutrality alone does not guarantee this condition. When both dollar neutral and beta-neutral, the portfolio hedges the market direction that explains a large share of common variance; portfolio returns come from other directions (idiosyncratic components or low-variance factors). Thus, the variance share explained by the first component does not by itself determine whether cross-sectional return differences can be predicted.

41-E05. Step 0: with centers 0.10 and 0.15, the midpoint is 0.125. Assign $\{0.10, 0.12\}$ and $\{0.15, 0.30, 0.35, 0.40\}$. The objective is $0+0.02^2+0+0.15^2+0.2^2+0.25^2=0.1254$. Updated centers are 0.11 and 0.30. Step 1: the midpoint is 0.205. Assign $\{0.10, 0.12, 0.15\}$ and $\{0.30, 0.35, 0.40\}$. The objective (new assignments, old centers) is $0.01^2+0.01^2+0.04^2+0+0.05^2+0.1^2=0.0143$. Updated centers are $0.1233, 0.35$. Step 2: the midpoint is 0.2367; assignments are unchanged. The objective is $0.0233^2+0.0033^2+0.0267^2+0.05^2+0+0.05^2=0.006267$. The centers no longer change, so the algorithm converges. The objective decreases monotonically: $0.1254 \to 0.0143 \to 0.006267$.

41-E06. Convert the returns, means, and standard deviations to decimal returns first; densities are measured "per unit decimal return," while normalized state probabilities are dimensionless. The prediction is $(0.6\times0.98+0.4\times0.05,\ 0.6\times0.02+0.4\times0.95)=(0.608, 0.392)$. The densities are $\phi(-0.01; 0.0005, 0.007)=18.50$ and $\phi(-0.01; -0.001, 0.02)=18.03$. Normalize $(0.608\times18.50,\ 0.392\times18.03)$ to obtain filtered probability $(0.614, 0.386)$. A -1% return is almost equally likely in both states, so the posterior remains close to the prior.

41-E07. Available: (a), (c), and (f). Unavailable: (b), which uses the next 10 days; (d), because although the filtering recursion itself uses only $x_{1:t}$, parameters estimated on the full sample contain test-period information (a subtler form of leakage, usually smaller than (e), but still invalid); and (e), because smoothing uses $x_{t+1:T}$.

41-E08. For the first pair, both partitions group the same points and only swap labels, so ARI $= 1$; elementwise agreement is $2/6 = 1/3$. For the second pair, comparing $[0, 0, 1, 1, 2, 2]$ with $[0, 1, 0, 1, 2, 2]$, ARI is $1/6 \approx 0.167$ and elementwise agreement is $4/6$. ARI measures partition agreement and is label-invariant; elementwise agreement depends on the labels and requires alignment first.

41-P01. Reference implementation.

```
import numpy as np
from quant_course import unsupervised_tools as ut


def rolling_in_out(R, window=120, test=20, k=2):
    rows = []
    for s in range(window, len(R) - test + 1, test):
        fit = ut.pca_fit(R.iloc[s - window:s], k, standardize=True)
        oos = ut.pca_reconstruction_r2(R.iloc[s:s + test], fit)
        rows.append((float(fit.explained_ratio.sum()), oos))
    return np.array(rows)


R, _ = ut.simulate_factor_panel(500, 30, factor_vols=(0.01, 0.004), idio_vol=0.015,
    ↪ seed=7)
a = rolling_in_out(R)
assert a[:, 0].mean() >= a[:, 1].mean()
R2 = R.copy()
R2.iloc[300:] *= 10                 # change only data from row 300 on
b = rolling_in_out(R2)
assert np.allclose(a[:8], b[:8])    # windows whose test ends by row 300 are
    ↪ unchanged
R3 = R.copy()
R3.iloc[120:140] += 0.05
assert not np.isclose(rolling_in_out(R3)[0, 1], a[0, 1])  # shifting the test changes
    ↪  R2
print(a.mean(axis=0).round(3))
```

The first test window is rows 120–139. The first eight windows end before row 300 and are therefore unaffected. The output reports averages of the two columns. This assertion for a fixed construction does not generalize to all data or establish that out-of-sample performance is always below in-sample performance.

41-P02. Reference implementation.

```
import numpy as np
from quant_course import unsupervised_tools as ut
m = ut.simulate_markov_regimes(500, p_stay=(0.97, 0.9), mus=(0.0005, -0.001),
                               sigmas=(0.008, 0.02), seed=3)
x = m["ret"].to_numpy()
kw = dict(means=[0.0005, -0.001], sds=[0.008, 0.02],
          transition=np.array([[0.97, 0.03], [0.1, 0.9]]), init=[1.0, 0.0])
f1, s1 = ut.hmm_filter_smooth(x, **kw)
x2 = x.copy()
x2[301:] = -0.05
f2, s2 = ut.hmm_filter_smooth(x2, **kw)
assert np.allclose(f1[:301], f2[:301]) and not np.allclose(s1[:301], s2[:301])
assert np.allclose(f1.sum(axis=1), 1) and np.allclose(s1.sum(axis=1), 1)
P = np.array([[0.98, 0.02], [0.05, 0.95]])
one, _ = ut.hmm_filter_smooth(np.array([-0.025]), means=[0.0005, -0.001], sds=[0.007,
    ↪  0.02],
                              transition=P, init=np.array([0.9, 0.1]) @ P)
assert np.allclose(one[0], [0.057059, 0.942941], atol=1e-6)
print("ok")
```

Metric conventions: let n be the number of daily return rows evaluated, including the first two flat days in this exercise. Annualized return is terminal wealth raised to $252/n$ minus 1 (CAGR). Sharpe is $\sqrt{252}$ times the arithmetic mean of daily net simple returns divided by their sample standard deviation (`ddof=1`); it is not CAGR divided by annualized volatility. Annual turnover is $252 \sum \text{turn}/n$. Drawdown includes initial wealth of 1. If the standard deviation is zero, Sharpe is undefined.

41-P03. Reference implementation (same information clock and cost proxy as the book implementation in `code/ch41_unsupervised.py`; includes all metrics requested by the exercise and checks three assertions. Section 8.4 shifts the full 3,000-day series before taking the test segment; this example slices first and shifts afterward, leaving the first two test days flat, so the numbers differ from Section 8.4):

```
import numpy as np, pandas as pd
from quant_course import unsupervised_tools as ut


def book(ret, signal, cost, lag=2):
    w = signal.shift(lag).fillna(0.0)
    turn = w.diff().abs().fillna(w.abs())
    net = w * ret - cost * turn
    wealth = np.concatenate([[1.0], np.cumprod(1 + net.to_numpy())])
    n = len(net)
    if n == 0 or np.any(1 + net.to_numpy() <= 0):
        raise ValueError("Require observations and positive wealth multipliers")
    sd = float(net.std(ddof=1)) if n > 1 else float("nan")
    ann_return = float(wealth[-1] ** (252 / n) - 1)
    sharpe = float(np.sqrt(252) * net.mean() / sd) if sd > 0 else float("nan")
    return {"wealth": wealth, "annual_return": ann_return, "sharpe": sharpe,
            "turnover": float(turn.sum()), "annual_turnover": float(turn.sum() * 252
    ↪ / n),
            "mdd": float((wealth / np.maximum.accumulate(wealth) - 1).min())}
```

```
m = ut.simulate_markov_regimes(3000, p_stay=(0.98, 0.95), mus=(0.0005, -0.001),
                               sigmas=(0.007, 0.02), seed=4102)

# The signal is sliced to the test window before shifting, so the first two test days
    ↪ are

# flat; code/ch41_unsupervised.py shifts the full series first (its §8.4 numbers
    ↪ differ).
ret, calm = m["ret"].iloc[1000:], (m["state"].iloc[1000:] == 0).astype(float)
bh = book(ret, pd.Series(1.0, index=ret.index), 0.0)
assert np.isclose(bh["wealth"][-1], np.prod(1 + ret.iloc[2:]))
same_day, lagged = book(ret, calm, 0.001, lag=0), book(ret, calm, 0.001, lag=2)
assert same_day["wealth"][-1] > lagged["wealth"][-1]
assert book(ret, calm, 0.001)["wealth"][-1] < book(ret, calm, 0.0)["wealth"][-1]
print(round(same_day["wealth"][-1], 3), round(lagged["wealth"][-1], 3))
```

41-C01. Complete reference answer. In the base scenario, PCA+Ridge has $R^2 = -0.008\%$ relative to the training mean, versus -0.113% for Ridge. IC is 0.0135 versus 0.0198, with paired $\Delta IC = -0.0063$ and descriptive $t = -0.86$. Sharpe at 10/20 bp is 0.25/−0.13, below Ridge's 0.45/0.07. In the sign-reversal scenario, $R^2 = -0.053\%$ versus -0.161%, IC is 0.0063 versus 0.0087, paired difference is −0.0023, and $t = -0.31$; Sharpe at 10/20 bp is −0.14/−0.52, slightly better than Ridge's −0.15/−0.53. "PCA is worse on every metric" would be inaccurate. Across outer folds 1–16, both scenarios use the same selected-k sequence: [30, 30, 30, 30, 5, 5, 15, 5, 5, 30, 30, 5, 30, 30, 30, 30]. That is 30 components in 10 folds, 5 in 5 folds, and 15 in 1 fold. The frozen summary has no fitdate field, so do not assign dates to these fold numbers. The two scenarios also show that better R^2 with respect to the training mean can coexist with lower IC: level error and cross-sectional `ranking/portfolio` performance are different quantities.

One testable construction makes several features noisy measurements of a common latent variable z and generates labels from z plus label noise. With a finite training sample and a strong common direction associated with z, a small number of PCs may aggregate measurement and discard nonpredictive low-variance directions, potentially outperforming Ridge on the original features; this is not guaranteed. Retain the full-k orthogonal-equivalence control, use the same scaler and α, and do not whiten; compare using training-only validation. This represents a setting where many correlated indicators measure the same economic quantity, not a mechanism already validated by this project.

Changing the k grid to its best value after viewing test results turns the outer test set into a selection set. A new idea can be tested in a new version, with the prior results disclosed, the original grid and negative results retained, and independent later dates used for evaluation; the old test data cannot be relabeled as unseen. PCA is a compression tool, and these constructed results do not support a market-alpha claim.

41-C02. Complete protocol example. First confirm the index, total-return series, tradable instrument, and data permission; fix the training period and unseen test dates. Use only past 21-day volatility and 63-day return as state features, with the scaler estimated within each training window. Fix $K = 2$, or compare 2 and 3 using training-only date validation and select by likelihood and stability, not test returns. Fit the HMM with Baum–Welch EM from multiple initializations within each training window; record initial values, convergence, likelihood, and transition matrices. Do not describe Gaussian-mixture hard-classification transition counts as maximum likelihood for the joint HMM. Freeze parameters, filter forward, form the signal at close t, execute at close $t + 1$, and earn the return ending at $t + 2$. Check that perturbing future data does not change earlier features or filtered probabilities.

For the actual strategy, use the Chapter 29 event ledger, one-way costs of 0/10/20 bp, and explicit assumptions for impact and financing. If a teaching proxy is also shown, state separately that it charges costs on the return-ending row and explain how it differs from the event ledger. Use buy and hold and a past-volatility rule as benchmarks. Report annualized return, Sharpe, drawdown, and turnover; results in crisis and calm periods; and results across initializations and rolling windows. Full-sample smoothing and centered windows are unavailable-to-trade diagnostics only and cannot be selected into the strategy. Interpret state labels from training-window centers; ARI does not require label alignment, while temporal stability requires block perturbation or rolling refits.

Before looking at test results, specify a decision rule; for example, do not adopt the strategy if its after-cost performance is no better than the past-volatility rule, its advantage appears in only one subperiod, or it is highly sensitive to initialization. A simulation with true parameters can serve as an unavailable oracle reference, not a guaranteed upper bound on after-cost returns or Sharpe: compare estimated and true transition matrices, false alarms and delay, turnover, and cost loss. Section 8.4's true-parameter median Sharpe of 0.38 versus -0.03 with training-estimated parameters shows that estimation error deserves investigation, but does not guarantee that EM will move estimates toward the true values. Real markets have no known true states or parameters; do not treat this simulation difference as a market result. Without authorization, real-market work remains blocked.

19. Further Reading

- **Must know:** Jolliffe and Cadima (2016), Section 2: choosing between covariance and correlation-matrix PCA and interpreting explained variance. Relevant to Sections 5–6.
- **Should know:** Laloux, Cizeau, Bouchaud, and Potters (1999): using random-matrix bounds for financial correlation matrices and assessing whether sample eigenvalues exceed noise. Hamilton (1989), Sections 3–4: Markov-switching models and `filtered/smoothed` probabilities.
- **Advanced:** Ang and Timmermann (2012): a review of regime switching in asset pricing; read Sections 2 and 4. Connor and Korajczyk (1986): statistical factor models based on asymptotic principal components. von Luxburg (2010): theory and limitations of clustering stability.
- **Optional:** scikit-learn 1.9.1 documentation for PCA, `KMeans`, and `adjusted_rand_score`, to check implementation details.

20. References

Reading statuses below refer to the original course author's historical review recorded in source_notes; BIL19 inherits those statuses without reopening publications or performing new online verification.

Full links and reading locations are in `references.md`, `references/sources.json`, and this chapter's `source_notes.md`.

- NUMPY: NumPy documentation (linalg.eigh). Used for eigendecomposition in Sections 6.1 and 8.1. Reading status: based on the local 2.5.3 implementation.
- SK_PITFALLS: scikit-learn "Common pitfalls" (data leakage and training-only preprocessing). Used for scaling in Section 5 and Section 12.1. Reading status: carried forward from A8-A.
- CH41_JC2016: Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: a review and recent developments. *Phil. Trans. R. Soc. A*, 374, 20150202. Used in Section 5 and Section 6.2 on scaling. Reading status: the AMC reconnaissance report covered Section 2(c); the author did not reread it.
- CH41_LCBP1999: Laloux, L., Cizeau, P., Bouchaud, J.-P., & Potters, M. (1999). Noise dressing of financial correlation matrices. *Physical Review Letters*, 83, 1467–1470. Used for the bound and assumptions in Section 6.3 and for Section 11. Reading status: see `source_notes.md`.
- CH41_HAMILTON1989: Hamilton, J. D. (1989). A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2), 357–384. Used for the HMM in Section 5 and filtering and smoothing in Section 6.6. Reading status: the reconnaissance report (R); the author did not read the full text.
- CH41_AT2012: Ang, A., & Timmermann, A. (2012). Regime changes and financial markets. *Annual Review of Financial Economics*, 4, 313–337. Used in Section 11 and Section 19. Reading status: reconnaissance report (R); the author did not read the full text.

- CH41_CK1986: Connor, G., & Korajczyk, R. A. (1986). Performance measurement with the arbitrage pricing theory. *Journal of Financial Economics*, 15(3), 373–394. Used in Section 19. Reading status: abstract only.
- CH41_VL2010: von Luxburg, U. (2010). Clustering stability: an overview. *Foundations and Trends in Machine Learning*, 2(3), 235–274. Used for cluster stability in Section 5 and Section 19. Reading status: reconnaissance report (R); the author did not read the full text.
- SK_KMEANS_DOC: scikit-learn 1.9.1 `KMeans` and `adjusted_rand_score` (documentation in local source). Used for k-means and ARI in Section 5. Reading status: checked by the author against local source documentation.

Specific passages read, reader, and which sources were consulted only at abstract level are recorded in `course/zh/chapters/ch41/source_notes.md`.

Notebook Reader Entry

The I3 Chapter 41 notebook has 12 cells, including 6 code cells. It freshly calculates the hand example, four PCA combinations, variance versus prediction, a 3,000-day state path, and the D11 post hoc diagnostic over seeds 4200–4249; it writes two new figures to a new `RUN_DIR` . It is not a frozen replay. This diagnostic is not promoted to a preregistered primary result. The notebook's bilingual projection is handled by BIL23; this batch did not run or modify the notebook.

Chapter 42: Deep Learning Foundations and Quantitative Applications

Author: Yuxiao Ma. This chapter uses frozen constructed results; no training was repeated. Main thread: The essence of a network architecture is a structural inductive bias on function space. In low-signal-to-noise financial time series, adding parameters does not automatically create a trading advantage. Strict boundary purging and a trading ledger with explicit fee assumptions are central to evaluating whether a complex deep model is useful. Prerequisites: the chain rule in Chapter 4; matrix dimensions in Chapter 5; Ridge regression in Chapter 8 and Chapter 39; the execution ledger in Chapter 29; time-causal validation in Chapter 31 and Chapter 38; and early stopping in Chapter 40. By the end, you should be able to calculate and implement MLP forward and backward propagation in pure NumPy; implement CNN local cross-correlation, RNN state recursion and gradient products, the modern LSTM constant-error carousel and Transformer strict causal masking; check the time boundaries for features, labels, and preprocessing before comparing models; and evaluate the performance of constructed signals under assumed fees.

---

1. Learning Objectives

After completing this chapter, readers should be able to:

1. **Compute**:
   - Given an input vector and weight matrices, calculate by hand the forward-pass output and intermediate activations of a multilayer perceptron (MLP) using matrix algebra;
   - Derive the analytical gradients of a mean squared error loss with L_2 regularization with respect to every layer's weights and biases, using the multivariate chain rule, and check the numerical gradients with central differences;
   - Independently calculate 1D convolutional neural network (CNN) valid cross-correlation, simple recurrent neural network (RNN) state recursion and cross-time gradient attenuation products, a modern long short-term memory network (LSTM with a forget gate) one-step gate, candidate, and state updates and its constant error carousel (CEC) direct partial derivative, as well as Transformer sinusoidal positional encoding, scaled dot-product attention, and a strict causal mask.
2. **Explain**:
   - Describe the structural inductive bias built into each deep-learning architecture and the geometric constraints it places on the hypothesis space;
   - Explain how low signal-to-noise ratio (SNR) and a small effective number of independent samples in financial asset-pricing data constrain high-capacity neural networks, and analyze overfitting through the bias-variance decomposition on the training-set distribution for fixed inputs;

- Explain that a causal attention mask can block look-ahead within a layer but cannot repair label leakage or look-ahead already present in the input features.

3. **Diagnose**:
   - Identify overfitting, vanishing gradients, and numerical overflow during deep-model training;
   - Diagnose test-set contamination in early stopping and ensure that best-epoch restoration is based strictly on an independent validation set;
   - Check for look-ahead leakage in feature standardization and ensure that means and variances are calculated using only mature training data.
4. **Implement**:
   - Write a lightweight MLP trainer and predictor in pure NumPy, supporting mini-batch stochastic gradient descent (SGD), weight decay, and validation-set early stopping;
   - Build a panel-data pipeline that follows a strict causal-time contract and purges boundary labels based on their settlement times;
   - Connect the model to a standard quantitative execution ledger and evaluate gross and net portfolio performance with one-way costs of 0 and 10 bps and an execution lag.

---

2. Why Quant Needs It

Across the life cycle of quantitative investment and research, researchers often explore moving from traditional linear factor models, such as Fama-French multifactor models or elastic-net regression, to complex nonlinear models. In quantitative finance, however, directly applying deep-learning paradigms from computer vision (CV) or natural language processing (NLP) often presents serious challenges:

1. **Structural inductive bias and overfitting**: A fully connected neural network (MLP), as a universal function approximator, has very high expressive capacity. Yet single-period returns in cross-sectional prediction usually have a low signal-to-noise ratio; much of their variation comes from unpredictable market microstructure disturbances or idiosyncratic shocks. Fitting dozens or even thousands of parameters directly to limited historical data makes a network prone to mistaking in-sample random noise for a pattern, which can push test-set R^2 down or even below zero.
2. **Prior structural constraints in specific architectures**: To curb the growth of degrees of freedom, modern deep learning uses architecture-specific inductive biases:
   - **CNNs** impose a prior of local-pattern translation equivariance through local connectivity and weight sharing;
   - **RNNs/LSTMs** impose a prior of Markov-style hidden-state evolution and controllable memory decay through autoregressive state transitions and gating;
   - **Transformers** use self-attention to overcome limits imposed by temporal distance and strict causal masking to prevent future information from flowing into the past. Quantitative researchers need to understand why these architectures were introduced, which structural assumptions each one makes, and when those assumptions are appropriate for financial time series.

3. **Strict time contracts and false excess-return traps**: Deep-model training can introduce look-ahead leakage inadvertently. Examples include failing to purge boundary samples for multi-period return labels, standardizing on the full sample, or assuming that a Transformer's causal attention mask automatically corrects future information in input features. High-complexity deep models can also produce sharply fluctuating cross-sectional predictions, resulting in very high rebalancing turnover. Gross Sharpe before fees may fall substantially once it is run through a real execution ledger with trading frictions.

This chapter explains the main operators through mathematical derivations and pure NumPy implementations, then compares the mean, Ridge, and a small MLP in a constructed low-SNR experiment with a fixed time split. The results describe the prediction and fee trade-off in this experiment; they cannot be extrapolated to real-market performance.

---

3. Intuition

3.1 Linear Models vs. Neural Networks: A Straightedge and Multidimensional Modeling Clay

A linear model such as OLS or Ridge is like a rigid wooden ruler: it can fit only a straight cross-sectional line between features and returns. If the true economic mechanism includes a nonlinear threshold effect—for example, momentum is strong when volatility is very low but decays sharply once volatility exceeds a threshold—the linear ruler cannot represent that bend.

An MLP is like a lump of multidimensional modeling clay. Each hidden-layer neuron represents an affine transformation with a bias, followed by a bend from a nonlinear activation such as hyperbolic tangent tanh or a rectified linear unit (ReLU). Composing the mappings across layers lets the network fold and partition feature space into complex nonlinear decision manifolds. But the more pliable the clay, the more easily random ripples on the water (noise) can pull it off course. In a low-SNR environment, weight decay (L_2 regularization) and early stopping add "stiffness" to the clay and help keep it from fitting in-sample noise too closely.

3.2 Why Specialized Structures Are Needed: Intuition for CNNs, RNNs, and Transformers

Flattening the past T days of prices into one vector and passing it directly to a fully connected network (MLP) has important shortcomings when modeling asset-price time series:

1. **Parameter growth and disregard for temporal locality**: A fully connected network gives the relationship between day $t-1$ and day t the same degrees of freedom as the relationship between day $t-100$ and day t. It ignores local translation structure in time series. The intuition behind a **CNN** is to slide a fixed-size small window (a convolution kernel) along the time axis to extract local patterns such as "short-cycle momentum" or "difference-based reversal," reusing the same kernel weights at every historical time.

2. **Remembering and forgetting long histories**: Financial states, such as volatility regimes, often have time-varying dependencies. The intuition behind an **RNN** is to maintain a "memory carrier," the hidden state h_t, and update it by mixing old memory with new input each day. Multiplying state Jacobians in a standard RNN can cause gradients to decay or explode exponentially during backpropagation. The intuition behind an **LSTM** is to introduce an additive channel controlled by gates—the "constant error carousel" (CEC)—so important information can pass across longer intervals when the forget gate is open.
3. **Global view and causal masking**: Although an LSTM mitigates vanishing gradients, its step-by-step temporal recursion limits direct associations across long time spans. The intuition behind a **Transformer** is to discard recurrence and directly calculate the association between any two days using Query-Key dot-product similarity. But quantitative prediction has a strict time constraint: **today cannot read tomorrow's information**. For this chapter's task, which predicts at each historical position using information available at that position, a Transformer therefore applies a strict causal mask to the upper triangle of the attention matrix ($\mathbf{M}_{i,j} = -\infty$ when $j > i$), ensuring that each position attends only to itself and the past. If the entire input window is available before one common forecast cutoff and only its final representation predicts a target after that cutoff, bidirectional encoding within the window does not automatically leak future information. Availability times, training/label boundaries, and preprocessing must still be checked.

---

4. Mathematical Foundations

4.1 Notation and Tensor-Dimension Contract

Before deriving the network, define each mathematical symbol and tensor dimension precisely. Let the batch contain B samples:

- Input feature matrix $\mathbf{X} \in \mathbb{R}^{B \times D_{\text{in}}}$: each row is one sample (an asset's feature vector on a particular date), and each column is one input feature;
- Target-label matrix $\mathbf{Y} \in \mathbb{R}^{B \times D_{\text{out}}}$: for a single-target return forecast, $D_{\text{out}} = 1$;
- First-layer weight matrix $\mathbf{W}_1 \in \mathbb{R}^{D_{\text{in}} \times D_{\text{hidden}}}$ and bias row vector $\mathbf{b}_1 \in \mathbb{R}^{1 \times D_{\text{hidden}}}$;
- Hidden-layer pre-activation $\mathbf{Z}_1 \in \mathbb{R}^{B \times D_{\text{hidden}}}$ and activation output $\mathbf{A}_1 \in \mathbb{R}^{B \times D_{\text{hidden}}}$;
- Second-layer weight matrix $\mathbf{W}_2 \in \mathbb{R}^{D_{\text{hidden}} \times D_{\text{out}}}$ and bias row vector $\mathbf{b}_2 \in \mathbb{R}^{1 \times D_{\text{out}}}$;
- Final network prediction $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times D_{\text{out}}}$.

4.2 Matrix Differentiation and Elementwise Operations

1. **Hadamard product (elementwise product)**: Denoted by $\odot$. For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{M \times N}$, $(\mathbf{A} \odot \mathbf{B})_{i,j} = A_{i,j} B_{i,j}$.
2. **Frobenius norm**: The squared Frobenius norm of matrix $\mathbf{W}$ is the sum of the squares of its entries: $\|\mathbf{W}\|_F^2 = \operatorname{tr}(\mathbf{W}^\top \mathbf{W}) = \sum_{i,j} W_{i,j}^2$.
3. **Hyperbolic tangent and its derivative**:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = \frac{1 - e^{-2z}}{1 + e^{-2z}}$$

   Its first derivative satisfies the analytic identity:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z)$$

4. **Logistic sigmoid and its derivative**:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

   Its first derivative is:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

---

5. Formal Definitions

5.1 Multilayer Perceptron (MLP)

Define the mathematical mapping of a single-hidden-layer regression MLP, $f : \mathbb{R}^{B \times D_{\text{in}}} \to \mathbb{R}^{B \times D_{\text{out}}}$, as follows:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1$$

$$\mathbf{A}_1 = \phi(\mathbf{Z}_1)$$

$$\hat{\mathbf{Y}} = \mathbf{A}_1 \mathbf{W}_2 + \mathbf{b}_2$$

Here, $\phi(\cdot)$ is a nonlinear activation function; this chapter uses the hyperbolic tangent tanh. In matrix addition, the bias vector is broadcast to every row.

5.2 Regularized Objective (MSE + L2 Regularization)

Given target labels $\mathbf{Y}$ and regularization strength $\lambda \geq 0$, define the batch loss as the sum of normalized mean squared error (MSE) and Frobenius-norm penalties on the weight matrices in all layers:

$$\begin{aligned}&\mathcal{L}(\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2)\\&= \frac{1}{B \cdot D_{\text{out}}} \|\hat{\mathbf{Y}} - \mathbf{Y}\|_F^2\\&+ \lambda \left(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2\right)\end{aligned}$$

Design notes:

1. The MSE term is normalized by $\frac{1}{B \cdot D_{\text{out}}}$ so the scalar loss remains on a consistent scale across different numbers of output dimensions (when $D_{\text{out}} = 1$, this is exactly equivalent to $\frac{1}{B}$).
2. This chapter does not apply an L_2 penalty to the bias vectors $\mathbf{b}_1, \mathbf{b}_2$, to avoid directly pulling the baseline output toward zero. This is a modeling choice; other tasks may penalize biases. Biases also affect local derivatives by changing activation locations.

5.3 One-Dimensional CNN Valid Cross-Correlation

Let the input $\mathbf{x}$ be a vector of length N and the filter $\mathbf{w}$ be a vector of length K, with $N \geq K$:

$$\mathbf{x} = [x_0, \ldots, x_{N-1}]^\top, \qquad \mathbf{w} = [w_0, \ldots, w_{K-1}]^\top.$$

In valid mode (without edge padding), the one-dimensional cross-correlation operator produces a feature map $\mathbf{y} = \mathbf{x} \star \mathbf{w}$ of length $N - K + 1$:

$$y_i = (\mathbf{x} \star \mathbf{w})_i = \sum_{j=0}^{K-1} x_{i+j} w_j, \quad \forall i \in \{0, 1, \ldots, N - K\}$$

Translation equivariance: If a time shift is applied to the input sequence, $T_\tau(\mathbf{x})_t = x_{t-\tau}$, cross-correlation satisfies $(T_\tau(\mathbf{x}) \star \mathbf{w})_i = (\mathbf{x} \star \mathbf{w})_{i-\tau}$; in other words, the operation commutes with translation: $f(g(x)) = g(f(x))$. The convolution operation itself provides **translation equivariance**, not translation invariance. The stated relation holds for an infinite sequence or under a consistent boundary definition; a finite valid window satisfies it only at common valid positions, and stride and padding also affect the result. Global pooling can provide invariance when translation only reorders the same set of features; after cropping or edge changes, the result still needs to be checked.

5.4 Recurrent Neural Networks and Modern Long Short-Term Memory (RNN & Modern LSTM)

1. **Simple recurrent neural network (Simple RNN)**:

$$h_t = \tanh(w_h h_{t-1} + w_x x_t + b)$$

2. **Modern long short-term memory with a forget gate (Modern LSTM with Forget Gate, Gers et al. 2000)**: For input $x_t \in \mathbb{R}^{D_{\text{in}}}$, previous hidden state $h_{t-1} \in \mathbb{R}^{D_h}$, and previous cell state $c_{t-1} \in \mathbb{R}^{D_h}$ at time t:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f x_t + \mathbf{U}_f h_{t-1} + \mathbf{b}_f) \quad \text{(forget gate)}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i x_t + \mathbf{U}_i h_{t-1} + \mathbf{b}_i) \quad \text{(input gate)}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c x_t + \mathbf{U}_c h_{t-1} + \mathbf{b}_c) \quad \text{(candidate cell state)}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad \text{(cell-state update)}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o x_t + \mathbf{U}_o h_{t-1} + \mathbf{b}_o) \quad \text{(output gate)}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad \text{(hidden-state output)}$$

The main text writes LSTM states as column vectors: $\mathbf{W}_g \in \mathbb{R}^{D_h \times D_{\text{in}}}$ and $\mathbf{U}_g \in \mathbb{R}^{D_h \times D_h}$. The code uses batch row vectors $x@W_g + h@U_g$, so its code weights are the transposes of the mathematical weights shown here; the two conventions agree in the scalar worked example. The six equations follow PyTorch's official modern LSTM equations; the Gers paper is cited only for the historical origin of the forget gate.

5.5 Transformer Sinusoidal Positional Encoding and Strict Causal Attention (PE & Causal Attention)

The triangular mask here serves per-position real-time prediction. A final-position forecast from a window already known at one cutoff may use another encoding structure, provided it respects that task's information set.

1. **Sinusoidal positional encoding (Vaswani et al. 2017)**: For position $pos \in \{0, 1, \ldots, T-1\}$ and channel indices $2i, 2i+1 \in \{0, 1, \ldots, d-1\}$:

$$\text{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$\text{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

2. **Causal scaled dot-product attention**: Let the sequence length be T, with feature projection matrices $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{T \times d_k}$ and $\mathbf{V} \in \mathbb{R}^{T \times d_v}$ (requiring $d_k > 0$, $d_v > 0$, and $T > 0$). The attention output is:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{M}\right)\mathbf{V}$$

The causal-mask matrix $\mathbf{M} \in \mathbb{R}^{T\times T}$ is defined by:

$$\mathbf{M}_{i,j} = \begin{cases} 0, & j \le i \\ -\infty, & j > i \end{cases}$$

The upper triangle is set strictly to $-\infty$, so after softmax normalization the upper-triangular attention weights are exactly zero in numerical computation.

---

6. Derivations

Complete matrix backpropagation: six steps and conditions

6.1 Full Matrix Chain-Rule Derivation of MLP Backpropagation Gradients

This section derives, without skipped steps, the partial derivatives of the scalar loss $\mathcal{L}$ with respect to every parameter.

Step 1: Differentiate the loss with respect to the final predictions $\hat{\mathbf{Y}}$. Expand the loss function:

$$\mathcal{L} = \frac{1}{B \cdot D_{\text{out}}} \sum_{i=1}^{B} \sum_{k=1}^{D_{\text{out}}} (\hat{Y}_{i,k} - Y_{i,k})^2 + \lambda \left(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2 \right)$$

Differentiate with respect to matrix element $\hat{Y}_{i,k}$:

$$\frac{\partial \mathcal{L}}{\partial \hat{Y}_{i,k}} = \frac{2}{B \cdot D_{\text{out}}} (\hat{Y}_{i,k} - Y_{i,k})$$

In matrix form, define the output error term $\boldsymbol{\delta}_2 \in \mathbb{R}^{B\times D_{\text{out}}}$:

$$\boldsymbol{\delta}_2 = \frac{\partial \mathcal{L}}{\partial \hat{\mathbf{Y}}} = \frac{2}{B \cdot D_{\text{out}}} (\hat{\mathbf{Y}} - \mathbf{Y})$$

Step 2: Differentiate with respect to second-layer parameters $\mathbf{W}_2$ and $\mathbf{b}_2$. By the chain rule, $\hat{\mathbf{Y}} = \mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2$. For weight entry $W_{2,j,k}$:

$$\frac{\partial \mathcal{L}}{\partial W_{2,j,k}} = \sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial \hat{Y}_{i,k}} \frac{\partial \hat{Y}_{i,k}}{\partial W_{2,j,k}} + 2\lambda W_{2,j,k} = \sum_{i=1}^{B} A_{1,i,j}\delta_{2,i,k} + 2\lambda W_{2,j,k}$$

The sum on the right is the matrix product $(\mathbf{A}_1^\top \boldsymbol{\delta}_2)_{j,k}$. Therefore, the full matrix gradient is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{A}_1^\top \boldsymbol{\delta}_2 + 2\lambda \mathbf{W}_2 \in \mathbb{R}^{D_{\text{hidden}}\times D_{\text{out}}}$$

Continued: the same complete example

For the bias term $\mathbf{b}_2 \in \mathbb{R}^{1\times D_{\text{out}}}$, the derivative of the bias for each sample is 1:

$$\frac{\partial \mathcal{L}}{\partial b_{2,k}} = \sum_{i=1}^{B} \delta_{2,i,k} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \mathbf{1}_{1\times B}\boldsymbol{\delta}_2 \in \mathbb{R}^{1\times D_{\text{out}}}$$

Step 3: Backpropagate the error to hidden-layer activations $\mathbf{A}_1$. Differentiate with respect to hidden activation entry $A_{1,i,j}$:

$$\frac{\partial \mathcal{L}}{\partial A_{1,i,j}} = \sum_{k=1}^{D_{\text{out}}} \frac{\partial \mathcal{L}}{\partial \hat{Y}_{i,k}} \frac{\partial \hat{Y}_{i,k}}{\partial A_{1,i,j}} = \sum_{k=1}^{D_{\text{out}}} \delta_{2,i,k} W_{2,j,k}$$

The sum on the right corresponds to matrix product $(\boldsymbol{\delta}_2 \mathbf{W}_2^\top)_{i,j}$. Thus:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{A}_1} = \boldsymbol{\delta}_2 \mathbf{W}_2^\top \in \mathbb{R}^{B\times D_{\text{hidden}}}$$

Step 4: Differentiate with respect to hidden-layer pre-activations $\mathbf{Z}_1$. Because the activation $\mathbf{A}_1 = \tanh(\mathbf{Z}_1)$ is applied elementwise, use the scalar derivative identity $\tanh'(z) = 1 - \tanh^2(z)$:

$$\frac{\partial \mathcal{L}}{\partial Z_{1,i,j}} = \frac{\partial \mathcal{L}}{\partial A_{1,i,j}} \cdot \tanh'(Z_{1,i,j}) = (\boldsymbol{\delta}_2 \mathbf{W}_2^\top)_{i,j} \cdot (1 - A_{1,i,j}^2)$$

Write this as a Hadamard product and define the hidden-layer local error $\boldsymbol{\delta}_1 \in \mathbb{R}^{B\times D_{\text{hidden}}}$:

$$\boldsymbol{\delta}_1 = \frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = (\boldsymbol{\delta}_2 \mathbf{W}_2^\top) \odot (1 - \mathbf{A}_1^2)$$

Step 5: Differentiate with respect to first-layer parameters $\mathbf{W}_1$ and $\mathbf{b}_1$. Since $\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1$, the derivation is the same as in Step 2:

$$\frac{\partial \mathcal{L}}{\partial W_{1,m,j}} = \sum_{i=1}^{B} X_{i,m}\delta_{1,i,j} + 2\lambda W_{1,m,j} \implies \frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \mathbf{X}^\top \boldsymbol{\delta}_1 + 2\lambda \mathbf{W}_1 \in \mathbb{R}^{D_{\text{in}}\times D_{\text{hidden}}}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \mathbf{1}_{1\times B}\boldsymbol{\delta}_1 \in \mathbb{R}^{1\times D_{\text{hidden}}}$$

Step 6: Equivalence of L_2 regularization and weight decay under ordinary SGD. Under a standard SGD optimizer, the weight update is:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{W}^{(t)} - \eta \left(\nabla_{\mathbf{W}} \mathcal{L}_{\text{MSE}} + 2\lambda \mathbf{W}^{(t)} \right) = (1 - 2\eta\lambda)\mathbf{W}^{(t)} - \eta \nabla_{\mathbf{W}} \mathcal{L}_{\text{MSE}}$$

Continued: the same complete example

The factor $(1 - 2\eta\lambda)$ reduces parameter magnitude. The algebraic identity does not itself require an inequality. For the decay factor to be positive and less than 1, however, we need $0 < 2\eta\lambda < 1$. Note that this strict equivalence holds only for ordinary SGD. With momentum or adaptive learning rates such as Adam, the L_2 regularization gradient is altered by smoothed gradient history and differs substantially from decoupled weight decay (AdamW).

6.2 Simple RNN State Recursion and Gradient-Product Attenuation

Consider the scalar Simple RNN state equation $h_t = \phi(w_h h_{t-1} + w_x x_t + b)$. The partial derivative of the loss at time T with respect to an earlier hidden state h_{T-k} is:

$$\frac{\partial \mathcal{L}_T}{\partial h_{T-k}} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{j=T-k+1}^{T} \frac{\partial h_j}{\partial h_{j-1}}$$

The one-step Jacobian is:

$$\frac{\partial h_j}{\partial h_{j-1}} = w_h \cdot \phi'(w_h h_{j-1} + w_x x_j + b)$$

If we use the linear activation $\phi(z) = z$, then $\frac{\partial h_j}{\partial h_{j-1}} = w_h$ and the gradient transfer factor reduces to a geometric sequence:

$$\frac{\partial h_T}{\partial h_{T-k}} = (w_h)^k$$

- If $|w_h| < 1$ (for example, $w_h = 0.5$), then across $k = 10$ steps the attenuation factor is $0.5^{10} \approx 9.76 \times 10^{-4}$, and the gradient of early input information approaches 0 during backpropagation (vanishing gradients).
- If $|w_h| > 1$, the product grows exponentially and can lead to exploding gradients.

6.3 Deriving the Modern LSTM Constant Error Carousel (CEC) Partial Derivative

Consider the modern LSTM cell-state transition:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Fix the gates and candidate state, and analyze the partial derivative along the direct linear path of cell memory. The Jacobian of the vector state is a diagonal matrix:

$$\left.\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}}\right|_{\text{fixed gates}} = \operatorname{diag}(\mathbf{f}_t).$$

The direct partial derivative of coordinate r is $f_{t,r}$; across k steps, the direct Jacobian is

$$\left.\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_{T-k}}\right|_{\text{fixed gates}} = \operatorname{diag}\left(\bigodot_{j=T-k+1}^{T} \mathbf{f}_j \right).$$

Only in the scalar case can this be abbreviated as $\partial c_t/\partial c_{t-1} = f_t$, with the multi-step product $\prod_{j=T-k+1}^{T} f_j$. If the forget gate is saturated in its open state ($f_j \approx 1$), the error can travel across a longer sequence through the additive channel. This is called the **constant error carousel (CEC)**. *Boundary note*: the total derivative also includes nonlinear gate-dependent paths through h_{t-1} (such as $\frac{\partial f_t}{\partial h_{t-1}}$ and $\frac{\partial i_t}{\partial h_{t-1}}$). When the network is deep or the forget gate is not saturated, gradients can still decay or oscillate. We cannot claim that an LSTM "eliminates all gradient problems."

6.4 Complete Workflow for Batches, One Optimization Step, and Early Stopping

One sample is a row of features and a mature label for one asset on one signal date; a mini-batch is a set of training rows selected for the current gradient step. One epoch passes through every training row. In this example, 2,136 rows are divided into batches of 32, giving 67 updates per epoch and a final batch of 24 rows; the loss is still divided by the actual number of rows in that batch. Rows are shuffled only within the 178 mature training dates, never across validation or test. This computational arrangement does not imply that real financial samples are independent.

Let the parameter set be $\theta = (W_1, b_1, W_2, b_2)$ and let $L_{\mathcal{B}}$ be the average data loss for batch $\mathcal{B}$. At each step, first calculate the forward pass, then calculate the full-parameter gradients from §6.1 and update $\theta \leftarrow \theta - \eta\nabla_\theta(L_{\mathcal{B}} + \lambda \sum W^2)$; the penalty applies only to weights. Full-batch gradient descent uses all training rows. Stochastic gradient descent (SGD) often refers to a one-row or mini-batch estimate. Mini-batch noise, computational cost, and stability all need to be considered together; a lower training loss does not imply better future predictions.

For example, §8.1 uses a learning rate of $\eta = 0.01$: $W_{1,00}$ changes from 0.2 to $0.2 - 0.01(0.019587) = 0.19980413$; b_2 changes from 0.05 to $0.05 - 0.01(0.063251) = 0.04936749$. This hand calculation uses $\lambda = 0.01$; the main experiment remains frozen at 0.001. Do not mix them.

```
Initialize weights; best_loss=+∞; bad_epochs=0; best_weights not yet set
For epoch=0,...,199:
    Shuffle training rows only; calculate MSE+L2 gradients in batches and update all
    ↪ parameters
    Calculate unpenalized MSE on the full validation set; reject non-finite results
    If val_mse < best_loss: copy all weights and biases, update best_epoch, set
    ↪ bad_epochs=0
    Otherwise increment bad_epochs; stop if bad_epochs>=15
Return a copy of best_weights, not the current parameters at stopping
Only after model selection is complete, predict from test features and evaluate in
    ↪ the original return units
```

The main experiment uses zero-based epoch indices: best_epoch=5 is the sixth pass, and stopped_epoch=20 means 21 passes were actually completed. Early stopping counts "15 consecutive epochs without a new low"; it does not require validation error to rise strictly each epoch. Validation-error fluctuations can also trigger stopping. The training API accepts only training and validation arrays, but the caller still has to ensure that test arrays were not passed under another name; a parameter name cannot prove that the whole study is leakage-free.

7. Financial Interpretation

7.1 Why Financial Time-Series SNR Constrains Model Complexity

In computer vision or speech recognition, inputs contain rich local redundancy and high SNR. In secondary-market asset pricing:

1. **Signal-to-noise characteristics**: Researchers should test whether the predictive signal is strong relative to noise. This chapter deliberately constructs a weak-signal, relatively high-noise case; it does not establish a universal SNR for all markets or frequencies.
2. **Limits on the effective number of independent samples**: Although high-frequency tick data are densely recorded, daily or weekly cross-sectional prediction provides only about 250 trading days a year. Ten years of history still gives only about 2,500 broad market cross-sections. A nonlinear network with many parameters can mistake in-sample noise for signal.
3. **Bias-variance trade-off**: For fixed input x, the generalization error averaged over the training-set distribution $\mathcal{D}$ decomposes as:

$$\mathbb{E}_{\mathcal{D},\epsilon}[(\hat{y}(x) - y)^2] = \text{Bias}^2(\hat{y}(x)) + \text{Var}_{\mathcal{D}}(\hat{y}(x)) + \sigma_\epsilon^2$$

Let $y = f_*(x) + \epsilon$, where new-observation noise is independent of the training set, has conditional mean 0, and conditional variance $\sigma_\epsilon^2(x)$. The left side must take expectations over both the training set and the new-observation noise. Write $\hat{y} - f_*$ as $\hat{y} - \mathbb{E}_{\mathcal{D}}[\hat{y}] + \mathbb{E}_{\mathcal{D}}[\hat{y}] - f_*$; the cross term has expectation 0, giving the decomposition above. Increasing capacity may reduce approximation bias, but it may also change estimation variance. There is no general monotonic theorem connecting parameter count and variance; regularization, the training algorithm, and the data distribution also matter.

7.2 The Cost of Trading Frictions for High-Turnover Signals

A nonlinear model may be sensitive to small input perturbations, causing the cross-sectional rank of daily predictions to change frequently:

- Frequent changes can result in high rebalancing frequency. If annualized turnover reaches 350+ times;
- Under a one-way trading-cost assumption of 10 bps (0.001), the annualized cost is on the order of:

$$\text{Annual Fee Drag} \approx \text{Turnover} \times \text{Fee Rate} \approx 360 \times 0.001 = 36.0\%$$

- This cost drag can substantially erode gross excess returns before fees and turn net Sharpe deeply negative. In actual investing, capital dynamics must be tracked in a daily execution ledger; static linear deductions cannot substitute for actual net returns.

Chapter 42 complete numerical example: all steps and conclusions

8. Numerical Example

This section gives small examples that can be recalculated by hand, covering MLP forward and backward propagation, CNN cross-correlation, RNN state recursion, LSTM one-step gating, sinusoidal positional encoding, and Transformer causal attention.

8.1 Hand Calculation of Forward and Backward Propagation for a 2→2→1 MLP

Take input feature vector $x = [0.5, -0.2]$, target $y = [0.1]$ ($B = 1$, $D_{\text{out}} = 1$). Set the network hyperparameters to:

$$\mathbf{W}_1 = \begin{pmatrix} 0.2 & -0.1 \\ 0.4 & 0.3 \end{pmatrix}, \quad \mathbf{b}_1 = [0.1, 0.0]$$

$$\mathbf{W}_2 = \begin{pmatrix} 0.5 \\ -0.2 \end{pmatrix}, \quad \mathbf{b}_2 = [0.05], \quad \lambda = 0.01$$

Step 1: Forward propagation

1. Hidden-layer pre-activation:

$$\mathbf{z}_1 = x\mathbf{W}_1 + \mathbf{b}_1 = [0.5(0.2) + (-0.2)(0.4) + 0.1,\ 0.5(-0.1) + (-0.2)(0.3) + 0.0]$$
$$= [0.12, -0.11]$$

2. Activations (hyperbolic tangent):

$$a_{1,1} = \tanh(0.12) \approx 0.119427, \quad a_{1,2} = \tanh(-0.11) \approx -0.109559$$

3. Output-layer calculation:

$$\hat{y} = a_{1,1}(0.5) + a_{1,2}(-0.2) + 0.05 = 0.119427(0.5) + (-0.109559)(-0.2) + 0.05 \approx 0.131625$$

4. Loss calculation:

$$\text{MSE} = \frac{1}{1 \times 1}(\hat{y} - y)^2 = (0.131625 - 0.1)^2 = (0.031625)^2 \approx 0.00100014$$

$$L_2 \text{ penalty} = 0.01 \times \left((0.2^2 + (-0.1)^2 + 0.4^2 + 0.3^2) + (0.5^2 + (-0.2)^2)\right)$$
$$= 0.01 \times (0.30 + 0.29) = 0.0059$$
$$\mathcal{L} \approx 0.00100014 + 0.005900 = 0.00690014$$

Step 2: Backpropagation gradients

1. Output error: $\delta_2 = \frac{2}{1 \times 1}(\hat{y} - y) = 2 \times 0.031625 = 0.063251$.
2. Second-layer gradients:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \mathbf{a}_1^\top \delta_2$$
$$+ 2\lambda \mathbf{W}_2 = \begin{pmatrix} 0.119427 \\ -0.109559 \end{pmatrix} (0.063251)$$
$$+ 0.02 \begin{pmatrix} 0.5 \\ -0.2 \end{pmatrix} = \begin{pmatrix} 0.017554 \\ -0.010930 \end{pmatrix}$$
$$\frac{\partial \mathcal{L}}{\partial b_2} = \delta_2 = 0.063251$$

Continued: the same complete example

3. Hidden-layer local error:

$$\begin{aligned}\boldsymbol{\delta}_1 &= (\delta_2 \mathbf{W}_2^\top) \odot (1 - \mathbf{a}_1^2) \\ &= [0.063251 \times 0.5,\ 0.063251 \times (-0.2)] \odot [1 - 0.119427^2,\ 1 - (-0.109559)^2] \\ &= [0.031625, -0.012650] \odot [0.985737, 0.987997] = [0.031174, -0.012498]\end{aligned}$$

4. First-layer gradients:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} &= x^\top \boldsymbol{\delta}_1 \\ &+ 2\lambda \mathbf{W}_1 = \begin{pmatrix} 0.5 \\ -0.2 \end{pmatrix} \begin{pmatrix} 0.031174 & -0.012498 \end{pmatrix} \\ &+ 0.02 \begin{pmatrix} 0.2 & -0.1 \\ 0.4 & 0.3 \end{pmatrix} \\ &= \begin{pmatrix} 0.019587 & -0.008249 \\ 0.001765 & 0.008500 \end{pmatrix} \\ \frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} &= \boldsymbol{\delta}_1 = [0.031174, -0.012498]\end{aligned}$$

The analytical derivatives match the central-difference numerical check in `tests/test_deep_learning.py` to a precision of 10^{-10}.

8.2 Hand Calculation of One-Dimensional CNN Valid Cross-Correlation

Input sequence $x = [1, 2, 3, 4]$ and filter kernel $w = [1, -1]$ (length $K = 2$).

- $i = 0$: $x[0]w[0] + x[1]w[1] = 1(1) + 2(-1) = -1$;
- $i = 1$: $x[1]w[0] + x[2]w[1] = 2(1) + 3(-1) = -1$;
- $i = 2$: $x[2]w[0] + x[3]w[1] = 3(1) + 4(-1) = -1$.

The feature map is exactly $[-1, -1, -1]$. This operation extracts the local difference of the first item minus the next; the example does not involve learning a convolution kernel.

8.3 Hand Calculation of Simple RNN State Recursion

Set the scalar Simple RNN parameters to $w_h = 0.5$, $w_x = 1.0$, and $b = 0.0$, with initial hidden state $h_{-1} = 0.0$. Let the input sequence be $x = [1.0, 2.0, 0.0]$ at $t = 0, 1, 2$ and use the identity activation $\phi(z) = z$:

- $t = 0$: $h_0 = 0.5(0.0) + 1.0(1.0) + 0.0 = 1.0$;
- $t = 1$: $h_1 = 0.5(1.0) + 1.0(2.0) + 0.0 = 2.5$;
- $t = 2$: $h_2 = 0.5(2.5) + 1.0(0.0) + 0.0 = 1.25$.

The state sequence is $[1.0, 2.5, 1.25]$. Across $k = 3$ steps, the gradient transfer factor is $\frac{\partial h_2}{\partial h_{-1}} = (w_h)^3 = (0.5)^3 = 0.125$.

Continued: the same complete example

8.4 Hand Calculation of a Modern LSTM One-Step Gating Example

Consider a scalar one-step LSTM with dimension 1. The input is $x_t = 0.5$, previous hidden state $h_{t-1} = 0.1$, and previous cell state $c_{t-1} = 0.2$. Set all weight matrices to $W = 1.0$, $U = 0.5$, and all biases to $b = 0.0$:

1. Linear pre-activation: $z = Wx_t + Uh_{t-1} + b = 1.0(0.5) + 0.5(0.1) + 0.0 = 0.55$;
2. Forget gate: $f_t = \sigma(0.55) = \frac{1}{1+e^{-0.55}} \approx 0.634135$;
3. Input gate: $i_t = \sigma(0.55) \approx 0.634135$;
4. Candidate cell state: $\tilde{c}_t = \tanh(0.55) \approx 0.500520$;
5. Cell-state update:
$$\begin{aligned} c_t &= f_t c_{t-1} + i_t \tilde{c}_t \\ &= 0.634135(0.2) + 0.634135(0.500520) \\ &= 0.126827 + 0.317397 \\ &= 0.444224 \end{aligned}$$
6. Output gate: $o_t = \sigma(0.55) \approx 0.634135$;
7. Hidden-state output:

$$h_t = o_t \tanh(c_t) = 0.634135 \times \tanh(0.444224) = 0.634135 \times 0.417143 \approx 0.264523$$

This exactly matches the output of the core `lstm_step` function.

8.5 Hand Calculation of Sinusoidal Positional Encoding (PE)

Let the sequence length be $T = 3$ and dimension $d = 4$. The channel scales are $10000^{0/4} = 1.0$ and $10000^{2/4} = 100.0$.

- $pos = 0$: $\mathrm{PE}_{0,0} = \sin(0) = 0.0$; $\mathrm{PE}_{0,1} = \cos(0) = 1.0$; $\mathrm{PE}_{0,2} = \sin(0) = 0.0$; $\mathrm{PE}_{0,3} = \cos(0) = 1.0$; $\Longrightarrow$ $\mathrm{PE}[0] = [0.0000, 1.0000, 0.0000, 1.0000]$.
- $pos = 1$: $\mathrm{PE}_{1,0} = \sin(1.0) \approx 0.8415$; $\mathrm{PE}_{1,1} = \cos(1.0) \approx 0.5403$; $\mathrm{PE}_{1,2} = \sin(0.01) \approx 0.0100$; $\mathrm{PE}_{1,3} = \cos(0.01) \approx 0.9999$; $\Longrightarrow$ $\mathrm{PE}[1] \approx [0.8415, 0.5403, 0.0100, 0.9999]$.
- $pos = 2$: $\mathrm{PE}_{2,0} = \sin(2.0) \approx 0.9093$; $\mathrm{PE}_{2,1} = \cos(2.0) \approx -0.4161$; $\mathrm{PE}_{2,2} = \sin(0.02) \approx 0.0200$; $\mathrm{PE}_{2,3} = \cos(0.02) \approx 0.9998$; $\Longrightarrow$ $\mathrm{PE}[2] \approx [0.9093, -0.4161, 0.0200, 0.9998]$.

Continued: the same complete example

8.6 Hand Calculation of Transformer Causal Attention and Output O for Three Tokens

Let $T = 3$, $d_k = 2$, and $d_v = 2$:

$$\mathbf{Q} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{V} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$$

1. Dot-product matrix $\mathbf{QK}^\top$:

$$\mathbf{QK}^\top = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 2 & 1 & 1 \end{pmatrix}$$

2. Scaled dot-product scores (divide by $\sqrt{2} \approx 1.414214$):

$$\mathbf{S} = \begin{pmatrix} 0.7071 & 0.7071 & 0.0000 \\ 0.7071 & 0.0000 & 0.7071 \\ 1.4142 & 0.7071 & 0.7071 \end{pmatrix}$$

3. Apply the causal mask (upper triangle set to $-\infty$):

$$\tilde{\mathbf{S}} = \begin{pmatrix} 0.7071 & -\infty & -\infty \\ 0.7071 & 0.0000 & -\infty \\ 1.4142 & 0.7071 & 0.7071 \end{pmatrix}$$

4. Calculate the row-wise softmax weight matrix $\mathbf{A}$:
   - Row 0: only the first entry is unmasked, so $\mathbf{A}[0] = [1.0000, 0.0000, 0.0000]$.
   - Row 1: valid entries are $[0.7071, 0.0000]$. Using 0.7071 as the baseline, $\exp(0) = 1.0$ and $\exp(-0.7071) \approx 0.493069$. The denominator is 1.493069. After normalization, $\mathbf{A}[1] \approx [0.669761, 0.330239, 0.000000]$.
   - Row 2: valid entries are $[1.4142, 0.7071, 0.7071]$. Using 1.4142 as the baseline, $\exp(0) = 1.0$, $\exp(-0.7071) \approx 0.493069$, and $\exp(-0.7071) \approx 0.493069$. The denominator is 1.986138. Divide each of the three valid terms by the denominator:

$$\mathbf{A}[2] \approx [0.503490, 0.248255, 0.248255].$$

5. Calculate the final output matrix $\mathbf{O} = \mathbf{AV}$ using unrounded A. Decimal coefficients below are displayed approximations; round O only at the end:
   - $\mathbf{O}[0] = 1.0 \times [1, 2] = [1.0000, 2.0000]$;
   - $\mathbf{O}[1] = 0.669761[1, 2] + 0.330239[3, 4] = [0.669761 + 0.990717, \ 1.339522 + 1.320956] \approx [1.6605, 2.6605]$;

Continued: the same complete example

- $\mathbf{O}[2] = 0.503490[1, 2] + 0.248255[3, 4] + 0.248255[5, 6] \approx [2.4895, 3.4895]$.

The upper triangle is strictly zero, so future key-value pairs cannot affect historical outputs.

---

9. Python Implementation

The NumPy teaching implementation for this chapter is in `src/quant_course/deep_learning.py`. The complete training path is the MLP: `fit_mlp` updates both layers' weights and biases, stops early based on validation-set MSE, and restores the best checkpoint; `predict` then produces predictions for the isolated test set. The matrix backpropagation in §6.1 and the optimization workflow in §6.4 connect in this path.

The remaining architectures are used to isolate mechanisms: `cnn_cross_correlation` performs one-dimensional valid cross-correlation with a fixed kernel and does not train that kernel; `rnn_states` is a scalar linear recursion corresponding to the identity-activation gradient-product toy form in §6.2, while a general nonlinear RNN also includes activation derivatives; `lstm_step` computes the gates, cell state, and hidden state for one cell step and does not train on a sequence; the Transformer code provides sinusoidal positional encoding, masked softmax, and Q/K/V causal-attention operators, not a full Transformer trainer. The hand calculations in §8 check these local operations; they are not end-to-end training comparisons across architectures.

9.1 Commands and Runtime Environment

Run these commands from the course source root.

- **Python interpreter**: `.venv/bin/python`
- **Core unit-test command**:

```
PYTHONPATH=src .venv/bin/python -m pytest tests/test_deep_learning.py -v
```

- **Standalone CLI experiment command**:

```
.venv/bin/python code/ch42_deep_learning.py --out build/batches/A8-C/reader/runs
    ↪ /ch42_trial_01
```

No-overwrite contract: if the specified `--out` directory already exists, the script raises an error (`FileExistsError`) and refuses to run, keeping historical experiment data immutable.

9.2 Fully Self-Contained Core-Interface Example

```
import numpy as np
from quant_course.deep_learning import (
    forward,
    loss_and_grad,
    fit_mlp,
    predict,
    generate_ch42_panel_data,
    split_ch42_panel,
)

# 1. Generate panel data and split it with time-boundary purging
x, raw_return, close, dates, assets = generate_ch42_panel_data(seed=4201)
splits = split_ch42_panel(x, raw_return, dates, assets)

# 2. Calculate feature and label means and standard deviations on the training set
    ↪ only
mu_X = np.mean(splits.X_train, axis=0)
sigma_X = np.std(splits.X_train, axis=0)
mu_y = float(np.mean(splits.y_train))
sigma_y = float(np.std(splits.y_train))

X_tr_s = (splits.X_train - mu_X) / sigma_X
y_tr_s = (splits.y_train - mu_y) / sigma_y
X_va_s = (splits.X_val - mu_X) / sigma_X
y_va_s = (splits.y_val - mu_y) / sigma_y
X_te_s = (splits.X_test - mu_X) / sigma_X

# 3. Train an MLP and automatically restore the best validation-set early-stopping
    ↪ checkpoint
mlp_res = fit_mlp(
    X_train=X_tr_s,
    y_train=y_tr_s,
    X_val=X_va_s,
    y_val=y_va_s,
    hidden_dim=8,
    lr=0.01,
    batch_size=32,
    l2_reg=0.001,
    max_epochs=200,
    patience=15,
    seed_init=4202,
    seed_shuffle=4203,
)

# 4. Predict and reverse standardization
pred_te_s = predict(X_te_s, mlp_res.W1, mlp_res.b1, mlp_res.W2, mlp_res.b2)
pred_te_real = pred_te_s * sigma_y + mu_y

print(f"MLP trained. Best Epoch: {mlp_res.best_epoch}, Stopped Epoch: {mlp_res.
    ↪ stopped_epoch}")
print(f"Test predictions shape: {pred_te_real.shape}, Mean prediction: {pred_te_real.
    ↪ mean():.6f}")
```

10. Visualization

The historical frozen CONSTRUCTED teaching CLI experiment produced five figures. This round reuses the original figures, stored under the relative directory `figures/root_r2/`:

10.1 Training and Validation Loss Curves with Early Stopping

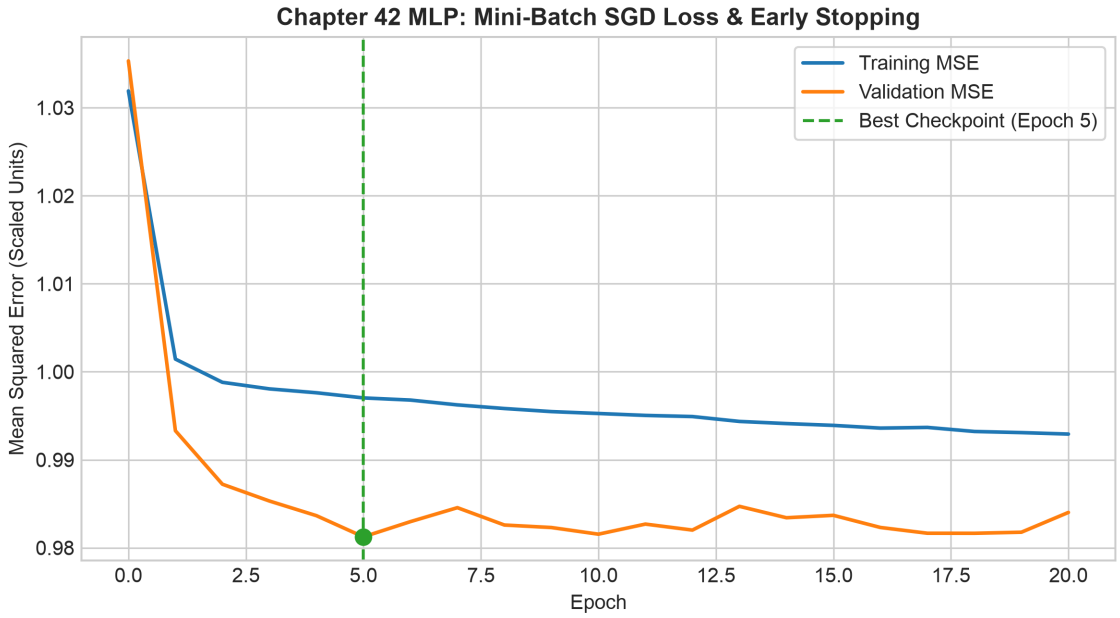
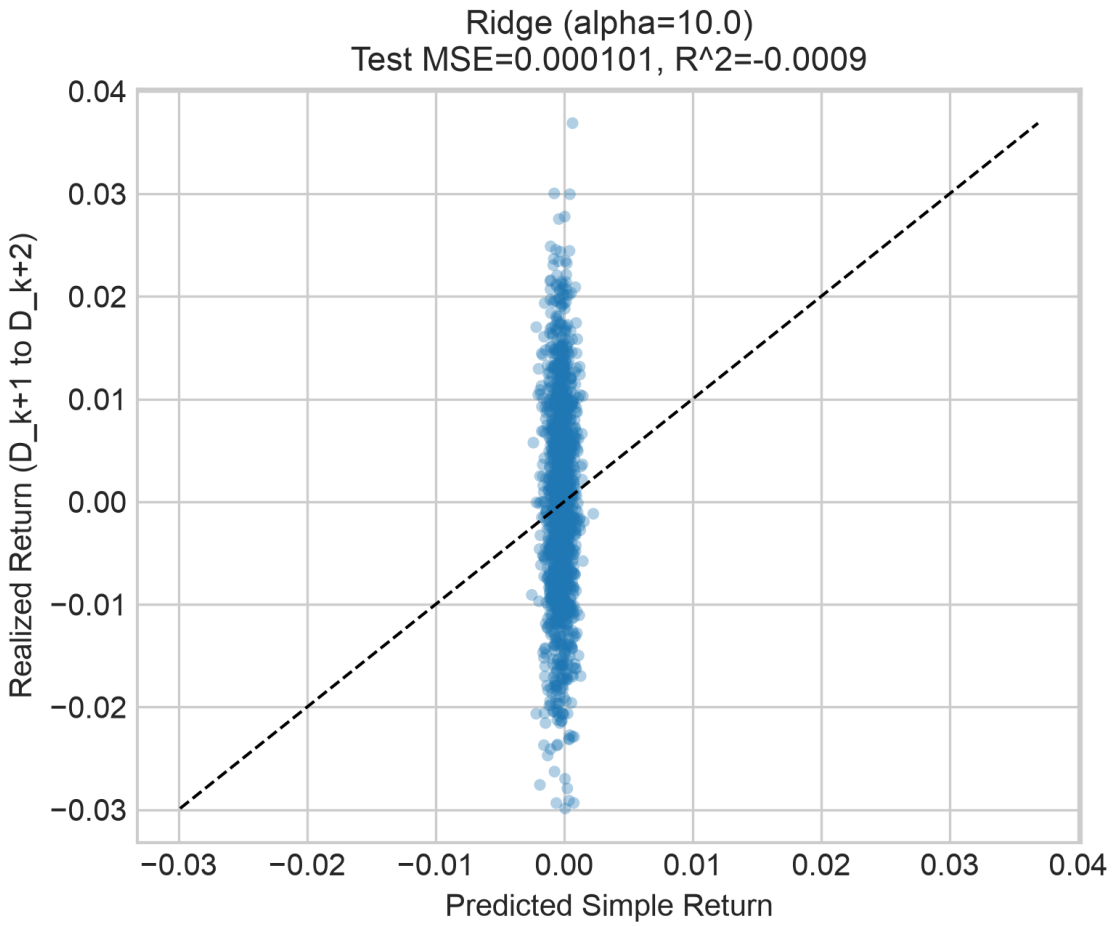


Figure 42.1: MLP training and validation loss curves

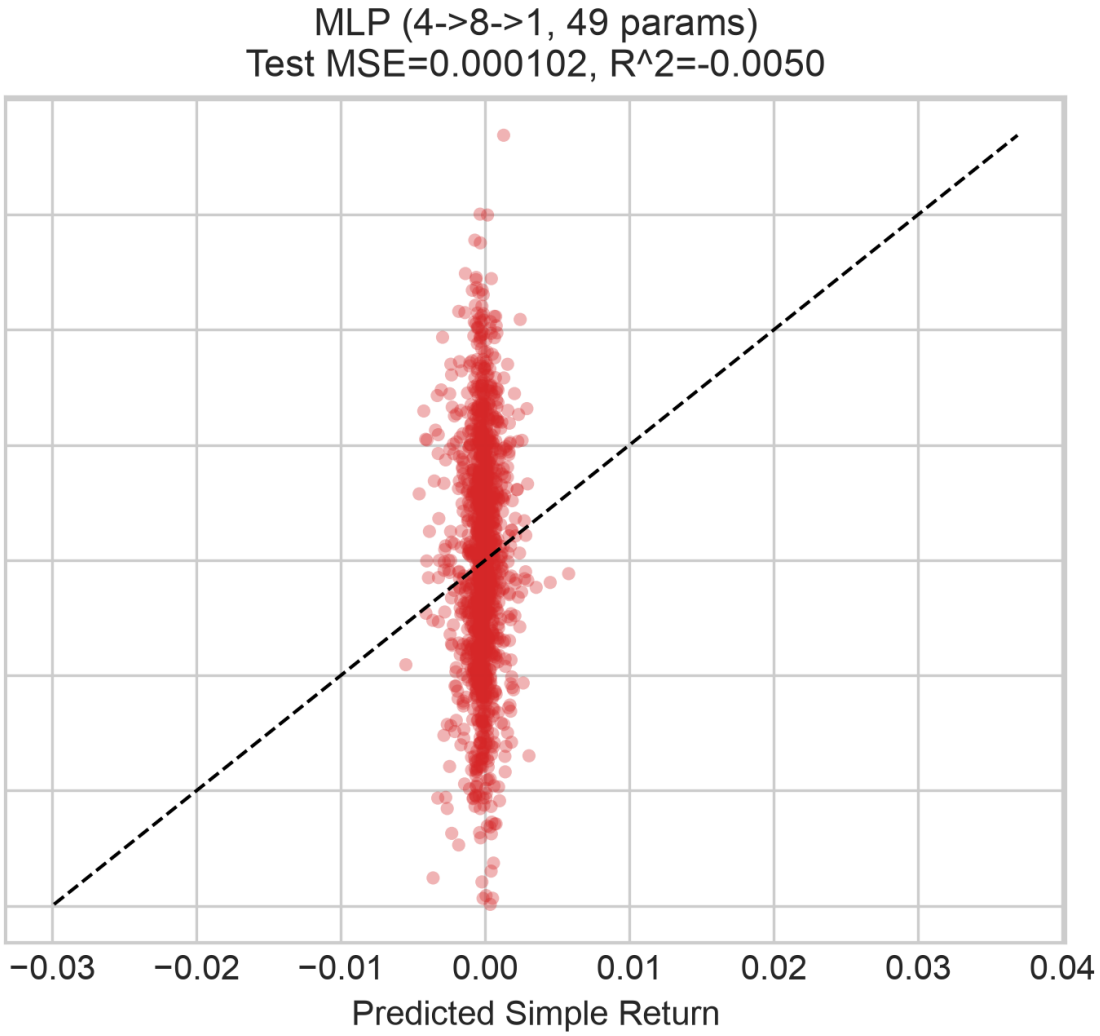
- **Figure notes**: The x-axis is epoch (0 to 20), and the y-axis is standardized mean squared error (MSE).
- **Observed pattern**: Training MSE (blue) declines overall, with a small rise from index 16 to 17. Validation MSE (orange) reaches its minimum at index 5 (0.9812), then fluctuates without reaching a new low. Patience=15 is exhausted at index 20, after 21 passes were actually completed; the weights from index 5 (the sixth pass) are restored (green dashed line). This observation does not establish that training or validation error must be monotone.

10.2 Test-Set Prediction Scatter: Ridge and MLP



(a)

- **Figure notes**: The left panel shows the best Ridge regression ($\alpha = 10$); the right panel shows the 4→8→1 MLP. The x-axis is predicted return and the y-axis is realized next-day simple return. The black dashed line is the ideal 45-degree fit.
- **Observed pattern**: The constructed labels vary much more widely than the model predictions. Both models' prediction ranges on the horizontal axis are clearly narrower than the realized-return range on the vertical axis, creating a narrow, nearly vertical scatter. This reflects how noise dominates realized returns in a low-SNR setting. The MLP test MSE (0.00010186) is slightly worse than Ridge (0.00010144), and both test-set R^2 values are slightly negative.

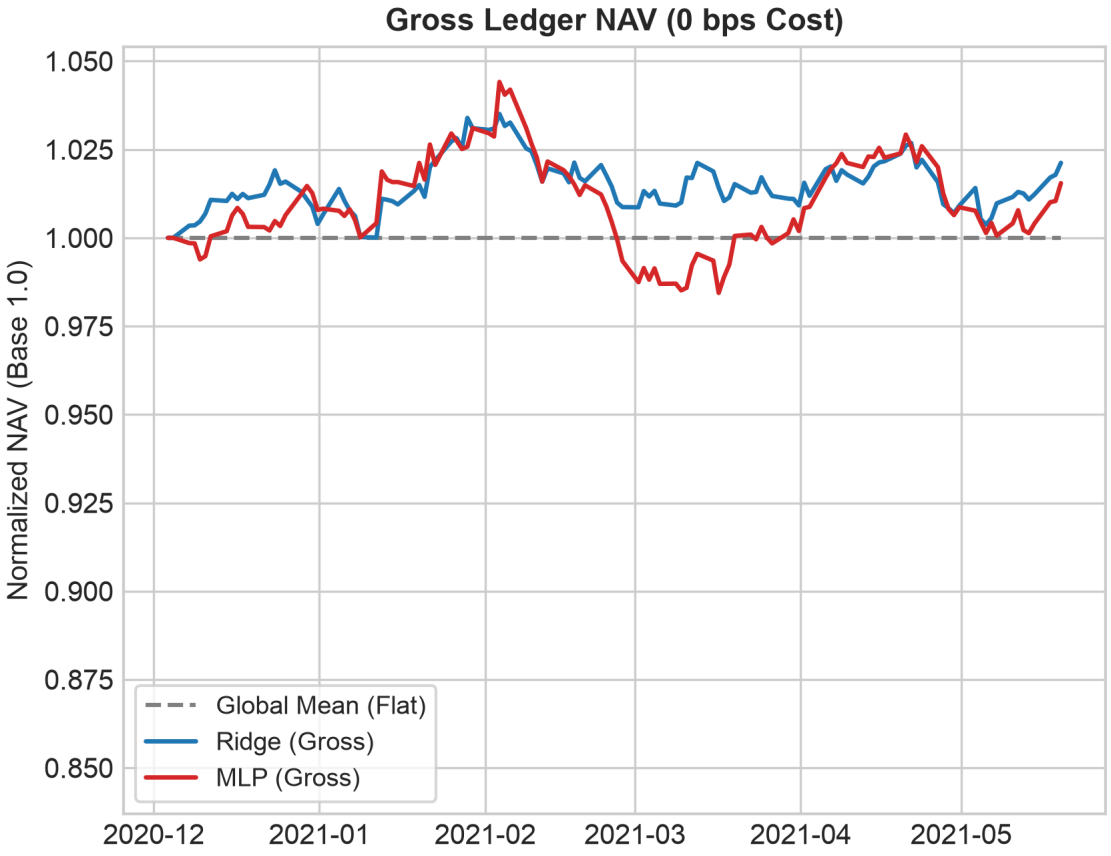


(b)

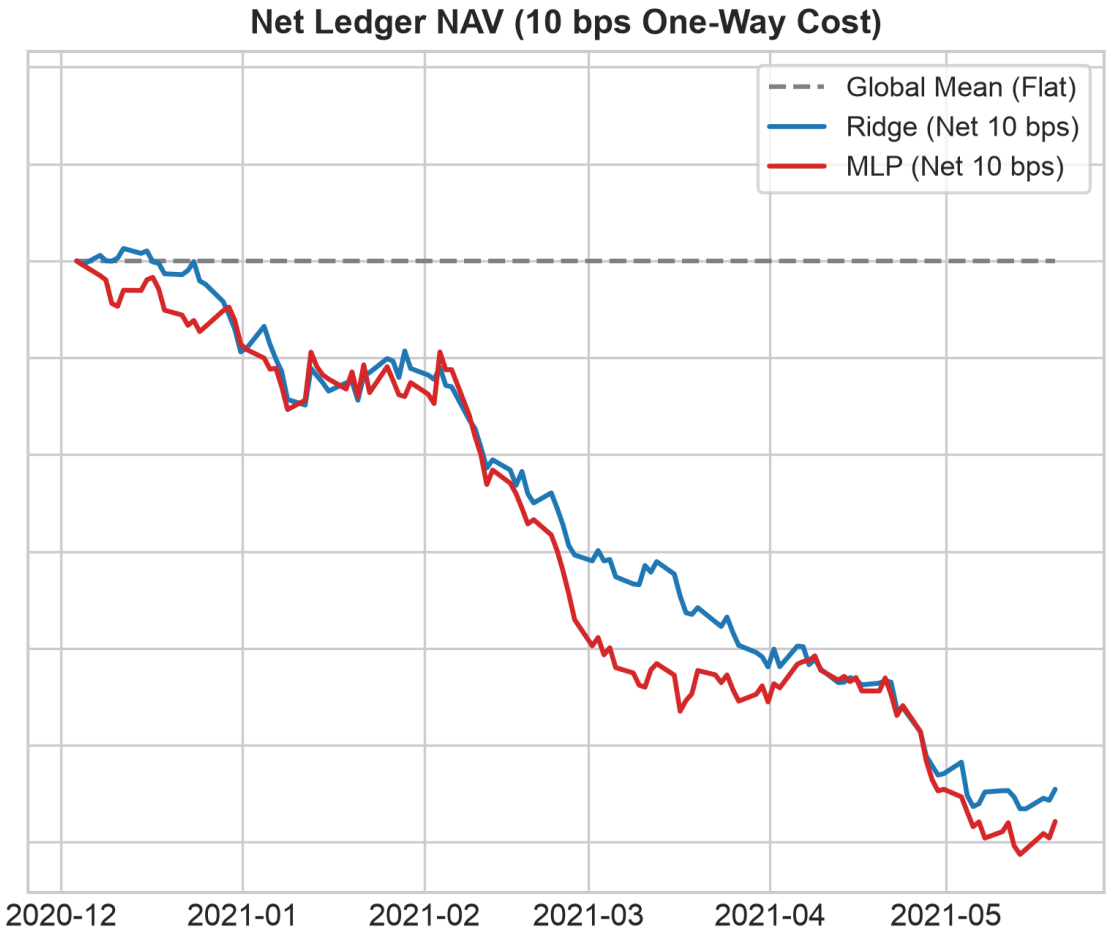
Figure 42.2: Predicted values versus realized returns

10.3 Gross and Net Cumulative Portfolio NAV from the Execution Ledger

- **Figure notes**: The left panel assumes zero cost (Gross); the right assumes a one-way cost of 10 bps (Net). The x-axis is ledger settlement date. The signal period is December 3, 2020 through May 17, 2021, with settlement extending to the final label date; the final settlement date is May 19, 2021. The y-axis is normalized portfolio NAV, with initial capital set to 1.0.
- **Observed pattern**: Under the zero-cost assumption, Ridge and MLP earn gross returns of +2.12% and +1.55%, respectively, in this constructed ledger, with gross Sharpe ratios of 0.817 and 0.501. Once the assumed one-way transaction cost of 10 bps is included, annualized turnover of 350+ times from frequent cross-sectional reordering reduces net NAV; Ridge and MLP end at about 0.864 and 0.855, respectively (net returns about -13.64% and -14.47%, net Sharpe ratios -5.47 and -4.69).


Gross Ledger NAV (0 bps Cost)
Normalized NAV (Base 1.0)
1.050
1.025
1.000
0.975
0.950
0.925
0.900
0.875
0.850
2020-12
2021-01
2021-02
2021-03
2021-04
2021-05
Global Mean (Flat)
Ridge (Gross)
MLP (Gross)

(a)



(b)

Figure 42.3: Cumulative portfolio NAV from the ledger

10.4 Daily Cross-Sectional Spearman IC on the Test Set

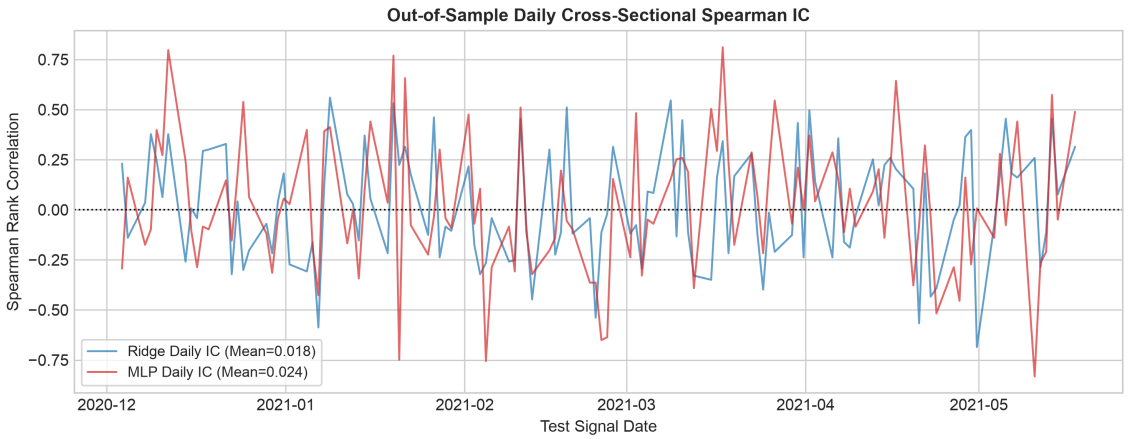


Figure 42.4: Daily cross-sectional Spearman IC on the test set

- **Figure notes**: The x-axis contains 118 mature test dates, and the y-axis is the Spearman rank correlation (IC) across 12 assets on each date.
- **Observed pattern**: In the frozen 118 daily IC rows, Ridge ranges from $[-0.6853, 0.5594]$ and MLP from $[-0.8322, 0.8112]$ (summarized directly from the frozen IC columns; predictions and training were not rerun). This shows how volatile cross-sectional return rankings are. Although the MLP mean IC (0.0241) is slightly higher than Ridge (0.0180), with a paired difference of $+0.0061$, time-series volatility means that this descriptive difference in means is not statistical significance or a net trading advantage.

10.5 Heatmap of Strictly Causal Transformer Attention Weights

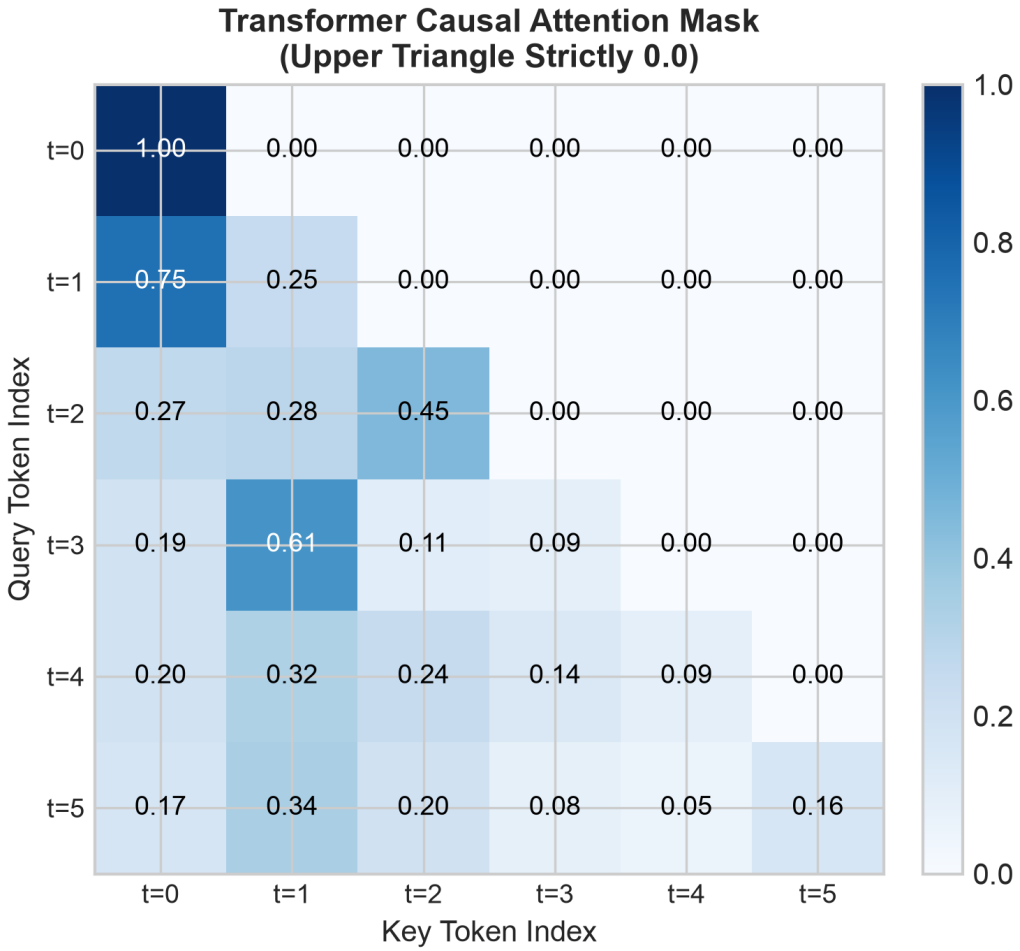


Figure 42.5: Heatmap of strictly causal Transformer attention weights

- **Figure notes**: The x-axis is the key-token time index ($t = 0 \dots 5$), and the y-axis is the query-token time index ($t = 0 \dots 5$).
- **Observed pattern**: The upper-triangular area, where key time is later than query time, is set strictly to 0.00, with values clearly shown in the matrix grid. This gives an engineering illustration that causal masking blocks information from flowing backward in time.

---

11. Real-Market Example / Experimental and Methodological Benchmark

Deep learning in asset pricing is widely discussed in academic and industry quantitative literature. This section clearly separates **verified methodological literature** from the **constructed teaching experiment run locally by the author**, to avoid conflation or fabricated data.

11.1 Review of Methodological Literature and Scope

1. **Backpropagation and structural bias (Goodfellow et al., 2016, §§6.5/9.2)**:
   - Backpropagation establishes a general computational framework for propagating error gradients layer by layer through local chain rules in composed functions;
   - Through local receptive fields and weight sharing, CNNs substantially reduce effective model degrees of freedom and build translation equivariance for time-series signals into the architecture as a prior.
2. **Recurrent memory and causal constraints on attention (Gers et al., 2000 abstract; PyTorch modern LSTM equations; Vaswani et al., 2017)**:
   - LSTM introduces a constant error carousel (CEC) with a forget gate, creating an additive channel for long-range memory flow;
   - Transformer uses dot-product self-attention to overcome sequence-distance limits and relies on a causal mask to block future information within attention calculations.
3. **No market-data run in this round**: No backtest on real secondary-market data was run in this round. All empirical numbers come from the frozen constructed teaching experiment in this chapter (seeds 4201, 4202, and 4203); they must not be extrapolated or described as real-market performance.

11.2 Summary of the Constructed Teaching Benchmark

Under the strictly frozen protocol, this chapter uses independent random seed 4201 to generate a synthetic asset panel with a weak nonlinear component $(0.0004x_0 + 0.0002(x_1^2 - 1))$ and independent Gaussian noise. The full empirical summary is in the table below:

Metric	Global training mean	Ridge	4→8→1 MLP	Teaching and methodological meaning
Parameter count	1 (scalar mean from training samples)	5 (4 weights + 1 bias)	49 (32 + 8 + 8 + 1)	Model complexity differs by nearly 50×
Test-set MSE	**0.00010135**	0.00010144	0.00010186	Mean baseline has the lowest MSE; MLP has the highest
Test-set R^2	0.000000	-0.000892	-0.004977	MLP and Ridge both have slightly negative out-of-sample R^2
Daily Spearman IC	NaN (constant; no variance)	0.0180	0.0241	Paired data are available on all 118 mature test days
Paired IC difference (MLP − Ridge)	—	baseline	+0.0061	Slight increase in cross-sectional rank correlation
Gross return (0 bps)	0.00%	+2.12%	+1.55%	Gross return before transaction costs

Metric	Global training mean	Ridge	4→8→1 MLP	Teaching and methodological meaning
Gross Sharpe (0 bps)	NaN	0.817	0.501	Gross performance before costs
Annualized turnover	0.0	351.8	360.2	Cross-sectional reordering results in high turnover
Net return (10 bps)	0.00%	**-13.64%**	**-14.47%**	Large drawdown after one-way 10 bps costs
Net Sharpe (10 bps)	NaN	**-5.468**	**-4.688**	Net performance is negative under this constructed setup

Why the negative results are retained: The experiment honestly reports the "negative results" that the MLP's out-of-sample MSE is worse than the one-parameter mean baseline and its net return suffers a severe drawdown under trading frictions. Quantitative research should not selectively report hand-picked seeds or intervals simply because a model is called "deep learning." Revealing the model's limits is a core value of quantitative engineering.

---

12. Common Mistakes

This section reviews four common traps in quantitative deep learning practice and provides runnable counterexamples and corrected code.

12.1 Trap 1: Full-Sample Standardization Causes Global Look-Ahead Leakage

- **Failure mechanism**: Calling `fit_transform` on all features before splitting the data lets the future test-set mean and extreme variance flow into the training inputs.
- **Self-contained verification code**:

```
import numpy as np

# Simulated data
rng = np.random.default_rng(42)
X_all = rng.normal(loc=0.0, scale=1.0, size=(100, 2))
X_all[80:] += 5.0  # A regime shift occurs in the future

# Incorrect: fit once on the full sample (leakage occurs)
mu_leaked = np.mean(X_all, axis=0)
sigma_leaked = np.std(X_all, axis=0)

# Correct: fit on the training set only (no leakage)
mu_clean = np.mean(X_all[:80], axis=0)
```

```
sigma_clean = np.std(X_all[:80], axis=0)

print(f"Leaked mean: {mu_leaked}, Clean train mean: {mu_clean}")
assert not np.allclose(mu_leaked, mu_clean), "Leakage alters training distribution!"
```

12.2 Trap 2: Cross-Period Time-Series Labels Are Not Purged at Boundaries

- **Failure mechanism**: When return labels span multiple dates (for example, signal D_k predicts returns from D_{k+1} to D_{k+2}), the outcomes for training samples $k = 178, 179$ are not settled until D_{180}, D_{181}. If the split is made directly at date 180, the final labels become available at or after validation begins and fail this frozen fitting snapshot's strict $<$ rule, so both 178 and 179 are removed. This strict convention does not imply that every other protocol using $\leq$ when boundary information has already arrived is leaky; interpret the actual clock and fitting snapshot.
- **Self-contained repair code**:

```
import pandas as pd
import numpy as np

dates = pd.bdate_range(start="2020-01-02", periods=20, tz="UTC") + pd.Timedelta(hours
    ↪ =16)

# Signal at k, execute one period later at k+1, hold one period through k+2; the
    ↪ label is then available
df = pd.DataFrame({
    "signal_date": dates[:18],
    "label_available_at": dates[2:],
})

val_first_signal = dates[10]

# Correct filter: keep only mature training samples settled before the validation set
    ↪ 's first signal time
train_clean = df[df["label_available_at"] < val_first_signal]
print(f"Total candidate dates: {len(df)}, Mature clean dates: {len(train_clean)}")
```

12.3 Trap 3: Assuming a Transformer Causal Mask Prevents Look-Ahead in Input Features

- **Failure mechanism**: A causal mask only ensures that the attention layer **does not actively query a future hidden state, given the current feature representations**. If the input feature itself already contains future returns—for example, `rolling(5).mean().shift(-1)` reads at t a window through $t+1$, or after-hours revisions are used without recording `available_at`—then the input at each time step is already contaminated. A causal mask cannot defend against this feature-level look-ahead leakage. A standard right-aligned `rolling(5).mean()` at t reads only $t-4$ through t and can be causal; `shift(-1)` pulls the next row's window result back to t. Do not infer look-ahead from "right-aligned" alone.

12.4 Trap 4: Monitoring the Test Set for Early Stopping or Failing to Restore the Best Epoch After Divergence

- **Failure mechanism**: During training, the network parameters at the lowest validation loss are not saved properly, and the parameters from the final epoch are returned as soon as early stopping triggers; or, more seriously, the test set itself is monitored for early stopping.
- **Self-contained checkpoint code**:

```
best_val_loss = float("inf")
best_weights = {}

# During training
for epoch in range(10):
    val_loss = 1.0 + (epoch - 3)**2 * 0.1  # Simulate the lowest loss at epoch=3
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        best_weights = {"epoch": epoch, "loss": val_loss}

# After training stops: restore best_weights, not the epoch=9 weights
assert best_weights["epoch"] == 3
print(f"Successfully restored checkpoint from epoch {best_weights['epoch']}")
```

---

13. Quantitative Research Application

A standardized end-to-end deep-learning research system should follow a workflow such as:

```
[Form a theoretical hypothesis] -> [Engineer features and record availability times]
    ↪ -> [Split strictly by time and purge boundary labels]
       |
       v
[Set one-parameter/low-parameter baselines (Mean, Ridge)] -> [Train deep model and
    ↪ early-stop on an independent validation set]
       |
       v
[Evaluate out-of-sample generalization (MSE, R^2, IC)] -> [Dollar-neutral targets and
    ↪  gross-exposure normalization]
       |
       v
[Connect a lagged execution ledger (Execution Lag = 1)] -> [Deduct multiple cost
    ↪ tiers (0, 10 bps)]
       |
       v
[Assess fee tolerance under fixed cost assumptions (Net Sharpe > 0? Is turnover
    ↪ tolerable?)]
```

The target-weight interface first demeans valid cross-sectional predictions, then normalizes to $\sum_i |w_i| = 1$, giving zero net dollar weight. It returns all zeros if there are too few valid assets or all predictions are identical. It does not guarantee beta neutrality, and realized weights drift after execution; the ledger remains authoritative.

13.1 Standards for the Main Research Stages

1. **Stage 1: Baseline first** Before starting any complex deep-learning project, set up two baselines: the overall mean fitted on the mature training block only (178 dates / 2,136 rows; no validation or test data) to test whether there is any statistical predictive power, and a regularized linear model such as Ridge to test whether nonlinear complexity adds value. If the deep model cannot beat a strongly constrained Ridge out of sample, reassess whether the network architecture's complexity is justified.
2. **Stage 2: Time contract and availability-time registration** Record the factor's `signal_time`, order execution `trade_time`, and label settlement `label_available_at` strictly. Apply boundary purging and never randomly shuffle training, validation, and test time blocks (No Shuffling Across Time Blocks). This does not conflict with reordering mini-batches within a mature training block.
3. **Stage 3: Stress-test transaction costs and friction tolerance** Pass prediction outputs to the execution ledger and test a stepwise transaction-cost range from 0 to 20 bps. Assess the strategy's breakeven cost and make sure it has a buffer against live-market slippage.

---

14. Chapter Summary

1. **Structural inductive bias is central to model selection**: Deep learning is not a universal black box. MLP, CNN, RNN/LSTM, and Transformer architectures each introduce different prior assumptions. Adding network depth blindly without enough prior structure can accelerate overfitting.
2. **Low financial SNR constrains parameter freedom**: The ratio of parameter count to the effective number of training samples is a risk-diagnostic clue, but cannot determine generalization error by itself. Under weak signals, a one-parameter mean and a low-parameter Ridge may remain robust out of sample because their estimation variance is lower.
3. **Strict time splits and boundary purging are essential**: For multi-period return labels, samples that cross the boundary between training and validation/test must be excluded. A causal mask blocks look-ahead within a layer but cannot replace strict time engineering at lower layers.
4. **Real trading frictions are central to quantitative-strategy evaluation**: Any high-turnover prediction signal must be evaluated in a complete execution ledger with explicit one-way friction assumptions (10 bps is assumed in this constructed example, not a measured real-market fee). Gross return without costs does not reveal actual investment viability.

15. Exercises

42-E01 (Conceptual: MLP Matrix Dimensions and Forward Pass)

A quantitative stock-selection model uses a $4 \to 8 \to 1$ MLP. It has four technical indicators as input, an eight-neuron hidden layer with a tanh activation, and next-day expected return as output.

1. Write the exact matrix dimensions for first-layer weight $\mathbf{W}_1$, first-layer bias $\mathbf{b}_1$, second-layer weight $\mathbf{W}_2$, and second-layer bias $\mathbf{b}_2$;
2. Calculate the model's total number of trainable parameters, ignoring regularization;
3. If the input is a cross-sectional feature matrix $\mathbf{X} \in \mathbb{R}^{32\times 4}$ containing 32 stocks, state the tensor shapes of hidden-layer pre-activation $\mathbf{Z}_1$, hidden activation $\mathbf{A}_1$, and final prediction $\hat{\mathbf{Y}}$ in the forward pass.

42-E02 (Mathematical: Expand the Analytical Gradient and L2 Term)

Consider the loss function $\mathcal{L} = \frac{1}{B\cdot D_{\text{out}}}\|\hat{\mathbf{Y}} - \mathbf{Y}\|_F^2 + \lambda(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$.

1. Derive in detail the regularization component of the second-layer weight gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2}$;
2. Explain why, in an ordinary SGD update $\mathbf{W} \leftarrow \mathbf{W} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$, L_2 regularization is equivalent to weight decay, and state the effective contraction condition;
3. Explain why bias vectors $\mathbf{b}_1, \mathbf{b}_2$ are usually not penalized with L_2 regularization.

42-E03 (Explanatory: Early Stopping and the Bias-Variance Trade-Off)

In the Chapter 42 experiment, the MLP reaches its lowest validation loss at epoch 5, but training does not stop until epoch 20.

1. Using the bias-variance trade-off from statistical learning theory, explain why validation error can first fall and then rise as training continues. Explain why the actual curve in this chapter cannot be summarized as necessarily U-shaped;
2. Why must the epoch-5 weight checkpoint be restored after stopping, instead of using the epoch-20 parameters directly?

42-E04 (Conceptual: CNN Cross-Correlation, Weight Sharing, and Translation Equivariance)

1. Explain how parameter sharing and local connectivity in a one-dimensional CNN reduce the number of model parameters;
2. Why does convolution provide translation equivariance rather than translation invariance? Explain the difference between cross-correlation in deep learning and mathematical convolution.

42-E05 (Mathematical: RNN Vanishing Gradients and the LSTM Constant Error Carousel)

1. Given the scalar Simple RNN state recursion $h_t = w_h h_{t-1} + x_t$, calculate the gradient transfer attenuation across 10 time steps if $|w_h| = 0.5$;
2. For a scalar modern LSTM, derive in detail the direct-path partial derivative $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ under the assumption that gates and the candidate state are fixed;
3. Explain how the constant error carousel (CEC) mechanism mitigates vanishing gradients in long-sequence backpropagation and why it cannot be said to eliminate every form of gradient attenuation.

42-E06 (Explanatory: Transformer Causal Mask and Research-Leakage Boundary)

1. Explain in detail why the upper-triangular elements of a causal mask in Transformer scaled dot-product attention are set to $-\infty$ rather than 0;
2. Make the case for the statement: "A causal mask only ensures that attention does not look ahead internally; it cannot prevent temporal look-ahead leakage in the construction of lower-layer input features." Give a concrete feature example from quantitative research.

---

16. Programming Exercises

42-P01 (Programming: Central-Difference Numerical Gradient Checker)

Write a Python function `check_mlp_gradients(X, y, W1, b1, W2, b2, l2_reg=0.01, eps=1e-6)` that:

1. Calls the analytical loss-and-gradient function to calculate analytical gradients ($\mathbf{dW}_1, \mathbf{db}_1$, $\mathbf{dW}_2, \mathbf{db}_2$);
2. For every scalar element of every parameter tensor, calculates a symmetric central-difference numerical gradient by perturbing the parameter by $+\epsilon$ and $-\epsilon$;
3. Asserts that the maximum absolute error for every parameter is less than 10^{-7} and prints a verification-passed report.

42-P02 (Programming: MLP Training with Early Stopping and Boundary Isolation)

Write a complete model-training and isolated-test evaluation script that:

1. Takes a constructed panel containing 360 dates;
2. Splits mature training, validation, and test data strictly over $[0, 177]$, $[180, 237]$, and $[240, 357]$, excluding boundary samples whose labels have not settled;
3. Fits standardization parameters for features and labels on the training set only;
4. Runs `fit_mlp` and reports unstandardized test-set MSE and R^2.

42-P03 (Programming: Transformer Future-Input Perturbation-Invariance Unit Test)

Write a pytest-style automated test function named `test_causal_attention_future_invariance()` that:

1. Constructs Query, Key, and Value matrices for five time steps with feature dimension 4;
2. Calculates causal-mask attention output $\mathbf{O}_{\text{orig}}$;
3. Perturbs the input values at the last two time steps ($t = 3, 4$) and recalculates output $\mathbf{O}_{\text{pert}}$;
4. Asserts that historical time-step outputs ($t = 0, 1, 2$) remain exactly unchanged (maximum absolute difference equals 0.0);
5. Includes a negative test: with causal masking disabled (`mask=False`), assert that the future perturbation changes historical outputs.

---

17. Challenge Problems

42-C01 (Challenge: Test-Set Peeking and Optimistically Biased Test Results)

Design and run a negative-control experiment that contaminates the test set:

1. In the normal procedure, early stopping monitors an independent validation set;
2. In the contaminated procedure, deliberately pass the test set to the early-stopping monitor and use test loss to select the best checkpoint;
3. Compare final test MSE and R^2 reported by the two procedures;
4. From a quantitative-research methodology perspective, write a 300-word report explaining why "selecting the best early-stopping checkpoint on the test set" is data snooping.

42-C02 (Challenge: Fee Tolerance and Turnover Trade-Off Across Several Cost Tiers)

Using test-set predictions from the Chapter 42 $4 \to 8 \to 1$ MLP experiment:

1. Connect to the Chapter 29 execution ledger and run backtests at one-way costs of 0, 2, 5, 10, and 20 bps (as a post hoc teaching sensitivity analysis, separate from the main protocol);
2. Plot net Sharpe as one-way transaction costs increase;
3. Use bisection to find, within a numerical tolerance, the breakeven fee rate at which the MLP strategy's net Sharpe crosses zero;
4. Discuss how smoothing network predictions (such as with exponential moving average, EMA) or adding a turnover-penalty loss could mitigate turnover drag.

18. Complete Solutions

42-E01 Complete Solution

1. **Tensor dimensions**:
   - Input dimension $D_{\text{in}} = 4$, hidden dimension $D_{\text{hidden}} = 8$, and output dimension $D_{\text{out}} = 1$;
   - First-layer weight $\mathbf{W}_1 \in \mathbb{R}^{4\times 8}$;
   - First-layer bias $\mathbf{b}_1 \in \mathbb{R}^{1\times 8}$;
   - Second-layer weight $\mathbf{W}_2 \in \mathbb{R}^{8\times 1}$;
   - Second-layer bias $\mathbf{b}_2 \in \mathbb{R}^{1\times 1}$.
2. **Total trainable parameter count**:
   - Weight parameters: $(4 \times 8) + (8 \times 1) = 32 + 8 = 40$;
   - Bias parameters: $8 + 1 = 9$;
   - Total: $40 + 9 = 49$ floating-point parameters.
3. **Batch tensor shapes**:
   - Input $\mathbf{X} \in \mathbb{R}^{32\times 4}$;
   - Hidden pre-activation $\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1 \in \mathbb{R}^{32\times 8}$;
   - Hidden activations $\mathbf{A}_1 = \tanh(\mathbf{Z}_1) \in \mathbb{R}^{32\times 8}$;
   - Final prediction $\hat{\mathbf{Y}} = \mathbf{A}_1\mathbf{W}_2 + \mathbf{b}_2 \in \mathbb{R}^{32\times 1}$.

42-E02 Complete Solution

1. **Derivation of the regularization gradient**: Expand the regularization term as a sum of squared entries:
$$g(\mathbf{W}_2) = \lambda\|\mathbf{W}_2\|_F^2 = \lambda \sum_{j=1}^{D_{\text{hidden}}} \sum_{k=1}^{D_{\text{out}}} W_{2,j,k}^2.$$
Differentiate with respect to a particular entry $W_{2,j,k}$: $\frac{\partial g}{\partial W_{2,j,k}} = 2\lambda W_{2,j,k}$. In matrix form, this is $2\lambda\mathbf{W}_2$.
2. **Weight-decay equivalence and contraction condition**: Under ordinary SGD, the parameter update is:
$$\begin{aligned}\mathbf{W} &\leftarrow \mathbf{W} - \eta\frac{\partial \mathcal{L}}{\partial \mathbf{W}} \\ &= \mathbf{W} - \eta\left(\nabla_{\mathbf{W}}\mathcal{L}_{\text{MSE}} + 2\lambda\mathbf{W}\right) \\ &= (1 - 2\eta\lambda)\mathbf{W} - \eta\nabla_{\mathbf{W}}\mathcal{L}_{\text{MSE}}\end{aligned}$$
When learning rate $\eta > 0$ and $\lambda > 0$, if $0 < 2\eta\lambda < 1$, the contraction factor $(1 - 2\eta\lambda)$ lies strictly between 0 and 1 and pulls the weight toward zero at each step. This is why it is equivalent to weight decay under ordinary SGD.
3. **Why biases are not regularized**: This chapter does not penalize biases so that baseline output and activation shifts remain free; this is not a mathematical requirement. Penalizing biases pulls them toward zero. Its practical effect must be assessed for the task, and we cannot claim that bias has no effect on local curvature or generalization.

42-E03 Complete Solution

1. **Why validation error may rise again**: Early in training, the model starts from random weights and learns the main nonlinear patterns in the data; it may gradually learn reusable relationships, reducing validation error. As training continues, a high-capacity network may use excess degrees of freedom to fit idiosyncratic random noise in the training samples. The bias term may then decline only slowly while estimation becomes more sensitive to training-sample noise, causing prediction error on unseen validation samples to rebound. The curve might be U-shaped, but it could also fluctuate, plateau, or continue declining. In this chapter, the validation curve reaches its recorded minimum at epoch 5 and then fluctuates slightly; the curve does not establish a monotone relationship between epoch and variance.
2. **Why the checkpoint must be restored**: Epoch-20 validation error is above the recorded minimum; this experiment does not establish that its true future error must also be higher. Early stopping treats "number of training epochs" as a discrete implicit regularization hyperparameter. The stopping rule means the validation minimum was not refreshed for 15 consecutive epochs; it does not require a continuous rise. Under the fixed selection rule, restore the checkpoint from index 5. Using index 20 directly would violate that rule and select parameters with higher observed validation error, but we cannot conclude that they would necessarily generalize worse to every new sample.

42-E04 Complete Solution

1. **Parameter sharing and local connectivity**:
   - Local connectivity: each convolutional neuron connects only to an adjacent local window of length K in the input sequence, rather than to the entire sequence of length N. This reduces its number of connections from $O(N)$ to $O(K)$;
   - Parameter sharing: the same convolution kernel slides across the entire time series, so every time step reuses the same K weights. This example has one input channel, one output channel, and one kernel, giving $K+1$ parameters, including bias, independent of the sequence length N. In general, with C_{in} input channels and C_{out} output channels, the parameter count is $C_{\text{out}}(KC_{\text{in}} + 1)$.
2. **Translation equivariance and cross-correlation**:
   - Convolution satisfies $f(g(x)) = g(f(x))$: input shifts commute with feature extraction, giving translation equivariance. Translation invariance instead requires $f(g(x)) = f(x)$. Global pooling can provide invariance when translation only reorders the same complete set of features; local pooling or edge cropping does not automatically guarantee invariance;
   - Mathematical convolution flips the kernel in time, $\sum_j x_{i-j} w_j$; cross-correlation does not flip it, $\sum_j x_{i+j} w_j$. Because deep-learning kernel weights are learned end to end by gradient descent, either convention has the same expressive power up to a flip. Frameworks implement cross-correlation without flipping the kernel.

42-E05 Complete Solution

1. **Simple RNN gradient attenuation**: From the derivation, $\frac{\partial h_{10}}{\partial h_0} = (w_h)^{10} = (0.5)^{10} = \frac{1}{1024} \approx 0.00097656$. The gradient has decayed by more than a factor of 1,000 after only 10 time steps of backpropagation.
2. **LSTM state partial derivative**: The cell-state update is $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. This exercise uses a scalar state; holding the gate and candidate values fixed, differentiate with respect to c_{t-1} (the vector version is $\text{diag}(\mathbf{f}_t)$):

$$\begin{aligned}\frac{\partial c_t}{\partial c_{t-1}} &= \frac{\partial (f_t \odot c_{t-1})}{\partial c_{t-1}} \\ &+ \frac{\partial (i_t \odot \tilde{c}_t)}{\partial c_{t-1}} = f_t \\ &+ 0 = f_t\end{aligned}$$

3. **How CEC helps and its limits**: CEC replaces the RNN's chain product with a direct path that includes a forget gate. When the forget gate remains highly active ($f_t \approx 1$), the gain on the direct gradient path is close to 1.0, allowing error to propagate through time. The total derivative is still constrained by nonlinear paths through hidden-state feedback; with greater depth or unsaturated gates, gradients can still decay. Therefore, it cannot be claimed to solve every gradient problem.

42-E06 Complete Solution

1. **Why set the mask to $-\infty$**: Softmax applies the natural exponential transform. Let the original score be S_{ij}. An additive mask value $M_{ij} = 0$ preserves S_{ij}, whose exponential is $\exp(S_{ij})$ and is generally not 1. Only if a future logit is directly overwritten with 0 is its exponential $\exp(0) = 1$. Neither operation strictly masks a future position. With a mask of $-\infty$, however, $\lim_{z \to -\infty} \exp(z) = 0.0$, so the future position's numerator is exactly zero in softmax normalization and its attention weight is strictly zero.
2. **Limits of causal masks and a counterexample**:
   - **Mechanism**: The causal mask applies strictly within the attention-matrix calculation. It only prevents output at time t from aggregating input at time $t+1$, given the input vectors at each time step.
   - **Counterexample**: A researcher defines a feature called "five-day moving-average breakout," but mistakenly codes it as `df['close'].rolling(5).mean().shift(-1)`. At t, this takes the moving average from $t+1$, with a window from $t-3$ through $t+1$ that includes tomorrow's closing price. When this feature is fed into a Transformer with a causal mask, the model has already received tomorrow's close in the input at time t. The causal mask cannot prevent this feature-level look-ahead.

42-P01 Complete Implementation and Runtime Check

```
import numpy as np
from quant_course.deep_learning import loss_and_grad

def check_mlp_gradients(X, y, W1, b1, W2, b2, l2_reg=0.01, eps=1e-6):
    """Check that MLP analytical gradients closely match numerical central
    ↪ differences."""
    loss, dW1, db1, dW2, db2 = loss_and_grad(X, y, W1, b1, W2, b2, l2_reg=l2_reg)
    param_dict = {"W1": (W1, dW1), "b1": (b1, db1), "W2": (W2, dW2), "b2": (b2, db2)}

    for name, (param, analytic_g) in param_dict.items():
        num_g = np.zeros_like(param)
        it = np.nditer(param, flags=["multi_index"])
        while not it.finished:
            idx = it.multi_index
            orig = param[idx]
            param[idx] = orig + eps
            l_plus, _, _, _, _ = loss_and_grad(X, y, W1, b1, W2, b2, l2_reg=l2_reg)
            param[idx] = orig - eps
            l_minus, _, _, _, _ = loss_and_grad(X, y, W1, b1, W2, b2, l2_reg=l2_reg)
            param[idx] = orig
            num_g[idx] = (l_plus - l_minus) / (2.0 * eps)
            it.iternext()

        max_err = float(np.max(np.abs(analytic_g - num_g)))
        assert max_err < 1e-7, f"Gradient mismatch on {name}: max_err={max_err:.2e}"
        print(f"[*] Parameter {name} gradient check passed with max_err = {max_err:.2
    ↪ e}")
    return True

# Runtime verification
if __name__ == "__main__":
    rng = np.random.default_rng(42)
    X = rng.normal(size=(4, 2))
    y = rng.normal(size=(4, 1))
    W1 = rng.normal(size=(2, 2))
    b1 = rng.normal(size=(1, 2))
    W2 = rng.normal(size=(2, 1))
    b2 = rng.normal(size=(1, 1))
    assert check_mlp_gradients(X, y, W1, b1, W2, b2)
```

42-P02 Complete Implementation and Runtime Check

```
import numpy as np
from quant_course.deep_learning import generate_ch42_panel_data, split_ch42_panel,
    ↪ fit_mlp, predict

def run_isolated_pipeline():
    # 1. Generate panel data and split by time
    x, raw_return, close, dates, assets = generate_ch42_panel_data(seed=4201)
    splits = split_ch42_panel(x, raw_return, dates, assets)

    # 2. Fit standardization parameters on the training set only
    mu_X = np.mean(splits.X_train, axis=0)
    sigma_X = np.std(splits.X_train, axis=0)
    mu_y = float(np.mean(splits.y_train))
    sigma_y = float(np.std(splits.y_train))

    X_train_s = (splits.X_train - mu_X) / sigma_X
    y_train_s = (splits.y_train - mu_y) / sigma_y
    X_val_s = (splits.X_val - mu_X) / sigma_X
    y_val_s = (splits.y_val - mu_y) / sigma_y
    X_test_s = (splits.X_test - mu_X) / sigma_X

    # 3. Train the MLP and restore the early-stopping checkpoint
    mlp_res = fit_mlp(
        X_train=X_train_s, y_train=y_train_s,
        X_val=X_val_s, y_val=y_val_s,
        hidden_dim=8, lr=0.01, batch_size=32, l2_reg=0.001,
        max_epochs=200, patience=15, seed_init=4202, seed_shuffle=4203
    )

    # 4. Predict on the test set and invert the transformation
    pred_s = predict(X_test_s, mlp_res.W1, mlp_res.b1, mlp_res.W2, mlp_res.b2)
    pred_real = pred_s * sigma_y + mu_y

    # 5. Evaluate MSE and R^2 in the original units
    mse_test = float(np.mean((splits.y_test - pred_real) ** 2))
    mse_mean_baseline = float(np.mean((splits.y_test - mu_y) ** 2))
    r2_test = 1.0 - mse_test / mse_mean_baseline

    print(f"Test MSE: {mse_test:.8f}, Test R^2: {r2_test:.6f}")
    return mse_test, r2_test

if __name__ == "__main__":
    run_isolated_pipeline()
```

42-P03 Complete Implementation and Runtime Check

```python
import numpy as np
from quant_course.deep_learning import causal_attention

def test_causal_attention_future_invariance():
    T, d_k = 5, 4
    rng = np.random.default_rng(42)
    Q = rng.normal(size=(T, d_k))
    K = rng.normal(size=(T, d_k))
    V = rng.normal(size=(T, d_k))

    # 1. Output with the causal mask
    weights_orig, out_orig = causal_attention(Q, K, V, mask=True)
    # Strictly verify that the upper triangle is zero
    assert np.allclose(np.triu(weights_orig, k=1), 0.0)

    # 2. Perturb only future time steps t=3, 4
    Q_pert, K_pert, V_pert = Q.copy(), K.copy(), V.copy()
    Q_pert[3:] += 10.0
    K_pert[3:] -= 5.0
    V_pert[3:] += 8.0

    _, out_pert = causal_attention(Q_pert, K_pert, V_pert, mask=True)

    # 3. Historical outputs at t=0, 1, 2 are unaffected
    history_diff = np.max(np.abs(out_orig[:3] - out_pert[:3]))
    assert history_diff == 0.0, f"Future leakage detected! diff={history_diff}"

    # 4. Negative test: without the mask, future perturbations contaminate historical
    ↪ outputs
    _, out_unmasked = causal_attention(Q, K, V, mask=False)
    _, out_unmasked_pert = causal_attention(Q_pert, K_pert, V_pert, mask=False)
    leakage = np.max(np.abs(out_unmasked[:3] - out_unmasked_pert[:3]))
    assert leakage > 1e-3, "Unmasked attention failed to detect leakage."
    print("[*] Causal attention invariance unit test passed.")

if __name__ == "__main__":
    test_causal_attention_future_invariance()
```

42-C01 Complete Runnable Code and Analysis Report

```
import numpy as np
from quant_course.deep_learning import generate_ch42_panel_data, split_ch42_panel,
    ↪ fit_mlp, predict

def run_c01_experiment():
    x, raw_return, close, dates, assets = generate_ch42_panel_data(seed=4201)
    splits = split_ch42_panel(x, raw_return, dates, assets)

    mu_X, sigma_X = np.mean(splits.X_train, axis=0), np.std(splits.X_train, axis=0)
    mu_y, sigma_y = float(np.mean(splits.y_train)), float(np.std(splits.y_train))

    X_tr_s = (splits.X_train - mu_X) / sigma_X
    y_tr_s = (splits.y_train - mu_y) / sigma_y
    X_va_s = (splits.X_val - mu_X) / sigma_X
    y_va_s = (splits.y_val - mu_y) / sigma_y
    X_te_s = (splits.X_test - mu_X) / sigma_X
    y_te_s = (splits.y_test - mu_y) / sigma_y

    # 1. Clean procedure: monitor the independent validation set
    res_clean = fit_mlp(X_tr_s, y_tr_s, X_va_s, y_va_s, hidden_dim=8, lr=0.01,
                        batch_size=32, l2_reg=0.001, max_epochs=200, patience=15,
                        seed_init=4202, seed_shuffle=4203)
    pred_clean_real = predict(
        X_te_s, res_clean.W1, res_clean.b1,
        res_clean.W2, res_clean.b2
    ) * sigma_y + mu_y
    mse_clean = float(np.mean((splits.y_test - pred_clean_real)**2))
    mse_base = float(np.mean((splits.y_test - mu_y)**2))
    r2_clean = 1.0 - mse_clean / mse_base

    # 2. Contaminated procedure: monitor the test set directly
    res_contaminated = fit_mlp(X_tr_s, y_tr_s, X_te_s, y_te_s, hidden_dim=8, lr=0.01,
                               batch_size=32, l2_reg=0.001, max_epochs=200, patience
    ↪ =15,
                               seed_init=4202, seed_shuffle=4203)
    pred_contam_real = predict(
        X_te_s, res_contaminated.W1, res_contaminated.b1,
        res_contaminated.W2, res_contaminated.b2
    ) * sigma_y + mu_y
    mse_contam = float(np.mean((splits.y_test - pred_contam_real)**2))
    r2_contam = 1.0 - mse_contam / mse_base

    print(f"Clean Pipeline        : Best Epoch={res_clean.best_epoch}"
          f", Test MSE={mse_clean:.8f}, R^2={r2_clean:.6f}")
    print(f"Contaminated Pipeline : Best Epoch={res_contaminated.best_epoch}"
          f", Test MSE={mse_contam:.8f}, R^2={r2_contam:.6f}")
    return {
        "clean": {"best_epoch": res_clean.best_epoch, "mse": mse_clean, "r2":
    ↪ r2_clean},
        "contaminated": {"best_epoch": res_contaminated.best_epoch,
```

```
                        "mse": mse_contam, "r2": r2_contam},
    }

if __name__ == "__main__":
    run_c01_experiment()
```

Post hoc teaching-run values inherited from the frozen historical run (not rerun in this round):

- Compliant procedure: restored Best Epoch $= 5$, test MSE $= 0.0001018575$, $R^2 = -0.004977$;
- Contaminated procedure: restored Best Epoch $= 17$, test MSE $= 0.0001015535$, $R^2 = -0.001978$.

Methodological analysis report: In the machine-learning development loop, the test set serves as an independent stand-in for an "unseen future." When researchers pass the test set to an early-stopping monitor, the model is effectively searching across training checkpoints for the one that minimizes loss on that specific test set. This is a classic form of data snooping: the test set is demoted from an "unbiased evaluation set" to a "model-selection set." The slight improvement reported on the test set then reflects survivor bias on data that are already known. In live operation, future market conditions cannot participate in choosing a model checkpoint in advance, so there is no guarantee that this overfit checkpoint will work out of sample.

42-C02 Complete Runnable Code, Bisection Root-Finding, and Sensitivity Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from quant_course.deep_learning import generate_ch42_panel_data, split_ch42_panel,
    ↪ fit_mlp, predict
from quant_course.ml_pipeline import backtest_predictions

def run_c02_experiment():
    x, raw_return, close, dates, assets = generate_ch42_panel_data(seed=4201)
    splits = split_ch42_panel(x, raw_return, dates, assets)

    mu_X, sigma_X = np.mean(splits.X_train, axis=0), np.std(splits.X_train, axis=0)
    mu_y, sigma_y = float(np.mean(splits.y_train)), float(np.std(splits.y_train))

    X_tr_s = (splits.X_train - mu_X) / sigma_X
    y_tr_s = (splits.y_train - mu_y) / sigma_y
    X_va_s = (splits.X_val - mu_X) / sigma_X
    y_va_s = (splits.y_val - mu_y) / sigma_y
    X_te_s = (splits.X_test - mu_X) / sigma_X

    mlp_res = fit_mlp(X_tr_s, y_tr_s, X_va_s, y_va_s, hidden_dim=8, lr=0.01,
                      batch_size=32, l2_reg=0.001, max_epochs=200, patience=15,
                      seed_init=4202, seed_shuffle=4203)
    pred_mlp = predict(X_te_s, mlp_res.W1, mlp_res.b1, mlp_res.W2, mlp_res.b2) *
    ↪ sigma_y + mu_y
```

```
    idx_test = pd.MultiIndex.from_product(
        [splits.test_dates, assets], names=["signal_time", "asset"]
    )
    pred_series = pd.Series(pred_mlp.ravel(), index=idx_test)

    # 1. Backtest at several cost levels (post hoc teaching exploration; units: bps)
    fee_grid = [0.0, 2e-4, 5e-4, 1e-3, 2e-3]
    grid_results = []
    for fee in fee_grid:
        bt = backtest_predictions(close, pred_series, rebalance_every=1, fee_rate=fee
    ↪ )
        ret = float(bt["metrics"]["total_return"])
        sharpe = float(bt["metrics"]["sharpe"])
        grid_results.append({"fee_bps": fee * 10000, "total_return": ret, "sharpe":
    ↪ sharpe})
        print(f"Fee {fee*10000:4.1f} bps: Return = {ret*100:6.2f}%, Sharpe = {sharpe
    ↪ :6.3f}")

    fig, ax = plt.subplots()
    ax.plot([row["fee_bps"] for row in grid_results],
            [row["sharpe"] for row in grid_results], marker="o")
    ax.axhline(0, color="gray", linestyle="--")
    ax.set(xlabel="One-way fee (bps)", ylabel="Annualized Sharpe",
           title="Constructed post-hoc cost sensitivity")
    plt.show()
    plt.close(fig)

    # 2. Find the breakeven cost where net Sharpe = 0 by bisection
    def sharpe_at_fee(f):
        bt = backtest_predictions(close, pred_series, rebalance_every=1, fee_rate=f)
        return float(bt["metrics"]["sharpe"])

    f_low, f_high = 0.0, 2e-4
    endpoints = [sharpe_at_fee(f_low), sharpe_at_fee(f_high)]
    assert np.isfinite(endpoints).all() and endpoints[0] > 0 > endpoints[1]
    for _ in range(30):
        f_mid = 0.5 * (f_low + f_high)
        s_mid = sharpe_at_fee(f_mid)
        if s_mid > 0:
            f_low = f_mid
        else:
            f_high = f_mid
    breakeven_bps = 0.5 * (f_low + f_high) * 10000
    print(f"\nNumerical Sharpe-Zero Fee Rate (Sharpe crosses 0): {breakeven_bps:.4f}
    ↪ bps")
    return grid_results, breakeven_bps

if __name__ == "__main__":
    run_c02_experiment()
```

Post hoc teaching-run values inherited from the frozen historical run (not rerun in this round):

- 0 bps (gross return): total return +1.55%, Sharpe = +0.501;
- 2 bps: total return -1.88%, Sharpe = -0.541;
- 5 bps: total return -6.80%, Sharpe = -2.101;
- 10 bps: total return -14.47%, Sharpe = -4.688;
- 20 bps: total return -27.96%, Sharpe = -9.779;
- **Bisection estimate of the breakeven one-way fee to numerical tolerance**: **0.9605 bps** (about 0.96 basis points).

Discussion:

1. **Turnover and fee sensitivity**: With annualized turnover as high as 360.2, the MLP strategy's net Sharpe falls quickly as the one-way fee rises. Net Sharpe crosses below zero when one-way cost reaches only 0.9605 bps;
2. **Protocol boundary**: The root found here is where annualized Sharpe is zero (mean daily return is zero); it is not the point where compounded total return is zero. After 30 bisection rounds, the fee-interval width is no more than $2/2^{30}$ bps. This 0/2/5/10/20 bps sensitivity analysis is a post-class extension and does not change the main protocol's frozen 0 bps and 10 bps benchmarks. This value describes only the fixed-proportional-cost tolerance of this constructed ledger; it is not capital capacity. No AUM, order-size, traded-volume, or market-impact model is present. Bisection requires finite, continuous Sharpe over the interval and opposite-signed endpoints. This run locates a crossing in that local interval; a global monotonic relationship cannot be inferred merely from changes in the Sharpe denominator. Real fees, slippage, and impact must be estimated separately by market, order, and capacity;
3. **Possible improvements**:
   - **Signal smoothing**: Apply an exponential moving average (EMA), such as $\alpha = 0.3$, to raw daily cross-sectional predictions to reduce sharp changes in target positions;
   - **Turnover-penalty loss**: Explicitly add a turnover penalty, $\lambda_{\text{turnover}} \sum_t \|w_t - w_{t-1}\|_1$, to the network's training objective. This is a research suggestion that has not been run and is not proven to improve this result. Actual weights must be defined using current and previous executions, prices, and NAV; a cross-time penalty must preserve temporal adjacency and cannot be applied directly to arbitrary neighboring rows in shuffled mini-batches. EMA has not been validated in this experiment either.

19. Further Reading

1. **Must Know (classic readings)**:
   - Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Read Chapter 6 closely (backpropagation and computation graphs) and §7.8 (early stopping as regularization).
   - Vaswani et al. (2017). *Attention Is All You Need*. Read §3.2 (scaled dot-product attention and causal masking) and §3.5 (positional encoding).
2. **Should Know (advanced recommendations)**:
   - Goodfellow et al., *Deep Learning*, Chapter 9 §§9.1–9.2. Understand the local connectivity, weight sharing, and cross-correlation reviewed for this chapter.
   - Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). *Learning to Forget: Continual Prediction with LSTM*. Understand why the forget gate became a core component of modern recurrent networks.
3. **Advanced (frontier research)**:
   - Goodfellow et al., *Deep Learning*, §§10.7 and 10.10.1. Compare long-term dependencies and the LSTM variants in the book; do not conflate them term by term with the modern equations in this chapter.
4. **Optional (further reference)**:
   - Scikit-learn official documentation: *Common pitfalls and recommended practices*. Review preprocessing leakage and time-series splitting.

20. References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Official online edition: Feedforward Networks §§6.2.2.1/6.5, Mini-batches and SGD §§8.1.2–8.1.3/8.3.1, Cross-correlation and equivariance §§9.1–9.2, and Long-term dependencies §10.7 and LSTM variants §10.10.1.
2. Gers, F. A., Schmidhuber, J., & Cummins, F. (2000). Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, 12(10), 2451–2471. Author PDF. The historical source check covered only the title, authors, and abstract; BIL20 inherits that reading scope; see the next item for the modern tanh-gate equations.
3. PyTorch Contributors. *torch.nn.LSTM*, official version 2.14 documentation. Modern six-equation form. Used only as the source for the equations; PyTorch was not installed or called in this chapter.
4. Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30. Author paper, arXiv:1706.03762. Sections §§3.2.1, 3.2.3, and 3.5 were read.
5. Scikit-learn Developers. Common pitfalls and recommended practices. The preprocessing-leakage section was read.

The historical reading scope, versions, and evidence are recorded in the shared source file `course/zh/chapters/ch42/source_notes.md`. BIL20 inherits those records and did not revisit the web sources. Suggested further reading does not mean that the full text was verified in this round.

Chapter 43: Bayesian Inference, Causal Identification, and Alternative Text Data

This chapter starts from frozen constructed results and retains the boundaries on template generalization and trust in summaries; no training or model call was performed.

Main thread: Prediction, probability updating, and intervention decisions in financial research have distinct mathematical foundations. Bayesian inference uses posterior probabilities to represent uncertainty with small samples and across groups; causal identification uses potential outcomes and DAGs to express confounding paths and identifies intervention effects under explicit assumptions; alternative data and large language models require feature extraction under strict contracts for point-in-time (PIT) eligibility, delayed ingestion, version revisions, and training-corpus cutoffs. Fitting a vocabulary across time, confusing retrospective smoothing with real-time prediction, or treating an LLM pretraining cutoff as historical PIT eligibility can all create fatal look-ahead bias. Prerequisites: Chapter 6: Probability and Chapter 7: Mathematical Statistics (conditional probability, Bayes' theorem, expectation, and variance); Chapter 8: Linear Regression (least squares and regularization); Chapter 12: Fundamental Quantitative Analysis and PIT (financial-statement releases and point-in-time availability); Chapter 22: Research Design (causal inference and confounding bias); and Chapter 31: Label Purging and Embargo and Chapter 38: Machine-Learning Pipelines (strict time-series validation and leakage-resistant features). By the end, you should be able to: derive the Beta-Binomial conjugate posterior and the joint predictive covariance of two future periods; calculate by hand partial-pooling shrinkage for a normal hierarchical mean with known hyperparameters and its posterior predictive variance; build a causal DAG and identify the ATE by stratified standardization under identification assumptions, calculate the reversal mechanism in Simpson's paradox, and diagnose violations of positivity; construct a rigorous after-hours earnings-call availability ledger and filter eligible versions under time-zone conversion and delayed ingestion; fit a TF-IDF vocabulary independently within the training window and compare text Ridge regression with a historical-mean baseline; calculate cosine similarity for dense word embeddings and illustrate sentiment reversal under negation; and design an offline LLM research ledger that includes prompt-version hashes and a review of training-corpus cutoffs.

1. Learning Objectives

1. **Master conjugate Bayesian updating and predictive distributions**: Starting from the prior and likelihood, derive the Beta-Binomial posterior parameters $(\alpha_{\text{post}}, \beta_{\text{post}})$ in full; derive the one-period future predictive mean $E[\tilde{y} \mid y]$ and the joint predictive probability for two future periods, $E[\tilde{y}_1\tilde{y}_2 \mid y] = E[p^2 \mid y]$; prove that two future Bernoulli trials are conditionally independent given p but correlated under the marginal posterior predictive distribution, with covariance exactly equal to the posterior variance $\text{Var}(p \mid y)$.
2. **Understand shrinkage through hierarchical models and partial pooling**: Using Section 5.4 of Gelman et al. (BDA3) as a reference, derive the precision-weighted shrinkage weight for each continuous-return asset group's posterior mean when hyperparameters (μ, τ^2, σ^2) are known: $w_j = \frac{n_j/\sigma^2}{1/\tau^2+n_j/\sigma^2}$. Calculate by hand how a small-sample group ($n = 4$) and a large-sample group ($n = 36$) shrink toward the whole-market prior mean $\mu = 0.0$, and derive the posterior predictive variance $\text{Var}_{\text{pred},j} = V_j+\sigma^2$, which includes uncertainty about the group and noise in a new observation. This chapter is limited to teaching with known hyperparameters; it does not claim to estimate unknown hyperparameters as empirical Bayes.
3. **Build causal graphs (DAGs) and a potential-outcomes framework**: Distinguish statistical association $P(Y \mid T = t)$ from causal intervention $P(Y \mid \text{do}(T = t))$; list the four assumptions for identifying the average treatment effect (ATE): consistency, conditional exchangeability, positivity/overlap, and the stable unit treatment value assumption (SUTVA, including no interference between units and no hidden treatment versions); calculate the Simpson's paradox example by hand, showing how an unadjusted difference in binary success rates of +0.38 (38 percentage points) becomes a negative observed difference after stratified standardization and, under the identification assumptions, an ATE of −0.10 (10 percentage points); stress that an adjusted association cannot be equated with unconditional true causality without the identification assumptions; and write an executable counterexample of collider bias and derive the negative covariance $-1/9$.
4. **Specify a point-in-time (PIT) contract for unstructured alternative data**: Build a multi-time-zone ledger containing `event_time`, `available_at`, `ingested_at`, `revision_time`, `signal_time`, `trade_time`, and `label_available_at`; use a U.S. after-hours earnings call at 16:05 to apply a 20-minute vendor delay and a 5-minute internal-ingestion delay, and work through daylight-saving-time (EST/EDT) and UTC conversions; implement exclusion of amended versions revised after the signal time and of records ingested too late; and distinguish public accessibility from authorization for commercial scraping, training, and redistribution.
5. **Build a leakage-resistant text-prediction pipeline aligned with its baseline**: Use 120 constructed earnings-call events generated from four repeated templates (with the `CONSTRUCTED_CALL_` prefix and an explicit teaching-only label); enforce time-series splits of 59 training documents, 29 validation documents, and 30 test documents (training and validation labels must mature strictly before the first signal time in the following block, `<`); fit the vocabulary and IDF independently within the training window and freeze them

before transforming validation and test documents; keep training features unchanged when unseen or out-of-vocabulary (OOV) terms appear in the test window; compare a global-mean baseline with TF-IDF + Ridge regression on the same test set; retain the negative low-SNR result (test MSE 2.3603×10^{-4} versus baseline 2.3274×10^{-4}, relative $R^2 = -0.0141$; time-series summary Spearman rank correlation +0.0763; cross-sectional IC is undefined because there is only one asset per day, and rank correlation is undefined for the constant-mean baseline); this chapter is prediction-only and specifies an interface to downstream portfolio accounting and fee calculations without inventing turnover from asset rotation.

6. **Understand the geometry beneath word embeddings and negation-based sentiment analysis**: Give the formal TF-IDF definition and a complete, step-by-step hand calculation for two teaching documents; use a hand-calculated 6×3 dense embedding matrix to explain angles between word vectors and mean pooling; and use a bag-of-words model (clarifying that bag-of-words representations do not assume statistical independence among words) to show errors under the negation terms "not" and "no," exposing the limits of unigram-frequency features and the need for n-grams or rule-based enhancements.
7. **Maintain a rigorous offline research ledger for large language models (LLMs)**: Create an offline prompt template, version control, and a SHA-256 check of the complete actual input text; clearly mark the model as not called (`not_run / unknown`) and use the pretraining cutoff only as a hypothetical counterexample; analyze why an LLM pretraining-corpus cutoff cannot establish verifiable historical PIT eligibility; explain why zero temperature and a seed do not eliminate hardware or underlying nondeterministic drift; and assess hallucinations, reproducibility, and fact-checking in quantitative research.

All run directories must be new paths that do not already exist. Use a different trial number when rerunning an example.

Completion criteria: exercises in Sections 15–17 can be derived or run independently; `code/ch43_bayesian_text.py` runs successfully in an independent output directory and generates all figures and the complete summary.

---

2. Why Quant Needs This

The companies and earnings-call scenarios below are constructed teaching examples, not established market facts. Conclusions may still fail if the prior is wrong, samples are dependent, or causal assumptions do not hold.

Quantitative researchers often face three common cognitive traps and modeling failures:

The first trap is the **illusion of point estimates with small samples**. In low-frequency macro-hedging or event-driven strategies (such as defaults, CEO departures, or major acquisitions), the historical sample may be tiny, perhaps only a dozen observations or fewer. Frequentist point estimates such as the sample mean can fluctuate sharply. If only four companies in an industry report positive earnings surprises and all four rise, the sample-frequency point estimate is 100%; this does not mean that frequentist inference asserts a 100% population win rate. A small-sample interval still reflects uncertainty. Bayesian methods provide an axiomatic framework for

updating uncertainty: a reasonable prior can shrink an extreme small-sample estimate toward the whole-market prior mean and yield a full posterior distribution and posterior predictive variance. Shrinkage can reduce sensitivity to small samples, but it does not guarantee stable optimizer settings; a misspecified prior can introduce bias.

The second trap is **treating a confounded association as an intervention effect**. In the constructed supplemental-call example in this chapter, the treatment group's overall target-attainment rate is 38 percentage points higher than the control group's. Companies with high analyst attention are more common in the treatment group and also have higher baseline attainment rates. After stratifying by attention, the observed difference is -10 percentage points within each stratum. This reversal shows how group weights affect the overall association. Interpreting the standardized difference as an ATE still requires the identification assumptions below; the table does not establish the true mechanism of management behavior or actual trading profit and loss.

The third trap is **look-ahead leakage in unstructured data and large-language-model applications**. As alternative data (news, research reports, calls, and social media) and LLMs become more common, text has become a central resource in quantitative research. Yet its time structure is fragile:

1. **Ingestion delay**: A news item is published after the close at 16:05, takes 20 minutes to reach the vendor, and another 5 minutes to enter the company's database. Before 16:30, the local quantitative research database cannot use this text. Generating a signal at 16:15 and assuming it could be executed after hours would amount to a time-traveling trade that is infeasible in live trading;
2. **Retrospective revisions**: Earnings-call transcripts are often corrected or supplemented by the issuer the following day. Backtesting a historical signal with the latest revised version leaks information about future corrections;
3. **Pretraining-corpus contamination**: LLM pretraining corpora are collected from the internet. Suppose a model's training cutoff is the end of 2023; its corpus could include reports published after events in 2022. The cutoff statement alone cannot prove that an output for a 2022 historical signal contains no future knowledge. Whether contamination actually occurred requires a dedicated experiment.

This chapter connects Bayesian inference with causal identification mathematically and implements a rigorous point-in-time text-processing pipeline, giving quantitative research a firm boundary around time and causality.

3. Intuition

We will build a clear mental picture from three examples in finance and statistics:

3.1 Bayesian Conjugate Updating and Partial Pooling of Asset Returns

- **Intuition for conjugate updating**: Let the win probability of a quantitative strategy be $p \in [0, 1]$. Before live trading, set $p \sim \text{Beta}(2, 2)$ based on prior knowledge of similar strategies across the market (prior expected win rate 0.5). If 8 of 10 constructed trades are profitable, the maximum-likelihood point estimate is 80%, but this point estimate does not express small-sample uncertainty. Under the prior specified here, Bayesian updating gives a posterior mean of approximately 71.4%, balancing prior knowledge with limited evidence.
- **Partial pooling of continuous asset returns**: Consider estimating continuous excess returns across industry or style groups. Let the center of the whole-market long-run unconditional excess-return prior be $\mu = 0.0$, between-group dispersion be $\tau = 0.02$, and the observation noise for an individual trade be $\sigma = 0.04$.
  - Group 1 is a newly listed technology stock with very little history ($n_1 = 4$) and sample mean $\bar{y}_1 = +0.03$ (+3%). Because the sample is small, the model assigns it a 50% shrinkage weight, pulling the posterior mean strongly back to +0.015;
  - Group 2 is a mature large-cap blue chip with a substantial history ($n_2 = 36$) and sample mean $\bar{y}_2 = +0.01$ (+1%). Given the stronger evidence, the model assigns the sample mean a 90% weight, yielding a posterior mean of +0.009. Partial pooling automatically balances "be cautious about groups with little evidence" against "respect groups supported by strong data."

3.2 Simpson's Paradox and Confounding by Analyst Attention

This constructed example contains 200 company records. The treatment is "a supplemental after-hours earnings call with updated guidance" ($T = 1$), and the outcome is whether the next-day opening excess return meets its target ($Y \in \{0, 1\}$). In the observed table, the target-attainment rates are 64% for the treatment group and 26% for the control group, an unadjusted difference of 38 percentage points. This is only an association. Stratify by analyst attention Z. In the high-attention stratum, 90 treated companies have 63 successes and 10 control companies have 8 successes, so the within-stratum difference is $0.70 - 0.80 = -0.10$. In the low-attention stratum, 10 treated companies have 1 success and 90 control companies have 18 successes, so the difference is also $0.10 - 0.20 = -0.10$. Ninety percent of the treatment group comes from the high-attention stratum, while 90% of the control group comes from the low-attention stratum. The overall association therefore has the opposite sign from the within-stratum differences. Using target-population weights of 50% for each stratum gives a standardized difference of -0.10. It can be interpreted as the target-population ATE only under consistency, conditional exchangeability, positivity, and the stated SUTVA assumptions. The four-cell table itself cannot verify those assumptions and is not evidence of a real market intervention.

3.3 Time Travel in Earnings-Call Text

A listed company ends an after-hours call at 16:05. You find its text in a historical database and use it to calculate a signal for a purchase at the next day's 09:30 open. The timeline appears to move forward, but the database text may be a version revised at 10:00 the following morning after review by the company's legal and investor-relations teams (for example, to correct a misspoken figure). If you calculate a sentiment score from that text at 09:00, your algorithm has effectively traveled through time. Freeze the historical ingestion ledger at the signal time and reject future versions strictly.

---

4. Mathematical Foundations

This chapter follows the course notation standard in `notation.md`. The prerequisite symbols, dimensions, and time contract are:

Symbol	Meaning	Dimension / unit	Conditions / interpretation
p	Success probability or parameter of a binary event in one trial	Scalar, dimensionless, $[0,1]$	Bernoulli-trial parameter
α, β	Beta-distribution hyperparameters	Positive real scalars, $\mathbb{R}_{++}$	Pseudo-successes and pseudo-failures
θ_j	Mean latent variable for group j in a hierarchical model	Scalar, same units as observations	Follows common prior $\mathcal{N}(\mu, \tau^2)$
μ, τ	Group prior mean and prior standard deviation (hyperparameters)	Scalar, return units	Restricted to known constants for this chapter's teaching example
σ	Observation-noise standard deviation for one sample	Scalar, return units	Known constant, $\sigma > 0$
$n_j, \bar{y}_j$	Sample size and within-group sample mean for group j	Integer, real number	$\bar{y}_j = \frac{1}{n_j}\sum_{i=1}^{n_j} y_{j,i}$
w_j	Posterior shrinkage weight for group j	Scalar, $[0,1]$	$w_j = \frac{n_j/\sigma^2}{1/\tau^2 + n_j/\sigma^2}$
V_j	Posterior variance of group mean θ_j	Scalar, squared return units	$1/V_j = 1/\tau^2 + n_j/\sigma^2$
Z, T, Y	Confounder, treatment variable, outcome variable	Discrete or continuous scalars	$T \in \{0,1\}, Y \in \{0,1\}$ in the binary case
$Y(1), Y(0)$	Potential outcomes	Scalars	Possible outcomes under hypothetical intervention $T = 1$ or $T = 0$
ATE	Average treatment effect	Scalar, outcome units	$\text{ATE} = E[Y(1) - Y(0)]$
event_time	Time when the real-world event occurs	UTC timestamp	Financial release, earnings-call time
available_at	Earliest time the external vendor can provide the text	UTC timestamp	Includes transmission and transcoding delay

Symbol	Meaning	Dimension / unit	Conditions / interpretation
ingested_at	Earliest time the local database has completed ingestion	UTC timestamp	Includes local ETL delay
signal_time	Decision time when the strategy generates a buy or sell signal	UTC timestamp	Availability and ingestion must both be no later than the signal
trade_time	Time an order is actually matched on the exchange	UTC timestamp	Strictly later than the signal time
label_available_at	Time a return target or evaluation label is fully mature and knowable	UTC timestamp	Training labels must be available before the fitting decision

5. Formal Definitions

5.1 Conjugate Bayesian Updating

The prior $p(\theta)$ is a probability distribution for parameter θ before observing the data. The likelihood $p(y \mid \theta)$ is a function of the parameter for fixed data y; it is usually not a normalized density over the parameter. Bayes' theorem combines them to form the posterior:

$$p(\theta \mid y) = \frac{p(y \mid \theta)p(\theta)}{p(y)}, \qquad p(y)$$
$$= \int p(y \mid \theta)p(\theta)\, d\theta > 0.$$

The denominator is the marginal likelihood, which normalizes the posterior to integrate to 1; for a discrete parameter, replace the integral with a sum. This does not conflate the probability of the data with the probability of the parameter.

In Bayesian statistics, a prior distribution $p(\theta)$ is called a **conjugate prior** for likelihood $p(y \mid \theta)$ when the posterior $p(\theta \mid y)$ belongs to the same distribution family as the prior. For a binomial likelihood $y \sim \text{Binomial}(n, p)$, the conjugate prior is the Beta distribution $\text{Beta}(\alpha, \beta)$. The posterior is also Beta:

$$p \mid y \sim \text{Beta}(\alpha + y, \beta + n - y)$$

5.2 Posterior Predictive Distribution

The posterior predictive distribution is the marginal probability distribution of a new future observation $\tilde{y}$ after observing existing data y. It is obtained by integrating over posterior uncertainty about the model parameter θ:

$$p(\tilde{y} \mid y) = \int_{\Theta} p(\tilde{y} \mid \theta)p(\theta \mid y)\, d\theta$$

Key result: For two future Bernoulli trials $(\tilde{y}_1, \tilde{y}_2)$ that share the same unknown parameter p, they are **conditionally independent** given p: $P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid p) = p^2$. After integrating out p, they are **correlated** under the marginal posterior predictive distribution:

$$\begin{aligned} P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid y) &= E[p^2 \mid y] \\ &= \mathrm{Var}(p \mid y) + (E[p \mid y])^2 \neq (E[p \mid y])^2 \end{aligned}$$

When the posterior concentration is finite and both Beta parameters are positive, the covariance is exactly $\mathrm{Var}(p \mid y) > 0$. It arises from uncertainty about the shared parameter; it does not mean one future outcome physically causes the other.

5.3 Normal Hierarchical Model and Partial Pooling

The closed form assumes independent group priors conditional on the known hyperparameters, and conditionally independent, homoskedastic normal observations within each group given its θ_j. New-observation noise is independent of θ_j and the old observations. With within-group time dependence or heteroskedasticity, σ^2/n_j and these fixed precision weights cannot be used directly. Consider J asset or analyst groups. The parameter for group j has a common prior $\theta_j \sim \mathcal{N}(\mu, \tau^2)$, and within-group observations satisfy $y_{j,i} \sim \mathcal{N}(\theta_j, \sigma^2)$. The sampling variance of the sample mean $\bar{y}_j$ is $\sigma_j^2 = \sigma^2/n_j$. When the hyperparameters (μ, τ^2, σ^2) are **known**, the posterior mean of group mean θ_j has the following closed-form partial-pooling solution (Equation 5.17 in Gelman et al., BDA3):

$$\begin{aligned} \hat{\theta}_j &= (1 - w_j)\mu + w_j \bar{y}_j, \quad w_j \\ &= \frac{n_j/\sigma^2}{1/\tau^2 + n_j/\sigma^2} = \frac{n_j}{n_j + \sigma^2/\tau^2} \end{aligned}$$

The posterior precision is the sum of the prior and sample precisions: $1/V_j = 1/\tau^2 + n_j/\sigma^2$. The posterior predictive variance of a new observation $\tilde{y}_j$ from this group is $\mathrm{Var}_{\mathrm{pred},j} = V_j + \sigma^2$.

5.4 Potential Outcomes and the Average Treatment Effect (ATE)

Let the binary treatment variable be $T \in \{0, 1\}$. For each unit, define a pair of potential outcomes:

- $Y(1)$: the potential outcome for the unit if it receives the treatment $(T = 1)$;
- $Y(0)$: the potential outcome for the unit if it does not receive the treatment $(T = 0)$. The fundamental difficulty in causal inference is that only one of the two potential outcomes can be observed for any unit in practice: $Y = T \cdot Y(1) + (1 - T) \cdot Y(0)$. The **average treatment effect (ATE)** for the target population is defined as:

$$\mathrm{ATE} = E[Y(1) - Y(0)]$$

5.5 Four Causal Identification Assumptions

Following Hernán & Robins (2020), statistical contrasts in observed data can be interpreted as a causal ATE only if the following four foundational assumptions hold:

1. **Consistency**: If a unit actually receives treatment $T = t$, its observed outcome Y equals its potential outcome $Y(t)$; that is, $Y = Y(T)$.
2. **Conditional exchangeability / no unmeasured confounding**: Conditional on adequately controlling for confounder covariates Z, the potential outcomes are conditionally independent of treatment assignment: $(Y(0), Y(1)) \perp T \mid Z$. In a DAG, this means that controlling for Z blocks all backdoor paths connecting T and Y.
3. **Positivity / overlap**: Within every confounder stratum z in the target population (that is, where $P(Z = z) > 0$), the probabilities of receiving treatment and control must both be strictly positive:
$$0 < P(T = 1 \mid Z = z) < 1$$
Positivity means every stratum has a positive probability of receiving either treatment; it **does not** mean that variables follow a normal distribution. If everyone in a stratum receives treatment or everyone receives control, the counterfactual outcome for the missing arm cannot be identified from the data. The algorithm must report an explicit error and reject the calculation; it must never fill the missing value silently.
4. **Stable unit treatment value assumption (SUTVA)**: This includes two closely related dimensions:
   - **No interference**: One unit's treatment does not affect another unit's potential outcome (for example, one stock having an earnings call and being bought by a strategy does not directly change the potential excess return of a stock without a call in the parallel world);
   - **No treatment variation / no hidden versions of treatment**: For a stated treatment level $T = t$, there are no unobserved versions of treatment that lead to different potential outcomes (for example, whether a CEO or CFO leads the call, or how quickly they speak, does not make the treatment $T = 1$ causally different under the declared treatment definition).

5.6 Stratified Standardization

When conditional exchangeability, positivity, consistency, and SUTVA all hold, the target-population causal ATE can be calculated by weighting stratum-specific conditional effects by the marginal distribution $P(Z)$ in the target population:

$$\text{ATE} = \sum_{z} \Big(E[Y \mid T = 1, Z = z] - E[Y \mid T = 0, Z = z] \Big) P(Z = z)$$

Emphasize that the numerical contrast from stratified standardization can be interpreted as the target-population causal ATE only when all four identification assumptions above hold. With unmeasured confounding, a SUTVA violation, or sample-selection bias, the adjusted statistical association remains a conditional association under a particular adjustment; it cannot be unconditionally claimed as a true causal effect.

5.7 Point-in-Time (PIT) Availability and Delay Contract

Inputs must have nonmissing, time-zone-aware values for event, revision, availability, ingestion, signal, trade, and label times. Under valid inputs and version semantics, the teaching selector requires one decision cutoff per `source_id` and positive, increasing integer version identifiers. Duplicate versions, contradictory times, and missing text cause errors; versions ingested too late are filtered out. License fields store source notes only; the program does not determine legal authorization. These checks also run at the direct training entry point. Alternative data must record the complete lifecycle timestamps for every version:

- Eligibility rule (iff, under the valid-input and version-semantics assumptions above): a version is eligible at the decision time if and only if $\max(\text{available_at}, \text{ingested_at}) \leq \text{signal_time}$.
- Version replacement: If an event has several revised versions, use only the latest compliant version ingested by the signal cutoff; never overwrite history retrospectively.

5.8 Bag-of-Words (BoW) and TF-IDF Feature Representation

- A bag-of-words model treats a text as an unordered multiset of term occurrences, ignoring word order, grammar, and syntactic dependencies.
- **Important clarification**: Bag-of-words feature representation itself **does not assume that words are statistically independent**. Treating feature conditional distributions as independent is an assumption of particular models such as the Naive Bayes classifier, not of the BoW vector representation. The fundamental limitation of BoW is that it discards order, so it cannot natively capture modifiers, phrase combinations, or polarity reversals caused by negation.
- Term frequency–inverse document frequency (TF-IDF) downweights common function words and gives greater weight to domain-specific information terms, producing continuous dense or sparse vectors that reflect term importance.

5.9 Offline LLM Research Ledger and Defenses Against Irreproducibility

Use of large language models (LLMs) in quantitative research should be governed by an offline-auditable ledger that verifies a cryptographic digest:

- Compute a SHA-256 cryptographic hash (digest) of the complete prompt (prompt template plus actual input text);
- A hash checks only whether a digest matches and requires an independently trusted expected digest. It is not encryption or immutable timestamped evidence. If both the input and the recorded digest change, a local recomputation may still match. An independently stored trusted digest can serve as a comparison, but this teaching code does not provide a trusted custody mechanism for that digest.
- Explicitly record the model call status as not called (`model_status: not_run`) and the model identifier as `not_run / unknown`; use the training-corpus cutoff only to test a hypothetical counterexample;

- Record reproducibility limits explicitly: `temperature=0` and a fixed seed do not guarantee bit-for-bit reproducibility across hardware architectures or inference-engine versions; underlying model updates or floating-point drift can materially change text features.

All causal tables in this chapter are constructed association data. The generator does not observe potential outcomes and cannot test identification assumptions such as no unmeasured confounding. A standardized difference can be interpreted as an ATE only if those assumptions hold.

6. Derivations

6.1 Deriving the Beta-Binomial Conjugate Posterior and Predictive Covariance

Step 1: Prior density and likelihood. Let parameter $p \in [0,1]$ follow a Beta prior:

$$f(p) = \frac{1}{\mathrm{B}(\alpha,\beta)} p^{\alpha-1}(1-p)^{\beta-1} \propto p^{\alpha-1}(1-p)^{\beta-1}$$

After observing y successes in n Bernoulli trials that are conditionally independent and identically distributed given the shared p, the binomial likelihood is:

$$L(y \mid p) = \binom{n}{y} p^{y}(1-p)^{n-y} \propto p^{y}(1-p)^{n-y}$$

Step 2: Calculate the posterior density. By Bayes' theorem, the posterior density is proportional to the product of the prior and likelihood:

$$f(p \mid y) \propto f(p)L(y \mid p) \propto p^{\alpha-1}(1-p)^{\beta-1} \cdot p^{y}(1-p)^{n-y} = p^{(\alpha+y)-1}(1-p)^{(\beta+n-y)-1}$$

Comparing with the kernel of a Beta distribution, we identify the posterior distribution directly:

$$p \mid y \sim \mathrm{Beta}(\alpha_{\text{post}}, \beta_{\text{post}}), \quad \alpha_{\text{post}} = \alpha + y, \quad \beta_{\text{post}} = \beta + n - y$$

Step 3: Posterior mean and variance. Using the first and second moments of the Beta distribution:

$$E[p \mid y] = \frac{\alpha_{\text{post}}}{\alpha_{\text{post}} + \beta_{\text{post}}}$$

$$\mathrm{Var}(p \mid y) = \frac{\alpha_{\text{post}}\beta_{\text{post}}}{(\alpha_{\text{post}} + \beta_{\text{post}})^2(\alpha_{\text{post}} + \beta_{\text{post}} + 1)}$$

Step 4: Predictive probabilities and covariance. For a single future trial $\tilde{y} \in \{0, 1\}$:

$$P(\tilde{y} = 1 \mid y) = \int_0^1 P(\tilde{y} = 1 \mid p) f(p \mid y)\, dp$$
$$= \int_0^1 p f(p \mid y)\, dp = E[p \mid y]$$

For two future trials $(\tilde{y}_1, \tilde{y}_2)$:

$$P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid y) = \int_0^1 P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid p) f(p \mid y)\, dp$$
$$= \int_0^1 p^2 f(p \mid y)\, dp = E[p^2 \mid y]$$

Using the variance identity $E[p^2 \mid y] = \mathrm{Var}(p \mid y) + (E[p \mid y])^2$, substitute the second raw moment of the Beta distribution:

$$E[p^2 \mid y] = \frac{\alpha_{\text{post}}(\alpha_{\text{post}} + 1)}{(\alpha_{\text{post}} + \beta_{\text{post}})(\alpha_{\text{post}} + \beta_{\text{post}} + 1)}$$

The covariance of success in the two periods is:

$$\begin{aligned}\mathrm{Cov}(\tilde{y}_1, \tilde{y}_2 \mid y) &= P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid y) \\ &\quad - P(\tilde{y}_1 = 1 \mid y) P(\tilde{y}_2 = 1 \mid y) \\ &= E[p^2 \mid y] - (E[p \mid y])^2 = \mathrm{Var}(p \mid y) > 0\end{aligned}$$

This proves the result. Under the finite, positive-parameter conditions above, the two trials are conditionally independent given p but positively correlated after integration. If the first-period outcome is then observed to be a success, the posterior for p updates and raises the conditional prediction of success in the second period. This is information updating, not a causal effect between the outcomes. The correlation in this example is $1/(10 + 4 + 1) = 1/15$.

6.2 Deriving Shrinkage for a Normal Hierarchical Mean (Partial Pooling)

Assumptions: Retain the independent group priors and conditionally independent, homoskedastic within-group observations specified in §5.3. Hyperparameters μ, τ^2, σ^2 are known. Group j has latent mean $\theta_j \sim \mathcal{N}(\mu, \tau^2)$, and within-group observations $y_{j,1}, \ldots, y_{j,n_j} \sim \mathcal{N}(\theta_j, \sigma^2)$. The sample mean $\bar{y}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} y_{j,i}$ satisfies $\bar{y}_j \mid \theta_j \sim \mathcal{N}(\theta_j, \sigma^2/n_j)$.

Step 1: Combine precisions under conjugate updating. The sampling precision of the group mean is n_j/σ^2, and the prior precision is $1/\tau^2$. By conjugacy of the normal distribution, the log posterior density is quadratic:

$$\begin{aligned}\ln p(\theta_j \mid \bar{y}_j) &= -\frac{1}{2\tau^2}(\theta_j - \mu)^2 - \frac{n_j}{2\sigma^2}(\bar{y}_j - \theta_j)^2 \\ &+ \text{const}\end{aligned}$$

Expand the coefficient of the quadratic term:

$$\begin{aligned}&-\frac{1}{2}\theta_j^2\left(\frac{1}{\tau^2} + \frac{n_j}{\sigma^2}\right) \\ &+ \theta_j\left(\frac{\mu}{\tau^2} + \frac{n_j\bar{y}_j}{\sigma^2}\right) \\ &+ \text{const}\end{aligned}$$

Read off the posterior precision $1/V_j$ and posterior variance V_j:

$$\begin{aligned}\frac{1}{V_j} &= \frac{1}{\tau^2} + \frac{n_j}{\sigma^2} \implies V_j \\ &= \frac{1}{\frac{1}{\tau^2} + \frac{n_j}{\sigma^2}}\end{aligned}$$

Step 2: Decompose the posterior mean into weighted terms. Dividing the linear coefficient by the quadratic coefficient gives the posterior mean $\hat{\theta}_j$:

$$\begin{aligned}\hat{\theta}_j &= \frac{\frac{\mu}{\tau^2} + \frac{n_j\bar{y}_j}{\sigma^2}}{\frac{1}{\tau^2} + \frac{n_j}{\sigma^2}} \\ &= \frac{1/\tau^2}{1/\tau^2 + n_j/\sigma^2}\mu + \frac{n_j/\sigma^2}{1/\tau^2 + n_j/\sigma^2}\bar{y}_j\end{aligned}$$

Define the shrinkage weight w_j:

$$w_j = \frac{n_j/\sigma^2}{1/\tau^2 + n_j/\sigma^2} = \frac{n_j}{n_j + \sigma^2/\tau^2}$$

The posterior mean is then a convex combination of the prior mean and sample mean:

$$\hat{\theta}_j = (1 - w_j)\mu + w_j\bar{y}_j$$

Step 3: Posterior predictive variance. For a new observation from the group, $\tilde{y}_j = \theta_j + \tilde{\epsilon}$, where $\tilde{\epsilon} \sim \mathcal{N}(0, \sigma^2)$ and is independent of θ_j and the old observations:

$$\begin{aligned}\operatorname{Var}(\tilde{y}_j \mid y) &= \operatorname{Var}(\theta_j \mid y) \\ &+ \operatorname{Var}(\tilde{\epsilon}) = V_j + \sigma^2\end{aligned}$$

This proves the result.

6.3 Deriving Simpson's Reversal and Causal Standardization

Let the binary confounder be $Z \in \{1, 2\}$, the binary treatment be $T \in \{0, 1\}$, and the binary outcome be $Y \in \{0, 1\}$.

Step 1: Unadjusted association difference. By the law of total probability, the unadjusted conditional expectation is:

$$E[Y \mid T = t] = \sum_z E[Y \mid T = t, Z = z] P(Z = z \mid T = t)$$

Therefore, the unadjusted difference is:

$$\begin{aligned} E[Y \mid T = 1] - E[Y \mid T = 0] = &\sum_z E[Y \mid T = 1, Z = z] P(Z = z \mid T = 1) \\ &- \sum_z E[Y \mid T = 0, Z = z] P(Z = z \mid T = 0) \end{aligned}$$

Note that $P(Z = z \mid T = 1) \neq P(Z = z \mid T = 0)$; a severe imbalance in weights can produce Simpson's reversal.

Step 2: Identification derivation for causal standardization. By consistency and conditional exchangeability $(Y(t) \perp T \mid Z)$:

$$E[Y(t) \mid Z = z] = E[Y(t) \mid T = t, Z = z] = E[Y \mid T = t, Z = z]$$

Take the marginal expectation over the target population $P(Z)$:

$$E[Y(t)] = \sum_z E[Y(t) \mid Z = z] P(Z = z) = \sum_z E[Y \mid T = t, Z = z] P(Z = z)$$

Under the identification assumptions above and positivity in the target population, subtracting gives the causal average treatment effect:

$$\text{ATE} = E[Y(1)] - E[Y(0)] = \sum_z \Big(E[Y \mid T = 1, Z = z] - E[Y \mid T = 0, Z = z] \Big) P(Z = z)$$

In this equation, both terms use the **same** marginal target-population weights $P(Z = z)$ in every stratum. Under the stated identification assumptions, this removes the confounding bias caused by Z in this DAG.

6.4 Formal TF-IDF Mathematics and a Step-by-Step Calculation for Two Documents

Step 1: Formal definition. Let the corpus contain $|D|$ documents and have a vocabulary of size M.

1. **Term frequency (TF)**: Let $f_{t,d}$ denote the number of occurrences of term t in document d. Standard term frequency uses the raw count, $\text{tf}(t, d) = f_{t,d}$.

2. **Smoothed inverse document frequency (IDF)**: To avoid division by zero and excessive weight on rare terms that occur only once, use the standard scikit-learn smoothed log formula:
$$\mathrm{idf}(t, D) = \ln\left(\frac{1 + |D|}{1 + \mathrm{df}(t)}\right) + 1$$
Here, $\mathrm{df}(t)$ is the number of documents in the corpus that contain term t.
3. **Weighted vector and L2 normalization**:
$$x(t, d) = \mathrm{tf}(t, d) \times \mathrm{idf}(t, D)$$
To reduce differences in document-length scale, apply L2 unit normalization when $\|\mathbf{x}(d)\|_2 > 0$:
$$\begin{aligned}\mathbf{v}(d) &= \frac{\mathbf{x}(d)}{\|\mathbf{x}(d)\|_2} \\ &= \frac{\mathbf{x}(d)}{\sqrt{\sum_{t=1}^{M} x(t, d)^2}}\end{aligned}$$

If a document is empty or all its terms are OOV, this implementation retains a zero vector and does not divide by zero; cosine similarity between a zero vector and another vector is undefined.

Step 2: Hand calculation for two teaching documents. Suppose the training corpus contains only two miniature documents:

- Document 1: d_1 = "growth profit"
- Document 2: d_2 = "growth loss" The corpus contains $|D| = 2$ documents.

1. **Extract the vocabulary and term frequencies**: The alphabetically sorted vocabulary is: `["growth", "loss", "profit"]` ($M = 3$).
   - Term-frequency vector for d_1: $\mathrm{tf}_1 = [1, 0, 1]$
   - Term-frequency vector for d_2: $\mathrm{tf}_2 = [1, 1, 0]$
2. **Calculate document frequency** df **and smoothed IDF** idf:
   - The term `"growth"` appears in both d_1 and d_2, so $\mathrm{df} = 2$:
$$\begin{aligned}\mathrm{idf}(\text{"growth"}) &= \ln\left(\frac{1 + 2}{1 + 2}\right) \\ &+ 1 = \ln(1) + 1 = 0.0 + 1.0 = 1.000000\end{aligned}$$
   - The term `"loss"` appears only in d_2, so $\mathrm{df} = 1$:
$$\begin{aligned}\mathrm{idf}(\text{"loss"}) &= \ln\left(\frac{1 + 2}{1 + 1}\right) \\ &+ 1 = \ln(1.5) + 1 \approx 0.405465 + 1.0 = 1.405465\end{aligned}$$
   - The term `"profit"` appears only in d_1, so $\mathrm{df} = 1$:
$$\begin{aligned}\mathrm{idf}(\text{"profit"}) &= \ln\left(\frac{1 + 2}{1 + 1}\right) \\ &+ 1 = \ln(1.5) + 1 \approx 0.405465 + 1.0 = 1.405465\end{aligned}$$
3. **Calculate the unnormalized feature vector** $\mathbf{x} = \mathrm{tf} \times \mathrm{idf}$:
   - $\mathbf{x}_1 = [1 \times 1.0, 0 \times 1.405465, 1 \times 1.405465] = [1.0, 0.0, 1.405465]$

- Magnitude $\|\mathbf{x}_1\|_2 = \sqrt{1.0^2 + 0^2 + 1.405465^2} = \sqrt{1.0 + 1.975332} = \sqrt{2.975332} \approx 1.724915$
- Normalized vector $\mathbf{v}_1$:

$$\mathbf{v}_1 = \left[\frac{1.0}{1.724915}, 0.0, \frac{1.405465}{1.724915}\right] \approx [0.579739, 0.000000, 0.814802]$$

- $\mathbf{x}_2 = [1 \times 1.0, 1 \times 1.405465, 0 \times 1.405465] = [1.0, 1.405465, 0.0]$
- Magnitude $\|\mathbf{x}_2\|_2 \approx 1.724915$
- Normalized vector $\mathbf{v}_2$:

$$\mathbf{v}_2 = \left[\frac{1.0}{1.724915}, \frac{1.405465}{1.724915}, 0.0\right] \approx [0.579739, 0.814802, 0.000000]$$

4. **Calculate cosine similarity between the two documents**:

$$\begin{aligned}\cos(\mathbf{v}_1, \mathbf{v}_2) &= \mathbf{v}_1 \cdot \mathbf{v}_2 \\ &= 0.579739^2 + 0.0 \times 0.814802 + 0.814802 \times 0.0 \\ &= 0.579739^2 \approx 0.336097\end{aligned}$$

Although the two documents share the common term `"growth"`, the feature terms `"profit"` and `"loss"` receive higher IDF weights and are orthogonal to one another. This lowers the cosine similarity after normalization to approximately 0.3361.

6.5 Deriving Dense Word-Embedding Geometry and Mean Pooling

Let each word map to a unit vector in a K-dimensional Euclidean space, $\mathbf{e}(w) \in \mathbb{R}^K$, with $\|\mathbf{e}(w)\|_2 = 1$.

1. **Mean pooling for a phrase or sentence**: For a text sequence $S = (w_1, w_2, \ldots, w_L)$ containing L terms, the unnormalized dense sentence representation is the simple arithmetic mean of its word vectors:

$$\mathbf{s} = \frac{1}{L} \sum_{l=1}^{L} \mathbf{e}(w_l)$$

2. **Projection onto the unit sphere** (requires $L > 0$ and $\|\mathbf{s}\|_2 > 0$):

$$\mathbf{u}_S = \frac{\mathbf{s}}{\|\mathbf{s}\|_2}$$

Even unit word vectors can cancel: the mean of $[1, 0]$ and $[-1, 0]$ is zero and cannot be normalized. Empty text also requires an explicit policy. This example is nonzero; a general pipeline should reject a zero vector or retain it explicitly and mark its cosine as undefined.

3. **Semantic similarity between two text passages** (both normalized vectors must exist):

$$\mathrm{Sim}(S_A, S_B) = \mathbf{u}_{S_A} \cdot \mathbf{u}_{S_B}$$

Compared with sparse, high-dimensional TF-IDF, dense word embeddings map synonyms (such as `"growth"` and `"expansion"`) to nearby regions of a hypersphere, avoiding the complete orthogonality of unigram bag-of-words features. In financial applications, however, generic embeddings without domain-specific fine-tuning may incorrectly judge antonyms (such as `"profit"` and `"loss"`) to be highly similar because they often occur in the same grammatical contexts. This is a key reason to audit embedding quality carefully in quantitative finance.

7. Financial Interpretation

7.1 How Does Bayesian Shrinkage Express Uncertainty in Portfolio Inputs?

In quantitative asset allocation (for example, the Black-Litterman model or multi-factor return forecasts), directly using historical sample means in mean-variance optimization is a well-known "error maximizer." Historical sample means are heavily affected by short-term, market-specific shocks. The financial interpretation of a normal hierarchical model (partial pooling) is that **all assets in the market share a baseline long-run risk-premium mean μ, but are heterogeneous in the cross-section, with variance τ^2.** For a small-cap stock with few disclosed observations ($n_j = 4$), the data contain little information, so the model shrinks its estimate strongly toward the market center (shrinkage weight $w_j = 0.5$). For a large-cap stock with substantial data ($n_j = 36$), the model gives more weight to its own sample ($w_j = 0.9$). In posterior predictive variance $\text{Var}_{\text{pred}} = V_j + \sigma^2$, a small sample increases uncertainty about the group mean, V_j; the future observation noise σ^2 is the same for both groups. The hyperparameters here are known and group data are not used to estimate shared hyperparameters; this differs from empirical Bayes or a full hierarchical model with unknown hyperparameters. This teaching example does not demonstrate improved live returns or optimization stability.

7.2 Confounding and Adverse Selection in Market-Maker Strategies

Consider a possible market-microstructure mechanism: the association between active order flow T and subsequent price change Y may be affected by trader information and order size, among other factors Z. The overall correlation cannot be interpreted directly as the causal price impact of a buy order. Associations stratified by informed and uninformed traders also require real data and an identification model. This chapter does not estimate actual price impact for either group and cannot support a claim that one group has a smaller impact.

7.3 Alternative-Data Delays and Backtest Vanity

When reporting alternative-data backtests to management, quantitative teams can easily show a Sharpe ratio that is too high. The source is often "timestamp fabrication": although news or text appeared at 16:05, vendor parsing may queue during network disruptions, or internal data-engineering ETL may face an after-hours concurrency bottleneck. Requiring both availability and ingestion times to be no later than the signal is necessary for the text to be available; trade timing, labels, sample selection, and model validity still require separate checks.

7.4 News Updates and Earnings-Call Revisions: Two Timing Scenarios

Constructed news item `CONSTRUCTED_NEWS_001` is published at 09:12 EDT on 2022-03-14, available from the vendor at 09:13, and ingested locally at 09:14. A 09:15 signal may use v1. Version v2, published at 09:16 and ingested at 09:18, cannot overwrite that historical signal even if it is more accurate. In UTC these times are 13:12, 13:13, 13:14, 13:15, 13:16, and 13:18 on the same date; this differs from the after-hours, cross-day earnings-call scenario in §8. The standalone code below reuses the complete field structure and changes only the constructed news record; it does not add the record to the frozen main experiment:

```
import pandas as pd
from quant_course.bayesian_text import (
    generate_constructed_text_dataset,
    filter_pit_eligible_documents,
)
base, _ = generate_constructed_text_dataset()
v1 = base.iloc[[0]].copy()
v1["source_id"] = "CONSTRUCTED_NEWS_001"
times = {
    'event_time': '2022-03-14T13:12:00Z',
    'revision_time': '2022-03-14T13:12:00Z',
    'available_at': '2022-03-14T13:13:00Z',
    'ingested_at': '2022-03-14T13:14:00Z',
    'signal_time': '2022-03-14T13:15:00Z',
    'trade_time': '2022-03-14T13:30:00Z',
    'label_start': '2022-03-14T13:30:00Z',
    'label_end': '2022-03-15T13:30:00Z',
    'label_available_at': '2022-03-15T13:30:00Z',
}
for field, stamp in times.items():
    v1[field] = pd.Timestamp(stamp)
v1["text"] = "constructed preliminary revenue growth report"
v2 = v1.copy()
v2["version_id"] = 2
v2["revision_time"] = pd.Timestamp("2022-03-14T13:16:00Z")
v2["available_at"] = pd.Timestamp("2022-03-14T13:17:00Z")
v2["ingested_at"] = pd.Timestamp("2022-03-14T13:18:00Z")
v2["text"] = "constructed corrected revenue decline report"
selected = filter_pit_eligible_documents(pd.concat([v1, v2], ignore_index=True))
assert selected["version_id"].tolist() == [1]
print(selected[["source_id", "version_id", "text"]])
```

Whether a news vendor preserves the original version and whether an earnings-call transcript is retrospectively corrected must be verified with the specific source. PIT filtering cannot recover an old version that was never archived. Training-window TF-IDF also cannot fix the error of using a future revision as historical input.

The Bayesian example also has model boundaries: the binomial update assumes trials are conditionally independent given p and that "success" is defined consistently. Dependent trades

cannot simply be treated as independent coin flips. Prior sensitivity can be calculated by hand: for the same 8/10 observations, the posterior mean is 9/12=0.75 under Beta(1,1) and 28/50= 0.56 under Beta(20,20). A stronger prior is harder to move with ten observations, but is not necessarily more accurate.

Chapter 43 complete numerical example: all steps and conclusions

8. Numerical Example

This section inherits constructed results from the historical A8-C frozen protocols `qa/a8/ch42_ch43_batch/EXPERIMENT_PROTOCOL.json` and `EXPERIMENT_ADDENDUM_v1.json` in the same directory. Historical hand/code checks and supplemental demonstrations retain their original status; BIL20 did not rerun them.

8.1 Hand Calculation: Beta-Binomial Example

- **Input**: Prior $\alpha = 2, \beta = 2$; $n = 10$ trials with $y = 8$ successes.
- **Posterior parameters**:

$$\alpha_{\text{post}} = 2 + 8 = 10, \quad \beta_{\text{post}} = 2 + 10 - 8 = 4$$

- **Posterior mean and variance**:

$$E[p \mid y] = \frac{10}{10+4} = \frac{10}{14} = \frac{5}{7} \approx 0.714286$$

$$\begin{aligned}\text{Var}(p \mid y) &= \frac{10 \times 4}{14^2 \times (14+1)} \\ &= \frac{40}{196 \times 15} = \frac{40}{2940} = \frac{2}{147} \approx 0.013605\end{aligned}$$

- **Prediction for future trials**:
  - Predicted probability of success on one trial: $P(\tilde{y} = 1 \mid y) = 10/14 = 5/7 \approx 0.714286$.
  - Joint probability of success in two trials:

$$\begin{aligned}E[p^2 \mid y] &= \frac{10 \times (10+1)}{14 \times (14+1)} \\ &= \frac{110}{210} = \frac{11}{21} \approx 0.523810\end{aligned}$$

  - Squared posterior mean: $(E[p \mid y])^2 = (10/14)^2 = 100/196 = 25/49 \approx 0.510204$.
  - Check the difference:

$$\begin{aligned}E[p^2 \mid y] - (E[p \mid y])^2 &= \frac{11}{21} - \frac{25}{49} \\ &= \frac{77-75}{147} = \frac{2}{147} \equiv \text{Var}(p \mid y)\end{aligned}$$

    The two values agree exactly.

Continued: the same complete example

8.2 Hand Calculation: Normal Hierarchical Shrinkage

Returns in this example are expressed as decimals: $0.02 = 2\%$ and $0.03 = 3\%$. Means and standard deviations use decimal return units; posterior variance V_j and predictive variance use squared units. For example, group 1 has $V_1 = 0.0002$, the uncertainty about its mean parameter; its square root is 0.014142, or about 1.4142 percentage points on the return scale. A future individual return also has observation noise, so predictive variance is $0.0002 + 0.0016 = 0.0018$, with square root about 0.042426 (4.2426 percentage points). If returns are expressed in percentage points instead, means and standard deviations are multiplied by 100, while variances must be multiplied by 100^2; the shrinkage weights do not change. Hyperparameters in this example remain known constants and are not estimated here.

- **Known hyperparameters**: Prior mean $\mu = 0.0$, prior standard deviation $\tau = 0.02$, and observation standard deviation $\sigma = 0.04$.
- **Precision constants**:
$$\tau^{-2} = \frac{1}{0.02^2} = 2500, \quad \sigma^{-2} = \frac{1}{0.04^2} = 625, \quad \frac{\sigma^2}{\tau^2} = \frac{0.0016}{0.0004} = 4$$
- **Group 1**: $n_1 = 4$, sample mean $\bar{y}_1 = 0.03$.
  - Shrinkage weight: $w_1 = \frac{4}{4+4} = 0.5$.
  - Posterior mean: $\hat{\theta}_1 = (1 - 0.5) \times 0.0 + 0.5 \times 0.03 = 0.015$.
  - Posterior precision: $1/V_1 = 2500 + 4 \times 625 = 5000 \implies V_1 = 1/5000 = 0.0002$.
  - Posterior standard deviation: $\sqrt{0.0002} \approx 0.014142$.
  - Posterior predictive variance: $\text{Var}_{\text{pred},1} = V_1 + \sigma^2 = 0.0002 + 0.0016 = 0.0018$.
  - Predictive standard deviation: $\sqrt{0.0018} \approx 0.042426$.
- **Group 2**: $n_2 = 36$, sample mean $\bar{y}_2 = 0.01$.
  - Shrinkage weight: $w_2 = \frac{36}{36+4} = \frac{36}{40} = 0.9$.
  - Posterior mean: $\hat{\theta}_2 = (1 - 0.9) \times 0.0 + 0.9 \times 0.01 = 0.009$.
  - Posterior precision: $1/V_2 = 2500 + 36 \times 625 = 2500 + 22500 = 25000 \implies V_2 = 1/25000 = 0.00004$.
  - Posterior standard deviation: $\sqrt{0.00004} \approx 0.006325$.
  - Posterior predictive variance: $\text{Var}_{\text{pred},2} = V_2 + \sigma^2 = 0.00004 + 0.0016 = 0.00164$.
  - Predictive standard deviation: $\sqrt{0.00164} \approx 0.040497$.

8.3 Hand Calculation: Simpson's Paradox and Stratification for Causal Inference

- **Definition of the evaluation metric**: The outcome $Y \in \{0, 1\}$ in this example indicates whether the next-day opening excess return reaches a benchmark threshold. Its mean $E[Y]$ is a dimensionless **success probability**; the difference between two probabilities multiplied by 100 is a difference in **percentage points (pp)**, **not** a percentage return.

Continued: the same complete example

- **Contingency-table data** (total sample $N = 200$):
  - High-attention stratum $Z = \text{high}$ (industry leaders):
    - Treatment group $T = 1$ (call held): $n = 90$, with 63 successes, so $E[Y \mid T = 1, Z = \text{high}] = 63/90 = 0.70$ (70%);
    - Control group $T = 0$ (no call): $n = 10$, with 8 successes, so $E[Y \mid T = 0, Z = \text{high}] = 8/10 = 0.80$ (80%);
    - Within-stratum contrast: $0.70 - 0.80 = -0.10$ (a 10-percentage-point reduction in the success rate).
  - Low-attention stratum $Z = \text{low}$ (micro-cap tail):
    - Treatment group $T = 1$ (call held): $n = 10$, with 1 success, so $E[Y \mid T = 1, Z = \text{low}] = 1/10 = 0.10$ (10%);
    - Control group $T = 0$ (no call): $n = 90$, with 18 successes, so $E[Y \mid T = 0, Z = \text{low}] = 18/90 = 0.20$ (20%);
    - Within-stratum contrast: $0.10 - 0.20 = -0.10$ (a 10-percentage-point reduction in the success rate).
- **Unadjusted crude contrast**:
  - Treatment group total: $n = 100$, with $63 + 1 = 64$ successes, so $E[Y \mid T = 1] = 64/100 = 0.64$ (64%);
  - Control group total: $n = 100$, with $8 + 18 = 26$ successes, so $E[Y \mid T = 0] = 26/100 = 0.26$ (26%);
  - Unadjusted crude difference: $0.64 - 0.26 = +0.38$ (the apparent success rate is 38 percentage points higher).
- **Standardized observed difference; interpreted as the ATE only under the identification assumptions in §5.5** (target-population distribution $P(Z = \text{high}) = 0.5, P(Z = \text{low}) = 0.5$):

$$\text{ATE} = 0.5 \times (-0.10) + 0.5 \times (-0.10) = -0.10$$

The unadjusted difference is +38 percentage points and the standardized difference is −10 percentage points; the latter is interpreted as the target-population ATE only if the identification conditions assumed in this example hold. The table demonstrates an association reversal caused by weights; it does not itself prove the true causal direction of treatment.

Continued: the same complete example

8.4 Side-by-Side Hand Calculation: TF-IDF for Two Documents

Based on the step-by-step derivation in §6.4, the term frequencies, smoothed inverse document frequencies, and feature vectors for d_1 = "growth profit" and d_2 = "growth loss" are shown below:

Term t	$\text{tf}(t, d_1)$	$\text{tf}(t, d_2)$	Document frequency $\text{df}(t)$	Smoothed $\text{idf}(t)$
`growth`	1	1	2	1.000000
`loss`	0	1	1	1.405465
`profit`	1	0	1	1.405465

Term t	Unnormalized $x_1(t)$	Unnormalized $x_2(t)$	L2-normalized $v_1(t)$	L2-normalized $v_2(t)$
`growth`	1.000000	1.000000	0.579739	0.579739
`loss`	0.000000	1.405465	0.000000	0.814802
`profit`	1.405465	0.000000	0.814802	0.000000

- Vector magnitudes: $\|\mathbf{x}_1\|_2 = \|\mathbf{x}_2\|_2 = \sqrt{1.0^2 + 1.405465^2} \approx 1.724915$
- Cosine similarity: $\cos(\mathbf{v}_1, \mathbf{v}_2) = 0.579739^2 + 0 + 0 = 0.336097$

8.5 Strict Time-Series Split and Constructed Results for Text Prediction

Under the frozen historical protocol, 120 constructed events are generated by repeating four fixed text templates; they are not 120 independent new texts. The 59 mature training, 29 validation, and 30 test documents each contain the same four templates. This tests extrapolation across time blocks only; it does not test generalization to new texts, templates, or company language. The OOV negative case checks only that the vocabulary and IDF remain unchanged; it is not a test of prediction ability on new text.

The 120 constructed earnings-call events (`CONSTRUCTED_CALL_000` through `CONSTRUCTED_CALL_119`) use a strict time-series split. A training label must be available strictly before the first signal time in the validation set (`label_available_at < next_signal_time`, `<`); a validation label must be available strictly before the first signal time in the test set.

- **Split sizes**:
  - Training: 59 eligible documents (indices 0..58; the label for index 59 matures at 09:30 on D_{61}, after the first validation signal at 09:00 on D_{61}, so it is strictly purged);
  - Validation: 29 eligible documents (indices 60..88; the label for index 89 matures at 09:30 on D_{91}, after the first test signal at 09:00 on D_{91}, so it is strictly purged);
  - Test: 30 documents (indices 90..119).
- **Training vocabulary size**: Fit strictly on the 59 training documents, yielding 42 unique terms.

Continued: the same complete example

- **Ridge hyperparameter tuning on validation**:
  - $\alpha = 0.1 \implies \text{MSE}_{\text{val}} = 0.00021835$
  - $\alpha = 1.0 \implies \text{MSE}_{\text{val}} = 0.00021697$
  - $\alpha = 10.0 \implies \text{MSE}_{\text{val}} = 0.00021133$ (best hyperparameter: $\alpha^* = 10.0$)
- **Final test-set comparison**:

Evaluation measure	Training-set global-mean baseline	TF-IDF + Ridge regression ($\alpha = 10.0$)	Economic / statistical interpretation
Test-set mean squared error (MSE)	0.00023274 (2.3274×10^{-4})	0.00023603 (2.3603×10^{-4})	Ridge prediction error is slightly larger than the naive historical constant mean
Relative test-set R^2	0.000000	-0.014136 (-1.41%)	Negative R^2 relative to the mean baseline (a typical negative result)
Time-series summary Spearman rank correlation	Undefined (constant prediction has zero variance)	+0.076274	Summary rank correlation across 30 test trading days
Daily cross-sectional IC	Undefined ($N = 1$ asset/day)	Undefined ($N = 1$ asset/day)	With one asset per day, cross-sectional covariance is mathematically undefined

This is a **negative result** from the constructed teaching experiment: when noisy returns overwhelm a weak text signal, a complex model can easily lose to a naive constant-mean baseline on the test set. Do not fill in zero or claim a trading advantage.

8.6 Hand Examples: Negation Sentiment and Word Embeddings

Each row of the hand-specified matrix corresponds to growth, profit, loss, decline, stable, and steady. Its three columns are teaching axes (strength, momentum, and uncertainty), not estimated financial factors:

$$E = \begin{pmatrix} 0.8 & 0.9 & 0.1 \\ 0.9 & 0.7 & 0.1 \\ -0.8 & -0.7 & 0.5 \\ -0.7 & -0.8 & 0.4 \\ 0.2 & 0.1 & -0.8 \\ 0.3 & 0.2 & -0.7 \end{pmatrix}.$$

First divide each row by its own norm. For example, the norms of growth and profit are $\sqrt{1.46}$ and $\sqrt{1.31}$, respectively, and their cosine similarity is:

$$\frac{1.36}{\sqrt{1.46 \cdot 1.31}} \approx 0.9834.$$

The sentence representation is the mean of its unit word vectors followed by normalization; this example ignores the connector “and.” The code below returns all vectors and similarities:

Continued: the same complete example

```
from quant_course.bayesian_text import hand_embedding_matrix,
    ↪ negation_sentiment_scorer
vectors, similarities = hand_embedding_matrix()
print(vectors, similarities)
print(negation_sentiment_scorer("earnings performance did not improve showing no
    ↪  growth"))
```

- **Negation reversal**: Sentence: "earnings performance did not improve showing no growth despite aggressive restructuring"
  - Naive unigram scoring with a fixed lexicon (BoW itself does not specify sentiment weights): it recognizes `"improve"` (+1) and `"growth"` (+1), giving a score of +2 (a severe error that classifies the sentence as highly positive);
  - Negation-window algorithm: `"not"` activates a three-word reversal window, flipping `"improve"` to −1; `"no"` activates the reversal window and flips `"growth"` to −1, for a total score of −2 (matching the intended negative meaning of this constructed sentence). This three-word window rule is not guaranteed to handle complex negation, double negation, sarcasm, or general financial text accurately.
- **Cosine similarities in the embedding space** (using a 6×3 manually specified, normalized embedding matrix; no real corpus was used for training):
  - Cosine similarity between `growth` and `profit`: +0.9834;
  - Cosine similarity between `growth` and `loss`: −0.8595;
  - Cosine similarity between the mean-pooled phrases S_1 ("growth and profit") and S_2 ("loss and decline"): −0.8836.

---

9. Python Implementation

The core module is `bayesian_text.py` under `src/quant_course`. The standalone script is `ch43_bayesian_text.py` under `code`; tests are in `test_bayesian_text.py` under `tests`. Historical execution counts and results are in the applicable A8-C/C19 QA records; BIL20 revises teaching only.

9.1 Execution Command and Run Contract

- Execution environment: the course-standard virtual environment, invoked from the course source root as `.venv/bin/python`.
- Standalone CLI invocation:

```
.venv/bin/python code/ch43_bayesian_text.py --out build/batches/A8-C/reader/runs
    ↪ /ch43_trial_01
```

- Safety guard: if the specified `--out` directory already exists, the script immediately raises `FileExistsError` and refuses to overwrite historical experimental evidence.

9.2 Core Function Interfaces and Standalone Example

The code first passes in the raw 60/30/30 time slices; `train_text_prediction_models` checks PIT eligibility again internally and filters training and validation labels using `label_available_at <` the next block's first `signal_time`. Index 59 has a label that matures at 09:30 on D_{61}, while the first validation signal is at 09:00 that day; index 89 is similarly later than the first test signal at 09:00 on D_{91}. The two rows are therefore purged separately, yielding 59/29/30 as in §8.5 and preventing immature labels from entering fitting or tuning. The test block has no following-fold fitting boundary, so its labels are used for retrospective evaluation. Equality does not satisfy the strict `<` condition.

The text-prediction path actually run in this chapter is TF-IDF + Ridge on constructed text. The 6×3 hand-specified embedding matrix in §8.6 is used only for a geometric calculation and was not a training feature for that predictor. The LLM section is an offline ledger and review exercise marked `not_run / unknown`. These are three distinct training and evaluation statuses.

```
# 1. Bayesian conjugate update and prediction
from quant_course.bayesian_text import beta_binomial_posterior_predictive
res_bayes = beta_binomial_posterior_predictive(
    alpha_prior=2.0, beta_prior=2.0, successes=8, trials=10
)

# 2. Normal hierarchical partial pooling
from quant_course.bayesian_text import normal_hierarchical_partial_pooling
res_normal = normal_hierarchical_partial_pooling(
    prior_mean=0.0, prior_sd=0.02, observation_sd=0.04,
    groups=[{"n": 4, "sample_mean": 0.03}, {"n": 36, "sample_mean": 0.01}]
)

# 3. Causal stratification and strict positivity defense
from quant_course.bayesian_text import (
    compute_stratified_ate,
    PositivityViolationError,
    generate_causal_dataset,
)
df_causal = generate_causal_dataset()
ate_res = compute_stratified_ate(
    df_causal, z_col="Z", t_col="T", y_col="Y",
    target_p_z={"high": 0.5, "low": 0.5}
)

# 4. PIT text filtering and training-window TF-IDF
from quant_course.bayesian_text import (
    filter_pit_eligible_documents,
    TextFeaturePipeline,
    train_text_prediction_models,
)
from quant_course.bayesian_text import generate_constructed_text_dataset
raw_docs_df, _ = generate_constructed_text_dataset()
eligible_docs = filter_pit_eligible_documents(raw_docs_df, cutoff_col="signal_time")
train_df, val_df, test_df = (
    eligible_docs.iloc[:60], eligible_docs.iloc[60:90], eligible_docs.iloc[90:]
```

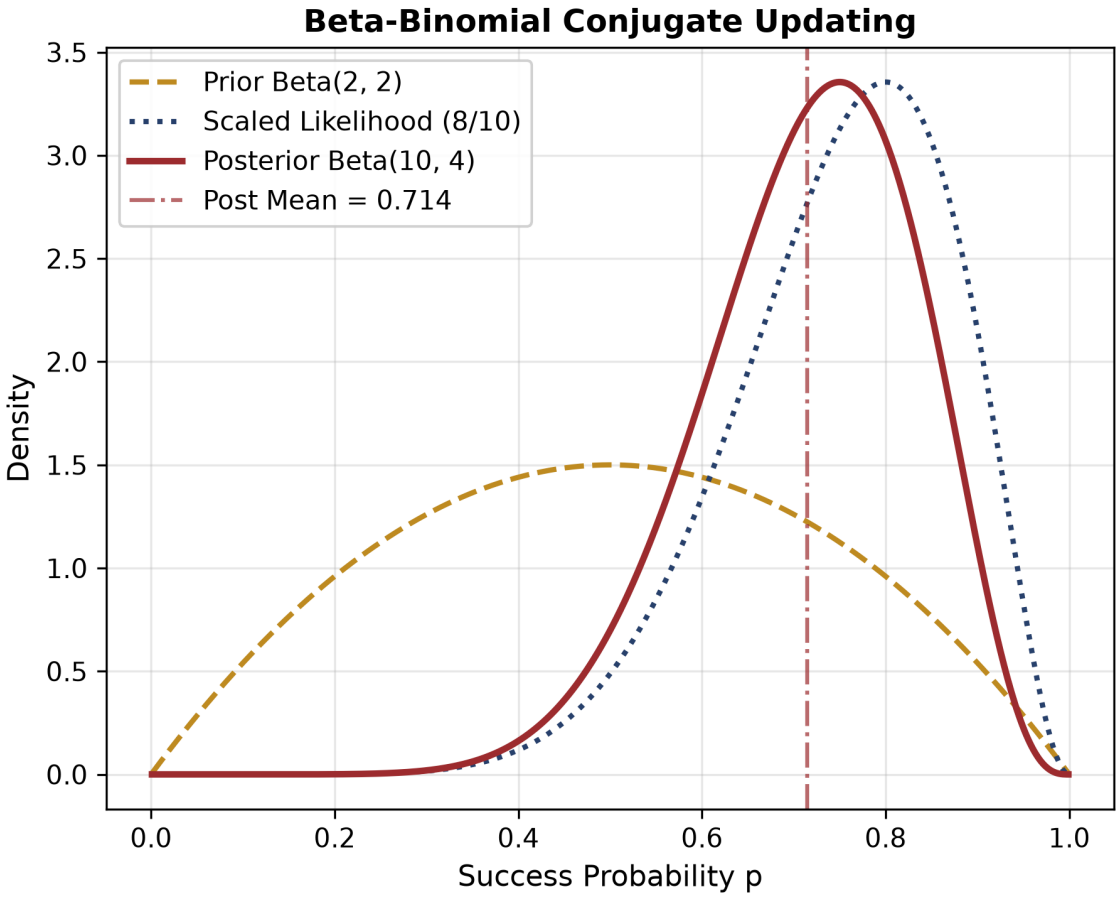
```
)
model_res = train_text_prediction_models(train_df, val_df, test_df, alpha_grid=[0.1,
    ↪ 1.0, 10.0])
```

---

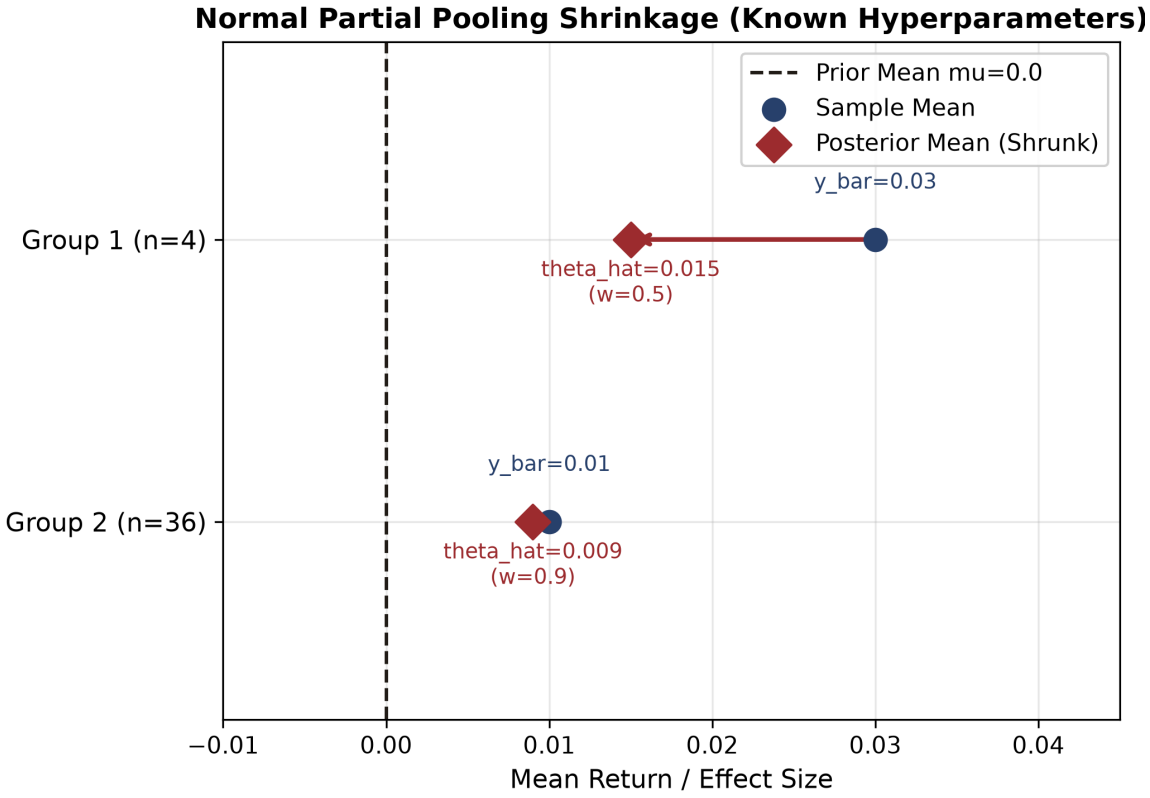
10. Visualization

The historical frozen CONSTRUCTED teaching run generated and saved five figures; this round reuses the original figures from `course/zh/chapters/ch43/figures/root_r3/`.

Figure 43-1: Bayesian Conjugate Updating and Normal Hierarchical Shrinkage



(a)



(b)

Figure 43-1 description:

- **Left panel (Beta-Binomial conjugate update)**: The horizontal axis is the binomial success probability $p \in [0, 1]$; the vertical axis is probability density (the scaled likelihood is used only to compare shapes and is not a probability density over the parameter). The gold dashed line is the prior Beta$(2, 2)$, the blue dotted line is the scaled likelihood (8 successes in 10 trials), and the red solid line is the posterior Beta$(10, 4)$. The posterior mean (red dash-dot line at 0.714) lies between the prior mean of 0.5 and the maximum-likelihood estimate of 0.8, clearly showing the Bayesian compromise with the data.
- **Right panel (normal hierarchical partial-pooling shrinkage)**: The horizontal axis is group-mean return and the vertical axis lists asset groups. The black dashed line is the whole-market prior mean $\mu = 0$. Blue circles are group sample means $\bar{y}$, and red diamonds are the shrunken posterior means $\hat{\theta}$. Group 1 (sample size $n = 4$) is strongly shrunk by 50% (arrow from 0.03 to 0.015); Group 2 (sample size $n = 36$) is shrunk by only 10% (from 0.01 to 0.009).

Figure 43-2: Simpson's Paradox and Standardization for Causal Inference

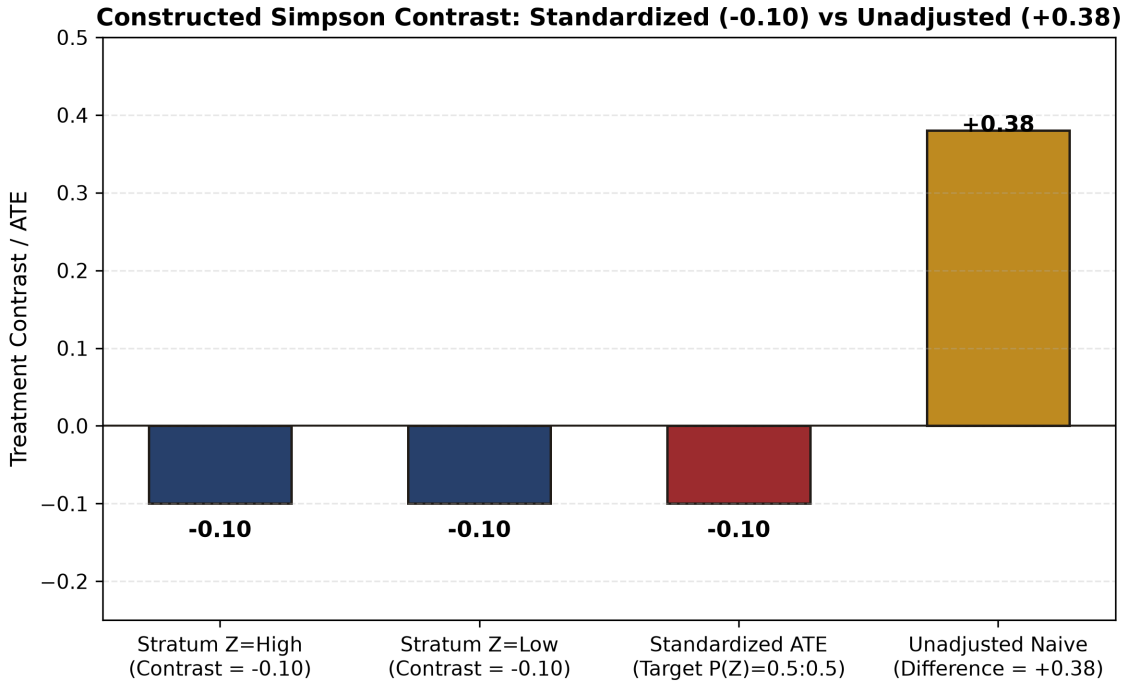


Figure 43-2 description:

- **Horizontal axis**: strata and evaluation measures; **vertical axis**: difference in binary success probability (0.10 corresponds to 10 percentage points).
- The first two dark-blue bars show the observed within-stratum probability differences. Within both the high- and low-attention strata, the call group is 10 percentage points below the no-call group (-0.10).
- The third red bar is the observed difference standardized to the target-population distribution $P(Z)$ (-0.10). Under the stated identification assumptions, it can be interpreted as the target-population ATE; the bar chart does not verify those assumptions.
- The fourth gold bar is the unstandardized association difference ($+0.38$). Comparing all four bars shows the reversal caused by group weights.

Figure 43-3: After-Hours Earnings-Call Timeline and PIT Contract

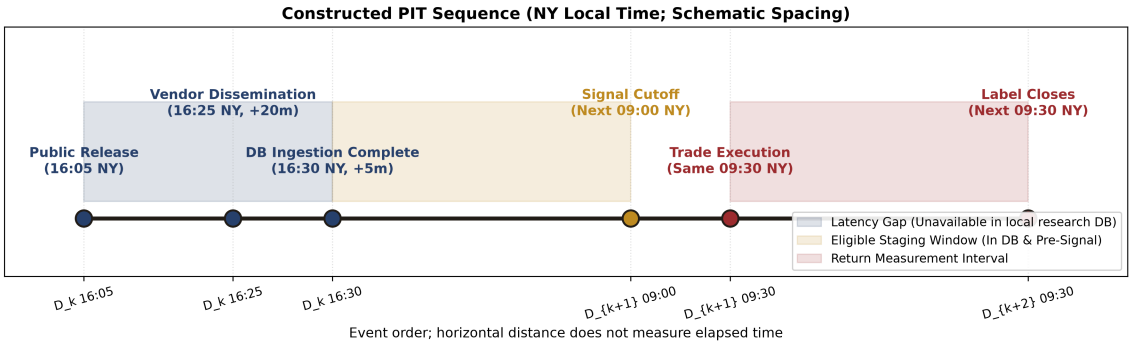
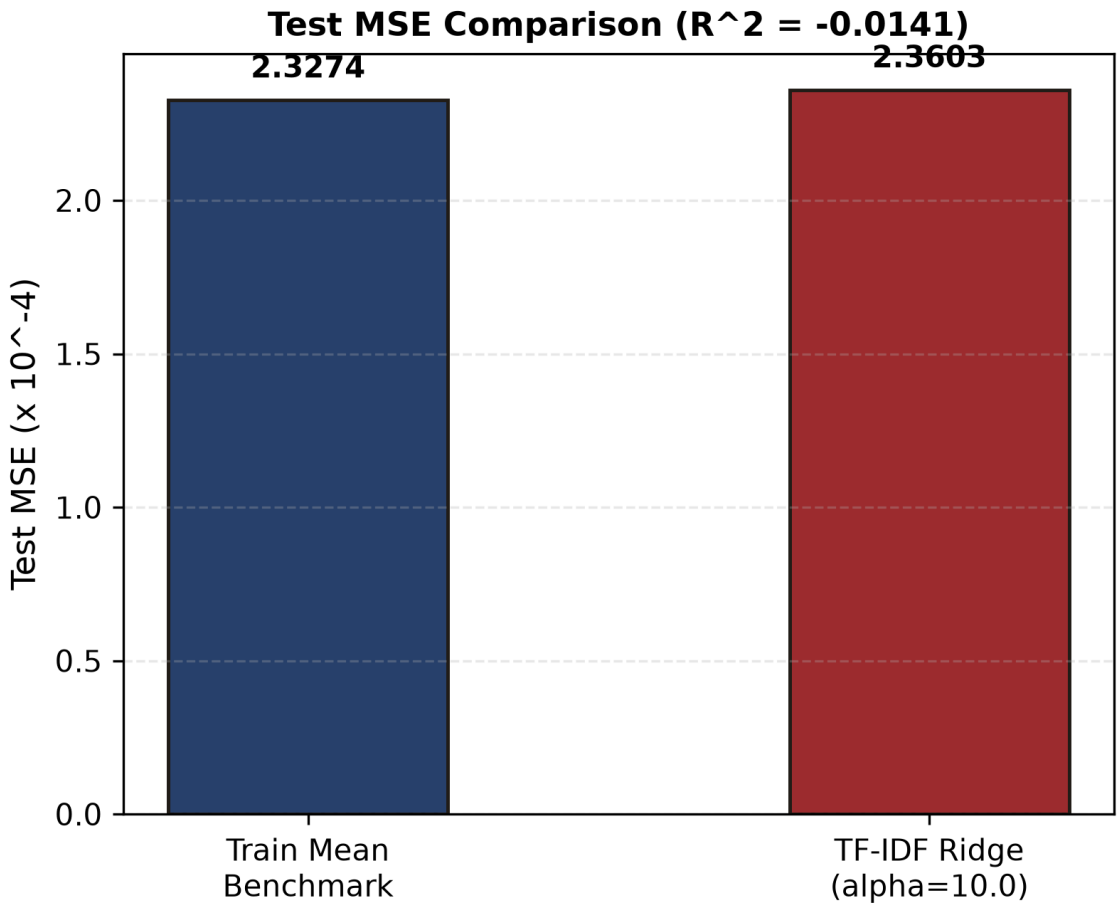


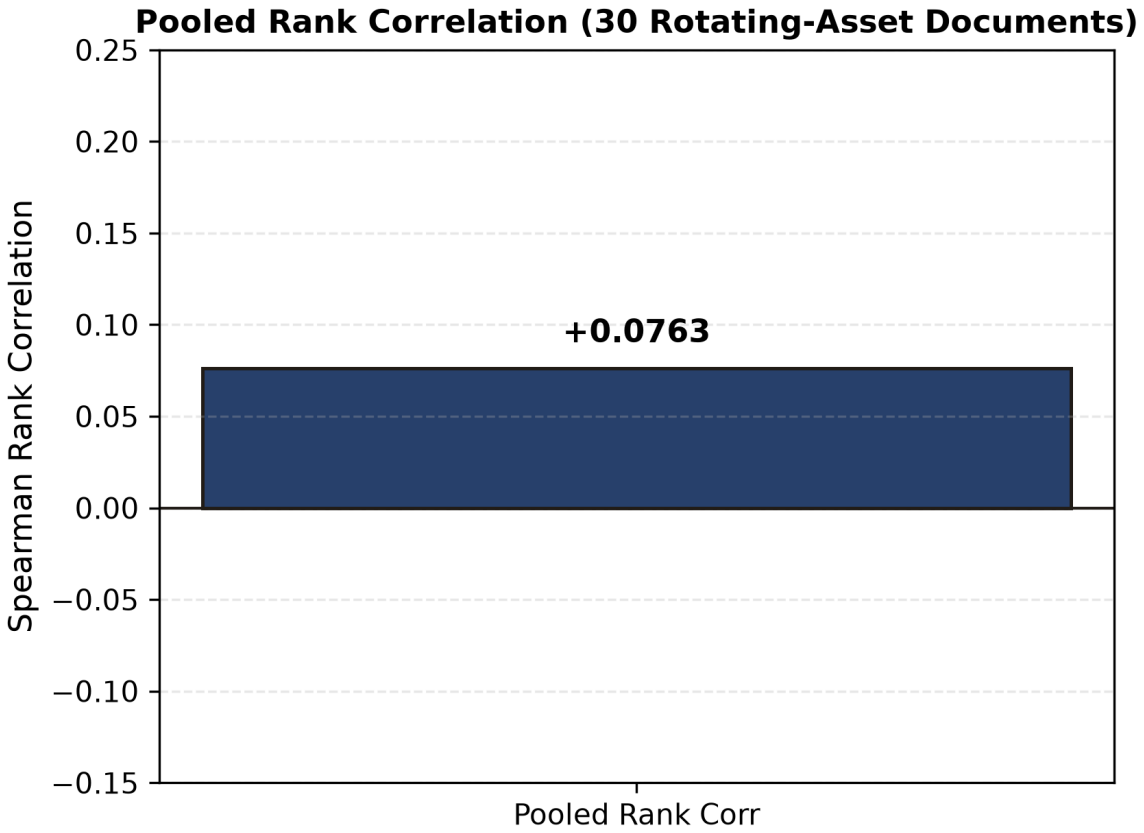
Figure 43-3 description:

- **Horizontal axis**: event order (local Eastern Time; spacing does not represent elapsed duration; see §7.4/43-E05 for UTC conversion); **vertical axis**: illustrative height.
- The blue shaded area (D_k 16:05 to 16:30) marks the 25-minute vendor-distribution and internal-ingestion delay. During this interval, **the local quantitative research database does not yet have the data available** (although the event became public in the external world at 16:05). Any local signal generated before 16:30 involves look-ahead leakage.
- The gold shaded area (D_k 16:30 to D_{k+1} 09:00) is the compliant computation window, after the data have been ingested and before the opening signal.
- The red shaded area (D_{k+1} 09:30 to D_{k+2} 09:30) is the one-day holding-period return measured after the opening trade; the label becomes fully mature only at the end of this interval.

Figure 43-4: Test-Set Prediction Performance (Low-SNR Result)



(a)

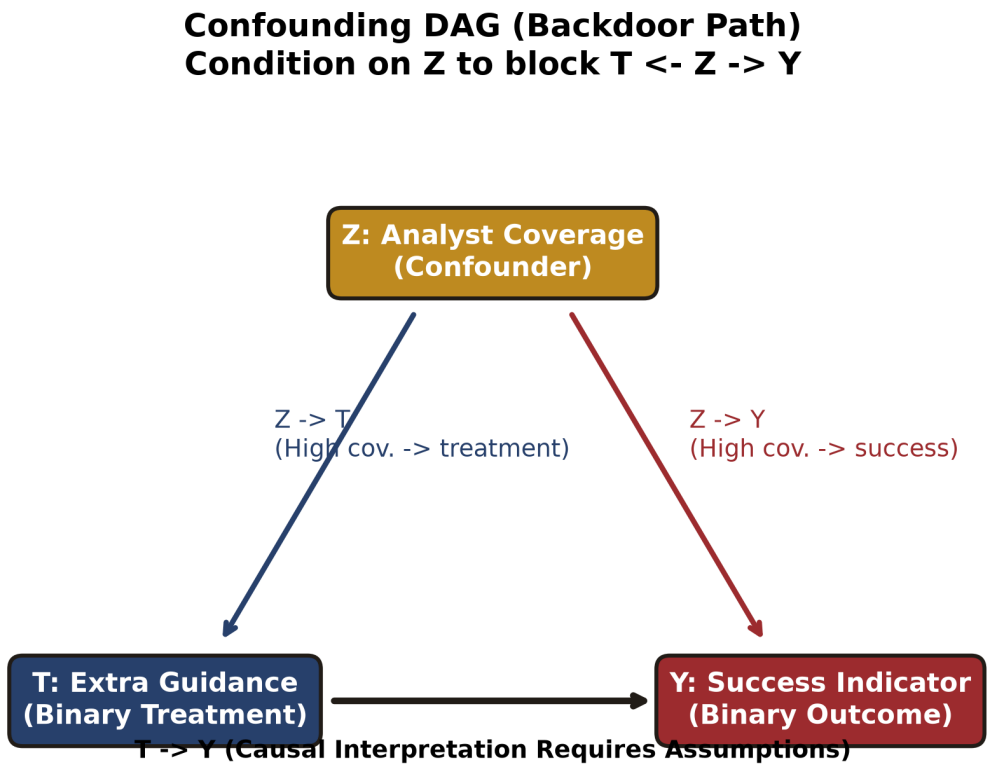


(b)

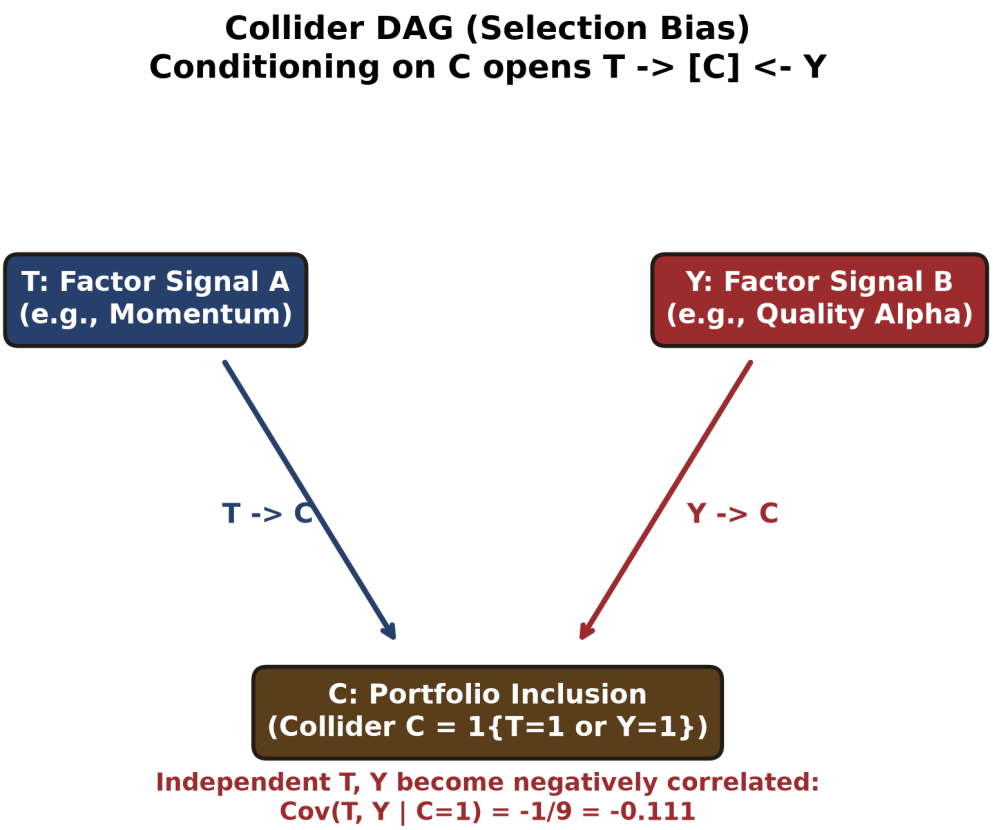
Figure 43-4 description:

- **Left panel**: Test-set mean squared error (MSE $\times 10^{-4}$): 0.00023274 for the training-set global-mean baseline and 0.00023603 for TF-IDF + Ridge regression; relative test-set R^2 is -0.0141.
- **Right panel**: The time-series summary Spearman rank correlation over 30 test trading days is +0.0763. Note explicitly that daily cross-sectional IC is mathematically undefined because the dataset has only one asset per trading day ($N = 1$); the training-set global-mean baseline also makes constant predictions with zero variance, so its rank correlation is undefined.
- **Observation**: In this chapter's constructed data, strong noise ($\sigma = 0.02$) greatly exceeds the weak text signal (0.0005). This sample has a positive summary correlation, but that is not enough to show that the model learned a reproducible ranking relationship, and its absolute mean squared error does not beat the naive mean baseline. This is a typical negative result in financial machine learning and cautions quantitative researchers against assuming that a text factor is profitable without considering the actual signal-to-noise ratio.

Figure 43-5: Causal Confounding and Collider DAGs



(a)



(b)

Figure 43-5 description:

- **Left panel (confounding DAG and backdoor path)**:
  - Node definitions: Z is a confounder (analyst attention), T is a binary treatment (whether additional guidance is provided), and Y is a binary success measure.
  - Directed edges: $Z \to T$ (in the constructed scenario, high-attention companies are more likely to provide additional guidance); $Z \to Y$ (in the constructed scenario, high-attention companies have a higher baseline target-attainment rate); and $T \to Y$ is the causal path to be identified. The standardized difference in this table is -0.10 and is interpreted as a probability effect only if the assumptions hold.
  - Blocking the backdoor path: The path $T \leftarrow Z \to Y$ creates a spurious positive association before adjustment ($+0.38$). Stratified standardization controlling for confounder Z blocks this path; under the stated identification assumptions, the standardized difference can be interpreted as the treatment effect.
- **Right panel (collider DAG and sample-selection bias)**:
  - Node definitions: T is a momentum-factor signal, Y is a quality-alpha-factor signal, and C is portfolio inclusion (a collider, defined as $C = \mathbb{I}(T = 1 \vee Y = 1)$).
  - Directed edges: $T \to C$ and $Y \to C$. The two factors are marginally independent in the full market ($\mathrm{Cov}(T, Y) \approx 0$).

– Mechanism of collider bias: If a researcher incorrectly restricts the backtest universe to "holdings in the selected portfolio" ($C = 1$), conditioning opens the collider path and creates a spurious negative correlation between otherwise independent factors, $\mathrm{Cov}(T, Y \mid C = 1) = -1/9 \approx -0.1111$.

---

11. Real-Market Example

In applications to real data, selection of unstructured text is constrained by technical interfaces, coverage bias, and legal authorization.

11.1 Real Regulatory Interfaces and the Boundary of PIT Review

The SEC's official API page describes public filing interfaces and processing delays. This round did not check an individual filing, field time zones, or original earnings-call text, so this is only a pointer to the interface; it is not proof of PIT eligibility or licensing for real text.

- **General API interfaces and physical latency**: When connecting to official data, quantitative engineering must distinguish the time an event physically occurs from the time it is ingested locally. External text remains unavailable until it has been parsed and stored in the local database, and therefore cannot be used before the system clock reaches that completion time.
- **Calendar and trading rules**: A real quantitative production system must use the actual exchange calendar (such as NYSE holidays and half-day sessions). The synthetic weekday calendar used in this chapter removes holiday complexity to emphasize the teaching point; it is not equivalent to a live-trading calendar with complex closure rules.

11.2 Alternative-Data Licensing, Coverage, and Sample-Selection Bias

Quantitative teams selecting alternative data must watch for two major nontechnical risks:

1. **Public accessibility versus legal authorization**: Text on a public webpage may be publicly accessible, but that does not mean it is licensed for compliant commercial scraping, feature extraction, model training, or redistribution of trades based on derived features. Training a strategy on unauthorized text may create serious compliance and legal exposure. This teaching batch records the licensing gap explicitly: no real earnings-call transcript corpus with full commercial authorization was provided, so this chapter uses constructed text with the `CONSTRUCTED_CALL_` prefix.
2. **Selection bias caused by vendor coverage**: Suppose an alternative-data vendor provides high-frequency earnings-call sentiment features, with an illustrative coverage scenario: 90% of high-analyst-attention (large-cap) companies are covered, but only 10% of low-attention (small- and mid-cap) companies are covered. If a researcher tests a factor only in the sample with vendor data (selection indicator $S = 1$), attention Z changes the sample composition because it affects both data availability S and company fundamentals Y. If

high- and low-attention firms each make up half of the population, high-attention firms make up $0.9/(0.9+0.1) = 0.9$ of the covered sample. With baseline success rates of 0.8 and 0.2, respectively, the population mean is 0.5 while the covered-sample mean is 0.74. This 0.24 difference comes from selection, not from a new factor effect. The edges $Z \to S$ and $Z \to Y$ alone do not make S a collider; collider status also requires two independent causes to point into S.

---

12. Common Mistakes

Mistake 1: Confusing Causal Positivity with Normality

- **Incorrect belief**: "Causal identification requires every feature to follow a normal distribution."
- **Correction**: Positivity/overlap means that within every confounder stratum, both the treatment and control groups have nonzero assignment probability: $0 < P(T = 1 \mid Z = z) < 1$.
- **Diagnostic snippet (do not run independently)**:

```
# Faulty approach: no control group in a stratum, so fill in 0 and calculate the
    ↪ difference
diff_stratum = mean_t1 - 0.0  # This seriously breaks identification of the
    ↪ potential outcome!
```

- **Corresponding repair snippet**:

```
if n_t1 == 0 or n_t0 == 0:
    raise PositivityViolationError(f"Positivity violated in stratum {z}!")
```

In a finite sample, the absence of one treatment arm in a stratum is a sample-support/computability safeguard; it does not by itself prove that the population treatment probability is 0 or 1. If a target-population stratum has positive weight but no observations at all, it also cannot be handled by silently dropping it or renormalizing the remaining weights.

Mistake 2: Fitting Vocabulary and IDF on the Full Sample Leaks Information Across Time

- **Incorrect belief**: "Text is unsupervised, and TF-IDF vocabulary extraction does not look at future labels, so fitting on the full sample is not leakage."
- **Correction**: If new test-set terms enter the IDF before training, they change the scale of the training features. More seriously, terms unique to major future events (such as a patent name or restructuring code name) can enter the vocabulary early, giving the model prior information about their future importance at historical dates.
- **Repair rule**: Fit the vocabulary and IDF on the training set only, then transform the validation and test sets; ignore unseen terms (OOV).

Mistake 3: Treating an LLM Pretraining Cutoff as Historical PIT Eligibility

- **Incorrect belief**: "The model's training corpus ends in December 2023, so using it to score 2022 news is strictly PIT-compliant."
- **Correction**: That assumed cutoff is later than the historical signal, so later reports may have been absorbed into the corpus. A cutoff statement alone cannot confirm that the model memorized a particular company, nor does it establish that every output is contaminated; retain evidence about the model, corpus timing, and control experiments.

---

13. Quantitative Research Application

13.1 End-to-End Quantitative Research Design: After-Hours Text to Next-Day Execution

Build an event-driven signal-extraction workflow based on earnings calls:

1. **Monitor the event**: Begin monitoring filing feeds at 16:00 Eastern Time. The call ends at 16:05; a third-party vendor produces a transcript at 16:25 and records `available_at`; the local quantitative system finishes ingestion at 16:30 and records `ingested_at`.
2. **Check point-in-time eligibility**: At 09:00 Eastern Time the next day (the signal-generation time), call `filter_pit_eligible_documents`. Exclude any version ingested after 09:00 or revised after 09:00.
3. **Proposed extension: purged walk-forward fitting**: In future work, TF-IDF and Ridge could be fitted in rolling windows under a separately frozen protocol. The procedure actually run in this chapter uses fixed train/validation/test blocks; it is not completed rolling validation. Note: if the label for the last day in a window has not matured (`label_available_at >= next_signal_time`), it must be strictly purged.
4. **Generate the opening signal**: At 09:00 the next day, output a continuous predicted return $\hat{y}$ for each asset.

13.2 Prediction-Only Interface and Downstream Execution-System Specification

This chapter is strictly limited to the **prediction stage for signal generation and features**. In constructed data that rotates one asset through a time series, turnover or transaction costs cannot be calculated by simply differencing the signs of consecutive predictions; sign flips for different single assets do not form a continuous holding ledger for the same portfolio. The downstream execution system must connect to the course-standard portfolio and execution ledger (such as the standardized interface in Chapter 29):

- **Required downstream-ledger fields**: `doc_id`, `trade_time`, `asset_id`, `target_return_pred`, `allocated_weight`, `execution_price`, `one_way_fee_bps`.
- **Boundary and separation of responsibilities**: The text-prediction model outputs only a point-in-time-compliant $\hat{y}$. `one_way_fee_bps / 10000` is the `fee_rate` input expected by the existing backtest function; these fields belong to a proposed adapter, not the

old function's parameters. A complete asset-price panel and a holding policy are also required. The downstream system is responsible for mapping $\hat{y}$ to target weights across assets (accounting for concentration and risk budgets), estimating slippage with a market-impact model, and producing a complete portfolio PnL ledger subject to borrow constraints.

---

14. Chapter Summary

1. **The nature of inference**: Bayesian inference represents uncertainty about knowledge with probability distributions. Beta-Binomial updating shows how data move a prior; the joint prediction for two future periods is dependent because of parameter uncertainty, with covariance equal to the posterior variance. With known hyperparameters, the normal hierarchical model demonstrates adaptive shrinkage through partial pooling.
2. **The distinction between causality and statistics**: Statistical correlation is not a causal effect. Confounders can create Simpson's paradox by changing marginal proportions in treatment assignment. Stratified standardization recovers the true ATE only under consistency, conditional exchangeability, strict positivity, and no interference.
3. **Engineering safeguards for text and PIT**: Alternative-data and LLM research must maintain a strict time boundary. Fit vocabularies only within the training window; review earnings-call ingestion and revision timestamps strictly; and record prompt hashes and training-corpus cutoff boundaries in offline LLM ledgers. Respecting negative results under low signal-to-noise conditions is a basic part of mature quantitative research.

---

15. Exercises

- **43-E01** (Beta-Binomial conjugate posterior and posterior prediction): Let $p \sim \text{Beta}(2, 2)$ and observe 8 successes in 10 trials. Find the posterior parameters, posterior mean, and posterior variance; derive the one-period future predicted success rate and the joint success rate for two future periods, $E[p^2 \mid y]$; and explain why $E[p^2 \mid y] \neq (E[p \mid y])^2$.
- **43-E02** (Normal hierarchical partial-pooling calculation): Let the prior mean be $\mu = 0$, the prior standard deviation $\tau = 0.02$, and the observation standard deviation $\sigma = 0.04$. Group 1 has 4 observations with mean 0.03; Group 2 has 36 observations with mean 0.01. Calculate both groups' shrinkage weights, posterior means, posterior precisions, and posterior predictive variances for a future observation.
- **43-E03** (Simpson's paradox and causal identification with confounding): Let $Z \in \{\text{high}, \text{low}\}$, with target-population probability $P(Z = \text{high}) = 0.5$. Using the contingency table in §8.3, calculate the unadjusted difference and standardized observed difference, state the identification assumptions needed to interpret the latter as an ATE, and explain the mathematical reason for the sign reversal.
- **43-E04** (Boundary of the causal positivity assumption): Explain the positivity assumption in causal inference. Suppose stratum z^* has **positive weight in the target population**.

If every unit in that stratum receives treatment ($P(T = 1 \mid Z = z^*) = 1$), is the causal ATE identifiable? Why or why not?

- **43-E05** (Daylight-saving-time conversion for the PIT calendar): A listed company releases earnings after hours at 16:05 Eastern Time on March 14, 2022 (the day after the U.S. daylight-saving-time change). Write the corresponding UTC timestamp and, with a 25-minute ingestion delay, determine the earliest time the text can be used for a signal generated at 09:00 Eastern Time.
- **43-E06** (Negation and the limits of bag-of-words): Explain why the fixed positive/negative lexicon used in this chapter scores the unigram bag-of-words phrase `"did not improve showing no growth"` as highly positive. How could an algorithm repair this?

---

16. Programming Exercises

- **43-P01** (Verify predictive covariance): Write a function `test_predictive_covariance()` that accepts any legal α, β, y, n and asserts, within floating-point precision, that the numerically computed $E[p^2 \mid y] - (E[p \mid y])^2$ equals $\mathrm{Var}(p \mid y)$.
- **43-P02** (Enforce causal positivity): Write a function `compute_stratified_ate_safe()` that raises a specifically defined `PositivityViolationError` when an input stratum has no control or no treatment units, and includes the violating stratum's name and sample counts in the error.
- **43-P03** (Implement a PIT document-version selector): Write a function `filter_pit_eligible_documents()` that takes a DataFrame with records for multiple revisions and returns the latest compliant unique record before a specified signal timestamp.
- **43-P04** (Fit TF-IDF independently within the training window): Implement `TextFeaturePipeline` so that `fit()` can be called only on training data. When the test set contains unseen terms (OOV), assert that calling `transform()` does not change the internal vocabulary or IDF.

---

17. Challenge Problems

- **43-C01** (Collider bias and sample-selection bias): Let T and Y be independent, each with distribution Bernoulli(0.5), with no causal effect between them. Let the inclusion rule $C = \mathbb{I}(T = 1 \vee Y = 1)$ define the DAG $T \to C \leftarrow Y$. Derive and demonstrate in code that incorrectly selecting the sample by $C = 1$ induces a spurious negative correlation between T and Y. What does this warn about in quantitative stock selection, such as selecting only within the S&P 500 constituents?
- **43-C02** (Offline LLM research ledger and defenses against irreproducibility): Design an auditable offline teaching ledger for an LLM. Write a JSON Schema that includes the prompt-template version, SHA-256 hash, and a check of the time difference between the model's pretraining cutoff and the historical signal date. Explain why changing both the

input and the digest can pass a local check, describe the role of an independently trusted reference digest, and list three engineering red lines for guarding against nondeterministic API outputs.

- **43-C03 (Generalization Boundary of Template Experiments)**: The 120 events contain only 4 fixed templates, and each mature training, validation, and test set contains all 4. What does a valid time split establish? Can the unchanged-vocabulary OOV test or the 30 test events be interpreted as evidence of generalization to new text?

18. Complete Solutions

43-E01 Complete Solution

1. **Calculate the posterior parameters**:

$$\alpha_{\text{post}} = \alpha + y = 2 + 8 = 10, \quad \beta_{\text{post}} = \beta + n - y = 2 + 10 - 8 = 4$$

2. **Posterior mean and variance**:

$$E[p \mid y] = \frac{10}{10+4} = \frac{10}{14} = \frac{5}{7} \approx 0.714286$$

$$\text{Var}(p \mid y) = \frac{10 \times 4}{14^2 \times (14+1)} = \frac{40}{196 \times 15} = \frac{40}{2940} = \frac{2}{147} \approx 0.013605$$

3. **One-period and two-period predictions**:
   - One-period prediction: $P(\tilde{y} = 1 \mid y) = E[p \mid y] = 10/14 = 5/7$.
   - Two-period prediction: $P(\tilde{y}_1 = 1, \tilde{y}_2 = 1 \mid y) = E[p^2 \mid y] = \frac{10\times11}{14\times15} = \frac{110}{210} = \frac{11}{21} \approx 0.523810$.
4. **Why they are unequal**: The squared posterior mean is $(10/14)^2 = 100/196 = 25/49 \approx 0.510204$. The difference $E[p^2 \mid y] - (E[p \mid y])^2 = \frac{11}{21} - \frac{25}{49} = \frac{77-75}{147} = \frac{2}{147} \equiv \text{Var}(p \mid y) > 0$. Explanation: The two trials are independent conditional on p. Integrating over their shared uncertain parameter gives predictive covariance $\text{Var}(p \mid y) > 0$, so they are positively correlated under the marginal predictive distribution. If a new first-trial success is subsequently observed, the posterior for p is then updated, raising the second trial's conditional prediction given that new outcome.

43-E02 Complete Solution

1. **Group 1 calculation**:
   - Prior precision $\tau^{-2} = 2500$; observation variance $\sigma^2 = 0.0016$.
   - Sample precision $n_1/\sigma^2 = 4/0.0016 = 2500$.
   - Posterior precision $1/V_1 = 2500 + 2500 = 5000 \implies V_1 = 0.0002$.
   - Shrinkage weight $w_1 = 2500/5000 = 0.5$.
   - Posterior mean $\hat{\theta}_1 = (1 - 0.5) \times 0.0 + 0.5 \times 0.03 = 0.015$.
   - Posterior predictive variance $\mathrm{Var}_{\mathrm{pred},1} = V_1 + \sigma^2 = 0.0002 + 0.0016 = 0.0018$.
2. **Group 2 calculation**:
   - Sample precision $n_2/\sigma^2 = 36/0.0016 = 22500$.
   - Posterior precision $1/V_2 = 2500 + 22500 = 25000 \implies V_2 = 0.00004$.
   - Shrinkage weight $w_2 = 22500/25000 = 0.9$.
   - Posterior mean $\hat{\theta}_2 = (1 - 0.9) \times 0.0 + 0.9 \times 0.01 = 0.009$.
   - Posterior predictive variance $\mathrm{Var}_{\mathrm{pred},2} = V_2 + \sigma^2 = 0.00004 + 0.0016 = 0.00164$.

43-E03 Complete Solution

1. **Unadjusted difference**: $E[Y \mid T = 1] = 64/100 = 0.64$, $E[Y \mid T = 0] = 26/100 = 0.26$. Difference $= 0.64 - 0.26 = +0.38$.
2. **Stratum-specific contrasts and standardized ATE**: High-attention stratum: $63/90 - 8/10 = 0.70 - 0.80 = -0.10$. Low-attention stratum: $1/10 - 18/90 = 0.10 - 0.20 = -0.10$. Standardized ATE $= 0.5 \times (-0.10) + 0.5 \times (-0.10) = -0.10$.
3. **Mathematical reason**: Ninety percent of the treatment group is in the high-success stratum $Z = \text{high}$, while 90% of the control group is in the low-success stratum $Z = \text{low}$. The different stratum composition reverses the association. The standardized difference is interpretable as a negative ATE only under assumptions such as consistency, conditional exchangeability, positivity in the target support, and SUTVA; the table does not test those assumptions.

43-E04 Complete Solution

1. **Definition of positivity**: In every stratum with $P(Z = z) > 0$, require $0 < P(T = 1 \mid Z = z) < 1$.
2. **Why the effect is not identifiable**: Because z^* has positive weight in the target population, if $P(T = 1 \mid Z = z^*) = 1$, then every unit in that stratum receives treatment and $P(T = 0 \mid Z = z^*) = 0$. The data contain no control observations in that stratum, so there is no observable term for the counterfactual outcome $E[Y(0) \mid Z = z^*]$ (its denominator

is zero). Without a strong parametric extrapolation assumption, the nonparametric causal ATE is not identifiable in theory.

43-E05 Complete Solution

1. **Time-zone conversion**: U.S. markets switched to daylight saving time (EDT, UTC−4) on March 13, 2022. On March 14, 2022, 16:05 EDT converts to `2022-03-14 20:05:00 UTC`.
2. **Delay and signal**:
   - A 20-minute vendor-distribution delay gives `2022-03-14 20:25:00 UTC`;
   - A 5-minute internal-database ingestion delay gives `2022-03-14 20:30:00 UTC` (16:30 EDT);
   - The next-day signal is at 09:00 EDT (that is, `2022-03-15 13:00:00 UTC`);
   - **Compare the complete UTC dates and times**:

     $$2022-03-14\ 20:30:00\ \text{UTC} < 2022-03-15\ 13:00:00\ \text{UTC}$$

     The time margin is 16 hours 30 minutes. Comparison must use complete UTC dates and timestamps, such as `2022-03-14 20:30:00 UTC` and `2022-03-15 13:00:00 UTC`. If the date crossing is carelessly ignored and the within-day time strings are compared directly (`20:30 < 13:00`), availability will be judged incorrectly. The system must compare absolute timestamps with time-zone awareness.

43-E06 Complete Solution

1. **Limitation of unigram bag-of-words**: A bag-of-words model counts each term separately without order and does not assume statistical independence among terms; it loses syntactic modifiers and word-order relationships. The fixed sentiment lexicon in this example assigns positive polarity weight +1 to `"improve"` and `"growth"`. If negation terms such as `"not"` and `"no"` are missing from the vocabulary or filtered as stop words, naive bag-of-words addition returns +2 and completely reverses the strongly negative meaning, labeling it highly positive.
2. **Possible repairs for this example (none guarantees general language accuracy)**:
   - Option 1: Add n-gram features (bigrams), extracting `"not_improve"` and `"no_growth"` as separate combined terms with negative polarity;
   - Option 2: Use a sliding window (such as a three-word window). When a negation term (`"not"`, `"no"`, or `"never"`) is encountered, activate a reversal flag and multiply the polarity of subsequent modified sentiment terms by −1.

43-P01 Reference Implementation

```python
import numpy as np
from quant_course.bayesian_text import beta_binomial_posterior_predictive

def verify_predictive_covariance(
    alpha: float, beta: float, successes: int, trials: int
) -> dict[str, float]:
    """Verify that predictive covariance equals posterior variance for arbitrary
    ↪ legal parameters."""
    res = beta_binomial_posterior_predictive(
        alpha_prior=alpha, beta_prior=beta, successes=successes, trials=trials
    )
    cov = res["joint_two_successes"] - (res["single_predictive_mean"] ** 2)
    var_post = res["posterior_variance"]
    assert np.isclose(cov, var_post, atol=1e-12), f"Covariance {cov} != Variance {
    ↪ var_post}"
    return {
        "alpha_post": res["alpha_post"],
        "beta_post": res["beta_post"],
        "predictive_covariance": cov,
        "posterior_variance": var_post,
    }

# Example calls: supports arbitrary legal hyperparameters
test_case_1 = verify_predictive_covariance(2.0, 2.0, 8, 10)
test_case_2 = verify_predictive_covariance(1.0, 3.0, 5, 20)
assert np.isclose(test_case_1["predictive_covariance"], 2.0 / 147.0)
```

43-P02 Reference Implementation

When `target_p_z=None`, this reference function uses the empirical stratum distribution as its target; the shared core `compute_stratified_ate` defaults to equal weights of 0.5 for high and low, so these functions do not target the same population by default. They align only when the same valid weights are passed explicitly. Weights must be finite, nonnegative, and sum to 1. If a positive-weight target stratum is entirely absent, the function must raise an error rather than drop it and renormalize. The checks below guard against missing sample support; they do not prove population positivity or conditional exchangeability.

```
import pandas as pd
from quant_course.bayesian_text import PositivityViolationError

def compute_stratified_ate_safe(
    df: pd.DataFrame,
    z_col: str = "Z",
    t_col: str = "T",
    y_col: str = "Y",
    target_p_z: dict | None = None,
) -> dict:
    """Compute stratified ATE with a strict positivity guard that raises a custom
    ↪ exception."""
    import math
    if df.empty:
        raise ValueError("Input DataFrame is empty.")
    for col in (z_col, t_col, y_col):
        if col not in df:
            raise KeyError(col)
    if df[[z_col, t_col, y_col]].isna().any().any():
        raise ValueError("Causal variables cannot be missing.")
    if not set(df[t_col]).issubset({0, 1}):
        raise ValueError("Treatment must be binary.")
    if not all(math.isfinite(float(y)) for y in df[y_col]):
        raise ValueError("Outcomes must be finite.")
    strata = df[z_col].unique()
    if target_p_z is None:
        p_z = {z: len(df[df[z_col] == z]) / len(df) for z in strata}
    else:
        p_z = dict(target_p_z)
    if not p_z or any(not math.isfinite(float(p)) or not 0 <= p <= 1
                      for p in p_z.values()):
        raise ValueError("Target weights must be finite probabilities.")
    if not math.isclose(sum(p_z.values()), 1.0, rel_tol=0, abs_tol=1e-12):
        raise ValueError("Target probabilities must sum to one.")
    for z, p in p_z.items():
        if p > 0 and z not in set(strata):
            raise PositivityViolationError(
                f"Missing target stratum \'{z}\': treated_count=0, control_count=0."
            )
    stratum_contrasts = {}

    for z in strata:
        sub = df[df[z_col] == z]
        treated = sub[sub[t_col] == 1]
        control = sub[sub[t_col] == 0]
        n_t1, n_t0 = len(treated), len(control)
        if n_t1 == 0 or n_t0 == 0:
            raise PositivityViolationError(
                f"Positivity violation in stratum \'{z}\': "
                f"treated_count={n_t1}, control_count={n_t0}."
            )
        stratum_contrasts[z] = treated[y_col].mean() - control[y_col].mean()

    ate = sum(p * stratum_contrasts[z] for z, p in p_z.items() if p > 0)
```

```
    return {"ate": ate, "stratum_contrasts": stratum_contrasts}

# Test: assert that a positivity violation raises the expected exception
df_fail = pd.DataFrame([
    {"Z": "stratum_a", "T": 1, "Y": 1},
    {"Z": "stratum_a", "T": 1, "Y": 0},  # stratum_a has no T=0 control
    {"Z": "stratum_b", "T": 1, "Y": 1},
    {"Z": "stratum_b", "T": 0, "Y": 0},
])

try:
    compute_stratified_ate_safe(df_fail)
    raise AssertionError("Failed to catch positivity violation!")
except PositivityViolationError as e:
    assert "Positivity violation in stratum 'stratum_a'" in str(e)
```

43-P03 Reference Implementation

The selector first checks for nonempty fields, time-zone-aware timestamps, unique source/version pairs, and one cutoff per source. It requires the time order event≤revision≤availability≤ingestion and signal<trade≤label_start<label_end≤label_available. Given these valid-input conditions, it filters using max(availability, ingestion)≤signal, then selects the complete row with the maximum `version_id`. Version identifiers must represent version order; the function does not additionally prove that revision time increases strictly with the identifier within each source. Multiple historical cutoffs must be handled by grouping in the caller; they are not supported directly by this interface. PIT eligibility does not establish data rights, model validity, or validity of the overall study.

```
import pandas as pd
from quant_course.bayesian_text import (
    filter_pit_eligible_documents,
    generate_constructed_text_dataset,
)

def test_pit_selection_and_deduplication():
    """Verify PIT selection rejects late revisions without splicing corrupted columns
    ↪ ."""
    base_df, invalid_df = generate_constructed_text_dataset(seed=4302, n_documents
    ↪ =120)

    # Inject a late revision (for example, version 2 published after the signal)
    late_revision = invalid_df.iloc[0:1]
    combined = pd.concat([base_df, late_revision], ignore_index=True)

    # Filter strictly at the signal-time cutoff
    filtered = filter_pit_eligible_documents(combined, cutoff_col="signal_time")

    # Assert that version 1 is retained, version 2 is fully excluded, and the total
    ↪ is exactly 120
```

```
    assert len(filtered) == 120
    row_10 = filtered[filtered["doc_id"] == 10]
    assert len(row_10) == 1
    assert row_10.iloc[0]["version_id"] == 1

test_pit_selection_and_deduplication()
```

43-P04 Reference Implementation

```
import numpy as np
from quant_course.bayesian_text import TextFeaturePipeline

def test_oov_invariance_and_frozen_vocabulary():
    """Verify that a train-fitted feature extractor never changes vocabulary or IDF
    ↪ for test OOV terms."""
    train_texts = [
        "quarterly revenue grew strongly beating analyst consensus",
        "earnings performance contracted showing weak operating margins",
    ]
    pipeline = TextFeaturePipeline()
    pipeline.fit(train_texts)

    vocab_before = dict(pipeline.vocabulary_)
    idf_before = np.copy(pipeline.idf_)

    # Input contains entirely new, unseen terms (OOV) and a perturbation string
    test_oov_texts = [
        "completely novel unexpected synthetic jargon xyz123",
        "artificial unigram token with zero intersection",
    ]
    vec_test = pipeline.transform(test_oov_texts)

    # Assert that the internal vocabulary and IDF remain completely unchanged
    assert pipeline.vocabulary_ == vocab_before
    assert np.array_equal(pipeline.idf_, idf_before)
    # The entirely new text maps to an all-zero vector, with no NaN or unknown
    ↪ overflow
    assert np.allclose(vec_test, 0.0)

test_oov_invariance_and_frozen_vocabulary()
```

43-C01 Complete Solution, Executable Counterexample, and Rubric

The arrows in a DAG do not determine the sign of the association after selection; the negative sign in this problem depends on independent Bernoulli(0.5) inputs and the OR rule. If the independent inputs are retained but the rule is changed to $C = \mathbb{I}(T = Y)$, conditioning on $C = 1$ leaves only 00 and 11, giving correlation $+1$. This is a counterexample under a different rule; it does not replace the original problem.

1. **Theoretical argument and algebraic derivation**: Let $T \in \{0, 1\}$ (a momentum breakout) and $Y \in \{0, 1\}$ (a quality-alpha signal) be two binary stock-selection factors. Assume they are independent and identically distributed across the full market, with marginal probabilities $P(T = 1) = 0.5$ and $P(Y = 1) = 0.5$. The four joint states $(t, y) \in \{(0, 0), (0, 1), (1, 0), (1, 1)\}$ each have unconditional prior probability $1/4$. The unconditional covariance is therefore $\mathrm{Cov}(T, Y) = 0$. Suppose portfolio admission allows a stock to be selected when either factor is strong (the collider node $C = \mathbb{I}(T = 1 \vee Y = 1)$). When the researcher restricts the empirical analysis to the constituent universe ($C = 1$):
   - State $(0, 0)$ is fully excluded;
   - The remaining states $(0, 1)$, $(1, 0)$, and $(1, 1)$ each have conditional probability $1/3$ given $C = 1$.
   - Conditional expectations:

$$E[T \mid C = 1] = 0 \times \frac{1}{3} + 1 \times \frac{1}{3} + 1 \times \frac{1}{3} = \frac{2}{3}$$

$$E[Y \mid C = 1] = 1 \times \frac{1}{3} + 0 \times \frac{1}{3} + 1 \times \frac{1}{3} = \frac{2}{3}$$

$$E[TY \mid C = 1] = 0 \times \frac{1}{3} + 0 \times \frac{1}{3} + 1 \times \frac{1}{3} = \frac{1}{3}$$

   - Calculate the conditional covariance:

$$\begin{aligned}\mathrm{Cov}(T, Y \mid C = 1) &= E[TY \mid C = 1] - E[T \mid C = 1]E[Y \mid C = 1] \\ &= \frac{1}{3} - \left(\frac{2}{3}\right)^2 = \frac{1}{3} - \frac{4}{9} = -\frac{1}{9} \approx -0.111111\end{aligned}$$

   - Conditional correlation:

$$\mathrm{Var}(T \mid C = 1) = \frac{2}{3}\left(1 - \frac{2}{3}\right) = \frac{2}{9}, \quad \mathrm{Corr}(T, Y \mid C = 1) = \frac{-1/9}{2/9} = -0.5$$

The two factors, originally fully independent, acquire a strong negative correlation of -0.5 in the selected sample solely because the researcher conditions on collider C. Running the code on 2,000 samples gives an empirical covariance of about -0.1144, close to the theoretical value $-1/9$; the finite-sample value is not the theoretical limit.

```
from quant_course.bayesian_text import demonstrate_collider_bias
r = demonstrate_collider_bias(n=2000, seed=4301)
print(r)
assert abs(r["conditioned_on_collider_covariance"] + 1/9) < 0.03
```

2. **Warning for quantitative investing**: In quantitative stock selection, if a researcher tests a factor only within large-cap index constituents such as the CSI 300 or S&P 500, inclusion in the index has been jointly selected by factors such as market capitalization and profitability. If the admission mechanism has the collider structure in this example, it may induce an association between factors inside the index (for example, a spurious reversal between momentum and value among large-cap stocks). Testing only within the index does not itself prove that this structure or bias exists. Sample-selection bias (Berkson's paradox) could otherwise be mistaken for a tradable market anomaly.
3. **Scoring rubric**:
   - Full credit (10 points): Draw the collider DAG clearly, derive conditional covariance $-1/9$ and correlation -0.5 in full, and explain the quantitative connection to constituent admission and survivorship bias;
   - Passing (6 points): Explain qualitatively that conditioning on a collider induces negative correlation, but omit the algebra for the three equally likely states;
   - Failing (0–5 points): Confuse a confounder with a collider.

43-C02 Open-Ended Reference Solution and Scoring Rubric

1. **Teaching JSON Schema and Python validation implementation**:

```
import json
import hashlib
from jsonschema import validate, ValidationError

# Define the formal JSON Schema
AUDIT_LEDGER_SCHEMA = {
    "$schema": "https://json-schema.org/draft/2020-12/schema",
    "title": "LLMOfflineResearchAuditLedger",
    "type": "object",
    "required": [
        "prompt_template_version",
        "sample_input_text",
        "full_prompt_sha256_hash",
        "model_status",
        "model_slug",
        "declared_model_cutoff",
        "provenance_table",
        "reproducibility_limits"
```

```
    ],
    "properties": {
        "prompt_template_version": {"type": "string"},
        "sample_input_text": {"type": "string"},
        "full_prompt_sha256_hash": {"type": "string", "pattern": "^[a-f0-9]{64}$"},
        "model_status": {"type": "string", "enum": ["not_run", "offline_cached"]},
        "model_slug": {"type": "string"},
        "declared_model_cutoff": {"type": "string"},
        "provenance_table": {
            "type": "object",
            "required": ["source_id", "historical_signal_time", "cutoff_leakage_risk"
    ↪ ],
            "properties": {
                "source_id": {"type": "string"},
                "license": {"type": "string"},
                "historical_signal_time": {"type": "string"},
                "cutoff_leakage_risk": {"type": "string"}
            }
        },
        "reproducibility_limits": {
            "type": "array",
            "items": {"type": "string"}
        }
    }
}

def validate_and_hash_audit_ledger(ledger_dict: dict, trusted_expected_hash=None) ->
    ↪ bool:
    """Check local digest consistency; optionally compare an independently trusted
    ↪ digest."""
    validate(instance=ledger_dict, schema=AUDIT_LEDGER_SCHEMA)

    # Recompute the hash and check consistency
    template = (
        "You are an offline equity research audit assistant.\n"
        'Analyze the following corporate transcript snippet under strict '
        'Point-in-Time constraints:\n'
        "Snippet: \"{transcript_snippet}\"\n"
        'Identify whether the statement reports historical performance or '
        'provides future guidance.\n'
        'Output format: JSON with keys "type" '
        '(HISTORICAL/FORWARD_LOOKING) and "polarity" (POSITIVE/NEGATIVE/NEUTRAL).\n'
        "Do not assume post-event market price reactions."
    )
    expected_prompt = template.format(transcript_snippet=ledger_dict["
    ↪ sample_input_text"])
    expected_hash = hashlib.sha256(expected_prompt.encode("utf-8")).hexdigest()

    if ledger_dict["full_prompt_sha256_hash"] != expected_hash:
        raise ValueError("Recorded prompt digest does not match the current input.")
    if trusted_expected_hash is not None and expected_hash != trusted_expected_hash:
        raise ValueError("Current prompt differs from the independently trusted
    ↪ digest.")
    return True
```

```
def audit_cutoff_relation(ledger):
    # Unknown means unevaluated, not PIT-pass. A later cutoff flags risk, not
    ↪ confirmed contamination.
    import pandas as pd
    declared = ledger["declared_model_cutoff"]
    if declared == "unknown":
        return "not_evaluated"
    cutoff = pd.Timestamp(declared)
    signal = pd.Timestamp(ledger["provenance_table"]["historical_signal_time"])
    if cutoff.tz is None or signal.tz is None:
        raise ValueError("Require full timezone-aware cutoff and signal timestamps.")
    return "potential_future_corpus" if cutoff > signal else "
    ↪ cutoff_only_insufficient_for_PIT"

from quant_course.bayesian_text import llm_offline_audit_ledger
valid_ledger = llm_offline_audit_ledger()
assert validate_and_hash_audit_ledger(valid_ledger)

# Teaching assumption: this digest was preserved independently before local mutation.

# This assignment illustrates a comparison; it does not create trusted timestamp
    ↪ custody.
trusted_baseline = valid_ledger["full_prompt_sha256_hash"]
changed_both = llm_offline_audit_ledger(sample_text="changed input and matching
    ↪ digest")
assert validate_and_hash_audit_ledger(changed_both)  # local consistency alone passes
assert changed_both["full_prompt_sha256_hash"] != trusted_baseline
try:
    validate_and_hash_audit_ledger(changed_both, trusted_baseline)
    raise AssertionError("Changed prompt escaped independent-baseline comparison")
except ValueError:
    pass
assert audit_cutoff_relation(valid_ledger) == "not_evaluated"
tampered = dict(valid_ledger, sample_input_text="changed input")
try:
    validate_and_hash_audit_ledger(tampered)
    raise AssertionError("Tampered input escaped hash validation")
except ValueError:
    pass
scenario = dict(valid_ledger, declared_model_cutoff="2023-12-31T00:00:00+00:00")
assert audit_cutoff_relation(scenario) == "potential_future_corpus"

# Invalid-case test: required fields are missing and the 64-character hash is invalid
invalid_ledger = {
    "prompt_template_version": "v1.2",
    "sample_input_text": "sample",
    "full_prompt_sha256_hash": "invalid_short_hash", # Fails the regular-expression
    ↪ pattern
    "model_status": "not_run"
}
try:
    validate(instance=invalid_ledger, schema=AUDIT_LEDGER_SCHEMA)
    assert False, "Failed to reject invalid schema!"
except ValidationError:
    pass # Correctly rejected
```

Date fields in the schema are strings only; a syntactically valid string is not necessarily a valid time. As above, each must still be parsed as a complete time-zone-aware timestamp. `unknown` returns `not_evaluated`, not a pass. The schema allows `offline_cached`, but the actual instance in this chapter remains `not_run`; no LLM was called.

2. **Three engineering safeguards and defenses against irreproducibility**:
    - **Safeguard 1 (physically isolate inputs and hash the complete prompt)**: Never provide the model with retrospective analysis or subsequent price reactions. The hash must cover the formatted "template + actual text," not only the static template string. Changing both the input and local digest can still pass the consistency check, so compare with an independently trusted reference; do not call the hash encryption or immutable evidence.
    - **Safeguard 2 (account for determinism limits and version drift)**: In production, recognize that `temperature=0` and a fixed seed do not guarantee bit-for-bit reproducibility across GPU microarchitectures, CUDA drivers, or inference-engine versions. Silent model-service updates can shift features, so persist and archive snapshots of extracted results for important decision times.
    - **Safeguard 3 (do not equate a pretraining cutoff with PIT)**: Never treat a commercial LLM's pretraining cutoff (such as 2023-12) as a verifiable historical point-in-time record. Training data later than the historical signal may include subsequent reports, so the cutoff statement cannot prove that historical data were free from leakage. If the model was not called, record `not_run / unknown` accurately.
3. **Scoring rubric**:
    - Full credit (10 points): Provide a standard JSON Schema and Python validation that rejects invalid inputs, explain complete-prompt digest consistency and the independent-reference boundary, demonstrate changing the input and digest together, and analyze the three engineering safeguards in depth;
    - Passing (6 points): Provide valid JSON and hashing, but omit formal Schema-validation code or show limited understanding of nondeterministic drift;
    - Failing (0–5 points): Assume that an LLM is inherently PIT-compliant and overlook look-ahead leakage from its pretraining cutoff.

43-C03 Complete Solution

This experiment tests extrapolation across time blocks with a shared template support, label maturity, and training-only preprocessing. It does not hold out new text or templates and therefore cannot demonstrate generalization to new text. The OOV check shows that `transform` does not modify the training vocabulary or IDF; it does not show predictive accuracy on unseen language. A future study of language generalization would require a separately frozen protocol that holds out text templates or company language while retaining time constraints. This batch does not change the experiment or retrain the model.

19. Further Reading

- **Must know**:
  - Gelman, Carlin, Stern, Dunson, Vehtari, and Rubin (2013/2025). *Bayesian Data Analysis* (BDA3), Sections 2.4, 2.5, and 5.4. Read closely on conjugate priors, posterior-predictive integration, and the closed-form normal partial-pooling solution.
  - Hernán, M. A., and Robins, J. M. (2020). *Causal Inference: What If*, Part I (Chapters 1–3). Read closely on potential outcomes, conditional exchangeability, positivity, and SUTVA.
- **Should know**:
  - Stan Development Team. *Stan User's Guide*, the official pages on *Hierarchical Regression* and *Posterior Predictive Checks*; this round inherits the historical source check and reading scope documented in the shared source notes. Understand production-level ideas for Bayesian hierarchical models.
  - Official scikit-learn documentation: *Text Feature Extraction* and *Common Pitfalls in Machine Learning*. Study how to avoid leakage in text-feature extraction.
- **Advanced**:
  - Hernán and Robins, *What If*, §§6.3 and 7.3: Continue practicing identification decisions using the collider path and backdoor criterion reviewed here.
  - Glasserman, P., and Lin, C. (2023). *Assessing Look-Ahead Bias in Stock Return Predictions Generated By GPT Sentiment Analysis*. Study quantitative research on pretraining-corpus look-ahead leakage and backtest contamination.
- **Optional**:
  - Official SEC EDGAR Application Programming Interfaces guide. Learn about real regulatory-filing data interfaces.
  - OpenAI (2025). *Why Language Models Hallucinate*. Understand hallucination and irreproducibility in financial applications.

---

20. References

- `BDA3`: Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013/2025). *Bayesian Data Analysis* (Third Edition). CRC Press. Electronic edition: `https://users.aalto.fi/~ave/BDA3.pdf`. The historical source check recorded in the shared source notes is inherited in this round. [Supports derivations and examples in Sections 4, 5, 6.1, 6.2, 8.1, and 8.2]
- `STAN_UG`: Stan Development Team. (2026). *Stan User's Guide*. `https://mc-stan.org/docs/stan-users-guide/`. The historical source check recorded in the shared source notes is inherited in this round; Stan was not run. [Supports hierarchical-model design in Sections 5, 6.2, and 7]
- `HERNAN_ROBINS_2020`: Hernán, M. A., & Robins, J. M. (2020). *Causal Inference: What If*. Boca Raton: Chapman & Hall/CRC. `https://www.hsph.harvard.edu/miguel-hernan/causal-inference-book/`. The historical source check recorded in

the shared source notes is inherited in this round; the book was not read in full. [Supports causal-identification derivations and Simpson's paradox in Sections 3, 4, 5, 6.3, 7, and 8.3]

- `CH43_SK_TEXT`: Scikit-learn Developers. *Text feature extraction; Common pitfalls.* Historical official-documentation check accessed 2026-10-11 and inherited in this round. `https://scikit-learn.org/stable/modules/feature_extraction.html` & `https://scikit-learn.org/stable/common_pitfalls.html`. [Supports the TF-IDF pipeline in Sections 5, 6.4, 9, and 12]
- `SEC_EDGAR_API`: U.S. Securities and Exchange Commission. (2024-06-06; updated 2025-04-08; historical source check accessed 2026-10-11 and inherited in this round). *EDGAR Application Programming Interfaces (APIs).* `https://www.sec.gov/search-filings/edgar-application-programming-interfaces`. [A pointer to general interfaces and processing delays in Sections 7 and 11; does not prove PIT eligibility or licensing for specific text]
- `LLM_LOOKAHEAD_GLASSERMAN_LIN`: Glasserman, P., & Lin, C. (2023). *Assessing Look-Ahead Bias in Stock Return Predictions Generated By GPT Sentiment Analysis.* arXiv:2309.17322, v1 (the historical source check recorded in the shared source notes reviewed only the abstract and metadata; this round inherits it). [Supports mechanisms of LLM look-ahead bias in Sections 5, 7, 12, and 13]
- `OPENAI_HALLUCINATION_2025`: OpenAI. (2025). *Why Language Models Hallucinate.* Official research explainer, 2025-09-05; historical source check inherited this round. `https://openai.com/index/why-language-models-hallucinate/`. [Supports LLM hallucination and factual-review practices in Sections 5, 12, and 18]

The historical reading scope and its inheritance in this round are recorded in shared `course/zh/chapters/ch43/source_notes.md`; BIL20 did not revisit these web pages or execute the historical protocols.

Project 6 —Machine-Learning Forecasts, Costs, and Evidence

Project 6 — Machine Learning Alpha

The research objective remains planned; the base linear pipeline has a constructed-data implementation with limited-scope acceptance. There is no empirical study on real markets: real data remain **blocked**, and the complete Project 6 has not been completed or marked verified. This page distinguishes accepted teaching implementations from work still needed for the full project. Constructed results are not market evidence.

Current scope and status

Scope	Current status	Notes
A8-A base pipeline	Limited-scope accepted; constructed teaching validation	Date-grouped walk-forward validation, training-only tuning, zero and training-mean baselines, OLS, Ridge, Lasso, and Elastic Net, prediction-to-portfolio weights, and a costed ledger. See `ACCEPTED_VERSION.md`.
A8-R1 display revision	Limited-scope accepted	Revised the display of frozen constructed results and retained negative results. It is not a new training run or empirical study. See `ACCEPTED_DISPLAY_REVISION_v1.md`.
A8-B tree-model extension	Limited-scope accepted, pending formal integration	Tree-model comparisons, training-only feature importance, stability, and registered ablation implementation on constructed data. See `ACCEPTED_VERSION_v2.md` and `research_report/v2_trees.md`.
Complete Project 6	Not complete; not verified	Real-data work remains blocked. The BIL19 bilingual tree and PCA guide, full project answers, and Loom are now author candidates awaiting independent review; they do not constitute full-project acceptance.

This page, the data dictionary, and the base-pipeline report describe the A8-A foundation. They do not replace the A8-B report and do not mean that Project 6 or real-data research is complete. The BIL19 candidates are linked in the v2 teaching guide, v2 research report, project answer key, and P6 Loom; their acceptance remains pending independent review.

Research question and data

On a constructed panel, can time-grouped validation and training-only model selection map prices and characteristics available at the close to next-period return forecasts, then form a dollar-neutral portfolio after costs? The base configuration has 46 features: 3 characteristics with nonzero constructed loadings, 40 zero-loading noise features, and 3 price features. The loadings and scenario boundaries are in the frozen configuration and the base-pipeline report. These values check teaching mechanisms and code; they do not support a market-alpha claim.

Any real-data work must first satisfy the source, licensing, field, sample, and retention requirements in `data_manifest.json`. There is currently no authorized real-data input frozen for use.

Method and timing contract

1. Specify feature construction, availability times, label interval, and forecast horizon.
2. Use expanding-window validation grouped by date; first exclude training rows whose labels are not yet mature, then apply any required time isolation. Shuffled K-fold is forbidden, and assets from the same signal date must remain on the same side of a split.
3. Fit preprocessing and select baselines or model hyperparameters using training data; use the outer test set only for evaluation.
4. Convert predictions to demeaned, dollar-neutral target weights, apply the configured trading fees, and report prediction and portfolio metrics separately.

In the base panel, the signal is formed at the close on D_k. The label runs from the close on D_{k+1} to the close on D_{k+6}, covering 5 close-to-close intervals; it is available for fitting only after the endpoint close. The portfolio executes at the close of the next trading day after the signal and rebalances every 5 signal dates. The ledger marks NAV each day; a five-day label is not a daily portfolio holding return. See the data dictionary for the full field, missing-data, and fee definitions.

Base-layer entry points

- `configs/p06_base_v1.json`: the frozen constructed-data configuration. Use a new version for any new setting and preserve v1.
- `src/p06_pipeline.py` and the shared `quant_course.ml_pipeline`: construct inputs and labels, validate by date, generate predictions and weights, and calculate ledgers and metrics.
- `run_project06.py --data constructed --out <new-directory>`: starts a new, full base-pipeline run from constructed inputs. The output directory must not already exist.
- `run_project06.py --data real --out <new-directory>`: currently returns a blocked status without writing output.
- `tests/test_p06_pipeline.py` and `tests/test_ml_pipeline.py`: implementation checks from the historical acceptance process.

- `notebooks/p06_baseline.ipynb`: reader entry point in the delivered package; use its notebook labels to identify the execution mode.
- `data_manifest.json`, `data_dictionary.md`, and `research_report/README.md`: data requirements, field definitions, and historical constructed results.

Reader replay and a fresh run

The default P6 notebook in the I3 reader package is a replay of a frozen display. It reads the summaries, display audit, and NAV images under `fixtures/projects/project06/`; it does not call `p06_pipeline.run`, retrain, or recompute daily ICs or paired dates. The notebook creates a unique `RUN_DIR` only to display the packaged material. Creating that directory does not produce a new model run. Older notebook prose that calls this a "cold-start run" is stale; the actual code and reader-mode labels define the current behavior.

Running `run_project06.py --data constructed ...` separately reconstructs the panel, trains the models, and writes predictions, fold records, weights, ledgers, and summaries. That is a fresh run with its own identity. Do not treat it as the same run as the I3 display replay or its frozen values.

The two I3 `base/regime` summaries are byte-identical to the historical `rw_qa2` summaries. Compared with `qa_r3`, all summary fields match except `/seconds`. `qa_r3` and `rw_qa2` remain distinct run identities; this comparison does not establish that unexamined CSVs, weights, or ledgers are identical. The 13 corresponding NaN fields in the summaries mean that the metric is undefined; they must not be treated as zero. The historical paired-date IC audit has a separate frozen-row evidence source; see section 3.4 of the report.

The I3 reader notebook for P6 v2 is a separate frozen replay: 9 cells, including 4 code cells. It replays two summaries, IC date bases, and historical ablation and stability displays, and generates `historical_constructed_trees_sharpe.png` from frozen summaries into `RUN_DIR.` Fit is marked SKIPPED; it does not retrain. Keep this display distinct from the base notebook in this section and from any new full CLI run identity.

Deliverables and interpretation

- `data_manifest.json`: data source, licensing boundary, sample, units, SHA-256, and data version.
- `data_dictionary.md`: fields, units, keys, timestamps, labels, and missing-data rules.
- `notebooks/`: reader entry points; the implementation lives in `src/`.
- `src/`, `tests/`, and `configs/`: pipeline implementation, historical checks, and frozen configuration.
- `figures/`: constructed-result figures with axis and data notes.
- `research_report/`: constructed assumptions, methods, historical results, costs, negative results, and limitations.
- BIL19 has produced an author candidate for `solutions.md` and the bilingual answer set; independent review remains pending. Teaching simulations must not stand in for real-data research.

Reports must retain zero and negative results. Spearman IC, out-of-sample R^2, and net portfolio returns answer different questions and should be reported separately. In addition to proportional fees, borrowing fees, financing spreads, market impact, capacity, and margin costs are not modeled. Research conclusions from real data remain blocked.

BIL18 foundation materials have limited-scope acceptance pending formal integration. The BIL19 tree and PCA guide, report, answer key, and bilingual Loom are candidates awaiting independent review; BIL23 handles notebook language projections. Real-data research remains blocked and full Project 6 remains unverified.

The complete question bank and answers use P6-L01–L08 and P6-T01–T04. The latter cover label maturity, joint within-date permutation, full-dimensional PCA equivalence, and event-ledger calculations.

Project 6 Data Dictionary (A8-A Constructed Base Panel)

Scope and status: This dictionary describes the frozen A8-A base configuration and its constructed panel. It is used to check the timing contract, pipeline, and teaching mechanisms; it is not market evidence. Real-data research remains **blocked**. Licensing and field requirements are in `data_manifest.json`. The complete Project 6 is not complete or verified; the A8-B tree-model extension has its own configuration, report, and acceptance entry.

Keys, frequency, and sample

- Frequency: daily. The frozen constructed configuration uses 1,512 business-day index dates starting on 2015-01-02; this is not an exchange trading calendar.
- Panel key: `(signal_time, asset)`. All assets on the same `signal_time` remain together in any split.
- Assets: the constructed sample contains 40 assets, `A00` through `A39`, with no listings, delistings, splits, or dividends. A real-data project would need separate corporate-action handling (Chapters 29 and 30); an adjusted price must not be treated as an execution price.
- Frozen configuration: `configs/p06_base_v1.json`, SHA-256 `62f49e1cab11b6992a9c391180df5d6827f9c5c12bf34f94acde5134ee4c39f7`. This file is read-only; any new setting requires a new version.

Time fields and labels

Field	Definition	Value in the constructed panel	Constraint
`event_time`	Time of the source-data event	Closing price: that day's close	—
`available_at`	Time when the researcher can read the value in the system	Price-derived features and constructed `q_*`/`noise_*` are defined as available at that day's close	At the date level, no later than `signal_time`; the exact timestamp must be no later than signal cutoff `tau_k`
`signal_time`	Date when features are computed and a forecast is formed	After the close on `D[k]`	—
Decision time `tau_k`	Exact signal cutoff	After the closing price is final (for example, 16:10 local exchange time after a 16:00 close)	Data available only after the cutoff may be used no earlier than the next signal date
`trade_time`	Execution time for rebalancing from the forecast	Close of the next trading day, `D[k+1]`; `execution_lag = 1`	Later than `signal_time`
`label_start`	Start of the label return	Close at `trade_time = D[k+1]`	—
`label_end`	End of the label return	Close on `D[k+6]`; `h = 5`	—
`label_available_at`	Time when the label is mature for training	Close at `label_end`	A training row must satisfy `label_available_at <= fit_time`
`fit_time`	Fit time for an outer fold	First test `signal_time` in that fold	Use only labels mature by this time

The label is `y = close[label_end] / close[label_start] - 1`, in decimal units. Its return interval is `(label_start, label_end]`, covering five close-to-close intervals. It starts at the next trading day's close and ends at the close five trading days later. The label becomes mature only after the endpoint close. The portfolio ledger marks NAV at each daily close; this five-day label is not a daily portfolio holding return.

The 46 base features

The base configuration has 46 features: 3 slow-moving characteristics with nonzero constructed return loadings, 40 zero-loading noise features, and 3 price features. The constructed features `q_1` through `q_3` and `noise_01` through `noise_40` are unit-variance AR(1) series with `char_rho = 0.98`. The constructed daily simple-return loadings per unit of `q_1`, `q_2`, and `q_3` are 0.03%, 0.015%, and 0.0075%, respectively; the loadings on `noise_*` are zero. In the predefined `regime_flip_at_75pct` scenario, the nonzero loadings change sign at 75% of the dates. These are constructed settings.

Feature	Definition or constructed source	Unit	Available at	Missing-data rule
`rev_5`	`-(close[t] / close[t-5] - 1)`	Decimal	At that day's close	NaN for the first 5 days
`mom_60_5`	`close[t-5] / close[t-60] - 1`; skips the latest 5 days	Decimal	At that day's close	NaN for the first 60 days
`vol_20`	Sample standard deviation (`ddof=1`) of the last 20 daily simple returns	Decimal per day	At that day's close	NaN for the first 20 days
`q_1`, `q_2`, `q_3`	Constructed slow-moving AR(1) characteristics with nonzero loadings	Unit-variance feature units	At that day's close	None
`noise_01`…`noise_40`	Constructed slow-moving AR(1) noise characteristics with zero loadings	Unit-variance feature units	At that day's close	None

All constructed features are defined as available at the close under the configuration. A real-data project must separately check event time, system availability time, and signal cutoff; a date label alone does not prove the absence of look-ahead.

Missing data, validation, and metrics

The shared `quant_course.ml_pipeline` rules are: drop and count a training row if any feature or its label is NaN; do not forecast a test row with a NaN feature, assign it zero target weight, and count it; raise an error for infinite values. Keep every asset from a date on the same side of a split. Each training row must have a label mature by `fit_time`. Fit preprocessing and select hyperparameters using training data and inner validation only; evaluate the outer test set once. Shuffled K-fold is forbidden.

Report prediction metrics (such as out-of-sample `R^2` relative to the zero and training-mean baselines and daily Spearman IC) separately from portfolio metrics. Cross-sectional IC is undefined on a date when predictions are constant. Keep undefined values missing; never replace them with 0. The paired IC in section 3.4 uses descriptive mean differences over dates when both models have valid predictions.

Portfolio rule and costs

- `predictions_to_target_weights` demeans each prediction cross-section and then normalizes by the sum of absolute weights so that `sum(abs(w)) = 1` and `sum(w) = 0`. This is dollar neutrality, not beta neutrality. The portfolio stays flat if fewer than two forecasts are finite or if the cross-section has zero dispersion.
- Rebalance every 5 signal dates. The signal is formed at the close on `D[k]`; `run_backtest(lag=1)` executes at the close of the next trading day, `D[k+1]`.
- Configured one-way proportional fees are 0, 0.001, and 0.002 (0, 10, and 20 basis points); initial cash is 1,000,000. NAV is marked at each daily close; fees are charged on traded notional.
- Only the proportional fees above are included. Borrow fees, financing spreads, market impact, capacity, and margin costs are not modeled. Under the Chapter 29 convention, short-sale proceeds remain fully in cash.

Historical status and limits

The A8-A base implementation and constructed teaching validation were accepted within a limited scope, as was the A8-R1 display revision. The A8-B tree-model extension has a separate limited-scope acceptance entry. Constructed results describe model behavior under a frozen simulated design; they do not establish real-market predictability, returns, or alpha. Real-data research remains **blocked**, and the complete Project 6 has not been verified.

Project 6 Base Pipeline Report (A8-R1 Frozen Display)

> This is not an empirical research report. Real data remain **blocked**. All figures below come from a constructed panel (`data_class = CONSTRUCTED`) and illustrate the base pipeline's timing contract, splits, training-only tuning, portfolio rules, lagged execution, and cost ledger. The signals in the panel were specified by the author; no return or ranking is evidence about markets. A8-A's base implementation and A8-R1's display revision each have a limited-scope acceptance entry. The A8-B tree-model extension also has its own limited-scope acceptance entry. This report covers only the base layer; it does not establish that Project 6 or real-data research is complete.

1. Question and hypothesis (mechanism)

Can 46 features available at the close (3 signal features, 40 noise features, and 3 price features) be passed through a regularized linear model selected within training data to produce a dollar-neutral portfolio with positive returns after costs and outperform both the zero forecast and the training-mean baseline? In the construction, the three signal features have daily expected-return loadings of 0.03%, 0.015%, and 0.0075% per feature unit. The signal-to-noise ratio is lower than in the Chapter 39 construction.

2. Frozen design

Configuration: `configs/p06_base_v1.json` (read-only; SHA-256 `62f49e1cab11b6992a9c391180df5d6827f9c5c12bf34f94acde5134ee4c39f7`). It was frozen before any Project 6 run. Constructed results seen before the freeze (Chapters 38 and 39) are recorded in `seen_before_freeze`.

- Data: 40 assets and 1,512 dates starting on 2015-01-02; seed 6061. The predefined `regime_flip_at_75pct` scenario reverses the signal loadings at 75% of the dates.
- Timing: `signal_time = D_k` close; `trade_time = label_start = D_{k+1}` close; `label_end = label_available_at = D_{k+6}` close; `h = 5`.
- Validation: expanding-window walk-forward grouped by date, with a 504-date initial training window and 63-date test blocks over 16 folds (first fit time: 2016-12-08). Each fold excludes 200 rows whose labels are not yet available, calculated as $(\ell + h - 1) \times N = 200$. Inner selection uses two 63-date validation blocks and at least 252 training dates. Shuffled K-fold is forbidden.
- Models: zero forecast, training mean, OLS, Ridge (α from 10^2 through 10^7), Lasso (5 grid values), and Elastic Net (4×3 grid values). Select the candidate with the lowest mean inner-validation MSE.
- Portfolio: demean predictions cross-sectionally, normalize so absolute weights sum to 1, and maintain dollar neutrality. Rebalance every 5 signal dates; execute at the close of the next trading day after the signal. One-way proportional fees are 0, 10, and 20 basis points; initial cash is 1 million. NAV is marked at each daily close using the Chapter 29 `run_backtest`. A five-day label is not the same as a daily portfolio holding return. Borrow fees, financing spreads, market impact, capacity, and margin costs are not modeled.

3. Results (historical constructed runs)

Tables 3.1 and 3.2 retain the constructed results from the historical runs `build/batches/A8-A/r1/runs/qa_r3/cli/p06_base` and `p06_regime`. The two frozen summaries in the I3 reader package are byte-identical to the `rw_qa2` summaries under `qa/a8/ch37_ch39_batch/rework_r2/evidence/rw_qa2/`. Compared field by field with the `qa_r3` summaries, only the runtime field `/seconds` differs; every other summary field matches. `rw_qa2` and `qa_r3` have distinct run identities. This comparison does not establish that unexamined CSVs, weights, or ledgers are identical. The 13 corresponding NaN fields in each summary remain undefined and must not be read as zero.

The default P6 notebook in the I3 reader package reads frozen summaries, the display audit, and NAV images. Its actual code does not call `p06_pipeline.run`, retrain, or recompute daily ICs or paired dates. Older notebook prose that says it "cold-starts the pipeline from the frozen configuration" does not match its current code; the reader-mode label and actual code define it as a display replay. By contrast, running `run_project06.py --data constructed ...` reconstructs the panel, trains the models, and writes predictions, weights, ledgers, and summaries. That is a new run with a distinct identity. Do not describe it as the same run as the I3 replay.

The common-date audit in section 3.4 comes from a separate set of frozen daily rows and its A8-R1 `display_02` evidence. Its values were not derived from the summary JSON, and must not be merged with unexamined run files as if they shared one run identity.

3.1 Base scenario

Model	R^2_{OS} vs zero	vs training mean	Mean IC (own valid dates)	t statistic shown
Zero forecast	0	+0.861%	—	—
Training mean	−0.868%	0	—	—
OLS	−1.715%	−0.839%	0.0203	—
Ridge	−0.982%	−0.113%	0.0198	—
Lasso	−0.952%	−0.083%	0.0179	—
Elastic Net	−0.879%	−0.010%	0.0309	—

Model	Own valid dates	Gross total return / Sharpe	Net after 10 bp	Net after 20 bp
Zero forecast	0	0 / —	0 / —	0 / —
Training mean	0	0 / —	0 / —	0 / —
OLS	1,002	+20.57% / 0.79	+9.39% / 0.39	−0.76% / 0.00
Ridge	1,002	+21.88% / 0.84	+10.90% / 0.45	+0.91% / 0.07
Lasso	630	+6.51% / 0.36	−0.20% / 0.01	−6.50% / −0.33
Elastic Net	882	+30.92% / 1.19	+19.90% / 0.81	+9.81% / 0.43

Model	Max drawdown after 10 bp	Annual turnover
Zero forecast	0	0
Training mean	0	0
OLS	−14.2%	24.3
Ridge	−12.9%	23.6
Lasso	−9.3%	16.3
Elastic Net	−7.2%	22.0

The test period contains 16 folds × 63 = 1,008 signal dates (about four years). The labels for the final six dates extend beyond the sample, leaving 1,002 dates in the prediction metrics.

3.2 Scenario: signal loadings reverse at 75%

Model	Mean IC (own valid dates)	t statistic shown	Before costs	Net after 10 bp	Net after 20 bp
OLS	0.0089	—	+3.52% / 0.17	−6.09% / −0.22	−14.80% / −0.61
Ridge	0.0087	—	+5.05% / 0.23	−4.44% / −0.15	−13.08% / −0.53
Lasso	0.0010	—	−4.66% / −0.25	−10.10% / −0.57	−15.23% / −0.89
Elastic Net	0.0152	—	+9.61% / 0.45	+1.09% / 0.08	−6.77% / −0.29

3.3 How to read the results

1. **The zero forecast beats the training mean on out-of-sample R^2.** The training-mean baseline is −0.868% relative to zero. The historical-mean noise at the individual-stock level is large, consistent with the discussion after equation (19) in Gu, Kelly, and Xiu (2020) and section 11 of Chapter 37. The four fitted models (OLS, Ridge, Lasso, and Elastic Net) have negative R^2 relative to zero.
2. **Keep prediction and portfolio results separate.** For those four fitted models, R^2 relative to zero is negative, while cross-sectional ICs are positive and dollar-neutral portfolios earn positive returns before costs. This is consistent with the level-versus-cross-section decomposition in section 6.5 of Chapter 38.
3. **Costs matter.** Annual turnover is about 16–24 times. At a one-way fee of 20 basis points, OLS and Lasso are negative and Ridge is near zero.
4. **Retain negative results.** Lasso returns −0.20% after 10 basis points. In the loading-reversal scenario, OLS, Ridge, and Lasso are all negative after 10 basis points, while Elastic Net is near zero. The fold-by-fold IC figure (`p06_fold_ic_regime.png`) shows negative IC in most folds after the reversal.
5. **Tuning is unstable.** Ridge's selected alpha jumps between 10^4 and 10^7 across folds (the mechanism discussed in section 6.7 of Chapter 39). Lasso sets all coefficients to zero in some folds, so its IC covers only 630 dates.
6. Elastic Net performs best under this particular constructed setting. That result alone says nothing about real data; the same protocol would need to be run on authorized real data.

3.4 IC dates and common-date pairing (recomputed from existing frozen rows in A8-R1)

There are 1,002 candidate label dates in each scenario. Let `D_m` be a model's own valid-date set and $D_m \cap D_R$ its common-date set with Ridge. Compute both paired means and their difference only on the common set. Do not subtract Ridge's mean over all 1,002 dates from a sparse model's mean over its own dates. IC is undefined for constant predictions; do not fill it with zero or invent an observation.

Scenario / model	Own valid / candidate dates	Missing: constant-prediction dates	Common dates with Ridge	Model mean on pair
Base / Ridge	1002/1002	0	1002	0.01982995
Base / Lasso	630/1002	372	630	0.01785759
Base / Elastic Net	882/1002	120	882	0.03087240
Reversal / Ridge	1002/1002	0	1002	0.00865399
Reversal / Lasso	567/1002	435	567	0.00100360
Reversal / Elastic Net	819/1002	183	819	0.01521028

Scenario / model	Ridge mean on pair	Difference in paired means
Base / Ridge	0.01982995	+0.00000000
Base / Lasso	0.01613330	+0.00172429
Base / Elastic Net	0.02060216	+0.01027024
Reversal / Ridge	0.00865399	+0.00000000
Reversal / Lasso	0.00308692	−0.00208331
Reversal / Elastic Net	0.00909016	+0.00612013

Base Lasso and Elastic Net have 630/882 valid dates; in the reversal scenario they have 567/819. The corresponding missing counts are 372/120 and 435/183. A8-R1's frozen-row check confirmed that each of these missing dates had constant predictions. Ridge has 1,002 dates in both scenarios; OLS has 1,002 in both; the zero forecast and training mean have none. Daily and paired CSVs for all models, source hashes, and checks are under `qa/a8/limited_revision_ch39_ch43_p06_v1/evidence/display_02/`. This display audit did not modify the older summaries.

The A8-R1 display does not report the old IC t statistics. The old `ic_summary` dropped missing values and then computed NW(4) over observation indices; for a sparse daily series, a lag of four observations is not four trading days. The old values remain only in the previously accepted report and summary. A8-R1 computed no new HAC statistic and makes no claim of statistical significance. The daily series for models without missing dates was confirmed to be continuous in the original inspection, but the display removed the historical t-statistic inference throughout to avoid mixing conventions. P6 v1 and Chapter 39 are different experiments; these rows cannot be used to recompute Chapter 39's historical paired quantities.

Complete Q&A

- **P6-L01** · **Question:** Can the base Lasso IC of 0.01785759 be compared directly with Ridge's 0.01982995 to conclude that Lasso is weaker? **Answer:** No paired-superiority claim follows because the models have different valid-date sets. On the common 630 dates, Ridge's mean is 0.01613330 and the paired mean difference is +0.00172429. This remains descriptive; this display provides no valid inferential t statistic.
- **P6-L02** · **Question:** Does compressed NW(4) represent a four-trading-day HAC? **Answer:** No. For example, if valid trading-date indices are 0/2/3/5/8, a fourth-order lag over valid observations connects indices 0 and 8, which are eight trading days apart.

- **P6-L03 · Question:** Can Chapter 39 reuse these P6 paired results? **Answer:** No. The frozen row-level predictions needed for Chapter 39 are unavailable, so those paired values could not be recomputed. Do not retrain to fabricate historical observations.

4. Historical execution records

- The historical acceptance process checked `tests/test_p06_pipeline.py` (6 tests) and the shared `tests/test_ml_pipeline.py` (26 tests at c2; 34 after A8-A revision r2 added label-timing contract tests; mutation checks killed 11/11 mutants).
- Historically, `run_project06.py --data real` returned exit code 3 and wrote no files.
- The A8-A base notebook has a historical clean-kernel execution record. The `A8-R1/I3` reader notebook is a frozen display replay and does not repeat that training.
- Historical checks confirmed that every training row in each fold satisfied `label_available_at <= fit_time`, and the zero and training-mean portfolios had zero turnover. These are historical acceptance records, not results newly produced for this bilingual revision.

5. Limitations and remaining work

- **Real data:** An authorized daily equity panel that may be retained for internal training is required (see `data_manifest.json` for fields and licensing). No Project 5 universe or purchase candidate is inherited. Real-data work remains blocked.
- **Tree models, training-only feature importance, stability, and ablation** have a separate A8-B constructed-teaching implementation and limited-scope acceptance entry; see `ACCEPTED_VERSION_v2.md` and `research_report/v2_trees.md`. This base report does not replace the separate results.
- **Complete Project 6** remains unverified. BIL19 has author candidates for full bilingual teaching integration, the PCA extension, and the complete answer set; independent review is pending.
- The cost model includes proportional fees only. Borrowing, financing, market impact, and capacity are not modeled (modules from Chapters 35–36 may be connected in the complete project).

This bilingual candidate documents an existing frozen display and historical acceptance evidence. It is not new training, simulation, notebook execution, or real-data validation. Real-data work remains blocked and the complete project remains unverified; prior acceptance versions, receipts, and the frozen configuration are unchanged. The BIL19 candidates add tree and PCA teaching, stable questions P6-L04–P6-L08, complete answers, and a bilingual Loom; see the v2 guide and project-level solutions.md. The three original questions in this report are numbered P6-L01–P6-L03.

Reading the frozen constructed figures

All four frozen PNGs are retained byte for byte. They describe constructed results, supply no new significance test, and cannot fill Chapter 39's missing rows.

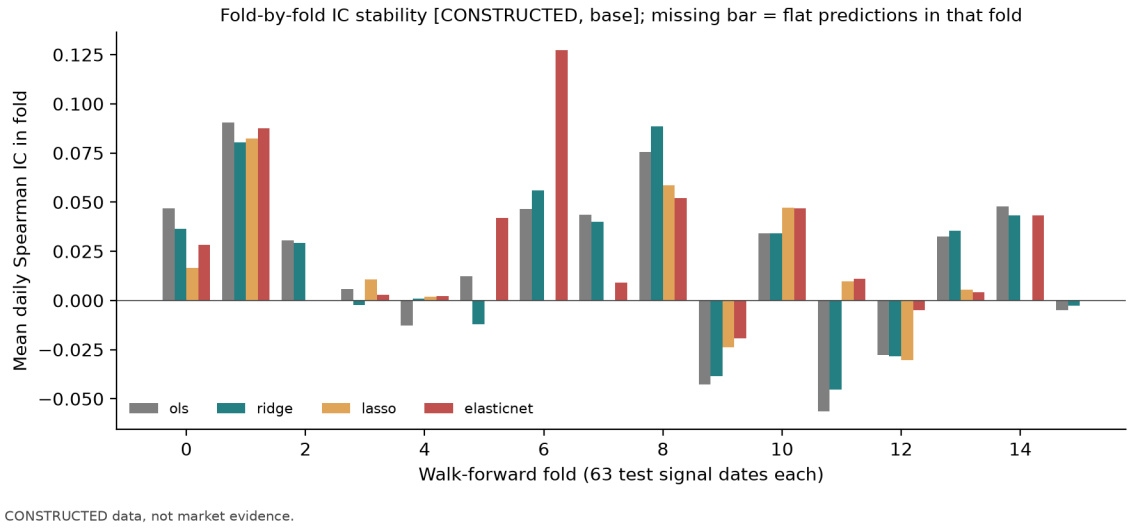


Figure 1: Base scenario: IC by fold

Horizontal axis: outer fold (63 test signal dates each); vertical axis: mean daily Spearman IC. Colors identify the four models; a missing bar denotes a flat predictor in that fold. Results vary across folds.

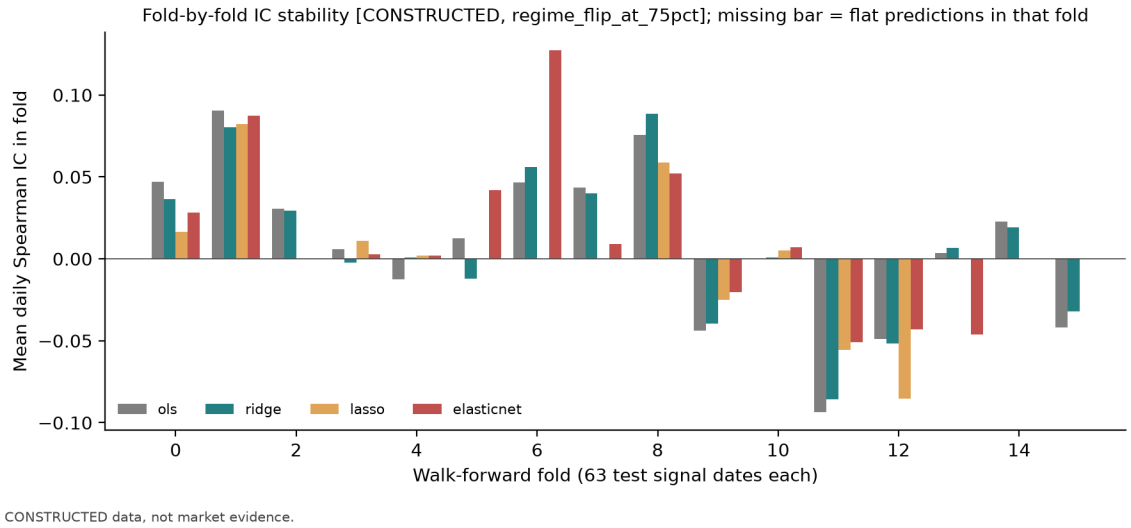


Figure 2: Loading-reversal scenario: IC by fold

The axes and colors match the base plot. Many folds after the loading reversal have negative IC; the final positive folds do not establish general regime adaptation.

Horizontal axis: ledger date; vertical axis: NAV in millions, starting at 1.0. The legend pairs each model's gross and 10-basis-point paths; proportional fees lower wealth.

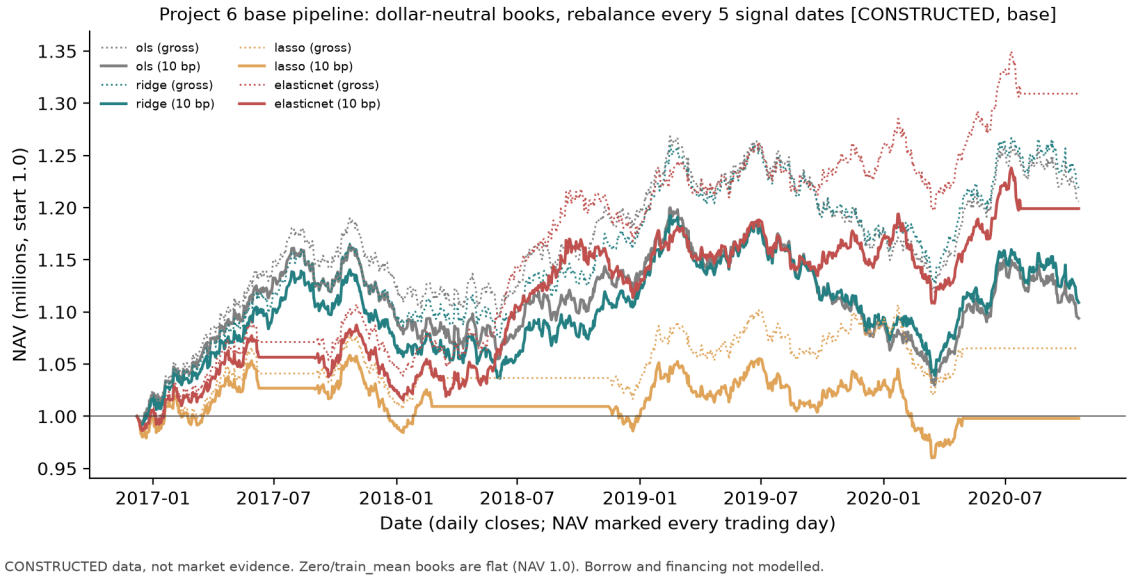


Figure 3: Base scenario: costed NAV

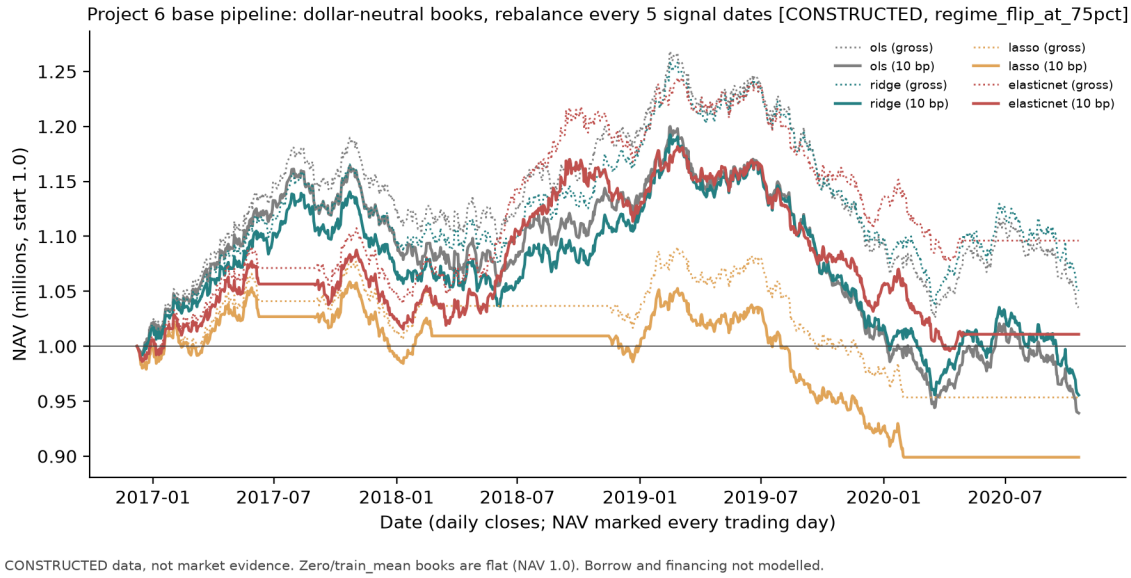


Figure 4: Loading-reversal scenario: costed NAV

The axes and fee convention match the base NAV plot. Wealth falls after the loading reversal for several models; the full path, including losses, must be retained.

Project 6 v2 Teaching Guide: Tree Models, Stability, Ablation, and PCA + Ridge

Author: Yuxiao Ma · **Revision date:** 2026-10-16.

Candidate status: This guide extends the A8-A foundation, which has limited-scope acceptance pending formal integration under BIL18. The A8-B constructed-data extension has its own limited-scope acceptance entry. The bilingual teaching materials in this BIL19 batch remain author candidates pending independent review. Real-data research remains **blocked**, and the complete Project 6 remains unverified. The numbers here are teaching evidence under a specified data-generating process, not market evidence.

This guide connects the linear baselines, tree models, PCA, portfolio accounting, and limits on inference. Full tables and frozen historical qualifications are in the base-pipeline report and v2 research report. The complete project question set and answers are in solutions.md.

1. Keep the experiment identity fixed

P6 v1 is a linear experiment with 46 features, 1,512 trading dates, and 16 folds. Its 46 columns comprise 3 constructed signal features, 40 constructed noise features, and 3 price features. Version 2 keeps the same frozen panel and the base and loading-reversal scenarios, then adds a single tree, a random forest, gradient boosting, and PCA + Ridge. It is a different experiment from the Chapter 40 interaction example: that example has 26 features, 1,260 dates, 12 folds, and a registered primary strength and a separate fivefold post hoc sensitivity. Do not transfer its parameters, dates, tables, or conclusions into P6.

The generator makes q sequences and noise sequences independent across sequences; each sequence is a stationary AR(1) over time with coefficient 0.98 and marginal variance 1. Thus, independence across generated sequences does not mean that each feature is IID over time. The price features `rev_5`, `mom_60_5`, and `vol_20` are derived from a return path with a common market shock, q signal, and idiosyncratic shock. The 43 characteristic columns and 3 derived price columns have different structures, and finite-sample covariance also affects the principal components.

The v1 implementation produces six prediction sequences: zero, `train_mean`, OLS, Ridge, Lasso, and Elastic Net. Version 2 adds four: a single tree, random forest, gradient boosting (HistGradientBoostingRegressor, HGB), and PCA + Ridge. There are 10 prediction sequences in the frozen run. The historical main table and comparison figures show only 8 fitted models, omitting the zero and `train_mean` baselines. PCA + Ridge is a linear model after dimension reduction, not a tree.

2. Timing contract and model comparison

The signal forms at the close on D_k. The label runs from the close on D_{k+1} to the close on D_{k+6}, spanning five close-to-close intervals; it matures only after the endpoint close. The portfolio executes at the close of the next trading day after the signal and rebalances every five signal dates. A daily NAV ledger is not the same thing as a five-day label.

Outer validation uses date-grouped expanding-window walk-forward folds: at least 504 training dates, 63 test dates per fold, and 16 folds. Exclude rows whose labels have not matured before fitting. Keep all assets on a signal date on the same side of a split. The inner loop uses the final two 63-date validation windows and at least 252 training dates; select parameters by mean validation MSE. The outer test set is for evaluation only. Gradient boosting uses HistGradientBoostingRegressor with `early_stopping`=False; its iteration cap is 300, selected with the other parameters in the inner date folds, followed by refitting on all outer-training rows.

The frozen grid below is not an unrestricted Cartesian product of every setting:

Model	Candidate settings	Selection
Single tree	Depth 2/3/4/6; at least 1,000 rows per leaf	4 candidates; mean inner MSE
RF	200 trees; `max_features`=0.33; `max_samples`=0.5; leaf size 500 or 2,000	2 candidates; mean inner MSE
HGB	(learning rate, depth)= (0.05,2)/(0.05,3)/(0.02,3); minimum leaf size 500; cap 300 rounds	3 candidates; average each round's MSE over the same two active inner folds
PCA+Ridge	k=5/15/30 crossed with alpha=$10^{4/10}$6; no whitening	6 candidates; fit Scaler and PCA only on current training rows

Read the metrics separately:

- R^2_{OS} measures error in asset-level return forecasts. The frozen tables report it against both the zero forecast and the training-mean baseline.
- Daily Spearman IC measures the cross-sectional ranking of forecasts and outcomes on a signal date. Its own-date mean uses dates where that model's IC is defined.
- ΔIC against Ridge is calculated only on dates where both models have defined IC. Both means and their difference must use that same common-date set.
- Sharpe, net return, drawdown, and turnover come from the portfolio ledger after applying the weights, execution timing, and costs. IC cannot substitute for a portfolio result.

There are $16 \times 63 = 1,008$ nominal test signal dates. After labels mature, each scenario has 1,002 candidate dates; labels for the final six dates extend beyond the sample. Lasso and Elastic Net produce constant forecasts on some dates, making the cross-sectional IC NaN (undefined), not zero. Their own valid dates are 630 and 882 in the base scenario, and 567 and 819 in the reversal scenario. Ridge comparisons use the same 630/882 and 567/819 common dates. Never subtract a sparse model's own-date mean from Ridge's mean over all 1,002 dates.

The historical summaries calculated NW(4) after dropping missing dates and indexing the remaining observations consecutively. A fourth-order lag over compressed observations is not

a four-trading-day HAC. The revised report does not use these compressed-series t values as trading-day inference and computes no new HAC. Historical t values for complete, continuous series remain explicitly unadjusted and descriptive. See the report for multiplicity limits.

3. Costs can change the portfolio conclusion

On each signal date, the forecast is demeaned cross-sectionally and scaled to a dollar-neutral target with gross exposure 1. The ledger rebalances every five signal dates and executes at the next trading day's close. The frozen one-way fee schedule is 0, 10, or 20 basis points, with initial capital of 1 million. Proportional fees are included; borrow fees, financing spreads, market impact, capacity, and margin costs are not modeled. Short-sale proceeds are credited to cash under the existing ledger convention; that does not mean real-world funding costs are fully represented.

In the base scenario, Elastic Net's historical Sharpe after 10/20 bp is 0.81/0.43, with net total returns of +19.90%/+9.81%. This is performance only under this constructed setting. In the reversal scenario all eight main fitted models have negative Sharpe at 20 bp; the flat `zero/train_mean` baselines have undefined Sharpe. Annual turnover is 35.1–36.2 for a single tree, 29.2–29.7 for RF, 36.2–36.5 for HGB, and about 14.7–24.3 for linear models. Greater model complexity did not remove turnover and cost pressure. Full per-model tables, returns, drawdowns, and ablations are in section 3 of the research report.

4. PCA + Ridge: dimension reduction is not automatically worse

In each fold, the frozen pipeline fits StandardScaler on training data, then fits full-SVD PCA and Ridge. Whitening is off, and no preprocessing is fitted on the outer test set. If all 46 components are retained under the same standardization, Ridge alpha, and no-whitening condition, PCA is an orthogonal rotation. The Ridge penalty norm is preserved, making the full-component model equivalent to Ridge in standardized coordinates. Truncation restricts coefficients to the retained subspace: it can reduce noise or discard a predictive direction, so it is not guaranteed to improve forecasts.

The observed results do not support saying that PCA + Ridge is uniformly worse. Its R^2 relative to the training mean is -0.008% in the base scenario, slightly better than Ridge at -0.113%; in the reversal scenario it is -0.053%, versus Ridge at -0.161%. Under reversal, its 10/20 bp Sharpe is $-0.14/-0.52$, also slightly higher than Ridge's $-0.15/-0.53$. On the other hand, PCA + Ridge's mean IC is 0.0135 and 0.0063 in the two scenarios, below Ridge's 0.0198 and 0.0087. These metrics answer different questions; do not selectively summarize them as an overall win or loss.

PCA selected 30 components in 10 folds, 5 in 5 folds, and 15 in 1 fold. This describes inner-loop choices under this setup; it does not establish that the selected dimension will remain effective on new samples, real data, or another cost structure. See the full comparisons in the research report.

5. How to read importance, ablation, and stability

Feature permutation importance is calculated on the final 63-date inner-validation window in each fold. Assets within a date are permuted together within a feature group, preserving the group's within-row structure; each column gets three repeats. The price-group and all-characteristics group permutations, and `RF/HGB` ablations that remove prices, use prices only, or retain the inner-loop top 10, are part of the frozen design. The top-10 selection uses inner-loop importance. Ablation differences are descriptive relative to the same model's full-feature version. The top-10 ablation reuses full-model parameters, a documented simplification.

Near-zero group permutation importance does not prove the group carries no information: correlated columns can substitute for one another. In the base case, RF and HGB IC both decline when price features are removed; HGB using only prices retains most of its IC. In the reversal case, HGB using only price features has higher IC than the full-feature version, but its t statistic is below 1.3, so treat this as an observation. A group-importance value is not a causal contribution.

Tree importance rankings are weakly stable across adjacent folds: mean rank correlations in the base and reversal scenarios are $-0.10/-0.04$ for a tree, 0.01/0.01 for RF, and 0.09/0.10 for HGB. RF predictions across three seeds correlate 0.962–0.982, with 10 bp Sharpe from 0.44 to 0.64. HGB is identical across its three seeds in this setup because early stopping is disabled and its fit is deterministic. Stability does not imply accuracy, and stable importance rankings do not establish economic value.

6. The evidence is constructed, limited, and unadjusted

Across the six main tree-model comparisons against Ridge (three tree models $\times$ two scenarios), the only absolute t statistic above 2 is the single tree in the reversal scenario: paired $\Delta IC = -0.0207$, unadjusted historical NW t $= -2.19$. The direction is worse for the single tree. The approximate unadjusted p value of 0.029 becomes about 0.17 after a Bonferroni adjustment over the six comparisons; ablations and seed comparisons add further multiplicity. The report preserves these unadjusted figures but does not call them a confirmatory finding.

The frozen configuration's `comparison.claims_rule` says only that the constructed data support no economic or alpha claim and that zero and negative results should be retained. It does not set a joint tree-model threshold of "NW t ≥ 2 for ΔIC in both scenarios and 20 bp Sharpe higher than Ridge and Elastic Net." Do not describe that joint threshold as a preregistered rule. This teaching report can describe the results and limits; it cannot establish real-market efficacy.

7. Notebook replay and the real-data boundary

The I3 reader notebook for P6 v2 has 9 cells, 4 of them code cells. It replays two historical summaries, date bases, and historical ablation and stability displays from fixed fixtures, and generates one new display plot, `historical_constructed_trees_sharpe.png`, from frozen summaries into `RUN_DIR`. The frozen output marks fit as SKIPPED. It does not retrain, generate new model results, or rerun the full CLI. The older "not accepted" banner is historical text; A8-B is currently limited-scope accepted pending formal integration. The BIL19 bilingual candidate remains pending independent review, and complete P6 remains unverified.

Real-data work remains blocked; the `data/PIT` state and A10-B are unverified. Do not describe these constructed mechanisms as empirical findings, return promises, or market alpha. Any future approved real-data project needs a new protocol version, verified data licensing and timing, and an explicit record that all previously seen constructed results are known in advance. None of that work was done in this batch.

8. Local source map

- Frozen P6 v1 configuration: `projects/project06/configs/p06_base_v1.json`; v2 configuration: `projects/project06/configs/p06_trees_v2.json`.
- Implementation: `projects/project06/src/p06_pipeline.py`, `projects/project06/src/p06_trees.py`, `src/quant_course/ml_pipeline.py`, and `src/quant_course/tree_models.py`.
- Result identity and frozen values: `projects/project06/research_report/v2_summary_base.json`, `v2_summary_regime.json`, and `v2_ic_date_bases.json`; original QA summaries are under `qa/a8/ch40_ch41_batch/evidence/qa_r1/`.
- Reports: `projects/project06/research_report/README.md` and `projects/project06/research_report/v2_trees.md`.
- P6 / Chapter 40 boundary review: `qa/bilingual_revision/BIL19/preparation/ROOT_READER_NOTES.md` and `qa/bilingual_revision/BIL19/PRE_FREEZE_REVIEW_01.md`.
- Notebook carry-forward and acceptance status: `qa/bilingual_revision/BIL23/preparation/CH40_CH41_P6V2_NOTEBOOK_CARRY_FORWARD.md`; A8-B receipt: `projects/project06/ACCEPTED_VERSION_v2.md` and `qa/bilingual_revision/BIL18/independent_acceptance/ACCEPTANCE_RECEIPT.json`.

Exercises: preserve the experiment and metric definition

P6-L04. Can the P6 experiment with 46 features, 1,512 dates, and 16 folds be merged into the Chapter 40 interaction experiment with 26 features, 1,260 dates, and 12 folds? Explain the design difference and the boundary around the fivefold post hoc comparison.

P6-L05. What are the six v1 prediction sequences and the four additions in v2? Why does the report say "8 main comparison models" while the frozen run has 10 prediction sequences? Which type of model is PCA + Ridge?

P6-L06. What does independence mean for the q and noise sequences? Are they IID over time? How are the three price columns generated? Why can the complete 46-dimensional feature set not be described as independent, identically distributed, and homoskedastic?

P6-L07. Using R^2 relative to the training mean, IC, and 10/20 bp Sharpe in both scenarios, determine whether PCA + Ridge is uniformly worse than Ridge. Explain why these findings can coexist.

P6-L08. Does the frozen configuration's `claims_rule` specify a joint threshold requiring "tree-model ΔIC NW t ≥ 2 in both scenarios and 20 bp Sharpe above both Ridge and Elastic Net"? How should the only main comparison with an absolute t statistic above 2 be described?

Research practice: dates, permutations, coordinates, and cash

These four new teaching exercises connect reading a report with independently checking a research process. They are not new project runs. Their stable IDs are P6-T01–P6-T04; complete answers are in this project's `solutions.md`, with calculation, proof, and code-diagnosis scaffolds in the Loom notes.

P6-T01 (Label maturity and date groups). Fitting occurs after the close information for D_{10} has arrived. A label starts at the close after its signal date and matures five close-to-close intervals later. Candidate signal dates are D_2, D_3, D_4, D_5, each with 40 assets. Give each date's execution time, label endpoint, and eligibility for this fit. With no other missing values, how many training rows remain? Why does randomly placing half of each day's assets in validation and half in training fail the date-based validation design even if all labels have matured?

P6-T02 (Diagnosing group permutation). There are three assets on each of two signal dates. On the first date, $q = (1, 2, 3)$; on the second, $q = (11, 12, 13)$. Both dates have $z = 2q$. Labels are $y = q$, measured in percentage points, and a fixed predictor is $f(q, z) = q$. The source-row orders for group permutation are $(3, 1, 2)$ and $(2, 1, 3)$, using one-based indices. Give the permuted (q, z) pairs and the MSE increase over all six rows, including units. Diagnose the code below and give direct assertions for date isolation, column multisets, and the group's joint structure. If the fixed predictor never reads a noise column, is its importance exactly zero or merely close to zero?

```
# WRONG: one global order mixes signal dates; two independent orders destroy pairs.
q_perm = q_values[global_order]
z_perm = z_values[another_global_order]
```

P6-T03 (The full-dimensional PCA control). Let X be the design matrix using a fixed training center and scale, let V be orthogonal, and set $Z = XV$. Prove that full-dimensional PCA+Ridge without whitening, using the same $\alpha > 0$, predicts identically to Ridge in the original standardized space. Then use $X^\top X = \mathrm{diag}(2, 8)$, $X^\top y = (2, 4)^\top$, $\alpha = 2$, and $V = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ to compute both coefficient vectors and transform back. What happens if only the first column of V is retained? Does maximum feature variance guarantee better prediction?

P6-T04 (Costs, weight drift, and event order). Fractional shares are allowed; ignore financing, dividends, borrow fees, and impact. Initial cash is 1,000 and both A and B close at 100. Target dollar weights are $+0.5, -0.5$, with dollar targets determined from pre-trade NAV. Charge a one-sided 10 bp fee to cash without resizing the dollar targets. The next closes are 110 and 90. Follow the order "mark positions, determine targets, execute and charge fees." Find cash and NAV after the first trade, NAV and actual weights before the next trade, and the traded notional, fee, and post-trade NAV when rebalancing to the same targets. Explain why a fixed-weight $wr - c|\Delta w|$ proxy is not an event-by-event ledger.

Project 6 complete calculation: target weights and actual weights after fees

Inputs. Use the two assets, 1,000 initial cash, and 10 bp fee from P6-T04. This is a teaching calculation, not a rerun of historical P6. Fractional shares are allowed and fees do not resize target dollar amounts.

Step 1: first execution. Buy five shares of A for 500 and short five shares of B for proceeds of 500. Total traded notional is 1,000, so the fee is $0.001 \times 1000 = 1$. Cash is $1000 - 500 + 500 - 1 = 999$. Net position value is $500 - 500 = 0$, giving NAV of 999.

Step 2: mark the next closes. A is worth 550 and the short B position is worth -450. Cash remains 999, so NAV is $999 + 550 - 450 = 1099$. Actual dollar weights are $550/1099 \approx 0.500455$ and $-450/1099 \approx -0.409463$, with cash weight $999/1099$. Relative price changes have moved the portfolio away from the dollar-neutral target. The return on the original 1,000 is 9.9%; actual positions cannot still be treated as weights $+0.5, -0.5$.

Step 3: rebalance and pay another fee. Pre-trade NAV of 1,099 sets dollar targets of $+549.5, -549.5$. Sell 0.5 dollars of A and short another 99.5 dollars of B: total traded notional is 100 and the fee is 0.1. Cash is $999 + 0.5 + 99.5 - 0.1 = 1098.9$ and net position value is zero, giving post-trade NAV of 1,098.9. Since targets used pre-trade NAV, each risky position's absolute weight after fees is $549.5/1098.9$, slightly above 0.5.

Outputs and conclusion. Fees are 1 and 0.1; the NAV path is 1000→999→1099→1098.9. Prices cause position-value drift, and rebalancing notional depends on the difference between target and existing positions. A simple fixed-weight-change formula omits these events. Historical project results still come from the existing Chapter 29 pipeline; this example explains ledger identities without replacing frozen numbers.

Project 6 v2: Tree-Model Comparison, Importance, Stability, and Ablation

Frozen A8-B results dated 2026-10-11; author revision dated 2026-10-16. Author: Yuxiao Ma. All results come from a teaching construction (`data_class` = CONSTRUCTED), not market evidence. A8-B has its own limited-scope acceptance entry, pending formal integration. The BIL19 bilingual candidate remains pending independent review. Real-data work is blocked, and complete Project 6 remains unverified. The base-pipeline report is README.md; its A8-A frozen content remains separately preserved.

1. Hypotheses and questions

On the v1 frozen constructed panel (a linear data-generating process with three loaded characteristics, 40 zero-loading characteristics, and three price features), this experiment asks: (1) do a single tree, random forest, or gradient boosting improve out-of-sample forecasts or net portfolio results over zero, training-mean, OLS, and regularized linear models; (2) are results stable across folds and seeds; and (3) how much information is associated with price and characteristic feature groups? The pre-run protocol predicted that trees would have no nonlinear structure to exploit under a linear generating process, would not meaningfully outperform Ridge or Elastic Net, and could turn over more because of piecewise-constant predictions.

The generator makes q sequences and noise sequences independent across sequences; each sequence is a stationary AR(1) over time with rho = 0.98 and marginal variance 1. They are not IID over time. The price features `rev_5`, `mom_60_5`, and `vol_20` are derived from a return path with common market, q-signal, and idiosyncratic shocks. Thus, the 43 `q/noise` characteristics and the 3 price-derived features have different structures, and the full 46-dimensional panel is not an independent, identically distributed, homoskedastic population. Interpret PCA using the actual covariance structure and pipeline, not an assumption that all 46 features are IID and equal-variance.

P6 uses 46 features, 1,512 dates, and 16 folds. The Chapter 40 interaction experiment is separate: 26 features, 1,260 dates, and 12 folds, with its registered primary strength and a separate fivefold post hoc sensitivity. Do not merge its parameters, outcomes, or post hoc analysis with P6.

2. Protocol and freeze

- Configuration: `p06_trees_v2.json`, SHA-256 `60101509bbe073678536f0850dd143777b7736be4ec2b6818e05cb736ce23cf4`; frozen before the first v2 run, read-only. It inherits `p06_base_v1.json`, SHA-256 `62f49e1cab11b6992a9c391180df5d6827f9c5c12bf34f94acde5134ee4c39f7`, checked at runtime.
- Seen before the freeze: A8-A v1 constructed outcomes in both the base and reversal scenarios (R^2, IC, and net Sharpe, recorded in the config), Chapter 38–39 constructed runs, and one timing-only probe. This was not a real-market preregistration.
- The generator and the base and `regime_flip_at_75pct` scenarios were unchanged; no scenario favorable to tree models was added.

- Timing: `signal_time` is the close on D_k; `trade_time` is the close on D_{k+1}; `label_end` and `label_available_at` are the close on D_{k+6}. A training row must satisfy `label_available_at` $\leq$`fit_time.` Outer expanding-window validation has a minimum 504 training dates, 63 dates per test block, and 16 folds. Inner selection uses the final two 63-date windows and at least 252 training dates.
- Models and grids are defined in the frozen configuration. Gradient boosting is HistGradientBoostingRegressor with `early_stopping`=False; its iteration count (maximum 300) is selected with other parameters by average inner date-fold MSE, then refitted on all outer-training rows.
- Portfolio: demean forecasts cross-sectionally, gross exposure 1, rebalance every five signal dates, execute at the next close, and use one-way fees of 0/10/20 bp in the Chapter 29 ledger; initial capital is 1 million. Borrowing, financing, market impact, and capacity are not modeled.
- Implementation: `p06_trees.py`; CLI `run_project06_v2.py`, whose real-data mode historically exits with code 3 without writing files; core logic is in `quant_course.tree_models` and the read-only reused `quant_course.ml_pipeline.`
- Baseline check: v1's six-model sequence was rerun with unmodified v1 code and matched the A8-A c3 summary value by value: 209 values per scenario and maximum relative difference 0.0.

The frozen v1 implementation generates six prediction sequences: zero, `train_mean`, OLS, Ridge, Lasso, and Elastic Net. Version 2 adds four: a single tree, RF, HGB, and PCA + Ridge, for 10 total sequences. The main comparison table and figures show eight fitted models and omit the two constant-forecast baselines. PCA + Ridge is not a tree.

3. Results

Numbers are from the author's historical QA run `qa_r1` (persistent evidence under `qa/a8/ch40_ch41_batch/evidence/qa_r1/`), with summary copies `v2_summary_base.json` and `v2_summary_regime.json`. The tables' R^2_{OS} is relative to the training mean. IC is the mean daily Spearman IC over a model's own defined dates. Parenthesized t values for continuous series are historical NW statistics with a four-trading-day bandwidth; ΔIC is the mean daily paired difference against Ridge over common dates.

The main tables have eight rows, not eight total prediction sequences. The zero and `train_mean` baselines are also present in the frozen run. Both produce a constant forecast and a flat portfolio: total return and turnover are zero; IC is undefined (NaN).

Date counts (added in R01 on 2026-10-11). There are 1,002 test dates with mature labels in each scenario. Own-date IC means and paired ΔIC means use different sets when a model has undefined IC dates. OLS, Ridge, PCA + Ridge, and the three tree models have defined IC on all 1,002 continuous dates. Lasso and Elastic Net sometimes produce constant predictions, making their cross-sectional IC undefined: 630 and 882 valid dates in the base scenario, and 567 and 819 in the reversal scenario. A sparse model must be compared with Ridge on the same common dates. For base Lasso, its mean on 630 dates is 0.0179 and Ridge's mean on those dates is 0.0161, difference +0.0017. For base Elastic Net on 882 dates, the means are 0.0309

and 0.0206, difference +0.0103. In reversal, the 567-date Lasso means are 0.0010 and 0.0031, difference −0.0021; the 819-date Elastic Net means are 0.0152 and 0.0091, difference +0.0061. See `v2_ic_date_bases.json`, independently recomputed from frozen prediction and label CSVs without changing the frozen summaries.

The historical summary retains sparse-series t diagnostics computed after dropping missing dates and treating the remaining observations as adjacent. A lag of four compressed observations is not a four-trading-day HAC. Those t statistics are not reported in the tables. The displayed dashes are intentional; no new HAC was calculated. NaN remains undefined, not zero.

3.1 Base scenario (1,002 test dates)

Model	R^2_{OS} vs training mean	IC (t)	ΔIC (t)	Sharpe 0 bp
OLS	−0.839%	0.0203 (2.24)	+0.0004 (0.18)	0.79
Ridge	−0.113%	0.0198 (2.22)	—	0.84
Lasso	−0.083%	0.0179 (—)[a]	+0.0017 (—)[a]	0.36
Elastic Net	−0.010%	0.0309 (—)[a]	+0.0103 (—)[a]	1.19
Single tree	−0.393%	0.0054 (0.64)	−0.0144 (−1.61)	0.21
Random forest	+0.037%	0.0266 (2.86)	+0.0067 (1.06)	1.13
HGB	+0.023%	0.0154 (1.79)	−0.0044 (−0.61)	0.76
PCA + Ridge	−0.008%	0.0135 (1.45)	−0.0063 (−0.86)	0.62

Model	10 bp	20 bp	Annual turnover	Max drawdown 10 bp
OLS	0.39	0.00	24.3	−14.2%
Ridge	0.45	0.07	23.6	−12.9%
Lasso	0.01	−0.33	16.3	−9.3%
Elastic Net	0.81	0.43	22.0	−7.2%
Single tree	−0.28	−0.75	35.1	−20.0%
Random forest	0.64	0.16	29.2	−12.3%
HGB	0.22	−0.32	36.2	−11.0%
PCA + Ridge	0.25	−0.13	23.3	−12.0%

Model	Total return 10 bp
OLS	+9.39%
Ridge	+10.90%
Lasso	−0.20%
Elastic Net	+19.90%
Single tree	−8.70%
Random forest	+15.98%
HGB	+5.02%
PCA + Ridge	+5.49%

(a) The summary has compressed-valid-observation t diagnostics, but the tables do not report them as trading-day inference.

The 16 test blocks contain $16 \times 63 = 1,008$ nominal signal dates (about four years). Labels for the final six dates extend beyond the sample, leaving 1,002 dates in prediction metrics.

3.2 Loading-reversal scenario (reversal begins on trading date 1,134; folds 11–16 fall after the reversal)

Model	R^2_{OS} vs training mean	IC (t)	ΔIC (t)	Sharpe 0 bp
OLS	−1.013%	0.0089 (0.97)	+0.0002 (0.10)	0.17
Ridge	−0.161%	0.0087 (0.95)	—	0.23
Lasso	−0.143%	0.0010 (—)[a]	−0.0021 (—)[a]	−0.25
Elastic Net	−0.061%	0.0152 (—)[a]	+0.0061 (—)[a]	0.45
Single tree	−0.604%	−0.0120 (−1.32)	−0.0207 (−2.19)	−0.66
Random forest	−0.042%	0.0129 (1.35)	+0.0042 (0.66)	0.41
HGB	−0.020%	0.0041 (0.46)	−0.0045 (−0.59)	0.16
PCA + Ridge	−0.053%	0.0063 (0.68)	−0.0023 (−0.31)	0.24

Model	10 bp	20 bp	Annual turnover	Max drawdown 10 bp
OLS	−0.22	−0.61	24.3	−21.7%
Ridge	−0.15	−0.53	23.7	−19.9%
Lasso	−0.57	−0.89	14.7	−15.1%
Elastic Net	0.08	−0.29	20.2	−15.7%
Single tree	−1.13	−1.60	36.2	−30.1%
Random forest	−0.08	−0.57	29.7	−21.0%
HGB	−0.38	−0.92	36.5	−20.0%
PCA + Ridge	−0.14	−0.52	23.5	−12.0%

Ridge's fold ICs after reversal (folds 11–16) are 0.001, −0.086, −0.052, 0.007, 0.019, and −0.032. Most are negative: all models learned the pre-reversal signal sign and needed later expanding-window data to adjust.

To retain R^2 against zero as well as the training-mean tables above, the historical R^2-vs-zero and 10/20 bp net-return values are listed below. They come from the same `qa_rl` summaries and were not recomputed.

Scenario	Model	R^2 vs zero	Net return 10 bp	Net return 20 bp
Base	OLS	−1.715%	+9.39%	−0.76%
Base	Ridge	−0.982%	+10.90%	+0.91%
Base	Lasso	−0.952%	−0.20%	−6.50%
Base	Elastic Net	−0.879%	+19.90%	+9.81%
Base	Single tree	−1.264%	−8.70%	−20.66%
Base	Random forest	−0.831%	+15.98%	+3.18%
Base	HGB	−0.845%	+5.02%	−9.14%
Base	PCA + Ridge	−0.876%	+5.49%	−3.91%
Reversal	OLS	−1.891%	−6.09%	−14.80%
Reversal	Ridge	−1.032%	−4.44%	−13.08%
Reversal	Lasso	−1.014%	−10.10%	−15.23%
Reversal	Elastic Net	−0.931%	+1.09%	−6.77%
Reversal	Single tree	−1.479%	−29.85%	−39.30%
Reversal	Random forest	−0.913%	−2.72%	−13.62%
Reversal	HGB	−0.890%	−10.55%	−22.71%
Reversal	PCA + Ridge	−0.923%	−4.24%	−12.84%

3.3 Ablation (ΔIC relative to the same model's full-feature version; Sharpe is 10/20 bp)

Ablation	Base IC	Base ΔIC (t)	Base Sharpe	Reversal IC
RF: no price	0.0207	−0.0059 (−2.05)	0.27 / −0.20	0.0082
RF: price only	0.0131	−0.0135 (−1.09)	−0.05 / −0.81	0.0141
RF: inner top 10	0.0146	−0.0120 (−1.85)	0.28 / −0.23	0.0082
HGB: no price	0.0080	−0.0074 (−1.56)	−0.22 / −0.76	−0.0013
HGB: price only	0.0145	−0.0009 (−0.08)	0.44 / −0.15	0.0130
HGB: inner top 10	0.0197	+0.0043 (0.58)	0.14 / −0.41	0.0141

Ablation	Reversal ΔIC (t)	Reversal Sharpe
RF: no price	−0.0047 (−1.62)	−0.34 / −0.81
RF: price only	+0.0013 (0.10)	−0.14 / −0.90
RF: inner top 10	−0.0047 (−0.77)	−0.06 / −0.56
HGB: no price	−0.0055 (−1.14)	−0.42 / −0.94
HGB: price only	+0.0089 (0.77)	0.19 / −0.40
HGB: inner top 10	+0.0099 (1.22)	−0.22 / −0.76

Grouped permutation importance (mean MSE increase $\times 10^{-4}$ over 16 folds, measured on the final inner validation window): in the base case, price group −0.0014 (HGB), −0.0006 (RF), and 0.0012 (tree); all-characteristics group 0.0495, 0.052, and 0.0585. In the reversal case, price group −0.0016 (HGB), −0.0009 (RF), and 0.0012 (tree); all-characteristics group 0.0477, 0.0373, and 0.0462.

3.4 Stability

- **Seeds (base scenario):** RF seeds 4061/4062/4063 yield ICs of 0.0266/0.0232/0.0237, Sharpe after 10 bp of 0.64/0.52/0.44, and prediction correlations of 0.962–0.982. HGB produces identical results for all three seeds because fitting is deterministic for this data with early stopping disabled.
- **Selected parameters:** a depth-2 tree in 14 of 16 folds; RF leaf size 2,000 in 10 folds and 500 in 6; HGB iteration counts from 2–96, with learning rate 0.02 selected in only 2 folds; PCA + Ridge selected 30 components in 10 folds, 5 in 5 folds, and 15 in 1.
- **Inner-loop importance ranking stability:** mean adjacent-fold rank correlations (the base and reversal scenarios) are tree −0.10/−0.04, RF 0.01/0.01, and HGB 0.09/0.10. Base-case top-10 appearance frequency: RF q1 in 11 folds, `noise_04` in 11, `noise_18` in 10, and q2 in 9; HGB q1 in 11, q2 in 10, `noise_09` in 10.
- **Post hoc diagnostic (not used for model selection):** outer-test-window permutation importance ranks q1 (0.0251 $\times 10^{-4}$), q2 (0.0069), and `noise_09` (0.0033) highest for HGB; RF ranks q1, `noise_04`, and q2 highest. RF mean final-model MDI by fold is q1 0.153, q2 0.112, `noise_04` 0.083; many zero-loading rows are around 0.03.

3.5 Figures

- Base model comparison and reversal model comparison: R^2_{OS}, IC, and 10/20 bp Sharpe for the eight fitted models.
- Base importance stability and reversal importance stability: fold heatmaps of HGB inner-loop permutation importance.
- Base ablation and reversal ablation: registered ablations.

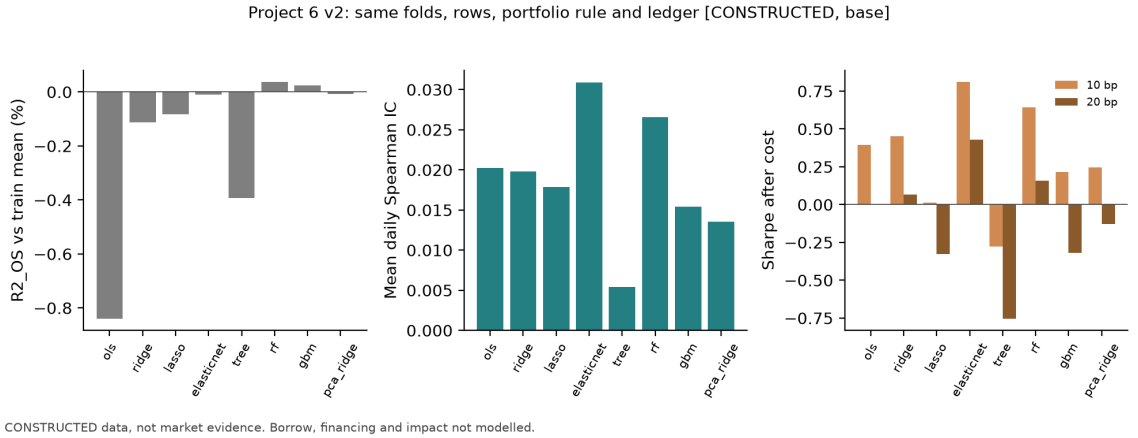


Figure 1: Base-scenario comparison of eight models: R-squared relative to the training mean, daily IC, and Sharpe at 10/20 bp. Zero and training-mean forecasts are separate flat baselines. Constructed data.

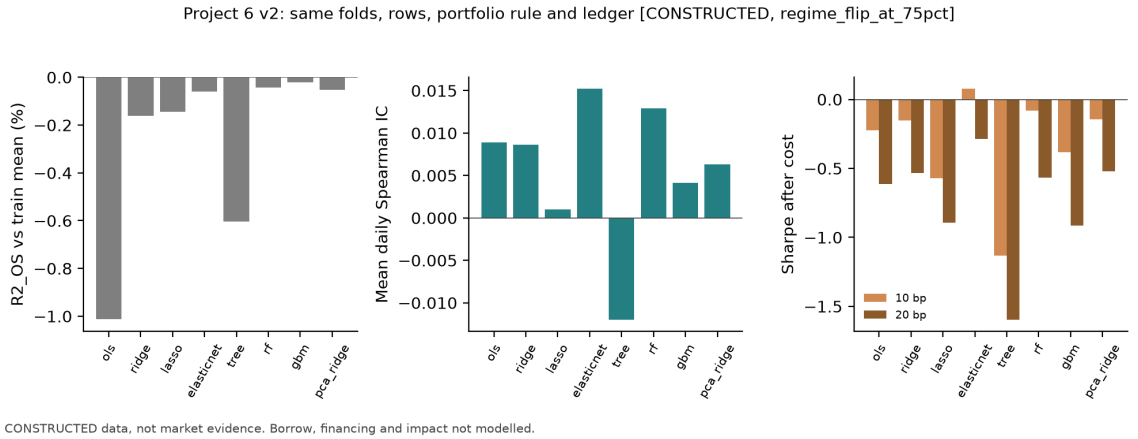


Figure 2: Loading-reversal comparison using the same metrics and axes; negative after-cost values remain visible. Constructed data.

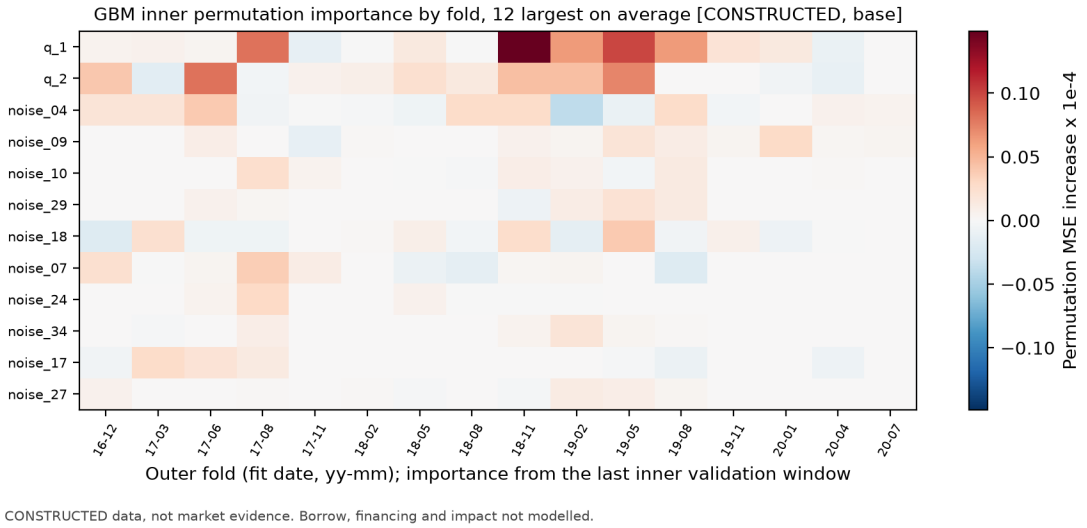


Figure 3: Base HGB inner-validation permutation-importance heatmap. Columns are the 16 outer-fit windows, rows the 12 displayed features, and color the MSE increase under within-date permutation. This is not a causal effect.

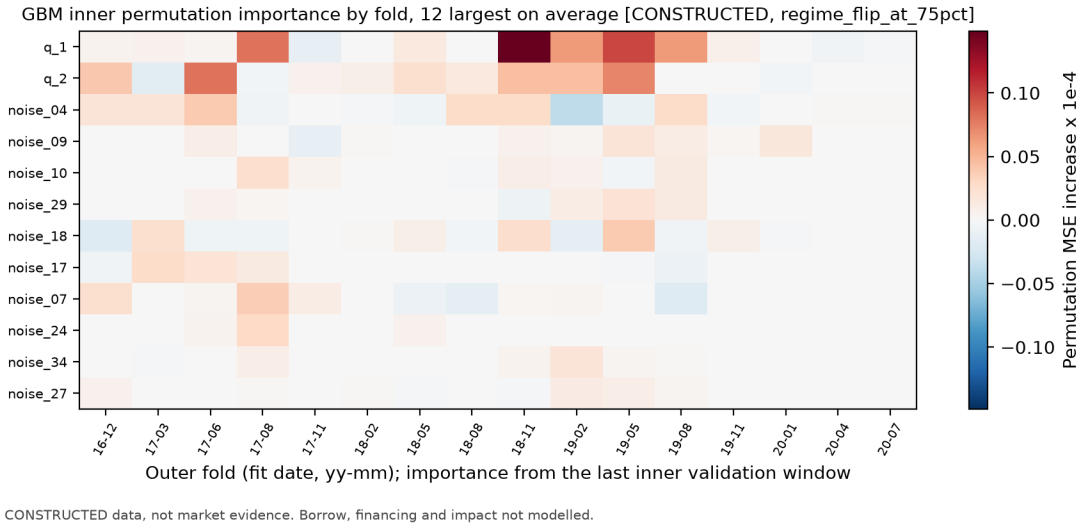


Figure 4: Reversal importance heatmap with the same axes, repeats, and loss convention as the base plot. Changes across windows do not establish a stable discovery.

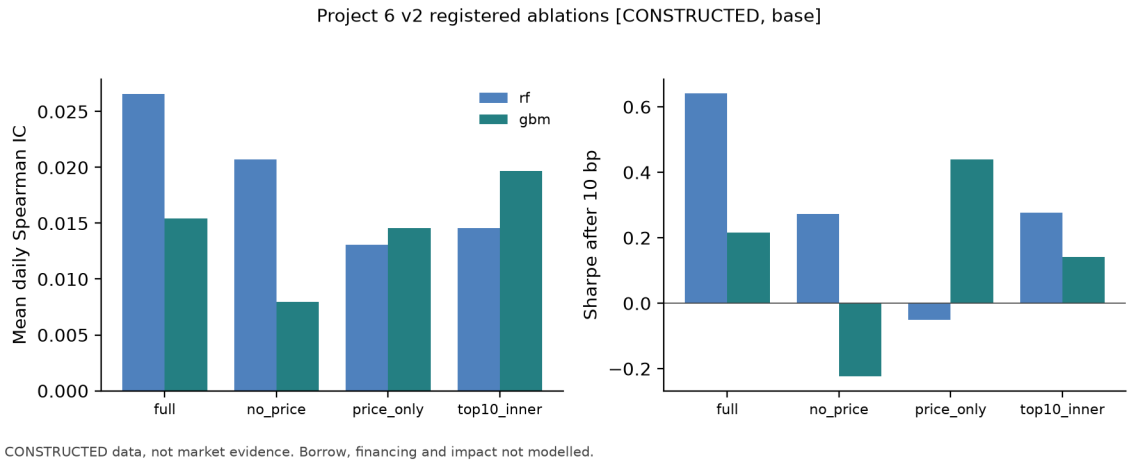


Figure 5: Base ablations: IC and after-cost outcomes when RF/HGB remove price features, use prices only, or keep the inner-selected top 10. Compare with the same model using all features.

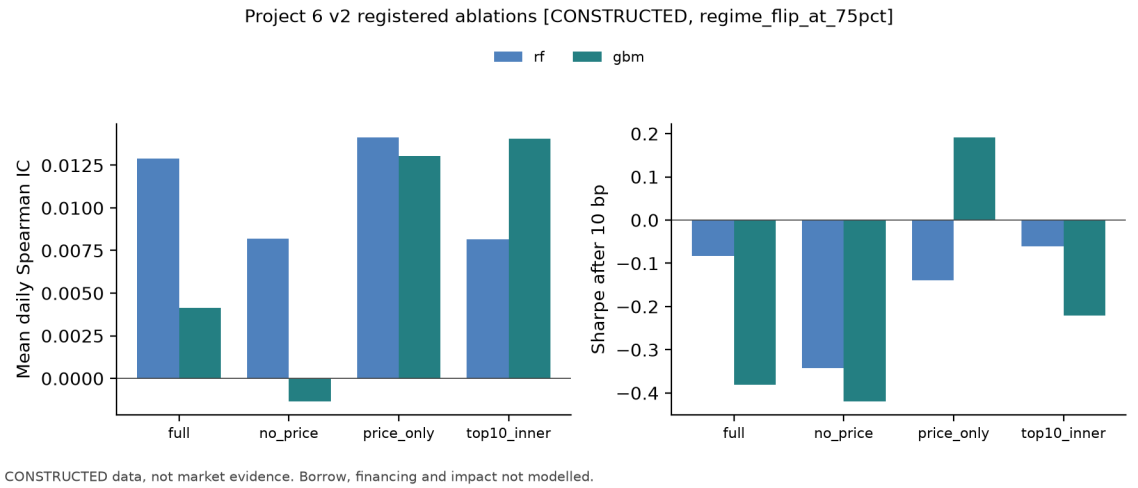


Figure 6: Reversal ablations retain all negative outcomes under the same model, fold, and cost definitions as the base scenario.

4. Interpretation

1. No model has R^2_{OS} above 0.04% relative to the training mean. Except for the reversal-scenario single tree at $t = -2.19$, all other main comparisons have $|t| < 2$; the single tree is worse ($\Delta IC = -0.0207$, unadjusted two-sided approximate $p \approx 0.029$; approximately 0.17 after Bonferroni correction only over three tree models × two scenarios). Ablations and seed comparisons were also registered, so this is an unadjusted descriptive comparison, not a confirmatory significant finding. Consistent with the prediction, trees found no additional structure to exploit under this linear data-generating process.
2. **Turnover:** annual tree turnover is 29.2–36.5 versus 14.7–24.3 for linear models. RF's base-scenario Sharpe is 1.13 before costs and 0.16 after 20 bp; Elastic Net has the highest Sharpe in both scenarios at all three cost levels.
3. **Overlap between price and characteristic information:** past returns contain slowly varying expected returns (feature AR coefficient 0.98), so price features carry indirect signal. When characteristics are present, grouped price permutation importance is close to zero (substitution); removing prices lowers IC for both tree models, while HGB with prices only retains nearly all its IC. In the reversal scenario, HGB with prices only has higher IC than its full-feature version, possibly because momentum features update with recent returns faster than pre-reversal feature coefficients; all related t values are below 1.3, so this is only an observation.
4. By IC, training-only top-10 selection lowers RF in both scenarios and raises HGB in both, as descriptive comparisons. After-cost Sharpe does not move consistently: HGB at 10 bp falls from 0.22 to 0.14 in base and rises from −0.38 to −0.22 in reversal. Permutation importance is noisy in 63-date validation windows, and zero-loading features often appear in the selected set.
5. PCA + Ridge does not rank uniformly below Ridge. Relative to the training mean, R^2 is −0.008%/−0.053% in the base and reversal scenarios, slightly better than Ridge's −0.113%/−0.161%. In reversal, 10/20 bp Sharpe of −0.14/−0.52 is also slightly higher than Ridge's −0.15/−0.53. By contrast, mean IC of 0.0135/0.0063 is below Ridge's 0.0198/0.0087. With all 46 components, the same standardization and alpha, and no whitening, PCA is an orthogonal rotation that preserves the Ridge norm, so it is equivalent to Ridge in the rotated coordinates. Truncation changes the available subspace and is not guaranteed to improve results. Interpret this alongside the AR(1) `q/noise` and derived price features; do not assume all 46 columns are independent and equal-variance.

5. Falsification conditions and conclusion

The frozen configuration's `comparison.claims_rule` only says to make no economic or alpha claim from constructed data and to retain zero and negative results. It does not specify a joint threshold requiring tree-model ΔIC NW t values of at least 2 in both scenarios and 20 bp Sharpe above both Ridge and Elastic Net. Do not attribute that threshold to the frozen protocol.

Across the six main comparisons (three tree models × two scenarios), the only absolute t statistic above 2 is the reversal single tree relative to Ridge: paired $\Delta IC = -0.0207$, historical unadjusted NW t = −2.19, in the direction of worse performance for the tree. Its approximate

unadjusted two-sided p value is 0.029; a Bonferroni adjustment over only these six comparisons gives about 0.17. Ablations and seed comparisons were also registered, so this remains an unadjusted description, not a confirmatory finding. All other main comparisons have absolute t values below 2.

The tree models followed the specified training-only pipeline, but showed no distinguishable overall improvement in this construction. This conclusion describes only the current generator, fold design, and cost ledger; it does not generalize to real markets.

6. Limitations

- Constructed data: the data-generating process is linear. Conclusions concern only this pipeline and construction, not real markets.
- Multiple comparisons: eight main-table models × two scenarios × three cost levels, plus six ablation variants in each scenario and three-seed stability comparisons were not multiplicity-adjusted; no result is described here as a significant finding.
- Costs: proportional fees only; borrow costs, financing, impact, and capacity are omitted. Short proceeds are fully credited to cash under the Chapter 29 convention.
- Importance: computed on 63-date inner windows with three repeats, so variance is high; grouped permutation has only two groups.
- Computation: `qa_r1` end-to-end time (the full `p06_trees.run`, including writes and plots, not fit time alone) was 1,100 seconds for base and 906 for reversal. Development run dev01 took 1,981 and 1,382 seconds under parallel load, with identical values. RF uses `n_jobs`=-1; floating-point summation order may vary by thread (post hoc refit matched original predictions within the default tolerance, asserted by `posthoc_test_importance`).

7. I3 notebook display replay and current scope

The I3 reader notebook for P6 v2 has 9 cells, 4 of them code cells. It replays two frozen summaries, the IC date bases, and historical ablation and stability displays from fixed fixtures, and generates `historical_constructed_trees_sharpe.png` from the frozen summaries into `RUN_DIR.` Fit is explicitly marked SKIPPED; there was no retraining, full CLI, or new model test. The old "not accepted" banner is historical text; A8-B is currently limited-scope accepted pending formal integration. The BIL19 bilingual candidate remains pending independent review, and complete P6 remains unverified. This author revision did not run the notebook, models, generator, or full CLI.

The Chapter 40 experiment's 26 features, 1,260 dates, 12 folds, registered primary strength, and separate fivefold post hoc sensitivity remain separate from P6's 46 features, 1,512 dates, and 16 folds. Data and point-in-time status and A10-B remain unverified; real research remains blocked.

8. Real-data prerequisites

See `data_manifest.json`: a licensed, retainable daily equity panel that can support internal training-derived statistics is required, with at least eight years. Create a new configuration version when it becomes available, rerun from the data node, and record all constructed results in this report and Chapters 37–41 as already seen.

Project 6 Questions and Complete Answers

Author: Yuxiao Ma · **Revision date:** 2026-10-16.

This file contains the stable Project 6 IDs P6-L01–P6-L08 and P6-T01–P6-T04. P6-L01–P6-L03 preserve the original questions in the base report; P6-L04–P6-L08 cover the BIL19 v2 tree and PCA extension. The questions also appear in the v2 teaching guide and the Loom study notes. Each answer stays in the same project as its prompt.

Project questions: work independently first

P6-L01. Can the base Lasso IC of 0.01785759 be compared directly with Ridge's 0.01982995 to conclude that Lasso is weaker?

P6-L02. Does compressed NW(4) represent a four-trading-day HAC?

P6-L03. Can Chapter 39 reuse these P6 paired results?

P6-L04. Can the P6 experiment with 46 features, 1,512 dates, and 16 folds be merged into the Chapter 40 interaction experiment with 26 features, 1,260 dates, and 12 folds? Explain the design difference and the boundary around the fivefold post hoc comparison.

P6-L05. What are the six v1 prediction sequences and the four additions in v2? Why does the report say "8 main comparison models" while the frozen run has 10 prediction sequences? Which type of model is PCA + Ridge?

P6-L06. What does independence mean for the q and noise sequences? Are they IID over time? How are the three price columns generated? Why can the complete 46-dimensional feature set not be described as independent, identically distributed, and homoskedastic?

P6-L07. Using R^2 relative to the training mean, IC, and 10/20 bp Sharpe in both scenarios, determine whether PCA + Ridge is uniformly worse than Ridge. Explain why these findings can coexist.

P6-L08. Does the frozen configuration's `claims_rule` specify a joint threshold requiring "tree-model ΔIC NW t ≥ 2 in both scenarios and 20 bp Sharpe above both Ridge and Elastic Net"? How should the only main comparison with an absolute t statistic above 2 be described?

P6-T01 (Label maturity and date groups). Fitting occurs after the close information for D_{10} has arrived. A label starts at the close after its signal date and matures five close-to-close intervals later. Candidate signal dates are D_2, D_3, D_4, D_5, each with 40 assets. Give each date's execution time, label endpoint, and eligibility for this fit. With no other missing values, how many training rows remain? Why does randomly placing half of each day's assets in validation and half in training fail the date-based validation design even if all labels have matured?

P6-T02 (Diagnosing group permutation). There are three assets on each of two signal dates. On the first date, $q = (1, 2, 3)$; on the second, $q = (11, 12, 13)$. Both dates have $z = 2q$. Labels are $y = q$, measured in percentage points, and a fixed predictor is $f(q, z) = q$. The source-row orders for group permutation are $(3, 1, 2)$ and $(2, 1, 3)$, using one-based indices. Give the permuted (q, z) pairs and the MSE increase over all six rows, including units. Diagnose the code below and give direct assertions for date isolation, column multisets, and the group's joint structure. If the fixed predictor never reads a noise column, is its importance exactly zero or merely close to zero?

```
# WRONG: one global order mixes signal dates; two independent orders destroy pairs.
q_perm = q_values[global_order]
z_perm = z_values[another_global_order]
```

P6-T03 (The full-dimensional PCA control). Let X be the design matrix using a fixed training center and scale, let V be orthogonal, and set $Z = XV$. Prove that full-dimensional PCA+Ridge without whitening, using the same $\alpha > 0$, predicts identically to Ridge in the original standardized space. Then use $X^\top X = \mathrm{diag}(2, 8)$, $X^\top y = (2, 4)^\top$, $\alpha = 2$, and $V = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ to compute both coefficient vectors and transform back. What happens if only the first column of V is retained? Does maximum feature variance guarantee better prediction?

P6-T04 (Costs, weight drift, and event order). Fractional shares are allowed; ignore financing, dividends, borrow fees, and impact. Initial cash is 1,000 and both A and B close at 100. Target dollar weights are $+0.5, -0.5$, with dollar targets determined from pre-trade NAV. Charge a one-sided 10 bp fee to cash without resizing the dollar targets. The next closes are 110 and 90. Follow the order "mark positions, determine targets, execute and charge fees." Find cash and NAV after the first trade, NAV and actual weights before the next trade, and the traded notional, fee, and post-trade NAV when rebalancing to the same targets. Explain why a fixed-weight $wr - c|\Delta w|$ proxy is not an event-by-event ledger.

P6-L01: Own-date IC and common-date pairing

Question. Can the base Lasso IC of 0.01785759 be compared directly with Ridge's 0.01982995 to conclude that Lasso is weaker?

Complete answer. No. The Lasso value is its mean over 630 dates on which its IC is defined; Ridge's 0.01982995 is its mean over all 1,002 dates. The sets differ, so their direct difference is not a paired comparison. On the common 630 dates, Lasso's mean is 0.01785759, Ridge's mean is 0.01613330, and the paired mean difference is +0.00172429. This is a descriptive common-date statistic; the display provides no valid inferential t statistic for it and supports no superiority claim. An IC for a constant forecast is NaN (undefined), not zero.

P6-L02: The time span in compressed NW(4)

Question. Does compressed NW(4) represent a four-trading-day HAC?

Complete answer. No. Suppose the valid trading-date indices are 0, 2, 3, 5, and 8. After missing dates are removed, a fourth-order lag over the compressed observations connects the first and fifth valid observations: date 0 and date 8, eight trading days apart. It is not a four-day lag on the trading calendar. The report retains the old compressed values only as historical diagnostics, not as four-trading-day HAC or valid current inference; no new HAC was calculated.

P6-L03: P6 cannot replace Chapter 39's paired data

Question. Can Chapter 39 reuse these P6 paired results?

Complete answer. No. P6 and Chapter 39 are different experiments. The frozen row-level predictions needed to reconstruct Chapter 39's historical common-date comparison are unavailable. P6 rows belong to a different configuration and run identity and cannot fill that gap. Retraining also cannot create historical realized predictions. Preserve the fact that Chapter 39's paired quantity could not be recomputed rather than substituting another project's numbers.

P6-L04: P6 and the Chapter 40 interaction experiment

Question. Can the P6 experiment with 46 features, 1,512 dates, and 16 folds be merged into the Chapter 40 interaction experiment with 26 features, 1,260 dates, and 12 folds? Explain the design difference and the boundary around the fivefold post hoc comparison.

Complete answer. No. P6's linear construction uses 46 features, 1,512 trading dates, and 16 outer folds. Chapter 40 uses a different constructed design with 26 features, 1,260 dates, and 12 folds. The generating process, sample, and split are different; Chapter 40's interaction structure is not a P6 feature or result. Chapter 40 also has a registered primary strength and a separate fivefold post hoc sensitivity; both belong to that chapter's experiment, not to P6's results, protocol thresholds, or inference. The methods may be taught side by side only if each experiment identity remains explicit.

P6-L05: Prediction sequences versus models in the main table

Question. What are the six v1 prediction sequences and the four additions in v2? Why does the report say "8 main comparison models" while the frozen run has 10 prediction sequences? Which type of model is PCA + Ridge?

Complete answer. Version 1 produces zero, `train_mean`, OLS, Ridge, Lasso, and Elastic Net. Version 2 adds a single tree, random forest, gradient boosting (the fitted estimator is HistGradientBoostingRegressor, or HGB), and PCA + Ridge, for 10 prediction sequences in the frozen run. The historical main table and figures show eight fitted models: OLS, Ridge, Lasso, Elastic Net, single tree, RF, HGB, and PCA + Ridge. They omit zero and `train_mean.` Both omitted baselines make constant forecasts and hold a flat portfolio, with total return and turnover of 0. PCA + Ridge is a dimension-reduced linear pipeline that standardizes and applies PCA before Ridge; it is not a tree.

P6-L06: Sequence independence, temporal dependence, and price features

Question. What does independence mean for the q and noise sequences? Are they IID over time? How are the three price columns generated? Why can the complete 46-dimensional feature set not be described as independent, identically distributed, and homoskedastic?

Complete answer. Different constructed q sequences are independent of one another, as are different noise sequences; the construction specifies independent q and noise innovations. Each q or noise sequence is still a stationary AR(1) over time, with coefficient 0.98 and marginal variance 1. Adjacent values are therefore dependent, so the sequences are not IID over time. Returns also include a common market shock, a signal term from q, and idiosyncratic shocks. The price features `rev_5`, `mom_60_5`, and `vol_20` are derived from this `return/price` path and therefore have a construction relationship to feature processes and past returns. The 46 columns combine 43 AR characteristic sequences and three derived price features; they are not draws from one independent, identically distributed, equal-variance population. This distinction matters when interpreting PCA, importance, and the word "noise."

P6-L07: Multiple metrics for PCA + Ridge

Question. Using R^2 relative to the training mean, IC, and 10/20 bp Sharpe in both scenarios, determine whether PCA + Ridge is uniformly worse than Ridge. Explain why these findings can coexist.

Complete answer. It is not uniformly worse. In the base scenario, PCA + Ridge has R^2 relative to the training mean of -0.008%, slightly better than Ridge at -0.113%. In the reversal scenario the values are -0.053% and -0.161%, respectively, again slightly better for PCA + Ridge. Its mean IC is lower: 0.0135/0.0063 across the base and reversal scenarios, versus Ridge's 0.0198/0.0087. Under reversal, the 10/20 bp Sharpes are $-0.14/-0.52$ for PCA + Ridge and $-0.15/-0.53$ for Ridge; both are negative, but PCA + Ridge is slightly higher numerically. R^2 measures asset-level forecast error, IC measures cross-sectional ranking, and Sharpe measures the

costed ledger under the specified portfolio rule. They measure different objects, so their rankings can differ. State the result by metric rather than declaring PCA universally better or worse.

P6-L08: The `claims_rule` and the only larger tree-model t statistic

Question. Does the frozen configuration's `claims_rule` specify a joint threshold requiring "tree-model ΔIC NW t ≥ 2 in both scenarios and 20 bp Sharpe above both Ridge and Elastic Net"? How should the only main comparison with an absolute t statistic above 2 be described?

Complete answer. No. The v2 configuration's `claims_rule` says not to make economic or alpha claims from constructed data and to retain zero and negative results. It does not specify that joint threshold, so do not attribute it to the frozen protocol. Among the six main tree-model comparisons (three tree types $\times$ two scenarios), the only absolute t statistic above 2 is the reversal-scenario single tree against Ridge: $\Delta IC = -0.0207$, historical unadjusted NW t $= -2.19$, in the direction of worse performance for the tree. The approximate unadjusted two-sided p value is 0.029; a Bonferroni approximation over only these six comparisons is about 0.17. The protocol also includes ablation and seed comparisons, so this is not a confirmatory significant finding. Describe it as an unadjusted, descriptive negative result within constructed data.

P6-T01: Complete answer

A signal on D_k executes at the close of D_{k+1} and its label matures at the close of D_{k+6}. The four candidates are $D_2 \to D_3 \to D_8$, $D_3 \to D_4 \to D_9$, $D_4 \to D_5 \to D_{10}$, and $D_5 \to D_6 \to D_{11}$. Because fitting occurs after the close information for D_{10} is available, the first three satisfy `label_available_at <= fit_time`; the last does not. With no other missing values, keep $3 \times 40 = 120$ rows, not 160. Equality at the cutoff is valid only after the relevant closing information has actually arrived. Randomly splitting assets within a date may satisfy maturity yet share the same daily shock and change the question from generalization to later dates. Keep each date intact and recheck maturity at every inner fitting time.

P6-T02: Complete answer and assertions

The first date becomes $(3, 6), (1, 2), (2, 4)$ and the second becomes $(12, 24), (11, 22), (13, 26)$. Labels retain their original order and the predictor still reads only q. Errors are $(2, -1, -1, 1, -1, 0)$, giving SSE of 8 and MSE of $8/6 = 4/3$ squared percentage points. Baseline MSE is zero, so the increase is also $4/3$. One shared row order preserves $z = 2q$ within each date; a global cross-date order destroys the date context.

These standard-library assertions check dates and joint structure directly, without training or calling the course implementation:

```
from collections import Counter
original = {"D1": [(1, 2), (2, 4), (3, 6)],
            "D2": [(11, 22), (12, 24), (13, 26)]}
orders = {"D1": [2, 0, 1], "D2": [1, 0, 2]}  # zero-based
permuted = {d: [rows[i] for i in orders[d]] for d, rows in original.items()}
for d in original:
    assert Counter(permuted[d]) == Counter(original[d])  # same-date pairs only
    for j in (0, 1):
        assert Counter(v[j] for v in permuted[d]) == Counter(v[j] for v in original[d
    ↪ ])
    assert all(z == 2*q for q, z in permuted[d])
```

If a fixed predictor never reads a column, permuting it changes no prediction or loss, so importance is exactly zero. A model fitted on a finite sample may happen to use a truly zero-loading column; its estimated importance need not be zero, so examine repeats and windows. Group permutation preserves dependence within the group, but not necessarily its relationship with unpermuted columns, so support remains a limitation.

P6-T03: Complete proof and coefficients

Set $\beta = Vb$. Full-dimensional orthogonality gives $VV^\top = V^\top V = I$, hence $X\beta = XVb = Zb$ and $\|\beta\|^2 = \|b\|^2$. The objectives $\|y - X\beta\|^2 + \alpha\|\beta\|^2$ and $\|y - Zb\|^2 + \alpha\|b\|^2$ are identical under a one-to-one change of coordinates. Prediction comparison also requires the same training center, scale, input rows, intercept convention, and alpha. Whitening rescales the directions, so the same-alpha penalty no longer has the same meaning.

Numerically, $\hat{\beta} = (X^\top X + 2I)^{-1} X^\top y = (2/4, 4/10)^\top = (0.5, 0.4)^\top$. Then $\hat{b} = V^\top \hat{\beta} = (0.4, -0.5)^\top$ and $V\hat{b} = (0.5, 0.4)^\top$. Retaining only the first column of V restricts coefficients to $\beta = (0, b_1)^\top$, giving $(0, 0.4)^\top$ here. The first original direction's predictive contribution is removed. Truncation may reduce estimation noise or discard true signal; ranking by feature variance alone cannot decide whether test prediction improves. The frozen P6 grid selected only 5/15/30 components; this full-dimensional control is a teaching derivation, not a new P6 experiment.

P6-T04: Complete ledger answer

The first trade buys five A shares and shorts five B shares. Total traded notional is 1,000, the fee is 1, cash is 999, net position value is zero, and NAV is 999. At the next closes, positions are worth 550 and -450, giving NAV of 1,099. Actual risky weights are approximately 0.500455 and -0.409463; net dollar exposure is 100/1099, so the positions no longer match the original target. Reset dollar targets from pre-trade NAV to $+549.5, -549.5$: sell 0.5 dollars of A and short another 99.5 dollars of B. Total traded notional is 100 and the fee is 0.1. Cash and NAV are 1,098.9, with zero net risky position value. After the fee, each risky position's absolute weight is $549.5/1098.9 \approx 0.500046$.

Fees depend on actual traded notional, cash updates at each event, and positions must first be marked at current prices to determine rebalancing requirements. Chapter 41's historical fixed-weight proxy charges costs on the return-ending row and approximates turnover from weight changes; it does not contain this complete event sequence. Its historical curve cannot replace the P6 event ledger. Financing, borrow fees, impact, and capacity are absent from this toy calculation; it verifies cash and NAV identities under the stated assumptions.

Chapter 44 — Stochastic Calculus and Derivatives

Yuxiao Ma · Advanced. Thread: start with no-arbitrage replication, build continuous pricing from available information and Itô quadratic variation, then check its boundaries through numerical results and cash ledgers.

1. Learning objectives

This chapter follows a European option from discrete replication to continuous replication, then checks analytical formulas, trees, and simulation against one another. You should be able to explain the origin of the Itô second-order term, construct a left-point integral for a specified information set, distinguish the physical measure from the pricing measure, and recompute a discrete hedging ledger with dividends and financing row by row.

The assessment actions are to calculate the stock and bond positions in one-period replication; derive the Black–Scholes PDE with continuous dividends; compute call and put Greeks in consistent units; distinguish MC sampling error, Euler discretization bias, and discrete hedging error; and use a payoff kink, an invalid tree probability, or a future-information counterexample to explain when an algorithm fails. Each action has exercises and full solutions in this chapter. Every experiment is labeled SIMULATED or CONSTRUCTED.

The level is Advanced. Prerequisites are Chapter 4 on Taylor expansions and partial derivatives, Chapter 6 on conditional expectations and normal/lognormal distributions, Chapter 9 on constraints and dimensions, Chapter 10 on contracts and payoffs, and Chapter 11 on price returns and total returns. Chapters 34–36 supply the boundaries concerning executable prices and costs. Ordinary equity alpha research follows its existing route and does not require completing stochastic calculus first.

2. Why a quant needs this

An investor forecasting next year's stock appreciation and a desk quoting a European call face different tasks. Forecasting estimates the physical distribution. No-arbitrage pricing seeks stock and cash positions whose payoff matches the option. If two portfolios have the same payoff in every model state, their current prices in an ideal market must agree; otherwise one could buy the cheaper portfolio and sell the dearer one.

A discrete tree uses two states to solve two equations. A continuous model has infinitely many paths. Adjusting the stock position to cancel instantaneous random risk requires calculating changes in functions of those paths. Brownian paths have no ordinary derivative, and the ordinary chain rule misses an accumulated second-order term. This is the concrete reason stochastic calculus enters continuous replication.

A pricing model also supplies risk sensitivities. A Delta hedger needs today's stock position, the next rebalancing time, and the cash effect of trading. A return forecaster needs the physical drift and sample evidence. Treating a risk-neutral distribution as a future-return forecast turns a pricing calculation into a purported trading signal.

3. Intuition: solve the replicating positions in two states

Teaching construction: the current stock price and European call strike are both 100. In one year the stock can only be 120 or 80; cash grows by a factor of 1.05, and there are no dividends. The option pays 20 or 0. Holding Δ shares and a current cash amount A requires both equations:

$$120\Delta + 1.05A = 20, \qquad 80\Delta + 1.05A = 0.$$

Subtracting gives 40Δ=20, hence Δ=0.5. Substitution in the down state gives A=−40/1.05. Negative cash means borrowing. In the up state, stock worth 60 minus debt of 40 leaves 20; in the down state, stock worth 40 minus debt of 40 leaves 0. Replication costs 50−40/1.05. No physical up probability is needed.

Choose an auxiliary probability p that makes the stock's expected growth factor 1.05: 1.2p+0.8(1−p)=1.05, so p=0.625. Weighting the option payoffs by this probability and dividing by 1.05 gives the same replication cost. This p comes from replication and discounting. The physical up probability could be 0.3 or 0.8 without changing the algebra. A multiperiod tree solves the replicating positions again at each node; its continuous limit becomes a predictable dynamic holding.

This tree permits only two states, no costs, and a common borrowing/lending rate. An additional independent risk state with only stock and a bond may prevent replication of every payoff. With dividends, price changes must be combined with cash received by the holder. The GBM model below retains one Brownian risk, continuous trading, and ideal financing, then checks the errors caused by discrete execution.

4. Mathematical foundations, units, and the information contract

Fix maturity T, current calendar time t, and remaining maturity τ=T−t, all measured in years. S is the ex-dividend stock price, K the strike, and V the value of an option on one share, all in the same quote currency. An actual contract multiplier is applied to the final amount; this chapter does not assume 100 shares. The continuously compounded annual risk-free rate and continuous dividend yield are r and q. Annualized volatility σ has dimension $\text{year}^{-1/2}$. Under the physical measure, μ specifically denotes the total-return drift including dividends, so the price drift is μ−q. Probability measures are $\mathbb{P}, \mathbb{Q}$; the put price is P_{put}, or a clearly local P, to keep the two uses distinct.

Brownian increments satisfy $\text{Var}(\Delta W) = \Delta t$, so W has units $\sqrt{\text{year}}$. A normal generator's scale parameter is the standard deviation: the code passes $\sqrt{\Delta t}$. Passing Δt would produce variance Δt^2. The return $\sigma\Delta$W and rΔt are dimensionless, allowing the terms in an exponent to be added.

The market price of risk $\lambda = (\mu - r)/\sigma$ has units $\text{year}^{-1/2}$: μ and r have units year^{-1}, while σ has units $\text{year}^{-1/2}$. Thus λW and λ^2t are dimensionless. Δ is the number of shares per replicated option and A is a cash amount; ΔS and A can be added. Where code retains the parameter name T, **API T=**τ in pricing, Greeks, and MC means years remaining from valuation to maturity,

rather than a calendar maturity date. The frozen example has current t=0, so numerically τ=T; its inputs are retained.

Here $f \in C^{1,2}$ means continuously differentiable once in time and twice in space. Nonsmooth payoff points at maturity are handled separately as boundaries. The call payoff $(S-K)^+$ has a kink at S=K. For T>t, the GBM Gaussian integral smooths the price on S>0; it does not make the terminal kink ordinarily differentiable.

Every strategy holding is determined at grid left endpoint t_i from information then available and earns the change over $(t_i, t_{i+1}]$. Numerical hedging observes S_i and rebalances immediately at that same S_i, an idealized simulation convention. Real-data applications must separately specify `available_at`, `signal_time`, submission, matching, and `trade_time`. An offline array calculation cannot establish instantaneous execution. Filtration and adaptedness below express "information then available" mathematically.

5. Formal definitions: from random paths to available information

5.1 Random-walk scaling and Brownian motion

Divide one year into N intervals, Δt=1/N, and let independent fair coins ξ_i take ± 1 with probability 1/2 each. Each scaled step is $\sqrt{\Delta t}\xi_i$. The N-step sum has mean 0 and variance $N\Delta t = 1$. Using $\Delta t\xi_i$ instead gives terminal variance 1/N and a limit degenerate at 0; leaving steps unscaled makes variance diverge. This explains diffusion scaling. The CLT first describes the normal limit of a fixed terminal value. Weak convergence of the entire path requires a functional CLT and tightness; this chapter cites that result and does not treat the finite-dimensional CLT as its proof. A finite coin path is not a Brownian path either. [CH06_DOC_13]

Standard Brownian motion has W_0=0, almost surely continuous sample paths, independent increments over disjoint intervals, and $W_t - W_s \sim N(0, t-s)$. We generate paths from these increments; connecting lines merely display finite observation points. For a compatible Brownian filtration, future increments must also be independent of current $\mathcal{F}_s$. This chapter uses the completed, right-continuous natural filtration. Adding advance information about the future can break that premise.

The difference quotient obeys $(W_{t+h} - W_t)/h \sim N(0, 1/h)$. For fixed finite M, $\Pr(|\Delta W/h| \leq M) \to 0$, so this family is not tight and cannot converge in probability to a finite derivative. Almost-sure nowhere differentiability is a stronger standard theorem requiring simultaneous control of all times; diverging variance alone is not its full proof. One may still take partial derivatives of a price function V(t,S), but cannot use an ordinary path velocity dW/dt.

5.2 Information, conditional expectation, and martingales

A filtration $\mathcal{F}_s \subseteq \mathcal{F}_t$ represents growing information. An adapted process has X_t measurable with respect to $\mathcal{F}_t$. Measurability lets X_t be determined from information available at t without looking at later increments. Continuous-time integration also requires suitable joint measurability, such as a progressively measurable or predictable integrand. Saying "adapted" at each time does not automatically remove those requirements.

Conditional expectation Y=$E[X \mid \mathcal{F}_s]$ is measurable at s and satisfies $E[Y1_A] = E[X1_A]$ for every $A \in \mathcal{F}_s$. In finite states it is an average with probabilities recalculated given the known state. A martingale M is adapted, has $E|M_t| < \infty$ at each time, and satisfies $E[M_t|\mathcal{F}_s] = M_s$ for s≤t. This is a conditional relation; a constant unconditional $E[M_t]$ is insufficient.

Make information concrete with four equally likely terminal states. Independent fair ±1 coins ξ and η are revealed in that order. Set $M_0 = 0, M_1 = \xi, M_2 = \xi + \eta$.

Terminal state (ξ,η)	Probability	$\mathcal{F}_0$ information group	$\mathcal{F}_1$ information group	$\mathcal{F}_2$ information group
(+1,+1)	1/4	All four states	Two states with ξ=+1	This terminal state
(+1,−1)	1/4	All four states	Two states with ξ=+1	This terminal state
(−1,+1)	1/4	All four states	Two states with ξ=−1	This terminal state
(−1,−1)	1/4	All four states	Two states with ξ=−1	This terminal state

Terminal state (ξ,η)	M_1	M_2
(+1,+1)	+1	2
(+1,−1)	+1	0
(−1,+1)	−1	0
(−1,−1)	−1	−2

Strictly, $\mathcal{F}_t$ is the σ-algebra of events decidable at that time; the information groups in the table are its atoms. In finite states, $\mathcal{F}_t$-measurable means constant within every group that cannot yet be distinguished. M_1 is respectively +1 and −1 throughout the two $\mathcal{F}_1$ groups and is available at time 1. M_2 still varies within each group and cannot be used early as M_1. Conditional expectation recalculates within-group probabilities: given ξ=+1, the two states have probability 1/2 each and mean (2+0)/2=1; given ξ=−1, their mean is (0−2)/2=−1. Thus $E[M_2 \mid \mathcal{F}_1] = \xi = M_1$; both have mean 0 under $\mathcal{F}_0$, forming a martingale. Similarly, W_T contains unrevealed increments when t<T and is generally not $\mathcal{F}_t$-measurable. A value being stored in an array does not turn future information into current information.

A finite-horizon counterexample uses t=0,1,2 and a fair ±1 coin ξ: X_0=0, X_1=ξ, X_2=−ξ. All three means are 0. At time 1 ξ is already known, so $E[X_2 \mid \mathcal{F}_1]$=−$\xi$≠$\xi$ and X is not a martingale. Replacing the terminal value by M_2=ξ+η with an independent fair coin η gives $E[M_2 \mid \mathcal{F}_1]$=$\xi$=$M_1$, satisfying the condition. Left-point holdings multiplied by independent future increments rely on this same conditioning step.

5.3 Quadratic variation and the domain of the Itô integral

Total variation is $\sup_\Pi \sum_i |X_{t_{i+1}} - X_{t_i}|$ over all finite partitions. For a continuously differentiable path it equals $\int |X'|dt$. Brownian total variation is almost surely infinite, while squared increments can accumulate to a finite value. Along deterministic partitions with maximum step tending to zero, Brownian quadratic variation satisfies

$$Q_\Pi = \sum_i (\Delta W_i)^2 \xrightarrow{L^2} T.$$

For the convergence needed here, $E[Q] = \sum_i \Delta t_i = T$. Independent Gaussian increments give $\mathrm{Var}((\Delta W_i)^2) = 2\Delta t_i^2$, hence $\mathrm{Var}(Q) \le 2T|\Pi| \to 0$. Therefore $E[(Q-T)^2] \to 0$, which also implies convergence in probability. Defining the quadratic variation process $[W]_t = t$ does not assert that every finite-grid QV equals t. Arbitrary path-dependent random partitions need separate treatment.

Start with a left-point simple process: $H_t = H_i$ on $(t_i, t_{i+1}]$, with H_i known at t_i, and define the integral as $\sum H_i \Delta W_i$. For a general progressively measurable H with $E\int_0^T H_t^2 dt < \infty$, approximate it by simple processes $H^{(n)}$ in $L^2(dt \times \mathbb{P})$, then extend the integral to the L^2 limit using the isometry. For continuous H=W or the smooth path functions used here, left-point approximations meet the required conditions. An arbitrary left-point grid alone cannot establish convergence for an irregular H. [CH44_MIT_ITO]

On this domain the Itô integral is a square-integrable martingale starting at 0. In particular $E[\int H dW]$=0, and the Itô isometry is

$$E\left[\left(\int_0^T H_t dW_t\right)^2\right] = E\int_0^T H_t^2 dt.$$

5.4 SDEs, GBM, and the pricing model

A stochastic differential equation (SDE), $dX = a(t,X)dt + b(t,X)dW$, abbreviates an integral equation. The notation alone establishes neither existence nor uniqueness. Local Lipschitz and linear-growth conditions are common sufficient conditions. Here constant-coefficient ABM and GBM can be checked directly through explicit solutions; we do not solve arbitrary-coefficient SDEs.

The full equation, for every t, is

$$X_t = X_0 + \int_0^t a(u, X_u)\, du + \int_0^t b(u, X_u)\, dW_u.$$

Integral	What accumulates	Conditions and units used here
Time integral $\int$a du	Drift times elapsed time	Locally integrable along the path; a has X units/year
Stochastic integral $\int$b dW	L² limit of currently known coefficients times future Brownian increments	Progressively measurable/predictable with finite $E\int b^2\,du$, or localization allowed by the theorem; b has X units/$\sqrt{}$year

The second term does not first compute an ordinary dW/du and then integrate it. The dX in SDE shorthand cannot simply be separated by ordinary differential rules. The logarithmic transformation below must retain the Itô second-order term.

A European option pays $h(S_T)$ only at fixed T: $(S_T - K)^+$ for a call and $(K - S_T)^+$ for a put. Continuous replication below assumes S>0, constant r/q/σ, σ>0, one Brownian risk, no transaction costs, divisible/shortable stock, borrowing and lending at the same r, continuous predictable trading, and integrability/admissibility restrictions excluding pathologies such as doubling strategies. Numerical pricing supports T=0 and σ=0 boundaries, while the measure-change and stochastic-Greeks derivations require positive T and σ.

6. Derivations: integrals, measures, and replication prices

6.1 Why the left-point integral has a second-order correction

For any finite sequence, $a(b-a) = [b^2 - a^2 - (b-a)^2]/2$. Substitute a=$W_i$ and b=W_{i+1}, then telescope the squared endpoint differences. With W_0=0 this gives

$$I_\Pi = \sum_i W_i \Delta W_i = \tfrac{1}{2}\left(W_T^2 - Q_\Pi\right).$$

This is an exact finite algebraic identity. Taking the L² limit of Q_Π from the previous section gives $\int_0^T W_t dW_t = (W_T^2 - T)/2$. For an ordinary smooth path starting at 0 the squared-increment sum tends to 0, so its ordinary chain rule has no $-$T/2 term. Right-point sampling instead gives $(W_T^2 + Q_\Pi)/2$ and a limit larger by T. The two choices use different information and define different integrals.

The simple-process proof of the isometry expands term by term. Cross terms i<j in $E[(\sum H_i \Delta W_i)^2]$ vanish after conditioning on $\mathcal{F}_{t_j}$: earlier factors are then known and $E[\Delta W_j \mid \mathcal{F}_{t_j}]$=0. Diagonal terms are $E[H_i^2]\Delta t_i$. Their sum is $E\sum_i H_i^2 \Delta t_i$. L² approximation by simple processes yields the general isometry. For H=W its right side is $\int_0^T E[W_t^2]dt = T^2/2$, agreeing with the variance of $(W_T^2 - T)/2$.

6.2 Itô's formula: which Taylor terms survive accumulation?

To see the increments, first expand for sufficiently smooth functions and locally bounded coefficients, evaluating all derivatives at the left endpoint:

$$\Delta f \approx f_t \Delta t + f_x \Delta X + \tfrac{1}{2} f_{xx} (\Delta X)^2.$$

Second-order time terms and time-space cross terms disappear in the accumulated limit; the squared space term survives. Writing $\Delta X \approx a_i \Delta t + b_i \Delta W$ gives the square $a_i^2 \Delta t^2 + 2 a_i b_i \Delta t \Delta W + b_i^2 (\Delta W)^2$. Local boundedness and grid refinement remove the first two terms; weighted quadratic variation turns the last into the integral of $b_t^2 dt$. The scale $O_p(\sqrt{\Delta t})$ is probabilistic, not a common ordinary big-O bound on every path.

Why random left-point weights still permit replacement of squared increments. Fix a deterministic partition $0 = t_0 < \cdots < t_n = T$. Let C_i be $\mathcal{F}_{t_i}$-measurable with $|C_i| \le K$ for one deterministic constant K. Put $\varepsilon_i = (\Delta W_i)^2 - \Delta t_i$ and $R_\Pi = \sum_i C_i \varepsilon_i$. Independence of the increment from left-point information and Gaussian second and fourth moments give

$$E[\varepsilon_i \mid \mathcal{F}_{t_i}] = 0, \qquad E[\varepsilon_i^2 \mid \mathcal{F}_{t_i}] = 2(\Delta t_i)^2.$$

For i<j, $C_i \varepsilon_i$ and C_j are known at t_j. Conditioning on $\mathcal{F}_{t_j}$ therefore gives $E[C_i \varepsilon_i C_j \varepsilon_j] = 0$. The weights need not be mutually independent: the next centered squared increment has conditional mean 0 given existing information. Consequently,

$$\begin{aligned} E[R_\Pi^2] &= 2 \sum_i E[C_i^2] (\Delta t_i)^2 \\ &\le 2K^2 \sum_i (\Delta t_i)^2 \le 2K^2 T |\Pi| \longrightarrow 0. \end{aligned}$$

Thus $\sum C_i (\Delta W_i)^2 - \sum C_i \Delta t_i \to 0$ in L^2. Writing the second sum as $\int C_t \, dt$ still requires the corresponding process and convergence conditions for its Riemann sums. The same conditioning gives $E[(\sum C_i \Delta t_i \Delta W_i)^2] \le K^2 \sum (\Delta t_i)^3 \le K^2 T |\Pi|^2$, while $\sum (\Delta t_i)^2 \le T |\Pi|$ controls the squared-time terms. Weights that use future values do not satisfy the conditioning premise.

For the smooth functions here, stopping within a compact region bounds the coefficient/derivative weights; localization then handles leaving that region. This connects the bounded calculation to the general Itô formula, without presenting the short bound as a full remainder proof. The conclusion is for deterministic partitions and does not extend here to arbitrary path-dependent random partitions.

For $f \in C^{1,2}$, Itô's formula is therefore

$$df(t, X_t) = (f_t + a f_x + \tfrac{1}{2} b^2 f_{xx}) dt + b f_x dW_t.$$

The Taylor argument explains the second-order term. The general theorem for $C^{1,2}$ functions and locally integrable coefficients still needs approximation and localization to control remainders. A single o(Δt) symbol does not replace those steps. The bookkeeping rules $(dW)^2 = dt$, $dt\, dW = 0$,

and dt^2=0 describe quadratic-variation contributions in integral limits; they are not pointwise ordinary algebra for finite random increments. [CH44_MIT_ITO, Lecture17]

For f=x^2, a=0, b=1, we have f_x=2x and f_{xx}=2, hence d(W^2)=2WdW+dt; integration recovers the previous result. Two correlated Brownian risks require cross quadratic variation $d[W^1, W^2] = \rho dt$. The implementation here is one-dimensional.

6.3 Explicit ABM/GBM solutions, moments, and discretization error

Arithmetic Brownian motion (ABM) writes dX=a dt+b dW with local symbols a in X units/year and b in X units/$\sqrt{}$year. Direct integration gives $X_t = X_0 + at + bW_t$, mean X_0+at, and variance b^2t. Its normal support includes negative values, which violates a nonnegative-stock requirement. Applications to spreads or state variables allowed to be negative must identify their object separately.

Geometric Brownian motion (GBM) has price drift β: dS=βSdt+σSdW. Here β is a local symbol for the price drift: β=μ−q under P and β=r−q under Q. For f(S)=log S, f'=1/S and f''=$-1/S^2$. Itô's formula gives

$$d\log S = (\beta - \sigma^2/2)dt + \sigma dW, \quad S_t = S_0 e^{(\beta-\sigma^2/2)t+\sigma W_t}.$$

We can also verify the candidate explicit solution backward rather than trust ordinary separation of variables. Let $F(t, w) = S_0 \exp[(\beta - \sigma^2/2)t + \sigma w]$. Its derivatives are $F_t = (\beta - \sigma^2/2)F$, $F_w = \sigma F, F_{ww} = \sigma^2 F$. Apply Itô to $F(t, W_t)$:

$$dS = \big[(\beta - \sigma^2/2)S + \sigma^2 S/2\big]dt + \sigma S\, dW = \beta S\, dt + \sigma S\, dW.$$

The exponential is strictly positive at finite t and finite Brownian values, so S_0>0 implies S_t>0. Positivity permits the logarithm and is also verified directly by the explicit solution. One may derive the logarithmic equation with stopping/localization from a positive initial value and then verify the positive solution; the $-\sigma^2/2$ term missed by the ordinary chain rule cannot be omitted.

The variable σW_t is N(0,σ^2t). For normal Y with mean m and variance v, its exponential moment $E[e^{uY}] = e^{um+u^2v/2}$ gives $E[S_t] = S_0 e^{\beta t}$, $E[S_t^2] = S_0^2 e^{(2\beta+\sigma^2)t}$, and $\mathrm{Var}(S_t) = S_0^2 e^{2\beta t}(e^{\sigma^2 t} - 1)$. These supply analytical references for the terminal distribution and simulation means; they are not a single path's compounded return.

Exact grid sampling is $S_{i+1} = S_i \exp[(\beta - \sigma^2/2)\Delta t + \sigma\Delta W_i]$; Euler–Maruyama uses $S_{i+1} = S_i(1 + \beta\Delta t + \sigma\Delta W_i)$. The latter factor can be negative: β=0, σ=1, dt=1, ΔW=−2 gives −100 from an initial 100, while the exact solution remains positive. This constructed counterexample diagnoses negative-price risk; it does not alter the frozen main simulation parameters.

Strong error compares paths driven by common noise. Here we estimate terminal $\sqrt{M^{-1}\sum(S_T^E - S_T^{exact})^2}$. Weak error compares expectations of a function; here g(S)=S only. We report the paired mean difference $\bar{D}$ and $SE = s_D/\sqrt{M}$. The population means of the exact and Euler solutions are respectively $S_0 e^{\beta T}$ and $S_0(1 + \beta dt)^N$. Higham (2001), §5 defines strong

order through expected absolute error and gives Euler strong order 1/2 and, under suitable conditions, weak order 1. Our terminal L^2 RMSE is a different error estimator, whose numerical values must not be interchanged with that L^1 measure. [CH44_HIGHAM2001]

For linear GBM we can derive the population RMSE directly. Set h=T/N, single-step Euler factor X=1+βh+$\sigma\Delta$W, and exact factor Y=exp[($\beta-\sigma^2/2$)h+$\sigma\Delta$W]. Gaussian exponential moments and independent increments give

$$\begin{aligned} E[X^2] &= (1+\beta h)^2 + \sigma^2 h, \\ E[Y^2] &= e^{(2\beta+\sigma^2)h}, \\ E[XY] &= e^{\beta h}[1 + (\beta+\sigma^2)h]. \end{aligned}$$

The terminal mean squared error is consequently

$$\begin{aligned} E[D^2]/S_0^2 = [(1+\beta h)^2 + \sigma^2 h]^N + e^{(2\beta+\sigma^2)T} \\ - 2e^{\beta T}[1 + (\beta+\sigma^2)h]^N. \end{aligned}$$

Expand each Nth power through N log(1+a h+b h^2). Combining its first-order h coefficients yields the leading mean squared error term

$$S_0^2 e^{(2\beta+\sigma^2)T}\sigma^4 T h/2.$$

Thus population RMSE for this GBM is O($\sqrt{}$h). Expanding the two means in §6.3 directly gives a mean difference of O(h). Finite-grid plots show compatibility of sample estimates with these restricted analytical references; they do not prove convergence orders for arbitrary SDEs or payoffs.

6.4 From P to Q: constant coefficients over a finite horizon

Fix finite T and $\sigma>0$, with μ the total-return drift including dividends. Define $\lambda=(\mu-r)/\sigma$ and $Z_t = e^{-\lambda W_t^{\mathbb{P}} - \lambda^2 t/2}$. This is a positive density. Gaussian exponential moments give $E[Z_t]=1$ directly, and independent future increments give $E[Z_t \mid \mathcal{F}_s]=Z_s$. Novikov's sufficient condition $E\exp[\frac{1}{2}\int_0^T \lambda^2 dt] < \infty$ equals $e^{\lambda^2 T/2}$ for constant finite λ and T and is satisfied. The conclusion depends on those conditions. [CH44_CMU_RNM]

Girsanov's theorem gives $d\mathbb{Q} = Z_T d\mathbb{P}$ and $W_t^{\mathbb{Q}} = W_t^{\mathbb{P}} + \lambda t$. In the constant case we can also see the tilted increments: multiplying the P density of ΔW by $e^{-\lambda\Delta W - \lambda^2\Delta t/2}$ and completing the square produces N($-\lambda\Delta$t,Δt). Thus $\Delta W_Q = \Delta W_P + \lambda\Delta t$ is again N(0,Δt). The density factorizes over disjoint increments, preserving independence, and adding deterministic drift preserves continuity. This is a restricted verification, not a full proof for general random λ.

Substitute $dW_P = dW_Q - \lambda dt$:

$$dS = (\mu-q)Sdt + \sigma S(dW_Q - \lambda dt) = [\mu - q - \sigma\lambda]Sdt + \sigma S dW_Q = (r-q)Sdt + \sigma S dW_Q.$$

The q terms cancel in λ because μ was defined as total-return drift. This change reweights probabilities on the same path space. It does not replace real investors' expected returns with r or establish known returns under P.

6.5 The correct martingales for a dividend-paying stock

Under Q, $d(e^{-rt}S_t) = -qe^{-rt}S_t dt + \sigma e^{-rt}S_t dW_Q$. For q≠0 the discounted ex-dividend stock itself is generally not a martingale; $E_Q[e^{-rT}S_T \mid \mathcal{F}_t] = e^{-rt}S_t e^{-q(T-t)}$. Include cash dividends in the discounted gain process:

$$G_t = e^{-rt}S_t + \int_0^t e^{-ru}qS_u du, \quad dG_t = \sigma e^{-rt}S_t dW_Q,$$

This is a true martingale under the finite-horizon GBM second-moment conditions here. Alternatively, reinvest dividends continuously: shares n_t satisfy dn=q n dt and n_0=1, so $n_t = e^{qt}$, wealth is $e^{qt}S_t$, and its discounted value $e^{-(r-q)t}S_t$ is also a true martingale. The two constructions use dividend cash differently; both include dividends in gains instead of ignoring the price drift.

6.6 From one-period replication to a multiperiod CRR tree

For q=0 define growth $R = e^{r\Delta t}$, stock multipliers u>d>0, and down/up payoffs V_d/V_u. Replication gives ΔuS+AR=V_u and ΔdS+AR=V_d. Subtracting yields $\Delta = (V_u - V_d)/[S(u-d)]$; substituting into the down state gives $A = R^{-1}(uV_d - dV_u)/(u-d)$. Hence

$$V = \Delta S + A = e^{-r\Delta t}\left[\frac{e^{r\Delta t}-d}{u-d}V_u + \frac{u-e^{r\Delta t}}{u-d}V_d\right].$$

No arbitrage requires d<R<u, making both auxiliary state weights strictly positive. The stock and bond payoff columns are linearly independent, with determinant proportional to u−d. Thus this two-state market replicates any payoff, showing how completeness appears in concrete equations.

The Cox–Ross–Rubinstein (CRR) binomial tree sets $u = e^{\sigma\sqrt{dt}}, d = 1/u$. Continuous dividends are explained through the reinvested total-return asset: starting with Δ shares on the left, reinvestment gives Δe^{qdt} shares on the right. The stock contributions to the two replication payoffs are $\Delta e^{qdt}uS$ and $\Delta e^{qdt}dS$. Subtraction adds a factor e^{-qdt} to the initial stock position; rearranging the initial cost yields

$$a = e^{(r-q)dt}, \quad p = (a-d)/(u-d), \quad V_{k,j} = e^{-rdt}[pV_{k+1,j+1} + (1-p)V_{k+1,j}].$$

This establishes the dividend-adjusted forward-growth relation for the tree. Price changes alone are not the total return of a dividend-paying stock. Terminal stock values are $S_{N,j} = S_0 u^j d^{N-j}$. Calculate $h(S_{N,j})$, then work backward using two-state replication at each node. Level k has k+1 nodes; update the entire level together rather than overwrite a neighbor before reading it.

If a is not strictly inside (d,u), the two-state model fails the strict no-arbitrage requirement. Use a finer valid grid or reject explicitly. This chapter does not clip probabilities; the deterministic σ=0 boundary is handled separately. As N increases, alignment of the payoff kink with the grid changes, often producing odd/even oscillation. Smaller errors on selected grids are numerical observations, not a claim of monotone convergence for consecutive N.

6.7 Continuous self-financing replication and the Black–Scholes/Merton PDE

Replicate one European option with Δ shares and cash A, so V=ΔS+A. Self-financing means changes in Δ are paid from A with no external injection; gains arise from existing positions:

$$dV = \Delta(dS + qSdt) + rAdt = \Delta dS + q\Delta Sdt + r(V - \Delta S)dt.$$

This gain equation includes dividends and financing. Applying the product rule to ΔS produces SdΔ and cross variation, which are matched by stock-rebalancing transfers in the cash account. V=ΔS+A alone does not justify treating Δ as constant indefinitely. Admissible strategies and the integral conditions here make these stochastic gain equations meaningful.

First consider instantaneous cost-free rebalancing with no interval gains. At S=100, selling 0.1 share reduces stock value by 10 currency units and raises cash by 10; ΔS+A is unchanged. The 10 is an internal transfer, not an additional gain. When the smooth pre-maturity Delta here is a semimartingale, the product formula can be written explicitly:

$$\begin{aligned} d(\Delta S) &= \Delta\, dS + S\, d\Delta + d[\Delta, S], \\ dA &= rA\, dt + q\Delta S\, dt - S\, d\Delta - d[\Delta, S]. \end{aligned}$$

Adding cancels the rebalancing terms and cross variation, recovering the self-financing gain equation. Borrowed cash A accrues at the same r. Continuous dividends enter cash rather than automatically increasing shares. This expansion applies to the sufficiently smooth Delta used here; a general predictable strategy need not admit such a dΔ calculation. The general self-financing definition uses the gain integrals of existing holdings.

Under P, $dS = (\mu - q)Sdt + \sigma S\, dW_P$. Itô gives

$$dV = [V_t + (\mu - q)SV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS}]dt + \sigma S V_S dW_P.$$

The random coefficient in the replicating gain equation is σSΔ, so σS>0 requires $\Delta = V_S$. Its drift then becomes $(\mu - q)SV_S + qSV_S + rV - rSV_S = \mu SV_S + rV - rSV_S$. Match the two drifts:

$$V_t + (\mu - q)SV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} = \mu SV_S + rV - rSV_S.$$

Subtract μSV_S from both sides and rearrange to obtain the partial differential equation (PDE):

$$V_t + (r - q)SV_S + \tfrac{1}{2}\sigma^2 S^2 V_{SS} - rV = 0, \qquad V(T, S) = h(S).$$

The physical μ disappears because the replicating holding cancels the same Brownian risk and its drift is constrained by financing and dividends. We have not assumed μ=r. An additional

independent untraded risk, jumps, or transaction costs can invalidate the original replication equation. Merton (1973), §7, equation (44) uses remaining maturity and $-V_\tau$; converting to calendar t gives this equation. [CH44_MERTON1973]

Spatial boundaries are C(t,0)=0 and $P_{put}(t,0) = Ke^{-r\tau}$. At large S the call approaches $Se^{-q\tau} - Ke^{-r\tau}$ and the put approaches 0. S=0 is a continuous-limit boundary only; the public API requires S>0. Growth conditions and these boundaries exclude economically meaningless PDE solutions. A terminal condition alone is not a reason to ignore the solution space.

A pricing map for the same payoff. Fix $h(S_T)$, current (t,S), τ=T−t, and common financing/dividend conventions. Each layer has a different task:

Layer	Task for the same payoff	Required basis/conditions
Continuous replication → PDE	Cancel the same noise and match financing drift	Self-financing, tradable risk, smooth price, admissible strategy
Q conditional expectation	Represent the PDE price as an expected discounted payoff	Restricted Feynman–Kac below, a true martingale, and an integrable terminal limit
BS Gaussian integral	Evaluate that expectation for constant-coefficient GBM	d_2 gives exercise probability; d_1 comes from a stock-weighted truncated integral
CRR tree / Q MC	Discrete replication / sampling calculation	Valid tree probabilities and grid error / independent pairs and SE

Discrete hedging error on P paths checks the execution ledger within a specified model, a separate task. Q does not forecast physical stock appreciation; sharing a payoff does not imply a common P/Q drift. The next section uses bounded call/put Delta to ensure martingale conditions, with analytical verification in §6.10. The reverse route in §6.9 integrates first and checks the PDE afterward.

6.8 The Feynman–Kac link: why a Q expectation is available here

The Feynman–Kac representation connects a diffusion-generator PDE solution to a conditional expectation. Here its scope is constant r/q/σ, σ>0, positive S, finite T, continuous call/put payoffs of at most linear growth, and solutions that are $C^{1,2}$ for τ>0 with the corresponding growth and integrability.

For u∈[t,T), apply Itô under Q to $Y_u = e^{-r(u-t)}V(u,S_u)$. Its drift is the discounted PDE left side and is zero. Its random term is $e^{-r(u-t)}\sigma S_u V_S dW_Q$. Call/put $|V_S|$ is bounded by a constant depending on q and T, and finite-horizon GBM has second moments. The stochastic integrand is therefore square integrable, making Y a true martingale suitable for conditional expectation. Payoff continuity and the growth bound allow an integrable limit at T, giving

$$V(t,S_t) = E_Q[e^{-r(T-t)}h(S_T) \mid \mathcal{F}_t].$$

The Q-GBM Markov property makes this expectation a function of current t and S alone. A local martingale does not automatically preserve its expectation as a true martingale does;

the moment conditions are part of the argument. In the reverse direction, the explicit Gaussian integral below produces a smooth price that can be checked against the PDE. No unconditional assertion is made for arbitrary irregular payoffs or degenerate/discontinuous coefficients.

6.9 Deriving calls, puts, d1, and d2 by Gaussian integration

Given current S and τ, the Q terminal value is $S_T = S\exp[m+vZ]$, where m=(r−q−$\sigma^2/2$)τ, v=$\sigma\sqrt{\tau}$, and Z~N(0,1). Exercise requires Z>−d2, where $d2 = [\log(S/K)+m]/v$. Its probability is Φ(d2), and the expected strike contribution is KΦ(d2).

The stock contribution requires an exponentially weighted truncated normal integral. With ϕ the standard normal density, completing the square gives $e^{vz}\phi(z) = e^{v^2/2}\phi(z-v)$. Therefore

$$E_Q[S_T 1_{S_T>K}] = Se^m \int_{-d2}^{\infty} e^{vz}\phi(z)dz = Se^{m+v^2/2}\Phi(d2+v).$$

Define d1=d2+v, with m+v^2/2=(r−q)τ. Discounting the stock and strike terms gives the Black–Scholes/Merton formula:

$$\begin{aligned} C &= Se^{-q\tau}\Phi(d1) - Ke^{-r\tau}\Phi(d2), \\ P_{put} &= Ke^{-r\tau}\Phi(-d2) - Se^{-q\tau}\Phi(-d1), \\ d1 &= \frac{\log(S/K)+(r-q+\sigma^2/2)\tau}{\sigma\sqrt{\tau}}, \quad d2 = d1-\sigma\sqrt{\tau}. \end{aligned}$$

The quantity d2 supplies the Q exercise probability. The d1 term comes from exponential weighting of the stock payoff; Φ(d1) is not the unweighted exercise probability under that same Q. Delta will be $e^{-q\tau}\Phi(d1)$. The terminal call-minus-put payoff S_T-K, or Φ(x)+Φ(−x)=1, gives put-call parity: $C - P_{put} = Se^{-q\tau} - Ke^{-r\tau}$. The discounting relation remains valid for negative r.

6.10 Greeks: define the derivatives, then cancel repeated terms

Greeks are local risk sensitivities, with other contract parameters held fixed. $\Delta = V_S$ is replicating shares per option; $\Gamma = V_{SS}$ is the change in Delta per unit S; $\mathcal{V} = V_\sigma$ is currency per unit annualized volatility parameter; $\Theta = V_t = -V_\tau$ is currency/year; and $\rho = V_r$ is currency per unit continuously compounded annual rate. A first-order amount is the derivative times the parameter change. $\Delta\sigma$=.01 and Δr=.01 each mean one percentage point. The derivative is not an exact price change for a large shock.

Begin analytical differentiation with $Se^{-q\tau}\phi(d1) = Ke^{-r\tau}\phi(d2)$. To prove it, substitute d1=d2+$\sigma\sqrt{\tau}$ into the Gaussian density ratio: $\log[\phi(d1)/\phi(d2)] = -d2\sigma\sqrt{\tau} - \sigma^2\tau/2$. Substitution for d2 leaves $-\log(S/K) - (r-q)\tau$. Thus the d1/d2 chain-rule terms cancel when differentiating in S, leaving $\Delta_{\text{call}} = e^{-q\tau}\Phi(d1)$. Put Delta follows by subtracting $e^{-q\tau}$ via parity. Differentiate once more using $\frac{\partial d1}{\partial S} = \frac{1}{S\sigma\sqrt{\tau}}$ to obtain Gamma.

Differentiating in σ makes the two CDF chain terms a common density coefficient times $(d1_\sigma - d2_\sigma) = \sqrt{\tau}$, giving $\mathcal{V} = Se^{-q\tau}\phi(d1)\sqrt{\tau}$. In the r derivative, $\frac{\partial d1}{\partial r} = \frac{\partial d2}{\partial r} = \frac{\sqrt{\tau}}{\sigma}$, so the

chain terms cancel and leave the strike-discount derivative. Theta follows from substituting Delta/Gamma into the PDE to obtain $-V_\tau$, or by differentiating directly in maturity. The results are

$$\Delta_C = e^{-q\tau}\Phi(d1), \qquad \Delta_P = -e^{-q\tau}\Phi(-d1),$$
$$\Gamma_C = \Gamma_P = \frac{e^{-q\tau}\phi(d1)}{S\sigma\sqrt{\tau}}, \qquad \mathcal{V}_C = \mathcal{V}_P = Se^{-q\tau}\phi(d1)\sqrt{\tau},$$
$$\rho_C = K\tau e^{-r\tau}\Phi(d2), \qquad \rho_P = -K\tau e^{-r\tau}\Phi(-d2).$$

$$\Theta_C = -\frac{Se^{-q\tau}\phi(d1)\sigma}{2\sqrt{\tau}} + qSe^{-q\tau}\Phi(d1) - rKe^{-r\tau}\Phi(d2),$$
$$\Theta_P = -\frac{Se^{-q\tau}\phi(d1)\sigma}{2\sqrt{\tau}} - qSe^{-q\tau}\Phi(-d1) + rKe^{-r\tau}\Phi(-d2).$$

The PDE provides a scenario-by-scenario check: $\Theta+(r-q)S\Delta+\sigma^2S^2\Gamma/2-rV=0$. Call and put Theta can have different signs; time decay need not be negative for every parameter choice. Central differences use (V(x+h)−V(x−h))/(2h), and Gamma uses a second difference divided by h^2. Calendar Theta negates the remaining-maturity difference. Finite h causes truncation error; overly small h amplifies rounding through cancellation. Left/right derivatives differ at the maturity kink. This API rejects all T=0/σ=0 Greeks, while the documentation explains that deterministic derivatives away from the kink are unimplemented.

6.11 Monte Carlo under Q and antithetic variates

This subsection shifts the current valuation time to 0. Its T is the shifted remaining maturity, namely τ=calendar maturity−current time on the original clock. The terminal subscript T below uses that shifted clock, and API T also equals τ. At an arbitrary current t, one must not pass the unadjusted calendar maturity to MC.

Monte Carlo (MC) draws independent Z_i and uses exact Q terminal values $S_T^{(i)} = S\exp[(r-q-\sigma^2/2)T+\sigma\sqrt{T}Z_i]$ with discounted payoffs $Y_i = e^{-rT}h(S_T^{(i)})$. A European payoff depends only on the terminal value, so exact sampling removes Euler time-discretization bias. Finite random samples retain sampling error. The estimator is mean(Y), with $SE = s_Y/\sqrt{M}$. For finite variance and sufficiently large M, the normal approximation gives 95% CI=mean(Y)±1.95996398454SE; this is not exact finite-sample coverage.

Antithetic variates use each independent Z_i to generate both Z_i and $-Z_i$, with $X_i = (Y_i^+ + Y_i^-)/2$. The M values X_i are independent across pairs; the two payoffs within a pair are correlated. The correct SE is $s_X/\sqrt{M}$, with $\mathrm{Var}(X) = [\mathrm{Var}(Y^+)+\mathrm{Var}(Y^-)+2\,\mathrm{Cov}]/4$. Antithetic correlation for the monotone vanilla payoffs here usually reduces variance; this numerical conclusion is not extended to every payoff. At the same number of independent observations, antithetic sampling uses twice as many terminal-payoff evaluations. Cost comparisons must state this.

The typical SE scale as sample size grows is $M^{-1/2}$; the realized pricing error in a particular batch need not decrease monotonically. One analytical price falling inside a CI is one random outcome. Analytical/financial identities, independent formula checks, and boundary rejection

jointly assess algorithm correctness. A random sample with M=1 cannot estimate s^2. Only deterministic σ=0/T=0 cases directly return SE=0 and a point interval.

6.12 A discrete Delta-hedging ledger: who receives cash each interval?

Simulate the P price drift $\mu-$q with initial capital V_0 equal to the Q model price. Buy Δ_0 shares, leaving cash $A_0 = V_0 - \Delta_0 S_0 - \text{fee}|\Delta_0|S_0$. Initial purchase costs are deducted from the capital rather than financed by an unreported injection.

Let dt=T/N. Hold the previous post-rebalance shares Δ_{i-1} and cash A_{i-1} fixed. Before rebalancing at the next endpoint, cash follows the protocol's approximate recursion:

$$A_i^{pre} = A_{i-1}e^{rdt} + \Delta_{i-1}S_{i-1}(e^{qdt} - 1).$$

Continuous-model cash dividends are $\int q\Delta S_u\, du$. This recursion approximates interval dividends using the left-point price and fixed old holdings, and books all interval cash flow at the right endpoint. It omits differing financing times for dividends received within the interval. This discretization differs from the continuous formula. Exact sampling of S terminal values does not make the whole hedging ledger exact continuous replication.

Observe S_i, compute the new Delta with remaining maturity $T - t_i$ for the following interval only, and trade $\delta\Delta = \Delta_i - \Delta_{i-1}$. Cash becomes

$$A_i = A_i^{pre} - \delta\Delta S_i - fee|\delta\Delta|S_i, \quad V_i^{portfolio} = A_i + \Delta_i S_i.$$

At maturity force Δ_N=0 and deduct liquidation costs. Final error is $A_N - h(S_N)$. Both cash and shares must reconcile. Negative cash A accrues financing at the same r; the simulation has no real borrowing/lending spread. The sum $\sum \text{fee}|\delta\Delta|S$ is nominal cash fees only. Their effect on terminal funds also includes financing opportunity cost, so the difference in mean errors across frequencies cannot simply be equated to mean fees.

7. Financial Interpretation: Prices, Probabilities, and Hedging Cash

Stock price changes must be distinguished from total stock returns. Holding Δ shares produces dividend income of ΔqSdt; the borrowing balance accrues interest at rate r. If dividends are omitted from the replication ledger, the stock term in the PDE is incorrectly written as r instead of r$-$q, and call and put prices and Deltas will then be inconsistent. Two pricing algorithms can be compared only when they use the same payoff, financing, and allocation of cash flows.

$\mathbb{Q}$ is a measure consistent with the model's no-arbitrage assumptions. In the complete-market model used in this chapter—one Brownian source of risk, a stock and a cash account, σ>0, continuous trading, and suitably integrable strategies—a replicable European payoff has a unique model price. Completeness means that every admissible payoff can be replicated with tradable assets; with no arbitrage but an incomplete market, the price need not be unique. The case σ=0 is a deterministic boundary, so the conversion formula based on the market price of risk λ=($\mu-$r)/σ cannot be applied directly.

Monte Carlo pricing computes expectations under Q. The discrete hedging experiment instead sets μ=.08, including dividends, under P to examine the error along the specified model paths. The two experiments share r, q, σ, and the payoff, but differ in purpose, measure, and drift. Hedging error is defined as the cash remaining after liquidation minus the option payoff. A positive value means the replicating portfolio has excess cash; a negative value means it is short of the required cash. This is not the investment return from holding one option.

Frictionless continuous replication provides a model benchmark. Discrete rebalancing leaves Gamma risk, and approximating dividends and cash flows over intervals introduces error. More frequent rebalancing also increases fees. The 10 bp fee is a constructed input; it has not been checked against order-book quotes or estimated market impact. Connecting the hedging chart to real profitability would still require validation of quotes, latency, borrowing, jump risk, and data permissions.

Chapter 44 complete numerical example: all steps and conclusions

8. Numerical Example: Frozen European Pricing and Hedging Experiment

All results are SIMULATED / CONSTRUCTED, with each payoff corresponding to one share. Inputs are S_0=K=100, T=1 year, r=.05, q=.02, and σ=.20. The protocol was fixed before the first run; the historical output was originally in `build/batches/A9-A/r1/runs/experiment_output_02` and has since been archived. Original run identities are recorded in the archive inventory. The numbers below retain the accepted historical independent root run; its existing results.json is available separately. The existing root results and archived author run retain distinct identities. This bilingual batch neither reran the experiment nor extracted the archive. All amounts are per-share teaching amounts in the same currency.

8.1 Intermediate Values for the Hand Calculation and Analytic Price

For the one-period tree without dividends, the replicating position is .5 shares and cash of −38.0952380952, giving a price of 11.9047619048. The up-state payoff is 20, the down-state payoff is 0, and the auxiliary probability is p=.625, matching the equation in §3. Its multipliers 1.2/.8 and growth factor 1.05 differ from those of the main CRR grid, so the difference between their prices must not be called an algorithm error.

For the main example, log(S/K)=0, (r−q+σ^2/2)T=.05, and $\sigma\sqrt{}$T=.2, so d1=.25 and d2=.05. The discounted stock amount is $100e^{-.02}$ = 98.0198673307, and the discounted strike is $100e^{-.05}$ = 95.1229424501. Multiplying each by its respective CDF and taking the difference gives a call price of 9.2270055082 and a put price of 6.3300806275. The parity difference is 2.8969248806, with a calculation residual of 0 within double-precision range. This identity checks dividends and discounting more directly than a single simulation interval covering the price.

Continued: the same complete example

8.2 CRR and Analytic Price Comparison

Steps	call	put	call−BS
10	9.03532574	6.13840086	-0.19167977
25	9.30117882	6.40425394	+0.07417331
50	9.18822483	6.29129994	-0.03878068
51	9.26328514	6.36636025	+0.03627963
100	9.20758997	6.31066509	-0.01941554
101	9.24530483	6.34837995	+0.01829933
200	9.21729155	6.32036667	-0.00971396
400	9.22214700	6.32522212	-0.00485851

The 25/50/51 and 100/101 grids show the effects of parity and payoff-grid alignment. Call and put errors overlap in the chart because of parity in this model. Valid probabilities are rechecked as the step size changes; connecting lines are only for reading adjacent choices. No claim of monotone convergence is made.

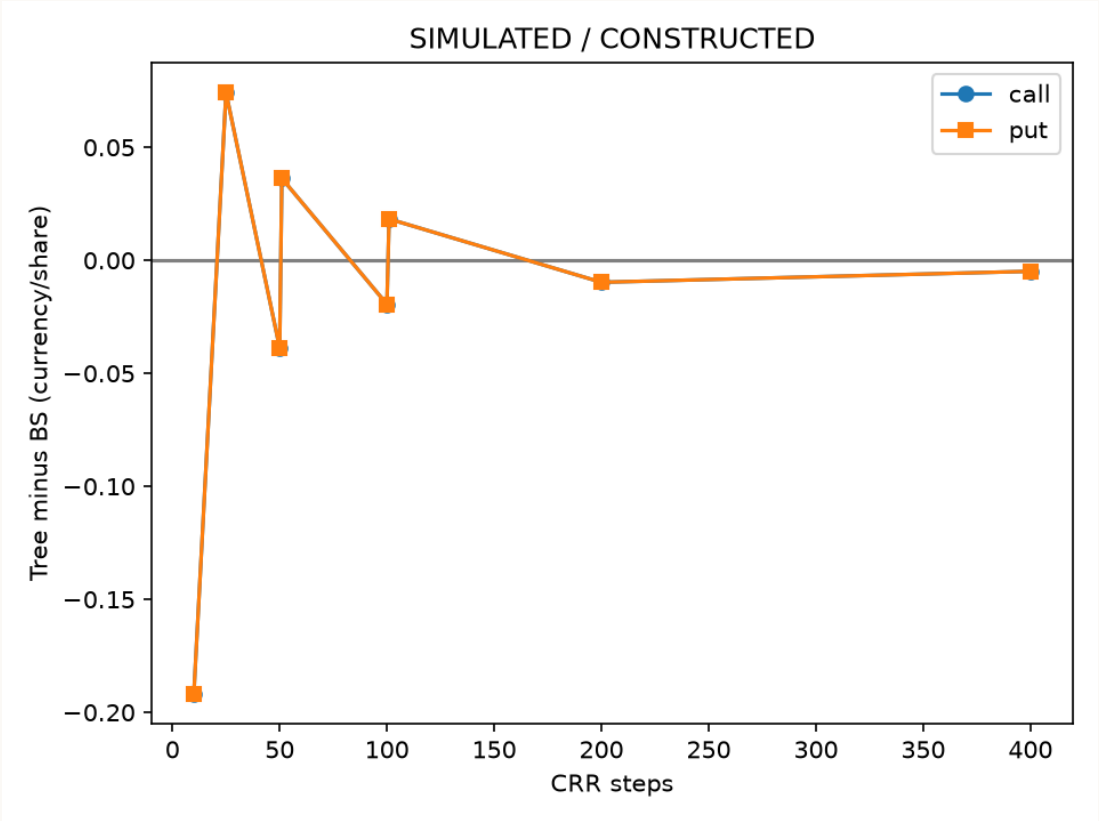


Figure 1: SIMULATED CRR minus BS: steps on the horizontal axis and per-share price error on the vertical axis; parity alignment and error scale are shown.

8.3 Exact Terminal-Value Monte Carlo under Q

Seed 4401. For antithetic sampling, the independent unit is the pair mean. Each row uses M independent pairs and 2M terminal payoff calculations; the 95% interval is a normal approximation.

Continued: the same complete example

Type	Independent pairs M	Price	SE	Approximate 95% CI
call	1000	8.970967	0.226342	[8.527344, 9.414590]
put	1000	6.168817	0.144720	[5.885170, 6.452464]
call	5000	9.221358	0.103914	[9.017691, 9.425026]
put	5000	6.322140	0.066202	[6.192386, 6.451894]
call	25000	9.210317	0.045884	[9.120386, 9.300248]
put	25000	6.322468	0.029532	[6.264586, 6.380350]
call	50000	9.208583	0.032531	[9.144823, 9.272343]
put	50000	6.318800	0.020913	[6.277811, 6.359790]

For 50000 ordinary independent paths, the call SE is 0.061862; for the same number of independent pairs, the SE is 0.032531. The number of payoff calculations differs, so this is not a comparison at equal computational cost. A single interval covering the analytic price does not prove the algorithm is correct, and the error need not decrease monotonically each time M increases.

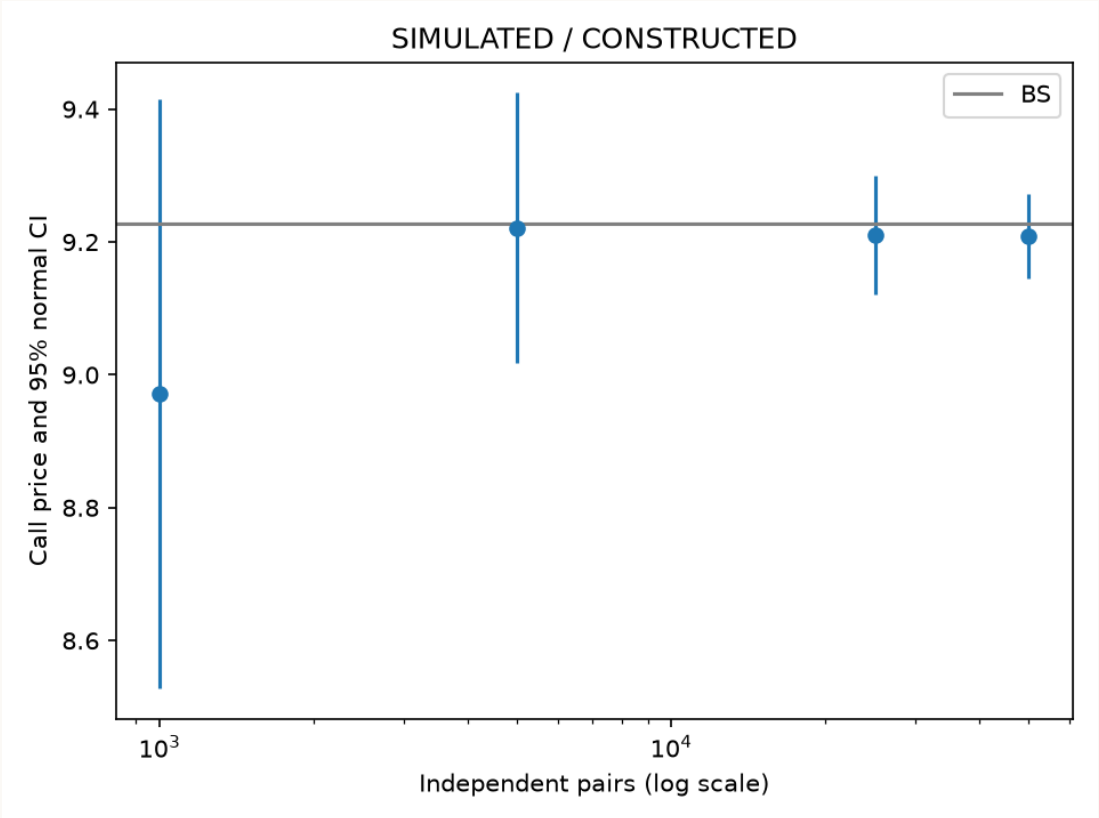


Figure 2: SIMULATED Q call MC: independent pair count on a logarithmic horizontal axis; price and approximate 95% normal intervals on the vertical axis; the gray line is the analytic price.

8.4 Finite-Grid Brownian Quadratic Variation and Its Limit

Seed 4402, with 1024 fine increments on each of 1024 paths, aggregated into nested grids; T=1.

Continued: the same complete example

Steps	Sample mean QV	Sample SD of QV	Maximum finite-identity residual	RMSE relative to Itô limit
16	1.013915	0.355776	6.66e-16	0.177937
64	1.002018	0.174564	4.44e-16	0.087245
256	1.000527	0.089581	8.88e-16	0.044769
1024	0.999746	0.045050	8.88e-16	0.022514

Each finite identity uses the actual QV for that row; only the last column replaces QV with its theoretical limit T. The sample means and SDs are compared with the theoretical values $E[QV]$=1 and Var(QV)=2/N. The numerical pattern for this seed and these grids is not a general proof for all partitions or paths.

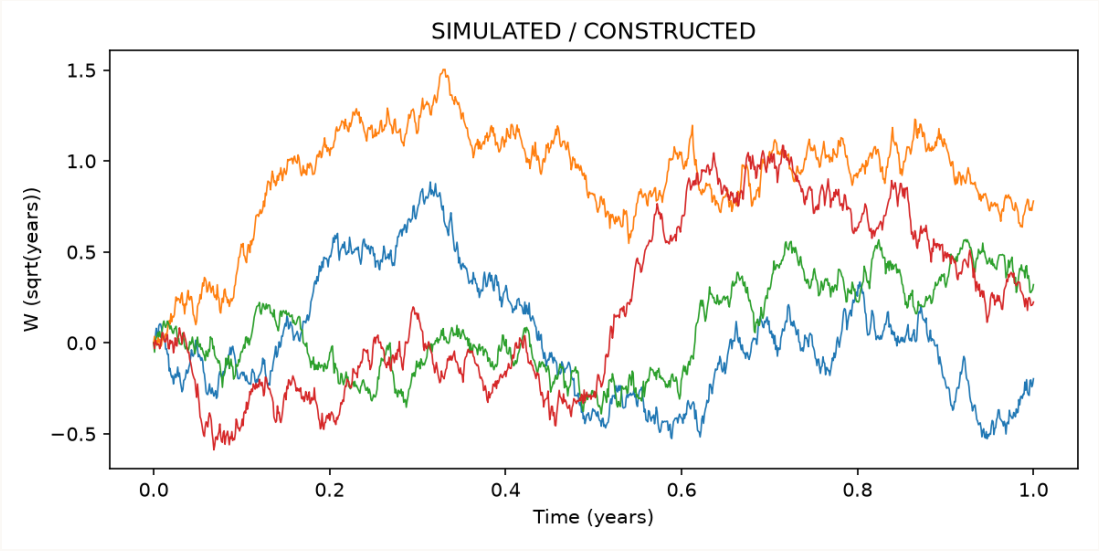


Figure 3: SIMULATED Brownian paths: years on the horizontal axis and W, in sqrt(years), on the vertical axis; four finitely sampled paths are shown. The connecting lines do not imply ordinary derivatives.

Continued: the same complete example

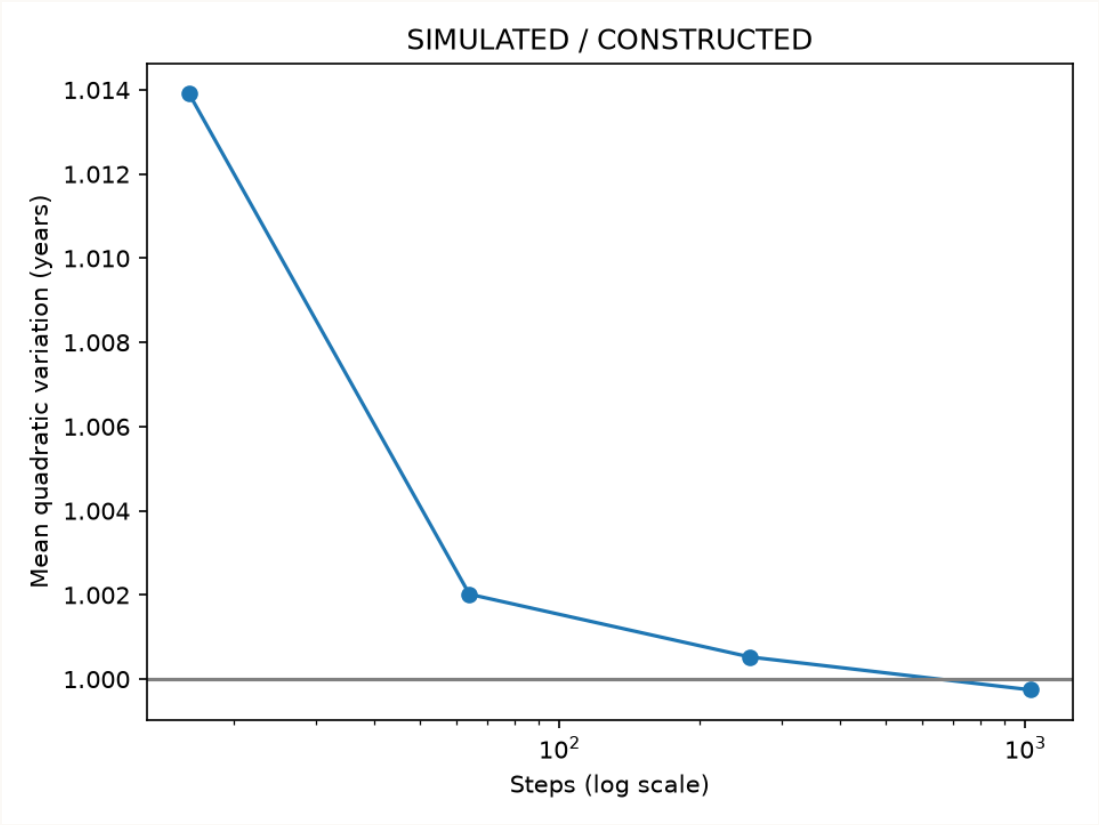


Figure 4: SIMULATED QV: steps on a logarithmic horizontal axis and the mean sum of squared increments across 1024 paths, in years, on the vertical axis; the gray line is T=1.

8.5 Exact GBM and Euler under P with Shared Increments

Seed 4403, with 4096 fine paths, dividend-inclusive μ=.08, and stock-price drift .06. Strong error uses the terminal difference D along the same Brownian path; the weak difference is the signed mean(D), and the paired SE is reported.

Steps	Strong RMSE	Paired weak mean difference	Paired SE	Population mean difference
16	0.770766	-0.024165	0.012039	-0.011915
64	0.374745	-0.002197	0.005856	-0.002985
256	0.186236	-0.000825	0.002910	-0.000746
1024	0.097747	-0.002740	0.001527	-0.000187

The strong-error scale on finer grids is consistent with Euler's strong order 1/2. The weak sample difference is larger at 1024 steps than at 256; the paired SE explains why monotonicity is not required. For the exact model, $E[S_1]$=106.183655; moments and the terminal distribution are given in §6.3. No negative Euler prices were observed on these grids, which describes only this sample; the separately constructed example with dW=−2 still produces a negative Euler price.

Continued: the same complete example

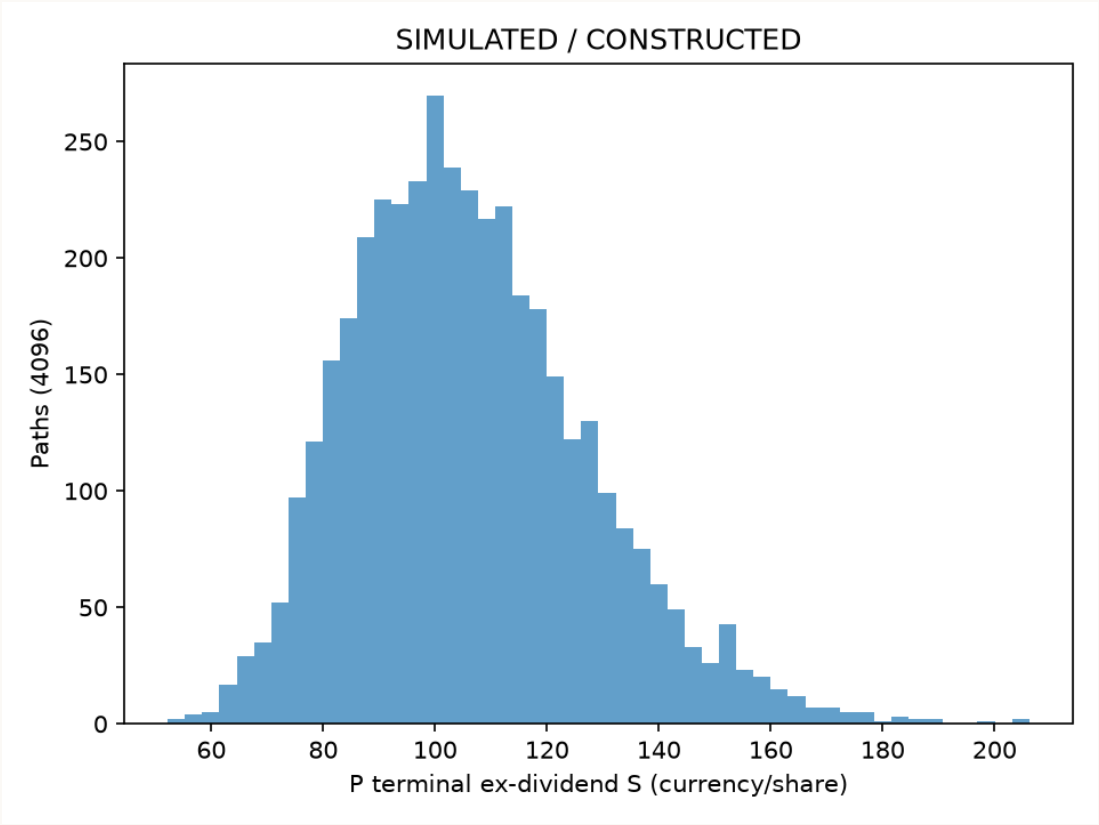


Figure 5: SIMULATED P GBM terminal distribution: ex-dividend price per share on the horizontal axis and counts across 4096 paths on the vertical axis; inputs are drift .06, volatility .2, and one year.

Continued: the same complete example

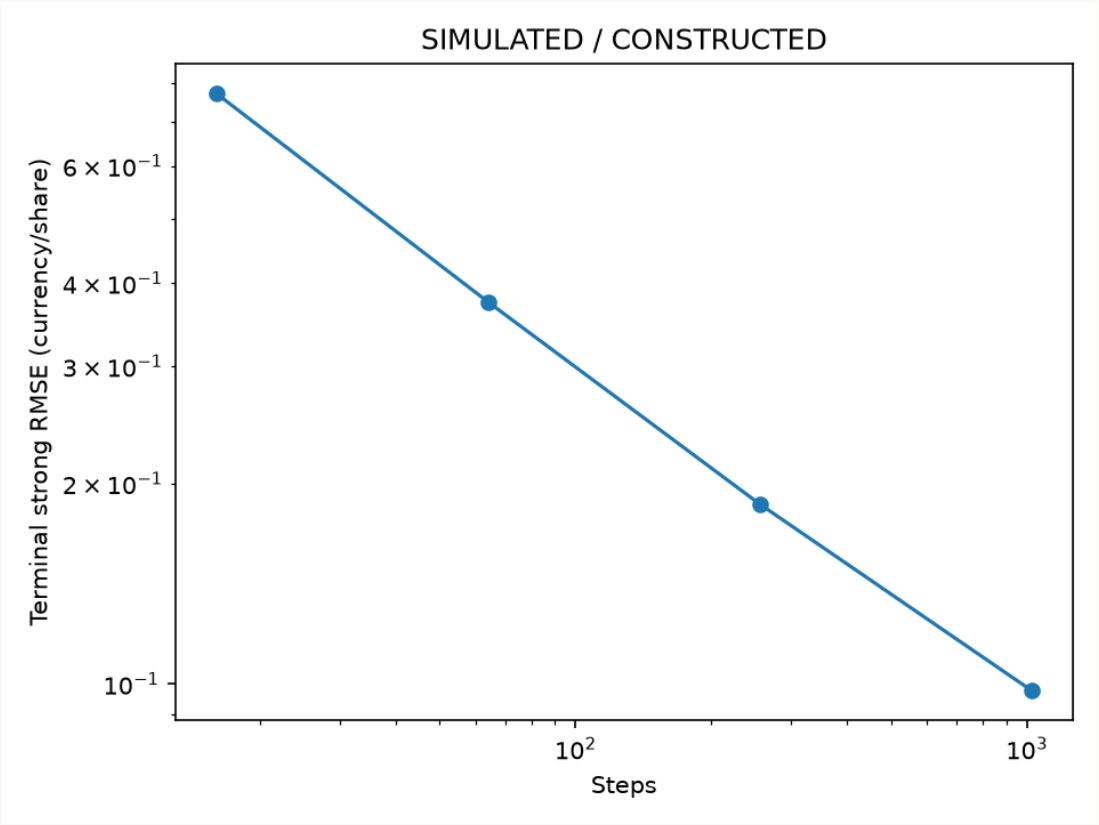


Figure 6: SIMULATED common-noise Euler error: steps on the horizontal axis and strong terminal RMSE in per-share amounts on a log-log scale on the vertical axis; only the scale in this sample is shown.

8.6 Greeks and Step Size

Vega and Rho are reported per unit parameter change; Theta is per calendar year. Centered differences were checked for calls and puts at S=80/100/120 with other inputs fixed. The maximum absolute error across different units is only an execution summary of per-item tolerances; it should not be treated as a single economic distance.

S/type	Delta	Gamma	Vega	Theta	Rho
80/call	0.189494	0.016802	21.506484	-2.528897	13.628799
80/put	-0.790704	0.016802	21.506484	0.658932	-81.494143
100/call	0.586851	0.018951	37.901158	-5.089319	49.458109
100/put	-0.393348	0.018951	37.901158	-2.293569	-45.664833
120/call	0.859931	0.008299	23.900201	-4.282714	79.130557
120/put	-0.120268	0.008299	23.900201	-1.879044	-15.992386

For every scenario and metric, the absolute centered-difference error is below 2.2e−6 in its own units, with h=.0001 and stock-price step $h_S = S \cdot h$. The base call Vega is 37.9011575100, so a 1 pp volatility shock has a first-order price effect of approximately .3790115751. The put Theta at S=80 is positive, so it is not valid to claim that time always erodes every option's value.

Continued: the same complete example

h ratio	Gamma difference	Gamma error
1e-02	0.0189477855	-2.79e-06
1e-04	0.0189505786	-1.20e-10
1e-06	0.0189515958	+1.02e-06
1e-08	0.0355271368	+1.66e-02

The very small h=1e−8 has already lost accuracy; the source code retains this negative diagnostic.

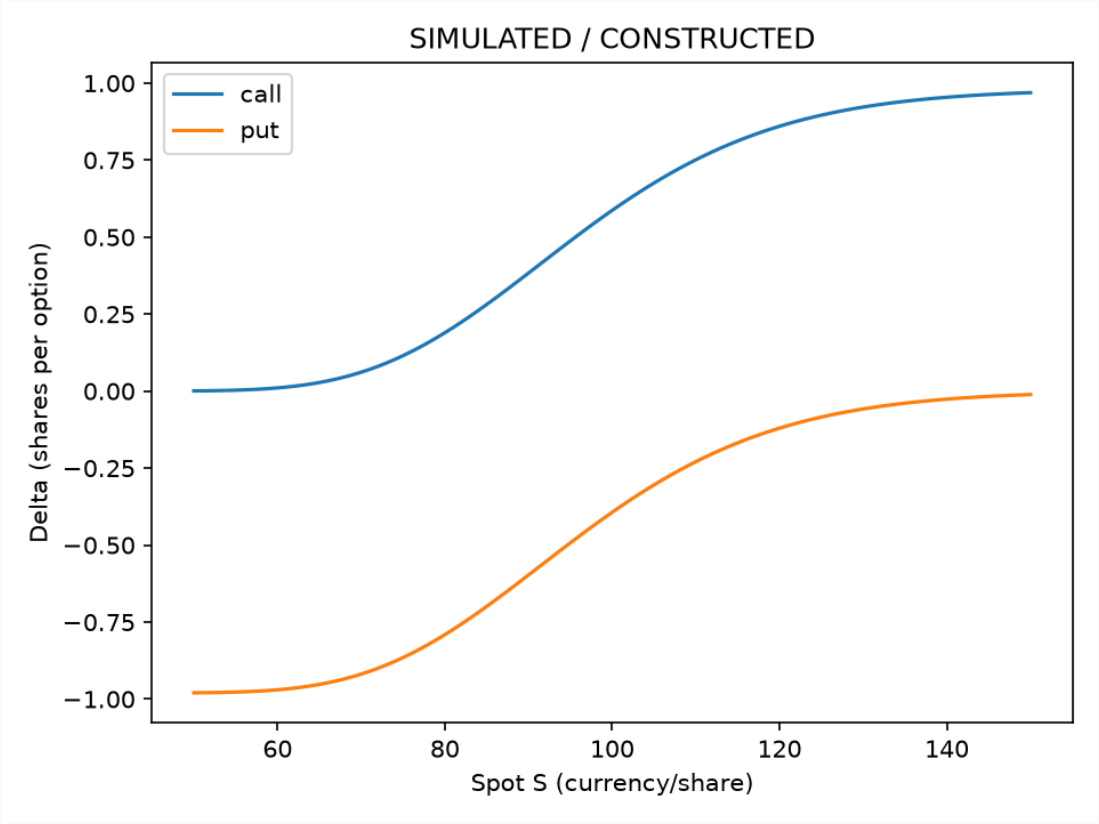


Figure 7: SIMULATED Delta: S per share on the horizontal axis and replicating shares per option on the vertical axis; call and put curves with fixed T/r/q/σ. The chart is not a trading signal.

8.7 Discrete Delta Hedging under P and Fees

Seed 4404, with 256 shared Brownian paths of 3276 fine steps, aggregated to 12/52/252 steps. The P stock-price drift μ−q=.06 is distinguished from the Q pricing drift .03. fee=.001 is a one-sided 10 bp fee, charged at initiation and liquidation; amounts are per share.

Steps/fee rate	Mean error	SD	5th percentile	95th percentile	Mean cash fee
12/0.0	-0.100174	1.982914	-3.299152	+2.940812	0.000000
12/0.001	-0.332425	2.000768	-3.551955	+2.751658	0.226679
52/0.0	-0.053391	0.952187	-1.694724	+1.431930	0.000000
52/0.001	-0.404548	0.977735	-2.202883	+1.122840	0.342607
252/0.0	-0.042639	0.432230	-0.786323	+0.580357	0.000000
252/0.001	-0.669815	0.476814	-1.561916	-0.039341	0.611734

Continued: the same complete example

Without fees, higher frequency lowers the error SD in this sample. With fees, higher frequency produces a more negative mean and larger average fees; the 95th percentile is also negative at 252 steps. This comparison retains replication shortfall and fees and does not promise real-world profitability at any frequency. The difference in mean error includes fee financing drag; total fees are a nominal sum.

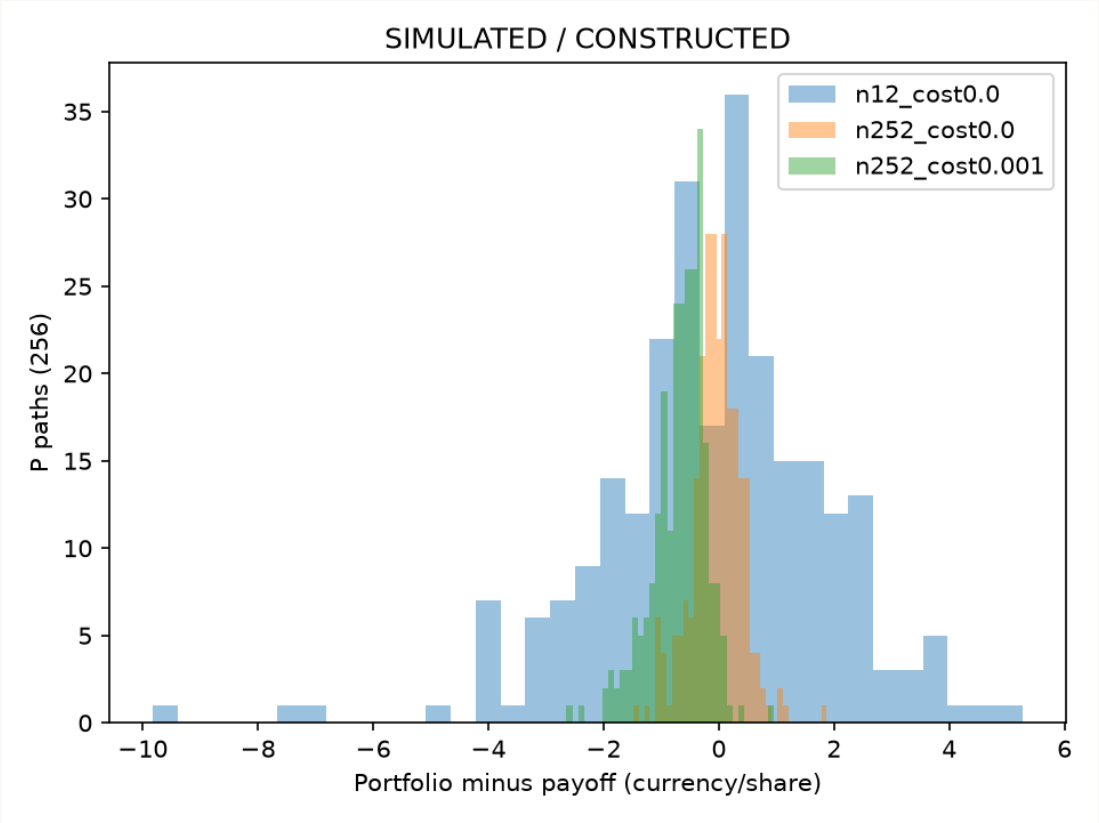


Figure 8: SIMULATED P hedge errors: liquidation portfolio value minus payoff per share on the horizontal axis, and counts across 256 paths on the vertical axis; risk and fees are compared across three frequencies and fee settings.

8.8 Cash and Share Reconciliation in a Complete Ledger

Path 0, 12 steps, fee=.001, with time i/12 years. The tables separate cash flows on the old position from the current rebalance. Unrounded values from the existing independent root run are in `ledger_n12_cost0.001.json`; the archived author payload is separately indexed above. Use the actual floating-point values to reconcile the ledger with the code in §18 rather than requiring 1e−10 agreement from six-digit displayed values.

Continued: the same complete example

i	S	New shares	Interval interest	Interval dividend	Rebalance cash flow
0	100.000000	0.586851	0.000000	0.000000	-58.685115
1	93.634880	0.450214	-0.206750	0.097890	12.793965
2	92.185735	0.407093	-0.153839	0.070318	3.975204
3	96.449460	0.495617	-0.137606	0.062599	-8.538091
4	98.288381	0.532052	-0.173605	0.079736	-3.581155
5	100.746167	0.587785	-0.188964	0.087230	-5.614901
6	108.452054	0.765829	-0.212857	0.098777	-19.309250
7	119.086899	0.927286	-0.294037	0.138542	-19.227440
8	119.499826	0.947869	-0.375048	0.184200	-2.459650
9	118.440463	0.960607	-0.386125	0.188941	-1.508684
10	130.009165	0.996218	-0.393254	0.189783	-4.629671
11	152.859291	0.998335	-0.413453	0.216042	-0.323635
12	149.238140	0.000000	-0.415630	0.254553	148.989617

i	Fee	New cash	Stock market value	Portfolio value	Option value/terminal payoff
0	0.058685	-49.516794	58.685115	9.168320	9.227006
1	0.012794	-36.844483	42.155771	5.311288	5.497746
2	0.003975	-32.956775	37.528141	4.571366	4.476903
3	0.008538	-41.578412	47.801964	6.223552	5.969667
4	0.003581	-45.257016	52.294519	7.037502	6.436110
5	0.005615	-50.979266	59.217090	8.237823	7.293182
6	0.019309	-70.421905	83.055745	12.633839	11.978397
7	0.019227	-89.824067	110.427659	20.603591	20.615407
8	0.002460	-92.477026	113.270211	20.793186	20.628805
9	0.001509	-94.184402	113.774758	19.590356	19.258906
10	0.004630	-99.022174	129.517407	30.495233	30.407686
11	0.000324	-99.543544	152.604737	53.061194	53.020537
12	0.148990	49.136007	0.000000	49.136007	49.238140

Initial cash equals the model price minus the stock purchase and initial fee. Over each interval, the carried share position receives dividends. On the final row, the share count and stock market value are zero, and liquidation cash minus payoff is the path error. Cash fees and financing costs reduce replication funds even when the terminal stock path is unchanged.

9. Python Implementation

Throughout this section, the public pricing parameter T denotes remaining time to maturity τ, in years. For example, if calendar time t=.25 and maturity T_{maturity}=1, pass T=.75 to the API. Theta remains the calendar-time derivative $-V_\tau$; its sign does not change because the API parameter is named T.

Core calculations are in `src/quant_course/stochastic_calculus.py` and `derivatives_pricing.py`, and are reused by the CLI and Notebook. Run the CLI from the course root using the existing .venv. All prices are in currency per share. The output directory must not already exist; if a run fails, retain that directory and continue with a new path.

```
.venv/bin/python code/ch44_stochastic_derivatives.py \
  --output-dir runs/reader_01
.venv/bin/python -m pytest \
  tests/test_stochastic_calculus.py tests/test_derivatives_pricing.py \
  tests/test_ch44_root.py -q
```

The CLI records fixed seeds, Monte Carlo pricing under Q and hedging under P, analytic/tree errors, Greek finite differences, shared Brownian increments, error estimators for every grid, figures, and one complete cash ledger. Run the Notebook from a clean kernel in first-cell order; it calls the core functions and generates PNGs during execution. Its smaller demonstration does not replace the full frozen CLI experiment.

Public pricing functions reject NaN, infinity, non-positive S/K, negative σ/T, and invalid call/put types; r may be negative. The tree rejects invalid probabilities rather than clipping them to 0 or 1. Prices at expiry and at zero volatility use the deterministic payoff. The Greeks API supports T>0 and σ>0; it explicitly rejects inputs at expiry or with zero volatility. At the payoff kink there is no ordinary unique derivative, while some deterministic derivatives can exist away from the kink; this API does not provide them. If extreme parameters exceed floating-point range, the function should raise a clear error rather than return a seemingly valid NaN.

The left-point integration function checks dimensions, shapes, and finite values; it cannot infer the information source of a numerical array. The caller is responsible for proving the integrand is adapted. The GBM function's drift is directly the stock-price drift: pass $\mu-$q=.06 under P or r−q=.03 under Q; the function does not subtract q again.

10. Visualization

The eight generated figures are placed beside the corresponding numerical results in §8, so reading the charts stays connected to the calculations. Inputs, units, random seeds, and sample sizes are described with each figure; the SIMULATED label appears on the charts. Brownian connecting lines do not imply smooth paths, the gray QV line is the population limit, tree-error connecting lines do not imply monotonicity, Monte Carlo error bars are sampling intervals, the Delta curve shows local model sensitivity, and the hedging histogram is under P rather than Q.

The figures can be rebuilt by the standalone CLI in a new directory; retain the output results together with the PNGs.

11. Real Market Example: A Testable Question from the Model

This chapter did not download a real option chain or report real market quotes and model-price differences. Agreement among the three pricing methods inside the model therefore checks only the numerical implementation and the relationship under the stated assumptions. With permission to use the data, you could ask a vendor a practical question: for the same underlying, expiry, strike, currency, timestamp, bid/ask quotes, and dividend curve, compute the implied volatility for each valid quote and test whether a single constant σ can explain them.

A volatility smile or skew means that implied volatility varies with contract characteristics such as strike. It is a pattern in model parameters backed out from prices, not an observed return distribution under the real-world measure, and it cannot be directly interpreted as trading Alpha. Before inverting mid-prices, check the bid/ask range, payoff convention, and dividends. If a constant-σ model shows systematic differences across many strikes, stochastic volatility, local volatility, or jump models may be needed; their hedging performance would still need to be tested.

The original Black–Scholes paper assumes no dividends and no transaction fees; Merton's dividend extension supports the q term in this chapter. Those theoretical sources do not mean that this chapter replicated their market studies. The real-market example remains pending validation with licensed data. Missing evidence includes quote URLs and permissions, download date and version, sample period, synchronized quotes and execution latency, currency-consistent rates and dividends, hashes, and reproducible cleaning. This chapter does not invent a volatility surface or market prices to fill that evidence table.

12. Common Mistakes: Runnable Counterexamples for Diagnosis

Mistake 1: Treating a Brownian path as an ordinary smooth path and calculating $\int W dW = W_T^2/2$, thereby omitting half the quadratic variation. On a finite grid, the correct value is $(W_T^2 - \sum \Delta W^2)/2$, whose limit is $(W_T^2 - T)/2$. The incorrect and repaired code in 44-P01 returns the difference between the two, instead of merely saying "remember randomness."

Mistake 2: Copying the terminal value W_T to every left endpoint as the integrand. The program can multiply the values, but the integrand uses future information; the resulting terminal sum is W_T^2, with expectation T, unlike the zero expectation of an adapted, square-integrable Itô integral. Correct array shape does not fix look-ahead. Perturbing a future suffix provides negative evidence about one code path's information dependence, but finite tests cannot prove that every input is adapted.

Mistake 3: Replacing r−q under Q with μ−q under P and then discounting at r. This computes a discounted expectation under P and generally does not replicate a European payoff. 44-P03 computes both the correct Q model and the incorrect drift construction for comparison; the role of Q follows from the model's assumptions of $\sigma>0$, completeness, and no arbitrage.

Mistake 4: Calculating the SE for correlated antithetic paths as though there were 2M independent samples. In 44-P04, both payoffs are generated from the same z and averaged within each pair first; the correct SE comes from the sample variance of the M pair means. The direction of error in the incorrect SE depends on payoff correlation; this chapter's example must not be stated as a theorem that antithetics reduce SE for every payoff.

Mistake 5: Treating Vega itself as the price change for a one-percentage-point volatility move, or treating Theta with respect to remaining time as Theta with respect to calendar time. 44-P04 provides centered differences in matching units and the 1 pp conversion. A step size that is too small can make Gamma inaccurate through cancellation; one that is too large introduces truncation error. More displayed decimal places do not guarantee greater accuracy.

Mistake 6: Recording only stock rebalancing in the hedging ledger, omitting interest on existing cash or dividends on the existing stock position, or retroactively crediting past dividends

using the new Delta. 44-P05 recomputes the old position's interval cash flows, current rebalance, and initial and liquidation fees row by row; it also shows the difference between the incorrect and correct ledgers. Using a future Delta or omitting fees changes the financial task.

13. Quant Research Application: From Model Checks to a Data Experiment

A research question could be: "Does increasing discrete hedging frequency reduce replication-loss risk after fees?" First fix the underlying/payoff and the sign convention for the error. Use frictionless continuous replication as the benchmark, then freeze the candidate frequencies, one-sided fee, P-path parameters, sample size, and seed. This chapter first examines sensitivity inside the model and retains the mean, SD, percentiles, and fees. A smaller error under one seed does not show that another market environment will behave better.

For real data, split the timeline into an estimation period for volatility, dividends, and fees, and a later period that is evaluated only once. This ensures every hedge uses only information available at that time. Record each bid/ask and execution eligibility, cash rates and borrowing spreads, settle the same fixed payoff, and compare with simple baselines such as no hedge, fixed Delta, and low-frequency Delta. Report pricing error, hedging risk, and net trading-strategy return separately.

Specify possible disconfirming outcomes in advance: a constant σ cannot explain the quote range; jump or tail replication shortfall exceeds the risk limit; fees from higher frequency offset the error reduction; or synchronized/licensed data are insufficient for reproduction. Any of these could prompt a revision to the model or execution assumptions. If parameters are changed after viewing the later period, label the analysis post hoc rather than rewriting it as preregistered. The results in this chapter deliver only the simulated teaching candidate described above, not a completed real-derivatives project acceptance.

14. Chapter Summary

The two-state replication equations give the risk-neutral probability; the information set constrains usable dynamic positions; Brownian quadratic variation explains the second-order Itô term; and GBM combines these conditions into a positive-price path. Continuous self-financing replication eliminates the real-world drift and leaves a PDE involving r, q, and σ. The risk-neutral conditional expectation connects the analytic solution to numerical methods.

In practice, first check the payoff and units, then the model and information conditions, and then choose the analytic formula, tree, or Monte Carlo method. Check Greeks with finite differences, calculate SE using the independent sampling unit, and check hedging error with a complete cash ledger. Numerical agreement supports the implementation; real quotes, costs, and jumps still determine the model's application boundary.

15. Exercises

44-E01: Scaling and the Scope of the Proof

For a fair-coin random walk over one year with N steps, let the step size be either $\xi/\sqrt{\text{N}}$ or ξ/N. Find the terminal variance in each case. What distribution does a Brownian difference quotient follow? Explain why this rules out a finite derivative in probability at a fixed time, and why it does not by itself prove almost-sure nondifferentiability everywhere. Derive the expected QV and its variance bound for a deterministic grid.

44-E02: Zero Unconditional Mean Does Not Imply a Martingale

Let ξ be fair ± 1 and define $X_0=0$, $X_1=\xi$, $X_2=-\xi$. Write the three means and $E[X_2 \mid \mathcal{F}_1]$, then determine whether X is a martingale. Next change X_2 to $\xi+\eta$, where η is an independent fair ± 1, and determine whether it is a martingale. Besides adaptation at each time, what measurability and integrability conditions does a continuous Itô integrand require?

44-E03: Finite Grid and Limit

For the path W=[0,.2,.1,.4], calculate the sum of squared increments, the left-point sum, and the right-point sum. Use telescoping sums to derive the general expressions for both sums. For Brownian motion with T=1, explain which equality holds only in the limit when QV is replaced by 1.

44-E04: GBM and Common Noise

Under P, let μ=.08, q=.02, σ=.2, and S_0=100. Find the stock-price drift, log-price drift, $E[S_T]$, $\text{Var}(S_T)$, and their expressions at T=1. Derive the Euler recursion and give a negative-price counterexample. Write the strong terminal RMSE and the paired estimator for the weak error with g(S)=S. Must the weak error be monotone across grids?

44-E05: Measure Change and Dividend-Adjusted Martingales

With the same parameters and r=.05, find λ, Z_t, W_Q, and the Q stock-price drift. Show that Novikov's condition holds over a finite horizon, then write the drift of the discounted ex-dividend price, the discounted return process including cash dividends, and the wealth process with dividends reinvested. Explain why the λ formula cannot be used when σ=0.

44-E06: One-Period Replication

Let S_0=K=100, S_u=120, S_d=80, the bank-account growth factor be 1.05, and q=0. Find the replicating share position, initial cash/bond position, call price, and auxiliary risk-neutral probability; check both states. How does the replication price change if the real-world probability of an up move is .3?

44-E07: Self-Financing and the PDE

Starting from V=ΔS+A, the dividend-financing flow in dA, and Itô's formula for dV, derive the PDE including q term by term. Explain why applying the ordinary product rule directly to ΔS would incorrectly treat rebalancing as an external contribution. State the European call/put terminal payoff and the σ>0 domain, as well as the borrowing and trading assumptions.

44-E08: d1/d2 and Conditional Pricing

Starting from the normally distributed log terminal price under Q, separately integrate $E_{\mathbb{Q}}[1_{\{S_T>K\}}]$ and $E_{\mathbb{Q}}[S_T 1_{\{S_T>K\}}]$ and explain the distinct roles of d2 and d1. Write put-call parity and the conditions for the restricted Feynman–Kac connection. Explain why the same formula cannot simply be evaluated using the P drift.

44-E09: Greek Units

Define Delta, Gamma, Vega, Theta, and Rho, with Theta measured in calendar time and Vega/Rho per unit parameter change. State the q-inclusive analytic relationships for calls and puts. How is a 1 pp volatility shock converted? Is the ordinary Delta unique at expiry when S=K? Why can a very small centered-difference step become inaccurate?

44-E10: Standard Error for Antithetic Monte Carlo

Given M independent z_i and correlated antithetic payoffs Y_i^+, Y_i^-, write the price estimate, pair-mean variance, SE, and approximate 95% normal CI. Explain why 2M paths cannot be treated as 2M independent observations, and what is established when an analytic price falls inside one interval.

44-E11: Information Groups and Weighted Quadratic Variation (New)

Using the four-terminal-state table in §5.2, fill in M_2 and the two $\mathcal{F}_1$ conditional means. Explain why M_2 is not $\mathcal{F}_1$-measurable while its conditional expectation is. For a fixed deterministic partition and a left-point-measurable C_i with $|C_i| \le K$, show the conditioning step that makes the i<j cross term vanish and derive the upper bound for $E[R_\Pi^2]$. What is the bound for T=1, four equal steps, and K=2? Does this prove the result for arbitrary random partitions or all remainder terms in the general Itô formula?

44-E12: Cash Transfer, Explicit Solution, and Time Convention (New)

With S=100, zero fees, and no interval return, let the share position change from .6 to .5 while cash is A=−50. Calculate total value before and after rebalancing term by term. Write the cash-offset identity for a smooth Delta in this chapter. For the explicit GBM solution, write F_t, F_w, and F_{ww} and verify the SDE and positivity. If calendar time is t=.25 and maturity is one year, what T should the pricing API receive? Give the units of λ.

16. Programming Exercises

44-P01: Finite Integral and Information Diagnostics

Generate 16 Brownian paths with 1024 steps using seed 4402. Check the maximum residual in the finite identity, the difference from the T limit, and that changing a future suffix does not affect the first-half integral. Then calculate the anticipating sum H=W_T, and show its nonzero mean theoretical reference and failure of the isometry. Do not label this process a valid Itô integrand. Empty arrays and NaN must be rejected.

44-P02: Nested Common Increments

Following the frozen protocol, aggregate 4096 fine paths of 1024 increments to grids of 16/64/256/1024 steps. Calculate the strong terminal RMSE between exact GBM and Euler, the paired weak mean difference and SE, and compare the analytic Euler mean with the exact mean. If independent noise is incorrectly used for the comparison, state what quantity the difference estimates. Do not use monotonicity of the weak sample mean as an acceptance criterion.

44-P03: Three Prices and Rejection Boundaries

Check the base call/put, parity, multiple CRR grids, and Monte Carlo with Q drift. Construct the incorrect discounted expectation that uses $\mu-q=.06$ and compare amounts using the same z. Test that invalid tree probabilities are rejected rather than clipped, negative rates are allowed, and prices at expiry/zero volatility are correct.

44-P04: Greeks and Correlated-Sample SE

For calls and puts at S=80/100/120, perform centered differences in matching units and scan Gamma error at h=.01/.0001/.000001/.00000001. Using 1000 independent z values with seed 4401, recompute the antithetic pair SE and the incorrect flattened SE and report both. The Greeks API must explicitly reject T=0/σ=0; with one independent random Monte Carlo observation, SE cannot be estimated and must be rejected.

44-P05: Reconcile the Hedging Ledger Row by Row

For the path [100,110,105,90], recompute each row for a one-year, three-step call with r=.05, q=.02, σ=.2, K=100, and fee=.001: financing on old cash, dividends on old shares, the number of shares and cash flow at rebalancing, one-sided fees, new cash, stock market value, final liquidation, and the payoff difference. Reconcile cash and shares and show the error caused by omitting dividends. Changing a future suffix must not alter the completed ledger prefix.

17. Challenge Problems

44-C01: Look-Ahead and Model Boundaries

Prerequisite: the fourth moment of a normal distribution. For fixed $H_t = W_T$, find the limit, mean, and second moment of the discrete sum and compare with $E\int H^2\,dt$. Explain why this does not establish a real-world arbitrage. Grading: 3 points for the algebra, 3 for the fourth moment/isometry, and 4 for the information and financial boundary.

44-C02: Disconfirming the Claim That Higher Hedging Frequency Is Better

Compare error SD, lower-tail percentiles, and mean fees for 12/52/252 steps, with and without fees, using the frozen P-path results. Propose a real-quote test plan with separated training and evaluation periods and two conditions that would disconfirm "higher frequency is better." Grading: 3 points for error/cost and units, 3 for a reproducible timeline/data-permission plan, and 4 for negative results and the boundary around post hoc exploration. This open-ended problem does not have one correct answer based only on which frequency has the highest mean.

18. Solutions / Solution Hints

44-E01

The terminal variances are N/N=1 for the first step size and N/N²=1/N for the second. The Brownian difference quotient has distribution N(0,1/h). For fixed M, the probability that it lies in [−M,M] tends to zero, ruling out convergence in probability to a finite random variable; proving nondifferentiability everywhere almost surely still requires control over all times. Independent squared increments give $E[QV] = T$ and $\mathrm{Var}(QV) = 2\sum_i \Delta t_i^2 \le 2T|\Pi|$, hence L² convergence. The CLT gives terminal and finite-dimensional distributions; weak convergence of paths additionally requires a functional CLT and tightness.

44-E02

All three means are zero. Since ξ is known at $\mathcal{F}_1$, the conditional expectation is $-\xi$, which is not ξ, so this is not a martingale. With X_2=ξ+η, the conditional mean is ξ because the independent η has mean zero; the conditional-mean relation at t=0 also holds. An Itô integrand must, for example, be progressively measurable or predictable and satisfy $E\int H^2\,dt < \infty$. Continuous adapted processes meet the measurability requirement; the stated square-integrability condition must still hold; "the left column of an array" does not establish the required information property.

44-E03

The increments are [.2,−.1,.3], QV=.04+.01+.09=.14, the left-point sum is $0\cdot .2+.2\cdot(-.1)+.1\cdot .3$=.01, and the right-point sum is $.2\cdot .2+.1\cdot(-.1)+.4\cdot .3$=.15. In general, the left sum is $(W_T^2 - W_0^2 - QV)/2$, while the right sum is $(W_T^2 - W_0^2 + QV)/2$; they differ by QV. For Brownian motion started at 0, the left sum tends to $(W_T^2 - 1)/2$ only when QV tends to T=1. The constructed sequence has terminal value .4 and QV=.14; neither is claimed to be a necessary Brownian outcome.

44-E04

β=μ−q=.06 and the log-price drift is $\beta-\sigma^2/2$=.04. $E[S_1] = 100e^{.06}$ and $\mathrm{Var}(S_1) = 10000e^{.12}(e^{.04}-1)$. Applying Itô's formula to log price gives the first-order term β and second-order term $-\sigma^2/2$ in the explicit solution; moments of the price follow from the exponential moments of a normal variable. The Euler update is $S_{i+1} = S_i(1+\beta dt+\sigma dW_i)$. With β=0, σ=1, dt=1, and dW=−2 it gives −100, a separately constructed boundary counterexample. With common noise, $D_m = S_T^E - S_T^{exact}$, strong RMSE=$\sqrt{}$mean(D²), weak mean difference=mean(D), and SE=std(D,ddof=1)/$\sqrt{}$M. Sampling error can make the weak estimate nonmonotone; the population mean bias can be calculated directly as $100(1+.06/N)^N - 100e^{.06}$.

44-E05

λ=(.08−.05)/.2=.15, $Z_t = \exp(-.15W_P(t) - .01125t)$, and $W_Q = W_P + .15t$. The Novikov expectation exp(.01125T) is finite for each fixed finite T; completing the square in the constant-increment density gives N(−λdt,dt), and the shifted process is Brownian. Subtracting $\sigma\lambda$=.03 from the P stock-price drift .06 gives the Q drift r−q=.03. The drift of the discounted ex-dividend price is $-qe^{-rt}S$; the gain process including cash dividends has drift 0. With dividends reinvested, the share count is exp(qt), and discounted wealth exp(−(r−q)t)S has drift 0. Under the finite-horizon GBM moment conditions it is a true martingale. When σ=0, the denominator is zero, so the λ formula does not define this measure change; the deterministic price must be handled separately.

44-E06

Subtracting the two state equations gives Δ=.5. In the down state, .5 · 80+1.05A=0, so A=−40/1.05; if Bo=1, the bond units are also A. The up-state value is 60−40=20 and the down-state value is 40−40=0, so the initial price is 50−40/1.05. The auxiliary probability is p=(1.05−.8)/(1.2−.8)=.625; its discounted expected payoff, (.625 · 20+.375 · 0)/1.05, equals the replication price. Changing the real-world up probability to .3 does not change the replicating payoff or cost; different state values or financing assumptions would require a new calculation.

44-E07

The self-financing gain equation is $dV = \Delta(dS + qSdt) + r(V - \Delta S)dt$. Substituting $\Delta = V_S$ and matching it with the Itô expansion in §6.7 eliminates $\mu S V_S$ from both sides and gives the PDE including q. The identity V=ΔS+A does not allow writing dV as only ΔdS while omitting cash, or treating S dΔ as external wealth. Self-financing requires stock rebalancing to be funded through the cash account; the gain equation is the replication condition. The call terminal payoff is (S−K)^+ and the put is (K−S)^+. The stochastic PDE applies for S>0, τ>0, and σ>0. The assumptions of no fees, continuous trading, same-rate borrowing/lending, and admissible strategies are listed in §5.4.

44-E08

Under Q, $\log(S_T/S) = (r - q - \sigma^2/2)\tau + \sigma\sqrt{\tau}Z$, and exercise occurs when $Z > -d2$, so $Q(S_T > K) = \Phi(d2)$. Completing the square in the truncated stock expectation shifts the threshold from −d2 by $\sigma\sqrt{\tau}$ under the exponential weight, giving $Se^{(r-q)\tau}\Phi(d1)$, where d1=d2+$\sigma\sqrt{\tau}$. Discounting these two terms separately gives the call. Put-call parity, $Se^{-q\tau} - Ke^{-r\tau}$, follows from the payoff difference $S_T - K$. For this chapter's positive S, constant-coefficient GBM, finite T, linearly growing call/put payoff, and $C^{1,2}$ solution for τ>0, Itô's formula and the integrable-martingale conditions connect the PDE to the Q conditional expectation. The chapter does not claim this for arbitrary irregular coefficients or payoffs. The P drift generally retains the risk premium and cannot replace the Q pricing task.

44-E09

Delta=$\frac{dV}{dS}$ (shares per option); Gamma=$\frac{d^2V}{dS^2}$ (change in Delta per unit S); Vega=$\frac{dV}{d\sigma}$ (currency per unit annualized-volatility parameter); Theta=$\frac{dV}{dt} = -\frac{dV}{d\tau}$ (currency per year); and Rho=$\frac{dV}{dr}$ (currency per unit continuously compounded annual rate). The analytic formulas and derivations are in the Greeks subsection of §6. Calls and puts have the same Gamma and Vega, and $\Delta_{\text{call}} - \Delta_{\text{put}} = e^{-q\tau}$. The signs of Theta and Rho terms involving q and r depend on whether the payoff is a call or a put. For a volatility change $\Delta\sigma$=.01, the first-order amount is .01 · Vega; this is not Vega itself. At expiry and S=K, the call's one-sided Deltas are 0 and 1, so there is no ordinary unique value. The API's rejection of all Greeks when T=0 or σ=0 defines its implementation domain; it does not claim that all derivatives away from the payoff

kink are nonexistent. Centered-difference subtraction of nearby values magnifies rounding error, and Gamma is especially sensitive because it is then divided by h_S^2.

44-E10

Let $X_i = (Y_i^+ + Y_i^-)/2$. The price is mean(X), $s_X^2 = \sum_i (X_i - \text{mean}(X))^2/(M-1)$, $SE = s_X/\sqrt{M}$, and the approximate 95% interval is mean(X)±1.95996398454SE. The variance is $\text{Var}(X) = [\text{Var}(Y^+) + \text{Var}(Y^-) + 2\,\text{Cov}(Y^+, Y^-)]/4$. Dependence within each pair means the formula for 2M independent observations omits the covariance term. The normal approximation requires independent pairs, a sufficiently large sample, and finite variance; one random observation cannot estimate a sample variance. One interval covering the analytic price is a random outcome of the interval procedure, not proof that the algorithm is correct. Algorithm checks also need financial identities, an independent recomputation, and boundary checks.

44-E11 (New Full Solution)

The four values of M_2 are 2, 0, 0, and −2; the conditional means for the two $\mathcal{F}_1$ groups are 1 and −1. Within each group M_2 takes different values, so it is not yet measurable; the conditional expectation is constant within each group and therefore measurable. Let $\varepsilon_i = (\Delta W_i)^2 - \Delta t_i$. Its conditional mean is 0 and its conditional second moment is $2(\Delta t_i)^2$. For i<j, $C_i\varepsilon_i C_j$ is $\mathcal{F}_{t_j}$-measurable, so the cross expectation is $E[C_i\varepsilon_i C_j E(\varepsilon_j \mid \mathcal{F}_{t_j})] = 0$. Hence $E[R_\Pi^2] = 2\sum_i E[C_i^2](\Delta t_i)^2 \le 2K^2T|\Pi|$. For the stated values, the bound is $2 \cdot 4 \cdot 1 \cdot .25{=}2$. This is an upper bound on expected squared error, not a pathwise error bound, and equality is not required. Refining the grid gives convergence in L^2. The result does not cover arbitrary random partitions; the general Itô formula still requires approximation, remainder control, and localization.

44-E12 (New Full Solution)

The initial stock value is 60, cash is −50, and total value is 10. Selling .1 share brings in 10; stock value becomes 50, cash becomes −40, and total value remains 10. For a smooth semimartingale Delta, $dA = rA\,dt + q\Delta S\,dt - S\,d\Delta - d[\Delta, S]$; adding this to the product formula gives $dV = \Delta\,dS + q\Delta S\,dt + rA\,dt$. For the exponential solution F, its three derivatives are $(\beta{-}\sigma^2/2)$F, σF, and σ^2F. The Itô drift adds σ^2F/2 back, giving βF; the stochastic term is σF. The initial value is S_0 and the solution is strictly positive. For the API, T=1−.25=.75 years. λ has units year$^{-1/2}$, so λW and λ^2t are dimensionless.

44-P01 Reference Implementation

```
import numpy as np
from quant_course.stochastic_calculus import brownian_increments, ito_left,
    ↪ quadratic_variation
dw = brownian_increments(16, 1024, 1., 4402)
w = np.column_stack([np.zeros(16), np.cumsum(dw, axis=1)])
qv = quadratic_variation(w)
i = ito_left(w, w[:, :-1])
ordinary = .5 * w[:, -1] ** 2
print("ordinary minus Ito first path", ordinary[0] - i[0])
assert np.allclose(ordinary - i, .5 * qv)
assert np.max(abs(i - .5 * (w[:, -1] ** 2 - qv))) < 1e-12
print("finite/limit RMSE", np.sqrt(np.mean((i - .5 * (w[:, -1] ** 2 - 1)) ** 2)))
future = w.copy()
future[:, 513:] += 3
assert np.array_equal(ito_left(w[:, :513], w[:, :512]), ito_left(future[:, :513],
    ↪ future[:, :512]))
lookahead = np.repeat(w[:, -1, None], 1024, axis=1)
wrong = ito_left(w, lookahead)
assert np.allclose(wrong, w[:, -1] ** 2)
print("anticipating first path", wrong[0])
for invalid in [np.empty((0, 2)), np.array([[0., np.nan]])]:
    try:
        quadratic_variation(invalid)
    except ValueError:
        pass
    else:
        raise AssertionError("invalid input accepted")
```

The correct finite algebraic difference is affected only by rounding; the ordinary chain rule exceeds the left sum by QV/2. The theoretical mean of the anticipating sum is $E[W_T^2]$=T rather than 0; its second moment is $3T^2$, while the isometry's right side is T^2. The sample mean across 16 paths need not equal T exactly.

44-P02 Reference Implementation

```
import math
import numpy as np
from quant_course.stochastic_calculus import brownian_increments, gbm_exact,
    ↪ gbm_euler
dw = brownian_increments(4096, 1024, 1., 4403)
for n in [16, 64, 256, 1024]:
    d = dw.reshape(4096, n, 1024 // n).sum(axis=2)
    ex = gbm_exact(100., .06, .2, 1/n, d)
    eu = gbm_euler(100., .06, .2, 1/n, d)
    error = eu[:, -1] - ex[:, -1]
    rmse = np.sqrt(np.mean(error**2))
    mean = error.mean()
    se = error.std(ddof=1) / np.sqrt(4096)
    population = 100 * ((1 + .06/n)**n - math.exp(.06))
    print(n, rmse, mean, se, population)
```

If independent noise is drawn for the two methods, the difference still estimates the difference between their means without bias, but its SE is larger; its pathwise RMSE is no longer the strong discretization error measured with common noise. This exercise requires aggregation of shared fine increments and retains the nonmonotone weak sample result; the four means need not be strictly monotone.

44-P03 Reference Implementation

```
import numpy as np
from quant_course.derivatives_pricing import bs_price, crr_price, mc_price
p = dict(S=100., K=100., T=1., r=.05, q=.02, sigma=.2)
c, put = bs_price(**p), bs_price(**p, kind="put")
assert abs(c - put - 100*np.exp(-.02) + 100*np.exp(-.05)) < 1e-10
print("BS", c, put)
for n in [10, 25, 50, 51, 100, 101, 200, 400]:
    print(n, crr_price(**p, steps=n) - c)
z = np.random.default_rng(4401).standard_normal(50000)
correct = np.exp(-.05) * np.maximum(100*np.exp(.01+.2*z)-100, 0)
wrong = np.exp(-.05) * np.maximum(100*np.exp(.04+.2*z)-100, 0)
print("wrong drift minus Q plain estimate", wrong.mean() - correct.mean())
assert bs_price(**dict(p, T=0)) == 0
assert bs_price(**dict(p, r=-.01)) >= 0
assert mc_price(**dict(p, sigma=0))["se"] == 0
try:
    crr_price(**dict(p, r=1., sigma=.01), steps=1)
except ValueError:
    pass
else:
    raise AssertionError("invalid probability clipped")
```

44-P04 Reference Implementation

```
import numpy as np
from quant_course.derivatives_pricing import bs_price, bs_greeks, mc_price
p = dict(S=100., K=100., T=1., r=.05, q=.02, sigma=.2)
for spot in [80., 100., 120.]:
    for kind in ["call", "put"]:
        a = dict(p, S=spot)
        g = bs_greeks(**a, kind=kind)
        h, hs = .0001, .01
        def v(key, bump):
            inputs = dict(a)
            inputs[key] += bump
            return bs_price(**inputs, kind=kind)
        fd = {
            "delta": (v("S", hs)-v("S", -hs))/(2*hs),
            "gamma": (v("S", hs)-2*bs_price(**a, kind=kind)+v("S", -hs))/hs**2,
            "vega": (v("sigma", h)-v("sigma", -h))/(2*h),
            "theta": -(v("T", h)-v("T", -h))/(2*h),
            "rho": (v("r", h)-v("r", -h))/(2*h),
        }
```

```
        for key in g:
            assert abs(fd[key]-g[key]) < 2e-5
        print(spot, kind, "Vega per 1pp", .01*g["vega"])
for h in [.01, .0001, .000001, .00000001]:
    hs = 100*h
    gamma = (bs_price(**dict(p, S=100+hs))-2*bs_price(**p)+bs_price(**dict(p, S=100-
    ↪ hs)))/hs**2
    print("h/Gamma error", h, gamma-bs_greeks(**p)["gamma"])
z = np.random.default_rng(4401).standard_normal(1000)
y = np.exp(-.05)*np.maximum(100*np.exp(.01+.2*z)-100, 0)
ya = np.exp(-.05)*np.maximum(100*np.exp(.01-.2*z)-100, 0)
x = (y+ya)/2
se = x.std(ddof=1)/np.sqrt(1000)
assert np.isclose(se, mc_price(**p, pairs=1000)["se"])
print("paired SE / wrong flat SE", se, np.r_[y, ya].std(ddof=1)/np.sqrt(2000))
for a in [dict(p, T=0), dict(p, sigma=0)]:
    try:
        bs_greeks(**a)
    except ValueError:
        pass
    else:
        raise AssertionError("unsupported boundary Greek accepted")
try:
    mc_price(**p, pairs=1)
except ValueError:
    pass
else:
    raise AssertionError("one random observation reported SE zero")
```

The reference implementation checks each of the five Greeks and gives the Vega conversion, Gamma step-size diagnostic, and correlated-sample Monte Carlo diagnostic. The core tests and CLI use the same unit contract.

44-P05 Reference Implementation

```
import math
import numpy as np
from quant_course.derivatives_pricing import bs_price, bs_greeks, delta_hedge
path = np.array([100., 110., 105., 90.])
r, q, sigma, fee, dt = .05, .02, .2, .001, 1/3
result = delta_hedge(path, 1., r, q, sigma, 100., cost_rate=fee)
cash = bs_price(100., 100., 1., r, q, sigma)
shares, total, cash_no_dividend = 0., 0., cash
for i, row in enumerate(result["ledger"]):
    if i:
        cash = cash*math.exp(r*dt) + shares*path[i-1]*math.expm1(q*dt)
        cash_no_dividend *= math.exp(r*dt)
    target = 0. if i == 3 else bs_greeks(path[i], 100., 1-i*dt, r, q, sigma)["delta"]
    change = target-shares
    cost = fee*abs(change)*path[i]
    cash -= change*path[i]+cost
    cash_no_dividend -= change*path[i]+cost
    shares = target
```

```
    total += cost
    assert abs(cash-row["cash"]) < 1e-10
    assert abs(shares-row["delta"]) < 1e-12
    assert abs(cash+shares*path[i]-row["portfolio_value"]) < 1e-10
assert abs(total-result["total_cost"]) < 1e-12
assert abs(cash-max(path[-1]-100, 0)-result["error"]) < 1e-10
print("missing dividend cash error", cash_no_dividend-cash)
changed = path.copy()
changed[-1] = 150.
prefix = delta_hedge(changed, 1., r, q, sigma, 100., cost_rate=fee)["ledger"]
assert prefix[:-1] == result["ledger"][:-1]
```

Dividends are approximated using the old share count and the left-endpoint stock price over each interval; financing is applied to old cash before rebalancing at the current price. The final target is 0, so liquidation incurs a fee. The incorrect `cash_no_dividend` ledger follows the same specified share positions but omits dividends, producing a cash difference; comparing only total stock return would miss it.

44-C01 Reference Argument

The discrete sum is $W_T \sum_i \Delta W_i = W_T^2$ because W_0=0. Its mean is T and its second moment is $E[W_T^4] = 3T^2$, whereas $E \int H^2\, dt$=T², so the Itô isometry does not apply. For t<T, H is not $\mathcal{F}_t$-measurable; the constructed position knows the terminal value in advance and is not an admissible strategy in this chapter. Real arbitrage requires available information, tradable assets, self-financing, and admissible strategies; the counterexample violates the first condition. Mentioning more general non-adapted integrals is optional; Malliavin calculus or other advanced theory is not required for credit.

44-C02 Reference Argument and Rubric

Read the frozen results in §8 and report error SD, the 5th percentile, and cash fees on the same 256 paths. Higher frequency may reduce frictionless error inside the model, but the fee-inclusive tail and mean need to be compared; simultaneous improvement is not guaranteed. Estimate volatility, fees, and dividends on a training period, then compare only in a later evaluation period under fixed quote, timing, and execution conventions. Give the data URL, permissions, version/hash, timezone, and next eligible execution time. If fees at higher frequency enlarge the replication-shortfall tail, or if every frequency breaches the risk limit under jumps, that disconfirms "higher frequency is always better." It is also possible that quotes cannot be synchronized and the result remains unresolved. Mark parameter changes as post hoc and retain negative results. The chapter's simulation does not substitute for completion of a real project.

19. Further Reading

Must Know: Review Taylor expansions and centered differences from Chapter 4, conditional expectation from Chapter 6, and payoffs and total returns from Chapters 10/11. For this chapter, focus on one-period replication, the second-order Itô term, self-financing with q, P/Q, and paired Monte Carlo SE.

Should Know: MIT 15.070J Lectures 15–17 on Itô integrals/formula and Lecture 19 on self-financing help check the left-point convention and square-integrability assumptions. Read the Black–Scholes (1973) no-dividend model assumption and Merton (1973) §7 dividend extension at the cited locations.

Advanced: CMU Risk Neutral Measures on Girsanov/Novikov and risk-neutral pricing. Measure theory, martingale representation, general Feynman–Kac conditions, and American-option free boundaries require additional prerequisites; this chapter does not provide complete proofs of those general theorems.

Optional: Hull, *Options, Futures, and Other Derivatives*, and Shreve, *Stochastic Calculus for Finance I/II*, are useful for a full course. The exact edition's copyright page was not checked, so no page numbers are invented. Before studying local/stochastic volatility and jumps, be able to independently reconcile the cash ledger in this chapter.

20. References

The key sources and the locations actually read for this chapter are listed in `source_notes.md`; the global source ledger is in references/sources.json. The original papers support theoretical claims. The inputs, outputs, code hashes, and run records for the chapter's simulations are a separate layer of evidence.

- CH44_BS1973: Black & Scholes, 1973, original paper hosted by Princeton courses, JPE 81(3), 637–654, DOI 10.1086/260062. Supports the no-dividend original model assumption and dynamic replication.
- CH44_MERTON1973: Merton, 1973, original paper hosted by Boston University, Bell J. Econ. Manag. Sci. 4(1), 141–183, DOI 10.2307/3003143, §7 equations 42–44 and footnote 62 support the dividend-inclusive PDE and European call.
- CH44_MIT_ITO: MIT 15.070J Fall 2013 lecture notes, Lectures 14–17 and 19; exact ranges are in the reading ledger.
- CH44_CMU_RNM: CMU Risk Neutral Measures, Chapter 4, Theorem 1.6 and Remark 1.9 support the density sign and Novikov condition.
- CH44_NUMPY_NORMAL: NumPy Generator.normal, supports the loc/scale/shape interface.
- CH44_HIGHAM2001: Desmond J. Higham (2001), original paper hosted by a university, SIAM Review 43(3), 525–546, DOI 10.1137/S0036144500378302; §§4–5 describe Euler, shared Brownian increments, and strong/weak error definitions. The original strong-error experiment uses L1; this chapter's RMSE is separately derived analytically.
- CH44_SCIPY_NORM: SciPy norm, supports the numerical standard-normal CDF/PDF interface.

Chapter 45 — Production Quant Pipelines

Learning thread: use the same offline constructed trades to connect five gates: data, timing, execution, reconciliation, and recovery. Check engineering facts, research design, and real-market evidence separately. Core thread: build a deterministic offline production quant system with strict point-in-time isolation, idempotent reruns, local transactional event accounting, residual reconciliation, and fault recovery. Prerequisites: Chapter 3, Chapter 29, Chapter 33, Chapter 35, and Chapter 38. After completing this chapter, you should be able to design a five-gate architecture, write a deterministic offline production pipeline with restart recovery and discrete reconciliation, and verify a causal timeline without look-ahead as well as ledger conservation.

1. Learning objectives

1. Computational and engineering objective: implement a deterministic quant production pipeline fully decoupled from external network dependencies. Given two constructed security assets (Security A and Security B) and initial cash of 10,000.000 USD, complete data validation, momentum-feature extraction, discrete target-share generation, simulated order matching, cash and position ledger updates, portfolio NAV calculation, and multidimensional reconciliation on a standard discrete-time grid.
2. Conceptual and theoretical objective: clarify how production systems differ from offline research prototypes. Master the five-gate architecture: the data gate (schema and finite-value checks); the timing gate (point-in-time partial-order constraints and holding-period isolation); the execution gate (limits, emergency halts, and idempotent matching); the reconciliation gate (cash, positions, gross and net returns, and fees); and the recovery gate (restart recovery, fault recovery, and atomic publication).
3. Fault and diagnosis objective: use the single recovery case that runs throughout the chapter (Asset A fills and is stored before a process interruption, while Asset B has not filled) to understand the state machine that prevents duplicate fills on rerun, protects the historical ledger, completes reconciliation, and atomically publishes results. Understand the boundary between single-filesystem atomic replacement with `os.replace` and SQLite local transactions: they are not atomic across resources.
4. Deliverable and completion criteria: connect the plan's 23 topics and the controlled recovery case into a traceable workflow; verify the hand calculation to three decimal places, with a cash reconciliation tolerance of 10^{-6} USD; and provide complete derivations, runnable verification code, and reference answers in Section 18 for every exercise (concept questions 45-E01–E06, programming questions 45-P01–P04, and challenge questions 45-C01–C02).

2. Why a quant needs this

Quant researchers often explore strategies in interactive Notebooks. Notebooks provide immediate feedback but lack production-grade constraints: cells can run out of order, variables share global memory, data loading can depend on dynamic network access or unversioned temporary files, and missing or exceptional values can be silently filled. When this research code goes straight into automated execution, it often exposes problems such as these:

1. Scheduling and retry jitter: a scheduled job (Cron or Airflow) can trigger the same logical-period task more than once after a timeout or network interruption. Without idempotency protection, the system may send the same-direction order to the market again within a short time and double the position.
2. Clock and information leakage: if the pipeline reads the machine's current wall clock directly while a market-data provider has push delays, code may mistake historical data for the latest quote. In a backtest, using adjusted prices that include future revisions or financial data obtained before disclosure can create irreproducible, fictitious performance.
3. Inconsistent state across systems: if a system crashes after sending a request to a trading interface but before persisting the local ledger, a restart without recovery reconciliation may be unable to determine the preceding order's status, creating a risk of uncontrolled orders.

A 2013 administrative order by the SEC records a deployment and control failure. On August 1, 2012, Knight's SMARS routing system sent millions of orders in about 45 minutes, generating more than 4 million executions across 154 stocks. The order records a final loss exceeding $460 million. The legacy Power Peg function could still be called, new code was missing from one server, and executed quantities were not correctly passed to that path. Section 11 uses this record to explain version checks and independent limits; it is not empirical evidence about a strategy in this course. Sources: SEC Release No. 34-70694, page 2 §1 and pages 5–6 §§12–17.

Data contracts must also reflect the value ranges of different financial instruments. For example, because of limited liability, ordinary listed equity shares have a theoretical and market price lower bound of 0. Physically delivered commodity futures, however, can settle below zero when delivery and storage costs are high; the May 2020 WTI crude oil contract settled at a negative price. A data system without an explicit domain contract can distort downstream model inputs by blindly assuming prices must be positive or by taking the absolute value of every negative observation.

The long-run robustness of a quant strategy depends not only on mathematical assumptions but also on the determinism, auditability, and recovery capacity of the engineering pipeline that runs on unreliable physical infrastructure.

3. Intuition

Think of a production quant pipeline as an assembly line with five independent verification gates connected in sequence.

The data gate checks fields, units, and values. After it passes, the timing gate confirms that decision information was available. The execution gate checks risk and existing fills first. The reconciliation gate recomputes cash and positions. The recovery gate checks the identity of the publication. Each gate preserves its failure reason. The five items below explain the input, action, and next step after failure, forming one workflow.

1. Gate One (Data Gate): validate input schema completeness, strong types, and numeric bounds. Prices must be finite positive numbers (or valid values under the instrument contract), the unit must be explicitly stated as USD/share, and share quantities may be nonnegative real numbers. Invalid values stop the run; implicit imputation is prohibited.
2. Gate Two (Timing Gate): enforce point-in-time (PIT) partial-order rules. The physical event time, external publication time, local ingestion time, and business snapshot time for decision inputs must all be no later than the signal time. Execution and matching must occur after the signal time.
3. Gate Three (Execution Gate): calculate the difference using the persisted state database. If the target number of shares for an asset is already fully met, the increment is 0; after an interruption, rerun only the unfinished difference. If a risk limit or circuit breaker is triggered, stop execution immediately and preserve the event.
4. Gate Four (Reconciliation Gate): reconstruct the internal ledger after matching and calculate fill notional, embedded spread cost, explicit commission, ending position market value, and portfolio NAV. Reject the run if the absolute difference between externally reported cash and internally calculated cash exceeds the floating-point tolerance (10^{-6} USD).
5. Gate Five (Recovery Gate): write calculated outputs to a separate staging directory with a run sequence (`staging_<run_key>_<attempt>`). Publication follows one of three explicit paths:
   - Path 1 (not yet published): the final directory does not exist. After the staging directory passes validation, publish it with `os.rename` on the same filesystem as the final directory and update the state to `COMPLETED`.
   - Path 2 (already published and valid): the final directory exists and passes hash validation, but the database registration is missing. Record `COMPLETED` directly; never trade again.
   - Path 3 (published but damaged): the final directory exists but fails hash validation. The system must reject explicitly and raise an alert, preserving all damaged evidence for investigation. It must never overwrite the directory or hide the damage with a retry.

Teaching boundary: this pipeline uses the Python standard library's JSON/CSV, `hashlib`, and `sqlite3`, and is limited to a single machine and process. It provides an engineering foundation for understanding the boundary between local database transactions and atomic filesystem commits, which are not atomic across resources.

4. Mathematical foundations

4.1 Discrete time axis and point-in-time partial order

Define a discrete time axis $\mathcal{T} \subset \mathbb{R}$. Represent every timestamp as a UTC ISO-8601 string and map them strictly monotonically. For each asset i in the security set $\mathcal{A} = \{A, B\}$, an input record is associated with the timestamp tuple

$$(t_{\text{event}}, t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}, t_{\text{signal}}, t_{\text{trade},i}, t_{\text{mark}})$$

and includes the permanent security identifier `permanent_security_id`, the preceding-price timestamp, and the data revision timestamp `revision_time`.

The timestamps mean:

- t_{event}: when the physical market event occurred (for example, the matching time of a trade).
- $t_{\text{available}}$: when an external provider made the data publicly available.
- t_{ingested}: when the local system received and persisted the data.
- $t_{\text{as_of}}$: the cutoff time for the business information snapshot, not a general legal effective date.
- t_{signal}: when the strategy model calculates features and generates target positions.
- $t_{\text{trade},i}$: when asset i is actually filled in the simulation.
- t_{mark}: the accounting period's asset valuation and closing time.

The necessary temporal gating condition is

$$\max(t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}) \le t_{\text{signal}} < t_{\text{trade},i} \le t_{\text{mark}}, \quad \forall i \in \mathcal{A}$$

Note: satisfying this temporal inequality is necessary to avoid look-ahead bias, but it is not sufficient. If a data source makes a backdated historical revision after t_{signal} (`revision_time` $> t_{\text{signal}}$) and the system has not retained historical snapshots, look-ahead leakage can still occur.

Place the trade clock from Section 8 and the label clock from §5.8 on the same timeline. All times in the table are UTC. The label becoming available only at 17:00 is a delayed version of the existing monitoring example; it does not change the execution configuration.

Time	Facts in this example	Permitted action
09-01 16:00 to before 16:05	Quote event occurs; available, ingested, and as_of times are no later than 16:05	Check the version and freshness available at that time; no positions are held yet
09-01 16:05	Signal generated	Use only inputs available before the signal to generate targets; do not use next-day trade or mark prices
09-02 09:00 / 09:01	A / B fill respectively	Each holding interval starts after that asset's own fill
09-02 16:00	Mark valuation; return-label interval ends	Close the books; the interval end satisfies only the label-maturity threshold
09-02 17:00	Delayed label actually becomes available and is ingested	Only after this time may its error enter monitoring; it cannot be updated at 16:30

Read §6.3 alongside this table: price changes before a fill are not holding-period returns. Then read §5.8: an interval ending does not mean the label has already reached the local system. The timing gate must also retain historical versions; the ordering in the table does not replace the check for future revisions above.

4.2 Discrete fills, embedded costs, and cash-ledger dynamics

The ledger derivation in this chapter uses the following explicit assumptions:

- Assumption A1: opening portfolio holdings are 0 (zero initial positions), so $H_{i,0} = 0$.
- Assumption A2: all orders are buy-only spot orders with full fills and no partial fills. The configured share quantity is a finite positive real number; if there is no positive signal, the target is 0. The worked example uses integers, but imposes no broker lot-size rule.
- Assumption A3: the account has one currency (USD) and no external cash flows.
- Assumption A4: cash deposit interest, financing interest, and securities-borrow costs are all 0.
- Assumption A5: market impact cost is 0.

Let opening cash be $\text{Cash}_0 \in \mathbb{R}^+$ (unit: USD). Asset i has holdings $H_{i,t} \in \mathbb{R}_{\geq 0}$ (unit: share). At trade time $t_{\text{trade},i}$, the market mid-price is $P_{\text{mid},i}$ (unit: USD/share). The buy/sell half-spread is $\phi_i \geq 0$ (a dimensionless proportion; 10 bps corresponds to $\phi_i = 0.0010$), and the explicit commission rate is $c_{\text{rate},i} \geq 0$ (a dimensionless proportion; 10 bps corresponds to $c_{\text{rate},i} = 0.0010$).

For a buy order with requested quantity $\Delta Q_i > 0$:

1. Fill price:
$$P_{\text{fill},i} = P_{\text{mid},i} \cdot (1 + \phi_i)$$
2. Actual fill notional (unit: USD):
$$N_i = \Delta Q_i \cdot P_{\text{fill},i} = \Delta Q_i \cdot P_{\text{mid},i} \cdot (1 + \phi_i)$$
3. Explicit commission (unit: USD):
$$C_i = N_i \cdot c_{\text{rate},i} = \Delta Q_i \cdot P_{\text{fill},i} \cdot c_{\text{rate},i}$$
4. Embedded spread cost (unit: USD):
$$\begin{aligned} S_i &= \Delta Q_i \cdot (P_{\text{fill},i} - P_{\text{mid},i}) \\ &= \Delta Q_i \cdot P_{\text{mid},i} \cdot \phi_i \end{aligned}$$
5. Total friction cost for one fill (unit: USD):
$$\text{Cost}_i = S_i + C_i$$
6. Cash-ledger deduction:
$$\Delta\text{Cash}_i = -(N_i + C_i) = -(\Delta Q_i \cdot P_{\text{fill},i} + C_i)$$
The spread cost S_i is already reflected in the fill price being higher than the mid-price. When notional N_i is deducted from cash, the spread is already embedded in that deduction. Never deduct S_i from cash a second time! Cash after the trade:
$$\text{Cash}_k = \text{Cash}_{k-1} + \Delta\text{Cash}_i$$

Position update:

$$H_{i,k} = H_{i,k-1} + \Delta Q_i$$

4.3 Asset valuation and the NAV conservation identity

At closing time t_{mark}, the market mark for asset i is $P_{\text{mark},i}$. Total ending market value of holdings (unit: USD):

$$V(\mathbf{H}, \mathbf{P}_{\text{mark}}) = \sum_{i \in \mathcal{A}} H_i \cdot P_{\text{mark},i}$$

Total portfolio net asset value (NAV, unit: USD):

$$\text{NAV} = \text{Cash} + V(\mathbf{H}, \mathbf{P}_{\text{mark}})$$

Portfolio gross PnL (using the mid-price at the time of the fill as the buy-price benchmark; unit: USD):

$$\text{GrossPnL} = \sum_{i \in \mathcal{A}} H_i \cdot (P_{\text{mark},i} - P_{\text{mid},i})$$

Total friction cost (unit: USD):

$$\text{TotalCost} = \sum_{i \in \mathcal{A}} \text{Cost}_i = \sum_{i \in \mathcal{A}} (S_i + C_i)$$

Net PnL (unit: USD):

$$\text{NetPnL} = \text{GrossPnL} - \text{TotalCost}$$

The ledger conservation identity (it holds exactly under assumptions A1–A5):

$$\text{NAV} - \text{Cash}_0 \equiv \text{NetPnL}$$

Note the difference in dimensions: USD PnL and NAV are absolute currency amounts (USD), not dimensionless rates of return. Cash reconciliation uses a floating-point tolerance of $\epsilon_{\text{cash}} = 10^{-6}$ USD. When checking share counts, the permitted position tolerance is $\epsilon_{\text{share}} = 10^{-6}$ share.

First connect the units with the same Asset A trade, then close the portfolio ledger with the Asset B trade in Section 8:

Step	Calculation in this example	Output units
Mid-price to fill price	$100 \times (1 + 0.0010) = 100.100$; 10 bps = 0.0010	USD/share; rate is dimensionless
Notional to cash	$10 \times 100.100 = 1001.000$; commission 1.001; cash change -1002.001	share × USD/share = USD
Mark to gross PnL	A: $10 \times (102 - 100) = 20$; B: $20 \times (49 - 50) = -20$	USD; portfolio gross PnL is 0
Gross to net PnL	$0 - (1 + 1 + 1.001 + 1.001) = -4.002$	USD; spread appears in the cost breakdown and is not deducted from cash again
Cash to NAV	$7995.998 + 2000 = 9995.998$; NAV $- 10000 = -4.002$	USD; this is not a decimal return

If net return normalized by opening cash is needed, this example gives $-4.002/10000 = -0.0004002$, or -0.04002%; it cannot be compared directly with the 10^{-6} USD amount tolerance. Use 10^{-6} share for quantity checks and 10^{-6} USD for amount checks.

4.4 Idempotency operator and execution state machine

Let the system's target vector be $\mathbf{Q}^* \in \mathbb{R}^M_{\geq 0}$. Assume input identity (`run_key` and input fingerprint) is fixed and fills recorded in the state database have been durably confirmed, with $0 \leq Q_{i,\text{filled}} \leq Q_i^*$. The order-increment operator produced by the execution engine is

$$\Delta\mathbf{Q} = \max(\mathbf{0}, \mathbf{Q}^* - \mathbf{Q}_{\text{filled}})$$

The system makes no new matching call if and only if $\Delta\mathbf{Q} = \mathbf{0}$. Applying the increment state transition twice in succession gives

$$\Gamma(\Gamma(s, \mathbf{Q}^*), \mathbf{Q}^*) = \Gamma(s, \mathbf{Q}^*)$$

Boundary condition: this idempotency proof depends strictly on confirmed fills in the local state database. It does not mean that an external broker naturally guarantees exactly-once execution when the network times out or an order's status is unknown. External idempotency depends on broker-side order-deduplication identifiers and active status queries.

4.5 Data lineage and cache fingerprint function

Define a hash fingerprint function for configuration, data, and code:

$$\begin{aligned}&\text{Fingerprint}\\&= \text{SHA256}\Big(\text{ConfigBytes} \,\|\, \text{InputBytes} \,\|\, \text{CodeHash}\Big)\end{aligned}$$

This equation illustrates lineage; it is not the exact character-by-character definition of the actual normalized serialization. Actual identity also includes the pipeline version and hashes of three reused historical modules; see Section 5.4. `CodeHash` includes the core module's source identity. Note: serializing a configuration dictionary as canonical JSON (sorted keys and compact separators) and hashing it does not make the runtime Python `dict` immutable. Preventing in-place changes still depends on coding conventions or an immutable mapping type.

5. Formal definitions: connect the five gates into an interpretable workflow

Section 4 gives the timing and cash identities. This section uses the same constructed task to explain why directories, files, state, and monitoring are kept separate. Each technique must answer three questions: what does it receive, how does it change the current decision, and what evidence does it leave for checking? The accepted engineering example uses local Python, CSV/JSON, and SQLite; optional-platform examples explain boundaries only. This bilingual revision inherits the existing evidence and does not rerun the course.

5.1 Project layers, installable packages, and cold-start imports

Project layering addresses the question, "Can the script still calculate the same ledger if a Notebook cell is removed?" The actual call graph is: `code/ch45_production_quant.py` and `capstone/run_offline.py` read parameters and call `src/quant_course/production_pipeline.py`; that core reuses historical cost-ledger and risk functions. The Notebook also calls the same core, and tests call the same public entry point. The directory has at least these roles:

```
pyproject.toml                  build backend, package discovery, and project
    ↪ dependency configuration
src/quant_course/production_pipeline.py   features, state, simulated ledger
code/ch45_production_quant.py   standalone command-line interface and companion plots
capstone/configs/offline_constructed.json   constructed configuration
notebooks/ch45_production_quant.ipynb      explanation, calls, and actual plots
tests/test_production_pipeline.py         counterexamples and boundaries
```

A source layout puts the importable package under `src/`. It is normally found by the environment after package installation; this batch reuses the existing virtual environment, and the entry point explicitly adds the course's `src` directory to the module search path, without reinstalling the package. From the course root, a reader can start a new process with `.venv/bin/python code/ch45_production_quant.py --help`; the import does not depend on memory from the previous Notebook session. A cold start rebuilds these dependencies instead of treating variables from an old kernel as inputs.

An installable package also needs build rules. Python Enhancement Proposal (PEP) 517 specifies how a build frontend calls a backend; PEP 518 specifies how build dependencies are declared in `pyproject.toml`. Neither requires a directory named `src`. This project's actual `[build-system]` selects `setuptools.build_meta`, and `[tool.setuptools]` maps package directories to `src`. The first answers "who builds?"; the second answers "where does package discovery start?" The distribution name `quant-course-a1` is also different from the import name `quant_course`. These boundaries make code identity traceable before the data gate; a successful import alone does not prove that a wheel was built or all dependencies were locked. The src-layout section of PyPA's documentation and the local configuration support this distinction.

5.2 Understand row-oriented and column-oriented reads with a four-row table

Comma-separated values (CSV) is suitable for the few records in this chapter that can be read by eye. Consider four constructed quotes that are already available; the final column is a teaching-source label and adds no real market data:

Asset	Event date	Price (USD/share)	Source label
AssetA	08-31	90	constructed
AssetB	08-31	45	constructed
AssetA	09-01	100	constructed
AssetB	09-01	50	constructed

To answer "What was the price on 09-01?", a CSV reader scans the four rows in order, parses the date, keeps the two matching rows, and selects the price column. Parquet, a columnar-storage format, organizes columns into column chunks and divides them into row groups; a query engine can use file metadata to locate the columns it needs. Projection pushdown passes the request to "select only the price column" down to the read layer. Filter pushdown passes the date condition down as well; if available statistics show that a row group's maximum date is 08-31, the engine may skip that group. If statistics are missing or the filter is unsupported, it still has to read the group; skipping is not guaranteed every time.

To calculate the read volume by hand, suppose each field uses 8 bytes and ignore metadata, compression, and parsing costs. This is only a layout model, not the actual size of a CSV or Parquet file. The whole table is $4 \times 4 \times 8 = 128$ bytes. Reading only the price column is $4 \times 1 \times 8 = 32$ bytes. If the two-row older group is then successfully skipped, the data portion is $2 \times 1 \times 8 = 16$ bytes, plus the cost of reading statistics metadata. These three intermediate values explain "what data can be skipped"; they are not a performance benchmark.

DuckDB is an in-process analytical query engine, while Parquet is a file format; they occupy different layers. The optional SQL `SELECT asset,current_price FROM read_parquet( 'quotes.parquet') WHERE event_time <= ...` shows how to pass the timing-gate condition to the engine. There is no actual `quotes.parquet` fixture, and DuckDB/pyarrow are absent from this batch, so this statement was not run. Actual acceptance still reads the small JSON/CSV inputs and explicitly checks sets, units, and timestamps. Missing packages do not prevent understanding column projection, and they certainly do not establish correct point-in-time handling: a table with future revisions is still not an input that was available at the time, however quickly it can be read.

5.3 Learn SQL, keys, and transaction boundaries from two fills

Structured Query Language (SQL) lets us retrieve facts from a persistent ledger with explicit conditions. The actual primary key of `fills` is the text `fill_id`, not an auto-incrementing sequence. Its fields are `asset`, `signed_quantity`, `fill_price`, and `commission`. The two rows from the default run are:

Asset	signed_quantity (share)	fill_price (USD/share)	commission (USD)
AssetA	10	100.1	1.001
AssetB	20	50.05	1.001

Filter by `run_key` first, count unique fills, and then aggregate cash flows: A's outflow is $10\times100.1+1.001 = 1002.001$; B's is $20\times50.05+1.001 = 1002.001$. Total outflow is 2004.002 USD, leaving cash of $10000-2004.002 = 7995.998$ USD. The accepted I3 PATH09 chain is shown below in full: start the same constructed CLI once from the root of the final complete course package, then pass its `state_db` and `run_key` to a read-only query. First save the following Python block as `print_examples/I2-PATH-09.py`; it matches the I3 script with the same name. These instructions apply to a complete course package containing the shared scripts and configuration. If an entry is absent from a working checkout, consult its explicitly labeled read-only source. This is a usage example, not a record of execution in this revision.

```
run="runs/path09_$(.venv/bin/python -c 'from uuid import uuid4; print(uuid4().hex)')"
state_db="$run/state.db"
run_key="reader"
.venv/bin/python capstone/run_offline.py --config capstone/configs/
    ↪ offline_constructed.json --run-key "$run_key" --state-db "$state_db" --output-
    ↪ dir "$run/artifacts"
.venv/bin/python print_examples/I2-PATH-09.py "$state_db" "$run_key"
```

```
"""Read-only cash check for the same CLI state database and run key."""
from pathlib import Path
import sqlite3,sys
state,key=Path(sys.argv[1]),sys.argv[2]
assert state.is_file(),"Complete the same constructed CLI first"
with sqlite3.connect(state.resolve().as_uri()+"?mode=ro",uri=True) as db:
    count=db.execute("SELECT COUNT(*),COUNT(DISTINCT fill_id) FROM fills WHERE
    ↪ run_key=?",(key,)).fetchone()
    spent=db.execute("SELECT SUM(signed_quantity*fill_price+commission) FROM fills
    ↪ WHERE run_key=?",(key,)).fetchone()[0]
assert count==(2,2)
assert spent is not None and abs(spent-2004.002)<=1e-6
assert abs((10000-spent)-7995.998)<=1e-6
print(count,spent,"USD; final cash",10000-spent)
```

A unique key can reject duplicate inserts, but a key alone is not enough: recovery must also check that fill times, quantities, fees, and order match the frozen target. Atomicity, consistency, isolation, and durability (ACID) describe database transactions. This core commits fill records first, then writes events and state separately; it cannot claim that fills and all audit events share one transaction. Recovery treats committed `fills` as the source of truth; a missing log row must

never cause cash to be deducted again. `BEGIN IMMEDIATE` is an optional SQLite mode that requests a write transaction early, and it can still encounter `SQLITE_BUSY`; this batch does not treat it as a distributed lock or turn the system into a multi-process trading platform. Directory publication uses a different resource and requires the three recovery checks in Section 6.4.

5.4 Configuration, lineage, cache, and the same logical task

Configuration separates initial cash of 10000 USD, a 10 bp spread, a 0.001 commission rate, and risk limits from the logic. A basis point (bp) is multiplied by 10^{-4} to convert it to a decimal return; this unit conversion is not the 10^{-6} USD amount-reconciliation tolerance. The configured share quantity may be a finite positive real number. The actual target is 0 for nonpositive momentum; 10 and 20 are integer values in this example, but 10.5 is also allowed. There is no broker lot-size or minimum-quantity check.

Data lineage associates normalized inputs, configuration, core source, and hashes of reused historical modules with the result. Sort JSON keys before serialization and calculate a SHA-256 identity; an input changed from 100 to 101, or a commission changed from 0.001 to 0.002, is a different logical input. A JSON hash does not make a Python dictionary immutable; the entry point must still avoid passing the same mutable object to a step that changes it.

Idempotency compares identity under the same `run_key`. The first constructed task writes two fills; if the scheduler triggers the same task again, A10 and B20 are already filled, the remaining amount is 0, and there are no new fills. If the key is the same but the input fingerprint differs, reject it and preserve `RUN_KEY_FINGERPRINT_MISMATCH`. A source repair also changes `code_hash` and the fingerprint; this is an expected identity change, and must not be described as byte-for-byte identical to the old revision. Cache validity must also be checked against the result hash anchored in SQLite; a mutable `checksums.json` inside the directory cannot independently prove its own integrity. The three publication-recovery cases are described in Section 6.4; a damaged published result cannot be hidden by creating a new attempt.

5.5 Scheduling, external APIs, and unknown outcomes

Scheduling answers when a task is triggered; idempotency answers what happens after a duplicate trigger. Suppose the task's logical date is 09-02, it is scheduled for 09:00, but queue backlog delays its actual start to 09:00:03, and a retry starts it again at 09:00:05. Both processes belong to the same logical key; different wall-clock seconds do not make them separate trading tasks. Apache Airflow's logical date usually represents the start of a data interval and differs from wall-clock execution time. `sched` is an in-process event queue and cannot replace a persistent scheduler or a market trading calendar. No scheduler was installed in this batch.

Retries of an Application Programming Interface (API) call also involve external side effects. The broker-interface scenario below was not run and does not mean this chapter sent an order:

Order	Evidence received	Permitted conclusion	Action
1	Client sends a request with a unique order ID	Request was sent; fill status is unknown	Record the identity awaiting confirmation
2	Wait times out and no confirmation arrives	Unknown does not mean unfilled	Stop new orders and query the original order's status
3a	Query confirms that 10 shares filled	The original side effect occurred	Reconcile these 10 shares; do not resend the full quantity
3b	Query remains unavailable	Status remains unknown	Continue preserving evidence and keep orders stopped

Reads such as data retrieval can be safely retried with a finite retry count and backoff. A write with side effects cannot be treated the same way. Offline simulation has no such network uncertainty. `after_first_fill` is a controlled interruption after the first simulated fill has been committed; it does not simulate an HTTP timeout. If B fills first, it still leaves only the first B fill. If B is the only asset with a target, recovery adds zero fills; if no target is positive, preflight rejects this fault configuration. `risk_before_second_fill` requires at least two positive targets, otherwise there is no second fill to intercept; explicitly rejecting the configuration reveals the test boundary more clearly than silently completing it.

5.6 Structured logs, metrics, and experiment tracking

Structured logging turns "where did it fail?" into searchable fields. In this example, actual events are in SQLite: `event_time` is the time the record was written, `run_key` identifies the logical task, `event_name` identifies the branch, and `details_json` contains the asset, `fill_id`, or exception reason. For example, a risk stop after the first fill preserves both `FILL_COMMITTED` and `RISK_LIMIT_EXCEEDED`; an independent terminal reconciliation difference records `TERMINAL_RECONCILIATION_FAILED` and does not publish a summary. Record time is not quote `event_time`, and a missing log entry does not mean no fill occurred. CLI stdout is an actual JSON summary; it did not create or start a `pipeline.log` service.

A production application can use the standard-library `logging` module to separate a library's logger from the application's handler. The following example prints one credential-free teaching event in the current small process; the actual core events remain in SQLite as described above:

```
import io
import json
import logging
from uuid import uuid4

stream = io.StringIO()
logger = logging.getLogger("ch45.teaching." + uuid4().hex)
logger.propagate = False
logger.setLevel(logging.INFO)
handler = logging.StreamHandler(stream)
logger.addHandler(handler)
logger.info(json.dumps({"run_key": "constructed", "event_name": "STOPPED",
                        "reason": "reconciliation", "contains_secret": False}))
assert json.loads(stream.getvalue())["event_name"] == "STOPPED"
print(stream.getvalue().strip())
```

Monitoring metrics compress many events. A counter accumulates attempts or unique simulated fills; a retry may increase attempts from 1 to 2 while unique fills remain at 2. A gauge can rise or fall; examples are cash of 7995.998 USD or A holdings of 10 share. A histogram accumulates observation counts by bucket: for teaching latencies of 0.02, 0.07, and 0.20 seconds and boundaries 0.05, 0.10, and $+\infty$, the cumulative counts are 1, 2, and 3. Their sum is 0.29 seconds, and their mean is $0.29/3 = 0.09667$ seconds. Bucket counts lose exact ordering information, so these three counts do not reveal an exact P99; the open-ended final bucket also gives no finite upper bound for P99. These are hand-calculated explanations of metrics that were not deployed.

Experiment tracking links metrics back to versions: record the run key, input/config/code hashes, state, and artifact path. Both the successful example with net PnL of -4.002 and a failed `STOPPED` run belong in the run index; do not record only profitable or completed tasks. MLflow can manage such runs, parameters, metrics, and artifacts, but this batch did not call its API or start a service. Data-gate rejection, reconciliation stops, and model alerts are different causes. Monitoring curves are interpretable only when the underlying events retain those distinctions.

5.7 Containers, configuration, and the lifecycle of secrets

A container packages the user-space runtime and entry point into image configuration; it does not remove differences in the host kernel or hardware. The actual `capstone/Dockerfile` skeleton is:

```
FROM python:3.14-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PYTHONPATH=/app/src
WORKDIR /app
RUN python -m pip install --no-cache-dir numpy pandas scipy matplotlib
COPY src/ /app/src/
COPY capstone/ /app/capstone/
COPY code/ /app/code/
ENTRYPOINT ["python", "capstone/run_offline.py"]
CMD ["--help"]
```

Read it step by step: FROM selects a base image; WORKDIR sets the relative path; RUN installs the reused core dependencies in a hypothetical build; COPY includes only the listed course directories; ENTRYPOINT points to the real Capstone entry point; and CMD displays help by default. If it is built later, the build context should be the course root, and `docker build -f capstone/Dockerfile .` will find `src`; `capstone/.dockerignore` is not the effective root ignore file for that course-root context. No image was built and no container was run here. RUN depends on package availability, so the offline data example must not be described as an offline image build.

This base tag and the unlocked pip versions can change over time, so they do not prove that the image is reproducible by digest. A future release should record the image digest, dependency lock, and platform conditions; this batch does not install another environment. Secrets management must also distinguish build-time and runtime: Docker's documentation says build ARG/ENV values persist in the image and should not carry build secrets; temporary build-secret mounts have a separate lifecycle. At runtime, a controlled credential service or secret mount can provide least-privilege values. Environment variables also require attention to access, redaction, and rotation. This example has no credentials, and the configuration snapshot and logs contain no secret values. Checking the configuration version during a model rollback does not "roll back" trades that have already occurred.

5.8 Model drift: calculate the statistic first, then determine when labels are available

A model-drift investigation begins with the input marginal distribution $P(X)$; it cannot directly establish that the conditional relationship $P(Y \mid X)$ or alpha has deteriorated. The two-sample Kolmogorov–Smirnov test (KS) statistic is $D = \sup_x |F_0(x) - F_1(x)|$. An empirical cumulative distribution function (ECDF) is "the number of samples no greater than x divided by the total number of samples." For a baseline $(1, 2, 3, 4)$ and a new sample $(2, 3, 4, 5)$, at x=1 the ECDF values are 1/4 and 0/4; at x=2 they are 2/4 and 1/4; and at x=4 they are 4/4 and 3/4. The largest difference is D=0.25, which is dimensionless. SciPy's official-documentation assumptions of independent samples from continuous distributions do not directly apply to this four-row teaching sample with autocorrelation and tied discrete values. We calculate D only; we do not report a classical p-value or financial significance.

The Population Stability Index (PSI) freezes bins first, then compares proportions. Take baseline counts $(3, 4, 3)$ and new counts $(2, 5, 3)$ from ten constructed observations. Thus $B = (0.3, 0.4, 0.3)$ and $A = (0.2, 0.5, 0.3)$. For positive proportions, $\text{PSI} = \sum_k (A_k - B_k) \ln(A_k/B_k)$. The first bin is $(-0.1)\ln(2/3) = 0.0405465$, the second is $0.1\ln(5/4) = 0.0223144$, the third is 0, and the total is 0.0628609, dimensionless. MathWorks' official formula supports this definition; MATLAB was not run in this example. If a bin is zero, the formula may be undefined or diverge. State the bin-merging or smoothing policy and its effect in advance; do not insert a number after the fact to hide an anomaly. The empirical thresholds 0.1 or 0.25 are not universal guarantees of significance, and this example does not automatically accept a model based on either one.

The Page–Hinkley method (PH) monitors changes in the mean of errors. Let $e_t = |y_t - \hat{y}_t|$; both y_t and its prediction are decimal returns, so e_t, the tolerance offset δ, and the alert threshold h are also decimal returns, never a USD ledger tolerance. For hand calculation, this chapter uses a PH-style fixed-baseline one-sided cumulative rule: $S_0 = M_0 = 0$, $S_t = S_{t-1} + e_t - \mu_0 - \delta$, $M_t = \min(M_{t-1}, S_t)$, with an alert when $S_t - M_t > h$. The cumulative/running mean uses every error through t: $\bar{e}_0 = 0$ and $\bar{e}_t = \bar{e}_{t-1} + (e_t - \bar{e}_{t-1})/t$. For the corresponding one-sided cumulative variant, let $V_0 = 0$, update only $V_t = V_{t-1} + e_t - \bar{e}_t - \delta$ at each step, then calculate $V_t - \min_{0 \le i \le t} V_i$. The current $\bar{e}_t$ is used only in the current increment; previously stored increments retain the $\bar{e}_i$ used at their own times. Do not substitute the final $\bar{e}_t$ into all historical increments. This is a separate, explicitly defined variant of the frozen-μ_0 rule. Compare sensitivity only under specified error paths, thresholds, and calibration conditions; there is no universal performance claim.

A cumulative mean is not a fixed-window moving mean. For $(0.10, 0.10, 0.10, 0.30)$, the fourth-step cumulative mean is $(0.10 + 0.10 + 0.10 + 0.30)/4 = 0.15$; the mean of the most recent two periods is $(0.10 + 0.30)/2 = 0.20$. The two-period window is a terminology counterexample only; this chapter neither teaches nor requires implementation of a third PH algorithm. The official River implementation also has a forgetting parameter and a two-sided option. It was not installed or called, and this example is not identical to its complete algorithm.

Freeze $\mu_0 = 0.10$, $\delta = 0.01$, and h=0.05. Suppose mature errors arrive in order as 0.10, 0.15, and 0.18. Step one: $S = -0.01$, $M = -0.01$, $S - M = 0$. Step two: $S = 0.03$, $M = -0.01$, $S - M = 0.04$. Step three: $S = 0.10$, $M = -0.01$, $S - M = 0.11$; only then does it exceed the teaching threshold. If a forecast is made at 09-01 16:05 and its return label interval ends at 09-02 16:00, that error cannot be computed at 09-02 15:59. It can be updated online after 16:00 once

the label interval has ended and the label is in hand. If the label is not published or ingested until 17:00, the system must wait until 17:00. Online updates after maturity and offline replay are both possible; future returns cannot be used early.

These statistics are intermediate evidence for "save feature and mature-label versions → calculate → check for data faults or economic change → decide whether to stop orders or roll back compatibly." Fixed thresholds must be calibrated on permitted historical blocks; they cannot be tuned by looking at final test performance. Automatic retraining is outside this batch's authorization. Paper trading usually uses virtual funds to inspect trading feedback; this batch has only offline constructed matching, with no real paper interface and no evidence of actual alpha.

6. Derivations

6.1 Derivation of discrete accounting, embedded spread, and net-asset reconciliation

Under assumptions A1–A5, we give a strict algebraic derivation.

Let opening cash be Cash_0, with initial holdings $H_{i,0} = 0$. At time $t_{\mathrm{trade},i}$, buy ΔQ_i shares. The fill price is $P_{\mathrm{fill},i} = P_{\mathrm{mid},i}(1 + \phi_i)$, and explicit commission is $C_i = \Delta Q_i \cdot P_{\mathrm{fill},i} \cdot c_{\mathrm{rate},i}$. The actual cash outflow is notional plus commission:

$$\mathrm{Outflow}_i = \Delta Q_i \cdot P_{\mathrm{fill},i} + C_i$$

Expand $P_{\mathrm{fill},i}$ into the mid-price and spread:

$$\begin{aligned}\Delta Q_i \cdot P_{\mathrm{fill},i} &= \Delta Q_i \cdot P_{\mathrm{mid},i} \\ &+ \Delta Q_i \cdot P_{\mathrm{mid},i} \cdot \phi_i = \Delta Q_i \cdot P_{\mathrm{mid},i} \\ &+ S_i\end{aligned}$$

where S_i is the embedded spread cost. Therefore, cash outflow is identically decomposed as

$$\begin{aligned}\mathrm{Outflow}_i &= \Delta Q_i \cdot P_{\mathrm{mid},i} + S_i \\ &+ C_i = \Delta Q_i \cdot P_{\mathrm{mid},i} + \mathrm{Cost}_i\end{aligned}$$

After all trades, ending cash is

$$\begin{aligned}\mathrm{Cash}_K &= \mathrm{Cash}_0 - \sum_{i \in \mathcal{A}} \mathrm{Outflow}_i \\ &= \mathrm{Cash}_0 - \sum_{i \in \mathcal{A}} \Delta Q_i \cdot P_{\mathrm{mid},i} - \sum_{i \in \mathcal{A}} \mathrm{Cost}_i\end{aligned}$$

At closing time t_{mark}, the market value of holdings is

$$\begin{aligned}V_K &= \sum_{i \in \mathcal{A}} H_i \cdot P_{\mathrm{mark},i} \\ &= \sum_{i \in \mathcal{A}} \Delta Q_i \cdot P_{\mathrm{mark},i}\end{aligned}$$

Total portfolio net asset value is

$$\begin{aligned}&\mathrm{NAV}_K\\&= \mathrm{Cash}_K + V_K\\&= \mathrm{Cash}_0 + \sum_{i\in\mathcal{A}} \Delta Q_i \cdot (P_{\mathrm{mark},i} - P_{\mathrm{mid},i}) - \sum_{i\in\mathcal{A}} \mathrm{Cost}_i\end{aligned}$$

Using the definitions $\mathrm{GrossPnL} = \sum_{i\in\mathcal{A}} \Delta Q_i \cdot (P_{\mathrm{mark},i} - P_{\mathrm{mid},i})$ and $\mathrm{TotalCost} = \sum_{i\in\mathcal{A}} \mathrm{Cost}_i$:

$$\mathrm{NAV}_K = \mathrm{Cash}_0 + \mathrm{GrossPnL} - \mathrm{TotalCost}$$

Since net PnL is defined as $\mathrm{NetPnL} = \mathrm{GrossPnL} - \mathrm{TotalCost}$, rearranging proves

$$\mathrm{NAV}_K - \mathrm{Cash}_0 \equiv \mathrm{GrossPnL} - \mathrm{TotalCost} \equiv \mathrm{NetPnL}$$

This derivation proves that notional uses the fill price P_{fill}, so the cash account has naturally paid the spread already. Gross PnL must therefore use the mid-price P_{mid} as the buy-price benchmark; the ledger's change in NAV then exactly matches net PnL.

6.2 Convergence and idempotency of the incremental execution state machine

Let the system's target vector be $\mathbf{Q}^* \in \mathbb{R}^M_{\geq 0}$. Assume input identity is fixed and the database contains the cumulative filled vector $\mathbf{Q}^{(k)} \in \mathbb{R}^M_{\geq 0}$, where $0 \leq \mathbf{Q}^{(k)} \leq \mathbf{Q}^*$. Define the incremental order function as

$$\delta^{(k)} = \max\left(\mathbf{0}, \mathbf{Q}^* - \mathbf{Q}^{(k)}\right)$$

A single offline execution fills the outstanding amount $\delta^{(k)}$, so

$$\begin{aligned}\mathbf{Q}^{(k+1)} &= \mathbf{Q}^{(k)} + \delta^{(k)}\\&= \mathbf{Q}^{(k)} + (\mathbf{Q}^* - \mathbf{Q}^{(k)}) = \mathbf{Q}^*\end{aligned}$$

After the target is reached, every later repeated trigger $j \geq 1$ gives

$$\begin{aligned}\delta^{(k+j)} &= \max\left(\mathbf{0}, \mathbf{Q}^* - \mathbf{Q}^*\right)\\&= \mathbf{0} \implies \mathbf{Q}^{(k+j+1)} = \mathbf{Q}^{(k+j)}\\&= \mathbf{Q}^*\end{aligned}$$

After its first complete execution, the system state converges to a fixed point, which proves idempotency mathematically.

6.3 Derivation of strict holding periods and execution-lag return decomposition

Consider the time interval from signal generation to the closing mark:

Let $t_{\text{signal}} < t_{\text{trade}} \leq t_{\text{mark}}$. Define price-change rates:

- Execution-lag price growth (dimensionless):

$$g = \frac{P_{\text{trade}}}{P_{\text{signal}}} - 1$$

- Actual holding-period asset return (dimensionless):

$$r = \frac{P_{\text{mark}}}{P_{\text{trade}}} - 1$$

If the signal time t_{signal} is incorrectly treated as the start of the holding period, the resulting fictitious return is

$$\begin{aligned} R_{\text{illusory}} &= \frac{P_{\text{mark}}}{P_{\text{signal}}} - 1 \\ &= \frac{P_{\text{trade}}(1+r)}{P_{\text{signal}}} - 1 \\ &= (1+g)(1+r) - 1 \end{aligned}$$

Expanding gives

$$R_{\text{illusory}} = 1 + g + r + g \cdot r - 1 = r + g(1+r)$$

The algebraic difference between the fictitious and actual returns is

$$R_{\text{illusory}} - r = g(1+r)$$

Units matter: R_{illusory} and r are dimensionless relative changes, while the strategy's holding PnL is an absolute monetary amount (USD):

$$\begin{aligned} &\Delta\text{PnL}_{\text{illusory}} - \Delta\text{PnL}_{\text{real}} \\ &= Q \cdot (P_{\text{mark}} - P_{\text{signal}}) - Q \cdot (P_{\text{mark}} - P_{\text{trade}}) \\ &= Q \cdot (P_{\text{trade}} - P_{\text{signal}}) = Q \cdot g \cdot P_{\text{signal}} \end{aligned}$$

This difference represents a market move that occurred while the system did not yet hold the asset. Counting it as strategy return is a look-ahead execution bias. The actual holding period must be defined strictly as $(t_{\text{trade}}, t_{\text{mark}}]$.

6.4 Cross-resource atomicity and crash recovery design

In a single-machine operating system, SQLite and the file system are separate storage resources and do not have a cross-resource two-phase commit (2PC) mechanism. To recover from crashes, the system defines three execution and publication paths:

1. Path 1 (not yet published): the final directory does not exist. Write the calculated output to a separate staging directory identified by the run, `staging_<run_key>_<attempt>`. After integrity and reconciliation checks pass, use the underlying `os.rename(staging, final)` system call for an atomic replacement. Finally, record the terminal state `COMPLETED` in the SQLite state database.
2. Path 2 (published and valid): after restart, the `final` directory exists and its report-file hash matches the value expected by the state database. This means the previous interruption happened after the atomic rename but before the database state write. Record `COMPLETED` in SQLite directly; do not trade again.
3. Path 3 (published but damaged): after restart, the `final` directory exists but a file is missing or its validation hash does not match. The system must reject explicitly and raise an alert, preserving the damaged files for investigation. It must not overwrite them or conceal the damage by starting a new retry.

During recovery, first ask "What evidence remains?" and then choose an action. The table follows only the local recovery rules for the same input identity and a controlled single-machine fault; it adds no new fault experiment.

Fault point or observed state	Persisted evidence that must be checked	Recovery action
Interruption after the first fill, before a final directory exists	Same run_key and input fingerprint; `fills` contains A with 10 shares and B with 0	Reconcile the ledger, then fill B's 20 shares without buying A again; reconcile and publish in a new attempt
No final directory during a later unpublished stage	Persisted fills and input identity; staged output is not yet formally published	Calculate the difference from `fills`, finish or rebuild the staged output, and validate it; do not infer no fills merely from `RUNNING`
Final directory is valid, but terminal-state registration was interrupted	Output integrity/hash, database's expected identity, and persisted fills agree	Record only `COMPLETED`; do not match again
Final directory exists but files are missing or the hash differs	Damaged output and database records	Reject and preserve the evidence; do not overwrite it or conceal it with a new attempt

If a real broker request times out while its fill status is unknown, local `fills` may not yet contain the external fact. Use the UNKNOWN query-and-stop path in §5.5; do not apply the first row and assume "not recorded locally" means "not filled externally." File publication and SQLite registration are still two resource operations, not a cross-resource transaction.

Physical boundary: `os.replace` provides an atomic switch of directory-entry names within the same file-system mount point; across mount points it raises `OSError(EXDEV)` and has no automatic fallback. Namespace atomicity also does not mean that the physical storage hardware

will preserve data through a power loss. The `RuntimeError` in the test is a controlled software exception, not evidence of a kill or power-loss test.

7. Financial interpretation

1. Trading friction and capacity constraints: many factors appear to have robust positive returns in theoretical research. In real execution, however, bid–ask spreads and explicit commissions impose deterministic costs. Strategies with high turnover can easily have their gains consumed by trading friction.
2. Trading halts and missing liquidity: offline models often assume a position can be rebalanced at the mid-price at any time. In real markets, an individual stock may be halted or have no liquidity at its limit-down price. The execution gate must support rejection while preserving exposure; it cannot assume every order fills without obstruction.
3. Clock offsets and spurious market relationships: financial-data-provider timestamps include network-transfer delays and internal cache times. Without strict PIT alignment, clock offsets can reverse cause and effect and create a spurious arbitrage factor that cannot earn money in live trading.

Chapter 45 complete numerical example: all steps and conclusions

8. Numerical example (constructed hand calculation)

This section follows the root numerical contract through a hand-calculable discrete derivation. The example is constructed for teaching and checks the pipeline's logic contract; it is not evidence of real strategy alpha.

8.1 Initial conditions and constructed data

- Initial cash: $\text{Cash}_0 = 10,000.000$ USD.
- Asset set: $\mathcal{A} = \{A, B\}$.
- Asset-set consistency check: input validation requires the observations, `simulator_execution`, and `simulator_marks` to contain exactly the same asset set, $\{A, B\}$. If any asset record is missing, reject immediately before creating a run record.
- Supported stress-test configuration: a simple-return shock must satisfy $r_{\text{shock}} \geq -1.0$ (to prevent an asset price from becoming negative). A general scenario allows positive return shocks, and loss is defined as Loss $= -$ScenarioPnL, so loss is negative when the scenario is profitable.
- Historical reference prices (one period earlier): $P_{A,\text{past}} = 90.000$ USD/share and $P_{B,\text{past}} = 45.000$ USD/share.
- Market prices observed at $t_{\text{obs}} =$ 2026-09-01T16:00Z:

$$P_{A,\text{obs}} = 100.000 \text{ USD/share}, \quad P_{B,\text{obs}} = 50.000 \text{ USD/share}$$

- Point-in-time validity check:

$$t_{\text{available}} \leq \text{16:05Z}, \quad t_{\text{ingested}} \leq \text{16:05Z}, \quad t_{\text{as_of}} \leq \text{16:05Z}$$

  Set the signal decision time to $t_{\text{signal}} =$ 2026-09-01T16:05Z. The condition $\max(t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}) \leq t_{\text{signal}}$ is satisfied, so the timing gate passes.

Continued: the same complete example

- Calculate the one-period momentum feature and check finiteness:

$$R_A = \frac{100.000 - 90.000}{90.000} = +11.111\% > 0$$

$$R_B = \frac{50.000 - 45.000}{45.000} = +11.111\% > 0$$

Both momentum returns are finite real numbers. If a prior price is close to the floating-point underflow limit (such as 10^{-320}) and division produces `inf`, the system detects the non-finite value immediately after feature calculation, sets the state to `STOPPED`, records a `FEATURE_VALIDATION_FAILED` event, and generates 0 fills.

- Generate target share quantities: because both asset momentum values are positive, the fixed engineering teaching rule generates fixed target quantities that are finite positive real numbers:

$$Q_A^* = 10.0 \text{ shares}, \quad Q_B^* = 20.0 \text{ shares}$$

8.2 Next-day execution and fault-injection semantics

Trade parameters: Half-spread $\phi = 10$ bps $= 0.0010$; explicit commission rate $c_{\text{rate}} = 10$ bps $= 0.0010$ (charged on actual fill notional); market impact, financing interest, and securities-borrow costs are all 0. The cash reconciliation tolerance is 10^{-6} USD.
Fault-injection count semantics: Fault configuration is based on the total number of actually committed fills, not on a hard-coded asset name:

- `fault="after_first_fill"`: interrupt after the first actual fill has been committed.
- `fault="reconcile_after_first_fill"`: inject a cash perturbation after the first fill has been committed.
- `fault="risk_before_second_fill"`: apply the risk block before the second actual fill. If fewer than two positive targets are active, reject this fault configuration before initialization.

Step 1: First discrete fill (Asset A)

Trade time: $t_{\text{trade},A} =$ 2026-09-02T09:00Z. Mid-price: $P_{\text{mid},A} = 100.000$ USD/share. Fill price:

$$P_{\text{fill},A} = 100.000 \times (1 + 0.0010) = 100.100 \text{ USD/share}$$

Actual fill notional:

$$N_A = 10 \times 100.100 = 1001.000 \text{ USD}$$

Explicit commission:

$$C_A = 1001.000 \times 0.0010 = 1.001 \text{ USD}$$

Embedded spread cost:

$$S_A = 10 \times (100.100 - 100.000) = 1.000 \text{ USD}$$

Continued: the same complete example

Total trading friction for Asset A:

$$\text{Cost}_A = S_A + C_A = 1.000 + 1.001 = 2.001 \text{ USD}$$

Cash deduction:

$$\Delta\text{Cash}_A = -(1001.000 + 1.001) = -1002.001 \text{ USD}$$

Cash balance after the fill:

$$\text{Cash}_A = 10000.000 - 1002.001 = 8997.999 \text{ USD}$$

Position: $H_A = 10$ shares. This fill is successfully persisted in the SQLite `fills` table.

Step 2: Inject a controlled interruption

With `fault="after_first_fill"`, raise a controlled exception, `RuntimeError` (the `after_first_fill` branch), after the first actual fill has been committed. At this point:

- Asset B has not been filled.
- The SQLite state database records the current state as `RUNNING`.
- The final output directory has not yet been created.

Step 3: Recover and rerun idempotently

Restart the pipeline using the same `run_key`:

1. Read the state database and confirm the input fingerprint matches; allow execution to continue. `after_first_fill` is a one-time fault for this logical run, so skip it automatically during recovery.
2. The increment operator reads the `fills` table and detects 10 shares already filled for Asset A:
$$\Delta Q_A = \max(0, 10 - 10) = 0 \text{ shares}$$
Skip Asset A; do not buy it again.
3. Asset B has 0 shares filled:
$$\Delta Q_B = \max(0, 20 - 0) = 20 \text{ shares}$$

Step 4: Second discrete fill (Asset B)

Trade time: $t_{\text{trade},B}$ = 2026-09-02T09:01Z. Mid-price: $P_{\text{mid},B} = 50.000$ USD/share. Fill price:

$$P_{\text{fill},B} = 50.000 \times (1 + 0.0010) = 50.050 \text{ USD/share}$$

Actual fill notional:

$$N_B = 20 \times 50.050 = 1001.000 \text{ USD}$$

Explicit commission:

$$C_B = 1001.000 \times 0.0010 = 1.001 \text{ USD}$$

Embedded spread cost:

$$S_B = 20 \times (50.050 - 50.000) = 1.000 \text{ USD}$$

Continued: the same complete example

Total trading friction for Asset B:

$$\text{Cost}_B = S_B + C_B = 1.000 + 1.001 = 2.001 \text{ USD}$$

Cash deduction:

$$\Delta\text{Cash}_B = -(1001.000 + 1.001) = -1002.001 \text{ USD}$$

Final cash balance after the fill:

$$\text{Cash}_{\text{final}} = 8997.999 - 1002.001 = 7995.998 \text{ USD}$$

Positions: $H_A = 10$ shares and $H_B = 20$ shares.

8.3 End-of-period marks and reconciliation

End-of-period valuation time: $t_{\text{mark}} = 2026\text{-}09\text{-}02\text{T}16{:}00\text{Z}$. Market mark prices:

$$P_{\text{mark},A} = 102.000 \text{ USD/share}, \quad P_{\text{mark},B} = 49.000 \text{ USD/share}$$

1. Total market value of ending positions:

$$V(\mathbf{H}) = 10 \times 102.000 + 20 \times 49.000 = 1020.000 + 980.000 = 2000.000 \text{ USD}$$

2. Portfolio net asset value (NAV):

$$\text{NAV} = \text{Cash} + V(\mathbf{H}) = 7995.998 + 2000.000 = 9995.998 \text{ USD}$$

3. Portfolio gross PnL (benchmarked to the mid-price at the fill):

$$\begin{aligned}\text{GrossPnL} &= 10 \times (102.000 - 100.000) + 20 \times (49.000 - 50.000) \\ &= +20.000 - 20.000 = 0.000 \text{ USD}\end{aligned}$$

4. Total trading friction:

$$\text{TotalCost} = \text{Cost}_A + \text{Cost}_B = 2.001 + 2.001 = 4.002 \text{ USD}$$

5. Portfolio net PnL:

$$\text{NetPnL} = \text{GrossPnL} - \text{TotalCost} = 0.000 - 4.002 = -4.002 \text{ USD}$$

6. Closed-loop check of the NAV and reconciliation identity:

$$\text{NAV} - \text{Cash}_0 = 9995.998 - 10000.000 = -4.002 \text{ USD}$$

$$\begin{aligned}\text{Residual} &= |\text{NetPnL} - (\text{NAV} - \text{Cash}_0)| \\ &= |-4.002 - (-4.002)| = 0.000 \text{ USD} \le 10^{-6} \text{ USD}.\end{aligned}$$

   If terminal reconciliation fails (for example, after injecting a reconciliation difference), the system sets the state to `STOPPED` during the closing check. It records the event name `TERMINAL_RECONCILIATION_FAILED`, preserves committed simulated fills, rejects publication, and does not allow a retry to hide the failure.

7. Holding intervals: Both dates are 2026-09-02 (UTC). Asset A's interval is (09:00, 16:00]; B's is (09:01, 16:00]. The full date and timezone remain in the Section 4.1 contract and original inputs.

9. Python implementation

The production-pipeline core is packaged in `quant_course.production_pipeline`. This section presents the standard call interface, standalone CLI, and Notebook instructions.

9.1 Call the same core

The core is in the production core, and the command line is in the chapter CLI. Run the following fragment from the course root in the existing environment. It does not copy the matching algorithm; each run creates a new demonstration directory and preserves the interruption evidence.

```
from pathlib import Path
from uuid import uuid4
import sqlite3
from quant_course.production_pipeline import run_pipeline

work = Path("runs") / ("reader_" + uuid4().hex)
work.mkdir(parents=True)
arguments = dict(run_key="teaching-task", state_db=work / "state.db",
                 output_dir=work / "artifacts")
try:
    run_pipeline(**arguments, fault="after_first_fill")
except RuntimeError:
    with sqlite3.connect(work / "state.db") as db:
        assert db.execute("SELECT COUNT(*) FROM fills").fetchone()[0] == 1
else:
    raise AssertionError("The fault branch did not trigger")
result = run_pipeline(**arguments)
assert result["positions"] == {"AssetA": 10.0, "AssetB": 20.0}
assert abs(result["terminal_nav"] - 9995.998) < 1e-6
assert result["reconciliation"]["status"] == "PASS"
with sqlite3.connect(work / "state.db") as db:
    assert db.execute("SELECT COUNT(*) FROM fills").fetchone()[0] == 2
print(result["final_cash"], result["terminal_nav"], result["net_pnl"])
```

9.2 Standalone command line

Run the following command from the course root. `state-db` holds the recovery state for the same logical task; `output-dir` is the final directory bound at the task's first execution. A new input requires a new task identity. The same key cannot be reused with a different input or target directory. The command below is a normal run. If `--fault after_first_fill` is added for a controlled-fault demonstration, that invocation exits nonzero; rerun with the same arguments without `fault` to check recovery.

```
PYTHONDONTWRITEBYTECODE=1 .venv/bin/python code/ch45_production_quant.py \
  --run-key ch45-example \
  --state-db runs/reader_cli_01/state.db \
  --output-dir runs/reader_cli_01/artifacts
```

`reader_cli_01` is the first demonstration path. For a new independent experiment, use a new directory name and preserve the original failed directory. Recovery of the same task continues with its original `state-db` and `output-dir`. The data frequency is constructed daily events; the signal is at 2026-09-01 16:05 UTC, simulated fills occur at 09:00/09:01 UTC the next day, and each return interval runs from its fill to 16:00 UTC. Freshness is checked relative to the logical `signal_time`; these historical demonstration dates are not presented as current live quotes.

9.3 Teaching Notebook execution rules

The teaching Notebook (`notebooks/ch45_production_quant.ipynb`) follows these principles:

1. Cells call only core functions in `quant_course.production_pipeline`; they do not reimplement matching and reconciliation algorithms.
2. The accepted I3 Notebook runs a normal constructed scenario once in a separate new directory, checks cash, NAV, and the residual, reads the event ledger, and generates one new plot. Multiple ledger-event stages are not a multi-month strategy.
3. Idempotency and recovery are covered in Section 9.1, 45-P04, the standalone `capstone/check_portable.py`, and existing tests. Do not claim the Notebook runs those scenarios; that recovery entry point is currently unavailable in the working checkout.
4. Execute top to bottom from a clean kernel and produce a clean structured summary and verification plots.

10. Visualizations

The following three figures come from a new R2 run. They use constructed data and carry no empirical meaning for market alpha or recovery success rates.

Figure 45.1: Five-gate result for the completed task

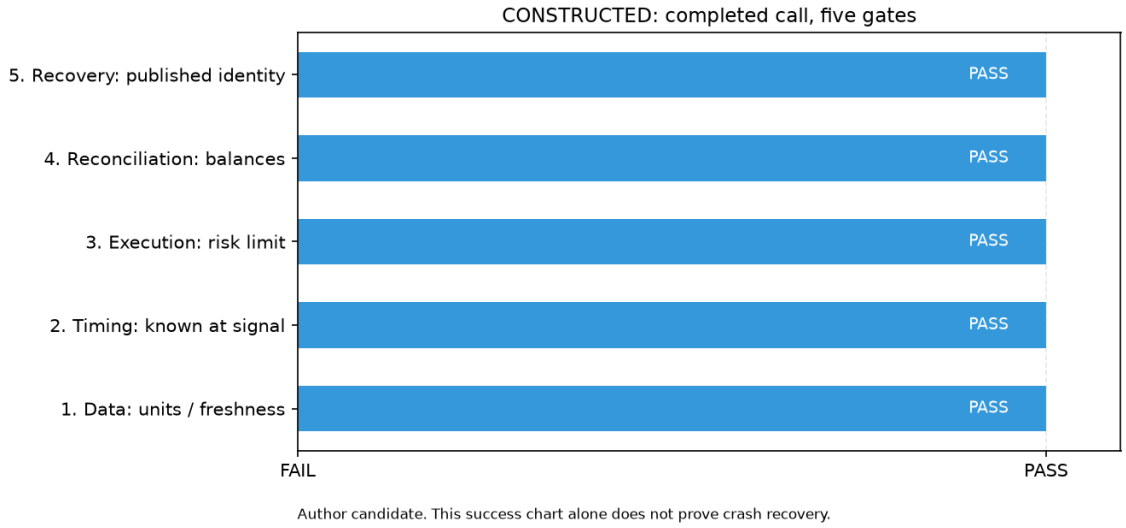


Figure 1: Constructed data: result of the five gates for this run

The vertical axis lists the data, timing, execution, reconciliation, and recovery gates; the horizontal axis is FAIL/PASS, not return. The figure was generated from the actual new CLI summary. All PASS means only that this normal run completed; interruption, damaged publication, and evidence from a B-first fill require separate inspection of faults and events and cannot be inferred from the success figure.

Figure 45.2: Holding intervals by asset

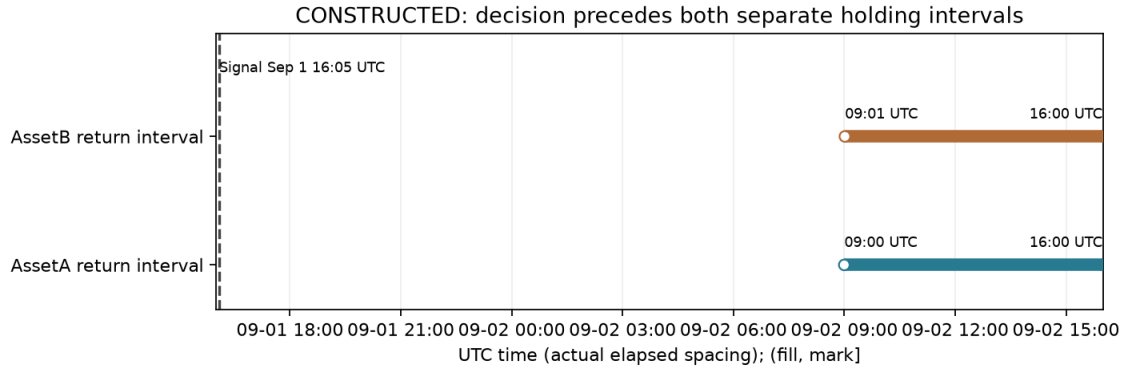


Figure 2: Constructed data: signal, fill, and mark times

The horizontal axis shows UTC clock time at the actual time intervals; the vertical axis distinguishes assets rather than prices. The previous day's 16:05 signal precedes both next-day fills; hollow markers show fill times and solid markers show the 16:00 mark. A is (09:00, 16:00] and B is (09:01, 16:00]; they cannot be combined into one return window. Inputs are bound by hash, and the new figure does not overwrite the old one.

Figure 45.3: Cash and NAV in the event ledger

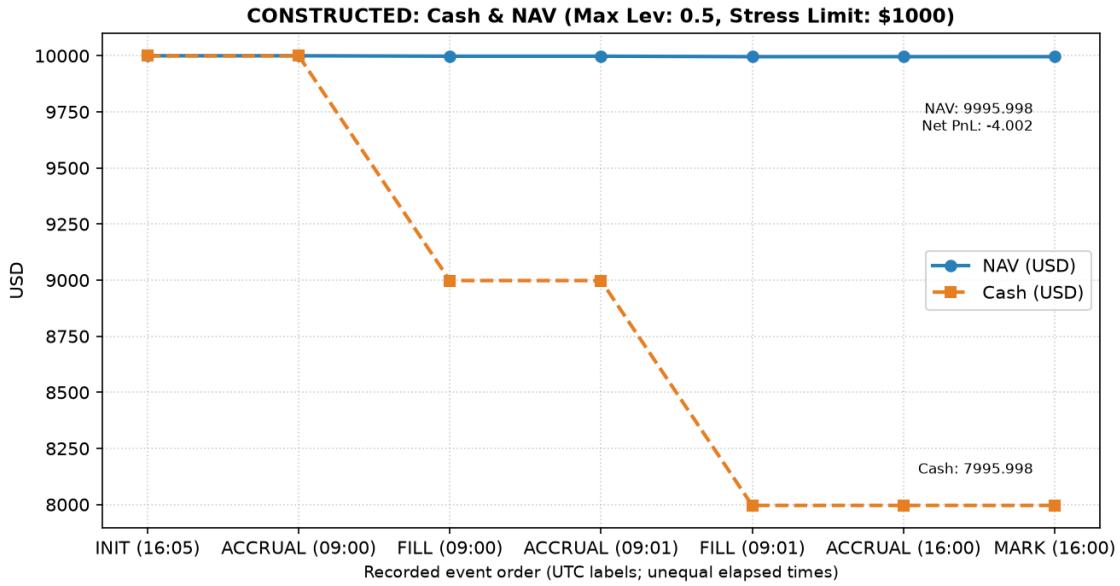


Figure 3: Constructed data: ledger cash and NAV

The horizontal axis shows actual ledger-event order and its UTC time slices; the vertical axis is USD. Orange cash falls from 10000 to 8997.999 and then 7995.998. Blue NAV also includes position value and ends at 9995.998. A fall in cash on a purchase does not mean an equal loss: most of the two cash outflows of 1002.001 USD each becomes assets. Total actual cost is 4.002 USD, gross PnL is 0, and net PnL is −4.002. See Section 8 and the independent Decimal recalculation for every intermediate amount; the amount tolerance is 1e−6 USD.

11. Real-market examples

11.1 Knight Capital's 2012 market-making system incident

According to facts established in the U.S. Securities and Exchange Commission's administrative order (SEC Release No. 34-70694): pages 5–6 §§12–17 state that the new RLP code was intended to replace the old Power Peg function in SMARS, but one of eight servers did not receive the new code. A parent order bearing a reuse flag triggered the old path, and executed quantities were not passed to SMARS, leading to duplicate child orders. Page 2 §1 reports more than 4 million executions, not exactly 4 million orders; the final loss exceeded $460 million. The engineering lessons extracted here concern deployment review, state propagation, and limiting additional orders; no further causal claim about the incident is made.

The following are engineering recommendations drawn from the incident facts; they are not verbatim requirements in the order:

1. Automate deployment and version checks: do not copy files manually to individual servers; cluster deployment must verify that versions are consistent.
2. Physically remove retired code: do not leave deprecated dead code in production indefinitely behind a logical switch.
3. Establish an independent, real-time risk circuit breaker: risk controls must be able to cut off order submission independently of the main business path and stop orders automatically when cumulative fills or abnormal exposure exceed limits.

11.2 Price floors for financial assets and domain-contract design

Input validation in financial-engineering systems must match the legal and physical properties of the underlying asset:

1. Limited-liability spot equity: shareholders have limited liability, and the theoretical and market price of a share satisfies $P \geq 0$. A negative spot quote can be treated as a data-source parsing error and blocked by the data gate.
2. Physically delivered commodity futures: in a specific delivery month, futures prices can be negative in theory and in practice when spot storage, transportation, and handling costs exceed the commodity's value. A data contract that rigidly assumes every instrument must have a positive price can cause quotes to fail parsing or reject a valid settlement.

Data-gate constraints must therefore be parameterized by asset class, distinguishing physical boundaries in the financial domain from data-format anomalies.

12. Common errors

This section gives six complete, self-contained counterexamples with assertion checks and contrasting fixes.

Diagnosis and repair: complete counterexample

Error 1: Look-ahead leakage in a point-in-time join

- Cause: using a financial statement's period-end date as its actual public release date in the join.
- Self-contained test code:

```
def test_pit_leakage_counterexample():
    market_records = [
        {"date": "2026-04-01", "symbol": "XYZ", "price": 100.0},
        {"date": "2026-04-30", "symbol": "XYZ", "price": 105.0}
    ]
    # The first-quarter reporting period ended on 03-31, but the filing was not
    ↪ made public until 04-28.
    fundamentals = [
```

Continued

```
        {"period_end": "2026-03-31", "available_at": "2026-04-28", "symbol": "
    ↪ XYZ", "eps": 1.50}
    ]
    # Incorrect logic: join on period_end, so data released on 04-28 appears on
    ↪ 04-01.
    bad_seen = None
    for m in market_records:
        if m["date"] == "2026-04-01":
            for f in fundamentals:
                if f["period_end"] <= m["date"]:
                    bad_seen = f["eps"]
    assert bad_seen == 1.50  # Look-ahead leakage occurs!

    # Correct logic: join strictly on available_at.
    good_seen = None
    for m in market_records:
        if m["date"] == "2026-04-01":
            for f in fundamentals:
                if f["available_at"] <= m["date"]:
                    good_seen = f["eps"]
    assert good_seen is None  # Correct isolation; no leakage occurred.
test_pit_leakage_counterexample()
```

Diagnosis and repair: complete counterexample

Error 2: Duplicate buying without idempotency

- Cause: blindly submitting the full order on rerun without comparing the persisted ledger.
- Self-contained test code:

```
def test_idempotency_counterexample():
    targets = {"A": 10, "B": 20}

    # Incorrect logic: a stateless loop produces a cumulative 20 shares of A
    ↪ across two runs.
    bad_fills = []
    for _ in range(2):
        for sym, shares in targets.items():
            bad_fills.append((sym, shares))
    assert sum(s for sym, s in bad_fills if sym == "A") == 20  # The position
    ↪ doubles!

    # Correct logic: compare with the persisted ledger and calculate only the
    ↪ difference.
    durable_fills = {"A": 10}  # Persisted fill left by the previous interrupted
    ↪  run.
    new_orders = []
    for sym, target_shares in targets.items():
        filled = durable_fills.get(sym, 0)
        delta = max(0, target_shares - filled)
        if delta > 0:
```

Continued

```
            new_orders.append((sym, delta))
            durable_fills[sym] = filled + delta
    assert new_orders == [("B", 20)]
    assert durable_fills == {"A": 10, "B": 20}
test_idempotency_counterexample()
```

Diagnosis and repair: complete counterexample

Error 3: `os.replace` fails across mount points

- Cause: `os.replace` cannot perform a single atomic inode directory-entry switch across different file-system mount points.
- Self-contained test code:

```
from pathlib import Path
from uuid import uuid4
import os

work = Path("runs") / ("rename_demo_" + uuid4().hex)
staging, final = work / "staging_attempt_01", work / "final_output"
staging.mkdir(parents=True)
(staging / "summary.json").write_text('{"status":"author_candidate"}')
assert not final.exists()
os.replace(staging, final)
assert (final / "summary.json").exists()
assert not staging.exists()  # It was renamed to final; its contents were not
    ↪ deleted.
print(final)
```

This demonstrates a successful rename on the same file system only; it does not establish that cross-device failure was tested. Rejection, event, and state checks for an actual rename failure are covered by the fault tests.

Diagnosis and repair: complete counterexample

Error 4: Direct `==` comparison of floating-point cash

- Cause: binary floating-point rounding can make an exact equality check fail. `==` is an equality comparison; identity comparison uses `is`. This example uses an absolute tolerance of 10^{-6} USD for amounts; it is not a universal precision for accounts of every size. A hand calculation with `Decimal` constructed from decimal strings can use exact assertions; see Error 6 and 45-P03.
- Self-contained test code:

```
def test_money_tolerance_comparison():
    v1 = 0.1 + 0.2
    v2 = 0.3
    assert v1 != v2  # Binary rounding means the values are not equal.
    assert abs(v1 - v2) <= 1e-6  # A financial tolerance correctly checks the
    ↪ ledger balance.
```

Continued

```
test_money_tolerance_comparison()
```

Diagnosis and repair: complete counterexample

Error 5: In-place configuration changes break historical traceability

- Cause: a Python `dict` is a mutable reference, and editing it directly can contaminate upstream configuration.
- Self-contained test code:

```
import copy

def test_immutable_config_copy():
    base_config = {"commission_bps": 10.0, "assets": ["A", "B"]}
    # Incorrect: reuse and modify the same reference.
    bad_cfg = base_config
    bad_cfg["commission_bps"] = 20.0
    assert base_config["commission_bps"] == 20.0  # The base configuration has
    ↪ been contaminated!

    # Correct: deep-copy the configuration or convert it to an immutable view.
    base_config["commission_bps"] = 10.0
    good_cfg = copy.deepcopy(base_config)
    good_cfg["commission_bps"] = 20.0
    assert base_config["commission_bps"] == 10.0  # The base configuration
    ↪ remains unchanged.
test_immutable_config_copy()
```

Diagnosis and repair: complete counterexample

Error 6: Deducting spread cost twice

- Cause: fill notional already includes the spread, but spread cost is deducted once more after notional.
- Self-contained test code:

```
from decimal import Decimal

def test_spread_double_deduction():
    cash = Decimal("10000.000")
    shares = Decimal("10")
    mid_price = Decimal("100.000")
    fill_price = Decimal("100.100")
    spread_cost = Decimal("1.000")
    commission = Decimal("1.001")

    # Incorrect deduction: spread_cost is deducted a second time.
    bad_cash = cash - (shares * fill_price + spread_cost + commission)
    assert bad_cash == Decimal("8996.999")  # USD 1.000 below the correct
    ↪ balance.
```

Continued

```
    # Correct deduction: notional plus explicit commission.
    good_cash = cash - (shares * fill_price + commission)
    assert good_cash == Decimal("8997.999")  # Correct ending cash.
test_spread_double_deduction()
```

13. Quant-research applications

The pipeline architecture provides underlying support for the Capstone integrated research project (Mini Quant Fund Research Project). Among its 18 research-report requirements, the pipeline supplies the following technical evidence:

1. Data dictionary and time contract (specification items 3 and 4): the data gate consumes the types, units, and timestamps defined by the data dictionary and enforces PIT rules.
2. Trading costs and reconciliation (specification item 9): spread and commission parameters are tied to matching logic, closing the accounting loop from Gross PnL to Net PnL.
3. Engineering cold start and run index: a fixed input fingerprint supports a clean-state rerun and validation of business results while preserving failed and rejected records. The Capstone technical protocol has been accepted, but actual research execution has not been authorized; out-of-sample results, the complete actual experiment set, and robustness conclusions have not been produced. A cold start in this chapter checks engineering execution only. Real paper trading and out-of-sample strategy acceptance are left for later.
4. Limitations and monitoring interface (specification item 17): the accepted engineering example records input versions, reconciliation residuals, and stop reasons. Model drift and rollback are teaching designs; no real model monitoring or retraining was run.

14. Chapter summary

A production quant pipeline requires four core engineering principles:

1. Temporal causality: clocks move monotonically forward, information must be available no later than the signal decision, and the holding-period return is defined strictly over $(t_{\text{trade}}, t_{\text{mark}}]$.
2. Idempotent state: every execution uses the persisted ledger as its sole source of truth and converges on rerun through the increment algorithm $\Delta Q = \max(0, Q^* - Q_{\text{filled}})$.
3. Ledger conservation: fill notional already embeds the spread cost, and the ending change in NAV equals gross PnL minus total friction cost within a tolerance of 10^{-6} USD.
4. Fault recovery: a single-machine environment has no natural cross-resource transaction. Lossless recovery relies on "write to separate staging → validate integrity → atomic replacement on the same file system → register in the state database."

15. Exercises

45-E01 (Conceptual)

The temporal data gate involves seven timestamps: $t_{\text{event}}, t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}, t_{\text{signal}}, t_{\text{trade}}, t_{\text{mark}}$. Explain the meaning of $t_{\text{as_of}}$ as the cutoff time for a business snapshot. Why is satisfying $\max(t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}) \leq t_{\text{signal}}$ necessary but not sufficient to avoid look-ahead bias?

45-E02 (Mathematical derivation)

Under assumptions A1–A5, let the buy fill price be $P_{\text{fill}} = P_{\text{mid}}(1+\phi)$ and explicit commission be C.

1. Write the analytic expressions for notional N, embedded spread cost S, and explicit commission C.
2. Derive and prove that if cash is reduced by $N+C$ and gross PnL uses P_{mid} as the benchmark, then $\text{NAV}_t - \text{Cash}_0 \equiv \text{GrossPnL} - (S + C)$.
3. If a system incorrectly deducts $N + S + C$ from cash, calculate the direction and value of the deviation in the reconciliation residual.

45-E03 (State machine)

Let the target position be $\mathbf{Q}^* = (10.0, 20.0)$ shares. The system is interrupted in a controlled way after matching Asset A (10.0 shares), when the database records $\mathbf{Q}_{\text{filled}} = (10.0, 0.0)$.

1. Calculate the increment vector after restart: $\Delta\mathbf{Q} = \max(\mathbf{0}, \mathbf{Q}^* - \mathbf{Q}_{\text{filled}})$.
2. Prove that if the system is triggered again after Asset B has filled, the increment is $\mathbf{0}$ and the system is at a fixed point.

45-E04 (System design)

In a single-machine environment, why can `os.replace` guarantee atomic replacement of directory entries only within the same file-system mount point? Why is atomic replacement of a directory entry not the same as durability through a physical power loss?

45-E05 (Risk controls and secrets)

1. List three different trigger dimensions for an emergency circuit breaker.
2. Explain why configuring secret credentials as environment variables does not make them absolutely secure, and state the specific requirements for log redaction.

45-E06 (Model monitoring)

Give the full names and mathematical definitions of the Kolmogorov–Smirnov test (KS), Population Stability Index (PSI), and Page–Hinkley test (PH). Using the fixed-baseline μ_0 rule in §5.8, state the zero initialization and minimum range; for $e_1 = 0.20$, $\mu_0 = 0.10$, $\delta = 0.01$, and $h = 0.05$, calculate by hand whether the first step triggers an alert. Explain how the increments differ between the fixed-baseline and cumulative/running-mean variants, and use $(0.10, 0.10, 0.10, 0.30)$ to distinguish a cumulative mean from the mean of the most recent two periods. Explain why an online update can occur only after a label has matured and is actually available.

16. Programming exercises

45-P01 (Reject before trading)

Call the chapter's core and inject NaN, Inf, a bool price, a wrong unit, a future `as-of`, and stale data one at a time. Use a new directory for each case. Confirm that the actual exception is raised and that SQLite is absent or `fills` contains 0 rows. Do not write a weaker validator to replace the core under test.

45-P02 (Deduplication through persistence)

Call the core with the fixed input to completion. After running again with the same run key, confirm that the fill count is still 2. Then change only the initial capital while retaining the original key and confirm that the fingerprint check rejects the run without adding fills. Read SQLite directly to verify; checking only the return value is insufficient.

45-P03 (High-precision Decimal hand reconciliation)

Use the Python standard library's `decimal.Decimal` to independently calculate portfolio reconciliation. Use opening cash of 10000.000 USD and the fill and mark data for Assets A and B. Assert that the reconciliation residual is exactly 0.

45-P04 (Controlled interruption and recovery test)

Write a test script that uses a controlled `RuntimeError` to simulate an interruption, then verify after restart that Asset A is not ordered again, Asset B is filled correctly, and the final holdings are 10 shares of A and 20 shares of B.

17. Challenge questions

45-C01 (Distributed execution and the double-spend boundary)

In a distributed quant system across hosts, why can neither SAGA nor two-phase commit (2PC) mathematically guarantee exactly-once external fills when the broker gateway lacks idempotent deduplication or an order has an unknown status after a timeout?

45-C02 (Online drift monitoring and adaptive rollback)

Analyze why online performance monitoring based on the Page–Hinkley statistic must depend on mature labels and a frozen baseline. Derive the trade-off between false-alarm rate and detection delay.

18. Exercise answers and hints

45-E01 Reference answer

1. Meaning of $t_{\text{as_of}}$: it is the cutoff time at which the data batch is treated as a business-information snapshot (for example, the 2026-09-01 closing snapshot). It defines the cutoff of the market facts reflected in that data and differs from a legal effective date.
2. Why the condition is not sufficient: the inequality $\max(t_{\text{available}}, t_{\text{ingested}}, t_{\text{as_of}}) \le t_{\text{signal}}$ guarantees only that some version of the data existed before t_{signal}. If the source later issues a historical revision (such as a revised macroeconomic indicator or financial disclosure), but a backtest reads the latest table containing the future revision without filtering by revision timestamp, the backtest has look-ahead data contamination. Timestamp ordering alone is therefore necessary, not sufficient.

45-E02 Reference answer

1. Analytic expressions: $N = \Delta Q \cdot P_{\text{mid}}(1+\phi)$; $S = \Delta Q \cdot P_{\text{mid}} \cdot \phi$; $C = N \cdot c_{\text{rate}}$.
2. Derivation: Cash outflow is $\Delta\text{Cash} = -(N+C)$. Ending position value is $V = \Delta Q \cdot P_{\text{mark}}$. Change in net asset value:

$$\begin{aligned}\Delta\text{NAV} &= \Delta\text{Cash} + V = -N - C + \Delta Q \cdot P_{\text{mark}} \\ &= -(\Delta Q \cdot P_{\text{mid}} + S) - C + \Delta Q \cdot P_{\text{mark}}\end{aligned}$$

   Rearranging gives

$$\Delta\text{NAV} = \Delta Q \cdot (P_{\text{mark}} - P_{\text{mid}}) - (S+C) = \text{GrossPnL} - (S+C) = \text{NetPnL}$$

   The identity is proved.
3. Analysis of the incorrect deduction: if $N + S + C$ is deducted, cash is overcharged by S and ending NAV is understated by S, so $\text{NAV}_{\text{err}} - \text{Cash}_0 = \text{NetPnL} - S$. The signed reconciliation difference is

$$D_{\text{signed}} = \text{NetPnL} - (\text{NAV}_{\text{err}} - \text{Cash}_0) = +S.$$

The reported residual is the absolute value: Residual $= |D_{\text{signed}}| = S$, all in USD. "Positive" describes the signed difference, not a direction for the absolute residual; cash and NAV are both too low. If spread is double-deducted on both fills, total $S = 2.000$ USD, incorrect NAV is 9993.998 USD, signed difference is +2.000 USD, and reported residual is 2.000 USD.

45-E03 Reference answer

1. Calculate the post-restart increment: $\Delta Q_A = \max(0, 10.0 - 10.0) = 0.0$ shares; $\Delta Q_B = \max(0, 20.0 - 0.0) = 20.0$ shares. The increment vector is $\Delta\mathbf{Q} = (0.0, 20.0)$.
2. Prove the rerun fixed point: After Asset B's 20.0 shares fill, the database records $\mathbf{Q}'_{\text{filled}} = (10.0, 20.0)$. On a further rerun, the increment is $\Delta\mathbf{Q}' = (\max(0, 10.0 - 10.0), \max(0, 20.0 - 20.0)) = (0.0, 0.0)$. The order increment is zero, the system state does not change, and it is at a convergent fixed point.

45-E04 Reference answer

1. On a POSIX file system, `os.replace` calls `rename` underneath and changes a directory-entry pointer in the same mount's inode table without copying the data; this provides namespace atomicity. Across file-system mount points, the inode cannot be shared, so atomic rename is unavailable and raises `EXDEV`; the call has no automatic copy fallback.
2. Namespace atomicity only guarantees that another process does not see a half-renamed directory. It does not guarantee that the physical disk cache has been flushed to durable storage at the instant of a power loss (fsync/flush). Unflushed sectors can still be damaged by a hardware power failure.

45-E05 Reference answer

1. Three circuit-breaker dimensions: excessive portfolio intraday maximum drawdown; a closing reconciliation residual above tolerance ($> 10^{-6}$ USD); and continuous loss of critical market data or severe clock drift.
2. Environment-variable security limitation: in a multi-tenant environment or on a shared host, privileged processes may inspect environment values through `/proc/<pid>/environ`; plaintext may also remain in memory when a process creates a core dump. Core credentials must never be written to logs, databases, or exception traces, and printed values must be masked.

45-E06 Reference answer

1. Full names and mathematical definitions:
   - Kolmogorov–Smirnov test (KS): measures the maximum distance between two empirical cumulative distribution functions, $D = \sup_x |F_0(x) - F_t(x)|$.
   - Population Stability Index (PSI): measures the relative entropy change in binned distributions, $\text{PSI} = \sum_{k=1}^{K} (A_k - B_k) \ln(A_k / B_k)$.

- Page–Hinkley test (PH): this problem uses the fixed-baseline one-sided teaching rule in §5.8. Freeze the historical baseline μ_0, set $U_0 = 0$, and define $U_t = U_{t-1} + e_t - \mu_0 - \delta = \sum_{i=1}^{t}(e_i - \mu_0 - \delta)$. The statistic is

$$PH_t = U_t - \min_{0 \le i \le t} U_i.$$

  Set $U_t = S_t$; then $M_t = \min_{0 \le i \le t} U_i$, matching the body definitions $S_0 = M_0 = 0$ and $M_t = \min(M_{t-1}, S_t)$. Alert when $PH_t > h$. The quantities e_t, μ_0, δ, and h are all decimal returns.

2. Zero-initial-value example: with $e_1 = 0.20$, $\mu_0 = 0.10$, and $\delta = 0.01$, $U_1 = 0.09$. Including $U_0 = 0$, the minimum is 0 and $PH_1 = 0.09 > 0.05$, so the first step triggers an alert. If one incorrectly uses $\min_{1 \le i \le 1} U_i = 0.09$, then $PH_1 = 0$ and the alert is missed.
3. Variants and terminology: the fixed-baseline rule always uses μ_0. At each step, the cumulative/running-mean variant first updates $\bar{e}_t = \bar{e}_{t-1} + (e_t - \bar{e}_{t-1})/t$, then updates $V_t = V_{t-1} + e_t - \bar{e}_t - \delta$ ($\bar{e}_0 = V_0 = 0$). Each historical increment uses its own $\bar{e}_i$, so $V_t = \sum_{i=1}^{t}(e_i - \bar{e}_i - \delta)$; do not recalculate the entire sum using the final $\bar{e}_t$. For $(0.10, 0.10, 0.10, 0.30)$, the cumulative mean at step four is 0.15 and the most-recent-two-period mean is 0.20. The two-period window is only for distinguishing terms; the question does not ask for a fixed-window PH implementation. There is no unconditional sensitivity ranking between the variants.
4. Updating with mature labels: $e_t = |y_t - \hat{y}_t|$ requires the return interval to have ended and the label to be actually available and ingested. If the label matures at 16:00 but is not received until 17:00, it cannot update the model at 16:30. After 17:00 it can be updated online or replayed offline; there is no requirement that processing occur only in an overnight batch.

45-P01 Reference answer

```
from copy import deepcopy
from pathlib import Path
from uuid import uuid4
import sqlite3
from quant_course.production_pipeline import constructed_inputs, run_pipeline

work = Path("runs") / ("answer_P01_" + uuid4().hex)
work.mkdir(parents=True)
cases = [("current_price", float("nan")), ("current_price", float("inf")),
         ("current_price", True), ("price_unit", "EUR/share"),
         ("as_of", "2026-09-01T16:06:00Z"),
         ("event_time", "2026-09-01T14:00:00Z")]
for number, (field, value) in enumerate(cases):
    data = deepcopy(constructed_inputs())
    data["observations"][0][field] = value
    folder = work / str(number)
    try:
        run_pipeline(inputs=data, run_key="invalid-input",
                     state_db=folder / "state.db", output_dir=folder / "artifacts")
    except (ValueError, TypeError):
        if (folder / "state.db").exists():
```

```
            with sqlite3.connect(folder / "state.db") as db:
                assert db.execute("SELECT COUNT(*) FROM fills").fetchone()[0] == 0
    else:
        raise AssertionError((field, "Invalid input was accepted"))
print("All 6 cases were rejected before any fill")
```

For the stale-data case, keep the `as-of` snapshot at 16:00 UTC but set the current quote's `event_time` to 14:00 UTC. By the 16:05 signal it is more than one hour old; a fresh snapshot label does not make an old quote fresh. This example checks the age of both the current quote's `event_time` and the `as-of` timestamp. The prior price, which was repaired first, is a lookback input and is not rejected by the current-quote one-hour threshold.

45-P02 Reference answer

```
from pathlib import Path
from uuid import uuid4
import sqlite3
from quant_course.production_pipeline import default_config, run_pipeline

work = Path("runs") / ("answer_P02_" + uuid4().hex)
work.mkdir(parents=True)
arguments = dict(run_key="same-logical-task", state_db=work / "state.db",
                 output_dir=work / "artifacts")
first = run_pipeline(**arguments)
second = run_pipeline(**arguments)
assert first == second
with sqlite3.connect(work / "state.db") as db:
    assert db.execute("SELECT COUNT(*) FROM fills").fetchone()[0] == 2
changed = default_config()
changed["initial_cash"] = 20000.0
try:
    run_pipeline(config=changed, **arguments)
except ValueError:
    pass
else:
    raise AssertionError("Different configuration was not rejected for the same key")
with sqlite3.connect(work / "state.db") as db:
    assert db.execute("SELECT COUNT(*) FROM fills").fetchone()[0] == 2
print("2 fills after rerun; still 2 after rejecting different input")
```

45-P03 Reference answer

```
from decimal import Decimal

def reconcile_handcalc():
    initial_cash = Decimal("10000.000")
    fills = [
        {"symbol": "A", "shares": Decimal("10.0"), "fill_price": Decimal("100.100"),
         "commission": Decimal("1.001"), "mid_price": Decimal("100.000"), "mark_price
    ↪ ": Decimal("102.000")},
        {"symbol": "B", "shares": Decimal("20.0"), "fill_price": Decimal("50.050"),
         "commission": Decimal("1.001"), "mid_price": Decimal("50.000"), "mark_price"
    ↪ : Decimal("49.000")}
    ]
    cash = initial_cash
    total_cost = Decimal("0")
    gross_pnl = Decimal("0")
    holdings_val = Decimal("0")

    for f in fills:
        notional = f["shares"] * f["fill_price"]
        cash -= (notional + f["commission"])
        spread_cost = f["shares"] * (f["fill_price"] - f["mid_price"])
        total_cost += (spread_cost + f["commission"])
        gross_pnl += f["shares"] * (f["mark_price"] - f["mid_price"])
        holdings_val += f["shares"] * f["mark_price"]

    nav = cash + holdings_val
    net_pnl = gross_pnl - total_cost
    residual = abs(net_pnl - (nav - initial_cash))

    assert cash == Decimal("7995.998")
    assert nav == Decimal("9995.998")
    assert gross_pnl == Decimal("0.000")
    assert total_cost == Decimal("4.002")
    assert net_pnl == Decimal("-4.002")
    assert residual <= Decimal("0.000001")  # Satisfies the 1e-6 tolerance.
reconcile_handcalc()
```

45-P04 Reference answer

The complete reference implementation is the fragment in Section 9.1. First catch the controlled `after_first_fill` exception and immediately query SQLite to prove that only one fill was committed. After recovery, assert two fills and the target positions/NAV. If the injected fault does not trigger, the `else` branch fails, preventing a normal run from being reported as successful recovery. This tests recovery after an in-process exception; it does not prove physical power-loss durability. Stronger `before_publish`, `after_publish_before_complete`, and actual rename-failure tests are in the chapter test source; failed staging directories are preserved.

45-C01 Reference answer

In a distributed system, sending an order to a broker is an externally controlled side effect. If the local system has a network partition or times out after sending, it cannot know whether the order has already been filled at the exchange. Even with 2PC or SAGA:

1. If the local system sends a compensating cancellation, cancellation can race with execution. If the order has already filled at the exchange, cancellation fails.
2. If the local system enters an unknown state and waits, it cannot automatically reconcile before the network recovers.
3. The system therefore needs the broker gateway to support idempotent deduplication keyed by `client_order_id` and an active order-status reconciliation interface. Without cooperation from the external interface, a one-sided distributed protocol cannot conjure an exactly-once guarantee.

45-C02 Reference answer

The fixed-baseline PH-style rule defined in Section 5.8 is $S_0 = M_0 = 0$, $S_t = S_{t-1} + e_t - \mu_0 - \delta$, and $M_t = \min(M_{t-1}, S_t)$; it alerts only when $S_t - M_t > h$. The quantities $e_t = |y_t - \hat{y}_t|$, μ_0, δ, and h are decimal returns, so the USD ledger tolerance does not apply. Set $U_t = S_t$ and $U_0 = 0$; the same statistic is $PH_t = U_t - \min_{0 \le i \le t} U_i$, so the minimum must include the zero initial value. This question discusses the fixed-baseline relationship; the cumulative/running-mean variant uses the current $\bar{e}_t$ for the current increment and does not recalculate earlier increments. It differs from a fixed-window moving mean and has no unconditional sensitivity advantage. River's implementation has other options for forgetting and two-sided detection and is not the same algorithm.

1. Wait until both the return interval ends and the label is actually available and ingested; both times must precede the update. The error can be calculated only then. A label that matures at 09-02 16:00 but arrives at 17:00 is still unavailable at 16:30. After 17:00 it may be updated online or replayed offline; it need not wait for an overnight batch.
2. For a fixed error path, if $h_2 > h_1$, the first time the larger threshold is crossed cannot precede the first time the smaller threshold is crossed; over a fixed observation period, the trigger set shrinks as the threshold rises. The actual false-alarm rate still needs calibration on permitted historical time blocks. If the persistent mean increment is $\Delta\mu > \delta$, a rough detection-delay approximation, ignoring noise and overshoot, is $h/(\Delta\mu - \delta)$ mature-label update periods; this is not a guarantee. Heavy tails, dependence, and repeated monitoring of multiple metrics can all change the actual false-alarm rate.
3. After an alert, first check data versions, freshness, transformations, and units; then decide whether to stop new orders or roll back compatibly. Existing cash and fills cannot be erased. This batch does not train, automatically retrain, or tune thresholds on the final test.

Open-question rubric: mature labels and units/frozen baseline (30%); path relationship and approximation conditions (30%); false-alarm calibration and multiple monitoring (20%); stopping orders and state-compatible rollback (20%).

19. Further reading

- Must Know:
  - The Twelve-Factor App: Config (https://12factor.net/config): practices for separating configuration and sensitive secrets.
  - Python sqlite3 Documentation (https://docs.python.org/3.14/library/sqlite3.html): standard-library explicit transaction control.
  - Python os.replace Documentation (https://docs.python.org/3.14/library/os.html): atomic rename on a single file system.
- Should Know:
  - Python Logging HOWTO (https://docs.python.org/3/howto/logging.html): separation of module loggers and application handlers.
  - PyPA Packaging Projects (https://packaging.python.org/en/latest/tutorials/packaging-projects/): standard buildable Python package layouts.
  - Apache Parquet Overview (https://parquet.apache.org/docs/overview/): columnar storage and encoding format.
- Advanced:
  - SEC Administrative Release No. 34-70694 (*In the Matter of Knight Capital Americas LLC*, https://www.sec.gov/litigation/admin/2013/34-70694.pdf): a major systems-engineering incident and missing automated risk controls.
  - SQLite Transaction Control (https://www.sqlite.org/lang_transaction.html): locking and write-lock concurrency.
- Optional:
  - DuckDB Parquet Overview (https://duckdb.org/docs/current/data/parquet/overview): in-process columnar queries and predicate pushdown.
  - Apache Airflow DAG Runs (https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dag-run.html): scheduling intervals and idempotent rerun concepts.

20. References

1. [CH45_PYPA_PACKAGING] Python Packaging Authority. *Packaging Python Projects*. URL: https://packaging.python.org/en/latest/tutorials/packaging-projects/. (Source read by the root reviewer: supports the `build-system` configuration and modern Python project packaging structure.)
2. [CH45_PYPA_SRC_LAYOUT] Python Packaging Authority. *Discussions: src-layout vs flat-layout*. URL: https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/. (Source read by the root reviewer: supports physical separation of the project root and package source.)
3. [CH45_PY_LOGGING] Python Software Foundation. *Logging HOWTO (Python 3 Documentation)*. URL: https://docs.python.org/3/howto/logging.html. (Source read by the root reviewer: supports the separation of structured logging and the responsibilities of application handlers.)
4. [CH45_PY314_LOGGING_COOKBOOK] Python Software Foundation. *Logging Cookbook (Python 3.14 Documentation)*. URL: https://docs.python.org/3.14/howto/logging-cookbook.html. (Source read by the root reviewer: explains the limits of writing to one file from multiple processes.)
5. [CH45_PARQUET_OVERVIEW] Apache Software Foundation. *Apache Parquet Documentation: Overview*. URL: https://parquet.apache.org/docs/overview/. (Source read by the root reviewer: supports the principles of columnar storage and separation from the file engine.)
6. [CH45_DOCKERFILE] Docker Inc. *Dockerfile reference and concepts*. URL: https://docs.docker.com/build/concepts/dockerfile/. (Source read by the root reviewer: supports basic container-image instructions and environment-reproducibility practice.)
7. [CH45_DOCKER_SECRETS] Docker Inc. *Docker Build Secrets*. URL: https://docs.docker.com/build/building/secrets/. (Source read by the root reviewer: supports separating build ARG/ENV from runtime credentials.)
8. [CH45_DUCKDB_PARQUET] DuckDB Foundation. *DuckDB Parquet Overview and Partial Reading*. URL: https://duckdb.org/docs/current/data/parquet/overview. (Source read by the root reviewer: supports teaching columnar predicate pushdown and vectorized in-memory queries.)
9. [CH45_PY314_SQLITE] Python Software Foundation. *sqlite3 — DB-API 2.0 interface for SQLite databases (Python 3.14 Documentation)*. URL: https://docs.python.org/3.14/library/sqlite3.html. (Source read by the root reviewer: supports explicit transaction control and atomic accounting.)
10. [CH45_PY314_OS_REPLACE] Python Software Foundation. *os.replace (Python 3.14 Documentation)*. URL: https://docs.python.org/3.14/library/os.html. (Source read by the root reviewer: supports single-file-system atomic rename and staged publication.)
11. [CH45_PY314_SCHED] Python Software Foundation. *sched — Event scheduler (Python 3.14 Documentation)*. URL: https://docs.python.org/3.14/library/sched.html. (Source read by the root reviewer: explains the general in-process scheduler.)

12. [CH45_PY314_URLLIB] Python Software Foundation. *urllib.request — Extensible library for opening URLs (Python 3.14 Documentation)*. URL: https://docs.python.org/3.14/library/urllib.request.html. (Source read by the root reviewer: explains the HTTP external-I/O boundary and offline teaching.)
13. [SK_PITFALLS] scikit-learn developers. *Common Pitfalls and Recommended Practices: Data Leakage*. URL: https://scikit-learn.org/stable/common_pitfalls.html. (Source read by the root reviewer: supports strict feature-transform and test-set isolation.)
14. [CH45_GOOGLE_ML_RULES] Martin Zinkevich. *Rules of Machine Learning: Best Practices for ML Engineering*. URL: https://developers.google.com/machine-learning/guides/rules-of-ml. (Source read by the root reviewer: supports reducing training-serving skew and monitoring data freshness.)
15. [CH45_12FACTOR_CONFIG] Adam Wiggins. *The Twelve-Factor App: III. Config*. URL: https://12factor.net/config. (Source rechecked by the root reviewer: supports separating secret credentials from environment configuration.)
16. [CH45_PYTEST_DOCS] Holger Krekel et al. *pytest: Simple powerful testing for Python*. URL: https://docs.pytest.org/en/stable/. (Source rechecked by the root reviewer: supports deterministic unit tests and fixed-asset isolation.)
17. [CH45_SQLITE_TRANSACTIONS] D. Richard Hipp et al. *SQLite Language: Transaction Control*. URL: https://www.sqlite.org/lang_transaction.html. (Source rechecked by the root reviewer: supports single-database ACID state persistence and lock-state explanation.)
18. [CH45_AIRFLOW_DAGRUN] Apache Software Foundation. *Apache Airflow: DAG Runs*. URL: https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dag-run.html. (Source rechecked by the root reviewer: supports task-scheduling intervals and idempotent rerun concepts.)
19. [CH45_PROMETHEUS_METRICS] Prometheus Authors. *Prometheus Metric Types*. URL: https://prometheus.io/docs/concepts/metric_types/. (Source rechecked by the root reviewer: supports production-monitoring Counter/Gauge/Histogram primitives.)
20. [CH45_MLFLOW_TRACKING] Databricks / Linux Foundation. *MLflow Tracking Concepts*. URL: https://mlflow.org/docs/latest/ml/tracking/tracking-api/. (Source rechecked by the root reviewer: supports associating experiment tracking, complete runs, and artifacts with version lineage.)
21. [CH45_SEC_KNIGHT_CAPITAL] U.S. Securities and Exchange Commission. *In the Matter of Knight Capital Americas LLC (Release No. 34-70694)*. URL: https://www.sec.gov/litigation/admin/2013/34-70694.pdf. (Source rechecked by the root reviewer: supports evidence on legacy dead-code risk and independent real-time risk circuit breakers.)
22. [CH45_R2_KS] SciPy. *ks_2samp*, statistic/Notes. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html. The root reviewer read the definition of D and the independent-continuous-sample assumption; webpage version 1.18.0 is not evidence of the local version.

23. [CH45_R2_PSI] MathWorks. *risk.validation.populationStabilityIndex*, More About. URL: https://www.mathworks.com/help/risk/risk.validation.populationstabilityindex.html. The root reviewer read the formula and input description; MATLAB was not run, and the empirical threshold is not treated as a universal guarantee.
24. [CH45_R2_PH] River. *PageHinkley*, definition/parameters/update. URL: https://riverml.xyz/dev/api/drift/PageHinkley/. The root reviewer read about online updates, running mean, forgetting, and two-sided options; this chapter's fixed-baseline example is not the library's complete implementation.
25. [CH45_R2_SRC] PyPA. *src layout vs flat layout.* URL: https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/. The root reviewer read the sections on directories, installation, and source CLI paths.
26. [CH45_R2_PEP517] Python. *PEP 517*, Abstract/Source trees/backend interface. URL: https://peps.python.org/pep-0517/.
27. [CH45_R2_PEP518] Python. *PEP 518*, Abstract/build-system. URL: https://peps.python.org/pep-0518/. This batch did not build or install the package.

The newly added R2 source scope for this chapter is recorded in `source_notes`. The original Page (1954) paper was not read directly; the fixed-baseline PH rule is an explicitly defined teaching variant in this chapter. Source references and review notes are inherited from the candidate and were not freshly web-verified during this translation.

Capstone —Research

Capstone: From Engineering Closure to Research Design

Package note. Run instructions in the engineering document apply to the final, complete course package. This Markdown-only candidate does not include runnable course entrypoints. Links into `outputs/` identify read-only I3 sources; they are not command working directories. Final package navigation will be mapped during integration.

This reader edition was added on 2026-10-14. The research question concerns the paired net monthly return difference between industry momentum strategy M0 and EW10 under the fixed revised French ten-industry history, hypothetical monthly allocations, and an all-in assumed cost. The A10-A design received limited acceptance, and the A10-A.1 v2 execution protocol received technical acceptance; G02 (dates) and G04 (statistical and cost conventions) are closed. G01 (the actual subject, use, and applicable rights evidence) remains unknown; G05 has not authorized A10-B, so execution is blocked. Historical `available` / `ingested` / `revision` times remain unknown. `full_preregistration`, `market_verified`, and `execution_authorized` are all `false`. Raw redistribution is `not_requested`, which does not mean `granted`. The historical data have been exposed and are not a blind holdout.

Reading order and prerequisites

Read Research Question, Sources, and Prior Exposure, then the Monthly Return Data Dictionary and Time and Label Contract, then the Full Mathematical Protocol. Finally, follow the 18-item report guide while completing the report template. The read-only I3 source map records source hashes, versions, and reader destinations; readers do not need production QA to understand this protocol.

Prerequisites by topic: Chapters 3 and 11 cover data sources and compounding; Chapter 15 covers RF / FF3 attribution; Chapters 22–25 cover hypotheses and signals; Chapters 29 and 31–33 cover costs, time, and inference; Chapters 35–36 cover cost units and execution boundaries; Chapter 38 covers label maturity; Chapter 45 covers identity, status, and recovery. Projects P1–P3 show historical statistics and benchmarks, P4 shows constructed costs, and P6 shows prediction, labels, and unresolved real-research boundaries. Direct links to the current English chapter sources: Chapter 3, Chapter 11, Chapter 15, Chapter 22, Chapter 23, Chapter 24, Chapter 25, Chapter 29, Chapter 31, Chapter 32, Chapter 33, Chapter 35, Chapter 36, Chapter 38, and Chapter 45. Return through the read-only body index, Loom index, and P1, P2, P3, P4, and P6.

The two configs and the two dictionaries

Material	Units and purpose	Use
`offline_constructed.json` and the engineering dictionary	AssetA / AssetB; `USD/share`, shares, second-level constructed timestamps; two fixed trades	Run the constructed example in an isolated environment using the engineering entrypoint; preserve the config bytes
`french_protocol_v2.json` and the monthly-return dictionary	Monthly decimal total returns for ten industries, portfolio weights, and normalized amounts	Read alongside the parameter teaching material and preserve the config bytes; there is no run command or implemented monthly-return adapter

The French parameter file has SHA-256 `d49ac62cff7d6496e478211921d507fe7216c18b0e4f13b57e48fc893eb12505`. Its candidate gate fields retain their source identity. The original main conversation closed G02 and G04 on 2026-10-13. Do not treat old `pending` fields as current open items, and do not use those closures to change `execution_authorized` to `true`. Full source hashes are recorded in the read-only source map.

The bridge from engineering to research

The engineering pipeline can carry over `run_id`, `input/config/code` identity, quality rejections, idempotency, and recovery semantics. Its two-asset, single-period `price/share` interface cannot represent multi-period monthly total returns for ten industries. A monthly-return ledger adapter would be needed; it has not been implemented and is not authorized. The following is a data-flow sketch only:

```
fixed snapshot / units / month key -> monthly quality gate -> cutoff and 11-term
    ↪ compounded score
-> fixed-order top-k and EW targets -> carry forward prior-month drifted amounts
-> solve post-cost NAV and target trades -> monthly total return -> month-end
    ↪ holdings / final liquidation
-> continuous 440-row ledger -> primary 132-row paired difference -> fixed statistics
    ↪  / risk / post-hoc attribution
```

A return or wealth index cannot stand in for a `USD/share` price. A 3,600-second staleness gate cannot be applied as a monthly-return cleaning rule. Without minute prices, volume, order-book data, or execution slots, TWAP / VWAP / POV cannot model real execution. The engineering example's USD 9,995.998 NAV and USD 4.002 cost explain only the constructed engineering example; they do not fill French research results.

This teaching package explains how to design and interpret the study. Target-market results do not exist, so it cannot answer the primary hypothesis. It does not register the full Capstone as verified or formally published.

Research Question, Sources, and Prior Exposure

The README states the current technical acceptance and stopping point. This document explains why M0 is compared with EW10 and what inputs can answer the question. Full parameters are in the protocol and the original-byte French protocol config in the read-only I3 source.

Primary question and hypotheses

In the accepted primary reporting window, 2015-09..2026-08 (future execution still requires authorization), does a fully invested long-only strategy that ranks industries by total return over the preceding 11 months ($t-12,\ldots,t-2$ for holding month t), holds the top three equally, and rebalances monthly produce a higher average net monthly return than the monthly equal-weight benchmark across ten industries, under the same 10 bps per unit of traded notional cost?

The economic-mechanism hypothesis is that common industry-level shocks and slow information diffusion may lead to medium-term return continuation. Reversal, industry risk premia, or concentration could produce the same observation. This study tests only the return difference; it does not identify a causal effect of "slow diffusion." Skipping the most recent month is a fixed signal definition, not a demonstrated reversal effect or evidence about data availability. Do not use unread papers to supply conclusions about the established literature.

The experimental unit is one calendar holding month's paired portfolio net-return difference, $d_t = r_{M,\text{net},t} - r_{EW,\text{net},t}$. The ten industries form that month's signal cross-section; they are not ten independent monthly replications. The primary endpoint is $\mu_d = E[d_t]$ (decimal per month). The directional alternative is $H_1 : \mu_d > 0$, with null $H_0 : \mu_d \leq 0$; assess it using the prespecified one-sided 95% lower bound and paired HAC inference over the monthly sequence. Economic significance is considered separately: whether the average net advantage reaches the illustrative prespecified threshold of 10 `bps/month`, alongside wealth, drawdown, and concentration. The threshold is a design judgment, not calibrated to actual capital costs.

The simplest baseline assigns 10% to each of the ten industries each month, with the same execution, cost, entry, and exit rules as the strategy. Net return must first come from a self-financing ledger. Subtract RF once only for Sharpe or attribution; the primary paired difference cancels the common same-month RF. There is no model training, factor screening, or maximum-Sharpe optimization.

Negative and zero outcomes also complete the research. If the interval crosses zero, evidence is insufficient under the sample and dependence assumptions. If the net mean is ≤ 0, report that this specification did not achieve a net advantage. If a gross advantage is consumed by costs, report the cost sensitivity. A non-significant result does not prove the strategy is always ineffective. If a data or licensing gate fails, the conclusion is feasibility-blocked, not a zero result from a market test.

Input: total-return portfolio categories, not OHLCV

The ten industries are supplier portfolio categories, not permanent security IDs; monthly returns follow the supplier's total-return convention. The observation is one paired portfolio return difference for a month, not ten independent industry experiments. The selected route can compare portfolio-level differences under fixed revised history and hypothetical costs. The institution-level, security-by-security PIT route was not selected; the P5 requirement for 16 assets and the P6 data-span requirement do not carry over.

Fixed input identity: `French_20261005_revised_history_A10A_v1`; snapshot date 2026-10-05; vintage 202608; coverage 1926-07..2026-08, 1,202 months. Industry download time in UTC: `2026-10-05T19:18:24.585420+00:00`; FF3 download time: `2026-10-05T21:11:10.296518+00:00`. Convert the original percentages to decimals once only. This edition reuses metadata from the accepted materials; it did not read new `CSV/ZIP` bytes or recompute data statistics. Names, hashes, and units are in the read-only I3 source-identity map. This package does not distribute the research-market `CSV/ZIP` files.

Historical source review was limited to the official ten-industry construction, coverage, and copyright sections; the factor `RF/definition` sections; the data homepage sections on `revisions/format` changes; and the CRSP changes body. The linked CRSP PDF was not read. Do not use that unread PDF or unchecked papers to prove the mechanism. This edition did not browse the web and makes no claim of new source review or of an absence of licensing terms across the whole site. Prior ICBC plans and older rights tables are historical records only; the current actual subject and use are unknown and are not presumed to be individual or bank-internal. Public download and copyright attribution do not replace rights evidence for the actual activity.

Exposure, degrees of freedom, and future access

Prior exposure	Known range and basis	Influence on the choice	Boundary
P1	README and manifest: NoDur / HiTec / Enrgy for 2010–2019, with descriptive results for 2020–2024	Carry forward unit and snapshot review; industry-wealth measures have already been shown	The historical design batch did not recompute its measures
P2	`README/config`: nine realized CAPM / FF3 / FF5 groups for 2010–2019; estimate on 2010–2014 and same-month attribution on 2015–2019	Choose FF3 rather than mixing FF3 and FF5; carry forward monthly `RF/HAC` conventions	Attribution is not prediction, and alpha does not decide whether momentum wins
P3	`README/config`: full ten-industry history 1990-01..2026-08; 60-month primary specification and multiple `window/cost` sensitivities; public report did not credibly find three estimated portfolios better than equal weight	Use a simple equal-weight baseline to reduce model search; this choice has a hindsight component	Net results and benchmark exposure already exist over the same history; it cannot be called an unused holdout

Prior exposure	Known range and basis	Influence on the choice	Boundary
P4	Cost-preparation `README/acceptance` entry: constructed signals, share ledger, and `cost/execution` experiments	Adopt exact cost-ledger boundaries and avoid residuals from industry-neutral projection	Constructed costs do not calibrate real markets
P5	`Manifest/acceptance` identity: paired constructed study; real panel blocked; 16-asset historical requirement belongs to P5	Do not inherit its `assets/dates` or choose a model based on paired performance	Not every constructed-performance row was reread; `user/old-session` exposure history is unknown
P6	`Manifest/acceptance` identity: `baseline/tree-model` constructed results and later presentation revisions; real study blocked and overall status not verified	Do not add an ML contest; reuse time-contract gates	The missing row-level predictions for Chapter 39 and the failed-artifact gaps remain open; do not claim the evidence is complete

Access by other sessions or people to the target momentum question is unknown; historical test status is `exposed_or_unknown` and `full_preregistration=false`. Locked reporting constrains future target runs and changes; it does not retroactively make the history a blind holdout.

Before any first authorized target run, preserve the run manifest, `code/config/data` hashes, all 13 prespecified paths, seed, accessors, and UTC times. Report the primary result and every prespecified sensitivity together, including positive, zero, and negative outcomes. Do not change parameters after viewing locked reporting. Preserve failed inputs and outputs for technical retries and show that the repair did not depend on returns. New economic hypotheses or performance-driven revisions require a new version and must be labeled as post-hoc exploration. A genuinely unused future sample must be supported by access records and frozen separately; do not invent future dates. No training, screening, fitted transformation, or maximum-Sharpe optimization is included.

The 18-item report guide turns the question into input → action → output → interpretation. The time contract explains unknown historical clocks and label maturity.

Data Dictionary for the French Monthly-Return Study

Identity: `French_20261005_revised_history_A10A_v1`. The wide table's primary key is `month`; the research long table's primary key is `(snapshot_id, month, industry_id)`. UTC is used for audit timestamps; `month` is a calendar interval, not a fabricated trading timestamp. There is no permanent security-level ID.

Field	Economic meaning / key	Type, unit, frequency	event_time	available_at

Field	Economic meaning / key	Type, unit, frequency	event_time	available_at
snapshot_id	Fixed input identity; part of composite key	string, version	End of snapshot coverage	Earliest release of current version: unknown
month	Calendar return month; primary key	YYYYMM string / monthly	Start and end of that month	Historical month-by-month availability: unknown
industry_id	Long-form ID for a ten-industry column; primary key	enum / monthly	Supplier group effective interval	Member release: unknown
NoDur / Durbl / Manuf / Enrgy / HiTec / Telcm / Shops / Hlth / Utils / Other (each column)	Value-weighted industry total return, including dividends; the same contract applies to every column	float, decimal simple total return / month	month interval	Unknown; do not fill with "known at month-end"
Mkt-RF	Published market excess return	float, decimal monthly spread	month interval	Unknown; post-hoc attribution only
SMB	Small-minus-big return spread	float, decimal monthly spread	Same as above	Same as above
HML	High-minus-low book-to-market return spread	float, decimal monthly spread	Same as above	Same as above
RF	Proxy for the month's risk-free return	float, decimal simple monthly return, not an annual rate	Same as above	Unknown; post-hoc statistics only, with any cash convention stated separately
signal_score	Compounded return over the prior 11 months, minus 1	float, dimensionless / monthly	$t-12, \ldots, t-2$	After all inputs are available; historical timing unknown
target_weight	Top 3 each 1/3, all others 0	float, capital weight / monthly	Month t when allocation takes effect	Signal cutoff
holding_value / cash / nav	Monetary amount and NAV	float or Decimal, illustrative USD or normalized unit / monthly	Ledger node	When node completes
traded_notional / fee	Traded amount and cost	Amount; fee rate as decimal, bps divided by 10,000	Rebalance node	When node completes
gross_return / net_return / label	Portfolio gross/net return and target-period return	float, decimal / monthly	Holding month t	After label end and source release/ingestion; historical times unknown
label_start / label_end / label_available	Label interval and maturity time	UTC timestamps / monthly	Interval boundary	Actual availability unknown; explicitly specified for constructed examples

Field	Economic meaning / key	Type, unit, frequency	event_time	available_at
cutoff / allocated_at	Information cutoff and hypothetical allocation-effective time	UTC timestamps	Protocol rule	Rule known in advance

Field	ingested_at	revision_at	as_of
snapshot_id	Manifest download UTC	Unknown; retain new versions	This candidate's fixed snapshot
month	Inherits file download	Historical time: unknown	Inherits snapshot
industry_id	Inherits file	Unknown	Inherits snapshot
NoDur / Durbl / Manuf / Enrgy / HiTec / Telcm / Shops / Hlth / Utils / Other (each column)	Industry file download UTC	Unknown	Fixed snapshot
Mkt-RF	FF3 download UTC	Unknown	Fixed FF3
SMB	Same as above	Same as above	Same as above
HML	Same as above	Same as above	Same as above
RF	FF3 download UTC	Unknown	Fixed FF3
signal_score	Computation time in UTC; only in A10-B if authorized	New version if config changes	Input cutoff and snapshot
target_weight	Future computation time	Protocol version	Corresponding cutoff
holding_value / cash / nav	Future ledger UTC	Do not overwrite prior runs	run_id
traded_notional / fee	Future ledger UTC	Run version	Same as above
gross_return / net_return / label	Future computation UTC	Snapshot version	Same as above
label_start / label_end / label_available	Future record	New version	Fit cutoff
cutoff / allocated_at	Protocol registration UTC	Revision chain	protocol_id

Missing fields are rejected or marked unknown; they are not replaced with zero. Subtract RF once from portfolio total return; do not subtract RF again from Mkt-RF / SMB / HML. Do not mix same-named FF5 columns into the FF3 dataset.

Corporate-event applicability

Splits and dividends are already reflected in the supplier's total-return methodology; do not add separate corporate-action cash. There is no security-level event table for an independent reconstruction. Delistings and the stock universe are represented only at the supplier's aggregate level; do not claim to have verified completeness of the underlying whole market. Financial-statement announcements and restatements are not direct feature inputs for the selected price-momentum question and therefore do not apply in this batch. Adding value or quality features would change the question and require new PIT fields; it is not a free option under the current protocol. Historical changes to portfolio classifications are not changes in tradable security IDs. Normalized wealth indices are for ledger demonstrations only and must not enter a real-price execution interface.

This dictionary is for the research plan. The two-asset engineering dictionary covers `USD/share`, shares, and second-level constructed runs. The interfaces are not interchangeable. The time contract specifies eligibility and label decisions; the protocol gives the full cost and statistical equations; the README gives current acceptance status.

Time and Label Contract: Reader Edition of Technically Accepted v2

The frequency is monthly and the audit timezone is UTC. The source contains only `YYYYMM`; do not rename it as an actual trading timestamp. Each monthly interval is the half-open interval `[B_t, B_(t+1))`, where `B_t` is 00:00:00 UTC on the first day of month t. This is a calendar label, not a definition of U.S. trading hours.

Decision, allocation, and holding period

The study assumes a signal cutoff $S_t = B_t - 1$ second and an `allocation/trade` marker $T_t = B_t$, so $S_t < T_t$. This marker means only that return allocation becomes effective; it does not simulate a real market fill. The holding label matches the source and spans the full calendar month $[B_t, B_{t+1})$. If the study later switches to real ETFs or opening-price execution, it must use a different data identity and match the actual return interval. A full French monthly return cannot simply be assigned to the next day's opening price.

$signal_{i,t} = \prod_{m=t-12}^{t-2}(1 + R_{i,m}) - 1$, exactly 11 months; $t - 1$ is excluded from the feature. For $t =$ 2026-01, the window is 2025-01..2025-11 and 2025-12 is skipped. This 11-month window and one-month skip provide no evidence about historical publication times.

For a real PIT row, the required eligibility condition is max(`available_at`, `ingested_at`, `revision_at (when applicable)`, `query_as_of`) $\le S_t$, with signal $< T_t$; if timestamps are equal and no ordering evidence exists, require strict $<$. `event_time` describes the economic event and is not forced into a total ordering with announcement time; an announcement can precede the event's effective time. `revision_at` records the external release of a revision and is not sorted by ingestion time. Select the latest eligible external version; reject a tie at the same publication time. `as_of` is the query cutoff, not a way to backdate the 2026 snapshot download time to 1990.

Historical monthly `available`, `ingested`, and `revision` times for the French data are unknown; the known download UTC records only ingestion of the 2026 snapshot. Therefore, strict historical PIT status is `unknown/blocked`; the bounded historical-analysis status is hypothetical simulation, within the scope accepted by the original main conversation. Delay sensitivities cannot turn `unknown` into `passed`.

RF and factors are used only for same-month excess returns, risk, or attribution after returns are realized; they are not features for decisions made in that month. Subtract the corresponding FF3 RF once from portfolio total return; do not subtract RF again from `Mkt-RF`, `SMB`, or `HML`. If cash is retained, using same-month RF for its return is an ex-post assumption and must be labeled as such; do not claim it was known at signal time. The primary specification is fully invested; cash only absorbs the reconciliation residual from costs.

Label maturity, purging, and gap

When fitting at cutoff F, every row must satisfy `label_end` $\leq F$, `label_available` $\leq F$, and `label_ingested` $\leq F$. Purge any label interval whose intersection with the validation interval is non-empty; adjacent half-open months share only an endpoint and do not intersect. A label that does not intersect can still be unpublished, so maturity and purging are separate checks. The fixed ranking has no model training and attribution is estimated only after the fact; retain this contract for implementation and teaching checks without inventing a 120-month "training" procedure.

For forward-only splits, all training data precede validation, so future-side embargo is 0. A one-month signal skip is a signal skip, not an embargo. Historical publication delay is unknown, so the source cannot determine a fixed maturity gap; record `pending_available_history` and do not assume that a one-month gap is enough. Cross-validation that trains on future data is outside this protocol's support domain.

Two explicit constructed rows

Row	event	available	ingested	revision
Normal constructed row	2026-01-31 19:00Z	1/31 20:00Z	1/31 21:00Z	1/31 22:00Z
Future announcement / immature row	2026-01-31 19:00Z	2/2 00:00Z	2/2 01:00Z	2/2 00:00Z

Row	query_as_of	signal	trade	label end / available
Normal constructed row	1/31 22:00Z	2/1 00:00Z	2/1 00:00:01Z	Matured prior label 1/1/1/2 (constructed)
Future announcement / immature row	2/2 01:00Z	2/1 00:00Z	2/1 00:00:01Z	February label end=3/1 00:00Z, available=3/2

Row	Step-by-step decision
Normal constructed row	Latest eligibility time 1/31 22:00 is $\leq$ signal, and signal $<$ trade, so eligible. This is a separate second-level example, not the primary monthly allocation rule.
Future announcement / immature row	available and ingested are later than signal, so reject the feature; label end and availability are also later than fit, so reject it for training.

The months and values above are explicitly CONSTRUCTED; the half-open endpoint is B_{t+1}. Do not copy constructed available times onto French rows. Applicability of corporate events is explained in the monthly-return dictionary; do not add separate event cash to aggregate total returns.

v2 confirmed calendar and continuous-stage contract

G02 dates are closed. Warm-up is 1988-07..1989-12; evaluation is 1990-01..2026-08; development ends 2004-08, validation is 2004-09..2015-08, and locked reporting is 2015-09..2026-08. Independent month-index checks established $440 = 176 + 132 + 132$; all segments are contiguous and non-overlapping. The primary mean test uses only the last 132 months.

Each of the 13 paths enters once with cash 1 at 1990-01 and then carries its actual prior drifted holdings/NAV forward every month. It does not reset, liquidate, or charge another entry cost at segment boundaries; it liquidates once in 2026-08. Thus the first primary-report row's pre_NAV carries from month-end 2015-08. The 2015-09 d is a row from the continuous paired ledger, not a separate experiment restarted with cash 1. The final primary month includes terminal liquidation. Segment-chart normalization changes display only; it does not change costs or returns.

The first-month windows are L11=1989-01..1989-11, L5=1989-07..1989-11, and L17=1988-07..1989-11; skip1 excludes December 1989. L11/skip2 uses 1988-12..1989-10. Warm-up does not initialize compounded account wealth. Signal $S_t = B_t - 1$ second and hypothetical allocation $T_t = B_t$ retain their ordering markers; the label is $[B_t, B_{t+1})$. No unknown historical French publication/ingestion/revision time has been filled in. The history is exposed and is not called a blind holdout; confirming dates does not clear G01/G05.

Check completeness of the 132-month HAC sample using month keys; do not drop a missing row and treat the next row as the next month. The forward-only design has future-side embargo = 0; the signal skip is not an embargo, and unknown historical label maturity must not be replaced

with simulated values. The `normal/future` examples above remain `CONSTRUCTED` and their eligibility clocks must not be copied to market data rows.

Return to the Capstone README. Full cost and HAC equations are in the technical protocol.

French Revised-History Monthly-Return Configuration: Technical Protocol

This document retains the complete technical body of the v2 protocol accepted on 2026-10-13, including the signal, continuous ledger, costs, and statistical conventions. Only the teaching navigation and candidate-stage process language have been adapted. G02 and G04 have received technical acceptance; the source freeze is unchanged. Current permissions and execution status are summarized in the Capstone README. Monthly labels and historical clocks are specified in the Time and Label Contract; required outputs and interpretations are listed in the report guide. A basis point (bps) is 10^{-4}; net asset value (NAV) is ledger net value; HAC means heteroskedasticity and autocorrelation consistent covariance estimation.

Question, assets, signal, and prior exposure

The bounded question is unchanged: under the fixed revised French ten-industry history, hypothetical monthly allocations, and all-in costs, does an equal-weighted long-only portfolio of the three industries with the highest compounded returns over the prior 11 months have a higher average net monthly return than the ten-industry monthly equal-weight benchmark? Slow industry-level information diffusion is a possible explanation to test; this design does not identify causation or establish a real-world tradable advantage. The primary direction is $H_1 : \mu_d > 0$, against $H_0 : \mu_d \leq 0$. A zero or negative result, an interval crossing zero, and a blocked gate have distinct interpretations. Negative results are retained, and a blocked study is not described as a zero market effect.

Asset order: NoDur, Durbl, Manuf, Enrgy, HiTec, Telcm, Shops, Hlth, Utils, Other. The primary signal is $\text{score}_{i,t} = \prod_{m=t-12}^{t-2}(1 + R_{i,m}) - 1$, exactly 11 months, skipping the most recent month $t - 1$. Sort scores descending and break ties stably by ascending manifest index; the primary top 3 receive 1/3 each. Every top-k sensitivity assigns $1/k$ to each selected industry. Gross and net exposure are both 1; leverage is 1; positions are fully invested long-only, with no financing, shorting, or negative cash. Missing, non-finite, or discontinuous-month data are rejected and logged; the asset pool is not silently changed, zero-filled, or dropped. There is no model training, factor screening, fitted standardization, or parameter optimization.

P3 has exposed the full 1990-01 through 2026-08 `ten-industry/equal-weight` history and sensitivities. `P1/P2` exposed three industries during 2010–2019, and P1 also showed descriptive results for 2020–2024. The full history was not blindly preregistered from scratch. The locked-reporting designation constrains future access and changes for this target; it does not make the sample a blind holdout. Any future target execution still requires G01 and G05. Public download availability does not establish authorization for the actual subject or use.

The one confirmed calendar and primary sample

Phase	Monthly interval, inclusive	Months	Use
Warm-up	1988-07..1989-12	18	Feature inputs; do not use these months to initialize account wealth
Development	1990-01..2004-08	176	Description and `implementation/contract` consistency; fixed rules, no tuning
Validation	2004-09..2015-08	132	Prespecified `consistency/description`; do not select a winner
Locked reporting	2015-09..2026-08	132	**The sole primary mean-test sample**, taken as 132 rows of the continuous ledger
Full evaluation	1990-01..2026-08	440	Full-period descriptions, figures, and ledger reconciliation; does not replace the primary 132-month test

The calendar arithmetic is $440 = (2026-1990)\times12+(8-1)+1$; $\lfloor 0.4\times440 \rfloor = 176$, $\lfloor 0.3\times440 \rfloor = 132$, leaving 132 months. The three full-month segments are contiguous, non-overlapping, and each is at least 24 months; the evaluation window is at least 120 months. For the first evaluation month, 1990-01, `L17/skip1` uses 1988-07..1989-11 and skips 1989-12; `L11/skip1` uses 1989-01..1989-11; L5 uses 1989-07..1989-11; `L11/skip2` uses 1988-12..1989-10. The general endpoints are $[t-\text{skip}-L,\ t-\text{skip}-1]$, containing exactly L months. Calendar arithmetic was checked in the accepted protocol's constructed checks without reading market-return values; this teaching adaptation did not rerun those checks.

Thirteen continuous paths and segment boundaries

The nine fixed strategies are: M0=`L11/skip1/k3/c10`; `C0/C25/C50` vary only the all-in cost rate; `L5/L17` vary only L; `K2/K4` vary only k; S2 changes only skip to 2. The four equal-weight cost paths are `EW0/EW10/EW25/EW50.` There are 13 paths total and zero optimization trials. Do not form a Cartesian grid or add independently reset subwindows. The primary difference is $d_t = r_{M0,\text{net},t} - r_{EW10,\text{net},t}$; the other eight strategies and three benchmark cost differences are prespecified descriptive sensitivities. They do not expand the significance-selection family or replace M0.

Each path invests normalized cash of 1 once at 1990-01, runs continuously through 2026-08, and liquidates fully in the final month. At 2004-09 and 2015-09, it directly carries forward the prior month's drifted holdings and NAV as they actually exist in the ledger. Here "actual holdings" means the actual state of the hypothetical ledger, not real-world trading. Segment boundaries

do not liquidate, charge a new entry cost, reset positions, or reset NAV. Locked-reporting `d` uses the 132 rows from the two continuous paths over 2015-09..2026-08; its first denominator carries forward the actual ending NAV at 2015-08, and its final row includes the 2026-08 liquidation cost.

For display, a segment chart may divide by the segment's starting NAV so its first point equals 1. This changes display only; it cannot recompute trade amounts or costs or alter monthly net returns. A separate run that restarts cash at 1 on 2015-09 is a new path and is not authorized by this protocol. Each path's gross allocation return $R_{\text{alloc},t}$, calculated from the existing target and that month's returns, may be reported as a no-cost compounded descriptive series. It is a same-signal ledger series, not a new selection specification. The explicit 0 bps `strategy/benchmark` is already within the 13-path budget; a "gross chart" cannot add another parameter path.

Continuous self-financing: drift, post-cost targets, entry and exit

Drift, self-financing, entry and terminal costs, and identities

Let $V_{\text{pre},t}$ be the prior month's actual ending NAV, or initial cash 1 for the first month. Let $x_{i,t}$ be the amount held after drift and before allocation. All targets satisfy $w_{i,t} \geq 0$ and $\sum_i w_{i,t} = 1$; post-cost NAV must solve the following equation. The supported domain requires $V_{\text{pre}} > 0$, finite nonnegative drifted holdings, valid simple monthly returns no lower than -1, and a positive ledger denominator for next-period normalization. A path that reaches zero must explicitly reject later `weight/return` division rather than insert a fabricated value. Finite cost rates must satisfy the fully invested long-only domain $0 \leq c < 1$. The solution conditions follow Chapter 29 in the current English reader; they do not change the agreed cost convention:

$$V_{post,t} = V_{pre,t} - c\sum_i |w_{i,t}V_{post,t} - x_{i,t}|, \quad F_t = c\sum_i |w_{i,t}V_{post,t} - x_{i,t}|.$$

The rate is $c = \text{bps}/10000$; costs and V share the same monetary unit. The sum of absolute purchases and sales is not divided by 2. After allocation, $x_{\text{post},i,t} = w_{i,t}V_{\text{post},t}$, end-of-month holding value is $x_{\text{end},i,t} = x_{\text{post},i,t}(1 + R_{i,t})$, and drifted weights are $x_{\text{end},i,t}/\sum_i x_{\text{end},i,t}$. In an ordinary month:

$$R_{alloc,t} = \sum_i w_{i,t}R_{i,t}, \qquad r_{net,t} = \frac{V_{post,t}}{V_{pre,t}}(1 + R_{alloc,t}) - 1.$$

At entry, $x_{i,1} = 0$, so $V_{\text{post},1} = 1/(1 + c)$; the first-month return is measured against initial cash 1 and includes the entry cost. In the final month, realize the holding return and then liquidate fully:

$$V_{\text{terminal}} = V_{\text{end},T}(1 - c), \qquad r_{\text{net},T}$$
$$= \frac{V_{\text{post},T}}{V_{\text{pre},T}}(1 + R_{\text{alloc},T})(1 - c) - 1.$$

Therefore,

$$\prod_{t=1}^{440}(1 + r_{\text{net},t}) = \frac{V_{\text{terminal}}}{V_{\text{initial}}}.$$

The products across segments also multiply to the same total, without another entry cost. The product over the 132 reporting rows equals final cash divided by end-of-month NAV at 2015-08. It must not be described as the return of a strategy restarted with cash 1.

Continued: the same complete example

The 10 bps rate is an uncalibrated all-in linear-friction scenario, conceptually including commissions, spread, and similar costs; `impact/capacity` is unknown and unsupported components are not reported as measured zeros. At traded notional 200.10 and $c = 0.001$, **all-in scenario friction is .20010**. Do not charge an additional spread on top or treat the rate as an annual 0.1%/12. The separate share example with a 5 bps half-spread embedded in fills and an explicit 10 bps commission is a different constructed setup, not a calibration of the primary 10 bps assumption. NAV-loss proportion C and trade-rate c have different definitions. The example $C = 0.001$ and gross $r = 0.02$ gives net $(1-C)(1+r)-1 = 0.01898$; this is a constructed unit example only.

Primary 132-month HAC mean inference

Let d_t be the paired net monthly difference over the 132 continuous primary-ledger rows, in decimal per month; do not subtract RF again. Set $n = 132$, $k = 1$, and X to a single column of ones; $\hat{\mu} = (1/n)\sum_{t=1}^{n} d_t$ and $u_t = d_t - \hat{\mu}$. Fix $L = 6$ as a count of complete calendar-month lags. All $\widehat{\gamma}_j$ use the common denominator n:

$$\widehat{\gamma}_j = \frac{1}{n}\sum_{t=j+1}^{n} u_t u_{t-j}, \qquad j = 0, \ldots, 6,$$
$$w_j = 1 - \frac{j}{7}, \qquad j = 1, \ldots, 6,$$
$$\widehat{\Omega} = \widehat{\gamma}_0 + 2\sum_{j=1}^{6} w_j \widehat{\gamma}_j.$$

The weights for lags 1–6 are 6/7, 5/7, 4/7, 3/7, 2/7, and 1/7; $\widehat{\gamma}_0$ uses $t = 1, \ldots, n$. The correction $q = n/(n-k) = 132/131$ applies to the **covariance**:

$$\mathrm{Var}(\hat{\mu}) = q\widehat{\Omega}/n = \widehat{\Omega}/(n-1),$$
$$\mathrm{SE}(\hat{\mu}) = \sqrt{\widehat{\Omega}/(n-1)}.$$

Do not replace the $\widehat{\gamma}_j$ denominator with $n - j$, and do not multiply $\widehat{\gamma}_j$ or SE by q a second time. For general k, use

$$q(X'X)^{-1}S(X'X)^{-1};$$

with constant X, $X'X = n$ and $S = n\widehat{\Omega}$, yielding the expression above. At $L = 0$,

$$\mathrm{Var}(\bar{d}) = \frac{\sum_t u_t^2}{n(n-1)} = \frac{s_d^2}{n};$$

the constructed check independently proves this. The full 440-month mean is descriptive only; do not use its n or degrees of freedom in place of the primary 132.

The statistic is $t_{\text{stat}} = \hat{\mu}/\text{SE}$. The one-sided 5% positive-direction test and one-sided 95% lower bound are $\hat{\mu} - t_{0.95,131}$ SE; the two-sided 95% interval is $\hat{\mu} \pm t_{0.975,131}$ SE. The $t(131)$ reference is an approximation and does not guarantee exact coverage under dependence or non-stationarity. If SE is `zero/non-finite`, months are missing, $n \leq L$, or parameters are not identified, report `undefined` / rejected explicitly with the reason. Do not fill months, silently reduce L, or set an undefined result to $p = 0$. Log numerical discrepancies; do not change the primary conclusion by choosing a different convention.

The local statsmodels constant-OLS mapping is `maxlags=6`, `weights_func=weights_bartlett`, `use_correction=True`, `use_t=True`. The original technical acceptance included constructed HAC and `calendar/continuous-cost-contract` checks. Those constructed checks are not research on the target market, and this reader adaptation did not rerun them.

For post-hoc FF3 attribution, subtract the same-month FF3 RF once from net total return; do not subtract RF again from published `Mkt-RF`, `SMB`, or `HML`. Keep months and currency aligned. Use an intercept and three factors, $k = 4$, with correction $n/(n-4)$; do not apply the primary $k = 1$ correction. Do not change the course's existing attribution interface or insert invented factors into the primary paired difference to call it. The shared same-month RF already cancels in the primary mean difference, so do not subtract RF again there.

Fixed bootstrap, secondary reporting, and negative results

For the primary 132-row paired d, use the circular block bootstrap with block length 12 months, 5,000 replications, and seed `20261012`; lengths 6 and 24 are prespecified interval sensitivities only. Initialize one dedicated Python `random.Random(seed)` generator once before the replication loop. All 5,000 replications consume that same evolving state, without reseeding inside the loop. Draw each required block start uniformly using `randrange(n)`, concatenate circular consecutive blocks until reaching $\lceil n/\text{block} \rceil$ blocks, and truncate to n; compute each bootstrap mean. Compute percentile quantiles by linear interpolation at position $(B-1)p$; the one-sided lower bound uses $p = 0.05$, and the two-sided interval uses 0.025 / 0.975. Report any disagreement with HAC; do not select the favorable interval or add another statistical selection family. This design is retained, and this batch did not run the target-sample bootstrap or a new selection.

CAGR, volatility, net-excess Sharpe, drawdown, costs, and weights are prespecified descriptive measures for a future authorized run. Development, validation, and the full 440 months do not replace the primary mean test and are not used for training or parameter selection. Keep the 10 `bps/month` illustrative net-advantage threshold separate from statistical significance. Preserve the original 18-item mapping for risk, industry concentration, tails, a uniform constructed shock, and FF3 attribution. With no trade `prices/volume`, stock IDs, or historical constituent versions, do not estimate real capacity or claim the absence of survivorship bias for the whole market. Do not invent `PSR/DSR/PBO` results to fill headings; where applicability is insufficient, mark `not_applicable` / `not_evaluated`. Retain all positive, zero, and negative results.

Capstone Research Report: 18-Item Teaching Guide

After reading this guide, you should be able to connect the research question through monthly returns, signals, targets, drift, costs, paired differences, and inference. The README gives current status; the protocol retains the full mathematics; the time contract and monthly-return dictionary define fields and eligibility. This guide specifies future outputs. **No target-market values have been computed**; the design is fixed.

Source codes and reading references

The question, exposure, historical source-review scope, and metadata for `S01/S07/S08/S09` are incorporated in Research Design and the read-only I3 source map. `S02/S05` refer to the Protocol and French protocol config in the read-only I3 source; S03 is the Time Contract, and S04 is the Monthly-Return Dictionary. S06 is the accepted source mapping for this guide's 18-item structure. `R/P` refer to the original-main acceptance decisions and current pending status, available from the README and source map. These codes locate sources; readers do not need to consult QA outside the package.

1. Research Hypothesis

Question to understand: Turn a proposed mechanism into a falsifiable comparison. Prerequisites: Chapters 22–25; Projects P1–P3.

Fixed design and upstream source: `S01/S02`: fixed French ten industries, `L11/skip1/top3`, EW10; $H_0 : \mu_d \leq 0$ / $H_1 : \mu_d > 0$; one month is the experimental unit; slow information diffusion is not an identified cause.

Why and how: A fixed asset set, frequency, baseline, and directional hypothesis produce one `d` per month. The future primary table reports n, net mean, one-sided lower bound, two-sided interval, and the 10 bps economic threshold; fix the columns before viewing results. The mechanism is a hypothesis, with risk premia, concentration, and reversal as alternatives.

Future output and interpretation: User decision, RQ, and fixed primary endpoint → research question; the one-sided net mean difference answers the selected question. Target-market result fields: not run.

Failure branch: Changing the hypothesis or baseline after seeing results. Retain future results for all 132 months; do not replace them with a generic "hypothesis proposed" statement.

2. Data Sources

Question to understand: How one data identity limits conclusions. Prerequisites: Chapters 3 and 10; Projects P1–P2.

Fixed design and upstream source: `S08/S09`, `R/P`: 2026-10-05 revised snapshot, `industry/FF3` identity, and download UTC; the actual `subject/use` for G01 is unknown, and raw redistribution is `not_requested`.

Why and how: First verify `data/config/run` identity and rights evidence for the actual activity; only then may inputs be read. Report snapshot, coverage, units, SHA, download UTC, and rights-status table. Full historical coverage does not mean the data were known at each historical point in time. Current materials contain metadata only; no new market files.

Future output and interpretation: Source text, license, and snapshot hash → authorized input; traceable `source/use` and revision scope. Target-market result fields: not run.

Failure branch: If licensing is unknown, do not execute real research; do not copy `CSV/ZIP` files. Source review remains within its historical scope; do not claim a new full-source review or rights grant.

3. Data Dictionary

Question to understand: Fields, keys, frequency, and units. Prerequisites: Chapters 2–3 and 11; Project P1.

Fixed design and upstream source: `S04/S03`: long-table key `(snapshot_id, month, industry_id)`, decimal total returns for ten industries, `FF3/RF`, `score/weight/amount/net-return/label-maturity` fields.

Why and how: Illustrative research long-table schema: `(snapshot_id, month, industry_id, total_return_decimal)`. This is a field example, not a market observation. Align FF3 by `month`; produce `missing/duplicate/version` logs. `industry_id` does not replace a permanent security ID.

Future output and interpretation: Fields, keys, and time units → valid monthly panel; consistent economic meaning. Target-market result fields: not run.

Failure branch: Duplicate keys or a second unit conversion. Future logs must record missingness and versions.

4. Point-in-Time Logic

Question to understand: A monthly label is not a historical publication clock. Prerequisites: Chapters 3, 12, 31, and 38.

Fixed design and upstream source: S03: half-open $[B_t, B_{t+1})$; $S = B_t - 1$ second and allocation $= B_t$ are hypothetical ordering markers only; `available` / `ingested` / `revision` remain unknown. The normal and future rows are `CONSTRUCTED`.

Why and how: On the normal constructed row, first require the latest eligibility time $\leq$ signal, then signal $<$ trade; reject the future row when eligibility is later than signal. Separately test `label_end` / `available` / `ingested` against fit cutoff F, then test half-open interval overlap. Adjacent labels can still be immature even if they do not overlap.

Future output and interpretation: Eligibility table and `normal/future` counterexample $\rightarrow$ input-eligibility status. The constructed PIT rule is testable; historical PIT remains unknown. Target-market result fields: not run.

Failure branch: Calling revised history PIT-valid or admitting a future label into training. Do not fill market history with skip1, download UTC, or constructed second-level timestamps.

5. Data Cleaning

Question to understand: What is rejected and why no silent filling is allowed. Prerequisites: Chapters 3 and 29–31.

Fixed design and upstream source: `S02/S04/S06`: reject duplicate `month/industry` keys, missing months, non-finite values, and missing values; convert original percentages to decimal by dividing by 100 once only; do not add corporate-action cash already included in total return.

Why and how: For every month, verify ten unique industry keys, a complete continuous month sequence, finite decimals, missing-value sentinels, and the one-time percentage conversion. Output rejection reasons and the retained schema; do not zero-fill, replace industries, or silently drop rows. Corporate-action cash is not added again to total returns.

Future output and interpretation: Schema counts and missing-sentinel log $\rightarrow$ `rejected/retained` lists; processing remains traceable. Target-market result fields: not run.

Failure branch: Silently dropping months, zero-filling, or applying the 3,600-second price-staleness rule to monthly returns. Do not fabricate future count tables.

6. Feature Engineering

Question to understand: Compounding endpoints and units. Prerequisites: Chapters 4, 11, and 25.

Fixed design and upstream source: `S02/S03/S05`: $\text{score}_{i,t} = \prod_{m=t-12}^{t-2}(1+R_{i,m}) - 1$, 11 terms; general window $[t-\text{skip}-L,\ t-\text{skip}-1]$; all first-evaluation-month windows are listed.

Why and how: Each `(1+R)` is a wealth multiplier; multiply 11 terms and subtract 1. Summing returns omits cross-products. For the first evaluation month, 1990-01, L11 uses 1989-01..1989-11 and skips 1989-12. A future score table records `t`, `input_start/end`, all 11 terms, score, and input identity.

Future output and interpretation: 11-month compounding formula, endpoints, and constructed expansion → score, using exactly $t-12, \ldots, t-2$. Target-market result fields: not run.

Failure branch: Treating a sum as compounding or using the holding-month return in the signal. Use the existing constructed example; do not recompute French data. The future output is a ten-industry score table.

7. Signal Construction

Question to understand: How deterministic sorting produces top-k. Prerequisites: Chapters 23, 25, and 28.

Fixed design and upstream source: `S02/S05`: fixed ten-industry order; descending score and ascending manifest index for ties; primary top3 at 1/3 each; each sensitivity assigns $1/k$; no fitting.

Why and how: Sort the ten scores uniquely by `(score descending, manifest index ascending)`. Assign $1/k$ where rank $\leq k$, otherwise 0. Output all ten `ranks/targets`, not only winners. Do not winsorize, standardize, or neutralize; any added fit requires a protocol revision.

Future output and interpretation: All ten industry scores and fixed tie order → top3; ranking is deterministic and not trained. Target-market result fields: not run.

Failure branch: Choosing industries after viewing future results or using hidden randomness for ties. The future `rank/target` table remains pending authorization.

8. Portfolio Construction

Question to understand: Targets, drift, and post-cost weights. Prerequisites: Chapters 9, 20–21, and 29; Project P3.

Fixed design and upstream source: S02: $V_{post} = V_{pre} - c\sum_i |w_i V_{post} - x_i|$; x_i is the drifted holding amount; fully invested long-only with gross=net=1; first-month value $1/(1+c)$; no segment reset.

Why and how: Carry forward x and V_{pre}, solve V_{post}, then trade $wV_{post} - x$; fees and NAV use the same amount unit. Check $\sum_i w_i = 1$, long-only exposure, cash conservation, and month-end drift. Future ledger columns: `pre/post/end NAV`, `x`, `w`, `trade`, `fee`, `terminal`. The full equation is in the protocol.

Future output and interpretation: `Targets/drift` table and self-financing equation → ledger; the long-only budget closes each month. Target-market result fields: not run.

Failure branch: Mixing `pre/post-fee` weights or drift; computing costs from something other than the sum of absolute `buy/sell` amounts; missing the target weights after costs; omitting `entry/terminal` costs; or resetting at segment boundaries all break cash conservation. The original engineering share example is a unit comparison only, not a French adapter.

9. Transaction Cost Model

Question to understand: Trading-notional costs and NAV drag. Prerequisites: Chapters 29 and 34–36; Project P4.

Fixed design and upstream source: `S02/S05/S06`: all-in cost is $c \times |\text{traded notional}|$, where $c = 0.001$; do not divide by 2 or charge spread again. $200.10 \times 0.001 = 0.20010$; $(1 - 0.001)(1 + 0.02) - 1 = 0.01898$ is a separate constructed amount example.

Why and how: $c = 10/10000 = 0.001$; USD 200.10 traded notional produces USD 0.20010 all-in cost. NAV-cost proportion $C = 0.001$ and gross return $r = 0.02$ produce net $(1 - C)(1 + r) - 1 = 0.01898$, a drag of $0.00102 = 10.2$ bps; C and c have different denominators. The future cost bridge uses month on the x-axis and one consistent amount unit or an explicitly stated NAV proportion on the y-axis; do not mix the two.

Future output and interpretation: Absolute traded notional / fee / gross-to-net bridge → net ledger; reconcile NAV under the cost scenario. Target-market result fields: not run.

Failure branch: Dividing by 2 and understating cost, charging spread twice, or claiming real capacity without data. The share example's `half-spread/commission` is another setup. Future cost bridge and capacity remain unknown.

10. Walk-Forward Validation

Question to understand: Fixed ranking still requires an access and phase contract. Prerequisites: Chapters 22, 31, and 38; Projects `P3/P6` are comparisons only.

Fixed design and upstream source: `S02/S03/S05/S07`: 18 warm-up months; $440 = 176 + 132 + 132$; sole primary segment 2015-09..2026-08; 13 continuous paths carry across boundaries; do not invent training; the history is exposed and is not a blind holdout.

Why and how: Assign rows using the `TIME_CONTRACT` phases: 176 development, 132 validation, and 132 locked-reporting months, total 440. This separates `display/reporting` only. The first primary row carries from 2015-08; the last primary row includes liquidation. Output a `month/phase/access/hash` table. A fixed ranking has no training; skip1 is not embargo.

Future output and interpretation: Confirmed 176/132/132 months, continuous ledger without reset, and `maturity/purge` table → 132-row locked report. Fix access and order first; ranking does not require invented training. Target-market result fields: not run.

Failure branch: Calling exposed history a holdout or running with unresolved window rules. `Run/code/config/data/hash/access` logs require authorization.

11. Performance Analysis

Question to understand: Difference between the primary net-difference test and descriptive statistics. Prerequisites: Chapters 7, 11, and 32.

Fixed design and upstream source: `S02/S05`: $d_t = r_{M0,net,t} - r_{EW10,net,t}$; primary $n = 132$, $k = 1$, common-n denominator for $\widehat{\gamma}_j$, Bartlett lag 6, $q = 132/131$ applied to covariance, $\text{SE} = \sqrt{\widehat{\Omega}/131}$, approximate $t(131)$; bootstrap block 12 / 5,000 / fixed seed `20261012`.

Why and how: Check continuous month keys for the primary 132-row `d`, then compute HAC6 and the fixed bootstrap. The primary interval plot uses net mean difference on the x-axis (`decimal/month` or clearly labeled `bps/month`) and specification on the y-axis, with 132 months, hash, and `one-/two-sided` intervals identified. The wealth plot uses month on the x-axis and NAV on the y-axis, for all 440 months descriptively only. CAGR differs from 12 times the monthly mean. If SE degenerates, do not report a p-value.

Future output and interpretation: Paired monthly `difference/HAC6` for the 132 locked-reporting rows; all 440 months and the first two phases are descriptive only → primary and secondary results. Separate net advantage from uncertainty. Target-market result fields: not run.

Failure branch: Showing only the highest Sharpe or best interval. Section 11 explains `one-/two-sided` intervals, degenerate SE, no fabricated `p=0` for missing months, and the 10 bps economic threshold. Provide future table columns and interpretation rules only; do not generate means, Sharpe ratios, or CIs now.

12. Risk Analysis

Question to understand: Concentration, tails, and missing volume beyond returns. Prerequisites: Chapters 18, 20–21, and 33–35.

Fixed design and upstream source: `S01/S02/S06`: top3 concentration, industry weights, maximum drawdown, and historical tails. A uniform constructed −10% shock checks only the risk formula; with no `volume/order` book, do not estimate ADV capacity.

Why and how: Interpret concentration in weights, maximum drawdown, and historical tails separately. Weight chart: month on x-axis, capital weight on y-axis. Drawdown chart: month on x-axis, loss relative to peak on y-axis. Label the monthly `frequency/window` for tail measures. The uniform constructed −10% shock checks only the risk formula; it does not measure market liquidity or ADV capacity.

Future output and interpretation: Industry `weights/drawdown/tails/constructed` shock → risk figures; explain return trade-offs, concentration, and loss. Target-market result fields: not run.

Failure branch: Estimating capacity without volume or claiming equal weights eliminate concentration risk. Mark future `weights/drawdown/tails` as not run; do not use the engineering USD 200 stress as a market-risk estimate.

13. Factor Attribution

Question to understand: Post-realization attribution and one RF subtraction. Prerequisites: Chapters 14–15; Project P2.

Fixed design and upstream source: `S02/S05`: subtract same-month FF3 RF once from net total return; do not subtract RF again from published `Mkt-RF` / `SMB` / `HML`. Include an intercept and use $k = 4$ correction $n/(n-4)$. The primary paired d_t does not subtract RF and uses $k = 1$.

Why and how: Set $y_t = \texttt{net_total}_t - \texttt{FF3_RF}_t$ once, and let X contain an intercept plus published `Mkt-RF` / `SMB` / `HML`; use $k = 4$ correction. Future tables report monthly alpha, beta, and HAC intervals. Residual chart: month on x-axis, `decimal/month` on y-axis. The primary d_t does not subtract RF; alpha is post-hoc attribution conditional on the specified model.

Future output and interpretation: Same-month net total return minus RF once / FF3 `k=4`; primary difference has no RF subtraction and `k=1` → `coefficient/residual` figure; historical model-related attribution. Target-market result fields: not run.

Failure branch: Mixing `FF3/FF5`, subtracting RF twice, or treating alpha as skill. Future `coefficients/residual` figure remains pending; do not mix FF3 and FF5 snapshots.

14. Robustness Tests

Question to understand: How to report all prespecified variations. Prerequisites: Chapters 22 and 30–32; Project P3.

Fixed design and upstream source: `S02/S05/S06`: report cost, skip, fixed-phase, and statistical block-length 6/24 sensitivities in full. Keep negative and zero results; do not choose conventions based on intervals.

Why and how: Report every prespecified `cost/skip` sensitivity, the fixed three calendar phases, and bootstrap `6/24-month` sensitivities. Future table columns include specification, fixed variation, window, mean, interval, and status. Segments are cuts through a continuous ledger. Report conflicting intervals and `negative/zero` results; do not select a favorable convention.

Future output and interpretation: Prespecified `costs/skips/three` prespecified phases → complete table; describe changes under fixed variations. Target-market result fields: not run.

Failure branch: Dropping `negative/zero` scenarios or adding and reporting only favorable post-hoc items. Fill in prespecified table columns and interpretation rules now; leave actual results blank.

15. Parameter Sensitivity

Question to understand: Why 9+4 is not a search contest. Prerequisites: Chapters 9, 22, 25, and 32.

Fixed design and upstream source: `S02/S05`: nine strategies M0, C0, C25, C50, L5, L17, K2, K4, and S2 plus four EW cost paths; zero optimization. Vary one factor at a time; do not make a Cartesian grid.

Why and how: List all 13 paths explicitly. A one-factor response plot uses cost in bps, L, skip, or k on the x-axis and descriptive net difference or risk on the y-axis, labeled with input identity and continuous window. There is no Cartesian product and zero optimization; do not replace M0 with the best point.

Future output and interpretation: Trial ledger for 9 strategies + 4 benchmarks → one-factor plots; describe response stability without selecting a winner. Target-market result fields: not run.

Failure branch: Exceeding the budget with a Cartesian product or replacing primary parameters after seeing results. Future response plots do not select the primary specification.

Path	L	skip	k	All-in bps per traded notional
M0	11	1	3	10
C0	11	1	3	0
C25	11	1	3	25
C50	11	1	3	50
L5	5	1	3	10
L17	17	1	3	10
K2	11	1	2	10
K4	11	1	4	10
S2	11	2	3	10
EW0	no ranking	n/a	10 equal weights	0
EW10	no ranking	n/a	10 equal weights	10
EW25	no ranking	n/a	10 equal weights	25
EW50	no ranking	n/a	10 equal weights	50

16. Overfitting Discussion

Question to understand: Fixed future procedures do not erase exposure. Prerequisites: Chapters 22, 30, 32, and 37; Projects P1–P3/P6.

Fixed design and upstream source: S07/S02: P3 saw the full 1990–2026 period; P1/P2 saw portions; other access is unknown. Not blind; full_preregistration=false. PSR/DSR/PBO may be N/A if applicability is insufficient.

Why and how: Record exposure to P3's full-history equal-weight/model results and partial exposure in P1/P2; the 9+4 degrees of freedom are fixed. Label future exploration as post-hoc and preserve the complete failure log. If evidence for applying PSR/DSR/PBO is insufficient, report N/A or not_evaluated rather than fabricate a statistic.

Future output and interpretation: Exposure/degrees of freedom/total attempts → interpretation boundary; fixed steps cannot remove hindsight bias from prior exposure. Target-market result fields: not run.

Failure branch: Filling in DSR/PBO without applicable evidence. Mark future exploration as post-hoc; do not invent DSR/PBO results.

17. Limitations

Question to understand: What this route cannot directly answer. Prerequisites: Chapters 3, 29, 33–36, and 45.

Fixed design and upstream source: R/P, S02/S04/S10: rights are unresolved, historical PIT is unknown, hypothetical allocations are not real fills or ETF replication, and costs are not calibrated to capacity. Retain old P6/39 gaps; the monthly-return adapter is not implemented.

Why and how: The support-domain table can be filled now: revised history, monthly allocation, and assumed linear costs can be designed; strict historical PIT, ETF replication, real fills, capacity, and causal mechanism cannot be established directly. G01 is unknown, G05 is not authorized, the monthly-return adapter is not implemented, and real-data gaps from `P6/39` remain.

Future output and interpretation: `Rights/PIT/trading/statistical` gaps → support-domain table; define what the study can and cannot establish. Target-market result fields: not run.

Failure branch: Turning `unknown` into zero or adding corporate-action cash on top of total return. Unknown stays unknown; engineering acceptance does not mean real research is complete.

18. Final Research Conclusion

Question to understand: Separate the design answer from the future research answer. Prerequisites: Chapters 22, 32, and 45; all prerequisite material.

Fixed design and upstream source: `R/P`, `S01/S02/S06`: reportable current state is `design/protocol` technically accepted and execution blocked. Future positive, zero, negative, or interval-crossing-zero outcomes have distinct interpretations. An engineering PASS does not answer $\mu_d > 0$.

Why and how: Current answer: the research design and technical protocol are accepted, but there are no target-market results, so it is not possible to judge whether $\mu_d > 0$. A future positive result with lower bound > 0 must still be compared with the 10 bps economic threshold; an interval crossing zero means insufficient evidence; net ≤ 0 means no net advantage for this specification; gross advantage consumed by costs indicates cost sensitivity; a failed gate means blocked, not zero effect.

Future output and interpretation: Primary paired net results / failed gates → answer the RQ. The result may be negative, zero, or insufficiently precise; blocked means no market conclusion. Target-market result fields: not run.

Failure branch: Treating engineering success as the result of all 18 research items. Provide future conclusion wording and failure branches, and never use constructed tables or engineering NAV `9995.998` as a French research conclusion.

Capstone Research Report Template

Status: The bounded research design and technical protocol have been accepted; target-market execution is blocked. See the current Capstone README. The engineering appendix below retains its historical identity and is not a French-market result. The original generic prompts are provided for comparison. The selected French protocol does not include model training, neutralization, a Cartesian grid, or mandatory `DSR/PBO` calculations; use the current design and report guide for each section.

1. Research Hypothesis

Fixed now: `S01/S02`: fixed French ten industries, `L11/skip1/top3`, EW10; $H_0 : \mu_d \leq 0$ / $H_1 : \mu_d > 0$; one month is the experimental unit; slow reaction is not an identified cause. See Guide item 1. Target-market output: **not run**.

Prior economic mechanism, target `assets/frequency`, falsifiable expectation, baseline, and registration date.

2. Data Sources

Fixed now: `S08/S09`, `R/P`: 2026-10-05 revised snapshot, `industry/FF3` identity, and download UTC; G01 `subject/use` is unknown; raw redistribution is `not_requested`. See Guide item 2. Target-market output: **not run**.

Source text and URL, license, download UTC, coverage, hashes, and supplier revisions; do not invent missing details.

3. Data Dictionary

Fixed now: `S04/S03`: long table `(snapshot_id, month, industry_id)`, decimal total returns for ten industries, `FF3/RF`, `score/weight/amount/net-return/label-maturity` fields. See Guide item 3. Target-market output: **not run**.

Fields, keys, types, units, frequency, time fields, missingness, and revision rules.

4. Point-in-Time Logic

Fixed now: S03: half-open $[B_t, B_{t+1})$; $S = B_t - 1$ second and allocation $= B_t$ are hypothetical ordering markers only; `available` / `ingested` / `revision` remain unknown; the `normal/future` rows are `CONSTRUCTED`. See Guide item 4. Target-market output: **not run**.

`Announcement/availability/ingestion/signal/trade/return/label` timeline; as-of join and counterexample tests.

5. Data Cleaning

Fixed now: `S02/S04/S06`: reject duplicate `month/industry` keys, missing months, non-finite values, and missing values; divide source percentages by 100 once only; do not add corporate-action cash already reflected in total return. See Guide item 5. Target-market output: **not run**.

Deduplication, missing values, bad prices, `splits/dividends/delisting`; `before/after` counts and traceable logs.

6. Feature Engineering

Fixed now: `S02/S03/S05`: $\text{score}_{i,t} = \prod_{m=t-12}^{t-2}(1 + R_{i,m}) - 1$, 11 terms; general window $[t - \text{skip} - L,\ t - \text{skip} - 1]$; first-evaluation-month endpoints are listed. See Guide item 6. Target-market output: **not run**.

Mathematical definition, units, window endpoints, train-only fitting, economic meaning, and ablation.

7. Signal Construction

Fixed now: `S02/S05`: fixed order of ten industries; descending score / ascending manifest index for ties; primary top3 at 1/3 each, sensitivities at $1/k$; no fitting. See Guide item 7. Target-market output: **not run**.

Ranking, winsorization, neutralization, missing-value treatment, and combined weights; verify direction and time.

8. Portfolio Construction

Fixed now: S02: $V_{post} = V_{pre} - c\sum_i |w_i V_{post} - x_i|$; x_i is the drifted holding amount; fully invested long-only, gross=net=1; first month $1/(1 + c)$; no segment reset. See Guide item 8. Target-market output: **not run**.

Budget, `gross/net`, `sector/beta`, leverage, cash, drift, feasibility, and rebalancing.

9. Transaction Cost Model

Fixed now: `S02/S05/S06`: all-in cost is $c \times |\text{traded notional}|$, $c = 0.001$; do not divide by 2 or charge spread again. $200.10 \times 0.001 = 0.20010$; $(1-0.001)(1+0.02)-1 = 0.01898$ is a separate constructed amount example. See Guide item 9. Target-market output: **not run**.

Commissions, spread, impact, financing, borrow, capacity, calibration limits, and gross-to-net reconciliation.

10. Walk-Forward Validation

Fixed now: `S02/S03/S05/S07`: 18 warm-up months; $440 = 176 + 132 + 132$; sole primary period 2015-09..2026-08; 13 paths continuous and carried across phase boundaries; do not invent training; history is exposed and is not a blind holdout. See Guide item 10. Target-market output: **not run**.

`Training/selection/final` test, rolling calendar, `overlap/purge/gap`, and `fit/prediction` availability times.

11. Performance Analysis

Fixed now: `S02/S05`: $d_t = r_{M0,net,t} - r_{EW10,net,t}$; primary $n = 132$, $k = 1$, common-n gamma denominator, Bartlett lag 6, $q = 132/131$ applied to covariance, SE $= \sqrt{\widehat{\Omega}/131}$, approximate $t(131)$; bootstrap block 12 / 5,000 / fixed seed `20261012`. See Guide item 11. Target-market output: **not run**.

`CAGR/Sharpe/drawdown`, intervals under dependence, simple benchmarks, and phase-specific descriptions; do not report only the best period.

12. Risk Analysis

Fixed now: `S01/S02/S06`: top3 concentration, industry weights, maximum drawdown, historical tails; a uniform constructed -10% shock checks only the risk formula; without `volume/order-book` data, do not estimate ADV capacity. See Guide item 12. Target-market output: **not run**.

Factors, industries, concentration, tails, liquidity, leverage, margin, and stress scenarios.

13. Factor Attribution

Fixed now: `S02/S05`: subtract same-month FF3 RF once from net total return; do not subtract RF again from published `Mkt-RF` / `SMB` / `HML`; use an intercept and $k = 4$ correction $n/(n-4)$. The primary paired d_t does not subtract RF and uses $k = 1$. See Guide item 13. Target-market output: **not run**.

Same-frequency, same-currency RF; `CAPM/FF`; HAC choice; residuals and scope of alpha interpretation.

14. Robustness Tests

Fixed now: `S02/S05/S06`: report all `cost/skip` sensitivities, the prespecified phases, and statistical block-length 6/24 sensitivities; retain `negative/zero` results and do not choose conventions based on intervals. See Guide item 14. Target-market output: **not run**.

Prespecified subsamples, alternate conventions, costs, delays, extreme periods, ablations, and alternate baselines.

15. Parameter Sensitivity

Fixed now: `S02/S05`: `M0/C0/C25/C50/L5/L17/K2/K4/S2` (nine strategies) plus four EW cost paths; zero optimization; one factor at a time, no Cartesian product. See Guide item 15. Target-market output: **not run**.

Parameter grid, all trials, stable regions, and figures; do not re-register based on the best test result.

16. Overfitting Discussion

Fixed now: `S07/S02`: P3 saw all of 1990–2026; `P1/P2` saw portions; other access is unknown; not blind and `full_preregistration=false`; `PSR/DSR/PBO` may be `N/A` if applicability is insufficient. See Guide item 16. Target-market output: **not run**.

Research degrees of freedom, multiple testing, `PSR/DSR/PBO` applicability, omitted searches, and limits on the final holdout.

17. Limitations

Fixed now: `R/P`, `S01/S02/S06`: rights are unresolved; historical PIT is unknown; hypothetical allocations are not actual fills or ETF replication; costs are not calibrated to capacity; retain old `P6/39` gaps; the monthly-return adapter is not implemented. See Guide item 17. Target-market output: **not run**.

Data, trading, statistical, model, market-regime, capacity, and deployment gaps; state unsupported inferences explicitly.

18. Final Research Conclusion

Fixed now: `R/P`, `S01/S02/S06`: current reportable status is `design/protocol` technically accepted and execution blocked; future positive, zero, negative, or interval-crossing-zero outcomes each have their own interpretation. Engineering PASS does not answer $\mu_d > 0$. See Guide item 18. Target-market output: **not run**.

Answer the initial hypothesis; distinguish evidence, conjecture, and unresolved questions. Report honestly if the net result is not economically meaningful.

Engineering Foundation Evidence Appendix (A9-B Engineering Base Interface)

The following preserves the historical status and figures from the original A9-B engineering appendix. These historical `pending` / candidate labels do not override the current A10-A / A10-A.1 acceptance status and do not mean that market research has been run. See the current research entry and the 18 guide items above for current conventions.

Historical status: The A9-B engineering-foundation phase was completed, providing an offline production pipeline, quality gates, persistent ledger, and reconciliation loop. All real research and strategy-development work was explicitly marked `pending_A10` at that time.

Engineering evidence and research status by report section

Report section	Historical status	Engineering evidence / artifact index	Note
1. Research Hypothesis	`pending_A10`	No strategy hypothesis; framework interface only	Formal research hypothesis awaited A10
2. Data Sources	`pending_A10`	`CONSTRUCTED_TEACHING_DATA` v1.0.0	No external real-data download
3. Data Dictionary	`pending_A10`	`capstone/data_dictionary.md` appendix	Constructed-field engineering dictionary only
4. Point-in-Time Logic	`author_candidate_pending_independent_acceptance`	$\max(\texttt{avail}, \texttt{ingest}, \texttt{as_of}) \leq \texttt{signal} < \texttt{trade}$	Constructed time contract and rejection tests; pending independent acceptance at the time
5. Data Cleaning	`author_candidate_pending_independent_acceptance`	Staleness threshold (≤ 3600 s), strictly positive prices, `non-null/non-duplicate` checks	Rejects the anomalies listed in the pre-fill contract
6. Feature Engineering	`pending_A10`	One-period teaching momentum $(P_t - P_{t-1})/P_{t-1} > 0$	Formal A10 feature engineering and ablation were pending
7. Signal Construction	`pending_A10`	Fixed teaching allocation quantities (A: 10, B: 20)	Cross-sectional ranking and neutralization awaited A10
8. Portfolio Construction	`pending_A10`	Fixed teaching positions; ex-ante leverage check $0.20 \leq 0.50$	Formal portfolio method awaited A10 preregistration
9. Transaction Cost Model	`author_candidate_pending_independent_acceptance`	10 bps spread embedded in fill price; 10 bps commission charged explicitly	No duplicate charge; total cost USD 4.002
10. Walk-Forward Validation	`pending_A10`	No rolling-window split run	Real rolling out-of-sample backtest awaited A10
11. Performance Analysis	`pending_A10`	Baseline ending NAV 9,995.998 USD; net loss -4.002 USD	Teaching example, not real strategy alpha
12. Risk Analysis	`author_candidate_pending_independent_acceptance`	-10% market-shock scenario loss USD 200.0 $\leq$ USD 1000.0 limit	Stress test and ex-ante leverage gate passed
13. Factor Attribution	`pending_A10`	No multi-factor decomposition	Attribution awaited A10 under Project 2 `units/RF/HAC` rules
14. Robustness Tests	`pending_A10`	No parameter-robustness perturbations	Subsample and sensitivity analyses awaited A10
15. Parameter Sensitivity	`pending_A10`	No parameter-grid scan	Parameter response surface awaited A10
16. Overfitting Discussion	`pending_A10`	No multiple-testing check	`PSR/DSR/PBO` calculations awaited A10
17. Limitations	`author_candidate_pending_independent_acceptance`	Offline, single-process environment; constructed teaching data; no live-market impact	Engineering boundaries stated; no extrapolation

Report section	Historical status	Engineering evidence / artifact index	Note
18. Final Research Conclusion	`pending_A10`	Constructed engineering-interface candidate; strategy-validity test awaited A10	No unsupported empirical investment conclusion

R2 engineering repair

R01–R04 counterexamples / STOPPED / overwrite and actual-order evidence are in the historical source `qa/a9/ch45_batch/revisions/r2_20261012`. The amount tolerance is 10^{-6} USD; real-valued share quantities and three recovery branches are described in `ENGINEERING_BASE.md`. All real-research sections were still `pending_A10` in this historical appendix. It does not fill real results or register acceptance.

Capstone Engineering Foundation Specification and Run Instructions (`ENGINEERING_BASE`)

Package note. The commands below describe the final, complete course package; this Markdown-only candidate does not include runnable course entrypoints. Links into `outputs/` identify read-only I3 sources and are not command working directories. The research protocol has G02 and G04 closed; G01 remains unknown and G05 is not authorized, so market execution is not authorized.

Status: `construction-only` (constructed-data baseline only; no real market data, live trading, or broker execution).

Version: `1.0.0` (A9-B Engineering Foundation Freeze)

Frozen configuration: `capstone/configs/offline_constructed.json`

---

1. Scope and invariants

1. **Fully offline execution:** no external network connections, online financial APIs, broker order interfaces, or requests for external keys or credentials.
2. **Deterministic constructed dataset:** data type is `CONSTRUCTED_TEACHING_DATA`, containing two teaching assets (`AssetA`, `AssetB`).

3. **Strict separation of causal and temporal information:**
   - Features and signal decisions may read only observations available at or before `signal_time` (`available_at`, `ingested_at`, `as_of`).
   - Next-day simulated fills (`09:00Z`, `09:01Z`) and the terminal mark (`16:00Z`) are for the simulator only and must not leak into the feature or signal layer.
4. **Research-work boundary:** this A9-B engineering freeze establishes only the engineering architecture and quality gates. Strategy findings and live-market checks in the 18-item research report are historically marked `pending_A10`; the current A10 design and protocol status is given in the Capstone README.

---

2. Portable run instructions for the final package

From the final package root, follow the reader README in the read-only I3 source to create an isolated environment. Create a new `runs/` subdirectory for each run and use the fixed constructed configuration. Keep the directory, state database, and task key consistent within each invocation. Use the package root as the command working directory:

```
run="runs/ch45_$(.venv/bin/python -c 'from uuid import uuid4; print(uuid4().hex)')"
.venv/bin/python code/ch45_production_quant.py --config capstone/configs/
    ↪ offline_constructed.json --run-key reader --state-db "$run/state.db" --output-
    ↪ dir "$run/artifacts"

run="runs/capstone_$(.venv/bin/python -c 'from uuid import uuid4; print(uuid4().hex)
    ↪ ')"
.venv/bin/python capstone/run_offline.py --config capstone/configs/
    ↪ offline_constructed.json --run-key reader --state-db "$run/state.db" --output-
    ↪ dir "$run/artifacts"
```

To inspect the two fills and reconcile cash read-only, follow the PATH09 full chain in the read-only I3 reader source, reusing the same state database and task key. The `recovery/idempotency` check entrypoint `capstone/check_portable.py` is restored byte-for-byte from the accepted read-only I3 source and runs from the package root. This document adaptation does not require rerunning the historical 47 checks.

The original A9-B version placed commands under `build/batches/A9-B/r2/...`. Those fixed directories are **historical audit run locations**, not instructions for the current portable package. The original source and SHA are retained in the historical acceptance note. Other engineering interfaces in this document are carried forward; the original acceptance decision, fixed configuration, and 18-item report template are not changed.

---

3. Lineage, logging, and persistent-state interfaces

3.1 SQLite state database schema (`state.db`)

- `runs` table:
  - `run_key TEXT PRIMARY KEY`: unique task key.
  - `fingerprint TEXT NOT NULL`: SHA-256 binding fingerprint for normalized inputs, configuration, code hashes, and historical module hashes.
  - `status TEXT NOT NULL`: run status (`RUNNING`, `STOPPED`, `COMPLETED`).
  - `created_at TEXT`, `updated_at TEXT`, `output_dir TEXT`.
- `events` table:
  - `id INTEGER PRIMARY KEY AUTOINCREMENT`: auto-incrementing event sequence number.
  - `run_key TEXT`, `event_name TEXT`, `event_time TEXT`, `details_json TEXT`.
- `fills` table:
  - `fill_id TEXT PRIMARY KEY`: unique fill key.
  - `run_key TEXT`, `order_id TEXT`, `parent_id TEXT`, `slot_index INTEGER`, `asset TEXT`, `fill_time TEXT`.
  - `signed_quantity REAL`, `mid_price REAL`, `fill_price REAL`, `notional REAL`.
  - `half_spread_usd REAL`, `temp_impact_usd REAL`, `perm_impact_usd REAL`, `commission REAL`.

3.2 Idempotency and failure-recovery guarantees

1. **Same key, same input:** read existing artifacts and hashes, reuse them, and do not generate duplicate fills (the fill count remains stable).
2. **Same key, different input:** reject execution and record a `RUN_KEY_FINGERPRINT_MISMATCH` event.
3. **Tampered cache:** validate the expected hash anchored in the SQLite record and `checksums.json`. If tampering is detected, reject explicitly, record `CACHED_ARTIFACT_CORRUPT`, and never overwrite silently.
4. **Recovery after interruption:** rebuild the historical ledger from persisted fills, avoid charging costs twice, and continue unfinished orders.
5. **Risk-control or reconciliation failure:** persist the `STOPPED` state immediately and block subsequent fills.

---

4. Business artifacts and runtime separation

The following files are atomically published under `--output-dir`:

- `business_summary.json`: deterministic business summary (initial cash, ending cash, ending NAV, `gross/net` PnL, cost breakdown, and independent reconciliation status).
- `fills.csv`: fill ledger.
- `ledger_events.csv`: ledger-event history.
- `signals_targets.csv`: signals and target holdings.
- `checksums.json`: SHA-256 checksums for business files; SQLite also anchors the hashes of published files.
- `runtime_metadata.json`: non-deterministic metadata such as elapsed time, `start/end` times, process PID, and environment information.
- `report_evidence_interface.json`: evidence-interface definition for the 18-section report (generated by the Capstone entrypoint).
- `figures/`: figures (`gate_outcomes.png` for gate outcomes and `cash_nav_timeline.png` for cash and NAV over time).

`normalized_inputs.json`, `normalized_config.json`, and `lineage.json` contain normalized inputs, configuration, and version lineage, all bound to the published hashes. Structured events are stored in SQLite `events`; stdout contains a JSON summary. No Python logging platform or Prometheus service is started. Figures are generated after CLI publication and are not part of the core cache commit transaction; body figures are stored under a separate candidate manifest.

Recovery coverage is limited to the controlled Python exceptions and local `SQLite/directory-publication` checks in this batch. There was no power-loss test, distributed multi-process test, or live broker simulation. The Dockerfile is a reference configuration: the image was not built, the container was not run, and its base tag and unlocked dependencies do not establish a byte-frozen environment.

`capstone/.dockerignore` is a skeleton reference. For a build using the course-root context described above, the effective default ignore file belongs at the build-context root. This Dockerfile copies only the specified `src/capstone/code` paths and does not copy `build` or `.venv`; this description does not replace an actual image-build check.

R2-limited repair interface

The amount tolerance is 10^{-6} USD. Fixed configured quantities are finite positive real numbers; a non-positive signal target is zero. The examples use integer quantities, and broker lot size is not validated. The observation, execution, and valuation asset sets must match and are validated before state creation. Simple-return stress values must be finite and at least -1; positive shocks are allowed as general scenarios. loss $= -$scenarioPnL may be negative. If momentum is non-finite after calculation, write `FEATURE_VALIDATION_FAILED` / `STOPPED`, retain zero fills,

and publish nothing. If independent terminal reconciliation fails, write `TERMINAL_RECONCILIATION_FAILED` / `STOPPED`, retain existing fills, and reject retries.

The three publication-recovery branches are: no publication, so create a new attempt; valid publication, so finish its completion; damaged publication, so reject it. Failure order is counted by cumulative actual fills and is not tied to an asset name. Recovery does not inject the first fill again if it already exists. A designated failure with no sufficient positive target is rejected before database creation. Monitoring may update after labels mature, but future labels cannot be used early; this batch does not run model monitoring.

Capstone Engineering Data Dictionary and Historical Template

This page retains the two-asset, `USD/share`, share, and second-level constructed-engineering fields, together with the historical general template. The French research plan uses the monthly-return dictionary; current status and the separate purposes of the two configs are described in the README. This reader package does not distribute French research-market data. Each field must be defined independently; a template must not be presented as an observation.

Field	Type / unit	Primary key / frequency	Economic meaning	`event_time`

Field	`available_at`	`revision_time`	Missingness / cleaning	Source / version

Base entities include permanent security ID, exchange, currency, quotes, corporate actions, financial-statement versions, historical constituents, factors, orders, fills, positions, and cash. The specific fields depend on available data. A label with `label_end` later than the training cutoff cannot enter that fit.

Engineering Foundation Data Dictionary Appendix

Status: A9-B constructed teaching-data baseline (`CONSTRUCTED_TEACHING_DATA` v1.0.0). No real externally downloaded data or live-market observations.

Field	Type / unit	Primary key / frequency	Economic meaning	event_time

Field	Type / unit	Primary key / frequency	Economic meaning	event_time
asset / security_id	string (symbol)	composite primary key	Permanent asset identifier (AssetA, AssetB)	-
prior_price	float (USD/share)	cross-sectional observation / single period	Reference price before momentum calculation	2026-08-31T16:00Z (prior_event_time)
prior_event_time	ISO8601 UTC	prior timestamp	Time the prior price observation was produced	2026-08-31T16:00Z
current_price	float (USD/share)	cross-sectional observation / single period	Current reference price for the decision	2026-09-01T16:00Z
event_time	ISO8601 UTC	time the fact occurred	Price-matching/fill time	observation itself
available_at	ISO8601 UTC	data-availability time	Time the external system could access the data	-
ingested_at	ISO8601 UTC	system-ingestion time	Time the local system ingested the data	-
as_of	ISO8601 UTC	cross-sectional reference time	Time the observation is effective	-
revision_time	ISO8601 UTC	revision-effective time	Time of the latest revision of the observation	-
momentum	float (dimensionless scalar)	calculated measure	One-period return momentum $(P_t - P_{t-1})/P_{t-1}$	2026-09-01T16:05Z
target_quantity	float (share)	target portfolio holding	Target rebalance quantities (A: 10, B: 20)	2026-09-01T16:05Z
trade_time	ISO8601 UTC	matching time	Discrete simulated fill time	next day 09:00Z / 09:01Z
mid_price	float (USD/share)	simulated fill reference	Mid-price at the matching time	next day 09:00Z / 09:01Z
fill_price	float (USD/share)	execution price	Execution price including half-spread $(P_{\text{mid}} + \Delta)$	trade_time
commission	float (USD)	explicit fee amount	Trading commission (10 bps of notional)	trade_time
mark_time	ISO8601 UTC	valuation/settlement time	Terminal mark-to-market time	2026-09-02T16:00Z
mark_price	float (USD/share)	terminal valuation price	Terminal mark-to-market price (A: 102, B: 49)	2026-09-02T16:00Z
cash	float (USD)	account cash balance	Portfolio liquidity cash	continuously marked
nav	float (USD)	net asset value	NAV (cash + marked holdings)	continuously marked

Field	available_at	revision_time	Missingness / cleaning	Source / version
asset / security_id	-	-	Must not be null or duplicated	A9-B constructed standard v1.0.0
prior_price	2026-09-01T16:00Z	2026-09-01T16:00Z	Must be finite and positive (> 0)	A9-B constructed quotes v1.0.0
prior_event_time	2026-09-01T16:00Z	2026-09-01T16:00Z	Must be strictly earlier than event_time	Constructed metadata v1.0.0
current_price	2026-09-01T16:00Z	2026-09-01T16:00Z	Must be finite and positive (> 0)	A9-B constructed quotes v1.0.0
event_time	synchronous	2026-09-01T16:00Z	Must be timezone-aware UTC	Constructed metadata v1.0.0
available_at	2026-09-01T16:00Z	2026-09-01T16:00Z	Must satisfy available_at $\leq$ signal_time	Constructed metadata v1.0.0
ingested_at	2026-09-01T16:00Z	2026-09-01T16:00Z	Must satisfy ingested_at $\leq$ signal_time	Constructed metadata v1.0.0
as_of	2026-09-01T16:00Z	2026-09-01T16:00Z	Must satisfy as_of $\leq$ signal_time; staleness ≤ 3600 s	Constructed metadata v1.0.0
revision_time	2026-09-01T16:00Z	2026-09-01T16:00Z	Must satisfy revision_time $\leq$ signal_time	Constructed metadata v1.0.0
momentum	2026-09-01T16:05Z	-	Must be finite	Derived strategy feature v1.0.0
target_quantity	2026-09-01T16:05Z	-	Must be nonnegative	Strategy target configuration v1.0.0
trade_time	next day	-	Must satisfy trade_time $>$ signal_time	Simulator matching data v1.0.0
mid_price	next day	-	Must be finite and positive (> 0)	Simulator quotes v1.0.0
fill_price	trade_time	-	Spread is embedded; do not charge it twice	Ledger fill history v1.0.0
commission	trade_time	-	Must be nonnegative (≥ 0)	Ledger fill history v1.0.0
mark_time	2026-09-02T16:00Z	-	Must satisfy mark_time $\geq$ trade_time	Simulator valuation mark v1.0.0
mark_price	2026-09-02T16:00Z	-	Must be finite and positive (> 0)	Simulator valuation mark v1.0.0
cash	real time	-	Balance-sheet reconciliation closes; ending value 7995.998	Single-period event cash ledger v1.0.0
nav	real time	-	Ending value 9995.998, net PnL -4.002	Single-period event NAV ledger v1.0.0

R2 support domain and units

`fixed_quantities` in the config must be finite, positive share quantities; a non-positive signal target is `0 share`. There is no lot-size validation. Execution and marks must contain exactly the same asset set as observations. General stress simple returns must be finite and ≥ -1; positive shocks are allowed. Positive raw prices do not guarantee that a ratio-derived feature will be finite, so check momentum after calculation. Reconciliation uses an absolute tolerance of 10^{-6} USD; share checks use share units separately and must not reuse a USD tolerance.

Historical Capstone Engineering Acceptance Note

Historical decision date: 2026-10-12. **I3 reader adaptation date:** 2026-10-14. **English edition:** 2026-10-16.

This is a historical summary. On 2026-10-13, later `A10-A/A10-A.1` decisions closed G02 (dates) and G04 (statistical and cost conventions) for the current research design and technical protocol. G01 rights for the actual `subject/use` remain unknown, G05 has not authorized A10-B, and target-market execution remains unauthorized. Those later decisions do not amend or expand the historical engineering acceptance described here.

Only the R2 engineering foundation and the specified constructed-data checks received limited acceptance from the original main conversation. Full research, real-data permissions, final strategy, and conclusions remained `pending_A10`; the full Capstone was not `verified`. This historical decision was not expanded by the portable-reader summary.

The original acceptance-receipt SHA-256 is `3b6d1a037d659106c2727aff8301e053cd5ca666f656c1eb38aaffe0bbf0672b`; the R2 manifest SHA-256 is `ff4a12666599d71f5511af728a7ad8d5ab0dac1a03676b4f085f7b449efb1d1b`. Teaching `body/Loom` content came from original R3; the older PH version was not written back.

Read-only I3 source package references

The links below identify files in the read-only I3 reader package; they are source references, not evidence that this BIL22 working root is runnable. Links identify historical sources; unavailable runtime entries are explicitly marked in the current engineering instructions.

Engineering Base in the read-only I3 package, offline entrypoint, fixed constructed config, data dictionary, and 18-item report template were the reader-package entrypoints. The shared production core was `src/quant_course/production_pipeline.py`. The identities of these canonical files were accepted and were not copied or modified here. I3 added the accepted research design, technical protocol, and 18-item report guide; market results were not run. Constructed-engineering success is not a French research conclusion. Docker was not built or run; no credentials or schedules were configured and A10-B was not executed.

Original audit sources (not distributed with the portable package)

The paths below are source-path metadata relative to the original course root. They identify historical provenance and are not local links or runtime dependencies. Bound runs, failures, and audit evidence remain in their original historical directories. The portable package does not copy the full `QA/Part` VII trees.

Original source path	SHA-256
`qa/a9/ch45_independent_acceptance/r3_ph_20261012/FINAL_ACCEPTANCE.md`	`5bd9d1d6cf2ef42e6f3a42bef91100dc36d9ce25225142974a22277aeb3e4cc5`
`qa/a9/ch45_independent_acceptance/r3_ph_20261012/ACCEPTANCE_RECEIPT.json`	`3b6d1a037d659106c2727aff8301e053cd5ca666f656c1eb38aaffe0bbf0672b`
`part07/ACCEPTED_VERSION_ch45.md`	`46d8ba251909977b00faf60dd8cad8793a1bca20dc1ba269f1dd56f61d05d803`
`qa/a9/ch45_closeout/r3_accepted_20261012/CLOSEOUT_PLAN.md`	`2179be215ec36911b1dfcd37a27c5cfd7746246d84b5b6ba70019ddf349dc686`
`capstone/ENGINEERING_BASE.md`	`996b01ba9f873c7f6b27b2be32bc3b8091d4f8eff475d96ee92ace12c63001bb`
`capstone/ACCEPTED_ENGINEERING_VERSION.md`	`e6b4ab5b4a1c9b1fabb2b9e814bcc0a7c61ae3ed9cb32a90d8d3714e11945e06`

The original constructed checks covered data → signal → target → simulated fills → ledger → `risk/reconciliation` → report, including duplicate-run, recovery, and STOPPED cases. They did not include a real broker, licensing review, final test tuning, or a complete research conclusion. Historical closeout registration, backups, and recovery advice remain at the original sources listed above. This adaptation does not rewrite the original acceptance record, publish, archive, or delete anything.